```
; ******************************************************************************
; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.10
; ------------------------------------------------------------------------------
; Last Update: 10/02/2026 (Previous: 10/01/2026)
; ------------------------------------------------------------------------------
; Beginning: 04/01/2016
; ------------------------------------------------------------------------------
; Assembler: NASM version 2.15 (trdos386.s)
; ------------------------------------------------------------------------------
; Turkish Rational DOS
; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
;
; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
; unix386.s (03/01/2016)
;
; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
; TRDOS2.ASM (09/11/2011)
;
; Derived from 'IBM PC-XT-286' BIOS source code (1986)
; ******************************************************************************
; nasm trdos386.s -l trdos386.txt -o TRDOS386.SYS

KLOAD       equ 10000h ; Kernel loading address
     ; NOTE: Retro UNIX 8086 v1 boot code loads kernel at 1000h:0000h
KCODE       equ 08h ; Code segment descriptor (ring 0)
KDATA       equ 10h ; Data segment descriptor (ring 0)
; 19/03/2015
UCODE       equ 1Bh ; 18h + 3h  (ring 3)
UDATA       equ 23h ; 20h + 3h  (ring 3)
; 24/03/2015
TSS equ 28h ; Task state segment descriptor (ring 0)
; 19/03/2015
CORE equ 400000h  ; Start of USER's virtual/linear address space
                  ; (at the end of the 1st 4MB)
ECORE       equ 0FFC00000h ; End of USER's virtual address space (4GB - 4MB)
                  ; ULIMIT = (ECORE/4096) - 1 = 0FFBFFh (in GDT)
;; 27/12/2013
;KEND       equ KLOAD + 65536 ; (28/12/2013) (end of kernel space)
; 04/07/2016
KEND    equ KERNELFSIZE + KLOAD

; IBM PC/AT BIOS ----- 10/06/85 (postequ.inc)
;-------- CMOS TABLE LOCATION ADDRESS'S ------------------------------------
CMOS_SECONDS       EQU     00H            ; SECONDS (BCD)
CMOS_SEC_ALARM     EQU     01H            ; SECONDS ALARM (BCD)
CMOS_MINUTES       EQU     02H            ; MINUTES (BCD)
CMOS_MIN_ALARM     EQU     03H            ; MINUTES ALARM (BCD)
CMOS_HOURS  EQU     04H            ; HOURS (BCD
CMOS_HR_ALARM      EQU     05H            ; HOURS ALARM   (BCD)
CMOS_DAY_WEEK      EQU     06H            ; DAY OF THE WEEK  (BCD)
CMOS_DAY_MONTH     EQU     07H            ; DAY OF THE MONTH (BCD)
CMOS_MONTH  EQU     08H            ; MONTH (BCD)
CMOS_YEAR   EQU     09H            ; YEAR (TWO DIGITS) (BCD)
CMOS_CENTURY       EQU     32H            ; DATE CENTURY BYTE (BCD)
CMOS_REG_A  EQU     0AH            ; STATUS REGISTER A
CMOS_REG_B  EQU     0BH            ; STATUS REGISTER B  ALARM
CMOS_REG_C  EQU     0CH            ; STATUS REGISTER C  FLAGS
CMOS_REG_D  EQU     0DH            ; STATUS REGISTER D  BATTERY
CMOS_SHUT_DOWN     EQU     0FH            ; SHUTDOWN STATUS COMMAND BYTE
;----------------------------------------
;   CMOS EQUATES FOR THIS SYSTEM  ;
;------------------------------------------------------------------------------
CMOS_PORT   EQU     070H           ; I/O ADDRESS OF CMOS ADDRESS PORT
CMOS_DATA   EQU     071H           ; I/O ADDRESS OF CMOS DATA PORT
NMI         EQU     10000000B      ; DISABLE NMI INTERRUPTS MASK -
                                   ; HIGH BIT OF CMOS LOCATION ADDRESS

; Memory Allocation Table Address
; 05/11/2014
; 31/10/2014
MEM_ALLOC_TBL      equ     100000h        ; Memory Allocation Table at the end of
                                   ; the 1st 1 MB memory space.
                                   ; (This address must be aligned
                                   ;  on 128 KB boundary, if it will be
                                   ;  changed later.)
                                   ; ((lower 17 bits of 32 bit M.A.T.
                                   ;   address must be ZERO)).
                                   ; ((((Reason: 32 bit allocation
                                   ;     instructions, dword steps)))
                                   ; (((byte >> 12 --> page >> 5)))
;04/11/2014
PDE_A_PRESENT      equ     1              ; Present flag for PDE
PDE_A_WRITE equ     2              ; Writable (write permission) flag
PDE_A_USER  equ     4              ; User (non-system/kernel) page flag
;
PTE_A_PRESENT      equ     1              ; Present flag for PTE (bit 0)
PTE_A_WRITE equ     2              ; Writable (write permission) flag (bit 1)
PTE_A_USER  equ     4              ; User (non-system/kernel) page flag (bit 2)
PTE_A_ACCESS    equ        32             ; Accessed flag (bit 5) ; 09/03/2015

; 17/02/2015 (unix386.s)
; 10/12/2014 - 30/12/2014 (0B000h -> 9000h) (dsectrm2.s)
DPT_SEGM equ 09000h  ; FDPT segment (EDD v1.1, EDD v3)
;
HD0_DPT      equ 0     ; Disk parameter table address for hd0
HD1_DPT      equ 32    ; Disk parameter table address for hd1
HD2_DPT      equ 64    ; Disk parameter table address for hd2
HD3_DPT      equ 96    ; Disk parameter table address for hd3

; 15/11/2020
VBE3INFOSEG equ 97E0h ; 512 bytes before Video_Pg_Backup
; 15/12/2020
VBE3MODEINFOSEG equ 97C0h ; 512 bytes before VBE3INFOBLOCK

; 29/11/2020
VBE3INFOBLOCK equ 97E00h ; linear address (512 bytes)
VBE3MODEINFOBLOCK equ 97C00h ; linear address (256 bytes)
VBE3SAVERESTOREBLOCK equ 97600h ; linear address (2048 bytes)
VBE3CRTCINFOBLOCK equ 97D80h ; linear address (64 bytes) ; 17/01/2021
VBE3BIOSDATABLOCK equ 97000h ; linear address (1536 bytes)
VBE3STACKADDR equ 96000h ; linear address (1024 bytes)
; VBE3 32 bit Protected Mode Interface (16 bit) Selectors (in GDT)
VBE3CS equ 30h ; _vbe3_CS:
VBE3BDS equ 38h ; _vbe3_BDS:
VBE3A000 equ 40h ; _A0000Sel:
VBE3B000 equ 48h ; _B0000Sel:
VBE3B800 equ 50h ; _B8000Sel:
VBE3DS equ 58h ; _vbe3_DS:
VBE3SS equ 60h ; _vbe3_SS:
VBE3ES equ 68h ; _vbe3_ES:
KCODE16 equ 70h ; _16bit_CS:
; 14/01/2021
; 06/12/2020
VBE3VIDEOSTATE equ 95800h ; 2048 bytes
```

```
                                    ; 05/01/2021
                                    VGAFONT16USER equ 94000h ; 8x16 pixels user font (256 chars)
                                                       ; (reserved/allocated font space: 4096 bytes)

                                    VGAFONT8USER equ 95000h   ; 8x8 pixels user font (256 chars)
                                                      ; (reserved/allocated font space: 2048 bytes)
                                    ; 17/01/2021
                                    ; temporary (initial) location for EDID information
                                    VBE3EDIDINFOBLOCK equ 97D00h ; linear address (128 bytes)

                                    ; FDPT (Phoenix, Enhanced Disk Drive Specification v1.1, v3.0)
                                    ;      (HDPT: Programmer's Guide to the AMIBIOS, 1993)
                                    ;
                                    FDPT_CYLS   equ 0 ; 1 word, number of cylinders
                                    FDPT_HDS    equ 2 ; 1 byte, number of heads
                                    FDPT_TT            equ 3 ; 1 byte, A0h = translated FDPT with logical values
                                                     ; otherwise it is standard FDPT with physical values
                                    FDPT_PCMP   equ 5 ; 1 word, starting write precompensation cylinder
                                                     ; (obsolete for IDE/ATA drives)
                                    FDPT_CB            equ 8 ; 1 byte, drive control byte
                                                       ; Bits 7-6 : Enable or disable retries (00h = enable)
                                                       ; Bit 5 : 1 = Defect map is located at last cyl. + 1
                                                       ; Bit 4 : Reserved. Always 0
                                                       ; Bit 3 : Set to 1 if more than 8 heads
                                                       ; Bit 2-0 : Reserved. Always 0
                                    FDPT_LZ            equ 12 ; 1 word, landing zone (obsolete for IDE/ATA drives)
                                    FDPT_SPT    equ 14 ; 1 byte, sectors per track

                                    ; Floppy Drive Parameters Table (Programmer's Guide to the AMIBIOS, 1993)
                                    ; (11 bytes long) will be used by diskette handler/bios
                                    ; which is derived from IBM PC-AT BIOS (DISKETTE.ASM, 21/04/1986).

                                    ; 01/02/2016
                                    Logical_DOSDisks equ 90000h + 100h ; 26*256 = 6656 bytes
                                    Directory_Buffer equ 80000h ; max = 64K Bytes
                                    FAT_Buffer   equ 91C00h ; 1536 bytes (3 sectors)
                                    ; 15/02/2016
                                    Cluster_Buffer      equ 70000h ; max = 64K Bytes ; buffer for file read & write
                                    ; 11/04/2016
                                    Env_Page     equ 93000h ; 512 bytes (4096 bytes)
                                    Env_Page_Size       equ 512    ; (4096 bytes)
                                    ; 30/07/2016
                                    Video_Pg_Backup     equ 98000h ; Mode 3h, video page backup (32K, 8 pages)

                                    ; 29/11/2020
                                    ; Free/Reserved memory blocks (in 1st 1MB): 93200h to 96000h (available)
                                    ; 06/12/2020
                                    ; Free/Reserved memory blocks (in 1st 1MB): 93200h to 95800h (available)

                                    ; 15/12/2020
                                    LFB_ADDR    equ LFB_Info+LFBINFO.LFB_addr
                                    LFB_SIZE    equ LFB_Info+LFBINFO.LFB_size

                                    ; 04/12/2023 - TRDOS 386 v2.0.7
                                    SYSTEMSTACK_ADDR  equ 97000h      ; max. 3072 bytes (96400h-97000h)
                                    VBE3BIOSCODE_ADDR equ 60000h      ; Protected Mode Video Bios (64KB)
                                    ;AC97DMABUFFR_ADDR equ 40000h     ; AC97 & VIA VT8233 DMA Buffer (128KB)
                                    SB16DMABUFFR_ADDR equ 50000h      ; Sound Blaster 16 DMA Buffer (64KB)
                                    sb16_dma_buffer equ SB16DMABUFFR_ADDR

                                    ; 29/08/2023 - TRDOS 386 v2.0.6
                                        ; 30/11/2020
                                        ; 29/11/2020 - TRDOS 386 v2.0.3

                                    struc PMInfo  ;  VESA VBE3 PMInfoBlock ('PMID' block)

00000000 ????????                    .Signature:       resb 4  ; db 'PMID' ; PM Info Block Signature
00000004 ????                        .EntryPoint:      resw 1  ; Offset of PM entry point within BIOS
00000006 ????                        .PMInitialize: resw 1     ; Offset of PM initialization entry point
00000008 ????                        .BIOSDataSel:     resw 1  ; Selector to BIOS data area emulation block
0000000A ????                        .A0000Sel: resw 1  ; Selector to access A0000h physical mem
0000000C ????                        .B0000Sel: resw 1  ; Selector to access B0000h physical mem
0000000E ????                        .B8000Sel: resw 1  ; Selector to access B8000h physical mem
00000010 ????                        .CodeSegSel:      resw 1  ; Selector to access code segment as data
00000012 ??                          .InProtectMode: resb 1 ; Set to 1 when in protected mode
00000013 ??                          .Checksum: resb 1  ; Checksum byte for structure
                                     .size:

                                    endstruc

                                    [BITS 16]       ; We need 16-bit intructions for Real mode

                                    [ORG 0]
                                        ; 12/11/2014
                                        ; Save boot drive number (that is default root drive)
00000000 8816[8866]                     mov     [boot_drv], dl ; physical drv number

                                        ; Determine installed memory
                                        ; 31/10/2014
                                        ;
00000004 B801E8                         mov     ax, 0E801h ; Get memory size
00000007 CD15                           int     15h        ; for large configurations
00000009 7308                           jnc     short chk_ms
0000000B B488                           mov     ah, 88h    ; Get extended memory size
0000000D CD15                           int     15h
                                        ;
                                        ;mov    al, 17h ; Extended memory (1K blocks) low byte
                                        ;out    70h, al ; select CMOS register
                                        ;in     al, 71h ; read data (1 byte)
                                        ;mov    cl, al
                                        ;mov    al, 18h ; Extended memory (1K blocks) high byte
                                        ;out    70h, al ; select CMOS register
                                        ;in     al, 71h ; read data (1 byte)
                                        ;mov    ch, al
                                        ;
0000000F 89C1                           mov     cx, ax
00000011 31D2                           xor     dx, dx
                                    chk_ms:
00000013 890E[8466]                     mov     [mem_1m_1k], cx
00000017 8916[8666]                     mov     [mem_16m_64k], dx
                                        ; 24/11/2023
0000001B 8916[0E0F]                     mov     [real_mem_16m_64k], dx

                                        ; 05/11/2014
                                        ;and    dx, dx
                                        ;jz     short L2
0000001F 81F90004                          cmp     cx, 1024
                                        ;jnb    short L0
00000023 7351                           jnb     short V0 ; 14/11/2020
                                                 ; insufficient memory_error
                                                 ; Minimum 2 MB memory is needed...
                                        ; 05/11/2014
                                        ; (real mode error printing)
00000025 FB                             sti
```

```
00000026 BE[3A00]                   mov     si, msg_out_of_memory
00000029 BB0700                     mov     bx, 7
0000002C B40E                       mov     ah, 0Eh ; write tty
                                oom_1:
0000002E AC                         lodsb
0000002F 08C0                       or      al, al
00000031 7404                       jz      short oom_2
00000033 CD10                       int     10h
00000035 EBF7                       jmp     short oom_1
                                oom_2:
00000037 F4                             hlt
00000038 EBFD                       jmp     short oom_2

                                ; 20/02/2017
                                ; 05/11/2014
                                msg_out_of_memory:
0000003A 070D0A                     db      07h, 0Dh, 0Ah
0000003D 496E73756666696369-            db      'Insufficient memory !'
00000046 656E74206D656D6F72-
0000004F 792021
00000052 0D0A                       db      0Dh, 0Ah
                                _int13h_48h_buffer: ; 07/07/2016
00000054 284D696E696D756D20-        db      '(Minimum 2MB memory is needed.)'
0000005D 324D42206D656D6F72-
00000066 79206973206E656564-
0000006F 65642E29
00000073 0D0A00                     db      0Dh, 0Ah, 0
                                v0:
                                    ; 18/10/2023 - TRDOS 386 v2.0.7
                                    ; set video mode to 03h again
                                    ; (to reset video bios data in ROMBIOS DATA AREA)
00000076 B80300                     mov     ax, 3
00000079 CD10                       int     10h
                                    ; copy IVT and ROMBIOS DATA AREA to VBE3 BIOS data area
0000007B BF0097                     mov     di, VBE3BIOSDATABLOCK>>4
0000007E 8EC7                       mov     es, di
00000080 31F6                       xor     si, si
00000082 31FF                       xor     di, di
00000084 8EDE                       mov     ds, si ; 0
00000086 B90003                     mov     cx, 300h ; 600h / 2
00000089 F3A5                       rep     movsw
0000008B 0E                         push    cs
0000008C 1F                         pop     ds

                                    ; 24/11/2023
                                    ; 15/12/2020
                                    ;mov    si, [mem_16m_64k]
                                    ;mov    [real_mem_16m_64k], si

                                    ; 15/11/2020
                                    ; 14/11/2020 (TRDOS 386 v2.0.3)
                                    ; check VESA (VBE) VIDEO BIOS version

0000008D B8034F                     mov     ax, 4F03h  ; Return current VBE mode
00000090 CD10                       int     10h
00000092 83F84F                     cmp     ax, 004Fh  ; successful (vbe) function call
                                    ;jne    short L0   ; not a VESA VBE compatible bios
                                    ; 18/10/2023
00000095 7561                       jne     short V1   ; restore es

                                    ; 27/11/2023
                                    ; 24/11/2023 - temporary
                                    ;jmp    short V1

                                    ;mov    ah, 3
                                    ;;jmp   short V1

                                    ; 15/11/2020
00000097 BBE097                     mov     bx, VBE3INFOSEG  ; 97E0h for current version
0000009A 8EC3                       mov     es, bx
0000009C 31FF                       xor     di, di
0000009E 2666C70556424532           mov     dword [es:di], 'VBE2' ; request VESA VBE3 info
                                            ; es:di = buffer address (512 bytes)
                                    ;mov    ax, 4F00h ; Return VBE controller information
000000A6 86C4                       xchg    al, ah
000000A8 CD10                       int     10h

                                    ; dx = cs
                                    ; es = VBE3INFOSEG (97E0h)
                                    ; di = 0
                                    ; ss = (endofkernelfile/16)+16
                                    ; sp = 0FFFEh

000000AA 83F84F                     cmp     ax, 004Fh
000000AD 7549                       jne     short V1 ; old vga bios (not VESA compatible)

                                    ; 15/11/2020
000000AF 2666813D56455341           cmp     dword [es:di], 'VESA'
000000B7 753F                       jne     short V1

                                    ;mov    ax, [es:di+4]
                                    ;       ; ax = vbe version in BCD format (0200h or 0300h)
                                    ;mov    [vbe3], ah ; version number (major)

                                    ; 15/11/2020
000000B9 268A4505                   mov     al, [es:di+5]
                                            ; al = high byte of VBE version number (02h or 03h)

000000BD A2[7C09]                   mov     [vbe3], al ; version number (major)
                                                       ; 02h or 03h is expected
                                    ; 17/01/2021
                                    ; Read EDID
000000C0 B301                       mov     bl, 01h ; Read EDID
000000C2 31C9                       xor     cx, cx  ; Controller unit number
                                                    ; (00 = primary controller)
000000C4 31D2                       xor     dx, dx  ; EDID block number = 0
000000C6 B8C097                     mov     ax, VBE3MODEINFOSEG  ; 97C0h for current version
000000C9 8EC0                       mov     es, ax
000000CB BF0001                     mov     di, VBE3EDIDINFOBLOCK - VBE3MODEINFOBLOCK
                                    ; es:di = temporary address of 128 bytes EDID information
000000CE B8154F                     mov     ax, 4F15h ; VBE/DDC Services
000000D1 CD10                       int     10h
                                    ;cmp    ax, 4Fh
                                    ;jne    short v2
000000D3 A2[1D42]                   mov     [edid], al ; 4Fh > 0
                                ;v2:
                                    ; 17/01/2021
000000D6 31FF                       xor     di, di
                                    ; 15/12/2020
                                    ; Get linear frame buffer info (for VESA VBE mode 118h)
                                    ;mov    si, VBE3MODEINFOSEG  ; 97C0h for current version
                                    ;mov    es, si
                                    ; di = 0
000000D8 B91841                     mov     cx, 04118h  ; 1024*768, 24 bpp, LFB
```

```
000000DB B8014F                     mov     ax, 4F01h ; Return VBE mode information
000000DE CD10                       int     10h
                                    ;cmp    ax, 4Fh
                                    ;jne    short V1
                                    ; 19/12/2020
                                    ;mov    si, [es:di+MODEINFO.PhysBasePtr+2]
                                                    ; hw of LFB base address
                                    ; MODEINFO structure starts from offset -2
000000E0 268B752A                   mov     si, [es:di+MODEINFO.PhysBasePtr] ; hw of LFB addr
000000E4 8936[100F]                 mov     [def_LFB_addr], si ; k_LFB_size = 3145728 bytes
000000E8 81EE0001                   sub     si, 256

                                    ; 15/12/2020
                                    ; check memory and decrease it to 3.5 GB if it is 4GB
                                    ; (reserve upper memory for LFB)
000000EC 8B3E[8666]                 mov     di, [mem_16m_64k]
                                    ; 24/11/2023
                                    ;mov    [real_mem_16m_64k], di

000000F0 39F7                       cmp     di, si
000000F2 7604                       jna     short V1

000000F4 8936[8666]                 mov     [mem_16m_64k], si

                                    ; VESA VBE3 video hardware
                                    ; (example: NVIDIA GEFORCE FX550, 256 MB)
                                    ; uses upper memory from 0D0000000h to 0DFFFFFFFh

                                    ;;cmp   di, 0CF00h ; 3328 MB - 16MB
                                    ;jna    short V1  ; <= 3328 MB memory, it is not required
                                                      ; decrease
                                    ;cmp    al, 3
                                    ;jb     short V2
                                    ; VESA VBE 3
                                    ;mov    word [mem_16m_64k], 0CF00h ; 3328 MB - 16MB
                                    ;jmp    short V1
                                ;V2:
                                    ; VESA VBE 2
                                    ; Check Bochs/Qemu/VirtualBox Emulator
                                    ; LFB base address: 0E0000000h
                                    ;sub    ax, ax ; 0
                                    ;mov    dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
                                    ;out    dx, ax ; VBE_DISPI_INDEX_ID register
                                    ;inc    dx
                                    ;in     ax, dx
                                    ;and    al, 0F0h
                                    ;cmp    ax, 0B0C0h
                                    ;jne    short V1
                                    ;
                                    ; BOCHS/QEMU/VIRTUALBOX
                                    ;mov    word [mem_16m_64k], 0DF00h ; 3584 MB - 16MB
                                V1:
000000F8 1E                         push    ds
000000F9 07                         pop     es ; restore extra data segment
                                L0:

                                %include 'diskinit.s' ; 07/03/2015
                            <1> ; ****************************************************************************
                            <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - diskinit.s
                            <1> ; ----------------------------------------------------------------------------
                            <1> ; Last Update: 09/08/2022 (Previous: 29/08/2020 - Kernel v2.0.4)
                            <1> ; ----------------------------------------------------------------------------
                            <1> ; Beginning: 24/01/2016
                            <1> ; ----------------------------------------------------------------------------
                            <1> ; Assembler: NASM version 2.15 (trdos386.s)
                            <1> ; ----------------------------------------------------------------------------
                            <1> ; Turkish Rational DOS
                            <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                            <1> ;
                            <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                            <1> ; diskinit.inc (10/07/2015)
                            <1> ;
                            <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                            <1> ; ****************************************************************************
                            <1>
                            <1> ; Retro UNIX 386 v1 Kernel - DISKINIT.INC
                            <1> ; Last Modification: 12/07/2022 (Previous: 10/07/2015)
                            <1>
                            <1> ; DISK I/O SYSTEM INITIALIZATION - Erdogan Tan (Retro UNIX 386 v1 project)
                            <1>
                            <1> ; ///////// DISK I/O SYSTEM STRUCTURE INITIALIZATION ///////////////
                            <1>
                            <1>     ; 09/08/2022
                            <1>     ; 08/08/2022
                            <1>     ; 14/07/2022 (TRDOS 386 v2.0.5)
                            <1>     ; 12/07/2022 (Retro UNIX 386 v1.2)
                            <1>     ; 29/08/2020
                            <1>     ; 17/07/2020
                            <1>     ; 14/07/2020 (TRDOS 386 v2.0.2)
                            <1>     ; 10/12/2014 - 02/02/2015 - dsectrm2.s
                            <1> ;L0:
                            <1>     ; 12/11/2014 (Retro UNIX 386 v1 - beginning)
                            <1>     ; Detecting disk drives... (by help of ROM-BIOS)
000000FA BA7F00             <1>     mov     dx, 7Fh
                            <1> L1:
000000FD FEC2               <1>     inc     dl
000000FF B441               <1>     mov     ah, 41h ; Check extensions present
                            <1>                     ; Phoenix EDD v1.1 - EDD v3
00000101 BBAA55             <1>     mov     bx, 55AAh
00000104 CD13               <1>     int     13h
00000106 721A               <1>     jc      short L2
                            <1>
00000108 81FB55AA           <1>     cmp     bx, 0AA55h
0000010C 7514               <1>     jne     short L2
0000010E FE06[8B66]         <1>     inc     byte [hdc]      ; count of hard disks (EDD present)
00000112 8816[8A66]         <1>         mov     [last_drv], dl  ; last hard disk number
00000116 BB[0E66]           <1>     mov     bx, hd0_type - 80h
00000119 01D3               <1>     add     bx, dx
0000011B 880F               <1>     mov     [bx], cl ; Interface support bit map in CX
                            <1>                     ; Bit 0 - 1, Fixed disk access subset ready
                            <1>                     ; Bit 1 - 1, Drv locking and ejecting ready
                            <1>                     ; Bit 2 - 1, Enhanced Disk Drive Support
                            <1>                         ;            (EDD) ready (DPTE ready)
                            <1>                     ; Bit 3 - 1, 64bit extensions are present
                            <1>                         ;            (EDD-3)
                            <1>                     ; Bit 4 to 15 - 0, Reserved
0000011D 80FA83             <1>     cmp     dl, 83h ; drive number < 83h
00000120 72DB               <1>     jb      short L1
                            <1> L2:
                            <1>     ; 23/11/2014
                            <1>     ; 19/11/2014
00000122 30D2               <1>     xor     dl, dl  ; 0
                            <1>     ; 04/02/2016 (esi -> si)
00000124 BE[8C66]           <1>     mov     si, fd0_type
```

```
                                <1> L3:
                                <1>     ; 14/01/2015
00000127 8816[8966]             <1>     mov     [drv], dl
                                <1>     ;
0000012B B408                   <1>     mov     ah, 08h ; Return drive parameters
0000012D CD13                   <1>     int     13h
0000012F 7210                   <1>     jc      short L4
                                <1>             ; BL = drive type (for floppy drives)
                                <1>             ; DL = number of floppy drives
                                <1>             ;
                                <1>             ; ES:DI = Address of DPT from BIOS
                                <1>             ;
00000131 881C                   <1>     mov     [si], bl ;  Drive type
                                <1>                     ; 4 = 1.44 MB, 80 track, 3 1/2"
                                <1>     ; 14/01/2015
00000133 E8DD01                 <1>     call    set_disk_parms
                                <1>     ; 10/12/2014
00000136 81FE[8C66]             <1>     cmp     si, fd0_type
0000013A 7705                   <1>     ja      short L4
0000013C 46                     <1>     inc     si ; fd1_type
0000013D B201                   <1>     mov     dl, 1
0000013F EBE6                   <1>     jmp     short L3
                                <1> L4:
00000141 B27F                   <1>     mov     dl, 7Fh
                                <1>     ; 24/12/2014
00000143 803E[8B66]00           <1>     cmp     byte [hdc], 0 ; EDD present or not ?
                                <1>     ;ja     L10       ; yes, all fixed disk operations
                                <1>                       ; will be performed according to
                                <1>                       ; present EDD specification
                                <1>     ; 14/07/2022
00000148 7603                   <1>     jna     short L5
0000014A E98B00                 <1>     jmp     L10
                                <1>
                                <1> L5:
                                <1>     ; 17/07/2020
                                <1>     ; Note: Virtual CPU will not come here while
                                <1>     ; running in QEMU, Bochs, VirtualBox emulators !!!
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ; Older BIOS (INT 13h, AH = 48h is not available)
                                <1>
0000014D FEC2                   <1>     inc     dl
0000014F 8816[8966]             <1>         mov     [drv], dl
00000153 8816[8A66]             <1>         mov     [last_drv], dl ; 14/01/2015
00000157 B408                   <1>     mov     ah, 08h ; Return drive parameters
00000159 CD13                   <1>     int     13h     ; (conventional function)
                                <1>     ;jc     L13     ; fixed disk drive not ready
                                <1>     ; 14/07/2022
0000015B 7303                   <1>     jnc     short L6
0000015D E9A501                 <1>     jmp     L13
                                <1> L6:
00000160 8816[8B66]             <1>         mov     [hdc], dl ; number of drives
                                <1>     ;; 14/01/2013
                                <1>     ;;push  cx
00000164 E8AC01                 <1>     call    set_disk_parms
                                <1>     ;;pop   cx
                                <1>     ;
                                <1>     ;;and   cl, 3Fh  ; sectors per track (bits 0-6)
00000167 8A16[8966]             <1>         mov     dl, [drv]
0000016B BB0401                 <1>     mov     bx, 65*4 ; hd0 parameters table (INT 41h)
0000016E 80FA80                 <1>     cmp     dl, 80h
00000171 7603                   <1>     jna     short L7
00000173 83C314                 <1>     add     bx, 5*4  ; hd1 parameters table (INT 46h)
                                <1> L7:
00000176 31C0                   <1>     xor     ax, ax
00000178 8ED8                   <1>     mov     ds, ax
0000017A 8B37                   <1>         mov     si, [bx]
0000017C 8B4702                 <1>         mov     ax, [bx+2]
0000017F 8ED8                   <1>     mov     ds, ax
00000181 3A4C0E                 <1>         cmp     cl, [si+FDPT_SPT] ; sectors per track
                                <1>     ;jne    L12 ; invalid FDPT
                                <1>     ; 14/07/2022
00000184 7403                   <1>     je      short L7_8
00000186 E97801                 <1>     jmp     L12
                                <1> L7_8:
00000189 BF0000                 <1>     mov     di, HD0_DPT
0000018C 80FA80                 <1>     cmp     dl, 80h
0000018F 7603                   <1>     jna     short L8
00000191 BF2000                 <1>     mov     di, HD1_DPT
                                <1> L8:
                                <1>     ; 30/12/2014
00000194 B80090                 <1>     mov     ax, DPT_SEGM
00000197 8EC0                   <1>     mov     es, ax
                                <1>     ; 24/12/2014
00000199 B90800                 <1>     mov     cx, 8
0000019C F3A5                   <1>     rep     movsw  ; copy 16 bytes to the kernel's DPT location
0000019E 8CC8                   <1>     mov     ax, cs
000001A0 8ED8                   <1>     mov     ds, ax
                                <1>
                                <1>     ; 02/02/2015
                                <1>     ;mov    cl, [drv]
                                <1>     ;mov    bl, cl
                                <1>     ;mov    ax, 1F0h
                                <1>     ;and    bl, 1
                                <1>     ;jz     short L9
                                <1>     ;shl    bl, 4
                                <1>     ;sub    ax, 1F0h-170h
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ; (Only 1F0h port address must be valid for old ROM BIOSes)
000001A2 B8F001                 <1>     mov     ax, 1F0h
000001A5 B3A0                   <1>     mov     bl, 0A0h
000001A7 80FA80                 <1>     cmp     dl, 80h
000001AA 7603                   <1>     jna     short L9
                                <1>     ; dl = 81h
000001AC 80C310                 <1>     add     bl, 10h  ; slave disk
                                <1>     ;sub    ax, 1F0h-170h
                                <1> L9:
000001AF AB                     <1>     stosw   ; I/O PORT Base Address (1F0h, 170h)
000001B0 050602                 <1>     add     ax, 206h
000001B3 AB                     <1>     stosw   ; CONTROL PORT Address (3F6h, 376h)
000001B4 88D8                   <1>     mov     al, bl  ; bit 4, master/slave disk bit
                                <1>     ;add    al, 0A0h ; 17/07/2020
000001B6 AA                     <1>     stosb   ; Device/Head Register upper nibble
                                <1>     ;
000001B7 FE06[8966]             <1>     inc     byte [drv]
                                <1>     ;mov    bx, hd0_type - 80h
                                <1>     ;add    bx, cx
                                <1>     ; 09/08/2022 - BugFix
000001BB 30FF                   <1>     xor     bh, bh
000001BD 88D3                   <1>     mov     bl, dl
000001BF 81C3[0E66]             <1>     add     bx, hd0_type - 80h
000001C3 800F80                 <1>     or      byte [bx], 80h  ; present sign (when lower nibble is 0)
000001C6 A0[8B66]               <1>     mov     al, [hdc]
```

```
000001C9 FEC8                <1>     dec     al
                             <1>     ;jz     L13
                             <1>     ; 14/07/2022
000001CB 7408                <1>     jz      short L9_10
000001CD 80FA80              <1>     cmp     dl, 80h
                             <1>         ;jna       L5 ; Max. 2 hard disks  ; 17/07/2020
                             <1>     ; 14/07/2022
000001D0 7703                <1>     ja      short L9_10
000001D2 E978FF              <1>     jmp     L5
                             <1> L9_10:
000001D5 E92D01              <1>         jmp     L13
                             <1> L10:
000001D8 FEC2                <1>     inc     dl
                             <1>     ; 25/12/2014
000001DA 8816[8966]          <1>     mov     [drv], dl
000001DE B408                <1>     mov     ah, 08h ; Return drive parameters
000001E0 CD13                <1>     int     13h     ; (conventional function)
                             <1>     ;jc     L13
                             <1>     ; 14/07/2022
000001E2 72F1                <1>     jc      short L9_10
                             <1>     ; 14/01/2015
                             <1>     ;mov    dl, [drv]
                             <1>     ; 09/08/2022
                             <1>     ;push   dx
000001E4 51                  <1>     push    cx
000001E5 E82B01              <1>     call    set_disk_parms
000001E8 59                  <1>     pop     cx
                             <1>     ;pop    dx
                             <1>     ; 09/08/2022
000001E9 8A16[8966]          <1>     mov     dl, [drv]
                             <1>     ; 06/07/2016 (BugFix for >64K kernel files)
                             <1>     ; 04/02/2016 (esi -> si)
                             <1>     ;mov    si, _end ; 30 byte temporary buffer address
                             <1>     ;               ; at the '_end' of kernel.
                             <1>     ;mov    word [si], 30
                             <1>     ; 06/07/2016
000001ED BE[5400]            <1>     mov     si, _int13h_48h_buffer
                             <1>     ; 09/07/2016
000001F0 B81E00              <1>     mov     ax, 001Eh
000001F3 8824                <1>     mov     [si], ah ; 0
000001F5 46                  <1>     inc     si
000001F6 8904                <1>     mov     [si], ax
                             <1>     ; word [si] = 30
                             <1>     ;
000001F8 B448                <1>     mov     ah, 48h ; Get drive parameters (EDD function)
000001FA CD13                <1>     int     13h
                             <1>     ;jc     L13
                             <1>     ; 14/07/2022
000001FC 72D7                <1>     jc      short L9_10
                             <1>
                             <1>     ; 29/08/2020
                             <1>     ; 04/02/2016 (ebx -> bx)
                             <1>     ; 14/01/2015
000001FE 28FF                <1>     sub     bh, bh
00000200 88D3                <1>     mov     bl, dl
                             <1>     ;sub    bl, 80h
                             <1>     ; 29/08/2020
00000202 81C3[0E66]          <1>     add     bx, (hd0_type - 80h)
                             <1>     ;mov    al, [bx]
00000206 8A07                <1>     mov     al, [bx]
00000208 0C80                <1>     or      al, 80h
0000020A 8807                <1>     mov     [bx], al
0000020C 81EB[8C66]          <1>     sub     bx, hd0_type - 2 ; 15/01/2015
                             <1>     ;add    bx, drv.status
                             <1>     ;mov    [bx], al
                             <1>     ; 29/08/2020
00000210 8887[AE66]          <1>     mov     [bx+drv.status], al
                             <1>     ; 04/02/2016 (eax -> ax)
                             <1>     ;mov    ax, [si+16]
                             <1>     ; 14/07/2020
                             <1>     ;mov    di, [si+18]
                             <1>     ;;test  ax, [si+18]
                             <1>     ;test   ax, di ; 14/07/2020
                             <1>     ;jz     short L10_A0h ; (!) ; 17/07/2020
                             <1>                     ; 'CHS only' disks on EDD system
                             <1>                     ;  are reported with ZERO disk size
                             <1>                     ; (if so, we must not overwrite
                             <1>                     ; calculated disk size in 'set_disk_parms')
                             <1>     ; 29/08/2020
00000214 8B4410              <1>     mov     ax, [si+16]
00000217 8B7C12              <1>     mov     di, [si+18]
0000021A 09C0                <1>     or      ax, ax
0000021C 7504                <1>     jnz     short L10_LBA
0000021E 09FF                <1>     or      di, di
00000220 740B                <1>     jz      short L10_A0h
                             <1> L10_LBA:
                             <1>     ;sub    bx, drv.status
00000222 C1E302              <1>     shl     bx, 2
                             <1>     ;add    bx, drv.size ; disk size (in sectors)
                             <1>     ;mov    [bx], ax
                             <1>     ; 29/08/2020
00000225 8987[9266]          <1>     mov     [bx+drv.size], ax
                             <1>     ;mov    ax, [si+18]
                             <1>     ;;mov   [bx], ax
                             <1>     ;mov    [bx+2], ax ; BugFix ; 15/07/2020
                             <1>     ; 14/07/2020
                             <1>     ;mov    [bx+2], di ; 15/07/2020
                             <1>     ; 29/08/2020
00000229 89BF[9466]          <1>     mov     [bx+drv.size+2], di
                             <1> L10_A0h:
                             <1>     ; 17/07/2020
                             <1>     ; Note: Virtual CPU will jump here from above (!) test
                             <1>     ;       while running in QEMU
                             <1>
                             <1>     ; Jump here to fix a ZERO (LBA) disk size problem
                             <1>     ; for CHS disks (28/02/2015)
                             <1>
                             <1>     ; 30/12/2014
0000022D BF0000              <1>     mov     di, HD0_DPT
00000230 88D0                <1>     mov     al, dl
00000232 83E003              <1>     and     ax, 3
00000235 C0E005              <1>     shl     al, 5 ; * 32
00000238 01C7                <1>     add     di, ax
0000023A B80090              <1>     mov     ax, DPT_SEGM
0000023D 8EC0                <1>     mov     es, ax
                             <1>     ;
0000023F 88E8                <1>     mov     al, ch  ; max. cylinder number (bits 0-7)
00000241 88CC                <1>     mov     ah, cl
00000243 C0EC06              <1>     shr     ah, 6   ; max. cylinder number (bits 8-9)
00000246 40                  <1>     inc     ax      ; logical cylinders (limit 1024)
00000247 AB                  <1>     stosw
00000248 88F0                <1>     mov     al, dh  ; max. head number
                             <1>     ;
0000024A 30F6                <1>     xor     dh, dh  ; 29/08/2020 (dh = 0 is needed here)
```

```
                                <1>     ;
0000024C FEC0                   <1>     inc     al
0000024E AA                     <1>     stosb           ; logical heads (limits 256)
0000024F B0A0                   <1>     mov     al, 0A0h ; Indicates translated table
00000251 AA                     <1>     stosb
00000252 8A440C                 <1>     mov     al, [si+12]
00000255 AA                     <1>     stosb           ; physical sectors per track
00000256 31C0                   <1>     xor     ax, ax
                                <1>     ;dec    ax      ; 02/01/2015
00000258 AB                     <1>     stosw           ; precompensation (obsolete)
                                <1>     ;xor    al, al  ; 02/01/2015
00000259 AA                     <1>     stosb           ; reserved
0000025A B008                   <1>     mov     al, 8   ; drive control byte
                                <1>                     ; (do not disable retries,
                                <1>                     ; more than 8 heads)
0000025C AA                     <1>     stosb
0000025D 8B4404                 <1>     mov     ax, [si+4]
00000260 AB                     <1>     stosw           ; physical number of cylinders
                                <1>     ;push   ax      ; 02/01/2015
00000261 8A4408                 <1>     mov     al, [si+8]
00000264 AA                     <1>     stosb           ; physical num. of heads (limit 16)
00000265 29C0                   <1>     sub     ax, ax
                                <1>     ;pop    ax      ; 02/01/2015
00000267 AB                     <1>     stosw           ; landing zone (obsolete)
00000268 88C8                   <1>     mov     al, cl  ; logical sectors per track (limit 63)
0000026A 243F                   <1>     and     al, 3Fh
0000026C AA                     <1>     stosb
                                <1>     ;sub    al, al  ; checksum
                                <1>     ;stosb
                                <1>     ;
0000026D 83C61A                 <1>     add     si, 26   ; (BIOS) DPTE address pointer
00000270 AD                     <1>     lodsw
00000271 50                     <1>     push    ax ; *  ; (BIOS) DPTE offset
00000272 AD                     <1>     lodsw
00000273 50                     <1>     push    ax ; ** ; (BIOS) DPTE segment
                                <1>     ;
                                <1>     ; checksum calculation
00000274 89FE                   <1>     mov     si, di
00000276 06                     <1>     push    es
00000277 1F                     <1>     pop     ds
                                <1>     ;mov    cx, 16
00000278 B90F00                 <1>     mov     cx, 15
0000027B 29CE                   <1>     sub     si, cx
0000027D 30E4                   <1>     xor     ah, ah
                                <1>     ;del    cl
                                <1> L11:
0000027F AC                     <1>     lodsb
00000280 00C4                   <1>     add     ah, al
00000282 E2FB                   <1>     loop    L11
                                <1>     ;
00000284 88E0                   <1>     mov     al, ah
00000286 F6D8                   <1>     neg     al      ; -x+x = 0
00000288 AA                     <1>     stosb           ; put checksum in byte 15 of the tbl
                                <1>     ;
00000289 1F                     <1>     pop     ds ; ** ; (BIOS) DPTE segment
0000028A 5E                     <1>     pop     si ; * ; (BIOS) DPTE offset
                                <1>
                                <1>     ; 08/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; (Recent version of Retro UNIX 386 v1 'diskinit.s' file
                                <1>     ; -12/07/2022- does not contain following 2020 code) (*)
                                <1>
                                <1>     ; 14/07/2020 (TRDOS 386 v2.0.2)
                                <1>     ; 0FFFFh:0FFFFh = invalid DPTE address
0000028B 8B0C                   <1>     mov     cx, [si]
0000028D 8B4402                 <1>     mov     ax, [si+2]
00000290 21C1                   <1>     and     cx, ax
00000292 41                     <1>     inc     cx
00000293 7404                   <1>     jz      short L11c ; 0FFFFh:0FFFFh
00000295 0B04                   <1>     or      ax, [si]
00000297 752A                   <1>     jnz     short L11e ; <> 0
                                <1> L11c:
                                <1>     ; 17/07/2020
                                <1>     ; TRDOS 386 v2 DRVINIT assumptions:
                                <1>     ; (also by regarding QEMU, Bochs and VirtualBox settings)
                                <1>     ;       Hard disk 0 port address: 1F0h
                                <1>     ;       Hard disk 1 port address: 1F0h
                                <1>     ;       Hard disk 2 port address: 170h
                                <1>     ;       Hard disk 3 port address: 170h
                                <1>
                                <1>     ; in QEMU, hda=hd0 (1F0h) and hdb=hd1 (1F0h) -IRQ14-
                                <1>     ;      and hdc=hd2 (170h) and hdd=hd3 (170h) -IRQ15-
                                <1>
00000299 B8F001                 <1>     mov     ax, 1F0h
                                <1>
                                <1>     ; 15/07/2020
                                <1>     ; 14/07/2020
                                <1>     ; Invalid DPTE address...
                                <1>     ; Default DPTE parms must be set for DISK_IO_CONT
                                <1>     ; (diskio.s)
                                <1>     ; 17/07/2020
                                <1>
                                <1>     ;mov    bl, dl
                                <1>     ;and    bl, 1
                                <1>     ;jz     short L11d
                                <1>
0000029C B3A0                   <1>     mov     bl, 0A0h
                                <1>
0000029E F6C201                 <1>     test    dl, 1
000002A1 7403                   <1>     jz      short L11g  ; Master (as default, for 80h & 82h))
                                <1>     ;shl    bl, 4 ; bl = 16 (bit 4 = 1 -> slave)
000002A3 80C310                 <1>     add     bl, 10h  ; Slave (as default, for 81h & 83h)
                                <1> L11g:
                                <1>     ; 17/07/2020
000002A6 80FA82                 <1>     cmp     dl, 82h ; Hard disk 3 or 4 ?
000002A9 7203                   <1>     jb      short L11d ; Primary ATA channel (hd0, hd1)
                                <1>                        ; (port address = 1F0h)
                                <1>
                                <1>     ; Secondary ATA channel (hd2, hd3)
                                <1>     ; (port address = 170h)
                                <1>
000002AB 2D8000                 <1>     sub     ax, 1F0h-170h
                                <1> L11d:
                                <1>     ; 14/07/2020
000002AE AB                     <1>     stosw   ; I/O PORT Base Address (1F0h, 170h)
000002AF 050602                 <1>     add     ax, 206h
000002B2 AB                     <1>     stosw   ; CONTROL PORT Address (3F6h, 376h)
000002B3 88D8                   <1>     mov     al, bl  ; Master/Slave bit (0 = Master)
                                <1>     ; 17/07/2020
                                <1>     ;or     al, 0A0h ; CHS (LBA enable bit = 0)
                                <1>                      ; (Bits 5&7, reserved bits  =  1)
000002B5 30E4                   <1>     xor     ah, ah
                                <1>     ;stosb ; Device/Head Register upper nibble
000002B7 AB                     <1>     stosw
000002B8 30C0                   <1>     xor     al, al
```

```
000002BA B90500             <1>     mov     cx, 5
000002BD F3AB               <1>     rep     stosw ; clear remain part of the (fake) DPTE
000002BF 0E                 <1>     push    cs
000002C0 1F                 <1>     pop     ds
000002C1 EB2E               <1>     jmp     short L11f
                            <1>
                            <1>     ; 08/08/2022 (TRDOS 386 v2.0.5)
                            <1>     ; (Recent version of Retro UNIX 386 v1 'diskinit.s' file
                            <1>     ; -12/07/2022- does not contain above 2020 code) (*)
                            <1> L11e:
                            <1>     ; 23/02/2015
000002C3 57                 <1>     push    di
                            <1>     ; ES:DI points to DPTE (FDPTE) location
                            <1>     ;;mov   cx, 8
                            <1>     ;mov    cl, 8
000002C4 B90800             <1>     mov     cx, 8 ; 14/07/2020
000002C7 F3A5               <1>     rep     movsw
                            <1>     ;
                            <1>     ; 23/02/2015
                            <1>     ; (P)ATA drive and LBA validation
                            <1>     ; (invalidating SATA drives and setting
                            <1>     ; CHS type I/O for old type fixed disks)
000002C9 5B                 <1>     pop     bx
000002CA 8CC8               <1>     mov     ax, cs
000002CC 8ED8               <1>     mov     ds, ax
000002CE 268B07             <1>     mov     ax, [es:bx]
000002D1 3DF001             <1>     cmp     ax, 1F0h
000002D4 7413               <1>     je      short L11a
000002D6 3D7001             <1>     cmp     ax, 170h
000002D9 740E               <1>     je      short L11a
                            <1>     ; invalidation
                            <1>     ; (because base port address is not 1F0h or 170h)
                            <1>     ;xor    bh, bh
                            <1>     ;mov    bl, dl
                            <1>     ; 29/08/2020
000002DB 89D3               <1>     mov     bx, dx
                            <1>     ;sub    bl, 80h
                            <1>     ;mov    byte [bx+hd0_type], 0 ; not a valid disk drive !
                            <1>         ;or byte [bx+drv.status+2], 0F0h ; (failure sign)
                            <1>     ; 29/08/2020
000002DD C687[0E66]00       <1>     mov     byte [bx+hd0_type-80h], 0
000002E2 808F[3066]F0       <1>     or      byte [bx+drv.status-7Eh], 0F0h
000002E7 EB0F               <1>     jmp     short L11b
                            <1> L11a:
                            <1>     ; LBA validation
000002E9 268A4704           <1>     mov     al, [es:bx+4] ; Head register upper nibble
000002ED A840               <1>     test    al, 40h ; LBA bit (bit 6)
000002EF 7507               <1>     jnz     short L11b ; LBA type I/O is OK! (E0h or F0h)
                            <1> L11f:
                            <1>     ; force CHS type I/O for this drive (A0h or B0h)
                            <1>     ;sub    bh, bh
                            <1>     ;mov    bl, dl
                            <1>     ; 29/08/2020
000002F1 89D3               <1>     ;xor    dh, dh ; 0
                            <1>     mov     bx, dx
                            <1>     ;sub    bl, 80h ; 26/02/2015
                            <1>         ;and        byte [bx+drv.status+2], 0FEh ; clear bit 0
                            <1>                             ; bit 0 = LBA ready bit
                            <1>     ; 29/08/2020
000002F3 80A7[3066]FE       <1>     and     byte [bx+drv.status-7Eh], 0FEh
                            <1>     ; 'diskio' procedure will check this bit !
                            <1> L11b:
000002F8 3A16[8A66]         <1>     cmp     dl, [last_drv] ; 25/12/2014
000002FC 7307               <1>         jnb     short L13
000002FE E9D7FE             <1>         jmp     L10
                            <1>
                            <1> L12:
                            <1>     ; Restore data registers
00000301 8CC8               <1>     mov     ax, cs
00000303 8ED8               <1>     mov     ds, ax
                            <1> L13:
                            <1>     ; 13/12/2014
00000305 0E                 <1>     push    cs
00000306 07                 <1>     pop     es
                            <1> L14:
                            <1>     ; clear keyboard buffer
00000307 B411               <1>     mov     ah, 11h
00000309 CD16               <1>     int     16h
0000030B 7440               <1>     jz      short L16 ; no keys in keyboard buffer
0000030D B010               <1>     mov     al, 10h
0000030F CD16               <1>     int     16h
00000311 EBF4               <1>     jmp     short L14
                            <1>
                            <1> set_disk_parms:
                            <1>     ; 08/08/2022 - TRDOS 386 v2.0.5
                            <1>     ; 09/05/2022 - Retro UNIX 386 v1.2
                            <1>     ; 29/08/2020 - TRDOS 386 v2.0.2
                            <1>     ; 04/02/2016 (ebx -> bx)
                            <1>     ; 10/07/2015
                            <1>     ; 14/01/2015
                            <1>     ;push   bx
00000313 28FF               <1>     sub     bh, bh
00000315 8A1E[8966]         <1>     mov     bl, [drv]
00000319 80FB80             <1>     cmp     bl, 80h
0000031C 7203               <1>     jb      short sdp0
0000031E 80EB7E             <1>     sub     bl, 7Eh
                            <1> sdp0:
                            <1>     ;add    bx, drv.status
                            <1>     ;mov    byte [bx], 80h ; 'Present' flag
                            <1>     ; 29/08/2020
00000321 C687[AE66]80       <1>     mov     byte [bx+drv.status], 80h
                            <1>     ;
00000326 88E8               <1>     mov     al, ch ; last cylinder (bits 0-7)
00000328 88CC               <1>     mov     ah, cl ;
0000032A C0EC06             <1>     shr     ah, 6  ; last cylinder (bits 8-9)
                            <1>     ;sub    bx, drv.status
0000032D D0E3               <1>     shl     bl, 1
                            <1>     ;add    bx, drv.cylinders
0000032F 40                 <1>     inc     ax  ; convert max. cyl number to cyl count
                            <1>     ;mov    [bx], ax
                            <1>     ; 08/08/2022
                            <1>     ; 29/08/2020
                            <1>     ;mov    [bx+drv.cylinders], ax
                            <1>     ;
00000330 50                 <1>     push    ax ; ** cylinders
                            <1>     ;sub    bx, drv.cylinders
                            <1>     ;add    bx, drv.heads
00000331 30E4               <1>     xor     ah, ah
00000333 88F0               <1>     mov     al, dh ; heads
00000335 40                 <1>     inc     ax
                            <1>     ;mov    [bx], ax
                            <1>     ; 08/08/2022
                            <1>     ; 29/08/2020
```

```
                                <1>     ;mov    [bx+drv.heads], ax
                                <1>     ;sub    bx, drv.heads
                                <1>         ;add        bx, drv.spt
00000336 30ED                   <1>     xor     ch, ch
00000338 80E13F                 <1>     and     cl, 3Fh ; sectors (bits 0-6)
                                <1>     ;mov    [bx], cx
                                <1>         ; 08/08/2022
                                <1>     ; 29/08/2020
                                <1>     ;mov    [bx+drv.spt], cx
                                <1>     ;sub    bx, drv.spt
0000033B D1E3                   <1>     shl     bx, 1
                                <1>     ;add    bx, drv.size ; disk size (in sectors)
                                <1>     ; LBA size = cylinders * heads * secpertrack
0000033D F7E1                   <1>     mul     cx
0000033F 89C2                   <1>     mov     dx, ax  ; heads*spt
00000341 58                     <1>     pop     ax ; ** cylinders
                                <1>     ; 09/05/2022 (fd0&fd1 drv.size = cyls*spt*heads)
                                <1>     ;dec    ax ; 1 cylinder reserved (!?) ; (*)
00000342 F7E2                   <1>     mul     dx ; cylinders * (heads*spt)
                                <1>     ;mov    [bx], ax
                                <1>     ;mov    [bx+2], dx
                                <1>     ; 29/08/2020
00000344 8987[9266]             <1>     mov     [bx+drv.size], ax
00000348 8997[9466]             <1>     mov     [bx+drv.size+2], dx
                                <1>     ;
                                <1>     ;pop    bx
0000034C C3                     <1>     retn
                                <1>
                                <1> L16:; 28/05/2016

                                        ; 10/11/2014
0000034D FA                             cli     ; Disable interrupts (clear interrupt flag)
                                        ; Reset Interrupt MASK Registers (Master&Slave)
                                        ;mov    al, 0FFh        ; mask off all interrupts
                                        ;out    21h, al         ; on master PIC (8259)
                                        ;jmp    $+2  ; (delay)
                                        ;out    0A1h, al        ; on slave PIC (8259)
                                        ;
                                        ; Disable NMI
0000034E B080                           mov     al, 80h
00000350 E670                           out     70h, al         ; set bit 7 to 1 for disabling NMI
                                        ;23/02/2015
                                        ;nop                    ;
                                        ;in     al, 71h         ; read in 71h just after writing out to 70h
                                                                ; for preventing unknown state (!?)
                                        ;
                                        ; 20/08/2014
                                        ; Moving the kernel 64 KB back (to physical address 0)
                                        ; DS = CS = 1000h
                                        ; 05/11/2014
00000352 31C0                           xor     ax, ax
00000354 8EC0                           mov     es, ax ; ES = 0
                                        ;
                                        ; 04/07/2016 - TRDOS 386 (64K - 128K kernel)
00000356 31F6                                   xor     si, si
00000358 31FF                           xor     di, di
                                        ;mov    cx, 16384
                                        ;rep    movsd
                                        ; 02/12/2023
0000035A B90080                         mov     cx, 32768
0000035D F3A5                           rep     movsw
                                        ;
0000035F 06                             push    es ; 0
00000360 68[6403]                       push    L17
00000363 CB                             retf
                                    L17:
00000364 B90010                         mov     cx, 1000h
00000367 8EC1                           mov     es, cx  ; 1000h
00000369 01C9                           add     cx, cx
0000036B 8ED9                           mov     ds, cx  ; 2000h
                                        ;sub    si, si
                                        ;sub    di, di
                                        ;mov    cx, 16384
                                        ;rep    movsd
                                        ; 02/12/2023
                                        ; si = di = 0
0000036D B90080                         mov     cx, 32768
00000370 F3A5                           rep     movsw

                                        ; Turn off the floppy drive motor
00000372 BAF203                             mov     dx, 3F2h
00000375 EE                                 out     dx, al ; 0 ; 31/12/2013

                                        ; Enable access to memory above one megabyte
                                    L18:
00000376 E464                           in      al, 64h
00000378 A802                           test    al, 2
0000037A 75FA                               jnz     short L18
0000037C B0D1                           mov     al, 0D1h        ; Write output port
0000037E E664                           out     64h, al
                                    L19:
00000380 E464                           in      al, 64h
00000382 A802                           test    al, 2
00000384 75FA                               jnz     short L19
00000386 B0DF                           mov     al, 0DFh        ; Enable A20 line
00000388 E660                           out     60h, al
                                    ;L20:
                                        ;
                                        ; Load global descriptor table register

                                            ;mov     ax, cs
                                            ;mov     ds, ax

0000038A 2E0F0116[F865]                     lgdt    [cs:gdtd]

00000390 0F20C0                             mov     eax, cr0
                                        ;or     al, 1   ; 24/07/2023
00000393 40                             inc     ax
00000394 0F22C0                         mov     cr0, eax

                                        ; Jump to 32 bit code

00000397 66                             db 66h                  ; Prefix for 32-bit
00000398 EA                             db 0EAh                 ; Opcode for far jump
00000399 [9F030000]                     dd StartPM              ; Offset to start, 32-bit
                                                                ; (1000h:StartPM = StartPM + 10000h)
0000039D 0800                           dw KCODE                ; This is the selector for CODE32_DESCRIPTOR,
                                                                ; assuming that StartPM resides in code32

                                    ; 20/02/2017


                                    [BITS 32]

```

```
                                StartPM:
                                    ; Kernel Base Address = 0 ; 30/12/2013
0000039F 66B81000                   mov ax, KDATA           ; Save data segment identifier
000003A3 8ED8                           mov ds, ax              ; Move a valid data segment into DS register
000003A5 8EC0                               mov es, ax              ; Move data segment into ES register
000003A7 8EE0                               mov fs, ax              ; Move data segment into FS register
000003A9 8EE8                               mov gs, ax              ; Move data segment into GS register
000003AB 8ED0                           mov ss, ax              ; Move data segment into SS register
                                    ;mov esp, 90000h        ; Move the stack pointer to 090000h
                                    ; 04/12/2023 - TRDOS 386 v2.0.7
                                    ;mov esp, 97000h      ; 3072 bytes system stack (96400h-97000h)
000003AD BC00700900                 mov esp, SYSTEMSTACK_ADDR ; 97000h (max. 3072 bytes)

                                clear_bss: ; Clear uninitialized data area
                                    ; 11/03/2015
000003B2 31C0                       xor     eax, eax ; 0
000003B4 B97B0A0000                 mov     ecx, (bss_end - bss_start)/4
                                    ;shr    ecx, 2 ; bss section is already aligned for double words
000003B9 BF[C3730100]               mov     edi, bss_start
000003BE F3AB                       rep     stosd

                                memory_init:
                                    ; Initialize memory allocation table and page tables
                                    ; 04/12/2023
                                    ; 29/11/2023
                                    ; 27/11/2023
                                    ; 23/11/2023 (TRDOS 386 v2.0.7)
                                    ; 24/07/2022 (TRDOS 386 v2.0.5)
                                    ; 18/04/2021 (TRDOS 386 v2.0.4)
                                    ; 16/11/2014
                                    ; 15/11/2014
                                    ; 07/11/2014
                                    ; 06/11/2014
                                    ; 05/11/2014
                                    ; 04/11/2014
                                    ; 31/10/2014 (Retro UNIX 386 v1 - Beginning)
                                    ;
                                ;   xor     eax, eax
                                ;   xor     ecx, ecx
000003C0 B108                       mov     cl, 8
000003C2 BF00001000                 mov     edi, MEM_ALLOC_TBL
000003C7 F3AB                       rep     stosd               ; clear Memory Allocation Table
                                                                ; for the first 1 MB memory
                                    ;
000003C9 668B0D[84660000]           mov     cx, [mem_1m_1k]           ; Number of contiguous KB between
                                                                ; 1 and 16 MB, max. 3C00h = 15 MB.
                                    ;shr    cx, 2               ; convert 1 KB count to 4 KB count
                                    ; 24/07/2022
000003D0 C1E902                     shr     ecx, 2
000003D3 890D[B8760100]             mov     [free_pages], ecx
000003D9 668B15[86660000]           mov     dx, [mem_16m_64k]  ; Number of contiguous 64 KB blocks
                                                                ; between 16 MB and 4 GB.
000003E0 6609D2                     or      dx, dx
000003E3 741A                       jz      short mi_0

                                ; 09/02/2026
                                ; 04/12/2023
                                %if 0
                                    ; 02/12/2023 - temporary (2816MB limit)
                                    ;cmp    dx, 44800 ; 0AF00h
                                    cmp     dx, 40704      ; (2560MB limit)
                                    jna     short mi_x
                                    ;mov    dx, 44800
                                    mov     dx, 40704
                                mi_x:
                                %endif
                                    ; 23/11/2023 - temporary
                                    ;and    dx, 3FFFh

000003E5 668915[86660000]           mov     [mem_16m_64k], dx

000003EC 6689D0                     mov     ax, dx
000003EF C1E004                     shl     eax, 4              ; 64 KB -> 4 KB (page count)
000003F2 0105[B8760100]             add     [free_pages], eax
000003F8 0500100000                 add     eax, 4096           ; 16 MB = 4096 pages
000003FD EB06                       jmp     short mi_1
                                mi_0:
                                    ;mov    ax, cx
                                    ; 24/07/2022
000003FF 89C8                       mov     eax, ecx
00000401 66050001                   add     ax, 256             ; add 256 pages for the first 1 MB
                                    ;add    eax, 256
                                mi_1:
00000405 A3[B4760100]               mov     [memory_size], eax ; Total available memory in pages
                                                                ; 1 alloc. tbl. bit = 1 memory page
                                                                ; 32 allocation bits = 32 mem. pages
                                    ;
0000040A 05FF7F0000                 add     eax, 32767          ; 32768 memory pages per 1 M.A.T. page
0000040F C1E80F                     shr     eax, 15             ; ((32768 * x) + y) pages (y < 32768)
                                                                ;  --> x + 1 M.A.T. pages, if y > 0
                                                                ;  --> x M.A.T. pages, if y = 0
00000412 66A3[C8760100]             mov     [mat_size], ax      ; Memory Alloc. Table Size in pages
00000418 C1E00C                     shl     eax, 12             ; 1 M.A.T. page = 4096 bytes
                                    ;                           ; Max. 32 M.A.T. pages (4 GB memory)
0000041B 89C3                       mov     ebx, eax            ; M.A.T. size in bytes
                                    ; Set/Calculate Kernel's Page Directory Address
0000041D 81C300001000               add     ebx, MEM_ALLOC_TBL
00000423 891D[B0760100]             mov     [k_page_dir], ebx  ; Kernel's Page Directory address
                                                                ; just after the last M.A.T. page
                                    ;
00000429 83E804                     sub     eax, 4              ; convert M.A.T. size to offset value
0000042C A3[C0760100]               mov     [last_page], eax   ; last page offset in the M.A.T.
                                    ;                           ; (allocation status search must be
                                                                ; stopped after here)
00000431 31C0                       xor     eax, eax
00000433 48                         dec     eax                 ; FFFFFFFFh (set all bits to 1)
                                    ;push   cx
                                    ; 18/04/2021
00000434 51                         push    ecx
                                    ; ecx = 3840 ; 27/11/2023
                                    ; (Note: ecx < 3840 if the total memory is less than 16 MB)
00000435 C1E905                     shr     ecx, 5              ; convert 1 - 16 MB page count to
                                                                ; count of 32 allocation bits
                                    ; ecx = 120 ; 27/11/2023
00000438 F3AB                       rep     stosd
                                    ;pop    cx
                                    ; 18/04/2021
0000043A 59                         pop     ecx
0000043B 40                         inc     eax                 ; 0
0000043C 80E11F                     and     cl, 31              ; remain bits
0000043F 7412                       jz      short mi_4
00000441 8907                       mov     [edi], eax          ; reset
                                mi_2:
00000443 0FAB07                     bts     [edi], eax          ; 06/11/2014
```

```
00000446 FEC9                    dec     cl
00000448 7404                    jz      short mi_3
0000044A FEC0                    inc     al
0000044C EBF5                    jmp     short mi_2
                             mi_3:
0000044E 28C0                    sub     al, al            ; 0
00000450 83C704                  add     edi, 4            ; 15/11/2014
                             mi_4:
00000453 6609D2                  or      dx, dx           ; check 16 MB to 4 GB memory space
00000456 7421                    jz      short mi_6       ; max. 16 MB memory, no more...
                                 ;
00000458 B900021000              mov     ecx, MEM_ALLOC_TBL + 512 ; End of first 16 MB memory
                                 ;
0000045D 29F9                    sub     ecx, edi         ; displacement (to end of 16 MB)
0000045F 7406                    jz      short mi_5       ; jump if EDI points to
                                                          ;         end of first 16 MB
00000461 D1E9                    shr     ecx, 1           ; convert to dword count
00000463 D1E9                    shr     ecx, 1           ; (shift 2 bits right)
00000465 F3AB                    rep     stosd            ; reset all bits for reserved pages
                                                          ; (memory hole under 16 MB)
                             mi_5:
00000467 6689D1                  mov     cx, dx           ; count of 64 KB memory blocks
0000046A D1E9                    shr     ecx, 1           ; 1 alloc. dword per 128 KB memory
0000046C 9C                      pushf                    ; 16/11/2014
0000046D 48                      dec     eax              ; FFFFFFFFh (set all bits to 1)
0000046E F3AB                    rep     stosd
00000470 40                      inc     eax              ; 0
00000471 9D                      popf                     ; 16/11/2014
00000472 7305                    jnc     short mi_6
00000474 6648                    dec     ax               ; eax = 0000FFFFh
00000476 AB                      stosd
00000477 6640                    inc     ax               ; 0
                             mi_6:
00000479 39DF                    cmp     edi, ebx         ; check if EDI points to
0000047B 730A                    jnb     short mi_7       ; end of memory allocation table
                                 ;                        ; (>= MEM_ALLOC_TBL + 4906)
0000047D 89D9                    mov     ecx, ebx         ; end of memory allocation table
0000047F 29F9                    sub     ecx, edi         ; convert displacement/offset
00000481 D1E9                    shr     ecx, 1           ; to dword count
00000483 D1E9                    shr     ecx, 1           ; (shift 2 bits right)
00000485 F3AB                    rep     stosd            ; reset all remain M.A.T. bits
                             mi_7:
                                 ; Reset M.A.T. bits in M.A.T. (allocate M.A.T. pages)
00000487 BA00001000              mov     edx, MEM_ALLOC_TBL
                                 ;sub    ebx, edx         ; Mem. Alloc. Tbl. size in bytes
                                 ;shr    ebx, 12          ; Mem. Alloc. Tbl. size in pages
0000048C 668B0D[C8760100]        mov     cx, [mat_size]   ; Mem. Alloc. Tbl. size in pages
00000493 89D7                    mov     edi, edx
00000495 C1EF0F                  shr     edi, 15          ; convert M.A.T. address to
                                                          ; byte offset in M.A.T.
                                                          ; (1 M.A.T. byte points to
                                                          ;           32768 bytes)
                                                          ; Note: MEM_ALLOC_TBL address
                                                          ; must be aligned on 128 KB
                                                          ; boundary!
00000498 01D7                    add     edi, edx         ; points to M.A.T.'s itself
                                 ; eax = 0
0000049A 290D[B8760100]          sub     [free_pages], ecx ; 07/11/2014
                             mi_8:
000004A0 0FB307                  btr     [edi], eax       ; clear bit 0 to bit x (1 to 31)
                                 ;dec    bl
000004A3 FEC9                    dec     cl
000004A5 7404                    jz      short mi_9
000004A7 FEC0                    inc     al
000004A9 EBF5                    jmp     short mi_8
                             mi_9:
                                 ;
                                 ; Reset Kernel's Page Dir. and Page Table bits in M.A.T.
                                 ;              (allocate pages for system page tables)

                                 ; edx = MEM_ALLOC_TBL
000004AB 8B0D[B4760100]          mov     ecx, [memory_size] ; memory size in pages (PTEs)
000004B1 81C1FF030000            add     ecx, 1023        ; round up (1024 PTEs per table)
000004B7 C1E90A                  shr     ecx, 10          ; convert memory page count to
                                                          ; page table count (PDE count)
                                 ;
000004BA 51                      push    ecx              ; (**) PDE count (<= 1024)
                                 ;
000004BB 41                      inc     ecx              ; +1 for kernel page directory
                                 ;
000004BC 290D[B8760100]          sub     [free_pages], ecx ; 07/11/2014
                                 ;
000004C2 8B35[B0760100]          mov     esi, [k_page_dir] ; Kernel's Page Directory address
000004C8 C1EE0C                  shr     esi, 12          ; convert to page number
                             mi_10:
000004CB 89F0                    mov     eax, esi         ; allocation bit offset
000004CD 89C3                    mov     ebx, eax
000004CF C1EB03                  shr     ebx, 3           ; convert to alloc. byte offset
000004D2 80E3FC                  and     bl, 0FCh         ; clear bit 0 and bit 1
                                                          ;   to align on dword boundary
000004D5 83E01F                  and     eax, 31          ; set allocation bit position
                                                          ;  (bit 0 to bit 31)
                                 ;
000004D8 01D3                    add     ebx, edx         ; offset in M.A.T. + M.A.T. address
                                 ;
000004DA 0FB303                  btr     [ebx], eax       ; reset relevant bit (0 to 31)
                                 ;
000004DD 46                      inc     esi              ; next page table
000004DE E2EB                    loop    mi_10            ; allocate next kernel page table
                                                          ; (ecx = page table count + 1)
                                 ;
000004E0 59                      pop     ecx              ; (**) PDE count (= pg. tbl. count)
                                 ;
                                 ; Initialize Kernel Page Directory and Kernel Page Tables
                                 ;
                                 ; Initialize Kernel's Page Directory
000004E1 8B3D[B0760100]          mov     edi, [k_page_dir]
000004E7 89F8                    mov     eax, edi
000004E9 0C03                    or      al, PDE_A_PRESENT + PDE_A_WRITE
                                                          ; supervisor + read&write + present
000004EB 89CA                    mov     edx, ecx         ; (**) PDE count (= pg. tbl. count)
                             mi_11:
000004ED 0500100000               add     eax, 4096        ; Add page size (PGSZ)
                                                           ; EAX points to next page table
000004F2 AB                      stosd
000004F3 E2F8                    loop    mi_11
000004F5 29C0                    sub     eax, eax         ; Empty PDE
                                 ;mov    cx, 1024         ; Entry count (PGSZ/4)
                                 ; 29/11/2023
000004F7 B504                    mov     ch, 4 ; cx = 4*256 = 1024
000004F9 29D1                    sub     ecx, edx
000004FB 7402                    jz      short mi_12
000004FD F3AB                    rep     stosd            ; clear remain (empty) PDEs
                                 ;
```

```
                                    ; Initialization of Kernel's Page Directory is OK, here.
                                mi_12:
                                    ; Initialize Kernel's Page Tables
                                    ;
                                    ; (EDI points to address of page table 0)
                                    ; eax = 0
000004FF 8B0D[B4760100]             mov     ecx, [memory_size] ; memory size in pages
00000505 89CA                       mov     edx, ecx       ; (***)
00000507 B003                       mov     al, PTE_A_PRESENT + PTE_A_WRITE
                                                           ; supervisor + read&write + present
                                mi_13:
00000509 AB                         stosd
0000050A 0500100000                 add     eax, 4096
0000050F E2F8                       loop    mi_13
                                    ;and    dx, 1023       ; (***)
                                    ; 30/08/2023
00000511 81E2FF030000               and     edx, 1023
00000517 7408                       jz      short mi_14
                                    ;mov    cx, 1024
                                    ; 30/08/2023
00000519 B504                       mov     ch, 4 ; 4*256 = 1024
                                    ;sub    cx, dx         ; from dx (<= 1023) to 1024
                                    ; 24/07/2022
0000051B 29D1                       sub     ecx, edx
0000051D 31C0                       xor     eax, eax
0000051F F3AB                       rep     stosd          ; clear remain (empty) PTEs
                                                           ; of the last page table
                                mi_14:
                                    ;  Initialization of Kernel's Page Tables is OK, here.
                                    ;
00000521 89F8                       mov     eax, edi       ; end of the last page table page
                                                            ; (beginging of user space pages)

00000523 C1E80F                     shr     eax, 15        ; convert to M.A.T. byte offset
00000526 24FC                       and     al, 0FCh       ; clear bit 0 and bit 1 for
                                                           ; aligning on dword boundary
00000528 A3[C4760100]               mov     [first_page], eax
0000052D A3[BC760100]               mov     [next_page], eax ; The first free page pointer
                                                            ; for user programs
                                                            ; (Offset in Mem. Alloc. Tbl.)
                                    ;
                                    ; Linear/FLAT (1 to 1) memory paging for the kernel is OK, here.
                                    ;

                                    ; Enable paging
                                    ;
00000532 A1[B0760100]                   mov     eax, [k_page_dir]
00000537 0F22D8                     mov     cr3, eax
0000053A 0F20C0                     mov     eax, cr0
0000053D 0D00000080                 or      eax, 80000000h ; set paging bit (bit 31)
00000542 0F22C0                     mov     cr0, eax
                                        ;jmp    KCODE:StartPMP

00000545 EA                         db 0EAh                ; Opcode for far jump
00000546 [4C050000]                     dd StartPMP                ; 32 bit offset
0000054A 0800                       dw KCODE               ; kernel code segment descriptor

                                StartPMP:
                                    ; 06/11//2014
                                    ; Clear video page 0
                                    ;
                                    ; Temporary Code
                                    ;
0000054C B9E8030000                 mov     ecx, 80*25/2
00000551 BF00800B00                 mov     edi, 0B8000h
                                    ; 30/01/2016
                                    ;xor    eax, eax       ; black background, black fore color
00000556 B800070007                 mov     eax, 07000700h  ; black background, light gray fore color
0000055B F3AB                       rep     stosd

                                    ; 19/08/2014
                                    ; Kernel Base Address = 0
                                    ; It is mapped to (physically) 0 in the page table.
                                    ; So, here is exactly 'StartPMP' address.

                                    ; 29/01/2016 (TRDOS 386 = TRDOS v2.0)
0000055D BE[5A390100]               mov     esi, starting_msg
                                    ;; 14/08/2015 (kernel version message will appear
                                    ;;             when protected mode and paging is enabled)
00000562 BF00800B00                 mov     edi, 0B8000h ; 27/08/2014

                                    ; 30/11/2020
                                    ; 14/11/2020 (TRDOS 386 v2.0.3)
                                    ;cmp    byte [vbe3], 3 ; 03h
                                    ;jne    short pkv_1
                                    ;;mov   ah, 0Bh ; Black background, light cyan forecolor
                                    ;; Light red TRDOS 386 version text shows VBE3 is ready !
                                    ;mov    ah, 0Ch ; Black background, light red forecolor
                                    ;jmp    short pkv_2
                                ;pkv_1:
00000567 B40A                       mov     ah, 0Ah ; Black background, light green forecolor
                                ;pkv_2:
                                    ; 20/08/2014
00000569 E8BF030000                 call    printk

                                    ; 'UNIX v7/x86' source code by Robert Nordier (1999)
                                    ; // Set IRQ offsets
                                    ;
                                    ;  Linux (v0.12) source code by Linus Torvalds (1991)
                                    ;
                                                                ;; ICW1
0000056E B011                       mov     al, 11h            ; Initialization sequence
00000570 E620                       out     20h, al            ;       8259A-1
                                    ; jmp   $+2
00000572 E6A0                       out     0A0h, al           ;       8259A-2
                                                                ;; ICW2
00000574 B020                       mov     al, 20h            ; Start of hardware ints (20h)
00000576 E621                       out     21h, al            ;       for 8259A-1
                                    ; jmp   $+2
00000578 B028                       mov     al, 28h            ; Start of hardware ints (28h)
0000057A E6A1                       out     0A1h, al           ;       for 8259A-2
                                                               ;
0000057C B004                       mov     al, 04h            ;; ICW3
0000057E E621                       out     21h, al            ;       IRQ2 of 8259A-1 (master)
                                    ; jmp   $+2
00000580 B002                       mov     al, 02h            ;       is 8259A-2 (slave)
00000582 E6A1                       out     0A1h, al           ;
                                                               ;; ICW4
00000584 B001                       mov     al, 01h            ;
00000586 E621                       out     21h, al            ;       8086 mode, normal EOI
                                    ; jmp   $+2
00000588 E6A1                       out     0A1h, al           ;       for both chips.

                                    ;mov    al, 0FFh       ; mask off all interrupts for now
```

```
                                    ;out   21h, al
                                    ;; jmp $+2
                                    ;out   0A1h, al

                                    ; 02/04/2015
                                    ; 26/03/2015 System call (INT 30h) modification
                                    ;  DPL = 3 (Interrupt service routine can be called from user mode)
                                    ;
                                    ;; Linux (v0.12) source code by Linus Torvalds (1991)
                                    ;  setup_idt:
                                    ;
                                        ;; 16/02/2015
                                    ;;mov     dword [DISKETTE_INT], fdc_int ; IRQ 6 handler
                                    ; 21/08/2014 (timer_int)
0000058A BE[F8350100]               mov     esi, ilist
0000058F 8D3D[C8730100]             lea     edi, [idt]
                                    ; 26/03/2015
00000595 B930000000                 mov     ecx, 48        ; 48 hardware interrupts (INT 0 to INT 2Fh)
                                    ; 02/04/2015
0000059A BB00000800                 mov     ebx, 80000h
                                rp_sidt1:
0000059F AD                         lodsd
000005A0 89C2                       mov     edx, eax
000005A2 66BA008E                   mov     dx, 8E00h
000005A6 6689C3                     mov     bx, ax
000005A9 89D8                       mov     eax, ebx       ; /* selector = 0x0008 = cs */
                                                           ; /* interrupt gate - dpl=0, present */
000005AB AB                         stosd   ; selector & offset bits 0-15
000005AC 89D0                       mov     eax, edx
000005AE AB                         stosd   ; attributes & offset bits 16-23
000005AF E2EE                       loop    rp_sidt1
                                    ; 15/04/2016
                                    ; TRDOS 386 (TRDOS v2.0) /// 32 sofware interrupts ///
                                    ;mov    cl, 16        ; 16 software interrupts (INT 30h to INT 3Fh)
000005B1 B120                       mov     cl, 32        ; 32 software interrupts (INT 30h to INT 4Fh)
                                rp_sidt2:
000005B3 AD                         lodsd
000005B4 21C0                       and     eax, eax
000005B6 7413                       jz      short rp_sidt3
000005B8 89C2                       mov     edx, eax
000005BA 66BA00EE                   mov     dx, 0EE00h     ; P=1b/DPL=11b/01110b
000005BE 6689C3                     mov     bx, ax
000005C1 89D8                       mov     eax, ebx       ; selector & offset bits 0-15
000005C3 AB                         stosd
000005C4 89D0                       mov     eax, edx
000005C6 AB                         stosd
000005C7 E2EA                       loop    rp_sidt2
000005C9 EB16                       jmp     short sidt_OK
                                rp_sidt3:
000005CB B8[A40D0000]               mov     eax, ignore_int
000005D0 89C2                       mov     edx, eax
000005D2 66BA00EE                   mov     dx, 0EE00h     ; P=1b/DPL=11b/01110b
000005D6 6689C3                     mov     bx, ax
000005D9 89D8                       mov     eax, ebx       ; selector & offset bits 0-15
                                rp_sidt4:
000005DB AB                         stosd
000005DC 92                         xchg    eax, edx
000005DD AB                         stosd
000005DE 92                         xchg    edx, eax
000005DF E2FA                       loop    rp_sidt4
                                sidt_OK:
000005E1 0F011D[FE650000]           lidt    [idtd]
                                    ;
                                    ; TSS descriptor setup ; 24/03/2015
000005E8 B8[48760100]               mov     eax, task_state_segment
000005ED 66A3[AA650000]             mov     [gdt_tss0], ax
000005F3 C1C010                     rol     eax, 16
000005F6 A2[AC650000]               mov     [gdt_tss1], al
000005FB 8825[AF650000]             mov     [gdt_tss2], ah
00000601 66C705[AE760100]68-        mov     word [tss.IOPB], tss_end - task_state_segment
00000609 00
                                            ;
                                            ; IO Map Base address (when this address points
                                            ; to end of the TSS, CPU does not use IO port
                                            ; permission bit map for RING 3 IO permissions,
                                            ; access to any IO ports in ring 3 will be forbidden.)
                                            ;
                                    ;mov    [tss.esp0], esp ; TSS offset 4
                                    ;mov    word [tss.ss0], KDATA ; TSS offset 8 (SS)
0000060A 66B82800                   mov     ax, TSS  ; It is needed when an interrupt
                                                     ; occurs (or a system call -software INT- is requested)
                                                     ; while cpu running in ring 3 (in user mode).
                                                     ; (Kernel stack pointer and segment will be loaded
                                                     ; from offset 4 and 8 of the TSS, by the CPU.)
0000060E 0F00D8                     ltr     ax  ; Load task register
                                    ;
                                esp0_set0:

                                ; 29/11/2023 - Erdogan Tan
                                ; ------------------------
                                ; If we read following -disabled- stack page setting code...
                                ; When the memory size >= 3GB, one of page tables conflicts with the stack page
                                ; at 4MB-4096 address. So, to leave stack pointer at 90000h is better/default.
                                ; (Problem may not appears for <= 2.5GB main memory but following code is also
                                ; defective because kernel stack page would be seen as unallocated in the M.A.T.
                                ; without by adding a memory allocation code.)
                                ;
                                ; 1st 4MB layout: 1MB kernel -base- reserved + max. 128 KB M.A.T. (at 100000h)
                                ;                + Kernel's page directory (4KB) -just after the M.A.T.-
                                ;            + Kernel's page tables (main memory size / 1024)
                                ; Note:
                                ; 1 or 2 additional kernel page table(s) may be needed for Linear Frame Buffer
                                ; .. but, it/they will not have to be contiguous with other kernel page tables.
                                ; ------------------------

                                ; 27/11/2023 - TRDOS 386 v2.0.7
                                %if 0
                                    ; 30/07/2015
                                    mov     ecx, [memory_size] ; memory size in pages
                                    shl     ecx, 12 ; convert page count to byte count
                                    cmp     ecx, CORE ; beginning of user's memory space (400000h)
                                                    ; (kernel mode virtual address)
                                    jna     short esp0_set1
                                    ;
                                    ; If available memory > CORE (end of the 1st 4 MB)
                                    ; set stack pointer to CORE
                                    ;(Because, PDE 0 is reserved for kernel space in user's page directory)
                                    ;(PDE 0 points to page table of the 1st 4 MB virtual address space)
                                    mov     ecx, CORE
                                esp0_set1:
                                    mov     esp, ecx ; top of kernel stack (**tss.esp0**)
                                %endif

                                esp0_set_ok:
```

```
                                    ; 30/07/2015 (**tss.esp0**)
00000611 8925[4C760100]             mov     [tss.esp0], esp       ; 90000h ; 29/11/2023
                                                          ; <-- 97000h ; 04/12/2023 (max. 3072 bytes)
00000617 66C705[50760100]10-            mov     word [tss.ss0], KDATA
0000061F 00
                                    ; 14/08/2015
                                    ; 10/11/2014 (Retro UNIX 386 v1 - Erdogan Tan)
                                    ;
                                    ;cli    ; Disable interrupts (for CPU)
                                    ;    (CPU will not handle hardware interrupts, except NMI!)
                                    ;
00000620 30C0                       xor     al, al         ; Enable all hardware interrupts!
00000622 E621                       out     21h, al        ; (IBM PC-AT compatibility)
00000624 EB00                       jmp     $+2            ; (All conventional PC-AT hardware
00000626 E6A1                       out     0A1h, al       ;  interrupts will be in use.)
                                                           ; (Even if related hardware component
                                                           ;  does not exist!)
                                    ; Enable NMI
00000628 B07F                       mov     al, 7Fh        ; Clear bit 7 to enable NMI (again)
0000062A E670                       out     70h, al
                                    ; 23/02/2015
0000062C 90                         nop
0000062D E471                       in      al, 71h        ; read in 71h just after writing out to 70h
                                                           ; for preventing unknown state (!?)
                                    ;
                                    ; Only a NMI can occur here... (Before a 'STI' instruction)
                                    ;
                                    ; 02/09/2014
                                    ;xor    bx, bx
                                    ; 24/07/2022
0000062F 31DB                       xor     ebx, ebx
00000631 66BA0002                   mov     dx, 0200h      ; Row 2, column 0  ; 07/03/2015
00000635 E83D1D0000                 call    _set_cpos      ; 24/01/2016

                                    ; 14/11/2020 (TRDOS 386 v2.0.3)
                                    ; Check VBE3 protected mode interface/feature(s)

                                        ;cmp    byte [vbe3], 3 ; 03h
                                        ;jne    short display_mem_info

                                        ; 20/11/2020
0000063A 803D[7C090000]02               cmp     byte [vbe3], 2 ; 02h
00000641 7707                           ja      short vbe3_pmid_chk
                                        ;;jb    short display_mem_info
                                        ;jb     display_mem_info ; 02/12/2020
00000643 7220                           jb      short jmp_display_mem_info ; 24/07/2022
00000645 E9EF010000                     jmp     check_boch_plex86_vbe

                                    vbe3_pmid_chk:
0000064A B9EA7F0000                     mov     ecx, 32768 - (20+2) ; 32766 - PMInfoBlockSize
0000064F BE02000C00                     mov     esi, 0C0002h ; 1st word of the video bios rom is 0AA55h

                                    chk_pmi_sign:
                                        ;mov    eax, [esi]
                                        ;cmp    eax, 'PMID'
                                        ; 30/11/2020
                                        ;cmp    al, 'P'
                                        ;jne    short chk_pmi_sign_next
00000654 813E504D4944                   cmp     dword [esi], 'PMID'
                                        ;je     short display_vbios_product_name
0000065A 740E                           je      short verify_pmib_chksum ; 15/11/2020
                                    ;chk_pmi_sign_next:
0000065C 46                             inc     esi  ; inc si
0000065D E2F5                           loop    chk_pmi_sign

                                    not_valid_pmib:
0000065F FE0D[7C090000]                 dec     byte [vbe3] ; 2 = VBE2 compatible
                                                            ; (vbe3 feature is defective in this vbios)
                                        ;jmp    short display_mem_info
                                    jmp_display_mem_info:     ; 24/07/2022
                                        ; 02/12/2020
00000665 E988010000                     jmp     display_mem_info

                                    verify_pmib_chksum:
                                        ; 18/10/2023
                                        ; (ATI RV370 video bios contains ZERO at Checksum offset.)
0000066A 31C0                           xor     eax, eax
0000066C 384613                         cmp     byte [esi+19], al ; 0 ; Checksum byte
0000066F 7611                           jna     short skip_verify_pmib_chksum ; may be ATI video bios
                                        ;cmp    dword [esi+16], 0C000h
                                        ;       ; CodeSegSel = 0C000h, InProtectMode = 0, Checksum = 0
                                        ;je     short skip_verify_pmib_chksum ; may be ATI video bios
                                        ; 18/10/2023
                                        ; 15/11/2020
                                        ;xor    eax, eax
                                        ;;mov   ecx, eax
                                        ;mov    cl, 20
00000671 66B91400                       mov     cx, 20 ; 30/11/2020
00000675 56                             push    esi
                                    pmib_sum_bytes:
00000676 AC                             lodsb
00000677 00C4                           add     ah, al
00000679 E2FB                           loop    pmib_sum_bytes
0000067B 5E                             pop     esi
0000067C 08E4                           or      ah, ah
0000067E 75DF                           jnz     short not_valid_pmib ; AH must be 0

                                        ; 18/10/2023
00000680 30C0                           xor     al, al  ; eax = 0

                                        ; 28/02/2021
                                        ; Set default (initial) truecolor bpp value to 32
                                        ; (for VBE3 video bios.. because vbe3 video bioses
                                        ;  use 32bpp -for truecolor modes- instead of 24bpp)
                                        ; (This setting may be changed via 'sysvideo' bx=0908h)

                                    skip_verify_pmib_chksum: ; 18/10/2023

00000682 C605[73730100]20               mov     byte [truecolor], 32 ; (RGB: 00RRGGBBh)

                                    display_vbios_product_name: ; 14/11/2020

                                        ; ESI points to 'PMID' (0C0000h + 'PMID' offset)

                                        ; 15/11/2020
                                        ;mov    [pmid_addr], si       ; PMInfoBlock offset
                                        ;               ; (in VGA bios, 0C0000h + offset)
                                        ; 02/12/2020
                                        ;push   esi ; * pmid_addr
00000689 89F7                           mov     edi, esi

                                        ;mov    esi, [VBE3INFOBLOCK+22] ; 097E00h + 16h
                                        ;               ; OemVendorNamePtr (seg16:off16)
0000068B 8B35067E0900                   mov     esi, [VBE3INFOBLOCK+6] ; 097E00h + 06h
```

```
                                                ; OemStringPtr (seg16:off16)
                                    ; 18/10/2023
                                    ;xor   al, al  ; eax = 0
00000691 6696                       xchg   ax, si  ; ax = offset, si = 0
00000693 C1EE0C                     shr    esi, 12 ; (to convert segment to base addr)
00000696 6601C6                     add    si, ax  ; esi has an address < 1 MB limit
                                                ; (OemVendorName is in VBE3INFOBLOCK)
                                                ; Example:
                                                ; TRDOS 386 v2.0.3 VESA VBE3 protected mode
                                                ; interface development reference is ...
                                                ; NVIDIA GeForce FX5500 VGA BIOS -C000h:029Ch-
                                                ; Version 4.34.20.54.00 -C000h:02EDh-
                                                ; ((OemString is 'NVIDIA'))
                                                ; ((OemVendorName is 'NVIDIA Corporation'))
                                                ; ((OemProductName is 'NV34 Board - p162-1nz))

                                    ;mov   ah, 0Eh ; Black background, yellow forecolor
                                    ; 30/11/2020
00000699 B40C                       mov    ah, 0Ch ; Black background, light red forecolor

0000069B E8E53A0000                 call   print_kmsg

                                    ;mov   ah, 07h

000006A0 BE[55730100]               mov    esi, vesa_vbe3_bios_msg
                                    ;call  print_kmsg
000006A5 E8E13A0000                 call   pkmsg_loop ; 30/11/2020

                                    ; 02/12/2020
                                    ;pop   edi ; * pmid_addr

                                ; 04/12/2023 - TRDOS 386 v2.0.7
                                %if 0
                                    ; 24/07/2022
                                    ; 29/11/2020
                                vbe3pminit:
                                    ; 30/11/2020
                                    ;cmp   byte [vbe3], 3 ; is VESA VBE3 PMI ready ?
                                    ;jne   short di4

                                    ; Allocate 64KB contiguous (kernel) memory block
                                    xor    eax, eax
                                    mov    ecx, 65536
                                    call   allocate_memory_block
                                    ;jc    short di4
                                    ;jc    di0 ; 30/11/2020
                                    ; 24/07/2022
                                    jnc    short vbe3pminit0
                                    jmp    di0

                                vbe3pminit0:
                                    ; of course this block must be in the 1st 16MB
                                    ; because vbe3 pmi segments will be 16 bit segments
                                    ; (80286 type segment descriptors in GDT)

                                    mov    [vbe3bios_addr], eax
                                %endif

                                    ; 04/12/2023 - TRDOS 386 v2.0.7      ; (+!*!+)
                                    ; fixed PM-VBIOS address (no need to add a memory block)
                                    ; in the reserved -and already allocated- area under 1MB
                                    ; (purpose: to prevent page faults if memory size > 2.5GB)
                                    ; ((User's page dir contains only the 1st 4MB of system mem
                                    ; as PDE. So, this causes to page faults during an interrupt
                                    ; in user mode, because if memory size > 2.5GB, kernel
                                    ; page tables overs/passes 4MB limit and PM-VBIOS
                                    ; is located after kernel page tables.))
                                vbe3pminit:
000006AA B800000600                 mov    eax, VBE3BIOSCODE_ADDR ; 60000h      ; (+!*!+)

                                    ; set [pmid_addr] to the new location
000006AF BE00000C00                 mov    esi, 0C0000h

                                    ; 30/11/2020
000006B4 29F7                       sub    edi, esi ; izolate offset
000006B6 01C7                       add    edi, eax ; new address
000006B8 893D[2C9D0100]             mov    [pmid_addr], edi ; new 'PMID' location

                                    ; Move VIDEO BIOS from 0C0000h to EAX
000006BE B900400000                 mov    ecx, 65536/4
000006C3 89C7                       mov    edi, eax ; 30/11/2020
000006C5 F3A5                       rep    movsd

                                    ; 02/12/2020
                                    ; 30/11/2020
                                    ; set vbe3 segment selectors

                                ; 04/12/2023 - TRDOS 386 v2.0.7   ; (+!*!+)
                                %if 0
                                    ; VBE3CS (VESA VBE3 video bios code segment)
                                    mov    edi, _vbe3_CS+2 ; base address bits 0..15
                                    stosw  ; edi = _vbe3_CS+4
                                    ror    eax, 16
                                    mov    [edi], al ; base address, bits 16..23

                                    ; VBE3DS ('CodeSegSel' in PMInfoBlock)
                                    mov    edi, _vbe3_DS+4 ; base addr bits 16..23
                                    mov    [edi], al
                                    rol    eax, 16
                                    mov    [edi-2], ax ; base address, bits 0..15
                                %endif
                                    ; VBE3BDS (BIOSDataSel in PMInfoBlock)
000006C7 BF[BA650000]               mov    edi, _vbe3_BDS+2 ; base addr bits 0..15
000006CC B800700900                 mov    eax, VBE3BIOSDATABLOCK ; 1536 bytes
000006D1 66AB                       stosw  ; edi = _vbe3_BDS+4
000006D3 C1E810                     shr    eax, 16
000006D6 8807                       mov    [edi], al ; base address, bits 16..23

                                    ; VBE3SS (1024 bytes)
000006D8 BF[E2650000]               mov    edi, _vbe3_SS+2 ; base addr bits 0..15
000006DD B800600900                 mov    eax, VBE3STACKADDR ; size = 1024 bytes
000006E2 66AB                       stosw  ; edi = _vbe3_SS+4
000006E4 C1E810                     shr    eax, 16
000006E7 8807                       mov    [edi], al ; base address, bits 16..23

                                    ; stack pointer (esp) will be set to 1020
                                    ; (before VBE3 PMI call)

                                    ; VBE3ES (max: 2048 bytes)
000006E9 BF[EA650000]               mov    edi, _vbe3_ES+2 ; base addr bits 0..15
000006EE B800760900                 mov    eax, VBE3SAVERESTOREBLOCK
000006F3 66AB                       stosw  ; edi = _vbe3_ES+4
000006F5 C1E810                     shr    eax, 16
000006F8 8807                       mov    [edi], al ; base address, bits 16..23
```

```
                                    ;Note: low word of _VBE3_ES base address will be
                                    ;      set -again- by VBE3 PMI caller routine

                                    ; 09/12/2020
                                    ;; set pmi32 (as VBE3 PMI is ready)
                                    ;inc   byte [pmi32] ; = 1

                                    ; KCODE16 (set PMI far return segment)
000006FA BF[F2650000]               mov    edi, _16bit_CS+2 ; base addr bits 0..15
000006FF B8[88070000]               mov    eax, pminit_return_addr16
00000704 66AB                       stosw  ; edi = _16bit_CS+4
00000706 C1E810                     shr    eax, 16
00000709 8807                       mov    [edi], al ; base address, bits 16..23

                                    ; 30/11/2020
                                    ; clear mem from VBE3 BIOS data area emu block
                                    ; to end of vbe3 buffers

                                    ; 18/10/2023 - TRDOS v2.0.7
                                    ; (VBE3BIOSDATABLOCK contains a copy of the 1st
                                    ; 1536 bytes of the memory, IVT, ROMBIOS DATA etc.)
                                    ; ((it must not be cleared here))
                                %if 0
                                    ; 01/12/2020
                                    mov    edi, VBE3BIOSDATABLOCK ; 97000h
                                    ;mov   cx, (VBE3INFOBLOCK-VBE3BIOSDATABLOCK)/4
                                    ;      ; ecx = 3584/4 double words
                                    ; 21/12/2020
                                    mov    cx, (VBE3MODEINFOBLOCK-VBE3BIOSDATABLOCK)/4
                                           ; ecx = 3072/4 double words
                                    ;xor   eax, eax
                                    xor    al, al
                                    rep    stosd
                                %endif
                                    ; 18/10/2023 - TRDOS v2.0.7
                                    ; (VBE3BIOSDATABLOCK contains a copy of the 1st
                                    ; 1536 bytes of the memory, IVT, ROMBIOS DATA etc.)
                                    ; ((it must not be cleared here))
0000070B BF00760900                 mov    edi, VBE3SAVERESTOREBLOCK ; 97600h
00000710 66B98001                   mov    cx, (VBE3MODEINFOBLOCK-VBE3SAVERESTOREBLOCK)/4
                                            ; ecx = 1536/4 = 384 double words
                                    ;xor   eax, eax
00000714 30C0                       xor    al, al
00000716 F3AB                       rep    stosd

                                    ; Filling PMInfoBlock selector fields
00000718 8B3D[2C9D0100]             mov    edi, [pmid_addr]
0000071E 66C747083800               mov    word [edi+PMInfo.BIOSDataSel], VBE3BDS
00000724 66C7470A4000               mov    word [edi+PMInfo.A0000Sel], VBE3A000
0000072A 66C7470C4800               mov    word [edi+PMInfo.B0000Sel], VBE3B000
00000730 66C7470E5000               mov    word [edi+PMInfo.B8000Sel], VBE3B800
00000736 66C747105800               mov    word [edi+PMInfo.CodeSegSel], VBE3DS
0000073C C6471201                   mov    byte [edi+PMInfo.InProtectMode], 1

                                    ; Calculate and write checksum byte
00000740 89FE                       mov    esi, edi
00000742 B113                       mov    cl, PMInfo.size - 1
                                    ;xor   ah, ah
                                pmid_chksum:
00000744 AC                         lodsb
00000745 00C4                       add    ah, al
00000747 E2FB                       loop   pmid_chksum
00000749 F6DC                       neg    ah ; 1 -> 255, 255 -> 1
0000074B 8826                       mov    [esi], ah  ; checksum

                                    ; far call PM initialization
                                    ; (VBE3 video bios will return via 'retf')

0000074D 668B4706                   mov    ax, [edi+PMInfo.PMInitialize]
                                    ; 30/11/2020
00000751 C1E010                     shl    eax, 16 ; save entry address in hw
                                    ; ax = 0

                                    ; 02/12/2020
00000754 68[AC070000]               push   pminit_ok ; normal, near return address

                                    ; 30/11/2020
                                _VBE3PMI_fcall:
                                    ; ax = function, hw of eax = entry address
00000759 9C                         pushf  ; save 32 bit flags
0000075A 56                         push   esi ; *
0000075B 55                         push   ebp ; **

0000075C 89E5                       mov    ebp, esp ; save 32 bit stack pointer

                                    ;mov   esi, eax
                                    ; 28/12/2025 (PMI32 BugFix)
0000075E 0FB7F0                     movzx  esi, ax

00000761 FA                         cli

                                    ; Disable interrupts (clear interrupt flag)
                                    ; Reset Interrupt MASK Registers (Master&Slave)
00000762 B0FF                       mov    al, 0FFh       ; mask off all interrupts
00000764 E621                       out    21h, al        ; on master PIC (8259)
00000766 EB00                       jmp    $+2  ; (delay)
00000768 E6A1                       out    0A1h, al       ; on slave PIC (8259)

                                    ; 02/12/2020
0000076A 66B86000                   mov    ax, VBE3SS
0000076E 8ED0                       mov    ss, ax

00000770 BCFC030000                 mov    esp, 1020 ; 30/11/2020

                                    ; 01/12/2020
                                    ;lss   esp, [stack16]

00000775 C1E810                     shr    eax, 16 ; now, entry address is in lw

                                    ; 30/11/2020 - 16 bit pm selector test (OK)
                                    ; (32 bit stack push/pop & retf with 32 bit code segment)
                                    ; (16 bit stack push/pop with 16 bit code segment)

                                    ; return
                                    ;push  KCODE16
                                    ;push  0 ; 30/11/2020 (pminit_return_addr16)

                                    ; 30/11/2020 (16 bit stack during retf from video bios)
00000778 C7042400007000             mov    dword [esp], KCODE16 << 16
                                                           ; ip = 0, cs = KCODE16
                                    ; 01/12/2020
                                    ;mov   dword [VBE3STACKADDR+1020], KCODE16*65536

```

```
                                    ;mov   [jumpfar16], eax

                                    ; 02/12/2020
                                    ; 30/11/2020 (32 bit stack during retf from kernel)
                                    ; far jump/call via retf
0000077F 6A30                       push   VBE3CS ; VBE3 video bios's code segment
00000781 50                         push   eax ; PMInitialize or EntryPoint

                                    ;mov   ax, si ; restore function
                                    ; 24/07/2022
00000782 89F0                       mov    eax, esi

                                    ; 02/12/2020
00000784 31F6                       xor    esi, esi ; (not necessary, it is not used)

00000786 CB                         retf   ; far return (to 16 bit code segment)

                                    ; 01/12/2020
                                    ;db    0EAh  ; far jump to 16 bit code segment
                                ;jumpfar16:
                                    ;dd    0
                                    ;dw    VBE3CS

                                ;stack16:
                                    ;dd    1020
                                    ;dw    VBE3SS

00000787 90                         align 2

                                pminit_return_addr16:
                                    ; 02/12/2020
                                    ; 30/11/2020
                                    ;;db   66h                ; Prefix for 32-bit
                                    ;db    0EAh               ; Opcode for far jump
                                    ;dd    pminit_return_addr32 ; 32 bit Offset
                                    ;dw    KCODE              ; 32 bit code segment
                                    ; 01/12/2020
00000788 EA[8F070000]0800           jmp    KCODE:pminit_return_addr32

                                pminit_return_addr32:
                                    ; restore 32 bit kernel selectors and 32 bit stack addr
0000078F BE10000000                 mov    esi, KDATA
00000794 8EDE                       mov    ds, si
00000796 8EC6                       mov    es, si
00000798 8ED6                       mov    ss, si
0000079A 89EC                       mov    esp, ebp  ; top of stack = iretd return addr

0000079C 5D                         pop    ebp ; **
0000079D 5E                         pop    esi ; *
0000079E 9D                         popf   ; restore 32 bit flags

                                    ; enable interrupts

0000079F FA                         cli

                                    ; 21/12/2020
000007A0 50                         push   eax

000007A1 30C0                       xor    al, al    ; Enable all hardware interrupts!
000007A3 E621                       out    21h, al   ; (IBM PC-AT compatibility)
000007A5 EB00                       jmp    $+2       ; (All conventional PC-AT hardware
000007A7 E6A1                       out    0A1h, al   ;  interrupts will be in use.)
                                                     ; (Even if related hardware component
                                                     ;  does not exist!)
000007A9 58                         pop    eax

000007AA FB                         sti

                                    ; top of stack = return address
                                    ; ('pminit_ok' for PMinit)

000007AB C3                         retn

                                pminit_ok:
                                    ; 03/12/2020
                                    ; (set [pmid_addr] to PMI entry point for next calls)
000007AC 8305[2C9D0100]04           add    dword [pmid_addr], PMInfo.EntryPoint ; + 4

                                    ; 17/01/2021
                                    ; copy EDID data from temporary location to final address
000007B3 803D[1D420000]4F           cmp    byte [edid], 4Fh
000007BA 7510                       jne    short vbe3h_chcl
                                    ;mov   ecx, 32 ; 128 bytes, 32 dwords
                                    ; 24/07/2022
000007BC 31C9                       xor    ecx, ecx
000007BE B120                       mov    cl, 32
000007C0 BE007D0900                 mov    esi, VBE3EDIDINFOBLOCK ; 97D00h
000007C5 BF[A49C0100]               mov    edi, edid_info
000007CA F3A5                       rep    movsd
                                    ; 17/01/2021
                                vbe3h_chcl:
                                    ; 16/01/2021
                                    ;; 06/12/2020
                                    ;; Save video mode 03h regs/dac/bios state
                                    ;
                                    ;mov   ax, 4F04h ; VESA VBE Function 04h
                                    ;               ; Save/Restore State
                                    ;sub   dl, dl  ; 0 = return buffer size
                                    ;mov   cx, 0Fh  ; ctrl/bios/dac/regs
                                    ;
                                    ;call  int10h_32bit_pmi
                                    ;; bx = number of 64-byte blocks to hold the state buff
                                    ;
                                    ;;mov  [vbe3stbufsize], bx
                                    ;; 16/01/2021
                                    ;or    word [vbe3stbsflags], 32768  ; set bit 15
                                    ;mov   ax, bx
                                    ;shl   ax, 6  ; * 64
                                    ;mov   [vbestatebufsize+30], ax
                                    ;
                                    ;; 06/12/2020
                                    ;; check 'vbe3stbufsize' (it must be <= 32)
                                    ;
                                    ;cmp   bx, 32
                                    ;ja    short display_mem_info ; light red forecolor
                                    ;
                                    ;; 16/01/2021
                                    ;or    byte [vbe3stbsflags], 1 ; set bit 0

                                    ; 30/11/2020
                                    ; Change VESA VBE3 BIOS text color in order to give
                                    ; "VBE3 PMI initialization has been successed" meaning

000007CC BE40810B00                 mov    esi, 0B8000h + 160*2 ; row 2
```

```
000007D1 89F7                       mov     edi, esi
                                vbe3h_chcl_next:
000007D3 66AD                       lodsw
000007D5 80FC0C                     cmp     ah, 0Ch ; light red forecolor
000007D8 7518                       jne     short display_mem_info
000007DA B40E                       mov     ah, 0Eh ; yellow forecolor
000007DC 66AB                       stosw
000007DE EBF3                       jmp     short vbe3h_chcl_next

                                di5:
                                    ; 18/10/2023 - TRDOS 386 v2.0.7
                                    ; ATI RV370 Video BIOS (VESA VBE2)
                                    ; has PMI. (as described in VESA VBE3 specification)
                                    ;
000007E0 FE05[7C090000]             inc     byte [vbe3]  ; [vbe3] = 2 -> 3
                                                ; will be decreased to 2 again if 'PMID'
                                                ; signature will not be found.
000007E6 E95FFEFFFF                 jmp     vbe3_pmid_chk ; check for ATI Video BIOS

                                di0:
                                    ; 30/11/2020
                                    ; Memory allocation error !
000007EB C605[7C090000]00           mov     byte [vbe3], 0 ; disable VBE3

                                display_mem_info:
                                    ; 19/12/2020
                                    ; temporary
                                    ; 24/11/2023
000007F2 803D[7C090000]02           cmp     byte [vbe3], 2
000007F9 7205                       jb      short dmi
000007FB E8A1390000                 call    default_lfb_info
                                dmi:
                                    ; 06/11/2014
00000800 E827390000                 call    memory_info
                                    ; 14/08/2015
                                    ;call   getch ; 28/02/2015

                                    ; 07/12/2023
                                    ; check EDID info for LCD monitor -screen resolution-
                                    ;       for modifying VGA mode 13h CRTC parameters
                                    ;       (if it is needed or not)

00000805 E82D010000                 call    video_mode_13h_parms

                                drv_init:
0000080A FB                         sti     ; Enable Interrupts
                                    ; 06/02/2015
0000080B 8B15[8E660000]             mov     edx, [hd0_type] ; hd0, hd1, hd2, hd3
00000811 668B1D[8C660000]           mov     bx, [fd0_type] ; fd0, fd1
                                    ; 22/02/2015
00000818 6621DB                     and     bx, bx
0000081B 756C                       jnz     short di1
                                    ;
0000081D 09D2                       or      edx, edx
0000081F 757A                       jnz     short di2
                                    ;
                                setup_error:
00000821 BE[FE380100]               mov     esi, setup_error_msg
                                psem:
00000826 AC                         lodsb
00000827 08C0                       or      al, al
                                    ;jz     short haltx ; 22/02/2015
00000829 747C                       jz      short di3
0000082B 56                         push    esi
                                    ; 13/05/2016
0000082C BB07000000                 mov     ebx, 7  ; Black background,
                                                    ; light gray forecolor
                                                    ; Video page 0 (BH=0)
00000831 E8A91A0000                 call    _write_tty
00000836 5E                         pop     esi
00000837 EBED                       jmp     short psem

                                check_boch_plex86_vbe:
                                    ; 20/10/2023
                                    ; 18/10/2023
                                    ; 20/11/2020
                                    ; check Bochs/Plex86 VGABios VBE extension
                                    ; (check if TRDOS 386 v2 is running on emulators)
                                    ; BOCHS/QEMU/VIRTUALBOX
                                    ;
                                    ; ref: vbe_display_api.txt

                                    ; bochs/plex86 VGAbios VBE source code
                                    ;  by Jeroen Janssen (2002)
                                    ;  and Volker Ruppert (2003-2020)

                                    ; 20/10/2023
                                    ; (ATI RV370 video bios has PMI support but
                                    ; it is a VESA VBE2 bios. So, even if [vbe3] is set
                                    ; to 3 for PMI functionality, it is better to display
                                    ; VESA VBE number as it is declared by the video bios.)

00000839 C605[5E730100]32           mov     byte [vbe_vnumber], "2" ; 20/10/2023

00000840 29C0                       sub     eax, eax ; 0
00000842 66BACE01                   mov     dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
00000846 66EF                       out     dx, ax ; VBE_DISPI_INDEX_ID register
                                    ;mov    ax, 0B0C0h ; VBE_DISPI_ID0
                                    ;mov    dx, 1CFh ; VBE_DISPI_IOPORT_DATA
00000848 6642                       inc     dx
                                    ;out    dx, ax
                                    ;nop
0000084A 66ED                       in      ax, dx
0000084C 80FCB0                     cmp     ah, 0B0h
                                    ;jne    short not_boch_qemu_vbe
                                    ; 18/10/2023
                                    ;jne    short display_mem_info
0000084F 758F                       jne     short di5  ; ATI VESA VBE2 bios or another

00000851 3CC5                       cmp     al, 0C5h ; it must be 0B0C4h or 0B0C5h ..
                                    ;ja     short not_boch_qemu_vbe
00000853 779D                       ja      short display_mem_info
00000855 3CC0                       cmp     al, 0C0h ; 0B0C0h to 0B0C5h .. ; Qemu
                                    ;cmp    al, 0C4h ; 0BC04h or 0B0C5h is OK ; Bochs
                                    ;jb     short not_boch_qemu_vbe
00000857 7299                       jb      short display_mem_info

                                    ; save VESA VBE2 bios (bochs/qemu) signature
                                    ; for enabling VBE2 functions in TRDOS 386 v2 kernel
00000859 A2[7D090000]               mov     [vbe2bios], al ; 0C4h or 0C5h (for BOCHS)
                                                           ; (0C0h-0C5h for QEMU)
                                    ; 20/10/2023
                                    ;mov    byte [vbe_vnumber], "2"
```

```
                                    ; 26/11/2020
                                    ; "BOCHS/QEMU/VIRTUALBOX VBE2 Video BIOS ..".
0000085E BE[40730100]               mov     esi, vbe2_bochs_vbios ; BOCH/QEMU vbios msg
00000863 B40E                       mov     ah, 0Eh  ; Yellow font
00000865 E81B390000                 call    print_kmsg

                                    ; this is not necessary ! (20/11/2020)
0000086A 803D[7D090000]C4           cmp     byte [vbe2bios], 0C4h
                                    ;jb     display_mem_info      ; (QEMU)
                                    ; 02/12/2023
00000871 720E                       jb      short not_boch_qemu_vbe

                                    ; Display kernel version message if 0E9h hack port
                                    ; is enabled (bochs emulator feature)
00000873 66BAE900                   mov     dx, 0E9h ; hack port for BOCHS
00000877 BE[99730100]               mov     esi, kernel_version_msg
                                kvmsg_next_char:
0000087C AC                         lodsb
0000087D 08C0                       or      al, al
0000087F 7505                       jnz     short put_kvmsg_in_hack_port
                                not_boch_qemu_vbe:
00000881 E96CFFFFFF                 jmp     display_mem_info
                                put_kvmsg_in_hack_port:
00000886 EE                         out     dx, al
00000887 EBF3                       jmp     short kvmsg_next_char

                                di1:
                                    ; supress 'jmp short T6'
                                    ;  (activate fdc motor control code)
00000889 66C705[DE090000]90-        mov     word [T5], 9090h ; nop
00000891 90
                                    ;
                                    ;mov    ax, int_0Eh    ; IRQ 6 handler
                                    ;mov    di, 0Eh*4      ; IRQ 6 vector
                                    ;stosw
                                    ;mov    ax, cs
                                    ;stosw
                                    ;; 16/02/2015
                                        ;;mov     dword [DISKETTE_INT], fdc_int ; IRQ 6 handler
                                    ;
00000892 E823460000                 CALL    DSKETTE_SETUP  ; Initialize Floppy Disks
                                    ;
00000897 09D2                       or      edx, edx
00000899 740C                           jz      short di3
                                di2:
0000089B E850460000                 call    DISK_SETUP     ; Initialize Fixed Disks
                                        ;jc setup_error
                                    ; 24/07/2022
000008A0 7305                       jnc     short di3
000008A2 E97AFFFFFF                 jmp     setup_error
                                di3:
000008A7 E854380000                 call    setup_rtc_int  ; 22/05/2015 (dsectrpm.s)
                                    ;
000008AC E8E32E0100                 call    display_disks ; 07/03/2015  (Temporary)
                                ;haltx:
                                    ; 14/08/2015
                                    ;call   getch ; 22/02/2015
                                    ;sti    ; Enable interrupts (for CPU)
                                ;   ; 29/01/2016
                                ;   sub     ah, ah ;  read time count
                                ;   call    int1Ah
                                ;   mov     edx, ecx ; 18.2 * seconds
                                ;md_info_msg_wait1:
                                ;   ; 29/01/2016
                                ;   mov     ah, 1
                                ;   call    int16h
                                ;   jz      short md_info_msg_wait2
                                ;   xor     ah, ah ; 0
                                ;       call    int16h
                                ;   jmp     short md_info_msg_ok
                                ;md_info_msg_wait2:
                                ;   sub     ah, ah  ; read time count
                                ;   call    int1Ah
                                ;   cmp     edx, ecx ; ; 18.2 * seconds
                                ;   jna     short md_info_msg_wait3
                                ;   xchg    edx, ecx
                                ;md_info_msg_wait3:
                                ;   sub     ecx, edx
                                ;   cmp     ecx, 127 ; 7 seconds (18.2 * 7)
                                ;   jb      short md_info_msg_wait1
                                ;md_info_msg_ok:

                                    ; 15/12/2020
                                    ; set initial values of LFB parameters

000008B1 803D[7C090000]02           cmp     byte [vbe3], 2
000008B8 7225                       jb      short di4

                                    ;mov    ax, [def_LFB_addr]
                                    ;shl    eax, 16
                                    ;mov    [LFB_ADDR], eax
                                    ;mov    eax, 1024*768*3
                                    ;mov    [LFB_SIZE], eax

000008BA BEFE7B0900                 mov     esi, VBE3MODEINFOBLOCK - 2
000008BF 66C7061801                 mov     word [esi], 0118h ; default vbe mode
                                                              ; 1024*768, 24bpp
000008C4 E8E2300000                 call    set_lfbinfo_table

                                    ;;;
                                    ; 28/11/2023
                                    ; 20/10/2023 - TRDOS 386 v2.0.7
000008C9 8B35[349D0100]             mov     esi, [LFB_ADDR] ; LFB base address (in bytes)
                                    ;mov    edx, [LFB_SIZE] ; 28/11/2023
                                    ;add    edx, 4095
                                    ;;;
                                    ;mov    edx, ((1024*768*4)+4095)>>12 ; 28/11/2023
000008CF BAE9070000                 mov     edx, ((1920*1080*4)+4095)>>12 ; 28/11/2023
                                    ; edx = LFB size in pages

                                    ; 29/11/2023
                                    ;mov    ebx, esi
                                    ;;add   ebx, 4095 ; LFB start addr is always in page boundary
                                    ;shr    ebx, 12 ; convert byte address to page address
000008D4 8B0D[B4760100]             mov     ecx, [memory_size]
                                    ;cmp    ebx, ecx
                                    ;jnb    short di4 ; LFB addr >= main memory size

                                    ; (set the overlapped pages as allocated for kernel)
                                    ;;;
                                    ; 28/11/2023
                                    ; (and set all of LFB pages in the kernel's page tables)

000008DA E8AC590000                 call    allocate_lfb_pages_for_kernel
```

```
                                di4:
                                    ; 08/09/2016
000008DF 0F20C0                     mov     eax, cr0
000008E2 A810                       test    al, 10h  ; Bit 4, ET (Extension Type)
000008E4 7408                       jz      short sysinit
                                    ; 27/02/2017
000008E6 FE05[74830100]             inc     byte [fpready]
                                    ; 80387 (FPU) is ready
000008EC DBE3                       fninit ; Initialize Floating-Point Unit
                                sysinit:
                                    ; 30/06/2015
000008EE E8A7630000                 call    sys_init
                                    ;
                                    ;jmp    cpu_reset ; 22/02/2015
                                hang:
                                    ; 23/02/2015
                                    ;sti                    ; Enable interrupts
000008F3 F4                         hlt
                                    ;
                                    ;nop
                                    ;; 03/12/2014
                                    ;; 28/08/2014
                                    ;mov    ah, 11h
                                    ;call   getc
                                    ;jz      _c8
                                    ;
                                    ; 23/02/2015
                                    ; 06/02/2015
                                    ; 07/09/2014
000008F4 31DB                       xor     ebx, ebx
000008F6 8A1D[DE760100]             mov     bl, [ptty]      ; active_page
000008FC 89DE                       mov     esi, ebx
                                    ;shl    si, 1
                                    ; 24/07/2022
000008FE D1E6                       shl     esi, 1
00000900 81C6[E0760100]             add     esi, ttychr
00000906 668B06                     mov     ax, [esi]
00000909 6621C0                     and     ax, ax
                                    ;jz     short _c8
0000090C 74E5                       jz      short hang
0000090E 66C7060000                 mov     word [esi], 0
00000913 80FB03                     cmp     bl, 3           ; Video page 3
                                    ;jb     short _c8
00000916 72DB                       jb      short hang
                                    ;
                                    ; 13/05/2016
                                    ; 07/09/2014
                                nxtl:
                                    ;push   bx
                                    ; 18/04/2021
00000918 53                         push    ebx
00000919 66BB0E00                   mov     bx, 0Eh         ; Yellow character
                                                            ; on black background
                                                            ; bh = 0 (video page 0)
                                                            ; Retro UNIX 386 v1 - Video Mode 0
                                                            ; (PC/AT Video Mode 3 - 80x25 Alpha.)
                                    ;push   ax
                                    ; 18/04/2021
0000091D 50                         push    eax
0000091E E8BC190000                 call    _write_tty
                                    ;pop    ax
                                    ; 18/04/2021
00000923 58                         pop     eax
                                    ;pop    bx
00000924 5B                         pop     ebx
00000925 3C0D                       cmp     al, 0Dh         ; carriage return (enter)
                                    ;jne    short _c8
00000927 75CA                       jne     short hang
00000929 B00A                       mov     al, 0Ah         ; next line
0000092B EBEB                       jmp     short nxtl

                                ;_c8:
                                ;   ; 25/08/2014
                                ;   cli                     ; Disable interrupts
                                ;   mov     al, [scounter + 1]
                                ;   and     al, al
                                ;   jnz     hang
                                ;   call    rtc_p
                                ;   jmp      hang

                                    ; 27/08/2014
                                    ; 20/08/2014
                                printk:
                                        ;mov    edi, [scr_row]
                                pkl:
0000092D AC                         lodsb
0000092E 08C0                       or      al, al
00000930 7404                       jz      short pkr
00000932 66AB                       stosw
00000934 EBF7                       jmp     short pkl
                                pkr:
00000936 C3                         retn

                                ; ************************************
                                video_mode_13h_parms:
                                    ; 07/12/2023 - TRDOS 386 v2.0.7
                                    ; Check EDID information for LCD monitors
                                    ; and change mode 13h parameters if it is required
                                    ; (if resolution > 1280x1024, it is LCD/panel monitor
                                    ; and Video Mode 13h parameters will be modified for 60HZ
                                    ; because LCD monitors does/can not display 320x200 70HZ
                                    ; standard VGA mode)

00000937 803D[1D420000]4F           cmp     byte [edid], 4Fh
0000093E 753B                       jne     short CRT_monitor

00000940 BE[A49C0100]               mov     esi, edid_info
00000945 83C626                     add     esi, 26h ; EDID Standard Timing Identification
00000948 B908000000                 mov     ecx, 8
                                chk_edid:
0000094D 66AD                       lodsw
0000094F 3C81                       cmp     al, 129 ; (1280/8)-31
00000951 770E                       ja      short LCD_monitor ; 16:9
00000953 3C01                       cmp     al, 1
00000955 7624                       jna     short CRT_monitor
00000957 E2F4                       loop    chk_edid
                                    ;jmp    short CRT_monitor
00000959 C3                         retn
                                mode13h_crtc_60hz:
0000095A 0B3EB9858FB8E2             db      0Bh, 3Eh, 0B9h, 85h, 8Fh, 0B8h, 0E2h
                                LCD_monitor:
                                    ; modify default (CRTC register) parameters for 60HZ
                                    ; (320x200 letterbox type screen will appear on LCD screen)
                                    ;
```

```
                                    ; Note: When/While [PMI32] -protected mode video bios-
                                    ;       feature is enabled -by user- for MODE 13h, internal
                                    ;       (TRDOS 386) VGA -video mode setting- parameters
                                    ;       will not be used (will be bypassed)
                                    ;       by TRDOS 386 kernel for all standard VGA modes.
                                    ;       ((NVIDIA/ATI Video Bios PMI will be used.))
                                    ;
                                    ;       ref: github, juj/60hz.cpp
                                    ;            https://gist.github.com/juj/
                                    ;
00000961 B107                       mov     cl, 7
00000963 BE[5A090000]               mov     esi, mode13h_crtc_60hz
00000968 BF[CA690000]               mov     edi, vga_mode_13h+10+6 ; CRTC Registers (index: 6)
0000096D A4                         movsb           ; Vertical Total Register
0000096E A4                         movsb           ; Overflow Register
0000096F BF[D4690000]               mov     edi, vga_mode_13h+10+16 ; CRTC Regs (index: 16)
00000974 A4                         movsb           ; Vertical Retrace Start Register
00000975 A4                         movsb           ; Vertical Retrace End Register
00000976 A4                         movsb           ; Vertical Display Enable/End Register
00000977 47                         inc     edi
00000978 47                         inc     edi
00000979 A4                         movsb           ; Vertical Blanking Start Register
0000097A A4                         movsb           ; Vertical Blanking End Register
                                CRT_monitor:
0000097B C3                         retn
                                ; ************************************

                                ; 14/11/2020 (TRDOS 386 v2.0.3)
0000097C 00                     vbe3:       db 0  ; VESA VBE version (must be 03h)
                                          ; for using video bios calls in protected mode
                                vbe2bios:
0000097D B0                         db 0B0h ;
                                ;pmid_addr:
                                    ;dw 0  ; > 0 if 'PMID' sign is found
                                    ;     ; ('pmid' offset addr in VGA bios seg, 0C000h)
                                    ;; 02/12/2020
                                    ;dw 0  ; 32 bit address in pmid_addr

                                ; 28/02/2017
                                ; 22/01/2017
                                ; 15/01/2017
                                ; 14/01/2017
                                ; 02/01/2017
                                ; 25/12/2016
                                ; 19/12/2016
                                ; 10/12/2016 (callback)
                                ; 06/06/2016
                                ; 23/05/2016
                                ; 22/05/2016 - TRDOS 386 (TRDOS v2.0) Timer Event Modifications
                                ; 25/07/2015
                                ; 14/05/2015 (multi tasking -time sharing- 'clock', x_timer)
                                ; 17/02/2015
                                ; 06/02/2015 (unix386.s)
                                ; 11/12/2014 - 22/12/2014 (dsectrm2.s)
                                ;
                                ; IBM PC-XT Model 286 Source Code - BIOS2.ASM (06/10/85)
                                ;
                                ;-- HARDWARE INT  08 H - ( IRQ LEVEL 0 ) --------------------------------------
                                ;   THIS ROUTINE HANDLES THE TIMER INTERRUPT FROM FROM CHANNEL 0 OF        :
                                ;   THE 8254 TIMER.  INPUT FREQUENCY IS 1.19318 MHZ AND THE DIVISOR        :
                                ;   IS 65536, RESULTING IN APPROXIMATELY 18.2 INTERRUPTS EVERY SECOND.     :
                                ;                                                                        :
                                ;   THE INTERRUPT HANDLER MAINTAINS A COUNT (40:6C) OF INTERRUPTS SINCE    :
                                ;   POWER ON TIME, WHICH MAY BE USED TO ESTABLISH TIME OF DAY.        :
                                ;   THE INTERRUPT HANDLER ALSO DECREMENTS THE MOTOR CONTROL COUNT (40:40)  :
                                ;   OF THE DISKETTE, AND WHEN IT EXPIRES, WILL TURN OFF THE           :
                                ;   DISKETTE MOTOR(s), AND RESET THE MOTOR RUNNING FLAGS.                  :
                                ;   THE INTERRUPT HANDLER WILL ALSO INVOKE A USER ROUTINE THROUGH           :
                                ;   INTERRUPT 1CH AT EVERY TIME TICK.  THE USER MUST CODE A           :
                                ;   ROUTINE AND PLACE THE CORRECT ADDRESS IN THE VECTOR TABLE.        :
                                ;----------------------------------------------------------------------------
                                ;

                                timer_int:  ; IRQ 0
                                ;int_08h:   ; Timer
                                    ; 14/10/2015
                                    ; Here, we are simulating system call entry (for task switch)
                                    ; (If multitasking is enabled,
                                    ; 'clock' procedure may jump to 'sysrelease')

0000097E 1E                         push    ds
0000097F 06                         push    es
00000980 0FA0                       push    fs
00000982 0FA8                       push    gs

00000984 60                         pushad  ; eax, ecx, edx, ebx, esp -before pushad-, ebp, esi, edi
00000985 66B91000                   mov     cx, KDATA
00000989 8ED9                       mov     ds, cx
0000098B 8EC1                       mov     es, cx
0000098D 8EE1                       mov     fs, cx
0000098F 8EE9                       mov     gs, cx

00000991 0F20D9                     mov     ecx, cr3
00000994 890D[7C8F0100]             mov     [cr3reg], ecx ; save current cr3 register value/content

                                    ; 14/01/2017
0000099A 3B0D[B0760100]             cmp     ecx, [k_page_dir]
000009A0 7409                       je      short T3

000009A2 8B0D[B0760100]             mov     ecx, [k_page_dir]
000009A8 0F22D9                     mov     cr3, ecx
                                T3:
                                    ;sti                        ; INTERRUPTS BACK ON
000009AB 66FF05[30770100]           INC     word [TIMER_LOW]    ; INCREMENT TIME
000009B2 7507                       JNZ     short T4            ; GO TO TEST_DAY
000009B4 66FF05[32770100]           INC     word [TIMER_HIGH]   ; INCREMENT HIGH WORD OF TIME
                                T4:                             ; TEST_DAY
000009BB 66833D[32770100]18         CMP     word [TIMER_HIGH],018H; TEST FOR COUNT EQUALING 24 HOURS
000009C3 7519                       JNZ     short T5            ; GO TO DISKETTE_CTL
000009C5 66813D[30770100]B0-        CMP     word [TIMER_LOW],0B0H
000009CD 00
000009CE 750E                       JNZ     short T5            ; GO TO DISKETTE_CTL

                                ;-----      TIMER HAS GONE 24 HOURS
                                    ;;SUB   AX,AX
                                    ;MOV    [TIMER_HIGH],AX
                                    ;MOV    [TIMER_LOW],AX
000009D0 29C0                       sub     eax, eax
000009D2 A3[30770100]               mov     [TIMER_LH], eax
                                    ;
000009D7 C605[34770100]01           MOV     byte [TIMER_OFL],1

                                ;-----      TEST FOR DISKETTE TIME OUT

```

```
                                T5:
                                    ; 23/12/2014
000009DE EB1D                       jmp     short T6                ; will be replaced with nop, nop
                                                                    ; (9090h) if a floppy disk
                                                                    ; is detected.
                                    ;mov    al,[CS:MOTOR_COUNT]
000009E0 A0[37770100]               mov     al, [MOTOR_COUNT]
000009E5 FEC8                       dec     al
                                    ;mov    [CS:MOTOR_COUNT], al    ; DECREMENT DISKETTE MOTOR CONTROL
000009E7 A2[37770100]               mov     [MOTOR_COUNT], al
                                    ;mov    [ORG_MOTOR_COUNT], al
000009EC 750F                       JNZ     short T6                ; RETURN IF COUNT NOT OUT
000009EE B0F0                       mov     al,0F0h
                                    ;AND    [CS:MOTOR_STATUS],al    ; TURN OFF MOTOR RUNNING BITS
000009F0 2005[36770100]             and     [MOTOR_STATUS], al
                                    ;and    [ORG_MOTOR_STATUS], al
000009F6 B00C                       MOV     AL,0CH                  ; bit 3 = enable IRQ & DMA,
                                                                    ; bit 2 = enable controller
                                                                    ;       1 = normal operation
                                                                    ;       0 = reset
                                                                    ; bit 0, 1 = drive select
                                                                    ; bit 4-7 = motor running bits
000009F8 66BAF203                   MOV     DX,03F2H                ; FDC CTL PORT
000009FC EE                         OUT     DX,AL                   ; TURN OFF THE MOTOR
                                T6:
                                    ;inc    word [CS:wait_count]    ; 22/12/2014 (byte -> word)
                                                                    ; TIMER TICK INTERRUPT
                                    ;;inc   word [wait_count] ;;27/02/2015
                                    ;INT    1CH                     ; TRANSFER CONTROL TO A USER ROUTINE
                                    ;cli
000009FD E85E040000                 call    u_timer                 ; TRANSFER CONTROL TO A USER ROUTINE
                                    ; 23/05/2016
00000A02 E83F110100                 call    clock                   ; Multi Tasking control procedure
                                T7:
                                    ; 14/10/2015
00000A07 B020                       MOV     AL,EOI                  ; GET END OF INTERRUPT MASK
00000A09 FA                         CLI                             ; DISABLE INTERRUPTS TILL STACK CLEARED
00000A0A E620                       OUT     INTA00,AL               ; END OF INTERRUPT TO 8259 - 1
                                    ;
                                    ;;;
                                    ; 21/08/2024
00000A0C 803D[45830100]FF           cmp     byte [p_change], 0FFh ; CTRL+BREAK signature
00000A13 7424                       je      short T9 ; current program will be forced to sysrelease
                                    ;;;
                                rtc_int_2:
                                    ; 26/12/2016
                                    ;mov    ecx, [cr3reg]
                                    ; 13/01/2017
00000A15 803D[E48E0100]00           cmp     byte [u.t_lock], 0    ; T_LOCK
00000A1C 7730                       ja      short timer_int_return  ; Timer Lock : 'sysrele' is needed !
                                    ; 28/02/2017
                                    ; We need to exit if the user's IRQ callback service is in progress!
                                    ; (To prevent a conflict!)
00000A1E 803D[E88E0100]00           cmp     byte [u.r_lock], 0    ; R_LOCK, IRQ callback service lock !
00000A25 7727                       ja      short timer_int_return  ; Timer Lock : 'sysrele' is needed !
                                    ; 15/01/2017
00000A27 803D[44830100]02           cmp     byte [priority], 2
00000A2E 733A                       jnb     short T8  ; current process has a timer event (15/01/2017)
                                    ; 22/05/2016
00000A30 803D[45830100]00           cmp     byte [p_change], 0 ; in 'set_run_sequence', in 'rtc_p'
00000A37 7615                       jna     short timer_int_return ; 23/05/2016

                                    ; 15/01/2017
                                    ; present process must be changed with high priority process
                                T9:         ; 21/08/2024
                                    ;xor    al, al
00000A39 31C0                       xor     eax, eax ; 26/12/2016
00000A3B A2[45830100]               mov     [p_change], al ; 0
                                    ;mov    byte [priority], 2 ; 15/01/2017 (there is a timer event)

00000A40 803D[608E0100]FF           cmp      byte [sysflg], 0FFh ; user or system space ?
00000A47 7416                       je      short rtc_int_3     ; user space ([sysflg]= 0FFh)

                                    ; system space, wait for 'sysret'
                                    ; to change running process
                                    ; with high priority (event) process

00000A49 A2[B48E0100]               mov     [u.quant], al ; 0

                                timer_int_return: ; 23/05/2016 - jump from 'rtc_int' ('rtc_int_2')
00000A4E 8B0D[7C8F0100]             mov     ecx, [cr3reg]  ; previous value/content of cr3 register
00000A54 0F22D9                     mov     cr3, ecx       ; restore cr3 register content
                                    ;
00000A57 61                         popad ; edi, esi, ebp, temp (icrement esp by 4), ebx, edx, ecx, eax
                                    ;
00000A58 0FA9                       pop     gs
00000A5A 0FA1                       pop     fs
00000A5C 07                         pop     es
00000A5D 1F                         pop     ds
                                    ;
00000A5E CF                         iretd   ; return from interrupt

                                rtc_int_3:
00000A5F FE05[608E0100]             inc     byte [sysflg]  ; now, we are in system space
                                    ;
00000A65 E945C00000                     jmp     sysrelease ; change running process immediatelly

                                T8:
                                    ; 13/01/2017 (eax -> ebx)
                                    ; callback checking... (19/12/2016)
00000A6A 31DB                       xor     ebx, ebx
00000A6C 871D[E08E0100]             xchg    ebx, [u.tcb] ; callback address (0 = normal return)
00000A72 09DB                       or      ebx, ebx
00000A74 74D8                       jz      short timer_int_return

                                    ; Set user's callback routine as return address from this interrupt
                                    ; and set normal return address as return address from callback
                                    ; routine!!! (19/12/2016)

                                    ; 14/01/2017
                                    ; 13/01/2017 - Timer Lock (T_LOCK)
00000A76 FE05[E48E0100]             inc     byte [u.t_lock]
00000A7C 8A0D[608E0100]             mov     cl, [sysflg]
00000A82 880D[E58E0100]             mov     [u.t_mode], cl

00000A88 8B2D[4C760100]             mov     ebp, [tss.esp0] ; kernel stack address (for ring 0)
00000A8E 83ED14                     sub     ebp, 20        ; eip, cs, eflags, esp, ss
00000A91 892D[648E0100]             mov     [u.sp], ebp
00000A97 8925[688E0100]             mov     [u.usp], esp

                                    ;or     word [ebp+8], 200h ; 22/01/2017, force enabling interrupts

00000A9D 8B44241C                   mov     eax, [esp+28] ; pushed eax
00000AA1 A3[6C8E0100]               mov     [u.r0], eax
```

```
00000AA6 E82CFF0000                call   wswap ; save user's registers & status

                                   ; software int is in ring 0 but timer int must return to ring 3
                                   ; so, ring 3 return address and stack registers
                                   ; (eip, cs, eflags, esp, ss)
                                   ; must be copied to timer int return
                                   ; eip will be replaced by callback service routine address

00000AAB C605[608E0100]FF          mov    byte [sysflg], 0FFh ; user mode

                                   ; system mode (system call)
                                   ;mov   ebp, [u.sp] ; EIP (u), CS (UCODE), EFLAGS (u),
                                                      ; ESP (u), SS (UDATA)

00000AB2 8B4510                    mov    eax, [ebp+16]  ; SS (UDATA
00000AB5 89E6                      mov    esi, esp
00000AB7 50                        push   eax
00000AB8 50                        push   eax
00000AB9 89E7                      mov    edi, esp
00000ABB 893D[688E0100]            mov    [u.usp], edi
00000AC1 B908000000                mov    ecx, ((ESPACE/4) - 4) ; except DS, ES, FS, GS
00000AC6 F3A5                      rep    movsd
00000AC8 B104                      mov    cl, 4
00000ACA F3AB                      rep    stosd
00000ACC 893D[648E0100]            mov    [u.sp], edi
00000AD2 89EE                      mov    esi, ebp
00000AD4 B105                      mov    cl, 5 ; EIP (u), CS (UCODE), EFLAGS (u), ESP (u), SS (UDATA)
00000AD6 F3A5                      rep    movsd

00000AD8 8B0D[C48E0100]            mov    ecx, [u.pgdir]
00000ADE 890D[7C8F0100]            mov    [cr3reg], ecx

                                   ; 13/01/207 (eax -> ebx)
                                   ; EBX = callback routine address (virtual, not physical address!)

                                   ; 09/01/2017
                                   ; !!! CALLBACK ROUTINE MUST BE ENDED/RETURNED WITH 'sysrele'
                                   ;     system call !!!
                                   ; 25/12/2016
                                   ; Callback Note: (19/12/2016)
                                   ; !!! CALLBACK ROUTINE MUST BE ENDED/RETURNED WITH 'RETN' !!!
                                   ;      pushf ; save flags
                                   ;      <callback service code>
                                   ;      popf  ; restore flags
                                   ;      retn ; return to normal running address
                                   ;

                                   ; 15/01/2017
                                   ; 14/01/2017
                                   ; 13/01/2017 (eax -> ebx)
                                   ; 10/01/2017
                               set_callback_addr:
                                   ; 09/01/2017 (**)
                                   ; 02/01/2017 (*)
                                   ; 25/12/2016 (*)
                                   ; 19/12/2016 (TRDOS 386 feature only!)
                                   ;
                                   ; This routine sets return address
                                   ; to start of user's interrupt
                                   ; service (callback) address
                                   ;; and sets callback 'retn' address to normal
                                   ;; return address of user's running code!
                                   ;
                                   ; INPUT:
                                   ;      EBX = callback routine/service address
                                   ;            (virtual, not physical address!)
                                   ;      [u.sp] = kernel stack, points to
                                   ;               user's EIP,CS,EFLAGS,ESP,SS
                                   ;               registers.
                                   ; OUTPUT:
                                   ;      EIP (user) = callback (service) address
                                   ;      CS (user) = UCODE
                                   ;      EFLAGS (user) = flags before callback
                                   ;       ESP (user) = ESP-4 (user, before callback)
                                   ;      [ESP](user) = EIP (user) before callback
                                   ;
                                   ; Note: If CPU was in user mode while entering
                                   ;      the timer interrupt service routine,
                                   ;      'IRET' will get return to callback routine
                                   ;      immediately. If CPU was in system/kernel mode
                                   ;      'iret' will get return to system call and
                                   ;      then, callback routine will be return address
                                   ;      from system call. (User's callback/service code
                                   ;      will be able to return to normal return address
                                   ;      via an 'retn' at the end.)
                                   ;
                                   ; Note(**): User's callback service code must be ended
                                   ;      with a 'sysrele' sytstem call ! (09/01/2017)
                                   ;
                                   ;      For example:
                                   ;
                                   ;      timer_callback:
                                   ;          ...
                                   ;          inc dword [time_counter]
                                   ;          ...
                                   ;          mov eax, 39 ; 'sysrele'
                                   ;          int 40h ; TRDOS 386 system call (interrupt)
                                   ;
                                   ;
                                   ;; Note(*): User's callback service code must preserve cpu
                                   ;;     flags if it has any instructions which changes
                                   ;;     flags in the service code. (25/12/2016)
                                   ;;
                                   ;;     For example:
                                   ;;
                                   ;;     timer_callback:
                                   ;;         pushf ; save flags
                                   ;;         ; this instruction changes zero flag
                                   ;;         inc dword [time_counter]
                                   ;;         popf ; restore flags
                                   ;;         retn ; return to normal user code
                                   ;;              (which is interrupted by the
                                   ;;               timer interput)
                                   ;;

                                   ; 15/01/2017
00000AE4 8B2D[648E0100]            mov    ebp, [u.sp]; kernel's stack, points to EIP (user)
00000AEA 895D00                    mov    [ebp], ebx
00000AED E95CFFFFFF                jmp    timer_int_return

                                   ; 15/01/2017
                                   ; 13/01/2017
                                   ; 19/12/2016
```

```
                                    ; 06/06/2016
                                    ; 23/05/2016
                                    ; 22/05/2016
                                    ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                                    ; 26/02/2015
                                    ; 07/09/2014
                                    ; 25/08/2014
                                rtc_int:       ; Real Time Clock Interrupt (IRQ 8)
                                    ; 22/05/2016
00000AF2 1E                         push   ds ; ** ; 23/05/2016
00000AF3 50                         push   eax ; *
00000AF4 66B81000                   mov    ax, KDATA
00000AF8 8ED8                       mov    ds, ax
                                    ;
00000AFA 8A25[2E770100]             mov    ah, [RTC_2Hz] ;  2 Hz interrupt to 1 Hz function
00000B00 80F401                     xor    ah, 1
00000B03 8825[2E770100]             mov    [RTC_2Hz], ah ; 1 = 0.5 second, 0 = 1 second
00000B09 753B                       jnz    short rtc_int_return ; half second
                                    ; 1 second
                                rtc_int_0:
                                    ; 22/05/2016
00000B0B 58                         pop    eax ; *
                                    ;
                                    ; 14/10/2015 ('timer_int')
                                    ; Here, we are simulating system call entry (for task switch)
                                    ; (If multitasking is enabled,
                                    ; 'clock' procedure may jump to 'sysrelease')
                                    ;push  ds ; ** ; 23/05/2016
00000B0C 06                         push   es
00000B0D 0FA0                       push   fs
00000B0F 0FA8                       push   gs
00000B11 60                         pushad  ; eax, ecx, edx, ebx, esp -before pushad-, ebp, esi, edi
00000B12 66B91000                   mov     cx, KDATA
                                        ;mov    ds, cx ; 06/06/2016
00000B16 8EC1                           mov     es, cx
00000B18 8EE1                           mov     fs, cx
00000B1A 8EE9                           mov     gs, cx
                                    ;
00000B1C 0F20D9                     mov    ecx, cr3
00000B1F 890D[7C8F0100]             mov    [cr3reg], ecx ; save current cr3 register value/content
                                    ;
00000B25 803D[E48E0100]00           cmp    byte [u.t_lock], 0 ; timer lock (callback) status ?
00000B2C 7711                       ja     short rtc_int_1          ; yes

                                    ; 15/01/2017
00000B2E 3B0D[B0760100]             cmp    ecx, [k_page_dir]
00000B34 7409                       je     short rtc_int_1

00000B36 8B0D[B0760100]             mov    ecx, [k_page_dir]
00000B3C 0F22D9                     mov    cr3, ecx
                                rtc_int_1:
                                    ; Timer event (kernel) functions must be performed with
                                    ; 1 second intervals - TRDOS 386 (TRDOS v2.0) feature ! -
                                    ;
                                    ; 25/08/2014
00000B3F E818030000                 call   rtc_p  ; 19/05/2016 - major modification

                                    ; 23/05/2016
00000B44 28E4                       sub    ah, ah ; 0
                                    ; 22/05/2016 - TRDOS 386 timer event modifications
                                rtc_int_return: ; 19/05/2016
                                    ; 22/02/2015 - dsectpm.s
                                    ; [ source: http://wiki.osdev.org/RTC ]
                                    ; read status register C to complete procedure
                                    ;(it is needed to get a next IRQ 8)
00000B46 B00C                       mov    al, 0Ch ;
00000B48 E670                       out    70h, al ; select register C
00000B4A 90                         nop
00000B4B E471                       in     al, 71h ; just throw away contents
                                    ; 22/02/2015
00000B4D B020                       MOV    AL,EOI         ; END OF INTERRUPT
                                    ;CLI                  ; DISABLE INTERRUPTS TILL STACK CLEARED
00000B4F E6A0                       OUT    INTB00,AL      ; FOR CONTROLLER #2

                                    ; 23/05/2016
00000B51 B020                       MOV    AL,EOI         ; GET END OF INTERRUPT MASK
00000B53 FA                         CLI                   ; DISABLE INTERRUPTS TILL STACK CLEARED
00000B54 E620                       OUT    INTA00,AL      ; END OF INTERRUPT TO 8259 - 1
                                    ;
                                    ; 23/05/2016
00000B56 20E4                       and    ah, ah
                                    ;jz    rtc_int_2
                                    ; 24/07/2022
00000B58 7505                       jnz    short rtc_int_4
00000B5A E9B6FEFFFF                 jmp    rtc_int_2
                                rtc_int_4:
                                    ; ah = 1 (half second)
00000B5F 58                         pop    eax ; *
00000B60 1F                         pop    ds  ; **
00000B61 CF                         iretd

                                ; ///////////////

                                    ; 28/08/2014
                                irq0:
                                        push       dword 0
00000B62 6A00
00000B64 EB48                       jmp    short which_irq
                                irq1:
00000B66 6A01                           push       dword 1
00000B68 EB44                       jmp    short which_irq
                                irq2:
00000B6A 6A02                           push       dword 2
00000B6C EB40                       jmp    short which_irq
                                irq3:
                                    ; 20/11/2015
                                    ; 24/10/2015
00000B6E 2EFF15[D01C0100]           call   dword [cs:com2_irq3]
00000B75 6A03                       push   dword 3
00000B77 EB35                       jmp    short which_irq
                                irq4:
                                    ; 20/11/2015
                                    ; 24/10/2015
00000B79 2EFF15[CC1C0100]           call   dword [cs:com1_irq4]
00000B80 6A04                           push       dword 4
00000B82 EB2A                       jmp    short which_irq
                                irq5:
00000B84 6A05                           push       dword 5
00000B86 EB26                       jmp    short which_irq
                                irq6:
00000B88 6A06                           push       dword 6
00000B8A EB22                       jmp    short which_irq
                                irq7:
00000B8C 6A07                           push       dword 7
00000B8E EB1E                       jmp    short which_irq
```

```
                                    irq8:
00000B90 6A08                               push       dword 8
00000B92 EB1A                           jmp    short which_irq
                                    irq9:
00000B94 6A09                               push       dword 9
00000B96 EB16                           jmp    short which_irq
                                    irq10:
00000B98 6A0A                               push       dword 10
00000B9A EB12                           jmp    short which_irq
                                    irq11:
00000B9C 6A0B                               push       dword 11
00000B9E EB0E                           jmp    short which_irq
                                    irq12:
00000BA0 6A0C                               push       dword 12
00000BA2 EB0A                           jmp    short which_irq
                                    irq13:
00000BA4 6A0D                               push       dword 13
00000BA6 EB06                           jmp    short which_irq
                                    irq14:
00000BA8 6A0E                               push       dword 14
00000BAA EB02                           jmp    short which_irq
                                    irq15:
00000BAC 6A0F                               push       dword 15
                                        ;jmp   short which_irq

                                        ; 22/01/2017
                                        ; 19/10/2015
                                        ; 29/08/2014
                                        ; 21/08/2014
                                    which_irq:
00000BAE 870424                         xchg   eax, [esp]  ; 28/08/2014
00000BB1 53                             push   ebx
00000BB2 56                             push   esi
00000BB3 57                             push   edi
00000BB4 1E                             push   ds
00000BB5 06                             push   es
                                        ;
00000BB6 88C3                           mov    bl, al
                                        ;
00000BB8 B810000000                     mov    eax, KDATA
00000BBD 8ED8                           mov    ds, ax
00000BBF 8EC0                           mov    es, ax
                                        ; 19/10/2015
00000BC1 FC                             cld
                                            ; 27/08/2014
00000BC2 8105[F0350100]A000-                add     dword [scr_row], 0A0h
00000BCA 0000
                                        ;
00000BCC B417                           mov    ah, 17h ; blue (1) background,
                                                       ; light gray (7) forecolor
00000BCE 8B3D[F0350100]                     mov     edi, [scr_row]
00000BD4 B049                           mov    al, 'I'
00000BD6 66AB                           stosw
00000BD8 B052                           mov    al, 'R'
00000BDA 66AB                           stosw
00000BDC B051                           mov    al, 'Q'
00000BDE 66AB                           stosw
00000BE0 B020                           mov    al, ' '
00000BE2 66AB                           stosw
00000BE4 88D8                           mov    al, bl
00000BE6 3C0A                           cmp    al, 10
00000BE8 7208                           jb     short ii1
00000BEA B031                           mov    al, '1'
00000BEC 66AB                           stosw
00000BEE 88D8                           mov    al, bl
00000BF0 2C0A                           sub    al, 10
                                    ii1:
00000BF2 0430                           add    al, '0'
00000BF4 66AB                           stosw
00000BF6 B020                           mov    al, ' '
00000BF8 66AB                           stosw
00000BFA B021                           mov    al, '!'
00000BFC 66AB                           stosw
00000BFE B020                           mov    al, ' '
00000C00 66AB                           stosw
                                        ; 23/02/2015
00000C02 80FB07                         cmp    bl, 7 ; check for IRQ 8 to IRQ 15
00000C05 7604                           jna    ii2
                                        ; 22/01/2017
00000C07 B020                           mov    al, 20h  ; END OF INTERRUPT COMMAND TO
00000C09 E6A0                           out    0A0h, al ; the 2nd 8259
                                    ii2:
00000C0B B020                           mov    al, 20h  ; END OF INTERRUPT COMMAND TO
00000C0D E620                           out    20h, al ; the 2nd 8259
00000C0F E9CA010000                     jmp     iiret
                                        ;
                                        ; 22/08/2014
                                        ;mov   al, 20h ; END OF INTERRUPT COMMAND TO 8259
                                        ;out   20h, al ; 8259 PORT
                                        ;
                                        ;pop   es
                                        ;pop   ds
                                        ;pop   edi
                                        ;pop   esi
                                        ;pop   ebx
                                        ;pop   eax
                                        ;iret

                                        ; 02/04/2015
                                        ; 25/08/2014
                                    exc0:
00000C14 6A00                               push       dword 0
00000C16 E990000000                         jmp     cpu_except
                                    exc1:
00000C1B 6A01                               push       dword 1
00000C1D E989000000                         jmp     cpu_except
                                    exc2:
00000C22 6A02                               push       dword 2
00000C24 E982000000                         jmp     cpu_except
                                    exc3:
00000C29 6A03                               push       dword 3
00000C2B EB7E                               jmp     cpu_except
                                    exc4:
00000C2D 6A04                               push       dword 4
00000C2F EB7A                               jmp     cpu_except
                                    exc5:
00000C31 6A05                               push       dword 5
00000C33 EB76                               jmp     cpu_except
                                    exc6:
00000C35 6A06                               push       dword 6
00000C37 EB72                               jmp     cpu_except
                                    exc7:
00000C39 6A07                               push       dword 7
00000C3B EB6E                               jmp     cpu_except
```

```
                                exc8:
                                    ; [esp] = Error code
00000C3D 6A08                           push    dword 8
00000C3F EB5C                           jmp     cpu_except_en
                                exc9:
00000C41 6A09                           push    dword 9
00000C43 EB66                           jmp     cpu_except
                                exc10:
                                    ; [esp] = Error code
00000C45 6A0A                           push    dword 10
00000C47 EB54                           jmp     cpu_except_en
                                exc11:
                                    ; [esp] = Error code
00000C49 6A0B                           push    dword 11
00000C4B EB50                           jmp     cpu_except_en
                                exc12:
                                    ; [esp] = Error code
00000C4D 6A0C                           push    dword 12
00000C4F EB4C                           jmp     cpu_except_en
                                exc13:
                                    ; [esp] = Error code
00000C51 6A0D                           push    dword 13
00000C53 EB48                           jmp     cpu_except_en
                                exc14:
                                    ; [esp] = Error code
00000C55 6A0E                           push    dword 14
00000C57 EB44                       jmp     short cpu_except_en
                                exc15:
00000C59 6A0F                           push    dword 15
00000C5B EB4E                           jmp     cpu_except
                                exc16:
00000C5D 6A10                           push    dword 16
00000C5F EB4A                           jmp     cpu_except
                                exc17:
                                    ; [esp] = Error code
00000C61 6A11                           push    dword 17
00000C63 EB38                       jmp     short cpu_except_en
                                exc18:
00000C65 6A12                           push    dword 18
00000C67 EB42                       jmp     short cpu_except
                                exc19:
00000C69 6A13                           push    dword 19
00000C6B EB3E                       jmp     short cpu_except
                                exc20:
00000C6D 6A14                           push    dword 20
00000C6F EB3A                       jmp     short cpu_except
                                exc21:
00000C71 6A15                           push    dword 21
00000C73 EB36                       jmp     short cpu_except
                                exc22:
00000C75 6A16                           push    dword 22
00000C77 EB32                       jmp     short cpu_except
                                exc23:
00000C79 6A17                           push    dword 23
00000C7B EB2E                       jmp     short cpu_except
                                exc24:
00000C7D 6A18                           push    dword 24
00000C7F EB2A                       jmp     short cpu_except
                                exc25:
00000C81 6A19                           push    dword 25
00000C83 EB26                       jmp     short cpu_except
                                exc26:
00000C85 6A1A                           push    dword 26
00000C87 EB22                       jmp     short cpu_except
                                exc27:
00000C89 6A1B                           push    dword 27
00000C8B EB1E                       jmp     short cpu_except
                                exc28:
00000C8D 6A1C                           push    dword 28
00000C8F EB1A                       jmp     short cpu_except
                                exc29:
00000C91 6A1D                           push    dword 29
00000C93 EB16                       jmp     short cpu_except
                                exc30:
00000C95 6A1E                           push    dword 30
00000C97 EB04                       jmp     short cpu_except_en
                                exc31:
00000C99 6A1F                           push    dword 31
00000C9B EB0E                           jmp     short cpu_except

                                    ; 19/10/2015
                                    ; 19/09/2015
                                    ; 01/09/2015
                                    ; 28/08/2015
                                    ; 28/08/2014
                                cpu_except_en:
00000C9D 87442404                   xchg    eax, [esp+4] ; Error code
00000CA1 36A3[80900100]             mov     [ss:error_code], eax
00000CA7 58                         pop     eax  ; Exception number
00000CA8 870424                     xchg    eax, [esp]
                                            ; eax = eax before exception
                                            ; [esp] -> exception number
                                            ; [esp+4] -> EIP to return
                                    ; 22/01/2017
                                    ; 19/10/2015
                                    ; 19/09/2015
                                    ; 01/09/2015
                                    ; 28/08/2015
                                    ; 29/08/2014
                                    ; 28/08/2014
                                    ; 25/08/2014
                                    ; 21/08/2014
                                cpu_except: ; CPU Exceptions
00000CAB FC                         cld
00000CAC 870424                     xchg    eax, [esp]
                                            ; eax = Exception number
                                            ; [esp] = eax (before exception)
00000CAF 53                         push    ebx
00000CB0 56                         push    esi
00000CB1 57                         push    edi
00000CB2 1E                         push    ds
00000CB3 06                         push    es
                                    ; 28/08/2015
00000CB4 66BB1000                   mov     bx, KDATA
00000CB8 8EDB                       mov     ds, bx
00000CBA 8EC3                       mov     es, bx
00000CBC 0F20DB                     mov     ebx, cr3
00000CBF 53                         push    ebx ; (*) page directory
                                    ; 19/10/2015
00000CC0 FC                         cld
                                    ; 25/03/2015
00000CC1 8B1D[B0760100]             mov     ebx, [k_page_dir]
00000CC7 0F22DB                     mov     cr3, ebx
                                    ; 28/08/2015
```

```
00000CCA 83F80E                     cmp     eax, 0Eh ; 14, PAGE FAULT
00000CCD 7510                       jne     short cpu_except_nfp
00000CCF E8FD4D0000                 call    page_fault_handler
00000CD4 21C0                       and     eax, eax
                                    ;jz     iiretp ; 01/09/2015
                                    ; 24/07/2022
00000CD6 7505                       jnz     short cpu_except_pf
00000CD8 E9FD000000                 jmp     iiretp
                                cpu_except_pf:     ; 24/07/2022
00000CDD B00E                       mov     al, 0Eh ; 14
                                cpu_except_nfp:
                                    ; 23/08/2016
00000CDF 803D[2E680000]03           cmp     byte [CRT_MODE], 3
00000CE6 7409                       je      short cpu_except_mode_3
00000CE8 50                         push    eax
00000CE9 B003                       mov     al, 3
00000CEB E8950E0000                 call    _set_mode
00000CF0 58                         pop     eax
                                cpu_except_mode_3:
                                    ; 02/04/2015
00000CF1 BB[F3080000]               mov     ebx, hang
00000CF6 875C241C                   xchg    ebx, [esp+28]
                                            ; EIP (points to instruction which faults)
                                            ; New EIP (hang)
00000CFA 891D[84900100]             mov     [FaultOffset], ebx
00000D00 C744242008000000           mov     dword [esp+32], KCODE ; kernel's code segment
00000D08 814C242400020000           or      dword [esp+36], 200h ; enable interrupts (set IF)
                                    ;
00000D10 88C4                       mov     ah, al
00000D12 240F                       and     al, 0Fh
00000D14 3C09                       cmp     al, 9
00000D16 7602                       jna     short h1ok
00000D18 0407                       add     al, 'A'-':'
                                h1ok:
00000D1A C0EC04                     shr     ah, 4
00000D1D 80FC09                     cmp     ah, 9
00000D20 7603                       jna     short h2ok
00000D22 80C407                     add     ah, 'A'-':'
                                h2ok:
00000D25 86E0                       xchg    ah, al
00000D27 66053030                   add     ax, '00'
00000D2B 66A3[48380100]             mov     [excnstr], ax
                                    ;
                                    ; 29/08/2014
00000D31 A1[84900100]               mov     eax, [FaultOffset]
00000D36 51                         push    ecx
00000D37 52                         push    edx
00000D38 89E3                       mov     ebx, esp
                                    ; 28/08/2015
00000D3A B910000000                 mov     ecx, 16  ; divisor value to convert binary number
                                                     ; to hexadecimal string
                                    ;mov    ecx, 10    ; divisor to convert
                                                       ; binary number to decimal string
                                b2d1:
00000D3F 31D2                       xor     edx, edx
00000D41 F7F1                       div     ecx
                                    ;push   dx
                                    ; 18/04/2021
00000D43 52                         push    edx
00000D44 39C8                       cmp     eax, ecx
00000D46 73F7                       jnb     short b2d1
00000D48 BF[53380100]               mov     edi, EIPstr ; EIP value
                                                        ; points to instruction which faults
                                    ; 28/08/2015
00000D4D 89C2                       mov     edx, eax
                                b2d2:
                                    ;add    al, '0'
00000D4F 8A82[0D420000]             mov     al, [edx+hexchrs]
00000D55 AA                         stosb               ; write hexadecimal digit to its place
00000D56 39E3                       cmp     ebx, esp
00000D58 7605                       jna     short b2d3
                                    ;pop    ax
                                    ; 18/04/2021
00000D5A 58                         pop     eax
00000D5B 88C2                       mov     dl, al
00000D5D EBF0                       jmp     short b2d2
                                b2d3:
00000D5F B068                       mov     al, 'h' ; 28/08/2015
00000D61 AA                         stosb
00000D62 B020                       mov     al, 20h     ; space
00000D64 AA                         stosb
00000D65 30C0                       xor     al, al      ; to do it an ASCIIZ string
00000D67 AA                         stosb
                                    ;
00000D68 5A                         pop     edx
00000D69 59                         pop     ecx
                                    ;
00000D6A B44F                       mov     ah, 4Fh ; red (4) background,
                                                    ; white (F) forecolor
00000D6C BE[38380100]               mov     esi, exc_msg ; message offset
                                    ;
                                    ; 20/01/2017 (!cpu exception!)
                                    ;
00000D71 8105[F0350100]A000-            add dword [scr_row], 0A0h
00000D79 0000
00000D7B 8B3D[F0350100]                 mov edi, [scr_row]
                                    ;
00000D81 C605[608E0100]00           mov     byte [sysflg], 0  ; system mode
00000D88 FB                             sti
                                    ;
00000D89 E89FFBFFFF                 call    printk
                                    ;
00000D8E B410                       mov     ah, 10h
00000D90 E87D010000                 call    int16h ; getc
                                    ;
00000D95 B003                       mov     al, 3
00000D97 E8E90D0000                 call    _set_mode
                                    ;
                                    ;mov    eax, 1
                                    ; 30/11/2023
00000D9C 31C0                       xor     eax, eax
00000D9E 40                         inc     eax
                                    ; eax = 1
00000D9F E928BE0000                 jmp     sysexit ; terminate process !!!

                                    ; 22/01/2017
                                    ; 18/04/2016
                                    ; 28/08/2015
                                    ; 23/02/2015
                                    ; 20/08/2014
                                ignore_int:
00000DA4 50                         push    eax
00000DA5 53                         push    ebx ; 23/02/2015
00000DA6 56                         push    esi
```

```
00000DA7 57                         push    edi
00000DA8 1E                         push    ds
00000DA9 06                         push    es
                                    ; 18/04/2016
00000DAA 66B81000                   mov     ax, KDATA
00000DAE 8ED8                       mov     ds, ax
00000DB0 8EC0                       mov     es, ax
                                    ; 28/08/2015
00000DB2 0F20D8                     mov     eax, cr3
00000DB5 50                         push    eax ; (*) page directory
                                    ;
00000DB6 B467                       mov     ah, 67h ; brown (6) background,
                                                    ; light gray (7) forecolor
00000DB8 BE[00370100]               mov     esi, int_msg ; message offset
                                piemsg:
                                        ; 27/08/2014
00000DBD 8105[F0350100]A000-            add     dword [scr_row], 0A0h
00000DC5 0000
00000DC7 8B3D[F0350100]                 mov     edi, [scr_row]
                                        ;
00000DCD E85BFBFFFF                 call    printk
                                    ;
                                    ; 23/02/2015
00000DD2 B020                       mov     al, 20h  ; END OF INTERRUPT COMMAND TO
00000DD4 E6A0                       out     0A0h, al ; the 2nd 8259
                                    ; 22/08/2014
00000DD6 B020                       mov     al, 20h ; END OF INTERRUPT COMMAND TO 8259
00000DD8 E620                       out     20h, al ; 8259 PORT
                                iiretp:
                                    ; 22/01/2017
                                    ; 01/09/2015
                                    ; 28/08/2015
00000DDA 58                         pop     eax ; (*) page directory
00000DDB 0F22D8                     mov     cr3, eax
                                iiret:
00000DDE 07                         pop     es
00000DDF 1F                         pop     ds
00000DE0 5F                         pop     edi
00000DE1 5E                         pop     esi
00000DE2 5B                         pop     ebx ; 29/08/2014
00000DE3 58                         pop     eax
00000DE4 CF                         iretd

                                    ; 23/05/2016
                                    ; 22/08/2014
                                    ; IBM PC/AT BIOS source code ----- 10/06/85 (bios.asm)
                                    ; (INT 1Ah)
                                    ;; Linux (v0.12) source code (main.c) by Linus Torvalds (1991)
                                time_of_day:
00000DE5 E83F570000                 call    UPD_IPR                 ; WAIT TILL UPDATE NOT IN PROGRESS
00000DEA 726F                           jc      short time_of_day_retn ; 23/05/2016
00000DEC B000                       mov     al, CMOS_SECONDS
00000DEE E86C570000                 call    CMOS_READ
00000DF3 A2[20770100]               mov     [time_seconds], al
00000DF8 B002                       mov     al, CMOS_MINUTES
00000DFA E860570000                 call    CMOS_READ
00000DFF A2[21770100]               mov     [time_minutes], al
00000E04 B004                       mov     al, CMOS_HOURS
00000E06 E854570000                 call    CMOS_READ
00000E0B A2[22770100]                   mov     [time_hours], al
00000E10 B006                       mov     al, CMOS_DAY_WEEK
00000E12 E848570000                 call    CMOS_READ
00000E17 A2[23770100]               mov     [date_wday], al
00000E1C B007                       mov     al, CMOS_DAY_MONTH
00000E1E E83C570000                 call    CMOS_READ
00000E23 A2[24770100]               mov     [date_day], al
00000E28 B008                       mov     al, CMOS_MONTH
00000E2A E830570000                 call    CMOS_READ
00000E2F A2[25770100]               mov     [date_month], al
00000E34 B009                       mov     al, CMOS_YEAR
00000E36 E824570000                 call    CMOS_READ
00000E3B A2[26770100]               mov     [date_year], al
00000E40 B032                       mov     al, CMOS_CENTURY
00000E42 E818570000                 call    CMOS_READ
00000E47 A2[27770100]               mov     [date_century], al
                                    ;
00000E4C B000                       mov     al, CMOS_SECONDS
00000E4E E80C570000                 call    CMOS_READ
00000E53 3A05[20770100]             cmp     al, [time_seconds]
00000E59 758A                       jne     short time_of_day

                                time_of_day_retn:
00000E5B C3                         retn

                                    ; 15/01/2017
                                    ; 10/06/2016
                                    ; 07/06/2016
                                    ; 06/06/2016
                                    ; 23/05/2016
                                rtc_p:
00000E5C B101                       mov     cl, 1 ; 15/01/2017
00000E5E EB02                       jmp     short rtc_p0
                                u_timer:
                                    ; Timer Events with 18.2 Hz Timer Ticks
                                    ; (and also timer events with RTC seconds)
00000E60 28C9                       sub     cl, cl ; mov cl, 0 ; 15/01/2017
                                rtc_p0:
                                    ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                                    ; Major Modification:
                                    ; Check and Perform Timer Events (for RTC)
                                    ; 25/08/2014 - 07/09/2014
                                    ; Retro UNIX 386 v1:
                                    ; Print Real Time Clock content

                                    ; 15/01/2017
00000E62 880D[44830100]             mov     byte [priority], cl ; 0 or 1 (not 2)
00000E68 8A2D[47830100]             mov     ch, [timer_events]
00000E6E 20ED                       and     ch, ch
00000E70 7420                       jz      short rtc_p3

00000E72 BE[808F0100]               mov     esi, timer_set  ; beginning address of
                                                            ; timer events space
                                rtc_p1:
00000E77 8B06                       mov     eax, [esi]
00000E79 20C0                       and     al, al ; 0 = free, >0 = process no.
00000E7B 7416                       jz      short rtc_p4
                                    ;
00000E7D C1C810                     ror     eax, 16
                                    ; ah = response value, al = interrupt type
                                    ; 15/01/2017
                                    ; cl = interrupt source
                                    ;       1 = RTC, 0 = PIT
00000E80 38C8                       cmp     al, cl
00000E82 750A                       jne     short rtc_p2 ; not as requested or undefined !
```

```
2869 00000E84 3C01                     cmp    al, 1 ; 1 ; RTC interrupt ?
2870 00000E86 7410                     je     short rtc_p5 ; yes, check for response
2871                                   ; 06/06/2016 - 18.2 Hz Timer Ticks
2872 00000E88 836E080A                 sub    dword [esi+8], 10 ; 1 tick = 10
2873 00000E8C 7613                     jna    short rtc_p6  ; continue for responding
2874                               rtc_p2:
2875                                   ; 15/01/2017 (cl -> ch)
2876                                   ; 07/06/2016
2877 00000E8E FECD                     dec    ch    ; remain count of timer events
2878 00000E90 7501                     jnz    short rtc_p4
2879                               rtc_p3:
2880 00000E92 C3                       retn
2881                               rtc_p4:
2882                                   ;cmp   esi, timer_set + 240 ; 15*16 (last event)
2883                                   ;jnb   short rtc_p3 ; end of timer event space
2884 00000E93 83C610                   add    esi, 16 ; next timer event
2885 00000E96 EBDF                     jmp    short rtc_p1
2886                               rtc_p5:
2887                                   ; current timer count ; 06/06/2016 (182)
2888 00000E98 816E08B6000000           sub    dword [esi+8], 182 ; 1 second (10*18.2)
2889 00000E9F 77ED                     ja     short rtc_p2  ; check for the next
2890                               rtc_p6:
2891                                   ; it is the time of response!
2892 00000EA1 8B5E04                   mov    ebx, [esi+4] ; set (count limit) value
2893 00000EA4 895E08                   mov    [esi+8], ebx ; reset count down value
2894                                                        ; to count limit
2895                                   ; 19/12/2016
2896                                   ; 10/12/2016 - timer callback modification
2897 00000EA7 8B7E0C                   mov    edi, [esi+12] ; response (or callback) address
2898 00000EAA 807E0100                 cmp    byte [esi+1], 0 ; >0 = callback
2899 00000EAE 762A                     jna    short rtc_p8
2900
2901                                   ; timer callback !
2902 00000EB0 0FB61E                   movzx  ebx, byte [esi] ; process number (>0)
2903 00000EB3 89D8                     mov    eax, ebx
2904 00000EB5 C0E302                   shl    bl, 2 ; *4
2905 00000EB8 89BB[FC8D0100]           mov    [ebx+p.tcb-4], edi ; user's callback service addr
2906 00000EBE 3A05[BD8E0100]           cmp    al, [u.uno]
2907 00000EC4 7521                     jne    short rtc_p9
2908 00000EC6 893D[E08E0100]           mov    [u.tcb], edi
2909                               rtc_p7:
2910                                   ; 15/01/2017
2911 00000ECC B002                     mov    al, 2
2912 00000ECE A2[44830100]             mov    [priority], al ; 2
2913                                   ; 10/01/2017
2914                                   ;mov   byte [u.pri], 2
2915 00000ED3 A2[B68E0100]             mov    [u.pri], al ; 2
2916 00000ED8 EBB4                     jmp    short rtc_p2
2917                               rtc_p8:
2918                                   ; response address is physical address of
2919                                   ; the program's response (signal return) byte
2920                                   ; 06/06/2016
2921                                   ;mov   edi, [esi+12] ; response address
2922 00000EDA 8827                     mov    [edi], ah     ; response value
2923                                   ;
2924 00000EDC C1C010                   rol    eax, 16
2925                                   ; 15/01/2017
2926 00000EDF 3A05[BD8E0100]           cmp    al, [u.uno] ; running process ?
2927 00000EE5 74E5                     je     short rtc_p7
2928                               rtc_p9:
2929                                   ; al = process number  ; 10/06/2016
2930 00000EE7 B202                     mov    dl, 2 ; priority, 2 = event (high)
2931 00000EE9 E80D0C0100               call   set_run_sequence ; 19/05/2016
2932 00000EEE EB9E                     jmp    short rtc_p2 ; 10/06/2016
2933
2934                               ; Default IRQ 7 handler against spurious IRQs (from master PIC)
2935                               ; 25/02/2015 (source: http://wiki.osdev.org/8259_PIC)
2936                               default_irq7:
2937                                   ;push  ax
2938                                   ; 18/04/2021
2939 00000EF0 50                       push   eax
2940 00000EF1 B00B                     mov    al, 0Bh  ; In-Service register
2941 00000EF3 E620                     out    20h, al
2942 00000EF5 EB00                         jmp short $+2
2943 00000EF7 EB00                     jmp short $+2
2944 00000EF9 E420                     in     al, 20h
2945 00000EFB 2480                     and    al, 80h ; bit 7 (is it real IRQ 7 or fake?)
2946 00000EFD 7404                         jz      short irq7_iret ; Fake (spurious) IRQ, do not send EOI
2947 00000EFF B020                         mov     al, 20h ; EOI
2948 00000F01 E620                     out    20h, al
2949                               irq7_iret:
2950                                   ;pop   ax
2951                                   ; 18/04/2021
2952 00000F03 58                       pop    eax
2953 00000F04 CF                       iretd
2954
2955                               bcd_to_ascii:
2956                                   ; 25/08/2014
2957                                   ; INPUT ->
2958                                   ;      al = Packed BCD number
2959                                   ; OUTPUT ->
2960                                   ;      ax  = ASCII word/number
2961                                   ;
2962                                   ; Erdogan Tan - 1998 (proc_hex) - TRDOS.ASM (2004-2011)
2963                                   ;
2964 00000F05 D410                     db     0D4h, 10h      ; Undocumented inst. AAM
2965                                                          ; AH = AL / 10h
2966                                                          ; AL = AL MOD 10h
2967 00000F07 660D3030                 or     ax, '00'       ; Make it ASCII based
2968
2969 00000F0B 86E0                         xchg       ah, al
2970
2971 00000F0D C3                       retn
2972
2973                               ; 15/12/2020
2974                               real_mem_16m_64k:
2975 00000F0E 0000                     dw     0       ; Real size of system memory (if > 16MB)
2976                                                   ; as number of 64K blocks - 256
2977                                                   ; (This is for saving real system memory
2978                                                   ; because if system memory is larger than
2979                                                   ; 3 GB and if a VESA VBE video bios
2980                                                   ; is detected, 'mem_16m_64K' may be
2981                                                   ; decreased to reserve LFB space
2982                                                   ; at the end of system memory.)
2983                                                   ; Upper memory space from LFB base address
2984                                                   ; to 4GB will not be included by M.A.T.
2985                               def_LFB_addr:
2986 00000F10 0000                     dw     0       ; HW of default LFB addr (for mode 118h)
2987
2988
2989                               %include 'keyboard.s' ; 07/03/2015
   1                           <1> ; ****************************************************************************
   2                           <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - keyboard.s
   3                           <1> ; ----------------------------------------------------------------------------
```

```
                                <1> ; Last Update: 07/08/2022 (Previous: 12/04/2021)
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Beginning: 17/01/2016
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Turkish Rational DOS
                                <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                                <1> ;
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; keyboard.inc (17/10/2015)
                                <1> ;
                                <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                                <1> ; ***************************************************************************
                                <1>
                                <1> ; Ref: Retro UNIX 386 v1.2 - keyboard.s - 11/06/2022
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - KEYBOARD.INC
                                <1> ; Last Modification: 17/10/2015
                                <1> ;              (Keyboard Data is in 'KYBDATA.INC')
                                <1> ;
                                <1> ; //////// KEYBOARD FUNCTIONS (PROCEDURES) ///////////////
                                <1>
                                <1> ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> ; 03/12/2014
                                <1> ; 26/08/2014
                                <1> ; KEYBOARD I/O
                                <1> ; (INT_16h - Retro UNIX 8086 v1 - U9.ASM, 30/06/2014)
                                <1>
                                <1> ;NOTE: 'k0' to 'k7' are name of OPMASK registers.
                                <1> ;   (The reason of using '_k' labels!!!) (27/08/2014)
                                <1> ;NOTE: 'NOT' keyword is '~' unary operator in NASM.
                                <1> ;   ('NOT LC_HC' --> '~LC_HC') (bit reversing operator)
                                <1>
                                <1> int16h:     ; 30/06/2015
                                <1> ;getc:
00000F12 9C                     <1>     pushfd  ; 28/08/2014
00000F13 0E                     <1>     push    cs
00000F14 E826000000             <1>     call    KEYBOARD_IO_1 ; getc_int
00000F19 C3                     <1>     retn
                                <1>
                                <1> ; 24/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1>     ;----- SHIFT STATUS
                                <1> _K3E:                               ; GET THE EXTENDED SHIFT STATUS FLAGS
00000F1A 8A25[FA670000]         <1>     mov     ah, [KB_FLAG_1]              ; GET SYSTEM SHIFT KEY STATUS
00000F20 80E404                 <1>     and     ah, SYS_SHIFT         ; MASK ALL BUT SYS KEY BIT
                                <1>     ;mov    cl, 5                 ; SHIFT THEW SYSTEMKEY BIT OVER TO
                                <1>     ;shl    ah, cl                ; BIT 7 POSITION
00000F23 C0E405                 <1>         shl ah, 5
00000F26 A0[FA670000]           <1>     mov     al, [KB_FLAG_1]              ; GET SYSTEM SHIFT STATES BACK
00000F2B 2473                   <1>     and     al, 01110011b         ; ELIMINATE SYS SHIFT, HOLD_STATE AND INS_SHIFT
00000F2D 08C4                   <1>     or      ah, al                  ; MERGE REMAINING BITS INTO AH
00000F2F A0[FC670000]           <1>     mov     al, [KB_FLAG_3]              ; GET RIGHT CTL AND ALT
00000F34 240C                   <1>     and     al, 00001100b         ; ELIMINATE LC_E0 AND LC_E1
00000F36 08C4                   <1>     or      ah, al                ; OR THE SHIFT FLAGS TOGETHER
                                <1> _K3:
00000F38 A0[F9670000]           <1>     mov     al, [KB_FLAG]         ; GET THE SHIFT STATUS FLAGS
                                <1>     ; 24/07/2022
00000F3D EB38                   <1>     jmp     short _KIO_EXIT              ; RETURN TO CALLER
                                <1>
                                <1> getc_int:
                                <1>     ; 28/02/2015
                                <1>     ; 03/12/2014 (derivation from pc-xt-286 bios source code -1986-,
                                <1>     ;             instead of pc-at bios - 1985-)
                                <1>     ; 28/08/2014 (_k1d)
                                <1>     ; 30/06/2014
                                <1>     ; 03/03/2014
                                <1>     ; 28/02/2014
                                <1>     ; Derived from "KEYBOARD_IO_1" procedure of IBM "pc-xt-286"
                                <1>     ; rombios source code (21/04/1986)
                                <1>     ;       'keybd.asm', INT 16H, KEYBOARD_IO
                                <1>     ;
                                <1>     ; KYBD --- 03/06/86  KEYBOARD BIOS
                                <1>     ;
                                <1>     ;--- INT 16 H ----------------------------------------------------------------
                                <1>     ; KEYBOARD I/O                                                          :
                                <1>     ;      THESE ROUTINES PROVIDE READ KEYBOARD SUPPORT                     :
                                <1>     ; INPUT                                                                 :
                                <1>     ;      (AH)= 00H  READ THE NEXT ASCII CHARACTER ENTERED FROM THE KEYBOARD,   :
                                <1>     ;                 RETURN THE RESULT IN (AL), SCAN CODE IN (AH).              :
                                <1>     ;                 THIS IS THE COMPATIBLE READ INTERFACE, EQUIVALENT TO THE   :
                                <1>     ;                  STANDARD PC OR PCAT KEYBOARD                                 :
                                <1>     ;-----------------------------------------------------------------------------:
                                <1>     ;      (AH)= 01H  SET THE ZERO FLAG TO INDICATE IF AN ASCII CHARACTER IS     :
                                <1>     ;                 AVAILABLE TO BE READ FROM THE KEYBOARD BUFFER.        :
                                <1>     ;                 (ZF)= 1 -- NO CODE AVAILABLE                                 :
                                <1>     ;                 (ZF)= 0 -- CODE IS AVAILABLE  (AX)= CHARACTER              :
                                <1>     ;                 IF (ZF)= 0, THE NEXT CHARACTER IN THE BUFFER TO BE READ IS :
                                <1>     ;                 IN (AX), AND THE ENTRY REMAINS IN THE BUFFER.              :
                                <1>     ;                 THIS WILL RETURN ONLY PC/PCAT KEYBOARD COMPATIBLE CODES    :
                                <1>     ;-----------------------------------------------------------------------------:
                                <1>     ;      (AH)= 02H  RETURN THE CURRENT SHIFT STATUS IN AL REGISTER            :
                                <1>     ;                 THE BIT SETTINGS FOR THIS CODE ARE INDICATED IN THE        :
                                <1>     ;                 EQUATES FOR @KB_FLAG                                          :
                                <1>     ;-----------------------------------------------------------------------------:
                                <1>     ;      (AH)= 03H  SET TYPAMATIC RATE AND DELAY                              :
                                <1>     ;            (AL) = 05H                                                     :
                                <1>     ;            (BL) = TYPAMATIC RATE (BITS 5 - 7 MUST BE RESET TO 0)          :
                                <1>     ;                                                                         :
                                <1>     ;                    REGISTER     RATE      REGISTER     RATE               :
                                <1>     ;                     VALUE     SELECTED     VALUE     SELECTED              :
                                <1>     ;                    ------------------------------------------            :
                                <1>     ;                    00H        30.0        10H        7.5                :
                                <1>     ;                    01H        26.7        11H        6.7                :
                                <1>     ;                    02H        24.0        12H        6.0                :
                                <1>     ;                    03H        21.8        13H        5.5                :
                                <1>     ;                    04H        20.0        14H        5.0                :
                                <1>     ;                    05H        18.5        15H        4.6                :
                                <1>     ;                    06H        17.1        16H        4.3                :
                                <1>     ;                    07H        16.0        17H        4.0                :
                                <1>     ;                    08H        15.0        18H        3.7                :
                                <1>     ;                    09H        13.3        19H        3.3                :
                                <1>     ;                    0AH        12.0        1AH        3.0                :
                                <1>     ;                    0BH        10.9        1BH        2.7                :
                                <1>         ;                0CH        10.0        1CH        2.5                :
                                <1>     ;                    0DH         9.2        1DH        2.3                :
                                <1>     ;                    0EH         8.6        1EH        2.1                :
                                <1>     ;                    0FH         8.0        1FH        2.0                :
                                <1>     ;                                                                         :
                                <1>     ;            (BH) = TYPAMATIC DELAY  (BITS 2 - 7 MUST BE RESET TO 0)        :
                                <1>     ;                                                                         :
                                <1>     ;                    REGISTER     DELAY                                    :
```

```
                                <1>     ;                    VALUE       VALUE                                  :
                                <1>     ;                   ------------------                                  :
                                <1>     ;                   00H        250 ms                                 :
                                <1>     ;                   01H        500 ms                                 :
                                <1>     ;                   02H        750 ms                                 :
                                <1>     ;                   03H       1000 ms                                 :
                                <1>     ;-----------------------------------------------------------------------:
                                <1>     ;      (AH)= 05H  PLACE ASCII CHARACTER/SCAN CODE COMBINATION IN KEYBOARD    :
                                <1>     ;                 BUFFER AS IF STRUCK FROM KEYBOARD                       :
                                <1>     ;                 ENTRY:  (CL) = ASCII CHARACTER                        :
                                <1>     ;                         (CH) = SCAN CODE                                :
                                <1>     ;                 EXIT:   (AH) = 00H = SUCCESSFUL OPERATION               :
                                <1>     ;                         (AL) = 01H = UNSUCCESSFUL - BUFFER FULL         :
                                <1>     ;                 FLAGS:  CARRY IF ERROR                                  :
                                <1>     ;-----------------------------------------------------------------------:
                                <1>     ;      (AH)= 10H  EXTENDED READ INTERFACE FOR THE ENHANCED KEYBOARD,         :
                                <1>     ;                 OTHERWISE SAME AS FUNCTION AH=0                         :
                                <1>     ;-----------------------------------------------------------------------:
                                <1>     ;      (AH)= 11H  EXTENDED ASCII STATUS FOR THE ENHANCED KEYBOARD,           :
                                <1>     ;                 OTHERWISE SAME AS FUNCTION AH=1                         :
                                <1>     ;-----------------------------------------------------------------------:
                                <1>     ;      (AH)= 12H  RETURN THE EXTENDED SHIFT STATUS IN AX REGISTER            :
                                <1>     ;                 AL = BITS FROM KB_FLAG, AH = BITS FOR LEFT AND RIGHT       :
                                <1>     ;                 CTL AND ALT KEYS FROM KB_FLAG_1 AND KB_FLAG_3             :
                                <1>     ; OUTPUT                                                                  :
                                <1>     ;      AS NOTED ABOVE, ONLY (AX) AND FLAGS CHANGED                         :
                                <1>     ;      ALL REGISTERS RETAINED                                               :
                                <1>     ;-----------------------------------------------------------------------
                                <1>
                                <1> ; 07/08/2022
                                <1> ; 24/07/2022 - TRDOS 386 v2.0.5
                                <1> ; 12/04/2021 - TRDOS 386 v2.0.3 (32 bit push/pop)
                                <1> ; 15/01/2017
                                <1> ; 14/01/2017
                                <1> ; 02/01/2017
                                <1> ; 29/05/2016
                                <1> ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> int32h:  ; Keyboard BIOS
                                <1>
                                <1> KEYBOARD_IO_1:
                                <1>     ;sti                        ; INTERRUPTS BACK ON
                                <1>     ; 29/05/2016
00000F3F 80642408BE             <1>         and     byte [esp+8], 10111110b ; clear zero flag and cary flag
                                <1>     ;
00000F44 1E                     <1>     push    ds                  ; SAVE CURRENT DS
00000F45 53                     <1>     push    ebx                 ; SAVE BX TEMPORARILY
                                <1>     ;push   ecx                 ; SAVE CX TEMPORARILY
00000F46 66BB1000               <1>         mov     bx, KDATA
00000F4A 8EDB                   <1>     mov     ds, bx              ; PUT SEGMENT VALUE OF DATA AREA INTO DS
                                <1>     ; 14/01/2017
00000F4C 8B1C24                 <1>     mov     ebx, [esp]
                                <1>     ;; 15/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;;mov   byte [intflg], 32h  ; keyboard interrupt
00000F4F FB                     <1>     sti
                                <1>     ;
00000F50 08E4                   <1>     or      ah, ah              ; CHECK FOR (AH)= 00H
00000F52 7433                   <1>     jz      short _K1           ; ASCII_READ
00000F54 FECC                   <1>     dec     ah                    ; CHECK FOR (AH)= 01H
00000F56 744C                   <1>         jz      short _K2             ; ASCII_STATUS
00000F58 FECC                   <1>     dec     ah                  ; CHECK FOR (AH)= 02H
00000F5A 74DC                   <1>         jz  short _K3           ; SHIFT STATUS
00000F5C FECC                   <1>     dec     ah                  ; CHECK FOR (AH)= 03H
00000F5E 746F                   <1>         jz      short _K300           ; SET TYPAMATIC RATE/DELAY
00000F60 80EC02                 <1>     sub     ah, 2               ; CHECK FOR (AH)= 05H
                                <1>         ;jz short _K500         ; KEYBOARD WRITE
                                <1>     ; 07/08/2022
00000F63 7505                   <1>     jnz     short _KIO1
00000F65 E988000000             <1>     jmp     _K500
                                <1> _KIO1:
00000F6A 80EC0B                 <1>     sub     ah, 11              ; AH =  10H
00000F6D 740C                   <1>     jz      short _K1E          ; EXTENDED ASCII READ
00000F6F FECC                   <1>     dec     ah                  ; CHECK FOR (AH)= 11H
00000F71 7422                   <1>     jz      short _K2E          ; EXTENDED_ASCII_STATUS
00000F73 FECC                   <1>     dec     ah                  ; CHECK FOR (AH)= 12H
00000F75 74A3                   <1>     jz      short _K3E          ; EXTENDED_SHIFT_STATUS
                                <1> _KIO_EXIT:
                                <1>     ; 02/01/2017
00000F77 FA                     <1>     cli
                                <1>     ;;mov   byte [intflg], 0 ;; 15/01/2017
                                <1>     ;
                                <1>     ;pop    ecx                 ; RECOVER REGISTER
00000F78 5B                     <1>     pop     ebx                 ; RECOVER REGISTER
00000F79 1F                     <1>     pop     ds                  ; RECOVER SEGMENT
00000F7A CF                     <1>     iretd                       ; INVALID COMMAND, EXIT
                                <1>
                                <1> ; 24/07/2022
                                <1> ;
                                <1> ;   ;----- SHIFT STATUS
                                <1> ;_K3E:                                  ; GET THE EXTENDED SHIFT STATUS FLAGS
                                <1> ;   mov     ah, [KB_FLAG_1]             ; GET SYSTEM SHIFT KEY STATUS
                                <1> ;   and     ah, SYS_SHIFT       ; MASK ALL BUT SYS KEY BIT
                                <1> ;   ;mov    cl, 5               ; SHIFT THEW SYSTEMKEY BIT OVER TO
                                <1> ;   ;shl    ah, cl              ; BIT 7 POSITION
                                <1> ;       shl ah, 5
                                <1> ;   mov     al, [KB_FLAG_1]             ; GET SYSTEM SHIFT STATES BACK
                                <1> ;   and     al, 01110011b       ; ELIMINATE SYS SHIFT, HOLD_STATE AND INS_SHIFT
                                <1> ;   or      ah, al                ; MERGE REMAINING BITS INTO AH
                                <1> ;   mov     al, [KB_FLAG_3]             ; GET RIGHT CTL AND ALT
                                <1> ;   and     al, 00001100b       ; ELIMINATE LC_E0 AND LC_E1
                                <1> ;   or      ah, al              ; OR THE SHIFT FLAGS TOGETHER
                                <1> ;_K3:
                                <1> ;   mov     al, [KB_FLAG]       ; GET THE SHIFT STATUS FLAGS
                                <1> ;   ; 24/07/2022
                                <1> ;   jmp     short _KIO_EXIT             ; RETURN TO CALLER
                                <1>
                                <1>     ;----- ASCII CHARACTER
                                <1> _K1E:
00000F7B E89F000000             <1>     call    _K1S                ; GET A CHARACTER FROM THE BUFFER (EXTENDED)
00000F80 E812010000             <1>     call    _KIO_E_XLAT         ; ROUTINE TO XLATE FOR EXTENDED CALLS
00000F85 EBF0                   <1>     jmp     short _KIO_EXIT       ; GIVE IT TO THE CALLER
                                <1> _K1:
00000F87 E893000000             <1>     call    _K1S                ; GET A CHARACTER FROM THE BUFFER
00000F8C E811010000             <1>     call    _KIO_S_XLAT         ; ROUTINE TO XLATE FOR STANDARD CALLS
00000F91 72F4                   <1>     jc      short _K1           ; CARRY SET MEANS TROW CODE AWAY
                                <1> _K1A:
00000F93 EBE2                   <1>     jmp     short _KIO_EXIT       ; RETURN TO CALLER
                                <1>
                                <1>     ;----- ASCII STATUS
                                <1> _K2E:
00000F95 E8D0000000             <1>     call    _K2S                ; TEST FOR CHARACTER IN BUFFER (EXTENDED)
00000F9A 7420                   <1>     jz      short _K2B          ; RETURN IF BUFFER EMPTY
00000F9C 9C                     <1>     pushf                       ; SAVE ZF FROM TEST
00000F9D E8F5000000             <1>     call    _KIO_E_XLAT         ; ROUTINE TO XLATE FOR EXTENDED CALLS
```

```
00000FA2 EB17                <1>     jmp    short _K2A             ; GIVE IT TO THE CALLER
                             <1> _K2:
00000FA4 E8C1000000          <1>     call   _K2S                   ; TEST FOR CHARACTER IN BUFFER
00000FA9 7411                <1>     jz     short _K2B             ; RETURN IF BUFFER EMPTY
00000FAB 9C                  <1>     pushf                         ; SAVE ZF FROM TEST
00000FAC E8F1000000          <1>     call   _KIO_S_XLAT            ; ROUTINE TO XLATE FOR STANDARD CALLS
00000FB1 7308                <1>     jnc    short _K2A             ; CARRY CLEAR MEANS PASS VALID CODE
00000FB3 9D                  <1>     popf                          ; INVALID CODE FOR THIS TYPE OF CALL
00000FB4 E866000000          <1>     call   _K1S                   ; THROW THE CHARACTER AWAY
00000FB9 EBE9                <1>     jmp    short _K2              ; GO LOOK FOR NEXT CHAR, IF ANY
                             <1> _K2A:
00000FBB 9D                  <1>     popf                          ; RESTORE ZF FROM TEST
                             <1> _K2B:
                             <1>     ; 02/01/2017
00000FBC FA                  <1>     cli
                             <1>     ;; mov  byte [intflg], 0 ;; 15/01/2017
                             <1>     ;
                             <1>     ;pop   ecx                    ; RECOVER REGISTER
00000FBD 5B                  <1>     pop    ebx                    ; RECOVER REGISTER
00000FBE 1F                  <1>     pop    ds                     ; RECOVER SEGMENT
                             <1>     ; (*) 29/05/2016
                             <1>     ; (*) retf 4                  ; THROW AWAY (e)FLAGS
00000FBF 7208                <1>     jc     short _k2d
00000FC1 7505                <1>     jnz    short _k2c
00000FC3 804C240840          <1>     or     byte [esp+8], 01000000b      ; set zero flag bit of eflags register
                             <1> _k2c:
00000FC8 CF                  <1>     iretd
                             <1> _k2d:
                             <1>     ; 29/05/2016 -set carry flag on stack-
                             <1>     ; [esp] = EIP
                             <1>     ; [esp+4] = CS
                             <1>     ; [esp+8] = E-FLAGS
00000FC9 804C240801          <1>     or     byte [esp+8], 1  ; set carry bit of eflags register
                             <1>     ; [esp+12] = ESP (user)
                             <1>     ; [esp+16] = SS (User)
00000FCE CF                  <1>     iretd
                             <1>
                             <1>     ; (*) 29/05/2016 - 'retf 4' intruction causes to stack fault
                             <1>     ; (OUTER-PRIVILEGE-LEVEL)
                             <1>     ; INTEL 80386 PROGRAMMER'S REFERENCE MANUAL 1986
                             <1>     ; // RETF instruction:
                             <1>     ;
                             <1>     ; IF OperandMode=32 THEN
                             <1>     ;    Load CS:EIP from stack;
                             <1>     ;    Set CS RPL to CPL;
                             <1>     ;    Increment ESP by 8 plus the immediate offset if it exists;
                             <1>     ;    Load SS:ESP from stack;
                             <1>     ; ELSE (* OperandMode=16 *)
                             <1>     ;    Load CS:IP from stack;
                             <1>     ;    Set CS RPL to CPL;
                             <1>     ;    Increment SP by 4 plus the immediate offset if it exists;
                             <1>     ;    Load SS:SP from stack;
                             <1>     ; FI;
                             <1>     ;
                             <1>     ; //
                             <1>
                             <1>     ; 24/07/2022
                             <1>     ;----- SET TYPAMATIC RATE AND DELAY
                             <1> _K300:
00000FCF 3C05                <1>     cmp    al, 5                  ; CORRECT FUNCTION CALL?
00000FD1 75A4                <1>     jne    short _KIO_EXIT              ; NO, RETURN
00000FD3 F6C3E0              <1>     test   bl, 0E0h               ; TEST FOR OUT-OF-RANGE RATE
00000FD6 759F                <1>     jnz    short _KIO_EXIT              ; RETURN IF SO
00000FD8 F6C7FC              <1>     test   bh, 0FCh               ; TEST FOR OUT-OF-RANGE DELAY
00000FDB 759A                <1>     jnz    short _KIO_EXIT              ; RETURN IF SO
00000FDD B0F3                <1>     mov    al, KB_TYPA_RD         ; COMMAND FOR TYPAMATIC RATE/DELAY
00000FDF E875060000          <1>     call   SND_DATA               ; SEND TO KEYBOARD
                             <1>     ;mov   cx, 5                  ; SHIFT COUNT
                             <1>     ;shl   bh, cl                 ; SHIFT DELAY OVER
00000FE4 C0E705              <1>     shl    bh, 5
00000FE7 88D8                <1>     mov    al, bl                 ; PUT IN RATE
00000FE9 08F8                <1>     or     al, bh                 ; AND DELAY
00000FEB E869060000          <1>     call   SND_DATA               ; SEND TO KEYBOARD
00000FF0 EB85                <1>         jmp     _KIO_EXIT               ; RETURN TO CALLER
                             <1>
                             <1>     ;----- WRITE TO KEYBOARD BUFFER
                             <1> _K500:
00000FF2 56                  <1>     push   esi                    ; SAVE SI (esi)
00000FF3 FA                  <1>     cli                           ;
00000FF4 8B1D[0A680000]      <1>            mov    ebx, [BUFFER_TAIL]    ; GET THE 'IN TO' POINTER TO THE BUFFER
00000FFA 89DE                <1>     mov    esi, ebx               ; SAVE A COPY IN CASE BUFFER NOT FULL
00000FFC E8D1000000          <1>     call   _K4                    ; BUMP THE POINTER TO SEE IF BUFFER IS FULL
00001001 3B1D[06680000]      <1>     cmp    ebx, [BUFFER_HEAD]     ; WILL THE BUFFER OVERRUN IF WE STORE THIS?
00001007 740D                <1>     je     short _K502            ; YES - INFORM CALLER OF ERROR
00001009 66890E              <1>     mov    [esi], cx              ; NO - PUT ASCII/SCAN CODE INTO BUFFER
0000100C 891D[0A680000]      <1>     mov    [BUFFER_TAIL], ebx     ; ADJUST 'IN TO' POINTER TO REFLECT CHANGE
00001012 28C0                <1>     sub    al, al                 ; TELL CALLER THAT OPERATION WAS SUCCESSFUL
00001014 EB02                <1>     jmp    short _K504            ; SUB INSTRUCTION ALSO RESETS CARRY FLAG
                             <1> _K502:
00001016 B001                <1>     mov    al, 01h                ; BUFFER FULL INDICATION
                             <1> _K504:
00001018 FB                  <1>     sti
00001019 5E                  <1>     pop    esi                    ; RECOVER SI (esi)
0000101A E958FFFFFF          <1>         jmp     _KIO_EXIT               ; RETURN TO CALLER WITH STATUS IN AL
                             <1>
                             <1>     ;----- READ THE KEY TO FIGURE OUT WHAT TO DO -----
                             <1> _K1S:
0000101F FA                  <1>     cli    ; 03/12/2014
00001020 8B1D[06680000]      <1>         mov     ebx, [BUFFER_HEAD]       ; GET POINTER TO HEAD OF BUFFER
00001026 3B1D[0A680000]      <1>         cmp     ebx, [BUFFER_TAIL]       ; TEST END OF BUFFER
                             <1>     ;jne   short _K1U             ; IF ANYTHING IN BUFFER SKIP INTERRUPT
0000102C 750F                <1>     jne    short _k1x ; 03/12/2014
                             <1>     ;
                             <1>     ; 03/12/2014
                             <1>     ; 28/08/2014
                             <1>     ; PERFORM OTHER FUNCTION ?? here !
                             <1>     ;; MOV  AX, 9002h             ; MOVE IN WAIT CODE & TYPE
                             <1>     ;; INT  15H                   ; PERFORM OTHER FUNCTION
                             <1> _K1T:                                  ; ASCII READ
0000102E FB                  <1>     sti                           ; INTERRUPTS BACK ON DURING LOOP
0000102F 90                  <1>     nop                           ; ALLOW AN INTERRUPT TO OCCUR
                             <1> _K1U:
00001030 FA                  <1>     cli                           ; INTERRUPTS BACK OFF
00001031 8B1D[06680000]      <1>         mov        ebx, [BUFFER_HEAD]    ; GET POINTER TO HEAD OF BUFFER
00001037 3B1D[0A680000]      <1>         cmp     ebx, [BUFFER_TAIL]       ; TEST END OF BUFFER
                             <1> _k1x:
0000103D 53                  <1>     push   ebx                    ; SAVE ADDRESS
0000103E 9C                  <1>     pushf                         ; SAVE FLAGS
0000103F E8C9060000          <1>     call   MAKE_LED               ; GO GET MODE INDICATOR DATA BYTE
00001044 8A1D[FB670000]      <1>     mov    bl, [KB_FLAG_2]        ; GET PREVIOUS BITS
0000104A 30C3                <1>     xor    bl, al                 ; SEE IF ANY DIFFERENT
0000104C 80E307              <1>     and    bl, 07h ; KB_LEDS      ; ISOLATE INDICATOR BITS
0000104F 7406                <1>     jz     short _K1V             ; IF NO CHANGE BYPASS UPDATE
00001051 E863060000          <1>     call   SND_LED1
```

```
00001056 FA                 <1>     cli                         ; DISABLE INTERRUPTS
                            <1> _K1V:
00001057 9D                 <1>     popf                        ; RESTORE FLAGS
00001058 5B                 <1>     pop     ebx                 ; RESTORE ADDRESS
00001059 74D3               <1>         je      short _K1T              ; LOOP UNTIL SOMETHING IN BUFFER
                            <1>     ;
0000105B 668B03             <1>     mov     ax, [ebx]           ; GET SCAN CODE AND ASCII CODE
0000105E E86F000000         <1>         call    _K4                     ; MOVE POINTER TO NEXT POSITION
00001063 891D[06680000]     <1>         mov     [BUFFER_HEAD], ebx      ; STORE VALUE IN VARIABLE
00001069 C3                 <1>     retn                        ; RETURN
                            <1>
                            <1>     ;----- READ THE KEY TO SEE IF ONE IS PRESENT -----
                            <1> _K2S:
0000106A FA                 <1>     cli                         ; INTERRUPTS OFF
0000106B 8B1D[06680000]     <1>         mov     ebx, [BUFFER_HEAD]      ; GET HEAD POINTER
00001071 3B1D[0A680000]     <1>         cmp     ebx, [BUFFER_TAIL]      ; IF EQUAL (Z=1) THEN NOTHING THERE
00001077 668B03             <1>     mov     ax, [ebx]
0000107A 9C                 <1>     pushf                       ; SAVE FLAGS
                            <1>     ;push   ax                  ; SAVE CODE
                            <1>     ; 12/04/2021
0000107B 50                 <1>     push    eax
0000107C E88C060000         <1>     call    MAKE_LED            ; GO GET MODE INDICATOR DATA BYTE
00001081 8A1D[FB670000]     <1>     mov     bl, [KB_FLAG_2]     ; GET PREVIOUS BITS
00001087 30C3               <1>     xor     bl, al              ; SEE IF ANY DIFFERENT
00001089 80E307             <1>     and     bl, 07h ; KB_LEDS   ; ISOLATE INDICATOR BITS
0000108C 7405               <1>     jz      short _K2T          ; IF NO CHANGE BYPASS UPDATE
0000108E E80F060000         <1>     call    SND_LED             ; GO TURN ON MODE INDICATORS
                            <1> _K2T:
                            <1>     ;pop    ax                  ; RESTORE CODE
                            <1>     ; 12/04/2021
00001093 58                 <1>     pop     eax
00001094 9D                 <1>     popf                        ; RESTORE FLAGS
00001095 FB                 <1>     sti                         ; INTERRUPTS BACK ON
00001096 C3                 <1>     retn                        ; RETURN
                            <1>
                            <1>     ;----- ROUTINE TO TRANSLATE SCAN CODE PAIRS FOR EXTENDED CALLS -----
                            <1> _KIO_E_XLAT:
00001097 3CF0               <1>     cmp     al, 0F0h            ; IS IT ONE OF THE FILL-INs?
00001099 7506               <1>     jne     short _KIO_E_RET    ; NO, PASS IT ON
0000109B 08E4               <1>         or  ah, ah              ; AH = 0 IS SPECIAL CASE
0000109D 7402               <1>         jz  short _KIO_E_RET        ; PASS THIS ON UNCHANGED
0000109F 30C0               <1>     xor     al, al              ; OTHERWISE SET AL = 0
                            <1> _KIO_E_RET:
000010A1 C3                 <1>     retn                        ; GO BACK
                            <1>
                            <1>     ;----- ROUTINE TO TRANSLATE SCAN CODE PAIRS FOR STANDARD CALLS -----
                            <1> _KIO_S_XLAT:
000010A2 80FCE0             <1>     cmp     ah, 0E0h            ; IS IT KEYPAD ENTER OR / ?
000010A5 750F               <1>     jne     short _KIO_S2       ; NO, CONTINUE
000010A7 3C0D               <1>     cmp     al, 0Dh             ; KEYPAD ENTER CODE?
000010A9 7408               <1>         je  short _KIO_S1       ; YES, MASSAGE A BIT
000010AB 3C0A               <1>     cmp     al, 0Ah             ; CTRL KEYPAD ENTER CODE?
000010AD 7404               <1>         je  short _KIO_S1       ; YES, MASSAGE THE SAME
000010AF B435               <1>     mov     ah, 35h             ; NO, MUST BE KEYPAD /
                            <1> _kio_ret: ; 03/12/2014
000010B1 F8                 <1>     clc
000010B2 C3                 <1>     retn
                            <1>     ;jmp    short _KIO_USE      ; GIVE TO CALLER
                            <1> _KIO_S1:
000010B3 B41C               <1>     mov     ah, 1Ch             ; CONVERT TO COMPATIBLE OUTPUT
                            <1>     ;jmp    short _KIO_USE      ; GIVE TO CALLER
000010B5 C3                 <1>     retn
                            <1> _KIO_S2:
000010B6 80FC84             <1>     cmp     ah, 84h             ; IS IT ONE OF EXTENDED ONES?
000010B9 7715               <1>     ja      short _KIO_DIS      ; YES, THROW AWAY AND GET ANOTHER CHAR
000010BB 3CF0               <1>     cmp     al, 0F0h            ; IS IT ONE OF THE FILL-INs?
000010BD 7506               <1>         jne short _KIO_S3       ; NO, TRY LAST TEST
000010BF 08E4               <1>     or      ah, ah              ; AH = 0 IS SPECIAL CASE
000010C1 740C               <1>         jz  short _KIO_USE      ; PASS THIS ON UNCHANGED
000010C3 EB0B               <1>     jmp     short _KIO_DIS      ; THROW AWAY THE REST
                            <1> _KIO_S3:
000010C5 3CE0               <1>     cmp     al, 0E0h            ; IS IT AN EXTENSION OF A PREVIOUS ONE?
                            <1>     ;jne    short _KIO_USE      ; NO, MUST BE A STANDARD CODE
000010C7 75E8               <1>     jne     short _kio_ret
000010C9 08E4               <1>     or      ah, ah              ; AH = 0 IS SPECIAL CASE
000010CB 7402               <1>         jz  short _KIO_USE      ; JUMP IF AH = 0
000010CD 30C0               <1>     xor     al, al              ; CONVERT TO COMPATIBLE OUTPUT
                            <1>     ;jmp    short _KIO_USE      ; PASS IT ON TO CALLER
                            <1> _KIO_USE:
                            <1>     ;clc                        ; CLEAR CARRY TO INDICATE GOOD CODE
000010CF C3                 <1>     retn                        ; RETURN
                            <1> _KIO_DIS:
000010D0 F9                 <1>     stc                         ; SET CARRY TO INDICATE DISCARD CODE
000010D1 C3                 <1>     retn                        ; RETURN
                            <1>
                            <1>     ;----- INCREMENT BUFFER POINTER ROUTINE -----
                            <1> _K4:
000010D2 43                 <1>     inc     ebx
000010D3 43                 <1>     inc     ebx                 ; MOVE TO NEXT WORD IN LIST
000010D4 3B1D[02680000]     <1>         cmp     ebx, [BUFFER_END]       ; AT END OF BUFFER?
                            <1>         ;jne    short _K5               ; NO, CONTINUE
000010DA 7206               <1>     jb      short _K5
000010DC 8B1D[FE670000]     <1>         mov     ebx, [BUFFER_START]     ; YES, RESET TO BUFFER BEGINNING
                            <1> _K5:
000010E2 C3                 <1>     retn
                            <1>
                            <1> ; 20/02/2015
                            <1> ; 05/12/2014
                            <1> ; 26/08/2014
                            <1> ; KEYBOARD (HARDWARE) INTERRUPT -  IRQ LEVEL 1
                            <1> ; (INT_09h - Retro UNIX 8086 v1 - U9.ASM, 07/03/2014)
                            <1> ;
                            <1> ; Derived from "KB_INT_1" procedure of IBM "pc-at"
                            <1> ; rombios source code (06/10/1985)
                            <1> ; 'keybd.asm', HARDWARE INT 09h - (IRQ Level 1)
                            <1>
                            <1> ; EQUATES (IBM PC-XT-286 BIOS, 1986, 'POSQEQU.INC')
                            <1>
                            <1> ;--------- 8042 COMMANDS -------------------------------------------------------
                            <1> ENA_KBD             equ     0AEh            ; ENABLE KEYBOARD COMMAND
                            <1> DIS_KBD             equ     0ADh            ; DISABLE KEYBOARD COMMAND
                            <1> SHUT_CMD    equ     0FEh            ; CAUSE A SHUTDOWN COMMAND
                            <1> ;--------- 8042 KEYBOARD INTERFACE AND DIAGNOSTIC CONTROL REGISTERS ------------
                            <1> STATUS_PORT equ     064h            ; 8042 STATUS PORT
                            <1> INPT_BUF_FULL       equ     00000010b       ; 1 = +INPUT BUFFER FULL
                            <1> PORT_A              equ     060h            ; 8042 KEYBOARD SCAN CODE/CONTROL PORT
                            <1> ;---------- 8042 KEYBOARD RESPONSE ----------------------------------------------
                            <1> KB_ACK              equ     0FAh            ; ACKNOWLEDGE PROM TRANSMISSION
                            <1> KB_RESEND   equ     0FEh            ; RESEND REQUEST
                            <1> KB_OVER_RUN equ     0FFh            ; OVER RUN SCAN CODE
                            <1> ;---------- KEYBOARD/LED COMMANDS -----------------------------------------------
                            <1> KB_ENABLE   equ     0F4h            ; KEYBOARD ENABLE
                            <1> LED_CMD             equ     0EDh            ; LED WRITE COMMAND
                            <1> KB_TYPA_RD  equ     0F3h            ; TYPAMATIC RATE/DELAY COMMAND
```

```
                        <1> ;---------- KEYBOARD SCAN CODES ----------------------------------------------
                        <1> NUM_KEY            equ    69             ; SCAN CODE FOR        NUMBER LOCK KEY
                        <1> SCROLL_KEY  equ    70             ; SCAN CODE FOR        SCROLL LOCK KEY
                        <1> ALT_KEY            equ    56             ; SCAN CODE FOR        ALTERNATE SHIFT KEY
                        <1> CTL_KEY            equ    29             ; SCAN CODE FOR        CONTROL KEY
                        <1> CAPS_KEY    equ    58             ; SCAN CODE FOR        SHIFT LOCK KEY
                        <1> DEL_KEY            equ    83             ; SCAN CODE FOR        DELETE KEY
                        <1> INS_KEY            equ    82             ; SCAN CODE FOR        INSERT KEY
                        <1> LEFT_KEY    equ    42             ; SCAN CODE FOR        LEFT SHIFT
                        <1> RIGHT_KEY   equ    54             ; SCAN CODE FOR        RIGHT SHIFT
                        <1> SYS_KEY            equ    84             ; SCAN CODE FOR        SYSTEM KEY
                        <1> ;---------- ENHANCED KEYBOARD SCAN CODES -------------------------------------
                        <1> ID_1        equ    0ABh           ; 1ST ID CHARACTER FOR KBX
                        <1> ID_2        equ    041h           ; 2ND ID CHARACTER FOR KBX
                        <1> ID_2A              equ    054h           ; ALTERNATE 2ND ID CHARACTER FOR KBX
                        <1> F11_M              equ    87             ; F11 KEY MAKE
                        <1> F12_M              equ    88             ; F12 KEY MAKE
                        <1> MC_E0              equ    224            ; GENERAL MARKER CODE
                        <1> MC_E1              equ    225            ; PAUSE KEY MARKER CODE
                        <1> ;---------- FLAG EQUATES WITHIN @KB_FLAG-------------------------------------
                        <1> RIGHT_SHIFT equ    00000001b      ; RIGHT SHIFT KEY DEPRESSED
                        <1> LEFT_SHIFT  equ    00000010b      ; LEFT SHIFT KEY DEPRESSED
                        <1> CTL_SHIFT   equ    00000100b      ; CONTROL SHIFT KEY DEPRESSED
                        <1> ALT_SHIFT   equ    00001000b      ; ALTERNATE SHIFT KEY DEPRESSED
                        <1> SCROLL_STATE       equ    00010000b      ; SCROLL LOCK STATE IS ACTIVE
                        <1> NUM_STATE   equ    00100000b      ; NUM LOCK STATE IS ACTIVE
                        <1> CAPS_STATE  equ    01000000b      ; CAPS LOCK STATE IS ACTIVE
                        <1> INS_STATE   equ    10000000b      ; INSERT STATE IS ACTIVE
                        <1> ;---------- FLAG EQUATES WITHIN   @KB_FLAG_1 -------------------------------------
                        <1> L_CTL_SHIFT equ    00000001b      ; LEFT CTL KEY DOWN
                        <1> L_ALT_SHIFT equ    00000010b      ; LEFT ALT KEY DOWN
                        <1> SYS_SHIFT   equ    00000100b      ; SYSTEM KEY DEPRESSED AND HELD
                        <1> HOLD_STATE  equ    00001000b      ; SUSPEND KEY HAS BEEN TOGGLED
                        <1> SCROLL_SHIFT       equ    00010000b      ; SCROLL LOCK KEY IS DEPRESSED
                        <1> NUM_SHIFT   equ    00100000b      ; NUM LOCK KEY IS DEPRESSED
                        <1> CAPS_SHIFT  equ    01000000b      ; CAPS LOCK KEY IS DEPRE55ED
                        <1> INS_SHIFT   equ    10000000b      ; INSERT KEY IS DEPRESSED
                        <1> ;---------- FLAGS EQUATES WITHIN @KB_FLAG_2 ----------------------------------
                        <1> KB_LEDS            equ    00000111b      ; KEYBOARD LED STATE BITS
                        <1> ;          equ    00000001b      ; SCROLL LOCK INDICATOR
                        <1> ;          equ    00000010b      ; NUM LOCK INDICATOR
                        <1> ;          equ    00000100b      ; CAPS LOCK INDICATOR
                        <1> ;          equ    00001000b      ; RESERVED (MUST BE ZERO)
                        <1> KB_FA              equ    00010000b      ; ACKNOWLEDGMENT RECEIVED
                        <1> KB_FE              equ    00100000b      ; RESEND RECEIVED FLAG
                        <1> KB_PR_LED   equ    01000000b      ; MODE INDICATOR UPDATE
                        <1> KB_ERR             equ    10000000b      ; KEYBOARD TRANSMIT ERROR FLAG
                        <1> ;---------- FLAGS EQUATES WITHIN @KB_FLAG_3 ----------------------------------
                        <1> LC_E1              equ    00000001b      ; LAST CODE WAS THE E1 HIDDEN CODE
                        <1> LC_E0              equ    00000010b      ; LAST CODE WAS THE E0 HIDDEN CODE
                        <1> R_CTL_SHIFT equ    00000100b      ; RIGHT CTL KEY DOWN
                        <1> R_ALT_SHIFT equ    00001000b      ; RIGHT ALT KEY DOWN
                        <1> GRAPH_ON    equ    00001000b      ; ALT GRAPHICS KEY DOWN (WT ONLY)
                        <1> KBX         equ    00010000b      ; ENHANCED KEYBOARD INSTALLED
                        <1> SET_NUM_LK  equ    00100000b      ; FORCE NUM LOCK IF READ ID AND KBX
                        <1> LC_AB              equ    01000000b      ; LAST CHARACTER WAS FIRST ID CHARACTER
                        <1> RD_ID              equ    10000000b      ; DOING A READ ID (MUST BE BIT0)
                        <1> ;
                        <1> ;---------- INTERRUPT EQUATES ------------------------------------------------
                        <1> EOI         equ    020h           ; END OF INTERRUPT COMMAND TO 8259
                        <1> INTA00             equ    020h           ; 8259 PORT
                        <1>
                        <1>
                        <1> kb_int:
                        <1>
                        <1> ; 24/07/2022 - TRDOS 386 v2.0.5
                        <1> ; 12/04/2021 - TRDOS 386 v2.0.3 (32 bit push/pop)
                        <1> ; 17/10/2015 ('ctrlbrk')
                        <1> ; 05/12/2014
                        <1> ; 04/12/2014 (derived from pc-xt-286 bios source code -1986-)
                        <1> ; 26/08/2014
                        <1> ;
                        <1> ; 03/06/86  KEYBOARD BIOS
                        <1> ;
                        <1> ;--- HARDWARE INT 09H -- (IRQ LEVEL 1) ----------------------------------------
                        <1> ;                                                                        ;
                        <1> ;   KEYBOARD INTERRUPT ROUTINE                                           ;
                        <1> ;                                                                        ;
                        <1> ;------------------------------------------------------------------------
                        <1>
                        <1> KB_INT_1:
000010E3 FB             <1>     sti                            ; ENABLE INTERRUPTS
                        <1>     ;push   ebp
000010E4 50             <1>     push    eax
000010E5 53             <1>     push    ebx
000010E6 51             <1>     push    ecx
000010E7 52             <1>     push    edx
000010E8 56             <1>     push    esi
000010E9 57             <1>     push    edi
000010EA 1E             <1>     push    ds
000010EB 06             <1>     push    es
000010EC FC             <1>     cld                            ; FORWARD DIRECTION
000010ED 66B81000       <1>     mov     ax, KDATA
000010F1 8ED8           <1>     mov     ds, ax
000010F3 8EC0           <1>     mov     es, ax
                        <1>     ;
                        <1>     ;-----  WAIT FOR KEYBOARD DISABLE COMMAND TO BE ACCEPTED
000010F5 B0AD           <1>     mov     al, DIS_KBD            ; DISABLE THE KEYBOARD COMMAND
000010F7 E84B050000     <1>     call    SHIP_IT                ; EXECUTE DISABLE
000010FC FA             <1>     cli                            ; DISABLE INTERRUPTS
000010FD B900000100     <1>     mov     ecx, 10000h            ; SET MAXIMUM TIMEOUT
                        <1> KB_INT_01:
00001102 E464           <1>     in      al, STATUS_PORT                ; READ ADAPTER STATUS
00001104 A802           <1>     test    al, INPT_BUF_FULL      ; CHECK INPUT BUFFER FULL STATUS BIT
00001106 E0FA           <1>     loopnz  KB_INT_01              ; WAIT FOR COMMAND TO BE ACCEPTED
                        <1>     ;
                        <1>     ;-----  READ CHARACTER FROM KEYBOARD INTERFACE
00001108 E460           <1>     in      al, PORT_A             ; READ IN THE CHARACTER
                        <1>     ;
                        <1>     ;-----  SYSTEM HOOK INT 15H - FUNCTION 4FH (ON HARDWARE INT LEVEL 9H)
                        <1>     ;mov    ah, 04Fh               ; SYSTEM INTERCEPT - KEY CODE FUNCTION
                        <1>     ;stc                           ; SET CY=1 (IN CASE OF IRET)
                        <1>     ;int    15h                    ; CASETTE CALL (AL)=KEY SCAN CODE
                        <1>     ;                              ; RETURNS CY=1 FOR INVALID FUNCTION
                        <1>     ;jc     KB_INT_02              ; CONTINUE IF CARRY FLAG SET ((AL)=CODE)
                        <1>     ;jmp    K26                    ; EXIT IF SYSTEM HANDLES SCAN CODE
                        <1>     ;                              ; EXİT HANDLES HARDWARE EOI AND ENABLE
                        <1>     ;
                        <1>     ;-----  CHECK FOR A RESEND COMMAND TO KEYBOARD
                        <1> KB_INT_02:                         ;         (AL)= SCAN CODE
0000110A FB             <1>     sti                            ; ENABLE INTERRUPTS AGAIN
0000110B 3CFE           <1>     cmp     al, KB_RESEND          ; IS THE INPUT A RESEND
0000110D 740E           <1>         je      short KB_INT_4             ; GO IF RESEND
                        <1>     ;
```

```
                            <1>     ;----- CHECK FOR RESPONSE TO A COMMAND TO KEYBOARD
0000110F 3CFA               <1>     cmp     al, KB_ACK              ; IS THE INPUT AN ACKNOWLEDGE
00001111 7514               <1>         jne     short KB_INT_2          ; GO IF NOT
                            <1>     ;
                            <1>     ;----- A COMMAND TO THE KEYBOARD WAS ISSUED
00001113 FA                 <1>     cli                             ; DISABLE INTERRUPTS
00001114 800D[FB670000]10   <1>     or      byte [KB_FLAG_2], KB_FA ; INDICATE ACK RECEIVED
                            <1>         ;jmp       K26                     ; RETURN IF NOT ACK RETURNED FOR DATA)
                            <1>     ; 12/04/2021
0000111B EB76               <1>     jmp     short ID_EX  ; K26
                            <1>     ;
                            <1>     ;----- RESEND THE LAST BYTE
                            <1> KB_INT_4:
0000111D FA                 <1>     cli                             ; DISABLE INTERRUPTS
0000111E 800D[FB670000]20   <1>     or      byte [KB_FLAG_2], KB_FE ; INDICATE RESEND RECEIVED
                            <1>         ;jmp       K26                     ; RETURN IF NOT ACK RETURNED FOR DATA)
                            <1>     ; 12/04/2021
00001125 EB6C               <1>     jmp     short ID_EX  ; K26
                            <1>     ;
                            <1> ;-----      UPDATE MODE INDICATORS IF CHANGE IN STATE
                            <1> KB_INT_2:
                            <1>     ;push   ax                      ; SAVE DATA IN
                            <1>     ; 12/04/2021
00001127 50                 <1>     push    eax
00001128 E8E0050000         <1>     call    MAKE_LED                ; GO GET MODE INDICATOR DATA BYTE
0000112D 8A1D[FB670000]     <1>     mov     bl, [KB_FLAG_2]         ; GET PREVIOUS BITS
00001133 30C3               <1>     xor     bl, al                  ; SEE IF ANY DIFFERENT
00001135 80E307             <1>     and     bl, KB_LEDS             ; ISOLATE INDICATOR BITS
00001138 7405               <1>     jz      short UP0               ; IF NO CHANGE BYPASS UPDATE
0000113A E863050000         <1>     call    SND_LED                 ; GO TURN ON MODE INDICATORS
                            <1> UP0:
                            <1>     ;pop    ax                      ; RESTORE DATA IN
                            <1>     ; 12/04/2021
0000113F 58                 <1>     pop     eax
                            <1> ;--------------------------------------------------------------------
                            <1> ;   START OF KEY PROCESSING                                         ;
                            <1> ;--------------------------------------------------------------------
00001140 88C4               <1>     mov     ah, al                  ; SAVE SCAN CODE IN AH ALSO
                            <1>     ;
                            <1>     ;----- TEST FOR OVERRUN SCAN CODE FROM KEYBOARD
00001142 3CFF               <1>     cmp     al, KB_OVER_RUN                 ; IS THIS AN OVERRUN CHAR
                            <1>         ;je K62                     ; BUFFER_FULL_BEEP
                            <1>     ; 12/04/2021
00001144 7505               <1>     jne     short K16
00001146 E9E8040000         <1>     jmp     K62
                            <1> K16:
0000114B 8A3D[FC670000]     <1>     mov     bh, [KB_FLAG_3]                 ; LOAD FLAGS FOR TESTING
                            <1>     ;
                            <1>     ;----- TEST TO SEE IF A READ_ID IS IN PROGRESS
00001151 F6C7C0             <1>     test    bh, RD_ID+LC_AB         ; ARE WE DOING A READ ID?
00001154 7442               <1>     jz      short NOT_ID            ; CONTINUE IF NOT
00001156 7914               <1>     jns     short TST_ID_2          ; IS THE RD_ID FLAG ON?
00001158 3CAB               <1>     cmp     al, ID_1                ; IS THIS THE 1ST ID CHARACTER?
0000115A 7507               <1>     jne     short RST_RD_ID
0000115C 800D[FC670000]40   <1>     or      byte [KB_FLAG_3], LC_AB ; INDICATE 1ST ID WAS OK
                            <1> RST_RD_ID:
00001163 8025[FC670000]7F   <1>     and     byte [KB_FLAG_3], ~RD_ID ; RESET THE READ ID FLAG
0000116A EB27               <1>         jmp short ID_EX             ; AND EXIT
                            <1>     ; 12/04/2021
                            <1>     ;jmp    K26
                            <1>     ;
                            <1> TST_ID_2:
0000116C 8025[FC670000]BF   <1>     and     byte [KB_FLAG_3], ~LC_AB ; RESET FLAG
00001173 3C54               <1>     cmp     al, ID_2A               ; IS THIS THE 2ND ID CHARACTER?
00001175 7415               <1>         je  short KX_BIT            ; JUMP IF SO
00001177 3C41               <1>     cmp     al, ID_2                ; IS THIS THE 2ND ID CHARACTER?
00001179 7518               <1>         jne short ID_EX             ; LEAVE IF NOT
                            <1>     ; 12/04/2021
                            <1>     ;jne    K26
                            <1>     ;
                            <1>     ;----- A READ ID SAID THAT IT WAS ENHANCED KEYBOARD
0000117B F6C720             <1>     test    bh, SET_NUM_LK                  ; SHOULD WE SET NUM LOCK?
0000117E 740C               <1>         jz      short KX_BIT                ; EXIT IF NOT
00001180 800D[F9670000]20   <1>     or      byte [KB_FLAG], NUM_STATE ; FORCE NUM LOCK ON
00001187 E816050000         <1>     call    SND_LED                 ; GO SET THE NUM LOCK INDICATOR
                            <1> KX_BIT:
0000118C 800D[FC670000]10   <1>     or      byte [KB_FLAG_3], KBX ; INDICATE ENHANCED KEYBOARD WAS FOUND
00001193 E9CB010000         <1> ID_EX:      jmp     K26                     ; EXIT
                            <1>     ;
                            <1> NOT_ID:
00001198 3CE0               <1>     cmp     al, MC_E0               ; IS THIS THE GENERAL MARKER CODE?
0000119A 7509               <1>     jne     short TEST_E1
0000119C 800D[FC670000]12   <1>     or      byte [KB_FLAG_3], LC_E0+KBX ; SET FLAG BIT, SET KBX, AND
000011A3 EB0B               <1>     jmp     short EXIT              ; THROW AWAY THIS CODE
                            <1>     ; 12/04/2021
                            <1>     ;jmp    K26A
                            <1> TEST_E1:
000011A5 3CE1               <1>     cmp     al, MC_E1               ; IS THIS THE PAUSE KEY?
000011A7 750C               <1>     jne     short NOT_HC
000011A9 800D[FC670000]11   <1>     or      byte [KB_FLAG_3], LC_E1+KBX ; SET FLAG BIT, SET KBX, AND
000011B0 E9B5010000         <1> EXIT:       jmp     K26A                    ; THROW AWAY THIS CODE
                            <1>     ;
                            <1> NOT_HC:
000011B5 247F               <1>     and     al, 07Fh                ; TURN OFF THE BREAK BIT
000011B7 F6C702             <1>     test    bh, LC_E0               ; LAST CODE THE E0 MARKER CODE
000011BA 7410               <1>     jz      short NOT_LC_E0                 ; JUMP IF NOT
                            <1>     ;
000011BC BF[E6660000]       <1>     mov     edi, _K6+6              ; IS THIS A SHIFT KEY?
000011C1 AE                 <1>     scasb
                            <1>     ;je     K26 ; K16B                ; YES, THROW AWAY & RESET FLAG
                            <1>     ; 12/04/2021
000011C2 745B               <1>     je      short K16B ; K26
000011C4 AE                 <1>     scasb
000011C5 756D               <1>     jne     short K16A              ; NO, CONTINUE KEY PROCESSING
                            <1>     ;jmp    short K16B              ; YES, THROW AWAY & RESET FLAG
000011C7 E997010000         <1>     jmp     K26
                            <1>     ;
                            <1> NOT_LC_E0:
000011CC F6C701             <1>     test    bh, LC_E1               ; LAST CODE THE E1 MARKER CODE?
000011CF 7425               <1>     jz      short T_SYS_KEY                 ; JUMP IF NOT
000011D1 B904000000         <1>     mov     ecx, 4                  ; LENGHT OF SEARCH
000011D6 BF[E4660000]       <1>     mov     edi, _K6+4              ; IS THIS AN ALT, CTL, OR SHIFT?
000011DB F2AE               <1>     repne   scasb                   ; CHECK IT
000011DD 74D1               <1>     je      short EXIT              ; THROW AWAY IF SO
                            <1>     ; 12/04/2021
                            <1>     ;je     K26A
                            <1>     ;
000011DF 3C45               <1>     cmp     al, NUM_KEY             ; IS IT THE PAUSE KEY?
000011E1 753C               <1>     jne     short K16B              ; NO, THROW AWAY & RESET FLAG
                            <1>     ; 12/04/2021
                            <1>     ;jne    K26
000011E3 F6C480             <1>     test    ah, 80h                 ; YES, IS IT THE BREAK OF THE KEY?
000011E6 7537               <1>     jnz     short K16B              ; YES, THROW THIS AWAY, TOO
                            <1>     ; 24/07/2022
```

```
                            <1>     ;jnz    K26
                            <1>         ; 20/02/2015
000011E8 F605[FA670000]08   <1>     test    byte [KB_FLAG_1],HOLD_STATE ; NO, ARE WE PAUSED ALREADY?
000011EF 752E               <1>     jnz     short K16B              ; YES, THROW AWAY
                            <1>     ; 12/04/2021
                            <1>     ;jnz    K26
000011F1 E9B1020000         <1>     jmp     K39P                    ; NO, THIS IS THE REAL PAUSE STATE
                            <1>     ;
                            <1>     ;----- TEST FOR SYSTEM KEY
                            <1> T_SYS_KEY:
000011F6 3C54               <1>     cmp     al, SYS_KEY             ; IS IT THE SYSTEM KEY?
000011F8 753A               <1>     jnz     short K16A              ; CONTINUE IF NOT
                            <1>     ;
000011FA F6C480             <1>     test    ah, 80h                 ; CHECK IF THIS A BREAK CODE
000011FD 7525               <1>     jnz     short K16C              ; DO NOT TOUCH SYSTEM INDICATOR IF TRUE
                            <1>     ;
000011FF F605[FA670000]04   <1>     test    byte [KB_FLAG_1], SYS_SHIFT ; SEE IF IN SYSTEM KEY HELD DOWN
00001206 7517               <1>     jnz     short K16B              ; IF YES, DO NOT PROCESS SYSTEM INDICATOR
                            <1>     ; 12/04/2021
                            <1>     ;jnz    K26
                            <1>     ;
00001208 800D[FA670000]04   <1>     or      byte [KB_FLAG_1], SYS_SHIFT ; INDICATE SYSTEM KEY DEPRESSED
0000120F B020               <1>     mov     al, EOI                 ; END OF INTERRUPT COMMAND
00001211 E620               <1>     out     20h, al ;out INTA00, al       ; SEND COMMAND TO INTERRUPT CONTROL PORT
                            <1>                                     ; INTERRUPT-RETURN-NO-EOI
00001213 B0AE               <1>     mov     al, ENA_KBD             ; INSURE KEYBOARD IS ENABLED
00001215 E82D040000         <1>     call    SHIP_IT                 ; EXECUTE ENABLE
                            <1>     ; !!! SYSREQ !!! function/system call (INTERRUPT) must be here !!!
                            <1>     ;MOV    AL, 8500H               ; FUNCTION VALUE FOR MAKE OF SYSTEM KEY
                            <1>     ;STI                            ; MAKE SURE INTERRUPTS ENABLED
                            <1>     ;INT    15H                     ; USER INTERRUPT
0000121A E957010000         <1>         jmp     K27A                    ; END PROCESSING
                            <1>     ;
0000121F E93F010000         <1> K16B:       jmp     K26                     ; IGNORE SYSTEM KEY
                            <1>     ;
                            <1> K16C:
00001224 8025[FA670000]FB   <1>     and     byte [KB_FLAG_1], ~SYS_SHIFT ; TURN OFF SHIFT KEY HELD DOWN
0000122B B020               <1>     mov     al, EOI                 ; END OF INTERRUPT COMMAND
0000122D E620               <1>     out     20h, al ;out INTA00, al ; SEND COMMAND TO INTERRUPT CONTROL PORT
                            <1>                                     ; INTERRUPT-RETURN-NO-EOI
                            <1>     ;MOV    AL, ENA_KBD             ; INSURE KEYBOARD IS ENABLED
                            <1>     ;CALL   SHIP_IT                 ; EXECUTE ENABLE
                            <1>     ;
                            <1>     ;MOV    AX, 8501H               ; FUNCTION VALUE FOR BREAK OF SYSTEM KEY
                            <1>     ;STI                            ; MAKE SURE INTERRUPTS ENABLED
                            <1>     ;INT    15H                     ; USER INTERRUPT
                            <1>     ;JMP    K27A                    ; INGONRE SYSTEM KEY
                            <1>     ;
0000122F E93B010000         <1>     jmp     K27                     ; IGNORE SYSTEM KEY
                            <1>     ;
                            <1>     ;----- TEST FOR SHIFT KEYS
                            <1> K16A:
00001234 8A1D[F9670000]     <1>     mov     bl, [KB_FLAG]           ; PUT STATE FLAGS IN BL
0000123A BF[E0660000]       <1>     mov     edi, _K6                ; SHIFT KEY TABLE offset
0000123F B908000000         <1>     mov     ecx, _K6L               ; LENGTH
00001244 F2AE               <1>     repne   scasb                   ; LOOK THROUGH THE TABLE FOR A MATCH
00001246 88E0               <1>     mov     al, ah                  ; RECOVER SCAN CODE
                            <1>         ;jne        K25                     ; IF NO MATCH, THEN SHIFT NOT FOUND
                            <1>     ; 12/04/2021
00001248 7405               <1>     je      short K17
0000124A E9FC000000         <1>     jmp     K25
                            <1>     ;
                            <1>     ;------SHIFT KEY FOUND
                            <1> K17:
0000124F 81EF[E1660000]     <1>         sub     edi, _K6+1              ; ADJUST PTR TO SCAN CODE MATCH
00001255 8AA7[E8660000]     <1>             mov     ah, [edi+_K7]           ; GET MASK INTO AH
0000125B B102               <1>     mov     cl, 2                   ; SETUP COUNT FOR FLAG SHIFTS
0000125D A880               <1>     test    al, 80h                 ; TEST FOR BREAK KEY
                            <1>     ;jnz    short K23               ; JUMP OF BREAK
                            <1>     ; 12/04/2021
0000125F 7405               <1>     jz      short K17C
00001261 E981000000         <1>     jmp     K23
                            <1>     ;
                            <1>     ;----- SHIFT MAKE FOUND, DETERMINE SET OR TOGGLE
                            <1> K17C:
00001266 80FC10             <1>     cmp     ah, SCROLL_SHIFT
00001269 7324               <1>     jae     short K18               ; IF SCROLL SHIFT OR ABOVE, TOGGLE KEY
                            <1>     ;
                            <1>     ;----- PLAIN SHIFT KEY, SET SHIFT ON
0000126B 0825[F9670000]     <1>     or      [KB_FLAG], ah           ; TURN ON SHIFT BIT
00001271 A80C               <1>         test        al, CTL_SHIFT+ALT_SHIFT ; IS IT ALT OR CTRL?
                            <1>     ;;jnz   short K17D              ; YES, MORE FLAGS TO SET
                            <1>     ;jz     K26                     ; NO, INTERRUPT RETURN
                            <1>     ; 12/04/2021
00001273 7415               <1>     jz      short k17f
                            <1> K17D:
00001275 F6C702             <1>     test    bh, LC_E0               ; IS THIS ONE OF NEW KEYS?
00001278 7408               <1>     jz      short K17E              ; NO, JUMP
0000127A 0825[FC670000]     <1>     or      [KB_FLAG_3], ah                 ; SET BITS FOR RIGHT CTRL, ALT
                            <1>     ;jmp    K26                     ; INTERRUPT RETURN
                            <1>     ; 12/04/2021
00001280 EB08               <1>     jmp     short k17f
                            <1> K17E:
00001282 D2EC               <1>     shr     ah, cl                  ; MOVE FLAG BITS TWO POSITIONS
00001284 0825[FA670000]     <1>     or      [KB_FLAG_1], ah                 ; SET BITS FOR LEFT CTRL, ALT
                            <1> k17f:       ; 12/04/2021
0000128A E9D4000000         <1>     jmp     K26
                            <1>     ;
                            <1>     ;----- TOGGLED SHIFT KEY, TEST FOR 1ST MAKE OR NOT
                            <1> K18:                                ; SHIFT-TOGGLE
0000128F F6C304             <1>     test    bl, CTL_SHIFT           ; CHECK CTL SHIFT STATE
00001292 7402               <1>     jz      short K18A                ; JUMP IF NOT CTL STATE
                            <1>         ;jnz        K25                     ; JUMP IF CTL STATE
                            <1>     ; 12/04/2021
00001294 EB1C               <1>     jmp     short k20a ; K25
                            <1> K18A:
00001296 3C52               <1>     cmp     al, INS_KEY             ; CHECK FOR INSERT KEY
00001298 7522               <1>     jne     short K22               ; JUMP IF NOT INSERT KEY
0000129A F6C308             <1>     test    bl, ALT_SHIFT           ; CHECK FOR ALTERNATE SHIFT
0000129D 7402               <1>             jz      short K18B              ; JUMP IF NOT ALTERNATE SHIFT
                            <1>         ;jnz        K25                     ; JUMP IF ALTERNATE SHIFT
                            <1>     ; 12/04/2021
0000129F EB11               <1>     jmp     short k20a ; K25
                            <1> K18B:
000012A1 F6C702             <1>     test    bh, LC_E0 ;20/02/2015 ; IS THIS NEW INSERT KEY?
000012A4 7516               <1>     jnz     short K22               ; YES, THIS ONE'S NEVER A '0'
                            <1> K19:
000012A6 F6C320             <1>     test    bl, NUM_STATE           ; CHECK FOR BASE STATE
000012A9 750C               <1>     jnz     short K21               ; JUMP IF NUM LOCK IS ON
000012AB F6C303             <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; TEST FOR SHIFT STATE
000012AE 740C               <1>     jz      short K22               ; JUMP IF BASE STATE
                            <1> K20:                                ; NUMERIC ZERO, NOT INSERT KEY
000012B0 88C4               <1>     mov     ah, al                  ; PUT SCAN CODE BACK IN AH
                            <1> k20a:       ; 12/04/2021
```

```
000012B2 E994000000          <1>         jmp     K25                     ; NUMERAL '0', STNDRD. PROCESSING
                             <1> K21:                                    ; MIGHT BE NUMERIC
000012B7 F6C303              <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT
000012BA 74F4                <1>     jz      short K20                   ; IS NUMERIC, STD. PROC.
                             <1>     ;
                             <1> K22:                                    ; SHIFT TOGGLE KEY HIT; PROCESS IT
000012BC 8425[FA670000]      <1>     test    ah, [KB_FLAG_1]             ; IS KEY ALREADY DEPRESSED
                             <1>         ;jnz       short K26                ; JUMP IF KEY ALREADY DEPRESSED
                             <1>     ; 12/04/2021
000012C2 75C6                <1>     jnz     short k17f ; K26
                             <1> K22A:
000012C4 0825[FA670000]      <1>         or      [KB_FLAG_1], ah         ; INDICATE THAT THE KEY IS DEPRESSED
000012CA 3025[F9670000]      <1>     xor     [KB_FLAG], ah               ; TOGGLE THE SHIFT STATE
                             <1>     ;
                             <1>     ;----- TOGGLE LED IF CAPS, NUM  OR SCROLL KEY DEPRESSED
000012D0 F6C470              <1>     test    ah, CAPS_SHIFT+NUM_SHIFT+SCROLL_SHIFT ; SHIFT TOGGLE?
000012D3 7407                <1>     jz      short K22B                  ; GO IF NOT
                             <1>     ;
                             <1>     ; 12/04/2021 (32 bit push/pop)
000012D5 50                  <1>     push    eax ; push ax               ; SAVE SCAN CODE AND SHIFT MASK
000012D6 E8C7030000          <1>     call    SND_LED                     ; GO TURN MODE INDICATORS ON
000012DB 58                  <1>     pop     eax ; pop ax                ; RESTORE SCAN CODE
                             <1> K22B:
000012DC 3C52                <1>     cmp     al, INS_KEY                 ; TEST FOR 1ST MAKE OF INSERT KEY
                             <1>         ;jne       short K26                ; JUMP IF NOT INSERT KEY
                             <1>     ; 12/04/2021
000012DE 75AA                <1>     jne     short k17f ; K26
000012E0 88C4                <1>     mov     ah, al                       ; SCAN CODE IN BOTH HALVES OF AX
000012E2 E999000000          <1>         jmp K28                          ; FLAGS UPDATED, PROC. FOR BUFFER
                             <1>     ;
                             <1>     ;----- BREAK SHIFT FOUND
                             <1> K23:                                    ; BREAK-SHIFT-FOUND
000012E7 80FC10              <1>     cmp     ah, SCROLL_SHIFT            ; IS THIS A TOGGLE KEY
000012EA F6D4                <1>     not     ah                          ; INVERT MASK
000012EC 7355                <1>     jae     short K24                   ; YES, HANDLE BREAK TOGGLE
000012EE 2025[F9670000]      <1>     and     [KB_FLAG], ah               ; TURN OFF SHIFT BIT
000012F4 80FCFB              <1>     cmp     ah, ~CTL_SHIFT              ; IS THIS ALT OR CTL?
000012F7 7730                <1>     ja      short K23D                  ; NO, ALL DONE
                             <1>     ;
000012F9 F6C702              <1>     test    bh, LC_E0                   ; 2ND ALT OR CTL?
000012FC 7408                <1>     jz      short K23A                  ; NO, HANSLE NORMALLY
000012FE 2025[FC670000]      <1>     and     [KB_FLAG_3], ah                  ; RESET BIT FOR RIGHT ALT OR CTL
00001304 EB08                <1>     jmp     short K23B                  ; CONTINUE
                             <1> K23A:
00001306 D2FC                <1>     sar     ah, cl                      ; MOVE THE MASK BIT TWO POSITIONS
00001308 2025[FA670000]      <1>     and     [KB_FLAG_1], ah                  ; RESET BIT FOR LEFT ALT AND CTL
                             <1> K23B:
0000130E 88C4                <1>     mov     ah, al                      ; SAVE SCAN CODE
00001310 A0[FC670000]        <1>     mov     al, [KB_FLAG_3]                  ; GET RIGHT ALT & CTRL FLAGS
00001315 D2E8                <1>     shr     al, cl                      ; MOVE TO BITS 1 & 0
00001317 0A05[FA670000]      <1>     or      al, [KB_FLAG_1]                  ; PUT IN LEFT ALŞT & CTL FLAGS
0000131D D2E0                <1>     shl     al, cl                      ; MOVE BACK TO BITS 3 & 2
0000131F 240C                <1>     and     al, ALT_SHIFT+CTL_SHIFT ; FILTER OUT OTHER GARBAGE
00001321 0805[F9670000]      <1>     or      [KB_FLAG], al               ; PUT RESULT IN THE REAL FLAGS
00001327 88E0                <1>     mov     al, ah
                             <1> K23D:
00001329 3CB8                <1>     cmp     al, ALT_KEY+80h                  ; IS THIS ALTERNATE SHIFT RELEASE
0000132B 7536                <1>     jne     short K26                   ; INTERRUPT RETURN
                             <1>     ;
                             <1>     ;----- ALTERNATE SHIFT KEY RELEASED, GET THE VALUE INTO BUFFER
0000132D A0[FD670000]        <1>     mov     al, [ALT_INPUT]
00001332 B400                <1>     mov     ah, 0                       ; SCAN CODE OF 0
00001334 8825[FD670000]      <1>     mov     [ALT_INPUT], ah             ; ZERO OUT THE FIELD
0000133A 3C00                <1>     cmp     al, 0                       ; WAS THE INPUT = 0?
0000133C 7425                <1>     je      short K26                   ; INTERRUPT_RETURN
                             <1>         ; 29/01/2016
                             <1>     ;jmp    K61                         ; IT WASN'T, SO PUT IN BUFFER
0000133E E9AB020000          <1>     jmp     _K60
                             <1>     ;
                             <1> K24:                                    ; BREAK-TOGGLE
00001343 2025[FA670000]      <1>     and     [KB_FLAG_1], ah             ; INDICATE NO LONGER DEPRESSED
00001349 EB18                <1>     jmp     short K26                   ; INTERRUPT_RETURN
                             <1>     ;
                             <1>     ;----- TEST FOR HOLD STATE
                             <1>                                         ; AL, AH = SCAN CODE
                             <1> K25:                                    ; NO-SHIFT-FOUND
0000134B 3C80                <1>     cmp     al, 80h                     ; TEST FOR BREAK KEY
0000134D 7314                <1>     jae     short K26                   ; NOTHING FOR BREAK CHARS FROM HERE ON
0000134F F605[FA670000]08    <1>     test    byte [KB_FLAG_1], HOLD_STATE ; ARE WE IN HOLD STATE
00001356 7428                <1>     jz      short K28                   ; BRANCH AROUND TEST IF NOT
00001358 3C45                <1>     cmp     al, NUM_KEY
0000135A 7407                <1>     je      short K26                   ; CAN'T END HOLD ON NUM_LOCK
0000135C 8025[FA670000]F7    <1>     and     byte [KB_FLAG_1], ~HOLD_STATE ; TURN OFF THE HOLD STATE BIT
                             <1> K26:
00001363 8025[FC670000]FC    <1>     and     byte [KB_FLAG_3], ~(LC_E0+LC_E1) ; RESET LAST CHAR H.C. FLAG
                             <1> K26A:                                        ; INTERRUPT-RETURN
0000136A FA                  <1>     cli                                 ; TURN OFF INTERRUPTS
0000136B B020                <1>     mov     al, EOI                     ; END OF INTERRUPT COMMAND
0000136D E620                <1>     out     20h, al ;out INTA00, al          ; SEND COMMAND TO INTERRUPT CONTROL PORT
                             <1> K27:                                    ; INTERRUPT-RETURN-NO-EOI
0000136F B0AE                <1>     mov     al, ENA_KBD                 ; INSURE KEYBOARD IS ENABLED
00001371 E8D1020000          <1>     call    SHIP_IT                     ; EXECUTE ENABLE
                             <1> K27A:
00001376 FA                  <1>     cli                                 ; DISABLE INTERRUPTS
                             <1>     ;;mov   byte [intflg], 0 ; 07/01/2017 ;; 15/01/2017
00001377 07                  <1>     pop     es                          ; RESTORE REGISTERS
00001378 1F                  <1>     pop     ds
00001379 5F                  <1>     pop     edi
0000137A 5E                  <1>     pop     esi
0000137B 5A                  <1>     pop     edx
0000137C 59                  <1>     pop     ecx
0000137D 5B                  <1>     pop     ebx
0000137E 58                  <1>     pop     eax
                             <1>     ;pop    ebp
0000137F CF                  <1>     iretd                               ; RETURN
                             <1>
                             <1>     ;----- NOT IN  HOLD STATE
                             <1> K28:                                    ; NO-HOLD-STATE
00001380 3C58                <1>     cmp     al, 88                      ; TEST FOR OUT-OF-RANGE SCAN CODES
00001382 77DF                <1>     ja      short K26                   ; IGNORE IF OUT-OF-RANGE
                             <1>     ;
00001384 F6C308              <1>     test    bl, ALT_SHIFT               ; ARE WE IN ALTERNATE SHIFT
00001387 740E                <1>         jz  short K28A                  ; IF NOT ALTERNATE
                             <1>         ; 12/04/2021
                             <1>     ;jz     K38
                             <1>     ;
00001389 F6C710              <1>     test    bh, KBX                     ; IS THIS THE ENCHANCED KEYBOARD?
0000138C 740E                <1>     jz      short K29                   ; NO, ALT STATE IS REAL
                             <1>      ;28/02/2015
0000138E F605[FA670000]04    <1>     test    byte [KB_FLAG_1], SYS_SHIFT ; YES, IS SYSREQ KEY DOWN?
00001395 7405                <1>     jz      short K29                   ;  NO, ALT STATE IS REAL
                             <1>     ; 12/04/2021
                             <1>     ;jnz    K38                         ; YES, THIS IS PHONY ALT STATE
                             <1>         ;                               ; DUE TO PRESSING SYSREQ
```

```
00001397 E9C4000000          <1> K28A:       jmp    K38
                             <1>     ;
                             <1>     ;----- TEST FOR RESET KEY SEQUENCE (CTL ALT DEL)
                             <1> K29:                            ; TEST-RESET
0000139C F6C304              <1>     test   bl, CTL_SHIFT        ; ARE WE IN CONTROL SHIFT ALSO?
0000139F 740B                <1>     jz     short K31            ; NO_RESET
000013A1 3C53                <1>     cmp    al, DEL_KEY          ; CTL-ALT STATE, TEST FOR DELETE KEY
000013A3 7507                <1>     jne    short K31            ; NO_RESET, IGNORE
                             <1>     ;
                             <1>     ;----- CTL-ALT-DEL HAS BEEN FOUND
                             <1>     ; 26/08/2014
                             <1> cpu_reset:
                             <1>     ; IBM PC/AT ROM BIOS source code - 10/06/85 (TEST4.ASM - PROC_SHUTDOWN)
                             <1>     ; Send FEh (system reset command) to the keyboard controller.
000013A5 B0FE                <1>     mov    al, SHUT_CMD         ; SHUTDOWN COMMAND
000013A7 E664                <1>     out    STATUS_PORT, al              ; SEND TO KEYBOARD CONTROL PORT
                             <1> khere:
000013A9 F4                  <1>     hlt                         ; WAIT FOR 80286 RESET
000013AA EBFD                <1>     jmp    short khere          ; INSURE HALT
                             <1>     ;
                             <1>     ;----- IN ALTERNATE SHIFT, RESET NOT FOUND
                             <1> K31:                            ; NO-RESET
000013AC 3C39                <1>     cmp    al, 57               ; TEST FOR SPACE KEY
000013AE 7507                <1>     jne    short K311           ; NOT THERE
000013B0 B020                <1>     mov    al, ' '              ; SET SPACE CHAR
                             <1> k31a:       ; 12/04/2021
000013B2 E929020000          <1>         jmp     K57                     ; BUFFER_FILL
                             <1> K311:
000013B7 3C0F                <1>     cmp    al, 15               ; TEST FOR TAB KEY
000013B9 7506                <1>     jne    short K312           ; NOT THERE
000013BB 66B800A5            <1>     mov    ax, 0A500h           ; SET SPECIAL CODE FOR ALT-TAB
                             <1>         ;jmp       K57                   ; BUFFER_FILL
                             <1>     ; 12/04/2021
000013BF EBF1                <1>     jmp    short k31a
                             <1> K312:
000013C1 3C4A                <1>     cmp    al, 74               ; TEST FOR KEY PAD -
                             <1>         ;je short K37B           ; GO PROCESS
                             <1>     ; 12/04/2021
000013C3 7404                <1>     je     short k312a
000013C5 3C4E                <1>     cmp    al, 78               ; TEST FOR KEY PAD +
                             <1>         ;je short K37B           ; GO PROCESS
                             <1>     ; 12/04/2021
000013C7 7505                <1>     jne    short K32
                             <1> k312a:
000013C9 E988000000          <1>     jmp    K37B
                             <1>     ;
                             <1>     ;----- LOOK FOR KEY PAD ENTRY
                             <1> K32:                            ; ALT-KEY-PAD
000013CE BF[BC660000]        <1>     mov    edi, K30             ; ALT-INPUT-TABLE offset
000013D3 B90A000000          <1>     mov    ecx, 10              ; LOOK FOR ENTRY USING KEYPAD
000013D8 F2AE                <1>     repne  scasb                ; LOOK FOR MATCH
000013DA 7521                <1>     jne    short K33            ; NO_ALT_KEYPAD
000013DC F6C702              <1>     test   bh, LC_E0            ; IS THIS ONE OF THE NEW KEYS?
000013DF 7579                <1>         jnz short K37C           ; YES, JUMP, NOT NUMPAD KEY
000013E1 81EF[BD660000]      <1>     sub    edi, K30+1           ; DI NOW HAS ENTRY VALUE
000013E7 A0[FD670000]        <1>     mov    al, [ALT_INPUT]      ; GET THE CURRENT BYTE
000013EC B40A                <1>     mov    ah, 10               ; MULTIPLY BY 10
000013EE F6E4                <1>     mul    ah
000013F0 6601F8              <1>     add    ax, di               ; ADD IN THE LATEST ENTRY
000013F3 A2[FD670000]        <1>     mov    [ALT_INPUT], al      ; STORE IT AWAY
                             <1> K32A:
000013F8 E966FFFFFF          <1>         jmp     K26                     ; THROW AWAY THAT KEYSTROKE
                             <1>     ;
                             <1>     ;----- LOOK FOR SUPERSHIFT ENTRY
                             <1> K33:                            ; NO-ALT-KEYPAD
000013FD C605[FD670000]00     <1>         mov     byte [ALT_INPUT], 0     ; ZERO ANY PREVIOUS ENTRY INTO INPUT
00001404 B91A000000          <1>     mov    ecx, 26              ; (DI),(ES) ALREADY POINTING
00001409 F2AE                <1>     repne  scasb                ; LOOK FOR MATCH IN ALPHABET
0000140B 7445                <1>     je     short K37A           ; MATCH FOUND, GO FILLL THE BUFFER
                             <1>     ;
                             <1>     ;----- LOOK FOR TOP ROW OF ALTERNATE SHIFT
                             <1> K34:                            ; ALT-TOP-ROW
0000140D 3C02                <1>     cmp    al, 2                ; KEY WITH '1' ON IT
0000140F 7245                <1>     jb     short K37B           ; MUST BE ESCAPE
00001411 3C0D                <1>     cmp    al, 13               ; IS IT IN THE REGION
00001413 7705                <1>     ja     short K35            ; NO, ALT SOMETHING ELSE
00001415 80C476              <1>     add    ah, 118              ; CONVERT PSEUDO SCAN CODE TO RANGE
00001418 EB38                <1>     jmp    short K37A           ; GO FILL THE BUFFER
                             <1>     ;
                             <1>     ;----- TRANSLATE ALTERNATE SHIFT PSEUDO SCAN CODES
                             <1> K35:                            ; ALT-FUNCTION
0000141A 3C57                <1>     cmp    al, F11_M            ; IS IT F11?
0000141C 7209                <1>     jb     short K35A ; 20/02/2015      ; NO, BRANCH
0000141E 3C58                <1>     cmp    al, F12_M            ; IS IT F12?
00001420 7705                <1>     ja     short K35A ; 20/02/2015      ; NO, BRANCH
00001422 80C434              <1>     add    ah, 52               ; CONVERT TO PSEUDO SCAN CODE
00001425 EB2B                <1>     jmp    short K37A           ; GO FILL THE BUFFER
                             <1> K35A:
00001427 F6C702              <1>     test   bh, LC_E0            ; DO WE HAVE ONE OF THE NEW KEYS?
0000142A 741B                <1>     jz     short K37            ; NO, JUMP
0000142C 3C1C                <1>     cmp    al, 28               ; TEST FOR KEYPAD ENTER
0000142E 7506                <1>         jne     short K35B              ; NOT THERE
00001430 66B800A6            <1>     mov    ax, 0A600h           ; SPECIAL CODE
                             <1>     ;jmp   K57                  ; BUFFER FILL
                             <1>     ; 12/04/2021
00001434 EB0C                <1>     jmp    short k35c
                             <1> K35B:
00001436 3C53                <1>     cmp    al, 83               ; TEST FOR DELETE KEY
00001438 7420                <1>     je     short K37C           ; HANDLE WITH OTHER EDIT KEYS
0000143A 3C35                <1>     cmp    al, 53               ; TEST FOR KEYPAD /
0000143C 75BA                <1>     jne    short K32A           ; NOT THERE, NO OTHER E0 SPECIALS
                             <1>         ; 12/04/2021
                             <1>     ;jne   K26
0000143E 66B800A4            <1>     mov    ax, 0A400h           ; SPECIAL CODE1
                             <1> k35c:       ; 12/04/2021
00001442 E999010000          <1>     jmp    K57                  ; BUFFER FILL
                             <1> K37:
00001447 3C3B                <1>     cmp    al, 59               ; TEST FOR FUNCTION KEYS (F1)
00001449 720B                <1>         jb      short K37B       ; NO FN, HANDLE W/OTHER EXTENDED
0000144B 3C44                <1>     cmp    al, 68               ; IN KEYPAD REGION?
0000144D 77A9                <1>     ja     short K32A           ; IF SO, IGNORE
                             <1>     ; 12/04/2021
                             <1>     ;ja    K26
0000144F 80C42D              <1>     add    ah, 45               ; CONVERT TO PSEUDO SCAN CODE
                             <1> K37A:
00001452 B000                <1>     mov    al, 0                ; ASCII CODE OF ZERO
                             <1>     ;jmp   K57                  ; PUT IT IN THE BUFFER
                             <1>     ; 12/04/2021
00001454 EBEC                <1>     jmp    short k35c
                             <1> K37B:
00001456 B0F0                <1>     mov    al, 0F0h             ; USE SPECIAL ASCII CODE
                             <1>     ;jmp   K57                  ; PUT IT IN THE BUFFER
                             <1>     ; 12/04/2021
00001458 EBE8                <1>     jmp    short k35c
```

```
                                <1> K37C:
0000145A 0450                   <1>     add     al, 80                  ; CONVERT SCAN CODE (EDIT KEYS)
0000145C 88C4                   <1>     mov     ah, al                  ; (SCAN CODE NOT IN AH FOR INSERT)
0000145E EBF2                   <1>     jmp      short K37A              ; PUT IT IN THE BUFFER
                                <1>     ;
                                <1>     ;----- NOT IN ALTERNATE SHIFT
                                <1> K38:                                ; NOT-ALT-SHIFT
                                <1>                                     ; BL STILL HAS SHIFT FLAGS
00001460 F6C304                 <1>     test    bl, CTL_SHIFT           ; ARE WE IN CONTROL SHIFT?
                                <1>     ;;jnz   short K38A              ; YES, START PROCESSING
                                <1>         ;jz K44                     ; NOT-CTL-SHIFT
                                <1>     ; 12/04/2021
00001463 7505                   <1>     jnz     short K38A              ; YES, START PROCESSING
00001465 E9AB000000             <1>     jmp     K44                     ; NOT-CTL-SHIFT
                                <1>     ;
                                <1>     ;----- CONTROL SHIFT, TEST SPECIAL CHARACTERS
                                <1>     ;----- TEST FOR BREAK
                                <1> K38A:
0000146A 3C46                   <1>     cmp     al, SCROLL_KEY          ; TEST FOR BREAK
0000146C 7530                   <1>     jne     short K39               ; JUMP, NO-BREAK
0000146E F6C710                 <1>     test    bh, KBX                 ; IS THIS THE ENHANCED KEYBOARD?
00001471 7405                   <1>     jz      short K38B              ; NO, BREAK IS VALID
00001473 F6C702                 <1>     test    bh, LC_E0               ; YES, WAS LAST CODE AN E0?
00001476 7426                   <1>     jz      short K39               ; NO-BREAK, TEST FOR PAUSE
                                <1> K38B:
00001478 8B1D[06680000]         <1>     mov     ebx, [BUFFER_HEAD]      ; RESET BUFFER TO EMPTY
0000147E 891D[0A680000]         <1>     mov     [BUFFER_TAIL], ebx
00001484 C605[F8670000]80       <1>     mov     byte [BIOS_BREAK], 80h  ; TURN ON BIOS_BREAK BIT
                                <1>     ;
                                <1>     ;----- ENABLE KEYBOARD
0000148B B0AE                   <1>     mov     al, ENA_KBD             ; ENABLE KEYBOARD
0000148D E8B5010000             <1>     call    SHIP_IT                 ; EXECUTE ENABLE
                                <1>     ;
                                <1>     ; CTRL+BREAK code here !!!
                                <1>     ;INT    1BH                     ; BREAK INTERRUPT VECTOR
                                <1>     ; 17/10/2015
00001492 E8A7590000             <1>     call    ctrlbrk ; control+break subroutine
                                <1>     ;
                                <1>     ;sub    ax, ax                  ; PUT OUT DUMMY CHARACTER
                                <1>         ; 12/04/2021
00001497 29C0                   <1>     sub     eax, eax
00001499 E942010000             <1>         jmp     K57                     ; BUFFER_FILL
                                <1>     ;
                                <1>     ;----- TEST FOR PAUSE
                                <1> K39:                                ; NO_BREAK
0000149E F6C710                 <1>     test    bh, KBX                 ; IS THIS THE ENHANCED KEYBOARD?
000014A1 7537                   <1>     jnz     short K41               ; YES, THEN THIS CAN'T BE PAUSE
000014A3 3C45                   <1>     cmp     al, NUM_KEY             ; LOOK FOR PAUSE KEY
000014A5 7533                   <1>     jne     short K41               ; NO-PAUSE
                                <1> K39P:
000014A7 800D[FA670000]08       <1>     or      byte [KB_FLAG_1], HOLD_STATE ; TURN ON THE HOLD FLAG
                                <1>     ;
                                <1>     ;----- ENABLE KEYBOARD
000014AE B0AE                   <1>     mov     al, ENA_KBD             ; ENABLE KEYBOARD
000014B0 E892010000             <1>     call    SHIP_IT                 ; EXECUTE ENABLE
                                <1> K39A:
000014B5 B020                   <1>     mov     al, EOI                 ; END OF INTERRUPT TO CONTROL PORT
000014B7 E620                   <1>     out     20h, al ;out INTA00, al       ; ALLOW FURTHER KEYSTROKE INTERRUPTS
                                <1>     ;
                                <1>     ;----- DURING PAUSE INTERVAL, TURN COLOR CRT BACK ON
000014B9 803D[2E680000]07       <1>         cmp     byte [CRT_MODE], 7      ; IS THIS BLACK AND WHITE CARD
000014C0 740A                   <1>         je      short K40               ; YES, NOTHING TO DO
000014C2 66BAD803               <1>     mov     dx, 03D8h               ; PORT FOR COLOR CARD
000014C6 A0[2F680000]           <1>         mov     al, [CRT_MODE_SET]      ; GET THE VALUE OF THE CURRENT MODE
000014CB EE                     <1>     out     dx, al                  ; SET THE CRT MODE, SO THAT CRT IS ON
                                <1>     ;
                                <1> K40:                                ; PAUSE-LOOP
000014CC F605[FA670000]08       <1>         test    byte [KB_FLAG_1], HOLD_STATE ; CHECK HOLD STATE FLAG
000014D3 75F7                   <1>     jnz     short K40               ; LOOP UNTIL FLAG TURNED OFF
                                <1>     ;
000014D5 E995FEFFFF             <1>         jmp     K27                     ; INTERRUPT_RETURN_NO_EOI
                                <1>         ;
                                <1>     ;----- TEST SPECIAL CASE KEY 55
                                <1> K41:                                ; NO-PAUSE
000014DA 3C37                   <1>     cmp     al, 55                  ; TEST FOR */PRTSC KEY
000014DC 7513                   <1>     jne     short K42               ; NOT-KEY-55
000014DE F6C710                 <1>     test    bh, KBX                 ; IS THIS THE ENHANCED KEYBOARD?
000014E1 7405                   <1>     jz      short K41A              ; NO, CTL-PRTSC IS VALID
000014E3 F6C702                 <1>     test    bh, LC_E0               ; YES, WAS LAST CODE AN E0?
000014E6 7421                   <1>     jz      short K42B              ; NO, TRANSLATE TO A FUNCTION
                                <1> K41A:
000014E8 66B80072               <1>     mov     ax, 114*256             ; START/STOP PRINTING SWITCH
000014EC E9EF000000             <1>         jmp     K57                     ; BUFFER_FILL
                                <1>     ;
                                <1>     ;----- SET UP TO TRANSLATE CONTROL SHIFT
                                <1> K42:                                ; NOT-KEY-55
000014F1 3C0F                   <1>     cmp     al, 15                  ; IS IT THE TAB KEY?
000014F3 7414                   <1>     je      short K42B              ; YES, XLATE TO FUNCTION CODE
000014F5 3C35                   <1>     cmp     al, 53                  ; IS IT THE / KEY?
000014F7 750E                   <1>     jne     short K42A              ; NO, NO MORE SPECIAL CASES
000014F9 F6C702                 <1>     test    bh, LC_E0               ; YES, IS IT FROM THE KEY PAD?
000014FC 7409                   <1>     jz      short K42A              ; NO, JUST TRANSLATE
000014FE 66B80095               <1>     mov     ax, 9500h               ; YES, SPECIAL CODE FOR THIS ONE
00001502 E9D9000000             <1>     jmp     K57                     ; BUFFER FILL
                                <1> K42A:
                                <1>     ;;mov   ebx, _K8                ; SET UP TO TRANSLATE CTL
00001507 3C3B                   <1>     cmp     al, 59                  ; IS IT IN CHARACTER TABLE?
                                <1>         ;jb short K45F                ; YES, GO TRANSLATE CHAR
                                <1>     ;;jb    K56 ; 20/02/2015
                                <1>     ;;jmp   K64 ; 20/02/2015
                                <1> K42B:
00001509 BB[F0660000]           <1>     mov     ebx, _K8                ; SET UP TO TRANSLATE CTL
                                <1>     ;jb     K56 ;; 20/02/2015
                                <1>     ; 12/04/2021
0000150E 7267                   <1>     jb      short K45F
00001510 E9B9000000             <1>     jmp     K64
                                <1>         ;
                                <1>     ;----- NOT IN CONTROL SHIFT
                                <1> K44:                                ; NOT-CTL-SHIFT
00001515 3C37                   <1>     cmp     al, 55                  ; PRINT SCREEN KEY?
00001517 7528                   <1>     jne     short K45               ; NOT PRINT SCREEN
00001519 F6C710                 <1>     test    bh, KBX                 ; IS THIS ENHANCED KEYBOARD?
0000151C 7407                   <1>     jz      short K44A              ; NO, TEST FOR SHIFT STATE
0000151E F6C702                 <1>     test    bh, LC_E0               ; YES, LAST CODE A MARKER?
00001521 7507                   <1>     jnz     short K44B              ; YES, IS PRINT SCREEN
00001523 EB41                   <1>     jmp     short K45C              ; NO, TRANSLATE TO '*' CHARACTER
                                <1> K44A:
00001525 F6C303                 <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; NOT 101 KBD, SHIFT KEY DOWN?
00001528 743C                   <1>     jz      short K45C              ; NO, TRANSLATE TO '*' CHARACTER
                                <1>     ;
                                <1>     ;----- ISSUE INTERRUPT TO INDICATE PRINT SCREEN FUNCTION
                                <1> K44B:
0000152A B0AE                   <1>     mov     al, ENA_KBD             ; INSURE KEYBOARD IS ENABLED
0000152C E816010000             <1>     call    SHIP_IT                 ; EXECUTE ENABLE
```

```
00001531 B020                <1>     mov     al, EOI                 ; END OF CURRENT INTERRUPT
00001533 E620                <1>     out     20h, al ;out INTA00, al         ; SO FURTHER THINGS CAN HAPPEN
                             <1>     ; Print Screen !!!              ; ISSUE PRINT SCREEN INTERRUPT (INT 05h)
                             <1>     ;PUSH   BP                      ; SAVE POINTER
                             <1>     ;INT    5H                      ; ISSUE PRINT SCREEN INTERRUPT
                             <1>     ;POP    BP                      ; RESTORE POINTER
00001535 8025[FC670000]FC    <1>         and     byte [KB_FLAG_3], ~(LC_E0+LC_E1) ; ZERO OUT THESE FLAGS
0000153C E92EFEFFFF          <1>         jmp     K27                         ; GO BACK WITHOUT EOI OCCURRING
                             <1>     ;
                             <1>     ;----- HANDLE IN-CORE KEYS
                             <1> K45:                                ; NOT-PRINT-SCREEN
00001541 3C3A                <1>     cmp     al, 58                  ; TEST FOR IN-CORE AREA
00001543 7734                <1>     ja      short K46               ; JUMP IF NOT
00001545 3C35                <1>     cmp     al, 53                  ; IS THIS THE '/' KEY?
00001547 7505                <1>     jne     short K45A              ; NO, JUMP
00001549 F6C702              <1>     test    bh, LC_E0               ; WAS THE LAST CODE THE MARKER?
0000154C 7518                <1>     jnz     short K45C              ; YES, TRANSLATE TO CHARACTER
                             <1> K45A:
0000154E B91A000000          <1>     mov     ecx, 26                 ; LENGHT OF SEARCH
00001553 BF[C6660000]        <1>     mov     edi, K30+10             ; POINT TO TABLE OF A-Z CHARS
00001558 F2AE                <1>     repne   scasb                   ; IS THIS A LETTER KEY?
                             <1>             ; 20/02/2015
0000155A 7505                <1>     jne     short K45B                ; NO, SYMBOL KEY
                             <1>     ;
0000155C F6C340              <1>     test    bl, CAPS_STATE          ; ARE WE IN CAPS_LOCK?
0000155F 750C                <1>     jnz     short K45D              ; TEST FOR SURE
                             <1> K45B:
00001561 F6C303              <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; ARE WE IN SHIFT STATE?
00001564 750C                <1>     jnz     short K45E              ; YES, UPPERCASE
                             <1>                                     ; NO, LOWERCASE
                             <1> K45C:
00001566 BB[48670000]        <1>     mov     ebx, K10                ; TRANSLATE TO LOWERCASE LETTERS
0000156B EB51                <1>     jmp     short K56
                             <1> K45D:                                       ; ALMOST-CAPS-STATE
0000156D F6C303              <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; CL ON. IS SHIFT ON, TOO?
00001570 75F4                <1>     jnz     short K45C              ; SHIFTED TEMP OUT OF CAPS STATE
                             <1> K45E:
00001572 BB[A0670000]        <1>     mov     ebx, K11                ; TRANSLATE TO UPPER CASE LETTERS
00001577 EB45                <1> K45F:       jmp     short K56
                             <1>     ;
                             <1>     ;----- TEST FOR KEYS F1 - F10
                             <1> K46:                                ; NOT IN-CORE AREA
00001579 3C44                <1>     cmp     al, 68                  ; TEST FOR F1 - F10
                             <1>     ;ja     short K47               ; JUMP IF NOT
                             <1>     ;jmp    short K53               ; YES, GO DO FN KEY PROCESS
0000157B 7635                <1>     jna     short K53
                             <1>     ;
                             <1>     ;----- HANDLE THE NUMERIC PAD KEYS
                             <1> K47:                                ; NOT F1 - F10
0000157D 3C53                <1>     cmp     al, 83                  ; TEST NUMPAD KEYS
0000157F 772D                <1>     ja      short K52               ; JUMP IF NOT
                             <1>     ;
                             <1>     ;----- KEYPAD KEYS, MUST TEST NUM LOCK FOR DETERMINATION
                             <1> K48:
00001581 3C4A                <1>     cmp     al, 74                  ; SPECIAL CASE FOR MINUS
00001583 74ED                <1>     je      short K45E              ; GO TRANSLATE
00001585 3C4E                <1>     cmp     al, 78                  ; SPECIAL CASE FOR PLUS
00001587 74E9                <1>     je      short K45E              ; GO TRANSLATE
00001589 F6C702              <1>     test    bh, LC_E0               ; IS THIS ONE OFTHE NEW KEYS?
0000158C 750A                <1>     jnz     short K49               ; YES, TRANSLATE TO BASE STATE
                             <1>     ;
0000158E F6C320              <1>     test    bl, NUM_STATE           ; ARE WE IN NUM LOCK
00001591 7514                <1>     jnz     short K50               ; TEST FOR SURE
00001593 F6C303              <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; ARE WE IN SHIFT STATE?
                             <1>     ;jnz    short K51               ; IF SHIFTED, REALLY NUM STATE
00001596 75DA                <1>     jnz     short K45E
                             <1>     ;
                             <1>     ;----- BASE CASE FOR KEYPAD
                             <1> K49:
00001598 3C4C                <1>     cmp     al, 76                  ; SPECIAL CASE FOR BASE STATE 5
0000159A 7504                <1>     jne     short K49A              ; CONTINUE IF NOT KEYPAD 5
0000159C B0F0                <1>     mov     al, 0F0h                ; SPECIAL ASCII CODE
0000159E EB40                <1>     jmp     short K57               ; BUFFER FILL
                             <1> K49A:
000015A0 BB[48670000]        <1>     mov     ebx, K10                ; BASE CASE TABLE
000015A5 EB27                <1>     jmp     short K64               ; CONVERT TO PSEUDO SCAN
                             <1>     ;
                             <1>     ;----- MIGHT BE NUM LOCK, TEST SHIFT STATUS
                             <1> K50:                                ; ALMOST-NUM-STATE
000015A7 F6C303              <1>         test    bl, LEFT_SHIFT+RIGHT_SHIFT
000015AA 75EC                <1>     jnz     short K49               ; SHIFTED TEMP OUT OF NUM STATE
000015AC EBC4                <1> K51:jmp     short K45E              ; REALLY NUM STATE
                             <1>     ;
                             <1>     ;----- TEST FOR THE NEW KEYS ON WT KEYBOARDS
                             <1> K52:                                ; NOT A NUMPAD KEY
000015AE 3C56                <1>     cmp     al, 86                  ; IS IT THE NEW WT KEY?
                             <1>     ;jne    short K53               ; JUMP IF NOT
                             <1>     ;jmp    short K45B              ; HANDLE WITH REST OF LETTER KEYS
000015B0 74AF                <1>     je      short K45B
                             <1>     ;
                             <1>     ;----- MUST BE F11 OR F12
                             <1> K53:                                ; F1 - F10 COME HERE, TOO
000015B2 F6C303              <1>     test    bl, LEFT_SHIFT+RIGHT_SHIFT ; TEST SHIFT STATE
000015B5 74E1                <1>     jz      short K49               ; JUMP, LOWER CASE PSEUDO SC'S
                             <1>             ; 20/02/2015
000015B7 BB[A0670000]        <1>     mov     ebx, K11                ; UPPER CASE PSEUDO SCAN CODES
000015BC EB10                <1>     jmp     short K64               ; TRANSLATE SCAN
                             <1>     ;
                             <1>     ;----- TRANSLATE THE CHARACTER
                             <1> K56:                                ; TRANSLATE-CHAR
000015BE FEC8                <1>     dec     al                      ; CONVERT ORIGIN
000015C0 D7                  <1>     xlat                            ; CONVERT THE SCAN CODE TO ASCII
000015C1 F605[FC670000]02    <1>     test    byte [KB_FLAG_3], LC_E0         ; IS THIS A NEW KEY?
000015C8 7416                <1>     jz      short K57               ; NO, GO FILL BUFFER
000015CA B4E0                <1>     mov     ah, MC_E0               ; YES, PUT SPECIAL MARKER IN AH
000015CC EB12                <1>     jmp     short K57               ; PUT IT INTO THE BUFFER
                             <1>     ;
                             <1>     ;----- TRANSLATE SCAN FOR PSEUDO SCAN CODES
                             <1> K64:                                ; TRANSLATE-SCAN-ORGD
000015CE FEC8                <1>     dec     al                      ; CONVERT ORIGIN
000015D0 D7                  <1>             xlat                            ; CTL TABLE SCAN
000015D1 88C4                <1>     mov     ah, al                  ; PUT VALUE INTO AH
000015D3 B000                <1>     mov     al, 0                   ; ZERO ASCII CODE
000015D5 F605[FC670000]02    <1>     test    byte [KB_FLAG_3], LC_E0         ; IS THIS A NEW KEY?
000015DC 7402                <1>     jz      short K57               ; NO, GO FILL BUFFER
000015DE B0E0                <1>     mov     al, MC_E0               ; YES, PUT SPECIAL MARKER IN AL
                             <1>     ;
                             <1>     ;----- PUT CHARACTER INTO BUFFER
                             <1> K57:                                ; BUFFER_FILL
000015E0 3CFF                <1>     cmp     al, -1                  ; IS THIS AN IGNORE CHAR
000015E2 7405                <1>     je      short K59               ; YES, DO NOTHING WITH IT
                             <1>     ;je     K26                     ; YES, DO NOTHING WITH IT
000015E4 80FCFF              <1>     cmp     ah, -1                  ; LOOK FOR -1 PSEUDO SCAN
                             <1>         ;;jne      short K61                ; NEAR_INTERRUPT_RETURN
```

```
                                <1>     ;je    K26                  ; INTERRUPT_RETURN
                                <1>     ; 12/04/2021
000015E7 7505                   <1>         jne short _K60          ; NEAR_INTERRUPT_RETURN
                                <1> K59:                            ; NEAR_INTERRUPT_RETURN
000015E9 E975FDFFFF             <1>     jmp    K26                  ; INTERRUPT_RETURN
                                <1>
                                <1> _K60: ; 29/01/2016
000015EE 80FC68                 <1>     cmp    ah, 68h ; ALT + F1 key
000015F1 721D                   <1>     jb     short K61
000015F3 80FC6F                 <1>     cmp    ah, 6Fh ; ALT + F8 key
000015F6 7718                   <1>     ja     short K61
                                <1>     ;
000015F8 8A1D[DE760100]         <1>     mov    bl, [ACTIVE_PAGE]
000015FE 80C368                 <1>     add    bl, 68h
00001601 38E3                   <1>     cmp    bl, ah
00001603 740B                   <1>     je     short K61
                                <1>     ; 24/07/2022
                                <1>     ;push  ax
00001605 50                     <1>     push   eax
00001606 88E0                   <1>     mov    al, ah
00001608 2C68                   <1>     sub    al, 68h
0000160A E8A4090000             <1>     call   set_active_page
0000160F 58                     <1>     pop    eax
                                <1>     ;pop   ax
                                <1> K61:                            ; NOT-CAPS-STATE
00001610 8B1D[0A680000]         <1>     mov    ebx, [BUFFER_TAIL]   ; GET THE END POINTER TO THE BUFFER
00001616 89DE                   <1>     mov    esi, ebx             ; SAVE THE VALUE
00001618 E8B5FAFFFF             <1>     call   _K4                  ; ADVANCE THE TAIL
0000161D 3B1D[06680000]         <1>     cmp    ebx, [BUFFER_HEAD]   ; HAS THE BUFFER WRAPPED AROUND
00001623 740E                   <1>     je     short K62            ; BUFFER_FULL_BEEP
00001625 668906                 <1>     mov    [esi], ax            ; STORE THE VALUE
00001628 891D[0A680000]         <1>     mov    [BUFFER_TAIL], ebx   ; MOVE THE POINTER UP
0000162E E930FDFFFF             <1>     jmp    K26
                                <1>     ;;cli                       ; TURN OFF INTERRUPTS
                                <1>     ;;mov  al, EOI              ; END OF INTERRUPT COMMAND
                                <1>     ;;out  INTA00, al           ; SEND COMMAND TO INTERRUPT CONTROL PORT
                                <1>     ;mov   al, ENA_KBD          ; INSURE KEYBOARD IS ENABLED
                                <1>     ;call  SHIP_IT              ; EXECUTE ENABLE
                                <1>     ;mov   ax, 9102h            ; MOVE IN POST CODE & TYPE
                                <1>     ;int   15h                  ; PERFORM OTHER FUNCTION
                                <1>     ;;and  byte [KB_FLAG_3],~(LC_E0+LC_E1) ; RESET LAST CHAR H.C. FLAG
                                <1>     ;jmp   K27A                 ; INTERRUPT_RETURN
                                <1>     ;;jmp   K27
                                <1>     ;
                                <1>     ;----- BUFFER IS FULL SOUND THE BEEPER
                                <1> K62:
00001633 B020                   <1>     mov    al, EOI              ; ENABLE INTERRUPT CONTROLLER CHIP
00001635 E620                   <1>     out    INTA00, al
00001637 66B9A602               <1>     mov    cx, 678              ; DIVISOR FOR 1760 HZ
0000163B B304                   <1>     mov    bl, 4                ; SHORT BEEP COUNT (1/16 + 1/64 DELAY)
0000163D E8960D0000             <1>     call   beep                 ; GO TO COMMON BEEP HANDLER
00001642 E928FDFFFF             <1>     jmp     K27                 ; EXIT
                                <1>
                                <1> SHIP_IT:
                                <1>     ;--------------------------------------------------------------------------------
                                <1>     ; SHIP_IT
                                <1>     ;      THIS ROUTINES HANDLES TRANSMISSION OF COMMAND AND DATA BYTES
                                <1>     ;      TO THE KEYBOARD CONTROLLER.
                                <1>     ;--------------------------------------------------------------------------------
                                <1>     ;
                                <1>
                                <1>     ;push  ax                   ; SAVE DATA TO SEND
                                <1>     ; 12/04/2021
00001647 50                     <1>     push   eax
                                <1>
                                <1>     ;----- WAIT FOR COMMAND TO ACCEPTED
00001648 FA                     <1>     cli                         ; DISABLE INTERRUPTS TILL DATA SENT
                                <1>     ; xor  ecx, ecx             ; CLEAR TIMEOUT COUNTER
00001649 B900000100             <1>     mov    ecx, 10000h
                                <1> S10:
0000164E E464                   <1>     in     al, STATUS_PORT              ; READ KEYBOARD CONTROLLER STATUS
00001650 A802                   <1>     test   al, INPT_BUF_FULL    ; CHECK FOR ITS INPUT BUFFER BUSY
00001652 E0FA                   <1>     loopnz S10                  ; WAIT FOR COMMAND TO BE ACCEPTED
                                <1>
                                <1>     ;pop   ax                   ; GET DATA TO SEND
                                <1>     ; 12/04/2021
00001654 58                     <1>     pop    eax
                                <1>
00001655 E664                   <1>     out    STATUS_PORT, al              ; SEND TO KEYBOARD CONTROLLER
00001657 FB                     <1>     sti                         ; ENABLE INTERRUPTS AGAIN
00001658 C3                     <1>     retn                        ; RETURN TO CALLER
                                <1>
                                <1>     ; 12/04/2021 (32 bit push/pop)
                                <1> SND_DATA:
                                <1>     ; ------------------------------------------------------------------------------
                                <1>     ; SND_DATA
                                <1>     ;      THIS ROUTINES HANDLES TRANSMISSION OF COMMAND AND DATA BYTES
                                <1>     ;      TO THE KEYBOARD AND RECEIPT OF ACKNOWLEDGEMENTS. IT ALSO
                                <1>     ;      HANDLES ANY RETRIES IF REQUIRED
                                <1>     ; ------------------------------------------------------------------------------
                                <1>     ;
00001659 50                     <1>     push   eax ; push ax        ; SAVE REGISTERS
0000165A 53                     <1>     push   ebx ; push bx
0000165B 51                     <1>     push   ecx
0000165C 88C7                   <1>     mov    bh, al               ; SAVE TRANSMITTED BYTE FOR RETRIES
0000165E B303                   <1>     mov    bl, 3                ; LOAD RETRY COUNT
                                <1> SD0:
00001660 FA                     <1>     cli                         ; DISABLE INTERRUPTS
00001661 8025[FB670000]CF       <1>     and    byte [KB_FLAG_2], ~(KB_FE+KB_FA) ; CLEAR ACK AND RESEND FLAGS
                                <1>     ;
                                <1>     ;----- WAIT FOR COMMAND TO BE ACCEPTED
00001668 B900000100             <1>     mov    ecx, 10000h          ; MAXIMUM WAIT COUNT
                                <1> SD5:
0000166D E464                   <1>     in     al, STATUS_PORT              ; READ KEYBOARD PROCESSOR STATUS PORT
0000166F A802                   <1>     test   al, INPT_BUF_FULL    ; CHECK FOR ANY PENDING COMMAND
00001671 E0FA                   <1>     loopnz SD5                  ; WAIT FOR COMMAND TO BE ACCEPTED
                                <1>     ;
00001673 88F8                   <1>     mov    al, bh               ; REESTABLISH BYTE TO TRANSMIT
00001675 E660                   <1>     out    PORT_A, al           ; SEND BYTE
00001677 FB                     <1>     sti                         ; ENABLE INTERRUPTS
                                <1>     ;mov   cx, 01A00h           ; LOAD COUNT FOR 10 ms+
00001678 B9FFFF0000             <1>     mov    ecx, 0FFFFh
                                <1> SD1:
0000167D F605[FB670000]30       <1>     test   byte [KB_FLAG_2], KB_FE+KB_FA ; SEE IF EITHER BIT SET
00001684 750F                   <1>     jnz    short SD3            ; IF SET, SOMETHING RECEIVED GO PROCESS
00001686 E2F5                   <1>     loop   SD1                  ; OTHERWISE WAIT
                                <1> SD2:
00001688 FECB                   <1>     dec    bl                   ; DECREMENT RETRY COUNT
0000168A 75D4                   <1>     jnz    short SD0            ; RETRY TRANSMISSION
0000168C 800D[FB670000]80       <1>     or     byte [KB_FLAG_2], KB_ERR ; TURN ON TRANSMIT ERROR FLAG
00001693 EB09                   <1>     jmp    short SD4            ; RETRIES EXHAUSTED FORGET TRANSMISSION
                                <1> SD3:
00001695 F605[FB670000]10       <1>     test   byte [KB_FLAG_2], KB_FA ; SEE IF THIS IS AN ACKNOWLEDGE
0000169C 74EA                   <1>     jz     short SD2            ; IF NOT, GO RESEND
```

```
                                <1> SD4:
0000169E 59                     <1>     pop     ecx                     ; RESTORE REGISTERS
0000169F 5B                     <1>     pop     ebx ; pop bx
000016A0 58                     <1>     pop     eax ; pop ax
000016A1 C3                     <1>     retn                            ; RETURN, GOOD TRANSMISSION
                                <1>
                                <1> SND_LED:
                                <1>     ; ---------------------------------------------------------------------------------
                                <1>     ; SND_LED
                                <1>     ;       THIS ROUTINES TURNS ON THE MODE INDICATORS.
                                <1>     ;
                                <1>     ;---------------------------------------------------------------------------------
                                <1>     ;
000016A2 FA                     <1>     cli                             ; TURN OFF INTERRUPTS
000016A3 F605[FB670000]40       <1>     test    byte [KB_FLAG_2], KB_PR_LED ; CHECK FOR MODE INDICATOR UPDATE
000016AA 755F                   <1>     jnz     short SL1               ; DON'T UPDATE AGAIN IF UPDATE UNDERWAY
                                <1>     ;
000016AC 800D[FB670000]40       <1>     or      byte [KB_FLAG_2], KB_PR_LED ; TURN ON UPDATE IN PROCESS
000016B3 B020                   <1>     mov     al, EOI                 ; END OF INTERRUPT COMMAND
000016B5 E620                   <1>     out     20h, al ;out INTA00, al       ; SEND COMMAND TO INTERRUPT CONTROL PORT
000016B7 EB11                   <1>     jmp     short SL0               ; GO SEND MODE INDICATOR COMMAND
                                <1> SND_LED1:
000016B9 FA                     <1>     cli                             ; TURN OFF INTERRUPTS
000016BA F605[FB670000]40       <1>     test    byte [KB_FLAG_2], KB_PR_LED ; CHECK FOR MODE INDICATOR UPDATE
000016C1 7548                   <1>     jnz     short SL1               ; DON'T UPDATE AGAIN IF UPDATE UNDERWAY
                                <1>     ;
000016C3 800D[FB670000]40       <1>     or      byte [KB_FLAG_2], KB_PR_LED ; TURN ON UPDATE IN PROCESS
                                <1> SL0:
000016CA B0ED                   <1>     mov     al, LED_CMD             ; LED CMD BYTE
000016CC E888FFFFFF             <1>     call    SND_DATA                ; SEND DATA TO KEYBOARD
000016D1 FA                     <1>     cli
000016D2 E836000000             <1>     call    MAKE_LED                ; GO FORM INDICATOR DATA BYTE
000016D7 8025[FB670000]F8       <1>     and     byte [KB_FLAG_2], 0F8h; ~KB_LEDS ; CLEAR MODE INDICATOR BITS
000016DE 0805[FB670000]         <1>     or      [KB_FLAG_2], al         ; SAVE PRESENT INDICATORS FOR NEXT TIME
000016E4 F605[FB670000]80       <1>     test    byte [KB_FLAG_2], KB_ERR ; TRANSMIT ERROR DETECTED
000016EB 750F                   <1>     jnz     short SL2               ; IF SO, BYPASS SECOND BYTE TRANSMISSION
                                <1>     ;
000016ED E867FFFFFF             <1>     call    SND_DATA                ; SEND DATA TO KEYBOARD
000016F2 FA                     <1>     cli                             ; TURN OFF INTERRUPTS
000016F3 F605[FB670000]80       <1>     test    byte [KB_FLAG_2], KB_ERR ; TRANSMIT ERROR DETECTED
000016FA 7408                   <1>     jz      short SL3               ; IF NOT, DON'T SEND AN ENABLE COMMAND
                                <1> SL2:
000016FC B0F4                   <1>     mov     al, KB_ENABLE           ; GET KEYBOARD CSA ENABLE COMMAND
000016FE E856FFFFFF             <1>     call    SND_DATA                ; SEND DATA TO KEYBOARD
00001703 FA                     <1>     cli                             ; TURN OFF INTERRUPTS
                                <1> SL3:
00001704 8025[FB670000]3F       <1>     and     byte [KB_FLAG_2], ~(KB_PR_LED+KB_ERR) ; TURN OFF MODE INDICATOR
                                <1> SL1:                                ; UPDATE AND TRANSMIT ERROR FLAG
0000170B FB                     <1>     sti                             ; ENABLE INTERRUPTS
0000170C C3                     <1>     retn                            ; RETURN TO CALLER
                                <1>
                                <1> MAKE_LED:
                                <1>     ;---------------------------------------------------------------------------------
                                <1>     ; MAKE_LED
                                <1>     ;       THIS ROUTINES FORMS THE DATA BYTE NECESSARY TO TURN ON/OFF
                                <1>     ;       THE MODE INDICATORS.
                                <1>     ;---------------------------------------------------------------------------------
                                <1>     ;
                                <1>     ;push   cx                      ; SAVE CX
0000170D A0[F9670000]           <1>     mov     al, [KB_FLAG]           ; GET CAPS & NUM LOCK INDICATORS
00001712 2470                   <1>     and     al, CAPS_STATE+NUM_STATE+SCROLL_STATE ; ISOLATE INDICATORS
                                <1>     ;mov    cl, 4                   ; SHIFT COUNT
                                <1>     ;rol    al, cl                  ; SHIFT BITS OVER TO TURN ON INDICATORS
00001714 C0C004                 <1>     rol     al, 4 ; 20/02/2015
00001717 2407                   <1>     and     al, 07h                 ; MAKE SURE ONLY MODE BITS ON
                                <1>     ;pop    cx
00001719 C3                     <1>     retn                            ; RETURN TO CALLER
                                <1>
                                <1> ; % include 'kybdata.s'   ; KEYBOARD DATA
                                <1>
                                <1>
                                <1> ; /// End Of KEYBOARD FUNCTIONS ///

                                    %include 'video.s' ; 07/03/2015
                                <1> ; ***************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.10 - video.s
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Last Update: 10/02/2026 (Previous: 29/11/2023 - Kernel v2.0.7)
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Beginning: 16/01/2016
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Turkish Rational DOS
                                <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                                <1> ;
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; video.inc (13/08/2015)
                                <1> ;
                                <1> ; Derived from 'IBM PC-AT' BIOS source code (1985)
                                <1> ; ***************************************************************************
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - VIDEO.INC
                                <1> ; Last Modification: 13/08/2015
                                <1> ;               (Video Data is in 'VIDATA.INC')
                                <1> ;
                                <1> ; ///////// VIDEO (CGA) FUNCTIONS ///////////////
                                <1>
                                <1> ; 16/01/2016 (32 bit modifications, TRDOS386 - TRDOS v2.0, video.s)
                                <1> ; INT 31H (TRDOS 386) = INT 10H (IBM PC/AT REAL MODE)
                                <1>
                                <1> ; IBM PC-AT BIOS Source Code
                                <1> ; TITLE VIDEO1 --- 06/10/85 VIDEO DISPLAY BIOS
                                <1>
                                <1> _int10h:
                                <1>     ; 23/03/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
0000171A 9C                     <1>     pushfd
0000171B 0E                     <1>     push    cs
0000171C E851000000             <1>     call    VIDEO_IO_1
00001721 C3                     <1>     retn
                                <1>
                                <1> ;--- INT 10 H ----------------------------------------------------------------
                                <1> ; VIDEO_IO                                                          :
                                <1> ;   THESE ROUTINES PROVIDE THE CRT DISPLAY INTERFACE                :
                                <1> ;   THE FOLLOWING FUNCTIONS ARE PROVIDED:                                   :
                                <1> ;                                                                   :
                                <1> ;    (AH)= 00H     SET MODE (AL) CONTAINS MODE VALUE                        :
                                <1> ;          (AL) = 00H  40X25 BW MODE (POWER ON DEFAULT)             :
                                <1> ;          (AL) = 01H  40X25 COLOR                                  :
                                <1> ;          (AL) = 02H  80X25 BW                                     :
                                <1> ;          (AL) = 03H  80X25 COLOR                                  :
                                <1> ;                        GRAPHICS MODES                             :
                                <1> ;          (AL) = 04H  320X200 COLOR                                :
```

```
<1> ;           (AL) = 05H  320X200 BW MODE                                 :
<1> ;           (AL) = 06H  640X200 BW MODE                                 :
<1> ;           (AL) = 07H   80X25 MONOCHROME (USED INTERNAL TO VIDEO ONLY)       :
<1> ;           *** NOTES -BW MODES OPERATE SAME AS COLOR MODES, BUT COLOR :
<1> ;                      BURST IS NOT ENABLED                                   :
<1> ;                     -CURSOR IS NOT DISPLAYED IN GRAPHICS MODE        :
<1> ;    (AH)= 01H     SET CURSOR TYPE                                                  :
<1> ;           (CH) = BITS 4-0 = START LINE FOR CURSOR                         :
<1> ;                  ** HARDWARE WILL ALWAYS CAUSE BLINK                 :
<1> ;                  ** SETTING BIT 5 OR 6 WILL CAUSE ERRATIC BLINKING   :
<1> ;                     OR NO CURSOR AT ALL                              :
<1> ;           (CL) = BITS 4-0 = END LINE FOR CURSOR                           :
<1> ;    (AH)= 02H     SET CURSOR POSITION                                     :
<1> ;           (DH,DL) = ROW,COLUMN  (00H,00H) IS UPPER LEFT                   :
<1> ;           (BH) = A PAGE NUMBER (MUST BE 00H FOR GRAPHICS MODES)           :
<1> ;    (AH)= 03H     READ CURSOR POSITION                                    :
<1> ;           (BH) = PAGE NUMBER (MUST BE 00H FOR GRAPHICS MODES)        :
<1> ;           ON EXIT (DH,DL) = ROW,COLUMN OF CURRENT CURSOR                  :
<1> ;                   (CH,CL) = CURSOR MODE CURRENTLY SET                :
<1> ;    (AH)= 04H     READ LIGHT PEN POSITION                                          :
<1> ;           ON EXIT:                                                   :
<1> ;           (AH) = 00H -- LIGHT PEN SWITCH NOT DOWN/NOT TRIGGERED           :
<1> ;           (AH) = 01H -- VALID LIGHT PEN VALUE IN REGISTERS           :
<1> ;                   (DH,DL) = ROW,COLUMN OF CHARACTER LP POSITION           :
<1> ;                   (CH) = RASTER LINE (0-199)                         :
<1> ;                   (BX) = PIXEL COLUMN (0-319,639)                         :
<1> ;    (AH)= 05H     SELECT ACTIVE DISPLAY PAGE (VALID ONLY FOR ALPHA MODES)          :
<1> ;           (AL) = NEW PAGE VALUE (0-7 FOR MODES 0&1, 0-3 FOR MODES 2&3)    :
<1> ;    (AH)= 06H     SCROLL ACTIVE PAGE UP                                   :
<1> ;           (AL) = NUMBER OF LINES. ( LINES BLANKED AT BOTTOM OF WINDOW )   :
<1> ;                   (AL) = 00H MEANS BLANK ENTIRE WINDOW               :
<1> ;           (CH,CL) = ROW,COLUMN OF UPPER LEFT CORNER OF SCROLL        :
<1> ;           (DH,DL) = ROW,COLUMN OF LOWER RIGHT CORNER OF SCROLL            :
<1> ;           (BH) = ATTRIBUTE TO BE USED ON BLANK LINE                  :
<1> ;    (AH)= 07H     SCROLL ACTIVE PAGE DOWN                                          :
<1> ;           (AL) = NUMBER OF LINES, INPUT LINES BLANKED AT TOP OF WINDOW    :
<1> ;                   (AL) = 00H MEANS BLANK ENTIRE WINDOW               :
<1> ;           (CH,CL) = ROW,COLUMN OF UPPER LEFT CORNER OF SCROLL        :
<1> ;           (DH,DL) = ROW,COLUMN OF LOWER RIGHT CORNER OF SCROLL            :
<1> ;           (BH) = ATTRIBUTE TO BE USED ON BLANK LINE                  :
<1> ;                                                                      :
<1> ;   CHARACTER HANDLING ROUTINES                                             :
<1> ;                                                                      :
<1> ;    (AH)= 08H     READ ATTRIBUTE/CHARACTER AT CURRENT CURSOR POSITION     :
<1> ;           (BH) = DISPLAY PAGE (VALID FOR ALPHA MODES ONLY)           :
<1> ;           ON EXIT:                                                   :
<1> ;           (AL) = CHAR READ                                           :
<1> ;           (AH) = ATTRIBUTE OF CHARACTER READ (ALPHA MODES ONLY)           :
<1> ;    (AH)= 09H     WRITE ATTRIBUTE/CHARACTER AT CURRENT CURSOR POSITION             :
<1> ;           (BH) = DISPLAY PAGE (VALID FOR ALPHA MODES ONLY)           :
<1> ;           (CX) = COUNT OF CHARACTERS TO WRITE                        :
<1> ;           (AL) = CHAR TO WRITE                                       :
<1> ;           (BL) = ATTRIBUTE OF CHARACTER (ALPHA)/COLOR OF CHAR (GRAPHICS)  :
<1> ;                    SEE NOTE ON WRITE DOT FOR BIT 7 OF BL = 1.        :
<1> ;    (AH) = 0AH    WRITE CHARACTER ONLY AT CURRENT CURSOR POSITION                  :
<1> ;           (BH) = DISPLAY PAGE (VALID FOR ALPHA MODES ONLY)           :
<1> ;           (CX) = COUNT OF CHARACTERS TO WRITE                        :
<1> ;           (AL) = CHAR TO WRITE                                       :
<1> ;                  NOTE: USE FUNCTION (AH)= 09H IN GRAPHICS MODES          :
<1> ;   FOR READ/WRITE CHARACTER INTERFACE WHILE IN GRAPHICS MODE, THE          :
<1> ;           CHARACTERS ARE FORMED FROM A CHARACTER GENERATOR IMAGE          :
<1> ;           MAINTAINED IN THE SYSTEM ROM. ONLY THE 1ST 128 CHARS            :
<1> ;           ARE CONTAINED THERE. TO READ/WRITE THE SECOND 128 CHARS,   :
<1> ;           THE USER MUST INITIALIZE THE POINTER AT INTERRUPT 1FH           :
<1> ;           (LOCATION 0007CH) TO POINT TO THE 1K BYTE TABLE CONTAINING :
<1> ;           THE CODE POINTS FOR THE SECOND 128 CHARS (128-255).        :
<1> ;   FOR WRITE CHARACTER INTERFACE IN GRAPHICS MODE, THE REPLICATION FACTOR   :
<1> ;           CONTAINED IN (CX) ON ENTRY WILL PRODUCE VALID RESULTS ONLY :
<1> ;           FOR CHARACTERS CONTAINED ON THE SAME ROW. CONTINUATION TO  :
<1> ;           SUCCEEDING LINES WILL NOT PRODUCE CORRECTLY.               :
<1> ;                                                                      :
<1> ;    GRAPHICS INTERFACE                                                      :
<1> ;    (AH)= 0BH     SET COLOR PALETTE                                       :
<1> ;           (BH) = PALETTE COLOR ID BEING SET (0-127)                  :
<1> ;           (BL) = COLOR VALUE TO BE USED WITH THAT COLOR ID           :
<1> ;                  NOTE: FOR THE CURRENT COLOR CARD, THIS ENTRY POINT HAS   :
<1> ;                          MEANING ONLY FOR 320X200 GRAPHICS.         :
<1> ;                  COLOR ID = 0 SELECTS THE BACKGROUND COLOR (0-15)        :
<1> ;                  COLOR ID = 1 SELECTS THE PALETTE TO BE USED:       :
<1> ;                          0 = GREEN(1)/RED(2)/YELLOW(3)              :
<1> ;                          1 = CYAN(1)/MAGENTA(2)/WHITE(3)                 :
<1> ;                  IN 40X25 OR 80X25 ALPHA MODES, THE VALUE SET FOR   :
<1> ;                          PALETTE COLOR 0 INDICATES THE BORDER COLOR :
<1> ;                          TO BE USED (VALUES 0-31, WHERE 16-31 SELECT :
<1> ;                          THE HIGH INTENSITY BACKGROUND SET.         :
<1> ;    (AH)= 0CH     WRITE DOT                                               :
<1> ;           (DX) = ROW NUMBER                                          :
<1> ;           (CX) = COLUMN NUMBER                                       :
<1> ;           (AL) = COLOR VALUE                                         :
<1> ;                   IF BIT 7 OF AL = 1, THEN THE COLOR VALUE IS EXCLUSIVE    :
<1> ;                   ORed WITH THE CURRENT CONTENTS OF THE DOT          :
<1> ;    (AH)= 0DH     READ DOT                                                :
<1> ;           (DX) = ROW NUMBER                                          :
<1> ;           (CX) = COLUMN NUMBER                                       :
<1> ;           (AL) = RETURNS THE DOT READ                                :
<1> ;                                                                      :
<1> ;   ASCII TELETYPE ROUTINE FOR OUTPUT                                       :
<1> ;                                                                      :
<1> ;    (AH)= 0EH     WRITE TELETYPE TO ACTIVE PAGE                           :
<1> ;           (AL) = CHAR TO WRITE                                       :
<1> ;           (BL) = FOREGROUND COLOR IN GRAPHICS MODE                   :
<1> ;           NOTE -- SCREEN WIDTH IS CONTROLLED BY PREVIOUS MODE SET         :
<1> ;    (AH)= 0FH     CURRENT VIDEO STATE                                     :
<1> ;           RETURNS THE CURRENT VIDEO STATE                                 :
<1> ;           (AL) = MODE CURRENTLY SET ( SEE (AH)=00H FOR EXPLANATION)  :
<1> ;           (AH) = NUMBER OR CHARACTER COLUMNS ON SCREEN               :
<1> ;           (BH) = CURRENT ACTIVE DISPLAY PAGE                         :
<1> ;    (AH)= 10H     RESERVED                                                :
<1> ;    (AH)= 11H     RESERVED                                                :
<1> ;    (AH)= 12H     RESERVED                                                :
<1> ;    (AH)= 13H     WRITE STRING                                            :
<1> ;                  ES:BP  -  POINTER T0 STRING TO BE WRITTEN          :
<1> ;                  CX     -  LENGTH OF CHARACTER STRING TO WRITTEN         :
<1> ;                  DX     -  CURSOR POSITION FOR STRING TO BE WRITTEN :
<1> ;                  BH     -  PAGE NUMBER                              :
<1> ;           (AL)= 00H      WRITE CHARACTER STRING                      :
<1> ;                  BL     -  ATTRIBUTE                                :
<1> ;                  STRING IS  <CHAR,CHAR, ... ,CHAR>                  :
<1> ;                  CURSOR NOT MOVED                                   :
<1> ;           (AL)= 01H      WRITE CHARACTER STRING AND MOVE CURSOR          :
<1> ;                  BL     -  ATTRIBUTE                                :
<1> ;                  STRING IS  <CHAR,CHAR, ... ,CHAR>                  :
<1> ;                  CURSOR MOVED                                       :
<1> ;           (AL)= 02H      WRITE CHARACTER AND ATTRIBUTE STRING        :
```

```
                                <1> ;                       (VALID FOR ALPHA MODES ONLY)                :
                                <1> ;                 STRING IS <CHAR,ATTR,CHAR,ATTR ..  ,CHAR,ATTR>            :
                                <1> ;                 CURSOR IS NOT MOVED                         :
                                <1> ;          (AL)= 03H WRITE CHARACTER AND ATTRIBUTE STRING AND MOVE CURSOR    :
                                <1> ;                       (VALID FOR ALPHA MODES ONLY)                :
                                <1> ;                 STRING IS <CHAR,ATTR,CHAR,ATTR ..  ,CHAR,ATTR>            :
                                <1> ;                 CURSOR IS MOVED                             :
                                <1> ;           NOTE:  CARRIAGE RETURN, LINE FEED, BACKSPACE, AND BELL ARE       :
                                <1> ;                  TREATED AS COMMANDS RATHER THAN PRINTABLE CHARACTERS.     :
                                <1> ;                                                               :
                                <1> ;   BX,CX,DX,SI,DI,BP,SP,DS,ES,SS PRESERVED DURING CALLS EXCEPT FOR           :
                                <1> ;   BX,CX,DX RETURN VALUES ON FUNCTIONS 03H,04H,0DH AND 0FH. ON ALL CALLS     :
                                <1> ;   AX IS MODIFIED.                                                          :
                                <1> ;--------------------------------------------------------------------------
                                <1>
00001722 [361B0000]             <1> M1: dd      SET_MODE        ; TABLE OF ROUTINES WITHIN VIDEO I/O
00001726 [F51E0000]             <1>     dd      SET_CTYPE
0000172A [231F0000]             <1>     dd      SET_CPOS
0000172E [471F0000]             <1>     dd      READ_CURSOR
                                <1>     ;dd     VIDEO_RETURN    ; READ_LPEN
00001732 [341B0000]             <1>     dd      set_mode_ncm    ; Set mode without clearing video memory
00001736 [951F0000]             <1>     dd      ACT_DISP_PAGE
0000173A [1F200000]             <1>     dd      SCROLL_UP
0000173E [36210000]             <1>     dd      SCROLL_DOWN
00001742 [B5210000]             <1>     dd      READ_AC_CURRENT
00001746 [12220000]             <1>     dd      WRITE_AC_CURRENT
0000174A [38220000]             <1>     dd      WRITE_C_CURRENT
0000174E [402B0000]             <1>     dd      SET_COLOR
00001752 [A82B0000]             <1>     dd      WRITE_DOT
00001756 [772B0000]             <1>     dd      READ_DOT
0000175A [C2220000]             <1>     dd      WRITE_TTY
0000175E [1C1B0000]             <1>     dd      VIDEO_STATE
00001762 [75360000]             <1>     dd      vga_pal_funcs   ; 10/08/2016 (TRDOS 386)
00001766 [8D300000]             <1>     dd      font_setup      ; 10/07/2016 (TRDOS 386)
0000176A [6B1B0000]             <1>     dd      VIDEO_RETURN    ; RESERVED
0000176E [38240000]             <1>     dd      WRITE_STRING    ; 23/06/2016 (TRDOS 386)
                                <1> M1L EQU     $ - M1
                                <1>
                                <1> ; 10/02/2026 - TRDOS 386 v2.0.10
                                <1> ; BugFix note:
                                <1> ; cr3 is changed to Kernel's Page Directory address in int 31h.
                                <1> ; Because, 'transfer_to_user_buffer' uses 'get_physical_addr_x'
                                <1> ; uses 'get_pde' uses 'get_pde' .. and 'get_pde' causes
                                <1> ; page fault error because of "mov eax, [edx]" code in it while
                                <1> ; edx contains an address at the beyond of the 1st 4MB.
                                <1> ; (Before this Bugfix, 'VGA_func_std' part of int 31h was running
                                <1> ; with user's page directry in cr3. (That was no problem if
                                <1> ; all page tables would be in the 1st 4MB. But if the RAM is larger
                                <1> ; than 2.5GB, page tables extent at the beyond of the 1st 4MB.)
                                <1> ; As result of the BugFix today, size of 4GB-LFBaddress RAM will be
                                <1> ; available to use.
                                <1> ;
                                <1> ;; Note: [cr3prev] is used for int 31h video interrupt and [cr3reg]
                                <1> ;; is used for timer -or RTC- interrupt.
                                <1> ;; (This separation is needed because timer interrupt can occur
                                <1> ;;  while video interrupt handler is in progress.)
                                <1>
                                <1>
                                <1> ; 22/12/2025
                                <1> ; 21/12/2025 - TRDOS 386 v2.0.10
                                <1> ; 29/11/2023 - TRDOS 386 v2.0.7
                                <1> ;   [VESA VBE3-PMI functions]
                                <1> ;   (fixing page table problems for LFB/PMI for >2.5GB main memory)
                                <1> ;   ((Addresses of kernel page tables and PMI videobios address
                                <1> ;     must be < 4MB or cr3 register must contain kernel's page dir
                                <1> ;     during video interrupt. Save-change-restore cr3 register
                                <1> ;     content here is a bugfix method to prevent page faults
                                <1> ;     for -real- computers with >2.5GB main memory.))
                                <1> ;    {INT 31h was not changing cr3 while it contains user's
                                <1> ;     page directory addr and accessing beyond of the 1st 4MB was
                                <1> ;     causing to page faults.. because, after the 1st 4MB, user's
                                <1> ;     page tables contain non-linear/virtual memory pages.}
                                <1> ;
                                <1> ; 02/08/2022 - TRDOS 386 v2.0.5
                                <1> ; 06/12/2020
                                <1> ; 05/12/2020
                                <1> ; 03/12/2020
                                <1> ; 27/11/2020 - TRDOS 386 v2.0.3
                                <1> ; 14/01/2017
                                <1> ; 02/01/2017
                                <1> ; 04/07/2016
                                <1> ; 12/05/2016
                                <1> ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> int31h:  ; Video BIOS
                                <1>
                                <1> ; BH = Video page number
                                <1> ; BL = Color/Attribute
                                <1> ; AH = Function number
                                <1> ; AL = Character
                                <1>
                                <1> VIDEO_IO_1:
                                <1>     ;sti                            ; INTERRUPTS BACK ON
00001772 FC                     <1>     cld                             ; SET DIRECTION FORWARD
                                <1>
                                <1>     ;cmp    ah, M1L/4               ; TEST FOR WITHIN TABLE RANGE
                                <1>     ;jnb    short M4                ; BRANCH TO EXIT IF NOT A VALID COMMAND
                                <1>
                                <1>     ; 26/11/2020
00001773 80FC14                 <1>     cmp     ah, M1L/4
00001776 7205                   <1>     jb      short VGA_func
                                <1>
00001778 80FC4F                 <1>     cmp     ah, 4Fh
0000177B 7546                   <1>     jne     short M4 ; invalid !
                                <1>
                                <1> VGA_func: ; 26/11/2020
0000177D 06                     <1>     push    es ; *
0000177E 1E                     <1>     push    ds ; **                 ; SAVE WORK AND PARAMETER REGISTERS
                                <1>
                                <1>     ; 26/11/2020
0000177F 50                     <1>     push    eax ; -
                                <1>
00001780 66B81000               <1>     mov     ax, KDATA               ; POINT DS: TO DATA SEGMENT
00001784 8ED8                   <1>     mov     ds, ax
00001786 8EC0                   <1>     mov     es, ax
                                <1>
                                <1>     ; 10/02/2026 - !!! BugFix !!!
00001788 0F20D8                 <1>     mov     eax, cr3
0000178B A3[60770100]           <1>     mov     [cr3prev], eax ; save current cr3 register value/content
00001790 A1[B0760100]           <1>     mov     eax, [k_page_dir]
00001795 0F22D8                 <1>     mov     cr3, eax
                                <1>
                                <1>     ; 26/11/2020
00001798 58                     <1>     pop     eax ; +
                                <1>     ;
```

```
00001799 FB                  <1>     sti
0000179A 80FC4F              <1>     cmp     ah, 4Fh
                             <1>     ;je     short VBE_func
                             <1>     ; 21/12/2025
0000179D 0F84BC000000        <1>     je      VBE_func
                             <1>
                             <1>     ; 04/12/2020
000017A3 A3[3C830100]        <1>     mov     [video_eax], eax
                             <1>
                             <1>     ; 21/12/2020
000017A8 803D[2E680000]FF    <1>     cmp     byte [CRT_MODE], 0FFh ; Current mode is a VESA VBE mode ?
000017AF 7213                <1>     jb      short VGA_func_std
                             <1>
000017B1 08E4                <1>     or      ah, ah                ; set mode ?
000017B3 750F                <1>     jnz     short VGA_func_std    ; no
                             <1>
000017B5 803D[7C090000]03    <1>     cmp     byte [vbe3], 3        ; (real) VESA VBE 3 video bios ?
000017BC 7506                <1>     jne     short VGA_func_std    ; no
                             <1>
000017BE E9CF010000          <1>     jmp     vesa_vbe3_pmi
                             <1>
                             <1>     ; 21/12/2020
                             <1> M4:                               ; COMMAND NOT VALID
000017C3 CF                  <1>     iretd                         ; DO NOTHING IF NOT IN VALID RANGE
                             <1>
                             <1> VGA_func_std:
                             <1>     ; 06/12/2020
                             <1>     ; 03/12/2020
000017C4 80FC0F              <1>     cmp     ah, 0Fh
                             <1>     ;ja     short VGA_funcs_0     ; only CGA funcs will be handled by
                             <1>     ; 21/12/2025
000017C7 777F                <1>     ja      VGA_funcs_0
                             <1>
                             <1>     ; 06/12/2020                  ; vga bios firmware
                             <1>     ; 03/12/2020
                             <1>     ;test   ah, 7Fh ; set mode ?
                             <1>     ;;or    ah, ah                ; only 'set mode' will be handled by
                             <1>     ;jnz    short VGA_funcs_0     ; vga bios firmware
                             <1>     ;jz     short vbe_pmi32_0
                             <1>
                             <1>     ; 28/11/2020
000017C9 803D[249D0100]00    <1>     cmp     byte [pmi32], 0               ; 32 bit protected mode interface for
000017D0 7676                <1>     jna     short VGA_funcs_0     ; video hardware's vga bios firmware
                             <1>                                   ; ([pmi32] > 0 if it is activated)
                             <1>                                   ; note:
                             <1>                                   ; [vbe3] = 3 is required to activate
                             <1>     ; 07/12/2020
000017D2 20E4                <1>     and     ah, ah ; is this set mode ?
000017D4 7413                <1>     jz      short vbe_pmi32_2 ; yes
                             <1>
000017D6 80FC04              <1>     cmp     ah, 04h  ; set mode ('no clear memory' option)
000017D9 746D                <1>     je      short VGA_funcs_0
                             <1>
                             <1>     ; 07/12/2020
000017DB 803D[2E680000]07    <1>     cmp     byte [CRT_MODE], 7 ; current mode > 7 ?
000017E2 7664                <1>     jna     short VGA_funcs_0  ; no
                             <1>
                             <1>     ; when mode 3 is active,
                             <1>     ; video bios functions are not redirected
                             <1>     ; to VESA VBE3 PMI except 'set mode' function
                             <1>
                             <1> vbe_pmi32_0:
                             <1>     ; 06/12/2020
                             <1>     ;or     ah, ah
                             <1>     ;jnz    short vbe_pmi32_2
                             <1>     ; ah = 0 ; 'set mode'
                             <1>     ;cmp    al, 3 ; 80x25 text mode, 16 colors, default mode for MainProg
                             <1>     ;jne    short vbe_pmi32_1
                             <1>     ;cmp    byte [CRT_MODE], 3 ; If current video mode <> 3 and requested
                             <1>     ;                     ; video mode is 3, use internal 'set mode';
                             <1>     ;                     ; otherwise, use vesa vbe 3 bios's 'set mode'.
                             <1>     ;jne    short VGA_funcs_0
                             <1> vbe_pmi32_1:
000017E4 E9A9010000          <1>     jmp     vesa_vbe3_pmi
                             <1>
                             <1> ;vbe_pmi32_2:
                             <1>     ;cmp    ah, 04h ; set mode (no clear mem option)
                             <1>     ;jne    short vbe_pmi32_1
                             <1>
                             <1> vbe_pmi32_2:
                             <1>
                             <1>     ; 22/12/2025
                             <1>     ; 21/12/2025 - TRDOS 386 v2.0.10
                             <1>     ; set character height (needed) for next system calls
                             <1>     ;;;;
                             <1> vbe_pmi32_3:
000017E9 56                  <1>     push    esi
000017EA 50                  <1>     push    eax
000017EB 88C4                <1>     mov     ah, al
000017ED 80E47F              <1>     and     ah, 7Fh
000017F0 BE[4A680000]        <1>     mov     esi, vga_modes
                             <1> vbe_pmi32_3_1:
000017F5 AC                  <1>     lodsb
000017F6 38C4                <1>     cmp     ah, al
000017F8 7408                <1>     je      short vbe_pmi32_3_2
000017FA 81FE[5A680000]      <1>     cmp     esi, vga_modes+vga_mode_count
00001800 72F3                <1>     jb      short vbe_pmi32_3_1
                             <1> vbe_pmi32_3_2:
00001802 81EE[4A680000]      <1>     sub     esi, vga_modes
00001808 C1E602              <1>     shl     esi, 2  ; dword
0000180B 81C6[5A680000]      <1>     add     esi, vga_mode_tbl_ptr
00001811 8B36                <1>     mov     esi, [esi]
00001813 66AD                <1>     lodsw
00001815 A2[30680000]        <1>     mov     [CRT_COLS], al
0000181A FEC4                <1>     inc     ah
0000181C 8825[36680000]      <1>     mov     [VGA_ROWS], ah
00001822 8A06                <1>     mov     al, [esi]
                             <1>     ;lodsb
00001824 A2[32680000]        <1>     mov     [CHAR_HEIGHT], al
                             <1> vbe_pmi32_4:
00001829 58                  <1>     pop     eax
0000182A 5E                  <1>     pop     esi
                             <1>     ;;;;
                             <1>
                             <1>     ; 07/12/2020
0000182B 803D[2E680000]07    <1>     cmp     byte [CRT_MODE], 7 ; current mode > 7 ?
00001832 77B0                <1>     ja      short vbe_pmi32_1  ; yes
                             <1>
00001834 3C07                <1>     cmp     al, 7  ; requested mode > 7 ?
00001836 7610                <1>     jna     short VGA_funcs_0  ; no (CGA)
                             <1>
00001838 3C13                <1>     cmp     al, 13h
0000183A 76A8                <1>     jna     short vbe_pmi32_1
                             <1>
```

```
0000183C A880                <1>     test    al, 80h
0000183E 7408                <1>     jz      short VGA_funcs_0  ; unknown or special
                             <1>
00001840 3C87                <1>     cmp     al, 87h ; requested mode > 7 ?
                             <1>                     ; (with no clear mem ops)
00001842 7604                <1>     jna     short VGA_funcs_0  ; no (CGA)
                             <1>
00001844 3C93                <1>     cmp     al, 93h
00001846 769C                <1>     jna     short vbe_pmi32_1
                             <1>
                             <1>     ; > 13h video modes are unknown or special
                             <1>     ; they must be handled by kernel
                             <1>
                             <1>     ; CGA video modes will be handled by kernel
                             <1>
                             <1> VGA_funcs_0:
00001848 52                  <1>     push    edx ; ***
00001849 51                  <1>     push    ecx ; ****
0000184A 53                  <1>     push    ebx ; *****
0000184B 56                  <1>     push    esi ; ******
0000184C 57                  <1>     push    edi ; *******
0000184D 55                  <1>     push    ebp ; ********
                             <1>
                             <1>     ;mov    [video_eax], eax ; 12/05/2016
0000184E BF00800B00          <1>     mov     edi, 0B8000h           ; GET offset FOR COLOR CARD
                             <1>
                             <1>     ; 23/03/2016
00001853 C0E402              <1>     shl     ah, 2  ; dword         ; TIMES 2 FOR WORD TABLE LOOKUP
00001856 0FB6F4              <1>     movzx   esi, ah                ; MOVE OFFSET INTO LOOK UP REGISTER (SI)
                             <1>     ;mov    ah, [CRT_MODE]         ; MOVE CURRENT MODE INTO (AH) REGISTER
                             <1>
                             <1>     ;;15/01/2017
                             <1>     ; 14/01/2017
                             <1>     ; 02/01/2017
                             <1>     ;;mov  byte [intflg], 31h  ; video interrupt
                             <1>     ;sti ; 26/11/2020
                             <1>     ;
                             <1>
00001859 FFA6[22170000]      <1>     jmp     dword [esi+M1]         ; GO TO SELECTED FUNCTION
                             <1>
                             <1> VBE_func:
                             <1>     ; 26/11/2020
                             <1>     ;sti
0000185F 55                  <1>     push    ebp ; *** ; 27/11/2020
00001860 56                  <1>     push    esi ; ****
                             <1>
                             <1>     ; Note:
                             <1>     ; ebx, ecx, edx, edi, ebp registers
                             <1>     ; must be saved by VBE functions and
                             <1>     ; eax register must be set
                             <1>     ; (according to VBE 3 standard)
                             <1>     ; before return from this interrupt
                             <1>     ; (every function must restore and set
                             <1>     ; registers except esp, esi, es, ds)
                             <1>
                             <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
                             <1>     ; cr3 = [k_page_dir] ; kernel's cr3
                             <1>     ; [cr3prev] = previous (user's) cr3
                             <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                             <1>     ;mov    esi, cr3
                             <1>     ;push   esi ; *****
                             <1>     ;;cmp   esi, [k_page_dir]
                             <1>     ;;je    short VBE_func_x
                             <1>     ;mov    esi, [k_page_dir]
                             <1>     ;mov    cr3, esi
                             <1> ;VBE_func_x:
00001861 803D[7C090000]02    <1>     cmp     byte [vbe3], 2
00001868 7749                <1>     ja      short VESA_VBE3_PMI_CALL ; VBE3 video bios ('PMID')
                             <1>     ;je     short VBE_func_0  ; Bochs/Qemu/VirtualBox emulator
0000186A 7271                <1>     jb      short VBE_unknown ; invalid ([vbe3] = 0)
                             <1>
                             <1>     ;jmp    VESA_VBE3_PMI_CALL
                             <1>
                             <1> VBE_func_0:
                             <1>     ; Bochs/Plex86 VGAbios VBE extension
                             <1>     ; (TRDOS 386 v2.0.3 can use VBE graphics modes on emulators)
                             <1>     ; BOCHS/QEMU/VIRTUALBOX
                             <1>
0000186C 8A25[7D090000]      <1>     mov     ah, [vbe2bios]
00001872 80FCC0              <1>     cmp     ah, 0C0h
00001875 7266                <1>     jb      short VBE_unknown
00001877 80FCC5              <1>     cmp     ah, 0C5h
0000187A 7761                <1>     ja      short VBE_unknown
                             <1>
                             <1>     ; TRDOS 386 is running on BOCHS or QEMU
0000187C B44F                <1>     mov     ah, 4Fh
                             <1> VBE_func_1:
0000187E 0FB6F0              <1>     movzx   esi, al ; VESA VBE function number
                             <1>     ;shl    si, 2 ; dword
                             <1>     ; 02/08/2022
00001881 C1E602              <1>     shl     esi, 2
00001884 6683FE14            <1>     cmp     si, N1L
00001888 7353                <1>     jnb     short VBE_unknown
                             <1>     ;sti
                             <1>
0000188A FF96[9F180000]      <1>     call    dword [esi+N1] ; call VBE function
                             <1>
                             <1>     ;jmp    short VBE_bios_return
                             <1>
                             <1> VBE_bios_return:
00001890 FA                  <1>     cli
                             <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
00001891 8B35[60770100]      <1>     mov     esi, [cr3prev] ; restore pğrevious (user's) cr3
                             <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                             <1>     ;pop    esi ; *****
                             <1>     ;;cmp   esi, [k_page_dir]
                             <1>     ;;je    short VBE_bios_return_x
00001897 0F22DE              <1>     mov     cr3, esi
                             <1> ;VBE_bios_return_x:
0000189A 5E                  <1>     pop     esi ; ****
0000189B 5D                  <1>     pop     ebp ; *** ; 27/11/2020
0000189C 07                  <1>     pop     es  ; **
0000189D 1F                  <1>     pop     ds  ; *
0000189E CF                  <1>     iretd
                             <1>
                             <1>     ; 26/11/2020
                             <1> N1:
0000189F [00190000]          <1>     dd      vbe_biosfn_return_ctrl_info
000018A3 [2D390000]          <1>     dd      vbe_biosfn_return_mode_info
000018A7 [E7390000]          <1>     dd      vbe_biosfn_set_mode
000018AB [B73A0000]          <1>     dd      vbe_biosfn_return_current_mode
000018AF [D63A0000]          <1>     dd      vbe_biosfn_save_restore_state
                             <1>     ;dd     vbe_biosfn_display_window_ctrl
                             <1>     ;dd     vbe_biosfn_set_get_log_scanline
```

```
                                <1>     ;dd     vbe_biosfn_set_get_disp_start
                                <1>     ;dd     vbe_biosfn_set_get_dac_pal_frm
                                <1>     ;dd     vbe_biosfn_set_get_palette_data
                                <1>
                                <1> N1L EQU     $ - N1
                                <1>
                                <1> VESA_VBE3_PMI_CALL: ; VESA VBE video bios (firmware) functions
                                <1>                     ; by using VESA VBE3 Protected Mode Interface
                                <1>
                                <1>     ; 02/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 29/11/2020
                                <1>     ; 26/11/2020 - TRDOS 386 v2.0.3 video.s
                                <1>
                                <1>     ; We are here because..
                                <1>     ; 'PMID' has been verified by TRDOS 386 v2.0.3 kernel.
                                <1>     ; (Otherwise bochs/plex86 compatible VBE functions and
                                <1>     ;  modes would be used on BOCHS/QEMU/VIRTUALBOX emulators
                                <1>     ;  or only standard/old VGA graphics modes would be used.)
                                <1>
                                <1>     ; (TRDOS 386 v2.0.3 can use VESA VBE graphics modes if
                                <1>     ;  the video bios is full compatible with VBE3 standard)
                                <1>
                                <1>     ; 29/11/2020
                                <1>
000018B3 0FB6F0                 <1>     movzx   esi, al ; VESA VBE 3 function number
                                <1>     ;shl    si, 2 ; dword
                                <1>     ; 02/08/2022
000018B6 C1E602                 <1>     shl     esi, 2
000018B9 6683FE14               <1>     cmp     si, P1L
000018BD 731E                   <1>     jnb     short VBE_unknown
                                <1>     ;sti
                                <1>
000018BF 57                     <1>     push    edi ; *****
                                <1>
000018C0 FF96[C9180000]         <1>     call    dword [esi+P1] ; call VBE 3 function
                                <1>
000018C6 5F                     <1>     pop     edi ; *****
                                <1>
000018C7 EBC7                   <1>     jmp     short VBE_bios_return
                                <1>
                                <1> P1:
000018C9 [E3180000]             <1>     dd      vbe3_pmfn_return_ctrl_info
000018CD [06190000]             <1>     dd      vbe3_pmfn_return_mode_info
000018D1 [42190000]             <1>     dd      vbe3_pmfn_set_mode
000018D5 [3D190000]             <1>     dd      vbe3_pmfn_return_current_mode
000018D9 [401A0000]             <1>     dd      vbe3_pmfn_save_restore_state
                                <1>     ;dd     vbe3_pmfn_display_window_ctrl
                                <1>     ;dd     vbe3_pmfn_set_get_log_scanline
                                <1>     ;dd     vbe3_pmfn_set_get_disp_start
                                <1>     ;dd     vbe3_pmfn_set_get_dac_pal_frm
                                <1>     ;dd     vbe3_pmfn_set_get_palette_data
                                <1>     ;dd     vbe3_pmfn_return_pmi ; invalid for TRDOS 386 v2
                                <1>     ;dd     vbe3_pmfn_set_get_pixel_clock
                                <1>
                                <1> P1L EQU     $ - P1
                                <1>
                                <1> ;   ; 29/11/2020
                                <1> ;   mov     edi, VBE3MODEINFOBLOCK >> 4 ; / 16
                                <1> ;
                                <1> ;   cmp     al, 04h
                                <1> ;   jb      short vbe3_pm_f ; function: 4F00h to 4F03h
                                <1> ;   ja      short vbe3_pmi_f5B ; function: 4F05h to 4F0Bh
                                <1> ;
                                <1> ;   ; check buffer length (must be <= 2048 bytes)
                                <1> ;
                                <1> ;   and     dl, dl ; 0
                                <1> ;   jz      short vbe3_pm_f
                                <1> ;                   ; Return Save/Restore State buffer size
                                <1> ;
                                <1> ;   push    ebx  ; buffer address
                                <1> ;   push    edx  ; function: save (01h) or restore (02h)
                                <1> ;   call
                                <1> ;
                                <1> ;vbe3_pm_f03:
                                <1> ;   cmp     al, 2
                                <1> ;   ja      short vbe3_pm_f ; function 4F03h
                                <1> ;   jb      short vbe3_pm_f1
                                <1> ;
                                <1> ;
                                <1> ;vbe3_pm_f1:
                                <1> ;
                                <1> ;
                                <1> ;vbe3_pmi_f5B:
                                <1> ;   cmp     al, 09h
                                <1> ;   jna     short vbe3_pm_f ; funcs 05h to 09h are usable
                                <1> ;
                                <1> ;   cmp     al, 0Bh ; Get/Set pixel clock, last function
                                <1> ;   jne     short VBE_unknown
                                <1> ;                   ; (do not use 'uncertain' functions
                                <1> ;                   ; because of system-user buff transfers)
                                <1> ;vbe3_pm_f:
                                <1> ;   mov     byte [vbe3_pm_fn], al ; set
                                <1> ;   ; prepare 16 bit pm segments & registers for pmi call
                                <1> ;   call    VESA_VBE3_PM_FUNCTION
                                <1> ;
                                <1> ;
                                <1>
                                <1>     ; 26/11/2020
                                <1> VBE_unknown:
000018DD 66B80001               <1>     mov     ax, 0100h  ; ah = 1 : Function call failed
                                <1>                        ; al = 0 : Function is not supported
                                <1>
000018E1 EBAD                   <1>     jmp     short VBE_bios_return
                                <1>
                                <1> vbe3_pmfn_return_ctrl_info:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 12/12/2020
                                <1>     ;
                                <1>     ; VBE function 4F00h - Return VBE Controller Information
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       EDI = Pointer to buffer in which to place
                                <1>     ;             VbeInfoBlock structure
                                <1>     ;
                                <1>     ;       AX = 4F00h
                                <1>     ; Output:
                                <1>     ;       AX = VBE return status
                                <1>     ;            AX = 004Fh -> succeeded
                                <1>     ;            AX <> 004Fh -> failed
                                <1>     ;
                                <1>     ; Modified registers: eax (+ edi for kernel's own call)
                                <1>
                                <1>     ; NOTE: TRDOS 386 v2 (v2.0.3) kernel calls this function
                                <1>     ; during startup while cpu is in real mode
```

```
                                <1>     ; (by using int 10h, 4F02h) and saves VbeInfoBlock at
                                <1>     ; VBE3INFOBLOCK address (97E00h for TRDOS 386 v2.0.3).
                                <1>     ;
                                <1>     ; So...
                                <1>     ; This VBE function is adjusted to return/move same info
                                <1>     ; from VBE3INFOBLOCK to user's buffer in EDI.
                                <1>
                                <1>     ; int 31h (int 10h) entrance
                                <1>
000018E3 21FF                   <1>     and     edi, edi
000018E5 7423                   <1>     jz      short vbe3_func_fail ; invalid buffer address !
                                <1>
                                <1> ;_vbe3_pmfn_return_ctrl_info:
                                <1>     ;or     edi, edi
                                <1>     ;jnz    short _vbe_biosfn_return_ctrl_info
                                <1>     ;
                                <1>     ;; this option may not be necessary - 12/12/2020
                                <1>     ;
                                <1>     ;; edi = 0, kernel forces to get ctrl info again
                                <1>     ;;         by using VESA VBE3 video bios's pmi
                                <1>     ;
                                <1>     ;;push  edi
                                <1>     ;; far call to VESA VBE3 PMI
                                <1>     ;;mov   ax, 4F00h ; Return VBE Controller Info
                                <1>     ;mov    edi, VBE3INFOBLOCK-VBE3SAVERESTOREBLOCK
                                <1>     ;; ES selector base address = VBE3SAVERESTOREBLOCK
                                <1>     ;call   int10h_32bit_pmi
                                <1>     ;;pop   edi
                                <1>     ;mov    edi, VBE3INFOBLOCK ; retn to the kernel sub
                                <1>     ;cmp    ax, 004Fh
                                <1>     ;je     short vbe_ctrl_info_retn
                                <1>     ;stc
                                <1>     ;retn
                                <1>
                                <1> _vbe_biosfn_return_ctrl_info:
000018E7 57                     <1>     push    edi
000018E8 51                     <1>     push    ecx
000018E9 BE007E0900             <1>     mov     esi, VBE3INFOBLOCK
                                <1>     ;mov    ecx, 512
                                <1>     ; 02/08/2022
000018EE 29C9                   <1>     sub     ecx, ecx
000018F0 B502                   <1>     mov     ch, 2
                                <1>     ; ecx = 512
000018F2 E824F20000             <1>     call    transfer_to_user_buffer
000018F7 59                     <1>     pop     ecx
000018F8 5F                     <1>     pop     edi
000018F9 720F                   <1>     jc      short vbe3_func_fail
                                <1>
000018FB 31C0                   <1>     xor     eax, eax
000018FD B04F                   <1>     mov     al, 4Fh ; successful
                                <1> ;vbe_ctrl_info_retn:
000018FF C3                     <1>     retn
                                <1>
                                <1> vbe_biosfn_return_ctrl_info:
                                <1>     ; 12/12/2020
                                <1>     ;
                                <1>     ; VBE function 4F00h - Return VBE Controller Information
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       EDI = Pointer to buffer in which to place
                                <1>     ;             VbeInfoBlock structure
                                <1>     ;
                                <1>     ;       AX = 4F00h
                                <1>     ; Output:
                                <1>     ;       AX = VBE return status
                                <1>     ;             AX = 004Fh -> succeeded
                                <1>     ;             AX <> 004Fh -> failed
                                <1>     ;
                                <1>     ; Modified registers: eax
                                <1>
00001900 21FF                   <1>     and     edi, edi
00001902 7406                   <1>     jz      short vbe3_func_fail ; invalid buffer addr !
00001904 EBE1                   <1>     jmp     short _vbe_biosfn_return_ctrl_info
                                <1>
                                <1> vbe3_pmfn_return_mode_info:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 21/12/2020
                                <1>     ; 12/12/2020
                                <1>     ;
                                <1>     ; VBE function 4F01h - Return VBE Mode Information
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       CX = Mode number (VESA VBE mode number)
                                <1>     ;       EDI = Pointer to ModeInfoBlock structure
                                <1>     ;             (256 bytes) -User's buffer address-
                                <1>     ;       EDI = 0 -> kernel call
                                <1>     ;                  (do not transfer ModeInfoBlock
                                <1>     ;                  to user's buffer address)
                                <1>     ;       AX = 4F01h
                                <1>     ; Output:
                                <1>     ;       AX = VBE return status
                                <1>     ;             AX = 004Fh -> succeeded
                                <1>     ;             AX <> 004Fh -> failed
                                <1>     ;
                                <1>     ; Modified registers: eax, esi, edi
                                <1>
                                <1>     ; int 31h (int 10h) entrance
                                <1>
00001906 09FF                   <1>     or      edi, edi
00001908 7506                   <1>     jnz     short _vbe3_pmfn_return_mode_info
                                <1>
                                <1> vbe3_func_fail:
0000190A B84F010000             <1>     mov     eax, 014Fh ; ah = 1 : Function call failed
                                <1>                        ; al = 4Fh : Function is supported
0000190F C3                     <1>     retn
                                <1>
                                <1>     ; jump from '_vbe_biosfn_return_mode_info'
                                <1> _vbe3_pmfn_return_mode_info:
00001910 57                     <1>     push    edi
                                <1>
                                <1>     ;; clear vbe3 'mode info block' buffer
                                <1>     ;push   ecx
                                <1>     ;xor    eax, eax
                                <1>     ;mov    ecx, 256/4
                                <1>     ;mov    edi, VBE3MODEINFOBLOCK
                                <1>     ;rep    stosd
                                <1>     ;pop    ecx
                                <1>
                                <1>     ; far call to VESA VBE3 PMI
                                <1>     ;mov    ax, 4F01h ; Return VBE Mode Information
00001911 BF00060000             <1>     mov     edi, VBE3MODEINFOBLOCK-VBE3SAVERESTOREBLOCK
                                <1>     ; ES selector base address = VBE3SAVERESTOREBLOCK
00001916 E807010000             <1>     call    int10h_32bit_pmi
                                <1>
```

```
0000191B 5F                <1>     pop     edi
                           <1>
                           <1>     ;cmp    ax, 004Fh
                           <1>     ;jne    short vbe3_func_retn  ; failed !
                           <1>
0000191C 21FF              <1>     and     edi, edi
                           <1>     ;jz     short vbe3_func_success
                           <1>     ; 21/12/2020
0000191E 741C              <1>     jz      short vbe3_func_retn
                           <1>
00001920 6683F84F          <1>     cmp     ax, 004Fh
00001924 7516              <1>     jne     short vbe3_func_retn  ; failed !
                           <1>
00001926 51                <1>     push    ecx
00001927 BE007C0900        <1>     mov     esi, VBE3MODEINFOBLOCK
                           <1>     ;mov    ecx, 256
                           <1>     ; 02/08/2022
0000192C 29C9              <1>     sub     ecx, ecx
0000192E FEC5              <1>     inc     ch
                           <1>     ; ecx = 256
00001930 E8E6F10000        <1>     call    transfer_to_user_buffer
00001935 59                <1>     pop     ecx
00001936 72D2              <1>     jc      short vbe3_func_fail
                           <1>
00001938 31C0              <1>     xor     eax, eax
0000193A B04F              <1>     mov     al, 4Fh ; successful
                           <1> vbe3_func_success:
                           <1> vbe3_func_retn:
0000193C C3                <1>     retn
                           <1>
                           <1> vbe3_pmfn_return_current_mode:
                           <1>     ; 12/12/2020
                           <1>     ;
                           <1>     ; VBE function 4F03h - Return Current VBE Mode
                           <1>     ;
                           <1>     ; Input:
                           <1>     ;       none (AX = 4F03h)
                           <1>     ; Output:
                           <1>     ;       AX = VBE return status
                           <1>     ;            AX = 004Fh -> succeeded
                           <1>     ;            AX <> 004Fh -> failed
                           <1>     ;       BX = Current VBE mode
                           <1>     ;         bit 0-13 = Mode number
                           <1>     ;         bit 14 = 0 Windowed frame buffer model
                           <1>     ;                = 1 Linear frame buffer model
                           <1>     ;         bit 15
                           <1>     ;             = 0 Memory cleared at last mode set
                           <1>     ;             = 1 Memory not cleared at last mode set
                           <1>     ;
                           <1>     ; Modified registers: eax, ebx
                           <1>
                           <1>     ; int 31h (int 10h) entrance
                           <1>
                           <1>     ; far call to VESA VBE3 PMI
                           <1>
                           <1>     ;mov   eax, 4F03h ; Return Current VBE Mode
                           <1> vbe3_pmfn_far_call:
                           <1>     ; ES selector base address = VBE3SAVERESTOREBLOCK
                           <1>     ;call  int10h_32bit_pmi
                           <1>     ;retn
0000193D E9E0000000        <1>     jmp     int10h_32bit_pmi
                           <1>
                           <1> vbe3_pmfn_set_mode:
                           <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                           <1>     ; 22/12/2020
                           <1>     ; 21/12/2020
                           <1>     ; 12/12/2020
                           <1>     ;
                           <1>     ; VBE function 4F02h - Set VBE Mode
                           <1>     ;
                           <1>     ; Input:
                           <1>     ;       BX = Desired Mode to set
                           <1>     ;         bit 0-13 = Mode number
                           <1>     ;         bit 14 = 0 Windowed frame buffer model
                           <1>     ;                = 1 Linear frame buffer model
                           <1>     ;         bit 15
                           <1>     ;             = 0 Memory cleared at last mode set
                           <1>     ;             = 1 Memory not cleared at last mode set
                           <1>     ; Output:
                           <1>     ;       AX = VBE return status
                           <1>     ;            AX = 004Fh -> succeeded
                           <1>     ;            AX <> 004Fh -> failed
                           <1>     ;
                           <1>     ; Modified registers: eax, ebx, esi (21/12/2020)
                           <1>
                           <1>     ; int 31h (int 10h) entrance
                           <1>
                           <1>     ; 22/12/2020 (VESA VBE3 feature)
                           <1>     ; BX bit 11 is flag for
                           <1>     ;       user specified CRTC values for refresh rate
                           <1>     ; 'test bh, 8'
                           <1>     ; if bit 11 is set, EDI points to 'CRTCInfoBlock'
                           <1>
                           <1>     ; 22/12/2020
                           <1>     ;; test bx for VBE video mode
                           <1>     ;test  bh, 1
                           <1>     ;jnz   short vbe3_sm_0
                           <1>
                           <1>     ;; use internal VBE mode set procedure
                           <1>     ;; for non-vbe (std VGA/CGA) modes
                           <1>     ;
                           <1>     ;; (it is useful -as 4F02h function-
                           <1>     ;;  to jump 'vbe_biosfn_set_mode'
                           <1>     ;;  instead of direct jump to '_set_mode')
                           <1>     ;; ((eliminates additional push-pops and settings))
                           <1>
                           <1>     ;jmp   vbe_biosfn_set_mode
                           <1>
                           <1> vbe3_sm_0:
                           <1>     ;;push ds  ; *
                           <1>     ;;push es  ; **
                           <1>     ;;push ebp ; ***
                           <1>     ;;push esi ; ****
                           <1>
                           <1>     ; Fit bx to VESA VBE2 type mode setting
                           <1>     ; (bx bit 11 is used for custom CRTC values in VBE3)
                           <1>     ; clear bit 9 to 11 (clear bh bit 1 to bit 3)
                           <1>
                           <1>     ; 22/12/2020
00001942 57                <1>     push    edi ; *****
00001943 F6C708            <1>     test    bh, 8  ; Use user specified CRTC values
00001946 7530              <1>     jnz     short vbe3_sm_3  ; for refresh rate
                           <1> vbe3_sm_4:
00001948 80E7C1            <1>     and     bh, 0C1h ; use bit 15, 14, 8 only (for bh)
```

```
                                <1>     ;mov    [vbe_mode_x], bh
                                <1>
0000194B 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 3 ; is current mode 03h ?
00001952 7509                   <1>     jne     short vbe3_sm_1    ; no
                                <1>
                                <1>     ; save mode 03h video pages and cursor positions
00001954 57                     <1>     push    edi ; **!***
00001955 51                     <1>     push    ecx ; ******
                                <1>     ;push   esi
                                <1>
00001956 E8CC040000             <1>     call    save_mode3_multiscreen
                                <1>
                                <1>     ;pop    esi
0000195B 59                     <1>     pop     ecx ; ******
0000195C 5F                     <1>     pop     edi ; **!***
                                <1> vbe3_sm_1:
                                <1>     ; ax = 4F02h
                                <1>     ; bx = video mode number (vbe2 type)
0000195D E8C0000000             <1>     call    int10h_32bit_pmi
                                <1>                     ; call to far call to VBE3 PMI
                                <1>
00001962 6683F84F               <1>     cmp     ax, 004Fh ; succeeded ?
00001966 750E                   <1>     jne     short vbe3_sm_2
                                <1>     ; set current mode byte/sign to extended (SVGA) mode
00001968 C605[2E680000]FF       <1>     mov     byte [CRT_MODE], 0FFh ; VESA VBE mode
                                <1>     ; set current VBE mode word to bx input
0000196F 66891D[269D0100]       <1>     mov     [video_mode], bx
                                <1> vbe3_sm_2:
                                <1>     ; 22/12/2020
00001976 5F                     <1>     pop     edi ; *****
00001977 C3                     <1>     retn
                                <1>
                                <1> vbe3_sm_3:
                                <1>     ; 22/12/2020
                                <1>     ; copy user's CRTCInfoBlock to the buffer
00001978 51                     <1>     push    ecx
00001979 89FE                   <1>     mov     esi, edi
0000197B BF807D0900             <1>     mov     edi, VBE3CRTCINFOBLOCK
                                <1>     ;mov    ecx, 64
                                <1>     ; 02/08/2022
00001980 29C9                   <1>     sub     ecx, ecx
00001982 B140                   <1>     mov     cl, 64
00001984 E8DCF10000             <1>     call    transfer_from_user_buffer
00001989 59                     <1>     pop     ecx
                                <1>     ; set offset (es base addr is VBE3SAVERESTOREBLOCK)
0000198A 81EF00760900           <1>     sub     edi, VBE3SAVERESTOREBLOCK
00001990 EBB6                   <1>     jmp     short vbe3_sm_4
                                <1>
                                <1> vesa_vbe3_pmi:
                                <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; 12/12/2020
                                <1>     ; 08/12/2020
                                <1>     ; 07/12/2020
                                <1>     ; 05/12/2020, 06/12/2020
                                <1>     ; 03/12/2020, 04/12/2020
                                <1>     ; 28/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; VGA BIOS functions via
                                <1>     ; VESA VBE3 Protected Mode Inface
                                <1>     ; [vbe3] = 3 and [pmi32] > 0
                                <1>
                                <1>     ; 04/12/2020
                                <1>     ; Only 'set mode' will be redirected to vbe3 video bios
                                <1>     ; (by setting mode 3 multiscreen parameters before and after)
                                <1>
                                <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
                                <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                                <1>     ;push   eax
                                <1>     ;mov    eax, cr3
                                <1>     ;xchg   eax, [esp] ; **!**
                                <1>     ;push   eax
                                <1>     ;;cmp   esi, [k_page_dir]
                                <1>     ;;je    short vesa_vbe3_pmi_x
                                <1>     ;mov    eax, [k_page_dir]
                                <1>     ;mov    cr3, eax
                                <1> ;;vesa_vbe3_pmi_x:
                                <1>     ;pop    eax
                                <1>
                                <1>     ; 06/12/2020
00001992 20E4                   <1>     and     ah, ah  ; 0 = set mode function
00001994 7405                   <1>     jz      short vbe3_pmi_0
00001996 E980000000             <1>     jmp     vbe3_pmi_9
                                <1>
                                <1> vbe3_pmi_0:
                                <1>     ; 07/12/2020
0000199B 88C4                   <1>     mov     ah, al
0000199D 80E480                 <1>     and     ah, 80h ; 0 or 80h
000019A0 30E0                   <1>     xor     al, ah ; 8?h -> 0?h
                                <1>
                                <1>     ;cmp    al, 13h ; mode number above 13h is returned
                                <1>     ;jna    short vbe3_pmi_1
                                <1>     ;               ; back to default code due to uncertainty
                                <1>     ;               ; (>13h is not std for all svga bioses)
                                <1>     ;jmp    VGA_funcs_0
                                <1> vbe3_pmi_1:
                                <1>     ; 07/12/2020
                                <1>     ; Possible cases for VBE3 (PMI, ah=0) set mode:
                                <1>     ; current mode > 07h and requested mode: any
                                <1>     ; current mode <= 07h and requested mode > 07h
                                <1>
                                <1>     ; 06/12/2020
000019A2 8825[4B830100]         <1>     mov     byte [noclearmem], ah ; 0 or 80h
                                <1>     ; check current video mode if it is 03h
000019A8 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 3 ; current mode
000019AF 750B                   <1>     jne     short vbe3_pmi_3
                                <1>     ; 07/12/2020
                                <1>     ; check new video mode if it is 03h also
                                <1>     ;cmp    al, 3
                                <1>     ;jne    short vbe3_pmi_2
                                <1>     ;mov    byte [p_crt_mode], 80h        ; clear video memory
                                <1>     ;jmp    short vbe3_pmi_5
                                <1> vbe3_pmi_2:
                                <1>     ; case 1:
                                <1>     ; Current mode is 03h and new mode is not 03h
                                <1>
                                <1>     ; save video pages and cursor positions
000019B1 56                     <1>     push    esi
000019B2 57                     <1>     push    edi
000019B3 51                     <1>     push    ecx
                                <1>
                                <1>     ; 12/12/2020
                                <1>     ;mov    esi, 0B8000h ; mode 3 video memory
                                <1>     ;mov    edi, 98000h  ; backup location
                                <1>     ;mov    ecx, (0B8000h-0B0000h)/4
                                <1>     ;rep    movsd
```

```
                            <1>     ;
                            <1>     ;mov   byte [p_crt_mode], 3 ; previous mode, backup sign
                            <1>     ;xchg  cl, [ACTIVE_PAGE]
                            <1>     ;mov   [p_crt_page], cl  ; save as previous active page
                            <1>     ;
                            <1>     ;; save cursor positions
                            <1>     ;mov   esi, CURSOR_POSN
                            <1>     ;mov   edi, cursor_pposn ; cursor positions backup
                            <1>     ;mov   cl, 4
                            <1>     ;rep   movsd
                            <1>
                            <1>     ; 12/12/2020
000019B4 E86E040000         <1>     call   save_mode3_multiscreen
                            <1>
000019B9 59                 <1>     pop    ecx
000019BA 5F                 <1>     pop    edi
000019BB 5E                 <1>     pop    esi
                            <1> vbe3_pmi_3:
                            <1>     ; 08/12/2020
                            <1>     ; 07/12/2020
                            <1>     ; case 3 or case 4
000019BC A2[2E680000]       <1>     mov    [CRT_MODE], al
000019C1 3C03               <1>     cmp    al, 3
000019C3 7407               <1>     je     short vbe3_pmi_4
                            <1>     ; case 4:
                            <1>     ; Current mode is not 03h and also new mode is not 03h
000019C5 800D[49830100]80   <1>     or     byte [p_crt_mode], 80h  ; 83h (case 1 -> case 4)
                            <1>     ;jmp   short vbe3_pmi_5
                            <1> vbe3_pmi_4:
                            <1>     ; case 3:
                            <1>     ;
                            <1>     ; Current mode is not 03h and new mode is 03h
                            <1>
                            <1> ;vbe3_pmi_5:
                            <1>     ;mov   [CRT_MODE], al
                            <1>
000019CC E851000000         <1>     call   int10h_32bit_pmi
                            <1>
000019D1 803D[2E680000]03   <1>     cmp    byte [CRT_MODE], 3  ; new video mode
                            <1>     ;jne   vbe3_pmi_8     ; video mode <> 03h
000019D8 7532               <1>     jne    short vbe3_pmi_8
                            <1>
                            <1>     ;push  eax ; 04/12/2020
000019DA 53                 <1>     push   ebx
000019DB 51                 <1>     push   ecx
000019DC 52                 <1>     push   edx
000019DD 57                 <1>     push   edi ; 03/12/2020
                            <1>
                            <1>     ; 12/12/2020
000019DE 56                 <1>     push   esi
000019DF E875040000         <1>     call   restore_mode3_multiscreen
000019E4 5E                 <1>     pop    esi
                            <1>     ; AL = active video page
                            <1>
                            <1>     ; 12/12/2020
                            <1>     ;mov   al, [p_crt_page] ; previous mode 3 active page
                            <1>     ;
                            <1>     ;;test byte [p_crt_mode], 7Fh ; 83h or 80h or 03h
                            <1>     ;;jz   short vbe3_pmi_6 ; do not restore video pages
                            <1>     ;                       ; clear current video page
                            <1>     ;; case 3
                            <1>     ;
                            <1>     ;; ([p_crt_mode] = 03h)
                            <1>     ;
                            <1>     ;; New video mode is 3 while current video mode is not 3
                            <1>     ;; (multi screen) video pages will be restored from 098000h
                            <1>     ;
                            <1>     ;; restore video pages and cursor positions
                            <1>     ;
                            <1>     ;mov   [ACTIVE_PAGE], al ; current mode 3 active page
                            <1>     ;
                            <1>     ;push  esi
                            <1>     ;
                            <1>     ;; restore video pages
                            <1>     ;mov   esi, 98000h
                            <1>     ;mov   edi, 0B8000h
                            <1>     ;;mov  ecx, 2000h
                            <1>     ;mov   cx, 2000h ; 8K dwords (32K)
                            <1>     ;rep   movsd
                            <1>     ;
                            <1>     ;mov   [p_crt_mode], cl ; reset ('case 3' end condition)
                            <1>     ;
                            <1>     ;; restore cursor positions
                            <1>     ;mov   esi, cursor_pposn
                            <1>     ;mov   edi, CURSOR_POSN
                            <1>     ;;mov  ecx, 4 ; restore all cursor positions (16 bytes)
                            <1>     ;mov   cl, 4
                            <1>     ;rep   movsd
                            <1>     ;
                            <1>     ;pop   esi
                            <1>     ;
                            <1>     ;; 07/12/2020
                            <1>     ;; restore CRT_START according to ACTIVE_PAGE
                            <1>     ;mov   [CRT_START], cx ; 0
                            <1>     ;
                            <1>     ;; check active page and set it again if it is not 0
                            <1>     ;or    al, al
                            <1>     ;jz    short vbe3_pmi_7
                            <1>     ;
                            <1>     ;mov   cl, al
                            <1> ;vbe3_pmi_5:
                            <1>     ;add   word [CRT_START], 4096
                            <1>     ;dec   cl
                            <1>     ;jnz   short vbe3_pmi_5
                            <1>
000019E5 B405               <1>     mov    ah, 05h ; set current video page
                            <1>     ;al = video page
000019E7 E836000000         <1>     call   int10h_32bit_pmi
                            <1>
                            <1>     ; check current cursor position & set it again if not 0,0
                            <1>     ;movzx ebx, byte [ACTIVE_PAGE]
000019EC 0FB6D8             <1>     movzx  ebx, al
000019EF D0E3               <1>     shl    bl, 1
000019F1 81C3[CE760100]     <1>     add    ebx, CURSOR_POSN
000019F7 668B13             <1>     mov    dx, [ebx]
000019FA 6621D2             <1>     and    dx, dx
000019FD 7409               <1>     jz     short vbe3_pmi_7
                            <1>
                            <1>     ;dx = cursor position (dl = column, dh = row)
                            <1>     ;mov   bh, [ACTIVE_PAGE] ; 06/12/2020
000019FF 88C7               <1>     mov    bh, al
00001A01 B402               <1>     mov    ah, 02h ; set cursor position
00001A03 E81A000000         <1>     call   int10h_32bit_pmi
                            <1>
```

```
                                <1>     ;jmp   short vbe3_pmi_7
                                <1>
                                <1> ;vbe3_pmi_6:
                                <1> ;   ; 07/12/2020
                                <1> ;   ; case 1, previous mode is 03h, current mode is 03h
                                <1> ;   ; 03/12/2020
                                <1> ;   cmp    byte [noclearmem], 0
                                <1> ;   jna    short vbe3_pmi_7 ; do not clear memory
                                <1> ;   ; clear video page
                                <1> ;   mov    ecx, 1024 ; 4096/4
                                <1> ;   mov    eax, 07200720h
                                <1> ;   mov    edi, 0B8000h ; [crt_base]
                                <1> ;   add    di, [CRT_START]
                                <1> ;   rep    stosd  ; FILL THE REGEN BUFFER WITH BLANKS
                                <1>
                                <1> vbe3_pmi_7:
00001A08 5F                     <1>     pop    edi
00001A09 5A                     <1>     pop    edx
00001A0A 59                     <1>     pop    ecx
00001A0B 5B                     <1>     pop    ebx
                                <1>     ;pop   eax ; 04/12/2020
                                <1> vbe3_pmi_8:
                                <1>     ; 04/12/2020
                                <1>     ;(TRDOS 386 v2.0.3, INT 31h, ah=0 return)
00001A0C 31C0                   <1>     xor    eax, eax  ; eax = 0 -> succesful
                                <1> vesa_vbe3_pmi_retn:
                                <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
00001A0E 50                     <1>     push   eax
00001A0F A1[60770100]           <1>     mov    eax, [cr3prev] ; restore previous (user's) cr3
                                <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                                <1>     ;xchg  eax, [esp] ; **!**
                                <1>     ;;cmp  eax, [k_page_dir]
                                <1>     ;;je   short vesa_vbe3_pmi_retn_x
00001A14 0F22D8                 <1>     mov    cr3, eax
                                <1> ;;vesa_vbe3_pmi_retn_x:
00001A17 58                     <1>     pop    eax
                                <1>     ;
00001A18 07                     <1>     pop    es  ; **
00001A19 1F                     <1>     pop    ds  ; *
00001A1A CF                     <1>     iretd
                                <1>
                                <1> vbe3_pmi_9:
                                <1>     ; 06/12/2020
                                <1>     ;cmp   ah, 10h ; Set/Get Palette Registers
                                <1>     ;jnb   short vbe3_pmi_10
                                <1>     ; 05/12/2020
00001A1B E802000000             <1>     call   int10h_32bit_pmi
00001A20 EBEC                   <1>     jmp    short vesa_vbe3_pmi_retn
                                <1>
                                <1> ;vbe3_pmi_10:
                                <1>     ; 06/12/2020
                                <1>     ;jmp   VGA_funcs_0
                                <1>
                                <1> int10h_32bit_pmi:
                                <1>     ; 03/12/2020
                                <1>     ; 28/11/2020
                                <1>     ; calling standard VGA Bios (INT 10h) functions
                                <1>     ; by using 32 bit protected mode interface of
                                <1>     ; VESA VBE3 Video Bios (with 'PMID' signature)
                                <1>
                                <1>     ; 03/12/2020
                                <1>     ; eax, ebx, ecx, edx, edi will be used by vbios pmi
                                <1>     ; (esi and ebp will not be used)
                                <1>
                                <1>     ; 03/12/2020
00001A22 56                     <1>     push   esi
00001A23 C1E010                 <1>     shl    eax, 16  ; move function number (ax) to hw
00001A26 8B35[2C9D0100]         <1>     mov    esi, [pmid_addr] ; linear address of
                                <1>                             ; PMInfo.Entrypoint pointer
                                <1>     ;mov   ax, [esi+PMInfo.EntryPoint]
00001A2C 668B06                 <1>     mov    ax, [esi]
00001A2F C1C010                 <1>     rol    eax, 16 ; move PM entry address to hw
                                <1>                    ; and move function number to lw (ax)
00001A32 5E                     <1>     pop    esi
                                <1>
                                <1>     ; top of stack: ; (*)
                                <1>     ; return (the caller) address of "int10h_32bit_pmi"
                                <1>
00001A33 E921EDFFFF             <1>     jmp    _VBE3PMI_fcall ; will return to the caller (*)
                                <1>
                                <1> _vbe3_pmfn_srs_8:
                                <1>     ; 17/01/2021
00001A38 31DB                   <1>     xor    ebx, ebx ; points to VBE3SAVERESTOREBLOCK
                                <1> _vbe3_pmfn_srs_9: ; 24/01/2021
00001A3A 66B8044F               <1>     mov    ax, 4F04h
00001A3E EBE2                   <1>     jmp    short int10h_32bit_pmi
                                <1>
                                <1> vbe3_pmfn_save_restore_state:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 24/01/2021
                                <1>     ; 23/01/2021
                                <1>     ; 16/01/2021 - 17/01/2021
                                <1>     ; 14/01/2021
                                <1>     ;
                                <1>     ; VBE function 4F04h - Save/Restore Video State
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;      DL = sub function
                                <1>     ;      CL = requested state
                                <1>     ;      EBX = pointer to buffer (if DL<>00h)
                                <1>     ;      AX = 4F04h
                                <1>     ; Output:
                                <1>     ;      AX = 004Fh (successful)
                                <1>     ;      AH > 0 -> error
                                <1>     ;      BX = Number of 64-byte blocks
                                <1>     ;           to hold the state buffer (if DL=00h)
                                <1>
                                <1>     ; Modified registers: eax, ebx, esi, edi
                                <1>
00001A40 21DB                   <1>     and    ebx, ebx ; user's buffer address
00001A42 750A                   <1>     jnz    short _vbe3_pmfn_save_restore_state
                                <1>
00001A44 08D2                   <1>     or     dl, dl
00001A46 740C                   <1>     jz     short _vbe3_pmfn_srs_0
                                <1>
                                <1>     ; function failed
                                <1>     ;;mov  eax, 0100h
                                <1>     ;sub   eax, eax
                                <1>     ;inc   ah  ; eax = 0100h
                                <1>     ;retn
                                <1>     ; 16/01/2021
                                <1> _vbe3_pmfn_srs_fail:
00001A48 B84F010000             <1>     mov    eax, 014Fh ; ah = 1 : Function call failed
                                <1>                       ; al = 4Fh : Function is supported
```

```
                                <1> _vbe3_srs_retn:
00001A4D C3                     <1>     retn
                                <1>
                                <1> _vbe3_pmfn_save_restore_state:
00001A4E 20D2                   <1>     and    dl, dl
00001A50 7555                   <1>     jnz    short _vbe3_pmfn_srs_2
                                <1> _vbe3_pmfn_srs:
00001A52 31DB                   <1>     xor    ebx, ebx
                                <1> _vbe3_pmfn_srs_0:
                                <1>     ; 24/01/2021
00001A54 83F90F                 <1>     cmp    ecx, 0Fh
                                <1>     ;ja    short _vbe3_pmfn_srs_1
00001A57 77EF                   <1>     ja     short _vbe3_pmfn_srs_fail
                                <1>
                                <1>     ; !!! CLEAR CL BIT 2 !!!
                                <1>     ; (when bit 2 is set, function causes cpu exception)
                                <1>     ; BIOS data will not be saved and restored
                                <1>     ; (to prevent protected mode page fault error)
00001A59 80E1FD                 <1>     and    cl, ~2 ; and cl, not 2
                                <1>
                                <1>     ; 24/01/2021
                                <1>     ;mov   bl, 1
00001A5C FEC3                   <1>     inc    bl ; = 1
                                <1>     ;shl   bx, cl
                                <1>     ; 02/08/2022
00001A5E D3E3                   <1>     shl    ebx, cl
00001A60 66231D[309D0100]       <1>     and    bx, [vbe3stbsflags]
00001A67 7415                   <1>     jz     short _vbe3_pmfn_srs_1
                                <1>     ;mov   bx, cx
00001A69 89CB                   <1>     mov    ebx, ecx ; <= 15
00001A6B D0E3                   <1>     shl    bl, 1 ; 0, 2, 8 .. 30
00001A6D 668B9B[873B0000]       <1>     mov    bx, [vbestatebufsize+ebx]
00001A74 89DF                   <1>     mov    edi, ebx
                                <1>     ; edi = state buffer size in bytes
                                <1>     ;shr   bx, 6 ; / 64
                                <1>     ; 02/08/2022
00001A76 C1EB06                 <1>     shr    ebx, 6
                                <1>     ;mov   ax, 4Fh
00001A79 28E4                   <1>     sub    ah, ah
00001A7B B04F                   <1>     mov    al, 4Fh
00001A7D C3                     <1>     retn
                                <1> _vbe3_pmfn_srs_1:
                                <1>     ; ax = 4F04h
                                <1>     ;call  int10h_32bit_pmi
                                <1>     ; 24/01/2021
                                <1>     ;call  _vbe3_pmfn_srs_8
                                <1>     ; ebx = 0
00001A7E E8B7FFFFFF             <1>     call   _vbe3_pmfn_srs_9
00001A83 6683F84F               <1>     cmp    ax, 004Fh
00001A87 75C4                   <1>     jne    short _vbe3_srs_retn
                                <1>     ; 24/01/2021
                                <1>     ;cmp   ecx, 0Fh
                                <1>     ;ja    short _vbe3_srs_retn
                                <1>     ; 24/01/2021
                                <1>     ;mov   ax, 1
00001A89 B001                   <1>     mov    al, 1
                                <1>     ;shl   ax, cl
                                <1>     ; 02/08/2022
00001A8B D3E0                   <1>     shl    eax, cl
00001A8D 660905[309D0100]       <1>     or     [vbe3stbsflags], ax ; set flag for state option
                                <1>     ; 23/01/2021
00001A94 89DF                   <1>     mov    edi, ebx
00001A96 89C8                   <1>     mov    eax, ecx
00001A98 D0E0                   <1>     shl    al, 1
                                <1>     ;shl   di, 6 ; * 64
                                <1>     ; 02/08/2022
00001A9A C1E706                 <1>     shl    edi, 6
00001A9D 6689B8[873B0000]       <1>     mov    [vbestatebufsize+eax], di
                                <1>                            ; save buf size for option
                                <1>     ;xchg  edi, ebx
                                <1>     ; edi = state buffer size in bytes
00001AA4 B04F                   <1>     mov    al, 4Fh
00001AA6 C3                     <1>     retn
                                <1>
                                <1> _vbe3_pmfn_srs_2:
                                <1>     ; 24/01/2021
                                <1>     ; !!! CLEAR CL BIT 2 !!!
                                <1>     ; (when bit 2 is set, function causes cpu exception)
                                <1>     ; BIOS data will not be saved and restored
                                <1>     ; (to prevent protected mode page fault error)
                                <1>
00001AA7 F6C1FD                 <1>     test   cl, ~2 ; test cl, not 2
00001AAA 749C                   <1>     jz     short _vbe3_pmfn_srs_fail
                                <1>
00001AAC 80FA02                 <1>     cmp    dl, 2
00001AAF 7748                   <1>     ja     short _vbe3_pmfn_srs_5
                                <1>
                                <1>     ;and   cl, ~2 ; and cl, not 2
                                <1>
00001AB1 53                     <1>     push   ebx ; * ; buffer address
                                <1>     ; save or restore state
                                <1>     ; (get required buffer size at first)
00001AB2 52                     <1>     push   edx ; **
00001AB3 28D2                   <1>     sub    dl, dl ; 0
00001AB5 E898FFFFFF             <1>     call   _vbe3_pmfn_srs
00001ABA 5A                     <1>     pop    edx ; **
                                <1>     ; 24/01/2021
00001ABB 5B                     <1>     pop    ebx ; *
00001ABC 08E4                   <1>     or     ah, ah
00001ABE 7538                   <1>     jnz    short _vbe3_pmfn_srs_4 ; error
                                <1>
                                <1>     ; edi = buffer size in bytes
00001AC0 81FF00080000           <1>     cmp    edi, 2048
00001AC6 772B                   <1>     ja     short _vbe3_pmfn_srs_3
                                <1>
00001AC8 80FA01                 <1>     cmp    dl, 1
00001ACB 7531                   <1>     jne    short _vbe3_pmfn_srs_6 ; restore state
                                <1>
                                <1>     ; save video state
                                <1>     ;xor   ebx, ebx ; points to VBE3SAVERESTOREBLOCK
                                <1>     ;mov   ax, 4F04h
                                <1>     ;call  int10h_32bit_pmi
                                <1>
                                <1>     ; 24/01/2021
00001ACD E842000000             <1>     call   _vbe3_pmfn_srs_7
                                <1>
00001AD2 6683F84F               <1>     cmp    ax, 004Fh
00001AD6 7520                   <1>     jne    short _vbe3_pmfn_srs_4
                                <1>
00001AD8 09DB                   <1>     or     ebx, ebx  ; kernel ('sysvideo') ?
00001ADA 741C                   <1>     jz     short _vbe3_pmfn_srs_4 ; yes
                                <1>
                                <1>     ; the caller is user
00001ADC 51                     <1>     push   ecx ; *
```

```
00001ADD 89F9                <1>     mov     ecx, edi ; state buffer size
00001ADF BE00760900          <1>     mov     esi, VBE3SAVERESTOREBLOCK ; source
                             <1>                                       ; (vbe3 pmi buff)
00001AE4 89DF                <1>     mov     edi, ebx       ; destination (user buff)
00001AE6 E830F00000          <1>     call    transfer_to_user_buffer
00001AEB 59                  <1>     pop     ecx ; *
00001AEC 7205                <1>     jc      short _vbe3_pmfn_srs_3
                             <1>
00001AEE 29C0                <1>     sub     eax, eax
00001AF0 B04F                <1>     mov     al, 4Fh
00001AF2 C3                  <1>     retn
                             <1>
                             <1>     ; 24/01/2021
                             <1> _vbe3_pmfn_srs_3:
00001AF3 B84F010000          <1>     mov     eax, 014Fh
                             <1> _vbe3_pmfn_srs_4:
00001AF8 C3                  <1>     retn
                             <1> _vbe3_pmfn_srs_5:
00001AF9 31C0                <1>     xor     eax, eax
00001AFB FEC4                <1>     inc     ah
                             <1>     ; eax = 0100h, function is not supported
00001AFD C3                  <1>     retn
                             <1>
                             <1> _vbe3_pmfn_srs_6:
                             <1>     ; restore video state
                             <1>     ; 24/01/2021
                             <1>     ;pop    ebx ; *
                             <1>     ; 23/01/2021
00001AFE 09DB                <1>     or      ebx, ebx ; 0 ?
00001B00 7412                <1>     jz      short _vbe3_pmfn_srs_7 ; 'sysvideo' call
                             <1>     ; 24/01/2021
                             <1>     ;jz     _vbe3_pmfn_srs_8
00001B02 89DE                <1>     mov     esi, ebx
                             <1>     ; esi = user's video state buffer
00001B04 51                  <1>     push    ecx ; *
00001B05 89F9                <1>     mov     ecx, edi ; state buffer size
00001B07 BF00760900          <1>     mov     edi, VBE3SAVERESTOREBLOCK ; destination
                             <1>                                       ; (vbe3 pmi buff)
                             <1>     ;mov    esi, ebx       ; source (user buff)
00001B0C E854F00000          <1>     call    transfer_from_user_buffer
00001B11 59                  <1>     pop     ecx ; *
00001B12 72DF                <1>     jc      short _vbe3_pmfn_srs_3
                             <1> _vbe3_pmfn_srs_7:
00001B14 53                  <1>     push    ebx ; *
                             <1>     ; restore video state
                             <1>     ;xor    ebx, ebx ; points to VBE3SAVERESTOREBLOCK
                             <1>     ;mov    ax, 4F04h
                             <1>     ;call   int10h_32bit_pmi
                             <1>     ; 17/01/2021
00001B15 E81EFFFFFF          <1>     call    _vbe3_pmfn_srs_8
00001B1A 5B                  <1>     pop     ebx ; *
00001B1B C3                  <1>     retn
                             <1>
                             <1> VIDEO_STATE:
                             <1>     ; 26/06/2016
                             <1>     ; 12/05/2016
                             <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>
                             <1> ;--------------------------------------------------
                             <1> ; VIDEO STATE
                             <1> ;  RETURNS THE CURRENT VIDEO STATE IN AX
                             <1> ;  AH = NUMBER OF COLUMNS ON THE SCREEN
                             <1> ;  AL = CURRENT VIDEO MODE
                             <1> ;  BH = CURRENT ACTIVE PAGE
                             <1> ;--------------------------------------------------
                             <1>
00001B1C 8A25[30680000]      <1>     mov     ah, [CRT_COLS] ; GET NUMBER OF COLUMNS
00001B22 A0[2E680000]        <1>     mov     al, [CRT_MODE] ; CURRENT MODE
                             <1>     ;movzx  esi, al
                             <1>     ;mov    ah, [esi+M6]
                             <1>     ; BH = active page
00001B27 8A3D[DE760100]      <1>     mov     bh, [ACTIVE_PAGE] ; GET CURRENT ACTIVE PAGE
00001B2D FA                  <1>     cli     ; 02/01/2017
00001B2E 5D                  <1>     pop     ebp            ; RECOVER REGISTERS
00001B2F 5F                  <1>     pop     edi
00001B30 5E                  <1>     pop     esi
00001B31 59                  <1>     pop     ecx            ; DISCARD SAVED BX
00001B32 EB41                <1>     jmp     short M15      ; RETURN TO CALLER
                             <1>
                             <1> set_mode_ncm:
                             <1>     ; 17/11/2020 (TRDOS 386 v2.0.3)
                             <1>     ; 04/07/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; set mode without clearing the video memory
                             <1>     ; (ony for graphics modes)
                             <1>
                             <1>     ;cmp    al, 7 ; IBM PC CGA modes
                             <1>     ;jna    short SET_MODE ; normal function (clear)
                             <1>     ;; do not clear memory
                             <1>     ;;mov   [noclearmem], al ; > 0
                             <1>     ;mov    byte [noclearmem], 80h ; 17/11/2020
                             <1>     ;call   _set_mode
                             <1>     ;mov    byte [noclearmem], 0
                             <1>         ;jmp     short VIDEO_RETURN
                             <1>
                             <1>     ; 17/11/2020 (TRDOS v2.0.3)
00001B34 0C80                <1>     or      al, 80h ; not clear memory option
                             <1>
                             <1>     ; 05/12/2020
                             <1>     ; 27/11/2020
                             <1>     ; 17/11/2020
                             <1>     ; 08/08/2016, 10/08/2016
                             <1>     ; 29/07/2016, 30/07/2016
                             <1>     ; 25/07/2016, 26/07/2016, 27/07/2016
                             <1>     ; 02/07/2016, 18/07/2016, 23/07/2016
                             <1>     ; 24/06/2016, 26/06/2016
                             <1>     ; 29/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> SET_MODE:
                             <1>     ; For 32 bit TRDOS and Retro UNIX 386:
                             <1>     ;       valid video mode: 03h only!
                             <1>     ;       (VGA modes will be selected with another routine)
                             <1>     ;
                             <1>     ; set_txt_mode ; 80*25 (16 fore colors, 8 back colors)
                             <1>
                             <1>     ; 27/11/2020
                             <1>
                             <1>     ; Check if current mode is
                             <1>     ; Bochs/Plex86 VBE graphics mode
00001B36 803D[2E680000]FF    <1>     cmp     byte [CRT_MODE], 0FFh ; VESA VBE graphics mode
00001B3D 7220                <1>     jb      short _set_mode_       ; signature
                             <1>                                    ; VBE mode number is in
                             <1>                                    ; [video_mode] bit 0to8
00001B3F 88C3                <1>     mov     bl, al ; save video mode
00001B41 E88D230000          <1>     call    dispi_get_enable
00001B46 50                  <1>     push    eax ; save current VBE dispi status
```

```
                                <1>     ; Disable Bochs/Plex86 VBE dispi
                                <1>     ;mov    ax, 0 ; VBE_DISPI_DISABLED
00001B47 31C0                   <1>     xor     eax, eax ; 0
00001B49 E85A230000             <1>     call    dispi_set_enable
00001B4E 88D8                   <1>     mov     al, bl ; restore video mode
00001B50 E830000000             <1>     call    _set_mode
00001B55 58                     <1>     pop     eax ; restore current VBE dispi status
00001B56 7313                   <1>     jnc     short VIDEO_RETURN
                                <1>     ; ! unimplemented or invalid video mode number !
                                <1>     ; VBE dispi must be enabled again
                                <1>     ; (return to run on current VBE graphics mode)
                                <1>     ;;mov   al, [video_mode+1] ; bit 8 to 15
                                <1>     ;;and   al, 0C0h ; isolate bit 14 and bit 15
                                <1>     ;;or    al, 1 ; VBE_DISPI_ENABLED
00001B58 E84B230000             <1>     call    dispi_set_enable
00001B5D EB07                   <1>     jmp     short _video_func_err
                                <1>
                                <1> _set_mode_:
                                <1>     ; VGA bios (non-VBE) 'setmode' procedure
                                <1>
                                <1>     ; 26/11/2020 (TRDOS v2.0.3)
                                <1>
                                <1> ;-----------------------------------------------------
                                <1> ; SET MODE                                          :
                                <1> ;   THIS ROUTINE INITIALIZES THE ATTACHMENT TO      :
                                <1> ;   THE SELECTED MODE, THE SCREEN IS BLANKED.       :
                                <1> ; INPUT                                             :
                                <1> ;   (AL) - MODE SELECTED (RANGE 0-7)                :
                                <1> ; OUTPUT                                            :
                                <1> ;   NONE                                            :
                                <1> ;-----------------------------------------------------
                                <1>
00001B5F E821000000             <1>     call    _set_mode ; 24/06/2016 (set_txt_mode)
                                <1>     ; 26/11/2020
00001B64 7305                   <1>     jnc     short VIDEO_RETURN
                                <1>
                                <1>     ; 26/11/2020
                                <1> _video_func_err:
00001B66 31C0                   <1>     xor     eax, eax ; function call failed
00001B68 48                     <1>     dec     eax  ; 0FFFFFFFFh ; - 1
00001B69 EB05                   <1>     jmp     short _video_return
                                <1>
                                <1> ; 12/05/2016
                                <1> ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> ;-----      NORMAL RETURN FROM ALL VIDEO RETURNS
                                <1>
                                <1> VIDEO_RETURN:
00001B6B A1[3C830100]           <1>     mov     eax, [video_eax] ; 12/05/2016
                                <1> _video_return:
00001B70 FA                     <1>     cli ; 02/01/2017
00001B71 5D                     <1>     pop     ebp ; ******** ; 26/11/2020
00001B72 5F                     <1>     pop     edi ; *******
00001B73 5E                     <1>     pop     esi ; ******
00001B74 5B                     <1>     pop     ebx ; *****
                                <1> M15:; VIDEO_RETURN_C
                                <1>     ;;15/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;;mov   byte [intflg], 0
                                <1>     ;
                                <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
                                <1>     ;       restore previous (user's) cr3
00001B75 8B0D[60770100]         <1>     mov     ecx, [cr3prev]
00001B7B 0F22D9                 <1>     mov     cr3, ecx
                                <1>     ;
00001B7E 59                     <1>     pop     ecx ; **** ; 26/11/2020
00001B7F 5A                     <1>     pop     edx ; ***
00001B80 1F                     <1>     pop     ds  ; **
00001B81 07                     <1>     pop     es  ; * ; RECOVER SEGMENTS
00001B82 CF                     <1>     iretd           ; ALL DONE
                                <1>
                                <1> set_txt_mode:
                                <1>
                                <1>     ; 29/07/2016
                                <1>     ; 27/06/2016
00001B83 B003                   <1>     mov     al, 3 ; 26/11/2020 (bit 7 = 0)
                                <1>
                                <1>     ; 17/11/2020 (TRDOS v2.0.3)
                                <1>     ;mov    byte [noclearmem], 0
                                <1>
                                <1> ; 04/08/2022
                                <1> ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1> ; 12/04/2021
                                <1> ; 10/08/2016
                                <1> ; 08/08/2016
                                <1> ; 30/07/2016
                                <1> ; 29/07/2016
                                <1> ; 25/07/2016 - 26/07/2016 - 27/07/2016
                                <1> ; 07/07/2016 - 18/07/2016 - 23/07/2016
                                <1> ; 02/07/2016 - 03/07/2016 - 04/07/2016
                                <1> ; 26/06/2016
                                <1> ; 24/06/2016 (set_txt_mode -> _set_mode)
                                <1> ; 17/06/2016
                                <1> ; 29/05/2016
                                <1> ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> _set_mode:
                                <1>     ; 12/12/2020
                                <1>     ; 26/11/2020
                                <1>     ; call from 'biosfn_set_video_mode'
                                <1>     ;       (bochs/plex86 video bios code)
                                <1>     ; call from 'SET_MODE'
                                <1>     ;       (TRDOS 386 v2 default, IBM PC/AT rom bios code)
                                <1>     ; continue from 'set_txt_mode'
                                <1>
                                <1>     ; INPUT:
                                <1>     ;       al = VGA video mode
                                <1>     ; RETURN:
                                <1>     ;       cf = 1 -> video mode not implemented
                                <1>     ;       cf = 0 -> OK
                                <1>     ;
                                <1>     ; Modified registers: eax, bx, ecx, esi, edi, (ebp)
                                <1>
                                <1>     ; 17/11/2020 (TRDOS v2.0.3)
                                <1>     ; no clear memory option
                                <1>     ; (from mode number byte bit 7)
00001B85 88C4                   <1>     mov     ah, al
00001B87 80E480                 <1>     and     ah, 80h
                                <1>     ;mov    [noclearmem], al
00001B8A 8825[4B830100]         <1>     mov     [noclearmem], ah
                                <1>     ;and    al, 7Fh ; clear bit 7
                                <1>     ;;xor   [noclearmem], al ; clear bit 0 to 6
                                <1>     ; 26/11/2020
00001B90 30E0                   <1>     xor     al, ah ; and al, 7Fh
```

```
                                <1>
                                <1>     ; 19/11/2020
                                <1>
                                <1>     ; Video mode 03h action principle:
                                <1>     ;
                                <1>     ; for case 1:
                                <1>     ; Current mode is 03h and next/requested mode is not 03h
                                <1>     ;       - save mode (set mode 03h flag)
                                <1>     ;       - save 8 video pages (which are will be restored)
                                <1>     ;       - save active page number (which will be reactivated)
                                <1>     ;       - set active page to 0 always (no multi screen)
                                <1>     ;       - save 8 cursor positions (which will be restored)
                                <1>     ;       - use 'noclearmem' option
                                <1>     ;       [p_crt_mode] = 0 -> 03h
                                <1>     ;
                                <1>     ; for case 2:
                                <1>     ; Current mode is 03h and next/requested mode is also 03h
                                <1>     ;       - clear active video page if 'noclearmem' is not set
                                <1>     ;       [p_crt_mode] = 0 -> 80h -> 0
                                <1>     ;
                                <1>     ; for case 3:
                                <1>     ; Current mode is not 03h and next/requested mode is 03h
                                <1>     ;       - restore video pages (8 video pages were saved)
                                <1>     ;       - restore active page number (which were saved)
                                <1>     ;       - restore 8 cursor positions (which were saved)
                                <1>     ;       - reset/clear mode 03h flag
                                <1>     ;       [p_crt_mode] = 03h -> 0
                                <1>     ;
                                <1>     ; for case 4:
                                <1>     ; Current mode is not 03h and next/requested mode is not 03h
                                <1>     ;       - use 'noclearmem' option
                                <1>     ;       - set active page to 0 always
                                <1>     ;       [p_crt_mode] = 03h -> 83h -> 03h
                                <1>     ;
                                <1>     ; initial (boot time) values:
                                <1>     ;       [p_crt_mode] = 0 ("there isn't a page backup, yet")
                                <1>     ;         [CRT_MODE] = 3 (kernel's starting mode)
                                <1>
                                <1>     ; 26/11/2020
00001B92 3C03                   <1>     cmp     al, 03h     ; mode 3, 80x25 text, 16 colors
00001B94 7515                   <1>     jne     short _sm_0 ; (default mode for TRDOS 386 mainprog)
                                <1>
                                <1>     ; case 2 or case 3
                                <1>
                                <1>     ; check current video mode if it is 03h
00001B96 08E4                   <1>     or      ah, ah ; 80h or 0 ('noclearmem' option)
00001B98 7521                   <1>     jnz     short _sm_1 ; do not clear display page
                                <1>
                                <1>     ; 26/11/2020
                                <1>     ; Note:
                                <1>     ; [CRT_MODE] = 0FFh for VESA VBE video modes
                                <1>     ; [video_mode] = standard VGA and VESA VBE video modes
                                <1>
00001B9A 3805[2E680000]         <1>     cmp     [CRT_MODE], al ; 03h
00001BA0 7520                   <1>     jne     short _sm_2    ; case 3 ([p_crt_mode] = 03h)
                                <1>
                                <1>     ; case 2
                                <1>
                                <1>     ; [p_crt_mode] = 0
                                <1>
                                <1>     ; 19/11/2020
                                <1>     ; If '_set_mode' procedure is called for video mode 3
                                <1>     ;     while video mode is 3, video page will be cleared
                                <1>     ;     and cursor position of video page will be reset.
                                <1>
                                <1>     ; clear display page
00001BA2 C605[49830100]80       <1>     mov     byte [p_crt_mode], 80h ; clear page sign
00001BA9 EB1C                   <1>     jmp     short _sm_3    ; bypass save video page routine
                                <1> _sm_0:
                                <1>     ; case 1 or case 4
                                <1>
                                <1>     ; 05/12/2020
00001BAB 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 3 ; is current mode 03h?
00001BB2 7507                   <1>     jne     short _sm_1    ; case 4 ; [p_crt_mode] = 03h
                                <1>
                                <1>     ; case 1
                                <1>     ; [p_crt_mode] = 0
                                <1>
                                <1>     ; 19/11/2020
                                <1>     ; If '_set_mode' procedure is called for a video mode
                                <1>     ;     except video mode 3 while current video mode
                                <1>     ;     is 3, all video pages of mode 3 will be copied
                                <1>     ;     to 98000h address as backup, before mode change.
                                <1>
                                <1> _sm_save_pm:
                                <1>     ; 12/12/2020
                                <1>     ;; 03/07/2016
                                <1>     ;; save video pages
                                <1>     ;mov   esi, 0B8000h
                                <1>     ;mov   edi, 98000h ; 30/07/2016
                                <1>     ;mov   ecx, (0B8000h-0B0000h)/4
                                <1>     ;rep   movsd
                                <1>
                                <1>     ;mov   byte [p_crt_mode], 3 ; previous mode, backup sign
                                <1>     ;; 26/11/2020
                                <1>     ;xchg  cl, [ACTIVE_PAGE]
                                <1>     ;mov   [p_crt_page], cl  ; save as previous active page
                                <1>     ;
                                <1>     ;; save cursor positions
                                <1>     ;mov   esi, CURSOR_POSN
                                <1>     ;mov   edi, cursor_pposn ; cursor positions backup
                                <1>     ;mov   cl, 4
                                <1>     ;rep   movsd
                                <1>
                                <1>     ; 12/12/2020
00001BB4 E86E020000             <1>     call    save_mode3_multiscreen
                                <1>
                                <1>     ; 29/07/2016
                                <1>     ;;mov  [ACTIVE_PAGE], cl ; 0
                                <1>     ;xchg  cl, [ACTIVE_PAGE]
                                <1>     ;mov   [p_crt_page], cl  ; previous page (for mode 3)
                                <1>
                                <1>     ; [ACTIVE_PAGE] = 0
                                <1>
00001BB9 EB07                   <1>     jmp     short _sm_2    ; case 1 - 19/11/2020
                                <1> _sm_1:
                                <1>     ; 26/11/2020
                                <1>     ; 19/11/2020
                                <1>
                                <1>     ; case 4
00001BBB 800D[49830100]80       <1>     or      byte [p_crt_mode], 80h
                                <1>                             ; here [p_crt_mode] must be 83h
                                <1>                             ;        (for case 4)
                                <1>                             ; (because video mode 03h
```

```
                                <1>                             ; was changed before as in case 1)
                                <1>
                                <1> _sm_2:      ; case 4 (jump to _sm_2) - 19/11/2020
                                <1>
                                <1>     ; 19/11/2020
                                <1>     ; case 3
                                <1>     ; If '_set_mode' procedure is called for video mode 3
                                <1>     ;     while video mode is not 3 and if there is video
                                <1>     ;     page backup for video mode 3, all (of 8) mode 3
                                <1>     ;     video pages will be restored from 98000h.
                                <1>
00001BC2 A2[2E680000]           <1>     mov    [CRT_MODE], al  ; save mode in global variable
                                <1> _sm_3:
                                <1>     ; 04/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 30/07/2016
                                <1>     ; 26/07/2016
                                <1>     ; 25/07/2016
                                <1>     ; set_mode_vga:
                                <1>     ; 18/07/2016
                                <1>     ; 14/07/2016
                                <1>     ; 09/07/2016
                                <1>     ; 04/07/2016
                                <1>     ; 03/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; /// video mode 13h ///
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgatables.h'
                                <1>     ;
                                <1>     ; Oracle VirtualBox 5.0.24 VGABios Source Code
                                <1>     ; ('vgabios.c', 'vgatables.h', 'vgafonts.h', 'vgarom.asm')
                                <1>     ;
00001BC7 88C4                   <1>     mov    ah, al
00001BC9 B910000000             <1>     mov    ecx, vga_mode_count
00001BCE BE[4A680000]           <1>     mov    esi, vga_modes
00001BD3 31DB                   <1>     xor    ebx, ebx
                                <1> _sm_4:
00001BD5 AC                     <1>     lodsb
00001BD6 38C4                   <1>     cmp    ah, al
00001BD8 7406                   <1>     je     short _sm_5
00001BDA FEC3                   <1>     inc    bl
00001BDC E2F7                   <1>     loop   _sm_4
                                <1>
                                <1>     ; UNIMPLEMENTED VIDEO MODE !
                                <1>     ;xor   eax, eax
                                <1>         ;mov       [video_eax], eax ; 0
                                <1>
                                <1>     ; 26/11/2020
00001BDE F9                     <1>     stc    ; unimplemented video mode ! (cf=1)
                                <1>
00001BDF C3                     <1>     retn
                                <1>
                                <1> ;-----      EBX POINTS TO CORRECT ROW OF INITIALIZATION TABLE
                                <1>
                                <1> _sm_5:      ; 25/07/2016
                                <1>     ;mov   esi, ebx
                                <1>     ;add   esi, vga_memmodel
                                <1>     ;mov   al, [esi]
                                <1>     ; 19/11/2020
00001BE0 8A83[9A680000]         <1>     mov    al, [ebx+vga_memmodel]
00001BE6 A2[62830100]           <1>     mov    [VGA_MTYPE], al
                                <1>
00001BEB 89DF                   <1>     mov    edi, ebx
00001BED 81C7[AA680000]         <1>     add    edi, vga_dac_s
00001BF3 C0E302                 <1>     shl    bl, 2 ; byte -> dword
00001BF6 81C3[5A680000]         <1>     add    ebx, vga_mode_tbl_ptr
                                <1>
                                <1>     ;mov   dword [VGA_BASE], 0B8000h
                                <1>     ;cmp   ah, 0Dh ; [CRT_MODE]
                                <1>     ;jb    short M9
                                <1>     ;mov   dword [VGA_BASE], 0A0000h
                                <1> ;M9:
00001BFC 8B33                   <1>     mov    esi, [ebx]
00001BFE 89F3                   <1>     mov    ebx, esi
00001C00 83C614                 <1>     add    esi, vga_p_cm_pos ; ebx + 20
00001C03 668B06                 <1>     mov    ax, [esi]       ; get the cursor mode from the table
00001C06 66A3[47680000]         <1>     mov    [CURSOR_MODE], ax ; save cursor mode (initial value)
                                <1>     ; al = 6, ah = 7
                                <1>     ; al = 0Dh, ah = 0Eh ; 25/07/2016
00001C0C E893020000             <1>     call   cursor_shape_fix
                                <1>     ; al = 14, ah = 15 (If [CHAR_HEIGHT] = 16)
00001C11 668906                 <1>     mov    [esi], ax
                                <1>
00001C14 56                     <1>     push   esi ; *
                                <1>
                                <1>     ; 17/04/2021
00001C15 B603                   <1>     mov    dh, 03h
                                <1>     ;
00001C17 8A25[35680000]         <1>     mov    ah, [VGA_MODESET_CTL]
00001C1D 80E408                 <1>     and    ah, 8 ; default palette loading ?
00001C20 7520                   <1>     jnz    short _sm_6
                                <1>     ;mov   dx, 3C6h ; VGAREG_PEL_MASK (DAC mask register)
                                <1>     ; 17/04/2021
00001C22 B2C6                   <1>     mov    dl, 0C6h
00001C24 B0FF                   <1>     mov    al, 0FFh ; PEL mask
00001C26 EE                     <1>     out    dx, al
00001C27 8A27                   <1>     mov    ah, [edi] ; DAC model (selection number)
00001C29 E800100000             <1>     call   load_dac_palette
                                <1>     ; ecx = 0
00001C2E F605[35680000]02       <1>     test   byte [VGA_MODESET_CTL], 2 ; gray scale summing
00001C35 740B                   <1>     jz     short _sm_6
00001C37 53                     <1>     push   ebx
00001C38 29DB                   <1>     sub    ebx, ebx ; sub bl, bl
                                <1>     ;mov   cx, 256
                                <1>     ; 02/08/2022
                                <1>     ;sub   ecx, ecx
                                <1>     ; ecx = 0
00001C3A FEC5                   <1>     inc    ch
                                <1>     ; ecx = 256
00001C3C E83F100000             <1>     call   gray_scale_summing
00001C41 5B                     <1>     pop    ebx
                                <1> _sm_6:
                                <1>     ; Reset Attribute Ctl flip-flop
                                <1>     ;mov   dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 17/03/2021
00001C42 B2DA                   <1>     mov    dl, 0DAh ; dx = 3DAh
00001C44 EC                     <1>     in     al, dx
                                <1>     ; Set Attribute Ctl
00001C45 89DE                   <1>     mov    esi, ebx ; addr of params tbl for selected mode
00001C47 83C623                 <1>     add    esi, 35  ; actl regs
00001C4A 30E4                   <1>     xor    ah, ah ; 0
                                <1>     ;mov   dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 17/04/2021
00001C4C B2C0                   <1>     mov    dl, 0C0h
```

```
                                <1> _sm_7:
00001C4E 88E0                   <1>     mov     al, ah
00001C50 EE                     <1>     out     dx, al ; index
00001C51 AC                     <1>     lodsb
                                <1>     ; DX = 3C0h = VGAREG_ACTL_WRITE_DATA
00001C52 EE                     <1>     out     dx, al ; value
00001C53 FEC4                   <1>     inc     ah
00001C55 80FC14                 <1>     cmp     ah, 20 ; number of actl registers
00001C58 72F4                   <1>     jb      short _sm_7
                                <1>     ;
00001C5A 88E0                   <1>     mov     al, ah ; 20
00001C5C EE                     <1>     out     dx, al ; index
00001C5D 28C0                   <1>     sub     al, al ; 0
00001C5F EE                     <1>     out     dx, al ; value
                                <1>     ;
                                <1>     ; Set Sequencer Ctl
00001C60 89DE                   <1>     mov     esi, ebx ; addr of params tbl for selected mode
00001C62 83C605                 <1>     add     esi, 5 ; sequ regs
                                <1>     ;
                                <1>     ;mov    dx, 3C4h  ; VGAREG_SEQU_ADDRESS
                                <1>     ; 17/04/2021
00001C65 B2C4                   <1>     mov     dl, 0C4h
00001C67 EE                     <1>     out     dx, al ; 0
                                <1>     ;inc    dx ; 3C5h ; VGAREG_SEQU_DATA
                                <1>     ; 17/04/2021
00001C68 FEC2                   <1>     inc     dl ; dx = 3C5h
00001C6A B003                   <1>     mov     al, 3
00001C6C EE                     <1>     out     dx, al
00001C6D B401                   <1>     mov     ah, 1
                                <1> _sm_8:
00001C6F 88E0                   <1>     mov     al, ah
                                <1>     ;mov    dx, 3C4h ; VGAREG_SEQU_ADDRESS
                                <1>     ;dec    dx
                                <1>     ; 17/04/2021
00001C71 FECA                   <1>     dec     dl ; dx = 3C4h
00001C73 EE                     <1>     out     dx, al ; index
00001C74 AC                     <1>     lodsb
                                <1>     ;inc    dx ; 3C5h ; VGAREG_SEQU_DATA
                                <1>     ; 17/04/2021
00001C75 FEC2                   <1>     inc     dl
00001C77 EE                     <1>     out     dx, al
00001C78 80FC04                 <1>     cmp     ah, 4 ; number of sequ regs
00001C7B 7304                   <1>     jnb     short _sm_9
00001C7D FEC4                   <1>     inc     ah
00001C7F EBEE                   <1>     jmp     short _sm_8
                                <1> _sm_9:
                                <1>     ; Set Grafx Ctl
00001C81 89DE                   <1>     mov     esi, ebx ; addr of params tbl for selected mode
00001C83 83C637                 <1>     add     esi, 55 ; grdc regs
00001C86 30E4                   <1>     xor     ah, ah ; 0
                                <1> _sm_10:
00001C88 88E0                   <1>     mov     al, ah
                                <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                                <1>     ; 17/04/2021
00001C8A B2CE                   <1>     mov     dl, 0CEh
00001C8C EE                     <1>     out     dx, al
00001C8D AC                     <1>     lodsb
                                <1>     ;inc    dx ; 3CFh ; VGAREG_GRDC_DATA
                                <1>     ; 17/04/2021
00001C8E FEC2                   <1>     inc     dl ; 3CFh
00001C90 EE                     <1>     out     dx, al
00001C91 FEC4                   <1>     inc     ah
00001C93 80FC09                 <1>     cmp     ah, 9 ; number of grdc regs
00001C96 72F0                   <1>     jb      short _sm_10
                                <1>     ;
                                <1>     ; Disable CRTC write protection
                                <1>     ;mov    dx, 3D4h  ; VGAREG_VGA_CRTC_ADDRESS
                                <1>     ; 17/04/2021
00001C98 B2D4                   <1>     mov     dl, 0D4h
                                <1>     ;mov    al, 11h
                                <1>     ;out    dx, al
                                <1>     ;inc    dx
                                <1>     ;sub    al, al
                                <1>     ;out    dx, al
00001C9A 66B81100               <1>     mov     ax, 11h
00001C9E 66EF                   <1>     out     dx, ax
00001CA0 89DE                   <1>     mov     esi, ebx ; addr of params tbl for selected mode
00001CA2 83C60A                 <1>     add     esi, 10 ; crtc regs
                                <1>     ; ah = 0
                                <1> _sm_11:
00001CA5 88E0                   <1>     mov     al, ah
                                <1>     ; dx = 3D4h = VGAREG_VGA_CRTC_ADDRESS
00001CA7 EE                     <1>     out     dx, al ; index
00001CA8 AC                     <1>     lodsb
                                <1>     ;inc    dx  ; VGAREG_VGA_CRTC_ADDRESS + 1
                                <1>     ; 17/04/2021
00001CA9 FEC2                   <1>     inc     dl
00001CAB EE                     <1>     out     dx, al ; value
00001CAC 80FC18                 <1>     cmp     ah, 24 ; number of crtc registers - 1
00001CAF 7306                   <1>     jnb     short _sm_12
00001CB1 FEC4                   <1>     inc     ah
                                <1>     ;dec    dx ; 3D4h
                                <1>     ; 17/04/2021
00001CB3 FECA                   <1>     dec     dl
00001CB5 EBEE                   <1>     jmp     short _sm_11
                                <1> _sm_12:
                                <1>     ; Set the misc register
                                <1>     ;mov    dx, 3CCh ; VGAREG_READ_MISC_OUTPUT
                                <1>     ; 17/04/2021
00001CB7 B2CC                   <1>     mov     dl, 0CCh
00001CB9 8A4309                 <1>     mov     al, [ebx+9] ; misc reg
00001CBC EE                     <1>     out     dx, al
                                <1>     ;
                                <1>     ; Enable video
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 17/04/2021
00001CBD B2C0                   <1>     mov     dl, 0C0h
00001CBF B020                   <1>     mov     al, 20h
00001CC1 EE                     <1>         out     dx, al   ; set bit 5 to 1
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 17/04/2021
00001CC2 B2DA                   <1>     mov     dl, 0DAh
00001CC4 EC                     <1>     in      al, dx
                                <1>     ;
                                <1>     ; 17/11/2020
                                <1>     ;cmp    byte [noclearmem], 0
                                <1>         ;ja short _sm_15
                                <1>
00001CC5 F605[4B830100]80       <1>     test    byte [noclearmem], 80h
00001CCC 753B                   <1>     jnz     short _sm_15
                                <1>
                                <1>     ; 29/07/2016
00001CCE 31C0                   <1>     xor     eax, eax
                                <1>     ;mov    ecx, 4000h ; 16K words (32K)
```

```
                                <1>     ; 02/08/2022
00001CD0 29C9                   <1>     sub     ecx, ecx
00001CD2 B540                   <1>     mov     ch, 40h
                                <1>     ; ecx = 4000h
00001CD4 803D[62830100]02       <1>     cmp      byte [VGA_MTYPE], 2  ; CTEXT, MTEXT, CGA
00001CDB 7715                   <1>     ja      short _sm_14    ; no ? (0A0000h)
00001CDD BF00800B00             <1>     mov     edi, 0B8000h
00001CE2 7409                   <1>     je      short _sm_13   ; CGA graphics mode
                                <1>     ; 08/08/2016
00001CE4 A3[5E830100]           <1>     mov     [VGA_INT43H], eax ; 0 ; default font
00001CE9 66B82007               <1>     mov     ax, 0720h      ; CGA text mode
                                <1> _sm_13:
00001CED 66F3AB                 <1>     rep     stosw
00001CF0 EB17                   <1>     jmp     short _sm_15
                                <1>
                                <1> _sm_14:
00001CF2 BF00000A00             <1>     mov     edi, 0A0000h
                                <1>     ; ecx = 16384 dwords (64K)
                                <1>     ;mov   dx, 3C4h ; VGAREG_SEQU_ADDRESS
                                <1>     ; 17/04/2021
00001CF7 B2C4                   <1>     mov     dl, 0C4h
00001CF9 B002                   <1>     mov     al, 2
00001CFB EE                     <1>     out     dx, al
                                <1>     ;mov   dx, 3C5h ; VGAREG_SEQU_DATA
                                <1>     ;inc   dx
                                <1>     ; 17/04/2021
00001CFC FEC2                   <1>     inc     dl ; 3C5h
00001CFE EC                     <1>     in      al, dx ; mmask
                                <1>     ;push  ax
                                <1>     ; 12/04/2021
00001CFF 50                     <1>     push    eax
00001D00 B00F                   <1>     mov     al, 0Fh ; all planes
00001D02 EE                     <1>     out     dx, al
00001D03 30C0                   <1>     xor     al, al ; 0
00001D05 F3AB                   <1>     rep     stosd  ; ecx = 163684 (64K)
                                <1>     ;pop   ax
                                <1>     ; 12/04/2021
00001D07 58                     <1>     pop     eax
00001D08 EE                     <1>     out     dx, al  ; mmask
                                <1> _sm_15:
                                <1>     ; ebx = addr of params tbl for selected mode
                                <1>     ; 10/08/2016
00001D09 668B03                 <1>     mov     ax, [ebx] ; num of columns, 'twidth'
00001D0C A2[30680000]           <1>     mov     [CRT_COLS], al
                                <1>     ;; 26/07/2016
                                <1>     ;; CRTC_ADDRESS = 3D4h (always)
                                <1>     ;mov   ah, [ebx+1] ; num of rows, 'theightm1'
00001D11 FEC4                   <1>     inc     ah ; 09/07/2016
00001D13 8825[36680000]         <1>     mov     [VGA_ROWS], ah
                                <1>     ; 10/08/2016
00001D19 8A4302                 <1>     mov     al, [ebx+2]
00001D1C A2[32680000]           <1>     mov     [CHAR_HEIGHT], al
                                <1>     ; 29/07/2016
                                <1>     ; length of regen buffer in bytes
00001D21 668B4B03               <1>     mov     cx, [ebx+3] ; 'slength_l'
00001D25 66890D[4C830100]       <1>     mov     [CRT_LEN], cx
                                <1>     ;
                                <1>     ; 27/07/2016
00001D2C 30E4                   <1>     xor     ah, ah
00001D2E A0[DE760100]           <1>     mov     al, [ACTIVE_PAGE] ; may be > 0 for mode 3
                                <1>     ;mul   word [CRT_LEN] ; 4096 for mode 3
00001D33 66F7E1                 <1>     mul     cx ; 29/07/2016
00001D36 66A3[CC760100]         <1>     mov     [CRT_START], ax
                                <1>     ;
00001D3C B060                   <1>     mov     al, 60h
                                <1>     ;cmp   byte [noclearmem], 0
                                <1>     ;jna   short _sm_16
                                <1>     ;add   al, 80h
00001D3E 0A05[4B830100]         <1>     or      al, [noclearmem] ; 17/11/2020
                                <1> _sm_16:
00001D44 A2[33680000]           <1>     mov     [VGA_VIDEO_CTL], al
00001D49 C605[34680000]F9       <1>     mov     byte [VGA_SWITCHES], 0F9h
00001D50 8025[35680000]7F       <1>     and     byte [VGA_MODESET_CTL], 7Fh
                                <1>
00001D57 5E                     <1>     pop     esi ; *
                                <1>
                                <1>     ; 26/07/2016
                                <1>     ; 07/07/2016
00001D58 668B0D[47680000]       <1>     mov     cx, [CURSOR_MODE] ; restore cursor mode (initial value)
00001D5F 66870E                 <1>     xchg    cx, [esi] ; cl = start line, ch = end line
                                <1>                       ; reset to initial value
00001D62 86E9                   <1>     xchg    ch, cl  ; ch = start line, cl = end line
00001D64 66890D[47680000]       <1>     mov     [CURSOR_MODE], cx ; save (fixed) cursor mode
                                <1>
                                <1>     ; 27/07/2016
00001D6B 803D[62830100]02       <1>     cmp     byte [VGA_MTYPE], 2 ; CTEXT, MTEXT
00001D72 7317                   <1>     jnb     short _sm_17
                                <1>
                                <1>     ; Set cursor shape
                                <1>     ;mov   cx, 0607h
                                <1>     ;call  _set_ctype
                                <1>
                                <1>     ; 29/07/2016
00001D74 B40A                   <1>     mov     ah, 10  ; 6845 register for cursor set
00001D76 E827060000             <1>     call    m16     ; output cx register
                                <1>
                                <1>     ; 25/07/2016
00001D7B 803D[2E680000]03       <1>         cmp     byte [CRT_MODE], 03h
00001D82 7507                   <1>     jne     short _sm_17
                                <1>     ; 26/07/2016
                                <1>
00001D84 A0[DE760100]           <1>     mov     al, [ACTIVE_PAGE]
00001D89 EB0B                   <1>     jmp     short _sm_18
                                <1> _sm_17:
                                <1>     ; Set cursor pos for page 0..7
                                <1>     ;sub   ax, ax ; eax = 0
00001D8B 29C0                   <1>     sub     eax, eax ; 17/11/2020
00001D8D BF[CE760100]           <1>     mov     edi, CURSOR_POSN
00001D92 AB                     <1>     stosd
00001D93 AB                     <1>     stosd
00001D94 AB                     <1>     stosd
00001D95 AB                     <1>     stosd
                                <1>     ;; Set active page 0
                                <1>     ;mov   [ACTIVE_PAGE], al ; 0
                                <1> _sm_18:
                                <1>     ; 29/07/2016
00001D96 803D[62830100]02       <1>     cmp     byte [VGA_MTYPE], 2 ; CTEXT, MTEXT
                                <1>     ;jnb   _sm_23
                                <1>     ; 04/08/2022
00001D9D 7205                   <1>     jb      short _sm_24
00001D9F E914020000             <1>     jmp     _set_active_page
                                <1> _sm_24:
                                <1>     ;cmp   byte [CHAR_HEIGHT], 16
                                <1>     ;je    short _sm_19
```

```
                                <1>
                                <1>     ;; copy and activate 8x16 font
                                <1>
                                <1>     ; 26/07/2016
00001DA4 B004                   <1>     mov     al, 04h
                                <1>     ;sub    bl, bl
                                <1>     ; AX = 1104H ; Load ROM 8x16 Character Set
                                <1>     ; (BL = font block to load (EGA: 0-3; VGA: 0-7))
00001DA6 E84C160000             <1>     call    load_text_8_16_pat
                                <1>
                                <1>     ; video_func_1103h:
                                <1>     ; biosfn_set_text_block_specifier:
                                <1>     ; BL = font block selector code
                                <1>     ; NOTE: TRDOS 386 only uses and sets font block 0
                                <1>     ; (It is as BL = 0 for TRDOS 386)
00001DAB 66BAC403               <1>     mov     dx, 3C4h ; VGAREG_SEQU_ADDRESS
                                <1>     ;;mov   ah, bl
                                <1>     ;sub    ah, ah ; 0
                                <1>     ;mov    al, 03h
                                <1>     ; 19/11/2020
00001DAF 66B80300               <1>     mov     ax, 03h
00001DB3 66EF                   <1>     out     dx, ax
                                <1> _sm_19:
                                <1>     ; 29/07/2016
                                <1>     ; 26/07/2016
                                <1>     ; 24/06/2016
                                <1>     ;mov    edi, 0B8000h
                                <1>     ;mov    cx, 4000h ; 16K words (32K)
                                <1>     ;
00001DB5 30C0                   <1>     xor     al, al
00001DB7 3805[49830100]         <1>         cmp     [p_crt_mode], al ; 0
00001DBD 7705                   <1>         ja      short _sm_20 ; 03h, 80h or 83h
                                <1>
                                <1>     ; case 1 - 19/11/2020
                                <1>     ;
                                <1>     ; If [pc_crt_mode] = 0, that means, previous mode is 03h
                                <1>     ; and current mode is not 03h (case 1)
                                <1>
                                <1>     ; 30/07/2016
                                <1>     ; 24/06/2016
                                <1>     ; TRDOS 386 (TRDOS v2) 'set mode' modification
                                <1>     ; (for multiscreen feature):
                                <1>     ; If '_set_mode' procedure is called for video mode 3
                                <1>     ;     while video mode is 3, video page will be cleared
                                <1>     ;     and cursor position of video page will be reset.
                                <1>     ; If '_set_mode' procedure is called for a video mode
                                <1>     ;     except video mode 3, while current video mode
                                <1>     ;     is 3, all video pages of mode 3 will be copied
                                <1>     ;     to 98000h address as backup, before mode change.
                                <1>     ; If '_set_mode' procedure is called for video mode 3
                                <1>     ;     while video mode is not 3 and if there is video
                                <1>     ;     page backup for video mode 3, all (of 8) mode 3
                                <1>     ;     video pages will be restored from 98000h.
                                <1>
                                <1>     ; 19/11/2020
                                <1>     ;mov    [ACTIVE_PAGE], al ; 0
                                <1>
                                <1>     ; Here,
                                <1>     ; video memory already cleared if [noclearmem] <> 80h
                                <1>
                                <1>     ;mov    ax, 0720h
                                <1>     ;mov    cx, 4000h ; 16K words (32K)
                                <1>     ;mov    edi, 0B8000h
                                <1>     ;rep    stosw
                                <1>     ;sub    al, al
                                <1>
                                <1>     ;jmp    short _sm_23
                                <1>
                                <1>     ; Set hardware side for the new active video page
                                <1>
00001DBF E9F4010000             <1>     jmp     _set_active_page ; 19/11/2020
                                <1>
                                <1> _sm_20:
                                <1>     ; 19/11/2020
                                <1>     ; case 2 or case 3 or case 4 - 19/11/2020
                                <1>
                                <1>     ; 19/11/2020
00001DC4 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 3  ; new video mode
00001DCB 754E                   <1>     jne     short _sm_22 ; al = 0 (& video mode <> 03h)
                                <1>                          ; case 4 - 19/11/2020
                                <1>                          ; ([p_crt_mode] = 83h)
                                <1>
                                <1>     ; case 2 or case 3 - 19/11/2020
                                <1>
                                <1>     ;movzx  ebx, byte [ACTIVE_PAGE]
                                <1>     ; 19/11/2020
00001DCD A0[4A830100]           <1>     mov     al, [p_crt_page] ; previous mode 3 active page
                                <1>     ;movzx  ebx, al
                                <1>     ;shl    bl, 1 ; * 2
                                <1>     ;add    ebx, CURSOR_POSN
                                <1>
                                <1>     ; 29/07/2016
00001DD2 F605[49830100]7F       <1>     test    byte [p_crt_mode], 7Fh ; 83h or 80h or 03h
00001DD9 740F                   <1>     jz      short _sm_21   ; do not restore video pages
                                <1>                            ; case 2 - 19/11/2020
                                <1>     ; case 3 - 19/11/2020
                                <1>
                                <1>     ; ([p_crt_mode] = 03h)
                                <1>
                                <1>     ; New video mode is 3 while current video mode is not 3
                                <1>     ; (multi screen) video pages will be restored from 098000h
                                <1>
                                <1>     ; 19/11/2020
00001DDB A2[DE760100]           <1>     mov     [ACTIVE_PAGE], al ; current mode 3 active page
                                <1>
                                <1>     ; 12/12/2020
                                <1>     ;; restore video pages
                                <1>     ;mov    esi, 98000h ; 30/07/2016
                                <1>     ;mov    edi, 0B8000h
                                <1>     ;mov    cx, 2000h ; 8K dwords (32K)
                                <1>     ;rep    movsd
                                <1>     ;
                                <1>     ;; 19/11/2020
                                <1>     ;mov    [p_crt_mode], cl ; reset ('case 3' end condition)
                                <1>     ;
                                <1>     ;; restore cursor positions
                                <1>     ;mov    esi, cursor_pposn
                                <1>     ;mov    edi, CURSOR_POSN
                                <1>     ;;mov   ecx, 4 ; restore all cursor positions (16 bytes)
                                <1>     ;mov    cl, 4
                                <1>     ;rep    movsd
                                <1>
                                <1>     ; 12/12/2020
00001DE0 E89A000000             <1>     call    _restore_mode3_multiscreen
```

```
                                <1>
                                <1>     ;jmp   short _sm_23 ; do not clear current video pages
                                <1>
                                <1>     ; 19/11/2020
00001DE5 E9CE010000             <1>     jmp    _set_active_page
                                <1>
                                <1> _sm_21:
                                <1>     ; 19/11/2020
                                <1>     ; case 2
                                <1>     ;
                                <1>     ; ([p_crt_mode] = 80h)
                                <1>     ;
                                <1>     ; User has requested to set video mode 3 again while
                                <1>     ; current video mode is 3.. that means, set mode 03h
                                <1>     ; parameters again and clear video page.
                                <1>     ; ('noclearmem' option effects the result)
                                <1>
                                <1>     ; 19/11/2020
00001DEA F605[4B830100]80       <1>     test   byte [noclearmem], 80h
00001DF1 7528                   <1>     jnz    short _sm_22   ; 'do not clear video memory'
                                <1>                           ; continue with current text
                                <1>                           ; (user's option/choice)
                                <1>     ; clear video page
                                <1>     ;mov   cx, [CRT_LEN] ; 4096
                                <1>     ;shr   cx, 1 ; 2048 ; 16/11/2020
00001DF3 66B82007               <1>     mov    ax, 0720h
                                <1>     ; 26/11/2020
                                <1>     ;mov   ecx, 2048 ; 4096/2
                                <1>     ; 02/08/2022
00001DF7 29C9                   <1>     sub    ecx, ecx
00001DF9 B508                   <1>     mov    ch, 08h
                                <1>     ; ecx = 0800h
00001DFB BF00800B00             <1>     mov    edi, 0B8000h ; [crt_base]
00001E00 66033D[CC760100]       <1>     add    di, [CRT_START]
00001E07 66F3AB                 <1>     rep    stosw  ; FILL THE REGEN BUFFER WITH BLANKS
                                <1>     ;
                                <1>     ; 19/11/2020
00001E0A A0[DE760100]           <1>     mov    al, [ACTIVE_PAGE] ; 0 to 7 (for video mode 3)
00001E0F 0FB6D8                 <1>     movzx  ebx, al
00001E12 D0E3                   <1>     shl    bl, 1
00001E14 66898B[CE760100]       <1>     mov    [ebx+CURSOR_POSN], cx ; reset cursor position
                                <1> _sm_22:
                                <1>     ;mov   [p_crt_mode], al ; 0 ; reset
                                <1>     ; 19/11/2020
                                <1>     ;and   byte [p_crt_mode], 3 ; 83h -> 3, 80h -> 0
00001E1B 8025[49830100]7F       <1>     and    byte [p_crt_mode], 7Fh ; 83h -> 3, 80h -> 0
                                <1> _sm_23:
                                <1>     ; al = video page number
                                <1>     ; [CRT_LEN] = length of regen buffer in bytes
                                <1>     ;call  _set_active_page
                                <1>     ; 16/11/2020
00001E22 E991010000             <1>     jmp    _set_active_page
                                <1>
                                <1> ;-----     NORMAL RETURN FROM ALL VIDEO RETURNS
                                <1>     ;retn
                                <1>
                                <1> save_mode3_multiscreen:
                                <1>     ; 02/08/2022 (TRDOS v2.0.5)
                                <1>     ; 12/12/2020 (TRDOS v2.0.3)
                                <1>     ; save mode 03h video pages and cursor positions
                                <1>     ;
                                <1>     ; Modified registers: ecx (=0), esi, edi
                                <1>
                                <1>     ; 12/12/2020
                                <1>     ; moved here from '_set_mode'
                                <1>     ; 03/07/2016
                                <1>     ; save video pages
00001E27 BE00800B00             <1>     mov    esi, 0B8000h
00001E2C BF00800900             <1>     mov    edi, 98000h ; 30/07/2016
                                <1>     ;mov   ecx, (0B8000h-0B0000h)/4
                                <1>     ; 02/08/2022
00001E31 29C9                   <1>     sub    ecx, ecx
00001E33 B520                   <1>     mov    ch, 20h
                                <1>     ; ecx = 2000h
00001E35 F3A5                   <1>     rep    movsd
                                <1>
00001E37 C605[49830100]03       <1>     mov    byte [p_crt_mode], 3 ; previous mode, backup sign
                                <1>     ; 26/11/2020
00001E3E 860D[DE760100]         <1>     xchg   cl, [ACTIVE_PAGE]
00001E44 880D[4A830100]         <1>     mov    [p_crt_page], cl  ; save as previous active page
                                <1>
                                <1>     ; save cursor positions
00001E4A BE[CE760100]           <1>     mov    esi, CURSOR_POSN
00001E4F BF[4E830100]           <1>     mov    edi, cursor_pposn ; cursor positions backup
00001E54 B104                   <1>     mov    cl, 4
00001E56 F3A5                   <1>     rep    movsd
00001E58 C3                     <1>     retn
                                <1>
                                <1> restore_mode3_multiscreen:
                                <1>     ; 02/08/2022 (TRDOS v2.0.5)
                                <1>     ; 12/12/2020 (TRDOS v2.0.3)
                                <1>     ; restore mode 03h video pages and cursor positions
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;    settings from the last 'save_mode3_multiscreen'
                                <1>     ;
                                <1>     ; Output:
                                <1>     ;    AL = active video page = [ACTIVE_PAGE]
                                <1>     ;
                                <1>     ; Modified registers: al, ecx (=0), esi, edi
                                <1>
00001E59 A0[4A830100]           <1>     mov    al, [p_crt_page] ; previous mode 3 active page
00001E5E A2[DE760100]           <1>     mov    [ACTIVE_PAGE], al ; current mode 3 active page
                                <1>
                                <1>     ; 12/12/2020
                                <1>     ; moved here from 'vesa_vbe3_pmi'
                                <1>
                                <1>     ; 07/12/2020
                                <1>     ; restore CRT_START according to ACTIVE_PAGE
                                <1>     ;mov   [CRT_START], cx ; 0
                                <1>     ; 12/12/2020
00001E63 66C705[CC760100]00-    <1>     mov    word [CRT_START], 0
00001E6B 00                     <1>
                                <1>
                                <1>     ; check active page and set it again if it is not 0
00001E6C 08C0                   <1>     or     al, al
                                <1>     ;;jz   short vbe3_pmi_7
                                <1>     ;jz    short _restore_mode3_multiscreen
00001E6E 740F                   <1>     jz     short r_m3_ms_1
00001E70 88C1                   <1>     mov    cl, al
                                <1> ;vbe3_pmi_5:
                                <1> r_m3_ms_0:
00001E72 668105[CC760100]00-    <1>     add    word [CRT_START], 4096
00001E7A 10                     <1>
```

```
00001E7B FEC9                <1>     dec    cl
                             <1>     ;jnz   short vbe3_pmi_5
00001E7D 75F3                <1>     jnz    short r_m3_ms_0
                             <1> r_m3_ms_1:
                             <1>     ; 12/12/2020
                             <1>     ; moved here from '_set_mode'
                             <1> _restore_mode3_multiscreen:
                             <1>     ; Modified registers: ecx, esi, edi
                             <1>
                             <1>     ; restore video pages
00001E7F BE00800900          <1>     mov    esi, 98000h ; 30/07/2016
00001E84 BF00800B00          <1>     mov    edi, 0B8000h
                             <1>     ;mov   cx, 2000h ; 8K dwords (32K)
                             <1>     ;mov   ecx, 2000h
                             <1>     ; 02/08/2022
00001E89 29C9                <1>     sub    ecx, ecx
00001E8B B520                <1>     mov    ch, 20h
                             <1>     ; ecx = 2000h
00001E8D F3A5                <1>     rep    movsd
                             <1>
                             <1>     ; 19/11/2020
00001E8F 880D[49830100]      <1>     mov    [p_crt_mode], cl ; reset ('case 3' end condition)
                             <1>
                             <1>     ; restore cursor positions
00001E95 BE[4E830100]        <1>     mov    esi, cursor_pposn
00001E9A BF[CE760100]        <1>     mov    edi, CURSOR_POSN
                             <1>     ;mov   ecx, 4 ; restore all cursor positions (16 bytes)
00001E9F B104                <1>     mov    cl, 4
00001EA1 F3A5                <1>     rep    movsd
00001EA3 C3                  <1>     retn
                             <1>
                             <1> cursor_shape_fix:
                             <1>     ; 12/04/2021
                             <1>     ; 07/07/2016
                             <1>     ; (Cursor start and cursor end line values -6,7-
                             <1>     ; will be fixed depending on character height)
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', ' biosfn_set_cursor_shape (CH,CL)'
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;      AL = cursor start line (=6)
                             <1>     ;      AH = cursor end line (=7)
                             <1>     ; OUTPUT ->
                             <1>     ;      AL = cursor start line (=14)
                             <1>     ;      AH = cursor end line (=15)
                             <1>     ;
                             <1>     ;; if((modeset_ctl&0x01)&&(cheight>8)&&(CL<8)&&(CH<0x20))
                             <1>
                             <1>     ;test  byte [VGA_MODESET_CTL], 1 ; VGA active
                             <1>     ;jz    short csf_3
00001EA4 803D[32680000]08    <1>     cmp    byte [CHAR_HEIGHT], 8
00001EAB 7647                <1>     jna    short csf_3
00001EAD 80FC08              <1>     cmp    ah, 8
00001EB0 7342                <1>     jnb    short csf_3
00001EB2 3C20                <1>     cmp    al, 20h
00001EB4 733E                <1>     jnb    short csf_3
                             <1>     ;
                             <1>     ;push  ax
                             <1>     ; 12/04/2021
00001EB6 50                  <1>     push   eax
                             <1>     ; {
                             <1>     ; if(CL!=(CH+1))
00001EB7 FEC0                <1>     inc    al
00001EB9 38C4                <1>     cmp    ah, al   ; ah != al + 1
00001EBB 740F                <1>         je      short csf_1
                             <1>     ; CH = ((CH+1) * cheight / 8) -1;
00001EBD 8A25[32680000]      <1>     mov    ah, [CHAR_HEIGHT]
00001EC3 F6E4                <1>     mul    ah
00001EC5 C0E803              <1>     shr    al, 3 ; / 8
00001EC8 FEC8                <1>     dec    al ; - 1
00001ECA EB0E                <1>     jmp    short csf_2
                             <1> csf_1:
                             <1>     ; }
                             <1>     ; else         ; ah = al + 1
                             <1>     ; {
00001ECC FEC4                <1>     inc    ah     ; ah = ah + 1
                             <1>     ; CH = ((CL+1) * cheight / 8) - 2;
00001ECE A0[32680000]        <1>     mov    al, [CHAR_HEIGHT]
00001ED3 F6E4                <1>     mul    ah
00001ED5 C0E803              <1>     shr    al, 3 ; / 8
00001ED8 2C02                <1>     sub    al, 2 ; - 2
                             <1>     ; al = 14 (if [CHAR_HEIGHT] = 16)
                             <1> csf_2:
00001EDA 880424              <1>     mov    [esp], al
00001EDD 8A642401            <1>     mov    ah, [esp+1]
                             <1>     ; CL = ((CL+1) * cheight / 8) - 1;
00001EE1 FEC4                <1>     inc    ah
00001EE3 A0[32680000]        <1>     mov    al, [CHAR_HEIGHT]
00001EE8 F6E4                <1>     mul    ah
00001EEA C0E803              <1>     shr    al, 3 ; / 8
00001EED FEC8                <1>     dec    al ; - 1
00001EEF 88442401            <1>     mov    [esp+1], al
                             <1>     ; ah = 15 (if [CHAR_HEIGHT] = 16)
                             <1>     ;
                             <1>     ;pop   ax
                             <1>     ; 12/04/2021
00001EF3 58                  <1>     pop    eax
                             <1> csf_3:
00001EF4 C3                  <1>     retn
                             <1>
                             <1> SET_CTYPE:
                             <1>     ; 04/08/2022 (TRDOS 386 v2.0.5)
                             <1>     ; 12/09/2016
                             <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
00001EF5 803D[2E680000]07    <1>     cmp    byte [CRT_MODE], 7
                             <1>     ;ja    VIDEO_RETURN ; 12/09/2016
                             <1>     ; 04/08/2022
00001EFC 7738                <1>     ja     short set_cpos_inv_vp
00001EFE E805000000          <1>     call   _set_ctype
00001F03 E963FCFFFF          <1>         jmp     VIDEO_RETURN
                             <1>
                             <1> _set_ctype:
                             <1>     ; 02/09/2014 (Retro UNIX 386 v1)
                             <1>     ;
                             <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                             <1>
                             <1>     ; (CH) = BITS 4-0 = START LINE FOR CURSOR
                             <1>     ;  ** HARDWARE WILL ALWAYS CAUSE BLINK
                             <1>     ;  ** SETTING BIT 5 OR 6 WILL CAUSE ERRATIC BLINKING
                             <1>     ;     OR NO CURSOR AT ALL
                             <1>     ; (CL) = BITS 4-0 = END LINE FOR CURSOR
```

```
                                <1>
                                <1> ;-----------------------------------------------
                                <1> ; SET_CTYPE
                                <1> ;   THIS ROUTINE SETS THE CURSOR VALUE
                                <1> ; INPUT
                                <1> ;   (CX) HAS CURSOR VALUE CH-START LINE, CL-STOP LINE
                                <1> ; OUTPUT
                                <1> ;   NONE
                                <1> ;-----------------------------------------------
                                <1>
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ;
                                <1>     ; 07/07/2016
                                <1>     ; Fixing cursor start and stop line depending on
                                <1>     ; current character height (=16)
                                <1>     ; (Note: Default/initial values are 6 and 7.
                                <1>     ; If set values are 6 (start) & 7 (stop) and
                                <1>     ; [CHAR_HEIGHT] = 16 :
                                <1>     ; After fixing, start line will be 14, stop line
                                <1>     ; will be 15.)
                                <1>
                                <1>     ;mov   ax, cx
                                <1>     ; 02/08/2022
00001F08 89C8                   <1>     mov    eax, ecx
                                <1>
00001F0A 86C4                   <1>     xchg   al, ah
                                <1>     ; AL = start line, AH = stop line
00001F0C E893FFFFFF             <1>     call   cursor_shape_fix
                                <1>     ; AL = start line (fixed), AH = stop line (fixed)
                                <1>     ;mov   cx, ax
                                <1>     ; 02/08/2022
00001F11 89C1                   <1>     mov    ecx, eax
00001F13 86E9                   <1>     xchg   ch, cl
                                <1>     ; CH = start line (fixed), CL = stop line (fixed)
                                <1>     ;
00001F15 B40A                   <1>     mov    ah, 10  ; 6845 register for cursor set
00001F17 66890D[47680000]       <1>     mov    [CURSOR_MODE], cx ; save in data area
                                <1>     ;call  m16    ; output cx register
                                <1>     ;retn
00001F1E E97F040000             <1>         jmp     m16
                                <1>
                                <1> SET_CPOS:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 12/09/2016
                                <1>     ; 07/07/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
00001F23 80FF07                 <1>     cmp    bh, 7 ; video page > 7 ; 07/07/2016
                                <1>     ;ja    VIDEO_RETURN
                                <1>     ; 02/08/2022
00001F26 770E                   <1>     ja     short set_cpos_inv_vp
                                <1>     ;
00001F28 803D[2E680000]07       <1>     cmp    byte [CRT_MODE], 7
00001F2F 770A                   <1>     ja     short vga_set_cpos ; 12/09/2016
00001F31 E841040000             <1>     call   _set_cpos
                                <1> set_cpos_inv_vp:   ; 02/08/2022
00001F36 E930FCFFFF             <1>         jmp     VIDEO_RETURN
                                <1>
                                <1> vga_set_cpos:
                                <1>     ; 12/09/2016
                                <1>     ; 09/07/2016
                                <1>     ; set cursor position
                                <1>     ; NOTE: Hardware cursor position will not be set
                                <1>     ;   in any VGA modes (>7)
                                <1>     ;   But, cursor position will be saved into
                                <1>     ;   [CURSOR_POSN].
                                <1>     ;   TRDOS 386 (TRDOS v2.0) uses only one page
                                <1>     ;   (page 0) for all graphics modes.
                                <1>
00001F3B 668915[CE760100]       <1>     mov    [CURSOR_POSN], dx ; save cursor pos for pg 0
                                <1>     ; 04/08/2016
                                <1>     ;mov   bh, [ACTIVE_PAGE] ; = 0
                                <1>     ;call  _set_cpos
00001F42 E924FCFFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> READ_CURSOR:
                                <1>     ; 12/09/2016
                                <1>     ; 07/07/2016
                                <1>     ; 12/05/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>
                                <1> ;-----------------------------------------------------
                                <1> ; READ_CURSOR
                                <1> ;   THIS ROUTINE READS THE CURRENT CURSOR VALUE FROM THE
                                <1> ;   845, FORMATS IT, AND SENDS IT BACK TO THE CALLER
                                <1> ; INPUT
                                <1> ;   BH - PAGE OF CURSOR
                                <1> ; OUTPUT
                                <1> ;   DX - ROW, COLUMN OF THE CURRENT CURSOR POSITION
                                <1> ;   CX - CURRENT CURSOR MODE
                                <1> ;-----------------------------------------------------
                                <1>
                                <1>     ; BH = Video page number (0 to 7)
                                <1>
                                <1>     ; 07/07/2016
00001F47 80FF07                 <1>     cmp    bh, 7 ; video page > 7 (invalid)
00001F4A 7606                   <1>     jna    short read_cursor_1
                                <1>     ; invalid video page (input)
00001F4C 31C9                   <1>     xor    ecx, ecx ; 0
00001F4E 31D2                   <1>     xor    edx, edx ; 0
00001F50 EB15                   <1>     jmp    short read_cursor_2
                                <1> read_cursor_1:
                                <1>     ; 12/09/2016
00001F52 803D[2E680000]07       <1>     cmp    byte [CRT_MODE], 7 ; vga mode
00001F59 772F                   <1>     ja     short vga_get_cpos
                                <1>     ;
00001F5B E81D000000             <1>     call   get_cpos
00001F60 0FB70D[47680000]       <1>     movzx  ecx, word [CURSOR_MODE]
                                <1> read_cursor_2:
00001F67 5D                     <1>     pop    ebp
00001F68 5F                     <1>     pop    edi
00001F69 5E                     <1>     pop    esi
00001F6A 5B                     <1>     pop    ebx
00001F6B 58                     <1>     pop    eax  ; DISCARD SAVED CX AND DX
00001F6C 58                     <1>     pop    eax
                                <1>     ;
                                <1>     ; 10/02/2026 - TRDOS 386 v2.0.10
                                <1>     ; restore previous (user's) cr3
00001F6D A1[60770100]           <1>     mov    eax, [cr3prev]
00001F72 0F22D8                 <1>     mov    cr3, eax
                                <1>     ;
00001F75 A1[3C830100]           <1>     mov    eax, [video_eax] ; 12/05/2016
                                <1>     ;;15/01/2017
```

```
                                <1>     ;;mov   byte [intflg], 0 ; 07/01/2017
00001F7A 1F                     <1>     pop     ds
00001F7B 07                     <1>     pop     es
00001F7C CF                     <1>     iretd
                                <1>
                                <1> get_cpos:
                                <1>     ; 12/05/2016
                                <1>     ; 16/01/2016
                                <1>     ; BH = Video page number (0 to 7)
                                <1>     ;
00001F7D D0E7                   <1>     shl     bh, 1 ; WORD OFFSET
00001F7F 0FB6F7                 <1>     movzx   esi, bh
00001F82 0FB796[CE760100]       <1>     movzx   edx, word [esi+CURSOR_POSN]
00001F89 C3                     <1>     retn
                                <1>
                                <1> vga_get_cpos:
                                <1>     ; 12/09/2016
                                <1>     ; get cursor position (vga)
00001F8A 0FB715[CE760100]       <1>     movzx   edx, word [CURSOR_POSN] ; cursor pos for pg 0
00001F91 31C9                   <1>     xor     ecx, ecx ; Cursor Mode = 0 (invalid)
00001F93 EBD2                   <1>     jmp      short read_cursor_2
                                <1>
                                <1> ACT_DISP_PAGE:
                                <1>     ; 07/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;--------------------------------------------------
                                <1> ; ACT_DISP_PAGE
                                <1> ;   THIS ROUTINE SETS THE ACTIVE DISPLAY PAGE, ALLOWING
                                <1> ;   THE FULL USE OF THE MEMORY SET ASIDE FOR THE VIDEO ATTACHMENT
                                <1> ; INPUT
                                <1> ;   AL HAS THE NEW ACTIVE DISPLAY PAGE
                                <1> ; OUTPUT
                                <1> ;   THE 6845 IS RESET TO DISPLAY THAT PAGE
                                <1> ;--------------------------------------------------
                                <1>     ; 07/07/2016
00001F95 3C07                   <1>     cmp     al, 7   ; > 7 = invalid video page number
                                <1>     ;ja     VIDEO_RETURN
00001F97 7715                   <1>         ja  short adp_2 ; 18/11/2020
                                <1>     ;cmp    byte [CRT_MODE], 3
                                <1>     ;je     short adp_1
                                <1>     ; 18/11/2020
00001F99 8A25[2E680000]         <1>     mov     ah, [CRT_MODE]
00001F9F 80FC03                 <1>     cmp     ah, 3
00001FA2 7605                   <1>     jna     short adp_1 ; mode 01h, 00h (01h), 02h (03h), 03h
00001FA4 80FC07                 <1>     cmp     ah, 7       ; mode 07h (03h)
00001FA7 7505                   <1>     jne     short adp_2
                                <1>     ;and    al, al
                                <1>         ;jnz        VIDEO_RETURN
                                <1>     ;;sub   al, al ; 0 ; force to page 0
                                <1> adp_1:
00001FA9 E805000000             <1>     call    set_active_page
                                <1> adp_2:
00001FAE E9B8FBFFFF             <1>         jmp     VIDEO_RETURN
                                <1>
                                <1> set_active_page:   ; tty_sw
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 09/12/2017
                                <1>     ; 26/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 30/06/2015
                                <1>     ; 04/03/2014  (act_disp_page --> tty_sw)
                                <1>     ; 10/12/2013
                                <1>     ; 04/12/2013
                                <1>     ;
00001FB3 A2[DE760100]           <1>     mov     [ACTIVE_PAGE], al ; save active page value ; [ptty]
                                <1> _set_active_page:
                                <1>     ; 27/06/2015
                                <1>     ;movzx  ebx, al
                                <1>     ; 02/08/2022
00001FB8 0FB6C0                 <1>     movzx   eax, al
00001FBB 89C3                   <1>     mov     ebx, eax
                                <1>     ;
                                <1>     ;cbw    ; 07/09/2014 (ah=0)
                                <1>     ; 02/08/2022
                                <1>     ;sub    ah, ah ; 09/12/2017
00001FBD 66F725[4C830100]       <1>     mul     word [CRT_LEN]  ; get saved length of regen buffer
                                <1>                             ; display page times regen length
                                <1>     ; 10/12/2013
00001FC4 66A3[CC760100]         <1>     mov     [CRT_START], ax ; save start address for later
                                <1>     ;mov    cx, ax ; start address to cx
                                <1>     ; 02/08/2022
00001FCA 89C1                   <1>     mov     ecx, eax
                                <1> _M16:
                                <1>     ;;sar   cx, 1
                                <1>     ;shr    cx, 1   ; divide by 2 for 6845 handling
                                <1>     ; 02/08/2022
00001FCC D1E9                   <1>     shr     ecx, 1
00001FCE B40C                   <1>     mov     ah, 12  ; 6845 register for start address
00001FD0 E8CD030000             <1>     call    m16
                                <1>     ;sal    bx, 1
                                <1>     ; 01/09/2014
00001FD5 D0E3                   <1>     shl     bl, 1   ; *2 for word offset
00001FD7 81C3[CE760100]         <1>     add     ebx, CURSOR_POSN
00001FDD 668B13                 <1>     mov     dx, [ebx] ; get cursor for this page
                                <1>     ; 16/01/2016
                                <1>     ;call   m18
                                <1>     ;retn
00001FE0 E9A9030000             <1>     jmp     m18
                                <1>
                                <1> position:
                                <1>     ; 02/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ; 24/06/2016
                                <1>     ; 12/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 27/06/2015
                                <1>     ; 02/09/2014
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; 04/12/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;----------------------------------------
                                <1> ; POSITION
                                <1> ;   THIS SERVICE ROUTINE CALCULATES THE REGEN BUFFER ADDRESS
                                <1> ;   OF A CHARACTER IN THE ALPHA MODE
                                <1> ; INPUT
                                <1> ;   AX = ROW, COLUMN POSITION
                                <1> ; OUTPUT
                                <1> ;   AX = OFFSET OF CHAR POSITION IN REGEN BUFFER
```

```
                                <1> ;----------------------------------------
                                <1>
                                <1>     ; DX = ROW, COLUMN POSITION
                                <1>     ;movzx  eax, byte [CRT_COLS] ; 27/06/2015
00001FE5 31C0                   <1>     xor     eax, eax ; 02/09/2014
00001FE7 B050                   <1>     mov     al, 80   ; determine bytes to row
00001FE9 F6E6                   <1>     mul     dh       ; row value
                                <1>     ;xor    dh, dh   ; 0
                                <1>     ;add    ax, dx   ; add column value to the result
                                <1>     ; 16/04/2021
00001FEB 00D0                   <1>     add     al, dl
00001FED 80D400                 <1>     adc     ah, 0
                                <1>     ; 02/08/2022
00001FF0 D1E0                   <1>     shl     eax, 1
                                <1>     ;shl    ax, 1   ; * 2 for attribute bytes
                                <1>             ; EAX = AX = OFFSET OF CHAR POSITION IN REGEN BUFFER
00001FF2 C3                     <1>     retn
                                <1>
                                <1> find_position:
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ; 24/06/2016
                                <1>     ; 12/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 27/06/2015
                                <1>     ; 07/09/2014
                                <1>     ; 02/09/2014
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>
00001FF3 0FB6CF                 <1>     movzx   ecx, bh ; video page number
                                <1>     ; 17/04/2021
                                <1>     ;mov    esi, ecx
                                <1>     ;shl    si, 1
                                <1>     ;mov    dx, [esi+CURSOR_POSN]
                                <1>     ;jz     short p21
                                <1>     ;xor    si, si
                                <1>     ; 17/04/2021
00001FF6 31F6                   <1>     xor     esi, esi
00001FF8 D0E1                   <1>     shl     cl, 1
00001FFA 668B91[CE760100]       <1>     mov     dx, [ecx+CURSOR_POSN]
00002001 740B                   <1>     jz      short p21
00002003 D0E9                   <1>     shr     cl, 1
                                <1> p20:
00002005 660335[4C830100]       <1>     add     si, [CRT_LEN] ; 24/06/2016
                                <1>     ;add    si, 80*25*2 ; add length of buffer for one page
0000200C E2F7                   <1>     loop    p20
                                <1> p21:
0000200E 6621D2                 <1>     and     dx, dx
00002011 7407                   <1>     jz      short p22
00002013 E8CDFFFFFF             <1>     call    position ; determine location in regen in page
00002018 01C6                   <1>     add     esi, eax ; add location to start of regen page
                                <1> p22:
                                <1>     ;mov    dx, [addr_6845] ; get base address of active display
                                <1>     ;mov    dx, 03D4h ; I/O address of color card
                                <1>     ;add    dx, 6   ; point at status port
0000201A 66BADA03               <1>     mov     dx, 03DAh ; status port
                                <1>     ; cx = 0
0000201E C3                     <1>     retn
                                <1>
                                <1> SCROLL_UP:
                                <1>     ; 04/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 07/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 12/05/2016
                                <1>     ; 30/01/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 07/09/2014
                                <1>     ; 02/09/2014
                                <1>     ; 01/09/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ; 04/04/2014
                                <1>     ; 04/12/2013
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;---------------------------------------------
                                <1> ; SCROLL UP
                                <1> ;   THIS ROUTINE MOVES A BLOCK OF CHARACTERS UP
                                <1> ;   ON THE SCREEN
                                <1> ; INPUT
                                <1> ;   (AH) = CURRENT CRT MODE
                                <1> ;   (AL) = NUMBER OF ROWS TO SCROLL
                                <1> ;   (CX) = ROW/COLUMN OF UPPER LEFT CORNER
                                <1> ;   (DX) = ROW/COLUMN OF LOWER RIGHT CORNER
                                <1> ;   (BH) = ATTRIBUTE TO BE USED ON BLANKED LINE
                                <1> ;   (DS) = DATA SEGMENT
                                <1> ;   (ES) = REGEN BUFFER SEGMENT
                                <1> ; OUTPUT
                                <1> ;   NONE -- THE REGEN BUFFER IS MODIFIED
                                <1> ;---------------------------------------------
                                <1>
                                <1>     ; 07/07/2016
0000201F 38F5                   <1>     cmp     ch, dh
                                <1>     ;ja     VIDEO_RETURN
                                <1>     ; 04/08/2022
00002021 7709                   <1>     ja      short _s_u_retn
                                <1>
00002023 38D1                   <1>     cmp     cl, dl
                                <1>     ;ja     VIDEO_RETURN
                                <1>     ; 04/08/2022
00002025 7705                   <1>     ja      short _s_u_retn
                                <1>     ;
00002027 E805000000             <1>     call    _scroll_up
                                <1> _s_u_retn:
0000202C E93AFBFFFF             <1>             jmp     VIDEO_RETURN
                                <1>
                                <1> _scroll_up:  ; from 'write_tty'
                                <1>     ;
                                <1>     ; cl = left upper column
                                <1>     ; ch = left upper row
                                <1>     ; dl = right lower column
                                <1>     ; dh = right lower row
                                <1>     ;
                                <1>     ; al = line count
                                <1>     ; bl = attribute to be used on blanked line
                                <1>     ; bh = video page number (0 to 7)
                                <1>
00002031 E89B000000             <1>     call    test_line_count ; 16/01/2016
                                <1>
00002036 8A25[2E680000]         <1>     mov     ah, [CRT_MODE] ; current video mode
                                <1>     ;;cmp   byte [CRT_MODE], 4
                                <1>     ;cmp    ah, 4 ; 07/07/2016
                                <1>     ;jnb    GRAPHICS_UP ; 26/06/2016
                                <1>     ; 18/11/2020
0000203C 80FC04                 <1>     cmp     ah, 4
0000203F 720A                   <1>     jb      short n0
```

```
00002041 80FC07              <1>     cmp    ah, 7 ; TEST FOR BW CARD
                             <1>                  ;(80x25 text, mono)
00002044 7405                <1>     je     short n0 ; same with mode 3 for TRDOS 386
00002046 E91B050000          <1>     jmp    GRAPHICS_UP
                             <1> n0:
                             <1>     ; 07/07/2016
0000204B 80FF07              <1>     cmp    bh, 7 ; video page number
0000204E 7606                <1>     jna    short n1
00002050 8A3D[DE760100]      <1>     mov    bh, [ACTIVE_PAGE]
                             <1> n1:
00002056 88DC                <1>     mov    ah, bl ; attribute
                             <1>     ;push  ax ; *
                             <1>     ; 12/04/2021
00002058 50                  <1>     push   eax ; *
                             <1>     ;mov   esi, [CRT_BASE]
00002059 BE00800B00          <1>         mov     esi, 0B8000h
0000205E 3A3D[DE760100]      <1>         cmp     bh, [ACTIVE_PAGE]
00002064 750B                <1>     jne    short n2
                             <1>     ;
00002066 66A1[CC760100]      <1>         mov     ax, [CRT_START]
0000206C 6601C6              <1>         add     si, ax
0000206F EB11                <1>         jmp     short n4
                             <1> n2:
00002071 20FF                <1>         and     bh, bh
00002073 740D                <1>     jz     short n4
00002075 88F8                <1>     mov    al, bh
                             <1> n3:
00002077 660335[4C830100]    <1>         add si, [CRT_LEN]
0000207E FEC8                <1>         dec al
00002080 75F5                <1>     jnz    short n3
                             <1> n4:
00002082 E85B000000          <1>     call   scroll_position ; 16/01/2016
00002087 7420                <1>         jz      short n6
                             <1>
00002089 01CE                <1>         add     esi, ecx ; from address for scroll
0000208B 88F5                <1>     mov    ch, dh  ; #rows in block
0000208D 28C5                <1>     sub    ch, al  ; #rows to be moved
                             <1> n5:
0000208F E88A000000          <1>     call   n10 ; 16/01/2016
                             <1>
00002094 51                  <1>         push       ecx
00002095 0FB60D[30680000]    <1>     movzx  ecx, byte [CRT_COLS]
0000209C 00C9                <1>     add    cl, cl
0000209E 01CE                <1>         add esi, ecx  ; next line
000020A0 01CF                <1>         add edi, ecx
000020A2 59                  <1>     pop    ecx
                             <1>
000020A3 FECD                <1>     dec    ch      ; count of lines to move
000020A5 75E8                <1>     jnz    short n5 ; row loop
                             <1>     ; ch = 0
000020A7 88C6                <1>     mov    dh, al   ; #rows
                             <1> n6:
                             <1>     ; attribute in ah
000020A9 B020                <1>     mov    al, ' '  ; fill with blanks
                             <1> n7:
000020AB E87B000000          <1>     call   n11 ; 16/01/2016
                             <1>
000020B0 8A0D[30680000]      <1>     mov    cl, [CRT_COLS]
000020B6 00C9                <1>     add    cl, cl
000020B8 01CF                <1>         add edi, ecx
                             <1>
000020BA FECE                <1>     dec    dh
000020BC 75ED                <1>     jnz    short n7
                             <1> n16:
000020BE 3A3D[DE760100]      <1>     cmp    bh, [ACTIVE_PAGE]
000020C4 750A                <1>     jne    short n8
                             <1>
                             <1>     ;cmp   byte [CRT_MODE], 7 ; is this the black and white card
                             <1>     ;je    short n8          ; if so, skip the mode reset
                             <1>
000020C6 A0[2F680000]        <1>     mov    al, [CRT_MODE_SET] ; get the value of mode set
000020CB 66BAD803            <1>     mov    dx, 03D8h ; always set color card port
000020CF EE                  <1>     out    dx, al
                             <1> n8:
000020D0 C3                  <1>     retn
                             <1>
                             <1> test_line_count:
                             <1>     ; 12/04/2021
                             <1>     ; 12/05/2016
                             <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 07/09/2014 (scroll_up)
000020D1 08C0                <1>     or     al, al
000020D3 740C                <1>     jz     short al_set2
                             <1>     ;push  dx
                             <1>     ; 12/04/2021
000020D5 52                  <1>     push   edx
000020D6 28EE                <1>     sub    dh, ch  ; subtract upper row from lower row number
000020D8 FEC6                <1>     inc    dh      ; adjust difference by 1
000020DA 38C6                <1>     cmp    dh, al  ; line count = amount of rows in window?
000020DC 7502                <1>     jne    short al_set1 ; if not the we're all set
000020DE 30C0                <1>     xor    al, al  ; otherwise set al to zero
                             <1> al_set1:
                             <1>     ;pop   dx
                             <1>     ; 12/04/2021
000020E0 5A                  <1>     pop    edx
                             <1> al_set2:
000020E1 C3                  <1>     retn
                             <1>
                             <1> scroll_position:
                             <1>     ; 12/04/2021
                             <1>     ; 26/06/2016
                             <1>     ; 30/01/2016
                             <1>         ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 07/09/2014 (scroll_up)
                             <1>
                             <1>     ; (*) [esp+4] = ax (al = line count, ah = attribute)
                             <1>
                             <1>     ;push  dx
                             <1>     ; 12/04/2021
000020E2 52                  <1>     push   edx
000020E3 6689CA              <1>     mov    dx, cx  ; now, upper left position in DX
000020E6 E8FAFEFFFF          <1>     call   position
000020EB 01C6                <1>     add    esi, eax
000020ED 89F7                <1>     mov    edi, esi
                             <1>     ;pop   dx      ; lower right position in DX
                             <1>     ; 12/04/2021
000020EF 5A                  <1>     pop    edx
000020F0 6629CA              <1>     sub    dx, cx
000020F3 FEC6                <1>     inc    dh      ; dh = #rows
000020F5 FEC2                <1>     inc    dl      ; dl = #cols in block
000020F7 59                  <1>     pop    ecx     ; return address
                             <1>     ;pop   ax      ; * ; al = line count, ah = attribute
                             <1>     ; 12/04/2021
000020F8 58                  <1>     pop    eax ; (*)
```

```
000020F9 51                 <1>     push    ecx     ; return address
000020FA 0FB7C8             <1>     movzx   ecx, ax
000020FD 8A25[30680000]     <1>     mov     ah, [CRT_COLS]
00002103 F6E4               <1>     mul     ah      ; determine offset to from address
                            <1>     ;add    ax, ax  ; *2 for attribute byte
                            <1>     ; 02/08/2022
                            <1>     ;shl    eax, 1
00002105 01C0               <1>     add     eax, eax
                            <1>     ;
                            <1>     ;push   ax      ; offset
                            <1>     ;push   dx
                            <1>     ; 12/04/2021
00002107 50                 <1>     push    eax     ; offset
00002108 52                 <1>     push    edx
                            <1>     ;
                            <1>     ; 04/04/2014
00002109 66BADA03           <1>     mov     dx, 3DAh ; guaranteed to be color card here
                            <1> n9:                         ; wait_display_enable
0000210D EC                 <1>         in      al, dx   ; get port
0000210E A808               <1>     test    al, RVRT ; wait for vertical retrace
00002110 74FB               <1>     jz      short n9 ; wait_display_enable
00002112 B025               <1>     mov     al, 25h
00002114 B2D8               <1>     mov     dl, 0D8h ; address control port
00002116 EE                 <1>     out     dx, al  ; turn off video during vertical retrace
                            <1>     ;pop    dx      ; #rows, #cols
                            <1>             ;pop    ax      ; offset
                            <1>     ; 12/04/2021
00002117 5A                 <1>     pop     edx     ; #rows, #cols
00002118 58                 <1>             pop     eax     ; offset
00002119 6691               <1>     xchg    ax, cx  ;
                            <1>     ; ecx = offset, al = line count, ah = attribute
                            <1>     ;
0000211B 08C0               <1>     or      al, al
0000211D C3                 <1>     retn
                            <1> n10:
                            <1>     ; Move rows
0000211E 88D1               <1>     mov     cl, dl  ; get # of cols to move
00002120 56                 <1>     push    esi
00002121 57                 <1>     push    edi     ; save start address
                            <1> n10r:
00002122 66A5               <1>     movsw           ; move that line on screen
00002124 FEC9               <1>     dec     cl
00002126 75FA               <1>         jnz     short n10r
00002128 5F                 <1>     pop     edi
00002129 5E                 <1>     pop     esi     ; recover addresses
0000212A C3                 <1>     retn
                            <1> n11:
                            <1>     ; Clear rows
                            <1>                     ; dh =  #rows
0000212B 88D1               <1>         mov cl, dl  ; get # of cols to clear
0000212D 57                 <1>         push    edi     ; save address
                            <1> n11r:
0000212E 66AB               <1>         stosw           ; store fill character
00002130 FEC9               <1>     dec     cl
00002132 75FA               <1>         jnz     short n11r
00002134 5F                 <1>         pop     edi     ; recover address
00002135 C3                 <1>     retn
                            <1>
                            <1> SCROLL_DOWN:
                            <1>     ; 12/04/2021
                            <1>     ; 07/07/2016
                            <1>     ; 27/06/2016
                            <1>     ; 26/06/2016
                            <1>     ; 12/05/2016
                            <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                            <1>     ;
                            <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                            <1>
                            <1> ;--------------------------------------------
                            <1> ; SCROLL DOWN
                            <1> ;   THIS ROUTINE MOVES THE CHARACTERS WITHIN A DEFINED
                            <1> ;   BLOCK DOWN ON THE SCREEN, FILLING THE TOP LINES
                            <1> ;   WITH A DEFINED CHARACTER
                            <1> ; INPUT
                            <1> ;   (AH) = CURRENT CRT MODE
                            <1> ;   (AL) = NUMBER OF LINES TO SCROLL
                            <1> ;   (CX) = UPPER LEFT CORNER OF RECION
                            <1> ;   (DX) = LOWER RIGHT CORNER OF REGION
                            <1> ;   (BH) = FILL CHARACTER
                            <1> ;   (DS) = DATA SEGMENT
                            <1> ;   (ES) = REGEN SEGMENT
                            <1> ; OUTPUT
                            <1> ;   NONE -- SCREEN IS SCROLLED
                            <1> ;--------------------------------------------
                            <1>
                            <1>     ; 07/07/2016
00002136 38F5               <1>     cmp     ch, dh
                            <1>     ;ja     VIDEO_RETURN
00002138 7709               <1>     ja      short _s_d_retn ; 18/11/2020
0000213A 38D1               <1>     cmp     cl, dl
                            <1>     ;ja     VIDEO_RETURN
0000213C 7705               <1>     ja      short _s_d_retn ; 18/11/2020
                            <1>     ;
0000213E E805000000         <1>     call    _scroll_down
                            <1> _s_d_retn:
00002143 E923FAFFFF         <1>         jmp     VIDEO_RETURN
                            <1>
                            <1> _scroll_down: ; 27/06/2016
                            <1>
                            <1>     ; cl = left upper column
                            <1>     ; ch = left upper row
                            <1>     ; dl = right lower column
                            <1>     ; dh = right lower row
                            <1>     ;
                            <1>     ; al = line count
                            <1>     ; bl = attribute to be used on blanked line
                            <1>     ; bh = video page number (0 to 7)
                            <1>
                            <1>     ; !!!!
00002148 FD                 <1>     std             ; DIRECTION FOR SCROLL DOWN
                            <1>     ; !!!!
00002149 E883FFFFFF         <1>     call    test_line_count ; 16/01/2016
                            <1>
0000214E 8A25[2E680000]     <1>     mov     ah, [CRT_MODE] ; current video mode
                            <1>     ;;cmp   byte [CRT_MODE], 4
                            <1>     ;cmp    ah, 4 ; 07/07/2016
                            <1>     ;jnb    GRAPHICS_DOWN ; 26/06/2016
                            <1>     ; 18/11/2020
00002154 80FC04             <1>     cmp     ah, 4
00002157 720A               <1>     jb      short _n0
00002159 80FC07             <1>     cmp     ah, 7 ; TEST FOR BW CARD
                            <1>                   ; (80x25 text, mono)
0000215C 7405               <1>     je      short _n0 ; same with mode 3 for TRDOS 386
0000215E E9D5060000         <1>     jmp     GRAPHICS_DOWN
```

```
                                <1> _n0:
                                <1>     ; 07/07/2016
00002163 80FF07                 <1>     cmp     bh, 7 ; video page number
00002166 7606                   <1>     jna     short n12
00002168 8A3D[DE760100]         <1>     mov     bh, [ACTIVE_PAGE]
                                <1>     ;
                                <1> n12:                ; CONTINUE_DOWN
0000216E 88DC                   <1>     mov     ah, bl
                                <1>     ;push   ax      ; * ; save attribute in ah
                                <1>     ; 12/04/2021
00002170 50                     <1>     push    eax
00002171 6689D0                 <1>     mov     ax, dx  ; LOWER RIGHT CORNER
00002174 E869FFFFFF             <1>     call    scroll_position         ; GET REGEN LOCATION
00002179 741F                   <1>     jz      short n14
0000217B 29CE                   <1>     sub     esi, ecx  ; SI IS FROM ADDRESS
0000217D 88F5                   <1>     mov     ch, dh  ; #rows in block
0000217F 28C5                   <1>     sub     ch, al  ; #rows to be moved
                                <1> n13:
00002181 E898FFFFFF             <1>     call    n10     ; MOVE ONE ROW
                                <1>
00002186 51                     <1>     push    ecx
00002187 8A0D[30680000]         <1>     mov     cl, [CRT_COLS]
0000218D 00C9                   <1>     add     cl, cl
0000218F 29CE                   <1>         sub esi, ecx  ; next line
00002191 29CF                   <1>         sub edi, ecx
00002193 59                     <1>         pop ecx
                                <1>
00002194 FECD                   <1>     dec     ch      ; count of lines to move
00002196 75E9                   <1>     jnz     short n13 ; row loop
                                <1>     ; ch = 0
00002198 88C6                   <1>     mov     dh, al  ; #rows
                                <1> n14:
                                <1>     ; attribute in ah
0000219A B020                   <1>     mov     al, ' ' ; fill with blanks
                                <1> n15:
0000219C E88AFFFFFF             <1>     call    n11 ; 16/01/2016
                                <1>
000021A1 8A0D[30680000]         <1>     mov     cl, [CRT_COLS]
000021A7 00C9                   <1>     add     cl, cl
000021A9 29CF                   <1>         sub edi, ecx
                                <1>
000021AB FECE                   <1>     dec     dh
000021AD 75ED                   <1>     jnz     short n15
                                <1>     ;
                                <1>     ; 18/11/2020
000021AF FC                     <1>     cld     ; clear direction flag
                                <1>     ;
000021B0 E909FFFFFF             <1>     jmp     n16 ; 27/06/2016
                                <1>
                                <1> ;   cmp     bh, [ACTIVE_PAGE]
                                <1> ;   jne     short n16
                                <1> ;
                                <1> ;   ;cmp    byte [CRT_MODE], 7    ; is this the black and white card
                                <1> ;   ;je     short n16             ; if so, skip the mode reset
                                <1> ;
                                <1> ;   mov     al, [CRT_MODE_SET] ; get the value of mode set
                                <1> ;   mov     dx, 03D8h ; always set color card port
                                <1> ;   out     dx, al
                                <1> ;n16:
                                <1> ;   ; !!!!
                                <1> ;   cld             ; Clear direction flag !
                                <1> ;   ; !!!!
                                <1> ;   retn
                                <1>
                                <1> READ_AC_CURRENT:
                                <1>     ; 08/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 12/05/2016
                                <1>     ; 18/01/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1>     ; 08/07/2016
000021B5 803D[2E680000]07       <1>         cmp     byte [CRT_MODE], 7 ; 6!?
000021BC 7607                   <1>     jna     short read_ac_c
000021BE 31C0                   <1>     xor     eax, eax
000021C0 E9ABF9FFFF             <1>         jmp     _video_return
                                <1> read_ac_c:
000021C5 E805000000             <1>     call    _read_ac_current
                                <1>     ; 12/05/2016
                                <1>         ;jmp     VIDEO_RETURN
000021CA E9A1F9FFFF             <1>     jmp     _video_return
                                <1>
                                <1> ;---------------------------------------------------------------------
                                <1> ; READ_AC_CURRENT                                                   :
                                <1> ;   THIS ROUTINE READS THE ATTRIBUTE AND CHARACTER AT THE CURRENT     :
                                <1> ;   CURSOR POSITION AND RETURNS THEM TO THE CALLER                    :
                                <1> ; INPUT                                                               :
                                <1> ;   (AH) = CURRENT CRT MODE                                           :
                                <1> ;   (BH) = DISPLAY PAGE ( ALPHA MODES ONLY )                  :
                                <1> ;   (DS) = DATA SEGMENT                                       :
                                <1> ;   (ES) = REGEN SEGMENT                                      :
                                <1> ; OUTPUT                                                      :
                                <1> ;   (AL) = CHARACTER READ                                     :
                                <1> ;   (AH) = ATTRIBUTE READ                                     :
                                <1> ;---------------------------------------------------------------------
                                <1>
                                <1> _read_ac_current:
                                <1>     ; 26/06/2016
                                <1>     ; 12/05/2016
                                <1>     ; 18/01/2016
                                <1>
000021CF 8A25[2E680000]         <1>     mov     ah, [CRT_MODE] ; current video mode
000021D5 80FC04                 <1>     cmp     ah, 4
000021D8 720A                   <1>     jb      short p10
                                <1>     ; 18/11/2020
                                <1>     ;cmp    byte [CRT_MODE], 4
                                <1>     ;jnb    GRAPHICS_READ ; 26/06/2016
                                <1>
000021DA 80FC07                 <1>     cmp     ah, 7 ; TEST FOR BW CARD (80x25 monochrome text)
000021DD 7405                   <1>     je      short p10       ; same with mode 3 in TRDOS 386
000021DF E9A0080000             <1>     jmp     GRAPHICS_READ
                                <1> p10:
000021E4 E80AFEFFFF             <1>     call    find_position  ; GET REGEN LOCATION AND PORT ADDRESS
                                <1>     ;
                                <1>     ; esi = regen location
                                <1>     ; dx = status port
                                <1>     ;
                                <1>
                                <1>     ; 18/11/2020
                                <1>     ; convert display mode to a zero value
                                <1>     ; for 80 column color mode
                                <1>     ;mov    ah, [CRT_MODE]
```

```
                                <1>     ;sub    ah, 2
                                <1>     ;shr    ah, 1
                                <1>     ;jnz    short p13
                                <1>
                                <1>     ; 05/12/2020
                                <1>     ; 18/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ;xor    bl, bl  ; 0
000021E9 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 03h ; 80x25 color text
                                <1>     ;je     short p11    ; Note: Only mode 03h and mode 01h are
                                <1>     ;inc    bl      ; 1  ;       in use by TRDOS 386 as text modes
                                <1>     ;jmp    short p14    ;       (07h, 00h and 02h are redirected)
000021F0 7516                   <1>     jne     short p13
                                <1>
                                <1>     ; 05/12/2020
000021F2 3A3D[DE760100]         <1>     cmp     bh, [ACTIVE_PAGE]
000021F8 750E                   <1>     jne     short p13
                                <1>
                                <1>     ; WAIT FOR HORIZONTAL RETRACE OR VERTICAL RETRACE IF COLOR 80
                                <1> p11:
000021FA FB                     <1>     sti             ; enable interrupts first
                                <1>     ; 05/12/2020
000021FB 90                     <1>     nop
                                <1>     ; 18/11/2020
                                <1>     ;or     bl, bl
                                <1>     ;jnz    short p13 ; mode 01h (and mode 00h) - 40x25 color text
000021FC FA                     <1>     cli             ; block interrupts for single loop
000021FD EC                     <1>     in      al, dx  ; get status from the adapter
000021FE A801                   <1>     test    al, RHRZ ; is horizontal retrace low
00002200 75F8                   <1>     jnz     short p11 ; wait until it is
                                <1> p12:                ;  wait for either retrace high
00002202 EC                     <1>     in      al, dx ; get status again
00002203 A809                   <1>     test    al, RVRT+RHRZ ; is horizontal or vertical retrace high
00002205 74FB                   <1>     jz      short p12 ; wait until either retrace active
                                <1> ;p14:
00002207 FB                     <1>     sti
                                <1> p13:
00002208 81C600800B00           <1>     add     esi, 0B8000h
0000220E 668B06                 <1>     mov     ax, [esi]
                                <1>
00002211 C3                     <1>     retn    ; 18/01/2016
                                <1>
                                <1> WRITE_AC_CURRENT:
                                <1>     ; 08/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 24/06/2016
                                <1>     ; 12/05/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;---------------------------------------------------------------
                                <1> ; WRITE_AC_CURRENT                                          :
                                <1> ;   THTS ROUTINE WRITES THE ATTRIBUTE AND CHARACTER            :
                                <1> ;   AT THE CURRENT CURSOR POSITION                             :
                                <1> ; INPUT                                                        :
                                <1> ;   (AH) = CURRENT CRT MODE                                    :
                                <1> ;   (BH) = DISPLAY PAGE                                     :
                                <1> ;   (CX) = COUNT OF CHARACTERS TO WRITE                     :
                                <1> ;   (AL) = CHAR TO WRITE                                    :
                                <1> ;   (BL) = ATTRIBUTE OF CHAR TO WRITE                       :
                                <1> ;   (DS) = DATA SEGMENT                                     :
                                <1> ;   (ES) = REGEN SEGMENT                                    :
                                <1> ; OUTPUT                                                    :
                                <1> ;   DISPLAY REGEN BUFFER UPDATED                            :
                                <1> ;---------------------------------------------------------------
                                <1>
                                <1>     ; 08/07/2016
00002212 803D[2E680000]07       <1>     cmp     byte [CRT_MODE], 7 ; 6!?
00002219 760A                   <1>     jna     short write_ac_c
                                <1>
0000221B E8B60A0000             <1>     call    vga_write_char_attr
00002220 E946F9FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> write_ac_c:
00002225 E834000000             <1>     call    _write_c_current
                                <1>
0000222A 0FB6F7                 <1>     movzx   esi, bh ; video page number (0 to 7)
0000222D 889E[37680000]         <1>     mov     [esi+chr_attrib], bl ; color/attribute
                                <1>
00002233 E933F9FFFF             <1>         jmp     VIDEO_RETURN
                                <1>
                                <1> WRITE_C_CURRENT:
                                <1>     ; 08/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 12/05/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;---------------------------------------------------------------
                                <1> ; WRITE_C_CURRENT                                           :
                                <1> ;   THIS ROUTINE WRITES THE CHARACTER AT                    :
                                <1> ;   THE CURRENT CURSOR POSITION, ATTRIBUTE UNCHANGED        :
                                <1> ; INPUT                                                        :
                                <1> ;   (AH) = CURRENT CRT MODE                                    :
                                <1> ;   (BH) = DISPLAY PAGE                                     :
                                <1> ;   (CX) = COUNT OF CHARACTERS TO WRITE                     :
                                <1> ;   (AL) = CHAR TO WRITE                                    :
                                <1> ;   (DS) = DATA SEGMENT                                     :
                                <1> ;   (ES) = REGEN SEGMENT                                    :
                                <1> ; OUTPUT                                                    :
                                <1> ;   DISPLAY REGEN BUFFER UPDATED                            :
                                <1> ;---------------------------------------------------------------
                                <1>
                                <1>     ; 08/07/2016
00002238 803D[2E680000]07       <1>     cmp     byte [CRT_MODE], 7 ; 6!?
0000223F 760A                   <1>     jna     short write_c_c
                                <1>
00002241 E8900A0000             <1>     call    vga_write_char_only
00002246 E920F9FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> write_c_c:
                                <1>     ;and    bh, 7 ; video page number (<= 7)
0000224B 0FB6F7                 <1>     movzx   esi, bh
0000224E 8A9E[37680000]         <1>     mov     bl, [esi+chr_attrib]
                                <1>
00002254 E805000000             <1>     call    _write_c_current
00002259 E90DF9FFFF             <1>         jmp     VIDEO_RETURN
                                <1>
                                <1> _write_c_current:  ; from 'write_tty'
                                <1>     ; 12/04/2021
                                <1>     ; 18/11/2020
                                <1>     ; 26/06/2016
                                <1>     ; 24/06/2016
```

```
                                <1>     ; 12/05/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; 18/01/2014
                                <1>     ; 04/12/2013
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85 VIDEO DISPLAY BIOS
                                <1>
                                <1>     ; 18/11/2020
0000225E 8A25[2E680000]         <1>     mov     ah, [CRT_MODE] ; current video mode
00002264 80FC04                 <1>     cmp     ah, 4
00002267 720A                   <1>     jb      short p40
                                <1>
                                <1>     ;cmp    byte [CRT_MODE], 4
                                <1>         ;jnb       GRAPHICS_WRITE ; 26/06/2016
                                <1>
00002269 80FC07                 <1>     cmp     ah, 7 ; TEST FOR BW CARD
0000226C 7405                   <1>     je      short p40
0000226E E963070000             <1>     jmp     GRAPHICS_WRITE
                                <1> p40:
                                <1>     ; al = character
                                <1>     ; bl = color/attribute
                                <1>     ; bh = video page
                                <1>     ; cx = count of characters to write
                                <1>     ;push   dx
                                <1>     ; 12/04/2021
00002273 52                     <1>     push    edx ; *
00002274 88DC                   <1>     mov     ah, bl  ; color/attribute (12/05/2016)
                                <1>     ;push   ax      ; save character & attribute/color
                                <1>     ;push   cx
                                <1>     ; 12/04/2021
00002276 50                     <1>     push    eax ; ** ; save character & attribute/color
00002277 51                     <1>     push    ecx ; ***
00002278 E876FDFFFF             <1>     call    find_position  ; get regen location and port address
                                <1>     ;pop    cx
                                <1>     ; 12/04/2021
0000227D 59                     <1>     pop     ecx ; ***
                                <1>     ; esi = regen location
                                <1>     ; dx = status port
                                <1>     ;
0000227E 81C600800B00           <1>     add     esi, 0B8000h ; 30/08/2014 (crt_base)
                                <1>     ;
                                <1>     ; 18/11/2020
                                <1>     ; convert display mode to a zero value
                                <1>     ; for 80 column color mode
                                <1>     ;mov    ah, [CRT_MODE]
                                <1>     ;sub    ah, 2
                                <1>     ;shr    ah, 1
                                <1>     ;jnz    short p44    ; 26/06/2016
                                <1>
                                <1>     ; 05/12/2020
                                <1>     ; 18/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ;xor    bl, bl ; 0
00002284 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 03h ; 80x25 color text
                                <1>     ;je     short p41    ; Note: Only mode 03h and mode 01h are
                                <1>     ;inc    bl      ; 1  ;       in use by TRDOS 386 as text modes
                                <1>     ;jmp    short p43    ;       (07h, 00h and 02h are redirected)
                                <1>     ; 05/12/2020
0000228B 751A                   <1>     jne     short p44
                                <1> p46:
                                <1>     ; 05/12/2020
0000228D 3A3D[DE760100]         <1>     cmp      bh, [ACTIVE_PAGE]
00002293 7512                   <1>     jne     short p44
                                <1>
                                <1>     ; WAIT FOR HORIZONTAL RETRACE OR VERTICAL RETRACE IF COLOR 80
                                <1> p41:
                                <1>     ; 05/12/2020
00002295 FB                     <1>     sti             ; enable interrupts first
00002296 90                     <1>     nop
                                <1>     ; 18/11/2020
                                <1>     ;or     bl, bl
                                <1>     ;jnz    short p44 ; mode 01h (and mode 00h) - 40x25 color text
00002297 FA                     <1>     cli             ; block interrupts for single loop
00002298 EC                     <1>     in      al, dx  ; get status from the adapter
00002299 A808                   <1>     test    al, RVRT ; check for vertical retrace first
0000229B 7509                   <1>     jnz     short p43 ; Do fast write now if vertical retrace
0000229D A801                   <1>     test    al, RHRZ ; is horizontal retrace low
0000229F 75F4                   <1>     jnz     short p41 ; wait until it is
                                <1> p42:                ;  wait for either retrace high
000022A1 EC                     <1>     in      al, dx ; get status again
000022A2 A809                   <1>     test    al, RVRT+RHRZ ; is horizontal or vertical retrace high
000022A4 74FB                   <1>     jz      short p42 ; wait until either retrace active
                                <1> p43:
000022A6 FB                     <1>     sti
                                <1> p44:
000022A7 668B0424               <1>     mov     ax, [esp] ; restore the character (al) & attribute (ah)
000022AB 668906                 <1>     mov     [esi], ax
                                <1>
000022AE 6649                   <1>     dec     cx
000022B0 740D                   <1>     jz      short p45
                                <1>
000022B2 46                     <1>     inc     esi
000022B3 46                     <1>     inc     esi
                                <1>
                                <1>     ; 05/12/2020
000022B4 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 03h
000022BB 75EA                   <1>     jne     short p44
                                <1>     ;jmp    short p41
000022BD EBCE                   <1>     jmp     short p46
                                <1> p45:
                                <1>     ;pop    ax
                                <1>     ;pop    dx
                                <1>     ; 12/04/2021
000022BF 58                     <1>     pop     eax ; **
000022C0 5A                     <1>     pop     edx ; *
000022C1 C3                     <1>     retn
                                <1>
                                <1> ; 09/07/2016
                                <1> ; 26/06/2016
                                <1> ; 24/06/2016
                                <1> ; 12/05/2016
                                <1> ; 18/01/2016
                                <1> ; 16/01/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; 30/06/2015
                                <1> ; 27/06/2015
                                <1> ; 11/03/2015
                                <1> ; 02/09/2014
                                <1> ; 30/08/2014
                                <1> ; VIDEO FUNCTIONS
                                <1> ; (write_tty - Retro UNIX 8086 v1 - U9.ASM, 01/02/2014)
                                <1>
                                <1> WRITE_TTY:
                                <1>     ; 09/12/2017
                                <1>     ; 09/07/2016
```

```
                                <1>     ; 01/07/2016
                                <1>     ; 26/06/2016
                                <1>     ; 24/06/2016
                                <1>     ; 13/05/2016
                                <1>     ; 12/05/2016
                                <1>     ; 30/01/2016
                                <1>     ; 18/01/2016
                                <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 13/08/2015
                                <1>     ; 02/09/2014
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ; 01/02/2014 (Retro UNIX 8086 v1 - last update)
                                <1>     ; 03/12/2013 (Retro UNIX 8086 v1 - beginning)
                                <1>     ; (Modified registers: EAX, EBX, ECX, EDX, ESI, EDI)
                                <1>     ;
                                <1>     ; INPUT -> AL = Character to be written
                                <1>     ;         BL = Color (Forecolor, Backcolor)
                                <1>     ;         BH = Video Page (0 to 7)
                                <1>
                                <1>     ; 09/07/2016
000022C2 803D[2E680000]07       <1>         cmp     byte [CRT_MODE], 7
000022C9 760A                   <1>     jna     short write_tty_cga
                                <1>
000022CB E8EB0C0000             <1>     call    vga_write_teletype
000022D0 E996F8FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> write_tty_cga:
                                <1>     ; 13/05/2016
                                <1>     ;call   _write_tty
                                <1>     ; 01/07/2016
000022D5 E818000000             <1>     call    _write_tty_m3
000022DA E98CF8FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> RVRTequ     00001000b       ; VIDEO VERTICAL RETRACE BIT
                                <1> RHRZequ     00000001b       ; VIDEO HORIZONTAL RETRACE BIT
                                <1>
                                <1> ; Derived from "WRITE_TTY" procedure of IBM "pc-at" rombios source code
                                <1> ; (06/10/1985), 'video.asm', INT 10H, VIDEO_IO
                                <1> ;
                                <1> ; 06/10/85  VIDEO DISPLAY BIOS
                                <1> ;
                                <1> ;--- WRITE_TTY ----------------------------------------------------------------
                                <1> ;                                                                   :
                                <1> ;   THIS INTERFACE PROVIDES A TELETYPE LIKE INTERFACE TO THE                :
                                <1> ;   VIDEO CARDS. THE INPUT CHARACTER IS WRITTEN TO THE CURRENT              :
                                <1> ;   CURSOR POSITION, AND THE CURSOR IS MOVED TO THE NEXT POSITION.          :
                                <1> ;   IF THE CURSOR LEAVES THE LAST COLUMN OF THE FIELD, THE COLUMN           :
                                <1> ;   IS SET TO ZERO, AND THE ROW VALUE IS INCREMENTED. IF THE ROW            :
                                <1> ;   ROW VALUE LEAVES THE FIELD, THE CURSOR IS PLACED ON THE LAST ROW,       :
                                <1> ;   FIRST COLUMN, AND THE ENTIRE SCREEN IS SCROLLED UP ONE LINE.            :
                                <1> ;   WHEN THE SCREEN IS SCROLLED UP, THE ATTRIBUTE FOR FILLING THE           :
                                <1> ;   NEWLY BLANKED LINE IS READ FROM THE CURSOR POSITION ON THE PREVIOUS              :
                                <1> ;   LINE BEFORE THE SCROLL, IN CHARACTER MODE. IN GRAPHICS MODE,            :
                                <1> ;   THE 0 COLOR IS USED.                                                    :
                                <1> ;   ENTRY --                                                                :
                                <1> ;     (AH) = CURRENT CRT MODE                                               :
                                <1> ;     (AL) = CHARACTER TO BE WRITTEN                                        :
                                <1> ;       NOTE THAT BACK SPACE, CARRIAGE RETURN, BELL AND LINE FEED ARE :
                                <1> ;       HANDLED AS COMMANDS RATHER THAN AS DISPLAY GRAPHICS CHARACTERS:
                                <1> ;     (BL) = FOREGROUND COLOR FOR CHAR WRITE IF CURRENTLY IN A GRAPHICS MODE  :
                                <1> ;   EXIT --                                                                 :
                                <1> ;     ALL REGISTERS SAVED                                                   :
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> ; 02/08/2022 (TRDOS 386 v2.0.5)
                                <1> ; 18/11/2020 (TRDOS 386 v2.0.3)
                                <1> ; 09/12/2017
                                <1> ; 08/07/2016
                                <1> ; 26/06/2016
                                <1> ; 24/06/2016
                                <1> _write_tty:
                                <1>     ; 13/05/2016
                                <1>     ; --- 18/11/2020 ---
                                <1>     ; NOTE:
                                <1>     ; Only kernel calls "_write_tty" procedure...
                                <1>     ; TRDOS 386 v2 kernel uses video mode 3 for displaying
                                <1>     ; (some error) messages and also mainprog command interpreter
                                <1>     ; (in kernel) uses "_write_tty".
                                <1>     ; So, here video mode must be set to 3 if it is not 3.
                                <1>
000022DF FA                     <1>     cli     ; disable interrupts
                                <1>     ;
                                <1>     ; 01/09/2014
000022E0 803D[2E680000]03       <1>     cmp     byte [CRT_MODE], 3
000022E7 7409                   <1>     je      short _write_tty_m3
                                <1>     ;
                                <1> set_mode_3:
000022E9 53                     <1>     push    ebx
000022EA 50                     <1>     push    eax
                                <1>     ;call   _set_mode
                                <1>     ; 18/11/2020
000022EB E893F8FFFF             <1>     call    set_txt_mode  ; set video mode to 03h
000022F0 58                     <1>     pop     eax
000022F1 5B                     <1>     pop     ebx
                                <1>     ;
                                <1> _write_tty_m3: ; 24/06/2016 (m3 -> _write_tty_m3)
000022F2 0FB6F7                 <1>     movzx   esi, bh ; 12/05/2016
                                <1>     ;shl    si, 1
                                <1>     ; 02/08/2022
000022F5 D1E6                   <1>     shl     esi, 1
000022F7 81C6[CE760100]         <1>     add     esi, CURSOR_POSN
000022FD 668B16                 <1>     mov     dx, [esi]
                                <1>     ;
                                <1>     ; dx now has the current cursor position
                                <1>     ;
00002300 3C0D                   <1>     cmp     al, 0Dh ; CR    ; is it carriage return or control character
                                <1>     ;jbe    short u8
                                <1>     ; 17/04/2021
00002302 7241                   <1>     jb      short u8
00002304 746F                   <1>     je      short u9
                                <1>     ;
                                <1>     ; write the char to the screen
                                <1> u0:
                                <1>     ; al = character
                                <1>     ; bl = attribute/color
                                <1>     ; bh = video page number (0 to 7)
                                <1>     ;
                                <1>     ;mov    cx, 1  ; 24/06/2016
                                <1>     ; 02/08/2022
00002306 29C9                   <1>     sub     ecx, ecx
00002308 FEC1                   <1>     inc     cl ; ecx = 1
                                <1>     ; cx = count of characters to write
                                <1>     ;
0000230A E84FFFFFFF             <1>     call    _write_c_current ; 16/01/2015
```

```
                                <1>     ;
                                <1>     ; position the cursor for next char
0000230F FEC2                   <1>     inc     dl              ; next column
00002311 3A15[30680000]         <1>     cmp     dl, [CRT_COLS]  ; test for column overflow
                                <1>     ;jne    _set_cpos
                                <1>     ; 02/08/2022
00002317 7402                   <1>     je      short u13
00002319 EB5C                   <1>     jmp     _set_cpos
                                <1> u13:
0000231B B200                   <1>     mov     dl, 0           ; column = 0
                                <1> u10:                        ; (line feed found)
0000231D 80FE18                 <1>     cmp     dh, 25-1        ; check for last row
00002320 721F                   <1>     jb      short u6
                                <1>     ;
                                <1>     ; scroll required
                                <1> u1:
                                <1>     ; SET CURSOR POSITION (04/12/2013)
00002322 E850000000             <1>     call    _set_cpos
                                <1>     ;
                                <1>     ; determine value to fill with during scroll
                                <1> u2:
                                <1>     ; bh = video page number
                                <1>     ;
00002327 E8A3FEFFFF             <1>     call    _read_ac_current ; 18/01/2016
                                <1>     ;
                                <1>     ; al = character, ah = attribute
                                <1>     ; bh = video page number
                                <1>     ; 18/11/2020
0000232C 88E3                   <1>     mov     bl, ah ; color/attribute
                                <1> u3:
                                <1>     ;;mov   ax, 0601h       ; scroll one line
                                <1>     ;;sub   cx, cx          ; upper left corner
                                <1>     ;;mov   dh, 25-1        ; lower right row
                                <1>     ;;;mov  dl, [CRT_COLS]
                                <1>     ;mov    dl, 80          ; lower right column
                                <1>     ;;dec   dl
                                <1>     ;;mov   dl, 79
                                <1>
                                <1>     ;;call  scroll_up       ; 04/12/2013
                                <1>     ;;; 11/03/2015
                                <1>     ; 02/09/2014
                                <1>     ;;;mov  cx, [crt_ulc] ; Upper left corner  (0000h)
                                <1>     ;;;mov  dx, [crt_lrc] ; Lower right corner (184Fh)
                                <1>     ; 11/03/2015
                                <1>     ;sub    cx, cx
                                <1>     ; 17/04/2021
0000232E 29C9                   <1>     sub     ecx, ecx
                                <1>     ;mov    dx, 184Fh ; dl = 79 (column), dh = 24 (row)
                                <1>     ; 18/11/2020
00002330 B618                   <1>     mov     dh, 25-1
00002332 8A15[30680000]         <1>     mov     dl, [CRT_COLS]
00002338 FECA                   <1>     dec     dl
                                <1>     ;
0000233A B001                   <1>     mov     al, 1           ; scroll 1 line up
                                <1>             ; ah = attribute
                                <1>     ;mov    bl, al ; 12/05/2016
0000233C E9F0FCFFFF             <1>     jmp     _scroll_up      ; 16/01/2016
                                <1> ;u4:
                                <1>     ;;int   10h             ; video-call return
                                <1>                             ; scroll up the screen
                                <1>                             ; tty return
                                <1> ;u5:
                                <1>     ;retn                   ; return to the caller
                                <1>
                                <1> u6:                         ; set-cursor-inc
00002341 FEC6                   <1>     inc     dh              ; next row
                                <1>                             ; set cursor
                                <1> ;u7:
                                <1>     ;;mov   ah, 02h
                                <1>     ;;jmp   short u4        ; establish the new cursor
                                <1>     ;call   _set_cpos
                                <1>     ;jmp    short u5
00002343 EB32                   <1>     jmp      _set_cpos
                                <1>
                                <1>     ; check for control characters
                                <1> u8:
                                <1>     ;je     short u9 ; 17/04/2021
00002345 3C0A                   <1>     cmp     al, 0Ah         ; is it a line feed (0Ah)
00002347 74D4                   <1>     je      short u10
00002349 3C07                   <1>     cmp     al, 07h         ; is it a bell
0000234B 747C                   <1>     je      short u11
0000234D 3C08                   <1>     cmp     al, 08h         ; is it a backspace
                                <1>     ;jne    short u0
0000234F 741C                   <1>     je      short bs        ; 12/12/2013
                                <1>     ; 12/12/2013 (tab stop)
00002351 3C09                   <1>     cmp     al, 09h         ; is it a tab stop
00002353 75B1                   <1>     jne     short u0
00002355 88D0                   <1>     mov     al, dl
                                <1>     ;cbw
00002357 30E4                   <1>     xor     ah, ah ; 09/12/2017
00002359 B108                   <1>     mov     cl, 8
0000235B F6F1                   <1>     div     cl
0000235D 28E1                   <1>     sub     cl, ah
                                <1> ts:
                                <1>     ; 02/09/2014
                                <1>     ; 01/09/2014
                                <1>     ;mov    al, 20h
                                <1> tsloop:
                                <1>     ;push   cx
                                <1>     ; 12/04/2021
0000235F 51                     <1>     push    ecx ; *
                                <1>     ; 18/11/2020
                                <1>     ;push   ax
                                <1>     ;;mov   bh, [ACTIVE_PAGE]
                                <1>     ; 05/12/2020
                                <1>     ;push   bx
00002360 B020                   <1>     mov     al, 20h ; al = space (blank)
                                <1>                     ; bl = color/attribute
00002362 E88BFFFFFF             <1>     call    _write_tty_m3 ; 24/06/2016 (m3 -> _write_tty_m3)
                                <1>     ; 05/12/2020
                                <1>     ; bx is preserved in '_write_tty_m3'
                                <1>     ; 18/11/2020
                                <1>     ;pop    bx  ; BL = color/attribute, Bh = video page
                                <1>     ;pop    ax  ; AL = character
                                <1>     ; 12/04/2021
                                <1>     ;pop    cx
00002367 59                     <1>     pop     ecx ; *
00002368 FEC9                   <1>     dec     cl
0000236A 75F3                   <1>     jnz     short tsloop
0000236C C3                     <1>     retn
                                <1> bs:
                                <1>     ; back space found
                                <1>
0000236D 08D2                   <1>     or      dl, dl          ; is it already at start of line
```

```
                                <1>     ;je     short u7        ; set_cursor
0000236F 7406                   <1>     jz      short _set_cpos
                                <1>     ;dec    dx              ; no -- just move it back
                                <1>     ; 17/04/2021
00002371 FECA                   <1>     dec     dl              ; move to 1 column back
                                <1>     ;jmp    short u7
00002373 EB02                   <1>     jmp     short _set_cpos
                                <1>
                                <1>     ; carriage return found
                                <1> u9:
00002375 B200                   <1>     mov     dl, 0           ; move to first column
                                <1>     ;jmp    short u7
                                <1>     ;jmp    short _set_cpos ; 30/01/2016
                                <1>
                                <1>     ; line feed found
                                <1> ;u10:
                                <1> ;   cmp     dh, 25-1        ; bottom of screen
                                <1> ;   jne     short u6        ; no, just set the cursor
                                <1> ;       jmp     u1              ; yes, scroll the screen
                                <1>
                                <1> _set_cpos:
                                <1>     ; 02/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ; 12/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 27/06/2015
                                <1>     ; 01/09/2014
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ;
                                <1>     ; 04/12/2013 - 12/12/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; VIDEO.ASM - 06/10/85  VIDEO DISPLAY BIOS
                                <1>     ;
                                <1> ;---------------------------------------------
                                <1> ; SET_CPOS
                                <1> ;   THIS ROUTINE SETS THE CURRENT CURSOR POSITION TO THE
                                <1> ;   NEW X-Y VALUES PASSED
                                <1> ; INPUT
                                <1> ;   DX - ROW,COLUMN OF NEW CURSOR
                                <1> ;   BH - DISPLAY PAGE OF CURSOR
                                <1> ; OUTPUT
                                <1> ;   CURSOR ID SET AT 6845 IF DISPLAY PAGE IS CURRENT DISPLAY
                                <1> ;---------------------------------------------
                                <1>     ;
00002377 BE[CE760100]           <1>     mov     esi, CURSOR_POSN
0000237C 0FB6C7                 <1>         movzx   eax, bh    ; BH = video page number
                                <1> ;   or      al, al
                                <1> ;   jz      short _set_cpos_0
0000237F D0E0                   <1>         shl     al, 1   ; word offset
00002381 01C6                   <1>         add     esi, eax
                                <1> ;_set_cpos_0:
00002383 668916                 <1>     mov     [esi], dx ; save the pointer
00002386 383D[DE760100]         <1>     cmp     [ACTIVE_PAGE], bh
0000238C 7532                   <1>     jne     short m17
                                <1>     ;call   m18     ; CURSOR SET
                                <1> ;m17:                       ; SET_CPOS_RETURN
                                <1>     ; 01/09/2014
                                <1> ;   retn
                                <1>             ; DX = row/column
                                <1> m18:
0000238E E852FCFFFF             <1>     call    position ; determine location in regen buffer
00002393 668B0D[CC760100]       <1>     mov     cx, [CRT_START]
0000239A 6601C1                 <1>     add     cx, ax  ; add char position in regen buffer
                                <1>                     ; to the start address (offset) for this page
0000239D 66D1E9                 <1>     shr     cx, 1   ; divide by 2 for char only count
000023A0 B40E                   <1>     mov     ah, 14  ; register number for cursor
                                <1>     ;call   m16     ; output value to the 6845
                                <1>     ;retn
                                <1>
                                <1>     ;----- THIS ROUTINE OUTPUTS THE CX REGISTER
                                <1>     ;       TO THE 6845 REGISTERS NAMED IN (AH)
                                <1> m16:
000023A2 FA                     <1>     cli
                                <1>     ;mov    dx, [addr_6845] ; address register
000023A3 66BAD403               <1>     mov     dx, 03D4h ; I/O address of color card
000023A7 88E0                   <1>     mov     al, ah  ; get value
000023A9 EE                     <1>     out     dx, al  ; register set
                                <1>     ;inc    dx      ; data register
                                <1>     ; 17/04/2021
000023AA FEC2                   <1>     inc     dl
000023AC EB00                   <1>     jmp     $+2     ; i/o delay
000023AE 88E8                   <1>     mov     al, ch  ; data
000023B0 EE                     <1>     out     dx, al
                                <1>     ;dec    dx
                                <1>     ; 17/04/2021
000023B1 FECA                   <1>     dec     dl
000023B3 88E0                   <1>     mov     al, ah
000023B5 FEC0                   <1>     inc     al      ; point to other data register
000023B7 EE                     <1>     out     dx, al  ; set for second register
                                <1>     ;inc    dx
                                <1>     ; 17/04/2021
000023B8 FEC2                   <1>     inc     dl
000023BA EB00                   <1>     jmp     $+2     ; i/o delay
000023BC 88C8                   <1>     mov     al, cl  ; second data value
000023BE EE                     <1>     out     dx, al
000023BF FB                     <1>     sti
                                <1> m17:
000023C0 C3                     <1>     retn
                                <1>
                                <1> _beep:
                                <1>     ; 12/02/2021 (TRDOS v2.0.3)
000023C1 FA                     <1>     cli
000023C2 E811000000             <1>     call    beep
000023C7 FB                     <1>     sti
000023C8 C3                     <1>     retn
                                <1>
                                <1> beeper:
                                <1>     ; 04/08/2016
                                <1>     ; 12/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; 18/01/2014
                                <1>     ; 03/12/2013
                                <1>     ; bell found
                                <1> u11:
000023C9 FB                     <1>     sti
000023CA 3A3D[DE760100]         <1>     cmp     bh, [ACTIVE_PAGE]
000023D0 7552                   <1>     jne     short u12       ; Do not sound the beep
                                <1>                             ; if it is not written on the active page
                                <1> beeper_gfx: ; 04/08/2016
000023D2 66B93305               <1>     mov     cx, 1331        ; divisor for 896 hz tone
000023D6 B31F                   <1>     mov     bl, 31          ; set count for 31/64 second for beep
                                <1>     ;call   beep            ; sound the pod bell
                                <1>     ;jmp    short u5        ; tty_return
                                <1>     ;retn
                                <1>
```

```
                                <1> TIMER       equ     040h            ; 8254 TIMER - BASE ADDRESS
                                <1> PORT_B      equ     061h            ; PORT B READ/WRITE DIAGNOSTIC REGISTER
                                <1> GATE2       equ     00000001b       ; TIMER 2 INPUT CATE CLOCK BIT
                                <1> SPK2 equ    00000010b       ; SPEAKER OUTPUT DATA ENABLE BIT
                                <1>
                                <1> beep:
                                <1>     ; 12/02/2021
                                <1>     ; 07/02/2015
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; 18/01/2014
                                <1>     ; 03/12/2013
                                <1>     ;
                                <1>     ; TEST4.ASM - 06/10/85  POST AND BIOS UTILITY ROUTINES
                                <1>     ;
                                <1>     ; ROUTINE TO SOUND THE BEEPER USING TIMER 2 FOR TONE
                                <1>     ;
                                <1>     ; ENTRY:
                                <1>     ;    (BL) = DURATION COUNTER ( 1 FOR 1/64 SECOND )
                                <1>     ;    (CX) = FREQUENCY DIVISOR (1193180/FREQUENCY) (1331 FOR 886 HZ)
                                <1>     ; EXIT:                 :
                                <1>     ;    (AX),(BL),(CX) MODIFIED.
                                <1>
000023D8 9C                     <1>     pushfd  ; 18/01/2014  ; save interrupt status
000023D9 FA                     <1>     cli                   ; block interrupts during update
000023DA B0B6                   <1>     mov     al, 10110110b ; select timer 2, lsb, msb binary
000023DC E643                   <1>     out     TIMER+3, al   ; write timer mode register
000023DE EB00                   <1>     jmp     $+2           ; I/O delay
000023E0 88C8                   <1>     mov     al, cl        ; divisor for hz (low)
000023E2 E642                   <1>     out     TIMER+2,AL    ; write timer 2 count - lsb
000023E4 EB00                   <1>     jmp     $+2           ; I/O delay
000023E6 88E8                   <1>     mov     al, ch        ; divisor for hz (high)
000023E8 E642                   <1>     out     TIMER+2, al   ; write timer 2 count - msb
000023EA E461                   <1>     in      al, PORT_B    ; get current setting of port
000023EC 88C4                   <1>     mov     ah, al        ; save that setting
000023EE 0C03                   <1>     or      al, GATE2+SPK2 ; gate timer 2 and turn speaker on
000023F0 E661                   <1>     out     PORT_B, al    ; and restore interrupt status
                                <1>     ; 12/02/2021
000023F2 9D                     <1>     popfd   ; 18/01/2014
                                <1>     ;sti
                                <1> g7:                       ; 1/64 second per count (bl)
000023F3 B90B040000             <1>     mov     ecx, 1035     ; delay count for 1/64 of a second
000023F8 E828000000             <1>     call    waitf         ; go to beep delay 1/64 count
000023FD FECB                   <1>     dec     bl            ; (bl) length count expired?
000023FF 75F2                   <1>     jnz     short g7      ; no - continue beeping speaker
                                <1>     ;
00002401 9C                     <1>     pushfd  ; 12/02/2021  ; save interrupt status
00002402 FA                     <1>     cli     ; 18/01/2014  ; block interrupts during update
00002403 E461                   <1>     in      al, PORT_B    ; get current port value
                                <1>         ;or al, not (GATE2+SPK2) ; isolate current speaker bits in case
00002405 0CFC                   <1>         or      al, ~(GATE2+SPK2)
00002407 20C4                   <1>         and ah, al        ; someone turned them off during beep
00002409 88E0                   <1>     mov     al, ah        ; recover value of port
                                <1>         ;or al, not (GATE2+SPK2) ; force speaker data off
0000240B 0CFC                   <1>     or      al, ~(GATE2+SPK2) ; isolate current speaker bits in case
0000240D E661                   <1>     out     PORT_B, al    ; and stop speaker timer
0000240F 9D                     <1>     popfd   ; 12/02/2021          ; restore interrupt flag state
                                <1>     ;sti
                                <1>     ;mov    ecx, 1035     ; force 1/64 second delay (short)
                                <1>     ; 17/04/2021
00002410 66B90B04               <1>     mov     cx, 1035
00002414 E80C000000             <1>     call    waitf         ; minimum delay between all beeps
00002419 9C                     <1>     pushfd                ; save interrupt status
0000241A FA                     <1>     cli                   ; block interrupts during update
0000241B E461                   <1>     in      al, PORT_B    ; get current port value in case
0000241D 2403                   <1>     and     al, GATE2+SPK2 ; someone turned them on
0000241F 08E0                   <1>     or      al, ah        ; recover value of port_b
00002421 E661                   <1>     out     PORT_B, al    ; restore speaker status
00002423 9D                     <1>     popfd                 ; restore interrupt flag state
                                <1> u12:
00002424 C3                     <1>     retn
                                <1>
                                <1> REFRESH_BIT equ    00010000b      ; REFRESH TEST BIT
                                <1>
                                <1> WAITF:
                                <1> waitf:
                                <1>     ; 30/08/2014 (Retro UNIX 386 v1)
                                <1>     ; 03/12/2013
                                <1>     ;
                                <1> ;   push    ax            ; save work register (ah)
                                <1> ;waitf1:
                                <1>                           ; use timer 1 output bits
                                <1> ;   in      al, PORT_B    ; read current counter output status
                                <1> ;   and     al, REFRESH_BIT       ; mask for refresh determine bit
                                <1> ;   cmp     al, ah        ; did it just change
                                <1> ;   je      short waitf1  ; wait for a change in output line
                                <1> ;   ;
                                <1> ;   mov     ah, al        ; save new lflag state
                                <1> ;   loop    waitf1        ; decrement half cycles till count end
                                <1> ;   ;
                                <1> ;   pop     ax            ; restore (ah)
                                <1> ;   retn                  ; return (cx)=0
                                <1>
                                <1> ; 06/02/2015 (unix386.s <-- dsectrm2.s)
                                <1> ; 17/12/2014 (dsectrm2.s)
                                <1> ; WAITF
                                <1> ; /// IBM PC-XT Model 286 System BIOS Source Code - Test 4 - 06/10/85 ///
                                <1> ;
                                <1> ;---WAITF-------------------------------------------------------------------
                                <1> ;   FIXED TIME WAIT ROUTINE (HARDWARE CONTROLLED - NOT PROCESSOR)
                                <1> ; ENTRY:
                                <1> ;   (CX) =  COUNT OF 15.085737 MICROSECOND INTERVALS TO WAIT
                                <1> ;           MEMORY REFRESH TIMER 1 OUTPUT USED AS REFERENCE
                                <1> ; EXIT:
                                <1> ;           AFTER (CX) TIME COUNT (PLUS OR MINUS 16 MICROSECONDS)
                                <1> ;   (CX) = 0
                                <1> ;--------------------------------------------------------------------------
                                <1>
                                <1> ; Refresh period: 30 micro seconds (15-80 us)
                                <1> ; (16/12/2014 - AWARDBIOS 1999 - ATORGS.ASM, WAIT_REFRESH)
                                <1>
                                <1> ;WAITF:                                   ; DELAY FOR (CX)*15.085737 US
                                <1>     ;push   AX                    ; SAVE WORK REGISTER (AH)
                                <1>     ; 11/04/2021
00002425 50                     <1>     push    eax
                                <1>     ; 16/12/2014
                                <1>     ;shr    cx, 1                 ; convert to count of 30 micro seconds
00002426 D1E9                   <1>     shr     ecx, 1  ; 21/02/2015
                                <1> ;17/12/2014
                                <1> ;WAITF1:
                                <1> ;   in      al, PORT_B   ;061h    ; READ CURRENT COUNTER OUTPUT STATUS
                                <1> ;   and     al, REFRESH_BIT       ;00010000b ; MASK FOR REFRESH DETERMINE BIT
                                <1> ;   cmp     al, ah                ; DID IT JUST CHANGE
                                <1> ;   je      short WAITF1          ; WAIT FOR A CHANGE IN OUTPUT LINE
                                <1> ;   mov     ah, al                ; SAVE NEW FLAG STATE
```

```
                                <1> ;   loop    WAITF1                  ; DECREMENT HALF CYCLES TILL COUNT END
                                <1>     ;
                                <1>     ; 17/12/2014
                                <1>     ;
                                <1>     ; Modification from 'WAIT_REFRESH' procedure of AWARD BIOS - 1999
                                <1>     ;
                                <1> ;WAIT_REFRESH:  Uses port 61, bit 4 to have CPU speed independent waiting.
                                <1> ;   INPUT:  CX = number of refresh periods to wait
                                <1> ;                   (refresh periods = 1 per 30 microseconds on most machines)
                                <1> WR_STATE_0:
00002428 E461                   <1>     in      al, PORT_B              ; IN AL,SYS1
0000242A A810                   <1>     test    al, 010h
0000242C 74FA                   <1>     JZ      short WR_STATE_0
                                <1> WR_STATE_1:
0000242E E461                   <1>     in      al, PORT_B              ; IN AL,SYS1
00002430 A810                   <1>     test    al, 010h
00002432 75FA                   <1>     jnz     short WR_STATE_1
00002434 E2F2                   <1>         loop    WR_STATE_0
                                <1>     ;
                                <1>     ;pop    ax                      ; RESTORE (AH)
                                <1>     ; 11/04/2021
00002436 58                     <1>     pop     eax
00002437 C3                     <1>     retn                            ; (CX) = 0
                                <1>
                                <1> ; 03/08/2022
                                <1> ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 09/07/2016
                                <1> ; 01/07/2016
                                <1> ; 24/06/2016
                                <1> ; 23/06/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;------------------------------------------------------------------------------
                                <1> ; WRITE_STRING                                                                 :
                                <1> ;   THIS ROUTINE WRITES A STRING OF CHARACTERS TO THE CRT.                     :
                                <1> ; INPUT                                                                  :
                                <1> ;   (AL) = WRITE STRING COMMAND  0 - 3                                   :
                                <1> ;   (BH) = DISPLAY PAGE (ACTIVE PAGE)                                    :
                                <1> ;   (CX) = COUNT OF CHARACTERS TO WRITE, IF (CX) = 0 THEN RETURN               :
                                <1> ;   (DX) = CURSOR POSITION FOR START OF STRING WRITE                     :
                                <1> ;   (BL) = ATTRIBUTE OF CHARACTER TO WRITE IF (AL) = 0  OR     (AL) = 1       :
                                <1> ;   (EBP) = SOURCE STRING OFFSET                                         :
                                <1> ; OUTPUT                                                                 :
                                <1> ;   NONE                                                                 :
                                <1> ;------------------------------------------------------------------------------
                                <1>
                                <1> ; AL = 00h: Assign all characters the attribute in BL; do not update cursor
                                <1> ; AL = 01h: Assign all characters the attribute in BL; update cursor
                                <1> ; AL = 02h: Use attributes in string; do not update cursor
                                <1> ; AL = 03h: Use attributes in string; update cursor
                                <1>
                                <1> WRITE_STRING:
                                <1>     ; 03/08/2022
                                <1>     ; 02/08/2022
                                <1>     ; 12/09/2016
                                <1>     ; 09/07/2016
                                <1>     ;cmp    byte [CRT_MODE], 7 ; 6?!
                                <1>     ;ja     VIDEO_RETURN            ; not a valid function for VGA modes
                                <1>     ;
00002438 A2[48830100]           <1>     mov     [w_str_cmd], al                 ; save (AL) command
0000243D 3C04                   <1>     cmp     al, 4                   ; TEST FOR INVALID WRITE STRING OPTION
                                <1>     ;jnb    VIDEO_RETURN            ; IF OPTION INVALID THEN RETURN
                                <1>     ; 02/08/2022
0000243F 7361                   <1>     jnb     short P55
                                <1>
                                <1>         ;jcxz      VIDEO_RETURN            ; IF ZERO LENGTH STRING THEN RETURN
                                <1>
00002441 67E35E                 <1>         jcxz    P55                     ; 01/07/2016
                                <1>
                                <1>     ; 01/07/2016
                                <1>     ;and    ecx, 0FFFFh
                                <1>     ; ecx = byte count
                                <1>     ;push   ecx
00002444 89EE                   <1>     mov     esi, ebp ; user buffer
00002446 BF00000700             <1>     mov     edi, Cluster_Buffer  ; system buffer
0000244B E815E70000             <1>     call    transfer_from_user_buffer
                                <1>     ;pop    ecx
                                <1>     ;jc     VIDEO_RETURN
                                <1>     ; 02/08/2022
00002450 7250                   <1>     jc      short P55
                                <1>     ; ecx = transfer (byte) count = character count
00002452 BD00000700             <1>     mov     ebp, Cluster_Buffer
                                <1>     ; 12/09/2016
00002457 803D[2E680000]07       <1>     cmp     byte [CRT_MODE], 7 ; 6?!
                                <1>     ;ja     vga_write_string
                                <1>     ; 02/08/2022
0000245E 7605                   <1>     jna     short P57
00002460 E99C000000             <1>     jmp     vga_write_string
                                <1> P57:
00002465 0FB6F7                 <1>     movzx   esi, bh                 ; GET CURRENT CURSOR PAGE
                                <1>     ;sal    si, 1                   ; CONVERT TO PAGE OFFSET (SI = PAGE)
                                <1>     ; 02/08/2022
00002468 D1E6                   <1>     sal     esi, 1
                                <1>     ; *****
0000246A 66FFB6[CE760100]       <1>     push    word [esi+CURSOR_POSN]; SAVE CURRENT CURSOR POSITION IN STACK
                                <1>
                                <1>     ;mov    ax, 0200h               ; SET NEW CURSOR POSITION
                                <1>     ;int    10h
                                <1> P50next:
00002471 51                     <1>     push    ecx ; ****
00002472 53                     <1>     push    ebx ; *** ; 18/11/2020
00002473 56                     <1>     push    esi ; **
00002474 52                     <1>     push    edx ; *
00002475 E8FDFEFFFF             <1>     call    _set_cpos
                                <1> P50:
0000247A 8A4500                 <1>     mov     al, [ebp]               ; GET CHARACTER FROM INPUT STRING
0000247D 45                     <1>     inc     ebp                     ; BUMP POINTER TO CHARACTER
                                <1>
                                <1> ;-----      TEST FOR SPECIAL CHARACTER'S
                                <1>
0000247E 3C08                   <1>     cmp     al, 08h                 ; IS IT A BACKSPACE
00002480 7410                   <1>     je      short P51               ; BACK_SPACE
00002482 3C0D                   <1>     cmp     al, 0Dh ; CR            ; IS IT CARRIAGE RETURN
00002484 740C                   <1>     je      short P51               ; CAR_RET
00002486 3C0A                   <1>     cmp     al, 0Ah ; LF            ; IS IT A LINE FEED
00002488 7408                   <1>     je      short P51               ; LINE_FEED
                                <1>     ; 18/11/2020
0000248A 3C09                   <1>     cmp     al, 09h                 ; is it a tab stop
0000248C 7404                   <1>     je      short P51
                                <1>     ;
0000248E 3C07                   <1>     cmp     al, 07h                 ; IS IT A BELL
00002490 7515                   <1>     jne     short P52               ; IF NOT THEN DO WRITE CHARACTER
                                <1> P51:
                                <1>     ;mov    ah, 0Eh                 ; TTY_CHARACTER_WRITE
                                <1>     ;int    10h                     ; WRITE TTY CHARACTER TO THE CRT
```

```
                                <1>
00002492 E85BFEFFFF             <1>     call    _write_tty_m3
                                <1>
00002497 5A                     <1>     pop     edx ; *
00002498 5E                     <1>     pop     esi ; **
                                <1>
00002499 668B96[CE760100]       <1>     mov     dx, [esi+CURSOR_POSN] ; GET CURRENT CURSOR POSITION
000024A0 EB44                   <1>     jmp     short P54             ; SET CURSOR POSITION AND CONTINUE
                                <1> P55:
000024A2 E9C4F6FFFF             <1>     jmp     VIDEO_RETURN
                                <1> P52:
                                <1>     ;mov    cx, 1                 ; SET CHARACTER WRITE AMOUNT TO ONE
                                <1>     ; 02/08/2022
000024A7 29C9                   <1>     sub     ecx, ecx
000024A9 FEC1                   <1>     inc     cl
                                <1>     ; ecx = 1
000024AB 803D[48830100]02       <1>     cmp     byte [w_str_cmd], 2   ; IS THE ATTRIBUTE IN THE STRING
000024B2 7204                   <1>     jb      short P53             ; IF NOT THEN SKIP
000024B4 8A5D00                 <1>     mov     bl, [ebp]             ; ELSE GET NEW ATTRIBUTE
000024B7 45                     <1>     inc     ebp                   ; BUMP STRING POINTER
                                <1> P53:
                                <1>     ;mov    ah, 09h               ; GOT_CHARACTER
                                <1>     ;int    10h                   ; WRITE CHARACTER TO THE CRT
                                <1>
000024B8 E8A1FDFFFF             <1>     call    _write_c_current
                                <1>
000024BD 5A                     <1>     pop     edx ; *
                                <1>
                                <1>     ; 05/12/2020
                                <1>     ; bx is preserved in '_write_c_current'
                                <1>     ; 18/11/2020
                                <1>     ;mov    ebx, [esp+4]   ; ***
                                <1>
000024BE 0FB6F7                 <1>     movzx   esi, bh ; video page number (0 to 7)
000024C1 889E[37680000]         <1>     mov     [esi+chr_attrib], bl ; color/attribute
                                <1>
000024C7 FEC2                   <1>     inc     dl                    ; INCREMENT COLUMN COUNTER
000024C9 3A15[30680000]         <1>     cmp     dl, [CRT_COLS]        ; IF COLS ARE WITHIN RANGE FOR THIS MODE
                                <1>     ;jb     short P54             ;    THEN GO TO COLUMNS SET
000024CF 7214                   <1>     jb      short P56 ; 05/12/2020
000024D1 FEC6                   <1>     inc     dh                    ; BUMP ROW COUNTER BY ONE
000024D3 28D2                   <1>     sub     dl, dl                ; SET COLUMN COUNTER TO ZERO
000024D5 80FE19                 <1>     cmp     dh, 25                ; IF ROWS ARE LESS THAN 25 THEN
                                <1>     ;jb     short P54             ; GO TO ROWS_COLUMNS_SET
000024D8 720B                   <1>     jb      short P56 ; 05/12/2020
                                <1>
                                <1>     ; 18/11/2020
                                <1>     ;mov    ax, 0E0Ah             ; ELSE SCROLL SCREEN
                                <1>     ;int    10h                   ; RESET ROW COUNTER TO 24
                                <1>
                                <1>     ; 18/11/2020
000024DA B00A                   <1>     mov     al, 0Ah ; line feed
                                <1>
000024DC E811FEFFFF             <1>     call    _write_tty_m3
                                <1>
000024E1 66BA0018               <1>     mov     dx, 1800h             ; Column = 0, Row = 24
                                <1> P56:
                                <1>     ; 05/12/2020
                                <1>     ; 18/11/2020
000024E5 5E                     <1>     pop     esi ; **
                                <1> P54:                              ; ROW_COLUMNS_SET
                                <1>     ;mov    ax, 0200h             ; SET NEW CURSOR POSITION COMMAND
                                <1>     ;int    10h                   ; ESTABLISH NEW CURSOR POSITION
                                <1>
                                <1>     ; 18/11/2020
000024E6 5B                     <1>     pop     ebx ; ***
000024E7 59                     <1>     pop     ecx ; ****
                                <1>
                                <1>     ;loop   P50                   ; DO IT ONCE MORE UNTIL (CX) = ZERO
000024E8 6649                   <1>     dec     cx
000024EA 7585                   <1>     jnz     short P50next
                                <1>
000024EC 665A                   <1>     pop     dx  ; *****           ; RESTORE OLD CURSOR COORDINATES
                                <1>
000024EE F605[48830100]01       <1>     test    byte [w_str_cmd], 1   ; IF CURSOR WAS NOT TO BE MOVED
                                <1>     ;jnz    VIDEO_RETURN          ; THEN EXIT WITHOUT RESETTING OLD VALUE
                                <1>     ; 03/08/2022
000024F5 7505                   <1>     jnz     short P58
                                <1>
                                <1>     ;mov    ax, 0200h             ; ELSE RESTORE OLD CURSOR POSITION
                                <1>     ;int    10h
                                <1>                                   ; DONE - EXIT WRITE STRING
000024F7 E87BFEFFFF             <1>     call    _set_cpos
                                <1> P58:
000024FC E96AF6FFFF             <1>     jmp     VIDEO_RETURN          ; RETURN TO CALLER
                                <1>
                                <1> vga_write_string:
                                <1>     ; 04/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 12/09/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', ' biosfn_write_string'
                                <1>
                                <1>     ; INPUT                                                         :
                                <1>     ;       (AL) = WRITE STRING COMMAND  0 - 3                      :
                                <1>     ;       (BH) = DISPLAY PAGE (ACTIVE PAGE)                       :
                                <1>     ;       (CX) = COUNT OF CHARACTERS TO WRITE, IF (CX) = 0 THEN RETURN        :
                                <1>     ;       (DX) = CURSOR POSITION FOR START OF STRING WRITE        :
                                <1>     ;       (BL) = ATTRIBUTE OF CHARACTER TO WRITE IF (AL) = 0  OR     (AL) = 1  :
                                <1>     ;       (EBP) = SOURCE STRING OFFSET                            :
                                <1>     ; OUTPUT                                                        :
                                <1>     ;       NONE                                                    :
                                <1>     ;-----------------------------------------------------------------------;
                                <1>
                                <1>     ; AL = 00h: Assign all characters the attribute in BL; do not update cursor
                                <1>     ; AL = 01h: Assign all characters the attribute in BL; update cursor
                                <1>     ; AL = 02h: Use attributes in string; do not update cursor
                                <1>     ; AL = 03h: Use attributes in string; update cursor
                                <1>
                                <1>     ; biosfn_write_string(GET_AL(),GET_BH(),GET_BL(),CX,GET_DH(),GET_DL(),ES,BP);
                                <1>     ; static void biosfn_write_string (flag,page,attr,count,row,col,seg,offset)
                                <1>
                                <1>     ; // Read curs info for the page
                                <1>     ; biosfn_get_cursor_pos(page,&dummy,&oldcurs);
                                <1>     ; bh = video page = 0
                                <1>     ;movzx  esi, word [CURSOR_POSN] ; current cursor position for video page 0
                                <1>
                                <1>     ; // if row=0xff special case : use current cursor position
                                <1>     ; if(row==0xff)
                                <1>     ;  {col=oldcurs&0x00ff;
                                <1>     ;   row=(oldcurs&0xff00)>>8;
                                <1>     ;  }
```

```
                                <1>
                                <1>     ;mov    al, [w_str_cmd]
                                <1>
00002501 80FEFF                 <1>     cmp     dh, 0FFh
00002504 7407                   <1>     je      short vga_wstr_1 ; user current cursor position
                                <1> vga_wstr_0:
                                <1>     ; set cursor position
00002506 668915[CE760100]       <1>     mov     [CURSOR_POSN], dx ; save cursor pos for pg 0
                                <1> vga_wstr_1:
0000250D 66FF35[CE760100]       <1>     push    word [CURSOR_POSN] ; *
                                <1>
                                <1>     ; ebp = string offset in system buffer (user buffer was copied to)
                                <1>
                                <1>     ; while(count--!=0)
                                <1>     ;  {
                                <1>     ;   car=read_byte(seg,offset++);
                                <1>     ;   if((flag&0x02)!=0)
                                <1>     ;    attr=read_byte(seg,offset++);
                                <1>     ;    biosfn_write_teletype(car,page,attr,WITH_ATTR);
                                <1>     ;  }
                                <1>
                                <1>     ;push   eax ; **
                                <1>     ;test   al, 2
00002514 F605[48830100]02       <1>     test    byte [w_str_cmd], 2
0000251B 751D                   <1>     jnz     short vga_wstr_3
0000251D 881D[DF760100]         <1>     mov     [ccolor], bl
                                <1> vga_wstr_2:
00002523 51                     <1>     push    ecx
00002524 8A4500                 <1>     mov     al, [ebp]
00002527 E88F0A0000             <1>     call    vga_write_teletype
0000252C 59                     <1>     pop     ecx
0000252D 6649                   <1>     dec     cx
0000252F 741E                   <1>     jz      short vga_wstr_4
00002531 45                     <1>     inc     ebp
00002532 8A1D[DF760100]         <1>     mov     bl, [ccolor]
00002538 EBE9                   <1>     jmp     short vga_wstr_2
                                <1> vga_wstr_3:
0000253A 51                     <1>     push    ecx
0000253B 8A4500                 <1>     mov     al, [ebp]
0000253E 45                     <1>     inc     ebp
0000253F 8A5D00                 <1>     mov     bl, [ebp]
00002542 E8740A0000             <1>     call    vga_write_teletype
00002547 59                     <1>     pop     ecx
00002548 6649                   <1>     dec     cx
0000254A 7403                   <1>     jz      short vga_wstr_4
0000254C 45                     <1>     inc     ebp
0000254D EBEB                   <1>     jmp     short vga_wstr_3
                                <1> vga_wstr_4:
                                <1>     ; // Set back curs pos
                                <1>     ; if((flag&0x01)==0)
                                <1>     ;  biosfn_set_cursor_pos(page,oldcurs);
                                <1>     ; }
                                <1>     ;pop    eax ; **
0000254F 665A                   <1>     pop     dx ; word [CURSOR_POSN] ; *
                                <1>     ;test   al, 1
00002551 F605[48830100]01       <1>     test    byte [w_str_cmd], 1
                                <1>     ;jnz    VIDEO_RETURN
                                <1>     ; 04/08/2022
00002558 7507                   <1>     jnz     short vga_wstr_5
0000255A 668915[CE760100]       <1>     mov     [CURSOR_POSN], dx
                                <1> vga_wstr_5:
00002561 E905F6FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 07/07/2016
                                <1> ; 27/06/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;-------------------------------------------------------
                                <1> ;  SCROLL UP
                                <1> ;   THIS ROUTINE SCROLLS UP THE INFORMATION ON THE CRT
                                <1> ; ENTRY ---
                                <1> ;  CH,CL = UPPER LEFT CORNER OF REGION TO SCROLL
                                <1> ;  DH,DL = LOWER RIGHT CORNER OF REGION TO SCROLL
                                <1> ;   BOTH OF THE ABOVE ARE IN CHARACTER POSITIONS
                                <1> ;  BH = FILL VALUE FOR BLANKED LINES
                                <1> ;  AL = # LINES TO SCROLL (AL=0 MEANS BLANK THE ENTIRE FIELD)
                                <1> ;  DS = DATA SEGMENT
                                <1> ;  ES = REGEN SEGMENT
                                <1> ; EXIT --
                                <1> ;  NOTHING, THE SCREEN IS SCROLLED
                                <1> ;-------------------------------------------------------
                                <1>
                                <1>     ; cl = upper left column
                                <1>     ; ch = upper left row
                                <1>     ; dl = lower rigth column
                                <1>     ; dh = lower right row
                                <1>     ;
                                <1>     ; al = line count (AL=0 means blank entire fields)
                                <1>     ; bl = fill value for blanked lines
                                <1>     ; bh = unused
                                <1>
                                <1> GRAPHICS_UP:
                                <1>     ; 07/07/2016
                                <1>     ; AH = Current video mode, [CRT_MODE]
00002566 80FC07                 <1>     cmp     ah, 7
00002569 7762                   <1>     ja      short vga_graphics_up
                                <1>     ;je     n0
                                <1>
0000256B 88C7                   <1>     mov     bh, al                  ; save line count in BH
                                <1>     ;mov    ax, cx                  ; GET UPPER LEFT POSITION INTO AX REG
                                <1>     ; 03/08/2022
0000256D 89C8                   <1>     mov     eax, ecx
                                <1>
                                <1> ;-----      USE CHARACTER SUBROUTINE FOR POSITIONING
                                <1> ;-----      ADDRESS RETURNED IS MULTIPLIED BY 2 FROM CORRECT VALUE
                                <1>
0000256F E8BA050000             <1>     call    GRAPH_POSN
                                <1>     ;movzx  edi, ax                 ; SAVE RESULT AS DESTINATION ADDRESS
                                <1>     ; 03/08/2022
00002574 89C7                   <1>     mov     edi, eax
                                <1>
                                <1> ;-----      DETERMINE SIZE OF WINDOW
                                <1>
00002576 6629CA                 <1>     sub     dx, cx
00002579 6681C20101             <1>         add     dx, 101h                ; ADJUST VALUES
0000257E C0E602                 <1>     sal     dh, 2                   ; MULTIPLY ROWS BY 4 AT 8 VERT DOTS/CHAR
                                <1>                                     ; AND EVEN/ODD ROWS
                                <1> ;-----      DETERMINE CRT MODE
                                <1>
00002581 803D[2E680000]06       <1>     cmp     byte [CRT_MODE], 6      ; TEST FOR MEDIUM RES
00002588 7304                   <1>         jnc short _R7_                ; FIND_SOURCE
                                <1>
                                <1> ;-----      MEDIUM RES UP
0000258A D0E2                   <1>     sal     dl, 1                   ; # COLUMNS * 2, SINCE 2 BYTES/CHAR
```

```
                            <1>     ;sal    di, 1                   ; OFFSET *2 SINCE 2 BYTES/CHAR
                            <1>     ; 03/08/2022
0000258C D1E7               <1>     sal     edi, 1
                            <1>
                            <1> ;-----      DETERMINE THE SOURCE ADDRESS IN THE BUFFER
                            <1> _R7_:                                       ; FIND_SOURCE
0000258E 81C700800B00       <1>     add     edi, 0B8000h
00002594 C0E702             <1>     sal     bh, 2                   ; multiply number of lines by 4
00002597 7430               <1>         jz  short _R11                ; IF ZERO, THEN BLANK ENTIRE FIELD
00002599 B050               <1>     mov     al, 80                  ; 80 BYTES/ROW
0000259B F6E7               <1>     mul     bh                      ; determine offset to source
                            <1>     ;movzx  esi, ax                 ; offset to source
                            <1>     ; 03/08/2022
0000259D 89C6               <1>     mov     esi, eax
0000259F 01FE               <1>     add     esi, edi                ; SET UP SOURCE
000025A1 88F4               <1>     mov     ah, dh                  ; NUMBER OF ROWS IN FIELD
000025A3 28FC               <1>     sub     ah, bh                  ; determine number to move
                            <1>
                            <1> ;-----      LOOP THROUGH, MOVING ONE ROW AT A TIME, BOTH EVEN AND ODD FIELDS
                            <1> _R8:                                        ; ROW_LOOP
000025A5 E8FD030000         <1>         call    _R17                    ; MOVE ONE ROW
000025AA 6681EEB01F         <1>     sub     si, 2000h-80            ; MOVE TO NEXT ROW
000025AF 6681EFB01F         <1>     sub     di, 2000h-80
000025B4 FECC               <1>     dec     ah                      ; NUMBER OF ROWS TO MOVE
000025B6 75ED               <1>         jnz     short _R8               ; CONTINUE TILL ALL MOVED
                            <1>
                            <1> ;-----      FILL IN THE VACATED LINE(S)
                            <1> _R9:                                        ; CLEAR ENTRY
000025B8 88D8               <1>     mov     al, bl                  ; attribute to fill with
                            <1> _R10_:
000025BA E804040000         <1>         call    _R18                    ; CLEAR THAT ROW
000025BF 6681EFB01F         <1>     sub     di, 2000h-80            ; POINT TO NEXT LINE
000025C4 FECF               <1>     dec     bh                      ; number of lines to fill
000025C6 75F2               <1>     jnz     short _R10_               ; CLEAR LOOP
000025C8 C3                 <1>     retn                            ; EVERYYHING DONE
                            <1>
                            <1> _R11:                                       ; BLANK_FIELD
000025C9 88F7               <1>     mov     bh, dh                  ; set blank count to everything in field
000025CB EBEB               <1>         jmp     short _R9               ; CLEAR THE FIELD
                            <1>
                            <1> vga_graphics_up:
                            <1>     ; 03/08/2022 - TRDOS 386 Kertnel v2.0.5
                            <1>     ; 12/04/2021
                            <1>     ; 08/08/2016
                            <1>     ; 07/08/2016
                            <1>     ; 04/08/2016
                            <1>     ; 01/08/2016
                            <1>     ; 31/07/2016
                            <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                            <1>     ;
                            <1>     ; derived from 'Plex86/Bochs VGABios' source code
                            <1>     ; vgabios-0.7a (2011)
                            <1>     ; by the LGPL VGABios developers Team (2001-2008)
                            <1>     ; 'vgabios.c', 'biosfn_scroll'
                            <1>     ;
                            <1>
                            <1>     ; cl = upper left column
                            <1>     ; ch = upper left row
                            <1>     ; dl = lower rigth column
                            <1>     ; dh = lower right row
                            <1>     ;
                            <1>     ; al = line count (AL=0 means blank entire fields)
                            <1>     ; bl = fill value for blanked lines
                            <1>     ; bh = unused
                            <1>     ;
                            <1>     ; ah = [CRT_MODE], current video mode
                            <1>
000025CD 88C7               <1>     mov     bh, al ; 31/07/2016
000025CF BE[52680000]       <1>     mov     esi, vga_g_modes
000025D4 89F7               <1>     mov     edi, esi
000025D6 83C708             <1>     add     edi, vga_g_mode_count
                            <1> vga_g_up_0:
000025D9 AC                 <1>     lodsb
000025DA 38E0               <1>     cmp     al, ah ; [CRT_MODE]
000025DC 7405               <1>     je      short vga_g_up_1
000025DE 39FE               <1>     cmp     esi, edi
000025E0 72F7               <1>     jb      short vga_g_up_0
                            <1>     ;xor    bh, bh ; 31/07/2016)
000025E2 C3                 <1>     retn    ; nothing to do
                            <1> vga_g_up_1:
000025E3 88F8               <1>     mov     al, bh ; 31/07/2016
000025E5 83C64F             <1>     add     esi, vga_g_memmodel - (vga_g_modes + 1)
                            <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
                            <1>
                            <1>     ; if(rlr>=nbrows)rlr=nbrows-1;
                            <1>     ; if(clr>=nbcols)clr=nbcols-1;
                            <1>     ; if(nblines>nbrows)nblines=0;
                            <1>     ; cols=clr-cul+1;
                            <1>
000025E8 3A35[36680000]     <1>     cmp     dh, [VGA_ROWS]
000025EE 7208               <1>     jb      short vga_g_up_2
000025F0 8A35[36680000]     <1>     mov     dh, [VGA_ROWS]
000025F6 FECE               <1>     dec     dh
                            <1> vga_g_up_2:
000025F8 3A15[30680000]     <1>     cmp     dl, [CRT_COLS]  ; = [VGA_COLS]
000025FE 7208               <1>     jb      short vga_g_up_3
00002600 8A15[30680000]     <1>     mov     dl, [CRT_COLS]
00002606 FECA               <1>     dec     dl
                            <1> vga_g_up_3:
00002608 3A05[36680000]     <1>     cmp     al, [VGA_ROWS]
0000260E 7602               <1>     jna     short vga_g_up_4
00002610 28C0               <1>     sub     al, al ; 0
                            <1> vga_g_up_4:
00002612 88D7               <1>     mov     bh, dl ; clr
00002614 28CF               <1>     sub     bh, cl ; cul
00002616 FEC7               <1>     inc     bh ; cols = clr-cul+1
                            <1>
00002618 20C0               <1>     and     al, al ; nblines = 0
0000261A 7559               <1>     jnz     short vga_g_up_6
0000261C 20ED               <1>     and     ch, ch ; rul = 0
0000261E 7555               <1>     jnz     short vga_g_up_6
00002620 20C9               <1>     and     cl, cl ; cul = 0
00002622 7551               <1>     jnz     short vga_g_up_6
                            <1>
                            <1>     ;push   ax
                            <1>     ; 12/04/2021
00002624 50                 <1>     push    eax
00002625 A0[36680000]       <1>     mov     al, [VGA_ROWS]
0000262A FEC8               <1>     dec     al
0000262C 38C6               <1>     cmp     dh, al ; rlr = nbrows-1
0000262E 7545               <1>     jne     short vga_g_up_5
00002630 A0[30680000]       <1>         mov     al, [CRT_COLS]  ; = VGA_COLS
00002635 FEC8               <1>     dec     al
00002637 38C2               <1>     cmp     dl, al ; clr = nbcols-1
```

```
                                <1>     ;jne    short vga_g_up_5
                                <1>     ;;pop   ax
                                <1>     ; 12/04/2021
00002639 58                     <1>     pop     eax
0000263A 7539                   <1>     jne     short vga_g_up_5
                                <1>
0000263C 66B80502               <1>     mov     ax, 0205h
00002640 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002644 66EF                   <1>     out     dx, ax
00002646 A0[36680000]           <1>     mov     al, [VGA_ROWS]
0000264B 8A25[30680000]         <1>     mov     ah, [CRT_COLS] ; = [VGA_COLS]
00002651 F6E4                   <1>     mul     ah
00002653 0FB7D0                 <1>     movzx   edx, ax
                                <1>     ; 08/08/2016
00002656 0FB605[32680000]       <1>     movzx   eax, byte [CHAR_HEIGHT]
0000265D F7E2                   <1>     mul     edx
                                <1>     ; eax = byte count
0000265F 89C1                   <1>     mov     ecx, eax
                                <1>     ;; 07/08/2016
                                <1>     ;shl    dx, 3 ; * 8 ; * [CHAR_HEIGHT]
                                <1>     ;mov    ecx, edx
00002661 88D8                   <1>     mov     al, bl ; fill value for blanked lines
00002663 BF00000A00             <1>     mov     edi, 0A0000h
00002668 F3AA                   <1>     rep     stosb
                                <1>
                                <1>     ;mov    ax, 5
                                <1>     ; 03/08/2022
0000266A 30E4                   <1>     xor     ah, ah
0000266C B005                   <1>     mov     al, 5
                                <1>
0000266E 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002672 66EF                   <1>     out     dx, ax ; 0005h
                                <1>
00002674 C3                     <1>     retn
                                <1>
                                <1> vga_g_up_5:
                                <1>     ;;pop   ax
                                <1>     ; 12/04/2021
                                <1>     ;pop    eax
                                <1>
                                <1> vga_g_up_6:
                                <1>     ; [ESI] = VGA memory model number for current video mode
                                <1>         ;
                                <1>         ; LINEAR8 equ 5
                                <1>         ; PLANAR4 equ 4
                                <1>         ; PLANAR1 equ 3
                                <1>
00002675 803E04                 <1>     cmp     byte [esi], PLANAR4
00002678 7424                   <1>     je      short vga_g_up_planar
0000267A 803E03                 <1>     cmp     byte [esi], PLANAR1
0000267D 741F                   <1>     je      short vga_g_up_planar
                                <1> vga_g_up_linear8:
                                <1>     ; 07/07/2016 (TEMPORARY)
                                <1>     ;
                                <1>     ; cl = upper left column ; cul
                                <1>     ; ch = upper left row ; rul
                                <1>     ; dl = lower rigth column ; clr
                                <1>     ; dh = lower right row ; rlr
                                <1>
                                <1> vga_g_up_l0:
                                <1>     ;{for(i=rul;i<=rlr;i++)
                                <1>     ; if((i+nblines>rlr)||(nblines==0))
0000267F 08C0                   <1>     or      al, al
00002681 7414                   <1>     jz      short vga_g_up_l2
00002683 88C4                   <1>     mov     ah, al
00002685 00EC                   <1>     add     ah, ch ; i+nblines
                                <1>     ;jc     short vga_g_up_l2
00002687 38F4                   <1>     cmp     ah, dh
00002689 770C                   <1>     ja      short vga_g_up_l2
                                <1>     ; else
                                <1>     ;  vgamem_copy_pl4(cul,i+nblines,i,cols,nbcols,cheight);
0000268B E8F2000000             <1>     call    vgamem_copy_l8
                                <1> vga_g_up_l1:
00002690 FEC5                   <1>     inc     ch
00002692 38F5                   <1>     cmp     ch, dh
00002694 76E9                   <1>     jna     short vga_g_up_l0
00002696 C3                     <1>     retn
                                <1> vga_g_up_l2:
                                <1>     ; vgamem_fill_pl4(cul,i,cols,nbcols,cheight,attr);
00002697 E84C010000             <1>     call    vgamem_fill_l8
0000269C EBF2                   <1>     jmp     short vga_g_up_l1
                                <1>
                                <1> vga_g_up_planar:
                                <1>     ; cl = upper left column ; cul
                                <1>     ; ch = upper left row ; rul
                                <1>     ; dl = lower rigth column ; clr
                                <1>     ; dh = lower right row ; rlr
                                <1> vga_g_up_pl0:
                                <1>     ;{for(i=rul;i<=rlr;i++)
                                <1>     ; if((i+nblines>rlr)||(nblines==0))
0000269E 20C0                   <1>     and     al, al
000026A0 7414                   <1>     jz      short vga_g_up_pl2
000026A2 88C4                   <1>     mov     ah, al
000026A4 00EC                   <1>     add     ah, ch ; i+nblines
                                <1>     ;jc     short vga_g_up_pl2
000026A6 38F4                   <1>     cmp     ah, dh
000026A8 770C                   <1>     ja      short vga_g_up_pl2
                                <1>     ; else
                                <1>     ;  vgamem_copy_pl4(cul,i+nblines,i,cols,nbcols,cheight);
000026AA E80E000000             <1>     call    vgamem_copy_pl4
                                <1> vga_g_up_pl1:
000026AF FEC5                   <1>     inc     ch
000026B1 38F5                   <1>     cmp     ch, dh
000026B3 76E9                   <1>     jna     short vga_g_up_pl0
000026B5 C3                     <1>     retn
                                <1> vga_g_up_pl2:
                                <1>     ; vgamem_fill_pl4(cul,i,cols,nbcols,cheight,attr);
000026B6 E870000000             <1>     call    vgamem_fill_pl4
000026BB EBF2                   <1>     jmp     short vga_g_up_pl1
                                <1>
                                <1> vgamem_copy_pl4:
                                <1>     ; 08/08/2016
                                <1>     ; 07/08/2016
                                <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgamem_copy_pl4'
                                <1>     ;
                                <1>     ; vgamem_copy_pl4(xstart,ysrc,ydest,cols,nbcols,cheight)
                                <1>     ; cl = xstart, ah = ysrc (i+nblines), ch = ydest (i),
                                <1>     ; bh = cols, [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight
                                <1>
```

```
                                <1>     ; src=ysrc*cheight*nbcols+xstart;
                                <1>     ; dest=ydest*cheight*nbcols+xstart;
                                <1>
000026BD 52                     <1>     push    edx
000026BE 50                     <1>     push    eax
                                <1>
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0105)
000026BF 66B80501               <1>     mov     ax, 0105h
000026C3 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
000026C7 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ; 07/08/2016
                                <1>     ;mov     ah, [esp+1]
                                <1>     ;movzx  edx, ah ; ysrc
000026C9 0FB6542401             <1>     movzx   edx, byte [esp+1]
                                <1>     ; 08/08/2016
000026CE 0FB605[32680000]       <1>     movzx   eax, byte [CHAR_HEIGHT]
000026D5 8A25[30680000]         <1>     mov     ah, [CRT_COLS] ; nbcols
000026DB F6E4                   <1>     mul     ah
                                <1>     ;; 07/08/2016
                                <1>     ;movzx  eax, byte [CRT_COLS]
                                <1>     ;shl    ax, 3 ; * 8 ; * [CHAR_HEIGHT]
000026DD 50                     <1>     push    eax ; cheight * nbcols
000026DE F7E2                   <1>     mul     edx ; * ysrc
                                <1>     ; eax = ysrc * cheight * nbcols
                                <1>     ; edx = 0
000026E0 88CA                   <1>     mov     dl, cl ; edx = xstart
000026E2 01D0                   <1>     add     eax, edx
000026E4 89C6                   <1>     mov     esi, eax ; src
000026E6 88EA                   <1>     mov     dl, ch ; ydest
000026E8 58                     <1>     pop     eax ; cheight * nbcols
000026E9 F7E2                   <1>     mul     edx
                                <1>     ; eax = ydest * cheight * nbcols
000026EB 88CA                   <1>     mov     dl, cl ; edx = xstart
000026ED 01D0                   <1>     add     eax, edx
000026EF 89C7                   <1>     mov     edi, eax ; dest
                                <1>     ; esi = src
                                <1>     ; edi = dest
                                <1>     ; for(i=0;i<cheight;i++)
                                <1>     ; {
                                <1>     ;  memcpyb(0xa000,dest+i*nbcols,0xa000,src+i*nbcols,cols);
                                <1>     ; }
000026F1 51                     <1>     push    ecx
000026F2 B900000A00             <1>     mov     ecx, 0A0000h
000026F7 01CE                   <1>     add     esi, ecx
000026F9 01CF                   <1>     add     edi, ecx
                                <1>     ; 08/08/2016
000026FB 8A35[32680000]         <1>     mov     dh, [CHAR_HEIGHT]
                                <1>     ;; 07/08/2016
                                <1>     ;mov    dh, 8 ; 07/08/2016
00002701 28D2                   <1>     sub     dl, dl ; i
                                <1> vgamem_copy_pl4_0:
00002703 56                     <1>     push    esi
00002704 57                     <1>     push    edi
00002705 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS]
0000270C F6E2                   <1>     mul     dl
                                <1>     ; eax = i * nbcols
0000270E 01C7                   <1>     add     edi, eax ; dest+i*nbcols
00002710 01C6                   <1>     add     esi, eax
00002712 0FB6CF                 <1>     movzx   ecx, bh ; cols
00002715 F3A4                   <1>     rep     movsb
00002717 5F                     <1>     pop     edi
00002718 5E                     <1>     pop     esi
00002719 FECE                   <1>     dec     dh
0000271B 75E6                   <1>     jnz     short vgamem_copy_pl4_0
                                <1> vgamem_copy_pl4_1:
0000271D 59                     <1>     pop     ecx
                                <1>
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0005);
0000271E 66B80500               <1>     mov     ax, 0005h
00002722 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002726 66EF                   <1>     out     dx, ax
                                <1>
00002728 58                     <1>     pop     eax
00002729 5A                     <1>     pop     edx
                                <1>
0000272A C3                     <1>     retn
                                <1>
                                <1> vgamem_fill_pl4:
                                <1>     ; 08/08/2016
                                <1>     ; 07/08/2016
                                <1>     ; 04/08/2016
                                <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgamem_fill_pl4'
                                <1>     ;
                                <1>     ; vgamem_fill_pl4(xstart,ystart,cols,nbcols,cheight,attr)
                                <1>     ; cl = xstart, edi = ch = ystart, bh = cols,
                                <1>     ; [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight, attr = 0
                                <1>
                                <1>     ; dest=ystart*cheight*nbcols+xstart;
0000272B 52                     <1>     push    edx
0000272C 50                     <1>     push    eax
                                <1>
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0205)
0000272D 66B80502               <1>     mov     ax, 0205h
00002731 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002735 66EF                   <1>     out     dx, ax
                                <1>
                                <1>             ; 08/08/2016
00002737 0FB605[32680000]       <1>     movzx    eax, byte [CHAR_HEIGHT]
0000273E F6E5                   <1>     mul     ch
                                <1>     ;; 07/08/2016
                                <1>     ;movzx  eax, ch
                                <1>     ;shl    ax, 3 ; * 8 ; * [CHAR_HEIGHT]
00002740 0FB615[30680000]       <1>     movzx   edx, byte [CRT_COLS] ; = [VGA_COLS]
00002747 F7E2                   <1>     mul     edx
                                <1>     ; edx  = 0
00002749 88CA                   <1>     mov     dl, cl
0000274B 01D0                   <1>     add     eax, edx
0000274D 89C7                   <1>     mov     edi, eax
                                <1>     ; edi = dest
                                <1>     ; for(i=0;i<cheight;i++)
                                <1>     ; {
                                <1>     ;  memsetb(0xa000,dest+i*nbcols,attr,cols);
                                <1>     ; }
0000274F 81C700000A00           <1>     add     edi, 0A0000h
00002755 51                     <1>     push    ecx
                                <1>     ; 08/08/2016
00002756 8A35[32680000]         <1>     mov     dh, [CHAR_HEIGHT]
                                <1>     ;; 07/08/2016
```

```
                                <1>     ;mov    dh, 8 ; 07/08/2016
0000275C 28D2                   <1>     sub     dl, dl ; i
                                <1> vgamem_fill_pl4_0:
0000275E 57                     <1>     push    edi
0000275F 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS]
00002766 F6E2                   <1>     mul     dl
                                <1>     ; eax = i * nbcols
00002768 01C7                   <1>     add     edi, eax ; dest+i*nbcols
0000276A 88D8                   <1>     mov     al, bl ; attr ; 04/08/2016
0000276C 0FB6CF                 <1>     movzx   ecx, bh ; cols
0000276F F3AA                   <1>     rep     stosb
00002771 5F                     <1>     pop     edi
00002772 75EA                   <1>     jnz     short vgamem_fill_pl4_0
                                <1> vgamem_fill_pl4_1:
00002774 59                     <1>     pop     ecx
                                <1>
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0005);
00002775 66B80500               <1>     mov     ax, 0005h
00002779 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
0000277D 66EF                   <1>     out     dx, ax
                                <1>
0000277F 58                     <1>     pop     eax
00002780 5A                     <1>     pop     edx
                                <1>
00002781 C3                     <1>     retn
                                <1>
                                <1> vgamem_copy_l8:
                                <1>     ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 08/08/2016
                                <1>     ; 07/08/2016
                                <1>     ; 06/08/2016
                                <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; TEMPORARY
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgamem_copy_pl4'
                                <1>     ;
                                <1>     ; vgamem_copy_pl4(xstart,ysrc,ydest,cols,nbcols,cheight)
                                <1>     ; cl = xstart, ah = ysrc (i+nblines), ch = ydest (i),
                                <1>     ; bh = cols, [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight
                                <1>
                                <1>     ; src=ysrc*cheight*nbcols+xstart;
                                <1>     ; dest=ydest*cheight*nbcols+xstart;
                                <1>
00002782 52                     <1>     push    edx
00002783 50                     <1>     push    eax
                                <1>
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0105)
                                <1>     ;mov    ax, 0105h
                                <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                                <1>     ;out    dx, ax
                                <1>
                                <1>     ;mov    ah, [esp+1]
                                <1>
00002784 0FB6D4                 <1>     movzx   edx, ah ; ysrc
                                <1>     ; 08/08/2016
00002787 0FB605[32680000]       <1>     movzx   eax, byte [CHAR_HEIGHT]
0000278E 8A25[30680000]         <1>     mov     ah, [CRT_COLS] ; nbcols
00002794 F6E4                   <1>     mul     ah
                                <1>     ;; 07/08/2016
                                <1>     ;movzx  eax, byte [CRT_COLS]
                                <1>     ;shl    ax, 3 ; * 8 ; * [CHAR_HEIGHT]
00002796 50                     <1>     push    eax ; cheight * nbcols
00002797 F7E2                   <1>     mul     edx ; * ysrc
                                <1>     ; eax = ysrc * cheight * nbcols
                                <1>     ; edx = 0
00002799 88CA                   <1>     mov     dl, cl ; edx = xstart
0000279B 01D0                   <1>     add     eax, edx
0000279D 89C6                   <1>     mov     esi, eax ; src
                                <1>     ;shl    si, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
0000279F C1E603                 <1>     shl     esi, 3
000027A2 88EA                   <1>     mov     dl, ch ; ydest
000027A4 58                     <1>     pop     eax ; cheight * nbcols
000027A5 F7E2                   <1>     mul     edx
                                <1>     ; eax = ydest * cheight * nbcols
000027A7 88CA                   <1>     mov     dl, cl ; edx = xstart
000027A9 01D0                   <1>     add     eax, edx
000027AB 89C7                   <1>     mov     edi, eax ; dest
                                <1>     ;shl    di, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
000027AD C1E703                 <1>     shl     edi, 3
                                <1>     ; esi = src
                                <1>     ; edi = dest
                                <1>     ; for(i=0;i<cheight;i++)
                                <1>     ; {
                                <1>     ;  memcpyb(0xa000,dest+i*nbcols,0xa000,src+i*nbcols,cols);
                                <1>     ; }
000027B0 51                     <1>     push    ecx
000027B1 B900000A00             <1>     mov     ecx, 0A0000h
000027B6 01CE                   <1>     add     esi, ecx
000027B8 01CF                   <1>     add     edi, ecx
                                <1>     ; 08/08/2016
000027BA 8A35[32680000]         <1>     mov     dh, [CHAR_HEIGHT]
                                <1>     ;; 07/08/2016
                                <1>     ;mov    dh, 8 ; 07/08/2016
000027C0 28D2                   <1>     sub     dl, dl ; i
                                <1> vgamem_copy_l8_0:
000027C2 56                     <1>     push    esi
000027C3 57                     <1>     push    edi
000027C4 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS]
000027CB F6E2                   <1>     mul     dl
                                <1>     ; eax = i * nbcols
                                <1>     ;shl    ax, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
000027CD C1E003                 <1>     shl     eax, 3
000027D0 01C7                   <1>     add     edi, eax ; dest+i*nbcols
000027D2 01C6                   <1>     add     esi, eax
000027D4 0FB6CF                 <1>     movzx   ecx, bh ; cols
                                <1>     ;shl    cx, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
000027D7 C1E103                 <1>     shl     ecx, 3
000027DA F3A4                   <1>     rep     movsb
000027DC 5F                     <1>     pop     edi
000027DD 5E                     <1>     pop     esi
000027DE FEC2                   <1>     inc     dl ; 06/08/2016
000027E0 FECE                   <1>     dec     dh
000027E2 75DE                   <1>     jnz     short vgamem_copy_l8_0
                                <1> vgamem_copy_l8_1:
000027E4 59                     <1>     pop     ecx
                                <1>
```

```
                                <1>     ;; outw(VGAREG_GRDC_ADDRESS, 0x0005);
                                <1>     ;mov   ax, 0005h
                                <1>     ;mov   dx, 3CEh ; VGAREG_GRDC_ADDRESS
                                <1>     ;out   dx, ax
                                <1>
000027E5 58                     <1>     pop    eax
000027E6 5A                     <1>     pop    edx
                                <1>
000027E7 C3                     <1>     retn
                                <1>
                                <1> vgamem_fill_l8:
                                <1>     ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 08/08/2016
                                <1>     ; 07/08/2016
                                <1>     ; 06/08/2016
                                <1>     ; 04/08/2016
                                <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; TEMPORARY
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgamem_fill_pl4'
                                <1>     ;
                                <1>     ; vgamem_fill_pl4(xstart,ystart,cols,nbcols,cheight,attr)
                                <1>     ; cl = xstart, edi = ch = ystart, bh = cols,
                                <1>     ; [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight, attr = 0
                                <1>
                                <1>     ; dest=ystart*cheight*nbcols+xstart;
000027E8 52                     <1>     push   edx
000027E9 50                     <1>     push   eax
                                <1>
                                <1>     ;; outw(VGAREG_GRDC_ADDRESS, 0x0205)
                                <1>     ;mov   ax, 0205h
                                <1>     ;mov   dx, 3CEh ; VGAREG_GRDC_ADDRESS
                                <1>     ;out   dx, ax
                                <1>
                                <1>         ; 08/08/2016
000027EA 0FB605[32680000]       <1>     movzx   eax, byte [CHAR_HEIGHT]
000027F1 F6E5                   <1>     mul    ch
                                <1>     ;; 07/08/2016
                                <1>     ;movzx eax, ch
                                <1>     ;shl   ax, 3 ; * 8 ; * [CHAR_HEIGHT]
000027F3 0FB615[30680000]       <1>     movzx  edx, byte [CRT_COLS] ; = [VGA_COLS]
000027FA F7E2                   <1>     mul    edx
                                <1>     ; edx  = 0
000027FC 88CA                   <1>     mov    dl, cl
000027FE 01D0                   <1>     add    eax, edx
00002800 89C7                   <1>     mov    edi, eax
                                <1>     ;shl   di, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
00002802 C1E703                 <1>     shl    edi, 3
                                <1>     ; edi = dest
                                <1>     ; for(i=0;i<cheight;i++)
                                <1>     ; {
                                <1>     ;  memsetb(0xa000,dest+i*nbcols,attr,cols);
                                <1>     ; }
00002805 81C700000A00           <1>     add    edi, 0A0000h
0000280B 51                     <1>     push   ecx
                                <1>     ; 08/08/2016
0000280C 8A35[32680000]         <1>     mov    dh, [CHAR_HEIGHT]
                                <1>     ;; 07/08/2016
                                <1>     ;mov   dh, 8 ; 07/08/2016
00002812 28D2                   <1>     sub    dl, dl ; i
                                <1> vgamem_fill_l8_0:
00002814 57                     <1>     push   edi
00002815 0FB605[30680000]       <1>     movzx  eax, byte [CRT_COLS]
0000281C F6E2                   <1>     mul    dl
                                <1>     ; eax = i * nbcols
                                <1>     ;shl   ax, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
0000281E C1E003                 <1>     shl    eax, 3
00002821 01C7                   <1>     add    edi, eax ; dest+i*nbcols
00002823 88D8                   <1>     mov    al, bl ; attr ; 04/08/2016
00002825 0FB6CF                 <1>     movzx  ecx, bh ; cols
                                <1>     ;shl   cx, 3 ; * 8 ; 06/08/2016
                                <1>     ; 02/08/2022
00002828 C1E103                 <1>     shl    ecx, 3
0000282B F3AA                   <1>     rep    stosb
0000282D 5F                     <1>     pop    edi
0000282E FEC2                   <1>     inc    dl ; 06/08/2016
00002830 FECE                   <1>     dec    dh
00002832 75E0                   <1>     jnz    short vgamem_fill_l8_0
                                <1> vgamem_fill_l8_1:
00002834 59                     <1>     pop    ecx
                                <1>
                                <1>     ;; outw(VGAREG_GRDC_ADDRESS, 0x0005);
                                <1>     ;mov   ax, 0005h
                                <1>     ;mov   dx, 3CEh ; VGAREG_GRDC_ADDRESS
                                <1>     ;out   dx, ax
                                <1>
00002835 58                     <1>     pop    eax
00002836 5A                     <1>     pop    edx
                                <1>
00002837 C3                     <1>     retn
                                <1>
                                <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 07/07/2016
                                <1> ; 27/06/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;-------------------------------------------------------
                                <1> ; SCROLL DOWN
                                <1> ;  THIS ROUTINE SCROLLS DOWN THE INFORMATION ON THE CRT
                                <1> ; ENTRY --
                                <1> ;  CH,CL = UPPER LEFT CORNER OF REGION TO SCROLL
                                <1> ;  DH,DL = LOWER RIGHT CORNER OF REGION TO SCROLL
                                <1> ;   BOTH OF THE ABOVE ARE IN CHARACTER POSITIONS
                                <1> ;  BH = FILL VALUE FOR BLANKED LINES
                                <1> ;  AL = # LINES TO SCROLL (AL=0 MEANS BLANK THE ENTIRE FIELD)
                                <1> ;  DS = DATA SEGMENT
                                <1> ;  ES = REGEN SEGMENT
                                <1> ; EXIT --
                                <1> ;  NOTHING, THE SCREEN IS SCROLLED
                                <1> ;-------------------------------------------------------
                                <1>
                                <1>     ; cl = upper left column
                                <1>     ; ch = upper left row
                                <1>     ; dl = lower rigth column
                                <1>     ; dh = lower right row
                                <1>     ;
                                <1>     ; al = line count (AL=0 means blank entire fields)
                                <1>     ; bl = fill value for blanked lines
                                <1>     ; bh = unused
```

```
                                <1>
                                <1> GRAPHICS_DOWN:
                                <1>     ; 07/07/2016
                                <1>     ; ah = Current video mode, [CRT_MODE]
                                <1>     ; std                           ; SET DIRECTION
00002838 80FC07                 <1>     cmp     ah, 7
0000283B 7769                   <1>         ja  short vga_graphics_down ; 03/08/2022
                                <1>     ;je     _n0
                                <1>
0000283D 88C7                   <1>     mov     bh, al                  ; save line count in BH
                                <1>     ;mov    ax, dx                  ; GET LOWER RIGHT POSITION INTO AX REG
                                <1>     ; 03/08/2022
0000283F 89D0                   <1>     mov     eax, edx
                                <1>
                                <1> ;-----      USE CHARACTER SUBROUTINE FOR POSITIONING
                                <1> ;-----      ADDRESS RETURNED IS MULTIPLIED BY 2 FROM CORRECT VALUE
                                <1>
00002841 E8E8020000             <1>     call    GRAPH_POSN
                                <1>     ;movzx  edi, ax                 ; SAVE RESULT AS DESTINATION ADDRESS
                                <1>     ; 03/08/2022
00002846 89C7                   <1>     mov     edi, eax
                                <1>
                                <1> ;-----      DETERMINE SIZE OF WINDOW
                                <1>
00002848 6629CA                 <1>     sub     dx, cx
0000284B 6681C20101             <1>         add     dx, 101h                ; ADJUST VALUES
00002850 C0E602                 <1>     sal     dh, 2                   ; MULTIPLY ROWS BY 4 AT 8 VERT DOTS/CHAR
                                <1>                                     ; AND EVEN/ODD ROWS
                                <1> ;-----      DETERMINE CRT MODE
                                <1>
00002853 803D[2E680000]06       <1>     cmp     byte [CRT_MODE], 6      ; TEST FOR MEDIUM RES
0000285A 7305                   <1>     jnc     short _R12              ; FIND_SOURCE_DOWN
                                <1>
                                <1> ;-----      MEDIUM RES DOWN
0000285C D0E2                   <1>     sal     dl, 1                   ; # COLUMNS * 2, SINCE 2 BYTES/CHAR
                                <1>     ;sal    di, 1                   ; OFFSET *2 SINCE 2 BYTES/CHAR
                                <1>     ; 03/08/2022
0000285E D1E7                   <1>     sal     edi, 1
                                <1>     ;inc    di                      ; POINT TO LAST BYTE
                                <1>     ; 03/08/2022
00002860 47                     <1>     inc     edi
                                <1>
                                <1> ;-----      DETERMINE THE SOURCE ADDRESS IN THE BUFFER
                                <1>
                                <1> _R12:                                       ; FIND_SOURCE_DOWN
00002861 81C700800B00           <1>     add     edi, 0B8000h
00002867 6681C7F000             <1>     add     di, 240                 ; POINT TO LAST ROW OF PIXELS
0000286C C0E702                 <1>     sal     bh, 2                   ; multiply number of lines by 4
0000286F 74(06)                 <1>     jz      short 6                 ; IF ZERO, THEN BLANK ENTIRE FIELD
00002871 B050                   <1>     mov     al, 80                  ; 80 BYTES/ROW
00002873 F6E7                   <1>     mul     bh                      ; determine offset to source
00002875 89FE                   <1>     mov     esi, edi                ; SET UP SOURCE
                                <1>     ;sub    si, ax                  ; SUBTRACT THE OFFSET
                                <1>     ; 03/08/2022
00002877 29C6                   <1>     sub     esi, eax
00002879 88F4                   <1>     mov     ah, dh                  ; NUMBER OF ROWS IN FIELD
0000287B 28FC                   <1>     sub     ah, bh                  ; determine number to move
                                <1>
                                <1> ;-----      LOOP THROUGH, MOVING ONE ROW AT A TIME, BOTH EVEN AND ODD FIELDS
                                <1>
                                <1> _R13:                                       ; ROW_LOOP_DOWN
0000287D E825010000             <1>         call    _R17                    ; MOVE ONE ROW
00002882 6681EE5020             <1>     sub     si, 2000h+80            ; MOVE TO NEXT ROW
00002887 6681EF5020             <1>     sub     di, 2000h+80
0000288C FECC                   <1>     dec     ah                      ; NUMBER OF ROWS TO MOVE
0000288E 75ED                   <1>     jnz     short _R13              ; CONTINUE TILL ALL MOVED
                                <1>
                                <1> ;-----      FILL IN THE VACATED LINE(S)
                                <1> _R14:                                       ; CLEAR_ENTRY_DOWN
00002890 88D8                   <1>     mov     al, bl                  ; attribute to fill with
                                <1> _R15_:                                      ; CLEAR_LOOP_DOWN
00002892 E82C010000             <1>     call    _R18                    ; CLEAR A ROW
00002897 6681EF5020             <1>     sub     di, 2000h+80            ; POINT TO NEXT LINE
0000289C FECF                   <1>     dec     bh                      ; number of lines to fill
0000289E 75F2                   <1>         jnz short _R15_             ; CLEAR_LOOP_DOWN
                                <1>     ; 18/11/2020
000028A0 FC                     <1>     cld                             ; RESET THE DIRECTION FLAG
                                <1>
000028A1 C3                     <1>     retn                            ; EVERYTHING DONE
                                <1>
                                <1> _R16:                                       ; BLANK_FIELD_DOWN
000028A2 88F7                   <1>     mov     bh, dh                  ; set blank count to everything in field
000028A4 EBEA                   <1>         jmp     short _R14              ; CLEAR THE FIELD
                                <1>
                                <1> vga_graphics_down:
                                <1>     ; 03/08/2022 - TRDOS 386 Kertnel v2.0.5
                                <1>     ; 12/04/2021
                                <1>     ; 08/08/2016
                                <1>     ; 07/08/2016
                                <1>     ; 31/07/2016
                                <1>     ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_scroll'
                                <1>     ;
                                <1>
                                <1>     ; cl = upper left column
                                <1>     ; ch = upper left row
                                <1>     ; dl = lower rigth column
                                <1>     ; dh = lower right row
                                <1>     ;
                                <1>     ; al = line count (AL=0 means blank entire fields)
                                <1>     ; bl = fill value for blanked lines
                                <1>     ; bh = unused
                                <1>     ;
                                <1>     ; ah = [CRT_MODE], current video mode
                                <1>
000028A6 FC                     <1>         cld     ; !!! Clear direction flag !!!
                                <1>
000028A7 88C7                   <1>     mov     bh, al ; 31/07/2016
                                <1>
000028A9 BE[4A680000]           <1>     mov     esi, vga_modes
000028AE 89F7                   <1>     mov     edi, esi
000028B0 83C710                 <1>     add     edi, vga_mode_count
                                <1> vga_g_down_0:
000028B3 AC                     <1>     lodsb
000028B4 38E0                   <1>     cmp     al, ah ; [CRT_MODE]
000028B6 7405                   <1>     je      short vga_g_down_1
000028B8 39FE                   <1>     cmp     esi, edi
000028BA 72F7                   <1>     jb      short vga_g_down_0
                                <1>     ; xor   bh, bh ; 31/07/2016
000028BC C3                     <1>     retn    ; nothing to do
```

```
                                <1> vga_g_down_1:
000028BD 88F8                   <1>     mov     al, bh ; 31/07/2016
000028BF 83C64F                 <1>     add     esi, vga_memmodel - (vga_modes + 1)
                                <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
                                <1>
                                <1>     ; if(rlr>=nbrows)rlr=nbrows-1;
                                <1>     ; if(clr>=nbcols)clr=nbcols-1;
                                <1>     ; if(nblines>nbrows)nblines=0;
                                <1>     ; cols=clr-cul+1;
                                <1>
000028C2 3A35[36680000]         <1>     cmp     dh, [VGA_ROWS]
000028C8 7208                   <1>     jb      short vga_g_down_2
000028CA 8A35[36680000]         <1>     mov     dh, [VGA_ROWS]
000028D0 FECE                   <1>     dec     dh
                                <1> vga_g_down_2:
000028D2 3A15[30680000]         <1>     cmp     dl, [CRT_COLS]  ; = [VGA_COLS]
000028D8 7208                   <1>     jb      short vga_g_down_3
000028DA 8A15[30680000]         <1>     mov     dl, [CRT_COLS]
000028E0 FECA                   <1>     dec     dl
                                <1> vga_g_down_3:
000028E2 3A05[36680000]         <1>     cmp     al, [VGA_ROWS]
000028E8 7602                   <1>     jna     short vga_g_down_4
000028EA 28C0                   <1>     sub     al, al ; 0
                                <1> vga_g_down_4:
000028EC 88F7                   <1>     mov     bh, dh ; clr
000028EE 28CF                   <1>     sub     bh, cl ; cul
000028F0 FEC7                   <1>     inc     bh ; cols = clr-cul+1
                                <1>
000028F2 20C0                   <1>     and     al, al ; nblines = 0
000028F4 7559                   <1>     jnz     short vga_g_down_6
000028F6 20ED                   <1>     and     ch, ch ; rul = 0
000028F8 7555                   <1>     jnz     short vga_g_down_6
000028FA 20C9                   <1>     and     cl, cl ; cul = 0
000028FC 7551                   <1>     jnz     short vga_g_down_6
                                <1>
000028FE 50                     <1>     push    eax ; push ax ; 12/04/2021
000028FF A0[36680000]           <1>     mov     al, [VGA_ROWS]
00002904 FEC8                   <1>     dec     al
00002906 38C6                   <1>     cmp     dh, al ; rlr = nbrows-1
00002908 7545                   <1>     jne     short vga_g_down_5
0000290A A0[30680000]           <1>         mov     al, [CRT_COLS]  ; = VGA_COLS
0000290F FEC8                   <1>     dec     al
00002911 38C2                   <1>     cmp     dl, al ; clr = nbcols-1
                                <1>     ;jne    short vga_g_down_5
                                <1>     ; 12/04/2021
00002913 58                     <1>     pop     eax ; pop ax
00002914 7539                   <1>     jne     short vga_g_down_5
                                <1>
00002916 66B80502               <1>     mov     ax, 0205h
0000291A 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
0000291E 66EF                   <1>     out     dx, ax
00002920 A0[36680000]           <1>     mov     al, [VGA_ROWS]
00002925 8A25[30680000]         <1>     mov     ah, [CRT_COLS] ; = [VGA_COLS]
0000292B F6E4                   <1>     mul     ah
0000292D 0FB7D0                 <1>     movzx   edx, ax
                                <1>     ; 08/08/2016
00002930 0FB605[32680000]       <1>     movzx   eax, byte [CHAR_HEIGHT]
00002937 F7E2                   <1>     mul     edx
                                <1>     ; eax = byte count
00002939 89C1                   <1>     mov     ecx, eax
                                <1>     ;; 07/08/2016
                                <1>     ;shl    dx, 3 ; * 8 ; * [CHAR_HEIGHT]
                                <1>     ;mov    ecx, edx
0000293B 88D8                   <1>     mov     al, bl ; fill value for blanked lines
0000293D BF00000A00             <1>     mov     edi, 0A0000h
00002942 F3AA                   <1>     rep     stosb
                                <1>
                                <1>     ; 03/08/2022
00002944 30E4                   <1>     xor     ah, ah ; 0
                                <1>
00002946 B005                   <1>     mov     al, 5
00002948 66BACE03               <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
0000294C 66EF                   <1>     out     dx, ax ; 0005h
                                <1>
0000294E C3                     <1>     retn
                                <1>
                                <1> vga_g_down_5:
                                <1>     ; 12/04/2021
                                <1>     ;pop    eax  ; pop ax
                                <1>
                                <1> vga_g_down_6:
                                <1>     ; [ESI] = VGA memory model number for current video mode
                                <1>         ;
                                <1>         ; LINEAR8 equ 5
                                <1>         ; PLANAR4 equ 4
                                <1>         ; PLANAR1 equ 3
                                <1>
0000294F 803E04                 <1>     cmp     byte [esi], PLANAR4
00002952 742C                   <1>     je      short vga_g_down_planar
00002954 803E03                 <1>     cmp     byte [esi], PLANAR1
00002957 7427                   <1>     je      short vga_g_down_planar
                                <1> vga_g_down_linear8:
                                <1>     ; 07/07/2016 (TEMPORARY)
                                <1>     ;
                                <1>     ; cl = upper left column ; cul
                                <1>     ; ch = upper left row ; rul
                                <1>     ; dl = lower rigth column ; clr
                                <1>     ; dh = lower right row ; rlr
                                <1>
                                <1> vga_g_down_l0:
                                <1>     ;{for(i=rlr;i>=rul;i--)
                                <1>         ; if((i<rul+nblines)||(nblines==0))
00002959 08C0                   <1>     or      al, al
0000295B 741C                   <1>     jz      short vga_g_down_l2
0000295D 88C4                   <1>     mov     ah, al
0000295F 00EC                   <1>     add     ah, ch
                                <1>     ;jc     short vga_g_down_l2
00002961 86EE                   <1>     xchg    ch, dh
00002963 38E5                   <1>     cmp     ch, ah
00002965 7212                   <1>     jb      short vga_g_down_l2
00002967 88EC                   <1>     mov     ah, ch
00002969 28C4                   <1>     sub     ah, al ; ah = i - nblines
                                <1>     ; else
                                <1>     ; vgamem_copy_pl4(cul,i,i-nblines,cols,nbcols,cheight);
0000296B E812FEFFFF             <1>     call    vgamem_copy_l8
                                <1> vga_g_down_l1:
00002970 86F5                   <1>     xchg    dh, ch
00002972 FECE                   <1>     dec     dh
00002974 38EE                   <1>     cmp     dh, ch
00002976 73E1                   <1>     jnb     short vga_g_down_l0
00002978 C3                     <1>     retn
                                <1>
                                <1> vga_g_down_l2:
                                <1>     ; vgamem_fill_pl4(cul,i,cols,nbcols,cheight,attr);
```

```
00002979 E86AFEFFFF          <1>     call    vgamem_fill_l8
0000297E EBF0                <1>     jmp     short vga_g_down_l1
                             <1>
                             <1> vga_g_down_planar:
                             <1>     ; cl = upper left column ; cul
                             <1>     ; ch = upper left row ; rul
                             <1>     ; dl = lower rigth column ; clr
                             <1>     ; dh = lower right row ; rlr
                             <1> vga_g_down_pl0:
                             <1>     ;{for(i=rlr;i>=rul;i--)
                             <1>         ; if((i<rul+nblines)||(nblines==0))
00002980 08C0                <1>     or      al, al
00002982 741C                <1>     jz      short vga_g_down_pl2
00002984 88C4                <1>     mov     ah, al
00002986 00EC                <1>     add     ah, ch
                             <1>     ;jc     short vga_g_down_pl2
00002988 86EE                <1>     xchg    ch, dh
0000298A 38E5                <1>     cmp     ch, ah
0000298C 7212                <1>     jb      short vga_g_down_pl2
0000298E 88EC                <1>     mov     ah, ch
00002990 28C4                <1>     sub     ah, al ; ah = i - nblines
                             <1>     ; else
                             <1>     ; vgamem_copy_pl4(cul,i,i-nblines,cols,nbcols,cheight);
00002992 E826FDFFFF          <1>     call    vgamem_copy_pl4
                             <1> vga_g_down_pl1:
00002997 86F5                <1>     xchg    dh, ch
00002999 FECE                <1>     dec     dh
0000299B 38EE                <1>     cmp     dh, ch
0000299D 73E1                <1>     jnb     short vga_g_down_pl0
0000299F C3                  <1>     retn
                             <1>
                             <1> vga_g_down_pl2:
                             <1>     ; vgamem_fill_pl4(cul,i,cols,nbcols,cheight,attr);
000029A0 E886FDFFFF          <1>     call    vgamem_fill_pl4
000029A5 EBF0                <1>     jmp     short vga_g_down_pl1
                             <1>
                             <1>
                             <1>
                             <1> ; 27/06/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1>
                             <1> ;-----      ROUTINE TO MOVE ONE ROW OF INFORMATION
                             <1>
                             <1> _R17:
000029A7 0FB6CA              <1>     movzx   ecx, dl                 ; NUMBER OF BYTES IN THE ROW
000029AA 56                  <1>     push    esi
000029AB 57                  <1>     push    edi                     ; SAVE POINTERS
000029AC F3A4                <1>     rep     movsb                   ; MOVE THE EVEN FIELD
000029AE 5F                  <1>     pop     edi
000029AF 5E                  <1>     pop     esi
000029B0 6681C60020          <1>     add     si, 2000h
000029B5 6681C70020          <1>     add     di, 2000h               ; POINT TO THE ODD FIELD
000029BA 56                  <1>     push    esi
000029BB 57                  <1>     push    edi                     ; SAVE THE POINTERS
000029BC 88D1                <1>     mov     cl, dl                  ; COUNT BACK
000029BE F3A4                <1>     rep     movsb                   ; MOVE THE ODD FIELD
000029C0 5F                  <1>     pop     edi
000029C1 5E                  <1>     pop     esi                     ; POINTERS BACK
000029C2 C3                  <1>     retn                            ; RETURN TO CALLER
                             <1>
                             <1> ;-----      CLEAR A SINGLE ROW
                             <1>
                             <1> _R18:
000029C3 0FB6CA              <1>     movzx   ecx, dl                 ; NUMBER OF BYTES IN FIELD
000029C6 57                  <1>     push    edi                     ; SAVE POINTER
000029C7 F3AA                <1>     rep     stosb                   ; STORE THE NEW VALUE
000029C9 5F                  <1>     pop     edi                     ; POINTER BACK
000029CA 6681C70020          <1>     add     di, 2000h               ; POINT TO ODD FIELD
000029CF 57                  <1>     push    edi
000029D0 88D1                <1>     mov     cl, dl
000029D2 F3AA                <1>     rep     stosb                   ; FILL THE ODD FIELD
000029D4 5F                  <1>     pop     edi
000029D5 C3                  <1>     retn                            ; RETURN TO CALLER
                             <1>
                             <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 04/07/2016
                             <1> ; 01/07/2016
                             <1> ; 30/06/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1> ;--------------------------------------------------
                             <1> ; GRAPHICS WRITE
                             <1> ;  THIS ROUTINE WRITES THE ASCII CHARACTER TO THE CURRENT
                             <1> ;  POSITION ON THE SCREEN.
                             <1> ; ENTRY --
                             <1> ;  AL = CHARACTER TO WRITE
                             <1> ;  BL = COLOR ATTRIBUTE TO BE USED FOR FOREGROUND COLOR
                             <1> ;   IF BIT 7 IS SET, THE CHAR IS XOR'D INTO THE REGEN BUFFER
                             <1> ;   (0 IS USED FOR THE BACKGROUND COLOR)
                             <1> ;  CX = NUMBER OF CHARS TO WRITE
                             <1> ;  DS = DATA SEGMENT
                             <1> ;  ES = REGEN SEGMENT
                             <1> ; EXIT --
                             <1> ;  NOTHING IS RETURNED
                             <1> ;
                             <1> ; GRAPHICS READ
                             <1> ;  THIS ROUTINE READS THE ASCII CHARACTER AT THE CURRENT CURSOR
                             <1> ;  POSITION ON THE SCREEN BY MATCHING THE DOTS ON THE SCREEN TO THE
                             <1> ;  CHARACTER GENERATOR CODE POINTS
                             <1> ; ENTRY --
                             <1> ;  NONE (0 IS ASSUMED AS THE BACKGROUND COLOR)
                             <1> ; EXIT --
                             <1> ;  AL = CHARACTER READ AT THAT POSITION (0 RETURNED IF NONE FOUND)
                             <1> ;
                             <1> ; FOR BOTH ROUTINES, THE IMAGES USED TO FORM CHARS ARE CONTAINED IN ROM
                             <1> ;  FOR THE 1ST 128 CHARS.  TO ACCESS CHARS IN THE SECOND HALF, THE USER
                             <1> ;  MUST INITIALIZE THE VECTOR AT INTERRUPT 1FH (LOCATION 0007CH) TO
                             <1> ;  POINT TO THE USER SUPPLIED TABLE OF GRAPHIC IMAGES (8X8 BOXES).
                             <1> ;  FAILURE TO DO SO WILL CAUSE IN STRANGE RESULTS
                             <1> ;--------------------------------------------------
                             <1>
                             <1> GRAPHICS_WRITE:
000029D6 25FF000000          <1>     and     eax, 0FFh               ; ZERO TO HIGH OF CODE POINT
000029DB 50                  <1>     push    eax                     ; SAVE CODE POINT VALUE
                             <1>
                             <1> ;-----      DETERMINE POSITION IN REGEN BUFFER TO PUT CODE POINTS
                             <1>
000029DC E846010000          <1>     call    S26                     ; FIND LOCATION IN REGEN BUFFER
000029E1 89C7                <1>     mov     edi, eax                ; REGEN POINTER IN DI
                             <1>
                             <1> ;-----      DETERMINE REGION TO GET CODE POINTS FROM
                             <1>
000029E3 58                  <1>     pop     eax                     ; RECOVER CODE POINT
                             <1>
000029E4 BE[404D0100]        <1>     mov     esi, CRT_CHAR_GEN       ; OFFSET OF IMAGES
```

```
                                <1>
                                <1> ;-----      DETERMINE GRAPHICS MODE IN OPERATION
                                <1>                                     ; DETERMINE_MODE
                                <1>     ;sal    ax, 3                   ; MULTIPLY CODE POINT VALUE BY 8
                                <1>     ; 03/08/2022
000029E9 C1E003                 <1>     sal     eax, 3
000029EC 01C6                   <1>     add     esi, eax                ; SI HAS OFFSET OF DESIRED CODES
                                <1>
000029EE 803D[2E680000]06       <1>     cmp     byte [CRT_MODE], 6
000029F5 7231                   <1>     jc      short S6                ; TEST FOR MEDIUM RESOLUTION MODE
                                <1>
                                <1> ;-----      HIGH RESOLUTION MODE
                                <1>
000029F7 81C700800B00           <1>     add     edi, 0B8000h
                                <1> S1:                                 ; HIGH_CHAR
000029FD 57                     <1>     push    edi                     ; SAVE REGEN POINTER
000029FE 56                     <1>     push    esi                     ; SAVE CODE POINTER
000029FF B604                   <1>     mov     dh, 4                   ; NUMBER OF TIMES THROUGH LOOP
                                <1> S2:
00002A01 AC                     <1>     lodsb                           ; GET BYTE FROM CODE POINTS
00002A02 F6C380                 <1>     test    bl, 80h                 ; SHOULD WE USE THE FUNCTION
00002A05 7515                   <1>     jnz     short S5                ; TO PUT CHAR IN
00002A07 AA                     <1>     stosb                           ; STORE IN REGEN BUFFER
00002A08 AC                     <1>     lodsb
                                <1> S4:
00002A09 8887FF1F0000           <1>     mov     [edi+2000h-1], al       ; STORE IN SECOND HALF
00002A0F 83C74F                 <1>     add     edi, 79                 ; MOVE TO NEXT ROW IN REGEN
00002A12 FECE                   <1>     dec     dh                      ; DONE WITH LOOP
00002A14 75EB                   <1>     jnz     short S2
00002A16 5E                     <1>     pop     esi
00002A17 5F                     <1>     pop     edi                     ; RECOVER REGEN POINTER
00002A18 47                     <1>     inc     edi                     ; POINT TO NEXT CHAR POSITION
00002A19 E2E2                   <1>     loop    S1                      ; MORE CHARS TO WRITE
00002A1B C3                     <1>     retn
                                <1>
                                <1> S5:
00002A1C 3207                   <1>     xor     al, [edi]               ; EXCLUSIVE OR WITH CURRENT
00002A1E AA                     <1>     stosb                           ; STORE THE CODE POINT
00002A1F AC                     <1>     lodsb                           ; AGAIN FOR ODD FIELD
00002A20 3287FF1F0000           <1>     xor     al, [edi+2000h-1]
00002A26 EBE1                   <1>     jmp     short S4                ; BACK TO MAINSTREAM
                                <1>
                                <1> ;-----      MEDIUM RESOLUTION WRITE
                                <1> S6:                                 ; MED_RES_WRITE
00002A28 88DA                   <1>     mov     dl, bl                  ; SAVE HIGH COLOR BIT
                                <1>     ; 03/08/2022
00002A2A D1E7                   <1>     sal     edi, 1
                                <1>     ;sal    di, 1                   ; OFFSET*2 SINCE 2 BYTES/CHAR
                                <1>                                     ; EXPAND BL TO FULL WORD OF COLOR
00002A2C 80E303                 <1>     and     bl, 3                   ; ISOLATE THE COLOR BITS ( LOW 2 BITS )
00002A2F B055                   <1>     mov     al, 055h                ; GET BIT CONVERSION MULTIPLIER
00002A31 F6E3                   <1>     mul     bl                      ; EXPAND 2 COLOR BITS TO 4 REPLICATIONS
00002A33 88C3                   <1>     mov     bl, al                  ; PLACE BACK IN WORK REGISTER
00002A35 88C7                   <1>     mov     bh, al                  ; EXPAND TO 8 REPLICATIONS OF COLOR BITS
00002A37 81C700800B00           <1>     add     edi, 0B8000h
                                <1> S7:                                         ; MED_CHAR
00002A3D 57                     <1>     push    edi                     ; SAVE REGEN POINTER
00002A3E 56                     <1>     push    esi                     ; SAVE THE CODE POINTER
00002A3F B604                   <1>     mov     dh, 4                   ; NUMBER OF LOOPS
                                <1> S8:
00002A41 AC                     <1>     lodsb                           ; GET CODE POINT
00002A42 E8B1000000             <1>     call    S21                     ; DOUBLE UP ALL THE BITS
00002A47 6621D8                 <1>     and     ax, bx                  ; CONVERT TO FOREGROUND COLOR ( 0 BACK )
00002A4A 86E0                   <1>     xchg    ah, al                  ; SWAP HIGH/LOW BYTES FOR WORD MOVE
00002A4C F6C280                 <1>     test    dl, 80h                 ; IS THIS XOR FUNCTION
00002A4F 7403                   <1>     jz      short S9                ; NO, STORE IT IN AS IS
00002A51 663307                 <1>     xor     ax, [edi]               ; DO FUNCTION WITH LOW/HIGH
                                <1> S9:
00002A54 668907                 <1>     mov     [edi], ax               ; STORE FIRST BYTE HIGH, SECOND LOW
00002A57 AC                     <1>     lodsb                           ; GET CODE POINT
00002A58 E89B000000             <1>     call    S21
00002A5D 6621D8                 <1>     and     ax, bx                  ; CONVERT TO COLOR
00002A60 86E0                   <1>     xchg    ah, al                  ; SWAP HIGH/LOW BYTES FOR WORD MOVE
00002A62 F6C280                 <1>     test    dl, 80h                 ; AGAIN, IS THIS XOR FUNCTION
00002A65 7407                   <1>     jz      short _S10              ; NO, JUST STORE THE VALUES
00002A67 66338700200000         <1>     xor     ax, [edi+2000h]                 ; FUNCTION WITH FIRST HALF LOW
                                <1> _S10:
00002A6E 66898700200000         <1>     mov     [edi+2000h], ax                 ; STORE SECOND PORTION HIGH
00002A75 6683C750               <1>     add     di, 80                  ; POINT TO NEXT LOCATION
00002A79 FECE                   <1>     dec     dh
00002A7B 75C4                   <1>     jnz     short S8                ; KEEP GOING
00002A7D 5E                     <1>     pop     esi                     ; RECOVER CODE POINTER
00002A7E 5F                     <1>     pop     edi                     ; RECOVER REGEN POINTER
00002A7F 47                     <1>     inc     edi                     ; POINT TO NEXT CHAR POSITION
00002A80 47                     <1>     inc     edi
00002A81 E2BA                   <1>     loop    S7                      ; MORE TO WRITE
00002A83 C3                     <1>     retn
                                <1>
                                <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 04/07/2016
                                <1> ; 01/07/2016
                                <1> ; 30/06/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;----------------------------------------
                                <1> ; GRAPHICS READ
                                <1> ;----------------------------------------
                                <1> GRAPHICS_READ:
00002A84 E89E000000             <1>     call    S26                     ; CONVERTED TO OFFSET IN REGEN
00002A89 89C6                   <1>     mov     esi, eax                ; SAVE IN SI
00002A8B 81C600800B00           <1>     add     esi, 0B8000h            ; 01/07/2016
00002A91 83EC08                 <1>     sub     esp, 8                  ; ALLOCATE SPACE FOR THE READ CODE POINT
00002A94 89E5                   <1>     mov     ebp, esp                ; POINTER TO SAVE AREA
                                <1>
                                <1> ;-----      DETERMINE GRAPHICS MODES
00002A96 B604                   <1>     mov     dh, 4                   ; number of passes ; 01/07/2016
00002A98 803D[2E680000]06       <1>     cmp     byte [CRT_MODE], 6
00002A9F 7219                   <1>     jc      short S12               ; MEDIUM RESOLUTION
                                <1>
                                <1> ;-----      HIGH RESOLUTION READ
                                <1> ;-----      GET VALUES FROM REGEN BUFFER AND CONVERT TO CODE POINT
                                <1>     ;mov    dh,4                    ; NUMBER OF PASSES
                                <1> S11:
00002AA1 8A06                   <1>     mov     al, [esi]               ; GET FIRST BYTE
00002AA3 884500                 <1>     mov     [ebp], al               ; SAVE IN STORAGE AREA
00002AA6 45                     <1>     inc     ebp                     ; NEXT LOCATION
00002AA7 8A8600200000           <1>     mov     al, [esi+2000h]                 ; GET LOWER REGION BYTE
00002AAD 884500                 <1>     mov     [ebp], al               ; ADJUST AND STORE
00002AB0 45                     <1>     inc     ebp
00002AB1 83C650                 <1>     add     esi, 80                 ; POINTER INTO REGEN
00002AB4 FECE                   <1>     dec     dh                      ; LOOP CONTROL
00002AB6 75E9                   <1>     jnz     short S11               ; DO IT SOME MORE
00002AB8 EB1C                   <1>     jmp     short S14               ; GO MATCH THE SAVED CODE POINTS
                                <1>
                                <1> ;-----      MEDIUM RESOLUTION READ
```

```
                                <1> S12:
                                <1>     ;sal    si, 1                   ; OFFSET*2 SINCE 2 BYTES/CHAR
                                <1>     ; 03/08/2022
00002ABA D1E6                   <1>     sal     esi, 1
                                <1>     ;mov    dh, 4                   ; NUMBER OF PASSES
                                <1> S13:
00002ABC E84B000000             <1>     call    S23                     ; GET BYTES FROM REGEN INTO SINGLE SAVE
00002AC1 81C6FE1F0000           <1>     add     esi, 2000h-2            ; GO TO LOWER REGION
00002AC7 E840000000             <1>     call    S23                     ; GET THIS PAIR INTO SAVE
00002ACC 81EEB21F0000           <1>     sub     esi, 2000h-80+2               ; ADJUST POINTER BACK INTO UPPER
00002AD2 FECE                   <1>     dec     dh
00002AD4 75E6                   <1>     jnz     short S13               ; KEEP GOING UNTIL ALL 8 DONE
                                <1>
                                <1> ;-----      SAVE AREA HAS CHARACTER IN IT, MATCH IT
                                <1> S14:                                ; FIND_CHAR
00002AD6 BF[404D0100]           <1>     mov     edi, CRT_CHAR_GEN       ; ESTABLISH ADDRESSING
00002ADB 83ED08                 <1>     sub     ebp, 8                  ; ADJUST POINTER TO START OF SAVE AREA
00002ADE 89EE                   <1>     mov     esi, ebp
                                <1> S15:
                                <1>     ;mov    ax, 256                 ; NUMBER TO TEST AGAINST
                                <1>     ; 03/08/2022
00002AE0 29C0                   <1>     sub     eax, eax
00002AE2 FEC4                   <1>     inc     ah
                                <1>     ; eax = 256
                                <1> S16:
00002AE4 56                     <1>     push    esi                     ; SAVE SAVE AREA POINTER
00002AE5 57                     <1>     push    edi                     ; SAVE CODE POINTER
                                <1>     ;mov    ecx, 4                  ; NUMBER OF WORDS TO MATCH
                                <1>     ;repe   cmpsw                   ; COMPARE THE 8 BYTES AS WORDS
00002AE6 A7                     <1>     cmpsd                           ; compare first 4 bytes
00002AE7 7501                   <1>     jne     short S17               ;
00002AE9 A7                     <1>     cmpsd                           ; compare last 4 bytes
                                <1> S17:
00002AEA 5F                     <1>     pop     edi                     ; RECOVER THE POINTERS
00002AEB 5E                     <1>     pop     esi
                                <1>     ;jz     short S18               ; IF ZERO FLAG SET, THEN MATCH OCCURRED
00002AEC 7406                   <1>     je      short S18
                                <1>     ;                               ; NO MATCH, MOVE ON TO NEXT
00002AEE 83C708                 <1>     add     edi, 8                  ; NEXT CODE POINT
                                <1>     ;dec    ax                      ; LOOP CONTROL
                                <1>     ; 03/08/2022
00002AF1 48                     <1>     dec     eax
00002AF2 75F0                   <1>     jnz     short S16               ; DO ALL OF THEM
                                <1>
                                <1> ;-----      CHARACTER IS FOUND ( AL=0 IF NOT FOUND )
                                <1> S18:
00002AF4 83C408                 <1>     add     esp, 8                  ; READJUST THE STACK, THROW AWAY SAVE
00002AF7 C3                     <1>     retn                            ; ALL DONE
                                <1>
                                <1> ; 12/04/2021
                                <1> ; 30/06/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;---------------------------------------------
                                <1> ; EXPAND BYTE
                                <1> ;  THIS ROUTINE TAKES THE BYTE IN AL AND DOUBLES ALL
                                <1> ;  OF THE BITS, TURNING THE 8 BITS INTO 16 BITS.
                                <1> ;  THE RESULT IS LEFT IN AX
                                <1> ;---------------------------------------------
                                <1> S21:
                                <1>     ; 03/08/2022
                                <1>     ;push   cx                      ; SAVE REGISTER
                                <1>     ; 12/04/2021
00002AF8 51                     <1>     push    ecx
                                <1>     ;;mov   cx, 8                   ; SHIFT COUNT REGISTER FOR ONE BYTE
                                <1>     ;mov    cl, 8
00002AF9 B408                   <1>     mov     ah, 8
                                <1> S22:
00002AFB D0C8                   <1>     ror     al, 1                   ; SHIFT BITS, LOW BIT INTO CARRY FLAG
                                <1>     ;rcr    bp, 1                   ; MOVE CARRY FLAG (LOW BIT INTO RESULTS
                                <1>     ;sar    bp, 1                   ; SIGN EXTEND HIGH BIT (DOUBLE IT)
                                <1>     ; 03/08/2022
00002AFD 66D1D9                 <1>     rcr     cx, 1
00002B00 66D1F9                 <1>     sar     cx, 1
                                <1>
                                <1>     ;;loop  S22                     ; REPEAT FOR ALL 8 BITS
                                <1>     ;dec    cl
                                <1>     ;jnz    short S22
                                <1>     ;xchg   ax, bp                  ; MOVE RESULTS TO PARAMETER REGISTER
                                <1>     ; 03/08/5022
00002B03 FECC                   <1>     dec     ah
00002B05 75F4                   <1>     jnz     short S22
00002B07 6689C8                 <1>     mov     ax, cx
                                <1>     ;pop    cx                      ; RECOVER REGISTER
                                <1>     ; 12/04/2021
00002B0A 59                     <1>     pop     ecx
00002B0B C3                     <1>     retn                            ; ALL DONE
                                <1>
                                <1> ; 01/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                                <1> ;-----------------------------------------------
                                <1> ; MED_READ_BYTE
                                <1> ; THIS ROUTINE WILL TAKE 2 BYTES FROM THE REGEN BUFFER,
                                <1> ;  COMPARE AGAINST THE CURRENT FOREGROUND COLOR, AND PLACE
                                <1> ;  THE CORRESPONDING ON/OFF BIT PATTERN INTO THE CURRENT
                                <1> ;  POSITION IN THE SAVE AREA
                                <1> ; ENTRY --
                                <1> ;  SI,DS = POINTER TO REGEN AREA OF INTEREST
                                <1> ;  BX = EXPANDED FOREGROUND COLOR
                                <1> ;  BP = POINTER TO SAVE AREA
                                <1> ; EXIT --
                                <1> ;  SI AND BP ARE INCREMENTED
                                <1> ;-----------------------------------------------
                                <1> S23:
00002B0C 66AD                   <1>     lodsw                           ; GET FIRST BYTE AND SECOND BYTES
00002B0E 86C4                   <1>     xchg    al, ah                  ; SWAP FOR COMPARE
                                <1>     ;mov    cx, 0C000h              ; 2 BIT MASK TO TEST THE ENTRIES
                                <1>     ; 02/08/2022
00002B10 29C9                   <1>     sub     ecx, ecx
00002B12 B5C0                   <1>     mov     ch, 0C0h
                                <1>     ; ecx = 0C000h
                                <1>     ;mov    dl, 0                   ; RESULT REGISTER
                                <1>     ; 03/08/2022
00002B14 28D2                   <1>     sub     dl, dl
                                <1> S24:
                                <1>     ;test   ax, cx                  ; IS THIS SECTION BACKCROUND?
                                <1>         ; 03/08/2022
00002B16 85C8                   <1>     test    eax, ecx
00002B18 7401                   <1>     jz      short S25                 ; IF ZERO, IT IS BACKGROUND (CARRY=0)
00002B1A F9                     <1>     stc                             ; WASN'T, SO SET CARRY
                                <1> S25:
00002B1B D0D2                   <1>     rcl     dl, 1                   ; MOVE THAT BIT INTO THE RESULT
                                <1>     ;shr    cx, 2                   ; MOVE THE MASK TO THE RIGHT BY 2 BITS
                                <1>     ; 02/08/2022
00002B1D C1E902                 <1>     shr     ecx, 2
```

```
00002B20 73F4                <1>     jnc     short S24               ; DO IT AGAIN IF MASK DIDN'T FALL OUT
00002B22 885500              <1>     mov     [ebp], dl               ; STORE RESULT IN SAVE AREA
00002B25 45                  <1>     inc     ebp                     ; ADJUST POINTER
00002B26 C3                  <1>     retn                            ; ALL DONE
                             <1>
                             <1> ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 30/06/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1> ;----------------------------------------
                             <1> ; V4_POSITION
                             <1> ;  THIS ROUTINE TAKES THE CURSOR POSITION CONTAINED IN
                             <1> ;  THE MEMORY LOCATION, AND CONVERTS IT INTO AN OFFSET
                             <1> ;  INTO THE REGEN BUFFER, ASSUMING ONE BYTE/CHAR.
                             <1> ;  FOR MEDIUM RESOLUTION GRAPHICS, THE NUMBER MUST
                             <1> ;  BE DOUBLED.
                             <1> ; ENTRY -- NO REGISTERS,MEMORY LOCATION @CURSOR_POSN IS USED
                             <1> ; EXIT--
                             <1> ;  AX CONTAINS OFFSET INTO REGEN BUFFER
                             <1> ;----------------------------------------
                             <1> S26:
00002B27 0FB705[CE760100]    <1>     movzx   eax, word [CURSOR_POSN]       ; GET CURRENT CURSOR
                             <1> GRAPH_POSN:
00002B2E 53                  <1>     push    ebx                     ; SAVE REGISTER
00002B2F 0FB6D8              <1>     movzx   ebx, al                 ; SAVE A COPY OF CURRENT CURSOR
00002B32 A0[30680000]        <1>     mov     al, [CRT_COLS]          ; GET BYTES PER COLUMN
00002B37 F6E4                <1>     mul     ah                      ; MULTIPLY BY ROWS
                             <1>     ;shl    ax, 2                   ; MULTIPLY * 4 SINCE 4 ROWS/BYTE
                             <1>     ; 02/08/2022
00002B39 C1E002              <1>     shl     eax, 2
00002B3C 01D8                <1>     add     eax, ebx                ; DETERMINE OFFSET
00002B3E 5B                  <1>     pop     ebx                     ; RECOVER POINTER
00002B3F C3                  <1>     retn                            ; ALL DONE
                             <1>
                             <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 09/07/2016
                             <1> ; 01/07/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1> ;---------------------------------------------
                             <1> ; SET_COLOR
                             <1> ;   THIS ROUTINE WILL ESTABLISH THE BACKGROUND COLOR, THE OVERSCAN COLOR,
                             <1> ;   AND THE FOREGROUND COLOR SET FOR MEDIUM RESOLUTION GRAPHICS
                             <1> ; INPUT
                             <1> ;   (BH) HAS COLOR ID
                             <1> ;           IF BH=0, THE BACKGROUND COLOR VALUE IS SET
                             <1> ;                    FROM THE LOW BITS OF BL (0-31)
                             <1> ;           IF BH=1, THE PALETTE SELECTION IS MADE
                             <1> ;                   BASED ON THE LOW BIT OF BL:
                             <1> ;                           0 = GREEN, RED, YELLOW FOR COLORS 1,2,3
                             <1> ;                           1 = BLUE, CYAN, MAGENTA FOR COLORS 1,2,3
                             <1> ;   (BL) HAS THE COLOR VALUE TO BE USED
                             <1> ; OUTPUT
                             <1> ;   THE COLOR SELECTION IS UPDATED
                             <1> ;---------------------------------------------
                             <1> SET_COLOR:
00002B40 803D[2E680000]07    <1>         cmp     byte [CRT_MODE], 7      ; 09/07/2016
                             <1>     ;ja     VIDEO_RETURN            ; nothing to do for VGA modes
                             <1>     ; 03/08/2022
00002B47 7605                <1>     jna     short M21
00002B49 E91DF0FFFF          <1>     jmp     VIDEO_RETURN
                             <1> M21:
                             <1>     ;mov    dx, [ADDR_6845]               ; I/O PORT FOR PALETTE
                             <1>     ;mov    dx, 3D4h
                             <1>     ;add    dx, 5                   ; OVERSCAN PORT
00002B4E 66BAD903            <1>     mov     dx, 3D9h
00002B52 A0[31680000]        <1>     mov     al, [CRT_PALETTE]       ; GET THE CURRENT PALETTE VALUE
00002B57 08FF                <1>     or      bh, bh                  ; IS THIS COLOR 0?
00002B59 7512                <1>     jnz     short M20               ; OUTPUT COLOR 1
                             <1>
                             <1> ;-----      HANDLE COLOR 0 BY SETTING THE BACKGROUND COLOR
                             <1>
00002B5B 24E0                <1>     and     al, 0E0h                ; TURN OFF LOW 5 BITS OF CURRENT
00002B5D 80E31F              <1>     and     bl, 01Fh                ; TURN OFF HIGH 3 BITS OF INPUT VALUE
00002B60 08D8                <1>     or      al, bl                  ; PUT VALUE INTO REGISTER
                             <1> M19:                                ; OUTPUT THE PALETTE
00002B62 EE                  <1>     out     dx, al                  ; OUTPUT COLOR SELECTION TO 3D9 PORT
00002B63 A2[31680000]        <1>     mov     [CRT_PALETTE], al       ; SAVE THE COLOR VALUE
00002B68 E9FEEFFFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1> ;-----      HANDLE COLOR 1 BY SELECTING THE PALETTE TO BE USED
                             <1>
                             <1> M20:
00002B6D 24DF                <1>     and     al, 0DFH                ; TURN OFF PALETTE SELECT BIT
00002B6F D0EB                <1>     shr     bl, 1                   ; TEST THE LOW ORDER BIT OF BL
00002B71 73EF                <1>     jnc     short M19               ; ALREADY DONE
00002B73 0C20                <1>     or      al, 20H                 ; TURN ON PALETTE SELECT BIT
00002B75 EBEB                <1>     jmp     short M19               ; GO DO IT
                             <1>
                             <1> ; 03/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 09/07/2016
                             <1> ; 01/07/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1> ;---------------------------------------------
                             <1> ; READ DOT -- WRITE DOT
                             <1> ; THESE ROUTINES WILL WRITE A DOT, OR READ THE
                             <1> ;  DOT AT THE INDICATED LOCATION
                             <1> ; ENTRY --
                             <1> ;   DX = ROW (0-199)       (THE ACTUAL VALUE DEPENDS ON THE MODE)
                             <1> ;   CX = COLUMN ( 0-639) ( THE VALUES ARE NOT RANGE CHECKED )
                             <1> ;   AL = DOT VALUE TO WRITE (1,2 OR 4 BITS DEPENDING ON MODE,
                             <1> ;   REQUIRED FOR WRITE DOT ONLY, RIGHT JUSTIFIED)
                             <1> ;   BIT 7 OF AL = 1 INDICATES XOR THE VALUE INTO THE LOCATION
                             <1> ;   DS = DATA SEGMENT
                             <1> ;   ES = REGEN SEGMENT
                             <1> ;
                             <1> ; EXIT
                             <1> ;   AL = DOT VALUE READ, RIGHT JUSTIFIED, READ ONLY
                             <1> ;---------------------------------------------
                             <1>
                             <1> READ_DOT:
                             <1>     ; 09/07/2016
00002B77 8A25[2E680000]      <1>     mov     ah,  [CRT_MODE]
00002B7D 80FC07              <1>     cmp     ah, 7 ; 6!?
00002B80 760A                <1>     jna     short read_dot_cga
                             <1>
00002B82 E8B3030000          <1>     call    vga_read_pixel
                             <1>     ; al = pixel value
                             <1> read_dot_retn:      ; 03/08/2022
00002B87 E9E4EFFFFF          <1>     jmp     _video_return
                             <1>
                             <1> read_dot_cga:
                             <1>     ;je     VIDEO_RETURN ; 7
00002B8C 80FC04              <1>     cmp     ah, 4 ; graphics ?
                             <1>     ;jb     VIDEO_RETURN ; no, text mode, nothing to do
                             <1>     ; 03/08/2022
```

```
00002B8F 72F6                <1>     jb      short read_dot_retn
                             <1>
00002B91 E84F000000          <1>     call    R3                      ; DETERMINE BYTE POSITION OF DOT
00002B96 8A06                <1>     mov     al, [esi]               ; GET THE BYTE
00002B98 20E0                <1>     and     al, ah                  ; MASK OFF THE OTHER BITS IN THE BYTE
00002B9A D2E0                <1>     shl     al, cl                  ; LEFT JUSTIFY THE VALUE
00002B9C 88F1                <1>     mov     cl, dh                  ; GET NUMBER OF BITS IN RESULT
00002B9E D2C0                <1>     rol     al, cl                  ; RIGHT JUSTIFY THE RESULT
                             <1>     ;jmp    VIDEO_RETURN            ; RETURN FROM VIDEO I/O
00002BA0 0FB6C0              <1>     movzx   eax, al
00002BA3 E9C8EFFFFF          <1>     jmp     _video_return
                             <1>
                             <1> ; 03/08/2022
                             <1> ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 12/04/2021
                             <1> ; 09/07/2016
                             <1> ; 01/07/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1>
                             <1> WRITE_DOT:
                             <1>     ; 09/07/2016
00002BA8 8A25[2E680000]      <1>     mov     ah, [CRT_MODE]
00002BAE 80FC07              <1>     cmp     ah, 7 ; 6!?
00002BB1 760A                <1>     jna     short write_dot_cga
                             <1>
00002BB3 E8F2020000          <1>     call    vga_write_pixel
                             <1> write_dot_retn:    ; 03/08/2022
00002BB8 E9AEEFFFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1> write_dot_cga:
                             <1>     ;je     VIDEO_RETURN ; 7
00002BBD 80FC04              <1>     cmp     ah, 4 ; graphics ?
                             <1>     ;jb     VIDEO_RETURN ; no, text mode, nothing to do
                             <1>     ; 03/08/2022
00002BC0 72F6                <1>     jb      short write_dot_retn
                             <1>
                             <1>     ;;push  ax                      ; SAVE DOT VALUE
                             <1>     ;push   ax                      ; TWICE
                             <1>     ; 12/04/2021
00002BC2 50                  <1>     push    eax
00002BC3 E81D000000          <1>     call    R3                      ; DETERMINE BYTE POSITION OF THE DOT
00002BC8 D2E8                <1>     shr     al, cl                  ; SHIFT TO SET UP THE BITS FOR OUTPUT
00002BCA 20E0                <1>     and     al, ah                  ; STRIP OFF THE OTHER BITS
00002BCC 8A0E                <1>     mov     cl, [esi]               ; GET THE CURRENT BYTE
                             <1>     ;pop    bx                      ; RECOVER XOR FLAG
                             <1>     ; 12/04/2021
00002BCE 5B                  <1>     pop     ebx
00002BCF F6C380              <1>     test    bl, 80h                 ; IS IT ON
00002BD2 750D                <1>     jnz     short R2                ; YES, XOR THE DOT
00002BD4 F6D4                <1>     not     ah                      ; SET MASK TO REMOVE THE INDICATED BITS
00002BD6 20E1                <1>     and     cl, ah
00002BD8 08C8                <1>     or      al, cl                  ; OR IN THE NEW VALUE OF THOSE BITS
                             <1> R1:                                 ; FINISH_DOT
00002BDA 8806                <1>     mov     [esi], al               ; RESTORE THE BYTE IN MEMORY
                             <1>     ;;pop   AX
00002BDC E98AEFFFFF          <1>     jmp     VIDEO_RETURN            ; RETURN FROM VIDEO I/O
                             <1> R2:                                 ; XOR_DOT
00002BE1 30C8                <1>     xor     al, cl                  ; EXCLUSIVE OR THE DOTS
00002BE3 EBF5                <1>     jmp     short R1                ; FINISH UP THE WRITING
                             <1>
                             <1> ; 02/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 01/07/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> ; VIDEO1.ASM - 24/03/1985 (IBM PC-AT BIOS source code)
                             <1>
                             <1> ;---------------------------------------------
                             <1> ; THIS SUBROUTINE DETERMINES THE REGEN BYTE LOCATION OF THE
                             <1> ; INDICATED ROW COLUMN VALUE IN GRAPHICS MODE.
                             <1> ; ENTRY --
                             <1> ;  DX = ROW VALUE (0-199)
                             <1> ;  CX = COLUMN VALUE (0-639)
                             <1> ; EXIT --
                             <1> ;  SI = OFFSET INTO REGEN BUFFER FOR BYTE OF INTEREST
                             <1> ;  AH = MASK TO STRIP OFF THE BITS OF INTEREST
                             <1> ;  CL = BITS TO SHIFT TO RIGHT JUSTIFY THE MASK IN AH
                             <1> ;  DH = # BITS IN RESULT
                             <1> ;  BX = MODIFIED
                             <1> ;-----------------------------------------------
                             <1> R3:
                             <1>
                             <1> ;-----      DETERMINE 1ST BYTE IN INDICATED ROW BY MULTIPLYING ROW VALUE BY 40
                             <1> ;-----       ( LOW BIT OF ROW DETERMINES EVEN/ODD, 80 BYTES/ROW )
                             <1>
00002BE5 0FB7F0              <1>     movzx   esi, ax                 ; WILL SAVE AL AND AH DURING OPERATION
00002BE8 B028                <1>     mov     al, 40
00002BEA F6E2                <1>     mul     dl                      ; AX= ADDRESS OF START OF INDICATED ROW
00002BEC A808                <1>     test    al, 08H                 ; TEST FOR EVEN/ODD ROW CALCULATED
00002BEE 7404                <1>     JZ      short R4                ; JUMP IF EVEN ROW
00002BF0 6605D81F            <1>     add     ax, 2000h-40            ; OFFSET TO LOCATION OF ODD ROWS ADJUST
                             <1> R4:                                 ; EVEN_ROW
                             <1>     ;xchg   si, ax                  ; MOVE POINTER TO (SI) AND RECOVER (AX)
                             <1>     ; 02/08/2022
00002BF4 96                  <1>     xchg    esi, eax
00002BF5 81C600800B00        <1>     add     esi, 0B8000h
                             <1>     ;mov    dx, cx                  ; COLUMN VALUE TO DX
00002BFB 0FB7D1              <1>     movzx   edx, cx
                             <1>
                             <1> ;-----      DETERMINE GRAPHICS MODE CURRENTLY IN EFFECT
                             <1>
                             <1> ; SET UP THE REGISTERS ACCORDING TO THE MODE
                             <1> ; CH = MASK FOR LOW OF COLUMN ADDRESS ( 7/3 FOR HIGH/MED RES )
                             <1> ; CL = # OF ADDRESS BITS IN COLUMN VALUE ( 3/2 FOR H/M )
                             <1> ; BL = MASK TO SELECT BITS FROM POINTED BYTE ( 80H/C0H FOR H/M )
                             <1> ; BH = NUMBER OF VALID BITS IN POINTED BYTE ( 1/2 FOR H/M )
                             <1>
00002BFE 66BBC002            <1>     mov     bx, 2C0h
00002C02 66B90203            <1>     mov     cx, 302h                ; SET PARMS FOR MED RES
00002C06 803D[2E680000]06    <1>     cmp     byte [CRT_MODE], 6
00002C0D 7208                <1>     jc      short R5                ; HANDLE IF MED RES
00002C0F 66BB8001            <1>     mov     bx, 180h
00002C13 66B90307            <1>     mov     cx, 703h                ; SET PARMS FOR HIGH RES
                             <1>
                             <1> ;-----      DETERMINE BIT OFFSET IN BYTE FROM COLUMN MASK
                             <1> R5:
00002C17 20D5                <1>     and     ch, dl                  ; ADDRESS OF PEL WITHIN BYTE TO CH
                             <1>
                             <1> ;-----      DETERMINE BYTE OFFSET FOR THIS LOCATION IN COLUMN
                             <1>
                             <1>     ;shr    dx, cl                  ; SHIFT BY CORRECT AMOUNT
                             <1>     ;add    si, dx                  ; INCREMENT THE POINTER
                             <1>     ; 02/08/2022
00002C19 D3EA                <1>     shr     edx, cl
00002C1B 01D6                <1>     add     esi, edx
00002C1D 88FE                <1>     mov     dh, bh                  ; GET THE # OF BITS IN RESULT TO DH
                             <1>
```

```
                                <1> ;-----      MULTIPLY BH (VALID BITS IN BYTE) BY CH (BIT OFFSET)
                                <1>
00002C1F 28C9                   <1>     sub     cl, cl                  ; ZERO INTO STORAGE LOCATION
                                <1> R6:
00002C21 D0C8                   <1>     ror     al, 1                   ; LEFT JUSTIFY VALUE IN AL (FOR WRITE)
00002C23 00E9                   <1>     add     cl, ch                  ; ADD IN THE BIT OFFSET VALUE
00002C25 FECF                   <1>     dec     bh                      ; LOOP CONTROL
00002C27 75F8                   <1>     jnz     short R6                ; ON EXIT, CL HAS COUNT TO RESTORE BITS
00002C29 88DC                   <1>     mov     ah, bl                  ;  GET MASK TO AH
00002C2B D2EC                   <1>     shr     ah, cl                  ;  MOVE THE MASK TO CORRECT LOCATION
00002C2D C3                     <1>     retn                            ;  RETURN WITH EVERYTHING SET UP
                                <1>
                                <1> load_dac_palette:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 29/07/2016
                                <1>     ; 23/07/2016
                                <1>     ; 03/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; (set_mode_vga)
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'load_dac_palette'
                                <1>     ;
                                <1>     ; Oracle VirtualBox 5.0.24 VGABios Source Code
                                <1>     ; ('vgabios.c', 'vgatables.h', 'vgafonts.h', 'vgarom.asm')
                                <1>     ;
                                <1>     ; INPUT -> AH = DAC selection number (3, 2 or 1)
                                <1>     ; OUTPUT -> ECX = 0, AX = 0
                                <1>     ; (Modifed registers: EAX, ECX, EDX, ESI)
                                <1>     ;
00002C2E 66BAC803               <1>     mov     dx, 3C8h  ; VGAREG_DAC_WRITE_ADDRESS
00002C32 28C0                   <1>     sub     al, al ; 0
00002C34 EE                     <1>     out     dx, al ; 0 ; color index, always 0 at the beginning
                                <1>     ;inc    dx   ; 3C9h ; VGAREG_DAC_DATA
                                <1>     ; 02/08/2022
00002C35 FEC2                   <1>     inc     dl ; dx = 3C9h
                                <1>     ;mov    ecx, 256   ; always 256*3 values
                                <1>     ; 02/08/2022
00002C37 31C9                   <1>     xor     ecx, ecx
00002C39 FEC5                   <1>     inc     ch
                                <1>     ; ecx = 256
                                <1>
                                <1>     ;push   esi
00002C3B 88E0                   <1>     mov     al, ah
00002C3D B43F                   <1>     mov     ah, 3Fh ; 3Fh except DAC selection number 3
00002C3F 3C02                   <1>     cmp     al, 2
00002C41 7414                   <1>     je      short l_dac_p_2
00002C43 7719                   <1>     ja      short l_dac_p_3
00002C45 20C0                   <1>     and     al, al
00002C47 7507                   <1>     jnz     short l_dac_p_1
                                <1> l_dac_p_0:
00002C49 BE[00480100]           <1>     mov     esi, palette0
00002C4E EB15                   <1>     jmp     short l_dac_p_4
                                <1> l_dac_p_1:
00002C50 BE[C0480100]           <1>     mov     esi, palette1
00002C55 EB0E                   <1>     jmp     short l_dac_p_4
                                <1> l_dac_p_2:
00002C57 BE[80490100]           <1>     mov     esi, palette2
00002C5C EB07                   <1>     jmp     short l_dac_p_4
                                <1> l_dac_p_3:
00002C5E B4FF                   <1>     mov     ah, 0FFh ; dac registers
00002C60 BE[404A0100]           <1>     mov     esi, palette3
                                <1> l_dac_p_4:
00002C65 AC                     <1>     lodsb
00002C66 EE                     <1>     out     dx, al  ; Red
00002C67 AC                     <1>     lodsb
00002C68 EE                     <1>     out     dx, al  ; Green
00002C69 AC                     <1>     lodsb
00002C6A EE                     <1>     out     dx, al  ; Blue
00002C6B 20E4                   <1>     and     ah, ah
00002C6D 7405                   <1>     jz      short l_dac_p_5
00002C6F FECC                   <1>     dec     ah
00002C71 E2F2                   <1>     loop    l_dac_p_4
                                <1>     ;pop    esi
00002C73 C3                     <1>     retn
                                <1> l_dac_p_5:
                                <1>     ; 29/07/2016
00002C74 FEC9                   <1>     dec     cl
00002C76 7407                   <1>     jz      short l_dac_p_7
                                <1>     ;
00002C78 28C0                   <1>     sub     al, al ; 0
                                <1> l_dac_p_6:
00002C7A EE                     <1>     out     dx, al ; outb(VGAREG_DAC_DATA,0);
00002C7B EE                     <1>     out     dx, al
00002C7C EE                     <1>     out     dx, al
00002C7D E2FB                   <1>     loop    l_dac_p_6
                                <1> l_dac_p_7:
                                <1>     ;pop    esi
00002C7F C3                     <1>     retn
                                <1>
                                <1> gray_scale_summing:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 12/04/2021
                                <1>     ; 03/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; (set_mode_vga)
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_perform_gray_scale_summing'
                                <1>     ;
                                <1>     ; Oracle VirtualBox 5.0.24 VGABios Source Code
                                <1>     ; ('vgabios.c', 'vgatables.h', 'vgafonts.h', 'vgarom.asm')
                                <1>     ;
                                <1>
                                <1>     ; INPUT -> EBX = Start address (color index <= 255)
                                <1>     ;         ECX = Count (<= 256)
                                <1>     ; OUTPUT -> (E)CX = 0
                                <1>     ; (Modifed registers: EAX, ECX, EDX, EBX)
                                <1>
00002C80 66BADA03               <1>     mov     dx, 3DAh ; VGAREG_ACTL_RESET
00002C84 EC                     <1>     in      al, dx
00002C85 30C0                   <1>     xor     al, al ; 0
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 03/08/2022
00002C87 B2C0                   <1>     mov     dl, 0C0h
00002C89 EE                     <1>     out     dx, al  ; clear bit 5
                                <1>                     ; (while loading palette registers)
                                <1>     ; set read address and switch to read mode
                                <1> g_s_s_1:
                                <1>     ;mov    dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
                                <1>     ; 03/08/2022
00002C8A B2C7                   <1>     mov     dl, 0C7h
00002C8C 88D8                   <1>     mov     al, bl
00002C8E EE                     <1>     out     dx, al
```

```
                                <1>     ; get 6-bit wide RGB data values
                                <1>     ; intensity = (0.3*Red)+(0.59*Green)+(0.11*Blue)
                                <1>     ; i = ( ( 77*r + 151*g + 28*b ) + 0x80 ) >> 8;
                                <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
                                <1>     ; 03/08/2022
00002C8F B2C9                   <1>     mov     dl, 0C9h
00002C91 EC                     <1>     in      al, dx ; red
00002C92 B44D                   <1>     mov     ah, 77 ; 0.3* Red
00002C94 F6E4                   <1>     mul     ah
                                <1>     ;push   ax
                                <1>     ; 12/04/2021
00002C96 50                     <1>     push    eax
00002C97 EC                     <1>     in      al, dx ; green
00002C98 B497                   <1>     mov     ah, 151  ; 0.59 * Green
00002C9A F6E4                   <1>     mul     ah
                                <1>     ;push   ax
                                <1>     ; 12/04/2021
00002C9C 50                     <1>     push    eax
00002C9D EC                     <1>     in      al, dx ; blue
00002C9E B41C                   <1>     mov     ah, 28 ; 0.11 * Blue
00002CA0 F6E4                   <1>     mul     ah
                                <1>     ;pop    dx
                                <1>     ; 12/04/2021
00002CA2 5A                     <1>     pop     edx
00002CA3 6601D0                 <1>     add     ax, dx
                                <1>     ;pop    dx
                                <1>     ; 12/04/2021
00002CA6 5A                     <1>     pop     edx
00002CA7 6601D0                 <1>     add     ax, dx
00002CAA 66058000               <1>     add     ax, 80h
00002CAE B03F                   <1>     mov     al, 3Fh
00002CB0 38C4                   <1>     cmp     ah, al  ; if(i>0x3f)i=0x3f
00002CB2 7602                   <1>     jna     short g_s_s_2
00002CB4 88C4                   <1>     mov     ah, al
                                <1> g_s_s_2:
                                <1>     ;mov    dx, 3C8h  ; VGAREG_DAC_WRITE_ADDRESS
                                <1>     ; 03/08/2022
00002CB6 B2C8                   <1>     mov     dl, 0C8h
00002CB8 88D8                   <1>     mov     al, bl ; color index
00002CBA EE                     <1>     out     dx, al
00002CBB 88E0                   <1>     mov     al, ah ; intensity
                                <1>     ;inc    dx ; 3C9h ; VGAREG_DAC_DATA
                                <1>     ; 03/08/2022
00002CBD FEC2                   <1>     inc     dl
00002CBF EE                     <1>     out     dx, al ; R (R=G=B)
00002CC0 88E0                   <1>     mov     al, ah ; intensity
00002CC2 EE                     <1>     out     dx, al ; G (R=G=B)
00002CC3 88E0                   <1>     mov     al, ah ; intensity
00002CC5 EE                     <1>     out     dx, al ; B (R=G=B)
                                <1>     ;dec    cx
                                <1>     ; 03/08/2022
00002CC6 49                     <1>     dec     ecx
00002CC7 7404                   <1>     jz      short g_s_s_3
00002CC9 FEC3                   <1>     inc     bl    ; next color index value
00002CCB EBBD                   <1>     jmp     short g_s_s_1
                                <1> g_s_s_3:
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 03/08/2022
00002CCD B2DA                   <1>     mov     dl, 0DAh
00002CCF EC                     <1>     in      al, dx
00002CD0 B020                   <1>     mov     al, 20h
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 03/08/2022
00002CD2 B2C0                   <1>     mov     dl, 0C0h
00002CD4 EE                     <1>     out     dx, al ; 20h -> set bit 5
                                <1>                     ; (after loading palette regs)
00002CD5 C3                     <1>     retn
                                <1>
                                <1> vga_write_char_attr:
                                <1> vga_write_char_only:
                                <1>     ; 08/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_write_char_attr'
                                <1>     ; 'biosfn_write_char_only'
                                <1>
                                <1>     ; INPUT ->
                                <1>     ; [CRT_MODE] = current video mode (>7)
                                <1>     ; CX = Count of characters to write
                                <1>     ; AL = Character to write
                                <1>     ; BL = Color of character
                                <1>     ; OUTPUT ->
                                <1>     ; Regen buffer updated
                                <1>
00002CD6 8A25[2E680000]         <1>     mov     ah, [CRT_MODE]
00002CDC 668B15[CE760100]       <1>     mov     dx, [CURSOR_POSN] ; cursor pos for page 0
                                <1>
00002CE3 BE[4A680000]           <1>     mov     esi, vga_modes
00002CE8 89F7                   <1>     mov     edi, esi
00002CEA 83C710                 <1>     add     edi, vga_mode_count
                                <1> vga_wca_0:
00002CED AC                     <1>     lodsb
00002CEE 38E0                   <1>     cmp     al, ah ; [CRT_MODE]
00002CF0 7405                   <1>     je      short vga_wca_2
00002CF2 39FE                   <1>     cmp     esi, edi
00002CF4 72F7                   <1>     jb      short vga_wca_0
                                <1> vga_wca_1:
00002CF6 C3                     <1>     retn    ; nothing to do
                                <1> vga_wca_2:
00002CF7 83C64F                 <1>     add     esi, vga_memmodel - (vga_modes + 1)
                                <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
                                <1>
                                <1>     ; biosfn_write_char_attr (car,page,attr,count)
                                <1>     ; AL = car, page = 0, BL = attr, CX = count
00002CFA 803E04                 <1>     cmp     byte [esi], PLANAR4
00002CFD 741D                   <1>     je      short vga_wca_planar
00002CFF 803E03                 <1>     cmp     byte [esi], PLANAR1
00002D02 7418                   <1>     je      short vga_wca_planar
                                <1> vga_wca_linear8:
                                <1>     ; while((count-->0) && (xcurs<nbcols))
                                <1>     ; CX = count
00002D04 6621C9                 <1>     and     cx, cx
00002D07 74ED                   <1>     jz      short vga_wca_1
00002D09 3A15[30680000]         <1>     cmp     dl, [CRT_COLS]
00002D0F 73E5                   <1>     jnb     short vga_wca_1
                                <1>     ; write_gfx_char_lin(car,attr,xcurs,ycurs,nbcols);
                                <1>     ; AL = car, BL = attr, DL = xcurs, DH = ycurs,
                                <1>     ; [CRT_COLS] = nbcols
00002D11 E81E000000             <1>     call    write_gfx_char_lin
00002D16 6649                   <1>     dec     cx ; count
00002D18 FEC2                   <1>     inc     dl ; xcurs
00002D1A EBE8                   <1>     jmp     short vga_wca_linear8
```

```
                                <1> vga_wca_planar:
                                <1>     ; while((count-->0) && (xcurs<nbcols))
                                <1>     ; CX = count
00002D1C 6621C9                 <1>     and     cx, cx
00002D1F 74D5                   <1>     jz      short vga_wca_1
00002D21 3A15[30680000]         <1>     cmp     dl, [CRT_COLS]
00002D27 73CD                   <1>     jnb     short vga_wca_1
                                <1>     ; write_gfx_char_pl4(car,attr,xcurs,ycurs,nbcols,cheight);
                                <1>     ; AL = car, BL = attr, DL = xcurs, DH = ycurs,
                                <1>     ; [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight
00002D29 E8A7000000             <1>     call    write_gfx_char_pl4
00002D2E 6649                   <1>     dec     cx ; count
00002D30 FEC2                   <1>     inc     dl ; xcurs
00002D32 EBE8                   <1>     jmp     short vga_wca_planar
                                <1>
                                <1> write_gfx_char_lin:
                                <1>     ; 02/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 08/01/2021
                                <1>     ; 05/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; 08/08/2016
                                <1>     ; 31/07/2016
                                <1>     ; 08/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'write_gfx_char_lin'
                                <1>
                                <1>     ; write_gfx_char_lin(car,attr,xcurs,ycurs,nbcols)
                                <1>     ; INPUT ->
                                <1>     ; AL = car, BL = attr, DL = xcurs, DH = ycurs,
                                <1>     ; [CRT_COLS] = nbcols
                                <1>     ; OUTPUT ->
                                <1>     ; Regen buffer updated
                                <1>
00002D34 51                     <1>     push    ecx
00002D35 53                     <1>     push    ebx
00002D36 52                     <1>     push    edx
00002D37 50                     <1>     push    eax
                                <1>     ; addr=xcurs*8+ycurs*nbcols*64;
                                <1>     ; 08/08/2016
00002D38 0FB6F0                 <1>     movzx   esi, al ; car
                                <1>     ; 08/01/2021
                                <1>     ;movzx  eax, dh ; ycurs
                                <1>     ;mov    ah, [CRT_COLS] ; nbcols
                                <1>     ;mul    ah
00002D3B A0[30680000]           <1>     mov     al, [CRT_COLS]
00002D40 F6E6                   <1>     mul     dh
                                <1>     ;shl    ax, 6 ; * 64
                                <1>     ;shl    ax, 3 ; 8 * ycurs * [CRT_COLS]
                                <1>     ; 02/08/2022
00002D42 C1E003                 <1>     shl     eax, 3
                                <1>     ;sub    dh, dh
                                <1>     ;shl    dx, 3 ; xcurs * 8
                                <1>     ;movzx  edi, dx
00002D45 BF00000A00             <1>     mov     edi, 0A0000h
00002D4A 30F6                   <1>     xor     dh, dh
00002D4C 6689D7                 <1>     mov     di, dx
                                <1>     ;movzx  edi, dl
00002D4F 66C1E703               <1>     shl     di, 3 ; xcurs * 8
                                <1>     ;xor    dh, dh
00002D53 8A15[32680000]         <1>     mov     dl, [CHAR_HEIGHT]
00002D59 66F7E2                 <1>     mul     dx
                                <1>     ; eax = ycurs*nbcols*8*[CHAR_HEIGHT]
                                <1>     ;add    edi, eax ; addr
                                <1>     ;add    edi, 0A0000h
00002D5C 6601C7                 <1>     add     di, ax
                                <1>     ;shl    si, 3 ; car * 8
00002D5F 30E4                   <1>     xor     ah, ah
00002D61 A0[32680000]           <1>     mov     al, [CHAR_HEIGHT]
00002D66 66F7E6                 <1>     mul     si
00002D69 6689C6                 <1>     mov     si, ax
                                <1>     ;; esi = src = car * 8
                                <1>     ; esi = src = car * [CHAR_HEIGHT]
                                <1>     ; i = 0
                                <1>     ;add    esi, vgafont8 ; fdata [src+i]
                                <1>     ; 08/08/2016
00002D6C A1[5E830100]           <1>     mov     eax, [VGA_INT43H]
00002D71 09C0                   <1>     or      eax, eax ; 0 ?
00002D73 743E                   <1>     jz      short wfxl_7 ; yes, default font
                                <1>     ;cmp    eax, vgafont16
                                <1>         ;je short wgfxl_0
                                <1>     ;cmp    eax, vgafont14
                                <1>     ;je     short wgfxl_0
                                <1>     ;cmp    eax, vgafont8
                                <1>     ;je     short wgfxl_0
                                <1>     ;; 05/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ;; user font
                                <1>     ;mov    eax, VGAFONTUSR ; 8x16 or 8x8 or 8x14 font
                                <1>     ;                       ; (256 characters)
                                <1> wgfxl_0:
00002D75 01C6                   <1>     add     esi, eax
                                <1> wgfxl_1:
00002D77 28FF                   <1>     sub     bh, bh ; i = 0
                                <1> wgfxl_2:
                                <1>     ; for(i=0;i<8;i++)
00002D79 57                     <1>     push    edi ; addr
00002D7A 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS] ; nbcols
00002D81 F6E7                   <1>     mul     bh ; nbcols*i
                                <1>     ;shl    ax, 3 ; i*nbcols*8
                                <1>     ; 02/08/2022
00002D83 C1E003                 <1>     shl     eax, 3
                                <1>     ; dest=addr+i*nbcols*8;
00002D86 01C7                   <1>     add     edi, eax ; dest + j ; j = 0
00002D88 B180                   <1>     mov     cl, 80h ; mask = 0x80;
                                <1>     ; esi = fdata + src + i
                                <1>     ; for(j=0;j<8;j++)
00002D8A 29D2                   <1>     sub     edx, edx ; j = 0
                                <1> wgfxl_3:
00002D8C 8A06                   <1>     mov     al, [esi] ; al = fdata[src+i]
00002D8E 20C8                   <1>     and     al, cl ; if (fdata[src+i] & mask)
00002D90 7402                   <1>     jz      short wgfxl_4  ; data = 0, zf = 1
00002D92 88D8                   <1>     mov     al, bl ; data = attr;
                                <1> wgfxl_4:
                                <1>     ; write_byte(0xa000,dest+j,data);
00002D94 AA                     <1>     stosb  ; dest + j (+ 0A0000h)
                                <1>     ;inc    dl ; j++
                                <1>     ;cmp    dl, 8
00002D95 80FA07                 <1>     cmp     dl, 7
00002D98 720E                   <1>     jb      short wgfxl_5
00002D9A 5F                     <1>     pop     edi
                                <1>     ; 08/08/2016
                                <1>     ;cmp    bh, 7
                                <1>     ;jnb    short wgfxl_6
```

```
00002D9B FEC7                <1>     inc     bh ; i++
00002D9D 3A3D[32680000]      <1>     cmp     bh, [CHAR_HEIGHT]
00002DA3 7309                <1>     jnb     short wgfxl_6
00002DA5 46                  <1>     inc     esi
00002DA6 EBD1                <1>     jmp     short wgfxl_2
                             <1> wgfxl_5:
00002DA8 D0E9                <1>     shr     cl, 1 ; mask >>= 1;
00002DAA FEC2                <1>     inc     dl ; j++
00002DAC EBDE                <1>         jmp     short wgfxl_3
                             <1> wgfxl_6:
00002DAE 58                  <1>     pop     eax
00002DAF 5A                  <1>     pop     edx
00002DB0 5B                  <1>     pop     ebx
00002DB1 59                  <1>     pop     ecx
00002DB2 C3                  <1>     retn
                             <1> wfxl_7:
                             <1>     ; 08/01/2021
                             <1>     ; 05/01/2021
                             <1>     ; Default font (8x8 or 8x14 or 8x16)
00002DB3 A0[32680000]        <1>     mov     al, [CHAR_HEIGHT]
00002DB8 3C08                <1>     cmp     al, 8
00002DBA 7507                <1>     jne     short wfxl_8
00002DBC B8[404D0100]        <1>     mov     eax, vgafont8
00002DC1 EBB2                <1>     jmp     short wgfxl_0
                             <1> wfxl_8:
00002DC3 3C0E                <1>     cmp     al, 14
00002DC5 7507                <1>     jne     short wfxl_9
00002DC7 B8[40550100]        <1>     mov     eax, vgafont14
00002DCC EBA7                <1>     jmp     short wgfxl_0
                             <1> wfxl_9:
00002DCE B8[40630100]        <1>     mov     eax, vgafont16
00002DD3 EBA0                <1>     jmp     short wgfxl_0
                             <1>
                             <1> write_gfx_char_pl4:
                             <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 08/08/2016
                             <1>     ; 08/07/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', 'write_gfx_char_pl4'
                             <1>
                             <1>     ; write_gfx_char_pl4(car,attr,xcurs,ycurs,nbcols,cheight)
                             <1>     ; INPUT ->
                             <1>     ; AL = car, BL = attr, DL = xcurs, DH = ycurs,
                             <1>     ; [CRT_COLS] = nbcols, [CHAR_HEIGHT] = cheight
                             <1>     ; OUTPUT ->
                             <1>     ; Regen buffer updated
                             <1>
00002DD5 51                  <1>     push    ecx
00002DD6 53                  <1>     push    ebx
00002DD7 52                  <1>     push    edx
00002DD8 50                  <1>     push    eax
                             <1> wgfxpl_f0:
                             <1>     ; switch(cheight)
00002DD9 8A25[32680000]      <1>     mov     ah, [CHAR_HEIGHT]
00002DDF 80FC10              <1>     cmp     ah, 16 ; case 16:
00002DE2 7507                <1>     jne     short wgfxpl_f1
                             <1>     ; fdata = &vgafont16;
00002DE4 BE[40630100]        <1>     mov     esi, vgafont16
00002DE9 EB13                <1>     jmp     short wgfxpl_f3
                             <1> wgfxpl_f1:
00002DEB 80FC0E              <1>     cmp     ah, 14 ; case 14:
00002DEE 7507                <1>     jne     short wgfxpl_f2
00002DF0 BE[40550100]        <1>     mov     esi, vgafont14
00002DF5 EB07                <1>     jmp     short wgfxpl_f3
                             <1> wgfxpl_f2:
                             <1>     ; default:
                             <1>     ;  fdata = &vgafont8;
00002DF7 BE[404D0100]        <1>     mov     esi, vgafont8
00002DFC B408                <1>     mov     ah, 8
                             <1> wgfxpl_f3:
                             <1>     ; al = car
00002DFE F6E4                <1>     mul     ah ; ah = cheight
00002E00 25FFFF0000          <1>     and     eax, 0FFFFh ; car * cheight
                             <1>     ; src = car * cheight;
00002E05 01C6                <1>     add     esi, eax ; esi = fdata[src+i]
                             <1>     ; addr=xcurs*8+ycurs*nbcols*64;
00002E07 88F0                <1>     mov     al, dh ; ycurs
00002E09 8A25[30680000]      <1>     mov     ah, [CRT_COLS] ; nbcols
00002E0F F6E4                <1>     mul     ah
                             <1>     ; 08/08/2016
                             <1>     ;shl    ax, 6 ; * 64
                             <1>     ;shl    ax, 3 ; * 8
                             <1>     ; 02/08/2022
00002E11 C1E003              <1>     shl     eax, 3
                             <1>     ;sub    dh, dh ; 0
                             <1>     ;shl    dx, 3 ; xcurs * 8
                             <1>     ;movzx  edi, dx
00002E14 0FB6FA              <1>     movzx   edi, dl
                             <1>     ;shl    di, 3 ; xcurs * 8
                             <1>     ; 02/08/2022
00002E17 C1E703              <1>     shl     edi, 3
00002E1A 30F6                <1>     xor     dh, dh
00002E1C 8A15[32680000]      <1>     mov     dl, [CHAR_HEIGHT]
00002E22 66F7E2              <1>     mul     dx
                             <1>     ; eax = ycurs*nbcols*8*[CHAR_HEIGHT]
00002E25 01C7                <1>     add     edi, eax ; addr
00002E27 81C700000A00        <1>     add     edi, 0A0000h
                             <1>     ;
                             <1>     ; outw(VGAREG_SEQU_ADDRESS, 0x0f02);
                             <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0205);
00002E2D 66BAC403            <1>     mov     dx, 3C4h ; VGAREG_SEQU_ADDRESS
00002E31 66B8020F            <1>     mov     ax, 0F02h
00002E35 66EF                <1>     out     dx, ax
00002E37 66BACE03            <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002E3B 66B80502            <1>     mov     ax, 0205h
00002E3F 66EF                <1>     out     dx, ax
                             <1>     ;
00002E41 66BACE03            <1>     mov     dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002E45 F6C380              <1>     test    bl, 80h ; if(attr&0x80)
00002E48 7406                <1>     jz      short wgfxpl_f4 ; else
                             <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x1803);
00002E4A 66B80318            <1>     mov     ax, 1803h
00002E4E EB04                <1>     jmp     short wgfxpl_f5
                             <1> wgfxpl_f4:
                             <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0003);
00002E50 66B80300            <1>     mov     ax, 0003h
                             <1> wgfxpl_f5:
00002E54 66EF                <1>     out     dx, ax
                             <1>     ;
00002E56 28FF                <1>     sub     bh, bh ; i = 0
                             <1> wgfxpl_0:
```

```
                                <1>     ; for(i=0;i<cheight;i++)
00002E58 57                     <1>     push    edi ; addr
00002E59 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS] ; nbcols
00002E60 F6E7                   <1>     mul     bh ; nbcols*i
                                <1>     ; dest=addr+i*nbcols
00002E62 01C7                   <1>     add     edi, eax ; dest
00002E64 B580                   <1>     mov     ch, 80h ; mask = 0x80;
                                <1>     ; for(j=0;j<8;j++)
00002E66 28C9                   <1>     sub     cl, cl ; j = 0
                                <1> wgfxpl_1:
00002E68 D2ED                   <1>     shr     ch, cl ; mask=0x80>>j;
                                <1>     ;
                                <1>     ; outw(VGAREG_GRDC_ADDRESS, (mask << 8) | 0x08);
                                <1>             ; read_byte(0xa000,dest);
                                <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002E6A 88EC                   <1>     mov     ah, ch
00002E6C B008                   <1>     mov     al, 8
00002E6E 66EF                   <1>     out     dx, ax
00002E70 8A07                   <1>     mov     al, [edi] ; ? (io delay?)
                                <1>     ;
00002E72 28C0                   <1>     sub     al, al ; attr = 0
                                <1>     ; if (fdata[src+i] & mask)
00002E74 842E                   <1>     test    byte [esi], ch
00002E76 7404                   <1>     jz      short wgfxpl_2  ; zf = 1
                                <1>     ; write_byte(0xa000,dest,attr&0x0f);
00002E78 88D8                   <1>     mov     al, bl ; attr;
00002E7A 240F                   <1>     and     al, 0Fh ; attr&0x0f
                                <1> wgfxpl_2:
                                <1>     ; write_byte(0xa000,dest,0x00);
00002E7C 8807                   <1>     mov     [edi], al ; dest (+ 0A0000h)
00002E7E FEC1                   <1>     inc     cl ; j++
00002E80 80F908                 <1>     cmp     cl, 8
00002E83 72E3                   <1>     jb      short wgfxpl_1
00002E85 5F                     <1>     pop     edi
                                <1>     ; 08/08/2016
                                <1>     ;cmp    bh, 7
                                <1>     ;jnb    short wgfxpl_3
00002E86 FEC7                   <1>     inc     bh ; j++
00002E88 3A3D[32680000]         <1>     cmp     bh, [CHAR_HEIGHT]
00002E8E 7303                   <1>     jnb     short wgfxpl_3
00002E90 46                     <1>     inc     esi
00002E91 EBC5                   <1>     jmp     short wgfxpl_0
                                <1> wgfxpl_3:
                                <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002E93 66B808FF               <1>     mov     ax, 0FF08h
00002E97 66EF                   <1>     out     dx, ax
00002E99 66B80500               <1>     mov     ax, 0005h
00002E9D 66EF                   <1>     out     dx, ax
00002E9F 66B80300               <1>     mov     ax, 0003h
00002EA3 66EF                   <1>     out     dx, ax
                                <1>     ;
00002EA5 58                     <1>     pop     eax
00002EA6 5A                     <1>     pop     edx
00002EA7 5B                     <1>     pop     ebx
00002EA8 59                     <1>     pop     ecx
00002EA9 C3                     <1>     retn
                                <1>
                                <1> vga_write_pixel:
                                <1>     ; 02/08/2022 (TRDOS 386 Kerbel v2.0.5)
                                <1>     ; 09/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_write_pixel'
                                <1>
                                <1>     ; INPUT ->
                                <1>     ;       DX = row (0-239)
                                <1>     ;       CX = column (0-799)
                                <1>     ;       AL = pixel value
                                <1>     ;       (AH = [CRT_MODE])
                                <1>     ; OUTPUT ->
                                <1>     ;       none
                                <1>
00002EAA 88C3                   <1>     mov     bl, al ; pixel value
                                <1>     ;mov    ah, [CRT_MODE]
00002EAC BE[4A680000]           <1>     mov     esi, vga_modes
00002EB1 89F7                   <1>     mov     edi, esi
00002EB3 83C710                 <1>     add     edi, vga_mode_count
                                <1> vga_wp_0:
00002EB6 AC                     <1>     lodsb
00002EB7 38E0                   <1>     cmp     al, ah ; [CRT_MODE]
00002EB9 7405                   <1>     je      short vga_wp_1
00002EBB 39FE                   <1>     cmp     esi, edi
00002EBD 72F7                   <1>     jb      short vga_wp_0
00002EBF C3                     <1>     retn    ; nothing to do
                                <1> vga_wp_1:
00002EC0 83C64F                 <1>     add     esi, vga_memmodel - (vga_modes + 1)
                                <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
00002EC3 BF00000A00             <1>     mov     edi, 0A0000h
                                <1>     ;
00002EC8 803E04                 <1>     cmp     byte [esi], PLANAR4
00002ECB 741C                   <1>     je      short vga_wp_planar
00002ECD 803E03                 <1>     cmp     byte [esi], PLANAR1
00002ED0 7417                   <1>     je      short vga_wp_planar
                                <1> vga_wp_linear8:
                                <1>     ; addr=CX+DX*(read_word(BIOSMEM_SEG,BIOSMEM_NB_COLS)*8);
00002ED2 0FB605[30680000]       <1>     movzx   eax, byte [CRT_COLS] ; = [VGA_COLS] ; nbcols
                                <1>             ;shl    ax, 3 ; * 8
                                <1>     ; 02/08/2022
00002ED9 C1E003                 <1>     shl     eax, 3
00002EDC 66F7E2                 <1>     mul     dx
00002EDF 50                     <1>     push    eax
                                <1>     ;mov    edi, 0A0000h
00002EE0 6601CF                 <1>     add     di, cx
00002EE3 58                     <1>     pop     eax
00002EE4 01C7                   <1>     add     edi, eax ; addr
                                <1>     ; write_byte(0xa000,addr,AL);
00002EE6 881F                   <1>     mov     [edi], bl
00002EE8 C3                     <1>     retn
                                <1> vga_wp_planar:
                                <1>     ; addr = CX/8+DX*read_word(BIOSMEM_SEG,BIOSMEM_NB_COLS);
00002EE9 0FB7C1                 <1>     movzx   eax, cx
00002EEC 66C1E803               <1>     shr     ax, 3 ; CX/8
00002EF0 50                     <1>     push    eax
00002EF1 28E4                   <1>     sub     ah, ah ; 0
00002EF3 A0[30680000]           <1>     mov     al, [CRT_COLS] ; = [VGA_COLS] ; nbcols
00002EF8 66F7E2                 <1>     mul     dx
                                <1>     ;mov    edi, 0A0000h
00002EFB 6601C7                 <1>         add     di, ax
00002EFE 58                     <1>     pop     eax
00002EFF 01C7                   <1>     add     edi, eax ; addr
00002F01 80E107                 <1>     and     cl, 7
00002F04 B580                   <1>     mov     ch, 80h ; mask
```

```
00002F06 D2ED                <1>     shr    ch, cl  ; mask = 0x80 >> (CX & 0x07);
                             <1>
                             <1>     ; outw(VGAREG_GRDC_ADDRESS, (mask << 8) | 0x08);
00002F08 66BACE03            <1>     mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002F0C 88EC                <1>     mov    ah, ch
00002F0E B008                <1>     mov    al, 8
00002F10 66EF                <1>     out    dx, ax
                             <1>     ; outw(VGAREG_GRDC_ADDRESS, 0x0205);
00002F12 66B80502            <1>     mov    ax, 0205h
00002F16 66EF                <1>     out    dx, ax
                             <1>     ; data = read_byte(0xa000,addr);
00002F18 8A07                <1>     mov    al, [edi] ; (delay?)
                             <1>     ; if (AL & 0x80)
                             <1>     ; {
                             <1>     ;  outw(VGAREG_GRDC_ADDRESS, 0x1803);
                             <1>     ; }
00002F1A F6C380              <1>     test   bl, 80h
00002F1D 7406                <1>     jz     short vga_wp_2
00002F1F 66B80318            <1>     mov    ax, 1803h
00002F23 66EF                <1>     out    dx, ax
                             <1> vga_wp_2:
                             <1>     ; write_byte(0xa000,addr,AL);
00002F25 881F                <1>     mov    [edi], bl
                             <1>     ;
                             <1>     ;mov   dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002F27 66B808FF            <1>     mov    ax, 0FF08h
00002F2B 66EF                <1>     out    dx, ax
00002F2D 66B80500            <1>     mov    ax, 0005h
00002F31 66EF                <1>     out    dx, ax
00002F33 66B80300            <1>     mov    ax, 0003h
00002F37 66EF                <1>     out    dx, ax
                             <1>     ;
00002F39 C3                  <1>     retn
                             <1>
                             <1> vga_read_pixel:
                             <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 09/07/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', 'biosfn_read_pixel'
                             <1>
                             <1>     ; INPUT ->
                             <1>     ;      DX = row (0-239)
                             <1>     ;      CX = column (0-799)
                             <1>     ;      (AH = [CRT_MODE])
                             <1>     ; OUTPUT ->
                             <1>     ;      AL = pixel value
                             <1>
                             <1>     ;mov   ah, [CRT_MODE]
00002F3A BE[4A680000]        <1>     mov    esi, vga_modes
00002F3F 89F7                <1>     mov    edi, esi
00002F41 83C710              <1>     add    edi, vga_mode_count
                             <1> vga_rp_0:
00002F44 AC                  <1>     lodsb
00002F45 38E0                <1>     cmp    al, ah ; [CRT_MODE]
00002F47 7405                <1>     je     short vga_rp_1
00002F49 39FE                <1>     cmp    esi, edi
00002F4B 72F7                <1>     jb     short vga_rp_0
00002F4D C3                  <1>     retn   ; nothing to do
                             <1> vga_rp_1:
00002F4E 83C64F              <1>     add    esi, vga_memmodel - (vga_modes + 1)
                             <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
00002F51 BF00000A00          <1>     mov    edi, 0A0000h
                             <1>     ;
00002F56 803E04              <1>     cmp    byte [esi], PLANAR4
00002F59 741C                <1>     je     short vga_rp_planar
00002F5B 803E03              <1>     cmp    byte [esi], PLANAR1
00002F5E 7417                <1>     je     short vga_rp_planar
                             <1> vga_rp_linear8:
                             <1>     ; addr=CX+DX*(read_word(BIOSMEM_SEG,BIOSMEM_NB_COLS)*8);
00002F60 0FB605[30680000]    <1>     movzx  eax, byte [CRT_COLS] ; = [VGA_COLS] ; nbcols
                             <1>            ;shl   ax, 3 ; * 8
                             <1>     ; 02/08/2022
00002F67 C1E003              <1>     shl    eax, 3
00002F6A 66F7E2              <1>     mul    dx
00002F6D 50                  <1>     push   eax
                             <1>     ;mov   edi, 0A0000h
00002F6E 6601CF              <1>     add    di, cx
00002F71 58                  <1>     pop    eax
00002F72 01C7                <1>     add    edi, eax ; addr
                             <1>     ; attr=read_byte(0xa000,addr);
00002F74 8A07                <1>     mov    al, [edi] ; pixel value
00002F76 C3                  <1>     retn
                             <1> vga_rp_planar:
                             <1>     ; addr = CX/8+DX*read_word(BIOSMEM_SEG,BIOSMEM_NB_COLS);
00002F77 0FB7C1              <1>     movzx  eax, cx
00002F7A 66C1E803            <1>     shr    ax, 3 ; CX/8
00002F7E 50                  <1>     push   eax
00002F7F 28E4                <1>     sub    ah, ah ; 0
00002F81 A0[30680000]        <1>     mov    al, [CRT_COLS] ; = [VGA_COLS] ; nbcols
00002F86 66F7E2              <1>     mul    dx
                             <1>     ;mov   edi, 0A0000h
00002F89 6601C7              <1>         add     di, ax
00002F8C 58                  <1>     pop    eax
00002F8D 01C7                <1>     add    edi, eax ; addr
00002F8F 80E107              <1>     and    cl, 7
00002F92 B580                <1>     mov    ch, 80h ; mask
00002F94 D2ED                <1>     shr    ch, cl  ; mask = 0x80 >> (CX & 0x07);
                             <1>     ; attr = 0x00;
00002F96 30DB                <1>     xor    bl, bl ; attr = bl = 0,
00002F98 30C9                <1>     xor    cl, cl ; i = cl = 0
                             <1>     ; for(i=0;i<4;i++)
                             <1>            ; {
                             <1>            ;  outw(VGAREG_GRDC_ADDRESS, (i << 8) | 0x04);
                             <1>            ;  data = read_byte(0xa000,addr) & mask;
                             <1>            ;  if (data > 0) attr |= (0x01 << i);
                             <1>            ; }
                             <1> vga_rp_2:
00002F9A 88CC                <1>     mov    ah, cl ; i << 8
00002F9C B004                <1>     mov    al, 4  ; | 0x04
00002F9E 66BACE03            <1>     mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
00002FA2 66EF                <1>     out    dx, ax
                             <1>     ; data = read_byte(0xa000,addr) & mask;
00002FA4 8A07                <1>     mov    al, [edi]
00002FA6 20E8                <1>     and    al, ch ; & mask
                             <1>     ; if (data > 0) attr |= (0x01 << i);
00002FA8 08C0                <1>     or     al, al
00002FAA 7408                <1>     jz     short vga_rp_3 ; al = 0
00002FAC B701                <1>     mov    bh, 1
00002FAE D2E7                <1>     shl    bh, cl ; (0x01 << i)
00002FB0 08FB                <1>     or     bl, bh ; attr |= (0x01 << i)
00002FB2 88D8                <1>     mov    al, bl ; pixel value
```

```
                                <1> vga_rp_3:
00002FB4 C3                     <1>     retn
                                <1>
                                <1> vga_beeper:
                                <1>     ; 04/08/2016  (TRDOS 386 = TRDOS v2.0)
00002FB5 FB                     <1>     sti
                                <1>     ;mov   bh, [ACTIVE_PAGE]
00002FB6 E917F4FFFF             <1>         jmp     beeper_gfx
                                <1>
                                <1> vga_write_teletype:
                                <1>     ; 03/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 12/04/2021 (TRDOS 386 v2.0.3, 32 bit push/pop)
                                <1>     ; 09/12/2017
                                <1>     ; 06/08/2016
                                <1>     ; 04/08/2016
                                <1>     ; 01/08/2016
                                <1>     ; 31/07/2016
                                <1>     ; 09/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_write_teletype'
                                <1>     ; 'biosfn_write_char_only'
                                <1>
                                <1>     ; INPUT ->
                                <1>     ; [CRT_MODE] = current video mode (>7)
                                <1>     ; AL = Character to write
                                <1>     ; BL = Color of character
                                <1>     ; OUTPUT ->
                                <1>     ; Regen buffer updated
                                <1>
                                <1>     ; biosfn_write_teletype (car, page, attr, flag)
                                <1>     ; car = character (AL)
                                <1>     ; page = 0
                                <1>     ; attr = color (BL)
                                <1>     ; 'flag' not used
                                <1>
00002FBB 8A25[2E680000]         <1>     mov     ah, [CRT_MODE]
00002FC1 88C7                   <1>     mov     bh, al ; character
00002FC3 668B15[CE760100]       <1>     mov     dx, [CURSOR_POSN] ; cursor pos for page 0
                                <1>
00002FCA BE[52680000]           <1>     mov     esi, vga_g_modes
00002FCF 89F7                   <1>     mov     edi, esi
00002FD1 83C708                 <1>     add     edi, vga_g_mode_count
                                <1> vga_wtty_0:
00002FD4 AC                     <1>     lodsb
00002FD5 38E0                   <1>     cmp     al, ah ; [CRT_MODE]
00002FD7 7405                   <1>     je      short vga_wtty_2
00002FD9 39FE                   <1>     cmp     esi, edi
00002FDB 72F7                   <1>     jb      short vga_wtty_0
                                <1> vga_wtty_1:
00002FDD C3                     <1>     retn    ; nothing to do
                                <1> vga_wtty_2:
00002FDE 80FF07                 <1>     cmp     bh, 07h ; bell (beep)
00002FE1 74D2                   <1>     je      short vga_beeper  ; u11
00002FE3 80FF08                 <1>     cmp     bh, 08h ; backspace
00002FE6 7508                   <1>     jne     short vga_wtty_3
                                <1>     ; if(xcurs>0)xcurs--;
00002FE8 08D2                   <1>     or      dl, dl ; xcurs (column)
00002FEA 74F1                   <1>     jz      short vga_wtty_1
00002FEC FECA                   <1>     dec     dl ; xcurs--;
00002FEE EB55                   <1>         jmp     short vga_wtty_12
                                <1> vga_wtty_3:
00002FF0 80FF0D                 <1>     cmp     bh, 0Dh ; carriage return (\r)
00002FF3 7504                   <1>     jne     short vga_wtty_4
                                <1>     ; xcurs=0;
00002FF5 28D2                   <1>     sub     dl, dl ; 0
00002FF7 EB4C                   <1>         jmp     short vga_wtty_12
                                <1> vga_wtty_4:
00002FF9 80FF0A                 <1>     cmp     bh, 0Ah ; new line (\n)
00002FFC 7504                   <1>     jne     short vga_wtty_5
                                <1>     ; ycurs++;
00002FFE FEC6                   <1>     inc     dh ; next row
00003000 EB5E                   <1>         jmp     short vga_wtty_11
                                <1> vga_wtty_5:
00003002 80FF09                 <1>     cmp     bh, 09h ; tab stop
00003005 7523                   <1>     jne     short vga_wtty_8
00003007 88D0                   <1>     mov     al, dl
                                <1>     ;cbw
00003009 30E4                   <1>     xor     ah, ah ; 09/12/2017
0000300B B108                   <1>     mov     cl, 8
0000300D F6F1                   <1>     div     cl
0000300F 28E1                   <1>     sub     cl, ah
                                <1>     ;
00003011 B720                   <1>     mov     bh, 20h ; space
                                <1> vga_wtty_6: ; tab stop loop
                                <1>     ;push   cx
                                <1>     ;push   bx
                                <1>     ; 12/04/2021
00003013 51                     <1>     push    ecx
00003014 53                     <1>     push    ebx
00003015 E810000000             <1>     call    vga_wtty_8
                                <1>     ;pop    bx  ; bh = character, bl = color
                                <1>     ;pop    cx
                                <1>     ; 12/04/2021
0000301A 5B                     <1>     pop     ebx  ; bh = character, bl = color
0000301B 59                     <1>     pop     ecx
0000301C FEC9                   <1>     dec     cl
0000301E 7409                   <1>     jz      short vga_wtty_7
00003020 668B15[CE760100]       <1>     mov     dx, [CURSOR_POSN] ; new cursor position (pg 0)
00003027 EBEA                   <1>     jmp     short vga_wtty_6
                                <1> vga_wtty_7:
00003029 C3                     <1>     retn
                                <1>     ;
                                <1> vga_wtty_8:
0000302A 83C64F                 <1>     add     esi, vga_g_memmodel - (vga_g_modes + 1)
                                <1>     ; [ESI] = VGA memory model number (LINEAR8, PLANAR4, PLANAR1)
0000302D BF00000A00             <1>     mov     edi, 0A0000h
                                <1>     ;
00003032 88F8                   <1>     mov     al, bh ; character
                                <1>     ;
00003034 803E04                 <1>     cmp     byte [esi], PLANAR4
00003037 7414                   <1>     je      short vga_wtty_planar
00003039 803E03                 <1>     cmp     byte [esi], PLANAR1
0000303C 740F                   <1>     je      short vga_wtty_planar
                                <1> vga_wtty_linear8:
                                <1>     ; write_gfx_char_lin(car,attr,xcurs,ycurs,nbcols);
                                <1>     ; AL = car, BL = attr (color), DL = xcurs, DH = ycurs,
                                <1>     ; [CRT_COLS] = nbcols
0000303E E8F1FCFFFF             <1>     call    write_gfx_char_lin
00003043 EB0D                   <1>     jmp     short vga_wtty_9
                                <1>
                                <1> vga_wtty_12:
```

```
                                <1>     ; 09/07/2016
                                <1>     ; set cursor position
                                <1>     ; NOTE: Hardware cursor position will not be set
                                <1>     ;   in any VGA modes (>7)
                                <1>     ;   But, cursor position will be saved into
                                <1>     ;   [CURSOR_POSN].
                                <1>     ;   TRDOS 386 (TRDOS v2.0) uses only one page
                                <1>     ;   (page 0) for all graphics modes.
                                <1>
00003045 668915[CE760100]       <1>     mov    [CURSOR_POSN], dx ; save cursor pos for pg 0
                                <1>     ; 04/08/2016
                                <1>     ;mov   bh, [ACTIVE_PAGE] ; = 0
                                <1>     ;call  _set_cpos
0000304C C3                     <1>     retn
                                <1>
                                <1> vga_wtty_planar:
                                <1>     ; write_gfx_char_pl4(car,attr,xcurs,ycurs,nbcols,cheight);
                                <1>     ; AL = car, BL = attr (color), DL = xcurs, DH = ycurs,
                                <1>     ; [CRT_COLS]= nbcols, [CHAR_HEIGHT] = cheight
0000304D E883FDFFFF             <1>     call   write_gfx_char_pl4
                                <1> vga_wtty_9:
00003052 FEC2                   <1>     inc    dl ; xcurs++;
                                <1> vga_wtty_10:
                                <1>     ; Do we need to wrap ?
                                <1>     ; if(xcurs==nbcols)
00003054 3A15[30680000]         <1>     cmp    dl, [CRT_COLS] ; [VGA_COLS]
0000305A 7204                   <1>     jb     short vga_wtty_11 ; no
0000305C 28D2                   <1>     sub    dl, dl ; xcurs=0;
0000305E FEC6                   <1>     inc    dh ;  ycurs++;
                                <1> vga_wtty_11:
                                <1>     ; Do we need to scroll ?
                                <1>     ; if(ycurs==nbrows)
00003060 3A35[36680000]         <1>     cmp    dh, [VGA_ROWS]
00003066 72DD                   <1>     jb     short vga_wtty_12 ; no
                                <1>     ;
                                <1>     ; biosfn_scroll (nblines,attr,rul,cul,rlr,clr,page,dir)
                                <1>     ; al = nblines = 1, bl = attr (color) = 0
                                <1>     ; ch = rul, cl = cul, dh = rlr, dl = clr, page = 0
                                <1>     ; dir = SCROLL_UP
                                <1>
00003068 B001                   <1>     mov    al, 1
0000306A 28DB                   <1>     sub    bl, bl ; 0 ; blank/black line (attr=0) will be used
                                <1>     ;sub   cx, cx ; 0,0
                                <1>     ; 03/08/2022
0000306C 29C9                   <1>     sub    ecx, ecx
                                <1>
                                <1>     ; 06/08/2016
0000306E 8A35[36680000]         <1>     mov    dh, [VGA_ROWS]
00003074 FECE                   <1>     dec    dh ; nbrows -1
                                <1>
                                <1>     ;push  dx ; 04/08/2016
                                <1>     ; 12/04/2021
00003076 52                     <1>     push   edx
00003077 8A15[30680000]         <1>     mov    dl, [CRT_COLS]
0000307D FECA                   <1>     dec    dl ; nbcols -1
                                <1>
0000307F 8A25[2E680000]         <1>     mov    ah, [CRT_MODE]
                                <1>
                                <1>     ; biosfn_scroll(0x01,0x00,0,0,nbrows-1,nbcols-1,page,SCROLL_UP);
00003085 E843F5FFFF             <1>     call   vga_graphics_up
                                <1>     ; 04/08/2016
                                <1>     ;pop   dx
                                <1>     ; 12/04/2021
0000308A 5A                     <1>     pop    edx
                                <1>
                                <1>     ;dec   dh ; ycurs-=1
0000308B EBB8                   <1>     jmp    short vga_wtty_12
                                <1>
                                <1> font_setup:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 09/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; 09/07/2016
                                <1>     ; character generator (font loading) functions
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'int10_func'
                                <1>
                                <1>     ; AX = 1100H ; Load User-Defined Font (EGA/VGA)
                                <1>     ;
                                <1>         ; BH =  height of each character (bytes per character definition)
                                <1>         ; (BL = font block to load (EGA: 0-3; VGA: 0-7))
                                <1>     ; CX =  number of characters to redefine (<=256)
                                <1>         ; DX =  ASCII code of the first character defined at ES:BP
                                <1>         ; EBP =    address of font-definition information
                                <1>     ;       (in user's memory space)
                                <1>
                                <1>     ; case 0x11:
                                <1>             ; switch(GET_AL())
                                <1>             ; {
                                <1>     ; case 0x00:
                                <1>         ; case 0x10:
                                <1>         ; biosfn_load_text_user_pat(GET_AL(),ES,BP,CX,DX,GET_BL(),GET_BH());
                                <1>         ; break;
                                <1>
                                <1>     ; AX = 1110H ; Load and Activate User-Defined Font (EGA/VGA)
0000308D 08C0                   <1>     or     al, al ; 0
0000308F 7404                   <1>     jz     short font_setup_0
00003091 3C10                   <1>     cmp    al, 10h
00003093 7511                   <1>     jne    short font_setup_1
                                <1> font_setup_0:
00003095 E8CE000000             <1>     call   transfer_user_fonts
0000309A 721C                   <1>     jc     short font_setup_error
0000309C E8AD010000             <1>     call   load_text_user_pat
000030A1 E9C5EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_1:
                                <1>     ; AX = 1101H ; Load ROM 8x14 Character Set (EGA/VGA)
                                <1>     ; case 0x01:
                                <1>         ; case 0x11:
                                <1>         ; biosfn_load_text_8_14_pat(GET_AL(),GET_BL());
                                <1>         ; break;
000030A6 3C01                   <1>     cmp    al, 1
000030A8 7404                   <1>     je     short font_setup_2
000030AA 3C11                   <1>     cmp    al, 11h
000030AC 7511                   <1>     jne    short font_setup_3
                                <1> font_setup_2:
                                <1>     ; AX = 1111H ; Load and Activate ROM 8x14 Character Set (EGA/VGA)
                                <1>     ; (BL = font block to load (EGA: 0-3; VGA: 0-7))
000030AE E8C9020000             <1>     call   load_text_8_14_pat
000030B3 E9B3EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_error:
000030B8 29C0                   <1>     sub    eax, eax ; 0 -> fonts could not be loaded
000030BA E9B1EAFFFF             <1>     jmp    _video_return
```

```
                                <1> font_setup_3:
                                <1>     ; AX = 1102H ; Load ROM 8x8 Character Set (EGA/VGA)
                                <1>     ; case 0x02:
                                <1>         ; case 0x12:
                                <1>         ; biosfn_load_text_8_8_pat(GET_AL(),GET_BL());
                                <1>         ; break;
000030BF 3C02                   <1>     cmp     al, 2
000030C1 7404                   <1>     je      short font_setup_4
000030C3 3C12                   <1>     cmp     al, 12h
000030C5 750A                   <1>     jne     short font_setup_5
                                <1> font_setup_4:
                                <1>     ; AX = 1112H ; Load and Activate ROM 8x8 Character Set (EGA/VGA)
                                <1>     ; (BL = font block to load (EGA: 0-3; VGA: 0-7))
000030C7 E8E1020000             <1>     call    load_text_8_8_pat
000030CC E99AEAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_5:
                                <1>     ; AX = 1104H ; Load ROM 8x16 Character Set (EGA/VGA)
                                <1>     ; case 0x04:
                                <1>         ; case 0x14:
                                <1>         ; biosfn_load_text_8_16_pat(GET_AL(),GET_BL());
                                <1>         ; break;
000030D1 3C04                   <1>     cmp     al, 4
000030D3 7404                   <1>     je      short font_setup_6
000030D5 3C14                   <1>     cmp     al, 14h
000030D7 750A                   <1>     jne     short font_setup_7
                                <1> font_setup_6:
                                <1>     ; AX = 1114H ; Load and Activate ROM 8x16 Character Set (EGA/VGA)
                                <1>     ; (BL = font block to load (EGA: 0-3; VGA: 0-7))
000030D9 E819030000             <1>     call    load_text_8_16_pat
000030DE E988EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_7:
                                <1>     ; Note: AX=1120h (Setup INT 1Fh, EXT_PTR) is not needed
                                <1>     ; for TRDOS 386 (TRDOS v2.0) video functionality;
                                <1>     ; because, originally EXT_PTR (font address) was used for
                                <1>     ; chars 80h to 0FFh (after the first 128 ASCII char fonts), for
                                <1>     ; CGA graphics mode; currenty, 'vgafont8' address has 256 chars!
                                <1>     ;
                                <1>     ; case 0x20:
                                <1>         ; biosfn_load_gfx_8_8_chars(ES,BP);
                                <1>         ; break;
                                <1>     ; case 0x21:
                                <1>         ; biosfn_load_gfx_user_chars(ES,BP,CX,GET_BL(),GET_DL());
                                <1>         ; break;
                                <1>     ; AX = 1121H ; Setup User-Defined Font for Graphics Mode (VGA)
                                <1>     ; BL   screen rows code: 00H = user-specified (in DL)
                                <1>         ;                        01H = 14 rows
                                <1>         ;                        02H = 25 rows
                                <1>         ;                        03H = 43 rows
                                <1>         ; CX   bytes per character definition
                                <1>         ; DL   (when BL=0) custom number of character rows on screen
                                <1>         ; EBP  address of font-definition information (user's mem space)
                                <1>
000030E3 3C21                   <1>     cmp     al, 21h
000030E5 7531                   <1>     jne     short font_setup_9
                                <1>
                                <1>     ; TRDOS 386 modification !
                                <1>     ; dh = 0 -> 256 characters
                                <1>     ; dh = 80h -> second 128 characters
                                <1>     ; dh = 0FFh -> first 128 characters
                                <1>
                                <1>     ; 09/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ;push   ebx
000030E7 51                     <1>     push    ecx
000030E8 52                     <1>     push    edx
000030E9 30D2                   <1>     xor     dl, dl
000030EB 88CF                   <1>     mov     bh, cl ; character height
                                <1>     ;mov    cx, 100h ; 256
                                <1>     ; 03/08/2022
000030ED 31C9                   <1>     xor     ecx, ecx
000030EF FEC5                   <1>     inc     ch
                                <1>     ; ecx = 100h
000030F1 08F6                   <1>     or      dh, dh ; 0
000030F3 7410                   <1>     jz      short  font_setup_8
000030F5 FECD                   <1>     dec     ch ; cx = 0
000030F7 80FEFF                 <1>     cmp     dh, 0FFh
000030FA 7409                   <1>     je      short font_setup_8 ; 1st 128 chars
                                <1>     ; 2nd 128 chars
000030FC 80FE80                 <1>     cmp     dh, 80h
000030FF 75B7                   <1>     jne     short font_setup_error ; invalid !
00003101 88F1                   <1>     mov     cl, dh
00003103 86D6                   <1>     xchg    dl, dh
                                <1>     ; number of chars, cx = 80h
                                <1>     ; start char, dl = 80h
                                <1> font_setup_8:
00003105 E85E000000             <1>     call    transfer_user_fonts
0000310A 5A                     <1>     pop     edx
0000310B 59                     <1>     pop     ecx
                                <1>     ;pop    ebx
0000310C 72AA                   <1>     jc      short font_setup_error
                                <1>     ; ebp = user's font data address in system's memory space
0000310E E82F030000             <1>     call    load_gfx_user_chars
00003113 E953EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_9:
                                <1>     ; case 0x22:
                                <1>         ; biosfn_load_gfx_8_14_chars(GET_BL());
                                <1>         ; break;
00003118 3C22                   <1>     cmp     al, 22h
0000311A 750A                   <1>     jne     short font_setup_10
0000311C E85D030000             <1>     call    load_gfx_8_14_chars
00003121 E945EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_10:
                                <1>     ; case 0x23:
                                <1>         ; biosfn_load_gfx_8_8_dd_chars(GET_BL());
                                <1>         ; break;
00003126 3C23                   <1>     cmp     al, 23h
00003128 750A                   <1>     jne     short font_setup_11
0000312A E890030000             <1>     call    load_gfx_8_8_chars
0000312F E937EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_11:
                                <1>     ; case 0x24:
                                <1>         ; biosfn_load_gfx_8_16_chars(GET_BL());
                                <1>         ; break;
00003134 3C24                   <1>     cmp     al, 24h
00003136 750A                   <1>     jne     short font_setup_12
00003138 E8C3030000             <1>     call    load_gfx_8_16_chars
0000313D E929EAFFFF             <1>         jmp     VIDEO_RETURN
                                <1> font_setup_12:
                                <1>     ; case 0x30:
                                <1>         ; biosfn_get_font_info(GET_BH(),&ES,&BP,&CX,&DX);
                                <1>         ; break;
00003142 3C30                   <1>     cmp     al, 30h
00003144 750A                   <1>     jne     short font_setup_13
00003146 E8F6030000             <1>     call    get_font_info
                                <1>     ; eax = return value (info: 4 bytes for 4 parms)
```

```
                                <1>     ; eax = 0 -> invalid function (input)
0000314B E920EAFFFF             <1>         jmp     _video_return
                                <1> font_setup_13:
00003150 3C03                   <1>     cmp     al, 03h ; AX = 1103h
00003152 750D                   <1>     jne     short font_setup_14
                                <1>     ; biosfn_set_text_block_specifier:
                                <1>     ; BL = font block selector code
                                <1>     ; NOTE: TRDOS 386 only uses and sets font block 0
                                <1>     ; (It is as BL = 0 for TRDOS 386)
00003154 66BAC403               <1>     mov     dx, 3C4h ; VGAREG_SEQU_ADDRESS
                                <1>     ;mov    ah, bl
00003158 28E4                   <1>     sub     ah, ah ; 0
                                <1>     ;mov    al, 03h
0000315A 66EF                   <1>     out     dx, ax
0000315C E90AEAFFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> font_setup_14:
00003161 29C0                   <1>     sub     eax, eax ; 0 = invalid function
00003163 E908EAFFFF             <1>         jmp     _video_return
                                <1>
                                <1> transfer_user_fonts:
                                <1>     ; 02/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 19/01/2021
                                <1>     ; 09/01/2021
                                <1>     ; 05/01/2021 (TRDOS 386 v2.0.3)
                                <1>
                                <1>     ; BH =  height of each character (bytes per character)
                                <1>     ; CX =  number of characters to redefine (<=256)
                                <1>         ; DX =  ASCII code of the first character defined at EBP
                                <1>         ; EBP =    address of font-definition information
                                <1>     ;       (in user's memory space)
                                <1>
                                <1>     ; Modified registers: eax, edx, ecx, esi, edi, ebp
                                <1>     ;
                                <1>     ; output:
                                <1>     ;       ebp = user font data address in system memory
                                <1>
00003168 81E2FFFF0000           <1>     and     edx, 0FFFFh
0000316E 81E1FFFF0000           <1>     and     ecx, 0FFFFh
00003174 7537                   <1>     jnz     short transfer_user_fonts_5
                                <1>
00003176 09D2                   <1>     or      edx, edx
00003178 7531                   <1>     jnz     short transfer_user_fonts_4
0000317A 09ED                   <1>     or      ebp, ebp
0000317C 752D                   <1>     jnz     short transfer_user_fonts_4
                                <1>
                                <1>     ; cx = 0, dx = 0, ebp = 0
                                <1>     ; copy system font to user font
                                <1>
0000317E B140                   <1>     mov     cl, 64 ; 64 dwords
                                <1>
00003180 80FF10                 <1>     cmp     bh, 16
00003183 7417                   <1>     je      short transfer_user_fonts_2
00003185 80FF08                 <1>     cmp     bh, 8
00003188 7406                   <1>     je      short transfer_user_fonts_1
                                <1>
0000318A BD[40550100]           <1>     mov     ebp, vgafont14
0000318F C3                     <1>     retn
                                <1>
                                <1> transfer_user_fonts_1:
00003190 BF00500900             <1>     mov     edi, VGAFONT8USER
00003195 BE[404D0100]           <1>     mov     esi, vgafont8
0000319A EB0A                   <1>     jmp     short transfer_user_fonts_3
                                <1>
                                <1> transfer_user_fonts_2:
0000319C BF00400900             <1>     mov     edi, VGAFONT16USER
000031A1 BE[40630100]           <1>     mov     esi, vgafont16
                                <1> transfer_user_fonts_3:
000031A6 89FD                   <1>     mov     ebp, edi
000031A8 F3A5                   <1>     rep     movsd
000031AA C3                     <1>     retn
                                <1>
                                <1> transfer_user_fonts_4:
000031AB F9                     <1>     stc
000031AC C3                     <1>     retn
                                <1>
                                <1> transfer_user_fonts_5:
000031AD 09ED                   <1>     or      ebp, ebp
000031AF 74FA                   <1>     jz      short transfer_user_fonts_4 ; invalid address !
                                <1>
000031B1 6681F90001             <1>     cmp     cx, 256
000031B6 77F3                   <1>     ja      short transfer_user_fonts_4
000031B8 29D1                   <1>     sub     ecx, edx
000031BA 76EF                   <1>     jna     short transfer_user_fonts_4
                                <1>
000031BC 80FF0E                 <1>     cmp     bh, 14 ; 8x14 font
                                <1>                    ; (there is not an alternative buffer)
000031BF 7524                   <1>     jne     short transfer_user_fonts_6
                                <1>
                                <1>     ; use system's 8x14 font space if permission flag is 1
000031C1 F605[869D0100]80       <1>     test    byte [ufont], 80h
000031C8 74E1                   <1>     jz      short transfer_user_fonts_4 ; not allowed
                                <1>
                                <1>     ; permission is given (for vgafont14 location etc.)
                                <1>     ; (for permanent font modification)
                                <1>     ;
                                <1>     ; 19/01/2021
                                <1>     ; Note: Permission flag can be set by 'root' while
                                <1>     ;       system is not in multi tasking/user mode
                                <1>     ;       while [multi_tasking] = 0 and [u.uid] = 0
                                <1>
                                <1>     ;push   edx
                                <1>     ; 02/08/2022
000031CA 87D1                   <1>     xchg    edx, ecx
                                <1>     ;xor    ah, ah
                                <1>     ; 02/08/2022
000031CC 31C0                   <1>     xor     eax, eax
000031CE 88F8                   <1>     mov     al, bh ; mov al, 14
                                <1>     ;mul    cx
                                <1>     ; 02/08/2022
000031D0 F7E2                   <1>     mul     edx  ; char count * 14
                                <1>     ; 02/08/2022
000031D2 89CA                   <1>     mov     edx, ecx ; ascii code
000031D4 89C1                   <1>     mov     ecx, eax
                                <1>             ; ecx = byte count
                                <1>     ;pop    edx
                                <1>     ; 02/08/2022
                                <1>     ;xor    eax, eax
000031D6 30E4                   <1>     xor     ah, ah
000031D8 88F8                   <1>     mov     al, bh ; mov ax, 14 ; bytes per character
                                <1>     ;mul    dx
                                <1>     ;mov    dx, ax ; char offset
                                <1>     ; 02/08/2022
000031DA F7E2                   <1>     mul     edx
```

```
000031DC 89C2                <1>     mov     edx, eax ; char offset
000031DE BF[40550100]        <1>     mov     edi, vgafont14
000031E3 EB48                <1>     jmp     short transfer_user_fonts_8
                             <1> transfer_user_fonts_6:
000031E5 80FF08              <1>     cmp     bh, 8 ; 8x8 font
000031E8 7520                <1>     jne     short transfer_user_fonts_7 ; 8x16 font
                             <1>     ;shl    dx, 3 ; * 8
                             <1>     ;shl    cx, 3 ; * 8
                             <1>     ; 02/08/2022
000031EA C1E203              <1>     shl     edx, 3 ; byte offset
000031ED C1E103              <1>     shl     ecx, 3 ; byte count
                             <1>     ; 09/01/2021
000031F0 BF00500900          <1>     mov     edi, VGAFONT8USER
000031F5 F605[869D0100]08    <1>     test    byte [ufont], 8  ; already loaded ?
000031FC 752F                <1>     jnz     short transfer_user_fonts_8 ; yes
000031FE BE[404D0100]        <1>     mov     esi, vgafont8
00003203 E839000000          <1>     call    transfer_user_fonts_10
00003208 EB23                <1>     jmp     short transfer_user_fonts_8
                             <1> transfer_user_fonts_7:
0000320A 80FF10              <1>     cmp     bh, 16 ; 8x16 font
0000320D 759C                <1>     jne     short transfer_user_fonts_4  ; invalid !
                             <1>     ;shl    dx, 4 ; * 16
                             <1>     ;shl    cx, 4 ; * 16
                             <1>     ; 02/08/2022
0000320F C1E204              <1>     shl     edx, 4 ; byte offset
00003212 C1E104              <1>     shl     ecx, 4 ; byte count
00003215 BF00400900          <1>     mov     edi, VGAFONT16USER
0000321A F605[869D0100]10    <1>     test    byte [ufont], 16  ; already loaded ?
00003221 750A                <1>     jnz     short transfer_user_fonts_8 ; yes
00003223 BE[40630100]        <1>     mov     esi, vgafont16
00003228 E814000000          <1>     call    transfer_user_fonts_10
                             <1> transfer_user_fonts_8:
0000322D 01D7                <1>     add     edi, edx ; char offset
                             <1>     ; 09/07/2016
                             <1>     ;and    ecx, 0FFFFh
                             <1>     ; ECX = byte count
                             <1>     ;push   ecx
0000322F 89EE                <1>     mov     esi, ebp ; user's font buffer
                             <1>     ; 09/01/2021
00003231 89FD                <1>     mov     ebp, edi ; system addr for user's font
                             <1>     ; 05/01/2021
                             <1>     ;mov    edi, Cluster_Buffer  ; system buffer
00003233 E82DD90000          <1>     call    transfer_from_user_buffer
                             <1>     ;pop    ecx
                             <1>     ; ecx = transfer (byte) count = character count
00003238 7206                <1>     jc      short transfer_user_fonts_9
                             <1>     ; 05/01/2021
                             <1>     ;mov    ebp, Cluster_Buffer
                             <1>
0000323A 083D[869D0100]      <1>     or      byte [ufont], bh
                             <1>                     ; 8x8 or 8x16 user font ready
                             <1> transfer_user_fonts_9:
00003240 C3                  <1>     retn
                             <1>
                             <1> transfer_user_fonts_10:
                             <1>     ; 02/08/2022
                             <1>     ; 09/01/2021
00003241 56                  <1>     push    esi
00003242 57                  <1>     push    edi
00003243 51                  <1>     push    ecx
                             <1>     ;mov    cx, 64
                             <1>     ; 02/08/2022
00003244 31C9                <1>     xor     ecx, ecx
00003246 B140                <1>     mov     cl, 64
00003248 F3A5                <1>     rep     movsd
0000324A 59                  <1>     pop     ecx
0000324B 5F                  <1>     pop     edi
0000324C 5E                  <1>     pop     esi
0000324D C3                  <1>     retn
                             <1>
                             <1> load_text_user_pat:
                             <1>     ; 02/08/2022 (TRDOS 3386 v2.0.5)
                             <1>     ; 26/07/2016
                             <1>     ; 09/07/2016
                             <1>     ; load user defined (EGA/VGA) text fonts
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', 'biosfn_load_text_user_pat'
                             <1>
                             <1>     ; biosfn_load_text_user_pat (AL,ES,BP,CX,DX,BL,BH)
                             <1>
                             <1>     ; get_font_access();
                             <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                             <1>     ; for(i=0;i<CX;i++)
                             <1>     ; {
                             <1>     ;  src = BP + i * BH;
                             <1>     ;  dest = blockaddr + (DX + i) * 32;
                             <1>     ;  memcpyb(0xA000, dest, ES, src, BH);
                             <1>     ; }
                             <1>     ; release_font_access();
                             <1>     ; if(AL>=0x10)
                             <1>     ; {
                             <1>     ; set_scan_lines(BH);
                             <1>     ; }
                             <1>
0000324E 50                  <1>     push    eax
0000324F E839000000          <1>     call    get_font_access
00003254 28DB                <1>     sub     bl, bl ; i = 0
                             <1>     ; 02/08/2022
                             <1>     ; ecx <= 256
00003256 30ED                <1>     xor     ch, ch
                             <1> ltup_1:
00003258 88D8                <1>     mov     al, bl
0000325A F6E7                <1>     mul     bh
                             <1>     ;movzx  esi, ax
                             <1>     ; 02/08/2022
0000325C 89C6                <1>     mov     esi, eax
0000325E 01EE                <1>     add     esi, ebp
00003260 88D8                <1>     mov     al, bl
00003262 28E4                <1>     sub     ah, ah
                             <1>     ;add    ax, dx ; (DX + i)
                             <1>     ;shl    ax, 5  ; * 32
                             <1>     ; 02/08/2022
00003264 01D0                <1>     add     eax, edx
00003266 C1E005              <1>     shl     eax, 5
                             <1>     ;movzx  edi, ax
                             <1>     ; 02/08/2022
00003269 89C7                <1>     mov     edi, eax
0000326B 81C700000A00        <1>     add     edi, 0A0000h
00003271 51                  <1>     push    ecx
                             <1>     ;movzx  ecx, bh
                             <1>     ; 02/08/2022
```

```
00003272 88F9              <1>     mov     cl, bh
00003274 F3A4              <1>     rep     movsb
00003276 59                <1>     pop     ecx
00003277 FEC3              <1>     inc     bl
00003279 38CB              <1>     cmp     bl, cl
0000327B 75DB              <1>     jne     short ltup_1
                           <1>     ;
0000327D E83E000000        <1>     call    release_font_access
                           <1>     ;
00003282 58                <1>     pop     eax
                           <1>     ; if(AL>=0x10)
00003283 3C10              <1>     cmp     al, 10h
00003285 7205              <1>     jb      short ltup_2
                           <1>     ; set_scan_lines(BH);
00003287 E86F000000        <1>     call    set_scan_lines
                           <1> ltup_2:
0000328C C3                <1>     retn
                           <1>
                           <1> get_font_access:
                           <1>     ; 02/08/2022
                           <1>     ; 09/07/2016
                           <1>     ;
                           <1>     ; derived from 'Plex86/Bochs VGABios' source code
                           <1>     ; vgabios-0.7a (2011)
                           <1>     ; by the LGPL VGABios developers Team (2001-2008)
                           <1>     ; 'vgabios.c', 'get_font_access'
                           <1>
                           <1>     ; get_font_access()
0000328D 52                <1>     push    edx
0000328E 66BAC403          <1>     mov     dx, 3C4h ; VGAREG_SEQU_ADDRESS
                           <1>     ; 02/08/2022
00003292 31C0              <1>     xor     eax, eax
                           <1>     ;mov    ax, 0100h
00003294 B401              <1>     mov     ah, 1
                           <1>     ; ax = 0100h
00003296 66EF              <1>     out     dx, ax
00003298 66B80204          <1>     mov     ax, 0402h
0000329C 66EF              <1>     out     dx, ax
0000329E 66B80407          <1>     mov     ax, 0704h
000032A2 66EF              <1>     out     dx, ax
000032A4 66B80003          <1>     mov     ax, 0300h
000032A8 66EF              <1>     out     dx, ax
                           <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                           <1>     ; 02/08/2022
000032AA B2CE              <1>     mov     dl, 0CEh
000032AC 66B80402          <1>     mov     ax, 0204h
000032B0 66EF              <1>     out     dx, ax
000032B2 66B80500          <1>     mov     ax, 0005h
000032B6 66EF              <1>     out     dx, ax
000032B8 66B80604          <1>     mov     ax, 0406h
000032BC 66EF              <1>     out     dx, ax
000032BE 5A                <1>     pop     edx
000032BF C3                <1>     retn
                           <1>
                           <1> release_font_access:
                           <1>     ; 02/08/2022
                           <1>     ; 29/07/2016
                           <1>     ; 09/07/2016
                           <1>     ;
                           <1>     ; derived from 'Plex86/Bochs VGABios' source code
                           <1>     ; vgabios-0.7a (2011)
                           <1>     ; by the LGPL VGABios developers Team (2001-2008)
                           <1>     ; 'vgabios.c', 'release_font_access'
                           <1>
000032C0 66BAC403          <1>     mov     dx, 3C4h ; VGAREG_SEQU_ADDRESS
                           <1>     ;mov    ax, 0100h
                           <1>     ; 02/08/2022
000032C4 29C0              <1>     sub     eax, eax
000032C6 B401              <1>     mov     ah, 1
                           <1>     ; ax = 0100h
000032C8 66EF              <1>     out     dx, ax
000032CA 66B80203          <1>     mov     ax, 0302h
000032CE 66EF              <1>     out     dx, ax
000032D0 66B80403          <1>     mov     ax, 0304h
000032D4 66EF              <1>     out     dx, ax
000032D6 66B80003          <1>     mov     ax, 0300h
000032DA 66EF              <1>     out     dx, ax
                           <1>     ;mov    dx, 3CCh ; VGAREG_READ_MISC_OUTPUT
                           <1>     ; 02/08/2022
000032DC B2CC              <1>     mov     dl, 0CCh
000032DE EC                <1>     in      al, dx
000032DF 2401              <1>     and     al, 01h
000032E1 C0E002            <1>     shl     al, 2
000032E4 0C0A              <1>     or      al, 0Ah
000032E6 88C4              <1>     mov     ah, al
000032E8 B006              <1>     mov     al, 06h
                           <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                           <1>     ; 02/08/2022
000032EA B2CE              <1>     mov     dl, 0CEh
000032EC 66EF              <1>     out     dx, ax
000032EE 66B80400          <1>     mov     ax, 0004h
000032F2 66EF              <1>     out     dx, ax
000032F4 66B80510          <1>     mov     ax, 1005h
000032F8 66EF              <1>     out     dx, ax
000032FA C3                <1>     retn
                           <1>
                           <1> set_scan_lines:
                           <1>     ; 02/08/2022
                           <1>     ; 09/07/2016
                           <1>     ;
                           <1>     ; derived from 'Plex86/Bochs VGABios' source code
                           <1>     ; vgabios-0.7a (2011)
                           <1>     ; by the LGPL VGABios developers Team (2001-2008)
                           <1>     ; 'vgabios.c', 'set_scan_lines'
                           <1>
                           <1>     ; set_scan_lines(lines)
                           <1>     ; BH = lines
                           <1>
                           <1>     ; outb(crtc_addr, 0x09);
000032FB 66BAD403          <1>     mov     dx, 3D4h ; CRTC_ADDRESS = 3D4h (always)
000032FF B009              <1>     mov     al, 09h
00003301 EE                <1>     out     dx, al
                           <1>     ; crtc_r9 = inb(crtc_addr+1);
                           <1>     ;inc    dx ; 3D5h
                           <1>     ; 02/08/2022
00003302 FEC2              <1>     inc     dl
00003304 EC                <1>     in      al, dx
                           <1>     ; crtc_r9 = (crtc_r9 & 0xe0) | (lines - 1);
00003305 24E0              <1>     and     al, 0E0h
00003307 FECF              <1>     dec     bh ; lines - 1
00003309 08F8              <1>     or      al, bh
                           <1>     ; outb(crtc_addr+1, crtc_r9);
0000330B EE                <1>     out     dx, al
                           <1>     ;inc    bh
```

```
                                <1>     ; if(lines==8)
                                <1>     ;cmp   bh, 8
0000330C 80FF07                 <1>     cmp    bh, 7
0000330F 7506                   <1>     jne    short ssl_1
                                <1>     ; biosfn_set_cursor_shape(0x06,0x07);
00003311 66B90706               <1>     mov    cx, 0607h
00003315 EB06                   <1>     jmp    short ssl_2
                                <1> ssl_1:
                                <1>     ; biosfn_set_cursor_shape(lines-4,lines-3);
00003317 88F9                   <1>     mov    cl, bh ; lines - 1
00003319 88CD                   <1>     mov    ch, cl ; lines - 1 (16 -> 15)
0000331B FECD                   <1>     dec    ch  ; lines - 2 (16 -> 14)
                                <1> ssl_2:
                                <1>     ; CH = start line, CL = stop line
0000331D B40A                   <1>     mov    ah, 10  ; 6845 register for cursor set
0000331F 66890D[47680000]       <1>     mov    [CURSOR_MODE], cx ; save in data area
00003326 E877F0FFFF             <1>     call   m16    ; output cx register
                                <1>     ; write_word(BIOSMEM_SEG,BIOSMEM_CHAR_HEIGHT, lines);
0000332B FEC7                   <1>     inc    bh ; lines
0000332D 883D[32680000]         <1>     mov    [CHAR_HEIGHT], bh
                                <1>     ;  outb(crtc_addr, 0x12);
00003333 66BAD403               <1>     mov    dx, 3D4h ; CRTC_ADDRESS
00003337 B012                   <1>     mov    al, 12h
00003339 EE                     <1>     out    dx, al
                                <1>     ; vde = inb(crtc_addr+1);
                                <1>     ;inc   dx
                                <1>     ; 02/08/2022
0000333A FEC2                   <1>     inc    dl
0000333C EC                     <1>     in     al, dx
0000333D 88C4                   <1>     mov    ah, al
                                <1>     ; outb(crtc_addr, 0x07);
                                <1>     ;dec   dx
                                <1>     ; 02/08/2022
0000333F FECA                   <1>     dec    dl
00003341 B007                   <1>     mov    al, 07h
00003343 EE                     <1>     out    dx, al
                                <1>     ; ovl = inb(crtc_addr+1);
                                <1>     ;inc   dx
                                <1>     ; 02/08/2022
00003344 FEC2                   <1>     inc    dl
00003346 EC                     <1>     in     al, dx
                                <1>     ; vde += (((ovl & 0x02) << 7) + ((ovl & 0x40) << 3) + 1);
00003347 88E2                   <1>     mov    dl, ah ; vde
00003349 88C6                   <1>     mov    dh, al ; ovl
                                <1>     ;and   ax, 02h
                                <1>     ;shl   ax, 7
                                <1>     ; 02/08/2022
0000334B 31C0                   <1>     xor    eax, eax
0000334D 88F0                   <1>     mov    al, dh
0000334F 2402                   <1>     and    al, 2
00003351 C1E007                 <1>     shl    eax, 7
                                <1>     ;mov   cx, ax ; (ovl & 0x02) << 7)
                                <1>     ; 02/08/2022
00003354 89C1                   <1>     mov    ecx, eax
00003356 28E4                   <1>     sub    ah, ah
00003358 88F0                   <1>     mov    al, dh ; ovl
                                <1>     ;and   ax, 40h
                                <1>     ;shl   ax, 3  ; (ovl & 0x40) << 3)
                                <1>     ;inc   ax ; + 1
                                <1>     ;add   ax, cx
                                <1>     ; 02/08/2022
0000335A 2440                   <1>     and    al, 40h
0000335C C1E003                 <1>     shl    eax, 3
0000335F 40                     <1>     inc    eax
00003360 01C8                   <1>     add    eax, ecx
00003362 30F6                   <1>     xor    dh, dh
                                <1>     ;add   ax, dx ; + vde
                                <1>     ; 02/08/2022
00003364 01D0                   <1>     add    eax, edx
                                <1>     ; rows = vde / lines;
00003366 F6F7                   <1>     div    bh
                                <1>     ;dec   al ; rows -1
                                <1>     ; write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, rows-1);
00003368 A2[36680000]           <1>     mov    [VGA_ROWS], al ; rows (not 'rows-1' !)
                                <1>     ; write_word(BIOSMEM_SEG,BIOSMEM_PAGE_SIZE, rows * cols * 2);
                                <1>     ;mov   ah, [CRT_COLS]
                                <1>     ;mul   ah
                                <1>     ; 17/11/2020
0000336D F625[30680000]         <1>     mul    byte [CRT_COLS]
                                <1>     ;shl   ax, 1
                                <1>     ; 02/08/2022
00003373 D1E0                   <1>     shl    eax, 1
00003375 66A3[4C830100]         <1>     mov    [CRT_LEN], ax
0000337B C3                     <1>     retn
                                <1>
                                <1> load_text_8_14_pat:
                                <1>     ; 02/08/2022 (TRDOS 3386 v2.0.5)
                                <1>     ; 26/07/2016
                                <1>     ; 25/07/2016
                                <1>     ; 23/07/2016
                                <1>     ; 09/07/2016
                                <1>     ; load user defined (EGA/VGA) text fonts
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_load_text_8_14_pat'
                                <1>
                                <1>     ; biosfn_load_text_8_14_pat (AL,BL)
                                <1>
                                <1>     ; get_font_access();
                                <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                                <1>     ; for(i=0;i<0x100;i++)
                                <1>     ; {
                                <1>     ;  src = i * 14;
                                <1>     ;  dest = blockaddr + i * 32;
                                <1>     ;  memcpyb(0xA000, dest, 0xC000, vgafont14+src, 14);
                                <1>     ; }
                                <1>     ; release_font_access();
                                <1>     ; if(AL>=0x10)
                                <1>     ; {
                                <1>     ; set_scan_lines(14);
                                <1>     ; }
                                <1>
0000337C 50                     <1>     push   eax
0000337D E80BFFFFFF             <1>     call   get_font_access
                                <1>
                                <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                                <1>     ;mov   dl, bl
                                <1>     ;and   dl, 3
                                <1>     ;shl   dx, 14
                                <1>     ;xchg  dx, bx
                                <1>     ;and   dl, 4
                                <1>     ;shl   dx, 11
```

```
                                <1>     ;add   dx, bx
                                <1>
                                <1>     ;xor   dx, dx  ; blockaddr = 0
                                <1>     ; Always block 0 for TRDOS 386 ! (blockaddr=0)
                                <1>
00003382 28DB                   <1>     sub    bl, bl ; i = 0
00003384 B70E                   <1>     mov    bh, 14
00003386 BE[40550100]           <1>     mov    esi, vgafont14
0000338B BF00000A00             <1>     mov    edi, 0A0000h
                                <1>     ; 02/08/2022
00003390 29C9                   <1>     sub    ecx, ecx
                                <1> lt8_14_1:
                                <1>     ;mov   al, bl
                                <1>     ;mul   bh
                                <1>     ;movzx esi, ax
                                <1>     ;add   esi, vgafont14
                                <1>     ;mov   al, bl
                                <1>     ;sub   ah, ah
                                <1>     ;shl   ax, 5 ; * 32
                                <1>     ;;add  ax, dx ; blockaddr + i * 32;
                                <1>     ;movzx edi, ax ; dest
                                <1>     ;add   edi, 0A0000h
                                <1>     ;02/08/2022
                                <1>     ;movzx ecx, bh
00003392 88F9                   <1>     mov    cl, bh
00003394 F3A4                   <1>     rep    movsb
00003396 83C712                 <1>     add    edi, 18 ; 32 - 14
00003399 FEC3                   <1>     inc    bl
0000339B 75F5                   <1>     jnz    short lt8_14_1
                                <1>     ;
0000339D E81EFFFFFF             <1>     call   release_font_access
                                <1>     ;
000033A2 58                     <1>     pop    eax
                                <1>     ; if(AL>=0x10)
000033A3 3C10                   <1>     cmp    al, 10h
000033A5 7205                   <1>     jb     short lt8_14_4
                                <1>     ; BH = 14
                                <1>     ; set_scan_lines(14);
000033A7 E84FFFFFFF             <1>     call   set_scan_lines
                                <1> lt8_14_4:
000033AC C3                     <1>     retn
                                <1>
                                <1> load_text_8_8_pat:
                                <1>     ; 02/08/2022 (TRDOS 3386 v2.0.5)
                                <1>     ; 05/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; 26/07/2016
                                <1>     ; 25/07/2016
                                <1>     ; 23/07/2016
                                <1>     ; 09/07/2016
                                <1>     ; load user defined (EGA/VGA) text fonts
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_load_text_8_8_pat'
                                <1>
                                <1>     ; biosfn_load_text_8_8_pat (AL,BL)
                                <1>
                                <1>     ; get_font_access();
                                <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                                <1>     ; for(i=0;i<0x100;i++)
                                <1>     ; {
                                <1>     ;  src = i * 8;
                                <1>     ;  dest = blockaddr + i * 32;
                                <1>     ;  memcpyb(0xA000, dest, 0xC000, vgafont8+src, 8);
                                <1>     ; }
                                <1>     ; release_font_access();
                                <1>     ; if(AL>=0x10)
                                <1>     ; {
                                <1>     ; set_scan_lines(8);
                                <1>     ; }
                                <1>
000033AD 50                     <1>     push   eax
000033AE E8DAFEFFFF             <1>     call   get_font_access
                                <1>
                                <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                                <1>     ;mov   dl, bl
                                <1>     ;and   dl, 3
                                <1>     ;shl   dx, 14
                                <1>     ;xchg  dx, bx
                                <1>     ;and   dl, 4
                                <1>     ;shl   dx, 11
                                <1>     ;add   dx, bx
                                <1>
                                <1>     ;xor   dx, dx  ; blockaddr = 0
                                <1>     ; Always block 0 for TRDOS 386 ! (blockaddr=0)
                                <1>
000033B3 28DB                   <1>     sub    bl, bl ; i = 0
000033B5 B708                   <1>     mov    bh, 8
                                <1>     ;mov   esi, vgafont8
000033B7 BF00000A00             <1>     mov    edi, 0A0000h
                                <1>
                                <1>     ; 02/08/2022
000033BC 29C9                   <1>     sub    ecx, ecx
                                <1>
                                <1>     ; 05/01/2021
000033BE F605[869D0100]80       <1>     test   byte [ufont], 80h
000033C5 7410                   <1>     jz     short lt8_8_0
                                <1>            ; user font permission (after set mode)
000033C7 F605[869D0100]08       <1>     test   byte [ufont], 8
000033CE 7407                   <1>     jz     short lt8_8_0
000033D0 BE00500900             <1>     mov    esi, VGAFONT8USER
000033D5 EB05                   <1>     jmp    short lt8_8_1
                                <1> lt8_8_0:
000033D7 BE[404D0100]           <1>     mov    esi, vgafont8
                                <1> lt8_8_1:
                                <1>     ;mov   al, bl
                                <1>     ;mul   bh
                                <1>     ;movzx esi, ax
                                <1>     ;add   esi, vgafont8
                                <1>     ;mov   al, bl
                                <1>     ;sub   ah, ah
                                <1>     ;shl   ax, 5 ; * 32
                                <1>     ;;add  ax, dx ; blockaddr + i * 32;
                                <1>     ;movzx edi, ax ; dest
                                <1>     ;add   edi, 0A0000h
                                <1>     ; 02/08/2022
                                <1>     ;movzx ecx, bh
000033DC 88F9                   <1>     mov    cl, bh
000033DE F3A4                   <1>     rep    movsb
000033E0 83C718                 <1>     add    edi, 24 ; 32 - 8
000033E3 FEC3                   <1>     inc    bl
000033E5 75F5                   <1>     jnz    short lt8_8_1
                                <1>     ;
```

```
000033E7 E8D4FEFFFF          <1>     call    release_font_access
                             <1>     ;
000033EC 58                  <1>     pop     eax
                             <1>     ; if(AL>=0x10)
000033ED 3C10                <1>     cmp     al, 10h
000033EF 7205                <1>     jb      short lt8_8_2
                             <1>     ; BH = 8
                             <1>     ; set_scan_lines(8);
000033F1 E805FFFFFF          <1>     call    set_scan_lines
                             <1> lt8_8_2:
000033F6 C3                  <1>     retn
                             <1>
                             <1> load_text_8_16_pat:
                             <1>     ; 02/08/2022 (TRDOS 3386 v2.0.5)
                             <1>     ; 05/01/2021 (TRDOS 386 v2.0.3)
                             <1>     ; 26/07/2016
                             <1>     ; 25/07/2016
                             <1>     ; 23/07/2016
                             <1>     ; 09/07/2016
                             <1>     ; load user defined (EGA/VGA) text fonts
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', 'biosfn_load_text_8_16_pat'
                             <1>
                             <1>     ; biosfn_load_text_8_16_pat (AL,BL)
                             <1>
                             <1>     ; get_font_access();
                             <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                             <1>     ; for(i=0;i<0x100;i++)
                             <1>     ; {
                             <1>     ;  src = i * 16;
                             <1>     ;  dest = blockaddr + i * 32;
                             <1>     ;  memcpyb(0xA000, dest, 0xC000, vgafont16+src, 16);
                             <1>     ; }
                             <1>     ; release_font_access();
                             <1>     ; if(AL>=0x10)
                             <1>     ; {
                             <1>     ; set_scan_lines(16);
                             <1>     ; }
                             <1>
000033F7 50                  <1>     push    eax
000033F8 E890FEFFFF          <1>     call    get_font_access
                             <1>
                             <1>     ; blockaddr = ((BL & 0x03) << 14) + ((BL & 0x04) << 11);
                             <1>     ;mov    dl, bl
                             <1>     ;and    dl, 3
                             <1>     ;shl    dx, 14
                             <1>     ;xchg   dx, bx
                             <1>     ;and    dl, 4
                             <1>     ;shl    dx, 11
                             <1>     ;add    dx, bx
                             <1>
                             <1>     ;xor    dx, dx  ; blockaddr = 0
                             <1>     ; Always block 0 for TRDOS 386 ! (blockaddr=0)
                             <1>
000033FD 28DB                <1>     sub     bl, bl ; i = 0
000033FF B710                <1>     mov     bh, 16
                             <1>     ;mov    esi, vgafont16
00003401 BF00000A00          <1>     mov     edi, 0A0000h
                             <1>     ;movzx  eax, bh
                             <1>     ; 02/08/2022
00003406 29C0                <1>     sub     eax, eax
00003408 88F8                <1>     mov     al, bh
                             <1>
                             <1>     ; 05/01/2021
0000340A F605[869D0100]80    <1>     test    byte [ufont], 80h
00003411 7410                <1>     jz      short lt8_16_0
                             <1>             ; user font permission (after set mode)
00003413 F605[869D0100]10    <1>     test    byte [ufont], 16
0000341A 7407                <1>     jz      short lt8_16_0
0000341C BE00400900          <1>     mov     esi, VGAFONT16USER
00003421 EB05                <1>     jmp     short lt8_16_1
                             <1> lt8_16_0:
00003423 BE[40630100]        <1>     mov     esi, vgafont16
                             <1> lt8_16_1:
                             <1>     ;mov    al, bl
                             <1>     ;mul    bh
                             <1>     ;movzx  esi, ax
                             <1>     ;add    esi, vgafont16
                             <1>     ;mov    al, bl ; i
                             <1>     ;sub    ah, ah
                             <1>     ;shl    ax, 5 ; * 32
                             <1>     ;;add   ax, dx ; blockaddr + i * 32;
                             <1>     ;movzx  edi, ax ; dest
                             <1>     ;add    edi, 0A0000h
                             <1>     ;movzx  ecx, bh
00003428 89C1                <1>     mov     ecx, eax ; 16
0000342A F3A4                <1>     rep     movsb
0000342C 01C7                <1>     add     edi, eax ; add edi, 16
0000342E FEC3                <1>     inc     bl
00003430 75F6                <1>     jnz     short lt8_16_1
                             <1>     ;
00003432 E889FEFFFF          <1>     call    release_font_access
                             <1>     ;
00003437 58                  <1>     pop     eax
                             <1>     ; if(AL>=0x10)
00003438 3C10                <1>     cmp     al, 10h
0000343A 7205                <1>     jb      short lt8_16_2
                             <1>     ; BH = 16
                             <1>     ; set_scan_lines(16);
0000343C E8BAFEFFFF          <1>     call    set_scan_lines
                             <1> lt8_16_2:
00003441 C3                  <1>     retn
                             <1>
                             <1> load_gfx_user_chars:
                             <1>     ; 08/01/2021
                             <1>     ; 05/01/2021 (TRDOS 386 v2.0.3)
                             <1>     ; 08/08/2016
                             <1>     ; 10/07/2016
                             <1>     ; Setup User-Defined Font for Graphics Mode (VGA)
                             <1>     ;
                             <1>     ; derived from 'Plex86/Bochs VGABios' source code
                             <1>     ; vgabios-0.7a (2011)
                             <1>     ; by the LGPL VGABios developers Team (2001-2008)
                             <1>     ; 'vgabios.c', 'biosfn_load_gfx_user_chars'
                             <1>
                             <1>     ; biosfn_load_gfx_user_chars (ES,BP,CX,BL,DL)
                             <1>     ; /* set 0x43 INT pointer */
                             <1>     ; write_word(0x0, 0x43*4, BP);
                             <1>     ; write_word(0x0, 0x43*4+2, ES);
                             <1>
                             <1>     ; 08/01/2021
```

```
                                <1>
                                <1>     ; BL   screen rows code: 00H = user-specified (in DL)
                                <1>         ;                        01H = 14 rows
                                <1>         ;                        02H = 25 rows
                                <1>         ;                        03H = 43 rows
                                <1>         ; CX   bytes per character definition
                                <1>         ; DL   (when BL=0) custom number of character rows on screen
                                <1>         ; EBP  address of font-definition information (user's mem space)
                                <1>
                                <1>     ; 05/01/2021
                                <1>     ;xor   eax, eax
                                <1>     ;dec   eax ; 0FFFFFFFFh (user defined fonts)
                                <1>     ;mov   [VGA_INT43H], eax
                                <1>
                                <1>     ; 08/01/2021
                                <1>     ; ebp = video font data (buffer) address
00003442 892D[5E830100]         <1>     mov    [VGA_INT43H], ebp
                                <1>
                                <1>     ; switch (BL) {
                                <1>     ; case 0:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, DL-1);
                                <1>     ;    break;
00003448 20DB                   <1>     and    bl, bl
0000344A 7508                   <1>     jnz    short l_gfx_uc_1
0000344C 8815[36680000]         <1>     mov    [VGA_ROWS], dl  ; not DL-1 !
00003452 EB23                   <1>     jmp    short l_gfx_uc_4
                                <1> l_gfx_uc_1:
                                <1>     ; case 1:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 13);
                                <1>     ;    break;
00003454 FECB                   <1>     dec    bl
00003456 7509                   <1>     jnz    short l_gfx_uc_2
                                <1>     ; bl = 1
00003458 C605[36680000]0E       <1>     mov    byte [VGA_ROWS], 14  ; not 13 !
0000345F EB16                   <1>     jmp    short l_gfx_uc_4
                                <1> l_gfx_uc_2:
00003461 FECB                   <1>     dec    bl
00003463 740B                   <1>     jz     short l_gfx_uc_3 ; bl = 2
00003465 FECB                   <1>     dec    bl
00003467 750E                   <1>     jnz    short l_gfx_uc_4 ; bl > 3
                                <1>     ; bl = 3
                                <1>     ; case 3:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 42);
                                <1>     ;    break;
00003469 C605[36680000]2B       <1>     mov    byte [VGA_ROWS], 43  ; not 42 !
                                <1> l_gfx_uc_3:
                                <1>     ; case 2:
                                <1>     ; default:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 24);
                                <1>     ;    break;
                                <1>     ; bl = 2 or bl > 3
00003470 C605[36680000]19       <1>     mov    byte [VGA_ROWS], 25  ; not 24 !
                                <1>     ; }
                                <1> l_gfx_uc_4:
                                <1>     ; write_byte(BIOSMEM_SEG, BIOSMEM_CHAR_HEIGHT, CX);
00003477 880D[32680000]         <1>     mov    [CHAR_HEIGHT], cl
                                <1>     ; }
0000347D C3                     <1>     retn
                                <1>
                                <1> load_gfx_8_14_chars:
                                <1>     ; 08/08/2016
                                <1>     ; 10/07/2016
                                <1>     ; Setup ROM 8x14 Font for Graphics Mode (VGA)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_load_gfx_8_14_chars'
                                <1>
                                <1>     ; biosfn_load_gfx_8_14_chars (BL)
                                <1>     ; /* set 0x43 INT pointer */
                                <1>     ; write_word(0x0, 0x43*4, &vgafont14);
                                <1>     ; write_word(0x0, 0x43*4+2, 0xC000);
0000347E C705[5E830100]-        <1>     mov    dword [VGA_INT43H], vgafont14
00003484 [40550100]             <1>
                                <1>
                                <1>     ; BL    screen rows code: 00H = user-specified (in DL)
                                <1>         ;                        01H = 14 rows
                                <1>         ;                        02H = 25 rows
                                <1>         ;                        03H = 43 rows
                                <1>         ; DL    (when BL=0) custom number of char rows on screen
                                <1>
                                <1>     ; switch (BL) {
                                <1>     ; case 0:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, DL-1);
                                <1>     ;    break;
00003488 20DB                   <1>     and    bl, bl
0000348A 7508                   <1>     jnz    short l_gfx_8_14c_1
0000348C 8815[36680000]         <1>     mov    [VGA_ROWS], dl  ; not DL-1 !
00003492 EB23                   <1>     jmp    short l_gfx_8_14c_4
                                <1> l_gfx_8_14c_1:
                                <1>     ; case 1:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 13);
                                <1>     ;    break;
00003494 FECB                   <1>     dec    bl
00003496 7509                   <1>     jnz    short l_gfx_8_14c_2
                                <1>     ; bl = 1
00003498 C605[36680000]0E       <1>     mov    byte [VGA_ROWS], 14  ; not 13 !
0000349F EB16                   <1>     jmp    short l_gfx_8_14c_4
                                <1> l_gfx_8_14c_2:
000034A1 FECB                   <1>     dec    bl
000034A3 740B                   <1>     jz     short l_gfx_8_14c_3 ; bl = 2
000034A5 FECB                   <1>     dec    bl
000034A7 750E                   <1>     jnz    short l_gfx_8_14c_4 ; bl > 3
                                <1>     ; bl = 3
                                <1>     ; case 3:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 42);
                                <1>     ;    break;
000034A9 C605[36680000]2B       <1>     mov    byte [VGA_ROWS], 43  ; not 42 !
                                <1> l_gfx_8_14c_3:
                                <1>     ; case 2:
                                <1>     ; default:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 24);
                                <1>     ;    break;
                                <1>     ; bl = 2 or bl > 3
000034B0 C605[36680000]19       <1>     mov    byte [VGA_ROWS], 25  ; not 24 !
                                <1>     ; }
                                <1> l_gfx_8_14c_4:
                                <1>     ; write_byte(BIOSMEM_SEG, BIOSMEM_CHAR_HEIGHT, 14);
000034B7 C605[32680000]0E       <1>         mov    byte [CHAR_HEIGHT], 14
                                <1>     ; }
000034BE C3                     <1>     retn
                                <1>
                                <1> load_gfx_8_8_chars:
```

```
                                <1>     ; 08/08/2016
                                <1>     ; 10/07/2016
                                <1>     ; Setup ROM 8x14 Font for Graphics Mode (VGA)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_load_gfx_8_8_dd_chars'
                                <1>
                                <1>     ; biosfn_load_gfx_8_8_dd_chars (BL)
                                <1>     ; /* set 0x43 INT pointer */
                                <1>     ; write_word(0x0, 0x43*4, &vgafont8);
                                <1>     ; write_word(0x0, 0x43*4+2, 0xC000);
000034BF C705[5E830100]-        <1>     mov    dword [VGA_INT43H], vgafont8
000034C5 [404D0100]             <1>
                                <1>
                                <1>     ; BL    screen rows code: 00H = user-specified (in DL)
                                <1>         ;                         01H = 14 rows
                                <1>         ;                         02H = 25 rows
                                <1>         ;                         03H = 43 rows
                                <1>         ; DL    (when BL=0) custom number of char rows on screen
                                <1>
                                <1>     ; switch (BL) {
                                <1>     ; case 0:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, DL-1);
                                <1>     ;    break;
000034C9 20DB                   <1>     and    bl, bl
000034CB 7508                   <1>     jnz    short l_gfx_8_8c_1
000034CD 8815[36680000]         <1>     mov    [VGA_ROWS], dl  ; not DL-1 !
000034D3 EB23                   <1>     jmp    short l_gfx_8_8c_4
                                <1> l_gfx_8_8c_1:
                                <1>     ; case 1:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 13);
                                <1>     ;    break;
000034D5 FECB                   <1>     dec    bl
000034D7 7509                   <1>     jnz    short l_gfx_8_8c_2
                                <1>     ; bl = 1
000034D9 C605[36680000]0E       <1>     mov    byte [VGA_ROWS], 14  ; not 13 !
000034E0 EB16                   <1>     jmp    short l_gfx_8_8c_4
                                <1> l_gfx_8_8c_2:
000034E2 FECB                   <1>     dec    bl
000034E4 740B                   <1>     jz     short l_gfx_8_8c_3 ; bl = 2
000034E6 FECB                   <1>     dec    bl
000034E8 750E                   <1>     jnz    short l_gfx_8_8c_4 ; bl > 3
                                <1>     ; bl = 3
                                <1>     ; case 3:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 42);
                                <1>     ;    break;
000034EA C605[36680000]2B       <1>     mov    byte [VGA_ROWS], 43  ; not 42 !
                                <1> l_gfx_8_8c_3:
                                <1>     ; case 2:
                                <1>     ; default:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 24);
                                <1>     ;    break;
                                <1>     ; bl = 2 or bl > 3
000034F1 C605[36680000]19       <1>     mov    byte [VGA_ROWS], 25  ; not 24 !
                                <1>     ; }
                                <1> l_gfx_8_8c_4:
                                <1>     ; write_byte(BIOSMEM_SEG, BIOSMEM_CHAR_HEIGHT, 8);
000034F8 C605[32680000]08       <1>         mov     byte [CHAR_HEIGHT], 8
                                <1>     ; }
000034FF C3                     <1>     retn
                                <1>
                                <1> load_gfx_8_16_chars:
                                <1>     ; 08/08/2016
                                <1>     ; 10/07/2016
                                <1>     ; Setup ROM 8x14 Font for Graphics Mode (VGA)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_load_gfx_8_16_chars'
                                <1>
                                <1>     ; biosfn_load_gfx_8_16_chars (BL)
                                <1>     ; /* set 0x43 INT pointer */
                                <1>     ; write_word(0x0, 0x43*4, &vgafont16);
                                <1>     ; write_word(0x0, 0x43*4+2, 0xC000);
00003500 C705[5E830100]-        <1>     mov    dword [VGA_INT43H], vgafont16
00003506 [40630100]             <1>
                                <1>
                                <1>     ; BL    screen rows code: 00H = user-specified (in DL)
                                <1>         ;                         01H = 14 rows
                                <1>         ;                         02H = 25 rows
                                <1>         ;                         03H = 43 rows
                                <1>         ; DL    (when BL=0) custom number of char rows on screen
                                <1>
                                <1>     ; switch (BL) {
                                <1>     ; case 0:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, DL-1);
                                <1>     ;    break;
0000350A 20DB                   <1>     and    bl, bl
0000350C 7508                   <1>     jnz    short l_gfx_8_16c_1
0000350E 8815[36680000]         <1>     mov    [VGA_ROWS], dl  ; not DL-1 !
00003514 EB23                   <1>     jmp    short l_gfx_8_16c_4
                                <1> l_gfx_8_16c_1:
                                <1>     ; case 1:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 13);
                                <1>     ;    break;
00003516 FECB                   <1>     dec    bl
00003518 7509                   <1>     jnz    short l_gfx_8_16c_2
                                <1>     ; bl = 1
0000351A C605[36680000]0E       <1>     mov    byte [VGA_ROWS], 14  ; not 13 !
00003521 EB16                   <1>     jmp    short l_gfx_8_16c_4
                                <1> l_gfx_8_16c_2:
00003523 FECB                   <1>     dec    bl
00003525 740B                   <1>     jz     short l_gfx_8_16c_3 ; bl = 2
00003527 FECB                   <1>     dec    bl
00003529 750E                   <1>     jnz    short l_gfx_8_16c_4 ; bl > 3
                                <1>     ; bl = 3
                                <1>     ; case 3:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 42);
                                <1>     ;    break;
0000352B C605[36680000]2B       <1>     mov    byte [VGA_ROWS], 43  ; not 42 !
                                <1> l_gfx_8_16c_3:
                                <1>     ; case 2:
                                <1>     ; default:
                                <1>     ;    write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, 24);
                                <1>     ;    break;
                                <1>     ; bl = 2 or bl > 3
00003532 C605[36680000]19       <1>     mov    byte [VGA_ROWS], 25  ; not 24 !
                                <1>     ; }
                                <1> l_gfx_8_16c_4:
                                <1>     ; write_byte(BIOSMEM_SEG, BIOSMEM_CHAR_HEIGHT, 16);
00003539 C605[32680000]10       <1>         mov     byte [CHAR_HEIGHT], 16
```

```
                                <1>     ; }
00003540 C3                     <1>     retn
                                <1>
                                <1> get_font_info:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 08/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; 19/09/2016
                                <1>     ; 08/08/2016
                                <1>     ; 10/07/2016
                                <1>     ; Get Current Character Generator Info (VGA)
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'biosfn_get_font_info'
                                <1>
                                <1>     ; Modified for TRDOS 386 !
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;    AX = 1130h
                                <1>     ;    BL = 0 -> Get info for current VGA font
                                <1>     ;              (BH = unused)
                                <1>     ;    19/09/2016
                                <1>     ;    BL > 0 -> Get requested character font data
                                <1>     ;        BL = 1 -> vgafont8
                                <1>     ;         BL = 2 -> vgafont14
                                <1>     ;        BL = 3 -> vgafont16
                                <1>     ;       ;08/01/2021
                                <1>     ;        BL = 4 -> user defined 8x8 font
                                <1>     ;        BL = 5 -> user defined 8x14 font
                                <1>     ;        BL = 6 -> user defined 8x16 font
                                <1>     ;        BL > 6 -> Invalid function (for now!)
                                <1>     ;        BH = ASCII code of the first character
                                <1>     ;        ECX = Number of characters from the 1st char
                                <1>     ;        ECX >= 256 -> All (256-BH) characters
                                <1>     ;        ECX = 0 -> All characters (BH = unused)
                                <1>     ;         EDX = User's Buffer Address
                                <1>     ; OUTPUT ->
                                <1>     ;    AL = height (scanlines), bytes per character
                                <1>     ;    AH = screen rows
                                <1>     ;    Byte 16-23 of EAX = number of columns
                                <1>     ;    Byte 24-31 of EAX =
                                <1>     ;        0 -> default font (not configured yet)
                                <1>     ;        0FFh -> user defined font
                                <1>     ;        14 = vgafont14
                                <1>     ;         8 = vgafont8
                                <1>     ;        16 = vgafont16
                                <1>     ;   If BL input > 0 ->
                                <1>     ;       EAX = Actual transfer count
                                <1>     ;
00003541 20DB                   <1>     and     bl, bl
00003543 740F                   <1>     jz      short gfi_1
                                <1>     ; invalid function (input)
                                <1>     ; 08/01/2021
00003545 80FB04                 <1>     cmp     bl, 4
00003548 7263                   <1>     jb      short gfi_5
0000354A 7441                   <1>     je      short gfi_3
0000354C 80FB06                 <1>     cmp     bl, 6
0000354F 744C                   <1>     je      short gfi_4
                                <1>     ; bh = 5 or bh > 6
                                <1> gfi_0:
00003551 31C0                   <1>     xor     eax, eax ; 0
00003553 C3                     <1>     retn
                                <1> gfi_1:
00003554 A0[32680000]           <1>     mov     al, [CHAR_HEIGHT]
00003559 8A25[36680000]         <1>     mov     ah, [VGA_ROWS]
0000355F C1E010                 <1>     shl     eax, 16
00003562 A0[30680000]           <1>     mov     al, [CRT_COLS]
00003567 8B0D[5E830100]         <1>     mov     ecx, [VGA_INT43H]
0000356D 21C9                   <1>     and     ecx, ecx
0000356F 7418                   <1>     jz      short gfi_2 ; 0 = default font
                                <1>     ; 08/01/2021
00003571 FECC                   <1>     dec     ah ; 0FFh
00003573 81F900400900           <1>     cmp     ecx, VGAFONT16USER
00003579 740E                   <1>     je      short gfi_2
0000357B 81F900500900           <1>     cmp     ecx, VGAFONT8USER
00003581 7406                   <1>     je      short gfi_2
00003583 8A25[32680000]         <1>     mov     ah, [CHAR_HEIGHT] ; font size = height
                                <1> gfi_2:
00003589 C1C010                 <1>     rol     eax, 16
0000358C C3                     <1>     retn
                                <1> gfi_3:
                                <1>     ; 08/01/2021
0000358D F605[869D0100]08       <1>     test    byte [ufont], 08h ; 8x8 user font
00003594 74BB                   <1>     jz      short gfi_0 ; not loaded !
00003596 BE00500900             <1>     mov     esi, VGAFONT8USER ; *
                                <1>     ;mov    bl, 8
                                <1>     ;jmp    short gfi_8
0000359B EB4A                   <1>     jmp     short gfi_10
                                <1> gfi_4:
                                <1>     ; 08/01/2021
0000359D F605[869D0100]10       <1>     test    byte [ufont], 10h ; 8x16 user font
000035A4 74AB                   <1>     jz      short gfi_0 ; not loaded !
000035A6 BE00400900             <1>     mov     esi, VGAFONT16USER ; *
000035AB EB15                   <1>     jmp     short gfi_7
                                <1> gfi_5:
000035AD 80FB02                 <1>     cmp     bl, 2
000035B0 7230                   <1>     jb      short gfi_9
000035B2 7709                   <1>     ja      short gfi_6
                                <1>     ; BL = 2 -> vgafont14
000035B4 BE[40550100]           <1>     mov     esi, vgafont14 ; *
000035B9 B30E                   <1>     mov     bl, 14
000035BB EB07                   <1>     jmp     short gfi_8
                                <1> gfi_6:
                                <1>     ; BL = 3 -> vgafont16
000035BD BE[40630100]           <1>     mov     esi, vgafont16 ; *
                                <1> gfi_7:
000035C2 B310                   <1>     mov     bl, 16
                                <1> gfi_8:
000035C4 89D7                   <1>     mov     edi, edx ; **
000035C6 09C9                   <1>     or      ecx, ecx
000035C8 7421                   <1>     jz      short gfi_11 ; all chars from the 00h
                                <1>     ;mov    al, bh ; character index
                                <1>     ; 03/08/2022
000035CA 0FB6C7                 <1>     movzx   eax, bh
000035CD F6E3                   <1>     mul     bl ; char index * char height/size
                                <1>     ;movzx  edx, ax
                                <1>     ; 03/08/2022
                                <1>     ;add    esi, edx ; *
000035CF 01C6                   <1>     add     esi, eax
000035D1 31D2                   <1>     xor     edx, edx
000035D3 FECA                   <1>     dec     dl
                                <1>     ; edx = 0FFh = 255
                                <1>     ;mov    dx, 255
```

```
000035D5 28FA                <1>     sub    dl, bh
                             <1>     ;inc   dx
                             <1>     ; 03/08/2022
000035D7 42                  <1>     inc    edx
000035D8 39D1                <1>     cmp    ecx, edx
000035DA 770F                <1>     ja     short gfi_11
000035DC 7411                <1>     je     short gfi_12
000035DE 89D1                <1>     mov    ecx, edx
000035E0 EB0D                <1>     jmp    short gfi_12
                             <1> gfi_9:
                             <1>     ;BL = 1 -> vgafont8
000035E2 BE[404D0100]        <1>     mov    esi, vgafont8 ; *
                             <1> gfi_10:
000035E7 B308                <1>     mov    bl, 8
000035E9 EBD9                <1>     jmp    short gfi_8
                             <1> gfi_11:
                             <1>     ;mov   ecx, 256
                             <1>     ; 03/08/2022
000035EB 29C9                <1>     sub    ecx, ecx
000035ED FEC5                <1>     inc    ch
                             <1>     ; ecx = 100h
                             <1> gfi_12:
                             <1>     ; 08/01/2021
000035EF 89C8                <1>     mov    eax, ecx ; character count
000035F1 30FF                <1>     xor    bh, bh
000035F3 66F7E3              <1>     mul    bx ; char count * char height/size
000035F6 89C1                <1>     mov    ecx, eax
                             <1>
                             <1>     ; ESI = source address in system space
                             <1>     ; EDI = user's buffer address
                             <1>     ; ECX = transfer (byte) count
000035F8 E81ED50000          <1>     call   transfer_to_user_buffer
000035FD 89C8                <1>     mov    eax, ecx ; actual transfer count
000035FF C3                  <1>     retn
                             <1>
                             <1> set_all_palette_reg:
                             <1>     ; 03/08/2022
                             <1>     ; 10/08/2016
                             <1>     ; Set All Palette Registers and Overscan
                             <1>     ; EDX = Address of 17 bytes;
                             <1>     ; an rgbRGB value for each of 16 palette
                             <1>     ; registers plus one for the border.
                             <1>
00003600 89D6                <1>     mov    esi, edx ; user buffer
                             <1>     ;mov   ecx, 17
                             <1>     ; 03/08/2022
00003602 29C9                <1>     sub    ecx, ecx
00003604 B111                <1>     mov    cl, 17
00003606 89E7                <1>     mov    edi, esp
00003608 83EC14              <1>     sub    esp, 20
0000360B E855D50000          <1>     call   transfer_from_user_buffer
                             <1>     ;jc    VIDEO_RETURN
                             <1>
00003610 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
00003614 EC                  <1>     in     al, dx
                             <1>     ;mov   cl, 0
                             <1>     ; 03/08/2022
00003615 28C9                <1>     sub    cl, cl
                             <1>     ;mov   dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
00003617 B2C0                <1>     mov    dl, 0C0h
                             <1> set_palette_loop:
00003619 88C8                <1>     mov    al, cl
0000361B EE                  <1>     out    dx, al
0000361C 8A07                <1>     mov    al, [edi]
0000361E EE                  <1>     out    dx, al
0000361F 47                  <1>     inc    edi
00003620 FEC1                <1>     inc    cl
00003622 80F910              <1>     cmp    cl, 10h
00003625 75F2                <1>     jne    short set_palette_loop
00003627 B011                <1>     mov    al, 11h
00003629 EE                  <1>     out    dx, al
0000362A 8A07                <1>     mov    al, [edi]
0000362C EE                  <1>     out    dx, al
0000362D B020                <1>     mov    al, 20h
0000362F EE                  <1>     out    dx, al
                             <1>     ; ifdef VBOX
                             <1>     ;mov   dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
00003630 B2DA                <1>     mov    dl, 0DAh
00003632 EC                  <1>     in     al, dx
                             <1>     ; endif ; VBOX
00003633 83C414              <1>     add    esp, 20
00003636 E930E5FFFF          <1>     jmp    VIDEO_RETURN
                             <1>
                             <1>     ; 07/08/2022
                             <1> toggle_intensity:
                             <1>     ; 03/08/2022
                             <1>     ; 10/08/2016
                             <1>     ; Select Foreground Blink or Bold Background
                             <1>     ; BL = 00h = enable bold backgrounds
                             <1>     ;          (16 background colors)
                             <1>         ;      01h = enable blinking foreground
                             <1>     ;          (8 background colors)
                             <1>
0000363B 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
0000363F EC                  <1>     in     al, dx
                             <1>     ;mov   dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
00003640 B2C0                <1>     mov    dl, 0C0h
00003642 B010                <1>     mov    al, 10h
00003644 EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C1h ; VGAREG_ACTL_READ_DATA
                             <1>     ; 03/08/2022
00003645 FEC2                <1>     inc    dl ; dx = 3C1h
00003647 EC                  <1>     in     al, dx
00003648 24F7                <1>     and    al, 0F7h
0000364A 80E301              <1>     and    bl, 01h
0000364D C0E303              <1>     shl    bl, 3
00003650 08D8                <1>     or     al, bl
                             <1>     ;mov   dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
00003652 FECA                <1>     dec    dl ; dx = 3C0h
00003654 EE                  <1>     out    dx, al
00003655 B020                <1>     mov    al, 20h
00003657 EE                  <1>     out    dx, al
                             <1>     ; ifdef VBOX
                             <1>     ;mov   dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
00003658 B2DA                <1>     mov    dl, 0DAh
0000365A EC                  <1>     in     al, dx
                             <1>     ; endif ; VBOX
0000365B E90BE5FFFF          <1>     jmp    VIDEO_RETURN
                             <1>
```

```
                                <1>     ; 07/08/2022
                                <1> vga_palf_unknown:
00003660 29C0                   <1>     sub     eax, eax ; 0 = invalid function
00003662 E909E5FFFF             <1>         jmp     _video_return
                                <1>
                                <1>     ; 07/08/2022
                                <1> vga_palf_101B:
00003667 3C1B                   <1>     cmp     al, 1Bh
                                <1>     ;jne    short vga_palf_unknown
00003669 77F5                   <1>     ja      short vga_palf_unknown
                                <1>
0000366B E810F6FFFF             <1>     call    gray_scale_summing
00003670 E9F6E4FFFF             <1>         jmp     VIDEO_RETURN
                                <1>
                                <1> vga_pal_funcs:
                                <1>     ; 07/08/2022
                                <1>     ; 10/08/2016
                                <1>     ; VGA Palette functions
                                <1>     ;
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios developers Team (2001-2008)
                                <1>     ; 'vgabios.c', 'vgarom.asm'
                                <1>
00003675 3C00                   <1>     cmp     al, 0
00003677 7467                   <1>         je      short set_single_palette_reg
                                <1> vga_palf_1002:
00003679 3C02                   <1>     cmp     al, 2
0000367B 7483                   <1>         je      short set_all_palette_reg
                                <1>     ; 07/08/2022
0000367D 7702                   <1>     ja      short vga_palf_1003
0000367F EB5D                   <1>     jmp     short set_overscan_border_color
                                <1>     ; 07/08/2022
                                <1> ;vga_palf_1001:
                                <1> ;   cmp     al, 1
                                <1> ;   je      short set_overscan_border_color
                                <1> vga_palf_1003:
00003681 3C03                   <1>     cmp     al, 3
00003683 74B6                   <1>     je      short toggle_intensity
                                <1> vga_palf_1007:
00003685 3C07                   <1>     cmp     al, 7
00003687 747F                   <1>         je      short get_single_palette_reg
00003689 72D5                   <1>     jb      short vga_palf_unknown
                                <1> vga_palf_1008:
0000368B 3C08                   <1>     cmp     al, 8
0000368D 7477                   <1>         je      short read_overscan_border_color
                                <1> vga_palf_1009:
0000368F 3C09                   <1>     cmp     al, 9
                                <1>     ;je     short get_all_palette_reg
                                <1>     ; 07/08/2022
00003691 7505                   <1>     jne     short vga_palf_1010
00003693 E966010000             <1>     jmp     get_all_palette_reg
                                <1> vga_palf_1010:
00003698 3C10                   <1>     cmp     al, 10h
                                <1>     ;je     short set_single_dac_reg
                                <1>     ; 07/08/2022
0000369A 7707                   <1>     ja      short vga_palf_1012
0000369C 72C2                   <1>     jb      short vga_palf_unknown
0000369E E908010000             <1>     jmp     set_single_dac_reg
                                <1> vga_palf_1012:
000036A3 3C12                   <1>     cmp     al, 12h
                                <1>     ;je     short set_all_dac_reg
                                <1>     ; 07/08/2022
000036A5 7707                   <1>     ja      short vga_palf_1013
000036A7 72B7                   <1>     jb      short vga_palf_unknown
000036A9 E916010000             <1>     jmp     set_all_dac_reg
                                <1> vga_palf_1013:
000036AE 3C13                   <1>     cmp     al, 13h
                                <1>     ;je     short select_video_dac_color_page
                                <1>     ; 07/08/2022
000036B0 7505                   <1>     jne     short vga_palf_1015
000036B2 E992010000             <1>     jmp     select_video_dac_color_page
                                <1> vga_palf_1015:
000036B7 3C15                   <1>     cmp     al, 15h
                                <1>     ;je     short read_single_dac_reg
                                <1>     ; 07/08/2022
000036B9 7707                   <1>     ja      short vga_palf_1017
000036BB 72A3                   <1>     jb      short vga_palf_unknown
000036BD E98D000000             <1>     jmp     read_single_dac_reg
                                <1> vga_palf_1017:
000036C2 3C17                   <1>     cmp     al, 17h
                                <1>     ;je     short read_all_dac_reg
                                <1>     ; 07/08/2022
000036C4 7707                   <1>     ja      short vga_palf_1018
000036C6 7298                   <1>     jb      short vga_palf_unknown
000036C8 E9A0000000             <1>     jmp     read_all_dac_reg
                                <1> vga_palf_1018:
000036CD 3C18                   <1>     cmp     al, 18h
000036CF 7464                   <1>         je  short set_pel_mask
                                <1> vga_palf_1019:
000036D1 3C19                   <1>     cmp     al, 19h
000036D3 746C                   <1>         je  short read_pel_mask
                                <1> vga_palf_101A:
000036D5 3C1A                   <1>     cmp     al, 1Ah
                                <1>         ;je short read_video_dac_state
                                <1>     ; 07/08/2022
000036D7 758E                   <1>     jne     short vga_palf_101B
000036D9 E9AD010000             <1>     jmp     read_video_dac_state
                                <1>
                                <1>     ; 07/08/2022
                                <1> set_overscan_border_color:
                                <1>     ; 10/08/2016
                                <1>     ; Set Overscan/Border Color Register
                                <1>     ; BH = 6-bit RGB color to display
                                <1>     ;      for that attribute
                                <1>
000036DE B311                   <1>     mov     bl, 11h
                                <1>     ; 07/08/2022
                                <1>     ;jmp    short set_single_palette_reg
                                <1>
                                <1> set_single_palette_reg:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 12/04/2021 (TRDOS 386 v2.0.3, 32 bit push/pop)
                                <1>     ; 10/08/2016
                                <1>     ; Set One Palette Register
                                <1>     ; BL = register number to set
                                <1>     ;     (a 4-bit attribute nibble: 00h-0Fh)
                                <1>     ; BH = 6-bit RGB color to display
                                <1>     ;      for that attribute
                                <1>
000036E0 80FB14                 <1>     cmp     bl, 14h
                                <1>     ;;ja    short no_actl_reg1
                                <1>     ;ja     VIDEO_RETURN
                                <1>     ; 03/08/2022
```

```
000036E3 7605                <1>     jna     short sspr_1
000036E5 E981E4FFFF          <1>     jmp     VIDEO_RETURN
                             <1> sspr_1:
                             <1>     ;push   ax
                             <1>     ;push   dx
                             <1>     ; 12/04/2021
000036EA 50                  <1>     push    eax
000036EB 52                  <1>     push    edx
000036EC 66BADA03            <1>     mov     dx, 3DAh ; VGAREG_ACTL_RESET
000036F0 EC                  <1>     in      al, dx
                             <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
000036F1 B2C0                <1>     mov     dl, 0C0h
000036F3 88D8                <1>     mov     al, bl
000036F5 EE                  <1>     out     dx, al
000036F6 88F8                <1>     mov     al, bh
000036F8 EE                  <1>     out     dx, al
000036F9 B020                <1>     mov     al, 20h
000036FB EE                  <1>     out     dx, al
                             <1>     ; ifdef VBOX
                             <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
000036FC B2DA                <1>     mov     dl, 0DAh
000036FE EC                  <1>     in      al, dx
                             <1>     ; endif ; VBOX
                             <1>     ;pop    dx
                             <1>     ;pop    ax
                             <1>     ; 12/04/2021
000036FF 5A                  <1>     pop     edx
00003700 58                  <1>     pop     eax
                             <1> ;no_actl_reg1:
00003701 E965E4FFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1>     ; 07/08/2022
                             <1> read_overscan_border_color:
                             <1>     ; 10/08/2016
                             <1>     ; Read Overscan Register
                             <1>     ; OUTPUT:
                             <1>     ; BH = current rgbRGB value
                             <1>     ;      of the overscan/border register
                             <1>
00003706 B311                <1>     mov     bl, 11h
                             <1>     ; 07/08/2022
                             <1>     ;jmp    short get_single_palette_reg
                             <1>
                             <1> get_single_palette_reg:
                             <1>     ; 03/08/2022
                             <1>     ; 10/08/2016
                             <1>     ; Read One Palette Register
                             <1>         ; INPUT:
                             <1>     ; BL = Palette register to read (00h-0Fh)
                             <1>     ; OUTPUT:
                             <1>     ; BH = Current rgbRGB value of specified register
                             <1>     ;      for that attribute
                             <1>
00003708 80FB14              <1>     cmp     bl, 14h
                             <1>     ;;ja    short no_actl_reg2
                             <1>     ;ja     VIDEO_RETURN
                             <1>     ; 03/08/2022
0000370B 7605                <1>     jna     short gspr_1
0000370D E959E4FFFF          <1>     jmp     VIDEO_RETURN
                             <1> gspr_1:
00003712 66BADA03            <1>     mov     dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
00003716 B2DA                <1>     mov     dl, 0DAh
00003718 EC                  <1>     in      al, dx
                             <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
00003719 B2C0                <1>     mov     dl, 0C0h
0000371B 88D8                <1>     mov     al, bl
0000371D EE                  <1>     out     dx, al
                             <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
                             <1>     ; 03/08/2022
0000371E FEC2                <1>     inc     dl ; dx = 3C1h
00003720 EC                  <1>     in      al, dx
00003721 8844240D            <1>     mov     [esp+13], al ; bh
                             <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
00003725 B2DA                <1>     mov     dl, 0DAh
00003727 EC                  <1>     in      al, dx
                             <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 03/08/2022
00003728 B2C0                <1>     mov     dl, 0C0h
0000372A B020                <1>     mov     al, 20h
0000372C EE                  <1>     out     dx, al
                             <1>     ; ifdef VBOX
                             <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 03/08/2022
0000372D B2DA                <1>     mov     dl, 0DAh
0000372F EC                  <1>     in      al, dx
                             <1>     ; endif ; VBOX
00003730 E936E4FFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1> set_pel_mask:
                             <1>     ; 10/08/2016
                             <1>     ; BL = mask value
00003735 66BAC603            <1>     mov     dx, 3C6h ; VGAREG_PEL_MASK
00003739 88D8                <1>     mov     al, bl
0000373B EE                  <1>     out     dx, al
0000373C E92AE4FFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1> read_pel_mask:
                             <1>     ; 10/08/2016
                             <1>     ; Output: BL = mask value
00003741 66BAC603            <1>     mov     dx, 3C6h ; VGAREG_PEL_MASK
00003745 EC                  <1>     in      al, dx
00003746 8844240C            <1>     mov     [esp+12], al ; bl
0000374A E91CE4FFFF          <1>     jmp     VIDEO_RETURN
                             <1>
                             <1> read_single_dac_reg:
                             <1>     ; 02/08/2022
                             <1>     ; 10/08/2016
                             <1>     ; Read One DAC Color Register
                             <1>     ; INPUT:
                             <1>     ; BX = color register to read (0-255)
                             <1>     ; OUTPUT:
                             <1>     ; CH = green value (00h-3Fh)
                             <1>         ; CL = blue value  (00h-3Fh)
                             <1>         ; DH = red value   (00h-3Fh)
                             <1>
0000374F 66BAC703            <1>     mov     dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
00003753 88D8                <1>     mov     al, bl
00003755 EE                  <1>     out     dx, al
                             <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
```

```
                                <1>     ; 02/08/2022
00003756 B2C9                   <1>     mov     dl, 0C9h
00003758 EC                     <1>     in      al, dx
00003759 88442415               <1>     mov     [esp+21], al ; dh
0000375D EC                     <1>     in      al, dx
0000375E 88C5                   <1>     mov     ch, al
00003760 EC                     <1>     in      al, dx
00003761 88C1                   <1>     mov     cl, al
00003763 66894C2410             <1>     mov     [esp+16], cx ; cx
00003768 E9FEE3FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> read_all_dac_reg:
                                <1>     ; 02/08/2022
                                <1>     ; 12/08/2016
                                <1>     ; 11/08/2016
                                <1>     ; 10/08/2016
                                <1>     ; Read a Block of DAC Color Registers
                                <1>         ; BX = first DAC register to read (0-00FFh)
                                <1>         ; ECX = number of registers to read (0-00FFh)
                                <1>         ; EDX = addr of a buffer to hold R,G,B values
                                <1>     ;       (CX*3 bytes long)
                                <1>
0000376D 89D7                   <1>     mov     edi, edx ; user buffer
0000376F 89CA                   <1>     mov     edx, ecx
                                <1>     ;shl    dx, 1 ; *2
                                <1>     ; 02/08/2022
00003771 D1E2                   <1>     shl     edx, 1
00003773 01CA                   <1>     add     edx, ecx ; edx = 3*ecx
00003775 89E5                   <1>     mov     ebp, esp
00003777 89EE                   <1>     mov     esi, ebp
00003779 29D6                   <1>     sub     esi, edx
0000377B 6683E6FC               <1>     and     si, 0FFFCh ; (dword alignment)
0000377F 89F4                   <1>     mov     esp, esi
00003781 52                     <1>     push    edx ; 3*ecx
00003782 66BAC703               <1>     mov     dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
00003786 88D8                   <1>     mov     al, bl
00003788 EE                     <1>     out     dx, al
00003789 66BAC903               <1>     mov     dx, 3C9h ; VGAREG_DAC_DATA
0000378D 89F3                   <1>     mov     ebx, esi
                                <1> read_dac_loop:
0000378F EC                     <1>     in      al, dx
00003790 8803                   <1>     mov     [ebx], al
00003792 43                     <1>     inc     ebx
00003793 EC                     <1>     in      al, dx
00003794 8803                   <1>     mov     [ebx], al
00003796 43                     <1>     inc     ebx
00003797 EC                     <1>     in      al, dx
00003798 8803                   <1>     mov     [ebx], al
0000379A 43                     <1>     inc     ebx
                                <1>     ;dec    cx
                                <1>     ; 02/08/2022
0000379B 49                     <1>     dec     ecx
0000379C 75F1                   <1>     jnz     short read_dac_loop
0000379E 59                     <1>     pop     ecx ; 3*ecx
                                <1>     ; ECX = transfer (byte) count
                                <1>     ; ESI = source address in system space
                                <1>     ; EDI = user's buffer address
0000379F E877D30000             <1>     call    transfer_to_user_buffer
000037A4 89EC                   <1>     mov     esp, ebp
000037A6 E9C0E3FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> set_single_dac_reg:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 12/04/2021 (TRDOS 386 v2.0.3, 32 bit push/pop)
                                <1>     ; 10/08/2016
                                <1>     ; Set One DAC Color Register
                                <1>     ; BX = color register to set (0-255)
                                <1>         ; CH = green value (00h-3Fh)
                                <1>         ; CL = blue value  (00h-3Fh)
                                <1>         ; DH = red value   (00h-3Fh)
                                <1>
                                <1>     ;push   dx
                                <1>     ; 12/04/2021
000037AB 52                     <1>     push    edx
000037AC 66BAC803               <1>     mov     dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
000037B0 88D8                   <1>     mov     al, bl
000037B2 EE                     <1>     out     dx, al
                                <1>     ;;mov   dx, 3C9h ; VGAREG_DAC_DATA
                                <1>     ;inc    dx
                                <1>     ; 03/08/2022
000037B3 FEC2                   <1>     inc     dl
                                <1>     ;pop    ax
                                <1>     ; 12/04/2021
000037B5 58                     <1>     pop     eax
000037B6 88E0                   <1>     mov     al, ah
000037B8 EE                     <1>     out     dx, al
000037B9 88E8                   <1>     mov     al, ch
000037BB EE                     <1>     out     dx, al
000037BC 88C8                   <1>     mov     al, cl
000037BE EE                     <1>     out     dx, al
000037BF E9A7E3FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> set_all_dac_reg:
                                <1>     ; 02/08/2022
                                <1>     ; 12/08/2016
                                <1>     ; 11/08/2016
                                <1>     ; 10/08/2016
                                <1>     ; Set a Block of DAC Color Register
                                <1>     ; BX = first DAC register to set (0-00FFh)
                                <1>     ; ECX = number of registers to set (0-00FFh)
                                <1>     ; EDX = addr of a table of R,G,B values
                                <1>     ;       (it will be CX*3 bytes long)
                                <1>
000037C4 89D6                   <1>     mov     esi, edx ; user buffer
000037C6 89CA                   <1>     mov     edx, ecx
                                <1>     ;shl    cx, 1 ; *2
                                <1>     ; 02/08/2022
000037C8 D1E1                   <1>     shl     ecx, 1
000037CA 01D1                   <1>     add     ecx, edx ; ecx = 3*ecx
000037CC 89E5                   <1>     mov     ebp, esp
000037CE 89EF                   <1>     mov     edi, ebp
000037D0 29CF                   <1>     sub     edi, ecx
000037D2 6683E7FC               <1>     and     di, 0FFFCh ; (dword alignment)
000037D6 89FC                   <1>     mov     esp, edi
000037D8 E888D30000             <1>     call    transfer_from_user_buffer
                                <1>     ;jc     VIDEO_RETURN
                                <1>
000037DD 89D1                   <1>     mov     ecx, edx
000037DF 66BAC803               <1>     mov     dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
000037E3 88D8                   <1>     mov     al, bl
000037E5 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
                                <1>     ; 02/08/2022
000037E6 FEC2                   <1>     inc     dl
```

```
                                <1> set_dac_loop:
000037E8 8A07                   <1>     mov     al, [edi]
000037EA EE                     <1>     out     dx, al
000037EB 47                     <1>     inc     edi
000037EC 8A07                   <1>     mov     al, [edi]
000037EE EE                     <1>     out     dx, al
000037EF 47                     <1>     inc     edi
000037F0 8A07                   <1>     mov     al, [edi]
000037F2 EE                     <1>     out     dx, al
000037F3 47                     <1>     inc     edi
                                <1>     ;dec    cx
                                <1>     ; 02/08/2022
000037F4 49                     <1>     dec     ecx
000037F5 75F1                   <1>     jnz     short set_dac_loop
000037F7 89EC                   <1>     mov     esp, ebp
000037F9 E96DE3FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> get_all_palette_reg:
                                <1>     ; 03/08/2022
                                <1>     ; 10/08/2016
                                <1>     ; Read All Palette Registers
                                <1>     ; EDX = Address of 17-byte buffer
                                <1>     ;       to receive data
                                <1>
000037FE 89D7                   <1>     mov     edi, edx
00003800 89E3                   <1>     mov     ebx, esp
00003802 89DE                   <1>     mov     esi, ebx
00003804 83EC14                 <1>     sub     esp, 20
                                <1>
00003807 B100                   <1>     mov     cl, 0
                                <1> get_palette_loop:
00003809 66BADA03               <1>     mov     dx, 3DAh ; VGAREG_ACTL_RESET
0000380D EC                     <1>     in      al, dx
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 03/08/2022
0000380E B2C0                   <1>     mov     dl, 0C0h
00003810 88C8                   <1>     mov     al, cl
00003812 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
                                <1>     ; 03/08/2022
                                <1>     ;mov    dl, 0C1h
00003813 FEC2                   <1>     inc     dl
00003815 EC                     <1>     in      al, dx
00003816 8803                   <1>     mov     [ebx], al
00003818 43                     <1>     inc     ebx
00003819 FEC1                   <1>     inc     cl
0000381B 80F910                 <1>     cmp     cl, 10h
0000381E 75E9                   <1>     jne     short get_palette_loop
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 03/08/2022
00003820 B2DA                   <1>     mov     dl, 0DAh
00003822 EC                     <1>     in      al, dx
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 03/08/2022
00003823 B2C0                   <1>     mov     dl, 0C0h
00003825 B011                   <1>     mov     al, 11h
00003827 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
                                <1>     ; 03/08/2022
00003828 FEC2                   <1>     inc     dl ; dx = 3C1h
0000382A EC                     <1>     in      al, dx
0000382B 8803                   <1>     mov     [ebx], al
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 03/08/2022
0000382D B2DA                   <1>     mov     dl, 0DAh
0000382F EC                     <1>     in      al, dx
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 03/08/2022
00003830 B2C0                   <1>     mov     dl, 0C0h
00003832 B020                   <1>     mov     al, 20h
00003834 EE                     <1>     out     dx, al
                                <1>     ; ifdef VBOX
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 03/08/2022
00003835 B2DA                   <1>     mov     dl, 0DAh
00003837 EC                     <1>     in      al, dx
                                <1>     ; endif ; VBOX
                                <1>
                                <1>     ;mov    ecx, 17 ; transfer (byte) count
                                <1>     ; 03/08/2022
00003838 29C9                   <1>     sub     ecx, ecx
0000383A B111                   <1>     mov     cl, 17
                                <1>
                                <1>     ; ESI = source address in system space
                                <1>     ; EDI = user's buffer address
0000383C E8DAD20000             <1>     call    transfer_to_user_buffer
                                <1>
00003841 83C414                 <1>     add     esp, 20
00003844 E922E3FFFF             <1>     jmp     VIDEO_RETURN
                                <1>
                                <1> select_video_dac_color_page:
                                <1>     ; 02/08/2022 (TRDOS 386 v2.0.5, code optimization)
                                <1>     ; 12/04/2021 (TRDOS 386 v2.0.3, 32 bit push/pop)
                                <1>     ; 10/08/2016
                                <1>     ; DAC Color Paging Functions
                                <1>     ; BL = 00H = select color paging mode
                                <1>         ;       BH = paging mode
                                <1>         ;            00h = 4 blocks of 64 registers
                                <1>         ;            01h = 16 blocks of 16 registers
                                <1>     ; BL = 01H = activate color page
                                <1>         ;       BH = DAC color page number
                                <1>         ;            00h-03h (4-page/64-reg mode)
                                <1>         ;            00h-0Fh (16-page/16-reg mode)
                                <1>
00003849 66BADA03               <1>     mov     dx, 3DAh ; VGAREG_ACTL_RESET
0000384D EC                     <1>     in      al, dx
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 02/08/2022
0000384E B240                   <1>     mov     dl, 40h
00003850 B010                   <1>     mov     al, 10h
00003852 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
                                <1>     ; 02/08/2022
00003853 FEC2                   <1>     inc     dl ; mov dl, 0C1h
00003855 EC                     <1>     in      al, dx
00003856 80E301                 <1>     and     bl, 01h
00003859 750C                   <1>     jnz     short set_dac_page
0000385B 247F                   <1>     and     al, 07Fh
0000385D C0E707                 <1>     shl     bh, 7
00003860 08F8                   <1>     or      al, bh
                                <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                                <1>     ; 02/08/2022
00003862 FECA                   <1>     dec     dl ; mov dl, 0C0h
00003864 EE                     <1>     out     dx, al
```

```
00003865 EB19                <1>     jmp    short set_actl_normal
                             <1> set_dac_page:
                             <1>     ;push   ax
                             <1>     ; 12/04/2021
00003867 50                  <1>     push   eax
00003868 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
0000386C EC                  <1>     in     al, dx
                             <1>     ;mov   dx, 3C0h ; VGAREG_ACTL_ADDRESS
                             <1>     ; 02/08/2022
0000386D B2C0                <1>     mov    dl, 0C0h
0000386F B014                <1>     mov    al, 14h
00003871 EE                  <1>     out    dx, al
                             <1>     ;pop   ax
                             <1>     ; 12/04/2021
00003872 58                  <1>     pop    eax
00003873 2480                <1>     and    al, 80h
00003875 7503                <1>     jnz    short set_dac_16_page
00003877 C0E702              <1>     shl    bh, 2
                             <1> set_dac_16_page:
0000387A 80E70F              <1>     and    bh, 0Fh
0000387D 88F8                <1>     mov    al, bh
0000387F EE                  <1>     out    dx, al
                             <1> set_actl_normal:
00003880 B020                <1>     mov    al, 20h
00003882 EE                  <1>     out    dx, al
                             <1>     ; ifdef VBOX
                             <1>     ;mov   dx, 3DAh ; VGAREG_ACTL_RESET
                             <1>     ; 02/08/2022
00003883 B2DA                <1>     mov    dl, 0DAh
00003885 EC                  <1>     in     al, dx
                             <1>     ; endif ; VBOX
00003886 E9E0E2FFFF          <1>     jmp    VIDEO_RETURN
                             <1>
                             <1> read_video_dac_state:
                             <1>     ; 10/08/2016
                             <1>     ; Query DAC Color Paging State
                             <1>     ; Output:
                             <1>     ; BH = current active DAC color page
                             <1>         ; BL = current active DAC paging mode
                             <1>
0000388B 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
0000388F EC                  <1>     in     al, dx
00003890 66BAC003            <1>     mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
00003894 B010                <1>     mov    al, 10h
00003896 EE                  <1>     out    dx, al
00003897 66BAC103            <1>     mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
0000389B EC                  <1>     in     al, dx
0000389C 88C3                <1>     mov    bl, al
0000389E C0EB07              <1>     shr    bl, 7
000038A1 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
000038A5 EC                  <1>     in     al, dx
000038A6 66BAC003            <1>     mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
000038AA B014                <1>     mov    al, 14h
000038AC EE                  <1>     out    dx, al
000038AD 66BAC103            <1>     mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
000038B1 EC                  <1>     in     al, dx
000038B2 88C7                <1>     mov    bh, al
000038B4 80E70F              <1>     and    bh, 0Fh
000038B7 F6C301              <1>     test   bl, 01
000038BA 7503                <1>     jnz    short get_dac_16_page
000038BC C0EF02              <1>     shr    bh, 2
                             <1> get_dac_16_page:
000038BF 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
000038C3 EC                  <1>     in     al, dx
000038C4 66BAC003            <1>     mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
000038C8 B020                <1>     mov    al, 20h
000038CA EE                  <1>     out    dx, al
                             <1>     ; ifdef VBOX
000038CB 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
000038CF EC                  <1>     in     al, dx
                             <1>     ; endif ; VBOX
000038D0 66895C240C          <1>     mov    [esp+12], bx ; bx
000038D5 E991E2FFFF          <1>     jmp    VIDEO_RETURN
                             <1>
                             <1> ; 23/11/2020 - TRDOS 386 v2.0.3
                             <1> ; VBE 2 BOCHS/QEMU emulator extensions
                             <1> ;   for TRDOS 386 v2 kernel (video bios)
                             <1>
                             <1> ; BOCH/QEMU VBE2 VGA BIOS code
                             <1> ;   by Jeroen Janssen (2002)
                             <1> ;   by Volker Rupper (2003-2020)
                             <1> ; vbe.c (02/01/2020)
                             <1>
                             <1> ; vbe.h (02/01/2020)
                             <1>
                             <1> VBE_DISPI_BANK_ADDRESS    equ    0A0000h
                             <1> VBE_DISPI_BANK_SIZE_KB    equ    64
                             <1>
                             <1> VBE_DISPI_MAX_XRES equ    2560
                             <1> VBE_DISPI_MAX_YRES equ    1600
                             <1>
                             <1> VBE_DISPI_IOPORT_INDEX    equ    01CEh
                             <1> VBE_DISPI_IOPORT_DATA     equ    01CFh
                             <1>
                             <1> VBE_DISPI_INDEX_ID equ    00h
                             <1> VBE_DISPI_INDEX_XRES      equ    01h
                             <1> VBE_DISPI_INDEX_YRES      equ    02h
                             <1> VBE_DISPI_INDEX_BPP       equ    03h
                             <1> VBE_DISPI_INDEX_ENABLE    equ    04h
                             <1> VBE_DISPI_INDEX_BANK      equ    05h
                             <1> VBE_DISPI_INDEX_VIRT_WIDTH equ   06h
                             <1> VBE_DISPI_INDEX_VIRT_HEIGHT equ  07h
                             <1> VBE_DISPI_INDEX_X_OFFSET equ     08h
                             <1> VBE_DISPI_INDEX_Y_OFFSET equ     09h
                             <1> VBE_DISPI_INDEX_VIDEO_MEMORY_64K equ 0Ah
                             <1> VBE_DISPI_INDEX_DDC       equ    0Bh
                             <1>
                             <1> VBE_DISPI_ID0             equ    0B0C0h
                             <1> VBE_DISPI_ID1             equ    0B0C1h
                             <1> VBE_DISPI_ID2             equ    0B0C2h
                             <1> VBE_DISPI_ID3             equ    0B0C3h
                             <1> VBE_DISPI_ID4             equ    0B0C4h
                             <1> VBE_DISPI_ID5             equ    0B0C5h
                             <1>
                             <1> VBE_DISPI_DISABLED equ    00h
                             <1> VBE_DISPI_ENABLED  equ    01h
                             <1> VBE_DISPI_GETCAPS  equ    02h
                             <1> VBE_DISPI_8BIT_DAC equ    20h
                             <1> VBE_DISPI_LFB_ENABLED     equ    40h
                             <1> VBE_DISPI_NOCLEARMEM      equ    80h
                             <1>
                             <1> VBE_DISPI_LFB_PHYSICAL_ADDRESS equ 0E0000000h
                             <1>
                             <1> ; ***
```

```
                                <1>
                                <1> ;// VBE Return Status Info
                                <1> ;// AL
                                <1> VBE_RETURN_STATUS_SUPPORTED       equ     4Fh
                                <1> VBE_RETURN_STATUS_UNSUPPORTED     equ     00h
                                <1> ;// AH
                                <1> VBE_RETURN_STATUS_SUCCESSFULL     equ     00h
                                <1> VBE_RETURN_STATUS_FAILED  equ     01h
                                <1> VBE_RETURN_STATUS_NOT_SUPPORTED   equ     02h
                                <1> VBE_RETURN_STATUS_INVALID equ     03h
                                <1>
                                <1> ;// VBE Mode Numbers
                                <1>
                                <1> VBE_MODE_VESA_DEFINED             equ     0100h
                                <1> VBE_MODE_REFRESH_RATE_USE_CRTC    equ     0800h
                                <1> VBE_MODE_LINEAR_FRAME_BUFFER      equ     4000h
                                <1> VBE_MODE_PRESERVE_DISPLAY_MEMORY equ     8000h
                                <1>
                                <1> ;// Mode Attributes
                                <1>
                                <1> VBE_MODE_ATTRIBUTE_SUPPORTED                equ 0001h
                                <1> VBE_MODE_ATTRIBUTE_EXTENDED_INFO_AVAILABLE equ  0002h
                                <1> VBE_MODE_ATTRIBUTE_COLOR_MODE               equ 0008h
                                <1> VBE_MODE_ATTRIBUTE_GRAPHICS_MODE     equ 0010h
                                <1> VBE_MODE_ATTRIBUTE_LINEAR_FRAME_BUFFER_MODE equ 0080h
                                <1> VBE_MODE_ATTRIBUTE_DOUBLE_SCAN_MODE         equ 0100h
                                <1> VBE_MODE_ATTRIBUTE_INTERLACE_MODE    equ 0200h
                                <1>
                                <1> ;// Window attributes
                                <1>
                                <1> VBE_WINDOW_ATTRIBUTE_RELOCATABLE equ     01h
                                <1> VBE_WINDOW_ATTRIBUTE_READABLE      equ    02h
                                <1> VBE_WINDOW_ATTRIBUTE_WRITEABLE     equ    04h
                                <1>
                                <1> ;/* Video memory */
                                <1> VGAMEM_GRAPH equ 0A000h
                                <1> VGAMEM_CTEXT equ 0B800h
                                <1> ;VGAMEM_MTEXT equ 0B000h
                                <1>
                                <1> ;// Memory model
                                <1>
                                <1> ;VBE_MEMORYMODEL_TEXT_MODE equ     00h
                                <1> ;VBE_MEMORYMODEL_CGA_GRAPHICS     equ     01h
                                <1> ;VBE_MEMORYMODEL_PLANAR           equ     03h
                                <1> VBE_MEMORYMODEL_PACKED_PIXEL      equ     04h
                                <1> ;VBE_MEMORYMODEL_NON_CHAIN_4_256 equ     05h
                                <1> VBE_MEMORYMODEL_DIRECT_COLOR      equ     06h
                                <1> ;VBE_MEMORYMODEL_YUV              equ     07h
                                <1>
                                <1> ;// DirectColorModeInfo
                                <1>
                                <1> ;VBE_DIRECTCOLOR_COLOR_RAMP_PROGRAMMABLE equ 01h
                                <1> VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE equ 02h
                                <1>
                                <1> VBE_DISPI_TOTAL_VIDEO_MEMORY_MB   equ 16
                                <1>
                                <1> ; 24/11/2020
                                <1> ; vbe.c
                                <1>
                                <1> %if 1
                                <1>
                                <1> _vbe_biosfn_return_mode_info:
                                <1>     ; 15/12/2020
                                <1>     ; 12/12/2020
                                <1>     ; Return VBE Mode Information
                                <1>     ; (call from 'sysvideo')
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       cx = video (bios) mode
                                <1>     ; Output:
                                <1>     ;       cf = 0 -> (successful)
                                <1>     ;          MODE_INFO_LIST addr contains MODEINFO
                                <1>     ;       cf = 1 -> error
                                <1>     ;
                                <1>     ; Modified registers: eax, edx, edi
                                <1>     ;
                                <1>
                                <1>     ; pushes for subroutine stack pops compatibility
                                <1>
                                <1>     ;push   ds  ; *
                                <1>     ;push   es  ; **
                                <1>
000038DA 55                     <1>     push    ebp ; ***
000038DB 56                     <1>     push    esi ; ****
                                <1>
000038DC 31FF                   <1>     xor     edi, edi  ; mov edi, 0
                                <1>
000038DE 803D[7C090000]03       <1>     cmp     byte [vbe3], 3
000038E5 7221                   <1>     jb      short _vbe_rmi_1
                                <1>
                                <1>     ;sub    edi, edi  ; 0 = kernel call (sign)
                                <1>     ;       ; no transfer to user's buffer
                                <1>
                                <1>     ; cx = Video mode (for 4F01h, with LFB flag)
                                <1>
000038E7 66B8014F               <1>     mov     ax, 4F01h
                                <1>
000038EB E820E0FFFF             <1>     call    _vbe3_pmfn_return_mode_info
                                <1>
000038F0 6683F84F               <1>     cmp     ax, 004Fh
000038F4 7533                   <1>     jne     short _vbe_rmi_2 ; fail
                                <1>
                                <1>     ; 15/12/2020
                                <1>     ; cx = vbe video mode
000038F6 80E501                 <1>     and     ch, 01h ; clear LFB flag
000038F9 BEFE7B0900             <1>     mov     esi, VBE3MODEINFOBLOCK - 2
000038FE 66890E                 <1>     mov     [esi], cx ; MODEINFO.mode
00003901 E8A5000000             <1>     call    set_lfbinfo_table
00003906 EB22                   <1>     jmp     short _vbe_rmi_3  ; cf = 0
                                <1> _vbe_rmi_1:
00003908 803D[7C090000]02       <1>     cmp     byte [vbe3], 2
0000390F 7219                   <1>     jb      short _vbe_rmi_3 ; cf = 1
00003911 A0[7D090000]           <1>     mov     al, [vbe2bios] ; 0C0h-0C5h for emu (*)
00003916 3CC0                   <1>     cmp     al, 0C0h ; BOCHS/QEMU/VIRTUALBOX (*) ?
00003918 7210                   <1>     jb      short _vbe_rmi_3  ; cf = 1
0000391A 3CC5                   <1>     cmp     al, 0C5h ; (*)
0000391C 770B                   <1>     ja      short _vbe_rmi_2 ; unknown vbios !?
                                <1>
                                <1>     ;xor    edi, edi  ; 0 = kernel call (sign)
                                <1>     ;       ; no transfer to user's buffer
                                <1>
                                <1>     ;mov    ax, 4F01h
                                <1>
                                <1>     ; cx = Video mode (for 4F01h, with LFB flag)
```

```
                                <1>
0000391E E80A000000             <1>     call    vbe_biosfn_return_mode_info
00003923 6683F84F               <1>     cmp     ax, 004Fh ; successful ?
00003927 7401                   <1>     je      short _vbe_rmi_3  ; cf = 0
                                <1> _vbe_rmi_2:
00003929 F9                     <1>     stc
                                <1>     ; cf = 1
                                <1> _vbe_rmi_3:
0000392A 5E                     <1>     pop     esi ; ****
0000392B 5D                     <1>     pop     ebp ; ***
                                <1>
                                <1>     ;pop    es  ; **
                                <1>     ;pop    ss  ; *
                                <1>
0000392C C3                     <1>     retn
                                <1>
                                <1>
                                <1> ; * (TRDOS 386, INT 31h, VESA Video Bios functions)
                                <1> ; * -------------------------------------------------------
                                <1> ; * Function 01h - Return VBE Mode Information
                                <1> ; * -------------------------------------------------------
                                <1> ; * Input:
                                <1> ; *         AX = 4F01h
                                <1> ; *         CX = Mode number
                                <1> ; *    (ES:DI) EDI = Pointer to ModeInfoBlock structure
                                <1> ; * Output:
                                <1> ; *         AX = VBE Return Status
                                <1> ; *
                                <1> ; *-------------------------------------------------------
                                <1> ; *
                                <1>
                                <1> vbe_biosfn_return_mode_info:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 15/12/2020
                                <1>     ; 14/12/2020
                                <1>     ; 12/12/2020
                                <1>     ; 11/12/2020 (TRDOS 386 v2.0.3)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       cx = video (bios) mode
                                <1>     ;       edi = ModeInfoBlock buffer address
                                <1>     ;             (in user's memory space)
                                <1>     ;       (ax = 4F01h)
                                <1>     ; Output:
                                <1>     ;       ax = 004Fh (successful)
                                <1>     ;       ah > 0 -> error
                                <1>     ;
                                <1>     ; Modified registers: esi
                                <1>
                                <1>     ;;push  ds  ; *
                                <1>     ;;push  es  ; **
                                <1>     ;;push  ebp ; ***
                                <1>     ;;push  esi ; ****
                                <1>
0000392D F6C501                 <1>     test    ch, 1
00003930 7505                   <1>     jnz     short vbe_rmi_1
                                <1>
                                <1>     ; mode number < 100h
                                <1>     ; CGA/VGA mode is not proper this VBE function
                                <1>
00003932 29C0                   <1>     sub     eax, eax
                                <1> vbe_rmi_0:
                                <1>     ;mov    ax, 0100h  ; Function is not supported
00003934 B401                   <1>     mov     ah, 1
00003936 C3                     <1>     retn
                                <1> vbe_rmi_1:
00003937 52                     <1>     push    edx ; *****
00003938 51                     <1>     push    ecx ; ******
00003939 53                     <1>     push    ebx ; *******
0000393A 57                     <1>     push    edi ; ********
                                <1>
                                <1>     ; 14/12/2020
0000393B 89CB                   <1>     mov     ebx, ecx
                                <1>
                                <1>     ;xor    eax, eax
0000393D 80E7C1                 <1>     and     bh, 0C1h ; use bit 15, 14, 8 only (for bh)
00003940 883D[259D0100]         <1>     mov     [vbe_mode_x], bh
                                <1>     ;and    bx, 1FFh
00003946 80E701                 <1>     and     bh, 1
                                <1>     ;mov    bh, 1
                                <1>
                                <1>     ; Alternative 2 (instead of 'Mode_info_find_mode')
00003949 E86A060000             <1>     call    set_mode_info_list ; (alternative 2)
                                <1>
                                <1>     ; eax = 0
                                <1>
                                <1>     ;mov    bx, [esi] ; mode
                                <1>
                                <1>     ; Alternative 1 (instead of 'set_mode_info_list')
                                <1>     ;call   mode_info_find_mode ; (alternative 1)
                                <1>
0000394E 09F6                   <1>     or      esi, esi
                                <1>     ; 14/12/2020
00003950 744C                   <1>     jz      short vbe_rmi_4       ; VBE mode number is wrong
                                <1>                                   ; or it is not supported
                                <1>
                                <1>     ; 15/12/2020
                                <1>     ;mov    bx, [esi] ; mode
                                <1>
                                <1>     ; 12/12/2020
                                <1>     ;call   set_lfbinfo_table
                                <1>
00003952 F605[259D0100]40       <1>     test    byte [vbe_mode_x], 40h ; LFB model ?
00003959 7404                   <1>     jz      short vbe_rmi_2
                                <1>
0000395B C6461C01               <1>     mov     byte [esi+MODEINFO.NumberOfBanks], 1
                                <1> vbe_rmi_2:
                                <1>     ; (vbe.c, 02/01/2020, vruppert)
                                <1>     ; 11/12/2020 (Erdogan Tan, video.s)
                                <1>     ; Bochs Graphics Adapter
                                <1>     ; vendor_id: 1111h, device id: 1234h
                                <1>
0000395F E835070000             <1>     call    pci_get_lfb_addr
                                <1>     ;or     eax, eax
00003964 7404                   <1>     jz      short vbe_rmi_3
                                <1>     ; zf = 0, ax > 0 (high word of LFB address)
                                <1>     ; set/change LFB address in MODEINFO structure
00003966 6689462C               <1>     mov     [esi+MODEINFO.PhysBasePtr+2], ax
                                <1>     ; 12/12/2020
                                <1>     ;mov    [edi+LFBINFO.LFB_addr+2], ax
                                <1> vbe_rmi_3:
                                <1>     ;test   byte [esi+MODEINFO.WinAAttributes], 1
                                <1>     ;               ; VBE_WINDOW_ATTRIBUTE_RELOCATABLE = 1
                                <1>         ;jz short vbe_rmi_4
```

```
                                <1>     ;; 11/12/2020
                                <1>     ;; In fact, this is far call address in (Bochs/BGA) Video Bios
                                <1>     ;; Direct user access to kernel subroutines is not possible
                                <1>     ;; in TRDOS 386. Also, TRDOS 386 kernel will support only LFB.
                                <1>     ;; Bank select may be a seperate sysvideo function in future
                                <1>     ;; (if it will be required).
                                <1>     ;mov    dword [esi+MODEINFO.WinFuncPtr], dispi_set_bank_farcall
                                <1> ;vbe_rmi_4:
                                <1>     ; 12/12/2020
0000396A E83C000000             <1>     call    set_lfbinfo_table
                                <1>
                                <1>     ; 11/12/2020
                                <1>     ; copy 68 bytes of MODE_INFO_LIST to user
                                <1>
0000396F 8B3C24                 <1>     mov     edi, [esp]  ; user's buffer address
                                <1>     ; 12/12/2020
00003972 09FF                   <1>     or      edi, edi ; 0 = kernel call
                                <1>                      ; (call from '_vbe_biosfn_return_mode_info')
00003974 7431                   <1>     jz      short vbe_rmi_6
                                <1>
                                <1>     ; 15/12/2020
                                <1>     ; prepare 256 bytes MODEINFO buffer at VBE3MODEINFOBLOCK
                                <1>     ; and then, copy buffer conttent to user's buffer
00003976 57                     <1>     push    edi
00003977 BE[449D0100]           <1>     mov     esi, MODE_INFO_LIST + 2       ; MODEINFO.ModeAttributes
0000397C BF007C0900             <1>     mov     edi, VBE3MODEINFOBLOCK
                                <1>     ;mov    ecx, 66/4  ; 66 bytes
                                <1>     ; 03/08/2022
00003981 29C9                   <1>     sub     ecx, ecx
00003983 B110                   <1>     mov     cl, 66/4
00003985 F3A5                   <1>     rep     movsd
00003987 31C0                   <1>     xor     eax, eax
00003989 B12F                   <1>     mov     cl, (256-68)/4 ; 188 bytes
0000398B F3AB                   <1>     rep     stosd
0000398D 66AB                   <1>     stosw   ; 2 bytes
0000398F 5F                     <1>     pop     edi
00003990 BE007C0900             <1>     mov     esi, VBE3MODEINFOBLOCK
                                <1>     ;mov    cx, 256
00003995 FEC5                   <1>     inc     ch ; cx = 256
00003997 E87FD10000             <1>     call    transfer_to_user_buffer
0000399C 7309                   <1>     jnc     short vbe_rmi_5
                                <1> vbe_rmi_4:
                                <1>     ;mov    eax, 014Fh ; fail/error
0000399E 31C0                   <1>     xor     eax, eax
000039A0 B401                   <1>     mov     ah, 01h
                                <1>     ;jmp    short vbe_rmi_6
000039A2 E981000000             <1>     jmp     vbe_sm_ret1 ; 11/12/2020
                                <1> vbe_rmi_5:
                                <1>     ; 256 bytes of MODEINFO have been transferred to user
                                <1>     ;mov    eax, 4Fh ; succesfull
                                <1> vbe_rmi_6: ; 12/12/2020
000039A7 31C0                   <1>     xor     eax, eax
                                <1> ;vbe_rmi_6:
000039A9 EB7D                   <1>     jmp     vbe_sm_ret1 ; 11/12/2020
                                <1>
                                <1>     ;pop    edi ; ********
                                <1>     ;pop    ebx ; *******
                                <1>     ;pop    ecx ; ******
                                <1>     ;pop    edx ; *****
                                <1>
                                <1>     ;;pop   esi ; ****
                                <1>     ;;pop   ebp ; ***
                                <1>     ;;pop   es  ; **
                                <1>     ;;pop   ds  ; *
                                <1>
                                <1>     ;retn
                                <1>
                                <1> set_lfbinfo_table:
                                <1>     ; 19/12/2020
                                <1>     ; 11/12/2020
                                <1>     ; Set/Fill LFBINFO structure/table
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       esi = Mode info list address
                                <1>     ; Output:
                                <1>     ;       LFB_Info address is filled with LFBINFO
                                <1>     ;       edi = LFB_Info address
                                <1>     ;
                                <1>     ; Modified registers: eax, edx (=0), edi
                                <1>
000039AB BF[329D0100]           <1>     mov     edi, LFB_Info
000039B0 8B462A                 <1>     mov     eax, [esi+MODEINFO.PhysBasePtr]
000039B3 894702                 <1>     mov     [edi+LFBINFO.LFB_addr], eax ; LFB address
                                <1>     ;mov    ax, [esi+MODEINFO.mode]
000039B6 668B06                 <1>     mov     ax, [esi]
000039B9 668907                 <1>     mov     [edi+LFBINFO.mode],ax
000039BC 8A461B                 <1>     mov     al, [esi+MODEINFO.BitsPerPixel]
000039BF 88470E                 <1>     mov     [edi+LFBINFO.bpp], al
000039C2 29C0                   <1>     sub     eax, eax
000039C4 668B4614               <1>     mov     ax, [esi+MODEINFO.XResolution]
000039C8 6689470A               <1>     mov     [edi+LFBINFO.X_res], ax
000039CC 89C2                   <1>     mov     edx, eax ; 19/12/2020
000039CE 668B4616               <1>     mov     ax, [esi+MODEINFO.YResolution]
000039D2 6689470C               <1>     mov     [edi+LFBINFO.Y_res], ax
                                <1>     ; eax = Y_res ; screen height
                                <1>     ; 19/12/2020
000039D6 F7E2                   <1>     mul     edx ; X_res*Y_res
                                <1>     ; edx = 0
000039D8 8A570E                 <1>     mov     dl, [edi+LFBINFO.bpp]
                                <1>     ; Note:
                                <1>     ; Bits per pixel may be 8,16,24,32 for TRDOS 386 v2.
                                <1>     ; (4 bits for pixel is not used for VESA modes here)
000039DB C0EA03                 <1>     shr     dl, 3 ; convert bits to byte
000039DE F7E2                   <1>     mul     edx
                                <1>     ; eax = screen/page/buffer size in bytes
000039E0 894706                 <1>     mov     [edi+LFBINFO.LFB_size], eax
                                <1>     ; edx = 0
                                <1>     ; clear reserved byte in LFBINFO structure/table
000039E3 88570F                 <1>     mov     [edi+LFBINFO.reserved], dl ; not necessary
000039E6 C3                     <1>     retn
                                <1>
                                <1> ; * (TRDOS 386, INT 31h, VESA Video Bios functions)
                                <1> ; * ------------------------------------------------------
                                <1> ; * Function 02h - Set VBE Mode
                                <1> ; * ------------------------------------------------------
                                <1> ; * Input:
                                <1> ; *         AX = 4F02h
                                <1> ; *         BX = Desired Mode to set
                                <1> ; * Output:
                                <1> ; *         AX = VBE Return Status
                                <1> ; *
                                <1> ; *------------------------------------------------------
                                <1> ; *
                                <1>
```

```
                                <1> vbe_biosfn_set_mode:
                                <1>     ; 07/03/2021
                                <1>     ; 12/12/2020
                                <1>     ; 11/12/2020 (LFBINFO table for VESA VBE modes)
                                <1>     ; 27/11/2020
                                <1>     ; 25/11/2020
                                <1>     ; 23/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; (ref: vbe.c, 02/01/2020, vruppert)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       bx = video (bios) mode
                                <1>     ;       ax = 4F02h
                                <1>     ; Output:
                                <1>     ;       ax = 004Fh (successful)
                                <1>     ;       ah > 0 -> error
                                <1>     ;
                                <1>     ; Modified registers: esi
                                <1>
                                <1>     ; 27/11/2020
                                <1>
                                <1>     ;;push ds  ; *
                                <1>     ;;push es  ; **
                                <1>     ;;push ebp ; ***
                                <1>     ;;push esi ; ****
                                <1>
                                <1>     ; 11/12/2020
000039E7 52                     <1>     push    edx ; *****
000039E8 51                     <1>     push    ecx ; ******
000039E9 53                     <1>     push    ebx ; *******
000039EA 57                     <1>     push    edi ; ********
                                <1>
                                <1>     ;xor    eax, eax
000039EB 80E7C1                 <1>     and     bh, 0C1h ; use bit 15, 14, 8 only (for bh)
000039EE 883D[259D0100]         <1>     mov     [vbe_mode_x], bh
000039F4 80E701                 <1>     and     bh, 1
000039F7 753C                   <1>     jnz     short vbe_sm_3  ; VESA VBE mode
                                <1>
                                <1>     ;;test bx, 4000h ; VBE_MODE_LINEAR_FRAME_BUFFER
                                <1>     ;test   bh, 40h
                                <1>     ;jz     short vbe_sm_0
                                <1>     ;; lfb_flag
                                <1>     ;mov    al, 40h ; VBE_DISPI_LFB_ENABLED
                                <1> vbe_sm_0:
                                <1>     ; 27/11/2020
000039F9 B080                   <1>     mov     al, 80h
                                <1>     ;test   bh, 80h ; VBE_MODE_PRESERVE_DISPLAY_MEMORY
                                <1>     ;jnz    short vbe_sm_1 ; no_clear
                                <1>     ;; clear
                                <1>     ;sub    al, al ; 0
000039FB 8405[259D0100]         <1>     test    [vbe_mode_x], al ; 80h
00003A01 7402                   <1>     jz      short vbe_sm_1 ; clear display memory
                                <1>     ; no_clear
00003A03 08C3                   <1>     or      bl, al ; VBE_MODE_PRESERVE_DISPLAY_MEMORY
                                <1> vbe_sm_1:
                                <1>     ; check non vesa mode
                                <1>     ;;cmp  bx, 100h ; VBE_MODE_VESA_DEFINED
                                <1>     ;;jna  short vbe_sm_2
                                <1>     ;and   bh, 1
                                <1>     ;jnz   short vbe_sm_3
                                <1>
                                <1>     ; BX <= 1FFh
                                <1>
                                <1>     ; 27/11/2020
                                <1>     ;or    bl, al  ; al = 80h if no_clear option is set
                                <1>     ;              ; al = 0 if no_clear option is not set
                                <1>
                                <1>     ; 25/11/2020
                                <1>     ; VBE DISPI will be disabled in 'biosfn_set_video_mode'
                                <1>
                                <1>     ;xor   al, al ; 0 ; VBE_DISPI_DISABLED
                                <1>     ;call  dispi_set_enable
                                <1>
                                <1>     ; call the vgabios in order to set the video mode
                                <1>     ; this allows for going back to textmode with a VBE call
                                <1>     ; (some applications expect that to work)
                                <1>
                                <1>     ;and   bx, 0FFh
                                <1>
                                <1>     ; 27/11/2020
                                <1> biosfn_set_video_mode:
                                <1>     ; _call: call subroutine
                                <1>     ; 26/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; (ref: vgabios.c, 02/01/2020, vruppert)
                                <1>     ; Input:
                                <1>     ;       bl = VGA video (bios) mode
                                <1>     ; Output:
                                <1>     ;       cf = 1 -> error
                                <1>     ;       cf = 0 -> ok
                                <1>     ;
                                <1>     ; Modified registers: esi
                                <1>
                                <1>     ; 'dispi_set_enable(VBE_DISPI_DISABLED);'
                                <1>
                                <1>     ;mov   ax, 0 ; VBE_DISPI_DISABLED
00003A05 31C0                   <1>     xor     eax, eax ; 0
00003A07 E89C040000             <1>     call    dispi_set_enable
                                <1>
00003A0C 88D8                   <1>     mov     al, bl
                                <1>     ;jmp   _set_mode ; (in 'biosfn_set_video_mode' sub)
00003A0E E872E1FFFF             <1>     call    _set_mode ; will return with cf=1 only if
                                <1>                     ; desired mode is not implemented
                                <1>     ; _retn: return from subroutine
00003A13 721A                   <1>     jc      short vbe_sm_2 ; 25/11/2020
                                <1>
                                <1>     ; 26/11/2020
00003A15 31C0                   <1>     xor     eax, eax
00003A17 A0[2E680000]           <1>     mov     al, [CRT_MODE]
                                <1>     ; 27/11/2020
00003A1C 8A25[4B830100]         <1>     mov     ah, [noclearmem] ; 80h or 0
                                <1>     ;and   ah 80h
00003A22 66A3[269D0100]         <1>     mov     [video_mode], ax ; bit 15 = no_clear flag
                                <1>                              ; bit 14 = 0 (not LFB model)
                                <1> vbe_sm_ret1:
                                <1>     ; 11/12/2020
                                <1>     ; (vbe_rmi_4 and vbe_rmi_6 jump here)
                                <1>     ; 27/11/2020
00003A28 B04F                   <1>     mov     al, 4Fh ; Function call successful
                                <1>     ; eax = 004Fh
                                <1> vbe_sm_ret2:
                                <1>     ; 11/12/2020
00003A2A 5F                     <1>     pop     edi ; ********
00003A2B 5B                     <1>     pop     ebx ; *******
00003A2C 59                     <1>     pop     ecx ; ******
00003A2D 5A                     <1>     pop     edx ; *****
```

```
                                <1>
                                <1>     ;;pop  esi ; ****
                                <1>     ;;pop  ebp ; ***
                                <1>     ;;pop  es  ; **
                                <1>     ;;pop  ds  ; *
                                <1>
00003A2E C3                     <1>     retn
                                <1>
                                <1> vbe_sm_2:
                                <1>     ;mov   ax, 0100h ; Function is not supported
                                <1>     ; 27/11/2020
00003A2F 31C0                   <1>     xor    eax, eax
00003A31 B401                   <1>     mov    ah, 01h
                                <1>     ; eax = 0100h
                                <1>     ;retn
00003A33 EBF5                   <1>     jmp    short vbe_sm_ret2
                                <1>
                                <1> vbe_sm_3:
                                <1>     ; 12/12/2020
                                <1>     ; check current mode, if it is 03h
                                <1>     ; save page contents and cursor positions
00003A35 803D[2E680000]03       <1>     cmp    byte [CRT_MODE], 03h
                                <1>     ;jne   short vbe_sm_0
00003A3C 7505                   <1>     jne    short vbe_sm_4 ; 07/03/2021
00003A3E E8E4E3FFFF             <1>     call   save_mode3_multiscreen
                                <1>     ; set current mode to extended (SVGA) mode
                                <1>     ;mov   byte [CRT_MODE], 0FFh ; VESA VBE mode
                                <1> vbe_sm_4:
                                <1>     ; 27/11/2020
                                <1>     ; bx = mode (bit 0 to 8)
                                <1>
                                <1>     ; 25/11/2020
                                <1>
                                <1>     ; Alternative 2 (instead of 'Mode_info_find_mode')
                                <1>     ;push  edi
00003A43 E870050000             <1>     call   set_mode_info_list ; (alternative 2)
                                <1>     ;pop   edi
                                <1>
                                <1>     ;mov   bx, [esi] ; mode
                                <1>
                                <1>     ; Alternative 1 (instead of 'set_mode_info_list')
                                <1>     ;call  mode_info_find_mode ; (alternative 1)
                                <1>
00003A48 09F6                   <1>     or     esi, esi
00003A4A 74E3                   <1>     jz     short vbe_sm_2 ; VBE mode number is wrong
                                <1>                           ; or it is not supported
                                <1>
                                <1>     ; 11/12/2020
00003A4C 668B1E                 <1>     mov    bx, [esi] ; mode
                                <1>
                                <1>     ; 27/11/2020
00003A4F 0A3D[259D0100]         <1>     or     bh, [vbe_mode_x]
                                <1>
                                <1>     ; save VESA VBE mode
00003A55 66891D[269D0100]       <1>     mov    [video_mode], bx
                                <1>            ; 27/11/2020
                                <1>            ; bit 0 to 8 = VESA VBE mode
                                <1>            ; bit 9 to 13 = 0 (bit 0 to 13 = mode)
                                <1>            ; bit 14 = Linear/Flat Frame Buffer flag
                                <1>            ; bit 15 = 'memory not cleared
                                <1>            ;         at last mode set' flag
                                <1>
                                <1>     ; first disable current mode
                                <1>     ; (when switching between vesa modes)
                                <1>     ; 'dispi_set_enable(VBE_DISPI_DISABLED);'
                                <1>
                                <1>     ;mov   ax, VBE_DISPI_DISABLED ; 0
00003A5C 29C0                   <1>     sub    eax, eax ; 0
                                <1>
00003A5E E845040000             <1>     call   dispi_set_enable
                                <1>
                                <1>     ; 11/12/2020
00003A63 8A461B                 <1>     mov    al, [esi+MODEINFO.BitsPerPixel]
                                <1>     ; ah = 0
                                <1>
                                <1>     ;cmp   byte [esi+MODEINFO.BitsPerPixel], 8
00003A66 3C08                   <1>     cmp    al, 8
00003A68 750B                   <1>     jne    short vbe_sm_5
                                <1>
                                <1>     ; 11/12/2020
                                <1>     ;push  edi
00003A6A 50                     <1>     push   eax
                                <1>     ; 'load_dac_palette(3);'
00003A6B 56                     <1>     push   esi
00003A6C B403                   <1>     mov    ah, 3  ; palette3, 256 colors
00003A6E E8BBF1FFFF             <1>     call   load_dac_palette
00003A73 5E                     <1>     pop    esi
                                <1>     ; 11/12/2020
00003A74 58                     <1>     pop    eax
                                <1>     ;pop   edi
                                <1> vbe_sm_5:
                                <1>     ;'dispi_set_bpp(cur_info->info.BitsPerPixel);'
                                <1>     ; 11/12/2020 (al = bits per pixel, ah = 0)
                                <1>     ;xor   ah, ah
                                <1>     ;mov   al, [esi+MODEINFO.BitsPerPixel]
00003A75 E841040000             <1>     call   dispi_set_bpp
                                <1>         ;'dispi_set_xres(cur_info->info.XResolution);'
00003A7A 668B4614               <1>     mov    ax, [esi+MODEINFO.XResolution]
00003A7E E83E040000             <1>     call   dispi_set_xres
                                <1>         ;'dispi_set_yres(cur_info->info.YResolution);'
00003A83 668B4616               <1>     mov    ax, [esi+MODEINFO.YResolution]
00003A87 E83B040000             <1>     call   dispi_set_yres
                                <1>
                                <1>     ;'dispi_set_bank(0);'
                                <1>     ;xor   ax, ax
00003A8C 31C0                   <1>     xor    eax, eax ; 0
00003A8E E83A040000             <1>     call   dispi_set_bank
                                <1>         ;'dispi_set_enable(VBE_DISPI_ENABLED|no_clear|lfb_flag);'
                                <1>     ;mov   ax, di
                                <1>     ; ah = 0 ; 27/11/2020
00003A93 A0[259D0100]           <1>     mov    al, [vbe_mode_x] ; restore VBE mode bit 14 & 15
00003A98 0C01                   <1>     or     al, 1 ; VBE_DISPI_ENABLED
00003A9A E809040000             <1>     call   dispi_set_enable
                                <1>
                                <1>         ; 'vga_compat_setup();'
00003A9F E83E040000             <1>     call   vga_compat_setup
                                <1>
                                <1>     ; 11/12/2020
00003AA4 E802FFFFFF             <1>     call   set_lfbinfo_table
                                <1>
                                <1>     ; 26/11/2020
00003AA9 31C0                   <1>     xor    eax, eax
00003AAB FEC8                   <1>     dec    al
00003AAD A2[2E680000]           <1>     mov    [CRT_MODE], al ; 0FFh = VESA VBE mode sign
```

```
                                <1>
                                <1>     ; 27/11/2020
00003AB2 E971FFFFFF             <1>     jmp    vbe_sm_ret1 ; Function call successful
                                <1>
                                <1>     ; 27/11/2020
                                <1>     ;mov   al, 4Fh
                                <1>     ;      ; eax = 004Fh = Function call successful
                                <1>     ;jmp   short vbe_sm_ret2
                                <1>
                                <1> ; * (TRDOS 386, INT 31h, VESA Video Bios functions)
                                <1> ; * -------------------------------------------------------
                                <1> ; * Function 03h - Return Current VBE Mode
                                <1> ; * -------------------------------------------------------
                                <1> ; * Input:
                                <1> ; *        AX = 4F03h
                                <1> ; * Output:
                                <1> ; *        AX = VBE Return Status
                                <1> ; *        BX = Current VBE Mode
                                <1> ; *
                                <1> ; *-------------------------------------------------------
                                <1> ; *
                                <1>
                                <1> vbe_biosfn_return_current_mode:
                                <1>     ; 11/12/2020
                                <1>     ; 27/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; (ref: vbe.c, 02/01/2020, vruppert)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;      none
                                <1>     ; Output:
                                <1>     ;      ax = 004Fh (successful)
                                <1>     ;      ah > 0 -> error
                                <1>     ;      bx = current video (bios) mode (if ah = 0)
                                <1>     ;
                                <1>     ; Modified registers: eax, ebx
                                <1>
                                <1>     ; 27/11/2020
                                <1>
                                <1>     ;;push ds  ; *
                                <1>     ;;push es  ; **
                                <1>     ;;push ebp ; ***
                                <1>     ;;push esi ; ****
                                <1>
                                <1>     ;push  edx ; *****
                                <1>
                                <1>     ; (vbe.c)
                                <1>     ;call  dispi_get_enable
                                <1>     ;      ; ax = vbe display interface status
                                <1>     ;and   al, 1 ; VBE_DISPI_ENABLED
                                <1>     ;jnz   short vbe_gm_1  ; VBE graphics mode
                                <1>
00003AB7 A0[2E680000]           <1>     mov    al, [CRT_MODE] ; current cga/vga mode
00003ABC 3CFF                   <1>     cmp    al, 0FFh ; VBE extension signature
00003ABE 720E                   <1>     jb     short vbe_gm_1 ; get CGA/VGA mode
                                <1>
                                <1>     ; get VBE mode
                                <1> vbe_gm_0:
00003AC0 66A1[269D0100]         <1>     mov    ax, [video_mode]
                                <1>            ; BX bits:
                                <1>            ; bit 0 to 8 = VESA VBE video mode
                                <1>            ; bit 9 to 13 = 0
                                <1>            ; bit 14 = last mode set LFB option
                                <1>            ;         1 - linear/flat frame buffer
                                <1>            ;         0 - windowed frame buffer
                                <1>            ; bit 15 = last mode set no_clear option
                                <1>            ;         0 - video memory cleared
                                <1>            ;         1 - video memory not cleared
                                <1>
                                <1> vbe_gm_return:
                                <1>     ;pop   edx ; ******
00003AC6 0FB7D8                 <1>     movzx  ebx, ax
                                <1> ;vbe_srs_retn:
00003AC9 31C0                   <1>     xor    eax, eax ; 0
00003ACB B04F                   <1>     mov    al, 4Fh ; ax = 004Fh (successful)
00003ACD C3                     <1>     retn
                                <1>
                                <1> vbe_gm_1:
                                <1>     ; legacy (old, standard) CGA/VGA bios video mode
00003ACE 8A25[4B830100]         <1>     mov    ah, [noclearmem] ; 80h or 0
                                <1>            ; BX bits:
                                <1>            ; bit 0 to 7 = video mode
                                <1>            ; bit 8 to 13 = 0
                                <1>            ; bit 14 = 0 (not LFB mode) CGA/VGA
                                <1>            ; bit 15 = 1 if [noclearmem] = 80h
                                <1>            ;         0 if [noclearmem] = 0
00003AD4 EBF0                   <1>     jmp    short vbe_gm_return
                                <1>
                                <1> ; * (TRDOS 386, INT 31h, VESA Video Bios functions)
                                <1> ; * -------------------------------------------------------
                                <1> ; * Function 04h - Save/Restore State
                                <1> ; * -------------------------------------------------------
                                <1> ; * Input:
                                <1> ; *        AX = 4F04h
                                <1> ; *            DL = 00h Return Save/Restore State buff size
                                <1> ; *                 01h Save State
                                <1> ; *                 02h Restore State
                                <1> ; *            CX = Requested states
                                <1> ; *             bit 0 - controller hardware state
                                <1> ; *             bit 1 - BIOS data state
                                <1> ; *             bit 2 - DAC state
                                <1> ; *             bit 3 - register state
                                <1> ; *    (ES:BX) EBX = Pointer to buffer (if DL <> 00h)
                                <1> ; * Output:
                                <1> ; *        AX = VBE Return Status
                                <1> ; *        BX = Number of 64-byte blocks
                                <1> ; *             to hold the state buffer (if DL=00h)
                                <1> ; *
                                <1> ; *-------------------------------------------------------
                                <1> ; *
                                <1>
                                <1> vbe_biosfn_save_restore_state:
                                <1>     ; 23/01/2021
                                <1>     ; 16/01/2021
                                <1>     ; 14/01/2021
                                <1>     ; 13/01/2021
                                <1>     ; 12/01/2021
                                <1>     ; 11/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; (ref: vbe.c, 02/01/2020, vruppert)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;      dl = sub function
                                <1>     ;      cl = requested state
                                <1>     ;      ebx = pointer to buffer (if dl<>00h)
                                <1>     ; Output:
```

```
                                <1>     ;       ax = 004Fh (successful)
                                <1>     ;       ah > 0 -> error
                                <1>     ;       bx = Number of 64-byte blocks
                                <1>     ;            to hold the state buffer (if DL=00h)
                                <1>
                                <1>     ; Modified registers: eax, ebx, edi
                                <1>
                                <1>     ; 14/01/2021
00003AD6 09DB                   <1>     or      ebx, ebx ; user's buffer address
00003AD8 750A                   <1>     jnz     short _vbe_biosfn_save_restore_state
                                <1>
00003ADA 20D2                   <1>     and     dl, dl
00003ADC 7406                   <1>     jz      short _vbe_biosfn_save_restore_state
                                <1>
                                <1>     ; function failed
                                <1>     ;mov    eax, 0100h
                                <1>     ;xor    eax, eax
                                <1>     ;inc    ah  ; eax = 0100h
                                <1>     ; 16/01/2021
00003ADE B84F010000             <1>     mov     eax, 014Fh
00003AE3 C3                     <1>     retn
                                <1>
                                <1> _vbe_biosfn_save_restore_state:
                                <1>     ; 23/01/2021
                                <1>     ; 14/01/2021
                                <1>     ; ebx = 0 if the caller is kernel ('sysvideo')
                                <1>
                                <1>     ; 13/01/2021
00003AE4 57                     <1>     push    edi
00003AE5 52                     <1>     push    edx
00003AE6 51                     <1>     push    ecx
                                <1>
                                <1>     ; 23/01/2021
                                <1>     ; 12/01/2021
00003AE7 80FA02                 <1>     cmp     dl, 2
00003AEA 7757                   <1>     ja      short vbe_srs_7 ; 23/01/2021
                                <1>                     ; invalid sub function
00003AEC 83F90F                 <1>     cmp     ecx, 0Fh
00003AEF 7752                   <1>     ja      short vbe_srs_7 ; invalid !
                                <1>
00003AF1 20D2                   <1>     and     dl, dl
00003AF3 7515                   <1>     jnz     short vbe_srs_4
                                <1>
                                <1>     ; DL = 0
                                <1>     ; Return Save/Restore State buffer size
                                <1>
                                <1>     ;mov    ebx, ecx
                                <1>     ;shl    bl, 1
                                <1>     ;mov    bx, [ebx+vbestatebufsize]
00003AF5 E881000000             <1>     call    vbe_srs_gbs
                                <1>
                                <1> ;   ; 11/01/2021
                                <1> ;   test    cl, 8
                                <1> ;   jz      short vbe_srs_3
                                <1> ;   ; vbe_biosfn_read_video_state_size();
                                <1> ;   ; return 9 * 2;
                                <1> ;   mov     bl, 18 ; register state size
                                <1> ;vbe_srs_0:
                                <1> ;   test    cl, 1
                                <1> ;   jz      short vbe_srs_1
                                <1> ;   ; size += 0x46;
                                <1> ;   add     bl, 70 ; controller state size
                                <1> ;vbe_srs_1:
                                <1> ;   test    cl, 2
                                <1> ;   jz      short vbe_srs_2
                                <1> ;   ; size += (5 + 8 + 5) * 2 + 6;
                                <1> ;   ;add    bl, 42 ; BIOS data state size ; Bochs/Plex86
                                <1> ;   ; 12/01/2021
                                <1> ;   add     bl, 40 ; TRDOS 386 v2 VBIOS data state size
                                <1> ;vbe_srs_2:
                                <1> ;   test    cl, 4
                                <1> ;   jz      short vbe_srs_3
                                <1> ;   ; size += 3 + 256 * 3 + 1;
                                <1> ;   add     bx, 772 ; DAC state size
                                <1>
                                <1> vbe_srs_3:
00003AFA 6683C33F               <1>     add     bx, 63
00003AFE 66C1EB06               <1>     shr     bx, 6 ; / 64
                                <1>
                                <1> vbe_srs_retn:
00003B02 31C0                   <1>     xor     eax, eax ; 0
                                <1> vbe_srs_0:  ; 16/01/2021
00003B04 B04F                   <1>     mov     al, 4Fh ; ax = 004Fh (successful)
                                <1> ;vbe_srs_0:
                                <1>     ; 13/01/2021
00003B06 59                     <1>     pop     ecx
00003B07 5A                     <1>     pop     edx
00003B08 5F                     <1>     pop     edi
                                <1>
00003B09 C3                     <1>     retn
                                <1>
                                <1>     ; 23/01/2021
                                <1> ;vbe_srs_10:
                                <1>     ;; 14/01/2021
                                <1>     ; return to 'sysvideo'
                                <1>     ;mov    ebx, ecx ; transfer count
                                <1>     ;       ; (byte count for saving current video state)
                                <1>     ;jmp    short vbe_srs_retn
                                <1>
                                <1> vbe_srs_4:
                                <1>     ; 23/01/2021
00003B0A 80E10F                 <1>     and     cl, 0Fh ; 8, 4, 2, 1
00003B0D 7434                   <1>     jz      short vbe_srs_7 ; cx = 0 -> invalid !
                                <1>
00003B0F BF00760900             <1>     mov     edi, VBE3SAVERESTOREBLOCK
                                <1>
00003B14 80FA01                 <1>     cmp     dl, 1
00003B17 7730                   <1>     ja      short vbe_srs_8
                                <1>
                                <1>     ; save video state
                                <1>
00003B19 F6C107                 <1>     test    cl, 07h ; 4, 2, 1
00003B1C 740A                   <1>     jz      short vbe_srs_5  ; vbe dispi regs state
                                <1>
00003B1E E884000000             <1>     call    biosfn_save_video_state
                                <1>     ; edi = current position
                                <1>     ;       in VBE3SAVERESTOREBLOCK
                                <1>     ;       (VGA save_state offset)
                                <1>     ; modified regs: edi, eax, edx, ch
00003B23 F6C108                 <1>     test    cl, 8
00003B26 7405                   <1>     jz      short vbe_srs_6
                                <1> vbe_srs_5:
00003B28 E8AB010000             <1>     call    vbe_biosfn_save_video_state
                                <1>     ; edi = end position
```

```
                                <1>     ;       in VBE3SAVERESTOREBLOCK
                                <1>     ;       (VGA save_state offset)
                                <1>     ; modified regs: edi, eax, edx, ch
                                <1> vbe_srs_6:
                                <1>     ; 23/01/2021
00003B2D 21DB                   <1>     and     ebx, ebx
00003B2F 74D1                   <1>     jz      short vbe_srs_retn ; the caller is kernel
                                <1>
00003B31 BE00760900             <1>     mov     esi, VBE3SAVERESTOREBLOCK
00003B36 29F7                   <1>     sub     edi, esi
00003B38 89F9                   <1>     mov     ecx, edi ; transfer count in bytes
                                <1>
                                <1>     ;; 14/01/2021
                                <1>     ;and    ebx, ebx
                                <1>     ;jz     short vbe_srs_10 ; the caller is kernel
                                <1>
00003B3A 89DF                   <1>     mov     edi, ebx ; user's buffer address
00003B3C E8DACF0000             <1>     call    transfer_to_user_buffer
00003B41 73BF                   <1>     jnc     short vbe_srs_retn
                                <1> vbe_srs_7:
                                <1>     ; // function failed
                                <1>     ;mov    eax, 0100h
00003B43 31C0                   <1>     xor     eax, eax
00003B45 FEC4                   <1>     inc     ah  ; eax = 0100h
                                <1>     ; 16/01/2021
                                <1>     ; ax = 0014Fh
                                <1>     ;retn
                                <1>     ; 13/01/2021
00003B47 EBBB                   <1>     jmp     short vbe_srs_0
                                <1> vbe_srs_8:
                                <1>     ;cmp    dl, 2
                                <1>     ;jne    short vbe_srs_7
                                <1>     ;               ; invalid sub function
                                <1>
                                <1>     ; 14/01/2021
00003B49 09DB                   <1>     or      ebx, ebx ; user's buffer address
                                <1>     ;jnz    short vbe_srs_11
                                <1>
                                <1>     ; the caller is kernel ('sysvideo')
                                <1>     ;jmp    short vbe_srs_12
                                <1>     ; 23/01/2021
00003B4B 7414                   <1>     jz      short vbe_srs_12 ; 'sysvideo' call
                                <1> vbe_srs_11:
00003B4D 89DE                   <1>     mov     esi, ebx ; user's buffer address
                                <1>     ; 23/01/2021
                                <1>     ;push   ebx
                                <1>
00003B4F E827000000             <1>     call    vbe_srs_gbs
                                <1>
                                <1>     ; restore video state
                                <1>
                                <1>     ;mov    edi, VBE3SAVERESTOREBLOCK
00003B54 51                     <1>     push    ecx
00003B55 89D9                   <1>     mov     ecx, ebx ; transfer count in bytes
00003B57 E809D00000             <1>     call    transfer_from_user_buffer
00003B5C 59                     <1>     pop     ecx
                                <1>     ; 23/01/2021
                                <1>     ;pop    ebx
00003B5D 89F3                   <1>     mov     ebx, esi
00003B5F 72E2                   <1>     jc      short vbe_srs_7
                                <1>
                                <1> vbe_srs_12:
                                <1>     ;mov    esi, VBE3SAVERESTOREBLOCK
00003B61 89FE                   <1>     mov     esi, edi
                                <1>
00003B63 F6C107                 <1>     test    cl, 07h ; 4, 2, 1
00003B66 740C                   <1>     jz      short vbe_srs_9       ; vbe dispi regs state
                                <1>
00003B68 E8A2010000             <1>     call    biosfn_restore_video_state
00003B6D 72D4                   <1>     jc      short vbe_srs_7 ; invalid buffer content !
                                <1>     ; esi = current position
                                <1>     ;       in VBE3SAVERESTOREBLOCK
                                <1>     ;       (VGA save_state offset)
                                <1>     ; modified regs: esi, eax, edx, ch
00003B6F F6C108                 <1>     test    cl, 8
                                <1>     ;jz     short vbe_srs_10
                                <1>     ; 23/01/2020
00003B72 EB8E                   <1>     jmp     short vbe_srs_retn
                                <1> vbe_srs_9:
00003B74 E8F1020000             <1>     call    vbe_biosfn_restore_video_state
                                <1>
                                <1>     ; modified regs: esi, eax, edx, ch
                                <1>
00003B79 EB87                   <1>     jmp     short vbe_srs_retn
                                <1>
                                <1> ;vbe_srs_10:
                                <1> ;   ; successful
                                <1> ;   xor     eax, eax ; 0
                                <1> ;   mov     al, 4Fh ; ax = 004Fh (successful)
                                <1> ;   retn
                                <1>
                                <1> vbe_srs_gbs:
                                <1>     ; return buffer size according to flags
00003B7B 89CB                   <1>     mov     ebx, ecx ; options/flags
00003B7D D0E3                   <1>     shl     bl, 1
00003B7F 668B9B[873B0000]       <1>     mov     bx, [ebx+vbestatebufsize]
00003B86 C3                     <1>     retn
                                <1>
                                <1> vbestatebufsize:
                                <1>     ; ---------------------------------------
                                <1>     ; CL = 0  1   2    3    4    5    6    7
                                <1>     ; ---------------------------------------
00003B87 0000460028006E0004-    <1>     dw      0, 70, 40, 110, 772, 842, 812, 882
00003B90 034A032C037203         <1>
                                <1>     ; ---------------------------------------
                                <1>     ; CL = 8   9   10  11   12   13   14   15
                                <1>     ; ---------------------------------------
00003B97 120058003A00800016-    <1>     dw      18, 88, 58, 128, 790, 860, 830, 900
00003BA0 035C033E038403         <1>
                                <1>
                                <1> ; 11/01/2021
                                <1> VGAREG_ACTL_ADDRESS       equ 3C0h
                                <1> VGAREG_ACTL_WRITE_DATA    equ 3C0h
                                <1> VGAREG_ACTL_READ_DATA     equ 3C1h
                                <1>
                                <1> VGAREG_INPUT_STATUS       equ 3C2h
                                <1> VGAREG_WRITE_MISC_OUTPUT equ 3C2h
                                <1> VGAREG_VIDEO_ENABLE       equ 3C3h
                                <1> VGAREG_SEQU_ADDRESS       equ 3C4h
                                <1> VGAREG_SEQU_DATA   equ 3C5h
                                <1>
                                <1> VGAREG_PEL_MASK           equ 3C6h
                                <1> VGAREG_DAC_STATE   equ 3C7h
                                <1> VGAREG_DAC_READ_ADDRESS   equ 3C7h
```

```
                                <1> VGAREG_DAC_WRITE_ADDRESS equ 3C8h
                                <1> VGAREG_DAC_DATA           equ 3C9h
                                <1>
                                <1> VGAREG_READ_FEATURE_CTL   equ 3CAh
                                <1> VGAREG_READ_MISC_OUTPUT   equ 3CCh
                                <1>
                                <1> VGAREG_GRDC_ADDRESS       equ 3CEh
                                <1> VGAREG_GRDC_DATA   equ 3CFh
                                <1>
                                <1> ;VGAREG_MDA_CRTC_ADDRESS equ 3B4h
                                <1> ;VGAREG_MDA_CRTC_DATA     equ 3B5h
                                <1> VGAREG_VGA_CRTC_ADDRESS   equ 3D4h
                                <1> VGAREG_VGA_CRTC_DATA      equ 3D5h
                                <1>
                                <1> ;VGAREG_MDA_WRITE_FEATURE_CTL equ 3BAh
                                <1> VGAREG_VGA_WRITE_FEATURE_CTL equ 3DAh
                                <1> VGAREG_ACTL_RESET  equ 3DAh
                                <1>
                                <1> ;VGAREG_MDA_MODECTL       equ 3B8h
                                <1> VGAREG_CGA_MODECTL equ 3D8h
                                <1> VGAREG_CGA_PALETTE equ 3D9h
                                <1>
                                <1> biosfn_save_video_state:
                                <1>     ; 03/08/2022 (TRDOS 386 v2.0.5)
                                <1>     ; 22/01/2021
                                <1>     ; 12/01/2021
                                <1>     ; 11/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; (vgabios.c)
                                <1>
                                <1>     ; modified registers: eax, edx, edi, ch
                                <1>
                                <1>     ;mov   edi, VBE3SAVERESTOREBLOCK
                                <1>
                                <1>     ; input: edi = state buffer address
                                <1>
00003BA7 F6C101                 <1>     test   cl, 1
00003BAA 0F8485000000           <1>     jz     bfn_svs_4
                                <1>
00003BB0 66BAC403               <1>     mov    dx, VGAREG_SEQU_ADDRESS ; 3C7h
00003BB4 EC                     <1>     in     al, dx
00003BB5 AA                     <1>     stosb
                                <1>     ;mov   dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003BB6 B2D4                   <1>     mov    dl, 0D4h
00003BB8 EC                     <1>     in     al, dx
00003BB9 AA                     <1>     stosb
                                <1>     ;mov   dx, VGAREG_GRDC_ADDRESS ; 3CEh
00003BBA B2CE                   <1>         mov dl, 0CEh
00003BBC EC                     <1>     in     al, dx
00003BBD AA                     <1>     stosb
                                <1>     ;mov   dx, VGAREG_ACTL_RESET ; 3DAh
00003BBE B2DA                   <1>     mov    dl, 0DAh
00003BC0 EC                     <1>     in     al, dx
                                <1>     ;mov   dx, VGAREG_ACTL_ADDRESS ; 3C0h
00003BC1 B2C0                   <1>     mov    dl, 0C0h
00003BC3 EC                     <1>     in     al, dx
00003BC4 AA                     <1>     stosb
00003BC5 88C4                   <1>     mov    ah, al ; ar_index
                                <1>     ;mov   dx, VGAREG_READ_FEATURE_CTL ; 3CAh
00003BC7 B2CA                   <1>     mov    dl, 0CAh
00003BC9 EC                     <1>     in     al, dx
00003BCA AA                     <1>     stosb
                                <1>     ; (5 bytes are writen above)
                                <1>
                                <1>     ; for(i=1;i<=4;i++){
00003BCB B001                   <1>     mov    al, 1
                                <1>     ;;mov  dx, VGAREG_SEQU_ADDRESS ; 3C4h
                                <1>     ;mov   dl, 0C4h
00003BCD B504                   <1>     mov    ch, 4
                                <1> bfn_svs_0:
                                <1>     ; outb(VGAREG_SEQU_ADDRESS, i);
                                <1>     ;mov   dx, VGAREG_SEQU_ADDRESS ; 3C4h
00003BCF B2C4                   <1>     mov    dl, 0C4h
00003BD1 EE                     <1>     out    dx, al
                                <1>     ;mov   dx, VGAREG_SEQU_DATA  ; 3C5h
00003BD2 FEC2                   <1>     inc    dl  ; dx = 3C5h
                                <1>     ; inb(VGAREG_SEQU_DATA)
00003BD4 50                     <1>     push   eax
00003BD5 EC                     <1>     in     al, dx
00003BD6 AA                     <1>     stosb  ; (4 bytes in loop)
00003BD7 58                     <1>     pop    eax
                                <1>     ;mov   dx, VGAREG_SEQU_ADDRESS ; 3C4h
                                <1>     ;dec   dl
00003BD8 FEC0                   <1>     inc    al  ; i++
00003BDA FECD                   <1>     dec    ch
00003BDC 75F1                   <1>     jnz    short bfn_svs_0
                                <1>
                                <1>     ; outb(VGAREG_SEQU_ADDRESS, 0);
00003BDE 28C0                   <1>     sub    al, al ; 0
00003BE0 EE                     <1>     out    dx, al
                                <1>     ; inb(VGAREG_SEQU_DATA)
                                <1>     ;mov   dx, VGAREG_SEQU_DATA ; 3C5h
00003BE1 FEC2                   <1>     inc    dl  ; dx = 3C5h
00003BE3 EC                     <1>     in     al, dx
00003BE4 AA                     <1>     stosb  ; (+1 byte)
                                <1>
                                <1>         ; for(i=0;i<=0x18;i++) {
00003BE5 28C0                   <1>     sub    al, al ; 0
                                <1>     ;;mov  dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
                                <1>     ;mov   dl, 0D4h
00003BE7 B519                   <1>     mov    ch, 25
                                <1> bfn_svs_1:
                                <1>         ; outb(crtc_addr,i);
                                <1>     ;mov   dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003BE9 B2D4                   <1>     mov    dl, 0D4h
00003BEB EE                     <1>     out    dx, al
                                <1>     ;mov   dx, VGAREG_VGA_CRTC_DATA ; 3D5h
00003BEC FEC2                   <1>     inc    dl  ; dx = 3D5h
                                <1>     ; inb(crtc_addr+1)
00003BEE 50                     <1>     push   eax
00003BEF EC                     <1>     in     al, dx
00003BF0 AA                     <1>     stosb  ; (25 bytes in loop)
00003BF1 58                     <1>     pop    eax
                                <1>     ;mov   dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
                                <1>     ;dec   dl
00003BF2 FEC0                   <1>     inc    al  ; i++
00003BF4 FECD                   <1>     dec    ch
00003BF6 75F1                   <1>     jnz    short bfn_svs_1
                                <1>
00003BF8 80E420                 <1>     and    ah, 20h  ; (ar_index & 0x20)
                                <1>         ; for(i=0;i<=0x13;i++) {
00003BFB 28C0                   <1>     sub    al, al ; 0
00003BFD B514                   <1>     mov    ch, 20
                                <1> bfn_svs_2:
                                <1>     ; inb(VGAREG_ACTL_RESET);
```

```
                                <1>     ;mov    dx, VGAREG_ACTL_RESET ; 3DAh
00003BFF B2DA                   <1>     mov     dl, 0DAh
00003C01 50                     <1>     push    eax
00003C02 EC                     <1>     in      al, dx
00003C03 8A0424                 <1>     mov     al, [esp]
                                <1>     ; outb(VGAREG_ACTL_ADDRESS, i | (ar_index & 0x20));
00003C06 08E0                   <1>     or      al, ah
                                <1>     ;mov    dx, VGAREG_ACTL_ADDRESS ; 3C0h
00003C08 B2C0                   <1>     mov     dl, 0C0h
00003C0A EE                     <1>     out     dx, al
                                <1>     ;mov    dx, VGAREG_ACTL_READ_DATA ; 3C1h
                                <1>     ;mov    dl, 0C1h
00003C0B FEC2                   <1>     inc     dl
00003C0D EC                     <1>     in      al, dx
00003C0E AA                     <1>     stosb   ; (20 bytes in loop)
00003C0F 58                     <1>     pop     eax
00003C10 FEC0                   <1>     inc     al  ; i++
00003C12 FECD                   <1>     dec     ch
00003C14 75E9                   <1>     jnz     short bfn_svs_2
                                <1>
                                <1>     ; inb(VGAREG_ACTL_RESET);
                                <1>     ;mov    dx, VGAREG_ACTL_RESET ; 3DAh
00003C16 B2DA                   <1>     mov     dl, 0DAh
00003C18 EC                     <1>     in      al, dx
                                <1>
                                <1>         ; for(i=0;i<=8;i++) {
00003C19 28C0                   <1>     sub     al, al ; 0
                                <1>     ;;mov   dx, VGAREG_GRDC_ADDRESS ; 3CEh
                                <1>     ;mov    dl, 0CEh
00003C1B B509                   <1>     mov     ch, 9
                                <1> bfn_svs_3:
                                <1>     ; outb(VGAREG_GRDC_ADDRESS,i)
                                <1>     ;mov    dx, VGAREG_GRDC_ADDRESS ; 3CEh
00003C1D B2CE                   <1>     mov     dl, 0CEh
00003C1F EE                     <1>     out     dx, al
                                <1>     ; inb(VGAREG_ACTL_READ_DATA)
00003C20 50                     <1>     push    eax
                                <1>     ;mov    dx, VGAREG_GRDC_DATA ; 3CFh
                                <1>     ;mov    dl, 0CFh
00003C21 FEC2                   <1>     inc     dl
00003C23 EC                     <1>     in      al, dx
00003C24 AA                     <1>     stosb   ; (9 bytes in loop)
00003C25 58                     <1>     pop     eax
                                <1>     ;dec    dl
00003C26 FEC0                   <1>     inc     al  ; i++
00003C28 FECD                   <1>     dec     ch
00003C2A 75F1                   <1>     jnz     short bfn_svs_3
                                <1>
                                <1>     ; write_word(ES, BX, crtc_addr); BX+= 2;
                                <1>     ; (offset 64)
00003C2C 66B8D403               <1>     mov     ax, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
00003C30 66AB                   <1>     stosw   ; (2 bytes (1 word))
                                <1>
                                <1>         ; /* XXX: read plane latches */
00003C32 31C0                   <1>     xor     eax, eax  ; 0
00003C34 AB                     <1>             stosd   ; (4 bytes)
                                <1>
                                <1>     ; (total 70 bytes are written above as controller hardware state)
                                <1>
                                <1> bfn_svs_4:
                                <1>     ; 12/01/2021 (TRDOS 386 v2.0.3)
00003C35 F6C102                 <1>     test    cl, 2
00003C38 7476                   <1>     jz      short bfn_svs_6
                                <1>
                                <1>     ; VIDEO BIOS DATA
                                <1>     ; !!! this data is valid for TRDOS 386 v2 kernel only !!!
                                <1>     ; (this is not same with BOCHS/PLEX86 video bios, BIOS data)
                                <1>
                                <1>     ; if (CX & 2) {
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_CURRENT_MODE)); BX++;
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_NB_COLS)); BX += 2;
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_PAGE_SIZE)); BX += 2;
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_CRTC_ADDRESS)); BX += 2;
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS)); BX++;
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_CHAR_HEIGHT)); BX += 2;
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_VIDEO_CTL)); BX++;
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_SWITCHES)); BX++;
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_MODESET_CTL)); BX++;
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_CURSOR_TYPE)); BX += 2;
                                <1>         ;for(i=0;i<8;i++) {
                                <1>         ;   write_word(ES, BX, read_word(BIOSMEM_SEG, BIOSMEM_CURSOR_POS+2*i));
                                <1>         ;   BX += 2;
                                <1>         ;}
                                <1>         ;write_word(ES, BX, read_word(BIOSMEM_SEG,BIOSMEM_CURRENT_START)); BX += 2;
                                <1>         ;write_byte(ES, BX, read_byte(BIOSMEM_SEG,BIOSMEM_CURRENT_PAGE)); BX++;
                                <1>         ;/* current font */
                                <1>         ;write_word(ES, BX, read_word(0, 0x1f * 4)); BX += 2;
                                <1>         ;write_word(ES, BX, read_word(0, 0x1f * 4 + 2)); BX += 2;
                                <1>         ;write_word(ES, BX, read_word(0, 0x43 * 4)); BX += 2;
                                <1>         ;write_word(ES, BX, read_word(0, 0x43 * 4 + 2)); BX += 2;
                                <1>
                                <1>     ; !!! save TRDOS 386 v2 kernel spesific video bios data !!!
                                <1>     ; (which is/are used by 'SET_MODE' function and/or it's sub functions)
                                <1>
00003C3A 66B8D403               <1>     mov     ax, 3D4h ; CRTC_ADDR, always 3D4h (color VGA) for TRDOS 386 v2
00003C3E 66AB                   <1>     stosw
00003C40 A0[2E680000]           <1>     mov     al, [CRT_MODE] ; Current video mode (0FFh for VESA VBE modes)
00003C45 AA                     <1>     stosb
00003C46 A0[2F680000]           <1>     mov     al, [CRT_MODE_SET] ; 29h for mode 03h ; TRDOS 386 feature only !
00003C4B AA                     <1>     stosb
00003C4C 66A1[269D0100]         <1>     mov     ax, [video_mode] ; Current VESA VBE (SVGA, extended VGA) mode
00003C52 66AB                   <1>     stosw                   ; (valid if [CRT_MODE] = 0FFh)
00003C54 66A1[4C830100]         <1>     mov     ax, [CRT_LEN] ; page size (in bytes)
00003C5A 66AB                   <1>     stosw
00003C5C 66A1[CC760100]         <1>     mov     ax, [CRT_START] ; video page start offset
00003C62 66AB                   <1>     stosw
00003C64 A0[30680000]           <1>     mov     al, [CRT_COLS] ; nbcols, characters per row
00003C69 AA                     <1>     stosb
00003C6A A0[36680000]           <1>     mov     al, [VGA_ROWS] ; nbrows, (character) rows per page (not rows-1)
00003C6F AA                     <1>     stosb
00003C70 A0[32680000]           <1>     mov     al, [CHAR_HEIGHT] ; character font height (8 or 16 or 14)
00003C75 AA                     <1>     stosb
00003C76 A0[33680000]           <1>     mov     al, [VGA_VIDEO_CTL] ; ROM BIOS DATA AREA Offset 87h
00003C7B AA                     <1>     stosb
00003C7C A0[34680000]           <1>     mov     al, [VGA_SWITCHES] ; feature bit switches
00003C81 AA                     <1>     stosb
00003C82 A0[35680000]           <1>     mov     al, [VGA_MODESET_CTL] ; basic mode set options
00003C87 AA                     <1>     stosb
                                <1>     ; followings are only used by TRDOS 386 v2 (IBM PC/AT ROMBIOS) code
                                <1>     ; (bochs/plex86 does not use and return those)
00003C88 A0[31680000]           <1>     mov     al, [CRT_PALETTE] ; current color palette ; TRDOS 386 feature only !
00003C8D AA                     <1>     stosb
00003C8E A0[DE760100]           <1>     mov     al, [ACTIVE_PAGE] ; current video page
00003C93 AA                     <1>     stosb
```

```
00003C94 66A1[47680000]      <1>     mov    ax, [CURSOR_MODE] ; cursor type
00003C9A 66AB                <1>     stosw
                             <1>     ;mov   eax, [CURSOR_POSN] ; cursor position for video page 0 and 1
                             <1>     ;stosd
                             <1>     ;mov   eax, [CURSOR_POSN+4] ; cursor position for video page 2 and 3
                             <1>     ;stosd
                             <1>     ;mov   eax, [CURSOR_POSN+8] ; cursor position for video page 4 and 5
                             <1>     ;stosd
                             <1>     ;mov   eax, [CURSOR_POSN+12] ; cursor position for video page 6 and 7
                             <1>     ;stosd
00003C9C 56                  <1>     push   esi
00003C9D B504                <1>     mov    ch, 4
00003C9F BE[CE760100]        <1>     mov    esi, CURSOR_POSN
                             <1> bfn_svs_5:
00003CA4 A5                  <1>     movsd
00003CA5 FECD                <1>     dec    ch
00003CA7 75FB                <1>     jnz    short bfn_svs_5
00003CA9 5E                  <1>     pop    esi
                             <1>     ; (font addr) protected mode address in kernel's/system memory space
                             <1>     ; (not accessable/meaningful address value by user)
00003CAA A1[5E830100]        <1>     mov    eax, [VGA_INT43H] ; VGA current (default) font address
00003CAF AB                  <1>     stosd
                             <1>
                             <1>     ; (total 40 bytes are written above as BIOS data state)
                             <1>
                             <1> bfn_svs_6:
                             <1>     ; 12/01/2021
00003CB0 F6C104              <1>     test   cl, 4
00003CB3 7422                <1>     jz     short bfn_svs_8
                             <1>
                             <1>     ;/* XXX: check this */
                             <1>            ; /* read/write mode dac */
                             <1>         ;write_byte(ES, BX, inb(VGAREG_DAC_STATE)); BX++;
                             <1>     ;      ; /* pix address */
                             <1>         ;write_byte(ES, BX, inb(VGAREG_DAC_WRITE_ADDRESS)); BX++;
                             <1>         ;write_byte(ES, BX, inb(VGAREG_PEL_MASK)); BX++;
                             <1>         ;// Set the whole dac always, from 0
                             <1>         ;outb(VGAREG_DAC_WRITE_ADDRESS,0x00);
                             <1>         ;for(i=0;i<256*3;i++) {
                             <1>         ;   write_byte(ES, BX, inb(VGAREG_DAC_DATA)); BX++;
                             <1>         ;}
                             <1>         ;write_byte(ES, BX, 0); BX++; /* color select register */
                             <1>
                             <1>     ; /* read/write mode dac */
00003CB5 66BAC703            <1>     mov    dx, 3C7h ; VGAREG_DAC_STATE
00003CB9 EC                  <1>     in     al, dx
00003CBA AA                  <1>     stosb
                             <1>     ; /* pix address */
                             <1>     ;mov   dx, VGAREG_DAC_WRITE_ADDRESS ; 3C8h
                             <1>     ;mov   dl, 0C8h
00003CBB FEC2                <1>     inc    dl
00003CBD EC                  <1>     in     al, dx
00003CBE AA                  <1>     stosb
                             <1>     ;mov   dx, VGAREG_PEL_MASK  ; 3C6h
00003CBF B2C6                <1>     mov    dl, 0C6h
00003CC1 EC                  <1>     in     al, dx
00003CC2 AA                  <1>     stosb
                             <1>     ;// Set the whole dac always, from 0
00003CC3 30C0                <1>     xor    al, al ; 0
                             <1>     ;mov   dx, VGAREG_DAC_WRITE_ADDRESS ; 3C8h
00003CC5 B2C8                <1>     mov    dl, 0C8h
00003CC7 EE                  <1>     out    dx, al
                             <1>
00003CC8 51                  <1>     push   ecx ; 22/01/2021
                             <1>     ;for(i=0;i<256*3;i++) {
                             <1>     ;mov   ecx, 256*3 ; 768 bytes
                             <1>     ; 03/08/2022
00003CC9 29C9                <1>     sub    ecx, ecx
00003CCB B503                <1>     mov    ch, 3
                             <1>     ; ecx = 300h = 768
                             <1>     ;mov   dx, VGAREG_DAC_DATA ; 3C9h
                             <1>     ;mov   dl, 0C9h
00003CCD FEC2                <1>     inc    dl ; dx = 3C9h
                             <1> bfn_svs_7:
00003CCF EC                  <1>     in     al, dx
00003CD0 AA                  <1>     stosb
00003CD1 E2FC                <1>     loop   bfn_svs_7
00003CD3 59                  <1>     pop    ecx ; 22/01/2021
                             <1>
                             <1>     ; /* color select register */
00003CD4 28C0                <1>     sub    al, al ; 0
00003CD6 AA                  <1>     stosb
                             <1>
                             <1>     ; (total 772 bytes are written above as DAC state)
                             <1> bfn_svs_8:
00003CD7 C3                  <1>     retn
                             <1>
                             <1> vbe_biosfn_save_video_state:
                             <1>     ; 23/01/2021
                             <1>     ; 13/01/2021
                             <1>     ; 12/01/2021 (TRDOS 386 v2.0.3)
                             <1>     ; (vbe.c)
                             <1>
                             <1>     ; modified registers: eax, edx, edi, ch
                             <1>
                             <1>     ; input: edi = state buffer address
                             <1>     ; output:
                             <1>     ;       VBE DISPI register contents will be saved
                             <1>     ;       (18 bytes, 9 words)
                             <1>
                             <1>     ; outw(VBE_DISPI_IOPORT_INDEX,VBE_DISPI_INDEX_ENABLE);
                             <1>     ; enable = inw(VBE_DISPI_IOPORT_DATA);
                             <1>     ; write_word(ES, BX, enable);
                             <1>     ; BX += 2;
                             <1>     ; if (!(enable & VBE_DISPI_ENABLED))
                             <1>     ;      return;
                             <1>     ; for(i = VBE_DISPI_INDEX_XRES;
                             <1>     ;              i <= VBE_DISPI_INDEX_Y_OFFSET; i++) {
                             <1>     ;    if (i != VBE_DISPI_INDEX_ENABLE) {
                             <1>     ;        outw(VBE_DISPI_IOPORT_INDEX, i);
                             <1>     ;        write_word(ES, BX, inw(VBE_DISPI_IOPORT_DATA));
                             <1>         ;    BX += 2;
                             <1>     ;        }
                             <1>     ; }
                             <1>
00003CD8 66BACE01            <1>     mov    dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                             <1>     ;mov   eax, 04h  ; VBE_DISPI_INDEX_ENABLE
                             <1>      ;03/08/2022
00003CDC 66EF                <1>     out    dx, ax
                             <1>     ;mov   dx, 01CFh ; VBE_DISPI_IOPORT_DATA
00003CDE FEC2                <1>     inc    dl
00003CE0 66ED                <1>     in     ax, dx ; enable (status)
00003CE2 66AB                <1>     stosw
00003CE4 6683E001            <1>     and    ax, 1 ; VBE_DISPI_ENABLED
```

```
00003CE8 7505                <1>     jnz    short vbe_bfn_svs_0
                             <1>     ; 23/01/2021
                             <1>     ; ax = 0
                             <1>     ; VBE_DISPI_DISABLED
                             <1>     ; 13/01/2021
                             <1>     ; clear remain 8 bytes
                             <1>     ;xor   eax, eax
00003CEA AB                  <1>     stosd  ; 2
00003CEB AB                  <1>     stosd  ; 2
00003CEC AB                  <1>     stosd  ; 2
00003CED AB                  <1>     stosd  ; 2
00003CEE C3                  <1>     retn
                             <1> vbe_bfn_svs_0:
                             <1>     ; VBE_DISPI_ENABLED
                             <1>
                             <1>     ;sub   eax, eax
00003CEF 28C0                <1>     sub    al, al ; eax = 0
                             <1>
                             <1>     ; from VBE_DISPI_INDEX_XRES
                             <1>     ;   to VBE_DISPI_INDEX_BPP
                             <1>
00003CF1 B503                <1>     mov    ch, 3
                             <1>     ; al = 0 ; VBE_DISPI_INDEX_XRES - 1
                             <1>
00003CF3 E804000000          <1>     call   vbe_bfn_svs_1
                             <1>
                             <1>     ; from VBE_DISPI_INDEX_BANK
                             <1>     ;   to VBE_DISPI_INDEX_Y_OFFSET
                             <1>
00003CF8 FEC0                <1>     inc    al
                             <1>     ; al = 4 ; VBE_DISPI_INDEX_BANK - 1
                             <1>
00003CFA B505                <1>     mov    ch, 5
                             <1> vbe_bfn_svs_1:
00003CFC FEC0                <1>     inc    al ; from VBE_DISPI_INDEX_XRES
                             <1>                ;   to VBE_DISPI_INDEX_BPP
                             <1>     ;mov   dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
00003CFE FECA                <1>     dec    dl ; 1CEh
00003D00 66EF                <1>     out    dx, ax
00003D02 50                  <1>     push   eax
                             <1>     ;mov   dx, 01CFh ; VBE_DISPI_IOPORT_DATA
00003D03 FEC2                <1>     inc    dl ; 1CFh
00003D05 66ED                <1>     in     ax, dx
00003D07 66AB                <1>     stosw
00003D09 58                  <1>     pop    eax
00003D0A FECD                <1>     dec    ch
00003D0C 75EE                <1>     jnz    short vbe_bfn_svs_1
00003D0E C3                  <1>     retn
                             <1>
                             <1> biosfn_restore_video_state:
                             <1>     ; 22/01/2021
                             <1>     ; 13/01/2021
                             <1>     ; 12/01/2021 (TRDOS 386 v2.0.3)
                             <1>     ; (vgabios.c)
                             <1>
                             <1>     ; modified registers: eax, edx, esi, edi, ch
                             <1>
                             <1>     ;mov   esi, VBE3SAVERESTOREBLOCK
                             <1>
                             <1>     ; input: esi = state buffer address
                             <1>
00003D0F F6C101              <1>     test   cl, 1
00003D12 0F84A9000000        <1>     jz     bfn_rvs_6
                             <1>
00003D18 66817E40D403        <1>     cmp    word [esi+64], 3D4h ; must be 3D4h
00003D1E 7402                <1>     je     short bfn_rvs_0
                             <1>                    ; it is seen as valid buffer
00003D20 F9                  <1>     stc
00003D21 C3                  <1>     retn
                             <1>
                             <1> bfn_rvs_0:
00003D22 89F7                <1>     mov    edi, esi ; addr1
00003D24 83C605              <1>     add    esi, 5 ; skip 1st 5 bytes for now
                             <1>
                             <1>     ; // Reset Attribute Ctl flip-flop
                             <1>         ; inb(VGAREG_ACTL_RESET);
00003D27 66BADA03            <1>     mov    dx, 3DAh ; VGAREG_ACTL_RESET
00003D2B EC                  <1>     in     al, dx
                             <1>
                             <1>     ; for(i=1;i<=4;i++){
00003D2C B001                <1>     mov    al, 1
                             <1>     ;;mov  dx, VGAREG_SEQU_ADDRESS ; 3C4h
                             <1>     ;mov   dl, 0C4h
00003D2E B504                <1>     mov    ch, 4
                             <1> bfn_rvs_1:
                             <1>     ; outb(VGAREG_SEQU_ADDRESS, i);
                             <1>     ;mov   dx, VGAREG_SEQU_ADDRESS ; 3C4h
00003D30 B2C4                <1>     mov    dl, 0C4h
00003D32 EE                  <1>     out    dx, al
                             <1>     ;mov   dx, VGAREG_SEQU_DATA  ; 3C5h
00003D33 FEC2                <1>     inc    dl  ; dx = 3C5h
                             <1>     ; outb(VGAREG_SEQU_DATA)
00003D35 50                  <1>     push   eax
00003D36 AC                  <1>     lodsb  ; (4 bytes in loop)
00003D37 EE                  <1>     out    dx, al
00003D38 58                  <1>     pop    eax
                             <1>     ;mov   dx, VGAREG_SEQU_ADDRESS ; 3C4h
                             <1>     ;dec   dl
00003D39 FEC0                <1>     inc    al  ; i++
00003D3B FECD                <1>     dec    ch
00003D3D 75F1                <1>     jnz    short bfn_rvs_1
                             <1>
                             <1>     ; outb(VGAREG_SEQU_ADDRESS, 0);
00003D3F 28C0                <1>     sub    al, al ; 0
00003D41 EE                  <1>     out    dx, al
                             <1>     ; outb(VGAREG_SEQU_DATA)
                             <1>     ;mov   dx, VGAREG_SEQU_DATA ; 3C5h
00003D42 FEC2                <1>     inc    dl  ; dx = 3C5h
00003D44 AC                  <1>     lodsb  ; (+1 byte)
00003D45 EE                  <1>     out    dx, al
                             <1>
                             <1>     ; // Disable CRTC write protection
                             <1>     ; outw(crtc_addr,0x0011);
                             <1>     ;mov   dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003D46 B2D4                <1>     mov    dl, 0D4h
00003D48 66B81100            <1>     mov    ax, 11h
00003D4C 66EF                <1>     out    dx, ax
                             <1>
                             <1>     ; // Set CRTC regs
                             <1>
                             <1>     ; for(i=0;i<=0x18;i++) {
                             <1>         ;   if (i != 0x11) {
00003D4E 28C0                <1>     sub    al, al ; 0
                             <1>     ;;mov  dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
```

```
                                <1>     ;mov    dl, 0D4h
00003D50 B519                   <1>     mov     ch, 25
                                <1> bfn_rvs_2:
                                <1>         ; outb(crtc_addr,i);
                                <1>     ;mov    dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003D52 B2D4                   <1>     mov     dl, 0D4h
00003D54 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, VGAREG_VGA_CRTC_DATA ; 3D5h
00003D55 FEC2                   <1>     inc     dl  ; dx = 3D5h
                                <1>     ; inb(crtc_addr+1)
00003D57 50                     <1>     push    eax
00003D58 AC                     <1>     lodsb   ; (25 bytes in loop)
00003D59 EE                     <1>     out     dx, al
00003D5A 58                     <1>     pop     eax
                                <1>     ;mov    dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
                                <1>     ;dec    dl
00003D5B FEC0                   <1>     inc     al  ; i++
00003D5D 3C11                   <1>     cmp     al, 17 ; 11h
00003D5F 7505                   <1>     jne     short bfn_rvs_3
00003D61 AC                     <1>     lodsb
00003D62 88C4                   <1>     mov     ah, al ; *
00003D64 B012                   <1>     mov     al, 18
                                <1> bfn_rvs_3:
00003D66 FECD                   <1>     dec     ch
00003D68 75E8                   <1>     jnz     short bfn_rvs_2
                                <1>
                                <1>     ; // select crtc base address
                                <1>         ; v = inb(VGAREG_READ_MISC_OUTPUT) & ~0x01;
                                <1>     ;if (crtc_addr = 0x3d4)
                                <1>     ;   v |= 0x01;
                                <1>     ; outb(VGAREG_WRITE_MISC_OUTPUT, v);
                                <1>
                                <1>     ;;mov   dx, VGAREG_READ_MISC_OUTPUT ; 3CCh
                                <1>     ;mov    dl, 0CCh
                                <1>     ;in     al, dl
                                <1>     ;and    al, 1
                                <1>     ;;mov   dx, VGAREG_WRITE_MISC_OUTPUT ; 3C2h
                                <1>     ;mov    dl, 0C2h
                                <1>     ;or     al, 1
                                <1>     ;out    dx, al
                                <1>
                                <1>     ; // enable write protection if needed
                                <1>     ;outb(crtc_addr, 0x11);
                                <1>         ;outb(crtc_addr+1, read_byte(ES, BX - 0x18 + 0x11));
                                <1>     ;mov    dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003D6A B2D4                   <1>     mov     dl, 0D4h
00003D6C B011                   <1>     mov     al, 11h
00003D6E EE                     <1>     out     dx, al
00003D6F 88E0                   <1>     mov     al, ah ; *
00003D71 FEC2                   <1>     inc     dl ; dx = 3D5h
00003D73 EE                     <1>     out     dx, al
                                <1>
                                <1>     ; // Set Attribute Ctl
00003D74 8A6703                 <1>     mov     ah, [edi+3] ; addr1+3, ah = ar_index
00003D77 80E420                 <1>     and     ah, 20h  ; (ar_index & 0x20)
                                <1>
                                <1>         ; inb(VGAREG_ACTL_RESET);
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
00003D7A B2DA                   <1>     mov     dl, 0DAh
00003D7C EC                     <1>     in      al, dx
                                <1>
                                <1>         ; for(i=0;i<=0x13;i++) {
00003D7D 28C0                   <1>     sub     al, al ; 0
00003D7F B514                   <1>     mov     ch, 20
                                <1> bfn_rvs_4:
                                <1>     ; outb(VGAREG_ACTL_ADDRESS, i | (ar_index & 0x20));
00003D81 50                     <1>     push    eax
00003D82 08E0                   <1>     or      al, ah
                                <1>     ;mov    dx, VGAREG_ACTL_ADDRESS ; 3C0h
00003D84 B2C0                   <1>     mov     dl, 0C0h
00003D86 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, VGAREG_ACTL_WRITE_DATA ; 3C0h
                                <1>     ;mov    dl, 0C0h
00003D87 AC                     <1>     lodsb   ; (20 bytes in loop)
00003D88 EE                     <1>     out     dx, al
00003D89 58                     <1>     pop     eax
00003D8A FEC0                   <1>     inc     al  ; i++
00003D8C FECD                   <1>     dec     ch
00003D8E 75F1                   <1>     jnz     short bfn_rvs_4
                                <1>
                                <1>     ; outb(VGAREG_ACTL_ADDRESS, ar_index);
                                <1>     ;mov    dx, VGAREG_ACTL_ADDRESS ; 3C0h
                                <1>     ;mov    dl, 0C0h
00003D90 88E0                   <1>     mov     al, ah ; ar_index
00003D92 EE                     <1>     out     dx, al
                                <1>
                                <1>     ; inb(VGAREG_ACTL_RESET);
                                <1>     ;mov    dx, VGAREG_ACTL_RESET ; 3DAh
00003D93 B2DA                   <1>     mov     dl, 0DAh
00003D95 EC                     <1>     in      al, dx
                                <1>
                                <1>         ; for(i=0;i<=8;i++) {
00003D96 28C0                   <1>     sub     al, al ; 0
                                <1>     ;;mov   dx, VGAREG_GRDC_ADDRESS ; 3CEh
                                <1>     ;mov    dl, 0CEh
00003D98 B509                   <1>     mov     ch, 9
                                <1> bfn_rvs_5:
                                <1>     ; outb(VGAREG_GRDC_ADDRESS,i)
                                <1>     ;mov    dx, VGAREG_GRDC_ADDRESS ; 3CEh
00003D9A B2CE                   <1>     mov     dl, 0CEh
00003D9C EE                     <1>     out     dx, al
                                <1>     ; outb(VGAREG_ACTL_READ_DATA)
00003D9D 50                     <1>     push    eax
                                <1>     ;mov    dx, VGAREG_GRDC_DATA ; 3CFh
                                <1>     ;mov    dl, 0CFh
00003D9E FEC2                   <1>     inc     dl
00003DA0 AC                     <1>     lodsb   ; (9 bytes in loop)
00003DA1 EE                     <1>     out     dx, al
00003DA2 58                     <1>     pop     eax
                                <1>     ;dec    dl
00003DA3 FEC0                   <1>     inc     al  ; i++
00003DA5 FECD                   <1>     dec     ch
00003DA7 75F1                   <1>     jnz     short bfn_rvs_5
                                <1>
                                <1>     ; BX += 2; /* crtc_addr */     ; 3D4h
                                <1>         ; BX += 4; /* plane latches */ ; 0
00003DA9 83C606                 <1>     add     esi, 6
00003DAC 56                     <1>     push    esi ; *
                                <1>
                                <1>     ;outb(VGAREG_SEQU_ADDRESS, read_byte(ES, addr1)); addr1++;
                                <1>         ;outb(crtc_addr, read_byte(ES, addr1)); addr1++;
                                <1>         ;outb(VGAREG_GRDC_ADDRESS, read_byte(ES, addr1)); addr1++;
                                <1>         ;addr1++;
                                <1>         ;outb(crtc_addr - 0x4 + 0xa, read_byte(ES, addr1)); addr1++;
```

```
                                <1>
00003DAD 89FE                   <1>     mov     esi, edi ; start of state buffer
                                <1>
                                <1>     ;mov    dx, VGAREG_SEQU_ADDRESS ; 3C7h
00003DAF B2C7                   <1>     mov     dl, 0C7h
00003DB1 AC                     <1>     lodsb
00003DB2 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, VGAREG_VGA_CRTC_ADDRESS ; 3D4h
00003DB3 B2D4                   <1>     mov     dl, 0D4h
00003DB5 AC                     <1>     lodsb
00003DB6 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, VGAREG_GRDC_ADDRESS ; 3CEh
00003DB7 B2CE                   <1>         mov dl, 0CEh
00003DB9 AC                     <1>     lodsb
00003DBA EE                     <1>     out     dx, al
00003DBB AC                     <1>     lodsb   ; addr1++
                                <1>     ;mov    dx, VGAREG_VGA_WRITE_FEATURE_CTL ; 3DAh
00003DBC B2DA                   <1>     mov     dl, 0DAh
00003DBE AC                     <1>     lodsb
00003DBF EE                     <1>     out     dx, al
                                <1>
00003DC0 5E                     <1>     pop     esi ; *
                                <1>
                                <1>     ; (total 70 bytes are read above as controller hardware state)
                                <1>
                                <1> bfn_rvs_6:
                                <1>     ; 13/01/2021
00003DC1 F6C102                 <1>     test    cl, 2
00003DC4 747D                   <1>     jz      short bfn_rvs_9
                                <1>
                                <1>     ; VIDEO BIOS DATA
                                <1>     ; !!! this data is valid for TRDOS 386 v2 kernel only !!!
                                <1>     ; (this is not same with BOCHS/PLEX86 video bios, BIOS data)
                                <1>
                                <1>     ; if (CX & 2) {
                                <1>     ;write_byte(BIOSMEM_SEG,BIOSMEM_CURRENT_MODE, read_byte(ES, BX)); BX++;
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_NB_COLS, read_word(ES, BX)); BX += 2;
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_PAGE_SIZE, read_word(ES, BX)); BX += 2;
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_CRTC_ADDRESS, read_word(ES, BX)); BX += 2;
                                <1>         ;write_byte(BIOSMEM_SEG,BIOSMEM_NB_ROWS, read_byte(ES, BX)); BX++;
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_CHAR_HEIGHT, read_word(ES, BX)); BX += 2;
                                <1>         ;write_byte(BIOSMEM_SEG,BIOSMEM_VIDEO_CTL, read_byte(ES, BX)); BX++;
                                <1>         ;write_byte(BIOSMEM_SEG,BIOSMEM_SWITCHES, read_byte(ES, BX)); BX++;
                                <1>         ;write_byte(BIOSMEM_SEG,BIOSMEM_MODESET_CTL, read_byte(ES, BX)); BX++;
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_CURSOR_TYPE, read_word(ES, BX)); BX += 2;
                                <1>         ;for(i=0;i<8;i++) {
                                <1>         ;   write_word(BIOSMEM_SEG, BIOSMEM_CURSOR_POS+2*i, read_word(ES, BX));
                                <1>         ;   BX += 2;
                                <1>         ;}
                                <1>         ;write_word(BIOSMEM_SEG,BIOSMEM_CURRENT_START, read_word(ES, BX)); BX += 2;
                                <1>         ;write_byte(BIOSMEM_SEG,BIOSMEM_CURRENT_PAGE, read_byte(ES, BX)); BX++;
                                <1>         ;/* current font */
                                <1>         ;write_word(0, 0x1f * 4, read_word(ES, BX)); BX += 2;
                                <1>         ;write_word(0, 0x1f * 4 + 2, read_word(ES, BX)); BX += 2;
                                <1>         ;write_word(0, 0x43 * 4, read_word(ES, BX)); BX += 2;
                                <1>         ;write_word(0, 0x43 * 4 + 2, read_word(ES, BX)); BX += 2;
                                <1>
                                <1>     ; !!! save TRDOS 386 v2 kernel spesific video bios data !!!
                                <1>     ; (which is/are used by 'SET_MODE' function and/or it's sub functions)
                                <1>
00003DC6 66AD                   <1>     lodsw   ; CRTC_ADDR, always 3D4h (color VGA) for TRDOS 386 v2
                                <1>     ; skip 3D4h check if it is already checked
00003DC8 F6C101                 <1>     test    cl, 1
00003DCB 7508                   <1>     jnz     short bfn_rvs_7
00003DCD 663DD403               <1>     cmp     ax, 3D4h
00003DD1 7402                   <1>     je      short bfn_rvs_7
00003DD3 F9                     <1>     stc
00003DD4 C3                     <1>     retn
                                <1> bfn_rvs_7:
00003DD5 AC                     <1>     lodsb
00003DD6 A2[2E680000]           <1>     mov     [CRT_MODE], al ; Current video mode (0FFh for VESA VBE modes)
00003DDB AC                     <1>     lodsb
00003DDC A2[2F680000]           <1>     mov     [CRT_MODE_SET], al ; 29h for mode 03h ; TRDOS 386 feature only !
00003DE1 66AD                   <1>     lodsw
00003DE3 66A3[269D0100]         <1>     mov     [video_mode], ax ; Current VESA VBE (SVGA, extended VGA) mode
00003DE9 66AD                   <1>     lodsw           ; (valid if [CRT_MODE] = 0FFh)
00003DEB 66A3[4C830100]         <1>     mov     [CRT_LEN], ax ; page size (in bytes)
00003DF1 66AD                   <1>     lodsw
00003DF3 66A3[CC760100]         <1>     mov     [CRT_START], ax ; video page start offset
00003DF9 AC                     <1>     lodsb
00003DFA A2[30680000]           <1>     mov     [CRT_COLS], al ; nbcols, characters per row
00003DFF AC                     <1>     lodsb
00003E00 A2[36680000]           <1>     mov     [VGA_ROWS], al ; nbrows, (character) rows per page (not rows-1)
00003E05 AC                     <1>     lodsb
00003E06 A2[32680000]           <1>     mov     [CHAR_HEIGHT], al ; character font height (8 or 16 or 14)
00003E0B AC                     <1>     lodsb
00003E0C A2[33680000]           <1>     mov     [VGA_VIDEO_CTL], al ; ROM BIOS DATA AREA Offset 87h
00003E11 AC                     <1>     lodsb
00003E12 A2[34680000]           <1>     mov     [VGA_SWITCHES], al ; feature bit switches
00003E17 AC                     <1>     lodsb
00003E18 A2[35680000]           <1>     mov     [VGA_MODESET_CTL], al ; basic mode set options
                                <1>     ; followings are only used by TRDOS 386 v2 (IBM PC/AT ROMBIOS) code
                                <1>     ; (bochs/plex86 does not use and return those)
00003E1D AC                     <1>     lodsb
00003E1E A2[31680000]           <1>     mov     [CRT_PALETTE], al ; current color palette ; TRDOS 386 feature only !
00003E23 AC                     <1>     lodsb
00003E24 A2[DE760100]           <1>     mov     [ACTIVE_PAGE], al ; current video page
00003E29 66AD                   <1>     lodsw
00003E2B 66A3[47680000]         <1>     mov     [CURSOR_MODE], ax ; cursor type
                                <1>     ;lodsd
                                <1>     ;mov    [CURSOR_POSN], eax ; cursor position for video page 0 and 1
                                <1>     ;lodsd
                                <1>     ;mov    [CURSOR_POSN+4], eax ; cursor position for video page 2 and 3
                                <1>     ;lodsd
                                <1>     ;mov    [CURSOR_POSN+8], eax ; cursor position for video page 4 and 5
                                <1>     ;lodsd
                                <1>     ;mov    [CURSOR_POSN+12], eax ; cursor position for video page 6 and 7
00003E31 B504                   <1>     mov     ch, 4
00003E33 BF[CE760100]           <1>     mov     edi, CURSOR_POSN
                                <1> bfn_rvs_8:
00003E38 A5                     <1>     movsd
00003E39 FECD                   <1>     dec     ch
00003E3B 75FB                   <1>     jnz     short bfn_rvs_8
                                <1>     ; (font addr) protected mode address in kernel's/system memory space
                                <1>     ; (not accessable/meaningful address value by user)
00003E3D AD                     <1>     lodsd
00003E3E A3[5E830100]           <1>     mov     [VGA_INT43H], eax ; VGA current (default) font address
                                <1>
                                <1>     ; (total 40 bytes are read&written above as BIOS data state)
                                <1> bfn_rvs_9:
                                <1>     ; 13/01/2021
00003E43 F6C104                 <1>     test    cl, 4
00003E46 7421                   <1>     jz      short bfn_rvs_11
                                <1>
```

```
                                <1>     ;BX++;
                                <1>         ;v = read_byte(ES, BX); BX++;
                                <1>         ;outb(VGAREG_PEL_MASK, read_byte(ES, BX)); BX++;
                                <1>         ;// Set the whole dac always, from 0
                                <1>         ;outb(VGAREG_DAC_WRITE_ADDRESS,0x00);
                                <1>         ;for(i=0;i<256*3;i++) {
                                <1>         ;   outb(VGAREG_DAC_DATA, read_byte(ES, BX)); BX++;
                                <1>         ;}
                                <1>         ;BX++;
                                <1>         ;outb(VGAREG_DAC_WRITE_ADDRESS, v);
                                <1>
                                <1>     ; /* read/write mode dac */
00003E48 AC                     <1>     lodsb   ; skip ; VGAREG_DAC_STATE
00003E49 AC                     <1>     lodsb
00003E4A 88C4                   <1>     mov     ah, al ; * ; v
00003E4C AC                     <1>     lodsb
00003E4D 66BAC603               <1>     mov     dx, VGAREG_PEL_MASK  ; 3C6h
00003E51 EE                     <1>     out     dx, al
                                <1>     ;// Set the whole dac always, from 0
00003E52 30C0                   <1>     xor     al, al ; 0
                                <1>     ;mov    dx, VGAREG_DAC_WRITE_ADDRESS ; 3C8h
00003E54 B2C8                   <1>     mov     dl, 0C8h
00003E56 EE                     <1>     out     dx, al
                                <1>
00003E57 51                     <1>     push    ecx ; 22/01/2021
                                <1>     ;for(i=0;i<256*3;i++) {
                                <1>     ;mov    ecx, 256*3 ; 768 bytes
                                <1>     ; 03/08/2022
00003E58 29C9                   <1>     sub     ecx, ecx
00003E5A B503                   <1>     mov     ch, 3
                                <1>     ; ecx = 300h = 768
                                <1>     ;mov    dx, VGAREG_DAC_DATA ; 3C9h
                                <1>     ;mov    dl, 0C9h
00003E5C FEC2                   <1>     inc     dl ; dx = 3C9h
                                <1> bfn_rvs_10:
00003E5E AC                     <1>     lodsb
00003E5F EE                     <1>     out     dx, al
00003E60 E2FC                   <1>     loop    bfn_rvs_10
00003E62 59                     <1>     pop     ecx ; 22/01/2021
                                <1>
                                <1>     ; /* color select register */
00003E63 AC                     <1>     lodsb    ; skip
                                <1>
00003E64 88E0                   <1>     mov     al, ah ; * ; v
                                <1>
                                <1>     ;mov    dx, VGAREG_DAC_WRITE_ADDRESS ; 3C8h
                                <1>     ;mov    dl, 0C8h
00003E66 FECA                   <1>     dec     dl ; dx  = 3C8h
00003E68 EE                     <1>     out     dx, al ; * ; v
                                <1>
                                <1>     ; (total 772 bytes are read above as DAC state)
                                <1> bfn_rvs_11:
00003E69 C3                     <1>     retn
                                <1>
                                <1> vbe_biosfn_restore_video_state:
                                <1>     ; 23/01/2021
                                <1>     ; 13/01/2021 (TRDOS 386 v2.0.3)
                                <1>     ; (vbe.c)
                                <1>
                                <1>     ; modified registers: eax, edx, esi, ch
                                <1>
                                <1>     ; input: esi = state buffer address
                                <1>     ; output:
                                <1>     ;       VBE DISPI register contents will be restored
                                <1>     ;       (18 bytes, 9 words)
                                <1>
                                <1>     ; enable = read_word(ES, BX);
                                <1>     ; BX += 2;
                                <1>     ;
                                <1>     ; if (!(enable & VBE_DISPI_ENABLED)) {
                                <1>         ;   outw(VBE_DISPI_IOPORT_INDEX,VBE_DISPI_INDEX_ENABLE);
                                <1>         ;   outw(VBE_DISPI_IOPORT_DATA, enable);
                                <1>     ; } else {
                                <1>         ;   outw(VBE_DISPI_IOPORT_INDEX, VBE_DISPI_INDEX_XRES);
                                <1>         ;   outw(VBE_DISPI_IOPORT_DATA, read_word(ES, BX));
                                <1>         ;   BX += 2;
                                <1>         ;   outw(VBE_DISPI_IOPORT_INDEX, VBE_DISPI_INDEX_YRES);
                                <1>         ;   outw(VBE_DISPI_IOPORT_DATA, read_word(ES, BX));
                                <1>         ;   BX += 2;
                                <1>         ;   outw(VBE_DISPI_IOPORT_INDEX, VBE_DISPI_INDEX_BPP);
                                <1>         ;   outw(VBE_DISPI_IOPORT_DATA, read_word(ES, BX));
                                <1>         ;   BX += 2;
                                <1>         ;   outw(VBE_DISPI_IOPORT_INDEX,VBE_DISPI_INDEX_ENABLE);
                                <1>         ;   outw(VBE_DISPI_IOPORT_DATA, enable);
                                <1>     ;
                                <1>         ;  for(i = VBE_DISPI_INDEX_BANK; i <= VBE_DISPI_INDEX_Y_OFFSET; i++)
                                <1>     ;    {
                                <1>         ;     outw(VBE_DISPI_IOPORT_INDEX, i);
                                <1>         ;     outw(VBE_DISPI_IOPORT_DATA, read_word(ES, BX));
                                <1>         ;     BX += 2;
                                <1>         ;    }
                                <1>         ; }
                                <1>
00003E6A 66AD                   <1>     lodsw   ; enable (status, enabled=1, disabled=0)
00003E6C 66BACE01               <1>     mov     dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ; 23/01/2021
00003E70 6683E001               <1>     and     ax, 1 ; VBE_DISPI_ENABLED
00003E74 750B                   <1>     jnz     short vbe_bfn_rvs_1
                                <1>     ; ax = 0
                                <1>     ; VBE_DISPI_DISABLED
                                <1> vbe_bfn_rvs_0:
                                <1>     ; enable (disable) dispi
                                <1>     ; dx = 01CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ; ah = 0
00003E76 50                     <1>     push    eax
00003E77 B004                   <1>     mov     al, 04h ; VBE_DISPI_INDEX_ENABLE
00003E79 66EF                   <1>     out     dx, ax
                                <1>     ;mov    dx, 01CFh ; VBE_DISPI_IOPORT_DATA
00003E7B FEC2                   <1>     inc     dl
00003E7D 58                     <1>     pop     eax
00003E7E 66EF                   <1>     out     dx, ax ; enable (or disable)
00003E80 C3                     <1>     retn
                                <1> vbe_bfn_rvs_1:
                                <1>     ; VBE_DISPI_ENABLED
                                <1>
                                <1>     ; from VBE_DISPI_INDEX_XRES
                                <1>     ;   to VBE_DISPI_INDEX_BPP
                                <1>
00003E81 B503                   <1>     mov     ch, 3
00003E83 28C0                   <1>     sub     al, al ; 0 ; VBE_DISPI_INDEX_XRES - 1
                                <1>     ; ax = 0
                                <1>
00003E85 E80B000000             <1>     call    vbe_bfn_rvs_2
                                <1>
```

```
                                <1>         ;outw(VBE_DISPI_IOPORT_INDEX,VBE_DISPI_INDEX_ENABLE);
                                <1>     ;outw(VBE_DISPI_IOPORT_DATA, enable);
                                <1>
                                <1>     ; 23/01/2021
00003E8A B001                   <1>     mov     al, 1 ; VBE_DISPI_ENABLED
                                <1>     ; ax = 1
00003E8C E8E5FFFFFF             <1>     call    vbe_bfn_rvs_0
                                <1>
                                <1>     ; from VBE_DISPI_INDEX_BANK
                                <1>     ;   to VBE_DISPI_INDEX_Y_OFFSET
                                <1>
00003E91 B505                   <1>     mov     ch, 5
                                <1>     ; 23/01/2021
00003E93 B004                   <1>     mov     al, 4  ; VBE_DISPI_INDEX_BANK - 1
                                <1>     ; ax = 4
                                <1> vbe_bfn_rvs_2:
00003E95 FEC0                   <1>     inc     al ; from VBE_DISPI_INDEX_XRES
                                <1>                ; to VBE_DISPI_INDEX_BPP
                                <1>     ;mov    dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ;mov    dl, 0CEh
00003E97 66EF                   <1>     out     dx, ax
00003E99 50                     <1>     push    eax
                                <1>     ;mov    dx, 01CFh ; VBE_DISPI_IOPORT_DATA
00003E9A FEC2                   <1>     inc     dl ; 1CFh
00003E9C 66AD                   <1>     lodsw
00003E9E 66EF                   <1>     out     dx, ax
00003EA0 58                     <1>     pop     eax
00003EA1 FECA                   <1>     dec     dl ; 1CEh
00003EA3 FECD                   <1>     dec     ch
00003EA5 75EE                   <1>     jnz     short vbe_bfn_rvs_2
00003EA7 C3                     <1>     retn
                                <1>
                                <1> ; ---------------------------------------------------------
                                <1>
                                <1> dispi_set_enable:
                                <1>     ; 03/08/2022
                                <1>     ; 23/11/2020
                                <1>     ; Input:
                                <1>     ;       ax = VBE_DISPI_ENABLED = 1
                                <1>     ;            or VBE_DISPI_DISABLED = 0
                                <1>     ;
                                <1>     ; Modified registers: none
                                <1>
                                <1>     ;push   edx
                                <1>     ;push   eax
                                <1>     ;mov    dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ;mov    ax, 04h   ; VBE_DISPI_INDEX_ENABLE
                                <1>     ;out    dx, ax
                                <1>     ;pop    eax
                                <1>     ;;mov   dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                                <1>     ;;mov   dl, 0CFh
                                <1>     ;inc    dl
                                <1>     ;out    dx, ax
                                <1>     ;pop    edx
                                <1>     ;retn
                                <1>
                                <1>     ; 25/11/2020
                                <1>     ; Modified registers: edx
                                <1>     ;;push  edx
                                <1>     ;mov    dx, 04h ; VBE_DISPI_INDEX_ENABLE
                                <1>     ; 03/08/2022
00003EA8 28F6                   <1>     sub     dh, dh
00003EAA B204                   <1>     mov     dl, 04h ; VBE_DISPI_INDEX_ENABLE
                                <1>
                                <1>     ;;call  dispi_set_parms
                                <1>     ;;pop   edx
                                <1>     ;;retn
                                <1>     ;jmp    short dispi_set_parms
                                <1>
                                <1> dispi_set_parms:
                                <1>     ; 03/08/2022
                                <1>     ; 25/11/2020
                                <1>     ; Input:
                                <1>     ;       ax = data
                                <1>     ;       dx = vbe dispi register index
                                <1>     ;
                                <1>     ; Modified registers: edx
                                <1>
00003EAC 50                     <1>     push    eax
                                <1>     ;mov    ax, dx
                                <1>     ; 03/08/2022
00003EAD 89D0                   <1>     mov     eax, edx
00003EAF 66BACE01               <1>     mov     dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
00003EB3 66EF                   <1>     out     dx, ax
00003EB5 58                     <1>     pop     eax
                                <1>     ;mov    dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                                <1>     ;mov    dl, 0CFh
00003EB6 FEC2                   <1>     inc     dl
00003EB8 66EF                   <1>     out     dx, ax
00003EBA C3                     <1>     retn
                                <1>
                                <1> dispi_set_bpp:
                                <1>     ; 03/08/2022
                                <1>     ; 25/11/2020
                                <1>     ; Input:
                                <1>     ;       ax = Bits per pixel value
                                <1>     ;            (8,16,24,32)
                                <1>     ;
                                <1>     ; Modified registers: none
                                <1>
                                <1>     ;push   edx
                                <1>     ;push   eax
                                <1>     ;mov    dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ;mov    ax, 03h   ; VBE_DISPI_INDEX_BPP
                                <1>     ;out    dx, ax
                                <1>     ;pop    eax
                                <1>     ;;mov   dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                                <1>     ;;mov   dl, 0CFh
                                <1>     ;inc    dl
                                <1>     ;out    dx, ax
                                <1>     ;pop    edx
                                <1>     ;retn
                                <1>
                                <1>     ; 25/11/2020
                                <1>     ; Modified registers: edx
                                <1>     ;;push  edx
                                <1>     ;mov    dx, 03h ; VBE_DISPI_INDEX_BPP
                                <1>     ; 03/08/2022
00003EBB 28F6                   <1>     sub     dh, dh
00003EBD B203                   <1>     mov     dl, 03h ; VBE_DISPI_INDEX_BPP
                                <1>
                                <1>     ;;call  dispi_set_parms
                                <1>     ;;pop   edx
                                <1>     ;;retn
```

```
00003EBF EBEB                <1>     jmp    short dispi_set_parms
                             <1>
                             <1> dispi_set_xres:
                             <1>     ; 03/08/2022
                             <1>     ; 25/11/2020
                             <1>     ; Input:
                             <1>     ;      ax = X resolution (screen witdh)
                             <1>     ;           (320,640,800,1024,1280,1920)
                             <1>     ;
                             <1>     ; Modified registers: none
                             <1>
                             <1>     ;push  edx
                             <1>     ;push  eax
                             <1>     ;mov   dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                             <1>     ;mov   ax, 01h   ; VBE_DISPI_INDEX_XRES
                             <1>     ;out   dx, ax
                             <1>     ;pop   eax
                             <1>     ;;mov  dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                             <1>     ;;mov  dl, 0CFh
                             <1>     ;inc   dl
                             <1>     ;out   dx, ax
                             <1>     ;pop   edx
                             <1>     ;retn
                             <1>
                             <1>     ; 25/11/2020
                             <1>     ; Modified registers: edx
                             <1>     ;;push edx
                             <1>     ;mov   dx, 01h ; VBE_DISPI_INDEX_XRES
                             <1>     ; 03/08/2022
00003EC1 28F6                <1>     sub    dh, dh
00003EC3 B201                <1>     mov    dl, 01h ; VBE_DISPI_INDEX_XRES
                             <1>     ;;call dispi_set_parms
                             <1>     ;;pop  edx
                             <1>     ;;retn
00003EC5 EBE5                <1>     jmp    short dispi_set_parms
                             <1>
                             <1> dispi_set_yres:
                             <1>     ; 03/08/2022
                             <1>     ; 25/11/2020
                             <1>     ; Input:
                             <1>     ;      ax = Y resolution (screen height)
                             <1>     ;           (200,400,600,720,768,1080)
                             <1>     ;
                             <1>     ; Modified registers: none
                             <1>
                             <1>     ;push  edx
                             <1>     ;push  eax
                             <1>     ;mov   dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                             <1>     ;mov   ax, 02h   ; VBE_DISPI_INDEX_YRES
                             <1>     ;out   dx, ax
                             <1>     ;pop   eax
                             <1>     ;;mov  dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                             <1>     ;;mov  dl, 0CFh
                             <1>     ;inc   dl
                             <1>     ;out   dx, ax
                             <1>     ;pop   edx
                             <1>     ;retn
                             <1>
                             <1>     ; 25/11/2020
                             <1>     ; Modified registers: edx
                             <1>     ;;push edx
                             <1>     ;mov   dx, 02h ; VBE_DISPI_INDEX_YRES
                             <1>     ; 03/08/2022
00003EC7 28F6                <1>     sub    dh, dh
00003EC9 B202                <1>     mov    dl, 02h ; VBE_DISPI_INDEX_YRES
                             <1>     ;;call dispi_set_parms
                             <1>     ;;pop  edx
                             <1>     ;;retn
00003ECB EBDF                <1>     jmp    short dispi_set_parms
                             <1>
                             <1> dispi_set_bank:
                             <1>     ; 03/08/2022
                             <1>     ; 25/11/2020
                             <1>     ; Input:
                             <1>     ;      ax = video memory bank number
                             <1>     ;
                             <1>     ; Modified registers: none
                             <1>
                             <1>     ;push  edx
                             <1>     ;push  eax
                             <1>     ;mov   dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                             <1>     ;mov   ax, 05h   ; VBE_DISPI_INDEX_BANK
                             <1>     ;out   dx, ax
                             <1>     ;pop   eax
                             <1>     ;;mov  dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                             <1>     ;;mov  dl, 0CFh
                             <1>     ;inc   dl
                             <1>     ;out   dx, ax
                             <1>     ;pop   edx
                             <1>     ;retn
                             <1>
                             <1>     ; 25/11/2020
                             <1>     ; Modified registers: edx
                             <1>     ;;push edx
                             <1>     ;mov   dx, 05h ; VBE_DISPI_INDEX_BANK
                             <1>     ; 03/08/2022
00003ECD 28F6                <1>     sub    dh, dh
00003ECF B205                <1>     mov    dl, 05h ; VBE_DISPI_INDEX_BANK
                             <1>     ;;call dispi_set_parms
                             <1>     ;;pop  edx
                             <1>     ;;retn
00003ED1 EBD9                <1>     jmp    short dispi_set_parms
                             <1>
                             <1> dispi_get_enable:
                             <1>     ; 03/08/2022
                             <1>     ; 27/11/2020
                             <1>     ; Input:
                             <1>     ;      none
                             <1>     ; Output:
                             <1>     ;      ax = vbe dispi status
                             <1>     ;
                             <1>     ; Modified registers: eax
                             <1>
                             <1>     ;push  edx
                             <1>     ;mov   dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
                             <1>     ;mov   ax, 04h   ; VBE_DISPI_INDEX_ENABLE
                             <1>     ;out   dx, ax
                             <1>     ;;mov  dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                             <1>     ;;mov  dl, 0CFh
                             <1>     ;inc   dl
                             <1>     ;in    ax, dx
                             <1>     ;pop   edx
                             <1>     ;retn
                             <1>
```

```
                                <1>     ; 27/11/2020
                                <1>     ; Modified registers: eax, edx
                                <1>     ;;push  edx
                                <1>     ;mov    ax, 04h ; VBE_DISPI_INDEX_ENABLE
                                <1>     ; 03/08/2022
00003ED3 28E4                   <1>     sub     ah, ah
00003ED5 B004                   <1>     mov     al, 04h ; VBE_DISPI_INDEX_ENABLE
                                <1>
                                <1>     ;;call  dispi_get_parms
                                <1>     ;;pop   edx
                                <1>     ;;retn
                                <1>     ;jmp    short dispi_get_parms
                                <1>
                                <1> dispi_get_parms:
                                <1>     ; 25/11/2020
                                <1>     ; Input:
                                <1>     ;       ax = vbe dispi register index
                                <1>     ; output:
                                <1>     ;       ax = data
                                <1>     ;
                                <1>     ; Modified registers: eax, edx
                                <1>
00003ED7 66BACE01               <1>     mov     dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
00003EDB 66EF                   <1>     out     dx, ax
                                <1>     ;mov    dx, 01CFh ; VBE_DISPI_IOPORT_DATA
                                <1>     ;mov    dl, 0CFh
00003EDD FEC2                   <1>     inc     dl
00003EDF 66ED                   <1>     in      ax, dx
00003EE1 C3                     <1>     retn
                                <1>
                                <1> vga_compat_setup:
                                <1>     ; 03/08/2022
                                <1>     ; 26/11/2020
                                <1>     ; 25/11/2020
                                <1>     ; VGA compatibility setup
                                <1>     ; (vbe.c, 02/01/2020, vruppert)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       none
                                <1>     ;
                                <1>     ; Modified registers: eax, edx
                                <1>
                                <1>     ; 26/11/2020
                                <1>     ;push   eax
                                <1>     ;push   edx
                                <1>
                                <1>     ; set CRT X resolution
00003EE2 66BACE01               <1>     mov     dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     ;mov    ax, 01h  ; VBE_DISPI_INDEX_XRES
                                <1>     ; 03/08/2022
00003EE6 31C0                   <1>     xor     eax, eax
00003EE8 FEC0                   <1>     inc     al
                                <1>     ; eax = 1
00003EEA 66EF                   <1>     out     dx, ax
                                <1>     ;mov    dx, 1CFh ; VBE_DISPI_IOPORT_DATA
00003EEC FEC2                   <1>     inc     dl
00003EEE 66ED                   <1>     in      ax, dx
00003EF0 50                     <1>     push    eax
00003EF1 66BAD403               <1>     mov     dx, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
00003EF5 66B81100               <1>     mov     ax, 0011h ; Vertical retrace end register
00003EF9 66EF                   <1>     out     dx, ax
                                <1>     ;pop    eax
                                <1>     ;push   eax
00003EFB 8B0424                 <1>     mov     eax, [esp]
                                <1>     ;shr    ax, 3 ; / 8 for pixel to character
                                <1>     ;dec    ax  ; - 1 (EGA or VGA?)
                                <1>     ; 03/08/2022
00003EFE C1E803                 <1>     shr     eax, 3
00003F01 48                     <1>     dec     eax
00003F02 88C4                   <1>     mov     ah, al
00003F04 B001                   <1>     mov     al, 01h ; Horizontal display end register
00003F06 66EF                   <1>     out     dx, ax
00003F08 58                     <1>     pop     eax
                                <1>
00003F09 E89C000000             <1>     call    vga_set_virt_width
                                <1>
                                <1>     ; set CRT Y resolution
                                <1>     ; 03/08/2022
00003F0E 66BACE01               <1>     mov     dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
00003F12 66B80200               <1>     mov     ax, 02h  ; VBE_DISPI_INDEX_YRES
00003F16 66EF                   <1>     out     dx, ax
                                <1>     ;mov    dx, 1CFh ; VBE_DISPI_IOPORT_DATA
                                <1>     ; 03/08/2022
00003F18 FEC2                   <1>     inc     dl
00003F1A 66ED                   <1>     in      ax, dx
00003F1C 50                     <1>     push    eax
00003F1D 66BAD403               <1>     mov     dx, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
00003F21 88C4                   <1>     mov     ah, al
00003F23 B012                   <1>     mov     al, 12h ; Vertical display end register
00003F25 66EF                   <1>     out     dx, ax
00003F27 58                     <1>     pop     eax
00003F28 B007                   <1>     mov     al, 07h ; Overflow register
00003F2A EE                     <1>     out     dx, al
                                <1>     ;inc    dx
                                <1>     ; 03/08/2022
00003F2B FEC2                   <1>     inc     dl
00003F2D EC                     <1>     in      al, dx ; read overflow register
00003F2E 24BD                   <1>     and     al, 0BDh ; clear VDE 9th and 10th bits
00003F30 F6C401                 <1>     test    ah, 01h
00003F33 7402                   <1>     jz      short bit8_clear
00003F35 0C02                   <1>     or      al, 02h ; VDE 9th bit (bit 8) in bit 1
                                <1> bit8_clear:
00003F37 F6C402                 <1>     test    ah, 02h
00003F3A 7402                   <1>     jz      short bit9_clear
00003F3C 0C40                   <1>     or      al, 40h ; VDE 10th bit (bit 9) in bit 6
                                <1> bit9_clear:
00003F3E EE                     <1>     out     dx, al
                                <1>
                                <1>     ; other settings
                                <1>     ;mov    dx, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
                                <1>     ; 03/08/2022
00003F3F B2D4                   <1>     mov     dl, 0D4h
00003F41 66B80900               <1>     mov     ax, 0009h ; Maximum scan line register
00003F45 66EF                   <1>     out     dx, ax  ; Reset
00003F47 B017                   <1>     mov     al, 17h ; Mode control register
00003F49 EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3D5h ; VGAREG_VGA_CRTC_DATA
00003F4A FEC2                   <1>     inc     dl
00003F4C EC                     <1>     in      al, dx  ; Read mode control register
00003F4D 0C03                   <1>     or      al, 03h ; Set SRS and CMS bits
00003F4F EE                     <1>     out     dx, al
                                <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                                <1>     ; 03/08/2022
00003F50 B2DA                   <1>     mov     dl, 0DAh
```

```
00003F52 EC                <1>     in      al, dx   ; clear flip-flop
                           <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                           <1>     ; 03/08/2022
00003F53 B2C0              <1>     mov     dl, 0C0h
00003F55 B010              <1>     mov     al, 10h ; Mode control register
00003F57 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
00003F58 FEC2              <1>     inc     dl
00003F5A EC                <1>     in      al, dx
00003F5B 0C01              <1>     or      al, 01h ; select graphics mode
                           <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
00003F5D FECA              <1>     dec     dl
00003F5F EE                <1>     out     dx, al ; Write to mode control register
00003F60 B020              <1>     mov     al, 20h ; Palette RAM <-> display memory
00003F62 EE                <1>     out     dx, al ; Write to attribute addr register
                           <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                           <1>     ; 03/08/2022
00003F63 B2CE              <1>     mov     dl, 0CEh
00003F65 66B80605          <1>     mov     ax, 0506h ; Misc. register, graph, mm 1
00003F69 66EF              <1>     out     dx, ax
                           <1>     ;mov    dx, 3C4h ; VGAREG_SEQU_ADDRESS
                           <1>     ; 03/08/2022
00003F6B B2C4              <1>     mov     dl, 0C4h
00003F6D 66B8020F          <1>     mov     ax, 0F02h ; Map mask register, all planes
00003F71 66EF              <1>     out     dx, ax
                           <1>
                           <1>     ; settings for >= 8bpp
                           <1>
                           <1>     ;mov    dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
                           <1>     ;mov    ax, 03h ; VBE_DISPI_INDEX_BPP
                           <1>     ;out    dx, ax
                           <1>     ;;mov   dx, 1CFh ; VBE_DISPI_IOPORT_DATA
                           <1>     ;inc    dl
                           <1>     ;in     ax, dx
                           <1>     ;cmp    al, 08h  ; < 8 bits per pixel
                           <1>     ;jb     short vga_compat_end
                           <1>
                           <1>     ;mov    dx, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
                           <1>     ; 03/08/2022
00003F73 B2D4              <1>     mov     dl, 0D4h
00003F75 B014              <1>     mov     al, 14h  ; Underline location register
00003F77 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3D5h ; VGAREG_VGA_CRTC_DATA
00003F78 FEC2              <1>     inc     dl
00003F7A EC                <1>     in      al, dx
00003F7B 0C40              <1>     or      al, 40h  ; enable double word mode
00003F7D EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3DAh ; VGAREG_ACTL_RESET
                           <1>     ; 03/08/2022
00003F7E B2DA              <1>     mov     dl, 0DAh
00003F80 EC                <1>     in      al, dx   ; clear flip-flop
                           <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
                           <1>     ; 03/08/2022
00003F81 B2C0              <1>     mov     dl, 0C0h
00003F83 B010              <1>     mov     al, 10h  ; Mode control register
00003F85 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3C1h ; VGAREG_ACTL_READ_DATA
00003F86 FEC2              <1>     inc     dl
00003F88 EC                <1>     in      al, dx
00003F89 0C40              <1>     or      al, 40h  ; Pixel clock select is 1
                           <1>     ;mov    dx, 3C0h ; VGAREG_ACTL_ADDRESS
00003F8B FECA              <1>     dec     dl
00003F8D EE                <1>     out     dx, al   ; update mode control reggister
00003F8E B020              <1>     mov     al, 20h  ; select display memory as PAS
00003F90 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3C4h ; VGAREG_SEQU_ADDRESS
                           <1>     ; 03/08/2022
00003F91 B2C4              <1>     mov     dl, 0C4h
00003F93 B004              <1>     mov     al, 04h ; Memory mode register
00003F95 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3C5h ; VGAREG_SEQU_DATA
00003F96 FEC2              <1>     inc     dl
00003F98 EC                <1>     in      al, dx
00003F99 0C08              <1>     or      al, 08h ; enable chain four
00003F9B EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3CEh ; VGAREG_GRDC_ADDRESS
                           <1>     ; 03/08/2022
00003F9C B2CE              <1>     mov     dl, 0CEh
00003F9E B005              <1>     mov     al, 05h  ; Mode register
00003FA0 EE                <1>     out     dx, al
                           <1>     ;mov    dx, 3CFh ; VGAREG_GRDC_DATA
00003FA1 FEC2              <1>     inc     dl
00003FA3 EC                <1>     in      al, dx
00003FA4 249F              <1>     and     al, 9Fh  ; clear shift register
00003FA6 0C40              <1>     or      al, 40h  ; set shift register to 2
00003FA8 EE                <1>     out     dx, al
                           <1>
                           <1> vga_compat_end:
                           <1>     ;pop    edx
                           <1>     ;pop    eax
00003FA9 C3                <1>     retn
                           <1>
                           <1> vga_set_virt_width:
                           <1>     ; 03/08/2022
                           <1>     ; 27/11/2020
                           <1>     ; 25/11/2020
                           <1>     ; (vbe.c, 02/01/2020, vruppert)
                           <1>     ;
                           <1>     ; Input:
                           <1>     ;       AX = resolution (screen width)
                           <1>     ;
                           <1>     ; Modified registers: eax, edx
                           <1>
                           <1>     ;;push  ebx
                           <1>     ;push   edx
                           <1>     ;push   eax
                           <1>     ;mov    ebx, eax
                           <1>     ;call   dispi_get_bpp ; bits per pixel
                           <1>     ;cmp    al, 4
                           <1>     ;ja     short set_width_svga  ; 8, 16, 24, 32
                           <1>     ;shr    bx, 1
                           <1> ;set_width_svga:
                           <1>     ;shr    bx, 3
                           <1>     ;mov    eax, [esp]
                           <1>     ;shr    ax, 3   ; / 8, bytes per row
                           <1>     ; 03/08/2022
00003FAA C1E803            <1>     shr     eax, 3
00003FAD 66BAD403          <1>     mov     dx, 3D4h ; VGAREG_VGA_CRTC_ADDRESS
                           <1>     ;mov    ah, bl  ;
00003FB1 88C4              <1>     mov     ah, al  ; width in bytes
00003FB3 B013              <1>     mov     al, 13h ; offset register
00003FB5 66EF              <1>     out     dx, ax  ; index (3D4h) and data (3D5h)
                           <1>     ;pop    eax
                           <1>     ;pop    edx
```

```
                                <1>     ;;pop   ebx
00003FB7 C3                     <1>     retn
                                <1>
                                <1> ; 24/11/2020
                                <1>
                                <1> struc bmi ; BOCHS/PLEX86 MODE INFO structure/table
00000000 ????                   <1>  .mode:      resw 1
00000002 ????                   <1>  .width:  resw 1
00000004 ????                   <1>  .height: resw 1
00000006 ????                   <1>  .depth:  resw 1
                                <1>  .size:
                                <1> endstruc
                                <1>
                                <1> ; 24/11/2020
                                <1> struc MODEINFO
00000000 ????                   <1>  .mode:                   resw 1  ; 1XXh
00000002 ????                   <1>  .ModeAttributes:  resw 1
00000004 ??                     <1>  .WinAAttributes:  resb 1
00000005 ??                     <1>  .WinBAttributes:  resb 1 ; = 0
00000006 ????                   <1>  .WinGranularity:  resw 1
00000008 ????                   <1>  .WinSize:         resw 1
0000000A ????                   <1>  .WinASegment:            resw 1
0000000C ????                   <1>  .WinBSegment:            resw 1 ; = 0
0000000E ????????               <1>  .WinFuncPtr:             resd 1 ; = 0
00000012 ????                   <1>  .BytesPerScanLine:       resw 1
00000014 ????                   <1>  .XResolution:            resw 1
00000016 ????                   <1>  .YResolution:            resw 1
00000018 ??                     <1>  .XCharSize:              resb 1
00000019 ??                     <1>  .YCharSize:              resb 1
0000001A ??                     <1>  .NumberOfPlanes:  resb 1
0000001B ??                     <1>  .BitsPerPixel:           resb 1
0000001C ??                     <1>  .NumberOfBanks:   resb 1
0000001D ??                     <1>  .MemoryModel:            resb 1
0000001E ??                     <1>  .BankSize:        resb 1 ; = 0
0000001F ??                     <1>  .NumberOfImagePages:     resb 1
00000020 ??                     <1>  .Reserved_page:   resb 1 ; = 0
00000021 ??                     <1>  .RedMaskSize:            resb 1
00000022 ??                     <1>  .RedFieldPosition:       resb 1
00000023 ??                     <1>  .GreenMaskSize:   resb 1
00000024 ??                     <1>  .GreenFieldPosition:     resb 1
00000025 ??                     <1>  .BlueMaskSize:           resb 1
00000026 ??                     <1>  .BlueFieldPosition:      resb 1
00000027 ??                     <1>  .RsvdMaskSize:           resb 1
00000028 ??                     <1>  .RsvdFieldPosition:      resb 1
00000029 ??                     <1>  .DirectColorModeInfo:    resb 1
0000002A ????????               <1>  .PhysBasePtr:            resd 1
0000002E ????????               <1>  .OffScreenMemOffset:     resd 1 ; = 0
00000032 ????                   <1>  .OffScreenMemSize:       resw 1 ; = 0
00000034 ????                   <1>  .LinBytesPerScanLine:    resw 1
00000036 ??                     <1>  .BnkNumberOfPages:       resb 1
00000037 ??                     <1>  .LinNumberOfPages:       resb 1
00000038 ??                     <1>  .LinRedMaskSize:  resb 1
00000039 ??                     <1>  .LinRedFieldPosition1: resb 1
0000003A ??                     <1>  .LinGreenMaskSize1:      resb 1
0000003B ??                     <1>  .LinGreenFieldPosition:resb 1
0000003C ??                     <1>  .LinBlueMaskSize:        resb 1
0000003D ??                     <1>  .LinBlueFieldPosition:   resb 1
0000003E ??                     <1>  .LinRsvdMaskSize:        resb 1
0000003F ??                     <1>  .LinRsvdFieldPosition:   resb 1
00000040 ????????               <1>  .MaxPixelClock:   resd 1 ; = 0
                                <1> .size:
                                <1> endstruc
                                <1>
                                <1> ; 10/12/2020
                                <1> struc LFBINFO
00000000 ????                   <1>  .mode:                   resw 1  ; 1XXh
00000002 ????????               <1>  .LFB_addr:        resd 1
00000006 ????????               <1>  .LFB_size:        resd 1
0000000A ????                   <1>  .X_res:           resw 1
0000000C ????                   <1>  .Y_res:           resw 1
0000000E ??                     <1>  .bpp:                    resb 1
0000000F ??                     <1>  .reserved:        resb 1
                                <1> .size:      ; 16 bytes
                                <1> endstruc
                                <1>
                                <1> set_mode_info_list:
                                <1>     ; 14/12/2020
                                <1>     ; 11/12/2020
                                <1>     ; 24/11/2020
                                <1>     ; (vbetables-gen.c)
                                <1>     ; Input:
                                <1>     ;       BX = VBE mode (including bochs special modes)
                                <1>     ; Output:
                                <1>     ;       ;;EAX = MODE_INFO_LIST address
                                <1>     ;       EAX = 0 ; 11/12/2020
                                <1>     ;       ESI = MODE_INFO_LIST address ; 11/12/2020
                                <1>     ;       (if mode is not found, ESI = 0)
                                <1>     ;
                                <1>     ; Modified registers: eax, ebx, ecx, edx, esi, edi
                                <1>
00003FB8 BE[BA6B0000]           <1>     mov     esi, b_vbe_modes ; bochs mode info base table
00003FBD BF[429D0100]           <1>     mov     edi, MODE_INFO_LIST ; mode info list (4F01h)
                                <1> sml_0:
00003FC2 66AD                   <1>     lodsw
00003FC4 6639D8                 <1>     cmp     ax, bx ; is mode number same ?
00003FC7 7410                   <1>     je      short sml_1 ; yes
00003FC9 AD                     <1>     lodsd
00003FCA 66AD                   <1>     lodsw
00003FCC 81FE[7A6C0000]         <1>     cmp     esi, end_of_b_vbe_modes
00003FD2 72EE                   <1>     jb      short sml_0
                                <1>     ; not found
00003FD4 31C0                   <1>     xor     eax, eax ; 0
                                <1>     ; 11/12/2020
00003FD6 31F6                   <1>     xor     esi, esi
00003FD8 C3                     <1>     retn
                                <1> sml_1:
00003FD9 66AB                   <1>     stosw   ; mode
00003FDB AD                     <1>     lodsd   ; width, height
                                <1>     ; 14/12/2020
00003FDC 89C1                   <1>     mov     ecx, eax
00003FDE 50                     <1>     push    eax ; ***
00003FDF 29C0                   <1>     sub     eax, eax ; clear high word of eax
00003FE1 66AD                   <1>     lodsw   ; depth
00003FE3 50                     <1>     push    eax ; **
                                <1>
                                <1>     ;add    al, 7 ; only for 15 bit colors (not used here)
00003FE4 C0E803                 <1>     shr     al, 3 ; / 8
                                <1>     ; 14/12/2020
00003FE7 66F7E1                 <1>     mul     cx  ; pitch = width * ((depth+7)/8)
                                <1>             ; ax = pitch
00003FEA 50                     <1>     push    eax ; * ; high word of eax = 0
00003FEB C1E910                 <1>     shr     ecx, 16
                                <1>     ;mul    cx
                                <1>     ;mov    cx, ax
```

```
00003FEE 31D2              <1>     xor     edx, edx  ; clear high word of edx
00003FF0 F7E1              <1>     mul     ecx ; height * pitch
00003FF2 89C1              <1>     mov     ecx, eax
00003FF4 B800000001        <1>     mov     eax, VBE_DISPI_TOTAL_VIDEO_MEMORY_MB * 1024 * 1024
00003FF9 F7F1              <1>     div     ecx
                           <1>             ; eax = pages = vram_size / (height*pitch)
                           <1>
                           <1>     ;mov    cx, ax
00003FFB 89C1              <1>     mov     ecx, eax ; pages
                           <1>
00003FFD 66B89B00          <1>     mov     ax, MODE_ATTRIBUTES
00004001 66AB              <1>     stosw   ; ModeAttributes
00004003 B007              <1>     mov     al, WINA_ATTRIBUTES
00004005 AA                <1>     stosb   ; WinAAttributes
00004006 30C0              <1>     xor     al, al ; WinBAttributes = 0
00004008 AA                <1>     stosb
00004009 66B84000          <1>     mov     ax, VBE_DISPI_BANK_SIZE_KB
0000400D 66AB              <1>     stosw   ; WinGranularity
0000400F 66AB              <1>     stosw   ; WinSize
00004011 66B800A0          <1>     mov     ax, VGAMEM_GRAPH
00004015 66AB              <1>     stosw   ; WinASegment
00004017 29C0              <1>     sub     eax, eax
00004019 66AB              <1>     stosw   ; WinBSegment = 0
0000401B AB                <1>     stosd   ; WinFuncPtr = 0
                           <1>
0000401C 58                <1>     pop     eax ; * ; pitch
0000401D 89C3              <1>     mov     ebx, eax  ; high word of ebx = 0 ; 14/12/2020
0000401F 66AB              <1>     stosw   ; BytesPerScanLine
                           <1>
00004021 5A                <1>     pop     edx ; ** ; depth (bits per pixel)
00004022 58                <1>     pop     eax ; *** width, height
                           <1>
                           <1>     ; // Mandatory information for VBE 1.2 and above
                           <1>
00004023 66AB              <1>     stosw   ; XResolution (width)
00004025 C1E810            <1>     shr     eax, 16
00004028 50                <1>     push    eax ; **** height
00004029 66AB              <1>     stosw   ; YResolution (height)
0000402B B008              <1>     mov     al, 8
0000402D AA                <1>     stosb   ; XCharSize  ; char width
0000402E B010              <1>     mov     al, 16
00004030 AA                <1>     stosb   ; YCharSize  ; char height
00004031 B001              <1>     mov     al, 1
00004033 AA                <1>     stosb   ; NumberOfPlanes
                           <1>     ;movzx  eax, dl
00004034 88D0              <1>     mov     al, dl ; eax <= 32
00004036 AA                <1>     stosb   ; BitsPerPixel
                           <1>     ; Number of banks = (height * pitch + 65535) / 65536
00004037 58                <1>     pop     eax ; **** ; height
                           <1>     ; 14/12/2020
00004038 52                <1>     push    edx ; ***** ; depth ; edx <= 32
00004039 F7E3              <1>     mul     ebx  ; pitch (ebx) * height (eax)
                           <1>     ;mov    edx, [esp] ; *****
                           <1>     ;mov    dl, [esp] ; *****
0000403B 05FFFF0000        <1>     add     eax, 65535
00004040 C1E810            <1>     shr     eax, 16 ; / 65536 ; <= 127 ; 14/12/2020
00004043 AA                <1>     stosb   ; NumberOfBanks
                           <1>     ; 14/12/2020
                           <1>     ;cmp    dl, 8 ; 8 bits per pixel
00004044 803C2408          <1>     cmp     byte [esp], 8
00004048 7704              <1>     ja      short sml_2
0000404A B004              <1>     mov     al, VBE_MEMORYMODEL_PACKED_PIXEL
0000404C EB02              <1>     jmp     short sml_3
                           <1> sml_2:
                           <1>     ; 16, 24, 32 bits per pixel
0000404E B006              <1>     mov     al, VBE_MEMORYMODEL_DIRECT_COLOR
                           <1> sml_3:
00004050 AA                <1>     stosb
00004051 30C0              <1>     xor     al, al ; 0
00004053 AA                <1>     stosb   ; BankSize = 0
00004054 49                <1>     dec     ecx ; pages - 1
                           <1>     ; NumberOfImagePages = 262 for 320x200x8 mode
                           <1>     ;;mov   ax, 255
                           <1>     ; 14/12/2020
                           <1>     ;mov    al, 255
00004055 FEC8              <1>     dec     al ; 255
00004057 39C1              <1>     cmp     ecx, eax ; ecx <= 261, eax = 255
                           <1>     ;cmp    cx, ax
00004059 7302              <1>     jnb     short sml_4
0000405B 88C8              <1>     mov     al, cl
                           <1> sml_4:
0000405D AA                <1>     stosb   ; NumberOfImagePages (1 byte)
0000405E 28C0              <1>     sub     al, al
00004060 AA                <1>     stosb   ; Reserved_page = 0
00004061 58                <1>     pop     eax ; ***** ; depth
00004062 88C1              <1>     mov     cl, al
                           <1>     ; eax < = 32
00004064 2C08              <1>     sub     al, 8 ; 8->0, 16->8, 24->16, 32->24
00004066 BE[7A6C0000]      <1>     mov     esi, direct_color_fields
0000406B 01C6              <1>     add     esi, eax
0000406D 56                <1>     push    esi ; ******
0000406E AD                <1>     lodsd   ; RedMaskSize (AL), RedFieldPosition (AH)
                           <1>             ; GreenMaskSize (16), GreenFieldPosition (24)
0000406F AB                <1>     stosd
00004070 AD                <1>     lodsd   ; BlueMaskSize (AL), BlueFieldPosition (AH)
                           <1>             ; RsvdMaskSize (16), RsvdFieldPosition (24)
00004071 AB                <1>     stosd
00004072 5E                <1>     pop     esi ; ******
                           <1>
00004073 30C0              <1>     xor     al, al ; 0
00004075 80F920            <1>     cmp     cl, 32
00004078 7202              <1>     jb      short sml_5
0000407A B002              <1>     mov     al, VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
                           <1> sml_5:
0000407C AA                <1>     stosb   ; DirectColorModeInfo
                           <1>
                           <1>     ; // Mandatory information for VBE 2.0 and above
                           <1>
0000407D B8000000E0        <1>     mov     eax, VBE_DISPI_LFB_PHYSICAL_ADDRESS
00004082 AB                <1>     stosd   ; PhysBasePtr
00004083 29C0              <1>     sub     eax, eax
00004085 AB                <1>     stosd   ; OffScreenMemOffset = 0
00004086 66AB              <1>     stosw   ; OffScreenMemSize = 0
                           <1>
                           <1>     ;// Mandatory information for VBE 3.0 and above
                           <1>
                           <1>     ; ebx = pitch
00004088 6689D8            <1>     mov     ax, bx
                           <1>     ;stosw
                           <1>
                           <1>     ;xor    al, al
                           <1>             ;stosb  ; BnkNumberOfPages = 0
                           <1>             ;stosb  ; LinNumberOfPages = 0
                           <1>
```

```
0000408B AB                 <1>     stosd   ; pitch (word), 0 (byte), 0 (byte)
                            <1>
0000408C AD                 <1>     lodsd   ; LinRedMaskSize (AL), LinRedFieldPosition (AH)
                            <1>             ; LinGreenMaskSize (16), LinGreenFieldPosition (24)
0000408D AB                 <1>     stosd
0000408E AD                 <1>     lodsd   ; LinBlueMaskSize (AL), LinBlueFieldPosition (AH)
                            <1>             ; LinRsvdMaskSize (16), LinRsvdFieldPosition (24)
0000408F AB                 <1>     stosd
                            <1>
00004090 29C0               <1>     sub     eax, eax
00004092 AB                 <1>     stosd   ; MaxPixelClock = 0
                            <1>
                            <1>     ;mov    eax, MODE_INFO_LIST
                            <1>     ; 11/12/2020
00004093 BE[429D0100]       <1>     mov     esi, MODE_INFO_LIST
                            <1>
00004098 C3                 <1>     retn
                            <1>
                            <1> ; end of set_mode_info_list ; edi = set_mode_info_list + 68
                            <1>
                            <1> pci_get_lfb_addr:
                            <1>     ; 11/12/2020
                            <1>     ; Get linear frame buffer base from PCI
                            <1>     ; (vgabios.c, 02/01/2020, vruppert)
                            <1>     ;
                            <1>     ; Input:
                            <1>     ;       ax = PCI device vendor id
                            <1>     ; Output:
                            <1>     ;       ax  = LFB address (high 16 bit) (zf=0)
                            <1>     ;       eax = 0 -> not found (error) (zf=1)
                            <1>     ;
                            <1>     ; Modified registers: eax
                            <1>
00004099 53                 <1>     push    ebx
0000409A 51                 <1>     push    ecx
0000409B 52                 <1>     push    edx
                            <1>     ;
0000409C 89C3               <1>     mov     ebx, eax
0000409E 31C9               <1>     xor     ecx, ecx
000040A0 28D2               <1>     sub     dl, dl ; mov dl, 0
000040A2 E842000000         <1>     call    pci_read_reg
000040A7 6683F8FF           <1>     cmp     ax, 0FFFFh
000040AB 7417               <1>     je      short pci_get_lfb_addr_fail
                            <1> pci_get_lfb_addr_next_dev:
000040AD 28D2               <1>     sub     dl, dl ; mov dl, 0
000040AF E835000000         <1>     call    pci_read_reg
000040B4 6639D8             <1>     cmp     ax, bx ; check vendor
000040B7 740F               <1>     je      short pci_get_lfb_addr_found
000040B9 6683C108           <1>     add     cx, 08h
000040BD 6681F90002         <1>     cmp     cx, 200h ; search bus 0 and 1
000040C2 72E9               <1>     jb      short pci_get_lfb_addr_next_dev
                            <1> pci_get_lfb_addr_fail:
000040C4 31C0               <1>     xor     eax, eax ; no LFB
                            <1>     ; zf = 1
000040C6 EB1D               <1>     jmp     short pci_get_lfb_addr_return
                            <1> pci_get_lfb_addr_found:
000040C8 B210               <1>     mov     dl, 10h ; I/O space 0
000040CA E81A000000         <1>     call    pci_read_reg
000040CF 66A9F1FF           <1>     test    ax, 0FFF1h
000040D3 740D               <1>     jz      short pci_get_lfb_addr_success
000040D5 B214               <1>     mov     dl, 14h ; I/O space 1
000040D7 E80D000000         <1>     call    pci_read_reg
000040DC 66A9F1FF           <1>     test    ax, 0FFF1h
000040E0 75E2               <1>     jnz     short pci_get_lfb_addr_fail
                            <1> pci_get_lfb_addr_success:
000040E2 C1E810             <1>     shr     eax, 16 ; LFB address (hw)
                            <1>     ; zf = 0
                            <1> pci_get_lfb_addr_return:
000040E5 5A                 <1>     pop     edx
000040E6 59                 <1>     pop     ecx
000040E7 5B                 <1>     pop     ebx
000040E8 C3                 <1>     retn
                            <1>
                            <1> pci_read_reg:
                            <1>     ; 11/12/2020
                            <1>     ; Read PCI register
                            <1>     ; (vgabios.c, 02/01/2020, vruppert)
                            <1>     ;
                            <1>     ; Input:
                            <1>     ;       cx = device/function
                            <1>     ;       dl = register
                            <1>     ; Output:
                            <1>     ;       eax = value
                            <1>     ;
                            <1>     ; Modified registers: eax, edx
                            <1>
000040E9 B800008000         <1>     mov     eax, 00800000h
000040EE 6689C8             <1>     mov     ax, cx
000040F1 C1E008             <1>     shl     eax, 8
000040F4 88D0               <1>     mov     al, dl
000040F6 66BAF80C           <1>     mov     dx, 0CF8h
000040FA EF                 <1>     out     dx, eax
000040FB 80C204             <1>     add     dl, 4 ; mov dx, 0CFCh
000040FE ED                 <1>     in      eax, dx
000040FF C3                 <1>     retn
                            <1>
                            <1> %endif
                            <1>
                            <1> ; -----------
                            <1>
                            <1> %if 0
                            <1>
                            <1> mode_info_find_mode:
                            <1>     ; 25/11/2020
                            <1>     ; Input:
                            <1>     ;       bx = VESA VBE2 video mode (+ bochs extensions)
                            <1>     ; Output:
                            <1>     ;       esi = mode info address (for BX input)
                            <1>     ;       esi = 0 -> not found
                            <1>     ;
                            <1>     ; Modified registers: eax, esi
                            <1>
                            <1>     xor     eax, eax
                            <1>     mov     esi, MODE_INFO_LIST
                            <1> mifm_1:
                            <1>     mov     ax, [esi]
                            <1>     cmp     ax, bx
                            <1>     je      short mifm_2
                            <1>     add     esi, MODEINFO.size ; add esi, 68
                            <1>     cmp     esi, VBE_VESA_MODE_END_OF_LIST
                            <1>     jb      short mifm_1
                            <1>     ; not found
                            <1>     sub     esi, esi ; 0
                            <1> mifm_2
```

```
<1>     retn
<1>
<1> dispi_get_bpp:
<1>     ; 28/11/2020
<1>     ; Input:
<1>     ;       none
<1>     ; Output:
<1>     ;       al = Bits per pixel
<1>     ;            (8,16,24,32)
<1>     ;       ah = Bytes per pixel
<1>     ;            (1,2,3,4)
<1>     ;
<1>     ; Modified registers: none
<1>
<1>     ;push   edx
<1>     ;mov    dx, 01CEh ; VBE_DISPI_IOPORT_INDEX
<1>     ;mov    ax, 03h   ; VBE_DISPI_INDEX_BPP
<1>     ;out    dx, ax
<1>     ;;mov   dx, 01CFh ; VBE_DISPI_IOPORT_DATA
<1>     ;;mov   dl, 0CFh
<1>     ;inc    dl
<1>     ;in     ax, dx
<1>     ;mov    ah, al
<1>     ;shr    ah, 3 ; / 8
<1>     ;;test  al, 7 ; 15 bit graphics mode
<1>     ;;jz    short get_bpp_noinc
<1>     ;;inc   ah
<1> ;;get_bpp_noinc:
<1>     ;pop    edx
<1>     ;retn
<1>
<1>     ; 28/11/2020
<1>     ; Modified registers: edx
<1>     ;push   edx
<1>     mov     dx, 03h ; VBE_DISPI_INDEX_BPP
<1>     call    dispi_get_parms
<1>     ;pop    edx
<1>     ;retn
<1>     mov     ah, al
<1>     shr     ah, 3 ; / 8
<1>     ;test   al, 7 ; 15 bit graphips mode
<1>     ;jz     short get_bpp_noinc
<1>     ;inc    ah
<1> ;get_bpp_noinc:
<1>     ;pop    edx
<1>     retn
<1>
<1> restore_vesa_video_state:
<1>     ; 02/08/2022 (TRDOS 386 v2.0.5)
<1>     ; 14/01/2021
<1>     ; 06/12/2020
<1>     ; Input:
<1>     ;       [vbe3stbufsize] <= 32 ; <= 32*64 bytes
<1>     ; Output:
<1>     ;       AX = 004Fh (successed)
<1>     ;
<1>     ;       eax = 0 -> buffer size problem
<1>     ;       eax > 0 and ax <> 004Fh -> failed
<1>     ;
<1>     ; Modified regs: eax, ebx, ecx, edx, esi, edi
<1>
<1>     ;movzx  ecx, word [vbe3stbufsize]
<1>     ;cmp    cx, 32 ; 32 * 64 bytes
<1>     ;ja     short r_v_b_s_fail
<1>
<1>     movzx   ecx, byte [vbe3stbufsize]; <=32
<1>     ;shl    cx, 4 ; dword count for movsd
<1>     ; 02/08/2022
<1>     shl     ecx, 4 ; <= 32*16 dwords (32*64 bytes)
<1>     mov     edi, VBE3SAVERESTOREBLOCK ; destination
<1>                                       ; (vbe3 pmi buff)
<1>     mov     esi, VBE3VIDEOSTATE ; source (kernel buff)
<1>     rep     movsd
<1>
<1>     mov     ax, 4F04h
<1>     mov     dl, 02h ; restore
<1>     ;mov    cx, 0Fh
<1>     mov     cl, 0Fh
<1>     xor     ebx, ebx ; points to VBE3SAVERESTOREBLOCK
<1>     jmp     short int10h_32bit_pmi
<1>
<1> ;s_v_b_s_fail:
<1> ;r_v_b_s_fail:
<1> ;   xor     eax, eax
<1> ;   retn
<1>
<1> save_vesa_video_state:
<1>     ; 02/08/2022 (TRDOS 386 v2.0.5)
<1>     ; 14/01/2021
<1>     ; 06/12/2020
<1>     ; Input:
<1>     ;       [vbe3stbufsize] <= 32 ; <= 32*64 bytes
<1>     ; Output:
<1>     ;       AX = 004Fh (successed)
<1>     ;
<1>     ;       eax = 0 -> buffer size problem
<1>     ;       eax > 0 and ax <> 004Fh -> failed
<1>     ;
<1>     ; Modified regs: eax, ebx, ecx, edx, esi, edi
<1>
<1>     ;cmp    word [vbe33stbufsize], 32
<1>     ;                      ; 32 * 64 bytes
<1>     ;ja     short s_v_b_s_fail
<1>
<1>     mov     ax, 4F04h
<1>     mov     dl, 01h ; save
<1>     ;mov    cx, 0Fh
<1>     mov     cl, 0Fh
<1>     xor     ebx, ebx ; points to VBE3SAVERESTOREBLOCK
<1>
<1>     call    int10h_32bit_pmi
<1>
<1>     movzx   ecx, byte [vbe3stbufsize]; <=32
<1>     ;shl    cx, 4 ; dword count for movsd
<1>     ; 02/08/2022
<1>     shl     ecx, 4 ; <= 32*16 dwords (32*64 bytes)
<1>     mov     esi, VBE3SAVERESTOREBLOCK ; destination
<1>                                       ; (vbe3 pmi buff)
<1>     mov     edi, VBE3VIDEOSTATE ; source (kernel buff)
<1>     rep     movsd
<1>     retn
<1>
<1> dispi_set_bank_farcall:
<1>     ; 03/08/2022
```

```
                                <1>     ; 12/04/2021 (32 bit push/pop)
                                <1>     ; 11/12/2020
                                <1>     ; (This may be 'sysvideo' function, later)
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       bx = 0000h, set bank number
                                <1>     ;          = 0100h, get bank number
                                <1>     ;       dx = bank number (if bx = 0)
                                <1>     ; Output:
                                <1>     ;       dx = bank number
                                <1>
                                <1>     cmp     bx, 0100h
                                <1>     je      short dispi_set_bank_farcall_get
                                <1>     or      bx, bx
                                <1>     ;jnz    dispi_set_bank_farcall_error
                                <1>     ; 03/08/2022
                                <1>     jz      short dsbfcall_1
                                <1>     jmp     dispi_set_bank_farcall_error
                                <1> dsbfcall_1:
                                <1>     mov     ax, dx
                                <1>     ;push   dx
                                <1>     ;push   ax
                                <1>     ; 12/04/2021
                                <1>     push    edx
                                <1>     push    eax
                                <1>     mov     ax, VBE_DISPI_INDEX_BANK
                                <1>     mov     dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     out     dx, ax
                                <1>     ;pop    ax
                                <1>     ; 12/04/2021
                                <1>     pop     eax
                                <1>     ;;mov   dx, VBE_DISPI_IOPORT_DATA
                                <1>     ;mov    dl, 0CFh
                                <1>     inc     dl ; 1CFh = VBE_DISPI_IOPORT_DATA
                                <1>     out     dx, ax
                                <1>     in      ax, dx
                                <1>     ;pop    dx
                                <1>     ; 12/04/2021
                                <1>     pop     edx
                                <1>     cmp     dx, ax
                                <1>     jne     short dispi_set_bank_farcall_error
                                <1>     mov     ax, 004Fh
                                <1>     retn    ; retf for real mode far call
                                <1> dispi_set_bank_farcall_get:
                                <1>     mov     ax, VBE_DISPI_INDEX_BANK
                                <1>     ; 03/08/2022
                                <1>     mov     dx, 1CEh ; VBE_DISPI_IOPORT_INDEX
                                <1>     out     dx, ax
                                <1>     ;;mov   dx, VBE_DISPI_IOPORT_DATA
                                <1>     ;mov    dl, 0CFh
                                <1>     ; 03/08/2022
                                <1>     inc     dl ; 1CFh = VBE_DISPI_IOPORT_DATA
                                <1>     in      ax, dx
                                <1>     mov     dx, ax
                                <1>     retn    ; retf for real mode far call
                                <1> dispi_set_bank_farcall_error:
                                <1>     mov     ax, 014Fh
                                <1>     retn    ; retf for real mode far call
                                <1>
                                <1> %endif
                                <1>
                                <1> ; % include 'vidata.s' ; VIDEO DATA
                                <1>
                                <1> ; /// End Of VIDEO FUNCTIONS ///

                                    setup_rtc_int:
                                    ; source: http://wiki.osdev.org/RTC
00004100 FA                             cli             ; disable interrupts
                                        ; default int frequency is 1024 Hz (Lower 4 bits of register A is 0110b or 6)
                                        ; in order to change this ...
                                        ; frequency  = 32768 >> (rate-1) --> 32768 >> 5 = 1024
                                        ; (rate must be above 2 and not over 15)
                                        ; new rate = 15 --> 32768 >> (15-1) = 2 Hz
00004101 B08A                           mov     al, 8Ah
00004103 E670                           out     70h, al ; set index to register A, disable NMI
00004105 90                             nop
00004106 E471                           in      al, 71h ; get initial value of register A
00004108 88C4                           mov     ah, al
0000410A 80E4F0                         and     ah, 0F0h
0000410D B08A                           mov     al, 8Ah
0000410F E670                           out     70h, al ; reset index to register A
00004111 88E0                           mov     al, ah
00004113 0C0F                           or      al, 0Fh ; new rate (0Fh -> 15)
00004115 E671                           out     71h, al ; write only our rate to A. Note, rate is the bottom 4 bits.
                                        ; enable RTC interrupt
00004117 B08B                           mov     al, 8Bh ;
00004119 E670                           out     70h, al ; select register B and disable NMI
0000411B 90                             nop
0000411C E471                           in      al, 71h ; read the current value of register B
0000411E 88C4                           mov     ah, al  ;
00004120 B08B                           mov     al, 8Bh ;
00004122 E670                           out     70h, al ; set the index again (a read will reset the index to register B)
00004124 88E0                           mov     al, ah  ;
00004126 0C40                           or      al, 40h ;
00004128 E671                           out     71h, al ; write the previous value ORed with 0x40. This turns on bit 6 of register B
0000412A FB                             sti
0000412B C3                             retn

                                    ; Write memory information
                                    ; 29/01/2016
                                    ; 06/11/2014
                                    ; 14/08/2015
                                    memory_info:
0000412C A1[B4760100]                   mov     eax, [memory_size] ; in pages
00004131 50                             push    eax
00004132 C1E00C                         shl     eax, 12           ; in bytes
00004135 BB0A000000                     mov     ebx, 10
0000413A 89D9                           mov     ecx, ebx          ; 10
0000413C BE[71380100]                   mov     esi, mem_total_b_str
00004141 E8D8000000                     call    bintdstr
00004146 58                             pop     eax
00004147 B107                           mov     cl, 7
00004149 BE[95380100]                   mov     esi, mem_total_p_str
0000414E E8CB000000                     call    bintdstr
                                        ; 14/08/2015
00004153 E8E3000000                     call    calc_free_mem
                                        ; edx = calculated free pages
                                        ; ecx = 0
00004158 A1[B8760100]                   mov     eax, [free_pages]
0000415D 39D0                           cmp     eax, edx ; calculated free mem value
                                                ; and initial free mem value are same or not?
0000415F 751D                           jne     short pmim ; print mem info with '?' if not
00004161 52                             push    edx ; free memory in pages
                                        ;mov    eax, edx
```

```
00004162 C1E00C                     shl     eax, 12 ; convert page count
                                                    ; to byte count
00004165 B10A                       mov     cl, 10
00004167 BE[B5380100]               mov     esi, free_mem_b_str
0000416C E8AD000000                 call    bintdstr
00004171 58                         pop     eax
00004172 B107                       mov     cl, 7
00004174 BE[D9380100]               mov     esi, free_mem_p_str
00004179 E8A0000000                 call    bintdstr
                                pmim:
0000417E BE[5F380100]               mov     esi, msg_memory_info
                                    ;
00004183 B407                       mov     ah, 07h ; Black background,
                                                    ; light gray forecolor
                                print_kmsg: ; 29/01/2016
00004185 8825[DF760100]             mov     [ccolor], ah
                                pkmsg_loop:
0000418B AC                         lodsb
0000418C 08C0                       or      al, al
0000418E 7410                       jz      short pkmsg_ok
00004190 56                         push    esi
                                    ; 13/05/2016
00004191 0FB61D[DF760100]           movzx   ebx, byte [ccolor]
                                                    ; Video page 0 (bh=0)
00004198 E842E1FFFF                 call    _write_tty
0000419D 5E                         pop     esi
0000419E EBEB                       jmp     short pkmsg_loop
                                pkmsg_ok:
000041A0 C3                         retn

                                ; 19/12/2020
                                ; temporary
                                ; Write default liner frame buffer address
                                ;
                                default_lfb_info:
000041A1 66A1[100F0000]             mov     ax, [def_LFB_addr] ; high word
                                    ; 24/11/2023 - temporary
000041A7 6609C0                     or      ax, ax
000041AA 74F4                       jz      short pkmsg_ok
000041AC E829000000                 call    wordtohex
000041B1 A3[31390100]               mov     dword [lfb_addr_str], eax
000041B6 BE[1A390100]               mov     esi, msg_lfb_addr
000041BB B40F                       mov     ah, 0Fh ; Black background,
                                                    ; white forecolor
000041BD EBC6                       jmp     short print_kmsg

                                ; Convert binary number to hexadecimal string
                                ; 10/05/2015
                                ; dsectpm.s (28/02/2015)
                                ; Retro UNIX 386 v1 - Kernel v0.2.0.6
                                ; 01/12/2014
                                ; 25/11/2014
                                ;
                                bytetohex:
                                    ; INPUT ->
                                    ;       AL = byte (binary number)
                                    ; OUTPUT ->
                                    ;       AX = hexadecimal string
                                    ;
000041BF 53                         push    ebx
000041C0 31DB                       xor     ebx, ebx
000041C2 88C3                       mov     bl, al
000041C4 C0EB04                     shr     bl, 4
000041C7 8A9B[0D420000]             mov     bl, [ebx+hexchrs]
000041CD 86D8                       xchg    bl, al
000041CF 80E30F                     and     bl, 0Fh
000041D2 8AA3[0D420000]             mov     ah, [ebx+hexchrs]
000041D8 5B                         pop     ebx
000041D9 C3                         retn

                                wordtohex:
                                    ; INPUT ->
                                    ;       AX = word (binary number)
                                    ; OUTPUT ->
                                    ;       EAX = hexadecimal string
                                    ;
000041DA 53                         push    ebx
000041DB 31DB                       xor     ebx, ebx
000041DD 86E0                       xchg    ah, al
000041DF 6650                       push    ax ; * save ax
000041E1 88E3                       mov     bl, ah
000041E3 C0EB04                     shr     bl, 4
000041E6 8A83[0D420000]             mov     al, [ebx+hexchrs]
000041EC 88E3                       mov     bl, ah
000041EE 80E30F                     and     bl, 0Fh
000041F1 8AA3[0D420000]             mov     ah, [ebx+hexchrs]
000041F7 C1E010                     shl     eax, 16 ; ax -> hw of eax
000041FA 6658                       pop     ax ; * restore ax
000041FC 5B                         pop     ebx
000041FD EBC0                       jmp     short bytetohex
                                    ;mov    bl, al
                                    ;shr    bl, 4
                                    ;mov    bl, [ebx+hexchrs]
                                    ;xchg   bl, al
                                    ;and    bl, 0Fh
                                    ;mov    ah, [ebx+hexchrs]
                                    ;pop    ebx
                                    ;retn

                                dwordtohex:
                                    ; INPUT ->
                                    ;       EAX = dword (binary number)
                                    ; OUTPUT ->
                                    ;       EDX:EAX = hexadecimal string
                                    ;
000041FF 50                         push    eax
00004200 C1E810                     shr     eax, 16
00004203 E8D2FFFFFF                 call    wordtohex
00004208 89C2                       mov     edx, eax
0000420A 58                         pop     eax
                                    ;call   wordtohex
                                    ;retn
                                    ; 18/04/2021
0000420B EBCD                       jmp     short wordtohex

                                ; 10/05/2015
                                hex_digits:
                                hexchrs:
0000420D 303132333435363738-        db '0123456789ABCDEF'
00004216 39414243444546
                                ; 19/01/2021 - VESA EDID ready flag (4Fh)
0000421D 00                     edid:       db 0

                                ; Convert binary number to decimal/numeric string
```

```
                                    ; 06/11/2014
                                    ; Temporary Code
                                    ;

                                    bintdstr:
                                        ; EAX = binary number
                                        ; ESI = decimal/numeric string address
                                        ; EBX = divisor (10)
                                        ; ECX = string length (<=10)
0000421E 01CE                           add     esi, ecx
                                    btdstr0:
00004220 4E                             dec     esi
00004221 31D2                           xor     edx, edx
00004223 F7F3                           div     ebx
00004225 80C230                         add     dl, 30h
00004228 8816                           mov     [esi], dl
0000422A FEC9                           dec     cl
0000422C 740C                           jz      short btdstr2 ; 08/09/2016
0000422E 09C0                           or      eax, eax
00004230 75EE                           jnz     short btdstr0
                                    btdstr1:
00004232 4E                             dec     esi
00004233 C60620                             mov     byte [esi], 20h ; blank space
00004236 FEC9                           dec     cl
00004238 75F8                           jnz     short btdstr1
                                    btdstr2:
0000423A C3                             retn

                                    ; Calculate free memory pages on M.A.T.
                                    ; 06/11/2014
                                    ; Temporary Code
                                    ;

                                    calc_free_mem:
0000423B 31D2                           xor     edx, edx
                                        ;xor    ecx, ecx
0000423D 668B0D[C8760100]               mov     cx, [mat_size] ; in pages
00004244 C1E10A                         shl     ecx, 10 ; 1024 dwords per page
00004247 BE00001000                     mov     esi, MEM_ALLOC_TBL
                                    cfm0:
0000424C AD                             lodsd
0000424D 51                             push    ecx
0000424E B920000000                     mov     ecx, 32
                                    cfm1:
00004253 D1E8                           shr     eax, 1
00004255 7301                           jnc     short cfm2
00004257 42                             inc     edx
                                    cfm2:
00004258 E2F9                           loop    cfm1
0000425A 59                             pop     ecx
0000425B E2EF                           loop    cfm0
0000425D C3                             retn

                                    %include 'diskio.s'  ; 07/03/2015
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.8 - diskio.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 23/06/2024 (Previous: 02/12/2023 - Kernel v2.0.7)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Turkish Rational DOS
                                <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                                <1> ;
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; diskio.inc (22/08/2015)
                                <1> ;
                                <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                                <1> ; ****************************************************************************
                                <1> ; Ref: Retro UNIX 386 v1.1 (Kernel v0.2.1.5) 'diskio' modification: 12/07/2022
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - DISKIO.INC
                                <1> ; Last Modification: 22/08/2015
                                <1> ;   (Initialized Disk Parameters Data is in 'DISKDATA.INC')
                                <1> ;   (Uninitialized Disk Parameters Data is in 'DISKBSS.INC')
                                <1>
                                <1> ; DISK I/O SYSTEM - Erdogan Tan (Retro UNIX 386 v1 project)
                                <1>
                                <1> ; ///////// DISK I/O SYSTEM ///////////////
                                <1>
                                <1> ; 11/04/2021
                                <1> ;; 06/02/2015
                                <1> ;diskette_io:
                                <1>     ;clc ; 20/07/2020
                                <1>     ;pushfd
                                <1>     ;push   cs
                                <1>     ;;call  DISKETTE_IO_1
                                <1>     ;;retn
                                <1>
                                <1> ;;;;;; DISKETTE I/O ;;;;;;;;;;;;;;;;;;;; 06/02/2015 ;;;
                                <1> ;////////////////////////////////////////////////////
                                <1>
                                <1> ; DISKETTE I/O - Erdogan Tan (Retro UNIX 386 v1 project)
                                <1> ; 20/02/2015
                                <1> ; 06/02/2015 (unix386.s)
                                <1> ; 16/12/2014 - 02/01/2015 (dsectrm2.s)
                                <1> ;
                                <1> ; Code (DELAY) modifications - AWARD BIOS 1999 (ADISK.EQU, COMMON.MAC)
                                <1> ;
                                <1> ; ADISK.EQU
                                <1>
                                <1> ;----- Wait control constants
                                <1>
                                <1> ;amount of time to wait while RESET is active.
                                <1>
                                <1> WAITCPU_RESET_ON   EQU     21              ;Reset on must last at least 14us
                                <1>                                            ;at 250 KBS xfer rate.
                                <1>                                            ;see INTEL MCS, 1985, pg. 5-456
                                <1>
                                <1> WAITCPU_FOR_STATUS EQU     100             ;allow 30 microseconds for
                                <1>                                            ;status register to become valid
                                <1>                                            ;before re-reading.
                                <1>
                                <1> ;After sending a byte to NEC, status register may remain
                                <1> ;incorrectly set for 24 us.
                                <1>
                                <1> WAITCPU_RQM_LOW            EQU     24              ;number of loops to check for
                                <1>                                            ;RQM low.
                                <1>
                                <1> ; COMMON.MAC
                                <1> ;
                                <1> ;   Timing macros
```

```
<1> ;
<1>
<1> %macro              SIODELAY 0                      ; SHORT IODELAY
<1>             jmp short $+2
<1> %endmacro
<1>
<1> %macro              IODELAY  0                      ; NORMAL IODELAY
<1>             jmp short $+2
<1>             jmp short $+2
<1> %endmacro
<1>
<1> %macro              NEWIODELAY 0
<1>             out 0EBh, al
<1> %endmacro
<1>
<1> ; (According to) AWARD BIOS 1999 - ATORGS.ASM (dw -> equ, db -> equ)
<1> ;;; WAIT_FOR_MEM
<1> ;WAIT_FDU_INT_LO   equ     017798          ; 2.5 secs in 30 micro units.
<1> ;WAIT_FDU_INT_HI   equ     1
<1> WAIT_FDU_INT_LH            equ     83334           ; 27/02/2015 (2.5 seconds waiting)
<1> ;;; WAIT_FOR_PORT
<1> ;WAIT_FDU_SEND_LO  equ     16667           ; .5 secons in 30 us units.
<1> ;WAIT_FDU_SEND_HI  equ     0
<1> WAIT_FDU_SEND_LH   equ     16667           ; 27/02/2015
<1> ;Time to wait while waiting for each byte of NEC results = .5
<1> ;seconds.  .5 seconds = 500,000 micros.  500,000/30 = 16,667.
<1> ;WAIT_FDU_RESULTS_LO       equ     16667           ; .5 seconds in 30 micro units.
<1> ;WAIT_FDU_RESULTS_HI       equ     0
<1> WAIT_FDU_RESULTS_LH        equ     16667  ; 27/02/2015
<1> ;;; WAIT_REFRESH
<1> ;amount of time to wait for head settle, per unit in parameter
<1> ;table = 1 ms.
<1> WAIT_FDU_HEAD_SETTLE       equ     33              ; 1 ms in 30 micro units.
<1>
<1>
<1> ; /////////////// DISKETTE I/O ///////////////
<1>
<1> ; 11/12/2014 (copy from IBM PC-XT Model 286 BIOS - POSTEQU.INC)
<1>
<1> ;---------------------------------------
<1> ;   EQUATES USED BY POST AND BIOS :
<1> ;---------------------------------------
<1>
<1> ;--------- 8042 KEYBOARD INTERFACE AND DIAGNOSTIC CONTROL REGISTERS ------------
<1> ;PORT_A            EQU     060H            ; 8042 KEYBOARD SCAN CODE/CONTROL PORT
<1> ;PORT_B            EQU     061H            ; PORT B READ/WRITE DIAGNOSTIC REGISTER
<1> ;REFRESH_BIT       EQU     00010000B       ; REFRESH TEST BIT
<1>
<1> ;---------------------------------------
<1> ;   CMOS EQUATES FOR THIS SYSTEM  :
<1> ;-------------------------------------------------------------------------------
<1> ;CMOS_PORT  EQU     070H            ; I/O ADDRESS OF CMOS ADDRESS PORT
<1> ;CMOS_DATA  EQU     071H            ; I/O ADDRESS OF CMOS DATA PORT
<1> ;NMI        EQU     10000000B       ; DISABLE NMI INTERRUPTS MASK -
<1>                                     ;  HIGH BIT OF CMOS LOCATION ADDRESS
<1>
<1> ;---------- CMOS TABLE LOCATION ADDRESS'S ## ----------------------------------
<1> CMOS_DISKETTE      EQU     010H            ; DISKETTE DRIVE TYPE BYTE          ;
<1> ;           EQU     011H            ; - RESERVED                        ;C
<1> CMOS_DISK   EQU     012H            ; FIXED DISK TYPE BYTE              ;H
<1> ;           EQU     013H            ; - RESERVED                        ;E
<1> CMOS_EQUIP  EQU     014H            ; EQUIPMENT WORD LOW BYTE           ;C
<1>
<1> ;---------- DISKETTE EQUATES --------------------------------------------------
<1> INT_FLAG    EQU     10000000B       ; INTERRUPT OCCURRENCE FLAG
<1> DSK_CHG     EQU     10000000B       ; DISKETTE CHANGE FLAG MASK BIT
<1> DETERMINED  EQU     00010000B       ; SET STATE DETERMINED IN STATE BITS
<1> HOME        EQU     00010000B       ; TRACK 0 MASK
<1> SENSE_DRV_ST       EQU     00000100B       ; SENSE DRIVE STATUS COMMAND
<1> TRK_SLAP    EQU     030H            ; CRASH STOP (48 TPI DRIVES)
<1> QUIET_SEEK  EQU     00AH            ; SEEK TO TRACK 10
<1> ;MAX_DRV    EQU     2               ; MAX NUMBER OF DRIVES
<1> HD12_SETTLE EQU     15              ; 1.2 M HEAD SETTLE TIME
<1> HD320_SETTLE       EQU     20              ; 320 K HEAD SETTLE TIME
<1> MOTOR_WAIT  EQU     37              ; 2 SECONDS OF COUNTS FOR MOTOR TURN OFF
<1>
<1> ;---------- DISKETTE ERRORS ---------------------------------------------------
<1> ;TIME_OUT   EQU     080H            ; ATTACHMENT FAILED TO RESPOND
<1> ;BAD_SEEK   EQU     040H            ; SEEK OPERATION FAILED
<1> BAD_NEC     EQU     020H            ; DISKETTE CONTROLLER HAS FAILED
<1> BAD_CRC     EQU     010H            ; BAD CRC ON DISKETTE READ
<1> MED_NOT_FND EQU     00CH            ; MEDIA TYPE NOT FOUND
<1> DMA_BOUNDARY       EQU     009H            ; ATTEMPT TO DMA ACROSS 64K BOUNDARY
<1> BAD_DMA     EQU     008H            ; DMA OVERRUN ON OPERATION
<1> MEDIA_CHANGE       EQU     006H            ; MEDIA REMOVED ON DUAL ATTACH CARD
<1> RECORD_NOT_FND     EQU     004H            ; REQUESTED SECTOR NOT FOUND
<1> WRITE_PROTECT      EQU     003H            ; WRITE ATTEMPTED ON WRITE PROTECT DISK
<1> BAD_ADDR_MARK      EQU     002H            ; ADDRESS MARK NOT FOUND
<1> BAD_CMD     EQU     001H            ; BAD COMMAND PASSED TO DISKETTE I/O
<1>
<1> ;---------- DISK CHANGE LINE EQUATES ------------------------------------------
<1> NOCHGLN     EQU     001H            ; NO DISK CHANGE LINE AVAILABLE
<1> CHGLN               EQU     002H            ; DISK CHANGE LINE AVAILABLE
<1>
<1> ;---------- MEDIA/DRIVE STATE INDICATORS --------------------------------------
<1> TRK_CAPA    EQU     00000001B       ; 80 TRACK CAPABILITY
<1> FMT_CAPA    EQU     00000010B       ; MULTIPLE FORMAT CAPABILITY (1.2M)
<1> DRV_DET     EQU     00000100B       ; DRIVE DETERMINED
<1> MED_DET     EQU     00010000B       ; MEDIA DETERMINED BIT
<1> DBL_STEP    EQU     00100000B       ; DOUBLE STEP BIT
<1> RATE_MSK    EQU     11000000B       ; MASK FOR CLEARING ALL BUT RATE
<1> RATE_500    EQU     00000000B       ; 500 KBS DATA RATE
<1> RATE_300    EQU     01000000B       ; 300 KBS DATA RATE
<1> RATE_250    EQU     10000000B       ; 250 KBS DATA RATE
<1> STRT_MSK    EQU     00001100B       ; OPERATION START RATE MASK
<1> SEND_MSK    EQU     11000000B       ; MASK FOR SEND RATE BITS
<1>
<1> ;---------- MEDIA/DRIVE STATE INDICATORS COMPATIBILITY -------------------------
<1> M3D3U               EQU     00000000B       ; 360 MEDIA/DRIVE NOT ESTABLISHED
<1> M3D1U               EQU     00000001B       ; 360 MEDIA,1.2DRIVE NOT ESTABLISHED
<1> M1D1U               EQU     00000010B       ; 1.2 MEDIA/DRIVE NOT ESTABLISHED
<1> MED_UNK     EQU     00000111B       ; NONE OF THE ABOVE
<1>
<1> ;---------- INTERRUPT EQUATES -------------------------------------------------
<1> ;EOI        EQU     020H            ; END OF INTERRUPT COMMAND TO 8259
<1> ;INTA00             EQU     020H            ; 8259 PORT
<1> INTA01              EQU     021H            ; 8259 PORT
<1> INTB00              EQU     0A0H            ; 2ND 8259
<1> INTB01              EQU     0A1H            ;
<1>
<1> ;-------------------------------------------------------------------------------
<1> DMA08               EQU     008H            ; DMA STATUS REGISTER PORT ADDRESS
<1> DMA         EQU     000H            ; DMA CH.0 ADDRESS REGISTER PORT ADDRESS
<1> DMA18               EQU     0D0H            ; 2ND DMA STATUS PORT ADDRESS
```

```
<1> DMA1        EQU     0C0H            ; 2ND DMA CH.0 ADDRESS REGISTER ADDRESS
<1> ;-------------------------------------------------------------------------------
<1> ;TIMER              EQU     040H            ; 8254 TIMER - BASE ADDRESS
<1>
<1> ;-------------------------------------------------------------------------------
<1> DMA_PAGE    EQU     081H            ; START OF DMA PAGE REGISTERS
<1>
<1> ; 06/02/2015 (unix386.s, protected mode modifications)
<1> ; (unix386.s <-- dsectrm2.s)
<1> ; 11/12/2014 (copy from IBM PC-XT Model 286 BIOS - DSEG.INC)
<1>
<1> ; 27/05/2016 - TRDOS 386 (TRDOS v2.0)
<1> ; 10/12/2014
<1> ;
<1> ;int40h:
<1> ;   pushf
<1> ;   push    cs
<1> ;   ;cli
<1> ;   call    DISKETTE_IO_1
<1> ;   retn
<1>
<1> ; DSKETTE ----- 04/21/86 DISKETTE BIOS
<1> ; (IBM PC XT Model 286 System BIOS Source Code, 04-21-86)
<1> ;
<1>
<1> ;-- INT13H ----------------------------------------------------------------
<1> ; DISKETTE I/O
<1> ;   THIS INTERFACE PROVIDES ACCESS TO THE 5 1/4 INCH 360 KB,
<1> ;   1.2 MB, 720 KB AND 1.44 MB DISKETTE DRIVES.
<1> ; INPUT
<1> ;   (AH) =  00H RESET DISKETTE SYSTEM
<1> ;           HARD RESET TO NEC, PREPARE COMMAND, RECALIBRATE REQUIRED
<1> ;           ON ALL DRIVES
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH)= 01H  READ THE STATUS OF THE SYSTEM INTO (AH)
<1> ;           @DISKETTE_STATUS FROM LAST OPERATION IS USED
<1> ;-------------------------------------------------------------------------------
<1> ;   REGISTERS FOR READ/WRITE/VERIFY/FORMAT
<1> ;   (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHECKED)
<1> ;   (DH) - HEAD NUMBER (0-1 ALLOWED, NOT VALUE CHECKED)
<1> ;   (CH) - TRACK NUMBER (NOT VALUE CHECKED)
<1> ;           MEDIA   DRIVE   TRACK NUMBER
<1> ;           320/360 320/360     0-39
<1> ;           320/360 1.2M        0-39
<1> ;           1.2M    1.2M        0-79
<1> ;           720K    720K        0-79
<1> ;           1.44M   1.44M       0-79
<1> ;   (CL) -  SECTOR NUMBER (NOT VALUE CHECKED, NOT USED FOR FORMAT)
<1> ;           MEDIA   DRIVE   SECTOR NUMBER
<1> ;           320/360 320/360      1-8/9
<1> ;           320/360 1.2M         1-8/9
<1> ;           1.2M    1.2M         1-15
<1> ;           720K    720K         1-9
<1> ;           1.44M   1.44M        1-18
<1> ;   (AL)    NUMBER OF SECTORS (NOT VALUE CHECKED)
<1> ;           MEDIA   DRIVE   MAX NUMBER OF SECTORS
<1> ;           320/360 320/360      8/9
<1> ;           320/360 1.2M         8/9
<1> ;           1.2M    1.2M         15
<1> ;           720K    720K          9
<1> ;           1.44M   1.44M        18
<1> ;
<1> ;   (ES:BX) - ADDRESS OF BUFFER (NOT REQUIRED FOR VERIFY)
<1> ;
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH)= 02H  READ THE DESIRED SECTORS INTO MEMORY
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH)= 03H  WRITE THE DESIRED SECTORS FROM MEMORY
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH)= 04H  VERIFY THE DESIRED SECTORS
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH)= 05H  FORMAT THE DESIRED TRACK
<1> ;           (ES,BX) MUST POINT TO THE COLLECTION OF DESIRED ADDRESS FIELDS
<1> ;           FOR THE TRACK. EACH FIELD IS COMPOSED OF 4 BYTES, (C,H,R,N),
<1> ;           WHERE C = TRACK NUMBER, H=HEAD NUMBER, R = SECTOR NUMBER,
<1> ;           N= NUMBER OF BYTES PER SECTOR (00=128,01=256,02=512,03=1024),
<1> ;           THERE MUST BE ONE ENTRY FOR EVERY SECTOR ON THE TRACK.
<1> ;           THIS INFORMATION IS USED TO FIND THE REQUESTED SECTOR DURING
<1> ;           READ/WRITE ACCESS.
<1> ;           PRIOR TO FORMATTING A DISKETTE, IF THERE EXISTS MORE THAN
<1> ;           ONE SUPPORTED MEDIA FORMAT TYPE WITHIN THE DRIVE IN QUESTION,
<1> ;           THEN "SET DASD TYPE" (INT 13H, AH = 17H) OR 'SET MEDIA TYPE'
<1> ;           (INT 13H, AH =  18H) MUST BE CALLED TO SET THE DISKETTE TYPE
<1> ;           THAT IS TO BE FORMATTED. IF "SET DASD TYPE" OR "SET MEDIA TYPE"
<1> ;           IS NOT CALLED, THE FORMAT ROUTINE WILL ASSUME THE
<1> ;           MEDIA FORMAT TO BE THE MAXIMUM CAPACITY OF THE DRIVE.
<1> ;
<1> ;           THESE PARAMETERS OF DISK BASE MUST BE CHANGED IN ORDER TO
<1> ;           FORMAT THE FOLLOWING MEDIAS:
<1> ;           -------------------------------------------
<1> ;           : MEDIA  :     DRIVE      : PARM 1 : PARM 2 :
<1> ;           -------------------------------------------
<1> ;           : 320K   : 320K/360K/1.2M :  50H   :   8    :
<1> ;           : 360K   : 320K/360K/1.2M :  50H   :   9    :
<1> ;           : 1.2M   : 1.2M           :  54H   :  15    :
<1> ;           : 720K   : 720K/1.44M     :  50H   :   9    :
<1> ;           : 1.44M  : 1.44M          :  6CH   :  18    :
<1> ;           -------------------------------------------
<1> ;           NOTES: - PARM 1 = GAP LENGTH FOR FORMAT
<1> ;                  - PARM 2 = EOT (LAST SECTOR ON TRACK)
<1> ;                  - DISK BASE IS POINTED BY DISK POINTER LOCATED
<1> ;                    AT ABSOLUTE ADDRESS 0:78.
<1> ;                  - WHEN FORMAT OPERATIONS ARE COMPLETE, THE PARAMETERS
<1> ;                    SHOULD BE RESTORED TO THEIR RESPECTIVE INITIAL VALUES.
<1> ;-------------------------------------------------------------------------------
<1> ;   (AH) = 08H READ DRIVE PARAMETERS
<1> ;   REGISTERS
<1> ;     INPUT
<1> ;       (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHECKED)
<1> ;        ** 27/05/2016 - TRDOS 386 (TRDOS v2.0) **
<1> ;            ** EBX = Buffer address for floppy disk parameters table **
<1> ;     OUTPUT
<1> ;       (ES:DI) POINTS TO DRIVE PARAMETER TABLE
<1> ;       *** TRDOS 386 note: floppy disk parameter table (16 bytes)
<1> ;       will be returned to user in EBX, buffer address *** 27/05/2016 ***
<1> ;
<1> ;       (CH) - LOW ORDER 8 OF 10 BITS MAXIMUM NUMBER OF TRACKS
<1> ;       (CL) - BITS 7 & 6 - HIGH ORDER TWO BITS OF MAXIMUM TRACKS
<1> ;              BITS 5 THRU 0 - MAXIMUM SECTORS PER TRACK
<1> ;       (DH) - MAXIMUM HEAD NUMBER
<1> ;       (DL) - NUMBER OF DISKETTE DRIVES INSTALLED
<1> ;       (BH) - 0
<1> ;       (BL) - BITS 7 THRU 4 - 0
<1> ;              BITS 3 THRU 0 - VALID DRIVE TYPE VALUE IN CMOS
```

```
                                <1> ;       (AX) - 0
                                <1> ;    UNDER THE FOLLOWING CIRCUMSTANCES:
                                <1> ;       (1) THE DRIVE NUMBER IS INVALID,
                                <1> ;       (2) THE DRIVE TYPE IS UNKNOWN AND CMOS IS NOT PRESENT,
                                <1> ;       (3) THE DRIVE TYPE IS UNKNOWN AND CMOS IS BAD,
                                <1> ;       (4) OR THE DRIVE TYPE IS UNKNOWN AND THE CMOS DRIVE TYPE IS INVALID
                                <1> ;       THEN ES,AX,BX,CX,DH,DI=0 ; DL=NUMBER OF DRIVES.
                                <1> ;       IF NO DRIVES ARE PRESENT THEN: ES,AX,BX,CX,DX,DI=0.
                                <1> ;       @DISKETTE_STATUS = 0 AND CY IS RESET.
                                <1> ;-------------------------------------------------------------------------
                                <1> ;   (AH)= 15H  READ DASD TYPE
                                <1> ;   OUTPUT REGISTERS
                                <1> ;   (AH) - ON RETURN IF CARRY FLAG NOT SET, OTHERWISE ERROR
                                <1> ;          00 - DRIVE NOT PRESENT
                                <1> ;          01 - DISKETTE, NO CHANGE LINE AVAILABLE
                                <1> ;          02 - DISKETTE, CHANGE LINE AVAILABLE
                                <1> ;          03 - RESERVED (FIXED DISK)
                                <1> ;   (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHECKED)
                                <1> ;-------------------------------------------------------------------------
                                <1> ;   (AH)= 16H  DISK CHANGE LINE STATUS
                                <1> ;   OUTPUT REGISTERS
                                <1> ;   (AH) - 00 - DISK CHANGE LINE NOT ACTIVE
                                <1> ;          06 - DISK CHANGE LINE ACTIVE & CARRY BIT ON
                                <1> ;   (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHECKED)
                                <1> ;-------------------------------------------------------------------------
                                <1> ;   (AH)= 17H  SET DASD TYPE FOR FORMAT
                                <1> ;   INPUT REGISTERS
                                <1> ;   (AL) -  00 - NOT USED
                                <1> ;          01 - DISKETTE 320/360K IN 360K DRIVE
                                <1> ;          02 - DISKETTE 360K IN 1.2M DRIVE
                                <1> ;          03 - DISKETTE 1.2M IN 1.2M DRIVE
                                <1> ;          04 - DISKETTE 720K IN 720K DRIVE
                                <1> ;   (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHECKED:
                                <1> ;          (DO NOT USE WHEN DISKETTE ATTACH CARD USED)
                                <1> ;-------------------------------------------------------------------------
                                <1> ;   (AH)= 18H  SET MEDIA TYPE FOR FORMAT
                                <1> ;   INPUT REGISTERS
                                <1> ;   (CH) - LOW ORDER 8 OF 10 BITS MAXIMUM TRACKS
                                <1> ;   (CL) - BITS 7 & 6 - HIGH ORDER TWO BITS OF MAXIMUM TRACKS
                                <1> ;          BITS 5 THRU 0 - MAXIMUM SECTORS PER TRACK
                                <1> ;   (DL) - DRIVE NUMBER (0-1 ALLOWED, VALUE CHACKED)
                                <1> ;   OUTPUT REGISTERS:
                                <1> ;   (ES:DI) - POINTER TO DRIVE PARAMETERS TABLE FOR THIS MEDIA TYPE,
                                <1> ;             UNCHANGED IF (AH) IS NON-ZERO
                                <1> ;   (AH) - 00H, CY = 0, TRACK AND SECTORS/TRACK COMBINATION IS SUPPORTED
                                <1> ;        - 01H, CY = 1, FUNCTION IS NOT AVAILABLE
                                <1> ;        - 0CH, CY = 1, TRACK AND SECTORS/TRACK COMBINATION IS NOT SUPPORTED
                                <1> ;        - 80H, CY = 1, TIME OUT (DISKETTE NOT PRESENT)
                                <1> ;-------------------------------------------------------------------------
                                <1> ;   DISK CHANGE STATUS IS ONLY CHECKED WHEN A MEDIA SPECIFIED IS OTHER
                                <1> ;   THAN 360 KB DRIVE. IF THE DISK CHANGE LINE IS FOUND TO BE
                                <1> ;   ACTIVE THE FOLLOWING ACTIONS TAKE PLACE:
                                <1> ;          ATTEMPT TO RESET DISK CHANGE LINE TO INACTIVE STATE.
                                <1> ;          IF ATTEMPT SUCCEEDS SET DASD TYPE FOR FORMAT AND RETURN DISK
                                <1> ;          CHANGE ERROR CODE
                                <1> ;          IF ATTEMPT FAILS RETURN TIMEOUT ERROR CODE AND SET DASD TYPE
                                <1> ;          TO A PREDETERMINED STATE INDICATING MEDIA TYPE UNKNOWN.
                                <1> ;   IF THE DISK CHANGE LINE IN INACTIVE PERFORM SET DASD TYPE FOR FORMAT.
                                <1> ;
                                <1> ; DATA VARIABLE -- @DISK_POINTER
                                <1> ;   DOUBLE WORD POINTER TO THE CURRENT SET OF DISKETTE PARAMETERS
                                <1> ;-------------------------------------------------------------------------
                                <1> ; OUTPUT FOR ALL FUNCTIONS
                                <1> ;   AH = STATUS OF OPERATION
                                <1> ;          STATUS BITS ARE DEFINED IN THE EQUATES FOR @DISKETTE_STATUS
                                <1> ;          VARIABLE IN THE DATA SEGMENT OF THIS MODULE
                                <1> ;   CY = 0  SUCCESSFUL OPERATION (AH=0 ON RETURN, EXCEPT FOR READ DASD
                                <1> ;          TYPE AH=(15)).
                                <1> ;   CY = 1  FAILED OPERATION (AH HAS ERROR REASON)
                                <1> ;   FOR READ/WRITE/VERIFY
                                <1> ;          DS,BX,DX,CX PRESERVED
                                <1> ;   NOTE: IF AN ERROR IS REPORTED BY THE DISKETTE CODE, THE APPROPRIATE
                                <1> ;          ACTION IS TO RESET THE DISKETTE, THEN RETRY THE OPERATION.
                                <1> ;          ON READ ACCESSES, NO MOTOR START DELAY IS TAKEN, SO THAT
                                <1> ;          THREE RETRIES ARE REQUIRED ON READS TO ENSURE THAT THE
                                <1> ;          PROBLEM IS NOT DUE TO MOTOR START-UP.
                                <1> ;-------------------------------------------------------------------------
                                <1> ;
                                <1> ; DISKETTE STATE MACHINE - ABSOLUTE ADDRESS 40:90 (DRIVE A) & 91 (DRIVE B)
                                <1> ;
                                <1> ;   ---------------------------------------------------------------
                                <1> ;   |       |       |       |       |       |       |       |       |
                                <1> ;   |   7   |   6   |   5   |   4   |   3   |   2   |   1   |   0   |
                                <1> ;   |       |       |       |       |       |       |       |       |
                                <1> ;   ---------------------------------------------------------------
                                <1> ;   |       |       |       |       |       |       |
                                <1> ;   |       |       |       |       |       -----------------
                                <1> ;   |       |       |       |       |             |
                                <1> ;   |       |       |       |       |    RESERVED         |
                                <1> ;   |       |       |       |       |                 PRESENT STATE
                                <1> ;   |       |       |       |       |       000: 360K IN 360K DRIVE UNESTABLISHED
                                <1> ;   |       |       |       |       |       001: 360K IN 1.2M DRIVE UNESTABLISHED
                                <1> ;   |       |       |       |       |       010: 1.2M IN 1.2M DRIVE UNESTABLISHED
                                <1> ;   |       |       |       |       |       011: 360K IN 360K DRIVE ESTABLISHED
                                <1> ;   |       |       |       |       |       100: 360K IN 1.2M DRIVE ESTABLISHED
                                <1> ;   |       |       |       |       |       101: 1.2M IN 1.2M DRIVE ESTABLISHED
                                <1> ;   |       |       |       |       |       110: RESERVED
                                <1> ;   |       |       |       |       |       111: NONE OF THE ABOVE
                                <1> ;   |       |       |       |
                                <1> ;   |       |       |       ------>MEDIA/DRIVE ESTABLISHED
                                <1> ;   |       |       |
                                <1> ;   |       |       --------------->       DOUBLE STEPPING REQUIRED (360K IN 1.2M
                                <1> ;   |       |                       DRIVE)
                                <1> ;   |       |
                                <1> ;   ------------------------------>      DATA TRANSFER RATE FOR THIS DRIVE:
                                <1> ;
                                <1> ;                                        00: 500 KBS
                                <1> ;                                        01: 300 KBS
                                <1> ;                                        10: 250 KBS
                                <1> ;                                        11: RESERVED
                                <1> ;
                                <1> ;
                                <1> ;-------------------------------------------------------------------------
                                <1> ; STATE OPERATION STARTED - ABSOLUTE ADDRESS 40:92 (DRIVE A) & 93 (DRIVE B)
                                <1> ;-------------------------------------------------------------------------
                                <1> ; PRESENT CYLINDER NUMBER - ABSOLUTE ADDRESS 40:94 (DRIVE A) & 95 (DRIVE B)
                                <1> ;-------------------------------------------------------------------------
                                <1>
                                <1> struc MD
00000000 ??                     <1>     .SPEC1        resb   1      ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
00000001 ??                     <1>     .SPEC2        resb   1      ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
00000002 ??                     <1>     .OFF_TIM      resb   1      ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
00000003 ??                     <1>     .BYT_SEC      resb   1      ; 512 BYTES/SECTOR
00000004 ??                     <1>     .SEC_TRK      resb   1      ; EOT (LAST SECTOR ON TRACK)
```

```
00000005 ??                 <1>     .GAP           resb   1       ; GAP LENGTH
00000006 ??                 <1>     .DTL           resb   1       ; DTL
00000007 ??                 <1>     .GAP3          resb   1       ; GAP LENGTH FOR FORMAT
00000008 ??                 <1>     .FIL_BYT       resb   1       ; FILL BYTE FOR FORMAT
00000009 ??                 <1>     .HD_TIM        resb   1       ; HEAD SETTLE TIME (MILLISECONDS)
0000000A ??                 <1>     .STR_TIM       resb   1       ; MOTOR START TIME (1/8 SECONDS)
0000000B ??                 <1>     .MAX_TRK       resb   1       ; MAX. TRACK NUMBER
0000000C ??                 <1>     .RATE          resb   1       ; DATA TRANSFER RATE
                            <1> endstruc
                            <1>
                            <1> BIT7OFF     EQU     7FH
                            <1> BIT7ON      EQU     80H
                            <1>
                            <1> ; 30/08/2020 - TRDOS 386 v2
                            <1>
                            <1> ;;int13h: ; 16/02/2015
                            <1> ;; 16/02/2015 - 21/02/2015
                            <1> ; 17/07/2022 - TRDOS 386 v2.0.5
                            <1> ;int40h:
                            <1> ;; 11/04/2021
                            <1> ;diskette_io:
                            <1> ;   clc ; 20/07/2020
                            <1> ;   pushfd
                            <1> ;   push    cs
                            <1> ;   call    DISKETTE_IO_1
                            <1> ;   retn
                            <1>
                            <1>     ; 09/08/2022
                            <1>     ; 06/08/2022
                            <1>     ; 17/07/2022 - TRDOS 386 v2.0.5
                            <1>     ; (jump from DISK_IO)
                            <1> DISKETTE_IO_1:
                            <1>     ;sti ; 17/07/2022              ; INTERRUPTS BACK ON
                            <1>
                            <1>     ;push   ebp                    ; USER REGISTER
                            <1>     ;push   edi                    ; USER REGISTER
                            <1>     ;push   edx                    ; HEAD #, DRIVE # OR USER REGISTER
                            <1>     ;push   ebx                    ; BUFFER OFFSET PARAMETER OR REGISTER
                            <1>     ;push   ecx                    ; TRACK #-SECTOR # OR USER REGISTER
                            <1>     ;mov    ebp, esp               ; BP     => PARAMETER LIST DEP. ON AH
                            <1>                                    ; [BP]   = SECTOR #
                            <1>                                    ; [BP+1] = TRACK #
                            <1>                                    ; [BP+2] = BUFFER OFFSET
                            <1>                                    ; FOR RETURN OF DRIVE PARAMETERS:
                            <1>                                    ; CL/[BP] = BITS 7&6 HI BITS OF MAX CYL
                            <1>                                    ;           BITS 0-5 MAX SECTORS/TRACK
                            <1>                                    ; CH/[BP+1] = LOW 8 BITS OF MAX CYL.
                            <1>                                    ; BL/[BP+2] = BITS 7-4 = 0
                            <1>                                    ;             BITS 3-0 = VALID CMOS TYPE
                            <1>                                    ; BH/[BP+3] = 0
                            <1>                                    ; DL/[BP+4] = # DRIVES INSTALLED
                            <1>                                    ; DH/[BP+5] = MAX HEAD #
                            <1>                                    ; DI/[BP+6] = OFFSET TO DISK BASE
                            <1>     ;push   es ; 06/02/2015
                            <1>     ;push   ds                     ; BUFFER SEGMENT PARM OR USER REGISTER
                            <1>     ;push   esi                    ; USER REGISTERS
                            <1>     ;;call  DDS                    ; SEGMENT OF BIOS DATA AREA TO DS
                            <1>     ;;mov   cx, cs
                            <1>     ;;mov   ds, cx
                            <1>     ;mov    cx, KDATA
                            <1>     ;mov    ds, cx
                            <1>     ;mov    es, cx
                            <1>
                            <1>     ; 17/07/2022
                            <1>     ; Registers are also on stack
                            <1>     ; (with same contents)
                            <1>     ; in following order:
                            <1>     ;
                            <1>     ;    ebx = esp+20
                            <1>     ;    ecx = esp+16
                            <1>     ;    edx = esp+12
                            <1>     ;    esi = esp+8
                            <1>     ;    edi = esp+4
                            <1>     ;
                            <1>     ; [esp] = caller's return address (from 'DISK_IO')
                            <1>     ;
                            <1>     ; cs = KCODE == KDATA
                            <1>     ; ds = es = ss = KDATA
                            <1>
                            <1>     ; 17/07/2022
0000425E 55                 <1>     push    ebp
0000425F 89DD               <1>     mov     ebp, ebx
                            <1>
                            <1>     ;cmp    ah, (FNC_TAE-FNC_TAB)/2       ; CHECK FOR > LARGEST FUNCTION
00004261 80FC19             <1>     cmp     ah, (FNC_TAE-FNC_TAB)/4       ; 18/02/2015
                            <1>     ;jb     short OK_FUNC          ; FUNCTION OK
                            <1>     ;mov    ah, 14h                ; REPLACE WITH KNOWN INVALID FUNCTION
                            <1>     ; 09/08/2022
00004264 730F               <1>     jnb     short INV_FUNC
                            <1> OK_FUNC:
00004266 80FC01             <1>     cmp     ah, 1                  ; RESET OR STATUS ?
00004269 760C               <1>     jbe     short OK_DRV           ; IF RESET OR STATUS DRIVE ALWAYS OK
0000426B 80FC08             <1>     cmp     ah, 8                  ; READ DRIVE PARMS ?
0000426E 7407               <1>     je      short OK_DRV           ; IF SO DRIVE CHECKED LATER
00004270 80FA01             <1>     cmp     dl, 1                  ; DRIVES 0 AND 1 OK
00004273 7602               <1>     jbe     short OK_DRV           ; IF 0 OR 1 THEN JUMP
                            <1> INV_FUNC:
00004275 B414               <1>     mov     ah, 14h                ; REPLACE WITH KNOWN INVALID FUNCTION
                            <1> OK_DRV:
                            <1>     ; 17/07/2022
                            <1>     ;xor    ecx, ecx
                            <1>     ;;mov   esi, ecx ; 08/02/2015
                            <1>     ;mov    edi, ecx ; 08/02/2015
                            <1>     ;mov    cl, ah                 ; CL = FUNCTION
                            <1>     ;;xor   ch, ch                 ; CX = FUNCTION
                            <1>     ;;shl   cl, 1                  ; FUNCTION TIMES 2
                            <1>     ;shl    cl, 2 ; 20/02/2015     ; FUNCTION TIMES 4 (for 32 bit offset)
                            <1>     ;mov    ebx, FNC_TAB           ; LOAD START OF FUNCTION TABLE
                            <1>     ;add    ebx, ecx               ; ADD OFFSET INTO TABLE => ROUTINE
                            <1>
                            <1>     ; 17/07/2022
00004277 29DB               <1>     sub     ebx, ebx
00004279 88E3               <1>     mov     bl, ah                 ; BL = FUNCTION
0000427B C0E302             <1>     shl     bl, 2 ; * 4
0000427E 81C3[9D420000]     <1>     add     ebx, FNC_TAB           ; [EBX] = FUNCTION ADDRESS
                            <1>
00004284 88F4               <1>     mov     ah, dh                 ; AX = HEAD #,# OF SECTORS OR DASD TYPE
                            <1>     ;xor    dh, dh                 ; DX = DRIVE #
                            <1>     ;mov    si, ax                 ; SI = HEAD #,# OF SECTORS OR DASD TYPE
                            <1>     ;mov    di, dx                 ; DI = DRIVE #
                            <1>
00004286 0FB7F0             <1>     movzx   esi, ax                ; ESI = HEAD #,# OF SECTORS OR DASD TYPE
00004289 0FB6FA             <1>     movzx   edi, dl                ; EDI = DRIVE #
                            <1>
```

```
                                <1>     ; CH = cylinder number (low 8 bit)
                                <1>     ; CL = sector number (and high 2 bits of cylinder number)
                                <1>
                                <1>     ; 06/08/2022
                                <1>     ; 11/12/2014
                                <1>         ;mov       [cfd], dl               ; current floppy drive (for 'GET_PARM')
                                <1>     ; 06/08/2022
                                <1>     ; EDI = (current) DRIVE #
                                <1>     ;
0000428C 8A25[38770100]         <1>     mov    ah, [DSKETTE_STATUS]  ; LOAD STATUS TO AH FOR STATUS FUNCTION
00004292 C605[38770100]00       <1>     mov    byte [DSKETTE_STATUS], 0 ; INITIALIZE FOR ALL OTHERS
                                <1>
                                <1> ;   THROUGHOUT THE DISKETTE BIOS, THE FOLLOWING INFORMATION IS CONTAINED IN
                                <1> ;   THE FOLLOWING MEMORY LOCATIONS AND REGISTERS. NOT ALL DISKETTE BIOS
                                <1> ;   FUNCTIONS REQUIRE ALL OF THESE PARAMETERS.
                                <1> ;
                                <1> ;           DI      : DRIVE #
                                <1> ;           SI-HI   : HEAD #
                                <1> ;           SI-LOW  : # OF SECTORS OR DASD TYPE FOR FORMAT
                                <1> ;           ES      : BUFFER SEGMENT
                                <1> ;           [BP]    : SECTOR #
                                <1> ;           [BP+1]  : TRACK #
                                <1> ;           [BP+2]  : BUFFER OFFSET
                                <1> ;
                                <1> ;   ACROSS CALLS TO SUBROUTINES THE CARRY FLAG (CY=1), WHERE INDICATED IN
                                <1> ;   SUBROUTINE PROLOGUES, REPRESENTS AN EXCEPTION RETURN (NORMALLY AN ERROR
                                <1> ;   CONDITION). IN MOST CASES, WHEN CY = 1, @DSKETTE_STATUS CONTAINS THE
                                <1> ;   SPECIFIC ERROR CODE.
                                <1>
                                <1>     ; 17/07/2022
                                <1>     ; EBX = pointer to function address
                                <1>     ; EBP = buffer address
                                <1>     ; EDI = drive number (0 or 1)
                                <1>     ; ESI = head number (byte 1) and sector count or disk type (byte 0)
                                <1>     ; CH = cylinder number (low 8 bit)
                                <1>     ; CL = sector number (and high 2 bits of cylinder number)
                                <1>
                                <1>                                     ; (AH) = @DSKETTE_STATUS
00004299 FF13                   <1>     call   dword [ebx]             ; CALL THE REQUESTED FUNCTION
                                <1>
                                <1>     ;pop   esi                     ; RESTORE ALL REGISTERS
                                <1>     ;pop   ds
                                <1>     ;pop   es      ; 06/02/2015
                                <1>     ;pop   ecx
                                <1>     ;pop   ebx
                                <1>     ;pop   edx
                                <1>     ;pop   edi
                                <1>     ;mov   ebp, esp
                                <1>     ;push  eax
                                <1>     ;pushfd
                                <1>     ;pop   eax
                                <1>     ;;mov  [bp+6], ax
                                <1>     ;mov   [ebp+12], eax  ; 18/02/2015, flags
                                <1>     ;pop   eax
                                <1>     ;pop   ebp
                                <1>     ;iretd
                                <1>
                                <1>     ; 17/07/2022
0000429B 5D                     <1>     pop    ebp
                                <1>
                                <1>     ; 17/07/2022
                                <1>     ; Stack order:
                                <1>     ;    ebx = esp+20
                                <1>     ;    ecx = esp+16
                                <1>     ;    edx = esp+12
                                <1>     ;    esi = esp+8
                                <1>     ;    edi = esp+4
                                <1>     ;
                                <1>     ; [esp] = caller's return address (from 'DISK_IO')
                                <1>
                                <1>     ; CF = disk i/o status (1 = error)
                                <1>     ; AH = error code (if > 0 and cf = 1)
                                <1>
                                <1>     ; 17/07/2022
0000429C C3                     <1>     retn   ; return to the caller of 'DISK_IO'
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1> ; DW --> dd (06/02/2015)
0000429D [01430000]             <1> FNC_TAB     dd     DSK_RESET             ; AH = 00H; RESET
000042A1 [6C430000]             <1>     dd     DSK_STATUS             ; AH = 01H; STATUS
000042A5 [7C430000]             <1>     dd     DSK_READ               ; AH = 02H; READ
000042A9 [89430000]             <1>     dd     DSK_WRITE              ; AH = 03H; WRITE
000042AD [9D440000]             <1>     dd     DSK_VERF               ; AH = 04H; VERIFY
000042B1 [AD440000]             <1>     dd     DSK_FORMAT             ; AH = 05H; FORMAT
000042B5 [2B450000]             <1>     dd     FNC_ERR                ; AH = 06H; INVALID
000042B9 [2B450000]             <1>     dd     FNC_ERR                ; AH = 07H; INVALID
000042BD [37450000]             <1>     dd     DSK_PARMS              ; AH = 08H; READ DRIVE PARAMETERS
000042C1 [2B450000]             <1>     dd     FNC_ERR                ; AH = 09H; INVALID
000042C5 [2B450000]             <1>     dd     FNC_ERR                ; AH = 0AH; INVALID
000042C9 [2B450000]             <1>     dd     FNC_ERR                ; AH = 0BH; INVALID
000042CD [2B450000]             <1>     dd     FNC_ERR                ; AH = 0CH; INVALID
000042D1 [2B450000]             <1>     dd     FNC_ERR                ; AH = 0DH; INVALID
000042D5 [2B450000]             <1>     dd     FNC_ERR                ; AH = 0EH; INVALID
000042D9 [2B450000]             <1>     dd     FNC_ERR                ; AH = 0FH; INVALID
000042DD [2B450000]             <1>     dd     FNC_ERR                ; AH = 10H; INVALID
000042E1 [2B450000]             <1>     dd     FNC_ERR                ; AH = 11H; INVALID
000042E5 [2B450000]             <1>     dd     FNC_ERR                ; AH = 12H; INVALID
000042E9 [2B450000]             <1>     dd     FNC_ERR                ; AH = 13H; INVALID
000042ED [2B450000]             <1>     dd     FNC_ERR                ; AH = 14H; INVALID
000042F1 [F4450000]             <1>     dd     DSK_TYPE               ; AH = 15H; READ DASD TYPE
000042F5 [14460000]             <1>     dd     DSK_CHANGE             ; AH = 16H; CHANGE STATUS
000042F9 [43460000]             <1>     dd     FORMAT_SET             ; AH = 17H; SET DASD TYPE
000042FD [B8460000]             <1>     dd     SET_MEDIA              ; AH = 18H; SET MEDIA TYPE
                                <1> FNC_TAE EQU     $                        ; END
                                <1>
                                <1>     ; 17/07/2022 - TRDOS 386 v2.0.5
                                <1> ;-----------------------------------------------------------------------------
                                <1> ; DISK_RESET       (AH = 00H)
                                <1> ;           RESET THE DISKETTE SYSTEM.
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;-----------------------------------------------------------------------------
                                <1> DSK_RESET:
                                <1>     ; 17/07/2022
00004301 66BAF203               <1>     mov    dx, 03F2h               ; ADAPTER CONTROL PORT
00004305 FA                     <1>     cli                            ; NO INTERRUPTS
00004306 A0[36770100]           <1>     mov    al, [MOTOR_STATUS]      ; GET DIGITAL OUTPUT REGISTER REFLECTION
0000430B 243F                   <1>     and    al, 00111111b           ; KEEP SELECTED AND MOTOR ON BITS
0000430D C0C004                 <1>     rol    al, 4                   ; MOTOR VALUE TO HIGH NIBBLE
                                <1>                                    ; DRIVE SELECT TO LOW NIBBLE
00004310 0C08                   <1>     or     al, 00001000b           ; TURN ON INTERRUPT ENABLE
00004312 EE                     <1>     out    dx, al                  ; RESET THE ADAPTER
00004313 C605[35770100]00       <1>     mov    byte [SEEK_STATUS], 0 ; SET RECALIBRATE REQUIRED ON ALL DRIVES
                                <1>     ;jmp   $+2                     ; WAIT FOR I/O
```

```
                            <1>     ;jmp    $+2                     ; WAIT FOR I/O (TO INSURE MINIMUM
                            <1>                                     ;       PULSE WIDTH)
                            <1>     ; 19/12/2014
                            <1>     NEWIODELAY
 83 0000431A E6EB           <2>  out 0EBh, al
                            <1>
                            <1>     ; 17/12/2014
                            <1>     ; AWARD BIOS 1999 - RESETDRIVES (ADISK.ASM)
0000431C B915000000         <1>     mov     ecx, WAITCPU_RESET_ON ; cx = 21 -- Min. 14 micro seconds !?
                            <1> wdw1:
                            <1>     NEWIODELAY   ; 27/02/2015
 83 00004321 E6EB           <2>  out 0EBh, al
00004323 E2FC               <1>     loop    wdw1
                            <1>     ;
00004325 0C04               <1>     or      al, 00000100b           ; TURN OFF RESET BIT
00004327 EE                 <1>     out     dx, al                  ; RESET THE ADAPTER
                            <1>     ; 16/12/2014
                            <1>     IODELAY
 78 00004328 EB00           <2>  jmp short $+2
 79 0000432A EB00           <2>  jmp short $+2
                            <1>     ;
                            <1>     ;sti                            ; ENABLE THE INTERRUPTS
0000432C E8DD0A0000         <1>     call    WAIT_INT                ; WAIT FOR THE INTERRUPT
00004331 7230               <1>     jc      short DR_ERR            ; IF ERROR, RETURN IT
                            <1>     ;mov    cx, 11000000b           ; CL = EXPECTED @NEC_STATUS
                            <1>     ; 17/07/2022
                            <1>     ;xor    ch, ch
00004333 B1C0               <1>     mov     cl, 11000000b
                            <1> NXT_DRV:
                            <1>     ;push   cx                      ; SAVE FOR CALL
                            <1>     ; 11/04/2021
00004335 51                 <1>     push    ecx
00004336 B8[62430000]       <1>     mov     eax, DR_POP_ERR         ; LOAD NEC_OUTPUT ERROR ADDRESS
0000433B 50                 <1>     push    eax                     ; "
0000433C B408               <1>     mov     ah, 08h                 ; SENSE INTERRUPT STATUS COMMAND
0000433E E8C3090000         <1>     call    NEC_OUTPUT
00004343 58                 <1>     pop     eax                     ; THROW AWAY ERROR RETURN
00004344 E8F40A0000         <1>     call    RESULTS                 ; READ IN THE RESULTS
                            <1>     ;pop    cx                      ; RESTORE AFTER CALL
                            <1>     ; 11/04/2021
00004349 59                 <1>     pop     ecx
0000434A 7217               <1>     jc      short DR_ERR            ; ERROR RETURN
0000434C 3A0D[39770100]     <1>     cmp     cl, [NEC_STATUS]        ; TEST FOR DRIVE READY TRANSITION
00004352 750F               <1>     jnz     short DR_ERR            ; EVERYTHING OK
00004354 FEC1               <1>     inc     cl                      ; NEXT EXPECTED @NEC_STATUS
00004356 80F9C3             <1>     cmp     cl, 11000011b           ; ALL POSSIBLE DRIVES CLEARED
00004359 76DA               <1>     jbe     short NXT_DRV           ; FALL THRU IF 11000100B OR >
                            <1>     ;
0000435B E800040000         <1>     call    SEND_SPEC               ; SEND SPECIFY COMMAND TO NEC
                            <1> RESBAC:
                            <1>     ; 06/08/2022
00004360 EB10               <1>     jmp     short SETUP_END_X
                            <1>     ;call   SETUP_END               ; VARIOUS CLEANUPS
                            <1>     ;;mov   bx, si                  ; GET SAVED AL TO BL
                            <1>     ;; 17/07/2022
                            <1>     ;mov    ebx, esi
                            <1>     ;mov    al, bl                  ; PUT BACK FOR RETURN
                            <1>     ;retn
                            <1>
                            <1> DR_POP_ERR:
                            <1>     ;pop    cx                      ; CLEAR STACK
                            <1>     ; 11/04/2021
00004362 59                 <1>     pop     ecx
                            <1> DR_ERR:
00004363 800D[38770100]20   <1>     or      byte [DSKETTE_STATUS], BAD_NEC ; SET ERROR CODE
                            <1>     ;jmp    short RESBAC            ; RETURN FROM RESET
                            <1>     ; 06/08/2022
0000436A EB06               <1>     jmp     short SETUP_END_X
                            <1>
                            <1> ;--------------------------------------------------------------------------
                            <1> ; DISK_STATUS      (AH = 01H)
                            <1> ;   DISKETTE STATUS.
                            <1> ;
                            <1> ; ON ENTRY: AH : STATUS OF PREVIOUS OPERATION
                            <1> ;
                            <1> ; ON EXIT:  AH, @DSKETTE_STATUS, CY REFLECT STATUS OF PREVIOUS OPERATION.
                            <1> ;--------------------------------------------------------------------------
                            <1> DSK_STATUS:
0000436C 8825[38770100]     <1>     mov     [DSKETTE_STATUS], ah    ; PUT BACK FOR SETUP END
                            <1> SETUP_END_X:       ; 06/08/2022
00004372 E809010000         <1>     call    SETUP_END               ; VARIOUS CLEANUPS
                            <1>     ;mov    bx, si                  ; GET SAVED AL TO BL
                            <1>     ;mov    al, bl                  ; PUT BACK FOR RETURN
                            <1>     ; 06/08/2022
00004377 89F3               <1>     mov     ebx, esi
00004379 88D8               <1>     mov     al, bl
0000437B C3                 <1>     retn
                            <1>
                            <1> ;--------------------------------------------------------------------------
                            <1> ; DISK_READ (AH = 02H)
                            <1> ;   DISKETTE READ.
                            <1> ;
                            <1> ; ON ENTRY: DI      : DRIVE #
                            <1> ;           SI-HI   : HEAD #
                            <1> ;           SI-LOW  : # OF SECTORS
                            <1> ;           ES      : BUFFER SEGMENT
                            <1> ;           [BP]    : SECTOR #
                            <1> ;           [BP+1]  : TRACK #
                            <1> ;           [BP+2]  : BUFFER OFFSET
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                            <1> ;--------------------------------------------------------------------------
                            <1>
                            <1> ; 06/02/2015, ES:BX -> EBX (unix386.s)
                            <1>
                            <1> DSK_READ:
0000437C 8025[36770100]7F   <1>     and     byte [MOTOR_STATUS], 01111111b ; INDICATE A READ OPERATION
00004383 66B846E6           <1>     mov     ax, 0E646h              ; AX = NEC COMMAND, DMA COMMAND
                            <1>     ;call   RD_WR_VF                ; COMMON READ/WRITE/VERIFY
                            <1>     ;retn
                            <1>     ; 06/08/2022
00004387 EB0B               <1>     jmp     short RD_WR_VF
                            <1>
                            <1> ;--------------------------------------------------------------------------
                            <1> ; DISK_WRITE       (AH = 03H)
                            <1> ;   DISKETTE WRITE.
                            <1> ;
                            <1> ; ON ENTRY: DI      : DRIVE #
                            <1> ;           SI-HI   : HEAD #
                            <1> ;           SI-LOW  : # OF SECTORS
                            <1> ;           ES      : BUFFER SEGMENT
                            <1> ;           [BP]    : SECTOR #
                            <1> ;           [BP+1]  : TRACK #
                            <1> ;           [BP+2]  : BUFFER OFFSET
```

```
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;-------------------------------------------------------------------------------
                                <1>
                                <1> ; 06/02/2015, ES:BX -> EBX (unix386.s)
                                <1>
                                <1> DSK_WRITE:
00004389 66B84AC5               <1>     mov     ax, 0C54Ah              ; AX = NEC COMMAND, DMA COMMAND
0000438D 800D[36770100]80       <1>     or      byte [MOTOR_STATUS], 10000000b ; INDICATE WRITE OPERATION
                                <1>     ;call   RD_WR_VF                ; COMMON READ/WRITE/VERIFY
                                <1>     ;retn
                                <1>     ; 06/08/2022
                                <1>     ;jmp    short RD_WR_VF
                                <1>
                                <1>     ; 09/08/2022
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ;-------------------------------------------------------------------------------
                                <1> ; RD_WR_VF
                                <1> ;   COMMON READ, WRITE AND VERIFY:
                                <1> ;   MAIN LOOP FOR STATE RETRIES.
                                <1> ;
                                <1> ; ON ENTRY: AH = READ/WRITE/VERIFY NEC PARAMETER
                                <1> ;           AL = READ/WRITE/VERIFY DMA PARAMETER
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;-------------------------------------------------------------------------------
                                <1> RD_WR_VF:
                                <1>     ; 02/12/2023
                                <1>     ; 11/08/2022
                                <1>     ; 09/08/2022
                                <1>     ; 07/08/2022
                                <1>     ; 06/08/2022
                                <1>     ; 11/04/2021 (32 bit push/pop, AX -> EAX)
                                <1>
00004394 50                     <1>     push    eax                     ; SAVE DMA, NEC PARAMETERS
                                <1>
                                <1>     ; 02/12/2023
                                <1>     ; (diskette change check for 'dir', 28 seconds)
00004395 A0[31770100]           <1>     mov     al, [TIMER_LOW+1]
0000439A D0E8                   <1>     shr     al, 1
0000439C A2[9A7E0100]           <1>     mov     [P_TIMER], al
                                <1>
000043A1 E803040000             <1>     call    XLAT_NEW                ; TRANSLATE STATE TO PRESENT ARCH.
000043A6 E86A040000             <1>     call    SETUP_STATE             ; INITIALIZE START AND END RATE
000043AB 58                     <1>     pop     eax                     ; RESTORE READ/WRITE/VERIFY
                                <1> DO_AGAIN:
000043AC 50                     <1>     push    eax                     ; SAVE READ/WRITE/VERIFY PARAMETER
000043AD E8F8040000             <1>     call    MED_CHANGE              ; MEDIA CHANGE AND RESET IF CHANGED
000043B2 58                     <1>     pop     eax                     ; RESTORE READ/WRITE/VERIFY
                                <1>     ;jc     short RWV_END           ; MEDIA CHANGE ERROR OR TIME-OUT
                                <1>     ; 07/08/2022
000043B3 7305                   <1>     jnc     short RWV
000043B5 E9BC000000             <1>     jmp     RWV_END
                                <1> RWV:
000043BA 50                     <1>     push    eax                     ; SAVE READ/WRITE/VERIFY PARAMETER
000043BB 8AB7[43770100]         <1>     mov     dh, [DSK_STATE+edi]     ; GET RATE STATE OF THIS DRIVE
000043C1 80E6C0                 <1>     and     dh, RATE_MSK            ; KEEP ONLY RATE
000043C4 E83E080000             <1>     call    CMOS_TYPE               ; RETURN DRIVE TYPE IN AL (AL)
                                <1>     ;;20/02/2015
                                <1>     ;;jc    short RWV_ASSUME        ; ERROR IN CMOS
000043C9 7447                   <1>     jz      short RWV_ASSUME ; 20/02/2015
000043CB 3C01                   <1>     cmp     al, 1                   ; 40 TRACK DRIVE?
000043CD 750D                   <1>     jne     short RWV_1             ; NO, BYPASS CMOS VALIDITY CHECK
000043CF F687[43770100]01       <1>     test    byte [DSK_STATE+edi], TRK_CAPA ; CHECK FOR 40 TRACK DRIVE
000043D6 7411                   <1>     jz      short RWV_2             ; YES, CMOS IS CORRECT
                                <1>     ;mov    al, 2                   ; CHANGE TO 1.2M
                                <1>     ; 06/08/2022
000043D8 FEC0                   <1>     inc     al ; al = 2
000043DA EB0D                   <1>     jmp     short RWV_2
                                <1> RWV_1:
                                <1>     ; 09/08/2022
                                <1>     ;jb     short RWV_2             ; NO DRIVE SPECIFIED, CONTINUE
000043DC F687[43770100]01       <1>     test    byte [DSK_STATE+edi], TRK_CAPA ; IS IT REALLY 40 TRACK?
000043E3 7504                   <1>     jnz     short RWV_2             ; NO, 80 TRACK
000043E5 B001                   <1>     mov     al, 1                   ; IT IS 40 TRACK, FIX CMOS VALUE
000043E7 EB00                   <1>     jmp     short rwv_3
                                <1> RWV_2:
                                <1>     ; 09/08/2022
                                <1>     ;or     al, al                  ; TEST FOR NO DRIVE
                                <1>     ;jz     short RWV_ASSUME        ; ASSUME TYPE, USE MAX TRACK
                                <1> rwv_3:
000043E9 E855030000             <1>     call    DR_TYPE_CHECK           ; RTN CS:BX = MEDIA/DRIVE PARAM TBL.
000043EE 7222                   <1>     jc      short RWV_ASSUME        ; TYPE NOT IN TABLE (BAD CMOS)
                                <1>
                                <1> ;-----      SEARCH FOR MEDIA/DRIVE PARAMETER TABLE
                                <1>
000043F0 57                     <1>     push    edi                     ; SAVE DRIVE #
                                <1>     ; 09/08/2022
                                <1>     ;xor    ebx, ebx                ; EBX = INDEX TO DR_TYPE TABLE
000043F1 BB[08660000]           <1>     mov     ebx, DR_TYPE
                                <1>     ;mov    ecx, DR_CNT             ; ECX = LOOP COUNT
000043F6 B106                   <1>     mov     cl, DR_CNT
                                <1> RWV_DR_SEARCH:
                                <1>     ;mov    ah, [DR_TYPE+ebx]       ; GET DRIVE TYPE
000043F8 8A23                   <1>     mov     ah, [ebx]
000043FA 80E47F                 <1>     and     ah, BIT7OFF             ; MASK OUT MSB
000043FD 38E0                   <1>     cmp     al, ah                  ; DRIVE TYPE MATCH?
                                <1>     ; 09/08/2022
                                <1>     ;cmp    dl, ah
000043FF 7509                   <1>     jne     short RWV_NXT_MD        ; NO, CHECK NEXT DRIVE TYPE
                                <1> RWV_DR_FND:
                                <1>     ;mov    edi, [DR_TYPE+ebx+1]    ; EDI = MEDIA/DRIVE PARAMETER TABLE
00004401 43                     <1>     inc     ebx
00004402 8B3B                   <1>     mov     edi, [ebx]
00004404 4B                     <1>     dec     ebx
                                <1> RWV_MD_SEARH:
00004405 3A770C                 <1>         cmp dh, [edi+MD.RATE]       ; MATCH?
00004408 741D                   <1>     je      short RWV_MD_FND        ; YES, GO GET 1ST SPECIFY BYTE
                                <1> RWV_NXT_MD:
0000440A 83C305                 <1>     add     ebx, 5                  ; CHECK NEXT DRIVE TYPE
                                <1>     ;loop   RWV_DR_SEARCH
0000440D FEC9                   <1>     dec     cl
0000440F 75E7                   <1>     jnz     short RWV_DR_SEARCH
00004411 5F                     <1>     pop     edi                     ; RESTORE DRIVE #
                                <1>
                                <1> ;-----      ASSUME PRIMARY DRIVE IS INSTALLED AS SHIPPED
                                <1>
                                <1> RWV_ASSUME:
00004412 BB[26660000]           <1>     mov     ebx, MD_TBL1            ; POINT TO 40 TRACK 250 KBS
00004417 F687[43770100]01       <1>     test    byte [DSK_STATE+edi], TRK_CAPA ; TEST FOR 80 TRACK
0000441E 740A                   <1>     jz      short RWV_MD_FND1       ; MUST BE 40 TRACK
00004420 BB[40660000]           <1>     mov     ebx, MD_TBL3            ; POINT TO 80 TRACK 500 KBS
00004425 EB03                   <1>     jmp     short RWV_MD_FND1       ; GO SPECIFY PARAMTERS
                                <1>
```

```
                            <1> ;-----      CS:BX POINTS TO MEDIA/DRIVE PARAMETER TABLE
                            <1>
                            <1> RWV_MD_FND:
00004427 89FB               <1>     mov     ebx, edi                ; BX = MEDIA/DRIVE PARAMETER TABLE
00004429 5F                 <1>     pop     edi                     ; RESTORE DRIVE #
                            <1>
                            <1> ;-----      SEND THE SPECIFY COMMAND TO THE CONTROLLER
                            <1>
                            <1> RWV_MD_FND1:
0000442A E85A030000         <1>     call    SEND_SPEC_MD
0000442F E8E3040000         <1>     call    CHK_LASTRATE            ; ZF=1 ATTEMP RATE IS SAME AS LAST RATE
00004434 7405               <1>     jz      short RWV_DBL           ; YES,SKIP SEND RATE COMMAND
00004436 E8BC040000         <1>     call    SEND_RATE               ; SEND DATA RATE TO NEC
                            <1> RWV_DBL:
0000443B 53                 <1>     push    ebx                     ; SAVE MEDIA/DRIVE PARAM TBL ADDRESS
0000443C E825070000         <1>     call    SETUP_DBL               ; CHECK FOR DOUBLE STEP
00004441 5B                 <1>     pop     ebx                     ; RESTORE ADDRESS
00004442 7225               <1>     jc      short CHK_RET           ; ERROR FROM READ ID, POSSIBLE RETRY
                            <1>     ;pop    eax                     ; RESTORE NEC COMMAND
                            <1>     ;push   eax                     ; SAVE NEC COMMAND
                            <1>     ; 09/08/2022
00004444 8B0424             <1>     mov     eax, [esp]
                            <1>     ;push   ebx                     ; SAVE MEDIA/DRIVE PARAM TBL ADDRESS
00004447 E8E0040000         <1>     call    DMA_SETUP               ; SET UP THE DMA
                            <1>     ;pop    ebx
0000444C 58                 <1>     pop     eax                     ; RESTORE NEC COMMAND
0000444D 7231               <1>     jc      short RWV_BAC           ; CHECK FOR DMA BOUNDARY ERROR
0000444F 50                 <1>     push    eax                     ; SAVE NEC COMMAND
00004450 53                 <1>     push    ebx                     ; SAVE MEDIA/DRIVE PARAM TBL ADDRESS
                            <1>     ; 11/08/2022
00004451 8B5C2420           <1>     mov     ebx, [esp+32] ; ECX
00004455 E866050000         <1>     call    NEC_INIT                ; INITIALIZE NEC
0000445A 5B                 <1>     pop     ebx                     ; RESTORE ADDRESS
0000445B 720C               <1>     jc      short CHK_RET           ; ERROR - EXIT
0000445D E88C050000         <1>     call    RWV_COM                 ; OP CODE COMMON TO READ/WRITE
00004462 7205               <1>     jc      short CHK_RET           ; ERROR - EXIT
00004464 E8D2050000         <1>     call    NEC_TERM                ; TERMINATE, GET STATUS, ETC.
                            <1> CHK_RET:
00004469 E871060000         <1>     call    RETRY                   ; CHECK FOR, SETUP RETRY
0000446E 58                 <1>     pop     eax                     ; RESTORE READ/WRITE PARAMETER
0000446F 7305               <1>     jnc     short RWV_END           ; CY = 0 NO RETRY
00004471 E936FFFFFF         <1>         jmp DO_AGAIN                  ; CY = 1 MEANS RETRY
                            <1> RWV_END:
00004476 E81C060000         <1>     call    DSTATE                  ; ESTABLISH STATE IF SUCCESSFUL
0000447B E8AE060000         <1>     call    NUM_TRANS               ; AL = NUMBER TRANSFERRED
                            <1> RWV_BAC:                            ; BAD DMA ERROR ENTRY
                            <1>     ; 06/08/2022
                            <1>     ;push   eax                     ; SAVE NUMBER TRANSFERRED
                            <1>     ;call   XLAT_OLD                ; TRANSLATE STATE TO COMPATIBLE MODE
                            <1>     ;pop    eax                     ; RESTORE NUMBER TRANSFERRED
                            <1>     ;call   SETUP_END               ; VARIOUS CLEANUPS
                            <1>     ;retn
                            <1>
                            <1> ;--------------------------------------------------------------------------------
                            <1> ; SETUP_END
                            <1> ;   RESTORES @MOTOR_COUNT TO PARAMETER PROVIDED IN TABLE
                            <1> ;   AND LOADS @DSKETTE_STATUS TO AH, AND SETS CY.
                            <1> ;
                            <1> ; ON EXIT:
                            <1> ;   AH, @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                            <1> ;--------------------------------------------------------------------------------
                            <1> SETUP_END:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                            <1>     ;mov    dl, 2                   ; GET THE MOTOR WAIT PARAMETER
                            <1>     ;push   ax                      ; SAVE NUMBER TRANSFERRED
                            <1>     ; 11/04/2021
00004480 50                 <1>     push    eax
                            <1>     ; 06/08/2022
00004481 B002               <1>     mov     al, 2                   ; GET THE MOTOR WAIT PARAMETER
00004483 E888070000         <1>     call    GET_PARM
00004488 8825[37770100]     <1>     mov     [MOTOR_COUNT], ah       ; STORE UPON RETURN
                            <1>     ;pop    ax                      ; RESTORE NUMBER TRANSFERRED
                            <1>     ; 11/04/2021
0000448E 58                 <1>     pop     eax
0000448F 8A25[38770100]     <1>     mov     ah, [DSKETTE_STATUS]    ; GET STATUS OF OPERATION
00004495 08E4               <1>     or      ah, ah                  ; CHECK FOR ERROR
00004497 7403               <1>     jz      short NUN_ERR           ; NO ERROR
00004499 30C0               <1>     xor     al, al                  ; CLEAR NUMBER RETURNED
                            <1>     ; 06/08/2022
0000449B F9                 <1>     stc
                            <1> NUN_ERR:
                            <1>     ;cmp    ah, 1                   ; SET THE CARRY FLAG TO INDICATE
                            <1>     ;cmc                            ; SUCCESS OR FAILURE
0000449C C3                 <1>     retn
                            <1>
                            <1> ;--------------------------------------------------------------------------------
                            <1> ; DISK_VERF (AH = 04H)
                            <1> ;   DISKETTE VERIFY.
                            <1> ;
                            <1> ; ON ENTRY: DI      : DRIVE #
                            <1> ;           SI-HI   : HEAD #
                            <1> ;           SI-LOW  : # OF SECTORS
                            <1> ;           ES      : BUFFER SEGMENT
                            <1> ;           [BP]    : SECTOR #
                            <1> ;           [BP+1]  : TRACK #
                            <1> ;           [BP+2]  : BUFFER OFFSET
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                            <1> ;--------------------------------------------------------------------------------
                            <1> DSK_VERF:
0000449D 8025[36770100]7F   <1>     and     byte [MOTOR_STATUS], 01111111b ; INDICATE A READ OPERATION
000044A4 66B842E6           <1>     mov     ax, 0E642h              ; AX = NEC COMMAND, DMA COMMAND
                            <1>     ;call   RD_WR_VF                ; COMMON READ/WRITE/VERIFY
                            <1>     ;retn
                            <1>     ; 06/08/2022
000044A8 E9E7FEFFFF         <1>     jmp     RD_WR_VF
                            <1>
                            <1> ;--------------------------------------------------------------------------------
                            <1> ; DISK_FORMAT       (AH = 05H)
                            <1> ;   DISKETTE FORMAT.
                            <1> ;
                            <1> ; ON ENTRY: DI      : DRIVE #
                            <1> ;           SI-HI   : HEAD #
                            <1> ;           SI-LOW  : # OF SECTORS
                            <1> ;           ES      : BUFFER SEGMENT
                            <1> ;           [BP]    : SECTOR #
                            <1> ;           [BP+1]  : TRACK #
                            <1> ;           [BP+2]  : BUFFER OFFSET
                            <1> ;           @DISK_POINTER POINTS TO THE PARAMETER TABLE OF THIS DRIVE
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                            <1> ;--------------------------------------------------------------------------------
                            <1> DSK_FORMAT:
                            <1>     ; 11/08/2022
```

```
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
000044AD E8F7020000             <1>     call    XLAT_NEW                ; TRANSLATE STATE TO PRESENT ARCH.
000044B2 E89F030000             <1>     call    FMT_INIT                ; ESTABLISH STATE IF UNESTABLISHED
000044B7 800D[36770100]80       <1>     or      byte [MOTOR_STATUS], 10000000b ; INDICATE WRITE OPERATION
000044BE E8E7030000             <1>     call    MED_CHANGE              ; CHECK MEDIA CHANGE AND RESET IF SO
000044C3 7261                   <1>         jc      short FM_DON            ; MEDIA CHANGED, SKIP
000044C5 E896020000             <1>     call    SEND_SPEC               ; SEND SPECIFY COMMAND TO NEC
000044CA E848040000             <1>     call    CHK_LASTRATE            ; ZF=1 ATTEMPT RATE IS SAME AS LAST RATE
000044CF 7405                   <1>         jz      short FM_WR             ; YES, SKIP SPECIFY COMMAND
000044D1 E821040000             <1>     call    SEND_RATE               ; SEND DATA RATE TO CONTROLLER
                                <1> FM_WR:
000044D6 E8C8040000             <1>     call    FMTDMA_SET              ; SET UP THE DMA FOR FORMAT
000044DB 7249                   <1>         jc      short FM_DON            ; RETURN WITH ERROR
000044DD B44D                   <1>     mov     ah, 04Dh                ; ESTABLISH THE FORMAT COMMAND
                                <1>     ; 11/08/2022
000044DF 8B5C2418               <1>     mov     ebx, [esp+24] ; ECX
000044E3 E8D8040000             <1>     call    NEC_INIT                ; INITIALIZE THE NEC
000044E8 723C                   <1>     jc      short FM_DON            ; ERROR - EXIT
000044EA B8[26450000]           <1>         mov     eax, FM_DON             ; LOAD ERROR ADDRESS
000044EF 50                     <1>     push    eax                     ; PUSH NEC_OUT ERROR RETURN
                                <1>     ;mov    dl, 3                   ; BYTES/SECTOR VALUE TO NEC
                                <1>     ; 06/08/2022
000044F0 B003                   <1>     mov     al, 3
000044F2 E819070000             <1>     call    GET_PARM
000044F7 E80A080000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    dl, 4                   ; SECTORS/TRACK VALUE TO NEC
                                <1>     ; 06/08/2022
000044FC B004                   <1>     mov     al, 4
000044FE E80D070000             <1>     call    GET_PARM
00004503 E8FE070000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    dl, 7                   ; GAP LENGTH VALUE TO NEC
                                <1>     ; 06/08/2022
00004508 B007                   <1>     mov     al, 7
0000450A E801070000             <1>     call    GET_PARM
0000450F E8F2070000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    dl, 8                   ; FILLER BYTE TO NEC
                                <1>     ; 06/08/2022
00004514 B008                   <1>     mov     al, 8
00004516 E8F5060000             <1>     call    GET_PARM
0000451B E8E6070000             <1>     call    NEC_OUTPUT
00004520 58                     <1>     pop     eax                     ; THROW AWAY ERROR
00004521 E815050000             <1>     call    NEC_TERM                ; TERMINATE, RECEIVE STATUS, ETC,
                                <1> FM_DON:
                                <1>     ; 06/08/2022
                                <1>     ;call   XLAT_OLD                ; TRANSLATE STATE TO COMPATIBLE MODE
                                <1>     ; 06/08/2022
00004526 E947FEFFFF             <1>     jmp     SETUP_END_X
                                <1>     ;call   SETUP_END               ; VARIOUS CLEANUPS
                                <1>     ;; 06/08/2022
                                <1>     ;mov    ebx, esi                ; GET SAVED AL TO BL
                                <1>     ;mov    al, bl                  ; PUT BACK FOR RETURN
                                <1>     ;retn
                                <1>
                                <1> ;---------------------------------------------------------------------------
                                <1> ; FNC_ERR
                                <1> ;   INVALID FUNCTION REQUESTED OR INVALID DRIVE:
                                <1> ;   SET BAD COMMAND IN STATUS.
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;---------------------------------------------------------------------------
                                <1> FNC_ERR:                            ; INVALID FUNCTION REQUEST
                                <1>     ;mov    ax, si
                                <1>     ; 06/08/2022
0000452B 89F0                   <1>     mov     eax, esi                ; RESTORE AL
0000452D B401                   <1>     mov     ah, BAD_CMD             ; SET BAD COMMAND ERROR
0000452F 8825[38770100]         <1>     mov     [DSKETTE_STATUS], ah    ; STORE IN DATA AREA
00004535 F9                     <1>     stc                             ; SET CARRY INDICATING ERROR
00004536 C3                     <1>     retn
                                <1>
                                <1> ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 30/08/2020
                                <1> ; 29/08/2020
                                <1> ; 01/06/2016
                                <1> ; 28/05/2016
                                <1> ; 27/05/2016 - TRDOS 386 (TRDOS v.2.0)
                                <1> ;---------------------------------------------------------------------------
                                <1> ; DISK_PARMS       (AH = 08H)
                                <1> ;   READ DRIVE PARAMETERS.
                                <1> ;
                                <1> ; ON ENTRY: DI : DRIVE #
                                <1> ;           ; 27/05/2016
                                <1> ;           EBX = Buffer Address for floppy disk parameters table (16 bytes)
                                <1> ;
                                <1> ; ON EXIT:  CL/[BP]   = BITS 7 & 6 HI 2 BITS OF MAX CYLINDER
                                <1> ;                       BITS 0-5 MAX SECTORS/TRACK
                                <1> ;           CH/[BP+1] = LOW 8 BITS OF MAX CYLINDER
                                <1> ;           BL/[BP+2] = BITS 7-4 = 0
                                <1> ;                       BITS 3-0 = VALID CMOS DRIVE TYPE
                                <1> ;           BH/[BP+3] = 0
                                <1> ;           DL/[BP+4] = # DRIVES INSTALLED (VALUE CHECKED)
                                <1> ;           DH/[BP+5] = MAX HEAD #
                                <1> ;         ** 27/05/2016 - TRDOS 386 (TRDOS v2.0) **
                                <1> ;             ** EBX = Buffer address for floppy disk parameters table **
                                <1> ;           ;DI/[BP+6] = OFFSET TO DISK_BASE
                                <1> ;           ;ES        = SEGMENT OF DISK_BASE
                                <1> ;
                                <1> ;           AX        = 0
                                <1> ;
                                <1> ;           NOTE : THE ABOVE INFORMATION IS STORED IN THE USERS STACK AT
                                <1> ;                  THE LOCATIONS WHERE THE MAIN ROUTINE WILL POP THEM
                                <1> ;                  INTO THE APPROPRIATE REGISTERS BEFORE RETURNING TO THE
                                <1> ;                  CALLER.
                                <1> ;---------------------------------------------------------------------------
                                <1> DSK_PARMS:
                                <1>     ; 09/08/2022
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ;
                                <1>     ; Registers on stack:
                                <1>     ;    ebx = esp+28
                                <1>     ;    ecx = esp+24
                                <1>     ;    edx = esp+20
                                <1>     ;    esi = esp+16
                                <1>     ;    edi = esp+12
                                <1>     ;    return address from DSKETTE_IO_1 = esp+8
                                <1>     ;    ebp = esp+4
                                <1>     ;    return address from DSK_PARMS = esp
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ebp = buffer address
                                <1>     ;   edi = drive number (0 or 1)
                                <1>
                                <1>     ; OUTPUT:
                                <1>     ;   ebx = [esp+28] ((BL = cmos type, BH = 0))
                                <1>     ;   ecx = [esp+24] ((CL = sectors per track, CH = tracks - 1))
```

```
                                <1>     ;   edx = [esp+20] ((DL = floppy drive count, DH = heads - 1))
                                <1>     ;   user's buffer = FDPT table
                                <1>
00004537 E86D020000             <1>     call    XLAT_NEW                ; TRANSLATE STATE TO PRESENT ARCH,
                                <1>
                                <1>     ;;mov   word [bp+2], 0          ; DRIVE TYPE = 0
                                <1>             ;;mov   ax, [EQUIP_FLAG]        ; LOAD EQUIPMENT FLAG FOR # DISKETTES
                                <1>             ;;and   al, 11000001b           ; KEEP DISKETTE DRIVE BITS
                                <1>             ;;mov   dl, 2                   ; DISKETTE DRIVES = 2
                                <1>             ;;cmp   al, 01000001b           ; 2 DRIVES INSTALLED ?
                                <1>             ;;jz    short STO_DL            ; IF YES JUMP
                                <1>             ;;dec   dl                      ; DISKETTE DRIVES = 1
                                <1>             ;;cmp   al, 00000001b           ; 1 DRIVE INSTALLED ?
                                <1>             ;;jnz   short NON_DRV           ; IF NO JUMP
                                <1>
0000453C 29D2                   <1>     sub     edx, edx
0000453E 66A1[8C660000]         <1>     mov     ax, [fd0_type]
00004544 6621C0                 <1>     and     ax, ax
00004547 747C                   <1>         jz      short NON_DRV
00004549 FEC2                   <1>     inc     dl
0000454B 20E4                   <1>     and     ah, ah
0000454D 7402                   <1>     jz      short STO_DL
0000454F FEC2                   <1>     inc     dl
                                <1> STO_DL:
                                <1>     ; 30/08/2020
                                <1>     ;cmp    dx, di
                                <1>     ; 06/08/2022
00004551 39FA                   <1>     cmp     edx, edi
00004553 7670                   <1>     jna     short NON_DRV
                                <1>     ;
                                <1>     ;;mov   [bp+4], dl              ; STORE NUMBER OF DRIVES
                                <1>     ;mov    [ebp+8], edx ; 20/02/2015
                                <1>
                                <1>     ; 06/08/2022
00004555 FEC6                   <1>     inc     dh ; number of heads - 1
                                <1>     ; dh = 1
                                <1>     ; dl = number of floppy/diskette drives (DL)
00004557 89542414               <1>     mov     [esp+20], edx
                                <1>
                                <1>     ; 11/04/2021
                                <1>     ;cmp    di, 1                   ; CHECK FOR VALID DRIVE
                                <1>     ;;ja    short NON_DRV1          ; DRIVE INVALID
                                <1>     ;ja     NON_DRV1 ; 29/08/2020
                                <1>     ;
                                <1>     ;;mov   byte [bp+5], 1          ; MAXIMUM HEAD NUMBER =      1
                                <1>     ; 06/08/2022
                                <1>     ;mov    byte [ebp+9], 1 ; 20/02/2015
                                <1>
0000455B E8A7060000             <1>     call    CMOS_TYPE               ; RETURN DRIVE TYPE IN AL
                                <1>     ;;20/02/2015
                                <1>     ;;jc    short CHK_EST           ; IF CMOS BAD CHECKSUM ESTABLISHED
                                <1>     ;;or    al, al                  ; TEST FOR NO DRIVE TYPE
00004560 740F                   <1>     jz      short CHK_EST           ; JUMP IF SO
00004562 E8DC010000             <1>     call    DR_TYPE_CHECK           ; RTN CS:BX = MEDIA/DRIVE PARAM TBL
00004567 7208                   <1>     jc      short CHK_EST           ; TYPE NOT IN TABLE (POSSIBLE BAD CMOS)
                                <1>     ;mov    [bp+2], al              ; STORE VALID CMOS DRIVE TYPE
                                <1>         ;mov        [ebp+4], al ; 06/02/2015
00004569 8A4B04                 <1>     mov     cl, [ebx+MD.SEC_TRK]    ; GET SECTOR/TRACK
0000456C 8A6B0B                 <1>         mov ch, [ebx+MD.MAX_TRK]    ; GET MAX. TRACK NUMBER
0000456F EB36                   <1>     jmp     short STO_CX            ; CMOS GOOD, USE CMOS
                                <1> CHK_EST:
00004571 8AA7[43770100]         <1>     mov     ah, [DSK_STATE+edi]     ; LOAD STATE FOR THIS DRIVE
00004577 F6C410                 <1>     test    ah, MED_DET             ; CHECK FOR ESTABLISHED STATE
0000457A 744D                   <1>     jz      short NON_DRV1          ; CMOS BAD/INVALID OR UNESTABLISHED
                                <1> USE_EST:
0000457C 80E4C0                 <1>     and     ah, RATE_MSK            ; ISOLATE STATE
0000457F 80FC80                 <1>     cmp     ah, RATE_250            ; RATE 250 ?
00004582 755C                   <1>     jne     short USE_EST2          ; NO, GO CHECK OTHER RATE
                                <1>
                                <1> ;-----      DATA RATE IS 250 KBS, TRY 360 KB TABLE FIRST
                                <1>
00004584 B001                   <1>     mov     al, 1                   ; DRIVE TYPE 1 (360KB)
00004586 E8B8010000             <1>     call    DR_TYPE_CHECK           ; RTN CS:BX = MEDIA/DRIVE PARAM TBL
0000458B 8A4B04                 <1>         mov cl, [ebx+MD.SEC_TRK]    ; GET SECTOR/TRACK
0000458E 8A6B0B                 <1>         mov ch, [ebx+MD.MAX_TRK]    ; GET MAX. TRACK NUMBER
00004591 F687[43770100]01       <1>     test    byte [DSK_STATE+edi], TRK_CAPA ; 80 TRACK ?
00004598 740D                   <1>     jz      short STO_CX            ; MUST BE 360KB DRIVE
                                <1>
                                <1> ;-----      IT IS 1.44 MB DRIVE
                                <1>
                                <1> PARM144:
0000459A B004                   <1>     mov     al, 4                   ; DRIVE TYPE 4 (1.44MB)
0000459C E8A2010000             <1>     call    DR_TYPE_CHECK           ; RTN CS:BX = MEDIA/DRIVE PARAM TBL
000045A1 8A4B04                 <1>     mov     cl, [ebx+MD.SEC_TRK]    ; GET SECTOR/TRACK
000045A4 8A6B0B                 <1>     mov     ch, [ebx+MD.MAX_TRK]    ; GET MAX. TRACK NUMBER
                                <1> STO_CX:
                                <1>     ;mov    [ebp], ecx              ; SAVE POINTER IN STACK FOR RETURN
                                <1>     ; 06/08/2022
000045A7 894C2418               <1>     mov     [esp+24], ecx ; spt (cl), tracks - 1 (ch)
                                <1> ES_DI:
                                <1>     ;mov    [bp+6], bx              ; ADDRESS OF MEDIA/DRIVE PARM TABLE
                                <1>     ;mov    [ebp+12], ebx ; 06/02/2015
                                <1>     ;mov    ax, cs                  ; SEGMENT MEDIA/DRIVE PARAMETER TABLE
                                <1>     ;mov    es, ax                  ; ES IS SEGMENT OF TABLE
                                <1>     ;
                                <1>     ; 28/05/2016
                                <1>     ; 27/05/2016
                                <1>     ; return floppy disk parameters table to user
                                <1>     ; in user's buffer, which is pointed by EBX
                                <1>
                                <1>     ; 09/08/2022
                                <1>     ;movzx  eax, al
000045AB 29C9                   <1>     sub     ecx, ecx
000045AD 88C1                   <1>     mov     cl, al
                                <1>     ;;mov   [ebp+4], eax  ; ebx   ; drive type (for floppy drives)
                                <1>     ; 06/08/2022
                                <1>     ;mov    [esp+28], eax ; drive type
                                <1>     ; 09/08/2022
000045AF 894C241C               <1>     mov     [esp+28], ecx ; drive type
                                <1>
                                <1>     ; 06/08/2022
                                <1>     ;push   edi
                                <1>
                                <1>     ;mov    edi, [ebp+4]            ; ebx (input), user's buffer address
                                <1>     ; 06/08/2022
                                <1>     ;mov    edi, ebp ; [esp+28] ; user's buffer address
                                <1>     ;
                                <1>     ;; 29/08/2020
                                <1>     ;or     edi, edi
                                <1>     ;jz     short no_copy_fdpt
                                <1>     ; 06/08/2022
000045B3 09ED                   <1>     or      ebp, ebp ; [esp+28] = ebx ; user's buffer address if > 0
000045B5 740B                   <1>     jz      short DP_OUT
                                <1>     ; 06/08/2022
```

```
                                <1>     ;push   edi
000045B7 89EF                   <1>     mov     edi, ebp
                                <1>     ;
                                <1>     ; 06/08/2022
                                <1>     ; 01/06/2016 (Int 33h, disk type return for floppy disks, in BL)
                                <1>     ;mov    [user_buffer], eax    ; 01/06/2016 (overwrite ebx return value)
                                <1>
                                <1>     ;(Int 33h, Function 08h will replace user's buffer addr with disk type!)
                                <1>     ;
000045B9 89DE                   <1>     mov     esi, ebx              ; floppy disk parameter table (16 bytes)
                                <1>     ;mov    ecx, 16 ; 16 bytes
                                <1>         ; 09/08/2022
                                <1>     ; ecx < 256
                                <1>     ; 06/08/2022
                                <1>     ;sub    ecx, ecx
000045BB B110                   <1>     mov     cl, 16
000045BD E859C50000             <1>     call     transfer_to_user_buffer ; trdosk6.s (16/05/2016)
                                <1> no_copy_fdpt:
                                <1>     ; 06/08/2022
                                <1>     ;pop    edi
                                <1> DP_OUT:
                                <1>     ; 06/08/2022
                                <1>     ;call   XLAT_OLD              ; TRANSLATE STATE TO COMPATIBLE MODE
                                <1>     ;xor    ax, ax                ; CLEAR
                                <1>     ; 06/08/2022
000045C2 31C0                   <1>     xor     eax, eax
                                <1>     ;clc
000045C4 C3                     <1>     retn
                                <1>
                                <1> ;-----      NO DRIVE PRESENT HANDLER
                                <1>
                                <1> NON_DRV:
                                <1>     ;;mov   byte [bp+4], 0        ; CLEAR NUMBER OF DRIVES
                                <1>     ;mov    [ebp+8], edx ; 0 ; 20/02/2015
                                <1>     ; 06/08/2022
000045C5 89542414               <1>     mov     [esp+20], edx ; 0 ; number of floppy drives and heads are 0
                                <1> NON_DRV1:
000045C9 6681FF8000             <1>     cmp     di, 80h               ; CHECK FOR FIXED MEDIA TYPE REQUEST
000045CE 7206                   <1>     jb      short NON_DRV2        ; CONTINUE IF NOT REQUEST FALL THROUGH
                                <1>
                                <1> ;-----      FIXED DISK REQUEST FALL THROUGH ERROR
                                <1>
                                <1>     ; 06/08/2022
                                <1>     ;call   XLAT_OLD              ; ELSE TRANSLATE TO COMPATIBLE MODE
                                <1>     ;mov    ax, si                ; RESTORE AL
                                <1>     ; 06/08/2022
000045D0 89F0                   <1>     mov     eax, esi
000045D2 B401                   <1>     mov     ah, BAD_CMD           ; SET BAD COMMAND ERROR
000045D4 F9                     <1>     stc
000045D5 C3                     <1>     retn
                                <1>
                                <1> NON_DRV2:
                                <1>     ;xor    ax, ax                ; CLEAR PARMS IF NO DRIVES OR CMOS BAD
000045D6 31C0                   <1>     xor     eax, eax
                                <1>     ;mov    [ebp], ax             ; TRACKS, SECTORS/TRACK = 0
                                <1>     ; 06/08/2022
000045D8 89442418               <1>     mov     [esp+24], eax ; spt and max. track number is 0
                                <1>
                                <1>     ;;mov   [bp+5], ah            ; HEAD = 0
                                <1>     ; 06/08/2022
                                <1>     ;mov    [ebp+9], ah ; 06/02/2015
                                <1>     ; 06/08/2022
                                <1>     ;mov    [esp+21], ah ; 0
                                <1>
                                <1>     ;;mov   [bp+6], ax            ; OFFSET TO DISK_BASE = 0
                                <1>     ; 06/08/2022
                                <1>     ;mov    [ebp+12], eax
                                <1>
                                <1>     ;;mov   es, ax                ; ES IS SEGMENT OF TABLE
                                <1>     ;jmp    short DP_OUT
                                <1>
                                <1>     ; 06/08/2022
                                <1>     ; 30/08/2020
                                <1>     ;call   XLAT_OLD
                                <1>     ;;mov   ah, NOT_RDY ; drive not ready
000045DC B407                   <1>     mov     ah, INIT_FAIL ; DRIVE PARAMETER ACTIVITY FAILED
000045DE F9                     <1>     stc     ; cf -> 1, ah = 'drive not ready' error code
000045DF C3                     <1>     retn
                                <1>
                                <1> ;-----      DATA RATE IS EITHER 300 KBS OR 500 KBS, TRY 1.2 MB TABLE FIRST
                                <1>
                                <1> USE_EST2:
000045E0 B002                   <1>     mov     al, 2                 ; DRIVE TYPE 2 (1.2MB)
000045E2 E85C010000             <1>     call    DR_TYPE_CHECK         ; RTN CS:BX = MEDIA/DRIVE PARAM TBL
000045E7 8A4B04                 <1>         mov     cl, [ebx+MD.SEC_TRK]    ; GET SECTOR/TRACK
000045EA 8A6B0B                 <1>         mov     ch, [ebx+MD.MAX_TRK]    ; GET MAX. TRACK NUMBER
000045ED 80FC40                 <1>     cmp     ah, RATE_300          ; RATE 300 ?
000045F0 74B5                   <1>     jz      short STO_CX          ; MUST BE 1.2MB DRIVE
000045F2 EBA6                   <1>     jmp     short PARM144         ; ELSE, IT IS 1.44MB DRIVE
                                <1>
                                <1> ; 30/08/2020
                                <1>
                                <1> ;--------------------------------------------------------------------------------
                                <1> ; DISK_TYPE (AH = 15H)
                                <1> ;   THIS ROUTINE RETURNS THE TYPE OF MEDIA INSTALLED.
                                <1> ;
                                <1> ;  ON ENTRY:       DI = DRIVE #
                                <1> ;
                                <1> ;  ON EXIT: AH = DRIVE TYPE, CY=0
                                <1> ;--------------------------------------------------------------------------------
                                <1> DSK_TYPE:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
000045F4 E8B0010000             <1>     call    XLAT_NEW              ; TRANSLATE STATE TO PRESENT ARCH.
000045F9 8A87[43770100]         <1>     mov     al, [DSK_STATE+edi]   ; GET PRESENT STATE INFORMATION
000045FF 08C0                   <1>     or      al, al                ; CHECK FOR NO DRIVE
00004601 740D                   <1>     jz      short NO_DRV
00004603 B401                   <1>     mov     ah, NOCHGLN           ; NO CHANGE LINE FOR 40 TRACK DRIVE
00004605 A801                   <1>     test    al, TRK_CAPA          ; IS THIS DRIVE AN 80 TRACK DRIVE?
00004607 7402                   <1>     jz      short DT_BACK         ; IF NO JUMP
00004609 B402                   <1>     mov     ah, CHGLN             ; CHANGE LINE FOR 80 TRACK DRIVE
                                <1> DT_BACK:
                                <1>     ; 06/08/2022
                                <1>     ;;push  ax                    ; SAVE RETURN VALUE
                                <1>     ; 11/04/2021
                                <1>     ;push   eax
                                <1>     ;call   XLAT_OLD              ; TRANSLATE STATE TO COMPATIBLE MODE
                                <1>     ;; 11/04/2021
                                <1>     ;pop    eax
                                <1>     ;; pop  ax
                                <1>     ;clc                          ; NO ERROR
                                <1>     ;mov    bx, si                ; GET SAVED AL TO BL
                                <1>     ; 06/08/2022
0000460B 89F3                   <1>     mov     ebx, esi
0000460D 88D8                   <1>     mov     al, bl                ; PUT BACK FOR RETURN
```

```
0000460F C3                 <1>     retn
                            <1> NO_DRV:
                            <1>     ;xor    ah, ah                  ; NO DRIVE PRESENT OR UNKNOWN
                            <1>     ;jmp    short DT_BACK
                            <1>
                            <1>     ; 06/08/2022
                            <1>     ; 30/08/2020
                            <1>     ;call   XLAT_OLD
00004610 29C0               <1>     sub     eax, eax
00004612 F9                 <1>     stc     ; cf = 1 -> drive not ready, ah = 0 (disk type = 0)
00004613 C3                 <1>     retn
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; DISK_CHANGE      (AH = 16H)
                            <1> ;   THIS ROUTINE RETURNS THE STATE OF THE DISK CHANGE LINE.
                            <1> ;
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ;
                            <1> ; ON EXIT:  AH = @DSKETTE_STATUS
                            <1> ;                00 - DISK CHANGE LINE INACTIVE, CY = 0
                            <1> ;                06 - DISK CHANGE LINE ACTIVE, CY = 1
                            <1> ;-----------------------------------------------------------------------------
                            <1> DSK_CHANGE:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004614 E890010000         <1>     call    XLAT_NEW                ; TRANSLATE STATE TO PRESENT ARCH.
00004619 8A87[43770100]     <1>     mov     al, [DSK_STATE+edi]     ; GET MEDIA STATE INFORMATION
0000461F 08C0               <1>     or      al, al                  ; DRIVE PRESENT ?
00004621 7417               <1>     jz      short DC_NON            ; JUMP IF NO DRIVE
00004623 A801               <1>     test    al, TRK_CAPA            ; 80 TRACK DRIVE ?
00004625 7407               <1>     jz      short SETIT             ; IF SO , CHECK CHANGE LINE
                            <1> DC0:
00004627 E86B080000         <1>         call    READ_DSKCHNG            ; GO CHECK STATE OF DISK CHANGE LINE
0000462C 7407               <1>     jz      short FINIS             ; CHANGE LINE NOT ACTIVE
                            <1> SETIT:
0000462E C605[38770100]06   <1>     mov     byte [DSKETTE_STATUS], MEDIA_CHANGE ; INDICATE MEDIA REMOVED
                            <1>
                            <1> FINIS:      ; 06/08/2022
                            <1>     ;call   XLAT_OLD                ; TRANSLATE STATE TO COMPATIBLE MODE
                            <1>     ; 06/08/2022
00004635 E938FDFFFF         <1>     jmp     SETUP_END_X
                            <1>     ;call   SETUP_END               ; VARIOUS CLEANUPS
                            <1>     ;; 06/08/2022
                            <1>     ;mov    ebx, esi                ; GET SAVED AL TO BL
                            <1>     ;mov    al, bl                  ; PUT BACK FOR RETURN
                            <1>     ;retn
                            <1> DC_NON:
0000463A 800D[38770100]80   <1>     or      byte [DSKETTE_STATUS], TIME_OUT ; SET TIMEOUT, NO DRIVE
00004641 EBF2               <1>     jmp     short FINIS
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; FORMAT_SET       (AH = 17H)
                            <1> ;   THIS ROUTINE IS USED TO ESTABLISH THE TYPE OF MEDIA TO BE USED
                            <1> ;   FOR THE FOLLOWING FORMAT OPERATION.
                            <1> ;
                            <1> ; ON ENTRY: SI LOW = DASD TYPE FOR FORMAT
                            <1> ;           DI     = DRIVE #
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS REFLECTS STATUS
                            <1> ;           AH = @DSKETTE_STATUS
                            <1> ;           CY = 1 IF ERROR
                            <1> ;-----------------------------------------------------------------------------
                            <1> FORMAT_SET:
00004643 E861010000         <1>     call    XLAT_NEW                ; TRANSLATE STATE TO PRESENT ARCH.
                            <1>     ;push   si                      ; SAVE DASD TYPE
                            <1>     ; 11/04/2021
00004648 56                 <1>     push    esi
                            <1>     ;mov    ax, si                  ; AH = ? , AL = DASD TYPE
                            <1>     ;xor    ah, ah                  ; AH = 0 , AL = DASD TYPE
                            <1>     ;mov    si, ax                  ; SI = DASD TYPE
                            <1>     ; 11/04/2021
00004649 89F0               <1>     mov     eax, esi
0000464B 0FB6F0             <1>     movzx   esi, al
0000464E 80A7[43770100]0F   <1>     and     byte [DSK_STATE+edi], ~(MED_DET+DBL_STEP+RATE_MSK) ; CLEAR STATE
                            <1>     ;dec    si                      ; CHECK FOR 320/360K MEDIA & DRIVE
                            <1>     ; 11/04/2021
00004655 4E                 <1>     dec     esi
00004656 7509               <1>     jnz     short NOT_320           ; BYPASS IF NOT
00004658 808F[43770100]90   <1>     or      byte [DSK_STATE+edi], MED_DET+RATE_250 ; SET TO 320/360
0000465F EB45               <1>     jmp     short S0
                            <1>
                            <1> NOT_320:
00004661 E844020000         <1>     call    MED_CHANGE              ; CHECK FOR TIME_OUT
00004666 803D[38770100]80   <1>     cmp     byte [DSKETTE_STATUS], TIME_OUT
0000466D 7437               <1>     jz      short S0                ; IF TIME OUT TELL CALLER
                            <1> S3:
                            <1>     ;dec    si                      ; CHECK FOR 320/360K IN 1.2M DRIVE
                            <1>     ; 11/04/2021
0000466F 4E                 <1>     dec     esi
00004670 7509               <1>     jnz     short NOT_320_12        ; BYPASS IF NOT
00004672 808F[43770100]70   <1>     OR      byte [DSK_STATE+edi], MED_DET+DBL_STEP+RATE_300 ; SET STATE
00004679 EB2B               <1>     jmp     short S0
                            <1>
                            <1> NOT_320_12:
                            <1>     ;dec    si                      ; CHECK FOR 1.2M MEDIA IN 1.2M DRIVE
                            <1>     ; 11/04/2021
0000467B 4E                 <1>     dec     esi
0000467C 7509               <1>     jnz     short NOT_12            ; BYPASS IF NOT
0000467E 808F[43770100]10   <1>     or      byte [DSK_STATE+edi], MED_DET+RATE_500 ; SET STATE VARIABLE
00004685 EB1F               <1>     jmp     short S0                ; RETURN TO CALLER
                            <1>
                            <1> NOT_12:
                            <1>     ;dec    si                      ; CHECK FOR SET DASD TYPE 04
                            <1>     ; 11/04/2021
00004687 4E                 <1>     dec     esi
00004688 7525               <1>     jnz     short FS_ERR            ; BAD COMMAND EXIT IF NOT VALID TYPE
                            <1>
0000468A F687[43770100]04   <1>     test    byte [DSK_STATE+edi], DRV_DET ; DRIVE DETERMINED ?
00004691 740B               <1>     jz      short ASSUME            ; IF STILL NOT DETERMINED ASSUME
00004693 B050               <1>     mov     al, MED_DET+RATE_300
00004695 F687[43770100]02   <1>         test    byte [DSK_STATE+edi], FMT_CAPA ; MULTIPLE FORMAT CAPABILITY ?
0000469C 7502               <1>     jnz     short OR_IT_IN          ; IF 1.2 M THEN DATA RATE 300
                            <1>
                            <1> ASSUME:
0000469E B090               <1>     mov     al, MED_DET+RATE_250    ; SET UP
                            <1>
                            <1> OR_IT_IN:
000046A0 0887[43770100]     <1>     or      [DSK_STATE+edi], al     ; OR IN THE CORRECT STATE
                            <1> S0:
                            <1>     ; 06/08/2022
                            <1>     ;call   XLAT_OLD                ; TRANSLATE STATE TO COMPATIBLE MODE
000046A6 E8D5FDFFFF         <1>     call    SETUP_END               ; VARIOUS CLEANUPS
                            <1>     ;pop    BX                      ; GET SAVED AL TO BL
                            <1>     ; 11/04/2021
000046AB 5B                 <1>     pop     ebx
```

```
000046AC 88D8                <1>     mov    al, bl                ; PUT BACK FOR RETURN
000046AE C3                  <1>     retn
                             <1>
                             <1> FS_ERR:
000046AF C605[38770100]01    <1>     mov    byte [DSKETTE_STATUS], BAD_CMD ; UNKNOWN STATE,BAD COMMAND
000046B6 EBEE                <1>     jmp    short S0
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; SET_MEDIA (AH = 18H)
                             <1> ;   THIS ROUTINE SETS THE TYPE OF MEDIA AND DATA RATE
                             <1> ;   TO BE USED FOR THE FOLLOWING FORMAT OPERATION.
                             <1> ;
                             <1> ; ON ENTRY:
                             <1> ;   [BP]   = SECTOR PER TRACK
                             <1> ;   [BP+1] = TRACK #
                             <1> ;   DI     = DRIVE #
                             <1> ;
                             <1> ; ON EXIT:
                             <1> ;   @DSKETTE_STATUS REFLECTS STATUS
                             <1> ;   IF NO ERROR:
                             <1> ;           AH = 0
                             <1> ;           CY = 0
                             <1> ;           ES = SEGMENT OF MEDIA/DRIVE PARAMETER TABLE
                             <1> ;           DI/[BP+6] = OFFSET OF MEDIA/DRIVE PARAMETER TABLE
                             <1> ;   IF ERROR:
                             <1> ;           AH = @DSKETTE_STATUS
                             <1> ;           CY = 1
                             <1> ;--------------------------------------------------------------------------------
                             <1> SET_MEDIA:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
000046B8 E8EC000000          <1>     call   XLAT_NEW              ; TRANSLATE STATE TO PRESENT ARCH.
000046BD F687[43770100]01    <1>         test    byte [DSK_STATE+edi], TRK_CAPA ; CHECK FOR CHANGE LINE AVAILABLE
000046C4 7415                <1>     jz     short SM_CMOS         ; JUMP IF 40 TRACK DRIVE
000046C6 E8DF010000          <1>     call   MED_CHANGE            ; RESET CHANGE LINE
000046CB 803D[38770100]80    <1>     cmp    byte [DSKETTE_STATUS], TIME_OUT ; IF TIME OUT TELL CALLER
000046D2 746A                <1>     je     short SM_RTN
000046D4 C605[38770100]00    <1>     mov    byte [DSKETTE_STATUS], 0 ; CLEAR STATUS
                             <1> SM_CMOS:
000046DB E827050000          <1>     call   CMOS_TYPE             ; RETURN DRIVE TYPE IN (AL)
                             <1>     ;;20/02/2015
                             <1>     ;;jc   short MD_NOT_FND      ; ERROR IN CMOS
                             <1>     ;;or   al, al                ; TEST FOR NO DRIVE
000046E0 745C                <1>     jz     short SM_RTN          ; RETURN IF SO
000046E2 E85C000000          <1>     call   DR_TYPE_CHECK         ; RTN CS:BX = MEDIA/DRIVE PARAM TBL
000046E7 7233                <1>     jc     short MD_NOT_FND      ; TYPE NOT IN TABLE (BAD CMOS)
000046E9 57                  <1>     push   edi                   ; SAVE REG.
000046EA 31DB                <1>     xor    ebx, ebx              ; BX = INDEX TO DR. TYPE TABLE
000046EC B906000000          <1>     mov    ecx, DR_CNT           ; CX = LOOP COUNT
                             <1> DR_SEARCH:
000046F1 8AA3[08660000]      <1>     mov    ah, [DR_TYPE+ebx]     ; GET DRIVE TYPE
000046F7 80E47F              <1>     and    ah, BIT7OFF           ; MASK OUT MSB
000046FA 38E0                <1>     cmp    al, ah                ; DRIVE TYPE MATCH ?
000046FC 7518                <1>     jne    short NXT_MD          ; NO, CHECK NEXT DRIVE TYPE
                             <1> DR_FND:
000046FE 8BBB[09660000]      <1>     mov    edi, [DR_TYPE+ebx+1]  ; DI = MEDIA/DRIVE PARAM TABLE
                             <1> MD_SEARCH:
00004704 8A6704              <1>         mov ah, [edi+MD.SEC_TRK]    ; GET SECTOR/TRACK
                             <1>     ;cmp   [ebp], ah             ; MATCH?
                             <1>     ; 06/08/2022
00004707 38642418            <1>     cmp    [esp+24], ah  ; CL ; spt
0000470B 7509                <1>     jne    short NXT_MD          ; NO, CHECK NEXT MEDIA
0000470D 8A670B              <1>         mov     ah, [edi+MD.MAX_TRK]    ; GET MAX. TRACK #
                             <1>     ;cmp   [ebp+1], ah           ; MATCH?
                             <1>     ; 06/08/2022
00004710 38642419            <1>     cmp    [esp+25], ah  ; CH ; heads - 1
00004714 740F                <1>     je     short MD_FND          ; YES, GO GET RATE
                             <1> NXT_MD:
                             <1>     ;add   bx, 3                 ; CHECK NEXT DRIVE TYPE
00004716 83C305              <1>         add ebx, 5 ; 18/02/2015
00004719 E2D6                <1>     loop   DR_SEARCH
0000471B 5F                  <1>     pop    edi                   ; RESTORE REG.
                             <1> MD_NOT_FND:
0000471C C605[38770100]0C    <1>     mov    byte [DSKETTE_STATUS], MED_NOT_FND ; ERROR, MEDIA TYPE NOT FOUND
00004723 EB19                <1>     jmp    short SM_RTN          ; RETURN
                             <1> MD_FND:
00004725 8A470C              <1>         mov al, [edi+MD.RATE]       ; GET RATE
00004728 3C40                <1>     cmp    al, RATE_300          ; DOUBLE STEP REQUIRED FOR RATE 300
0000472A 7502                <1>     jne    short MD_SET
0000472C 0C20                <1>     or     al, DBL_STEP
                             <1> MD_SET:
                             <1>     ;;mov  [bp+6], di            ; SAVE TABLE POINTER IN STACK
                             <1>     ; 06/08/2022
                             <1>     ;mov   [ebp+12], edi ; 18/02/2015
                             <1>
0000472E 0C10                <1>     or     al, MED_DET           ; SET MEDIA ESTABLISHED
00004730 5F                  <1>     pop    edi
00004731 80A7[43770100]0F    <1>     and    byte [DSK_STATE+edi], ~(MED_DET+DBL_STEP+RATE_MSK) ; CLEAR STATE
00004738 0887[43770100]      <1>     or     [DSK_STATE+edi], al
                             <1>     ;mov   ax, cs                ; SEGMENT OF MEDIA/DRIVE PARAMETER TABLE
                             <1>     ;mov   es, ax                ; ES IS SEGMENT OF TABLE
                             <1> SM_RTN:
                             <1>     ; 06/08/2022
                             <1>     ;call  XLAT_OLD              ; TRANSLATE STATE TO COMPATIBLE MODE
                             <1>     ;call  SETUP_END             ; VARIOUS CLEANUPS
                             <1>     ;retn
                             <1>     ; 06/08/2022
0000473E E93DFDFFFF          <1>     jmp    SETUP_END
                             <1>
                             <1> ;--------------------------------------------------------------
                             <1> ; DR_TYPE_CHECK                                                 :
                             <1> ;   CHECK IF THE GIVEN DRIVE TYPE IN REGISTER (AL)              :
                             <1> ;   IS SUPPORTED IN BIOS DRIVE TYPE TABLE                       :
                             <1> ; ON ENTRY:                                            :
                             <1> ;   AL = DRIVE TYPE                                             :
                             <1> ; ON EXIT:                                             :
                             <1> ;   CY = 0  DRIVE TYPE SUPPORTED                       :
                             <1> ;         EBX = OFFSET TO MEDIA/DRIVE PARAMETER TABLE    :
                             <1> ;   CY = 1  DRIVE TYPE NOT SUPPORTED                   :
                             <1> ; REGISTERS ALTERED: EBX, AH ; 11/07/2022                       :
                             <1> ;--------------------------------------------------------------
                             <1> DR_TYPE_CHECK:
                             <1>     ; 09/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 12/07/2022
                             <1>     ; 11/07/2022
                             <1>     ; 08/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                             <1>     ; 24/12/2021
                             <1>     ;push  eax ; 11/07/2022
                             <1>     ;push  ecx ; 08/07/2022
                             <1>     ;xor   ebx,ebx               ; EBX = INDEX TO DR_TYPE TABLE
00004743 BB[08660000]        <1>     mov    ebx, DR_TYPE
                             <1>     ;;mov  ecx, DR_CNT           ; ECX = LOOP COUNT
                             <1>     ;mov   cl, DR_CNT
00004748 B406                <1>     mov    ah, DR_CNT ; 11/07/2022
                             <1> TYPE_CHK:
```

```
                            <1>     ;;mov   ah, [DR_TYPE+ebx]     ; GET DRIVE TYPE
                            <1>     ;mov    ah, [ebx]
                            <1>     ;cmp    al, ah                ; DRIVE TYPE MATCH?
0000474A 3A03               <1>     cmp     al, [ebx] ; 11/07/2022
0000474C 740E               <1>     je      short DR_TYPE_VALID   ; YES, RETURN WITH CARRY RESET
                            <1>     ; 16/02/2015 (32 bit address modification)
0000474E 83C305             <1>     add     ebx, 5                ; CHECK NEXT DRIVE TYPE
                            <1>     ;loop   TYPE_CHK
                            <1>     ;dec    cl
00004751 FECC               <1>     dec     ah ; 11/07/2022
00004753 75F5               <1>     jnz     short TYPE_CHK
                            <1>     ;
00004755 BB[67660000]       <1>     mov     ebx, MD_TBL6          ; 1.44MB fd parameter table
                            <1>                                   ; Default for GET_PARM (11/12/2014)
                            <1>     ;
0000475A F9                 <1>     stc                           ; DRIVE TYPE NOT FOUND IN TABLE
                            <1>     ;jmp    short TYPE_RTN
                            <1>     ; 12/07/2022
0000475B C3                 <1>     retn
                            <1> DR_TYPE_VALID:
                            <1>     ;mov    ebx, [DR_TYPE+ebx+1]  ; EBX = MEDIA TABLE
0000475C 43                 <1>     inc     ebx
0000475D 8B1B               <1>     mov     ebx, [ebx]
                            <1> TYPE_RTN:
                            <1>     ;pop    ecx ; 08/07/2022
                            <1>     ; 24/12/2021
                            <1>     ;pop    eax ; 11/07/2022
0000475F C3                 <1>     retn
                            <1>
                            <1> ;---------------------------------------------------------------
                            <1> ; SEND_SPEC                                                   :
                            <1> ;   SEND THE SPECIFY COMMAND TO CONTROLLER USING DATA FROM     :
                            <1> ;   THE DRIVE PARAMETER TABLE POINTED BY @DISK_POINTER  :
                            <1> ; ON ENTRY: @DISK_POINTER = DRIVE PARAMETER TABLE              :
                            <1> ; ON EXIT:  NONE                                              :
                            <1> ; REGISTERS ALTERED: CX, DX                                    :
                            <1> ;---------------------------------------------------------------
                            <1> SEND_SPEC:
                            <1>     ; 06/08/2022
00004760 50                 <1>     push    eax                   ; SAVE AX
00004761 B8[87470000]       <1>     mov     eax, SPECBAC          ; LOAD ERROR ADDRESS
00004766 50                 <1>     push    eax                   ; PUSH NEC_OUT ERROR RETURN
00004767 B403               <1>     mov     ah, 03h               ; SPECIFY COMMAND
00004769 E898050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
                            <1>     ;sub    dl, dl                ; FIRST SPECIFY BYTE
                            <1>     ; 06/08/2022
0000476E 28C0               <1>     sub     al, al ; 0
00004770 E89B040000         <1>     call    GET_PARM              ; GET PARAMETER TO AH
00004775 E88C050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
                            <1>     ;mov    dl, 1                 ; SECOND SPECIFY BYTE
                            <1>     ; 06/08/2022
0000477A B001               <1>     mov     al, 1
0000477C E88F040000         <1>     call    GET_PARM              ; GET PARAMETER TO AH
00004781 E880050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
00004786 58                 <1>     pop     eax                   ; POP ERROR RETURN
                            <1> SPECBAC:
00004787 58                 <1>     pop     eax                   ; RESTORE ORIGINAL AX VALUE
00004788 C3                 <1>     retn
                            <1>
                            <1> ;---------------------------------------------------------------
                            <1> ; SEND_SPEC_MD                                                 :
                            <1> ;   SEND THE SPECIFY COMMAND TO CONTROLLER USING DATA FROM     :
                            <1> ;   THE MEDIA/DRIVE PARAMETER TABLE POINTED BY (CS:BX)  :
                            <1> ; ON ENTRY: CS:BX = MEDIA/DRIVE PARAMETER TABLE         :
                            <1> ; ON EXIT:  NONE                                        :
                            <1> ; REGISTERS ALTERED: AX                                        :
                            <1> ;---------------------------------------------------------------
                            <1> SEND_SPEC_MD:
00004789 50                 <1>     push    eax                   ; SAVE RATE DATA
0000478A B8[A7470000]       <1>     mov     eax, SPEC_ESBAC               ; LOAD ERROR ADDRESS
0000478F 50                 <1>     push    eax                   ; PUSH NEC_OUT ERROR RETURN
00004790 B403               <1>     mov     ah, 03h               ; SPECIFY COMMAND
00004792 E86F050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
00004797 8A23               <1>         mov     ah, [ebx+MD.SPEC1]      ; GET 1ST SPECIFY BYTE
00004799 E868050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
0000479E 8A6301             <1>         mov     ah, [ebx+MD.SPEC2]      ; GET SECOND SPECIFY BYTE
000047A1 E860050000         <1>     call    NEC_OUTPUT            ; OUTPUT THE COMMAND
000047A6 58                 <1>     pop     eax                   ; POP ERROR RETURN
                            <1> SPEC_ESBAC:
000047A7 58                 <1>     pop     eax                   ; RESTORE ORIGINAL AX VALUE
000047A8 C3                 <1>     retn
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; XLAT_NEW
                            <1> ;   TRANSLATES DISKETTE STATE LOCATIONS FROM COMPATIBLE
                            <1> ;   MODE TO NEW ARCHITECTURE.
                            <1> ;
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ;-----------------------------------------------------------------------------
                            <1> XLAT_NEW:
                            <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                            <1>     ; 11/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
000047A9 83FF01             <1>     cmp     edi, 1                ; VALID DRIVE
000047AC 7709               <1>     ja      short XN_OUT          ; IF INVALID BACK
000047AE 80BF[43770100]00   <1>     cmp     byte [DSK_STATE+edi], 0       ; NO DRIVE ?
000047B5 7401               <1>     jz      short DO_DET          ; IF NO DRIVE ATTEMPT DETERMINE
                            <1>
                            <1>     ;;mov   cx, di                ; CX = DRIVE NUMBER
                            <1>     ;mov    ecx, edi
                            <1>     ;or     cl, cl
                            <1>     ;jz     short XN_0
                            <1>     ;shl    cl, 2                 ; CL = SHIFT COUNT, A=0, B=4
                            <1>     ;mov    al, [HF_CNTRL]        ; DRIVE INFORMATION
                            <1>     ;ror    al, cl                ; TO LOW NIBBLE
                            <1> ;XN_0:
                            <1>     ;and    al, DRV_DET+FMT_CAPA+TRK_CAPA ; KEEP DRIVE BITS
                            <1>         ;and       byte [DSK_STATE+edi], ~(DRV_DET+FMT_CAPA+TRK_CAPA)
                            <1>     ;or     [DSK_STATE+edi], al   ; UPDATE DRIVE STATE
                            <1> XN_OUT:
000047B7 C3                 <1>     retn
                            <1>
                            <1> DO_DET:
                            <1>     ;call   DRIVE_DET             ; TRY TO DETERMINE
                            <1>     ;retn
                            <1>     ;jmp    DRIVE_DET
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; DRIVE_DET
                            <1> ;   DETERMINES WHETHER DRIVE IS 80 OR 40 TRACKS AND
                            <1> ;   UPDATES STATE INFORMATION ACCORDINGLY.
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ;-----------------------------------------------------------------------------
                            <1> DRIVE_DET:
                            <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
```

```
                                <1>     ; 08/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
000047B8 E88E040000             <1>     call    MOTOR_ON                ; TURN ON MOTOR IF NOT ALREADY ON
000047BD E8FA050000             <1>     call    RECAL                   ; RECALIBRATE DRIVE
000047C2 724E                   <1>     jc      short DD_BAC            ; ASSUME NO DRIVE PRESENT
000047C4 B530                   <1>     mov     ch, TRK_SLAP            ; SEEK TO TRACK 48
000047C6 E872050000             <1>     call    SEEK
000047CB 7245                   <1>     jc      short DD_BAC            ; ERROR NO DRIVE
000047CD B50B                   <1>     mov     ch, QUIET_SEEK+1        ; SEEK TO TRACK 10
                                <1> SK_GIN:
000047CF FECD                   <1>     dec     ch                      ; DECREMENT TO NEXT TRACK
                                <1>     ;push   cx                      ; SAVE TRACK
                                <1>     ; 11/04/2021
000047D1 51                     <1>     push    ecx
000047D2 E866050000             <1>     call    SEEK
000047D7 723A                   <1>     jc      short POP_BAC           ; POP AND RETURN
000047D9 B8[13480000]           <1>     mov     eax, POP_BAC            ; LOAD NEC OUTPUT ERROR ADDRESS
000047DE 50                     <1>     push    eax
000047DF B404                   <1>     mov     ah, SENSE_DRV_ST        ; SENSE DRIVE STATUS COMMAND BYTE
000047E1 E820050000             <1>     call    NEC_OUTPUT              ; OUTPUT TO NEC
                                <1>     ;mov    ax, di                  ; AL = DRIVE
                                <1>     ; 06/08/2022
000047E6 89F8                   <1>     mov     eax, edi
000047E8 88C4                   <1>     mov     ah, al                  ; AH = DRIVE
000047EA E817050000             <1>     call    NEC_OUTPUT              ; OUTPUT TO NEC
000047EF E849060000             <1>     call    RESULTS                 ; GO GET STATUS
000047F4 58                     <1>     pop     eax                     ; THROW AWAY ERROR ADDRESS
                                <1>     ;pop    cx                      ; RESTORE TRACK
                                <1>     ; 11/04/2021
000047F5 59                     <1>     pop     ecx
000047F6 F605[39770100]10       <1>     test    byte [NEC_STATUS], HOME       ; TRACK 0 ?
000047FD 74D0                   <1>     jz      short SK_GIN            ; GO TILL TRACK 0
000047FF 08ED                   <1>     or      ch, ch                  ; IS HOME AT TRACK 0
00004801 7408                   <1>     jz      short IS_80             ; MUST BE 80 TRACK DRIVE
                                <1>
                                <1> ;   DRIVE IS A 360; SET DRIVE TO DETERMINED;
                                <1> ;   SET MEDIA TO DETERMINED AT RATE 250.
                                <1>
00004803 808F[43770100]94       <1>     or      byte [DSK_STATE+edi], DRV_DET+MED_DET+RATE_250
0000480A C3                     <1>     retn                            ; ALL INFORMATION SET
                                <1> IS_80:
0000480B 808F[43770100]01       <1>     or      byte [DSK_STATE+edi], TRK_CAPA ; SETUP 80 TRACK CAPABILITY
                                <1> DD_BAC:
00004812 C3                     <1>     retn
                                <1> POP_BAC:
                                <1>     ;pop    cx                      ; THROW AWAY
                                <1>     ; 11/04/2021
00004813 59                     <1>     pop     ecx
00004814 C3                     <1>     retn
                                <1>
                                <1>
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>
                                <1> %if 0
                                <1>
                                <1> ;--------------------------------------------------------------------------------
                                <1> ; XLAT_OLD
                                <1> ;   TRANSLATES DISKETTE STATE LOCATIONS FROM NEW
                                <1> ;   ARCHITECTURE TO COMPATIBLE MODE.
                                <1> ;
                                <1> ; ON ENTRY: DI = DRIVE
                                <1> ;--------------------------------------------------------------------------------
                                <1> XLAT_OLD:
                                <1>     cmp     edi, 1                  ; VALID DRIVE ?
                                <1>     ja      short XO_OUT              ; IF INVALID BACK
                                <1>         cmp byte [DSK_STATE+edi], 0       ; NO DRIVE ?
                                <1>     jz      short XO_OUT            ; IF NO DRIVE TRANSLATE DONE
                                <1>
                                <1> ;-----      TEST FOR SAVED DRIVE INFORMATION ALREADY SET
                                <1>
                                <1>     ;mov    cx, di                  ; CX = DRIVE NUMBER
                                <1>     ; 06/08/2022
                                <1>     mov     ecx, edi
                                <1>     shl     cl, 2                   ; CL = SHIFT COUNT, A=0, B=4
                                <1>     mov     ah, FMT_CAPA            ; LOAD MULTIPLE DATA RATE BIT MASK
                                <1>     ror     ah, cl                  ; ROTATE BY MASK
                                <1>     test    [HF_CNTRL], ah          ; MULTIPLE-DATA RATE DETERMINED ?
                                <1>     jnz     short SAVE_SET          ; IF SO, NO NEED TO RE-SAVE
                                <1>
                                <1> ;-----      ERASE DRIVE BITS IN @HF_CNTRL FOR THIS DRIVE
                                <1>
                                <1>     mov     ah, DRV_DET+FMT_CAPA+TRK_CAPA ; MASK TO KEEP
                                <1>     ror     ah, cl                  ; FIX MASK TO KEEP
                                <1>     not     ah                      ; TRANSLATE MASK
                                <1>     and     [HF_CNTRL], ah          ; KEEP BITS FROM OTHER DRIVE INTACT
                                <1>
                                <1> ;-----      ACCESS CURRENT DRIVE BITS AND STORE IN @HF_CNTRL
                                <1>
                                <1>     mov     al, [DSK_STATE+edi]     ; ACCESS STATE
                                <1>     and     al, DRV_DET+FMT_CAPA+TRK_CAPA ; KEEP DRIVE BITS
                                <1>     ror     al, cl                  ; FIX FOR THIS DRIVE
                                <1>     or      [HF_CNTRL], al          ; UPDATE SAVED DRIVE STATE
                                <1>
                                <1> ;-----      TRANSLATE TO COMPATIBILITY MODE
                                <1>
                                <1> SAVE_SET:
                                <1>     mov     ah, [DSK_STATE+edi]     ; ACCESS STATE
                                <1>     mov     bh, ah                  ; TO BH FOR LATER
                                <1>     and     ah, RATE_MSK            ; KEEP ONLY RATE
                                <1>     cmp     ah, RATE_500            ; RATE 500 ?
                                <1>     jz      short CHK_144           ; YES 1.2/1.2 OR 1.44/1.44
                                <1>     mov     al, M3D1U               ; AL = 360 IN 1.2 UNESTABLISHED
                                <1>     cmp     ah, RATE_300            ; RATE 300 ?
                                <1>     jnz     short CHK_250           ; NO, 360/360, 720/720 OR 720/1.44
                                <1>     test    bh, DBL_STEP            ; CHECK FOR DOUBLE STEP
                                <1>     jnz     short TST_DET           ; MUST BE 360 IN 1.2
                                <1> UNKNO:
                                <1>     mov     al, MED_UNK             ; NONE OF THE ABOVE
                                <1>     jmp     short AL_SET            ; PROCESS COMPLETE
                                <1> CHK_144:
                                <1>     call    CMOS_TYPE               ; RETURN DRIVE TYPE IN (AL)
                                <1>     ;;20/02/2015
                                <1>     ;;jc    short UNKNO             ; ERROR, SET 'NONE OF ABOVE'
                                <1>     jz      short UNKNO ;; 20/02/2015
                                <1>     cmp     al, 2                   ; 1.2MB DRIVE ?
                                <1>     jne     short UNKNO             ; NO, GO SET 'NONE OF ABOVE'
                                <1>     mov     al, M1D1U               ; AL = 1.2 IN 1.2 UNESTABLISHED
                                <1>     jmp     short TST_DET
                                <1> CHK_250:
                                <1>     mov     al, M3D3U               ; AL = 360 IN 360 UNESTABLISHED
                                <1>     cmp     ah, RATE_250            ; RATE 250 ?
                                <1>     jnz     short UNKNO             ; IF SO FALL IHRU
                                <1>     test    bh, TRK_CAPA            ; 80 TRACK CAPABILITY ?
                                <1>     jnz     short UNKNO             ; IF SO JUMP, FALL THRU TEST DET
                                <1> TST_DET:
```

```
                                <1>     test    bh, MED_DET           ; DETERMINED ?
                                <1>     jz      short AL_SET          ; IF NOT THEN SET
                                <1>     add     al, 3                 ; MAKE DETERMINED/ESTABLISHED
                                <1> AL_SET:
                                <1>     and     byte [DSK_STATE+edi], ~(DRV_DET+FMT_CAPA+TRK_CAPA) ; CLEAR DRIVE
                                <1>     or      [DSK_STATE+edi], al   ; REPLACE WITH COMPATIBLE MODE
                                <1> XO_OUT:
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> ;------------------------------------------------------------------------------
                                <1> ; SETUP_STATE:     INITIALIZES START AND END RATES.
                                <1> ;------------------------------------------------------------------------------
                                <1> SETUP_STATE:
00004815 F687[43770100]10       <1>     test    byte [DSK_STATE+edi], MED_DET ; MEDIA DETERMINED ?
0000481C 7537                   <1>     jnz     short J1C             ; NO STATES IF DETERMINED
0000481E 66B84000               <1>         mov     ax, (RATE_500*256)+RATE_300 ; AH = START RATE, AL = END RATE
00004822 F687[43770100]04       <1>     test    byte [DSK_STATE+edi], DRV_DET ; DRIVE ?
00004829 740D                   <1>     jz      short AX_SET          ; DO NOT KNOW DRIVE
0000482B F687[43770100]02       <1>     test    byte [DSK_STATE+edi], FMT_CAPA ; MULTI-RATE?
00004832 7504                   <1>     jnz     short AX_SET          ; JUMP IF YES
00004834 66B88080               <1>         mov     ax, RATE_250*257  ; START A END RATE 250 FOR 360 DRIVE
                                <1> AX_SET:
00004838 80A7[43770100]1F       <1>     and     byte [DSK_STATE+edi], ~(RATE_MSK+DBL_STEP) ; TURN OFF THE RATE
0000483F 08A7[43770100]         <1>     or      [DSK_STATE+edi], ah   ; RATE FIRST TO TRY
00004845 8025[40770100]F3       <1>     and     byte [LASTRATE], ~STRT_MSK ; ERASE LAST TO TRY RATE BITS
0000484C C0C804                 <1>     ror     al, 4                 ; TO OPERATION LAST RATE LOCATION
0000484F 0805[40770100]         <1>     or      [LASTRATE], al        ; LAST RATE
                                <1> J1C:
00004855 C3                     <1>     retn
                                <1>
                                <1> ;------------------------------------------------------------------------------
                                <1> ;  FMT_INIT: ESTABLISH STATE IF UNESTABLISHED AT FORMAT TIME.
                                <1> ;------------------------------------------------------------------------------
                                <1> FMT_INIT:
00004856 F687[43770100]10       <1>     test    byte [DSK_STATE+edi], MED_DET ; IS MEDIA ESTABLISHED
0000485D 7546                   <1>     jnz     short F1_OUT          ; IF SO RETURN
0000485F E8A3030000             <1>     call    CMOS_TYPE             ; RETURN DRIVE TYPE IN AL
                                <1>     ;; 20/02/2015
                                <1>     ;;jc    short CL_DRV          ; ERROR IN CMOS ASSUME NO DRIVE
00004864 7440                   <1>     jz      short CL_DRV ;; 20/02/2015
00004866 FEC8                   <1>     dec     al                    ; MAKE ZERO ORIGIN
                                <1>     ;;JS    short CL_DRV          ; NO DRIVE IF AL 0
00004868 8AA7[43770100]         <1>     mov     ah, [DSK_STATE+edi]   ; AH = CURRENT STATE
0000486E 80E40F                 <1>     and     ah, ~(MED_DET+DBL_STEP+RATE_MSK) ; CLEAR
00004871 08C0                   <1>     or      al, al                ; CHECK FOR 360
00004873 7505                   <1>     jnz     short N_360           ; IF 360 WILL BE 0
00004875 80CC90                 <1>     or      ah, MED_DET+RATE_250  ; ESTABLISH MEDIA
00004878 EB25                   <1>     jmp     short SKP_STATE               ; SKIP OTHER STATE PROCESSING
                                <1> N_360:
0000487A FEC8                   <1>     dec     al                    ; 1.2 M DRIVE
0000487C 7505                   <1>     jnz     short N_12            ; JUMP IF NOT
                                <1> F1_RATE:
0000487E 80CC10                 <1>     or      ah, MED_DET+RATE_500  ; SET FORMAT RATE
00004881 EB1C                   <1>     jmp     short SKP_STATE               ; SKIP OTHER STATE PROCESSING
                                <1> N_12:
00004883 FEC8                   <1>     dec     al                    ; CHECK FOR TYPE 3
00004885 750F                   <1>     jnz     short N_720           ; JUMP IF NOT
00004887 F6C404                 <1>     test    ah, DRV_DET           ; IS DRIVE DETERMINED
0000488A 7410                   <1>     jz      short ISNT_12         ; TREAT AS NON 1.2 DRIVE
0000488C F6C402                 <1>     test    ah, FMT_CAPA          ; IS 1.2M
0000488F 740B                   <1>     jz      short ISNT_12         ; JUMP IF NOT
00004891 80CC50                 <1>     or      ah, MED_DET+RATE_300  ; RATE 300
00004894 EB09                   <1>     jmp     short SKP_STATE               ; CONTINUE
                                <1> N_720:
00004896 FEC8                   <1>     dec     al                    ; CHECK FOR TYPE 4
00004898 750C                   <1>     jnz     short CL_DRV          ; NO DRIVE, CMOS BAD
0000489A EBE2                   <1>     jmp     SHORT F1_RATE
                                <1> ISNT_12:
0000489C 80CC90                 <1>     or      ah, MED_DET+RATE_250  ; MUST BE RATE 250
                                <1> SKP_STATE:
0000489F 88A7[43770100]         <1>     mov     [DSK_STATE+edi], ah   ; STORE AWAY
                                <1> F1_OUT:
000048A5 C3                     <1>     retn
                                <1> CL_DRV:
000048A6 30E4                   <1>     xor     ah, ah                ; CLEAR STATE
000048A8 EBF5                   <1>     jmp     short SKP_STATE               ; SAVE IT
                                <1>
                                <1> ;------------------------------------------------------------------------------
                                <1> ; MED_CHANGE
                                <1> ;   CHECKS FOR MEDIA CHANGE, RESETS MEDIA CHANGE,
                                <1> ;   CHECKS MEDIA CHANGE AGAIN.
                                <1> ;
                                <1> ; ON EXIT:  CY = 1 MEANS MEDIA CHANGE OR TIMEOUT
                                <1> ;           @DSKETTE_STATUS = ERROR CODE
                                <1> ;------------------------------------------------------------------------------
                                <1> MED_CHANGE:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
000048AA E8E8050000             <1>     call    READ_DSKCHNG          ; READ DISK CHANCE LINE STATE
000048AF 7445                   <1>     jz      short MC_OUT          ; BYPASS HANDLING DISK CHANGE LINE
000048B1 80A7[43770100]EF       <1>     and     byte [DSK_STATE+edi], ~MED_DET ; CLEAR STATE FOR THIS DRIVE
                                <1>
                                <1> ;   THIS SEQUENCE ENSURES WHENEVER A DISKETTE IS CHANGED THAT
                                <1> ;   ON THE NEXT OPERATION THE REQUIRED MOTOR START UP TIME WILL
                                <1> ;   BE WAITED. (DRIVE MOTOR MAY GO OFF UPON DOOR OPENING).
                                <1>
                                <1>     ;mov    cx, di                ; CL = DRIVE 0
                                <1>     ; 06/08/2022
000048B8 89F9                   <1>     mov     ecx, edi
000048BA B001                   <1>     mov     al, 1                 ; MOTOR ON BIT MASK
000048BC D2E0                   <1>     shl     al, cl                ; TO APPROPRIATE POSITION
000048BE F6D0                   <1>     not     al                    ; KEEP ALL BUT MOTOR ON
000048C0 FA                     <1>     cli                           ; NO INTERRUPTS
000048C1 2005[36770100]         <1>     and     [MOTOR_STATUS], al    ; TURN MOTOR OFF INDICATOR
000048C7 FB                     <1>     sti                           ; INTERRUPTS ENABLED
000048C8 E87E030000             <1>     call    MOTOR_ON              ; TURN MOTOR ON
                                <1>
                                <1> ;-----      THIS SEQUENCE OF SEEKS IS USED TO RESET DISKETTE CHANGE SIGNAL
                                <1>
000048CD E82FFAFFFF             <1>     call    DSK_RESET             ; RESET NEC
000048D2 B501                   <1>     mov     ch, 1                 ; MOVE TO CYLINDER 1
000048D4 E864040000             <1>     call    SEEK                  ; ISSUE SEEK
000048D9 30ED                   <1>     xor     ch, ch                ; MOVE TO CYLINDER 0
000048DB E85D040000             <1>     call    SEEK                  ; ISSUE SEEK
000048E0 C605[38770100]06       <1>     mov     byte [DSKETTE_STATUS], MEDIA_CHANGE ; STORE IN STATUS
                                <1> OK1:
000048E7 E8AB050000             <1>     call    READ_DSKCHNG          ; CHECK MEDIA CHANGED AGAIN
000048EC 7407                   <1>     jz      short OK2             ; IF ACTIVE, NO DISKETTE, TIMEOUT
                                <1> OK4:
000048EE C605[38770100]80       <1>     mov     byte [DSKETTE_STATUS], TIME_OUT ; TIMEOUT IF DRIVE EMPTY
                                <1> OK2:
000048F5 F9                     <1>     stc                           ; MEDIA CHANGED, SET CY
                                <1> MC_OUT:     ; 06/08/2022 (cf = 0)
```

```
000048F6 C3                 <1>     retn
                            <1> ;MC_OUT:
                            <1>     ;clc                        ; NO MEDIA CHANGED, CLEAR CY
                            <1>     ;retn
                            <1>
                            <1> ;-------------------------------------------------------------------------------
                            <1> ; SEND_RATE
                            <1> ;   SENDS DATA RATE COMMAND TO NEC
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ; ON EXIT:  NONE
                            <1> ; REGISTERS ALTERED: DX
                            <1> ;-------------------------------------------------------------------------------
                            <1> SEND_RATE:
                            <1>     ;push   ax                  ; SAVE REG.
                            <1>     ; 11/04/2021
000048F7 50                 <1>     push    eax
000048F8 8025[40770100]3F   <1>     and     byte [LASTRATE], ~SEND_MSK ; ELSE CLEAR LAST RATE ATTEMPTED
000048FF 8A87[43770100]     <1>     mov     al, [DSK_STATE+edi]   ; GET RATE STATE OF THIS DRIVE
00004905 24C0               <1>     and     al, SEND_MSK          ; KEEP ONLY RATE BITS
00004907 0805[40770100]     <1>     or      [LASTRATE], al        ; SAVE NEW RATE FOR NEXT CHECK
0000490D C0C002             <1>     rol     al, 2                 ; MOVE TO BIT OUTPUT POSITIONS
00004910 66BAF703           <1>     mov     dx, 03F7h             ; OUTPUT NEW DATA RATE
00004914 EE                 <1>     out     dx, al
                            <1>     ;pop    ax                  ; RESTORE REG.
                            <1>     ; 11/04/2021
00004915 58                 <1>     pop     eax
00004916 C3                 <1>     retn
                            <1>
                            <1> ;-------------------------------------------------------------------------------
                            <1> ; CHK_LASTRATE
                            <1> ;   CHECK PREVIOUS DATE RATE SNT TO THE CONTROLLER.
                            <1> ; ON ENTRY:
                            <1> ;   DI = DRIVE #
                            <1> ; ON EXIT:
                            <1> ;   ZF =  1 DATA RATE IS THE SAME AS THE LAST RATE SENT TO NEC
                            <1> ;   ZF =  0 DATA RATE IS DIFFERENT FROM LAST RATE
                            <1> ; REGISTERS ALTERED: DX
                            <1> ;-------------------------------------------------------------------------------
                            <1> CHK_LASTRATE:
                            <1>     ; 13/07/2022 - TRDOS 386 v2.0.5
                            <1>     ;push   ax                  ; SAVE REG.
                            <1>     ; 11/04/2021
00004917 50                 <1>     push    eax
                            <1>     ; 13/07/2022 (BugFix)
00004918 8A25[40770100]     <1>     mov     ah, [LASTRATE]              ; GET LAST DATA RATE SELECTED
0000491E 8A87[43770100]     <1>     mov     al, [DSK_STATE+edi]   ; GET RATE STATE OF THIS DRIVE
00004924 6625C0C0           <1>         and ax, SEND_MSK*257        ; KEEP ONLY RATE BITS OF BOTH
00004928 38E0               <1>     cmp     al, ah                ; COMPARE TO PREVIOUSLY TRIED
                            <1>                                   ; ZF = 1 RATE IS THE SAME
                            <1>     ;pop    ax                  ; RESTORE REG.
                            <1>     ; 11/04/2021
0000492A 58                 <1>     pop     eax
0000492B C3                 <1>     retn
                            <1>
                            <1> ;-------------------------------------------------------------------------------
                            <1> ; DMA_SETUP
                            <1> ;   THIS ROUTINE SETS UP THE DMA FOR READ/WRITE/VERIFY OPERATIONS.
                            <1> ;
                            <1> ; ON ENTRY: AL = DMA COMMAND
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                            <1> ;-------------------------------------------------------------------------------
                            <1>
                            <1> ; SI = Head #, # of Sectors or DASD Type
                            <1>
                            <1> ; 22/08/2015
                            <1> ; 08/02/2015 - Protected Mode Modification
                            <1> ; 06/02/2015 - 07/02/2015
                            <1> ; NOTE: Buffer address must be in 1st 16MB of Physical Memory (24 bit limit).
                            <1> ; (DMA Addres = Physical Address)
                            <1> ; (Retro UNIX 386 v1 Kernel/System Mode Virtual Address = Physical Address)
                            <1> ;
                            <1> ; 09/08/2022
                            <1> ; 06/08/2022
                            <1> ; 04/02/2016 (clc)
                            <1> ; 20/02/2015 modification (source: AWARD BIOS 1999, DMA_SETUP)
                            <1> ; 16/12/2014 (IODELAY)
                            <1>
                            <1> DMA_SETUP:
                            <1>     ; 09/08/2022
                            <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                            <1>     ;; 20/02/2015
                            <1>     ;;mov   edx, [ebp+4]          ; Buffer address
                            <1>     ; 06/08/2022
                            <1>     ;mov    edx, ebp ; buffer address
                            <1>     ;test   edx, 0FF000000h             ; 16 MB limit (22/08/2015, bugfix)
0000492C F7C5000000FF       <1>     test    ebp, 0FF000000h
00004932 7566               <1>     jnz     short dma_bnd_err_stc
                            <1>     ; 09/08/2022
00004934 89EA               <1>     mov     edx, ebp
                            <1>     ;
                            <1>     ;;push ax                   ; DMA command
                            <1>     ; 11/04/2021
00004936 50                 <1>     push    eax
                            <1>     ; 06/08/2022
                            <1>     ;push   edx                 ; *
                            <1>     ;mov    dl, 3               ; GET BYTES/SECTOR PARAMETER
                            <1>     ; 06/08/2022
00004937 B003               <1>     mov     al, 3               ; GET BYTES/SECTOR PARAMETER
00004939 E8D2020000         <1>     call    GET_PARM            ;
0000493E 88E1               <1>     mov     cl, ah              ; SHIFT COUNT (0=128, 1=256, 2=512 ETC)
                            <1>     ;mov    ax, si              ; Sector count
                            <1>     ; 06/08/2022
00004940 89F0               <1>     mov     eax, esi
00004942 88C4               <1>     mov     ah, al              ; AH = # OF SECTORS
00004944 28C0               <1>     sub     al, al              ; AL = 0, AX = # SECTORS * 256
                            <1>     ;shr    ax, 1               ; AX = # SECTORS * 128
                            <1>     ; 06/08/2022
00004946 D1E8               <1>     shr     eax, 1
                            <1>     ;shl    ax, cl              ; SHIFT BY PARAMETER VALUE
                            <1>     ;dec    ax                  ; -1 FOR DMA VALUE
                            <1>     ;mov    cx, ax
00004948 D3E0               <1>     shl     eax, cl
0000494A 48                 <1>     dec     eax
0000494B 89C1               <1>     mov     ecx, eax
                            <1>     ; 06/08/2022
                            <1>     ;pop    edx                 ; *
                            <1>     ;;pop   ax
                            <1>     ; 11/04/2021
0000494D 58                 <1>     pop     eax
0000494E 3C42               <1>     cmp     al, 42h
00004950 7507               <1>         jne     short NOT_VERF
                            <1>     ; 09/08/2022
00004952 BA0000FF00         <1>     mov     edx, 0FF0000h
```

```
                                <1>     ; 06/08/2022
                                <1>     ;mov    ebp, 0FF0000h
00004957 EB08                   <1>     jmp     short J33
                                <1> NOT_VERF:
00004959 6601CA                 <1>     add     dx, cx                  ; check for overflow
0000495C 723D                   <1>     jc      short dma_bnd_err
                                <1>     ;
0000495E 6629CA                 <1>     sub     dx, cx                  ; Restore start address
                                <1> J33:
00004961 FA                     <1>     cli                             ; DISABLE INTERRUPTS DURING DMA SET-UP
00004962 E60C                   <1>     out     DMA+12, al              ; SET THE FIRST/LA5T F/F
                                <1>     IODELAY                         ; WAIT FOR I/O
  78 00004964 EB00              <2>  jmp short $+2
  79 00004966 EB00              <2>  jmp short $+2
00004968 E60B                   <1>     out     DMA+11, al              ; OUTPUT THE MODE BYTE
                                <1>     ;mov    eax, edx                ; Buffer address
                                <1>     ; 06/08/2022
                                <1>     ;mov    eax, ebp
                                <1>     ; 09/08/2022
0000496A 89D0                   <1>     mov     eax, edx
0000496C E604                   <1>     out     DMA+4, al               ; OUTPUT LOW ADDRESS
                                <1>     IODELAY                         ; WAIT FOR I/O
  78 0000496E EB00              <2>  jmp short $+2
  79 00004970 EB00              <2>  jmp short $+2
00004972 88E0                   <1>     mov     al, ah
00004974 E604                   <1>     out     DMA+4, al               ; OUTPUT HIGH ADDRESS
00004976 C1E810                 <1>     shr     eax, 16
                                <1>     IODELAY                         ; I/O WAIT STATE
  78 00004979 EB00              <2>  jmp short $+2
  79 0000497B EB00              <2>  jmp short $+2
0000497D E681                   <1>     out     081h, al                ; OUTPUT HIGHEST BITS TO PAGE REGISTER
                                <1>     IODELAY
  78 0000497F EB00              <2>  jmp short $+2
  79 00004981 EB00              <2>  jmp short $+2
                                <1>     ;mov    ax, cx                  ; Byte count - 1
                                <1>     ; 06/08/2022
00004983 89C8                   <1>     mov     eax, ecx
00004985 E605                   <1>     out     DMA+5, al               ; LOW BYTE OF COUNT
                                <1>     IODELAY                         ; WAIT FOR I/O
  78 00004987 EB00              <2>  jmp short $+2
  79 00004989 EB00              <2>  jmp short $+2
0000498B 88E0                   <1>     mov     al, ah
0000498D E605                   <1>     out     DMA+5, al               ; HIGH BYTE OF COUNT
                                <1>     IODELAY
  78 0000498F EB00              <2>  jmp short $+2
  79 00004991 EB00              <2>  jmp short $+2
00004993 FB                     <1>     sti                             ; RE-ENABLE INTERRUPTS
00004994 B002                   <1>     mov     al, 2                   ; MODE FOR 8237
00004996 E60A                   <1>     out     DMA+10, al              ; INITIALIZE THE DISKETTE CHANNEL
                                <1>
00004998 F8                     <1>     clc     ; 04/02/2016
00004999 C3                     <1>     retn
                                <1>
                                <1> dma_bnd_err_stc:
0000499A F9                     <1>     stc
                                <1> dma_bnd_err:
0000499B C605[38770100]09       <1>     mov     byte [DSKETTE_STATUS], DMA_BOUNDARY ; SET ERROR
000049A2 C3                     <1>     retn                            ; CY SET BY ABOVE IF ERROR
                                <1>
                                <1> ;; 16/12/2014
                                <1> ;;  CLI                             ; DISABLE INTERRUPTS DURING DMA SET-UP
                                <1> ;;  OUT     DMA+12,AL               ; SET THE FIRST/LA5T F/F
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  OUT     DMA+11,AL               ; OUTPUT THE MODE BYTE
                                <1> ;;  ;SIODELAY
                                <1> ;;      ;cmp        AL,42H                  ; DMA VERIFY COMMAND
                                <1> ;;      ;JNE        short NOT_VERF          ; NO
                                <1> ;;      ;xor        AX,AX                   ; START ADDRESS
                                <1> ;;      ;jmp        SHORT J33
                                <1> ;;;NOT_VERF:
                                <1> ;;  ;mov    AX,ES                   ; GET THE ES VALUE
                                <1> ;;  ;ROL    AX,4                    ; ROTATE LEFT
                                <1> ;;  ;mov    CH,AL                   ; GET HIGHEST NIBBLE OF ES TO CH
                                <1> ;;  ;and    AL,11110000B            ; ZERO THE LOW NIBBLE FROM SEGMENT
                                <1> ;;  ;add    AX,[BP+2]               ; TEST FOR CARRY FROM ADDITION
                                <1> ;;  mov     eax,[ebp+4] ; 06/02/2015
                                <1> ;;  ;jnc    short J33
                                <1> ;;  ;inc    CH                      ; CARRY MEANS HIGH 4 BITS MUST BE INC
                                <1> ;;;J33:
                                <1> ;;  push    eax                     ; SAVE START ADDRESS
                                <1> ;;  OUT     DMA+4,AL                ; OUTPUT LOW ADDRESS
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  mov     AL,AH
                                <1> ;;  OUT     DMA+4,AL                ; OUTPUT HIGH ADDRESS
                                <1> ;;  shr     eax, 16      ; 07/02/2015
                                <1> ;;  ;mov    AL,CH                   ; GET HIGH 4 BITS
                                <1> ;;  ;jmp    $+2                     ; I/O WAIT STATE
                                <1> ;;  IODELAY
                                <1> ;;  ;and    AL,00001111B
                                <1> ;;  OUT     081H,AL                 ; OUTPUT HIGH 4 BITS TO PAGE REGISTER
                                <1> ;;  ;SIODELAY
                                <1> ;;
                                <1> ;;;----- DETERMINE COUNT
                                <1> ;;  sub     eax, eax ; 08/02/2015
                                <1> ;;  mov     AX,SI                   ; AL =  # OF SECTORS
                                <1> ;;  xchg    AL,AH                   ; AH =  # OF SECTORS
                                <1> ;;  sub     AL,AL                   ; AL = 0, AX = # SECTORS * 256
                                <1> ;;  SHR     AX,1                    ; AX = # SECTORS * 128
                                <1> ;;  push    AX                      ; SAVE # OF SECTORS * 128
                                <1> ;;  mov     DL,3                    ; GET BYTES/SECTOR PARAMETER
                                <1> ;;  call    GET_PARM                ; "
                                <1> ;;  mov     CL,AH                   ; SHIFT COUNT (0=128, 1=256, 2=512 ETC)
                                <1> ;;  pop     AX                      ; AX = # SECTORS * 128
                                <1> ;;  SHL     AX,CL                   ; SHIFT BY PARAMETER VALUE
                                <1> ;;  dec     AX                      ; -1 FOR DMA VALUE
                                <1> ;;  push    eax  ; 08/02/2015       ; SAVE COUNT VALUE
                                <1> ;;  OUT     DMA+5,AL                ; LOW BYTE OF COUNT
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  mov     AL,AH
                                <1> ;;  OUT     DMA+5,AL                ; HIGH BYTE OF COUNT
                                <1> ;;  ;IODELAY
                                <1> ;;  STI                             ; RE-ENABLE INTERRUPTS
                                <1> ;;  pop     ecx  ; 08/02/2015       ; RECOVER COUNT VALUE
                                <1> ;;  pop     eax  ; 08/02/2015       ; RECOVER ADDRESS VALUE
                                <1> ;;  ;add    AX, CX                  ; ADD, TEST FOR 64K OVERFLOW
                                <1> ;;  add     ecx,eax ; 08/02/2015
                                <1> ;;  mov     AL, 2                   ; MODE FOR 8237
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  SIODELAY
                                <1> ;;  OUT     DMA+10,AL               ; INITIALIZE THE DISKETTE CHANNEL
                                <1> ;;  ;jnc    short NO_BAD            ; CHECK FOR ERROR
```

```
                                <1> ;;  jc      short dma_bnd_err ; 08/02/2015
                                <1> ;;  and     ecx,0FFF00000h ; 16 MB limit
                                <1> ;;  jz      short NO_BAD
                                <1> ;;dma_bnd_err:
                                <1> ;;  mov     byte [DSKETTE_STATUS],DMA_BOUNDARY ; SET ERROR
                                <1> ;;NO_BAD:
                                <1> ;;  retn                            ; CY SET BY ABOVE IF ERROR
                                <1>
                                <1> ;--------------------------------------------------------------------------------
                                <1> ; FMTDMA_SET
                                <1> ;   THIS ROUTINE SETS UP THE DMA CONTROLLER FOR A FORMAT OPERATION.
                                <1> ;
                                <1> ; ON ENTRY: NOTHING REQUIRED
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;--------------------------------------------------------------------------------
                                <1>
                                <1> FMTDMA_SET:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ;; 20/02/2015 modification
                                <1>     ;;mov   edx, [ebp+4]            ; Buffer address
                                <1>     ; 06/08/2022
                                <1>     ;mov    edx, ebp ; buffer address
                                <1>     ; 06/08/2022
                                <1>     ;test   edx, 0FF000000h                 ; 16 MB limit
000049A3 F7C5000000FF           <1>     test    ebp, 0FF000000h
000049A9 75EF                   <1>     jnz     short dma_bnd_err_stc
                                <1>     ;
                                <1>     ;;push  dx                      ; *
                                <1>     ;; 11/04/2021
                                <1>     ; 06/08/2022
                                <1>     ;push   edx
                                <1>     ;mov    dl, 4                   ; SECTORS/TRACK VALUE IN PARM TABLE
                                <1>     ; 06/08/2022
000049AB B004                   <1>     mov     al, 4                   ; SECTORS/TRACK VALUE IN PARM TABLE
000049AD E85E020000             <1>     call    GET_PARM                ; "
000049B2 88E0                   <1>     mov     al, ah                  ; AL = SECTORS/TRACK VALUE
000049B4 28E4                   <1>     sub     ah, ah                  ; AX = SECTORS/TRACK VALUE
                                <1>     ;shl    ax, 2                   ; AX = SEC/TRK * 4 (OFFSET C,H,R,N)
                                <1>     ; 06/08/2022
000049B6 C1E002                 <1>     shl     eax, 2
                                <1>     ;dec    ax                      ; -1 FOR DMA VALUE
000049B9 48                     <1>     dec     eax
                                <1>     ;mov    cx, ax
000049BA 89C1                   <1>     mov     ecx, eax
                                <1>     ;;pop   dx                      ; *
                                <1>     ;; 11/04/2021
                                <1>     ; 06/08/2022
                                <1>     ;pop    edx
                                <1>     ; 06/08/2022
000049BC B04A                   <1>     mov     al, 04Ah
                                <1>     ;
000049BE EB99                   <1>     jmp     short NOT_VERF ; 06/08/2022
                                <1>
                                <1> ;; 06/08/2022
                                <1> ;   add     dx, cx                  ; check for overflow
                                <1> ;   jc      short dma_bnd_err
                                <1> ;   ;
                                <1> ;   sub     dx, cx                  ; Restore start address
                                <1> ;   ;
                                <1> ;   ;mov    al, 04Ah                ; WILL WRITE TO THE DISKETTE
                                <1> ;   cli                             ; DISABLE INTERRUPTS DURING DMA SET-UP
                                <1> ;   out     DMA+12, al              ; SET THE FIRST/LA5T F/F
                                <1> ;   IODELAY                         ; WAIT FOR I/O
                                <1> ;   out     DMA+11, al              ; OUTPUT THE MODE BYTE
                                <1> ;   mov     eax, edx                ; Buffer address
                                <1> ;   out     DMA+4, al               ; OUTPUT LOW ADDRESS
                                <1> ;   IODELAY                         ; WAIT FOR I/O
                                <1> ;   mov     al, ah
                                <1> ;   out     DMA+4, al               ; OUTPUT HIGH ADDRESS
                                <1> ;   shr     eax, 16
                                <1> ;   IODELAY                         ; I/O WAIT STATE
                                <1> ;   out     081h, al                ; OUTPUT HIGHEST BITS TO PAGE REGISTER
                                <1> ;   IODELAY
                                <1> ;   ;mov    ax, cx                  ; Byte count - 1
                                <1> ;   ; 06/08/2022
                                <1> ;   mov     eax, ecx
                                <1> ;   out     DMA+5, al               ; LOW BYTE OF COUNT
                                <1> ;   IODELAY                         ; WAIT FOR I/O
                                <1> ;   mov     al, ah
                                <1> ;   out     DMA+5, al               ; HIGH BYTE OF COUNT
                                <1> ;   IODELAY
                                <1> ;   sti                             ; RE-ENABLE INTERRUPTS
                                <1> ;   mov     al, 2                   ; MODE FOR 8237
                                <1> ;   out     DMA+10, al              ; INITIALIZE THE DISKETTE CHANNEL
                                <1> ;
                                <1> ;   ; 06/08/2022
                                <1> ;   clc
                                <1> ;   retn
                                <1>
                                <1> ;; 08/02/2015 - Protected Mode Modification
                                <1> ;;  mov     AL,04AH                 ; WILL WRITE TO THE DISKETTE
                                <1> ;;  CLI                             ; DISABLE INTERRUPTS DURING DMA SET-UP
                                <1> ;;  OUT     DMA+12,AL               ; SET THE FIRST/LA5T F/F
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  OUT     DMA+11,AL               ; OUTPUT THE MODE BYTE
                                <1> ;;  ;mov    AX,ES                   ; GET THE ES VALUE
                                <1> ;;  ;ROL    AX,4                    ; ROTATE LEFT
                                <1> ;;  ;mov    CH,AL                   ; GET HIGHEST NIBBLE OF ES TO CH
                                <1> ;;  ;and    AL,11110000B            ; ZERO THE LOW NIBBLE FROM SEGMENT
                                <1> ;;  ;add    AX,[BP+2]               ; TEST FOR CARRY FROM ADDITION
                                <1> ;;  ;jnc    short J33A
                                <1> ;;  ;inc    CH                      ; CARRY MEANS HIGH 4 BITS MUST BE INC
                                <1> ;;  mov     eax,[ebp+4] ; 08/02/2015
                                <1> ;;;J33A:
                                <1> ;;  push    eax ; 08/02/2015        ; SAVE START ADDRESS
                                <1> ;;  OUT     DMA+4,AL                ; OUTPUT LOW ADDRESS
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  mov     AL,AH
                                <1> ;;  OUT     DMA+4,AL                ; OUTPUT HIGH ADDRESS
                                <1> ;;  shr     eax,16 ; 08/02/2015
                                <1> ;;  ;mov    AL,CH                   ; GET HIGH 4 BITS
                                <1> ;;  ;jmp    $+2                     ; I/O WAIT STATE
                                <1> ;;  IODELAY
                                <1> ;;  ;and    AL,00001111B
                                <1> ;;  OUT     081H,AL                 ; OUTPUT HIGH 4 BITS TO PAGE REGISTER
                                <1> ;;
                                <1> ;;;----- DETERMINE COUNT
                                <1> ;;  sub     eax,eax ; 08/02/2015
                                <1> ;;  mov     DL,4                    ; SECTORS/TRACK VALUE IN PARM TABLE
                                <1> ;;  call    GET_PARM                ; "
                                <1> ;;  xchg    AL,AH                   ; AL = SECTORS/TRACK VALUE
```

```
                                <1> ;;  sub     AH,AH                   ; AX = SECTORS/TRACK VALUE
                                <1> ;;  SHL     AX,2                    ; AX = SEC/TRK * 4 (OFFSET C,H,R,N)
                                <1> ;;  dec     AX                      ; -1 FOR DMA VALUE
                                <1> ;;  push    eax     ; 08/02/2015    ; SAVE # OF BYTES TO BE TRANSFERED
                                <1> ;;  OUT     DMA+5,AL                ; LOW BYTE OF COUNT
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  IODELAY
                                <1> ;;  mov     AL,AH
                                <1> ;;  OUT     DMA+5,AL                ; HIGH BYTE OF COUNT
                                <1> ;;  STI                             ; RE-ENABLE INTERRUPTS
                                <1> ;;  pop     ecx     ; 08/02/2015    ; RECOVER COUNT VALUE
                                <1> ;;  pop     eax     ; 08/02/2015    ; RECOVER ADDRESS VALUE
                                <1> ;;  ;add    AX,CX                   ; ADD, TEST FOR 64K OVERFLOW
                                <1> ;;  add     ecx,eax ; 08/02/2015
                                <1> ;;  mov     AL,2                    ; MODE FOR 8237
                                <1> ;;  ;jmp    $+2                     ; WAIT FOR I/O
                                <1> ;;  SIODELAY
                                <1> ;;  OUT     DMA+10,AL               ; INITIALIZE THE DISKETTE CHANNEL
                                <1> ;;  ;jnc    short FMTDMA_OK                 ; CHECK FOR ERROR
                                <1> ;;  jc      short fmtdma_bnd_err ; 08/02/2015
                                <1> ;;  and     ecx,0FFF00000h ; 16 MB limit
                                <1> ;;  jz      short FMTDMA_OK
                                <1> ;;  stc     ; 20/02/2015
                                <1> ;;fmtdma_bnd_err:
                                <1> ;;  mov     byte [DSKETTE_STATUS],DMA_BOUNDARY ; SET ERROR
                                <1> ;;FMTDMA_OK:
                                <1> ;;  retn                            ; CY SET BY ABOVE IF ERROR
                                <1>
                                <1> ;--------------------------------------------------------------------------------
                                <1> ; NEC_INIT
                                <1> ;   THIS ROUTINE SEEKS TO THE REQUESTED TRACK AND INITIALIZES
                                <1> ;   THE NEC FOR THE READ/WRITE/VERIFY/FORMAT OPERATION.
                                <1> ;
                                <1> ; ON ENTRY: AH = NEC COMMAND TO BE PERFORMED
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;--------------------------------------------------------------------------------
                                <1> NEC_INIT:
                                <1>     ; 11/08/2022
                                <1>     ; EBX = user's ECX register content (on stack)
                                <1>     ; 10/08/2022
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ;push   ax                      ; SAVE NEC COMMAND
                                <1>     ; 11/04/2021
000049C0 50                     <1>     push    eax
000049C1 E885020000             <1>     call    MOTOR_ON                ; TURN MOTOR ON FOR SPECIFIC DRIVE
                                <1>
                                <1> ;-----      DO THE SEEK OPERATION
                                <1>
                                <1>     ;mov    ch, [ebp+1]             ; CH = TRACK #
                                <1>     ; 10/08/2022
000049C6 88FD                   <1>     mov     ch, bh ; CH = TRACK #
000049C8 E870030000             <1>     call    SEEK                    ; MOVE TO CORRECT TRACK
                                <1>     ;pop    ax                      ; RECOVER COMMAND
                                <1>     ; 11/04/2021
000049CD 58                     <1>     pop     eax
000049CE 721D                   <1>     jc      short ER_1              ; ERROR ON SEEK
000049D0 BB[ED490000]           <1>     mov     ebx, ER_1               ; LOAD ERROR ADDRESS
000049D5 53                     <1>     push    ebx                     ; PUSH NEC_OUT ERROR RETURN
                                <1>
                                <1> ;-----      SEND OUT THE PARAMETERS TO THE CONTROLLER
                                <1>
000049D6 E82B030000             <1>     call    NEC_OUTPUT              ; OUTPUT THE OPERATION COMMAND
                                <1>     ;mov    ax, si                  ; AH = HEAD #
                                <1>     ; 06/08/2022
000049DB 89F0                   <1>     mov     eax, esi
000049DD 89FB                   <1>     mov     ebx, edi                ; BL = DRIVE #
000049DF C0E402                 <1>     sal     ah, 2                   ; MOVE IT TO BIT 2
000049E2 80E404                 <1>     and     ah, 00000100b           ; ISOLATE THAT BIT
000049E5 08DC                   <1>     or      ah, bl                  ; OR IN THE DRIVE NUMBER
000049E7 E81A030000             <1>     call    NEC_OUTPUT              ; FALL THRU CY SET IF ERROR
000049EC 5B                     <1>     pop     ebx                     ; THROW AWAY ERROR RETURN
                                <1> ER_1:
000049ED C3                     <1>     retn
                                <1>
                                <1> ;--------------------------------------------------------------------------------
                                <1> ; RWV_COM
                                <1> ;   THIS ROUTINE SENDS PARAMETERS TO THE NEC SPECIFIC TO THE
                                <1> ;   READ/WRITE/VERIFY OPERATIONS.
                                <1> ;
                                <1> ; ON ENTRY: CS:BX = ADDRESS OF MEDIA/DRIVE PARAMETER TABLE
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;--------------------------------------------------------------------------------
                                <1> RWV_COM:
                                <1>     ; 11/08/2022
                                <1>     ; 10/08/2022
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
000049EE B8[3A4A0000]           <1>     mov     eax, ER_2               ; LOAD ERROR ADDRESS
000049F3 50                     <1>     push    eax                     ; PUSH NEC_OUT ERROR RETURN
                                <1>     ;mov    ah, [ebp+1]             ; OUTPUT TRACK #
                                <1>     ; 11/08/2022
000049F4 8A642425               <1>     mov     ah, [esp+37] ; CH = OUTPUT TRACK #
000049F8 E809030000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    ax, si                  ; OUTPUT HEAD #
                                <1>     ; 06/08/2022
000049FD 89F0                   <1>     mov     eax, esi
000049FF E802030000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    ah, [ebp]               ; OUTPUT SECTOR #
                                <1>     ; 11/08/2022
00004A04 8A642424               <1>     mov     ah, [esp+36] ; CL = OUTPUT SECTOR #
00004A08 E8F9020000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    dl, 3                   ; BYTES/SECTOR PARAMETER FROM BLOCK
                                <1>     ; 06/08/2022
00004A0D B003                   <1>     mov     al, 3
00004A0F E8FC010000             <1>     call    GET_PARM                ; ... TO THE NEC
00004A14 E8ED020000             <1>     call    NEC_OUTPUT              ; OUTPUT TO CONTROLLER
                                <1>     ;mov    dl, 4                   ; EOT PARAMETER FROM BLOCK
                                <1>     ; 06/08/2022
00004A19 B004                   <1>     mov     al, 4
00004A1B E8F0010000             <1>     call    GET_PARM                ; ... TO THE NEC
00004A20 E8E1020000             <1>     call    NEC_OUTPUT              ; OUTPUT TO CONTROLLER
00004A25 8A6305                 <1>     mov     ah, [ebx+MD.GAP]          ; GET GAP LENGTH
                                <1> _R15:
00004A28 E8D9020000             <1>     call    NEC_OUTPUT
                                <1>     ;mov    dl, 6                   ; DTL PARAMETER PROM BLOCK
                                <1>     ; 06/08/2022
00004A2D B006                   <1>     mov     al, 6
00004A2F E8DC010000             <1>     call    GET_PARM                ; ... TO THE NEC
00004A34 E8CD020000             <1>     call    NEC_OUTPUT              ; OUTPUT TO CONTROLLER
00004A39 58                     <1>     pop     eax                     ; THROW AWAY ERROR EXIT
                                <1> ER_2:
00004A3A C3                     <1>     retn
                                <1>
                                <1> ;--------------------------------------------------------------------------------
```

```
                                <1> ; NEC_TERM
                                <1> ;   THIS ROUTINE WAITS FOR THE OPERATION THEN ACCEPTS THE STATUS
                                <1> ;   FROM THE NEC FOR THE READ/WRITE/VERIFY/FORWAT OPERATION.
                                <1> ;
                                <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                                <1> ;-------------------------------------------------------------------------------
                                <1> NEC_TERM:
                                <1>
                                <1> ;-----      LET THE OPERATION HAPPEN
                                <1>
00004A3B 56                     <1>     push    esi                     ; SAVE HEAD #, # OF SECTORS
00004A3C E8CD030000             <1>     call    WAIT_INT                ; WAIT FOR THE INTERRUPT
00004A41 9C                     <1>     pushf
00004A42 E8F6030000             <1>     call    RESULTS                 ; GET THE NEC STATUS
00004A47 724B                   <1>     jc      short SET_END_POP
00004A49 9D                     <1>     popf
00004A4A 723E                   <1>     jc      short SET_END           ; LOOK FOR ERROR
                                <1>
                                <1> ;-----      CHECK THE RESULTS RETURNED BY THE CONTROLLER
                                <1>
00004A4C FC                     <1>     cld                             ; SET THE CORRECT DIRECTION
00004A4D BE[39770100]           <1>     mov     esi, NEC_STATUS                 ; POINT TO STATUS FIELD
00004A52 AC                     <1>     lodsb                           ; GET ST0
00004A53 24C0                   <1>     and     AL, 11000000B           ; TEST FOR NORMAL TERMINATION
00004A55 7433                   <1>     jz      short SET_END
00004A57 3C40                   <1>     cmp     AL,01000000B            ; TEST FOR ABNORMAL TERMINATION
00004A59 7527                   <1>     jnz     short J18               ; NOT ABNORMAL, BAD NEC
                                <1>
                                <1> ;-----      ABNORMAL TERMINATION, FIND OUT WHY
                                <1>
00004A5B AC                     <1>     lodsb                           ; GET ST1
00004A5C D0E0                   <1>     sal     al, 1                   ; TEST FOR EDT FOUND
00004A5E B404                   <1>     mov     ah, RECORD_NOT_FND
00004A60 7222                   <1>     jc      short J19
00004A62 C0E002                 <1>     sal     al, 2
00004A65 B410                   <1>     mov     ah, BAD_CRC
00004A67 721B                   <1>     jc      short J19
00004A69 D0E0                   <1>     sal     al, 1                   ; TEST FOR DMA OVERRUN
00004A6B B408                   <1>     mov     ah, BAD_DMA
00004A6D 7215                   <1>     jc      short J19
00004A6F C0E002                 <1>     sal     al, 2                   ; TEST FOR RECORD NOT FOUND
00004A72 B404                   <1>     mov     ah, RECORD_NOT_FND
00004A74 720E                   <1>     jc      short J19
00004A76 D0E0                   <1>     sal     al, 1
00004A78 B403                   <1>     mov     ah, WRITE_PROTECT       ; TEST FOR WRITE_PROTECT
00004A7A 7208                   <1>     jc      short J19
00004A7C D0E0                   <1>     sal     al, 1                   ; TEST MISSING ADDRESS MARK
00004A7E B402                   <1>     mov     ah, BAD_ADDR_MARK
00004A80 7202                   <1>     jc      short J19
                                <1>
                                <1> ;-----      NEC MUST HAVE FAILED
                                <1> J18:
00004A82 B420                   <1>     mov     ah, BAD_NEC
                                <1> J19:
00004A84 0825[38770100]         <1>     or      [DSKETTE_STATUS], ah
                                <1> SET_END:
00004A8A 803D[38770100]01       <1>     cmp     byte [DSKETTE_STATUS], 1 ; SET ERROR CONDITION
00004A91 F5                     <1>     cmc
00004A92 5E                     <1>     pop     esi
00004A93 C3                     <1>     retn                            ; RESTORE HEAD #, # OF SECTORS
                                <1>
                                <1> SET_END_POP:
00004A94 9D                     <1>     popf
00004A95 EBF3                   <1>     jmp     short SET_END
                                <1>
                                <1> ;-------------------------------------------------------------------------------
                                <1> ; DSTATE:   ESTABLISH STATE UPON SUCCESSFUL OPERATION.
                                <1> ;-------------------------------------------------------------------------------
                                <1> DSTATE:
00004A97 803D[38770100]00       <1>     cmp     byte [DSKETTE_STATUS], 0 ; CHECK FOR ERROR
00004A9E 753E                   <1>     jnz     short SETBAC            ; IF ERROR JUMP
00004AA0 808F[43770100]10       <1>     or      byte [DSK_STATE+edi], MED_DET
                                <1>                                     ; NO ERROR, MARK MEDIA AS DETERMINED
00004AA7 F687[43770100]04       <1>     test    byte [DSK_STATE+edi], DRV_DET ; DRIVE DETERMINED ?
00004AAE 752E                   <1>     jnz     short SETBAC            ; IF DETERMINED NO TRY TO DETERMINE
00004AB0 8A87[43770100]         <1>     mov     al, [DSK_STATE+edi]     ; LOAD STATE
00004AB6 24C0                   <1>     and     al, RATE_MSK            ; KEEP ONLY RATE
00004AB8 3C80                   <1>     cmp     al, RATE_250            ; RATE 250 ?
00004ABA 751B                   <1>     jne     short M_12              ; NO, MUST BE 1.2M OR 1.44M DRIVE
                                <1>
                                <1> ;-----      CHECK IF IT IS 1.44M
                                <1>
00004ABC E846010000             <1>     call    CMOS_TYPE               ; RETURN DRIVE TYPE IN (AL)
                                <1>     ;;20/02/2015
                                <1>     ;;jc    short M_12              ; CMOS BAD
00004AC1 7414                   <1>     jz      short M_12 ;; 20/02/2015
00004AC3 3C04                   <1>     cmp     al, 4                   ; 1.44MB DRIVE ?
00004AC5 7410                   <1>     je      short M_12              ; YES
                                <1> M_720:
00004AC7 80A7[43770100]FD       <1>     and     byte [DSK_STATE+edi], ~FMT_CAPA ; TURN OFF FORMAT CAPABILITY
00004ACE 808F[43770100]04       <1>     or      byte [DSK_STATE+edi], DRV_DET ; MARK DRIVE DETERMINED
00004AD5 EB07                   <1>     jmp     short SETBAC            ; BACK
                                <1> M_12:
00004AD7 808F[43770100]06       <1>     or      byte [DSK_STATE+edi], DRV_DET+FMT_CAPA
                                <1>                                     ; TURN ON DETERMINED & FMT CAPA
                                <1> SETBAC:
00004ADE C3                     <1>     retn
                                <1>
                                <1> ;-------------------------------------------------------------------------------
                                <1> ; RETRY
                                <1> ;   DETERMINES WHETHER A RETRY IS NECESSARY.
                                <1> ;   IF RETRY IS REQUIRED THEN STATE INFORMATION IS UPDATED FOR RETRY.
                                <1> ;
                                <1> ; ON EXIT:  CY = 1 FOR RETRY, CY = 0 FOR NO RETRY
                                <1> ;-------------------------------------------------------------------------------
                                <1> RETRY:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004ADF 803D[38770100]00       <1>     cmp     byte [DSKETTE_STATUS], 0 ; GET STATUS OF OPERATION
00004AE6 7445                   <1>     jz      short NO_RETRY          ; SUCCESSFUL OPERATION
00004AE8 803D[38770100]80       <1>     cmp     byte [DSKETTE_STATUS], TIME_OUT ; IF TIME OUT NO RETRY
00004AEF 743C                   <1>     jz      short NO_RETRY
00004AF1 8AA7[43770100]         <1>     mov     ah, [DSK_STATE+edi]     ; GET MEDIA STATE OF DRIVE
00004AF7 F6C410                 <1>     test    ah, MED_DET             ; ESTABLISHED/DETERMINED ?
00004AFA 7531                   <1>     jnz     short NO_RETRY          ; IF ESTABLISHED STATE THEN TRUE ERROR
00004AFC 80E4C0                 <1>     and     ah, RATE_MSK            ; ISOLATE RATE
00004AFF 8A2D[40770100]         <1>     mov     ch, [LASTRATE]          ; GET START OPERATION STATE
00004B05 C0C504                 <1>     rol     ch, 4                   ; TO CORRESPONDING BITS
00004B08 80E5C0                 <1>     and     ch, RATE_MSK            ; ISOLATE RATE BITS
00004B0B 38E5                   <1>     cmp     ch, ah                  ; ALL RATES TRIED
00004B0D 741E                   <1>     je      short NO_RETRY          ; IF YES, THEN TRUE ERROR
                                <1>
                                <1> ;   SETUP STATE INDICATOR FOR RETRY ATTEMPT TO NEXT RATE
                                <1> ;    00000000B (500) -> 10000000B (250)
                                <1> ;    10000000B (250) -> 01000000B (300)
```

```
                                <1> ;    01000000B (300) -> 00000000B (500)
                                <1>
00004B0F 80FC01                 <1>     cmp     ah, RATE_500+1          ; SET CY FOR RATE 500
00004B12 D0DC                   <1>     rcr     ah, 1                   ; TO NEXT STATE
00004B14 80E4C0                 <1>     and     ah, RATE_MSK            ; KEEP ONLY RATE BITS
00004B17 80A7[43770100]1F       <1>     and     byte [DSK_STATE+edi], ~(RATE_MSK+DBL_STEP)
                                <1>                                     ; RATE, DBL STEP OFF
00004B1E 08A7[43770100]         <1>     or      [DSK_STATE+edi], ah     ; TURN ON NEW RATE
00004B24 C605[38770100]00       <1>     mov     byte [DSKETTE_STATUS], 0  ; RESET STATUS FOR RETRY
00004B2B F9                     <1>     stc                             ; SET CARRY FOR RETRY
00004B2C C3                     <1>     retn                            ; RETRY RETURN
                                <1>
                                <1> NO_RETRY:
                                <1>     ; 06/08/2022
                                <1>     ;clc                            ; CLEAR CARRY NO RETRY
00004B2D C3                     <1>     retn                            ; NO RETRY RETURN
                                <1>
                                <1> ;---------------------------------------------------------------------------
                                <1> ; NUM_TRANS
                                <1> ;   THIS ROUTINE CALCULATES THE NUMBER OF SECTORS THAT WERE
                                <1> ;   ACTUALLY TRANSFERRED TO/FROM THE DISKETTE.
                                <1> ;
                                <1> ; ON ENTRY: [BP+1] = TRACK
                                <1> ;           SI-HI  = HEAD
                                <1> ;           [BP]   = START SECTOR
                                <1> ;
                                <1> ; ON EXIT:  AL = NUMBER ACTUALLY TRANSFERRED
                                <1> ;---------------------------------------------------------------------------
                                <1> NUM_TRANS:
                                <1>     ; 10/08/2022
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004B2E 30C0                   <1>     xor     al, al                  ; CLEAR FOR ERROR
                                <1>     ;cmp    byte [DSKETTE_STATUS], 0
00004B30 3805[38770100]         <1>     cmp     [DSKETTE_STATUS], al ; 0 ; CHECK FOR ERROR
00004B36 752D                   <1>     jnz     short NT_OUT            ; IF ERROR 0 TRANSFERRED
                                <1>     ;mov    dl, 4                   ; SECTORS/TRACK OFFSET TO DL
                                <1>     ; 06/08/2022
00004B38 B004                   <1>     mov     al, 4
00004B3A E8D1000000             <1>     call    GET_PARM                ; AH = SECTORS/TRACK
00004B3F 8A1D[3E770100]         <1>     mov     bl, [NEC_STATUS+5]      ; GET ENDING SECTOR
                                <1>     ;mov    cx, si                  ; CH = HEAD # STARTED
                                <1>     ; 06/08/2022
00004B45 89F1                   <1>     mov     ecx, esi
00004B47 3A2D[3D770100]         <1>     cmp     ch, [NEC_STATUS+4]      ; GET HEAD ENDED UP ON
00004B4D 750E                   <1>     jnz     short DIF_HD            ; IF ON SAME HEAD, THEN NO ADJUST
00004B4F 8A2D[3C770100]         <1>     mov     ch, [NEC_STATUS+3]      ; GET TRACK ENDED UP ON
                                <1>     ;cmp    ch, [ebp+1]             ; IS IT ASKED FOR TRACK
                                <1>     ; 10/08/2022
00004B55 3A6C241D               <1>     cmp     ch, [esp+29] ; CH = TRACK #
00004B59 7404                   <1>     jz      short SAME_TRK          ; IF SAME TRACK NO INCREASE
00004B5B 00E3                   <1>     add     bl, ah                  ; ADD SECTORS/TRACK
                                <1> DIF_HD:
00004B5D 00E3                   <1>     add     bl, ah                  ; ADD SECTORS/TRACK
                                <1> SAME_TRK:
                                <1>     ;sub    bl, [ebp]               ; SUBTRACT START FROM END
                                <1>     ; 10/08/2022
00004B5F 2A5C241C               <1>     sub     bl, [esp+28] ; CL = SECTOR #
00004B63 88D8                   <1>     mov     al, bl                  ; TO AL
                                <1> NT_OUT:
00004B65 C3                     <1>     retn
                                <1>
                                <1> ;---------------------------------------------------------------------------
                                <1> ; SETUP_DBL
                                <1> ;   CHECK DOUBLE STEP.
                                <1> ;
                                <1> ; ON ENTRY :       DI = DRIVE
                                <1> ;
                                <1> ; ON EXIT : CY = 1 MEANS ERROR
                                <1> ;---------------------------------------------------------------------------
                                <1> SETUP_DBL:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004B66 8AA7[43770100]         <1>     mov     ah, [DSK_STATE+edi]     ; ACCESS STATE
00004B6C F6C410                 <1>     test    ah, MED_DET             ; ESTABLISHED STATE ?
00004B6F 7578                   <1>     jnz     short NO_DBL            ; IF ESTABLISHED THEN DOUBLE DONE
                                <1>
                                <1> ;-----      CHECK FOR TRACK 0 TO SPEED UP ACKNOWLEDGE OF UNFORMATTED DISKETTE
                                <1>
00004B71 C605[35770100]00       <1>     mov     byte [SEEK_STATUS], 0 ; SET RECALIBRATE REQUIRED ON ALL DRIVES
00004B78 E8CE000000             <1>     call    MOTOR_ON                ; ENSURE MOTOR STAY ON
00004B7D B500                   <1>     mov     ch, 0                   ; LOAD TRACK 0
00004B7F E8B9010000             <1>     call    SEEK                    ; SEEK TO TRACK 0
00004B84 E861000000             <1>     call    READ_ID                 ; READ ID FUNCTION
00004B89 7243                   <1>     jc      short SD_ERR            ; IF ERROR NO TRACK 0
                                <1>
                                <1> ;-----      INITIALIZE START AND MAX TRACKS (TIMES 2 FOR BOTH HEADS)
                                <1>
00004B8B 66B95004               <1>     mov     cx, 0450h               ; START, MAX TRACKS
00004B8F F687[43770100]01       <1>     test    byte [DSK_STATE+edi], TRK_CAPA ; TEST FOR 80 TRACK CAPABILITY
00004B96 7402                   <1>     jz      short CNT_OK            ; IF NOT COUNT IS SETUP
00004B98 B1A0                   <1>     mov     cl, 0A0h                ; MAXIMUM TRACK 1.2 MB
                                <1>
                                <1> ;   ATTEMPT READ ID OF ALL TRACKS, ALL HEADS UNTIL SUCCESS; UPON SUCCESS,
                                <1> ;   MUST SEE IF ASKED FOR TRACK IN SINGLE STEP MODE = TRACK ID READ; IF NOT
                                <1> ;   THEN SET DOUBLE STEP ON.
                                <1>
                                <1> CNT_OK:
                                <1>     ; 11/04/2021 (32 bit push/pop)
00004B9A C605[37770100]FF       <1>         mov     byte [MOTOR_COUNT], 0FFh ; ENSURE MOTOR STAYS ON FOR OPERATION
00004BA1 51                     <1>     push    ecx                     ; SAVE TRACK, COUNT
00004BA2 C605[38770100]00       <1>     mov     byte [DSKETTE_STATUS], 0 ; CLEAR STATUS, EXPECT ERRORS
                                <1>     ;xor    ax, ax                  ; CLEAR AX
                                <1>     ; 06/08/2022
00004BA9 31C0                   <1>     xor     eax, eax
00004BAB D0ED                   <1>     shr     ch, 1                   ; HALVE TRACK, CY = HEAD
00004BAD C0D003                 <1>     rcl     al, 3                   ; AX = HEAD IN CORRECT BIT
00004BB0 50                     <1>     push    eax                     ; SAVE HEAD
00004BB1 E887010000             <1>     call    SEEK                    ; SEEK TO TRACK
00004BB6 58                     <1>     pop     eax                     ; RESTORE HEAD
                                <1>     ;or     di, ax                  ; DI = HEAD OR'ED DRIVE
                                <1>     ; 06/08/2022
00004BB7 09C7                   <1>     or      edi, eax
00004BB9 E82C000000             <1>     call    READ_ID                 ; READ ID HEAD 0
00004BBE 9C                     <1>     pushf                           ; SAVE RETURN FROM READ_ID
00004BBF 6681E7FB00             <1>     and     di, 11111011b           ; TURN OFF HEAD 1 BIT
00004BC4 9D                     <1>     popf                            ; RESTORE ERROR RETURN
00004BC5 59                     <1>     pop     ecx                     ; RESTORE COUNT
00004BC6 7308                   <1>     jnc     short DO_CHK            ; IF OK, ASKED = RETURNED TRACK ?
00004BC8 FEC5                   <1>     inc     ch                      ; INC FOR NEXT TRACK
00004BCA 38CD                   <1>     cmp     ch, cl                  ; REACHED MAXIMUM YET
00004BCC 75CC                   <1>     jnz     short CNT_OK            ; CONTINUE TILL ALL TRIED
                                <1>
                                <1> ;-----      FALL THRU, READ ID FAILED FOR ALL TRACKS
                                <1>
                                <1> SD_ERR:
```

```
00004BCE F9                  <1>     stc                         ; SET CARRY FOR ERROR
00004BCF C3                  <1>     retn                        ; SETUP_DBL ERROR EXIT
                             <1>
                             <1> DO_CHK:
00004BD0 8A0D[3C770100]      <1>     mov     cl, [NEC_STATUS+3]  ; LOAD RETURNED TRACK
00004BD6 888F[45770100]      <1>     mov     [DSK_TRK+edi], cl   ; STORE TRACK NUMBER
00004BDC D0ED                <1>     shr     ch, 1               ; HALVE TRACK
00004BDE 38CD                <1>     cmp     ch, cl              ; IS IT THE SAME AS ASKED FOR TRACK
00004BE0 7407                <1>     jz      short NO_DBL        ; IF SAME THEN NO DOUBLE STEP
00004BE2 808F[43770100]20    <1>     or      byte [DSK_STATE+edi], DBL_STEP ; TURN ON DOUBLE STEP REQUIRED
                             <1> NO_DBL:
                             <1>     ; 06/08/2022
                             <1>     ;clc                        ; CLEAR ERROR FLAG
00004BE9 C3                  <1>     retn
                             <1>
                             <1> ;-----------------------------------------------------------------------------
                             <1> ; READ_ID
                             <1> ;   READ ID FUNCTION.
                             <1> ;
                             <1> ; ON ENTRY: DI : BIT 2 = HEAD; BITS 1,0 = DRIVE
                             <1> ;
                             <1> ; ON EXIT:  DI : BIT 2 IS RESET, BITS 1,0 = DRIVE
                             <1> ;           @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION
                             <1> ;-----------------------------------------------------------------------------
                             <1> READ_ID:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004BEA B8[064C0000]        <1>     mov     eax, ER_3           ; MOVE NEC OUTPUT ERROR ADDRESS
00004BEF 50                  <1>     push    eax
00004BF0 B44A                <1>     mov     ah, 4Ah             ; READ ID COMMAND
00004BF2 E80F010000          <1>     call    NEC_OUTPUT          ; TO CONTROLLER
                             <1>     ;mov    ax, di              ; DRIVE # TO AH, HEAD 0
                             <1>     ; 06/08/2022
00004BF7 89F8                <1>     mov     eax, edi
00004BF9 88C4                <1>     mov     ah, al
00004BFB E806010000          <1>     call    NEC_OUTPUT          ; TO CONTROLLER
00004C00 E836FEFFFF          <1>     call    NEC_TERM            ; WAIT FOR OPERATION, GET STATUS
00004C05 58                  <1>     pop     eax                 ; THROW AWAY ERROR ADDRESS
                             <1> ER_3:
00004C06 C3                  <1>     retn
                             <1>
                             <1> ;-----------------------------------------------------------------------------
                             <1> ; CMOS_TYPE
                             <1> ;   RETURNS DISKETTE TYPE FROM CMOS
                             <1> ;
                             <1> ; ON ENTRY: DI = DRIVE #
                             <1> ;
                             <1> ; ON EXIT:  AL = TYPE; CY REFLECTS STATUS
                             <1> ;-----------------------------------------------------------------------------
                             <1>
                             <1> CMOS_TYPE: ; 11/12/2014
00004C07 8A87[8C660000]      <1>     mov     al, [edi+fd0_type]
00004C0D 20C0                <1>     and     al, al ; 18/12/2014
00004C0F C3                  <1>     retn
                             <1>
                             <1> ;CMOS_TYPE:
                             <1> ;   mov     al, CMOS_DIAG       ; CMOS DIAGNOSTIC STATUS BYTE ADDRESS
                             <1> ;   call    CMOS_READ           ; GET CMOS STATUS
                             <1> ;   test    al, BAD_BAT+BAD_CKSUM ; BATTERY GOOD AND CHECKSUM VALID
                             <1> ;   stc                         ; SET CY = 1 INDICATING ERROR FOR RETURN
                             <1> ;   jnz     short BAD_CM        ; ERROR IF EITHER BIT ON
                             <1> ;   mov     al, CMOS_DISKETTE   ; ADDRESS OF DISKETTE BYTE IN CMOS
                             <1> ;   call    CMOS_READ           ; GET DISKETTE BYTE
                             <1> ;   or      di, di              ; SEE WHICH DRIVE IN QUESTION
                             <1> ;   jnz     short TB            ; IF DRIVE 1, DATA IN LOW NIBBLE
                             <1> ;   ror     al, 4               ; EXCHANGE NIBBLES IF SECOND DRIVE
                             <1> ;TB:
                             <1> ;   and     al, 0Fh             ; KEEP ONLY DRIVE DATA, RESET CY, 0
                             <1> ;BAD_CM:
                             <1> ;   retn                        ; CY, STATUS OF READ
                             <1>
                             <1> ;-----------------------------------------------------------------------------
                             <1> ; GET_PARM
                             <1> ;   THIS ROUTINE FETCHES THE INDEXED POINTER FROM THE DISK_BASE
                             <1> ;   BLOCK POINTED TO BY THE DATA VARIABLE @DISK_POINTER. A BYTE FROM
                             <1> ;   THAT TABLE IS THEN MOVED INTO AH, THE INDEX OF THAT BYTE BEING
                             <1> ;   THE PARAMETER IN DL.
                             <1> ;
                             <1> ; ON ENTRY: DL = INDEX OF BYTE TO BE FETCHED
                             <1> ;
                             <1> ; ON EXIT:  AH = THAT BYTE FROM BLOCK
                             <1> ;           AL, DH DESTROYED
                             <1> ;-----------------------------------------------------------------------------
                             <1> GET_PARM:
                             <1>     ; 09/08/2022
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; AL = INDEX
                             <1>     ; EDI = (current) DRIVE #
                             <1>     ;;push  ds
                             <1>     ;push   esi
                             <1>     ;;sub   ax, ax              ; DS = 0, BIOS DATA AREA
                             <1>     ;;mov   ds, ax
                             <1>     ;;mov   ax, cs
                             <1>     ;;mov   ds, ax
                             <1>     ; 08/02/2015 (protected mode modifications, bx -> ebx)
                             <1>     ;xchg   edx, ebx            ; BL = INDEX
                             <1>     ; 06/08/2022
00004C10 53                  <1>     push    ebx                 ; SAVE EBX
                             <1>     ;movzx  ebx, dl             ; EBX = INDEX
00004C11 0FB6D8              <1>     movzx   ebx, al ; 06/08/2022
                             <1>     ;;sub   bh, bh              ; BX = INDEX
                             <1>     ;and    ebx, 0FFh
                             <1>     ;;lds   si, [DISK_POINTER]  ; POINT TO BLOCK
                             <1>     ;
                             <1>     ; 17/12/2014
                             <1>     ;mov    ax, [cfd]           ; current (AL) and previous fd (AH)
                             <1>     ; 06/08/2022
00004C14 89F8                <1>     mov     eax, edi            ; EDI = DRIVE #
                             <1>     ;cmp    al, ah
00004C16 3A05[7D660000]      <1>     cmp     al, [pfd]
00004C1C 7423                <1>     je      short gpndc
00004C1E A2[7D660000]        <1>     mov     [pfd], al           ; current drive -> previous drive
00004C23 53                  <1>     push    ebx ; 08/02/2015
                             <1>     ;mov    bl, al
                             <1>     ;; 11/12/2014
                             <1>     ;mov    al, [ebx+fd0_type]  ; Drive type (0,1,2,3,4)
                             <1>     ; 09/08/2022
00004C24 8A87[8C660000]      <1>     mov     al, [edi+fd0_type]  ; Drive type (0,1,2,3,4)
                             <1>     ; 18/12/2014
00004C2A 20C0                <1>     and     al, al
00004C2C 7507                <1>     jnz     short gpdtc
00004C2E BB[67660000]        <1>     mov     ebx, MD_TBL6        ; 1.44 MB param. tbl. (default)
00004C33 EB05                <1>         jmp     short gpdpu
                             <1> gpdtc:
00004C35 E809FBFFFF          <1>     call    DR_TYPE_CHECK
```

```
                                <1>     ; cf = 1 -> ebx points to 1.44MB fd parameter table (default)
                                <1> gpdpu:
00004C3A 891D[04660000]         <1>     mov     [DISK_POINTER], ebx
00004C40 5B                     <1>     pop     ebx
                                <1> gpndc:
                                <1>     ;mov    esi, [DISK_POINTER] ; 08/02/2015, si -> esi
                                <1>     ; 06/08/2022
00004C41 031D[04660000]         <1>     add     ebx, [DISK_POINTER]
                                <1>     ;mov    ah, [esi+ebx]         ; GET THE WORD
00004C47 8A23                   <1>     mov     ah, [ebx]
                                <1>     ;xchg   edx, ebx              ; RESTORE BX
                                <1>     ; 06/08/2022
00004C49 5B                     <1>     pop     ebx                   ; RESTORE EBX
                                <1>     ;pop    esi
                                <1>     ;;pop   ds
00004C4A C3                     <1>     retn
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1> ; MOTOR_ON
                                <1> ;   TURN MOTOR ON AND WAIT FOR MOTOR START UP TIME. THE @MOTOR_COUNT
                                <1> ;   IS REPLACED WITH A SUFFICIENTLY HIGH NUMBER (0FFH) TO ENSURE
                                <1> ;   THAT THE MOTOR DOES NOT GO OFF DURING THE OPERATION. IF THE
                                <1> ;   MOTOR NEEDED TO BE TURNED ON, THE MULTI-TASKING HOOK FUNCTION
                                <1> ;   (AX=90FDH, INT 15) IS CALLED TELLING THE OPERATING SYSTEM
                                <1> ;   THAT THE BIOS IS ABOUT TO WAIT FOR MOTOR START UP. IF THIS
                                <1> ;   FUNCTION RETURNS WITH CY = 1, IT MEANS THAT THE MINIMUM WAIT
                                <1> ;   HAS BEEN COMPLETED. AT THIS POINT A CHECK IS MADE TO ENSURE
                                <1> ;   THAT THE MOTOR WASN'T TURNED OFF BY THE TIMER. IF THE HOOK DID
                                <1> ;   NOT WAIT, THE WAIT FUNCTION (AH=086H) IS CALLED TO WAIT THE
                                <1> ;   PRESCRIBED AMOUNT OF TIME. IF THE CARRY FLAG IS SET ON RETURN,
                                <1> ;   IT MEANS THAT THE FUNCTION IS IN USE AND DID NOT PERFORM THE
                                <1> ;   WAIT. A TIMER 1 WAIT LOOP WILL THEN DO THE WAIT.
                                <1> ;
                                <1> ; ON ENTRY: DI = DRIVE #
                                <1> ; ON EXIT:  AX,CX,DX DESTROYED
                                <1> ;-----------------------------------------------------------------------------
                                <1> MOTOR_ON:
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
00004C4B 53                     <1>     push    ebx                   ; SAVE REG.
00004C4C E825000000             <1>     call    TURN_ON               ; TURN ON MOTOR
00004C51 7221                   <1>     jc      short MOT_IS_ON               ; IF CY=1 NO WAIT
                                <1>     ; 06/08/2022
                                <1>     ;call   XLAT_OLD              ; TRANSLATE STATE TO COMPATIBLE MODE
00004C53 E851FBFFFF             <1>     call    XLAT_NEW              ; TRANSLATE STATE TO PRESENT ARCH,
                                <1>     ;call   TURN_ON               ; CHECK AGAIN IF MOTOR ON
                                <1>     ;jc     short MOT_IS_ON               ; IF NO WAIT MEANS IT IS ON
                                <1> M_WAIT:
                                <1>     ;mov    dl, 10                ; GET THE MOTOR WAIT PARAMETER
                                <1>     ; 06/08/2022
00004C58 B00A                   <1>     mov     al, 10
00004C5A E8B1FFFFFF             <1>     call    GET_PARM
                                <1>     ;mov    al, ah                ; AL = MOTOR WAIT PARAMETER
                                <1>     ;xor    ah, ah                ; AX = MOTOR WAIT PARAMETER
                                <1>     ;cmp    al, 8                 ; SEE IF AT LEAST A SECOND IS SPECIFIED
00004C5F 80FC08                 <1>     cmp     ah, 8
                                <1>     ;jae    short GP2             ; IF YES, CONTINUE
00004C62 7702                   <1>     ja      short J13
                                <1>     ;mov    al, 8                 ; ONE SECOND WAIT FOR MOTOR START UP
00004C64 B408                   <1>     mov     ah, 8
                                <1>
                                <1> ;-----      AS CONTAINS NUMBER OF 1/8 SECONDS (125000 MICROSECONDS) TO WAIT
                                <1> GP2:
                                <1> ;-----      FOLLOWING LOOPS REQUIRED WHEN RTC WAIT FUNCTION IS ALREADY IN USE
                                <1> J13:                              ; WAIT FOR 1/8 SECOND PER (AL)
                                <1>     ;mov    ecx, 8286             ; COUNT FOR 1/8 SECOND AT 15.085737 US
                                <1>     ; 11/04/2021
00004C66 B947100000             <1>     mov     ecx, 4167 ; count of 30 micro seconds * (1/8)
00004C6B E8B5D7FFFF             <1>     call    WAITF                 ; GO TO FIXED WAIT ROUTINE
                                <1>     ;dec    al                    ; DECREMENT TIME VALUE
00004C70 FECC                   <1>     dec     ah
00004C72 75F2                   <1>     jnz     short J13             ; ARE WE DONE YET
                                <1> MOT_IS_ON:
00004C74 5B                     <1>     pop     ebx                   ; RESTORE REG.
00004C75 C3                     <1>     retn
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1> ; TURN_ON
                                <1> ;   TURN MOTOR ON AND RETURN WAIT STATE.
                                <1> ;
                                <1> ; ON ENTRY: DI = DRIVE #
                                <1> ;
                                <1> ; ON EXIT:  CY = 0 MEANS WAIT REQUIRED
                                <1> ;           CY = 1 MEANS NO WAIT REQUIRED
                                <1> ;           AX,BX,CX,DX DESTROYED
                                <1> ;-----------------------------------------------------------------------------
                                <1> TURN_ON:
00004C76 89FB                   <1>     mov     ebx, edi              ; BX = DRIVE #
00004C78 88D9                   <1>     mov     cl, bl                ; CL = DRIVE #
00004C7A C0C304                 <1>     rol     bl, 4                 ; BL = DRIVE SELECT
00004C7D FA                     <1>     cli                           ; NO INTERRUPTS WHILE DETERMINING STATUS
00004C7E C605[37770100]FF       <1>     mov     byte [MOTOR_COUNT], 0FFh ; ENSURE MOTOR STAYS ON FOR OPERATION
00004C85 A0[36770100]           <1>     mov     al, [MOTOR_STATUS]    ; GET DIGITAL OUTPUT REGISTER REFLECTION
00004C8A 2430                   <1>     and     al, 00110000b         ; KEEP ONLY DRIVE SELECT BITS
00004C8C B401                   <1>     mov     ah, 1                 ; MASK FOR DETERMINING MOTOR BIT
00004C8E D2E4                   <1>     shl     ah, cl                ; AH = MOTOR ON, A=00000001, B=00000010
                                <1>
                                <1> ;  AL = DRIVE SELECT FROM @MOTOR_STATUS
                                <1> ;  BL = DRIVE SELECT DESIRED
                                <1> ;  AH = MOTOR ON MASK DESIRED
                                <1>
00004C90 38D8                   <1>     cmp     al, bl                ; REQUESTED DRIVE ALREADY SELECTED ?
00004C92 7508                   <1>     jnz     short TURN_IT_ON      ; IF NOT SELECTED JUMP
00004C94 8425[36770100]         <1>     test    ah, [MOTOR_STATUS]    ; TEST MOTOR ON BIT
00004C9A 7535                   <1>     jnz     short NO_MOT_WAIT     ; JUMP IF MOTOR ON AND SELECTED
                                <1>
                                <1> TURN_IT_ON:
00004C9C 08DC                   <1>     or      ah, bl                ; AH = DRIVE SELECT AND MOTOR ON
00004C9E 8A3D[36770100]         <1>     mov     bh, [MOTOR_STATUS]    ; SAVE COPY OF @MOTOR_STATUS BEFORE
00004CA4 80E70F                 <1>     and     bh, 00001111b         ; KEEP ONLY MOTOR BITS
00004CA7 8025[36770100]CF       <1>     and     byte [MOTOR_STATUS], 11001111b ; CLEAR OUT DRIVE SELECT
00004CAE 0825[36770100]         <1>     or      [MOTOR_STATUS], ah    ; OR IN DRIVE SELECTED AND MOTOR ON
00004CB4 A0[36770100]           <1>     mov     al, [MOTOR_STATUS]    ; GET DIGITAL OUTPUT REGISTER REFLECTION
00004CB9 88C3                   <1>     mov     bl, al                ; BL=@MOTOR_STATUS AFTER, BH=BEFORE
00004CBB 80E30F                 <1>     and     bl, 00001111b         ; KEEP ONLY MOTOR BITS
00004CBE FB                     <1>     sti                           ; ENABLE INTERRUPTS AGAIN
00004CBF 243F                   <1>     and     al, 00111111b         ; STRIP AWAY UNWANTED BITS
00004CC1 C0C004                 <1>     rol     al, 4                 ; PUT BITS IN DESIRED POSITIONS
00004CC4 0C0C                   <1>     or      al, 00001100b         ; NO RESET, ENABLE DMA/INTERRUPT
00004CC6 66BAF203               <1>     mov     dx, 03F2h             ; SELECT DRIVE AND TURN ON MOTOR
00004CCA EE                     <1>     out     dx, al
00004CCB 38FB                   <1>     cmp     bl, bh                ; NEW MOTOR TURNED ON ?
                                <1>     ;jz     short NO_MOT_WAIT     ; NO WAIT REQUIRED IF JUST SELECT
00004CCD 7403                   <1>     je      short no_mot_w1 ; 27/02/2015
00004CCF F8                     <1>     clc                           ; (re)SET CARRY MEANING WAIT
```

```
00004CD0 C3                 <1>     retn
                            <1>
                            <1> NO_MOT_WAIT:
00004CD1 FB                 <1>     sti
                            <1> no_mot_w1: ; 27/02/2015
00004CD2 F9                 <1>     stc                         ; SET NO WAIT REQUIRED
                            <1>     ;sti                        ; INTERRUPTS BACK ON
00004CD3 C3                 <1>     retn
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; HD_WAIT
                            <1> ;   WAIT FOR HEAD SETTLE TIME.
                            <1> ;
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ;
                            <1> ; ON EXIT:  AX,BX,CX,DX DESTROYED
                            <1> ;-----------------------------------------------------------------------------
                            <1> HD_WAIT:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                            <1>     ;mov    dl,9                ; GET HEAD SETTLE PARAMETER
                            <1>     ; 06/08/2022
00004CD4 B009               <1>     mov     al, 9
00004CD6 E835FFFFFF         <1>     call    GET_PARM
00004CDB 08E4               <1>     or      ah, ah  ; 17/12/2014
00004CDD 7519               <1>     jnz     short DO_WAT
00004CDF F605[36770100]80   <1>         test    byte [MOTOR_STATUS], 10000000b ; SEE IF A WRITE OPERATION
                            <1>     ;jz     short ISNT_WRITE    ; IF NOT, DO NOT ENFORCE ANY VALUES
                            <1>     ;or     ah, ah              ; CHECK FOR ANY WAIT?
                            <1>     ;jnz    short DO_WAT        ; IF THERE DO NOT ENFORCE
00004CE6 741D               <1>     jz      short HW_DONE
00004CE8 B40F               <1>     mov     ah, HD12_SETTLE             ; LOAD 1.2M HEAD SETTLE MINIMUM
00004CEA 8A87[43770100]     <1>     mov     al, [DSK_STATE+edi]     ; LOAD STATE
00004CF0 24C0               <1>     and     al, RATE_MSK            ; KEEP ONLY RATE
00004CF2 3C80               <1>     cmp     al, RATE_250            ; 1.2 M DRIVE ?
00004CF4 7502               <1>     jnz     short DO_WAT            ; DEFAULT HEAD SETTLE LOADED
                            <1> ;GP3:
00004CF6 B414               <1>     mov     ah, HD320_SETTLE        ; USE 320/360 HEAD SETTLE
                            <1> ;   jmp     short DO_WAT
                            <1>
                            <1> ;ISNT_WRITE:
                            <1> ;   or      ah, ah                  ; CHECK FOR NO WAIT
                            <1> ;   jz      short HW_DONE           ; IF NOT WRITE AND 0 ITS OK
                            <1>
                            <1> ;-----      AH CONTAINS NUMBER OF MILLISECONDS TO WAIT
                            <1> DO_WAT:
                            <1> ;   mov     al, ah                  ; AL = # MILLISECONDS
                            <1> ;   ;xor    ah, ah                  ; AX = # MILLISECONDS
                            <1> J29:                                ;      1 MILLISECOND LOOP
                            <1>     ;;mov   cx, WAIT_FDU_HEAD_SETTLE ; 33 ; 1 ms in 30 micro units.
                            <1>     ;mov    ecx, 66                 ; COUNT AT 15.085737 US PER COUNT
                            <1>     ; 11/04/2021
                            <1>     ;mov    ecx, WAIT_FDU_HEAD_SETTLE ; 33
                            <1>     ; 06/08/2022
00004CF8 29C9               <1>     sub     ecx, ecx
00004CFA B121               <1>     mov     cl, WAIT_FDU_HEAD_SETTLE ; 33
00004CFC E824D7FFFF         <1>     call    WAITF                   ; DELAY FOR 1 MILLISECOND
                            <1>     ;dec    al                      ; DECREMENT THE COUNT
00004D01 FECC               <1>     dec     ah
00004D03 75F3               <1>     jnz     short J29               ; DO AL MILLISECOND # OF TIMES
                            <1> HW_DONE:
00004D05 C3                 <1>     retn
                            <1>
                            <1> ;-----------------------------------------------------------------------------
                            <1> ; NEC_OUTPUT
                            <1> ;   THIS ROUTINE SENDS A BYTE TO THE NEC CONTROLLER AFTER TESTING
                            <1> ;   FOR CORRECT DIRECTION AND CONTROLLER READY THIS ROUTINE WILL
                            <1> ;   TIME OUT IF THE BYTE IS NOT ACCEPTED WITHIN A REASONABLE AMOUNT
                            <1> ;   OF TIME, SETTING THE DISKETTE STATUS ON COMPLETION.
                            <1> ;
                            <1> ; ON ENTRY:         AH = BYTE TO BE OUTPUT
                            <1> ;
                            <1> ; ON EXIT:  CY = 0  SUCCESS
                            <1> ;           CY = 1  FAILURE -- DISKETTE STATUS UPDATED
                            <1> ;                   IF A FAILURE HAS OCCURRED, THE RETURN IS MADE ONE LEVEL
                            <1> ;                   HIGHER THAN THE CALLER OF NEC OUTPUT. THIS REMOVES THE
                            <1> ;                   REQUIREMENT OF TESTING AFTER EVERY CALL OF NEC_OUTPUT.
                            <1> ;           AX,CX,DX DESTROYED
                            <1> ;-----------------------------------------------------------------------------
                            <1>
                            <1> ; 09/12/2014 [Erdogan Tan]
                            <1> ;   (from 'PS2 Hardware Interface Tech. Ref. May 88', Page 09-05.)
                            <1> ; Diskette Drive Controller Status Register (3F4h)
                            <1> ;   This read only register facilitates the transfer of data between
                            <1> ;   the system microprocessor and the controller.
                            <1> ; Bit 7 - When set to 1, the Data register is ready to transfer data
                            <1> ;     with the system micrprocessor.
                            <1> ; Bit 6 - The direction of data transfer. If this bit is set to 0,
                            <1> ;     the transfer is to the controller.
                            <1> ; Bit 5 - When this bit is set to 1, the controller is in the non-DMA mode.
                            <1> ; Bit 4 - When this bit is set to 1, a Read or Write command is being executed.
                            <1> ; Bit 3 - Reserved.
                            <1> ; Bit 2 - Reserved.
                            <1> ; Bit 1 - When this bit is set to 1, dskette drive 1 is in the seek mode.
                            <1> ; Bit 0 - When this bit is set to 1, dskette drive 1 is in the seek mode.
                            <1>
                            <1> ; Data Register (3F5h)
                            <1> ; This read/write register passes data, commands and parameters, and provides
                            <1> ; diskette status information.
                            <1>
                            <1> NEC_OUTPUT:
                            <1>     ; 09/08/2022
                            <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                            <1>     ;
                            <1>     ;push   bx                      ; SAVE REG.
00004D06 66BAF403           <1>     mov     dx, 03F4h               ; STATUS PORT
                            <1>     ;mov    bl,2                    ; HIGH ORDER COUNTER
                            <1>     ;xor    cx, cx                  ; COUNT FOR TIME OUT
                            <1>     ; 16/12/2014
                            <1>     ; waiting for (max.) 0.5 seconds
                            <1>         ;;mov       byte [wait_count], 0 ;; 27/02/2015
                            <1>     ;
                            <1>     ; 17/12/2014
                            <1>     ; Modified from AWARD BIOS 1999 - ADISK.ASM - SEND_COMMAND
                            <1>     ;
                            <1>     ;WAIT_FOR_PORT:         Waits for a bit at a port pointed to by DX to
                            <1>     ;               go on.
                            <1>     ;INPUT:
                            <1>     ;       AH=Mask for isolation bits.
                            <1>     ;       AL=pattern to look for.
                            <1>     ;       DX=Port to test for
                            <1>     ;       BH:CX=Number of memory refresh periods to delay.
                            <1>     ;            (normally 30 microseconds per period.)
                            <1>     ;
                            <1>     ;WFP_SHORT:
```

```
                            <1>     ;       Wait for port if refresh cycle is short (15-80 Us range).
                            <1>     ;
                            <1>
                            <1> ;   mov     bl, WAIT_FDU_SEND_HI+1; 0+1
                            <1> ;   mov     cx, WAIT_FDU_SEND_LO  ; 16667
00004D0A B91B410000         <1>     mov     ecx, WAIT_FDU_SEND_LH   ; 16667 (27/02/2015)
                            <1> ;
                            <1> ;WFPS_OUTER_LP:
                            <1> ;   ;
                            <1> ;WFPS_CHECK_PORT:
                            <1> J23:
00004D0F EC                 <1>     in      al, dx                  ; GET STATUS
00004D10 24C0               <1>     and     al, 11000000b           ; KEEP STATUS AND DIRECTION
00004D12 3C80               <1>     cmp     al, 10000000b           ; STATUS 1 AND DIRECTION 0 ?
00004D14 7418               <1>     jz      short J27               ; STATUS AND DIRECTION OK
                            <1> WFPS_HI:
00004D16 E461               <1>     in      al, PORT_B  ; 061h      ; SYS1 ; wait for hi to lo
00004D18 A810               <1>     test    al, 010h                ; transition on memory
00004D1A 75FA               <1>     jnz     short WFPS_HI           ; refresh.
                            <1> WFPS_LO:
00004D1C E461               <1>     in      al, PORT_B              ; SYS1
00004D1E A810               <1>     test    al, 010h
00004D20 74FA               <1>     jz      short WFPS_LO
                            <1>     ;loop   short WFPS_CHECK_PORT
00004D22 E2EB               <1>     loop    J23     ; 27/02/2015
                            <1> ;   ;
                            <1> ;   dec     bl
                            <1> ;   jnz     short WFPS_OUTER_LP
                            <1> ;   jmp     short WFPS_TIMEOUT      ; fail
                            <1> ;J23:
                            <1> ;   in      al, dx                  ; GET STATUS
                            <1> ;   and     al, 11000000b           ; KEEP STATUS AND DIRECTION
                            <1> ;   cmp     al, 10000000b           ; STATUS 1 AND DIRECTION 0 ?
                            <1> ;   jz      short J27               ; STATUS AND DIRECTION OK
                            <1>     ;loop   J23                     ; CONTINUE TILL CX EXHAUSTED
                            <1>     ;dec    bl                      ; DECREMENT COUNTER
                            <1>     ;jnz    short J23               ; REPEAT TILL DELAY FINISHED, CX = 0
                            <1>
                            <1>     ;;27/02/2015
                            <1>     ;16/12/2014
                            <1>         ;;cmp      byte [wait_count], 10   ; (10/18.2 seconds)
                            <1>     ;;jb    short J23
                            <1>
                            <1> ;WFPS_TIMEOUT:
                            <1>
                            <1> ;-----      FALL THRU TO ERROR RETURN
                            <1>
00004D24 800D[38770100]80   <1>     or      byte [DSKETTE_STATUS], TIME_OUT
                            <1>     ;pop    bx                      ; RESTORE REG.
00004D2B 58                 <1>     pop     eax ; 08/02/2015        ; DISCARD THE RETURN ADDRESS
00004D2C F9                 <1>     stc                             ; INDICATE ERROR TO CALLER
00004D2D C3                 <1>     retn
                            <1>
                            <1> ;-----      DIRECTION AND STATUS OK; OUTPUT BYTE
                            <1>
                            <1> J27:
00004D2E 88E0               <1>     mov     al, ah                  ; GET BYTE TO OUTPUT
                            <1>     ;inc    dx                      ; DATA PORT = STATUS PORT + 1
                            <1>     ; 06/08/2022
00004D30 FEC2               <1>     inc     dl
00004D32 EE                 <1>     out     dx, al                  ; OUTPUT THE BYTE
                            <1>     ;;NEWIODELAY  ;; 27/02/2015
                            <1>     ; 27/02/2015
                            <1>     ;pushf                          ; SAVE FLAGS
                            <1>     ; 09/08/2022
                            <1>     ; cf = 0, zf = 1
                            <1>     ;mov    ecx, 3                  ; 30 TO 45 MICROSECONDS WAIT FOR
                            <1>     ; 11/04/2021
                            <1>     ;mov    ecx, 2
                            <1>     ; 06/08/2022
00004D33 29C9               <1>     sub     ecx, ecx
00004D35 B102               <1>     mov     cl, 2
00004D37 E8E9D6FFFF         <1>     call    WAITF                   ; NEC FLAGS UPDATE CYCLE
                            <1>     ; 09/08/2022
                            <1>     ; cf = 0, zf = 1
                            <1>     ;popf                           ; RESTORE FLAGS FOR EXIT
                            <1>     ;pop    bx                      ; RESTORE REG
00004D3C C3                 <1>     retn                            ; CY = 0 FROM TEST INSTRUCTION
                            <1>
                            <1> ;-------------------------------------------------------------------------------
                            <1> ; SEEK
                            <1> ;   THIS ROUTINE WILL MOVE THE HEAD ON THE NAMED DRIVE TO THE NAMED
                            <1> ;   TRACK. IF THE DRIVE HAS NOT BEEN ACCESSED SINCE THE DRIVE
                            <1> ;   RESET COMMAND WAS ISSUED, THE DRIVE WILL BE RECALIBRATED.
                            <1> ;
                            <1> ; ON ENTRY: DI = DRIVE #
                            <1> ;           CH = TRACK #
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION.
                            <1> ;           AX,BX,CX DX DESTROYED
                            <1> ;-------------------------------------------------------------------------------
                            <1> SEEK:
00004D3D 89FB               <1>     mov     ebx, edi                ; BX = DRIVE #
00004D3F B001               <1>     mov     al, 1                   ; ESTABLISH MASK FOR RECALIBRATE TEST
                            <1>     ; 06/08/2022
                            <1>     ;xchg   cl, bl                  ; SET DRIVE VALUE INTO CL
                            <1>     ;rol    al, cl                  ; SHIFT MASK BY THE DRIVE VALUE
                            <1>     ;xchg   cl, bl                  ; RECOVER TRACK VALUE
                            <1>     ; 06/08/2022
00004D41 84C3               <1>     test    bl, al ; test bl, 1
00004D43 7402               <1>     jz      short seek_0
00004D45 FEC0               <1>     inc     al ; shl al, 1
                            <1> seek_0:
00004D47 8405[35770100]     <1>     test    al, [SEEK_STATUS]       ; TEST FOR RECALIBRATE REQUIRED
00004D4D 7526               <1>     jnz     short J28A              ; JUMP IF RECALIBRATE NOT REQUIRED
                            <1>
00004D4F 0805[35770100]     <1>     or      [SEEK_STATUS], al       ; TURN ON THE NO RECALIBRATE BIT IN FLAG
00004D55 E862000000         <1>     call    RECAL                   ; RECALIBRATE DRIVE
00004D5A 730E               <1>     jnc     short AFT_RECAL               ; RECALIBRATE DONE
                            <1>
                            <1> ;-----      ISSUE RECALIBRATE FOR 80 TRACK DISKETTES
                            <1>
00004D5C C605[38770100]00   <1>     mov     byte [DSKETTE_STATUS], 0 ; CLEAR OUT INVALID STATUS
00004D63 E854000000         <1>     call    RECAL                   ; RECALIBRATE DRIVE
00004D68 7251               <1>     jc      short RB                ; IF RECALIBRATE FAILS TWICE THEN ERROR
                            <1>
                            <1> AFT_RECAL:
00004D6A C687[45770100]00   <1>         mov     byte [DSK_TRK+edi], 0    ; SAVE NEW CYLINDER AS PRESENT POSITION
00004D71 08ED               <1>     or      ch, ch                  ; CHECK FOR SEEK TO TRACK 0
00004D73 743F               <1>     jz      short DO_WAIT           ; HEAD SETTLE, CY = 0 IF JUMP
                            <1>
                            <1> ;-----      DRIVE IS IN SYNCHRONIZATION WITH CONTROLLER, SEEK TO TRACK
                            <1>
00004D75 F687[43770100]20   <1> J28A:       test    byte [DSK_STATE+edi], DBL_STEP ; CHECK FOR DOUBLE STEP REQUIRED
```

```
00004D7C 7402                <1>     jz      short _R7             ; SINGLE STEP REQUIRED BYPASS DOUBLE
00004D7E D0E5                <1>     shl     ch, 1                 ; DOUBLE NUMBER OF STEP TO TAKE
                             <1>
00004D80 3AAF[45770100]      <1> _R7:cmp     ch, [DSK_TRK+edi]     ; SEE IF ALREADY AT THE DESIRED TRACK
00004D86 7433                <1>     je      short RB              ; IF YES, DO NOT NEED TO SEEK
                             <1>
00004D88 BA[BB4D0000]        <1>     mov     edx, NEC_ERR          ; LOAD RETURN ADDRESS
00004D8D 52                  <1>     push    edx ; (*)             ; ON STACK FOR NEC OUTPUT ERROR
00004D8E 88AF[45770100]      <1>     mov     [DSK_TRK+edi], ch     ; SAVE NEW CYLINDER AS PRESENT POSITION
00004D94 B40F                <1>     mov     ah, 0Fh               ; SEEK COMMAND TO NEC
00004D96 E86BFFFFFF          <1>     call    NEC_OUTPUT
00004D9B 89FB                <1>     mov     ebx, edi              ; BX = DRIVE #
00004D9D 88DC                <1>     mov     ah, bl                ; OUTPUT DRIVE NUMBER
00004D9F E862FFFFFF          <1>     call    NEC_OUTPUT
00004DA4 8AA7[45770100]      <1>     mov     ah, [DSK_TRK+edi]     ; GET CYLINDER NUMBER
00004DAA E857FFFFFF          <1>     call    NEC_OUTPUT
00004DAF E827000000          <1>     call    CHK_STAT_2            ; ENDING INTERRUPT AND SENSE STATUS
                             <1>
                             <1> ;-----     WAIT FOR HEAD SETTLE
                             <1>
                             <1> DO_WAIT:
00004DB4 9C                  <1>     pushf                         ; SAVE STATUS
00004DB5 E81AFFFFFF          <1>     call    HD_WAIT               ; WAIT FOR HEAD SETTLE TIME
00004DBA 9D                  <1>     popf                          ; RESTORE STATUS
                             <1> RB:
                             <1> NEC_ERR:
                             <1>     ; 08/02/2015 (code trick here from original IBM PC/AT DISKETTE.ASM)
                             <1>     ; (*) nec_err -> retn (push edx -> pop edx) -> nec_err -> retn
00004DBB C3                  <1>     retn                          ; RETURN TO CALLER
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; RECAL
                             <1> ;   RECALIBRATE DRIVE
                             <1> ;
                             <1> ; ON ENTRY: DI = DRIVE #
                             <1> ;
                             <1> ; ON EXIT:  CY REFLECTS STATUS OF OPERATION.
                             <1> ;--------------------------------------------------------------------------------
                             <1> RECAL:
                             <1>     ;push   cx
                             <1>     ; 11/04/2021
00004DBC 51                  <1>     push    ecx
00004DBD B8[D94D0000]        <1>     mov     eax, RC_BACK          ; LOAD NEC_OUTPUT ERROR
00004DC2 50                  <1>     push    eax
00004DC3 B407                <1>     mov     ah, 07h               ; RECALIBRATE COMMAND
00004DC5 E83CFFFFFF          <1>     call    NEC_OUTPUT
00004DCA 89FB                <1>     mov     ebx, edi              ; BX = DRIVE #
00004DCC 88DC                <1>     mov     ah, bl
00004DCE E833FFFFFF          <1>     call    NEC_OUTPUT            ; OUTPUT THE DRIVE NUMBER
00004DD3 E803000000          <1>     call    CHK_STAT_2            ; GET THE INTERRUPT AND SENSE INT STATUS
00004DD8 58                  <1>     pop     eax                   ; THROW AWAY ERROR
                             <1> RC_BACK:
                             <1>     ;pop    cx
                             <1>     ; 11/04/2021
00004DD9 59                  <1>     pop     ecx
00004DDA C3                  <1>     retn
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; CHK_STAT_2
                             <1> ;   THIS ROUTINE HANDLES THE INTERRUPT RECEIVED AFTER RECALIBRATE,
                             <1> ;   OR SEEK TO THE ADAPTER. THE INTERRUPT IS WAITED FOR, THE
                             <1> ;   INTERRUPT STATUS SENSED, AND THE RESULT RETURNED TO THE CALLER.
                             <1> ;
                             <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION.
                             <1> ;--------------------------------------------------------------------------------
                             <1> CHK_STAT_2:
00004DDB B8[034E0000]        <1>         mov     eax, CS_BACK            ; LOAD NEC_OUTPUT ERROR ADDRESS
00004DE0 50                  <1>     push    eax
00004DE1 E828000000          <1>     call    WAIT_INT              ; WAIT FOR THE INTERRUPT
00004DE6 721A                <1>     jc      short J34             ; IF ERROR, RETURN IT
00004DE8 B408                <1>     mov     ah, 08h               ; SENSE INTERRUPT STATUS COMMAND
00004DEA E817FFFFFF          <1>     call    NEC_OUTPUT
00004DEF E849000000          <1>     call    RESULTS               ; READ IN THE RESULTS
00004DF4 720C                <1>     jc      short J34
00004DF6 A0[39770100]        <1>     mov     al, [NEC_STATUS]      ; GET THE FIRST STATUS BYTE
00004DFB 2460                <1>     and     al, 01100000B         ; ISOLATE THE BITS
00004DFD 3C60                <1>     cmp     al, 01100000B         ; TEST FOR CORRECT VALUE
00004DFF 7403                <1>     jz      short J35             ; IF ERROR, GO MARK IT
00004E01 F8                  <1>     clc                           ; GOOD RETURN
                             <1> J34:
00004E02 58                  <1>     pop     eax                   ; THROW AWAY ERROR RETURN
                             <1> CS_BACK:
00004E03 C3                  <1>     retn
                             <1> J35:
00004E04 800D[38770100]40    <1>     or      byte [DSKETTE_STATUS], BAD_SEEK
00004E0B F9                  <1>     stc                           ; ERROR RETURN CODE
00004E0C EBF4                <1>     jmp     short J34
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; WAIT_INT
                             <1> ;   THIS ROUTINE WAITS FOR AN INTERRUPT TO OCCUR A TIME OUT ROUTINE
                             <1> ;   TAKES PLACE DURING THE WAIT, SO THAT AN ERROR MAY BE RETURNED
                             <1> ;   IF THE DRIVE IS NOT READY.
                             <1> ;
                             <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION.
                             <1> ;--------------------------------------------------------------------------------
                             <1>
                             <1> ; 17/12/2014
                             <1> ; 2.5 seconds waiting !
                             <1> ;(AWARD BIOS - 1999, WAIT_FDU_INT_LOW, WAIT_FDU_INT_HI)
                             <1> ; amount of time to wait for completion interrupt from NEC.
                             <1>
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1> WAIT_INT:
00004E0E FB                  <1>     sti                           ; TURN ON INTERRUPTS, JUST IN CASE
                             <1>     ; 06/08/2022
                             <1>     ;clc                          ; CLEAR TIMEOUT INDICATOR
                             <1>     ;
                             <1>     ;mov    bl, 10                ; CLEAR THE COUNTERS
                             <1>     ;xor    cx, cx                ; FOR 2 SECOND WAIT
                             <1>
                             <1>     ; Modification from AWARD BIOS - 1999 (ATORGS.ASM, WAIT
                             <1>     ;
                             <1>     ;WAIT_FOR_MEM:
                             <1>     ;       Waits for a bit at a specified memory location pointed
                             <1>     ;       to by ES:[DI] to become set.
                             <1>     ;INPUT:
                             <1>     ;       AH= Mask to test with.
                             <1>     ;       ES:[DI] = memory location to watch.
                             <1>     ;       BH:CX= Number of memory refresh periods to delay.
                             <1>     ;            (normally 30 microseconds per period.)
                             <1>
                             <1>     ; waiting for (max.) 2.5 secs in 30 micro units.
                             <1> ;   mov     cx, WAIT_FDU_INT_LO           ; 017798
```

```
                            <1> ;;  mov     bl, WAIT_FDU_INT_HI
                            <1> ;   mov     bl, WAIT_FDU_INT_HI + 1
                            <1>     ; 27/02/2015
00004E0F B986450100         <1>     mov     ecx, WAIT_FDU_INT_LH  ; 83334 (2.5 seconds)
                            <1> WFMS_CHECK_MEM:
00004E14 F605[35770100]80   <1>     test    byte [SEEK_STATUS], INT_FLAG
                            <1>                                   ; TEST FOR INTERRUPT OCCURRING
00004E1B 7516               <1>         jnz     short J37
                            <1> WFMS_HI:
00004E1D E461               <1>     in      al, PORT_B  ; 061h    ; SYS1, wait for lo to hi
00004E1F A810               <1>     test    al, 010h              ; transition on memory
00004E21 75FA               <1>     jnz     short WFMS_HI         ; refresh.
                            <1> WFMS_LO:
00004E23 E461               <1>     in      al, PORT_B            ; SYS1
00004E25 A810               <1>     test    al, 010h
00004E27 74FA               <1>     jz      short WFMS_LO
00004E29 E2E9               <1>         loop       WFMS_CHECK_MEM
                            <1>
                            <1> ;WFMS_OUTER_LP:
                            <1> ;;  or      bl, bl                ; check outer counter
                            <1> ;;  jz      short J36A            ; WFMS_TIMEOUT
                            <1> ;   dec     bl
                            <1> ;   jz      short J36A
                            <1> ;   jmp     short WFMS_CHECK_MEM
                            <1>
                            <1>     ; 17/12/2014
                            <1>     ; 16/12/2014
                            <1> ;   mov     byte [wait_count], 0  ; Reset (INT 08H) counter
                            <1> ;J36:
                            <1> ;   test    byte [SEEK_STATUS], INT_FLAG
                            <1> ;                                 ; TEST FOR INTERRUPT OCCURRING
                            <1> ;   jnz     short J37
                            <1>
                            <1>     ; 16/12/2014
                            <1>     ;loop   J36                   ; COUNT DOWN WHILE WAITING
                            <1>     ;dec    bl                    ; SECOND LEVEL COUNTER
                            <1>     ;jnz    short J36
                            <1> ;       cmp     byte [wait_count], 46   ; (46/18.2 seconds)
                            <1> ;   jb      short J36
                            <1>
                            <1> ;WFMS_TIMEOUT:
                            <1> ;J36A:
00004E2B 800D[38770100]80   <1>     or      byte [DSKETTE_STATUS], TIME_OUT ; NOTHING HAPPENED
00004E32 F9                 <1>     stc                           ; ERROR RETURN
                            <1> J37:
00004E33 9C                 <1>     pushf                         ; SAVE CURRENT CARRY
00004E34 8025[35770100]7F   <1>     and     byte [SEEK_STATUS], ~INT_FLAG ; TURN OFF INTERRUPT FLAG
00004E3B 9D                 <1>     popf                          ; RECOVER CARRY
00004E3C C3                 <1>     retn                          ; GOOD RETURN CODE
                            <1>
                            <1> ;--------------------------------------------------------------------------
                            <1> ; RESULTS
                            <1> ;   THIS ROUTINE WILL READ ANYTHING THAT THE NEC CONTROLLER RETURNS
                            <1> ;   FOLLOWING AN INTERRUPT.
                            <1> ;
                            <1> ; ON EXIT:  @DSKETTE_STATUS, CY REFLECT STATUS OF OPERATION.
                            <1> ;           AX,BX,CX,DX DESTROYED
                            <1> ;--------------------------------------------------------------------------
                            <1> RESULTS:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
00004E3D 57                 <1>     push    edi
00004E3E BF[39770100]       <1>     mov     edi, NEC_STATUS               ; POINTER TO DATA AREA
00004E43 B307               <1>     mov     bl, 7                 ; MAX STATUS BYTES
00004E45 66BAF403           <1>     mov     dx, 03F4h             ; STATUS PORT
                            <1>
                            <1> ;-----      WAIT FOR REQUEST FOR MASTER
                            <1>
                            <1> _R10:
                            <1>     ; 16/12/2014
                            <1>     ; wait for (max) 0.5 seconds
                            <1>     ;mov    bh, 2                 ; HIGH ORDER COUNTER
                            <1>     ;xor    cx, cx                ; COUNTER
                            <1>
                            <1>     ;Time to wait while waiting for each byte of NEC results = .5
                            <1>     ;seconds.  .5 seconds = 500,000 micros.  500,000/30 = 16,667.
                            <1>     ; 27/02/2015
00004E49 B91B410000         <1>     mov     ecx, WAIT_FDU_RESULTS_LH ; 16667
                            <1>     ;mov    cx, WAIT_FDU_RESULTS_LO  ; 16667
                            <1>     ;mov    bh, WAIT_FDU_RESULTS_HI+1 ; 0+1
                            <1>
                            <1> WFPSR_OUTER_LP:
                            <1>     ;
                            <1> WFPSR_CHECK_PORT:
                            <1> J39:                              ; WAIT FOR MASTER
00004E4E EC                 <1>     in      al, dx                ; GET STATUS
00004E4F 24C0               <1>     and     al, 11000000b         ; KEEP ONLY STATUS AND DIRECTION
00004E51 3CC0               <1>     cmp     al, 11000000b         ; STATUS 1 AND DIRECTION 1 ?
00004E53 7418               <1>     jz      short J42             ; STATUS AND DIRECTION OK
                            <1> WFPSR_HI:
00004E55 E461               <1>     in      al, PORT_B     ; 061h ; SYS1 ; wait for hi to lo
00004E57 A810               <1>     test    al, 010h              ; transition on memory
00004E59 75FA               <1>     jnz     short WFPSR_HI        ; refresh.
                            <1> WFPSR_LO:
00004E5B E461               <1>     in      al, PORT_B            ; SYS1
00004E5D A810               <1>     test    al, 010h
00004E5F 74FA               <1>     jz      SHORT WFPSR_LO
00004E61 E2EB               <1>     loop    WFPSR_CHECK_PORT
                            <1>
                            <1>     ;; 27/02/2015
                            <1>     ;;dec   bh
                            <1>     ;;jnz   short WFPSR_OUTER_LP
                            <1>     ;jmp    short WFPSR_TIMEOUT   ; fail
                            <1>
                            <1>     ;;mov   byte [wait_count], 0
                            <1> ;J39:                                     ; WAIT FOR MASTER
                            <1> ;   in      al, dx                ; GET STATUS
                            <1> ;   and     al, 11000000b         ; KEEP ONLY STATUS AND DIRECTION
                            <1> ;   cmp     al, 11000000b         ; STATUS 1 AND DIRECTION 1 ?
                            <1> ;   jz      short J42             ; STATUS AND DIRECTION OK
                            <1>     ;loop   J39                   ; LOOP TILL TIMEOUT
                            <1>     ;dec    bh                    ; DECREMENT HIGH ORDER COUNTER
                            <1>     ;jnz    short J39             ; REPEAT TILL DELAY DONE
                            <1>     ;
                            <1>     ;;cmp   byte [wait_count], 10  ; (10/18.2 seconds)
                            <1>     ;;jb    short J39
                            <1>
                            <1> ;WFPSR_TIMEOUT:
00004E63 800D[38770100]80   <1>     or      byte [DSKETTE_STATUS], TIME_OUT
00004E6A F9                 <1>     stc                           ; SET ERROR RETURN
00004E6B EB28               <1>     jmp     short POPRES          ; POP REGISTERS AND RETURN
                            <1>
                            <1> ;-----      READ IN THE STATUS
                            <1>
                            <1> J42:
```

```
00004E6D EB00                <1>     jmp    $+2                  ; I/O DELAY
                             <1>     ;inc   dx                   ; POINT AT DATA PORT
                             <1>     ; 06/08/2022
00004E6F FEC2                <1>     inc    dl
00004E71 EC                  <1>     in     al, dx               ; GET THE DATA
                             <1>     ; 16/12/2014
                             <1>     NEWIODELAY
  83 00004E72 E6EB           <2>  out 0EBh, al
00004E74 8807                <1>         mov     [edi], al       ; STORE THE BYTE
00004E76 47                  <1>     inc    edi                  ; INCREMENT THE POINTER
                             <1>
                             <1>     ; 16/12/2014
                             <1> ;   push   cx
                             <1> ;   mov    cx, 30
                             <1> ;wdw2:
                             <1> ;   NEWIODELAY
                             <1> ;   loop   wdw2
                             <1> ;   pop    cx
                             <1>
                             <1>     ;;mov  ecx, 3               ; MINIMUM 24 MICROSECONDS FOR NEC
                             <1>     ; 11/04/2021
                             <1>     ;mov   ecx, 2
                             <1>     ; 06/08/2022
00004E77 29C9                <1>     sub    ecx, ecx
00004E79 B102                <1>     mov    cl, 2
00004E7B E8A5D5FFFF          <1>     call   WAITF                ; WAIT 30 TO 45 MICROSECONDS
                             <1>     ;dec   dx                   ; POINT AT STATUS PORT
                             <1>     ; 06/08/2022
00004E80 FECA                <1>     dec    dl
00004E82 EC                  <1>     in     al, dx               ; GET STATUS
                             <1>     ; 16/12/2014
                             <1>     NEWIODELAY
  83 00004E83 E6EB           <2>  out 0EBh, al
                             <1>     ;
00004E85 A810                <1>     test   al, 00010000b        ; TEST FOR NEC STILL BUSY
00004E87 740C                <1>     jz     short POPRES         ; RESULTS DONE ?
                             <1>
00004E89 FECB                <1>     dec    bl                   ; DECREMENT THE STATUS COUNTER
00004E8B 75BC                <1>         jnz     short _R10              ; GO BACK FOR MORE
00004E8D 800D[38770100]20    <1>     or     byte [DSKETTE_STATUS], BAD_NEC ; TOO MANY STATUS BYTES
00004E94 F9                  <1>     stc                         ; SET ERROR FLAG
                             <1>
                             <1> ;-----     RESULT OPERATION IS DONE
                             <1> POPRES:
00004E95 5F                  <1>     pop    edi
00004E96 C3                  <1>     retn                        ; RETURN WITH CARRY SET
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; READ_DSKCHNG
                             <1> ;   READS THE STATE OF THE DISK CHANGE LINE.
                             <1> ;
                             <1> ; ON ENTRY: DI = DRIVE #
                             <1> ;
                             <1> ; ON EXIT:  DI = DRIVE #
                             <1> ;           ZF = 0 : DISK CHANGE LINE INACTIVE
                             <1> ;           ZF = 1 : DISK CHANGE LINE ACTIVE
                             <1> ;           AX,CX,DX DESTROYED
                             <1> ;--------------------------------------------------------------------------------
                             <1> READ_DSKCHNG:
00004E97 E8AFFDFFFF          <1>     call   MOTOR_ON             ; TURN ON THE MOTOR IF OFF
00004E9C 66BAF703            <1>     mov    dx, 03F7h            ; ADDRESS DIGITAL INPUT REGISTER
00004EA0 EC                  <1>     in     al, dx               ; INPUT DIGITAL INPUT REGISTER
00004EA1 A880                <1>     test   al, DSK_CHG          ; CHECK FOR DISK CHANGE LINE ACTIVE
00004EA3 C3                  <1>     retn                        ; RETURN TO CALLER WITH ZERO FLAG SET
                             <1>
                             <1> fdc_int:
                             <1>       ; 30/07/2015
                             <1>       ; 16/02/2015
                             <1> ;int_0Eh: ; 11/12/2014
                             <1>
                             <1> ;--- HARDWARE INT 0EH -- ( IRQ LEVEL 6 ) -------------------------------------
                             <1> ; DISK_INT
                             <1> ;   THIS ROUTINE HANDLES THE DISKETTE INTERRUPT.
                             <1> ;
                             <1> ; ON EXIT:  THE INTERRUPT FLAG IS SET IN @SEEK_STATUS.
                             <1> ;--------------------------------------------------------------------------------
                             <1> DISK_INT_1:
                             <1>     ;push  AX                   ; SAVE WORK REGISTER
                             <1>     ; 11/04/2021
00004EA4 50                  <1>     push   eax
00004EA5 1E                  <1>     push   ds
00004EA6 66B81000            <1>     mov    ax, KDATA
00004EAA 8ED8                <1>     mov    ds, ax
00004EAC 800D[35770100]80    <1>         or  byte [SEEK_STATUS], INT_FLAG ; TURN ON INTERRUPT OCCURRED
00004EB3 B020                <1>     mov     al, EOI                     ; END OF INTERRUPT MARKER
00004EB5 E620                <1>     out    INTA00, al           ; INTERRUPT CONTROL PORT
00004EB7 1F                  <1>     pop    ds
                             <1>     ;pop   ax                   ; RECOVER REGISTER
                             <1>     ; 11/04/2021
00004EB8 58                  <1>     pop    eax
00004EB9 CF                  <1>     iretd                       ; RETURN FROM INTERRUPT
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1> ; DSKETTE_SETUP
                             <1> ;   THIS ROUTINE DOES A PRELIMINARY CHECK TO SEE WHAT TYPE OF
                             <1> ;   DISKETTE DRIVES ARE ATTACH TO THE SYSTEM.
                             <1> ;--------------------------------------------------------------------------------
                             <1>
                             <1> ; 09/08/2022 - TRDOS 386 Kernel v2.0.5
                             <1> ; 29/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>
                             <1> DSKETTE_SETUP:
                             <1>     ;push  ax                   ; SAVE REGISTERS
                             <1>     ;push  bx
                             <1>     ;push  cx
00004EBA 52                  <1>     push   edx
                             <1>     ;push  di
                             <1>     ;;push ds
                             <1>     ; 14/12/2014
                             <1>     ;mov   word [DISK_POINTER], MD_TBL6
                             <1>     ;mov   [DISK_POINTER+2], cs
                             <1>     ;
                             <1>     ;or    byte [RTC_WAIT_FLAG], 1      ; NO RTC WAIT, FORCE USE OF LOOP
00004EBB 31FF                <1>     xor    edi, edi             ; INITIALIZE DRIVE POINTER
                             <1>     ; 09/08/2022
                             <1>     ;mov   esi, eax
00004EBD 31C0                <1>     xor    eax, eax ; eax = 0
00004EBF 66A3[43770100]      <1>     mov    [DSK_STATE], ax              ; INITIALIZE STATES
00004EC5 8025[40770100]33    <1>     and    byte [LASTRATE], ~(STRT_MSK+SEND_MSK) ; CLEAR START & SEND
00004ECC 800D[40770100]C0    <1>     or     byte [LASTRATE], SEND_MSK ; INITIALIZE SENT TO IMPOSSIBLE
00004ED3 A2[35770100]        <1>     mov    [SEEK_STATUS], al    ; INDICATE RECALIBRATE NEEDED
00004ED8 A2[37770100]        <1>     mov    [MOTOR_COUNT], al    ; INITIALIZE MOTOR COUNT
00004EDD A2[36770100]        <1>     mov    [MOTOR_STATUS], al   ; INITIALIZE DRIVES TO OFF STATE
00004EE2 A2[38770100]        <1>     mov    [DSKETTE_STATUS], al ; NO ERRORS
```

```
                                <1>     ;
                                <1>     ; 28/02/2015
                                <1>     ;mov    word [cfd], 100h
00004EE7 E815F4FFFF             <1>     call    DSK_RESET
00004EEC 5A                     <1>     pop     edx
00004EED F8                     <1>     clc     ; 29/05/2016
00004EEE C3                     <1>     retn
                                <1>
                                <1> ;SUP0:
                                <1> ;   call    DRIVE_DET               ; DETERMINE DRIVE
                                <1> ;   call    XLAT_OLD                ; TRANSLATE STATE TO COMPATIBLE MODE
                                <1> ;   ; 02/01/2015
                                <1> ;   ;inc    di                      ; POINT TO NEXT DRIVE
                                <1> ;   ;cmp    di, MAX_DRV             ; SEE IF DONE
                                <1> ;   ;jnz    short SUP0              ; REPEAT FOR EACH ORIVE
                                <1> ;       cmp     byte [fd1_type], 0
                                <1> ;   jna     short sup1
                                <1> ;   or      di, di
                                <1> ;   jnz     short sup1
                                <1> ;   inc     di
                                <1> ;       jmp     short SUP0
                                <1> ;sup1:
                                <1> ;   mov     byte [SEEK_STATUS], 0 ; FORCE RECALIBRATE
                                <1> ;   ;and    byte [RTC_WAIT_FLAG], 0FEh ; ALLOW FOR RTC WAIT
                                <1> ;   call    SETUP_END               ; VARIOUS CLEANUPS
                                <1> ;   ;;pop   ds                      ; RESTORE CALLERS REGISTERS
                                <1> ;   ;pop    di
                                <1> ;   pop     edx
                                <1> ;   ;pop    cx
                                <1> ;   ;pop    bx
                                <1> ;   ;pop    ax
                                <1> ;   retn
                                <1>
                                <1> ;////////////////////////////////////////////////////
                                <1> ;; END OF DISKETTE I/O ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
                                <1>
                                <1> ; 17/04/2021 (TRDOS 386 v2.0.4)
                                <1>
                                <1> ; 11/04/2021
                                <1> ;int13h: ; 21/02/2015
                                <1>     ;pushfd
                                <1>     ;push   cs
                                <1>     ;;call  DISK_IO
                                <1>     ;;retn
                                <1>
                                <1> ;;;;;; DISK I/O ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;; 21/02/2015 ;;;
                                <1> ;////////////////////////////////////////////////////////////////
                                <1>
                                <1> ; DISK I/O - Erdogan Tan (Retro UNIX 386 v1 project)
                                <1> ; 18/02/2016
                                <1> ; 17/02/2016
                                <1> ; 23/02/2015
                                <1> ; 21/02/2015 (unix386.s)
                                <1> ; 22/12/2014 - 14/02/2015 (dsectrm2.s)
                                <1> ;
                                <1> ; Original Source Code:
                                <1> ; DISK ----- 09/25/85 FIXED DISK BIOS
                                <1> ; (IBM PC XT Model 286 System BIOS Source Code, 04-21-86)
                                <1> ;
                                <1> ; Modifications: by reference of AWARD BIOS 1999 (D1A0622)
                                <1> ;            Source Code - ATORGS.ASM, AHDSK.ASM
                                <1> ;
                                <1>
                                <1> ;The wait for controller to be not busy is 10 seconds.
                                <1> ;10,000,000 / 30 = 333,333. 333,333 decimal = 051615h
                                <1> ;;WAIT_HDU_CTLR_BUSY_LO   equ 1615h
                                <1> ;;WAIT_HDU_CTLR_BUSY_HI   equ 05h
                                <1> WAIT_HDU_CTRL_BUSY_LH     equ 51615h     ; 21/02/2015
                                <1>
                                <1> ;The wait for controller to issue completion interrupt is 10 seconds.
                                <1> ;10,000,000 / 30 = 333,333.  333,333 decimal = 051615h
                                <1> ;;WAIT_HDU_INT_LO  equ 1615h
                                <1> ;;WAIT_HDU_INT_HI  equ 05h
                                <1> WAIT_HDU_INT_LH           equ 51615h     ; 21/02/2015
                                <1>
                                <1> ;The wait for Data request on read and write longs is
                                <1> ;2000 us. (?)
                                <1> ;;WAIT_HDU_DRQ_LO  equ 1000       ; 03E8h
                                <1> ;;WAIT_HDU_DRQ_HI  equ 0
                                <1> WAIT_HDU_DRQ_LH           equ 1000       ; 21/02/2015
                                <1>
                                <1> ; Port 61h (PORT_B)
                                <1> SYS1equ 61h        ; PORT_B  (diskette.inc)
                                <1>
                                <1> ; 23/12/2014
                                <1> %define CMD_BLOCK       ebp-8  ; 21/02/2015
                                <1>
                                <1> ;--- INT 13H ----------------------------------------------------------------
                                <1> ;                                                                       :
                                <1> ; FIXED DISK I/O INTERFACE                                              :
                                <1> ;                                                                       :
                                <1> ;   THIS INTERFACE PROVIDES ACCESS TO 5 1/4" FIXED DISKS THROUGH           :
                                <1> ;   THE IBM FIXED DISK CONTROLLER.                                          :
                                <1> ;                                                                       :
                                <1> ;   THE  BIOS  ROUTINES  ARE  MEANT  TO  BE  ACCESSED  THROUGH          :
                                <1> ;   SOFTWARE  INTERRUPTS  ONLY.    ANY  ADDRESSES  PRESENT     IN           :
                                <1> ;   THESE  LISTINGS  ARE  INCLUDED        ONLY  FOR  COMPLETENESS,          :
                                <1> ;   NOT  FOR  REFERENCE.  APPLICATIONS   WHICH  REFERENCE  ANY          :
                                <1> ;   ABSOLUTE  ADDRESSES  WITHIN  THE  CODE      SEGMENTS  OF  BIOS        :
                                <1> ;   VIOLATE  THE  STRUCTURE  AND  DESIGN  OF  BIOS.                     :
                                <1> ;                                                                       :
                                <1> ;-----------------------------------------------------------------------:
                                <1> ;                                                                       :
                                <1> ; INPUT  (AH)= HEX COMMAND VALUE                                        :
                                <1> ;                                                                       :
                                <1> ;   (AH)= 00H  RESET DISK (DL = 80H,81H) / DISKETTE                     :
                                <1> ;   (AH)= 01H  READ THE STATUS OF THE LAST DISK OPERATION INTO (AL)       :
                                <1> ;              NOTE: DL < 80H - DISKETTE                                :
                                <1> ;                    DL > 80H - DISK                                    :
                                <1> ;   (AH)= 02H  READ THE DESIRED SECTORS INTO MEMORY                     :
                                <1> ;   (AH)= 03H  WRITE THE DESIRED SECTORS FROM MEMORY                    :
                                <1> ;   (AH)= 04H  VERIFY THE DESIRED SECTORS                                   :
                                <1> ;   (AH)= 05H  FORMAT THE DESIRED TRACK                                 :
                                <1> ;   (AH)= 06H  UNUSED                                                   :
                                <1> ;   (AH)= 07H  UNUSED                                                   :
                                <1> ;   (AH)= 08H  RETURN THE CURRENT DRIVE PARAMETERS                          :
                                <1> ;   (AH)= 09H  INITIALIZE DRIVE PAIR CHARACTERISTICS                    :
                                <1> ;              INTERRUPT 41 POINTS TO DATA BLOCK FOR DRIVE 0            :
                                <1> ;              INTERRUPT 46 POINTS TO DATA BLOCK FOR DRIVE 1            :
                                <1> ;   (AH)= 0AH  READ LONG                                                :
                                <1> ;   (AH)= 0BH  WRITE LONG  (READ & WRITE LONG ENCOMPASS 512 + 4 BYTES ECC) :
                                <1> ;   (AH)= 0CH  SEEK                                                     :
                                <1> ;   (AH)= 0DH  ALTERNATE DISK RESET (SEE DL)                            :
                                <1> ;   (AH)= 0EH  UNUSED                                                   :
```

```
<1> ;   (AH)= 0FH  UNUSED                                                :
<1> ;   (AH)= 10H  TEST DRIVE READY                                      :
<1> ;   (AH)= 11H  RECALIBRATE                                           :
<1> ;   (AH)= 12H  UNUSED                                                :
<1> ;   (AH)= 13H  UNUSED                                                :
<1> ;   (AH)= 14H  CONTROLLER INTERNAL DIAGNOSTIC                        :
<1> ;   (AH)= 15H  READ DASD TYPE                                        :
<1> ;                                                                    :
<1> ;-----------------------------------------------------------------------------
<1> ;                                                                    :
<1> ;   REGISTERS USED FOR FIXED DISK OPERATIONS                         :
<1> ;                                                                    :
<1> ;           (DL)    -  DRIVE NUMBER     (80H-81H FOR DISK. VALUE CHECKED)  :
<1> ;           (DH)    -  HEAD NUMBER     (0-15 ALLOWED, NOT VALUE CHECKED)  :
<1> ;           (CH)    -  CYLINDER NUMBER  (0-1023, NOT VALUE CHECKED)(SEE CL):
<1> ;           (CL)    -  SECTOR NUMBER    (1-17, NOT VALUE CHECKED)              :
<1> ;                                                                    :
<1> ;                      NOTE: HIGH 2 BITS OF CYLINDER NUMBER ARE PLACED     :
<1> ;                            IN THE HIGH 2 BITS OF THE CL REGISTER             :
<1> ;                            (10 BITS TOTAL)                         :
<1> ;                                                                    :
<1> ;           (AL)    -  NUMBER OF SECTORS (MAXIMUM POSSIBLE RANGE 1-80H,    :
<1> ;                                         FOR READ/WRITE LONG 1-79H)       :
<1> ;                                                                    :
<1> ;           (ES:BX) -  ADDRESS OF BUFFER FOR READS AND WRITES,       :
<1> ;                      (NOT REQUIRED FOR VERIFY)                     :
<1> ;                                                                    :
<1> ;           FORMAT (AH=5) ES:BX POINTS TO A 512 BYTE BUFFER. THE FIRST     :
<1> ;                      2*(SECTORS/TRACK) BYTES CONTAIN F,N FOR EACH SECTOR.:
<1> ;                      F = 00H FOR A GOOD SECTOR                     :
<1> ;                          80H FOR A BAD SECTOR                      :
<1> ;                      N = SECTOR NUMBER                             :
<1> ;                      FOR AN INTERLEAVE OF 2 AND 17 SECTORS/TRACK             :
<1> ;                      THE TABLE SHOULD BE:                          :
<1> ;                                                                    :
<1> ;              DB      00H,01H,00H,0AH,00H,02H,00H,0BH,00H,03H,00H,0CH     :
<1> ;              DB      00H,04H,00H,0DH,00H,05H,00H,0EH,00H,06H,00H,0FH     :
<1> ;              DB      00H,07H,00H,10H,00H,08H,00H,11H,00H,09H      :
<1> ;                                                                    :
<1> ;-----------------------------------------------------------------------------
<1>
<1> ;-----------------------------------------------------------------------------
<1> ; OUTPUT                                                             :
<1> ;   AH = STATUS OF CURRENT OPERATION                                 :
<1> ;        STATUS BITS ARE DEFINED IN THE EQUATES BELOW                :
<1> ;   CY = 0  SUCCESSFUL OPERATION (AH=0 ON RETURN)                          :
<1> ;   CY = 1  FAILED OPERATION (AH HAS ERROR REASON)                         :
<1> ;                                                                    :
<1> ;   NOTE:   ERROR 11H  INDICATES THAT THE DATA READ HAD A RECOVERABLE      :
<1> ;           ERROR WHICH WAS CORRECTED BY THE ECC ALGORITHM.  THE DATA     :
<1> ;           IS PROBABLY GOOD,   HOWEVER THE BIOS ROUTINE INDICATES AN     :
<1> ;           ERROR TO ALLOW THE CONTROLLING PROGRAM A CHANCE TO DECIDE     :
<1> ;           FOR ITSELF.  THE  ERROR  MAY  NOT  RECUR  IF  THE DATA IS     :
<1> ;           REWRITTEN.                                               :
<1> ;                                                                    :
<1> ;   IF DRIVE PARAMETERS WERE REQUESTED (DL >= 80H),                  :
<1> ;      INPUT:                                                        :
<1> ;        (DL) = DRIVE NUMBER                                         :
<1> ;        ; 27/05/2016 - TRDOS 386 (TRDOS v2.0)                                  :
<1> ;        EBX = Buffer address for fixed disk parameters table (32 bytes)   :
<1> ;      OUTPUT:                                                       :
<1> ;        (DL) = NUMBER OF CONSECUTIVE ACKNOWLEDGING DRIVES ATTACHED (1-2)  :
<1> ;              (CONTROLLER CARD ZERO TALLY ONLY)                          :
<1> ;        (DH) = MAXIMUM USEABLE VALUE FOR HEAD NUMBER                :
<1> ;        (CH) = MAXIMUM USEABLE VALUE FOR CYLINDER NUMBER                 :
<1> ;        (CL) = MAXIMUM USEABLE VALUE FOR SECTOR NUMBER              :
<1> ;              AND CYLINDER NUMBER HIGH BITS                         :
<1> ;                                                                    :
<1> ;   IF READ DASD TYPE WAS REQUESTED,                                 :
<1> ;                                                                    :
<1> ;   AH = 0 - NOT PRESENT                                              :
<1> ;        1 - DISKETTE - NO CHANGE LINE AVAILABLE                     :
<1> ;        2 - DISKETTE - CHANGE LINE AVAILABLE                        :
<1> ;        3 - FIXED DISK                                              :
<1> ;                                                                    :
<1> ;   CX,DX = NUMBER OF 512 BYTE BLOCKS WHEN AH = 3                            :
<1> ;                                                                    :
<1> ;   REGISTERS WILL BE PRESERVED EXCEPT WHEN THEY ARE USED TO RETURN      :
<1> ;   INFORMATION.                                                     :
<1> ;                                                                    :
<1> ;   NOTE: IF AN ERROR IS REPORTED BY THE DISK CODE, THE APPROPRIATE      :
<1> ;           ACTION IS TO RESET THE DISK, THEN RETRY THE OPERATION.           :
<1> ;                                                                    :
<1> ;-----------------------------------------------------------------------------
<1>
<1> SENSE_FAIL  EQU     0FFH            ; NOT IMPLEMENTED
<1> NO_ERR              EQU     0E0H            ; STATUS ERROR/ERROR REGISTER=0
<1> WRITE_FAULT EQU     0CCH            ; WRITE FAULT ON SELECTED DRIVE
<1> UNDEF_ERR   EQU     0BBH            ; UNDEFINED ERROR OCCURRED
<1> NOT_RDY     EQU     0AAH            ; DRIVE NOT READY
<1> TIME_OUT    EQU     80H             ; ATTACHMENT FAILED TO RESPOND
<1> BAD_SEEK    EQU     40H             ; SEEK OPERATION FAILED
<1> BAD_CNTLR   EQU     20H             ; CONTROLLER HAS FAILED
<1> DATA_CORRECTED      EQU     11H             ; ECC CORRECTED DATA ERROR
<1> BAD_ECC     EQU     10H             ; BAD ECC ON DISK READ
<1> BAD_TRACK   EQU     0BH             ; NOT IMPLEMENTED
<1> BAD_SECTOR  EQU     0AH             ; BAD SECTOR FLAG DETECTED
<1> ;DMA_BOUNDARY       EQU     09H             ; DATA EXTENDS TOO FAR
<1> INIT_FAIL   EQU     07H             ; DRIVE PARAMETER ACTIVITY FAILED
<1> BAD_RESET   EQU     05H             ; RESET FAILED
<1> ;RECORD_NOT_FND     EQU     04H             ; REQUESTED SECTOR NOT FOUND
<1> ;BAD_ADDR_MARK      EQU     02H             ; ADDRESS MARK NOT FOUND
<1> ;BAD_CMD    EQU     01H             ; BAD COMMAND PASSED TO DISK I/O
<1>
<1> ;-------------------------------------------------------
<1> ;                                                    :
<1> ; FIXED DISK PARAMETER TABLE                          :
<1> ;  -  THE TABLE IS COMPOSED OF A BLOCK DEFINED AS:    :
<1> ;                                                    :
<1> ;  +0       (1 WORD) - MAXIMUM NUMBER OF CYLINDERS          :
<1> ;  +2       (1 BYTE) - MAXIMUM NUMBER OF HEADS       :
<1> ;  +3       (1 WORD) - NOT USED/SEE PC-XT            :
<1> ;  +5       (1 WORD) - STARTING WRITE PRECOMPENSATION CYL    :
<1> ;  +7       (1 BYTE) - MAXIMUM ECC DATA BURST LENGTH :
<1> ;  +8       (1 BYTE) - CONTROL BYTE                  :
<1> ;              BIT   7 DISABLE RETRIES -OR-      :
<1> ;              BIT   6 DISABLE RETRIES           :
<1> ;              BIT   3 MORE THAN 8 HEADS         :
<1> ;  +9       (3 BYTES)- NOT USED/SEE PC-XT            :
<1> ; +12       (1 WORD) - LANDING ZONE                  :
<1> ; +14       (1 BYTE) - NUMBER OF SECTORS/TRACK       :
<1> ; +15       (1 BYTE) - RESERVED FOR FUTURE USE       :
<1> ;                                                    :
<1> ;    - TO DYNAMICALLY DEFINE A SET OF PARAMETERS:
```

```
                                <1> ;      BUILD A TABLE FOR UP TO 15 TYPES AND PLACE      :
                                <1> ;      THE CORRESPONDING VECTOR INTO INTERRUPT 41      :
                                <1> ;      FOR DRIVE 0 AND INTERRUPT 46 FOR DRIVE 1.:
                                <1> ;                                                      :
                                <1> ;------------------------------------------------------
                                <1>
                                <1> ;------------------------------------------------------
                                <1> ;                                                      :
                                <1> ; HARDWARE SPECIFIC VALUES                             :
                                <1> ;                                                      :
                                <1> ;  -  CONTROLLER I/O PORT                              :
                                <1> ;                                                      :
                                <1> ;     > WHEN READ FROM:                                :
                                <1> ;   HF_PORT+0 - READ DATA (FROM CONTROLLER TO CPU)     :
                                <1> ;   HF_PORT+1 - GET ERROR REGISTER                     :
                                <1> ;   HF_PORT+2 - GET SECTOR COUNT                       :
                                <1> ;   HF_PORT+3 - GET SECTOR NUMBER                      :
                                <1> ;   HF_PORT+4 - GET CYLINDER LOW                       :
                                <1> ;   HF_PORT+5 - GET CYLINDER HIGH (2 BITS)             :
                                <1> ;   HF_PORT+6 - GET SIZE/DRIVE/HEAD                    :
                                <1> ;   HF_PORT+7 - GET STATUS REGISTER                    :
                                <1> ;                                                      :
                                <1> ;     > WHEN WRITTEN TO:                               :
                                <1> ;   HF_PORT+0 - WRITE DATA (FROM CPU TO CONTROLLER) :
                                <1> ;   HF_PORT+1 - SET PRECOMPENSATION CYLINDER           :
                                <1> ;   HF_PORT+2 - SET SECTOR COUNT                       :
                                <1> ;   HF_PORT+3 - SET SECTOR NUMBER                      :
                                <1> ;   HF_PORT+4 - SET CYLINDER LOW                       :
                                <1> ;   HF_PORT+5 - SET CYLINDER HIGH (2 BITS)             :
                                <1> ;   HF_PORT+6 - SET SIZE/DRIVE/HEAD                    :
                                <1> ;   HF_PORT+7 - SET COMMAND REGISTER                   :
                                <1> ;                                                      :
                                <1> ;------------------------------------------------------
                                <1>
                                <1> ;HF_PORT    EQU     01F0H   ; DISK PORT
                                <1> ;HF1_PORT   equ     0170h
                                <1> ;HF_REG_PORT        EQU     03F6H
                                <1> ;HF1_REG_PORT       equ     0376h
                                <1>
                                <1> HDC1_BASEPORT       equ     1F0h
                                <1> HDC2_BASEPORT       equ     170h
                                <1>
00004EEF 90                     <1> align 2
                                <1>
                                <1> ;-----              STATUS REGISTER
                                <1>
                                <1> ST_ERROR    EQU     00000001B       ;
                                <1> ST_INDEX    EQU     00000010B       ;
                                <1> ST_CORRCTD  EQU     00000100B       ; ECC CORRECTION SUCCESSFUL
                                <1> ST_DRQ              EQU     00001000B       ;
                                <1> ST_SEEK_COMPL       EQU     00010000B       ; SEEK COMPLETE
                                <1> ST_WRT_FLT  EQU     00100000B       ; WRITE FAULT
                                <1> ST_READY    EQU     01000000B       ;
                                <1> ST_BUSY     EQU     10000000B       ;
                                <1>
                                <1> ;-----              ERROR REGISTER
                                <1>
                                <1> ERR_DAM     EQU     00000001B       ; DATA ADDRESS MARK NOT FOUND
                                <1> ERR_TRK_0   EQU     00000010B       ; TRACK 0 NOT FOUND ON RECAL
                                <1> ERR_ABORT   EQU     00000100B       ; ABORTED COMMAND
                                <1> ;           EQU     00001000B       ; NOT USED
                                <1> ERR_ID              EQU     00010000B       ; ID NOT FOUND
                                <1> ;           EQU     00100000B       ; NOT USED
                                <1> ERR_DATA_ECC        EQU     01000000B
                                <1> ERR_BAD_BLOCK       EQU     10000000B
                                <1>
                                <1>
                                <1> RECAL_CMD   EQU     00010000B       ; DRIVE RECAL  (10H)
                                <1> READ_CMD    EQU     00100000B       ;      READ    (20H)
                                <1> WRITE_CMD   EQU     00110000B       ;      WRITE   (30H)
                                <1> VERIFY_CMD  EQU     01000000B       ;      VERIFY  (40H)
                                <1> FMTTRK_CMD  EQU     01010000B       ; FORMAT TRACK (50H)
                                <1> INIT_CMD    EQU     01100000B       ;   INITIALIZE (60H)
                                <1> SEEK_CMD    EQU     01110000B       ;      SEEK    (70H)
                                <1> DIAG_CMD    EQU     10010000B       ; DIAGNOSTIC   (90H)
                                <1> SET_PARM_CMD        EQU     10010001B       ; DRIVE PARMS  (91H)
                                <1> NO_RETRIES  EQU     00000001B       ; CHD MODIFIER (01H)
                                <1> ECC_MODE    EQU     00000010B       ; CMD MODIFIER (02H)
                                <1> BUFFER_MODE EQU     00001000B       ; CMD MODIFIER (08H)
                                <1>
                                <1> ;MAX_FILE   EQU     2
                                <1> ;S_MAX_FILE EQU     2
                                <1> MAX_FILE    equ     4               ; 22/12/2014
                                <1> S_MAX_FILE  equ     4               ; 22/12/2014
                                <1>
                                <1> DELAY_1     EQU     25H             ; DELAY FOR OPERATION COMPLETE
                                <1> DELAY_2     EQU     0600H           ; DELAY FOR READY
                                <1> DELAY_3     EQU     0100H           ; DELAY FOR DATA REQUEST
                                <1>
                                <1> HF_FAIL     EQU     08H             ; CMOS FLAG IN BYTE 0EH
                                <1>
                                <1> ;-----              COMMAND BLOCK REFERENCE
                                <1>
                                <1> ;CMD_BLOCK      EQU     BP-8            ; @CMD_BLOCK REFERENCES BLOCK HEAD IN SS
                                <1>                                         ;  (BP) POINTS TO COMMAND BLOCK TAIL
                                <1>                                         ;       AS DEFINED BY THE "ENTER" PARMS
                                <1> ; 19/12/2014
                                <1> ORG_VECTOR  equ     4*13h           ; INT 13h vector
                                <1> DISK_VECTOR equ     4*40h           ; INT 40h vector (for floppy disks)
                                <1> ;HDISK_INT  equ     4*76h           ; Primary HDC - Hardware interrupt (IRQ14)
                                <1> ;HDISK_INT1 equ     4*76h           ; Primary HDC - Hardware interrupt (IRQ14)
                                <1> ;HDISK_INT2 equ     4*77h           ; Secondary HDC - Hardware interrupt (IRQ15)
                                <1> ;HF_TBL_VEC equ     4*41h           ; Pointer to 1st fixed disk parameter table
                                <1> ;HF1_TBL_VEC        equ     4*46h           ; Pointer to 2nd fixed disk parameter table
                                <1>
                                <1> align 2
                                <1>
                                <1> ;-------------------------------------------------------------
                                <1> ; FIXED DISK I/O SETUP                                        :
                                <1> ;                                                           :
                                <1> ;  -  ESTABLISH TRANSFER VECTORS FOR THE FIXED DISK           :
                                <1> ;  -  PERFORM POWER ON DIAGNOSTICS                            :
                                <1> ;     SHOULD AN ERROR OCCUR A "1701" MESSAGE IS DISPLAYED      :
                                <1> ;                                                           :
                                <1> ;-------------------------------------------------------------
                                <1>
                                <1> ; 09/08/2022
                                <1> ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 29/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>
                                <1> DISK_SETUP:
                                <1>     ;CLI
                                <1>     ;;mov   ax, ABS0                ; GET ABSOLUTE SEGMENT
                                <1>     ;xor    ax, ax
```

```
                                <1>     ;mov   ds, ax                ; SET SEGMENT REGISTER
                                <1>     ;mov   ax, [ORG_VECTOR]      ; GET DISKETTE VECTOR
                                <1>     ;mov   [DISK_VECTOR], ax     ;  INTO INT 40H
                                <1>     ;mov   ax, [ORG_VECTOR+2]
                                <1>     ;mov   [DISK_VECTOR+2], ax
                                <1>     ;mov   word [ORG_VECTOR], DISK_IO ; FIXED DISK HANDLER
                                <1>     ;mov   [ORG_VECTOR+2], cs
                                <1>     ; 1st controller (primary master, slave)   - IRQ 14
                                <1>     ;;mov  word [HDISK_INT], HD_INT   ; FIXED DISK INTERRUPT
                                <1>     ;mov   word [HDISK_INT1], HD_INT  ;
                                <1>     ;;mov  [HDISK_INT+2], cs
                                <1>     ;mov   [HDISK_INT1+2], cs
                                <1>     ; 2nd controller (secondary master, slave) - IRQ 15
                                <1>     ;mov   word [HDISK_INT2], HD1_INT ;
                                <1>     ;mov   [HDISK_INT2+2], cs
                                <1>     ;
                                <1>     ;;mov  word [HF_TBL_VEC], HD0_DPT   ; PARM TABLE DRIVE 80
                                <1>     ;;mov  word [HF_TBL_VEC+2], DPT_SEGM
                                <1>     ;;mov  word [HF1_TBL_VEC], HD1_DPT  ; PARM TABLE DRIVE 81
                                <1>     ;;mov  word [HF1_TBL_VEC+2], DPT_SEGM
                                <1>     ;push  cs
                                <1>     ;pop   ds
                                <1>     ;mov   word [HDPM_TBL_VEC], HD0_DPT  ; PARM TABLE DRIVE 80h
                                <1>     ;mov   word [HDPM_TBL_VEC+2], DPT_SEGM
00004EF0 C705[4C770100]0000-    <1>     mov    dword [HDPM_TBL_VEC], (DPT_SEGM*16)+HD0_DPT
00004EF8 0900                   <1>
                                <1>     ;mov   word [HDPS_TBL_VEC], HD1_DPT  ; PARM TABLE DRIVE 81h
                                <1>     ;mov   word [HDPS_TBL_VEC+2], DPT_SEGM
00004EFA C705[50770100]2000-    <1>     mov    dword [HDPS_TBL_VEC], (DPT_SEGM*16)+HD1_DPT
00004F02 0900                   <1>
                                <1>     ;mov   word [HDSM_TBL_VEC], HD2_DPT  ; PARM TABLE DRIVE 82h
                                <1>     ;mov   word [HDSM_TBL_VEC+2], DPT_SEGM
00004F04 C705[54770100]4000-    <1>     mov    dword [HDSM_TBL_VEC], (DPT_SEGM*16)+HD2_DPT
00004F0C 0900                   <1>
                                <1>     ;mov   word [HDSS_TBL_VEC], HD3_DPT  ; PARM TABLE DRIVE 83h
                                <1>     ;mov   word [HDSS_TBL_VEC+2], DPT_SEGM
00004F0E C705[58770100]6000-    <1>     mov    dword [HDSS_TBL_VEC], (DPT_SEGM*16)+HD3_DPT
00004F16 0900                   <1>
                                <1>     ;
                                <1>     ;;in   al, INTB01            ; TURN ON SECOND INTERRUPT CHIP
                                <1>     ;;;and al, 0BFh
                                <1>     ;;and  al, 3Fh               ; enable IRQ 14 and IRQ 15
                                <1>     ;;;jmp $+2
                                <1>     ;;IODELAY
                                <1>     ;;out  INTB01, al
                                <1>     ;;IODELAY
                                <1>     ;;in   al, INTA01            ; LET INTERRUPTS PASS THRU TO
                                <1>     ;;and  al, 0FBh              ;  SECOND CHIP
                                <1>     ;;;jmp $+2
                                <1>     ;;IODELAY
                                <1>     ;;out  INTA01, al
                                <1>     ;
                                <1>     ;sti
                                <1>     ;;push ds                    ; MOVE ABS0 POINTER TO
                                <1>     ;;pop  es                    ; EXTRA SEGMENT POINTER
                                <1>     ;;;call DDS                  ; ESTABLISH DATA SEGMENT
                                <1>     ;;mov  byte [DISK_STATUS1], 0       ; RESET THE STATUS INDICATOR
                                <1>     ;;mov  byte [HF_NUM], 0      ; ZERO NUMBER OF FIXED DISKS
                                <1>     ;;mov  byte [CONTROL_BYTE], 0
                                <1>     ;;mov  byte [PORT_OFF], 0    ; ZERO CARD OFFSET
                                <1>     ; 20/12/2014 - private code by Erdogan Tan
                                <1>                  ; (out of original PC-AT, PC-XT BIOS code)
                                <1>     ;mov   si, hd0_type
00004F18 BE[8E660000]           <1>     mov    esi, hd0_type
                                <1>     ;;mov  cx, 4
                                <1>     ;mov   ecx, 4
                                <1>     ; 06/08/2022
00004F1D 29C9                   <1>     sub    ecx, ecx
00004F1F B104                   <1>     mov    cl, 4
                                <1> hde_l:
00004F21 AC                     <1>     lodsb
00004F22 3C80                   <1>     cmp    al, 80h               ; 8?h = existing
00004F24 7206                   <1>     jb     short _L4
00004F26 FE05[48770100]         <1>     inc    byte [HF_NUM]         ; + 1 hard (fixed) disk drives
                                <1> _L4: ; 26/02/2015
00004F2C E2F3                   <1>     loop   hde_l
                                <1> ;_L4:                                   ; 0 <= [HF_NUM] =< 4
                                <1> ;L4:
                                <1>     ;; 31/12/2014 - cancel controller diagnostics here
                                <1>     ;;;mov cx, 3  ; 26/12/2014 (Award BIOS 1999)
                                <1>     ;;mov  cl, 3
                                <1>     ;;
                                <1>     ;;mov  DL, 80H               ; CHECK THE CONTROLLER
                                <1> ;;hdc_dl:
                                <1>     ;;mov  AH, 14H               ; USE CONTROLLER DIAGNOSTIC COMMAND
                                <1>     ;;INT  13H                   ; CALL BIOS WITH DIAGNOSTIC COMMAND
                                <1>     ;;;jc  short CTL_ERRX        ; DISPLAY ERROR MESSAGE IF BAD RETURN
                                <1>     ;;;jc  short POD_DONE ;22/12/2014
                                <1>     ;;jnc  short hdc_reset0
                                <1>     ;;loop hdc_dl
                                <1>     ;;; 27/12/2014
                                <1>     ;;stc
                                <1>     ;;retn
                                <1>     ;
                                <1> ;;hdc_reset0:
                                <1>     ; 18/01/2015
00004F2E 8A0D[48770100]         <1>     mov    cl, [HF_NUM]
00004F34 20C9                   <1>     and    cl, cl
00004F36 740D                   <1>     jz     short POD_DONE
                                <1>     ;
00004F38 B27F                   <1>     mov    dl, 7Fh
                                <1> hdc_reset1:
00004F3A FEC2                   <1>     inc    dl
                                <1>     ;; 31/12/2015
                                <1>     ;;push dx
                                <1>     ;;push cx
                                <1>     ;;push ds
                                <1>     ;;sub  ax, ax
                                <1>     ;;mov  ds, ax
                                <1>     ;;mov  ax, [TIMER_LOW]              ; GET START TIMER COUNTS
                                <1>     ;;pop  ds
                                <1>     ;;mov  bx, ax
                                <1>     ;;add  ax, 6*182             ; 60 SECONDS* 18.2
                                <1>     ;;mov  cx, ax
                                <1>     ;;mov  word [wait_count], 0  ; 22/12/2014 (reset wait counter)
                                <1>     ;;
                                <1>     ;; 31/12/2014 - cancel HD_RESET_1
                                <1>     ;;call HD_RESET_1            ; SET UP DRIVE 0, (1,2,3)
                                <1>     ;;pop  cx
                                <1>     ;;pop  dx
                                <1>     ;;
                                <1>     ; 18/01/2015
00004F3C B40D                   <1>     mov    ah, 0Dh ; ALTERNATE RESET
                                <1>     ;int   13h
```

```
00004F3E E8CC000000          <1>     call    int13h
00004F43 E2F5                <1>     loop    hdc_reset1
                             <1>     ;clc    ; 29/05/2016
                             <1> POD_DONE:
00004F45 C3                  <1>     retn
                             <1>
                             <1> ;;-----     POD_ERROR
                             <1>
                             <1> ;;CTL_ERRX:
                             <1> ;   ;mov    SI,OFFSET F1782         ; CONTROLLER ERROR
                             <1> ;   ;call   SET_FAIL                ; DO NOT IPL FROM DISK
                             <1> ;   ;call   E_MSG                   ; DISPLAY ERROR AND SET (BP) ERROR FLAG
                             <1> ;   ;jmp    short POD_DONE
                             <1>
                             <1> ;;HD_RESET_1:
                             <1> ;;  ;push   BX                      ; SAVE TIMER LIMITS
                             <1> ;;  ;push   CX
                             <1> ;;RES_1: mov        AH,09H                  ; SET DRIVE PARAMETERS
                             <1> ;;  INT     13H
                             <1> ;;  jc      short RES_2
                             <1> ;;  mov     AH,11h                  ; RECALIBRATE DRIVE
                             <1> ;;  INT     13H
                             <1> ;;  jnc     short RES_CK            ; DRIVE OK
                             <1> ;;RES_2: ;call      POD_TCHK                ; CHECK TIME OUT
                             <1> ;;  cmp     word [wait_count], 6*182 ; waiting time (in timer ticks)
                             <1> ;;                                  ; (30 seconds)
                             <1> ;;  ;cmc
                             <1> ;;  ;jnc    short RES_1
                             <1> ;;  jb      short RES_1
                             <1> ;;;RES_FL: ;mov    SI,OFFSET F1781         ; INDICATE DISK 1 FAILURE;
                             <1> ;;  ;test   DL,1
                             <1> ;;  ;jnz    short RES_E1
                             <1> ;;  ;mov    SI,OFFSET F1780         ; INDICATE DISK 0 FAILURE
                             <1> ;;  ;call   SET_FAIL                ; DO NOT TRY TO IPL DISK 0
                             <1> ;;  ;jmp    SHORT RES_E1
                             <1> ;;RES_ER: ; 22/12/2014
                             <1> ;;RES_OK:
                             <1> ;;  ;pop    CX                      ; RESTORE TIMER LIMITS
                             <1> ;;  ;pop    BX
                             <1> ;;  retn
                             <1> ;;
                             <1> ;;RES_RS: mov       AH,00H                  ; RESET THE DRIVE
                             <1> ;;  INT     13H
                             <1> ;;RES_CK: mov       AH,08H                  ; GET MAX CYLINDER,HEAD,SECTOR
                             <1> ;;  mov     BL,DL                   ; SAVE DRIVE CODE
                             <1> ;;  INT     13H
                             <1> ;;  jc      short RES_ER
                             <1> ;;  mov     [NEC_STATUS],CX         ; SAVE MAX CYLINDER, SECTOR
                             <1> ;;  mov     DL,BL                   ; RESTORE DRIVE CODE
                             <1> ;;RES_3: mov        AX,0401H                ; VERIFY THE LAST SECTOR
                             <1> ;;  INT     13H
                             <1> ;;  jnc     short RES_OK            ; VERIFY OK
                             <1> ;;  cmp     AH,BAD_SECTOR           ; OK ALSO IF JUST ID READ
                             <1> ;;  JE      short RES_OK
                             <1> ;;  cmp     AH,DATA_CORRECTED
                             <1> ;;  JE      short RES_OK
                             <1> ;;  cmp     AH,BAD_ECC
                             <1> ;;  JE      short RES_OK
                             <1> ;;  ;call   POD_TCHK                ; CHECK FOR TIME OUT
                             <1> ;;  cmp     word [wait_count], 6*182 ; waiting time (in timer ticks)
                             <1> ;;                                  ; (60 seconds)
                             <1> ;;  cmc
                             <1> ;;  jc      short RES_ER            ; FAILED
                             <1> ;;  mov     CX,[NEC_STATUS]         ; GET SECTOR ADDRESS, AND CYLINDER
                             <1> ;;  mov     AL,CL                   ; SEPARATE OUT SECTOR NUMBER
                             <1> ;;  and     AL,3FH
                             <1> ;;  dec     AL                      ; TRY PREVIOUS ONE
                             <1> ;;  jz      short RES_RS            ; WE'VE TRIED ALL SECTORS ON TRACK
                             <1> ;;  and     CL,0C0H                 ; KEEP CYLINDER BITS
                             <1> ;;  OR      CL,AL                   ; MERGE SECTOR WITH CYLINDER BITS
                             <1> ;;  mov     [NEC_STATUS],CX         ; SAVE CYLINDER, NEW SECTOR NUMBER
                             <1> ;;  jmp     short RES_3             ; TRY AGAIN
                             <1> ;;;RES_ER: mov      SI,OFFSET F1791         ; INDICATE DISK 1 ERROR
                             <1> ;;  ;test   DL,1
                             <1> ;;  ;jnz    short RES_E1
                             <1> ;;  ;mov    SI,OFFSET F1790         ; INDICATE DISK 0 ERROR
                             <1> ;;;RES_E1:
                             <1> ;;  ;call   E_MSG                   ; DISPLAY ERROR AND SET (BP) ERROR FLAG
                             <1> ;;;RES_OK:
                             <1> ;;  ;pop    CX                      ; RESTORE TIMER LIMITS
                             <1> ;;  ;pop    BX
                             <1> ;;  ;retn
                             <1> ;
                             <1> ;;SET_FAIL:
                             <1> ;   ;mov    AX,X*(CMOS_DIAG+NMI)    ; GET CMOS ERROR BYTE
                             <1> ;   ;call   CMOS_READ
                             <1> ;   ;OR     AL,HF_FAIL              ; SET DO NOT IPL FROM DISK FLAG
                             <1> ;   ;xchg   AH,AL                   ; SAVE IT
                             <1> ;   ;call   CMOS_WRITE              ; PUT IT OUT
                             <1> ;   ;retn
                             <1> ;
                             <1> ;;POD_TCHK:                         ; CHECK FOR 30 SECOND TIME OUT
                             <1> ;   ;pop    AX                      ; SAVE RETURN
                             <1> ;   ;pop    CX                      ; GET TIME OUT LIMITS
                             <1> ;   ;pop    BX
                             <1> ;   ;push   BX                      ; AND SAVE THEM AGAIN
                             <1> ;   ;push   CX
                             <1> ;   ;push   AX
                             <1> ;   ;push   ds
                             <1> ;   ;xor    ax, ax
                             <1> ;   ;mov    ds, ax                  ; RESTORE RETURN
                             <1> ;   ;mov    AX, [TIMER_LOW]                 ; AX = CURRENT TIME
                             <1> ;   ;                               ; BX = START TIME
                             <1> ;   ;                               ; CX = END TIME
                             <1> ;   ;pop    ds
                             <1> ;   ;cmp    BX,CX
                             <1> ;   ;JB     short TCHK1             ; START < END
                             <1> ;   ;cmp    BX,AX
                             <1> ;   ;JB     short TCHKG             ; END < START < CURRENT
                             <1> ;   ;jmp    SHORT TCHK2             ; END, CURRENT < START
                             <1> ;;TCHK1: cmp        AX,BX
                             <1> ;;  JB      short TCHKNG            ; CURRENT < START < END
                             <1> ;;TCHK2: cmp        AX,CX
                             <1> ;;  JB      short TCHKG             ; START < CURRENT < END
                             <1> ;;                                  ; OR CURRENT < END < START
                             <1> ;;TCHKNG: STC                               ; CARRY SET INDICATES TIME OUT
                             <1> ;;  retn
                             <1> ;;TCHKG: CLC                                ; INDICATE STILL TIME
                             <1> ;;  retn
                             <1> ;;
                             <1> ;;int_13h:
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   FIXED DISK BIOS ENTRY POINT   :
```

```
                                <1> ;----------------------------------------
                                <1>
                                <1> ; 17/07/2022
                                <1> ; 16/07/2022
                                <1> ; 13/07/2022 - TRDOS 386 v2.0.5
                                <1> ; 15/01/2017
                                <1> ; 14/01/2017
                                <1> ; 07/01/2017
                                <1> ; 02/01/2017
                                <1> ; 01/06/2016
                                <1> ; 16/05/2016, 27/05/2016, 28/05/2016, 29/05/2016
                                <1> ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> int33h: ; DISK I/O
                                <1>     ; 29/05/2016
00004F46 80642408FE             <1>     and    byte [esp+8], 11111110b  ; clear carry bit of eflags register
                                <1>
                                <1>     ; 13/07/2022
00004F4B 06                     <1>     push   es
                                <1>     ; 16/05/2016
00004F4C 1E                     <1>     push   ds
00004F4D 53                     <1>     push   ebx ; user's buffer address (virtual)
                                <1>     ; 13/07/2022
00004F4E 51                     <1>     push   ecx
00004F4F 52                     <1>     push   edx
00004F50 56                     <1>     push   esi
00004F51 57                     <1>     push   edi
                                <1>
                                <1>     ;mov   bx, KDATA ; System (Kernel's) data segment
                                <1>     ;mov   ds, bx
                                <1>     ; 13/07/2022
00004F52 66BF1000               <1>     mov    di, KDATA
00004F56 8EDF                   <1>     mov    ds, di
00004F58 8EC7                   <1>     mov    es, di
                                <1>
                                <1>     ;;15/01/2017
                                <1>     ; 14/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;;mov  byte [intflg], 33h  ; disk io interrupt
                                <1>     ;pop   ebx
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ;pop   dword [user_buffer] ; 01/06/2016
00004F5A 891D[40830100]         <1>     mov    [user_buffer], ebx
                                <1>
00004F60 C605[7A7C0100]00       <1>     mov    byte [scount], 0 ; sector count for transfer
00004F67 80FC03                 <1>     cmp    ah, 03h ; chs write
00004F6A 773C                   <1>     ja     short int33h_2
00004F6C 7407                   <1>     je     short int33h_0
00004F6E 80FC02                 <1>     cmp    ah, 02h ; chs read
00004F71 7267                   <1>     jb     short int33h_5
00004F73 EB5A                   <1>     jmp    short int33h_4
                                <1> int33h_0:
                                <1>     ;; 17/07/2022 - 64K r/w buffer limit check ?
                                <1>     ;cmp   al, 80h ; 128
                                <1>     ;ja    short int33h_8 ; error
                                <1>     ;; 17/07/2022 - zero r/w count check ?
                                <1>     ;or    al, al
                                <1>     ;jz    short int33h_8 ; error
                                <1>
                                <1>     ; 17/07/2022 (buffer limit and zero count check)
00004F75 FEC8                   <1>     dec    al
00004F77 781A                   <1>     js     short int33h_8 ; error
00004F79 FEC0                   <1>     inc    al
                                <1>
                                <1>     ; transfer user's buffer content to sector buffer
00004F7B 51                     <1>     push   ecx
00004F7C 0FB6C8                 <1>     movzx  ecx, al
                                <1> int33h_1:
                                <1>     ; 13/07/2022
                                <1>     ;push  esi
                                <1>     ;mov   esi, [user_buffer]
00004F7F 89DE                   <1>     mov    esi, ebx
                                <1>     ; esi = user's buffer address (virtual, ebx)
                                <1>     ;push  edi
                                <1>     ;push  es
00004F81 50                     <1>     push   eax
                                <1>     ;mov   ax, KDATA
                                <1>     ;mov   es, ax
00004F82 BF00000700             <1>     mov    edi, Cluster_Buffer
00004F87 C1E109                 <1>     shl    ecx, 9 ; * 512
00004F8A E8D6BB0000             <1>     call   transfer_from_user_buffer
                                <1>            ; (ecx and eax will be modified)
00004F8F 58                     <1>     pop    eax
                                <1>     ; 13/07/2022
                                <1>     ;pop   es
                                <1>     ;pop   edi
                                <1>     ;pop   esi
00004F90 59                     <1>     pop    ecx
00004F91 7347                   <1>     jnc    short int33h_5
                                <1>
                                <1>     ;mov   ebx, [user_buffer] ; 01/06/2016
                                <1>     ;pop   ds
                                <1>
                                <1> int33h_8:
                                <1>     ; 13/07/2022
00004F93 B8FF000000             <1>     mov    eax, 0FFh ; Unknown error !?
                                <1> int33h_9:
00004F98 5F                     <1>     pop    edi
00004F99 5E                     <1>     pop    esi
00004F9A 5A                     <1>     pop    edx
00004F9B 59                     <1>     pop    ecx
00004F9C 5B                     <1>     pop    ebx
00004F9D 1F                     <1>     pop    ds
00004F9E 07                     <1>     pop    es
                                <1>
                                <1>     ; 13/07/2022
00004F9F 7305                   <1>     jnc    short int33h_7
                                <1>
                                <1>     ;;15/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;cli
                                <1>     ;;mov  byte [ss:intflg], 0 ; 07/01/2017
                                <1>     ;
                                <1>     ; (*) 29/05/2016
                                <1>     ; (*) retf 4 ; skip eflags on stack
                                <1>
                                <1>     ; 29/05/2016 -set carry flag on stack-
                                <1>     ; [esp] = EIP
                                <1>     ; [esp+4] = CS
                                <1>     ; [esp+8] = E-FLAGS
00004FA1 804C240801             <1>     or     byte [esp+8], 1  ; set carry bit of eflags register
                                <1>     ; [esp+12] = ESP (user)
                                <1>     ; [esp+16] = SS (User)
                                <1>     ;
```

```
                                <1>     ; 13/07/2022
                                <1> int33h_7:
00004FA6 FA                     <1>     cli
                                <1>     ;;15/01/2017
                                <1>     ;;mov  byte [ss:intflg], 0 ; 07/01/2017
                                <1>     ; cf = 0  ; use eflags which is in stack
00004FA7 CF                     <1>     iretd
                                <1>
                                <1>     ; (*) 29/05/2016 - 'retf 4' intruction causes to stack fault
                                <1>     ; (OUTER-PRIVILEGE-LEVEL)
                                <1>     ; INTEL 80386 PROGRAMMER'S REFERENCE MANUAL 1986
                                <1>     ; // RETF instruction:
                                <1>     ;
                                <1>     ; IF OperandMode=32 THEN
                                <1>     ;    Load CS:EIP from stack;
                                <1>     ;    Set CS RPL to CPL;
                                <1>     ;    Increment ESP by 8 plus the immediate offset if it exists;
                                <1>     ;    Load SS:ESP from stack;
                                <1>     ; ELSE (* OperandMode=16 *)
                                <1>     ;    Load CS:IP from stack;
                                <1>     ;    Set CS RPL to CPL;
                                <1>     ;    Increment ESP by 4 plus the immediate offset if it exists;
                                <1>     ;    Load SS:ESP from stack;
                                <1>     ; FI;
                                <1>     ;
                                <1>     ; //
                                <1>
                                <1> int33h_2:
00004FA8 80FC05                 <1>     cmp    ah, 05h ; format track
00004FAB 7709                   <1>     ja     short int33h_3
00004FAD 722B                   <1>     jb     short int33h_5
00004FAF 51                     <1>     push   ecx
                                <1>     ;mov   ecx, 1
                                <1>     ; 17/07/2022
00004FB0 31C9                   <1>     xor    ecx, ecx
00004FB2 FEC1                   <1>     inc    cl
                                <1>     ; ecx = 1
00004FB4 EBC9                   <1>     jmp    short int33h_1
                                <1> int33h_3:
00004FB6 80FC1C                 <1>     cmp    ah, 1Ch ; LBA write
00004FB9 771F                   <1>     ja     short int33h_5
00004FBB 74B8                   <1>     je     short int33h_0
00004FBD 80FC1B                 <1>     cmp    ah, 1Bh ; LBA read
00004FC0 740D                   <1>     je     short int33h_4
00004FC2 80FC08                 <1>     cmp    ah, 08h ; get disk parameters
00004FC5 7513                   <1>     jne    short int33h_5
                                <1>     ; 01/06/2016
00004FC7 8B1D[40830100]         <1>     mov    ebx, [user_buffer] ; user's buffer address
00004FCD EB10                   <1>     jmp    short int33h_6
                                <1> int33h_4:
                                <1>     ;; 17/07/2022 - 64K r/w buffer limit check ?
                                <1>     ;cmp   al, 80h ; 128
                                <1>     ;ja    short int33h_8 ; error
                                <1>     ;; 17/07/2022 - zero r/w count check ?
                                <1>     ;or    al, al
                                <1>     ;jz    short int33h_8 ; error
                                <1>
                                <1>     ; 17/07/2022 (buffer limit and zero count check)
00004FCF FEC8                   <1>     dec    al
00004FD1 78C0                   <1>     js     short int33h_8 ; error
00004FD3 FEC0                   <1>     inc    al
                                <1>
00004FD5 A2[7A7C0100]           <1>     mov    byte [scount], al ; <= 128 sectors
                                <1> int33h_5:
00004FDA BB00000700             <1>     mov    ebx, Cluster_Buffer ; max. 65536 bytes
                                <1>                                ; buf. addr: 70000h
                                <1>     ;mov   byte [ClusterBuffer_Valid], 0
                                <1> int33h_6:
                                <1>     ; 13/07/2022
                                <1>     ;pop   ds
                                <1>     ;pushfd
                                <1>     ;push  cs
                                <1>
00004FDF E83B000000             <1>     call   DISK_IO
                                <1>
                                <1>     ;;mov  ebx, [cs:user_buffer] ; 01/06/2016
                                <1>     ;mov   ebx, [user_buffer] ; 13/07/2022
00004FE4 72B2                   <1>     jc     short int33h_9
                                <1>     ;
                                <1>     ;cmp   byte [cs:scount], 0
                                <1>     ;jna   short int33h_7
00004FE6 803D[7A7C0100]00       <1>     cmp    byte [scount], 0 ; 13/07/2022
00004FED 76A9                   <1>     jna    short int33h_9
                                <1>
                                <1>     ; 13/07/2022
                                <1>
                                <1>     ; transfer sector buffer content to user's buffer
                                <1>     ;push  es
                                <1>     ;push  ds
                                <1>     ;push  eax
                                <1>     ;mov   ax, KDATA
                                <1>     ;mov   ds, ax
                                <1>     ;mov   es, ax
                                <1>     ;push  ecx
                                <1>     ;push  esi
                                <1>     ;push  edi
                                <1>
                                <1>     ; 13/07/2022
00004FEF 50                     <1>     push   eax
00004FF0 0FB60D[7A7C0100]       <1>     movzx  ecx, byte [scount]
00004FF7 C1E109                 <1>     shl    ecx, 9 ; * 512 bytes
                                <1>     ;mov   edi, ebx ; user's buffer address
00004FFA 8B3D[40830100]         <1>     mov    edi, [user_buffer] ; 13/07/2022
00005000 BE00000700             <1>     mov    esi, Cluster_Buffer
00005005 E811BB0000             <1>     call   transfer_to_user_buffer
                                <1>            ; (ecx and eax will be modified)
0000500A 58                     <1>     pop    eax
                                <1>
                                <1>     ; 13/07/2022
0000500B 7286                   <1>     jc     short int33h_8 ; eax = 0FFh
0000500D EB89                   <1>     jmp    short int33h_9 ; cf = 0
                                <1>
                                <1>     ;pop   edi
                                <1>     ;pop   esi
                                <1>     ;pop   ecx
                                <1>     ;pop   eax
                                <1>     ;pop   ds
                                <1>     ;pop   es
                                <1>     ;jc    short int33h_8
                                <1> ;int33h_7:
                                <1>     ;cli
                                <1>     ;;;15/01/2017
                                <1>     ;;;mov byte [ss:intflg], 0 ; 07/01/2017
                                <1>     ;; cf = 0  ; use eflags which is in stack
```

```
                                <1>     ;iretd
                                <1> ;int33h_8:
                                <1>     ;mov    eax, 0FFh ; Unknown error !?
                                <1>     ; 13/07/2022
                                <1>     ;xor    eax, eax
                                <1>     ;dec    al  ; eax = 0FFh
                                <1>     ;jmp    short int33h_9
                                <1>
                                <1> ;int33h_9:
                                <1>     ;; cf = 1
                                <1>     ;
                                <1>     ;; (*) 29/05/2016
                                <1>     ;; (*) retf 4 ; skip eflags on stack
                                <1>     ;; Note: This 'retf 4' was wrong, -it was causing
                                <1>     ;;       to stack errors in ring 3-
                                <1>     ;;      POP sequence of 'retf 4' is as
                                <1>     ;;        "eip, cs, eflags, esp, ss, +4 bytes"
                                <1>         ;;       it is not as "eip, cs, +4 bytes, esp, ss" !
                                <1>     ;
                                <1>     ;; 29/05/2016 -set carry flag on stack-
                                <1>     ;or     byte [esp+8], 1  ; set carry bit of eflags register
                                <1>     ;;iretd
                                <1>     ;jmp    short int33h_7 ; 07/01/2017
                                <1>
                                <1> ;; 11/04/2021
                                <1> ;int13h: ; 21/02/2015
                                <1> ;   clc ; 11/04/2021
                                <1> ;   pushfd
                                <1> ;   push    cs
                                <1> ;   call    DISK_IO
                                <1> ;   retn
                                <1>
                                <1> int13h:
                                <1>     ; 13/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; Note: DISK_IO sets registers on stack
                                <1>     ;       as return parameters. So,
                                <1>     ;       stack order (at the entry of 'DISK_IO')
                                <1>     ;       must be same with 'int33h:' as above.
                                <1>
                                <1>     ;push   es ; not necessary
                                <1>     ;push   ds ; not necessary
                                <1>     ;
                                <1>     ; following pushes are necessary
                                <1>     ; for setting registers -return values- in DISK_IO
0000500F 53                     <1>     push    ebx
00005010 51                     <1>     push    ecx
00005011 52                     <1>     push    edx
00005012 56                     <1>     push    esi
00005013 57                     <1>     push    edi
                                <1>     ;push   ebp
                                <1>     ;mov    ebp, esp
                                <1>     ; edi = ebp+4
                                <1>     ; esi = ebp+8
                                <1>     ; edx = ebp+12
                                <1>     ; ecx = ebp+16
                                <1>     ; ebx = ebp+20
                                <1>     ;
00005014 E806000000             <1>     call    DISK_IO
                                <1>     ;
00005019 5F                     <1>     pop     edi
0000501A 5E                     <1>     pop     esi
0000501B 5A                     <1>     pop     edx
0000501C 59                     <1>     pop     ecx
0000501D 5B                     <1>     pop     ebx
                                <1>     ;
                                <1>     ;pop    ds
                                <1>     ;pop    es
                                <1>     ;
0000501E C3                     <1>     retn
                                <1>
                                <1> ; 10/08/2022
                                <1> ; 07/08/2022
                                <1> ; 17/07/2022
                                <1> ; 13/07/2022 - TRDOS 386 v2.0.5
                                <1> ; 18/04/2021 - TRDOS 386 v2.0.4
                                <1> ; 11/04/2021 - TRDOS 386 v2.0.3
                                <1> ; 30/08/2020
                                <1> ; 09/12/2017
                                <1> ; 29/05/2016
                                <1> ; 27/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>
                                <1> DISK_IO:
                                <1>     ; 10/08/2022
                                <1>     ; 17/07/2022
                                <1>     ; 13/07/2022
                                <1>     ; Registers are also on stack
                                <1>     ; (with same contents)
                                <1>     ; in following order:
                                <1>     ;
                                <1>     ;    ebx = esp+20
                                <1>     ;    ecx = esp+16
                                <1>     ;    edx = esp+12
                                <1>     ;    esi = esp+8
                                <1>     ;    edi = esp+4
                                <1>
                                <1>     ; cs = KCODE (== KDATA base address)
                                <1>     ; ss = KDATA
                                <1>     ; ds = KDATA
                                <1>     ; es = KDATA
                                <1>
                                <1>     ; 17/07/2022
0000501F FB                     <1>     sti                             ; ENABLE INTERRUPTS
                                <1>
00005020 80FA80                 <1>     cmp     dl, 80h                 ; TEST FOR FIXED DISK DRIVE
                                <1>     ;jae    short A1                ; YES, HANDLE HERE
                                <1>     ;;;int  40h                     ; DISKETTE HANDLER
                                <1>     ;;call  int40h
                                <1>     ;jb     DISKETTE_IO_1
                                <1> ;RET_2:
                                <1>     ;;retf  2                       ; BACK TO CALLER
                                <1>     ;retf   4
                                <1>     ; 11/04/2021
00005023 7305                   <1>     jnb     short A1
00005025 E934F2FFFF             <1>     jmp     DISKETTE_IO_1
                                <1> A1:
                                <1>     ;sti     ; 17/07/2022           ; ENABLE INTERRUPTS
                                <1>
                                <1>     ;; 04/01/2015
                                <1>     ;;or    ah, ah
                                <1>     ;;jnz   short A2
                                <1>     ;;int   40h                     ; RESET NEC WHEN AH=0
                                <1>     ;;sub   ah, ah
                                <1>
0000502A 80FA83                 <1>     cmp     dl, (80h + S_MAX_FILE - 1)
```

```
                                <1>     ;ja     short RET_2
0000502D 760A                   <1>     jna     short _A0
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; (here, DS is KDATA segment already)
                                <1>     ;
                                <1>     ; 29/05/2016
                                <1>     ;push   ds
                                <1>     ; 11/04/2021
                                <1>     ;push   eax
                                <1>     ;mov    ax, KDATA
                                <1>     ;mov    ds, ax
                                <1>     ; 11/04/2021
                                <1>     ;pop    eax
                                <1>
0000502F B4AA                   <1>     mov     ah, 0AAh        ; Hard disk drive not ready !
                                <1>                             ; (Programmer's guide to AMIBIOS, 1992)
00005031 8825[47770100]         <1>     mov     byte [DISK_STATUS1], ah
                                <1>     ; 13/07/2022
                                <1>     ;pop    ds
                                <1>     ;jmp    short RET_2
00005037 F9                     <1>     stc
00005038 C3                     <1>     retn
                                <1> _A0:
                                <1>     ; 18/01/2015
00005039 08E4                   <1>     or      ah, ah
0000503B 742C                   <1>     jz      short A4
0000503D 80FC0D                 <1>     cmp     ah, 0Dh ; Alternate reset
00005040 7504                   <1>     jne     short A2
00005042 28E4                   <1>     sub     ah, ah  ; Reset
00005044 EB23                   <1>     jmp     short A4
                                <1> A2:
                                <1>     ; 13/07/2022
00005046 80FC08                 <1>     cmp     ah, 08h                 ; GET PARAMETERS IS A SPECIAL CASE
00005049 7505                   <1>     jne     short A3
0000504B E911040000             <1>         jmp GET_PARM_N
                                <1> A3:
                                <1>     ; 13/07/2022
00005050 80FC15                 <1>     cmp     ah, 15h                 ; READ DASD TYPE IS ALSO
00005053 7514                   <1>     jne     short A4
00005055 E9AC030000             <1>         jmp READ_DASD_TYPE ; Return Drive Type
                                <1>                             ; (Programmer's guide to AMIBIOS, 1992)
                                <1>     ; 13/07/2022
                                <1> int33h_bad_cmd:
                                <1>     ; 16/05/2016
                                <1>     ; 30/01/2015
                                <1>     ; 29/05/2016
                                <1>     ;push   ds
                                <1>     ;push   eax
                                <1>     ;mov    ax, KDATA
                                <1>     ;mov    ds, ax
                                <1>     ;pop    eax
                                <1>
0000505A B401                   <1>     mov     ah, BAD_CMD
0000505C 8825[47770100]         <1>     mov     [DISK_STATUS1], ah ; BAD_CMD  ; COMMAND ERROR
                                <1>         ;jmp       short RET_2
                                <1>     ; 13/07/2022
                                <1> ;RET_2:
                                <1>     ; (*) 29/05/2016
                                <1>     ; (*) retf 4
                                <1>     ;or     byte [esp+8], 1 ; set carry bit of eflags register
                                <1>     ;iretd
                                <1>
                                <1>     ; 13/07/2022
00005062 F9                     <1>     stc
                                <1>     ; cf = 1, ah = BAD_CMD
00005063 C3                     <1>     retn
                                <1> _A4:
                                <1>     ; 13/07/2022
                                <1>     ; 02/02/2015
00005064 80FC1D                 <1>     cmp     ah, 1Dh                 ; (Temporary for Retro UNIX 386 v1)
                                <1>     ; 12/01/2015
                                <1>     ;cmc
                                <1>     ;jnc    short A4
                                <1>     ; 13/07/2022
00005067 73F1                   <1>     jnb     short int33h_bad_cmd
                                <1> A4:                                 ; SAVE REGISTERS DURING OPERATION
00005069 C8080000               <1>     enter   8, 0                    ; SAVE (BP) AND MAKE ROOM FOR @CMD_BLOCK
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; ENTER 8, 0
                                <1>     ;;push  ebp
                                <1>     ;;mov   ebp, esp
                                <1>     ;;sub   esp, 8
                                <1>     ;
                                <1>     ;push   ebx                     ;  IN THE STACK, THE COMMAND BLOCK IS:
                                <1>     ;push   ecx                     ;   @CMD_BLOCK == BYTE PTR [BP]-8
                                <1>     ;push   edx
                                <1>     ;push   esi
                                <1>     ;push   edi
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; edi = ebp+8
                                <1>     ; esi = ebp+12
                                <1>     ; edx = ebp+16
                                <1>     ; ecx = ebp+20
                                <1>     ; ebx = ebp+24
                                <1>
                                <1>     ;;04/01/2015
                                <1>     ;;or    ah, ah                  ; CHECK FOR RESET
                                <1>     ;;jnz   short A5
                                <1>     ;;mov   dl, 80h                 ; FORCE DRIVE 80 FOR RESET
                                <1> ;;A5:
                                <1>     ; 13/07/2022
0000506D E880000000             <1>     call    DISK_IO_CONT            ; PERFORM THE OPERATION
                                <1>     ;;call DDS                      ; ESTABLISH SEGMENT
00005072 8A25[47770100]         <1>     mov     ah, [DISK_STATUS1]      ; GET STATUS FROM OPERATION
                                <1>     ;(*) cmp ah, 1                  ; SET THE CARRY FLAG TO INDICATE
                                <1>                                     ; SUCCESS OR FAILURE
                                <1>     ;pop    edi                     ; RESTORE REGISTERS
                                <1>     ;pop    esi
                                <1>     ;pop    edx
                                <1>     ;pop    ecx
                                <1>     ;pop    ebx
                                <1>
00005078 C9                     <1>     leave                           ; ADJUST (SP) AND RESTORE (BP)
                                <1>
                                <1>     ;retf  2                        ; THROW AWAY SAVED FLAGS
                                <1>     ; (*) 29/05/2016
                                <1>     ; (*) retf 4
                                <1>
                                <1>     ; 13/07/2022
00005079 80FC01                 <1>     cmp     ah, 1
                                <1>     ;jc     short _A5
```

```
                            <1>     ;or    byte [esp+8], 1 ; set carry bit of eflags register
                            <1> ;_A5:
                            <1>     ;iretd
                            <1>     ; 10/08/2022
0000507C F5                 <1>     cmc
                            <1>     ; 13/07/2022
0000507D C3                 <1>     retn
                            <1>
                            <1> ; 21/02/2015
                            <1> ;       dw --> dd
                            <1>     ; 13/07/2022
                            <1> D1:                                 ; FUNCTION TRANSFER TABLE
0000507E [95520000]         <1>     dd     DISK_RESET               ; 00h
00005082 [F5520000]         <1>     dd     RETURN_STATUS            ; 01h
00005086 [08530000]         <1>     dd     DISK_READ                ; 02h
0000508A [64530000]         <1>     dd     DISK_WRITE               ; 03h
0000508E [EE530000]         <1>     dd     DISK_VERF                ; 04h
00005092 [D8530000]         <1>     dd     FMT_TRK                  ; 05h
00005096 [2B520000]         <1>     dd     BAD_COMMAND              ; 06h   FORMAT BAD SECTORS
0000509A [2B520000]         <1>     dd     BAD_COMMAND              ; 07h   FORMAT DRIVE
0000509E [2B520000]         <1>     dd     BAD_COMMAND              ; 08h   RETURN PARAMETERS
000050A2 [E1540000]         <1>     dd     INIT_DRV                 ; 09h
000050A6 [02530000]         <1>     dd     RD_LONG                  ; 0Ah
000050AA [5E530000]         <1>     dd     WR_LONG                  ; 0Bh
000050AE [52550000]         <1>     dd     DISK_SEEK                ; 0Ch
000050B2 [95520000]         <1>     dd     DISK_RESET               ; 0Dh
000050B6 [2B520000]         <1>     dd     BAD_COMMAND              ; 0Eh   READ BUFFER
000050BA [2B520000]         <1>     dd     BAD_COMMAND              ; 0Fh   WRITE BUFFER
000050BE [7A550000]         <1>     dd     TST_RDY                  ; 10h
000050C2 [AE550000]         <1>     dd     HDISK_RECAL              ; 11h
000050C6 [2B520000]         <1>     dd     BAD_COMMAND              ; 12h   MEMORY DIAGNOSTIC
000050CA [2B520000]         <1>     dd     BAD_COMMAND              ; 13h   DRIVE DIAGNOSTIC
000050CE [E4550000]         <1>     dd     CTLR_DIAGNOSTIC          ; 14h   CONTROLLER DIAGNOSTIC
                            <1>     ;; 02/02/2015 (Temporary - Retro UNIX 386 v1 - DISK I/O test)
000050D2 [2B520000]         <1>     dd     BAD_COMMAND              ; 15h
000050D6 [2B520000]         <1>     dd     BAD_COMMAND              ; 16h
000050DA [2B520000]         <1>     dd     BAD_COMMAND              ; 17h
000050DE [2B520000]         <1>     dd     BAD_COMMAND              ; 18h
000050E2 [2B520000]         <1>     dd     BAD_COMMAND              ; 19h
000050E6 [2B520000]         <1>     dd     BAD_COMMAND              ; 1Ah
000050EA [08530000]         <1>     dd     DISK_READ                ; 1Bh ; LBA read
000050EE [64530000]         <1>     dd     DISK_WRITE               ; 1Ch ; LBA write
                            <1> D1L     EQU    $ - D1
                            <1>
                            <1>     ; 02/12/2023
                            <1>     ; 01/12/2023 - TRDOS 386 v2.0.7
                            <1>     ; 07/08/2022
                            <1>     ; 17/07/2022 - TRDOS 386 v2.0.5
                            <1> DISK_IO_CONT:
                            <1>     ;;call DDS                      ; ESTABLISH SEGMENT
                            <1>     ; 11/04/2021
000050F2 80FC01             <1>     cmp    ah, 01h                  ; RETURN STATUS
000050F5 7505               <1>     jne    short SU0
000050F7 E9F9010000         <1>     jmp    RETURN_STATUS
                            <1> SU0:
000050FC C605[47770100]00   <1>     mov    byte [DISK_STATUS1], 0        ; RESET THE STATUS INDICATOR
                            <1>     ; 13/07/2022
00005103 89DE               <1>     mov    esi, ebx ; 21/02/2015 ; SAVE DATA ADDRESS
00005105 8A1D[48770100]     <1>     mov    bl, [HF_NUM]             ; GET NUMBER OF DRIVES
0000510B 80E27F             <1>     and    dl, 7Fh                  ; GET DRIVE AS 0 OR 1
                            <1>                                     ; (get drive number as 0 to 3)
                            <1>     ; 14/02/2015
0000510E 38D3               <1>     cmp    bl, dl
                            <1>     ;jbe   short BAD_COMMAND        ; INVALID DRIVE
                            <1>     ; 07/08/2022
00005110 7705               <1>     ja     short SU0X
00005112 E914010000         <1>     jmp    BAD_COMMAND
                            <1> SU0X:
                            <1>     ;;03/01/2015
00005117 29DB               <1>     sub    ebx, ebx
00005119 88D3               <1>     mov    bl, dl
0000511B 883D[5C770100]     <1>     mov    [LBAMode], bh  ; 0
                            <1>
                            <1>     ;test  byte [ebx+hd0_type], 1        ; LBA ready ?
                            <1>     ;jz    short su1                ; no
                            <1>     ;inc   byte [LBAMode]
                            <1> ;su1:
                            <1>     ; 11/04/2021 (32 bit push/pop)
                            <1>     ; 21/02/2015 (32 bit modification)
                            <1>     ; 04/01/2015
00005121 50                 <1>     push   eax ; ***
                            <1>     ;push  es  ; **
00005122 52                 <1>     push   edx ; *
00005123 50                 <1>     push   eax ; ****
00005124 E8AA060000         <1>     call   GET_VEC                  ; GET DISK PARAMETERS
                            <1>     ; 02/02/2015
                            <1>     ;mov   ax, [ES:BX+16] ; I/O port base address (1F0h, 170h)
00005129 668B4310           <1>     mov    ax, [ebx+16]
0000512D 66A3[7E660000]     <1>     mov    [HF_PORT], ax
                            <1>     ;mov   dx, [ES:BX+18] ; control port address (3F6h, 376h)
00005133 668B5312           <1>     mov    dx, [ebx+18]
00005137 668915[80660000]   <1>     mov    [HF_REG_PORT], dx
                            <1>     ;mov   al, [ES:BX+20] ; head register upper nibble (A0h,B0h,E0h,F0h)
0000513E 8A4314             <1>     mov    al, [ebx+20]
                            <1>     ; 23/02/2015
00005141 A840               <1>     test   al, 40h  ; LBA bit (bit 6)
00005143 7406               <1>     jz     short su1
00005145 FE05[5C770100]     <1>     inc    byte [LBAMode] ; 1
                            <1> su1:
0000514B C0E804             <1>     shr    al, 4
0000514E 2401               <1>     and    al, 1
00005150 A2[82660000]       <1>     mov    [hf_m_s], al
                            <1>     ;
                            <1>     ; 03/01/2015
                            <1>     ;mov   al, [ES:BX+8]         ; GET CONTROL BYTE MODIFIER
00005155 8A4308             <1>     mov    al, [ebx+8]
                            <1>     ;mov   dx, [HF_REG_PORT]     ; Device Control register
00005158 EE                 <1>     out    dx, al                   ; SET EXTRA HEAD OPTION
                            <1>                                     ; -here-
                            <1>                                     ; Control Byte: (= 08h)
                            <1>                                     ;  bit 0 - 0
                            <1>                                     ;  bit 1 - nIEN (1 = disable irq)
                            <1>                                     ;  bit 2 - SRST (software RESET)
                            <1>                                     ;  bit 3 - use extra heads (8 to 15)
                            <1>                                     ;          -always set to 1-
                            <1>                                     ;  (bits 3 to 7 are reserved
                            <1>                                     ;          for ATA devices)
00005159 8A25[49770100]     <1>     mov    ah, [CONTROL_BYTE]       ; SET EXTRA HEAD OPTION IN
0000515F 80E4C0             <1>     and    ah, 0C0h                 ; CONTROL BYTE
00005162 08C4               <1>     or     ah, al
00005164 8825[49770100]     <1>     mov    [CONTROL_BYTE], ah
                            <1>
                            <1>     ; 11/04/2021 (32 bit push/pop)
                            <1>     ; 04/01/2015
```

```
0000516A 58                <1>     pop    eax ; ****
0000516B 5A                <1>     pop    edx ; * ; 14/02/2015
0000516C 20E4              <1>     and    ah, ah  ; Reset function ?
0000516E 7506              <1>     jnz    short su2
                           <1>     ;pop   es ; **
00005170 58                <1>     pop    eax ; ***
00005171 E91F010000        <1>         jmp     DISK_RESET
                           <1> su2:
00005176 803D[5C770100]00  <1>     cmp    byte [LBAMode], 0
0000517D 765E              <1>     jna    short su3
                           <1>     ;
                           <1>     ; 02/02/2015 (LBA read/write function calls)
0000517F 80FC1B            <1>     cmp    ah, 1Bh
00005182 720A              <1>     jb     short lbarw1
00005184 80FC1C            <1>     cmp    ah, 1Ch
00005187 7759              <1>     ja     short invldfnc
                           <1>     ;;pop  edx ; * ; 14/02/2015
                           <1>     ;mov   ax, cx ; Lower word of LBA address (bits 0-15)
                           <1>
                           <1>     ; 01/12/2023 (48 bit LBA rw)
00005189 E9AD000000        <1>     jmp    lba_read_write
                           <1>
                           <1> lbarw1:
                           <1>     ; convert CHS to LBA
                           <1>     ;
                           <1>     ; LBA calculation - AWARD BIOS - 1999 - AHDSK.ASM
                           <1>     ; LBA = "# of Heads" * Sectors/Track * Cylinder + Head * Sectors/Track
                           <1>     ;       + Sector - 1
                           <1>     ; 11/04/2021 (32 bit push/pop)
0000518E 52                <1>     push   edx ; * ;; 14/02/2015
                           <1>     ;xor   dh, dh
0000518F 31D2              <1>     xor    edx, edx
                           <1>     ;mov   dl, [ES:BX+14] ; sectors per track (logical)
00005191 8A530E            <1>     mov    dl, [ebx+14]
                           <1>     ;xor   ah, ah
00005194 31C0              <1>     xor    eax, eax
                           <1>     ;mov   al, [ES:BX+2]  ; heads (logical)
00005196 8A4302            <1>     mov    al, [ebx+2]
00005199 FEC8              <1>     dec    al
0000519B 6640              <1>     inc    ax             ; 0 = 256
0000519D 66F7E2            <1>     mul    dx
                           <1>            ; AX = # of Heads * Sectors/Track
000051A0 6689CA            <1>     mov    dx, cx
                           <1>     ;and   cx, 3Fh ; sector  (1 to 63)
000051A3 83E13F            <1>     and    ecx, 3Fh
000051A6 86D6              <1>     xchg   dl, dh
000051A8 C0EE06            <1>     shr    dh, 6
                           <1>            ; DX = cylinder (0 to 1023)
                           <1>     ;mul   dx
                           <1>            ; DX:AX = # of Heads" * Sectors/Track * Cylinder
000051AB F7E2              <1>     mul    edx
000051AD FEC9              <1>     dec    cl ; sector - 1
                           <1>     ;add   ax, cx
                           <1>     ;adc   dx, 0
                           <1>            ; DX:AX = # of Heads" * Sectors/Track * Cylinder + Sector - 1
000051AF 01C8              <1>     add    eax, ecx
                           <1>     ; 11/04/2021 (32 bit push/pop)
000051B1 59                <1>     pop    ecx ; * ; ch = head, cl = drive number (zero based)
                           <1>     ;push  dx
                           <1>     ;push  ax
000051B2 50                <1>     push   eax
                           <1>     ; 13/07/2022
000051B3 29C0              <1>     sub    eax, eax
                           <1>     ;mov   al, [ES:BX+14] ; sectors per track (logical)
000051B5 8A430E            <1>     mov    al, [ebx+14]
000051B8 F6E5              <1>     mul    ch
                           <1>            ; AX = Head * Sectors/Track
                           <1>         ; 13/07/2022
                           <1>     ;movzx eax, ax ; 09/12/2017
                           <1>     ;pop   dx
000051BA 5A                <1>     pop    edx
                           <1>     ;add   ax, dx
                           <1>     ;pop   dx
                           <1>     ;adc   dx, 0 ; add carry bit
000051BB 01D0              <1>     add    eax, edx
                           <1> lbarw2:
000051BD 29D2              <1>     sub    edx, edx ; 21/02/2015
000051BF 88CA              <1>     mov    dl, cl ; 21/02/2015
000051C1 C645F800          <1>         mov     byte [CMD_BLOCK], 0 ; Features Register
                           <1>                            ; NOTE: Features register (1F1h, 171h)
                           <1>                            ; is not used for ATA device R/W functions.
                           <1>                            ; It is old/obsolete 'write precompensation'
                           <1>                            ; register and error register
                           <1>                            ; for old ATA/IDE devices.
                           <1>     ; 18/01/2014
                           <1>     ;mov   ch, [hf_m_s]   ; Drive 0 (master) or 1 (slave)
000051C5 8A0D[82660000]    <1>     mov    cl, [hf_m_s]
                           <1>     ;shl   ch, 4          ; bit 4 (drive bit)
                           <1>     ;or    ch, 0E0h       ; bit 5 = 1
                           <1>                            ; bit 6 = 1 = LBA mode
                           <1>                            ; bit 7 = 1
000051CB 80C90E            <1>     or     cl, 0Eh ; 1110b
                           <1>     ;and   dh, 0Fh        ; LBA byte 4 (bits 24 to 27)
000051CE 25FFFFFF0F        <1>     and    eax, 0FFFFFFFh
000051D3 C1E11C            <1>     shl    ecx, 28 ; 21/02/2015
                           <1>     ;or    dh, ch
000051D6 09C8              <1>     or     eax, ecx
                           <1>     ;;mov  [CMD_BLOCK+2], al ; LBA byte 1 (bits 0 to 7)
                           <1>                              ; (Sector Number Register)
                           <1>     ;;mov  [CMD_BLOCK+3], ah ; LBA byte 2 (bits 8 to 15)
                           <1>                              ; (Cylinder Low Register)
                           <1>     ;mov   [CMD_BLOCK+2], ax ; LBA byte 1, 2
                           <1>     ;mov   [CMD_BLOCK+4], dl ; LBA byte 3 (bits 16 to 23)
                           <1>                              ; (Cylinder High Register)
                           <1>     ;;mov  [CMD_BLOCK+5], dh ; LBA byte 4 (bits 24 to 27)
                           <1>                              ; (Drive/Head Register)
                           <1>
                           <1>     ;mov   [CMD_BLOCK+4], dx ; LBA byte 4, LBA & DEV select bits
000051D8 8945FA            <1>     mov    [CMD_BLOCK+2], eax ; 21/02/2015
                           <1>     ; 14/02/2015
                           <1>     ;mov   dl, cl ; Drive number (INIT_DRV)
000051DB EB36              <1>     jmp    short su4
                           <1> su3:
                           <1>     ; 07/08/2022
                           <1>     ; 13/07/2022
                           <1>     ; 02/02/2015
                           <1>     ; (1Bh & 1Ch functions are not valid for CHS mode)
000051DD 80FC14            <1>     cmp    ah, 14h
000051E0 7603              <1>     jna    short chsfnc
                           <1> invldfnc:
                           <1>         ; 14/02/2015
                           <1>     ;pop   es ; **
                           <1>     ; 11/04/2021
000051E2 58                <1>     pop    eax ; ***
```

```
000051E3 EB46                <1>         jmp     short BAD_COMMAND
                             <1> chsfnc:
                             <1>     ;mov    ax, [ES:BX+5]         ; GET WRITE PRE-COMPENSATION CYLINDER
000051E5 668B4305            <1>     mov     ax, [ebx+5]
                             <1>     ;shr    ax, 2
                             <1>     ; 07/08/2022
000051E9 C1E802              <1>     shr     eax, 2
000051EC 8845F8              <1>     mov     [CMD_BLOCK], al
                             <1>
                             <1>     ;;mov   al, [ES:BX+8]         ; GET CONTROL BYTE MODIFIER
                             <1>     ;;push  edx ; *
                             <1>     ;;mov   dx, [HF_REG_PORT]
                             <1>     ;;out   dx, al                ; SET EXTRA HEAD OPTION
                             <1>     ;;pop   edx ; *
                             <1>     ;;pop   es  ; **
                             <1>     ;;mov   ah, [CONTROL_BYTE]    ; SET EXTRA HEAD OPTION IN
                             <1>     ;;and   ah, 0C0h              ; CONTROL BYTE
                             <1>     ;;or    ah, al
                             <1>     ;;mov   [CONTROL_BYTE], ah
                             <1>
000051EF 88C8                <1>     mov     al, cl                ; GET SECTOR NUMBER
000051F1 243F                <1>     and     al, 3Fh
000051F3 8845FA              <1>     mov     [CMD_BLOCK+2], al
000051F6 886DFB              <1>     mov     [CMD_BLOCK+3], ch     ; GET CYLINDER NUMBER
000051F9 88C8                <1>     mov     al, cl
000051FB C0E806              <1>     shr     al, 6
000051FE 8845FC              <1>     mov     [CMD_BLOCK+4], al     ; CYLINDER HIGH ORDER 2 BITS
                             <1>
                             <1>     ;;05/01/2015
                             <1>     ;;mov   al, dl                ; DRIVE NUMBER
00005201 A0[82660000]        <1>     mov     al, [hf_m_s]
00005206 C0E004              <1>     shl     al, 4
00005209 80E60F              <1>     and     dh, 0Fh               ; HEAD NUMBER
0000520C 08F0                <1>     or      al, dh
                             <1>     ;or     al, 80h or 20h
0000520E 0CA0                <1>     or      al, 80h+20h           ; ECC AND 512 BYTE SECTORS
00005210 8845FD              <1>     mov     [CMD_BLOCK+5], al     ; ECC/SIZE/DRIVE/HEAD
                             <1> su4:
                             <1>     ;pop    es ; **
                             <1>         ;; 14/02/2015
                             <1>         ;;pop      ax
                             <1>         ;;mov      [CMD_BLOCK+1], al     ; SECTOR COUNT
                             <1>         ;;push     ax
                             <1>         ;;mov      al, ah                ; GET INTO LOW BYTE
                             <1>         ;;xor      ah, ah                ; ZERO HIGH BYTE
                             <1>         ;;sal      ax, 1                 ; *2 FOR TABLE LOOKUP
                             <1>         ; 11/04/2021
00005213 58                  <1>     pop     eax ; ***
00005214 8845F9              <1>         mov     [CMD_BLOCK+1], al
00005217 29DB                <1>         sub ebx, ebx
                             <1>     ;xor    bh, bh
00005219 88E3                <1>     mov      bl, ah
                             <1>         ;sal       bx, 1
                             <1>         ; 17/07/2022
0000521B C1E302              <1>     sal     ebx, 2 ; 32 bit offset (21/02/2015)
                             <1>     ;mov    si, ax                ; PUT INTO SI FOR BRANCH
                             <1>         ; 13/07/2022
                             <1>     ;cmp    bx, D1L               ; TEST WITHIN RANGE
                             <1>     ;jnb    short BAD_COMMAND_POP
0000521E 83FB74              <1>     cmp     ebx, D1L              ; TEST WITHIN RANGE
00005221 7308                <1>     jnb     short BAD_COMMAND
                             <1>         ;xchg      bx, si
00005223 87DE                <1>         xchg       ebx, esi
                             <1>     ;;;pop  ax                    ; RESTORE AX
                             <1>     ;;;pop  bx                    ; AND DATA ADDRESS
                             <1>
                             <1>     ;;push  cx
                             <1>     ;;push  ax                    ; ADJUST ES:BX
                             <1>     ;mov    cx, bx                ; GET 3 HIGH ORDER NIBBLES OF BX
                             <1>     ;shr    cx, 4
                             <1>     ;mov    ax, es
                             <1>     ;add    ax, cx
                             <1>     ;mov    es, ax
                             <1>     ;and    bx, 000Fh             ; ES:BX CHANGED TO ES:000X
                             <1>     ;;pop   ax
                             <1>     ;;pop   cx
                             <1>     ;;jmp   word [CS:SI+D1]
                             <1>     ;jmp    word [SI+D1]
                             <1>
00005225 FFA6[7E500000]      <1>     jmp     dword [esi+D1]
                             <1>
                             <1>     ; 07/08/2022
                             <1>     ; 13/07/2022
                             <1> BAD_COMMAND:
0000522B C605[47770100]01    <1>         mov byte [DISK_STATUS1], BAD_CMD ; COMMAND ERROR
00005232 B000                <1>     mov     al, 0
00005234 C3                  <1>     retn
                             <1>
                             <1> ; ---------------------------------------------------
                             <1> ; 48 bit LBA read/write
                             <1> ; ---------------------------------------------------
                             <1>
                             <1> su10:       ; 01/12/2023 ; 28 bit LBA r/w
00005235 89C8                <1>     mov     eax, ecx ; LBA address (21/02/2015)
                             <1>     ; 13/07/2022
00005237 88D1                <1>     mov     cl, dl ; 14/02/2015
00005239 EB82                <1>     jmp     short lbarw2
                             <1>
                             <1> lba_read_write:
                             <1>     ; 02/12/2023
                             <1>     ; 01/12/2023 - TRDOS 386 v2.0.7 (48 bit LBA rw)
0000523B 81F9FFFFFF0F        <1>     cmp     ecx, 0FFFFFFFh
00005241 731B                <1>     jnb     short su6
                             <1>     ;movzx  eax, byte [esp] ; ***
00005243 31C0                <1>     xor     eax, eax
00005245 8A0424              <1>     mov     al, [esp] ; *** ; sector count
00005248 01C8                <1>     add     eax, ecx
0000524A 730B                <1>     jnc     short su5
0000524C C605[47770100]17    <1>     mov     byte [DISK_STATUS1], ERR_INV_PARAMETER
                             <1>     ;jmp    short su7
                             <1>     ; 20/06/2024
00005253 58                  <1>     pop     eax ; ***
00005254 B000                <1>     mov     al, 0
00005256 C3                  <1>     retn
                             <1> su5:
00005257 3DFFFFFF0F          <1>     cmp     eax, 0FFFFFFFh ; 28 bit limit
0000525C 76D7                <1>     jna     short su10
                             <1> su6:
                             <1>
                             <1> ; 20/06/2024 (TRDOS 386 v2.0.8)
                             <1> %if 0
                             <1>     ; 48 bit LBA r/w
                             <1>     mov     al, [hf_m_s]   ; (+!+) ; 02/12/2023
                             <1>     shl     al, 4
```

```
                                <1>     ; 02/12/2023
                                <1>     ;add   al, 0E0h
                                <1>     add    al, 40h
                                <1>     ;;add  al, 0Eh
                                <1>     ;add   al, 04h
                                <1>     ;shl   al, 4
                                <1>     mov    dx, [HF_PORT]
                                <1>     add    dl, 6  ; hd base port + 6
                                <1>     out    dx, al
                                <1>     ; 02/12/2023
                                <1>     mov    [CMD_BLOCK+6], al
                                <1>     inc    edx    ; hd base port + 7
                                <1>     in     al, dx
                                <1>     NEWIODELAY
                                <1>     ;test  al, 128       ; READY ?
                                <1>     and    al, 128
                                <1>     jz     short su8
                                <1>     in     al, dx
                                <1>     ;test  al, 128
                                <1>     and    al, 128
                                <1>     jz     short su8
                                <1>     mov    byte [DISK_STATUS1], TIME_OUT
                                <1> su7:
                                <1>     pop    eax ; ***
                                <1>     mov    al, 0
                                <1>     retn
                                <1> su8:
                                <1>     ;mov   dx, [HF_PORT]
                                <1>     ;inc   edx
                                <1>     ;inc   edx    ; hd base port + 2
                                <1>     sub    dl, 5  ; hd base port + 2
                                <1>     ;xor   al, al
                                <1>     out    dx, al ; sector count hb (bits 8 to 15) = 0
                                <1>     inc    edx    ; hd base port + 3
                                <1>     mov    eax, ecx ; LBA disk sector address
                                <1>     rol    eax, 8
                                <1>     out    dx, al ; LBA byte 4 (bits 24 to 31)
                                <1>     inc    edx    ; hd base port + 4
                                <1>     xor    al, al
                                <1>     out    dx, al ; LBA byte 5 (bits 32 to 39) = 0
                                <1>     inc    edx    ; hd base port + 5
                                <1>     ;sub   al, al
                                <1>     out    dx, al ; LBA byte 6 (bits 40 to 47) = 0
                                <1>
                                <1>     mov    al, [esp] ; ***
                                <1>     ; 02/12/2023
                                <1>     mov    [CMD_BLOCK+1], al
                                <1>
                                <1>     sub    dl, 3  ; hd base port + 2
                                <1>     out    dx, al ; sector count lb (bits 0 to 7)
                                <1>     inc    edx    ; hd base port + 3
                                <1>     mov    eax, ecx ; LBA disk sector address
                                <1>     out    dx, al ; LBA byte 1 (bits 0 to 7)
                                <1>     inc    edx    ; hd base port + 4
                                <1>     shr    eax, 8
                                <1>     out    dx, al ; LBA byte 2 (bits 8 to 15)
                                <1>     inc    edx    ; hd base port + 5
                                <1>     shr    eax, 8
                                <1>     out    dx, al ; LBA byte 3 (bits 16 to 23)
                                <1>
                                <1>     inc     edx   ; hd base port + 6
                                <1>
                                <1>     ; 02/12/2023 (not necessary) (+!+)
                                <1>     ;mov   al, [hf_m_s]
                                <1>     ;shl   al, 4
                                <1>     ;add   al, 40h
                                <1>     ;out   dx, al
                                <1>
                                <1>     pop    eax    ; ***
                                <1>
                                <1>     inc    edx    ; dx = hd base port + 7
                                <1>                   ;      command/status port
                                <1>     ;xchg  esi, ebx
                                <1>     ;mov   edi, ebx
                                <1>     mov    edi, esi ; sector buffer
                                <1>     cmp    ah, 1Ch
                                <1>     jne    short su9
                                <1>     mov    al, 34h ; WRITE SECTOR(S) EXT
                                <1>     out    dx, al
                                <1>     jmp    CMD_WX
                                <1> su9:
                                <1>     mov    al, 24h ; READ SECTOR(S) EXT
                                <1>     out    dx, al
                                <1>     jmp    CMD_RX
                                <1> %else
                                <1>     ; 20/06/2024
0000525E 58                     <1>     pop    eax    ; ***
0000525F C645F800               <1>     mov    byte [CMD_BLOCK], 0
00005263 8845F9                 <1>     mov    [CMD_BLOCK+1], al ; sector count to r/w
00005266 894DFA                 <1>     mov    [CMD_BLOCK+2], ecx ; LBA disk sector address
                                <1>
                                <1>     ;;;
                                <1>     ; 23/06/2024
00005269 C605[5C770100]FF       <1>     mov    byte [LBAMode], 0FFh
                                <1>     ;
00005270 A0[82660000]           <1>     mov    al, [hf_m_s]
00005275 C0E004                 <1>     shl    al, 4
                                <1>     ;add   al, 40h
00005278 0C40                   <1>     or     al, 40h
                                <1>     ; al = 40h (for master), 50h (for slave)
0000527A A2[82660000]           <1>     mov    [hf_m_s], al
                                <1>     ;;;
                                <1>
0000527F 80FC1C                 <1>     cmp    ah, 1Ch
00005282 7509                   <1>     jne    short su7
                                <1>     ; AH = 1Ch ; TRDOS 386 v2 - INT 33h - LBA WRITE
                                <1>     ; esi = sector buffer
                                <1>     ; 48 bit LBA write
00005284 C645FE34               <1>     mov    byte [CMD_BLOCK+6], 34h ; WRITE SECTOR(S) EXT
00005288 E9DD000000             <1>     jmp    CMD_WX
                                <1> su7:
                                <1>     ; AH = 1Bh ; TRDOS 386 v2 - INT 33h - LBA READ
0000528D 89F7                   <1>     mov    edi, esi ; sector buffer
                                <1>     ; 48 bit LBA read
0000528F C645FE24               <1>     mov    byte [CMD_BLOCK+6], 24h ; READ SECTOR(S) EXT
00005293 EB79                   <1>     jmp    CMD_RX
                                <1>
                                <1> %endif
                                <1>
                                <1> ; ---------------------------------------------------
                                <1>
                                <1> ; 09/08/2022
                                <1> ; 07/08/2022
                                <1> ; 17/07/2022
```

```
                                <1> ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1> ; 10/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   RESET THE DISK SYSTEM  (AH=00H) :
                                <1> ;----------------------------------------
                                <1>
                                <1> ; 18-1-2015 : one controller reset (not other one)
                                <1>
                                <1> DISK_RESET:
00005295 FA                     <1>     cli
00005296 E4A1                   <1>     in      al, INTB01              ; GET THE MASK REGISTER
                                <1>     ;jmp    $+2
                                <1>     IODELAY
00005298 EB00                   <2>  jmp short $+2
0000529A EB00                   <2>  jmp short $+2
                                <1>     ;and    al, 0BFh                ; ENABLE FIXED DISK INTERRUPT
0000529C 243F                   <1>     and     al, 3Fh                 ; 22/12/2014 (IRQ 14 & IRQ 15)
0000529E E6A1                   <1>     out     INTB01, al
000052A0 FB                     <1>     sti                             ; START INTERRUPTS
                                <1>     ; 14/02/2015
                                <1>     ;mov    di, dx
                                <1>     ; 24/12/2021
000052A1 89D7                   <1>     mov     edi, edx
                                <1>     ; 04/01/2015
                                <1>     ;xor    di,di
                                <1> drst0:
000052A3 B004                   <1>     mov     al, 04h  ; bit 2 - SRST
                                <1>     ;mov    dx, HF_REG_PORT
000052A5 668B15[80660000]       <1>     mov     dx, [HF_REG_PORT]
000052AC EE                     <1>     out     dx, al                  ; RESET
                                <1> ;   mov     cx, 10                  ; DELAY COUNT
                                <1> ;DRD:       dec     cx
                                <1> ;   jnz     short DRD               ; WAIT 4.8 MICRO-SEC
                                <1>     ;mov    cx, 2                   ; wait for 30 micro seconds
                                <1>         ;mov       ecx, 2 ; 21/02/2015
                                <1>     ; 10/07/2022
000052AD 29C9                   <1>     sub     ecx, ecx
000052AF B102                   <1>     mov     cl, 2
000052B1 E86FD1FFFF             <1>     call    WAITF                   ; (Award Bios 1999 - WAIT_REFRESH,
                                <1>                                         ; 40 micro seconds)
000052B6 A0[49770100]           <1>     mov     al, [CONTROL_BYTE]
000052BB 240F                   <1>     and     al, 0Fh                 ; SET HEAD OPTION
000052BD EE                     <1>     out     dx, al                  ; TURN RESET OFF
000052BE E85B040000             <1>     call    NOT_BUSY
000052C3 7514                   <1>     jnz     short DRERR             ; TIME OUT ON RESET
000052C5 668B15[7E660000]       <1>     mov     dx, [HF_PORT]
000052CC FEC2                   <1>     inc     dl  ; HF_PORT+1
                                <1>     ; 02/01/2015 - Award BIOS 1999 - AHDSK.ASM
                                <1>         ;mov       cl, 10
                                <1>         ;mov       ecx, 10 ; 21/02/2015
                                <1>     ; 17/07/2022
                                <1>     ;xor    ecx, ecx
000052CE B10A                   <1>     mov     cl, 10
                                <1> drst1:
000052D0 EC                     <1>     in      al, dx                  ; GET RESET STATUS
000052D1 3C01                   <1>     cmp     al, 1
                                <1>     ; 04/01/2015
000052D3 740C                   <1>     jz      short drst2
                                <1>     ;jnz    short DRERR             ; BAD RESET STATUS
                                <1>             ; Drive/Head Register - bit 4
                                <1>     ;loop   drst1
                                <1>     ; 17/07/2022
000052D5 FEC9                   <1>     dec     cl
000052D7 75F7                   <1>     jnz     short drst1
                                <1> DRERR:
000052D9 C605[47770100]05       <1>     mov     byte [DISK_STATUS1], BAD_RESET ; CARD FAILED
000052E0 C3                     <1>     retn
                                <1> drst2:
                                <1>     ; 14/02/2015
                                <1>     ;mov    dx, di
                                <1>     ; 07/08/2022
000052E1 89FA                   <1>     mov     edx, edi
                                <1> ;drst3:
                                <1> ;   ; 05/01/2015
                                <1> ;   shl     di, 1
                                <1> ;   ; 04/01/2015
                                <1> ;   mov     ax, [di+hd_cports]
                                <1> ;   cmp     ax, [HF_REG_PORT]
                                <1> ;   je      short drst4
                                <1> ;   mov     [HF_REG_PORT], ax
                                <1> ;   ; 03/01/2015
                                <1> ;   mov     ax, [di+hd_ports]
                                <1> ;       mov     [HF_PORT], ax
                                <1> ;   ; 05/01/2014
                                <1> ;   shr     di, 1
                                <1> ;   ; 04/01/2015
                                <1> ;   jmp     short drst0    ; reset other controller
                                <1> ;drst4:
                                <1> ;   ; 05/01/2015
                                <1> ;   shr     di, 1
                                <1> ;   mov     al, [di+hd_dregs]
                                <1> ;   and     al, 10h ; bit 4 only
                                <1> ;   shr     al, 4 ; bit 4  -> bit 0
                                <1> ;   mov     [hf_m_s], al ; (0 = master, 1 = slave)
                                <1>     ;
                                <1> ; 09/08/2022
                                <1> ; (('INIT_DRV' prodedure sets [CMD_BLOCKS+5] value))
                                <1> ;
                                <1> ;   mov     al, [hf_m_s] ; 18/01/2015
                                <1> ;   test    al, 1
                                <1> ;   ;jnz    short drst6
                                <1> ;       jnz     short drst4
                                <1> ;   and     byte [CMD_BLOCK+5], 0EFh ; SET TO DRIVE 0
                                <1> ;drst5:
                                <1> drst3:
000052E3 E8F9010000             <1>     call    INIT_DRV                ; SET MAX HEADS
                                <1>     ;mov    dx, di
000052E8 E8C1020000             <1>     call    HDISK_RECAL             ; RECAL TO RESET SEEK SPEED
                                <1>     ; 04/01/2014
                                <1> ;   inc     di
                                <1> ;   mov     dx, di
                                <1> ;   cmp     dl, [HF_NUM]
                                <1> ;   jb      short drst3
                                <1> ;DRE:
000052ED C605[47770100]00       <1>     mov     byte [DISK_STATUS1], 0        ; IGNORE ANY SET UP ERRORS
000052F4 C3                     <1>     retn
                                <1> ;drst6:
                                <1> drst4:              ; Drive/Head Register - bit 4
                                <1> ;   or      byte [CMD_BLOCK+5], 010h ; SET TO DRIVE 1
                                <1> ;       ;jmp    short drst5
                                <1> ;       jmp     short drst3
                                <1>
                                <1> ;----------------------------------------
```

```
                                <1> ;   DISK STATUS ROUTINE  (AH = 01H) :
                                <1> ;----------------------------------------
                                <1>
                                <1> RETURN_STATUS:
000052F5 A0[47770100]           <1>     mov     al, [DISK_STATUS1]    ; OBTAIN PREVIOUS STATUS
000052FA C605[47770100]00       <1>     mov     byte [DISK_STATUS1], 0; RESET STATUS
00005301 C3                     <1>     retn
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   READ LONG           (AH = 0AH) :
                                <1> ;----------------------------------------
                                <1>
                                <1> RD_LONG:
                                <1>     ;mov    @CMD_BLOCK+6, READ_CMD OR ECC_MODE
00005302 C645FE22               <1>     mov      byte [CMD_BLOCK+6], READ_CMD + ECC_MODE
00005306 EB04                   <1>     jmp     short COMMANDI
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   DISK READ ROUTINE    (AH = 02H) :
                                <1> ;----------------------------------------
                                <1>
                                <1> DISK_READ:
00005308 C645FE20               <1>     mov     byte [CMD_BLOCK+6], READ_CMD
                                <1>         ;jmp       COMMANDI
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ; COMMANDI                        :
                                <1> ;   REPEATEDLY INPUTS DATA TILL   :
                                <1> ;   NSECTOR RETURNS ZERO          :
                                <1> ;----------------------------------------
                                <1> COMMANDI:
                                <1>     ; 16/07/2022
                                <1>     ;       (check 64K boundary is not needed)
                                <1>     ;call   CHECK_DMA             ; CHECK 64K BOUNDARY ERROR
                                <1>     ;jc     short CMD_ABORT
                                <1>
                                <1>     ;mov    di, bx
0000530C 89DF                   <1>     mov     edi, ebx ; 21/02/2015
                                <1> CMD_RX:
                                <1>     ; 20/06/2024 (48 bit LBA r/w modification)
0000530E E821030000             <1>     call    COMMAND               ; OUTPUT COMMAND
00005313 7548                   <1>     jnz     short CMD_ABORT
                                <1> ;CMD_RX: ; 01/12/2023 (48 bit LBA read)
                                <1> CMD_I1:
00005315 E8D4030000             <1>     call    _WAIT                 ; WAIT FOR DATA REQUEST INTERRUPT
0000531A 7541                   <1>     jnz     short TM_OUT          ; TIME OUT
                                <1> cmd_i1x:
                                <1>     ; 18/02/2016
                                <1>     ;;mov   cx, 256               ; SECTOR SIZE IN WORDS
                                <1>     ;mov    ecx, 256 ; 21/02/2015
                                <1>     ; 16/07/2022
0000531C 29C9                   <1>     sub     ecx, ecx
0000531E FEC5                   <1>     inc     ch  ; ecx = 256
                                <1>     ;mov    dh, HF_PORT
00005320 668B15[7E660000]       <1>     mov     dx, [HF_PORT]
00005327 FA                     <1>     cli
00005328 FC                     <1>     cld
00005329 66F36D                 <1>     rep     insw                  ; GET THE SECTOR
0000532C FB                     <1>     sti
                                <1>
0000532D F645FE02               <1>     test    byte [CMD_BLOCK+6], ECC_MODE ; CHECK FOR NORMAL INPUT
00005331 7418                   <1>     jz      short CMD_I3
00005333 E80D040000             <1>     call    WAIT_DRQ              ; WAIT FOR DATA REQUEST
00005338 7223                   <1>     jc      short TM_OUT
                                <1>     ;mov    dx, HF_PORT
0000533A 668B15[7E660000]       <1>     mov     dx, [HF_PORT]
                                <1>     ;;mov   cx, 4                 ; GET ECC BYTES
                                <1>     ;mov    ecx, 4 ; mov cx, 4
                                <1>     ; 16/07/2022
00005341 31C9                   <1>     xor     ecx, ecx
00005343 B104                   <1>     mov     cl, 4
                                <1> CMD_I2:
00005345 EC                     <1>     in      al, dx
00005346 8807                   <1>     mov     [edi], al ; 21/02/2015; GO SLOW FOR BOARD
00005348 47                     <1>     inc     edi
00005349 E2FA                   <1>     loop    CMD_I2
                                <1> CMD_I3:
                                <1>     ; wait for 400 ns
0000534B 80C207                 <1>     add     dl, 7
0000534E EC                     <1>     in      al, dx
0000534F EC                     <1>     in      al, dx
00005350 EC                     <1>     in      al, dx
                                <1>     ;
00005351 E8E4010000             <1>     call    CHECK_STATUS
00005356 7505                   <1>     jnz     short CMD_ABORT               ; ERROR RETURNED
00005358 FE4DF9                 <1>     dec     byte [CMD_BLOCK+1]    ; CHECK FOR MORE
                                <1>     ;jnz    short CMD_I1
0000535B 75BF                   <1>     jnz     short cmd_i1x ; 18/02/2016
                                <1> CMD_ABORT:
                                <1> TM_OUT:
0000535D C3                     <1>     retn
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   WRITE LONG          (AH = 0BH) :
                                <1> ;----------------------------------------
                                <1>
                                <1> WR_LONG:
                                <1>     ;mov    @CMD_BLOCK+6, WRITE_CMD OR ECC_MODE
0000535E C645FE32               <1>     mov     byte [CMD_BLOCK+6], WRITE_CMD + ECC_MODE
00005362 EB04                   <1>     jmp     short COMMANDO
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   DISK WRITE ROUTINE   (AH = 03H) :
                                <1> ;----------------------------------------
                                <1>
                                <1> DISK_WRITE:
00005364 C645FE30               <1>     mov     byte [CMD_BLOCK+6], WRITE_CMD
                                <1>     ;jmp    COMMANDO
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ; COMMANDO                        :
                                <1> ;   REPEATEDLY OUTPUTS DATA TILL  :
```

```
                            <1> ;   NSECTOR RETURNS ZERO         :
                            <1> ;----------------------------------------
                            <1> COMMANDO:
                            <1>     ; 16/07/2022
                            <1>     ;       (check 64K boundary is not needed)
                            <1>     ;call   CHECK_DMA             ; CHECK 64K BOUNDARY ERROR
                            <1>     ;jc     short CMD_ABORT
                            <1> CMD_OF:
00005368 89DE               <1>     mov     esi, ebx ; 21/02/2015
                            <1> CMD_WX:
                            <1>     ; 20/06/2024 (48 bit LBA r/w modification)
0000536A E8C5020000         <1>     call    COMMAND               ; OUTPUT COMMAND
0000536F 75EC               <1>     jnz     short CMD_ABORT
                            <1> ;CMD_WX:    ; 01/12/2023 (48 bit LBA write)
00005371 E8CF030000         <1>     call    WAIT_DRQ              ; WAIT FOR DATA REQUEST
00005376 72E5               <1>     jc      short TM_OUT          ; TOO LONG
                            <1> CMD_O1:
                            <1>     ; 16/07/2022
00005378 668B15[7E660000]   <1>     mov     dx, [HF_PORT]
                            <1>
                            <1>     ;mov    ecx, 256 ; 21/02/2015
0000537F 31C9               <1>     xor     ecx, ecx
00005381 FEC5               <1>     inc     ch
                            <1>     ; ecx = 256
00005383 FA                 <1>     cli
00005384 FC                 <1>     cld
                            <1>     ;rep    outsw
                            <1>     ; 01/12/2023 - TRDOS 386 v2.0.7
                            <1> CMD_O1_L:
00005385 666F               <1>     outsw
00005387 EB00               <1>     jmp     $+2
00005389 E2FA               <1>     loop    CMD_O1_L
                            <1>
0000538B FB                 <1>     sti
                            <1>
0000538C F645FE02           <1>     test    byte [CMD_BLOCK+6], ECC_MODE ; CHECK FOR NORMAL OUTPUT
00005390 7418               <1>     jz      short CMD_O3
00005392 E8AE030000         <1>     call    WAIT_DRQ              ; WAIT FOR DATA REQUEST
00005397 72C4               <1>     jc      short TM_OUT
                            <1>     ;mov    dx, HF_PORT
00005399 668B15[7E660000]   <1>     mov     dx, [HF_PORT]
                            <1>                                   ; OUTPUT THE ECC BYTES
                            <1>     ;mov    ecx, 4  ; mov cx, 4
                            <1>     ; 16/07/2022
000053A0 29C9               <1>     sub     ecx, ecx
000053A2 B104               <1>     mov     cl, 4
                            <1> CMD_O2:
000053A4 8A06               <1>     mov     al, [esi]
000053A6 EE                 <1>     out     dx, al
000053A7 46                 <1>     inc     esi
000053A8 E2FA               <1>     loop    CMD_O2
                            <1> CMD_O3:
000053AA E83F030000         <1>     call    _WAIT                 ; WAIT FOR SECTOR COMPLETE INTERRUPT
000053AF 75AC               <1>     jnz     short TM_OUT          ; ERROR RETURNED
000053B1 E884010000         <1>     call    CHECK_STATUS
000053B6 75A5               <1>     jnz     short CMD_ABORT
000053B8 F605[41770100]08   <1>     test    byte [HF_STATUS], ST_DRQ ; CHECK FOR MORE
000053BF 75B7               <1>     jnz     short CMD_O1
                            <1>     ;mov    dx, HF_PORT+2         ; CHECK RESIDUAL SECTOR COUNT
000053C1 668B15[7E660000]   <1>     mov     dx, [HF_PORT]
000053C8 80C202             <1>     add     dl, 2
                            <1>     ;inc    dl
                            <1>     ;inc    dl
000053CB EC                 <1>     in      al, dx                ;
000053CC A8FF               <1>     test    al, 0FFh              ;
000053CE 7407               <1>     jz      short CMD_O4          ; COUNT = 0 OK
000053D0 C605[47770100]BB   <1>     mov     byte [DISK_STATUS1], UNDEF_ERR
                            <1>                                   ; OPERATION ABORTED - PARTIAL TRANSFER
                            <1> CMD_O4:
000053D7 C3                 <1>     retn
                            <1>
                            <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   FORMATTING          (AH = 05H) :
                            <1> ;----------------------------------------
                            <1>
                            <1> FMT_TRK:                          ; FORMAT TRACK (AH = 005H)
000053D8 C645FE50           <1>     mov     byte [CMD_BLOCK+6], FMTTRK_CMD
                            <1>     ;push   es
                            <1>     ;push   bx
000053DC 53                 <1>     push    ebx
000053DD E8F1030000         <1>     call    GET_VEC               ; GET DISK PARAMETERS ADDRESS
                            <1>     ;mov    al, [ES:BX+14]        ; GET SECTORS/TRACK
000053E2 8A430E             <1>     mov     al, [ebx+14]
000053E5 8845F9             <1>     mov     [CMD_BLOCK+1], al     ; SET SECTOR COUNT IN COMMAND
000053E8 5B                 <1>     pop     ebx
                            <1>     ;pop    bx
                            <1>     ;pop    es
                            <1>     ;jmp    short CMD_OF          ; GO EXECUTE THE COMMAND
                            <1>     ; 01/12/2023
000053E9 E97AFFFFFF         <1>     jmp     CMD_OF
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   DISK VERIFY         (AH = 04H) :
                            <1> ;----------------------------------------
                            <1>
                            <1> DISK_VERF:
000053EE C645FE40           <1>     mov     byte [CMD_BLOCK+6], VERIFY_CMD
000053F2 E83D020000         <1>     call    COMMAND
000053F7 750C               <1>     jnz     short VERF_EXIT               ; CONTROLLER STILL BUSY
000053F9 E8F0020000         <1>     call    _WAIT                 ; (Original: CALL WAIT)
000053FE 7505               <1>     jnz     short VERF_EXIT               ; TIME OUT
00005400 E835010000         <1>     call    CHECK_STATUS
                            <1> VERF_EXIT:
00005405 C3                 <1>     retn
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   READ DASD TYPE      (AH = 15H) :
                            <1> ;----------------------------------------
                            <1>
                            <1> RETURN_DRIVE_TYPE:
                            <1>     ; 10/08/2022
                            <1>     ; 13/07/2022
                            <1>     ; (Ref: Programmer's Guide to the AMIBIOS -Page 214-, 1992)
                            <1>     ;
                            <1>     ; INPUT:
                            <1>     ;       DL = Disk number (>= 80h)
                            <1>     ;   TRDOS 386 v2.0.5 Feature:
                            <1>     ;       If AL = 0FFh, return disk size in ECX
                            <1>     ;               otherwise in CX:DX
                            <1>     ; OUTPUT:
                            <1>     ;       AH = 00h - No drive present
                            <1>     ;          = 03h - Hard disk drive
```

```
                                <1>     ;       CF = 0 - No error
                                <1>     ;          = 1 - Error
                                <1>     ;   TRDOS 386 v2.0.5 Feature:
                                <1>     ;       AL = 00h - LBA not ready !
                                <1>     ;          = 01h - LBA ready
                                <1>     ;               (28 bit or 48 bit LBA r/w depending
                                <1>     ;               on disk size in CX:DX)
                                <1>     ;       CX:DX = Number of 512 byte sectors
                                <1>     ;
                                <1>     ; (Note: High words of ECX and EDX will be zero at return)
                                <1>     ; ((If AL input is 0FFh, disk size will be in ECX only))
                                <1>
                                <1> READ_DASD_TYPE:
                                <1> READ_D_T:                           ; GET DRIVE PARAMETERS
                                <1>     ;push  ds                       ; SAVE REGISTERS
                                <1>
                                <1>     ;;push es
                                <1>     ; 18/04/2021
                                <1>     ;push  ebx
                                <1>     ;;call DDS                      ; ESTABLISH ADDRESSING
                                <1>     ;;push cs
                                <1>     ;;pop  ds
                                <1>
                                <1>     ; 18/04/2021
                                <1>     ;mov   bx, KDATA
                                <1>     ;mov   ds, bx
                                <1>     ;;mov  es, bx
                                <1>     ;mov   byte [DISK_STATUS1], 0
                                <1>     ;mov   bl, [HF_NUM]             ; GET NUMBER OF DRIVES
                                <1>     ;and   dl, 7Fh                  ; GET DRIVE NUMBER
                                <1>     ;cmp   bl, dl
                                <1>     ;jbe   short RDT_NOT_PRESENT ; RETURN DRIVE NOT PRESENT
                                <1>
                                <1>     ;mov   ax, KDATA
                                <1>     ;mov   ds, ax
                                <1>
00005406 C605[47770100]00       <1>     mov    byte [DISK_STATUS1], 0
0000540D 8A0D[48770100]         <1>     mov    cl, [HF_NUM]
00005413 80E27F                 <1>     and    dl, 7Fh
00005416 38D1                   <1>     cmp    cl, dl
00005418 7631                   <1>     jbe    short RDT_NOT_PRESENT
                                <1>
                                <1>     ; 18/04/2021 - TRDOS 386 v2.0.4
                                <1>
                                <1>     ;call  GET_VEC                  ; GET DISK PARAMETER ADDRESS
                                <1>     ;
                                <1>     ;;mov  al, [ES:BX+2]            ; HEADS
                                <1>     ;mov   al, [ebx+2]  ; heads (logical)
                                <1>     ;;;mov cl, [ES:BX+14]
                                <1>     ;;mov  cl, [ebx+14]
                                <1>     ;; 17/04/2021
                                <1>     ;mov   ah, [ebx+14] ; sectors per track (logical)
                                <1>     ;;imul cl                       ; * NUMBER OF SECTORS
                                <1>     ;;mov  cx, [ES:BX]              ; MAX NUMBER OF CYLINDERS
                                <1>     ;mov   cx, [ebx]    ; cylinders (logical)
                                <1>     ;; 02/01/2015
                                <1>     ;; ** leave the last cylinder as reserved for diagnostics **
                                <1>     ;; (Also in Award BIOS - 1999, AHDSK.ASM, FUN15 -> sub ax, 1)
                                <1>     ;dec   cx                       ; LEAVE ONE FOR DIAGNOSTICS
                                <1>     ;;imul cx                       ; NUMBER OF SECTORS
                                <1>     ;; 17/04/2021
                                <1>     ;mul   ah
                                <1>     ;; ax = spt*heads
                                <1>     ;mul   cx
                                <1>     ;; dx:ax = number of sectors
                                <1>     ;
                                <1>     ;mov   cx, dx                   ; HIGH ORDER HALF
                                <1>     ;mov   dx, ax                   ; LOW ORDER HALF
                                <1>
                                <1>     ; 18/04/2021
0000541A B102                   <1>     mov    cl, 2
0000541C 00CA                   <1>     add    dl, cl ; hd0 = 2
                                <1>
                                <1>     ; 13/07/2022
0000541E 0FB6D2                 <1>     movzx  edx, dl
00005421 8A9A[AE660000]         <1>     mov    bl, [edx+drv.status]
00005427 80E301                 <1>     and    bl, 1 ; LBA ready bit (bit 0)
                                <1>
0000542A D2E2                   <1>     shl    dl, cl ; * 4
                                <1>
                                <1>     ;mov   eax, [edx+drv.size]
                                <1>     ;mov   dx, ax
                                <1>     ;shr   eax, 16
                                <1>     ;mov   cx, ax
                                <1>
0000542C 8B8A[92660000]         <1>     mov    ecx, [edx+drv.size]
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ;cmp   al, 0FFh   ; return disk size in ecx ?
                                <1>     ;je    short RDT1 ; yes
00005432 FEC0                   <1>     inc    al  ; 0FFh -> 0
00005434 740A                   <1>     jz     short RDT1
                                <1>
00005436 6689CA                 <1>     mov    dx, cx
00005439 C1E910                 <1>     shr    ecx, 16
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; ebx = esp+20
                                <1>     ; ecx = esp+16
                                <1>     ; edx = esp+12
                                <1>     ; esi = esp+8
                                <1>     ; edi = esp+4
                                <1>
                                <1>     ; return disk size in user's registers
0000543C 8954240C               <1>     mov    [esp+12], edx
                                <1>     ; cx:dx = disk size
                                <1> RDT1:
                                <1>     ;mov   [esp+16], ecx
                                <1>
                                <1>     ;;sub  al, al
00005440 29C0                   <1>     sub    eax, eax
00005442 B403                   <1>     mov    ah, 03h                  ; INDICATE FIXED DISK
00005444 88D8                   <1>     mov    al, bl
                                <1>     ; al = 1 -> LBA r/w ready/applicable
                                <1>     ;    = 0 -> LBA r/w not ready/applicable
                                <1>     ; cf = 0
                                <1> RDT2:
00005446 894C2410               <1>     mov    [esp+16], ecx
                                <1> ;RDT2:
                                <1>     ; 13/07/2022
                                <1>     ; 18/04/2021
                                <1>     ;pop   ebx                      ; RESTORE REGISTERS
                                <1>     ;;pop  es
                                <1>     ;pop   ds
```

```
                                <1>     ; (*) clc                       ; CLEAR CARRY
                                <1>     ;retf  2
                                <1>     ; (*) 29/05/2016
                                <1>     ; (*) retf 4
                                <1>     ;and   byte [esp+8], 0FEh ; clear carry bit of eflags register
                                <1>     ;iretd
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; [DISK_STATUS1] = 0
                                <1>     ; ah = 3
                                <1>     ; al = 0 or 1 (LBA ready)
                                <1>     ; cf = 0
                                <1>
0000544A C3                     <1>     retn
                                <1>
                                <1> RDT_NOT_PRESENT:
                                <1>     ;;sub  ax, ax                   ; DRIVE NOT PRESENT RETURN
                                <1>     ;; 18/04/2021
                                <1>     ;sub   eax, eax
                                <1>     ;;mov  cx, ax                   ; ZERO BLOCK COUNT
                                <1>     ;;mov  dx, ax
                                <1>     ;mov   ecx, eax
                                <1>     ;mov   edx, eax
                                <1>     ;jmp   short RDT2
                                <1>     ; 13/07/2022
0000544B 29C9                   <1>     sub    ecx, ecx
0000544D 88C1                   <1>     mov    cl, al ; if AL = 0FFh, disk size will be in ECX
                                <1>                   ; if not, disk size will be in CX:DX
0000544F 29C0                   <1>     sub    eax, eax
00005451 FEC1                   <1>     inc    cl ; 0FFh -> 0
00005453 80F901                 <1>     cmp    cl, 1
00005456 72EE                   <1>     jb     short RDT2 ; ecx = 0
                                <1>     ; 10/08/2022
00005458 28C9                   <1>     sub    cl, cl
                                <1>     ; ecx = 0
0000545A 8944240C               <1>     mov    [esp+12], eax ; edx = 0
0000545E F9                     <1>     stc
0000545F EBE5                   <1>     jmp    short RDT2 ; cf = 1, eax = 0
                                <1>
                                <1> ; 10/08/2022
                                <1> ; 07/08/2022
                                <1> ; 13/07/2022 - TRDOS 386 v2.0.5
                                <1> ; 28/05/2016
                                <1> ; 27/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   GET PARAMETERS      (AH = 08H) :
                                <1> ;----------------------------------------
                                <1>
                                <1> GET_PARM_N:
                                <1>     ; ebx = user's buffer address for parameters table
                                <1>     ; 10/08/2022
                                <1>     ; 13/07/2022
                                <1>     ; (if ebx = 0, HDPT will not be returned to user)
                                <1>     ;
                                <1> ;GET_PARM:                          ; GET DRIVE PARAMETERS
                                <1>     ;push  ds                       ; SAVE REGISTERS
                                <1>     ;push  es
                                <1>
                                <1>     ;push  ebx
00005461 89DF                   <1>     mov    edi, ebx ; 13/07/2022
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; ((IBM PC XT-286 ROM BIOS source code remainders))
                                <1>     ;;mov  ax, ABS0                 ; ESTABLISH ADDRESSING
                                <1>     ;;mov  ds, ax
                                <1>
                                <1>     ;;test dl, 1                    ; CHECK FOR DRIVE 1
                                <1>     ;;jz   short G0
                                <1>     ;;les  bx, @HF1_TBL_VEC
                                <1>     ;;jmp  short G1
                                <1> ;;G0:
                                <1>     ;les   bx, @HF_TBL_VEC
                                <1> ;;G1:
                                <1>     ;;call DDS                      ; ESTABLISH SEGMENT
                                <1>
                                <1>     ; 13/07/2022
                                <1>     ; 22/12/2014
                                <1>     ;;push cs
                                <1>     ;;pop  ds
                                <1>     ;mov   bx, KDATA
                                <1>     ;mov   ds, bx
                                <1>     ;mov   es, bx  ; 27/05/2016
                                <1>     ;
                                <1>     ; 18/04/2021
00005463 29C9                   <1>     sub    ecx, ecx
                                <1>     ;
00005465 80EA80                 <1>     sub    dl, 80h
00005468 80FA04                 <1>     cmp    dl, MAX_FILE             ; TEST WITHIN RANGE
0000546B 7344                   <1>     jae    short G2 ; 13/07/2022
                                <1>     ;
                                <1>     ; 21/02/2015
0000546D 31DB                   <1>     xor    ebx, ebx
                                <1>     ; 18/04/2021
                                <1>     ;sub   ecx, ecx
                                <1>     ; 22/12/2014
0000546F 88D3                   <1>     mov    bl, dl
                                <1>     ;xor   bh, bh
00005471 C0E302                 <1>     shl    bl, 2                    ; convert index to offset
                                <1>     ;add   bx, HF_TBL_VEC
00005474 81C3[4C770100]         <1>     add    ebx, HF_TBL_VEC
                                <1>     ;mov   ax, [bx+2]
                                <1>     ;mov   es, ax                   ; dpt segment
                                <1>     ;mov   bx, [bx]                 ; dpt offset
0000547A 8B1B                   <1>     mov    ebx, [ebx] ; 32 bit offset
                                <1>     ; 18/04/2021
0000547C 29D2                   <1>     sub    edx, edx
0000547E 8815[47770100]         <1>     mov    [DISK_STATUS1], dl ; 0
                                <1>
                                <1>     ;mov   byte [DISK_STATUS1], 0
                                <1>         ;mov       ax, [ES:BX]           ; MAX NUMBER OF CYLINDERS
00005484 668B03                 <1>     mov    ax, [ebx]
                                <1>     ;;sub  ax, 2                    ; ADJUST FOR 0-N
00005487 6648                   <1>     dec    ax                       ; max. cylinder number
00005489 88C5                   <1>     mov    ch, al
0000548B 66250003               <1>     and    ax, 0300h                ; HIGH TWO BITS OF CYLINDER
                                <1>     ;shr   ax, 1
                                <1>     ;shr   ax, 1
                                <1>     ; 13/07/2022
                                <1>     ;shr   ax, 2
                                <1>     ; 07/08/2022
0000548F C1E802                 <1>     shr    eax, 2
                                <1>     ;or    al, [ES:BX+14]           ; SECTORS
00005492 0A430E                 <1>     or     al, [ebx+14]
```

```
00005495 88C1                <1>     mov     cl, al
                             <1>     ;mov    dh, [ES:BX+2]          ; HEADS
00005497 8A7302              <1>     mov     dh, [ebx+2]
0000549A FECE                <1>     dec     dh                     ; 0-N RANGE
0000549C 8A15[48770100]      <1>     mov     dl, [HF_NUM]           ; DRIVE COUNT
                             <1>     ;;sub   ax, ax
                             <1>     ; 18/04/2021
                             <1>     ;sub    eax, eax
                             <1>
                             <1>     ; 27/12/2014
                             <1>     ;mov    di, bx                 ; HDPT offset
                             <1>
                             <1>     ; 13/07/2022
                             <1>     ; ebx = esp+20
                             <1>     ; ecx = esp+16
                             <1>     ; edx = esp+12
                             <1>     ; esi = esp+8
                             <1>     ; edi = esp+4
                             <1>
                             <1>     ; 13/07/2022
                             <1>     ; set return register contents/values
000054A2 894C2410            <1>     mov     [esp+16], ecx
000054A6 8954240C            <1>     mov     [esp+12], edx
                             <1>
                             <1>     ; is hard disk parameters table requested ?
000054AA 09FF                <1>     or      edi, edi ; (edi = [ebp+24] = ebx)
000054AC 751B                <1>     jnz     short G3 ; yes
                             <1>
000054AE 29C0                <1>     sub     eax, eax
                             <1>
                             <1>     ; [DISK_STATUS1] = 0
                             <1>     ; eax = 0
                             <1>     ; cf = 0
                             <1>
000054B0 C3                  <1>     retn
                             <1>
                             <1> G2:
                             <1>     ;mov    ah, INIT_FAIL
                             <1>     ;mov    byte [DISK_STATUS1], ah ; (INIT_FAIL)
                             <1>     ;                              ; OPERATION FAILED
                             <1>     ;sub    al, al
                             <1>     ;sub    dx, dx
                             <1>     ;sub    cx, cx
                             <1>     ; 18/04/2021
000054B1 29C0                <1>     sub     eax, eax
000054B3 B407                <1>     mov     ah, INIT_FAIL
000054B5 8825[47770100]      <1>     mov      [DISK_STATUS1], ah    ; OPERATION FAILED
                             <1>
                             <1>     ; 13/07/2022
                             <1>     ;sub    edx, edx
                             <1>     ;sub    ecx, ecx
                             <1>     ; ecx = 0
000054BB 894C240C            <1>     mov     [esp+12], ecx ; 0 ; edx (heads-1, drive count)
000054BF 894C2410            <1>     mov     [esp+16], ecx ; 0 ; ecx (cylinders-1, sectors)
000054C3 894C2414            <1>     mov     [esp+20], ecx ; 0 ; ebx (HDPT address)
                             <1>
000054C7 F9                  <1>     stc
000054C8 C3                  <1>     retn
                             <1>
                             <1>     ; 13/07/2022
                             <1>     ; 29/05/2016 (*)
                             <1>     ;;stc                          ; SET ERROR FLAG
                             <1>     ;;jmp   short G5
                             <1>     ;jmp    short _G6
                             <1>
                             <1> G3:
                             <1>     ; 27/05/2016
                             <1>     ; return fixed disk parameters table to user
                             <1>     ; in user's buffer, which is pointed by EBX
                             <1>
                             <1>     ;xchg   edi, [esp]             ; ebx (input)-> edi, edi -> [esp]
                             <1>     ; 13/07/2022
                             <1>     ;pop    edi
                             <1>     ; edi = user's buffer address
                             <1>     ;push   esi
000054C9 89DE                <1>     mov     esi, ebx               ; hard disk parameter table (32 bytes)
                             <1>     ;mov    ebx, edi               ; ebx = user's buffer address
000054CB 51                  <1>     push    ecx
                             <1>     ;push   eax
                             <1>     ;mov    ecx, 32 ; 32 bytes
000054CC 30ED                <1>     xor     ch, ch
000054CE B120                <1>     mov     cl, 32
                             <1>     ; ecx = 32
000054D0 E846B60000          <1>     call    transfer_to_user_buffer ; trdosk6.s (16/05/2016)
                             <1>     ;pop    eax
000054D5 59                  <1>     pop     ecx
                             <1>     ; 10/08/2022
                             <1>     ;pop    esi
                             <1>     ;pop    edi
000054D6 7306                <1>     jnc     short G4
                             <1>     ; 29/05/2016 (*)
000054D8 B8FF000000          <1>     mov     eax, 0FFh ; unknown error !
                             <1>     ; [DISK_STATUS1] = 0
                             <1>     ; ah = 0, al = 0FFh
                             <1>     ; cf = 1
                             <1>
000054DD C3                  <1>     retn
                             <1> ;_G6:
                             <1>     ;or     byte [esp+16], 1 ; set carry bit of eflags register
                             <1> ;G5:
                             <1>     ; 27/05/2016
                             <1>     ;pop    ebx                    ; RESTORE REGISTERS
                             <1>     ;pop    es
                             <1>     ;pop    ds
                             <1>     ;;retf 2
                             <1>     ; (*) 29/05/2016
                             <1>     ; (*) retf 4
                             <1>     ; (*) or byte [esp+8], 1 ; set carry bit of eflags register
                             <1>     ;iretd
                             <1>
                             <1> G4:
                             <1>     ; 13/07/2022
000054DE 31C0                <1>     xor     eax, eax
                             <1>
                             <1>     ; [user_buffer] = [ebp+24] = HDPT
                             <1>     ; [DISK_STATUS1] = 0
                             <1>     ; eax = 0
                             <1>     ; cf = 0
                             <1>
000054E0 C3                  <1>     retn
                             <1>
                             <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                             <1>
                             <1> ;----------------------------------------
```

```
                                <1> ;   INITIALIZE DRIVE     (AH = 09H) :
                                <1> ;----------------------------------------
                                <1>     ; 03/01/2015
                                <1>     ; According to ATA-ATAPI specification v2.0 to v5.0
                                <1>     ; logical sector per logical track
                                <1>     ; and logical heads - 1 would be set but
                                <1>     ; it is seen as it will be good
                                <1>     ; if physical parameters will be set here
                                <1>     ; because, number of heads <= 16.
                                <1>     ; (logical heads usually more than 16)
                                <1>     ; NOTE: ATA logical parameters (software C, H, S)
                                <1>     ;       == INT 13h physical parameters
                                <1>
                                <1> ;INIT_DRV:
                                <1> ;   mov     byte [CMD_BLOCK+6], SET_PARM_CMD
                                <1> ;   call    GET_VEC                 ; ES:BX -> PARAMETER BLOCK
                                <1> ;   mov     al, [es:bx+2]           ; GET NUMBER OF HEADS
                                <1> ;   dec     al                      ; CONVERT TO 0-INDEX
                                <1> ;   mov     ah, [CMD_BLOCK+5]       ; GET SDH REGISTER
                                <1> ;   and     ah, 0F0h                ; CHANGE HEAD NUMBER
                                <1> ;   or      ah, al                  ; TO MAX HEAD
                                <1> ;   mov     [CMD_BLOCK+5], ah
                                <1> ;   mov     al, [es:bx+14]          ; MAX SECTOR NUMBER
                                <1> ;   mov     [CMD_BLOCK+1], al
                                <1> ;   sub     ax, ax
                                <1> ;   mov     [CMD_BLOCK+3], al       ; ZERO FLAGS
                                <1> ;   call    COMMAND                 ; TELL CONTROLLER
                                <1> ;   jnz     short INIT_EXIT                 ; CONTROLLER BUSY ERROR
                                <1> ;   call    NOT_BUSY                ; WAIT FOR IT TO BE DONE
                                <1> ;   jnz     short INIT_EXIT                 ; TIME OUT
                                <1> ;   call    CHECK_STATUS
                                <1> ;INIT_EXIT:
                                <1> ;   retn
                                <1>
                                <1> ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ; 04/01/2015
                                <1> ; 02/01/2015 - Derived from from AWARD BIOS 1999
                                <1> ;                           AHDSK.ASM - INIT_DRIVE
                                <1> INIT_DRV:
                                <1>     ;xor    ah,ah
000054E1 31C0                   <1>     xor     eax, eax ; 21/02/2015
000054E3 B00B                   <1>     mov     al, 11 ; Physical heads from translated HDPT
000054E5 3825[5C770100]         <1>         cmp     [LBAMode], ah   ; 0
000054EB 7702                   <1>     ja      short idrv0
000054ED B002                   <1>     mov     al, 2  ; Physical heads from standard HDPT
                                <1> idrv0:
                                <1>     ; DL = drive number (0 based)
000054EF E8DF020000             <1>     call    GET_VEC
                                <1>     ;push   bx
000054F4 53                     <1>     push    ebx ; 21/02/2015
                                <1>     ;add    bx, ax
000054F5 01C3                   <1>     add     ebx, eax
                                <1>     ;; 05/01/2015
000054F7 8A25[82660000]         <1>     mov     ah, [hf_m_s] ; drive number (0= master, 1= slave)
                                <1>     ;;and   ah, 1
000054FD C0E404                 <1>     shl     ah, 4
00005500 80CCA0                 <1>     or      ah, 0A0h  ; Drive/Head register - 10100000b (A0h)
                                <1>     ;mov    al, [es:bx]
00005503 8A03                   <1>     mov     al, [ebx] ; 21/02/2015
00005505 FEC8                   <1>     dec     al      ; last head number
                                <1>     ;and    al, 0Fh
00005507 08E0                   <1>     or      al, ah  ; lower 4 bits for head number
                                <1>     ;
00005509 C645FE91               <1>     mov     byte [CMD_BLOCK+6], SET_PARM_CMD
0000550D 8845FD                 <1>     mov     [CMD_BLOCK+5], al
                                <1>     ;pop    bx
00005510 5B                     <1>     pop     ebx
00005511 29C0                   <1>     sub     eax, eax ; 21/02/2015
00005513 B004                   <1>     mov     al, 4  ; Physical sec per track from translated HDPT
00005515 803D[5C770100]00       <1>     cmp     byte [LBAMode], 0
0000551C 7702                   <1>     ja      short idrv1
0000551E B00E                   <1>     mov     al, 14 ; Physical sec per track from standard HDPT
                                <1> idrv1:
                                <1>     ;xor    ah, ah
                                <1>     ;add    bx, ax
00005520 01C3                   <1>     add     ebx, eax ; 21/02/2015
                                <1>     ;mov    al, [es:bx]
                                <1>                     ; sector number
00005522 8A03                   <1>     mov     al, [ebx]
00005524 8845F9                 <1>     mov     [CMD_BLOCK+1], al
00005527 28C0                   <1>     sub     al, al
00005529 8845FB                 <1>     mov     [CMD_BLOCK+3], al ; ZERO FLAGS
0000552C E803010000             <1>     call    COMMAND         ; TELL CONTROLLER
00005531 751E                   <1>     jnz     short INIT_EXIT         ; CONTROLLER BUSY ERROR
00005533 E8E6010000             <1>     call    NOT_BUSY        ; WAIT FOR IT TO BE DONE
00005538 7517                   <1>     jnz     short INIT_EXIT         ; TIME OUT
                                <1>     ; 16/07/2022
                                <1>     ;call   CHECK_STATUS
                                <1>     ;jmp    short CHECK_STATUS
                                <1> ;INIT_EXIT:
                                <1>     ;retn
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 10/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   CHECK FIXED DISK STATUS      :
                                <1> ;----------------------------------------
                                <1> CHECK_STATUS:
0000553A E831020000             <1>     call    CHECK_ST                ; CHECK THE STATUS BYTE
                                <1>     ;jnz    short CHECK_S1          ; AN ERROR WAS FOUND
                                <1>     ; 10/07/2022
0000553F 7510                   <1>     jnz     short CHECK_S2
00005541 A801                   <1>     test    al, ST_ERROR            ; WERE THERE ANY OTHER ERRORS
00005543 7405                   <1>     jz      short CHECK_S1          ; NO ERROR REPORTED
00005545 E866020000             <1>     call    CHECK_ER                ; ERROR REPORTED
                                <1> CHECK_S1:
0000554A 803D[47770100]00       <1>     cmp     byte [DISK_STATUS1], 0        ; SET STATUS FOR CALLER
                                <1> CHECK_S2:
                                <1> INIT_EXIT:  ; 10/07/2022
00005551 C3                     <1>     retn
                                <1>
                                <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   SEEK                 (AH = 0CH) :
                                <1> ;----------------------------------------
                                <1>
                                <1> DISK_SEEK:
00005552 C645FE70               <1>         mov byte [CMD_BLOCK+6], SEEK_CMD
00005556 E8D9000000             <1>     call    COMMAND
0000555B 751C                   <1>     jnz     short DS_EXIT           ; CONTROLLER BUSY ERROR
0000555D E88C010000             <1>     call    _WAIT
```

```
00005562 7515                <1>         jnz short DS_EXIT         ; TIME OUT ON SEEK
00005564 E8D1FFFFFF          <1>     call    CHECK_STATUS
00005569 803D[47770100]40    <1>         cmp byte [DISK_STATUS1], BAD_SEEK
00005570 7507                <1>     jne     short DS_EXIT
00005572 C605[47770100]00    <1>         mov byte [DISK_STATUS1], 0
                             <1> DS_EXIT:
00005579 C3                  <1>     retn
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   TEST DISK READY      (AH = 10H) :
                             <1> ;----------------------------------------
                             <1>
                             <1> TST_RDY:                          ; WAIT FOR CONTROLLER
0000557A E89F010000          <1>     call    NOT_BUSY
0000557F 752C                <1>     jnz     short TR_EX
                             <1>     ;;;
                             <1>     ; 23/06/2024
00005581 803D[5C770100]FF    <1>     cmp     byte [LBAMode], 0FFh
00005588 7507                <1>     jne     short tst_28bit_rdy
0000558A A0[82660000]        <1>     mov     al, [hf_m_s]
0000558F EB03                <1>     jmp     short tst_48bit_rdy
                             <1> tst_28bit_rdy:
                             <1>     ;;;
00005591 8A45FD              <1>     mov     al, [CMD_BLOCK+5]     ; SELECT DRIVE
                             <1> tst_48bit_rdy:
00005594 668B15[7E660000]    <1>     mov     dx, [HF_PORT]
0000559B 80C206              <1>     add     dl, 6
0000559E EE                  <1>     out     dx, al
0000559F E8CC010000          <1>     call    CHECK_ST              ; CHECK STATUS ONLY
000055A4 7507                <1>     jnz     short TR_EX
000055A6 C605[47770100]00    <1>     mov     byte [DISK_STATUS1], 0        ; WIPE OUT DATA CORRECTED ERROR
                             <1> TR_EX:
000055AD C3                  <1>     retn
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   RECALIBRATE         (AH = 11H) :
                             <1> ;----------------------------------------
                             <1>
                             <1> HDISK_RECAL:
000055AE C645FE10            <1>         mov byte [CMD_BLOCK+6], RECAL_CMD ; 10h, 16
000055B2 E87D000000          <1>     call    COMMAND               ; START THE OPERATION
000055B7 7523                <1>     jnz     short RECAL_EXIT      ; ERROR
000055B9 E830010000          <1>     call    _WAIT                 ; WAIT FOR COMPLETION
000055BE 7407                <1>     jz      short RECAL_X         ; TIME OUT ONE OK ?
000055C0 E829010000          <1>     call    _WAIT                 ; WAIT FOR COMPLETION LONGER
000055C5 7515                <1>     jnz     short RECAL_EXIT      ; TIME OUT TWO TIMES IS ERROR
                             <1> RECAL_X:
000055C7 E86EFFFFFF          <1>     call    CHECK_STATUS
000055CC 803D[47770100]40    <1>     cmp     byte [DISK_STATUS1], BAD_SEEK ; SEEK NOT COMPLETE
000055D3 7507                <1>     jne     short RECAL_EXIT      ; IS OK
000055D5 C605[47770100]00    <1>     mov     byte [DISK_STATUS1], 0
                             <1> RECAL_EXIT:
000055DC 803D[47770100]00    <1>         cmp byte [DISK_STATUS1], 0
000055E3 C3                  <1>     retn
                             <1>
                             <1> ;----------------------------------------
                             <1> ;      CONTROLLER DIAGNOSTIC (AH = 14H) :
                             <1> ;----------------------------------------
                             <1>
                             <1> CTLR_DIAGNOSTIC:
                             <1>     ; 07/08/2022 - TRDOS 386 v2.0.5
000055E4 FA                  <1>     cli                           ; DISABLE INTERRUPTS WHILE CHANGING MASK
000055E5 E4A1                <1>     in      al, INTB01            ; TURN ON SECOND INTERRUPT CHIP
                             <1>     ;and    al, 0BFH
000055E7 243F                <1>     and     al, 3Fh               ; enable IRQ 14 & IRQ 15
                             <1>     ;jmp    $+2
                             <1>     IODELAY
000055E9 EB00                <2>  jmp short $+2
000055EB EB00                <2>  jmp short $+2
000055ED E6A1                <1>     out     INTB01, al
                             <1>     IODELAY
000055EF EB00                <2>  jmp short $+2
000055F1 EB00                <2>  jmp short $+2
000055F3 E421                <1>     in      al, INTA01            ; LET INTERRUPTS PASS THRU TO
000055F5 24FB                <1>     and     al, 0FBh              ;  SECOND CHIP
                             <1>     ;jmp    $+2
                             <1>     IODELAY
000055F7 EB00                <2>  jmp short $+2
000055F9 EB00                <2>  jmp short $+2
000055FB E621                <1>     out     INTA01, al
000055FD FB                  <1>     sti
000055FE E81B010000          <1>     call    NOT_BUSY              ; WAIT FOR CARD
00005603 7526                <1>     jnz     short CD_ERR          ; BAD CARD
                             <1>     ;mov    dx, PORT+7
00005605 668B15[7E660000]    <1>     mov     dx, [HF_PORT]
0000560C 80C207              <1>     add     dl, 7
0000560F B090                <1>     mov     al, DIAG_CMD          ; START DIAGNOSE
00005611 EE                  <1>     out     dx, al
00005612 E807010000          <1>     call    NOT_BUSY              ; WAIT FOR IT TO COMPLETE
00005617 B480                <1>     mov     ah, TIME_OUT
00005619 7512                <1>     jnz     short CD_EXIT         ; TIME OUT ON DIAGNOSTIC
                             <1>     ;mov    dx, HF_PORT+1         ; GET ERROR REGISTER
0000561B 668B15[7E660000]    <1>     mov     dx, [HF_PORT]
00005622 FEC2                <1>     inc     dl
00005624 EC                  <1>     in      al, dx
                             <1>     ; 07/08/2022
                             <1>     ;mov    [HF_ERROR], al        ; SAVE IT
00005625 B400                <1>     mov     ah, 0
00005627 3C01                <1>     cmp     al, 1                 ; CHECK FOR ALL OK
00005629 7402                <1>     je      short CD_EXIT
                             <1> CD_ERR:
0000562B B420                <1>     mov     ah, BAD_CNTLR
                             <1> CD_EXIT:
0000562D 8825[47770100]      <1>     mov     [DISK_STATUS1], ah
00005633 C3                  <1>     retn
                             <1>
                             <1>     ; 20/06/2024 - TRDOS 386 v2.0.8
                             <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 10/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                             <1>
                             <1> ;-------------------------------------------------------
                             <1> ; COMMAND                                        :
                             <1> ;   THIS ROUTINE OUTPUTS THE COMMAND BLOCK              :
                             <1> ; OUTPUT                                         :
                             <1> ;   BL = STATUS                                  :
                             <1> ;   BH = ERROR REGISTER                          :
                             <1> ;-------------------------------------------------------
                             <1>
                             <1> COMMAND:
                             <1>     ;push   ebx ; 10/07/2022      ; WAIT FOR SEEK COMPLETE AND READY
                             <1>     ;;mov   ecx, DELAY_2          ; SET INITIAL DELAY BEFORE TEST
                             <1> COMMAND1:
                             <1>     ;;push  ecx                   ; SAVE LOOP COUNT
00005634 E841FFFFFF          <1>     call    TST_RDY               ; CHECK DRIVE READY
```

```
                                <1>     ;;pop   ecx
                                <1>     ;pop    ebx ; 10/07/2022
00005639 7418                   <1>     jz      short COMMAND2          ; DRIVE IS READY
0000563B 803D[47770100]80       <1>         cmp byte [DISK_STATUS1], TIME_OUT ; TST_RDY TIMED OUT--GIVE UP
                                <1>     ;jz     short CMD_TIMEOUT
                                <1>     ;;loop  COMMAND1                ; KEEP TRYING FOR A WHILE
                                <1>     ;jmp    short COMMAND4          ; ITS NOT GOING TO GET READY
00005642 7507                   <1>     jne     short COMMAND4
                                <1> CMD_TIMEOUT:
00005644 C605[47770100]20       <1>     mov     byte [DISK_STATUS1], BAD_CNTLR
                                <1> COMMAND4:
                                <1>     ;;pop   ebx ; 10/07/2022
0000564B 803D[47770100]00       <1>         cmp byte [DISK_STATUS1], 0; SET CONDITION CODE FOR CALLER
00005652 C3                     <1>     retn
                                <1> COMMAND2:
                                <1>     ;;pop   ebx ; 10/07/2022
                                <1>     ;push   edi ; 10/07/2022
00005653 C605[42770100]00       <1>     mov     byte [HF_INT_FLAG], 0 ; RESET INTERRUPT FLAG
0000565A FA                     <1>     cli                             ; INHIBIT INTERRUPTS WHILE CHANGING MASK
0000565B E4A1                   <1>     in      al, INTB01              ; TURN ON SECOND INTERRUPT CHIP
                                <1>     ;and    al, 0BFh
0000565D 243F                   <1>     and     al, 3Fh                 ; Enable IRQ 14 & 15
                                <1>     ;jmp    $+2
                                <1>     IODELAY
0000565F EB00                   <2>  jmp short $+2
00005661 EB00                   <2>  jmp short $+2
00005663 E6A1                   <1>     out     INTB01, al
00005665 E421                   <1>     in      al, INTA01              ; LET INTERRUPTS PASS THRU TO
00005667 24FB                   <1>     and     al, 0FBh                ; SECOND CHIP
                                <1>     ;jmp    $+2
                                <1>     IODELAY
00005669 EB00                   <2>  jmp short $+2
0000566B EB00                   <2>  jmp short $+2
0000566D E621                   <1>     out     INTA01, al
0000566F FB                     <1>     sti
                                <1>     ;xor    edi, edi                ; INDEX THE COMMAND TABLE
                                <1>     ; 10/07/2022
00005670 31C9                   <1>     xor     ecx, ecx
                                <1>
                                <1>     ;mov    dx, HF_PORT+1           ; DISK ADDRESS
00005672 668B15[7E660000]       <1>     mov     dx, [HF_PORT]
00005679 FEC2                   <1>     inc     dl
                                <1>
                                <1>     ; 20/06/2024
                                <1>     ;mov    al, [CMD_BLOCK+6]
                                <1>     ;cmp    al, 24h ; READ SECTOR(S) EXT
                                <1>     ;je     short COMMAND5
                                <1>     ;cmp    al, 34h ; WRITE SECTOR(S) EXT
                                <1>     ;je     short COMMAND5
                                <1>     ; 23/06/2024
0000567B 803D[5C770100]FF       <1>     cmp     byte [LBAMode], 0FFh
00005682 7428                   <1>     je      short COMMAND5 ; 48 bit LBA read/write
                                <1>
                                <1>     ; 20/06/2024
                                <1>     ; dx = hd base port + 1 ; 1F1h or 171h
                                <1>
00005684 F605[49770100]C0       <1>     test    byte [CONTROL_BYTE], 0C0h ; CHECK FOR RETRY SUPPRESSION
0000568B 740E                   <1>     jz      short COMMAND3
                                <1>     ; 20/06/2024
                                <1>     ;mov    al, [CMD_BLOCK+6]       ; YES-GET OPERATION CODE
0000568D 24F0                   <1>     and     al, 0F0h                ; GET RID OF MODIFIERS
0000568F 3C20                   <1>     cmp     al, 20h                 ; 20H-40H IS READ, WRITE, VERIFY
00005691 7208                   <1>     jb      short COMMAND3
00005693 3C40                   <1>     cmp     al, 40h
00005695 7704                   <1>     ja      short COMMAND3
00005697 804DFE01               <1>     or      byte [CMD_BLOCK+6], NO_RETRIES
                                <1>                                     ; VALID OPERATION FOR RETRY SUPPRESS
                                <1> COMMAND3:
                                <1>     ;mov    al, [CMD_BLOCK+edi]     ; GET THE COMMAND STRING BYTE
                                <1>     ; 10/07/2022
0000569B 8A440DF8               <1>     mov     al, [CMD_BLOCK+ecx]
0000569F EE                     <1>     out     dx, al                  ; GIVE IT TO CONTROLLER
                                <1>     IODELAY
000056A0 EB00                   <2>  jmp short $+2
000056A2 EB00                   <2>  jmp short $+2
                                <1>     ;inc    edi                     ; NEXT BYTE IN COMMAND BLOCK
                                <1>     ; 10/07/2022
000056A4 41                     <1>     inc     ecx
                                <1>     ;inc    dx                      ; NEXT DISK ADAPTER REGISTER
000056A5 42                     <1>     inc     edx   ; 10/07/2022
                                <1>     ;cmp    di, 7 ; 01/01/2015      ; ALL DONE?
                                <1>     ;jne    short COMMAND3          ; NO--GO DO NEXT ONE
000056A6 80F907                 <1>     cmp     cl, 7 ; 10/07/2022
000056A9 72F0                   <1>     jb      short COMMAND3
                                <1>     ;pop    edi ; 10/07/2022
000056AB C3                     <1>     retn                            ; ZERO FLAG IS SET
                                <1>
                                <1>     ; 20/06/2024
                                <1> COMMAND5:
                                <1>     ; 48 bit LBA r/w
000056AC 8A45F8                 <1>     mov     al, [CMD_BLOCK] ; 0
000056AF EE                     <1>     out     dx, al  ; hd base port + 1
                                <1>     ;
000056B0 80C205                 <1>     add     dl, 5   ; hd base port + 6 ; 1F6h or 176h
000056B3 A0[82660000]           <1>     mov     al, [hf_m_s]
                                <1>     ; 23/06/2024
                                <1>     ;shl    al, 4
                                <1>     ;;add   al, 40h
                                <1>     ;or     al, 40h
                                <1>     ;; al = 40h (for master), 50h (for slave)
                                <1>     ;
000056B8 EE                     <1>     out     dx, al
000056B9 80EA04                 <1>     sub     dl, 4   ; hd base port + 2 ; 1F2h or 172h
000056BC 30C0                   <1>     xor     al, al  ; 0
000056BE EE                     <1>     out     dx, al  ; sector count hb (bits 8 to 15) = 0
000056BF 42                     <1>     inc     edx     ; hd base port + 3
000056C0 8B45FA                 <1>     mov     eax, [CMD_BLOCK+2]
000056C3 C1C008                 <1>     rol     eax, 8
000056C6 EE                     <1>     out     dx, al  ; LBA byte 4 (bits 24 to 31)
000056C7 42                     <1>     inc     edx     ; hd base port + 4
000056C8 30C0                   <1>     xor     al, al
000056CA EE                     <1>     out     dx, al  ; LBA byte 5 (bits 32 to 39) = 0
000056CB 42                     <1>     inc     edx     ; hd base port + 5
000056CC 28C0                   <1>     sub     al, al
000056CE EE                     <1>     out     dx, al  ; LBA byte 6 (bits 40 to 47) = 0
000056CF 8A45F9                 <1>     mov     al, [CMD_BLOCK+1] ; sector count
000056D2 80EA03                 <1>     sub     dl, 3   ; hd base port + 2
000056D5 EE                     <1>     out     dx, al  ; sector count lb (bits 0 to 7)
000056D6 C1E808                 <1>     shr     eax, 8
000056D9 42                     <1>     inc     edx     ; hd base port + 3 ; 1F3h or 173h
000056DA EE                     <1>     out     dx, al  ; LBA byte 1 (bits 0 to 7)
000056DB C1E808                 <1>     shr     eax, 8
000056DE 42                     <1>     inc     edx     ; hd base port + 4
000056DF EE                     <1>     out     dx, al  ; LBA byte 2 (bits 8 to 15)
```

```
000056E0 C1E808              <1>     shr     eax, 8
000056E3 42                  <1>     inc     edx     ; hd base port + 5
000056E4 EE                  <1>     out     dx, al  ; LBA byte 3 (bits 16 to 23
000056E5 42                  <1>     inc     edx     ; hd base port + 6
000056E6 8A45FE              <1>     mov     al, [CMD_BLOCK+6] ; 48 bit read or write command
                             <1>     ; al = 24h or 34h
000056E9 42                  <1>     inc     edx     ; hd base port + 7 ; 1F7h or 177h
000056EA EE                  <1>     out     dx, al
000056EB 31C0                <1>     xor     eax, eax ; 0
                             <1>     ; zf = 1
000056ED C3                  <1>     retn
                             <1>
                             <1> ;CMD_TIMEOUT:
                             <1> ;   mov     byte [DISK_STATUS1], BAD_CNTLR
                             <1> ;COMMAND4:
                             <1> ;   pop     ebx
                             <1> ;   cmp     byte [DISK_STATUS1], 0        ; SET CONDITION CODE FOR CALLER
                             <1> ;   retn
                             <1>
                             <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 10/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   WAIT FOR INTERRUPT            :
                             <1> ;----------------------------------------
                             <1> ;WAIT:
                             <1> _WAIT:
000056EE FB                  <1>     sti                             ; MAKE SURE INTERRUPTS ARE ON
                             <1>     ;sub    cx, cx                  ; SET INITIAL DELAY BEFORE TEST
                             <1>     ;clc
                             <1>     ;mov    ax, 9000h               ; DEVICE WAIT INTERRUPT
                             <1>     ;int    15h
                             <1>     ;jc     short WT2               ; DEVICE TIMED OUT
                             <1>     ;mov    bl, DELAY_1             ; SET DELAY COUNT
                             <1>
                             <1>     ;mov    bl, WAIT_HDU_INT_HI
                             <1>     ;; 21/02/2015
                             <1>     ;;mov   bl, WAIT_HDU_INT_HI + 1
                             <1>     ;;mov   cx, WAIT_HDU_INT_LO
000056EF B915160500          <1>     mov     ecx, WAIT_HDU_INT_LH
                             <1>                                     ; (AWARD BIOS -> WAIT_FOR_MEM)
                             <1> ;-----      WAIT LOOP
                             <1>
                             <1> WT1:
                             <1>     ;test   byte [HF_INT_FLAG], 80h       ; TEST FOR INTERRUPT
000056F4 F605[42770100]C0    <1>     test    byte [HF_INT_FLAG], 0C0h
                             <1>     ;loopz  WT1
000056FB 7512                <1>     jnz     short WT3               ; INTERRUPT--LETS GO
                             <1>     ;dec    bl
                             <1>     ;jnz    short WT1               ; KEEP TRYING FOR A WHILE
                             <1>
                             <1> WT1_hi:
000056FD E461                <1>     in      al, SYS1 ; 61h (PORT_B)       ; wait for lo to hi
000056FF A810                <1>     test    al, 10h                 ; transition on memory
00005701 75FA                <1>     jnz     short WT1_hi            ; refresh.
                             <1> WT1_lo:
00005703 E461                <1>     in      al, SYS1                ; 061h (PORT_B)
00005705 A810                <1>     test    al, 10h
00005707 74FA                <1>     jz      short WT1_lo
00005709 E2E9                <1>     loop    WT1
                             <1>     ;;or    bl, bl
                             <1>     ;;jz    short WT2
                             <1>     ;;dec   bl
                             <1>     ;;jmp   short WT1
                             <1>     ;dec    bl
                             <1>     ;jnz    short WT1
                             <1> WT2:
                             <1>     ; 10/07/2022
                             <1>     ;mov    byte [DISK_STATUS1], TIME_OUT ; REPORT TIME OUT ERROR
0000570B B080                <1>     mov     al, TIME_OUT
0000570D EB07                <1>     jmp     short WT4
                             <1> WT3:
                             <1>     ;mov    byte [DISK_STATUS1], 0
                             <1>     ;mov    byte [HF_INT_FLAG], 0
0000570F 28C0                <1>     sub     al, al ; 0
00005711 A2[42770100]        <1>     mov     byte [HF_INT_FLAG], al
                             <1> WT4:
                             <1> NB2:
00005716 A2[47770100]        <1>     mov     byte [DISK_STATUS1], al
                             <1>
                             <1>     ;cmp    byte [DISK_STATUS1], 0        ; SET CONDITION CODE FOR CALLER
0000571B 20C0                <1>     and     al, al
                             <1>     ; zf = 0 -> time out, zf = 1 -> ok
0000571D C3                  <1>     retn
                             <1>
                             <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 10/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   WAIT FOR CONTROLLER NOT BUSY  :
                             <1> ;----------------------------------------
                             <1> NOT_BUSY:
0000571E FB                  <1>     sti                             ; MAKE SURE INTERRUPTS ARE ON
                             <1>     ;push   ebx
                             <1>     ;sub    cx, cx                  ; SET INITIAL DELAY BEFORE TEST
0000571F 668B15[7E660000]    <1>     mov     dx, [HF_PORT]
00005726 80C207              <1>     add     dl, 7                   ; Status port (HF_PORT+7)
                             <1>     ;mov    bl, DELAY_1
                             <1>                                     ; wait for 10 seconds
                             <1>     ;mov    cx, WAIT_HDU_INT_LO     ; 1615h
                             <1>     ;;mov   bl, WAIT_HDU_INT_HI     ;   05h
                             <1>     ;mov    bl, WAIT_HDU_INT_HI + 1
00005729 B915160500          <1>     mov     ecx, WAIT_HDU_INT_LH    ; 21/02/2015
                             <1>     ;
                             <1>     ;;mov   byte [wait_count], 0    ; Reset wait counter
                             <1> NB1:
0000572E EC                  <1>     in      al, dx                  ; CHECK STATUS
                             <1>     ;test   al, ST_BUSY
0000572F 2480                <1>     and     al, ST_BUSY
                             <1>     ;loopnz NB1
00005731 74E3                <1>     jz      short NB2 ; al = 0      ; NOT BUSY--LETS GO
                             <1>     ;dec    bl
                             <1>     ;jnz    short NB1               ; KEEP TRYING FOR A WHILE
                             <1>
                             <1> NB1_hi:
00005733 E461                <1>     in      al, SYS1                ; wait for hi to lo
00005735 A810                <1>     test    al, 010h                ; transition on memory
00005737 75FA                <1>     jnz     short NB1_hi            ; refresh.
                             <1> NB1_lo:
00005739 E461                <1>     in      al, SYS1
0000573B A810                <1>     test    al, 010h
0000573D 74FA                <1>     jz      short NB1_lo
0000573F E2ED                <1>     loop    NB1
                             <1>     ;dec    bl
```

```
                            <1>     ;jnz    short NB1
                            <1>     ;
                            <1>     ;;cmp   byte [wait_count], 182  ; 10 seconds (182 timer ticks)
                            <1>     ;;jb    short NB1
                            <1>     ;
                            <1>     ;mov    byte [DISK_STATUS1], TIME_OUT ; REPORT TIME OUT ERROR
                            <1>     ;jmp    short NB3
00005741 B080               <1>     mov     al, TIME_OUT
                            <1> ;NB2:
00005743 EBD1               <1>     jmp     short NB2 ; 10/07/2022
                            <1>
                            <1> ;   ;mov    byte [DISK_STATUS1], 0
                            <1> ;;NB3:
                            <1> ;   ;pop    ebx
                            <1> ;   mov     [DISK_STATUS1], al    ;;; will be set after return
                            <1> ;   ;cmp    byte [DISK_STATUS1], 0       ; SET CONDITION CODE FOR CALLER
                            <1> ;   or      al, al                ; (zf = 0 --> timeout)
                            <1> ;   retn
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   WAIT FOR DATA REQUEST         :
                            <1> ;----------------------------------------
                            <1> WAIT_DRQ:
                            <1>     ;mov    cx, DELAY_3
                            <1>     ;mov    dx, HF_PORT+7
00005745 668B15[7E660000]   <1>     mov     dx, [HF_PORT]
0000574C 80C207             <1>     add     dl, 7
                            <1>     ;;mov   bl, WAIT_HDU_DRQ_HI   ; 0
                            <1>     ;mov    cx, WAIT_HDU_DRQ_LO   ; 1000 (30 milli seconds)
                            <1>                                   ; (but it is written as 2000
                            <1>                                   ; micro seconds in ATORGS.ASM file
                            <1>                                   ; of Award Bios - 1999, D1A0622)
0000574F B9E8030000         <1>     mov     ecx, WAIT_HDU_DRQ_LH ; 21/02/2015
                            <1> WQ_1:
00005754 EC                 <1>     in      al, dx                ; GET STATUS
00005755 A808               <1>     test    al, ST_DRQ            ; WAIT FOR DRQ
00005757 7516               <1>     jnz     short WQ_OK
                            <1>     ;loop   WQ_1                  ; KEEP TRYING FOR A SHORT WHILE
                            <1> WQ_hi:
00005759 E461               <1>     in      al, SYS1              ; wait for hi to lo
0000575B A810               <1>     test    al, 010h              ; transition on memory
0000575D 75FA               <1>     jnz     short WQ_hi           ; refresh.
                            <1> WQ_lo:
0000575F E461               <1>     in      al, SYS1
00005761 A810               <1>     test    al, 010h
00005763 74FA               <1>     jz      short WQ_lo
00005765 E2ED               <1>     loop    WQ_1
                            <1>
00005767 C605[47770100]80   <1>     mov     byte [DISK_STATUS1], TIME_OUT ; ERROR
0000576E F9                 <1>     stc
                            <1> WQ_OK:
0000576F C3                 <1>     retn
                            <1> ;WQ_OK:
                            <1>     ;clc
                            <1>     ;ret
                            <1>
                            <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   CHECK FIXED DISK STATUS BYTE  :
                            <1> ;----------------------------------------
                            <1> CHECK_ST:
                            <1>     ;mov    dx, HF_PORT+7         ; GET THE STATUS
00005770 668B15[7E660000]   <1>     mov     dx, [HF_PORT]
00005777 80C207             <1>     add     dl, 7
                            <1>
                            <1>     ; 17/02/2016
                            <1>     ;(http://wiki.osdev.org/ATA_PIO_Mode)
                            <1>     ;"delay 400ns to allow drive to set new values of BSY and DRQ"
0000577A EC                 <1>     in      al, dx
                            <1>     ;in     al, dx ; 100ns
                            <1>     ;in     al, dx ; 100ns
                            <1>     ;in     al, dx ; 100ns
                            <1>     NEWIODELAY ; 18/02/2016 (AWARD BIOS - 1999, 'CKST' in AHSDK.ASM)
  83 0000577B E6EB          <2>  out 0EBh, al
                            <1>     ;
                            <1>
                            <1>     ; 16/07/2022
0000577D A2[41770100]       <1>     mov     [HF_STATUS], al
                            <1>     ;mov    ah, 0
00005782 28E4               <1>     sub     ah, ah ; 0
00005784 A880               <1>     test    al, ST_BUSY           ; IF STILL BUSY
00005786 751A               <1>     jnz     short CKST_EXIT              ; REPORT OK
00005788 B4CC               <1>     mov     ah, WRITE_FAULT
0000578A A820               <1>     test    al, ST_WRT_FLT        ; CHECK FOR WRITE FAULT
0000578C 7514               <1>     jnz     short CKST_EXIT
0000578E B4AA               <1>     mov     ah, NOT_RDY
00005790 A840               <1>     test    al, ST_READY          ; CHECK FOR NOT READY
00005792 740E               <1>     jz      short CKST_EXIT
00005794 B440               <1>     mov     ah, BAD_SEEK
00005796 A810               <1>     test    al, ST_SEEK_COMPL     ; CHECK FOR SEEK NOT COMPLETE
00005798 7408               <1>     jz      short CKST_EXIT
0000579A B411               <1>     mov     ah, DATA_CORRECTED
0000579C A804               <1>     test    al, ST_CORRCTD        ; CHECK FOR CORRECTED ECC
0000579E 7502               <1>     jnz     short CKST_EXIT
                            <1>     ;mov    ah, 0
000057A0 30E4               <1>     xor     ah, ah ; 0
                            <1> CKST_EXIT:
000057A2 8825[47770100]     <1>     mov     [DISK_STATUS1], ah    ; SET ERROR FLAG
000057A8 80FC11             <1>     cmp     ah, DATA_CORRECTED    ; KEEP GOING WITH DATA CORRECTED
000057AB 7402               <1>     je      short CKST_EX1
                            <1>     ;cmp    ah, 0
000057AD 20E4               <1>     and     ah, ah
                            <1> CKST_EX1:
000057AF C3                 <1>     retn
                            <1>
                            <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   CHECK FIXED DISK ERROR REGISTER :
                            <1> ;----------------------------------------
                            <1> CHECK_ER:
                            <1>     ;mov    dx, HF_PORT+1         ; GET THE ERROR REGISTER
000057B0 668B15[7E660000]   <1>     mov     dx, [HF_PORT]         ;
000057B7 FEC2               <1>     inc     dl
000057B9 EC                 <1>     in      al, dx
                            <1>     ; 16/07/2022
                            <1>     ;mov    [HF_ERROR], al
                            <1>     ;push   ebx  ; 21/02/2015
000057BA 29C9               <1>     sub     ecx, ecx
                            <1>     ;mov    ecx, 8                ; TEST ALL 8 BITS
000057BC B108               <1>     mov     cl, 8
                            <1> CK1:
000057BE D0E0               <1>     shl     al, 1                 ; MOVE NEXT ERROR BIT TO CARRY
```

```
000057C0 7202                <1>     jc      short CK2               ; FOUND THE ERROR
000057C2 E2FA                <1>     loop    CK1                     ; KEEP TRYING
                             <1> CK2:
                             <1>     ;mov    ebx, ERR_TBL            ; COMPUTE ADDRESS OF
                             <1>     ;add    ebx, ecx                ; ERROR CODE
000057C4 81C1[74660000]      <1>     add     ecx, ERR_TBL ; 16/07/2022
                             <1>
                             <1>     ;;;mov  ah, byte [cs:bx]        ; GET ERROR CODE
                             <1>     ;;mov   ah, [bx]
                             <1>     ;mov    ah, [ebx] ; 21/02/2015
000057CA 8A21                <1>     mov     ah, [ecx]
                             <1> CKEX:
000057CC 8825[47770100]      <1>     mov     [DISK_STATUS1], ah      ; SAVE ERROR CODE
                             <1>     ; 16/07/2022
                             <1>     ;pop    ebx
                             <1>     ;;cmp   ah, 0
                             <1>     ;and    ah, ah
000057D2 C3                  <1>     retn
                             <1>
                             <1> ;-------------------------------------------------------
                             <1> ; CHECK_DMA                                   :
                             <1> ;  -CHECK ES:BX AND # SECTORS TO MAKE SURE THAT IT WILL :
                             <1> ;   FIT WITHOUT SEGMENT OVERFLOW.             :
                             <1> ;  -ES:BX HAS BEEN REVISED TO THE FORMAT SSSS:000X      :
                             <1> ;  -OK IF # SECTORS < 80H (7FH IF LONG READ OR WRITE)   :
                             <1> ;  -OK IF # SECTORS = 80H (7FH) AND BX <= 00H (04H)     :
                             <1> ;  -ERROR OTHERWISE                                     :
                             <1> ;-------------------------------------------------------
                             <1>
                             <1>     ; 16/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; (not needed for hard disks and 32 bit OS)
                             <1>
                             <1> ;CHECK_DMA:
                             <1> ;   ; 11/04/2021 (32 bit push/pop)
                             <1> ;   push    eax                     ; SAVE REGISTERS
                             <1> ;   mov     ax, 8000h               ; AH = MAX # SECTORS
                             <1> ;                                   ; AL = MAX OFFSET
                             <1> ;   test    byte [CMD_BLOCK+6], ECC_MODE
                             <1> ;   jz      short CKD1
                             <1> ;   mov     ah, 7F04h               ; ECC IS 4 MORE BYTES
                             <1> ;CKD1:
                             <1> ;   cmp     ah, [CMD_BLOCK+1]       ; NUMBER OF SECTORS
                             <1> ;   ja      short CKDOK             ; IT WILL FIT
                             <1> ;   jb      short CKDERR            ; TOO MANY
                             <1> ;   cmp     al, bl                  ; CHECK OFFSET ON MAX SECTORS
                             <1> ;   ;jb     short CKDERR            ; ERROR
                             <1> ;   jnb     short CKDOK ; 16/07/2022
                             <1> ;;CKDOK:
                             <1> ;   ;clc                            ; CLEAR CARRY
                             <1> ;   ;pop    eax
                             <1> ;   ;retn                           ; NORMAL RETURN
                             <1> ;CKDERR:
                             <1> ;   ;stc                            ; INDICATE ERROR
                             <1> ;   mov     byte [DISK_STATUS1], DMA_BOUNDARY
                             <1> ;CKDOK:
                             <1> ;   pop     eax
                             <1> ;   retn
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   SET UP EBX-> DISK PARMS               :
                             <1> ;----------------------------------------
                             <1>
                             <1> ; INPUT -> DL = 0 based drive number
                             <1> ; OUTPUT -> EBX = disk parameter table address
                             <1>
                             <1> GET_VEC:
                             <1>     ;sub    ax, ax                  ; GET DISK PARAMETER ADDRESS
                             <1>     ;mov    es, ax
                             <1>     ;test   dl, 1
                             <1>     ;jz     short GV_0
                             <1> ;   les     bx, [HF1_TBL_VEC]       ; ES:BX -> DRIVE PARAMETERS
                             <1> ;   jmp     short GV_EXIT
                             <1> ;GV_0:
                             <1> ;   les     bx,[HF_TBL_VEC]               ; ES:BX -> DRIVE PARAMETERS
                             <1> ;
000057D3 31DB                <1>     xor     ebx, ebx
000057D5 88D3                <1>     mov     bl, dl
                             <1>     ;02/01/2015
                             <1>     ;xor    bh, bh
                             <1>     ;shl    bl, 1                   ; port address offset
                             <1>     ;mov    ax, [bx+hd_ports]       ; Base port address (1F0h, 170h)
                             <1>     ;shl    bl, 1                   ; dpt pointer offset
000057D7 C0E302              <1>     shl     bl, 2
                             <1>     ;add    bx, HF_TBL_VEC          ; Disk parameter table pointer
000057DA 81C3[4C770100]      <1>     add     ebx, HF_TBL_VEC ; 21/02/2015
                             <1>     ;push   word [bx+2]             ; dpt segment
                             <1>     ;pop    es
                             <1>     ;mov    bx, [bx]                ; dpt offset
000057E0 8B1B                <1>     mov     ebx, [ebx]
                             <1> ;GV_EXIT:
000057E2 C3                  <1>     retn
                             <1>
                             <1> hdc1_int: ; 21/02/2015
                             <1> ;--- HARDWARE INT 76H -- ( IRQ LEVEL 14 ) -----------------------
                             <1> ;                                                               :
                             <1> ;   FIXED DISK INTERRUPT ROUTINE                                :
                             <1> ;                                                               :
                             <1> ;-------------------------------------------------------------
                             <1>
                             <1> ; 22/12/2014
                             <1> ; IBM PC-XT Model 286 System BIOS Source Code - DISK.ASM (HD_INT)
                             <1> ;    '11/15/85'
                             <1> ; AWARD BIOS 1999 (D1A0622)
                             <1> ;   Source Code - ATORGS.ASM (INT_HDISK, INT_HDISK1)
                             <1>
                             <1> ;int_76h:
                             <1> HD_INT:
                             <1>     ; 11/04/2021 (32 bit push/pop)
000057E3 50                  <1>     push    eax
000057E4 1E                  <1>     push    ds
                             <1>     ;call   DDS
                             <1>     ; 21/02/2015 (32 bit, 386 pm modification)
000057E5 66B81000            <1>     mov     ax, KDATA
000057E9 8ED8                <1>     mov     ds, ax
                             <1>     ;
                             <1>     ;;mov   @HF_INT_FLAG,0FFH       ; ALL DONE
                             <1>         ;mov        byte [CS:HF_INT_FLAG], 0FFh
000057EB C605[42770100]FF    <1>     mov     byte [HF_INT_FLAG], 0FFh
                             <1>     ;
000057F2 52                  <1>     push    edx
000057F3 66BAF701            <1>     mov     dx, HDC1_BASEPORT+7     ; Status Register (1F7h)
                             <1>                                     ; Clear Controller
                             <1> Clear_IRQ1415:                                ; (Award BIOS - 1999)
000057F7 EC                  <1>     in      al, dx                  ;
```

```
000057F8 5A                 <1>     pop     edx
                            <1>     NEWIODELAY
  83 000057F9 E6EB          <2>  out 0EBh, al
                            <1>     ;
000057FB B020               <1>     mov     al, EOI             ; NON-SPECIFIC END OF INTERRUPT
000057FD E6A0               <1>     out     INTB00, al          ; FOR CONTROLLER #2
                            <1>     ;jmp    $+2                 ; WAIT
                            <1>     NEWIODELAY
  83 000057FF E6EB          <2>  out 0EBh, al
00005801 E620               <1>     out     INTA00, al          ; FOR CONTROLLER #1
00005803 1F                 <1>     pop     ds
                            <1>     ;sti                        ; RE-ENABLE INTERRUPTS
                            <1>     ;mov    ax, 9100h           ; DEVICE POST
                            <1>     ;int    15h                 ;  INTERRUPT
                            <1> irq15_iret: ; 25/02/2015
00005804 58                 <1>     pop     eax
00005805 CF                 <1>     iretd                       ; RETURN FROM INTERRUPT
                            <1>
                            <1> hdc2_int: ; 21/02/2015
                            <1> ;--- HARDWARE INT 77H -- ( IRQ LEVEL 15 ) -----------------------
                            <1> ;                                                               :
                            <1> ;   FIXED DISK INTERRUPT ROUTINE                                :
                            <1> ;                                                               :
                            <1> ;-------------------------------------------------------------
                            <1>
                            <1> ;int_77h:
                            <1> HD1_INT:
                            <1>     ; 11/04/2021 (32 bit push/pop)
00005806 50                 <1>     push    eax
                            <1>     ; Check if that is a spurious IRQ (from slave PIC)
                            <1>     ; 25/02/2015 (source: http://wiki.osdev.org/8259_PIC)
00005807 B00B               <1>     mov     al, 0Bh  ; In-Service Register
00005809 E6A0               <1>     out     0A0h, al
0000580B EB00               <1>         jmp short $+2
0000580D EB00               <1>     jmp short $+2
0000580F E4A0               <1>     in      al, 0A0h
00005811 2480               <1>     and     al, 80h ; bit 7 (is it real IRQ 15 or fake?)
00005813 74EF               <1>     jz      short irq15_iret ; Fake (spurious) IRQ, do not send EOI)
                            <1>     ;
00005815 1E                 <1>     push    ds
                            <1>     ;call   DDS
                            <1>     ; 21/02/2015 (32 bit, 386 pm modification)
00005816 66B81000           <1>     mov     ax, KDATA
0000581A 8ED8               <1>     mov     ds, ax
                            <1>     ;
                            <1>     ;;mov   @HF_INT_FLAG,0FFH      ; ALL DONE
                            <1>         ;or byte [CS:HF_INT_FLAG],0C0h
0000581C 800D[42770100]C0   <1>     or      byte [HF_INT_FLAG], 0C0h
                            <1>     ;
00005823 52                 <1>     push    edx
00005824 66BA7701           <1>     mov     dx, HDC2_BASEPORT+7   ; Status Register (177h)
                            <1>                                   ; Clear Controller (Award BIOS 1999)
00005828 EBCD               <1>     jmp     short Clear_IRQ1415
                            <1>
                            <1> ;%include 'diskdata.inc' ; 11/03/2015
                            <1> ;%include 'diskbss.inc' ; 11/03/2015
                            <1>
                            <1> ;///////////////////////////////////////////////////////////////
                            <1> ;; END OF DISK I/O SYTEM ///
                                %include 'memory.s'  ; 09/03/2015
                            <1> ; ***************************************************************************
                            <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.8 - memory.s
                            <1> ; ---------------------------------------------------------------------------
                            <1> ; Last Update: 09/02/2026  (Previous: 04/06/2024 - Kernel v2.0.8)
                            <1> ; ---------------------------------------------------------------------------
                            <1> ; Beginning: 24/01/2016
                            <1> ; ---------------------------------------------------------------------------
                            <1> ; Assembler: NASM version 2.15 (trdos386.s)
                            <1> ; ---------------------------------------------------------------------------
                            <1> ; Turkish Rational DOS
                            <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                            <1> ;
                            <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                            <1> ; memory.inc (18/10/2015)
                            <1> ; ***************************************************************************
                            <1>
                            <1> ; MEMORY.ASM - Retro UNIX 386 v1 MEMORY MANAGEMENT FUNCTIONS (PROCEDURES)
                            <1> ; Retro UNIX 386 v1 Kernel (unix386.s, v0.2.0.14) - MEMORY.INC
                            <1> ; Last Modification: 18/10/2015
                            <1>
                            <1> ; ///////// MEMORY MANAGEMENT FUNCTIONS (PROCEDURES) ///////////////
                            <1>
                            <1> ;;04/11/2014 (unix386.s)
                            <1> ;PDE_A_PRESENT     equ     1               ; Present flag for PDE
                            <1> ;PDE_A_WRITE       equ     2               ; Writable (write permission) flag
                            <1> ;PDE_A_USER equ     4               ; User (non-system/kernel) page flag
                            <1> ;;
                            <1> ;PTE_A_PRESENT     equ     1               ; Present flag for PTE (bit 0)
                            <1> ;PTE_A_WRITE       equ     2               ; Writable (write permission) flag (bit 1)
                            <1> ;PTE_A_USER equ     4               ; User (non-system/kernel) page flag (bit 2)
                            <1> ;PTE_A_ACCESS   equ        32              ; Accessed flag (bit 5) ; 09/03/2015
                            <1>
                            <1> ; 27/04/2015
                            <1> ; 09/03/2015
                            <1> PAGE_SIZE   equ 4096                ; page size in bytes
                            <1> PAGE_SHIFT  equ 12                  ; page table shift count
                            <1> PAGE_D_SHIFT       equ 22  ; 12 + 10       ; page directory shift count
                            <1> PAGE_OFF    equ 0FFFh               ; 12 bit byte offset in page frame
                            <1> PTE_MASK    equ 03FFh               ; page table entry mask
                            <1> PTE_DUPLICATED  equ 200h            ; duplicated page sign (AVL bit 0)
                            <1> PDE_A_CLEAR equ 0F000h              ; to clear PDE attribute bits
                            <1> PTE_A_CLEAR equ 0F000h              ; to clear PTE attribute bits
                            <1> LOGIC_SECT_SIZE equ 512                     ; logical sector size
                            <1> ERR_MAJOR_PF       equ 0E0h                 ; major error: page fault
                            <1> ERR_MINOR_IM       equ 4 ;15/10/2016 (1->4); insufficient (out of) memory
                            <1> ERR_MINOR_RO       equ 5 ;09/02/2026 ; access error (write attempt on read only page)
                            <1> ERR_MINOR_PV       equ 6 ;15/10/2016 (1->4); protection violation
                            <1> SWP_DISK_READ_ERR     equ 40
                            <1> SWP_DISK_NOT_PRESENT_ERR   equ 41
                            <1> SWP_SECTOR_NOT_PRESENT_ERR equ 42
                            <1> SWP_NO_FREE_SPACE_ERR      equ 43
                            <1> SWP_DISK_WRITE_ERR         equ 44
                            <1> SWP_NO_PAGE_TO_SWAP_ERR    equ 45
                            <1> PTE_A_ACCESS_BIT equ 5  ; Bit 5 (accessed flag)
                            <1> SECTOR_SHIFT     equ 3  ; sector shift (to convert page block number)
                            <1> ; 12/07/2016
                            <1> PTE_SHARED   equ 400h               ; AVL bit 1, direct memory access bit
                            <1>                                     ; (Indicates that the page is not allocated
                            <1>                                     ; for the process, it is a shared or system
                            <1>                                          ; page, it must not be deallocated!)
                            <1> ; 14/12/2020
                            <1> ; (Linear Frame Buffer - video memory mark : AVL bit 1, outside M.A.T.)
                            <1> PDE_EXTERNAL       equ 400h        ; Page directory entry for external memory blocks
                            <1> PTE_EXTERNAL       equ 400h        ; Allocated kernel pages for Linear Frame Buffer
```

```
<1>                             ; (Out of memory allocation table)
<1>
<1> ;
<1> ;; Retro Unix 386 v1 - paging method/principles
<1> ;;
<1> ;; 10/10/2014
<1> ;; RETRO UNIX 386 v1 - PAGING METHOD/PRINCIPLES
<1> ;;
<1> ;; KERNEL PAGE MAP: 1 to 1 physical memory page map
<1> ;;  (virtual address = physical address)
<1> ;; KERNEL PAGE TABLES:
<1> ;;  Kernel page directory and all page tables are
<1> ;;  on memory as initialized, as equal to physical memory
<1> ;;  layout. Kernel pages can/must not be swapped out/in.
<1> ;;
<1> ;;  what for: User pages may be swapped out, when accessing
<1> ;;  a page in kernel/system mode, if it would be swapped out,
<1> ;;  kernel would have to swap it in! But it is also may be
<1> ;;  in use by a user process. (In system/kernel mode
<1> ;;  kernel can access all memory pages even if they are
<1> ;;  reserved/allocated for user processes. Swap out/in would
<1> ;;  cause conflicts.)
<1> ;;
<1> ;;  As result of these conditions,
<1> ;;  all kernel pages must be initialized as equal to
<1> ;;  physical layout for preventing page faults.
<1> ;;  Also, calling "allocate page" procedure after
<1> ;;  a page fault can cause another page fault (double fault)
<1> ;;  if all kernel page tables would not be initialized.
<1> ;;
<1> ;;  [first_page] = Beginning of users space, as offset to
<1> ;;  memory allocation table. (double word aligned)
<1> ;;
<1> ;;  [next_page] = first/next free space to be searched
<1> ;;  as offset to memory allocation table. (dw aligned)
<1> ;;
<1> ;;  [last_page] = End of memory (users space), as offset
<1> ;;  to memory allocation table. (double word aligned)
<1> ;;
<1> ;; USER PAGE TABLES:
<1> ;;  Demand paging (& 'copy on write' allocation method) ...
<1> ;;          'ready only' marked copies of the
<1> ;;          parent process's page table entries (for
<1> ;;          same physical memory).
<1> ;;          (A page will be copied to a new page after
<1> ;;           if it causes R/W page fault.)
<1> ;;
<1> ;;  Every user process has own (different)
<1> ;;  page directory and page tables.
<1> ;;
<1> ;;  Code starts at virtual address 0, always.
<1> ;;  (Initial value of EIP is 0 in user mode.)
<1> ;;  (Programs can be written/developed as simple
<1> ;;   flat memory programs.)
<1> ;;
<1> ;; MEMORY ALLOCATION STRATEGY:
<1> ;;  Memory page will be allocated by kernel only
<1> ;;          (in kernel/system mode only).
<1> ;;  * After a
<1> ;;    - 'not present' page fault
<1> ;;    - 'writing attempt on read only page' page fault
<1> ;;  * For loading (opening, reading) a file or disk/drive
<1> ;;  * As responce to 'allocate additional memory blocks'
<1> ;;    request by running process.
<1> ;;  * While creating a process, allocating a new buffer,
<1> ;;    new page tables etc.
<1> ;;
<1> ;;  At first,
<1> ;;  - 'allocate page' procedure will be called;
<1> ;,     if it will return with a valid (>0) physical address
<1> ;;     (that means the relevant M.A.T. bit has been RESET)
<1> ;;     relevant memory page/block will be cleared (zeroed).
<1> ;;  - 'allocate page' will be called for allocating page
<1> ;;     directory, page table and running space (data/code).
<1> ;;  - every successful 'allocate page' call will decrease
<1> ;;    'free_pages' count (pointer).
<1> ;;  - 'out of (insufficient) memory error' will be returned
<1> ;;    if 'free_pages' points to a ZERO.
<1> ;;  - swapping out and swapping in (if it is not a new page)
<1> ;;    procedures will be called as responce to 'out of memory'
<1> ;;    error except errors caused by attribute conflicts.
<1> ;;   (swapper functions)
<1> ;;
<1> ;;  At second,
<1> ;;  - page directory entry will be updated then page table
<1> ;;    entry will be updated.
<1> ;;
<1> ;; MEMORY ALLOCATION TABLE FORMAT:
<1> ;;  - M.A.T. has a size according to available memory as
<1> ;;    follows:
<1> ;;            - 1 (allocation) bit per 1 page (4096 bytes)
<1> ;;            - a bit with value of 0 means allocated page
<1> ;;            - a bit with value of 1 means a free page
<1> ;,  - 'free_pages' pointer holds count of free pages
<1> ;;    depending on M.A.T.
<1> ;;          (NOTE: Free page count will not be checked
<1> ;;          again -on M.A.T.- after initialization.
<1> ;;          Kernel will trust on initial count.)
<1> ;,  - 'free_pages' count will be decreased by allocation
<1> ;;    and it will be increased by deallocation procedures.
<1> ;;
<1> ;;  - Available memory will be calculated during
<1> ;;    the kernel's initialization stage (in real mode).
<1> ;;    Memory allocation table and kernel page tables
<1> ;;    will be formatted/sized as result of available
<1> ;;    memory calculation before paging is enabled.
<1> ;;
<1> ;; For 4GB Available/Present Memory: (max. possible memory size)
<1> ;;  - Memory Allocation Table size will be 128 KB.
<1> ;;  - Memory allocation for kernel page directory size
<1> ;;    is always 4 KB. (in addition to total allocation size
<1> ;;    for page tables)
<1> ;;  - Memory allocation for kernel page tables (1024 tables)
<1> ;;    is 4 MB (1024*4*1024 bytes).
<1> ;;  - User (available) space will be started
<1> ;;    at 6th MB of the memory (after 1MB+4MB).
<1> ;;  - The first 640 KB is for kernel's itself plus
<1> ;;    memory allocation table and kernel's page directory
<1> ;;    (D0000h-EFFFFh may be used as kernel space...)
<1> ;;  - B0000h to B7FFFh address space (32 KB) will be used
<1> ;;    for buffers.
<1> ;;  - ROMBIOS, VIDEO BUFFER and VIDEO ROM space are reserved.
<1> ;,    (A0000h-AFFFFh, C0000h-CFFFFh, F0000h-FFFFFh)
<1> ;;  - Kernel page tables start at 100000h (2nd MB)
```

```
<1> ;;
<1> ;; For 1GB Available Memory:
<1> ;;  - Memory Allocation Table size will be 32 KB.
<1> ;;  - Memory allocation for kernel page directory size
<1> ;;    is always 4 KB. (in addition to total allocation size
<1> ;;    for page tables)
<1> ;;  - Memory allocation for kernel page tables (256 tables)
<1> ;;    is 1 MB (256*4*1024 bytes).
<1> ;;  - User (available) space will be started
<1> ;;    at 3th MB of the memory (after 1MB+1MB).
<1> ;;  - The first 640 KB is for kernel's itself plus
<1> ;;    memory allocation table and kernel's page directory
<1> ;;    (D0000h-EFFFFh may be used as kernel space...)
<1> ;;  - B0000h to B7FFFh address space (32 KB) will be used
<1> ;;    for buffers.
<1> ;;  - ROMBIOS, VIDEO BUFFER and VIDEO ROM space are reserved.
<1> ;,    (A0000h-AFFFFh, C0000h-CFFFFh, F0000h-FFFFFh)
<1> ;;  - Kernel page tables start at 100000h (2nd MB).
<1> ;;
<1> ;;
<1>
<1>
<1> ;;*******************************************************************************
<1> ;;
<1> ;; RETRO UNIX 386 v1 - Paging (Method for Copy On Write paging principle)
<1> ;; DEMAND PAGING - PARENT&CHILD PAGE TABLE DUPLICATION PRINCIPLES (23/04/2015)
<1>
<1> ;; Main factor: "sys fork" system call
<1> ;;
<1> ;;         FORK
<1> ;;                        |----> parent - duplicated PTEs, read only pages
<1> ;;  writable pages ---->|
<1> ;;                        |----> child - duplicated PTEs, read only pages
<1> ;;
<1> ;; AVL bit (0) of Page Table Entry is used as duplication sign
<1> ;;
<1> ;; AVL Bit 0 [PTE Bit 9] = 'Duplicated PTE belongs to child' sign/flag (if it is set)
<1> ;; Note: Dirty bit (PTE bit 6) may be used instead of AVL bit 0 (PTE bit 9)
<1> ;;       -while R/W bit is 0-.
<1> ;;
<1> ;; Duplicate page tables with writable pages (the 1st sys fork in the process):
<1> ;; # Parent's Page Table Entries are updated to point same pages as read only,
<1> ;;   as duplicated PTE bit  -AVL bit 0, PTE bit 9- are reset/clear.
<1> ;; # Then Parent's Page Table is copied to Child's Page Table.
<1> ;; # Child's Page Table Entries are updated as duplicated child bit
<1> ;;   -AVL bit 0, PTE bit 9- is set.
<1> ;;
<1> ;; Duplicate page tables with read only pages (several sys fork system calls):
<1> ;; # Parent's read only pages are copied to new child pages.
<1> ;;   Parent's PTE attributes are not changed.
<1> ;;   (Because, there is another parent-child fork before this fork! We must not
<1> ;;    destroy/mix previous fork result).
<1> ;; # Child's Page Table Entries (which are corresponding to Parent's
<1> ;;   read only pages) are set as writable (while duplicated PTE bit is clear).
<1> ;; # Parent's PTEs with writable page attribute are updated to point same pages
<1> ;;   as read only, (while) duplicated PTE bit is reset (clear).
<1> ;; # Parent's Page Table Entries (with writable page attribute) are duplicated
<1> ;;   as Child's Page Table Entries without copying actual page.
<1> ;; # Child 's Page Table Entries (which are corresponding to Parent's writable
<1> ;;   pages) are updated as duplicated PTE bit (AVL bit 0, PTE bit 9- is set.
<1> ;;
<1> ;; !? WHAT FOR (duplication after duplication):
<1> ;; In UNIX method for sys fork (a typical 'fork' application in /etc/init)
<1> ;; program/executable code continues from specified location as child process,
<1> ;; returns back previous code location as parent process, every child after
<1> ;; every sys fork uses last image of code and data just prior the fork.
<1> ;; Even if the parent code changes data, the child will not see the changed data
<1> ;; after the fork. In Retro UNIX 8086 v1, parent's process segment (32KB)
<1> ;; was copied to child's process segment (all of code and data) according to
<1> ;; original UNIX v1 which copies all of parent process code and data -core-
<1> ;; to child space -core- but swaps that core image -of child- on to disk.
<1> ;; If I (Erdogan Tan) would use a method of to copy parent's core
<1> ;; (complete running image of parent process) to the child process;
<1> ;; for big sizes, i would force Retro UNIX 386 v1 to spend many memory pages
<1> ;; and times only for a sys fork. (It would excessive reservation for sys fork,
<1> ;; because sys fork usually is prior to sys exec; sys exec always establishes
<1> ;; a new/fresh core -running space-, by clearing all code/data content).
<1> ;; 'Read Only' page flag ensures page fault handler is needed only for a few write
<1> ;; attempts between sys fork and sys exec, not more... (I say so by thinking
<1> ;; of "/etc/init" content, specially.) sys exec will clear page tables and
<1> ;; new/fresh pages will be used to load and run new executable/program.
<1> ;; That is what for i have preferred "copy on write", "duplication" method
<1> ;; for sharing same read only pages between parent and child processes.
<1> ;; That is a pitty i have to use new private flag (AVL bit 0, "duplicated PTE
<1> ;; belongs to child" sign) for cooperation on duplicated pages between a parent
<1> ;; and it's child processes; otherwise parent process would destroy data belongs
<1> ;; to its child or vice versa; or some pages would remain unclaimed
<1> ;; -deallocation problem-.
<1> ;; Note: to prevent conflicts, read only pages must not be swapped out...
<1> ;;
<1> ;; WHEN PARENT TRIES TO WRITE IT'S READ ONLY (DUPLICATED) PAGE:
<1> ;; # Page fault handler will do those:
<1> ;;   - 'Duplicated PTE' flag (PTE bit 9) is checked (on the failed PTE).
<1> ;;   - If it is reset/clear, there is a child uses same page.
<1> ;;   - Parent's read only page -previous page- is copied to a new writable page.
<1> ;;   - Parent's PTE is updated as writable page, as unique page (AVL=0)
<1> ;;   - (Page fault handler whill check this PTE later, if child process causes to
<1> ;;     page fault due to write attempt on read only page. Of course, the previous
<1> ;;     read only page will be converted to writable and unique page which belongs
<1> ;;     to child process.)
<1> ;; WHEN CHILD TRIES TO WRITE IT'S READ ONLY (DUPLICATED) PAGE:
<1> ;; # Page fault handler will do those:
<1> ;;   - 'Duplicated PTE' flag (PTE bit 9) is checked (on the failed PTE).
<1> ;;   - If it is set, there is a parent uses -or was using- same page.
<1> ;;   - Same PTE address within parent's page table is checked if it has same page
<1> ;;     address or not.
<1> ;;   - If parent's PTE has same address, child will continue with a new writable page.
<1> ;;     Parent's PTE will point to same (previous) page as writable, unique (AVL=0).
<1> ;;   - If parent's PTE has different address, child will continue with it's
<1> ;;     own/same page but read only flag (0) will be changed to writable flag (1) and
<1> ;;     'duplicated PTE (belongs to child)' flag/sign will be cleared/reset.
<1> ;;
<1> ;; NOTE: When a child process is terminated, read only flags of parent's page tables
<1> ;;       will be set as writable (and unique) in case of child process was using
<1> ;;       same pages with duplicated child PTE sign... Depending on sys fork and
<1> ;;       duplication method details, it is not possible multiple child processes
<1> ;;       were using same page with duplicated PTEs.
<1> ;;
<1> ;;*******************************************************************************
<1>
<1> ;; 08/10/2014
<1> ;; 11/09/2014 - Retro UNIX 386 v1 PAGING (further) draft
<1> ;;         by Erdogan Tan (Based on KolibriOS 'memory.inc')
<1>
```

```
                                <1> ;; 'allocate_page' code is derived and modified from KolibriOS
                                <1> ;; 'alloc_page' procedure in 'memory.inc'
                                <1> ;; (25/08/2014, Revision: 5057) file
                                <1> ;; by KolibriOS Team (2004-2012)
                                <1>
                                <1> allocate_page:
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;       (temporary modifications)
                                <1>     ; 01/07/2015
                                <1>     ; 05/05/2015
                                <1>     ; 30/04/2015
                                <1>     ; 16/10/2014
                                <1>     ; 08/10/2014
                                <1>     ; 09/09/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT -> none
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      EAX = PHYSICAL (real/flat) ADDRESS OF THE ALLOCATED PAGE
                                <1>     ;      (corresponding MEMORY ALLOCATION TABLE bit is RESET)
                                <1>     ;
                                <1>     ;      CF = 1 and EAX = 0
                                <1>     ;                if there is not a free page to be allocated
                                <1>     ;
                                <1>     ; Modified Registers -> none (except EAX)
                                <1>     ;
0000582A A1[B8760100]           <1>     mov    eax, [free_pages]
0000582F 21C0                   <1>     and    eax, eax
00005831 7438                   <1>     jz     short out_of_memory
                                <1>     ;
00005833 53                     <1>     push   ebx
00005834 51                     <1>     push   ecx
                                <1>     ;
00005835 BB00001000             <1>     mov    ebx, MEM_ALLOC_TBL   ; Memory Allocation Table offset
0000583A 89D9                   <1>     mov    ecx, ebx
                                <1>                                 ; NOTE: 32 (first_page) is initial
                                <1>                                 ; value of [next_page].
                                <1>                                 ; It points to the first available
                                <1>                                 ; page block for users (ring 3) ...
                                <1>                                 ; (MAT offset 32 = 1024/32)
                                <1>                                 ; (at the of the first 4 MB)
0000583C 031D[BC760100]         <1>     add    ebx, [next_page] ; Free page searching starts from here
                                <1>                             ; next_free_page >> 5
00005842 030D[C0760100]         <1>     add    ecx, [last_page] ; Free page searching ends here
                                <1>                             ; (total_pages - 1) >> 5
                                <1> al_p_scan:
00005848 39CB                   <1>     cmp    ebx, ecx
0000584A 770A                   <1>     ja     short al_p_notfound
                                <1>     ;
                                <1>     ; 01/07/2015
                                <1>     ; AMD64 Architecture Programmer's Manual
                                <1>     ; Volume 3:
                                <1>     ; General-Purpose and System Instructions
                                <1>     ;
                                <1>     ; BSF - Bit Scan Forward
                                <1>     ;
                                <1>     ;   Searches the value in a register or a memory location
                                <1>     ;   (second operand) for the least-significant set bit.
                                <1>     ;   If a set bit is found, the instruction clears the zero flag (ZF)
                                <1>     ;   and stores the index of the least-significant set bit in a destination
                                <1>     ;   register (first operand). If the second operand contains 0,
                                <1>     ;   the instruction sets ZF to 1 and does not change the contents of the
                                <1>     ;   destination register. The bit index is an unsigned offset from bit 0
                                <1>     ;   of the searched value
                                <1>     ;
0000584C 0FBC03                 <1>     bsf    eax, [ebx] ; Scans source operand for first bit set (1).
                                <1>                       ; Clear ZF if a bit is found set (1) and
                                <1>                       ; loads the destination with an index to
                                <1>                       ; first set bit. (0 -> 31)
                                <1>                       ; Sets ZF to 1 if no bits are found set.
0000584F 751C                   <1>     jnz    short al_p_found ; ZF = 0 -> a free page has been found
                                <1>                     ;
                                <1>                     ; NOTE: a Memory Allocation Table bit
                                <1>                     ;      with value of 1 means
                                <1>                     ;      the corresponding page is free
                                <1>                     ;      (Retro UNIX 386 v1 feature only!)
00005851 83C304                 <1>     add    ebx, 4
                                <1>                     ; We return back for searching next page block
                                <1>                     ; NOTE: [free_pages] is not ZERO; so,
                                <1>                     ;      we always will find at least 1 free page here.
00005854 EBF2                   <1>         jmp     short al_p_scan
                                <1>     ;
                                <1> al_p_notfound:
00005856 81E900001000           <1>     sub    ecx, MEM_ALLOC_TBL
0000585C 890D[BC760100]         <1>     mov    [next_page], ecx ; next/first free page = last page
                                <1>                             ; (deallocate_page procedure will change it)
00005862 31C0                   <1>     xor    eax, eax
00005864 A3[B8760100]           <1>     mov    [free_pages], eax ; 0
00005869 59                     <1>     pop    ecx
0000586A 5B                     <1>     pop    ebx
                                <1>     ;
                                <1> ; 17/04/2021
                                <1> ; ('swap_out' procedure call is disabled as temporary)
                                <1>
                                <1> out_of_memory:
                                <1> ;   call   swap_out
                                <1> ;   jnc    short al_p_ok  ; [free_pages] = 0, re-allocation by swap_out
                                <1> ;   ;
                                <1> ;   sub    eax, eax ; 0
0000586B F9                     <1>     stc
0000586C C3                     <1>     retn
                                <1>
                                <1> al_p_found:
0000586D 89D9                   <1>     mov    ecx, ebx
0000586F 81E900001000           <1>     sub    ecx, MEM_ALLOC_TBL
00005875 890D[BC760100]         <1>     mov    [next_page], ecx ; Set first free page searching start
                                <1>                             ; address/offset (to the next)
0000587B FF0D[B8760100]         <1>         dec     dword [free_pages] ; 1 page has been allocated (X = X-1)
                                <1>     ;
00005881 0FB303                 <1>     btr    [ebx], eax      ; The destination bit indexed by the source value
                                <1>                            ; is copied into the Carry Flag and then cleared
                                <1>                            ; in the destination.
                                <1>                            ;
                                <1>                            ; Reset the bit which is corresponding to the
                                <1>                            ; (just) allocated page.
                                <1>     ; 01/07/2015 (4*8 = 32, 1 allocation byte = 8 pages)
00005884 C1E103                 <1>     shl    ecx, 3          ; (page block offset * 32) + page index
00005887 01C8                   <1>     add    eax, ecx        ; = page number
00005889 C1E00C                 <1>     shl    eax, 12         ; physical address of the page (flat/real value)
                                <1>     ; EAX = physical address of memory page
                                <1>     ;
                                <1>     ; NOTE: The relevant page directory and page table entry will be updated
                                <1>     ;       according to this EAX value...
0000588C 59                     <1>     pop    ecx
```

```
0000588D 5B                 <1>     pop     ebx
                            <1> al_p_ok:
0000588E C3                 <1>     retn
                            <1>
                            <1> make_page_dir:
                            <1>     ; 18/04/2015
                            <1>     ; 12/04/2015
                            <1>     ; 23/10/2014
                            <1>     ; 16/10/2014
                            <1>     ; 09/10/2014 ; (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       none
                            <1>     ; OUTPUT ->
                            <1>     ;       (EAX = 0)
                            <1>     ;       cf = 1 -> insufficient (out of) memory error
                            <1>     ;       cf = 0 ->
                            <1>     ;       u.pgdir = page directory (physical) address of the current
                            <1>     ;                 process/user.
                            <1>     ;
                            <1>     ; Modified Registers -> EAX
                            <1>     ;
0000588F E896FFFFFF         <1>     call    allocate_page
00005894 7216               <1>     jc      short mkpd_error
                            <1>     ;
00005896 A3[C48E0100]       <1>     mov     [u.pgdir], eax    ; Page dir address for current user/process
                            <1>                               ; (Physical address)
                            <1> clear_page:
                            <1>     ; 18/04/2015
                            <1>     ; 09/10/2014 ; (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       EAX = physical address of the page
                            <1>     ; OUTPUT ->
                            <1>     ;       all bytes of the page will be cleared
                            <1>     ;
                            <1>     ; Modified Registers -> none
                            <1>     ;
0000589B 57                 <1>     push    edi
0000589C 51                 <1>     push    ecx
0000589D 50                 <1>     push    eax
0000589E B900040000         <1>     mov     ecx, PAGE_SIZE / 4
000058A3 89C7               <1>     mov     edi, eax
000058A5 31C0               <1>     xor     eax, eax
000058A7 F3AB               <1>     rep     stosd
000058A9 58                 <1>     pop     eax
000058AA 59                 <1>     pop     ecx
000058AB 5F                 <1>     pop     edi
                            <1> mkpd_error:
                            <1> mkpt_error:
000058AC C3                 <1>     retn
                            <1>
                            <1> make_page_table:
                            <1>     ; 23/06/2015
                            <1>     ; 18/04/2015
                            <1>     ; 12/04/2015
                            <1>     ; 16/10/2014
                            <1>     ; 09/10/2014 ; (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       EBX = virtual (linear) address
                            <1>     ;       ECX = page table attributes (lower 12 bits)
                            <1>     ;             (higher 20 bits must be ZERO)
                            <1>     ;             (bit 0 must be 1)
                            <1>     ;       u.pgdir = page directory (physical) address
                            <1>     ; OUTPUT ->
                            <1>     ;       EDX = Page directory entry address
                            <1>     ;       EAX = Page table address
                            <1>     ;       cf = 1 -> insufficient (out of) memory error
                            <1>     ;       cf = 0 -> page table address in the PDE (EDX)
                            <1>     ;
                            <1>     ; Modified Registers -> EAX, EDX
                            <1>     ;
000058AD E878FFFFFF         <1>     call    allocate_page
000058B2 72F8               <1>     jc      short mkpt_error
000058B4 E811000000         <1>     call    set_pde
000058B9 EBE0               <1>     jmp     short clear_page
                            <1>
                            <1> make_page:
                            <1>     ; 24/07/2015
                            <1>     ; 23/06/2015 ; (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       EBX = virtual (linear) address
                            <1>     ;       ECX = page attributes (lower 12 bits)
                            <1>     ;             (higher 20 bits must be ZERO)
                            <1>     ;             (bit 0 must be 1)
                            <1>     ;       u.pgdir = page directory (physical) address
                            <1>     ; OUTPUT ->
                            <1>     ;       EBX = Virtual address
                            <1>     ;       (EDX = PTE value)
                            <1>     ;       EAX = Physical address
                            <1>     ;       cf = 1 -> insufficient (out of) memory error
                            <1>     ;
                            <1>     ; Modified Registers -> EAX, EDX
                            <1>     ;
000058BB E86AFFFFFF         <1>     call    allocate_page
000058C0 7207               <1>     jc      short mkp_err
000058C2 E821000000         <1>     call    set_pte
000058C7 73D2               <1>     jnc     short clear_page ; 18/04/2015
                            <1> mkp_err:
000058C9 C3                 <1>     retn
                            <1>
                            <1>
                            <1> set_pde:    ; Set page directory entry (PDE)
                            <1>     ; 20/07/2015
                            <1>     ; 18/04/2015
                            <1>     ; 12/04/2015
                            <1>     ; 23/10/2014
                            <1>     ; 10/10/2014 ; (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       EAX = physical address
                            <1>     ;             (use present value if EAX = 0)
                            <1>     ;       EBX = virtual (linear) address
                            <1>     ;       ECX = page table attributes (lower 12 bits)
                            <1>     ;             (higher 20 bits must be ZERO)
                            <1>     ;             (bit 0 must be 1)
                            <1>     ;       u.pgdir = page directory (physical) address
                            <1>     ; OUTPUT ->
                            <1>     ;       EDX = PDE address
                            <1>     ;       EAX = page table address (physical)
                            <1>     ;       ;(CF=1 -> Invalid page address)
                            <1>     ;
```

```
                                <1>     ; Modified Registers -> EDX
                                <1>     ;
000058CA 89DA                   <1>     mov     edx, ebx
000058CC C1EA16                 <1>     shr     edx, PAGE_D_SHIFT ; 22
000058CF C1E202                 <1>     shl     edx, 2 ; offset to page directory (1024*4)
000058D2 0315[C48E0100]         <1>     add     edx, [u.pgdir]
                                <1>     ;
000058D8 21C0                   <1>     and     eax, eax
000058DA 7506                   <1>     jnz     short spde_1
                                <1>     ;
000058DC 8B02                   <1>     mov     eax, [edx]  ; old PDE value
                                <1>     ;test   al, 1
                                <1>     ;jz     short spde_2
000058DE 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h  ; clear lower 12 bits
                                <1> spde_1:
                                <1>     ;and    cx, 0FFFh
000058E2 8902                   <1>     mov     [edx], eax
000058E4 66090A                 <1>     or      [edx], cx
000058E7 C3                     <1>     retn
                                <1> ;spde_2: ; error
                                <1> ;   stc
                                <1> ;   retn
                                <1>
                                <1> set_pte:    ; Set page table entry (PTE)
                                <1>     ; 24/07/2015
                                <1>     ; 20/07/2015
                                <1>     ; 23/06/2015
                                <1>     ; 18/04/2015
                                <1>     ; 12/04/2015
                                <1>     ; 10/10/2014 ; (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = physical page address
                                <1>     ;             (use present value if EAX = 0)
                                <1>     ;       EBX = virtual (linear) address
                                <1>     ;       ECX = page attributes (lower 12 bits)
                                <1>     ;             (higher 20 bits must be ZERO)
                                <1>     ;             (bit 0 must be 1)
                                <1>     ;       u.pgdir = page directory (physical) address
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = physical page address
                                <1>     ;       (EDX = PTE value)
                                <1>     ;       EBX = virtual address
                                <1>     ;
                                <1>     ;       CF = 1 -> error
                                <1>     ;
                                <1>     ; Modified Registers -> EAX, EDX
                                <1>     ;
000058E8 50                     <1>     push    eax
000058E9 A1[C48E0100]           <1>     mov     eax, [u.pgdir] ; 20/07/2015
000058EE E837000000             <1>     call    get_pde
                                <1>             ; EDX = PDE address
                                <1>             ; EAX = PDE value
000058F3 5A                     <1>     pop     edx ; physical page address
000058F4 722A                   <1>     jc      short spte_err ; PDE not present
                                <1>     ;
000058F6 53                     <1>     push    ebx ; 24/07/2015
000058F7 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear lower 12 bits
                                <1>                     ; EDX = PT address (physical)
000058FB C1EB0C                 <1>     shr     ebx, PAGE_SHIFT ; 12
000058FE 81E3FF030000           <1>     and     ebx, PTE_MASK  ; 03FFh
                                <1>                     ; clear higher 10 bits (PD bits)
00005904 C1E302                 <1>     shl     ebx, 2   ; offset to page table (1024*4)
00005907 01C3                   <1>     add     ebx, eax
                                <1>     ;
00005909 8B03                   <1>     mov     eax, [ebx] ; Old PTE value
0000590B A801                   <1>     test    al, 1
0000590D 740C                   <1>     jz      short spte_0
0000590F 09D2                   <1>     or      edx, edx
00005911 750F                   <1>     jnz     short spte_1
00005913 662500F0               <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear lower 12 bits
00005917 89C2                   <1>     mov     edx, eax
00005919 EB09                   <1>     jmp     short spte_2
                                <1> spte_0:
                                <1>     ; If this PTE contains a swap (disk) address,
                                <1>     ; it can be updated by using 'swap_in' procedure
                                <1>     ; only!
0000591B 21C0                   <1>     and     eax, eax
0000591D 7403                   <1>     jz      short spte_1
                                <1>     ; 24/07/2015
                                <1>     ; swapped page ! (on disk)
0000591F 5B                     <1>     pop     ebx
                                <1> spte_err:
00005920 F9                     <1>     stc
00005921 C3                     <1>     retn
                                <1> spte_1:
00005922 89D0                   <1>     mov     eax, edx
                                <1> spte_2:
00005924 09CA                   <1>     or      edx, ecx
                                <1>     ; 23/06/2015
00005926 8913                   <1>     mov     [ebx], edx ; PTE value in EDX
                                <1>     ; 24/07/2015
00005928 5B                     <1>     pop     ebx
00005929 C3                     <1>     retn
                                <1>
                                <1> get_pde:    ; Get present value of the relevant PDE
                                <1>     ; 20/07/2015
                                <1>     ; 18/04/2015
                                <1>     ; 12/04/2015
                                <1>     ; 10/10/2014 ; (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EBX = virtual (linear) address
                                <1>     ;       EAX = page directory (physical) address
                                <1>     ; OUTPUT ->
                                <1>     ;       EDX = Page directory entry address
                                <1>     ;       EAX = Page directory entry value
                                <1>     ;       CF = 1 -> PDE not present or invalid ?
                                <1>     ; Modified Registers -> EDX, EAX
                                <1>     ;
0000592A 89DA                   <1>     mov     edx, ebx
0000592C C1EA16                 <1>     shr     edx, PAGE_D_SHIFT ; 22  (12+10)
0000592F C1E202                 <1>     shl     edx, 2 ; offset to page directory (1024*4)
00005932 01C2                   <1>     add     edx, eax ; page directory address (physical)
00005934 8B02                   <1>     mov     eax, [edx]
00005936 A801                   <1>     test    al, PDE_A_PRESENT ; page table is present or not !
00005938 751F                   <1>     jnz     short gpte_retn
0000593A F9                     <1>     stc
                                <1> gpde_retn:
0000593B C3                     <1>     retn
                                <1>
                                <1> get_pte:
                                <1>             ; Get present value of the relevant PTE
                                <1>     ; 29/07/2015
```

```
                                <1>     ; 20/07/2015
                                <1>     ; 18/04/2015
                                <1>     ; 12/04/2015
                                <1>     ; 10/10/2014 ; (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EBX = virtual (linear) address
                                <1>     ;       EAX = page directory (physical) address
                                <1>     ; OUTPUT ->
                                <1>     ;       EDX = Page table entry address (if CF=0)
                                <1>     ;             Page directory entry address (if CF=1)
                                <1>     ;            (Bit 0 value is 0 if PT is not present)
                                <1>     ;       EAX = Page table entry value (page address)
                                <1>     ;       CF = 1 -> PDE not present or invalid ?
                                <1>     ; Modified Registers -> EAX, EDX
                                <1>     ;
0000593C E8E9FFFFFF             <1>     call    get_pde
00005941 72F8                   <1>     jc      short gpde_retn       ; page table is not present
                                <1>     ;jnc    short gpte_1
                                <1>     ;retn
                                <1> ;gpte_1:
00005943 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear lower 12 bits
00005947 89DA                   <1>     mov     edx, ebx
00005949 C1EA0C                 <1>     shr     edx, PAGE_SHIFT ; 12
0000594C 81E2FF030000           <1>     and     edx, PTE_MASK  ; 03FFh
                                <1>                     ; clear higher 10 bits (PD bits)
00005952 C1E202                 <1>     shl     edx, 2 ; offset from start of page table (1024*4)
00005955 01C2                   <1>     add     edx, eax
00005957 8B02                   <1>     mov     eax, [edx]
                                <1> gpte_retn:
00005959 C3                     <1>     retn
                                <1>
                                <1> deallocate_page_dir:
                                <1>     ; 15/09/2015
                                <1>     ; 05/08/2015
                                <1>     ; 30/04/2015
                                <1>     ; 28/04/2015
                                <1>     ; 17/10/2014
                                <1>     ; 12/10/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = PHYSICAL ADDRESS OF THE PAGE DIRECTORY (CHILD)
                                <1>     ;       EBX = PHYSICAL ADDRESS OF THE PARENT'S PAGE DIRECTORY
                                <1>     ; OUTPUT ->
                                <1>     ;       All of page tables in the page directory
                                <1>     ;       and page dir's itself will be deallocated
                                <1>     ;       except 'read only' duplicated pages (will be converted
                                <1>     ;       to writable pages).
                                <1>     ;
                                <1>     ; Modified Registers -> EAX
                                <1>     ;
                                <1>     ;
0000595A 56                     <1>     push    esi
0000595B 51                     <1>     push    ecx
0000595C 50                     <1>     push    eax
0000595D 89C6                   <1>     mov     esi, eax
0000595F 31C9                   <1>     xor     ecx, ecx
                                <1>     ; The 1st PDE points to Kernel Page Table 0 (the 1st 4MB),
                                <1>     ; it must not be deallocated
00005961 890E                   <1>     mov     [esi], ecx ; 0 ; clear PDE 0
                                <1> dapd_0:
00005963 AD                     <1>     lodsd
00005964 A801                   <1>     test    al, PDE_A_PRESENT ; bit 0, present flag (must be 1)
00005966 7409                   <1>     jz      short dapd_1
00005968 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
0000596C E812000000             <1>     call    deallocate_page_table
                                <1> dapd_1:
00005971 41                     <1>     inc     ecx ; page directory entry index
00005972 81F900040000           <1>     cmp     ecx, PAGE_SIZE / 4 ; 1024
00005978 72E9                   <1>     jb      short dapd_0
                                <1> dapd_2:
0000597A 58                     <1>     pop     eax
0000597B E869000000             <1>     call    deallocate_page       ; deallocate the page dir's itself
00005980 59                     <1>     pop     ecx
00005981 5E                     <1>     pop     esi
00005982 C3                     <1>     retn
                                <1>
                                <1> deallocate_page_table:
                                <1>     ; 29/08/2023 - TRDOS 386 v2.0.6
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;       (temporary modifications)
                                <1>     ; 12/07/2016
                                <1>     ; 19/09/2015
                                <1>     ; 15/09/2015
                                <1>     ; 05/08/2015
                                <1>     ; 30/04/2015
                                <1>     ; 28/04/2015
                                <1>     ; 24/10/2014
                                <1>     ; 23/10/2014
                                <1>     ; 12/10/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS OF THE PAGE TABLE
                                <1>     ;       EBX = PHYSICAL ADDRESS OF THE PARENT'S PAGE DIRECTORY
                                <1>     ;       (ECX = page directory entry index)
                                <1>     ; OUTPUT ->
                                <1>     ;       All of pages in the page table and page table's itself
                                <1>     ;       will be deallocated except 'read only' duplicated pages
                                <1>     ;       (will be converted to writable pages).
                                <1>     ;
                                <1>     ; Modified Registers -> EAX
                                <1>     ;
00005983 56                     <1>     push    esi
00005984 57                     <1>     push    edi
00005985 52                     <1>     push    edx
00005986 50                     <1>     push    eax ; *
00005987 89C6                   <1>     mov     esi, eax
00005989 31FF                   <1>     xor     edi, edi ; 0
                                <1> dapt_0:
0000598B AD                     <1>     lodsd
0000598C A801                   <1>     test    al, PTE_A_PRESENT ; bit 0, present flag (must be 1)
0000598E 744C                   <1>     jz      short dapt_1
                                <1>     ;
00005990 A802                   <1>     test    al, PTE_A_WRITE   ; bit 1, writable (r/w) flag
                                <1>                               ; (must be 1)
00005992 753D                   <1>     jnz     short dapt_3
                                <1>     ; Read only -duplicated- page (belongs to a parent or a child)
00005994 66A90002               <1>         test    ax, PTE_DUPLICATED ; Was this page duplicated
                                <1>                                    ; as child's page ?
00005998 7442                   <1>     jz      short dapt_4 ; Clear PTE but don't deallocate the page!
                                <1>     ; check the parent's PTE value is read only & same page or not..
                                <1>     ; ECX = page directory entry index (0-1023)
0000599A 53                     <1>     push    ebx
0000599B 51                     <1>     push    ecx
```

```
                            <1>     ;shl    cx, 2 ; *4
                            <1>     ; 29/08/2023
0000599C C1E102             <1>     shl     ecx, 2
0000599F 01CB               <1>     add     ebx, ecx ; PDE offset (for the parent)
000059A1 8B0B               <1>     mov     ecx, [ebx]
000059A3 F6C101             <1>     test    cl, PDE_A_PRESENT ; present (valid) or not ?
000059A6 7427               <1>     jz      short dapt_2   ; parent process does not use this page
000059A8 6681E100F0         <1>     and     cx, PDE_A_CLEAR ; 0F000h ; Clear attribute bits
                            <1>     ; EDI = page table entry index (0-1023)
000059AD 89FA               <1>     mov     edx, edi
                            <1>     ;shl    dx, 2 ; *4
                            <1>     ; 29/08/2023
000059AF C1E202             <1>     shl     edx, 2
000059B2 01CA               <1>     add     edx, ecx ; PTE offset (for the parent)
000059B4 8B1A               <1>     mov     ebx, [edx]
000059B6 F6C301             <1>     test    bl, PTE_A_PRESENT ; present or not ?
000059B9 7414               <1>     jz      short dapt_2   ; parent process does not use this page
000059BB 662500F0           <1>     and     ax, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
000059BF 6681E300F0         <1>     and     bx, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
000059C4 39D8               <1>     cmp     eax, ebx       ; parent's and child's pages are same ?
000059C6 7507               <1>     jne     short dapt_2   ; not same page
                            <1>                            ; deallocate the child's page
000059C8 800A02             <1>         or      byte [edx], PTE_A_WRITE ; convert to writable page (parent)
000059CB 59                 <1>     pop     ecx
000059CC 5B                 <1>     pop     ebx
000059CD EB0D               <1>     jmp     short dapt_4
                            <1>
                            <1> ; 17/04/2021
                            <1> ; ('dapt_1' is disabled as temporary)
                            <1> ;
                            <1> ;dapt_1:
                            <1> ;   or      eax, eax       ; swapped page ?
                            <1> ;   jz      short dapt_5   ; no
                            <1> ;                          ; yes
                            <1> ;   shr     eax, 1
                            <1> ;   call    unlink_swap_block ; Deallocate swapped page block
                            <1> ;                             ; on the swap disk (or in file)
                            <1> ;   jmp     short dapt_5
                            <1> dapt_2:
000059CF 59                 <1>     pop     ecx
000059D0 5B                 <1>     pop     ebx
                            <1> dapt_3:
                            <1>     ; 12/07/2016
000059D1 66A90004           <1>     test    ax, PTE_SHARED ; shared or direct memory access indicator
000059D5 7505               <1>     jnz     short dapt_4   ; AVL bit 1 = 1, do not deallocate this page!
                            <1>     ;
                            <1>     ;and    ax, PTE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
000059D7 E80D000000         <1>     call    deallocate_page ; set the mem allocation bit of this page
                            <1> dapt_4:
                            <1>     ; 20/10/2023
                            <1>     ; PTE clearing here was/is not necessary
                            <1>     ; because this page table is being deallocated
                            <1>     ; and it's PTEs will be ineffective.
                            <1>
                            <1>     ;mov    dword [esi-4], 0 ; clear/reset PTE (child, dupl. as parent)
                            <1>
                            <1> dapt_1:     ; 17/04/2021 (temporary)
                            <1> dapt_5:
000059DC 47                 <1>     inc     edi ; page table entry index
000059DD 81FF00040000       <1>     cmp     edi, PAGE_SIZE / 4 ; 1024
000059E3 72A6               <1>     jb      short dapt_0
                            <1>     ;
000059E5 58                 <1>     pop     eax ; *
000059E6 5A                 <1>     pop     edx
000059E7 5F                 <1>     pop     edi
000059E8 5E                 <1>     pop     esi
                            <1>     ;
                            <1>     ;call   deallocate_page       ; deallocate the page table's itself
                            <1>     ;retn
                            <1>
                            <1> deallocate_page:
                            <1>     ; 15/09/2015
                            <1>     ; 28/04/2015
                            <1>     ; 10/03/2015
                            <1>     ; 17/10/2014
                            <1>     ; 12/10/2014 (Retro UNIX 386 v1 - beginning)
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS OF THE ALLOCATED PAGE
                            <1>     ; OUTPUT ->
                            <1>     ;       [free_pages] is increased
                            <1>     ;       (corresponding MEMORY ALLOCATION TABLE bit is SET)
                            <1>     ;       CF = 1 if the page is already deallocated
                            <1>     ;              (or not allocated) before.
                            <1>     ;
                            <1>     ; Modified Registers -> EAX
                            <1>     ;
000059E9 53                 <1>     push    ebx
000059EA 52                 <1>     push    edx
                            <1>     ;
000059EB C1E80C             <1>     shr     eax, PAGE_SHIFT      ; shift physical address to
                            <1>                                  ; 12 bits right
                            <1>                                  ; to get page number
000059EE 89C2               <1>     mov     edx, eax
                            <1>     ; 15/09/2015
000059F0 C1EA03             <1>     shr     edx, 3               ; to get offset to M.A.T.
                            <1>                                  ; (1 allocation bit = 1 page)
                            <1>                                  ; (1 allocation bytes = 8 pages)
000059F3 80E2FC             <1>     and     dl, 0FCh             ; clear lower 2 bits
                            <1>                                  ; (to get 32 bit position)
                            <1>     ;
000059F6 BB00001000         <1>     mov     ebx, MEM_ALLOC_TBL   ; Memory Allocation Table address
000059FB 01D3               <1>     add     ebx, edx
000059FD 83E01F             <1>     and     eax, 1Fh             ; lower 5 bits only
                            <1>                                  ; (allocation bit position)
00005A00 3B15[BC760100]     <1>     cmp     edx, [next_page]     ; is the new free page address lower
                            <1>                                  ; than the address in 'next_page' ?
                            <1>                                  ; (next/first free page value)
00005A06 7306               <1>     jnb     short dap_1          ; no
00005A08 8915[BC760100]     <1>     mov     [next_page], edx     ; yes
                            <1> dap_1:
00005A0E 0FAB03             <1>     bts     [ebx], eax           ; unlink/release/deallocate page
                            <1>                                  ; set relevant bit to 1.
                            <1>                                  ; set CF to the previous bit value
                            <1>     ;cmc                         ; complement carry flag
                            <1>     ;jc     short dap_2          ; do not increase free_pages count
                            <1>                                  ; if the page is already deallocated
                            <1>                                  ; before.
00005A11 FF05[B8760100]     <1>         inc     dword [free_pages]
                            <1> dap_2:
00005A17 5A                 <1>     pop     edx
00005A18 5B                 <1>     pop     ebx
00005A19 C3                 <1>     retn
                            <1>
```

```
                                <1> ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
                                <1> ;;                                                                    ;;
                                <1> ;; Copyright (C) KolibriOS team 2004-2012. All rights reserved. ;;
                                <1> ;; Distributed under terms of the GNU General Public License    ;;
                                <1> ;;                                                                    ;;
                                <1> ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
                                <1>
                                <1> ;;$Revision: 5057 $
                                <1>
                                <1>
                                <1> ;;align 4
                                <1> ;;proc alloc_page
                                <1>
                                <1> ;;        pushfd
                                <1> ;;        cli
                                <1> ;;        push    ebx
                                <1> ;;;//-
                                <1> ;;        cmp     [pg_data.pages_free], 1
                                <1> ;;        jle     .out_of_memory
                                <1> ;;;//-
                                <1> ;;
                                <1> ;;        mov     ebx, [page_start]
                                <1> ;;        mov     ecx, [page_end]
                                <1> ;;.l1:
                                <1> ;;        bsf     eax, [ebx];
                                <1> ;;        jnz     .found
                                <1> ;;        add     ebx, 4
                                <1> ;;        cmp     ebx, ecx
                                <1> ;;        jb      .l1
                                <1> ;;        pop     ebx
                                <1> ;;        popfd
                                <1> ;;        xor     eax, eax
                                <1> ;;        ret
                                <1> ;;.found:
                                <1> ;;;//-
                                <1> ;;        dec     [pg_data.pages_free]
                                <1> ;;        jz      .out_of_memory
                                <1> ;;;//-
                                <1> ;;        btr     [ebx], eax
                                <1> ;;        mov     [page_start], ebx
                                <1> ;;        sub     ebx, sys_pgmap
                                <1> ;;        lea     eax, [eax+ebx*8]
                                <1> ;;        shl     eax, 12
                                <1> ;;;//-       dec [pg_data.pages_free]
                                <1> ;;        pop     ebx
                                <1> ;;        popfd
                                <1> ;;        ret
                                <1> ;;;//-
                                <1> ;;.out_of_memory:
                                <1> ;;        mov     [pg_data.pages_free], 1
                                <1> ;;        xor     eax, eax
                                <1> ;;        pop     ebx
                                <1> ;;        popfd
                                <1> ;;        ret
                                <1> ;;;//-
                                <1> ;;endp
                                <1>
                                <1> duplicate_page_dir:
                                <1>     ; 21/09/2015
                                <1>     ; 31/08/2015
                                <1>     ; 20/07/2015
                                <1>     ; 28/04/2015
                                <1>     ; 27/04/2015
                                <1>     ; 18/04/2015
                                <1>     ; 12/04/2015
                                <1>     ; 18/10/2014
                                <1>     ; 16/10/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       [u.pgdir] = PHYSICAL (real/flat) ADDRESS of the parent's
                                <1>     ;                   page directory.
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX =  PHYSICAL (real/flat) ADDRESS of the child's
                                <1>     ;              page directory.
                                <1>     ;       (New page directory with new page table entries.)
                                <1>     ;       (New page tables with read only copies of the parent's
                                <1>     ;       pages.)
                                <1>     ;       EAX = 0 -> Error (CF = 1)
                                <1>     ;
                                <1>     ; Modified Registers -> none (except EAX)
                                <1>     ;
00005A1A E80BFEFFFF             <1>     call    allocate_page
00005A1F 723E                   <1>     jc      short dpd_err
                                <1>     ;
00005A21 55                     <1>     push    ebp ; 20/07/2015
00005A22 56                     <1>     push    esi
00005A23 57                     <1>     push    edi
00005A24 53                     <1>     push    ebx
00005A25 51                     <1>     push    ecx
00005A26 8B35[C48E0100]         <1>     mov     esi, [u.pgdir]
00005A2C 89C7                   <1>     mov     edi, eax
00005A2E 50                     <1>     push    eax ; save child's page directory address
                                <1>     ; 31/08/2015
                                <1>     ; copy PDE 0 from the parent's page dir to the child's page dir
                                <1>     ; (use same system space for all user page tables)
00005A2F A5                     <1>     movsd
00005A30 BD00004000             <1>     mov     ebp, 1024*4096 ; pass the 1st 4MB (system space)
00005A35 B9FF030000             <1>     mov     ecx, (PAGE_SIZE / 4) - 1 ; 1023
                                <1> dpd_0:
00005A3A AD                     <1>     lodsd
                                <1>     ;or     eax, eax
                                <1>         ;jnz     short dpd_1
00005A3B A801                   <1>     test    al, PDE_A_PRESENT ;  bit 0 =  1
00005A3D 7508                   <1>     jnz     short dpd_1
                                <1>     ; 20/07/2015 (virtual address at the end of the page table)
00005A3F 81C500004000           <1>     add     ebp, 1024*4096 ; page size * PTE count
00005A45 EB0F                   <1>     jmp     short dpd_2
                                <1> dpd_1:
00005A47 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear attribute bits
00005A4B 89C3                   <1>     mov     ebx, eax
                                <1>     ; EBX = Parent's page table address
00005A4D E81F000000             <1>     call    duplicate_page_table
00005A52 720C                   <1>     jc      short dpd_p_err
                                <1>     ; EAX = Child's page table address
00005A54 0C07                   <1>     or      al, PDE_A_PRESENT + PDE_A_WRITE + PDE_A_USER
                                <1>                     ; set bit 0, bit 1 and bit 2 to 1
                                <1>                     ; (present, writable, user)
                                <1> dpd_2:
00005A56 AB                     <1>     stosd
00005A57 E2E1                   <1>     loop    dpd_0
                                <1>     ;
00005A59 58                     <1>     pop     eax  ; restore child's page directory address
                                <1> dpd_3:
00005A5A 59                     <1>     pop     ecx
```

```
00005A5B 5B                  <1>     pop     ebx
00005A5C 5F                  <1>     pop     edi
00005A5D 5E                  <1>     pop     esi
00005A5E 5D                  <1>     pop     ebp ; 20/07/2015
                             <1> dpd_err:
00005A5F C3                  <1>     retn
                             <1> dpd_p_err:
                             <1>     ; release the allocated pages missing (recover free space)
00005A60 58                  <1>     pop     eax  ; the new page directory address (physical)
00005A61 8B1D[C48E0100]      <1>     mov     ebx, [u.pgdir] ; parent's page directory address
00005A67 E8EEFEFFFF          <1>     call    deallocate_page_dir
00005A6C 29C0                <1>     sub     eax, eax ; 0
00005A6E F9                  <1>     stc
00005A6F EBE9                <1>     jmp     short dpd_3
                             <1>
                             <1> duplicate_page_table:
                             <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                             <1>     ;       (temporary modifications)
                             <1>     ; 20/02/2017
                             <1>     ; 21/09/2015
                             <1>     ; 20/07/2015
                             <1>     ; 05/05/2015
                             <1>     ; 28/04/2015
                             <1>     ; 27/04/2015
                             <1>     ; 18/04/2015
                             <1>     ; 18/10/2014
                             <1>     ; 16/10/2014 (Retro UNIX 386 v1 - beginning)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       EBX = PHYSICAL (real/flat) ADDRESS of the parent's page table.
                             <1>     ;        20/02/2017
                             <1>     ;       EBP = Linear address of the page (from 'duplicate_page_dir')
                             <1>     ;             (Linear address = CORE + user's virtual address)
                             <1>     ; OUTPUT ->
                             <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS of the child's page table.
                             <1>     ;             (with 'read only' attribute of page table entries)
                             <1>     ;       20/02/2017
                             <1>     ;       EBP = Next linear page address (for 'duplicate_page_dir')
                             <1>     ;
                             <1>     ;       CF = 1 -> error
                             <1>     ;
                             <1>     ; Modified Registers -> EBP (except EAX)
                             <1>     ;
00005A71 E8B4FDFFFF          <1>     call    allocate_page
00005A76 7258                <1>     jc      short dpt_err
                             <1>     ;
00005A78 50                  <1>     push    eax ; *
00005A79 56                  <1>     push    esi
00005A7A 57                  <1>     push    edi
00005A7B 52                  <1>     push    edx
00005A7C 51                  <1>     push    ecx
                             <1>     ;
00005A7D 89DE                <1>     mov     esi, ebx
00005A7F 89C7                <1>     mov     edi, eax
00005A81 89C2                <1>     mov     edx, eax
00005A83 81C200100000        <1>     add     edx, PAGE_SIZE
                             <1> dpt_0:
00005A89 AD                  <1>     lodsd
00005A8A 21C0                <1>     and     eax, eax
00005A8C 7432                <1>     jz      short dpt_3
00005A8E A801                <1>     test    al, PTE_A_PRESENT ;  bit 0 =  1
                             <1>     ; 17/04/2021 (temporary)
                             <1>     ;jnz    short dpt_1
00005A90 7439                <1>     jz      short dpt_p_err
                             <1>
                             <1> ; 17/04/2021
                             <1> ; ('reload_page' procedure call is disabled as temporary)
                             <1> ;
                             <1> ;   ; 20/07/2015
                             <1> ;   ; ebp = virtual (linear) address of the memory page
                             <1> ;   call    reload_page ; 28/04/2015
                             <1> ;   jc      short dpt_p_err
                             <1> dpt_1:
                             <1>     ; 21/09/2015
00005A92 89C1                <1>     mov     ecx, eax
00005A94 662500F0            <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear attribute bits
00005A98 F6C102              <1>     test    cl, PTE_A_WRITE ; writable page ?
00005A9B 751A                <1>     jnz     short dpt_2
                             <1>     ; Read only (parent) page
                             <1>     ;       - there is a third process which uses this page -
                             <1>     ; Allocate a new page for the child process
00005A9D E888FDFFFF          <1>     call    allocate_page
00005AA2 7227                <1>     jc      short dpt_p_err
00005AA4 57                  <1>     push    edi
00005AA5 56                  <1>     push    esi
00005AA6 89CE                <1>     mov     esi, ecx
00005AA8 89C7                <1>     mov     edi, eax
00005AAA B900040000          <1>     mov     ecx, PAGE_SIZE/4
00005AAF F3A5                <1>     rep     movsd   ; copy page (4096 bytes)
00005AB1 5E                  <1>     pop     esi
00005AB2 5F                  <1>     pop     edi
                             <1>     ;
                             <1>
                             <1> ; 17/04/2021
                             <1> ; ('add_to_swap_queue' procedure call is disabled as temporary)
                             <1> ;
                             <1> ;   push    ebx
                             <1> ;   push    eax
                             <1> ;   ; 20/07/2015
                             <1> ;   mov     ebx, ebp
                             <1> ;   ; ebx = virtual (linear) address of the memory page
                             <1> ;   call    add_to_swap_queue
                             <1> ;   pop     eax
                             <1> ;   pop     ebx
                             <1>
                             <1>     ; 21/09/2015
00005AB3 0C07                <1>     or      al, PTE_A_USER+PTE_A_WRITE+PTE_A_PRESENT
                             <1>             ; user + writable + present page
00005AB5 EB09                <1>     jmp     short dpt_3
                             <1> dpt_2:
                             <1>     ;or     ax, PTE_A_USER+PTE_A_PRESENT
00005AB7 0C05                <1>     or      al, PTE_A_USER+PTE_A_PRESENT
                             <1>                 ; (read only page!)
00005AB9 8946FC              <1>     mov     [esi-4], eax ; update parent's PTE
00005ABC 660D0002            <1>     or       ax, PTE_DUPLICATED  ; (read only page & duplicated PTE!)
                             <1> dpt_3:
00005AC0 AB                  <1>     stosd  ; EDI points to child's PTE
                             <1>     ;
00005AC1 81C500100000        <1>     add     ebp, 4096 ; 20/07/2015 (next page)
                             <1>     ;
00005AC7 39D7                <1>     cmp     edi, edx
00005AC9 72BE                <1>     jb      short dpt_0
                             <1> dpt_p_err:
00005ACB 59                  <1>     pop     ecx
```

```
00005ACC 5A                     <1>     pop     edx
00005ACD 5F                     <1>     pop     edi
00005ACE 5E                     <1>     pop     esi
00005ACF 58                     <1>     pop     eax ; *
                                <1> dpt_err:
00005AD0 C3                     <1>     retn
                                <1>
                                <1> page_fault_handler:       ; CPU EXCEPTION 0Eh (14) : Page Fault !
                                <1>     ; 09/02/2026 - TRDOS 386 v2.0.10
                                <1>     ; 29/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;       (temporary modifications)
                                <1>     ; 21/09/2015
                                <1>     ; 19/09/2015
                                <1>     ; 17/09/2015
                                <1>     ; 28/08/2015
                                <1>     ; 20/07/2015
                                <1>     ; 28/06/2015
                                <1>     ; 03/05/2015
                                <1>     ; 30/04/2015
                                <1>     ; 18/04/2015
                                <1>     ; 12/04/2015
                                <1>     ; 30/10/2014
                                <1>     ; 11/09/2014
                                <1>     ; 10/09/2014 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; Note: This is not an interrupt/exception handler.
                                <1>     ;      This is a 'page fault remedy' subroutine
                                <1>     ;      which will be called by standard/uniform
                                <1>     ;      exception handler.
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      [error_code] = 32 bit ERROR CODE (lower 5 bits are valid)
                                <1>     ;
                                <1>     ;      cr2 = the virtual (linear) address
                                <1>     ;            which has caused to page fault (19/09/2015)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      (corresponding PAGE TABLE ENTRY is mapped/set)
                                <1>     ;      EAX = 0 -> no error
                                <1>     ;      EAX > 0 -> error code in EAX (also CF = 1)
                                <1>     ;
                                <1>     ; Modified Registers -> none (except EAX)
                                <1>     ;
                                <1>         ;
                                <1>         ; ERROR CODE:
                                <1>     ;       31  .....     4   3   2   1   0
                                <1>     ;      +---+-- --+---+---+---+---+---+
                                <1>     ;      |   Reserved  | I | R | U | W | P |
                                <1>     ;      +---+-- --+---+---+---+---+---+
                                <1>     ;
                                <1>     ; P : PRESENT -       when set, the page fault was caused by
                                <1>     ;             a page-protection violation. When not set,
                                <1>     ;             it was caused by a non-present page.
                                <1>     ; W : WRITE   -       when set, the page fault was caused by
                                <1>     ;             a page write. When not set, it was caused
                                <1>     ;             by a page read.
                                <1>     ; U : USER    -       when set, the page fault was caused
                                <1>     ;             while CPL = 3.
                                <1>     ;             This does not necessarily mean that
                                <1>     ;             the page fault was a privilege violation.
                                <1>     ; R : RESERVD -       when set, the page fault was caused by
                                <1>     ;     WRITE   reading a 1 in a reserved field.
                                <1>     ; I : INSTRUC -       when set, the page fault was caused by
                                <1>     ;     FETCH   an instruction fetch
                                <1>     ;
                                <1>     ;; x86 (32 bit) VIRTUAL ADDRESS TRANSLATION
                                <1>     ;  31               22                  12 11                    0
                                <1>     ; +------------------+------------------+-----------------------+
                                <1>             ; | PAGE DIR. ENTRY # | PAGE TAB. ENTRY # |        OFFSET         |
                                <1>             ; +-------------------+-------------------+-----------------------+
                                <1>     ;
                                <1>
                                <1>     ;; CR3 REGISTER (Control Register 3)
                                <1>     ;  31                                   12             5 4 3 2   0
                                <1>     ; +---------------------------------------+-------------+---+-----+
                                <1>             ; |                                       |                 |P|P|     |
                                <1>             ; |   PAGE DIRECTORY TABLE BASE ADDRESS   |  reserved       |C|W|rsvrd|
                                <1>             ; |                                       |         |D|T|     |
                                <1>     ; +---------------------------------------+-------------+---+-----+
                                <1>     ;
                                <1>     ;      PWT    - WRITE THROUGH
                                <1>     ;      PCD    - CACHE DISABLE
                                <1>     ;
                                <1>     ;
                                <1>     ;; x86 PAGE DIRECTORY ENTRY (4 KByte Page)
                                <1>     ;  31                                   12 11  9 8 7 6 5 4 3 2 1 0
                                <1>     ; +---------------------------------------+-----+---+-+-+---+-+-+-+
                                <1>             ; |                                       |     | | | | |P|P|U|R| |
                                <1>             ; |     PAGE TABLE BASE ADDRESS 31..12    | AVL |G|0|D|A|C|W|/|/|P|
                                <1>             ; |                                       |     | | | | |D|T|S|W| |
                                <1>     ; +---------------------------------------+-----+---+-+-+---+-+-+-+
                                <1>     ;
                                <1>         ;       P      - PRESENT
                                <1>         ;       R/W    - READ/WRITE
                                <1>         ;       U/S    - USER/SUPERVISOR
                                <1>     ;       PWT    - WRITE THROUGH
                                <1>     ;       PCD    - CACHE DISABLE
                                <1>     ;       A      - ACCESSED
                                <1>         ;       D      - DIRTY (IGNORED)
                                <1>     ;       PAT    - PAGE ATTRIBUTE TABLE INDEX (CACHE BEHAVIOR)
                                <1>     ;       G      - GLOBAL       (IGNORED)
                                <1>         ;       AVL    - AVAILABLE FOR SYSTEMS PROGRAMMER USE
                                <1>     ;
                                <1>     ;
                                <1>     ;; x86 PAGE TABLE ENTRY (4 KByte Page)
                                <1>     ;  31                                   12 11  9 8 7 6 5 4 3 2 1 0
                                <1>     ; +---------------------------------------+-----+---+-+-+---+-+-+-+
                                <1>             ; |                                       |     | |P| | |P|P|U|R| |
                                <1>             ; |     PAGE FRAME BASE ADDRESS 31..12    | AVL |G|A|D|A|C|W|/|/|P|
                                <1>             ; |                                       |     | |T| | |D|T|S|W| |
                                <1>     ; +---------------------------------------+-----+---+-+-+---+-+-+-+
                                <1>     ;
                                <1>         ;       P      - PRESENT
                                <1>         ;       R/W    - READ/WRITE
                                <1>         ;       U/S    - USER/SUPERVISOR
                                <1>     ;       PWT    - WRITE THROUGH
                                <1>     ;       PCD    - CACHE DISABLE
                                <1>     ;       A      - ACCESSED
                                <1>         ;       D      - DIRTY
                                <1>     ;       PAT    - PAGE ATTRIBUTE TABLE INDEX (CACHE BEHAVIOR)
                                <1>     ;       G      - GLOBAL
                                <1>         ;       AVL    - AVAILABLE FOR SYSTEMS PROGRAMMER USE
                                <1>     ;
```

```
                                <1>     ;
                                <1>     ;; 80386 PAGE TABLE ENTRY (4 KByte Page)
                                <1>     ;  31                                   12 11  9 8 7 6 5 4 3 2 1 0
                                <1>     ; +-------------------------------------+-----+-+-+-+-+---+-+-+-+
                                <1>             ; |                                     |     | | | | | | |U|R| |
                                <1>             ; |     PAGE FRAME BASE ADDRESS 31..12  | AVL |0|0|D|A|0|0|/|/|P|
                                <1>             ; |                                     |     | | | | | | |S|W| |
                                <1>             ; +-------------------------------------+-----+-+-+-+-+---+-+-+-+
                                <1>     ;
                                <1>         ;       P      - PRESENT
                                <1>         ;       R/W    - READ/WRITE
                                <1>         ;       U/S    - USER/SUPERVISOR
                                <1>         ;       D      - DIRTY
                                <1>         ;       AVL    - AVAILABLE FOR SYSTEMS PROGRAMMER USE
                                <1>     ;
                                <1>         ;       NOTE: 0 INDICATES INTEL RESERVED. DO NOT DEFINE.
                                <1>     ;
                                <1>     ;
                                <1>     ;; Invalid Page Table Entry
                                <1>     ; 31                                                          1 0
                                <1>             ; +-----------------------------------------------------------+-+
                                <1>             ; |                                                           | |
                                <1>             ; |                           AVAILABLE                       |0|
                                <1>             ; |                                                           | |
                                <1>             ; +-----------------------------------------------------------+-+
                                <1>     ;
                                <1>
00005AD1 53                     <1>     push    ebx
00005AD2 52                     <1>     push    edx
00005AD3 51                     <1>     push    ecx
                                <1>     ;
                                <1>     ; 21/09/2015 (debugging)
00005AD4 FF05[DC8E0100]         <1>     inc     dword [u.pfcount] ; page fault count for running process
00005ADA FF05[88900100]         <1>     inc     dword [PF_Count]  ; total page fault count
                                <1>     ; 28/06/2015
                                <1>     ;mov    edx, [error_code] ; Lower 5 bits are valid
00005AE0 8A15[80900100]         <1>     mov     dl, [error_code]
                                <1>     ;
00005AE6 F6C201                 <1>     test    dl, 1   ; page fault was caused by a non-present page
                                <1>                     ; sign
00005AE9 742D                   <1>     jz      short pfh_alloc_np
                                <1>     ;
                                <1>     ; If it is not a 'write on read only page' type page fault
                                <1>     ; major page fault error with minor reason must be returned without
                                <1>     ; fixing the problem. 'sys_exit with error' will be needed
                                <1>     ; after return here!
                                <1>     ; Page fault will be remedied, by copying page contents
                                <1>     ; to newly allocated page with write permission;
                                <1>     ; sys_fork -> sys_exec -> copy on write, demand paging method is
                                <1>     ; used for working with minimum possible memory usage.
                                <1>     ; sys_fork will duplicate page directory and tables of parent
                                <1>     ; process with 'read only' flag. If the child process attempts to
                                <1>     ; write on these read only pages, page fault will be directed here
                                <1>     ; for allocating a new page with same data/content.
                                <1>     ;
                                <1>     ; IMPORTANT : Retro UNIX 386 v1 (and SINGLIX and TR-DOS)
                                <1>     ; will not force to separate CODE and DATA space
                                <1>     ; in a process/program...
                                <1>     ; CODE segment/section may contain DATA!
                                <1>     ; It is flat, smoth and simplest programming method already as in
                                <1>     ; Retro UNIX 8086 v1 and MS-DOS programs.
                                <1>     ;
00005AEB F6C202                 <1>     test    dl, 2   ; page fault was caused by a page write
                                <1>                     ; sign
00005AEE 7418                   <1>         jz      short pfh_p_err
                                <1>     ; 31/08/2015
00005AF0 F6C204                 <1>     test    dl, 4   ; page fault was caused while CPL = 3 (user mode)
                                <1>                     ; sign. (U+W+P = 4+2+1 = 7)
00005AF3 741B                   <1>         jz  short pfh_pv_err
                                <1>     ;
                                <1>     ; make a new page and copy the parent's page content
                                <1>     ; as the child's new page content
                                <1>     ;
00005AF5 0F20D3                 <1>     mov     ebx, cr2 ; CR2 contains the linear address
                                <1>                      ; which has caused to page fault
00005AF8 E884000000             <1>     call    copy_page
00005AFD 7202                   <1>         jc  short pfh_im_err ; insufficient memory
                                <1>     ;
00005AFF EB7A                   <1>         jmp     pfh_cpp_ok
                                <1>
                                <1>     ; 29/11/2023
                                <1> pfh_im_err:
00005B01 B8E4000000             <1>     mov     eax, ERR_MAJOR_PF + ERR_MINOR_IM ; Error code in AX
                                <1>                     ; Major (Primary) Error: Page Fault
                                <1>                     ; Minor (Secondary) Error: Insufficient Memory !
00005B06 EB75                   <1>     jmp     short pfh_err_retn
                                <1>
                                <1>     ; 29/11/2023
                                <1> pfh_p_err: ; 09/03/2015
                                <1>     ; 09/02/2026 - temporary
00005B08 B8E5000000             <1>     mov     eax, ERR_MAJOR_PF + ERR_MINOR_RO ; Error code in AX
00005B0D F9                     <1>     stc
00005B0E EB6D                   <1>     jmp     short pfh_err_retn
                                <1>
                                <1> pfh_pv_err:
                                <1>     ; Page fault was caused by a protection-violation
00005B10 B8E6000000             <1>     mov     eax, ERR_MAJOR_PF + ERR_MINOR_PV ; Error code in AX
                                <1>                     ; Major (Primary) Error: Page Fault
                                <1>                     ; Minor (Secondary) Error: Protection violation !
00005B15 F9                     <1>     stc
00005B16 EB65                   <1>     jmp     short pfh_err_retn
                                <1>
                                <1> pfh_alloc_np:
00005B18 E80DFDFFFF             <1>     call    allocate_page  ; (allocate a new page)
00005B1D 72E2                   <1>         jc  short pfh_im_err ; 'insufficient memory' error
                                <1> pfh_chk_cpl:
                                <1>     ; EAX = Physical (base) address of the allocated (new) page
                                <1>             ; (Lower 12 bits are ZERO, because
                                <1>             ;       the address is on a page boundary)
00005B1F 80E204                 <1>     and     dl, 4   ; CPL = 3 ?
00005B22 7505                   <1>     jnz     short pfh_um
                                <1>                     ; Page fault handler for kernel/system mode (CPL=0)
00005B24 0F20DB                 <1>     mov     ebx, cr3 ; CR3 (Control Register 3) contains physical address
                                <1>                      ; of the current/active page directory
                                <1>                      ; (Always kernel/system mode page directory, here!)
                                <1>                      ; Note: Lower 12 bits are 0. (page boundary)
00005B27 EB06                   <1>     jmp     short pfh_get_pde
                                <1>     ;
                                <1> pfh_um:                     ; Page fault handler for user/appl. mode (CPL=3)
00005B29 8B1D[C48E0100]         <1>     mov     ebx, [u.pgdir] ; Page directory of current/active process
                                <1>                     ; Physical address of the USER's page directory
                                <1>                     ; Note: Lower 12 bits are 0. (page boundary)
                                <1> pfh_get_pde:
00005B2F 80CA03                 <1>     or      dl, 3   ; USER + WRITE + PRESENT or SYSTEM + WRITE + PRESENT
```

```
00005B32 0F20D1          <1>     mov     ecx, cr2 ; CR2 contains the virtual address
                         <1>                      ; which has been caused to page fault
                         <1>                      ;
00005B35 C1E914          <1>     shr     ecx, 20  ; shift 20 bits right
00005B38 80E1FC          <1>     and     cl, 0FCh ; mask lower 2 bits to get PDE offset
                         <1>     ;
00005B3B 01CB            <1>     add     ebx, ecx ; now, EBX points to the relevant page dir entry
00005B3D 8B0B            <1>     mov     ecx, [ebx] ; physical (base) address of the page table
00005B3F F6C101          <1>     test    cl, 1   ; check bit 0 is set (1) or not (0).
00005B42 740B            <1>     jz      short pfh_set_pde ; Page directory entry is not valid,
                         <1>                               ; set/validate page directory entry
00005B44 6681E100F0      <1>     and     cx, PDE_A_CLEAR ; 0F000h ; Clear attribute bits
00005B49 89CB            <1>     mov     ebx, ecx ; Physical address of the page table
00005B4B 89C1            <1>     mov     ecx, eax ; new page address (physical)
00005B4D EB16            <1>     jmp     short pfh_get_pte
                         <1> pfh_set_pde:
                         <1>     ;; NOTE: Page directories and page tables never be swapped out!
                         <1>     ;;       (So, we know this PDE is empty or invalid)
                         <1>     ;
00005B4F 08D0            <1>     or      al, dl   ; lower 3 bits are used as U/S, R/W, P flags
00005B51 8903            <1>     mov     [ebx], eax ; Let's put the new page directory entry here !
00005B53 30C0            <1>     xor     al, al   ; clear lower (3..8) bits
00005B55 89C3            <1>     mov     ebx, eax
00005B57 E8CEFCFFFF      <1>     call    allocate_page   ; (allocate a new page)
00005B5C 72A3            <1>     jc      short pfh_im_err   ; 'insufficient memory' error
                         <1> pfh_spde_1:
                         <1>     ; EAX = Physical (base) address of the allocated (new) page
00005B5E 89C1            <1>     mov     ecx, eax
00005B60 E836FDFFFF      <1>     call    clear_page ; Clear page content
                         <1> pfh_get_pte:
00005B65 0F20D0          <1>     mov     eax, cr2 ; virtual address
                         <1>                      ; which has been caused to page fault
00005B68 89C7            <1>     mov     edi, eax ; 20/07/2015
00005B6A C1E80C          <1>     shr     eax, 12  ; shift 12 bit right to get
                         <1>                      ; higher 20 bits of the page fault address
00005B6D 25FF030000      <1>     and     eax, 3FFh ; mask PDE# bits, the result is PTE# (0 to 1023)
00005B72 C1E002          <1>     shl     eax, 2  ; shift 2 bits left to get PTE offset
00005B75 01C3            <1>     add     ebx, eax ; now, EBX points to the relevant page table entry
                         <1> ; 17/04/2021 temporary
                         <1> ;   mov     eax, [ebx] ; get previous value of pte
                         <1> ;           ; bit 0 of EAX is always 0 (otherwise we would not be here)
                         <1>
                         <1> ; 17/04/2021
                         <1> ; ('swap_in' procedure call has been disabled as temporary)
                         <1> ;
                         <1> ;   and     eax, eax
                         <1> ;   jz      short pfh_gpte_1
                         <1> ;   ; 20/07/2015
                         <1> ;   xchg    ebx, ecx ; new page address (physical)
                         <1> ;   push    ebp ; 20/07/2015
                         <1> ;   mov     ebp, cr2
                         <1> ;           ; ECX = physical address of the page table entry
                         <1> ;           ; EBX = Memory page address (physical!)
                         <1> ;           ; EAX = Swap disk (offset) address
                         <1> ;           ; EBP = virtual address (page fault address)
                         <1> ;   call    swap_in
                         <1> ;   pop     ebp
                         <1> ;   jc       short pfh_err_retn
                         <1> ;   xchg    ecx, ebx
                         <1> ;           ; EBX = physical address of the page table entry
                         <1> ;           ; ECX = new page
                         <1> pfh_gpte_1:
00005B77 08D1            <1>     or      cl, dl  ; lower 3 bits are used as U/S, R/W, P flags
00005B79 890B            <1>     mov     [ebx], ecx ; Let's put the new page table entry here !
                         <1> pfh_cpp_ok:
                         <1> ; 17/04/2021
                         <1> ; ('add_to_swap_queue' procedure call has been disabled as temporary)
                         <1> ;
                         <1> ;   ; 20/07/2015
                         <1> ;   mov     ebx, cr2
                         <1> ;   call    add_to_swap_queue
                         <1>     ;
                         <1>     ; The new PTE (which contains the new page) will be added to
                         <1>     ; the swap queue, here.
                         <1>     ; (Later, if memory will become insufficient,
                         <1>     ; one page will be swapped out which is at the head of
                         <1>     ; the swap queue by using FIFO and access check methods.)
                         <1>     ;
00005B7B 31C0            <1>     xor     eax, eax  ; 0
                         <1>     ;
                         <1> pfh_err_retn:
00005B7D 59              <1>     pop     ecx
00005B7E 5A              <1>     pop     edx
00005B7F 5B              <1>     pop     ebx
00005B80 C3              <1>     retn
                         <1>
                         <1> copy_page:
                         <1>     ; 29/08/2023 (TRDOS 386 v2.0.6)
                         <1>     ; 22/09/2015
                         <1>     ; 21/09/2015
                         <1>     ; 19/09/2015
                         <1>     ; 07/09/2015
                         <1>     ; 31/08/2015
                         <1>     ; 20/07/2015
                         <1>     ; 05/05/2015
                         <1>     ; 03/05/2015
                         <1>     ; 18/04/2015
                         <1>     ; 12/04/2015
                         <1>     ; 30/10/2014
                         <1>     ; 18/10/2014 (Retro UNIX 386 v1 - beginning)
                         <1>     ;
                         <1>     ; INPUT ->
                         <1>     ;       EBX = Virtual (linear) address of source page
                         <1>     ;            (Page fault address)
                         <1>     ; OUTPUT ->
                         <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS OF THE ALLOCATED PAGE
                         <1>     ;       (corresponding PAGE TABLE ENTRY is mapped/set)
                         <1>     ;       EAX = 0 (CF = 1)
                         <1>     ;              if there is not a free page to be allocated
                         <1>     ;       (page content of the source page will be copied
                         <1>     ;       onto the target/new page)
                         <1>     ;
                         <1>     ; Modified Registers -> ecx, ebx (except EAX)
                         <1>     ;
00005B81 56              <1>     push    esi
00005B82 57              <1>     push    edi
                         <1>     ;push   ebx
                         <1>     ;push   ecx
00005B83 31F6            <1>     xor     esi, esi
00005B85 C1EB0C          <1>     shr     ebx, 12 ; shift 12 bits right to get PDE & PTE numbers
00005B88 89D9            <1>     mov     ecx, ebx ; save page fault address (as 12 bit shifted)
00005B8A C1EB08          <1>     shr     ebx, 8  ; shift 8 bits right and then
00005B8D 80E3FC          <1>     and     bl, 0FCh ; mask lower 2 bits to get PDE offset
00005B90 89DF            <1>     mov     edi, ebx ; save it for the parent of current process
```

```
00005B92 031D[C48E0100]      <1>     add     ebx, [u.pgdir] ; EBX points to the relevant page dir entry
00005B98 8B03                <1>     mov     eax, [ebx] ; physical (base) address of the page table
00005B9A 662500F0            <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear attribute bits
00005B9E 89CB                <1>     mov     ebx, ecx   ; (restore higher 20 bits of page fault address)
00005BA0 81E3FF030000        <1>     and     ebx, 3FFh  ; mask PDE# bits, the result is PTE# (0 to 1023)
                             <1>     ;shl    bx, 2      ; shift 2 bits left to get PTE offset
                             <1>     ; 29/08/2023
00005BA6 C1E302              <1>     shl     ebx, 2
00005BA9 01C3                <1>     add     ebx, eax   ; EBX points to the relevant page table entry
                             <1>     ; 07/09/2015
00005BAB 66F7030002          <1>         test    word [ebx], PTE_DUPLICATED ; (Does current process share this
                             <1>                                 ; read only page as a child process?)
00005BB0 7509                <1>     jnz     short cpp_0 ; yes
00005BB2 8B0B                <1>     mov     ecx, [ebx] ; PTE value
00005BB4 6681E100F0          <1>     and     cx, PTE_A_CLEAR ; 0F000h  ; clear page attributes
00005BB9 EB31                <1>     jmp     short cpp_1
                             <1> cpp_0:
00005BBB 89FE                <1>     mov     esi, edi
00005BBD 0335[C88E0100]      <1>     add     esi, [u.ppgdir] ; the parent's page directory entry
00005BC3 8B06                <1>     mov     eax, [esi] ; physical (base) address of the page table
00005BC5 662500F0            <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear attribute bits
00005BC9 89CE                <1>     mov     esi, ecx   ; (restore higher 20 bits of page fault address)
00005BCB 81E6FF030000        <1>     and     esi, 3FFh  ; mask PDE# bits, the result is PTE# (0 to 1023)
                             <1>     ;shl    si, 2      ; shift 2 bits left to get PTE offset
                             <1>     ; 29/08/2023
00005BD1 C1E602              <1>     shl     esi, 2
00005BD4 01C6                <1>     add     esi, eax   ; EDX points to the relevant page table entry
00005BD6 8B0E                <1>     mov     ecx, [esi] ; PTE value of the parent process
                             <1>     ; 21/09/2015
00005BD8 8B03                <1>     mov     eax, [ebx] ; PTE value of the child process
00005BDA 662500F0            <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear page attributes
                             <1>     ;
00005BDE F6C101              <1>     test    cl, PTE_A_PRESENT ; is it a present/valid page ?
00005BE1 7424                <1>     jz      short cpp_3 ; the parent's page is not same page
                             <1>     ;
00005BE3 6681E100F0          <1>     and     cx, PTE_A_CLEAR ; 0F000h ; clear page attributes
00005BE8 39C8                <1>     cmp     eax, ecx   ; Same page?
00005BEA 751B                <1>     jne     short cpp_3 ; Parent page and child page are not same
                             <1>                         ; Convert child's page to writable page
                             <1> cpp_1:
00005BEC E839FCFFFF          <1>     call    allocate_page
00005BF1 721A                <1>     jc      short cpp_4 ; 'insufficient memory' error
00005BF3 21F6                <1>     and     esi, esi    ; check ESI is valid or not
00005BF5 7405                <1>     jz      short cpp_2
                             <1>             ; Convert read only page to writable page
                             <1>             ;(for the parent of the current process)
                             <1>     ;and    word [esi], PTE_A_CLEAR ; 0F000h
                             <1>     ; 22/09/2015
00005BF7 890E                <1>     mov     [esi], ecx
00005BF9 800E07              <1>     or      byte [esi], PTE_A_PRESENT + PTE_A_WRITE + PTE_A_USER
                             <1>                             ; 1+2+4 = 7
                             <1> cpp_2:
00005BFC 89C7                <1>     mov     edi, eax ; new page address of the child process
                             <1>     ; 07/09/2015
00005BFE 89CE                <1>     mov     esi, ecx ; the page address of the parent process
00005C00 B900040000          <1>     mov     ecx, PAGE_SIZE / 4
00005C05 F3A5                <1>     rep     movsd ; 31/08/2015
                             <1> cpp_3:
00005C07 0C07                <1>     or      al, PTE_A_PRESENT + PTE_A_WRITE + PTE_A_USER ; 1+2+4 = 7
00005C09 8903                <1>     mov     [ebx], eax ; Update PTE
00005C0B 28C0                <1>     sub     al, al ; clear attributes
                             <1> cpp_4:
                             <1>     ;pop    ecx
                             <1>     ;pop    ebx
00005C0D 5F                  <1>     pop     edi
00005C0E 5E                  <1>     pop     esi
00005C0F C3                  <1>     retn
                             <1>
                             <1> ;; 28/04/2015
                             <1> ;; 24/10/2014
                             <1> ;; 21/10/2014 (Retro UNIX 386 v1 - beginning)
                             <1> ;; SWAP_PAGE_QUEUE (4096 bytes)
                             <1> ;;
                             <1> ;;   0000   0001   0002   0003   ....   1020   1021   1022   1023
                             <1> ;; +------+------+------+------+-    -+------+------+------+------+
                             <1> ;; |  pg1 |  pg2 |  pg3 |  pg4 | .... |pg1021|pg1022|pg1023|pg1024|
                             <1> ;; +------+------+------+------+-    -+------+------+------+------+
                             <1> ;;
                             <1> ;; [swpq_last] = 0 to 4096 (step 4) -> the last position on the queue
                             <1> ;;
                             <1> ;; Method:
                             <1> ;;  Swap page queue is a list of allocated pages with physical
                             <1> ;;  addresses (system mode virtual adresses = physical addresses).
                             <1> ;;  It is used for 'swap_in' and 'swap_out' procedures.
                             <1> ;;  When a new page is being allocated, swap queue is updated
                             <1> ;;  by 'swap_queue_shift' procedure, header of the queue (offset 0)
                             <1> ;;  is checked for 'accessed' flag. If the 1st page on the queue
                             <1> ;;  is 'accessed' or 'read only', it is dropped from the list;
                             <1> ;;  other pages from the 2nd to the last (in [swpq_last]) shifted
                             <1> ;;  to head then the 2nd page becomes the 1st and '[swpq_last]'
                             <1> ;;  offset value becomes it's previous offset value - 4.
                             <1> ;;  If the 1st page of the swap page queue is not 'accessed'
                             <1> ;;  the queue/list is not shifted.
                             <1> ;;  After the queue/list shift, newly allocated page is added
                             <1> ;;  to the tail of the queue at the [swpq_count*4] position.
                             <1> ;;  But, if [swpq_count] > 1023, the newly allocated page
                             <1> ;;  will not be added to the tail of swap page queue.
                             <1> ;;
                             <1> ;;  During 'swap_out' procedure, swap page queue is checked for
                             <1> ;;  the first non-accessed, writable page in the list,
                             <1> ;;  from the head to the tail. The list is shifted to left
                             <1> ;;  (to the head) till a non-accessed page will be found in the list.
                             <1> ;;  Then, this page       is swapped out (to disk) and then it is dropped
                             <1> ;;  from the list by a final swap queue shift. [swpq_count] value
                             <1> ;;  is changed. If all pages on the queue' are 'accessed',
                             <1> ;;  'insufficient memory' error will be returned ('swap_out'
                             <1> ;;  procedure will be failed)...
                             <1> ;;
                             <1> ;;  Note: If the 1st page of the queue is an 'accessed' page,
                             <1> ;;  'accessed' flag of the page will be reset (0) and that page
                             <1> ;;  (PTE) will be added to the tail of the queue after
                             <1> ;;  the check, if [swpq_count] < 1023. If [swpq_count] = 1024
                             <1> ;;  the queue will be rotated and the PTE in the head will be
                             <1> ;;  added to the tail after resetting 'accessed' bit.
                             <1> ;;
                             <1> ;;
                             <1> ;;
                             <1> ;; SWAP DISK/FILE (with 4096 bytes swapped page blocks)
                             <1> ;;
                             <1> ;;  00000000  00000004  00000008  0000000C   ...   size-8    size-4
                             <1> ;; +---------+---------+---------+---------+-- --+---------+---------+
                             <1> ;; |descriptr| page(1) | page(2) | page(3) | ... |page(n-1)| page(n) |
                             <1> ;; +---------+---------+---------+---------+-- --+---------+---------+
                             <1> ;;
```

```
<1> ;; [swpd_next] = the first free block address in swapped page records
<1> ;;                  for next free block search by 'swap_out' procedure.
<1> ;; [swpd_size] = swap disk/file size in sectors (512 bytes)
<1> ;;          NOTE: max. possible swap disk size is 1024 GB
<1> ;;          (entire swap space must be accessed by using
<1> ;;          31 bit offset address)
<1> ;; [swpd_free] = free block (4096 bytes) count in swap disk/file space
<1> ;; [swpd_start] = absolute/start address of the swap disk/file
<1> ;;            0 for file, or beginning sector of the swap partition
<1> ;; [swp_drv] = logical drive description table addr. of swap disk/file
<1> ;;
<1> ;;
<1> ;; Method:
<1> ;;  When the memory (ram) becomes insufficient, page allocation
<1> ;;  procedure swaps out a page from memory to the swap disk
<1> ;;  (partition) or swap file to get a new free page at the memory.
<1> ;;  Swapping out is performed by using swap page queue.
<1> ;;
<1> ;;  Allocation block size of swap disk/file is equal to page size
<1> ;;  (4096 bytes). Swapping address (in sectors) is recorded
<1> ;;  into relevant page file entry as 31 bit physical (logical)
<1> ;;  offset address as 1 bit shifted to left for present flag (0).
<1> ;;  Swapped page address is between 1 and swap disk/file size - 4.
<1> ;;  Absolute physical (logical) address of the swapped page is
<1> ;;  calculated by adding offset value to the swap partition's
<1> ;;  start address. If the swap device (disk) is a virtual disk
<1> ;;  or it is a file, start address of the swap disk/volume is 0,
<1> ;;  and offset value is equal to absolute (physical or logical)
<1> ;;  address/position. (It has not to be ZERO if the swap partition
<1> ;;  is in a partitioned virtual hard disk.)
<1> ;;
<1> ;;  Note: Swap addresses are always specified/declared in sectors,
<1> ;;  not in bytes or       in blocks/zones/clusters (4096 bytes) as unit.
<1> ;;
<1> ;;  Swap disk/file allocation is mapped via 'Swap Allocation Table'
<1> ;;  at memory as similar to 'Memory Allocation Table'.
<1> ;;
<1> ;;  Every bit of Swap Allocation Table repsesents one swap block
<1> ;;  (equal to page size) respectively. Bit 0 of the S.A.T. byte 0
<1> ;;  is reserved for swap disk/file block 0 as descriptor block
<1> ;;  (also for compatibility with PTE). If bit value is ZERO,
<1> ;;  it means relevant (respective) block is in use, and,
<1> ;;  of course, if bit value is 1, it means relevant (respective)
<1> ;;      swap disk/file block is free.
<1> ;;  For example: bit 1 of the byte 128 repsesents block 1025
<1> ;;  (128*8+1) or sector (offset) 8200 on the swap disk or
<1> ;;  byte (offset/position) 4198400 in the swap file.
<1> ;;  4GB swap space is represented via 128KB Swap Allocation Table.
<1> ;;  Initial layout of Swap Allocation Table is as follows:
<1> ;;  ------------------------------------------------------------
<1> ;;  01111111111111111111111111 .... 11111111111111111111111111111
<1> ;;  ------------------------------------------------------------
<1> ;;  (0 is reserved block, 1s represent free blocks respectively.)
<1> ;;  (Note: Allocation cell/unit of the table is bit, not byte)
<1> ;;
<1> ;;  ............................................................
<1> ;;
<1> ;;  'swap_out' procedure checks 'free_swap_blocks' count at first,
<1> ;;  then it searches Swap Allocation Table if free count is not
<1> ;;  zero. From begining the [swpd_next] dword value, the first bit
<1> ;;  position with value of 1 on the table is converted to swap
<1> ;;  disk/file offset address, in sectors (not 4096 bytes block).
<1> ;;  'ldrv_write' procedure is called with ldrv (logical drive
<1> ;;  number of physical swap disk or virtual swap disk)
<1> ;;  number, sector offset (not absolute sector -LBA- number),
<1> ;;  and sector count (8, 512*8 = 4096) and buffer adress
<1> ;;  (memory page). That will be a direct disk write procedure.
<1> ;;  (for preventing late memory allocation, significant waiting).
<1> ;;  If disk write procedure returns with error or free count of
<1> ;;  swap blocks is ZERO, 'swap_out' procedure will return with
<1> ;;  'insufficient memory error' (cf=1).
<1> ;;
<1> ;;  (Note: Even if free swap disk/file blocks was not zero,
<1> ;;  any disk write error will not be fixed by 'swap_out' procedure,
<1> ;;  in other words, 'swap_out' will not check the table for other
<1> ;;  free blocks after a disk write error. It will return to
<1> ;;  the caller with error (CF=1) which means swapping is failed.
<1> ;;
<1> ;;  After writing the page on to swap disk/file address/sector,
<1> ;;  'swap_out' procesure returns with that swap (offset) sector
<1> ;;  address (cf=0).
<1> ;;
<1> ;;  ............................................................
<1> ;;
<1> ;;  'swap_in' procedure loads addressed (relevant) swap disk or
<1> ;;  file sectors at specified memory page. Then page allocation
<1> ;;  procedure updates relevant page table entry with 'present'
<1> ;;  attribute. If swap disk or file reading fails there is nothing
<1> ;;  to do, except to terminate the process which is the owner of
<1> ;;  the swapped page.
<1> ;;
<1> ;;  'swap_in' procedure sets the relevant/respective bit value
<1> ;;  in the Swap Allocation Table (as free block). 'swap_in' also
<1> ;;  updates [swpd_first] pointer if it is required.
<1> ;;
<1> ;;  ............................................................
<1> ;;
<1> ;;  Note: If [swap_enabled] value is ZERO, that means there is not
<1> ;;  a swap disk or swap file in use... 'swap_in' and 'swap_out'
<1> ;;  procedures ans 'swap page que' procedures will not be active...
<1> ;;  'Insufficient memory' error will be returned by 'swap_out'
<1> ;;  and 'general protection fault' will be returned by 'swap_in'
<1> ;;  procedure, if it is called mistakenly (a wrong value in a PTE).
<1> ;;
<1>
<1> ; 17/04/2021
<1> ; ('swap_in' procedure call is disabled as temporary)
<1>
<1> swap_in:
<1>     ; 31/08/2015
<1>     ; 20/07/2015
<1>     ; 28/04/2015
<1>     ; 18/04/2015
<1>     ; 24/10/2014 (Retro UNIX 386 v1 - beginning)
<1>     ;
<1>     ; INPUT ->
<1>     ;       EBX = PHYSICAL (real/flat) ADDRESS OF THE MEMORY PAGE
<1>     ;       EBP = VIRTUAL (LINEAR) ADDRESS (page fault address)
<1>     ;       EAX = Offset Address for the swapped page on the
<1>     ;             swap disk or in the swap file.
<1>     ;
<1>     ; OUTPUT ->
<1>     ;       EAX = 0 if loading at memory has been successful
<1>     ;
```

```
<1>     ;       CF = 1 -> swap disk reading error (disk/file not present
<1>     ;                 or sector not present or drive not ready
<1>     ;            EAX = Error code
<1>     ;            [u.error] = EAX
<1>     ;                      = The last error code for the process
<1>     ;                        (will be reset after returning to user)
<1>     ;
<1>     ; Modified Registers -> EAX
<1>     ;
<1>
<1> ;       cmp     dword [swp_drv], 0
<1> ;   jna     short swpin_dnp_err
<1> ;
<1> ;   cmp     eax, [swpd_size]
<1> ;   jnb     short swpin_snp_err
<1> ;
<1> ;   push    esi
<1> ;   push    ebx
<1> ;   push    ecx
<1> ;   mov     esi, [swp_drv]
<1> ;   mov     ecx, PAGE_SIZE / LOGIC_SECT_SIZE  ; 8 !
<1> ;           ; Note: Even if corresponding physical disk's sector
<1> ;           ; size different than 512 bytes, logical disk sector
<1> ;           ; size is 512 bytes and disk reading procedure
<1> ;           ; will be performed for reading 4096 bytes
<1> ;           ; (2*2048, 8*512).
<1> ;   ; ESI = Logical disk description table address
<1> ;   ; EBX = Memory page (buffer) address (physical!)
<1> ;   ; EAX = Sector adress (offset address, logical sector number)
<1> ;   ; ECX = Sector count ; 8 sectors
<1> ;   push    eax
<1> ;   call    logical_disk_read
<1> ;   pop     eax
<1> ;   jnc     short swpin_read_ok
<1> ;   ;
<1> ;   mov     eax, SWP_DISK_READ_ERR ; drive not ready or read error
<1> ;   mov     [u.error], eax
<1> ;   jmp     short swpin_retn
<1> ;   ;
<1> ;swpin_read_ok:
<1> ;   ; EAX = Offset address (logical sector number)
<1> ;   call    unlink_swap_block  ; Deallocate swap block
<1> ;   ;
<1> ;   ; EBX = Memory page (buffer) address (physical!)
<1> ;   ; 20/07/2015
<1> ;   mov     ebx, ebp ; virtual address (page fault address)
<1> ;       and     bx, ~PAGE_OFF ; ~0FFFh ; reset bits, 0 to 11
<1> ;   mov     bl, [u.uno] ; current process number
<1> ;   ; EBX = Virtual (Linear) address & process number combination
<1> ;   call    swap_queue_shift
<1> ;   ; eax = 0 ; 10/06/2016 (if ebx input > 0, eax output = 0)
<1> ;   ;sub    eax, eax  ; 0 ; Error Code = 0  (no error)
<1> ;   ; zf = 1
<1> ;swpin_retn:
<1> ;   pop     ecx
<1> ;   pop     ebx
<1> ;   pop     esi
<1> ;   retn
<1> ;
<1> ;swpin_dnp_err:
<1> ;   mov     eax, SWP_DISK_NOT_PRESENT_ERR
<1> ;swpin_err_retn:
<1> ;   mov     [u.error], eax
<1> ;   stc
<1> ;   retn
<1> ;
<1> ;swpin_snp_err:
<1> ;   mov     eax, SWP_SECTOR_NOT_PRESENT_ERR
<1> ;   jmp     short swpin_err_retn
<1>
<1> ; 17/04/2021
<1> ; ('swap_out' procedure call is disabled as temporary)
<1>
<1> swap_out:
<1>     ; 10/06/2016
<1>     ; 07/06/2016
<1>         ; 23/05/2016
<1>     ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
<1>     ; 24/10/2014 - 31/08/2015 (Retro UNIX 386 v1)
<1>     ;
<1>     ; INPUT ->
<1>     ;       none
<1>     ;
<1>     ; OUTPUT ->
<1>     ;       EAX = Physical page address (which is swapped out
<1>     ;             for allocating a new page)
<1>     ;       CF = 1 -> swap disk writing error (disk/file not present
<1>     ;                 or sector not present or drive not ready
<1>     ;            EAX = Error code
<1>     ;            [u.error] = EAX
<1>     ;                      = The last error code for the process
<1>     ;                        (will be reset after returning to user)
<1>     ;
<1>     ; Modified Registers -> none (except EAX)
<1>     ;
<1>
<1> ;   cmp     word [swpq_count], 1
<1> ;       jc      swpout_im_err ; 'insufficient memory'
<1> ;
<1> ;       ;cmp    dword [swp_drv], 1
<1> ;   ;jc     short swpout_dnp_err ; 'swap disk/file not present'
<1> ;
<1> ;       cmp     dword [swpd_free], 1
<1> ;       jc      swpout_nfspc_err ; 'no free space on swap disk'
<1> ;
<1> ;   push    ebx ; *
<1> ;swpout_1:
<1> ;   ; 10/06/2016
<1> ;   xor     ebx, ebx ; shift the queue and return a PTE value
<1> ;   call    swap_queue_shift
<1> ;   and     eax, eax        ; 0 = empty queue (improper entries)
<1> ;       jz      swpout_npts_err         ; There is not any proper PTE
<1> ;                                       ; pointer in the swap queue
<1> ;   ; EAX = PTE value of the page
<1> ;   ; EBX = PTE address of the page
<1> ;   and     ax, PTE_A_CLEAR ; 0F000h ; clear attribute bits
<1> ;   ;
<1> ;   ; 07/06/2016
<1> ;   ; 19/05/2016
<1> ;   ; check this page is in timer events or not
<1> ;
<1> ;swpout_timer_page_0:
<1> ;   push    edx ; **
<1> ;
```

```
<1> ;   ; 07/06/2016
<1> ;   cmp     byte [timer_events], 0
<1> ;   jna     short swpout_2
<1> ;   ;
<1> ;   mov     dl, [timer_events]
<1> ;
<1> ;   push    ecx ; ***
<1> ;   push    ebx ; ****
<1> ;   mov     ebx, timer_set ; beginning address of timer event
<1> ;                          ; structures
<1> ;swpout_timer_page_1:
<1> ;   mov     cl, [ebx]
<1> ;   or      cl, cl ; 0 = free, >0 = process number
<1> ;   jz      short swpout_timer_page_3
<1> ;   mov     ecx, [ebx+12] ; response (signal return) address
<1> ;   and     cx, PTE_A_CLEAR ; clear offset part (right 12 bits)
<1> ;                           ; of the response byte address, to
<1> ;                           ; get beginning of the page address)
<1> ;   cmp     eax, ecx
<1> ;   jne     short swpout_timer_page_2 ; not same page
<1> ;
<1> ;   ; !same page!
<1> ;   ;
<1> ;   ; NOTE: // 19/05/2016 // - TRDOS 386 feature only ! -
<1> ;   ; This page will be used by the kernel to put timer event
<1> ;   ; response (signal return) byte at the requested address;
<1> ;   ; in order to prevent a possible wrong write (while
<1> ;   ; this page is swapped out) on physical memory,
<1> ;   ; we must protect this page against to be swapped out!
<1> ;   ;
<1> ;   pop     ebx ; ****
<1> ;   pop     ecx ; ***
<1> ;   pop     edx ; **
<1> ;   jmp     short swpout_1 ; do not swap out this page !
<1> ;
<1> ;swpout_timer_page_2:
<1> ;   ; 07/06/2016
<1> ;   dec     dl
<1> ;   jz      short swpout_timer_page_4
<1> ;swpout_timer_page_3:
<1> ;   ;cmp    ebx, timer_set + 240 ; last timer event (15*16)
<1> ;   ;jnb    short swpout_timer_page_4
<1> ;   add     ebx, 16
<1> ;   jmp     short swpout_timer_page_1
<1> ;
<1> ;swpout_timer_page_4:
<1> ;   pop     ebx ; ****
<1> ;   pop     ecx ; ***
<1> ;swpout_2:
<1> ;   mov     edx, ebx              ; Page table entry address
<1> ;   mov     ebx, eax              ; Buffer (Page) Address
<1> ;   ;
<1> ;   call    link_swap_block
<1> ;   jnc     short swpout_3        ; It may not be needed here
<1> ;                                 ; because [swpd_free] value
<1> ;                                 ; was checked at the beginging.
<1> ;   pop     edx ; **
<1> ;   pop     ebx ; *
<1> ;   jmp     short swpout_nfspc_err
<1> ;swpout_3:
<1> ;   test    eax, 80000000h ; test bit 31 (this may not be needed!)
<1> ;   jnz     short swpout_nfspc_err  ; 10/06/2016 (bit 31 = 1 !)
<1> ;   ;
<1> ;   push    esi ; **
<1> ;   push    ecx ; ***
<1> ;   push    eax ; sector address ; (31 bit !, bit 31 = 0)
<1> ;   mov     esi, [swp_drv]
<1> ;   mov     ecx, PAGE_SIZE / LOGIC_SECT_SIZE  ; 8 !
<1> ;           ; Note: Even if corresponding physical disk's sector
<1> ;           ; size different than 512 bytes, logical disk sector
<1> ;           ; size is 512 bytes and disk writing procedure
<1> ;           ; will be performed for writing 4096 bytes
<1> ;           ; (2*2048, 8*512).
<1> ;   ; ESI = Logical disk description table address
<1> ;   ; EBX = Buffer (Page) address
<1> ;   ; EAX = Sector adress (offset address, logical sector number)
<1> ;   ; ECX = Sector count ; 8 sectors
<1> ;   ; edx = PTE address
<1> ;   call    logical_disk_write
<1> ;   ; edx = PTE address
<1> ;   pop     ecx ; sector address
<1> ;   jnc     short swpout_write_ok
<1> ;   ;
<1> ;   ;; call unlink_swap_block ; this block must be left as 'in use'
<1> ;swpout_dw_err:
<1> ;   mov     eax, SWP_DISK_WRITE_ERR ; drive not ready or write error
<1> ;   mov     [u.error], eax
<1> ;   jmp     short swpout_retn
<1> ;   ;
<1> ;swpout_write_ok:
<1> ;   ; EBX = Buffer (page) address
<1> ;   ; EDX = Page Table Entry address
<1> ;   ; ECX = Swap disk sector (file block) address (31 bit)
<1> ;   shl     ecx, 1  ; 31 bit sector address from bit 1 to bit 31
<1> ;   mov     [edx], ecx
<1> ;           ; bit 0 = 0 (swapped page)
<1> ;   mov     eax, ebx
<1> ;swpout_retn:
<1> ;   pop     ecx ; ***
<1> ;   pop     esi ; **
<1> ;   pop     ebx ; *
<1> ;   retn
<1> ;
<1> ;;swpout_dnp_err:
<1> ;;  mov     eax, SWP_DISK_NOT_PRESENT_ERR ; disk not present
<1> ;;  jmp     short swpout_err_retn
<1> ;swpout_nfspc_err:
<1> ;   mov     eax, SWP_NO_FREE_SPACE_ERR ; no free space
<1> ;swpout_err_retn:
<1> ;   mov     [u.error], eax
<1> ;   ;stc
<1> ;   retn
<1> ;swpout_npts_err:
<1> ;   mov     eax, SWP_NO_PAGE_TO_SWAP_ERR
<1> ;   pop     ebx
<1> ;   jmp     short swpout_err_retn
<1> ;swpout_im_err:
<1> ;   mov     eax, ERR_MINOR_IM ; insufficient (out of) memory
<1> ;   jmp     short swpout_err_retn
<1>
<1> ; 17/04/2021
<1> ; ('swap_queue_shift' procedure call is disabled as temporary)
<1>
<1> swap_queue_shift:
```

```
<1>     ; 26/03/2017
<1>     ; 10/06/2016
<1>     ; 09/06/2016 - TRDOS 386 (TRDOS v2.0)
<1>     ; 23/10/2014 - 20/07/2015 (Retro UNIX 386 v1)
<1>     ;
<1>     ; INPUT ->
<1>     ;      EBX = Virtual (linear) address (bit 12 to 31)
<1>     ;            and process number combination (bit 0 to 11)
<1>     ;      EBX = 0 -> shift/drop from the head (offset 0)
<1>     ;
<1>     ; OUTPUT ->
<1>     ;      If EBX input > 0
<1>     ;         the queue will be shifted 4 bytes (dword),
<1>     ;         from the tail to the head, up to entry offset
<1>     ;         which points to EBX input value or nothing
<1>     ;         to do if EBX value is not found on the queue.
<1>     ;         (The entry -with EBX value- will be removed
<1>     ;         from the queue if it is found.)
<1>     ;
<1>     ;         EAX = 0
<1>     ;
<1>     ;      If EBX input = 0
<1>     ;         the queue will be shifted 4 bytes (dword),
<1>     ;         from the tail to the head, if the PTE address
<1>     ;         which is pointed in head of the queue is marked
<1>     ;         as "accessed" or it is marked as "non present".
<1>     ;         (If "accessed" flag of the PTE -which is pointed
<1>     ;         in the head- is set -to 1-, it will be reset
<1>     ;         -to 0- and then, the queue will be rotated
<1>     ;         -without dropping pointer of the PTE from
<1>     ;         the queue- for 4 bytes on head to tail direction.
<1>     ;         Pointer in the head will be moved into the tail,
<1>     ;         other PTEs will be shifted on head direction.)
<1>     ;
<1>     ;         Swap queue will be shifted up to the first
<1>     ;         'present' or 'non accessed' page will be found
<1>     ;         (as pointed) on the queue head (then it will be
<1>       ;           removed/dropped from the queue).
<1>     ;
<1>     ;         EAX (> 0) = PTE value of the page which is
<1>     ;               (it's pointer -virtual address-) dropped
<1>     ;               (removed) from swap queue.
<1>     ;         EBX = PTE address of the page (if EAX > 0)
<1>     ;               which is (it's pointer -virtual address-)
<1>     ;               dropped (removed) from swap queue.
<1>     ;
<1>     ;         EAX = 0 -> empty swap queue !
<1>     ;
<1>     ; Modified Registers -> EAX, EBX
<1>     ;
<1> ;   movzx   eax, word [swpq_count]  ; Max. 1024
<1> ;   and     ax, ax
<1> ;   jz      short swpqs_retn
<1> ;   push    edi
<1> ;   push    esi
<1> ;   push    ecx
<1> ;   mov     esi, swap_queue
<1> ;   mov     ecx, eax
<1> ;   or      ebx, ebx
<1> ;   jz      short swpqs_7
<1> ;swpqs_1:
<1> ;   lodsd
<1> ;   cmp     eax, ebx
<1> ;   je      short swpqs_2
<1> ;   loop    swpqs_1
<1> ;   ; 10/06/2016
<1> ;   sub     eax, eax
<1> ;   jmp     short swpqs_6
<1> ;swpqs_2:
<1> ;   mov     edi, esi
<1> ;   sub     edi, 4
<1> ;swpqs_3:
<1> ;   dec     word [swpq_count]
<1> ;   jz      short swpqs_5
<1> ;swpqs_4:
<1> ;   dec     ecx
<1> ;   rep     movsd   ; shift up (to the head)
<1> ;swpqs_5:
<1> ;   xor     eax, eax
<1> ;   mov     [edi], eax
<1> ;swpqs_6:
<1> ;   pop     ecx
<1> ;   pop     esi
<1> ;   pop     edi
<1> ;swpqs_retn:
<1> ;   retn
<1> ;swpqs_7:
<1> ;   mov     edi, esi ; head
<1> ;   lodsd
<1> ;   ; 20/07/2015
<1> ;   mov     ebx, eax
<1> ;   and     ebx, ~PAGE_OFF ; ~0FFFh
<1> ;                  ; ebx = virtual address (at page boundary)
<1> ;   and     eax, PAGE_OFF ; 0FFFh
<1> ;                  ; ax = process number (1 to 4095)
<1> ;   cmp     al, [u.uno]
<1> ;           ; Max. 16 (nproc) processes for Retro UNIX 386 v1
<1> ;   jne     short swpqs_8
<1> ;   mov     eax, [u.pgdir]
<1> ;   jmp     short swpqs_9
<1> ;swpqs_8:
<1> ;   ; 09/06/2016
<1> ;   cmp     byte [eax+p.stat-1], 0
<1> ;   jna     short swpqs_3     ; free (or terminated) process
<1> ;   cmp     byte [eax+p.stat-1], 2 ; waiting
<1> ;   ja      short swpqs_3    ; zombie (3) or undefined ?
<1> ;
<1> ;   ;shl    ax, 2
<1> ;   shl     al, 2
<1> ;   mov     eax, [eax+p.upage-4]
<1> ;   or      eax, eax
<1> ;   jz      short swpqs_3 ; invalid upage
<1> ;   add     eax, u.pgdir - user
<1> ;                   ; u.pgdir value for the process
<1> ;                   ; is in [eax]
<1> ;   mov     eax, [eax]
<1> ;   and     eax, eax
<1> ;   jz      short swpqs_3 ; invalid page directory
<1> ;swpqs_9:
<1> ;   push    edx
<1> ;   ; eax = page directory
<1> ;   ; ebx = virtual address
<1> ;   call    get_pte
<1> ;   mov     ebx, edx       ; PTE address
```

```
<1> ;   pop     edx
<1> ;   ; 10/06/2016
<1> ;   jc      short swpqs_13 ; empty PDE
<1> ;   ; EAX = PTE value
<1> ;   test    al, PTE_A_PRESENT ; bit 0 = 1
<1> ;   jz      short swpqs_13  ; Drop non-present page
<1> ;                           ; from the queue (head)
<1> ;   test    al, PTE_A_WRITE       ; bit 1 = 0 (read only)
<1> ;   jz      short swpqs_13  ; Drop read only page
<1> ;                           ; from the queue (head)
<1> ;   ;test   al, PTE_A_ACCESS ; bit 5 = 1 (Accessed)
<1> ;   ;jnz    short swpqs_11  ; present
<1> ;                           ; accessed page
<1> ;       btr     eax, PTE_A_ACCESS_BIT ; reset 'accessed' bit
<1> ;   jc      short swpqs_11  ; accessed page
<1> ;
<1> ;   dec     ecx
<1> ;   mov     [swpq_count], cx
<1> ;       jz      short swpqs_10
<1> ;           ; esi = head + 4
<1> ;           ; edi = head
<1> ;   rep     movsd   ; n = 1 to k-1, [n - 1] = [n]
<1> ;swpqs_10:
<1> ;   mov     [edi], ecx ; 0
<1> ;   jmp     short swpqs_6 ; 26/03/2017
<1> ;
<1> ;swpqs_11:
<1> ;   mov     [ebx], eax      ; save changed attribute
<1> ;   ; Rotation (head -> tail)
<1> ;   dec     ecx     ; entry count -> last entry number
<1> ;   jz      short swpqs_10
<1> ;           ; esi = head + 4
<1> ;           ; edi = head
<1> ;   mov     eax, [edi] ; 20/07/2015
<1> ;   rep     movsd   ; n = 1 to k-1, [n - 1] = [n]
<1> ;   mov     [edi], eax ; head -> tail ; [k] = [1]
<1> ;
<1> ;   mov     cx, [swpq_count]
<1> ;
<1> ;swpqs_12:
<1> ;   mov     esi, swap_queue ; head
<1> ;       jmp     swpqs_7
<1> ;
<1> ;swpqs_13:
<1> ;   dec     ecx
<1> ;   mov     [swpq_count], cx
<1> ;       jz      swpqs_5
<1> ;   jmp     short swpqs_12
<1>
<1> ; 17/04/2021
<1> ; ('add_to_swp_queue' procedure call is disabled as temporary)
<1>
<1> add_to_swap_queue:
<1>     ; 20/02/2017
<1>     ; 20/07/2015
<1>     ; 24/10/2014 (Retro UNIX 386 v1 - beginning)
<1>     ;
<1>     ; Adds new page to swap queue
<1>     ; (page directories and page tables must not be added
<1>     ; to swap queue)
<1>     ;
<1>     ; INPUT ->
<1>     ;       EBX = Linear (Virtual) addr for current process
<1>     ;       [u.uno]
<1>     ;       20/02/2017
<1>     ;       (Linear address = CORE + user's virtual address)
<1>     ;
<1>     ; OUTPUT ->
<1>     ;       EAX = [swpq_count]
<1>     ;             (after the PTE has been added)
<1>     ;       EAX = 0 -> Swap queue is full, (1024 entries)
<1>     ;             the PTE could not be added.
<1>     ;
<1>     ; Modified Registers -> EAX
<1>     ;
<1> ;   push    ebx
<1> ;       and     bx, ~PAGE_OFF ; ~0FFFh ; reset bits, 0 to 11
<1> ;   mov     bl, [u.uno] ; current process number
<1> ;   call    swap_queue_shift ; drop from the queue if
<1> ;                            ; it is already on the queue
<1> ;           ; then add it to the tail of the queue
<1> ;   movzx   eax, word [swpq_count]
<1> ;   cmp     ax, 1024
<1> ;   jb      short atsq_1
<1> ;   sub     ax, ax
<1> ;   pop     ebx
<1> ;   retn
<1> ;atsq_1:
<1> ;   push    esi
<1> ;   mov     esi, swap_queue
<1> ;   and     ax, ax
<1> ;   jz      short atsq_2
<1> ;   shl     ax, 2   ; convert to offset
<1> ;   add     esi, eax
<1> ;   shr     ax, 2
<1> ;atsq_2:
<1> ;   inc     ax
<1> ;   mov     [esi], ebx ; Virtual address + [u.uno] combination
<1> ;   mov     [swpq_count], ax
<1> ;   pop     esi
<1> ;   pop     ebx
<1> ;   retn
<1>
<1> ; 17/04/2021
<1> ; ('unlink_swap_block' procedure call is disabled as temporary)
<1>
<1> unlink_swap_block:
<1>     ; 15/09/2015
<1>     ; 30/04/2015
<1>     ; 18/04/2015
<1>     ; 24/10/2014 (Retro UNIX 386 v1 - beginning)
<1>     ;
<1>     ; INPUT ->
<1>     ;       EAX = swap disk/file offset address
<1>     ;             (bit 1 to bit 31)
<1>     ; OUTPUT ->
<1>     ;       [swpd_free] is increased
<1>     ;       (corresponding SWAP DISK ALLOC. TABLE bit is SET)
<1>     ;
<1>     ; Modified Registers -> EAX
<1>     ;
<1> ;   push    ebx
<1> ;   push    edx
<1> ;   ;
```

```
<1> ;   shr     eax, SECTOR_SHIFT+1  ;3+1 ; shift sector address to
<1> ;                                ; 3 bits right
<1> ;                                ; to get swap block/page number
<1> ;   mov     edx, eax
<1> ;   ; 15/09/2015
<1> ;   shr     edx, 3               ; to get offset to S.A.T.
<1> ;                                ; (1 allocation bit = 1 page)
<1> ;                                ; (1 allocation bytes = 8 pages)
<1> ;   and     dl, 0FCh             ; clear lower 2 bits
<1> ;                                ; (to get 32 bit position)
<1> ;   ;
<1> ;   mov     ebx, swap_alloc_table ; Swap Allocation Table address
<1> ;   add     ebx, edx
<1> ;   and     eax, 1Fh             ; lower 5 bits only
<1> ;                                ; (allocation bit position)
<1> ;   cmp     eax, [swpd_next]      ; is the new free block addr. lower
<1> ;                                ; than the address in 'swpd_next' ?
<1> ;                                ; (next/first free block value)
<1> ;   jnb     short uswpbl_1       ; no
<1> ;   mov     [swpd_next], eax      ; yes
<1> ;uswpbl_1:
<1> ;   bts     [ebx], eax           ; unlink/release/deallocate block
<1> ;                                ; set relevant bit to 1.
<1> ;                                ; set CF to the previous bit value
<1> ;   cmc                          ; complement carry flag
<1> ;   jc      short uswpbl_2       ; do not increase swfd_free count
<1> ;                                ; if the block is already deallocated
<1> ;                                ; before.
<1> ;       inc     dword [swpd_free]
<1> ;uswpbl_2:
<1> ;   pop     edx
<1> ;   pop     ebx
<1> ;   retn
<1>
<1> ; 17/04/2021
<1> ; ('ink_swap_block' procedure call is disabled as temporary)
<1>
<1> link_swap_block:
<1>     ; 01/07/2015
<1>     ; 18/04/2015
<1>     ; 24/10/2014 (Retro UNIX 386 v1 - beginning)
<1>     ;
<1>     ; INPUT -> none
<1>     ;
<1>     ; OUTPUT ->
<1>     ;       EAX = OFFSET ADDRESS OF THE ALLOCATED BLOCK (4096 bytes)
<1>     ;             in sectors (corresponding
<1>     ;             SWAP DISK ALLOCATION TABLE bit is RESET)
<1>     ;
<1>     ;       CF = 1 and EAX = 0
<1>     ;                  if there is not a free block to be allocated
<1>     ;
<1>     ; Modified Registers -> none (except EAX)
<1>     ;
<1>
<1> ;   ;mov    eax, [swpd_free]
<1> ;   ;and    eax, eax
<1> ;   ;jz     short out_of_swpspc
<1> ;   ;
<1> ;   push    ebx
<1> ;   push    ecx
<1> ;   ;
<1> ;   mov     ebx, swap_alloc_table ; Swap Allocation Table offset
<1> ;   mov     ecx, ebx
<1> ;   add     ebx, [swpd_next] ; Free block searching starts from here
<1> ;                            ; next_free_swap_block >> 5
<1> ;   add     ecx, [swpd_last] ; Free block searching ends here
<1> ;                            ; (total_swap_blocks - 1) >> 5
<1> ;lswbl_scan:
<1> ;   cmp     ebx, ecx
<1> ;   ja      short lswbl_notfound
<1> ;   ;
<1> ;   bsf     eax, [ebx] ; Scans source operand for first bit set (1).
<1> ;                      ; Clears ZF if a bit is found set (1) and
<1> ;                      ; loads the destination with an index to
<1> ;                      ; first set bit. (0 -> 31)
<1> ;                      ; Sets ZF to 1 if no bits are found set.
<1> ;   ; 01/07/2015
<1> ;   jnz     short lswbl_found ; ZF = 0 -> a free block has been found
<1> ;                     ;
<1> ;                     ; NOTE:  a Swap Disk Allocation Table bit
<1> ;                     ;       with value of 1 means
<1> ;                     ;       the corresponding page is free
<1> ;                     ;       (Retro UNIX 386 v1 feaure only!)
<1> ;   add     ebx, 4
<1> ;                     ; We return back for searching next page block
<1> ;                     ; NOTE: [swpd_free] is not ZERO; so,
<1> ;                     ;      we always will find at least 1 free block here.
<1> ;   jmp     short lswbl_scan
<1> ;   ;
<1> ;lswbl_notfound:
<1> ;   sub     ecx, swap_alloc_table
<1> ;   mov     [swpd_next], ecx ; next/first free page = last page
<1> ;                            ; (unlink_swap_block procedure will change it)
<1> ;   xor     eax, eax
<1> ;   mov     [swpd_free], eax
<1> ;   stc
<1> ;lswbl_ok:
<1> ;   pop     ecx
<1> ;   pop     ebx
<1> ;   retn
<1> ;   ;
<1> ;;out_of_swpspc:
<1> ;;  stc
<1> ;;  retn
<1> ;
<1> ;lswbl_found:
<1> ;   mov     ecx, ebx
<1> ;   sub     ecx, swap_alloc_table
<1> ;   mov     [swpd_next], ecx ; Set first free block searching start
<1> ;                            ; address/offset (to the next)
<1> ;       dec     dword [swpd_free] ; 1 block has been allocated (X = X-1)
<1> ;   ;
<1> ;   btr     [ebx], eax       ; The destination bit indexed by the source value
<1> ;                            ; is copied into the Carry Flag and then cleared
<1> ;                            ; in the destination.
<1> ;                            ;
<1> ;                            ; Reset the bit which is corresponding to the
<1> ;                            ; (just) allocated block.
<1> ;   shl     ecx, 5           ; (block offset * 32) + block index
<1> ;   add     eax, ecx         ; = block number
<1> ;   shl     eax, SECTOR_SHIFT ; 3, sector (offset) address of the block
<1> ;                            ; 1 block =  8 sectors
<1> ;   ;
```

```
                                <1> ;   ; EAX = offset address of swap disk/file sector (beginning of the block)
                                <1> ;   ;
                                <1> ;   ; NOTE: The relevant page table entry will be updated
                                <1> ;   ;       according to this EAX value...
                                <1> ;   ;
                                <1> ;   jmp    short lswbl_ok
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('logical_disk_read' procedure call is disabled as temporary)
                                <1>
                                <1> logical_disk_read:
                                <1>     ; 20/07/2015
                                <1>     ; 09/03/2015 (temporary code here)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      ESI = Logical disk description table address
                                <1>     ;      EBX = Memory page (buffer) address (physical!)
                                <1>     ;      EAX = Sector adress (offset address, logical sector number)
                                <1>     ;      ECX = Sector count
                                <1>     ;
                                <1>     ;
                                <1> ;   retn
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('logical_disk_write' procedure call is disabled as temporary)
                                <1>
                                <1> logical_disk_write:
                                <1>     ; 20/07/2015
                                <1>     ; 09/03/2015 (temporary code here)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      ESI = Logical disk description table address
                                <1>     ;      EBX = Memory page (buffer) address (physical!)
                                <1>     ;      EAX = Sector adress (offset address, logical sector number)
                                <1>     ;      ECX = Sector count
                                <1>     ;
                                <1> ;   retn
                                <1>
                                <1> get_physical_addr:
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;      (temporary modifications)
                                <1>     ;
                                <1>     ; 26/03/2017
                                <1>     ; 20/02/2017
                                <1>     ; 27/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 18/10/2015
                                <1>     ; 29/07/2015
                                <1>     ; 20/07/2015
                                <1>     ; 04/06/2015
                                <1>     ; 20/05/2015
                                <1>     ; 28/04/2015
                                <1>     ; 18/04/2015
                                <1>     ; Get physical address
                                <1>     ;     (allocates a new page for user if it is not present)
                                <1>     ;
                                <1>     ; (This subroutine is needed for mapping user's virtual
                                <1>     ; (buffer) address to physical address (of the buffer).)
                                <1>     ; ('sys write', 'sys read' system calls...)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      EBX = virtual address
                                <1>     ;      u.pgdir = page directory (physical) address
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      EAX = physical address
                                <1>     ;      EBX = linear address
                                <1>     ;      EDX = physical address of the page frame
                                <1>     ;            (with attribute bits)
                                <1>     ;      ECX = byte count within the page frame
                                <1>     ;
                                <1>     ; Modified Registers -> EAX, EBX, ECX, EDX
                                <1>     ;
00005C10 81C300004000           <1>     add    ebx, CORE ; 18/10/2015
                                <1> get_physical_addr_x: ; 27/05/2016
00005C16 A1[C48E0100]           <1>     mov    eax, [u.pgdir]
00005C1B E81CFDFFFF             <1>     call   get_pte
                                <1>            ; EDX = Page table entry address (if CF=0)
                                <1>            ;       Page directory entry address (if CF=1)
                                <1>            ;       (Bit 0 value is 0 if PT is not present)
                                <1>            ; EAX = Page table entry value (page address)
                                <1>            ;       CF = 1 -> PDE not present or invalid ?
00005C20 731C                   <1>     jnc    short gpa_1
                                <1>     ;
00005C22 E803FCFFFF             <1>     call   allocate_page
00005C27 723F                   <1>     jc     short gpa_im_err  ; 'insufficient memory' error
                                <1> gpa_0:
00005C29 E86DFCFFFF             <1>     call   clear_page
                                <1>     ; EAX = Physical (base) address of the allocated (new) page
00005C2E 0C07                   <1>     or     al, PDE_A_PRESENT + PDE_A_WRITE + PDE_A_USER ; 4+2+1 = 7
                                <1>                          ; lower 3 bits are used as U/S, R/W, P flags
                                <1>                          ; (user, writable, present page)
00005C30 8902                   <1>     mov    [edx], eax ; Let's put the new page directory entry here !
00005C32 A1[C48E0100]           <1>     mov    eax, [u.pgdir]
00005C37 E800FDFFFF             <1>     call   get_pte
00005C3C 722A                   <1>     jc     short gpa_im_err ; 'insufficient memory' error
                                <1> gpa_1:
                                <1>     ; EAX = PTE value, EDX = PTE address
00005C3E A801                   <1>     test   al, PTE_A_PRESENT
00005C40 7516                   <1>     jnz    short gpa_3 ; 26/03/2017
00005C42 09C0                   <1>     or     eax, eax
00005C44 7446                   <1>     jz     short gpa_7  ; Allocate a new page
                                <1>
                                <1> ; 17/04/2021 (TRDOS 386 v2.0.4)
                                <1> ; ('reload_page' procedure call is disabled as temporary)
00005C46 EB20                   <1>     jmp    short gpa_im_err  ; temporary !
                                <1>
                                <1>     ; 20/07/2015
                                <1> ;   push   ebp
                                <1> ;   mov    ebp, ebx ; virtual (linear) address
                                <1> ;   ; reload swapped page
                                <1> ;   call   reload_page ; 28/04/2015
                                <1> ;   pop    ebp
                                <1> ;   jc     short gpa_retn
                                <1> gpa_2:
                                <1>     ; 26/03/2017
                                <1>     ; 20/02/2017
                                <1>     ; If a page will contain a Signal Response Byte
                                <1>     ; it must not be swapped out, because
                                <1>     ; timer service or irq callback service
                                <1>     ; will write a signal return/response byte
                                <1>     ; directly by using physical address of Signal
                                <1>     ; Response Byte.(Even if process is not running,
                                <1>     ; or it is running with swapped out pages.)
                                <1>     ;
```

```
                                <1>     ; 'no_page_swap' will be set by 'systimer' or
                                <1>     ; 'syscalbac' sistem functions/calls. (*)
                                <1>     ;
00005C48 803D[52880100]00       <1>     cmp     byte [no_page_swap], 0
00005C4F 761D                   <1>     jna     short gpa_4 ; this page can be swapped out
                                <1>     ; this page must not be swapped out
                                <1>     ; but 'no_page_swap' must be reset here
                                <1>     ; immediately for other callers (*)
                                <1>     ; (otherwise, swap queue would not be long enough)
00005C51 E844000000             <1>     call    gpa_8 ; 26/03/2017
00005C56 EB16                   <1>     jmp     short gpa_5
                                <1> gpa_3:
                                <1>     ; 26/03/2017
00005C58 803D[52880100]00       <1>     cmp     byte [no_page_swap], 0
00005C5F 7611                   <1>     jna     short gpa_6 ; this page can be swapped out
00005C61 E834000000             <1>     call    gpa_8
00005C66 EB0A                   <1>     jmp     short gpa_6
                                <1>
                                <1> gpa_im_err:
00005C68 B804000000             <1>     mov     eax, ERR_MINOR_IM ; Insufficient memory (minor) error!
                                <1>                               ; Major error = 0 (No protection fault)
00005C6D C3                     <1>     retn
                                <1> gpa_4:
                                <1> ; 17/04/2021 (TRDOS 386 v2.0.4)
                                <1> ; ('add_to_swap_queue' procedure call is disabled as temporary)
                                <1>
                                <1>     ; 20/07/2015
                                <1>     ; 20/05/2015
                                <1>     ; add this page to swap queue
                                <1> ;   push    eax
                                <1> ;   ; EBX = Linear (CORE+virtual) address ; 20/02/2017
                                <1> ;   call    add_to_swap_queue
                                <1> ;   pop     eax
                                <1> gpa_5:
                                <1>             ; PTE address in EDX
                                <1>             ; virtual address in EBX
                                <1>     ; EAX = memory page address
00005C6E 0C07                   <1>     or      al, PTE_A_PRESENT + PTE_A_USER + PTE_A_WRITE
                                <1>                               ; present flag, bit 0 = 1
                                <1>                               ; user flag, bit 2 = 1
                                <1>                               ; writable flag, bit 1 = 1
00005C70 8902                   <1>     mov     [edx], eax  ; Update PTE value
                                <1> gpa_6:
                                <1>     ; 18/10/2015
00005C72 89D9                   <1>     mov     ecx, ebx
00005C74 81E1FF0F0000           <1>     and     ecx, PAGE_OFF
00005C7A 89C2                   <1>     mov     edx, eax
00005C7C 662500F0               <1>     and     ax, PTE_A_CLEAR
00005C80 01C8                   <1>     add     eax, ecx
00005C82 F7D9                   <1>     neg     ecx ; 1 -> -1 (0FFFFFFFFh), 4095 (0FFFh) -> -4095
00005C84 81C100100000           <1>     add     ecx, PAGE_SIZE
00005C8A F8                     <1>     clc
                                <1> gpa_retn:
00005C8B C3                     <1>     retn
                                <1> gpa_7:
00005C8C E899FBFFFF             <1>     call    allocate_page
00005C91 72D5                   <1>     jc      short gpa_im_err ; 'insufficient memory' error
00005C93 E803FCFFFF             <1>     call    clear_page
00005C98 EBAE                   <1>     jmp     short gpa_2
                                <1>
                                <1> gpa_8: ; 26/03/2017
00005C9A C605[52880100]00       <1>     mov     byte [no_page_swap], 0
                                <1>
00005CA1 C3                     <1>     retn    ; 17/04/2021 (temporary)
                                <1>
                                <1> ; 17/04/2021 (TRDOS 386 v2.0.4)
                                <1> ; ('swap_queue_shift' procedure call is disabled as temporary)
                                <1> ;
                                <1> ;   push    ebx
                                <1> ;   push    eax ; 26/03/2017
                                <1> ;       and     bx, ~PAGE_OFF ; ~0FFFh ; reset bits, 0 to 11
                                <1> ;   mov     bl, [u.uno] ; current process number
                                <1> ;   call    swap_queue_shift ; drop from the queue if
                                <1> ;                           ; it is already on the queue
                                <1> ;   pop     eax ; 26/03/2017
                                <1> ;   pop     ebx
                                <1> ;
                                <1> ;   retn
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('reload_page' procedure call is disabled as temporary)
                                <1>
                                <1> reload_page:
                                <1>     ; 20/07/2015
                                <1>     ; 28/04/2015 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; Reload (Restore) swapped page at memory
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EBP = Virtual (linear) memory address
                                <1>     ;       EAX = PTE value (swap disk sector address)
                                <1>     ;       (Swap disk sector address = bit 1 to bit 31 of EAX)
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS OF RELOADED PAGE
                                <1>     ;
                                <1>     ;       CF = 1 and EAX = error code
                                <1>     ;
                                <1>     ; Modified Registers -> none (except EAX)
                                <1>     ;
                                <1> ;   shr     eax, 1   ; Convert PTE value to swap disk address
                                <1> ;   push    ebx      ;
                                <1> ;   mov     ebx, eax ; Swap disk (offset) address
                                <1> ;   call    allocate_page
                                <1> ;   jc      short rlp_im_err
                                <1> ;   xchg    eax, ebx
                                <1> ;   ; EBX = Physical memory (page) address
                                <1> ;   ; EAX = Swap disk (offset) address
                                <1> ;   ; EBP = Virtual (linear) memory address
                                <1> ;   call    swap_in
                                <1> ;   jc      short rlp_swp_err  ; (swap disk/file read error)
                                <1> ;   mov     eax, ebx
                                <1> ;rlp_retn:
                                <1> ;   pop     ebx
                                <1> ;   retn
                                <1> ;
                                <1> ;rlp_im_err:
                                <1> ;   mov     eax, ERR_MINOR_IM ; Insufficient memory (minor) error!
                                <1> ;                             ; Major error = 0 (No protection fault)
                                <1> ;   jmp     short rlp_retn
                                <1> ;
                                <1> ;rlp_swp_err:
                                <1> ;   mov     eax, SWP_DISK_READ_ERR ; Swap disk read error !
                                <1> ;   jmp     short rlp_retn
                                <1>
```

```
                                <1> copy_page_dir:
                                <1>     ; 17/04/2021 (temporary modifications)
                                <1>     ; 19/09/2015
                                <1>     ; temporary - 07/09/2015
                                <1>     ; 07/09/2015 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       [u.pgdir] = PHYSICAL (real/flat) ADDRESS of the parent's
                                <1>     ;                   page directory.
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX =  PHYSICAL (real/flat) ADDRESS of the child's
                                <1>     ;              page directory.
                                <1>     ;       (New page directory with new page table entries.)
                                <1>     ;       (New page tables with read only copies of the parent's
                                <1>     ;       pages.)
                                <1>     ;       EAX = 0 -> Error (CF = 1)
                                <1>     ;
                                <1>     ; Modified Registers -> none (except EAX)
                                <1>     ;
00005CA2 E883FBFFFF             <1>     call    allocate_page
00005CA7 723E                   <1>     jc      short cpd_err
                                <1>     ;
00005CA9 55                     <1>     push    ebp ; 20/07/2015
00005CAA 56                     <1>     push    esi
00005CAB 57                     <1>     push    edi
00005CAC 53                     <1>     push    ebx
00005CAD 51                     <1>     push    ecx
00005CAE 8B35[C48E0100]         <1>     mov     esi, [u.pgdir]
00005CB4 89C7                   <1>     mov     edi, eax
00005CB6 50                     <1>     push    eax ; save child's page directory address
                                <1>     ; copy PDE 0 from the parent's page dir to the child's page dir
                                <1>     ; (use same system space for all user page tables)
00005CB7 A5                     <1>     movsd
00005CB8 BD00004000             <1>     mov     ebp, 1024*4096 ; pass the 1st 4MB (system space)
00005CBD B9FF030000             <1>     mov     ecx, (PAGE_SIZE / 4) - 1 ; 1023
                                <1> cpd_0:
00005CC2 AD                     <1>     lodsd
                                <1>     ;or     eax, eax
                                <1>         ;jnz     short cpd_1
00005CC3 A801                   <1>     test    al, PDE_A_PRESENT ;  bit 0 =  1
00005CC5 7508                   <1>     jnz     short cpd_1
                                <1>     ; (virtual address at the end of the page table)
00005CC7 81C500004000           <1>     add     ebp, 1024*4096 ; page size * PTE count
00005CCD EB0F                   <1>     jmp     short cpd_2
                                <1> cpd_1:
00005CCF 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear attribute bits
00005CD3 89C3                   <1>     mov     ebx, eax
                                <1>     ; EBX = Parent's page table address
00005CD5 E81F000000             <1>     call    copy_page_table
00005CDA 720C                   <1>     jc      short cpd_p_err
                                <1>     ; EAX = Child's page table address
00005CDC 0C07                   <1>     or      al, PDE_A_PRESENT + PDE_A_WRITE + PDE_A_USER
                                <1>                     ; set bit 0, bit 1 and bit 2 to 1
                                <1>                     ; (present, writable, user)
                                <1> cpd_2:
00005CDE AB                     <1>     stosd
00005CDF E2E1                   <1>     loop    cpd_0
                                <1>     ;
00005CE1 58                     <1>     pop     eax  ; restore child's page directory address
                                <1> cpd_3:
00005CE2 59                     <1>     pop     ecx
00005CE3 5B                     <1>     pop     ebx
00005CE4 5F                     <1>     pop     edi
00005CE5 5E                     <1>     pop     esi
00005CE6 5D                     <1>     pop     ebp
                                <1> cpd_err:
00005CE7 C3                     <1>     retn
                                <1> cpd_p_err:
                                <1>     ; release the allocated pages missing (recover free space)
00005CE8 58                     <1>     pop     eax  ; the new page directory address (physical)
00005CE9 8B1D[C48E0100]         <1>     mov     ebx, [u.pgdir] ; parent's page directory address
00005CEF E866FCFFFF             <1>     call    deallocate_page_dir
00005CF4 29C0                   <1>     sub     eax, eax ; 0
00005CF6 F9                     <1>     stc
00005CF7 EBE9                   <1>     jmp     short cpd_3
                                <1>
                                <1> copy_page_table:
                                <1>     ; 17/04/2021 (temporary modifications)
                                <1>     ; 19/09/2015
                                <1>     ; temporary - 07/09/2015
                                <1>     ; 07/09/2015 (Retro UNIX 386 v1 - beginning)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EBX = PHYSICAL (real/flat) ADDRESS of the parent's page table.
                                <1>     ;       EBP = page table entry index (from 'copy_page_dir')
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = PHYSICAL (real/flat) ADDRESS of the child's page table.
                                <1>     ;       EBP = (recent) page table index (for 'add_to_swap_queue')
                                <1>     ;       CF = 1 -> error
                                <1>     ;
                                <1>     ; Modified Registers -> EBP (except EAX)
                                <1>     ;
00005CF9 E82CFBFFFF             <1>     call    allocate_page
00005CFE 7244                   <1>     jc      short cpt_err
                                <1>     ;
00005D00 50                     <1>     push    eax ; *
                                <1>     ;push   ebx
00005D01 56                     <1>     push    esi
00005D02 57                     <1>     push    edi
00005D03 52                     <1>     push    edx
00005D04 51                     <1>     push    ecx
                                <1>     ;
00005D05 89DE                   <1>     mov     esi, ebx
00005D07 89C7                   <1>     mov     edi, eax
00005D09 89C2                   <1>     mov     edx, eax
00005D0B 81C200100000           <1>     add     edx, PAGE_SIZE
                                <1> cpt_0:
00005D11 AD                     <1>     lodsd
00005D12 A801                   <1>     test    al, PTE_A_PRESENT ;  bit 0 = 1
                                <1>     ;jnz    short cpt_1 (*)
                                <1>     ; 17/04/2021 (temporary (*)
                                <1>     ;and    eax, eax (*)
00005D14 741E                   <1>     jz      short cpt_2  ; 17/04/2021
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('reload_page' procedure call is disabled as temporary)
                                <1> ;
                                <1> ;   ; ebp = virtual (linear) address of the memory page
                                <1> ;   call    reload_page ; 28/04/2015
                                <1> ;   jc      short cpt_p_err
                                <1> cpt_1:
00005D16 662500F0               <1>     and     ax, PTE_A_CLEAR ; 0F000h ; clear attribute bits
00005D1A 89C1                   <1>     mov     ecx, eax
                                <1>     ; Allocate a new page for the child process
```

```
00005D1C E809FBFFFF          <1>     call    allocate_page
00005D21 721C                <1>     jc      short cpt_p_err
00005D23 57                  <1>     push    edi
00005D24 56                  <1>     push    esi
00005D25 89CE                <1>     mov     esi, ecx
00005D27 89C7                <1>     mov     edi, eax
00005D29 B900040000          <1>     mov     ecx, PAGE_SIZE/4
00005D2E F3A5                <1>     rep     movsd   ; copy page (4096 bytes)
00005D30 5E                  <1>     pop     esi
00005D31 5F                  <1>     pop     edi
                             <1>     ;
                             <1>
                             <1> ; 17/04/2021
                             <1> ; ('add_to_swap_queue' procedure call is disabled as temporary)
                             <1> ;
                             <1> ;   push    ebx
                             <1> ;   push    eax
                             <1> ;   mov     ebx, ebp
                             <1> ;   ; ebx = virtual address of the memory page
                             <1> ;   call    add_to_swap_queue
                             <1> ;   pop     eax
                             <1> ;   pop     ebx
                             <1>     ;
                             <1>     ;or     ax, PTE_A_USER+PTE_A_PRESENT
00005D32 0C07                <1>     or      al, PTE_A_USER+PTE_A_WRITE+PTE_A_PRESENT
                             <1> cpt_2:
00005D34 AB                  <1>     stosd  ; EDI points to child's PTE
                             <1>     ;
00005D35 81C500100000        <1>     add     ebp, 4096 ; 20/07/2015 (next page)
                             <1>     ;
00005D3B 39D7                <1>     cmp     edi, edx
00005D3D 72D2                <1>     jb      short cpt_0
                             <1> cpt_p_err:
00005D3F 59                  <1>     pop     ecx
00005D40 5A                  <1>     pop     edx
00005D41 5F                  <1>     pop     edi
00005D42 5E                  <1>     pop     esi
                             <1>     ;pop    ebx
00005D43 58                  <1>     pop     eax ; *
                             <1> cpt_err:
00005D44 C3                  <1>     retn
                             <1>
                             <1> allocate_memory_block:
                             <1>     ; 01/05/2017
                             <1>     ; 28/04/2017
                             <1>     ; 25/04/2017
                             <1>     ; 01/04/2016, 02/04/2016, 03/04/2016
                             <1>     ; 13/03/2016, 14/03/2016
                             <1>     ; 12/03/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; Allocating contiguous memory pages (in the kernel's memory space)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       EAX = Beginning address (physical)
                             <1>     ;       EAX = 0 -> Allocate memory block from the first proper aperture
                             <1>     ;       ECX = Number of bytes to be allocated
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;       1) cf = 0 -> successful
                             <1>     ;       EAX = Beginning (physical) address of the allocated memory block
                             <1>     ;       ECX = Number of allocated bytes (rounded up to page borders)
                             <1>     ;       2) cf = 1 -> unsuccessful
                             <1>     ;        2.1) If EAX > 0 ->
                             <1>     ;             (Number of requested pages is more than # of free pages
                             <1>     ;              but contiguous free pages -the aperture- is not enough!)
                             <1>     ;             EAX = Beginning address of available aperture
                             <1>     ;                   (one of all aperture with max. aperture size/length)
                             <1>     ;             ECX = Size of available aperture (memory block) in bytes
                             <1>     ;        2.2) If EAX = 0 -> Out of memory error
                             <1>     ;                   (number of free pages is less than requested number)
                             <1>     ;             ECX = Total number of free bytes (free pages * 4096)
                             <1>     ;                   (It is not number of contiguous free bytes)
                             <1>     ;
                             <1>     ; (Modified Registers -> EAX, ECX)
                             <1>     ;
                             <1>     ; PURPOSE: Loading a file at memory for copying or running etc.
                             <1>     ; If this procedure returns with cf is set, ECX contains maximum
                             <1>     ; available space and EAX contains the beginning address of it.
                             <1>     ; If EAX has zero, ECX contains total number of free bytes.
                             <1>     ; If requested block has been successfully allocated (by rounding up to
                             <1>     ; the last page border), it must be deallocated later by using
                             <1>     ; 'deallocate_memory_block' procedure.
                             <1>
00005D45 52                  <1>     push    edx ; *
00005D46 BAFF0F0000          <1>     mov     edx, PAGE_SIZE - 1   ; 4095
00005D4B 01D0                <1>     add     eax, edx
00005D4D 01D1                <1>     add     ecx, edx
00005D4F C1E90C              <1>     shr     ecx, PAGE_SHIFT            ; 12
                             <1>
                             <1>     ; ECX = number of contiguous pages to be allocated
00005D52 8B15[B8760100]      <1>     mov     edx, [free_pages]
                             <1>     ; 01/05/2017
                             <1>     ;or     ecx, ecx
                             <1>     ;jz     short amb3
                             <1>     ; If ECX=0, set cf to 1 and return with max. available mem block size
                             <1>
00005D58 39D1                <1>     cmp     ecx, edx
00005D5A 7760                <1>     ja      short amb_3
                             <1>
00005D5C C1E80C              <1>     shr     eax, PAGE_SHIFT      ; 12
                             <1>
00005D5F 89C2                <1>     mov     edx, eax            ; page number
00005D61 C1EA03              <1>     shr     edx, 3              ; to get offset to M.A.T.
                             <1>                                 ; (1 allocation bit = 1 page)
                             <1>                                 ; (1 allocation bytes = 8 pages)
00005D64 80E2FC              <1>     and     dl, 0FCh            ; clear lower 2 bits
                             <1>                                 ; (to get 32 bit position)
00005D67 53                  <1>     push    ebx ; **
                             <1> amb_0:
00005D68 890D[70820100]      <1>     mov     [mem_ipg_count], ecx ; initial (reset) value of page count
00005D6E 890D[74820100]      <1>     mov     [mem_pg_count], ecx
00005D74 31C9                <1>     xor     ecx, ecx ; 0
00005D76 890D[78820100]      <1>     mov     [mem_aperture], ecx ; 0
00005D7C 890D[7C820100]      <1>     mov     [mem_max_aperture], ecx ; 0
                             <1>
00005D82 BB00001000          <1>     mov     ebx, MEM_ALLOC_TBL   ; Memory Allocation Table address.
00005D87 3B15[BC760100]      <1>     cmp     edx, [next_page]     ; Is the beginning page address lower
                             <1>                                 ; than the address in 'next_page' ?
                             <1>                                 ; (the first/next free page of user space)
00005D8D 7208                <1>     jb      short amb_1
00005D8F 3B15[C0760100]      <1>     cmp     edx, [last_page]     ; is the beginning page address higher
                             <1>                                 ; than the address in 'last_page' ?
                             <1>                                 ; (end of the memory)
00005D95 7606                <1>     jna     short amb_2         ; no
                             <1> amb_1:
```

```
00005D97 8B15[BC760100]      <1>     mov     edx, [next_page]     ; M.A.T. offset (1 M.A.T. byte = 8 pages)
                             <1> amb_2:
00005D9D 01D3                <1>     add     ebx, edx
                             <1>
                             <1>     ; 28/04/2017
                             <1>     ;xor    ecx, ecx
00005D9F 0FBC0B              <1>     bsf     ecx, [ebx]          ; 0 to 31
00005DA2 89D0                <1>     mov     eax, edx
00005DA4 C1E003              <1>     shl     eax, 3              ; *8
00005DA7 01C8                <1>     add     eax, ecx            ; beginning page number
                             <1>
00005DA9 A3[80820100]        <1>     mov     [mem_pg_pos], eax    ; beginning page no (for curr. mem. aperture)
00005DAE A3[84820100]        <1>     mov     [mem_max_pg_pos], eax ; beginning page no for max. mem. aperture
                             <1>
00005DB3 83E01F              <1>     and     eax, 1Fh            ; lower 5 bits only (0 to 31)
                             <1>                                 ; (allocation bit position)
00005DB6 750E                <1>     jnz     short amb_4         ; 0
00005DB8 B120                <1>     mov     cl, 32
00005DBA EB4B                <1>     jmp     short amb_10
                             <1>
                             <1> amb_3:      ; out_of_memory
00005DBC 31C0                <1>     xor     eax, eax ; 0
00005DBE 89D1                <1>     mov     ecx, edx ; free pages
00005DC0 C1E10C              <1>     shl     ecx, PAGE_SHIFT
00005DC3 5A                  <1>     pop     edx ; *
00005DC4 F9                  <1>     stc
00005DC5 C3                  <1>     retn
                             <1> amb_4:
00005DC6 8B13                <1>     mov     edx, [ebx]
00005DC8 88C1                <1>     mov     cl, al ; 1 to 31
00005DCA D3EA                <1>     shr     edx, cl
00005DCC 89D0                <1>     mov     eax, edx
                             <1> amb_5:
00005DCE D1E8                <1>     shr     eax, 1 ; (***)
00005DD0 7317                <1>     jnc     short amb_7
00005DD2 FF05[78820100]      <1>     inc     dword [mem_aperture]
00005DD8 FF0D[74820100]      <1>     dec     dword [mem_pg_count]
00005DDE 747F                <1>     jz      short amb_15
                             <1> amb_6:
                             <1>     ; 28/04/2017
00005DE0 FEC1                <1>     inc     cl
00005DE2 80F920              <1>     cmp     cl, 32
00005DE5 730D                <1>     jnb     short amb_9
00005DE7 EBE5                <1>     jmp     short amb_5
                             <1> amb_7:
00005DE9 50                  <1>     push    eax ; (***) allocation bits (in shifted status)
00005DEA E828010000          <1>     call    amb_26 ; set maximum memory aperture (free memory block size)
00005DEF 58                  <1>     pop     eax ; (***)
00005DF0 EBEE                <1>     jmp     short amb_6
                             <1> amb_8:
                             <1>     ; 28/04/2017
00005DF2 B120                <1>     mov     cl, 32
                             <1> amb_9:
00005DF4 89DA                <1>     mov     edx, ebx
00005DF6 81EA00001000        <1>     sub     edx, MEM_ALLOC_TBL
00005DFC 3B15[C0760100]      <1>     cmp     edx, [last_page]
00005E02 7336                <1>     jnb     short amb_14 ; contiguous pages not enough
00005E04 83C304              <1>     add     ebx, 4
                             <1> amb_10:
00005E07 8B03                <1>     mov     eax, [ebx]
00005E09 21C0                <1>     and     eax, eax
00005E0B 7408                <1>         jz      short amb_11 ; there is not a free page bit in this alloc dword
00005E0D 40                  <1>     inc     eax ; 0FFFFFFFFh -> 0
00005E0E 740C                <1>     jz      short amb_12 ; all of bits are set (32 free pages)
00005E10 48                  <1>     dec     eax
00005E11 28C9                <1>     sub     cl, cl ; 0
00005E13 EBB9                <1>     jmp     short amb_5
                             <1> amb_11:
00005E15 E8FD000000          <1>     call    amb_26 ; set maximum memory aperture (free memory block size)
00005E1A EBD8                <1>     jmp     short amb_9
                             <1> amb_12:
00005E1C 390D[74820100]      <1>     cmp     [mem_pg_count], ecx ; 32
00005E22 7306                <1>     jnb     short amb_13
00005E24 8B0D[74820100]      <1>     mov     ecx, [mem_pg_count]
                             <1> amb_13:
00005E2A 010D[78820100]      <1>     add     [mem_aperture], ecx
00005E30 290D[74820100]      <1>     sub     [mem_pg_count], ecx
00005E36 7627                <1>     jna     short amb_15
00005E38 EBBA                <1>     jmp     short amb_9 ; 01/05/2017
                             <1> amb_14:
00005E3A E8D8000000          <1>     call    amb_26 ; 28/04/2017
00005E3F A1[84820100]        <1>     mov     eax, [mem_max_pg_pos] ; begin address of max. mem aperture
00005E44 8B0D[7C820100]      <1>     mov     ecx, [mem_max_aperture] ; max. (largest) memory aperture
00005E4A F9                  <1>     stc
00005E4B E9BC000000          <1>         jmp     amb_25
                             <1>
                             <1> allocate_lfb_memory_block:
                             <1>     ; 28/11/2023
                             <1>     ; 20/10/2023 - TRDOS 386 v2.0.7
                             <1>     ; (short way to set the LFB page bits on the M.A.T.)
                             <1>     ; called from 'allocate_lfb_pages_for_kernel'
00005E50 A3[80820100]        <1>     mov     [mem_pg_pos], eax    ; LFB start/base address as page number
00005E55 890D[78820100]      <1>     mov     [mem_aperture], ecx  ; number of LFB pages
                             <1>                                  ; (!which overlapping main memory!)
00005E5B 52                  <1>     push    edx ; *
00005E5C 53                  <1>     push    ebx ; **
00005E5D EB0B                <1>     jmp     short amb_16
                             <1>
                             <1> amb_15: ; OK !
00005E5F A1[80820100]        <1>     mov     eax, [mem_pg_pos]    ; Beginning address as page number
00005E64 8B0D[78820100]      <1>     mov     ecx, [mem_aperture]  ; Free contiguous page count (>=1)
                             <1> amb_16:
                             <1>     ; allocate contiguous memory pages (via memory allocation table bits)
00005E6A 89C2                <1>     mov     edx, eax
                             <1>     ; 25/04/2017
00005E6C C1EA03              <1>     shr     edx, 3          ; 8 pages in one allocation byte
00005E6F 80E2FC              <1>     and     dl, 0FCh        ; clear lower 2 bits
                             <1>                             ; (for dword/32bit positioning)
                             <1>
00005E72 BB00001000          <1>     mov     ebx, MEM_ALLOC_TBL
00005E77 01D3                <1>     add     ebx, edx
00005E79 83E01F              <1>     and     eax, 1Fh ; 31
                             <1>     ; 03/04/2016
00005E7C BA20000000          <1>     mov     edx, 32
00005E81 28C2                <1>     sub     dl, al
00005E83 39CA                <1>     cmp     edx, ecx        ; ecx >= 1
00005E85 7602                <1>     jna     short amb_17
00005E87 89CA                <1>     mov     edx, ecx
                             <1> amb_17:
00005E89 29D1                <1>     sub     ecx, edx
00005E8B 51                  <1>     push    ecx ; ***
00005E8C 89D1                <1>     mov     ecx, edx
                             <1> amb_18:
00005E8E 0FB303              <1>     btr     [ebx], eax      ; The destination bit indexed by the source value
```

```
                                <1>                             ; is copied into the Carry Flag and then cleared
                                <1>                             ; in the destination.
                                <1>     ; 20/10/2023
                                <1>     ; (for LFB page allocation..
                                <1>     ;  -here, to prevent using LFB pages for another allocation later-
                                <1>     ;  /// the destination bit must be 0 -already- ..
                                <1>     ;  so, following 'jnc' is not necessary but i am not removing it
                                <1>     ;  for understanding why free page count is decreased here.)
                                <1>     ;
                                <1>     ;jnc   short amb_28 ; requested page is already allocated
                                <1>
00005E91 FF0D[B8760100]         <1>     dec     dword [free_pages] ; 1 page has been allocated (X = X-1)
                                <1> ;amb_28:    ; 20/10/2023
00005E97 49                     <1>     dec     ecx
00005E98 7404                   <1>     jz      short amb_19
00005E9A FEC0                   <1>     inc     al
00005E9C EBF0                   <1>     jmp     short amb_18
                                <1> amb_19:
00005E9E 59                     <1>     pop     ecx ; ***
                                <1>     ;and    ecx, ecx ; 0 ?
                                <1>     ;jz     short amb_22
                                <1>     ; 28/11/2023
00005E9F E31E                   <1>     jecxz   amb_22
                                <1>     ; 01/04/2016
00005EA1 B020                   <1>     mov     al, 32
                                <1> amb_20:
00005EA3 83C304                 <1>     add     ebx, 4
00005EA6 39C1                   <1>     cmp     ecx, eax ; 32
00005EA8 7305                   <1>     jnb     short amb_21
                                <1>     ; ECX < 32
00005EAA 28C0                   <1>     sub     al, al ; 0
00005EAC 50                     <1>     push    eax ; 0 ***
00005EAD EBDF                   <1>     jmp     short amb_18
                                <1> amb_21:
00005EAF 2905[B8760100]         <1>     sub     [free_pages], eax   ; [free_pages] = [free_pages] - 32
00005EB5 C70300000000           <1>     mov     dword [ebx], 0     ; reset 32 bits
00005EBB 29C1                   <1>     sub     ecx, eax ; 32
00005EBD 75E4                   <1>     jnz     short amb_20
                                <1> amb_22:
00005EBF A1[80820100]           <1>     mov     eax, [mem_pg_pos]   ; Beginning address as page number
00005EC4 8B0D[78820100]         <1>     mov     ecx, [mem_aperture] ; Free contiguous page count
                                <1>     ; [next_page] update
00005ECA 89C2                   <1>     mov     edx, eax
                                <1>     ; 03/04/2016
00005ECC C1EA03                 <1>     shr     edx, 3              ; to get offset to M.A.T.
                                <1>                                 ; (1 allocation bit = 1 page)
                                <1>                                 ; (1 allocation bytes = 8 pages)
00005ECF 80E2FC                 <1>     and     dl, 0FCh            ; clear lower 2 bits
                                <1>                                 ; (to get 32 bit position)
00005ED2 3B15[BC760100]         <1>     cmp     edx, [next_page] ; first free page pointer offset
00005ED8 7732                   <1>     ja      short amb_25
00005EDA BB00001000             <1>     mov     ebx, MEM_ALLOC_TBL
00005EDF 833C1300               <1>     cmp     dword [ebx+edx], 0
00005EE3 7721                   <1>     ja      short amb_24
00005EE5 89C2                   <1>     mov     edx, eax
00005EE7 01CA                   <1>     add     edx, ecx
00005EE9 C1EA03                 <1>     shr     edx, 3
00005EEC 80E2FC                 <1>     and     dl, 0FCh
                                <1> amb_23:
00005EEF 833C1300               <1>     cmp     dword [ebx+edx], 0
00005EF3 7711                   <1>     ja      short amb_24
00005EF5 83C204                 <1>     add     edx, 4
00005EF8 3B15[C0760100]         <1>     cmp     edx, [last_page]    ; last page pointer offset
00005EFE 76EF                   <1>     jna     short amb_23
00005F00 8B15[C4760100]         <1>     mov     edx, [first_page]   ; (for) beginning of user's space
                                <1> amb_24:
00005F06 8915[BC760100]         <1>     mov     [next_page], edx
                                <1> amb_25:
00005F0C 9C                     <1>     pushf
00005F0D C1E00C                 <1>     shl     eax, PAGE_SHIFT            ; convert to phy. address in bytes
00005F10 C1E10C                 <1>     shl     ecx, PAGE_SHIFT            ; convert to byte counts
00005F13 9D                     <1>     popf
00005F14 5B                     <1>     pop     ebx ; **
00005F15 5A                     <1>     pop     edx ; *
00005F16 C3                     <1>     retn
                                <1>
                                <1> amb_26:     ; set maximum free memory aperture (free memory block size)
00005F17 89DA                   <1>     mov     edx, ebx ; current address
00005F19 81EA00001000           <1>     sub     edx, MEM_ALLOC_TBL ; MAT beginning address
                                <1>     ; 02/04/2016
00005F1F C1E203                 <1>     shl     edx, 3 ; MAT byte offset * 8 = page number base
00005F22 01CA                   <1>     add     edx, ecx ; current page number (ecx = 0 to 32)
                                <1>     ;
00005F24 A1[78820100]           <1>     mov     eax, [mem_aperture]
00005F29 21C0                   <1>     and     eax, eax
00005F2B 7421                   <1>         jz      short amb_27
00005F2D C705[78820100]0000-    <1>         mov     dword [mem_aperture], 0
00005F35 0000                   <1>
00005F37 3B05[7C820100]         <1>     cmp     eax, [mem_max_aperture]
00005F3D 760F                   <1>     jna     short amb_27
00005F3F A3[7C820100]           <1>     mov     [mem_max_aperture], eax
                                <1>     ; 25/04/2017
00005F44 A1[80820100]           <1>     mov     eax, [mem_pg_pos]
                                <1>     ; EAX = Beginning page number of the max. aperture
00005F49 A3[84820100]           <1>     mov     [mem_max_pg_pos], eax
                                <1> amb_27:
00005F4E 8915[80820100]         <1>     mov     [mem_pg_pos], edx ; current page
                                <1>
00005F54 A1[70820100]           <1>     mov     eax, [mem_ipg_count] ; initial (reset) value of page count
00005F59 A3[74820100]           <1>     mov     [mem_pg_count], eax
                                <1>
00005F5E C3                     <1>     retn
                                <1>
                                <1> ; 04/06/2024
                                <1> %if 0
                                <1> deallocate_memory_block_x:
                                <1>     ; 26/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; deallocate pages after rounding up
                                <1>     ; ((audio buffer size etc. may not be rounded up to page boundary))
                                <1>     add     ecx, PAGE_SIZE - 1   ; 4095
                                <1> %endif
                                <1>
                                <1> deallocate_memory_block:
                                <1>     ; 03/04/2016
                                <1>     ; 14/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; Deallocating contiguous memory pages (in the kernel's memory space)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Beginning address (physical)
                                <1>     ;       ECX = Number of bytes to be deallocated
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       Memory Allocation Table bits will be updated
                                <1>     ;       [free_pages] will be changed (increased)
```

```
                                <1>     ;
                                <1>     ; (Modified Registers -> EAX, ECX)
                                <1>     ;
                                <1>     ; PURPOSE: Unloading/Freeing a file -or an allocated memory block-
                                <1>     ; at memory after copying, running, saving, reading, writing etc.
                                <1>     ;
                                <1>
00005F5F 52                     <1>     push    edx ; *
00005F60 53                     <1>     push    ebx ; **
                                <1>
00005F61 C1E80C                 <1>     shr     eax, PAGE_SHIFT             ; 12
00005F64 C1E90C                 <1>     shr     ecx, PAGE_SHIFT             ; 12
                                <1>
                                <1>     ; EAX = Beginning page number
                                <1>     ; ECX = Number of contiguous pages to be deallocated
                                <1> damb_0:
                                <1>     ; deallocate contiguous memory pages (via memory allocation table bits)
00005F67 89C2                   <1>     mov     edx, eax
00005F69 C1EA03                 <1>     shr     edx, 3              ; to get offset to M.A.T.
                                <1>                                 ; (1 allocation bit = 1 page)
                                <1>                                 ; (1 allocation bytes = 8 pages)
00005F6C 80E2FC                 <1>     and     dl, 0FCh            ; clear lower 2 bits
                                <1>                                 ; (to get 32 bit position)
00005F6F 3B15[BC760100]         <1>     cmp     edx, [next_page] ; next free page
00005F75 7306                   <1>     jnb     short damb_1
00005F77 8915[BC760100]         <1>     mov     [next_page], edx
                                <1> damb_1:
00005F7D BB00001000             <1>     mov     ebx, MEM_ALLOC_TBL
00005F82 01D3                   <1>     add     ebx, edx
00005F84 83E01F                 <1>     and     eax, 1Fh ; 31
                                <1>
                                <1>     ; 03/04/2016
00005F87 BA20000000             <1>     mov     edx, 32
00005F8C 28C2                   <1>     sub     dl, al
00005F8E 39CA                   <1>     cmp     edx, ecx
00005F90 7602                   <1>     jna     short damb_2
00005F92 89CA                   <1>     mov     edx, ecx
                                <1> damb_2:
00005F94 29D1                   <1>     sub     ecx, edx
00005F96 51                     <1>     push    ecx ; ***
00005F97 89D1                   <1>     mov     ecx, edx
                                <1> damb_3:
00005F99 0FAB03                 <1>     bts     [ebx], eax          ; unlink/release/deallocate page
                                <1>                                 ; set relevant bit to 1.
                                <1>                                 ; set CF to the previous bit value
00005F9C FF05[B8760100]         <1>     inc     dword [free_pages]   ; 1 page has been deallocated (X = X+1)
00005FA2 49                     <1>     dec     ecx
00005FA3 7404                   <1>     jz      short damb_4
00005FA5 FEC0                   <1>     inc     al
00005FA7 EBF0                   <1>     jmp     short damb_3
                                <1> damb_4:
00005FA9 59                     <1>     pop     ecx ; ***
00005FAA 21C9                   <1>     and     ecx, ecx ; 0 ?
00005FAC 741E                   <1>     jz      short damb_7
                                <1>     ; 03/04/2016
00005FAE B020                   <1>     mov     al, 32
                                <1> damb_5:
00005FB0 83C304                 <1>     add     ebx, 4
00005FB3 39C1                   <1>     cmp     ecx, eax ; 32
00005FB5 7305                   <1>     jnb     short damb_6
                                <1>     ; ECX < 32
00005FB7 28C0                   <1>     sub     al, al ; 0
00005FB9 50                     <1>     push    eax ; 0 ***
00005FBA EBDD                   <1>     jmp     short damb_3
                                <1> damb_6:
00005FBC 0105[B8760100]         <1>     add     [free_pages], eax ; [free_pages] = [free_pages] + 32
00005FC2 C703FFFFFFFF           <1>     mov     dword [ebx], 0FFFFFFFFh ; set 32 bits
00005FC8 29C1                   <1>     sub     ecx, eax ; 32
00005FCA 75E4                   <1>     jnz     short damb_5
                                <1> damb_7:
00005FCC 5B                     <1>     pop     ebx ; **
00005FCD 5A                     <1>     pop     edx ; *
00005FCE C3                     <1>     retn
                                <1>
                                <1> direct_memory_access:
                                <1>     ; 20/10/2023
                                <1>     ; 17/04/2021 (temporary modifications)
                                <1>     ; 22/07/2017
                                <1>     ; 12/05/2017
                                <1>     ; 16/07/2016
                                <1>     ; 12/07/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; This processure will be called to map
                                <1>     ; user's (ring 3) page tables to access phsical
                                <1>     ; (flat/linear) memory addresses, directly (without
                                <1>     ;  kernel's data transfer functions).
                                <1>     ;
                                <1>     ; Purpose: Video memory access and shared memory access.
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Beginning address (physical).
                                <1>     ;       EBX = User's buffer address ; 12/05/2017
                                <1>     ;       ECX = Number of contiguous pages to be mapped.
                                <1>     ; OUTPUT ->
                                <1>     ;       User's page directory and pages tables
                                <1>     ;       will be updated.
                                <1>     ;
                                <1>     ;       If an old page table entry has valid page address,
                                <1>     ;       that page will be deallocated just before PTE will
                                <1>     ;       be changed for direct (1 to 1) memory page access.
                                <1>     ;
                                <1>     ;       If old PTE value points to a swapped page,
                                <1>       ;       that page (block) will be unlinked on swap disk.
                                <1>     ;
                                <1>     ;       Newly allocated pages (except page tables) will not
                                <1>     ;       be applied to Memory Allocation Table.
                                <1>     ;       AVL bit 1 (PTE bit 10) of page table entry will be
                                <1>     ;       used to indicate shared (direct) memory page; then,
                                <1>     ;       this page will not be deallocated later during
                                <1>     ;       process termination. (Memory Allocation Table and
                                <1>     ;       free memory count will not be affected.
                                <1>     ;       (Except deallocating page table's itself.)
                                <1>     ;
                                <1>     ;       CF = 1 -> error (EAX = error code)
                                <1>     ;       CF = 0 -> success (EAX = beginning address)
                                <1>     ;
                                <1>     ;; (Modified Registers -> none)
                                <1>     ; Modified registers: ebp, edx, ecx, ebx, esi, edi
                                <1>     ;
                                <1>
                                <1>     ;push   ebp
                                <1>     ;push   ebx
                                <1>     ;push   ecx
                                <1>     ;push   edx
00005FCF 662500F0               <1>     and     ax, PTE_A_CLEAR ; clear page offset
```

```
00005FD3 50                 <1>     push    eax
                            <1>     ;and    ecx, ecx ; page count
                            <1>     ;jz     dmem_acc_7  ; 'insufficient memory' error
00005FD4 89C5               <1>     mov     ebp, eax
00005FD6 81C300004000       <1>     add     ebx, CORE ; 12/05/2017
                            <1> dmem_acc_0:
00005FDC 891D[288D0100]     <1>     mov     [base_addr], ebx ; 12/05/2017
00005FE2 A1[C48E0100]       <1>     mov     eax, [u.pgdir] ; page dir address (physical)
00005FE7 E850F9FFFF         <1>     call    get_pte
                            <1>             ; EDX = Page table entry address (if CF=0)
                            <1>             ;       Page directory entry address (if CF=1)
                            <1>             ;       (Bit 0 value is 0 if PT is not present)
                            <1>             ; EAX = Page table entry value (page address)
                            <1>             ;       CF = 1 -> PDE not present or invalid ?
00005FEC 7321               <1>     jnc     short dmem_acc_1
                            <1>     ; 20/10/2023
                            <1>     ; Allocate a page as a new page table
00005FEE E837F8FFFF         <1>     call    allocate_page
                            <1>     ;jc     dmem_acc_7  ; 'insufficient memory' error
                            <1>     ; 17/04/2021
00005FF3 7215               <1>     jc      short _dmem_acc_7
                            <1>     ;
00005FF5 E8A1F8FFFF         <1>     call    clear_page
                            <1>     ; EAX = Physical (base) address of the allocated (new) page
00005FFA 0C07               <1>     or      al, PDE_A_PRESENT + PDE_A_WRITE + PDE_A_USER ; 4+2+1 = 7
                            <1>                         ; lower 3 bits are used as U/S, R/W, P flags
                            <1>                         ; (user, writable, present page)
00005FFC 8902               <1>     mov     [edx], eax ; Let's put the new page directory entry here !
00005FFE A1[C48E0100]       <1>     mov     eax, [u.pgdir]
00006003 E834F9FFFF         <1>     call    get_pte
                            <1>         ;jc dmem_acc_7 ; 'insufficient memory' error
                            <1>     ; 17/04/2021
00006008 7305               <1>     jnc     short dmem_acc_1
                            <1> _dmem_acc_7:
0000600A E985000000         <1>     jmp     dmem_acc_7
                            <1> dmem_acc_1:
                            <1>     ; EAX = PTE value, EDX = PTE address
0000600F A801               <1>     test    al, PTE_A_PRESENT
                            <1>     ;jnz    short dmem_acc_2   ; 17/04/2021 (*)
                            <1>     ; 17/04/2021 (temporary)
00006011 745F               <1>     jz      short short dmem_acc_6  ; ! temporary ! (*)
                            <1>     ; 20/10/2023
                            <1>     ; If the page table entry is zero or invalid -not present-
                            <1>     ; it will be directy allocated for the physical memory section
                            <1>     ; without using the M.A.T. (Specially for LFB access -generally-
                            <1>     ; is at the beyod of the physical main memory end.)
                            <1>     ; (M.A.T bits are not used for direct memory access. To cancel
                            <1>     ; direct memory access also is out of the M.A.T scope because
                            <1>     ; direct accessed physical pages are marked as SHARED on their
                            <1>     ; PTE. Shared pages are not dealloaced.)
                            <1>
                            <1> ; 17/04/2021
                            <1> ; (following code is disabled as temporary)
                            <1> ;
                            <1> ;   or      eax, eax
                            <1> ;   jz      short dmem_acc_6   ; Change PTE
                            <1> ;   shr     eax, 1         ; swap disk block (8 sectors) address
                            <1> ;   ; unlink swap disk block
                            <1> ;   call    unlink_swap_block
                            <1> ;   jmp     short dmem_acc_6
                            <1>
                            <1> dmem_acc_2:
00006013 A802               <1>     test    al, PTE_A_WRITE   ; bit 1, writable (r/w) flag
                            <1>                             ; (must be 1)
00006015 7550               <1>     jnz     short dmem_acc_4
                            <1>     ; Read only -duplicated- page (belongs to a parent or a child)
00006017 66A90002           <1>         test    ax, PTE_DUPLICATED ; Was this page duplicated
                            <1>                             ; as child's page ?
0000601B 7455               <1>     jz      short dmem_acc_5 ; Change PTE but don't deallocate the page!
                            <1>
                            <1>     ;push   edi
                            <1>     ;push   esi
                            <1>
0000601D 51                 <1>     push    ecx
                            <1>     ;push   ebx
0000601E 8B1D[C88E0100]     <1>     mov     ebx, [u.ppgdir] ; parent's page dir address (physical)
                            <1>
                            <1>     ; check the parent's PTE value is read only & same page or not..
00006024 89EF               <1>     mov     edi, ebp
00006026 C1EF16             <1>     shr     edi, PAGE_D_SHIFT ; 22
                            <1>     ; EDI = page directory entry index (0-1023)
00006029 89EE               <1>     mov     esi, ebp
0000602B C1EE0C             <1>     shr     esi, PAGE_SHIFT ; 12
0000602E 81E6FF030000       <1>     and     esi, PTE_MASK
                            <1>     ; ESI = page table entry index (0-1023)
                            <1>
00006034 66C1E702           <1>     shl     di, 2 ; * 4
00006038 01FB               <1>     add     ebx, edi ; PDE offset (for the parent)
0000603A 8B0F               <1>     mov     ecx, [edi]
0000603C F6C101             <1>     test    cl, PDE_A_PRESENT ; present (valid) or not ?
0000603F 7425               <1>     jz      short dmem_acc_3      ; parent process does not use this page
00006041 6681E100F0         <1>     and     cx, PDE_A_CLEAR ; 0F000h ; Clear attribute bits
00006046 66C1E602           <1>     shl     si, 2 ; *4
0000604A 01CE               <1>     add     esi, ecx ; PTE offset (for the parent)
0000604C 8B1E               <1>     mov     ebx, [esi]
0000604E F6C301             <1>     test    bl, PTE_A_PRESENT ; present or not ?
00006051 7413               <1>     jz      short dmem_acc_3      ; parent process does not use this page
00006053 662500F0           <1>     and     ax, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
00006057 6681E300F0         <1>     and     bx, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
0000605C 39D8               <1>     cmp     eax, ebx       ; parent's and child's pages are same ?
0000605E 7506               <1>     jne     short dmem_acc_3      ; not same page
                            <1>                             ; deallocate the child's page
00006060 800E02             <1>         or      byte [esi], PTE_A_WRITE ; convert to writable page (parent)
                            <1>     ;pop    ebx
00006063 59                 <1>     pop     ecx
00006064 EB0C               <1>     jmp     short dmem_acc_5
                            <1> dmem_acc_3:
                            <1>     ;pop    ebx
00006066 59                 <1>     pop     ecx
                            <1> dmem_acc_4:
00006067 66A90004           <1>     test    ax, PTE_SHARED ; shared or direct memory access indicator
0000606B 7505               <1>     jnz     short dmem_acc_5   ; AVL bit 1 = 1, do not deallocate this page!
                            <1>     ;
                            <1>     ;and    ax, PTE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
0000606D E877F9FFFF         <1>     call    deallocate_page
                            <1> dmem_acc_5:
                            <1>     ;pop    esi
                            <1>     ;pop    edi
                            <1> dmem_acc_6:
00006072 89E8               <1>     mov     eax, ebp ; physical page (offset=0) address
                            <1>     ; EAX = memory page address
                            <1>     ; EDX = PTE entry address (physical)
00006074 660D0704           <1>     or      ax, PTE_A_PRESENT+PTE_A_USER+PTE_A_WRITE+PTE_SHARED
                            <1>                     ; present flag, bit 0 = 1
```

```
                                <1>                     ; user flag, bit 2 = 1
                                <1>                     ; writable flag, bit 1 = 1
                                <1>                     ; direct memory access flag, bit 10 = 1
                                <1>                     ; (This page must not be deallocated!)
00006078 8902                   <1>     mov     [edx], eax  ; Update PTE value
0000607A 49                     <1>     dec     ecx ; remain count of contiguous pages
0000607B 741E                   <1>     jz      short dmem_acc_8
0000607D 81C500100000           <1>     add     ebp, PAGE_SIZE ; next physical page address
                                <1>     ; 22/07/2017
                                <1>     ;mov    eax, ebp
                                <1>     ; 12/05/2017
00006083 8B1D[288D0100]         <1>     mov     ebx, [base_addr] ; linear address (virtual+CORE)
00006089 81C300100000           <1>     add     ebx, PAGE_SIZE ; next linear address
0000608F E948FFFFFF             <1>         jmp     dmem_acc_0
                                <1> dmem_acc_7:  ; ERROR !
00006094 C7042404000000         <1>     mov     dword [esp], ERR_MINOR_IM
                                <1>             ; Insufficient memory (minor) error!
                                <1>             ; Major error = 0 (No protection fault)
                                <1>     ; cf = 1
                                <1> dmem_acc_8:
0000609B 58                     <1>     pop     eax
                                <1>     ;pop    edx
                                <1>     ;pop    ecx
                                <1>     ;pop    ebx
                                <1>     ;pop    ebp
0000609C C3                     <1>     retn
                                <1>
                                <1> deallocate_user_pages:
                                <1>     ; 20/05/2017
                                <1>     ; 15/05/2017
                                <1>     ; 20/02/2017
                                <1>     ; 19/02/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; Deallocate virtually contiguous user pages (memory block)
                                <1>     ; (caller: 'sysdalloc' system call)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EBX = VIRTUAL ADDRESS (beginning address)
                                <1>     ;       ECX = byte count
                                <1>     ;       [u.pgdir] = user's page directory
                                <1>     ;       [u.ppdir] = parent's page directory
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;    If CF = 0
                                <1>     ;       EAX = Deallocated memory bytes
                                <1>     ;         (Even if shared or read only pages will not be
                                <1>     ;          deallocated on M.A.T., this byte count will be
                                <1>     ;          returned as virtually deallocated bytes; in fact
                                <1>     ;          virtually deallocated user pages * 4096.)
                                <1>     ;       EBX = Virtual address (as rounded up)
                                <1>     ;    If CF = 1
                                <1>     ;       EAX = 0 (there is not any deallocated pages)
                                <1>     ;
                                <1>     ; Note: Empty page tables will not be deallocated!!!
                                <1>     ;     (they will be deallocated at process termination stage)
                                <1>     ;
                                <1>     ; Modified Registers -> EAX, EDX, ESI, EDI, EBX, ECX, EBP
                                <1>     ;
0000609D 89DE                   <1>     mov     esi, ebx
0000609F 89F7                   <1>     mov     edi, esi
000060A1 01CF                   <1>     add     edi, ecx
000060A3 81C6FF0F0000           <1>     add     esi, PAGE_SIZE - 1  ; 4095 (round up)
000060A9 C1EE0C                 <1>     shr     esi, PAGE_SHIFT
000060AC C1EF0C                 <1>     shr     edi, PAGE_SHIFT
000060AF 89F8                   <1>     mov     eax, edi ; end page
000060B1 29F0                   <1>     sub     eax, esi ; end page - start page
000060B3 0F86C8000000           <1>     jna     da_u_pd_err  ; < 1
000060B9 89F3                   <1>     mov     ebx, esi
000060BB C1E30C                 <1>     shl     ebx, PAGE_SHIFT ; virtual address (as rounded up)
000060BE 53                     <1>     push    ebx ; *
000060BF 89C1                   <1>     mov     ecx, eax ; page count
000060C1 C1E00C                 <1>     shl     eax, PAGE_SHIFT ; byte count as adjusted
000060C4 50                     <1>     push    eax ; **
000060C5 8B1D[C48E0100]         <1>     mov     ebx, [u.pgdir] ; physical addr of user's page dir
000060CB 81C600040000           <1>     add     esi, CORE/PAGE_SIZE
000060D1 89F7                   <1>     mov     edi, esi
000060D3 81E7FF030000           <1>     and     edi, PTE_MASK ; PTE entry in the page table
000060D9 57                     <1>     push    edi ; *** ; PTE index (of page directory)
000060DA C1EE0A                 <1>     shr     esi, PAGE_D_SHIFT - PAGE_SHIFT ; 22-12=10
000060DD 89F2                   <1>     mov     edx, esi
                                <1>     ; EDX = PDE index
000060DF C1E602                 <1>     shl     esi, 2 ; convert PDE index to dword offset
000060E2 01DE                   <1>     add     esi, ebx ; add page directory address
                                <1> da_u_pd_1:
000060E4 AD                     <1>     lodsd
                                <1>     ;
000060E5 89F5                   <1>     mov     ebp, esi ; 20/02/2017
                                <1>     ; EBP = next PDE address
                                <1>     ;
000060E7 A801                   <1>     test    al, PDE_A_PRESENT ; bit 0, present flag (must be 1)
000060E9 0F8487000000           <1>     jz      da_u_pd_3 ; 20/05/2017
000060EF 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
                                <1>     ; EAX = PHYSICAL (flat) ADDRESS OF THE PAGE TABLE
000060F3 8B3C24                 <1>     mov     edi, [esp] ; ***
                                <1>     ; EDI = PTE index (of complete page directory)
                                <1>     ;and    edi, PTE_MASK ; PTE entry in the page table
000060F6 C1E702                 <1>     shl     edi, 2 ; convert PTE index to dword offset
000060F9 89FE                   <1>     mov     esi, edi ; PTE offset in page table (0-4092)
000060FB 01C6                   <1>     add     esi, eax ; now, esi points to requested PTE
                                <1> da_u_pt_0:
000060FD AD                     <1>     lodsd
000060FE A801                   <1>     test    al, PTE_A_PRESENT ; bit 0, present flag (must be 1)
00006100 7452                   <1>     jz      short da_u_pt_1
                                <1>     ;
00006102 A802                   <1>     test    al, PTE_A_WRITE   ; bit 1, writable (r/w) flag
                                <1>                               ; (must be 1)
00006104 753C                   <1>     jnz     short da_u_pt_3
                                <1>     ; Read only -duplicated- page (belongs to a parent or a child)
00006106 66A90002               <1>         test    ax, PTE_DUPLICATED ; Was this page duplicated
                                <1>                                    ; as child's page ?
0000610A 7441                   <1>     jz      short da_u_pt_4 ; Clear PTE but don't deallocate the page!
                                <1>     ;
                                <1>     ; check the parent's PTE value is read only & same page or not..
                                <1>     ; EDX = page directory entry index (0-1023)
0000610C 52                     <1>     push    edx ; ****
                                <1>     ; EDI = page table entry offset (0-4092)
0000610D 8B1D[C88E0100]         <1>     mov     ebx, [u.ppgdir] ; page directory of the parent process
00006113 66C1E202               <1>     shl     dx, 2 ; *4
00006117 01D3                   <1>     add     ebx, edx ; PDE address (for the parent)
00006119 8B13                   <1>     mov     edx, [ebx] ; page table address
0000611B F6C201                 <1>     test    dl, PDE_A_PRESENT ; present (valid) or not ?
0000611E 7421                   <1>     jz      short da_u_pt_2       ; parent process does not use this page
00006120 6681E200F0             <1>     and     dx, PDE_A_CLEAR ; 0F000h ; Clear attribute bits
                                <1>     ; EDI = page table entry offset (0-4092)
```

```
00006125 01D7                <1>     add     edi, edx       ; PTE address (for the parent)
00006127 8B1F                <1>     mov     ebx, [edi]
00006129 F6C301              <1>     test    bl, PTE_A_PRESENT ; present or not ?
0000612C 7413                <1>     jz      short da_u_pt_2       ; parent process does not use this page
0000612E 662500F0            <1>     and     ax, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
00006132 6681E300F0          <1>     and     bx, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
00006137 39D8                <1>     cmp     eax, ebx       ; parent's and child's pages are same ?
00006139 7506                <1>     jne     short da_u_pt_2       ; not same page
                             <1>                            ; deallocate the child's page
0000613B 800F02              <1>         or      byte [edi], PTE_A_WRITE ; convert to writable page (parent)
0000613E 5A                  <1>     pop     edx ; ****
0000613F EB0C                <1>     jmp     short da_u_pt_4
                             <1>
                             <1> ; 17/04/2021
                             <1> ; ('da_u_pt_1' is disabled as temporary)
                             <1>
                             <1> ;da_u_pt_1:
                             <1> ;   or      eax, eax       ; swapped page ?
                             <1> ;   jz      short da_u_pt_5       ; no
                             <1> ;                          ; yes
                             <1> ;   shr     eax, 1
                             <1> ;   call    unlink_swap_block ; Deallocate swapped page block
                             <1> ;                            ; on the swap disk (or in file)
                             <1> ;   jmp     short da_u_pt_5
                             <1> da_u_pt_2:
00006141 5A                  <1>     pop     edx ; ****
                             <1> da_u_pt_3:
00006142 66A90004            <1>     test    ax, PTE_SHARED ; shared or direct memory access indicator
00006146 7505                <1>     jnz     short da_u_pt_4       ; AVL bit 1 = 1, do not deallocate this page!
                             <1>     ;
                             <1>     ;and    ax, PTE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
00006148 E89CF8FFFF          <1>     call    deallocate_page ; set the mem allocation bit of this page
                             <1> da_u_pt_4:
0000614D C746FC00000000      <1>     mov     dword [esi-4], 0 ; clear/reset PTE (child, dupl. as parent)
                             <1> ; 17/04/2021 (temporary)
                             <1> da_u_pt_1:
                             <1> da_u_pt_5:
                             <1>     ; 20/05/2017
00006154 58                  <1>     pop     eax ; *** PTE index (of page directory)
00006155 49                  <1>     dec     ecx ; remain page count
00006156 7426                <1>     jz      short da_u_pd_4
00006158 40                  <1>     inc     eax ; next PTE
00006159 6625FF03            <1>     and     ax, PTE_MASK ; PTE entry index in the page table
0000615D 50                  <1>     push    eax ; *** (save again)
                             <1>     ;mov    edi, eax
                             <1>     ;and    di, PTE_MASK
                             <1>     ;cmp    edi, PAGE_SIZE / 4 ; 1024
                             <1>     ;jnb    short da_u_pd_2
0000615E 89C7                <1>     mov     edi, eax
00006160 C1E702              <1>     shl     edi, 2 ; convert index to dword offset
                             <1>     ;test   ax, PTE_MASK ; 3FFh
00006163 09C0                <1>     or      eax, eax
00006165 7596                <1>     jnz     short da_u_pt_0 ; 1-1023
                             <1> da_u_pd_2:
00006167 42                  <1>     inc     edx
                             <1>     ; 20/05/2017
00006168 6681E2FF03          <1>     and     dx, PTE_MASK  ; 3FFh
0000616D 740F                <1>     jz      short da_u_pd_4  ; 0 (1024)
                             <1>     ;cmp    edx, 1024
                             <1>     ;jnb    short da_u_pd_4
0000616F 89EE                <1>     mov     esi, ebp ; 20/02/2017
00006171 E96EFFFFFF          <1>     jmp     da_u_pd_1
                             <1> da_u_pd_3:
                             <1>     ; 15/05/2017 (empty page directory entry)
00006176 81E900040000        <1>     sub     ecx, 1024
0000617C 77E9                <1>     ja      short da_u_pd_2 ; 20/05/2017
                             <1> da_u_pd_4:
0000617E 58                  <1>     pop     eax ; **
0000617F 5B                  <1>     pop     ebx ; *
00006180 C3                  <1>     retn
                             <1>
                             <1> da_u_pd_err:
00006181 31C0                <1>     xor     eax, eax
00006183 F9                  <1>     stc
00006184 C3                  <1>     retn
                             <1>
                             <1> allocate_user_pages:
                             <1>     ; 20/05/2017
                             <1>     ; 01/05/2017, 02/05/2017, 15/05/2017
                             <1>     ; 04/03/2017
                             <1>     ; 20/02/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>     ; Allocate physically contiguous user pages (memory block)
                             <1>     ; (caller: 'sysalloc' system call)
                             <1>     ;
                             <1>     ; Note: This procedure does not alloc a page's itself
                             <1>     ;       (page bits) on Memory Allocation Table.
                             <1>     ;       (allocate_memory_block is needed before this proc)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       EAX = PHYSICAL ADDRESS (beginning address)
                             <1>     ;       EBX = VIRTUAL ADDRESS (beginning address)
                             <1>     ;       ECX = byte count (>=4096)
                             <1>     ;       [u.pgdir] = user's page directory
                             <1>     ;
                             <1>     ;       Note: All addresses are (must be) already adjusted
                             <1>     ;       to page borders, otherwise, lower 12bits of addresses
                             <1>     ;       and byte count would be truncated.
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;       none
                             <1>     ;
                             <1>     ;       CF = 1 -> insufficient memory error
                             <1>     ;
                             <1>     ; Note: All pages will be allocated in physical page order
                             <1>     ;       from the beginning page address.
                             <1>     ;       * A new page table will be added to the page dir
                             <1>     ;         when the requested PDE is invalid.
                             <1>     ;       * Those pages will not be added to swap queue
                             <1>     ;         because main purpose of this allocation is to
                             <1>     ;         set a direct memory access (DMA controller) buffer.
                             <1>     ;        (Swapping out a page in a DMA buffer would be wrong!)
                             <1>     ;       * Previous content of page tables (PTEs) would be
                             <1>     ;         (should be) deallocated before entering this
                             <1>     ;         procedure. So, new page table entries (PTEs)
                             <1>     ;         directly will be written without checking
                             <1>     ;         their previous content.
                             <1>     ;       * Only solution to increase free memory by removing
                             <1>     ;         that non-swappable memory block is to terminate
                             <1>     ;         the process or to wait until the process will
                             <1>     ;         deallocate that memory block as itself. ('sysdalloc')
                             <1>     ;         (No problem, if the process does not grab all of
                             <1>     ;         -very big amount of- free memory by using
                             <1>     ;         'sysalloc' system call!?)
```

```
                                <1>     ;        (Even if the process has grabbed all of free memory,
                                <1>     ;        no problem if the process is not running in
                                <1>     ;        multitasking mode. No problem in multitasking
                                <1>     ;        mode if there is not another process which is running
                                <1>     ;        or waiting or sleeping for an event as it's pages
                                <1>     ;        are swapped-out. But a new process can not start to
                                <1>     ;        run if all of free memory has beeen allocated
                                <1>     ;        by running processes. Deallocation -'sysdalloc'-
                                <1>     ;        or terminate a running process is needed
                                <1>     ;        in order to run a new process.)
                                <1>     ;
                                <1>     ; Modified Registers -> EAX, EDX, ESI, EDI, EBX, ECX, EBP
                                <1>     ;
                                <1>
                                <1>     ; 01/05/2017
00006185 662500F0               <1>     and     ax, ~PAGE_OFF
00006189 6681E300F0             <1>     and     bx, ~PAGE_OFF
                                <1>     ; 02/05/2017
0000618E BD00F0FFFF             <1>     mov     ebp, 0FFFFF000h ; 4 Giga Bytes - 4096 Bytes (for Stack)
00006193 C1E90C                 <1>     shr     ecx, PAGE_SHIFT ; page count
00006196 83F901                 <1>     cmp     ecx, 1
00006199 7251                   <1>     jb      short a_u_im_retn
0000619B 89C2                   <1>     mov     edx, eax
0000619D 01CA                   <1>     add     edx, ecx
0000619F 724B                   <1>     jc      short a_u_im_retn
000061A1 39D5                   <1>     cmp     ebp, edx
000061A3 7247                   <1>     jb      short a_u_im_retn
000061A5 89DA                   <1>     mov     edx, ebx
000061A7 81C200004000           <1>     add     edx, CORE
000061AD 723D                   <1>     jc      short a_u_im_retn
000061AF 01CA                   <1>     add     edx, ecx
000061B1 7239                   <1>     jc      short a_u_im_retn
000061B3 39D5                   <1>     cmp     ebp, edx
000061B5 7235                   <1>     jb      short a_u_im_retn
                                <1>     ;
000061B7 89C5                   <1>     mov     ebp, eax ; physical address
000061B9 89DE                   <1>     mov     esi, ebx
000061BB 81C600004000           <1>     add     esi, CORE ; start of user's memory (4M)
000061C1 C1EE0C                 <1>     shr     esi, PAGE_SHIFT ; higher 20 bits of the linear address
                                <1>     ;shr    ecx, PAGE_SHIFT ; page count
000061C4 8B1D[C48E0100]         <1>     mov     ebx, [u.pgdir] ; physical addr of user's page dir
000061CA 89F7                   <1>     mov     edi, esi
000061CC 81E7FF030000           <1>     and     edi, PTE_MASK ; PTE entry index in the page table
000061D2 57                     <1>     push    edi  ; * ; PTE index (in page directory)
000061D3 C1EE0A                 <1>     shr     esi, PAGE_D_SHIFT - PAGE_SHIFT ; 22-12=10
000061D6 89F2                   <1>     mov     edx, esi
                                <1>     ; EDX = PDE index
000061D8 C1E602                 <1>     shl     esi, 2 ; convert PDE index to dword offset
000061DB 01DE                   <1>     add     esi, ebx ; add page directory address
                                <1> a_u_pd_0:
000061DD AD                     <1>     lodsd
                                <1>     ;
000061DE 89F3                   <1>     mov     ebx, esi ; next PDE address
                                <1>     ;
000061E0 A801                   <1>     test    al, PDE_A_PRESENT ; bit 0, present flag (must be 1)
000061E2 7513                   <1>     jnz     short a_u_pd_2
                                <1>     ;
                                <1>     ; empty PDE (it does not point to valid page table address)
000061E4 E841F6FFFF             <1>     call    allocate_page  ; (allocate a new page table)
000061E9 7302                   <1>     jnc     short a_u_pd_1 ; OK... now, we have a new page table.
                                <1>     ; cf = 1
                                <1>     ; There is not a free memory page to allocate a new page table !!!
000061EB 5E                     <1>     pop     esi ; *
                                <1> a_u_im_retn:
000061EC C3                     <1>     retn    ; return to 'sysalloc' with 'insufficient memory' error
                                <1>     ;
                                <1> a_u_pd_1: ; clear the new page table content
                                <1>     ; EAX = Physical (base) address of the new page table
000061ED E8A9F6FFFF             <1>     call    clear_page ; Clear page content
                                <1>     ;
000061F2 0C07                   <1>     or      al, PDE_A_PRESENT + PDE_A_WRITE + PDE_A_USER
                                <1>              ; set bit 0, bit 1 and bit 2 to 1
                                <1>              ; (present, writable, user)
000061F4 8946FC                 <1>     mov     [esi-4], eax
                                <1> a_u_pd_2:
000061F7 662500F0               <1>     and     ax, PDE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
                                <1>     ; EAX = PHYSICAL (flat) ADDRESS OF THE PAGE TABLE
000061FB 8B3C24                 <1>     mov     edi, [esp] ; *
                                <1>     ; EDI = PTE index (of page directory)
                                <1>     ;and    edi, PTE_MASK ; PTE entry index in the page table
                                <1>     ; EBX = next PDE address
000061FE 89FE                   <1>     mov     esi, edi ; PTE index in page table (0-1023)
00006200 C1E702                 <1>     shl     edi, 2 ; convert PTE index to dword offset
00006203 01C7                   <1>     add     edi, eax ; now, edi points to requested PTE
                                <1> a_u_pt_0:
                                <1>     ; 02/05/2017
00006205 8B07                   <1>     mov     eax, [edi]
                                <1>     ;
00006207 A801                   <1>     test    al, PTE_A_PRESENT ; bit 0, present flag (must be 1)
00006209 7445                   <1>     jz      short a_u_pt_1
                                <1>     ;
0000620B A802                   <1>     test    al, PTE_A_WRITE   ; bit 1, writable (r/w) flag
                                <1>                               ; (must be 1)
0000620D 7550                   <1>     jnz     short a_u_pt_3
                                <1>     ; Read only -duplicated- page (belongs to a parent or a child)
0000620F 66A90002               <1>         test    ax, PTE_DUPLICATED ; Was this page duplicated
                                <1>                                  ; as child's page ?
00006213 7455                   <1>     jz      short a_u_pt_4 ; Clear PTE but don't deallocate the page!
                                <1>     ;
                                <1>     ; check the parent's PTE value is read only & same page or not..
                                <1>     ; EDX = page directory entry index (0-1023)
00006215 52                     <1>     push    edx ; **
00006216 53                     <1>     push    ebx ; ***
                                <1>     ; ESI = page table entry index (0-1023)
                                <1>     ;push   esi ; **** ; 20/05/2017
00006217 8B1D[C88E0100]         <1>     mov     ebx, [u.ppgdir] ; page directory of the parent process
0000621D 66C1E202               <1>     shl     dx, 2 ; *4
00006221 01D3                   <1>     add     ebx, edx ; PTE address,0 (for the parent)
00006223 8B13                   <1>     mov     edx, [ebx] ; page table address
00006225 F6C201                 <1>     test    dl, PDE_A_PRESENT ; present (valid) or not ?
00006228 7433                   <1>     jz      short a_u_pt_2 ; parent process does not use this page
0000622A 6681E200F0             <1>     and     dx, PDE_A_CLEAR ; 0F000h ; Clear attribute bits
0000622F 66C1E602               <1>     shl     si, 2 ; *4
                                <1>     ; ESI = page table entry offset (0-4092)
00006233 01D6                   <1>     add     esi, edx       ; PTE address (for the parent)
00006235 8B1E                   <1>     mov     ebx, [esi]
00006237 F6C301                 <1>     test    bl, PTE_A_PRESENT ; present or not ?
0000623A 7421                   <1>     jz      short a_u_pt_2 ; parent process does not use this page
0000623C 662500F0               <1>     and     ax, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
00006240 6681E300F0             <1>     and     bx, PTE_A_CLEAR ; 0F000h ; Clear attribute bits
00006245 39D8                   <1>     cmp     eax, ebx       ; parent's and child's pages are same ?
00006247 7514                   <1>     jne     short a_u_pt_2 ; not same page
                                <1>                            ; deallocate the child's page
00006249 800E02                 <1>         or      byte [esi], PTE_A_WRITE ; convert to writable page (parent)
```

```
                                <1>     ;pop    esi ; **** ; 20/05/2017
0000624C 5B                     <1>     pop     ebx ; ***
0000624D 5A                     <1>     pop     edx ; **
0000624E EB1A                   <1>     jmp     short a_u_pt_4
                                <1> a_u_pt_1:
00006250 09C0                   <1>     or      eax, eax        ; swapped page ?
00006252 7416                   <1>     jz      short a_u_pt_4 ; no
                                <1>                             ; yes
00006254 D1E8                   <1>     shr     eax, 1
00006256 E8B5F9FFFF             <1>     call    unlink_swap_block ; Deallocate swapped page block
                                <1>                               ; on the swap disk (or in file)
0000625B EB0D                   <1>     jmp     short a_u_pt_4
                                <1> a_u_pt_2:
                                <1>     ;pop    esi ; **** ; 20/05/2017
0000625D 5B                     <1>     pop     ebx ; ***
0000625E 5A                     <1>     pop     edx ; **
                                <1> a_u_pt_3:
0000625F 66A90004               <1>     test    ax, PTE_SHARED ; shared or direct memory access indicator
00006263 7505                   <1>     jnz     short a_u_pt_4 ; AVL bit 1 = 1, do not deallocate this page!
                                <1>     ;
                                <1>     ;and    ax, PTE_A_CLEAR ; 0F000h ; clear lower 12 (attribute) bits
00006265 E87FF7FFFF             <1>     call    deallocate_page ; set the mem allocation bit of this page
                                <1>     ;
                                <1> a_u_pt_4:
0000626A 89E8                   <1>     mov     eax, ebp ; physical address
0000626C 0C07                   <1>     or      al, PTE_A_PRESENT + PTE_A_WRITE + PTE_A_USER ; 04/03/2017
0000626E AB                     <1>     stosd
0000626F 5E                     <1>     pop     esi ; * ; 20/05/2017
00006270 49                     <1>     dec     ecx ; remain page count
00006271 7417                   <1>     jz      short a_u_pd_5
00006273 81C500100000           <1>     add     ebp, PAGE_SIZE
00006279 46                     <1>     inc     esi ; next PTE (index)
                                <1>     ; 20/05/2017
                                <1>     ;cmp    esi, PAGE_SIZE/4 ; 1024
                                <1>     ;jb     short a_u_pt_0
0000627A 6681E6FF03             <1>     and     si, PTE_MASK ; 3FFh (0 to 1023)
0000627F 56                     <1>     push    esi ; *
00006280 7583                   <1>     jnz     short a_u_pt_0 ; > 0 (<1024)
                                <1> a_u_pd_3:
00006282 42                     <1>     inc     edx
                                <1> ;   cmp     edx, 1024
                                <1> ;   jnb     short a_u_pd_4 ; 02/05/2017 (error!, ecx > 0)
00006283 89DE                   <1>     mov     esi, ebx ; the next PDE address
00006285 E953FFFFFF             <1>     jmp     a_u_pd_0
                                <1> a_u_pd_4:
                                <1>     ; 02/05/2017
                                <1> ;   stc
                                <1> a_u_pd_5:
                                <1>     ; 20/05/2017
                                <1>     ;pop    edi ; *
0000628A C3                     <1>     retn
                                <1>
                                <1>     ; 28/11/2023
                                <1> ;a_lfb_k__err:
                                <1>     ;retn
                                <1>
                                <1> allocate_lfb_pages_for_kernel:
                                <1>     ; 02/12/2023
                                <1>     ; 29/11/2023
                                <1>     ; 28/11/2023
                                <1>     ; 20/10/2023 - TRDOS 386 v2.0.7
                                <1>     ; (only the overlapped main memory pages will be allocated)
                                <1>     ; 28/11/2023
                                <1>     ; (but all LFB pages will be added to the kernel's page tables)
                                <1>     ; (NOTE: LFB size -which will be used- is the size of the max.
                                <1>     ;  recognized screen resolution. For example: 1024*768*4 bytes)
                                <1>     ; 15/12/2020
                                <1>     ; 14/12/2020 - TRDOS 386 v2.0.3
                                <1>     ; Set kernel page tables for linear frame buffer
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       ;[LFB_ADDR] = linear frame buffer base address
                                <1>     ;       ;[LFB_SIZE] = linear frame buffer size in bytes
                                <1>     ;       ; 29/11/2023
                                <1>     ;       ;ebx = Linear frame buffer base address as page num
                                <1>     ;       ; 20/10/2023
                                <1>     ;       esi = linear frame buffer base address
                                <1>     ;       ecx = memory size in pages = [memory_size]
                                <1>     ;       ; 28/11/2023
                                <1>     ;       edx = linear frame buffer size in pages
                                <1>     ; Output:
                                <1>     ;       none
                                <1>     ;       cf = 1 -> error
                                <1>     ;
                                <1>     ; Modified registers: eax, ebx, ecx, edx, esi, edi, ebp
                                <1>
                                <1>
                                <1> ; 02/12/2023 ; (!**!)
                                <1> ; ('memory size' -for M.A.T.- is always <= LFB stat/base address)
                                <1> %if 0
                                <1>
                                <1>     ; 29/11/2023
                                <1>     ; allocate LFB blocks which < memory size at first
                                <1>
                                <1>     ;mov    eax, edx
                                <1>     ;shl    eax, 12
                                <1>     ;add    eax, esi
                                <1>     ;jc     short a_lfb_k__err ; > 4GB limit
                                <1>     ;
                                <1>     ;;mov   eax, 1048576 ; page count of (full) 4GB memory
                                <1>     ;;sub   eax, edx
                                <1>     ;;cmp   eax, esi
                                <1>     ;jb     short a_lfb_k__err ; > 4GB limit
                                <1>
                                <1>     mov     ebx, esi ; LFB base/start address
                                <1>     shr     ebx, 12  ; convert byte address to page address
                                <1>     ; ebx = LFB start/base page number
                                <1>
                                <1>     sub     ecx, ebx
                                <1>     jna     short a_lfb_k_2 ; skip M.A.T. bit allocation
                                <1>     cmp     ecx, edx
                                <1>     jna     short a_lfb_k_0
                                <1>     mov     ecx, edx
                                <1> a_lfb_k_0:
                                <1>     sub     edx, ecx
                                <1>     jna     short a_lfb_k_1
                                <1>
                                <1>     ; edx = remain count of LFB pages which are out of the M.A.T.
                                <1>     ; ebx = LFB start/base page number
                                <1>     ; ecx = page allocation count in the M.A.T.
                                <1>
                                <1>     ;mov    eax, 4096
                                <1>     ;mul    ecx
                                <1>     mov     eax, ecx
```

```
                                <1>     shl     eax, 12 ; * 4096
                                <1>
                                <1>     add     esi, eax
                                <1>     ; esi = LFB address just after the main memory if there is
                                <1> a_lfb_k_1:
                                <1>     mov     eax, ebx ; LFB start/base page number
                                <1>     call    allocate_lfb_memory_block
                                <1>
                                <1>     ; here if there is, remain (the 2nd) part of the LFB
                                <1>     ; will be allocated by using new kernel page table(s)
                                <1>     ; (but M.A.T. bits will not be cleared because
                                <1>     ;  the 2nd LFB part is out of the M.A.T.)
                                <1>
                                <1>     and     edx, edx
                                <1>     jz      short a_lfb_k_6  ; all LFB pages have been allocated
                                <1>
                                <1> %endif
                                <1>
                                <1> a_lfb_k_2:
                                <1>     ; 29/11/2023
0000628B C1EE0C                 <1>     shr     esi, 12 ; convert LFB address to page number
                                <1>     ; edx = count of pages to be added (in kernel page tables)
                                <1>     ; 28/11/2023
0000628E 89D5                   <1>     mov     ebp, edx
00006290 81C2FF030000           <1>     add     edx, 1023 ; round up
00006296 C1EA0A                 <1>     shr     edx, 10  ; / 1024
                                <1>     ; edx = count of kernel page tables to be added
                                <1>     ; (edx = 1 or edx = 2)
                                <1>     ; ((1024*768*4 = 3145728 bytes, 768 PTEs, 1 page table))
                                <1>     ; ((1920*1080*4 = 8294400 bytes, 2025 PTEs, 2 page tables))
00006299 E88CF5FFFF             <1>     call    allocate_page
                                <1>     ;jc     short a_lfb_k__err
                                <1>     ; (mem alloc error is not expected at this startup stage)
                                <1>     ; eax = physical address of the new page table
0000629E 89F7                   <1>     mov     edi, esi  ; LFB start/base page number
000062A0 C1EF0A                 <1>     shr     edi, 10
                                <1>     ; edi = PDE entry number of the Linear Frame Buffer addr
                                <1>     ; (edi = 832 for 0D0000000h, edi = 896 for 0E0000000h)
000062A3 C1E702                 <1>     shl     edi, 2
                                <1>     ; edi = PDE offset
000062A6 033D[B0760100]         <1>     add     edi, [k_page_dir] ; Kernel's Page Dir Address
000062AC 50                     <1>     push    eax ; +
000062AD 660D0304               <1>     or      ax, PDE_A_PRESENT + PDE_A_WRITE + PDE_EXTERNAL
                                <1>                             ; supervisor + read&write + present
                                <1>                             ; + external memory block (LFB)
000062B1 AB                     <1>     stosd
                                <1>
000062B2 4A                     <1>     dec     edx
000062B3 740C                   <1>     jz      short a_lfb_k_3       ; only 1 new page table
                                <1>
                                <1>     ; the 2nd page table
000062B5 E870F5FFFF             <1>     call    allocate_page
                                <1>     ;jc     short a_lfb_k__err
                                <1>     ; (mem alloc error is not expected at this startup stage)
                                <1>     ; eax = physical address of the new page table
000062BA 89C3                   <1>     mov     ebx, eax
000062BC 660D0304               <1>     or      ax, PDE_A_PRESENT + PDE_A_WRITE + PDE_EXTERNAL
                                <1>                             ; supervisor + read&write + present
                                <1>                             ; + external memory block (LFB)
000062C0 AB                     <1>     stosd
                                <1>
                                <1> a_lfb_k_3:
                                <1>     ; set new PTEs
                                <1>     ;
                                <1>     ; ebp = count of pages (PTEs) to be allocated
000062C1 5F                     <1>     pop     edi ; +  ; 1st page table address
                                <1>     ; 29/11/2023
000062C2 B900040000             <1>     mov     ecx, 1024      ; number of PTEs in a page table
000062C7 89F0                   <1>     mov     eax, esi       ; LFB base/start page number
                                <1>                            ; (may be the 2nd part of the LFB)
000062C9 C1E00C                 <1>     shl     eax, 12
000062CC 660D0304               <1>     or      ax, PTE_A_PRESENT + PTE_A_WRITE + PTE_EXTERNAL
                                <1>                             ; supervisor + read&write + present
                                <1>                             ; + external memory block (LFB)
                                <1> a_lfb_k_4:
000062D0 AB                     <1>     stosd                  ; save as PTE
000062D1 4D                     <1>     dec     ebp
000062D2 740D                   <1>     jz      short a_lfb_k_5
000062D4 0500100000             <1>     add     eax, 4096      ; the next page address
000062D9 E2F5                   <1>     loop    a_lfb_k_4
                                <1>
                                <1>     ;and    ebx, ebx
                                <1>     ;jz     short a_lfb_k_6
000062DB 89DF                   <1>     mov     edi, ebx       ; the 2nd page table address
                                <1>     ;xor    ebx, ebx
                                <1>     ;mov    ecx, 1024
000062DD B504                   <1>     mov     ch, 4 ; cx = 4*256 = 1024
000062DF EBEF                   <1>     jmp     short a_lfb_k_4 ; only 1 more loop (no the 3rd pt)
                                <1>                             ; (ebp <= 1024 here)
                                <1> a_lfb_k_5:
000062E1 49                     <1>     dec     ecx
000062E2 7404                   <1>     jz      short a_lfb_k_6
000062E4 31C0                   <1>     xor     eax, eax       ; clear page table entry (empty)
000062E6 F3AB                   <1>     rep     stosd
                                <1> a_lfb_k_6:
                                <1>     ; 29/11/2023
000062E8 C3                     <1>     retn
                                <1>
                                <1> ; 28/11/2023
                                <1> %if 0
                                <1>
                                <1> allocate_lfb_pages_for_kernel:
                                <1>     ; 20/10/2023 - TRDOS 386 v2.0.7
                                <1>     ; (only the overlapped main memory pages will be allocated)
                                <1>     ; ((if LFB base/start address is greater than -or equal to-
                                <1>     ;      main memory size, this procedure is not called/used))
                                <1>     ; 15/12/2020
                                <1>     ; 14/12/2020 - TRDOS 386 v2.0.3
                                <1>     ; Set kernel page tables for linear frame buffer
                                <1>     ;
                                <1>     ; Input:
                                <1>     ;       ;[LFB_ADDR] = linear frame buffer base address
                                <1>     ;       ;[LFB_SIZE] = linear frame buffer size in bytes
                                <1>     ;       ; 20/10/2023
                                <1>     ;       eax = Linear frame buffer base address as page num
                                <1>     ;             (eax < memory size)
                                <1>     ;       esi = linear frame buffer base address
                                <1>     ;       ecx = memory size in pages = [memory_size]
                                <1>     ; Output:
                                <1>     ;       none
                                <1>     ;       cf = 1 -> error
                                <1>     ;
                                <1>     ; Modified registers: eax, ecx, edx, edi
                                <1>
```

```
<1>     ; 20/10/2023
<1>     ;mov    edi, [LFB_ADDR]
<1>     ;mov    edx, [LFB_SIZE]
<1>
<1>     ;;;
<1>     ; 20/10/2023
<1>     mov     edi, eax ; LFB base/start page number
<1>     ;mov    ecx, [memory_size]
<1>     sub     ecx, eax
<1>
<1>     ; ecx = number of pages to be set as (already) allocated
<1>     ;;;
<1>
<1>     ; 20/10/2023
<1>     shr     edi, 10 ; convert page number to PDE entry number
<1>                     ; (1024 pages per page table -the shift
<1>                     ; result is a page table index number
<1>                     ; or page directory entry number <= 1023)
<1>     ;shr    edi, 22 ; convert address to page number
<1>                     ; and then convert it to PDE entry offset
<1>                     ; (1 PDE is for 4MB, 22 bit shift)
<1>
<1>     shl     di, 2   ; * 4 to obtain the PDE offset
<1>
<1>     ; 20/10/2023
<1>     ;;add   edx, 4095
<1>     ;shr    edx, 12 ; convert LFB size to LFB page count
<1>
<1>     ;mov    ecx, edx ; * ; LFB page count
<1>     ; 20/10/2023
<1>     mov     edx, ecx ; *
<1>                     ; 20/10/2023
<1>                     ; edx = page count (to be set as allocated)
<1>                     ;       from the LFB start page
<1>                     ; (it may be <= linear frame buffer size)
<1>
<1>     add     ecx, 1023 ; page count + 1023
<1>     shr     ecx, 10 ; convert to page directory entry count
<1>                     ; (page table count)
<1>     push    ecx ; **
<1>     shl     ecx, 12 ; convert to byte count
<1>
<1>     ; 20/10/2023
<1>     ; (absolute allocation is needed instead of the first fit)
<1>     ;xor    eax, eax ; first available pages
<1>
<1>     ; 20/10/2023
<1>     ;mov    esi, [LFB_ADDR]
<1>     mov     eax, esi ; linear frame buffer's base/start address
<1>
<1>     ; allocate contiguous memory block for these kernel pages
<1>
<1>     ;call   allocate_memory_block
<1>     ; 20/10/2023
<1>     ; (short way to set LFB page bits on the 'M.A.T.)
<1>     call    allocate_lfb_memory_block ; in 'allocate_memory_block'
<1>
<1>     ; eax = start address of (contiguous) memory block
<1>     pop     ecx ; ** ; PDE count
<1>     jnc     short a_lfb_k_1
<1>     ; error (cf=1)
<1>     retn
<1> a_lfb_k_1:
<1>     ; Allocate (new) page tables in kernel's page directory
<1>     push    ecx ; PDE (page table) count
<1>     push    eax ; start address of contiguous memory pages
<1>                 ; (at page boundary)
<1>     ; edi = 1st page directory entry offset
<1>     add     edi, [k_page_dir] ; Kernel's Page Dir Address
<1> a_lfb_k_2:
<1>     or      ax, PDE_A_PRESENT + PDE_A_WRITE + PDE_EXTERNAL
<1>                             ; supervisor + read&write + present
<1>                             ; + external memory block (LFB)
<1>     stosd
<1>     add     eax, 4096
<1>     loop    a_lfb_k_2
<1>
<1>     pop     edi ; start addr of contiguous memory pages
<1>     pop     ecx ; page table (PDE) count
<1>
<1>     ; Allocate pages in (new) kernel page tables
<1>
<1>     ; (Note: page tables are contiguous in pyhsical memory)
<1>     shl     ecx, 10 ; * 1024, convert to (total) PTE count
<1>
<1>     mov     eax, [LFB_ADDR]
<1>     ; edx = LFB page count
<1>     ;and    ax, ~4095  ; lw of LFB address is 0
<1> a_lfb_k_3:
<1>     or      ax, PTE_A_PRESENT + PTE_A_WRITE + PTE_EXTERNAL
<1>                             ; supervisor + read&write + present
<1>                             ; + external memory block (LFB)
<1>     stosd
<1>     dec     edx
<1>     jz      short a_lfb_k_4 ; LFB size has been completed (!?)
<1>     add     eax, 4096
<1>     loop    a_lfb_k_3
<1>
<1>     retn
<1>
<1> a_lfb_k_4:
<1>     ; clear PTEs for empty/free pages
<1>     ;       (if there are after LFB !?)
<1>     xor     eax, eax ; clear page table entry (empty)
<1>     rep     stosd
<1>     retn
<1>
<1> %endif
<1>
<1> ;deallocate_lfb_pages_for_kernel:
<1>     ; 15/12/2020
<1>     ; 14/12/2020 - TRDOS 386 v2.0.3
<1>     ; Reset/Release kernel page tables
<1>     ;       which are used for linear frame buffer
<1>     ; (this procedure will be called by kernel only)
<1>     ;
<1>     ; Input:
<1>     ;       [LFB_ADDR] = linear frame buffer base address
<1>     ;       [FFB_SIZE] = linear frame buffer size in bytes
<1>     ; Output:
<1>     ;       none
<1>     ;
<1>     ; Modified registers: eax, ecx, edi
<1>
```

```
                                <1>     ;mov    edi, [LFB_ADDR]
                                <1>     ;mov    ecx, [LFB_SIZE]
                                <1>     ;
                                <1>     ;shr    edi, 22 ; convert address to page number
                                <1>     ;               ; and then convert it to PDE entry offset
                                <1>     ;               ; (1 PDE is for 4MB, 22 bit shift)
                                <1>     ;
                                <1>     ;shl    di, 2   ; * 4 for offset
                                <1>     ;
                                <1>     ;;add   ecx, 4095
                                <1>     ;shr    ecx, 12 ; convert LFB size to page count
                                <1>     ;
                                <1>     ;add    ecx, 1023 ; page count + 1023
                                <1>     ;shr    ecx, 10 ; convert to page directory entry count
                                <1>     ;               ; (page table count)
                                <1>     ;push   ecx ; *
                                <1>     ;shl    ecx, 12 ; convert to byte count
                                <1>     ;
                                <1>     ;xor    eax, eax ; first available pages
                                <1>     ;
                                <1>     ;; deallocate contiguous memory block for kernel pages
                                <1>     ;
                                <1>     ;call   deallocate_memory_block
                                <1>     ;
                                <1>     ;pop    ecx ; * ; PDE count
                                <1>     ;
                                <1>     ;; Release/Free PDEs (page tables) in kernel's page dir
                                <1>     ;; edi = 1st page directory entry offset
                                <1>     ;add    edi, [k_page_dir] ; Kernel's Page Dir Address
                                <1>     ;sub    eax, eax ; clear (also invalidate)
                                <1>     ;rep    stosd
                                <1>     ;
                                <1>     ;retn
                                <1>
                                <1> ; /// End Of MEMORY MANAGEMENT FUNCTIONS ///
                                <1>
                                <1> ;; Data:
                                <1>
                                <1> ; 09/03/2015
                                <1> ;swpq_count: dw 0 ; count of pages on the swap que
                                <1> ;swp_drv:    dd 0 ; logical drive description table address of the swap drive/disk
                                <1> ;swpd_size:  dd 0 ; size of swap drive/disk (volume) in sectors (512 bytes).
                                <1> ;swpd_free:  dd 0 ; free page blocks (4096 bytes) on swap disk/drive (logical)
                                <1> ;swpd_next:  dd 0 ; next free page block
                                <1> ;swpd_last:  dd 0 ; last swap page block
                                    %include 'timer.s'   ; 17/01/2015
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - timer.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 08/08/2022  (Previous: 18/04/2021)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 17/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Turkish Rational DOS
                                <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                                <1> ;
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ;
                                <1> ; Derived from 'IBM PC-AT' BIOS source code (1985)
                                <1> ; ****************************************************************************
                                <1>
                                <1> ; TRDOS 386  (TRDOS v2.0) Kernel - TIMER & REAL TIME CLOCK (BIOS) FUNCTIONS
                                <1>
                                <1> ; IBM PC-AT BIOS Source Code ('BIOS2.ASM')
                                <1> ; TITLE BIOS2 ---- 06/10/85 BIOS INTERRUPT ROUTINES
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1> ;
                                <1> ; ///////// TIMER (& REAL TIME CLOCK) FUNCTIONS ///////////////
                                <1>
                                <1> int1Ah:
                                <1>     ; 29/01/2016
                                <1>     ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
000062E9 9C                     <1>     pushfd
000062EA 0E                     <1>     push    cs
000062EB E801000000             <1>     call    TIME_OF_DAY_1
000062F0 C3                     <1>     retn
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> ;--- INT 1A H -- (TIME OF DAY) ------------------------------------------------
                                <1> ;       THIS BIOS ROUTINE ALLOWS THE CLOCKS TO BE SET OR READ                :
                                <1> ;                                                                   :
                                <1> ; PARAMETERS:                                                                :
                                <1> ;     (AH) = 00H  READ THE CURRENT SETTING AND RETURN WITH,                  :
                                <1> ;                     (CX) = HIGH PORTION OF COUNT                           :
                                <1> ;                     (DX) = LOW PORTION OF COUNT                            :
                                <1> ;                     (AL) = 0 TIMER HAS NOT PASSED 24 HOURS SINCE LAST READ :
                                <1> ;                            1 IF ON ANOTHER DAY. (RESET TO ZERO AFTER READ) :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 01H  SET THE CURRENT CLOCK USING,                               :
                                <1> ;                (CX) = HIGH PORTION OF COUNT                       :
                                <1> ;                (DX) = LOW PORTION OF COUNT.                       :
                                <1> ;                                                                   :
                                <1> ;             NOTE: COUNTS OCCUR AT THE RATE OF 1193180/65536 COUNTS/SECOND   :
                                <1> ;                   (OR ABOUT 18.2 PER SECOND -- SEE EQUATES)                :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 02H  READ THE REAL TIME CLOCK AND RETURN WITH,                  :
                                <1> ;                     (CH) = HOURS IN BCD (00-23)                            :
                                <1> ;                     (CL) = MINUTES IN BCD (00-59)                          :
                                <1> ;                     (DH) = SECONDS IN BCD (00-59)                          :
                                <1> ;                     (DL) = DAYLIGHT SAVINGS ENABLE (00-01)                 :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 03H  SET THE REAL TIME CLOCK USING,                             :
                                <1> ;                    (CH) = HOURS IN BCD (00-23)                             :
                                <1> ;                    (CL) = MINUTES IN BCD (00-59)                           :
                                <1> ;                    (DH) = SECONDS IN BCD (00-59)                           :
                                <1> ;                    (DL) = 01 IF DAYLIGHT SAVINGS ENABLE OPTION, ELSE 00.   :
                                <1> ;                                                                   :
                                <1> ;             NOTE: (DL) = 00 IF DAYLIGHT SAVINGS TIME ENABLE IS NOT ENABLED. :
                                <1> ;                   (DL) = 01 ENABLES TWO SPECIAL UPDATES THE LAST SUNDAY IN  :
                                <1> ;                APRIL   (1:59:59 --> 3:00:00 AM) AND THE LAST SUNDAY IN      :
                                <1> ;                    OCTOBER (1:59:59 --> 1:00:00 AM) THE FIRST TIME.         :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 04H  READ THE DATE FROM THE REAL TIME CLOCK AND RETURN WITH,    :
                                <1> ;                     (CH) = CENTURY IN BCD (19 OR 20)                       :
                                <1> ;                     (CL) = YEAR IN BCD (00-99)                             :
                                <1> ;                     (DH) = MONTH IN BCD (01-12)                            :
                                <1> ;                     (DL) = DAY IN BCD (01-31).                             :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 05H  SET THE DATE INTO THE REAL TIME CLOCK USING,               :
                                <1> ;                    (CH) = CENTURY IN BCD (19 OR 20)                        :
```

```
                                <1> ;                    (CL) = YEAR IN BCD (00-99)                     :
                                <1> ;                    (DH) = MONTH IN BCD (01-12)                        :
                                <1> ;                    (DL) = DAY IN BCD (01-31).                     :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 06H  SET THE ALARM TO INTERRUPT AT SPECIFIED TIME,            :
                                <1> ;                    (CH) = HOURS IN BCD (00-23 (OR FFH))                   :
                                <1> ;                    (CL) = MINUTES IN BCD (00-59 (OR FFH))                 :
                                <1> ;                    (DH) = SECONDS IN BCD (00-59 (OR FFH))                 :
                                <1> ;                                                                   :
                                <1> ;     (AH) = 07H  RESET THE ALARM INTERRUPT FUNCTION.                      :
                                <1> ;                                                                   :
                                <1> ; NOTES: FOR ALL RETURNS CY= 0 FOR SUCCESSFUL OPERATION.                   :
                                <1> ;        FOR (AH)= 2, 4, 6 - CARRY FLAG SET IF REAL TIME CLOCK NOT OPERATING. :
                                <1> ;        FOR (AH)= 6 - CARRY FLAG SET IF ALARM ALREADY ENABLED.            :
                                <1> ;        FOR THE ALARM FUNCTION (AH = 6) THE USER MUST SUPPLY A ROUTINE AND   :
                                <1> ;         INTERCEPT THE CORRECT ADDRESS IN THE VECTOR TABLE FOR INTERRUPT 4AH.       :
                                <1> ;         USE 0FFH FOR ANY "DO NOT CARE" POSITION FOR INTERVAL INTERRUPTS.    :
                                <1> ;        INTERRUPTS ARE DISABLED DURING DATA MODIFICATION.                 :
                                <1> ;        AH & AL ARE RETURNED MODIFIED AND NOT DEFINED EXCEPT WHERE INDICATED.      :
                                <1> ;-------------------------------------------------------------------------------
                                <1>
                                <1> ; 29/07/2022
                                <1> ; 15/01/2017
                                <1> ; 14/01/2017
                                <1> ; 07/01/2017
                                <1> ; 02/01/2017
                                <1> ; 29/05/2016
                                <1> ; 29/01/2016
                                <1> ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> ; 29/05/2016
                                <1> ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> int35h:  ; Date/Time functions
                                <1>
                                <1> TIME_OF_DAY_1:
                                <1>     ; 07/08/5022
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ;sti                            ; INTERRUPTS BACK ON
                                <1>     ; 29/05/2016
000062F1 80642408FE             <1>     and     byte [esp+8], 11111110b      ; clear carry bit of eflags register
                                <1>     ;
000062F6 80FC08                 <1>     cmp     ah, (RTC_TBE-RTC_TB)/4; CHECK IF COMMAND IN VALID RANGE (0-7)
000062F9 F5                     <1>     cmc                             ; COMPLEMENT CARRY FOR ERROR EXIT
                                <1>     ; (*) jc short TIME_9           ; EXIT WITH CARRY = 1 IF NOT VALID
000062FA 721A                   <1>     jc      short _TIME_9 ; 29/05/2016
                                <1>
000062FC 1E                     <1>     push    ds
000062FD 56                     <1>     push    esi
000062FE 66BE1000               <1>     mov     si, KDATA               ; kernel data segment
00006302 8EDE                   <1>     mov     ds, si
                                <1>
                                <1>     ;;15/01/2017
                                <1>     ; 14/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;;mov   byte [intflg], 35h    ; date & time interrupt
                                <1>     ;sti
                                <1>     ;
00006304 C0E402                 <1>     shl     ah, 2                   ; convert function to dword offset
00006307 0FB6F4                 <1>     movzx   esi, ah                 ; PLACE INTO ADDRESSING REGISTER
                                <1>     ;cli                            ; NO INTERRUPTS DURING TIME FUNCTIONS
0000630A FF96[1C630000]         <1>     call    [esi+RTC_TB]            ; VECTOR TO FUNCTION REQUESTED WITH CY=0
                                <1>                                     ; RETURN WITH CARRY FLAG SET FOR RESULT
                                <1>     ;sti                            ; INTERRUPTS BACK ON
00006310 B400                   <1>     mov     ah, 0                   ; CLEAR (AH) TO ZERO
00006312 5E                     <1>     pop     esi                     ; RECOVER USERS REGISTER
00006313 1F                     <1>     pop     ds                      ; RECOVER USERS SEGMENT SELECTOR
                                <1>
                                <1>     ;;15/01/2017
                                <1>     ; 02/01/2017
                                <1>     ;;mov   byte [ss:intflg], 0 ; 07/01/2017
                                <1>
                                <1> ;TIME_9:
                                <1>                                     ; RETURN WITH CY= 0 IF NO ERROR
                                <1>     ; (*) 29/05/2016
                                <1>     ; (*) retf 4 ; skip eflags on stack
00006314 7305                   <1>     jnc     short _TIME_10
                                <1> _TIME_9:
                                <1>     ; 29/05/2016 -set carry flag on stack-
                                <1>     ; [esp] = EIP
                                <1>     ; [esp+4] = CS
                                <1>     ; [esp+8] = E-FLAGS
00006316 804C240801             <1>     or      byte [esp+8], 1        ; set carry bit of eflags register
                                <1>     ; [esp+12] = ESP (user)
                                <1>     ; [esp+16] = SS (User)
                                <1> _TIME_10:
0000631B CF                     <1>     iretd
                                <1>
                                <1>     ; (*) 29/05/2016 - 'ref 4' intruction causes to stack fault
                                <1>     ; (OUTER-PRIVILEGE-LEVEL)
                                <1>     ; INTEL 80386 PROGRAMMER'S REFERENCE MANUAL 1986
                                <1>     ; // RETF instruction:
                                <1>     ;
                                <1>     ; IF OperandMode=32 THEN
                                <1>     ;    Load CS:EIP from stack;
                                <1>     ;    Set CS RPL to CPL;
                                <1>     ;    Increment eSP by 8 plus the immediate offset if it exists;
                                <1>     ;    Load SS:eSP from stack;
                                <1>     ; ELSE (* OperandMode=16 *)
                                <1>     ;    Load CS:IP from stack;
                                <1>     ;    Set CS RPL to CPL;
                                <1>     ;    Increment eSP by 4 plus the immediate offset if it exists;
                                <1>     ;    Load SS:eSP from stack;
                                <1>     ; FI;
                                <1>     ;
                                <1>     ; //
                                <1>                                     ; ROUTINE VECTOR TABLE (AH)=
                                <1> RTC_TB:
0000631C [3C630000]             <1>     dd      RTC_00                  ; 0 = READ CURRENT CLOCK COUNT
00006320 [4F630000]             <1>     dd      RTC_10                  ; 1 = SET CLOCK COUNT
00006324 [5D630000]             <1>     dd      RTC_20                  ; 2 = READ THE REAL TIME CLOCK TIME
00006328 [8B630000]             <1>     dd      RTC_30                  ; 3 = SET REAL TIME CLOCK TIME
0000632C [C8630000]             <1>     dd      RTC_40                  ; 4 = READ THE REAL TIME CLOCK DATE
00006330 [EE630000]             <1>     dd      RTC_50                  ; 5 = SET REAL TIME CLOCK DATE
00006334 [4D640000]             <1>     dd      RTC_60                  ; 6 = SET THE REAL TIME CLOCK ALARM
00006338 [9F640000]             <1>     dd      RTC_70                  ; 7 = RESET ALARM
                                <1>
                                <1> RTC_TBE     equ     $
                                <1>
                                <1> RTC_00:                             ; READ TIME COUNT
0000633C A0[34770100]           <1>     mov     al, [TIMER_OFL]                 ; GET THE OVERFLOW FLAG
00006341 C605[34770100]00       <1>     mov     byte [TIMER_OFL], 0   ; AND THEN RESET THE OVERFLOW FLAG
00006348 8B0D[30770100]         <1>         mov     ecx, [TIMER_LH]         ; GET COUNT OF TIME
0000634E C3                     <1>     retn
                                <1>
```

```
                                <1> RTC_10:                         ; SET TIME COUNT
0000634F 890D[30770100]         <1>         mov     [TIMER_LH], ecx         ; SET TIME COUNT
00006355 C605[34770100]00       <1>     mov     byte [TIMER_OFL], 0     ; RESET OVERFLOW FLAG
0000635C C3                     <1>     retn                            ; RETURN WITH NO CARRY
                                <1>
                                <1> RTC_20:                         ; GET RTC TIME
0000635D E8C7010000             <1>     call    UPD_IPR                 ; CHECK FOR UPDATE IN PROCESS
00006362 7226                   <1>     jc      short RTC_29            ; EXIT IF ERROR (CY= 1)
                                <1>
00006364 B000                   <1>     mov     al, CMOS_SECONDS        ; SET ADDRESS OF SECONDS
00006366 E8F4010000             <1>     call    CMOS_READ               ; GET SECONDS
0000636B 88C6                   <1>     mov     dh, al                  ; SAVE
0000636D B00B                   <1>     mov     al, CMOS_REG_B          ; ADDRESS ALARM REGISTER
0000636F E8EB010000             <1>     call    CMOS_READ               ; READ CURRENT VALUE OF DSE BIT
00006374 2401                   <1>     and     al, 00000001b           ; MASK FOR VALID DSE BIT
00006376 88C2                   <1>     mov     dl, al                  ; SET [DL] TO ZERO FOR NO DSE BIT
00006378 B002                   <1>     mov     al, CMOS_MINUTES        ; SET ADDRESS OF MINUTES
0000637A E8E0010000             <1>     call    CMOS_READ               ; GET MINUTES
0000637F 88C1                   <1>     mov     cl, al                  ; SAVE
00006381 B004                   <1>         mov     al, CMOS_HOURS          ; SET ADDRESS OF HOURS
                                <1> RTC_41:             ; 29/07/2022
00006383 E8D7010000             <1>     call    CMOS_READ               ; GET HOURS
00006388 88C5                   <1>     mov     ch, al                  ; SAVE
                                <1>     ; 29/07/2022
                                <1>     ;clc                            ; SET CY= 0
                                <1> RTC_29:
0000638A C3                     <1>     retn                            ; RETURN WITH RESULT IN CARRY FLAG
                                <1>
                                <1> RTC_30:                         ; SET RTC TIME
0000638B E899010000             <1>     call    UPD_IPR                 ; CHECK FOR UPDATE IN PROCESS
00006390 7305                   <1>     jnc     short RTC_35            ; GO AROUND IF CLOCK OPERATING
00006392 E8AD010000             <1>     call    RTC_STA                 ; ELSE TRY INITIALIZING CLOCK
                                <1> RTC_35:
00006397 88F4                   <1>     mov     ah, dh                  ; GET TIME BYTE - SECONDS
00006399 B000                   <1>     mov     al, CMOS_SECONDS        ; ADDRESS SECONDS
0000639B E894000000             <1>     call    CMOS_WRITE              ; UPDATE SECONDS
000063A0 88CC                   <1>     mov     ah, cl                  ; GET TIME BYTE - MINUTES
000063A2 B002                   <1>     mov     al, CMOS_MINUTES        ; ADDRESS MINUTES
000063A4 E88B000000             <1>     call    CMOS_WRITE              ; UPDATE MINUTES
000063A9 88EC                   <1>     mov     ah, ch                  ; GET TIME BYTE - HOURS
000063AB B004                   <1>     mov     al, CMOS_HOURS          ; ADDRESS HOURS
000063AD E882000000             <1>     call    CMOS_WRITE              ; UPDATE ADDRESS
                                <1>     ;mov    al, CMOS_REG_B          ; ADDRESS ALARM REGISTER
                                <1>     ;mov    ah, al
000063B2 66B80B0B               <1>     mov     ax, CMOS_REG_B * 257
000063B6 E8A4010000             <1>     call    CMOS_READ               ; READ CURRENT TIME
000063BB 2462                   <1>     and     al, 01100010b           ; MASK FOR VALID BIT POSITIONS
000063BD 0C02                   <1>     or      al, 00000010b           ; TURN ON 24 HOUR MODE
000063BF 80E201                 <1>     and     dl, 00000001b           ; USE ONLY THE DSE BIT
000063C2 08D0                   <1>     or      al, dl                  ; GET DAY LIGHT SAVINGS TIME BIT (OSE)
000063C4 86E0                   <1>     xchg    ah, al                  ; PLACE IN WORK REGISTER AND GET ADDRESS
                                <1>     ;call   CMOS_WRITE              ; SET NEW ALARM SITS
                                <1>     ;clc                            ; SET CY= 0
                                <1>     ;retn                           ; RETURN WITH CY= 0
                                <1>     ; 29/07/2022
000063C6 EB6C                   <1>     jmp     short CMOS_WRITE
                                <1>
                                <1> RTC_40:                         ; GET RTC DATE
000063C8 E85C010000             <1>     call    UPD_IPR                 ; CHECK FOR UPDATE IN PROCESS
                                <1>     ;jc     short RTC_49            ; EXIT IF ERROR (CY= 1)
                                <1>     ; 07/08/2022
000063CD 72BB                   <1>     jc      short RTC_29
                                <1>
000063CF B007                   <1>     mov     al, CMOS_DAY_MONTH      ; ADDRESS DAY OF MONTH
000063D1 E889010000             <1>     call    CMOS_READ               ; READ DAY OF MONTH
000063D6 88C2                   <1>     mov     dl, al                  ; SAVE
000063D8 B008                   <1>     mov     al, CMOS_MONTH          ; ADDRESS MONTH
000063DA E880010000             <1>     call    CMOS_READ               ; READ MONTH
000063DF 88C6                   <1>     mov     dh, al                  ; SAVE
000063E1 B009                   <1>     mov     al, CMOS_YEAR           ; ADDRESS YEAR
000063E3 E877010000             <1>     call    CMOS_READ               ; READ YEAR
000063E8 88C1                   <1>     mov     cl, al                  ; SAVE
000063EA B032                   <1>     mov     al, CMOS_CENTURY        ; ADDRESS CENTURY LOCATION
                                <1> ; 29/07/2022
                                <1> ;   call    CMOS_READ               ; GET CENTURY BYTE
                                <1> ;   mov     ch, al                  ; SAVE
                                <1> ;   ; 29/07/2022
                                <1> ;   ;clc                            ; SET CY=0
                                <1> ;RTC_49:
                                <1> ;   retn                            ; RETURN WITH RESULTS IN CARRY FLAG
                                <1>
                                <1>     ; 29/07/2022
000063EC EB95                   <1>     jmp     short RTC_41
                                <1>
                                <1>
                                <1> RTC_50:                         ; SET RTC DATE
000063EE E836010000             <1>     call    UPD_IPR                 ; CHECK FOR UPDATE IN PROCESS
000063F3 7305                   <1>     jnc     short RTC_55            ; GO AROUND IF NO ERROR
000063F5 E84A010000             <1>     call    RTC_STA                 ; ELSE INITIALIZE CLOCK
                                <1> RTC_55:
000063FA 66B80600               <1>     mov     ax, CMOS_DAY_WEEK       ; ADDRESS OF DAY OF WEEK BYTE
000063FE E831000000             <1>     call    CMOS_WRITE              ; LOAD ZEROS TO DAY OF WEEK
00006403 88D4                   <1>     mov     ah, dl                  ; GET DAY OF MONTH BYTE
00006405 B007                   <1>     mov     al, CMOS_DAY_MONTH      ; ADDRESS DAY OF MONTH BYTE
00006407 E828000000             <1>     call    CMOS_WRITE              ; WRITE OF DAY OF MONTH REGISTER
0000640C 88F4                   <1>     mov     ah, dh                  ; GET MONTH
0000640E B008                   <1>     mov     al, CMOS_MONTH          ; ADDRESS MONTH BYTE
00006410 E81F000000             <1>     call    CMOS_WRITE              ; WRITE MONTH REGISTER
00006415 88CC                   <1>     mov     ah, cl                  ; GET YEAR BYTE
00006417 B009                   <1>     mov     al, CMOS_YEAR           ; ADDRESS YEAR REGISTER
00006419 E816000000             <1>     call    CMOS_WRITE              ; WRITE YEAR REGISTER
0000641E 88EC                   <1>     mov     ah, ch                  ; GET CENTURY BYTE
00006420 B032                   <1>     mov     al, CMOS_CENTURY        ; ADDRESS CENTURY BYTE
00006422 E80D000000             <1>     call    CMOS_WRITE              ; WRITE CENTURY LOCATION
                                <1>     ;mov    al, CMOS_REG_B          ; ADDRESS ALARM REGISTER
                                <1>     ;mov    ah, al
00006427 66B80B0B               <1>     mov     ax, CMOS_REG_B * 257
0000642B E82F010000             <1>     call    CMOS_READ               ; READ CURRENT SETTINGS
00006430 247F                   <1>     and     al, 07Fh                ; CLEAR 'SET BIT'
00006432 86E0                   <1>     xchg    ah, al                  ; MOVE TO WORK REGISTER
                                <1>     ;call   CMOS_WRITE              ; AND START CLOCK UPDATING
                                <1>     ;clc                            ; SET CY= 0
                                <1>     ;retn                           ; RETURN CY=0
                                <1>     ; 29/07/2022
                                <1>     ;jmp    short CMOS_WRITE
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> ; 08/08/2022
                                <1> ; 29/07/2022 (TRDOS 386 v2.0.5)
                                <1> ; 18/04/2021 (TRDOS 386 v2.0.4)
                                <1> ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> ;--- CMOS_WRITE ---------------------------------------------------------------
```

```
                                <1> ;   WRITE BYTE TO CMOS SYSTEM CLOCK CONFIGURATION TABLE              :
                                <1> ;                                                                   :
                                <1> ; INPUT: (AL)=     CMOS TABLE ADDRESS TO BE WRITTEN TO                    :
                                <1> ;           BIT    7 = 0 FOR NMI ENABLED AND 1 FOR NMI DISABLED ON EXIT    :
                                <1> ;           BITS 6-0 = ADDRESS OF TABLE LOCATION TO WRITE                  :
                                <1> ;     (AH)= NEW VALUE TO BE PLACED IN THE ADDRESSED TABLE LOCATION         :
                                <1> ;                                                                   :
                                <1> ; OUTPUT:   VALUE IN (AH) PLACED IN LOCATION (AL) WITH NMI LEFT DISABLED   :
                                <1> ;           IF BIT 7 OF (AL) IS ON, DURING THE CMOS UPDATE BOTH NMI AND    :
                                <1> ;           NORMAL INTERRUPTS ARE DISABLED TO PROTECT CMOS DATA INTEGRITY. :
                                <1> ;           THE CMOS ADDRESS REGISTER IS POINTED TO A DEFAULT VALUE AND    :
                                <1> ;           THE INTERRUPT FLAG RESTORED TO THE ENTRY STATE ON RETURN.      :
                                <1> ;           ONLY THE CMOS LOCATION AND THE NMI STATE IS CHANGED.              :
                                <1> ;-------------------------------------------------------------------------
                                <1>
                                <1>     ; 08/08/2022
                                <1> CMOS_WRITE:                         ; WRITE (AH) TO LOCATION (AL)
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ;pushf                          ; SAVE INTERRUPT ENABLE STATUS AND FLAGS
                                <1>     ;;push ax                       ; SAVE WORK REGISTER VALUES
                                <1>     ; 18/04/2021
                                <1>     ;push  eax
00006434 D0C0                   <1>     rol    al, 1                    ; MOVE NMI BIT TO LOW POSITION
00006436 F9                     <1>     stc                             ; FORCE NMI BIT ON IN CARRY FLAG
00006437 D0D8                   <1>     rcr    al, 1                    ; HIGH BIT ON TO DISABLE NMI - OLD IN CY
00006439 FA                     <1>     cli                             ; DISABLE INTERRUPTS
0000643A E670                   <1>     out    CMOS_PORT, al            ; ADDRESS LOCATION AND DISABLE NMI
0000643C 88E0                   <1>     mov    al, ah                   ; GET THE DATA BYTE TO WRITE
0000643E E671                   <1>     out    CMOS_DATA, al            ; PLACE IN REQUESTED CMOS LOCATION
00006440 B01E                   <1>     mov    al, CMOS_SHUT_DOWN*2     ; GET ADDRESS OF DEFAULT LOCATION
                                <1>     ;mov   al, CMOS_REG_D*2         ; GET ADDRESS OF DEFAULT LOCATION
00006442 D0D8                   <1>     rcr    al, 1                    ; PUT ORIGINAL NMI MASK BIT INTO ADDRESS
00006444 E670                   <1>     out    CMOS_PORT, al            ; SET DEFAULT TO READ ONLY REGISTER
                                <1>     ;nop                            ; I/O DELAY
                                <1>     ; 29/07/2022
00006446 E6EB                   <1>     out    0EBh, al ; NEWIODELAY    ; AWARD BIOS 1999, ATIME.ASM
00006448 E471                   <1>     in     al, CMOS_DATA            ; OPEN STANDBY LATCH
                                <1>     ;;pop  ax                       ; RESTORE WORK REGISTERS
                                <1>     ; 18/04/2021
                                <1>     ;pop   eax
                                <1>     ;popf
                                <1>     ; 29/07/2022
                                <1>     ;clc
                                <1>     ; 08/08/2022
0000644A 30C0                   <1>     xor    al, al
0000644C C3                     <1>     retn
                                <1>
                                <1> ;-------------------------------------------------------------------------
                                <1>
                                <1> RTC_60:                             ; SET RTC ALARM
0000644D B00B                   <1>     mov    al, CMOS_REG_B           ; ADDRESS ALARM
0000644F E80B010000             <1>     call   CMOS_READ                ; READ ALARM REGISTER
00006454 A820                   <1>     test   al, 20h                  ; CHECK FOR ALARM ALREADY ENABLED
00006456 F9                     <1>     stc                             ; SET CARRY IN CASE OF ERROR
00006457 7541                   <1>     jnz    short RTC_69             ; ERROR EXIT IF ALARM SET
00006459 E8CB000000             <1>     call   UPD_IPR                  ; CHECK FOR UPDATE IN PROCESS
0000645E 7305                   <1>     jnc    short RTC_65             ; SKIP INITIALIZATION IF NO ERROR
00006460 E8DF000000             <1>     call   RTC_STA                  ; ELSE INITIALIZE CLOCK
                                <1> RTC_65:
00006465 88F4                   <1>     mov    ah, dh                   ; GET SECONDS BYTE
00006467 B001                   <1>     mov    al, CMOS_SEC_ALARM       ; ADDRESS THE SECONDS ALARM REGISTER
00006469 E8C6FFFFFF             <1>     call   CMOS_WRITE               ; INSERT SECONDS
0000646E 88CC                   <1>     mov    ah, cl                   ; GET MINUTES PARAMETER
00006470 B003                   <1>     mov    al, CMOS_MIN_ALARM       ; ADDRESS MINUTES ALARM REGISTER
00006472 E8BDFFFFFF             <1>     call   CMOS_WRITE               ; INSERT MINUTES
00006477 88EC                   <1>     mov    ah, ch                   ; GET HOURS PARAMETER
00006479 B005                   <1>     mov    al, CMOS_HR_ALARM        ; ADDRESS HOUR ALARM REGISTER
0000647B E8B4FFFFFF             <1>     call   CMOS_WRITE               ; INSERT HOURS
00006480 E4A1                   <1>     in     al, INTB01               ; READ SECOND INTERRUPT MASK REGISTER
00006482 24FE                   <1>     and    al, 0FEh                 ; ENABLE ALARM TIMER BIT (CY= 0)
00006484 E6A1                   <1>     out    INTB01, al               ; WRITE UPDATED MASK
                                <1>     ;mov   al, CMOS_REG_B           ; ADDRESS ALARM REGISTER
                                <1>     ;mov   ah, al
00006486 66B80B0B               <1>     mov    ax, CMOS_REG_B * 257
0000648A E8D0000000             <1>     call   CMOS_READ                ; READ CURRENT ALARM REGISTER
0000648F 247F                   <1>     and    al, 07Fh                 ; ENSURE SET BIT TURNED OFF
00006491 0C20                   <1>     or     al, 20h                  ; TURN ON ALARM ENABLE
00006493 86E0                   <1>     xchg   ah, al                   ; MOVE MASK TO OUTPUT REGISTER
00006495 E89AFFFFFF             <1>     call   CMOS_WRITE               ; WRITE NEW ALARM MASK
                                <1>     ; 29/07/2022
                                <1>     ;clc                            ; SET CY= 0
                                <1> RTC_69:
0000649A 66B80000               <1>     mov    ax, 0                    ; CLEAR AX REGISTER
0000649E C3                     <1>     retn                            ; RETURN WITH RESULTS IN CARRY FLAC
                                <1>
                                <1> RTC_70:                             ; RESET ALARM
                                <1>     ;mov   al, CMOS_REG_B           ; ADDRESS ALARM REGISTER
                                <1>     ;mov   ah, al
0000649F 66B80B0B               <1>     mov    ax, CMOS_REG_B * 257     ; ADDRESS ALARM REGISTER (TO BOTH AH,AL)
000064A3 E8B7000000             <1>     call   CMOS_READ                ; READ ALARM REGISTER
000064A8 2457                   <1>     and    al, 57h                  ; TURN OFF ALARM ENABLE
000064AA 86E0                   <1>     xchg   ah, al                   ; SAVE DATA AND RECOVER ADDRESS
                                <1>     ;call  CMOS_WRITE               ; RESTORE NEW VALUE
                                <1>     ;clc                            ; SET CY= 0
                                <1>     ;retn                           ; RETURN WITH NO CARRY
                                <1>     ; 29/07/2022
000064AC EB86                   <1>     jmp    short CMOS_WRITE
                                <1>
                                <1> ;-------------------------------------------------------------------------
                                <1>
                                <1> ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>
                                <1> ;--- HARDWARE INT 70 H -- ( IRQ LEVEL 8) ---------------------------------------
                                <1> ; ALARM INTERRUPT HANDLER (RTC)                                          :
                                <1> ;       THIS ROUTINE HANDLES THE PERIODIC AND ALARM INTERRUPTS FROM THE CMOS  :
                                <1> ;       TIMER. INPUT FREQUENCY IS 1.024 KHZ OR APPROXIMATELY 1024 INTERRUPTS  :
                                <1> ;       EVERY SECOND FOR THE PERIODIC INTERRUPT. FOR THE ALARM FUNCTION,      :
                                <1> ;       THE INTERRUPT WILL OCCUR AT THE DESIGNATED TIME.                      :
                                <1> ;                                                                   :
                                <1> ;       INTERRUPTS ARE ENABLED WHEN THE EVENT OR ALARM FUNCTION IS ACTIVATED. :
                                <1> ;       FOR THE EVENT INTERRUPT, THE HANDLER WILL DECREMENT THE WAIT COUNTER  :
                                <1> ;       AND WHEN IT EXPIRES WILL SET THE DESIGNATED LOCATION TO 80H. FOR      :
                                <1> ;       THE ALARM INTERRUPT. THE USER MUST PROVIDE A ROUTINE TO INTERCEPT     :
                                <1> ;       THE CORRECT ADDRESS FROM THE VECTOR TABLE INVOKED BY INTERRUPT 4AH    :
                                <1> ;       PRIOR TO SETTING THE REAL TIME CLOCK ALARM (INT 1AH, AH= 06H).        :
                                <1> ;-------------------------------------------------------------------------
                                <1>
                                <1> RTC_A_INT: ; 07/01/2017
                                <1> ;RTC_INT:                           ; ALARM INTERRUPT
000064AE 1E                     <1>     push   ds                       ; LEAVE INTERRUPTS DISABLED
000064AF 50                     <1>     push   eax                      ; SAVE REGISTERS
000064B0 57                     <1>     push   edi
                                <1> RTC_I_1:                            ; CHECK FOR SECOND INTERRUPT
000064B1 66B88C8B               <1>     mov    ax, 256*(CMOS_REG_B+NMI)+CMOS_REG_C+NMI ; ALARM AND STATUS
```

```
000064B5 E670                <1>     out     CMOS_PORT, al          ; WRITE ALARM FLAG MASK ADDRESS
000064B7 90                  <1>     nop                            ; I/O DELAY
000064B8 EB00                <1>     jmp     short $+2
000064BA E471                <1>     in      al, CMOS_DATA          ; READ AND RESET INTERRUPT REQUEST FLAGS
000064BC A860                <1>     test    al, 01100000b          ; CHECK FOR EITHER INTERRUPT PENDING
000064BE 745B                <1>     jz      short  RTC_I_9         ; EXIT IF NOT A VALID RTC INTERRUPT
                             <1>
000064C0 86E0                <1>     xchg    ah, al                 ; SAVE FLAGS AND GET ENABLE ADDRESS
000064C2 E670                <1>     out     CMOS_PORT, al          ; WRITE ALARM ENABLE MASK ADDRESS
000064C4 90                  <1>     nop                            ; I/O DELAY
000064C5 EB00                <1>     jmp     short $+2
000064C7 E471                <1>     in      al, CMOS_DATA          ; READ CURRENT ALARM ENABLE MASK
000064C9 20E0                <1>     and     al, ah                 ; ALLOW ONLY SOURCES THAT ARE ENABLED
000064CB A840                <1>     test    al, 01000000b          ; CHECK FOR PERIODIC INTERRUPT
000064CD 7439                <1>     jz      short RTC_I_5          ; SKIP IF NOT A PERIODIC INTERRUPT
                             <1>
                             <1> ;-----      DECREMENT WAIT COUNT BY INTERRUPT INTERVAL
                             <1>
000064CF 66BF1000            <1>     mov     di, KDATA              ; kernel data segment
000064D3 8EDF                <1>     mov     ds, di
                             <1>
000064D5 812D[28770100]D003- <1>     sub     dword [RTC_LH], 976    ; DECREMENT COUNT BY 1/1024
000064DD 0000                <1>
000064DF 7327                <1>     jnc     short RTC_I_5          ; SKIP TILL 32 BIT WORD LESS THAN ZERO
                             <1>
                             <1> ;-----      TURN OFF PERIODIC INTERRUPT ENABLE
                             <1>
                             <1>     ;push   ax                     ; SAVE INTERRUPT FLAG MASK
                             <1>     ; 18/04/2021
000064E1 50                  <1>     push    eax
000064E2 66B88B8B            <1>     mov     ax, 257*(CMOS_REG_B+NMI) ; INTERRUPT ENABLE REGISTER
000064E6 E670                <1>     out     CMOS_PORT, al          ; WRITE ADDRESS TO CMOS CLOCK
000064E8 90                  <1>     nop                            ; I/O DELAY
000064E9 EB00                <1>     jmp     short $+2
000064EB E471                <1>     in      al, CMOS_DATA          ; READ CURRENT ENABLES
000064ED 24BF                <1>     and     al, 0BFh               ; TURN OFF PIE
000064EF 86C4                <1>     xchg    al, ah                 ; GET CMOS ADDRESS AND SAVE VALUE
000064F1 E670                <1>     out     CMOS_PORT, al          ; ADDRESS REGISTER B
000064F3 86C4                <1>     xchg    al, ah                 ; GET NEW INTERRUPT ENABLE MASK
000064F5 E671                <1>     out     CMOS_DATA, al          ; SET MASK IN INTERRUPT ENABLE REGISTER
000064F7 C605[2C770100]00    <1>     mov     byte [RTC_WAIT_FLAG], 0       ; SET FUNCTION ACTIVE FLAG OFF
000064FE 8B3D[2D770100]      <1>     mov     edi, [USER_FLAG]       ; SET UP (DS:DI) TO POINT TO USER FLAG
00006504 C60780              <1>     mov     byte [edi], 80h               ; TURN ON USERS FLAG
                             <1>     ;pop    ax                     ; GET INTERRUPT SOURCE BACK
                             <1>     ; 18/04/2021
00006507 58                  <1>     pop     eax
                             <1> RTC_I_5:
00006508 A820                <1>     test    al, 00100000b          ; TEST FOR ALARM INTERRUPT
0000650A 740D                <1>     jz      short RTC_I_7          ; SKIP USER INTERRUPT CALL IF NOT ALARM
                             <1>
0000650C B00D                <1>     mov     al, CMOS_REG_D         ; POINT TO DEFAULT READ ONLY REGISTER
0000650E E670                <1>     out     CMOS_PORT, al          ; ENABLE NMI AND CMOS ADDRESS TO DEFAULT
00006510 FB                  <1>     sti                            ; INTERRUPTS BACK ON NOW
00006511 52                  <1>     push    edx
00006512 E8E7B60000          <1>     call    INT4Ah                 ; TRANSFER TO USER ROUTINE
00006517 5A                  <1>     pop     edx
00006518 FA                  <1>     cli                            ; BLOCK INTERRUPT FOR RETRY
                             <1> RTC_I_7:                           ; RESTART ROUTINE TO HANDLE DELAYED
00006519 EB96                <1>     jmp     short RTC_I_1          ;  ENTRY AND SECOND EVENT BEFORE DONE
                             <1>
                             <1> RTC_I_9:                           ; EXIT - NO PENDING INTERRUPTS
0000651B B00D                <1>     mov     al, CMOS_REG_D         ; POINT TO DEFAULT READ ONLY REGISTER
0000651D E670                <1>     out     CMOS_PORT, al          ; ENABLE NMI AND CMOS ADDRESS TO DEFAULT
0000651F B020                <1>     mov     al, EOI                ; END OF INTERRUPT MASK TO 8259 - 2
00006521 E6A0                <1>     out     INTB00, al             ; TO 8259 - 2
00006523 E620                <1>     out     INTA00,al              ; TO 8259 - 1
00006525 5F                  <1>     pop     edi                    ; RESTORE REGISTERS
00006526 58                  <1>     pop     eax
00006527 1F                  <1>     pop     ds
00006528 CF                  <1>     iretd                          ; END OF INTERRUPT
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1>
                             <1>     ; 29/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 22/08/2014 (Retro UNIX 386 v1)
                             <1>     ; IBM PC/AT BIOS source code ----- 10/06/85 (bios2.asm)
                             <1> UPD_IPR:                           ; WAIT TILL UPDATE NOT IN PROGRESS
00006529 51                  <1>     push    ecx
                             <1>
                             <1>     ; 29/05/2016
0000652A B968110000          <1>     mov     ecx, ((1984+244)*4)/2 ; AWARD BIOS 1999, ATIME.ASM
                             <1>                                   ; 'WAITCPU_CK_UD_STAT'
                             <1>                                   ; (244Us + 1984Us)
                             <1>                                   ; (assume each read takes
                             <1>                                   ;  2 microseconds).
                             <1>     ;mov    ecx, 65535
                             <1>             ;mov cx, 800          ; SET TIMEOUT LOOP COUNT (= 800)
                             <1> UPD_10:
0000652F B00A                <1>     mov     al, CMOS_REG_A         ; ADDRESS STATUS REGISTER A
00006531 FA                  <1>     cli                            ; NO TIMER INTERRUPTS DURING UPDATES
00006532 E828000000          <1>     call    CMOS_READ              ; READ UPDATE IN PROCESS FLAG
00006537 A880                <1>     test    al, 80h                ; IF UIP BIT IS ON ( CANNOT READ TIME )
00006539 7406                <1>     jz      short UPD_90           ; EXIT WITH CY= 0 IF CAN READ CLOCK NOW
0000653B FB                  <1>     sti                            ; ALLOW INTERRUPTS WHILE WAITING
0000653C E2F1                <1>     loop    UPD_10                 ; LOOP TILL READY OR TIMEOUT
0000653E 31C0                <1>     xor     eax, eax ; xor ax, ax  ; CLEAR RESULTS IF ERROR
00006540 F9                  <1>     stc                            ; SET CARRY FOR ERROR
                             <1> UPD_90:
00006541 59                  <1>     pop     ecx                    ; RESTORE CALLERS REGISTER
00006542 FA                  <1>     cli                            ; INTERRUPTS OFF DURING SET
00006543 C3                  <1>     retn                           ; RETURN WITH CY FLAG SET
                             <1>
                             <1>     ; 18/04/2021
                             <1> RTC_STA:                   ; INITIALIZE REAL TIME CLOCK
                             <1>     ;mov    al, CMOS_REG_A         ; ADDRESS REGISTER A AND LOAD DATA MASK
                             <1>     ;mov    ah, 26h
00006544 66B80A26            <1>     mov     ax, (26h*100h)+CMOS_REG_A
00006548 E8E7FEFFFF          <1>     call    CMOS_WRITE             ; INITIALIZE STATUS REGISTER A
                             <1>     ;mov    al, CMOS_REG_B         ; SET "SET BIT" FOR CLOCK INITIALIZATION
                             <1>     ;mov    ah, 82h
0000654D 66B80B82            <1>     mov     ax, (82h*100h)+CMOS_REG_B
00006551 E8DEFEFFFF          <1>     call    CMOS_WRITE             ; AND 24 HOUR MODE TO REGISTER B
00006556 B00C                <1>     mov     al, CMOS_REG_C         ; ADDRESS REGISTER C
00006558 E802000000          <1>     call    CMOS_READ              ; READ REGISTER C TO INITIALIZE
0000655D B00D                <1>     mov     al, CMOS_REG_D         ; ADDRESS REGISTER D
                             <1>     ; 18/04/2021
                             <1>     ;call   CMOS_READ              ; READ REGISTER D TO INITIALIZE
                             <1>     ;retn
                             <1>     ;jmp    short CMOS_READ ; 18/04/2021
                             <1>
                             <1> ;--------------------------------------------------------------------------------
                             <1>
                             <1>     ; 29/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 18/04/2021 - TRDOS 386 v2.0.4
```

```
                                <1>     ; 29/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 22/08/2014 (Retro UNIX 386 v1)
                                <1>     ; IBM PC/AT BIOS source code ----- 10/06/85 (test4.asm)
                                <1>
                                <1> ;--- CMOS_READ ------------------------------------------------------------
                                <1> ;           READ BYTE FROM CMOS_SYSTEM CLOCK CONFIGURATION TABLE              :
                                <1> ;                                                                             :
                                <1> ; INPUT: (AL)=     CMOS_TABLE ADDRESS TO BE READ                              :
                                <1> ;           BIT    7 = 0 FOR NMI ENABLED AND 1 FOR NMI DISABLED ON EXIT    :
                                <1> ;           BITS 6-0 = ADDRESS OF TABLE LOCATION TO READ                :
                                <1> ;                                                                       :
                                <1> ; OUTPUT: (AL)     VALUE AT LOCATION (AL) MOVED INTO (AL). IF BIT 7 OF (AL) WAS   :
                                <1> ;           ON THEN NMI LEFT DISABLED, DURING THE CMOS READ BOTH NMI AND   :
                                <1> ;           NORMAL INTERRUPTS ARE DISABLED TO PROTECT CMOS DATA INTEGRITY. :
                                <1> ;           THE CMOS ADDRESS REGISTER IS POINTED TO A DEFAULT VALUE AND    :
                                <1> ;           THE INTERRUPT FLAG RESTORED TO THE ENTRY STATE ON RETURN.      :
                                <1> ;           ONLY THE (AL) REGISTER AND THE NMI STATE IS CHANGED.              :
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> CMOS_READ:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ;pushf                          ; SAVE INTERRUPT ENABLE STATUS AND FLAGS
0000655F D0C0                   <1>     rol     al, 1                   ; MOVE NMI BIT TO LOW POSITION
00006561 F9                     <1>     stc                             ; FORCE NMI BIT ON IN CARRY FLAG
00006562 D0D8                   <1>     rcr     al, 1                   ; HIGH BIT ON TO DISABLE NMI - OLD IN CY
00006564 FA                     <1>     cli                             ; DISABLE INTERRUPTS
00006565 E670                   <1>     out     CMOS_PORT, al           ; ADDRESS LOCATION AND DISABLE NMI
                                <1>     ; 29/05/2016
                                <1>     ;nop                            ; I/O DELAY
00006567 E6EB                   <1>     out     0EBh, al ; NEWIODELAY ; AWARD BIOS 1999, ATIME.ASM
                                <1>     ;
00006569 E471                   <1>     in      al, CMOS_DATA           ; READ THE REQUESTED CMOS LOCATION
                                <1>     ;push   ax      ; SAVE (AH) REGISTER VALUE AND CMOS BYTE
                                <1>     ; 18/04/2021
0000656B 50                     <1>     push    eax
                                <1>     ; 15/03/2015 ; IBM PC/XT Model 286 BIOS source code
                                <1>                  ; ----- 10/06/85 (test4.asm)
0000656C B01E                   <1>     mov     al, CMOS_SHUT_DOWN*2  ; GET ADDRESS OF DEFAULT LOCATION
                                <1>     ;mov    al, CMOS_REG_D*2      ; GET ADDRESS OF DEFAULT LOCATION
0000656E D0D8                   <1>     rcr     al, 1                 ; PUT ORIGINAL NMI MASK BIT INTO ADDRESS
00006570 E670                   <1>     out     CMOS_PORT, al         ; SET DEFAULT TO READ ONLY REGISTER
                                <1>     ;pop    ax                    ; RESTORE (AH) AND (AL), CMOS BYTE
                                <1>     ; 18/04/2021
00006572 58                     <1>     pop     eax
                                <1>     ; 29/07/2022
                                <1>     ;popf
00006573 F8                     <1>     clc     ; 29/07/2022
00006574 C3                     <1>     retn                          ; RETURN WITH FLAGS RESTORED
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> ; /// End Of TIMER FUNCTIONS ///

00006575 90<rep 8h>                 Align 16

                                    gdt:; Global Descriptor Table
                                        ; 02/12/2020
                                        ; (30/07/2015, conforming cs)
                                        ; (26/03/2015)
                                        ; (24/03/2015, tss)
                                        ; (19/03/2015)
                                        ; (29/12/2013)
                                        ;
00006580 0000000000000000               dw 0, 0, 0, 0         ; NULL descriptor
                                    gdt_kcode:
                                        ; 18/08/2014
                                                        ; 8h kernel code segment, base = 00000000h
                                        ;dw 0FFFFh, 0, 9E00h, 00CFh   ; KCODE  ; 30/12/2016
00006588 FFFF0000009ACF00               dw 0FFFFh, 0, 9A00h, 00CFh    ; KCODE
                                    gdt_kdata:
                                                        ; 10h kernel data segment, base = 00000000h
00006590 FFFF00000092CF00               dw 0FFFFh, 0, 9200h, 00CFh    ; KDATA
                                    gdt_ucode:
                                                        ; 1Bh user code segment, base address = 400000h ; CORE
                                        ;dw 0FBFFh, 0, 0FE40h, 00CFh  ; UCODE  ; 30/12/2016
00006598 FFFB000040FACF00               dw 0FBFFh, 0, 0FA40h, 00CFh   ; UCODE
                                    gdt_udata:
                                                        ; 23h user data segment, base address = 400000h ; CORE
000065A0 FFFB000040F2CF00               dw 0FBFFh, 0, 0F240h, 00CFh   ; UDATA
                                    gdt_tss:
                                                        ; Task State Segment
000065A8 6700                           dw 0067h ; Limit = 103 ; (104-1, tss size = 104 byte,
                                                                 ;  no IO permission in ring 3)
                                    gdt_tss0:
000065AA 0000                           dw 0  ; TSS base address, bits 0-15
                                    gdt_tss1:
000065AC 00                             db 0  ; TSS base address, bits 16-23
                                                        ; 49h
000065AD E9                             db 11101001b ; 0E9h => P=1/DPL=11/0/1/0/B/1 --> B = Task is busy (1)
000065AE 00                             db 0 ; G/0/0/AVL/LIMIT=0000 ; (Limit bits 16-19 = 0000) (G=0, 1 byte)
                                    gdt_tss2:
000065AF 00                             db 0  ; TSS base address, bits 24-31

                                        ; 04/12/2023 - TRDOS v2.0.7
                                        ; 30/11/2020
                                        ; 29/11/2020 - TRDOS v2.0.3
                                        ; VESA VBE3 VIDE BIOS 32 BIT PMI SEGMENTS (16 bit segments)
                                                        ; 30h ; VBE3CS
                                    _vbe3_CS:  ; vesa vbe3 bios uses this as code seg (same addr with _vbe3_DS)
                                        ; limit = 65536, base addr = 0, P/DPL/1/Type/C/R/A = 9Ah, 16 bit
                                        ;dw 0FFFFh, 0, 9A00h, 0 ; Note: base addr will be initialized
                                        ; 04/12/2023 - TRDOS 386 v2.0.7      ; (+!*!+)
000065B0 FFFF0000069A0000               dw 0FFFFh, 0, 9A06h, 0; VBE3BIOSCODE_ADDR = 60000h
                                                        ; 38h ; VBE3BDS
                                    _vbe3_BDS: ; vesa vbe3 bios uses this as equivalent of rombios data segment
                                        ; limit = 1536, base addr = 0, P/DPL/1/Type/E/W/A = 92h, 16 bit
000065B8 FF05000000920000               dw 05FFh, 0, 9200h, 0 ; Note: base addr will be initialized
                                                        ; 40h ; VBE3A000
                                    _A0000Sel: ; VGA default video memory address
                                        ; limit = 65536, base addr = 0A0000h, 16 bit
000065C0 FFFF00000A920000               dw 0FFFFh, 0, 920Ah, 0
                                                        ; 48h ; VBE3B000
                                    _B0000Sel: ; MDA (monochrome) video memory address
                                        ; limit = 65536, base addr = 0B0000h, 16 bit
000065C8 FFFF00000B920000               dw 0FFFFh, 0, 920Bh, 0
                                                        ; 50h ; VBE3B800
                                    _B8000Sel: ; CGA video memory address
                                        ; limit = 32768, base addr = 0B8000h, 16 bit
000065D0 FF7F00800B920000               dw 07FFFh, 8000h, 920Bh, 0
                                                        ; 58h ; VBE3DS
                                    _vbe3_DS: ; vesa vbe3 bios uses this as data seg (CodeSegSel in PMInfoBlock)
                                        ; limit = 65536, base addr = 0, P/DPL/1/Type/E/W/A = 92h, 16 bit
                                        ;dw 0FFFFh, 0, 9200h, 0 ; Note: base addr will be initialized
                                        ; 04/12/2023 - TRDOS 386 v2.0.7      ; (+!*!+)
```

```
000065D8 FFFF000006920000            dw 0FFFFh, 0, 9206h, 0; VBE3BIOSCODE_ADDR = 60000h
                                                    ; 60h ; VBE3SS
                                 _vbe3_SS: ; kernel's stack segment but 16 bit version (same stack addr)
                                     ; limit = 1024, base addr = 0, P/DPL/1/Type/E/W/A = 92h, 16 bit
000065E0 FF03000000920000            dw 03FFh, 0, 9200h, 0 ; Note: base addr will be initialized
                                                    ; 68h ; VBE3ES
                                 _vbe3_ES: ; extra 16 bit segment points to buffers in kernel's mem space
                                     ; limit = 2048, base addr = 0, P/DPL/1/Type/E/W/A = 92h, 16 bit
000065E8 FF07000000920000            dw 07FFh, 0, 9200h,0 ; Note: base addr will be initialized
                                                    ; 70h ; KODE16
                                 _16bit_CS: ; 16 bit code segment points to kernel's far return addr
                                     ; limit = 16M, base addr = 0, P/DPL/1/Type/E/W/A = 92h, 16 bit
000065F0 FFFF0000009AFF00            dw 0FFFFh, 0, 9A00h, 00FFh ; Note: base addr will be initialized

                                 gdt_end:
                                     ;; 9Eh = 1001 1110b (GDT byte 5) P=1/DPL=00/1/TYPE=1110,
                                                                ;; Type= 1 (code)/C=1/R=1/A=0
                                             ; P= Present, DPL=0=ring 0,  1= user (0= system)
                                             ; 1= Code C= Conforming, R= Readable, A= Accessed

                                     ;; 9Ah = 1001 1010b (GDT byte 5) P=1/DPL=00/1/TYPE=1010,
                                                                ;; Type= 1 (code)/C=0/R=1/A=0
                                             ; P= Present, DPL=0=ring 0,  1= user (0= system)
                                             ; 1= Code C= non-Conforming, R= Readable, A= Accessed

                                     ;; 92h = 1001 0010b (GDT byte 5) P=1/DPL=00/1/TYPE=1010,
                                                                ;; Type= 0 (data)/E=0/W=1/A=0
                                             ; P= Present, DPL=0=ring 0,  1= user (0= system)
                                             ; 0= Data E= Expansion direction (1= down, 0= up)
                                             ; W= Writeable, A= Accessed

                                     ;; FEh = 1111 1110b (GDT byte 5) P=1/DPL=11/1/TYPE=1110,
                                                                ;; Type= 1 (code)/C=1/R=1/A=0
                                             ; P= Present, DPL=3=ring 3,  1= user (0= system)
                                             ; 1= Code C= Conforming, R= Readable, A= Accessed

                                     ;; FAh = 1111 1010b (GDT byte 5) P=1/DPL=11/1/TYPE=1010,
                                                                ;; Type= 1 (code)/C=0/R=1/A=0
                                             ; P= Present, DPL=3=ring 3,  1= user (0= system)
                                             ; 1= Code C= non-Conforming, R= Readable, A= Accessed

                                     ;; F2h = 1111 0010b (GDT byte 5) P=1/DPL=11/1/TYPE=0010,
                                                                ;; Type= 0 (data)/E=0/W=1/A=0
                                             ; P= Present, DPL=3=ring 3,  1= user (0= system)
                                             ; 0= Data E= Expansion direction (1= down, 0= up)

                                     ;; CFh = 1100 1111b (GDT byte 6) G=1/B=1/0/AVL=0, Limit=1111b (3)

                                             ;; Limit = FFFFFh (=> FFFFFh+1= 100000h) // bits 0-15, 48-51 //
                                             ;       = 100000h * 1000h (G=1) = 4GB
                                             ;; Limit = FFBFFh (=> FFBFFh+1= FFC00h) // bits 0-15, 48-51 //
                                             ;       = FFC00h * 1000h (G=1) = 4GB - 4MB
                                             ; G= Granularity (1= 4KB), B= Big (32 bit),
                                             ; AVL= Available to programmers

                                 gdtd:
000065F8 7700                            dw gdt_end - gdt - 1    ; Limit (size)
000065FA [80650000]                      dd gdt                  ; Address of the GDT

                                     ; 20/08/2014
                                 idtd:
000065FE 7F02                            dw idt_end - idt - 1    ; Limit (size)
00006600 [C8730100]                     dd idt                  ; Address of the IDT

                                 ; 20/02/2017
                                 ;;; 11/03/2015
                                 %include 'diskdata.s'     ; DISK (BIOS) DATA (initialized)
                             <1> ; ****************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - diskdata.s
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Last Update: 06/08/2022 (Previous: 24/01/2016)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Beginning: 24/01/2016
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.15 (trdos386.s)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Turkish Rational DOS
                             <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                             <1> ;
                             <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                             <1> ; diskdata.inc (11/03/2015)
                             <1> ;
                             <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                             <1> ; ****************************************************************************
                             <1>
                             <1> ; Retro UNIX 386 v1 Kernel - DISKDATA.INC
                             <1> ; Last Modification: 11/03/2015
                             <1> ;   (Initialized Disk Parameters Data section for 'DISKIO.INC')
                             <1> ;
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   80286 INTERRUPT LOCATIONS     :
                             <1> ;   REFERENCED BY POST & BIOS     :
                             <1> ;----------------------------------------
                             <1>
00006604 [67660000]          <1> DISK_POINTER:      dd      MD_TBL6        ; Pointer to Diskette Parameter Table
                             <1>
                             <1> ; IBM PC-XT Model 286 source code ORGS.ASM (06/10/85) - 14/12/2014
                             <1> ;---------------------------------------------------------------
                             <1> ; DISK_BASE                                                     :
                             <1> ;   THIS IS THE SET OF PARAMETERS REQUIRED FOR                  :
                             <1> ;   DISKETTE OPERATION. THEY ARE POINTED AT BY THE                :
                             <1> ;   DATA VARIABLE @DISK_POINTER. TO MODIFY THE PARAMETERS,        :
                             <1> ;   BUILD ANOTHER PARAMETER BLOCK AND POINT AT IT                 :
                             <1> ;---------------------------------------------------------------
                             <1>
                             <1> ;DISK_BASE:
                             <1> ;   DB      11011111B       ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
                             <1> ;   DB      2               ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
                             <1> ;   DB      MOTOR_WAIT      ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
                             <1> ;   DB      2               ; 512 BYTES/SECTOR
                             <1> ;   ;DB     15              ; EOT (LAST SECTOR ON TRACK)
                             <1> ;   db      18              ; (EOT for 1.44MB diskette)
                             <1> ;   DB      01BH            ; GAP LENGTH
                             <1> ;   DB      0FFH            ; DTL
                             <1> ;   ;DB     054H            ; GAP LENGTH FOR FORMAT
                             <1> ;   db      06ch            ; (for 1.44MB dsikette)
                             <1> ;   DB      0F6H            ; FILL BYTE FOR FORMAT
                             <1> ;   DB      15              ; HEAD SETTLE TIME (MILLISECONDS)
                             <1> ;   DB      8               ; MOTOR START TIME (1/8 SECONDS)
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   ROM BIOS DATA AREAS           :
                             <1> ;----------------------------------------
```

```
                                <1>
                                <1> ;DATA               SEGMENT AT 40H        ; ADDRESS= 0040:0000
                                <1>
                                <1> ;@EQUIP_FLAG        DW      ?             ; INSTALLED HARDWARE FLAGS
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   DISKETTE DATA AREAS           :
                                <1> ;----------------------------------------
                                <1>
                                <1> ;@SEEK_STATUS       DB      ?             ; DRIVE RECALIBRATION STATUS
                                <1> ;                                         ; BIT 3-0 = DRIVE 3-0 RECALIBRATION
                                <1> ;                                         ; BEFORE NEXT SEEK IF BIT IS = 0
                                <1> ;@MOTOR_STATUS      DB      ?             ; MOTOR STATUS
                                <1> ;                                         ; BIT 3-0 = DRIVE 3-0 CURRENTLY RUNNING
                                <1> ;                                         ; BIT 7 = CURRENT OPERATION IS A WRITE
                                <1> ;@MOTOR_COUNT       DB      ?             ; TIME OUT COUNTER FOR MOTOR(S) TURN OFF
                                <1> ;@DSKETTE_STATUS DB         ?             ; RETURN CODE STATUS BYTE
                                <1> ;                                         ; CMD_BLOCK  IN STACK FOR DISK OPERATION
                                <1> ;@NEC_STATUS        DB      7 DUP(?)      ; STATUS BYTES FROM DISKETTE OPERATION
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   POST AND BIOS WORK DATA AREA  :
                                <1> ;----------------------------------------
                                <1>
                                <1> ;@INTR_FLAG DB      ?             ; FLAG INDICATING AN INTERRUPT HAPPENED
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   TIMER DATA AREA               :
                                <1> ;----------------------------------------
                                <1>
                                <1> ; 17/12/2014  (IRQ 0 - INT 08H)
                                <1> ;TIMER_LOW  equ     46Ch          ; Timer ticks (counter)  @ 40h:006Ch
                                <1> ;TIMER_HIGH equ     46Eh          ; (18.2 timer ticks per second)
                                <1> ;TIMER_OFL  equ     470h          ; Timer - 24 hours flag  @ 40h:0070h
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   ADDITIONAL MEDIA DATA         :
                                <1> ;----------------------------------------
                                <1>
                                <1> ;@LASTRATE  DB      ?             ; LAST DISKETTE DATA RATE SELECTED
                                <1> ;@DSK_STATE DB      ?             ; DRIVE 0 MEDIA STATE
                                <1> ;           DB      ?             ; DRIVE 1 MEDIA STATE
                                <1> ;           DB      ?             ; DRIVE 0 OPERATION START STATE
                                <1> ;           DB      ?             ; DRIVE 1 OPERATION START STATE
                                <1> ;@DSK_TRK   DB      ?             ; DRIVE 0 PRESENT CYLINDER
                                <1> ;           DB      ?             ; DRIVE 1 PRESENT CYLINDER
                                <1>
                                <1> ;DATA               ENDS                  ; END OF BIOS DATA SEGMENT
                                <1>
                                <1> ;-------------------------------------------------------
                                <1> ;   DRIVE TYPE TABLE                                :
                                <1> ;-------------------------------------------------------
                                <1>             ; 16/02/2015 (unix386.s, 32 bit modifications)
                                <1> DR_TYPE:
00006608 01                     <1>             DB      01            ; DRIVE TYPE, MEDIA TABLE
                                <1>                 ;DW       MD_TBL1
00006609 [26660000]             <1>             dd      MD_TBL1
0000660D 82                     <1>             DB      02+BIT7ON
                                <1>             ;DW     MD_TBL2
0000660E [33660000]             <1>                 dd      MD_TBL2
00006612 02                     <1> DR_DEFAULT: DB      02
                                <1>                 ;DW       MD_TBL3
00006613 [40660000]             <1>             dd      MD_TBL3
00006617 03                     <1>             DB      03
                                <1>                 ;DW       MD_TBL4
00006618 [4D660000]             <1>             dd      MD_TBL4
0000661C 84                     <1>             DB      04+BIT7ON
                                <1>                 ;DW       MD_TBL5
0000661D [5A660000]             <1>             dd      MD_TBL5
00006621 04                     <1>             DB      04
                                <1>                 ;DW       MD_TBL6
00006622 [67660000]             <1>             dd      MD_TBL6
                                <1> DR_TYPE_E       equ $                 ; END OF TABLE
                                <1> ;DR_CNT            EQU     (DR_TYPE_E-DR_TYPE)/3
                                <1> DR_CNT             equ     (DR_TYPE_E-DR_TYPE)/5
                                <1> ;-------------------------------------------------------
                                <1> ;   MEDIA/DRIVE PARAMETER TABLES                    :
                                <1> ;-------------------------------------------------------
                                <1> ;-------------------------------------------------------
                                <1> ;   360 KB MEDIA IN 360 KB DRIVE                    :
                                <1> ;-------------------------------------------------------
                                <1> MD_TBL1:
00006626 DF                     <1>     DB      11011111B     ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
00006627 02                     <1>     DB      2             ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
00006628 25                     <1>     DB      MOTOR_WAIT    ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
00006629 02                     <1>     DB      2             ; 512 BYTES/SECTOR
0000662A 09                     <1>     DB      09            ; EOT (LAST SECTOR ON TRACK)
0000662B 2A                     <1>     DB      02AH          ; GAP LENGTH
0000662C FF                     <1>     DB      0FFH          ; DTL
0000662D 50                     <1>     DB      050H          ; GAP LENGTH FOR FORMAT
0000662E F6                     <1>     DB      0F6H          ; FILL BYTE FOR FORMAT
0000662F 0F                     <1>     DB      15            ; HEAD SETTLE TIME (MILLISECONDS)
00006630 08                     <1>     DB      8             ; MOTOR START TIME (1/8 SECONDS)
00006631 27                     <1>     DB      39            ; MAX. TRACK NUMBER
00006632 80                     <1>     DB      RATE_250      ; DATA TRANSFER RATE
                                <1> ;-------------------------------------------------------
                                <1> ;   360 KB MEDIA IN 1.2 MB DRIVE                    :
                                <1> ;-------------------------------------------------------
                                <1> MD_TBL2:
00006633 DF                     <1>     DB      11011111B     ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
00006634 02                     <1>     DB      2             ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
00006635 25                     <1>     DB      MOTOR_WAIT    ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
00006636 02                     <1>     DB      2             ; 512 BYTES/SECTOR
00006637 09                     <1>     DB      09            ; EOT (LAST SECTOR ON TRACK)
00006638 2A                     <1>     DB      02AH          ; GAP LENGTH
00006639 FF                     <1>     DB      0FFH          ; DTL
0000663A 50                     <1>     DB      050H          ; GAP LENGTH FOR FORMAT
0000663B F6                     <1>     DB      0F6H          ; FILL BYTE FOR FORMAT
0000663C 0F                     <1>     DB      15            ; HEAD SETTLE TIME (MILLISECONDS)
0000663D 08                     <1>     DB      8             ; MOTOR START TIME (1/8 SECONDS)
0000663E 27                     <1>     DB      39            ; MAX. TRACK NUMBER
0000663F 40                     <1>     DB      RATE_300      ; DATA TRANSFER RATE
                                <1> ;-------------------------------------------------------
                                <1> ;   1.2 MB MEDIA IN 1.2 MB DRIVE                    :
                                <1> ;-------------------------------------------------------
                                <1> MD_TBL3:
00006640 DF                     <1>     DB      11011111B     ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
00006641 02                     <1>     DB      2             ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
00006642 25                     <1>     DB      MOTOR_WAIT    ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
00006643 02                     <1>     DB      2             ; 512 BYTES/SECTOR
00006644 0F                     <1>     DB      15            ; EOT (LAST SECTOR ON TRACK)
00006645 1B                     <1>     DB      01BH          ; GAP LENGTH
00006646 FF                     <1>     DB      0FFH          ; DTL
00006647 54                     <1>     DB      054H          ; GAP LENGTH FOR FORMAT
```

```
00006648 F6                 <1>     DB      0F6H            ; FILL BYTE FOR FORMAT
00006649 0F                 <1>     DB      15              ; HEAD SETTLE TIME (MILLISECONDS)
0000664A 08                 <1>     DB      8               ; MOTOR START TIME (1/8 SECONDS)
0000664B 4F                 <1>     DB      79              ; MAX. TRACK NUMBER
0000664C 00                 <1>     DB      RATE_500        ; DATA TRANSFER RATE
                            <1> ;-------------------------------------------------------
                            <1> ;   720 KB MEDIA IN 720 KB DRIVE                :
                            <1> ;-------------------------------------------------------
                            <1> MD_TBL4:
0000664D DF                 <1>     DB      11011111B       ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
0000664E 02                 <1>     DB      2               ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
0000664F 25                 <1>     DB      MOTOR_WAIT      ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
00006650 02                 <1>     DB      2               ; 512 BYTES/SECTOR
00006651 09                 <1>     DB      09              ; EOT (LAST SECTOR ON TRACK)
00006652 2A                 <1>     DB      02AH            ; GAP LENGTH
00006653 FF                 <1>     DB      0FFH            ; DTL
00006654 50                 <1>     DB      050H            ; GAP LENGTH FOR FORMAT
00006655 F6                 <1>     DB      0F6H            ; FILL BYTE FOR FORMAT
00006656 0F                 <1>     DB      15              ; HEAD SETTLE TIME (MILLISECONDS)
00006657 08                 <1>     DB      8               ; MOTOR START TIME (1/8 SECONDS)
00006658 4F                 <1>     DB      79              ; MAX. TRACK NUMBER
00006659 80                 <1>     DB      RATE_250        ; DATA TRANSFER RATE
                            <1> ;-------------------------------------------------------
                            <1> ;   720 KB MEDIA IN 1.44 MB DRIVE               :
                            <1> ;-------------------------------------------------------
                            <1> MD_TBL5:
0000665A DF                 <1>     DB      11011111B       ; SRT=D, HD UNLOAD=0F - 1ST SPECIFY BYTE
0000665B 02                 <1>     DB      2               ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
0000665C 25                 <1>     DB      MOTOR_WAIT      ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
0000665D 02                 <1>     DB      2               ; 512 BYTES/SECTOR
0000665E 09                 <1>     DB      09              ; EOT (LAST SECTOR ON TRACK)
0000665F 2A                 <1>     DB      02AH            ; GAP LENGTH
00006660 FF                 <1>     DB      0FFH            ; DTL
00006661 50                 <1>     DB      050H            ; GAP LENGTH FOR FORMAT
00006662 F6                 <1>     DB      0F6H            ; FILL BYTE FOR FORMAT
00006663 0F                 <1>     DB      15              ; HEAD SETTLE TIME (MILLISECONDS)
00006664 08                 <1>     DB      8               ; MOTOR START TIME (1/8 SECONDS)
00006665 4F                 <1>     DB      79              ; MAX. TRACK NUMBER
00006666 80                 <1>     DB      RATE_250        ; DATA TRANSFER RATE
                            <1> ;-------------------------------------------------------
                            <1> ;   1.44 MB MEDIA IN 1.44 MB DRIVE                  :
                            <1> ;-------------------------------------------------------
                            <1> MD_TBL6:
00006667 AF                 <1>     DB      10101111B       ; SRT=A, HD UNLOAD=0F - 1ST SPECIFY BYTE
00006668 02                 <1>     DB      2               ; HD LOAD=1, MODE=DMA - 2ND SPECIFY BYTE
00006669 25                 <1>     DB      MOTOR_WAIT      ; WAIT TIME AFTER OPERATION TILL MOTOR OFF
0000666A 02                 <1>     DB      2               ; 512 BYTES/SECTOR
0000666B 12                 <1>     DB      18              ; EOT (LAST SECTOR ON TRACK)
0000666C 1B                 <1>     DB      01BH            ; GAP LENGTH
0000666D FF                 <1>     DB      0FFH            ; DTL
0000666E 6C                 <1>     DB      06CH            ; GAP LENGTH FOR FORMAT
0000666F F6                 <1>     DB      0F6H            ; FILL BYTE FOR FORMAT
00006670 0F                 <1>     DB      15              ; HEAD SETTLE TIME (MILLISECONDS)
00006671 08                 <1>     DB      8               ; MOTOR START TIME (1/8 SECONDS)
00006672 4F                 <1>     DB      79              ; MAX. TRACK NUMBER
00006673 00                 <1>     DB      RATE_500        ; DATA TRANSFER RATE
                            <1>
                            <1> ; << diskette.inc >>
                            <1> ; ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
                            <1> ;
                            <1> ;----------------------------------------
                            <1> ;   ROM BIOS DATA AREAS          :
                            <1> ;----------------------------------------
                            <1>
                            <1> ;DATA               SEGMENT AT 40H          ; ADDRESS= 0040:0000
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   FIXED DISK DATA AREAS        :
                            <1> ;----------------------------------------
                            <1>
                            <1> ;DISK_STATUS1:      DB      0               ; FIXED DISK STATUS
                            <1> ;HF_NUM:    DB      0               ; COUNT OF FIXED DISK DRIVES
                            <1> ;CONTROL_BYTE:      DB      0               ; HEAD CONTROL BYTE
                            <1> ;@PORT_OFF  DB      ?               ; RESERVED (PORT OFFSET)
                            <1>
                            <1> ;----------------------------------------
                            <1> ;   ADDITIONAL MEDIA DATA        :
                            <1> ;----------------------------------------
                            <1>
                            <1> ;@LASTRATE  DB      ?               ; LAST DISKETTE DATA RATE SELECTED
                            <1> ;HF_STATUS  DB      0               ; STATUS REGISTER
                            <1> ;HF_ERROR   DB      0               ; ERROR REGISTER
                            <1> ;HF_INT_FLAG        DB      0               ; FIXED DISK INTERRUPT FLAG
                            <1> ;HF_CNTRL   DB      0               ; COMBO FIXED DISK/DISKETTE CARD BIT 0=1
                            <1> ;@DSK_STATE DB      ?               ; DRIVE 0 MEDIA STATE
                            <1> ;           DB      ?               ; DRIVE 1 MEDIA STATE
                            <1> ;           DB      ?               ; DRIVE 0 OPERATION START STATE
                            <1> ;           DB      ?               ; DRIVE 1 OPERATION START STATE
                            <1> ;@DSK_TRK   DB      ?               ; DRIVE 0 PRESENT CYLINDER
                            <1> ;           DB      ?               ; DRIVE 1 PRESENT CYLINDER
                            <1>
                            <1> ;DATA               ENDS                    ; END OF BIOS DATA SEGMENT
                            <1> ;
                            <1> ; ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
                            <1>
                            <1> ERR_TBL:
00006674 E0                 <1>     db      NO_ERR
00006675 024001BB           <1>     db      BAD_ADDR_MARK,BAD_SEEK,BAD_CMD,UNDEF_ERR
00006679 04BB100A           <1>     db      RECORD_NOT_FND,UNDEF_ERR,BAD_ECC,BAD_SECTOR
                            <1>
                            <1> ; 06/08/2022
                            <1> ; 17/12/2014 (mov ax, [cfd])
                            <1> ; 11/12/2014
                            <1> ;cfd:               db      0               ; current floppy drive (for GET_PARM)
                            <1> ; 17/12/2014                                ; instead of 'DISK_POINTER'
                            <1> ;pfd:               db      1               ; previous floppy drive (for GET_PARM)
                            <1>                                     ; (initial value of 'pfd
                            <1>                                     ; must be different then 'cfd' value
                            <1>                                     ; to force updating/initializing
                            <1>                                     ; current drive parameters)
                            <1> ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                            <1> ; 11/07/2022 - Retro UNIX 386 v1.1 (Kernel v0.2.1.5)
0000667D FF                 <1> pfd:        db      0FFh
                            <1>
                            <1> align 2
                            <1>
0000667E F001               <1> HF_PORT:    dw      1F0h  ; Default = 1F0h
                            <1>                           ; (170h)
00006680 F603               <1> HF_REG_PORT:        dw      3F6h  ; HF_PORT + 206h
                            <1>
                            <1> ; 05/01/2015
00006682 00                 <1> hf_m_s:         db      0     ; (0 = Master, 1 = Slave)
                            <1>
```

```
                                <1> ; ****************************************************************************
                                    Align 2

                                    ; 04/11/2014 (Retro UNIX 386 v1)
00006684 0000                       mem_1m_1k:   dw 0  ; Number of contiguous KB between
                                                       ; 1 and 16 MB, max. 3C00h = 15 MB.
00006686 0000                       mem_16m_64k: dw 0  ; Number of contiguous 64 KB blocks
                                                     ; between 16 MB and 4 GB.

                                    ; 12/11/2014 (Retro UNIX 386 v1)
00006688 00                         boot_drv:    db 0 ; boot drive number (physical)
                                    ; 24/11/2014
00006689 00                         drv:     db 0
0000668A 00                         last_drv:    db 0 ; last hdd
0000668B 00                         hdc:         db 0  ; number of hard disk drives
                                                     ; (present/detected)

                                    ; 24/11/2014 (Retro UNIX 386 v1)
                                    ; Physical drive type & flags
0000668C 00                         fd0_type:    db 0  ; floppy drive type
0000668D 00                         fd1_type:    db 0  ; 4 = 1.44 Mb, 80 track, 3.5" (18 spt)
                                                     ; 6 = 2.88 Mb, 80 track, 3.5" (36 spt)
                                                     ; 3 = 720 Kb, 80 track, 3.5" (9 spt)
                                                     ; 2 = 1.2 Mb, 80 track, 5.25" (15 spt)
                                                     ; 1 = 360 Kb, 40 track, 5.25" (9 spt)
0000668E 00                         hd0_type:    db 0  ; EDD status for hd0 (bit 7 = present flag)
0000668F 00                         hd1_type:    db 0  ; EDD status for hd1 (bit 7 = present flag)
00006690 00                         hd2_type:    db 0  ; EDD status for hd2 (bit 7 = present flag)
00006691 00                         hd3_type:    db 0  ; EDD status for hd3 (bit 7 = present flag)
                                                     ; bit 0 - Fixed disk access subset supported
                                                     ; bit 1 - Drive locking and ejecting
                                                     ; bit 2 - Enhanced disk drive support
                                                     ; bit 3 = Reserved (64 bit EDD support)
                                                     ; (If bit 0 is '1' Retro UNIX 386 v1
                                                     ; will interpret it as 'LBA ready'!)

                                    ; 08/08/2022
                                    ; (drv.cylinders, drv.spt, drv.spt will not be used now on)
                                    ; ('diskio.inc')
                                    ; ((spt and heads and cylinder counts will be taken from DPT))

                                    ;; 11/03/2015 - 10/07/2015
                                    ;drv.cylinders: dw 0,0,0,0,0,0,0
                                    ;drv.heads:     dw 0,0,0,0,0,0,0
                                    ;drv.spt:       dw 0,0,0,0,0,0,0
                                    ; 12/07/2022 - 11/03/2015
00006692 000000000000000000-        drv.size:      dd 0,0,0,0,0,0,0
0000669B 000000000000000000-
000066A4 000000000000000000-
000066AD 00
000066AE 00000000000000             drv.status:    db 0,0,0,0,0,0,0
000066B5 00000000000000             drv.error:     db 0,0,0,0,0,0,0
                                    ;

                                    Align 2

                                    ;;; 11/03/2015
                                    %include 'kybdata.s'      ; KEYBOARD (BIOS) DATA
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - kybdata.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 24/07/2022 (Previous: 17/01/2016)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 17/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Turkish Rational DOS
                                <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                                <1> ;
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; kybdata.inc (11/03/2015)
                                <1> ;
                                <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                                <1> ; ****************************************************************************
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - KYBDATA.INC
                                <1> ; Last Modification: 11/03/2015
                                <1> ;            (Data Section for 'KEYBOARD.INC')
                                <1> ;
                                <1> ; ///////// KEYBOARD DATA ///////////////
                                <1>
                                <1> ; 24/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1> ; 05/12/2014
                                <1> ; 04/12/2014 (derived from pc-xt-286 bios source code -1986-)
                                <1> ; 03/06/86  KEYBOARD BIOS
                                <1>
                                <1> ;-----------------------------------------------------------------------------
                                <1> ;   KEY IDENTIFICATION SCAN TABLES
                                <1> ;-----------------------------------------------------------------------------
                                <1>
                                <1> ;-----      TABLES FOR ALT CASE ------------
                                <1> ;-----      ALT-INPUT-TABLE
000066BC 524F50514B             <1> K30:db      82,79,80,81,75
000066C1 4C4D474849             <1>     db      76,77,71,72,73        ; 10 NUMBER ON KEYPAD
                                <1> ;-----      SUPER-SHIFT-TABLE
000066C6 101112131415           <1>     db      16,17,18,19,20,21     ; A-Z TYPEWRITER CHARS
000066CC 161718191E1F           <1>     db      22,23,24,25,30,31
000066D2 202122232425           <1>     db      32,33,34,35,36,37
000066D8 262C2D2E2F30           <1>     db      38,44,45,46,47,48
000066DE 3132                   <1>     db      49,50
                                <1>
                                <1> ;-----      TABLE OF SHIFT KEYS AND MASK VALUES
                                <1> ;-----      KEY_TABLE
000066E0 52                     <1> _K6:    db      INS_KEY                 ; INSERT KEY
000066E1 3A4546381D             <1>     db      CAPS_KEY,NUM_KEY,SCROLL_KEY,ALT_KEY,CTL_KEY
000066E6 2A36                   <1>         db      LEFT_KEY,RIGHT_KEY
                                <1> _K6L    equ     $-_K6
                                <1>
                                <1> ;-----      MASK_TABLE
000066E8 80                     <1> _K7:    db      INS_SHIFT               ; INSERT MODE SHIFT
000066E9 4020100804             <1>     db      CAPS_SHIFT,NUM_SHIFT,SCROLL_SHIFT,ALT_SHIFT,CTL_SHIFT
000066EE 0201                   <1>     db      LEFT_SHIFT,RIGHT_SHIFT
                                <1>
                                <1> ;-----      TABLES FOR CTRL CASE         ;---- CHARACTERS ------
000066F0 1BFF00FFFFFF           <1> _K8:db      27,-1,0,-1,-1,-1      ; Esc, 1, 2, 3, 4, 5
000066F6 1EFFFFFFFF1F           <1>     db      30,-1,-1,-1,-1,31     ; 6, 7, 8, 9, 0, -
                                <1>     ;db     -1,127,-1,17,23,5     ; =, Bksp, Tab, Q, W, E
000066FC FF7F94111705           <1>     db      -1,127,148,17,23,5 ; 24/07/2022
00006702 12141915090F           <1>     db      18,20,25,21,9,15      ; R, T, Y, U, I, O
00006708 101B1D0AFF01           <1>     db      16,27,29,10,-1,1      ; P, [, ], Enter, Ctrl, A
0000670E 13040607080A           <1>     db      19,4,6,7,8,10         ; S, D, F, G, H, J
00006714 0B0CFFFFFFFF           <1>     db      11,12,-1,-1,-1,-1     ; K, L, :, ', `, LShift
```

```
0000671A 1C1A18031602        <1>     db      28,26,24,3,22,2              ; Bkslash, Z, X, C, V, B
00006720 0E0DFFFFFFFF        <1>     db      14,13,-1,-1,-1,-1     ; N, M, ,, ., /, RShift
00006726 96FF20FF            <1>     db      150,-1,' ',-1         ; *, ALT, Spc, CL
                             <1>     ;                             ;----- FUNCTIONS ------
0000672A 5E5F60616263        <1>     db      94,95,96,97,98,99     ; F1 - F6
00006730 64656667FFFF        <1>     db      100,101,102,103,-1,-1 ; F7 - F10, NL, SL
00006736 778D848E738F        <1>     db      119,141,132,142,115,143      ; Home, Up, PgUp, -, Left, Pad5
0000673C 749075917692        <1>     db      116,144,117,145,118,146 ; Right, +, End, Down, PgDn, Ins
00006742 93FFFFFF898A        <1>     db      147,-1,-1,-1,137,138  ; Del, SysReq, Undef, WT, F11, F12
                             <1>
                             <1> ;-----      TABLES FOR LOWER CASE ----------
00006748 1B3132333435363738- <1> K10:db      27,'1234567890-=',8,9
00006751 39302D3D0809        <1>
00006757 71776572747975696F- <1>     db      'qwertyuiop[]',13,-1,'asdfghjkl;',39
00006760 705B5D0DFF61736466- <1>
00006769 67686A6B6C3B27      <1>
00006770 60FF5C7A786376626E- <1>     db      96,-1,92,'zxcvbnm,./',-1,'*',-1,' ',-1
00006779 6D2C2E2FFF2AFF20FF  <1>
                             <1> ;-----      LC TABLE SCAN
00006782 3B3C3D3E3F          <1>     db      59,60,61,62,63        ; BASE STATE OF F1 - F10
00006787 4041424344          <1>     db      64,65,66,67,68
0000678C FFFF                <1>     db      -1,-1                 ; NL, SL
                             <1>
                             <1> ;-----      KEYPAD TABLE
0000678E 474849FF4BFF        <1> K15:db      71,72,73,-1,75,-1     ; BASE STATE OF KEYPAD KEYS
00006794 4DFF4F50515253      <1>     db      77,-1,79,80,81,82,83
0000679B FFFF5C8586          <1>     db      -1,-1,92,133,134      ; SysRq, Undef, WT, F11, F12
                             <1>
                             <1> ;-----      TABLES FOR UPPER CASE ----------
000067A0 1B21402324255E262A- <1> K11:db      27,'!@#$%',94,'&*()_+',8,0
000067A9 28295F2B0800        <1>
000067AF 5157455254595549 4F- <1>     db      'QWERTYUIOP{}',13,-1,'ASDFGHJKL:"'
000067B8 507B7D0DFF41534446- <1>
000067C1 47484A4B4C3A22      <1>
000067C8 7EFF7C5A584356424E- <1>     db      126,-1,'|ZXCVBNM<>?',-1,'*',-1,' ',-1
000067D1 4D3C3E3FFF2AFF20FF  <1>
                             <1> ;-----      UC TABLE SCAN
000067DA 5455565758          <1> K12:db      84,85,86,87,88        ; SHIFTED STATE OF F1 - F10
000067DF 595A5B5C5D          <1>     db      89,90,91,92,93
000067E4 FFFF                <1>     db      -1,-1                 ; NL, SL
                             <1>
                             <1> ;-----      NUM STATE TABLE
000067E6 3738392D3435362B31- <1> K14:db      '789-456+1230.'              ; NUMLOCK STATE OF KEYPAD KEYS
000067EF 3233302E            <1>
                             <1>     ;
000067F3 FFFF7C8788          <1>     db      -1,-1,124,135,136     ; SysRq, Undef, WT, F11, F12
                             <1>
                             <1> ; 26/08/2014
                             <1> ; Retro UNIX 8086 v1 - UNIX.ASM (03/03/2014)
                             <1> ; Derived from IBM "pc-at"
                             <1> ; rombios source code (06/10/1985)
                             <1> ; 'dseg.inc'
                             <1>
                             <1> ;--------------------------------------;
                             <1> ;   SYSTEM DATA AREA              ;
                             <1> ;--------------------------------------
000067F8 00                  <1> BIOS_BREAK  db      0             ; BIT 7=1 IF BREAK KEY HAS BEEN PRESSED
                             <1>
                             <1> ;--------------------------------------
                             <1> ;   KEYBOARD DATA AREAS           ;
                             <1> ;--------------------------------------
                             <1>
000067F9 00                  <1> KB_FLAG             db      0             ; KEYBOARD SHIFT STATE AND STATUS FLAGS
000067FA 00                  <1> KB_FLAG_1   db      0             ; SECOND BYTE OF KEYBOARD STATUS
000067FB 00                  <1> KB_FLAG_2   db      0             ; KEYBOARD LED FLAGS
000067FC 00                  <1> KB_FLAG_3   db      0             ; KEYBOARD MODE STATE AND TYPE FLAGS
000067FD 00                  <1> ALT_INPUT   db      0             ; STORAGE FOR ALTERNATE KEY PAD ENTRY
000067FE [0E680000]          <1> BUFFER_START        dd      KB_BUFFER      ; OFFSET OF KEYBOARD BUFFER START
00006802 [2E680000]          <1> BUFFER_END  dd      KB_BUFFER + 32 ; OFFSET OF END OF BUFFER
00006806 [0E680000]          <1> BUFFER_HEAD dd      KB_BUFFER      ; POINTER TO HEAD OF KEYBOARD BUFFER
0000680A [0E680000]          <1> BUFFER_TAIL dd      KB_BUFFER      ; POINTER TO TAIL OF KEYBOARD BUFFER
                             <1> ; ------    HEAD = TAIL INDICATES THAT THE BUFFER IS EMPTY
0000680E 0000<rep 10h>       <1> KB_BUFFER   times   16 dw 0        ; ROOM FOR 16 SCAN CODE ENTRIES
                             <1>
                             <1> ; /// End Of KEYBOARD DATA ///
                                 %include 'vidata.s'        ; VIDEO (BIOS) DATA
                             <1> ; ****************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.10 - vidata.s
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Last Update: 21/12/2025 (Previous: 17/10/2023 - Kernel v2.0.7)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Beginning: 16/01/2016
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.15 (trdos386.s)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Turkish Rational DOS
                             <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                             <1> ;
                             <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                             <1> ; vidata.inc (11/03/2015)
                             <1> ;
                             <1> ; Derived from 'IBM PC-AT' BIOS source code (1985)
                             <1> ; ****************************************************************************
                             <1>
                             <1> ; Retro UNIX 386 v1 Kernel - VIDATA.S
                             <1> ; Last Modification: 11/03/2015
                             <1> ;               (Data section for 'VIDEO.INC')
                             <1> ;
                             <1> ; ///////// VIDEO DATA ///////////////
                             <1>
                             <1> ;----------------------------------------
                             <1> ;   VIDEO DISPLAY DATA AREA               ;
                             <1> ;----------------------------------------
0000682E 03                  <1> CRT_MODE:   db      3       ; CURRENT DISPLAY MODE (TYPE)
0000682F 29                  <1> CRT_MODE_SET:       db      29h     ; CURRENT SETTING OF THE 3X8 REGISTER
                             <1>                             ; (29h default setting for video mode 3)
                             <1>                             ; Mode Select register Bits
                             <1>                             ;   BIT 0 - 80x25 (1), 40x25 (0)
                             <1>                             ;   BIT 1 - ALPHA (0), 320x200 GRAPHICS (1)
                             <1>                             ;   BIT 2 - COLOR (0), BW (1)
                             <1>                             ;   BIT 3 - Video Sig. ENABLE (1), DISABLE (0)
                             <1>                             ;   BIT 4 - 640x200 B&W Graphics Mode (1)
                             <1>                             ;   BIT 5 - ALPHA mode BLINKING (1)
                             <1>                             ;   BIT 6, 7 - Not Used
                             <1>
                             <1> ; Mode 0 - 2Ch = 101100b  ; 40x25 text, 16 gray colors
                             <1> ; Mode 1 - 28h = 101000b  ; 40x25 text, 16 fore colors, 8 back colors
                             <1> ; Mode 2 - 2Dh = 101101b  ; 80x25 text, 16 gray colors
                             <1> ; Mode 3 - 29h = 101001b  ; 80x25 text, 16 fore color, 8 back color
                             <1> ; Mode 4 - 2Ah = 101010b  ; 320x200 graphics, 4 colors
                             <1> ; Mode 5 - 2Eh = 101110b  ; 320x200 graphics, 4 gray colors
                             <1> ; Mode 6 - 1Eh = 011110b  ; 640x200 graphics, 2 colors
                             <1> ; Mode 7 - 29h = 101001b  ; 80x25 text, black & white colors
                             <1> ; Mode & 37h = Video signal OFF
```

```
                                <1>
                                <1> ; 24/06/2016
00006830 50                     <1> CRT_COLS:   db      80      ; Number of columns
                                <1>
                                <1> ; 01/07/2016
00006831 00                     <1> CRT_PALETTE:        db      0       ; Current palette setting
                                <1>
                                <1> ; 03/07/2016
00006832 10                     <1> CHAR_HEIGHT:        db      16      ; Default character height
00006833 60                     <1> VGA_VIDEO_CTL:      db      60h     ; ROM BIOS DATA AREA Offset 87h
00006834 F9                     <1> VGA_SWITCHES:       db      0F9h    ; Feature Bit Switches (the basic screen)
00006835 51                     <1> VGA_MODESET_CTL: db         051h    ; Basic mode set options (VGA video flags)
                                <1>                             ; ROM BIOS DATA AREA Offset 89h
                                <1>                             ; Bit 7, 4 : Mode
                                <1>                             ;        01 : 400-line mode
                                <1>                             ; Bit 6 : Display switch enabled = 1
                                <1>                             ; Bit 5 : Reserved = 0
                                <1>                             ; Bit 3 : Default palette loading
                                <1>                             ;        disabled = 0
                                <1>                             ; Bit 2 : Color monitor = 0
                                <1>                             ; Bit 1 = Gray scale summing
                                <1>                             ;        disabled = 0
                                <1>                             ; Bit 0 = VGA active = 1
00006836 19                     <1> VGA_ROWS:   db      25
                                <1>
                                <1> ; 16/01/2016
                                <1> chr_attrib:  ; Character color/attributes for video pages (0 to 7)
00006837 0707070707070707       <1>     db      07h, 07h, 07h, 07h, 07h, 07h, 07h, 07h
                                <1> ; 30/01/2016
                                <1> vmode:
0000683F 0303030303030303       <1>     db      3,3,3,3,3,3,3,3 ; video modes for pseudo screens
                                <1>
                                <1> CURSOR_MODE: ; cursor start (ch) = 14, cursor end (cl) = 15
00006847 0F0E                   <1>     db      15, 14 ; 07/07/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>
                                <1> ;align 4
                                <1> ;VGA_BASE: ; 26/07/2016
                                <1> ;   dd      0B8000h  ; (Mode < 0Dh) or 0A0000h (mode >= 0Dh)
                                <1>
00006849 90                     <1> align 2
                                <1>
                                <1> vga_modes:
                                <1>     ; 25/07/2016
                                <1>     ; 09/07/2016
                                <1>     ; 03/07/2016
                                <1>     ; valid (implemented) video modes (>7, extension to IBM PC CGA modes)
0000684A 0302010007040506       <1>     db      03h, 02h, 01h, 00h, 07h, 04h, 05h, 06h
                                <1> vga_g_modes: ; 31/07/2016
00006852 13F0126A0D0E1011       <1>     db      13h, 0F0h, 12h, 6Ah, 0Dh, 0Eh, 10h, 11h
                                <1> vga_mode_count equ $ - vga_modes
                                <1> vga_g_mode_count equ $ - vga_g_modes
                                <1>
                                <1> vga_mode_tbl_ptr:
                                <1>     ; 25/07/2016
0000685A [BA680000]             <1>     dd      vga_mode_03h
0000685E [BA680000]             <1>     dd      vga_mode_03h ; mode 02h -> mode 03h
00006862 [FA680000]             <1>     dd      vga_mode_01h
00006866 [FA680000]             <1>     dd      vga_mode_01h ; mode 00h -> mode 01h
                                <1>     ;dd     vga_mode_07h
0000686A [BA680000]             <1>     dd      vga_mode_03h ; mode 07h -> mode 03h
0000686E [3A690000]             <1>     dd      vga_mode_04h
00006872 [3A690000]             <1>     dd      vga_mode_04h ; mode 05h -> mode 04h
00006876 [7A690000]             <1>     dd      vga_mode_06h
                                <1> vga_g_mode_tbl_ptr: ; 21/12/2025
0000687A [BA690000]             <1>     dd      vga_mode_13h
0000687E [FA690000]             <1>     dd      vga_mode_F0h
00006882 [3A6A0000]             <1>     dd      vga_mode_12h
00006886 [7A6A0000]             <1>     dd      vga_mode_6Ah
0000688A [BA6A0000]             <1>     dd      vga_mode_0Dh
0000688E [FA6A0000]             <1>     dd      vga_mode_0Eh
00006892 [3A6B0000]             <1>     dd      vga_mode_10h
00006896 [7A6B0000]             <1>     dd      vga_mode_11h
                                <1>
                                <1> vga_memmodel:
                                <1>     ; 25/07/2016
                                <1>     ; 07/07/2016
                                <1>     CTEXT  equ 0
                                <1>     ;MTEXT  equ 1
                                <1>     MTEXT  equ 0 ; mode 07h -> mode 03h
                                <1>     CGA    equ 2
                                <1>     LINEAR8 equ 5
                                <1>     PLANAR4 equ 4
                                <1>     PLANAR1 equ 3
0000689A 0000000000020202       <1>     db      CTEXT, CTEXT, CTEXT, CTEXT, MTEXT, CGA, CGA, CGA
                                <1> vga_g_memmodel: ; 31/07/2016
000068A2 0504040404040403       <1>     db      LINEAR8, PLANAR4, PLANAR4, PLANAR4, PLANAR4, PLANAR4, PLANAR4, PLANAR1
                                <1> ;vga_pixbits:
                                <1> ;   ; 25/07/2016
                                <1> ;   ; 08/07/2016
                                <1> ;   db      4, 4, 4, 4, 4, 2, 2, 1, 8, 4, 4, 4, 4, 4, 4, 1
                                <1> vga_dac_s:
000068AA 020202020001010103-     <1>     db      2, 2, 2, 2, 0, 1, 1, 1, 3, 3, 2, 2, 1, 1, 2, 2
000068B3 03020201010202         <1>
                                <1>                                             ; (vgatables.h, VGAMODES, dac)
                                <1>                                             ; 17/11/2020
                                <1> vga_params:
                                <1>     ; 23/11/2020
                                <1>     ; 16/11/2020
                                <1>     ; 09/11/2020, 10/11/2020, 11/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; 25/07/2016
                                <1>     ; 19/07/2016
                                <1>     ; 03/07/2016
                                <1>     ; derived from 'Plex86/Bochs VGABios' source code
                                <1>     ; vgabios-0.7a (2011)
                                <1>     ; by the LGPL VGABios Developers Team (2001-2008)
                                <1>     ; 'vgatables.h'
                                <1>     ; Oracle VirtualBox 5.0.24 VGABios Source Code
                                <1>     ; ('vgabios.c', 'vgatables.h', 'vgafonts.h', 'vgarom.asm')
                                <1>
                                <1>     ; 09/11/2020
                                <1>     ; Block Structure of Video Parameter Table
                                <1>     ;
                                <1>     ; Offset       # Bytes        Contents
                                <1>     ;
                                <1>     ;  0           1              # columns
                                <1>     ;  1           1              # rows - 1
                                <1>     ;  2           1              Pixels/character
                                <1>     ;  3-4         2              Page length
                                <1>     ;  5-8         4              Sequencer Registers
                                <1>     ;  9           1              Miscellaneous Register
                                <1>     ;  10-34       25             CRTC Registers
                                <1>     ;  35-54       20             Attribute Registers
                                <1>     ;  55-63       9              Graphics Controller Registers
                                <1>     ;
```

```
                                <1>     ; Ref: Programmer's Guide to EGA, VGA, and Super VGA cards
                                <1>     ; (Richard F. Ferraro, 1994)
                                <1>
                                <1>     ;
                                <1> vga_mode_03h:  ; mode 03h, 80*25 text, CGA colors
                                <1>     ; 11/11/2020
000068BA 5018100010             <1>     db      80, 24, 16, 00h, 10h ; tw, th-1, ch, slength (5)
000068BF 00030002               <1>     db      00h, 03h, 00h, 02h ; sequ regs (4)
000068C3 67                     <1>     db      67h     ; misc reg (1)
000068C4 5F4F50825581BF1F       <1>     db      5Fh, 4Fh, 50h, 82h, 55h, 81h, 0BFh, 1Fh
                                <1>     ; 09/11/2020
                                <1>     ;db     5Fh, 4Fh, 50h, 82h, 54h, 80h, 0BFh, 1Fh
000068CC 004F                   <1>     db      00h, 4Fh
                                <1> vga_p_cm_pos equ $ - vga_mode_03h
000068CE 0D0E00000000           <1>     db      0Dh, 0Eh, 00h, 00h, 00h, 00h
000068D4 9C8E8F281F96B9A3       <1>     db      9Ch, 8Eh, 8Fh, 28h, 1Fh, 96h, 0B9h, 0A3h
000068DC FF                     <1>     db      0FFh    ; crtc_regs (25)
000068DD 0001020304051407       <1>     db      00h, 01h, 02h, 03h, 04h, 05h, 14h, 07h
000068E5 38393A3B3C3D3E3F       <1>     db      38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh
                                <1>     ; 17/10/2023
000068ED 0C000F08               <1>     db      0Ch, 00h, 0Fh, 08h  ; actl regs (20)
                                <1>     ;db     0Ch, 00h, 0Fh, 00h  ; 19/11/2020
000068F1 0000000000100E0FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 10h, 0Eh, 0Fh, 0FFh ; grdc regs (9)
                                <1>     ; 09/11/2020
                                <1>     ;db     00h, 00h, 00h, 00h, 00h, 10h, 0Eh, 00h, 0FFh ; grdc regs (9)
                                <1> vga_mode_01h:      ; mode 01h, 40*25 text, CGA colors
000068FA 2818100008             <1>     db      40, 24, 16, 00h, 08h ; tw, th-1, ch, slength
000068FF 08030002               <1>     db      08h, 03h, 00h, 02h  ; sequ regs
00006903 67                     <1>     db      67h     ; misc reg
00006904 2D2728902BA0BF1F       <1>     db      2Dh, 27h, 28h, 90h, 2Bh, 0A0h, 0BFh, 1Fh
0000690C 004F0D0E00000000       <1>     db      00h, 4Fh, 0Dh, 0Eh, 00h, 00h, 00h, 00h
00006914 9C8E8F141F96B9A3       <1>     db      9Ch, 8Eh, 8Fh, 14h, 1Fh, 96h, 0B9h, 0A3h
0000691C FF                     <1>     db      0FFh    ; crtc_regs
0000691D 0001020304051407       <1>     db      00h, 01h, 02h, 03h, 04h, 05h, 14h, 07h
00006925 38393A3B3C3D3E3F       <1>     db      38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh
                                <1>     ;db     0Ch, 00h, 0Fh, 08h  ; actl regs (20)
0000692D 0C000F00               <1>     db      0Ch, 00h, 0Fh, 00h  ; 19/11/2020
00006931 0000000000100E0FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 10h, 0Eh, 0Fh, 0FFh ; grdc regs
                                <1> ;vga_mode_07h:     ; mode 07h, 80*25 text, mono color
                                <1> ;   db      80, 24, 16, 00h, 10h  ; tw, th-1, ch, slength
                                <1> ;   db      00h, 03h, 00h, 02h ; sequ regs
                                <1> ;   db      66h     ; misc reg
                                <1> ;   db      5Fh, 4Fh, 50h, 82h, 55h, 81h, 0BFh, 1Fh
                                <1> ;   db      00h, 4Fh, 0Dh, 0Eh, 00h, 00h, 00h, 00h
                                <1> ;   db      9Ch, 8Eh, 8Fh, 28h, 0Fh, 96h, 0B9h, 0A3h
                                <1> ;   db      0FFh    ; crtc regs
                                <1> ;   db      00h, 08h, 08h, 08h, 08h, 08h, 08h, 08h
                                <1> ;   db      10h, 18h, 18h, 18h, 18h, 18h, 18h, 18h
                                <1> ;   db      0Eh, 00h, 0Fh, 08h  ; actl regs
                                <1> ;   db      00h, 00h, 00h, 00h, 00h, 10h, 0Ah, 0Fh, 0FFh ; grdc regs
                                <1> vga_mode_04h:      ; 320*200 graphics, 4 colors, CGA
                                <1>     ; 11/11/2020
0000693A 2818080040             <1>     db      40, 24, 8, 00h, 40h ; tw, th-1, ch, slength
0000693F 09030002               <1>     db      09h, 03h, 00h, 02h ; sequ regs
00006943 63                     <1>     db      63h     ; misc reg
00006944 2D2728902B80BF1F       <1>     db      2Dh, 27h, 28h, 90h, 2Bh, 80h, 0BFh, 1Fh
0000694C 00C1000000000000       <1>     db      00h, 0C1h, 00h, 00h, 00h, 00h, 00h, 00h
00006954 9C8E8F140096B9A2       <1>     db      9Ch, 8Eh, 8Fh, 14h, 00h, 96h, 0B9h, 0A2h
0000695C FF                     <1>     db      0FFh    ; crtc_regs
0000695D 0013151702040607       <1>     db      00h, 13h, 15h, 17h, 02h, 04h, 06h, 07h
00006965 1011121314151617       <1>     db      10h, 11h, 12h, 13h, 14h, 15h, 16h, 17h
0000696D 01000300               <1>     db      01h, 00h, 03h, 00h ; actl regs
00006971 0000000000300F0FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 30h, 0Fh, 0Fh, 0FFh ; grdc regs
                                <1> vga_mode_06h:      ; 640*200 graphics, 2 colors, CGA
                                <1>     ; 11/11/2020
0000697A 5018080040             <1>     db      80, 24, 8, 00h, 40h   ;       tw, th-1, ch, slength
0000697F 01010006               <1>     db      01h, 01h, 00h, 06h ; sequ regs
00006983 63                     <1>     db      63h     ; misc reg
00006984 5F4F50825480BF1F       <1>     db      5Fh, 4Fh, 50h, 82h, 54h, 80h, 0BFh, 1Fh
0000698C 00C1000000000000       <1>     db      00h, 0C1h, 00h, 00h, 00h, 00h, 00h, 00h
00006994 9C8E8F280096B9C2       <1>     db      9Ch, 8Eh, 8Fh, 28h, 00h, 96h, 0B9h, 0C2h
0000699C FF                     <1>     db      0FFh    ; crtc regs
0000699D 0017171717171717       <1>     db      00h, 17h, 17h, 17h, 17h, 17h, 17h, 17h
000069A5 1717171717171717       <1>     db      17h, 17h, 17h, 17h, 17h, 17h, 17h, 17h
000069AD 01000100               <1>     db      01h, 00h, 01, 00h  ; actl regs
000069B1 0000000000000D0FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 00h, 0Dh, 0Fh, 0FFh ; grdc regs
                                <1> vga_mode_13h:  ; mode 13h, 300*200, 256 colors, linear
                                <1>     ; 11/11/2020
                                <1>     ;db     40, 24, 8, 00h, 20h ; tw, th-1, ch, slength (5)
                                <1>     ; 23/11/2020 - 10/11/2020
000069BA 28180800FA             <1>     db      40, 24, 8, 00h, 0FAh ; tw, th-1, ch, slength (5)
000069BF 010F000E               <1>     db      01h, 0Fh, 00h, 0Eh ; sequ regs (4)
000069C3 63                     <1>     db      63h     ; misc reg (1)
000069C4 5F4F50825480BF1F       <1>     db      5Fh, 4Fh, 50h, 82h, 54h, 80h, 0BFh, 1Fh
000069CC 0041000000000000       <1>     db      00h, 041h, 00h, 00h, 00h, 00h, 00h, 00h
000069D4 9C8E8F284096B9A3       <1>     db      9Ch, 8Eh, 8Fh, 28h, 40h, 96h, 0B9h, 0A3h
                                <1>     ;db     9Ch, 0Eh, 8Fh, 28h, 40h, 96h, 0B9h, 0A3h ; 17/10/2023
000069DC FF                     <1>     db      0FFh    ; crtc regs (25)
000069DD 0001020304050607       <1>     db      00h, 01h, 02h, 03h, 04h, 05h, 06h, 07h
000069E5 08090A0B0C0D0E0F       <1>     db      08h, 09h, 0Ah, 0Bh, 0Ch, 0Dh, 0Eh, 0Fh
000069ED 41000F00               <1>     db      41h, 00h, 0Fh, 00h  ; actl regs (20)
                                <1>     ; 10/11/2020
                                <1>     ;db     41h, 01h, 0Fh, 13h  ; actl regs (20)
000069F1 000000000040050FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 40h, 05h, 0Fh, 0FFh ; grdc regs (9)
                                <1> vga_mode_setl equ $ - vga_mode_13h  ; = 64
                                <1> vga_mode_F0h:  ; mode X ; 320*240, 256 colors, planar
000069FA 2818080000             <1>     db      40, 24, 8, 00h, 00h ; tw, th-1, ch, slength
000069FF 010F0006               <1>     db      01h, 0Fh, 00h, 06h ; sequ regs
00006A03 E3                     <1>     db      0E3h    ; misc reg
00006A04 5F4F508254800D3E       <1>     db      5Fh, 4Fh, 50h, 82h, 54h, 80h, 0Dh, 3Eh
00006A0C 0041000000000000       <1>     db      00h, 41h, 00h, 00h, 00h, 00h, 00h, 00h
00006A14 EAACDF2800E706E3       <1>     db      0EAh, 0ACh, 0DFh, 28h, 00h, 0E7h, 06h, 0E3h
00006A1C FF                     <1>     db      0FFh    ; crtc regs (25)
00006A1D 0001020304050607       <1>     db      00h, 01h, 02h, 03h, 04h, 05h, 06h, 07h
00006A25 08090A0B0C0D0E0F       <1>     db      08h, 09h, 0Ah, 0Bh, 0Ch, 0Dh, 0Eh, 0Fh
00006A2D 41000F00               <1>     db      41h, 00h, 0Fh, 00h  ; actl regs
00006A31 000000000040050FFF     <1>     db      00h, 00h, 00h, 00h, 00h, 40h, 05h, 0Fh, 0FFh ; grdc regs
                                <1> vga_mode_12h:  ; mode 12h, 640*480, 16 colors, planar
                                <1>     ; 11/11/2020
                                <1>     ;db     80, 29, 16, 0, 0 ; tw, th-1, ch, slength
                                <1>     ; 09/11/2020
00006A3A 501D1000A0             <1>     db      80, 29, 16, 00h, 0A0h ; tw, th-1, ch, slength
00006A3F 010F0006               <1>     db      01h, 0Fh, 00h, 06h ; sequ regs
00006A43 E3                     <1>     db      0E3h    ; misc reg
00006A44 5F4F508254800B3E       <1>     db      5Fh, 4Fh, 50h, 82h, 54h, 80h, 0Bh, 3Eh
                                <1>     ; 09/11/2020
                                <1>     ;db     5Fh, 4Fh, 50h, 82h, 53h, 9Fh, 0Bh, 3Eh
00006A4C 0040000000000000       <1>     db      00h, 40h, 00h, 00h, 00h, 00h, 00h, 00h
00006A54 EA8CDF2800E704E3       <1>     db      0EAh, 8Ch, 0DFh, 28h, 00h, 0E7h, 04h, 0E3h
                                <1>     ; 09/11/2020
                                <1>     ;db     0E9h, 8Bh, 0DFh, 28h, 00h, 0E7h, 04h, 0E3h
00006A5C FF                     <1>     db      0FFh    ; crtc regs
00006A5D 0001020304051407       <1>     db      00h, 01h, 02h, 03h, 04h, 05h, 14h, 07h
00006A65 38393A3B3C3D3E3F       <1>     db      38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh
```

```
00006A6D 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006A71 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> vga_mode_6Ah:  ; mode 6Ah, 800*600, 16 colors, planar
                             <1>     ; 11/11/2020
00006A7A 6424100000          <1>     db     100, 36, 16, 00h, 00h ; tw, th-1, ch, slength
00006A7F 010F0006            <1>     db     01h, 0Fh, 00h, 06h ; sequ regs
00006A83 E3                  <1>     db     0E3h   ; misc reg
00006A84 7F6363836B1B72F0    <1>     db     7Fh, 63h, 63h, 83h, 6Bh, 1Bh, 72h, 0F0h
00006A8C 0060000000000000    <1>     db     00h, 60h, 00h, 00h, 00h, 00h, 00h, 00h
00006A94 598D5732005773E3    <1>     db     59h, 8Dh, 57h, 32h, 00h, 57h, 73h, 0E3h
00006A9C FF                  <1>     db     0FFh   ; crtc regs
00006A9D 0001020304051407    <1>     db     00h, 01h, 02h, 03h, 04h, 05h, 14h, 07h
00006AA5 38393A3B3C3D3E3F    <1>     db     38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh
00006AAD 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006AB1 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> vga_mode_0Dh:  ; mode 0Dh, 320*200, 16 colors, planar
00006ABA 2818080020          <1>     db     40, 24, 8, 00h, 20h ; tw, th-1, ch, slength
00006ABF 090F0006            <1>     db     09h, 0Fh, 00h, 06h ; sequ regs
00006AC3 63                  <1>     db     63h    ; misc reg
00006AC4 2D2728902B80BF1F    <1>     db     2Dh, 27h, 28h, 90h, 2Bh, 80h, 0BFh, 1Fh
00006ACC 00C0000000000000    <1>     db     00h, 0C0h, 00h, 00h, 00h, 00h, 00h, 00h
00006AD4 9C8E8F140096B9E3    <1>     db     9Ch, 8Eh, 8Fh, 14h, 00h, 96h, 0B9h, 0E3h
00006ADC FF                  <1>     db     0FFh   ; crtc regs
00006ADD 0001020304050607    <1>     db     00h, 01h, 02h, 03h, 04h, 05h, 06h, 07h
00006AE5 1011121314151617    <1>     db     10h, 11h, 12h, 13h, 14h, 15h, 16h, 17h
00006AED 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006AF1 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> vga_mode_0Eh:  ; mode 0Eh, 640*200, 16 colors, planar
00006AFA 5018080040          <1>     db     80, 24, 8, 00h, 40h ; tw, th-1, ch, slength
00006AFF 010F0006            <1>     db     01h, 0Fh, 00h, 06h ; sequ regs
00006B03 63                  <1>     db     63h    ; misc reg
00006B04 5F4F50825480BF1F    <1>     db     5Fh, 4Fh, 50h, 82h, 54h, 80h, 0BFh, 1Fh
00006B0C 00C0000000000000    <1>     db     00h, 0C0h, 00h, 00h, 00h, 00h, 00h, 00h
00006B14 9C8E8F280096B9E3    <1>     db     9Ch, 8Eh, 8Fh, 28h, 00h, 96h, 0B9h, 0E3h
00006B1C FF                  <1>     db     0FFh   ; crtc regs
00006B1D 0001020304050607    <1>     db     00h, 01h, 02h, 03h, 04h, 05h, 06h, 07h
00006B25 1011121314151617    <1>     db     10h, 11h, 12h, 13h, 14h, 15h, 16h, 17h
00006B2D 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006B31 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> vga_mode_10h: ; mode 10h, 640*350, 16 colors, planar
00006B3A 50180E0080          <1>     db     80, 24, 14, 00h, 80h ; tw, th-1, ch, slength
00006B3F 010F0006            <1>     db     01h, 0Fh, 00h, 06h ; sequ regs
00006B43 A3                  <1>     db     0A3h   ; misc reg
00006B44 5F4F50825480BF1F    <1>     db     5Fh, 4Fh, 50h, 82h, 54h, 80h, 0BFh, 1Fh
00006B4C 0040000000000000    <1>     db     00h, 40h, 00h, 00h, 00h, 00h, 00h, 00h
00006B54 83855D280F63BAE3    <1>     db     83h, 85h, 5Dh, 28h, 0Fh, 63h, 0BAh, 0E3h
00006B5C FF                  <1>     db     0FFh   ; crtc regs
00006B5D 0001020304051407    <1>     db     00h, 01h, 02h, 03h, 04h, 05h, 14h, 07h
00006B65 38393A3B3C3D3E3F    <1>     db     38h, 39h, 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh
00006B6D 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006B71 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> vga_mode_11h: ; mode 11h, 640*480, mono color, planar
                             <1>     ; 11/11/2020
00006B7A 501D1000A0          <1>     db     80, 29, 16, 00h, 0A0h ; tw, th-1, ch, slength
00006B7F 010F0006            <1>     db     01h, 0Fh, 00h, 06h ; sequ regs
00006B83 E3                  <1>     db     0E3h   ; misc reg
00006B84 5F4F508254800B3E    <1>     db     5Fh, 4Fh, 50h, 82h, 54h, 80h, 0Bh, 3Eh
00006B8C 0040000000000000    <1>     db     00h, 40h, 00h, 00h, 00h, 00h, 00h, 00h
00006B94 EA8CDF2800E704C3    <1>     db     0EAh, 8Ch, 0DFh, 28h, 00h, 0E7h, 04h, 0C3h ; 11/11/2020
00006B9C FF                  <1>     db     0FFh   ; crtc regs
00006B9D 003F003F003F003F    <1>     db     00h, 3Fh, 00h, 3Fh, 00h, 3Fh, 00h, 3Fh
00006BA5 003F003F003F003F    <1>     db     00h, 3Fh, 00h, 3Fh, 00h, 3Fh, 00h, 3Fh
00006BAD 01000F00            <1>     db     01h, 00h, 0Fh, 00h  ; actl regs
00006BB1 000000000000050FFF  <1>     db     00h, 00h, 00h, 00h, 00h, 00h, 05h, 0Fh, 0FFh ; grdc regs
                             <1> end_of_vga_params:
                             <1>
                             <1> ; /// End Of VIDEO DATA ///
                             <1>
                             <1> ; 23/11/2020
                             <1> ; VBE 2 BOCHS/QEMU emulator extensions
                             <1> ;   for TRDOS 386 v2 kernel (video bios)
                             <1>
                             <1> ; vbetables.h by Volker Rupper (02/01/2020)
                             <1>
                             <1> b_vbe_modes:
                             <1>     ;/* standard VESA modes */
00006BBA 0001800290010800    <1>     dw 100h,  640, 400,  8
00006BC2 01018002E0010800    <1>     dw 101h,  640, 480,  8
00006BCA 0301200358020800    <1>     dw 103h,  800, 600,  8
00006BD2 0501000400030800    <1>     dw 105h, 1024, 768,  8
00006BDA 0E014001C8001000    <1>     dw 10Eh,  320, 200, 16
00006BE2 0F014001C8001800    <1>     dw 10Fh,  320, 200, 24
00006BEA 11018002E0011000    <1>     dw 111h,  640, 480, 16
00006BF2 12018002E0011800    <1>     dw 112h,  640, 480, 24
00006BFA 1401200358021000    <1>     dw 114h,  800, 600, 16
00006C02 1501200358021800    <1>     dw 115h,  800, 600, 24
00006C0A 1701000400031000    <1>     dw 117h, 1024, 768, 16
00006C12 1801000400031800    <1>     dw 118h, 1024, 768, 24
                             <1>
                             <1> ;/* BOCHS/PLEX86 'own' mode numbers */
00006C1A 40014001C8002000    <1>     dw 140h,  320,  200, 32
00006C22 4101800290012000    <1>     dw 141h,  640,  400, 32
00006C2A 42018002E0012000    <1>     dw 142h,  640,  480, 32
00006C32 4301200358022000    <1>     dw 143h,  800,  600, 32
00006C3A 4401000400032000    <1>     dw 144h, 1024,  768, 32
00006C42 46014001C8000800    <1>     dw 146h,  320,  200,  8
00006C4A 8D010005D0021000    <1>     dw 18Dh, 1280,  720, 16
00006C52 8E010005D0021800    <1>     dw 18Eh, 1280,  720, 24
00006C5A 8F010005D0022000    <1>     dw 18Fh, 1280,  720, 32
00006C62 9001800738041000    <1>     dw 190h, 1920, 1080, 16
00006C6A 9101800738041800    <1>     dw 191h, 1920, 1080, 24
00006C72 9201800738042000    <1>     dw 192h, 1920, 1080, 32
                             <1>
                             <1> end_of_b_vbe_modes:
                             <1>
                             <1> MA1 equ VBE_MODE_ATTRIBUTE_SUPPORTED
                             <1> MA2 equ VBE_MODE_ATTRIBUTE_EXTENDED_INFO_AVAILABLE
                             <1> MA3 equ VBE_MODE_ATTRIBUTE_COLOR_MODE
                             <1> MA4 equ VBE_MODE_ATTRIBUTE_LINEAR_FRAME_BUFFER_MODE
                             <1> MA5 equ     VBE_MODE_ATTRIBUTE_GRAPHICS_MODE
                             <1>
                             <1> MODE_ATTRIBUTES equ MA1|MA2|MA3|MA4|MA5
                             <1>
                             <1> WA1 equ VBE_WINDOW_ATTRIBUTE_RELOCATABLE
                             <1> WA2 equ     VBE_WINDOW_ATTRIBUTE_READABLE
                             <1> WA3 equ VBE_WINDOW_ATTRIBUTE_WRITEABLE
                             <1>
                             <1> WINA_ATTRIBUTES equ WA1|WA2|WA3
                             <1>
                             <1> ; 24/11/2020
                             <1>
                             <1> %if 0
                             <1>
                             <1> MODE_INFO_LIST:
```

```
<1>
<1> ; 24/11/2020
<1> ; '%if 0' disables 24 mode info tables here, until %endif
<1> ; ('set_mode_info_list' will set only 1 list for selected mode)
<1> ; (Purpose: To save about 1 KB kernel size by removing fixed data)
<1>
<1>                     dw      0100h ; 640x400x8
<1> ModeAttributes1:    dw      MODE_ATTRIBUTES
<1> WinAAttributes1:    db      WINA_ATTRIBUTES
<1> WinBAttributes1:    db      0
<1> WinGranularity1:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize1:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment1:               dw      VGAMEM_GRAPH
<1> WinBSegment1:               dw      0000h
<1> WinFuncPtr1:                dd      0
<1> BytesPerScanLine1:          dw      640
<1> XResolution1:               dw      640
<1> YResolution1:               dw      400
<1> XCharSize1:         db      8
<1> YCharSize1:         db      16
<1> NumberOfPlanes1:    db      1
<1> BitsPerPixel1:              db      8
<1> NumberOfBanks1:             db      4
<1> MemoryModel1:               db      VBE_MEMORYMODEL_PACKED_PIXEL
<1> BankSize1:          db      0
<1> NumberOfImagePages1:        db      64
<1> Reserved_page1:             db      0
<1> RedMaskSize1:               db      0
<1> RedFieldPosition1:          db      0
<1> GreenMaskSize1:             db      0
<1> GreenFieldPosition1:        db      0
<1> BlueMaskSize1:              db      0
<1> BlueFieldPosition1:         db      0
<1> RsvdMaskSize1:              db      0
<1> RsvdFieldPosition1:         db      0
<1> DirectColorModeInfo1:       db      0
<1> PhysBasePtr1:               dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset1:        dd      0
<1> OffScreenMemSize1:          dw      0
<1> LinBytesPerScanLine1:       dw      640
<1> BnkNumberOfPages1:          db      0
<1> LinNumberOfPages1:          db      0
<1> LinRedMaskSize1:    db      0
<1> LinRedFieldPosition1:       db      0
<1> LinGreenMaskSize1:          db      0
<1> LinGreenFieldPosition1: db0
<1> LinBlueMaskSize1:   db      0
<1> LinBlueFieldPosition1:      db      0
<1> LinRsvdMaskSize1:   db      0
<1> LinRsvdFieldPosition1:      db      0
<1> MaxPixelClock1:             dd      0
<1>
<1>                     dw      0101h ; 640x480x8
<1> ModeAttributes2:    dw      MODE_ATTRIBUTES
<1> WinAAttributes2:    db      WINA_ATTRIBUTES
<1> WinBAttributes2:    db      0
<1> WinGranularity2:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize2:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment2:               dw      VGAMEM_GRAPH
<1> WinBSegment2:               dw      0000h
<1> WinFuncPtr2:                dd      0
<1> BytesPerScanLine2: dw       640
<1> XResolution2:               dw      640
<1> YResolution2:               dw      480
<1> XCharSize2:         db      8
<1> YCharSize2:         db      16
<1> NumberOfPlanes2:    db      1
<1> BitsPerPixel2:              db      8
<1> NumberOfBanks2:             db      5
<1> MemoryModel2:               db      VBE_MEMORYMODEL_PACKED_PIXEL
<1> BankSize2:          db      0
<1> NumberOfImagePages2:        db      53
<1> Reserved_page2:             db      0
<1> RedMaskSize2:               db      0
<1> RedFieldPosition2: db       0
<1> GreenMaskSize2:             db      0
<1> GreenFieldPosition2:        db      0
<1> BlueMaskSize2:              db      0
<1> BlueFieldPosition2:         db      0
<1> RsvdMaskSize2:              db      0
<1> RsvdFieldPosition2:         db      0
<1> DirectColorModeInfo2:       db      0
<1> PhysBasePtr2:               dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset2:        dd      0
<1> OffScreenMemSize2: dw       0
<1> LinBytesPerScanLine2:       dw      640
<1> BnkNumberOfPages2:          db      0
<1> LinNumberOfPages2:          db      0
<1> LinRedMaskSize2:    db      0
<1> LinRedFieldPosition2:       db      0
<1> LinGreenMaskSize2:          db      0
<1> LinGreenFieldPosition2: db0
<1> LinBlueMaskSize2:   db      0
<1> LinBlueFieldPosition2:      db      0
<1> LinRsvdMaskSize2:   db      0
<1> LinRsvdFieldPosition2:      db      0
<1> MaxPixelClock2:             dd      0
<1>
<1>                     dw      0103h ; 800x600x8
<1> ModeAttributes3:    dw      MODE_ATTRIBUTES
<1> WinAAttributes3:    db      WINA_ATTRIBUTES
<1> WinBAttributes3:    db      0
<1> WinGranularity3:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize3:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment3:               dw      VGAMEM_GRAPH
<1> WinBSegment3:               dw      0000h
<1> WinFuncPtr3:                dd      0
<1> BytesPerScanLine3: dw       800
<1> XResolution3:               dw      800
<1> YResolution3:               dw      600
<1> XCharSize3:         db      8
<1> YCharSize3:         db      16
<1> NumberOfPlanes3:    db      1
<1> BitsPerPixel3:              db      8
<1> NumberOfBanks3:             db      8
<1> MemoryModel3:               db      VBE_MEMORYMODEL_PACKED_PIXEL
<1> BankSize3:          db      0
<1> NumberOfImagePages3:        db      33
<1> Reserved_page3:             db      0
<1> RedMaskSize3:               db      0
<1> RedFieldPosition3: db       0
<1> GreenMaskSize3:             db      0
<1> GreenFieldPosition3:        db      0
<1> BlueMaskSize3:              db      0
```

```
<1> BlueFieldPosition3:       db      0
<1> RsvdMaskSize3:            db      0
<1> RsvdFieldPosition3:       db      0
<1> DirectColorModeInfo3:     db      0
<1> PhysBasePtr3:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset3:      dd      0
<1> OffScreenMemSize3: dw     0
<1> LinBytesPerScanLine3:     dw      800
<1> BnkNumberOfPages3:        db      0
<1> LinNumberOfPages3:        db      0
<1> LinRedMaskSize3:   db     0
<1> LinRedFieldPosition3:     db      0
<1> LinGreenMaskSize3:        db      0
<1> LinGreenFieldPosition:    db      0
<1> LinBlueMaskSize:   db     0
<1> LinBlueFieldPosition:     db      0
<1> LinRsvdMaskSize:   db     0
<1> LinRsvdFieldPosition:     db      0
<1> MaxPixelClock:            dd      0
<1>
<1>                     dw      0105h ; 1024x768x8
<1> ModeAttributes4:    dw      MODE_ATTRIBUTES
<1> WinAAttributes4:    db      WINA_ATTRIBUTES
<1> WinBAttributes4:    db      0
<1> WinGranularity4:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize4:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment4:               dw      VGAMEM_GRAPH
<1> WinBSegment4:               dw      0000h
<1> WinFuncPtr4:                dd      0
<1> BytesPerScanLine4: dw       1024
<1> XResolution4:               dw      1024
<1> YResolution4:               dw      768
<1> XCharSize4:         db      8
<1> YCharSize4:         db      16
<1> NumberOfPlanes4:    db      1
<1> BitsPerPixel4:              db      8
<1> NumberOfBanks4:             db      12
<1> MemoryModel4:               db      VBE_MEMORYMODEL_PACKED_PIXEL
<1> BankSize4:          db      0
<1> NumberOfImagePages4:        db      20
<1> Reserved_page4:             db      0
<1> RedMaskSize4:               db      0
<1> RedFieldPosition4: db       0
<1> GreenMaskSize4:             db      0
<1> GreenFieldPosition4:        db      0
<1> BlueMaskSize4:              db      0
<1> BlueFieldPosition4:         db      0
<1> RsvdMaskSize4:              db      0
<1> RsvdFieldPosition4:         db      0
<1> DirectColorModeInfo4:       db      0
<1> PhysBasePtr4:               dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset4:        dd      0
<1> OffScreenMemSize4: dw       0
<1> LinBytesPerScanLine4:       dw      1024
<1> BnkNumberOfPages4:          db      0
<1> LinNumberOfPages4:          db      0
<1> LinRedMaskSize4:   db       0
<1> LinRedFieldPosition4:       db      0
<1> LinGreenMaskSize4:          db      0
<1> LinGreenFieldPosition4: db 0
<1> LinBlueMaskSize4:  db       0
<1> LinBlueFieldPosition4:      db      0
<1> LinRsvdMaskSize4:  db       0
<1> LinRsvdFieldPosition4:      db      0
<1> MaxPixelClock4:             dd      0
<1>
<1>                     dw      010Eh ; 320x200x16
<1> ModeAttributes5:    dw      MODE_ATTRIBUTES
<1> WinAAttributes5:    db      WINA_ATTRIBUTES
<1> WinBAttributes5:    db      0
<1> WinGranularity5:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize5:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment5:               dw      VGAMEM_GRAPH
<1> WinBSegment5:               dw      0000h
<1> WinFuncPtr5:                dd      0
<1> BytesPerScanLine5: dw       640
<1> XResolution5:               dw      320
<1> YResolution5:               dw      200
<1> XCharSize5:         db      8
<1> YCharSize5:         db      16
<1> NumberOfPlanes5:    db      1
<1> BitsPerPixel5:              db      16
<1> NumberOfBanks5:             db      2
<1> MemoryModel5:               db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize5:          db      0
<1> NumberOfImagePages5:        db      130
<1> Reserved_page5:             db      0
<1> RedMaskSize5:               db      5
<1> RedFieldPosition5: db       11
<1> GreenMaskSize5:             db      6
<1> GreenFieldPosition5:        db      5
<1> BlueMaskSize5:              db      5
<1> BlueFieldPosition5:         db      0
<1> RsvdMaskSize5:              db      0
<1> RsvdFieldPosition5:         db      0
<1> DirectColorModeInfo5:       db      0
<1> PhysBasePtr5:               dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset5:        dd      0
<1> OffScreenMemSize5: dw       0
<1> LinBytesPerScanLine5:       dw      640
<1> BnkNumberOfPages5:          db      0
<1> LinNumberOfPages5:          db      0
<1> LinRedMaskSize5:   db       5
<1> LinRedFieldPosition5:       db      11
<1> LinGreenMaskSize5: db       6
<1> LinGreenFieldPosition5:     db      5
<1> LinBlueMaskSize5:  db       5
<1> LinBlueFieldPosition5:      db      0
<1> LinRsvdMaskSize5:  db       0
<1> LinRsvdFieldPosition5:      db      0
<1> MaxPixelClock5:             dd      0
<1>
<1>                     dw      010Fh ; 320x200x24
<1> ModeAttributes6:    dw      MODE_ATTRIBUTES
<1> WinAAttributes6:    db      WINA_ATTRIBUTES
<1> WinBAttributes6:    db      0
<1> WinGranularity6:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize6:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment6:               dw      VGAMEM_GRAPH
<1> WinBSegment6:               dw      0000h
<1> WinFuncPtr6:                dd      0
<1> BytesPerScanLine6: dw       960
<1> XResolution6:               dw      320
<1> YResolution6:               dw      200
```

```
                        <1> XCharSize6:        db      8
                        <1> YCharSize6:        db      16
                        <1> NumberOfPlanes6:   db      1
                        <1> BitsPerPixel6:             db      24
                        <1> NumberOfBanks6:            db      3
                        <1> MemoryModel6:              db      VBE_MEMORYMODEL_DIRECT_COLOR
                        <1> BankSize6:         db      0
                        <1> NumberOfImagePages6:       db      86
                        <1> Reserved_page6:            db      0
                        <1> RedMaskSize6:              db      8
                        <1> RedFieldPosition6: db      16
                        <1> GreenMaskSize6:            db      8
                        <1> GreenFieldPosition6:       db      8
                        <1> BlueMaskSize6:             db      8
                        <1> BlueFieldPosition6:        db      0
                        <1> RsvdMaskSize6:             db      0
                        <1> RsvdFieldPosition6:        db      0
                        <1> DirectColorModeInfo6:      db      0
                        <1> PhysBasePtr6:              dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
                        <1> OffScreenMemOffset6:       dd      0
                        <1> OffScreenMemSize6: dw      0
                        <1> LinBytesPerScanLine6:      dw      960
                        <1> BnkNumberOfPages6:         db      0
                        <1> LinNumberOfPages6:         db      0
                        <1> LinRedMaskSize6:   db      8
                        <1> LinRedFieldPosition6:      db      16
                        <1> LinGreenMaskSize6: db      8
                        <1> LinGreenFieldPosition6:    db      8
                        <1> LinBlueMaskSize6:  db      8
                        <1> LinBlueFieldPosition6:     db      0
                        <1> LinRsvdMaskSize6:  db      0
                        <1> LinRsvdFieldPosition6:     db      0
                        <1> MaxPixelClock6:            dd      0
                        <1>
                        <1>                    dw      0111h ; 640x480x16
                        <1> ModeAttributes7:   dw      MODE_ATTRIBUTES
                        <1> WinAAttributes7:   db      WINA_ATTRIBUTES
                        <1> WinBAttributes7:   db      0
                        <1> WinGranularity7:   dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinSize7:          dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinASegment7:              dw      VGAMEM_GRAPH
                        <1> WinBSegment7:              dw      0000h
                        <1> WinFuncPtr7:               dd      0
                        <1> BytesPerScanLine7: dw      1280
                        <1> XResolution7:              dw      640
                        <1> YResolution7:              dw      480
                        <1> XCharSize7:        db      8
                        <1> YCharSize7:        db      16
                        <1> NumberOfPlanes7:   db      1
                        <1> BitsPerPixel7:             db      16
                        <1> NumberOfBanks7:            db      10
                        <1> MemoryModel7:              db      VBE_MEMORYMODEL_DIRECT_COLOR
                        <1> BankSize7:         db      0
                        <1> NumberOfImagePages7:       db      26
                        <1> Reserved_page7:            db      0
                        <1> RedMaskSize7:              db      5
                        <1> RedFieldPosition7: db      11
                        <1> GreenMaskSize7:            db      6
                        <1> GreenFieldPosition7:       db      5
                        <1> BlueMaskSize7:             db      5
                        <1> BlueFieldPosition7:        db      0
                        <1> RsvdMaskSize7:             db      0
                        <1> RsvdFieldPosition7:        db      0
                        <1> DirectColorModeInfo7:      db      0
                        <1> PhysBasePtr7:              dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS,
                        <1> OffScreenMemOffset7:       dd      0
                        <1> OffScreenMemSize7: dw      0
                        <1> LinBytesPerScanLine7:      dw      1280
                        <1> BnkNumberOfPages7:         db      0
                        <1> LinNumberOfPages7:         db      0
                        <1> LinRedMaskSize7:   db      5
                        <1> LinRedFieldPosition7:      db      11
                        <1> LinGreenMaskSize7: db      6
                        <1> LinGreenFieldPosition7:    db      5
                        <1> LinBlueMaskSize7:  db      5
                        <1> LinBlueFieldPosition7:     db      0
                        <1> LinRsvdMaskSize7:  db      0
                        <1> LinRsvdFieldPosition7:     db      0
                        <1> MaxPixelClock7:            dd      0
                        <1>
                        <1>                    dw      0112h ; 640x480x24
                        <1> ModeAttributes8:   dw      MODE_ATTRIBUTES
                        <1> WinAAttributes8:   db      WINA_ATTRIBUTES
                        <1> WinBAttributes8:   db      0
                        <1> WinGranularity8:   dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinSize8:          dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinASegment8:              dw      VGAMEM_GRAPH
                        <1> WinBSegment8:              dw      0000h
                        <1> WinFuncPtr8:               dd      0
                        <1> BytesPerScanLine8: dw      1920
                        <1> XResolution8:              dw      640
                        <1> YResolution8:              dw      480
                        <1> XCharSize8:        db      8
                        <1> YCharSize8:        db      16
                        <1> NumberOfPlanes8:   db      1
                        <1> BitsPerPixel8:             db      24
                        <1> NumberOfBanks8:            db      15
                        <1> MemoryModel8:              db      VBE_MEMORYMODEL_DIRECT_COLOR
                        <1> BankSize8:         db      0
                        <1> NumberOfImagePages8:       db      17
                        <1> Reserved_page8:            db      0
                        <1> RedMaskSize8:              db      8
                        <1> RedFieldPosition8: db      16
                        <1> GreenMaskSize8:            db      8
                        <1> GreenFieldPosition8:       db      8
                        <1> BlueMaskSize8:             db      8
                        <1> BlueFieldPosition8:        db      0
                        <1> RsvdMaskSize8:             db      0
                        <1> RsvdFieldPosition8:        db      0
                        <1> DirectColorModeInfo8:      db      0
                        <1> PhysBasePtr8:              dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
                        <1> OffScreenMemOffset8:       dd      0
                        <1> OffScreenMemSize8: dw      0
                        <1> LinBytesPerScanLine8:      dw      1920
                        <1> BnkNumberOfPages8:         db      0
                        <1> LinNumberOfPages8:         db      0
                        <1> LinRedMaskSize8:   db      8
                        <1> LinRedFieldPosition8:      db      16
                        <1> LinGreenMaskSize8: db      8
                        <1> LinGreenFieldPosition8:    db      8
                        <1> LinBlueMaskSize8:  db      8
                        <1> LinBlueFieldPosition8:     db      0
                        <1> LinRsvdMaskSize8:  db      0
                        <1> LinRsvdFieldPosition8:     db      0
```

```
<1> MaxPixelClock8:           dd      0
<1>
<1>                     dw      0114h ; 800x600x16
<1> ModeAttributes9:    dw      MODE_ATTRIBUTES
<1> WinAAttributes9:    db      WINA_ATTRIBUTES
<1> WinBAttributes9:    db      0
<1> WinGranularity9:    dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize9:           dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment9:               dw      VGAMEM_GRAPH
<1> WinBSegment9:               dw      0000h
<1> WinFuncPtr9:                dd      0
<1> BytesPerScanLine9: dw       1600
<1> XResolution9:               dw      800
<1> YResolution9:               dw      600
<1> XCharSize9:         db      8
<1> YCharSize9:         db      16
<1> NumberOfPlanes9:    db      1
<1> BitsPerPixel9:              db      16
<1> NumberOfBanks9:             db      15
<1> MemoryModel9:               db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize9:          db      0
<1> NumberOfImagePages9:        db      16
<1> Reserved_page9:             db      0
<1> RedMaskSize9:               db      5
<1> RedFieldPosition9: db       11
<1> GreenMaskSize9:             db      6
<1> GreenFieldPosition9:        db      5
<1> BlueMaskSize9:              db      5
<1> BlueFieldPosition9:         db      0
<1> RsvdMaskSize9:              db      0
<1> RsvdFieldPosition9:         db      0
<1> DirectColorModeInfo9:       db      0
<1> PhysBasePtr9:               dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset9:        dd      0
<1> OffScreenMemSize9: dw       0
<1> LinBytesPerScanLine9:       dw      1600
<1> BnkNumberOfPages9:          db      0
<1> LinNumberOfPages9:          db      0
<1> LinRedMaskSize9:    db      5
<1> LinRedFieldPosition9:       db      11
<1> LinGreenMaskSize9: db       6
<1> LinGreenFieldPosition9:     db      5
<1> LinBlueMaskSize9:   db      5
<1> LinBlueFieldPosition9:      db      0
<1> LinRsvdMaskSize9:   db      0
<1> LinRsvdFieldPosition9:      db      0
<1> MaxPixelClock9:             dd      0
<1>
<1>                     dw      0115h ; 800x600x24
<1> ModeAttributes10:   dw      MODE_ATTRIBUTES
<1> WinAAttributes10:   db      WINA_ATTRIBUTES
<1> WinBAttributes10:   db      0
<1> WinGranularity10:   dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize10:          dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment10:              dw      VGAMEM_GRAPH
<1> WinBSegment10:              dw      0000h
<1> WinFuncPtr10:               dd      0
<1> BytesPerScanLine10:         dw      2400
<1> XResolution10:              dw      800
<1> YResolution10:              dw      600
<1> XCharSize10:                db      8
<1> YCharSize10:                db      16
<1> NumberOfPlanes10:   db      1
<1> BitsPerPixel10:             db      24
<1> NumberOfBanks10:    db      22
<1> MemoryModel10:              db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize10:         db      0
<1> NumberOfImagePages10:       db      10
<1> Reserved_page10:    db      0
<1> RedMaskSize10:              db      8
<1> RedFieldPosition10:         db      16
<1> GreenMaskSize10:    db      8
<1> GreenFieldPosition10:       db      8
<1> BlueMaskSize10:             db      8
<1> BlueFieldPosition10:        db      0
<1> RsvdMaskSize10:             db      0
<1> RsvdFieldPosition10:        db      0
<1> DirectColorModeInfo10:      db      0
<1> PhysBasePtr10:              dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset10:       dd      0
<1> OffScreenMemSize10:         dw      0
<1> LinBytesPerScanLine10:      dw      2400
<1> BnkNumberOfPages10:         db      0
<1> LinNumberOfPages10:         db      0
<1> LinRedMaskSize10:   db      8
<1> LinRedFieldPosition10:      db      16
<1> LinGreenMaskSize10:         db      8
<1> LinGreenFieldPosition10:db  8
<1> LinBlueMaskSize10: db       8
<1> LinBlueFieldPosition10:     db      0
<1> LinRsvdMaskSize10: db       0
<1> LinRsvdFieldPosition10:     db      0
<1> MaxPixelClock10:    dd      0
<1>
<1>                     dw      0117h ; 1024x768x16
<1> ModeAttributes11:   dw      MODE_ATTRIBUTES
<1> WinAAttributes11:   db      WINA_ATTRIBUTES
<1> WinBAttributes11:   db      0
<1> WinGranularity11:   dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize11:          dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment11:              dw      VGAMEM_GRAPH
<1> WinBSegment11:              dw      0000h
<1> WinFuncPtr11:               dd      0
<1> BytesPerScanLine11:         dw      2048
<1> XResolution11:              dw      1024
<1> YResolution11:              dw      768
<1> XCharSize11:                db      8
<1> YCharSize11:                db      16
<1> NumberOfPlanes11:   db      1
<1> BitsPerPixel11:             db      16
<1> NumberOfBanks11:    db      24
<1> MemoryModel11:              db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize11:         db      0
<1> NumberOfImagePages11:       db      9
<1> Reserved_page11:    db      0
<1> RedMaskSize11:              db      5
<1> RedFieldPosition11:         db      11
<1> GreenMaskSize11:    db      6
<1> GreenFieldPosition11:       db      5
<1> BlueMaskSize11:             db      5
<1> BlueFieldPosition11:        db      0
<1> RsvdMaskSize11:             db      0
<1> RsvdFieldPosition11:        db      0
<1> DirectColorModeInfo11:      db      0
```

```
<1> PhysBasePtr11:            dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS,
<1> OffScreenMemOffset11:     dd      0
<1> OffScreenMemSize11:       dw      0
<1> LinBytesPerScanLine11:    dw      2048
<1> BnkNumberOfPages11:       db      0
<1> LinNumberOfPages11:       db      0
<1> LinRedMaskSize11:  db     5
<1> LinRedFieldPosition11:    db      11
<1> LinGreenMaskSize11:       db      6
<1> LinGreenFieldPosition11:db 5
<1> LinBlueMaskSize11: db     5
<1> LinBlueFieldPosition11:   db      0
<1> LinRsvdMaskSize11: db     0
<1> LinRsvdFieldPosition11:   db      0
<1> MaxPixelClock11:   dd     0
<1>
<1>                    dw     0118h ; 1024x768x24
<1> ModeAttributes12:  dw     MODE_ATTRIBUTES
<1> WinAAttributes12:  db     WINA_ATTRIBUTES
<1> WinBAttributes12:  db     0
<1> WinGranularity12:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize12:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment12:            dw      VGAMEM_GRAPH
<1> WinBSegment12:            dw      0000h
<1> WinFuncPtr12:             dd      0
<1> BytesPerScanLine12:       dw      3072
<1> XResolution12:            dw      1024
<1> YResolution12:            dw      768
<1> XCharSize12:              db      8
<1> YCharSize12:              db      16
<1> NumberOfPlanes12:  db     1
<1> BitsPerPixel12:           db      24
<1> NumberOfBanks12:   db     36
<1> MemoryModel12:            db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize12:        db     0
<1> NumberOfImagePages12:     db      6
<1> Reserved_page12:   db     0
<1> RedMaskSize12:            db      8
<1> RedFieldPosition12:       db      16
<1> GreenMaskSize12:   db     8
<1> GreenFieldPosition12:     db      8
<1> BlueMaskSize12:           db      8
<1> BlueFieldPosition12:      db      0
<1> RsvdMaskSize12:           db      0
<1> RsvdFieldPosition12:      db      0
<1> DirectColorModeInfo12:    db      0
<1> PhysBasePtr12:            dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset12:     dd      0
<1> OffScreenMemSize12:       dw      0
<1> LinBytesPerScanLine12:    dw      3072
<1> BnkNumberOfPages12:       db      0
<1> LinNumberOfPages12:       db      0
<1> LinRedMaskSize12:  db     8
<1> LinRedFieldPosition12:    db      16
<1> LinGreenMaskSize12:       db      8
<1> LinGreenFieldPosition12:db 8
<1> LinBlueMaskSize12: db     8
<1> LinBlueFieldPosition12:   db      0
<1> LinRsvdMaskSize12: db     0
<1> LinRsvdFieldPosition12:   db      0
<1> MaxPixelClock12:   dd     0
<1>
<1>                    dw     0140h ; 320x200x32
<1> ModeAttributes13:  dw     MODE_ATTRIBUTES
<1> WinAAttributes13:  db     WINA_ATTRIBUTES
<1> WinBAttributes13:  db     0
<1> WinGranularity13:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize13:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment13:            dw      VGAMEM_GRAPH
<1> WinBSegment13:            dw      0000h
<1> WinFuncPtr13:             dd      0
<1> BytesPerScanLine13:       dw      1280
<1> XResolution13:            dw      320
<1> YResolution13:            dw      200
<1> XCharSize13:              db      8
<1> YCharSize13:              db      16
<1> NumberOfPlanes13:  db     1
<1> BitsPerPixel13:           db      32
<1> NumberOfBanks13:   db     4
<1> MemoryModel13:            db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize13:        db     0
<1> NumberOfImagePages13:     db      64
<1> Reserved_page13:   db     0
<1> RedMaskSize13:            db      8
<1> RedFieldPosition13:       db      16
<1> GreenMaskSize13:   db     8
<1> GreenFieldPosition13:     db      8
<1> BlueMaskSize13:           db      8
<1> BlueFieldPosition13:      db      0
<1> RsvdMaskSize13:           db      8
<1> RsvdFieldPosition13:      db      24
<1> DirectColorModeInfo13:    db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
<1> PhysBasePtr13:            dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset13:     dd      0
<1> OffScreenMemSize13:       dw      0
<1> LinBytesPerScanLine13:    dw      1280
<1> BnkNumberOfPages13:       db      0
<1> LinNumberOfPages13:       db      0
<1> LinRedMaskSize13:  db     8
<1> LinRedFieldPosition13:    db      16
<1> LinGreenMaskSize13:       db      8
<1> LinGreenFieldPosition13:db 8
<1> LinBlueMaskSize13: db     8
<1> LinBlueFieldPosition13:   db      0
<1> LinRsvdMaskSize13: db     8
<1> LinRsvdFieldPosition13:   db      24
<1> MaxPixelClock13:   dd     0
<1>
<1>                    dw     0141h ; 640x400x32
<1> ModeAttributes14:  dw     MODE_ATTRIBUTES
<1> WinAAttributes14:  db     WINA_ATTRIBUTES
<1> WinBAttributes14:  db     0
<1> WinGranularity14:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize14:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment14:            dw      VGAMEM_GRAPH
<1> WinBSegment14:            dw      0000h
<1> WinFuncPtr14:             dd      0
<1> BytesPerScanLine14:       dw      2560
<1> XResolution14:            dw      640
<1> YResolution14:            dw      400
<1> XCharSize14:              db      8
<1> YCharSize14:              db      16
<1> NumberOfPlanes14:  db     1
<1> BitsPerPixel14:           db      32
```

```
<1> NumberOfBanks14:   db      16
<1> MemoryModel14:             db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize14:        db      0
<1> NumberOfImagePages14:      db      15
<1> Reserved_page14:   db      0
<1> RedMaskSize14:             db      8
<1> RedFieldPosition14:        db      16
<1> GreenMaskSize14:   db      8
<1> GreenFieldPosition14:      db      8
<1> BlueMaskSize14:            db      8
<1> BlueFieldPosition14:       db      0
<1> RsvdMaskSize14:            db      8
<1> RsvdFieldPosition14:       db      24
<1> DirectColorModeInfo14:     db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
<1> PhysBasePtr14:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset14:      dd      0
<1> OffScreenMemSize14:        dw      0
<1> LinBytesPerScanLine14:     dw      2560
<1> BnkNumberOfPages14:        db      0
<1> LinNumberOfPages14:        db      0
<1> LinRedMaskSize14:  db      8
<1> LinRedFieldPosition14:     db      16
<1> LinGreenMaskSize14:        db      8
<1> LinGreenFieldPosition14:db 8
<1> LinBlueMaskSize14: db      8
<1> LinBlueFieldPosition14:    db      0
<1> LinRsvdMaskSize14: db      8
<1> LinRsvdFieldPosition14:    db      24
<1> MaxPixelClock14:   dd      0
<1>
<1>                    dw      0142 ; 640x480x32
<1> ModeAttributes15:  dw      MODE_ATTRIBUTES
<1> WinAAttributes15:  db      WINA_ATTRIBUTES
<1> WinBAttributes15:  db      0
<1> WinGranularity15:  dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize15:         dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment15:             dw      VGAMEM_GRAPH
<1> WinBSegment15:             dw      0000h
<1> WinFuncPtr15:              dd      0
<1> BytesPerScanLine15:        dw      2560
<1> XResolution15:             dw      640
<1> YResolution15:             dw      480
<1> XCharSize15:               db      8
<1> YCharSize15:               db      16
<1> NumberOfPlanes15:  db      1
<1> BitsPerPixel15:            db      32
<1> NumberOfBanks15:   db      19
<1> MemoryModel15:             db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize15:        db      0
<1> NumberOfImagePages15:      db      12
<1> Reserved_page15:   db      0
<1> RedMaskSize15:             db      8
<1> RedFieldPosition15:        db      16
<1> GreenMaskSize15:   db      8
<1> GreenFieldPosition15:      db      8
<1> BlueMaskSize15:            db      8
<1> BlueFieldPosition15:       db      0
<1> RsvdMaskSize15:            db      8
<1> RsvdFieldPosition15:       db      24
<1> DirectColorModeInfo15:     db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE,
<1> PhysBasePtr15:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS,
<1> OffScreenMemOffset15:      dd      0
<1> OffScreenMemSize15:        dw      0
<1> LinBytesPerScanLine15:     dw      2560
<1> BnkNumberOfPages15:        db      0
<1> LinNumberOfPages15:        db      0
<1> LinRedMaskSize15:  db      8
<1> LinRedFieldPosition15:     db      16
<1> LinGreenMaskSize15:        db      8
<1> LinGreenFieldPosition15:db 8
<1> LinBlueMaskSize15: db      8
<1> LinBlueFieldPosition15:    db      0
<1> LinRsvdMaskSize15: db      8
<1> LinRsvdFieldPosition15:    db      24
<1> MaxPixelClock15:   dd      0
<1>
<1>                    dw      0143h ; 800x600x32
<1> ModeAttributes16:  dw      MODE_ATTRIBUTES
<1> WinAAttributes16:  db      WINA_ATTRIBUTES
<1> WinBAttributes16:  db      0
<1> WinGranularity16:  dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize16:         dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment16:             dw      VGAMEM_GRAPH
<1> WinBSegment16:             dw      0000h
<1> WinFuncPtr16:              dd      0
<1> BytesPerScanLine16:        dw      3200
<1> XResolution16:             dw      800
<1> YResolution16:             dw      600
<1> XCharSize16:               db      8
<1> YCharSize16:               db      16
<1> NumberOfPlanes16:  db      1
<1> BitsPerPixel16:            db      32
<1> NumberOfBanks16:   db      30
<1> MemoryModel16:             db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize16:        db      0
<1> NumberOfImagePages16:      db      7
<1> Reserved_page16:   db      0
<1> RedMaskSize16:             db      8
<1> RedFieldPosition16:        db      16
<1> GreenMaskSize16:   db      8
<1> GreenFieldPosition16:      db      8
<1> BlueMaskSize16:            db      8
<1> BlueFieldPosition16:       db      0
<1> RsvdMaskSize16:            db      8
<1> RsvdFieldPosition16:       db      24
<1> DirectColorModeInfo16:     db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE,
<1> PhysBasePtr16:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS,
<1> OffScreenMemOffset16:      dd      0
<1> OffScreenMemSize16:        dw      0
<1> LinBytesPerScanLine16:     dw      3200
<1> BnkNumberOfPages16:        db      0
<1> LinNumberOfPages16:        db      0
<1> LinRedMaskSize16:  db      8
<1> LinRedFieldPosition16:     db      16
<1> LinGreenMaskSize16:        db      8
<1> LinGreenFieldPosition16:db 8
<1> LinBlueMaskSize16: db      8
<1> LinBlueFieldPosition16:    db      0
<1> LinRsvdMaskSize16: db      8
<1> LinRsvdFieldPosition16:    db      24
<1> MaxPixelClock16:   dd      0
<1>
<1>                    dw      0144h ; 1024x768x32
<1> ModeAttributes17:  dw      MODE_ATTRIBUTES
```

```
<1> WinAAttributes17:  db      WINA_ATTRIBUTES
<1> WinBAttributes17:  db      0
<1> WinGranularity17:  dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize17:         dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment17:             dw      VGAMEM_GRAPH
<1> WinBSegment17:             dw      0000h
<1> WinFuncPtr17:              dd      0
<1> BytesPerScanLine17:        dw      4096
<1> XResolution17:             dw      1024
<1> YResolution17:             dw      768
<1> XCharSize17:               db      8
<1> YCharSize17:               db      16
<1> NumberOfPlanes17:  db      1
<1> BitsPerPixel17:            db      32
<1> NumberOfBanks17:   db      48
<1> MemoryModel17:             db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize17:        db      0
<1> NumberOfImagePages17:      db      4
<1> Reserved_page17:   db      0
<1> RedMaskSize17:             db      8
<1> RedFieldPosition17:        db      16
<1> GreenMaskSize17:   db      8
<1> GreenFieldPosition17:      db      8
<1> BlueMaskSize17:            db      8
<1> BlueFieldPosition17:       db      0
<1> RsvdMaskSize17:            db      8
<1> RsvdFieldPosition17:       db      24
<1> DirectColorModeInfo17:     db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
<1> PhysBasePtr17:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset17:      dd      0
<1> OffScreenMemSize17:        dw      0
<1> LinBytesPerScanLine17:     dw      4096
<1> BnkNumberOfPages17:        db      0
<1> LinNumberOfPages17:        db      0
<1> LinRedMaskSize17:  db      8
<1> LinRedFieldPosition17:     db      16
<1> LinGreenMaskSize17:        db      8
<1> LinGreenFieldPosition17:db 8
<1> LinBlueMaskSize17: db      8
<1> LinBlueFieldPosition17:    db      0
<1> LinRsvdMaskSize17: db      8
<1> LinRsvdFieldPosition17:    db      24
<1> MaxPixelClock17:   dd      0
<1>
<1>                    dw      0146h ; 320x200x8
<1> ModeAttributes18:  dw      MODE_ATTRIBUTES
<1> WinAAttributes18:  db      WINA_ATTRIBUTES
<1> WinBAttributes18:  db      0
<1> WinGranularity18:  dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize18:         dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment18:             dw      VGAMEM_GRAPH
<1> WinBSegment18:             dw      0000h
<1> WinFuncPtr18:              dd      0
<1> BytesPerScanLine18:        dw      320
<1> XResolution18:             dw      320
<1> YResolution18:             dw      200
<1> XCharSize18:               db      8
<1> YCharSize18:               db      16
<1> NumberOfPlanes18:  db      1
<1> BitsPerPixel18:            db      8
<1> NumberOfBanks18:   db      1
<1> MemoryModel18:             db      VBE_MEMORYMODEL_PACKED_PIXEL
<1> BankSize18:        db      0
<1> NumberOfImagePages18:      db      255 ; 261 in vbetables.h (03/01/2020) !
<1> Reserved_page18:   db      0
<1> RedMaskSize18:             db      0
<1> RedFieldPosition18:        db      0
<1> GreenMaskSize18:   db      0
<1> GreenFieldPosition18:      db      0
<1> BlueMaskSize18:            db      0
<1> BlueFieldPosition18:       db      0
<1> RsvdMaskSize18:            db      0
<1> RsvdFieldPosition18:       db      0
<1> DirectColorModeInfo18:     db      0
<1> PhysBasePtr18:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset18:      dd      0
<1> OffScreenMemSize18:        dw      0
<1> LinBytesPerScanLine18:     dw      320
<1> BnkNumberOfPages18:        db      0
<1> LinNumberOfPages18:        db      0
<1> LinRedMaskSize18:  db      0
<1> LinRedFieldPosition18:     db      0
<1> LinGreenMaskSize18:        db      0
<1> LinGreenFieldPosition18:db 0
<1> LinBlueMaskSize18:         db      0
<1> LinBlueFieldPosition18: db 0
<1> LinRsvdMaskSize18:         db      0
<1> LinRsvdFieldPosition18: db 0
<1> MaxPixelClock18:   dd      0
<1>
<1>                    dw      018Dh ; 1280x720x16
<1> ModeAttributes19:  dw      MODE_ATTRIBUTES
<1> WinAAttributes19:  db      WINA_ATTRIBUTES
<1> WinBAttributes19:  db      0
<1> WinGranularity19:  dw      VBE_DISPI_BANK_SIZE_KB
<1> WinSize19:         dw      VBE_DISPI_BANK_SIZE_KB
<1> WinASegment19:             dw      VGAMEM_GRAPH
<1> WinBSegment19:             dw      0000h
<1> WinFuncPtr19:              dd      0
<1> BytesPerScanLine19:        dw      2560
<1> XResolution19:             dw      1280
<1> YResolution19:             dw      720
<1> XCharSize19:               db      8
<1> YCharSize19:               db      16
<1> NumberOfPlanes19:  db      1
<1> BitsPerPixel19:            db      16
<1> NumberOfBanks19:   db      29
<1> MemoryModel19:             db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize19:        db      0
<1> NumberOfImagePages19:      db      8
<1> Reserved_page19:   db      0
<1> RedMaskSize19:             db      5
<1> RedFieldPosition19:        db      11
<1> GreenMaskSize19:   db      6
<1> GreenFieldPosition19:      db      5
<1> BlueMaskSize19:            db      5
<1> BlueFieldPosition19:       db      0
<1> RsvdMaskSize19:            db      0
<1> RsvdFieldPosition19:       db      0
<1> DirectColorModeInfo19:     db      0
<1> PhysBasePtr19:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset19:      dd      0
<1> OffScreenMemSize19:        dw      0
<1> LinBytesPerScanLine19:     dw      2560
```

```
<1> BnkNumberOfPages19:       db      0
<1> LinNumberOfPages19:       db      0
<1> LinRedMaskSize19:  db     5
<1> LinRedFieldPosition19:    db      11
<1> LinGreenMaskSize19:       db      6
<1> LinGreenFieldPosition19:db5
<1> LinBlueMaskSize19: db     5
<1> LinBlueFieldPosition19:   db      0
<1> LinRsvdMaskSize19: db     0
<1> LinRsvdFieldPosition19:   db      0
<1> MaxPixelClock19:   dd     0
<1>
<1>                    dw     018Eh ; 1280x720x24
<1> ModeAttributes20:  dw     MODE_ATTRIBUTES
<1> WinAAttributes20:  db     WINA_ATTRIBUTES
<1> WinBAttributes20:  db     0
<1> WinGranularity20:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize20:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment20:            dw      VGAMEM_GRAPH
<1> WinBSegment20:            dw      0000h
<1> WinFuncPtr20:             dd      0
<1> BytesPerScanLine20:       dw      3840
<1> XResolution20:            dw      1280
<1> YResolution20:            dw      720
<1> XCharSize20:              db      8
<1> YCharSize20:              db      16
<1> NumberOfPlanes20:  db     1
<1> BitsPerPixel20:           db      24
<1> NumberOfBanks20:   db     43
<1> MemoryModel20:            db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize20:        db     0
<1> NumberOfImagePages20:     db      5
<1> Reserved_page20:   db     0
<1> RedMaskSize20:            db      8
<1> RedFieldPosition20:       db      16
<1> GreenMaskSize20:   db     8
<1> GreenFieldPosition20:     db      8
<1> BlueMaskSize20:           db      8
<1> BlueFieldPosition20:      db      0
<1> RsvdMaskSize20:           db      0
<1> RsvdFieldPosition20:      db      0
<1> DirectColorModeInfo20:    db      0
<1> PhysBasePtr20:            dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset20:     dd      0
<1> OffScreenMemSize20:       dw      0
<1> LinBytesPerScanLine20:    dw      3840
<1> BnkNumberOfPages20:       db      0
<1> LinNumberOfPages20:       db      0
<1> LinRedMaskSize20:  db     8
<1> LinRedFieldPosition20:    db      16
<1> LinGreenMaskSize20:       db      8
<1> LinGreenFieldPosition20:db8
<1> LinBlueMaskSize20: db     8
<1> LinBlueFieldPosition20:   db      0
<1> LinRsvdMaskSize20: db     0
<1> LinRsvdFieldPosition20:   db      0
<1> MaxPixelClock20:   dd     0
<1>
<1>                    dw     018Fh ; 1280x720x32
<1> ModeAttributes21:  dw     MODE_ATTRIBUTES
<1> WinAAttributes21:  db     WINA_ATTRIBUTES
<1> WinBAttributes21:  db     0
<1> WinGranularity21:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize21:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment21:            dw      VGAMEM_GRAPH
<1> WinBSegment21:            dw      0000h
<1> WinFuncPtr21:             dd      0
<1> BytesPerScanLine21:       dw      5120
<1> XResolution21:            dw      1280
<1> YResolution21:            dw      720
<1> XCharSize21:              db      8
<1> YCharSize21:              db      16
<1> NumberOfPlanes21:  db     1
<1> BitsPerPixel21:           db      32
<1> NumberOfBanks21:   db     57
<1> MemoryModel21:            db      VBE_MEMORYMODEL_DIRECT_COLOR
<1> BankSize21:        db     0
<1> NumberOfImagePages21:     db      3
<1> Reserved_page21:   db     0
<1> RedMaskSize21:            db      8
<1> RedFieldPosition21:       db      16
<1> GreenMaskSize21:   db     8
<1> GreenFieldPosition21:     db      8
<1> BlueMaskSize21:           db      8
<1> BlueFieldPosition21:      db      0
<1> RsvdMaskSize21:           db      8
<1> RsvdFieldPosition21:      db      24
<1> DirectColorModeInfo21:    db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
<1> PhysBasePtr21:            dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
<1> OffScreenMemOffset21:     dd      0
<1> OffScreenMemSize21:       dw      0
<1> LinBytesPerScanLine21:    dw      5120
<1> BnkNumberOfPages21:       db      0
<1> LinNumberOfPages21:       db      0
<1> LinRedMaskSize21:  db     8
<1> LinRedFieldPosition21:    db      16
<1> LinGreenMaskSize21:       db      8
<1> LinGreenFieldPosition21:db8
<1> LinBlueMaskSize21: db     8
<1> LinBlueFieldPosition21:   db      0
<1> LinRsvdMaskSize21: db     8
<1> LinRsvdFieldPosition21:   db      24
<1> MaxPixelClock21:   dd     0
<1>
<1>                    dw     0190h ; 1920x1080x16
<1> ModeAttributes22:  dw     MODE_ATTRIBUTES
<1> WinAAttributes22:  db     WINA_ATTRIBUTES
<1> WinBAttributes22:  db     0
<1> WinGranularity22:  dw     VBE_DISPI_BANK_SIZE_KB
<1> WinSize22:         dw     VBE_DISPI_BANK_SIZE_KB
<1> WinASegment22:            dw      VGAMEM_GRAPH
<1> WinBSegment22:            dw      0000h
<1> WinFuncPtr22:             dd      0
<1> BytesPerScanLine22:       dw      3840
<1> XResolution22:            dw      1920
<1> YResolution22:            dw      1080
<1> XCharSize22:              db      8
<1> YCharSize22:              db      16
<1> NumberOfPlanes22:  db     1
<1> BitsPerPixel22:           db      16
<1> NumberOfBanks22:   db     64
<1> MemoryModel22:            db      VBE_MEMORYMODEL_DIRECT_COLOR,
<1> BankSize22:        db     0
<1> NumberOfImagePages22:     db      3
```

```
                        <1> Reserved_page22:   db      0
                        <1> RedMaskSize22:             db      5
                        <1> RedFieldPosition22:        db      11
                        <1> GreenMaskSize22:   db      6
                        <1> GreenFieldPosition22:      db      5
                        <1> BlueMaskSize22:            db      5
                        <1> BlueFieldPosition22:       db      0
                        <1> RsvdMaskSize22:            db      0
                        <1> RsvdFieldPosition22:       db      0
                        <1> DirectColorModeInfo22:     db      0
                        <1> PhysBasePtr22:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
                        <1> OffScreenMemOffset22:      dd      0
                        <1> OffScreenMemSize22:        dw      0
                        <1> LinBytesPerScanLine22:     dw      3840
                        <1> BnkNumberOfPages22:        db      0
                        <1> LinNumberOfPages22:        db      0
                        <1> LinRedMaskSize22:  db      5
                        <1> LinRedFieldPosition22:     db      11
                        <1> LinGreenMaskSize22:        db      6
                        <1> LinGreenFieldPosition22:db 5
                        <1> LinBlueMaskSize22: db      5
                        <1> LinBlueFieldPosition22:    db      0
                        <1> LinRsvdMaskSize22: db      0
                        <1> LinRsvdFieldPosition22:    db      0
                        <1> MaxPixelClock22:   dd      0
                        <1>
                        <1>                    dw      0191h ; 1920x1080x24
                        <1> ModeAttributes23:  dw      MODE_ATTRIBUTES
                        <1> WinAAttributes23:  db      WINA_ATTRIBUTES
                        <1> WinBAttributes23:  db      0
                        <1> WinGranularity23:  dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinSize23:         dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinASegment23:             dw      VGAMEM_GRAPH
                        <1> WinBSegment23:             dw      0000h
                        <1> WinFuncPtr23:              dd      0
                        <1> BytesPerScanLine23:        dw      5760
                        <1> XResolution23:             dw      1920
                        <1> YResolution23:             dw      1080
                        <1> XCharSize23:               db      8
                        <1> YCharSize23:               db      16
                        <1> NumberOfPlanes23:  db      1
                        <1> BitsPerPixel23:            db      24
                        <1> NumberOfBanks23:   db      95
                        <1> MemoryModel23:             db      VBE_MEMORYMODEL_DIRECT_COLOR
                        <1> BankSize23:        db      0
                        <1> NumberOfImagePages23:      db      1
                        <1> Reserved_page23:   db      0
                        <1> RedMaskSize23:             db      8
                        <1> RedFieldPosition23:        db      16
                        <1> GreenMaskSize23:   db      8
                        <1> GreenFieldPosition23:      db      8
                        <1> BlueMaskSize23:            db      8
                        <1> BlueFieldPosition23:       db      0
                        <1> RsvdMaskSize23:            db      0
                        <1> RsvdFieldPosition23:       db      0
                        <1> DirectColorModeInfo23:     db      0
                        <1> PhysBasePtr23:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
                        <1> OffScreenMemOffset23:      dd      0
                        <1> OffScreenMemSize23:        dw      0
                        <1> LinBytesPerScanLine23:     dw      5760
                        <1> BnkNumberOfPages23:        db      0
                        <1> LinNumberOfPages23:        db      0
                        <1> LinRedMaskSize23:  db      8
                        <1> LinRedFieldPosition23:     db      16
                        <1> LinGreenMaskSize23:        db      8
                        <1> LinGreenFieldPosition23:db 8
                        <1> LinBlueMaskSize23: db      8
                        <1> LinBlueFieldPosition23:    db      0
                        <1> LinRsvdMaskSize23: db      0
                        <1> LinRsvdFieldPosition23:    db      0
                        <1> MaxPixelClock23:   dd      0
                        <1>
                        <1>                    dw      0192h ; 1920x1080x32
                        <1> ModeAttributes24:  dw      MODE_ATTRIBUTES
                        <1> WinAAttributes24:  db      WINA_ATTRIBUTES
                        <1> WinBAttributes24:  db      0
                        <1> WinGranularity24:  dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinSize24:         dw      VBE_DISPI_BANK_SIZE_KB
                        <1> WinASegment24:             dw      VGAMEM_GRAPH
                        <1> WinBSegment24:             dw      0000h
                        <1> WinFuncPtr24:              dd      0
                        <1> BytesPerScanLine24:        dw      7680
                        <1> XResolution24:             dw      1920
                        <1> YResolution24:             dw      1080
                        <1> XCharSize24:               db      8
                        <1> YCharSize24:               db      16
                        <1> NumberOfPlanes24:  db      1
                        <1> BitsPerPixel24:            db      32
                        <1> NumberOfBanks24:   db      127
                        <1> MemoryModel24:             db      VBE_MEMORYMODEL_DIRECT_COLOR
                        <1> BankSize24:        db      0
                        <1> NumberOfImagePages24:      db      1
                        <1> Reserved_page24:   db      0
                        <1> RedMaskSize24:             db      8
                        <1> RedFieldPosition24:        db      16
                        <1> GreenMaskSize24:   db      8
                        <1> GreenFieldPosition24:      db      8
                        <1> BlueMaskSize24:            db      8
                        <1> BlueFieldPosition24:       db      0
                        <1> RsvdMaskSize24:            db      8
                        <1> RsvdFieldPosition24:       db      24
                        <1> DirectColorModeInfo24:     db      VBE_DIRECTCOLOR_RESERVED_BITS_AVAILABLE
                        <1> PhysBasePtr24:             dd      VBE_DISPI_LFB_PHYSICAL_ADDRESS
                        <1> OffScreenMemOffset24:      dd      0
                        <1> OffScreenMemSize24:        dw      0
                        <1> LinBytesPerScanLine24:     dw      7680
                        <1> BnkNumberOfPages24:        db      0
                        <1> LinNumberOfPages24:        db      0
                        <1> LinRedMaskSize24:  db      8
                        <1> LinRedFieldPosition24:     db      16
                        <1> LinGreenMaskSize24:        db      8
                        <1> LinGreenFieldPosition24:db 8
                        <1> LinBlueMaskSize24: db      8
                        <1> LinBlueFieldPosition24:    db      0
                        <1> LinRsvdMaskSize24: db      8
                        <1> LinRsvdFieldPosition24:    db      24
                        <1> MaxPixelClock24:   dd      0
                        <1>
                        <1> VBE_VESA_MODE_END_OF_LIST: dw      0
                        <1>
                        <1> %endif
                        <1>
                        <1> ; 24/11/2020
                        <1>
```

```
                                <1> direct_color_fields:
                                <1>     ; 24/11/2020
                                <1>
                                <1> ; (vbetables-gen.c)
                                <1> ; // Direct Color fields
                                <1> ; (required for direct/6 and YUV/7 memory models)
                                <1> ; switch(pm->depth) {
                                <1>
                                <1> ;case 8:
00006C7A 00                     <1> r_size_8:  db 0
00006C7B 00                     <1> r_pos_8:   db 0
00006C7C 00                     <1> g_size_8:  db 0
00006C7D 00                     <1> g_pos_8:   db 0
00006C7E 00                     <1> b_size_8:  db 0
00006C7F 00                     <1> b_pos_8:   db 0
00006C80 00                     <1> a_size_8:  db 0
00006C81 00                     <1> a_pos_8:   db 0
                                <1>
                                <1> ;case 16:
00006C82 05                     <1> r_size_16:  db 5
00006C83 0B                     <1> r_pos_16:   db 11
00006C84 06                     <1> g_size_16:  db 6
00006C85 05                     <1> g_pos_16:   db 5
00006C86 05                     <1> b_size_16:  db 5
00006C87 00                     <1> b_pos_16:   db 0
00006C88 00                     <1> a_size_16:  db 0
00006C89 00                     <1> a_pos_16:   db 0
                                <1>
                                <1> ;case 24:
00006C8A 08                     <1> r_size_24:  db 8
00006C8B 10                     <1> r_pos_24:   db 16
00006C8C 08                     <1> g_size_24:  db 8
00006C8D 08                     <1> g_pos_24:   db 8
00006C8E 08                     <1> b_size_24:  db 8
00006C8F 00                     <1> b_pos_24:   db 0
00006C90 00                     <1> a_size_24:  db 0
00006C91 00                     <1> a_pos_24:   db 0
                                <1>
                                <1> ;case 32:
00006C92 08                     <1> r_size_32:  db 8
00006C93 10                     <1> r_pos_32:   db 16
00006C94 08                     <1> g_size_32:  db 8
00006C95 08                     <1> g_pos_32:   db 8
00006C96 08                     <1> b_size_32:  db 8
00006C97 00                     <1> b_pos_32:   db 0
00006C98 08                     <1> a_size_32:  db 8
00006C99 18                     <1> a_pos_32:   db 24
                                    ;%include 'diskdata.s'    ; DISK (BIOS) DATA (initialized)
                                    ;;;

                                    Align 2

                                    %include 'sysdefs.s' ; 24/01/2015
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.5) - SYSTEM DEFINITIONS : sysdefs.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 23/07/2022  (Previous: 31/12/2017)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.11 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; sysdefs.inc (14/11/2015)
                                <1> ; ****************************************************************************
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - SYSDEFS.INC
                                <1> ; Last Modification: 14/11/2015
                                <1> ;
                                <1> ; ///////// RETRO UNIX 386 V1 SYSTEM DEFINITIONS ///////////////
                                <1> ; (Modified from
                                <1> ;   Retro UNIX 8086 v1 system definitions in 'UNIX.ASM', 01/09/2014)
                                <1> ; ((UNIX.ASM (RETRO UNIX 8086 V1 Kernel), 11/03/2013 - 01/09/2014))
                                <1> ;   UNIX.ASM (MASM 6.11) --> SYSDEFS.INC (NASM 2.11)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ;
                                <1> ; Derived from UNIX Operating System (v1.0 for PDP-11)
                                <1> ; (Original) Source Code by Ken Thompson (1971-1972)
                                <1> ; <Bell Laboratories (17/3/1972)>
                                <1> ; <Preliminary Release of UNIX Implementation Document>
                                <1> ;
                                <1> ; ****************************************************************************
                                <1>
                                <1> nproc       equ     16  ; number of processes
                                <1> ;nfiles equ 50
                                <1> nttyequ     8   ; 8+1 -> 8 (10/05/2013)
                                <1> nbufequ     4   ; 6 ;; 21/08/2015 - 'namei' buffer problem when nbuf > 4
                                <1>             ; NOTE: If fd0 super block buffer addres is beyond of the 1st
                                <1>             ; 32K, DMA r/w routine or someting else causes a jump to
                                <1>             ; kernel panic routine (in 'alloc' routine, in u5.s)
                                <1>             ; because of invalid buffer content (r/w error).
                                <1>             ; When all buffers are set before the end of the 1st 32k,
                                <1>             ; there is no problem!? (14/11/2015)
                                <1>
                                <1> ;csgmnt     equ     2000h   ; 26/05/2013 (segment of process 1)
                                <1> ;core       equ     0           ; 19/04/2013
                                <1> ;ecore      equ     32768 - 64  ; 04/06/2013 (24/05/2013)
                                <1>     ; (if total size of argument list and arguments is 128 bytes)
                                <1>     ; maximum executable file size = 32768-(64+40+128-6) = 32530 bytes
                                <1>     ; maximum stack size = 40 bytes (+6 bytes for 'IRET' at 32570)
                                <1>     ; initial value of user's stack pointer = 32768-64-128-2 = 32574
                                <1>     ;       (sp=32768-args_space-2 at the beginning of execution)
                                <1>     ; argument list offset = 32768-64-128 = 32576 (if it is 128 bytes)
                                <1>     ; 'u' structure offset (for the '/core' dump file) = 32704
                                <1>     ; '/core' dump file size = 32768 bytes
                                <1>
                                <1> ; 08/03/2014
                                <1> ;sdsegmnt equ       6C0h  ; 256*16 bytes (swap data segment size for 16 processes)
                                <1> ; 19/04/2013 Retro UNIX 8086 v1 feaure only !
                                <1> ;;sdsegmnt equ      740h  ; swap data segment (for user structures and registers)
                                <1>
                                <1> ; 30/08/2013
                                <1> time_count equ 4 ; 10 --> 4 01/02/2014
                                <1>
                                <1> ; 05/02/2014
                                <1> ; process status
                                <1> ;SFREE      equ 0
                                <1> ;SRUN       equ 1
                                <1> ;SWAIT      equ 2
                                <1> ;SZOMB      equ 3
                                <1> ;SSLEEP     equ 4 ; Retro UNIX 8086 V1 extension (for sleep and wakeup)
                                <1>
                                <1> ; 09/03/2015
                                <1> userdata equ 80000h ; user structure data address for current user ; temporary
```

```
<1> swap_queue equ 90000h - 2000h ; swap queue address ; temporary
<1> swap_alloc_table equ 0D0000h  ;  swap allocation table address ; temporary
<1>
<1> ; 17/09/2015
<1> ESPACE equ 48 ; [u.usp] (at 'sysent') - [u.sp] value for error return
<1>
<1> ; 31/12/2017
<1> ; 19/02/2017
<1> ; 15/10/2016
<1> ; 20/05/2016
<1> ; 19/05/2016
<1> ; 18/05/2016
<1> ; 29/04/2016
<1> ; TRDOS 386 (TRDOS v2.0) system calls - temporary List
<1> ; 14/07/2013 - 21/09/2015 (Retro UNIX 8086 & 386 system calls)
<1> _ver        equ 0 ; Get TRDOS version (v2.0)
<1> _exit       equ 1
<1> _fork       equ 2
<1> _read       equ 3
<1> _write      equ 4
<1> _open       equ 5
<1> _close      equ 6
<1> _wait       equ 7
<1> _creat      equ 8
<1> _rename     equ 9  ; TRDOS 386, Rename File (31/12/2017)
<1> _delete     equ 10 ; TRDOS 386, Delete File (29/12/2017)
<1> _exec       equ 11
<1> _chdir      equ 12
<1> _time       equ 13 ; TRDOS 386, Get Sys Date&Time (30/12/2017)
<1> _mkdir      equ 14
<1> _chmod      equ 15 ; TRDOS 386, Change Attributes (30/12/2017)
<1> _rmdir      equ 16 ; TRDOS 386, Remove Directory (29/12/2017)
<1> _break      equ 17
<1> _drive      equ 18 ; TRDOS 386, Get/Set Current Drv (30/12/2017)
<1> _seek       equ 19
<1> _tell       equ 20
<1> _memequ 21 ; TRDOS 386, Get Total&Free Mem (31/12/2017)
<1> _prompt     equ 22 ; TRDOS 386, Change Cmd Prompt (31/12/2017)
<1> _path       equ 23 ; TRDOS 386, Get/Set Run Path (31/12/2017)
<1> _envequ 24 ; TRDOS 386, Get/Set Env Vars (31/12/2017)
<1> _stime      equ 25 ; TRDOS 386, Set Sys Date&Time (30/12/2017)
<1> _quit       equ 26
<1> _intr       equ 27
<1> _direqu 28 ; TRDOS 386, Get Curr Drive&Dir (30/12/2017)
<1> _emt        equ 29
<1> _ldrvt      equ 30 ; TRDOS 386, Get Logical DOS DDT (30/12/2017)
<1> _video  equ 31 ; TRDOS 386 Video Functions (16/05/2016)
<1> _audio      equ 32 ; TRDOS 386 Video Functions (16/05/2016)
<1> _timer      equ 33 ; TRDOS 386 Timer Functions (18/05/2016)
<1> _sleep      equ 34 ; Retro UNIX 8086 v1 feature only !
<1> _msgequ 35 ; Retro UNIX 386 v1 feature only !
<1> _geterr     equ 36 ; Retro UNIX 386 v1 feature only !
<1> _fpsave equ 37 ; TRDOS 386 FPU state option (28/02/2017)
<1> _pri        equ 38 ; change priority - TRDOS 386 (20/05/2016)
<1> _rele       equ 39 ; TRDOS 386 (19/05/2016)
<1> _fffequ 40 ; Find First File - TRDOS 386 (15/10/2016)
<1> _fnfequ 41 ; Find Next File - TRDOS 386 (15/10/2016)
<1> _alloc      equ 42 ; Allocate memory - TRDOS 386 (19/02/2017)
<1>             ; TRDOS 386 (19/02/2017) DMA buff fuctions
<1> _dalloc equ 43 ; Deallocate mem - TRDOS 386 (19/02/2017)
<1> _calbac equ 44 ; Set IRQ callback - TRDOS 386 (20/02/2017)
<1> _dmaequ 45 ; DMA service - TRDOS 386 (20/08/2017)
<1>
<1> %macro sys 1-4
<1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
<1>     ; 03/09/2015
<1>     ; 13/04/2015
<1>     ; Retro UNIX 386 v1 system call.
<1>     %if %0 >= 2
<1>         mov ebx, %2
<1>         %if %0 >= 3
<1>             mov ecx, %3
<1>             %if %0 = 4
<1>                mov edx, %4
<1>             %endif
<1>         %endif
<1>     %endif
<1>     mov eax, %1
<1>     ;int 30h
<1>     int 40h ; TRDOS 386 (TRDOS v2.0)
<1> %endmacro
<1>
<1> ; TRDOS 386 system calls, interrupt number
<1> ; 25/12/2016
<1> SYSCALL_INT_NUM   equ '40' ; '40h'
<1>
<1> ; 13/05/2015 - ERROR CODES
<1> ERR_FILE_NOT_OPEN  equ 10 ; 'file not open !' error
<1> ERR_FILE_ACCESS    equ 11 ; 'permission denied !' error
<1> ; 14/05/2015
<1> ERR_DIR_ACCESS     equ 11 ; 'permission denied !' error
<1> ERR_FILE_NOT_FOUND equ 12 ; 'file not found !' error
<1> ERR_TOO_MANY_FILES equ 13 ; 'too many open files !' error
<1> ERR_DIR_EXISTS     equ 14 ; 'directory already exists !' error
<1> ; 16/05/2015
<1> ERR_DRV_NOT_RDY    equ 15 ; 'drive not ready !' error
<1> ; 18/05/2015
<1> ERR_DEV_NOT_RDY    equ 15 ; 'device not ready !' error
<1> ERR_DEV_ACCESS     equ 11 ; 'permission denied !' error
<1> ERR_DEV_NOT_OPEN   equ 10 ; 'device not open !' error
<1> ; 07/06/2015
<1> ERR_FILE_EOF          equ 16 ; 'end of file !' error
<1> ERR_DEV_VOL_SIZE   equ 16 ; 'out of volume !' error
<1> ; 09/06/2015
<1> ERR_DRV_READ          equ 17 ; 'disk read error !'
<1> ERR_DRV_WRITE         equ 18 ; 'disk write error !'
<1> ; 16/06/2015
<1> ERR_NOT_DIR    equ 19 ; 'not a (valid) directory !' error
<1> ERR_FILE_SIZE         equ 20 ; 'file size error !'
<1> ; 22/06/2015
<1> ERR_NOT_SUPERUSER  equ 11 ; 'permission denied !' error
<1> ERR_NOT_OWNER      equ 11 ; 'permission denied !' error
<1> ERR_NOT_FILE       equ 11 ; 'permission denied !' error
<1> ; 23/06/2015
<1> ERR_FILE_EXISTS    equ 14 ; 'file already exists !' error
<1> ERR_DRV_NOT_SAME   equ 21 ; 'not same drive !' error
<1> ERR_DIR_NOT_FOUND  equ 12 ; 'directory not found !' error
<1> ERR_NOT_EXECUTABLE equ 22 ; 'not executable file !' error
<1> ; 27/06/2015
<1> ERR_INV_PARAMETER  equ 23 ; 'invalid parameter !' error
<1> ERR_INV_DEV_NAME   equ 24 ; 'invalid device name !' error
<1> ; 29/06/2015
<1> ERR_TIME_OUT          equ 25 ; 'time out !' error
<1> ERR_DEV_NOT_RESP   equ 25 ; 'device not responding !' error
```

```
                                <1> ; 10/10/2016
                                <1> ERR_INV_FILE_NAME  equ 26 ; 'invalid file name !' error
                                <1> ERR_INV_FLAGS         equ 23 ; 'invalid flags !' error
                                <1> ; For code compatibility with previous version of TRDOS (2011)
                                <1> ; (Temporary error codes for current TRDOS 386 -2016- version)
                                <1> ERR_NO_MORE_FILES  equ 12 ; 'no more files !' error
                                <1> ERR_PATH_NOT_FOUND equ  3 ; 'path not found !' error
                                <1>                       ; 'dir not found !' ; TRDOS 8086
                                <1> ERR_NOT_FOUND         equ  2 ; 'file not found !' ; TRDOS 8086
                                <1> ERR_DISK_SPACE        equ 39 ; 'out of volume !' TRDOS 8086
                                <1>                       ; 'insufficient disk space !' ; 27h
                                <1> ERR_DISK_WRITE        equ 30 ; 'disk write protected !' ; 16/10/2016
                                <1> ERR_ACCESS_DENIED  equ  5 ; 'access denied !' ; TRDOS 8086
                                <1> ; 28/02/2017
                                <1> ERR_PERM_DENIED       equ 11 ; 'permission denied !' error
                                <1> ; 18/05/2016
                                <1> ERR_MISC       equ 27 ; miscellaneous/other errors
                                <1> ; 15/10/2016
                                <1> ; TRDOS 8086 -> TRDOS 386 (0Bh -> 28)
                                <1> ERR_INV_FORMAT        equ 28 ; 'invalid format !' error
                                <1> ; TRDOS 8086 -> TRDOS 386 (0Dh -> 29)
                                <1> ERR_INV_DATA          equ 29 ; 'invalid data !' error
                                <1> ; TRDOS 8086 -> TRDOS 386 (0Eh -> 20)
                                <1> ERR_ZERO_LENGTH       equ 20  ; 'zero length !' error
                                <1> ; TRDOS 8086 -> TRDOS 386 (15h -> 17, 1Dh -> 18, 1Eh -> 17)
                                <1> ERR_DRV_NR_READ       equ 17 ; 'drive not ready or read error !'
                                <1> ERR_DRV_NR_WRITE   equ 18 ; 'drive not ready or write error !'
                                <1> ; 15/10/2016
                                <1> ERR_INV_PATH_NAME  equ 19 ; 'bad path name !' error
                                <1> ERR_BAD_CMD_ARG       equ  1 ; 'bad command argument !' ; TRDOS 8086
                                <1> ERR_INV_FNUMBER       equ  1 ; 'invalid function number !' ; TRDOS 8086
                                <1> ERR_BIG_FILE          equ  8 ; 'big file & out of memory ! ; TRDOS 8086
                                <1> ERR_BIG_DATA          equ  8 ; 'big data & out of memory ! ; TRDOS 8086
                                <1> ERR_CLUSTER    equ 35 ; 'cluster not available !' ; TRDOS 8086
                                <1> ERR_OUT_OF_MEMORY  equ  4 ; 'out of memory !'
                                <1>                       ; 'insufficient memory !'
                                <1> ERR_P_VIOLATION       equ 6 ; 'protection violation !'
                                <1> ERR_PAGE_FAULT        equ 224 ;'page fault !' ;0E0h
                                <1> ERR_SWP_DISK_READ             equ 40
                                <1> ERR_SWP_DISK_NOT_PRESENT   equ 41
                                <1> ERR_SWP_SECTOR_NOT_PRESENT equ 42
                                <1> ERR_SWP_NO_FREE_SPACE      equ 43
                                <1> ERR_SWP_DISK_WRITE         equ 44
                                <1> ERR_SWP_NO_PAGE_TO_SWAP    equ 45
                                <1> ; 10/04/2017
                                <1> ERR_BUFFER     equ 46  ; 'buffer error !'
                                <1> ; 28/08/2017 (20/08/2017)
                                <1> ERR_DMA               equ -1  ; DMA buffer (allocation/misc.) error!
                                <1>
                                <1> ; 26/08/2015
                                <1> ; 24/07/2015
                                <1> ; 24/06/2015
                                <1> MAX_ARG_LEN    equ 256 ; max. length of sys exec arguments
                                <1> ; 01/07/2015
                                <1> MAX_MSG_LEN    equ 255 ; max. msg length for 'sysmsg'
                                <1> ;
                                <1> ; 06/10/2016
                                <1> ;OPENFILES     equ 10  ; max. number of open files (system)
                                <1> ; 23/07/2022
                                <1> OPENFILES      equ 32  ; max. number of open files (system)
                                <1> ; 07/10/2016
                                <1> ;NUMOFDEVICES         equ 20  ; max. num of available devices (sys)
                                    %include 'trdosk0.s' ; 04/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.0) - DEFINITIONS : trdosk0.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 29/02/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 04/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.11 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ****************************************************************************
                                <1> ; TRDOS2.ASM (c) 2004-2011 Erdogan TAN [ 17/01/2004 ] Last Update: 09/11/2011
                                <1> ;
                                <1> ; Masterboot / Partition Table at Beginning+1BEh
                                <1> ptBootable       equ 0
                                <1> ptBeginHead      equ 1
                                <1> ptBeginSector    equ 2
                                <1> ptBeginCylinder  equ 3
                                <1> ptFileSystemID   equ 4
                                <1> ptEndHead        equ 5
                                <1> ptEndSector      equ 6
                                <1> ptEndCylinder    equ 7
                                <1> ptStartSector    equ 8
                                <1> ptSectors        equ 12
                                <1>
                                <1> ; Boot Sector Parameters at 7C00h
                                <1> DataArea1     equ -4
                                <1> DataArea2     equ -2
                                <1> BootStart     equ 0h
                                <1> OemName       equ 03h
                                <1> BytesPerSec   equ 0Bh
                                <1> SecPerClust   equ 0Dh
                                <1> ResSectors    equ 0Eh
                                <1> FATs          equ 10h
                                <1> RootDirEnts   equ 11h
                                <1> Sectors       equ 13h
                                <1> Media         equ 15h
                                <1> FATSecs       equ 16h
                                <1> SecPerTrack   equ 18h
                                <1> Heads         equ 1Ah
                                <1> Hidden1       equ 1Ch
                                <1> Hidden2       equ 1Eh
                                <1> HugeSec1      equ 20h
                                <1> HugeSec2      equ 22h
                                <1> DriveNumber   equ 24h
                                <1> Reserved1     equ 25h
                                <1> bootsignature equ 26h
                                <1> VolumeID      equ 27h
                                <1> VolumeLabel   equ 2Bh
                                <1> FileSysType   equ 36h
                                <1> Reserved2     equ 3Eh             ; Starting cluster of P2000
                                <1>
                                <1> ; FAT32 BPB Structure
                                <1> FAT32_FAT_Size equ 36
                                <1> FAT32_RootFClust equ 44
                                <1> FAT32_FSInfoSec equ 48
                                <1> FAT32_DrvNum equ 64
                                <1> FAT32_BootSig equ 66
                                <1> FAT32_VolID equ 67
                                <1> FAT32_VolLab equ 71
```

```
<1> FAT32_FilSysType equ 82
<1>
<1> ; BIOS Disk Parameters
<1> DPDiskNumber  equ 0h
<1> DPDType       equ 1h
<1> DPReturn      equ 2h
<1> DPHeads       equ 3h
<1> DPCylinders   equ 4h
<1> DPSecPerTrack equ 6h
<1> DPDisks       equ 7h
<1> DPTableOff    equ 8h
<1> DPTableSeg    equ 0Ah
<1> DPNumOfSecs   equ 0Ch
<1>
<1> ; BIOS INT 13h Extensions (LBA extensions)
<1> ; Just After DP Data (DPDiskNumber+)
<1> DAP_PacketSize equ 10h  ; If extensions present, this byte will be >=10h
<1> DAP_Reserved1 equ 11h   ; Reserved Byte
<1> DAP_NumOfBlocks equ 12h ; Value of this byte must be 0 to 127
<1> DAP_Reserved2 equ 13h   ; Reserved Byte
<1> DAP_Destination equ 14h ; Address of Transfer Buffer as SEGMENT:OFFSET
<1> DAP_LBA_Address equ 18h ; LBA=(C1*H0+H1)*S0+S1-1
<1>                         ; C1= Selected Cylinder Number
<1>                         ; H0= Number Of Heads (Maximum Head Number + 1)
<1>                         ; H1= Selected Head Number
<1>                         ; S0= Maximum Sector Number
<1>                         ; S1= Selected Sector Number
<1>                         ; QUAD WORD
<1> ; DAP_Flat_Destination equ 20h ; 64 bit address, if value in 4h is FFFF:FFFFh
<1>                               ; QUAD WORD (Also, value in 0h must be 18h)
<1>                               ; TR-DOS will not use 64 bit Flat Address
<1>
<1> ; INT 13h Function 48h "Get Enhanced Disk Drive Parameters"
<1> ; Just After DP Data (DPDiskNumber+)
<1> GetDParams_48h equ 20h ; Word. Data Length, must be 26 (1Ah) for short data.
<1> GDP_48h_InfoFlag equ 22h ; Word
<1> ; Bit 1 = 1 -> The geometry returned in bytes 4-15 is valid.
<1> GDP_48h_NumOfPCyls equ 24h ; Double Word. Number physical cylinders.
<1> GDP_48h_NumOfPHeads equ 28h ; Double Word. Number of physical heads.
<1> GDP_48h_NumOfPSpT equ 2Ch ; Double word. Num of physical sectors per track.
<1> GDP_48h_LBA_Sectors equ 30h ; 8 bytes. Number of physical/LBA sectors.
<1> GDP_48h_BytesPerSec equ 38h ; Word. Number of bytes in a sector.
<1>
<1> ; TR-DOS Standalone Program Extensions to the DiskParams Block
<1> ; Just After DP Data (DPDiskNumber+)
<1> TRDP_CurrentSector equ 3Ah  ; DX:AX (LBA)
<1> TRDP_SectorCount equ 3Eh    ; CX (or Counter)
<1>
<1>
<1> ; DOS Logical Disks
<1> LD_Name equ 0
<1> LD_DiskType equ 1
<1> LD_PhyDrvNo equ 2
<1> LD_FATType equ 3
<1> LD_FSType equ 4
<1> LD_LBAYes equ 5
<1> LD_BPB equ 6
<1> LD_FATBegin equ 96
<1> LD_ROOTBegin equ 100
<1> LD_DATABegin equ 104
<1> LD_StartSector equ 108
<1> LD_TotalSectors equ 112
<1> LD_FreeSectors equ 116
<1> LD_Clusters equ 120
<1> LD_PartitionEntry equ 124
<1> LD_DParamEntry equ 125
<1> LD_MediaChanged equ 126
<1> LD_CDirLevel equ 127
<1> LD_CurrentDirectory equ 128
<1>
<1> ; Singlix FS Extensions to DOS Logical Disks
<1> ; 03/01/2010 (LD_BPB compatibility for CHS r/w)
<1>
<1> LD_FS_Name equ 0
<1> LD_FS_DiskType equ 1
<1> LD_FS_PhyDrvNo equ 2
<1> LD_FS_FATType equ 3
<1> LD_FS_FSType equ 4
<1> LD_FS_LBAYes equ 5
<1> LD_FS_BPB equ 6
<1> LD_FS_MediaAttrib equ 6
<1> LD_FS_VersionMajor equ 7
<1> LD_FS_RootDirD equ 8
<1> LD_FS_MATLocation equ 12
<1> LD_FS_Reserved1 equ 16 ;1 reserved byte
<1> LD_FS_BytesPerSec equ 17 ; LD_BPB + 0Bh
<1> LD_FS_Reserved2 equ 19 ;2 reserved byte
<1> LD_FS_DATLocation equ 20
<1> LD_FS_DATSectors equ 24
<1> LD_FS_Reserved3 equ 28 ;3 reserved word
<1> LD_FS_SecPerTrack equ 30 ; LD_BPB + 18h
<1> LD_FS_NumHeads equ 32    ; LD_BPB + 1Ah
<1> LD_FS_UnDelDirD equ 34
<1> LD_FS_Reserved4 equ 38 ;4 reserved word
<1> LD_FS_VolumeSerial equ 40
<1> LD_FS_VolumeName equ 44
<1> LD_FS_BeginSector equ 108
<1> LD_FS_VolumeSize equ 112
<1> LD_FS_FreeSectors equ 116
<1> LD_FS_FirstFreeSector equ 120
<1> LD_FS_PartitionEntry equ 124
<1> LD_FS_DParamEntry equ 125
<1> LD_FS_MediaChanged equ 126
<1> LD_FS_CDirLevel equ 127
<1> LD_FS_CDIR_Converted equ 128
<1>
<1> ; Valid FAT Types
<1> FS_FAT12 equ 1
<1> FS_FAT16_CHS equ 2
<1> FS_FAT32_CHS equ 3
<1> FS_FAT16_LBA equ 4
<1> FS_FAT32_LBA equ 5
<1>
<1> ; Cursor Location
<1> CCCpointer equ  0450h   ; BIOS data, current cursor column
<1> ; FAT Clusters EOC sign
<1> FAT12EOC equ 0FFFh
<1> FAT16EOC equ 0FFFFh
<1> ;FAT32EOC equ 0FFFFFFFh ; It is not direct usable for 8086 code
<1> ; BAD Cluster
<1> FAT12BADC equ 0FF7h
<1> FAT16BADC equ 0FFF7h
<1> ;FAT32BADC equ 0FFFFFF7h ; It is not direct usable for 8086 code
<1> ; MS-DOS FAT16 FS (Maximum Possible) Last Cluster Number= 0FFF6h
```

```
                                <1>
                                <1> ; TRFS
                                <1>
                                <1> bs_FS_JmpBoot equ 0 ; jmp short bsBootCode
                                <1>                     ; db 0EBh, db 3Fh, db 90h
                                <1> bs_FS_Identifier equ 3  ; db 'FS', db 0
                                <1> bs_FS_BytesPerSec equ 6 ; dw 512
                                <1> bs_FS_MediaAttrib equ 8 ; db 3
                                <1> bs_FS_PartitionID equ 9 ; db 0A1h
                                <1> bs_FS_VersionMaj equ 10 ; db 01h
                                <1> bs_FS_VersionMin equ 11 ; db 0
                                <1> bs_FS_BeginSector equ 12   ; dd 0
                                <1> bs_FS_VolumeSize equ 16 ; dd 2880
                                <1> bs_FS_StartupFD equ 20 ; dd 0
                                <1> bs_FS_MATLocation equ 24 ; dd 1
                                <1> bs_FS_RootDirD equ 28 ; dd 8
                                <1> bs_FS_SystemConfFD equ 32 ; dd 0
                                <1> bs_FS_SwapFD equ 36 ; dd 0
                                <1> bs_FS_UnDelDirD equ 40 ; dd 0
                                <1> bs_FS_DriveNumber equ 44 ; db 0
                                <1> bs_FS_LBA_Ready equ 45 ; db 0
                                <1> bs_FS_MagicWord equ 46
                                <1> bs_FS_SecPerTrack equ 46 ; db 0A1h
                                <1> bs_FS_Heads equ 47 ; db 01h
                                <1> bs_FS_OperationSys equ 48 ; db "TR-SINGLIX v1.0b"
                                <1> bs_FS_Terminator equ 64 ; db 0
                                <1> bs_FS_BootCode equ 65
                                <1>
                                <1> FS_MAT_DATLocation equ 12
                                <1> FS_MAT_DATScount equ 16
                                <1> FS_MAT_FreeSectors equ 20
                                <1> FS_MAT_FirstFreeSector equ 24
                                <1> FS_RDT_VolumeSerialNo equ 28
                                <1> FS_RDT_VolumeName equ 64
                                <1>
                                <1> ; FAT12 + FAT16 + FAT32
                                <1> BS_JmpBoot equ 0
                                <1> BS_OEMName equ 3
                                <1> BPB_BytsPerSec equ 11
                                <1> BPB_SecPerClust equ 13
                                <1> BPB_RsvdSecCnt equ 14
                                <1> BPB_NumFATs equ 16
                                <1> BPB_RootEntCnt equ 17
                                <1> BPB_TotalSec16 equ 19
                                <1> BPB_Media equ 21
                                <1> BPB_FATSz16 equ 22
                                <1> BPB_SecPerTrk equ 24
                                <1> BPB_NumHeads equ 26
                                <1> BPB_HiddSec equ 28
                                <1> BPB_TotalSec32 equ 32
                                <1>
                                <1> ; FAT12 and FAT16 only
                                <1> BS_DrvNum equ 36
                                <1> BS_Reserved1 equ 37
                                <1> BS_BootSig equ 38
                                <1> BS_VolID equ 39
                                <1> BS_VolLab equ 43
                                <1> BS_FilSysType equ 54 ; 8 bytes
                                <1> BS_BootCode equ 62
                                <1>
                                <1> ; FAT32 only
                                <1> BPB_FATSz32 equ 36 ; FAT32, 4 bytes
                                <1> BPB_ExtFlags equ 40 ; FAT32, 2 bytes
                                <1> BPB_FSVer equ 42 ; FAT32, 2 bytes
                                <1> BPB_RootClus equ 44 ; FAT32, 4 bytes
                                <1> BPB_FSInfo equ 48 ; FAT 32, 2 bytes
                                <1> BPB_BkBootSec equ 50 ; FAT32, 2 bytes
                                <1> BPB_Reserved equ 52 ; FAT32, 12 bytes
                                <1> BS_FAT32_DrvNum equ 64 ; FAT32, 1 byte
                                <1> BS_FAT32_Reserved1 equ 65 ; FAT32, 1 byte
                                <1> BS_FAT32_BootSig equ 66 ; FAT32, 1 byte
                                <1> BS_FAT32_VolID equ 67 ; FAT32, 4 bytes
                                <1> BS_FAT32_VolLab equ 71 ; FAT32, 11 bytes
                                <1> BS_FAT32_FilSysType equ 82 ; FAT32, 8 bytes
                                <1> BS_FAT32_BootCode equ 90
                                <1>
                                <1> ; 29/02/2016
                                <1> ;(FAT32 Free Cluster Count & First Free Cluster values)
                                <1> ;[BPB_Reserved] = Free Cluster Count (offset 52)
                                <1> ;[BPB_Reserved+4] = First Free Cluster (offset 56)
                                <1>
                                <1> BS_Validation equ 510
                                <1>
                                <1> ; 15/02/2016
                                <1> ; FILE.ASM - 09/10/2011
                                <1> ; Directory Entry Structure
                                <1> ; 29/10/2009 (According to Microsoft FAT32 File System Specification)
                                <1> DirEntry_Name equ 0
                                <1> DirEntry_Attr equ 11
                                <1> DirEntry_NTRes equ 12
                                <1> DirEntry_CrtTimeTenth equ 13
                                <1> DirEntry_CrtTime equ 14
                                <1> DirEntry_CrtDate equ 16
                                <1> DirEntry_LastAccDate equ 18
                                <1> DirEntry_FstClusHI equ 20
                                <1> DirEntry_WrtTime equ 22
                                <1> DirEntry_WrtDate equ 24
                                <1> DirEntry_FstClusLO equ 26
                                <1> DirEntry_FileSize equ 28
                                    %include 'trdosk1.s' ; 04/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.9) - SYS INIT : trdosk1.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 26/09/2024 (Previous: 25/07/2022)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 04/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ****************************************************************************
                                <1> ; TRDOS2.ASM (c) 2004-2011 Erdogan TAN [ 17/01/2004 ] Last Update: 09/11/2011
                                <1> ;
                                <1>
                                <1> sys_init:
                                <1>     ; 26/09/2024 (TRDOS v2.0.9)
                                <1>     ; 18/04/2021 (TRDOS 386 v2.0.4)
                                <1>     ; 20/01/2018  (v2.0.1)
                                <1>     ; 23/01/2017  (v2.0.0)
                                <1>     ; 07/05/2016
                                <1>     ; 02/05/2016
                                <1>     ; 24/04/2016
                                <1>     ; 14/04/2016
```

```
                            <1>     ; 13/04/2016
                            <1>     ; 30/03/2016
                            <1>     ; 24/01/2016
                            <1>     ; 06/01/2016
                            <1>     ; 04/01/2016 (TRDOS 386 v2.0 - Beginning)
                            <1>
                            <1>     ; 23/01/2017 - reset timer frequency (to 18.2Hz)
00006C9A B036               <1>     mov     al, 00110110b ; 36h
00006C9C E643               <1>     out     43h, al
00006C9E 31C0               <1>     xor     eax, eax  ; sub       al, al ; 0
00006CA0 E640               <1>     out     40h, al ; LB
00006CA2 E640               <1>     out     40h, al ; HB
                            <1>     ;
                            <1>     ; 30/03/2016
                            <1>     ; Clear Logical DOS Disk Description Tables Area
                            <1>     ;xor    eax, eax
00006CA4 BF00010900         <1>     mov     edi, Logical_DOSDisks
00006CA9 B980060000         <1>     mov     ecx, 6656/4 ; 26*256 = 6656 bytes
00006CAE F3AB               <1>     rep     stosd ; 1664 times 4 bytes
                            <1>
00006CB0 B83F3A2F00         <1>     mov     eax, '?:/'
00006CB5 A3[77770100]       <1>     mov     [Current_Dir_Drv], eax
                            <1>
                            <1>     ; Logical DRV INIT (only for hard disks)
00006CBA E8C3030000         <1>     call    ldrv_init  ; trdosk2.s
                            <1>
                            <1>     ; When floppy_drv_init call is disabled
                            <1>     ; media changed sign is needed
                            <1>     ; for proper drive initialization
                            <1>
00006CBF BE00010900         <1>     mov     esi, Logical_DOSDisks
00006CC4 B001               <1>     mov     al, 1 ; Initialization sign (invalid_fd_parameter)
00006CC6 83C67E             <1>     add     esi, LD_MediaChanged ; Media Change Status = 1 (init needed)
00006CC9 8806               <1>     mov     [esi], al ; A:
                            <1>     ; Temporary - 26/09/2024
                            <1>     ;;mov   dword [esi+LD_Clusters], -1 ; *
                            <1>     ;dec    dword [esi+LD_Clusters]
00006CCB 81C600010000       <1>     add     esi, 100h
00006CD1 8806               <1>     mov     [esi], al ; B:
                            <1>     ;;mov   dword [esi+LD_Clusters], -1 ; *
                            <1>     ;dec    dword [esi+LD_Clusters]
                            <1>
                            <1> _current_drive_bootdisk:
00006CD3 8A15[88660000]     <1>     mov     dl, [boot_drv] ; physical drive number
00006CD9 80FAFF             <1>     cmp     dl, 0FFh
00006CDC 740A               <1>     je      short _last_dos_diskno_check
                            <1> _boot_drive_check:
00006CDE 80FA80             <1>     cmp     dl, 80h
00006CE1 7218               <1>     jb      short _current_drive_a
00006CE3 80EA7E             <1>     sub     dl, 7Eh ; C = 2 , D = 3
00006CE6 EB13               <1>     jmp     short _current_drive_a
                            <1>
                            <1> _last_dos_diskno_check:
00006CE8 8A15[8D2E0100]     <1>     mov     dl, [Last_DOS_DiskNo]
00006CEE 80FA02             <1>     cmp     dl, 2
00006CF1 7706               <1>     ja      short _current_drive_c
00006CF3 7406               <1>     je      short _current_drive_a
00006CF5 30D2               <1>     xor     dl, dl ; A:
00006CF7 EB02               <1>     jmp     short _current_drive_a
                            <1>
                            <1> _current_drive_c:
00006CF9 B202               <1>     mov     dl, 2 ; C:
                            <1>
                            <1> _current_drive_a:
00006CFB 8815[89660000]     <1>     mov     [drv], dl
00006D01 BE[8F2E0100]       <1>         mov     esi, msg_CRLF_temp
00006D06 E8AF000000         <1>     call    print_msg
                            <1>
00006D0B 8A15[89660000]     <1>     mov     dl, [drv]
                            <1> _default_drive_c:
00006D11 E8DA090000         <1>     call    change_current_drive
00006D16 731C               <1>     jnc     short _start_mainprog
                            <1>
                            <1> _drv_not_ready_error:
00006D18 BE[53310100]       <1>     mov     esi, msgl_drv_not_ready
00006D1D E898000000         <1>     call    print_msg
                            <1>         ;jmp       _end_of_mainprog
                            <1>
                            <1>     ; 20/01/2018
00006D22 B202               <1>     mov     dl, 2
00006D24 3815[89660000]     <1>     cmp     [drv], dl
00006D2A 736C               <1>     jnb     short _end_of_mainprog
00006D2C 8815[89660000]     <1>     mov     [drv], dl
00006D32 EBDD               <1>     jmp     short _default_drive_c
                            <1>
                            <1> _start_mainprog:
                            <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                            <1>     ; 18/04/2021 (TRDOS 386 v2.0.4 - Beginning)
                            <1>     ; 07/01/2017
                            <1>     ; 07/05/2016
                            <1>     ; 02/05/2016
                            <1>     ; 24/04/2016 (TRDOS 386 v2)
                            <1>     ; Retro UNIX 386 v1, 'sys_init' (u0.s)
                            <1>     ; 23/06/2015
                            <1>
                            <1>     ; 02/05/2016
                            <1>     ; 24/04/2016
                            <1>     ;mov    ax, 1
                            <1>     ; 18/04/2021 - TRDOS 386 v2.0.4
00006D34 31C0               <1>     xor     eax, eax
00006D36 FEC0               <1>     inc     al  ; eax = 1
00006D38 A2[BD8E0100]       <1>     mov     [u.uno], al
00006D3D 66A3[548E0100]     <1>     mov     [mpid], ax
00006D43 66A3[408D0100]     <1>     mov     [p.pid], ax
00006D49 A2[A08D0100]       <1>     mov     [p.stat], al
00006D4E C605[B48E0100]04   <1>     mov     byte [u.quant], time_count  ; 07/01/2017
                            <1>     ;
00006D55 A1[B0760100]       <1>     mov     eax, [k_page_dir]
00006D5A A3[C48E0100]       <1>     mov     [u.pgdir], eax ; reset
                            <1>     ;
00006D5F E8C6EAFFFF         <1>     call    allocate_page
                            <1>     ;jc     panic
                            <1>     ; 25/07/2022
00006D64 7305               <1>     jnc     short _start_mainprog_1
00006D66 E9AE000000         <1>     jmp     panic
                            <1>
                            <1> _start_mainprog_1:
00006D6B A3[C08E0100]       <1>     mov     [u.upage], eax ; user structure page
00006D70 A3[B08D0100]       <1>     mov     [p.upage], eax
00006D75 E821EBFFFF         <1>     call    clear_page
                            <1>     ;
                            <1>     ; 24/08/2015
00006D7A FE0D[608E0100]     <1>     dec     byte [sysflg] ; FFh = ready for system call
                            <1>                           ; 0 = executing a system call
```

```
                                <1>     ; 13/04/2016
                                <1>     ; Clear Environment Variables Page/Area
00006D80 BF00300900             <1>     mov     edi, Env_Page ; 93000h
00006D85 B980000000             <1>     mov     ecx, Env_Page_Size / 4; 512/4  (4096/4)
00006D8A 31C0                   <1>     xor     eax, eax
00006D8C F3AB                   <1>     rep     stosd
                                <1>
                                <1>     ; 14/04/2016
00006D8E E880300000             <1>     call    mainprog_startup_configuration
                                <1>
00006D93 E8870A0000             <1>         call    dos_prompt
                                <1>
                                <1> _end_of_mainprog:
00006D98 BE[8F2E0100]           <1>         mov     esi, msg_CRLF_temp
00006D9D E818000000             <1>     call    print_msg
00006DA2 BE[952E0100]           <1>     mov     esi, mainprog_Version
00006DA7 E80E000000             <1>     call    print_msg
                                <1>     ; 24/01/2016
00006DAC 28E4                   <1>     sub     ah, ah
00006DAE E85FA1FFFF             <1>     call    int16h ; call getch
00006DB3 E9EDA5FFFF             <1>     jmp     cpu_reset
                                <1>
00006DB8 EBFE                   <1> infinitiveloop: jmp short infinitiveloop
                                <1>
                                <1> print_msg:
                                <1>     ; 13/05/2016
                                <1>     ; 04/01/2016
                                <1>     ; 01/07/2015
                                <1>     ; 13/03/2015 (Retro UNIX 386 v1)
                                <1>     ; 07/03/2014 (Retro UNIX 8086 v1)
                                <1>     ; (Modified registers: EAX, EBX, ECX, EDX, ESI, EDI)
                                <1>     ;
00006DBA 8A3D[DE760100]         <1>     mov     bh, [ACTIVE_PAGE] ; 04/01/2016 (ptty)
                                <1>     ;mov    bl, 07h ; Black background, light gray forecolor
                                <1>
00006DC0 AC                     <1>     lodsb
                                <1> pmsg1:
00006DC1 56                     <1>     push    esi
                                <1>     ;mov    bh, [ACTIVE_PAGE] ; 04/01/2016 (ptty)
00006DC2 B307                   <1>     mov     bl, 07h ; Black background, light gray forecolor
00006DC4 E816B5FFFF             <1>     call    _write_tty
00006DC9 5E                     <1>     pop     esi
00006DCA AC                     <1>     lodsb
00006DCB 20C0                   <1>     and     al, al
00006DCD 75F2                   <1>     jnz     short pmsg1
00006DCF C3                     <1>     retn
                                <1>
                                <1> clear_screen:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 06/12/2020
                                <1>     ; 03/12/2020 (TRDOS 386 v2.0.3)
                                <1>     ; 13/05/2016
                                <1>     ; 30/01/2016
                                <1>     ; 24/01/2016
                                <1>     ; 04/01/2016
00006DD0 0FB61D[DE760100]       <1>     movzx   ebx, byte [ACTIVE_PAGE] ; video page number (0 to 7)
                                <1>     ; 25/07/2022
                                <1>     ;sub    al, al
                                <1>     ; al = 0
00006DD7 8AA3[3F680000]         <1>     mov     ah, [ebx+vmode] ; default = 03h (80x25 text)
00006DDD 80FC04                 <1>     cmp     ah, 4
00006DE0 7205                   <1>     jb      short cls1
00006DE2 80FC07                 <1>     cmp     ah, 7
00006DE5 752B                   <1>     jne     short vga_clear
                                <1> cls1:
                                <1>     ;mov    bh, bl
                                <1>     ;mov    bl, 7
00006DE7 3A25[2E680000]         <1>     cmp     ah, [CRT_MODE] ; current video mode ?
00006DED 740D                   <1>     je      short cls2 ; yes (current video mode = 3)
                                <1>     ;;call  set_mode_3 ; set video mode to 3 (& clear screen)
                                <1>     ;;retn
                                <1>     ; 06/12/2020
                                <1>     ;cmp    byte [pmi32], 0
                                <1>     ; 25/07/2022
00006DEF 383D[249D0100]         <1>     cmp     [pmi32], bh ; 0
00006DF5 771B                   <1>     ja      short vga_clear
00006DF7 E9EDB4FFFF             <1>     jmp     set_mode_3
                                <1> cls2:
00006DFC 88DF                   <1>     mov     bh, bl ; video page (0 to 7)
00006DFE B307                   <1>     mov     bl, 07h ; attribute to be used on blanked line
                                <1>     ; 25/07/2022
                                <1>     ;sub    al, al ; 0 = entire window
                                <1>     ;xor    cx, cx
                                <1>     ; 25/07/2022
                                <1>     ; al = 0
00006E00 31C9                   <1>     xor     ecx, ecx
00006E02 66BA4F18               <1>     mov     dx, 184Fh
00006E06 E826B2FFFF             <1>     call    _scroll_up ; 24/01/2016
                                <1>     ;
                                <1>     ;mov    bh, [ACTIVE_PAGE] ; video page number (0 to 7)
                                <1>     ;xor    dx, dx
                                <1>     ; 25/07/2022
00006E0B 31D2                   <1>     xor     edx, edx
                                <1>     ;call   _set_cpos ; 24/01/2016
                                <1>     ;;retn
                                <1>     ; 03/12/2020
00006E0D E965B5FFFF             <1>     jmp     _set_cpos ; returns to the caller of this proc
                                <1> ;cls3:
                                <1> ;   retn
                                <1>
                                <1>     ; 06/12/2020
                                <1> vga_clear:
                                <1>     ; 03/12/2020
                                <1>     ; set mode by using _int10h
                                <1>     ; (also clears screen)
                                <1>     ;mov    al, ah
                                <1>     ;sub    ah, ah  ; set current video mode
                                <1>     ; 25/07/2022
00006E12 86E0                   <1>     xchg    ah, al
                                <1>     ; ah = 0
                                <1>     ; al = video mode
                                <1>     ;call   _int10h ; simulates int 10h in TRDOS 386 kernel
                                <1>     ;jmp    short cls3
00006E14 E901A9FFFF             <1>     jmp     _int10h ; returns to the caller of this proc
                                <1>
                                <1> panic:
                                <1>     ; 13/05/2016 (TRDOS 386 = TRDOS v2)
                                <1>     ; 13/03/2015 (Retro UNIX 386 v1)
                                <1>     ; 07/03/2014 (Retro UNIX 8086 v1)
00006E19 BE[3F390100]           <1>     mov     esi, panic_msg
00006E1E E897FFFFFF             <1>     call    print_msg
                                <1> key_to_reboot:
                                <1>         ; 24/01/2016
00006E23 28E4                   <1>         sub     ah, ah
```

```
00006E25 E8E8A0FFFF          <1>         call    int16h    ; call getch
                             <1>         ; wait for a character from the current tty
                             <1>     ;
00006E2A B00A                <1>     mov    al, 0Ah
00006E2C 8A3D[DE760100]      <1>     mov    bh, [ptty] ; [ACTIVE_PAGE]
00006E32 B307                <1>     mov    bl, 07h ; Black background,
                             <1>                    ; light gray forecolor
00006E34 E8A6B4FFFF          <1>     call   _write_tty
00006E39 E967A5FFFF          <1>     jmp    cpu_reset
                             <1>
                             <1> ctrlbrk:
                             <1>     ; 21/08/2024 (TRDOS 386 Kernel v2.0.9)
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 12/11/2015
                             <1>     ; 13/03/2015 (Retro UNIX 386 v1)
                             <1>     ; 06/12/2013 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; INT 1Bh (control+break) handler
                             <1>     ;
                             <1>             ; Retro Unix 8086 v1 feature only!
                             <1>             ;
                             <1>
                             <1>     ; 25/07/2022
00006E3E 52                  <1>     push   edx
00006E3F 31D2                <1>     xor    edx, edx
                             <1>
                             <1> ; 21/08/2024 - TRDOS 386 v2.0.9
                             <1> %if 0
                             <1>     ;cmp   word [u.intr], 0
                             <1>     cmp    [u.intr], dx ; 0
                             <1>     jna    short cbrk4
                             <1> cbrk0:
                             <1>     ; 12/11/2015
                             <1>     ; 06/12/2013
                             <1>     ;cmp   word [u.quit], 0
                             <1>     cmp    [u.quit], dx ; 0 ; 25/07/2022
                             <1>     jz     short cbrk4
                             <1> %endif
                             <1>     ; 20/09/2013
                             <1>     ;push  ax
00006E41 50                  <1>     push   eax ; 25/07/2022
00006E42 A0[DE760100]        <1>     mov    al, [ptty]
                             <1>
                             <1> ; 21/08/2024 - TRDOS 386 v2.0.9
                             <1> %if 1
00006E47 3A05[A28E0100]      <1>     cmp    al, [u.ttyn]
00006E4D 7509                <1>     jne    short cbrk3
                             <1> %else
                             <1>     ; 12/11/2015
                             <1>     ;
                             <1>     ; ctrl+break (EOT, CTRL+D) from serial port
                             <1>     ; or ctrl+break from console (pseudo) tty
                             <1>     ; (!redirection!)
                             <1>
                             <1>     cmp    al, 8 ; serial port tty nums > 7
                             <1>         jb      short cbrk1 ; console (pseudo) tty
                             <1>
                             <1>     ; Serial port interrupt handler sets [ptty]
                             <1>     ; to the port's tty number (as temporary).
                             <1>     ;
                             <1>     ; If active process is using a stdin or
                             <1>     ; stdout redirection (by the shell),
                             <1>         ; console tty keyboard must be available
                             <1>     ; to terminate running process,
                             <1>     ; in order to prevent a deadlock.
                             <1>
                             <1>     ; 25/07/2022
                             <1>     ;push  edx
                             <1>     ;movzx edx, byte [u.uno]
                             <1>     mov    dl, [u.uno]
                             <1>     cmp     al, [edx+p.ttyc-1] ; console tty (rw)
                             <1>     ;pop   edx
                             <1>     je     short cbrk2
                             <1> cbrk1:
                             <1>     inc    al  ; [u.ttyp] : 1 based tty number
                             <1>     ; 06/12/2013
                             <1>     cmp    al, [u.ttyp]   ; recent open tty (r)
                             <1>     je     short cbrk2
                             <1>         cmp     al, [u.ttyp+1] ; recent open tty (w)
                             <1>     jne    short cbrk3
                             <1> cbrk2:
                             <1>     ;; 06/12/2013
                             <1>     ;mov   ax, [u.quit]
                             <1>     ;and   ax, ax
                             <1>     ;jz    short cbrk3
                             <1> %endif
                             <1>     ;xor   ax, ax ; 0
                             <1>     ;dec   ax
                             <1>     ; 0FFFFh = 'ctrl+brk' keystroke
                             <1>     ; 25/07/2022
                             <1>     ;xor   eax, eax ; 0
                             <1>     ;dec   eax ; -1 ; 0FFFFFFFFh
                             <1>     ;mov   [u.quit], ax
                             <1>     ; 21/08/2024
                             <1>     ; set CTRL+BREAK flag (even if it is not activated)
                             <1>     ; (u.intr is it's activation flag, 0 = disabled))
00006E4F 66C705[BA8E0100]FF- <1>     mov    word [u.quit], -1 ; 0FFFFh
00006E57 FF                  <1>
                             <1> cbrk3:
                             <1>     ;pop   ax
00006E58 58                  <1>     pop    eax ; 25/07/2022
                             <1> cbrk4:
                             <1>     ; 25/07/2022
00006E59 5A                  <1>     pop    edx
00006E5A C3                  <1>     retn
                             <1>
                             <1> ; 31/12/2017
                             <1> ; TRDOS 386 - 30/12/2017
                             <1> %define get_rtc_date RTC_40
                             <1> %define get_rtc_time RTC_20
                             <1> %define     set_rtc_date RTC_50
                             <1> %define set_rtc_time RTC_30
                             <1> get_rtc_date_time:
                             <1> ; Retro UNIX 8086 v1 - UNIX.ASM (01/09/2014)
                             <1> ;epoch:
                             <1>     ; 18/04/2021 (TRDOS 386 v2.0.3)
                             <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 15/03/2015 (Retro UNIX 386 v1 - 32 bit version)
                             <1>     ; 09/04/2013 (Retro UNIX 8086 v1 - UNIX.ASM)
                             <1>     ; 'epoch' procedure prototype:
                             <1>     ;                    UNIXCOPY.ASM, 10/03/2013
                             <1>     ; 14/11/2012
                             <1>     ; unixboot.asm (boot file configuration)
                             <1>     ; version of "epoch" procedure in "unixproc.asm"
```

```
                                <1>     ; 21/7/2012
                                <1>     ; 15/7/2012
                                <1>     ; 14/7/2012
                                <1>     ; Erdogan Tan - RETRO UNIX v0.1
                                <1>     ; compute current date and time as UNIX Epoch/Time
                                <1>     ; UNIX Epoch: seconds since 1/1/1970 00:00:00
                                <1>     ;
                                <1>         ;  ((Modified registers: EAX, EDX, ECX, EBX))
                                <1>     ;
                                <1>
                                <1>     ; 18/04/2021
                                <1>     ; INPUT:
                                <1>     ;       none (real time clock)
                                <1>     ; OUTPUT:
                                <1>     ;       eax = unix epoch time value
                                <1>     ;           (seconds since 1/1/1970 00:00:00)
                                <1>
00006E5B E8FDF4FFFF             <1>     call    get_rtc_time          ; Return Current Time
                                <1>         ;xchg      ch, cl ; 18/04/2021
00006E60 66890D[7A730100]       <1>         mov        [hour], cx    ; BCD, cl = minute, ch = hour
                                <1>         ;xchg      dh, dl ; 18/04/2021
                                <1>     ;mov    [second], dx  ; BCD, dh = second, dl = dse
                                <1>     ; 18/04/2021
00006E67 8835[7E730100]         <1>     mov     [second], dh  ; second
                                <1>     ;
00006E6D E856F5FFFF             <1>         call       get_rtc_date          ; Return Current Date
                                <1>         ;xchg      ch, cl ; 18/04/2021
00006E72 66890D[74730100]       <1>         mov        [year], cx    ; BCD, cl = year, ch = century
                                <1>         ;xchg      dh, dl ; 18/04/2021
00006E79 668915[76730100]       <1>         mov        [month], dx   ; BCD, dl = day, dh = month
                                <1>     ;
                                <1>     ;mov    al, [hour]    ; Hour
                                <1>         ; 18/04/2021
00006E80 A0[7B730100]           <1>     mov     al, [hour+1]  ; Hour
                                <1>             ; AL <-- BCD number
00006E85 D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006E87 D50A                   <1>         aad        ; AX= AH*10+AL
                                <1>     ;mov    [hour], al
                                <1>     ;mov    al, [hour+1]  ; Minute
00006E89 8605[7A730100]         <1>     xchg    al, [hour]    ; [hour] = hour, al = minute
                                <1>             ; AL <-- BCD number
00006E8F D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006E91 D50A                   <1>         aad        ; AX= AH*10+AL
00006E93 A2[7C730100]           <1>     mov     [minute], al
00006E98 A0[7E730100]           <1>     mov     al, [second]  ; Second
                                <1>             ; AL <-- BCD number
00006E9D D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006E9F D50A                   <1>         aad        ; AX= AH*10+AL
00006EA1 A2[7E730100]           <1>     mov     [second], al
00006EA6 66A1[74730100]         <1>     mov     ax, [year]    ; Year (century)
                                <1>     ; 18/04/2021
                                <1>     ;push   eax ; puhs ax
                                <1>     ;mov    al, ah ; century ; 18/04/2021
                                <1>             ; AL <-- BCD number
00006EAC D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006EAE D50A                   <1>         aad        ; AX= AH*10+AL
                                <1>     ;mov    ah, 100
                                <1>     ;mul    ah
                                <1>     ;mov    [year], ax
                                <1>     ; 18/04/2021
                                <1>     ; ax = al = year (0 to 99)
00006EB0 668705[74730100]       <1>     xchg    ax, [year]    ; [year+1] = century -> ah
                                <1>     ;pop    eax ; pop ax
00006EB7 88E0                   <1>     mov     al, ah  ; century
                                <1>             ; AL <-- BCD number
00006EB9 D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006EBB D50A                   <1>         aad        ; AX= AH*10+AL
                                <1>     ; 18/04/2021
00006EBD B464                   <1>     mov     ah, 100       ; 100*(century byte of year)
00006EBF F6E4                   <1>     mul     ah
                                <1>     ;
00006EC1 660105[74730100]       <1>     add     [year], ax
                                <1>     ;mov    al, [month]   ; Month
                                <1>     ; 18/04/2021
00006EC8 A0[77730100]           <1>     mov     al, [month+1] ; Month
                                <1>             ; AL <-- BCD number
00006ECD D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006ECF D50A                   <1>         aad        ; AX= AH*10+AL
                                <1>     ;mov    [month], al
                                <1>         ;mov       al, [month+1] ; Day
                                <1>     ; 18/04/2021
00006ED1 8605[76730100]         <1>     xchg    al, [month]   ; [month] = month, al = day
                                <1>             ; AL <-- BCD number
00006ED7 D410                   <1>         db  0D4h, 10h             ; Undocumented inst. AAM
                                <1>                                   ; AH = AL / 10h
                                <1>                                   ; AL = AL MOD 10h
00006ED9 D50A                   <1>         aad        ; AX= AH*10+AL
00006EDB A2[78730100]           <1>         mov     [day], al
                                <1>
00006EE0 C3                     <1>     retn    ; 30/12/2017
                                <1>
                                <1> epoch:
00006EE1 E875FFFFFF             <1>     call    get_rtc_date_time ; TRDOS 386 - 30/12/2017
                                <1>
                                <1> convert_to_epoch:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 15/03/2015 (Retro UNIX 386 v1 - 32 bit modification)
                                <1>     ; 09/04/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; ((Modified registers: EAX, EDX, EBX))
                                <1>     ;
                                <1>     ; Derived from DALLAS Semiconductor
                                <1>     ; Application Note 31 (DS1602/DS1603)
                                <1>     ; 6 May 1998
00006EE6 29C0                   <1>     sub     eax, eax
00006EE8 66A1[74730100]         <1>     mov     ax, [year]
00006EEE 662DB207               <1>     sub     ax, 1970
00006EF2 BA6D010000             <1>     mov     edx, 365
00006EF7 F7E2                   <1>     mul     edx
00006EF9 31DB                   <1>     xor     ebx, ebx
00006EFB 8A1D[76730100]         <1>     mov     bl, [month]
```

```
00006F01 FECB                <1>     dec     bl
00006F03 D0E3                <1>     shl     bl, 1
                             <1>     ;sub    edx, edx
00006F05 668B93[80730100]    <1>     mov     dx, [EBX+DMonth]
00006F0C 8A1D[78730100]      <1>         mov     bl, [day]
00006F12 FECB                <1>     dec     bl
00006F14 01D0                <1>     add     eax, edx
00006F16 01D8                <1>     add     eax, ebx
                             <1>                     ; EAX = days since 1/1/1970
00006F18 668B15[74730100]    <1>     mov     dx, [year]
00006F1F 6681EAB107          <1>     sub     dx, 1969
                             <1>     ;shr    dx, 1
                             <1>     ;shr    dx, 1
                             <1>     ; 25/07/2022
00006F24 C1EA02              <1>     shr     edx, 2
                             <1>             ; (year-1969)/4
00006F27 01D0                <1>     add     eax, edx
                             <1>                     ; + leap days since 1/1/1970
00006F29 803D[76730100]02    <1>     cmp     byte [month], 2       ; if past february
00006F30 760E                <1>     jna     short cte1
00006F32 668B15[74730100]    <1>     mov     dx, [year]
00006F39 6683E203            <1>     and     dx, 3 ; year mod 4
00006F3D 7501                <1>     jnz     short cte1
                             <1>                     ; and if leap year
                             <1>     ;add    eax, 1 ; add this year's leap day (february 29)
                             <1>     ; 25/07/2022
00006F3F 40                  <1>     inc     eax
                             <1> cte1:                       ; compute seconds since 1/1/1970
00006F40 BA18000000          <1>     mov     edx, 24
00006F45 F7E2                <1>     mul     edx
00006F47 8A15[7A730100]      <1>     mov     dl, [hour]
00006F4D 01D0                <1>     add     eax, edx
                             <1>             ; EAX = hours since 1/1/1970 00:00:00
                             <1>     ;mov    ebx, 60
00006F4F B33C                <1>     mov     bl, 60
00006F51 F7E3                <1>     mul     ebx
00006F53 8A15[7C730100]      <1>     mov     dl, [minute]
00006F59 01D0                <1>     add     eax, edx
                             <1>             ; EAX = minutes since 1/1/1970 00:00:00
                             <1>     ;mov    ebx, 60
00006F5B F7E3                <1>     mul     ebx
00006F5D 8A15[7E730100]      <1>     mov     dl, [second]
00006F63 01D0                <1>     add     eax, edx
                             <1>             ; EAX -> seconds since 1/1/1970 00:00:00
00006F65 C3                  <1>     retn
                             <1>
                             <1> ;set_date_time:
                             <1> convert_from_epoch:
                             <1>     ; 25/07/2022 (v2.0.5)
                             <1>     ; 18/04/2021 (v2.0.4)
                             <1>     ; 31/12/2017 (v2.0.0)
                             <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 15/03/2015 (Retro UNIX 386 v1 - 32 bit version)
                             <1>     ; 20/06/2013 (Retro UNIX 8086 v1)
                             <1>     ; 'convert_from_epoch' procedure prototype:
                             <1>     ;                   UNIXCOPY.ASM, 10/03/2013
                             <1>     ;
                             <1>     ; ((Modified registers: EAX, EDX, ECX, EBX))
                             <1>     ;
                             <1>     ; Derived from DALLAS Semiconductor
                             <1>     ; Application Note 31 (DS1602/DS1603)
                             <1>     ; 6 May 1998
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ; EAX = Unix (Epoch) Time
                             <1>     ;
00006F66 31D2                <1>     xor     edx, edx
                             <1>     ;mov    ecx, 60
                             <1>     ; 25/07/2022
00006F68 29C9                <1>     sub     ecx, ecx
00006F6A B13C                <1>     mov     cl, 60
00006F6C F7F1                <1>     div     ecx
                             <1>     ;mov    [imin], eax   ; whole minutes
                             <1>                     ; since 1/1/1970
00006F6E 668915[7E730100]    <1>     mov     [second], dx  ; leftover seconds
00006F75 29D2                <1>     sub     edx, edx
00006F77 F7F1                <1>     div     ecx
                             <1>     ;mov    [ihrs], eax   ; whole hours
                             <1>     ;                   ; since 1/1/1970
00006F79 668915[7C730100]    <1>     mov     [minute], dx  ; leftover minutes
00006F80 31D2                <1>     xor     edx, edx
                             <1>     ;mov    cx, 24
00006F82 B118                <1>     mov     cl, 24
00006F84 F7F1                <1>     div     ecx
                             <1>     ;mov    [iday], ax   ; whole days
                             <1>                          ; since 1/1/1970
00006F86 668915[7A730100]    <1>     mov     [hour], dx   ; leftover hours
00006F8D 05DB020000          <1>     add     eax, 365+366 ; whole day since
                             <1>                          ; 1/1/1968
                             <1>     ;mov    [iday], ax
00006F92 50                  <1>     push    eax
00006F93 29D2                <1>     sub     edx, edx
                             <1>     ;mov    ecx, (4*365)+1 ; 4 years = 1461 days
                             <1>     ; 25/07/2022
00006F95 66B9B505            <1>     mov     cx, (4*365)+1
00006F99 F7F1                <1>     div     ecx
00006F9B 59                  <1>     pop     ecx
                             <1>     ;mov    [lday], ax   ; count of quadyrs (4 years)
                             <1>     ;push   dx
                             <1>     ; 18/04/2021
00006F9C 52                  <1>     push    edx
                             <1>     ;mov    [qday], dx   ; days since quadyr began
00006F9D 6683FA3C            <1>     cmp     dx, 31+29    ; if past feb 29 then
00006FA1 F5                  <1>     cmc                  ; add this quadyr's leap day
00006FA2 83D000              <1>     adc     eax, 0       ; to # of qadyrs (leap days)
                             <1>     ;mov    [lday], ax   ; since 1968
                             <1>     ;mov    cx, [iday]
00006FA5 91                  <1>     xchg    ecx, eax     ; ECX = lday, EAX = iday
00006FA6 29C8                <1>     sub     eax, ecx     ; iday - lday
                             <1>     ;mov    ecx, 365
                             <1>     ; 25/07/2022
00006FA8 66B96D01            <1>     mov     cx, 365
00006FAC 31D2                <1>     xor     edx, edx
                             <1>     ; EAX = iday-lday, EDX = 0
00006FAE F7F1                <1>     div     ecx
                             <1>     ;mov    [iyrs], ax   ; whole years since 1968
                             <1>     ;jday = iday - (iyrs*365) - lday
                             <1>     ;mov    [jday], dx   ; days since 1/1 of current year
                             <1>     ;add    eax, 1968
00006FB0 6605B007            <1>     add     ax, 1968     ; compute year
00006FB4 66A3[74730100]      <1>     mov     [year], ax
                             <1>     ;mov    cx, dx
                             <1>     ; 25/07/2022
00006FBA 89D1                <1>     mov     ecx, edx
```

```
                                <1>     ;;mov   dx, [qday]
                                <1>     ;pop    dx
                                <1>     ; 18/04/2021
00006FBC 5A                     <1>     pop     edx
00006FBD 6681FA6D01             <1>     cmp     dx, 365     ; if qday <= 365 and qday >= 60
00006FC2 7708                   <1>     ja      short cfe1   ; jday = jday +1
00006FC4 6683FA3C               <1>     cmp     dx, 60       ; if past 2/29 and leap year then
00006FC8 F5                     <1>         cmc             ; add a leap day to the # of whole
                                <1>     ;adc    cx, 0        ; days since 1/1 of current year
                                <1>     ; 25/07/2022
00006FC9 83D100                 <1>     adc     ecx, 0
                                <1> cfe1:
                                <1>     ;mov    [jday], cx
                                <1>     ;mov    bx, 12       ; estimate month
                                <1>     ; 18/04/2021
00006FCC 29DB                   <1>     sub     ebx, ebx
00006FCE B30C                   <1>     mov     bl, 12
00006FD0 66BA6E01               <1>     mov     dx, 366      ; mday, max. days since 1/1 is 365
00006FD4 6683E003               <1>     and     ax, 11b      ; year mod 4 (and dx, 3)
                                <1> cfe2:       ; Month calculation  ; 0 to 11  (11 to 0)
                                <1>     ;cmp    cx, dx       ; mday = # of days passed from 1/1
                                <1>     ; 25/07/2022
00006FD8 39D1                   <1>     cmp     ecx, edx
00006FDA 731B                   <1>     jnb     short cfe3
                                <1>     ;dec    bx           ; month = month - 1
                                <1>     ;shl    bx, 1
                                <1>     ; 18/04/2021
00006FDC FECB                   <1>     dec     bl
00006FDE D0E3                   <1>     shl     bl, 1
00006FE0 668B93[80730100]       <1>     mov     dx, [EBX+DMonth] ; # elapsed days at 1st of month
                                <1>     ; 18/04/2021
                                <1>     ;shr    bx, 1        ; bx = month - 1 (0 to 11)
00006FE7 D0EB                   <1>     shr     bl, 1
                                <1>     ;cmp    bx, 1        ; if month > 2 and year mod 4  = 0
00006FE9 80FB01                 <1>     cmp     bl, 1
00006FEC 76EA                   <1>     jna     short cfe2   ; then mday = mday + 1
00006FEE 76E8                   <1>     jna     short cfe2   ; then mday = mday + 1
00006FF0 08C0                   <1>     or      al, al       ; if past 2/29 and leap year then
00006FF2 75E4                   <1>     jnz     short cfe2   ; add leap day (to mday)
                                <1>     ;inc    dx           ; mday = mday + 1
                                <1>     ; 25/07/2022
00006FF4 42                     <1>     inc     edx
00006FF5 EBE1                   <1>     jmp     short cfe2
                                <1> cfe3:
                                <1>     ;inc    bx           ; -> bx = month, 1 to 12
                                <1>     ; 18/04/2021
00006FF7 FEC3                   <1>     inc     bl
00006FF9 66891D[76730100]       <1>     mov     [month], bx
                                <1>     ;sub    cx, dx       ; day = jday - mday + 1
                                <1>     ; 25/07/2022
00007000 29D1                   <1>     sub     ecx, edx
                                <1>     ;inc    cx
                                <1>     ; 18/04/2021
00007002 FEC1                   <1>     inc     cl
                                <1>     ;mov    [day], cx
00007004 880D[78730100]         <1>     mov     [day], cl
                                <1>
                                <1>     ; eax, ebx, ecx, edx is changed at return
                                <1>     ; output ->
                                <1>     ; [year], [month], [day], [hour], [minute], [second]
                                <1>
0000700A C3                     <1>     retn    ; 31/12/2017 (TRDOS 386)
                                <1>
                                <1> set_rtc_date_time:
                                <1>     ; 31/12/2017 (v2.0.0)
                                <1>     ; 30/12/2017 (TRDOS 386)
                                <1>     ; 15/03/2015 (Retro UNIX 386 v1 - 32 bit version)
                                <1>     ; 20/06/2013 (Retro UNIX 8086 v1)
0000700B E80F000000             <1>     call    set_date_bcd
                                <1>     ; Set real-time clock date
00007010 E8D9F3FFFF             <1>     call    set_rtc_date ; RTC_50
                                <1>     ; Set real-time clock time
00007015 E832000000             <1>     call    set_time_bcd
0000701A E96CF3FFFF             <1>     jmp     set_rtc_time ; RTC_30
                                <1>
                                <1> ; 31/12/2017
                                <1> set_date_bcd:
0000701F A0[75730100]           <1>         mov     al, [year+1]
00007024 D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
00007026 D510                   <1>     db      0D5h, 10h    ; Undocumented inst. AAD
                                <1>                          ; AL = AH * 10h + AL
00007028 88C5                   <1>     mov     ch, al ; century (BCD)
0000702A A0[74730100]           <1>     mov     al, [year]
0000702F D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
00007031 D510                   <1>     db      0D5h, 10h    ; Undocumented inst. AAD
                                <1>                          ; AL = AH * 10h + AL
00007033 88C1                   <1>     mov     cl, al ; year (BCD)
00007035 A0[76730100]           <1>         mov         al, [month]
0000703A D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
0000703C D510                   <1>     db      0D5h, 10h    ; Undocumented inst. AAD
                                <1>                          ; AL = AH * 10h + AL
0000703E 88C6                   <1>     mov     dh, al ; month (BCD)
00007040 A0[78730100]           <1>     mov     al, [day]
00007045 D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
00007047 D510                   <1>     db      0D5h, 10h    ; Undocumented inst. AAD
                                <1>                          ; AL = AH * 10h + AL
                                <1>     ; 18/04/2021
00007049 88C2                   <1>     mov     dl, al ; day (BCD)
0000704B C3                     <1>     retn    ; 30/12/2017
                                <1>
                                <1> ; 31/12/2017
                                <1> set_time_bcd:
                                <1>         ; Read real-time clock time
                                <1>     ; (get day light saving time bit status)
0000704C FA                     <1>     cli
0000704D E8D7F4FFFF             <1>     call    UPD_IPR              ; CHECK FOR UPDATE IN PROCESS
                                <1>     ; cf = 1 -> al = 0
00007052 7207                   <1>         jc      short stime1
00007054 B00B                   <1>     mov     al, CMOS_REG_B       ; ADDRESS ALARM REGISTER
00007056 E804F5FFFF             <1>     call    CMOS_READ            ; READ CURRENT VALUE OF DSE BIT
                                <1> stime1:
0000705B FB                     <1>     sti
0000705C 2401                   <1>     and     al, 00000001b        ; MASK FOR VALID DSE BIT
0000705E 88C2                   <1>     mov     dl, al               ; SET [DL] TO ZERO FOR NO DSE BIT
                                <1>     ; DL = 1 or 0 (day light saving time)
                                <1>
00007060 A0[7A730100]           <1>     mov     al, [hour]
00007065 D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
00007067 D510                   <1>     db      0D5h,10h     ; Undocumented inst. AAD
                                <1>                          ; AL = AH * 10h + AL
00007069 88C5                   <1>     mov     ch, al ; hour (BCD)
0000706B A0[7C730100]           <1>         mov     al, [minute]
00007070 D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
00007072 D510                   <1>     db      0D5h,10h     ; Undocumented inst. AAD
```

```
                                <1>                         ; AL = AH * 10h + AL
00007074 88C1                   <1>     mov     cl, al      ; minute (BCD)
00007076 A0[7E730100]           <1>         mov     al, [second]
0000707B D40A                   <1>     aam     ; ah = al / 10, al = al mod 10
0000707D D510                   <1>     db      0D5h,10h    ; Undocumented inst. AAD
                                <1>                         ; AL = AH * 10h + AL
0000707F 88C6                   <1>     mov     dh, al      ; second (BCD)
00007081 C3                     <1>     retn    ; 30/12/2017
                                %include 'trdosk2.s' ; 04/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - DRV INIT : trdosk2.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 19/12/2025 (Previous: 22/05/2024, TRDOS 386 v2.0.8)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 04/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.14 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ****************************************************************************
                                <1> ; DRV_INIT.ASM (c) 2009-2011 Erdogan TAN  [26/09/2009] Last Update: 07/08/2011
                                <1> ;
                                <1>
                                <1> ldrv_init: ; Logical Drive Initialization
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 22/05/2024 (TRDOS 386 Kernel v2.0.8)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 30/08/2020
                                <1>     ; 25/08/2020
                                <1>     ; 11/08/2020 - 13/08/2020
                                <1>     ; 17/07/2020 - 20/07/2020
                                <1>     ; 14/07/2020 - 15/07/2020
                                <1>     ; 30/01/2018
                                <1>     ; 27/12/2017
                                <1>     ; 12/02/2016
                                <1>     ; 06/01/2016
                                <1>     ;       ('diskinit.inc', 'diskio.inc' integration)
                                <1>     ; 04/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 07/08/2011
                                <1>     ; 20/09/2009
                                <1>     ; 2005
                                <1>
                                <1>     ; 15/07/2020
                                <1>     ;movzx  ecx, byte [HF_NUM] ; number of fixed disks
                                <1>     ;cmp    cl, 1
                                <1>     ;jnb    short load_hd_partition_tables
                                <1>
00007082 A0[48770100]           <1>     mov     al, [HF_NUM] ; number of fixed disks
00007087 20C0                   <1>     and     al, al
00007089 7501                   <1>     jnz     short load_hd_partition_tables
                                <1>
                                <1>     ; no any hard disks
0000708B C3                     <1>     retn
                                <1>
                                <1> load_hd_partition_tables:
                                <1>     ;mov    esi, [HDPM_TBL_VEC] ; primary master disk FDPT
                                <1>     ; 15/07/2020
0000708C BE[4C770100]           <1>     mov     esi, HDPM_TBL_VEC
00007091 BF[767B0100]           <1>     mov     edi, PTable_hd0
00007096 B280                   <1>     mov     dl, 80h
                                <1>     ; 15/07/2020
00007098 A2[8B660000]           <1>     mov     [hdc], al
                                <1>     ;xor    ecx, ecx ; 0
                                <1> load_next_hd_partition_table:
                                <1>     ; 20/07/2020
0000709D 31C9                   <1>     xor     ecx, ecx ; 0
                                <1>     ;push   ecx
0000709F 57                     <1>     push    edi ; *
                                <1>     ;push   esi ; FDPT (+ DPTE) address
                                <1>     ; 15/07/2020
000070A0 AD                     <1>     lodsd
000070A1 56                     <1>     push    esi ; ** ; next FDPT (+ DPTE) address ptr
                                <1>
                                <1>     ;mov    al, [esi+20] ; DPTE offset 4
                                <1>     ;and    al, 40h ;  LBA bit (bit 6)
                                <1>     ;;shr   al, 6
                                <1>     ;mov    [HD_LBA_yes], al
                                <1>
                                <1>     ; 15/07/2020
000070A2 8A4814                 <1>     mov     cl, [eax+20]
000070A5 80E140                 <1>     and     cl, 40h
                                <1>     ;mov    [HD_LBA_yes], cl
                                <1>     ; 22/05/2024 (BugFix)
000070A8 0FB6C2                 <1>     movzx   eax, dl
000070AB 05[F67B0100]           <1>     add     eax, HD_LBA_yes - 80h
000070B0 8808                   <1>     mov     [eax], cl
                                <1>
000070B2 E81A030000             <1>     call    load_masterboot
                                <1>     ;jc     short pass_pt_this_hard_disk
                                <1>     ; 13/08/2020
                                <1>     ;jc     pass_pt_this_hard_disk
                                <1>     ; 25/07/2022
000070B7 7302                   <1>     jnc     short load_mbr_ok
000070B9 EB7C                   <1>     jmp     pass_pt_this_hard_disk
                                <1>
                                <1> load_mbr_ok:
000070BB BB[347B0100]           <1>     mov     ebx, PartitionTable
000070C0 89DE                   <1>     mov     esi, ebx
                                <1>     ;mov    ecx, 16
000070C2 B110                   <1>     mov     cl, 16
000070C4 F3A5                   <1>     rep     movsd
000070C6 89DE                   <1>     mov     esi, ebx
                                <1>     ;mov    byte [hdc], 4 ; 4 - partition index
                                <1>     ; 15/07/2020
000070C8 C605[7B7C0100]04       <1>     mov     byte [PP_Counter], 4
                                <1> loc_validate_hdp_partition:
                                <1>     ;cmp    byte [esi+ptFileSystemID], 0
                                <1>     ;jna    short loc_validate_next_hdp_partition2
                                <1>     ; 13/08/2020
000070CF 8A4604                 <1>     mov     al, [esi+ptFileSystemID]
000070D2 20C0                   <1>     and     al, al
000070D4 7449                   <1>     jz      short loc_validate_next_hdp_partition2
                                <1>
000070D6 56                     <1>     push    esi ; *** ; Masterboot partition table offset
000070D7 52                     <1>     push    edx ; **** ; dl = Physical drive number
                                <1>
                                <1>     ; 13/08/2020
000070D8 3C05                   <1>     cmp     al, 05h  ; Extended partition CHS
000070DA 7404                   <1>     je      short loc_set_ep_counter
000070DC 3C0F                   <1>     cmp     al, 0Fh  ; Extended partition LBA
000070DE 7511                   <1>     jne     short loc_validate_next_hdp_partition0
                                <1>
                                <1>     ;;inc   byte [PP_Counter]
```

```
                                <1>     ; 15/07/2020
                                <1>     ;inc   byte [EP_Counter] ; disk has valid partition(s)
                                <1>
                                <1> loc_set_ep_counter:
                                <1>     ; 13/08/2020
000070E0 803D[7C7C0100]80       <1>     cmp    byte [EP_Counter], 80h
000070E7 7334                   <1>     jnb    short loc_validate_next_hdp_partition1
                                <1>
000070E9 8815[7C7C0100]         <1>     mov    byte [EP_Counter], dl ; disk drv has extd. part.
                                <1>
000070EF EB2C                   <1>     jmp    short loc_validate_next_hdp_partition1
                                <1>
                                <1> loc_validate_next_hdp_partition0:
000070F1 31FF                   <1>     xor    edi, edi ; 0
                                <1>     ; Input -> ESI = PartitionTable offset
                                <1>     ; DL = Hard disk drive number
                                <1>     ; EDI = 0 -> Primary Partition
                                <1>     ; EDI > 0 -> Extended Partition's Start Sector
000070F3 E87E010000             <1>     call   validate_hd_fat_partition
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     jnc    short loc_set_valid_hdp_partition_entry
                                <1>
                                <1>     ;pop   edx
                                <1>     ;push  edx
                                <1>     mov    edx, [esp]  ; ****
                                <1>     mov    esi, [esp+4] ; *** ; 30/01/2018
                                <1>     call   validate_hd_fs_partition
                                <1> %endif
000070F8 7223                   <1>     jc     short loc_validate_next_hdp_partition1
                                <1>
                                <1> loc_set_valid_hdp_partition_entry:
000070FA 8A0D[8D2E0100]         <1>     mov    cl, [Last_DOS_DiskNo]
00007100 80C141                 <1>     add    cl, 'A'
                                <1>     ; ESI = Logical dos drive description table address
00007103 880E                   <1>     mov    [esi+LD_Name], cl
                                <1>     ; 15/07/2020
00007105 8A4602                 <1>     mov    al, [esi+LD_PhyDrvNo] ; Physical drive number
                                <1>     ;mov   al, [esp] ; ****
00007108 2C7F                   <1>     sub    al, 7Fh
                                <1>            ; AL = 1 to 4
0000710A C0E002                 <1>     shl    al, 2 ; AL = 4 to 16
                                <1>
0000710D 8A15[7B7C0100]         <1>     mov    dl, [PP_Counter]
                                <1>
                                <1>     ;sub   al, [PP_Counter]
00007113 28D0                   <1>     sub    al, dl ; [PP_Counter] ; 4 - partition index
                                <1>
                                <1>     ; AL = Partition entry/index, 0 based
                                <1>     ;  0 -> hd 0, Partition Table offset = 0
                                <1>     ; 15 -> hd 3, Partition Table offset = 3
                                <1>
                                <1>     ;mov   [esi+LD_PartitionEntry], al
                                <1>
                                <1>     ; 15/07/2020
00007115 B404                   <1>     mov    ah, 4
                                <1>     ;sub   ah, [PP_Counter]
00007117 28D4                   <1>     sub    ah, dl
                                <1>
                                <1>     ; AH = Primary partition index, 0 to 3 ; pt entry
                                <1>     ;              (4 to 7 for logical disk partitions)
                                <1>
                                <1>     ;mov   [esi+LD_DParamEntry], ah
00007119 6689467C               <1>     mov    [esi+LD_PartitionEntry], ax
                                <1>
                                <1> loc_validate_next_hdp_partition1:
0000711D 5A                     <1>     pop    edx ; **** ; dl = Physical drive number
0000711E 5E                     <1>     pop    esi ; *** ; Masterboot partition table offset
                                <1>
                                <1> loc_validate_next_hdp_partition2:
                                <1>     ; ESI = PartitionTable offset
                                <1>     ; DL = Hard/Fixed disk drive number
                                <1>
                                <1>     ;dec   byte [hdc] ; 4 - partition index
                                <1>     ;jz    short pass_pt_this_hard_disk
                                <1>     ; 15/07/2020
0000711F FE0D[7B7C0100]         <1>     dec    byte [PP_Counter] ; 4 - partition index
00007125 7410                   <1>     jz     short pass_pt_this_hard_disk
                                <1>
00007127 83C610                 <1>     add    esi, 16 ; 10h
0000712A EBA3                   <1>     jmp    short loc_validate_hdp_partition
                                <1>
                                <1> loc_not_any_extd_partitions:
                                <1>     ; 15/07/2020
0000712C C3                     <1>     retn
                                <1>
                                <1> loc_next_hd_partition_table:
0000712D FEC2                   <1>     inc    dl
                                <1>     ; 15/07/2020
                                <1>     ;add   esi, 32 ; next FDPT address
0000712F 83C740                 <1>     add    edi, 64 ; next partition table destination
00007132 E966FFFFFF             <1>         jmp     load_next_hd_partition_table
                                <1>
                                <1> pass_pt_this_hard_disk:
                                <1>     ;pop   esi ; FDPT (+ DPTE) address
                                <1>     ; 15/07/2020
00007137 5E                     <1>     pop    esi ; ** ; next FDPT (+ DPTE) address ptr
00007138 5F                     <1>     pop    edi ; * ; Ptable_hd?
                                <1>     ;pop   ecx
                                <1>     ;loop  loc_next_hd_partition_table
00007139 FE0D[8B660000]         <1>     dec    byte [hdc]
0000713F 75EC                   <1>     jnz    short loc_next_hd_partition_table
                                <1>
                                <1>     ;cmp   byte [PP_Counter], 1
                                <1>     ;jnb   short load_extended_dos_partitions
                                <1>     ;; Empty partition table
                                <1>     ;retn
                                <1>
                                <1>     ; 11/08/2020
                                <1>     ; 17/07/2020
                                <1> check_extended_partitions:
                                <1>     ; 15/07/2020
                                <1>     ;cmp   byte [EP_Counter], 0
                                <1>     ;jna   short loc_not_any_extd_partitions
                                <1>     ; 13/08/2020
00007141 A0[7C7C0100]           <1>     mov    al, [EP_Counter] ; 1st disk drv has extd partition
00007146 08C0                   <1>     or     al, al ; 0 ?
00007148 74E2                   <1>     jz     short loc_not_any_extd_partitions
                                <1>
                                <1> load_extended_dos_partitions:
                                <1>     ;mov   byte [hdc], 80h
                                <1>     ; 13/08/2020
0000714A A2[8B660000]           <1>     mov    byte [hdc], al ; 1st disk drv has extd partition
```

```
                                <1>     ; 25/08/2020
0000714F 2C80                   <1>     sub     al, 80h
00007151 740E                   <1>     jz      short loc_set_ext_ptable_hd0
00007153 C0E006                 <1>     shl     al, 6 ; * 64
00007156 0FB6F0                 <1>     movzx   esi, al
00007159 81C6[767B0100]         <1>     add     esi, PTable_hd0
0000715F EB05                   <1>     jmp     short next_hd_extd_partition
                                <1>
                                <1>     ; 25/08/2020
                                <1> loc_set_ext_ptable_hd0:
00007161 BE[767B0100]           <1>     mov     esi, PTable_hd0
                                <1>
                                <1> next_hd_extd_partition:
                                <1>     ; 17/07/2020
                                <1>     ;mov    byte [EP_Counter], 0 ; Reset for each physical disk
                                <1>     ; 13/08/2020
                                <1>     ;mov    byte [LD_Counter], 0 ; Reset logical drive index
00007166 66C705[7C7C0100]00-    <1>     mov     word [EP_Counter], 0 ; Reset EP index and LD index
0000716E 00                     <1>
                                <1>
0000716F 56                     <1>     push    esi ; **** ; PTable_hd? offset
                                <1>
00007170 C605[7B7C0100]04       <1>     mov     byte [PP_Counter], 4
                                <1>                             ; set for each extd partition table
                                <1>     ;;mov   ecx, 4
                                <1>     ;mov    cl, 4
00007177 8A15[8B660000]         <1>     mov     dl, [hdc]
                                <1> hd_check_fs_id_05h:
0000717D 8A4604                 <1>     mov     al, [esi+ptFileSystemID]
00007180 3C05                   <1>     cmp     al, 05h ; Is it an extended dos partition ?
00007182 7411                   <1>     je      short loc_set_ep_start_sector ; yes
                                <1> hd_check_fs_id_0Fh:
00007184 3C0F                   <1>     cmp     al, 0Fh ; Is it an extended win4 (LBA mode) partition ?
00007186 740D                   <1>     je      short loc_set_ep_start_sector ; yes
                                <1>
                                <1> continue_to_check_ep:
                                <1>     ;add    esi, 16
                                <1>     ;loop   hd_check_fs_id_05h
                                <1>     ; 15/07/2020
                                <1>     ;dec    cl
                                <1>     ;jz     short continue_check_ep_next_disk
00007188 FE0D[7B7C0100]         <1>     dec     byte [PP_Counter] ; 4 --> 0
0000718E 7432                   <1>     jz      short continue_check_ep_next_disk
00007190 83C610                 <1>     add     esi, 16
00007193 EBE8                   <1>     jmp     short hd_check_fs_id_05h
                                <1>
                                <1> loc_set_ep_start_sector:
                                <1>     ; dl = [hdc] ; Drive number
                                <1>     ; 15/07/2020
00007195 8B4E08                 <1>     mov     ecx, [esi+ptStartSector]
                                <1>     ; 30/08/2020
00007198 890D[7E7C0100]         <1>     mov     [MBR_EP_StartSector], ecx
                                <1>     ; 20/07/2020
                                <1> loc_validate_hde_partition_next:
                                <1>     ; 22/05/2024 (BugFix)
0000719E 0FB6FA                 <1>     movzx   edi, dl
000071A1 81C7[F67B0100]         <1>     add     edi, HD_LBA_yes - 80h
                                <1>     ;
000071A7 890D[827C0100]         <1>     mov     [EP_StartSector], ecx ; Extended partition's start sector
000071AD BB[76790100]           <1>         mov ebx, MasterBootBuff
                                <1>     ; 22/05/2024
000071B2 803F01                 <1>     cmp     byte [edi], 1 ; LBA ready = Yes
                                <1>     ;cmp    byte [HD_LBA_yes], 1 ; LBA ready = Yes
000071B5 7227                   <1>     jb      short loc_hd_load_ep_05h ; cf = 1 ; 20/07/2020
                                <1>     ; 11/08/2020
                                <1>     ; (BugFix for extended partition type 05h beyond CHS limit)
                                <1>     ; (Infact if extended partition starts at the beyond of CHS limit,
                                <1>     ;  it's partition ID must be 0Fh but they/somebodies had used 05h.)
                                <1>     ;cmp    al, 05h
                                <1>     ;je     short loc_hd_load_ep_05h
                                <1> loc_hd_load_ep_0Fh:
                                <1>     ; 04/01/2016
                                <1>     ;push   ecx
                                <1>     ; 15/07/2020
                                <1>     ;mov    ecx, [esi+ptStartSector] ; sector number
                                <1>     ;mov    ebx, MasterBootBuff ; buffer address
                                <1>     ; LBA read/write (with private LBA function)
                                <1>     ;((Retro UNIX 386 v1 - DISK I/O code by Erdogan Tan))
                                <1>     ; dl = physical drive number (0,1, 80h, 81h, 82h, 83h)
                                <1>     ;mov    ah, 1Bh ; LBA read
                                <1>     ;mov    al, 1 ; sector count
000071B7 66B8011B               <1>     mov     ax, 1B01h
000071BB E84FDEFFFF             <1>     call    int13h
                                <1>     ;pop    ecx
                                <1>     ;jnc    short loc_hd_move_ep_table
                                <1>     ; 15/07/2020
000071C0 732E                   <1>     jnc     short loc_validate_hde_partition
                                <1>
                                <1> continue_check_ep_next_disk:
                                <1>     ; 15/07/2020
                                <1>     ;pop    edi ; PTable_ep?
000071C2 5E                     <1>     pop     esi ; **** ; PTable_hd?
000071C3 A0[48770100]           <1>     mov     al, [HF_NUM] ; number of hard disks
000071C8 047F                   <1>     add     al, 7Fh
000071CA 3805[8B660000]         <1>     cmp     [hdc], al
000071D0 730B                   <1>     jnb     short loc_validating_hd_partitions_ok
000071D2 83C640                 <1>     add     esi, 64
                                <1>     ; 15/07/2020
                                <1>     ;add    edi, 64
000071D5 FE05[8B660000]         <1>     inc     byte [hdc]
000071DB EB89                   <1>     jmp     short next_hd_extd_partition
                                <1>
                                <1> loc_validating_hd_partitions_ok:
                                <1>     ; 15/07/2020
                                <1>     ;mov    al, [Last_DOS_DiskNo]
                                <1> loc_drv_init_retn:
000071DD C3                     <1>     retn
                                <1>
                                <1> loc_hd_load_ep_05h:
                                <1>     ; 20/07/2020 ('diskio.s', int13h, cf = 1 -> bugfix)
                                <1>     ;clc    ; (Bug: int13h would not clear carry flag bit,
                                <1>     ;       ; even if there would not be an error)
                                <1>     ;       ; ((Fix: now, int13h procedure clears carry flag
                                <1>     ;       ;  at the entrance of it.. 20/07/2020))
                                <1>     ; 15/07/2020
                                <1>     ;push   ecx
000071DE 8A7601                 <1>     mov     dh, [esi+ptBeginHead]
000071E1 668B4E02               <1>         mov     cx, [esi+ptBeginSector]
000071E5 66B80102               <1>     mov     ax, 0201h ; Read 1 sector
                                <1>     ;mov    ebx, MasterBootBuff
000071E9 E821DEFFFF             <1>     call    int13h ; 20/07/2020
                                <1>                    ; 'diskio.s' modification, 'clc'
                                <1>     ;pop    ecx
000071EE 72D2                   <1>     jc      short continue_check_ep_next_disk
```

```
                                <1>     ; 15/07/2020
                                <1>     ;jmp   short loc_validate_hde_partition
                                <1>
                                <1>     ; 15/07/2020
                                <1> ;loc_hd_move_ep_table:
                                <1>     ;;pop  edi
                                <1>     ;;push edi  ; PTable_ep?
                                <1>     ;mov   edi, [esp]
                                <1>         ;mov       esi, PartitionTable ; Extended
                                <1>     ;mov   ebx, esi
                                <1>     ;;mov  ecx, 16
                                <1>     ;mov   cl, 16
                                <1>            ;rep   movsd
                                <1>     ;mov   esi, ebx
                                <1> ;loc_set_hde_sub_partition_count:
                                <1>     ;mov   byte [PP_Counter], 4
                                <1>     ;mov   byte [EP_Counter], 0
                                <1>
                                <1> loc_validate_hde_partition:
                                <1>     ; 13/08/2020
                                <1>     ; 15/07/2020
                                <1>     ;mov   byte [PP_Counter], 4
000071F0 BE[347B0100]           <1>     mov    esi, PartitionTable ; (in MasterBootBuff)
                                <1>     ; 13/08/2020
                                <1>     ;jmp   short get_minidisk_partition_entry
                                <1>
                                <1> ;get_minidisk_partition_entry:
                                <1> ;   ; 20/07/2020
                                <1> ;   cmp    byte [esi+ptFileSystemID], 0
                                <1> ;   ja     short loc_validate_minidisk_partition
                                <1> ;   ; 13/08/2020
                                <1> ;   jmp    short continue_check_ep_next_disk
                                <1>
                                <1> ;   ; 11/08/2020
                                <1> ;get_minidisk_partition_entry_next:
                                <1> ;   ; 13/08/2020
                                <1> ;   ;dec   byte [PP_Counter]
                                <1> ;   ;jz    short continue_check_ep_next_disk
                                <1> ;   ; 20/07/2020
                                <1> ;;get_minidisk_partition_entry_next:
                                <1> ;   ; 13/08/2020
                                <1> ;   cmp    esi, PartitionTable+64
                                <1> ;   jnb    short continue_check_ep_next_disk
                                <1> ;
                                <1> ;   add    esi, 16 ; 10h
                                <1> ;   ;jmp   short get_minidisk_partition_entry
                                <1>
                                <1>     ; 13/08/2020
                                <1> get_minidisk_partition_entry:
                                <1>     ; 20/07/2020
000071F5 807E0400               <1>     cmp    byte [esi+ptFileSystemID], 0
000071F9 76C7                   <1>     jna    short continue_check_ep_next_disk ; 13/08/2020
                                <1>
                                <1> loc_validate_minidisk_partition:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/08/2020
                                <1>     ; 20/07/2020
                                <1>     ;push  esi ; *** ; Extended partition table offset
                                <1>
                                <1>     ; 13/08/2020
000071FB FE05[7C7C0100]         <1>     inc    byte [EP_Counter] ; current (sub partition) index
                                <1>                              ; in current extended partition
00007201 BF[827C0100]           <1>     mov    edi, EP_StartSector
                                <1>
                                <1>     ; Input -> ESI = PartitionTable offset
                                <1>     ; DL = Hard disk drive number
                                <1>     ; EDI = Extended partition start sector pointer
00007206 E86B000000             <1>     call   validate_hd_fat_partition
                                <1>     ;pop   ecx ; *
0000720B 7308                   <1>     jnc    short loc_set_valid_hde_partition_entry
                                <1>                    ; jump down to deep !!!
                                <1>
                                <1>     ;pop   esi ; *** ; Extended partition table offset
                                <1>     ; 13/08/2020
                                <1>     ;mov   esi, PartitionTable
                                <1>
                                <1>     ; 11/08/2020
                                <1>     ; ESI = Extended partition table offset
0000720D 8A15[8B660000]         <1>     mov    dl, [hdc]
                                <1>
                                <1>     ;; DL = Hard disk drive number
                                <1>     ;dec   byte [PP_Counter]
                                <1>     ;jz    short continue_check_ep_next_disk
                                <1>     ;add   esi, 16 ; 10h
                                <1>     ;mov   dl, [hdc]
                                <1>     ;jmp   short get_minidisk_partition_entry
                                <1>
                                <1>     ; 11/08/2020
                                <1>     ;jmp   short get_minidisk_partition_entry_next
                                <1>
                                <1>     ; 23/08/2020
00007213 EB3E                   <1>     jmp    short validate_next_minidisk_partition_ok
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ;; jumping down to deep levels !!!
                                <1>     ; ((That is a pitty microsoft preferred ep table chain
                                <1>     ; instead of a single table as mbr partition table!?))
                                <1>
                                <1> loc_set_valid_hde_partition_entry:
                                <1>     ; 15/07/2020
00007215 A0[8B660000]           <1>     mov    al, [hdc] ; Hard disk drive number (>=80h)
0000721A 88C2                   <1>     mov    dl, al ; mov dl, [hdc]
0000721C 2C7F                   <1>     sub    al, 7Fh
                                <1>                ; 1 to 4
0000721E C0E002                 <1>     shl    al, 2 ; 4 to 16
00007221 2A05[7B7C0100]         <1>     sub    al, [PP_Counter] ; al - (4 - partition index)
                                <1>                    ; (disk number * 4) + partition index
                                <1>
                                <1>     ; AL = Partition entry/index, 0 based
                                <1>         ;  0 -> hd 0, Partition Table offset = 0
                                <1>         ; 15 -> hd 3, Partition Table offset = 3
                                <1>
                                <1>     ;mov   ah, 4 ; Logical dos partition (>= 4)
                                <1>     ;add   ah, [EP_Counter]
                                <1>            ; Logical disk partition index = 4 to 7
                                <1>            ; (Primary disk partition index = 0 to 3)
                                <1>
                                <1>     ; 13/08/2020
00007227 8A25[7D7C0100]         <1>     mov    ah, [LD_Counter] ; Logical drive index number
                                <1>                             ; (in current extended partition)
0000722D 80C404                 <1>     add    ah, 4 ; 4 to 7
                                <1>
                                <1>     ; 15/07/2020
                                <1>     ; CX -> AX
```

```
                                <1>     ;; 06/01/2016 (TRDOS v2.0)
                                <1>     ;; BUGFIX *
                                <1>     ;;mov   [esi+LD_PartitionEntry], cl
                                <1>     ;;mov   [esi+LD_DParamEntry], ch
                                <1>     ;mov    [esi+LD_PartitionEntry], cx
00007230 6689467C               <1>     mov     [esi+LD_PartitionEntry], ax
                                <1>
00007234 8A0D[8D2E0100]         <1>     mov     cl, [Last_DOS_DiskNo]
0000723A 80C141                 <1>     add     cl, 'A'
0000723D 880E                   <1>     mov     [esi+LD_Name], cl
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ;cmp    cl, 'Z'
                                <1>     ;jb     short logical_drive_count_ok_for_next
                                <1>     ;pop    esi ; ***
                                <1>     ;pop    esi ; ****
                                <1>     ;retn
                                <1>
                                <1> ;logical_drive_count_ok_for_next:
                                <1>
                                <1>     ;; 15/07/2020
                                <1>     ;inc    byte [EP_Counter]
                                <1>     ; 13/08/2020
0000723F FE05[7D7C0100]         <1>     inc     byte [LD_Counter]
                                <1>
                                <1>     ;mov    dl, [hdc]
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ;; Now,
                                <1>     ;; we are swimming in deep of an extended partition !!!
                                <1>     ; (! sub or chained extended partition tables !)
                                <1>     ; ((Logical dos partitions in extended partition were called
                                <1>     ;  as 'mini disk partition' in msdos 6.0 source code.))
                                <1>
                                <1> validate_next_minidisk_partition:
                                <1>     ; 13/08/2020
                                <1>     ;pop    esi ; *** ; Extended partition table offset
                                <1>
                                <1>     ; 17/07/2020
                                <1>     ;cmp    byte [EP_Counter], 4
                                <1>     ; 13/08/2020
00007245 803D[7D7C0100]04       <1>     cmp     byte [LD_Counter], 4 ; maximum 4 logical disks
                                <1>                                  ; per extended partition
                                <1>     ;jnb    continue_check_ep_next_disk
                                <1>     ; 25/07/2022
0000724C 7205                   <1>     jb      short validate_next_minidisk_partition_ok
0000724E E96FFFFFFF             <1>     jmp     continue_check_ep_next_disk
                                <1>
                                <1> validate_next_minidisk_partition_ok:
                                <1>     ; 13/08/2020
                                <1>     ;dec    byte [PP_Counter] ; 4 --> 0
                                <1>     ;jz     continue_check_ep_next_disk
                                <1>
                                <1>     ;cmp    esi, PartitionTable+64
                                <1>     ;jnb    continue_check_ep_next_disk
                                <1>
                                <1>     ;add    esi, 16
                                <1>     ; 13/08/2020
00007253 BE[447B0100]           <1>     mov     esi, PartitionTable+16
                                <1>
                                <1>     ; 20/07/2020
00007258 8A4604                 <1>     mov     al, [esi+ptFileSystemID]
                                <1>
                                <1>     ; 20/07/2020
0000725B 3C05                   <1>     cmp     al, 05h ; Is it an extended dos partition ?
0000725D 7409                   <1>     je      short loc_minidisk_next_ep_lba_chs ; 17/07/2020
0000725F 3C0F                   <1>     cmp     al, 0Fh ; Is it an extended win4 (LBA mode) partition ?
                                <1>     ;jne    continue_check_ep_next_disk ; AL must be 0 here
                                <1>                                         ; (when it is not 05h or 0Fh)
                                <1>                                         ; If AL is not ZERO -> EP Bug!
                                <1>                                         ; (!Microsoft DOS convention!)
                                <1>     ; 25/07/2022
00007261 7405                   <1>     je      short loc_minidisk_next_ep_lba_chs
00007263 E95AFFFFFF             <1>     jmp     continue_check_ep_next_disk
                                <1>
                                <1> loc_minidisk_next_ep_lba_chs:
                                <1>     ; 17/07/2020
00007268 8B4E08                 <1>     mov     ecx, [esi+ptStartSector] ; relative start sector number
                                <1>     ;add    ecx, [EP_StartSector]
                                <1>     ; 30/08/2020
0000726B 030D[7E7C0100]         <1>     add     ecx, [MBR_EP_StartSector]
                                <1>     ; 20/07/2020
00007271 E928FFFFFF             <1>     jmp     loc_validate_hde_partition_next
                                <1>
                                <1> validate_hd_fat_partition:
                                <1>     ; 17/07/2020
                                <1>     ; 15/07/2020
                                <1>     ;       (optimization)
                                <1>     ; 14/07/2020
                                <1>     ;       (fat16 -big- partition search bugfix)
                                <1>     ; 27/12/2017
                                <1>     ; 12/02/2016
                                <1>     ; 07/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 07/08/2011
                                <1>     ; 23/07/2011
                                <1>     ; Input
                                <1>     ;   DL = Hard/Fixed Disk Drive Number
                                <1>     ;   ESI = PartitionTable offset
                                <1>     ;   EDI = Extend. Part. Start Sector Pointer
                                <1>     ;   EDI = 0 -> Primary Partition
                                <1>     ;   byte [Last_DOS_DiskNo]
                                <1>     ; Output
                                <1>     ;  cf=0 -> Validated
                                <1>     ;   ESI = Logical dos drv desc. table
                                <1>     ;   EBX = FAT boot sector buffer
                                <1>     ;   byte [Last_DOS_DiskNo]
                                <1>     ;  cf=1 -> Not a valid FAT partition
                                <1>     ; EAX, EDX, ECX, EDI -> changed
                                <1>
                                <1>     ;mov    esi, PartitionTable
00007276 8A6604                 <1>     mov     ah, [esi+ptFileSystemID]
00007279 B002                   <1>     mov     al, 2 ; 27/12/2017
0000727B 80FC06                 <1>     cmp     ah, 06h ; FAT16 CHS partition (>=32MB)
                                <1>     ; 12/02/2016
                                <1>     ;;jb    short loc_not_a_valid_fat_partition2
                                <1>     ;jnb    short vhdp_FAT16_32
                                <1>     ; 14/07/2020 (BugFix)
0000727E 7711                   <1>     ja      short vhdp_FAT16_32
00007280 7425                   <1>     je      short loc_set_valid_hd_partition_params
                                <1>
                                <1> vhdp_FAT12_16:
                                <1>     ; 27/12/2017
00007282 FEC8                   <1>     dec     al ; mov al, 1
00007284 38C4                   <1>     cmp     ah, al ; 1 ; FAT12 partition
```

```
00007286 741F                <1>     je      short loc_set_valid_hd_partition_params
                             <1>     ;
00007288 FEC0                <1>     inc     al ; mov al, 2
0000728A 80FC04              <1>     cmp     ah, 04h ; FAT16 CHS partition (< 32MB)
0000728D 7418                <1>     je      short loc_set_valid_hd_partition_params
                             <1>
                             <1>     ; 15/07/2020
                             <1>     ; (ah = 05h, 02h or 03h)
                             <1> loc_not_a_valid_fat_partition1:
0000728F F9                  <1>     stc
                             <1>     ; cf=1
00007290 C3                  <1>     retn
                             <1>
                             <1> vhdp_FAT16_32:
                             <1>     ; 15/07/2020
                             <1>     ;mov    al, 3
00007291 FEC0                <1>     inc     al
00007293 80FC0C              <1>     cmp     ah, 0Ch ; FAT32 LBA partition
00007296 740F                <1>     je      short loc_set_valid_hd_partition_params
00007298 7706                <1>     ja      short vhdp_check_FAT16_lba
                             <1>
                             <1> vhdp_check_FAT32_chs:
0000729A 80FC0B              <1>     cmp     ah, 0Bh ; FAT32 CHS partition
0000729D 7408                <1>     je      short loc_set_valid_hd_partition_params
                             <1>     ;jne    short loc_not_a_valid_fat_partition1
                             <1>
                             <1>     ;stc
                             <1> loc_not_a_valid_fat_partition2:
0000729F C3                  <1>     retn
                             <1>
                             <1> vhdp_check_FAT16_lba:
000072A0 80FC0E              <1>     cmp     ah, 0Eh ; FAT16 LBA partition
000072A3 75EA                <1>     jne     short loc_not_a_valid_fat_partition1
                             <1>
                             <1>     ;mov    al, 2
000072A5 FEC8                <1>     dec     al
                             <1>
                             <1> loc_set_valid_hd_partition_params:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 30/07/2022
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/07/2020
                             <1>     ;inc    byte [Last_DOS_DiskNo] ; > 1
                             <1>     ;
000072A7 31DB                <1>     xor     ebx, ebx
000072A9 8A3D[8D2E0100]      <1>     mov     bh, [Last_DOS_DiskNo] ; * 256
000072AF FEC7                <1>     inc     bh ; 15/07/2020
000072B1 81C300010900        <1>     add     ebx, Logical_DOSDisks
                             <1>     ;
000072B7 C6430102            <1>     mov     byte [ebx+LD_DiskType], 2
000072BB 885302              <1>     mov     byte [ebx+LD_PhyDrvNo], dl
                             <1>     ;mov    byte [ebx+LD_FATType], al ; 2 or 3
                             <1>     ;mov    byte [ebx+LD_FSType], ah ; 06h, 0Eh, 0Bh, 0Ch
000072BE 66894303            <1>     mov     word [ebx+LD_FATType], ax
                             <1>     ;
000072C2 8B4E08              <1>     mov     ecx, [esi+ptStartSector]
000072C5 09FF                <1>     or      edi, edi
000072C7 7402                <1>     jz      short pass_hd_FAT_ep_start_sector_adding
                             <1> loc_add_hd_FAT_ep_start_sector:
                             <1>     ; 17/07/2020
000072C9 030F                <1>     add     ecx, [edi]
                             <1> pass_hd_FAT_ep_start_sector_adding:
000072CB 894B6C              <1>     mov     [ebx+LD_StartSector], ecx
                             <1> loc_hd_FAT_logical_drv_init:
000072CE 89DD                <1>     mov     ebp, ebx
                             <1>     ;mov    dl, [ebx+LD_PhyDrvNo]
000072D0 A0[767C0100]        <1>     mov     al, [HD_LBA_yes] ; 07/01/2016
000072D5 884305              <1>     mov     [ebx+LD_LBAYes], al
000072D8 BB[867C0100]        <1>     mov     ebx, DOSBootSectorBuff ; buffer address
000072DD 08C0                <1>     or      al, al
000072DF 740C                <1>     jz      short loc_hd_FAT_drv_init_load_bs_chs
                             <1> loc_hd_FAT_drv_init_load_bs_lba:
                             <1>     ; DL = Physical drive number
                             <1>     ;mov    ecx, [esi+ptStartSector] ; sector number
                             <1>     ;mov    ebx, DOSBootSectorBuff ; buffer address
                             <1>     ; LBA read/write (with private LBA function)
                             <1>     ;((Retro UNIX 386 v1 - DISK I/O code by Erdogan Tan))
                             <1>     ; dl = physical drive number (0,1, 80h, 81h, 82h, 83h)
000072E1 B41B                <1>     mov     ah, 1Bh ; LBA read
000072E3 B001                <1>     mov     al, 1 ; sector count
000072E5 E825DDFFFF          <1>     call    int13h
000072EA 7313                <1>     jnc     short loc_hd_drv_FAT_boot_validation
                             <1> loc_not_a_valid_fat_partition3:
000072EC C3                  <1>     retn
                             <1> loc_hd_FAT_drv_init_load_bs_chs:
000072ED 8A7601              <1>     mov     dh, [esi+ptBeginHead]
000072F0 668B4E02            <1>     mov     cx, [esi+ptBeginSector]
000072F4 66B80102            <1>     mov     ax, 0201h ; Read 1 sector
                             <1>     ;mov    ebx, DOSBootSectorBuff
000072F8 E812DDFFFF          <1>     call    int13h
000072FD 72ED                <1>     jc      short loc_not_a_valid_fat_partition3
                             <1> loc_hd_drv_FAT_boot_validation:
                             <1>     ;mov    esi, DOSBootSectorBuff
000072FF 89DE                <1>     mov     esi, ebx
00007301 6681BEFE01000055AA  <1>     cmp     word [esi+BS_Validation], 0AA55h
0000730A 751C                <1>     jne     short loc_not_a_valid_fat_partition4
0000730C 807E15F8            <1>     cmp     byte [esi+BPB_Media], 0F8h
00007310 7516                <1>     jne     short loc_not_a_valid_fat_partition4
                             <1>     ; 19/12/2025
00007312 66817E0B0002        <1>     cmp     word [esi+BPB_BytsPerSec], 512
00007318 750E                <1>     jne     short loc_not_a_valid_fat_partition4
                             <1>
                             <1>     ; 25/07/2022
0000731A 31C9                <1>     xor     ecx, ecx
                             <1>
                             <1>     ; 27/12/2017
0000731C 807D0303            <1>     cmp     byte [ebp+LD_FATType], 3
00007320 7508                <1>     jne     short loc_hd_FAT16_BPB
                             <1>
                             <1> loc_hd_drv_FAT32_boot_validation:
00007322 807E4229            <1>     cmp     byte [esi+BS_FAT32_BootSig], 29h
00007326 7413                <1>     je      short loc_hd_FAT32_BPB
                             <1>
                             <1> loc_not_a_valid_fat_partition4:
00007328 F9                  <1>     stc
00007329 C3                  <1>     retn
                             <1>
                             <1> loc_hd_FAT16_BPB:
0000732A 807E2629            <1>     cmp     byte [esi+BS_BootSig], 29h
0000732E 75F8                <1>     jne     short loc_not_a_valid_fat_partition4
                             <1>
00007330 66837E1600          <1>     cmp     word [esi+BPB_FATSz16], 0
00007335 7604                <1>     jna     short loc_hd_big_FAT16_BPB
                             <1>     ;mov    ecx, 32
```

```
                                <1>     ; 25/07/2022
00007337 B120                   <1>     mov     cl, 32
                                <1>     ; ecx = 32
00007339 EB02                   <1>     jmp     short loc_hd_move_FAT_BPB
                                <1>
                                <1> loc_hd_FAT32_BPB:
                                <1>     ;cmp    word [esi+BPB_FATSz16], 0
                                <1>     ;ja     short loc_not_a_valid_fat_partition4
                                <1> loc_hd_big_FAT16_BPB:
                                <1>     ;mov    ecx, 45
                                <1>     ; 25/07/2022
0000733B B12D                   <1>     mov     cl, 45
                                <1>     ; ecx = 45
                                <1> loc_hd_move_FAT_BPB:
0000733D 89EF                   <1>     mov     edi, ebp
                                <1>     ;mov    esi, ebx ; Boot sector
0000733F 57                     <1>     push    edi
00007340 83C706                 <1>     add     edi, LD_BPB
00007343 66F3A5                 <1>     rep     movsw
00007346 5E                     <1>     pop     esi
00007347 0FB74614               <1>     movzx   eax, word [esi+LD_BPB+BPB_RsvdSecCnt]
0000734B 03466C                 <1>     add     eax, [esi+LD_StartSector]
0000734E 894660                 <1>     mov     [esi+LD_FATBegin], eax
00007351 807E0303               <1>     cmp     byte [esi+LD_FATType], 3
00007355 7223                   <1>     jb      short loc_set_FAT16_RootDirLoc
                                <1> loc_set_FAT32_RootDirLoc:
00007357 8B462A                 <1>     mov     eax, [esi+LD_BPB+BPB_FATSz32]
0000735A 0FB65E16               <1>         movzx       ebx, byte [esi+LD_BPB+BPB_NumFATs]
0000735E F7E3                   <1>     mul     ebx
00007360 034660                 <1>     add     eax, [esi+LD_FATBegin]
                                <1> loc_set_FAT32_data_begin:
00007363 894668                 <1>     mov     [esi+LD_DATABegin], eax
00007366 894664                 <1>     mov     [esi+LD_ROOTBegin], eax
                                <1>     ; If Root Directory Cluster <> 2 then
                                <1>     ; change the beginning sector value
                                <1>     ; of the root dir by adding sector offset.
00007369 8B4632                 <1>     mov     eax, [esi+LD_BPB+BPB_RootClus]
                                <1>     ;sub    eax, 2
                                <1>     ; 30/07/2022
0000736C 48                     <1>     dec     eax ; 2 -> 1
0000736D 48                     <1>     dec     eax ; 1 -> 0
0000736E 7433                   <1>     jz      short short loc_set_32bit_FAT_total_sectors
                                <1>     ;movzx  ebx, byte [esi+LD_BPB+BPB_SecPerClust]
00007370 8A5E13                 <1>     mov     bl, [esi+LD_BPB+BPB_SecPerClust]
00007373 F7E3                   <1>     mul     ebx
00007375 014664                 <1>     add     [esi+LD_ROOTBegin], eax
00007378 EB29                   <1>     jmp     short loc_set_32bit_FAT_total_sectors
                                <1>     ;
                                <1> loc_set_FAT16_RootDirLoc:
0000737A 0FB64616               <1>     movzx   eax, byte [esi+LD_BPB+BPB_NumFATs]
0000737E 0FB7561C               <1>     movzx   edx, word [esi+LD_BPB+BPB_FATSz16]
00007382 F7E2                   <1>     mul     edx
00007384 034660                 <1>     add     eax, [esi+LD_FATBegin]
00007387 894664                 <1>     mov     [esi+LD_ROOTBegin], eax
                                <1> loc_set_FAT16_data_begin:
0000738A 894668                 <1>     mov     [esi+LD_DATABegin], eax
                                <1>     ;mov    eax, 20h  ; Size of a directory entry
                                <1>     ;;movzx edx, word [esi+LD_BPB+BPB_RootEntCnt]
                                <1>         ;mov       dx, [esi+LD_BPB+BPB_RootEntCnt]
                                <1>         ;mul       edx
                                <1>     ;;mov   ecx, 511
                                <1>     ;mov    cx, 511
                                <1>     ;add    eax, ecx
                                <1>     ;inc    ecx ; 512
                                <1>     ;div    ecx
                                <1>     ; 14/07/2020
0000738D 0FB74617               <1>     movzx   eax, word [esi+LD_BPB+BPB_RootEntCnt]
00007391 6683C00F               <1>     add     ax, 15
                                <1>     ;shr    ax, 4 ; / 16 ; (16 entries per sector)
                                <1>     ; 25/07/2022
00007395 C1E804                 <1>     shr     eax, 4
00007398 014668                 <1>     add     [esi+LD_DATABegin], eax
                                <1>     ;movzx  eax, word [esi+LD_BPB+BPB_TotalSec16]
0000739B 668B4619               <1>     mov     ax, [esi+LD_BPB+BPB_TotalSec16]
                                <1>     ;test   ax, ax
                                <1>     ; 25/07/2022
0000739F 85C0                   <1>     test    eax, eax
                                <1>     ;jz     short loc_set_32bit_FAT_total_sectors
                                <1> ;loc_set_16bit_FAT_total_sectors:
                                <1>     ;mov    [esi+LD_TotalSectors], eax
                                <1>     ;jmp    short loc_set_hd_FAT_cluster_count
                                <1>     ; 14/07/2020
000073A1 7503                   <1>     jnz     short loc_set_hd_FAT_cluster_count
                                <1> loc_set_32bit_FAT_total_sectors:
000073A3 8B4626                 <1>     mov     eax, [esi+LD_BPB+BPB_TotalSec32]
                                <1>     ;mov    [esi+LD_TotalSectors], eax
                                <1> loc_set_hd_FAT_cluster_count:
000073A6 894670                 <1>     mov     [esi+LD_TotalSectors], eax ; 14/07/2020
000073A9 8B5668                 <1>     mov     edx, [esi+LD_DATABegin]
000073AC 2B566C                 <1>     sub     edx, [esi+LD_StartSector]
000073AF 29D0                   <1>     sub     eax, edx
000073B1 31D2                   <1>     xor     edx, edx ; 0
000073B3 0FB64E13               <1>         movzx   ecx, byte [esi+LD_BPB+BPB_SecPerClust]
000073B7 F7F1                   <1>         div ecx
000073B9 894678                 <1>     mov     [esi+LD_Clusters], eax
                                <1>     ; Maximum Valid Cluster Number= EAX +1
                                <1>     ; with 2 reserved clusters= EAX +2
                                <1> loc_set_hd_FAT_fs_free_sectors:
                                <1>     ;mov    dword [esi+LD_FreeSectors], 0
000073BC E831000000             <1>     call    get_free_FAT_sectors
000073C1 720D                   <1>     jc      short loc_validate_hd_FAT_partition_retn
000073C3 894674                 <1>     mov     [esi+LD_FreeSectors], eax
000073C6 C6467E06               <1>     mov     byte [esi+LD_MediaChanged], 6  ; Volume Name Reset
                                <1>
                                <1>     ; 15/07/2020
000073CA FE05[8D2E0100]         <1>     inc     byte [Last_DOS_DiskNo] ; > 1
                                <1>
                                <1>     ;mov    cl, [Last_DOS_DiskNo]
                                <1>     ;add    cl, 'A'
                                <1>     ;mov    [esi+LD_FS_Name], cl
                                <1>
                                <1> loc_validate_hd_FAT_partition_retn:
000073D0 C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>
                                <1> validate_hd_fs_partition:
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 03/02/2018
                                <1>     ; 09/12/2017
                                <1>     ; 13/02/2016
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
```

```
<1>     ; 29/01/2011
<1>     ; 23/07/2011
<1>     ; Input
<1>     ;   DL = Hard/Fixed Disk Drive Number
<1>     ;   ESI = PartitionTable offset
<1>     ;   byte [Last_DOS_DiskNo]
<1>     ; Output
<1>     ;  cf=0 -> Validated
<1>     ;   ESI = Logical dos drv desc. table
<1>     ;   EBX = Singlix FS boot sector buffer
<1>     ;   byte [Last_DOS_DiskNo]
<1>     ;  cf=1 -> Not a valid 'Singlix FS' partition
<1>     ; EAX, EDX, ECX, EDI -> changed
<1>
<1>     ;mov    esi, PartitionTable
<1>     mov     ah, [esi+ptFileSystemID]
<1>     cmp     ah, 0A1h ; SINGLIX FS1 (trfs1) partition
<1>     jne     short loc_validate_hd_fs_partition_stc_retn
<1> loc_set_valid_hd_fs_partition_params:
<1>     inc     byte [Last_DOS_DiskNo] ; > 1
<1>     xor     al, al ; mov al, 0
<1>     ;mov    [drv], dl
<1>     sub     ebx, ebx ; 0
<1>     mov     bh, [Last_DOS_DiskNo]
<1>     add     ebx, Logical_DOSDisks
<1>     mov     byte [ebx+LD_DiskType], 2
<1>     mov     [ebx+LD_PhyDrvNo], dl
<1>     ;mov    [ebx+LD_FATType], al ; 0
<1>     ;mov    [ebx+LD_FSType], ah
<1>     mov     [ebx+LD_FATType], ax
<1>     ;mov    eax, [esi+ptStartSector]
<1>     ;mov    [ebx+LD_StartSector], eax
<1> loc_hd_fs_logical_drv_init:
<1>     mov     ebp, ebx ; 10/01/2016
<1>     ;mov    dl, [ebx+LD_PhyDrvNo]
<1>     mov     al, [HD_LBA_yes] ; 10/01/2016
<1>     mov     [ebx+LD_LBAYes], al
<1>     mov     esi, ebx
<1>     mov     ebx, DOSBootSectorBuff ; buffer address
<1>     or      al, al
<1>     jnz     short loc_hd_fs_drv_init_load_bs_lba
<1> loc_hd_fs_drv_init_load_bs_chs:
<1>     mov     dh, [esi+ptBeginHead]
<1>     mov     cx, [esi+ptBeginSector]
<1>     mov     ax, 0201h ; Read 1 sector
<1>     ;mov    ebx, DOSBootSectorBuff
<1>     call    int13h
<1>     jnc     short loc_hd_drv_fs_boot_validation
<1> loc_validate_hd_fs_partition_err_retn:
<1>     retn
<1> loc_validate_hd_fs_partition_stc_retn:
<1>     stc
<1>     retn
<1> loc_hd_fs_drv_init_load_bs_lba:
<1>     ; DL = Physical drive number
<1>     ;mov    esi, ebx
<1>     mov     ecx, [esi+ptStartSector] ; sector number
<1>     ;mov    ebx, DOSBootSectorBuff ; buffer address
<1>     ; LBA read/write (with private LBA function)
<1>     ;((Retro UNIX 386 v1 - DISK I/O code by Erdogan Tan))
<1>     ; dl = physical drive number (0,1, 80h, 81h, 82h, 83h)
<1>     mov     ah, 1Bh ; LBA read
<1>     mov     al, 1 ; sector count
<1>     call    int13h
<1>     jc      short loc_validate_hd_fs_partition_err_retn
<1> loc_hd_drv_fs_boot_validation:
<1>     ;mov    esi, DOSBootSectorBuff
<1>     mov     esi, ebx ; Boot sector buffer
<1>     cmp     word [esi+BS_Validation], 0AA55h
<1>     jne     short loc_validate_hd_fs_partition_stc_retn
<1>         ;
<1>     ;Singlix FS Extensions to TR-DOS (7/6/2009)
<1>     cmp     word [esi+bs_FS_Identifier], 'FS' ; 03/02/2018
<1>     jne     short loc_validate_hd_fs_partition_stc_retn
<1>         ;'A1h' check is not necessary
<1>     ;  if 'FS' check is passed as OK/Yes.
<1>     cmp     byte [esi+bs_FS_PartitionID], 0A1h
<1>     jne     short loc_validate_hd_fs_partition_stc_retn
<1>     ;
<1>     mov     edi, ebp ; 10/01/2016
<1>     ;
<1>     mov     al, byte [esi+bs_FS_LBA_Ready]
<1>     mov     [edi+LD_FS_LBAYes], al
<1>     ;
<1>     ; 03/01/2010 CHS -> DOS FAT/BPB compatibility fix
<1>     mov     al, [esi+bs_FS_MediaAttrib]
<1>     mov     byte [edi+LD_FS_MediaAttrib], al
<1>     ;
<1>     mov     al, [esi+bs_FS_VersionMaj]
<1>     mov     [edi+LD_FS_VersionMajor], al
<1>     ;
<1>     mov     ax, [esi+bs_FS_BytesPerSec]
<1>     mov     [edi+LD_FS_BytesPerSec], ax
<1>     mov     al, [esi+bs_FS_SecPerTrack]
<1>     xor     ah, ah ; 09/12/2017
<1>     mov     [edi+LD_FS_SecPerTrack], ax
<1>     mov     al, [esi+bs_FS_Heads]
<1>     mov     [edi+LD_FS_NumHeads], ax
<1>     ;
<1>     mov     eax, [esi+bs_FS_UnDelDirD]
<1>     mov     [edi+LD_FS_UnDelDirD], eax
<1>     mov     edx, [esi+bs_FS_MATLocation]
<1>     mov     [edi+LD_FS_MATLocation], edx
<1>     mov     eax, [esi+bs_FS_RootDirD]
<1>     mov     [edi+LD_FS_RootDirD], eax
<1>     mov     eax, [esi+bs_FS_BeginSector]
<1>     mov     [edi+LD_FS_BeginSector], eax
<1>     mov     eax, [edi+bs_FS_VolumeSize]
<1>     mov     [edi+LD_FS_VolumeSize], eax
<1>     ;
<1>     mov     eax, edx ; [edi+LD_FS_MATLocation]
<1>     add     eax, [edi+LD_FS_BeginSector]
<1>     mov     esi, edi
<1> mread_hd_fs_MAT_sector:
<1>         ;mov       ebx, DOSBootSectorBuff
<1>     ;mov    ecx, 1
<1>     ; 25/07/2022
<1>     sub     ecx, ecx
<1>     inc     cl
<1>     ; ecx = 1
<1>     call    disk_read
<1>     jc      short loc_validate_hd_fs_partition_retn
<1>     ; EDI will not be changed
<1>     mov     esi, ebx
```

```
                                <1> use_hdfs_mat_sector_params:
                                <1>     mov     eax, [esi+FS_MAT_DATLocation]
                                <1>     mov     [edi+LD_FS_DATLocation], eax
                                <1>     mov     eax, [esi+FS_MAT_DATScount]
                                <1>     mov     [edi+LD_FS_DATSectors], eax
                                <1>     mov     eax, [esi+FS_MAT_FreeSectors]
                                <1>         mov     [edi+LD_FS_FreeSectors], eax
                                <1>     mov     eax, [esi+FS_MAT_FirstFreeSector]
                                <1>     mov     [edi+LD_FS_FirstFreeSector], eax
                                <1>     mov     eax, [edi+LD_FS_RootDirD]
                                <1>     add     eax, [edi+LD_FS_BeginSector]
                                <1>     mov     esi, edi
                                <1> read_hd_fs_RDT_sector:
                                <1>     mov     ebx, DOSBootSectorBuff
                                <1>     ;mov    ecx, 1
                                <1>     mov     cl, 1
                                <1>     call    disk_read
                                <1>     jc      short loc_validate_hd_fs_partition_retn
                                <1>     ; EDI will not be changed
                                <1>     mov     esi, ebx
                                <1> use_hdfs_RDT_sector_params:
                                <1>     mov     eax, [esi+FS_RDT_VolumeSerialNo]
                                <1>     mov     [edi+LD_FS_VolumeSerial], eax
                                <1>     push    edi
                                <1>     ;mov    ecx, 16
                                <1>     mov     cl, 16
                                <1>     add     esi, FS_RDT_VolumeName
                                <1>     add     edi, LD_FS_VolumeName
                                <1>     rep     movsd ; 64 bytes
                                <1>     pop     esi
                                <1>             ; Volume Name Reset
                                <1>         mov     byte [esi+LD_FS_MediaChanged], 6
                                <1>     ;
                                <1>         ;mov       cl, [Last_DOS_DiskNo]
                                <1>     ;add    cl, 'A'
                                <1>     ;mov    [esi+LD_FS_Name], cl
                                <1>
                                <1> loc_validate_hd_fs_partition_retn:
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> load_masterboot:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 14/07/2020 (Reset function has been removed)
                                <1>     ;
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 2005 - 2011
                                <1>     ; input -> DL = drive number
                                <1> ;   mov     ah, 0Dh ; Alternate disk reset
                                <1> ;   call    int13h
                                <1> ;   jnc     short pass_reset_error
                                <1> ;harddisk_error:
                                <1> ;   retn
                                <1> ;pass_reset_error:
000073D1 BB[76790100]           <1>     mov     ebx, MasterBootBuff
                                <1>     ;mov    ax, 0201h
                                <1>     ;mov    cx, 1
                                <1>     ; 25/07/2022
                                <1>     ;xor    ecx, ecx
                                <1>     ;inc    cl
000073D6 B101                   <1>     mov     cl, 1
                                <1>     ; ecx = 1
000073D8 89C8                   <1>     mov     eax, ecx ; ch = cylinder = 0
000073DA B402                   <1>     mov     ah, 2  ; chs read
                                <1>     ; eax = 0201h
000073DC 30F6                   <1>     xor     dh, dh ; head = 0
000073DE E82CDCFFFF             <1>     call    int13h
000073E3 720C                   <1>     jc      short harddisk_error
                                <1>     ;
000073E5 66813D[747B0100]55-    <1>     cmp     word [MBIDCode], 0AA55h
000073ED AA                     <1>
000073EE 7401                   <1>     je      short load_masterboot_ok
000073F0 F9                     <1>     stc
                                <1> harddisk_error:
                                <1> load_masterboot_ok:
000073F1 C3                     <1>     retn
                                <1>
                                <1> get_free_FAT_sectors:
                                <1>     ; 29/08/2023
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 21/12/2017
                                <1>     ; 29/02/2016
                                <1>     ; 13/02/2016
                                <1>     ; 04/02/2016
                                <1>     ; 07/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 11/07/2010
                                <1>     ; 21/06/2009
                                <1>     ; INPUT: ESI = Logical DOS Drive Description Table address
                                <1>     ; OUTPUT: STC => Error
                                <1>         ;  cf = 0 and EAX = Free FAT sectors
                                <1>     ; Also, related parameters and FAT buffer will be reset and updated
                                <1>
000073F2 31C0                   <1>     xor     eax, eax
                                <1>     ;mov    [esi+LD_FreeSectors], eax ; Reset
                                <1>
000073F4 807E0302               <1>         cmp     byte [esi+LD_FATType], 2
000073F8 7653                   <1>     jna     short loc_gfc_get_fat_free_clusters
                                <1>
                                <1>     ; 29/02/2016
000073FA 48                     <1>     dec     eax ; 0FFFFFFFFh
000073FB 89463A                 <1>     mov     [esi+LD_BPB+BPB_Reserved], eax ; Free cluster count (reset)
000073FE 89463E                 <1>     mov     [esi+LD_BPB+BPB_Reserved+4], eax ; First Free Cluster (reset)
00007401 40                     <1>     inc     eax ; 0
                                <1>     ;
00007402 668B4636               <1>     mov     ax, [esi+LD_BPB+BPB_FSInfo]
00007406 03466C                 <1>     add     eax, [esi+LD_StartSector]
                                <1>
00007409 BB[867C0100]           <1>     mov     ebx, DOSBootSectorBuff
                                <1>     ;mov    ecx, 1
                                <1>     ; 25/07/2022
0000740E 31C9                   <1>     xor     ecx, ecx
00007410 FEC1                   <1>     inc     cl
                                <1>     ; ecx = 1
00007412 E8DCA50000             <1>     call    disk_read
00007417 7301                   <1>     jnc     short loc_gfc_check_fsinfo_signs
                                <1> retn_gfc_get_fsinfo_sec:
00007419 C3                     <1>     retn
                                <1>
                                <1> loc_gfc_check_fsinfo_signs:
0000741A BB[867C0100]           <1>     mov     ebx, DOSBootSectorBuff ; 13/02/2016
0000741F 813B52526141           <1>         cmp     dword [ebx], 41615252h
00007425 7524                   <1>     jne     short retn_gfc_get_fsinfo_stc
                                <1>     ;add    ebx, 484
```

```
                                <1>     ;cmp    dword [ebx], 61417272h
00007427 81BBE4010000727241-    <1>     cmp     dword [ebx+484], 61417272h
00007430 61                     <1>
00007431 7518                   <1>     jne     short retn_gfc_get_fsinfo_stc
                                <1>     ;add    ebx, 4
                                <1>     ;mov    eax, [ebx]
00007433 8B83E8010000           <1>     mov     eax, [ebx+488]
                                <1>     ; 29/02/2016
00007439 89463A                 <1>     mov     [esi+LD_BPB+BPB_Reserved], eax ; Free cluster count
0000743C 8B93EC010000           <1>     mov     edx, [ebx+492]
                                <1>     ; 29/08/2023 (BugFix)
00007442 89563E                 <1>     mov     [esi+LD_BPB+BPB_Reserved+4], edx ; First Free Cluster
                                <1>     ; 21/12/2017
00007445 89C3                   <1>     mov     ebx, eax ; (initial value = 0FFFFFFFFh)
00007447 43                     <1>     inc     ebx ; 0FFFFFFFFh -> 0
00007448 7513                   <1>     jnz     short short retn_from_get_free_fat32_clusters
0000744A C3                     <1>     retn
                                <1>
                                <1> retn_gfc_get_fsinfo_stc:
0000744B F9                     <1>     stc
0000744C C3                     <1>     retn
                                <1>
                                <1> loc_gfc_get_fat_free_clusters:
                                <1>     ;mov    eax, 2
0000744D B002                   <1>     mov     al, 2
                                <1>     ;mov    [FAT_CurrentCluster], eax
                                <1> loc_gfc_loop_get_next_cluster:
0000744F E8354B0000             <1>     call    get_next_cluster
00007454 730E                   <1>     jnc     short loc_gfc_free_fat_clusters_cont
00007456 21C0                   <1>     and     eax, eax
00007458 7411                   <1>     jz      short loc_gfc_pass_inc_free_cluster_count
                                <1>
                                <1> retn_from_get_free_fat_clusters:
0000745A 8B4674                 <1>     mov     eax, [esi+LD_FreeSectors] ; Free clusters !
                                <1> retn_from_get_free_fat32_clusters:
0000745D 0FB65E13               <1>         movzx      ebx, byte [esi+LD_BPB+BPB_SecPerClust]
00007461 F7E3                   <1>         mul     ebx
                                <1>     ;mov    [esi+LD_FreeSectors], eax ; Free sectors
                                <1> retn_get_free_sectors_calc:
00007463 C3                     <1>     retn
                                <1>
                                <1> loc_gfc_free_fat_clusters_cont:
00007464 09C0                   <1>     or      eax, eax
00007466 7503                   <1>     jnz     short loc_gfc_pass_inc_free_cluster_count
00007468 FF4674                 <1>     inc     dword [esi+LD_FreeSectors] ; Free clusters !
                                <1>
                                <1> loc_gfc_pass_inc_free_cluster_count:
                                <1>     ;mov    eax, [FAT_CurrentCluster]
0000746B 89C8                   <1>     mov     eax, ecx ; [FAT_CurrentCluster]
0000746D 3B4678                 <1>     cmp     eax, [esi+LD_Clusters]
00007470 77E8                   <1>     ja      short retn_from_get_free_fat_clusters
00007472 40                     <1>     inc     eax
                                <1>     ;mov    [FAT_CurrentCluster], eax
00007473 EBDA                   <1>     jmp     short loc_gfc_loop_get_next_cluster
                                <1>
                                <1> floppy_drv_init:
                                <1>     ; 19/12/2025 (TRDOS 386 Kenrel v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 09/12/2017
                                <1>     ; 06/07/2016
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 24/07/2011
                                <1>     ; 04/07/2009
                                <1>     ; INPUT ->
                                <1>     ;       DL = Drive number (0,1)
                                <1>     ; OUTPUT ->
                                <1>     ;       BL = drive name
                                <1>     ;       BH = drive number
                                <1>     ;       ESI = Logical DOS drv description table
                                <1>     ;       EAX = Volume serial number
                                <1>
00007475 BE[8C660000]           <1>     mov     esi, fd0_type ; 10/01/2016
0000747A BF00010900             <1>     mov     edi, Logical_DOSDisks
0000747F 08D2                   <1>     or      dl, dl
00007481 7407                   <1>     jz      short loc_drv_init_fd0_fd1
00007483 81C700010000           <1>     add     edi, 100h
00007489 46                     <1>     inc     esi ; fd1_type ; 10/01/2016
                                <1> loc_drv_init_fd0_fd1:
0000748A C6477E00               <1>     mov     byte [edi+LD_MediaChanged], 0
0000748E 803E01                 <1>     cmp     byte [esi], 1 ; type (>0 if it is existing)
                                <1>             ; 4 = 1.44 MB, 80 track, 3 1/2"
00007491 7221                   <1>     jb      short read_fd_boot_sector_retn
00007493 885702                 <1>     mov     [edi+LD_PhyDrvNo], dl
                                <1> read_fd_boot_sector:
00007496 30F6                   <1>     xor     dh, dh
00007498 B904000000             <1>     mov     ecx, 4 ; Retry Count
                                <1> read_fd_boot_sector_again:
0000749D 51                     <1>     push    ecx
                                <1>     ;mov    cx, 1
0000749E B101                   <1>     mov     cl, 1
000074A0 66B80102               <1>     mov     ax, 0201h ; Read 1 sector
000074A4 BB[867C0100]           <1>     mov     ebx, DOSBootSectorBuff
000074A9 E861DBFFFF             <1>     call    int13h
000074AE 59                     <1>     pop     ecx
000074AF 7304                   <1>     jnc     short use_fd_boot_sector_params
000074B1 E2EA                   <1>     loop    read_fd_boot_sector_again
                                <1>
                                <1> read_fd_boot_sector_stc_retn:
000074B3 F9                     <1>     stc
                                <1> read_fd_boot_sector_retn:
000074B4 C3                     <1>     retn
                                <1>
                                <1> use_fd_boot_sector_params:
                                <1>     ;mov    esi, DOSBootSectorBuff
000074B5 89DE                   <1>     mov     esi, ebx
000074B7 6681BEFE01000055AA     <1>     cmp     word [esi+BS_Validation], 0AA55h
000074C0 75F1                   <1>     jne     short read_fd_boot_sector_stc_retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>         cmp     word [esi+bs_FS_Identifier], 'SF'
                                <1>     ;jne    use_fd_fatfs_boot_sector_params
                                <1>     ; 25/07/2022
                                <1>     je      short use_fdfs_boot_sector_params
                                <1>     jmp     use_fd_fatfs_boot_sector_params
                                <1> use_fdfs_boot_sector_params:
                                <1>     mov     al, [esi+bs_FS_LBA_Ready]
                                <1>     mov     [edi+LD_FS_LBAYes], al
                                <1>     ;
                                <1>     ; 03/01/2010 CHS -> DOS FAT/BPB compatibility fix
                                <1>     mov     al, [esi+bs_FS_MediaAttrib]
                                <1>     mov     [edi+LD_FS_MediaAttrib], al
                                <1>     ;
                                <1>         mov al, [esi+bs_FS_VersionMaj]
```

```
                                <1>     mov     byte [edi+LD_FS_VersionMajor], al
                                <1>     mov     ax, [esi+bs_FS_BytesPerSec]
                                <1>     mov     [edi+LD_FS_BytesPerSec], ax
                                <1>     mov     al, [esi+bs_FS_SecPerTrack]
                                <1>     sub     ah, ah ; 09/12/2017
                                <1>     mov     [edi+LD_FS_SecPerTrack], ax
                                <1>     mov     al, [esi+bs_FS_Heads]
                                <1>     mov     [edi+LD_FS_NumHeads], ax
                                <1>     ;
                                <1>     mov     eax, [esi+bs_FS_UnDelDirD]
                                <1>     mov     [edi+LD_FS_UnDelDirD], eax
                                <1>     mov     eax, [esi+bs_FS_MATLocation]
                                <1>     mov     [edi+LD_FS_MATLocation], eax
                                <1>     mov     eax, [esi+bs_FS_RootDirD]
                                <1>     mov     [edi+LD_FS_RootDirD], eax
                                <1>     mov     eax, [esi+bs_FS_BeginSector]
                                <1>     mov     [edi+LD_FS_BeginSector], eax
                                <1>     mov     eax, [esi+bs_FS_VolumeSize]
                                <1>     mov     [edi+LD_FS_VolumeSize], eax
                                <1>     ;
                                <1>     mov     esi, edi
                                <1>     mov     eax, [esi+LD_FS_MATLocation]
                                <1>     ;add    eax, [edi+LD_FS_BeginSector]
                                <1> read_fd_MAT_sector_again:
                                <1>     ;mov    ebx, DOSBootSectorBuff
                                <1>     ;mov    ecx, 1
                                <1>     mov     cl, 1
                                <1>     call    chs_read
                                <1>     mov     esi, ebx
                                <1>     ;jnc    short use_fdfs_mat_sector_params
                                <1>     ;jmp    short read_fd_boot_sector_retn
                                <1>     ;retn
                                <1>     ; 25/07/2022
                                <1>     jc      short read_fd_RDT_sector_retn
                                <1> use_fdfs_mat_sector_params:
                                <1>     mov     eax, [esi+FS_MAT_DATLocation]
                                <1>     mov     [edi+LD_FS_DATLocation], eax
                                <1>     mov     eax, [esi+FS_MAT_DATScount]
                                <1>     mov     [edi+LD_FS_DATSectors], eax
                                <1>     mov     eax, [edi+FS_MAT_FreeSectors]
                                <1>     mov     [edi+LD_FS_FreeSectors], eax
                                <1>     mov     eax, [esi+FS_MAT_FirstFreeSector]
                                <1>     mov     [edi+LD_FS_FirstFreeSector], eax
                                <1>     ;
                                <1>     mov     esi, edi
                                <1>     mov     eax, [esi+LD_FS_RootDirD]
                                <1> read_fd_RDT_sector_again:
                                <1>     ;mov    ebx, DOSBootSectorBuff
                                <1>     ;mov    cx, 1
                                <1>     mov     cl, 1
                                <1>     call    chs_read
                                <1>     mov     esi, ebx
                                <1>     jc      short read_fd_RDT_sector_retn
                                <1> use_fdfs_RDT_sector_params:
                                <1>     mov     eax, [esi+FS_RDT_VolumeSerialNo]
                                <1>     mov     [edi+LD_FS_VolumeSerial], eax
                                <1>     push    edi
                                <1>     ;mov    ecx, 16
                                <1>     mov     cl, 16
                                <1>     add     esi, FS_RDT_VolumeName
                                <1>     add     edi, LD_FS_VolumeName
                                <1>     rep     movsd ; 64 bytes
                                <1>     pop     esi
                                <1>     mov     byte [esi+LD_FATType], 0
                                <1>     mov     byte [esi+LD_FSType], 0A1h
                                <1>         jmp     loc_cont_use_fd_boot_sector_params
                                <1>
                                <1> read_fd_RDT_sector_stc_retn:
                                <1>     stc
                                <1> read_fd_RDT_sector_retn:
                                <1>     retn
                                <1> %endif
                                <1>
                                <1> use_fd_fatfs_boot_sector_params:
000074C2 807E2629               <1>     cmp     byte [esi+BS_BootSig], 29h
                                <1>     ;jne    short read_fd_RDT_sector_stc_retn
                                <1>     ; 19/12/2025
000074C6 75EB                   <1>     jne     short read_fd_boot_sector_stc_retn
000074C8 807E15F0               <1>     cmp     byte [esi+BPB_Media], 0F0h
                                <1>     ;jb     short read_fd_RDT_sector_retn
                                <1>     ; 19/12/2025
000074CC 72E6                   <1>     jb      short read_fd_boot_sector_retn
000074CE 57                     <1>     push    edi
000074CF 83C706                 <1>     add     edi, LD_BPB
                                <1>     ;mov    ecx, 16
000074D2 B110                   <1>     mov     cl, 16
000074D4 F3A5                   <1>     rep     movsd ; 64 bytes
000074D6 5E                     <1>     pop     esi
000074D7 31C0                   <1>     xor     eax, eax
000074D9 89466C                 <1>     mov     [esi+LD_StartSector], eax ; 0
000074DC 668B461C               <1>     mov     ax, [esi+LD_BPB+BPB_FATSz16]
000074E0 8A4E16                 <1>     mov     cl, [esi+LD_BPB+BPB_NumFATs]
000074E3 F7E1                   <1>     mul     ecx
                                <1>     ; edx = 0 !
000074E5 668B5614               <1>     mov     dx, [esi+LD_BPB+BPB_RsvdSecCnt]
000074E9 66895660               <1>     mov     [esi+LD_FATBegin], dx
                                <1>     ; 25/07/2022
000074ED 01D0                   <1>     add     eax, edx
                                <1>     ;add    ax, dx
000074EF 894664                 <1>     mov     [esi+LD_ROOTBegin], eax
000074F2 894668                 <1>     mov     [esi+LD_DATABegin], eax
000074F5 668B5617               <1>     mov     dx, [esi+LD_BPB+BPB_RootEntCnt]
                                <1>     ;;shl   edx, 5 ; * 32 (Size of a directory entry)
                                <1>     ;shl    dx, 5
                                <1>     ;;add   edx, 511
                                <1>     ;add    dx, 511
                                <1>     ;;shr   edx, 9 ; edx = ((edx*32)+511) / 512
                                <1>     ;shr    dx, 9
000074F9 6683C20F               <1>     add     dx, 15 ; 06/07/2016 (+(512/32)-1)
                                <1>     ;shr    dx, 4 ; / 16 (==16 entries per sector)
                                <1>     ; 25/07/2022
000074FD C1EA04                 <1>     shr     edx, 4
00007500 015668                 <1>     add     [esi+LD_DATABegin], edx ; + rd sectors
                                <1>     ;movzx  eax, word [esi+LD_BPB+BPB_TotalSec16]
00007503 668B4619               <1>     mov     ax, [esi+LD_BPB+BPB_TotalSec16]
00007507 894670                 <1>     mov     [esi+LD_TotalSectors], eax
0000750A 2B4668                 <1>     sub     eax, [esi+LD_DATABegin]
                                <1>     ;movzx  ecx, byte [esi+LD_BPB+BPB_SecPerClust]
0000750D 8A4E13                 <1>     mov     cl, [esi+LD_BPB+BPB_SecPerClust]
00007510 80F901                 <1>     cmp     cl, 1
00007513 7604                   <1>     jna     short save_fd_fatfs_cluster_count
                                <1>     ; 25/07/2022
00007515 29D2                   <1>     sub     edx, edx
                                <1>     ;sub    dx, dx ; 0
```

```
                                <1>     ;sub    dl, dl ; 06/07/2016
00007517 F7F1                   <1>     div     ecx
                                <1> save_fd_fatfs_cluster_count:
00007519 894678                 <1>     mov     [esi+LD_Clusters], eax
                                <1>
                                <1>       ; Maximum Valid Cluster Number = EAX +1
                                <1>       ; with 2 reserved clusters = EAX +2
                                <1>
                                <1> reset_FAT_buffer_decriptors:
0000751C 29C0                   <1>     sub     eax, eax ; 0
0000751E A2[8A7E0100]           <1>     mov     [FAT_BuffValidData], al ; 0
00007523 A2[8B7E0100]           <1>     mov     [FAT_BuffDrvName], al ; 0
00007528 A3[8E7E0100]           <1>     mov     [FAT_BuffSector], eax ; 0
                                <1>
                                <1> read_fd_FAT_sectors:
0000752D BB001C0900             <1>     mov     ebx, FAT_Buffer
00007532 668B4614               <1>     mov     ax, [esi+LD_BPB+BPB_RsvdSecCnt]
                                <1>     ;mov    ecx, 3
00007536 B103                   <1>     mov     cl, 3 ; 3 sectors
00007538 E8BCA40000             <1>     call    chs_read
0000753D 7240                   <1>     jc      short read_fd_FAT_sectors_retn
                                <1> use_fd_FAT_sectors:
0000753F 8A4602                 <1>     mov     al, [esi+LD_PhyDrvNo]
00007542 0441                   <1>     add     al, 'A'
00007544 A2[8B7E0100]           <1>     mov     [FAT_BuffDrvName], al
00007549 C605[8A7E0100]01       <1>     mov     byte [FAT_BuffValidData], 1
00007550 E82B000000             <1>     call    fd_init_calculate_free_clusters
00007555 7228                   <1>     jc      short read_fd_FAT_sectors_retn
                                <1>
                                <1> loc_use_fd_boot_sector_params_FAT:
00007557 C6460301               <1>     mov     byte [esi+LD_FATType], 1 ; FAT 12
0000755B C6460401               <1>     mov     byte [esi+LD_FSType], 1
0000755F 8B462D                 <1>         mov     eax, [esi+LD_BPB+VolumeID]
                                <1> loc_cont_use_fd_boot_sector_params:
00007562 8A7E02                 <1>     mov     bh, [esi+LD_PhyDrvNo]
00007565 887E7D                 <1>     mov     [esi+LD_DParamEntry], bh
00007568 88FB                   <1>     mov     bl, bh
0000756A 80C341                 <1>     add     bl, 'A'
0000756D 881E                   <1>     mov     byte [esi+LD_Name], bl
0000756F C6460101               <1>     mov     byte [esi+LD_DiskType], 1
00007573 C6460500               <1>     mov     byte [esi+LD_LBAYes], 0
00007577 C6467C00               <1>     mov     byte [esi+LD_PartitionEntry], 0
0000757B C6467E06               <1>     mov     byte [esi+LD_MediaChanged], 6 ; Volume Name Reset
                                <1>
                                <1> read_fd_FAT_sectors_retn:
0000757F C3                     <1>     retn
                                <1>
                                <1> fd_init_calculate_free_clusters:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 09/12/2017
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 04/07/2009
                                <1>     ; INPUT ->
                                <1>     ;     ESI = Logical DOS drive description table address
                                <1>     ; OUTPUT ->
                                <1>     ;    [ESI+LD_FreeSectors] will be set
                                <1>
00007580 29C0                   <1>     sub     eax, eax
00007582 894674                 <1>     mov     [esi+LD_FreeSectors], eax ; 0
00007585 B002                   <1>     mov     al, 2 ; eax = 2
                                <1>
                                <1> fd_init_loop_get_next_cluster:
00007587 E828000000             <1>     call    fd_init_get_next_cluster
0000758C 7225                   <1>     jc      short fd_init_calculate_free_clusters_retn
                                <1>
                                <1> fd_init_free_fat_clusters:
                                <1>     ;cmp    eax, 0
                                <1>     ;ja     short fd_init_pass_inc_free_cluster_count
                                <1>     ;and    eax, eax
                                <1>     ;jnz    short fd_init_pass_inc_free_cluster_count
                                <1>     ;and    ax, ax
0000758E 21C0                   <1>     and     eax, eax ; 25/07/2022
00007590 7503                   <1>     jnz     short fd_init_pass_inc_free_cluster_count
                                <1>     ; 25/07/2022
00007592 FF4674                 <1>     inc     dword [esi+LD_FreeSectors]
                                <1>         ;inc       word [esi+LD_FreeSectors]
                                <1>
                                <1> fd_init_pass_inc_free_cluster_count:
                                <1>     ; 25/07/2022
00007595 A1[867E0100]           <1>     mov     eax, [FAT_CurrentCluster]
                                <1>     ;mov    ax, [FAT_CurrentCluster]
0000759A 3B4678                 <1>     cmp     eax, [esi+LD_Clusters]
                                <1>     ;cmp    ax, [esi+LD_Clusters]
0000759D 7703                   <1>     ja      short short retn_from_fd_init_calculate_free_clusters
0000759F 40                     <1>     inc     eax
                                <1>     ;inc    ax
000075A0 EBE5                   <1>     jmp     short fd_init_loop_get_next_cluster
                                <1>
                                <1> retn_from_fd_init_calculate_free_clusters:
                                <1>     ; 25/07/2022
                                <1>     ;xor    eax, eax
000075A2 30E4                   <1>     xor     ah, ah
000075A4 8A4613                 <1>     mov     al, [esi+LD_BPB+BPB_SecPerClust]
000075A7 3C01                   <1>     cmp     al, 1
000075A9 7608                   <1>     jna     short fd_init_calculate_free_clusters_retn
                                <1>     ;;movzx eax, al
                                <1>     ;xor    ah, ah ; 09/12/2017
                                <1>     ; 25/07/2022
000075AB 8B4E74                 <1>     mov     ecx, [esi+LD_FreeSectors]
                                <1>     ;mov    cx, [esi+LD_FreeSectors] ; Count of free clusters
000075AE F7E1                   <1>     mul     ecx
                                <1>     ;mul    cx
000075B0 894674                 <1>     mov     [esi+LD_FreeSectors], eax
                                <1>     ;mov    [esi+LD_FreeSectors], ax
                                <1> fd_init_calculate_free_clusters_retn:
000075B3 C3                     <1>     retn
                                <1>
                                <1> fd_init_get_next_cluster:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 04/02/2016
                                <1>     ; 02/02/2016
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 04/07/2009
                                <1>     ; INPUT ->
                                <1>     ;    EAX = Current cluster
                                <1>     ;    ESI = Logical DOS drive description table address
                                <1>     ;    EDX = 0
                                <1>     ; OUTPUT ->
                                <1>     ;    EAX = Next cluster
                                <1>
000075B4 A3[867E0100]           <1>     mov     [FAT_CurrentCluster], eax
                                <1> fd_init_get_next_cluster_readnext:
000075B9 29D2                   <1>     sub     edx, edx ; 0
000075BB BB00040000             <1>     mov     ebx, 1024 ; 400h
```

```
000075C0 F7F3                <1>     div     ebx
                             <1>     ; EAX = Count of 3 FAT sectors
                             <1>     ; EDX = Buffer entry index
000075C2 89C1                <1>     mov     ecx, eax
                             <1>     ;mov    eax, 3
000075C4 B003                <1>     mov     al, 3
000075C6 F7E2                <1>     mul     edx ; Multiply by 3
                             <1>     ;shr    ax, 1 ; Divide by 2
                             <1>     ; 25/07/2022
000075C8 D1E8                <1>     shr     eax, 1
000075CA 89C3                <1>     mov     ebx, eax ; Buffer byte offset
000075CC 81C3001C0900        <1>     add     ebx, FAT_Buffer
000075D2 89C8                <1>     mov     eax, ecx
                             <1>     ;mov    edx, 3
000075D4 66BA0300            <1>     mov     dx, 3
000075D8 F7E2                <1>     mul     edx
                             <1>     ; EAX = FAT Beginning Sector
                             <1>     ; EDX = 0
000075DA 8A0E                <1>     mov     cl, [esi+LD_Name]
                             <1>     ;cmp    byte [FAT_BuffValidData], 0
                             <1>     ;jna    short fd_init_load_FAT_sectors0
000075DC 3A0D[8B7E0100]      <1>     cmp     cl, [FAT_BuffDrvName]
000075E2 751D                <1>     jne     short fd_init_load_FAT_sectors0
000075E4 3B05[8E7E0100]      <1>     cmp     eax, [FAT_BuffSector]
000075EA 751B                <1>     jne     short fd_init_load_FAT_sectors1
                             <1>     ; 25/07/2022
000075EC A1[867E0100]        <1>     mov     eax, [FAT_CurrentCluster]
                             <1>     ;mov    al, [FAT_CurrentCluster]
000075F1 D1E8                <1>     shr     eax, 1
                             <1>     ;shr    al, 1
000075F3 668B03              <1>     mov     ax, [ebx]
000075F6 7305                <1>     jnc     short fd_init_gnc_even
                             <1>     ;shr    ax, 4
                             <1>     ; 25/04/2022
000075F8 C1E804              <1>     shr     eax, 4
                             <1> fd_init_gnc_clc_retn:
000075FB F8                  <1>     clc
000075FC C3                  <1>     retn
                             <1>
                             <1> fd_init_gnc_even:
000075FD 80E40F              <1>     and     ah, 0Fh
00007600 C3                  <1>     retn
                             <1>
                             <1> fd_init_load_FAT_sectors0:
00007601 880D[8B7E0100]      <1>     mov     [FAT_BuffDrvName], cl
                             <1> fd_init_load_FAT_sectors1:
00007607 C605[8A7E0100]00    <1>     mov     byte [FAT_BuffValidData], 0
0000760E A3[8E7E0100]        <1>     mov     [FAT_BuffSector], eax
00007613 034660              <1>     add     eax, [esi+LD_FATBegin]
00007616 BB001C0900          <1>     mov     ebx, FAT_Buffer
                             <1>     ;movzx  ecx, word [esi+LD_BPB+BPB_FATSz16]
0000761B 668B4E1C            <1>     mov     cx, [esi+LD_BPB+BPB_FATSz16]
                             <1>     ;sub    cx, [FAT_BuffSector]
                             <1>         ; 25/07/2022
0000761F 2B0D[8E7E0100]      <1>     sub     ecx, [FAT_BuffSector]
                             <1>     ;sub    edx, edx
00007625 B203                <1>     mov     dl, 3
                             <1>     ; edx = 3
                             <1>     ;;cmp   ecx, 3
                             <1>     ;cmp    cx, 3
00007627 39D1                <1>     cmp     ecx, edx ; 3
00007629 7602                <1>     jna     short fdinit_pass_fix_sector_count_3
                             <1>     ;;mov   ecx, 3
                             <1>     ;mov    ecx, 3
0000762B 89D1                <1>     mov     ecx, edx ; 3
                             <1> fdinit_pass_fix_sector_count_3:
0000762D E8C7A30000          <1>     call    chs_read
00007632 730D                <1>     jnc     short fd_init_FAT_sectors_no_load_error
00007634 C605[8A7E0100]00    <1>     mov     byte [FAT_BuffValidData], 0
                             <1>             ; Drv not ready or read Error !
0000763B B80F000000          <1>     mov     eax, ERR_DRV_NOT_RDY ; 15
                             <1>     ;xor    edx, edx
00007640 C3                  <1>     retn
                             <1>
                             <1> fd_init_FAT_sectors_no_load_error:
00007641 C605[8A7E0100]01    <1>     mov     byte [FAT_BuffValidData], 1
00007648 A1[867E0100]        <1>     mov     eax, [FAT_CurrentCluster]
0000764D E967FFFFFF          <1>         jmp     fd_init_get_next_cluster_readnext
                             <1>
                             <1> get_FAT_volume_name:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 12/09/2009
                             <1>     ; INPUT ->
                             <1>     ;       BH = Logical DOS drive number (0,1,2,3,4 ...)
                             <1>     ;        BL = 0
                             <1>     ; OUTPUT ->
                             <1>     ;       CF = 0 -> ESI = Volume name address
                             <1>     ;       CF = 1 -> Root volume name not found
                             <1>
                             <1>     ;mov    ah, 0FFh
                             <1>     ;mov    al, [Last_Dos_DiskNo]
                             <1>     ;cmp    al, bh
                             <1>     ;jb      short loc_gfvn_dir_load_err
                             <1>
00007652 89DE                <1>     mov     esi, ebx
00007654 81E600FF0000        <1>     and     esi, 0FF00h ; esi = bh
0000765A 81C600010900        <1>     add     esi, Logical_DOSDisks
00007660 8A06                <1>     mov      al, [esi+LD_Name]
00007662 8A6603              <1>     mov      ah, [esi+LD_FATType]
                             <1>     ; 19/12/2025
                             <1>     ;cmp    ah, 1
                             <1>     ;jb     short loc_gfvn_dir_load_err
00007665 3C41                <1>     cmp     al, 'A'
00007667 720C                <1>     jb      short loc_gfvn_dir_load_err
00007669 80FC02              <1>     cmp     ah, 2
0000766C 7708                <1>     ja      short get_FAT32_root_cluster
                             <1>
0000766E E8544A0000          <1>     call     load_FAT_root_directory
00007673 730B                <1>     jnc      short loc_get_volume_name
                             <1>
                             <1> loc_gfvn_dir_load_err:
                             <1> loc_get_volume_name_retn: ; 29/08/2023
00007675 C3                  <1>     retn
                             <1>
                             <1> get_FAT32_root_cluster:
00007676 8B4632              <1>     mov     eax, [esi+LD_BPB+BPB_RootClus]
00007679 E8C74A0000          <1>     call     load_FAT_sub_directory
0000767E 72F5                <1>     jc      short loc_get_volume_name_retn
                             <1>
                             <1> loc_get_volume_name:
00007680 BE00000800          <1>         mov     esi, Directory_Buffer
                             <1>     ;xor    cx, cx ; 0
```

```
                                <1>     ; 25/07/2022
00007685 31C9                   <1>     xor    ecx, ecx ; 0
                                <1> check_root_volume_name:
00007687 8A06                   <1>     mov    al, [esi]
00007689 08C0                   <1>     or      al, al
0000768B 74E8                   <1>     jz      short loc_get_volume_name_retn
0000768D 807E0B08               <1>     cmp     byte [esi+0Bh], 08h
00007691 74E2                   <1>     je      short loc_get_volume_name_retn
00007693 663B0D[A07E0100]       <1>     cmp     cx, [DirBuff_LastEntry]
0000769A 7306                   <1>     jnb     short pass_check_root_volume_name
                                <1>     ;inc   cx
                                <1>     ; 25/07/2022
0000769C 41                     <1>     inc    ecx
0000769D 83C620                 <1>     add     esi, 32
000076A0 EBE5                   <1>     jmp     short check_root_volume_name
                                <1>
                                <1>     ; 29/08/2023
                                <1> ;loc_get_volume_name_retn:
                                <1>     ;retn
                                <1>
                                <1> pass_check_root_volume_name:
000076A2 803D[9C7E0100]03       <1>     cmp    byte [DirBuff_FATType], 3
000076A9 7230                   <1>     jb     short loc_get_volume_name_retn_xor
                                <1>
000076AB BB001C0900             <1>     mov    ebx, FAT_Buffer
000076B0 BE00010900             <1>     mov    esi, Logical_DOSDisks
000076B5 31C0                   <1>     xor    eax, eax
000076B7 8A25[9B7E0100]         <1>     mov    ah, [DirBuff_DRV]
000076BD 80EC41                 <1>     sub    ah, 'A'
000076C0 01C6                   <1>     add    esi, eax
000076C2 A1[A27E0100]           <1>     mov    eax, [DirBuff_Cluster]
000076C7 E8BD480000             <1>     call   get_next_cluster
000076CC 7305                   <1>     jnc    short loc_gfvn_load_FAT32_dir_cluster
                                <1>
000076CE 83F801                 <1>     cmp     eax, 1
000076D1 F5                     <1>     cmc
000076D2 C3                     <1>     retn
                                <1>
                                <1> loc_gfvn_load_FAT32_dir_cluster:
000076D3 E86D4A0000             <1>     call   load_FAT_sub_directory
000076D8 73A6                   <1>     jnc    short loc_get_volume_name
000076DA C3                     <1>     retn
                                <1>
                                <1> loc_get_volume_name_retn_xor:
000076DB 31C0                   <1>     xor    eax, eax
000076DD C3                     <1>     retn
                                <1>
                                <1> get_media_change_status:
                                <1>     ; 10/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 09/09/2009
                                <1>     ; INPUT:
                                <1>     ;     DL = Drive number (physical)
                                <1>     ; OUTPUT: clc & AH = 6 media changed
                                <1>     ;     clc & AH = 0 media not changed
                                <1>     ;     stc -> Drive not ready or an error
                                <1>
000076DE B416                   <1>     mov    ah, 16h
000076E0 E82AD9FFFF             <1>     call   int13h
000076E5 80FC06                 <1>     cmp    ah, 06h
000076E8 7405                   <1>     je     short loc_gmc_status_retn
000076EA 08E4                   <1>     or     ah, ah
000076EC 7401                   <1>     jz     short loc_gmc_status_retn
                                <1> loc_gmc_status_stc_retn:
000076EE F9                     <1>     stc
                                <1> loc_gmc_status_retn:
000076EF C3                     <1>     retn
                                %include 'trdosk3.s' ; 06/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - MAIN PROGRAM : trdosk3.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 19/12/2025  (Previous: 26/09/2024, TRDOS 386 v2.0.9)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 06/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; MAINPROG.ASM (09/11/2011)
                                <1> ; ****************************************************************************
                                <1> ; MAINPROG.ASM [ TRDOS KERNEL - COMMAND EXECUTER SECTION - MAIN PROGRAM ]
                                <1> ; (c) 2004-2011  Erdogan TAN  [ 17/01/2004 ]  Last Update: 09/11/2011
                                <1> ; CMD_INTR.ASM [ TRDOS Command Interpreter Procedure ] Last Update: 09/11/2011
                                <1> ; DIR.ASM [ DIRECTORY FUNCTIONS ] Last Update: 09/10/2011
                                <1> ; FILE.ASM [ FILE FUNCTIONS ] Last Update: 09/10/2011
                                <1>
                                <1> change_current_drive:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 16/10/2016
                                <1>     ; 02/02/2016
                                <1>     ; 15/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 18/08/2011
                                <1>     ; 09/09/2009
                                <1>     ; INPUT:
                                <1>     ;   DL = Logical DOS Drive Number
                                <1>     ; OUTPUT:
                                <1>     ;  cf=1 -> Not successful
                                <1>     ;   EAX = Error code
                                <1>     ;  cf=0 ->
                                <1>     ;   EAX = 0 (successful)
                                <1>
000076F0 31DB                   <1>     xor    ebx, ebx
000076F2 88D7                   <1>     mov    bh, dl
                                <1>
                                <1>     ;cmp   dl, 1
                                <1>     ;jna   short loc_ccdrv_initial_media_change_check
                                <1>     ;cmp   bh, [Last_Dos_DiskNo]
                                <1>     ;ja    short loc_ccdrv_drive_not_ready_err
                                <1>
                                <1> loc_ccdrv_initial_media_change_check:
000076F4 BE00010900              <1>     mov    esi, Logical_DOSDisks
000076F9 01DE                   <1>     add    esi, ebx
                                <1> loc_ccdrv_dos_drive_name_check:
000076FB 80FA02                 <1>     cmp    dl, 2
000076FE 720E                   <1>     jb     short loc_ccdrv_dos_drive_name_check_ok
                                <1>
00007700 8A06                   <1>     mov    al, [esi+LD_Name]
00007702 2C41                   <1>     sub    al, 'A'
00007704 38D0                   <1>     cmp    al, dl
00007706 7406                   <1>     je     short loc_ccdrv_dos_drive_name_check_ok
                                <1>
                                <1> loc_ccdrv_drive_not_ready_err:
                                <1>     ; 16/10/2016 (15h -> 15)
                                <1>     ;mov   eax, 15 ; Drive not ready
```

```
                                <1>     ; 26/07/2022
00007708 29C0                   <1>     sub     eax, eax
0000770A B00F                   <1>     mov     al, 15
                                <1> loc_change_current_drive_stc_retn:
0000770C F9                     <1>     stc
0000770D C3                     <1>     retn
                                <1>
                                <1> loc_ccdrv_dos_drive_name_check_ok:
0000770E 8A667E                 <1>     mov     ah, [esi+LD_MediaChanged]
00007711 80FC06                 <1>     cmp     ah, 6  ; VOLUME NAME CHECK/MOVE SIGN
00007714 7455                   <1>     je      short loc_ccdrv_get_FAT_volume_name_0
                                <1>
00007716 80FA01                 <1>     cmp     dl, 1
00007719 7778                   <1>     ja      short loc_gmcs_init_drv_hd
                                <1>
                                <1> loc_gmcs_init_drv_fd:
0000771B 08E4                   <1>     or      ah, ah
                                <1>     ; AH = 1 is initialization sign (invalid_fd_parameter)
0000771D 7517                   <1>     jnz     short loc_ccdrv_call_fd_init
                                <1>
0000771F E8BAFFFFFF             <1>     call    get_media_change_status
00007724 72E2                   <1>     jc      short loc_ccdrv_drive_not_ready_err
                                <1>
00007726 20E4                   <1>     and     ah, ah
00007728 7471                   <1>     jz      short loc_change_current_drv3
                                <1>
0000772A 80F406                 <1>     xor     ah, 6
0000772D 75D9                   <1>     jnz     short loc_ccdrv_drive_not_ready_err
                                <1>
                                <1> loc_ccdrv_call_fd_init_check_vol_id:
0000772F E81C090000             <1>     call    get_volume_serial_number
00007734 730D                   <1>     jnc     short loc_ccdrv_check_vol_serial
                                <1>
                                <1> loc_ccdrv_call_fd_init:
00007736 E83AFDFFFF             <1>     call    floppy_drv_init
0000773B 731A                   <1>     jnc     short loc_reset_drv_fd_current_dir
                                <1>
                                <1> loc_ccdrv_fdinit_fail_retn:
                                <1>     ; 16/10/2016
0000773D B80F000000             <1>     mov     eax, 15 ; Drive not ready
00007742 C3                     <1>     retn
                                <1>
                                <1> loc_ccdrv_check_vol_serial:
00007743 A3[6C770100]           <1>     mov     [Current_VolSerial], eax
                                <1>     ;mov    dl, bh
00007748 E828FDFFFF             <1>     call    floppy_drv_init
0000774D 72EE                   <1>     jc      short loc_ccdrv_fdinit_fail_retn
                                <1>
0000774F 3B05[6C770100]         <1>     cmp     eax, [Current_VolSerial]
00007755 7440                   <1>     je      short loc_change_current_drv2
                                <1>
                                <1> loc_reset_drv_fd_current_dir:
00007757 31C0                   <1>     xor     eax, eax
00007759 88467F                 <1>         mov [esi+LD_CDirLevel], al
0000775C 89F7                   <1>     mov     edi, esi
0000775E 81C780000000           <1>     add     edi, LD_CurrentDirectory
00007764 B920000000             <1>     mov     ecx, 32
00007769 F3AB                   <1>     rep     stosd
                                <1>
                                <1> loc_ccdrv_get_FAT_volume_name_0:
                                <1>     ; 19/12/2025
                                <1>     ;mov    al, [esi+LD_FATType]
                                <1>     ;or     al, al
                                <1>     ;jz     short loc_change_current_drv2
                                <1>
0000776B 56                     <1>     push    esi
                                <1>     ;cmp    al, 2
                                <1>     ; 19/12/2025
0000776C 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
00007770 7705                   <1>     ja      short loc_ccdrv_get_FAT32_vol_name
                                <1>
                                <1> loc_ccdrv_get_FAT2_16_vol_name:
00007772 83C631                 <1>     add     esi, LD_BPB + VolumeLabel
00007775 EB03                   <1>     jmp     short loc_ccdrv_get_FAT_volume_name_1
                                <1>
                                <1> loc_ccdrv_get_FAT32_vol_name:
00007777 83C64D                 <1>     add     esi, LD_BPB + FAT32_VolLab
                                <1> loc_ccdrv_get_FAT_volume_name_1:
0000777A 53                     <1>     push    ebx
0000777B 56                     <1>     push    esi
0000777C E8D1FEFFFF             <1>     call    get_FAT_volume_name
00007781 5F                     <1>     pop     edi
00007782 5B                     <1>     pop     ebx
                                <1>     ; BL = 0
00007783 720B                   <1>     jc      short loc_change_current_drv1
00007785 20C0                   <1>     and     al, al
00007787 7407                   <1>     jz      short loc_change_current_drv1
                                <1>
                                <1> loc_ccdrv_move_FAT_volume_name:
00007789 B90B000000             <1>     mov     ecx, 11
0000778E F3A4                   <1>     rep     movsb
                                <1>
                                <1> loc_change_current_drv1:
00007790 5E                     <1>     pop     esi
00007791 EB04                   <1>     jmp     short loc_change_current_drv2
                                <1>
                                <1> loc_gmcs_init_drv_hd:
00007793 08E4                   <1>     or      ah, ah
00007795 7404                   <1>     jz      short loc_change_current_drv3
                                <1>     ; BL = 0, BH = Logical DOS drive number
                                <1> loc_change_current_drv2:
00007797 C6467E00               <1>     mov     byte [esi+LD_MediaChanged], 0
                                <1> loc_change_current_drv3:
0000779B 883D[76770100]         <1>     mov     [Current_Drv], bh
                                <1>
                                <1>     ;call   restore_current_directory
                                <1>     ;retn
                                <1>
                                <1> restore_current_directory:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 11/02/2016
                                <1>     ; 15/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 25/01/2010
                                <1>     ; 12/10/2009
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ESI = Logical DOS Drive Description Table
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;   ESI = Logical DOS Drive Description Table
                                <1>     ;   EDI = offset Current_Dir_Drv
                                <1>
000077A1 8A4603                 <1>     mov     al, [esi+LD_FATType]
```

```
000077A4 A2[75770100]        <1>     mov    [Current_FATType], al
                             <1>
000077A9 8A26                <1>     mov    ah, [esi+LD_Name]
000077AB 8825[77770100]      <1>     mov    [Current_Dir_Drv], ah
                             <1>
                             <1>     ; 19/12/2025
                             <1>     ;and   al, al
                             <1>     ;jz    short loc_restore_FS_current_directory
                             <1>
                             <1> loc_restore_FAT_current_directory:
000077B1 8A667F              <1>     mov    ah, [esi+LD_CDirLevel]
000077B4 8825[74770100]      <1>     mov    [Current_Dir_Level], ah
000077BA 08E4                <1>     or     ah, ah
000077BC 7410                <1>         jz  short loc_ccdrv_reset_cdir_FAT_12_16_32_fcluster
                             <1>
000077BE 0FB6D4              <1>     movzx  edx, ah
000077C1 C0E204              <1>     shl    dl, 4 ; * 16
000077C4 01F2                <1>         add edx, esi
000077C6 8B828C000000        <1>     mov    eax, [edx+LD_CurrentDirectory+12]
000077CC EB24                <1>     jmp    short loc_ccdrv_reset_cdir_FAT_fcluster
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> loc_restore_FS_current_directory:
                             <1>     ;call  load_current_FS_directory
                             <1>     ;retn
                             <1>     ; 26/07/2022
                             <1>     jmp    load_current_FS_directory
                             <1> %endif
                             <1>
                             <1> loc_ccdrv_reset_cdir_FAT_12_16_32_fcluster:
000077CE 3C03                <1>     cmp    al, 3
000077D0 7205                <1>     jb     short loc_ccdrv_reset_cdir_FAT_12_16_fcluster
                             <1> loc_ccdrv_reset_cdir_FAT32_fcluster:
000077D2 8B4632              <1>     mov    eax, [esi+LD_BPB+FAT32_RootFClust]
000077D5 EB02                <1>     jmp    short loc_ccdrv_check_rootdir_sign
                             <1> loc_ccdrv_reset_cdir_FAT_12_16_fcluster:
                             <1>     ;xor   al, al  ; xor eax, eax
                             <1>     ; 26/07/2022
000077D7 31C0                <1>     xor    eax, eax
                             <1>     ;xor   edx, edx
                             <1> loc_ccdrv_check_rootdir_sign:
000077D9 80BE8000000000      <1>     cmp    byte [esi+LD_CurrentDirectory], 0
000077E0 7510                <1>     jne    short loc_ccdrv_reset_cdir_FAT_fcluster
                             <1> loc_ccdrv_set_rootdir_FAT_fcluster:
000077E2 89868C000000        <1>         mov     [esi+LD_CurrentDirectory+12], eax
000077E8 C78680000000524F4F- <1>     mov    dword [esi+LD_CurrentDirectory], 'ROOT'
000077F1 54                  <1>
                             <1>
                             <1> loc_ccdrv_reset_cdir_FAT_fcluster:
000077F2 A3[70770100]        <1>     mov    [Current_Dir_FCluster], eax
                             <1>
000077F7 BF[D47E0100]        <1>     mov    edi, PATH_Array
000077FC 89F2                <1>     mov    edx, esi
000077FE 81C680000000        <1>     add    esi, LD_CurrentDirectory
00007804 B920000000          <1>     mov    ecx, 32
00007809 F3A5                <1>     rep    movsd
                             <1>
0000780B E8042B0000          <1>     call   change_prompt_dir_string
                             <1>
00007810 89D6                <1>     mov    esi, edx
                             <1>
00007812 29C0                <1>         sub eax, eax
                             <1>        ;sub edx, edx
00007814 BF[77770100]        <1>     mov    edi, Current_Dir_Drv
                             <1>
00007819 A2[8E2E0100]        <1>     mov    [Restore_CDIR], al ; 0
0000781E C3                  <1>     retn
                             <1>
                             <1> dos_prompt:
                             <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 06/05/2016
                             <1>     ; 30/01/2016
                             <1>     ; 29/01/2016
                             <1>     ; 16/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 15/09/2011
                             <1>     ; 13/09/2009
                             <1>     ; 2004-2005
                             <1>
                             <1>     ; 06/05/2016
0000781F C705[34830100]-     <1>     mov    dword [mainprog_return_addr], return_from_cmd_interpreter
00007825 [D1780000]          <1>
                             <1>
                             <1> loc_TRDOS_prompt:
00007829 BF[76780100]        <1>     mov    edi, TextBuffer
0000782E C6075B              <1>     mov    byte [edi], "["
00007831 47                  <1>     inc    edi
00007832 BE[EA2E0100]        <1>     mov    esi, TRDOSPromptLabel
                             <1> get_next_prompt_label_char:
00007837 803E20              <1>     cmp    byte [esi], 20h
0000783A 7203                <1>     jb     short pass_prompt_label
0000783C A4                  <1>     movsb
0000783D EBF8                <1>     jmp    short get_next_prompt_label_char
                             <1> pass_prompt_label:
0000783F C6075D              <1>     mov    byte [edi], "]"
00007842 47                  <1>     inc    edi
00007843 C60720              <1>     mov    byte [edi], 20h
00007846 47                  <1>     inc    edi
00007847 BE[77770100]        <1>     mov    esi, Current_Dir_Drv
0000784C 66A5                <1>     movsw
0000784E A4                  <1>     movsb
                             <1> loc_prompt_current_directory:
0000784F 803E20              <1>     cmp    byte [esi], 20h
00007852 7203                <1>     jb     short pass_prompt_current_directory
00007854 A4                  <1>     movsb
00007855 EBF8                <1>     jmp    short loc_prompt_current_directory
                             <1> pass_prompt_current_directory:
00007857 C6073E              <1>     mov    byte [edi], '>'
0000785A 47                  <1>     inc    edi
0000785B C60700              <1>     mov    byte [edi], 0
0000785E BE[76780100]        <1>     mov    esi, TextBuffer
00007863 E852F5FFFF          <1>     call   print_msg
                             <1>
                             <1>     ;sub   bh, bh ; video page = 0
                             <1>     ;call  get_cpos ; get cursor position
00007868 668B15[CE760100]    <1>     mov    dx, [CURSOR_POSN] ; video page 0
0000786F 8815[D6770100]      <1>     mov    [CursorColumn], dl
                             <1>
                             <1>     ; 30/01/2016 (to show cursor on the row, again)
                             <1>     ; (Initial color attributes of video page 0 is 0)
                             <1>     ; (see: 'StartPMP' in trdos386.s)
                             <1>     ;
                             <1>     ;mov   edi, 0B8000h ; start of video page 0
                             <1>     ;movzx ecx, dl ; column
```

```
                            <1>     ;mov    al, 80
                            <1>     ;mul    dh
                            <1>     ;add    ax, cx
                            <1>     ;shl    ax, 1 ; character + attribute
                            <1>     ;add    di, ax ; (2*80*row) + (2*column)
                            <1>     ;neg    cl
                            <1>     ;add    cl, 80
                            <1>     ;mov    ax, 700h ;  ah = 7 (color attribute)
                            <1>     ;rep    stosw
                            <1>
                            <1> loc_rw_char:
00007875 E8AB000000         <1>     call    rw_char
                            <1> loc_move_command:
0000787A BE[26780100]       <1>     mov     esi, CommandBuffer
0000787F 89F7               <1>     mov     edi, esi
00007881 31C9               <1>     xor     ecx, ecx
                            <1> first_command_char:
00007883 AC                 <1>     lodsb
00007884 3C20               <1>     cmp     al, 20h
00007886 770C               <1>     ja      short pass_space_control
00007888 723F               <1>     jb      short loc_move_cmd_arguments_ok
0000788A 81FE[75780100]     <1>     cmp     esi, CommandBuffer + 79
00007890 72F1               <1>     jb      short first_command_char
00007892 EB35               <1>     jmp     short loc_move_cmd_arguments_ok
                            <1>
                            <1>     ; 26/07/2022
                            <1> pass_space_control:
00007894 3C61               <1>     cmp     al, 61h
00007896 7206               <1>     jb      short pass_capitalize
00007898 3C7A               <1>     cmp     al, 7Ah
0000789A 7702               <1>     ja      short pass_capitalize
0000789C 24DF               <1>     and     al, 0DFh
                            <1> pass_capitalize:
0000789E AA                 <1>     stosb
0000789F FEC1               <1>     inc     cl
000078A1 81FE[75780100]     <1>     cmp     esi, CommandBuffer + 79
                            <1>     ;jb     short next_command_char
                            <1>     ; 26/07/2022
000078A7 7320               <1>     jnb     short loc_move_cmd_arguments_ok
                            <1>
                            <1> next_command_char:
000078A9 AC                 <1>     lodsb
000078AA 3C20               <1>     cmp     al, 20h
000078AC 77E6               <1>     ja      short pass_space_control
000078AE 7219               <1>     jb      short loc_move_cmd_arguments_ok
                            <1>
                            <1> loc_1st_cmd_arg: ; 30/01/2016
000078B0 AC                 <1>     lodsb
000078B1 3C20               <1>     cmp     al, 20h
000078B3 74FB               <1>     je      short loc_1st_cmd_arg
000078B5 7212               <1>     jb      short loc_move_cmd_arguments_ok
                            <1>
000078B7 C60700             <1>         mov     byte [edi], 0
000078BA 47                 <1>     inc     edi
                            <1>
                            <1> loc_move_cmd_arguments:
000078BB AA                 <1>     stosb
000078BC 81FE[75780100]     <1>     cmp     esi, CommandBuffer + 79
000078C2 7305               <1>     jnb     short loc_move_cmd_arguments_ok
000078C4 AC                 <1>         lodsb
000078C5 3C20               <1>     cmp     al, 20h
000078C7 73F2               <1>     jnb     short loc_move_cmd_arguments
                            <1>     ; 26/07/2022
                            <1>     ;jmp    short loc_move_cmd_arguments_ok
                            <1>
                            <1> ; 26/07/2022
                            <1> ;pass_space_control:
                            <1> ;   cmp     al, 61h
                            <1> ;   jb      short pass_capitalize
                            <1> ;   cmp     al, 7Ah
                            <1> ;   ja      short pass_capitalize
                            <1> ;   and     al, 0DFh
                            <1> ;pass_capitalize:
                            <1> ;   stosb
                            <1> ;   inc     cl
                            <1> ;   cmp     esi, CommandBuffer + 79
                            <1> ;   jb      short next_command_char
                            <1>
                            <1> loc_move_cmd_arguments_ok:
000078C9 C60700             <1>         mov     byte [edi], 0
                            <1>
                            <1> call_command_interpreter:
000078CC E8A8070000         <1>     call    command_interpreter
                            <1>
                            <1> return_from_cmd_interpreter:
000078D1 B950000000         <1>         mov ecx, 80
                            <1>     ;mov    cx, 80
000078D6 BF[26780100]       <1>     mov     edi, CommandBuffer
000078DB 30C0               <1>     xor     al, al
000078DD F3AA               <1>     rep     stosb
                            <1>     ;cmp    byte [Program_Exit], 0
                            <1>     ;ja     short loc_terminate_trdos
                            <1>
                            <1>     ; 16/01/2016
000078DF 803D[2E680000]03   <1>     cmp     byte [CRT_MODE], 3 ; 80*25 color
000078E6 7419               <1>     je      short pass_set_txt_mode
                            <1>
000078E8 E896A2FFFF         <1>     call    set_txt_mode ; set vide mode to 03h
                            <1>     ; 07/01/2017
000078ED 30C0               <1>     xor     al, al
                            <1>
                            <1> loc_check_active_page:
                            <1>     ;xor    al, al
000078EF 3805[DE760100]     <1>     cmp     [ACTIVE_PAGE], al ; 0
                            <1>         ;je loc_TRDOS_prompt
                            <1>     ; 26/07/2022
000078F5 7405               <1>     je      short loc_prompt_again
                            <1>
                            <1>     ; AL = 0 = video page 0
000078F7 E8B7A6FFFF         <1>     call    set_active_page
                            <1> loc_prompt_again: ; 26/07/2022
000078FC E928FFFFFF         <1>         jmp     loc_TRDOS_prompt ; infinitive loop
                            <1>
                            <1> pass_set_txt_mode:
00007901 BE[17390100]       <1>     mov     esi, nextline
00007906 E8AFF4FFFF         <1>     call    print_msg
0000790B EBE2               <1>     jmp     short loc_check_active_page
                            <1>
                            <1> ;rw_char:
                            <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                            <1> loc_arrow:
0000790D 80FC4B             <1>     cmp     ah, 4Bh
00007910 7426               <1>     je      short loc_back
00007912 80FC53             <1>     cmp     ah, 53h
```

```
00007915 7421                <1>     je      short loc_back
00007917 80FC4D              <1>     cmp     ah, 4Dh
0000791A 7509                <1>     jne     short readnextchar
0000791C 80FA4F              <1>     cmp     dl, 79
0000791F 7304                <1>     jnb     short readnextchar
00007921 FEC2                <1>     inc     dl
00007923 EB1D                <1>     jmp     short set_cursor_pos
                             <1>
                             <1> rw_char:
                             <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 13/05/2016
                             <1>     ; 30/01/2016
                             <1>     ; 29/01/2016
                             <1>     ; 17/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 2004-2005
                             <1>
                             <1>     ; DH = cursor row, DL = cursor column
                             <1>     ; BH = 0 = video page number (active page)
                             <1>
                             <1>     ;xor   bh, bh ; 0 = video page 0
                             <1>
                             <1> readnextchar:
00007925 30E4                <1>     xor     ah, ah
00007927 E8E695FFFF          <1>     call    int16h
0000792C 20C0                <1>     and     al, al
0000792E 74DD                <1>     jz      short loc_arrow
00007930 3CE0                <1>     cmp     al, 0E0h
00007932 74D9                <1>     je      short loc_arrow
00007934 3C08                <1>     cmp     al, 08h
00007936 752A                <1>     jne     short char_return
                             <1> loc_back:
00007938 3A15[D6770100]      <1>     cmp     dl, [CursorColumn]
0000793E 76E5                <1>     jna     short readnextchar
                             <1> prev_column:
00007940 FECA                <1>     dec     dl
                             <1> set_cursor_pos:
                             <1>     ;push   dx
00007942 52                  <1>     push    edx ; 29/12/2017
                             <1>     ;xor   bh, bh ; 0 = video page 0
                             <1>     ; DH = Row, DL = Column
00007943 E82FAAFFFF          <1>     call    _set_cpos ; 17/01/2016
00007948 5A                  <1>         pop edx ; 29/12/2017
                             <1>     ;pop   dx
                             <1>     ;movzx ebx, dl
00007949 88D3                <1>     mov     bl, dl
0000794B 2A1D[D6770100]      <1>     sub     bl, [CursorColumn]
00007951 B020                <1>     mov     al, 20h
00007953 8883[26780100]      <1>     mov     [CommandBuffer+ebx], al
                             <1>     ;sub   bh, bh ; video page 0
                             <1>     ;mov   cx, 1
00007959 B307                <1>     mov     bl, 7 ; color attribute
                             <1>     ; bh = 0 ; 26/07/2022
0000795B E8FEA8FFFF          <1>     call    _write_c_current ; 17/01/2016
                             <1>     ;mov   dx, [CURSOR_POSN]
00007960 EBC3                <1>     jmp     short readnextchar
                             <1>
                             <1>     ; 26/07/2022
                             <1> ;loc_arrow:
                             <1> ;   cmp     ah, 4Bh
                             <1> ;   je      short loc_back
                             <1> ;   cmp     ah, 53h
                             <1> ;   je      short loc_back
                             <1> ;   cmp     ah, 4Dh
                             <1> ;   jne     short readnextchar
                             <1> ;   cmp     dl, 79
                             <1> ;   jnb     short readnextchar
                             <1> ;   inc     dl
                             <1> ;   jmp     short set_cursor_pos
                             <1>
                             <1> char_return:
00007962 0FB6DA              <1>     movzx   ebx, dl
00007965 2A1D[D6770100]      <1>     sub     bl, [CursorColumn]
0000796B 3C20                <1>     cmp     al, 20h
0000796D 721D                <1>     jb      short loc_escape
0000796F 8883[26780100]      <1>     mov     [CommandBuffer+ebx], al
00007975 80FA4F              <1>     cmp     dl, 79
00007978 73AB                <1>     jnb     short readnextchar
0000797A 66BB0700            <1>     mov     bx, 7 ; color attribute
0000797E E85CA9FFFF          <1>     call    _write_tty
00007983 668B15[CE760100]    <1>     mov     dx, [CURSOR_POSN] ; video page 0
0000798A EB99                <1>         jmp short readnextchar ; 26/07/2022
                             <1>
                             <1> loc_escape:
0000798C 3C1B                <1>     cmp     al, 1Bh
0000798E 7414                <1>     je      short rw_char_retn
                             <1>     ;
00007990 3C0D                <1>     cmp     al, 0Dh ; CR
00007992 7591                <1>         jne short readnextchar ; 26/07/2022
                             <1>
                             <1>     ; 13/05/2016
00007994 66BB0700            <1>     mov     bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh=0)
00007998 E842A9FFFF          <1>     call    _write_tty
                             <1>     ;mov   bx, 7  ; attribute/color
                             <1>                   ; video page 0 (bh=0)
0000799D B00A                <1>     mov     al, 0Ah ; LF
0000799F E83BA9FFFF          <1>     call    _write_tty
                             <1> rw_char_retn:
000079A4 C3                  <1>     retn
                             <1>
                             <1> show_date:
                             <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>         ; 2004-2005
                             <1>
                             <1>     ;mov   ah, 04h
                             <1>     ;call  int1Ah
000079A5 E81EEAFFFF          <1>     call    RTC_40  ; GET RTC DATE
                             <1>
000079AA 88D0                <1>     mov     al, dl
000079AC E85495FFFF          <1>     call    bcd_to_ascii
000079B1 66A3[D62F0100]      <1>     mov     [Day], ax
                             <1>
000079B7 88F0                <1>     mov     al, dh
000079B9 E84795FFFF          <1>     call    bcd_to_ascii
000079BE 66A3[D92F0100]      <1>     mov     [Month], ax
                             <1>
000079C4 88E8                <1>     mov     al, ch
000079C6 E83A95FFFF          <1>     call    bcd_to_ascii
000079CB 66A3[DC2F0100]      <1>     mov     [Century], ax
                             <1>
000079D1 88C8                <1>     mov     al, cl
000079D3 E82D95FFFF          <1>     call    bcd_to_ascii
000079D8 66A3[DE2F0100]      <1>     mov     [Year], ax
```

```
                                <1>
000079DE BE[C62F0100]           <1>     mov     esi, Msg_Show_Date
                                <1>     ;call   print_msg
                                <1>     ;retn
                                <1>     ; 26/07/2022
000079E3 E9D2F3FFFF             <1>     jmp     print_msg
                                <1>
                                <1> set_date:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/05/2016
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>         ; 2004-2005
                                <1>
000079E8 BE[AA2F0100]           <1>     mov     esi, Msg_Enter_Date
000079ED E8C8F3FFFF             <1>     call    print_msg
                                <1>
                                <1> loc_enter_day_1:
000079F2 30E4                   <1>     xor     ah, ah
000079F4 E81995FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
000079F9 3C0D                   <1>     cmp     al, 13
                                <1>     ;je     loc_set_date_retn
                                <1>     ; 26/07/2022
000079FB 7404                   <1>     je      short set_date_0
000079FD 3C1B                   <1>     cmp     al, 27
                                <1>     ;je     loc_set_date_retn
                                <1>     ; 26/07/2022
000079FF 7511                   <1>     jne     short set_date_1
                                <1>
                                <1> set_date_0:
                                <1> ;loc_set_date_retn:
00007A01 BE[17390100]           <1>     mov     esi, nextline
                                <1>     ;call   print_msg
                                <1>     ;retn
                                <1>     ; 26/07/2022
00007A06 E9AFF3FFFF             <1>     jmp     print_msg
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_date_stc_0:
                                <1>     ;xor    bh, bh ; video page 0
00007A0B E8B9A9FFFF             <1>     call    beeper ; BEEP !
00007A10 EBE0                   <1>     jmp     short loc_enter_day_1
                                <1>
                                <1>     ; 26/07/2022
                                <1> set_date_1:
00007A12 A2[D62F0100]           <1>     mov     [Day], al
00007A17 3C30                   <1>     cmp     al, '0'
00007A19 72F0                   <1>     jb      short loc_set_date_stc_0
00007A1B 3C33                   <1>     cmp     al, '3'
00007A1D 77EC                   <1>     ja      short loc_set_date_stc_0
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007A1F B307                   <1>     mov     bl, 7
00007A21 E8B9A8FFFF             <1>     call    _write_tty
                                <1> loc_enter_day_2:
00007A26 30E4                   <1>     xor     ah, ah
00007A28 E8E594FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007A2D 3C1B                   <1>     cmp     al, 27
                                <1>         ;je loc_set_date_retn
                                <1>     ; 26/07/2022
00007A2F 74D0                   <1>     je      short set_date_0
00007A31 A2[D72F0100]           <1>     mov     [Day+1], al
00007A36 3C30                   <1>     cmp     al, '0'
00007A38 7211                   <1>         jb      short loc_set_date_stc_1
00007A3A 3C39                   <1>     cmp     al, '9'
00007A3C 770D                   <1>         ja      short loc_set_date_stc_1
00007A3E 803D[D62F0100]33       <1>     cmp     byte [Day], '3'
00007A45 7219                   <1>     jb      short pass_set_day_31
00007A47 3C31                   <1>     cmp     al, '1'
                                <1>         ;ja loc_set_date_stc_1
                                <1>     ; 26/07/2022
00007A49 7615                   <1>     jna     short pass_set_day_31
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_date_stc_1:
00007A4B E8E4010000             <1>     call    check_for_backspace
00007A50 7407                   <1>     je      short loc_set_date_bs_1
                                <1>     ;xor    bh, bh ; video page 0
00007A52 E872A9FFFF             <1>     call    beeper ; BEEP !
00007A57 EBCD                   <1>     jmp     short loc_enter_day_2
                                <1> loc_set_date_bs_1:
00007A59 E8C4010000             <1>     call    write_backspace
00007A5E EB92                   <1>     jmp     short loc_enter_day_1
                                <1>
                                <1> pass_set_day_31:
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007A60 B307                   <1>     mov     bl, 7
00007A62 E878A8FFFF             <1>     call    _write_tty
                                <1> loc_enter_separator_1:
00007A67 28E4                   <1>     sub     ah, ah ; 0
00007A69 E8A494FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007A6E 3C1B                   <1>     cmp     al, 27
                                <1>         ;je loc_set_date_retn
                                <1>     ; 26/07/2022
00007A70 748F                   <1>     je      short set_date_0
00007A72 3C2D                   <1>     cmp     al, '-'
00007A74 7443                   <1>     je      short pass_set_date_separator_1
00007A76 3C2F                   <1>     cmp     al, '/'
                                <1>     ;jne    loc_set_date_stc_2
                                <1>     ; 26/07/2022
00007A78 743F                   <1>     je      short pass_set_date_separator_1
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_date_stc_2:
00007A7A E8B5010000             <1>     call    check_for_backspace
00007A7F 7407                   <1>     je      short loc_set_date_bs_2
                                <1>     ;xor    bh, bh ; video page 0
00007A81 E843A9FFFF             <1>     call    beeper ; BEEP !
00007A86 EBDF                   <1>     jmp     short loc_enter_separator_1
                                <1> loc_set_date_bs_2:
00007A88 E895010000             <1>     call    write_backspace
00007A8D EB97                   <1>     jmp     short loc_enter_day_2
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_date_stc_3:
00007A8F E8A0010000             <1>     call    check_for_backspace
00007A94 7407                   <1>     je      short loc_set_date_bs_3
                                <1>     ;xor    bh, bh ; video page 0
00007A96 E82EA9FFFF             <1>     call    beeper ; BEEP !
```

```
00007A9B EB23                <1>     jmp     short loc_enter_month_1
                             <1> loc_set_date_bs_3:
00007A9D E880010000          <1>     call    write_backspace
00007AA2 EBC3                <1>     jmp     short loc_enter_separator_1
                             <1>
                             <1>     ; 26/07/2022
                             <1> loc_set_date_stc_4:
00007AA4 E88B010000          <1>     call    check_for_backspace
00007AA9 7407                <1>     je      short loc_set_date_bs_4
                             <1>     ;xor    bh, bh ; video page 0
00007AAB E819A9FFFF          <1>     call    beeper ; BEEP !
00007AB0 EB2D                <1>     jmp     short loc_enter_month_2
                             <1> loc_set_date_bs_4:
00007AB2 E86B010000          <1>     call    write_backspace
00007AB7 EB07                <1>     jmp     short loc_enter_month_1
                             <1>
                             <1> pass_set_date_separator_1:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007AB9 B307                <1>     mov     bl, 7
00007ABB E81FA8FFFF          <1>     call    _write_tty
                             <1> loc_enter_month_1:
00007AC0 30E4                <1>     xor      ah, ah ; 0
00007AC2 E84B94FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007AC7 3C1B                <1>     cmp     al, 27
                             <1>         ;je loc_set_date_retn
                             <1>     ; 26/07/2022
                             <1>     ;je     short loc_set_date_ok
                             <1>     ; 07/08/2022
00007AC9 741F                <1>     je      short jmp_loc_set_date_ok
00007ACB A2[D92F0100]        <1>     mov     [Month], al
00007AD0 3C30                <1>     cmp     al, '0'
00007AD2 72BB                <1>         jb      short loc_set_date_stc_3
00007AD4 3C31                <1>     cmp     al, '1'
00007AD6 77B7                <1>         ja  short loc_set_date_stc_3
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007AD8 B307                <1>     mov     bl, 7
00007ADA E800A8FFFF          <1>     call    _write_tty
                             <1> loc_enter_month_2:
00007ADF 30E4                <1>     xor      ah, ah
00007AE1 E82C94FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007AE6 3C1B                <1>     cmp     al, 27
                             <1>         ;;je       loc_set_date_retn
                             <1>     ; 26/07/2022
                             <1>     ;je     short loc_set_date_ok
                             <1>     ; 07/08/2022
00007AE8 7505                <1>     jne     short loc_enter_month_3
                             <1> jmp_loc_set_date_ok:
00007AEA E996000000          <1>     jmp     loc_set_date_ok
                             <1> loc_enter_month_3:
00007AEF A2[DA2F0100]        <1>     mov     [Month+1], al
00007AF4 3C30                <1>     cmp     al, '0'
00007AF6 72AC                <1>         jb  short loc_set_date_stc_4
00007AF8 3C39                <1>     cmp     al, '9'
00007AFA 77A8                <1>         ja  short loc_set_date_stc_4
00007AFC 803D[D92F0100]31    <1>     cmp     byte [Month], '1'
00007B03 7204                <1>     jb      short pass_set_month_12
00007B05 3C32                <1>     cmp     al, '2'
00007B07 779B                <1>         ja  short loc_set_date_stc_4
                             <1> pass_set_month_12:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007B09 B307                <1>     mov     bl, 7
00007B0B E8CFA7FFFF          <1>     call    _write_tty
                             <1> loc_enter_separator_2:
00007B10 28E4                <1>     sub      ah, ah
00007B12 E8FB93FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007B17 3C1B                <1>     cmp     al, 27
                             <1>         ;je loc_set_date_retn
                             <1>     ; 26/07/2022
00007B19 746A                <1>     je      short loc_set_date_ok
00007B1B 3C2D                <1>     cmp     al, '-'
00007B1D 7404                <1>     je      short pass_set_date_separator_2
00007B1F 3C2F                <1>     cmp     al, '/'
00007B21 756C                <1>         jne short loc_set_date_stc_5
                             <1> pass_set_date_separator_2:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007B23 B307                <1>     mov     bl, 7
00007B25 E8B5A7FFFF          <1>     call    _write_tty
                             <1> loc_enter_year_1:
00007B2A 30E4                <1>     xor     ah, ah
00007B2C E8E193FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007B31 3C1B                <1>     cmp     al, 27
                             <1>         ;je loc_set_date_retn
                             <1>     ; 26/07/2022
00007B33 7450                <1>     je      short loc_set_date_ok
00007B35 A2[DE2F0100]        <1>     mov     [Year], al
00007B3A 3C30                <1>     cmp     al, '0'
00007B3C 726C                <1>         jb  short loc_set_date_stc_6
00007B3E 3C39                <1>     cmp     al, '9'
00007B40 7768                <1>         ja  short loc_set_date_stc_6
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007B42 B307                <1>     mov     bl, 7
00007B44 E896A7FFFF          <1>     call    _write_tty
                             <1> loc_enter_year_2:
00007B49 30E4                <1>     xor     ah, ah
00007B4B E8C293FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007B50 3C1B                <1>     cmp     al, 27
                             <1>     ;je     short loc_set_date_retn
                             <1>     ; 26/07/2022
00007B52 7431                <1>     je      short loc_set_date_ok
00007B54 A2[DF2F0100]        <1>     mov     byte [Year+1], al
00007B59 3C30                <1>     cmp     al, '0'
00007B5B 7268                <1>         jb  short loc_set_date_stc_7
00007B5D 3C39                <1>     cmp     al, '9'
00007B5F 7764                <1>         ja  short loc_set_date_stc_7
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007B61 B307                <1>     mov     bl, 7
00007B63 E877A7FFFF          <1>     call    _write_tty
```

```
                                <1> loc_set_date_get_lchar_again:
00007B68 28E4                   <1>     sub     ah, ah ; 0
00007B6A E8A393FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007B6F 3C0D                   <1>     cmp     al, 13 ; ENTER key
00007B71 746D                   <1>     je      short loc_set_date_progress
00007B73 3C1B                   <1>     cmp     al, 27 ; ESC key
                                <1>     ;je     short loc_set_date_retn
                                <1>     ; 26/07/2022
00007B75 740E                   <1>     je      short loc_set_date_ok
                                <1>     ;
00007B77 E8B8000000             <1>     call    check_for_backspace
00007B7C 75EA                   <1>     jne     short loc_set_date_get_lchar_again
                                <1>
                                <1> loc_set_date_bs_8:
00007B7E E89F000000             <1>     call    write_backspace
00007B83 EBC4                   <1>     jmp     short loc_enter_year_2
                                <1>
                                <1> loc_set_date_ok:
                                <1> ;loc_set_date_retn:
00007B85 BE[17390100]           <1>     mov     esi, nextline
                                <1>     ;call   print_msg
                                <1>     ;retn
                                <1>     ; 26/07/2022
00007B8A E92BF2FFFF             <1>     jmp     print_msg
                                <1>
                                <1>     ; 26/07/2022
                                <1> ;loc_set_date_stc_0:
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_day_1
                                <1> ;loc_set_date_stc_1:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_date_bs_1
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_day_2
                                <1> ;loc_set_date_bs_1:
                                <1> ;   call    write_backspace
                                <1> ;   jmp      loc_enter_day_1
                                <1> ;loc_set_date_stc_2:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_date_bs_2
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_separator_1
                                <1> ;loc_set_date_bs_2:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_day_2
                                <1> ;loc_set_date_stc_3:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_date_bs_3
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_month_1
                                <1> ;loc_set_date_bs_3:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_separator_1
                                <1> ;loc_set_date_stc_4:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_date_bs_4
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_month_2
                                <1> ;loc_set_date_bs_4:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_month_1
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_date_stc_5:
00007B8F E8A0000000             <1>     call    check_for_backspace
00007B94 740A                   <1>     je      short loc_set_date_bs_5
                                <1>     ;xor    bh, bh ; video page 0
00007B96 E82EA8FFFF             <1>     call    beeper ; BEEP !
00007B9B E970FFFFFF             <1>     jmp     loc_enter_separator_2
                                <1> loc_set_date_bs_5:
00007BA0 E87D000000             <1>     call    write_backspace
00007BA5 E935FFFFFF             <1>     jmp     loc_enter_month_2
                                <1> loc_set_date_stc_6:
00007BAA E885000000             <1>     call    check_for_backspace
00007BAF 740A                   <1>         je      short loc_set_date_bs_6
                                <1>     ;xor    bh, bh ; video page 0
00007BB1 E813A8FFFF             <1>     call    beeper ; BEEP !
00007BB6 E96FFFFFFF             <1>     jmp     loc_enter_year_1
                                <1> loc_set_date_bs_6:
00007BBB E862000000             <1>     call    write_backspace
00007BC0 E94BFFFFFF             <1>     jmp     loc_enter_separator_2
                                <1> loc_set_date_stc_7:
00007BC5 E86A000000             <1>     call    check_for_backspace
00007BCA 740A                   <1>     je      short loc_set_date_bs_7
                                <1>     ;xor    bh, bh ; video page 0
00007BCC E8F8A7FFFF             <1>     call    beeper ; BEEP !
00007BD1 E973FFFFFF             <1>     jmp     loc_enter_year_2
                                <1> loc_set_date_bs_7:
00007BD6 E847000000             <1>     call    write_backspace
00007BDB E94AFFFFFF             <1>     jmp     loc_enter_year_1
                                <1>
                                <1> loc_set_date_progress:
                                <1>     ; Get Current Date
                                <1>     ;mov    ah, 04h
                                <1>     ;call   int1Ah
00007BE0 E8E3E7FFFF             <1>     call    RTC_40  ; GET RTC DATE
                                <1>     ; CH = century (in BCD)
                                <1>
00007BE5 66A1[DE2F0100]         <1>     mov     ax, [Year]
00007BEB 662D3030               <1>     sub     ax, '00'
00007BEF C0E004                 <1>     shl     al, 4 ; * 16
00007BF2 88C1                   <1>     mov     cl, al
00007BF4 00E1                   <1>     add     cl, ah
00007BF6 66A1[D92F0100]         <1>     mov     ax, [Month]
00007BFC 662D3030               <1>     sub     ax, '00'
00007C00 C0E004                 <1>     shl     al, 4 ; * 16
00007C03 88C6                   <1>     mov     dh, al
00007C05 00E6                   <1>     add     dh, ah
00007C07 66A1[D62F0100]         <1>     mov     ax, [Day]
00007C0D 662D3030               <1>     sub     ax, '00'
00007C11 C0E004                 <1>     shl     al, 4 ; * 16
00007C14 88C2                   <1>     mov     dl, al
00007C16 00E2                   <1>     add     dl, ah
                                <1>
                                <1>     ;mov    ah, 05h
                                <1>     ;call   int1Ah
00007C18 E8D1E7FFFF             <1>     call    RTC_50  ; SET RTC DATE
                                <1>
```

```
                                <1> ;loc_set_date_retn:
                                <1> ;   mov     esi, nextline
                                <1> ;   ;call   print_msg
                                <1> ;   ;retn
                                <1> ;   ; 26/07/2022
                                <1> ;   jmp     print_msg
                                <1>
                                <1>     ; 26/07/2022
00007C1D E963FFFFFF              <1>     jmp     loc_set_date_ok
                                <1>
                                <1> write_backspace:
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
00007C22 B008                    <1>     mov     al, 08h ; BACKSPACE
                                <1>     ; 13/05/2016
00007C24 66BB0700                <1>     mov     bx, 7 ; bl = attribute/color
                                <1>                   ; bh = video page = 0
00007C28 E8B2A6FFFF              <1>     call    _write_tty
00007C2D B020                    <1>     mov     al, 20h ; BLANK/SPACE char
                                <1>     ;mov    bx, 7 ; attribute/color
                                <1>     ;call   _write_c_current
                                <1>     ;retn
00007C2F E92AA6FFFF              <1>     jmp     _write_c_current
                                <1>
                                <1> check_for_backspace:
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
00007C34 663D080E                <1>     cmp     ax, 0E08h
00007C38 7410                    <1>     je      short cfbs_retn
00007C3A 663DE04B                <1>     cmp     ax, 4BE0h
00007C3E 740A                    <1>     je      short cfbs_retn
00007C40 663D004B                <1>     cmp     ax, 4B00h
00007C44 7404                    <1>     je      short cfbs_retn
00007C46 663DE053                <1>     cmp     ax, 53E0h
                                <1> cfbs_retn:
00007C4A C3                      <1>     retn
                                <1>
                                <1> show_time:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>         ; 2004-2005
                                <1>
                                <1>     ;mov    ah, 02h
                                <1>     ;call   int1Ah
00007C4B E80DE7FFFF              <1>     call    RTC_20  ; GET RTC TIME
                                <1>
00007C50 88E8                    <1>     mov     al, ch
00007C52 E8AE92FFFF              <1>     call    bcd_to_ascii
00007C57 66A3[04300100]          <1>     mov     [Hour], ax
                                <1>
00007C5D 88C8                    <1>     mov     al, cl
00007C5F E8A192FFFF              <1>     call    bcd_to_ascii
00007C64 66A3[07300100]          <1>     mov     [Minute], ax
                                <1>
00007C6A 88F0                    <1>     mov     al, dh
00007C6C E89492FFFF              <1>     call    bcd_to_ascii
00007C71 66A3[0A300100]          <1>     mov     [Second], ax
                                <1>
00007C77 BE[F42F0100]            <1>     mov     esi, Msg_Show_Time
                                <1>     ;call   print_msg
                                <1>     ;retn
                                <1>     ; 26/07/2022
00007C7C E939F1FFFF              <1>     jmp     print_msg
                                <1>
                                <1> set_time:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/05/2016
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>         ; 2004-2005
                                <1>
00007C81 BE[E32F0100]            <1>     mov     esi, Msg_Enter_Time
00007C86 E82FF1FFFF              <1>     call    print_msg
                                <1>
                                <1> loc_enter_hour_1:
00007C8B 30E4                    <1>     xor     ah, ah
00007C8D E88092FFFF              <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007C92 3C0D                    <1>     cmp     al, 13 ; ENTER key
00007C94 742D                    <1>         je  short loc_set_time_retn
00007C96 3C1B                    <1>     cmp     al, 27 ; ESC key
00007C98 7429                    <1>         je  short loc_set_time_retn
                                <1> set_time_0:
00007C9A A2[04300100]            <1>     mov     [Hour], al
00007C9F 3C30                    <1>     cmp     al, '0'
00007CA1 7204                    <1>         jb  short loc_set_time_stc_0
00007CA3 3C32                    <1>     cmp     al, '2'
                                <1>     ;ja     loc_set_time_stc_0
                                <1>     ; 26/07/2022
00007CA5 7626                    <1>     jna     short set_time_1
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_time_stc_0:
                                <1>     ;xor    bh, bh ; video page 0
00007CA7 E81DA7FFFF              <1>     call    beeper ; BEEP !
00007CAC EBDD                    <1>     jmp     short loc_enter_hour_1
                                <1>
                                <1> loc_set_time_stc_1:
00007CAE E881FFFFFF              <1>     call    check_for_backspace
00007CB3 7407                    <1>     je      short loc_set_time_bs_1
                                <1>     ;xor    bh, bh ; video page 0
00007CB5 E80FA7FFFF              <1>     call    beeper ; BEEP !
00007CBA EB18                    <1>     jmp     short loc_enter_hour_2
                                <1> loc_set_time_bs_1:
00007CBC E861FFFFFF              <1>     call    write_backspace
00007CC1 EBC8                    <1>     jmp     short loc_enter_hour_1
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_time_retn:
00007CC3 BE[17390100]            <1>     mov     esi, nextline
                                <1>     ;call   print_msg
                                <1>     ;retn
00007CC8 E9EDF0FFFF              <1>     jmp     print_msg
                                <1>
                                <1> set_time_1:
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007CCD B307                    <1>     mov     bl, 7
00007CCF E80BA6FFFF              <1>     call    _write_tty
                                <1> loc_enter_hour_2:
00007CD4 30E4                    <1>     xor     ah, ah
00007CD6 E83792FFFF              <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007CDB 3C1B                    <1>     cmp     al, 27
00007CDD 74E4                    <1>     je      short loc_set_time_retn
00007CDF A2[05300100]            <1>     mov     [Hour+1], al
```

```
00007CE4 3C30                <1>     cmp     al, '0'
00007CE6 72C6                <1>     jb      short loc_set_time_stc_1
00007CE8 3C39                <1>     cmp     al, '9'
00007CEA 77C2                <1>     ja      short loc_set_time_stc_1
00007CEC 803D[04300100]32    <1>         cmp     byte [Hour], '2'
00007CF3 7204                <1>     jb      short pass_set_time_24
00007CF5 3C34                <1>     cmp     al, '4'
00007CF7 77B5                <1>     ja      short loc_set_time_stc_1
                             <1> pass_set_time_24:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007CF9 B307                <1>     mov     bl, 7
00007CFB E8DFA5FFFF          <1>     call    _write_tty
                             <1> loc_enter_time_separator_1:
00007D00 28E4                <1>     sub     ah, ah ; 0
00007D02 E80B92FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007D07 3C1B                <1>     cmp     al, 27
00007D09 74B8                <1>     je      short loc_set_time_retn
00007D0B 3C3A                <1>     cmp     al, ':'
                             <1>     ;jne    loc_set_time_stc_2
                             <1>     ; 26/07/2022
00007D0D 7415                <1>     je      short set_time_2
                             <1>
                             <1>     ; 26/07/2022
                             <1> loc_set_time_stc_2:
00007D0F E820FFFFFF          <1>     call    check_for_backspace
00007D14 7407                <1>     je      short loc_set_time_bs_2
                             <1>     ;xor    bh, bh ; video page 0
00007D16 E8AEA6FFFF          <1>     call    beeper ; BEEP !
00007D1B EBE3                <1>     jmp     short loc_enter_time_separator_1
                             <1> loc_set_time_bs_2:
00007D1D E800FFFFFF          <1>     call    write_backspace
00007D22 EBB0                <1>     jmp     short loc_enter_hour_2
                             <1>
                             <1> set_time_2:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007D24 B307                <1>     mov     bl, 7
00007D26 E8B4A5FFFF          <1>     call    _write_tty
                             <1> loc_enter_minute_1:
00007D2B 30E4                <1>     xor      ah, ah
00007D2D E8E091FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007D32 3C1B                <1>     cmp     al, 27
00007D34 748D                <1>     je      short loc_set_time_retn
00007D36 A2[07300100]        <1>     mov     [Minute], al
00007D3B 3C30                <1>     cmp     al, '0'
00007D3D 7204                <1>     jb      short loc_set_time_stc_3
00007D3F 3C35                <1>     cmp     al, '5'
                             <1>     ;ja     loc_set_time_stc_3
                             <1>     ; 26/07/2022
00007D41 7615                <1>     jna     short set_time_3
                             <1>
                             <1>     ; 26/07/2022
                             <1> loc_set_time_stc_3:
00007D43 E8ECFEFFFF          <1>     call    check_for_backspace
00007D48 7407                <1>     je      short loc_set_time_bs_3
                             <1>     ;xor    bh, bh ; video page 0
00007D4A E87AA6FFFF          <1>     call    beeper ; BEEP !6
00007D4F EBDA                <1>     jmp     short loc_enter_minute_1
                             <1> loc_set_time_bs_3:
00007D51 E8CCFEFFFF          <1>     call    write_backspace
00007D56 EBA8                <1>     jmp     short loc_enter_time_separator_1
                             <1>
                             <1> set_time_3:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007D58 B307                <1>     mov     bl, 7
00007D5A E880A5FFFF          <1>     call    _write_tty
                             <1> loc_enter_minute_2:
00007D5F 30E4                <1>     xor      ah, ah
00007D61 E8AC91FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007D66 3C1B                <1>     cmp     al, 27
                             <1>     ;je     short loc_set_time_retn
                             <1>     ; 07/08/2022
00007D68 7505                <1>     jne     short loc_enter_minute_3
00007D6A E954FFFFFF          <1>     jmp     loc_set_time_retn
                             <1> loc_enter_minute_3:
00007D6F A2[08300100]        <1>     mov     [Minute+1], al
00007D74 3C30                <1>     cmp     al, '0'
00007D76 7204                <1>     jb      short loc_set_time_stc_4
00007D78 3C39                <1>     cmp     al, '9'
                             <1>     ;ja     loc_set_time_stc_4
                             <1>     ; 26/07/2022
00007D7A 7615                <1>     jna     short set_time_4
                             <1>
                             <1>     ; 26/07/2022
                             <1> loc_set_time_stc_4:
00007D7C E8B3FEFFFF          <1>     call    check_for_backspace
00007D81 7407                <1>     je      short loc_set_time_bs_4
                             <1>     ;xor    bh, bh ; video page 0
00007D83 E841A6FFFF          <1>     call    beeper ; BEEP !
00007D88 EBD5                <1>     jmp     short loc_enter_minute_2
                             <1> loc_set_time_bs_4:
00007D8A E893FEFFFF          <1>     call    write_backspace
00007D8F EB9A                <1>     jmp     short loc_enter_minute_1
                             <1>
                             <1> set_time_4:
                             <1>     ; 13/05/2016
                             <1>     ;mov    bx, 7 ; attribute/color (bl)
                             <1>                   ; video page 0 (bh)
00007D91 B307                <1>     mov     bl, 7
00007D93 E847A5FFFF          <1>     call    _write_tty
                             <1> loc_enter_time_separator_2:
00007D98 66C705[0A300100]30- <1>     mov     word [Second], 3030h
00007DA0 30                  <1>
00007DA1 28E4                <1>     sub      ah, ah
00007DA3 E86A91FFFF          <1>     call    int16h
                             <1>     ; AL = ASCII Code of the Character
00007DA8 3C0D                <1>     cmp     al, 13
                             <1>         ;je short loc_set_time_progress
                             <1>     ; 07/08/2022
00007DAA 7505                <1>     jne     short loc_enter_time_separator_3
                             <1> jmp_loc_set_time_progress:
00007DAC E9D4000000          <1>     jmp     loc_set_time_progress
                             <1> loc_enter_time_separator_3:
00007DB1 3C1B                <1>     cmp     al, 27
                             <1>     ;;je    loc_set_time_retn
                             <1>     ; 26/07/2022
```

```
                                <1>     ;je     short loc_set_time_ok
                                <1>     ; 07/08/2022
00007DB3 7505                   <1>     jne     short loc_enter_time_separator_4
00007DB5 E982000000             <1>     jmp     loc_set_time_ok
                                <1> loc_enter_time_separator_4:
00007DBA 3C3A                   <1>     cmp     al, ':'
00007DBC 7563                   <1>         jne short loc_set_time_stc_5
                                <1>
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007DBE B307                   <1>     mov     bl, 7
00007DC0 E81AA5FFFF             <1>     call    _write_tty
                                <1> loc_enter_second_1:
00007DC5 30E4                   <1>     xor      ah, ah
00007DC7 E84691FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007DCC 3C0D                   <1>     cmp     al, 13
                                <1>     ;je     short loc_set_time_progress
                                <1>     ; 07/08/2022
00007DCE 74DC                   <1>     je      short jmp_loc_set_time_progress
00007DD0 3C1B                   <1>     cmp     al, 27
                                <1>     ;;je    loc_set_time_retn
                                <1>     ; 26/07/2022
                                <1>     ;je     short loc_set_time_ok
                                <1>     ; 07/08/2022
00007DD2 7502                   <1>     jne     short loc_enter_second_2
00007DD4 EB66                   <1>     jmp     loc_set_time_ok
                                <1> loc_enter_second_2:
00007DD6 A2[0A300100]           <1>     mov     [Second], al
00007DDB 3C30                   <1>     cmp     al, '0'
00007DDD 7267                   <1>     jb      short loc_set_time_stc_6
00007DDF 3C35                   <1>     cmp     al, '5'
00007DE1 7763                   <1>     ja      short loc_set_time_stc_6
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007DE3 B307                   <1>     mov     bl, 7
00007DE5 E8F5A4FFFF             <1>     call    _write_tty
                                <1> loc_enter_second_3:
00007DEA 30E4                   <1>     xor      ah, ah
00007DEC E82191FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007DF1 3C1B                   <1>     cmp     al, 27
                                <1>     ;je     short loc_set_time_retn
                                <1>     ; 26/07/2022
00007DF3 7447                   <1>     je      short loc_set_time_ok
00007DF5 3C30                   <1>     cmp     al, '0'
00007DF7 7271                   <1>         jb  short loc_set_time_stc_7
00007DF9 3C39                   <1>     cmp     al, '9'
00007DFB 776D                   <1>         ja  short loc_set_time_stc_7
                                <1>     ; 13/05/2016
                                <1>     ;mov    bx, 7 ; attribute/color (bl)
                                <1>                   ; video page 0 (bh)
00007DFD B307                   <1>     mov     bl, 7
00007DFF E8DBA4FFFF             <1>     call    _write_tty
                                <1> loc_set_time_get_lchar_again:
00007E04 28E4                   <1>     sub     ah, ah ; 0
00007E06 E80791FFFF             <1>     call    int16h
                                <1>     ; AL = ASCII Code of the Character
00007E0B 3C0D                   <1>     cmp     al, 13
00007E0D 7476                   <1>     je      short loc_set_time_progress
00007E0F 3C1B                   <1>     cmp     al, 27
                                <1>     ;je     short loc_set_time_retn
                                <1>     ; 07/08/2022
00007E11 7429                   <1>     je      short loc_set_time_ok
                                <1>     ;
00007E13 E81CFEFFFF             <1>     call    check_for_backspace
00007E18 75EA                   <1>     jne     short loc_set_time_get_lchar_again
                                <1>
                                <1> loc_set_time_bs_8:
00007E1A E803FEFFFF             <1>     call    write_backspace
00007E1F EBC9                   <1>     jmp     short loc_enter_second_3
                                <1>
                                <1> ;   ; 26/07/2022
                                <1> ;loc_set_time_retn:
                                <1> ;   mov     esi, nextline
                                <1> ;   ;call   print_msg
                                <1> ;   ;retn
                                <1> ;   jmp     print_msg
                                <1>
                                <1>     ; 26/07/2022
                                <1> ;loc_set_time_stc_0:
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_hour_1
                                <1> ;loc_set_time_stc_1:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_time_bs_1
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_hour_2
                                <1> ;loc_set_time_bs_1:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_hour_1
                                <1> ;loc_set_time_stc_2:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_time_bs_2
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_time_separator_1
                                <1> ;loc_set_time_bs_2:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_hour_2
                                <1> ;loc_set_time_stc_3:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_time_bs_3
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !6
                                <1> ;   jmp     loc_enter_minute_1
                                <1> ;loc_set_time_bs_3:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_time_separator_1
                                <1> ;loc_set_time_stc_4:
                                <1> ;   call    check_for_backspace
                                <1> ;   je      short loc_set_time_bs_4
                                <1> ;   ;xor    bh, bh ; video page 0
                                <1> ;   call    beeper ; BEEP !
                                <1> ;   jmp     loc_enter_minute_2
                                <1> ;loc_set_time_bs_4:
                                <1> ;   call    write_backspace
                                <1> ;   jmp     loc_enter_minute_1
                                <1>
                                <1>     ; 26/07/2022
```

```
                                <1> loc_set_time_stc_5:
00007E21 E80EFEFFFF             <1>     call    check_for_backspace
00007E26 740A                   <1>     je      short loc_set_time_bs_5
                                <1>     ;xor    bh, bh ; video page 0
00007E28 E89CA5FFFF             <1>     call    beeper ; BEEP !
00007E2D E966FFFFFF             <1>     jmp     loc_enter_time_separator_2
                                <1> loc_set_time_bs_5:
00007E32 E8EBFDFFFF             <1>     call    write_backspace
00007E37 E923FFFFFF             <1>     jmp     loc_enter_minute_2
                                <1>
                                <1>     ; 26/07/2022
                                <1> loc_set_time_ok:
00007E3C BE[17390100]           <1>     mov     esi, nextline
                                <1>     ;call   print_msg
                                <1>     ;retn
00007E41 E974EFFFFF             <1>     jmp     print_msg
                                <1>
                                <1>     ; 07/08/2022
                                <1> loc_set_time_stc_6:
00007E46 E8E9FDFFFF             <1>     call    check_for_backspace
00007E4B 7413                   <1>     je      short loc_set_time_bs_6
                                <1>     ;xor    bh, bh ; video page 0
00007E4D E877A5FFFF             <1>     call    beeper ; BEEP !
00007E52 66C705[0A300100]30-    <1>     mov     word [Second], 3030h
00007E5A 30                     <1>
00007E5B E965FFFFFF             <1>     jmp     loc_enter_second_1
                                <1> loc_set_time_bs_6:
00007E60 E8BDFDFFFF             <1>     call    write_backspace
00007E65 E92EFFFFFF             <1>     jmp     loc_enter_time_separator_2
                                <1> loc_set_time_stc_7:
00007E6A E8C5FDFFFF             <1>     call    check_for_backspace
00007E6F 740A                   <1>     je      short loc_set_time_bs_7
                                <1>     ;xor    bh, bh ; video page 0
00007E71 E853A5FFFF             <1>     call    beeper ; BEEP !
00007E76 E96FFFFFFF             <1>     jmp     loc_enter_second_3
                                <1> loc_set_time_bs_7:
00007E7B E8A2FDFFFF             <1>     call    write_backspace
00007E80 E940FFFFFF             <1>     jmp     loc_enter_second_1
                                <1>
                                <1> loc_set_time_progress:
                                <1>     ; Get Current Time
                                <1>     ;mov    ah, 02h
                                <1>     ;call   int1Ah
00007E85 E8D3E4FFFF             <1>     call    RTC_20  ; GET RTC TIME
                                <1>     ;DL = Daylight Savings Enable option (0-1)
                                <1>
00007E8A 66A1[04300100]         <1>     mov     ax, [Hour]
00007E90 662D3030               <1>     sub     ax, '00'
00007E94 C0E004                 <1>     shl     al, 4 ; * 16
00007E97 88C5                   <1>     mov     ch, al
00007E99 00E5                   <1>     add     ch, ah
00007E9B 66A1[07300100]         <1>     mov     ax, [Minute]
00007EA1 662D3030               <1>     sub     ax, '00'
00007EA5 C0E004                 <1>     shl     al, 4 ; * 16
00007EA8 88C1                   <1>     mov     cl, al
00007EAA 00E1                   <1>     add     cl, ah
00007EAC 66A1[0A300100]         <1>     mov     ax, [Second]
00007EB2 662D3030               <1>     sub     ax, '00'
00007EB6 C0E004                 <1>     shl     al, 4 ; * 16
00007EB9 88C6                   <1>     mov     dh, al
00007EBB 00E6                   <1>     add     dh, ah
                                <1>
                                <1>     ;mov    ah, 03h
                                <1>     ;call   int1Ah
00007EBD E8C9E4FFFF             <1>     call    RTC_30  ; SET RTC TIME
                                <1>
                                <1>     ; 26/07/2022
00007EC2 E975FFFFFF             <1>     jmp     loc_set_time_ok
                                <1>
                                <1> print_volume_info:
                                <1>     ; 19/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 01/03/2016
                                <1>     ; 08/02/2016
                                <1>     ; 06/02/2016
                                <1>     ; 04/02/2016
                                <1>     ; 18/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 25/10/2009
                                <1>     ;
                                <1>     ; "Volume Serial No: "
                                <1>     ;
                                <1>     ; INPUT  : AL = DOS Drive Number
                                <1>     ; OUTPUT : AH = FS Type
                                <1>     ;          AL = DOS Drive Name
                                <1>     ; CF = 0 -> OK
                                <1>     ; CF = 1 -> Drive not ready
                                <1>
00007EC7 88C4                   <1>     mov     ah, al
00007EC9 28C0                   <1>     sub     al, al
00007ECB 0FB7F0                 <1>     movzx   esi, ax
00007ECE 81C600010900           <1>     add     esi, Logical_DOSDisks
00007ED4 8A06                   <1>     mov     al, [esi]
00007ED6 3C41                   <1>     cmp     al, 'A'
00007ED8 7304                   <1>     jnb     short loc_pvi_set_vol_name
00007EDA 8A6604                 <1>     mov     ah, [esi+LD_FSType]
00007EDD C3                     <1>     retn
                                <1>
                                <1> loc_pvi_set_vol_name:
00007EDE A2[3E300100]           <1>     mov     [Vol_Drv_Name], al
00007EE3 56                     <1>     push    esi
00007EE4 E833010000             <1>     call    move_volume_name_and_serial_no ;;;
00007EE9 7302                   <1>     jnc     short loc_pvi_mvn_ok
00007EEB 5E                     <1>     pop     esi
00007EEC C3                     <1>     retn
                                <1>
                                <1> loc_pvi_mvn_ok:
00007EED 8B3424                 <1>     mov     esi, [esp]
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FSType], 0A1h
                                <1>     jne     short loc_pvi_fat_vol_size
                                <1>     mov     eax, [esi+LD_FS_VolumeSize]
                                <1>     ; 19/12/2025
                                <1>     ;movzx  ebx, word [esi+LD_FS_BytesPerSec]
                                <1>     jmp     short loc_vol_size_mul32
                                <1> %endif
                                <1>
                                <1> loc_pvi_fat_vol_size:
00007EF0 8B4670                 <1>     mov     eax, [esi+LD_TotalSectors]
00007EF3 8B4670                 <1>     mov     eax, [esi+LD_TotalSectors]
                                <1>     ; 19/12/2025
                                <1>     ;movzx  ebx, word [esi+LD_BPB+BPB_BytsPerSec]
                                <1> loc_vol_size_mul32:
                                <1>     ; 19/12/2025
```

```
00007EF6 BB00020000          <1>     mov     ebx, 512
00007EFB F7E3                <1>     mul     ebx
00007EFD 09D2                <1>     or      edx, edx
00007EFF 7507                <1>     jnz     short loc_vol_size_in_kbytes
                             <1> loc_vol_size_in_bytes:
00007F01 B9[1C300100]        <1>     mov     ecx, VolSize_Bytes
00007F06 EB0B                <1>     jmp     short loc_write_vol_size_str
                             <1> loc_vol_size_in_kbytes:
                             <1>     ;mov    bx, 1024
                             <1>     ; 19/12/2025
00007F08 D1E3                <1>     shl     ebx, 1 ; ebx = 1024
00007F0A F7F3                <1>     div     ebx
00007F0C B9[0F300100]        <1>     mov     ecx, VolSize_KiloBytes
00007F11 31D2                <1>     xor     edx, edx ; 0
                             <1> loc_write_vol_size_str:
00007F13 890D[AC7E0100]      <1>     mov     [VolSize_Unit1], ecx
                             <1>     ;
00007F19 BF[C27E0100]        <1>     mov     edi, Vol_Tot_Sec_Str_End
                             <1>         ;mov       byte [edi], 0
00007F1E B90A000000          <1>     mov     ecx, 10
                             <1> loc_write_vol_size_chr:
00007F23 F7F1                <1>     div     ecx
00007F25 80C230              <1>     add     dl, '0'
00007F28 4F                  <1>     dec     edi
00007F29 8817                <1>     mov     [edi], dl
00007F2B 85C0                <1>     test    eax, eax
00007F2D 7404                <1>     jz      short loc_write_vol_size_str_ok
00007F2F 28D2                <1>     sub     dl, dl ; 0
00007F31 EBF0                <1>     jmp     short loc_write_vol_size_chr
                             <1>
                             <1> loc_write_vol_size_str_ok:
00007F33 893D[B47E0100]      <1>     mov     [Vol_Tot_Sec_Str_Start], edi
                             <1>     ;
00007F39 BF[27300100]        <1>     mov     edi, Vol_FS_Name
00007F3E 8A4E03              <1>     mov     cl, [esi+LD_FATType]
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     and     cl, cl ; 0 ?
                             <1>     jnz     short loc_write_vol_FAT_str_1
                             <1>     mov     word [edi], 'TR'
                             <1>     mov     dword [edi+4], ' FS1'
                             <1>     ; 19/12/2025
                             <1>     ;;movzx ebx, word [esi+LD_FS_BytesPerSec]
                             <1>     ;mov    bx, [esi+LD_FS_BytesPerSec]
                             <1>     mov     eax, [esi+LD_FS_FreeSectors]
                             <1>     jmp     short loc_vol_freespace_mul32
                             <1> %endif
                             <1>
                             <1> loc_write_vol_FAT_str_1:
00007F41 66B83332            <1>     mov     ax, '32' ; FAT32
00007F45 80F902              <1>     cmp     cl, 2 ; [esi+LD_FATType]
00007F48 7708                <1>     ja      short loc_write_vol_FAT_str_2
00007F4A 66B83132            <1>     mov     ax, '12' ; FAT12
00007F4E 7202                <1>     jb      short loc_write_vol_FAT_str_2
00007F50 B436                <1>     mov     ah, '6'  ; FAT16
                             <1> loc_write_vol_FAT_str_2:
00007F52 C70746415420        <1>     mov     dword [edi], 'FAT '
00007F58 66894704            <1>     mov     word [edi+4], ax
                             <1>     ; 19/12/2025
                             <1>     ;;movzx ebx, word [esi+LD_BPB+BPB_BytsPerSec]
                             <1>     ;mov    bx, [esi+LD_BPB+BPB_BytsPerSec]
00007F5C 8B4674              <1>     mov     eax, [esi+LD_FreeSectors]
                             <1>
                             <1> loc_vol_freespace_recalc0:
                             <1>     ; 01/03/2016
00007F5F 83F8FF              <1>     cmp     eax, 0FFFFFFFFh
00007F62 7209                <1>     jb      short loc_vol_freespace_mul32
                             <1>     ;inc    eax ; 0
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     and     cl, cl ; byte [esi+LD_FATType]
                             <1>     jz      short loc_vol_freespace_mul32
                             <1> %endif
                             <1>     ; 19/12/2025
                             <1>     ;push   ebx
00007F64 66BB00FF            <1>     mov     bx, 0FF00h ; recalculate free sectors
00007F68 E804460000          <1>     call    calculate_fat_freespace
                             <1>     ;pop    ebx
                             <1>
                             <1>     ; 19/12/2025
                             <1>     ; (if cf = 1, eax = -1)
                             <1>     ; if eax = -1 , free sector count is invalid
                             <1>
                             <1> loc_vol_freespace_mul32:
00007F6D B9[1C300100]        <1>     mov     ecx, VolSize_Bytes
00007F72 BF[D27E0100]        <1>     mov     edi, Vol_Free_Sectors_Str_End
                             <1>     ;mov    byte [edi], 0
00007F77 40                  <1>     inc     eax ; * ; -1 -> 0
00007F78 7506                <1>     jnz     short loc_vol_freespace_mul32_@
00007F7A 4F                  <1>     dec     edi
00007F7B C6073F              <1>     mov     byte [edi], '?'
00007F7E EB32                <1>     jmp     short loc_write_vol_fspace_str_ok
                             <1>
                             <1> loc_vol_freespace_mul32_@:
00007F80 48                  <1>     dec     eax ; *
                             <1>     ; 19/12/2025
00007F81 BB00020000          <1>     mov     ebx, 512
00007F86 F7E3                <1>     mul     ebx
00007F88 09D2                <1>     or      edx, edx
                             <1>     ;jnz    short loc_vol_fspace_in_kbytes
                             <1> loc_vol_fspace_in_bytes:
                             <1>     ; 19/12/2025
                             <1>     ;mov    ecx, VolSize_Bytes
                             <1>     ;jmp    short loc_write_vol_fspace_str
00007F8A 740B                <1>     jz      short loc_write_vol_fspace_str
                             <1> loc_vol_fspace_in_kbytes:
                             <1>     ;mov    bx, 1024
                             <1>     ; 19/12/2025
00007F8C D1E3                <1>     shl     ebx, 1 ; ebx = 1024
00007F8E F7F3                <1>     div     ebx
00007F90 B9[0F300100]        <1>     mov     ecx, VolSize_KiloBytes
00007F95 31D2                <1>     xor     edx, edx ; 0
                             <1> loc_write_vol_fspace_str:
00007F97 890D[B07E0100]      <1>     mov     [VolSize_Unit2], ecx
                             <1>     ;
                             <1>     ; 19/12/2025
                             <1>     ;mov    edi, Vol_Free_Sectors_Str_End
                             <1>         ;mov       byte [edi], 0
00007F9D B90A000000          <1>     mov     ecx, 10
                             <1> loc_write_vol_fspace_chr:
00007FA2 F7F1                <1>     div     ecx
00007FA4 80C230              <1>     add     dl, '0'
```

```
00007FA7 4F                 <1>     dec     edi
00007FA8 8817               <1>     mov     [edi], dl
00007FAA 85C0               <1>     test    eax, eax
00007FAC 7404               <1>     jz      short loc_write_vol_fspace_str_ok
00007FAE 28D2               <1>     sub     dl, dl ; 0
00007FB0 EBF0               <1>     jmp     short loc_write_vol_fspace_chr
                            <1>
                            <1> loc_write_vol_fspace_str_ok:
00007FB2 893D[C47E0100]     <1>     mov     [Vol_Free_Sectors_Str_Start], edi
                            <1>     ;
00007FB8 BE[25300100]       <1>     mov     esi, Volume_in_drive
00007FBD E8F8EDFFFF         <1>     call    print_msg
00007FC2 BE[65300100]       <1>     mov     esi, Vol_Name
00007FC7 E8EEEDFFFF         <1>     call    print_msg
00007FCC BE[17390100]       <1>     mov     esi, nextline
00007FD1 E8E4EDFFFF         <1>     call    print_msg
                            <1>     ;
00007FD6 BE[C6300100]       <1>     mov     esi, Vol_Total_Sector_Header
00007FDB E8DAEDFFFF         <1>     call    print_msg
00007FE0 8B35[B47E0100]     <1>     mov     esi, [Vol_Tot_Sec_Str_Start]
00007FE6 E8CFEDFFFF         <1>     call    print_msg
00007FEB 8B35[AC7E0100]     <1>     mov     esi, [VolSize_Unit1]
00007FF1 E8C4EDFFFF         <1>     call    print_msg
                            <1>     ;
00007FF6 BE[D7300100]       <1>     mov     esi, Vol_Free_Sectors_Header
00007FFB E8BAEDFFFF         <1>     call    print_msg
00008000 8B35[C47E0100]     <1>     mov     esi, [Vol_Free_Sectors_Str_Start]
00008006 E8AFEDFFFF         <1>     call    print_msg
0000800B 8B35[B07E0100]     <1>     mov     esi, [VolSize_Unit2]
00008011 E8A4EDFFFF         <1>     call    print_msg
                            <1>     ;
00008016 5E                 <1>     pop     esi
                            <1>
                            <1>     ;mov    ah, [esi+LD_FSType]
                            <1>     ;mov    al, [esi+LD_FATType]
00008017 668B4603           <1>     mov     ax, [esi+LD_FATType]
                            <1>
0000801B C3                 <1>     retn
                            <1>
                            <1> move_volume_name_and_serial_no:
                            <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                            <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                            <1>     ; 08/02/2016  (TRDOS 386 = TRDOS v2.0)
                            <1>     ; this routine will be called by
                            <1>     ; "print_volume_info" and "print_directory"
                            <1>     ; INPUT ->
                            <1>     ;       ESI = Logical DOS drv descripton table address
                            <1>     ; OUTPUT ->
                            <1>     ;       *Volume name will be moved to text area
                            <1>     ;       *Volume serial number will be converted to
                            <1>     ;        text and will be moved to text area
                            <1>     ;   cf = 1 -> invalid/unknown dos drive
                            <1>     ;   cf = 0 -> ecx = 0
                            <1>     ;
                            <1>     ; (eax, edx, ecx, esi, edi will be changed)
                            <1>
                            <1>     ; 26/07/2022
0000801C 31C9               <1>     xor     ecx, ecx
                            <1>
0000801E BF[65300100]       <1>     mov     edi, Vol_Name
                            <1>
                            <1>     ;mov    ah, [esi+LD_FSType]
                            <1>     ; 19/12/2025
00008023 8A4603             <1>     mov     al, [esi+LD_FATType]
                            <1>     ;mov    ax, [esi+LD_FATType]
                            <1>
                            <1> ; 19/12/2025
                            <1> %if 0
                            <1>     cmp     ah, 0A1h
                            <1>     je      short mvn_2
                            <1>     or      ah, ah
                            <1>     jz      short mvn_0
                            <1>     or      al, al
                            <1>     jnz     short mvn_1
                            <1> mvn_0:
                            <1>     mov     al, [esi]
                            <1>     stc
                            <1>     retn
                            <1> %endif
                            <1>
                            <1> mvn_1:
00008026 3C02               <1>     cmp     al, 2
00008028 7708               <1>     ja      short mvn_3
                            <1>     ;or     al, al
                            <1>     ;jz     short mvn_2
0000802A 8B462D             <1>     mov     eax, [esi+LD_BPB+VolumeID]
0000802D 83C631             <1>     add     esi, LD_BPB+VolumeLabel
00008030 EB06               <1>     jmp     short mvn_4
                            <1>
                            <1> ; 19/12/2025
                            <1> %if 0
                            <1> mvn_2:
                            <1>     mov     eax, [esi+LD_FS_VolumeSerial]
                            <1>     add     esi, LD_FS_VolumeName
                            <1>     ;mov    ecx, 16
                            <1>     ; 26/07/2022
                            <1>     mov     cl, 16
                            <1>     rep     movsd
                            <1>     jmp     short mvn_5
                            <1> %endif
                            <1>
                            <1> mvn_3:
00008032 8B4649             <1>     mov     eax, [esi+LD_BPB+FAT32_VolID]
00008035 83C64D             <1>     add     esi, LD_BPB+FAT32_VolLab
                            <1> mvn_4:
                            <1>     ;mov    ecx, 11
                            <1>     ; 26/07/2022
00008038 B10B               <1>     mov     cl, 11
0000803A F3A4               <1>     rep     movsb
0000803C C60700             <1>     mov     byte [edi], 0
                            <1> mvn_5:
                            <1>     ;mov    [Current_VolSerial], eax
0000803F E8BBC1FFFF         <1>     call    dwordtohex
00008044 8915[BA300100]     <1>     mov     [Vol_Serial1], edx
0000804A A3[BF300100]       <1>     mov     [Vol_Serial2], eax
                            <1>     ; ecx = 0
0000804F C3                 <1>     retn
                            <1>
                            <1> get_volume_serial_number:
                            <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                            <1>     ; 19/01/2016 (TRDOS 386 = TRDOS v2.0)
                            <1>     ; 08/08/2010
                            <1>     ;
                            <1>     ; INPUT -> DL = Logical DOS Drive number
```

```
                                <1>     ; OUTPUT -> EAX = Volume serial number
                                <1>     ;          BL= FAT Type
                                <1>     ;          BH = Logical DOS drv Number (DL input)
                                <1>     ; cf = 1 -> Drive not ready
                                <1>
00008050 31DB                   <1>     xor     ebx, ebx
00008052 88D7                   <1>     mov     bh, dl
00008054 3815[8D2E0100]         <1>     cmp     [Last_DOS_DiskNo], dl
0000805A 7304                   <1>     jnb     short loc_gvsn_start
                                <1> loc_gvsn_stc_retn:
0000805C 31C0                   <1>     xor     eax, eax
0000805E F9                     <1>     stc
0000805F C3                     <1>         retn
                                <1> loc_gvsn_start:
00008060 56                     <1>     push    esi
00008061 BE00010900             <1>     mov     esi, Logical_DOSDisks
00008066 01DE                   <1>     add     esi, ebx
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     mov     bl, [esi+LD_FATType]
                                <1>     and     bl, bl
                                <1>     jz      short loc_gvsn_fs
                                <1> %endif
00008068 80FB02                 <1>     cmp     bl, 2
0000806B 7705                   <1>     ja      short loc_gvsn_fat32
                                <1> loc_gvsn_fat:
0000806D 83C62D                 <1>     add     esi, LD_BPB + VolumeID
00008070 EB03                   <1>     jmp     short loc_gvsn_return
                                <1> loc_gvsn_fat32:
00008072 83C649                 <1>     add     esi, LD_BPB + FAT32_VolID
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     jmp     short loc_gvsn_return
                                <1> loc_gvsn_fs:
                                <1>     cmp     byte [esi+LD_FSType], 0A1h
                                <1>     jne     short loc_gvsn_stc_retn
                                <1>     add     esi, LD_FS_VolumeSerial
                                <1> %endif
                                <1>
                                <1> loc_gvsn_return:
00008075 8B06                   <1>     mov     eax, [esi]
00008077 5E                     <1>     pop     esi
00008078 C3                     <1>     retn
                                <1>
                                <1> ; CMD_INTR.ASM [ TRDOS Command Interpreter Procedure ]
                                <1> ; 09/11/2011
                                <1> ; 29/01/2005
                                <1>
                                <1> command_interpreter:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 16/10/2016
                                <1>     ; 12/10/2016
                                <1>     ; 13/05/2016
                                <1>     ; 07/05/2016
                                <1>     ; 04/03/2016
                                <1>     ; 04/02/2016
                                <1>     ; 03/02/2016
                                <1>     ; 30/01/2016
                                <1>     ; 29/01/2016 (TRDOS 386 = TRDOS 2.0)
                                <1>     ; 15/09/2011
                                <1>     ; 29/01/2005
                                <1>
                                <1>     ; Input: ecx = command word length (CL)
                                <1>     ;        CommandBuffer = Command string offset
                                <1>
00008079 C605[647F0100]00       <1>     mov     byte [Program_Exit], 0
                                <1>
                                <1>     ;cmp    cl, 4
                                <1>     ;ja     c_6
                                <1>     ;jb     c_2
                                <1>
                                <1>     ; 26/07/2022
00008080 80F903                 <1>     cmp     cl, 3
00008083 777C                   <1>     ja      short c_4
00008085 7405                   <1>     je      short c_3
00008087 E9D2010000             <1>     jmp     c_2
                                <1> c_3:
                                <1> cmp_cmd_dir:
0000808C BF[F62E0100]           <1>     mov     edi, Cmd_Dir
00008091 E8D1030000             <1>     call    cmp_cmd
                                <1>     ;jnc    print_directory_list
                                <1>     ; 26/07/2022
00008096 7205                   <1>     jc      short cmp_cmd_cls
00008098 E97D040000             <1>     jmp     print_directory_list
                                <1>
                                <1> cmp_cmd_cls:
0000809D B103                   <1>     mov     cl, 3
0000809F BF[322F0100]           <1>     mov     edi, Cmd_Cls
000080A4 E8BE030000             <1>     call    cmp_cmd
                                <1>         ;jnc        clear_screen
                                <1>     ; 26/07/2022
000080A9 7205                   <1>     jc      short cmp_cmd_ver
000080AB E920EDFFFF             <1>     jmp     clear_screen
                                <1>
                                <1> cmp_cmd_ver:
000080B0 B103                   <1>     mov     cl, 3
000080B2 BF[002F0100]           <1>     mov     edi, Cmd_Ver
000080B7 E8AB030000             <1>     call    cmp_cmd
000080BC 720A                   <1>     jc      short cmp_cmd_mem
                                <1>
000080BE BE[952E0100]           <1>     mov     esi, mainprog_Version
                                <1>     ;call   print_msg
000080C3 E9F2ECFFFF             <1>     jmp     print_msg
                                <1>     ;retn
                                <1>
                                <1> cmp_cmd_mem:
000080C8 B103                   <1>     mov     cl, 3
000080CA BF[682F0100]           <1>     mov     edi, Cmd_Mem
000080CF E893030000             <1>     call    cmp_cmd
                                <1>     ;jnc    memory_info
                                <1>     ; 26/07/2022
000080D4 7205                   <1>     jc      short cmp_cmd_del
000080D6 E951C0FFFF             <1>     jmp     memory_info
                                <1>
                                <1> cmp_cmd_del:
000080DB B103                   <1>     mov     cl, 3
000080DD BF[3B2F0100]           <1>     mov     edi, Cmd_Del
000080E2 E880030000             <1>     call    cmp_cmd
                                <1>         ;jnc        delete_file
                                <1>     ; 26/07/2022
000080E7 7205                   <1>     jc      short cmp_cmd_set
000080E9 E9E2100000             <1>     jmp     delete_file
                                <1>
                                <1> cmp_cmd_set:
```

```
000080EE B103                <1>     mov     cl, 3
000080F0 BF[2E2F0100]        <1>     mov     edi, Cmd_Set
000080F5 E86D030000          <1>     call    cmp_cmd
                             <1>     ;jnc    set_get_env
                             <1>     ; 26/07/2022
000080FA 720F                <1>     jc      short cmp_cmd_run
000080FC E9F1180000          <1>     jmp     set_get_env
                             <1>
                             <1>     ; 07/08/2022
                             <1> c_4:
                             <1>     ; 26/07/2022
00008101 80F904              <1>     cmp     cl, 4
00008104 741D                <1>     je      short cmp_cmd_4
00008106 E937020000          <1>     jmp     c_6
                             <1>
                             <1> cmp_cmd_run:
0000810B B103                <1>     mov     cl, 3
0000810D BF[2A2F0100]        <1>     mov     edi, Cmd_Run
00008112 E850030000          <1>     call    cmp_cmd
                             <1>     ; 07/05/2016
                             <1>         ;jc cmp_cmd_external
                             <1>     ; 26/07/2022
00008117 7305                <1>     jnc     short c3_run
00008119 E92C030000          <1>     jmp     cmp_cmd_external
                             <1> c3_run:
0000811E E9D71E0000          <1>     jmp     load_and_execute_file
                             <1>
                             <1> cmp_cmd_4:
                             <1>     ; 26/07/2022
                             <1> cmp_cmd_exit:
00008123 BF[042F0100]        <1>     mov     edi, Cmd_Exit
00008128 E83A030000          <1>     call    cmp_cmd
0000812D 7208                <1>     jc      short cmp_cmd_date
                             <1>
0000812F C605[647F0100]01    <1>         mov     byte [Program_Exit], 1
00008136 C3                  <1>         retn
                             <1>
                             <1> cmp_cmd_date:
00008137 B104                <1>     mov     cl, 4
00008139 BF[202F0100]        <1>     mov     edi, Cmd_Date
0000813E E824030000          <1>     call    cmp_cmd
00008143 720A                <1>         jc  short cmp_cmd_time
                             <1>
00008145 E85BF8FFFF          <1>     call    show_date
                             <1>     ;call   set_date
                             <1>     ;retn
                             <1>     ; 26/07/2022
0000814A E999F8FFFF          <1>     jmp     set_date
                             <1>
                             <1> cmp_cmd_time:
0000814F B104                <1>     mov     cl, 4
00008151 BF[252F0100]        <1>     mov     edi, Cmd_Time
00008156 E80C030000          <1>     call    cmp_cmd
0000815B 720A                <1>     jc      short cmp_cmd_show
                             <1>
0000815D E8E9FAFFFF          <1>     call    show_time
                             <1>     ;call   set_time
                             <1>     ;retn
                             <1>     ; 26/07/2022
00008162 E91AFBFFFF          <1>     jmp     set_time
                             <1>
                             <1> cmp_cmd_show:
00008167 B104                <1>     mov     cl, 4
00008169 BF[362F0100]        <1>     mov     edi, Cmd_Show
0000816E E8F4020000          <1>     call    cmp_cmd
                             <1>         ;jnc       show_file
                             <1>     ; 26/07/2022
00008173 7205                <1>     jc      short cmp_cmd_echo
00008175 E971090000          <1>     jmp     show_file
                             <1>
                             <1> cmp_cmd_echo:
0000817A B104                <1>     mov     cl, 4
0000817C BF[722F0100]        <1>     mov     edi, Cmd_Echo
00008181 E8E1020000          <1>     call    cmp_cmd
00008186 7224                <1>     jc      short cmp_cmd_copy
                             <1>
                             <1>     ; 22/11/2017
                             <1>     ; AL = 0
00008188 803E20              <1>     cmp     byte [esi], 20h
0000818B 7215                <1>     jb      short cmd_echo_nextline
                             <1>     ; 14/04/2016
0000818D 56                  <1>     push    esi
                             <1> cmd_echo_asciiz:
                             <1>     ;inc    esi
                             <1>     ;mov    al, [esi]
                             <1>     ; 22/11/2017
0000818E AC                  <1>     lodsb
0000818F 3C20                <1>     cmp     al, 20h
00008191 73FB                <1>     jnb     short cmd_echo_asciiz
00008193 4E                  <1>     dec     esi
00008194 C60600              <1>     mov     byte [esi], 0
00008197 5E                  <1>     pop     esi
00008198 89F7                <1>     mov     edi, esi
0000819A E81BECFFFF          <1>     call    print_msg
0000819F C60700              <1>     mov     byte [edi], 0
                             <1> cmd_echo_nextline:
000081A2 BE[85390100]        <1>     mov     esi, NextLine
                             <1>     ;call   print_msg
                             <1>     ;retn
000081A7 E90EECFFFF          <1>     jmp     print_msg
                             <1>
                             <1> cmp_cmd_copy:
000081AC B104                <1>     mov     cl, 4
000081AE BF[592F0100]        <1>     mov     edi, Cmd_Copy
000081B3 E8AF020000          <1>     call    cmp_cmd
                             <1>     ;jnc    copy_file
                             <1>     ; 26/07/2022
000081B8 7205                <1>     jc      short cmp_cmd_move
000081BA E995160000          <1>     jmp     copy_file
                             <1>
                             <1> cmp_cmd_move:
000081BF B104                <1>     mov     cl, 4
000081C1 BF[5E2F0100]        <1>     mov     edi, Cmd_Move
000081C6 E89C020000          <1>     call    cmp_cmd
                             <1>     ;jnc    move_file
                             <1>     ; 26/07/2022
000081CB 7205                <1>     jc      short cmp_cmd_path
000081CD E940150000          <1>     jmp     move_file
                             <1>
                             <1> cmp_cmd_path:
000081D2 B104                <1>     mov     cl, 4
000081D4 BF[632F0100]        <1>     mov     edi, Cmd_Path
000081D9 E889020000          <1>     call    cmp_cmd
                             <1>     ;jnc    set_get_path
```

```
                                <1>     ; 26/07/2022
000081DE 7205                   <1>     jc      short cmp_cmd_beep
000081E0 E995180000             <1>     jmp     set_get_path
                                <1>
                                <1> cmp_cmd_beep:
000081E5 B104                   <1>     mov     cl, 4
000081E7 BF[902F0100]           <1>     mov     edi, Cmd_Beep
000081EC E876020000             <1>     call    cmp_cmd
000081F1 720B                   <1>     jc      short cmp_cmd_find
                                <1>     ; 13/05/2016
000081F3 8A3D[DE760100]         <1>     mov     bh, [ptty] ; [ACTIVE_PAGE]
000081F9 E9CBA1FFFF             <1>     jmp     beeper
                                <1>
                                <1> cmp_cmd_find:
000081FE B104                   <1>     mov     cl, 4
00008200 BF[6D2F0100]           <1>     mov     edi, Cmd_Find
00008205 E85D020000             <1>     call    cmp_cmd
                                <1>         ;jc cmp_cmd_external
                                <1>     ; 26/07/2022
0000820A 7305                   <1>     jnc     short c4_find
0000820C E939020000             <1>     jmp     cmp_cmd_external
                                <1> c4_find:
                                <1>     ;call   find_and_list_files
00008211 E9FC200000             <1>     jmp     find_and_list_files
                                <1>     ;retn
                                <1>
                                <1> c_1:
00008216 AD                     <1>     lodsd
                                <1> cmp_cmd_help:
00008217 3C3F                   <1>     cmp     al, '?'
00008219 751D                   <1>         jne     short cmp_cmd_remark
                                <1>
0000821B BE[F62E0100]           <1>     mov     esi, Command_List
                                <1> cmd_help_next_w:
00008220 E895EBFFFF             <1>     call    print_msg
                                <1>
00008225 803E20                 <1>     cmp     byte [esi], 20h ; 0
00008228 7233                   <1>     jb      short cmd_help_retn
                                <1>
0000822A 56                     <1>     push    esi
0000822B BE[17390100]           <1>     mov     esi, nextline
00008230 E885EBFFFF             <1>     call    print_msg
00008235 5E                     <1>     pop     esi
00008236 EBE8                   <1>     jmp     short cmd_help_next_w
                                <1>
                                <1> cmp_cmd_remark:
00008238 3C2A                   <1>     cmp     al, '*'
                                <1>     ;jne    cmp_cmd_external
                                <1>     ; 26/07/2022
0000823A 7405                   <1>     je      short cmp_cmd_rem
0000823C E909020000             <1>     jmp     cmp_cmd_external
                                <1> cmp_cmd_rem:
00008241 46                     <1>     inc     esi
00008242 BF[D8770100]           <1>     mov     edi, Remark
00008247 8A06                   <1>     mov     al, [esi]
00008249 3C20                   <1>     cmp     al, 20h
0000824B 7707                   <1>     ja      short cmd_remark_write
0000824D 89FE                   <1>     mov     esi, edi ; Remark
0000824F E966EBFFFF             <1>     jmp     print_msg
                                <1>
                                <1> cmd_remark_write:
00008254 AA                     <1>     stosb
00008255 AC                     <1>     lodsb
00008256 3C20                   <1>     cmp     al, 20h
00008258 73FA                   <1>     jnb     short cmd_remark_write
0000825A C60700                 <1>     mov     byte [edi], 0
                                <1>
                                <1> cmd_help_retn:
                                <1> cmd_remark_retn:
                                <1> cd_retn:
0000825D C3                     <1>     retn
                                <1> c_2:
                                <1>     ; 26/07/2022
0000825E BE[26780100]           <1>     mov     esi, CommandBuffer
00008263 80F902                 <1>     cmp     cl, 2
                                <1>     ;ja     c_3
                                <1>     ;mov    esi, CommandBuffer
00008266 72AE                   <1>     jb      short c_1
                                <1>     ; 26/07/2022
                                <1>     ;jne    short c_1
                                <1>
                                <1> cmp_cmd_cd:
00008268 66AD                   <1>     lodsw
0000826A 663D4344               <1>     cmp     ax, 'CD'
0000826E 754E                   <1>     jne     short cmp_cmd_drive
00008270 46                     <1>         inc esi
                                <1> cd_0:
00008271 668B06                 <1>     mov     ax, [esi]
00008274 3C20                   <1>     cmp     al, 20h
00008276 76E5                   <1>     jna     short cd_retn
                                <1>     ; 10/02/2016
00008278 80FC3A                 <1>     cmp     ah, ':'
0000827B 7504                   <1>     jne     short cd_1
0000827D 46                     <1>     inc     esi
0000827E 46                     <1>     inc     esi
0000827F EB47                   <1>     jmp     short cd_2
                                <1>
                                <1> cd_1:       ; change current directory
                                <1>     ; 29/11/2009
                                <1>     ; AH = CDh     ; to separate 'CD' command from others
                                <1>                    ; for restoring current directory
                                <1>                    ; 0CDh sign is for saving cdir into
                                <1>                    ; DOS drv description table cdir area
                                <1>
00008281 B4CD                   <1>     mov     ah, 0CDh ; mov byte [CD_COMMAND], 0CDh
                                <1>
00008283 E863210000             <1>     call    change_current_directory
                                <1>     ;jnc    change_prompt_dir_string
                                <1>     ; 26/07/2022
00008288 7205                   <1>     jc      short cd_error_messages
0000828A E985200000             <1>     jmp     change_prompt_dir_string
                                <1>
                                <1> cd_error_messages:
0000828F 3C03                   <1>     cmp     al, 3
00008291 740C                   <1>     je      short cd_path_not_found
                                <1>     ; 16/10/2016 (15h -> 15)
00008293 3C0F                   <1>     cmp     al, 15 ; drive not ready error
00008295 7453                   <1>     je      short cd_drive_not_ready
00008297 3C11                   <1>     cmp     al, 17 ; read error
00008299 744F                   <1>     je      short cd_drive_not_ready
0000829B 3C13                   <1>     cmp     al, 19 ; ; Bad directory/path name
0000829D 7460                   <1>     je      short cd_command_failed
                                <1>
                                <1> cd_path_not_found:
```

```
0000829F 50                 <1>     push    eax ; 29/12/2017
                            <1>     ;push   ax
000082A0 BE[99310100]       <1>     mov     esi, Msg_Dir_Not_Found
000082A5 E810EBFFFF         <1>     call    print_msg
                            <1>     ;pop    ax
000082AA 58                 <1>     pop     eax ; 29/12/2017
000082AB 3A25[74770100]     <1>     cmp     ah, [Current_Dir_Level]
                            <1>         ;jnb       change_prompt_dir_string
                            <1>     ; 26/07/2022
000082B1 7306               <1>     jnb     short cd_cpds
000082B3 8825[74770100]     <1>     mov     [Current_Dir_Level], ah
                            <1> cd_cpds:
000082B9 E956200000         <1>         jmp     change_prompt_dir_string
                            <1>
                            <1> cmp_cmd_drive: ; change current drive
                            <1>     ; C:, D:, E: etc.
000082BE 80FC3A             <1>     cmp     ah, ':'
                            <1>     ;jne    cmp_cmd_external
                            <1>     ; 26/07/2022
000082C1 7405               <1>     je      short cd_2
                            <1> cmd_ext:
000082C3 E982010000         <1>     jmp     cmp_cmd_external
                            <1>
                            <1> cd_2:       ; 'CD C:', 'CD D:' ...
000082C8 803E20             <1>     cmp     byte [esi], 20h
                            <1>     ;ja     loc_cmd_failed
                            <1>     ; 26/07/2022
000082CB 7706               <1>     ja      short cd_failed
000082CD 24DF               <1>     and     al, 0DFh
000082CF 2C41               <1>     sub     al, 'A'
                            <1>     ;jc     loc_cmd_failed
                            <1>     ; 26/07/2022
000082D1 7305               <1>     jnc     short cd_3
                            <1> cd_failed:
000082D3 E97C010000         <1>     jmp     loc_cmd_failed
                            <1> cd_3:
000082D8 3A05[8D2E0100]     <1>         cmp     al, [Last_DOS_DiskNo]
000082DE 770A               <1>         ja  short cd_drive_not_ready
                            <1>
000082E0 88C2               <1>     mov     dl, al
000082E2 E809F4FFFF         <1>     call    change_current_drive
000082E7 7201               <1>     jc      short cd_drive_not_ready
000082E9 C3                 <1>     retn
                            <1>
                            <1> cd_drive_not_ready:
000082EA BE[56310100]       <1>     mov     esi, Msg_Not_Ready_Read_Err
000082EF E8C6EAFFFF         <1>     call    print_msg
                            <1>
                            <1> cd_fail_drive_restart:
000082F4 8A15[76770100]     <1>     mov     dl, [Current_Drv]
                            <1>     ;call   change_current_drive
000082FA E9F1F3FFFF         <1>         jmp     change_current_drive
                            <1>     ;retn
                            <1>
                            <1> cd_command_failed:
000082FF BE[37310100]       <1>     mov     esi, Msg_Bad_Command
00008304 E8B1EAFFFF         <1>     call    print_msg
00008309 EBE9               <1>     jmp     short cd_fail_drive_restart
                            <1>
                            <1> c_5:
                            <1> cmp_cmd_mkdir:
0000830B BF[532F0100]       <1>     mov     edi, Cmd_Mkdir
00008310 E852010000         <1>     call    cmp_cmd
                            <1>     ;jnc    make_directory
                            <1>     ; 26/07/2022
00008315 7205               <1>     jc      short cmp_cmd_rmdir
00008317 E9550A0000         <1>     jmp     make_directory
                            <1>
                            <1> cmp_cmd_rmdir:
0000831C B105               <1>     mov     cl, 5
0000831E BF[4D2F0100]       <1>     mov     edi, Cmd_Rmdir
00008323 E83F010000         <1>     call    cmp_cmd
                            <1>     ;jnc    delete_directory
                            <1>     ; 26/07/2022
00008328 7205               <1>     jc      short cmp_cmd_chdir
0000832A E95E0B0000         <1>     jmp     delete_directory
                            <1>
                            <1> cmp_cmd_chdir:
0000832F B105               <1>     mov     cl, 5
00008331 BF[8A2F0100]       <1>     mov     edi, Cmd_Chdir
00008336 E82C010000         <1>     call    cmp_cmd
                            <1>     ;jc     cmp_cmd_external
                            <1>     ; 26/07/2022
0000833B 7286               <1>     jc      short cmd_ext
                            <1>
0000833D E92FFFFFFF         <1>     jmp     cd_0
                            <1>
                            <1> c_6:
00008342 80F906             <1>     cmp     cl, 6
                            <1>     ;ja     c_8
                            <1>     ; 26/07/2022
00008345 72C4               <1>     jb      short c_5
00008347 7405               <1>     je      short cmd_6
00008349 E9E4000000         <1>     jmp     c_8
                            <1>
                            <1> cmd_6:
                            <1> cmp_cmd_prompt:
0000834E BF[092F0100]       <1>     mov     edi, Cmd_Prompt
00008353 E80F010000         <1>     call    cmp_cmd
00008358 722F               <1>         jc  short cmp_cmd_volume
                            <1> get_prompt_name_fchar:
0000835A AC                 <1>     lodsb
0000835B 3C20               <1>     cmp     al, 20h
0000835D 74FB               <1>     je      short get_prompt_name_fchar
0000835F 7713               <1>     ja      short loc_change_prompt_label
                            <1> default_command_prompt: ; 31/12/2017 ('sysprompt')
00008361 BE[EA2E0100]       <1>     mov     esi, TRDOSPromptLabel
00008366 C7065452444F       <1>     mov     dword [esi], "TRDO"
0000836C 66C746045300       <1>             mov     word [esi+4], "S"
                            <1> loc_cmd_prompt_return:
00008372 C3                 <1>     retn
                            <1>
                            <1> set_command_prompt: ; 31/12/2017 ('sysprompt')
00008373 AC                 <1>     lodsb
                            <1> loc_change_prompt_label:
                            <1>     ;mov    cx, 11
                            <1>     ; 26/07/2022
00008374 29C9               <1>     sub     ecx, ecx
00008376 B10B               <1>     mov     cl, 11
00008378 BF[EA2E0100]       <1>     mov     edi, TRDOSPromptLabel
                            <1> put_char_new_prompt_label:
0000837D AA                 <1>     stosb
0000837E AC                 <1>     lodsb
0000837F 3C20               <1>     cmp     al, 20h
```

```
00008381 7202                <1>     jb      short pass_put_new_prompt_label
00008383 E2F8                <1>     loop    put_char_new_prompt_label
                             <1> pass_put_new_prompt_label:
00008385 C60700              <1>     mov     byte [edi], 0
00008388 C3                  <1>     retn
                             <1>
                             <1> cmp_cmd_volume:
00008389 B106                <1>     mov     cl, 6
0000838B BF[102F0100]        <1>     mov     edi, Cmd_Volume
00008390 E8D2000000          <1>     call    cmp_cmd
00008395 7259                <1>         jc  short cmp_cmd_attrib
                             <1>
                             <1> cmd_vol1:
00008397 AC                  <1>     lodsb
00008398 3C20                <1>     cmp     al, 20h
0000839A 7707                <1>     ja      short cmd_vol2
0000839C A0[76770100]        <1>     mov     al, [Current_Drv]
000083A1 EB41                <1>     jmp     short cmd_vol4
                             <1> cmd_vol2:
000083A3 3C41                <1>     cmp     al, 'A'
                             <1>     ;jb     loc_cmd_failed
                             <1>     ; 26/07/2022
000083A5 722D                <1>     jb      short cmd_vol_failed_1
000083A7 3C7A                <1>     cmp     al, 'z'
                             <1>     ;ja     loc_cmd_failed
                             <1>     ; 26/07/2022
000083A9 7731                <1>     ja      short cmd_vol_failed_2
000083AB 3C5A                <1>     cmp     al, 'Z'
000083AD 7606                <1>     jna     short cmd_vol3
000083AF 3C61                <1>     cmp     al, 'a'
                             <1>         ;jb loc_cmd_failed
                             <1>     ; 26/07/2022
000083B1 722B                <1>     jb      short cmd_vol_failed_3
000083B3 24DF                <1>     and     al, 0DFh
                             <1> cmd_vol3:
000083B5 8A26                <1>     mov     ah, [esi]
000083B7 80FC3A              <1>     cmp     ah, ':'
000083BA 0F8594000000        <1>         jne     loc_cmd_failed
000083C0 2C41                <1>     sub     al, 'A'
000083C2 3A05[8D2E0100]      <1>         cmp     al, [Last_DOS_DiskNo]
000083C8 761A                <1>     jna     short cmd_vol4
                             <1>
000083CA BE[56310100]        <1>     mov     esi, Msg_Not_Ready_Read_Err
000083CF E9E6E9FFFF          <1>     jmp     print_msg
                             <1>
                             <1>     ; 26/07/2022
                             <1>     ; (numeric characters and underscore are allowed)
                             <1> cmd_vol_failed_1:
                             <1>     ; check for numeric characters
000083D4 3C30                <1>     cmp     al, '0'
000083D6 7204                <1>     jb      short cmd_vol_failed_2
000083D8 3C39                <1>     cmp     al, '9'
000083DA 76D9                <1>     jna     short cmd_vol3
                             <1> cmd_vol_failed_2:
000083DC EB76                <1>     jmp     loc_cmd_failed
                             <1> cmd_vol_failed_3:
000083DE 3C5F                <1>     cmp     al, '_' ; underline ?
000083E0 74D3                <1>     je      short cmd_vol3 ; is ok..
000083E2 EBF8                <1>     jmp     short cmd_vol_failed_2
                             <1>
                             <1> cmd_vol4:
000083E4 E8DEFAFFFF          <1>     call    print_volume_info
                             <1>     ;jc     cd_drive_not_ready
                             <1>     ; 26/07/2022
000083E9 7339                <1>     jnc     short cmd_vol5
000083EB E9FAFEFFFF          <1>     jmp     cd_drive_not_ready
                             <1> ;cmd_vol5:
                             <1> ;   retn
                             <1>
                             <1> cmp_cmd_attrib:
000083F0 B106                <1>     mov     cl, 6
000083F2 BF[3F2F0100]        <1>     mov     edi, Cmd_Attrib
000083F7 E86B000000          <1>     call    cmp_cmd
                             <1>     ;jnc    set_file_attributes
                             <1>     ; 26/07/2022
000083FC 7205                <1>     jc      short cmp_cmd_rename
000083FE E9D80E0000          <1>     jmp     set_file_attributes
                             <1>
                             <1> cmp_cmd_rename:
00008403 B106                <1>     mov     cl, 6
00008405 BF[462F0100]        <1>     mov     edi, Cmd_Rename
0000840A E858000000          <1>     call    cmp_cmd
                             <1>     ;jnc    rename_file
                             <1>     ; 26/07/2022
0000840F 7205                <1>     jc      short cmp_cmd_device
00008411 E9BF100000          <1>     jmp     rename_file
                             <1>
                             <1> cmp_cmd_device:
00008416 B106                <1>     mov     cl, 6
00008418 BF[7B2F0100]        <1>     mov     edi, Cmd_Device
0000841D E845000000          <1>     call    cmp_cmd
00008422 7226                <1>     jc      short cmp_cmd_external
                             <1>     ; 26/07/2022
                             <1> cmd_vol5:
                             <1> cmd_dev:
00008424 C3                  <1>     retn
                             <1>
                             <1> c_7:
                             <1> cmp_cmd_devlist:
00008425 BF[822F0100]        <1>     mov     edi, Cmd_DevList
0000842A E838000000          <1>     call    cmp_cmd
0000842F 7219                <1>         jc  short cmp_cmd_external
                             <1>
                             <1> loc_cmd_return:
00008431 C3                  <1>     retn
                             <1>
                             <1> c_8:
00008432 80F908              <1>         cmp cl, 8
00008435 7713                <1>     ja      short cmp_cmd_external
00008437 72EC                <1>     jb      short c_7
                             <1>
                             <1> cmp_cmd_longname:
00008439 BF[172F0100]        <1>     mov     edi, Cmd_LongName
0000843E E824000000          <1>     call    cmp_cmd
                             <1>     ;jnc    get_and_print_longname
                             <1>     ; 26/07/2022
00008443 7205                <1>     jc      short cmp_cmd_external
00008445 E95A060000          <1>     jmp     get_and_print_longname
                             <1>
                             <1> cmp_cmd_external:
                             <1>     ; 07/05/2016
                             <1>     ; 22/04/2016
0000844A BE[26780100]        <1>     mov     esi, CommandBuffer
0000844F E9A61B0000          <1>     jmp     loc_run_check_filename
```

```
                                <1>
                                <1> loc_cmd_failed:
00008454 803D[26780100]20       <1>     cmp     byte [CommandBuffer], 20h
0000845B 76D4                   <1>     jna     short loc_cmd_return
0000845D BE[37310100]           <1>     mov     esi, Msg_Bad_Command
                                <1> ;   call    print_msg
                                <1> ;loc_cmd_return:
                                <1> ;   retn
00008462 E953E9FFFF             <1>     jmp     print_msg
                                <1>
                                <1> cmp_cmd:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 29/01/2016 (TRDOS 386 = TRDOS v2.0)
00008467 BE[26780100]           <1>         mov esi, CommandBuffer
                                <1>         ; edi = internal command word (ASCIIZ)
                                <1>     ; ecx = command length (<=8)
                                <1> cmp_cmd_1:
0000846C AC                     <1>     lodsb
0000846D AE                     <1>     scasb
0000846E 750D                   <1>     jne     short cmp_cmd_3
00008470 E2FA                   <1>     loop    cmp_cmd_1
00008472 AC                     <1>     lodsb
00008473 3C20                   <1>     cmp     al, 20h
00008475 7703                   <1>     ja      short cmp_cmd_2
00008477 30C0                   <1>     xor     al, al
                                <1>     ; ZF = 1 -> internal command word matches
00008479 C3                     <1>     retn
                                <1> cmp_cmd_2:
                                <1>     ; ZF = 0 (CF = 0) -> external command word
0000847A 58                     <1>     pop     eax ; no return to the caller from here
0000847B EBCD                   <1>     jmp     short cmp_cmd_external ; 26/07/2022
                                <1> cmp_cmd_3:
0000847D F9                     <1>     stc
                                <1>     ; CF = 1 -> internal command word does not match
0000847E C3                     <1>     retn
                                <1>
                                <1> loc_run_cmd_failed:
                                <1>     ; 26/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/03/2016
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 07/12/2009 (CMD_INTR.ASM)
                                <1>     ; 29/11/2009
                                <1>
0000847F E863000000             <1>     call    restore_cdir_after_cmd_fail
                                <1>
                                <1> loc_run_cmd_failed_cmp_al:
                                <1>     ; End of Restore_CDIR code (29/11/2009)
                                <1>
00008484 3C01                   <1>     cmp     al, 1 ; Bad command or file name
00008486 74CC                   <1>     je      short loc_cmd_failed ; 26/07/2022
                                <1> loc_run_dir_not_found:
00008488 3C03                   <1>     cmp     al, 3
0000848A 750A                   <1>     jne     short loc_run_file_notfound_msg
                                <1>     ; Path not found (MS-DOS Error Code = 3)
0000848C BE[99310100]           <1>     mov     esi, Msg_Dir_Not_Found
00008491 E924E9FFFF             <1>     jmp     print_msg
                                <1>
                                <1> loc_run_file_notfound_msg:
00008496 3C02                   <1>     cmp     al, 2 ; File not found
00008498 750A                   <1>     jne     short loc_run_file_drv_read_err
                                <1>
                                <1> loc_print_file_notfound_msg:
0000849A BE[B0310100]           <1>         mov     esi, Msg_File_Not_Found
                                <1>     ;call   proc_printmsg
                                <1>     ;retn
0000849F E916E9FFFF             <1>     jmp     print_msg
                                <1>
                                <1> loc_run_file_drv_read_err:
                                <1>     ; Err: 17 (Read fault)
000084A4 3C11                   <1>     cmp     al, 17 ; Drive not ready or read error
000084A6 7404                   <1>     je      short loc_run_file_print_drv_read_err
                                <1>     ;
000084A8 3C0F                   <1>     cmp     al, 15 ; Drive not ready (or read error)
000084AA 750A                   <1>     jne     short loc_run_file_toobig
                                <1>
                                <1> loc_run_file_print_drv_read_err:
000084AC BE[56310100]           <1>     mov     esi, Msg_Not_Ready_Read_Err
000084B1 E904E9FFFF             <1>     jmp     print_msg
                                <1>
                                <1> loc_run_file_toobig:
000084B6 3C08                   <1>     cmp     al, 8 ; Not enough free memory to load&run file
000084B8 750A                   <1>     jne     short loc_run_file_perm_denied
000084BA BE[FB310100]           <1>     mov     esi, Msg_Insufficient_Memory
000084BF E9F6E8FFFF             <1>     jmp     print_msg
                                <1>
                                <1> loc_run_file_perm_denied:
                                <1>     ; 29/12/2017
000084C4 3C0B                   <1>     cmp     al, ERR_PERM_DENIED ; 11 ; Permission denied
000084C6 750A                   <1>     jne     short loc_run_misc_error
000084C8 BE[8F330100]           <1>     mov     esi, Msg_Permission_Denied
000084CD E9E8E8FFFF             <1>     jmp     print_msg
                                <1>
                                <1>     ; 15/03/2016
                                <1> print_misc_error_msg:
                                <1> loc_run_misc_error:
                                <1>     ; AL = Error code
000084D2 E8E8BCFFFF             <1>     call    bytetohex
000084D7 66A3[2F320100]         <1>         mov     [error_code_hex], ax
                                <1>
000084DD BE[12320100]           <1>     mov     esi, Msg_Error_Code
                                <1>     ;call   print_msg
                                <1>     ;retn
000084E2 E9D3E8FFFF             <1>     jmp     print_msg
                                <1>
                                <1> restore_cdir_after_cmd_fail:
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
000084E7 50                     <1>     push    eax
000084E8 8A3D[D37E0100]         <1>     mov     bh, [RUN_CDRV] ; it is set at the beginning
                                <1>                            ; of the 'run' command.
000084EE 3A3D[76770100]         <1>     cmp     bh, [Current_Drv]
000084F4 7409                   <1>     je      short loc_run_restore_cdir
000084F6 88FA                   <1>     mov     dl, bh
000084F8 E8F3F1FFFF             <1>     call    change_current_drive
000084FD EB19                   <1>     jmp     short loc_run_err_pass_restore_cdir
                                <1>
                                <1> loc_run_restore_cdir:
000084FF 803D[8E2E0100]00       <1>     cmp     byte [Restore_CDIR], 0
00008506 7610                   <1>     jna     short loc_run_err_pass_restore_cdir
00008508 30DB                   <1>     xor     bl, bl
0000850A 0FB7F3                 <1>     movzx   esi, bx
0000850D 81C600010900           <1>     add     esi, Logical_DOSDisks
00008513 E889F2FFFF             <1>     call    restore_current_directory
                                <1>
                                <1> loc_run_err_pass_restore_cdir:
```

```
00008518 58                    <1>     pop    eax
00008519 C3                    <1>     retn
                               <1>
                               <1> print_directory_list:
                               <1>     ; 02/12/2023 (TRDOS 386 v2.0.7)
                               <1>     ; 07/08/2022
                               <1>     ; 27/07/2022 (TRDOS 386 Kernel v2.0.5)
                               <1>     ; 10/02/2016
                               <1>     ; 08/02/2016 (TRDOS 386 = TRDOS v2.0)
                               <1>     ; 06/12/2009 ('cmp_cmd_dir')
                               <1>     ;
0000851A 66C705[14800100]00-   <1>     mov    word [AttributesMask], 0800h ; ..except volume names..
00008522 08                    <1>
00008523 A0[76770100]          <1>     mov    al, [Current_Drv]
00008528 A2[D37E0100]          <1>     mov    [RUN_CDRV], al
                               <1>     ; 02/12/2023
0000852D 3C02                  <1>     cmp    al, 2
0000852F 733A                  <1>     jnb    short get_dfname_fchar
00008531 A0[31770100]          <1>     mov    al, [TIMER_LOW+1]
00008536 D0E8                  <1>     shr    al, 1  ; 512/18.2 (>= 28 seconds)
00008538 8605[9A7E0100]        <1>     xchg   al, [P_TIMER]  ; 28 seconds
0000853E 3A05[9A7E0100]        <1>     cmp    al, [P_TIMER]
00008544 7425                  <1>     je     short get_dfname_fchar
00008546 E893F1FFFF            <1>     call   get_media_change_status
0000854B 7205                  <1>     jc     short pdl_chdrv
0000854D 80FC06                <1>     cmp    ah, 6
00008550 7519                  <1>     jne    short get_dfname_fchar
                               <1> pdl_chdrv:
00008552 56                    <1>     push   esi
00008553 8A15[D37E0100]        <1>     mov    dl, [RUN_CDRV]
00008559 E892F1FFFF            <1>     call   change_current_drive
0000855E 5E                    <1>     pop    esi
0000855F 730A                  <1>     jnc    short get_dfname_fchar
00008561 BE[56310100]          <1>     mov    esi, Msg_Not_Ready_Read_Err
                               <1>     ;call  print_msg
                               <1>     ;retn
00008566 E94FE8FFFF            <1>     jmp    print_msg
                               <1>
                               <1> get_dfname_fchar:
0000856B AC                    <1>     lodsb
0000856C 3C20                  <1>     cmp    al, 20h
0000856E 74FB                  <1>     je     short get_dfname_fchar
                               <1>     ;jb    loc_print_dir_call_all
                               <1>     ; 02/12/2023
00008570 7705                  <1>     ja     short get_dfname_fchar_2
00008572 E9E0000000            <1>     jmp    loc_print_dir_call_all
                               <1> get_dfname_fchar_2:
00008577 3C2D                  <1>     cmp    al, '-'
00008579 753C                  <1>     jne    short loc_print_dir_call_flt
                               <1> get_next_attr_char:
0000857B AC                    <1>     lodsb
0000857C 3C20                  <1>     cmp    al, 20h
0000857E 74FB                  <1>     je     short get_next_attr_char
                               <1>     ;jb    loc_cmd_failed
                               <1>     ; 27/07/2022
00008580 7305                  <1>     jnb    short pdl_1
                               <1> pdl_0:
00008582 E9CDFEFFFF            <1>     jmp    loc_cmd_failed
                               <1> pdl_1:
00008587 24DF                  <1>     and    al, 0DFh
00008589 3C44                  <1>     cmp    al, 'D' ; directories only ?
0000858B 750E                  <1>     jne    short pass_only_directories
0000858D AC                    <1>     lodsb
0000858E 3C20                  <1>     cmp    al, 20h
                               <1>     ;ja    loc_cmd_failed
                               <1>     ; 27/07/2022
00008590 77F0                  <1>     ja     short pdl_0
00008592 800D[14800100]10      <1>     or     byte [AttributesMask], 10h ; ..directory..
00008599 EB10                  <1>     jmp    short get_dfname_fchar_attr
                               <1> pass_only_directories:
0000859B 3C46                  <1>     cmp    al, 'F' ; files only ?
0000859D 7530                  <1>     jne    short check_attr_s ; 27/07/2022
0000859F AC                    <1>     lodsb
000085A0 3C20                  <1>     cmp    al, 20h
                               <1>     ;ja    loc_cmd_failed
                               <1>     ; 27/07/2022
000085A2 77DE                  <1>     ja     short pdl_0
000085A4 800D[15800100]10      <1>     or     byte [AttributesMask+1], 10h ; ..except directories..
                               <1> get_dfname_fchar_attr:
000085AB AC                    <1>     lodsb
000085AC 3C20                  <1>     cmp    al, 20h
000085AE 74FB                  <1>     je     short get_dfname_fchar_attr
                               <1>     ;jb    short loc_print_dir_call_all
                               <1>     ; 07/08/2022
000085B0 7305                  <1>     jnb    short loc_print_dir_call_flt
000085B2 E9A0000000            <1>     jmp    loc_print_dir_call_all
                               <1> loc_print_dir_call_flt:
000085B7 4E                    <1>     dec    esi
000085B8 BF[16800100]          <1>     mov    edi, FindFile_Drv
000085BD E8BC230000            <1>     call   parse_path_name
000085C2 7352                  <1>     jnc    short loc_print_dir_change_drv_1
000085C4 3C01                  <1>     cmp    al, 1
                               <1>     ;ja    loc_run_cmd_failed
                               <1>     ; 27/07/2022
000085C6 764E                  <1>     jna    short loc_print_dir_change_drv_1
                               <1> pdl_2:
000085C8 E9B2FEFFFF            <1>     jmp    loc_run_cmd_failed
                               <1>
                               <1>     ; 27/07/2022
                               <1> check_attr_s_cap:
000085CD 24DF                  <1>     and    al, 0DFh
                               <1> check_attr_s:
000085CF 3C53                  <1>     cmp    al, 'S'
000085D1 7510                  <1>     jne    short pass_attr_s
000085D3 800D[14800100]04      <1>     or     byte [AttributesMask], 4 ; system
000085DA AC                    <1>     lodsb
000085DB 3C20                  <1>     cmp    al, 20h
000085DD 74CC                  <1>     je     short get_dfname_fchar_attr
000085DF 7276                  <1>     jb     short loc_print_dir_call_all
000085E1 24DF                  <1>     and    al, 0DFh
                               <1> pass_attr_s:
000085E3 3C48                  <1>     cmp    al, 'H'
000085E5 7510                  <1>     jne    short pass_attr_h
000085E7 800D[14800100]02      <1>     or     byte [AttributesMask], 2 ; hidden
                               <1> pass_attr_shr:
000085EE AC                    <1>     lodsb
000085EF 3C20                  <1>     cmp    al, 20h
000085F1 74B8                  <1>     je     short get_dfname_fchar_attr
000085F3 7262                  <1>     jb     short loc_print_dir_call_all
000085F5 EBD6                  <1>     jmp    short check_attr_s_cap
                               <1> pass_attr_h:
000085F7 3C52                  <1>     cmp    al, 'R'
000085F9 7509                  <1>     jne    short pass_attr_r
000085FB 800D[14800100]01      <1>     or     byte [AttributesMask], 1 ; read only
```

```
00008602 EBEA                <1>     jmp    short pass_attr_shr
                             <1> pass_attr_r:
00008604 3C41                <1>     cmp    al, 'A'
                             <1>     ;jne   loc_cmd_failed
                             <1>     ; 27/07/2022
00008606 7405                <1>     je     short pass_attr_a
00008608 E947FEFFFF          <1>     jmp    loc_cmd_failed
                             <1> pass_attr_a:
0000860D 800D[14800100]20    <1>     or     byte [AttributesMask], 20h ; archive
00008614 EBD8                <1>     jmp    short pass_attr_shr
                             <1>
                             <1>     ; 07/08/2022
                             <1> loc_print_dir_change_drv_1:
00008616 8A15[16800100]      <1>     mov    dl, [FindFile_Drv]
                             <1> loc_print_dir_change_drv_2:
0000861C 3A15[D37E0100]      <1>     cmp    dl, [RUN_CDRV]
00008622 7407                <1>     je     short loc_print_dir_change_directory
00008624 E8C7F0FFFF          <1>     call   change_current_drive
                             <1>         ;jc loc_run_cmd_failed
                             <1>     ; 27/07/2022
00008629 729D                <1>     jc     short pdl_2
                             <1> loc_print_dir_change_directory:
0000862B 803D[17800100]20    <1>     cmp    byte [FindFile_Directory], 20h ; 0 or 20h ?
00008632 7619                <1>     jna    short pass_print_dir_change_directory
                             <1>
00008634 FE05[8E2E0100]      <1>     inc    byte [Restore_CDIR]
0000863A BE[17800100]        <1>     mov    esi, FindFile_Directory
0000863F 30E4                <1>     xor    ah, ah ; CD_COMMAND sign -> 0
00008641 E8A51D0000          <1>     call   change_current_directory
                             <1>         ;jc loc_run_cmd_failed
                             <1>     ; 27/07/2022
00008646 7280                <1>     jc     short pdl_2
                             <1>
                             <1> loc_print_dir_change_prompt_dir_string:
00008648 E8C71C0000          <1>     call   change_prompt_dir_string
                             <1>
                             <1> pass_print_dir_change_directory:
0000864D BE[58800100]        <1>     mov    esi, FindFile_Name
00008652 803E20              <1>     cmp    byte [esi], 20h ; ; 0 or 20h ?
00008655 7706                <1>     ja     short loc_print_dir_call
                             <1>
                             <1> loc_print_dir_call_all:
00008657 C7062A2E2A00        <1>     mov    dword [esi], '*.*'
                             <1> loc_print_dir_call:
0000865D E82D000000          <1>     call   print_directory
                             <1>
00008662 8A15[D37E0100]      <1>     mov    dl, [RUN_CDRV]  ; it is set at the beginning
00008668 3A15[76770100]      <1>     cmp    dl, [Current_Drv]
0000866E 7405                <1>     je     short loc_print_dir_call_restore_cdir_retn
                             <1>     ;call  change_current_drive
                             <1>     ;retn
                             <1>     ; 27/07/2022
00008670 E97BF0FFFF          <1>     jmp    change_current_drive
                             <1>
                             <1> loc_print_dir_call_restore_cdir_retn:
00008675 803D[8E2E0100]00    <1>     cmp    byte [Restore_CDIR], 0
0000867C 7610                <1>     jna    short pass_print_dir_call_restore_cdir_retn
                             <1>
0000867E BE00010900          <1>     mov    esi, Logical_DOSDisks
00008683 31C0                <1>     xor    eax, eax
00008685 88D4                <1>     mov    ah, dl
00008687 01C6                <1>     add    esi, eax
                             <1>
                             <1>     ;call  restore_current_directory
                             <1>     ; 27/07/2022
00008689 E913F1FFFF          <1>     jmp    restore_current_directory
                             <1>
                             <1> pass_print_dir_call_restore_cdir_retn:
0000868E C3                  <1>     retn
                             <1>
                             <1> print_directory:
                             <1>     ; 27/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 13/05/2016
                             <1>     ; 11/02/2016
                             <1>     ; 10/02/2016
                             <1>     ; 08/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 30/10/2010 ('proc_print_directory')
                             <1>     ; 19/09/2009
                             <1>     ; 2005
                             <1>     ; INPUT ->
                             <1>     ;      ESI = Asciiz File/Dir Name Address
                             <1>
0000868F 56                  <1>     push   esi
                             <1>
00008690 29C0                <1>     sub    eax, eax
                             <1>
00008692 66A3[A0800100]      <1>     mov    [Dir_Count], ax ; 0
00008698 66A3[9E800100]      <1>     mov    [File_Count], ax ; 0
0000869E A3[A2800100]        <1>     mov    [Total_FSize], eax ; 0
                             <1>
000086A3 E828E7FFFF          <1>     call    clear_screen
                             <1>
000086A8 31C9                <1>     xor    ecx, ecx
000086AA 8A2D[76770100]      <1>     mov     ch, [Current_Drv] ; DirBuff_Drv - 'A'
000086B0 A0[77770100]        <1>     mov     al, [Current_Dir_Drv]
000086B5 A2[54300100]        <1>     mov     [Dir_Drive_Name], al
000086BA BE00010900          <1>     mov    esi, Logical_DOSDisks
000086BF 01CE                <1>     add    esi, ecx
                             <1>
000086C1 E856F9FFFF          <1>     call   move_volume_name_and_serial_no
000086C6 730C                <1>     jnc    short print_dir_strlen_check
                             <1>
000086C8 5E                  <1>     pop    esi
000086C9 8A3D[DE760100]      <1>     mov    bh, [ptty] ; [ACTIVE_PAGE]
                             <1>     ;call  beeper
                             <1>     ;retn
000086CF E9F59CFFFF          <1>     jmp    beeper  ; beep ! and return
                             <1>
                             <1> print_dir_strlen_check:
000086D4 BE[79770100]        <1>     mov    esi, Current_Dir_Root
000086D9 BF[F1300100]        <1>     mov    edi, Dir_Str_Root
                             <1>
                             <1>     ;xor   ecx, ecx
000086DE 8A0D[D5770100]      <1>         mov     cl, [Current_Dir_StrLen]
000086E4 FEC1                <1>     inc    cl
000086E6 80F940              <1>     cmp    cl, 64
000086E9 760D                <1>     jna    short pass_print_dir_strlen_shorting
000086EB 46                  <1>     inc    esi
000086EC 01CE                <1>     add    esi, ecx
000086EE 83EE40              <1>     sub    esi, 64
000086F1 47                  <1>     inc    edi
000086F2 B82E2E2E20          <1>     mov    eax, '... '
000086F7 AB                  <1>     stosd
                             <1>
```

```
                                <1> pass_print_dir_strlen_shorting:
000086F8 F3A4                   <1>     rep     movsb
                                <1>
000086FA BE[47300100]           <1>     mov     esi, Dir_Drive_Str
000086FF E8B6E6FFFF             <1>     call    print_msg
                                <1>
00008704 BE[A6300100]           <1>     mov     esi, Vol_Serial_Header
00008709 E8ACE6FFFF             <1>     call    print_msg
                                <1>
0000870E BE[E6300100]           <1>     mov     esi, Dir_Str_Header
00008713 E8A2E6FFFF             <1>     call    print_msg
                                <1>
00008718 BE[15390100]           <1>     mov     esi, next2line
0000871D E898E6FFFF             <1>     call    print_msg
                                <1>
                                <1> loc_print_dir_first_file:
00008722 C605[B5800100]10       <1>     mov     byte [PrintDir_RowCounter], 16
00008729 66A1[14800100]         <1>     mov     ax, [AttributesMask]
0000872F 5E                     <1>     pop     esi
                                <1>
00008730 E845020000             <1>     call    find_first_file
                                <1>     ;jc     loc_dir_ok
                                <1>     ; 27/07/2022
00008735 7305                   <1>     jnc     short loc_dfname_use_this
00008737 E97B010000             <1>     jmp     loc_dir_ok
                                <1>
                                <1> loc_dfname_use_this:
                                <1>     ; bl =  File Attributes (bh = Long Name Entry Length)
0000873C F6C310                 <1>     test    bl, 10h  ; Is it a directory?
0000873F 741B                   <1>     jz      short loc_not_dir
                                <1>
00008741 66FF05[A0800100]       <1>     inc     word [Dir_Count]
00008748 89F2                   <1>     mov     edx, esi        ; FindFile_DirEntry address
0000874A BE[35320100]           <1>     mov     esi, Type_Dir  ; '<DIR>     '
0000874F BF[4C320100]           <1>     mov     edi, Dir_Or_FileSize
                                <1>     ; move 10 bytes
00008754 A5                     <1>     movsd
00008755 A5                     <1>     movsd
00008756 66A5                   <1>     movsw
00008758 89D6                   <1>     mov     esi, edx
0000875A EB36                   <1>     jmp      short loc_dir_attribute
                                <1>
                                <1> loc_not_dir:
0000875C 66FF05[9E800100]       <1>     inc     word [File_Count]
00008763 0105[A2800100]         <1>     add     [Total_FSize], eax
                                <1>
00008769 B90A000000             <1>     mov     ecx, 10  ; 32 bit divisor
0000876E 89CF                   <1>     mov     edi, ecx
00008770 81C7[4C320100]         <1>     add     edi, Dir_Or_FileSize
                                <1> loc_dir_rdivide:
00008776 29D2                   <1>     sub     edx, edx
00008778 F7F1                   <1>     div     ecx      ; remainder in dl (< 10)
0000877A 80C230                 <1>     add      dl, '0'         ; to make visible (ascii)
0000877D 4F                     <1>     dec     edi
0000877E 8817                   <1>     mov      [edi], dl
00008780 21C0                   <1>     and     eax, eax
00008782 75F2                   <1>     jnz     short loc_dir_rdivide
                                <1>
                                <1> loc_dir_fill_space:
00008784 81FF[4C320100]         <1>     cmp      edi, Dir_Or_FileSize
0000878A 7606                   <1>     jna      short loc_dir_attribute
0000878C 4F                     <1>     dec      edi
0000878D C60720                 <1>     mov      byte [edi], 20h
00008790 EBF2                   <1>     jmp      short loc_dir_fill_space
                                <1>
                                <1> loc_dir_attribute:
00008792 C705[57320100]2020-    <1>     mov     dword [File_Attribute], 20202020h
0000879A 2020                   <1>
                                <1>
0000879C 80FB20                 <1>     cmp     bl, 20h  ; Is it an archive file?
0000879F 7207                   <1>     jb      short loc_dir_pass_arch
000087A1 C605[5A320100]41       <1>     mov     byte [File_Attribute+3], 'A'
                                <1>
                                <1> loc_dir_pass_arch:
000087A8 80E307                 <1>     and     bl, 7
000087AB 7428                   <1>     jz      short loc_dir_file_name
000087AD 88DF                   <1>     mov     bh, bl
000087AF 80E303                 <1>     and     bl, 3
000087B2 38DF                   <1>     cmp     bh, bl
000087B4 7607                   <1>     jna     short loc_dir_pass_s
000087B6 C605[57320100]53       <1>     mov     byte [File_Attribute], 'S'
                                <1>
                                <1> loc_dir_pass_s:
000087BD 80E302                 <1>     and      bl,2
000087C0 7407                   <1>     jz       short loc_dir_pass_h
000087C2 C605[58320100]48       <1>     mov      byte [File_Attribute+1], 'H'
                                <1> loc_dir_pass_h:
000087C9 80E701                 <1>     and      bh,1
000087CC 7407                   <1>     jz       short loc_dir_file_name
000087CE C605[59320100]52       <1>     mov      byte [File_Attribute+2], 'R'
                                <1> loc_dir_file_name:
                                <1>     ;mov    bx, [esi+18h] ; Date
                                <1>     ;mov    dx, [esi+16h] ; Time
000087D5 8B5E16                 <1>     mov     ebx, [esi+16h]
000087D8 89F1                   <1>     mov     ecx, esi ; FindFile_DirEntry address
000087DA BF[3F320100]           <1>     mov      edi, File_Name
                                <1>     ; move 8 bytes
000087DF A5                     <1>     movsd
000087E0 A5                     <1>     movsd
000087E1 C60720                 <1>     mov     byte [edi], 20h
000087E4 47                     <1>     inc     edi
                                <1>     ; move 3 bytes
000087E5 66A5                   <1>     movsw
000087E7 A4                     <1>     movsb
000087E8 89CE                   <1>     mov     esi, ecx
                                <1>
                                <1> Dir_Time_start:
                                <1>     ;mov    ax, dx          ; Time
000087EA 6689D8                 <1>     mov     ax, bx
000087ED 66C1E805               <1>     shr     ax, 5           ; shift right 5 times
000087F1 6683E03F               <1>     and     ax, 0000111111b        ; Minute Mask
000087F5 D40A                   <1>     aam                     ; Q([AL]/10)->AH
                                <1>                             ; R([AL]/10)->AL
                                <1>                             ; [AL]+[AH]= Minute as BCD
000087F7 660D3030               <1>     or      ax, '00'        ; Convert to ASCII
000087FB 86E0                   <1>     xchg    ah, al
000087FD 66A3[6A320100]         <1>     mov     [File_Minute], ax
                                <1>
                                <1>     ;mov    al, dh
00008803 88F8                   <1>     mov     al, bh
00008805 C0E803                 <1>     shr     al, 3           ; shift right 3 times
00008808 D40A                   <1>     aam                     ; [AL]+[AH]= Hours as BCD
0000880A 660D3030               <1>     or      ax, '00'
0000880E 86E0                   <1>     xchg    ah, al
```

```
00008810 66A3[67320100]      <1>     mov     [File_Hour], ax
                             <1>
00008816 C1EB10              <1>     shr     ebx, 16        ; BX = Date
                             <1>
                             <1> Dir_Date_start:
                             <1>     ;mov    ax, bx         ; Date
                             <1>     ; 27/07/2022
00008819 89D8                <1>     mov     eax, ebx
0000881B 6683E01F            <1>     and     ax, 00011111b  ; Day Mask
0000881F D40A                <1>     aam                    ; Q([AL]/10)->AH
                             <1>                            ; R([AL]/10)->AL
                             <1>                            ; [AL]+[AH]= Day as BCD
00008821 660D3030            <1>     or      ax, '00'       ; Convert to ASCII
00008825 86C4                <1>     xchg    al, ah
                             <1>
00008827 66A3[5C320100]      <1>     mov     [File_Day], ax
                             <1>
                             <1>     ;mov    ax, bx
                             <1>     ; 27/07/2022
0000882D 89D8                <1>     mov     eax, ebx
                             <1>     ;shr    ax, 5          ; shift right 5 times
0000882F C1E805              <1>     shr     eax, 5
00008832 6683E00F            <1>     and     ax, 00001111b  ; Month Mask
00008836 D40A                <1>     aam
00008838 660D3030            <1>     or      ax, '00'
0000883C 86E0                <1>     xchg    ah, al
0000883E 66A3[5F320100]      <1>     mov     [File_Month], ax
                             <1>
                             <1>     ;mov    ax, bx
                             <1>     ; 27/07/2022
00008844 89D8                <1>     mov     eax, ebx
                             <1>     ;shr    ax, 9
00008846 C1E809              <1>     shr     eax, 9
00008849 6683E07F            <1>     and     ax, 01111111b  ; Result = Year - 1980
0000884D 6605BC07            <1>     add     ax, 1980
                             <1>
00008851 B10A                <1>     mov     cl, 10
00008853 F6F1                <1>     div     cl             ; Q -> AL, R -> AH
00008855 80CC30              <1>     or      ah, '0'
00008858 8825[65320100]      <1>     mov     [File_Year+3], ah
0000885E D40A                <1>     aam
00008860 86E0                <1>     xchg    ah, al
00008862 80CC30              <1>     or      ah, '0'   ; Convert to ASCII
00008865 8825[64320100]      <1>     mov     [File_Year+2], ah
0000886B D40A                <1>     aam
0000886D 86C4                <1>     xchg    al, ah
0000886F 660D3030            <1>     or      ax, '00'
00008873 66A3[62320100]      <1>     mov     [File_Year], ax
                             <1>
                             <1> loc_show_line:
00008879 56                  <1>     push    esi
0000887A BE[3F320100]        <1>     mov     esi, File_Name
0000887F E836E5FFFF          <1>     call    print_msg
00008884 BE[17390100]        <1>     mov     esi, nextline
00008889 E82CE5FFFF          <1>     call    print_msg
0000888E 5E                  <1>     pop     esi
                             <1>
0000888F FE0D[B5800100]      <1>     dec     byte [PrintDir_RowCounter]
00008895 740C                <1>     jz      short pause_dir_scroll ; 27/07/2022
                             <1>
                             <1> loc_next_entry:
00008897 E88B010000          <1>     call    find_next_file
                             <1>     ;jnc    loc_dfname_use_this
                             <1>     ; 27/07/2022
0000889C 7219                <1>     jc      short loc_dir_ok
0000889E E999FEFFFF          <1>     jmp     loc_dfname_use_this
                             <1>
                             <1>     ; 27/07/2022
                             <1> pause_dir_scroll:
000088A3 28E4                <1>     sub     ah, ah
000088A5 E86886FFFF          <1>     call    int16h
000088AA 3C1B                <1>     cmp     al, 1Bh
000088AC 7409                <1>     je      short loc_dir_ok
000088AE C605[B5800100]10    <1>     mov     byte [PrintDir_RowCounter], 16 ; Reset counter
000088B5 EBE0                <1>         jmp     short loc_next_entry
                             <1>
                             <1> loc_dir_ok:
000088B7 B90A000000          <1>     mov     ecx, 10
                             <1>     ;mov    ax, [Dir_Count]
                             <1>     ; 27/07/2022
000088BC 0FB705[A0800100]    <1>     movzx   eax, word [Dir_Count]
000088C3 BF[80320100]        <1>     mov     edi, Decimal_Dir_Count
                             <1>     ;cmp    ax, cx ; 10
                             <1>     ; 27/07/2022
000088C8 39C8                <1>     cmp     eax, ecx
000088CA 7216                <1>     jb      short pass_ddc
000088CC 47                  <1>     inc     edi
000088CD 6683F864            <1>     cmp     ax, 100
000088D1 720F                <1>     jb      short pass_ddc
000088D3 47                  <1>     inc     edi
000088D4 663DE803            <1>     cmp     ax, 1000
000088D8 7208                <1>     jb      short pass_ddc
000088DA 47                  <1>     inc     edi
000088DB 663D1027            <1>     cmp     ax, 10000
000088DF 7201                <1>     jb      short pass_ddc
000088E1 47                  <1>     inc     edi
                             <1> pass_ddc:
000088E2 886F01              <1>     mov     [edi+1], ch ; 0
                             <1> loc_ddc_rediv:
000088E5 31D2                <1>     xor     edx, edx
                             <1>     ;div    cx  ; 10
                             <1>     ; 27/07/2022
000088E7 F7F1                <1>     div     ecx
000088E9 80C230              <1>     add     dl, '0'
000088EC 8817                <1>     mov     [edi], dl
000088EE 4F                  <1>     dec     edi
                             <1>     ;or     ax, ax
                             <1>     ; 27/07/2022
000088EF 09C0                <1>     or      eax, eax
000088F1 75F2                <1>     jnz     short loc_ddc_rediv
                             <1>
000088F3 66A1[9E800100]      <1>     mov     ax, [File_Count]
000088F9 BF[6F320100]        <1>     mov     edi, Decimal_File_Count
                             <1>     ;cmp    ax, cx ; 10
                             <1>     ; 27/07/2022
000088FE 39C8                <1>     cmp     eax, ecx ; 10
00008900 7216                <1>     jb      short pass_dfc
00008902 47                  <1>     inc     edi
00008903 6683F864            <1>     cmp     ax, 100
00008907 720F                <1>     jb      short pass_dfc
00008909 47                  <1>     inc     edi
0000890A 663DE803            <1>     cmp     ax, 1000
0000890E 7208                <1>     jb      short pass_dfc
00008910 47                  <1>     inc     edi
```

```
00008911 663D1027            <1>     cmp     ax, 10000
00008915 7201                <1>     jb      short pass_dfc
00008917 47                  <1>     inc     edi
                             <1> pass_dfc:
                             <1>     ;mov    cx, 10
00008918 886F01              <1>     mov     [edi+1], ch ; 00
                             <1> loc_dfc_rediv:
                             <1>     ;;xor  dx, dx
                             <1>     ;xor   dl, dl
                             <1>     ;div   cx
                             <1>     ; 27/07/022
0000891B 31D2                <1>     xor     edx, edx
0000891D F7F1                <1>     div     ecx
0000891F 80C230              <1>     add     dl, '0'
00008922 8817                <1>     mov     [edi], dl
00008924 4F                  <1>     dec     edi
                             <1>     ;or     ax, ax
                             <1>     ; 27/07/2022
00008925 09C0                <1>     or      eax, eax
00008927 75F2                <1>     jnz     short loc_dfc_rediv
                             <1>
00008929 BF[B4800100]        <1>     mov     edi, TFS_Dec_End
                             <1>         ;mov    byte [edi], 0
0000892E A1[A2800100]        <1>     mov     eax, [Total_FSize]
                             <1>     ;mov    ecx, 10
                             <1> rediv_tfs_hex:
00008933 29D2                <1>     sub     edx, edx ; 27/07/2022
                             <1>     ;sub   dl, dl
00008935 F7F1                <1>     div     ecx
00008937 80C230              <1>     add     dl, '0'
0000893A 4F                  <1>     dec     edi
0000893B 8817                <1>     mov     [edi], dl
0000893D 21C0                <1>     and     eax, eax
0000893F 75F2                <1>     jnz     short rediv_tfs_hex
                             <1>
00008941 893D[A6800100]      <1>     mov     [TFS_Dec_Begin], edi
00008947 BE[6D320100]        <1>     mov     esi, Decimal_File_Count_Header
0000894C E869E4FFFF          <1>     call    print_msg
00008951 BE[75320100]        <1>     mov     esi, str_files
00008956 E85FE4FFFF          <1>     call    print_msg
0000895B BE[86320100]        <1>     mov     esi, str_dirs
00008960 E855E4FFFF          <1>     call    print_msg
00008965 8B35[A6800100]      <1>     mov     esi, [TFS_Dec_Begin]
0000896B E84AE4FFFF          <1>     call    print_msg
00008970 BE[97320100]        <1>     mov     esi, str_bytes
                             <1>     ;call  print_msg
                             <1>     ;retn
                             <1>     ; 27/07/2022
00008975 E940E4FFFF          <1>     jmp     print_msg
                             <1>
                             <1> find_first_file:
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 11/02/2016
                             <1>     ; 10/02/2016
                             <1>     ; 08/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 09/10/2011
                             <1>     ; 17/09/2009
                             <1>     ; 2005
                             <1>     ; INPUT ->
                             <1>     ;      ESI = ASCIIZ File/Dir Name Address (in Current Directory)
                             <1>     ;      AL = Attributes AND mask (The AND result must be equal to AL)
                             <1>     ;            bit 0 = Read Only
                             <1>     ;            bir 1 = Hidden
                             <1>     ;            bit 2 = System
                             <1>     ;            bit 3 = Volume Label
                             <1>     ;            bit 4 = Directory
                             <1>     ;            bit 5 = Archive
                             <1>     ;            bit 6 = Reserved, must be 0
                             <1>     ;            bit 7 = Reserved, must be 0
                             <1>     ;       AH = Attributes Negative AND mask (The AND result must be ZERO)
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;      CF = 1 -> Error, Error Code in EAX (AL)
                             <1>     ;      CF = 0 ->
                             <1>     ;           ESI = Directory Entry (FindFile_DirEntry) Location
                             <1>     ;           EDI = Directory Buffer Directory Entry Location
                             <1>     ;           EAX = File Size
                             <1>     ;            BL = Attributes of The File/Directory
                             <1>     ;            BH = Long Name Yes/No Status (>0 is YES)
                             <1>     ;             DX > 0 : Ambiguous filename chars are used
                             <1>     ;
                             <1>     ; (EAX, EBX, ECX, EDX, ESI, EDI will be changed)
                             <1>
0000897A 66A3[66800100]      <1>     mov     [FindFile_AttributesMask], ax
00008980 BF[68800100]        <1>     mov     edi, FindFile_DirEntry ; TR-DOS Fullfilename formatted buffer
00008985 31C0                <1>     xor     eax, eax
                             <1>     ;mov    ecx, 11
                             <1>     ; 28/07/2022
00008987 31C9                <1>     xor     ecx, ecx
00008989 B10B                <1>     mov     cl, 11
0000898B F3AB                <1>     rep     stosd   ; 44 bytes
                             <1>     ;stosw          ; +2 bytes
                             <1>
0000898D BF[58800100]        <1>     mov     edi, FindFile_Name ; FFF structure, offset 66
00008992 39FE                <1>     cmp     esi, edi
00008994 7408                <1>     je      short loc_fff_mfn_ok
00008996 89FA                <1>     mov     edx, edi
                             <1>     ; move 13 bytes
00008998 A5                  <1>     movsd
00008999 A5                  <1>     movsd
0000899A A5                  <1>     movsd
0000899B AA                  <1>     stosb
0000899C 89D6                <1>     mov     esi, edx
                             <1> loc_fff_mfn_ok:
0000899E BF[07800100]        <1>     mov     edi, Dir_Entry_Name ; Dir Entry Format File Name
000089A3 E80C1F0000          <1>     call    convert_file_name
000089A8 89FE                <1>     mov     esi, edi ; offset Dir_Entry_Name
                             <1>
000089AA 66A1[66800100]      <1>     mov     ax, [FindFile_AttributesMask]
                             <1>     ;xor    ecx, ecx
000089B0 30C9                <1>     xor     cl, cl
000089B2 E8511C0000          <1>     call    locate_current_dir_file
000089B7 726D                <1>     jc      short loc_fff_retn
                             <1>     ; EDI = Directory Entry
                             <1>     ; EBX = Directory Buffer Entry Index/Number
                             <1>
                             <1> loc_fff_fnf_ln_check:
000089B9 30ED                <1>     xor     ch, ch
000089BB 80F60F              <1>     xor     dh, 0Fh
000089BE 7408                <1>     jz      short loc_fff_longname_yes
000089C0 882D[65800100]      <1>     mov     [FindFile_LongNameYes], ch ; 0
000089C6 EB0C                <1>     jmp     short loc_fff_longname_no
                             <1>
                             <1> loc_fff_longname_yes:
```

```
                                <1>     ;inc    byte [FindFile_LongNameYes]
000089C8 8A0D[727F0100]         <1>     mov     cl, [LFN_EntryLength]
000089CE 880D[65800100]         <1>     mov     [FindFile_LongNameEntryLength], cl ; FindFile_LongNameYes
                                <1>
                                <1> loc_fff_longname_no:
                                <1>     ;mov    bx, [DirBuff_CurrentEntry]
000089D4 66891D[90800100]       <1>     mov     [FindFile_DirEntryNumber], bx
                                <1>     ;mov    dx, ax ; Ambiguous Filename chars used sign > 0
                                <1>     ; 28/07/2022
000089DB 89C2                   <1>     mov     edx, eax
                                <1>
000089DD A0[76770100]           <1>     mov     al, [Current_Drv]
000089E2 A2[16800100]           <1>     mov     [FindFile_Drv], al
                                <1>
000089E7 A1[70770100]           <1>     mov     eax, [Current_Dir_FCluster]
000089EC A3[88800100]           <1>     mov     [FindFile_DirFirstCluster], eax
                                <1>
000089F1 A1[A27E0100]           <1>     mov     eax, [DirBuff_Cluster]
000089F6 A3[8C800100]           <1>     mov     [FindFile_DirCluster], eax
                                <1>
000089FB 66FF05[92800100]       <1>     inc     word [FindFile_MatchCounter]
                                <1>
00008A02 89FB                   <1>     mov     ebx, edi
00008A04 89FE                   <1>     mov     esi, edi
00008A06 BF[68800100]           <1>     mov     edi, FindFile_DirEntry
00008A0B 89F8                   <1>     mov     eax, edi
00008A0D B108                   <1>     mov     cl, 8
00008A0F F3A5                   <1>     rep     movsd
00008A11 89C6                   <1>     mov     esi, eax
00008A13 89DF                   <1>     mov     edi, ebx
                                <1>
00008A15 A1[84800100]           <1>     mov     eax, [FindFile_DirEntry+28] ; File Size
                                <1>
00008A1A 8A1D[73800100]         <1>     mov     bl, [FindFile_DirEntry+11] ; File Attributes
00008A20 8A3D[65800100]         <1>     mov     bh, [FindFile_LongNameYes]
                                <1>
                                <1>     ;mov    cx, [DirBuff_EntryCounter]
                                <1>     ;mov    [FindFile_DirEntryNumber], cx
                                <1>     ;mov    cx, [FindFile_DirEntryNumber]
                                <1>     ; ecx = 0
                                <1>
                                <1> loc_fff_retn:
00008A26 C3                     <1>     retn
                                <1>
                                <1> find_next_file:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
                                <1>     ; 10/02/2016
                                <1>     ; 08/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 06/02/2011
                                <1>     ; 17/09/2009
                                <1>     ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       NONE, Find First File Parameters
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 1 -> Error, Error Code in EAX (AL)
                                <1>     ;       CF = 0 ->
                                <1>     ;           ESI = Directory Entry (FindFile_DirEntry) Location
                                <1>     ;           EDI = Directory Buffer Directory Entry Location
                                <1>     ;           EAX = File Size
                                <1>     ;             BL = Attributes of The File/Directory
                                <1>     ;             BH = Long Name Yes/No Status (>0 is YES)
                                <1>     ;              DX > 0 : Ambiguous filename chars are used
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, ESI, EDI will be changed)
                                <1>
00008A27 66833D[92800100]00     <1>     cmp     word [FindFile_MatchCounter], 0
00008A2F 7707                   <1>     ja      short loc_start_search_next_file
                                <1>
                                <1> loc_fnf_stc_retn:
00008A31 F9                     <1>     stc
                                <1> loc_fnf_ax12h_retn:
00008A32 B80C000000             <1>     mov     eax, 12 ; No More files
                                <1> ;loc_fnf_retn:
00008A37 C3                     <1>     retn
                                <1>
                                <1> loc_start_search_next_file:
00008A38 668B1D[90800100]       <1>     mov     bx, [FindFile_DirEntryNumber]
                                <1>     ;inc    bx
                                <1>     ; 28/07/2022
00008A3F 43                     <1>     inc     ebx
00008A40 663B1D[A07E0100]       <1>     cmp     bx, [DirBuff_LastEntry]
00008A47 7719                   <1>     ja      short loc_cont_search_next_file
                                <1>
                                <1> loc_fnf_search:
00008A49 BE[07800100]           <1>     mov     esi, Dir_Entry_Name
00008A4E 66A1[66800100]         <1>     mov     ax, [FindFile_AttributesMask]
                                <1>     ;xor    cx, cx
                                <1>     ; 28/07/2022
00008A54 31C9                   <1>     xor     ecx, ecx
00008A56 E8881C0000             <1>     call    find_directory_entry
                                <1>     ;jnc    loc_fff_fnf_ln_check
                                <1>     ; 28/07/2022
00008A5B 7205                   <1>     jc      short loc_cont_search_next_file
00008A5D E957FFFFFF             <1>     jmp     loc_fff_fnf_ln_check
                                <1>
                                <1> loc_cont_search_next_file:
00008A62 31DB                   <1>     xor     ebx, ebx
00008A64 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
00008A6A BE00010900             <1>     mov     esi, Logical_DOSDisks
00008A6F 01DE                   <1>     add     esi, ebx
                                <1>
00008A71 803D[74770100]00       <1>     cmp     byte [Current_Dir_Level], 0
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     jna     short loc_fnf_check_FAT_type
                                <1>
                                <1>     cmp     byte [esi+LD_FATType], 1
                                <1>     jb      short loc_fnf_ax12h_retn
                                <1>
                                <1>     jmp     short loc_fnf_check_next_cluster
                                <1> %else
                                <1>     ; 19/12/2025
00008A78 7706                   <1>     ja      short loc_fnf_check_next_cluster
                                <1> %endif
                                <1>
                                <1> loc_fnf_check_FAT_type:
00008A7A 807E0303               <1>     cmp     byte [esi+LD_FATType], 3
00008A7E 72B2                   <1>     jb      short loc_fnf_ax12h_retn
                                <1>
                                <1> loc_fnf_check_next_cluster:
00008A80 A1[A27E0100]           <1>     mov     eax, [DirBuff_Cluster]
00008A85 E8FF340000             <1>     call    get_next_cluster
```

```
00008A8A 7306                <1>     jnc     short loc_fnf_load_next_dir_cluster
00008A8C 09C0                <1>     or      eax, eax
00008A8E 74A1                <1>     jz      short loc_fnf_stc_retn
                             <1>     ;mov    eax, 17 ;Drive not ready or read error
00008A90 F5                  <1>     cmc     ;stc
                             <1> loc_fnf_retn:
00008A91 C3                  <1>     retn
                             <1>
                             <1> loc_fnf_load_next_dir_cluster:
00008A92 E8AE360000          <1>     call    load_FAT_sub_directory
00008A97 72F8                <1>     jc      short loc_fnf_retn
                             <1>     ;xor    bx, bx
                             <1>     ; 28/07/2022
00008A99 31DB                <1>     xor     ebx, ebx
00008A9B 66891D[90800100]    <1>     mov     [FindFile_DirEntryNumber], bx
00008AA2 EBA5                <1>     jmp     short loc_fnf_search
                             <1>
                             <1> get_and_print_longname:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 16/10/2016
                             <1>     ; 13/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 24/01/2010
                             <1>     ; 17/10/2009 (CMD_INTR.ASM, 'cmp_cmd_longname')
                             <1> get_longname_fchar:
00008AA4 803E20              <1>     cmp     byte [esi], 20h
00008AA7 7701                <1>     ja      short loc_find_longname
                             <1>     ;jb     short loc_longname_retn
                             <1>     ;inc    esi
                             <1>     ;je     short get_longname_fchar
                             <1> ;loc_longname_retn:
00008AA9 C3                  <1>     retn
                             <1> loc_find_longname:
00008AAA E86C1F0000          <1>     call    find_longname
00008AAF 731D                <1>     jnc     short loc_print_longname
                             <1>
00008AB1 08C0                <1>     or      al, al
00008AB3 7412                <1>     jz      short loc_longname_not_found
                             <1>
                             <1>     ; 16/10/2016 (15h -> 15, 17)
00008AB5 3C0F                <1>     cmp     al, 15
                             <1>     ;je     cd_drive_not_ready ; drive not ready
                             <1>     ; 28/07/2022
00008AB7 7505                <1>     jne     short loc_fln_err2
                             <1> loc_fln_err1:
00008AB9 E92CF8FFFF          <1>     jmp     cd_drive_not_ready
                             <1> loc_fln_err2:
                             <1>                                     ; or
00008ABE 3C11                <1>     cmp     al, 17                  ; read error
                             <1>     ;je     cd_drive_not_ready
                             <1>     ; 28/07/2022
                             <1>     ;je     short loc_fln_err1
                             <1>
                             <1> loc_ln_file_dir_not_found:
00008AC0 BE[C2310100]        <1>     mov     esi, Msg_File_Directory_Not_Found
                             <1>     ;;call  print_msg
                             <1>         ;;retn
                             <1>     ;jmp    print_msg
                             <1>     ; 28/07/2022
00008AC5 EB1F                <1>     jmp     short loc_lfn_err3
                             <1>
                             <1> loc_longname_not_found:
00008AC7 BE[E1310100]        <1>         mov     esi, Msg_LongName_Not_Found
                             <1>     ;;call  print_msg
                             <1>         ;;retn
                             <1>     ;jmp    print_msg
                             <1>     ; 28/07/2022
00008ACC EB18                <1>     jmp     short loc_lfn_err3
                             <1>
                             <1> loc_print_longname:
                             <1>     ;mov    esi, LongFileName
00008ACE BF[76780100]        <1>     mov     edi, TextBuffer
00008AD3 57                  <1>     push    edi
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     al, 0
                             <1>     ja      short loc_print_longname_1
                             <1> loc_print_FS_longname: ; Singlix FS (64 byte ASCIIZ file name)
                             <1>     lodsb
                             <1>     stosb
                             <1>     or      al, al
                             <1>     jnz     short loc_print_FS_longname
                             <1>     jmp     short loc_print_longname_2
                             <1> %endif
                             <1>     ;
                             <1> loc_print_longname_1: ; MS Windows long name (UNICODE chars)
00008AD4 66AD                <1>     lodsw
00008AD6 AA                  <1>     stosb
00008AD7 08C0                <1>     or      al, al
00008AD9 75F9                <1>     jnz     short loc_print_longname_1
                             <1>     ;
                             <1> loc_print_longname_2:
00008ADB 5E                  <1>     pop     esi
00008ADC E8D9E2FFFF          <1>     call    print_msg
00008AE1 BE[17390100]        <1>     mov     esi, nextline
                             <1> loc_lfn_err3:
                             <1>     ;call   print_msg
                             <1>     ;retn
00008AE6 E9CFE2FFFF          <1>     jmp     print_msg
                             <1>
                             <1> show_file:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 07/08/2022
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 18/02/2016
                             <1>     ; 17/02/2016
                             <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 13/09/2011 (CMD_INTR.ASM, 'cmp_cmd_show')
                             <1>     ; 08/11/2009
                             <1>
                             <1> loc_show_parse_path_name:
00008AEB BF[16800100]        <1>     mov     edi, FindFile_Drv
00008AF0 E8891E0000          <1>     call    parse_path_name
                             <1>     ;jc     loc_cmd_failed
                             <1>     ; 28/07/2022
00008AF5 7305                <1>     jnc     short loc_show_check_filename_exists
                             <1> show_file_err1:
00008AF7 E958F9FFFF          <1>     jmp     loc_cmd_failed
                             <1>
                             <1> loc_show_check_filename_exists:
00008AFC BE[58800100]        <1>     mov     esi, FindFile_Name
00008B01 803E20              <1>     cmp     byte [esi], 20h
                             <1>     ;jna    loc_cmd_failed
```

```
                                <1>     ; 28/07/2022
00008B04 76F1                   <1>     jna     short show_file_err1
                                <1>
                                <1>     ; 15/02/2016 (invalid file name check)
00008B06 E8CE010000             <1>     call    check_filename
00008B0B 730A                   <1>     jnc     short loc_show_change_drv
                                <1>
00008B0D BE[AD320100]           <1>     mov     esi, Msg_invalid_name_chars
00008B12 E9A3E2FFFF             <1>     jmp     print_msg
                                <1>
                                <1> loc_show_change_drv:
00008B17 8A35[76770100]         <1>     mov     dh, [Current_Drv]
00008B1D 8835[D37E0100]         <1>     mov     [RUN_CDRV], dh
00008B23 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
00008B29 38F2                   <1>     cmp     dl, dh
00008B2B 740C                   <1>     je      short loc_show_change_directory
00008B2D E8BEEBFFFF             <1>     call    change_current_drive
                                <1>     ;;jc    loc_file_rw_cmd_failed
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 28/07/2022
00008B32 7305                   <1>     jnc     short loc_show_change_directory
                                <1> show_file_err2:
00008B34 E946F9FFFF             <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1> loc_show_change_directory:
00008B39 803D[17800100]20       <1>     cmp     byte [FindFile_Directory], 20h
00008B40 7614                   <1>     jna     short loc_findload_showfile
                                <1>
00008B42 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
00008B48 BE[17800100]           <1>     mov     esi, FindFile_Directory
00008B4D 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
00008B4F E897180000             <1>     call    change_current_directory
                                <1>     ;;jc    loc_file_rw_cmd_failed
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 28/07/2022
00008B54 72DE                   <1>     jc      short show_file_err2
                                <1>
                                <1> ;loc_show_change_prompt_dir_string:
                                <1>     ;call   change_prompt_dir_string
                                <1>
                                <1> loc_findload_showfile:
                                <1>     ; 15/02/2016
00008B56 BE[58800100]           <1>     mov     esi, FindFile_Name
00008B5B BF[07800100]           <1>     mov     edi, Dir_Entry_Name ; Dir Entry Format File Name
00008B60 E84F1D0000             <1>     call    convert_file_name
00008B65 89FE                   <1>     mov     esi, edi ; offset Dir_Entry_Name
                                <1>
00008B67 28C0                   <1>     sub     al, al  ; Attrib AND mask = 0
                                <1>     ; Directory attribute : 10h
                                <1>     ; Volume name attribute: 8h
00008B69 B418                   <1>     mov     ah, 00011000b ; 18h (Attrib NAND, AND --> zero mask)
                                <1>     ;
                                <1>     ;xor    cx, cx
                                <1>     ; 28/07/2022
00008B6B 31C9                   <1>     xor     ecx, ecx
00008B6D E8961A0000             <1>     call    locate_current_dir_file
                                <1>     ;;jc    loc_file_rw_cmd_failed
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 28/07/2022
00008B72 72C0                   <1>     jc      short show_file_err2
                                <1>
                                <1> loc_show_load_file:
                                <1>     ; EDI = Directory Entry
00008B74 668B4714               <1>     mov     ax, [edi+DirEntry_FstClusHI] ; First Cluster High Word
00008B78 C1E010                 <1>     shl     eax, 16
00008B7B 668B471A               <1>     mov     ax, [edi+DirEntry_FstClusLO] ; First Cluster Low Word
00008B7F A3[C0800100]           <1>     mov     [Show_Cluster], eax
00008B84 8B471C                 <1>     mov     eax, [edi+DirEntry_FileSize] ; File Size
00008B87 21C0                   <1>     and     eax, eax ; Empty file !
                                <1>     ;jz     end_of_show_file
                                <1>     ; 28/07/2022
00008B89 7505                   <1>     jnz     short loc_show_load_file_set_size
00008B8B E994000000             <1>     jmp     end_of_show_file
                                <1>
                                <1> loc_show_load_file_set_size: ; 28/07/2022
00008B90 A3[C4800100]           <1>     mov     [Show_FileSize], eax
00008B95 31C0                   <1>     xor     eax, eax
00008B97 A3[C8800100]           <1>     mov     [Show_FilePointer], eax ; 0
00008B9C 66A3[CC800100]         <1>     mov     [Show_ClusterPointer], ax ; 0
00008BA2 29DB                   <1>     sub     ebx, ebx
00008BA4 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
00008BAA BE00010900             <1>     mov     esi, Logical_DOSDisks
00008BAF 01DE                   <1>     add     esi, ebx
00008BB1 8935[BC800100]         <1>     mov     [Show_LDDDT], esi ; Logical DOS Drv Description Table addr
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ja      short loc_show_calculate_cluster_size
                                <1>     ; Singlix FS
                                <1>     ; First Cluster Number is FDT number (in compatibility buffer)
                                <1>     mov     edx, [Show_Cluster] ; Compatibility dir. buffer value (FDT)
                                <1>     mov     [Show_FDT], edx
                                <1>     ; 19/12/2025
                                <1>     ;xor    eax, eax
                                <1>     mov     [Show_Cluster], eax ; Sector index  = 0
                                <1>                                 ; (next time it will be 1)
                                <1> %endif
                                <1>
                                <1> loc_show_calculate_cluster_size:
                                <1>     ;mov    bx, [esi+LD_BPB+BPB_BytsPerSec] ; FAT 12-16-32 (512)
                                <1>     ; 19/12/2025
00008BB7 66BB0002               <1>     mov     bx, 512
                                <1>     ; BX = 512 = [esi+LD_FS_BytesPerSec] ; Singlix FS
00008BBB 8A4613                 <1>     mov     al, [esi+LD_BPB+BPB_SecPerClust] ; FAT 12-16-32 (<= 128)
                                <1>     ; AL = 1 = [esi+LD_FS_Reserved2] ; SectPerClust for Singlix FS
00008BBE F7E3                   <1>     mul     ebx
                                <1>
                                <1>     ;cmp    eax, 65536 ; non-compatible (very big) cluster size
                                <1>     ;ja     short end_of_show_file
00008BC0 66A3[CE800100]         <1>     mov     [Show_ClusterSize], ax
                                <1>
                                <1> loc_start_show_file:
00008BC6 BE[17390100]           <1>     mov     esi, nextline
00008BCB E8EAE1FFFF             <1>     call    print_msg
                                <1>
00008BD0 A1[C0800100]           <1>     mov     eax, [Show_Cluster]
00008BD5 C605[D0800100]17       <1>     mov     byte [Show_RowCount], 23
                                <1>
                                <1>     ; 17/02/2016
00008BDC 8B35[BC800100]         <1>     mov     esi, [Show_LDDDT]
                                <1>
                                <1>     ; 07/08/2022
                                <1> loc_show_next_cluster:
```

```
                                <1>     ; 15/02/2016
00008BE2 BB00000700              <1>     mov     ebx, Cluster_Buffer ; 70000h (for current TRDOS 386 version)
                                <1>     ; ESI = Logical DOS drv description table address
00008BE7 E88F350000              <1>     call    read_cluster
                                <1>     ;;jc    loc_file_rw_cmd_failed
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 07/08/2022
00008BEC 7305                    <1>     jnc     short loc_show_nc_rc_ok
00008BEE E98CF8FFFF              <1>     jmp     loc_run_cmd_failed
                                <1> loc_show_nc_rc_ok:
00008BF3 31DB                    <1>     xor     ebx, ebx
                                <1> loc_show_next_byte:
00008BF5 803D[D0800100]00        <1>     cmp     byte [Show_RowCount], 0
00008BFC 7512                    <1>     jne     short pass_show_wait_for_key
00008BFE 30E4                    <1>     xor     ah, ah
00008C00 E80D83FFFF              <1>     call    int16h
00008C05 3C1B                    <1>     cmp     al, 1Bh
                                <1>     ;jne    short pass_exit_show
                                <1>     ; 28/07/2022
00008C07 741B                    <1>     je      short end_of_show_file
                                <1>
                                <1> ;end_of_show_file:
                                <1> ;pass_show_file:
                                <1> ;   mov     esi, nextline
                                <1> ;   call    print_msg
                                <1> ;   jmp     loc_file_rw_restore_retn
                                <1>
                                <1> pass_exit_show:
00008C09 C605[D0800100]14        <1>     mov     byte [Show_RowCount], 20
                                <1> pass_show_wait_for_key:
00008C10 81C300000700            <1>     add     ebx, Cluster_Buffer
00008C16 8A03                    <1>     mov     al, [ebx]
00008C18 3C0D                    <1>     cmp     al, 0Dh
                                <1>     ;jne    loc_show_check_tab_space
                                <1>     ; 28/07/2022
00008C1A 7417                    <1>     je      short loc_show_dec_row_count
00008C1C EB7E                    <1>     jmp     loc_show_check_tab_space
                                <1>
                                <1>     ; 07/08/2022
                                <1> loc_show_next:
                                <1>     ;and    bx, bx ; 65536 -> 0
                                <1>     ; 28/07/2022
00008C1E 21DB                    <1>     and     ebx, ebx
00008C20 75D3                    <1>     jnz     short loc_show_next_byte
00008C22 EBBE                    <1>     jmp      short loc_show_next_cluster
                                <1>
                                <1>     ; 19/12/2025
                                <1>     ; 28/07/2022
                                <1> end_of_show_file:
                                <1> pass_show_file:
00008C24 BE[17390100]            <1>     mov     esi, nextline
00008C29 E88CE1FFFF              <1>     call    print_msg
00008C2E E915010000              <1>     jmp     loc_file_rw_restore_retn
                                <1>
                                <1> loc_show_dec_row_count:    ; 28/07/2022
00008C33 FE0D[D0800100]          <1>     dec     byte [Show_RowCount]
                                <1> pass_show_dec_rowcount:
00008C39 B307                    <1>     mov     bl, 7 ; (light gray character color, black background)
00008C3B 8A3D[DE760100]          <1>     mov     bh, [ACTIVE_PAGE] ; [ptty]
00008C41 E89996FFFF              <1>     call    _write_tty
                                <1> loc_show_check_eof:
00008C46 FF05[C8800100]          <1>     inc     dword [Show_FilePointer]
00008C4C A1[C8800100]            <1>     mov     eax, [Show_FilePointer]
00008C51 3B05[C4800100]          <1>     cmp     eax, [Show_FileSize]
00008C57 73CB                    <1>     jnb     short end_of_show_file
00008C59 66FF05[CC800100]        <1>     inc     word [Show_ClusterPointer]
00008C60 0FB71D[CC800100]        <1>     movzx   ebx, word [Show_ClusterPointer]
                                <1>
                                <1>     ; 17/02/2016
                                <1>     ; (sector boundary -9 bits- check, 512 = 0)
00008C67 66F7C3FF01              <1>         test    bx, 1FFh ; 1 to 511
00008C6C 7587                    <1>     jnz     short loc_show_next_byte
                                <1>
                                <1>     ; 16/02/2016
00008C6E 8B35[BC800100]          <1>     mov     esi, [Show_LDDDT]
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ja      short loc_show_check_fat_cluster_size
                                <1>
                                <1>     ; Singlix FS
                                <1>     ; 1 sector, more... (cluster size = 1 sector)
                                <1>     mov     eax, [Show_Cluster]
                                <1>     inc     eax
                                <1>     mov     [Show_Cluster], eax
                                <1>
                                <1>     ; 07/08/2022
                                <1>     jmp     short loc_show_next
                                <1>
                                <1>     ; 28/07/2022
                                <1> end_of_show_file:
                                <1> pass_show_file:
                                <1>     mov     esi, nextline
                                <1>     call    print_msg
                                <1>     jmp     loc_file_rw_restore_retn
                                <1> %endif
                                <1>
                                <1> loc_show_check_fat_cluster_size:
                                <1>     ; 17/02/2016
00008C74 663B1D[CE800100]        <1>     cmp     bx, [Show_ClusterSize] ; cluster size in bytes
                                <1>         ;jb short loc_show_next_byte ; 28/07/2022
                                <1>     ; 07/08/2022
00008C7B 7305                    <1>     jnb     short loc_show_file_cluster_ok
00008C7D E973FFFFFF              <1>     jmp     loc_show_next_byte
                                <1>
                                <1> loc_show_file_cluster_ok:
00008C82 66C705[CC800100]00-     <1>     mov     word [Show_ClusterPointer], 0
00008C8A 00                      <1>
                                <1>
00008C8B A1[C0800100]            <1>     mov     eax, [Show_Cluster]
                                <1>     ;mov    esi, [Show_LDDDT]
                                <1> loc_show_get_next_cluster:
00008C90 E8F4320000              <1>     call    get_next_cluster
                                <1>     ;;jc    loc_file_rw_cmd_failed
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 28/07/2022
00008C95 7338                    <1>     jnc     short loc_show_update_ccluster
00008C97 E9E3F7FFFF              <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1> loc_show_check_tab_space:
00008C9C 3C09                    <1>     cmp     al, 09h
                                <1>     ;jne    short pass_show_dec_rowcount ; 28/07/2022
                                <1>     ; 07/08/2022
```

```
00008C9E 7402                <1>     je      short loc_show_put_tab_space
00008CA0 EB97                <1>     jmp     pass_show_dec_rowcount
                             <1> loc_show_put_tab_space:
00008CA2 8A3D[DE760100]      <1>     mov     bh, [ACTIVE_PAGE] ; [ptty]
00008CA8 E8D092FFFF          <1>     call    get_cpos
                             <1>     ; dl = cursor column
00008CAD 80E207              <1>     and     dl, 7 ; 18/02/2016
                             <1>     ;shr    bh, 1 ; [ACTIVE_PAGE]
00008CB0 8A3D[DE760100]      <1>     mov     bh, [ACTIVE_PAGE]
00008CB6 B307                <1>     mov     bl, 7 ; color attribute
                             <1> loc_show_put_space_chars:
00008CB8 B020                <1>     mov     al, 20h ; space
                             <1>     ;mov    bh, [ACTIVE_PAGE] ; [ptty]
                             <1>     ;mov    bl, 7 ; color attribute
                             <1>     ;push   dx
00008CBA 52                  <1>     push    edx ; 29/12/2017
00008CBB E81F96FFFF          <1>     call    _write_tty
00008CC0 5A                  <1>     pop     edx ; 29/12/2017
                             <1>     ;pop    dx
                             <1>     ; 18/02/2016
00008CC1 80FA07              <1>     cmp     dl, 7
                             <1>     ;jnb    short loc_show_check_eof ; 28/07/2022
                             <1>     ; 07/08/2022
00008CC4 7205                <1>     jb      short loc_show_next_tab_space
00008CC6 E97BFFFFFF          <1>     jmp     loc_show_check_eof
                             <1> loc_show_next_tab_space:
00008CCB FEC2                <1>     inc     dl
00008CCD EBE9                <1>     jmp     short loc_show_put_space_chars
                             <1>
                             <1> loc_show_update_ccluster:
00008CCF A3[C0800100]        <1>     mov     [Show_Cluster], eax
00008CD4 E909FFFFFF          <1>         jmp     loc_show_next_cluster
                             <1>
                             <1> check_filename:
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 10/10/2016
                             <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 07/08/2010 (FILE.ASM, 'proc_check_filename')
                             <1>     ; 10/07/2010
                             <1>     ; Derived from 'proc_check_filename'
                             <1>     ; in the old TRDOS.ASM (09/02/2005).
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Dot File Name Location
                             <1>     ; OUTPUT ->
                             <1>     ;       cf = 1 -> error code in AL
                             <1>     ;            AL = ERR_INV_FILE_NAME (=26)
                             <1>     ;                Invalid file name chars
                             <1>     ;       cf = 0 -> valid file name
                             <1>     ;
                             <1>     ; (EAX, ECX, EDI will be changed)
                             <1>
                             <1> check_invalid_filename_chars:
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 10/07/2010 (FILE.ASM, 'proc_check_invalid_filename_chars')
                             <1>     ; 10/02/2010
                             <1>     ; Derived from 'proc_check_invalid_filename_chars'
                             <1>     ; in the old TRDOS.ASM (09/02/2005).
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       ESI = ASCIIZ FileName
                             <1>     ; OUTPUT ->
                             <1>     ;       cf = 1 -> invalid
                             <1>     ;       cf = 0 -> valid
                             <1>     ;
                             <1>     ;(EAX, ECX, EDI will be changed)
                             <1>
00008CD9 56                  <1>     push    esi
                             <1>
00008CDA BF[962F0100]        <1>         mov     edi, invalid_fname_chars
00008CDF AC                  <1>     lodsb
                             <1> check_filename_next_char:
00008CE0 B914000000          <1>     mov     ecx, sizeInvFnChars
00008CE5 BF[962F0100]        <1>     mov     edi, invalid_fname_chars
                             <1> loc_scan_invalid_filename_char:
00008CEA AE                  <1>     scasb
00008CEB 741E                <1>     je      short loc_invalid_filename_stc
00008CED E2FB                <1>     loop    loc_scan_invalid_filename_char
00008CEF AC                  <1>     lodsb
00008CF0 3C1F                <1>     cmp     al, 1Fh  ; 20h and above
00008CF2 77EC                <1>     ja      short check_filename_next_char
                             <1>
                             <1> check_filename_dot:
00008CF4 8B3424              <1>     mov     esi, [esp]
                             <1>
00008CF7 B421                <1>     mov     ah, 21h
                             <1>     ;mov    ecx, 8
                             <1>     ; 28/07/2022
00008CF9 29C9                <1>     sub     ecx, ecx
00008CFB B108                <1>     mov     cl, 8
                             <1> loc_check_filename_next_char:
00008CFD AC                  <1>     lodsb
00008CFE 3C2E                <1>     cmp     al, 2Eh
00008D00 7511                <1>     jne     short pass_check_fn_dot_check
                             <1> loc_check_filename_ext_0:
00008D02 AC                  <1>     lodsb
00008D03 38E0                <1>     cmp     al, ah ; 21h
00008D05 7205                <1>     jb      short loc_invalid_filename
00008D07 3C2E                <1>     cmp     al, 2Eh
00008D09 7519                <1>     jne     short loc_check_filename_ext_1
                             <1>
                             <1> loc_invalid_filename_stc:
                             <1> loc_check_fn_stc_rtn:
00008D0B F9                  <1>     stc
                             <1> loc_invalid_filename:
                             <1>     ; 10/10/2016 (0Bh -> 26)
00008D0C B81A000000          <1>     mov     eax, ERR_INV_FILE_NAME ; (=26)
                             <1>     ; Invalid file name chars
                             <1> loc_check_fn_rtn:
00008D11 5E                  <1>     pop     esi
00008D12 C3                  <1>     retn
                             <1>
                             <1> pass_check_fn_dot_check:
00008D13 38E0                <1>     cmp     al, ah ; 21h
00008D15 7224                <1>     jb      short loc_check_fn_clc_rtn
00008D17 E2E4                <1>     loop    loc_check_filename_next_char
00008D19 AC                  <1>     lodsb
00008D1A 38E0                <1>     cmp     al, ah ; 21h
00008D1C 721D                <1>     jb      short loc_check_fn_clc_rtn
00008D1E 3C2E                <1>     cmp     al, 2Eh
00008D20 75E9                <1>     jne     short loc_check_fn_stc_rtn
00008D22 EBDE                <1>     jmp     short loc_check_filename_ext_0
                             <1>
```

```
                                <1> loc_check_filename_ext_1:
00008D24 AC                     <1>     lodsb
00008D25 38E0                   <1>     cmp     al, ah ; 21h
00008D27 7212                   <1>     jb      short loc_check_fn_clc_rtn
00008D29 3C2E                   <1>     cmp     al, 2Eh
00008D2B 74DE                   <1>     je      short loc_check_fn_stc_rtn
00008D2D AC                     <1>     lodsb
00008D2E 38E0                   <1>     cmp     al, ah ; 21h
00008D30 7209                   <1>     jb      short loc_check_fn_clc_rtn
00008D32 3C2E                   <1>     cmp     al, 2Eh
00008D34 74D5                   <1>     je      short loc_check_fn_stc_rtn
00008D36 AC                     <1>     lodsb
00008D37 38E0                   <1>     cmp     al, ah ; 21h
00008D39 73D0                   <1>     jnb     short loc_check_fn_stc_rtn
                                <1>
                                <1> loc_check_fn_clc_rtn:
00008D3B 5E                     <1>     pop     esi
00008D3C F8                     <1>     clc
00008D3D C3                     <1>     retn
                                <1>
                                <1> loc_print_deleted_message:
00008D3E BE[82330100]           <1>     mov     esi, Msg_Deleted
00008D43 E872E0FFFF             <1>     call    print_msg
                                <1>
                                <1>     ;clc
                                <1>
                                <1> loc_file_rw_restore_retn:
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 28/02/2010 (CMD_INTR.ASM)
                                <1> loc_file_rw_cmd_failed:
00008D48 9C                     <1>     pushf
00008D49 E899F7FFFF             <1>     call    restore_cdir_after_cmd_fail
00008D4E 9D                     <1>     popf
00008D4F 720D                   <1>     jc      short loc_file_rw_check_write_fault
00008D51 C3                     <1>     retn
                                <1>
                                <1> loc_permission_denied:
                                <1>     ; 27/02/2016
00008D52 BE[8F330100]           <1>     mov     esi, Msg_Permission_Denied
00008D57 E85EE0FFFF             <1>     call    print_msg
00008D5C EBEA                   <1>     jmp     short loc_file_rw_restore_retn
                                <1>
                                <1> loc_file_rw_check_write_fault:
                                <1>     ;cmp    al, 1Dh ; Write Fault
00008D5E 3C12                   <1>         cmp al, 18 ; 05/11/2016
                                <1>     ;jne    loc_run_cmd_failed_cmp_al
                                <1>     ; 28/07/2022
00008D60 7405                   <1>     je      short loc_file_rw_fault
00008D62 E91DF7FFFF             <1>     jmp     loc_run_cmd_failed_cmp_al
                                <1>
                                <1> loc_file_rw_fault:
00008D67 BE[77310100]           <1>     mov     esi, Msg_Not_Ready_Write_Err
                                <1>     ;call   print_msg
                                <1>     ;retn
00008D6C E949E0FFFF             <1>     jmp     print_msg
                                <1>
                                <1> make_directory:
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 12/03/2011 (CMD_INTR.ASM, 'cmp_cmd_mkdir')
                                <1>     ; 14/08/2010
                                <1>     ; 10/07/2010
                                <1>     ; 29/11/2009
                                <1>     ;
                                <1> get_mkdir_fchar:
                                <1>     ; esi = directory name
00008D71 803E20                 <1>     cmp     byte [esi], 20h
00008D74 7701                   <1>         ja  short loc_mkdir_parse_path_name
                                <1>
                                <1> loc_mkdir_nodirname_retn:
00008D76 C3                     <1>     retn
                                <1>
                                <1> loc_mkdir_parse_path_name:
00008D77 BF[16800100]           <1>     mov     edi, FindFile_Drv
00008D7C E8FD1B0000             <1>         call    parse_path_name
                                <1>     ;jc     loc_cmd_failed
                                <1>     ; 28/07/2022
00008D81 7305                   <1>     jnc     short loc_mkdir_check_dirname_exists
                                <1> loc_mkdir_cmd_failed:
00008D83 E9CCF6FFFF             <1>     jmp     loc_cmd_failed
                                <1>
                                <1> loc_mkdir_check_dirname_exists:
00008D88 BE[58800100]           <1>     mov     esi, FindFile_Name
00008D8D 803E20                 <1>     cmp     byte [esi], 20h
                                <1>     ;jna    loc_cmd_failed
                                <1>     ; 28/07/2022
00008D90 76F1                   <1>     jna     short loc_mkdir_cmd_failed
00008D92 8935[D4800100]         <1>     mov     [DelFile_FNPointer], esi
00008D98 E83CFFFFFF             <1>     call    check_filename
00008D9D 725B                   <1>     jc      short loc_mkdir_invalid_dir_name_chars
                                <1>
                                <1> loc_mkdir_drv:
00008D9F 8A35[76770100]         <1>     mov     dh, [Current_Drv]
00008DA5 8835[D37E0100]         <1>     mov     [RUN_CDRV], dh
                                <1>
00008DAB 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
00008DB1 38F2                   <1>     cmp     dl, dh
00008DB3 7409                   <1>     je      short loc_mkdir_change_directory
                                <1>
00008DB5 E836E9FFFF             <1>     call    change_current_drive
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
00008DBA 7302                   <1>     jnc     short loc_mkdir_change_directory
00008DBC EB8A                   <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_mkdir_change_directory:
00008DBE 803D[17800100]20       <1>     cmp     byte [FindFile_Directory], 20h
00008DC5 7614                   <1>     jna     short loc_mkdir_find_directory
                                <1>
00008DC7 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
00008DCD BE[17800100]           <1>     mov     esi, FindFile_Directory
00008DD2 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
00008DD4 E812160000             <1>     call    change_current_directory
00008DD9 7226                   <1>     jc      short loc_mkdir_check_error_code
                                <1>
                                <1> ;loc_mkdir_change_prompt_dir_string:
                                <1>     ;call   change_prompt_dir_string
                                <1>
                                <1> loc_mkdir_find_directory:
                                <1>     ;mov    esi, FindFile_Name
00008DDB 8B35[D4800100]         <1>     mov     esi, [DelFile_FNPointer]
                                <1>     ;xor    eax, eax
00008DE1 6631C0                 <1>     xor     ax, ax ; any name (dir, file, volume)
00008DE4 E891FBFFFF             <1>     call    find_first_file
```

```
00008DE9 7216                <1>     jc      short loc_mkdir_check_error_code
                             <1>
                             <1> loc_mkdir_directory_found:
00008DEB BE[DA320100]        <1>     mov     esi, Msg_Name_Exists
                             <1> loc_mkdir_inv_dname_chrs_msg:
00008DF0 E8C5DFFFFF          <1>     call    print_msg
00008DF5 E94EFFFFFF          <1>     jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_mkdir_invalid_dir_name_chars:
00008DFA BE[AD320100]        <1>     mov     esi, Msg_invalid_name_chars
                             <1>     ;call   print_msg
                             <1>         ;jmp       loc_file_rw_restore_retn
                             <1>     ; 28/07/2022
00008DFF EBEF                <1>     jmp     short loc_mkdir_inv_dname_chrs_msg
                             <1>
                             <1> loc_mkdir_check_error_code:
00008E01 3C02                <1>     cmp     al, 2
                             <1>     ;je     short loc_mkdir_directory_not_found
00008E03 7406                <1>     je      short loc_mkdir_ask_for_yes_no
00008E05 F9                  <1>     stc
00008E06 E93DFFFFFF          <1>         jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> loc_mkdir_directory_not_found:
                             <1> loc_mkdir_ask_for_yes_no:
00008E0B BE[FB320100]        <1>     mov     esi, Msg_DoYouWantMkdir
00008E10 E8A5DFFFFF          <1>     call    print_msg
00008E15 8B35[D4800100]      <1>     mov     esi, [DelFile_FNPointer]
00008E1B E89ADFFFFF          <1>     call    print_msg
00008E20 BE[1A330100]        <1>     mov     esi, Msg_YesNo
00008E25 E890DFFFFF          <1>     call    print_msg
                             <1>
00008E2A C605[24330100]20    <1>     mov     byte [Y_N_nextline], 20h
                             <1>
                             <1> loc_mkdir_ask_again:
00008E31 30E4                <1>     xor     ah, ah
00008E33 E8DA80FFFF          <1>     call    int16h
00008E38 3C1B                <1>     cmp     al, 1Bh
                             <1>     ;je     short loc_do_not_make_directory
00008E3A 743B                <1>     je      short loc_mkdir_y_n_escape
00008E3C 24DF                <1>     and     al, 0DFh ; y -> Y, n -> N
00008E3E 3C59                <1>     cmp     al, 'Y' ; 'yes'
00008E40 7404                <1>     je      short loc_mkdir_yes_make_directory
00008E42 3C4E                <1>     cmp     al, 'N' ; 'no'
00008E44 75EB                <1>     jne     short loc_mkdir_ask_again
                             <1>
                             <1> loc_do_not_make_directory:
                             <1> loc_mkdir_yes_make_directory:
00008E46 E830000000          <1>     call    y_n_answer ; 29/12/2017
                             <1>     ;cmp    al, 'Y' ; 'yes'
                             <1>     ;cmc
                             <1>         ;jnc       loc_file_rw_restore_retn
00008E4B 3C4E                <1>     cmp     al, 'N' ; 'no'
                             <1>     ;je     loc_file_rw_restore_retn
                             <1>     ; 28/07/2022
00008E4D 7505                <1>     jne     short loc_mkdir_call_make_sub_dir
00008E4F E9F4FEFFFF          <1>     jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_mkdir_call_make_sub_dir:
00008E54 8B35[D4800100]      <1>     mov     esi, [DelFile_FNPointer]
00008E5A B110                <1>     mov     cl, 10h ; Directory attributes
00008E5C E8001C0000          <1>     call    make_sub_directory
                             <1> loc_rename_file_ok: ; 06/03/2016
                             <1>         ;jc loc_file_rw_cmd_failed
                             <1>     ; 28/07/2022
00008E61 7305                <1>     jnc     short move_source_file_to_dest_OK
00008E63 E9E0FEFFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1> move_source_file_to_dest_OK:
00008E68 BE[28330100]        <1>     mov     esi, Msg_OK
00008E6D E848DFFFFF          <1>     call    print_msg
00008E72 E9D1FEFFFF          <1>     jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_mkdir_y_n_escape:
00008E77 B04E                <1>     mov     al, 'N' ; 'no'
00008E79 EBCB                <1>     jmp     short loc_do_not_make_directory
                             <1>
                             <1> y_n_answer:
                             <1>     ; 29/12/2017
00008E7B A2[24330100]        <1>     mov     [Y_N_nextline], al
                             <1>     ;push   ax
00008E80 50                  <1>     push    eax
00008E81 BE[24330100]        <1>     mov     esi, Y_N_nextline
00008E86 E82FDFFFFF          <1>     call    print_msg
00008E8B 58                  <1>     pop     eax
                             <1>     ;pop    ax
00008E8C C3                  <1>     retn
                             <1>
                             <1> delete_directory:
                             <1>     ; 17/07/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 29/12/2017
                             <1>     ; 15/10/2016
                             <1>     ; 01/03/2016, 06/03/2016
                             <1>     ; 27/02/2016, 28/02/2016, 29/02/2016
                             <1>     ; 26/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 16/10/2010 (CMD_INTR.ASM, 'cmp_cmd_rmdir')
                             <1>     ; 05/06/2010
                             <1>     ;
                             <1> get_fchar:
                             <1>     ; esi = directory name
00008E8D 803E20              <1>     cmp     byte [esi], 20h
00008E90 7701                <1>         ja  short loc_rmdir_parse_path_name
                             <1>
                             <1> loc_rmdir_nodirname_retn:
00008E92 C3                  <1>     retn
                             <1>
                             <1> loc_rmdir_parse_path_name:
00008E93 BF[16800100]        <1>     mov     edi, FindFile_Drv
00008E98 E8E11A0000          <1>     call    parse_path_name
                             <1>     ;jc     loc_cmd_failed
                             <1>     ; 28/07/2022
00008E9D 7305                <1>     jnc     short loc_rmdir_check_dirname_exists
                             <1> lc_del_dir_failed:
00008E9F E9B0F5FFFF          <1>     jmp     loc_cmd_failed
                             <1>
                             <1> loc_rmdir_check_dirname_exists:
00008EA4 BE[58800100]        <1>     mov     esi, FindFile_Name
00008EA9 803E20              <1>     cmp     byte [esi], 20h
                             <1>     ;jna    loc_cmd_failed
                             <1>     ; 28/07/2022
00008EAC 76F1                <1>     jna     short lc_del_dir_failed
00008EAE 8935[D4800100]      <1>     mov     [DelFile_FNPointer], esi
                             <1>
                             <1> loc_rmdir_drv:
00008EB4 8A35[76770100]      <1>     mov     dh, [Current_Drv]
```

```
00008EBA 8835[D37E0100]      <1>     mov     [RUN_CDRV], dh
                             <1>
                             <1>     ;;;;
                             <1>     ; 17/07/2025 - TRDOS 386 v2.0.10
00008EC0 8835[1C810100]      <1>     mov     [rmdir_drv], dh
00008EC6 8B1D[70770100]      <1>     mov     ebx, [Current_Dir_FCluster]
00008ECC 891D[18810100]      <1>     mov     [rmdir_dir_fcluster], ebx
                             <1>     ;;;;
                             <1>
00008ED2 8A15[16800100]      <1>     mov     dl, [FindFile_Drv]
00008ED8 38F2                <1>     cmp     dl, dh
00008EDA 7407                <1>     je      short loc_rmdir_change_directory
                             <1>
00008EDC E80FE8FFFF          <1>     call    change_current_drive
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 28/07/2022
                             <1>     ;jnc    short loc_rmdir_change_directory
                             <1>     ;jmp    loc_file_rw_cmd_failed
00008EE1 7233                <1>     jc      short loc_rmdir_chdrv_failed
                             <1>
                             <1> loc_rmdir_change_directory:
00008EE3 803D[17800100]20    <1>     cmp     byte [FindFile_Directory], 20h
00008EEA 7614                <1>     jna     short loc_rmdir_find_directory
                             <1>
00008EEC FE05[8E2E0100]      <1>     inc     byte [Restore_CDIR]
00008EF2 BE[17800100]        <1>     mov     esi, FindFile_Directory
00008EF7 30E4                <1>     xor     ah, ah ; CD_COMMAND sign -> 0
00008EF9 E8ED140000          <1>     call    change_current_directory
00008EFE 7211                <1>     jc      short loc_rmdir_check_error_code
                             <1>
                             <1> ;loc_rmdir_change_prompt_dir_string:
                             <1>     ;call   change_prompt_dir_string
                             <1>
                             <1> loc_rmdir_find_directory:
                             <1>     ;mov    esi, FindFile_Name
00008F00 8B35[D4800100]      <1>     mov     esi, [DelFile_FNPointer]
00008F06 66B81008            <1>     mov     ax, 0810h ; Only directories
00008F0A E86BFAFFFF          <1>     call    find_first_file
00008F0F 730A                <1>     jnc     short loc_rmdir_ambgfn_check
                             <1>
                             <1> loc_rmdir_check_error_code:
00008F11 3C02                <1>     cmp     al, 2
00008F13 740B                <1>     je      short loc_rmdir_directory_not_found
00008F15 F9                  <1>     stc
                             <1> loc_rmdir_chdrv_failed:
00008F16 E92DFEFFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> loc_rmdir_ambgfn_check:
00008F1B 6621D2              <1>     and     dx, dx ; Ambiguous filename chars used sign (DX>0)
00008F1E 740F                <1>     jz      short loc_rmdir_directory_found
                             <1>
                             <1> loc_rmdir_directory_not_found:
00008F20 BE[99310100]        <1>     mov     esi, Msg_Dir_Not_Found
00008F25 E890DEFFFF          <1>     call    print_msg
                             <1>
00008F2A E919FEFFFF          <1>     jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_rmdir_directory_found:
00008F2F 80E307              <1>     and     bl, 07h ; Attributes
                             <1>     ;jnz    loc_permission_denied
                             <1>     ; 28/07/2022
00008F32 7405                <1>     jz      short loc_rmdir_save_lnel
00008F34 E919FEFFFF          <1>     jmp     loc_permission_denied
                             <1>
                             <1> loc_rmdir_save_lnel: ; 28/02/2016
                             <1>         ;mov bh, [LongName_EntryLength]
00008F39 883D[DE800100]      <1>     mov     [DelFile_LNEL], bh ; Long name entry length (if > 0)
                             <1>     ; edi = Directory Entry Offset (DirBuff)
                             <1>     ; esi = Directory Entry (FFF Structure)
                             <1>     ;mov    [DelFile_DirEntryAddr], edi ; not required
                             <1>     ;mov    ax, [edi+20] ; First Cluster High Word
                             <1>         ;shl        eax, 16
                             <1>     ;mov    ax, [edi+26] ; First Cluster Low Word
                             <1>     ; ROOT Dir First Cluster = 0
                             <1>         ;cmp        eax, 2
                             <1>     ;jb     loc_update_direntry_1
                             <1>
                             <1> pass_rmdir_fc_check:
00008F3F 57                  <1>     push    edi ; * (29/02/2016)
                             <1>
00008F40 BE[2E330100]        <1>     mov     esi, Msg_DoYouWantRmDir
00008F45 E870DEFFFF          <1>     call    print_msg
00008F4A 8B35[D4800100]      <1>     mov     esi, [DelFile_FNPointer]
00008F50 E865DEFFFF          <1>     call    print_msg
00008F55 BE[1A330100]        <1>     mov     esi, Msg_YesNo
00008F5A E85BDEFFFF          <1>     call    print_msg
                             <1>
                             <1> loc_rmdir_ask_again:
00008F5F 30E4                <1>     xor     ah, ah
00008F61 E8AC7FFFFF          <1>     call    int16h
00008F66 3C1B                <1>     cmp     al, 1Bh
                             <1>     ;je     short loc_do_not_delete_directory
00008F68 742F                <1>         je  short loc_rmdir_y_n_escape ; 06/03/2016
00008F6A 24DF                <1>     and     al, 0DFh
00008F6C A2[24330100]        <1>     mov     [Y_N_nextline], al
00008F71 3C59                <1>     cmp     al, 'Y'
00008F73 7404                <1>     je      short loc_rmdir_yes_delete_directory
00008F75 3C4E                <1>     cmp     al, 'N'
00008F77 75E6                <1>     jne     short loc_rmdir_ask_again
                             <1>
                             <1> loc_do_not_delete_directory:
                             <1> loc_rmdir_yes_delete_directory:
00008F79 E8FDFEFFFF          <1>     call    y_n_answer ; 29/12/2017
00008F7E 5F                  <1>     pop     edi ; * (29/02/2016)
                             <1>     ;cmp    al, 'Y' ; 'yes'
                             <1>     ;cmc
                             <1>         ;jnc        loc_file_rw_restore_retn
00008F7F 3C4E                <1>     cmp     al, 'N' ; 'no'
                             <1>     ;je     loc_file_rw_restore_retn
                             <1>     ; 28/07/2022
                             <1>     ;jne    short loc_delete_sub_dir
                             <1>     ;jmp    loc_file_rw_restore_retn
00008F81 7411                <1>     je      short loc_rmdir_rw_restore_retn
                             <1>
                             <1> loc_delete_sub_dir:
                             <1>     ; 29/12/2017
00008F83 E85E000000          <1>     call    delete_sub_directory
00008F88 7213                <1>     jc      short loc_rmdir_cmd_failed
                             <1>
                             <1> loc_rmdir_ok:
00008F8A BE[28330100]        <1>     mov     esi, Msg_OK
00008F8F E826DEFFFF          <1>     call    print_msg
                             <1> loc_rmdir_rw_restore_retn: ; 28/07/2022
00008F94 E9AFFDFFFF          <1>     jmp     loc_file_rw_restore_retn
```

```
                                <1>
                                <1> loc_rmdir_y_n_escape:
00008F99 B04E                   <1>     mov     al, 'N' ; 'no'
00008F9B EBDC                   <1>         jmp     loc_do_not_delete_directory
                                <1>
                                <1> loc_rmdir_cmd_failed:
                                <1>     ; 29/12/2017
00008F9D 09C0                   <1>     or      eax, eax ; EAX = 0 -> Directory not empty!
00008F9F 7423                   <1>     jz      short loc_rmdir_directory_not_empty
                                <1>
                                <1>     ; EAX > 0 -> Error code in AL (or AX or EAX)
                                <1>
00008FA1 833D[927E0100]01       <1>     cmp     dword [FAT_ClusterCounter], 1
                                <1>     ;jb     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
00008FA8 7305                   <1>     jnb     short loc_rmdir_failed
00008FAA E999FDFFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_rmdir_failed:
00008FAF F9                     <1>     stc
                                <1> loc_rmdir_cmd_return:
                                <1>     ; 01/03/2016
00008FB0 9C                     <1>     pushf
                                <1>     ; ESI = Logical DOS Drive Description Table address
00008FB1 66BB00FF               <1>     mov     bx, 0FF00h ; BH = FFh -> use ESI for Drive parameters
                                <1>                ; BL = 0 -> Recalculate free cluster count
00008FB5 50                     <1>     push    eax
00008FB6 E8B6350000             <1>     call    calculate_fat_freespace
00008FBB 58                     <1>     pop     eax
00008FBC 9D                     <1>     popf
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ;jmp    loc_file_rw_restore_retn
                                <1>     ; 28/07/2022
00008FBD 73D5                   <1>     jnc     short loc_rmdir_rw_restore_retn
00008FBF E984FDFFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_rmdir_directory_not_empty:
00008FC4 BE[4F330100]           <1>     mov     esi, Msg_Dir_Not_Empty
00008FC9 E8ECDDFFFF             <1>     call    print_msg
                                <1>     ; 01/03/2016
00008FCE A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter]
00008FD3 09C0                   <1>     or      eax, eax ; 0 ?
                                <1>     ;jz     loc_file_rw_restore_retn
                                <1>     ; 28/07/2022
00008FD5 74BD                   <1>     jz      short loc_rmdir_rw_restore_retn
                                <1>
                                <1>     ; ESI = Logical DOS Drive Description Table address
00008FD7 66BB01FF               <1>     mov     bx, 0FF01h ; BH = FFh -> use ESI for Drive parameters
                                <1>                ; BL = 1 -> add free clusters
00008FDB E891350000             <1>     call    calculate_fat_freespace
00008FE0 09C9                   <1>     or      ecx, ecx
                                <1>     ;jz     loc_file_rw_restore_retn ; ecx = 0 -> OK
                                <1>     ;; ecx > 0 -> Error (Recalculation is needed)
                                <1>     ;jmp    short loc_rmdir_cmd_return
                                <1>     ; 28/07/2022
00008FE2 75CC                   <1>     jnz     short loc_rmdir_cmd_return
00008FE4 EBAE                   <1>     jmp     short loc_rmdir_rw_restore_retn
                                <1>     ;jmp    loc_file_rw_restore_retn
                                <1>
                                <1> delete_sub_directory:
                                <1>     ; 19/12/2025
                                <1>     ; 17/07/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 29/12/2017
                                <1>     ; (moved here from 'delete_directory' for 'sysrmdir' )
                                <1>
                                <1>     ; EDI = Directory buffer entry offset/address
                                <1>
                                <1> loc_rmdir_delete_short_name_check_dir_empty:
00008FE6 668B4714               <1>     mov     ax, [edi+20] ; First Cluster High Word
00008FEA C1E010                 <1>         shl eax, 16
00008FED 668B471A               <1>     mov     ax, [edi+26] ; First Cluster Low Word
                                <1>
                                <1>     ;;;
                                <1>     ; 17/07/2025
                                <1>     ; (sure it is not the current directory)
00008FF1 8A15[1C810100]         <1>     mov     dl, [rmdir_drv]
00008FF7 3A15[76770100]         <1>     cmp     dl, [Current_Drv]
00008FFD 7508                   <1>     jne     short loc_rmdir_delete_short_name_check_dot
00008FFF 3B05[18810100]         <1>     cmp     eax, [rmdir_dir_fcluster]
00009005 743F                   <1>     je      short loc_rmdir_permission_denied
                                <1> loc_rmdir_delete_short_name_check_dot:
                                <1>     ; (DOT and DOTDOT can not be deleted)
00009007 803F2E                 <1>     cmp     byte [edi], '.'
0000900A 750F                   <1>     jne     short loc_rmdir_delete_short_name_skip_dot
0000900C 66813F2E20             <1>     cmp     word [edi], '. '
00009011 7433                   <1>     je      short loc_rmdir_permission_denied
00009013 813F2E2E2020           <1>     cmp     dword [edi], '..  '
00009019 742B                   <1>     je      short loc_rmdir_permission_denied
                                <1> loc_rmdir_delete_short_name_skip_dot:
                                <1>     ;;;
                                <1>
                                <1>     ;mov    [DelFile_FCluster], eax
                                <1>
                                <1>     ;;mov   bx, [DirBuff_EntryCounter]
                                <1>     ;mov    bx, [FindFile_DirEntryNumber] ; 27/02/2016
                                <1>     ;mov    [DelFile_EntryCounter], bx
                                <1>
0000901B 29DB                   <1>     sub     ebx, ebx
                                <1>     ; 29/12/2017
0000901D 891D[927E0100]         <1>     mov     [FAT_ClusterCounter], ebx ; 0 ; Reset
                                <1>
00009023 8A3D[16800100]         <1>     mov     bh, [FindFile_Drv]
00009029 BE00010900             <1>     mov     esi, Logical_DOSDisks
0000902E 01DE                   <1>     add     esi, ebx
                                <1>
                                <1>     ; 19/12/2025
                                <1>     ;cmp    word [edi+DirEntry_NTRes], 01A1h
                                <1>     ;je     short loc_rmdir_check_fs_directory
                                <1>
                                <1>     ;cmp    byte [esi+LD_FATType], 1
                                <1>     ;jb     short loc_rmdir_get__last_cluster_0
                                <1>
                                <1>     ; 29/12/2017
00009030 83F802                 <1>     cmp     eax, 2
00009033 7306                   <1>     jnb     short loc_rmdir_get_last_cluster_1
                                <1>     ; eax < 2
                                <1> loc_rmdir_get_last_cluster_0:
                                <1>     ;mov    eax, ERR_INV_FORMAT ; invalid format!
00009035 B813000000             <1>     mov     eax, ERR_NOT_DIR ; not a valid directory!
                                <1>     ;stc
0000903A C3                     <1>     retn
                                <1>
                                <1> loc_rmdir_get_last_cluster_1:
```

```
0000903B 807E0303            <1>     cmp     byte [esi+LD_FATType], 3 ; FAT32
0000903F 750C                <1>     jne     short loc_rmdir_get_last_cluster_2
                             <1>
                             <1>     ; is it root directory ?
00009041 3B4632              <1>     cmp     eax, [esi+LD_BPB+BPB_RootClus]
00009044 7507                <1>     jne     short loc_rmdir_get_last_cluster_2
                             <1>
                             <1>     ; root directory can not be deleted !!
                             <1> loc_rmdir_permission_denied:
00009046 B80B000000          <1>     mov     eax, ERR_PERM_DENIED ; permission denied!
0000904B F9                  <1>     stc
0000904C C3                  <1>     retn
                             <1>
                             <1> loc_rmdir_get_last_cluster_2:
                             <1>     ; 29/12/2017
0000904D A3[D8800100]        <1>     mov     [DelFile_FCluster], eax
                             <1>
                             <1>     ;mov   dx, [DirBuff_EntryCounter]
00009052 668B15[90800100]    <1>     mov     dx, [FindFile_DirEntryNumber] ; 27/02/2016
00009059 668915[DC800100]    <1>     mov     [DelFile_EntryCounter], dx
                             <1>
00009060 8B15[A27E0100]      <1>     mov     edx, [DirBuff_Cluster]
00009066 8915[0C810100]      <1>     mov     [RmDir_ParentDirCluster], edx
                             <1>
0000906C 893D[08810100]      <1>     mov     [RmDir_DirEntryOffset], edi
                             <1>
                             <1>     ; 01/03/2016
                             <1>     ;mov   dword [FAT_ClusterCounter], 0 ; Reset
                             <1>
                             <1> loc_rmdir_get_last_cluster_3:
00009072 E878360000          <1>     call    get_last_cluster
                             <1>         ;jc loc_rmdir_cmd_failed
00009077 7211                <1>     jc      short loc_delete_sub_dir_retn ; 29/12/2017
                             <1>
00009079 3B05[D8800100]      <1>     cmp     eax, [DelFile_FCluster]
0000907F 750A                <1>     jne     short loc_rmdir_multi_dir_clusters
                             <1>
00009081 C605[07810100]00    <1>     mov     byte [RmDir_MultiClusters], 0
00009088 EB08                <1>     jmp     short pass_rmdir_multi_dir_clusters
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>
                             <1> loc_rmdir_check_fs_directory:
                             <1>     ; 29/12/2017
                             <1>     cmp     byte [esi+LD_FSType], 0A1h
                             <1>     jne     short loc_rmdir_permission_denied
                             <1>
                             <1> loc_rmdir_delete_fs_directory:
                             <1>     call    delete_fs_directory
                             <1>     ;jnc    loc_print_deleted_message
                             <1>     jnc     short loc_delete_sub_dir_retn ; 29/12/2017
                             <1>
                             <1>     ; EAX=0 -> Directory not empty !
                             <1>     ; EAX>0 -> Disk r/w error or another (misc) error
                             <1>
                             <1>     ;or     eax, eax
                             <1>     ;jz     loc_rmdir_directory_not_empty_2
                             <1>     ;;stc
                             <1>     ;;jmp   loc_file_rw_cmd_failed
                             <1>
                             <1> %endif
                             <1>
                             <1> loc_delete_sub_dir_retn:
0000908A C3                  <1>     retn
                             <1>
                             <1> loc_rmdir_multi_dir_clusters:
0000908B C605[07810100]01    <1>     mov     byte [RmDir_MultiClusters], 1
                             <1>
                             <1> pass_rmdir_multi_dir_clusters:
00009092 A3[10810100]        <1>     mov     [RmDir_DirLastCluster], eax
00009097 890D[14810100]      <1>     mov     [RmDir_PreviousCluster], ecx
                             <1>
                             <1> loc_rmdir_load_fat_sub_directory:
0000909D E8A3300000          <1>     call    load_FAT_sub_directory
                             <1>     ;jc     loc_rmdir_cmd_failed
000090A2 72E6                <1>     jc      short loc_delete_sub_dir_retn
                             <1>
                             <1> loc_rmdir_find_last_dir_entry:
000090A4 56                  <1>     push    esi
000090A5 BE[FA7F0100]        <1>     mov     esi, Dir_File_Name
000090AA C6062A              <1>     mov     byte [esi], '*'
000090AD C646082A            <1>     mov     byte [esi+8], '*'
000090B1 31DB                <1>     xor     ebx, ebx ; Entry offset  = 0
                             <1> loc_rmdir_find_last_dir_entry_next:
                             <1>     ;mov   ax, 0800h ; Except volume/long names
                             <1>     ;xor   cx, cx ; 0 = Find a valid file or dir name
                             <1>     ; 28/07/2022
000090B3 31C0                <1>     xor     eax, eax
000090B5 B408                <1>     mov     ah, 8
                             <1>     ; eax = 0800h
000090B7 31C9                <1>     xor     ecx, ecx ; 0
000090B9 E825160000          <1>     call    find_directory_entry
000090BE 7225                <1>     jc      short loc_rmdir_empty_dir_cluster
000090C0 83FB01              <1>     cmp     ebx, 1
000090C3 771B                <1>     ja      short loc_rmdir_directory_not_empty_1
                             <1> loc_rmdir_dot_entry_check:
000090C5 80FD2E              <1>     cmp     ch, '.' ; The first char of the dir entry
000090C8 7516                <1>     jne     short loc_rmdir_directory_not_empty_1
000090CA 08DB                <1>     or      bl, bl
000090CC 7506                <1>     jnz     short loc_rmdir_dotdot_entry_check
000090CE 807F0120            <1>     cmp     byte [edi+1], 20h
000090D2 EB06                <1>     jmp     short pass_rmdir_dot_entry_check
                             <1>
                             <1> loc_rmdir_dotdot_entry_check:
000090D4 66817F012E20        <1>     cmp     word [edi+1], '. '
                             <1> pass_rmdir_dot_entry_check:
000090DA 7504                <1>     jne     short loc_rmdir_directory_not_empty_1
000090DC FEC3                <1>     inc     bl
000090DE EBD3                <1>     jmp     short loc_rmdir_find_last_dir_entry_next
                             <1>
                             <1> loc_rmdir_directory_not_empty_1:
000090E0 58                  <1>     pop     eax ; pushed esi
000090E1 31C0                <1>     xor     eax, eax ; 0
                             <1> loc_rmdir_directory_not_empty_2:
                             <1> loc_delete_sub_dir_stc_retn:
000090E3 F9                  <1>     stc
000090E4 C3                  <1>     retn
                             <1>
                             <1> loc_rmdir_empty_dir_cluster:
000090E5 5E                  <1>     pop     esi
                             <1>
                             <1> loc_rmdir_set_prev_cluster_dir_last_cluster:
000090E6 803D[07810100]00    <1>     cmp     byte [RmDir_MultiClusters], 0
```

```
000090ED 7613                <1>     jna     short loc_rmdir_unlink_dir_last_cluster
                             <1>
000090EF A1[14810100]        <1>     mov     eax, [RmDir_PreviousCluster]
                             <1>     ;xor    ecx, ecx
000090F4 49                  <1>     dec     ecx ; FFFFFFFFh
000090F5 E876310000          <1>     call    update_cluster
000090FA 7306                <1>     jnc     short loc_rmdir_unlink_dir_last_cluster
                             <1>
                             <1>     ; 01/03/2016
                             <1>     ;cmp    eax, 1  ; eax = 0 -> end of cluster chain
                             <1>     ;cmc
                             <1>     ;jc     short loc_rmdir_cmd_failed
                             <1>     ;jmp    short loc_rmdir_save_fat_buffer
                             <1>     ; 29/12/2017
000090FC 21C0                <1>     and     eax, eax
000090FE 75E3                <1>     jnz     short loc_delete_sub_dir_stc_retn
00009100 EB12                <1>     jmp     short loc_rmdir_save_fat_buffer
                             <1>
                             <1> loc_rmdir_unlink_dir_last_cluster:
00009102 A1[10810100]        <1>     mov     eax, [RmDir_DirLastCluster]
00009107 31C9                <1>     xor     ecx, ecx ; 0
00009109 E862310000          <1>     call    update_cluster
0000910E 7327                <1>     jnc     short loc_rmdir_unlink_stc_retn_0Bh
                             <1>     ; Because of it is the last cluster
                             <1>     ; 'update_cluster' must return with eocc error
00009110 09C0                <1>     or      eax, eax
                             <1>     ;jz     short loc_rmdir_save_fat_buffer ; eocc
                             <1>     ;stc
                             <1>         ;jmp       short loc_rmdir_cmd_failed
                             <1>     ; 29/12/2017
00009112 75CF                <1>     jnz     short loc_delete_sub_dir_stc_retn
                             <1>
                             <1> loc_rmdir_save_fat_buffer:
00009114 803D[8A7E0100]02    <1>     cmp     byte [FAT_BuffValidData], 2
0000911B 7527                <1>     jne     short loc_rmdir_calculate_FAT_freespace
0000911D E8C2330000          <1>     call    save_fat_buffer
                             <1>     ;jc     short loc_rmdir_cmd_failed
                             <1>     ; 29/12/2017
00009122 7219                <1>     jc      short loc_rmdir_unlink_error_retn
                             <1>
                             <1>     ; 01/03/2016
00009124 803D[07810100]00    <1>     cmp     byte [RmDir_MultiClusters], 0
0000912B 7617                <1>     jna     short loc_rmdir_calculate_FAT_freespace
                             <1>
0000912D A1[D8800100]        <1>     mov     eax, [DelFile_FCluster]
00009132 E93BFFFFFF          <1>         jmp     loc_rmdir_get_last_cluster_3
                             <1>
                             <1> loc_rmdir_unlink_stc_retn_0Bh:
                             <1>     ; 15/10/2016 (0Bh -> 28)
00009137 B81C000000          <1>     mov     eax, ERR_INV_FORMAT ; 28 = Invalid format
                             <1> loc_rmdir_unlink_stc_retn:
0000913C F9                  <1>     stc
                             <1> loc_rmdir_unlink_error_retn:
0000913D C3                  <1>     retn
                             <1>
                             <1> loc_rmdir_delete_short_name_invalid_data:
                             <1>     ;mov    eax, 29 ; Invalid data (15/10/2016)
                             <1>     ; 28/07/2022
0000913E 29C0                <1>     sub     eax, eax
00009140 B01D                <1>     mov     al, 29
                             <1>     ;stc
                             <1>         ;jmp       loc_rmdir_cmd_failed
                             <1>     ; 29/12/2017
00009142 EBF8                <1>     jmp     short loc_rmdir_unlink_stc_retn
                             <1>
                             <1> loc_rmdir_calculate_FAT_freespace:
                             <1>     ;mov    eax, [FAT_ClusterCounter]
                             <1>     ; 29/12/2017
00009144 29C0                <1>     sub     eax, eax ; 0
00009146 8705[927E0100]      <1>     xchg    eax, [FAT_ClusterCounter]
                             <1>     ;
0000914C 66BB01FF            <1>     mov     bx, 0FF01h
                             <1>     ; BL = 1 -> Add EAX to free space count
                             <1>     ; BH = FFh ->
                             <1>     ; ESI = Logical DOS Drive Description Table address
00009150 E81C340000          <1>     call    calculate_fat_freespace
                             <1>
00009155 21C9                <1>     and     ecx, ecx ; ecx = 0 -> valid free sector count
00009157 7409                <1>     jz      short loc_rmdir_delete_short_name_continue
                             <1>
                             <1> loc_rmdir_recalculate_FAT_freespace:
00009159 66BB00FF            <1>         mov     bx, 0FF00h ; BL = 0 -> Recalculate free space
0000915D E80F340000          <1>     call    calculate_fat_freespace
                             <1>
                             <1> loc_rmdir_delete_short_name_continue:
00009162 A1[0C810100]        <1>     mov     eax, [RmDir_ParentDirCluster]
00009167 83F802              <1>     cmp     eax, 2
0000916A 7309                <1>     jnb     short loc_rmdir_del_short_name_load_sub_dir
0000916C E8562F0000          <1>     call    load_FAT_root_directory
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 29/12/2017
00009171 72CA                <1>     jc      short loc_rmdir_unlink_error_retn
00009173 EB07                <1>     jmp     short loc_rmdir_del_short_name_ld_chk_fclust
                             <1>
                             <1> loc_rmdir_del_short_name_load_sub_dir:
00009175 E8CB2F0000          <1>     call    load_FAT_sub_directory
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 29/12/2017
0000917A 72C1                <1>     jc      short loc_rmdir_unlink_error_retn
                             <1>
                             <1> loc_rmdir_del_short_name_ld_chk_fclust:
0000917C 0FB73D[08810100]    <1>     movzx   edi, word [RmDir_DirEntryOffset]
00009183 81C700000800        <1>     add     edi, Directory_Buffer
                             <1>
00009189 668B4714            <1>     mov     ax, [edi+20] ; First Cluster High Word
0000918D C1E010              <1>     shl     eax, 16
00009190 668B471A            <1>     mov     ax, [edi+26] ; First Cluster Low Word
                             <1>         ; Not necessary...
00009194 3B05[D8800100]      <1>     cmp     eax, [DelFile_FCluster]
0000919A 75A2                <1>     jne     short loc_rmdir_delete_short_name_invalid_data
                             <1>     ;
0000919C C607E5              <1>     mov     byte [edi], 0E5h ; 'Deleted' sign
                             <1>     ; 27/02/2016
                             <1>     ; TRDOS v1 has a bug here! it does not set
                             <1>     ; 'DirBuff_ValidData' to 2; as result of this bug,
                             <1>     ; 'save_directory_buffer' would not save the change !
0000919F C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2 ; change sign
                             <1>     ;
000091A6 E8E61B0000          <1>     call    save_directory_buffer
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 29/12/2017
000091AB 7290                <1>     jc      short loc_rmdir_unlink_error_retn
                             <1>
                             <1> loc_rmdir_del_long_name:
```

```
000091AD 0FB615[DE800100]    <1>     movzx   edx, byte [DelFile_LNEL]
000091B4 08D2                <1>     or      dl, dl
000091B6 7410                <1>     jz      short loc_rmdir_update_parent_dir_lmdt
                             <1>
000091B8 0FB705[DC800100]    <1>     movzx   eax, word [DelFile_EntryCounter]
000091BF 29D0                <1>     sub     eax, edx
                             <1>     ; 29/12/2017
000091C1 7205                <1>     jc      short loc_rmdir_update_parent_dir_lmdt
                             <1>
                             <1>     ; EAX = Directory Entry Number of the long name last entry
000091C3 E8091D0000          <1>     call    delete_longname
                             <1>
                             <1> loc_rmdir_update_parent_dir_lmdt:
000091C8 E84D1C0000          <1>     call    update_parent_dir_lmdt
                             <1>     ;jc     short loc_file_rw_cmd_failed
                             <1>     ; 29/12/2017
                             <1>     ;jc     short loc_rmdir_unlink_error_retn
                             <1>
                             <1> loc_delete_sub_directory_ok:
                             <1>     ; 29/12/2017
000091CD 31C0                <1>     xor     eax, eax ;  0 ;  cf = 0
000091CF C3                  <1>     retn
                             <1>
                             <1> delete_file:
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 29/02/2016
                             <1>     ; 28/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 09/08/2010 (CMD_INTR.ASM, 'cmp_cmd_del')
                             <1>     ; 28/02/2010
                             <1>
                             <1> get_delfile_fchar:
                             <1>     ; esi = file name
000091D0 803E20              <1>     cmp     byte [esi], 20h
000091D3 7701                <1>         ja  short loc_delfile_parse_path_name
                             <1>
                             <1> loc_delfile_nofilename_retn:
000091D5 C3                  <1>     retn
                             <1>
                             <1> loc_delfile_parse_path_name:
000091D6 BF[16800100]        <1>     mov     edi, FindFile_Drv
000091DB E89E170000          <1>     call    parse_path_name
                             <1>     ;jc     loc_cmd_failed
                             <1>     ; 28/07/2022
000091E0 7305                <1>     jnc     short loc_delfile_check_filename_exists
                             <1> loc_delfile_failed:
000091E2 E96DF2FFFF          <1>     jmp     loc_cmd_failed
                             <1>
                             <1> loc_delfile_check_filename_exists:
000091E7 BE[58800100]        <1>     mov     esi, FindFile_Name
000091EC 803E20              <1>     cmp     byte [esi], 20h
                             <1>     ;jna    loc_cmd_failed
                             <1>     ; 28/07/2022
000091EF 76F1                <1>     jna     short loc_delfile_failed
000091F1 8935[D4800100]      <1>     mov     [DelFile_FNPointer], esi
                             <1>
                             <1> loc_delfile_drv:
000091F7 8A15[16800100]      <1>     mov     dl, [FindFile_Drv]
000091FD 8A35[76770100]      <1>     mov     dh, [Current_Drv]
00009203 8835[D37E0100]      <1>     mov     [RUN_CDRV], dh
00009209 38F2                <1>     cmp     dl, dh
0000920B 7407                <1>     je      short loc_delfile_change_directory
                             <1>
0000920D E8DEE4FFFF          <1>     call    change_current_drive
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 28/07/2022
                             <1>     ;jnc    short loc_delfile_change_directory
                             <1>     ;jmp    loc_file_rw_cmd_failed
00009212 721D                <1>     jc      short loc_delfile_chdrv_failed
                             <1>
                             <1> loc_delfile_change_directory:
00009214 803D[17800100]20    <1>     cmp     byte [FindFile_Directory], 20h
0000921B 7619                <1>     jna     short loc_delfile_find
                             <1>
0000921D FE05[8E2E0100]      <1>     inc     byte [Restore_CDIR]
00009223 BE[17800100]        <1>     mov     esi, FindFile_Directory
00009228 30E4                <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000922A E8BC110000          <1>     call    change_current_directory
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 28/07/2022
0000922F 7305                <1>     jnc     short loc_delfile_chdir_ok
                             <1> loc_delfile_chdrv_failed:
                             <1> loc_delfile_fff_failed:
00009231 E912FBFFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> loc_delfile_chdir_ok: ; 28/07/2022
                             <1>
                             <1> ;loc_delfile_change_prompt_dir_string:
                             <1>     ;call   change_prompt_dir_string
                             <1>
                             <1> loc_delfile_find:
                             <1>     ;mov    esi, FindFile_Name
00009236 8B35[D4800100]      <1>     mov     esi, [DelFile_FNPointer]
0000923C 66B80018            <1>     mov     ax, 1800h ; Except volume label and dirs
00009240 E835F7FFFF          <1>     call    find_first_file
                             <1>     ;jc     loc_file_rw_cmd_failed
                             <1>     ; 28/07/2022
00009245 72EA                <1>     jc      short loc_delfile_fff_failed
                             <1>
                             <1> loc_delfile_ambgfn_check:
00009247 6621D2              <1>     and     dx, dx ; Ambiguous filename chars used sign (DX>0)
0000924A 740A                <1>     jz      short loc_delfile_found
                             <1>
                             <1> loc_file_not_found:
                             <1>     ;mov    eax, 2 ; File not found sign
                             <1>     ; 28/07/2022
0000924C 31C0                <1>     xor     eax, eax
0000924E B002                <1>     mov     al, 2
00009250 F9                  <1>     stc
00009251 E9F2FAFFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> loc_delfile_found:
00009256 80E307              <1>     and     bl, 07h ; Attributes
                             <1>     ;jnz    loc_permission_denied
                             <1>     ; 28/07/2022
00009259 7405                <1>     jz      short loc_delfile_attrb_ok
0000925B E9F2FAFFFF          <1>     jmp     loc_permission_denied
                             <1>
                             <1> loc_delfile_attrb_ok: ; 28/07/2022
                             <1>
                             <1> ;loc_delfile_found_save_lnel:
                             <1> ;   mov     [DelFile_LNEL], bh ; Long name entry length (if > 0)
                             <1>
                             <1> loc_delfile_ask_for_delete:
00009260 57                  <1>     push    edi ; * (29/02/2016)
```

```
                                <1>
00009261 BE[66330100]           <1>     mov     esi, Msg_DoYouWantDelete
00009266 E84FDBFFFF             <1>     call    print_msg
0000926B 8B35[D4800100]         <1>     mov     esi, [DelFile_FNPointer]
00009271 E844DBFFFF             <1>     call    print_msg
00009276 BE[1A330100]           <1>     mov     esi, Msg_YesNo
0000927B E83ADBFFFF             <1>     call    print_msg
                                <1>
                                <1> loc_delfile_ask_again:
00009280 30E4                   <1>     xor     ah, ah
00009282 E88B7CFFFF             <1>     call    int16h
00009287 3C1B                   <1>     cmp     al, 1Bh
                                <1>     ;je     short loc_do_not_delete_file
00009289 744C                   <1>     je      short loc_delfile_y_n_escape ; 06/03/2016
0000928B 24DF                   <1>     and     al, 0DFh
0000928D A2[24330100]           <1>     mov     [Y_N_nextline], al
00009292 3C59                   <1>     cmp     al, 'Y'
00009294 7404                   <1>     je      short loc_yes_delete_file
00009296 3C4E                   <1>     cmp     al, 'N'
00009298 75E6                   <1>     jne     short loc_delfile_ask_again
                                <1>
                                <1> loc_do_not_delete_file:
                                <1> loc_yes_delete_file:
0000929A E8DCFBFFFF             <1>     call    y_n_answer ; 29/12/2017
0000929F 5F                     <1>     pop     edi ; * (29/02/2016)
                                <1>     ;cmp    al, 'Y' ; 'yes'
                                <1>     ;cmc
                                <1>         ;jnc       loc_file_rw_restore_retn
000092A0 3C4E                   <1>     cmp     al, 'N' ; 'no'
                                <1>         ;je loc_file_rw_restore_retn
                                <1>     ; 28/07/2022
000092A2 7505                   <1>     jne     short loc_delete_file
000092A4 E99FFAFFFF             <1>     jmp     loc_file_rw_restore_retn
                                <1>
                                <1> loc_delete_file:
000092A9 8A3D[16800100]         <1>     mov     bh, [FindFile_Drv]
                                <1>     ;mov    bl, [DelFile_LNEL]
000092AF 8A1D[65800100]         <1>     mov     bl, [FindFile_LongNameEntryLength]
                                <1>     ;mov    cx, [DirBuff_EntryCounter]
000092B5 668B0D[90800100]       <1>     mov     cx, [FindFile_DirEntryNumber]
                                <1>     ; (*) EDI = Directory buffer entry offset/address
000092BC E8E81D0000             <1>     call    remove_file ; (FILE.ASM, 'proc_delete_file')
                                <1>     ;jnc    loc_print_deleted_message
                                <1>     ; 28/07/2022
000092C1 7205                   <1>     jc      short loc_delete_file_err1
000092C3 E976FAFFFF             <1>     jmp     loc_print_deleted_message
                                <1>
                                <1> loc_delete_file_err1:
                                <1>     ;cmp    al, 05h
000092C8 3C0B                   <1>     cmp     al, ERR_PERM_DENIED  ; 29/12/2017 (5 -> 11)
                                <1>     ;je     loc_permission_denied
                                <1>     ; 28/07/2022
000092CA 7505                   <1>     jne     short loc_delete_file_err2
000092CC E981FAFFFF             <1>     jmp     loc_permission_denied
                                <1> loc_delete_file_err2:
000092D1 F9                     <1>     stc
000092D2 E971FAFFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_delfile_y_n_escape:
000092D7 B04E                   <1>     mov     al, 'N' ; 'no'
000092D9 EBBF                   <1>     jmp     short loc_do_not_delete_file
                                <1>
                                <1> set_file_attributes:
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 26/09/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 06/03/2016
                                <1>     ; 04/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 10/07/2010 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_attrib')
                                <1>     ; 23/05/2010
                                <1>     ; 17/12/2000 (P2000.ASM)
                                <1>
                                <1>     ; esi = file or directory name
                                <1>     ;xor    ax, ax
                                <1>     ; 28/07/2022
000092DB 31C0                   <1>     xor     eax, eax
000092DD 66A3[B7330100]         <1>     mov     [Attr_Chars], ax
000092E3 A2[30810100]           <1>     mov     [Attributes], al
                                <1>
                                <1> get_attrib_fchar:
                                <1>     ; esi = file name
000092E8 8A06                   <1>     mov     al, [esi]
000092EA 3C20                   <1>     cmp     al, 20h
000092EC 7621                   <1>     jna     short loc_attr_file_nofilename_retn
                                <1>
                                <1> loc_scan_attrib_params:
000092EE 3C2D                   <1>     cmp     al, '-'
                                <1>     ;ja     loc_attr_file_parse_path_name
                                <1>     ;je     short loc_attr_space
                                <1>     ; 28/07/2022
000092F0 7207                   <1>     jb      short loc_sfa_1
000092F2 740E                   <1>     je      short loc_attr_space
000092F4 E911010000             <1>     jmp     loc_attr_file_parse_path_name
                                <1>
                                <1> loc_sfa_1:
                                <1>     ; 28/07/2022
000092F9 3C2B                   <1>     cmp     al, '+'
                                <1>     ;jne    loc_cmd_failed
000092FB 7405                   <1>     je      short loc_attr_space
                                <1> loc_sfa_2:
000092FD E952F1FFFF             <1>     jmp     loc_cmd_failed
                                <1> loc_attr_space:
00009302 8A6601                 <1>     mov     ah, [esi+1]
00009305 80FC20                 <1>     cmp     ah, 20h
00009308 7706                   <1>     ja      short pass_attr_space
                                <1>     ;jb     loc_cmd_failed
                                <1>     ; 28/07/2022
0000930A 72F1                   <1>     jb      short loc_sfa_2
0000930C 46                     <1>     inc     esi
0000930D EBF3                   <1>     jmp     short loc_attr_space
                                <1>
                                <1> loc_attr_file_nofilename_retn:
0000930F C3                     <1>     retn
                                <1>
                                <1> pass_attr_space:
00009310 80E4DF                 <1>     and     ah, 0DFh
00009313 80FC53                 <1>     cmp     ah, 'S'
                                <1>     ;ja     loc_cmd_failed
                                <1>     ; 28/07/2022
00009316 77E5                   <1>     ja      short loc_sfa_2
00009318 7204                   <1>     jb      short pass_attr_system
0000931A B404                   <1>     mov     ah, 04h ; System
0000931C EB1D                   <1>     jmp     short pass_attr_archive
                                <1>
```

```
                                <1> pass_attr_system:
0000931E 80FC48                 <1>     cmp     ah, 'H'
00009321 7706                   <1>     ja      short pass_attr_hidden
00009323 720F                   <1>     jb      short pass_attr_read_only
00009325 B402                   <1>     mov     ah, 02h ; Hidden
00009327 EB12                   <1>     jmp     short pass_attr_archive
                                <1>
                                <1> pass_attr_hidden:
00009329 80FC52                 <1>     cmp     ah, 'R'
                                <1>     ;ja     loc_cmd_failed
                                <1>     ; 28/07/2022
0000932C 77CF                   <1>     ja      short loc_sfa_2
0000932E 7204                   <1>     jb      short pass_attr_read_only ; Read only
00009330 B401                   <1>     mov     ah, 01h
00009332 EB07                   <1>     jmp     short pass_attr_archive
                                <1>
                                <1> pass_attr_read_only:
00009334 80FC41                 <1>     cmp     ah, 'A'
00009337 753B                   <1>     jne     short loc_chk_attr_enter
00009339 B420                   <1>     mov     ah, 20h ; Archive
                                <1>
                                <1> pass_attr_archive:
0000933B 3C2D                   <1>     cmp     al, '-'
0000933D 7508                   <1>     jne     short pass_reducing_attributes
0000933F 0825[B7330100]         <1>     or      [Attr_Chars], ah
00009345 EB06                   <1>     jmp     short loc_change_attributes_inc
                                <1>
                                <1> pass_reducing_attributes:
00009347 0825[B8330100]         <1>     or      [Attr_Chars+1], ah
                                <1>
                                <1> loc_change_attributes_inc:
0000934D 46                     <1>     inc     esi
0000934E 8A6601                 <1>     mov     ah, [esi+1]
00009351 80FC20                 <1>     cmp     ah, 20h
00009354 7228                   <1>     jb      short pass_change_attr
00009356 74F5                   <1>     je      short loc_change_attributes_inc
00009358 80FC2D                 <1>     cmp     ah, '-'
0000935B 770D                   <1>     ja      short loc_chk_next_attr_char1
0000935D 7405                   <1>     je      short loc_chk_next_attr_char0
0000935F 80FC2B                 <1>     cmp     ah, '+'
00009362 7506                   <1>     jne     short loc_chk_next_attr_char1
                                <1>
                                <1> loc_chk_next_attr_char0:
00009364 46                     <1>     inc     esi
00009365 668B06                 <1>     mov     ax, [esi]
00009368 EBA6                   <1>     jmp     short pass_attr_space
                                <1>
                                <1> loc_chk_next_attr_char1:
0000936A 803E2D                 <1>     cmp     byte [esi], '-'
0000936D 77A1                   <1>     ja      short pass_attr_space
0000936F E989000000             <1>         jmp     loc_attr_file_check_fname_fchar
                                <1>
                                <1> loc_chk_attr_enter:
00009374 80FC0D                 <1>     cmp     ah, 0Dh
                                <1>     ;jne    loc_cmd_failed
                                <1>     ; 28/07/202
00009377 7405                   <1>     je      short pass_change_attr
00009379 E9D6F0FFFF             <1>     jmp     loc_cmd_failed
                                <1>
                                <1> pass_change_attr:
0000937E A0[B7330100]           <1>     mov     al, [Attr_Chars]
00009383 F6D0                   <1>     not     al
00009385 2005[30810100]         <1>     and     [Attributes], al
0000938B A0[B8330100]           <1>     mov     al, [Attr_Chars+1]
00009390 0805[30810100]         <1>     or      [Attributes], al
                                <1>
                                <1> loc_show_attributes:
00009396 BE[17390100]           <1>     mov     esi, nextline
0000939B E81ADAFFFF             <1>     call    print_msg
                                <1>
                                <1> loc_show_attributes_no_nextline:
000093A0 C705[B7330100]4E4F-    <1>     mov     dword [Attr_Chars], 'NORM'
000093A8 524D                   <1>
000093AA 66C705[BB330100]41-    <1>     mov     word [Attr_Chars+4], 'AL'
000093B2 4C                     <1>
000093B3 BE[B7330100]           <1>     mov     esi, Attr_Chars
000093B8 A0[30810100]           <1>     mov     al, [Attributes]
000093BD A804                   <1>     test    al, 04h
000093BF 7406                   <1>     jz      short pass_put_attr_s
000093C1 66C7065300             <1>     mov     word [esi], 0053h     ; S
000093C6 46                     <1>     inc     esi
                                <1>
                                <1> pass_put_attr_s:
000093C7 A802                   <1>     test    al, 02h
000093C9 7406                   <1>     jz      short pass_put_attr_h
000093CB 66C7064800             <1>     mov     word [esi], 0048h     ; H
000093D0 46                     <1>     inc     esi
                                <1>
                                <1> pass_put_attr_h:
000093D1 A801                   <1>     test    al, 01h
000093D3 7406                   <1>     jz      short pass_put_attr_r
000093D5 66C7065200             <1>     mov     word [esi], 0052h     ; R
000093DA 46                     <1>     inc     esi
                                <1>
                                <1> pass_put_attr_r:
000093DB 3C20                   <1>     cmp     al, 20h
000093DD 7205                   <1>     jb      short pass_put_attr_a
000093DF 66C7064100             <1>     mov     word [esi], 0041h     ; A
                                <1>
                                <1> pass_put_attr_a:
000093E4 BE[AA330100]           <1>     mov     esi, Str_Attributes
000093E9 E8CCD9FFFF             <1>     call    print_msg
000093EE BE[17390100]           <1>     mov     esi, nextline
000093F3 E8C2D9FFFF             <1>     call    print_msg
000093F8 E94BF9FFFF             <1>     jmp     loc_file_rw_restore_retn
                                <1>
                                <1> loc_attr_file_check_fname_fchar:
000093FD 46                     <1>     inc     esi
000093FE 803E20                 <1>     cmp     byte [esi], 20h
00009401 74FA                   <1>     je      short loc_attr_file_check_fname_fchar
                                <1>         ;jb pass_change_attr
                                <1>     ; 28/07/2022
00009403 7705                   <1>     ja      short loc_attr_file_parse_path_name
00009405 E974FFFFFF             <1>     jmp     pass_change_attr
                                <1>
                                <1> loc_attr_file_parse_path_name:
0000940A BF[16800100]           <1>     mov     edi, FindFile_Drv
0000940F E86A150000             <1>     call    parse_path_name
                                <1>     ;jc     loc_cmd_failed
                                <1>     ; 28/07/2022
00009414 7305                   <1>     jnc     short loc_attr_file_check_filename_exists
                                <1> loc_sfa_3:
00009416 E939F0FFFF             <1>     jmp     loc_cmd_failed
                                <1>
```

```
                                <1> loc_attr_file_check_filename_exists:
0000941B BE[58800100]           <1>     mov     esi, FindFile_Name
00009420 803E20                 <1>     cmp     byte [esi], 20h
                                <1>     ;jna    loc_cmd_failed
                                <1>     ; 28/07/2022
00009423 76F1                   <1>     jna     short loc_sfa_3
00009425 8935[D4800100]         <1>     mov     [DelFile_FNPointer], esi
                                <1>
                                <1> loc_attr_file_drv:
0000942B 8A35[76770100]         <1>     mov     dh, [Current_Drv]
00009431 8835[D37E0100]         <1>     mov     [RUN_CDRV], dh
                                <1>
00009437 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
0000943D 38F2                   <1>     cmp     dl, dh
0000943F 7407                   <1>     je      short loc_attr_file_change_directory
                                <1>
00009441 E8AAE2FFFF             <1>     call    change_current_drive
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
00009446 722E                   <1>     jc      short loc_sfa_4
                                <1>
                                <1> loc_attr_file_change_directory:
00009448 803D[17800100]20       <1>         cmp     byte [FindFile_Directory], 20h
0000944F 7614                   <1>     jna     short loc_attr_file_find
                                <1>
00009451 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
                                <1>
00009457 BE[17800100]           <1>     mov     esi, FindFile_Directory
0000945C 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000945E E8880F0000             <1>     call    change_current_directory
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
00009463 7211                   <1>     jc      short loc_sfa_4
                                <1>
                                <1> ;loc_attr_file_change_prompt_dir_string:
                                <1>     ;call   change_prompt_dir_string
                                <1>
                                <1> loc_attr_file_find:
                                <1>     ;mov    esi, FindFile_Name
00009465 8B35[D4800100]         <1>     mov     esi, [DelFile_FNPointer]
0000946B 66B80008               <1>     mov     ax, 0800h ; Except volume labels
0000946F E806F5FFFF             <1>     call    find_first_file
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
00009474 7305                   <1>     jnc     short loc_attr_file_ambgfn_check
                                <1> loc_sfa_4:
00009476 E9CDF8FFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_attr_file_ambgfn_check:
0000947B 6609D2                 <1>     or      dx, dx ; Ambiguous filename chars used sign (DX>0)
                                <1>     ;       (Note: It was BX in TRDOS v1)
                                <1>     ;;jz    short loc_attr_file_found
                                <1>     ;jnz    loc_file_not_found ; 06/03/2016
                                <1>     ; 28/07/2022
0000947E 7405                   <1>     jz      short loc_attr_file_found
00009480 E9C7FDFFFF             <1>     jmp     loc_file_not_found
                                <1>
                                <1>     ;mov    eax, 2 ; File not found sign
                                <1>     ;stc
                                <1>     ;jmp    loc_file_rw_cmd_failed
                                <1>
                                <1> loc_attr_file_found:
                                <1>     ; EDI = Directory buffer entry offset/address
                                <1>     ; BL = File (or Directory) Attributes
                                <1>     ;       (Note: It was 'CL' in TRDOS v1)
                                <1>     ; mov   bl, [EDI+0Bh]
                                <1>
00009485 66833D[B7330100]00     <1>     cmp     word [Attr_Chars], 0
0000948D 770B                   <1>     ja      short loc_attr_file_change_attributes
0000948F 881D[30810100]         <1>     mov     [Attributes], bl
00009495 E9FCFEFFFF             <1>     jmp     loc_show_attributes
                                <1>
                                <1> loc_attr_file_change_attributes:
0000949A A0[B7330100]           <1>     mov     al, [Attr_Chars]
0000949F F6D0                   <1>     not     al
000094A1 20C3                   <1>     and     bl, al
000094A3 A0[B8330100]           <1>     mov     al, [Attr_Chars+1]
000094A8 08C3                   <1>     or      bl, al
                                <1>
                                <1>     ; 19/12/2025
                                <1>     ;cmp    word [edi+DirEntry_NTRes], 01A1h ; Singlix FS
                                <1>     ;je     short loc_attr_file_fs_check
                                <1>
000094AA 881D[30810100]         <1>     mov     [Attributes], bl
000094B0 885F0B                 <1>     mov     [edi+0Bh], bl  ; Attributes (New!)
                                <1>
                                <1>     ; 04/03/2016
                                <1>     ; TRDOS v1 has a bug here! it does not set
                                <1>     ; 'DirBuff_ValidData' to 2; as result of this bug,
                                <1>     ; 'save_directory_buffer' would not save the new attributes !
                                <1>
000094B3 C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
                                <1>
000094BA E8D2180000             <1>     call    save_directory_buffer
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ;jmp    short loc_print_attr_changed_message
                                <1>     ; 28/07/2022
000094BF 7305                   <1>     jnc     short loc_print_attr_changed_message
                                <1> loc_sfa_5:
000094C1 E982F8FFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1> loc_attr_file_fs_check:
                                <1>     sub     eax, eax
                                <1>         mov     ah, [DirBuff_DRV]
                                <1>     ; 26/09/2024 (BugFix)
                                <1>     sub     ah, 'A'
                                <1>     mov     esi, Logical_DOSDisks
                                <1>         add     esi, eax
                                <1>         cmp     byte [esi+LD_FSType], 0A1h
                                <1>     jnc     short loc_attr_file_change_fs_file_attributes
                                <1>     ; 29/12/2017 (0Dh -> 29)
                                <1>     mov     ax, 29 ; Invalid Data
                                <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> loc_attr_file_change_fs_file_attributes:
                                <1>     ; BL = New MS-DOS File Attributes
                                <1>     mov     al, bl ; File/Directory Attributes
                                <1>     xor     ah, ah ; Attributes in MS-DOS format sign
                                <1>     call    change_fs_file_attributes
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
                                <1>     jc      short loc_sfa_5
```

```
                                <1>
                                <1>     mov     [Attributes], bl
                                <1> %endif
                                <1>
                                <1> loc_print_attr_changed_message:
000094C6 BE[A5330100]           <1>     mov     esi, Msg_New
000094CB E8EAD8FFFF             <1>     call    print_msg
000094D0 E9CBFEFFFF             <1>     jmp     loc_show_attributes_no_nextline
                                <1>
                                <1> rename_file:
                                <1>     ; 25/07/2025 (BugFix)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/11/2017
                                <1>     ; 06/11/2016
                                <1>     ; 05/11/2016
                                <1>     ; 16/10/2016
                                <1>     ; 08/03/2016
                                <1>     ; 06/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 20/11/2010 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_rename')
                                <1>     ; 16/11/2010
                                <1>
                                <1> get_rename_source_fchar:
                                <1>     ; esi = file name
000094D5 803E20                 <1>     cmp     byte [esi], 20h
000094D8 7613                   <1>         jna short loc_rename_nofilename_retn
                                <1>
000094DA 8935[58810100]         <1>     mov     [SourceFilePath], esi
                                <1>
                                <1> rename_scan_source_file:
000094E0 46                     <1>     inc     esi
000094E1 803E20                 <1>     cmp     byte [esi], 20h
000094E4 7408                   <1>     je      short rename_scan_destination_file_1
                                <1>     ;;jb    short loc_rename_nofilename_retn
                                <1>     ;jb     loc_cmd_failed
                                <1>     ;jmp    short rename_scan_source_file
                                <1>     ; 28/07/2022
000094E6 77F8                   <1>     ja      short rename_scan_source_file
                                <1> loc_rename_failed:
000094E8 E967EFFFFF             <1>     jmp     loc_cmd_failed
                                <1>
                                <1> loc_rename_nofilename_retn: ; 08/03/2016
000094ED C3                     <1>     retn
                                <1>
                                <1> rename_scan_destination_file_1:
000094EE C60600                 <1>     mov     byte [esi], 0
                                <1>
                                <1> rename_scan_destination_file_2:
000094F1 46                     <1>     inc     esi
000094F2 803E20                 <1>     cmp     byte [esi], 20h
000094F5 74FA                   <1>     je      short rename_scan_destination_file_2
                                <1>     ;;jb    short loc_rename_nofilename_retn
                                <1>     ;jb     loc_cmd_failed
                                <1>     ; 28/07/2022
000094F7 72EF                   <1>     jb      short loc_rename_failed
                                <1>
000094F9 8935[5C810100]         <1>     mov     [DestinationFilePath], esi
                                <1>
                                <1> rename_scan_destination_file_3:
000094FF 46                     <1>     inc     esi
00009500 803E20                 <1>     cmp     byte [esi], 20h
00009503 77FA                   <1>     ja      short rename_scan_destination_file_3
                                <1>
00009505 C60600                 <1>     mov     byte [esi], 0
                                <1>
                                <1> loc_rename_save_current_drive:
00009508 8A35[76770100]         <1>     mov     dh, [Current_Drv]
0000950E 8835[D37E0100]         <1>     mov     byte [RUN_CDRV], dh
                                <1>
                                <1> loc_rename_sf_parse_path_name:
00009514 8B35[58810100]         <1>     mov     esi, [SourceFilePath]
0000951A BF[16800100]           <1>     mov     edi, FindFile_Drv
0000951F E85A140000             <1>     call    parse_path_name
                                <1>     ;jc     loc_cmd_failed
                                <1>     ; 28/07/2022
00009524 72C2                   <1>     jc      short loc_rename_failed
                                <1>
                                <1> loc_rename_sf_check_filename_exists:
00009526 BE[58800100]           <1>     mov     esi, FindFile_Name
0000952B 803E20                 <1>     cmp     byte [esi], 20h
                                <1>     ;jna    loc_cmd_failed
                                <1>     ; 28/07/2022
0000952E 76B8                   <1>     jna     short loc_rename_failed
                                <1>
                                <1>     ;mov    [DelFile_FNPointer], esi
                                <1>
                                <1> loc_rename_sf_drv:
                                <1>     ;mov    dh, [Current_Drv]
                                <1>     ;mov    [RUN_CDRV], dh
                                <1>
00009530 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
00009536 38F2                   <1>     cmp     dl, dh ; dh = [Current_Drv]
00009538 7407                   <1>     je      short rename_sf_change_directory
                                <1>
0000953A E8B1E1FFFF             <1>     call    change_current_drive
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
0000953F 722D                   <1>     jc      short loc_rename_fff_failed
                                <1>
                                <1> rename_sf_change_directory:
00009541 803D[17800100]20       <1>     cmp     byte [FindFile_Directory], 20h
00009548 7614                   <1>     jna     short rename_sf_find
                                <1>
0000954A FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
00009550 BE[17800100]           <1>     mov     esi, FindFile_Directory
00009555 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
00009557 E88F0E0000             <1>     call    change_current_directory
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
0000955C 7210                   <1>     jc      short loc_rename_fff_failed
                                <1>
                                <1> ;rename_sf_change_prompt_dir_string:
                                <1>     ;call   change_prompt_dir_string
                                <1>
                                <1> rename_sf_find:
                                <1>     ;mov    esi, [DelFile_FNPointer]
0000955E BE[58800100]           <1>     mov     esi, FindFile_Name
                                <1>
00009563 66B80008               <1>     mov     ax, 0800h ; Except volume labels
00009567 E80EF4FFFF             <1>     call    find_first_file
                                <1>     ;jc     loc_file_rw_cmd_failed
                                <1>     ; 28/07/2022
0000956C 7305                   <1>     jnc     short loc_rename_sf_ambgfn_check
                                <1>
                                <1> loc_rename_fff_failed:
```

```
0000956E E9D5F7FFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> loc_rename_sf_ambgfn_check:
00009573 6621D2              <1>     and     dx, dx ; Ambiguous filename chars used sign (DX>0)
                             <1>     ;       (Note: It was BX in TRDOS v1)
                             <1>     ;;jz    short loc_rename_sf_found
                             <1>     ;jnz    loc_file_not_found
                             <1>     ; 28/07/2022
00009576 7405                <1>     jz      short loc_rename_sf_found
00009578 E9CFFCFFFF          <1>     jmp     loc_file_not_found
                             <1>
                             <1>     ;mov    eax, 2 ; File not found sign
                             <1>     ;stc
                             <1>     ;jmp    loc_file_rw_cmd_failed
                             <1>
                             <1> loc_rename_sf_found:
                             <1>     ; EDI = Directory buffer entry offset/address
                             <1>     ; BL = File (or Directory) Attributes
                             <1>     ;       (Note: It was 'CL' in TRDOS v1)
                             <1>     ; mov   bl, [EDI+0Bh]
                             <1>
0000957D F6C307              <1>     test    bl, 07h ; Attributes, S-H-R
                             <1>     ;jnz    loc_permission_denied
                             <1>     ; 28/07/2022
00009580 7405                <1>     jz      short loc_rename_attrb_ok
00009582 E9CBF7FFFF          <1>     jmp     loc_permission_denied
                             <1>
                             <1> loc_rename_attrb_ok:
00009587 BE[16800100]        <1>         mov     esi, FindFile_Drv
0000958C BF[60810100]        <1>         mov     edi, SourceFile_Drv
00009591 B920000000          <1>     mov     ecx, 32
00009596 F3A5                <1>     rep     movsd
                             <1>
                             <1> loc_rename_df_parse_path_name:
00009598 8B35[5C810100]      <1>     mov     esi, [DestinationFilePath]
0000959E BF[16800100]        <1>     mov     edi, FindFile_Drv
000095A3 E8D6130000          <1>     call    parse_path_name
000095A8 7219                <1>     jc      short loc_rename_df_cmd_failed
                             <1>
                             <1>     ;mov    dh, [RUN_CDRV]
000095AA 8A35[76770100]      <1>     mov     dh, [Current_Drv]
                             <1>
                             <1>     ; 'rename' command is valid only for same dos drive and same dir!
                             <1>     ; ('move' command must be used if source file and destination file
                             <1>     ; directories are not same!)
000095B0 8A15[16800100]      <1>     mov     dl, [FindFile_Drv]
000095B6 38F2                <1>     cmp     dl, dh ; are source and destination drives different ?!
000095B8 7509                <1>     jne     short loc_rename_df_cmd_failed ; yes!
                             <1>
                             <1> rename_df_check_dirname_exists:
000095BA 803D[17800100]00    <1>     cmp     byte [FindFile_Directory], 0
000095C1 760B                <1>     jna     short rename_df_check_filename_exists
                             <1>
                             <1>     ; different source file and destination file directories !
                             <1> loc_rename_df_cmd_failed:
000095C3 B801000000          <1>     mov     eax, 1 ; TRDOS 'Bad command or file name' error
000095C8 F9                  <1>     stc
000095C9 E97AF7FFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> rename_df_check_filename_exists:
000095CE BE[58800100]        <1>     mov     esi, FindFile_Name
000095D3 E801F7FFFF          <1>     call    check_filename
                             <1>     ;jc     loc_mkdir_invalid_dir_name_chars
                             <1>     ; 28/07/2022
000095D8 7305                <1>     jnc     short loc_rename_file_name_ok
000095DA E91BF8FFFF          <1>     jmp     loc_mkdir_invalid_dir_name_chars
                             <1>
                             <1> loc_rename_file_name_ok:
                             <1>     ;mov    [DelFile_FNPointer], esi
                             <1>     ;cmp    byte [esi], 20h
                             <1>     ;ja     short loc_rename_df_find
                             <1>
                             <1>     ;mov    dh, [Current_Drv] ; dh has not been changed
                             <1>
                             <1> rename_df_drv_check_writable:
                             <1>     ;movzx  esi, dh
                             <1>     ;;movzx esi, byte [Current_Drv]
                             <1>     ;add    esi, Logical_DOSDisks
                             <1>     ; 25/07/2025 (BugFix)
000095DF 31C0                <1>     xor     eax, eax
000095E1 88F4                <1>     mov     ah, dh
000095E3 BE00010900          <1>     mov     esi, Logical_DOSDisks
000095E8 01C6                <1>     add     esi, eax
                             <1>
000095EA 88F2                <1>     mov     dl, dh ; dl = [Current_Drv]
000095EC 8A7601              <1>     mov     dh, [esi+LD_DiskType]
                             <1>
000095EF 80FE01              <1>     cmp     dh, 1 ; 0 = Invalid
000095F2 7310                <1>     jnb     short rename_df_compare_sf_df_name
                             <1>
                             <1>     ; 16/10/2016 (13h -> 30)
000095F4 B81E000000          <1>     mov     eax, 30 ; 'Disk write-protected' error
000095F9 8B1D[5C810100]      <1>     mov     ebx, [DestinationFilePath]
000095FF E944F7FFFF          <1>     jmp     loc_file_rw_cmd_failed
                             <1>
                             <1> rename_df_compare_sf_df_name:
00009604 BE[58800100]        <1>     mov     esi, FindFile_Name
00009609 BF[A2810100]        <1>     mov     edi, SourceFile_Name
                             <1>     ;mov    ecx, 12
                             <1>     ; 28/07/2022
0000960E 29C9                <1>     sub     ecx, ecx
00009610 B10C                <1>     mov     cl, 12
                             <1> rename_df_compare_sf_df_name_next:
00009612 AC                  <1>     lodsb
00009613 AE                  <1>     scasb
00009614 7506                <1>     jne     short loc_rename_df_find
00009616 08C0                <1>     or      al, al
00009618 74A9                <1>     jz      short loc_rename_df_cmd_failed
0000961A E2F6                <1>     loop    rename_df_compare_sf_df_name_next
                             <1>
                             <1> loc_rename_df_find:
                             <1>     ;mov    esi, [DelFile_FNPointer]
0000961C BE[58800100]        <1>     mov     esi, FindFile_Name
                             <1>
                             <1>     ;xor    ax, ax ; Any
                             <1>     ; 28/07/2022
00009621 31C0                <1>     xor     eax, eax ; 0 ; Any
00009623 E852F3FFFF          <1>     call    find_first_file
                             <1>     ;;jnc   short loc_rename_df_found
                             <1>     ;; 29/12/2017
                             <1>     ;jnc    loc_permission_denied
                             <1>     ; 28/07/2022
00009628 7205                <1>     jc      short loc_rename_df_check_error_code
0000962A E923F7FFFF          <1>     jmp     loc_permission_denied
```

```
                                <1>
                                <1> loc_rename_df_check_error_code:
                                <1>     ;cmp    eax, 2
0000962F 3C02                   <1>     cmp     al, 2 ; Not found error
00009631 7406                   <1>     je      short rename_df_move_find_struct_to_dest
00009633 F9                     <1>     stc
00009634 E90FF7FFFF             <1>     jmp     loc_file_rw_cmd_failed
                                <1>
                                <1> ;loc_rename_df_found:
                                <1>     ; 05/11/2016
                                <1>     ; Permission denied error
                                <1>     ;mov    eax, ERR_PERM_DENIED ; 29/12/2017
                                <1>     ;stc
                                <1>     ;jmp    loc_permission_denied  ; 06/11/2016
                                <1>
                                <1> rename_df_move_find_struct_to_dest:
00009639 BE[16800100]           <1>         mov     esi, FindFile_Drv
0000963E BF[E0810100]           <1>         mov     edi, DestinationFile_Drv
                                <1>     ;mov    ecx, 32
                                <1>     ; 28/07/2022
00009643 29C9                   <1>     sub     ecx, ecx
00009645 B120                   <1>     mov     cl, 32
00009647 F3A5                   <1>     rep     movsd
                                <1>
                                <1> loc_rename_df_process_q_sf:
                                <1>     ;mov    ecx, 12
00009649 B10C                   <1>     mov     cl, 12
0000964B BE[A2810100]           <1>     mov     esi, SourceFile_Name
00009650 BF[E6330100]           <1>     mov     edi, Rename_OldName
                                <1> rename_df_process_q_nml_1_sf:
00009655 AC                     <1>     lodsb
00009656 3C20                   <1>         cmp al, 20h
00009658 7603                   <1>         jna short rename_df_process_q_nml_2_sf
0000965A AA                     <1>     stosb
0000965B E2F8                   <1>     loop    rename_df_process_q_nml_1_sf
                                <1>
                                <1> rename_df_process_q_nml_2_sf:
0000965D C60700                 <1>     mov     byte [edi], 0
                                <1>
                                <1> loc_rename_df_process_q_df:
                                <1>     ;mov    ecx, 12
00009660 B10C                   <1>     mov     cl, 12
00009662 BE[22820100]           <1>     mov     esi, DestinationFile_Name
00009667 BF[F7330100]           <1>     mov     edi, Rename_NewName
                                <1> rename_df_process_q_nml_1_df:
0000966C AC                     <1>     lodsb
0000966D 3C20                   <1>     cmp     al, 20h
0000966F 7603                   <1>     jna     short loc_rename_df_process_q_nml_2_df
00009671 AA                     <1>     stosb
00009672 E2F8                   <1>     loop    rename_df_process_q_nml_1_df
                                <1>
                                <1> loc_rename_df_process_q_nml_2_df:
00009674 C60700                 <1>     mov     byte [edi], 0
                                <1>
                                <1> loc_rename_confirmation_question:
00009677 BE[BE330100]           <1>     mov     esi, Msg_DoYouWantRename
0000967C E839D7FFFF             <1>     call    print_msg
                                <1>
00009681 A0[BD810100]           <1>     mov     al, [SourceFile_DirEntry+11] ; Attributes
00009686 2410                   <1>     and     al, 10h
00009688 750C                   <1>     jnz     short rename_confirmation_question_dir
                                <1>
                                <1> rename_confirmation_question_file:
0000968A BE[D5330100]           <1>     mov     esi, Rename_File
0000968F E826D7FFFF             <1>     call    print_msg
00009694 EB0A                   <1>     jmp     short rename_confirmation_question_as
                                <1>
                                <1> rename_confirmation_question_dir:
00009696 BE[DB330100]           <1>     mov     esi, Rename_Directory
0000969B E81AD7FFFF             <1>     call    print_msg
                                <1>
                                <1> rename_confirmation_question_as:
000096A0 BE[E6330100]           <1>     mov     esi, Rename_OldName
000096A5 E810D7FFFF             <1>     call    print_msg
000096AA BE[F3330100]           <1>     mov     esi, Msg_File_rename_as
000096AF E806D7FFFF             <1>     call    print_msg
000096B4 BE[1A330100]           <1>     mov     esi, Msg_YesNo
000096B9 E8FCD6FFFF             <1>     call    print_msg
                                <1>
                                <1> loc_rename_ask_again:
000096BE 30E4                   <1>     xor     ah, ah
000096C0 E84D78FFFF             <1>     call    int16h
000096C5 3C1B                   <1>     cmp     al, 1Bh
000096C7 740F                   <1>     je      short loc_do_not_rename_file
000096C9 24DF                   <1>     and     al, 0DFh
000096CB A2[24330100]           <1>     mov     [Y_N_nextline], al
000096D0 3C59                   <1>     cmp     al, 'Y'
000096D2 7404                   <1>     je      short loc_yes_rename_file
000096D4 3C4E                   <1>     cmp     al, 'N'
000096D6 75E6                   <1>     jne     short loc_rename_ask_again
                                <1>
                                <1> loc_do_not_rename_file:
                                <1> loc_yes_rename_file:
000096D8 E89EF7FFFF             <1>     call    y_n_answer ; 29/12/2017
                                <1>     ;cmp    al, 'Y' ; 'yes'
                                <1>     ;cmc
                                <1>         ;jnc       loc_file_rw_restore_retn
000096DD 3C4E                   <1>     cmp     al, 'N' ; 'no'
                                <1>     ;je     loc_file_rw_restore_retn
                                <1>     ; 28/07/2022
000096DF 7505                   <1>     jne     short loc_rename_file_yes
000096E1 E962F6FFFF             <1>     jmp     loc_file_rw_restore_retn
                                <1>
                                <1> loc_rename_file_yes: ; 28/07/2022
000096E6 BE[F7330100]           <1>     mov     esi, Rename_NewName
000096EB 668B0D[DA810100]       <1>     mov     cx, [SourceFile_DirEntryNumber]
000096F2 66A1[C6810100]         <1>     mov     ax, [SourceFile_DirEntry+20] ; First Cluster, HW
000096F8 C1E010                 <1>     shl     eax, 16 ; 13/11/2017
000096FB 66A1[CC810100]         <1>     mov     ax, [SourceFile_DirEntry+26] ; First Cluster, LW
                                <1>
00009701 0FB61D[AF810100]       <1>     movzx   ebx, byte [SourceFile_LongNameEntryLength]
00009708 E8201A0000             <1>     call    rename_directory_entry
0000970D E94FF7FFFF             <1>     jmp     loc_rename_file_ok
                                <1> ;loc_rename_file_ok:
                                <1> ;   jc      loc_run_cmd_failed
                                <1> ;   mov     esi, Msg_OK
                                <1> ;   call    proc_printmsg
                                <1> ;   jmp     loc_file_rw_restore_retn
                                <1>
                                <1> move_file:
                                <1>     ; 07/08/2022
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 11/03/2016
                                <1>     ; 09/03/2016
```

```
                                <1>     ; 08/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 21/05/2011 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_move')
                                <1>     ; 23/04/2011
                                <1>
                                <1> get_move_source_fchar:
                                <1>     ; esi = file name
00009712 803E20                 <1>     cmp     byte [esi], 20h
00009715 7613                   <1>         jna short loc_move_nofilename_retn
                                <1>
00009717 8935[58810100]         <1>     mov     [SourceFilePath], esi
                                <1>
                                <1> move_scan_source_file:
0000971D 46                     <1>     inc     esi
0000971E 803E20                 <1>     cmp     byte [esi], 20h
00009721 7408                   <1>         je      short move_scan_destination_1
                                <1>     ;;jb    short loc_move_nofilename_retn
                                <1>     ;jb     loc_cmd_failed
                                <1>     ;jmp    short move_scan_source_file
                                <1>     ; 28/07/2022
00009723 77F8                   <1>     ja      short move_scan_source_file
                                <1> loc_move_failed:
00009725 E92AEDFFFF             <1>     jmp     loc_cmd_failed
                                <1>
                                <1> loc_move_nofilename_retn:
0000972A C3                     <1>     retn
                                <1>
                                <1> move_scan_destination_1:
0000972B C60600                 <1>     mov     byte [esi], 0
                                <1>
                                <1> move_scan_destination_2:
0000972E 46                     <1>     inc     esi
0000972F 803E20                 <1>     cmp     byte [esi], 20h
00009732 74FA                   <1>     je      short move_scan_destination_2
                                <1>     ;;jb    short loc_move_nofilename_retn
                                <1>     ;jb     loc_cmd_failed
                                <1>     ; 28/07/2022
00009734 72EF                   <1>     jb      short loc_move_failed
                                <1>
00009736 8935[5C810100]         <1>     mov     [DestinationFilePath], esi
                                <1>
                                <1> move_scan_destination_3:
0000973C 46                     <1>     inc     esi
0000973D 803E20                 <1>     cmp     byte [esi], 20h
00009740 77FA                   <1>     ja      short move_scan_destination_3
00009742 C60600                 <1>     mov     byte [esi], 0
                                <1>
                                <1> loc_move_scan_destination_OK:
00009745 8B35[58810100]         <1>     mov     esi, [SourceFilePath]
0000974B 8B3D[5C810100]         <1>     mov     edi, [DestinationFilePath]
                                <1>
00009751 B001                   <1>     mov     al, 1  ; move procedure Phase 1
00009753 E8411A0000             <1>     call    move_source_file_to_destination_file
00009758 7325                   <1>     jnc     short move_source_file_to_destination_question
                                <1>
                                <1> loc_move_cmd_failed_1:
0000975A 08C0                   <1>     or      al, al
                                <1>     ;jz     loc_cmd_failed
                                <1>     ; 28/07/2022
0000975C 74C7                   <1>     jz      short loc_move_failed
                                <1>
0000975E 3C11                   <1>     cmp     al, 11h
00009760 7409                   <1>     je      short loc_msg_not_same_device
                                <1>     ;cmp    al, 05h
                                <1>     ;cmp    al, ERR_PERM_DENIED ; 29/12/2017
                                <1>     ;jne    loc_run_cmd_failed
                                <1>     ;jmp    loc_permission_denied
00009762 3C0B                   <1>     cmp     al, ERR_PERM_DENIED
                                <1>     ;je     loc_permission_denied
                                <1>     ; 28/07/2022
00009764 7414                   <1>     je      short loc_move_perm_denied
00009766 E914EDFFFF             <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1>     ;mov    esi, Msg_Permission_denied
                                <1>     ;call   print_msg
                                <1>     ;jmp    loc_file_rw_restore_retn
                                <1>
                                <1> loc_msg_not_same_device:
0000976B BE[04340100]           <1>     mov     esi, msg_not_same_drv
00009770 E845D6FFFF             <1>     call    print_msg
00009775 E9CEF5FFFF             <1>     jmp     loc_file_rw_restore_retn
                                <1>
                                <1>     ; 28/07/2022
                                <1> loc_move_perm_denied:
0000977A E9D3F5FFFF             <1>     jmp     loc_permission_denied
                                <1>
                                <1> move_source_file_to_destination_question:
0000977F A0[60810100]           <1>         mov     al, [SourceFile_Drv]
00009784 0441                   <1>     add     al, 'A'
00009786 A2[66340100]           <1>     mov     [msg_source_file_drv], al
0000978B A0[E0810100]           <1>         mov     al, [DestinationFile_Drv]
00009790 0441                   <1>     add     al, 'A'
00009792 A2[85340100]           <1>     mov     [msg_destination_file_drv], al
                                <1>
00009797 57                     <1>     push    edi ; *
                                <1>
00009798 BE[4A340100]           <1>     mov     esi, msg_source_file
0000979D E818D6FFFF             <1>     call    print_msg
000097A2 BE[61810100]           <1>     mov     esi, SourceFile_Directory
000097A7 803E20                 <1>     cmp     byte [esi], 20h
000097AA 7605                   <1>     jna     short msftdfq_sfn
000097AC E809D6FFFF             <1>     call    print_msg
                                <1> msftdfq_sfn:
000097B1 BE[A2810100]           <1>     mov     esi, SourceFile_Name
000097B6 E8FFD5FFFF             <1>     call    print_msg
000097BB BE[69340100]           <1>     mov     esi, msg_destination_file
000097C0 E8F5D5FFFF             <1>     call    print_msg
000097C5 BE[E1810100]           <1>     mov     esi, DestinationFile_Directory
000097CA 803E20                 <1>     cmp     byte [esi], 20h
000097CD 7605                   <1>     jna     short msftdfq_dfn
000097CF E8E6D5FFFF             <1>     call    print_msg
                                <1> msftdfq_dfn:
000097D4 BE[22820100]           <1>     mov     esi, DestinationFile_Name
000097D9 E8DCD5FFFF             <1>     call    print_msg
000097DE BE[88340100]           <1>     mov     esi, msg_copy_nextline
000097E3 E8D2D5FFFF             <1>     call    print_msg
000097E8 BE[88340100]           <1>     mov     esi, msg_copy_nextline
000097ED E8C8D5FFFF             <1>     call    print_msg
                                <1>
                                <1> loc_move_ask_for_new_file_yes_no:
000097F2 BE[16340100]           <1>     mov     esi, Msg_DoYouWantMoveFile
000097F7 E8BED5FFFF             <1>     call    print_msg
000097FC BE[1A330100]           <1>     mov     esi, Msg_YesNo
00009801 E8B4D5FFFF             <1>     call    print_msg
                                <1> loc_move_ask_for_new_file_again:
```

```
00009806 30E4                <1>     xor     ah, ah
00009808 E80577FFFF          <1>     call    int16h
0000980D 3C1B                <1>     cmp     al, 1Bh
                             <1>     ;je     short loc_do_not_move_file
0000980F 743F                <1>     je      short loc_move_y_n_escape
00009811 24DF                <1>     and     al, 0DFh
00009813 A2[24330100]        <1>         mov     [Y_N_nextline], al
00009818 3C59                <1>     cmp     al, 'Y'
0000981A 7404                <1>     je      short loc_yes_move_file
0000981C 3C4E                <1>     cmp     al, 'N'
0000981E 75E6                <1>     jne     short loc_move_ask_for_new_file_again
                             <1>
                             <1> loc_do_not_move_file:
                             <1> loc_yes_move_file:
00009820 E856F6FFFF          <1>     call    y_n_answer ; 29/12/2017
00009825 5F                  <1>     pop     edi ; *
                             <1>     ;cmp    al, 'Y' ; 'yes'
                             <1>     ;cmc
                             <1>         ;jnc       loc_file_rw_restore_retn
00009826 3C4E                <1>     cmp     al, 'N' ; 'no'
                             <1>         ;je loc_file_rw_restore_retn
                             <1>     ; 28/07/2022
00009828 7421                <1>     je      short loc_move_rw_restore_retn
                             <1>
                             <1> loc_move_yes_move_file:
0000982A B002                <1>     mov     al, 2 ; move procedure Phase 2
0000982C E868190000          <1>     call    move_source_file_to_destination_file
                             <1>     ;;jc    short loc_move_cmd_failed_2
                             <1>         ;jnc       move_source_file_to_dest_OK
                             <1>     ; 28/07/2022
00009831 7205                <1>     jc      short loc_move_cmd_failed_2
                             <1>     ; 07/08/2022
00009833 E930F6FFFF          <1>     jmp     move_source_file_to_dest_OK
                             <1>
                             <1> ;move_source_file_to_destination_OK:
                             <1> ;   mov     esi, Msg_OK
                             <1> ;   call    print_msg
                             <1> ;   jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_move_cmd_failed_2:
00009838 3C27                <1>     cmp     al, 27h
                             <1>     ;jne    loc_run_cmd_failed
                             <1>     ; 28/07/2022
0000983A 7405                <1>     je      short loc_move_ids_err
0000983C E93EECFFFF          <1>     jmp     loc_run_cmd_failed
                             <1>
                             <1> loc_move_ids_err: ; 28/07/2022
00009841 BE[2F340100]        <1>     mov     esi, msg_insufficient_disk_space
00009846 E86FD5FFFF          <1>     call    print_msg
                             <1>
                             <1> loc_move_rw_restore_retn: ; 28/07/2022
0000984B E9F8F4FFFF          <1>     jmp     loc_file_rw_restore_retn
                             <1>
                             <1> loc_move_y_n_escape:
00009850 B04E                <1>     mov     al, 'N' ; 'no'
00009852 EBCC                <1>     jmp     short loc_do_not_move_file
                             <1>
                             <1> copy_file:
                             <1>     ; 31/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 15/10/2016
                             <1>     ; 24/03/2016
                             <1>     ; 21/03/2016
                             <1>     ; 15/03/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 21/05/2011 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_copy')
                             <1>     ; 01/08/2010
                             <1>
                             <1> get_copy_source_fchar:
                             <1>     ; esi = file name
00009854 803E20              <1>     cmp     byte [esi], 20h
00009857 7613                <1>         jna     short loc_copy_nofilename_retn
                             <1>
00009859 8935[58810100]      <1>     mov     [SourceFilePath], esi
                             <1>
                             <1> copy_scan_source_file:
0000985F 46                  <1>     inc     esi
00009860 803E20              <1>     cmp     byte [esi], 20h
00009863 7408                <1>     je      short copy_scan_destination_1
                             <1>     ;;jb    short loc_copy_nofilename_retn
                             <1>     ;jb     loc_cmd_failed
                             <1>     ;jmp    short copy_scan_source_file
                             <1>     ; 25/07/2022
00009865 73F8                <1>     jnb     short copy_scan_source_file
                             <1> copy_scan_destination_0:
00009867 E9E8EBFFFF          <1>     jmp     loc_cmd_failed
                             <1>
                             <1> loc_copy_nofilename_retn:
0000986C C3                  <1>     retn
                             <1>
                             <1> copy_scan_destination_1:
0000986D C60600              <1>     mov     byte [esi], 0
                             <1>
                             <1> copy_scan_destination_2:
00009870 46                  <1>     inc     esi
00009871 803E20              <1>     cmp     byte [esi], 20h
00009874 74FA                <1>     je      short copy_scan_destination_2
                             <1>     ;;jb    short loc_copy_nofilename_retn
                             <1>     ;jb     loc_cmd_failed
                             <1>     ; 25/07/2022
00009876 72EF                <1>     jb      short copy_scan_destination_0
                             <1>
00009878 8935[5C810100]      <1>     mov     [DestinationFilePath], esi
                             <1>
                             <1> copy_scan_destination_3:
0000987E 46                  <1>     inc     esi
0000987F 803E20              <1>     cmp     byte [esi], 20h
00009882 77FA                <1>     ja      short copy_scan_destination_3
00009884 C60600              <1>     mov     byte [esi], 0
                             <1>
                             <1> loc_copy_save_current_drive:
00009887 8A35[76770100]      <1>     mov     dh, [Current_Drv]
0000988D 8835[D37E0100]      <1>     mov     [RUN_CDRV], dh
                             <1>
                             <1> copy_source_file_to_destination_phase_1:
00009893 8B35[58810100]      <1>     mov     esi, [SourceFilePath]
00009899 8B3D[5C810100]      <1>     mov     edi, [DestinationFilePath]
                             <1>
0000989F B001                <1>     mov     al, 1  ; copy procedure Phase 1
000098A1 E8571B0000          <1>     call    copy_source_file_to_destination_file
000098A6 7327                <1>     jnc     short copy_source_file_to_destination_question
                             <1>
                             <1> loc_copy_cmd_failed_1:
                             <1>     ; 18/03/2016 (restore current drive and directory)
000098A8 08C0                <1>     or      al, al
```

```
000098AA 7507                <1>     jnz     short loc_copy_cmd_failed_2
                             <1>
000098AC FEC0                <1>         inc     al ; mov al, 1 ; Bad command or file name !
                             <1> loc_copy_cmd_failed_3:    ; 25/07/2022
000098AE E9CCEBFFFF          <1>     jmp     loc_run_cmd_failed
                             <1>
                             <1> loc_copy_cmd_failed_2:
000098B3 3C27                <1>     cmp     al, 27h ; Insufficient disk space
000098B5 7409                <1>     je      short loc_file_write_insuff_disk_space_msg
                             <1>
                             <1>     ; 29/12/2017
                             <1>     ;cmp    al, 05h
000098B7 3C0B                <1>     cmp     al, ERR_PERM_DENIED
                             <1>     ;jne    loc_run_cmd_failed
                             <1>     ; 25/07/2022
000098B9 75F3                <1>     jne     short loc_copy_cmd_failed_3
                             <1>
000098BB E992F4FFFF          <1>     jmp     loc_permission_denied
                             <1>
                             <1> loc_file_write_insuff_disk_space_msg:
000098C0 BE[2F340100]        <1>     mov     esi, msg_insufficient_disk_space
000098C5 E8F0D4FFFF          <1>     call    print_msg
000098CA E979F4FFFF          <1>         jmp     loc_file_rw_restore_retn
                             <1>
                             <1> copy_source_file_to_destination_question:
000098CF 57                  <1>     push    edi ; *
                             <1>
                             <1>     ; dh = source file attributes
                             <1>     ; dl > 0 -> destination file found
000098D0 20D2                <1>     and     dl, dl
000098D2 7446                <1>     jz      short copy_source_file_to_destination_pass_owrq
                             <1>
                             <1> loc_copy_ask_for_owr_yes_no:
000098D4 BE[8B340100]        <1>     mov     esi, Msg_DoYouWantOverWriteFile
000098D9 E8DCD4FFFF          <1>     call    print_msg
000098DE BE[22820100]        <1>     mov     esi, DestinationFile_Name
000098E3 E8D2D4FFFF          <1>     call    print_msg
000098E8 BE[1A330100]        <1>     mov     esi, Msg_YesNo
000098ED E8C8D4FFFF          <1>     call    print_msg
                             <1>
                             <1> loc_copy_ask_for_owr_again:
000098F2 30E4                <1>     xor     ah, ah
000098F4 E81976FFFF          <1>     call    int16h
000098F9 3C1B                <1>     cmp     al, 1Bh
                             <1>         ;je     loc_do_not_copy_file
000098FB 7419                <1>         je      short loc_copy_y_n_escape
000098FD 24DF                <1>     and     al, 0DFh
000098FF A2[24330100]        <1>         mov     [Y_N_nextline], al
00009904 3C59                <1>     cmp     al, 'Y'
                             <1>     ;je     loc_yes_copy_file
                             <1>     ; 25/07/2022
00009906 7505                <1>     jne     short loc_copy_ask_for_owr_n
00009908 E9AD000000          <1>     jmp     loc_yes_copy_file
                             <1>
                             <1> loc_copy_ask_for_owr_n: ; 25/07/2022
0000990D 3C4E                <1>     cmp     al, 'N'
                             <1>         ;je loc_do_not_copy_file
                             <1>     ;jmp    short loc_copy_ask_for_owr_again
                             <1>     ; 25/07/2022
0000990F 75E1                <1>     jne     short loc_copy_ask_for_owr_again
                             <1> loc_do_not_copy_file_j:
00009911 E9A4000000          <1>     jmp     loc_do_not_copy_file
                             <1>
                             <1> loc_copy_y_n_escape:
00009916 B04E                <1>     mov     al, 'N' ; 'no'
                             <1>     ;jmp    loc_do_not_copy_file
                             <1>     ; 25/07/2022
00009918 EBF7                <1>     jmp     short loc_do_not_copy_file_j
                             <1>
                             <1> copy_source_file_to_destination_pass_owrq:
0000991A A0[60810100]        <1>     mov     al, [SourceFile_Drv]
0000991F 0441                <1>     add     al, 'A'
00009921 A2[66340100]        <1>     mov     [msg_source_file_drv], al
00009926 A0[E0810100]        <1>         mov     al, [DestinationFile_Drv]
0000992B 0441                <1>     add     al, 'A'
0000992D A2[85340100]        <1>     mov     [msg_destination_file_drv], al
                             <1>
00009932 BE[4A340100]        <1>     mov     esi, msg_source_file
00009937 E87ED4FFFF          <1>     call    print_msg
0000993C BE[61810100]        <1>     mov     esi, SourceFile_Directory
00009941 803E20              <1>     cmp     byte [esi], 20h
00009944 7605                <1>     jna     short csftdfq_sfn
00009946 E86FD4FFFF          <1>     call    print_msg
                             <1> csftdfq_sfn:
0000994B BE[A2810100]        <1>     mov     esi, SourceFile_Name
00009950 E865D4FFFF          <1>     call    print_msg
00009955 BE[69340100]        <1>     mov     esi, msg_destination_file
0000995A E85BD4FFFF          <1>     call    print_msg
0000995F BE[E1810100]        <1>     mov     esi, DestinationFile_Directory
00009964 803E20              <1>     cmp     byte [esi], 20h
00009967 7605                <1>     jna     short csftdfq_dfn
00009969 E84CD4FFFF          <1>     call    print_msg
                             <1> csftdfq_dfn:
0000996E BE[22820100]        <1>     mov     esi, DestinationFile_Name
00009973 E842D4FFFF          <1>     call    print_msg
00009978 BE[88340100]        <1>     mov     esi, msg_copy_nextline
0000997D E838D4FFFF          <1>     call    print_msg
00009982 BE[88340100]        <1>     mov     esi, msg_copy_nextline
00009987 E82ED4FFFF          <1>     call    print_msg
                             <1>
                             <1> loc_copy_ask_for_new_file_yes_no:
0000998C BE[AA340100]        <1>     mov     esi, Msg_DoYouWantCopyFile
00009991 E824D4FFFF          <1>     call    print_msg
00009996 BE[1A330100]        <1>     mov     esi, Msg_YesNo
0000999B E81AD4FFFF          <1>     call    print_msg
                             <1>
                             <1> loc_copy_ask_for_new_file_again:
000099A0 30E4                <1>     xor     ah, ah
000099A2 E86B75FFFF          <1>     call    int16h
000099A7 3C1B                <1>     cmp     al, 1Bh
000099A9 740F                <1>     je      short loc_do_not_copy_file
000099AB 24DF                <1>     and     al, 0DFh
000099AD A2[24330100]        <1>         mov     [Y_N_nextline], al
000099B2 3C59                <1>     cmp     al, 'Y'
000099B4 7404                <1>     je      short loc_yes_copy_file
000099B6 3C4E                <1>     cmp     al, 'N'
000099B8 75E6                <1>     jne     short loc_copy_ask_for_new_file_again
                             <1>
                             <1> loc_do_not_copy_file:
                             <1> loc_yes_copy_file:
000099BA E8BCF4FFFF          <1>     call    y_n_answer ; 29/12/2017
000099BF 5F                  <1>     pop     edi ; *
                             <1>     ;cmp    al, 'Y' ; 'yes'
                             <1>     ;cmc
```

```
                                <1>         ;jnc       loc_file_rw_restore_retn
000099C0 3C4E                   <1>     cmp    al, 'N' ; 'no'
                                <1>     ;je    loc_file_rw_restore_retn
                                <1>     ; 25/07/2022
000099C2 7505                   <1>     jne    short copy_source_file_to_destination_pass_q
000099C4 E97FF3FFFF             <1>     jmp    loc_file_rw_restore_retn
                                <1>
                                <1> copy_source_file_to_destination_pass_q:
000099C9 B002                   <1>     mov    al, 2  ; copy procedure Phase 2
000099CB E82D1A0000             <1>     call   copy_source_file_to_destination_file
                                <1>     ;jc    short loc_file_write_check_disk_space_err
                                <1>
                                <1>     ; 31/08/2024
000099D0 9C                     <1>     pushf
                                <1>
                                <1>     ; 24/03/2016
                                <1>     ;;push cx
                                <1>     ;push  ecx ; 29/12/2017
                                <1>
000099D1 BE[88340100]           <1>     mov    esi, msg_copy_nextline
000099D6 E8DFD3FFFF             <1>     call   print_msg
                                <1>
                                <1>     ;pop   eax ; 29/12/2017
                                <1>     ;;pop  cx
                                <1>     ;pop   ax
                                <1>
                                <1>     ; 31/08/2024
000099DB 9D                     <1>     popf
000099DC 7305                   <1>     jnc    short copy_source_file_to_destination_OK
                                <1>
                                <1>     ; 31/08/2024
                                <1>     ;;or   cl, cl
                                <1>     ;;or   al, al
                                <1>     ;jz    short copy_source_file_to_destination_OK
                                <1>     ;
                                <1>     ;; 15/10/2016 (1Dh -> 18)
                                <1>     ;; 18/03/2016 (1Dh)
                                <1>     ;;cmp  cl, 18 ; write error
                                <1>     ;cmp   al, 18
                                <1>     ;jne   short copy_source_file_to_destination_not_OK
                                <1>     ;;
                                <1>     ;;mov  al, cl ; error number (write fault!)
                                <1>     ;stc
                                <1>
000099DE E965F3FFFF             <1>     jmp    loc_file_rw_cmd_failed
                                <1>
                                <1>     ; 31/08/2024
                                <1> ;copy_source_file_to_destination_not_OK:
                                <1>     ;mov   esi, Msg_read_file_error_before_EOF
                                <1>     ;call  print_msg
                                <1>     ;jmp   loc_file_rw_restore_retn
                                <1>
                                <1> copy_source_file_to_destination_OK:
000099E3 BE[28330100]           <1>     mov    esi, Msg_OK
000099E8 E8CDD3FFFF             <1>     call   print_msg
                                <1>
000099ED E956F3FFFF             <1>     jmp    loc_file_rw_restore_retn
                                <1>
                                <1> ;loc_file_write_check_disk_space_err:
                                <1>     ;cmp   al, 27h ; Insufficient disk space
                                <1>     ;je    loc_file_write_insuff_disk_space_msg
                                <1>         ;jb loc_file_rw_cmd_failed
                                <1>
                                <1>     ;call  print_misc_error_msg ; 15/03/2016
                                <1>         ;jmp       loc_file_rw_restore_retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1> change_fs_file_attributes:
                                <1>     ; 04/03/2016 ; Temporary
                                <1>     ; AL = File or directory attributes
                                <1>     ; AH = 0 -> Attributes are in MS-DOS format
                                <1>     ; AH > 0 -> Attributes are in SINGLIX format
                                <1>     ;push  ebx
                                <1>     ; ... do somethings here ...
                                <1>     ;pop   ebx
                                <1>     ; BL = File or directory attributes
                                <1>     retn
                                <1> %endif
                                <1>
                                <1> set_get_env:
                                <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 11/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 02/09/2011 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_set')
                                <1>     ; 2005 - 28/08/2011
                                <1> get_setenv_fchar:
                                <1>     ; esi = environment variable/string
000099F2 8A06                   <1>     mov    al, [esi]
000099F4 3C20                   <1>     cmp    al, 20h
000099F6 771E                   <1>     ja     short loc_find_env
                                <1>
000099F8 BE00300900             <1>     mov    esi, Env_Page
                                <1> loc_print_setline:
000099FD 803E00                 <1>     cmp    byte [esi], 0
00009A00 7613                   <1>     jna    short loc_setenv_retn
00009A02 E8B3D3FFFF             <1>     call   print_msg
00009A07 56                     <1>     push   esi
00009A08 BE[17390100]           <1>     mov    esi, nextline
00009A0D E8A8D3FFFF             <1>     call   print_msg
00009A12 5E                     <1>     pop    esi
00009A13 EBE8                   <1>     jmp    short loc_print_setline
                                <1>
                                <1> loc_setenv_retn:
00009A15 C3                     <1>     retn
                                <1>
                                <1> loc_find_env:
00009A16 3C3D                   <1>     cmp    al, '='
                                <1>     ;je    loc_cmd_failed
                                <1>     ; 25/07/2022
00009A18 7505                   <1>     jne    short loc_find_envr
00009A1A E935EAFFFF             <1>     jmp    loc_cmd_failed
                                <1>
                                <1> loc_find_envr:     ; 25/07/2022
00009A1F 56                     <1>     push   esi
                                <1> loc_repeat_env_equal_check:
00009A20 46                     <1>     inc    esi
00009A21 803E3D                 <1>     cmp    byte [esi], '='
00009A24 7430                   <1>     je     short pass_env_equal_check
00009A26 803E20                 <1>     cmp    byte [esi], 20h
00009A29 73F5                   <1>     jnb    short loc_repeat_env_equal_check
00009A2B C60600                 <1>     mov    byte [esi], 0
00009A2E 5E                     <1>     pop    esi
                                <1>     ; 25/07/2022 (*)
                                <1> loc_print_env_string:
```

```
00009A2F BF[76780100]        <1>     mov     edi, TextBuffer ; out buffer
00009A34 B9FF000000          <1>     mov     ecx, 255 ; maximum size (limit)
00009A39 30C0                <1>     xor     al, al ; 0 -> use [ESI]
00009A3B E877000000          <1>     call    get_environment_string
00009A40 72D3                <1>     jc      short loc_setenv_retn
                             <1>      ; 25/07/2022
                             <1> ;loc_print_env_string:
00009A42 BE[76780100]        <1>     mov     esi, TextBuffer
00009A47 E86ED3FFFF          <1>     call    print_msg
00009A4C BE[17390100]        <1>     mov     esi, nextline
                             <1>     ;call   print_msg
                             <1>     ;retn
                             <1>     ; 25/07/2022
00009A51 E964D3FFFF          <1>     jmp     print_msg
                             <1>
                             <1> pass_env_equal_check:
00009A56 46                  <1>     inc     esi
00009A57 803E20              <1>     cmp     byte [esi], 20h
00009A5A 73FA                <1>     jnb     short pass_env_equal_check
00009A5C C60600              <1>     mov     byte [esi], 0
                             <1>
                             <1> loc_call_set_env_string:
00009A5F 5E                  <1>     pop     esi
00009A60 E815010000          <1>     call    set_environment_string
00009A65 73AE                <1>     jnc     short loc_setenv_retn
                             <1>
                             <1> loc_set_cmd_failed:
00009A67 3C08                <1>     cmp     al, 08h
                             <1>     ;jne    loc_cmd_failed
                             <1>     ; 25/07/2022
00009A69 7405                <1>     je      short loc_set_cmd_failed_spc
00009A6B E9E4E9FFFF          <1>     jmp     loc_cmd_failed
                             <1>
                             <1> loc_set_cmd_failed_spc:
00009A70 BE[03350100]        <1>     mov     esi, Msg_No_Set_Space
                             <1>     ;call   print_msg
                             <1>     ;retn
                             <1>     ; 25/07/2022
00009A75 E940D3FFFF          <1>     jmp     print_msg
                             <1>
                             <1> set_get_path:
                             <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 11/04/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 03/09/2011 (TRDOS v1, CMD_INTR.ASM, 'cmp_cmd_path')
                             <1>     ; 2005
                             <1> get_path_fchar:
                             <1>     ; esi = path
00009A7A 803E20              <1>     cmp     byte [esi], 20h
00009A7D 7711                <1>     ja      short loc_set_path
                             <1>
00009A7F BE00300900          <1>     mov     esi, Env_Page
                             <1> loc_print_path:
00009A84 803E00              <1>     cmp     byte [esi], 0
00009A87 762D                <1>     jna     short loc_path_retn
                             <1>
00009A89 BE[632F0100]        <1>     mov     esi, Cmd_Path ; 'PATH' address
                             <1>     ; 25/07/2022 (*)
00009A8E EB9F                <1>     jmp     short loc_print_env_string
                             <1> ; 25/07/2022
                             <1> ;   mov     edi, TextBuffer ; out buffer
                             <1> ;   xor     al, al  ; use [ESI]
                             <1> ;   mov     ecx, 255 ; maximum size (limit)
                             <1> ;   call    get_environment_string
                             <1> ;   ;jc     short loc_path_retn
                             <1> ;   ; 25/07/2022
                             <1> ;   jnc     short loc_print_env_string
                             <1> ;   retn
                             <1>
                             <1> ;   mov     esi, TextBuffer
                             <1> ;   call    print_msg
                             <1> ;   mov     esi, nextline
                             <1> ;   ;call   print_msg
                             <1> ;loc_path_retn:
                             <1> ;   ;retn
                             <1> ;   ; 25/07/2022
                             <1> ;   jmp     print_msg
                             <1>
                             <1> loc_set_path:
00009A90 56                  <1>     push    esi
                             <1> loc_set_path_find_end:
00009A91 46                  <1>     inc     esi
00009A92 803E20              <1>     cmp     byte [esi], 20h
00009A95 73FA                <1>     jnb     short loc_set_path_find_end
00009A97 C60600              <1>     mov     byte [esi], 0
                             <1> loc_set_path_header:
00009A9A 5E                  <1>     pop     esi
                             <1> set_path_x: ; 31/12/2017 ('syspath')
00009A9B 4E                  <1>     dec     esi
00009A9C C6063D              <1>     mov     byte [esi], '='
00009A9F 4E                  <1>     dec     esi
00009AA0 C60648              <1>     mov     byte [esi], 'H'
00009AA3 4E                  <1>     dec     esi
00009AA4 C60654              <1>     mov     byte [esi], 'T'
00009AA7 4E                  <1>     dec     esi
00009AA8 C60641              <1>     mov     byte [esi], 'A'
00009AAB 4E                  <1>     dec     esi
00009AAC C60650              <1>     mov     byte [esi], 'P'
                             <1>
                             <1> loc_path_call_set_env_string:
00009AAF E8C6000000          <1>     call    set_environment_string
00009AB4 72B1                <1>         jc  short loc_set_cmd_failed
                             <1> loc_path_retn:     ; 25/07/2022
00009AB6 C3                  <1>     retn
                             <1>
                             <1> get_environment_string:
                             <1>     ; 12/04/2016
                             <1>     ; 11/04/2016
                             <1>     ; 05/04/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 02/09/2011 (TRDOS v1, MAINPROG.ASM)
                             <1>     ; 28/08/2011
                             <1>     ; INPUT->
                             <1>     ;       EDI = Output buffer
                             <1>     ;       CX = Buffer length (<= ENV_PAGE_SIZE)
                             <1>     ;
                             <1>     ;       AL > 0 = AL = String sequence number
                             <1>     ;       AL = 0 -> ESI = ASCIIZ Set word
                             <1>     ;               (environment variable)
                             <1>     ; OUTPUT ->
                             <1>     ;       ESI is not changed
                             <1>     ;       EDI is not changed
                             <1>     ;       EAX = String length (with zero tail)
                             <1>     ;       EDX = Environment variables page address
                             <1>     ;       CF = 1 -> Not found (EAX not valid)
                             <1>     ;
```

```
                                <1>     ; (Modified registers: EAX, EDX)
                                <1>
00009AB7 BA00300900              <1>     mov     edx, Env_Page
00009ABC 803A00                  <1>     cmp     byte [edx], 0
00009ABF 7474                    <1>     jz      short get_env_string_with_word_stc_retn
                                <1>
00009AC1 66890D[E2820100]        <1>     mov     [env_var_length], cx
                                <1>
00009AC8 51                      <1>     push    ecx ; *
00009AC9 56                      <1>     push    esi ; **
                                <1>
00009ACA 08C0                    <1>     or      al, al
00009ACC 7449                    <1>     jz      short get_env_string_with_word
                                <1>
                                <1> get_env_string_with_seq_number:
00009ACE B101                    <1>     mov     cl, 1
00009AD0 88C5                    <1>     mov     ch, al
00009AD2 31C0                    <1>     xor     eax, eax
00009AD4 89D6                    <1>     mov     esi, edx ; Env_Page
                                <1>
                                <1> get_env_string_seq_number_check:
00009AD6 38CD                    <1>     cmp     ch, cl
00009AD8 7726                    <1>     ja      short get_env_string_seq_number_next
                                <1>
                                <1> get_env_string_move_to_buff:
00009ADA 57                      <1>     push    edi ; ***
                                <1>
00009ADB 29D2                    <1>     sub     edx, edx
                                <1>
                                <1> get_env_string_seq_number_repeat1:
00009ADD 42                      <1>     inc     edx
00009ADE AC                      <1>     lodsb
00009ADF AA                      <1>     stosb
                                <1>
00009AE0 66FF0D[E2820100]        <1>     dec     word [env_var_length]
00009AE7 7508                    <1>     jnz     short get_env_string_seq_number_repeat3
                                <1>
                                <1> get_env_string_seq_number_repeat2:
00009AE9 20C0                    <1>     and     al, al
00009AEB 7408                    <1>     jz      short get_env_string_seq_number_ok
00009AED 42                      <1>     inc     edx
00009AEE AC                      <1>     lodsb
00009AEF EBF8                    <1>     jmp     short get_env_string_seq_number_repeat2
                                <1>
                                <1> get_env_string_seq_number_repeat3:
00009AF1 08C0                    <1>     or      al, al
00009AF3 75E8                    <1>     jnz     short get_env_string_seq_number_repeat1
                                <1>
                                <1> get_env_string_seq_number_ok:
00009AF5 5F                      <1>     pop     edi ; ***
00009AF6 89D0                    <1>     mov     eax, edx ; Length of the environment string
                                <1>                     ; (ASCIIZ, includes ZERO tail)
00009AF8 BA00300900               <1>     mov     edx, Env_Page
                                <1>
                                <1> get_env_string_stc_retn:
00009AFD 5E                      <1>     pop     esi ; **
00009AFE 59                      <1>     pop     ecx ; *
00009AFF C3                      <1>     retn
                                <1>
                                <1> get_env_string_seq_number_next:
00009B00 AC                      <1>     lodsb
00009B01 08C0                    <1>     or      al, al
00009B03 75FB                    <1>     jnz     short get_env_string_seq_number_next
                                <1>
00009B05 81FE00320900            <1>     cmp     esi, Env_Page + Env_Page_Size ; +512 (+4096)
00009B0B F5                      <1>     cmc
00009B0C 72EF                    <1>     jc      short get_env_string_stc_retn
                                <1>
00009B0E AC                      <1>     lodsb
00009B0F 3C01                    <1>     cmp     al, 1
00009B11 72EA                    <1>     jb      short get_env_string_stc_retn
00009B13 FEC1                    <1>     inc     cl
00009B15 EBBF                    <1>     jmp     short get_env_string_seq_number_check
                                <1>
                                <1> get_env_string_with_word:
00009B17 31C9                    <1>     xor     ecx, ecx
                                <1>
                                <1> get_env_string_calc_word_length:
00009B19 AC                      <1>     lodsb
00009B1A 3C20                    <1>     cmp     al, 20h
00009B1C 7211                    <1>     jb      short get_env_string_calc_word_length_ok
                                <1>     ;inc    cx
00009B1E FEC1                    <1>     inc     cl
                                <1>
00009B20 3C61                    <1>     cmp     al, 'a'
00009B22 72F5                    <1>     jb      short get_env_string_calc_word_length
00009B24 3C7A                    <1>     cmp     al, 'z'
00009B26 77F1                    <1>     ja      short get_env_string_calc_word_length
00009B28 24DF                    <1>     and     al, 0DFh
00009B2A 8846FF                  <1>     mov     [esi-1], al
00009B2D EBEA                    <1>     jmp     short get_env_string_calc_word_length
                                <1>
                                <1> get_env_string_calc_word_length_ok:
00009B2F 08C9                    <1>     or      cl, cl
00009B31 7506                    <1>     jnz     short get_env_string_calc_word_length_save
                                <1>
00009B33 5E                      <1>     pop     esi ; **
                                <1>
                                <1> get_env_string_stc_retn1:
00009B34 59                      <1>     pop     ecx ; *
                                <1>
                                <1> get_env_string_with_word_stc_retn:
00009B35 31C0                    <1>     xor     eax, eax
00009B37 F9                      <1>     stc
00009B38 C3                      <1>     retn
                                <1>
                                <1> get_env_string_calc_word_length_save:
00009B39 871C24                  <1>     xchg    ebx, [esp] ; **
00009B3C 89DE                    <1>     mov     esi, ebx
                                <1>             ; Start of the env string (to be searched)
                                <1>
00009B3E 57                      <1>     push    edi ; ***
00009B3F 89D7                    <1>     mov     edi, edx ; Env_Page
                                <1>
                                <1> get_env_string_compare:
00009B41 57                      <1>     push    edi ; ****
00009B42 51                      <1>     push    ecx ; ***** ; Variable name length
                                <1>
                                <1> get_env_string_compare_rep:
00009B43 AC                      <1>     lodsb
00009B44 AE                      <1>     scasb
00009B45 7511                    <1>     jne     short get_env_string_compare_next1
00009B47 E2FA                    <1>     loop    get_env_string_compare_rep
                                <1>
```

```
00009B49 803F3D          <1>     cmp     byte [edi], '='
00009B4C 750A            <1>     jne     short get_env_string_compare_next1
                         <1>
00009B4E 59              <1>     pop     ecx ; *****
00009B4F 5F              <1>     pop     edi ; ****
00009B50 89FE            <1>     mov     esi, edi
00009B52 5F              <1>     pop     edi ; ***
00009B53 871C24          <1>     xchg    ebx, [esp] ; **
00009B56 EB82            <1>     jmp     short get_env_string_move_to_buff
                         <1>
                         <1> get_env_string_compare_next1:
00009B58 89FE            <1>     mov     esi, edi
00009B5A 59              <1>     pop     ecx ; *****
00009B5B 5F              <1>     pop     edi ; ****
                         <1> get_env_string_compare_next2:
00009B5C 81FEFF310900    <1>     cmp     esi, Env_Page + Env_Page_Size - 1 ; +511 (+4095)
00009B62 7310            <1>     jnb     short get_env_string_compare_not_ok
00009B64 20C0            <1>     and     al, al
00009B66 AC              <1>     lodsb
00009B67 75F3            <1>     jnz     short get_env_string_compare_next2
00009B69 08C0            <1>     or      al, al
00009B6B 7407            <1>     jz      short get_env_string_compare_not_ok
00009B6D 4E              <1>     dec     esi ; 12/04/2016
00009B6E 89F7            <1>     mov     edi, esi
00009B70 89DE            <1>     mov     esi, ebx
00009B72 EBCD            <1>     jmp     short get_env_string_compare
                         <1>
                         <1> get_env_string_compare_not_ok:
00009B74 5F              <1>     pop     edi ; ***
00009B75 89DE            <1>     mov     esi, ebx
00009B77 5B              <1>     pop     ebx ; **
00009B78 EBBA            <1>     jmp     short get_env_string_stc_retn1
                         <1>
                         <1> set_environment_string:
                         <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                         <1>     ; 13/04/2016
                         <1>     ; 12/04/2016
                         <1>     ; 11/04/2016
                         <1>     ; 06/04/2016
                         <1>     ; 05/04/2016 (TRDOS 386 = TRDOS v2.0)
                         <1>     ; 02/09/2011 (TRDOS v1, MAINPROG.ASM)
                         <1>     ; 29/08/2011
                         <1>     ; 29/08/2011
                         <1>     ; INPUT->
                         <1>     ;       ESI = ASCIIZ environment string
                         <1>     ; OUTPUT ->
                         <1>     ;       ESI is not changed
                         <1>     ;       CF = 1 -> Could not set,
                         <1>     ;            insufficient environment space
                         <1>     ;
                         <1>     ; (EAX, EDX will be changed)
                         <1>     ;
                         <1>     ;    (EAX = Start address of the env string if > 0)
                         <1>     ;    (EDX = Environment string length)
                         <1>
00009B7A 56              <1>     push    esi ; *
                         <1>
00009B7B 31C0            <1>     xor     eax, eax
                         <1>
                         <1> set_env_chk_validation1:
00009B7D FEC4            <1>     inc     ah ; variable (string) length
00009B7F AC              <1>     lodsb
00009B80 3C3D            <1>     cmp     al, '='
00009B82 7415            <1>     je      short set_env_chk_validation2
00009B84 3C20            <1>     cmp     al, 20h
00009B86 720F            <1>     jb      short set_env_string_stc
                         <1>
                         <1>     ; 06/04/2016
00009B88 3C61            <1>     cmp     al, 'a'
00009B8A 72F1            <1>     jb      short set_env_chk_validation1
00009B8C 3C7A            <1>     cmp     al, 'z'
00009B8E 77ED            <1>     ja      short set_env_chk_validation1
00009B90 2C20            <1>     sub     al, 'a'-'A'
00009B92 8846FF          <1>     mov     [esi-1], al
00009B95 EBE6            <1>     jmp     short set_env_chk_validation1
                         <1>
                         <1> set_env_string_stc:
00009B97 5E              <1>     pop     esi ; *
                         <1>     ;stc
00009B98 C3              <1>     retn
                         <1>
                         <1> set_env_chk_validation2:
00009B99 51              <1>     push    ecx ; **
00009B9A 53              <1>     push    ebx ; ***
00009B9B 57              <1>     push    edi ; ****
                         <1>
                         <1>     ; 12/04/2016
                         <1>     ;mov    ebx, [esp+12]
                         <1>     ; 25/07/2022
00009B9C 8B54240C        <1>     mov     edx, [esp+12]
                         <1>
                         <1> set_env_chk_validation2w:
00009BA0 89F7            <1>     mov     edi, esi
00009BA2 4F              <1>     dec     edi
                         <1>
00009BA3 807FFF20        <1>     cmp     byte [edi-1], 20h
00009BA7 771A            <1>     ja      short set_env_chk_validation2z
                         <1>
00009BA9 56              <1>     push    esi
00009BAA 89FE            <1>     mov     esi, edi
00009BAC 4E              <1>     dec     esi
                         <1>
                         <1> set_env_chk_validation2x:
00009BAD 4E              <1>     dec     esi
                         <1>
                         <1>     ;cmp    esi, ebx
00009BAE 39D6            <1>     cmp     esi, edx ; 25/07/2022
00009BB0 7207            <1>     jb      short set_env_chk_validation2y
                         <1>
00009BB2 4F              <1>     dec     edi
                         <1>
00009BB3 8A06            <1>     mov     al, [esi]
00009BB5 8807            <1>     mov     [edi], al
                         <1>
00009BB7 EBF4            <1>     jmp     short set_env_chk_validation2x
                         <1>
                         <1> set_env_chk_validation2y:
00009BB9 5E              <1>     pop     esi
                         <1>
                         <1>     ;;mov   byte [ebx], 20h
                         <1>     ; 25/07/2022
                         <1>     ;mov    byte [edx], 20h
                         <1>
                         <1>     ;inc    ebx
```

```
                                <1>     ;mov    [esp+12], ebx
                                <1>     ; 25/07/2022
00009BBA 42                     <1>     inc     edx
00009BBB 8954240C               <1>     mov     [esp+12], edx
                                <1>
00009BBF FECC                   <1>     dec     ah ; 13/04/2016
                                <1>
00009BC1 EBDD                   <1>     jmp     short set_env_chk_validation2w
                                <1>
                                <1> set_env_chk_validation2z:
                                <1>     ;mov    edx, Env_Page
                                <1>     ;mov    edi, edx
                                <1>     ; 25/07/2022
00009BC3 BB00300900             <1>     mov     ebx, Env_Page
00009BC8 89DF                   <1>     mov     edi, ebx
                                <1>
                                <1> set_env_chk_validation3:
00009BCA AC                     <1>     lodsb
00009BCB 3C20                   <1>     cmp     al, 20h
00009BCD 74FB                   <1>     je      short set_env_chk_validation3
                                <1>
00009BCF 9C                     <1>     pushf
                                <1>
                                <1>     ; 12/04/2016
                                <1> set_env_chk_validation3n:
00009BD0 3C61                   <1>     cmp     al, 'a'
00009BD2 720C                   <1>     jb      short set_env_chk_validation3c
00009BD4 3C7A                   <1>     cmp     al, 'z'
00009BD6 7705                   <1>     ja      short set_env_chk_validation3x
00009BD8 2C20                   <1>     sub     al, 'a'-'A'
00009BDA 8846FF                 <1>     mov     [esi-1], al
                                <1>
                                <1> set_env_chk_validation3x:
00009BDD AC                     <1>     lodsb
00009BDE EBF0                   <1>     jmp     short set_env_chk_validation3n
                                <1>
                                <1> set_env_chk_validation3c:
00009BE0 3C20                   <1>     cmp     al, 20h
00009BE2 73F9                   <1>     jnb     short set_env_chk_validation3x
                                <1>
00009BE4 803F00                 <1>     cmp     byte [edi], 0
00009BE7 772B                   <1>     ja      short set_env_chk_validation4
                                <1>
00009BE9 9D                     <1>     popf
00009BEA 7222                   <1>     jb      short set_env_string_nothing
                                <1>
00009BEC B900020000             <1>     mov     ecx, Env_Page_Size ; 512 (4096)
                                <1>
                                <1>     ;mov    esi, ebx ; 12/04/2016
                                <1>     ; 25/07/2022
00009BF1 89D6                   <1>     mov     esi, edx
                                <1>
                                <1>     ; 25/07/2022
00009BF3 89CA                   <1>     mov     edx, ecx
                                <1>
                                <1> set_env_string_copy_to_envb:
00009BF5 AC                     <1>     lodsb
00009BF6 3C20                   <1>     cmp     al, 20h
00009BF8 7207                   <1>     jb      short set_env_string_copy_to_envb_z
00009BFA AA                     <1>     stosb
00009BFB E2F8                   <1>     loop    set_env_string_copy_to_envb
                                <1>
                                <1>     ; 11/04/2016
                                <1>     ;mov    edi, edx ; Env_Page
                                <1>     ; 25/07/2022
00009BFD 89DF                   <1>     mov     edi, ebx
                                <1>     ; 25/07/2022
                                <1>     ;mov    ecx, Env_Page_Size
00009BFF 89D1                   <1>     mov     ecx, edx
                                <1>
                                <1> set_env_string_copy_to_envb_z:
                                <1>     ; 25/07/2022
                                <1>     ;push   edx  ; Start address of the variable
                                <1>
                                <1>     ;;mov   edx, Env_Page_Size
                                <1>     ;; 25/07/2022
00009C01 29CA                   <1>     sub     edx, ecx ; variable (string) length
                                <1>
00009C03 28C0                   <1>     sub     al, al ; 0
00009C05 F3AA                   <1>     rep     stosb ; clear remain bytes of the env page
                                <1>
                                <1>     ;pop    eax  ; Start address of the variable
                                <1>     ; 25/07/2022
00009C07 89D8                   <1>     mov     eax, ebx
                                <1>
                                <1> set_env_string_allocate_envb_retn:  ; stc or clc return
00009C09 5F                     <1>     pop     edi ; ****
00009C0A 5B                     <1>     pop     ebx ; ***
00009C0B 59                     <1>     pop     ecx ; **
00009C0C 5E                     <1>     pop     esi ; *
00009C0D C3                     <1>     retn
                                <1>
                                <1> set_env_string_nothing:
00009C0E 31C0                   <1>     xor     eax, eax
00009C10 31D2                   <1>     xor     edx, edx ; 11/04/2016
00009C12 EBF5                   <1>     jmp     short set_env_string_allocate_envb_retn
                                <1>
                                <1> set_env_chk_validation4:
                                <1>     ; 11/04/2016
00009C14 9D                     <1>     popf
                                <1>
                                <1>     ;mov    esi, edx  ; Env_Page
                                <1>     ; 25/07/2022
00009C15 89DE                   <1>     mov     esi, ebx
                                <1>
                                <1> set_env_chk_validation5:
                                <1>     ;mov    edi, ebx ; ASCIIZ environment string address
                                <1>     ; 25/07/2022
00009C17 89D7                   <1>     mov     edi, edx
00009C19 0FB6CC                 <1>     movzx   ecx, ah ; Variable (string) length (with '=')
                                <1>
                                <1> set_env_chk_validation5_loop:
00009C1C AC                     <1>     lodsb
00009C1D AE                     <1>     scasb
00009C1E 7508                   <1>     jne     short set_env_chk_validation6
00009C20 E2FA                   <1>     loop    set_env_chk_validation5_loop
                                <1>
00009C22 3C3D                   <1>     cmp     al, '='
                                <1>         ;je set_env_change_variable
                                <1>     ; 25/07/2022
00009C24 7502                   <1>     jne     short set_env_chk_validation6
00009C26 EB7E                   <1>     jmp     set_env_change_variable
                                <1>
                                <1> set_env_chk_validation6:
```

```
00009C28 08C0                <1>     or      al, al ; 0
00009C2A 7403                <1>     jz      short set_env_chk_validation7
                             <1>
00009C2C AC                  <1>     lodsb
00009C2D EBF9                <1>     jmp     short set_env_chk_validation6
                             <1>
                             <1> set_env_chk_validation7:
00009C2F 88E1                <1>     mov     cl, ah
00009C31 01F1                <1>     add     ecx, esi
00009C33 81F9FF310900        <1>     cmp     ecx, Env_Page + Env_Page_Size - 1
                             <1>             ; 511 (4095)
                             <1>             ; strlen + '=' + 0
00009C39 72DC                <1>     jb      short set_env_chk_validation5
                             <1>
                             <1> set_env_chk_validation8: ; variable not found
00009C3B 0FB6F4              <1>     movzx   esi, ah  ; variable name length (with '=')
                             <1>     ;add    esi, ebx ; position just after of the '='
                             <1>     ; 25/07/2022
00009C3E 01D6                <1>     add     esi, edx
                             <1>
                             <1> set_env_chk_validation8_loop:
00009C40 AC                  <1>     lodsb
00009C41 3C20                <1>     cmp     al, 20h
00009C43 74FB                <1>     je      short set_env_chk_validation8_loop
00009C45 72C7                <1>     jb      short set_env_string_nothing
                             <1>
                             <1> set_env_chk_validation9:
00009C47 AC                  <1>     lodsb
00009C48 3C20                <1>     cmp     al, 20h
00009C4A 73FB                <1>     jnb     short set_env_chk_validation9
                             <1>
                             <1>     ; End of ASCIIZ environment string
                             <1>
                             <1> set_env_add_variable:
                             <1>     ;sub    esi, ebx ; variable+definition length
                             <1>     ; 25/07/2022
00009C4C 29D6                <1>     sub     esi, edx
                             <1>
00009C4E 56                  <1>     push    esi ; *****
                             <1>
                             <1>     ;mov    esi, edx ; Environment page address
                             <1>     ; 25/07/2022
00009C4F 89DE                <1>     mov     esi, ebx
                             <1>
00009C51 B900020000          <1>     mov     ecx, Env_Page_Size ; 512 (4096)
                             <1>
                             <1> set_env_add_variable_loop:
00009C56 AC                  <1>     lodsb
00009C57 20C0                <1>     and     al, al
00009C59 7406                <1>     jz      short set_env_add_variable_chk1 ; 0
00009C5B E2F9                <1>     loop    set_env_add_variable_loop
                             <1>
                             <1>     ; 11/04/2016
00009C5D 884EFF              <1>     mov     [esi-1], cl ; 0
00009C60 41                  <1>     inc     ecx
                             <1>
                             <1> set_env_add_variable_chk1:
00009C61 49                  <1>     dec     ecx
00009C62 7408                <1>     jz      short set_env_add_variable_nspc
00009C64 AC                  <1>     lodsb
00009C65 08C0                <1>     or      al, al
00009C67 740B                <1>     jz      short set_env_add_variable_chk2 ; 00
00009C69 49                  <1>     dec     ecx
00009C6A 75EA                <1>     jnz     short set_env_add_variable_loop
                             <1>
                             <1> set_env_add_variable_nspc: ; no space on environment page
00009C6C 58                  <1>     pop     eax ; *****
                             <1>     ;mov    eax, 8 ; No space for new environment string
                             <1>     ; 25/07/2022
00009C6D 29C0                <1>     sub     eax, eax
00009C6F B008                <1>     mov     al, 8
00009C71 F9                  <1>     stc
00009C72 EB95                <1>         jmp     short set_env_string_allocate_envb_retn
                             <1>
                             <1> set_env_add_variable_chk2:
00009C74 8B0C24              <1>     mov     ecx, [esp] ; *****
00009C77 4E                  <1>     dec     esi ; beginning address of the new variable
00009C78 89F0                <1>     mov     eax, esi
00009C7A 01C8                <1>     add     eax, ecx ; string length (with CR)
                             <1>     ;add    edx, Env_Page_Size ; 512 (4096)
                             <1>     ; 25/07/2022
00009C7C 81C300020000        <1>     add     ebx, Env_Page_Size
                             <1>     ;cmp    eax, edx
00009C82 39D8                <1>     cmp     eax, ebx ; 25/07/2022
00009C84 77E6                <1>     ja      short set_env_add_variable_nspc
00009C86 49                  <1>     dec     ecx ; except CR at the end
                             <1>     ;mov    edx, ecx ; 12/04/2016
                             <1>     ; 25/07/2022
00009C87 89CB                <1>     mov     ebx, ecx
00009C89 89F7                <1>     mov     edi, esi
00009C8B 893C24              <1>     mov     [esp], edi ; ***** ; Start address of new variable
                             <1>     ;mov    esi, ebx ; ASCIIZ environment string address
                             <1>     ; 25/07/2022
00009C8E 89D6                <1>     mov     esi, edx
00009C90 F3A4                <1>     rep     movsb
00009C92 28C0                <1>     sub     al, al
00009C94 AA                  <1>     stosb
00009C95 58                  <1>     pop     eax ; ***** ; Beginning address of new variable
00009C96 81FF00320900        <1>         cmp     edi, Env_Page + Env_Page_Size ; 12/04/2016
                             <1>     ;jnb    set_env_string_allocate_envb_retn ; OK !
                             <1>     ; 25/07/2022
00009C9C 7303                <1>     jnb     short set_env_add_variable_chk3
00009C9E 880F                <1>     mov     [edi], cl ; 0
00009CA0 F8                  <1>     clc     ; 13/04/2016
                             <1> set_env_add_variable_chk3:
00009CA1 E963FFFFFF          <1>         jmp     set_env_string_allocate_envb_retn ; OK !
                             <1>
                             <1> set_env_change_variable:
                             <1>     ; 06/04/2016
                             <1>     ; esi = Variable's address in environment page (after '=')
                             <1>     ; edi = ASCIIZ environment string address (after '=')
                             <1>
                             <1>     ; ah = variable length from start to the '='
00009CA6 8825[E2820100]      <1>     mov     [env_var_length], ah
                             <1>
00009CAC 28C9                <1>     sub     cl, cl ; ecx = 0
                             <1>
00009CAE 57                  <1>     push    edi ; *****
                             <1>
00009CAF 89F7                <1>     mov     edi, esi ; 11/04/2016
                             <1>
                             <1> set_env_change_variable_calc1:
00009CB1 AC                  <1>     lodsb
00009CB2 08C0                <1>     or      al, al
```

```
00009CB4 7403                <1>     jz      short set_env_change_variable_calc2
                             <1>
00009CB6 41                  <1>     inc     ecx ; length of environment string (after the '=')
                             <1>
00009CB7 EBF8                <1>     jmp     short set_env_change_variable_calc1
                             <1>
                             <1> set_env_change_variable_calc2:
00009CB9 8B3424              <1>     mov     esi, [esp] ; ASCIIZ environment string address
                             <1>
00009CBC 29D2                <1>     sub     edx, edx
                             <1>
                             <1> set_env_change_variable_calc3:
00009CBE AC                  <1>     lodsb
00009CBF 3C20                <1>     cmp     al, 20h
00009CC1 7203                <1>     jb      short set_env_change_variable_calc4
                             <1>
00009CC3 42                  <1>     inc     edx ; length of ASCIIZ string (after the '=')
                             <1>
00009CC4 EBF8                <1>     jmp     short set_env_change_variable_calc3
                             <1>
                             <1> set_env_change_variable_calc4:
00009CC6 C646FF00            <1>     mov     byte [esi-1], 0  ; put ZERO instead of CR
                             <1>
00009CCA 5E                  <1>     pop     esi ; ***** ; ASCIIZ string address (after '=')
                             <1>
                             <1>     ; EDI = Old variable's address (after '=')
                             <1>
                             <1>     ; compare the new string with the old string
00009CCB 39CA                <1>     cmp     edx, ecx
00009CCD 7718                <1>     ja      short set_env_change_variable_calc5 ; longer
                             <1>     ;jb     set_env_change_variable_calc9 ; shorter
                             <1>     ; 25/07/2022
00009CCF 7405                <1>     je      short set_env_change_variable_calc22
00009CD1 E98C000000          <1>     jmp     set_env_change_variable_calc9
                             <1>
                             <1> set_env_change_variable_calc22:
                             <1>     ;same length (simple copy)
00009CD6 0FB6C4              <1>     movzx   eax, ah
00009CD9 01C2                <1>     add     edx, eax
00009CDB F7D8                <1>     neg     eax
00009CDD 01F8                <1>     add     eax, edi
                             <1>     ; EAX = Start address of the variable
                             <1>     ; EDX = Variable length (without ZERO at the end of variable)
                             <1>
00009CDF F3A4                <1>     rep     movsb
00009CE1 F8                  <1>     clc     ; 13/04/2016
00009CE2 E922FFFFFF          <1>         jmp     set_env_string_allocate_envb_retn ; OK !
                             <1>
                             <1> set_env_change_variable_calc5:
                             <1>     ; 11/04/2016
00009CE7 52                  <1>     push    edx ; *****
00009CE8 29CA                <1>     sub     edx, ecx ; difference ; (the new string is longer)
00009CEA 89F3                <1>     mov     ebx, esi
00009CEC 89FE                <1>     mov     esi, edi
                             <1>
                             <1> set_env_change_variable_calc6:
00009CEE AC                  <1>     lodsb
00009CEF 20C0                <1>     and     al, al
00009CF1 75FB                <1>     jnz     short set_env_change_variable_calc6
                             <1>
00009CF3 81FE00320900        <1>     cmp     esi, Env_Page + Env_Page_Size ; 512 (4096)
00009CF9 0F836DFFFFFF        <1>         jnb     set_env_add_variable_nspc
                             <1>
00009CFF 89F9                <1>     mov     ecx, edi  ; current (old) variable's address
00009D01 89F7                <1>     mov     edi, esi  ; next variable's address
                             <1>
00009D03 AC                  <1>     lodsb
00009D04 08C0                <1>     or      al, al
00009D06 7417                <1>     jz      short set_env_change_variable_calc8 ; 00
                             <1>
                             <1> set_env_change_variable_calc7:
00009D08 AC                  <1>     lodsb
00009D09 20C0                <1>     and     al, al
00009D0B 75FB                <1>     jnz     short set_env_change_variable_calc7
                             <1>
00009D0D 81FE00320900        <1>     cmp     esi, Env_Page + Env_Page_Size ; 512 (4096)
                             <1>     ;jnb    set_env_add_variable_nspc
                             <1>     ; 25/07/2022
00009D13 7205                <1>     jb      short set_env_change_variable_calc24
                             <1> set_env_change_variable_calc23:
00009D15 E952FFFFFF          <1>     jmp     set_env_add_variable_nspc
                             <1>
                             <1> set_env_change_variable_calc24:
00009D1A AC                  <1>     lodsb
00009D1B 08C0                <1>     or      al, al
00009D1D 75E9                <1>     jnz     short set_env_change_variable_calc7
                             <1>
                             <1> set_env_change_variable_calc8:
00009D1F 4E                  <1>     dec     esi ; address of the second (last) 0 of the 00
                             <1>
00009D20 01F2                <1>     add     edx, esi ; final position of the last 0
                             <1>
00009D22 81FA00320900        <1>     cmp     edx, Env_Page + Env_Page_Size ; 512 (4096)
                             <1>     ;jnb     set_env_add_variable_nspc
                             <1>     ; 25/07/2022
00009D28 73EB                <1>     jnb     short set_env_change_variable_calc23
                             <1>
00009D2A 89C8                <1>     mov     eax, ecx ; old variable's address (after '=')
                             <1>
00009D2C 89F1                <1>     mov     ecx, esi
00009D2E 29F9                <1>     sub     ecx, edi ; count of bytes to move forward
                             <1>
                             <1>     ; 13/04/2016
00009D30 C60200              <1>     mov     byte [edx], 0
00009D33 89D7                <1>     mov     edi, edx
00009D35 29F2                <1>     sub     edx, esi ; difference (additional byte count)
00009D37 4F                  <1>     dec     edi ; the last zero address (first byte of the 00)
00009D38 89FE                <1>     mov     esi, edi
00009D3A 29D6                <1>     sub     esi, edx ; - displacement
                             <1>
00009D3C FA                  <1>     cli     ; disable interrupts
00009D3D FD                  <1>     std     ; backward
                             <1>
00009D3E F3A4                <1>     rep     movsb ; move ECX bytes from DS:ESI to ES:EDI
                             <1>
00009D40 FC                  <1>     cld     ; forward (default)
00009D41 FB                  <1>     sti     ; enable interrupts
                             <1>
00009D42 89C7                <1>     mov     edi, eax
00009D44 59                  <1>     pop     ecx ; ***** ; byte count (after '=')
00009D45 89CA                <1>     mov     edx, ecx
00009D47 89DE                <1>     mov     esi, ebx ; ASCIIZ string address (after '=')
00009D49 89FB                <1>     mov     ebx, edi
                             <1>
```

```
00009D4B F3A4                <1>     rep     movsb
                             <1>
00009D4D 880F                <1>     mov     [edi], cl ; 0 ; end of variable
                             <1>
00009D4F 0FB605[E2820100]    <1>     movzx   eax, byte [env_var_length]
00009D56 01C2                <1>     add     edx, eax ; variable length (total)
00009D58 F7D8                <1>     neg     eax
00009D5A 01D8                <1>     add     eax, ebx ; start address of the variable
00009D5C F8                  <1>     clc     ; 13/04/2016
00009D5D E9A7FEFFFF          <1>         jmp     set_env_string_allocate_envb_retn ; OK !
                             <1>
                             <1> set_env_change_variable_calc9:
                             <1>     ; 11/04/2016
00009D62 21D2                <1>     and     edx, edx ; is empty ?
00009D64 753B                <1>     jnz     short set_env_change_variable_calc15
                             <1>
00009D66 0FB6DC              <1>     movzx   ebx, ah
00009D69 F7DB                <1>     neg     ebx
00009D6B 01FB                <1>     add     ebx, edi
                             <1>
                             <1>     ; EBX = Start address of the variable (in env page)
                             <1>     ; EDX = Variable length = 0
                             <1>
00009D6D 89FE                <1>     mov     esi, edi
                             <1>
                             <1> set_env_change_variable_calc10:
00009D6F AC                  <1>     lodsb
00009D70 08C0                <1>     or      al, al
00009D72 75FB                <1>     jnz     short set_env_change_variable_calc10
                             <1>
00009D74 B9FF310900          <1>     mov     ecx, Env_Page + Env_Page_Size - 1
                             <1>
00009D79 39CE                <1>     cmp     esi, ecx ; +511 (+4095)
00009D7B 7604                <1>     jna     short set_env_change_variable_calc11
                             <1>
00009D7D 89CE                <1>     mov     esi, ecx
00009D7F 8806                <1>     mov     [esi], al ; 0
                             <1>
                             <1> set_env_change_variable_calc11:
00009D81 89DF                <1>     mov     edi, ebx ; old variable's start address
                             <1>
                             <1> set_env_change_variable_calc12:
00009D83 AC                  <1>     lodsb
00009D84 AA                  <1>     stosb
00009D85 20C0                <1>     and     al, al
00009D87 75FA                <1>     jnz     short set_env_change_variable_calc12
00009D89 39CE                <1>     cmp     esi, ecx
00009D8B 7706                <1>     ja      short set_env_change_variable_calc13
00009D8D AC                  <1>     lodsb
00009D8E AA                  <1>     stosb
00009D8F 20C0                <1>     and     al, al
00009D91 75F0                <1>     jnz     short set_env_change_variable_calc12
                             <1>
                             <1> set_env_change_variable_calc13:
00009D93 29F9                <1>     sub     ecx, edi
00009D95 7203                <1>     jb      short set_env_change_variable_calc14
00009D97 41                  <1>     inc     ecx ; 1-512 (1-4096)
00009D98 F3AA                <1>     rep     stosb ; al = 0
                             <1>
                             <1> set_env_change_variable_calc14:
00009D9A 29C0                <1>     sub     eax, eax ; Start address of the variable
                             <1>     ; EAX = 0 -> Variable is removed
                             <1>     ; EDX = Variable length = 0
                             <1>
00009D9C E968FEFFFF          <1>         jmp     set_env_string_allocate_envb_retn ; OK !
                             <1>
                             <1> set_env_change_variable_calc15:
00009DA1 52                  <1>     push    edx ; *****
00009DA2 F7DA                <1>     neg     edx
00009DA4 01CA                <1>     add     edx, ecx ; difference (the old string is longer)
00009DA6 89F3                <1>     mov     ebx, esi
00009DA8 89FE                <1>     mov     esi, edi
                             <1>
                             <1> set_env_change_variable_calc16:
00009DAA AC                  <1>     lodsb
00009DAB 20C0                <1>     and     al, al
00009DAD 75FB                <1>     jnz     short set_env_change_variable_calc16
                             <1>
00009DAF B900320900          <1>     mov     ecx, Env_Page + Env_Page_Size
                             <1>
00009DB4 39CE                <1>     cmp     esi, ecx ; +512 (+4096)
00009DB6 7605                <1>     jna     short set_env_change_variable_calc17
                             <1>
00009DB8 89CE                <1>     mov     esi, ecx
00009DBA 8846FF              <1>     mov     [esi-1], al ; 0
                             <1>
                             <1> set_env_change_variable_calc17:
00009DBD 89F9                <1>     mov     ecx, edi  ; current (old) variable's address
00009DBF 89F7                <1>     mov     edi, esi  ; next variable's address
                             <1>
00009DC1 AC                  <1>     lodsb
00009DC2 08C0                <1>     or      al, al
00009DC4 741D                <1>     jz      short set_env_change_variable_calc20
                             <1>
                             <1> set_env_change_variable_calc18:
00009DC6 AC                  <1>     lodsb
00009DC7 20C0                <1>     and     al, al
00009DC9 75FB                <1>     jnz     short set_env_change_variable_calc18
                             <1>
00009DCB 81FE00320900        <1>     cmp     esi, Env_Page + Env_Page_Size
00009DD1 720B                <1>     jb      short set_env_change_variable_calc19
00009DD3 740E                <1>     je      short set_env_change_variable_calc20
                             <1>
00009DD5 BEFF310900          <1>     mov     esi, Env_Page + Env_Page_Size - 1
00009DDA 8806                <1>     mov     [esi], al ; 0
00009DDC EB06                <1>     jmp     short set_env_change_variable_calc21
                             <1>
                             <1> set_env_change_variable_calc19:
00009DDE AC                  <1>     lodsb
00009DDF 08C0                <1>     or      al, al
00009DE1 75E3                <1>     jnz     short set_env_change_variable_calc18
                             <1>
                             <1> set_env_change_variable_calc20:
00009DE3 4E                  <1>     dec     esi ; address of the second (last) 0 of the 00
                             <1>
                             <1> set_env_change_variable_calc21:
                             <1>     ; edx = difference (byte count)
                             <1>
00009DE4 89C8                <1>     mov     eax, ecx ; old variable's address (after '=')
                             <1>
00009DE6 89F1                <1>     mov     ecx, esi
00009DE8 29F9                <1>     sub     ecx, edi ; count of bytes to move backward
                             <1>
00009DEA 89FE                <1>     mov     esi, edi ; next variable's address
```

```
00009DEC 29D7                <1>     sub    edi, edx ; (displacement)
                             <1>
00009DEE F3A4                <1>     rep    movsb
                             <1>
00009DF0 880F                <1>     mov    [edi], cl ; 0 ; 00 ; end of environment variables
                             <1>
00009DF2 89C7                <1>     mov    edi, eax
00009DF4 5A                  <1>     pop    edx ; ***** ; byte count (after '=')
00009DF5 89D1                <1>     mov    ecx, edx
00009DF7 89DE                <1>     mov    esi, ebx ; ASCIIZ string address (after '=')
00009DF9 89FB                <1>     mov    ebx, edi
                             <1>
00009DFB F3A4                <1>     rep    movsb
                             <1>
00009DFD 880F                <1>     mov    [edi], cl ; 0 ; end of variable
                             <1>
00009DFF 0FB605[E2820100]    <1>     movzx  eax, byte [env_var_length]
00009E06 01C2                <1>     add    edx, eax ; variable length (total)
00009E08 F7D8                <1>     neg    eax
00009E0A 01D8                <1>     add    eax, ebx ; start address of the variable
00009E0C F8                  <1>     clc    ; 13/04/2016
00009E0D E9F7FDFFFF          <1>         jmp     set_env_string_allocate_envb_retn ; OK !
                             <1>
                             <1>     ; 19/12/2025
                             <1> loc_load_mainprog_cfg_exit:
00009E12 C3                  <1>     retn
                             <1>
                             <1> mainprog_startup_configuration:
                             <1>     ; 19/12/2025 - TRDOS 386 Kernel v2.10.10
                             <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 22/11/2017
                             <1>     ; 06/05/2016
                             <1>     ; 14/04/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 17/09/2011 (TRDOS v1, MAINPROG.ASM)
                             <1>     ;
                             <1> loc_load_mainprog_cfg_file:
00009E13 BE[DD2E0100]        <1>     mov    esi, MainProgCfgFile
00009E18 66B80018            <1>     mov    ax, 1800h ; Except volume label and dirs
00009E1C E859EBFFFF          <1>     call   find_first_file
00009E21 72EF                <1>     jc     short loc_load_mainprog_cfg_exit
                             <1>
                             <1>     ;or    eax, eax
                             <1>     ;jz    short loc_load_mainprog_cfg_exit
                             <1>
                             <1> loc_start_mainprog_configuration:
                             <1>     ; ESI = FindFile_DirEntry Location
                             <1>     ; EAX = File Size
                             <1>
00009E23 A3[64770100]        <1>     mov    [MainProgCfg_FileSize], eax
                             <1>
00009E28 668B5614            <1>     mov    dx, [esi+DirEntry_FstClusHI]
00009E2C C1E210              <1>     shl    edx, 16
00009E2F 668B561A            <1>     mov    dx, [esi+DirEntry_FstClusLO]
                             <1>     ;mov   [csftdf_sf_cluster], edx
                             <1>     ; 19/12/2025
00009E33 8915[98820100]      <1>     mov    [MainProgCfg_cluster], edx
                             <1>
00009E39 89C1                <1>     mov    ecx, eax
00009E3B 29C0                <1>     sub    eax, eax
                             <1>
                             <1>     ; TRDOS 386 (TRDOS v2.0)
                             <1>     ; Allocate contiguous memory block for loading the file
                             <1>
                             <1>     ; eax = 0 (Allocate memory from the beginning)
                             <1>     ; ecx = File (Allocation) size in bytes
                             <1>
00009E3D E803BFFFFF          <1>     call   allocate_memory_block
00009E42 72CE                <1>     jc     short loc_load_mainprog_cfg_exit
                             <1>
                             <1>     ;mov   [csftdf_sf_mem_addr], eax ; loading address
                             <1>     ;mov   [csftdf_sf_mem_bsize], ecx ; block size
                             <1>     ; 19/12/2025
00009E44 A3[90820100]        <1>     mov    [MainProgCfg_mem_addr], eax ; loading address
00009E49 890D[94820100]      <1>     mov    [MainProgCfg_mem_bsize], ecx ; block size
                             <1>
00009E4F 31DB                <1>     xor    ebx, ebx
                             <1>     ;;mov  [csftdf_sf_rbytes], ebx ; 0, reset
                             <1>     ; 19/12/2025
                             <1>     ;mov   [MainProgCfg_rbytes], ebx ; 0, reset
                             <1>
00009E51 8A3D[76770100]      <1>     mov    bh, [Current_Drv] ; [FindFile_Drv]
00009E57 BE00010900          <1>     mov    esi, Logical_DOSDisks
00009E5C 01DE                <1>     add    esi, ebx
                             <1>
                             <1>     ;mov   ebx, [csftdf_sf_mem_addr] ; memory block address
                             <1>     ; 19/12/2025
00009E5E 8B1D[90820100]      <1>     mov    ebx, [MainProgCfg_mem_addr]
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp    byte [esi+LD_FATType], 0
                             <1>         ja  short loc_mcfg_load_fat_file
                             <1>
                             <1>     ;mov   dword [csftdf_r_size], 65536
                             <1>     ; 19/12/2025
                             <1>     mov    dword [MainProgCfg_r_size], 65536
                             <1>         jmp     loc_mcfg_load_fs_file
                             <1>
                             <1> loc_load_mainprog_cfg_exit:
                             <1>     retn
                             <1> %endif
                             <1>
                             <1> loc_mcfg_load_fat_file:
                             <1>     ;movzx eax, word [esi+LD_BPB+BytesPerSec]
                             <1>     ; 19/12/2025
00009E64 B800020000          <1>     mov    eax, 512
00009E69 0FB64E13            <1>     movzx  ecx, byte [esi+LD_BPB+SecPerClust]
00009E6D F7E1                <1>     mul    ecx
                             <1>     ;mov   [csftdf_r_size], eax
                             <1>     ; 19/12/2025
00009E6F A3[A0820100]        <1>     mov    [MainProgCfg_r_size], eax
                             <1>
                             <1> loc_mcfg_load_fat_file_next:
00009E74 E817010000          <1>     call   mcfg_read_fat_file_sectors
00009E79 7259                <1>         jc  short mcfg_deallocate_mem ; 25/07/2022
                             <1>
00009E7B 09D2                <1>     or     edx, edx ; edx > 0 -> EOF
00009E7D 74F5                <1>     jz     short loc_mcfg_load_fat_file_next
                             <1>
                             <1> loc_mcfg_load_fat_file_ok:
                             <1>     ; 06/05/2016
00009E7F C705[34830100]-     <1>     mov    dword [mainprog_return_addr], loc_mcfg_ci_return_addr
00009E85 [519F0000]          <1>
                             <1>     ;
```

```
                                <1>     ;mov    esi, [csftdf_sf_mem_addr]
                                <1>     ; 19/12/2025
00009E89 8B35[90820100]         <1>     mov     esi, [MainProgCfg_mem_addr]
00009E8F 8935[68770100]         <1>     mov     [MainProgCfg_LineOffset], esi
                                <1>
00009E95 A1[64770100]           <1>     mov     eax, [MainProgCfg_FileSize]
00009E9A 89C2                   <1>     mov     edx, eax
00009E9C 01F2                   <1>     add     edx, esi
                                <1>
                                <1> loc_mcfg_process_next_line_check:
00009E9E 89C1                   <1>     mov     ecx, eax
                                <1>
00009EA0 803E2A                 <1>     cmp     byte [esi], "*" ; Remark sign
00009EA3 7503                   <1>     jne     short loc_mcfg_process_next_line
00009EA5 46                     <1>     inc     esi
00009EA6 EB16                   <1>     jmp     short loc_move_mainprog_cfg_nl1
                                <1>
                                <1> loc_mcfg_process_next_line:
00009EA8 83F94F                 <1>     cmp     ecx, 79
00009EAB 7604                   <1>     jna     short loc_start_mainprog_cfg_process
                                <1>
                                <1>     ;mov    ecx, 79
                                <1>     ; 25/07/2022
00009EAD 29C9                   <1>     sub     ecx, ecx
00009EAF B14F                   <1>     mov     cl, 79
                                <1>
                                <1> loc_start_mainprog_cfg_process:
00009EB1 BF[26780100]           <1>     mov     edi, CommandBuffer
                                <1>
                                <1> loc_move_mainprog_cfg_line:
00009EB6 AC                     <1>     lodsb
00009EB7 3C20                   <1>     cmp     al, 20h
00009EB9 720C                   <1>     jb      short loc_move_mainprog_cfg_nl2
00009EBB AA                     <1>     stosb
00009EBC E2F8                   <1>     loop    loc_move_mainprog_cfg_line
                                <1>
                                <1> loc_move_mainprog_cfg_nl1:
00009EBE 39D6                   <1>     cmp     esi, edx ; + configuration file size
00009EC0 7322                   <1>     jnb     short loc_end_of_mainprog_cfg_line
00009EC2 AC                     <1>     lodsb
00009EC3 3C20                   <1>     cmp     al, 20h
00009EC5 73F7                   <1>     jnb     short loc_move_mainprog_cfg_nl1
                                <1>
                                <1> loc_move_mainprog_cfg_nl2:
00009EC7 39D6                   <1>     cmp     esi, edx
00009EC9 7319                   <1>     jnb     short loc_end_of_mainprog_cfg_line
00009ECB 8A06                   <1>     mov     al, [esi]
00009ECD 3C20                   <1>     cmp     al, 20h
00009ECF 7713                   <1>     ja      short loc_end_of_mainprog_cfg_line
00009ED1 46                     <1>     inc     esi
00009ED2 EBF3                   <1>     jmp     short loc_move_mainprog_cfg_nl2
                                <1>
                                <1>     ; 25/07/2022
                                <1> mcfg_deallocate_mem:
                                <1>     ;mov    eax, [csftdf_sf_mem_addr] ; start address
                                <1>     ;mov    ecx, [csftdf_sf_mem_bsize] ; block size
                                <1>     ; 19/12/2025
00009ED4 A1[90820100]           <1>     mov     eax, [MainProgCfg_mem_addr] ; start address
00009ED9 8B0D[94820100]         <1>     mov     ecx, [MainProgCfg_mem_bsize] ; block size
                                <1>
                                <1>     ;call   deallocate_memory_block
                                <1>     ;retn
00009EDF E97BC0FFFF             <1>     jmp     deallocate_memory_block
                                <1>
                                <1> loc_end_of_mainprog_cfg_line:
00009EE4 C60700                 <1>     mov     byte [edi], 0
                                <1>
00009EE7 8935[68770100]         <1>     mov     [MainProgCfg_LineOffset], esi
                                <1>
                                <1>     ; 22/11/2017
00009EED BE[2E780100]           <1>     mov     esi, CommandBuffer + 8
00009EF2 29FE                   <1>     sub     esi, edi
00009EF4 7606                   <1>     jna     short loc_move_mainprog_cfg_command
00009EF6 30C0                   <1>     xor     al, al
                                <1> loc_mainprog_cfg_clear_chrs:
00009EF8 AA                     <1>     stosb
00009EF9 4E                     <1>     dec     esi
00009EFA 75FC                   <1>     jnz     short loc_mainprog_cfg_clear_chrs
                                <1>
                                <1> loc_move_mainprog_cfg_command:
00009EFC BE[26780100]           <1>     mov     esi, CommandBuffer
00009F01 89F7                   <1>     mov     edi, esi
00009F03 31DB                   <1>     xor     ebx, ebx
                                <1>     ;xor    ecx, ecx
00009F05 30C9                   <1>     xor     cl, cl
                                <1>
                                <1> loc_move_mcfg_first_cmd_char:
00009F07 8A041E                 <1>     mov     al, [esi+ebx]
00009F0A FEC3                   <1>     inc     bl
00009F0C 3C20                   <1>     cmp     al, 20h
00009F0E 7712                   <1>     ja      short loc_move_mcfg_cmd_capitalizing
00009F10 7237                   <1>     jb      short loc_move_mcfg_cmd_arguments_ok
00009F12 80FB4F                 <1>     cmp     bl, 79
00009F15 72F0                   <1>     jb      short loc_move_mcfg_first_cmd_char
00009F17 EB30                   <1>     jmp     short loc_move_mcfg_cmd_arguments_ok
                                <1>
                                <1> loc_move_mcfg_next_cmd_char:
00009F19 8A041E                 <1>     mov     al, [esi+ebx]
00009F1C FEC3                   <1>     inc     bl
00009F1E 3C20                   <1>     cmp     al, 20h
00009F20 7614                   <1>     jna     short loc_move_mcfg_cmd_ok
                                <1>
                                <1> loc_move_mcfg_cmd_capitalizing:
00009F22 3C61                   <1>     cmp     al, 61h ; 'a'
00009F24 7206                   <1>     jb      short loc_move_mcfg_cmd_caps_ok
00009F26 3C7A                   <1>     cmp     al, 7Ah ; 'z'
00009F28 7702                   <1>     ja      short loc_move_mcfg_cmd_caps_ok
00009F2A 24DF                   <1>     and     al, 0DFh ; sub al, 'a'-'A'
                                <1>
                                <1> loc_move_mcfg_cmd_caps_ok:
00009F2C AA                     <1>     stosb
00009F2D FEC1                   <1>     inc     cl
00009F2F 80FB4F                 <1>     cmp     bl, 79
00009F32 72E5                   <1>     jb      short loc_move_mcfg_next_cmd_char
00009F34 EB13                   <1>     jmp     short loc_move_mcfg_cmd_arguments_ok
                                <1>
                                <1> loc_move_mcfg_cmd_ok:
00009F36 30C0                   <1>     xor     al, al ; 0
                                <1>
                                <1> loc_move_mcfg_cmd_arguments:
00009F38 8807                   <1>     mov     [edi], al
00009F3A 47                     <1>     inc     edi
00009F3B 80FB4F                 <1>     cmp     bl, 79
00009F3E 7309                   <1>     jnb     short loc_move_mcfg_cmd_arguments_ok
```

```
00009F40 8A041E              <1>     mov     al, [esi+ebx]
00009F43 FEC3                <1>     inc     bl
00009F45 3C20                <1>     cmp     al, 20h
00009F47 73EF                <1>     jnb     short loc_move_mcfg_cmd_arguments
                             <1>
                             <1> loc_move_mcfg_cmd_arguments_ok:
00009F49 C60700              <1>     mov     byte [edi], 0
                             <1>
                             <1> loc_mcfg_process_cmd_interpreter:
00009F4C E828E1FFFF          <1>     call    command_interpreter
                             <1>
                             <1> loc_mcfg_ci_return_addr:
00009F51 A1[64770100]        <1>     mov     eax, [MainProgCfg_FileSize]
00009F56 89C2                <1>     mov     edx, eax
00009F58 8B35[68770100]      <1>     mov     esi, [MainProgCfg_LineOffset]
00009F5E 01F2                <1>     add     edx, esi
                             <1>     ;add    eax, [csftdf_sf_mem_addr]
                             <1>     ; 19/12/2025
00009F60 0305[90820100]      <1>     add     eax, [MainProgCfg_mem_addr]
00009F66 29F0                <1>     sub     eax, esi
00009F68 0F8730FFFFFF        <1>         ja      loc_mcfg_process_next_line_check
                             <1>
00009F6E E861FFFFFF          <1>     call    mcfg_deallocate_mem
                             <1>
00009F73 B94F000000          <1>     mov     ecx, 79 ; 80 ?
00009F78 BF[26780100]        <1>     mov     edi, CommandBuffer
00009F7D 30C0                <1>     xor     al, al
00009F7F F3AA                <1>     rep     stosb
                             <1>
                             <1>     ; 06/05/2016
00009F81 BE[17390100]        <1>     mov     esi, nextline
00009F86 E82FCEFFFF          <1>     call    print_msg
00009F8B E98FD8FFFF          <1>     jmp     dos_prompt
                             <1>
                             <1>     ; 05/12/2025
                             <1> ;mcfg_read_file_sectors:
                             <1>     ; 14/04/2016
                             <1>     ;cmp    byte [esi+LD_FATType], 0
                             <1>         ;jna        short mcfg_read_fs_file_sectors
                             <1>
                             <1> mcfg_read_fat_file_sectors:
                             <1>     ; return:
                             <1>     ;   CF = 0 & EDX > 0 -> END OF FILE
                             <1>     ;   CF = 0 & EDX = 0 -> not EOF
                             <1>     ;   CF = 1 -> read error (error code in AL)
                             <1>
                             <1> mcfg_read_fat_file_secs_0:
00009F90 8B15[64770100]      <1>     mov     edx, [MainProgCfg_FileSize]
                             <1>     ;sub    edx, [csftdf_sf_rbytes]
                             <1>     ; 19/12/2025
00009F96 2B15[A8820100]      <1>     sub     edx, [MainProgCfg_rbytes]
                             <1>     ;cmp    edx, [csftdf_r_size]
00009F9C 3B15[A0820100]      <1>     cmp     edx, [MainProgCfg_r_size]
00009FA2 7306                <1>     jnb     short mcfg_read_fat_file_secs_1
                             <1>     ;mov    [csftdf_r_size], edx
                             <1>     ; 19/12/2025
00009FA4 8915[A0820100]      <1>     mov     [MainProgCfg_r_size], edx
                             <1>
                             <1> mcfg_read_fat_file_secs_1:
                             <1>     ;mov    eax, [csftdf_r_size]
                             <1>     ; 19/12/2025
00009FAA A1[A0820100]        <1>     mov     eax, [MainProgCfg_r_size]
00009FAF 29D2                <1>     sub     edx, edx
                             <1>     ;movzx  ecx, word [esi+LD_BPB+BytesPerSec]
                             <1>     ; 19/12/2025
00009FB1 B900020000          <1>     mov     ecx, 512
00009FB6 01C8                <1>     add     eax, ecx
00009FB8 48                  <1>     dec     eax
00009FB9 F7F1                <1>     div     ecx
00009FBB 89C1                <1>     mov     ecx, eax ; sector count
                             <1>     ;mov    eax, [csftdf_sf_cluster]
                             <1>     ; 19/12/2025
00009FBD A1[98820100]        <1>     mov     eax, [MainProgCfg_cluster]
                             <1>
                             <1>     ; EBX = memory block address (current)
                             <1>
00009FC2 E8B8210000          <1>     call    read_fat_file_sectors
00009FC7 7230                <1>     jc      short mcfg_read_fat_file_secs_3
                             <1>
                             <1>     ; EBX = next memory address
                             <1>
                             <1>     ;mov    eax, [csftdf_sf_rbytes]
                             <1>     ; 19/12/2025
00009FC9 A1[A8820100]        <1>     mov     eax, [MainProgCfg_rbytes]
                             <1>     ;add    eax, [csftdf_r_size]
00009FCE 0305[A0820100]      <1>     add     eax, [MainProgCfg_r_size]
00009FD4 8B15[64770100]      <1>     mov     edx, [MainProgCfg_FileSize]
00009FDA 39D0                <1>     cmp     eax, edx
00009FDC 731B                <1>     jnb     short mcfg_read_fat_file_secs_3 ; edx > 0
                             <1>     ;mov    [csftdf_sf_rbytes], eax
                             <1>     ; 19/12/2025
00009FDE A3[A8820100]        <1>     mov     [MainProgCfg_rbytes], eax
                             <1>
00009FE3 53                  <1>     push    ebx ; *
                             <1>     ; get next cluster (csftdf_r_size! bytes)
                             <1>     ;mov    eax, [csftdf_sf_cluster]
                             <1>     ; 19/12/2025
00009FE4 A1[98820100]        <1>     mov     eax, [MainProgCfg_cluster]
00009FE9 E89B1F0000          <1>     call    get_next_cluster
00009FEE 5B                  <1>     pop     ebx ; *
00009FEF 7301                <1>     jnc     short mcfg_read_fat_file_secs_2
                             <1>
                             <1>     ;mov    eax, 17; Read error !
00009FF1 C3                  <1>     retn
                             <1>
                             <1> mcfg_read_fat_file_secs_2:
00009FF2 29D2                <1>     sub     edx, edx ; 0
                             <1>     ;mov    [csftdf_sf_cluster], eax ; next cluster
                             <1>     ; 19/12/2025
00009FF4 A3[98820100]        <1>     mov     [MainProgCfg_cluster], eax
                             <1>
                             <1> ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>
                             <1> mcfg_read_fat_file_secs_3:
                             <1> ; 19/12/2025
                             <1> ;   ;retn
                             <1> ;
                             <1> ;mcfg_read_fs_file_sectors:
                             <1> ;   ;retn
                             <1> ;
                             <1> ;loc_mcfg_load_fs_file:
00009FF9 C3                  <1>     retn
                             <1>
                             <1> load_and_execute_file:
```

```
                                <1>     ; 02/12/2025
                                <1>     ; 04/07/2025
                                <1>     ; 03/07/2025 - TRDOS 386 Kernel v2.0.10
                                <1>     ; 03/09/2024 - TRDOS 386 Kernel v2.0.9
                                <1>     ; 30/08/2023 - TRDOS 386 Kernel v2.0.6
                                <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 04/01/2017
                                <1>     ; 06/05/2016 - 07/05/2016 - 11/05/2016
                                <1>     ; 23/04/2016 - 24/04/2016
                                <1>     ; 22/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 05/11/2011
                                <1>     ; (TRDOS v1, CMDINTR.ASM, 'cmp_cmd_run', 'cmp_cmd_external')
                                <1>     ; ('loc_run_check_filename')
                                <1>     ; 29/08/2011
                                <1>     ; 10/09/2011
                                <1>     ; INPUT->
                                <1>     ;       ESI = Path Name address (CommandBuffer address)
                                <1>     ; OUTPUT ->
                                <1>     ;       none (error message will be shown if an error will occur)
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, ESI, EDI, EBP will be changed)
                                <1>     ;
                                <1> loc_run_check_filename:
00009FFA 803E20                 <1>     cmp     byte [esi], 20h
                                <1>     ;jb     loc_cmd_failed
                                <1>     ; 25/07/2022
00009FFD 7237                   <1>     jb      short loc_run_ppn_failed
00009FFF 7703                   <1>     ja      short loc_run_check_filename_ok
0000A001 46                     <1>     inc     esi
0000A002 EBF6                   <1>     jmp     short loc_run_check_filename
                                <1>
                                <1> loc_run_check_filename_ok:
0000A004 C605[D7770100]00       <1>     mov     byte [CmdArgStart], 0 ; reset
0000A00B 56                     <1>     push    esi ; *
                                <1> loc_run_get_first_arg_pos:
0000A00C 46                     <1>     inc     esi
0000A00D 8A06                   <1>     mov     al, [esi]
0000A00F 3C20                   <1>     cmp     al, 20h
0000A011 77F9                   <1>     ja      short loc_run_get_first_arg_pos
0000A013 C60600                 <1>     mov     byte [esi], 0
                                <1> loc_run_get_external_arg_pos:
                                <1>     ; 11/05/2016
0000A016 46                     <1>     inc     esi
0000A017 8A06                   <1>     mov     al, [esi]
0000A019 3C20                   <1>     cmp     al, 20h
0000A01B 760C                   <1>     jna     short loc_run_parse_path_name
0000A01D 89F0                   <1>     mov     eax, esi
0000A01F 2D[26780100]           <1>     sub     eax, CommandBuffer
0000A024 A2[D7770100]           <1>     mov     byte [CmdArgStart], al
                                <1> loc_run_parse_path_name:
0000A029 5E                     <1>     pop     esi ; *
0000A02A BF[16800100]           <1>     mov     edi, FindFile_Drv
0000A02F E84A090000             <1>     call    parse_path_name
                                <1>     ;jc     loc_cmd_failed
                                <1>     ; 25/07/2022
0000A034 7305                   <1>     jnc     short loc_run_check_filename_exists
                                <1> loc_run_ppn_failed:
0000A036 E919E4FFFF             <1>     jmp     loc_cmd_failed
                                <1>
                                <1> loc_run_check_filename_exists:
0000A03B BE[58800100]           <1>     mov     esi, FindFile_Name
0000A040 803E20                 <1>     cmp     byte [esi], 20h
                                <1>     ;jna    loc_cmd_failed
                                <1>     ; 25/07/2022
0000A043 76F1                   <1>     jna     short loc_run_ppn_failed
                                <1>
                                <1> loc_run_check_exe_filename_ext:
0000A045 E896020000             <1>     call    check_prg_filename_ext
                                <1>     ;jc     loc_cmd_failed
                                <1>     ; 25/07/2022
0000A04A 72EA                   <1>     jc      short loc_run_ppn_failed
                                <1>     ; 03/07/2025
                                <1>     ; if al = 0 -> ah = file name length
                                <1>
                                <1> loc_run_check_exe_filename_ext_ok:
0000A04C 66A3[31830100]         <1>     mov     word [EXE_ID], ax ; 'P.' or 'P'+0
                                <1>
                                <1> loc_run_drv:
0000A052 C605[30830100]00       <1>     mov     byte [Run_Manual_Path], 0
0000A059 A1[70770100]           <1>     mov     eax, [Current_Dir_FCluster]
0000A05E A3[24830100]           <1>         mov     [Run_CDirFC], eax
                                <1>     ;
0000A063 8A35[76770100]         <1>     mov     dh, [Current_Drv]
0000A069 8835[D37E0100]         <1>     mov     [RUN_CDRV], dh
                                <1>
0000A06F 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
0000A075 38F2                   <1>     cmp     dl, dh
0000A077 7413                   <1>     je      short loc_run_change_directory
                                <1>
0000A079 8005[30830100]02       <1>     add     byte [Run_Manual_Path], 2
                                <1>
0000A080 E86BD6FFFF             <1>     call    change_current_drive
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 25/07/2022
0000A085 7305                   <1>     jnc     short loc_run_change_directory
0000A087 E9F3E3FFFF             <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1> loc_run_change_directory:
0000A08C 803D[17800100]20       <1>     cmp     byte [FindFile_Directory], 20h
0000A093 7624                   <1>     jna     short loc_run_find_executable_file
                                <1>
0000A095 FE05[30830100]         <1>     inc     byte [Run_Manual_Path]
                                <1>
0000A09B FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
                                <1>
0000A0A1 BE[17800100]           <1>     mov     esi, FindFile_Directory
0000A0A6 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000A0A8 E83E030000             <1>     call    change_current_directory
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 25/07/2022
0000A0AD 7305                   <1>     jnc     short loc_run_change_prompt_dir_string
0000A0AF E9CBE3FFFF             <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1> loc_run_change_prompt_dir_string:
0000A0B4 E85B020000             <1>     call    change_prompt_dir_string
                                <1>
                                <1> loc_run_find_executable_file:
0000A0B9 66C705[2C830100]00-    <1>     mov     word [Run_Auto_Path], 0
0000A0C1 00                     <1>
                                <1>
                                <1> loc_run_find_executable_file_next:
0000A0C2 BE[58800100]           <1>     mov     esi, FindFile_Name
                                <1> loc_run_find_program_file_next:
0000A0C7 66B80018               <1>     mov     ax, 1800h ; Except volume label and dirs
```

```
0000A0CB E8AAE8FFFF          <1>     call    find_first_file
                             <1>     ; ESI = Directory Entry (FindFile_DirEntry) Location
                             <1>     ; EDI = Directory Buffer Directory Entry Location
                             <1>     ; EAX = File size
                             <1>     ;jnc    loc_load_and_run_file
                             <1>     ; 25/07/2022
0000A0D0 7216                <1>     jc      short loc_run_program_file_not_found
                             <1>
                             <1>     ; 02/12/2025 (TRDOS 386 v2.0.10)
0000A0D2 8B471C              <1>     mov     eax, [edi+DirEntry_FileSize]
0000A0D5 21C0                <1>     and     eax, eax ; zero file size check?
0000A0D7 750A                <1>     jnz     short loc_load_and_run_file_@
0000A0D9 B814000000          <1>     mov     eax, ERR_FILE_SIZE
                             <1>             ; 20, 'file size error !'
                             <1>     ;mov    eax, ERR_ZERO_LENGTH
                             <1>             ; 20, 'zero length !' error
0000A0DE E99CE3FFFF          <1>     jmp     loc_run_cmd_failed
                             <1>
                             <1> loc_load_and_run_file_@:
0000A0E3 E957010000          <1>     jmp     loc_load_and_run_file
                             <1>
                             <1> loc_run_program_file_not_found:
0000A0E8 3C02                <1>     cmp     al, 2 ; file not found
                             <1>     ;jne    loc_run_cmd_failed
                             <1>     ; 25/07/2022
0000A0EA 7405                <1>     je      short loc_run_progr_file_chk_prg_ext
0000A0EC E98EE3FFFF          <1>     jmp     loc_run_cmd_failed
                             <1>
                             <1> loc_run_progr_file_chk_prg_ext: ; 25/07/2022
0000A0F1 66A1[31830100]      <1>     mov     ax, word [EXE_ID]
                             <1>
                             <1>     ;cmp    ah, '.' ; File name has extension sign
                             <1>     ;je     short loc_run_check_auto_path
                             <1>     ;or     al, al
                             <1>     ;jnz    short loc_run_check_auto_path
                             <1>     ; 03/07/2025
0000A0F7 663D502E            <1>     cmp     ax, 'P.'
0000A0FB 7420                <1>     je      short loc_run_check_auto_path
                             <1>
0000A0FD 80FC08              <1>     cmp     ah, 8 ; count of file name chars
0000A100 771B                <1>     ja      short loc_run_check_auto_path
                             <1>
                             <1> loc_run_change_file_ext_to_prg:
0000A102 0FB6DC              <1>     movzx   ebx, ah ; count of file name chars
0000A105 BE[58800100]        <1>     mov     esi, FindFile_Name
0000A10A 01F3                <1>     add     ebx, esi
                             <1>     ; 07/05/2016
0000A10C C7032E505247        <1>     mov     dword [ebx],  '.PRG'
0000A112 66C705[31830100]50- <1>     mov     word [EXE_ID], 'P.'
0000A11A 2E                  <1>
0000A11B EBAA                <1>     jmp     short loc_run_find_program_file_next
                             <1>
                             <1> loc_run_check_auto_path:
                             <1>     ; NOTE: /// 07/05/2016 ///
                             <1>     ; If the path is given, value of byte [Run_Manual_Path]
                             <1>     ; will not be ZERO. If so, file searching by using
                             <1>     ; Automatic Path (via 'PATH' environment variable)
                             <1>     ; will not be applicable, because the program file
                             <1>     ; is already/absolutely not found.
                             <1>
0000A11D A0[30830100]        <1>     mov     al, [Run_Manual_Path]
0000A122 08C0                <1>     or      al, al
                             <1>     ;jnz    loc_cmd_failed
                             <1>     ; 25/07/2022
0000A124 752B                <1>     jnz     short loc_run_cap_failed
                             <1>
                             <1> loc_run_check_auto_path_again:
                             <1>     ;cmp    word [Run_Auto_Path], 0FFFFh
                             <1>     ;       ; 0FFFFh = Not a valid run path (in ENV block)
                             <1>     ;;jnb   loc_cmd_failed
                             <1>     ;; 25/07/2022
                             <1>     ;jnb    short loc_run_cap_failed
                             <1>     ; 03/07/2025
0000A126 FF05[2C830100]      <1>     inc     dword [Run_Auto_Path] ; 0FFFFFFFFh -> 0
0000A12C 7423                <1>     jz      short loc_run_cap_failed ; -1 -> 0
                             <1>
                             <1>     ; 03/07/2025
0000A12E BF[76780100]        <1>     mov     edi, TextBuffer
0000A133 FF0D[2C830100]      <1>     dec     dword [Run_Auto_Path] ; 0 -> init auto path
0000A139 751D                <1>     jnz     short loc_run_chk_filename_ext_next
                             <1>
                             <1>     ;xor    al, al
0000A13B BE[632F0100]        <1>     mov     esi, Cmd_Path ; 'PATH'
                             <1>     ; 03/07/2025
                             <1>     ;mov    edi, TextBuffer
                             <1>     ; al = 0
                             <1>     ; 03/09/2024 (bugfix)
0000A140 66B90001            <1>     mov     cx, 256 ; TextBuffer (maximum) length
0000A144 E86EF9FFFF          <1>     call    get_environment_string
0000A149 7315                <1>     jnc     short loc_run_chk_filename_ext_again
                             <1>     ;mov    word [Run_Auto_Path], 0FFFFh ; invalid
                             <1>     ; 03/07/2025
0000A14B FF0D[2C830100]      <1>     dec     dword [Run_Auto_Path] ; -1 (invalid)
                             <1> loc_run_cap_failed: ; 25/07/2022
                             <1>     ;jmp    loc_cmd_failed
                             <1>     ; 30/08/2023
0000A151 B001                <1>     mov     al, 1 ; (force jump to loc_cmd_failed at the end)
0000A153 E927E3FFFF          <1>     jmp     loc_run_cmd_failed ; (restore cdir if it is needed)
                             <1>
                             <1>     ; 03/07/2025
                             <1> loc_run_chk_filename_ext_next:
0000A158 8B0D[28830100]      <1>     mov     ecx, [Run_Path_Length]
0000A15E EB09                <1>     jmp     short loc_run_chk_filename_ext_again_@
                             <1>
                             <1> loc_run_chk_filename_ext_again:
0000A160 89C1                <1>     mov     ecx, eax ; string length (with zero tail)
0000A162 49                  <1>     dec     ecx ; without zero tail
                             <1>     ; 03/07/2025
0000A163 890D[28830100]      <1>     mov     [Run_Path_Length], ecx
                             <1> loc_run_chk_filename_ext_again_@:
                             <1>     ; 04/07/2025
                             <1>     ;mov    ax, [EXE_ID]
                             <1>     ;cmp    ah, '.'
                             <1>     ;je     short loc_run_chk_auto_path_pos
                             <1>
                             <1> loc_run_change_file_ext_to_noext_again:
                             <1>     ; 04/07/2025
                             <1>     ;movzx  ebx, ah
0000A169 BE[58800100]        <1>     mov     esi, FindFile_Name
                             <1>     ;add    ebx, esi
0000A16E 31C0                <1>     xor     eax, eax
                             <1> loc_run_change_file_ext_to_noext_@:
0000A170 46                  <1>     inc     esi
0000A171 FEC4                <1>     inc     ah
```

```
                                <1>     ;cmp    al, '.'
0000A173 803E2E                 <1>     cmp     byte [esi], '.'
0000A176 75F8                   <1>     jne     short loc_run_change_file_ext_to_noext_@
0000A178 66A3[31830100]         <1>     mov     [EXE_ID], ax
                                <1>             ; al = 0, ah = file name length
                                <1>     ;sub    eax, eax
                                <1>     ;;mov   [ebx], eax ; 0 ; erase extension (.PRG)
                                <1>     ;mov    [esi], eax ; 0
0000A17E 8806                   <1>     mov     [esi], al ; 0
                                <1>
                                <1> loc_run_chk_auto_path_pos:
                                <1>     ;;movzx eax, word [Run_Auto_Path]
                                <1>     ;mov    ax, [Run_Auto_Path]
                                <1>     ; 03/07/2025
0000A180 A1[2C830100]           <1>     mov     eax, [Run_Auto_Path]
0000A185 39C8                   <1>     cmp     eax, ecx ; ecx = string length (except zero tail)
                                <1>     ;jnb    loc_cmd_failed
                                <1>     ; 25/07/2022
0000A187 73C8                   <1>     jnb     short loc_run_cap_failed
                                <1>
                                <1>     ; 03/07/2025
0000A189 09C0                   <1>     or      eax, eax
                                <1>     ;or     ax, ax
0000A18B 7502                   <1>     jnz     short loc_run_auto_path_pos_move
0000A18D B005                   <1>     mov     al, 5
                                <1>
                                <1> loc_run_auto_path_pos_move:
0000A18F 89FE                   <1>     mov     esi, edi ; offset TextBuffer
                                <1>     ; 04/07/2025
0000A191 BF[F6780100]           <1>     mov     edi, RunPathBuffer
0000A196 01C6                   <1>     add     esi, eax
                                <1>
                                <1> loc_run_auto_path_pos_space_loop:
0000A198 AC                     <1>     lodsb
0000A199 3C20                   <1>     cmp     al, 20h
0000A19B 74FB                   <1>     je      short loc_run_auto_path_pos_space_loop
                                <1>     ;jb     loc_cmd_failed
                                <1>     ; 25/07/2022
0000A19D 72B2                   <1>     jb      short loc_run_cap_failed
                                <1>     ;
                                <1> loc_run_auto_path_pos_move_next_@:
0000A19F AA                     <1>     stosb
                                <1> loc_run_auto_path_pos_move_next:
0000A1A0 AC                     <1>     lodsb
0000A1A1 3C3B                   <1>     cmp     al, ';'
0000A1A3 741B                   <1>     je      short loc_run_auto_path_pos_move_last_byte
0000A1A5 3C20                   <1>     cmp     al, 20h
0000A1A7 74F7                   <1>     je      short loc_run_auto_path_pos_move_next
                                <1>     ;jb     short loc_byte_ptr_end_of_path
                                <1>     ;stosb
                                <1>     ;jmp    short loc_run_auto_path_pos_move_next
                                <1>     ; 03/09/2024
0000A1A9 73F4                   <1>     jnb     short loc_run_auto_path_pos_move_next_@
                                <1>
                                <1> loc_byte_ptr_end_of_path:
0000A1AB 66C705[2C830100]FF-    <1>     mov     word [Run_Auto_Path], 0FFFFh ; end of path
0000A1B3 FF                     <1>
                                <1>     ; 03/07/2025
0000A1B4 C705[2C830100]FFFF-    <1>     mov     dword [Run_Auto_Path], 0FFFFFFFFh
0000A1BC FFFF                   <1>
0000A1BE EB0C                   <1>     jmp     short loc_run_auto_path_move_ok
                                <1>
                                <1> loc_run_auto_path_pos_move_last_byte:
0000A1C0 89F0                   <1>     mov     eax, esi
0000A1C2 2D[76780100]           <1>     sub     eax, TextBuffer
                                <1>     ;mov    [Run_Auto_Path], ax ; next path position
                                <1>     ; 03/07/2025
0000A1C7 A3[2C830100]           <1>     mov     [Run_Auto_Path], eax
                                <1>
                                <1> loc_run_auto_path_move_ok:
0000A1CC 4F                     <1>     dec     edi
0000A1CD B02F                   <1>     mov     al, '/'
0000A1CF 3807                   <1>     cmp     [edi], al
0000A1D1 7403                   <1>     je      short loc_run_auto_path_move_file_name
0000A1D3 47                     <1>     inc     edi
0000A1D4 8807                   <1>     mov     [edi], al
                                <1>
                                <1> loc_run_auto_path_move_file_name:
0000A1D6 47                     <1>     inc     edi
0000A1D7 BE[58800100]           <1>     mov     esi, FindFile_Name
                                <1>
                                <1> loc_run_auto_path_move_fn_loop:
0000A1DC AC                     <1>     lodsb
0000A1DD AA                     <1>     stosb
0000A1DE 08C0                   <1>     or      al, al
0000A1E0 75FA                   <1>     jnz     short loc_run_auto_path_move_fn_loop
                                <1>
                                <1>     ;mov    esi, TextBuffer
                                <1>     ; 04/07/2025
0000A1E2 BE[F6780100]           <1>     mov     esi, RunPathBuffer
0000A1E7 BF[16800100]           <1>     mov     edi, FindFile_Drv
0000A1EC E88D070000             <1>     call    parse_path_name
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 25/07/2022
                                <1>     ;jnc    short loc_run_change_current_drive
                                <1>     ;jmp    loc_run_cmd_failed
0000A1F1 723D                   <1>     jc      short loc_run_path_failed
                                <1>
                                <1> loc_run_change_current_drive:
0000A1F3 8A35[76770100]         <1>     mov     dh, [Current_Drv]
0000A1F9 8A15[16800100]         <1>     mov     dl, [FindFile_Drv]
0000A1FF 38F2                   <1>     cmp     dl, dh
0000A201 7407                   <1>     je      short loc_run_change_directory_again
                                <1>
0000A203 E8E8D4FFFF             <1>     call    change_current_drive
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 25/07/2022
                                <1>     ;jnc    short loc_run_change_directory_again
                                <1>     ;jmp    loc_run_cmd_failed
0000A208 7226                   <1>     jc      short loc_run_path_failed
                                <1>
                                <1> loc_run_change_directory_again:
0000A20A 803D[17800100]20       <1>     cmp     byte [FindFile_Directory], 20h
0000A211 7627                   <1>     jna     short loc_load_executable_cdir_chk_again
                                <1>
0000A213 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
0000A219 BE[17800100]           <1>     mov     esi, FindFile_Directory
0000A21E 30E4                   <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000A220 E8C6010000             <1>     call    change_current_directory
                                <1>     ;jc     loc_run_cmd_failed
                                <1>     ; 25/07/2022
0000A225 730E                   <1>     jnc     short loc_run_chg_prompt_dir_str_again
                                <1>
                                <1>     ; 03/07/2025
```

```
                                <1>     ; ignore 'directory not found' error
                                <1>     ; if auto path in progress
0000A227 803D[30830100]00       <1>     cmp     byte [Run_Manual_Path], 0
0000A22E 760A                   <1>     jna     short jmp_loc_run_find_executable_file_next
                                <1>
                                <1> loc_run_path_failed: ; 25/07/2022
0000A230 E94AE2FFFF             <1>     jmp     loc_run_cmd_failed
                                <1>
                                <1> loc_run_chg_prompt_dir_str_again:
0000A235 E8DA000000             <1>     call    change_prompt_dir_string
                                <1>
                                <1> loc_load_executable_cdir_chk_again:
                                <1> ; 04/07/2025
                                <1> %if 0
                                <1>     ; 03/07/2025
                                <1>     mov     al, [Current_Drv]
                                <1>     cmp     al, [RUN_CDRV]
                                <1>     jne     short jmp_loc_run_find_executable_file_next
                                <1>     ;
                                <1>     mov     eax, [Current_Dir_FCluster]
                                <1>     cmp     eax, [Run_CDirFC]
                                <1>     ;jne    loc_run_find_executable_file_next
                                <1>     ; 30/08/2023
                                <1>     je      short jmp_loc_run_check_auto_path_again
                                <1>
                                <1> jmp_loc_run_find_executable_file_next: ; 03/07/2025
                                <1>     jmp     loc_run_find_executable_file_next
                                <1> jmp_loc_run_check_auto_path_again:
                                <1>     xor     al, al ; 0
                                <1>     jmp     loc_run_check_auto_path_again
                                <1> %else
                                <1>     ; 04/07/2025
                                <1> jmp_loc_run_find_executable_file_next:
0000A23A E983FEFFFF             <1>     jmp     loc_run_find_executable_file_next
                                <1> %endif
                                <1>
                                <1> loc_load_and_run_file:
                                <1>     ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 13/11/2017
                                <1>     ; 04/01/2017
                                <1>     ; 23/04/2016
0000A23F BE[58800100]           <1>     mov     esi, FindFile_Name
0000A244 BF[76780100]           <1>     mov     edi, TextBuffer
                                <1>
                                <1>     ; 24/04/2016
0000A249 31D2                   <1>     xor     edx, edx
                                <1>     ;mov    word [argc], dx ; 0
                                <1>     ; 25/07/2022
0000A24B 8915[688F0100]         <1>     mov     dword [argc], edx
0000A251 8915[988E0100]         <1>     mov     dword [u.nread], edx ; 0
                                <1>
                                <1> loc_load_and_run_file_1:
0000A257 AC                     <1>     lodsb
0000A258 AA                     <1>     stosb
0000A259 FF05[988E0100]         <1>     inc     dword [u.nread]
0000A25F 20C0                   <1>     and     al, al
0000A261 75F4                   <1>     jnz     short loc_load_and_run_file_1
                                <1>
0000A263 A0[D7770100]           <1>     mov     al, [CmdArgStart]
0000A268 20C0                   <1>     and     al, al
0000A26A 7442                   <1>     jz      short loc_load_and_run_file_7
                                <1>
0000A26C 0FB6F0                 <1>     movzx   esi, al ; 11/05/2016
                                <1>     ;mov    ecx, 80
                                <1>     ; 25/07/2022
0000A26F 31C9                   <1>     xor     ecx, ecx
0000A271 B150                   <1>     mov     cl, 80
                                <1>     ;sub    ecx, esi
0000A273 28C1                   <1>     sub     cl, al
0000A275 81C6[26780100]         <1>     add     esi, CommandBuffer
                                <1>
                                <1>     ;inc    word [argc] ; 11/05/2016
                                <1>     ; 25/07/2022
0000A27B FF05[688F0100]         <1>     inc     dword [argc]
                                <1>
                                <1> loc_load_and_run_file_2:
0000A281 AC                     <1>     lodsb
0000A282 3C20                   <1>     cmp     al, 20h
0000A284 7716                   <1>     ja      short loc_load_and_run_file_5
0000A286 721D                   <1>     jb      short loc_load_and_run_file_6
                                <1>
                                <1> loc_load_and_run_file_3:
0000A288 803E20                 <1>     cmp     byte [esi], 20h
0000A28B 7707                   <1>     ja      short loc_load_and_run_file_4
0000A28D 7216                   <1>     jb      short loc_load_and_run_file_6
0000A28F 46                     <1>     inc     esi
0000A290 E2F6                   <1>     loop    loc_load_and_run_file_3
0000A292 EB11                   <1>     jmp     short loc_load_and_run_file_6
                                <1>
                                <1> loc_load_and_run_file_4:
0000A294 28C0                   <1>     sub     al, al ; 0
                                <1>     ;inc    word [argc]
                                <1>     ; 25/07/2022
0000A296 FF05[688F0100]         <1>     inc     dword [argc]
                                <1> loc_load_and_run_file_5:
0000A29C AA                     <1>     stosb
0000A29D FF05[988E0100]         <1>     inc     dword [u.nread]
0000A2A3 E2DC                   <1>     loop    loc_load_and_run_file_2
                                <1>
                                <1> loc_load_and_run_file_6:
0000A2A5 30C0                   <1>     xor     al, al ; 0
0000A2A7 AA                     <1>     stosb
0000A2A8 FF05[988E0100]         <1>     inc     dword [u.nread]
                                <1> loc_load_and_run_file_7:
0000A2AE 8807                   <1>     mov     [edi], al ; 0
                                <1>     ;inc    word [argc] ; 24/04/2016
                                <1>     ; 25/07/2022
0000A2B0 FF05[688F0100]         <1>     inc     dword [argc]
0000A2B6 FF05[988E0100]         <1>     inc     dword [u.nread] ; 24/04/2016
0000A2BC BE[76780100]           <1>     mov     esi, TextBuffer
0000A2C1 8B15[84800100]         <1>     mov     edx, [FindFile_DirEntry+DirEntry_FileSize]
0000A2C7 66A1[7C800100]         <1>     mov     ax, [FindFile_DirEntry+DirEntry_FstClusHI]
0000A2CD C1E010                 <1>     shl     eax, 16 ; 13/11/2017
0000A2D0 66A1[82800100]         <1>     mov     ax, [FindFile_DirEntry+DirEntry_FstClusLO]
                                <1>     ; EAX = First Cluster number
                                <1>     ; EDX = File Size
                                <1>     ; ESI = Argument list address
                                <1>     ; [argc] = argument count
                                <1>     ; [u.nread] = argument list length
0000A2D6 E8BA620000             <1>     call    load_and_run_file ; trdosk6.s
                                <1>         ;jc loc_run_cmd_failed ; 04/01/2017
                                <1> loc_load_and_run_file_8: ; 06/05/2016
0000A2DB E968EAFFFF             <1>     jmp     loc_file_rw_restore_retn
                                <1>
```

```
                                <1> check_prg_filename_ext:
                                <1>     ; 23/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 10/09/2011
                                <1>     ; (TRDOS v1, CMDINTR.ASM, 'proc_check_exe_filename_ext')
                                <1>     ; 14/11/2009
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Dot File Name
                                <1>     ; OUTPUT ->
                                <1>     ;     cf = 0 -> EXE_ID in AL
                                <1>     ;       ESI = Last char + 1 position
                                <1>     ;     cf = 1 -> Invalid executable file name
                                <1>     ;       or no file name extension if AH<=8
                                <1>     ;       AL = Last file name char
                                <1>     ;     cf = 0 -> AL='P' (PRG), AL=0 (no extension)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ESI)
                                <1>
0000A2E0 30E4                   <1>     xor     ah, ah
                                <1> loc_run_check_filename_ext:
0000A2E2 AC                     <1>     lodsb
0000A2E3 3C21                   <1>     cmp     al, 21h
0000A2E5 7229                   <1>     jb      short loc_check_exe_fn_retn
0000A2E7 FEC4                   <1>     inc     ah
0000A2E9 3C2E                   <1>     cmp     al, '.'
0000A2EB 75F5                   <1>     jne     short loc_run_check_filename_ext
                                <1>
                                <1> loc_run_check_filename_ext_dot:
0000A2ED 80FC02                 <1>     cmp     ah, 2  ; .??? is not valid
0000A2F0 88C4                   <1>     mov     ah, al ; '.'
0000A2F2 7219                   <1>     jb      short loc_check_prg_fn_retn
                                <1>
                                <1> loc_run_check_filename_ext_dot_ok:
0000A2F4 AC                     <1>     lodsb
0000A2F5 24DF                   <1>     and     al, 0DFh
                                <1>
                                <1> loc_run_check_filename_ext_prg:
0000A2F7 3C50                   <1>     cmp     al, 'P'
0000A2F9 7212                   <1>     jb      short loc_check_prg_fn_retn
0000A2FB 7711                   <1>     ja      short loc_check_prg_fn_stc
0000A2FD AC                     <1>     lodsb
0000A2FE 24DF                   <1>     and     al, 0DFh
0000A300 3C52                   <1>     cmp     al, 'R'
0000A302 750A                   <1>     jne     short loc_check_prg_fn_stc
0000A304 AC                     <1>     lodsb
0000A305 24DF                   <1>     and     al, 0DFh
0000A307 3C47                   <1>     cmp     al, 'G'
0000A309 7503                   <1>     jne     short loc_check_prg_fn_stc
                                <1>
0000A30B B050                   <1>     mov     al, 'P'
                                <1> loc_check_prg_fn_retn:
0000A30D C3                     <1>     retn
                                <1>
                                <1> ; 25/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>
                                <1> loc_check_prg_fn_stc:
0000A30E F9                     <1>     stc
0000A30F C3                     <1>     retn
                                <1>
                                <1> loc_check_exe_fn_retn:
0000A310 28C0                   <1>     sub     al, al ; 0
                                <1>     ;retn
                                <1>
                                <1> find_and_list_files:
                                <1>     ;retn
                                <1> set_exec_arguments:
0000A312 C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> ;delete_fs_directory:
                                <1> ;   xor     eax, eax
0000A313 C3                     <1>     retn
                                %include 'trdosk4.s' ; 24/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - Directory Functions : trdosk4.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 19/12/2025  (Previous: 03/09/2024, v2.0.9)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; DIR.ASM (09/10/2011)
                                <1> ; ****************************************************************************
                                <1>
                                <1> ; DIR.ASM  [ TRDOS KERNEL - COMMAND EXECUTER SECTION - DIRECTORY FUNCTIONS ]
                                <1> ; (c) 2004-2010  Erdogan TAN  [ 17/01/2004 ]  Last Update: 09/10/2011
                                <1> ; FILE.ASM [ FILE FUNCTIONS ] Last Update: 09/10/2011
                                <1>
                                <1> change_prompt_dir_string:
                                <1>     ; 05/10/2016
                                <1>     ; 24/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 27/03/2011
                                <1>     ; 09/10/2009
                                <1>     ; INPUT/OUTPUT => none
                                <1>     ; this procedure changes current directory string/text
                                <1>     ; 2005
                                <1>
0000A314 BE[D47E0100]           <1>     mov     esi, PATH_Array
                                <1> change_prompt_dir_str: ; 05/10/2016 (call from 'set_working_path')
0000A319 BF[7A770100]           <1>     mov     edi, Current_Directory
0000A31E 8A25[74770100]         <1>     mov     ah, [Current_Dir_Level]
0000A324 E807000000             <1>     call    set_current_directory_string
0000A329 880D[D5770100]         <1>     mov     [Current_Dir_StrLen], cl
                                <1>
0000A32F C3                     <1>     retn
                                <1>
                                <1> set_current_directory_string:
                                <1>     ; 11/08/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 24/01/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 27/03/2011
                                <1>     ; 09/10/2009
                                <1>     ; INPUT:
                                <1>     ;    ESI = Path Array Address
                                <1>     ;    EDI = Current Directory String Buffer
                                <1>     ;    AH = Current Directory Level
                                <1>     ; OUTPUT => EAX, EBX, ESI will be changed
                                <1>     ;    EDI will be same with input
                                <1>     ;    ECX = Current Directory String Length
                                <1>
0000A330 57                     <1>     push    edi
0000A331 80FC00                 <1>     cmp     ah, 0
0000A334 7651                   <1>     jna     short pass_write_path
0000A336 83C610                 <1>     add     esi, 16
```

```
0000A339 89F3                <1>     mov     ebx, esi
                             <1> loc_write_path:
                             <1>     ;mov    ecx, 8
                             <1>     ; 11/08/2022
0000A33B 29C9                <1>     sub     ecx, ecx
0000A33D B108                <1>     mov     cl, 8
                             <1> path_write_dirname1:
0000A33F AC                  <1>     lodsb
0000A340 3C20                <1>     cmp     al, 20h
0000A342 7612                <1>     jna     short pass_write_dirname1
0000A344 AA                  <1>     stosb
0000A345 81FF[D4770100]      <1>     cmp     edi, End_Of_Current_Dir_Str
0000A34B 733A                <1>     jnb     short pass_write_path
0000A34D E2F0                <1>     loop    path_write_dirname1
0000A34F 803E20              <1>     cmp     byte [esi], 20h
0000A352 7624                <1>     jna     short pass_write_dirname2
0000A354 EB0A                <1>     jmp      short loc_put_dot_cont_ext
                             <1> pass_write_dirname1:
0000A356 89DE                <1>     mov     esi, ebx
0000A358 83C608              <1>     add     esi, 8
0000A35B 803E20              <1>     cmp     byte [esi], 20h
0000A35E 7618                <1>     jna     short pass_write_dirname2
                             <1> loc_put_dot_cont_ext:
0000A360 C6072E              <1>     mov     byte [edi], "."
                             <1>     ;mov    ecx, 3
0000A363 B103                <1>     mov     cl, 3
                             <1> loc_check_dir_name_ext:
0000A365 AC                  <1>     lodsb
0000A366 47                  <1>     inc     edi
0000A367 3C20                <1>     cmp     al, 20h
0000A369 760D                <1>     jna     short pass_write_dirname2
0000A36B 8807                <1>     mov     [edi], al
0000A36D 81FF[D4770100]      <1>     cmp     edi, End_Of_Current_Dir_Str
0000A373 7312                <1>     jnb     short pass_write_path
0000A375 E2EE                <1>     loop     loc_check_dir_name_ext
0000A377 47                  <1>     inc     edi
                             <1> pass_write_dirname2:
0000A378 FECC                <1>     dec     ah
0000A37A 740B                <1>     jz       short pass_write_path
0000A37C 83C310              <1>     add     ebx, 16
0000A37F 89DE                <1>     mov     esi, ebx
0000A381 C6072F              <1>     mov     byte [edi],"/"
0000A384 47                  <1>     inc     edi
0000A385 EBB4                <1>     jmp     short loc_write_path
                             <1> pass_write_path:
0000A387 C60700              <1>     mov     byte [edi], 0
0000A38A 47                  <1>     inc     edi
0000A38B 89F9                <1>     mov     ecx, edi
0000A38D 5F                  <1>     pop     edi
0000A38E 29F9                <1>     sub     ecx, edi
                             <1>     ; ECX = Current Directory String Length
0000A390 C3                  <1>     retn
                             <1>
                             <1> get_current_directory:
                             <1>     ; 29/08/2023 (TRDOS 386 Kernel v2.0.6)
                             <1>     ; 11/08/2022
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/10/2016
                             <1>     ; 14/02/2016
                             <1>     ; 24/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 27/03/2011
                             <1>     ;
                             <1>     ; INPUT-> ESI = Current Directory Buffer
                             <1>     ;         DL = TRDOS Logical Dos Drive Number + 1
                             <1>     ;              (0 = Default/Current Drive)
                             <1>     ;
                             <1>     ;   Note: Required dir buffer length may be <= 92 bytes
                             <1>     ;         for TRDOS (7*12 name chars + 7 slash + 0)
                             <1>     ; OUTPUT ->  ESI = Current Directory Buffer
                             <1>     ;            EAX, EBX, ECX, EDX, EDI will be changed
                             <1>     ;            CX/CL = Current Directory String Length
                             <1>     ;           DL = Drive Number (0 based)
                             <1>     ;            (If input is 0, output is current drv number)
                             <1>     ;            DH = same with input
                             <1>     ;   cf = 0 -> AL = 0
                             <1>     ;   cf = 1 -> error code in AL
                             <1>
                             <1> loc_get_current_drive_0:
                             <1>     ; 29/08/2023
0000A391 29C0                <1>     sub     eax, eax ; 0
                             <1>     ;cmp    dl, 0
0000A393 38C2                <1>     cmp     dl, al
0000A395 7724                <1>     ja      short loc_get_current_drive_1
0000A397 8A15[76770100]      <1>     mov     dl, [Current_Drv]
                             <1>     ; 29/08/2023
                             <1>     ;jmp    short loc_get_current_drive_2
                             <1>
                             <1> loc_get_current_drive_2:
                             <1>     ; 29/08/2023
                             <1>     ;xor    eax, eax
                             <1>     ; eax = 0
0000A39D 88D4                <1>     mov     ah, dl
0000A39F 56                  <1>     push    esi ; (*)
0000A3A0 BE00010900          <1>     mov     esi, Logical_DOSDisks
0000A3A5 01C6                <1>     add     esi, eax
0000A3A7 8A06                <1>     mov     al, [esi+LD_Name]
0000A3A9 3C41                <1>     cmp     al, 'A'
0000A3AB 721C                <1>     jb      short loc_get_current_drive_not_ready_retn
                             <1>
                             <1>     ; 28/07/2022
0000A3AD 31C9                <1>     xor     ecx, ecx
                             <1>
0000A3AF 8A667F              <1>     mov     ah, [esi+LD_CDirLevel]
0000A3B2 08E4                <1>     or      ah, ah
0000A3B4 7519                <1>     jnz     short loc_get_current_drive_3
                             <1>
                             <1>     ;xor    ah, ah ; mov ah, 0
                             <1>
                             <1>     ; 11/08/2022 - BugFix (*)
0000A3B6 5E                  <1>     pop     esi ; (*) Current Directory Buffer address
                             <1>
0000A3B7 8826                <1>     mov     [esi], ah
                             <1>     ; 28/07/2022
                             <1>     ;xor    ecx, ecx
0000A3B9 EB2D                <1>     jmp     short loc_get_current_drive_4
                             <1>
                             <1>     ; 29/08/2023
                             <1> loc_get_current_drive_1:
0000A3BB FECA                <1>     dec     dl
0000A3BD 3A15[8D2E0100]      <1>     cmp     dl, [Last_DOS_DiskNo]
0000A3C3 76D8                <1>     jna     short loc_get_current_drive_2
                             <1>     ;mov    eax, 0Fh ; Invalid drive (Drive not ready!)
                             <1>     ;cmc    ; stc
                             <1>     ; 28/07/2022
```

```
                                <1>     ;sub    eax, eax ; 29/08/2023
                                <1>     ; eax = 0
0000A3C5 B00F                   <1>     mov     al, 0Fh
0000A3C7 F9                     <1>     stc
0000A3C8 C3                     <1>     retn
                                <1>
                                <1> loc_get_current_drive_not_ready_retn:
0000A3C9 5E                     <1>     pop     esi
                                <1>     ;mov    eax, 15
0000A3CA 66B80F00               <1>     mov     ax, 15 ; Drive not ready
0000A3CE C3                     <1>     retn
                                <1>
                                <1> loc_get_current_drive_3:
0000A3CF BF[D47E0100]           <1>         mov     edi, PATH_Array
0000A3D4 57                     <1>     push    edi
0000A3D5 81C680000000           <1>     add     esi, LD_CurrentDirectory
                                <1>     ;mov    ecx, 32
                                <1>     ; 28/07/2022
0000A3DB B120                   <1>     mov     cl, 32
0000A3DD F3A5                   <1>     rep     movsd
0000A3DF 5E                     <1>     pop     esi ; Path Array Address
0000A3E0 5F                     <1>     pop     edi ; pushed esi (current dir buffer offset)
                                <1>     ;
0000A3E1 E84AFFFFFF             <1>     call    set_current_directory_string
0000A3E6 89FE                   <1>     mov     esi, edi
                                <1>
                                <1> loc_get_current_drive_4:
0000A3E8 30C0                   <1>     xor     al, al
0000A3EA C3                     <1>     retn
                                <1>
                                <1> change_current_directory:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 02/03/2021 (TRDOS 386 v2.0.3) ((BugFix))
                                <1>     ; 19/02/2016
                                <1>     ; 11/02/2016
                                <1>     ; 10/02/2016
                                <1>     ; 08/02/2016
                                <1>     ; 06/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 18/09/2011 (DIR.ASM, 09/10/2011)
                                <1>     ; 04/10/2009
                                <1>     ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Directory string
                                <1>     ;       ah = CD command (CDh = save current dir string)
                                <1>     ; OUTPUT ->
                                <1>     ;       EDI = DOS Drive Description Table
                                <1>     ;       cf = 1 -> error
                                <1>     ;          EAX = Error code
                                <1>     ;       cf = 0 -> successful
                                <1>     ;          ESI = PATH_Array
                                <1>     ;          EAX = Current Directory First Cluster
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, ESI, EDI will be changed)
                                <1>
0000A3EB 8825[627F0100]         <1>     mov     [CD_COMMAND], ah
0000A3F1 803E2F                 <1>     cmp     byte [esi], '/'
0000A3F4 7505                   <1>     jne     short loc_ccd_cdir_level
0000A3F6 46                     <1>     inc     esi
0000A3F7 30C0                   <1>     xor     al, al
0000A3F9 EB05                   <1>     jmp     short loc_ccd_parse_path_name
                                <1> loc_ccd_cdir_level:
0000A3FB A0[74770100]           <1>     mov     al, [Current_Dir_Level]
                                <1> loc_ccd_parse_path_name:
0000A400 88C4                   <1>     mov     ah, al
0000A402 BF[D47E0100]           <1>     mov     edi, PATH_Array
                                <1>
                                <1> ; Reset directory levels > cdir level
                                <1>     ; is this required !?
                                <1>     ;
                                <1>     ; Relations:
                                <1>     ; MAINPROG.ASM (pass_ccdrv_reset_cdir_FAT_fcluster)
                                <1>     ; proc_parse_dir_name,
                                <1>     ; proc_change_current_directory (this procedure)
                                <1>     ; proc_change_prompt_dir_string
                                <1>
0000A407 0FB6C8                 <1>     movzx   ecx, al
0000A40A FEC1                   <1>     inc     cl
0000A40C C0E104                 <1>     shl     cl, 4
0000A40F 01CF                   <1>     add     edi, ecx
0000A411 B107                   <1>     mov     cl, 7
0000A413 28C1                   <1>     sub     cl, al
0000A415 C0E102                 <1>     shl     cl, 2
0000A418 89C3                   <1>     mov     ebx, eax
0000A41A 31C0                   <1>     xor     eax, eax ; 0
0000A41C F3AB                   <1>     rep     stosd
0000A41E 89D8                   <1>     mov     eax, ebx
                                <1>
0000A420 BF[D47E0100]           <1>     mov     edi, PATH_Array
                                <1>
0000A425 803E20                 <1>     cmp     byte [esi], 20h
0000A428 F5                     <1>     cmc
0000A429 7305                   <1>     jnc     short pass_ccd_parse_dir_name
                                <1>
                                <1>             ; ESI = Path name
                                <1>             ; AL = CCD_Level
0000A42B E83B010000             <1>         call    parse_dir_name
                                <1>             ; AL = CCD_Level
                                <1>             ; AH = Last_Dir_Level
                                <1>             ; (EDI = PATH_Array)
                                <1>
                                <1> pass_ccd_parse_dir_name:
0000A430 9C                     <1>     pushf
                                <1>
                                <1>     ;mov    [CCD_Level], al
                                <1>         ;mov        [Last_Dir_Level], ah
0000A431 66A3[587F0100]         <1>     mov     [CCD_Level], ax
                                <1>
0000A437 31DB                   <1>     xor     ebx, ebx
0000A439 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
0000A43F BE00010900             <1>     mov     esi, Logical_DOSDisks
0000A444 01DE                   <1>     add     esi, ebx
                                <1>
0000A446 9D                     <1>     popf
0000A447 720A                   <1>     jc      short loc_ccd_bad_path_name_retn
                                <1>
0000A449 8935[547F0100]         <1>     mov     [CCD_DriveDT], esi
                                <1>
0000A44F 3C07                   <1>     cmp     al, 7
0000A451 7208                   <1>     jb      short loc_ccd_load_child_dir
                                <1>
                                <1> loc_ccd_bad_path_name_retn:
0000A453 87F7                   <1>     xchg    esi, edi
                                <1>     ;mov    eax, 19 ; Bad directory/path name
```

```
                                <1>     ; 28/07/2022
0000A455 29C0                   <1>     sub     eax, eax
0000A457 B013                   <1>     mov     al, 19
0000A459 F9                     <1>     stc
                                <1> loc_ccd_retn_p:
0000A45A C3                     <1>     retn
                                <1>
                                <1> loc_ccd_load_child_dir:
                                <1>     ; AL = CCD_Level
0000A45B 08C0                   <1>     or      al, al
0000A45D 744C                   <1>     jz      short loc_ccd_load_root_dir
                                <1>
                                <1>     ;mov    cx, ax
                                <1>     ; 28/07/2022
0000A45F 89C1                   <1>     mov     ecx, eax
0000A461 C0E004                 <1>     shl     al, 4
0000A464 0FB6F0                 <1>     movzx   esi, al
0000A467 01FE                   <1>             add     esi, edi  ; offset PATH_Array
                                <1>
0000A469 8B460C                 <1>     mov     eax, [esi+12]
0000A46C 38E9                   <1>     cmp     cl, ch
                                <1>     ;je     loc_ccd_load_sub_directory
                                <1>     ; 28/07/2022
0000A46E 7505                   <1>     jne     short loc_ccd_1
0000A470 E992000000             <1>     jmp     loc_ccd_load_sub_directory
                                <1>
                                <1> loc_ccd_1:  ; 28/07/2022
0000A475 A3[70770100]           <1>     mov     [Current_Dir_FCluster], eax
                                <1>
                                <1> loc_ccd_load_child_dir_next:
0000A47A 83C610                 <1>     add     esi, 16 ; DOS DirEntry Format FileName Address
                                <1>
                                <1>     ; Directory attribute : 10h
0000A47D B010                   <1>     mov     al, 00010000b ; 10h (Attrib AND mask)
                                <1>     ;mov    ah, 11001000b ; C8h
                                <1>     ; Volume name attribute: 8h
0000A47F B408                   <1>     mov     ah, 00001000b ; 08h (Attrib NAND, AND --> zero mask)
                                <1>
                                <1>     ;xor    cx, cx
0000A481 31C9                   <1>     xor     ecx, ecx ; 02/03/2021
0000A483 E880010000             <1>     call    locate_current_dir_file
0000A488 733C                   <1>     jnc     short loc_ccd_set_dir_cluster_ptr
                                <1>
                                <1>     ; 19/02/2016
                                <1>     ;mov    edi, [CCD_DriveDT]
0000A48A 8A25[587F0100]         <1>     mov     ah, [CCD_Level]
0000A490 803D[627F0100]CD       <1>     cmp     byte [CD_COMMAND], 0CDh ;'CD' command or another
0000A497 7509                   <1>     jne     short loc_ccd_load_child_dir_err
                                <1>     ; It is better to save recent successful part
                                <1>     ; of the (requested) path as current directory.
                                <1>     ; (Otherwise the path would be reset to back
                                <1>     ; on the next 'CD' command.)
0000A499 88E1                   <1>     mov     cl, ah
0000A49B 50                     <1>     push    eax
0000A49C E8AE000000             <1>     call    loc_ccd_save_current_dir
0000A4A1 58                     <1>     pop     eax
                                <1> loc_ccd_load_child_dir_err:
0000A4A2 3C03                   <1>     cmp     al, 3  ; AL = 2 => File not found error
0000A4A4 7202                   <1>     jb      short loc_ccd_path_not_found_retn
0000A4A6 F9                     <1>     stc
0000A4A7 C3                     <1>     retn
                                <1>
                                <1> loc_ccd_path_not_found_retn:
0000A4A8 B003                   <1>     mov     al, 3  ; Path not found
0000A4AA C3                     <1>     retn
                                <1>
                                <1> loc_ccd_load_root_dir:     ; 19/12/2025
                                <1> loc_ccd_load_FAT_root_dir:
0000A4AB 803D[75770100]02       <1>     cmp     byte [Current_FATType], 2
0000A4B2 774B                   <1>     ja      short loc_ccd_load_FAT32_root_dir
                                <1>
                                <1>     ;mov    esi, [CCD_DriveDT]
                                <1>     ;push   esi
0000A4B4 E80E1C0000             <1>     call    load_FAT_root_directory
                                <1>     ;pop    edi ; Dos Drv Description Table
                                <1>
0000A4B9 89F7                   <1>     mov     edi, esi
0000A4BB BE[D47E0100]           <1>     mov     esi, PATH_Array
0000A4C0 7298                   <1>     jc      short loc_ccd_retn_p
                                <1>
0000A4C2 31C0                   <1>     xor     eax, eax
0000A4C4 EB58                   <1>         jmp short loc_ccd_set_cdfc
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1> loc_ccd_load_root_dir:
                                <1>     cmp     byte [Current_FATType], 1
                                <1>     jnb     short loc_ccd_load_FAT_root_dir
                                <1>
                                <1> loc_ccd_load_FS_root_dir:
                                <1>     call    load_FS_root_directory
                                <1>     jmp     short pass_ccd_load_FAT_sub_directory
                                <1>
                                <1> loc_ccd_load_FS_sub_directory_next:
                                <1>     call    load_FS_sub_directory
                                <1>     jmp     short pass_ccd_set_dir_cluster_ptr
                                <1> %endif
                                <1>
                                <1> loc_ccd_set_dir_cluster_ptr:
                                <1>     ; EDI = Directory Entry
0000A4C6 668B4714               <1>     mov     ax, [edi+20] ; First Cluster High Word
0000A4CA C1E010                 <1>     shl     eax, 16
0000A4CD 668B471A               <1>     mov     ax, [edi+26] ; First Cluster Low Word
                                <1>
0000A4D1 8B35[547F0100]         <1>     mov     esi, [CCD_DriveDT]
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [Current_FATType], 1
                                <1>     jb      short loc_ccd_load_FS_sub_directory_next
                                <1> %endif
                                <1>     ;push   esi
0000A4D7 E8691C0000             <1>     call    load_FAT_sub_directory
                                <1>     ;pop    edi ; Dos Drv Description Table
                                <1>
                                <1> pass_ccd_set_dir_cluster_ptr:
                                <1>     ;mov    edi, esi
0000A4DC BE[D47E0100]           <1>     mov     esi, PATH_Array
0000A4E1 7265                   <1>     jc      short loc_ccd_retn_c
                                <1>
0000A4E3 A1[A27E0100]           <1>     mov     eax, [DirBuff_Cluster]
                                <1>
0000A4E8 FE05[587F0100]         <1>     inc     byte [CCD_Level]
0000A4EE 0FB61D[587F0100]       <1>     movzx   ebx, byte [CCD_Level]
```

```
0000A4F5 C0E304              <1>     shl     bl, 4 ; * 16 (<= 128)
0000A4F8 01DE                <1>     add     esi, ebx ; 19/02/2016
0000A4FA 89460C              <1>     mov     [esi+12], eax
0000A4FD EB1F                <1>     jmp     short loc_ccd_set_cdfc
                             <1>
                             <1> loc_ccd_load_FAT32_root_dir:
0000A4FF BE[D47E0100]        <1>     mov     esi, PATH_Array
0000A504 8B460C              <1>     mov     eax, [esi+12]
                             <1>
                             <1> loc_ccd_load_sub_directory: ; 19/12/2025
0000A507 8B35[547F0100]      <1>     mov     esi, [CCD_DriveDT]
                             <1>
                             <1> loc_ccd_load_FAT_sub_directory:
                             <1>     ;push   esi
0000A50D E8331C0000          <1>     call    load_FAT_sub_directory
                             <1>     ;pop    edi ; Dos Drv Description Table
                             <1>
                             <1> pass_ccd_load_FAT_sub_directory:
                             <1>     ;mov    edi, esi
0000A512 BE[D47E0100]        <1>     mov     esi, PATH_Array
0000A517 722F                <1>     jc      short loc_ccd_retn_c
                             <1>
0000A519 A1[A27E0100]        <1>     mov     eax, [DirBuff_Cluster]
                             <1>
                             <1> loc_ccd_set_cdfc:
0000A51E 8A0D[587F0100]      <1>     mov     cl, [CCD_Level]
0000A524 880D[74770100]      <1>     mov     [Current_Dir_Level], cl
0000A52A A3[70770100]        <1>     mov     [Current_Dir_FCluster], eax
                             <1>
0000A52F 8A2D[597F0100]      <1>     mov     ch, [Last_Dir_Level]
0000A535 38E9                <1>     cmp     cl, ch
                             <1>     ;jb     loc_ccd_load_child_dir_next
                             <1>     ; 28/07/2022
0000A537 7305                <1>     jnb     short loc_ccd_2
0000A539 E93CFFFFFF          <1>     jmp     loc_ccd_load_child_dir_next
                             <1> loc_ccd_2:
0000A53E 803D[627F0100]CD    <1>     cmp     byte [CD_COMMAND], 0CDh ;'CD' command or another
0000A545 7408                <1>     je      short loc_ccd_save_current_dir
                             <1>
                             <1>         ; jne -> don't save, restore (the previous cdir) later !
                             <1>         ; (saving the cdir would prevent previous cdir restoration!)
                             <1>
0000A547 F8                  <1>     clc
                             <1>
                             <1> loc_ccd_retn_c:
0000A548 8B3D[547F0100]      <1>     mov     edi, [CCD_DriveDT]
0000A54E C3                  <1>     retn
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> loc_ccd_load_sub_directory:
                             <1>     mov     esi, [CCD_DriveDT]
                             <1>     cmp     byte [Current_FATType], 1
                             <1>     jnb     short loc_ccd_load_FAT_sub_directory
                             <1>     call    load_FS_sub_directory
                             <1>     jmp     short pass_ccd_load_FAT_sub_directory
                             <1> %endif
                             <1>
                             <1> loc_ccd_save_current_dir:
                             <1>     ; 02/03/2021 (TRDOS 386 v2.0.3) ((BugFix))
                             <1>     ; ('find_directory_entry' has been fixed to prevent large
                             <1>     ; ECX value > 65535)
                             <1>     ;
0000A54F BE[D47E0100]        <1>     mov     esi, PATH_Array ; 19/02/2016
0000A554 8B3D[547F0100]      <1>     mov     edi, [CCD_DriveDT]
0000A55A 57                  <1>     push    edi
0000A55B 83C77F              <1>         add     edi, LD_CDirLevel
0000A55E 880F                <1>     mov     [edi], cl
0000A560 47                  <1>     inc     edi ; LD_CurrentDirectory
0000A561 56                  <1>     push    esi
                             <1>     ;;mov   ecx, 32  ; always < 65536 (in this procedure)
0000A562 66B92000            <1>     mov     cx, 32
                             <1>     ; 02/03/2021
                             <1>     ;mov    ecx, 32
0000A566 F3A5                <1>     rep     movsd
                             <1>     ; Current directory has been saved to
                             <1>     ; the DOS drive description table, cdir area !
0000A568 5E                  <1>     pop     esi  ; PATH_Array
0000A569 5F                  <1>     pop     edi  ; Dos Drv Description Table
                             <1>
0000A56A C3                  <1>     retn
                             <1>
                             <1> parse_dir_name:
                             <1>     ; 11/02/2016
                             <1>     ; 10/02/2016
                             <1>     ; 07/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 18/09/2011
                             <1>     ; 17/10/2009
                             <1>     ; INPUT ->
                             <1>     ;       ESI = ASCIIZ Directory String Address
                             <1>     ;       AL = Current Directory Level
                             <1>     ;       EDI = Destination Adress
                             <1>     ;             (8 levels, each one 12+4 byte)
                             <1>     ; OUTPUT ->
                             <1>     ;       EDI = Dir Entry Formatted Array
                             <1>     ;             with zero cluster pointer at the last level
                             <1>     ;       AH = Last Dir Level
                             <1>     ;       AL = Current Dir Level
                             <1>     ;
                             <1>     ; (esi, ebx, ecx will be changed)
                             <1>
                             <1>     ;mov    [PATH_Array_Ptr], edi
0000A56B 88C4                <1>     mov     ah, al
0000A56D 66A3[F87F0100]      <1>     mov     [PATH_CDLevel], ax
                             <1> repeat_ppdn_check_slash:
0000A573 AC                  <1>     lodsb
0000A574 3C2F                <1>     cmp     al, '/'
0000A576 74FB                <1>     je      short repeat_ppdn_check_slash
0000A578 3C21                <1>     cmp     al, 21h
0000A57A 7219                <1>     jb      short loc_ppdn_retn
0000A57C 57                  <1>     push    edi
                             <1> loc_ppdn_get_dir_name:
0000A57D B90C000000          <1>     mov     ecx, 12
0000A582 BF[FA7F0100]        <1>     mov     edi, Dir_File_Name
                             <1> repeat_ppdn_get_dir_name:
0000A587 AA                  <1>     stosb
0000A588 AC                  <1>     lodsb
0000A589 3C2F                <1>     cmp     al, '/'
0000A58B 740A                <1>     je      short loc_check_level_dot_conv_dir_name
0000A58D 3C20                <1>     cmp     al, 20h
0000A58F 7605                <1>     jna     short loc_ppdn_end_of_path_scan
0000A591 E2F4                <1>     loop    repeat_ppdn_get_dir_name
0000A593 5F                  <1>     pop     edi
0000A594 F9                  <1>     stc
```

```
                                <1> loc_ppdn_retn:
0000A595 C3                     <1>     retn
                                <1>
                                <1> loc_ppdn_end_of_path_scan:
0000A596 4E                     <1>     dec     esi
                                <1> loc_check_level_dot_conv_dir_name:
0000A597 31C0                   <1>     xor     eax, eax
0000A599 AA                     <1>     stosb
0000A59A 89F3                   <1>     mov     ebx, esi
0000A59C BE[FA7F0100]           <1>     mov     esi, Dir_File_Name
0000A5A1 AC                     <1>     lodsb
                                <1> repeat_ppdn_name_check_dot:
0000A5A2 3C2E                   <1>     cmp     al, '.'
0000A5A4 7509                   <1>     jne     short loc_ppdn_convert_sub_dir_name
                                <1> repeat_ppdn_name_dot_dot:
0000A5A6 AC                     <1>     lodsb
0000A5A7 3C2E                   <1>     cmp     al, '.'
0000A5A9 743E                   <1>     je      short loc_ppdn_dot_dot
0000A5AB 3C21                   <1>     cmp     al, 21h
0000A5AD 7226                   <1>     jb      short pass_ppdn_convert_sub_dir_name
                                <1> loc_ppdn_convert_sub_dir_name:
0000A5AF 8A25[F97F0100]         <1>     mov     ah, [PATH_Level]
0000A5B5 80FC07                 <1>     cmp     ah, 7
0000A5B8 731B                   <1>     jnb     short pass_ppdn_convert_sub_dir_name
0000A5BA FEC4                   <1>     inc     ah
0000A5BC 8825[F97F0100]         <1>     mov     [PATH_Level], ah
0000A5C2 BE[FA7F0100]           <1>     mov     esi, Dir_File_Name
                                <1>     ;mov    edi, [PATH_Array_Ptr]
0000A5C7 B010                   <1>     mov     al, 16
0000A5C9 F6E4                   <1>     mul     ah
0000A5CB 8B3C24                 <1>     mov     edi, [esp]
                                <1>     ;push   edi
0000A5CE 01C7                   <1>     add     edi, eax
0000A5D0 E8DF020000             <1>     call    convert_file_name
                                <1>     ;pop    edi
                                <1> pass_ppdn_convert_sub_dir_name:
0000A5D5 89DE                   <1>     mov     esi, ebx
                                <1> repeat_ppdn_check_last_slash:
0000A5D7 AC                     <1>     lodsb
0000A5D8 3C2F                   <1>     cmp     al, '/'
0000A5DA 74FB                   <1>     je      short repeat_ppdn_check_last_slash
0000A5DC 3C21                   <1>     cmp     al, 21h
0000A5DE 739D                   <1>     jnb     short loc_ppdn_get_dir_name
                                <1> end_of_parse_dir_name:
0000A5E0 5F                     <1>     pop     edi
0000A5E1 F5                     <1>     cmc
                                <1>     ;mov    al, [PATH_CDLevel]
                                <1>     ;mov    ah, [PATH_Level]
0000A5E2 66A1[F87F0100]         <1>     mov     ax, [PATH_CDLevel]
0000A5E8 C3                     <1>     retn
                                <1>
                                <1> loc_ppdn_dot_dot:
0000A5E9 AC                     <1>     lodsb
0000A5EA 3C21                   <1>     cmp     al, 21h
0000A5EC 73F2                   <1>     jnb     short end_of_parse_dir_name
                                <1> loc_ppdn_dot_dot_prev_level:
0000A5EE 66A1[F87F0100]         <1>     mov     ax, [PATH_CDLevel]
0000A5F4 80EC01                 <1>     sub     ah, 1
0000A5F7 80D400                 <1>     adc     ah, 0
0000A5FA 38E0                   <1>     cmp     al, ah
0000A5FC 7602                   <1>     jna     short pass_ppdn_set_al_to_ah
0000A5FE 88E0                   <1>     mov     al, ah
                                <1> pass_ppdn_set_al_to_ah:
0000A600 66A3[F87F0100]         <1>     mov     [PATH_CDLevel], ax
0000A606 EBCD                   <1>     jmp     short pass_ppdn_convert_sub_dir_name
                                <1>
                                <1> locate_current_dir_file:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 26/08/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 26/08/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 20/11/2017
                                <1>     ; 14/02/2016
                                <1>     ; 13/02/2016
                                <1>     ; 10/02/2016
                                <1>     ; 06/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 14/08/2010
                                <1>     ; 19/09/2009
                                <1>       ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       ESI = DOS DirEntry Format FileName Address
                                <1>     ;       AL = Attributes Mask
                                <1>     ;       (<AL AND EntryAttrib> must be equal to AL)
                                <1>     ;       AH = Negative Attributes Mask (If AH>0)
                                <1>     ;       (<AH AND EntryAttrib> must be ZERO)
                                <1>     ;       CH > 0 Find First Free Dir Entry or Deleted Entry
                                <1>     ;       CL = 0 -> Return the First Free Dir Entry
                                <1>     ;       CL = E5h -> Return the 1st deleted entry
                                <1>     ;       CL = FFh -> Return the 1st deleted or free entry
                                <1>     ;       CL > 0 and CL <> E5h and CL <> FFh -> Return the first
                                <1>     ;            proper entry (which fits with Atributes Masks)
                                <1>     ;       CX = 0 Find Valid File/Directory/VolumeName
                                <1>     ;       ? = Any One Char
                                <1>     ;       * = Every Chars
                                <1>     ; OUTPUT ->
                                <1>     ;       EDI = Directory Entry Address (in Directory Buffer)
                                <1>     ;       ESI = DOS DirEntry Format FileName Address
                                <1>     ;       CF = 0 -> No Error, Proper Entry,
                                <1>     ;       DL = Attributes
                                <1>     ;       DH = Previous Entry Attr (LongName Check)
                                <1>     ;       AL > 0 -> Ambiguous filename wildcard "?" used
                                <1>     ;       AH > 0 -> Ambiguous filename wildcard "*" used
                                <1>     ;       AX = 0 -> Filename full fits with directory entry
                                <1>     ;       CH = The 1st Name Char of Current Dir Entry
                                <1>     ;       CF = 1 -> Proper entry not found, Error Code in EAX/AL
                                <1>     ;       CL = 0 and CH = 0 -> Free Entry (End Of Dir)
                                <1>     ;       CL = 0 and CH = E5h -> Deleted Entry fits with filters
                                <1>     ;       CL > 0 -> Entry not found, CH invalid
                                <1>     ;       CF = 0 ->
                                <1>     ;       EBX = Current Directory Entry Index/Number (BX)
                                <1>
                                <1>     ;mov    word [DirBuff_EntryCounter], 0 ; Zero Based
                                <1>
0000A608 8935[5C7F0100]         <1>     mov     [CDLF_FNAddress], esi
0000A60E 66A3[5A7F0100]         <1>     mov     [CDLF_AttributesMask], ax
0000A614 66890D[607F0100]       <1>     mov     [CDLF_DEType], cx
                                <1>
0000A61B 31DB                   <1>     xor     ebx, ebx
0000A61D 881D[707F0100]         <1>     mov     [PreviousAttr], bl ; 0  ; 13/02/2016
                                <1>
0000A623 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
0000A629 381D[9D7E0100]         <1>     cmp     byte [DirBuff_ValidData], bl ; 0
0000A62F 761D                   <1>     jna     short loc_lcdf_reload_current_dir2
0000A631 8A1D[9B7E0100]         <1>         mov     bl, [DirBuff_DRV]
```

```
0000A637 80EB41              <1>     sub     bl, 'A'
0000A63A 38DF                <1>     cmp     bh, bl
0000A63C 750E                <1>     jne     short loc_lcdf_reload_current_dir1
0000A63E 8B15[A27E0100]      <1>     mov     edx, [DirBuff_Cluster]
0000A644 3B15[70770100]      <1>     cmp     edx, [Current_Dir_FCluster]
0000A64A 7412                <1>     je      short loc_cdir_locatefile_search
                             <1>
                             <1> loc_lcdf_reload_current_dir1:
0000A64C 30DB                <1>     xor     bl, bl
                             <1> loc_lcdf_reload_current_dir2:
0000A64E 89DE                <1>     mov     esi, ebx
0000A650 81C600010900        <1>         add     esi, Logical_DOSDisks
0000A656 E866000000          <1>     call    reload_current_directory
0000A65B 734F                <1>     jnc     short loc_locatefile_search_again
0000A65D C3                  <1>     retn
                             <1>
                             <1> loc_cdir_locatefile_search:
0000A65E 31DB                <1>     xor     ebx, ebx
0000A660 55                  <1>     push    ebp ; 20/11/2017
0000A661 E87D000000          <1>     call    find_directory_entry
0000A666 5D                  <1>     pop     ebp ; 20/11/2017
0000A667 733B                <1>     jnc     short loc_cdir_locate_file_retn
                             <1>
                             <1> loc_locatefile_check_stc_reason:
0000A669 08ED                <1>     or      ch, ch
0000A66B 7436                <1>     jz      short loc_cdir_locate_file_stc_retn
                             <1>
                             <1> loc_locatefile_check_next_entryblock:
                             <1>     ;mov    bh, [Current_Drv]
                             <1>     ;sub    bl, bl
                             <1>     ;movzx  esi, bx
                             <1>     ; 28/07/2022
0000A66D 31DB                <1>         xor ebx, ebx
0000A66F 8A3D[76770100]      <1>     mov     bh, [Current_Drv]
0000A675 89DE                <1>     mov     esi, ebx
0000A677 81C600010900        <1>     add      esi, Logical_DOSDisks
                             <1>
0000A67D 803D[74770100]00    <1>     cmp     byte [Current_Dir_Level], 0
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     jna     short loc_locatefile_check_FAT_type
                             <1>
                             <1>     cmp     byte [Current_FATType], 1
                             <1>     jnb     short loc_locatefile_load_subdir_cluster
                             <1>     retn
                             <1> %else
                             <1>     ; 19/12/2025
0000A684 7709                <1>     ja      short loc_locatefile_load_subdir_cluster
                             <1> %endif
                             <1>
                             <1> loc_locatefile_check_FAT_type:
0000A686 803D[75770100]03    <1>     cmp     byte [Current_FATType], 3
0000A68D 7215                <1>     jb      short loc_cdir_locate_file_retn
                             <1>
                             <1> loc_locatefile_load_subdir_cluster:
0000A68F A1[A27E0100]        <1>     mov     eax, [DirBuff_Cluster]
0000A694 E8F0180000          <1>     call    get_next_cluster
0000A699 730A                <1>     jnc     short loc_locatefile_next_cluster
0000A69B 09C0                <1>     or      eax, eax
0000A69D 7504                <1>     jnz     short loc_locatefile_drive_not_ready_read_err
                             <1>     ;stc
                             <1>     ; 28/07/2022
                             <1> ;loc_locatefile_file_notfound:
                             <1>     ;mov    eax, 2 ; File/Directory/VolName not found
                             <1>     ;xor    eax, eax
                             <1>     ; 26/08/2024
                             <1>     ; eax = 0
0000A69F B002                <1>     mov     al, 2
0000A6A1 F9                  <1>     stc
0000A6A2 C3                  <1>     retn
                             <1>
                             <1> loc_locatefile_drive_not_ready_read_err:
                             <1>     ;mov    eax, 17 ;Drive not ready or read error
                             <1> loc_cdir_locate_file_stc_retn:
0000A6A3 F5                  <1>     cmc ;stc
                             <1> loc_cdir_locate_file_retn:
0000A6A4 C3                  <1>     retn
                             <1>
                             <1> loc_locatefile_next_cluster:
0000A6A5 E89B1A0000          <1>     call    load_FAT_sub_directory
                             <1>     ;jc     short loc_locatefile_drive_not_ready_read_err
0000A6AA 72F8                <1>     jc      short loc_cdir_locate_file_retn
                             <1>
                             <1> loc_locatefile_search_again:
0000A6AC 8B35[5C7F0100]      <1>     mov     esi, [CDLF_FNAddress]
0000A6B2 66A1[5A7F0100]      <1>     mov     ax, [CDLF_AttributesMask]
0000A6B8 668B0D[607F0100]    <1>     mov     cx, [CDLF_DEType]
0000A6BF EB9D                <1>     jmp     short loc_cdir_locatefile_search
                             <1>
                             <1> reload_current_directory:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 06/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 13/06/2010
                             <1>     ; 22/09/2009
                             <1>         ;
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Dos drive description table address
                             <1>
                             <1>     ;mov    al, [esi+LD_FATType]
0000A6C1 A0[75770100]        <1>     mov     al, [Current_FATType]
0000A6C6 3C02                <1>     cmp     al, 2
0000A6C8 770F                <1>     ja      short loc_reload_FAT_sub_directory
0000A6CA 8A25[74770100]      <1>     mov     ah, [Current_Dir_Level]
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     or      al, al
                             <1>     jz      short loc_reload_FS_directory
                             <1> %endif
0000A6D0 08E4                <1>     or      ah, ah
0000A6D2 7505                <1>     jnz     short loc_reload_FAT_sub_directory
                             <1> loc_reload_FAT_12_16_root_directory:
                             <1>     ;call   load_FAT_root_directory
                             <1>     ;retn
                             <1>     ; 28/07/2022
0000A6D4 E9EE190000          <1>     jmp     load_FAT_root_directory
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> loc_reload_FS_directory:
                             <1>     and     ah, ah
                             <1>     jnz     short loc_reload_FS_sub_directory
                             <1> loc_reload_FS_root_directory:
                             <1>     ;call   load_FS_root_directory
```

```
                                <1>     ;retn
                                <1>     ; 28/07/2022
                                <1>     jmp     load_FS_root_directory
                                <1> loc_reload_FS_sub_directory:
                                <1>     mov     eax, [Current_Dir_FCluster]
                                <1>     ;call   load_FS_sub_directory
                                <1>     ;retn
                                <1>     jmp     load_FS_sub_directory
                                <1> %endif
                                <1>
                                <1> loc_reload_FAT_sub_directory:
0000A6D9 A1[70770100]           <1>     mov     eax, [Current_Dir_FCluster]
                                <1>     ;call   load_FAT_sub_directory
                                <1>     ;retn
                                <1>     ; 28/07/2022
0000A6DE E9621A0000             <1>     jmp     load_FAT_sub_directory
                                <1>
                                <1> find_directory_entry:
                                <1>     ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 02/03/2021 (TRDOS 386 v2.0.3) ((BugFix))
                                <1>     ; 14/02/2016
                                <1>     ; 13/02/2016
                                <1>     ; 10/02/2016
                                <1>     ; 06/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 14/08/2010 (DIR.ASM, "proc_find_direntry")
                                <1>     ; 19/09/2009
                                <1>     ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Sub Dir or File Name Address
                                <1>     ;       AL = Attributes Mask
                                <1>     ;       (<AL AND EntryAttrib> must be equal to AL)
                                <1>     ;       AH = Negative Attributes Mask (If AH>0)
                                <1>     ;       (<AH AND EntryAttrib> must be ZERO)
                                <1>     ;       CH > 0 Find First Free Dir Entry or Deleted Entry
                                <1>     ;       CL = 0 -> Return the First Free Dir Entry
                                <1>     ;       CL = E5h -> Return the 1st deleted entry
                                <1>     ;       CL = FFh -> Return the 1st deleted or free entry
                                <1>     ;       CL > 0 and CL <> E5h and CL <> FFh -> Return the first
                                <1>     ;             proper entry (which fits with Atributes Masks)
                                <1>     ;       CX = 0 -> Find Valid File/Directory/VolumeName
                                <1>     ;       ? = Any One Char
                                <1>     ;       * = Every Chars
                                <1>     ;       EBX = Current Dir Entry (BX)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       EDI = Directory Entry Address (in DirectoryBuffer)
                                <1>     ;       ESI = Sub Dir or File Name Address
                                <1>     ;       CF = 0 -> No Error, Proper Entry,
                                <1>     ;       DL = Attributes
                                <1>     ;       DH = Previous Entry Attr (LongName Check)
                                <1>     ;       AL > 0 -> Ambiguous filename wildcard "?" used
                                <1>     ;       AH > 0 -> Ambiguous filename wildcard "*" used
                                <1>     ;       AX = 0 -> Filename full fits with directory entry
                                <1>     ;       EBX = CurrentDirEntry (BX)
                                <1>     ;       CH = The 1st Name Char of Current Dir Entry
                                <1>     ;       CF = 1 -> Proper entry not found, Error Code in AX/AL
                                <1>     ;       CL = 0 and CH = 0 -> Free Entry (End Of Dir)
                                <1>     ;       CL = 0 and CH = E5h -> Deleted Entry fits with filters
                                <1>     ;       CL > 0 -> Entry not found, CH invalid
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, EDI, EBP will be changed)
                                <1>
0000A6E3 663B1D[A07E0100]       <1>     cmp     bx, [DirBuff_LastEntry]
0000A6EA 7728                   <1>     ja      short loc_ffde_stc_retn_255 ; 28/07/2022
                                <1>
                                <1>     ;mov    [DirBuff_CurrentEntry], bx
                                <1>
0000A6EC BF00000800             <1>     mov     edi, Directory_Buffer
0000A6F1 66A3[6C7F0100]         <1>     mov     [FDE_AttrMask], ax
                                <1>
0000A6F7 29C0                   <1>     sub     eax, eax
                                <1>
                                <1>     ;;mov   [PreviousAttr], al ; 0 ;; 13/02/2016
0000A6F9 66A3[6E7F0100]         <1>     mov     [AmbiguousFileName], ax ; 0
                                <1>
0000A6FF 6689D8                 <1>     mov     ax, bx
                                <1>     ;shl    ax, 5 ; ; * 32 ; Directory entry size
                                <1>     ; 28/07/2022
0000A702 C1E005                 <1>     shl     eax, 5
0000A705 01C7                   <1>     add     edi, eax
                                <1>
0000A707 08ED                   <1>     or      ch, ch
                                <1>     ;jnz    loc_find_free_deleted_entry_0
                                <1>     ; 28/07/2022
0000A709 7405                   <1>     jz      short loc_fde_any_valid_entry_opt
0000A70B E911010000             <1>     jmp     loc_find_free_deleted_entry_0
                                <1>
                                <1> loc_fde_any_valid_entry_opt:
0000A710 08C9                   <1>     or      cl, cl
                                <1>         ;jnz        loc_ffde_stc_retn_255
                                <1>     ; 28/07/2022
0000A712 742E                   <1>     jz      short check_find_dir_entry
                                <1>
                                <1>     ; 28/07/2022
                                <1> loc_ffde_stc_retn_255:
                                <1>     ; 02/03/2021 (TRDOS 386 v2.0.3) ((BugFix))
                                <1>     ; (ECX must not be > 65535)
                                <1>     ; ((because 'loc_ccd_save_current_dir'
                                <1>     ;  sets CX to 32 for 'rep movsd'))
0000A714 66B9FFFF               <1>     mov     cx, 0FFFFh
                                <1>     ;xor    ecx, ecx
                                <1>     ;dec    ecx ; 0FFFFFFFFh
                                <1>     ;xor    eax, eax
                                <1> loc_find_direntry_stc_retn:
                                <1> loc_check_ffde_retn_1:
                                <1>     ;mov    ax, 2
                                <1>     ;mov    eax, 2 ; File Not Found
                                <1>     ; 28/07/2022
0000A718 29C0                   <1>     sub     eax, eax
0000A71A B002                   <1>     mov     al, 2
0000A71C 8A35[707F0100]         <1>     mov     dh, [PreviousAttr]
0000A722 66891D[9E7E0100]       <1>     mov     [DirBuff_CurrentEntry], bx
0000A729 F9                     <1>     stc
0000A72A C3                     <1>     retn
                                <1>
                                <1>     ; 28/07/2022
                                <1> loc_find_dir_next_entry_prevdeleted:
0000A72B 80CA80                 <1>     or      dl, 80h  ; Bit 7 -> deleted entry sign
                                <1>     ;jmp    short loc_find_dir_next_entry
                                <1>
                                <1>     ; 28/07/2022
                                <1> loc_find_dir_next_entry:
0000A72E 8815[707F0100]         <1>     mov     byte [PreviousAttr], dl ; LongName check
                                <1> loc_find_dir_next_entry_1:
```

```
0000A734 5E                  <1>     pop     esi
0000A735 83C720              <1>     add     edi, 32
                             <1>     ;;inc   word [DirBuff_EntryCounter]
                             <1>     ;inc    bx
                             <1>     ; 28/07/2022
0000A738 43                  <1>     inc     ebx
0000A739 663B1D[A07E0100]    <1>     cmp     bx, [DirBuff_LastEntry]
0000A740 77D2                <1>     ja      short loc_ffde_stc_retn_255
                             <1>         ; 28/07/2022
                             <1>     ;jmp    short check_find_dir_entry
                             <1>
                             <1> check_find_dir_entry:
0000A742 66A1[6C7F0100]      <1>     mov     ax, [FDE_AttrMask]
0000A748 8A2F                <1>     mov     ch, [edi]
0000A74A 80FD00              <1>     cmp     ch, 0 ; Is it never used entry?
                             <1>     ;jna    loc_find_direntry_stc_retn
                             <1>     ; 28/07/2022
0000A74D 7702                <1>     ja      short loc_fde_check_attrib
                             <1>     ; end of directory entries
0000A74F EBC7                <1>     jmp     loc_find_direntry_stc_retn
                             <1> loc_fde_check_attrib:
0000A751 56                  <1>     push    esi
0000A752 8A570B              <1>     mov     dl, [edi+0Bh] ; File attributes
0000A755 80FDE5              <1>     cmp     ch, 0E5h ; Is it a deleted file?
0000A758 74D1                <1>     je      short loc_find_dir_next_entry_prevdeleted
                             <1>
0000A75A 80FA0F              <1>     cmp     dl, 0Fh ; longname sub component check
0000A75D 7505                <1>     jne     short loc_check_attributes_mask
0000A75F E8A6010000          <1>     call    save_longname_sub_component
                             <1>
                             <1> loc_check_attributes_mask:
0000A764 88C6                <1>     mov     dh, al
0000A766 20D6                <1>     and     dh, dl
                             <1>     ; 28/07/2022
0000A768 38F0                <1>     cmp     al, dh
0000A76A 75C2                <1>     jne     short loc_find_dir_next_entry
0000A76C 20D4                <1>     and     ah, dl
0000A76E 75BE                <1>         jnz short loc_find_dir_next_entry
0000A770 80FA0F              <1>     cmp     dl, 0Fh
0000A773 7515                <1>     jne     short pass_direntry_attr_check
                             <1>
0000A775 3C0F                <1>     cmp     al, 0Fh ; AL = 0Fh -> find long name
0000A777 75B5                <1>     jne     short loc_find_dir_next_entry
                             <1>
0000A779 5E                  <1>     pop     esi
                             <1>     ;xor    ax, ax
                             <1>     ; 28/07/2022
                             <1>     ;sub    eax, eax
0000A77A 30C0                <1>     xor     al, al
0000A77C 8A35[707F0100]      <1>     mov     dh, [PreviousAttr]
0000A782 66891D[9E7E0100]    <1>     mov     [DirBuff_CurrentEntry], bx
0000A789 C3                  <1>     retn
                             <1>
                             <1> pass_direntry_attr_check:
0000A78A 89FD                <1>     mov     ebp, edi ; 14/02/2016
                             <1>     ;mov    ecx, 8
                             <1>     ; 28/07/2022
0000A78C 29C9                <1>     sub     ecx, ecx
0000A78E B108                <1>     mov     cl, 8
                             <1> loc_lodsb_find_dir:
0000A790 AC                  <1>     lodsb
0000A791 3C2A                <1>     cmp     al, '*'
0000A793 7508                <1>     jne     short pass_fde_ambiguous1_check
0000A795 FE05[6F7F0100]      <1>         inc     byte [AmbiguousFileName+1]
0000A79B EB23                <1>     jmp     short loc_check_direntry_extension
                             <1>
                             <1> pass_fde_ambiguous1_check:
0000A79D 3C3F                <1>     cmp     al, '?'
0000A79F 750D                <1>     jne     short pass_fde_ambiguous2_check
0000A7A1 FE05[6E7F0100]      <1>     inc     byte [AmbiguousFileName]
0000A7A7 803F20              <1>     cmp     byte [edi], 20h
0000A7AA 763E                <1>     jna     short loc_find_dir_next_entry_ebp
0000A7AC EB0F                <1>     jmp     short loc_scasb_find_dir_inc_di
                             <1>
                             <1> pass_fde_ambiguous2_check:
0000A7AE 3C20                <1>     cmp     al, 20h
0000A7B0 7507                <1>     jne     short loc_scasb_find_dir
0000A7B2 803F20              <1>     cmp     byte [edi], 20h
0000A7B5 7533                <1>     jne     short loc_find_dir_next_entry_ebp
0000A7B7 EB07                <1>     jmp     short loc_check_direntry_extension
                             <1>
                             <1> loc_scasb_find_dir:
0000A7B9 3A07                <1>     cmp     al, [edi]
0000A7BB 752D                <1>     jne     short loc_find_dir_next_entry_ebp
                             <1> loc_scasb_find_dir_inc_di:
0000A7BD 47                  <1>     inc     edi
0000A7BE E2D0                <1>     loop    loc_lodsb_find_dir
                             <1>
                             <1> loc_check_direntry_extension:
0000A7C0 BE08000000          <1>     mov     esi, 8
0000A7C5 89F7                <1>     mov     edi, esi ; 8
0000A7C7 033424              <1>     add     esi, [esp] ; Sub Dir or File Name Address
0000A7CA 01EF                <1>     add     edi, ebp
0000A7CC B103                <1>     mov     cl, 3
                             <1> loc_lodsb_find_dir_ext:
0000A7CE AC                  <1>     lodsb
0000A7CF 3C2A                <1>     cmp     al, '*'
0000A7D1 7508                <1>     jne     short pass_fde_ambiguous3_check
0000A7D3 FE05[6F7F0100]      <1>     inc     byte [AmbiguousFileName+1]
0000A7D9 EB1F                <1>     jmp     short loc_find_dir_proper_direntry
                             <1>
                             <1> pass_fde_ambiguous3_check:
0000A7DB 3C3F                <1>     cmp     al, '?'
0000A7DD 7512                <1>     jne     short pass_fde_ambiguous4_check
0000A7DF FE05[6E7F0100]      <1>     inc     byte [AmbiguousFileName]
0000A7E5 803F20              <1>     cmp     byte [edi], 20h
                             <1>     ;jna    short loc_find_dir_next_entry_ebp
                             <1>     ;jmp    short loc_scasb_find_dir_ext_inc_di
                             <1>     ; 28/07/2022
0000A7E8 7732                <1>     ja      short loc_scasb_find_dir_ext_inc_di
                             <1>
                             <1> loc_find_dir_next_entry_ebp:
0000A7EA 89EF                <1>     mov     edi, ebp ; 14/02/2016
0000A7EC E93DFFFFFF          <1>     jmp     loc_find_dir_next_entry ; 28/07/2022
                             <1>
                             <1> pass_fde_ambiguous4_check:
0000A7F1 3C20                <1>     cmp     al, 20h
0000A7F3 7523                <1>     jne     short loc_scasb_find_dir_ext
0000A7F5 803F20              <1>     cmp     byte [edi], 20h
                             <1>     ; 28/07/2022
0000A7F8 75F0                <1>     jne     short loc_find_dir_next_entry_ebp
                             <1>     ;jmp    short loc_find_dir_proper_direntry
                             <1>
                             <1> loc_find_dir_proper_direntry:
```

```
0000A7FA 30C9                <1>     xor     cl, cl
                             <1> loc_find_dir_proper_direntry_1:
0000A7FC 5E                  <1>     pop     esi
0000A7FD 89EF                <1>         mov     edi, ebp
0000A7FF 8A2F                <1>     mov     ch, [edi]
0000A801 8A570B              <1>     mov     dl, [edi+0Bh] ; Dir entry attributes
0000A804 66A1[6E7F0100]      <1>     mov     ax, [AmbiguousFileName]
                             <1> loc_find_dir_proper_direntry_2:
0000A80A 8A35[707F0100]      <1>     mov     dh, [PreviousAttr]
0000A810 66891D[9E7E0100]    <1>     mov     [DirBuff_CurrentEntry], bx
0000A817 C3                  <1>     retn
                             <1>
                             <1> loc_scasb_find_dir_ext:
0000A818 3A07                <1>     cmp     al, [edi]
0000A81A 75CE                <1>     jne     short loc_find_dir_next_entry_ebp
                             <1> loc_scasb_find_dir_ext_inc_di:
0000A81C 47                  <1>     inc     edi
0000A81D E2AF                <1>     loop    loc_lodsb_find_dir_ext
0000A81F EBDB                <1>     jmp     short loc_find_dir_proper_direntry_1
                             <1>
                             <1> loc_find_free_deleted_entry_0:
0000A821 66A1[6C7F0100]      <1>     mov     ax, [FDE_AttrMask]
0000A827 8A2F                <1>     mov     ch, [edi]
0000A829 8A570B              <1>     mov     dl, [edi+0Bh] ; File attributes
0000A82C 08C9                <1>     or      cl, cl
0000A82E 7407                <1>     jz      short loc_check_ffde_0_repeat
                             <1>     ;cmp    cl, 0E5h
                             <1>     ;je     short pass_loc_check_ffde_0_err
0000A830 80F9FF              <1>     cmp     cl, 0FFh
0000A833 7430                <1>     je      short loc_find_free_deleted_entry_1
0000A835 EB4A                <1>     jmp     short pass_loc_check_ffde_0_err
                             <1>
                             <1> loc_check_ffde_0_repeat:
0000A837 08ED                <1>     or      ch, ch
0000A839 7510                <1>     jnz     short loc_check_ffde_0_next
                             <1>
                             <1> loc_check_ffde_retn_2:
                             <1>     ;sub    ax, ax
                             <1>     ; 28/07/2022
0000A83B 29C0                <1>     sub     eax, eax
0000A83D 8A35[707F0100]      <1>     mov     dh, [PreviousAttr]
0000A843 66891D[9E7E0100]    <1>     mov     [DirBuff_CurrentEntry], bx
0000A84A C3                  <1>     retn
                             <1>
                             <1> loc_check_ffde_0_next:
                             <1>     ;inc    bx
                             <1>     ; 28/07/2022
0000A84B 43                  <1>     inc     ebx
0000A84C 83C720              <1>     add     edi, 32
                             <1>     ;inc    word [DirBuff_EntryCounter]
                             <1>
0000A84F 663B1D[A07E0100]    <1>         cmp bx, [DirBuff_LastEntry]
                             <1>     ;ja     short loc_ffde_stc_retn_255
                             <1>     ; 07/08/2022
0000A856 773A                <1>     ja      short jmp_ffde_stc_retn_255
                             <1>
0000A858 8815[707F0100]      <1>     mov     [PreviousAttr], dl
0000A85E 8A2F                <1>     mov     ch, [edi]
0000A860 8A570B              <1>     mov     dl, [edi+0Bh] ; file attributes
0000A863 EBD2                <1>     jmp     short loc_check_ffde_0_repeat
                             <1>
                             <1> loc_find_free_deleted_entry_1:
0000A865 28D2                <1>     sub     dl, dl
                             <1> loc_find_free_deleted_entry_2:
0000A867 20ED                <1>     and     ch, ch
0000A869 74D0                <1>     jz      short loc_check_ffde_retn_2
0000A86B 80FDE5              <1>     cmp     ch, 0E5h
0000A86E 74CB                <1>     je      short loc_check_ffde_retn_2
                             <1>     ;inc    bx
                             <1>     ; 28/07/2022
0000A870 43                  <1>     inc     ebx
0000A871 83C720              <1>     add     edi, 32
0000A874 663B1D[A07E0100]    <1>     cmp     bx, [DirBuff_LastEntry]
                             <1>     ;ja     short loc_ffde_stc_retn_255
                             <1>     ; 07/08/2022
0000A87B 7715                <1>     ja      short jmp_ffde_stc_retn_255
                             <1>
0000A87D 8A2F                <1>     mov     ch, [edi]
0000A87F EBE6                <1>     jmp     short loc_find_free_deleted_entry_2
                             <1>
                             <1> pass_loc_check_ffde_0_err:
0000A881 38CD                <1>     cmp     ch, cl
0000A883 741F                <1>     je      short loc_check_ffde_attrib
                             <1>
                             <1>     ;inc    bx
                             <1>     ; 28/07/2022
0000A885 43                  <1>     inc     ebx
0000A886 83C720              <1>     add     edi, 32
0000A889 663B1D[A07E0100]    <1>     cmp     bx, [DirBuff_LastEntry]
                             <1>         ;ja loc_ffde_stc_retn_255
                             <1>     ; 28/07/2022
0000A890 7605                <1>     jna     short loc_ffe_save_prev_attr
                             <1> jmp_ffde_stc_retn_255:     ; 07/08/2022
0000A892 E97DFEFFFF          <1>     jmp     loc_ffde_stc_retn_255
                             <1>
                             <1> loc_ffe_save_prev_attr: ; 28/07/2022
0000A897 8815[707F0100]      <1>     mov     [PreviousAttr], dl
0000A89D 8A2F                <1>     mov     ch, [edi]
0000A89F 8A570B              <1>     mov     dl, [edi+0Bh]
0000A8A2 EBDD                <1>     jmp     short pass_loc_check_ffde_0_err
                             <1>
                             <1> loc_check_ffde_attrib:
0000A8A4 88C6                <1>     mov     dh, al
0000A8A6 20D6                <1>     and     dh, dl
0000A8A8 38F0                <1>     cmp     al, dh
0000A8AA 759F                <1>     jne     short loc_check_ffde_0_next
0000A8AC 20D4                <1>     and     ah, dl
0000A8AE 759B                <1>     jnz     short loc_check_ffde_0_next
0000A8B0 30C9                <1>     xor     cl, cl
0000A8B2 EB87                <1>         jmp short loc_check_ffde_retn_2
                             <1>
                             <1> convert_file_name:
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 06/03/2016
                             <1>     ; 11/02/2016
                             <1>     ; 07/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 06/10/2009
                             <1>     ; 2005
                             <1>     ;
                             <1>     ; INPUT  ->
                             <1>     ;       ESI = Dot File Name Location
                             <1>     ;       EDI = Dir Entry Format File Name Location
                             <1>     ; OUTPUT ->
                             <1>     ;       EDI = Dir Entry Format File Name Location
```

```
                                <1>     ;       ESI = Dot File Name Location (capitalized)
                                <1>     ;
                                <1>     ; (ECX, AL will be changed)
                                <1>
0000A8B4 56                     <1>     push    esi
0000A8B5 57                     <1>     push    edi
                                <1>
                                <1>     ;mov    ecx, 11
                                <1>     ; 29/07 2022
0000A8B6 29C9                   <1>     sub     ecx, ecx
0000A8B8 B10B                   <1>     mov     cl, 11
0000A8BA B020                   <1>     mov     al, 20h
0000A8BC F3AA                   <1>     rep     stosb
                                <1>
0000A8BE 8B3C24                 <1>     mov     edi, [esp]
                                <1>
0000A8C1 B10C                   <1>     mov     cl, 12 ; file name length (max.)
                                <1>     ; 06/03/2016
                                <1>     ; Directory entry name limit (11 bytes) check for
                                <1>     ; 'rename_directory_entry' procedure.
                                <1>     ; (EDI points to Directory Entry)
                                <1>     ; (If the file name would not contain a dot
                                <1>     ; and file name length would be 12, this would cause to
                                <1>     ; overwrite the attributes byte of the directory entry.)
                                <1>     ;
0000A8C3 B50B                   <1>     mov     ch, 11 ; directory entry's name length
                                <1> loc_check_first_dot:
0000A8C5 8A06                   <1>     mov     al, [esi]
0000A8C7 3C2E                   <1>     cmp     al, 2Eh
0000A8C9 750C                   <1>     jne     short pass_check_first_dot
0000A8CB 8807                   <1>     mov     [edi], al
0000A8CD 47                     <1>     inc     edi
0000A8CE 46                     <1>     inc     esi
0000A8CF FEC9                   <1>     dec     cl
0000A8D1 75F2                   <1>     jnz     short loc_check_first_dot
                                <1>     ;;(ecx <= 12)
                                <1>     ;;loop  loc_check_first_dot
0000A8D3 EB30                   <1>     jmp     short stop_convert_file
                                <1>
                                <1> loc_get_fchar:
0000A8D5 8A06                   <1>     mov     al, [esi]
                                <1> pass_check_first_dot:
0000A8D7 3C61                   <1>     cmp     al, 61h ; 'a'
0000A8D9 7208                   <1>     jb      short pass_name_capitalize
0000A8DB 3C7A                   <1>     cmp     al, 7Ah ; 'z'
0000A8DD 7704                   <1>     ja      short pass_name_capitalize
0000A8DF 24DF                   <1>     and     al, 0DFh
0000A8E1 8806                   <1>     mov     [esi], al
                                <1> pass_name_capitalize:
0000A8E3 3C21                   <1>     cmp     al, 21h
0000A8E5 721E                   <1>     jb      short stop_convert_file
0000A8E7 3C2E                   <1>     cmp     al, 2Eh ; '.'
0000A8E9 750C                   <1>     jne     short pass_dot_space
                                <1> add_dot_space:
0000A8EB 80F904                 <1>     cmp     cl, 4
0000A8EE 760E                   <1>     jna     short inc_and_loop
0000A8F0 47                     <1>     inc     edi
0000A8F1 FECD                   <1>     dec     ch ; 06/03/2016
0000A8F3 FEC9                   <1>     dec     cl
0000A8F5 EBF4                   <1>     jmp     short add_dot_space
                                <1>
                                <1>     ;mov    al, 4
                                <1>     ;cmp    cl, al
                                <1>     ;jna    short inc_and_loop
                                <1>     ;sub    cl, al
                                <1>     ;add    edi, ecx
                                <1>     ;mov    cl, al
                                <1>     ;jmp    short inc_and_loop
                                <1>
                                <1> pass_dot_space:
0000A8F7 8807                   <1>     mov     [edi], al
                                <1> loc_after_double_dot:
                                <1>     ; 06/03/2016
0000A8F9 FECD                   <1>     dec     ch ; count down for 11 bytes dir entry limit
0000A8FB 740A                   <1>     jz      short stop_convert_file_x
0000A8FD 47                     <1>     inc     edi
                                <1> inc_and_loop:
0000A8FE FEC9                   <1>     dec     cl ; count down for 12 bytes filename limit
0000A900 7403                   <1>     jz      short stop_convert_file
0000A902 46                     <1>     inc     esi
                                <1>     ;;(ecx <= 12)
                                <1>     ;;loop  loc_get_fchar
0000A903 EBD0                   <1>     jmp     short loc_get_fchar
                                <1>
                                <1> stop_convert_file:
                                <1>     ; 06/03/2016
0000A905 30ED                   <1>     xor     ch, ch
                                <1>     ; ECX < 256 ; 'find_first_file' -> xor cl, cl
                                <1> stop_convert_file_x:
0000A907 5F                     <1>     pop     edi
0000A908 5E                     <1>     pop     esi
0000A909 C3                     <1>     retn
                                <1>
                                <1> save_longname_sub_component:
                                <1>     ; 13/02/2016
                                <1>     ; 06/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 28/02/2010
                                <1>     ; 17/10/2009
                                <1>     ; INPUT ->
                                <1>     ;       EDI = Directory Entry
                                <1>     ;       // This procedure is called
                                <1>     ;       // from 'find_directory_entry' procedure.
                                <1>     ;       // If the last entry returns with
                                <1>     ;       // a non-zero LongnameFound value and
                                <1>     ;       // if LFN_CheckSum value is equal to
                                <1>     ;       // the next shortname checksum,
                                <1>     ;       // long name is valid.
                                <1>     ;       // If a longname is longer than 65 bytes,
                                <1>     ;       // it is invalid for trdos. (>45h)
                                <1>
0000A90A 57                     <1>     push    edi
0000A90B 56                     <1>     push    esi
                                <1>     ;push   ebx
                                <1>     ;push   ecx
                                <1>     ;push   edx
0000A90C 50                     <1>     push    eax
                                <1>
0000A90D 29C9                   <1>     sub     ecx, ecx
                                <1>     ;sub    eax, eax
0000A90F B11A                   <1>     mov     cl, 26
                                <1>
0000A911 0FB607                 <1>     movzx   eax, byte [edi] ; LDIR_Order
0000A914 3C41                   <1>     cmp     al, 41h  ; 40h (last long entry sign) + 1
0000A916 722B                   <1>     jb      short pass_pslnsc_last_long_entry
```

```
                                <1>
0000A918 88C4                   <1>     mov     ah, al
0000A91A 80EC40                 <1>     sub     ah, 40h
0000A91D 8825[727F0100]         <1>     mov     [LFN_EntryLength], ah
                                <1>
0000A923 3C45                   <1>     cmp     al, 45h  ; 40h (last long entry sign) + 5
                                <1>             ; Max 130 byte length is usable in TRDOS
                                <1> ; 26*5 = 130
0000A925 7753                   <1>     ja      short loc_pslnsc_retn
                                <1>
0000A927 2407                   <1>     and     al, 07h ; 0Fh
0000A929 A2[717F0100]           <1>     mov     [LongNameFound], al
                                <1>
0000A92E FEC8                   <1>     dec     al
                                <1>     ;mov    cl, 26
0000A930 F6E1                   <1>     mul     cl
                                <1>
0000A932 89C6                   <1>     mov     esi, eax
0000A934 01CE                   <1>     add     esi, ecx
                                <1>             ; to make is an ASCIIZ string
                                <1>             ; with ax+26 bytes length
0000A936 81C6[747F0100]         <1>     add     esi, LongFileName
0000A93C 66C7060000             <1>     mov     word [esi], 0
0000A941 EB16                   <1>     jmp     short loc_pslsc_move_ldir_name2
                                <1>
                                <1> pass_pslnsc_last_long_entry:
0000A943 3C04                   <1>     cmp     al, 04h
0000A945 7733                   <1>     ja      short loc_pslnsc_retn
0000A947 FE0D[717F0100]         <1>     dec     byte [LongNameFound]
0000A94D 3A05[717F0100]         <1>     cmp     al, [LongNameFound]
0000A953 7525                   <1>     jne     short loc_pslnsc_retn
                                <1>
                                <1> loc_pslsc_move_ldir_name1:
0000A955 FEC8                   <1>     dec     al
                                <1>     ;mov    cl, 26
0000A957 F6E1                   <1>     mul     cl
                                <1>
                                <1> loc_pslsc_move_ldir_name2:
0000A959 8A4F0D                 <1>     mov     cl, [edi+0Dh] ; long name checksum
0000A95C 880D[737F0100]         <1>     mov     [LFN_CheckSum], cl
0000A962 89FE                   <1>     mov     esi, edi ; LDIR_Order
0000A964 BF[747F0100]           <1>     mov     edi, LongFileName
0000A969 01C7                   <1>     add     edi, eax
0000A96B 46                     <1>     inc     esi
0000A96C B105                   <1>     mov     cl, 5 ; chars 1 to 5
0000A96E 66F3A5                 <1>     rep     movsw
0000A971 83C603                 <1>     add     esi, 3
0000A974 A5                     <1>     movsd   ; char 6 & 7
0000A975 A5                     <1>     movsd   ; char 8 & 9
0000A976 A5                     <1>     movsd   ; char 10 & 11
0000A977 46                     <1>     inc     esi
0000A978 46                     <1>     inc     esi
0000A979 A5                     <1>     movsd    ; char 12 & 13
                                <1>
                                <1> loc_pslnsc_retn:
0000A97A 58                     <1>     pop     eax
                                <1>     ;pop    edx
                                <1>     ;pop    ecx
                                <1>     ;pop    ebx
0000A97B 5E                     <1>     pop     esi
0000A97C 5F                     <1>     pop     edi
                                <1>
0000A97D C3                     <1>     retn
                                <1>
                                <1> parse_path_name:
                                <1>     ; 02/12/2025 (TRDOS 386 v2.0.10, BugFix)
                                <1>     ; 03/09/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 09/08/2022
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 10/02/2016
                                <1>     ; 08/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 10/009/2011 ('proc_parse_pathname')
                                <1>     ; 27/11/2009
                                <1>     ; 05/12/2004
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Beginning of ASCIIZ pathname string
                                <1>     ;        EDI = Destination Address
                                <1>     ;              (which is TR-DOS FindFile data buffer)
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 1 -> Error
                                <1>     ;            EAX = Error Code (AL)
                                <1>     ;
                                <1>     ; (Modified registers: eax, ecx, esi, edi)
                                <1>
                                <1>     ; Clear the pathname bytes in TR-DOS Findfile data buffer
0000A97E 57                     <1>     push    edi
                                <1>     ;mov    ecx, 20  ; 80 bytes
                                <1>     ; 29/07/2022
0000A97F 29C9                   <1>     sub     ecx, ecx
0000A981 B114                   <1>     mov     cl, 20
0000A983 31C0                   <1>     xor     eax, eax
0000A985 F3AB                   <1>     rep     stosd
0000A987 5F                     <1>     pop     edi
                                <1>
0000A988 668B06                 <1>     mov     ax, [esi]
0000A98B 80FC3A                 <1>     cmp     ah, ':'
0000A98E 741C                   <1>     je      short loc_ppn_change_drive
0000A990 A0[76770100]           <1>     mov     al, [Current_Drv]
0000A995 EB33                   <1>     jmp     short pass_ppn_change_drive
                                <1>
                                <1> pass_ppn_cdir:
0000A997 8B35[96800100]         <1>     mov     esi, [First_Path_Pos]
0000A99D AC                     <1>     lodsb
                                <1> loc_ppn_get_filename:
0000A99E 83C741                 <1>     add     edi, 65 ; FindFile_Name location
                                <1>     ; TRDOS Filename length must not be more than 12 bytes
                                <1>     ;mov    ecx, 12
0000A9A1 B10C                   <1>     mov     cl, 12
                                <1> loc_ppn_get_fnchar_next:
0000A9A3 AA                     <1>     stosb
0000A9A4 AC                     <1>     lodsb
0000A9A5 3C21                   <1>     cmp     al, 21h
0000A9A7 7269                   <1>     jb      short loc_ppn_clc_return
0000A9A9 E2F8                   <1>         loop    loc_ppn_get_fnchar_next
                                <1> loc_ppn_return:
0000A9AB C3                     <1>     retn
                                <1>
                                <1> loc_ppn_change_drive:
                                <1>     ; 29/07/2022
                                <1>     ; ecx = 0
0000A9AC 24DF                   <1>     and     al, 0DFh
0000A9AE 2C41                   <1>     sub     al, 'A' ; A:
0000A9B0 7264                   <1>     jc      short loc_ppn_invalid_drive
```

```
0000A9B2 3805[8D2E0100]      <1>     cmp     [Last_DOS_DiskNo], al
0000A9B8 725C                <1>     jb      short loc_ppn_invalid_drive
                             <1>
0000A9BA 46                  <1>     inc     esi
0000A9BB 46                  <1>     inc     esi
0000A9BC 8A26                <1>     mov     ah, [esi]
0000A9BE 80FC21              <1>     cmp     ah, 21h
0000A9C1 7307                <1>     jnb     short pass_ppn_change_drive
                             <1>
                             <1> loc_ppn_cmd_failed:
                             <1>     ; File or directory name is not existing
0000A9C3 8807                <1>     mov     [edi], al ; Drv
0000A9C5 66B80100            <1>     mov     ax, 1 ; eax = 1
                             <1>     ; TR-DOS Error Code 01h = Bad Command Argument
                             <1>     ; MS-DOS Error Code 01h : Invalid Function Number
                             <1>     ;stc
                             <1>     ; (MainProg ErrMsg: "Bad command or file name!")
0000A9C9 C3                  <1>     retn
                             <1>
                             <1> pass_ppn_change_drive:
0000A9CA 8935[96800100]      <1>     mov     [First_Path_Pos], esi
                             <1>     ;mov    dword [Last_Slash_Pos], 0
                             <1>     ; 29/07/2022
0000A9D0 890D[9A800100]      <1>     mov     [Last_Slash_Pos], ecx ; 0
0000A9D6 AA                  <1>     stosb
0000A9D7 8A06                <1>     mov     al, [esi]
                             <1> loc_scan_ppn_dslash:
0000A9D9 3C2F                <1>     cmp     al, '/'
0000A9DB 7506                <1>     jne     short loc_scan_next_slash_pos
0000A9DD 8935[9A800100]      <1>     mov     [Last_Slash_Pos], esi
                             <1> loc_scan_next_slash_pos:
0000A9E3 46                  <1>     inc     esi
0000A9E4 8A06                <1>     mov     al, [esi]
0000A9E6 3C20                <1>     cmp     al, 20h
0000A9E8 77EF                <1>     ja      short loc_scan_ppn_dslash
                             <1>     ;cmp    dword [Last_Slash_Pos], 0
                             <1>     ; 09/08/2022
                             <1>     ;cmp    [Last_Slash_Pos], ecx ; 0 ?
                             <1>     ;jna    short pass_ppn_cdir
                             <1>
0000A9EA 8B0D[9A800100]      <1>     mov     ecx, [Last_Slash_Pos]
                             <1>     ; 03/09/2024
                             <1>     ;jcxz   pass_ppn_cdir
                             <1>     ; 02/12/2025
0000A9F0 E3A5                <1>     jecxz   pass_ppn_cdir
                             <1>
0000A9F2 8B35[96800100]      <1>     mov     esi, [First_Path_Pos]
0000A9F8 29F1                <1>     sub     ecx, esi
0000A9FA 41                  <1>     inc     ecx
                             <1>     ;cmp    ecx, 64
0000A9FB 80F940              <1>     cmp     cl, 64
0000A9FE 7715                <1>     ja      short loc_ppn_invalid_drive_stc
                             <1>
0000AA00 89F8                <1>     mov     eax, edi ; Dest Dir String Location (65 bytes)
0000AA02 F3A4                <1>     rep     movsb
                             <1>     ;mov    [edi], cl ; 0, End of Dir String
0000AA04 8B35[9A800100]      <1>     mov     esi, [Last_Slash_Pos]
0000AA0A 46                  <1>     inc     esi
0000AA0B 89C7                <1>     mov     edi, eax
0000AA0D AC                  <1>     lodsb
0000AA0E 3C21                <1>     cmp     al, 21h
0000AA10 738C                <1>     jnb     short loc_ppn_get_filename
                             <1> loc_ppn_clc_return:
                             <1>     ;clc
0000AA12 31C0                <1>     xor     eax, eax
0000AA14 C3                  <1>     retn
                             <1>
                             <1> loc_ppn_invalid_drive_stc:
0000AA15 F5                  <1>     cmc     ; stc
                             <1> loc_ppn_invalid_drive:
                             <1>     ; cf = 1
                             <1>     ; The Drive Letter/Char < "A" or > "Z"
0000AA16 66B80F00            <1>     mov     ax, 0Fh
                             <1>     ; MS-DOS Error Code 0Fh = Disk Drive Invalid
                             <1>     ; (MainProg ErrMsg: "Drive not ready or read error!")
0000AA1A C3                  <1>     retn
                             <1>
                             <1> find_longname:
                             <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 13/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 24/01/2010 (DIR.ASM, 'proc_find_longname')
                             <1>     ; 17/10/2009
                             <1>
                             <1>     ; INPUT ->
                             <1>     ;       ESI = DOS short file name address
                             <1>     ;       for example: "filename.ext"
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;       ESI = ASCIIZ longname address (cf = 0)
                             <1>     ;       cf = 1 -> error number returns in EAX (AL)
                             <1>     ;       AL = 0 & CF=1 -> longname not found
                             <1>     ;            the file/directory has no longname
                             <1>     ;       cf = 0 -> AL = FAT Type
                             <1>
                             <1>     ; 17/10/2009
                             <1>     ; ASCIIZ string will be returned
                             <1>     ; as LongFileName
                             <1>     ; clearing/reset is not needed
                             <1>     ;mov    ecx, 33
                             <1>     ;mov    edi, LongFileName
                             <1>     ;sub    ax, ax ; 0
                             <1>     ;rep    stosw
                             <1>
                             <1>     ;mov    byte [LongNameFound], 0
                             <1>
                             <1>     ; ESI = ASCIIZ file/directory name address
                             <1>     ;   AL = Attributes AND mask
                             <1>     ;       (Result of AND must be equal to AL)
                             <1>     ;   AH = Negative attributes mask
                             <1>     ;       (Result of AND must be ZERO)
0000AA1B 66B80008            <1>     mov     ax, 0800h
                             <1>             ; it must not be volume name or longname
0000AA1F E856DFFFFF          <1>     call    find_first_file
0000AA24 7207                <1>     jc      short loc_fln_retn
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> loc_fln_check_FAT_Type:
                             <1>     cmp     byte [Current_FATType], 1
                             <1>     jnb     short loc_fln_check_longname_yes_sign
                             <1>
                             <1>     ;call   get_fs_longname
                             <1>     ;retn
```

```
                                <1>     ; 29/07/2022
                                <1>     jmp     get_fs_longname
                                <1> %endif
                                <1>
                                <1> loc_fln_check_longname_yes_sign:
0000AA26 08FF                   <1>     or      bh, bh
0000AA28 7504                   <1>     jnz     short loc_fln_check_longnamefound_number
                                <1> loc_fln_longname_not_found_retn:
0000AA2A 31C0                   <1>     xor     eax, eax
                                <1>     ; cf = 1 & al = 0 -> longname not found
0000AA2C F9                     <1>     stc
                                <1> loc_fln_retn:
0000AA2D C3                     <1>     retn
                                <1>
                                <1> loc_fln_check_longnamefound_number:
                                <1>     ; 'LongNameFound' is set by
                                <1>         ; by 'save_longname_sub_component'
                                <1>     ; which is called from
                                <1>     ; 'find_directory_entry'
                                <1>     ; which is called from
                                <1>     ; 'find_first_file'
                                <1>     ; It must 1 if the longname is valid
0000AA2E 803D[717F0100]01       <1>         cmp     byte [LongNameFound], 1
0000AA35 75F3                   <1>     jne     short loc_fln_longname_not_found_retn
                                <1>
                                <1> loc_fln_calculate_checksum:
0000AA37 E813000000             <1>     call    calculate_checksum
                                <1>     ; AL = shortname checksum
                                <1>
                                <1> loc_fln_longname_validation:
                                <1>     ; 'LFN_CheckSum' has been set already
                                <1>     ; by 'save_longname_sub_component'
                                <1>     ; which is called from
                                <1>     ; 'find_directory_entry'
                                <1>     ; which is called from
                                <1>     ; 'find_first_file'
0000AA3C 3805[737F0100]         <1>     cmp     [LFN_CheckSum], al
0000AA42 75E6                   <1>     jne     short loc_fln_longname_not_found_retn
                                <1>
0000AA44 BE[747F0100]           <1>     mov     esi, LongFileName
0000AA49 A0[75770100]           <1>     mov     al, [Current_FATType]
0000AA4E C3                     <1>     retn
                                <1>
                                <1> calculate_checksum:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 17/10/2009 (DIR.ASM, 'proc_calculate_checksum')
                                <1>         ;
                                <1>     ; INPUT ->
                                <1>     ;       ESI = 11 byte DOS File Name location
                                <1>     ;       (in DOS Directory Entry Format)
                                <1>     ; OUTPUT ->
                                <1>     ;        AL = 8 bit checksum (CRC) value
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, ESI)
                                <1>
                                <1>     ; Erdogan Tan [ 17-10-2009 ]
                                <1>     ;  'ror al, 1' instruction
                                <1>
                                <1>     ; Erdogan Tan [ 20-06-2004 ]
                                <1>     ; This 8086 assembly code is an original code
                                <1>     ; which is adapted from C code in
                                <1>     ; Microsoft FAT32 File System Specification
                                <1>     ; Version 1.03, December 6, 2000
                                <1>     ; Page 28
                                <1>
0000AA4F 30C0                   <1>     xor     al, al
                                <1>     ;mov    ecx, 11
                                <1>     ; 29/07/2022
0000AA51 29C9                   <1>     sub     ecx, ecx
0000AA53 B10B                   <1>     mov     cl, 11
                                <1> loc_next_sum:
                                <1>     ;xor    ah, ah
                                <1>     ;test   al, 1
                                <1>     ;jz     short pass_ah_80h
                                <1>     ;mov    ah, 80h
                                <1> ;pass_ah_80h:
                                <1>     ;shr    al, 1
0000AA55 D0C8                   <1>     ror     al, 1 ; 17/10/2009
0000AA57 0206                   <1>     add     al, [esi]
0000AA59 46                     <1>     inc     esi
                                <1>     ;add    al, ah
0000AA5A E2F9                   <1>     loop    loc_next_sum
0000AA5C C3                     <1>     retn
                                <1>
                                <1> get_fs_longname:
                                <1>     ; temporary (13/02/2016)
0000AA5D 31C0                   <1>     xor     eax, eax
0000AA5F F9                     <1>     stc
0000AA60 C3                     <1>     retn
                                <1>
                                <1> make_sub_directory:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 07/08/2022
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 16/10/2016
                                <1>     ; 02/03/2016, 03/03/2016
                                <1>     ; 26/02/2016, 27/02/2016
                                <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 01/08/2011 (DIR.ASM, 'proc_make_directory')
                                <1>     ; 10/07/2010
                                <1>     ; INPUT ->
                                <1>     ;       ESI = ASCIIZ Directory Name
                                <1>     ;       CL = Directory Attributes
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = New sub dir's first cluster
                                <1>     ;       ESI = Logical Dos Drv Descr. Table Addr.
                                <1>     ;       CF = 1 -> error code in AL (EAX)
                                <1>
                                <1>     ;test   cl, 10h  ; directory
                                <1>     ;jz     short loc_make_directory_access_denied
                                <1>     ;test   cl, 08h ; volume name
                                <1>     ;jnz    short loc_make_directory_access_denied
                                <1>
0000AA61 80E107                 <1>     and     cl, 07h
0000AA64 880D[F4800100]         <1>     mov     byte [mkdir_attrib], cl
                                <1>
0000AA6A 56                     <1>     push    esi
0000AA6B 31DB                   <1>     xor     ebx, ebx
0000AA6D 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
0000AA73 BE00010900             <1>     mov     esi, Logical_DOSDisks
0000AA78 01DE                   <1>     add     esi, ebx
0000AA7A 5B                     <1>     pop     ebx
                                <1>
```

```
                                <1>     ; 10/07/2010 -> 1st writable disk check for trdos
                                <1>     ; LD_DiskType = 0 for write protection (read only)
0000AA7B 807E0101               <1>     cmp     byte [esi+LD_DiskType], 1 ; 0 = Invalid
0000AA7F 7308                   <1>     jnb     short loc_mkdir_check_file_sytem
                                <1>     ; 16/10/2016 (13h -> 30)
                                <1>     ;mov    eax, 30 ; 'Disk write-protected' error
                                <1>     ;mov    edx, 0
                                <1>     ; 29/07/2022
0000AA81 29C0                   <1>     sub     eax, eax
0000AA83 B01E                   <1>     mov     al, 30
0000AA85 29D2                   <1>     sub     edx, edx ; 0
0000AA87 F9                     <1>     stc
                                <1>     ; err retn: EDX = 0, EBX = Dir name offset
                                <1>     ;ESI = Logical DOS drive description table address
0000AA88 C3                     <1>     retn
                                <1>
                                <1> ;loc_make_directory_access_denied:
                                <1>     ;mov    ax, 05h ; access denied (invalid attributes input)
                                <1>     ;stc
                                <1>     ;retn
                                <1>
                                <1> loc_mkdir_check_file_sytem:
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 1
                                <1>     jnb     short loc_mkdir_check_free_sectors
                                <1>
                                <1> loc_make_fs_directory:
                                <1>     mov     eax, [Current_Dir_FCluster]
                                <1>
                                <1>     ; EAX = Parent directory DDT Address
                                <1>     ; ESI = Logical DOS Drive DT Address
                                <1>     ; EBX = Directory name offset (as ASCIIZ name)
                                <1>
                                <1>     ;call   make_fs_directory
                                <1>     ;retn
                                <1>     ; 29/07/2022
                                <1>     jmp     make_fs_directory
                                <1> %endif
                                <1>
                                <1> loc_mkdir_check_free_sectors:
0000AA89 0FB64613               <1>         movzx   eax, byte [esi+LD_BPB+SecPerClust]
0000AA8D 8B4E74                 <1>     mov     ecx, [esi+LD_FreeSectors]
0000AA90 39C1                   <1>     cmp     ecx, eax
0000AA92 7254                   <1>     jb      short loc_mkdir_insufficient_disk_space
                                <1>
                                <1> loc_make_fat_directory:
0000AA94 891D[E4800100]         <1>     mov     [mkdir_DirName_Offset], ebx
0000AA9A 890D[F0800100]         <1>     mov     [mkdir_FreeSectors], ecx
                                <1>
                                <1>     ;mov    al, [esi+LD_BPB+SecPerClust]
0000AAA0 A2[F6800100]           <1>     mov     byte [mkdir_SecPerClust], al
                                <1>
                                <1> loc_mkdir_gffc_1:
0000AAA5 E8F1160000             <1>     call    get_first_free_cluster
0000AAAA 722A                   <1>     jc      short loc_mkdir_gffc_retn
                                <1>
                                <1> ;loc_mkdir_gffc_1_cont:
                                <1>     ;cmp    eax, 2
                                <1>     ;jb     short loc_mkdir_gffc_insufficient_disk_space
                                <1>
                                <1> ;loc_mkdir_gffc_1_save_fcluster:
0000AAAC A3[E8800100]           <1>     mov     [mkdir_FFCluster], eax
                                <1>
                                <1> loc_mkdir_locate_ffe:
                                <1>     ; Current directory fcluster <> Directory buffer cluster
                                <1>     ; Current directory will be reloaded by
                                <1>     ; 'locate_current_dir_file' procedure
                                <1>     ;
                                <1>     ; ESI = Logical DOS Drive Description Table Address
                                <1>     ;push   esi ; 27/02/2016
0000AAB1 31C0                   <1>     xor     eax, eax
0000AAB3 89C1                   <1>         mov ecx, eax
0000AAB5 6649                   <1>     dec     cx ; FFFFh
                                <1>     ; CX = FFFFh -> find first deleted or free entry
                                <1>     ; ESI would be ASCIIZ filename address if the call
                                <1>     ; would not be for first free or deleted dir entry
0000AAB7 E84CFBFFFF             <1>     call    locate_current_dir_file
0000AABC 734A                   <1>     jnc     short loc_mkdir_set_ff_dir_entry_1
                                <1>     ;pop    esi
                                <1>     ; ESI = Logical DOS Drive Description Table Address
0000AABE 83F802                 <1>     cmp     eax, 2  ; cmp al, 2 ; File/Dir not found !
0000AAC1 7529                   <1>     jne     short loc_mkdir_stc_return
                                <1>
                                <1> loc_mkdir_add_new_cluster:
0000AAC3 3805[75770100]         <1>     cmp     byte [Current_FATType], al ; 2
                                <1>     ;cmp    byte ptr [esi+LD_FATType], 2
0000AAC9 770C                   <1>     ja      short loc_mkdir_add_new_cluster_check_fsc
0000AACB 803D[74770100]01       <1>     cmp     byte [Current_Dir_Level], 1
                                <1>     ;cmp    byte [esi+LD_CDirLevel], 1
0000AAD2 7303                   <1>     jnb     short loc_mkdir_add_new_cluster_check_fsc
                                <1>
0000AAD4 B00C                   <1>     mov     al, 12 ; No more files
                                <1> loc_mkdir_gffc_retn:
0000AAD6 C3                     <1>     retn
                                <1>
                                <1> loc_mkdir_add_new_cluster_check_fsc:
0000AAD7 8B0D[F0800100]         <1>     mov     ecx, [mkdir_FreeSectors]
                                <1>     ;movzx  eax, byte [mkdir_SecPerClust]
0000AADD A0[F6800100]           <1>     mov     al, [mkdir_SecPerClust]
                                <1>     ;shl    ax, 1 ; AX = 2 * AX
                                <1>     ; 29/07/2022
0000AAE2 D1E0                   <1>     shl     eax, 1
0000AAE4 39C1                   <1>     cmp     ecx, eax
0000AAE6 7350                   <1>     jnb     short loc_mkdir_add_new_subdir_cluster
                                <1>
                                <1> loc_mkdir_insufficient_disk_space:
                                <1>     ;mov    edx, ecx
                                <1> ;loc_mkdir_gffc_insufficient_disk_space:
                                <1>     ; 29/07/2022
                                <1>     ;mov    ax, 27h ; MSDOS err => insufficient disk space
                                <1>
                                <1>     ; err retn: EDX = Free sectors, EBX = Dir name offset
                                <1>         ; ESI -> Dos drive description table address
                                <1>     ;; ecx = edx
                                <1>
                                <1>     ;retn
                                <1>
                                <1>     ; 29/07/2022
0000AAE8 30E4                   <1>     xor     ah, ah
0000AAEA B027                   <1>     mov     al, 27h
                                <1>
                                <1> loc_mkdir_stc_return:
0000AAEC F9                     <1>     stc
```

```
0000AAED C3                  <1>     retn
                             <1>
                             <1> loc_mkdir_gffc_2:
0000AAEE E8A8160000          <1>     call    get_first_free_cluster
0000AAF3 72E1                <1>     jc      short loc_mkdir_gffc_retn
                             <1>
                             <1> ;loc_mkdir_gffc_1_cont:
                             <1>     ;cmp    eax, 2
                             <1>     ;jb     short loc_mkdir_gffc_insufficient_disk_space
                             <1>
                             <1> ;loc_mkdir_gffc_2_save_fcluster:
0000AAF5 A3[E8800100]        <1>     mov     [mkdir_FFCluster], eax
                             <1>
0000AAFA A1[EC800100]        <1>     mov     eax, [mkdir_LastDirCluster]
                             <1>
0000AAFF E841160000          <1>     call    load_FAT_sub_directory
0000AB04 72D0                <1>     jc      short loc_mkdir_gffc_retn
                             <1>
0000AB06 31FF                <1>     xor     edi, edi
                             <1> loc_mkdir_set_ff_dir_entry_1:
                             <1>     ; 27/02/2016
0000AB08 56                  <1>     push    esi ; Logical DOS Drv Desc. Tbl. address
                             <1>     ; EDI = Directory Entry Address
0000AB09 8B35[E4800100]      <1>     mov     esi, [mkdir_DirName_Offset]
0000AB0F A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
                             <1>
0000AB14 66B91000            <1>     mov     cx, 10h ; CL = Directory attribute
                             <1>                     ; CH = 0 -> File size is 0
0000AB18 0A0D[F4800100]      <1>     or      cl, [mkdir_attrib] ; S, H, R
0000AB1E E89B010000          <1>     call    make_directory_entry
                             <1>
0000AB23 5E                  <1>     pop     esi
                             <1>
0000AB24 C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2
0000AB2B E861020000          <1>     call    save_directory_buffer
                             <1>         ;jnc        loc_mkdir_set_ff_dir_entry_2
                             <1>     ; 29/07/2022
0000AB30 7205                <1>     jc      short loc_mkdir_return
0000AB32 E9CA000000          <1>     jmp     loc_mkdir_set_ff_dir_entry_2
                             <1>
                             <1> loc_mkdir_return:
0000AB37 C3                  <1>     retn
                             <1>
                             <1> loc_mkdir_add_new_subdir_cluster:
0000AB38 8B15[A27E0100]      <1>     mov     edx, [DirBuff_Cluster]
0000AB3E 8915[EC800100]      <1>     mov     [mkdir_LastDirCluster], edx
                             <1>
0000AB44 A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
0000AB49 E8F7150000          <1>     call    load_FAT_sub_directory
0000AB4E 72E7                <1>     jc      short loc_mkdir_return
                             <1>     ; eax = 0
                             <1>     ; ecx = directory buffer sector count (<= 128)
                             <1>
                             <1> pass_mkdir_add_new_subdir_cluster:
                             <1>
                             <1> ; 29/07/2022
                             <1> ;   ;sub    edi, edi ; 0
                             <1> ;   ; 29/07/2022 - BUGFIX !
                             <1> ;   mov     edi, Directory_Buffer
                             <1> ;
                             <1> ;   ;mov    al, 128 ; double word
                             <1> ;   ;mul    ecx ; ecx =  directory buffer sector count
                             <1> ;   ;mov    ecx, eax
                             <1> ;   ;shl    cx, 7 ; 128 * sector count
                             <1> ;   mov     ax, [esi+LD_BPB+BytesPerSec] ; 512
                             <1> ;   ;shr    ax, 2 ; 'byte count / 4' for 'stosd'
                             <1> ;   ; 29/07/2022
                             <1> ;   shr     eax, 2
                             <1> ;   ;mul    cx ; max = 128*(512/4) -> 16384 (stosd)
                             <1> ;   ;mov    cx, ax
                             <1> ;   ;sub    ax, ax ; 0
                             <1> ;   ; 29/07/2022
                             <1> ;   mul     ecx
                             <1> ;   mov     ecx, eax
                             <1> ;   sub     eax, eax
                             <1> ;   rep     stosd ; clear directory buffer
                             <1>
                             <1>     ; 29/07/2022
0000AB50 E85C010000          <1>     call    clear_directory_buffer
                             <1>
0000AB55 C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2
0000AB5C E830020000          <1>     call    save_directory_buffer
0000AB61 72D4                <1>     jc      short loc_mkdir_return
                             <1>
                             <1> loc_mkdir_save_added_cluster:
0000AB63 A1[EC800100]        <1>     mov     eax, [mkdir_LastDirCluster]
0000AB68 8B0D[E8800100]      <1>     mov     ecx, [mkdir_FFCluster]
                             <1>     ; 01/03/2016
0000AB6E 31D2                <1>     xor     edx, edx
0000AB70 8915[927E0100]      <1>     mov     [FAT_ClusterCounter], edx ; 0 ; reset
0000AB76 E8F5160000          <1>     call    update_cluster
0000AB7B 7304                <1>     jnc     short loc_mkdir_save_fat_buffer_0
0000AB7D 09C0                <1>     or      eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000AB7F 7518                <1>     jnz     short loc_mkdir_save_fat_buffer_stc_retn
                             <1>
                             <1> loc_mkdir_save_fat_buffer_0:
0000AB81 A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
0000AB86 A3[EC800100]        <1>     mov     [mkdir_LastDirCluster], eax
                             <1>
0000AB8B 31C9                <1>     xor     ecx, ecx
0000AB8D 49                  <1>     dec     ecx ; FFFFFFFFh
                             <1>     ; ESI = Logical DOS Drive Description Table address
0000AB8E E8DD160000          <1>     call    update_cluster
0000AB93 731A                <1>     jnc     short loc_mkdir_save_fat_buffer_1
0000AB95 09C0                <1>     or      eax, eax
0000AB97 7416                <1>     jz      short loc_mkdir_save_fat_buffer_1
                             <1>
                             <1> loc_mkdir_save_fat_buffer_stc_retn:
                             <1>     ; 01/03/2016
0000AB99 803D[927E0100]01    <1>     cmp     byte [FAT_ClusterCounter], 1
0000ABA0 720C                <1>     jb      short loc_mkdir_save_fat_buffer_retn
                             <1>
0000ABA2 66BB00FF            <1>     mov     bx, 0FF00h ; recalculate free space (BL = 0)
                             <1>                        ; (BH = FFh -> Use ESI as Drv Param. Tbl.)
0000ABA6 50                  <1>     push    eax
0000ABA7 E8C5190000          <1>     call    calculate_fat_freespace
0000ABAC 58                  <1>     pop     eax
0000ABAD F9                  <1>     stc
                             <1> loc_mkdir_save_fat_buffer_retn:
0000ABAE C3                  <1>     retn
                             <1>
                             <1> loc_mkdir_save_fat_buffer_1:
                             <1>     ; byte [FAT_BuffValidData] = 2
0000ABAF E830190000          <1>     call    save_fat_buffer
```

```
0000ABB4 72E3                <1>     jc      short loc_mkdir_save_fat_buffer_stc_retn
                             <1>
                             <1>     ; 01/03/2016
0000ABB6 803D[927E0100]01    <1>     cmp     byte [FAT_ClusterCounter], 1
0000ABBD 721B                <1>     jb      short loc_mkdir_save_fat_buffer_2
                             <1>
                             <1>     ; ESI = Logical DOS Drive Description Table address
0000ABBF A1[927E0100]        <1>     mov     eax, [FAT_ClusterCounter]
0000ABC4 66BB01FF            <1>     mov     bx, 0FF01h ; add free clusters
0000ABC8 E8A4190000          <1>     call    calculate_fat_freespace
                             <1>
                             <1>     ;inc    eax ; 0FFFFFFFFh -> 0 ; recalculation is needed!
                             <1>     ;jnz    short loc_mkdir_save_fat_buffer_2
                             <1>
                             <1>     ; ecx > 0 -> Recalculation is needed
0000ABCD 09C9                <1>     or      ecx, ecx
0000ABCF 7409                <1>     jz      short loc_mkdir_save_fat_buffer_2
                             <1>
0000ABD1 66BB00FF            <1>     mov     bx, 0FF00h ; ; recalculate free space
0000ABD5 E897190000          <1>     call    calculate_fat_freespace
                             <1>
                             <1> loc_mkdir_save_fat_buffer_2:
0000ABDA C605[F7800100]01    <1>     mov     byte [mkdir_add_new_cluster], 1
0000ABE1 E9B0000000          <1>     jmp     loc_mkdir_upd_parent_dir_lmdt
                             <1>
                             <1> loc_mkdir_update_sub_dir_cluster:
0000ABE6 A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
0000ABEB 29C9                <1>     sub     ecx, ecx ; 0
                             <1>     ; 01/03/2016
0000ABED 890D[927E0100]      <1>     mov     [FAT_ClusterCounter], ecx ; 0 ; Reset
0000ABF3 49                  <1>     dec     ecx ; 0FFFFFFFFh
                             <1>
                             <1>     ; ESI = Logical DOS Drive Description Table address
0000ABF4 E877160000          <1>     call    update_cluster
0000ABF9 7364                <1>     jnc     short loc_mkdir_save_fat_buffer_3
0000ABFB 09C0                <1>     or      eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000ABFD 7460                <1>     jz      short loc_mkdir_save_fat_buffer_3
                             <1>     ; 01/03/2016
0000ABFF EB98                <1>     jmp     short loc_mkdir_save_fat_buffer_stc_retn
                             <1>
                             <1> loc_mkdir_set_ff_dir_entry_2:
                             <1>     ; ESI = Logical DOS Drive Description Table address
0000AC01 A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
                             <1>     ; Load disk sectors as a directory cluster
0000AC06 E83A150000          <1>     call    load_FAT_sub_directory
0000AC0B 7251                <1>     jc      short retn_make_fat_directory
                             <1>
                             <1>     ; eax = 0
                             <1>     ; ecx = directory buffer sector count (<= 128)
                             <1>
                             <1> ; 29/07/2022
                             <1> ;   mov     edi, Directory_Buffer + 64 ; 26/02/2016
                             <1> ;
                             <1> ;   ; 02/03/2016
                             <1> ;   mov     ax, [esi+LD_BPB+BytesPerSec] ; 512
                             <1> ;   ;shr    ax, 2 ; 'byte count / 4' for 'stosd'
                             <1> ;   ; 29/07/2022
                             <1> ;   shr     eax, 2
                             <1> ;   mul     ecx
                             <1> ;   mov     ecx, eax
                             <1> ;   ;
                             <1> ;   ; 29/07/2022 - BUGFIX !
                             <1> ;   sub     ecx, 16 ; - 64 bytes
                             <1> ;                   ; (space for '.' & '..' entries)
                             <1> ;   ;sub    ax, ax
                             <1> ;   sub     eax, eax
                             <1> ;   rep     stosd
                             <1> ;
                             <1> ;   ;;mov   al, 128 ; double word (count in sector)
                             <1> ;   ;;mul   ecx ; ecx = directory buffer sector count
                             <1> ;   ;;mov   ecx, eax
                             <1> ;   ;shl    cx, 7 ; 128 * sector count
                             <1> ;   ;sub    ecx, 64 ; 29/07/2022
                             <1> ;   ;;sub   eax, eax
                             <1> ;   ;;sub   al, al ; 0
                             <1> ;   ;rep    stosd ; clear directory buffer
                             <1>
                             <1>     ; 29/07/2022
0000AC0D E89F000000          <1>     call    clear_directory_buffer
                             <1>
0000AC12 BF00000800          <1>     mov     edi, Directory_Buffer ; 26/02/2016
                             <1>
0000AC17 56                  <1>     push    esi
                             <1>
0000AC18 BE[F8800100]        <1>     mov     esi, mkdir_Name
0000AC1D 66C7062E00          <1>     mov     word [esi], 2Eh ; db '.', '0'
                             <1>
0000AC22 A1[E8800100]        <1>     mov     eax, [mkdir_FFCluster]
                             <1>     ;mov    cx, 10h ; CL = Directory attribute
                             <1>                     ; CH = 0 -> File size is 0
                             <1>     ; 29/07/2022
0000AC27 B110                <1>     mov     cl, 10h
0000AC29 E890000000          <1>     call    make_directory_entry
                             <1>
0000AC2E BF20000800          <1>     mov     edi, Directory_Buffer + 32 ; 26/02/2016
                             <1>
                             <1>     ; 03/03/2016
                             <1>     ; Following modification has been done according to
                             <1>     ; 'Microsoft Extensible Firmware Initiative
                             <1>     ; FAT32 File System Specification' document,
                             <1>     ; 'FAT: General Overview of On-Disk Format—Page 25'.
                             <1>     ; "Finally, you set DIR_FstClusLO and DIR_FstClusHI
                             <1>     ; for the dotdot entry (the second entry) to the
                             <1>     ; first cluster number of the directory in which you
                             <1>     ; just created the directory (value is 0 if this directory
                             <1>     ; is the root directory even for FAT32 volumes)."
                             <1>     ; (Correctness of this modification has been verified
                             <1>     ;  by using Windows 98 'scandisk.exe'.)
                             <1>
0000AC33 29C0                <1>     sub     eax, eax
0000AC35 3805[74770100]      <1>     cmp     byte [Current_Dir_Level], al ; 0
0000AC3B 7605                <1>     jna     short loc_mkdir_set_ff_dir_entry_3
0000AC3D A1[70770100]        <1>     mov     eax, [Current_Dir_FCluster] ; parent dir
                             <1> loc_mkdir_set_ff_dir_entry_3:
0000AC42 66C746012E00        <1>     mov     word [esi+1], 2Eh ; db '.', '0'
                             <1>
                             <1>     ;mov    cx, 10h
0000AC48 E871000000          <1>     call    make_directory_entry
                             <1>
0000AC4D 5E                  <1>     pop     esi
                             <1>
0000AC4E C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2
0000AC55 E837010000          <1>     call    save_directory_buffer
                             <1>     ;jnc    loc_mkdir_update_sub_dir_cluster
```

```
                                <1>     ; 29/07/2022
0000AC5A 7202                   <1>     jc      short retn_make_fat_directory
0000AC5C EB88                   <1>     jmp     loc_mkdir_update_sub_dir_cluster
                                <1>
                                <1> retn_make_fat_directory:
0000AC5E C3                     <1>     retn
                                <1>
                                <1> loc_mkdir_save_fat_buffer_3:
                                <1>     ; 01/03/2016
                                <1>     ; byte [FAT_BuffValidData] = 2
0000AC5F E880180000             <1>     call    save_fat_buffer
                                <1>     ;jc     short loc_mkdir_save_fat_buffer_stc_retn
                                <1>     ; 07/08/2022
0000AC64 7305                   <1>     jnc     short loc_mkdir_save_fat_buffer_4
0000AC66 E92EFFFFFF             <1>     jmp     loc_mkdir_save_fat_buffer_stc_retn
                                <1>
                                <1> loc_mkdir_save_fat_buffer_4:
0000AC6B 803D[927E0100]01       <1>     cmp     byte [FAT_ClusterCounter], 1
0000AC72 721B                   <1>     jb      short loc_mkdir_save_fat_buffer_5
                                <1>
                                <1>     ; ESI = Logical DOS Drive Description Table address
0000AC74 A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter]
0000AC79 66BB01FF               <1>     mov     bx, 0FF01h ; add free clusters
0000AC7D E8EF180000             <1>     call    calculate_fat_freespace
                                <1>
                                <1>     ;inc    eax ; 0FFFFFFFFh -> 0 ; recalculation is needed!
                                <1>         ;jnz    short loc_mkdir_save_fat_buffer_5
                                <1>
                                <1>     ; ecx > 0 -> Recalculation is needed
0000AC82 09C9                   <1>     or      ecx, ecx
0000AC84 7409                   <1>         jz      short loc_mkdir_save_fat_buffer_5
                                <1>
0000AC86 66BB00FF               <1>     mov     bx, 0FF00h ; recalculate free space
0000AC8A E8E2180000             <1>     call    calculate_fat_freespace
                                <1>
                                <1> loc_mkdir_save_fat_buffer_5:
0000AC8F C605[F7800100]00       <1>     mov     byte [mkdir_add_new_cluster], 0
                                <1>
                                <1> loc_mkdir_upd_parent_dir_lmdt:
0000AC96 E87F010000             <1>     call    update_parent_dir_lmdt
                                <1>
                                <1>     ; 01/03/2016
0000AC9B 803D[F7800100]00       <1>     cmp     byte [mkdir_add_new_cluster], 0
                                <1>     ;ja     loc_mkdir_gffc_2
                                <1>     ; 29/07/2022
0000ACA2 7605                   <1>     jna     short loc_mkdir_retn_new_dir_cluster
0000ACA4 E945FEFFFF             <1>     jmp     loc_mkdir_gffc_2
                                <1>
                                <1> loc_mkdir_retn_new_dir_cluster:
0000ACA9 A1[E8800100]           <1>     mov     eax, [mkdir_FFCluster]
0000ACAE 31D2                   <1>     xor     edx, edx
                                <1> loc_mkdir_retn:
0000ACB0 C3                     <1>     retn
                                <1>
                                <1> clear_directory_buffer:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ;
                                <1>     ; eax = 0
                                <1>     ; ecx = directory buffer sector count (<= 128)
                                <1>     ;
0000ACB1 BF00000800             <1>     mov     edi, Directory_Buffer
                                <1>     ;mov    al, 128 ; double word
                                <1>     ;mul    ecx ; ecx = directory buffer sector count
                                <1>     ;mov    ecx, eax
                                <1>     ;shl    cx, 7 ; 128 * sector count
                                <1>     ; 19/12/2025
                                <1>     ;mov    ax, [esi+LD_BPB+BytesPerSec] ; 512
                                <1>     ;shr    eax, 2 ; 'byte count / 4' for 'stosd'
                                <1>     ;mov    ax, 128 ; 512/4
                                <1>     ;mul    ecx ; max = 128*(512/4) -> 16384 (stosd)
                                <1>     ;mov    ecx, eax
0000ACB6 C1E107                 <1>     shl     ecx, 7 ; (512/4) * sector count
0000ACB9 29C0                   <1>     sub     eax, eax
0000ACBB F3AB                   <1>     rep     stosd ; clear directory buffer
0000ACBD C3                     <1>     retn
                                <1>
                                <1> make_directory_entry:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 02/03/2016
                                <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 09/08/2010 (DIR.ASM, 'proc_make_directory_entry')
                                <1>     ; 17/07/2010
                                <1>     ; INPUT ->
                                <1>     ;       EDI = Directory Entry Address
                                <1>     ;       ESI = Dot File Name Location
                                <1>     ;       EAX = First Cluster
                                <1>     ;       File Size = 0 (Must be set later)
                                <1>     ;       CL = Attributes
                                <1>     ;       CH = 0 (File size = 0)
                                <1>     ;       (If CH>0, File size is in dword [EBX]) (*)
                                <1>     ; OUTPUT ->
                                <1>     ;       EDI = Directory Entry Address
                                <1>     ;       ESI = Dot File Name Location (Capitalized)
                                <1>     ;       If CH input = 0, File Size = 0
                                <1>     ;       Otherwise file size is as dword [EBX] (*)
                                <1>     ;       DX = Date, AX = Time in DOS Dir Entry format
                                <1>     ;       EBX = same
                                <1>     ;       ECX = same
                                <1>
0000ACBE 51                     <1>     push    ecx
                                <1>
0000ACBF 884F0B                 <1>     mov     [edi+11], cl ; Attributes
0000ACC2 6689471A               <1>     mov     [edi+26], ax ; FClusterLw, 26
0000ACC6 C1E810                 <1>     shr     eax, 16
0000ACC9 66894714               <1>     mov     [edi+20], ax ; FClusterHw, 20
                                <1>     ;xor    ax, ax
                                <1>     ; 29/07/2022
0000ACCD 31C0                   <1>     xor     eax, eax
0000ACCF 6689470C               <1>     mov     [edi+12], ax ; NTReserved, 12
                                <1>                          ; CrtTimeTenth, 13
0000ACD3 08ED                   <1>     or      ch, ch
0000ACD5 7402                   <1>     jz      short loc_make_direntry_set_filesize
                                <1>
0000ACD7 8B03                   <1>     mov     eax, [ebx]
                                <1>
                                <1> loc_make_direntry_set_filesize:
0000ACD9 89471C                 <1>     mov     [edi+28], eax ; FileSize, 28
                                <1>
0000ACDC E8D3FBFFFF             <1>     call    convert_file_name
                                <1>     ; EDI = Dir Entry Format File Name Location
                                <1>     ; ESI = Dot File Name Location (capitalized)
                                <1>
0000ACE1 E816000000             <1>     call    convert_current_date_time
```

```
                                <1>     ; OUTPUT -> DX = Date in dos dir entry format
                                <1>         ;       AX = Time in dos dir entry format
0000ACE6 6689470E               <1>     mov     [edi+14], ax ; CrtTime, 14
0000ACEA 66895710               <1>     mov     [edi+16], dx ; CrtDate, 16
0000ACEE 66895712               <1>     mov     [edi+18], dx ; LastAccDate, 18
0000ACF2 66894716               <1>     mov     [edi+22], ax ; WrtTime, 14
0000ACF6 66895718               <1>     mov     [edi+24], dx ; WrtDate, 16
0000ACFA 59                     <1>     pop     ecx
                                <1>
0000ACFB C3                     <1>     retn
                                <1>
                                <1> convert_current_date_time:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 13/06/2010 (DIR.ASM, 'proc_convert_current_date_time')
                                <1>     ; converts date&time to dos dir entry format
                                <1>     ; INPUT -> none
                                <1>     ; OUTPUT -> DX = Date in dos dir entry format
                                <1>     ;           AX = Time in dos dir entry format
                                <1>
0000ACFC B404                   <1>     mov     ah, 04h ; Return Current Date
0000ACFE E8E6B5FFFF             <1>     call    int1Ah
                                <1>
0000AD03 88E8                   <1>     mov     al, ch ; <- century BCD
0000AD05 240F                   <1>     and     al, 0Fh
0000AD07 88EC                   <1>     mov     ah, ch
0000AD09 C0EC04                 <1>     shr     ah, 4
0000AD0C D50A                   <1>     aad
0000AD0E 88C5                   <1>     mov     ch, al ; -> century
                                <1>
0000AD10 88C8                   <1>     mov     al, cl ; <- year BCD
0000AD12 240F                   <1>     and     al, 0Fh
0000AD14 88CC                   <1>     mov     ah, cl
0000AD16 C0EC04                 <1>     shr     ah, 4
0000AD19 D50A                   <1>     aad
0000AD1B 88C1                   <1>     mov     cl, al ; -> year
                                <1>
                                <1>     ;mov    al, ch
                                <1>     ;mov    ah, 100
                                <1>     ;mul    ah
                                <1>     ; 29/07/2022
0000AD1D B064                   <1>     mov     al, 100
0000AD1F F6E5                   <1>     mul     ch
                                <1>
                                <1>     ;xor    ch, ch
                                <1>     ;add    ax, cx
                                <1>     ; 29/07/2022
0000AD21 00C8                   <1>     add     al, cl
0000AD23 80D400                 <1>     adc     ah, 0
0000AD26 662DBC07               <1>     sub     ax, 1980 ; ms-dos epoch
                                <1>     ;mov    cx, ax
                                <1>     ; 29/07/2022
0000AD2A 88C1                   <1>     mov     cl, al
                                <1>     ;mov    ecx, eax
                                <1>
0000AD2C 88F0                   <1>     mov     al, dh ; <- month in bcd
0000AD2E 240F                   <1>     and     al, 0Fh
0000AD30 88F4                   <1>     mov     ah, dh
0000AD32 C0EC04                 <1>     shr     ah, 4
0000AD35 D50A                   <1>     aad
0000AD37 88C6                   <1>     mov     dh, al ; -> month
                                <1>
0000AD39 88D0                   <1>     mov     al, dl ; <- day BCD
0000AD3B 240F                   <1>     and     al, 0Fh
0000AD3D 88D4                   <1>     mov     ah, dl
0000AD3F C0EC04                 <1>     shr     ah, 4
0000AD42 D50A                   <1>     aad
0000AD44 88C2                   <1>     mov     dl, al ; -> day
                                <1>
0000AD46 88C8                   <1>     mov     al, cl ; count of years from 1980
                                <1>     ;shl    ax, 4
                                <1>     ; 29/07/2022
                                <1>     ;mov    eax, ecx
0000AD48 C1E004                 <1>     shl     eax, 4
                                <1>
0000AD4B 08F0                   <1>     or      al, dh ; month of year, 1 to 12
                                <1>     ;shl    ax, 5
                                <1>     ; 29/07/2022
0000AD4D C1E005                 <1>     shl     eax, 5
0000AD50 08D0                   <1>     or      al, dl ; day of year, 1 to 31
                                <1>
                                <1>     ;push   ax ; push date
                                <1>     ; 29/07/2022
0000AD52 50                     <1>     push    eax
                                <1>
0000AD53 B402                   <1>     mov     ah, 02h ; Return Current Time
0000AD55 E88FB5FFFF             <1>     call    int1Ah
                                <1>
0000AD5A 88E8                   <1>     mov     al, ch ; <- hours BCD
0000AD5C 240F                   <1>     and     al, 0Fh
0000AD5E 88EC                   <1>     mov     ah, ch
0000AD60 C0EC04                 <1>     shr     ah, 4
0000AD63 D50A                   <1>     aad
0000AD65 88C5                   <1>     mov     ch, al ; -> hours
                                <1>
0000AD67 88C8                   <1>     mov     al, cl ; <- minutes BCD
0000AD69 240F                   <1>     and     al, 0Fh
0000AD6B 88CC                   <1>     mov     ah, cl
0000AD6D C0EC04                 <1>     shr     ah, 4
0000AD70 D50A                   <1>     aad
0000AD72 88C1                   <1>     mov     cl, al ; -> minutes
                                <1>
0000AD74 88F0                   <1>     mov     al, dh ; <- seconds BCD
0000AD76 240F                   <1>     and     al, 0Fh
0000AD78 88F4                   <1>     mov     ah, dh
0000AD7A C0EC04                 <1>     shr     ah, 4
0000AD7D D50A                   <1>     aad
0000AD7F 88C6                   <1>     mov     dh, al ; -> seconds
                                <1>
0000AD81 88E8                   <1>     mov     al, ch ; hours
                                <1>     ;shl    ax, 6
                                <1>     ; 29/07/2022
0000AD83 C1E006                 <1>     shl     eax, 6
0000AD86 08C8                   <1>     or      al, cl ; minutes
                                <1>     ;shl    ax, 5
                                <1>     ; 29/07/2022
0000AD88 C1E005                 <1>     shl     eax, 5
0000AD8B D0EE                   <1>     shr     dh, 1 ; 2 seconds
                                <1>     ; There is a bug in TRDOS v1 here !
                                <1>     ; it was 'or al, dl' !
0000AD8D 08F0                   <1>     or      al, dh ; seconds
                                <1>
                                <1>     ;pop    dx ; pop date
                                <1>     ; 29/07/2022
```

```
0000AD8F 5A                  <1>     pop    edx
                             <1>
0000AD90 C3                  <1>     retn
                             <1>
                             <1> save_directory_buffer:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 30/07/2022
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/10/2016
                             <1>     ; 23/03/2016
                             <1>     ; 26/02/2016
                             <1>     ; 22/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 01/08/2011
                             <1>     ; 14/03/2010
                             <1>     ; INPUT ->
                             <1>     ;       none
                             <1>     ; OUTPUT ->
                             <1>     ;  cf = 0 -> write OK...
                             <1>     ;  cf = 1 -> error code in AL (EAX)
                             <1>     ;  cf = 1 & AL = 0Dh => CH & CL = FS & FAT type
                             <1>     ;  EBX = Directory Buffer Address
                             <1>     ;
                             <1>     ;  (EAX, ECX, EDX will be modified)
                             <1>
0000AD91 BB00000800          <1>     mov    ebx, Directory_Buffer
0000AD96 803D[9D7E0100]02    <1>     cmp    byte [DirBuff_ValidData], 2
0000AD9D 7403                <1>     je     short loc_save_dir_buffer
0000AD9F 31C0                <1>     xor    eax, eax
0000ADA1 C3                  <1>     retn
                             <1>
                             <1> loc_save_dir_buffer:
0000ADA2 56                  <1>     push   esi
0000ADA3 31DB                <1>     xor    ebx, ebx
0000ADA5 8A3D[9B7E0100]      <1>         mov     bh, [DirBuff_DRV]
0000ADAB 80EF41              <1>     sub    bh, 'A'
0000ADAE BE00010900          <1>         mov     esi, Logical_DOSDisks
0000ADB3 01DE                <1>     add    esi, ebx
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>         mov cx, [esi+LD_FATType]
                             <1>     ; CH = FS Type (A1h for FS)
                             <1>     ; CL = FAT Type (0 for FS)
                             <1>     or     cl, cl
                             <1>     jz     short loc_save_dir_buff_stc_retn
                             <1> %endif
                             <1>
                             <1> loc_save_dir_buffer_check_cluster_no:
0000ADB5 A1[A27E0100]        <1>     mov    eax, [DirBuff_Cluster]
0000ADBA 28FF                <1>     sub    bh, bh ; ebx = 0
0000ADBC 09C0                <1>     or     eax, eax
0000ADBE 7537                <1>     jnz    short loc_save_sub_dir_buffer
0000ADC0 8A25[9C7E0100]      <1>     mov    ah, [DirBuff_FATType]
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     inc    bl ; bl = 1
                             <1>     cmp    ah, bl
                             <1>     jb     short loc_save_dir_buff_inv_data_retn
                             <1>     inc    bl ; bl = 2
                             <1>     cmp    bl, ah
                             <1>     jb     short loc_save_dir_buff_inv_data_retn
                             <1> %else
                             <1>     ; 19/12/2025
0000ADC6 80FC03              <1>     cmp    ah, 3 ; FAT32 ?
0000ADC9 7314                <1>     jnb    short loc_save_dir_buff_stc_retn ; yes
                             <1> %endif
                             <1>
                             <1> loc_save_root_dir_buffer:
0000ADCB 668B5E17            <1>     mov    bx, [esi+LD_BPB+RootDirEnts]
0000ADCF 6683C30F            <1>     add    bx, 15
                             <1>     ;shr   bx, 4 ; 16 dir entries per sector
                             <1>     ; 29/07/2022
0000ADD3 C1EB04              <1>     shr    ebx, 4
                             <1>     ;or    bx, bx
0000ADD6 09DB                <1>     or     ebx, ebx
0000ADD8 7405                <1>     jz     short loc_save_dir_buff_stc_retn
                             <1>     ;mov   ecx, ebx
0000ADDA 8B4664              <1>     mov    eax, [esi+LD_ROOTBegin] ; 26/02/2016
0000ADDD EB22                <1>     jmp    short loc_write_directory_to_disk
                             <1>
                             <1> loc_save_dir_buff_stc_retn:
0000ADDF F9                  <1>     stc
                             <1> loc_save_dir_buff_inv_data_retn:
                             <1>     ; 15/10/2016 (0Dh -> 29)
0000ADE0 B01D                <1>     mov    al, 29 ; Invalid data !
0000ADE2 C605[9D7E0100]00    <1>     mov    byte [DirBuff_ValidData], 0
0000ADE9 EB05                <1>     jmp    short loc_save_dir_buff_retn
                             <1>
                             <1> loc_write_directory_to_disk_err:
                             <1>     ; 15/10/2016 (disk write error code, 1Dh -> 18)
0000ADEB B812000000          <1>     mov    eax, 18 ; Drive not ready or write error
                             <1>
                             <1> loc_save_dir_buff_retn:
0000ADF0 BB00000800          <1>     mov    ebx, Directory_Buffer
0000ADF5 5E                  <1>     pop    esi
0000ADF6 C3                  <1>     retn
                             <1>
                             <1> loc_save_sub_dir_buffer:
                             <1>     ; ebx = 0
                             <1>     ;sub   eax, 2
                             <1>     ; 30/07/2022
0000ADF7 48                  <1>     dec    eax
0000ADF8 48                  <1>     dec    eax
0000ADF9 8A5E13              <1>     mov    bl, [esi+LD_BPB+SecPerClust]
0000ADFC F7E3                <1>     mul    ebx
0000ADFE 034668              <1>         add     eax, [esi+LD_DATABegin]
                             <1>     ;mov   ecx, ebx
                             <1>
                             <1> loc_write_directory_to_disk:
0000AE01 89D9                <1>     mov    ecx, ebx
0000AE03 BB00000800          <1>     mov    ebx, Directory_Buffer
0000AE08 E8D76B0000          <1>     call   disk_write
0000AE0D 72DC                <1>     jc     short loc_write_directory_to_disk_err
                             <1>
                             <1> loc_save_dir_buff_validate_retn:
0000AE0F C605[9D7E0100]01    <1>     mov    byte [DirBuff_ValidData], 1
0000AE16 31C0                <1>     xor    eax, eax
                             <1>     ; 26/02/2016
0000AE18 EBD6                <1>     jmp    short loc_save_dir_buff_retn
                             <1>
                             <1> update_parent_dir_lmdt:
                             <1>     ; 19/12/2025
                             <1>     ; 16/07/2025 (TRDOS 386 Kernel v2.0.10) (BugFix)
```

```
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 29/12/2017
                                <1>     ; 22/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 01/08/2011
                                <1>     ; 16/10/2010
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       none
                                <1>     ; OUTPUT ->
                                <1>     ;       (last modification date & time of the parent dir
                                <1>     ;       will be changed/updated)
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, EDI will be changed)
                                <1>
0000AE1A 29C0                   <1>     sub     eax, eax
0000AE1C 8A25[74770100]         <1>     mov     ah, [Current_Dir_Level]
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     mov     al, [Current_FATType]
                                <1>     cmp     al, 1
                                <1>     jb      short loc_UPDLMDT_proc_retn
                                <1> %endif
                                <1>
                                <1> loc_update_parent_dir_lm_date_time:
0000AE22 08E4                   <1>     or      ah, ah
0000AE24 7433                   <1>     jz      short loc_UPDLMDT_proc_retn
                                <1>
0000AE26 56                     <1>     push    esi ; *
0000AE27 8825[DF800100]         <1>     mov     [UPDLMDT_CDirLevel], ah
0000AE2D 8B15[70770100]         <1>     mov     edx, [Current_Dir_FCluster]
0000AE33 8915[E0800100]         <1>     mov     [UPDLMDT_CDirFCluster], edx
                                <1>
0000AE39 FECC                   <1>     dec     ah
                                <1>     ;mov    ecx, 12
                                <1>         ; 29/07/2022
0000AE3B 29C9                   <1>     sub     ecx, ecx
0000AE3D B10C                   <1>     mov     cl, 12
                                <1>     ;
0000AE3F BE[D47E0100]           <1>     mov      esi, PATH_Array
                                <1>
0000AE44 8825[74770100]         <1>     mov     [Current_Dir_Level], ah
0000AE4A 08E4                   <1>     or      ah, ah
0000AE4C 750C                   <1>     jnz     short loc_update_parent_dir_lmdt_load_sub_dir_1
0000AE4E 803D[75770100]02       <1>     cmp     byte [Current_FATType], 2
0000AE55 7709                   <1>     ja      short loc_update_parent_dir_lmdt_load_sub_dir_2
                                <1>     ; 19/12/2025
                                <1>     ;sub    al, al
                                <1>     ; eax = 0
0000AE57 EB0A                   <1>     jmp     short loc_update_parent_dir_lmdt_load_sub_dir_3
                                <1>
                                <1> loc_UPDLMDT_proc_retn:
0000AE59 C3                     <1>     retn
                                <1>
                                <1> loc_update_parent_dir_lmdt_load_sub_dir_1:
0000AE5A B010                   <1>     mov     al, 16
0000AE5C F6E4                   <1>     mul     ah
0000AE5E 01C6                   <1>     add     esi, eax
                                <1>
                                <1> loc_update_parent_dir_lmdt_load_sub_dir_2:
0000AE60 8B460C                 <1>     mov     eax, [esi+12] ; Parent Dir First Cluster
                                <1>
                                <1> loc_update_parent_dir_lmdt_load_sub_dir_3:
0000AE63 A3[70770100]           <1>     mov     [Current_Dir_FCluster], eax
                                <1>
0000AE68 83C610                 <1>     add     esi, 16
                                <1>     ;mov    di, Dir_File_Name
                                <1>     ; 16/07/2025 (BugFix)
0000AE6B BF[FA7F0100]           <1>     mov     edi, Dir_File_Name
0000AE70 F3A4                   <1>     rep     movsb
                                <1>
0000AE72 BE00010900             <1>     mov     esi, Logical_DOSDisks
0000AE77 29DB                   <1>     sub     ebx, ebx
0000AE79 8A3D[76770100]         <1>     mov     bh, [Current_Drv]
0000AE7F 01DE                   <1>     add     esi, ebx
0000AE81 E83BF8FFFF             <1>     call    reload_current_directory
0000AE86 722F                   <1>     jc      short loc_update_parent_dir_lmdt_restore_cdirlevel
                                <1>
                                <1> loc_update_parent_dir_lmdt_locate_dir:
0000AE88 BE[FA7F0100]           <1>     mov     esi, Dir_File_Name
                                <1>     ;xor    cx, cx
                                <1>     ; 29/07/2022
0000AE8D 31C9                   <1>     xor     ecx, ecx
0000AE8F 66B81008               <1>     mov     ax, 0810h ; Only directories
0000AE93 E870F7FFFF             <1>         call    locate_current_dir_file
                                <1>     ; EDI = DirBuff Directory Entry Address
0000AE98 721D                   <1>     jc      short loc_update_parent_dir_lmdt_restore_cdirlevel
                                <1>
0000AE9A E85DFEFFFF             <1>     call    convert_current_date_time
0000AE9F 66895712               <1>     mov     [edi+18], dx ; Last Access Date
0000AEA3 66895718               <1>     mov     [edi+24], dx ; Last Write Date
0000AEA7 66894716               <1>     mov     [edi+22], ax ; Last Write Time
                                <1>
0000AEAB C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
0000AEB2 E8DAFEFFFF             <1>     call    save_directory_buffer
                                <1>     ; 29/12/2017
                                <1>     ;jc     short loc_update_parent_dir_lmdt_restore_cdirlevel
                                <1>     ;xor    al, al
                                <1> loc_update_parent_dir_lmdt_restore_cdirlevel:
                                <1>     ;current directory level restoration
0000AEB7 8A25[DF800100]         <1>     mov     ah, [UPDLMDT_CDirLevel]
0000AEBD 8825[74770100]         <1>     mov     [Current_Dir_Level], ah
0000AEC3 8B15[E0800100]         <1>         mov     edx, [UPDLMDT_CDirFCluster]
0000AEC9 8915[70770100]         <1>     mov     [Current_Dir_FCluster], edx
                                <1>
0000AECF 5E                     <1>     pop     esi ; *
0000AED0 C3                     <1>     retn
                                <1>
                                <1> delete_longname:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 27/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 01/08/2011 (DIR.ASM, 'proc_delete_longname')
                                <1>     ; 14/03/2010
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Directory Entry (Index) Number (< 65536)
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 0 -> OK  (EAX = 0)
                                <1>     ;       cf = 1 -> error code in EAX (AL)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, EDX, ECX, EBX, EDI)
                                <1>
0000AED1 66A3[48810100]         <1>     mov     [DLN_EntryNumber], ax
0000AED7 C605[4A810100]40       <1>         mov     byte [DLN_40h], 40h
                                <1>
```

```
0000AEDE E857000000          <1>     call    locate_current_dir_entry
0000AEE3 7307                <1>     jnc     short loc_dln_check_attributes
0000AEE5 C3                  <1>     retn
                             <1>
                             <1> loc_dln_longname_not_found:
                             <1>     ;mov    eax, 2
                             <1>     ; 29/07/2022
0000AEE6 29C0                <1>     sub     eax, eax
0000AEE8 B002                <1>     mov     al, 2
0000AEEA F9                  <1>     stc
0000AEEB C3                  <1>     retn
                             <1>
                             <1> loc_dln_check_attributes:
0000AEEC B00F                <1>     mov     al, 0Fh  ; long name
0000AEEE 8A670B              <1>     mov     ah, [edi+0Bh] ; dir entry attributes
0000AEF1 38C4                <1>     cmp     ah, al
0000AEF3 75F1                <1>     jne     short loc_dln_longname_not_found
0000AEF5 8A27                <1>     mov     ah, [edi]
0000AEF7 2A25[4A810100]      <1>     sub     ah, [DLN_40h]
0000AEFD 76E7                <1>     jna     short loc_dln_longname_not_found
0000AEFF 80FC14              <1>     cmp     ah, 14h ; 84-64=20 -> 20*13=260 bytes
0000AF02 77E2                <1>     ja      short loc_dln_longname_not_found
                             <1>
0000AF04 C607E5              <1>     mov     byte [edi], 0E5h  ; deleted sign
0000AF07 C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2 ; changed/write sign
0000AF0E C605[4A810100]00    <1>     mov     byte [DLN_40h], 0 ; 40h -> 0
                             <1>
                             <1> loc_dln_delete_next_ln_entry:
0000AF15 80FC01              <1>     cmp     ah, 1
0000AF18 7616                <1>     jna     short loc_dln_longname_retn
                             <1> loc_dln_delete_next_ln_entry_0:
0000AF1A 66FF05[48810100]    <1>     inc     word [DLN_EntryNumber]
0000AF21 0FB705[48810100]    <1>     movzx   eax, word [DLN_EntryNumber]
0000AF28 E80D000000          <1>     call    locate_current_dir_entry
0000AF2D 73BD                <1>     jnc     short loc_dln_check_attributes
                             <1>
                             <1> loc_dln_longname_stc_retn:
0000AF2F C3                  <1>     retn
                             <1>
                             <1> loc_dln_longname_retn:
                             <1>     ;cmp    byte [DirBuff_ValidData], 2
                             <1>     ;jne    short loc_dln_longname_retn_xor_eax
0000AF30 E85CFEFFFF          <1>     call    save_directory_buffer
0000AF35 72F8                <1>     jc      short loc_dln_longname_stc_retn
                             <1>
                             <1> loc_dln_longname_retn_xor_eax:
0000AF37 31C0                <1>     xor     eax, eax
0000AF39 C3                  <1>     retn
                             <1>
                             <1> locate_current_dir_entry:
                             <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                             <1>     ; 30/07/2022
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 16/10/2016
                             <1>     ; 15/10/2016
                             <1>     ; 23/03/2016
                             <1>     ; 27/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 01/08/2011 (DIR.ASM, 'proc_locate_current_dir_entry')
                             <1>     ; 07/03/2010
                             <1>     ; INPUT ->
                             <1>     ;       EAX = Directory Entry (Index) Number (< 65536)
                             <1>     ; OUTPUT ->
                             <1>     ;       EDI = Directory Entry Address
                             <1>     ;       EAX = Cluster Number of Directory Buffer
                             <1>     ;       EBX = Directory Buffer Entry Offset
                             <1>     ;       ECX = DirBuff Valid Data identifier (CL)
                             <1>     ;       If CF = 0 and CL = 2 then
                             <1>     ;          directory buffer modified and
                             <1>     ;          must be written to disk.
                             <1>     ;       If CF = 0  and CL = 1 then
                             <1>     ;          dir buffer has been written to disk, already.
                             <1>     ;       CF = 1 -> Error code in EAX (AL)
                             <1>     ;
                             <1>     ; (Modified registers: EAX, EDX, ECX, EBX, EDI)
                             <1>
                             <1> loc_locate_current_dir_entry:
0000AF3A 56                  <1>     push    esi
0000AF3B 89C1                <1>     mov     ecx, eax
                             <1>     ;mov    edx, 32
                             <1>     ; 29/07/2022
0000AF3D 29D2                <1>     sub     edx, edx
0000AF3F B220                <1>     mov     dl, 32
0000AF41 F7E2                <1>     mul     edx
0000AF43 A3[54810100]        <1>     mov     [LCDE_ByteOffset], eax
0000AF48 31DB                <1>     xor     ebx, ebx
0000AF4A 8A3D[76770100]      <1>     mov     bh, [Current_Drv]
0000AF50 A0[9B7E0100]        <1>         mov     al, [DirBuff_DRV]
0000AF55 2C41                <1>     sub     al, 'A'
0000AF57 BE00010900          <1>         mov     esi, Logical_DOSDisks
0000AF5C 01DE                <1>     add     esi, ebx
0000AF5E 38C7                <1>     cmp     bh, al
                             <1>     ;jne    loc_lcde_reload_current_directory
                             <1>     ; 29/07/2022
0000AF60 7405                <1>     je      short loc_lcde_cdl_check
0000AF62 E986000000          <1>     jmp     loc_lcde_reload_current_directory
                             <1> loc_lcde_cdl_check:
                             <1>     ; 29/07/2022
0000AF67 31C0                <1>     xor     eax, eax
0000AF69 803D[74770100]00    <1>     cmp     byte [Current_Dir_Level], 0
0000AF70 7728                <1>     ja      short loc_lcde_calc_dirbuff_cluster_offset
                             <1>     ; 27/02/2016
                             <1>     ; TRDOS v1 has bug here for FAT32 fs !
                             <1>     ; (Root Directory Entries for FAT32 = 0)
0000AF72 807E0303            <1>     cmp     byte [esi+LD_FATType], 3  ; FAT32
0000AF76 7322                <1>     jnb     short loc_lcde_calc_dirbuff_cluster_offset
                             <1>
                             <1> loc_lcde_cdl_check_FAT12_16:
                             <1>     ; 29/07/2022
                             <1>     ;xor    eax, eax
0000AF78 668B4617            <1>     mov     ax, [esi+LD_BPB+RootDirEnts]
                             <1>     ;dec    ax
0000AF7C 48                  <1>     dec     eax
                             <1>     ;xor    dx, dx
                             <1>     ;cmp    ax, cx ; cx = Directory Entry (Index) Number
                             <1>     ; 29/07/2022
0000AF7D 39C8                <1>     cmp     eax, ecx
0000AF7F 720E                <1>     jb      short loc_lcde_stc_12h_retn
0000AF81 66890D[4C810100]    <1>     mov     [LCDE_EntryIndex], cx
0000AF88 31C0                <1>     xor     eax, eax
0000AF8A E985000000          <1>         jmp     loc_lcde_check_dir_buffer_cluster
                             <1>
                             <1> loc_lcde_stc_12h_retn:
0000AF8F 5E                  <1>     pop     esi
0000AF90 89CB                <1>     mov     ebx, ecx
```

```
0000AF92 89D1                <1>     mov     ecx, edx
                             <1>     ; 16/10/2016 (12h -> 12)
0000AF94 B80C000000          <1>     mov     eax, 12 ; No more files
0000AF99 C3                  <1>     retn
                             <1>
                             <1> loc_lcde_calc_dirbuff_cluster_offset:
                             <1>     ;mov    bl, [esi+LD_BPB+SecPerClust]
                             <1>     ;xor    bh, bh
                             <1>     ;mov    ax, [esi+LD_BPB+BytesPerSec]
                             <1>     ;mul    bx
                             <1>     ;;or    dx, dx ; If bytes per cluster > 32KB it is invalid
                             <1>     ;; 29/07/2022
                             <1>     ;or     dl, dl
                             <1>     ;jnz    short loc_lcde_invalid_format
                             <1>     ; 29/07/2022
0000AF9A 31DB                <1>     xor     ebx, ebx
0000AF9C 8A5E13              <1>     mov     bl, [esi+LD_BPB+SecPerClust]
                             <1>     ;mov    ax, [esi+LD_BPB+BytesPerSec]
                             <1>     ; 19/12/2025
0000AF9F 66B80002            <1>     mov     ax, 512
0000AFA3 F7E3                <1>     mul     ebx
0000AFA5 89C1                <1>     mov     ecx, eax
                             <1>     ;mov    cx, ax ; BYTES PER CLUSTER
0000AFA7 A1[54810100]        <1>     mov     eax, [LCDE_ByteOffset]
                             <1>     ;sub    edx, edx
0000AFAC F7F1                <1>     div     ecx
0000AFAE 3DFFFF0000          <1>     cmp     eax, 65535
0000AFB3 7746                <1>     ja      short loc_lcde_invalid_format
                             <1>
                             <1>     ; cluster sequence number of directory (< 65536)
0000AFB5 66A3[4E810100]      <1>     mov     [LCDE_ClusterSN], ax
                             <1>
                             <1>     ;mov    ax, dx ; byte offset in cluster (directory buffer)
                             <1>     ; 29/07/2022
0000AFBB 89D0                <1>     mov     eax, edx
                             <1>     ;mov    bx, 32  ; 1 dir entry = 32 bytes
0000AFBD B320                <1>         mov bl, 32
                             <1>     ;sub    dx, dx  ; 0
                             <1>     ;div    bx
0000AFBF 29D2                <1>     sub     edx, edx
0000AFC1 F7F3                <1>     div     ebx
0000AFC3 66A3[4C810100]      <1>     mov     [LCDE_EntryIndex], ax ; dir entry index/sequence number
                             <1>                                   ; (in directory buffer/cluster)
                             <1> loc_lcde_get_current_sub_dir_fcluster:
0000AFC9 A1[70770100]        <1>     mov     eax, [Current_Dir_FCluster]
                             <1>
                             <1> loc_lcde_get_next_cluster:
0000AFCE 66833D[4E810100]00  <1>     cmp     word [LCDE_ClusterSN], 0
0000AFD6 763C                <1>     jna     short loc_lcde_check_dir_buffer_cluster
0000AFD8 A3[50810100]        <1>     mov     [LCDE_Cluster], eax
0000AFDD E8A70F0000          <1>     call    get_next_cluster
0000AFE2 721E                <1>     jc      short loc_lcde_check_gnc_error
0000AFE4 66FF0D[4E810100]    <1>     dec     word [LCDE_ClusterSN]
0000AFEB EBE1                <1>     jmp     short loc_lcde_get_next_cluster
                             <1>
                             <1> loc_lcde_reload_current_directory:
0000AFED 51                  <1>     push    ecx
0000AFEE E8CEF6FFFF          <1>     call    reload_current_directory
0000AFF3 59                  <1>     pop     ecx
                             <1>     ;jnc    loc_lcde_cdl_check
                             <1>     ;pop    esi
                             <1>     ;retn
                             <1>     ; 09/08/2022
0000AFF4 727E                <1>     jc      short loc_lcde_retn
0000AFF6 E96CFFFFFF          <1>     jmp     loc_lcde_cdl_check
                             <1>
                             <1> loc_lcde_invalid_format:
                             <1>     ; 15/10/2016 (0Bh -> 28)
                             <1>     ;mov    eax, 28 ; Invalid Format !
                             <1>     ; 29/07/2022
0000AFFB 29C0                <1>     sub     eax, eax
0000AFFD B01C                <1>     mov     al, 28
                             <1> loc_lcde_drive_not_ready_read_err:
0000AFFF F9                  <1>     stc
0000B000 5E                  <1>     pop     esi
0000B001 C3                  <1>     retn
                             <1>
                             <1> loc_lcde_check_gnc_error:
0000B002 09C0                <1>     or      eax, eax
0000B004 75F9                <1>     jnz     short loc_lcde_drive_not_ready_read_err
0000B006 66FF0D[4E810100]    <1>     dec     word [LCDE_ClusterSN]
0000B00D 75EC                <1>     jnz     short loc_lcde_invalid_format
0000B00F A1[50810100]        <1>     mov     eax, [LCDE_Cluster]
                             <1>
                             <1> loc_lcde_check_dir_buffer_cluster:
0000B014 3B05[A27E0100]      <1>     cmp     eax, [DirBuff_Cluster]
0000B01A 755A                <1>     jne     short loc_lcde_load_dir_cluster
0000B01C 803D[9D7E0100]00    <1>     cmp     byte [DirBuff_ValidData], 0
0000B023 7726                <1>     ja      short lcde_check_dir_buffer_cluster_next
0000B025 803D[74770100]00    <1>     cmp     byte [Current_Dir_Level], 0
0000B02C 775D                <1>     ja      short loc_lcde_load_dir_cluster_0
                             <1>     ; 27/02/2016
                             <1>     ; TRDOS v1 has bug here for FAT32 fs !
0000B02E 807E0303            <1>     cmp     byte [esi+LD_FATType], 3  ; FAT32
0000B032 7357                <1>     jnb     short loc_lcde_load_dir_cluster_0
                             <1>     ;
0000B034 0FB74E17            <1>     movzx   ecx, word [esi+LD_BPB+RootDirEnts]
0000B038 6683C10F            <1>     add     cx, 15 ; round up (16 entries per sector)
                             <1>     ;shr    cx, 4 ; 1 sector contains 16 dir entries
                             <1>     ; 29/07/2022
0000B03C C1E904              <1>     shr     ecx, 4
0000B03F 8B4664              <1>         mov     eax, [esi+LD_ROOTBegin]
0000B042 EB52                <1>     jmp     short loc_lcde_load_dir_cluster_1
                             <1>
                             <1> loc_lcde_validate_dirBuff:
0000B044 C605[9D7E0100]01    <1>     mov     byte [DirBuff_ValidData], 1
                             <1>
                             <1> lcde_check_dir_buffer_cluster_next:
0000B04B 0FB71D[4C810100]    <1>     movzx   ebx, word [LCDE_EntryIndex]
0000B052 663B1D[A07E0100]    <1>     cmp     bx, [DirBuff_LastEntry]
0000B059 77A0                <1>     ja      short loc_lcde_invalid_format
                             <1>     ;mov    eax, 32
                             <1>     ; 29/07/2022
0000B05B 31C0                <1>     xor     eax, eax
0000B05D B020                <1>     mov     al, 32
0000B05F F7E3                <1>     mul     ebx
                             <1>     ;or     edx, edx
                             <1>     ;jnz    short loc_lcde_invalid_format
                             <1>
0000B061 BF00000800          <1>     mov     edi, Directory_Buffer
0000B066 01C7                <1>     add     edi, eax ; add entry offset to buffer address
                             <1>
                             <1> loc_lcde_dir_buffer_last_check:
0000B068 A1[A27E0100]        <1>     mov     eax, [DirBuff_Cluster]
```

```
0000B06D 0FB60D[9D7E0100]    <1>     movzx   ecx, byte [DirBuff_ValidData]
                             <1>
                             <1> loc_lcde_retn:
0000B074 5E                  <1>     pop     esi
0000B075 C3                  <1>     retn
                             <1>
                             <1> loc_lcde_load_dir_cluster:
                             <1>     ;cmp    byte [DirBuff_ValidData], 2
                             <1>     ;jne    short loc_lcde_load_dir_cluster_n2
0000B076 50                  <1>     push    eax
0000B077 E815FDFFFF          <1>     call    save_directory_buffer
0000B07C 58                  <1>     pop     eax
0000B07D 72F5                <1>     jc      short loc_lcde_retn
                             <1>
                             <1> loc_lcde_load_dir_cluster_n2:
0000B07F C605[9D7E0100]00    <1>     mov     byte [DirBuff_ValidData], 0
0000B086 A3[A27E0100]        <1>     mov     [DirBuff_Cluster], eax
                             <1>
                             <1> loc_lcde_load_dir_cluster_0:
                             <1>     ;sub    eax, 2
                             <1>     ; 30/07/2022
0000B08B 48                  <1>     dec     eax
0000B08C 48                  <1>     dec     eax
0000B08D 0FB64E13            <1>     movzx   ecx, byte [esi+LD_BPB+SecPerClust]
0000B091 F7E1                <1>     mul     ecx
0000B093 034668              <1>         add     eax, [esi+LD_DATABegin]
                             <1>
                             <1> loc_lcde_load_dir_cluster_1:
0000B096 BB00000800          <1>     mov     ebx, Directory_Buffer
                             <1>     ; ecx = sector count
0000B09B E853690000          <1>     call    disk_read
0000B0A0 73A2                <1>     jnc     short loc_lcde_validate_dirBuff
                             <1>
                             <1>     ; 15/10/2016
                             <1>     ; (Disk read error instead of drv not ready err)
0000B0A2 B811000000          <1>     mov     eax, 17 ; Drive not ready or read error !
0000B0A7 EBCB                <1>     jmp     short loc_lcde_retn
                             <1>
                             <1> remove_file:
                             <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/10/2016
                             <1>     ; 28/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 10/04/2011 (FILE.ASM, 'proc_delete_file')
                             <1>     ; 09/08/2010
                             <1>     ; INPUT ->
                             <1>     ;       EDI = Directory Buffer Entry Address
                             <1>     ;        CX = Directory Buffer Entry Counter/Index
                             <1>     ;        BL = Longname Entry Length
                             <1>     ;        BH = Logical DOS Drive Number
                             <1>
0000B0A9 29C0                <1>     sub     eax, eax
0000B0AB 88FC                <1>     mov     ah, bh
0000B0AD BE00010900          <1>     mov     esi, Logical_DOSDisks
0000B0B2 01C6                <1>     add     esi, eax
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     byte [esi+LD_FATType], 1
                             <1>     jnb     short loc_del_fat_file
                             <1>
                             <1>     cmp     byte [esi+LD_FSType], 0A1h
                             <1>     je      short loc_del_fs_file
                             <1>
                             <1> loc_del_file_invalid_format:
                             <1>     xor     ah, ah
                             <1>     ; 15/10/2016 (0Bh -> 28)
                             <1>     mov     al, 28  ; Invalid Format
                             <1>     stc
                             <1>     retn
                             <1>
                             <1> loc_del_fs_file:
                             <1>     ;call   delete_fs_file
                             <1>     ;retn
                             <1>     ; 29/07/2022
                             <1>     jmp     delete_fs_file
                             <1> %endif
                             <1>
                             <1> loc_del_fat_file:
0000B0B4 E808000000          <1>     call    delete_directory_entry
0000B0B9 7205                <1>     jc      short loc_del_file_err_retn
                             <1>
                             <1> loc_delfile_unlink_cluster_chain:
0000B0BB E85E160000          <1>     call    truncate_cluster_chain
                             <1>     ;jc     short loc_del_file_err_retn
                             <1>
                             <1> loc_delfile_return:
                             <1> loc_del_file_err_retn:
0000B0C0 C3                  <1>     retn
                             <1>
                             <1> delete_directory_entry:
                             <1>     ; 15/10/2016
                             <1>     ; 28/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 01/08/2011 (DIR.ASM, 'proc_delete_directory_entry')
                             <1>     ; 10/04/2011
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Logical Dos Drive Descripton Table Address
                             <1>     ;       EDI = Directory Buffer Entry Address
                             <1>     ;        CX = Directory Buffer Entry Counter/Index
                             <1>     ;        BL = Longname Entry Length
                             <1>     ; OUTPUT ->
                             <1>     ;       ESI = Logical dos drive descripton table address
                             <1>     ;       EAX = First cluster to be truncated/unlinked
                             <1>     ;        CF = 1 -> Error code in EAX (AL)
                             <1>     ;        CF = 0 & BH <> 0 -> LMDT write error  (BH = 1)
                             <1>     ;        CF = 0 & BL <> 0 -> Long name delete error (BL = FFh)
                             <1>     ;
                             <1>     ;  (EDI, EBX, ECX register contents will be changed)
                             <1>
0000B0C1 881D[DE800100]      <1>     mov     [DelFile_LNEL], bl
0000B0C7 66890D[DC800100]    <1>     mov     [DelFile_EntryCounter], cx
                             <1>
0000B0CE 668B4714            <1>     mov     ax, [edi+20] ; First Cluster High Word
0000B0D2 C1E010              <1>     shl     eax, 16
0000B0D5 668B471A            <1>     mov     ax, [edi+26] ; First Cluster Low Word
                             <1>
0000B0D9 A3[D8800100]        <1>     mov     [DelFile_FCluster], eax
                             <1>
                             <1> loc_del_short_name:
0000B0DE C607E5              <1>     mov     byte [edi], 0E5h  ; Deleted sign
                             <1>
0000B0E1 C605[9D7E0100]02    <1>     mov     byte [DirBuff_ValidData], 2
0000B0E8 E8A4FCFFFF          <1>     call    save_directory_buffer
0000B0ED 723D                <1>     jc      short loc_delete_direntry_err_return
```

```
                                <1>
                                <1> loc_del_long_name:
0000B0EF 0FB615[DE800100]       <1>     movzx   edx, byte [DelFile_LNEL]
0000B0F6 08D2                   <1>     or      dl, dl
0000B0F8 7416                   <1>     jz      short loc_del_dir_entry_update_parent_dir_lm_date
                                <1>
0000B0FA 8835[DE800100]         <1>     mov     [DelFile_LNEL], dh ; 0
                                <1>
0000B100 0FB705[DC800100]       <1>     movzx   eax, word [DelFile_EntryCounter]
0000B107 29D0                   <1>     sub     eax, edx
                                <1>     ;jnc    short loc_del_long_name_continue
0000B109 7205                   <1>     jc      short loc_del_dir_entry_update_parent_dir_lm_date
                                <1>
                                <1> ;loc_del_direntry_inv_data_return: ; 15/10/2016 (0Dh -> 29)
                                <1> ;   mov     eax, 29 ; 0Dh (TRDOS 8086) ; Invalid data
                                <1> ;   retn
                                <1>
                                <1> loc_del_long_name_continue:
                                <1>     ; AX = Directory Entry Number of the long name last entry
0000B10B E8C1FDFFFF             <1>     call    delete_longname
                                <1>     ;jc     short loc_delete_direntry_err_return
                                <1>
                                <1> loc_del_dir_entry_update_parent_dir_lm_date:
0000B110 801D[DE800100]00       <1>     sbb     byte [DelFile_LNEL], 0 ; 0FFh if cf = 1
                                <1>
0000B117 E8FEFCFFFF             <1>     call    update_parent_dir_lmdt
0000B11C B700                   <1>     mov     bh, 0
0000B11E 80D700                 <1>     adc     bh, 0
                                <1>
0000B121 8A1D[DE800100]         <1>     mov     bl, byte [DelFile_LNEL]
                                <1>
                                <1> loc_delete_direntry_return:
0000B127 A1[D8800100]           <1>     mov     eax, [DelFile_FCluster]
                                <1> loc_delete_direntry_err_return:
0000B12C C3                     <1>     retn
                                <1>
                                <1> rename_directory_entry:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 13/11/2017
                                <1>     ; 15/10/2016
                                <1>     ; 06/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 01/08/2011 (DIR.ASM, 'proc_rename_directory_entry')
                                <1>     ; 19/11/2010
                                <1>     ; INPUT -> (Current Directory)
                                <1>     ;       CX = Directory Entry Number
                                <1>     ;       EAX = First Cluster number of file or directory
                                <1>     ;       EBX = Longname Length (dir entry count) (< 256)
                                <1>     ;       ESI = New file (or directory) name (no path).
                                <1>     ;             (ASCIIZ string)
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 0 -> successfull
                                <1>     ;       CF = 1 -> error code in EAX (AL)
                                <1>     ;
                                <1>     ; (EAX, EBX, ECX, EDX, ESI, EDI will be changed)
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [Current_FATType], 0
                                <1>     ja      short loc_rename_directory_entry
                                <1>
                                <1>     ;call   rename_fs_file_or_directory
                                <1>     ;retn
                                <1>     ; 29/07/2022
                                <1>     jmp     rename_fs_file_or_directory
                                <1> %endif
                                <1>
                                <1> loc_rename_directory_entry:
0000B12D 881D[DE800100]         <1>     mov     [DelFile_LNEL], bl
0000B133 66890D[DC800100]       <1>     mov     [DelFile_EntryCounter], cx
0000B13A A3[D8800100]           <1>     mov     [DelFile_FCluster], eax
                                <1>
0000B13F 0FB7C1                 <1>     movzx   eax, cx
0000B142 E8F3FDFFFF             <1>     call    locate_current_dir_entry
0000B147 7307                   <1>     jnc     short loc_rename_direntry_check_fcluster
                                <1>
                                <1> loc_rename_direntry_pop_retn:
0000B149 C3                     <1>     retn
                                <1>
                                <1> loc_rename_direntry_pop_invd_retn:
                                <1>     ; 29/07/2022
                                <1>     ;stc
                                <1> loc_rename_direntry_invd_retn:
                                <1>     ; 15/10/2016 (0Dh -> 29)
                                <1>     ;mov    eax, 29 ; Invalid data
                                <1>     ; 29/07/2022
0000B14A 29C0                   <1>     sub     eax, eax
0000B14C B01D                   <1>     mov     al, 29
0000B14E F9                     <1>     stc
                                <1> loc_rename_retn:
0000B14F C3                     <1>     retn
                                <1>
                                <1> loc_rename_direntry_check_fcluster:
0000B150 668B5714               <1>     mov     dx, [edi+20] ; First Cluster HW
0000B154 C1E210                 <1>     shl     edx, 16 ; 13/11/2017
0000B157 668B571A               <1>     mov     dx, [edi+26] ; First Cluster LW
0000B15B 3B15[D8800100]         <1>     cmp     edx, [DelFile_FCluster]
0000B161 75E7                   <1>     jne     short loc_rename_direntry_pop_invd_retn
                                <1>     ; ESI = New file (or directory) name. (ASCIIZ string)
                                <1>     ; 06/03/2016
                                <1>     ; TRDOS v2 - NOTE: 'convert_file_name' procedure
                                <1>     ; has been modified for eliminating following situation.
                                <1>     ;
                                <1>     ; TRDOS v1 - NOTE: If file/dir name is more than 11 bytes
                                <1>     ; without a dot, attributes (edi+11) byte will be overwritten !
                                <1>     ; (Dot file name input must be proper for 11 byte dir entry
                                <1>     ;  type file name output.)
0000B163 E84CF7FFFF             <1>     call    convert_file_name
                                <1>
0000B168 C605[9D7E0100]02       <1>         mov     byte [DirBuff_ValidData], 2
0000B16F E81DFCFFFF             <1>     call    save_directory_buffer
0000B174 72D9                   <1>     jc      short loc_rename_retn
                                <1>
                                <1> loc_rename_direntry_del_ln:
0000B176 0FB615[DE800100]       <1>     movzx   edx, byte [DelFile_LNEL]
0000B17D 08D2                   <1>     or      dl, dl
0000B17F 7410                   <1>     jz      short loc_rename_direntry_update_parent_dir_lm_date
                                <1>
0000B181 0FB705[DC800100]       <1>     movzx   eax, word [DelFile_EntryCounter]
0000B188 29D0                   <1>     sub     eax, edx
0000B18A 72BE                   <1>     jc      short loc_rename_direntry_invd_retn
                                <1>
                                <1> loc_rename_direntry_del_ln_continue:
                                <1>     ; EAX = Directory Entry Number of the long name last entry
```

```
0000B18C E840FDFFFF          <1>     call    delete_longname
                             <1>
                             <1> loc_rename_direntry_update_parent_dir_lm_date:
0000B191 E884FCFFFF          <1>     call    update_parent_dir_lmdt
0000B196 31C0                <1>     xor     eax, eax
0000B198 C3                  <1>     retn
                             <1>
                             <1> move_source_file_to_destination_file:
                             <1>     ; 19/12/2025
                             <1>     ; 11/08/2025 (TRDOS 386 Kernel v2.0.10)
                             <1>     ; 07/08/2022
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/10/2016
                             <1>     ; 11/03/2016
                             <1>     ; 10/03/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 01/08/2011 (FILE.ASM)
                             <1>     ; 04/08/2010
                             <1>     ;
                             <1>     ;   Phase 1 -> Check destination file,
                             <1>     ;              'not found' is required
                             <1>     ;   Phase 2 -> Check source file
                             <1>     ;              'found' and proper attributes is required
                             <1>     ;   Phase 3 -> Make destination directory entry,
                             <1>     ;             add new dir cluster or section if it is required
                             <1>     ;   Phase 4 -> Delete source directory entry.
                             <1>     ;       cf = 1 causes to return before the phase 4.
                             <1>     ;    (source file protection against any possible errors)
                             <1>     ;
                             <1>     ; 08/05/2011 major modification
                             <1>     ;            -> destination file deleting is removed
                             <1>     ;            for msdos move/rename compatibility.
                             <1>     ;            (Access denied error will return if
                             <1>     ;            the destination file is found...)
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Source File Pathname (Asciiz)
                             <1>     ;        EDI = Destination File Pathname (Asciiz)
                             <1>     ;        AL = 0 --> Interrupt (System call)
                             <1>     ;        AL > 0 --> Command Interpreter (Question)
                             <1>     ;        AL = 1 --> Question Phase
                             <1>     ;        AL = 2 --> Progress Phase
                             <1>     ; OUTPUT ->
                             <1>     ;       cf = 0 -> OK
                             <1>     ;        EAX = Destination directory first cluster
                             <1>     ;        ESI = Logical DOS drive description table
                             <1>     ;        EBX = Destination file structure offset
                             <1>     ;        CX = 0 (CX > 0 --> calculate free space error)
                             <1>     ;        cf = 1 -> Error code in EAX (AL)
                             <1>     ;
                             <1>     ;  (EDX, ECX, EBX, ESI, EDI will be changed)
                             <1>
0000B199 3C02                <1>     cmp     al, 2
                             <1>     ;je     msftdf_df2_check_directory
                             <1>     ; 29/07/2022
0000B19B 7505                <1>     jne     short msftdf
0000B19D E96D010000          <1>     jmp     msftdf_df2_check_directory
                             <1> msftdf:
0000B1A2 A2[62820100]        <1>     mov     [move_cmd_phase], al
                             <1>
                             <1> msftdf_parse_sf_path:
                             <1>     ; ESI = ASCIIZ pathname (Source)
0000B1A7 57                  <1>     push    edi
0000B1A8 BF[60810100]        <1>     mov     edi, SourceFile_Drv
0000B1AD E8CCF7FFFF          <1>     call    parse_path_name
0000B1B2 5E                  <1>     pop     esi
0000B1B3 7211                <1>     jc      short msftdf_psf_retn
                             <1>
                             <1> msftdf_parse_df_path:
                             <1>     ; ESI = ASCIIZ pathname       (Destination)
0000B1B5 BF[E0810100]        <1>     mov     edi, DestinationFile_Drv
0000B1BA E8BFF7FFFF          <1>     call    parse_path_name
0000B1BF 7306                <1>     jnc     short msftdf_check_sf_drv
                             <1>
0000B1C1 3C01                <1>     cmp     al, 1 ; File or directory name is not existing
0000B1C3 7602                <1>     jna     short msftdf_check_sf_drv
                             <1>
                             <1> msftdf_stc_retn:
0000B1C5 F9                  <1>     stc
                             <1> msftdf_psf_retn:
0000B1C6 C3                  <1>     retn
                             <1>
                             <1> msftdf_check_sf_drv:
0000B1C7 A0[60810100]        <1>     mov     al, [SourceFile_Drv]
                             <1>
                             <1> msftdf_check_df_drv:
0000B1CC 8A15[E0810100]      <1>     mov     dl, [DestinationFile_Drv]
                             <1>
                             <1> msftdf_compare_sf_df_drv:
                             <1>     ; 11/08/2025
                             <1>     ;sub    ebx, ebx
                             <1>     ;mov    bh, [Current_Drv]
0000B1D2 38C2                <1>     cmp     dl, al
0000B1D4 7408                <1>     je      short msftdf_check_sf_df_drv_ok
                             <1>
                             <1> msftdf_not_same_drv:
                             <1>         ; DL = source file's drive number
0000B1D6 88C6                <1>     mov     dh, al ; destination file's drive number
                             <1>     ; 15/10/2016 (11h -> 21)
                             <1>     ;mov    eax, 21 ; Not the same drive
                             <1>     ; 29/07/2022
0000B1D8 29C0                <1>     sub     eax, eax
0000B1DA B015                <1>     mov     al, 21
0000B1DC F9                  <1>     stc
0000B1DD C3                  <1>     retn
                             <1>
                             <1> msftdf_check_sf_df_drv_ok:
0000B1DE 8815[63820100]      <1>     mov     [msftdf_sf_df_drv], dl
                             <1>
0000B1E4 29C0                <1>         sub eax, eax
0000B1E6 88D4                <1>     mov     ah, dl
0000B1E8 0500010900          <1>     add     eax, Logical_DOSDisks
0000B1ED A3[64820100]        <1>     mov     [msftdf_drv_offset], eax
                             <1>
                             <1>     ;cmp    dl, bh ; byte [Current_Drv]
                             <1>     ; 11/08/2025
0000B1F2 3A15[76770100]      <1>     cmp     dl, [Current_Drv]
0000B1F8 7407                <1>     je      short msftdf_df_check_directory
                             <1>
                             <1> msftdf_change_drv:
0000B1FA E8F1C4FFFF          <1>     call    change_current_drive
0000B1FF 726C                <1>     jc      short msftdf_df_error_retn
                             <1>
                             <1> msftdf_check_destination_file:
                             <1> msftdf_df_check_directory:
0000B201 BE[E1810100]        <1>     mov     esi, DestinationFile_Directory
```

```
0000B206 803E20              <1>     cmp     byte [esi], 20h
0000B209 760F                <1>     jna     short msftdf_df_find_1
                             <1>
                             <1> msftdf_df_change_directory:
0000B20B FE05[8E2E0100]      <1>     inc     byte [Restore_CDIR]
0000B211 30E4                <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000B213 E8D3F1FFFF          <1>     call    change_current_directory
0000B218 7253                <1>     jc      short msftdf_df_error_retn
                             <1>
                             <1> ;msftdf_df_change_prompt_dir_string:
                             <1> ;   call    change_prompt_dir_string
                             <1>
                             <1> msftdf_df_find_1:
0000B21A BE[22820100]        <1>         mov     esi, DestinationFile_Name
0000B21F 803E20              <1>     cmp     byte [esi], 20h
0000B222 7631                <1>     jna     short msftdf_df_copy_sf_name
                             <1>
                             <1> msftdf_df_find_2:
                             <1>     ;xor    ax, ax ; DestinationFile_AttributesMask -> any/zero
                             <1>     ; 11/08/2025
0000B224 31C0                <1>     xor     eax, eax
0000B226 E84FD7FFFF          <1>     call    find_first_file
                             <1>     ;jnc    msftdf_permission_denied_retn
                             <1>     ; 29/07/2022
0000B22B 7205                <1>     jc      short msftdf_df_check_error_code
0000B22D E98B000000          <1>     jmp     msftdf_permission_denied_retn
                             <1>
                             <1> msftdf_df_check_error_code:
                             <1>     ;cmp    eax, 2 ; File not found error
0000B232 3C02                <1>     cmp     al, 2
0000B234 7536                <1>     jne     short msftdf_df_stc_retn
                             <1>
                             <1> msftdf_df_check_fname:
                             <1>     ; 15/10/2016
0000B236 BE[22820100]        <1>     mov     esi, DestinationFile_Name ; *
0000B23B E899DAFFFF          <1>     call    check_filename
0000B240 7307                <1>     jnc     short msftdf_convert_df_direntry_name
                             <1>     ; invalid file name chars !
0000B242 B81A000000          <1>     mov     eax, ERR_INV_FILE_NAME  ; 26
0000B247 EB23                <1>     jmp     short msftdf_df_stc_retn
                             <1>
                             <1> msftdf_convert_df_direntry_name:
                             <1>     ;mov    esi, DestinationFile_Name ; *
0000B249 BF[32820100]        <1>     mov     edi, DestinationFile_DirEntry
0000B24E E861F6FFFF          <1>     call    convert_file_name
0000B253 EB19                <1>     jmp     short msftdf_restore_current_dir_1
                             <1>
                             <1> msftdf_df_copy_sf_name:
0000B255 89F7                <1>     mov     edi, esi
0000B257 57                  <1>     push    edi
0000B258 BE[A2810100]        <1>         mov     esi, SourceFile_Name
                             <1>     ;mov    ecx, 12
                             <1>     ; 29/07/2022
0000B25D 29C9                <1>     sub     ecx, ecx
0000B25F B10C                <1>     mov     cl, 12
                             <1> msftdf_df_copy_sf_name_loop:
0000B261 AC                  <1>     lodsb
0000B262 AA                  <1>         stosb
0000B263 08C0                <1>     or      al, al
0000B265 7402                <1>     jz      short msftdf_df_copy_sf_name_ok
0000B267 E2F8                <1>         loop    msftdf_df_copy_sf_name_loop
                             <1> msftdf_df_copy_sf_name_ok:
0000B269 5E                  <1>     pop     esi
0000B26A EBB8                <1>     jmp     short msftdf_df_find_2
                             <1>
                             <1> msftdf_df_stc_retn:
0000B26C F9                  <1>     stc
                             <1> msftdf_restore_cdir_failed:
                             <1> msftdf_df_error_retn:
0000B26D C3                  <1>     retn
                             <1>
                             <1> msftdf_restore_current_dir_1:
0000B26E 803D[8E2E0100]00    <1>     cmp     byte [Restore_CDIR], 0
0000B275 760D                <1>     jna     short msftdf_sf_check_directory
0000B277 8B35[64820100]      <1>     mov     esi, [msftdf_drv_offset]
0000B27D E81FC5FFFF          <1>     call    restore_current_directory
0000B282 72E9                <1>     jc      short msftdf_restore_cdir_failed
                             <1>
                             <1> msftdf_sf_check_directory:
0000B284 BE[61810100]        <1>     mov     esi, SourceFile_Directory
0000B289 803E20              <1>     cmp     byte [esi], 20h
0000B28C 760F                <1>     jna     short msftdf_sf_find
                             <1> msftdf_sf_change_directory:
0000B28E FE05[8E2E0100]      <1>     inc     byte [Restore_CDIR]
0000B294 30E4                <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000B296 E850F1FFFF          <1>     call    change_current_directory
0000B29B 7225                <1>     jc      short msftdf_return
                             <1>
                             <1> ;msftdf_sf_change_prompt_dir_string:
                             <1> ;   call    change_prompt_dir_string
                             <1>
                             <1> msftdf_sf_find:
0000B29D BE[A2810100]        <1>         mov     esi, SourceFile_Name  ; Offset 66
0000B2A2 66B80018            <1>     mov     ax, 1800h ; Only files
0000B2A6 E8CFD6FFFF          <1>     call    find_first_file
0000B2AB 7215                <1>     jc      short msftdf_return
                             <1>
                             <1> msftdf_sf_ambgfn_check:
0000B2AD 6609D2              <1>     or      dx, dx ; Ambiguous filename chars used sign (DX>0)
0000B2B0 7406                <1>     jz      short msftdf_sf_found
                             <1>
                             <1> msftdf_ambiguous_file_name_error:
                             <1>     ;mov    eax, 2 ; File not found error
                             <1>     ; 29/07/2022
0000B2B2 29C0                <1>     sub     eax, eax
0000B2B4 B002                <1>     mov     al, 2
0000B2B6 F9                  <1>     stc
0000B2B7 C3                  <1>     retn
                             <1>
                             <1> msftdf_sf_found:
0000B2B8 80E31F              <1>     and     bl, 1Fh ; Attributes, D-V-S-H-R
0000B2BB 7415                <1>     jz      short msftdf_save_sf_structure
                             <1>
                             <1> msftdf_permission_denied_retn:
                             <1>     ;mov    eax, 05h ; Access (Permission) denied !
                             <1>     ; 29/07/2022
0000B2BD 29C0                <1>     sub     eax, eax
0000B2BF B005                <1>     mov     al, 5
0000B2C1 F9                  <1>     stc
                             <1> msftdf_rest_cdir_err_retn:
                             <1> msftdf_return:
0000B2C2 C3                  <1>     retn
                             <1>
                             <1> msftdf_phase_1_return:
```

```
0000B2C3 31C0                <1>     xor     eax, eax
0000B2C5 A2[62820100]        <1>     mov     [move_cmd_phase], al ; 0
0000B2CA FEC0                <1>     inc     al ; mov al, 1
0000B2CC BB[0FB30000]        <1>     mov     ebx, msftdf_df2_check_directory
                             <1>     ;mov    edx, 0FFFFFFFFh
0000B2D1 C3                  <1>     retn
                             <1>
                             <1> msftdf_save_sf_structure:
0000B2D2 BE[68800100]        <1>     mov     esi, FindFile_DirEntry
0000B2D7 BF[B2810100]        <1>     mov     edi, SourceFile_DirEntry
                             <1>     ;mov    ecx, 8
                             <1>     ; 29/07/2022
0000B2DC 29C9                <1>     sub     ecx, ecx
0000B2DE B108                <1>     mov     cl, 8
0000B2E0 F3A5                <1>     rep     movsd
                             <1>
                             <1> msftdf_df_copy_sf_parameters:
                             <1>     ;mov    esi, 11
                             <1>     ;mov    edi, esi
                             <1>     ;add    esi, SourceFile_DirEntry
                             <1>     ;add    edi, DestinationFile_DirEntry
                             <1>     ; 11/08/2025
0000B2E2 BE[BD810100]        <1>     mov     esi, SourceFile_DirEntry+11
0000B2E7 BF[3D820100]        <1>     mov     edi, DestinationFile_DirEntry+11
                             <1>     ;mov    ecx, 21
0000B2EC B115                <1>     mov     cl, 21
0000B2EE F3A4                <1>     rep     movsb
                             <1>
                             <1> msftdf_restore_current_dir_2:
0000B2F0 803D[8E2E0100]00    <1>     cmp     byte [Restore_CDIR], 0
0000B2F7 760D                <1>     jna     short msftdf_df2_check_move_cmd_phase
0000B2F9 8B35[64820100]      <1>     mov     esi, [msftdf_drv_offset]
0000B2FF E89DC4FFFF          <1>     call    restore_current_directory
0000B304 72BC                <1>     jc      short msftdf_rest_cdir_err_retn
                             <1>
                             <1> msftdf_df2_check_move_cmd_phase:
0000B306 803D[62820100]01    <1>     cmp     byte [move_cmd_phase], 1
0000B30D 74B4                <1>     je      short msftdf_phase_1_return
                             <1>
                             <1> msftdf_df2_check_directory:
0000B30F BE[E1810100]        <1>     mov     esi, DestinationFile_Directory
0000B314 803E20              <1>     cmp     byte [esi], 20h
0000B317 760F                <1>     jna     short msftdf_make_dfde_locate_ffe_on_directory
                             <1> msftdf_df2_change_directory:
0000B319 FE05[8E2E0100]      <1>     inc     byte [Restore_CDIR]
0000B31F 30E4                <1>     xor     ah, ah ; CD_COMMAND sign -> 0
0000B321 E8C5F0FFFF          <1>     call    change_current_directory
0000B326 729A                <1>     jc      short msftdf_return
                             <1>
                             <1> ;msftdf_df2_change_prompt_dir_string:
                             <1> ;   call    change_prompt_dir_string
                             <1>
                             <1> msftdf_make_dfde_locate_ffe_on_directory:
                             <1>     ; Current directory fcluster <> Directory buffer cluster
                             <1>     ; Current directory will be reloaded by
                             <1>     ; 'locate_current_dir_file' procedure
                             <1>     ;
                             <1>     ;xor    ax, ax
0000B328 31C0                <1>     xor     eax, eax
0000B32A 89C1                <1>     mov     ecx, eax
0000B32C 6649                <1>     dec     cx ; FFFFh
                             <1>             ; CX = FFFFh -> find first deleted or free entry
                             <1>             ; ESI would be ASCIIZ filename address if the call
                             <1>             ; would not be for first free or deleted dir entry
0000B32E E8D5F2FFFF          <1>     call    locate_current_dir_file
                             <1>     ; 07/08/2022
0000B333 7328                <1>     jnc     short msftdf_make_dfde_set_ff_dir_entry
                             <1>
                             <1>     ;cmp    eax, 2
0000B335 3C02                <1>         cmp al, 2
0000B337 7520                <1>     jne     short msftdf_error_retn
                             <1>
                             <1> msftdf_add_new_dir_entry_check_fs:
0000B339 8B35[64820100]      <1>     mov     esi, [msftdf_drv_offset]
0000B33F A1[A27E0100]        <1>     mov     eax, [DirBuff_Cluster]
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     byte [esi+LD_FATType], 0
                             <1>     ja      short msftdf_add_new_subdir_cluster
                             <1>
                             <1> msftdf_add_new_fs_subdir_section:
                             <1>     ;CL=0, CH=E5h --> deleted entry, CH=0 --> free entry
                             <1>         ;xor        cx, cx
                             <1>     xor     ch, ch ; cx = 0 --> add a new subdir section
                             <1>     call    add_new_fs_section
                             <1>         jc  short msftdf_dsfde_error_retn
                             <1>     ;mov    [createfile_LastDirCluster], eax
                             <1>
                             <1>     call    load_FS_sub_directory
                             <1>     ;mov    ebx, Directory_Buffer
                             <1>     jnc     short msftdf_add_new_fs_subdir_section_ok
                             <1>     retn
                             <1> %endif
                             <1>
                             <1> msftdf_add_new_subdir_cluster:
0000B344 E8C3140000          <1>     call    add_new_cluster
0000B349 720F                <1>     jc      short msftdf_dsfde_error_retn
                             <1>
                             <1>     ;mov    [createfile_LastDirCluster], eax
                             <1>
0000B34B E8F50D0000          <1>     call    load_FAT_sub_directory
0000B350 7309                <1>     jnc     short msftdf_add_new_subdir_cluster_ok
                             <1>     ; EBX = Directory buffer address
                             <1>
                             <1> msftdf_ansdc_update_parent_dir_lmdt:
                             <1> msftdf_make_dfde_err_upd_pdir_lmdt:
0000B352 50                  <1>     push    eax
0000B353 E8C2FAFFFF          <1>     call    update_parent_dir_lmdt
0000B358 58                  <1>     pop     eax
                             <1>
                             <1> msftdf_error_retn:
0000B359 F9                  <1>     stc
                             <1> msftdf_dsfde_restore_cdir_failed:
                             <1> msftdf_dsfde_error_retn:
0000B35A C3                  <1>     retn
                             <1>
                             <1> msftdf_add_new_fs_subdir_section_ok:
                             <1> msftdf_add_new_subdir_cluster_ok:
0000B35B 89DF                <1>     mov     edi, ebx ; Directory buffer address
                             <1>
                             <1> msftdf_make_dfde_set_ff_dir_entry:
0000B35D 8B15[70770100]      <1>     mov     edx, [Current_Dir_FCluster]
0000B363 8915[C8820100]      <1>     mov     [createfile_FFCluster], edx
```

```
                                <1>     ; EDI = Directory entry offset
0000B369 BE[32820100]           <1>     mov     esi, DestinationFile_DirEntry
                                <1>     ;mov    ecx, 8
                                <1>     ; 29/07/2022
0000B36E 29C9                   <1>     sub     ecx, ecx
0000B370 B108                   <1>     mov     cl, 8
0000B372 F3A5                   <1>     rep     movsd
                                <1>
0000B374 C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
0000B37B E811FAFFFF             <1>     call    save_directory_buffer
0000B380 72D0                   <1>     jc      short msftdf_make_dfde_err_upd_pdir_lmdt
                                <1>
                                <1> msftdf_make_dfde_update_pdir_lmdt:
0000B382 E893FAFFFF             <1>     call    update_parent_dir_lmdt
                                <1>
                                <1> msftdf_dsfde_restore_current_dir_1:
0000B387 803D[8E2E0100]00       <1>     cmp     byte [Restore_CDIR], 0
0000B38E 760D                   <1>     jna     short msftdf_dsfde_check_directory
0000B390 8B35[64820100]         <1>     mov     esi, [msftdf_drv_offset]
0000B396 E806C4FFFF             <1>     call    restore_current_directory
0000B39B 72BD                   <1>     jc      short msftdf_dsfde_restore_cdir_failed
                                <1>
                                <1> msftdf_dsfde_check_directory:
0000B39D BE[61810100]           <1>     mov     esi, SourceFile_Directory
0000B3A2 803E20                 <1>     cmp     byte [esi], 20h
0000B3A5 760F                   <1>     jna     short msftdf_dsfde_find_file
                                <1>
                                <1> msftdf_dsfde_change_directory:
0000B3A7 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
0000B3AD 28E4                   <1>     sub     ah, ah ; CD_COMMAND sign -> 0
0000B3AF E837F0FFFF             <1>     call    change_current_directory
0000B3B4 72A4                   <1>     jc      short msftdf_dsfde_error_retn
                                <1>
                                <1> ;msftdf_dsfde_sf_change_prompt_dir_string:
                                <1> ;   call    change_prompt_dir_string
                                <1>
                                <1> msftdf_dsfde_find_file:
0000B3B6 BE[A2810100]           <1>     mov     esi, SourceFile_Name  ; Offset 66
0000B3BB 668B460E               <1>     mov     ax, [esi+14] ; 80 -> SourceFile_AttributesMask
0000B3BF E8B6D5FFFF             <1>     call    find_first_file
0000B3C4 7294                   <1>     jc      short msftdf_dsfde_error_retn
                                <1>
                                <1> msftdf_dsfde_delete_direntry:
0000B3C6 8B35[64820100]         <1>     mov     esi, [msftdf_drv_offset]
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ja      short msftdf_delete_FAT_direntry
                                <1>
                                <1>     xor     bl, bl
                                <1>     ; BL = 0 -> File
                                <1>     ; EDI -> Directory buffer entry offset/address
                                <1>     call    delete_fs_directory_entry
                                <1>     jnc     short msftdf_dsfde_restore_current_dir_2
                                <1>     retn
                                <1> %endif
                                <1>
                                <1> msftdf_delete_FAT_direntry:
0000B3CC 8A1D[65800100]         <1>     mov     bl, [FindFile_LongNameEntryLength]
0000B3D2 668B0D[90800100]       <1>     mov     cx, [FindFile_DirEntryNumber]
                                <1>     ; ESI = Logical DOS drive description table address
                                <1>     ; EDI = Directory buffer entry offset/address
0000B3D9 E8E3FCFFFF             <1>     call    delete_directory_entry
0000B3DE 721C                   <1>     jc      short msftdf_retn
                                <1>
                                <1> msftdf_dsfde_restore_current_dir_2:
0000B3E0 803D[8E2E0100]00       <1>     cmp     byte [Restore_CDIR], 0
0000B3E7 7607                   <1>     jna     short msftdf_new_dir_fcluster_retn
                                <1>     ;mov    esi, [msftdf_drv_offset]
0000B3E9 E8B3C3FFFF             <1>     call    restore_current_directory
0000B3EE 720C                   <1>     jc      short msftdf_retn
                                <1>
                                <1> msftdf_new_dir_fcluster_retn:
0000B3F0 31C9                   <1>     xor     ecx, ecx
0000B3F2 A1[C8820100]           <1>     mov     eax, [createfile_FFCluster]
0000B3F7 BB[E0810100]           <1>     mov     ebx, DestinationFile_Drv
                                <1>
                                <1> msftdf_retn:
0000B3FC C3                     <1>     retn
                                <1>
                                <1> copy_source_file_to_destination_file:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 31/08/2024
                                <1>     ; 30/08/2024
                                <1>     ; 29/08/2024
                                <1>     ; 26/08/2024
                                <1>     ; 25/08/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 17/10/2016
                                <1>     ; 16/10/2016
                                <1>     ; 15/10/2016
                                <1>     ; 30/03/2016, 31/03/2016
                                <1>     ; 24/03/2016, 25/03/2016, 28/03/2016
                                <1>     ; 21/03/2016, 22/03/2016, 23/03/2016
                                <1>     ; 16/03/2016, 17/03/2016, 18/03/2016
                                <1>     ; 15/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 02/09/2011 (FILE.ASM 'copy_source_file_to_destination_file')
                                <1>     ; 01/08/2010 - 18/05/2011
                                <1>     ;
                                <1>     ;   Command Interpreter phase 1 enter ->
                                <1>     ;           AL = 1 -> Caller is command interpreter
                                <1>     ;           AL = 2 -> The second call, re-enter/continue
                                <1>     ;   Phase 1 -> Check source file
                                <1>     ;              'found' is required
                                <1>     ;   Phase 2 -> Check destination file,
                                <1>     ;              save 'found' or 'not found' status
                                <1>     ;              'permission denied' error will be return
                                <1>     ;              if attributes have not for ordinary file
                                <1>     ;              without readonly attribute
                                <1>     ;   Command Interpreter phase 1 return ->
                                <1>     ;              DH = Source file attributes
                                <1>     ;              DL = Destination file found status
                                <1>     ;              EAX = 0
                                <1>     ;   Command Interpreter phase 2 enter ->
                                <1>     ;              AL = 2 -> Continue from the last position
                                <1>     ;              AH =
                                <1>     ;   Phase 3 -> Load source file or use read/write cluster method
                                <1>     ;   Phase 4 -> Create destination file if it is not found
                                <1>     ;   Phase 5 -> Open destination file
                                <1>     ;   Phase 6 -> Read from source and write to destination
                                <1>     ;   Phase 7 -> Unload source file, if it is loaded at memory
                                <1>     ;       cf = 1 causes to return before the phase 7
                                <1>     ;              but loaded file will be unloaded
```

```
                                <1>     ;             (allocated memory block will be deallocated)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Source File Pathname (Asciiz)
                                <1>     ;        EDI = Destination File Pathname (Asciiz)
                                <1>     ;        AL = 0 --> Interrupt (System call)
                                <1>     ;        AL > 0 --> Command Interpreter (Question)
                                <1>     ;        AL = 1 --> Question Phase
                                <1>     ;        AL = 2 --> Progress Phase
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      cf = 0 -> OK ; (*)
                                <1>     ;      EAX = Destination file first cluster
                                <1>     ;
                                <1>     ;      31/08/2024 - TRDOS v2.0.9 ; (*)
                                <1>     ; obsolete ;CL > 0 if there is file reading error before EOF
                                <1>     ;          ;           (incomplete copy)
                                <1>     ;          ;CH > 0 if file is (full) loaded at memory
                                <1>     ;
                                <1>     ;      cf = 1 -> Error code in AL (EAX) ; (*)
                                <1>     ;
                                <1>     ; (EBX, ECX, ESI, EDI register contents will be changed)
                                <1>
0000B3FD 3C02                   <1>     cmp    al, 2
                                <1>     ;je    csftdf2_check_cdrv
                                <1>     ; 29/07/2022
0000B3FF 7205                   <1>     jb     short csftdf_ph1
                                <1>     ;jmp   csftdf2_check_cdrv
0000B401 E943020000             <1>     jmp    csftdf_ph2
                                <1>
                                <1> ; Phase 1
                                <1>     ; 29/07/2022
                                <1> csftdf_ph1:
0000B406 A2[88820100]           <1>     mov    byte [copy_cmd_phase], al
                                <1>
0000B40B 57                     <1>     push   edi ; *
                                <1>
                                <1> csftdf_parse_sf_path:
0000B40C BF[60810100]           <1>     mov    edi, SourceFile_Drv
0000B411 E868F5FFFF             <1>     call   parse_path_name
0000B416 721C                   <1>     jc     short csftdf_parse_sf_path_failed
                                <1>
                                <1> csftdf_parse_df_path:
0000B418 5E                     <1>     pop    esi ; * (pushed edi)
                                <1>
                                <1> csftdf_sf_check_filename_exists:
0000B419 803D[A2810100]21       <1>     cmp    byte [SourceFile_Name], 21h
0000B420 7214                   <1>     jb     short csftdf_sf_file_not_found_error
                                <1>
0000B422 BF[E0810100]           <1>     mov    edi, DestinationFile_Drv
0000B427 E852F5FFFF             <1>     call   parse_path_name
0000B42C 730E                   <1>     jnc    short csftdf_check_sf_cdrv
                                <1>
0000B42E 3C01                   <1>     cmp    al, 1 ; File or directory name is not existing
0000B430 760A                   <1>     jna    short csftdf_check_sf_cdrv
                                <1>
                                <1> csftdf_parse_df_path_failed:
0000B432 F9                     <1>     stc
                                <1> csftdf_sf_error_retn:
0000B433 C3                     <1>     retn
                                <1>
                                <1> csftdf_parse_sf_path_failed:
0000B434 5F                     <1>     pop    edi ; *
                                <1>     ;jmp   short csftdf_sf_error_retn
                                <1>     ; 25/08/2024
0000B435 C3                     <1>     retn
                                <1>
                                <1> csftdf_sf_file_not_found_error:
0000B436 B802000000             <1>     mov    eax, 2 ; File not found
                                <1>     ;jmp   short csftdf_sf_error_retn
                                <1>     ; 25/08/2024
0000B43B C3                     <1>     retn
                                <1>
                                <1> csftdf_check_sf_cdrv:
0000B43C 8A3D[76770100]         <1>     mov    bh, [Current_Drv]
                                <1>
0000B442 883D[8B820100]         <1>     mov    [csftdf_cdrv], bh ; 23/03/2016
                                <1>
0000B448 8A15[60810100]         <1>     mov    dl, [SourceFile_Drv]
0000B44E 38FA                   <1>     cmp    dl, bh ; byte [Current_Drv]
0000B450 7407                   <1>     je     short csftdf_sf_check_directory
                                <1>
0000B452 E899C2FFFF             <1>     call   change_current_drive
0000B457 72DA                   <1>     jc     short csftdf_sf_error_retn
                                <1>
                                <1> csftdf_sf_check_directory:
0000B459 BE[61810100]           <1>     mov    esi, SourceFile_Directory
0000B45E 803E20                 <1>     cmp    byte [esi], 20h
0000B461 760F                   <1>     jna    short csftdf_find_sf
                                <1>
                                <1> csftdf_sf_change_directory:
0000B463 FE05[8E2E0100]         <1>     inc    byte [Restore_CDIR]
0000B469 30E4                   <1>     xor    ah, ah ; CD_COMMAND sign -> 0
0000B46B E87BEFFFFF             <1>     call   change_current_directory
0000B470 72C1                   <1>     jc     short csftdf_sf_error_retn
                                <1>
                                <1> ;csftdf_sf_change_prompt_dir_string:
                                <1> ;   call   change_prompt_dir_string
                                <1>
                                <1> csftdf_find_sf:
0000B472 BE[A2810100]           <1>     mov    esi, SourceFile_Name
0000B477 66B80018               <1>     mov    ax, 1800h ; Except volume label and dirs
0000B47B E8FAD4FFFF             <1>     call   find_first_file
0000B480 72B1                   <1>     jc     short csftdf_sf_error_retn
                                <1>
                                <1> csftdf_sf_ambgfn_check:
0000B482 6621D2                 <1>     and    dx, dx ; Ambiguous filename chars used sign (DX>0)
0000B485 7406                   <1>     jz     short csftdf_sf_found
                                <1>
                                <1> csftdf_ambiguous_file_name_error:
                                <1>     ;mov   eax, 2 ; File not found error
                                <1>     ; 29/07/2022
0000B487 29C0                   <1>     sub    eax, eax
0000B489 B002                   <1>     mov    al, 2
0000B48B F9                     <1>     stc
0000B48C C3                     <1>     retn
                                <1>
                                <1> csftdf_sf_found:
0000B48D A3[8C820100]           <1>     mov    [csftdf_filesize], eax
                                <1>
0000B492 09C0                   <1>     or     eax, eax
0000B494 7504                   <1>     jnz    short csftdf_set_source_file_direntry
                                <1>
                                <1> ;csftdf_sf_file_size_zero:
```

```
                                <1>     ;mov    eax, 20 ; TRDOS zero length (file size) error
                                <1>     ; 25/08/2024
                                <1>     ; eax = 0
                                <1>     ; 29/07/2022
                                <1>     ;sub    eax, eax
0000B496 B014                   <1>     mov     al, 20
0000B498 F9                     <1>     stc
0000B499 C3                     <1>     retn
                                <1>
                                <1> csftdf_set_source_file_direntry:
0000B49A BE[68800100]           <1>     mov     esi, FindFile_DirEntry
0000B49F BF[B2810100]           <1>     mov     edi, SourceFile_DirEntry
                                <1>     ;mov    ecx, 8
                                <1>     ; 29/07/2022
0000B4A4 31C9                   <1>     xor     ecx, ecx
0000B4A6 B108                   <1>     mov     cl, 8
0000B4A8 F3A5                   <1>     rep     movsd
                                <1>
                                <1> csftdf_sf_restore_cdrv:
                                <1>     ; 22/03/2016
0000B4AA 8A15[8B820100]         <1>     mov     dl, [csftdf_cdrv]
0000B4B0 3A15[76770100]         <1>     cmp     dl, [Current_Drv]
0000B4B6 7407                   <1>     je      short csftdf_sf_restore_cdir
0000B4B8 E833C2FFFF             <1>     call    change_current_drive
0000B4BD 724F                   <1>     jc      short csftdf_df_error_retn ; 30/03/2016
                                <1>
                                <1> csftdf_sf_restore_cdir:
0000B4BF 803D[8E2E0100]00       <1>     cmp     byte [Restore_CDIR], 0
0000B4C6 7612                   <1>     jna     short csftdf_df_check_filename_exists
0000B4C8 29C0                   <1>     sub     eax, eax
0000B4CA BE00010900             <1>     mov     esi, Logical_DOSDisks
0000B4CF 88D4                   <1>     mov     ah, dl ; byte [csftdf_cdrv]
0000B4D1 01C6                   <1>     add     esi, eax
0000B4D3 E8C9C2FFFF             <1>     call    restore_current_directory
0000B4D8 7234                   <1>     jc      short csftdf_df_error_retn
                                <1>
                                <1> csftdf_df_check_filename_exists:
0000B4DA 803D[22820100]20       <1>     cmp     byte [DestinationFile_Name], 20h
0000B4E1 7716                   <1>     ja      short csftdf_check_df_cdrv
                                <1>
                                <1> csftdf_copy_sf_name:
0000B4E3 BF[22820100]           <1>     mov     edi, DestinationFile_Name
0000B4E8 BE[A2810100]           <1>     mov     esi, SourceFile_Name
0000B4ED B10C                   <1>     mov     cl, 12
                                <1>
                                <1> csftdf_df_copy_sf_name_loop:
0000B4EF AC                     <1>     lodsb
0000B4F0 AA                     <1>     stosb
0000B4F1 08C0                   <1>     or      al, al
0000B4F3 7404                   <1>     jz      short csftdf_check_df_cdrv
0000B4F5 FEC9                   <1>     dec     cl
0000B4F7 75F6                   <1>     jnz     csftdf_df_copy_sf_name_loop
                                <1>
                                <1> csftdf_check_df_cdrv:
0000B4F9 8A15[E0810100]         <1>     mov     dl, [DestinationFile_Drv]
0000B4FF 3A15[76770100]         <1>     cmp     dl, [Current_Drv]
0000B505 7408                   <1>     je      short csftdf_df_check_directory
                                <1>
0000B507 E8E4C1FFFF             <1>     call    change_current_drive
0000B50C 7301                   <1>     jnc     short csftdf_df_check_directory
                                <1>
                                <1> csftdf_df_error_retn:
0000B50E C3                     <1>     retn
                                <1>
                                <1> csftdf_df_check_directory:
0000B50F BE[E1810100]           <1>     mov     esi, DestinationFile_Directory
0000B514 803E20                 <1>         cmp     byte [esi], 20h
0000B517 760F                   <1>     jna     short csftdf_find_df
                                <1>
                                <1> csftdf_df_change_directory:
0000B519 FE05[8E2E0100]         <1>     inc     byte [Restore_CDIR]
0000B51F 28E4                   <1>     sub     ah, ah ; CD_COMMAND sign -> 0
0000B521 E8C5EEFFFF             <1>     call    change_current_directory
0000B526 72E6                   <1>     jc      short csftdf_df_error_retn
                                <1>
                                <1> ;csftdf_df_change_prompt_dir_string:
                                <1> ;   call    change_prompt_dir_string
                                <1>
                                <1> csftdf_find_df:
                                <1>     ; 23/03/2016
0000B528 29DB                   <1>     sub     ebx, ebx
0000B52A 8A3D[E0810100]         <1>     mov     bh, [DestinationFile_Drv]
0000B530 81C300010900           <1>     add     ebx, Logical_DOSDisks
0000B536 891D[B8820100]         <1>     mov     [csftdf_df_drv_dt], ebx
                                <1>
0000B53C BE[22820100]           <1>     mov     esi, DestinationFile_Name
                                <1>     ;xor    ax, ax
                                <1>     ; 25/08/2024
0000B541 31C0                   <1>     xor     eax, eax
                                <1>             ; DestinationFile_AttributesMask -> any/zero
0000B543 E832D4FFFF             <1>     call    find_first_file
0000B548 7217                   <1>     jc      short csftdf_df_check_error_code
                                <1>
                                <1> csftdf_df_ambgfn_check:
0000B54A 6609D2                 <1>     or      dx, dx ; Ambiguous filename chars used sign (DX>0)
0000B54D 7529                   <1>     jnz     short csftdf_df_error_inv_fname
                                <1>
                                <1> csftdf_df_found:
0000B54F C605[8A820100]01       <1>     mov     byte [DestinationFileFound], 1
                                <1>     ; 17/10/2016 (cl -> bl)
0000B556 80E31F                 <1>     and     bl, 1Fh ; Attributes, D-V-S-H-R
0000B559 7452                   <1>     jz      short csftdf_df_save_first_cluster
                                <1>
                                <1> csftdf_df_permission_denied_retn:
                                <1>     ;mov    eax, 05h ; Access/Permission denied.
                                <1>     ; 29/07/2022
0000B55B 29C0                   <1>     sub     eax, eax
0000B55D B005                   <1>     mov     al, 5
                                <1> csftdf_df_error_stc_retn:
0000B55F F9                     <1>     stc
0000B560 C3                     <1>     retn
                                <1>
                                <1> csftdf_df_check_error_code:
                                <1>     ;cmp    eax, 2
0000B561 3C02                   <1>     cmp     al, 2
0000B563 75FA                   <1>     jne     short csftdf_df_error_stc_retn
                                <1>
0000B565 C605[8A820100]00       <1>     mov     byte [DestinationFileFound], 0
                                <1>
                                <1>     ; 15/10/2016
0000B56C BE[58800100]           <1>     mov     esi, FindFile_Name ; *
0000B571 E863D7FFFF             <1>     call    check_filename
0000B576 7306                   <1>     jnc     short csftdf_df_valid_fname
                                <1> csftdf_df_error_inv_fname: ; 'invalid file name !'
```

```
                                <1>     ;mov    eax, ERR_INV_FILE_NAME  ; 26
                                <1>     ; 29/07/2022
0000B578 29C0                   <1>     sub     eax, eax
0000B57A B01A                   <1>     mov     al, ERR_INV_FILE_NAME ; 26
0000B57C F9                     <1>     stc
0000B57D C3                     <1>     retn
                                <1>
                                <1> csftdf_df_valid_fname:
                                <1>     ; 21/03/2016
                                <1>     ; (Capitalized file name)
                                <1>     ;mov    esi, FindFile_Name ; * ; 15/10/2016
0000B57E BF[22820100]           <1>     mov     edi, DestinationFile_Name
0000B583 A5                     <1>     movsd
0000B584 A5                     <1>     movsd
0000B585 A5                     <1>     movsd
                                <1>     ;movsb
                                <1>
                                <1> csftdf_check_disk_free_size_0:
0000B586 A1[CE810100]           <1>     mov     eax, [SourceFile_DirEntry+DirEntry_FileSize]
                                <1>
                                <1> csftdf_check_disk_free_size_1:
                                <1>     ;sub    ebx, ebx
                                <1>     ;mov    esi, Logical_DOSDisks
                                <1>     ;mov    bh,  [DestinationFile_Drv]
                                <1>     ;add    esi, ebx
                                <1>
0000B58B 8B35[B8820100]         <1>     mov     esi, [csftdf_df_drv_dt] ; 23/03/2016
                                <1>
                                <1>     ;movzx  ecx, word [esi+LD_BPB+BytesPerSec] ; 17, LD_BPB + 0Bh
                                <1>     ; 19/12/2025
0000B591 B900020000             <1>     mov     ecx, 512
0000B596 01C8                   <1>     add     eax, ecx
0000B598 48                     <1>     dec     eax  ; file size (additional bytes) + 511 (round up)
                                <1> csftdf_check_disk_free_size_3: ; 16/03/2016
0000B599 29D2                   <1>     sub     edx, edx
0000B59B F7F1                   <1>     div     ecx ; bytes per sector
                                <1>
                                <1> csftdf_check_disk_free_size:
0000B59D 3B4674                 <1>     cmp     eax, [esi+LD_FreeSectors]
                                <1>         ;jb csftdf_check_disk_free_size_ok
                                <1>     ; 25/08/2024
                                <1>     ; 29/07/2022
                                <1>     ;jb     short csftdf_check_dfs_ok
0000B5A0 7705                   <1>     ja      short csftdf_df_insufficient_disk_space
                                <1>
                                <1>     ; 25/08/2024
                                <1>     ;cmp    byte [esi+LD_FATType], 0 ; FS needs FDT sector also.
                                <1>     ;;ja    csftdf_check_disk_free_size_ok
                                <1>     ;; 29/07/2022
                                <1>     ;jna    short csftdf_df_insufficient_disk_space
                                <1>
                                <1> csftdf_check_dfs_ok:
0000B5A2 E985000000             <1>     jmp     csftdf_check_disk_free_size_ok
                                <1>
                                <1> csftdf_df_insufficient_disk_space:
                                <1>     ;mov    eax, 27h ; insufficient disk space
                                <1>     ; 29/07/2022
0000B5A7 29C0                   <1>     sub     eax, eax
0000B5A9 B027                   <1>     mov     al, 27h
                                <1>     ;jmp    short csftdf_df_error_stc_retn
                                <1>     ; 25/08/2024
0000B5AB F9                     <1>     stc
0000B5AC C3                     <1>     retn
                                <1>
                                <1> csftdf_df_save_first_cluster:
                                <1>     ; ESI = FindFile_DirEntry (for the old destination file)
                                <1>     ; EAX = Old destination file size
                                <1>     ; 24/03/2016
                                <1>     ; EDI = Directory entry address (within Dir Buffer boundaries)
0000B5AD 81EF00000800           <1>     sub     edi, Directory_Buffer  ; (<65536)
                                <1>     ;shr    di, 5 ; Convert entry offset to entry index/number
                                <1>     ; 29/07/2022
0000B5B3 C1EF05                 <1>     shr     edi, 5
0000B5B6 66893D[5A820100]       <1>     mov     [DestinationFile_DirEntryNumber], di ; (<2048)
                                <1>
                                <1> csftdf_df_check_sf_df_fcluster:
0000B5BD 668B5614               <1>     mov     dx, [esi+DirEntry_FstClusHI]
0000B5C1 C1E210                 <1>     shl     edx, 16
0000B5C4 668B561A               <1>     mov     dx, [esi+DirEntry_FstClusLO]
0000B5C8 8915[9C820100]         <1>     mov     [csftdf_df_cluster], edx
                                <1> csftdf_df_check_sf_df_fcluster_1:
0000B5CE 668B15[C6810100]       <1>     mov     dx, [SourceFile_DirEntry+DirEntry_FstClusHI]
0000B5D5 C1E210                 <1>     shl     edx, 16
0000B5D8 668B15[CC810100]       <1>     mov     dx, [SourceFile_DirEntry+DirEntry_FstClusLO]
0000B5DF 3B15[9C820100]         <1>     cmp     edx, [csftdf_df_cluster]
0000B5E5 7512                   <1>     jne     short csftdf_df_check_sf_df_fcluster_ok
                                <1> csftdf_df_check_sf_df_drv:
0000B5E7 8A15[60810100]         <1>     mov     dl, [SourceFile_Drv]
0000B5ED 3A15[E0810100]         <1>     cmp     dl, [DestinationFile_Drv]
0000B5F3 7504                   <1>     jne     short csftdf_df_check_sf_df_fcluster_ok
                                <1>
                                <1>     ; source and destination files are same !
                                <1>     ; (they have same first cluster value on same logical disk)
                                <1>
0000B5F5 31C0                   <1>     xor     eax, eax ; mov eax, 0 -> Bad command or file name !
0000B5F7 F9                     <1>     stc
0000B5F8 C3                     <1>     retn
                                <1>
                                <1> csftdf_df_check_sf_df_fcluster_ok:
                                <1> csftdf_df_move_findfile_struct:
                                <1>     ; mov   esi, FindFile_DirEntry
0000B5F9 BF[32820100]           <1>     mov     edi, DestinationFile_DirEntry
                                <1>     ;mov    ecx, 8
0000B5FE 31C9                   <1>     xor     ecx, ecx
0000B600 B108                   <1>     mov     cl, 8
0000B602 F3A5                   <1>     rep     movsd
                                <1>
                                <1> ;csftdf_check_disk_free_size_2:
0000B604 89C2                   <1>     mov     edx, eax ; Old destination file size
                                <1>
                                <1>     ;mov    eax, [SourceFile_DirEntry+DirEntry_FileSize]
0000B606 A1[8C820100]           <1>     mov     eax, [csftdf_filesize] ; 23/03/2016
                                <1>
                                <1>     ;;sub   ecx, ecx ; 0
                                <1>     ;mov    esi, Logical_DOSDisks
                                <1>     ;mov    ch, [DestinationFile_Drv]
                                <1>     ;add    esi, ecx
                                <1>     ;
                                <1>     ;mov    [csftdf_df_drv_dt], esi
                                <1>
0000B60B 8B35[B8820100]         <1>     mov     esi, [csftdf_df_drv_dt] ; 23/03/2016
                                <1>
                                <1>     ;mov    cx, [esi+LD_BPB+BytesPerSec] ; 17, LD_BPB + 0Bh
                                <1>     ; 19/12/2025
```

```
0000B611 66B90002            <1>     mov     cx, 512
0000B615 01CA                <1>     add     edx, ecx ; + 512
0000B617 01C8                <1>     add     eax, ecx ; + 512
0000B619 4A                  <1>     dec     edx ; old file size + 511 (round up)
0000B61A 48                  <1>     dec     eax ; new file size + 511 (round up)
0000B61B F7D9                <1>     neg     ecx ; -512 ; 0FFFFFE00h
0000B61D 21CA                <1>     and     edx, ecx ; = old sector count * 512
0000B61F 21C8                <1>     and     eax, ecx ; = new sector count * 512
                             <1>
0000B621 29D0                <1>     sub     eax, edx ; new file size - old file size (on disk)
0000B623 7607                <1>     jna     short csftdf_check_disk_free_size_ok
                             <1>
0000B625 F7D9                <1>     neg     ecx ; 512 (bytes per sector) ; 200h
                             <1>     ; check free space for additional sectors
                             <1>     ; eax = number of additional sectors * bytes per sector
                             <1>     ; esi = Logical DOS drive number (of destination disk)
0000B627 E96DFFFFFF          <1>         jmp csftdf_check_disk_free_size_3
                             <1>
                             <1> csftdf_check_disk_free_size_ok:
                             <1>     ; 18/03/2016
                             <1> csftdf_df_check_copy_cmd_phase:
0000B62C A0[88820100]        <1>     mov     al, [copy_cmd_phase]
0000B631 3C01                <1>     cmp     al, 1
0000B633 7514                <1>     jne     short csftdf2_check_cdrv
                             <1>
0000B635 31C0                <1>     xor     eax, eax
0000B637 A2[88820100]        <1>     mov     [copy_cmd_phase], al ; 0
                             <1>
0000B63C 8A15[8A820100]      <1>     mov     dl, [DestinationFileFound]
0000B642 8A35[BD810100]      <1>     mov     dh, [SourceFile_DirEntry+11] ; Attributes
                             <1>
                             <1> csftdf_return:
0000B648 C3                  <1>     retn
                             <1>
                             <1> ; Phase 2
                             <1> csftdf_ph2:
                             <1>     ; 29/07/2022
                             <1> csftdf2_check_cdrv:
                             <1>     ; 18/03/2016
                             <1>     ; Here, destination drive and directory are ready !
                             <1>     ; (checking/restoring is not needed)
                             <1>     ; (Since at the end of the phase 1)
                             <1>
                             <1> ;   mov     dl, [DestinationFile_Drv]
                             <1> ;   cmp     dl, [Current_Drv]
                             <1> ;   je      short csftdf2_df_check_directory
                             <1> ;
                             <1> ;   call    change_current_drive
                             <1> ;   jc      short csftdf2_read_error
                             <1> ;
                             <1> ;csftdf2_df_check_directory:
                             <1> ;   mov     esi, DestinationFile_Directory
                             <1> ;   cmp     byte [esi], 20h
                             <1> ;   jna     short csftdf2_df_check_found_or_not
                             <1> ;
                             <1> ;csftdf2_df_change_directory:
                             <1> ;   inc     byte [Restore_CDIR]
                             <1> ;   xor     ah, ah ; CD_COMMAND sign -> 0
                             <1> ;   call    change_current_directory
                             <1> ;   jc      short csftdf2_stc_return
                             <1> ;
                             <1> ;;csftdf2_df_change_prompt_dir_string:
                             <1> ;;  call    change_prompt_dir_string
                             <1>
                             <1> csftdf2_df_check_found_or_not:
                             <1>     ; 21/03/2016
0000B649 803D[8A820100]00    <1>     cmp     byte [DestinationFileFound], 0
0000B650 7742                <1>     ja      short csftdf2_set_sf_percentage
                             <1>
                             <1> csftdf2_create_file:
0000B652 BE[22820100]        <1>     mov     esi, DestinationFile_Name
0000B657 A1[8C820100]        <1>     mov     eax, [csftdf_filesize]
0000B65C 30C9                <1>     xor     cl, cl ; 0
                             <1>
0000B65E 31DB                <1>     xor     ebx, ebx ; 0
0000B660 4B                  <1>     dec     ebx ; 0FFFFFFFFh
                             <1>
                             <1>     ; INPUT ->
                             <1>     ;       EAX -> File Size
                             <1>     ;       ESI = ASCIIZ File name
                             <1>     ;        CL = File attributes
                             <1>     ;       EBX = FFFFFFFFh -> empty file sign for FAT fs
                             <1>     ;       EBX <> FFFFFFFFh -> use file size for FAT fs
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;       EAX = New file's first cluster
                             <1>     ;               (0 for empty file) ; 29/08/2024
                             <1>     ;       ESI = Logical Dos Drv Descr. Table Addr.
                             <1>     ;       ;EBX = CreateFile_Size address
                             <1>     ;       ;ECX = Sectors per cluster (<256)
                             <1>     ;       ;EDX = Directory Entry Index/Number (<65536)
                             <1>     ;       ; 29/08/2024
                             <1>     ;       EBX = File Size (0 for a new, empty file)
                             <1>     ;       ECX = Directory Entry Index/Number (<2048)
                             <1>     ;            (in directory cluster, not in directory)
                             <1>     ;       EDX = Directory Cluster Number (of the file)
                             <1>     ;
                             <1>     ;       cf = 1 -> error code in AL (EAX)
                             <1>
0000B661 E80D060000          <1>     call    create_file
                             <1>     ;pop   esi
                             <1>     ;jc    csftdf2_rw_error
                             <1>     ; 29/07/2022
0000B666 7305                <1>     jnc     short csftdf2_create_file_OK
0000B668 E978050000          <1>     jmp     csftdf2_rw_error
                             <1>
                             <1> csftdf2_create_file_OK:
                             <1>
0000B66D A3[9C820100]        <1>     mov     [csftdf_df_cluster], eax
                             <1>     ; 27/08/2024
                             <1>     ; eax = 0
                             <1>
                             <1>     ; 24/03/2016
                             <1>     ;mov   [DestinationFile_DirEntryNumber], dx
                             <1>     ; 29/08/2024
                             <1>     ; (these 3 parameters are needed for
                             <1>     ;  loading same dir entry for first cluster update)
0000B672 8915[56820100]      <1>     mov     [DestinationFile_DirCluster], edx
0000B678 66890D[5A820100]    <1>     mov     [DestinationFile_DirEntryNumber], cx
                             <1>
                             <1>     ; 30/08/2024
                             <1>     ; edx -> ecx
                             <1>     ; 21/03/2016
0000B67F BE00000800          <1>     mov     esi, Directory_Buffer
```

```
                                <1>     ;shl    edx, 5 ; 32 * index number
0000B684 C1E105                 <1>     shl     ecx, 5
                                <1>     ;add    esi, edx
0000B687 01CE                   <1>     add     esi, ecx
0000B689 BF[32820100]           <1>     mov     edi, DestinationFile_DirEntry
                                <1>     ;mov    cl, 8 ; 32 bytes
                                <1>     ; 29/08/2024
0000B68E 29C9                   <1>     sub     ecx, ecx
0000B690 B108                   <1>     mov     cl,8
0000B692 F3A5                   <1>     rep     movsd
                                <1>
                                <1>     ;mov    cl, [esi] ; L.D.D.D.T.
                                <1>     ;mov    [DestinationFile_Drv], cl
                                <1>
                                <1> csftdf2_set_sf_percentage:
                                <1>     ; 17/03/2016
0000B694 31C0                   <1>     xor     eax, eax
0000B696 A2[B0820100]           <1>     mov     [csftdf_percentage], al ; 0, reset
                                <1>
0000B69B A3[A8820100]           <1>     mov     [csftdf_sf_rbytes], eax ; 0, reset
0000B6A0 A3[AC820100]           <1>     mov     [csftdf_df_wbytes], eax ; 0, reset
                                <1>
0000B6A5 8A25[60810100]         <1>     mov     ah, [SourceFile_Drv]
0000B6AB BE00010900             <1>     mov     esi, Logical_DOSDisks
0000B6B0 01C6                   <1>     add     esi, eax
                                <1>
0000B6B2 8935[B4820100]         <1>     mov     [csftdf_sf_drv_dt], esi ; 23/03/2016
                                <1>
0000B6B8 668B15[C6810100]       <1>     mov     dx, [SourceFile_DirEntry+DirEntry_FstClusHI]
0000B6BF C1E210                 <1>     shl     edx, 16
0000B6C2 668B15[CC810100]       <1>     mov     dx, [SourceFile_DirEntry+DirEntry_FstClusLO]
0000B6C9 8915[98820100]         <1>     mov     [csftdf_sf_cluster], edx
                                <1>
                                <1>     ; 16/03/2016
                                <1>     ; Note: Singlix FS boot sector parameters (for cluster
                                <1>     ;       related calculations) has same offset
                                <1>     ;       values from LD_BPB as in FAT file system.
                                <1>     ;       [esi+LD_BPB+SecPerClust] is 1 for Singlix FS.
                                <1>     ;
0000B6CF 0FB64E13               <1>     movzx   ecx, byte [esi+LD_BPB+SecPerClust]
0000B6D3 880D[DE810100]         <1>     mov     [SourceFile_SecPerClust], cl
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     ; 17/03/2016
                                <1>     cmp     [esi+LD_FATType], ch ; 0
                                <1>     ja      short csftdf2_set_sf_percent_rsize1
                                <1>
                                <1>     mov     eax, 65536 ; read/write buffer size for Singlix FS
                                <1>     jmp     short csftdf2_set_sf_percent_rsize2
                                <1> %endif
                                <1>
                                <1> csftdf2_set_sf_percent_rsize1:
                                <1>     ;mov    ax, [esi+LD_BPB+BytesPerSec]
                                <1>     ; 19/12/2025
0000B6D9 66B80002               <1>     mov     ax, 512
0000B6DD F7E1                   <1>     mul     ecx
                                <1>     ;sub    edx, edx
                                <1> csftdf2_set_sf_percent_rsize2:
0000B6DF A3[A0820100]           <1>     mov     [csftdf_r_size], eax
                                <1>
                                <1> csftdf2_set_df_percentage:
                                <1>     ;sub    eax, eax
                                <1>     ;mov    ah, [DestinationFile_Drv]
                                <1>     ;mov    edi, Logical_DOSDisks
                                <1>     ;add    edi, eax
                                <1>     ;mov    [csftdf_df_drv_dt], edi ; 17/03/2016
                                <1>
0000B6E4 8B3D[B8820100]         <1>     mov     edi, [csftdf_df_drv_dt] ; 23/03/2016
                                <1>
                                <1>     ; 16/03/2016
                                <1>     ; Note: Singlix FS boot sector parameters (for cluster
                                <1>     ;       related calculations) has same offset
                                <1>     ;       values from LD_BPB as in FAT file system.
                                <1>     ;       [edi+LD_BPB+SecPerClust] is 1 for Singlix FS.
                                <1>     ;
                                <1>     ;movzx  ecx, byte [edi+LD_BPB+SecPerClust]
0000B6EA 8A4F13                 <1>     mov     cl, [edi+LD_BPB+SecPerClust]
0000B6ED 880D[5E820100]         <1>     mov     [DestinationFile_SecPerClust], cl
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     ; 17/03/2016
                                <1>     cmp     [edi+LD_FATType], ch ; 0
                                <1>     ja      short csftdf2_set_df_percent_wsize1
                                <1>
                                <1>     mov     eax, 65536 ; read/write buffer size for Singlix FS
                                <1>     jmp     short csftdf2_set_df_percent_wsize2
                                <1> %endif
                                <1>
                                <1> csftdf2_set_df_percent_wsize1:
                                <1>     ;movzx  eax, word [edi+LD_BPB+BytesPerSec]
                                <1>     ; 19/12/2025
0000B6F3 B800020000             <1>     mov     eax, 512
0000B6F8 F7E1                   <1>     mul     ecx
                                <1>     ;sub    edx, edx
                                <1> csftdf2_set_df_percent_wsize2:
0000B6FA A3[A4820100]           <1>     mov     [csftdf_w_size], eax
                                <1>
0000B6FF A1[8C820100]           <1>     mov     eax, [csftdf_filesize]
                                <1>
0000B704 3D00000100             <1>     cmp     eax, 65536 ; 64KB     ; small file
0000B709 721F                   <1>     jb      short csftdf2_load_file ; do not display percentage
                                <1>
                                <1> csftdf2_reset_wf_percent_ptr_chk_64k:
0000B70B B201                   <1>     mov     dl, 1 ; 25/03/2016
                                <1>
0000B70D 3D00000400             <1>     cmp     eax, 65536*4 ; 256KB
0000B712 7310                   <1>     jnb     short csftdf2_enable_percentage_display ; big file
                                <1>
                                <1>     ; 64-128KB file size for floppy disks
0000B714 3815[60810100]         <1>     cmp     byte [SourceFile_Drv], dl ; 1 ; read from floppy disk ?
0000B71A 7608                   <1>     jna     short csftdf2_enable_percentage_display
                                <1>
0000B71C 3815[E0810100]         <1>     cmp     byte [DestinationFile_Drv], dl ; 1 ; write to floppy disk ?
0000B722 7706                   <1>     ja      short csftdf2_load_file
                                <1>
                                <1> csftdf2_enable_percentage_display:
0000B724 8815[B0820100]         <1>     mov     [csftdf_percentage], dl ; 1
                                <1>
                                <1> csftdf2_load_file:
                                <1>     ; 13/05/2016
                                <1>     ; 19/03/2016
                                <1>     ; 18/03/2016
```

```
                                <1>     ; 17/03/2016
0000B72A B40F                   <1>     mov     ah, 0Fh
0000B72C E8E95FFFFF             <1>     call    _int10h
                                <1>     ; 13/05/2016
0000B731 883D[B1820100]         <1>     mov     [csftdf_videopage], bh ; active video page
0000B737 B403                   <1>     mov     ah, 03h
0000B739 E8DC5FFFFF             <1>     call    _int10h
0000B73E 668915[B2820100]       <1>     mov     [csftdf_cursorpos], dx
                                <1>
0000B745 29C0                   <1>     sub     eax, eax
0000B747 A2[89820100]           <1>     mov     [csftdf_rw_err], al ; 0
                                <1>
                                <1> ; ///
                                <1> csftdf_sf_amb: ; 15/03/2016
0000B74C 8B0D[8C820100]         <1>     mov     ecx, [csftdf_filesize]; 23/03/2016
                                <1>
                                <1>     ; TRDOS 386 (TRDOS v2.0)
                                <1>     ; Allocate contiguous memory block for loading the file
                                <1>
                                <1>     ;mov   ecx, [SourceFile_DirEntry+DirEntry_FileSize]
                                <1>
                                <1>     ;sub   eax, eax ; First free memory aperture
                                <1>
                                <1>     ; eax = 0 (Allocate memory from the beginning)
                                <1>     ; ecx = File (Allocation) size in bytes
                                <1>
                                <1> ; 31/08/2024 - temporary
                                <1> ;%if 0
0000B752 E8EEA5FFFF             <1>     call    allocate_memory_block
0000B757 7304                   <1>     jnc     short loc_check_sf_save_loading_parms
                                <1> ;%endif
                                <1>
                                <1> csftdf_use_cluster_buff: ; 27/08/2024
                                <1>
0000B759 29C0                   <1>     sub     eax, eax
0000B75B 29C9                   <1>     sub     ecx, ecx
                                <1>
                                <1> loc_check_sf_save_loading_parms:
0000B75D A3[90820100]           <1>     mov     [csftdf_sf_mem_addr], eax ; loading address
0000B762 890D[94820100]         <1>     mov     [csftdf_sf_mem_bsize], ecx ; block size
                                <1> ; ///
                                <1>     ; 19/03/2016
0000B768 8B35[B4820100]         <1>     mov     esi, [csftdf_sf_drv_dt] ; logical dos drv desc. tbl.
                                <1>
                                <1>     ; 17/03/2016
0000B76E 09C0                   <1>     or      eax, eax ; contiguous free memory block address
                                <1>     ;jz     csftdf2_read_sf_cluster
                                <1>     ; 29/07/2022
0000B770 7505                   <1>     jnz     short csftdf2_1
0000B772 E949010000             <1>     jmp     csftdf2_read_sf_cluster
                                <1>
                                <1> csftdf2_1:
                                <1>     ; 18/03/2016
0000B777 8B1D[90820100]         <1>     mov     ebx, [csftdf_sf_mem_addr] ; memory block address
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>         ;jna       csftdf2_load_fs_file
                                <1>     ; 29/07/2022
                                <1>     ja      short csftdf2_load_fat_file
                                <1>     jmp     csftdf2_load_fs_file
                                <1> %endif
                                <1>
                                <1> csftdf2_load_fat_file:
0000B77D 53                     <1>     push    ebx ; *
                                <1>
                                <1> csftdf2_load_fat_file_next:
0000B77E BE[E6340100]           <1>     mov     esi, msg_reading
0000B783 E832B6FFFF             <1>     call    print_msg
                                <1>
0000B788 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
0000B78F 7605                   <1>     jna     short csftdf2_load_fat_file_1
                                <1>
0000B791 E87E000000             <1>     call    csftdf2_print_percentage ; 19/03/2016
                                <1>
                                <1> csftdf2_load_fat_file_1:
0000B796 8B35[B4820100]         <1>     mov     esi, [csftdf_sf_drv_dt]
0000B79C 5B                     <1>     pop     ebx ; *
                                <1>
                                <1> csftdf2_load_fat_file_2:
0000B79D E8AF000000             <1>     call    csftdf2_read_fat_file_sectors ; 19/03/2016
                                <1>     ;jc     csftdf2_rw_error ; eocc! or disk error!
                                <1>     ; 29/07/2022
0000B7A2 7305                   <1>     jnc     short csftdf2_load_fat_file_3
0000B7A4 E93C040000             <1>     jmp     csftdf2_rw_error
                                <1> csftdf2_load_fat_file_3:
0000B7A9 09D2                   <1>     or      edx, edx ; edx > 0 -> EOF
0000B7AB 7520                   <1>     jnz     short csftdf2_load_fat_file_ok
                                <1>
0000B7AD 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
0000B7B4 76E7                   <1>     jna     short csftdf2_load_fat_file_2
                                <1>
0000B7B6 53                     <1>     push    ebx ; *
                                <1>
                                <1>     ; Set cursor position
                                <1>     ; AH= 02h, BH= Page Number, DH= Row, DL= Column
0000B7B7 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000B7BD 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000B7C4 B402                   <1>     mov     ah, 2
0000B7C6 E84F5FFFFF             <1>     call    _int10h
0000B7CB EBB1                   <1>     jmp     short csftdf2_load_fat_file_next
                                <1>
                                <1> csftdf2_load_fat_file_ok:
0000B7CD 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
                                <1>     ;jna    csftdf2_save_file ; 25/03/2016
                                <1>     ; 29/07/2022
0000B7D4 7705                   <1>     ja      short csftdf2_2
0000B7D6 E941020000             <1>     jmp     csftdf2_save_file
                                <1> csftdf2_2:
                                <1>     ; "Reading... 100%"
0000B7DB BF[FE340100]           <1>     mov     edi, percentagestr
0000B7E0 B031                   <1>     mov     al, '1'
0000B7E2 AA                     <1>     stosb
0000B7E3 B030                   <1>     mov     al, '0'
0000B7E5 AA                     <1>     stosb
0000B7E6 AA                     <1>     stosb
                                <1>
0000B7E7 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000B7ED 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000B7F4 B402                   <1>     mov     ah, 2
0000B7F6 E81F5FFFFF             <1>     call    _int10h
                                <1>
0000B7FB BE[E6340100]           <1>     mov     esi, msg_reading
```

```
0000B800 E8B5B5FFFF          <1>     call    print_msg
                             <1>
0000B805 BE[FE340100]        <1>     mov     esi, percentagestr
0000B80A E8ABB5FFFF          <1>     call    print_msg
                             <1>
0000B80F E908020000          <1>     jmp     csftdf2_save_file ; 25/03/2016
                             <1>
                             <1> csftdf2_print_percentage:
                             <1>     ; 09/12/2017
                             <1>     ; 19/03/2016
                             <1>     ; 18/03/2016
0000B814 B020                <1>     mov     al, 20h
0000B816 BF[FE340100]        <1>     mov     edi, percentagestr
0000B81B AA                  <1>     stosb
0000B81C AA                  <1>     stosb
0000B81D A1[A8820100]        <1>     mov     eax, [csftdf_sf_rbytes]
                             <1>     ;mov    edx, 100
                             <1>     ; 29/07/2022
0000B822 29D2                <1>     sub     edx, edx
0000B824 B264                <1>     mov     dl, 100
0000B826 F7E2                <1>     mul     edx
0000B828 8B0D[8C820100]      <1>     mov     ecx, [csftdf_filesize]
0000B82E F7F1                <1>     div     ecx
0000B830 B10A                <1>     mov     cl, 10
0000B832 F6F1                <1>     div     cl
0000B834 80C430              <1>     add     ah, '0'
0000B837 8827                <1>     mov     [edi], ah
0000B839 20C0                <1>     and     al, al
0000B83B 740A                <1>     jz      short csftdf2_print_percent_1
0000B83D 4F                  <1>     dec     edi
                             <1>     ;cbw
0000B83E 28E4                <1>     sub     ah, ah ; 09/12/2017
0000B840 F6F1                <1>     div     cl
0000B842 80C430              <1>     add     ah, '0'
0000B845 8827                <1>     mov     [edi], ah
                             <1>     ;and    al, al
                             <1>     ;jz     short csftdf2_print_percent_1
                             <1>     ;dec    edi
                             <1>     ;mov    [edi], '1' ; 100%
                             <1>
                             <1> csftdf2_print_percent_1:
0000B847 BE[FE340100]        <1>     mov     esi, percentagestr
                             <1>     ;call   print_msg
                             <1>     ;retn
0000B84C E969B5FFFF          <1>     jmp     print_msg
                             <1>
                             <1> csftdf2_read_file_sectors:
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     ; 19/03/2016
                             <1>     cmp     byte [esi+LD_FATType], 0
                             <1>     ;jna    csftdf2_read_fs_file_sectors
                             <1>     ; 29/07/2022
                             <1>     ja      short csftdf2_read_fat_file_sectors
                             <1>     jmp     csftdf2_read_fs_file_sectors
                             <1> %endif
                             <1>
                             <1> csftdf2_read_fat_file_sectors:
                             <1>     ; 19/03/2016
                             <1>     ; 18/03/2016
                             <1>     ; return:
                             <1>     ;   CF = 0 & EDX > 0 -> END OF FILE
                             <1>     ;   CF = 0 & EDX = 0 -> not EOF
                             <1>     ;   CF = 1 -> read error (error code in AL)
                             <1>
                             <1> csftdf2_read_fat_file_secs_0:
0000B851 8B15[8C820100]      <1>     mov     edx, [csftdf_filesize]
0000B857 2B15[A8820100]      <1>     sub     edx, [csftdf_sf_rbytes]
0000B85D 3B15[A0820100]      <1>     cmp     edx, [csftdf_r_size]
0000B863 7306                <1>     jnb     short csftdf2_read_fat_file_secs_1
0000B865 8915[A0820100]      <1>     mov     [csftdf_r_size], edx
                             <1>
                             <1> csftdf2_read_fat_file_secs_1:
0000B86B A1[A0820100]        <1>     mov     eax, [csftdf_r_size]
0000B870 29D2                <1>     sub     edx, edx
                             <1>     ;movzx  ecx, word [esi+LD_BPB+BytesPerSec]
                             <1>     ; 19/12/2025
0000B872 B900020000          <1>     mov     ecx, 512
0000B877 01C8                <1>     add     eax, ecx
0000B879 48                  <1>     dec     eax
0000B87A F7F1                <1>     div     ecx
0000B87C 89C1                <1>     mov     ecx, eax ; sector count
0000B87E A1[98820100]        <1>     mov     eax, [csftdf_sf_cluster]
                             <1>
                             <1>     ; EBX = memory block address (current)
                             <1>
0000B883 E8F7080000          <1>     call    read_fat_file_sectors
0000B888 7235                <1>     jc      short csftdf2_read_fat_file_secs_3
                             <1>
                             <1>     ; EBX = next memory address
                             <1>
0000B88A A1[A8820100]        <1>     mov     eax, [csftdf_sf_rbytes]
0000B88F 0305[A0820100]      <1>     add     eax, [csftdf_r_size]
0000B895 8B15[8C820100]      <1>     mov     edx, [csftdf_filesize]
0000B89B 39D0                <1>     cmp     eax, edx
0000B89D 7320                <1>     jnb     short csftdf2_read_fat_file_secs_3 ; edx > 0
0000B89F A3[A8820100]        <1>     mov     [csftdf_sf_rbytes], eax
                             <1>
0000B8A4 53                  <1>     push    ebx ; *
                             <1>     ; get next cluster (csftdf_r_size! bytes)
0000B8A5 A1[98820100]        <1>     mov     eax, [csftdf_sf_cluster]
0000B8AA E8DA060000          <1>     call    get_next_cluster
0000B8AF 5B                  <1>     pop     ebx ; *
0000B8B0 7306                <1>     jnc     short csftdf2_read_fat_file_secs_2
                             <1>
                             <1>     ; 15/10/2016
                             <1>     ; Disk read error instead of drv not ready err
0000B8B2 B811000000          <1>     mov     eax, 17 ; Read error !
0000B8B7 C3                  <1>     retn
                             <1>
                             <1> csftdf2_read_fat_file_secs_2:
0000B8B8 29D2                <1>     sub     edx, edx ; 0
0000B8BA A3[98820100]        <1>     mov     [csftdf_sf_cluster], eax ; next cluster
                             <1>
                             <1> csftdf2_read_fat_file_secs_3:
0000B8BF C3                  <1>     retn
                             <1>
                             <1> csftdf2_read_sf_cluster:
                             <1>     ; 19/03/2016
0000B8C0 BB00000700          <1>     mov     ebx, Cluster_Buffer ; buffer address (64KB)
                             <1>
0000B8C5 803D[B0820100]00    <1>     cmp     byte [csftdf_percentage], 0
0000B8CC 760D                <1>     jna     short csftdf2_read_sf_clust_2
```

```
                                <1>
0000B8CE 53                     <1>     push    ebx ; *
                                <1>
                                <1> csftdf2_read_sf_clust_next:
0000B8CF E840FFFFFF             <1>     call    csftdf2_print_percentage
                                <1>
                                <1> csftdf2_read_sf_clust_0:
0000B8D4 8B35[B4820100]         <1>     mov     esi, [csftdf_sf_drv_dt]
                                <1> csftdf2_read_sf_clust_1:
0000B8DA 5B                     <1>     pop     ebx ; *
                                <1>
                                <1> csftdf2_read_sf_clust_2:
0000B8DB 89DA                   <1>     mov     edx, ebx
0000B8DD 0315[A0820100]         <1>     add     edx, [csftdf_r_size]
0000B8E3 81FA00000800           <1>     cmp     edx, Cluster_Buffer + 65536
0000B8E9 772B                   <1>     ja      short csftdf2_write_df_cluster
                                <1>
0000B8EB E861FFFFFF             <1>     call    csftdf2_read_file_sectors ; 19/03/2016
                                <1>     ;jc     csftdf2_save_fat_file_err2 ; eocc! or disk error!
                                <1>     ; 29/07/2022
0000B8F0 723C                   <1>     jc      short csftdf2_3
                                <1>
0000B8F2 09D2                   <1>     or      edx, edx ; edx > 0 -> EOF
0000B8F4 7520                   <1>     jnz     short csftdf2_write_df_cluster
                                <1>
0000B8F6 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
0000B8FD 76DC                   <1>     jna     short csftdf2_read_sf_clust_2
                                <1>
0000B8FF 53                     <1>     push    ebx ; *
                                <1>
                                <1>     ; Set cursor position
                                <1>     ; AH= 02h, BH= Page Number, DH= Row, DL= Column
0000B900 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000B906 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000B90D B402                   <1>     mov     ah, 2
0000B90F E8065EFFFF             <1>     call    _int10h
0000B914 EBB9                   <1>     jmp     short csftdf2_read_sf_clust_next
                                <1>
                                <1> csftdf2_write_df_cluster:
                                <1>
                                <1> ; 31/08/2024
                                <1> %if 1
                                <1>     ; 27/08/2024
                                <1>     ; 19/03/2016
                                <1>     ;mov    esi, [csftdf_df_drv_dt] ; (!)
                                <1>     ;mov    ebx, Cluster_Buffer ; buffer address (64KB)
                                <1>     ;;;
                                <1>     ; 27/08/2024
0000B916 E8FF020000             <1>     call    csftdf2_update_df_fclust
                                <1>     ; 31/08/2024
0000B91B 7305                   <1>     jnc     short csftdf2_update_df_fclust_ok
0000B91D E9C3020000             <1>     jmp     csftdf2_rw_error
                                <1>
                                <1> csftdf2_update_df_fclust_ok:
                                <1> %endif
                                <1>     ; esi = [csftdf_df_drv_dt]
                                <1>
                                <1>     ;mov    esi, [csftdf_df_drv_dt]
0000B922 BB00000700             <1>     mov     ebx, Cluster_Buffer ; buffer address (64KB)
                                <1>
                                <1> csftdf2_write_df_clust_next:
0000B927 E85B000000             <1>     call    csftdf2_write_file_sectors ; 19/03/2016
                                <1>     ;jc     csftdf2_save_fat_file_err2 ; eocc! or disk error!
                                <1>     ; 29/07/2022
0000B92C 7305                   <1>     jnc     short csftdf2_4
                                <1> csftdf2_3:
0000B92E E939020000             <1>     jmp     csftdf2_save_fat_file_err2
                                <1> csftdf2_4:
0000B933 09D2                   <1>     or      edx, edx ; edx > 0 -> EOF
0000B935 7513                   <1>     jnz     short csftdf2_rw_f_clust_ok
                                <1>
0000B937 81FB00000800           <1>     cmp     ebx, Cluster_Buffer + 65536
0000B93D 72E8                   <1>     jb      short csftdf2_write_df_clust_next
                                <1>
                                <1>     ; 31/08/2024
0000B93F 8B35[B4820100]         <1>     mov     esi, [csftdf_sf_drv_dt]
                                <1>
0000B945 E976FFFFFF             <1>     jmp     csftdf2_read_sf_cluster ; 31/08/2024
                                <1>
                                <1> csftdf2_rw_f_clust_ok:
0000B94A 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
                                <1>     ;jna    csftdf2_save_fat_file_4 ; 25/03/2016
                                <1>     ; 29/07/2022
0000B951 762A                   <1>     jna     short csftdf2_5
                                <1>
                                <1>     ; "100%"
0000B953 BF[FE340100]           <1>     mov     edi, percentagestr
0000B958 B031                   <1>     mov     al, '1'
0000B95A AA                     <1>     stosb
0000B95B B030                   <1>     mov     al, '0'
0000B95D AA                     <1>     stosb
0000B95E AA                     <1>     stosb
                                <1>
0000B95F 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000B965 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000B96C B402                   <1>     mov     ah, 2
0000B96E E8A75DFFFF             <1>     call    _int10h
                                <1>
0000B973 BE[FE340100]           <1>     mov     esi, percentagestr
0000B978 E83DB4FFFF             <1>     call    print_msg
                                <1> csftdf2_5:
0000B97D E974010000             <1>         jmp     csftdf2_save_fat_file_4
                                <1>
                                <1> csftdf2_load_fs_file:
                                <1>     ; temporary - 18/03/2016
0000B982 E95C020000             <1>         jmp     csftdf2_read_error
                                <1>
                                <1> csftdf2_write_file_sectors:
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     ; 31/08/2024
                                <1>     ; 30/08/2024
                                <1>     ; 19/03/2016
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ;jna    csftdf2_write_fs_file_sectors
                                <1>     ; 29/07/2022
                                <1>     ja      short csftdf2_write_fat_file_sectors
                                <1>     jmp     csftdf2_write_fs_file_sectors
                                <1> %endif
                                <1>
                                <1> csftdf2_write_fat_file_sectors:
                                <1>     ; 30/08/2024
                                <1>     ; 19/03/2016
```

```
                                <1>     ; 18/03/2016
                                <1>     ; return:
                                <1>     ;   CF = 0 & EDX > 0 -> END OF FILE
                                <1>     ;   CF = 0 & EDX = 0 -> not EOF
                                <1>     ;   CF = 1 -> write error (error code in AL)
                                <1>
                                <1> csftdf2_write_fat_file_secs_0:
0000B987 8B15[8C820100]         <1>     mov     edx, [csftdf_filesize]
0000B98D 2B15[AC820100]         <1>     sub     edx, [csftdf_df_wbytes]
0000B993 3B15[A4820100]         <1>     cmp     edx, [csftdf_w_size]
0000B999 7306                   <1>     jnb     short csftdf2_write_fat_file_secs_1
0000B99B 8915[A4820100]         <1>     mov     [csftdf_w_size], edx
                                <1>
                                <1> csftdf2_write_fat_file_secs_1:
                                <1>
                                <1> ; 31/08/2024
                                <1> %if 0
                                <1>     ;;;
                                <1>     ; 30/08/2024
                                <1>     mov     eax, [csftdf_df_cluster]
                                <1>     and     eax, eax
                                <1>     jnz     short csftdf2_write_fat_file_secs_@
                                <1>     push    ebx
                                <1>     call    add_new_cluster
                                <1>     pop     ebx
                                <1>     jc      short csftdf2_write_fat_file_secs_5
                                <1>     mov     [csftdf_df_cluster], eax
                                <1> csftdf2_write_fat_file_secs_@:
                                <1>     ;;;
                                <1> %endif
                                <1>
0000B9A1 A1[A4820100]           <1>     mov     eax, [csftdf_w_size]
0000B9A6 29D2                   <1>     sub     edx, edx
                                <1>     ;movzx  ecx, word [esi+LD_BPB+BytesPerSec]
                                <1>     ; 19/12/2025
0000B9A8 B900020000             <1>     mov     ecx, 512
0000B9AD 01C8                   <1>     add     eax, ecx
0000B9AF 48                     <1>     dec     eax
0000B9B0 F7F1                   <1>     div     ecx
0000B9B2 89C1                   <1>     mov     ecx, eax ; sector count
0000B9B4 A1[9C820100]           <1>     mov     eax, [csftdf_df_cluster]
                                <1>
                                <1>     ; EBX = memory block address (current)
                                <1>
0000B9B9 E8DF0E0000             <1>     call    write_fat_file_sectors
0000B9BE 7254                   <1>     jc      short csftdf2_write_fat_file_secs_4
                                <1>
                                <1>     ; EBX = next memory address
                                <1>
0000B9C0 A1[AC820100]           <1>     mov     eax, [csftdf_df_wbytes]
0000B9C5 0305[A4820100]         <1>     add     eax, [csftdf_w_size]
0000B9CB 8B15[8C820100]         <1>     mov     edx, [csftdf_filesize]
0000B9D1 39D0                   <1>     cmp     eax, edx
0000B9D3 733F                   <1>     jnb     short csftdf2_write_fat_file_secs_4
0000B9D5 A3[AC820100]           <1>     mov     [csftdf_df_wbytes], eax
                                <1>     ;
0000B9DA A3[4E820100]           <1>     mov     [DestinationFile_DirEntry+DirEntry_FileSize], eax
                                <1>
                                <1>
0000B9DF 53                     <1>     push    ebx ; *
                                <1>
                                <1>     ; 30/08/2024
0000B9E0 A1[9C820100]           <1>     mov     eax, [csftdf_df_cluster] ; last cluster
                                <1>
0000B9E5 803D[8A820100]01       <1>     cmp     byte [DestinationFileFound], 1
0000B9EC 720B                   <1>     jb      short csftdf2_write_fat_file_secs_2
                                <1>
                                <1>     ; get next cluster (csftdf_w_size! bytes)
                                <1>     ;mov    eax, [csftdf_df_cluster]
0000B9EE E896050000             <1>     call    get_next_cluster
0000B9F3 7317                   <1>     jnc     short csftdf2_write_fat_file_secs_3
                                <1>
0000B9F5 21C0                   <1>     and     eax, eax ; end of cluster chain!?
0000B9F7 751C                   <1>     jnz     short csftdf2_write_fat_file_secs_5 ; disk error !
                                <1>
                                <1> csftdf2_write_fat_file_secs_2:
                                <1>     ;mov    eax, [csftdf_df_cluster] ; last cluster
                                <1>     ; 30/08/2024
0000B9F9 E80E0E0000             <1>     call    add_new_cluster
0000B9FE 7215                   <1>     jc      short csftdf2_write_fat_file_secs_5
                                <1>
                                <1>     ; NOTE: Destination file size may be bigger than
                                <1>     ; source file size when the last reading fails after here.
                                <1>     ; (The last -empty- cluster of destination file must be
                                <1>     ; truncated and LMDT must be current date&time for partial
                                <1>     ; copy result!)
0000BA00 8B15[A4820100]         <1>     mov     edx, [csftdf_w_size] ; < = bytes per cluster
0000BA06 0115[4E820100]         <1>     add     [DestinationFile_DirEntry+DirEntry_FileSize], edx
                                <1>
                                <1> csftdf2_write_fat_file_secs_3:
                                <1>     ; 30/08/2024
                                <1>     ;pop    ebx ; *
0000BA0C 29D2                   <1>     sub     edx, edx ; 0
0000BA0E A3[9C820100]           <1>     mov     [csftdf_df_cluster], eax ; next cluster
                                <1>
                                <1>     ; 30/08/2024
0000BA13 5B                     <1>     pop     ebx ; *
                                <1>
                                <1> csftdf2_write_fat_file_secs_4:
0000BA14 C3                     <1>     retn
                                <1>
                                <1> csftdf2_write_fat_file_secs_5:
0000BA15 5B                     <1>     pop     ebx ; *
0000BA16 B812000000             <1>     mov     eax, 18 ; Write error !
                                <1>     ; 16/10/2016 (1Dh -> 18)
0000BA1B C3                     <1>     retn
                                <1>
                                <1> csftdf2_save_file:
                                <1>     ; 19/12/2025
                                <1>     ; 31/08/2024
                                <1>     ; 26/08/2024, 30/08/2024
                                <1>     ; 09/12/2017
                                <1>     ; 19/03/2016, 25/03/2016
                                <1>     ; 18/03/2016
0000BA1C 8B35[B8820100]         <1>     mov     esi, [csftdf_df_drv_dt] ; logical dos drv desc. tbl.
0000BA22 8B1D[90820100]         <1>     mov     ebx, [csftdf_sf_mem_addr] ; memory block address
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ;jna    csftdf2_save_fs_file
                                <1>     ; 29/07/2022
                                <1>     ja      short csftdf2_save_fat_file
                                <1>     jmp     csftdf2_save_fs_file
```

```
                                <1> %endif
                                <1>
                                <1> csftdf2_save_fat_file:
                                <1>
                                <1> ; 31/08/2024
                                <1> %if 1
                                <1>     ; 30/08/2024
0000BA28 53                     <1>     push    ebx ; *
                                <1>     ;;;
                                <1>     ; 27/08/2024
                                <1>     ;call   csftdf2_update_df_fclust
0000BA29 E8F2010000             <1>     call    csftdf2_update_df_fclust_@ ; 31/08/2024
0000BA2E 5B                     <1>     pop     ebx ; *
0000BA2F 7236                   <1>     jc      short csftdf2_save_fat_file_err
                                <1>     ; esi = [csftdf_df_drv_dt] ; (*)
                                <1>     ;;;
                                <1> %endif
                                <1>
0000BA31 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
0000BA38 7726                   <1>     ja      short csftdf2_save_fat_file_0 ; 30/08/2024
                                <1>
0000BA3A 53                     <1>     push    ebx ; **
                                <1>
                                <1>     ; Set cursor position
                                <1>     ; AH= 02h, BH= Page Number, DH= Row, DL= Column
0000BA3B 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000BA41 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000BA48 B402                   <1>     mov     ah, 2
0000BA4A E8CB5CFFFF             <1>     call    _int10h
                                <1>
0000BA4F BE[F2340100]           <1>     mov     esi, msg_writing
0000BA54 E861B3FFFF             <1>     call    print_msg
                                <1>
                                <1> csftdf2_save_fat_file_next:
0000BA59 5B                     <1>     pop     ebx ; **
                                <1>
                                <1>     ; 31/08/2024
0000BA5A 8B35[B8820100]         <1>     mov     esi, [csftdf_df_drv_dt]
                                <1>
                                <1> csftdf2_save_fat_file_0: ; 30/08/2024
                                <1>
                                <1> csftdf2_save_fat_file_1:
0000BA60 E822FFFFFF             <1>     call    csftdf2_write_file_sectors ; 19/03/2016
                                <1>     ;jc     csftdf2_rw_error ; eocc! or disk error!
                                <1>     ; 29/07/2022
0000BA65 7305                   <1>     jnc     short csftdf2_6
                                <1> csftdf2_save_fat_file_err: ; 27/08/2024
0000BA67 E979010000             <1>     jmp     csftdf2_rw_error
                                <1>
                                <1> csftdf2_6:
0000BA6C 09D2                   <1>     or      edx, edx ; edx > 0 -> EOF
0000BA6E 756C                   <1>         jnz     short csftdf2_save_fat_file_3 ; 25/03/2016
                                <1>
0000BA70 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
0000BA77 76E7                   <1>     jna     short csftdf2_save_fat_file_1
                                <1>
0000BA79 B020                   <1>     mov     al, 20h
0000BA7B BF[FE340100]           <1>     mov     edi, percentagestr
0000BA80 AA                     <1>     stosb
0000BA81 AA                     <1>     stosb
0000BA82 A1[AC820100]           <1>     mov     eax, [csftdf_df_wbytes]
                                <1>     ;mov    edx, 100
                                <1>     ; 29/07/2022
0000BA87 31D2                   <1>     xor     edx, edx
0000BA89 B264                   <1>     mov     dl, 100
0000BA8B F7E2                   <1>     mul     edx
0000BA8D 8B0D[8C820100]         <1>     mov     ecx, [csftdf_filesize]
0000BA93 F7F1                   <1>     div     ecx
0000BA95 B10A                   <1>     mov     cl, 10
0000BA97 F6F1                   <1>     div     cl
0000BA99 80C430                 <1>     add     ah, '0'
0000BA9C 8827                   <1>     mov     [edi], ah
0000BA9E 20C0                   <1>     and     al, al
0000BAA0 740A                   <1>     jz      short csftdf2_save_fat_file_2
0000BAA2 4F                     <1>     dec     edi
                                <1>     ;cbw
0000BAA3 30E4                   <1>     xor     ah, ah ; 09/12/2017
0000BAA5 F6F1                   <1>     div     cl
0000BAA7 80C430                 <1>     add     ah, '0'
0000BAAA 8827                   <1>     mov     [edi], ah
                                <1>     ;and    al, al
                                <1>     ;jz     short csftdf2_save_fat_file_2
                                <1>     ;dec    edi
                                <1>     ;mov    [edi], '1' ; 100%
                                <1>
                                <1> csftdf2_save_fat_file_2:
0000BAAC 53                     <1>     push    ebx ; *
                                <1>
0000BAAD E802000000             <1>     call    csftdf2_print_wr_percentage ; 25/03/2016
                                <1>
0000BAB2 EBA5                   <1>         jmp     csftdf2_save_fat_file_next
                                <1>
                                <1> csftdf2_print_wr_percentage:
                                <1>     ; Set cursor position
                                <1>     ; AH= 02h, BH= Page Number, DH= Row, DL= Column
0000BAB4 8A3D[B1820100]         <1>     mov     bh, [csftdf_videopage]
0000BABA 668B15[B2820100]       <1>     mov     dx, [csftdf_cursorpos]
0000BAC1 B402                   <1>     mov     ah, 2
0000BAC3 E8525CFFFF             <1>     call    _int10h
                                <1>
0000BAC8 BE[F2340100]           <1>     mov     esi, msg_writing
0000BACD E8E8B2FFFF             <1>     call    print_msg
                                <1>
0000BAD2 BE[FE340100]           <1>     mov     esi, percentagestr
                                <1>     ;call   print_msg
                                <1>     ;retn
0000BAD7 E9DEB2FFFF             <1>     jmp     print_msg
                                <1>
                                <1> csftdf2_save_fat_file_3:
0000BADC 803D[B0820100]00       <1>     cmp     byte [csftdf_percentage], 0
                                <1>         ;jna        csftdf2_save_fat_file_4 ; 25/03/2016
                                <1>     ; 29/07/2022
                                <1>     ;ja     short csftdf2_7
                                <1>     ;jmp    csftdf2_save_fat_file_4
                                <1>     ; 31/08/2024
0000BAE3 7611                   <1>     jna     short csftdf2_save_fat_file_4
                                <1>
                                <1> csftdf2_7:
                                <1>     ; "100%"
0000BAE5 BF[FE340100]           <1>     mov     edi, percentagestr
0000BAEA B031                   <1>     mov     al, '1'
0000BAEC AA                     <1>     stosb
0000BAED B030                   <1>     mov     al, '0'
0000BAEF AA                     <1>     stosb
```

```
0000BAF0 AA                  <1>     stosb
                             <1>
0000BAF1 E8BEFFFFFF          <1>     call    csftdf2_print_wr_percentage
                             <1>
                             <1> csftdf2_save_fat_file_4:
0000BAF6 803D[8A820100]00    <1>     cmp     byte [DestinationFileFound], 0
0000BAFD 7645                <1>     jna     short csftdf2_save_fat_file_6
                             <1>
0000BAFF 8B35[B8820100]      <1>     mov     esi, [csftdf_df_drv_dt] ; 31/03/2016
                             <1>
0000BB05 A1[9C820100]        <1>     mov     eax, [csftdf_df_cluster] ; last cluster
0000BB0A E87A040000          <1>     call    get_next_cluster
0000BB0F 7233                <1>     jc      short csftdf2_save_fat_file_6 ; eocc! or disk error!
                             <1>
0000BB11 A1[9C820100]        <1>     mov     eax, [csftdf_df_cluster] ; last cluster
                             <1>     ; 29/08/2024
0000BB16 31C9                <1>     xor     ecx, ecx
                             <1>     ;mov    [FAT_ClusterCounter], ecx ; 0 ; reset
0000BB18 49                  <1>     dec     ecx ; 0FFFFFFFFh
                             <1>     ;shr    ecx, 4 ; 28 bit ; 0FFFFFFFh
                             <1>     ; 29/08/2024
                             <1>     ; (update_cluster will save 28 bit cluster value)
                             <1>     ;mov    ecx, 0FFFFFFFh
0000BB19 E852070000          <1>     call    update_cluster
0000BB1E 7224                <1>     jc      short csftdf2_save_fat_file_6 ; really last cluster!?
                             <1>
0000BB20 A3[9C820100]        <1>     mov     [csftdf_df_cluster], eax ; next cluster
                             <1>
                             <1>     ; byte [FAT_BuffValidData] = 2
0000BB25 E8BA090000          <1>     call    save_fat_buffer
0000BB2A 730E                <1>     jnc     short csftdf2_save_fat_file_5
                             <1>
0000BB2C 8B15[8C820100]      <1>     mov     edx, [csftdf_filesize]
0000BB32 8915[4E820100]      <1>     mov     [DestinationFile_DirEntry+DirEntry_FileSize], edx
0000BB38 EB57                <1>     jmp     short csftdf2_save_fat_file_err3
                             <1>
                             <1> csftdf2_save_fat_file_5:
0000BB3A A1[9C820100]        <1>     mov     eax, [csftdf_df_cluster]
                             <1>
                             <1>     ; EAX = First cluster to be truncated/unlinked
                             <1>     ; ESI = Logical dos drive description table address
0000BB3F E8DA0B0000          <1>     call    truncate_cluster_chain
                             <1>
                             <1> csftdf2_save_fat_file_6:
                             <1>     ; 28/03/2016
0000BB44 BE[BD810100]        <1>     mov     esi, SourceFile_DirEntry+DirEntry_Attr ; +11 to + 18
0000BB49 BF[3D820100]        <1>     mov     edi, DestinationFile_DirEntry+DirEntry_Attr ; +11 to + 18
0000BB4E A4                  <1>     movsb ; +11
0000BB4F A5                  <1>     movsd ; +12 .. +15
0000BB50 66A5                <1>     movsw ; +16 .. +17
                             <1>             ; + 18
0000BB52 83C604              <1>     add     esi, 4
0000BB55 83C704              <1>     add     edi, 4
0000BB58 A5                  <1>     movsd   ; DirEntry_WrtTime ; +22 .. +25
                             <1>
0000BB59 8B15[8C820100]      <1>     mov     edx, [csftdf_filesize]
0000BB5F 8915[4E820100]      <1>     mov     [DestinationFile_DirEntry+DirEntry_FileSize], edx
                             <1>
0000BB65 E892F1FFFF          <1>     call    convert_current_date_time
                             <1>     ; DX = Date in dos dir entry format
                             <1>     ; AX = Time in dos dir entry format
0000BB6A EB4C                <1>     jmp     short csftdf2_save_fat_file_7
                             <1>
                             <1>     ; 31/08/2024
                             <1> ;csftdf2_save_fat_file_err1:
                             <1>     ;pop    ebx ; *
                             <1>
                             <1> csftdf2_save_fat_file_err2:
0000BB6C A1[AC820100]        <1>     mov     eax, [csftdf_df_wbytes]
0000BB71 8B15[4E820100]      <1>     mov     edx, [DestinationFile_DirEntry+DirEntry_FileSize]
0000BB77 39C2                <1>     cmp     edx, eax
0000BB79 7616                <1>     jna     short csftdf2_save_fat_file_err3
0000BB7B A1[9C820100]        <1>     mov     eax, [csftdf_df_cluster] ; last (empty) cluster
                             <1>     ; ESI = Logical dos drive description table address
0000BB80 E8990B0000          <1>     call    truncate_cluster_chain
0000BB85 720A                <1>     jc      short csftdf2_save_fat_file_err3
0000BB87 A1[AC820100]        <1>     mov     eax, [csftdf_df_wbytes]
0000BB8C A3[4E820100]        <1>     mov     [DestinationFile_DirEntry+DirEntry_FileSize], eax
                             <1> csftdf2_save_fat_file_err3:
0000BB91 E866F1FFFF          <1>     call    convert_current_date_time
                             <1>     ; DX = Date in dos dir entry format
                             <1>     ; AX = Time in dos dir entry format
0000BB96 C605[3F820100]00    <1>     mov     byte [DestinationFile_DirEntry+DirEntry_CrtTimeTenth], 0
0000BB9D 66A3[40820100]      <1>     mov     [DestinationFile_DirEntry+DirEntry_CrtTime], ax
0000BBA3 668915[42820100]    <1>     mov     [DestinationFile_DirEntry+DirEntry_CrtDate], dx
0000BBAA 66A3[48820100]      <1>     mov     [DestinationFile_DirEntry+DirEntry_WrtTime], ax
0000BBB0 668915[4A820100]    <1>     mov     [DestinationFile_DirEntry+DirEntry_WrtDate], dx
0000BBB7 F9                  <1>     stc
                             <1> csftdf2_save_fat_file_7:
0000BBB8 9C                  <1>     pushf
0000BBB9 668915[44820100]    <1>     mov     [DestinationFile_DirEntry+DirEntry_LastAccDate], dx
0000BBC0 BE[32820100]        <1>     mov     esi, DestinationFile_DirEntry
0000BBC5 BF00000800          <1>     mov     edi, Directory_Buffer
0000BBCA 0FB70D[5A820100]    <1>     movzx   ecx, word [DestinationFile_DirEntryNumber] ; (<2048)
                             <1>     ;shl    cx, 5 ; 32 * directory entry number
                             <1>     ; 29/07/2022
0000BBD1 C1E105              <1>     shl     ecx, 5
0000BBD4 01CF                <1>     add     edi, ecx
                             <1>     ;mov    ecx, 8
                             <1>     ;mov    cx, 8
                             <1>     ; 31/08/2024
                             <1>     ; 29/07/2022
0000BBD6 29C9                <1>     sub     ecx, ecx
                             <1>     ;sub    ch, ch
0000BBD8 B108                <1>     mov     cl, 8
0000BBDA F3A5                <1>     rep     movsd
0000BBDC 9D                  <1>     popf
0000BBDD 730B                <1>     jnc     short csftdf2_write_file_OK
                             <1>
                             <1> csftdf2_write_error:
                             <1>     ; 18/03/2016
                             <1>     ;mov    al, 1Dh ; write error
                             <1>     ; 31/08/2024
0000BBDF B012                <1>     mov     al, ERR_DRV_WRITE ; 18 ; write error
0000BBE1 EB02                <1>     jmp     short csftdf2_rw_error
                             <1>
                             <1>     ; 16/03/2016
                             <1> csftdf2_read_error:
0000BBE3 B011                <1>     mov     al, 17 ; ; Drive not ready or read error!
                             <1> csftdf2_rw_error:
0000BBE5 A2[89820100]        <1>     mov     [csftdf_rw_err], al
                             <1>
                             <1> csftdf2_write_file_OK:
                             <1>     ; 18/03/2016
```

```
0000BBEA C605[9D7E0100]02    <1>     mov    byte [DirBuff_ValidData], 2
0000BBF1 E89BF1FFFF          <1>     call   save_directory_buffer
                             <1>
                             <1>     ; Update last modification date&time of destination
                             <1>     ; file's (parent) directory
0000BBF6 E81FF2FFFF          <1>     call   update_parent_dir_lmdt
                             <1>     ;
0000BBFB A1[90820100]        <1>     mov    eax, [csftdf_sf_mem_addr] ; start address
                             <1>
0000BC00 21C0                <1>     and    eax, eax
0000BC02 7509                <1>     jnz    short csftdf2_dealloc_mblock
                             <1>
                             <1>     ; 31/08/2024
                             <1>     ;mov   ch, al ; 0 (Cluster r/w, not full loading)
                             <1> csftdf2_dealloc_retn:
                             <1>     ;mov   cl, [csftdf_rw_err]
                             <1>     ;mov   eax, [csftdf_df_cluster]
                             <1>     ; 31/08/2024
0000BC04 A0[89820100]        <1>     mov    al, [csftdf_rw_err]
0000BC09 3C01                <1>     cmp    al, 1
0000BC0B F5                  <1>     cmc
                             <1>     ; if al > 0 -> cf = 1
                             <1> csftdf2_write_df_error:   ; 31/08/2024
                             <1> csftdf2_write_df_clust_@@: ; 31/08/2024
0000BC0C C3                  <1>     retn
                             <1>
                             <1> csftdf2_dealloc_mblock:
0000BC0D 8B0D[94820100]      <1>     mov    ecx, [csftdf_sf_mem_bsize] ; block size
0000BC13 E847A3FFFF          <1>     call   deallocate_memory_block
                             <1>     ; 31/08/2024
                             <1>         ;mov       ch, 0FFh ; (File was full loaded at memory)
0000BC18 EBEA                <1>     jmp    short csftdf2_dealloc_retn
                             <1>
                             <1>     ;;;;
                             <1> csftdf2_update_df_fclust:
                             <1>     ; 31/08/2024
                             <1>     ; 27/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; update (directory entry of)
                             <1>     ; the new (created) destination file
                             <1>     ; with new first cluster (it was 0 before)
                             <1>
0000BC1A 8B35[B8820100]      <1>     mov    esi, [csftdf_df_drv_dt]       ; (!)
                             <1>
                             <1> csftdf2_update_df_fclust_@: ; 31/08/2024
0000BC20 A1[9C820100]        <1>     mov    eax, [csftdf_df_cluster]
0000BC25 21C0                <1>     and    eax, eax
0000BC27 75E3                <1>     jnz    short csftdf2_write_df_clust_@@
                             <1>     ; if eax = 0, add new cluster
                             <1> ; 31/08/2024
                             <1> %if 0
                             <1>     call   get_first_free_cluster
                             <1>     jc     short csftdf2_write_df_error
                             <1>     ; EAX >= 2 and EAX < FFFFFFFFh is valid
                             <1>
                             <1>     cmp    eax, 0FFFFFFFFh ; no free space
                             <1>     jne    short csftdf2_write_df_clust_@
                             <1>     inc    eax ; 0
                             <1>     stc
                             <1>     mov    al, 27h ; insufficient disk space
                             <1> csftdf2_write_df_error:
                             <1>     retn
                             <1> %else
                             <1>     ; 31/08/2024
0000BC29 E8DE0B0000          <1>     call   add_new_cluster
0000BC2E 72DC                <1>     jc     short csftdf2_write_df_error
                             <1> %endif
                             <1>
                             <1> csftdf2_write_df_clust_@:
                             <1>     ; 31/08/2024
0000BC30 A3[9C820100]        <1>     mov    [csftdf_df_cluster], eax
                             <1>     ;
                             <1>     ; eax = (new) first cluster
0000BC35 56                  <1>     push   esi
0000BC36 BE[32820100]        <1>     mov    esi, DestinationFile_DirEntry
0000BC3B 0FB73D[5A820100]    <1>     movzx  edi, word [DestinationFile_DirEntryNumber]
0000BC42 C1E705              <1>     shl    edi, 5 ; 32 * index number
0000BC45 81C700000800        <1>     add    edi, Directory_Buffer
0000BC4B C1C010              <1>     rol    eax, 16
0000BC4E 66894614            <1>     mov    [esi+DirEntry_FstClusHI],ax
0000BC52 C1C810              <1>     ror    eax, 16
0000BC55 6689461A            <1>     mov    [esi+DirEntry_FstClusLO],ax
0000BC59 B908000000          <1>     mov    ecx, 8 ; 32 bytes
0000BC5E F3A5                <1>     rep    movsd
0000BC60 5E                  <1>     pop    esi
                             <1>     ;
0000BC61 C605[9D7E0100]02    <1>     mov    byte [DirBuff_ValidData], 2 ; change sign
                             <1>
                             <1> ; 31/08/2024
                             <1> %if 0
                             <1>     call   save_directory_buffer
                             <1>     jc     short csftdf2_write_df_error
                             <1>     ;
                             <1>     ; +1 cluster is in use (as first cluster)
                             <1>     mov    ebx, [esi+LD_FreeSectors]
                             <1>     inc    ebx  ; 0FFFFFFFFh ? invalid ?
                             <1>     jz     short csftdf2_write_df_clust_@@ ; yes
                             <1>     xor    ebx, ebx
                             <1>     mov    bl, [esi+LD_BPB+BPB_SecPerClust]
                             <1>     sub    [esi+LD_FreeSectors], ebx
                             <1>     ;
                             <1> csftdf2_write_df_clust_@@:
                             <1>     retn
                             <1>     ;;;;
                             <1> %else
                             <1>     ; 31/08/2024
0000BC68 E924F1FFFF          <1>     jmp    save_directory_buffer
                             <1> %endif
                             <1>
                             <1> csftdf2_save_fs_file:
                             <1>     ; 16/10/2016 (1Dh -> 18)
                             <1>     ; temporary - (21/03/2016)
                             <1>     ;mov   eax, 18 ; write error
                             <1>     ; 29/07/2022
0000BC6D 31C0                <1>     xor    eax, eax
0000BC6F B012                <1>     mov    al, 18
0000BC71 F9                  <1>     stc
0000BC72 C3                  <1>     retn
                             <1>
                             <1> create_file:
                             <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                             <1>     ; 31/08/2024
                             <1>     ; 29/08/2024
                             <1>     ; 27/08/2024
                             <1>     ; 26/08/2024 (TRDOS 386 v2.0.9)
```

```
                                <1>     ; 30/07/2022
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 16/10/2016
                                <1>     ; 24/03/2016, 31/03/2016
                                <1>     ; 20/03/2016, 21/03/2016, 23/03/2016
                                <1>     ; 19/03/2016 (TRDOS 396 = TRDOS v2.0)
                                <1>     ; 03/09/2011 (FILE.ASM, 'proc_create_file')
                                <1>     ; 09/08/2010
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = File Size
                                <1>     ;       ESI = ASCIIZ File Name
                                <1>     ;       CL = File Attributes
                                <1>     ;       EBX = FFFFFFFFh -> create empty file
                                <1>     ;                          (only for FAT fs)
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = New file's first cluster
                                <1>     ;                (0 for empty file) ; 29/08/2024
                                <1>     ;       ESI = Logical Dos Drv Descr. Table Addr.
                                <1>     ;       ;EBX = CreateFile_Size address
                                <1>     ;       ;ECX = Sectors per cluster (<256)
                                <1>     ;       ;EDX = Directory Entry Index/Number (<65536)
                                <1>     ;       ; 29/08/2024
                                <1>     ;       EBX = File Size (0 for a new, empty file)
                                <1>     ;       ECX = Directory Entry Index/Number (<2048)
                                <1>     ;            (in directory cluster, not in directory)
                                <1>     ;       EDX = Directory Cluster Number (of the file)
                                <1>     ;
                                <1>     ;       cf = 1 -> error code in AL (EAX)
                                <1>     ;
                                <1>     ; (Modified registers: eax, ebx, ecx, edx, esi, edi)
                                <1>     ;
                                <1>
                                <1> ;   test    cl, 18h (directory or volume name)
                                <1> ;   jnz     short loc_createfile_access_denied
0000BC73 80E107                 <1>     and     cl, 07h ; S, H, R
0000BC76 880D[D8820100]         <1>         mov     [createfile_attrib], cl
                                <1>
0000BC7C 89D9                   <1>     mov     ecx, ebx
0000BC7E 89F3                   <1>     mov     ebx, esi ; ASCIIZ File Name address
0000BC80 29D2                   <1>     sub     edx, edx
0000BC82 8A35[76770100]         <1>         mov     dh, [Current_Drv]
0000BC88 BE00010900             <1>         mov     esi, Logical_DOSDisks
0000BC8D 01D6                   <1>     add     esi, edx
                                <1>
0000BC8F 8815[E1820100]         <1>     mov     [createfile_UpdatePDir], dl ; 0 ; 31/03/2016
                                <1>
                                <1>     ; LD_DiskType = 0 for write protection (read only)
0000BC95 807E0101               <1>     cmp     byte [esi+LD_DiskType], 1 ; 0 = Invalid
0000BC99 7308                   <1>     jnb     short loc_createfile_check_file_sytem
                                <1>     ; 16/10/2016 (TRDOS Error code: 30, disk write protected)
                                <1>     ;mov    eax, 30 ; 13h, MSDOS err : Disk write-protected
                                <1>     ;mov    dx, 0
                                <1>     ; 29/07/2022
0000BC9B 31D2                   <1>     xor     edx, edx ; 0
0000BC9D 31C0                   <1>     xor     eax, eax ; 0
0000BC9F B01E                   <1>     mov     al, 30
0000BCA1 F9                     <1>     stc
                                <1>     ; err retn: EDX = 0, EBX = File name offset
                                <1>     ; ESI -> Dos drive description table address
0000BCA2 C3                     <1>     retn
                                <1>
                                <1> ;loc_createfile_access_denied:
                                <1> ;   mov     eax, 05h ; access denied (invalid attributes input)
                                <1> ;   stc
                                <1> ;   retn
                                <1>
                                <1> loc_createfile_check_file_sytem:
                                <1>     ; 27/08/2024
0000BCA3 A3[C4820100]           <1>     mov     [createfile_size], eax
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 1
                                <1>     jnb     short loc_createfile_chk_empty_FAT_file_sign1
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ;mov    [createfile_size], eax
                                <1>     ; ESI = Logical Dos Drive Description Table address
                                <1>     ; EBX = ASCIIZ File Name address
                                <1>     jmp     create_fs_file
                                <1> %endif
                                <1>
                                <1> loc_createfile_chk_empty_FAT_file_sign1:
                                <1>     ; ECX = FFFFFFFFh -> create empty file if drive has FAT fs
0000BCA8 41                     <1>     inc     ecx
0000BCA9 7506                   <1>     jnz     short loc_createfile_chk_empty_FAT_file_sign2
                                <1>
0000BCAB 890D[C4820100]         <1>     mov     [createfile_size], ecx ; 0  ; empty file
                                <1>
                                <1> loc_createfile_chk_empty_FAT_file_sign2:
                                <1>     ; 19/12/2025
                                <1>     ; 23/03/2016
                                <1>     ;mov    cx, [esi+LD_BPB+BytesPerSec]
                                <1>     ;mov    [createfile_BytesPerSec], cx
                                <1>
                                <1>     ; EBX = ASCIIZ File Name address
0000BCB1 0FB65613               <1>     movzx   edx, byte [esi+LD_BPB+SecPerClust]
0000BCB5 8815[D9820100]         <1>     mov     [createfile_SecPerClust], dl
0000BCBB 8B4E74                 <1>     mov     ecx, [esi+LD_FreeSectors]
0000BCBE 39D1                   <1>     cmp     ecx, edx ; byte [createfile_SecPerClust]
0000BCC0 7306                   <1>     jnb     short loc_create_fat_file
                                <1>
                                <1> loc_createfile_insufficient_disk_space:
0000BCC2 B827000000             <1>     mov     eax, 27h
                                <1> loc_createfile_gffc_retn:
0000BCC7 C3                     <1>     retn
                                <1>
                                <1> loc_create_fat_file:
0000BCC8 891D[BC820100]         <1>     mov     [createfile_Name_Offset], ebx
0000BCCE 890D[C0820100]         <1>     mov     [createfile_FreeSectors], ecx
                                <1>
                                <1>     ; 27/08/2024
                                <1> ;loc_createfile_gffc_1:
                                <1>     ;call   get_first_free_cluster
                                <1>     ;jc     short loc_createfile_gffc_retn
                                <1>     ;
                                <1>     ;mov    [createfile_FFCluster], eax
                                <1>
                                <1> loc_createfile_locate_ffe_on_directory:
                                <1>     ; Current directory fcluster <> Directory buffer cluster
                                <1>     ; Current directory will be reloaded by
                                <1>     ; 'locate_current_dir_file' procedure
                                <1>     ;
```

```
                                <1>     ; ESI = Logical Dos Drv Desc. Table Adress
0000BCD4 56                     <1>     push    esi ; *
0000BCD5 31C0                   <1>     xor     eax, eax
                                <1>
0000BCD7 A3[927E0100]           <1>     mov     [FAT_ClusterCounter], eax ; 0
                                <1>     ; 21/03/2016
0000BCDC A2[E0820100]           <1>     mov     [createfile_wfc], al ; 0
                                <1>
0000BCE1 89C1                   <1>     mov     ecx, eax
0000BCE3 6649                   <1>     dec     cx ; FFFFh
                                <1>     ; CX = FFFFh -> find first deleted or free entry
                                <1>     ; ESI would be ASCIIZ filename address if the call
                                <1>     ; would not be for first free or deleted dir entry
0000BCE5 E81EE9FFFF             <1>     call    locate_current_dir_file
                                <1>     ;jnc    loc_createfile_set_ff_dir_entry
                                <1>     ; 29/07/2022
0000BCEA 7205                   <1>     jc      short loc_createfile_locate_file_err
                                <1>     ; 26/08/2024
                                <1>     ; cl = 0FFh, ch=0 or ch=0E5h
                                <1>     ; 27/08/2024
                                <1>     ;xor    ecx, ecx ; 26/08/2024
0000BCEC E9DA000000             <1>     jmp     loc_createfile_set_ff_dir_entry
                                <1>
                                <1> loc_createfile_locate_file_err: ; 29/07/2022
0000BCF1 5E                     <1>     pop     esi ; *
                                <1>     ; ESI = Logical DOS Drv. Description Table Address
0000BCF2 83F802                 <1>     cmp     eax, 2
0000BCF5 7402                   <1>     je      short loc_createfile_add_new_cluster
                                <1> loc_createfile_locate_file_stc_retn:
0000BCF7 F9                     <1>     stc
0000BCF8 C3                     <1>     retn
                                <1>
                                <1> loc_createfile_add_new_cluster:
0000BCF9 803D[75770100]02       <1>     cmp     byte [Current_FATType], 2
                                <1>     ;cmp    byte [esi+LD_FATType], 2
0000BD00 770C                   <1>     ja      short loc_createfile_add_new_cluster_check_fsc
0000BD02 803D[74770100]01       <1>     cmp     byte [Current_Dir_Level], 1
                                <1>     ;cmp    byte [esi+LD_CDirLevel], 1
0000BD09 7303                   <1>     jnb     short loc_createfile_add_new_cluster_check_fsc
                                <1>
                                <1>     ;mov    eax, 12
0000BD0B B00C                   <1>     mov     al, 12 ; No more files
                                <1>
                                <1> loc_createfile_anc_retn:
0000BD0D C3                     <1>     retn
                                <1>
                                <1> loc_createfile_add_new_cluster_check_fsc:
0000BD0E 8B0D[C0820100]         <1>     mov     ecx, [createfile_FreeSectors]
0000BD14 0FB605[D9820100]       <1>     movzx   eax, byte [createfile_SecPerClust]
                                <1>     ;shl    ax, 1 ; AX = 2 * AX
                                <1>     ; 29/07/2022
0000BD1B D1E0                   <1>     shl     eax, 1
0000BD1D 39C1                   <1>     cmp     ecx, eax
0000BD1F 72A1                   <1>         jb  short loc_createfile_insufficient_disk_space
                                <1>
                                <1> loc_createfile_add_new_subdir_cluster:
0000BD21 8B15[A27E0100]         <1>     mov     edx, [DirBuff_Cluster]
0000BD27 8915[CC820100]         <1>     mov     [createfile_LastDirCluster], edx
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ;;;
0000BD2D E869040000             <1>     call    get_first_free_cluster
0000BD32 72D9                   <1>     jc      short loc_createfile_anc_retn
0000BD34 A3[C8820100]           <1>     mov     [createfile_FFCluster], eax
                                <1>     ;;;
                                <1>     ; 27/08/2024
                                <1>     ;;;
                                <1>     ;mov    eax, [createfile_FFCluster]
                                <1>     ; eax = cluster address of the new directory sector
                                <1>     ;;;
                                <1>
0000BD39 E807040000             <1>     call    load_FAT_sub_directory
0000BD3E 72CD                   <1>     jc      short loc_createfile_anc_retn
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ; [DirBuff_Cluster] = [createfile_FFCluster]
                                <1>
                                <1> pass_createfile_add_new_subdir_cluster:
                                <1>     ; clear directory buffer (new dir sector) ; 27/08/2024
                                <1>     ;;movzx eax, word [esi+LD_BPB+BytesPerSec]
                                <1>     ;movzx  eax, word [createfile_BytesPerSec] ; 23/03/2016
                                <1>     ; 19/12/2025
                                <1>     ;mov    eax, 512
                                <1>     ; ecx = directory buffer sector count
                                <1>     ;       (sectors per cluster)
                                <1>     ;mul    ecx
                                <1>     ;mov    ecx, eax
                                <1>             ; (bytes per cluster or directory buffer size)
                                <1>     ;shr    ecx, 2 ; dword count
                                <1>     ; 19/12/2025
0000BD40 C1E107                 <1>     shl     ecx, 7 ; 512/4 (dword count)
                                <1>
0000BD43 29C0                   <1>     sub     eax, eax ; 0
0000BD45 F3AB                   <1>     rep     stosd
                                <1>     ;
0000BD47 C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
0000BD4E E83EF0FFFF             <1>     call    save_directory_buffer
0000BD53 72B8                   <1>     jc      short loc_createfile_anc_retn
                                <1>
                                <1> loc_createfile_save_added_subdir_cluster:
0000BD55 A1[CC820100]           <1>     mov     eax, [createfile_LastDirCluster]
0000BD5A 8B0D[C8820100]         <1>     mov     ecx, [createfile_FFCluster]
0000BD60 E80B050000             <1>     call    update_cluster
0000BD65 7304                   <1>     jnc     short loc_createfile_save_fat_buffer_0
0000BD67 09C0                   <1>     or      eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000BD69 7518                   <1>     jnz     short loc_createfile_save_fat_buffer_stc_retn
                                <1>
                                <1> loc_createfile_save_fat_buffer_0:
0000BD6B A1[C8820100]           <1>     mov     eax, [createfile_FFCluster]
0000BD70 A3[CC820100]           <1>     mov     [createfile_LastDirCluster], eax
                                <1>     ;mov    ecx, 0FFFFFFFh ; 28 bit
                                <1>     ; 29/08/2024
                                <1>     ; (update_cluster will save 28 bit cluster value)
                                <1>     ;mov    ecx, 0FFFFFFFFh
0000BD75 29C9                   <1>     sub     ecx, ecx
0000BD77 49                     <1>     dec     ecx ; 0FFFFFFFFh
0000BD78 E8F3040000             <1>     call    update_cluster
0000BD7D 7306                   <1>     jnc     short loc_createfile_save_fat_buffer_1
0000BD7F 09C0                   <1>     or      eax, eax ; Was it EOF ?
0000BD81 7402                   <1>     jz      short loc_createfile_save_fat_buffer_1 ; yes
                                <1>
                                <1> loc_createfile_save_fat_buffer_stc_retn:
0000BD83 F9                     <1>     stc
                                <1> loc_createfile_save_fat_buffer_retn:
```

```
                                <1> loc_createfile_gffc_2_stc_retn:
0000BD84 C3                     <1>     retn
                                <1>
                                <1> loc_createfile_save_fat_buffer_1:
                                <1>     ; byte [FAT_BuffValidData] = 2
0000BD85 E85A070000             <1>     call    save_fat_buffer
0000BD8A 72F8                   <1>     jc      short loc_createfile_save_fat_buffer_retn
                                <1>
0000BD8C 803D[927E0100]01       <1>     cmp     byte [FAT_ClusterCounter], 1
0000BD93 7222                   <1>     jb      short loc_createfile_save_fat_buffer_2
                                <1>
                                <1>     ; ESI = Logical DOS Drive Description Table address
0000BD95 A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter]
                                <1>
0000BD9A C605[927E0100]00       <1>     mov     byte [FAT_ClusterCounter], 0 ; 21/03/2016
                                <1>
0000BDA1 66BB01FF               <1>     mov     bx, 0FF01h ; add free clusters
0000BDA5 E8C7070000             <1>     call    calculate_fat_freespace
                                <1>
                                <1>     ;inc    eax ; 0FFFFFFFFh -> 0 ; recalculation is needed!
                                <1>     ;jnz    short loc_createfile_save_fat_buffer_2
                                <1>
                                <1>     ; ecx > 0 -> Recalculation is needed
0000BDAA 09C9                   <1>     or      ecx, ecx
0000BDAC 7409                   <1>     jz      short loc_createfile_save_fat_buffer_2
                                <1>
0000BDAE 66BB00FF               <1>     mov     bx, 0FF00h ; recalculate free space
0000BDB2 E8BA070000             <1>     call    calculate_fat_freespace
                                <1>
                                <1> loc_createfile_save_fat_buffer_2:
                                <1>     ;call  update_parent_dir_lmdt
                                <1>
                                <1>     ; 27/08/2024
                                <1> ;loc_createfile_gffc_2:
                                <1>     ;call  get_first_free_cluster
                                <1>     ;jc    short loc_createfile_gffc_2_stc_retn
                                <1>     ;
                                <1>     ;mov   [createfile_FFCluster], eax
                                <1>     ;
                                <1>
0000BDB7 A1[CC820100]           <1>     mov     eax, [createfile_LastDirCluster]
                                <1>
0000BDBC E884030000             <1>     call    load_FAT_sub_directory
0000BDC1 72C1                   <1>     jc      short loc_createfile_gffc_2_stc_retn
                                <1>
0000BDC3 BF00000800             <1>     mov     edi, Directory_Buffer
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ;sub   bx, bx ; directory entry index/number = 0
0000BDC8 29DB                   <1>     sub     ebx, ebx
                                <1>     ; 29/07/2022
                                <1>     ;sub   ecx, ecx
                                <1>
0000BDCA 56                     <1>     push    esi ; * ; 23/03/2016
                                <1>
                                <1> loc_createfile_set_ff_dir_entry:
                                <1>     ;;;
                                <1>     ; 29/08/2024
0000BDCB A1[A27E0100]           <1>     mov     eax, [DirBuff_Cluster]
0000BDD0 A3[CC820100]           <1>     mov     [createfile_LastDirCluster], eax
                                <1>     ;;;
                                <1>
                                <1>     ; 27/08/2024
0000BDD5 66891D[DA820100]       <1>     mov     [createfile_DirIndex], bx
                                <1>     ; 29/07/2022
                                <1>     ;mov   [createfile_DirIndex], cx ; 0
                                <1>
                                <1>     ;;;
0000BDDC A1[C4820100]           <1>     mov     eax, [createfile_size]
0000BDE1 09C0                   <1>     or      eax, eax
0000BDE3 740A                   <1>     jz      short loc_createfile_sffc
                                <1>     ; 31/08/2024
0000BDE5 8B3424                 <1>     mov     esi, [esp] ; * ; LDDDT address
0000BDE8 E8AE030000             <1>     call    get_first_free_cluster
0000BDED 7295                   <1>     jc      short loc_createfile_gffc_2_stc_retn
                                <1> loc_createfile_sffc:
0000BDEF A3[C8820100]           <1>     mov     [createfile_FFCluster], eax
0000BDF4 29C9                   <1>     sub     ecx, ecx ; 0
                                <1>     ;;;
                                <1>
                                <1>         ; EDI = Directory entry address
0000BDF6 8B35[BC820100]         <1>     mov     esi, [createfile_Name_Offset]
                                <1>     ;;;
                                <1>     ; 27/08/2024
                                <1>     ;mov   eax, [createfile_FFCluster]
                                <1>     ;;;
0000BDFC A3[D0820100]           <1>     mov     [createfile_Cluster], eax ; 24/03/2016
                                <1>     ;mov   ch, 0FFh
                                <1>         ; 29/07/2022
                                <1>     ; ecx = 0
0000BE01 FECD                   <1>     dec     ch ; 0 -> 0FFh
0000BE03 8A0D[D8820100]         <1>     mov     cl, [createfile_attrib] ; file attributes
                                <1>     ; CH > 0 -> File size is in [EBX]
0000BE09 BB[C4820100]           <1>     mov     ebx, createfile_size
                                <1>
0000BE0E E8ABEEFFFF             <1>     call    make_directory_entry
                                <1>
0000BE13 5E                     <1>     pop     esi ; * ; ESI = Logical Dos Drv Desc. Table address
                                <1>
0000BE14 C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
0000BE1B E871EFFFFF             <1>     call    save_directory_buffer
0000BE20 7213                   <1>     jc      short loc_createfile_set_ff_dir_entry_retn
                                <1>
0000BE22 C605[E1820100]01       <1>     mov     byte [createfile_UpdatePDir], 1 ; 31/03/2016
                                <1>
                                <1> loc_createfile_get_set_write_file_cluster:
0000BE29 A1[C4820100]           <1>     mov     eax, [createfile_size]
0000BE2E 09C0                   <1>     or      eax, eax ; 0 ?
0000BE30 755B                   <1>     jnz     short loc_createfile_get_set_wfc_cont ; no
0000BE32 40                     <1>     inc     eax ; eax = 1
                                <1>
                                <1>     ; 27/08/2024 (empty file)
                                <1>     ;;;
                                <1>     ; 23/03/2016
                                <1>     ;movzx ebx, byte [createfile_SecPerClust]
                                <1>     ;;movzx ecx, word [esi+LD_BPB+BytesPerSec] ; 512
                                <1>         ;movzx     ecx, word [createfile_BytesPerSec] ; 512
                                <1>     ;;;
0000BE33 EB72                   <1>     jmp     loc_createfile_set_cluster_count
                                <1>
                                <1> loc_createfile_set_ff_dir_entry_retn:
0000BE35 C3                     <1>     retn
                                <1>
                                <1> loc_createfile_write_fcluster_to_disk:
```

```
0000BE36 034668              <1>     add    eax, [esi+LD_DATABegin] ; convert to physical address
0000BE39 BB00000700          <1>     mov    ebx, Cluster_Buffer
                             <1>     ; ESI = Logical DOS Drv. Desc. Tbl. address
                             <1>     ; EAX = Disk address
                             <1>     ; EBX = Sector Buffer
                             <1>     ; ECX = sectors per cluster
0000BE3E E8A15B0000          <1>     call   disk_write
0000BE43 7211                <1>     jc     short loc_createfile_dsk_wr_err
                             <1>
                             <1> loc_createfile_update_fat_cluster:
                             <1>     ; 21/03/2016
0000BE45 803D[E0820100]00    <1>     cmp    byte [createfile_wfc], 0
0000BE4C 7711                <1>     ja     short loc_createfile_update_fat_cluster_n1
                             <1>
0000BE4E FE05[E0820100]      <1>     inc    byte [createfile_wfc] ; 1
0000BE54 EB1F                <1>     jmp    short loc_createfile_update_fat_cluster_n2
                             <1>
                             <1> loc_createfile_dsk_wr_err:
                             <1>     ; 16/10/2016 (1Dh -> 18)
                             <1>     ; 23/03/2016
                             <1>     ;mov   eax, 18 ; Drive not ready or write error !
                             <1>     ; 29/07/2022
0000BE56 29C0                <1>     sub    eax, eax
0000BE58 B012                <1>     mov    al, 18
                             <1> loc_cf_stc_retn:
0000BE5A E9BA000000          <1>     jmp    loc_createfile_stc_retn
                             <1>
                             <1> loc_createfile_update_fat_cluster_n1:
0000BE5F A1[D4820100]        <1>     mov    eax, [createfile_PCluster]
0000BE64 8B0D[D0820100]      <1>     mov    ecx, [createfile_Cluster]
0000BE6A E801040000          <1>     call   update_cluster
0000BE6F 7304                <1>     jnc    short loc_createfile_update_fat_cluster_n2
0000BE71 09C0                <1>     or     eax, eax ; EAX = 0 -> cluster value is 0 or eocc
                             <1>     ;jnz   loc_createfile_stc_retn
                             <1>     ; 29/07/2022
0000BE73 75E5                <1>     jnz    short loc_cf_stc_retn
                             <1>
                             <1> loc_createfile_update_fat_cluster_n2:
0000BE75 A1[D0820100]        <1>         mov eax, [createfile_Cluster]
                             <1>     ;mov   ecx, 0FFFFFFFh
                             <1>     ; 29/08/2024
                             <1>     ; (update_cluster will save 28 bit cluster value)
                             <1>     ;mov   ecx, 0FFFFFFFh
0000BE7A 29C9                <1>     sub    ecx, ecx
0000BE7C 49                  <1>     dec    ecx
0000BE7D E8EE030000          <1>     call   update_cluster
0000BE82 7353                <1>     jnc    short loc_createfile_save_fat_buffer_3
0000BE84 09C0                <1>     or     eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000BE86 744F                <1>     jz     short loc_createfile_save_fat_buffer_3
                             <1>
                             <1> loc_cf_upd_fat_fcluster_stc_retn:
0000BE88 E98C000000          <1>     jmp    loc_createfile_stc_retn
                             <1>
                             <1> loc_createfile_get_set_wfc_cont:
                             <1>     ;;movzx ecx, word [esi+LD_BPB+BytesPerSec] ; 512
                             <1>     ;movzx  ecx, word [createfile_BytesPerSec] ; 512
                             <1>     ; 19/12/2025
0000BE8D B900020000          <1>     mov    ecx, 512
0000BE92 01C8                <1>     add    eax, ecx
0000BE94 48                  <1>     dec    eax  ; add eax, 511
0000BE95 29D2                <1>     sub    edx, edx
0000BE97 F7F1                <1>     div    ecx
0000BE99 0FB61D[D9820100]    <1>     movzx  ebx, byte [createfile_SecPerClust]
0000BEA0 01D8                <1>     add    eax, ebx
0000BEA2 48                  <1>     dec    eax  ; add eax, SecPerClust - 1
                             <1>     ;xor   dx, dx
                             <1>     ; 27/08/2024
0000BEA3 31D2                <1>     xor    edx, edx
0000BEA5 F7F3                <1>     div    ebx
                             <1>
                             <1> loc_createfile_set_cluster_count:
0000BEA7 A3[DC820100]        <1>     mov    [createfile_CCount], eax
                             <1>
                             <1>     ; 31/08/2024
                             <1>     ;;;
0000BEAC 833D[C4820100]00    <1>     cmp    dword [createfile_size], 0 ; empty file ?
0000BEB3 7422                <1>     jz     short loc_createfile_save_fat_buffer_3
                             <1>     ;;;
                             <1>
0000BEB5 BF00000700          <1>     mov    edi, Cluster_Buffer
0000BEBA 89C8                <1>     mov    eax, ecx ; Bytes per Sector
0000BEBC F7E3                <1>     mul    ebx ; Sectors per Cluster
                             <1>     ; EAX = Bytes per Cluster
0000BEBE 89C1                <1>     mov    ecx, eax
0000BEC0 C1E902              <1>     shr    ecx, 2 ; dword count
0000BEC3 31C0                <1>     xor    eax, eax
0000BEC5 F3AB                <1>     rep    stosd ; clear cluster buffer
                             <1>
0000BEC7 A1[D0820100]        <1>     mov    eax, [createfile_Cluster] ; 24/03/2016
                             <1>
0000BECC 89D9                <1>     mov    ecx, ebx
                             <1>
                             <1> loc_createfile_get_set_wf_fclust_cont:
                             <1>     ;sub   eax, 2
                             <1>     ; 30/07/2022
0000BECE 48                  <1>     dec    eax
0000BECF 48                  <1>     dec    eax
0000BED0 F7E1                <1>     mul    ecx
                             <1>     ; EAX = Logical DOS disk address (offset)
0000BED2 E95FFFFFFF          <1>         jmp     loc_createfile_write_fcluster_to_disk
                             <1>
                             <1> loc_createfile_save_fat_buffer_3:
                             <1>     ; byte [FAT_BuffValidData] = 2
0000BED7 E808060000          <1>     call   save_fat_buffer
                             <1>     ;jc    loc_createfile_stc_retn
                             <1>     ; 29/07/2022
0000BEDC 72AA                <1>     jc     short loc_cf_upd_fat_fcluster_stc_retn
                             <1>
                             <1>     ; 21/03/2016
                             <1>     ;cmp   byte [FAT_ClusterCounter], 1
                             <1>     ;jb    short loc_createfile_save_fat_buffer_4
                             <1>     ; 31/08/2024
0000BEDE 803D[927E0100]00    <1>     cmp    byte [FAT_ClusterCounter], 0
0000BEE5 761B                <1>     jna    short loc_createfile_save_fat_buffer_4 ; cf = 0
                             <1>
                             <1>     ; ESI = Logical DOS Drive Description Table address
0000BEE7 A1[927E0100]        <1>     mov    eax, [FAT_ClusterCounter]
0000BEEC 66BB01FF            <1>     mov    bx, 0FF01h ; add free clusters
0000BEF0 E87C060000          <1>     call   calculate_fat_freespace
                             <1>
                             <1>     ;inc   eax ; 0FFFFFFFFh -> 0 ; recalculation is needed!
                             <1>     ;jnz   short loc_createfile_save_fat_buffer_4
                             <1>
                             <1>     ; ecx > 0 -> Recalculation is needed
```

```
0000BEF5 09C9                <1>     or      ecx, ecx
0000BEF7 7409                <1>     jz      short loc_createfile_save_fat_buffer_4
                             <1>
0000BEF9 66BB00FF            <1>     mov     bx, 0FF00h ; ; recalculate free space
0000BEFD E86F060000          <1>     call    calculate_fat_freespace
                             <1>
                             <1> loc_createfile_save_fat_buffer_4:
0000BF02 FF0D[DC820100]      <1>     dec     dword [createfile_CCount]
                             <1>     ;jz     short loc_createfile_upd_dir_modif_date_time
0000BF08 743D                <1>     jz      short loc_createfile_stc_retn_cc ; 31/03/2016
                             <1>
                             <1> loc_createfile_get_set_write_next_cluster:
0000BF0A E88C020000          <1>     call    get_first_free_cluster
0000BF0F 7208                <1>     jc      short loc_createfile_stc_retn
                             <1>
                             <1> loc_createfile_get_set_write_next_cluster_1:
0000BF11 83F8FF              <1>     cmp     eax, 0FFFFFFFFh
0000BF14 7211                <1>     jb      short loc_createfile_get_set_write_next_cluster_2
                             <1>
                             <1> ;loc_createfile_wnc_insufficient_disk_space:
                             <1>     ;mov    eax, 27h ; Insufficient disk space
                             <1>     ; 29/07/2022
                             <1>     ;xor    eax, eax
                             <1>     ; 29/08/2024
                             <1>     ; eax = FFFFFFFFh
0000BF16 40                  <1>     inc     eax ; eax = 0
0000BF17 B027                <1>     mov     al, 27h
                             <1>
                             <1> loc_createfile_stc_retn:
0000BF19 803D[E0820100]01    <1>     cmp     byte [createfile_wfc], 1
0000BF20 7324                <1>     jnb     short loc_createfile_err_retn
0000BF22 C3                  <1>     retn
                             <1>
                             <1> loc_createfile_wnc_inv_format_retn:
                             <1>     ;mov    eax, 28
0000BF23 B01C                <1>     mov     al, 28 ; Invalid format
0000BF25 EBF2                <1>     jmp     short loc_createfile_stc_retn
                             <1>
                             <1> loc_createfile_get_set_write_next_cluster_2:
0000BF27 83F802              <1>     cmp     eax, 2
0000BF2A 72F7                <1>     jb      short loc_createfile_wnc_inv_format_retn
                             <1>
                             <1> loc_createfile_get_set_write_next_cluster_3:
0000BF2C 8B0D[D0820100]      <1>     mov     ecx, [createfile_Cluster]
0000BF32 A3[D0820100]        <1>     mov     [createfile_Cluster], eax
0000BF37 890D[D4820100]      <1>     mov     [createfile_PCluster], ecx
0000BF3D 0FB60D[D9820100]    <1>     movzx   ecx, byte [createfile_SecPerClust]
0000BF44 EB88                <1>     jmp     short loc_createfile_get_set_wf_fclust_cont
                             <1>
                             <1> loc_createfile_err_retn:
0000BF46 F9                  <1>     stc
                             <1>
                             <1> ;loc_createfile_upd_dir_modif_date_time:
                             <1> loc_createfile_stc_retn_cc: ; 31/03/2016
0000BF47 9C                  <1>     pushf   ; cpu is here for an error return or completion
0000BF48 50                  <1>     push    eax ; error code if cf = 1
                             <1>
                             <1>     ;call   update_parent_dir_lmdt
                             <1>
                             <1> ;loc_createfile_stc_retn_cc:
0000BF49 A1[927E0100]        <1>     mov     eax, [FAT_ClusterCounter]
0000BF4E 09C0                <1>     or      eax, eax
0000BF50 741A                <1>     jz      short loc_createfile_stc_retn_pop_eax
0000BF52 8A3D[76770100]      <1>     mov     bh, [Current_Drv]
0000BF58 B301                <1>     mov     bl, 01h ; BL = 1 -> add clusters
                             <1>     ; NOTE: EAX value will be added to Free Cluster Count
                             <1>     ; (If EAX value is negative, Free Cluster Count will be decreased)
0000BF5A E812060000          <1>     call    calculate_fat_freespace
                             <1>         ; ESI = Logical DOS Drive Description Table Address
                             <1>         ;jc short loc_createfile_stc_retn_pop_eax_cf
0000BF5F 21C9                <1>     and     ecx, ecx ; cx = 0 -> valid free sector count
0000BF61 7409                <1>     jz      short loc_createfile_stc_retn_pop_eax
                             <1>
                             <1> loc_createfile_stc_retn_recalc_FAT_freespace:
0000BF63 66BB00FF            <1>     mov     bx, 0FF00h ; bh = 0FFh ->
                             <1>     ; ESI = Logical DOS Drv DT Addr
                             <1>     ; BL = 0 -> Recalculate
0000BF67 E805060000          <1>     call    calculate_fat_freespace
                             <1>
                             <1> loc_createfile_stc_retn_pop_eax:
0000BF6C 58                  <1>     pop     eax
0000BF6D 9D                  <1>     popf
0000BF6E 7218                <1>     jc      short loc_createfile_retn
                             <1>
                             <1> loc_createfile_retn_fcluster:
0000BF70 A1[C8820100]        <1>     mov     eax, [createfile_FFCluster]
                             <1>     ;mov    ebx, createfile_size
                             <1>     ;;movzx ecx, byte [esi+LD_BPB+SecPerClust]
                             <1>     ;movzx  ecx, byte [createfile_SecPerClust] ; 23/03/2016
                             <1>     ;movzx  edx, word [createfile_DirIndex]
                             <1>     ; 29/08/2024 - TRDOS 386 v2.0.9
0000BF75 8B1D[C4820100]      <1>     mov     ebx, [createfile_size]
0000BF7B 0FB70D[DA820100]    <1>     movzx   ecx, word [createfile_DirIndex] ; = directory entry index
0000BF82 8B15[CC820100]      <1>     mov     edx, [createfile_LastDirCluster] ; = [DirBuff_Cluster]
                             <1>     ; esi = Logical DOS Drive Description Table address
                             <1>     ; byte [esi] = drive name ; 'A','B','C','D'
                             <1>
                             <1> loc_createfile_retn:
0000BF88 C3                  <1>     retn
                             <1>
                             <1> ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                             <1> ; 28/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1> ;
                             <1> ;create_fs_file:
                             <1> ;   ; temporary (21/03/2016)
                             <1> ;   ;retn
                             <1> ;
                             <1> ;delete_fs_file:
                             <1> ;   ; temporary (28/02/2016)
                             <1> ;   ;retn
                             <1> ;
                             <1> ;rename_fs_file_or_directory:
                             <1> ;   ;retn
                             <1> ;
                             <1> ;make_fs_directory:
                             <1> ;   ; temporary (21/02/2016)
                             <1> ;   ;retn
                             <1> ;
                             <1> ;add_new_fs_section:
                             <1> ;   ; temporary (11/03/2016)
                             <1> ;   ;retn
                             <1> ;
                             <1> ;delete_fs_directory_entry:
                             <1> ;   ; temporary (11/03/2016)
```

```
                                <1> ;   ;retn
                                <1> ;
                                <1> ;csftdf2_read_fs_file_sectors:
                                <1> ;   ; temporary (19/03/2016)
                                <1> ;   ;retn
                                <1> ;
                                <1> ;csftdf2_write_fs_file_sectors:
                                <1> ;   ; temporary (19/03/2016)
                                <1> ;   retn
                                    %include 'trdosk5.s' ; 24/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - File System Procedures : trdosk5s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 21/12/2025 (Previous: 31/08/2024, v2.0.9)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.11 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; DRV_FAT.ASM (21/08/2011)
                                <1> ; ****************************************************************************
                                <1> ; DRV_FAT.ASM (c) 2005-2011 Erdogan TAN [ 07/07/2009 ] Last Update: 21/08/2011
                                <1>
                                <1> get_next_cluster:
                                <1>     ; 07/08/2022
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
                                <1>     ; 23/03/2016
                                <1>     ; 01/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 05/07/2011
                                <1>     ; 07/07/2009
                                <1>     ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Cluster Number (32 bit)
                                <1>     ;       ESI = Logical DOS Drive Parameters Table
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 0 -> No Error, EAX valid
                                <1>     ;       cf = 1 & EAX = 0 -> End Of Cluster Chain
                                <1>     ;       cf = 1 & EAX > 0 -> Error
                                <1>     ;       ECX = Current/Previous cluster (if CF = 0)
                                <1>     ;       EAX = Next Cluster Number (32 bit)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
0000BF89 A3[867E0100]           <1>     mov     [FAT_CurrentCluster], eax
                                <1> check_next_cluster_fat_type:
0000BF8E 29D2                   <1>     sub     edx, edx ; 0
0000BF90 807E0302               <1>     cmp      byte [esi+LD_FATType], 2
0000BF94 7243                   <1>     jb      short get_FAT12_next_cluster
                                <1>     ;ja     get_FAT32_next_cluster
                                <1>     ; 25/07/2022
0000BF96 7605                   <1>     jna     short get_FAT16_next_cluster
0000BF98 E9B2000000             <1>     jmp     get_FAT32_next_cluster
                                <1>
                                <1> get_FAT16_next_cluster:
0000BF9D BB00030000             <1>     mov     ebx, 300h ;768
0000BFA2 F7F3                   <1>     div     ebx
                                <1>     ; EAX = Count of 3 FAT sectors
                                <1>     ; EDX = Cluster Offset (< 768)
                                <1>     ;shl    dx, 1 ; Multiply by 2
                                <1>     ; 25/07/2022
0000BFA4 D1E2                   <1>     shl     edx, 1
0000BFA6 89D3                   <1>     mov     ebx, edx ; Byte Offset
0000BFA8 81C3001C0900           <1>     add     ebx, FAT_Buffer
0000BFAE 66BA0300               <1>     mov     dx, 3
0000BFB2 F7E2                   <1>     mul     edx
                                <1>     ; EAX = FAT Sector (<= 256)
                                <1>     ; EDX = 0
0000BFB4 8A0E                   <1>     mov     cl, [esi+LD_Name]
                                <1>     ;cmp    byte [FAT_BuffValidData], 0
0000BFB6 3815[8A7E0100]         <1>         cmp [FAT_BuffValidData], dl ; 0
0000BFBC 7674                   <1>     jna     short load_FAT_sectors0
0000BFBE 3A0D[8B7E0100]         <1>     cmp     cl, [FAT_BuffDrvName]
0000BFC4 756C                   <1>         jne short load_FAT_sectors0
0000BFC6 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
0000BFCC 756A                   <1>         jne short load_FAT_sectors1
                                <1>     ;movzx  eax, word [ebx]
0000BFCE 668B03                 <1>     mov     ax, [ebx]
                                <1>     ; 01/02/2016
                                <1>     ; DRV_FAT.ASM (21/08/2011) had a FATal bug here !
                                <1>     ; (cmp ah, 0Fh) ! (ax >= FF7h)
                                <1>     ; (how can i do a such mistake!?)
                                <1>     ;cmp    al, 0F7h
                                <1>     ;jb     short loc_pass_gnc_FAT16_eoc_check
                                <1>     ;cmp    ah, 0FFh
                                <1>     ;jb     short loc_pass_gnc_FAT16_eoc_check
0000BFD1 6683F8F7               <1>     cmp     ax, 0FFF7h
0000BFD5 724E                   <1>     jb      short loc_pass_gnc_FAT16_eoc_check
                                <1>     ; ax >= FFF7h (cluster 0002h to FFF6h is valid, in use)
0000BFD7 EB4A                   <1>     jmp     short loc_pass_gnc_FAT16_eoc_check_xor_eax
                                <1>
                                <1> get_FAT12_next_cluster:
0000BFD9 BB00040000             <1>     mov     ebx, 400h ;1024
0000BFDE F7F3                   <1>     div     ebx
                                <1>     ; EAX = Count of 3 FAT sectors
                                <1>     ; EDX = Cluster Offset (< 1024)
                                <1>     ; 25/07/2022
                                <1>     ;push   ax
0000BFE0 50                     <1>     push    eax
                                <1>     ;mov    ax, 3
0000BFE1 B003                   <1>     mov     al, 3
                                <1>     ;mul    dx      ; Multiply by 3
0000BFE3 F7E2                   <1>     mul     edx
                                <1>     ;shr    ax, 1   ; Divide by 2
0000BFE5 D1E8                   <1>         shr eax, 1
                                <1>     ;mov    bx, ax  ; Byte Offset
0000BFE7 89C3                   <1>     mov     ebx, eax
0000BFE9 81C3001C0900           <1>     add     ebx, FAT_Buffer
0000BFEF 58                     <1>     pop     eax
                                <1>     ;pop    ax
                                <1>     ;mov    dx, 3
0000BFF0 B203                   <1>     mov     dl, 3
0000BFF2 F7E2                   <1>     mul     edx
                                <1>     ; EAX = FAT Sector (<= 12)
                                <1>     ; EDX = 0
0000BFF4 8A0E                   <1>     mov     cl, [esi+LD_Name]
                                <1>     ;cmp    byte [FAT_BuffValidData], 0
0000BFF6 3815[8A7E0100]         <1>     cmp     [FAT_BuffValidData], dl ; 0
0000BFFC 7634                   <1>     jna     short load_FAT_sectors0
0000BFFE 3A0D[8B7E0100]         <1>     cmp     cl, [FAT_BuffDrvName]
0000C004 752C                   <1>     jne     short load_FAT_sectors0
0000C006 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
0000C00C 752A                   <1>     jne     short load_FAT_sectors1
```

```
0000C00E A1[867E0100]        <1>     mov     eax, [FAT_CurrentCluster]
                             <1>     ;shr    ax, 1
                             <1>     ; 25/07/2022
0000C013 D1E8                <1>     shr     eax, 1
                             <1>     ;movzx  eax, word [ebx]
0000C015 668B03              <1>     mov     ax, [ebx]
0000C018 7313                <1>     jnc     short get_FAT12_nc_even
                             <1>     ;shr    ax, 4
0000C01A C1E804              <1>     shr     eax, 4
                             <1> loc_gnc_fat12_eoc_check:
                             <1>     ;cmp    al, 0F7h
                             <1>     ;jb     short loc_pass_gnc_FAT16_eoc_check
                             <1>     ;cmp    ah, 0Fh
                             <1>     ;jb     short loc_pass_gnc_FAT16_eoc_check
0000C01D 663DF70F            <1>     cmp     ax, 0FF7h
0000C021 7202                <1>     jb      short loc_pass_gnc_FAT16_eoc_check
                             <1>     ; ax >= FF7h (cluster 0002h to FF6h is valid, in use)
                             <1>
                             <1> loc_pass_gnc_FAT16_eoc_check_xor_eax:
0000C023 31C0                <1>     xor     eax, eax ; 0
                             <1> loc_pass_gnc_FAT16_eoc_check:
                             <1> loc_pass_gnc_FAT32_eoc_check:
0000C025 8B0D[867E0100]      <1>     mov     ecx, [FAT_CurrentCluster]
0000C02B F5                  <1>     cmc
0000C02C C3                  <1>     retn
                             <1>
                             <1> get_FAT12_nc_even:
0000C02D 80E40F              <1>     and     ah, 0Fh
0000C030 EBEB                <1>     jmp     short loc_gnc_fat12_eoc_check
                             <1>
                             <1> load_FAT_sectors0:
0000C032 880D[8B7E0100]      <1>     mov     [FAT_BuffDrvName], cl
                             <1> load_FAT_sectors1:
                             <1>     ; 25/07/2022
                             <1>     ;sub    edx, edx
                             <1>     ; edx = 0
0000C038 A3[8E7E0100]        <1>     mov     [FAT_BuffSector], eax
0000C03D 89C3                <1>     mov     ebx, eax
0000C03F 034660              <1>         add eax, [esi+LD_FATBegin]
0000C042 B202                <1>     mov     dl, 2
                             <1>     ;cmp    byte [esi+LD_FATType], 2
0000C044 385603              <1>         cmp [esi+LD_FATType], dl ; 2
0000C047 7748                <1>     ja      short load_FAT_sectors3
0000C049 0FB74E1C            <1>     movzx   ecx, word [esi+LD_BPB+BPB_FATSz16]
0000C04D EB45                <1>     jmp     short load_FAT_sectors4
                             <1>
                             <1>     ; 07/08/2022
                             <1> get_FAT32_next_cluster:
0000C04F BB80010000          <1>     mov     ebx, 180h ;384
0000C054 F7F3                <1>     div     ebx
                             <1>     ; EAX = Count of 3 FAT sectors
                             <1>     ; EDX = Cluster Offset (< 384)
                             <1>     ;shl    dx, 2   ; Multiply by 4
                             <1>     ; 25/07/2022
0000C056 C1E202              <1>     shl     edx, 2
0000C059 89D3                <1>     mov     ebx, edx ; Byte Offset
0000C05B 81C3001C0900        <1>     add     ebx, FAT_Buffer
0000C061 66BA0300            <1>     mov     dx, 3
0000C065 F7E2                <1>     mul     edx
                             <1>         ; EAX = FAT Sector (<= 2097152) ; (FFFFFF7h * 4) / 512
                             <1>     ;       for 32KB cluster size:
                             <1>     ;       EAX <= 1024 = (4GB / 32KB) * 4) / 512
                             <1>     ; EDX = 0
0000C067 8A0E                <1>     mov     cl, [esi+LD_Name]
                             <1>     ;cmp    byte [FAT_BuffValidData], 0
0000C069 3815[8A7E0100]      <1>     cmp     [FAT_BuffValidData], dl ; 0
0000C06F 76C1                <1>     jna     short load_FAT_sectors0
0000C071 3A0D[8B7E0100]      <1>     cmp     cl, [FAT_BuffDrvName]
0000C077 75B9                <1>     jne     short load_FAT_sectors0
0000C079 3B05[8E7E0100]      <1>     cmp     eax, [FAT_BuffSector] ; 0, 3, 6, 9 ...
0000C07F 75B7                <1>     jne     short load_FAT_sectors1
0000C081 8B03                <1>     mov     eax, [ebx]
0000C083 25FFFFFF0F          <1>     and     eax, 0FFFFFFFh ; 28 bit Cluster
0000C088 3DF7FFFF0F          <1>     cmp     eax, 0FFFFFF7h
0000C08D 7296                <1>     jb      short loc_pass_gnc_FAT32_eoc_check
                             <1>     ; eax >= FFFFFF7h (cluster 0002h to FFFFFF6h is valid)
0000C08F EB92                <1>     jmp     short loc_pass_gnc_FAT16_eoc_check_xor_eax
                             <1>
                             <1> load_FAT_sectors3:
0000C091 8B4E2A              <1>     mov     ecx, [esi+LD_BPB+BPB_FATSz32]
                             <1> load_FAT_sectors4:
0000C094 29D9                <1>     sub     ecx, ebx ; [FAT_BuffSector]
                             <1>     ; 25/07/2022
0000C096 FEC2                <1>     inc     dl
                             <1>     ; edx = 3
                             <1>         ;cmp       ecx, 3
0000C098 39D1                <1>         cmp ecx, edx ; 3
0000C09A 7602                <1>     jna     short load_FAT_sectors5
                             <1>     ;mov    ecx, 3
                             <1>     ; 25/07/2022
0000C09C 89D1                <1>     mov     ecx, edx ; 3
                             <1> load_FAT_sectors5:
0000C09E BB001C0900          <1>     mov     ebx, FAT_Buffer
0000C0A3 E84B590000          <1>     call    disk_read
0000C0A8 730C                <1>     jnc     short load_FAT_sectors_ok
                             <1>     ; 15/10/2016 (15h -> 17)
                             <1>     ; 23/03/2016 (15h)
0000C0AA B811000000          <1>     mov     eax, 17 ; Drive not ready or read error
                             <1>     ;mov    byte [FAT_BuffValidData], 0
                             <1>     ; 25/07/2022
0000C0AF 8825[8A7E0100]      <1>     mov     byte [FAT_BuffValidData], ah ; 0
0000C0B5 C3                  <1>     retn
                             <1> load_FAT_sectors_ok:
0000C0B6 C605[8A7E0100]01    <1>     mov     byte [FAT_BuffValidData], 1
0000C0BD A1[867E0100]        <1>     mov     eax, [FAT_CurrentCluster]
0000C0C2 E9C7FEFFFF          <1>         jmp     check_next_cluster_fat_type
                             <1>
                             <1> load_FAT_root_directory:
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 23/10/2016
                             <1>     ; 15/10/2016
                             <1>     ; 07/02/2016
                             <1>     ; 02/02/2016
                             <1>     ; 01/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 21/05/2011
                             <1>     ; 22/08/2009
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Logical DOS Drive Description Table
                             <1>     ; OUTPUT ->
                             <1>     ;       cf = 1 -> Root directory could not be loaded
                             <1>     ;           EAX > 0 -> Error number
                             <1>     ;       cf = 0 -> EAX = 0
                             <1>     ;       ECX = Directory buffer size in sectors (CL)
```

```
                                <1>     ;       EBX = Directory buffer address
                                <1>     ;       NOTE: DirBuffer_Size is in bytes ! (word)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
                                <1>     ; NOTE: Only for FAT12 and FAT16 file systems !
                                <1>     ; (FAT32 fs root dir must be loaded as sub directory)
                                <1>
0000C0C7 8A1E                   <1>     mov     bl, [esi+LD_Name]
0000C0C9 8A7E03                 <1>     mov     bh, [esi+LD_FATType]
                                <1>
                                <1>     ;mov    [DirBuff_DRV], bl
                                <1>     ;mov    [DirBuff_FATType], bh
0000C0CC 66891D[9B7E0100]       <1>     mov     [DirBuff_DRV], bx
                                <1>
                                <1>     ;cmp    bh, 2
                                <1>     ;ja     short load_FAT32_root_dir0 ; FAT32 root dir
                                <1>
                                <1> load_FAT_root_dir0: ; 23/10/2016
0000C0D3 0FB75617               <1>     movzx   edx, word [esi+LD_BPB+RootDirEnts]
                                <1>
                                <1>     ;or     dx, dx ; 0 for FAT32 file systems
                                <1>     ;jz     short load_FAT32_root_dir0 ; FAT32 root dir
                                <1>
                                <1>     ; 25/07/2022
0000C0D7 89D0                   <1>     mov     eax, edx
0000C0D9 6681FA0002             <1>     cmp     dx, 512 ; Number of Root Dir Entries
0000C0DE 740B                   <1>     je      short lrd_mov_ecx_32
                                <1>     ;mov    eax, edx ; 25/07/2022
                                <1>     ; 23/10/2016
0000C0E0 89C1                   <1>     mov     ecx, eax
0000C0E2 6683C10F               <1>     add     cx, 15 ; round up
                                <1>     ;shr    cx, 4  ; 16 entries per sector (512/32)
                                <1>     ; 25/07/2022
0000C0E6 C1E904                 <1>     shr     ecx, 4
                                <1>     ; ecx = Root directory size in sectors
                                <1>     ;;shl   ax, 5 ; Root directory size in bytes
                                <1>     ; 25/07/2022
                                <1>     ;shl    eax, 5
                                <1>     ;;dec   dx    ; Last entry number of root dir
                                <1>     ;dec    edx
                                <1>     ; cx = Dir Buffer sector count
0000C0E9 EB04                   <1>     jmp     short lrd_check_dir_buffer
                                <1>
                                <1> lrd_mov_ecx_32:
                                <1>     ;mov    ecx, 32
                                <1>     ; 25/07/2022
0000C0EB 29C9                   <1>     sub     ecx, ecx
0000C0ED B120                   <1>     mov     cl, 32
                                <1>     ;dec    dx ; 511
                                <1>     ;mov    ax, 32*512
                                <1>
                                <1> lrd_check_dir_buffer:
                                <1>     ; 25/07/2022
0000C0EF 4A                     <1>     dec     edx ; root dir entries - 1
0000C0F0 C1E005                 <1>     shl     eax, 5 ; * 32
                                <1>     ;
0000C0F3 29DB                   <1>     sub     ebx, ebx ; 0
0000C0F5 881D[9D7E0100]         <1>     mov     [DirBuff_ValidData], bl ; 0
0000C0FB 668915[A07E0100]       <1>     mov     [DirBuff_LastEntry], dx
0000C102 891D[A27E0100]         <1>     mov     [DirBuff_Cluster], ebx ; 0
0000C108 66A3[A67E0100]         <1>     mov     [DirBuffer_Size], ax
                                <1>
0000C10E 8B4664                 <1>     mov     eax, [esi+LD_ROOTBegin]
                                <1> read_directory:
0000C111 BB00000800             <1>     mov     ebx, Directory_Buffer
0000C116 51                     <1>     push    ecx ; Directory buffer sector count
0000C117 53                     <1>     push    ebx
0000C118 E8D6580000             <1>     call    disk_read
0000C11D 5B                     <1>     pop     ebx
0000C11E 720B                   <1>     jc      short load_DirBuff_error
                                <1>
                                <1> validate_DirBuff_and_return:
0000C120 59                     <1>     pop     ecx ; Number of loaded sectors
0000C121 C605[9D7E0100]01       <1>     mov     byte [DirBuff_ValidData], 1
0000C128 31C0                   <1>     xor     eax, eax ; 0 = no error
0000C12A C3                     <1>     retn
                                <1>
                                <1> load_DirBuff_error:
0000C12B 89C8                   <1>     mov     eax, ecx ; remaining sectors
0000C12D 59                     <1>     pop     ecx ; sector count
0000C12E 29C1                   <1>     sub     ecx, eax ; Number of loaded sectors
                                <1>     ; 15/10/2016 (15h -> 17)
                                <1>     ;mov    eax, 17 ; DRV NOT READY OR READ ERROR !
                                <1>     ; 25/07/2022
                                <1>     ;sub    eax, eax
0000C130 B011                   <1>     mov     al, 17
0000C132 F9                     <1>     stc
0000C133 C3                     <1>         retn
                                <1>
                                <1> load_FAT32_root_directory:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 02/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical DOS Drive Description Table
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 1 -> Root directory could not be loaded
                                <1>     ;           EAX > 0 -> Error number
                                <1>     ;       cf = 0 -> EAX = 0
                                <1>     ;       ECX = Directory buffer size in sectors (CL)
                                <1>     ;       EBX = Directory buffer address
                                <1>     ;       NOTE: DirBuffer_Size is in bytes ! (word)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
0000C134 8A1E                   <1>     mov     bl, [esi+LD_Name]
0000C136 8A7E03                 <1>     mov     bh, [esi+LD_FATType]
                                <1>
                                <1>     ;mov    [DirBuff_DRV], bl
                                <1>     ;mov    [DirBuff_FATType], bh
0000C139 66891D[9B7E0100]       <1>     mov     [DirBuff_DRV], bx
                                <1>
                                <1> load_FAT32_root_dir0:
0000C140 8B4632                 <1>     mov     eax, [esi+LD_BPB+FAT32_RootFClust]
0000C143 EB0C                   <1>     jmp     short load_FAT_sub_dir0
                                <1>
                                <1> load_FAT_sub_directory:
                                <1>     ; 19/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 01/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 05/07/2011
                                <1>     ; 23/08/2009
                                <1>     ;
```

```
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical DOS Drive Description Table
                                <1>     ;       EAX = Cluster Number
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 1 -> Sub directory could not be loaded
                                <1>     ;           EAX > 0 -> Error number
                                <1>     ;       cf = 0 -> EAX = 0
                                <1>     ;       ECX = Directory buffer size in sectors (CL)
                                <1>     ;       EBX = Directory buffer address
                                <1>     ;
                                <1>     ;       NOTE: DirBuffer_Size is in bytes ! (word)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
0000C145 8A1E                   <1>     mov     bl, [esi+LD_Name]
0000C147 8A7E03                 <1>     mov     bh, [esi+LD_FATType]
                                <1>
                                <1>     ;mov    [DirBuff_DRV], bl
                                <1>     ;mov    [DirBuff_FATType], bh
0000C14A 66891D[9B7E0100]       <1>     mov     [DirBuff_DRV], bx
                                <1>
                                <1> load_FAT_sub_dir0:
0000C151 0FB64E13               <1>     movzx   ecx, byte [esi+LD_BPB+SecPerClust]
                                <1>
0000C155 882D[9D7E0100]         <1>     mov     [DirBuff_ValidData], ch ; 0
0000C15B A3[A27E0100]           <1>     mov     [DirBuff_Cluster], eax
                                <1>
                                <1>     ;movzx  eax, word [esi+LD_BPB+BytesPerSec] ; 512
                                <1>     ;mul    ecx
                                <1>     ;shr    eax, 5 ; directory entry count (dir size / 32)
                                <1>     ; 19/12/2025
0000C160 89C8                   <1>     mov     eax, ecx
0000C162 C1E004                 <1>     shl     eax, 4 ; 512/32 (16 entries per sector)
                                <1>
                                <1>     ;dec    ax ; last entry
                                <1>     ; 25/07/2022
0000C165 48                     <1>     dec     eax
0000C166 66A3[A07E0100]         <1>     mov     [DirBuff_LastEntry], ax
                                <1>
0000C16C A1[A27E0100]           <1>     mov     eax, [DirBuff_Cluster]
0000C171 83E802                 <1>     sub     eax, 2
0000C174 F7E1                   <1>     mul     ecx
0000C176 034668                 <1>     add     eax, [esi+LD_DATABegin]
                                <1>     ; ecx = sectors per cluster (dir buffer size <= 128 sectors)
0000C179 EB96                   <1>     jmp     short read_directory
                                <1>
                                <1> ; DRV_FS.ASM
                                <1>
                                <1> ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>
                                <1> ; 19/12/2025
                                <1> ;load_current_FS_directory:
                                <1> ;   ;retn
                                <1> ;load_FS_root_directory:
                                <1> ;   ;retn
                                <1> ;load_FS_sub_directory:
                                <1> ;   retn
                                <1>
                                <1> read_cluster:
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
                                <1>     ; 18/03/2016
                                <1>     ; 16/03/2016
                                <1>     ; 17/02/2016
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Cluster Number (Sector index for SINGLIX FS)
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ;       EBX = Cluster (File R/W) Buffer address (max. 64KB)
                                <1>     ;       Only for SINGLIX FS:
                                <1>     ;       EDX = File Number (The 1st FDT address)
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 1 -> Cluster can not be loaded at the buffer
                                <1>     ;           EAX > 0 -> Error number
                                <1>     ;       cf = 0 -> Cluster has been loaded at the buffer
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
0000C17B 0FB64E13               <1>     movzx   ecx, byte [esi+LD_BPB+BPB_SecPerClust]
                                <1>     ; CL = 1 = [esi+LD_FS_Reserved2] ; SectPerClust for Singlix FS
                                <1>
                                <1> read_file_sectors: ; 16/03/2016
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     ;cmp    byte [esi+LD_FATType], 0
                                <1>     ; 25/07/2022
                                <1>     cmp     [esi+LD_FATType], ch ; 0
                                <1>     jna     short read_fs_cluster
                                <1> %endif
                                <1>
                                <1> read_fat_file_sectors: ; 18/03/2016
0000C17F 83E802                 <1>     sub     eax, 2 ; Beginning cluster number is always 2
0000C182 0FB65613               <1>     movzx   edx, byte [esi+LD_BPB+BPB_SecPerClust] ; 18/03/2016
0000C186 F7E2                   <1>     mul     edx
0000C188 034668                 <1>     add     eax, [esi+LD_DATABegin] ; absolute address of the cluster
                                <1>
                                <1>     ; EAX = Disk sector address
                                <1>     ; ECX = Sector count
                                <1>     ; EBX = Buffer address
                                <1>     ; (EDX = 0)
                                <1>     ; ESI = Logical DOS drive description table address
                                <1>
0000C18B E863580000             <1>     call    disk_read
0000C190 7306                   <1>     jnc     short rclust_retn
                                <1>
                                <1>     ; 15/10/2016 (15h -> 17)
0000C192 B811000000             <1>     mov     eax, 17 ; Drive not ready or read error !
0000C197 C3                     <1>     retn
                                <1>
                                <1> rclust_retn:
0000C198 29C0                   <1>     sub     eax, eax ; 0
0000C19A C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>
                                <1> read_fs_cluster:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; Singlix FS
                                <1>
```

```
                                <1>     ; EAX = Cluster number is sector index number of the file (eax)
                                <1>
                                <1>     ; EDX = File number is the first File Descriptor Table address
                                <1>     ;       of the file. (Absolute address of the FDT).
                                <1>
                                <1>     ; eax = sector index (0 for the first sector)
                                <1>     ; edx = FDT0 address
                                <1>             ; 64 KB buffer = 128 sectors (limit)
                                <1>     ;mov    ecx, 128 ; maximum count of sectors (before eof)
                                <1>     ; 25/07/2022
                                <1>     sub     ecx, ecx
                                <1>     mov     cl, 128
                                <1>     ;call   read_fs_sectors
                                <1>     ;retn
                                <1>     ;jmp    short read_fs_sectors
                                <1>
                                <1> read_fs_sectors:
                                <1>     ; 15/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     stc
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> get_first_free_cluster:
                                <1>     ; 21/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 02/03/2016
                                <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 26/10/2010 (DRV_FAT.ASM, 'proc_get_first_free_cluster')
                                <1>     ; 10/07/2010
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 1 -> Error code in AL (EAX)
                                <1>     ;       cf = 0 ->
                                <1>     ;         EAX = Cluster number
                                <1>     ;         If EAX = FFFFFFFFh -> no free space
                                <1>     ;       If the drive has FAT32 fs:
                                <1>     ;         EBX = FAT32 FSI sector buffer address (if > 0)
                                <1>     ;
                                <1>     ; (Modified registers: eax, ebx, ecx, edx)
                                <1>     ;
                                <1>
0000C19B 8B4678                 <1>     mov     eax, [esi+LD_Clusters]
0000C19E 40                     <1>     inc     eax ; add eax, 1
0000C19F A3[28810100]           <1>     mov     [gffc_last_free_cluster], eax
                                <1>
0000C1A4 31DB                   <1>     xor     ebx, ebx ; 0 ; 02/03/2016
                                <1>
0000C1A6 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
0000C1AA 760D                   <1>     jna     short loc_gffc_get_first_fat_free_cluster0
                                <1>
                                <1> ; 21/12/2025 - TRDOS 386 v2.0.10
                                <1> %if 1
                                <1>     ; FAT32 - FSINFO - first free cluster
0000C1AC 31D2                   <1>     xor     edx, edx
0000C1AE 4A                     <1>     dec     edx ; -1
0000C1AF 87563E                 <1>     xchg    edx, [esi+LD_BPB+BPB_Reserved+4]
                                <1>
                                <1> %else
                                <1> loc_gffc_get_first_fat32_free_cluster:
                                <1>     ; 02/03/2016
                                <1>     call    get_fat32_fsinfo_sector_parms
                                <1>     jc      short loc_gffc_get_first_fat_free_cluster0
                                <1> %endif
                                <1>
                                <1> loc_gffc_check_fsinfo_parms:
                                <1>     ;;mov   ebx, DOSBootSectorBuff
                                <1>     ;cmp    dword [ebx], 41615252h
                                <1>     ;jne    short loc_gffc_fat32_fsinfo_err
                                <1>     ;cmp    dword [ebx+484], 61417272h
                                <1>     ;jne    short loc_gffc_fat32_fsinfo_err
                                <1>     ;mov    eax, [ebx+492] ; FSI_Next_Free
                                <1>     ;EAX = First free cluster
                                <1>     ;(from FAT32 FSInfo sector)
0000C1B2 89D0                   <1>     mov     eax, edx ; FSI_Next_Free (First Free Cluster)
0000C1B4 83F8FF                 <1>     cmp     eax, 0FFFFFFFFh ; invalid (unknown) !
0000C1B7 7204                   <1>     jb      short loc_gffc_get_first_fat_free_cluster1
                                <1>
                                <1>     ; Start from the 1st cluster of the FAT(32) file system
                                <1> loc_gffc_get_first_fat_free_cluster0:
                                <1>     ;mov    eax, 2
                                <1>     ; 25/07/2022
0000C1B9 29C0                   <1>     sub     eax, eax
0000C1BB B002                   <1>     mov     al, 2
                                <1>     ;xor    edx, edx
                                <1>
                                <1> loc_gffc_get_first_fat_free_cluster1:
0000C1BD 53                     <1>     push    ebx ; 02/03/2016
                                <1>
                                <1> loc_gffc_get_first_fat_free_cluster2:
0000C1BE A3[24810100]           <1>     mov     [gffc_first_free_cluster], eax
0000C1C3 A3[20810100]           <1>     mov     [gffc_next_free_cluster], eax
                                <1>
                                <1>     ; EBX = FAT32 FSINFO sector buffer address
                                <1>     ; (EBX = 0, if the drive has not got FAT32 fs or
                                <1>     ; FAT32 FSINFO sector buffer is invalid.)
                                <1>
                                <1> loc_gffc_get_first_fat_free_cluster3:
0000C1C8 E8BCFDFFFF             <1>     call    get_next_cluster
0000C1CD 7307                   <1>     jnc     short loc_gffc_get_first_fat_free_cluster4
0000C1CF 09C0                   <1>     or      eax, eax
0000C1D1 740B                   <1>     jz      short loc_gffc_first_free_fat_cluster_next
0000C1D3 5B                     <1>     pop     ebx ; 02/03/2016
0000C1D4 F5                     <1>     cmc     ; stc
0000C1D5 C3                     <1>     retn
                                <1>
                                <1> loc_gffc_get_first_fat_free_cluster4:
0000C1D6 21C0                   <1>     and     eax, eax ; next cluster value
0000C1D8 7504                   <1>     jnz     short loc_gffc_first_free_fat_cluster_next
0000C1DA 89C8                   <1>     mov     eax, ecx ; current (previous cluster) value
0000C1DC EB22                   <1>     jmp     short loc_gffc_check_for_set
                                <1>
                                <1> loc_gffc_first_free_fat_cluster_next:
0000C1DE A1[20810100]           <1>     mov     eax, [gffc_next_free_cluster]
0000C1E3 3B05[28810100]         <1>     cmp     eax, [gffc_last_free_cluster]
0000C1E9 7308                   <1>     jnb     short retn_stc_from_get_first_free_cluster
                                <1> pass_gffc_last_cluster_eax_check:
0000C1EB 40                     <1>     inc     eax ; add eax, 1
0000C1EC A3[20810100]           <1>     mov     [gffc_next_free_cluster], eax
0000C1F1 EBD5                   <1>     jmp     short loc_gffc_get_first_fat_free_cluster3
                                <1>
                                <1> retn_stc_from_get_first_free_cluster:
0000C1F3 A1[24810100]           <1>     mov     eax, [gffc_first_free_cluster]
```

```
0000C1F8 83F802              <1>     cmp     eax, 2
0000C1FB 7712                <1>     ja      short loc_gffc_check_previous_clusters
0000C1FD 29C0                <1>     sub     eax, eax
0000C1FF 48                  <1>     dec     eax ; FFFFFFFFh
                             <1>
                             <1> loc_gffc_check_for_set:
                             <1>     ; 02/03/2016
0000C200 5B                  <1>     pop     ebx
                             <1>
                             <1>     ; 21/12/2025
                             <1>     ;;;
0000C201 807E0302            <1>     cmp     byte [esi+LD_FATType], 2
0000C205 7603                <1>     jna     short loc_gffc_check_for_set_@
                             <1>
0000C207 89463E              <1>     mov     [esi+LD_BPB+BPB_Reserved+4], eax
                             <1>
                             <1> loc_gffc_check_for_set_@:
                             <1>     ;;;
                             <1>
                             <1>     ; EBX = FAT32 FSINFO sector buffer address
                             <1>     ; (EBX = 0, if the drive has not got FAT32 fs or
                             <1>     ; FAT32 FSINFO sector buffer is invalid.)
                             <1>
0000C20A 09DB                <1>     or      ebx, ebx
0000C20C 750D                <1>     jnz     short loc_gffc_set_ffree_fat32_cluster
                             <1>
                             <1>     ;cmp    byte [esi+LD_FATType], 3
                             <1>     ;jnb    short loc_gffc_set_ffree_fat32_cluster
                             <1>
                             <1>     ;xor    ebx, ebx ; 0
                             <1>
                             <1> loc_gffc_retn:
0000C20E C3                  <1>     retn
                             <1>
                             <1> loc_gffc_check_previous_clusters:
0000C20F 48                  <1>     dec     eax ; sub eax, 1
0000C210 A3[28810100]        <1>     mov     [gffc_last_free_cluster], eax
                             <1>     ;mov    eax, 2
                             <1>     ; 25/07/2022
0000C215 31C0                <1>     xor     eax, eax
0000C217 B002                <1>     mov     al, 2
                             <1>     ; eax = 2
                             <1>     ;xor    edx, edx
0000C219 EBA3                <1>     jmp     short loc_gffc_get_first_fat_free_cluster2
                             <1>
                             <1> loc_gffc_set_ffree_fat32_cluster:
                             <1>     ;call   set_first_free_cluster
                             <1>     ;retn
                             <1>     ;jmp    short set_first_free_cluster
                             <1>
                             <1> set_first_free_cluster:
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 15/10/2016
                             <1>     ; 23/03/2016
                             <1>     ; 02/03/2016
                             <1>     ; 29/02/2016
                             <1>     ; 26/02/2016
                             <1>     ; 21/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 21/08/2011 (DRV_FAT.ASM, 'proc_set_first_free_cluster')
                             <1>     ; 11/07/2010
                             <1>     ; INPUT ->
                             <1>     ;       ESI = Logical DOS Drive Description Table address
                             <1>     ;       EAX = First free cluster
                             <1>     ;       EBX = FSINFO sector buffer address
                             <1>     ;       ;;If EBX > 0, it is FSINFO sector buffer address
                             <1>     ;       ;;EBX = 0, if FSINFO sector is not loaded
                             <1>     ; OUTPUT->
                             <1>     ;       ESI = Logical DOS Drive Description Table address
                             <1>     ;       If EBX > 0, it is FSINFO sector buffer address
                             <1>     ;       EBX = 0, if FSINFO sector could not be loaded
                             <1>     ;       CF = 1 -> Error code in AL (EAX)
                             <1>     ;       CF = 0 -> first free cluster is successfully updated
                             <1>
                             <1>     ;cmp    byte [esi+LD_FATType], 3
                             <1>     ;jb     short loc_sffc_invalid_drive
                             <1>
                             <1>     ; Save First Free Cluster value for 'update_cluster'
0000C21B 89463E              <1>     mov     [esi+LD_BPB+BPB_Reserved+4], eax ; First free Cluster
                             <1>
                             <1>     ;or     ebx, ebx
                             <1>     ;jnz    short loc_sffc_read_fsinfo_sector
                             <1>
0000C21E 813B52526141        <1>     cmp      dword [ebx], 41615252h
0000C224 753C                <1>     jne     short loc_sffc_read_fsinfo_sector
0000C226 81BBE4010000727241- <1>     cmp     dword [ebx+484], 61417272h
0000C22F 61                  <1>
0000C230 7530                <1>     jne     short loc_sffc_read_fsinfo_sector
                             <1>
0000C232 3B83EC010000        <1>     cmp     eax, [ebx+492]  ; FSI_Next_Free
0000C238 741E                <1>     je      short loc_sffc_retn
                             <1>
                             <1> loc_sffc_write_fsinfo_sector:
                             <1>     ; EBX = FSINFO sector buffer
                             <1>     ; [CFS_FAT32FSINFOSEC] is set in 'get_fat32_fsinfo_sector_parms'
0000C23A 8983EC010000        <1>     mov     [ebx+492], eax
0000C240 A1[38810100]        <1>     mov     eax, [CFS_FAT32FSINFOSEC]
                             <1>     ;mov    ecx, 1
                             <1>     ; 25/07/2022
0000C245 31C9                <1>     xor     ecx, ecx
0000C247 FEC1                <1>     inc     cl
                             <1>     ; ecx = 1
0000C249 53                  <1>     push    ebx
0000C24A E895570000          <1>     call    disk_write
0000C24F 7208                <1>     jc       short loc_sffc_read_fsinfo_sector_err1
0000C251 5B                  <1>     pop     ebx
                             <1>
0000C252 8B83EC010000        <1>     mov     eax, [ebx+492] ; First (Next) Free Cluster
                             <1>
                             <1> loc_sffc_retn:
0000C258 C3                  <1>     retn
                             <1>
                             <1> ;loc_sffc_invalid_drive:
                             <1> ;   mov     eax, 0Fh ; MSDOS Error : Invalid drive
                             <1> ;   push    edx
                             <1>
                             <1> loc_sffc_read_fsinfo_sector_err1:
                             <1>     ; 25/07/2022
                             <1>     ;mov    ebx, 0
                             <1>     ; 15/10/2016 (1Dh -> 18)
                             <1>     ; 23/03/2016 (1Dh)
                             <1>     ;mov    eax, 18 ; Drive not ready or write error
0000C259 31C0                <1>     xor     eax, eax
0000C25B 89C3                <1>     mov     ebx, eax ; 0
0000C25D B012                <1>     mov     al, 18
```

```
0000C25F F9                     <1>     stc
                                <1> loc_sffc_read_fsinfo_sector_err2:
0000C260 5A                     <1>     pop     edx
0000C261 C3                     <1>     retn
                                <1>
                                <1> loc_sffc_read_fsinfo_sector:
0000C262 50                     <1>     push    eax
                                <1>
0000C263 E84F050000             <1>     call    get_fat32_fsinfo_sector_parms
0000C268 72F6                   <1>     jc      short loc_sffc_read_fsinfo_sector_err2
                                <1>
0000C26A 58                     <1>     pop     eax
                                <1>     ; EDX = First (Next) Free Cluster value from FSINFO sector
                                <1>     ; EAX = First Free Cluster value from 'get_next_cluster'
                                <1>     ; (edx = old value)
0000C26B 39D0                   <1>     cmp     eax, edx ; First free Cluster (eax = new value)
0000C26D 75CB                   <1>     jne     short loc_sffc_write_fsinfo_sector
                                <1>
0000C26F C3                     <1>     retn
                                <1>
                                <1> update_cluster:
                                <1>     ; 31/08/2024
                                <1>     ; 29/08/2024
                                <1>     ; 26/08/2024
                                <1>     ; 24/08/2024 (TRDOS 386 Kernel v2.0.9)
                                <1>     ; 07/08/2022
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 23/10/2016
                                <1>     ; 23/03/2016
                                <1>     ; 02/03/2016
                                <1>     ; 01/03/2016
                                <1>     ; 29/02/2016
                                <1>     ; 27/02/2016
                                <1>     ; 26/02/2016
                                <1>     ; 22/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 11/08/2011
                                <1>     ; 09/02/2005
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Cluster Number
                                <1>     ;       ECX = New Cluster Value
                                <1>     ;       ESI = Logical Dos Drive Parameters Table
                                <1>     ;
                                <1>     ;       /// dword [FAT_ClusterCounter] ///
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 0 -> No Error, EAX is valid
                                <1>     ;       cf = 1 & EAX = 0 -> End Of Cluster Chain
                                <1>     ;       cf = 1 & EAX > 0 -> Error
                                <1>     ;               (ECX -> any value)
                                <1>     ;       EAX = Next Cluster
                                <1>     ;       ECX = New Cluster Value
                                <1>     ;
                                <1>     ;       /// [FAT_ClusterCounter] is updated,
                                <1>     ;       /// decreased when a free cluster is assigned,
                                <1>     ;       /// increased if an assigned cluster is freed.
                                <1>     ;
                                <1>     ;
                                <1>     ; (Modified registers: EAX, EBX, -ECX-, EDX)
                                <1>
0000C270 A3[867E0100]           <1>     mov     [FAT_CurrentCluster], eax
0000C275 890D[2C810100]         <1>     mov     [ClusterValue], ecx
                                <1>
                                <1> loc_update_cluster_check_fat_buffer:
0000C27B 8A1E                   <1>     mov     bl, [esi+LD_Name]
0000C27D 381D[8B7E0100]         <1>     cmp     [FAT_BuffDrvName], bl
0000C283 7418                   <1>     je      short loc_update_cluster_check_fat_type
0000C285 803D[8A7E0100]02       <1>     cmp     byte [FAT_BuffValidData], 2
                                <1>     ;je     loc_uc_save_fat_buffer
                                <1>     ; 25/07/2022
0000C28C 7502                   <1>     jne     short loc_uc_reset_fat_buffer_validation
0000C28E EB66                   <1>     jmp     loc_uc_save_fat_buffer
                                <1>
                                <1> loc_uc_reset_fat_buffer_validation:
0000C290 C605[8A7E0100]00       <1>     mov     byte [FAT_BuffValidData], 0
                                <1>
                                <1> loc_uc_check_fat_type_reset_drvname:
0000C297 881D[8B7E0100]         <1>     mov     [FAT_BuffDrvName], bl
                                <1>
                                <1> loc_update_cluster_check_fat_type:
0000C29D 29D2                   <1>     sub     edx, edx ; 26/02/2016
0000C29F 8A5E03                 <1>     mov     bl, [esi+LD_FATType]
0000C2A2 83F802                 <1>     cmp     eax, 2
0000C2A5 7218                   <1>         jb  short update_cluster_inv_data
                                <1>
                                <1>     ;;;
                                <1>     ; 24/08/2024
                                <1>     ; edx = 0 ; 24/08/2024
0000C2A7 8B4E78                 <1>     mov     ecx, [esi+LD_Clusters]
0000C2AA 41                     <1>     inc     ecx
0000C2AB 890D[967E0100]         <1>     mov     [LastCluster], ecx
                                <1>     ;
0000C2B1 39C8                   <1>     cmp     eax, ecx
                                <1>     ;ja     short return_uc_fat32_stc ; 25/07/2022
                                <1>     ; edx = 0 ; (must be -1 or > 0 after here)
0000C2B3 7717                   <1>     ja      short return_uc_fat_stc
                                <1>     ;;;
                                <1>
0000C2B5 80FB02                 <1>     cmp     bl, 2
                                <1>         ;ja update_fat32_cluster
                                <1>     ; 25/07/2022
0000C2B8 7608                   <1>     jna     short loc_uc_check_fat_type_1
0000C2BA E97B010000             <1>     jmp     update_fat32_cluster
                                <1>
                                <1> ; 24/08/2024
                                <1> ;loc_uc_check_fat_type_1:
                                <1> ;   ;cmp    bl, 1
                                <1> ;   ;jb     short update_cluster_inv_data
                                <1> ;   mov     ecx, [esi+LD_Clusters]
                                <1> ;   inc     ecx
                                <1> ;   mov     [LastCluster], ecx
                                <1> ;   cmp     eax, ecx ; dword [LastCluster]
                                <1> ;   ;ja     return_uc_fat_stc
                                <1> ;   ; 25/07/2022
                                <1> ;   jna     short loc_uc_check_fat_type_2
                                <1> ;   ; 24/08/2024
                                <1> ;   ; edx = 0 ; (must be -1 or > 0 after here)
                                <1> ;   jmp     return_uc_fat_stc
                                <1>
                                <1>     ; 25/07/2022
                                <1> update_cluster_inv_data:
                                <1>     ;;mov   eax, 0Dh
                                <1>     ;mov    al, 0Dh  ; Invalid Data
                                <1>     ; 31/08/2024
0000C2BF B01D                   <1>     mov     al, ERR_INV_DATA ; 29 ; Invalid Data
```

```
0000C2C1 C3                 <1>     retn
                            <1>
                            <1>     ; 24/08/2024
                            <1> loc_uc_check_fat_type_1:
                            <1> loc_uc_check_fat_type_2:
                            <1>     ; TRDOS v1 has a FATal bug here !
                            <1>             ; or bl, bl ; cmp bl, 0
                            <1>             ; jz short update_fat12_cluster
                            <1>     ; !! It would destroy FAT12 floppy disk fs here !!
                            <1>     ; ('A:' disks of TRDOS v1 operating system project
                            <1>     ; had 'singlix fs', so, I could not differ this mistake
                            <1>     ; on a drive 'A:')
0000C2C2 80FB01             <1>     cmp     bl, 1 ; correct comparison is this !
                            <1>     ;jna    update_fat12_cluster
                            <1>     ; 25/07/2022
0000C2C5 770B               <1>     ja      short update_fat16_cluster
0000C2C7 E9CF000000         <1>     jmp     update_fat12_cluster
                            <1>
                            <1>     ; 24/08/2024
                            <1> ;return_uc_fat16_stc:
                            <1>     ; 25/07/2022
                            <1> return_uc_fat_stc:
                            <1>     ; 24/08/2024
                            <1>     ; edx = 0
0000C2CC 4A                 <1>     dec     edx
                            <1>     ; edx = -1 ; 0FFFFFFFFh
                            <1>     ; ecx > 0
                            <1> return_uc_fat16_stc:      ; 24/08/2024
                            <1> loc_fat_buffer_stc_0:     ; 24/08/2024
                            <1>     ; 01/03/2016
0000C2CD 31C0               <1>     xor     eax, eax
0000C2CF F9                 <1>     stc
0000C2D0 EB70               <1>     jmp     short loc_fat_buffer_stc_1
                            <1>
                            <1> update_fat16_cluster:
                            <1> pass_uc_fat16_errc:
                            <1>     ;sub    edx, edx
                            <1>     ; edx = 0
0000C2D2 BB00030000         <1>     mov     ebx, 300h ; 768
0000C2D7 F7F3               <1>     div     ebx
                            <1>     ; EAX = Count of 3 FAT sectors
                            <1>     ; DX = Cluster index in FAT buffer
                            <1>     ;mov    bx, dx
                            <1>     ; 25/07/2022
0000C2D9 89D3               <1>     mov     ebx, edx
                            <1>     ;shl    bx, 1 ; Multiply by 2
                            <1>     ; 25/07/2022
0000C2DB D1E3               <1>     shl     ebx, 1
0000C2DD 66BA0300           <1>     mov     dx, 3
0000C2E1 F7E2               <1>     mul     edx
                            <1>     ; EAX = FAT Sector
                            <1>     ; EDX = 0
                            <1>     ; EBX = Byte offset in FAT buffer
0000C2E3 8A0D[8A7E0100]     <1>     mov     cl, [FAT_BuffValidData]
0000C2E9 80F902             <1>     cmp     cl, 2
0000C2EC 7518               <1>     jne     short loc_uc_check_fat16_buff_sector_load
                            <1>
                            <1> loc_uc_check_fat16_buff_sector_save:
0000C2EE 3B05[8E7E0100]     <1>     cmp     eax, [FAT_BuffSector]
                            <1>     ;jne    short loc_uc_save_fat_buffer
                            <1>     ;jmp    short loc_update_fat16_cell
                            <1>     ; 07/08/2022
0000C2F4 741D               <1>     je      short loc_update_fat16_cell
                            <1>     ;jmp    loc_uc_save_fat_buffer
                            <1>
                            <1>     ; 07/08/2022
                            <1> loc_uc_save_fat_buffer:
                            <1>     ; byte [FAT_BuffValidData] = 2
0000C2F6 E8E9010000         <1>     call    save_fat_buffer
0000C2FB 7267               <1>         jc  short loc_fat_sectors_rw_error2
                            <1>     ;mov    byte [FAT_BuffValidData], 1
0000C2FD A1[867E0100]       <1>     mov     eax, [FAT_CurrentCluster]
                            <1>     ;mov    ecx, [ClusterValue]
                            <1>     ;jmp    short loc_update_cluster_check_fat_buffer
0000C302 8A1E               <1>     mov     bl, [esi+LD_Name] ; 01/03/2016
0000C304 EB8A               <1>         jmp loc_uc_reset_fat_buffer_validation
                            <1>
                            <1> loc_uc_check_fat16_buff_sector_load:
0000C306 80F901             <1>     cmp     cl, 1 ; byte [FAT_BuffValidData]
0000C309 7560               <1>         jne short loc_uc_load_fat_sectors
0000C30B 3B05[8E7E0100]     <1>     cmp     eax, [FAT_BuffSector]
0000C311 7558               <1>         jne short loc_uc_load_fat_sectors
                            <1>
                            <1> loc_update_fat16_cell:
                            <1> loc_update_fat16_buffer:
0000C313 81C3001C0900       <1>     add     ebx, FAT_Buffer ; 26/02/2016
                            <1>     ;movzx  eax, word [ebx]
0000C319 668B03             <1>     mov     ax, [ebx]
                            <1>     ; 01/03/2016
0000C31C 89C2               <1>     mov     edx, eax ; old value of the cluster
0000C31E A3[867E0100]       <1>     mov     [FAT_CurrentCluster], eax
0000C323 8B0D[2C810100]     <1>     mov     ecx, [ClusterValue] ; 32 bits
0000C329 66890B             <1>     mov     [ebx], cx ; 16 bits !
                            <1>
0000C32C C605[8A7E0100]02   <1>     mov     byte [FAT_BuffValidData], 2
                            <1>
0000C333 6683F802           <1>     cmp     ax, 2
0000C337 7294               <1>     jb      short return_uc_fat16_stc
0000C339 3B05[967E0100]     <1>     cmp     eax, [LastCluster]
0000C33F 778C               <1>     ja      short return_uc_fat16_stc
                            <1>
                            <1> loc_fat_buffer_updated:
                            <1>     ; 01/03/2016
0000C341 F8                 <1>     clc
                            <1> loc_fat_buffer_stc_1:
0000C342 9C                 <1>     pushf
0000C343 21C9               <1>     and     ecx, ecx
0000C345 7506               <1>     jnz     short loc_fat_buffer_updated_1
                            <1>
                            <1>     ; 01/03/2016
                            <1>     ; new value of the cluster = 0 (free)
                            <1>     ; increase free(d) cluster count
0000C347 FF05[927E0100]     <1>     inc     dword [FAT_ClusterCounter]
                            <1>
                            <1> loc_fat_buffer_updated_1: ; new value of the cluster > 0
0000C34D 09D2               <1>     or      edx, edx ; 02/03/2016
0000C34F 7506               <1>     jnz     short loc_fat_buffer_updated_2
                            <1>     ; old value of the cluster = 0 (it was free cluster)
                            <1>     ; decrease free(d) cluster count
0000C351 FF0D[927E0100]     <1>     dec     dword [FAT_ClusterCounter] ; it may be negative number
                            <1>
                            <1> loc_fat_buffer_updated_2:
0000C357 9D                 <1>     popf
0000C358 C3                 <1>     retn
```

```
                                <1>
                                <1>     ; 25/07/2022
                                <1> loc_fat_sectors_rw_error1:
                                <1>     ;mov    byte [FAT_BuffValidData], 0
                                <1>     ; 23/10/2016 (15h -> 17)
                                <1>     ; 23/03/2016
0000C359 B811000000             <1>     mov     eax, 17 ; Drive not ready or read error
0000C35E 8825[8A7E0100]         <1>     mov     [FAT_BuffValidData], ah ; 0
                                <1>
                                <1> loc_fat_sectors_rw_error2:
                                <1>     ;mov    eax, error code
                                <1>     ;mov    edx, 0
0000C364 8B0D[2C810100]         <1>     mov     ecx, [ClusterValue]
0000C36A C3                     <1>     retn
                                <1>
                                <1>     ; 25/07/2022
                                <1> loc_uc_load_fat_sectors:
0000C36B A3[8E7E0100]           <1>     mov     [FAT_BuffSector], eax
                                <1>
                                <1> load_uc_fat_sectors_zero:
0000C370 034660                 <1>     add     eax, [esi+LD_FATBegin]
0000C373 BB001C0900             <1>     mov     ebx, FAT_Buffer
0000C378 B903000000             <1>     mov     ecx, 3
0000C37D E871560000             <1>     call    disk_read
0000C382 72D5                   <1>     jc      short loc_fat_sectors_rw_error1
                                <1>
0000C384 C605[8A7E0100]01       <1>         mov     byte [FAT_BuffValidData], 1
0000C38B A1[867E0100]           <1>     mov     eax, [FAT_CurrentCluster]
0000C390 8B0D[2C810100]         <1>     mov     ecx, [ClusterValue]
0000C396 E902FFFFFF             <1>         jmp     loc_update_cluster_check_fat_type
                                <1>
                                <1> update_fat12_cluster:
                                <1> pass_uc_fat12_errc:
                                <1>     ;sub    edx, edx
0000C39B BB00040000             <1>     mov     ebx, 400h ; 1024
0000C3A0 F7F3                   <1>     div     ebx
                                <1>     ; EAX = Count of 3 FAT sectors
                                <1>     ; DX = Cluster index in FAT buffer
                                <1>     ;mov    cx, 3
                                <1>     ; 25/07/2022
0000C3A2 29C9                   <1>     sub     ecx, ecx
0000C3A4 B103                   <1>     mov     cl, 3
                                <1>     ; ecx = 3
                                <1>     ;mov    bx, ax
0000C3A6 89C3                   <1>     mov     ebx, eax
                                <1>     ;mov    ax, cx ; 3
0000C3A8 89C8                   <1>     mov     eax, ecx
                                <1>     ;mul    dx      ; Multiply by 3
0000C3AA F7E2                   <1>     mul     edx
                                <1>     ;shr    ax, 1  ; Divide by 2
0000C3AC D1E8                   <1>     shr     eax, 1
                                <1>     ;xchg   bx, ax
0000C3AE 93                     <1>     xchg    ebx, eax
                                <1>     ; EAX = Count of 3 FAT sectors
                                <1>     ; EBX = Byte Offset in FAT buffer
                                <1>     ;mul    cx  ; 3 * AX
0000C3AF F7E1                   <1>     mul     ecx ; 3 * EAX
                                <1>     ; EAX = FAT Beginning Sector
                                <1>     ; EDX = 0
0000C3B1 8A0D[8A7E0100]         <1>     mov     cl, [FAT_BuffValidData]
                                <1>     ; TRDOS v1 has a FATal bug here !
                                <1>     ; (it does not have 'cmp cl, 2' instruction here !
                                <1>     ;  while 'jne' is existing !)
0000C3B7 80F902                 <1>     cmp     cl, 2 ; 2 = dirty buffer (must be written to disk)
0000C3BA 750D                   <1>     jne     short loc_uc_check_fat12_buff_sector_load
                                <1>
                                <1> loc_uc_check_fat12_buff_sector_save:
0000C3BC 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
                                <1>         ;jne       short loc_uc_save_fat_buffer
                                <1>     ;jmp    short loc_update_fat12_cell
                                <1>     ; 07/08/2022
0000C3C2 7412                   <1>     je      short loc_update_fat12_cell
0000C3C4 E92DFFFFFF             <1>     jmp     loc_uc_save_fat_buffer
                                <1>
                                <1> loc_uc_check_fat12_buff_sector_load:
0000C3C9 80F901                 <1>     cmp     cl, 1 ; byte ptr [FAT_BuffValidData]
0000C3CC 759D                   <1>         jne     short loc_uc_load_fat_sectors
0000C3CE 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
0000C3D4 7595                   <1>     jne     short loc_uc_load_fat_sectors
                                <1>     ; 07/08/2022
                                <1>     ;je     short loc_update_fat12_cell
                                <1>     ;jmp    loc_uc_load_fat_sectors
                                <1>
                                <1> loc_update_fat12_cell:
0000C3D6 81C3001C0900           <1>     add     ebx, FAT_Buffer ; 26/02/2016
                                <1>     ;mov    cx, [FAT_CurrentCluster]
                                <1>     ; 25/07/2022
0000C3DC 8B0D[867E0100]         <1>     mov     ecx, [FAT_CurrentCluster]
                                <1>     ;shr    cx, 1
                                <1>     ; 25/07/2022
0000C3E2 D1E9                   <1>     shr     ecx, 1
0000C3E4 668B03                 <1>     mov     ax, [ebx]
                                <1>     ;mov    dx, ax
0000C3E7 89C2                   <1>     mov     edx, eax ; 25/07/2022
                                <1>     ; 24/08/2024 (*)
0000C3E9 8B0D[2C810100]         <1>     mov     ecx, [ClusterValue] ; 32 bits
0000C3EF 7336                   <1>     jnc     short uc_fat12_nc_even
                                <1>
                                <1>     ; 26/08/2024
                                <1>     ; ODD cluster number
                                <1>     ; eax = current value (before updated)
                                <1>     ; ecx = new value of the cluster
                                <1>     ; of the cluster
                                <1>     ; low 4 bit of al is high 4 bit
                                <1>     ; of the previous cluster
                                <1>     ;       (it must not be overwritten)
                                <1>
                                <1>     ;and    ax, 0Fh
                                <1>     ; 25/07/2022
0000C3F1 240F                   <1>     and     al, 0Fh
                                <1>     ; 24/08/2024 (*)
                                <1>     ;mov    ecx, [ClusterValue] ; 32 bits
                                <1>     ;shl    cx, 4
0000C3F3 C1E104                 <1>     shl     ecx, 4
                                <1>     ;or     cx, ax
0000C3F6 08C1                   <1>     or      cl, al ; 25/07/2022
                                <1>     ;mov    ax, dx
0000C3F8 89D0                   <1>     mov     eax, edx
0000C3FA 66890B                 <1>     mov     [ebx], cx  ; 16 bits !
                                <1>     ;shr    ax, 4 ; al(bit4..7)+ah(bit0..7)
                                <1>     ; 25/07/2022
0000C3FD C1E804                 <1>     shr     eax, 4
                                <1>
                                <1> update_fat12_buffer:
```

```
                                <1>     ;;;
                                <1>     ; 24/08/2024
0000C400 8B0D[2C810100]         <1>     mov     ecx, [ClusterValue]
                                <1>     ;;;
0000C406 A3[867E0100]           <1>     mov     [FAT_CurrentCluster], eax
0000C40B 89C2                   <1>     mov     edx, eax ; 01/03/2016
0000C40D C605[8A7E0100]02       <1>     mov     byte [FAT_BuffValidData], 2
0000C414 6683F802               <1>     cmp     ax, 2
0000C418 724A                   <1>         jb      short return_uc_fat12_stc
0000C41A 3B05[967E0100]         <1>     cmp     eax, [LastCluster]
0000C420 7742                   <1>         ja      short return_uc_fat12_stc
0000C422 E91AFFFFFF             <1>         jmp     loc_fat_buffer_updated
                                <1>
                                <1> uc_fat12_nc_even:
                                <1>     ; 26/08/2024
                                <1>     ; EVEN cluster number
                                <1>     ; eax = current value (before updated)
                                <1>     ; ecx = new value of the cluster
                                <1>     ; of the cluster
                                <1>     ; high 4 bit of ah is low 4 bit
                                <1>     ; of the next cluster (it must not be overwritten)
                                <1>
0000C427 662500F0               <1>     and     ax, 0F000h
                                <1>     ; 24/08/2024 (*)
                                <1>     ;mov    ecx, [ClusterValue] ; 32 bits
0000C42B 80E50F                 <1>     and     ch, 0Fh
                                <1>     ;or     cx, ax
                                <1>     ; 25/07/2022
0000C42E 09C1                   <1>     or      ecx, eax
                                <1>     ;mov    ax, dx
0000C430 89D0                   <1>     mov     eax, edx
0000C432 66890B                 <1>     mov     [ebx], cx ; 16 bits !
0000C435 80E40F                 <1>     and     ah, 0Fh ; al(bit0..7)+ah(bit0..3)
0000C438 EBC6                   <1>     jmp     short update_fat12_buffer
                                <1>
                                <1> update_fat32_cluster:
                                <1>     ; edx = 0 ; 24/08/2024
                                <1>     ;mov    ecx, [esi+LD_Clusters]
                                <1>     ;inc    ecx
                                <1>     ;mov    [LastCluster], ecx
                                <1>     ; 24/08/2024
                                <1>     ; ecx = [LastCluster]
                                <1>
                                <1>     ; 24/08/2024
                                <1>     ;cmp    eax, ecx
                                <1>     ;;ja    short return_uc_fat32_stc ; 25/07/2022
                                <1>     ;; 24/08/2024
                                <1>     ;; edx = 0 ; (must be -1 or > 0 after here)
                                <1>     ;ja     short return_uc_fat_stc
                                <1>
                                <1> pass_uc_fat32_errc:
                                <1>     ;sub    edx, edx
0000C43A BB80010000             <1>     mov     ebx, 180h ; 384
0000C43F F7F3                   <1>     div     ebx
                                <1>     ; EAX = Count of 3 FAT sectors
                                <1>     ; DX = Cluster index in FAT buffer
0000C441 89D3                   <1>     mov     ebx, edx
0000C443 C1E302                 <1>     shl     ebx, 2 ; Multiply by 4
                                <1>     ;mov    edx, 3
                                <1>     ; 25/07/2022
                                <1>     ;xor    dh, dh
                                <1>     ;mov    dl, 3
0000C446 66BA0300               <1>     mov     dx, 3
0000C44A F7E2                   <1>     mul     edx
                                <1>     ; EBX = Cluster Offset in FAT buffer
                                <1>     ; EAX = FAT Sector
                                <1>     ; EDX = 0
0000C44C 8A0D[8A7E0100]         <1>     mov     cl, [FAT_BuffValidData]
0000C452 80F902                 <1>     cmp     cl, 2
0000C455 7512                   <1>     jne     short loc_uc_check_fat32_buff_sector_load
                                <1>
                                <1> loc_uc_check_fat32_buff_sector_save:
0000C457 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
                                <1>     ;jne    loc_uc_save_fat_buffer
                                <1>     ;jmp    short loc_update_fat32_cell
                                <1>     ; 25/07/2022
0000C45D 741C                   <1>     je      short loc_update_fat32_cell
0000C45F E992FEFFFF             <1>     jmp     loc_uc_save_fat_buffer
                                <1>
                                <1> return_uc_fat12_stc:
                                <1> return_uc_fat32_stc:
                                <1>     ; 24/08/2024
0000C464 E964FEFFFF             <1>     jmp     loc_fat_buffer_stc_0 ; (*)
                                <1>     ; 25/07/2022
                                <1>     ;sub    eax, eax
                                <1>     ;stc
                                <1>     ;jmp    loc_fat_buffer_stc_1 ; (*)
                                <1>
                                <1> loc_uc_check_fat32_buff_sector_load:
0000C469 80F901                 <1>     cmp     cl, 1 ; byte [FAT_BuffValidData]
                                <1>     ;jne    loc_uc_load_fat_sectors
                                <1>     ; 25/07/2022
0000C46C 7508                   <1>     jne     short loc_uc_load_fat_sects
0000C46E 3B05[8E7E0100]         <1>     cmp     eax, [FAT_BuffSector]
                                <1>     ;jne    loc_uc_load_fat_sectors
                                <1>     ; 25/07/2022
0000C474 7405                   <1>     je      short loc_update_fat32_cell
                                <1> loc_uc_load_fat_sects:
0000C476 E9F0FEFFFF             <1>     jmp     loc_uc_load_fat_sectors
                                <1>
                                <1> loc_update_fat32_cell:
                                <1> loc_update_fat32_buffer:
0000C47B 81C3001C0900           <1>     add     ebx, FAT_Buffer ; 26/02/2016
0000C481 8B03                   <1>     mov     eax, [ebx]
0000C483 25FFFFFF0F             <1>     and     eax, 0FFFFFFFh ; 28 bit cluster value
                                <1>
0000C488 8B15[867E0100]         <1>     mov     edx, [FAT_CurrentCluster] ; 01/03/2016
                                <1>
0000C48E A3[867E0100]           <1>     mov     [FAT_CurrentCluster], eax
0000C493 8B0D[2C810100]         <1>     mov     ecx, [ClusterValue]
                                <1>     ;;;
                                <1>     ; 29/08/2024
0000C499 81E1FFFFFF0F           <1>     and     ecx, 0FFFFFFFh ; 28 bit cluster value
                                <1>     ;;;
0000C49F 890B                   <1>     mov     [ebx], ecx ; 29/02/2016
                                <1>
0000C4A1 C605[8A7E0100]02       <1>     mov     byte [FAT_BuffValidData], 2
                                <1>
                                <1>     ; 01/03/2016
0000C4A8 21C0                   <1>     and     eax, eax ; was it free cluster ?
0000C4AA 7513                   <1>     jnz     short loc_upd_fat32_c0
                                <1>
                                <1>     ;or     ecx, ecx ; it will be left free ?!
                                <1>     ;jz     short loc_upd_fat32_c3
```

```
                                <1>
0000C4AC 3B563E                 <1>     cmp     edx, [esi+LD_BPB+BPB_Reserved+4] ; First free cluster
0000C4AF 751F                   <1>     jne     short loc_upd_fat32_c3
                                <1>
0000C4B1 3B15[967E0100]         <1>     cmp     edx, [LastCluster]
0000C4B7 7206                   <1>     jb      short loc_upd_fat32_c0
                                <1>
                                <1>     ;mov    edx, 2 ; rewind !
                                <1>     ; 25/07/2022
0000C4B9 29D2                   <1>     sub     edx, edx
0000C4BB B202                   <1>     mov     dl, 2
0000C4BD EB0E                   <1>     jmp     short loc_upd_fat32_c2
                                <1>
                                <1> loc_upd_fat32_c0:
0000C4BF FF463E                 <1>     inc     dword [esi+LD_BPB+BPB_Reserved+4] ; set it to next cluster
0000C4C2 EB0C                   <1>     jmp     short loc_upd_fat32_c3
                                <1>
                                <1> loc_upd_fat32_c1:
0000C4C4 09C9                   <1>     or      ecx, ecx ; will it be free cluster ?
0000C4C6 7508                   <1>     jnz     short loc_upd_fat32_c3
                                <1>
0000C4C8 3B563E                 <1>     cmp     edx, [esi+LD_BPB+BPB_Reserved+4] ; First free cluster
0000C4CB 7303                   <1>     jnb     short loc_upd_fat32_c3
                                <1>
                                <1> loc_upd_fat32_c2:
0000C4CD 89563E                 <1>     mov     [esi+LD_BPB+BPB_Reserved+4], edx
                                <1>
                                <1> loc_upd_fat32_c3:
0000C4D0 89C2                   <1>     mov     edx, eax
                                <1>
                                <1> loc_upd_fat32_c4:
0000C4D2 83F802                 <1>     cmp     eax, 2
0000C4D5 728D                   <1>     jb      short return_uc_fat32_stc ; 25/07/2022
                                <1>
                                <1> pass_uc_fat32_c_zero_check_2:
0000C4D7 3B05[967E0100]         <1>     cmp     eax, [LastCluster]
0000C4DD 7785                   <1>     ja      short return_uc_fat32_stc ; 25/07/2022
                                <1>
0000C4DF E95DFEFFFF             <1>     jmp     loc_fat_buffer_updated
                                <1>
                                <1> save_fat_buffer:
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 31/08/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
                                <1>     ; 01/03/2016
                                <1>     ; 22/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 11/08/2011
                                <1>     ; 09/02/2005
                                <1>     ; INPUT ->
                                <1>     ;       None
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 0 -> OK.
                                <1>     ;       cf = 1 -> error code in AL (EAX)
                                <1>     ;
                                <1>     ;       EBX = FAT_Buffer address
                                <1>     ;
                                <1>     ; (EAX, EDX, ECX, EBX will be modified)
                                <1>
                                <1>     ;cmp    byte [FAT_BuffValidData], 2
                                <1>     ;je     short loc_save_fat_buff
                                <1>
                                <1> ;loc_save_fat_buffer_retn:
                                <1> ;   xor     eax, eax
                                <1> ;   retn
                                <1>
                                <1> loc_save_fat_buff:
0000C4E4 31D2                   <1>     xor     edx, edx
0000C4E6 8A35[8B7E0100]         <1>     mov     dh, [FAT_BuffDrvName]
0000C4EC 80FE41                 <1>     cmp     dh, 'A'
0000C4EF 724E                   <1>     jb      short loc_save_fat_buffer_inv_data_retn
0000C4F1 80EE41                 <1>     sub     dh, 'A'
0000C4F4 56                     <1>     push    esi ; *
0000C4F5 BE00010900             <1>         mov     esi, Logical_DOSDisks
0000C4FA 01D6                   <1>     add     esi, edx
                                <1>
0000C4FC 8A5603                 <1>     mov     dl, [esi+LD_FATType]
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     and     dl, dl
                                <1>     jz      short loc_save_fat_buffer_inv_data_pop_retn
                                <1> %endif
0000C4FF A1[8E7E0100]           <1>     mov     eax, [FAT_BuffSector]
0000C504 80FA02                 <1>     cmp     dl, 2
0000C507 772E                   <1>     ja      short loc_save_fat32_buff
                                <1>
                                <1> loc_save_fat_12_16_buff:
                                <1>     ; 01/03/2016
                                <1>     ; TRDOS v1 has a FATal bug here!
                                <1>     ; Correct code: mov dx, word ptr [FAT_BuffSector]+2
                                <1>     ; (DX:AX in TRDOS v1 -> EAX in TRDOS v2)
                                <1>     ;
0000C509 0FB74E1C               <1>     movzx   ecx, word [esi+LD_BPB+FATSecs]
0000C50D 29C1                   <1>     sub     ecx, eax
                                <1>     ; TRDOS v1 has a bug here... ('pop esi' was forgotten!)
                                <1>     ;jna    short loc_save_fat_buffer_inv_data_retn ; wrong addr!
0000C50F 762D                   <1>     jna     short loc_save_fat_buffer_inv_data_pop_retn ; correct addr.
                                <1>     ; 25/07/2022
                                <1>     ;jmp    short loc_save_fat_buffer_check_rs3
                                <1>
                                <1> loc_save_fat_buffer_check_rs3:
                                <1>     ; 25/07/2022
0000C511 29DB                   <1>     sub     ebx, ebx
0000C513 B303                   <1>     mov     bl, 3
                                <1>     ;cmp    ecx, 3
0000C515 39D9                   <1>     cmp     ecx, ebx ; 3
0000C517 7602                   <1>     jna     short loc_save_fat_buff_continue
0000C519 89D9                   <1>     mov     ecx, ebx ; mov ecx, 3
                                <1> loc_save_fat_buff_continue:
0000C51B BB001C0900             <1>     mov     ebx, FAT_Buffer
0000C520 034660                 <1>     add     eax, [esi+LD_FATBegin]
0000C523 51                     <1>     push    ecx
0000C524 E8BB540000             <1>     call    disk_write
0000C529 59                     <1>     pop     ecx
0000C52A 7239                   <1>     jc      short loc_save_FAT_buff_write_err
                                <1>
0000C52C 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
0000C530 7613                   <1>     jna     short loc_calc_2nd_fat12_16_addr
                                <1>
                                <1> loc_calc_2nd_fat32_addr:
0000C532 8B462A                 <1>     mov     eax, [esi+LD_BPB+FAT32_FAT_Size]
0000C535 EB12                   <1>     jmp     short loc_calc_2nd_fat_addr
                                <1>
                                <1>     ; 25/07/2022
```

```
                                <1> loc_save_fat32_buff:
0000C537 8B4E2A                 <1>     mov     ecx, [esi+LD_BPB+FAT32_FAT_Size]
0000C53A 29C1                   <1>     sub     ecx, eax
0000C53C 77D3                   <1>     ja      short loc_save_fat_buffer_check_rs3
                                <1>
                                <1> loc_save_fat_buffer_inv_data_pop_retn:
0000C53E 5E                     <1>     pop     esi ; *
                                <1> loc_save_fat_buffer_inv_data_retn:
                                <1>     ;mov    eax, 0Dh ; Invalid DATA
                                <1>     ; 25/07/2022
0000C53F 29C0                   <1>     sub     eax, eax
                                <1>     ;mov    al, 0Dh  ; Invalid Data
                                <1>     ; 31/08/2024
0000C541 B01D                   <1>     mov     al, ERR_INV_DATA ; 29 ; Invalid Data
0000C543 F9                     <1>     stc     ; cf = 1
0000C544 C3                     <1>     retn
                                <1>
                                <1> loc_calc_2nd_fat12_16_addr:
0000C545 0FB7461C               <1>     movzx   eax, word [esi+LD_BPB+FATSecs]
                                <1>
                                <1> loc_calc_2nd_fat_addr:
0000C549 034660                 <1>     add     eax, [esi+LD_FATBegin]
0000C54C 0305[8E7E0100]         <1>     add     eax, [FAT_BuffSector]
0000C552 BB001C0900             <1>     mov     ebx, FAT_Buffer
                                <1>     ; ecx = 1 to 3
0000C557 E888540000             <1>     call    disk_write
0000C55C 7207                   <1>     jc      short loc_save_FAT_buff_write_err
                                <1>     ; Valid buffer (1 = valid but do not save)
0000C55E C605[8A7E0100]01       <1>     mov     byte [FAT_BuffValidData], 1
                                <1>
                                <1> loc_save_FAT_buff_write_err:
0000C565 5E                     <1>     pop     esi ; *
0000C566 BB001C0900             <1>     mov     ebx, FAT_Buffer
                                <1>     ; 15/10/2016 (1Dh -> 18)
                                <1>     ; 23/03/2016 (1Dh)
0000C56B B812000000             <1>     mov     eax, 18 ; Drive not ready or write error
0000C570 C3                     <1>     retn
                                <1>
                                <1> calculate_fat_freespace:
                                <1>     ; 21/12/2025 (TRDOS 386 Kernel v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 23/03/2016
                                <1>     ; 02/03/2016
                                <1>     ; 01/03/2016
                                <1>     ; 29/02/2016
                                <1>     ; 22/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 30/04/2011
                                <1>     ; 03/04/2010
                                <1>     ; 2005
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Cluster count to be added or subtracted
                                <1>     ;       If BH = FFh, ESI = TR-DOS Logical Drive Description Table
                                <1>     ;       If BH < FFh, BH = TR-DOS Logical Drive Number
                                <1>     ;       BL:
                                <1>     ;       0 = Calculate, 1 = Add, 2 = Subtract, 3 = Get (Not Set/Calc)
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = Free Space in sectors
                                <1>     ;       ESI = Logical Dos Drive Description Table address
                                <1>     ;       BH = Logical Dos Drive Number (same with input value of BH)
                                <1>     ;       BL = Type of operation (same with input value of BL)
                                <1>     ;       ECX = 0 -> valid
                                <1>     ;       ECX > 0 -> error or invalid
                                <1>     ;       If EAX = FFFFFFFFh, it is 're-calculation needed'
                                <1>     ;                               sign due to r/w error
                                <1>     ;
                                <1>     ; (Modifed registers: eax, ebx, ecx, edx, esi)
                                <1>     ;
                                <1>
0000C571 66891D[32810100]       <1>     mov     [CFS_OPType], bx
0000C578 A3[34810100]           <1>     mov     [CFS_CC], eax
                                <1>
0000C57D 80FFFF                 <1>     cmp     bh, 0FFh
0000C580 740B                   <1>     je      short pass_calculate_freespace_get_drive_dt_offset
                                <1>
                                <1> loc_calculate_freespace_get_drive_dt_offset:
0000C582 31C0                   <1>     xor     eax, eax
0000C584 88FC                   <1>         mov     ah, bh
0000C586 BE00010900             <1>     mov     esi, Logical_DOSDisks
0000C58B 01C6                   <1>         add     esi, eax
                                <1>
                                <1> pass_calculate_freespace_get_drive_dt_offset:
0000C58D 08DB                   <1>     or      bl, bl
0000C58F 7436                   <1>     jz      short loc_reset_fcc
                                <1>
                                <1> loc_get_free_sectors:
0000C591 8B4674                 <1>     mov     eax, [esi+LD_FreeSectors]
                                <1>
                                <1>     ;xor    ecx, ecx
                                <1>     ;dec    ecx ; 0FFFFFFFFh
                                <1>     ;cmp    eax, ecx ; 29/02/2016
                                <1>     ;je     short loc_get_free_sectors_retn ; recalculation is needed!
                                <1>
                                <1>     ; 23/03/2016
0000C594 8B4E70                 <1>     mov     ecx, [esi+LD_TotalSectors]
0000C597 39C1                   <1>     cmp     ecx, eax ; Total sectors must be greater than Free sectors !
0000C599 7707                   <1>     ja      short loc_get_free_sectors_check_optype
                                <1>
0000C59B 31C0                   <1>     xor     eax, eax
0000C59D 48                     <1>     dec     eax ; 0FFFFFFFFh  ; recalculation is needed!
0000C59E 894674                 <1>     mov     [esi+LD_FreeSectors], eax ; reset (for recalculation)
                                <1>
                                <1> loc_get_free_sectors_retn:
0000C5A1 C3                     <1>     retn
                                <1>
                                <1> loc_get_free_sectors_check_optype:
0000C5A2 80FB03                 <1>     cmp     bl, 3
0000C5A5 7203                   <1>     jb      short loc_set_fcc_1 ; 25/07/2022
                                <1>
0000C5A7 29C9                   <1>     sub     ecx, ecx ; 0
                                <1>
0000C5A9 C3                     <1>     retn
                                <1>
                                <1> loc_set_fcc_1:
0000C5AA 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
                                <1>     ;ja     loc_update_FAT32_fs_info_fcc
                                <1>     ; 25/07/2022
0000C5AE 7605                   <1>     jna     short loc_set_fcc_2
0000C5B0 E9DD000000             <1>     jmp     loc_update_FAT32_fs_info_fcc
                                <1>
                                <1> loc_set_fcc_2:
                                <1>     ;mov    eax, [esi+LD_FreeSectors]
0000C5B5 0FB64E13               <1>     movzx   ecx, byte [esi+LD_BPB+SecPerClust]
0000C5B9 29D2                   <1>     sub     edx, edx
0000C5BB F7F1                   <1>     div     ecx
```

```
                                <1>     ;or     dx, dx
                                <1>     ;       ; DX -> Remain sectors < SecPerClust
                                <1>     ;       ; DX > 0 -> invalid free sector count
                                <1>     ;jnz    short loc_reset_fcc
                                <1>
                                <1> ;pass_set_fcc_div32:
0000C5BD A3[A87E0100]           <1>     mov     [FreeClusterCount], eax
0000C5C2 E986000000             <1>         jmp     loc_set_free_sectors_FAT12_FAT16
                                <1>
                                <1> loc_reset_fcc:
0000C5C7 31C0                   <1>     xor     eax, eax
0000C5C9 A3[A87E0100]           <1>     mov     [FreeClusterCount], eax ; 0
0000C5CE 8B5678                 <1>     mov     edx, [esi+LD_Clusters]
0000C5D1 42                     <1>     inc     edx
0000C5D2 8915[967E0100]         <1>     mov     [LastCluster], edx
                                <1>
0000C5D8 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
0000C5DC 7645                   <1>     jna     short loc_count_free_fat_clusters_0
                                <1>
0000C5DE 48                     <1>     dec     eax ; FFFFFFFFh
0000C5DF A3[3C810100]           <1>     mov     [CFS_FAT32FC], eax
                                <1>
                                <1>     ; 29/02/2016
0000C5E4 89463A                 <1>     mov     [esi+LD_BPB+BPB_Reserved], eax ; reset
0000C5E7 89463E                 <1>     mov     [esi+LD_BPB+BPB_Reserved+4], eax ; reset
                                <1>
                                <1>     ;mov    eax, 2
                                <1>     ; 25/07/2022
0000C5EA 40                     <1>     inc     eax ; eax = 0
0000C5EB B002                   <1>     mov     al, 2
                                <1>
                                <1> loc_count_fc_next_cluster_0:
0000C5ED 50                     <1>     push    eax
0000C5EE E896F9FFFF             <1>     call    get_next_cluster
0000C5F3 7310                   <1>     jnc     short loc_check_fat32_ff_cluster
0000C5F5 09C0                   <1>     or      eax, eax
0000C5F7 741E                   <1>     jz      short pass_inc_cfs_fcc_0
                                <1>
                                <1> loc_put_fcc_unknown_sign:
0000C5F9 58                     <1>     pop     eax
                                <1>     ; "Free count is Unknown" sign
                                <1>     ;mov    dword [FreeClusterCount], 0FFFFFFFFh
                                <1>
                                <1>     ; 29/02/2016
                                <1>     ; Save Free Cluster Count value in FAT32 'BPB_Reserved' area
                                <1>     ;mov    [esi+LD_BPB+BPB_Reserved], 0FFFFFFFFh ; unknown!
0000C5FA 8B15[3C810100]         <1>     mov     edx, [CFS_FAT32FC] ; First Free Cluster
                                <1>     ; Save First Free Cluster value in FAT32 'BPB_Reserved+4' area
0000C600 89563E                 <1>     mov     [esi+LD_BPB+BPB_Reserved+4], edx
                                <1>
0000C603 EB7D                   <1>         jmp     loc_put_fcc_invalid_sign
                                <1>
                                <1> loc_check_fat32_ff_cluster:
0000C605 09C0                   <1>     or      eax, eax
0000C607 750E                   <1>     jnz     short pass_inc_cfs_fcc_0
0000C609 58                     <1>     pop     eax
0000C60A A3[3C810100]           <1>     mov     [CFS_FAT32FC], eax
                                <1>     ;mov    dword [FreeClusterCount], 1
0000C60F FF05[A87E0100]         <1>     inc     dword [FreeClusterCount]
0000C615 EB27                   <1>     jmp     short pass_inc_cfs_fcc_1
                                <1>
                                <1> pass_inc_cfs_fcc_0:
0000C617 58                     <1>     pop     eax
                                <1>
                                <1> pass_inc_cfs_fcc_0c:
0000C618 40                     <1>     inc     eax ; add eax, 1
0000C619 3B05[967E0100]         <1>     cmp     eax, [LastCluster]
0000C61F 76CC                   <1>     jna     short loc_count_fc_next_cluster_0
0000C621 EB6F                   <1>     jmp     short loc_update_FAT32_fs_info_fcc
                                <1>
                                <1> loc_count_free_fat_clusters_0:
                                <1>     ;mov    eax, 2
0000C623 B002                   <1>     mov     al, 2
                                <1>
                                <1> loc_count_fc_next_cluster:
0000C625 50                     <1>     push    eax
0000C626 E85EF9FFFF             <1>     call    get_next_cluster
0000C62B 720C                   <1>     jc      short loc_count_fcc_stc
                                <1>
                                <1> loc_count_free_clusters_1:
0000C62D 21C0                   <1>     and     eax, eax
0000C62F 750C                   <1>     jnz     short pass_inc_cfs_fcc
                                <1>
0000C631 FF05[A87E0100]         <1>     inc     dword [FreeClusterCount]
0000C637 EB04                   <1>     jmp     short pass_inc_cfs_fcc
                                <1>
                                <1> loc_count_fcc_stc:
0000C639 09C0                   <1>     or      eax, eax
0000C63B 75BC                   <1>     jnz     short loc_put_fcc_unknown_sign ; 29/02/2016
                                <1>
                                <1> pass_inc_cfs_fcc:
0000C63D 58                     <1>     pop     eax
                                <1>
                                <1> pass_inc_cfs_fcc_1:
0000C63E 40                     <1>     inc     eax ; add eax, 1
0000C63F 3B05[967E0100]         <1>     cmp     eax, [LastCluster]
0000C645 76DE                   <1>     jna     short loc_count_fc_next_cluster
                                <1>
                                <1> loc_set_free_sectors:
0000C647 807E0302               <1>     cmp     byte [esi+LD_FATType], 2
0000C64B 7745                   <1>     ja      short loc_update_FAT32_fs_info_fcc
                                <1>
                                <1> loc_set_free_sectors_FAT12_FAT16:
0000C64D 803D[32810100]00       <1>     cmp     byte [CFS_OPType], 0
0000C654 761C                   <1>     jna     short pass_FAT_add_sub_fcc
0000C656 A1[34810100]           <1>     mov     eax, [CFS_CC]
0000C65B 803D[32810100]01       <1>     cmp     byte [CFS_OPType], 1
0000C662 7708                   <1>     ja      short pass_FAT_add_fcc
0000C664 0105[A87E0100]         <1>     add     [FreeClusterCount], eax
0000C66A EB06                   <1>     jmp     short pass_FAT_add_sub_fcc
                                <1>
                                <1> pass_FAT_add_fcc:
0000C66C 2905[A87E0100]         <1>     sub     [FreeClusterCount], eax
                                <1>
                                <1> pass_FAT_add_sub_fcc:
0000C672 0FB64613               <1>     movzx   eax, byte [esi+LD_BPB+SecPerClust]
0000C676 8B15[A87E0100]         <1>     mov     edx, [FreeClusterCount]
0000C67C F7E2                   <1>     mul     edx
                                <1>
0000C67E 31C9                   <1>     xor     ecx, ecx
0000C680 EB05                   <1>     jmp     short loc_cfs_retn_params
                                <1>
                                <1> loc_put_fcc_invalid_sign:
0000C682 29C0                   <1>             sub     eax, eax ; 0
```

```
0000C684 48                  <1>     dec     eax ; FFFFFFFFh
                             <1> loc_fat32_ffc_recalc_needed:
0000C685 89C1                <1>     mov     ecx, eax
                             <1>
                             <1> loc_cfs_retn_params:
0000C687 894674              <1>     mov     [esi+LD_FreeSectors], eax
0000C68A 0FB71D[32810100]    <1>     movzx   ebx, word [CFS_OPType]
0000C691 C3                  <1>     retn
                             <1>
                             <1> loc_update_FAT32_fs_info_fcc:
                             <1> loc_check_fcc_FSINFO_op:
                             <1>     ; 29/02/2016
                             <1>     ; EAX = Free cluster count (before this update) ; value from disk
                             <1>     ; EDX = First Free Cluster (before this update) ; value from disk
0000C692 803D[32810100]01    <1>     cmp     byte [CFS_OPType], 1
0000C699 7221                <1>     jb      short loc_cfs_FAT32_get_rcalc_parms ; 0 = recalculated
0000C69B 7406                <1>     je      short loc_check_fcc_FSINFO_op1 ; 1 = add
                             <1> loc_check_fcc_FSINFO_op2: ; subtract
0000C69D F71D[34810100]      <1>     neg     dword [CFS_CC] ; prepare to subtract ; 2 = sub (add negative)
                             <1> loc_check_fcc_FSINFO_op1:
                             <1>     ; 01/03/2016
0000C6A3 31D2                <1>     xor     edx, edx ; 0
0000C6A5 4A                  <1>     dec     edx ; 0FFFFFFFFh
0000C6A6 8B463A              <1>     mov     eax, [esi+LD_BPB+BPB_Reserved]
0000C6A9 39D0                <1>     cmp     eax, edx
                             <1>     ;jnb    short loc_put_fcc_invalid_sign
                             <1>     ; 21/12/2025
0000C6AB 73D8                <1>     jnb     short loc_fat32_ffc_recalc_needed
                             <1>
0000C6AD 0305[34810100]      <1>         add     eax, [CFS_CC] ; free cluster count on disk + current count
0000C6B3 72CD                <1>     jc      short loc_put_fcc_invalid_sign
                             <1>
0000C6B5 A3[A87E0100]        <1>     mov     [FreeClusterCount], eax
0000C6BA EB0E                <1>     jmp     short loc_cfs_write_FSINFO_sector
                             <1>
                             <1> loc_cfs_FAT32_get_rcalc_parms:
0000C6BC 8B15[3C810100]      <1>     mov     edx, [CFS_FAT32FC]
0000C6C2 A1[A87E0100]        <1>     mov     eax, [FreeClusterCount]
0000C6C7 89563E              <1>     mov     [esi+LD_BPB+BPB_Reserved+4], edx ; First Free Cluster
                             <1> loc_cfs_write_FSINFO_sector:
0000C6CA 89463A              <1>     mov     [esi+LD_BPB+BPB_Reserved], eax ; Free cluster count
                             <1>
0000C6CD E8BC000000          <1>     call    set_fat32_fsinfo_sector_parms
0000C6D2 72AE                <1>     jc      short loc_put_fcc_invalid_sign
                             <1>
                             <1> loc_set_FAT32_free_sectors:
                             <1>     ; 29/02/2016
                             <1>     ;mov    eax, [FreeClusterCount]
                             <1>     ;mov    ecx, eax
                             <1>     ;cmp    eax, 0FFFFFFFFh ; Invalid !
                             <1>     ;je     short loc_cfs_retn_params
                             <1>     ;
0000C6D4 8B0D[A87E0100]      <1>     mov     ecx, [FreeClusterCount]
0000C6DA 0FB64613            <1>     movzx   eax, byte [esi+LD_BPB+SecPerClust]
0000C6DE F7E1                <1>     mul     ecx
                             <1>     ; 29/02/2016
0000C6E0 31C9                <1>     xor     ecx, ecx ; 0
0000C6E2 09D2                <1>     or      edx, edx ; 0 ?
0000C6E4 759C                <1>         jnz short loc_put_fcc_invalid_sign ; 25/07/2022
0000C6E6 394670              <1>     cmp     [esi+LD_TotalSectors], eax ; Volume size in sectors
0000C6E9 7697                <1>         jna     short loc_put_fcc_invalid_sign
                             <1>     ;
                             <1> loc_set_FAT32_free_sectors_ok:
0000C6EB 31D2                <1>     xor     edx, edx ; 0
0000C6ED EB98                <1>         jmp     short loc_cfs_retn_params
                             <1>     ;
                             <1>
                             <1> get_last_cluster:
                             <1>     ; 22/10/2016
                             <1>     ; 27/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 12/06/2010 (DRV_FAT.ASM, 'proc_get_last_custer')
                             <1>     ; 06/06/2010
                             <1>     ; INPUT ->
                             <1>     ;       EAX = First Cluster Number
                             <1>     ;       ESI = Logical Dos Drive Parameters Table
                             <1>     ; OUTPUT ->
                             <1>     ;       cf = 0 -> No Error, EAX is valid
                             <1>     ;       cf = 1 -> EAX > 0 -> Error
                             <1>     ;       EAX = Last Cluster Number
                             <1>     ;        ECX = Previous Cluster -just before the last cluster-
                             <1>     ;        ; 22/10/2016
                             <1>     ;       [glc_index] = cluster index number of the last cluster
                             <1>     ;
                             <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                             <1>
0000C6EF 89C1                <1>     mov     ecx, eax
                             <1>
0000C6F1 C705[44810100]FFFF- <1>     mov     dword [glc_index], 0FFFFFFFFh ; 22/10/2016
0000C6F9 FFFF                <1>
                             <1>
                             <1> loc_glc_get_next_cluster_1:
0000C6FB 890D[40810100]      <1>     mov     [glc_prevcluster], ecx
                             <1>     ; 22/10/2016
0000C701 FF05[44810100]      <1>     inc     dword [glc_index]
                             <1>
                             <1> loc_glc_get_next_cluster_2:
0000C707 E87DF8FFFF          <1>     call    get_next_cluster
                             <1>     ; ecx = current/previous cluster
                             <1>     ; eax = next/last cluster
0000C70C 73ED                <1>     jnc     short loc_glc_get_next_cluster_1
                             <1>
0000C70E 09C0                <1>     or      eax, eax
0000C710 7509                <1>     jnz     short loc_glc_stc_retn
                             <1>
                             <1>     ; ecx = previous cluster
0000C712 89C8                <1>         mov eax, ecx
                             <1>
                             <1>     ; previous cluster becomes last cluster (ecx -> eax)
                             <1>     ; previous of previous cluster becomes previous cluster (ecx)
                             <1>
                             <1> loc_glc_prev_cluster_retn:
0000C714 8B0D[40810100]      <1>     mov     ecx, [glc_prevcluster]
0000C71A C3                  <1>     retn
                             <1>
                             <1> loc_glc_stc_retn:
0000C71B F5                  <1>     cmc     ;stc
0000C71C EBF6                <1>         jmp short loc_glc_prev_cluster_retn
                             <1>
                             <1> truncate_cluster_chain:
                             <1>     ; 31/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; 01/03/2016
                             <1>     ; 28/02/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 22/01/2011 (DRV_FAT.ASM, 'proc_truncate_cluster_chain')
                             <1>     ; 11/09/2010
```

```
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ;       EAX = First cluster to be truncated/unlinked
                                <1>     ; OUTPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ;       ECX = Count of truncated/removed clusters
                                <1>     ;       CF = 0 -> EAX = Free sectors
                                <1>     ;       CF = 1 -> Error code in EAX (AL)
                                <1>
                                <1>     ; NOTE: This procedure does not update lm date&time !
                                <1>
                                <1> loc_truncate_cc:
0000C71E 31C9                   <1>     xor     ecx, ecx ; mov ecx, 0
                                <1>     ;mov    byte [FAT_BuffValidData], 0
0000C720 890D[927E0100]         <1>     mov     [FAT_ClusterCounter], ecx ; 0 ; reset
                                <1>
                                <1>     ;;;
                                <1>     ; 31/08/2024
0000C726 21C0                   <1>     and     eax, eax ; 0
0000C728 740D                   <1>     jz      short loc_tcc_unlink_zero_cluster  ; zero
                                <1>     ;cmp    eax, 0FFFFFF7h
0000C72A 3DFFFFFF0F             <1>     cmp     eax, 0FFFFFFFh ; 28 bit cluster number limit (EOF)
0000C72F 7207                   <1>     jb      short loc_tcc_unlink_clusters
                                <1>
                                <1>     ; (possible FAT32) EOF signature...
                                <1>     ;       not a valid cluster number
                                <1>     ;
                                <1>     ; NOTE: update_cluster returns EOF (if eax > [LastCluster])
                                <1>     ;       instead of invalid data error
                                <1>
0000C731 B81D000000             <1>     mov     eax, ERR_INV_DATA  ; invalid cluster number
0000C736 F9                     <1>     stc
                                <1> loc_tcc_unlink_zero_cluster:      ; nothing to do
0000C737 C3                     <1>     retn
                                <1>     ;;;
                                <1>
                                <1> loc_tcc_unlink_clusters:
0000C738 E833FBFFFF             <1>     call    update_cluster
                                <1>     ; EAX = Next Cluster
                                <1>     ; ECX = Cluster Value
                                <1>     ; Note:
                                <1>     ; Returns count of unlinked clusters in
                                <1>     ; dword ptr FAT_ClusterCounter
0000C73D 73F9                   <1>     jnc     short loc_tcc_unlink_clusters
                                <1>
                                <1>     ; error or EOF (end of cluster chain) ; 31/08/2024
                                <1>
                                <1> pass_tcc_unlink_clusters:
0000C73F A2[4B810100]           <1>     mov     byte [TCC_FATErr], al
0000C744 803D[8A7E0100]02       <1>     cmp     byte [FAT_BuffValidData], 2
0000C74B 750E                   <1>     jne     short loc_tcc_calculate_FAT_freespace
0000C74D E892FDFFFF             <1>     call    save_fat_buffer
0000C752 7307                   <1>     jnc     short loc_tcc_calculate_FAT_freespace
0000C754 A2[4B810100]           <1>     mov     byte [TCC_FATErr], al ; Error
                                <1>     ;mov    byte [FAT_BuffValidData], 0
                                <1>
                                <1>     ; 01/03/2016
0000C759 EB12                   <1>     jmp     short loc_tcc_recalculate_FAT_freespace
                                <1>
                                <1> loc_tcc_calculate_FAT_freespace:
0000C75B A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter] ; signed (+-) number
0000C760 66BB01FF               <1>     mov     bx, 0FF01h ; BH = FFh -> ESI = Dos drv desc. table
                                <1>                        ; BL = 1 -> add cluster(s)
0000C764 E808FEFFFF             <1>     call    calculate_fat_freespace
0000C769 21C9                   <1>     and     ecx, ecx ; cx = 0 -> valid free sector count
0000C76B 7409                   <1>     jz      short pass_truncate_cc_recalc_FAT_freespace
                                <1>
                                <1> loc_tcc_recalculate_FAT_freespace:
0000C76D 66BB00FF               <1>     mov     bx, 0FF00h ; recalculate !
0000C771 E8FBFDFFFF             <1>     call    calculate_fat_freespace
                                <1>
                                <1> loc_tcc_calculate_FAT_freespace_err:
                                <1> pass_truncate_cc_recalc_FAT_freespace:
0000C776 8B0D[927E0100]         <1>     mov     ecx, [FAT_ClusterCounter]
                                <1>
0000C77C 803D[4B810100]00       <1>     cmp     byte [TCC_FATErr], 0
0000C783 7608                   <1>     jna     short loc_tcc_unlink_clusters_retn
                                <1>
                                <1> loc_tcc_unlink_clusters_error:
0000C785 0FB605[4B810100]       <1>     movzx   eax, byte [TCC_FATErr]
0000C78C F9                     <1>     stc
                                <1> loc_tcc_unlink_clusters_retn:
0000C78D C3                     <1>     retn
                                <1>
                                <1> set_fat32_fsinfo_sector_parms:
                                <1>     ; 15/10/2016
                                <1>     ; 23/03/2016
                                <1>     ; 29/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ;       [esi+LD_BPB+BPB_Reserved] = Free Cluster Count
                                <1>     ;       [esi+LD_BPB+BPB_Reserved+4] = First Free Cluster
                                <1>     ; OUTPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ;       CF = 0 -> OK..
                                <1>     ;       CF = 1 -> Error code in EAX (AL)
                                <1>     ;
                                <1>     ; (Modified registers: EAX, EBX, ECX, EDX)
                                <1>
0000C78E E824000000             <1>     call    get_fat32_fsinfo_sector_parms
0000C793 7221                   <1>     jc      short update_fat32_fsinfo_sector_retn
                                <1>
0000C795 8B463A                 <1>     mov     eax, [esi+LD_BPB+BPB_Reserved] ; Free Cluster Count
0000C798 8B563E                 <1>     mov     edx, [esi+LD_BPB+BPB_Reserved+4] ; First free Cluster
                                <1>
                                <1>         ;mov       ebx, DOSBootSectorBuff
0000C79B 8983E8010000           <1>     mov     [ebx+488], eax
0000C7A1 8993EC010000           <1>     mov     [ebx+492], edx
                                <1>
0000C7A7 A1[38810100]           <1>     mov     eax, [CFS_FAT32FSINFOSEC]
0000C7AC B901000000             <1>     mov     ecx, 1
0000C7B1 E82E520000             <1>     call    disk_write
                                <1>     ;jnc    short update_fat32_fsinfo_sector_retn
                                <1>
                                <1>     ; 15/10/2016 (1Dh -> 18)
                                <1>     ; 23/03/2016 (1Dh)
                                <1>     ;mov    eax, 18 ; Drive not ready or write error
                                <1>
                                <1> update_fat32_fsinfo_sector_retn:
0000C7B6 C3                     <1>     retn
                                <1>
                                <1> get_fat32_fsinfo_sector_parms:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
```

```
                                <1>     ; 23/03/2016
                                <1>     ; 01/03/2016
                                <1>     ; 29/02/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ; OUTPUT ->
                                <1>     ;       ESI = Logical dos drive description table address
                                <1>     ;       EBX = FSINFO sector buffer address (DOSBootSectorBuff)
                                <1>     ;       CF = 0 -> OK..
                                <1>     ;          EAX = FsInfo sector address
                                <1>     ;          ECX = Free cluster count
                                <1>     ;          EDX = First free cluster
                                <1>     ;       CF = 1 -> Error code in AL (EAX)
                                <1>     ;          EBX = 0
                                <1>     ;
                                <1>     ;       [CFS_FAT32FSINFOSEC] = FAT32 FSINFO sector address
                                <1>         ;
                                <1>     ; (Modified registers: EAX, EBX, ECX, EDX)
                                <1>
0000C7B7 0FB74636               <1>     movzx   eax, word [esi+LD_BPB+FAT32_FSInfoSec]
0000C7BB 03466C                 <1>     add     eax, [esi+LD_StartSector]
0000C7BE A3[38810100]           <1>     mov     [CFS_FAT32FSINFOSEC], eax
                                <1>
0000C7C3 BB[867C0100]           <1>         mov     ebx, DOSBootSectorBuff
                                <1>     ;mov    ecx, 1
                                <1>     ; 25/07/2022
0000C7C8 29C9                   <1>     sub     ecx, ecx
0000C7CA FEC1                   <1>     inc     cl
                                <1>     ; ecx = 1
0000C7CC E822520000             <1>     call    disk_read
0000C7D1 722F                   <1>     jc      short loc_read_FAT32_fsinfo_sec_err
                                <1>
0000C7D3 BB[867C0100]           <1>     mov     ebx, DOSBootSectorBuff
                                <1>
0000C7D8 813B52526141           <1>     cmp     dword [ebx], 41615252h
0000C7DE 751E                   <1>     jne     short loc_read_FAT32_fsinfo_sec_stc
                                <1>
0000C7E0 81BBE4010000727241-    <1>     cmp     dword [ebx+484], 61417272h
0000C7E9 61                     <1>
0000C7EA 7512                   <1>     jne     short loc_read_FAT32_fsinfo_sec_stc
                                <1>
0000C7EC A1[38810100]           <1>     mov     eax, [CFS_FAT32FSINFOSEC]
0000C7F1 8B8BE8010000           <1>     mov     ecx, [ebx+488] ; free cluster count
0000C7F7 8B93EC010000           <1>     mov     edx, [ebx+492] ; first (next) free cluster
                                <1>
0000C7FD C3                     <1>     retn
                                <1>
                                <1> loc_read_FAT32_fsinfo_sec_stc:
                                <1>     ; 15/10/2016 (0Bh -> 28)
                                <1>     ;mov    eax, 28 ; Invalid format!
                                <1>     ; 25/07/2022
0000C7FE B31C                   <1>     mov     bl, 28
0000C800 EB02                   <1>     jmp     short loc_read_FAT32_fsinfo_sec_stc_retn
                                <1>
                                <1> loc_read_FAT32_fsinfo_sec_err:
                                <1>     ; 15/10/2016 (15h -> 17)
                                <1>     ; 23/03/2016 (15h)
                                <1>     ;mov    eax, 17 ; Drive not ready or read error
                                <1>     ; 25/07/2022
0000C802 B311                   <1>     mov     bl, 17
                                <1> loc_read_FAT32_fsinfo_sec_stc_retn:
                                <1>     ; 25/07/2022
0000C804 29C0                   <1>     sub     eax, eax
0000C806 88D8                   <1>     mov     al, bl ; error code
                                <1>     ; eax = error code
0000C808 29DB                   <1>     sub     ebx, ebx ; 0
0000C80A F9                     <1>     stc
0000C80B C3                     <1>     retn
                                <1>
                                <1> add_new_cluster:
                                <1>     ; 30/08/2024
                                <1>     ; 27/08/2024
                                <1>     ; 25/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 15/10/2016
                                <1>     ; 16/05/2016
                                <1>     ; 18/03/2016, 24/03/2016
                                <1>     ; 11/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 30/07/2011 (DRV_FAT.ASM)
                                <1>     ; 11/09/2010
                                <1>     ; INPUT ->
                                <1>     ;       ESI = Logical dos drv desc. table address
                                <1>     ;       EAX = Last cluster
                                <1>     ; OUTPUT ->
                                <1>     ;       ESI = Logical dos drv desc. table address
                                <1>     ;       EAX = New Last cluster (next cluster)
                                <1>     ;       cf = 1 -> error code in EAX (AL)
                                <1>     ;       cf = 1 -> EBX = sectors per cluster
                                <1>     ;       ECX = Free sectors
                                <1>     ;;;     25/07/2022
                                <1>     ;       (EBX = sectors per cluster -not used-)
                                <1>     ;       EDX = 0 (if cf = 0)
                                <1>     ; NOTE:
                                <1>     ; This procedure does not update lm date&time !
                                <1>     ;    ; 30/08/2024
                                <1>     ; and doesn't update 1st clust and file size fields !
                                <1>     ;
                                <1>     ; (Modified registers: EAX, EBX, ECX, EDX, EDI)
                                <1>
0000C80C A3[68820100]           <1>     mov     [FAT_anc_LCluster], eax
                                <1>
0000C811 E885F9FFFF             <1>     call    get_first_free_cluster
0000C816 7206                   <1>     jc      short loc_add_new_cluster_retn
                                <1>     ; EAX >= 2 and EAX < FFFFFFFFh is valid
                                <1>
                                <1>     ;mov    edx, eax
                                <1>     ;
                                <1>     ;inc    edx
                                <1>     ;;jnz   short loc_add_new_cluster_check_ffc_eax
                                <1>     ;jnz    short loc_add_new_cluster_save_fcc
                                <1>
                                <1>     ; 27/08/2024
0000C818 40                     <1>     inc     eax ; (*)
0000C819 750D                   <1>     jnz     short loc_add_new_cluster_save_fcc
                                <1>
                                <1> loc_add_new_cluster_no_disk_space_retn:
                                <1>     ;mov    eax, 27h ; MSDOS err => insufficient disk space
                                <1>     ; 27/08/2024
                                <1>     ; eax = 0
                                <1>     ; 25/07/2022
                                <1>     ;xor    eax, eax
0000C81B B027                   <1>     mov     al, 27h
                                <1> loc_add_new_cluster_stc_retn:
0000C81D F9                     <1>     stc
```

```
                                <1> loc_add_new_cluster_retn:
                                <1>     ; 25/07/2022
                                <1>     ;movzx  ebx, byte [esi+LD_BPB+SecPerClust]
0000C81E 8B4E74                 <1>     mov     ecx, [esi+LD_FreeSectors]
                                <1>     ;xor    edx, edx
                                <1>     ;stc
0000C821 C3                     <1>     retn
                                <1>
                                <1> loc_anc_invalid_format_stc_retn:
                                <1>     ; 27/08/2024
                                <1>     ;stc
                                <1> loc_add_new_cluster_invalid_format_retn:
                                <1>     ; 15/10/2016 (0Bh -> 28)
                                <1>     ;mov    eax, 28 ; Invalid format
                                <1>     ;jmp    short loc_add_new_cluster_retn
                                <1>     ; 25/07/2022
0000C822 29C0                   <1>     sub     eax, eax
0000C824 B01C                   <1>     mov     al, 28
0000C826 EBF5                   <1>     jmp     short loc_add_new_cluster_stc_retn
                                <1>
                                <1> ;loc_add_new_cluster_check_ffc_eax:
                                <1> ;   cmp     eax, 2
                                <1> ;   jb      short loc_add_new_cluster_invalid_format_retn
                                <1>
                                <1> loc_add_new_cluster_save_fcc:
                                <1>     ;;;
                                <1>     ; 27/08/2024
0000C828 48                     <1>     dec     eax ; (*)
                                <1>     ;;;
0000C829 A3[6C820100]           <1>     mov     [FAT_anc_FFCluster], eax
                                <1>
                                <1> ; 27/08/2024 (TRDOS 386 v2.0.9)
                                <1> %if 0
                                <1>     sub     eax, 2
                                <1>     movzx   ebx, byte [esi+LD_BPB+SecPerClust]
                                <1>     mul     ebx
                                <1>     or      edx, edx
                                <1>     jnz     short loc_anc_invalid_format_stc_retn
                                <1>
                                <1> loc_add_new_cluster_allocate_cluster:
                                <1>     ; 18/03/2016
                                <1>     xchg    edx, eax ; eax = 0
                                <1>     ; 16/05/2016
                                <1>     ;cmp    [ClusterBuffer_Valid], al ; 0
                                <1>     ;jna    short loc_anc_clear_cluster_buffer
                                <1>     ;; 'copy' command,
                                <1>     ;; writing destination file clust after reading source file clust
                                <1>     ;mov    [ClusterBuffer_Valid], al ; 0 ; reset
                                <1>     ;jmp    short loc_add_new_cluster_write_nc_to_disk
                                <1>
                                <1> loc_anc_clear_cluster_buffer:
                                <1>     ; 11/03/2016
                                <1>     ; Clear buffer
                                <1>     mov     edi, Cluster_Buffer ; 70000h (for current TRDOS 386 version)
                                <1>     mov     ecx, ebx ; sector count
                                <1>     shl     ecx, 7 ; 1 sector = 512 bytes -> 128 double words
                                <1>     ;xor    eax, eax ; 0
                                <1>     rep     stosd
                                <1>
                                <1> loc_add_new_cluster_write_nc_to_disk:
                                <1>     ; 11/03/2016
                                <1>     ;xchg   eax, edx ; edx = 0, eax = sector offset
                                <1>     mov     eax, edx
                                <1>         add     eax, [esi+LD_DATABegin]
                                <1>     jc      short loc_add_new_cluster_invalid_format_retn
                                <1>
                                <1>     mov     ecx, ebx ; ECX = sectors per cluster (<256)
                                <1>     mov     ebx, Cluster_Buffer
                                <1>     call    disk_write
                                <1>     jnc     short loc_add_new_cluster_update_fat_nlc
                                <1>
                                <1>     ; 15/10/2016 (1Dh -> 18)
                                <1>     ;mov    eax, 18 ; Write Error
                                <1>     ; 25/07/2022
                                <1>     xor     eax, eax
                                <1>     mov     al, 18
                                <1>     jmp     short loc_add_new_cluster_stc_retn
                                <1>
                                <1> loc_add_new_cluster_update_fat_nlc:
                                <1>     mov     eax, [FAT_anc_FFCluster]
                                <1> %endif
                                <1>     ; 30/08/2024
                                <1>     ; eax = [FAT_anc_FFCluster] ; first free cluster
0000C82E 31C9                   <1>     xor     ecx, ecx ; 0
0000C830 890D[927E0100]         <1>     mov     [FAT_ClusterCounter], ecx ; 0 ; reset
0000C836 49                     <1>     dec     ecx ; -1 ; 0FFFFFFFFh
0000C837 850D[68820100]         <1>     test    [FAT_anc_LCluster], ecx       ; 0 ?
0000C83D 7416                   <1>     jz      short loc_add_new_cluster_update_fat_fc ; yes
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ; eax = (first free) cluster to be added as last cluster
                                <1>     ;;;
                                <1>     ;xor    ecx, ecx
                                <1>
                                <1> loc_add_new_cluster_update_fat_nlc: ; 30/08/2024
                                <1>     ;mov    [FAT_ClusterCounter], ecx ; 0 ; reset
                                <1>     ;dec    ecx ; 0FFFFFFFFh ; last cluster
0000C83F E82CFAFFFF             <1>     call    update_cluster
0000C844 7304                   <1>     jnc     short loc_add_new_cluster_update_fat_plc
0000C846 09C0                   <1>     or      eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000C848 75D3                   <1>     jnz     short loc_add_new_cluster_stc_retn
                                <1>
                                <1> loc_add_new_cluster_update_fat_plc:
0000C84A A1[68820100]           <1>     mov     eax, [FAT_anc_LCluster]
0000C84F 8B0D[6C820100]         <1>     mov     ecx, [FAT_anc_FFCluster]
                                <1> loc_add_new_cluster_update_fat_fc: ; 30/08/2024
0000C855 E816FAFFFF             <1>     call    update_cluster
0000C85A 7311                   <1>     jnc     short loc_add_new_cluster_save_fat_buffer
                                <1>
0000C85C 09C0                   <1>     or      eax, eax ; EAX = 0 -> cluster value is 0 or eocc
0000C85E 740D                   <1>     jz      short loc_add_new_cluster_save_fat_buffer
                                <1>
                                <1> loc_anc_save_fat_buffer_err_retn:
                                <1>     ;cmp    byte [FAT_ClusterCounter], 1
                                <1>     ;jb     short loc_add_new_cluster_retn
                                <1>
0000C860 66BB00FF               <1>     mov     bx, 0FF00h ; recalculate free space (BL = 0)
                                <1>                        ; (BH = FFh -> Use ESI as Drv Param. Tbl.)
0000C864 50                     <1>     push    eax
0000C865 E807FDFFFF             <1>     call    calculate_fat_freespace
0000C86A 58                     <1>     pop     eax
0000C86B EBB0                   <1>         jmp     loc_add_new_cluster_stc_retn
                                <1>
                                <1> loc_add_new_cluster_save_fat_buffer:
```

```
                                <1>     ;cmp    byte [FAT_BuffValidData], 2
                                <1>     ;jne    short loc_add_new_cluster_calc_FAT_freespace
                                <1>     ;Byte [FAT_BuffValidData] = 2
0000C86D E872FCFFFF             <1>     call    save_fat_buffer
0000C872 72EC                   <1>     jc      short loc_anc_save_fat_buffer_err_retn
                                <1>
                                <1> loc_add_new_cluster_calc_FAT_freespace:
                                <1>     ;mov    eax, 1 ; Only one Cluster
0000C874 A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter]
0000C879 66BB01FF               <1>     mov     bx, 0FF01h ; BH = FFh -> ESI -> Dos drv desc. table
                                <1>             ; BL = 1 -> add cluster(s)
                                <1>     ;mov    bl, 01h ; 27/08/2024
                                <1>     ; NOTE: EAX value will be added to Free Cluster Count
                                <1>     ; (Free Cluster Count is decreased when EAX value is negative)
0000C87D E8EFFCFFFF             <1>     call    calculate_fat_freespace
                                <1>     ; ECX = 0 -> no error, ECX > 0 -> error or invalid return
0000C882 21C9                   <1>     and     ecx, ecx ; ECX = 0 -> valid free sector count
0000C884 7409                   <1>     jz      short loc_add_new_cluster_return_cluster_number
                                <1>
                                <1> loc_add_new_cluster_recalc_FAT_freespace:
0000C886 66BB00FF               <1>     mov     bx, 0FF00h  ; recalculate free space
0000C88A E8E2FCFFFF             <1>         call    calculate_fat_freespace
                                <1>     ; cf = 0
                                <1> loc_add_new_cluster_return_cluster_number:
0000C88F 89C1                   <1>     mov     ecx, eax ; Free sector count
0000C891 A1[6C820100]           <1>     mov     eax, [FAT_anc_FFCluster]
                                <1>     ;mov    edi, Cluster_Buffer
                                <1>     ; 25/07/2022 (EBX is not used by callers of this sprocedure)
                                <1>     ;movzx  ebx, byte [esi+LD_BPB+SecPerClust]
0000C896 31D2                   <1>     xor     edx, edx ; 0
0000C898 C3                     <1>         retn
                                <1>
                                <1> write_cluster:
                                <1>     ; 19/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 31/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 15/10/2016
                                <1>     ; 21/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Cluster Number (Sector index for SINGLIX FS)
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ;       EBX = Cluster (File R/W) Buffer address (max. 64KB)
                                <1>     ;       Only for SINGLIX FS:
                                <1>     ;       EDX = File Number (The 1st FDT address)
                                <1>     ; OUTPUT ->
                                <1>     ;       cf = 1 -> Cluster can not be written onto disk
                                <1>     ;           EAX > 0 -> Error number
                                <1>     ;       cf = 0 -> Cluster has been written successfully
                                <1>     ;
                                <1>     ; (Modified registers: EAX, ECX, EBX, EDX)
                                <1>
0000C899 0FB64E13               <1>     movzx   ecx, byte [esi+LD_BPB+BPB_SecPerClust]
                                <1>     ; CL = 1 = [esi+LD_FS_Reserved2] ; SectPerClust for Singlix FS
                                <1>
                                <1> write_file_sectors: ; 16/03/2016
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     jna     short write_fs_cluster
                                <1> %endif
                                <1>
                                <1> write_fat_file_sectors:
                                <1>     ; 31/08/2024
                                <1>     ; ecx = sector count (may be different than sectors per cluster)
0000C89D 83E802                 <1>     sub     eax, 2 ; Beginning cluster number is always 2
0000C8A0 0FB65613               <1>     movzx   edx, byte [esi+LD_BPB+BPB_SecPerClust] ; 18/03/2016
0000C8A4 F7E2                   <1>     mul     edx
0000C8A6 034668                 <1>     add     eax, [esi+LD_DATABegin] ; absolute address of the cluster
                                <1>
                                <1>     ; EAX = Disk sector address
                                <1>     ; ECX = Sector count
                                <1>     ; EBX = Buffer address
                                <1>     ; (EDX = 0)
                                <1>     ; ESI = Logical DOS drive description table address
                                <1>
0000C8A9 E836510000             <1>     call    disk_write
0000C8AE 7306                   <1>     jnc     short wclust_retn
                                <1>
                                <1>     ; 15/10/2016 (1Dh -> 18)
0000C8B0 B812000000             <1>     mov     eax, 18 ; Drive not ready or write error !
0000C8B5 C3                     <1>     retn
                                <1>
                                <1> wclust_retn:
0000C8B6 29C0                   <1>     sub     eax, eax ; 0
0000C8B8 C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>
                                <1> write_fs_cluster:
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 21/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; Singlix FS
                                <1>
                                <1>     ; EAX = Cluster number is sector index number of the file (eax)
                                <1>
                                <1>     ; EDX = File number is the first File Descriptor Table address
                                <1>     ;       of the file. (Absolute address of the FDT).
                                <1>
                                <1>     ; eax = sector index (0 for the first sector)
                                <1>     ; edx = FDT0 address
                                <1>             ; 64 KB buffer = 128 sectors (limit)
                                <1>     ;mov    ecx, 128 ; maximum count of sectors (before eof)
                                <1>     ; 25/07/2022
                                <1>     sub     ecx, ecx
                                <1>     mov     cl, 128
                                <1>     ;call   write_fs_sectors
                                <1>     ;retn
                                <1>     ;jmp    short write_fs_sectors
                                <1>
                                <1> write_fs_sectors:
                                <1>     ; 21/03/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     stc
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> get_cluster_by_index:
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 29/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Beginning cluster
```

```
                                <1>     ;       EDX = Sector index in disk/file section
                                <1>     ;             (Only for SINGLIX file system!)
                                <1>     ;       ECX = Cluster sequence number after the beginning cluster
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = Cluster number
                                <1>     ;       cf = 1 -> Error code in AL (EAX)
                                <1>     ;
                                <1>     ;(Modified registers: EAX, ECX, EBX, EDX)
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 1
                                <1>         jb      short get_fs_section_by_index
                                <1> %endif
0000C8B9 3B4E78                 <1>     cmp     ecx, [esi+LD_Clusters]
0000C8BC 7206                   <1>     jb      short gcbi_1
                                <1> gcbi_0:
                                <1>     ;stc
                                <1>     ;mov    eax, 23h ; Cluster not available !
                                <1>                     ; MSDOS error code: FCB unavailable
                                <1>     ; 25/07/2022
0000C8BE 29C0                   <1>     sub     eax, eax
                                <1> gcbi_4:
0000C8C0 B023                   <1>     mov     al, 23h
0000C8C2 F9                     <1>     stc
0000C8C3 C3                     <1>     retn
                                <1> gcbi_1:
0000C8C4 51                     <1>     push    ecx
0000C8C5 E8BFF6FFFF             <1>     call    get_next_cluster
0000C8CA 59                     <1>     pop     ecx
0000C8CB 7203                   <1>     jc      short gcbi_3
0000C8CD E2F5                   <1>     loop    gcbi_1
                                <1> gcbi_2:
0000C8CF C3                     <1>     retn
                                <1> gcbi_3:
0000C8D0 09C0                   <1>     or      eax, eax
                                <1>     ;jz     short gcbi_0
                                <1>     ; 25/07/2022
0000C8D2 74EC                   <1>     jz      short gcbi_4
0000C8D4 F5                     <1>     cmc     ; stc
0000C8D5 C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>
                                <1> get_fs_section_by_index:
                                <1>     ; 29/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       EAX = Beginning FDT number/address
                                <1>     ;       EDX = Sector index in disk/file section
                                <1>     ;       ECX = Sector sequence number after the beginning FDT
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = FDT number/address
                                <1>     ;       EDX = Sector index of the section (0,1,2,3,4...)
                                <1>     ;       cf = 1 -> Error code in AL (EAX)
                                <1>     ;
                                <1>     ;(Modified registers: EAX, ECX, EBX, EDX)
                                <1>     ;
                                <1>     mov     eax, 0FFFFFFFFh
                                <1>     retn
                                <1>
                                <1> get_last_section:
                                <1>     ; 22/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       EAX = (The 1st) FDT number/address
                                <1>     ;       ESI = Logical DOS Drive Description Table address
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = FDT number/address of the last section
                                <1>     ;       EDX = Last sector of the section (0,1,2,3,4...)
                                <1>     ;       [glc_index] = sector index number of the last sector
                                <1>     ;                     (for file, not for the last section)
                                <1>     ;
                                <1>     ;       cf = 1 -> Error code in AL (EAX)
                                <1>     ;
                                <1>     ;(Modified registers: EAX, ECX, EBX, EDX)
                                <1>     ;
                                <1>     mov     eax, 0
                                <1>     mov     edx, 0
                                <1>     retn
                                <1>
                                <1> %endif
                                    %include 'trdosk6.s' ; 24/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - MAIN PROGRAM : trdosk6.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 10/01/2026  (Previous: 27/09/2024, v2.0.9)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; u1.s (27/17/2015), u2.s (03/01/2016)
                                <1> ; ****************************************************************************
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; INT_21H.ASM (c) 2009-2011 Erdogan TAN  [14/11/2009] Last Update: 08/11/2011
                                <1>
                                <1> ; Ref: Retro UNIX 386 v1.2 Kernel (v0.2.2.3) - ux.s - 15/07/2022
                                <1>
                                <1> sysent: ; < enter to system call >
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 17/03/2017
                                <1>     ; 03/03/2017
                                <1>     ; 19/02/2017
                                <1>     ; 13/01/2017
                                <1>     ; 06/06/2016
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 16/04/2015 - 19/10/2015 (Retro UNIX 386 v1)
                                <1>     ; 10/04/2013 - 18/01/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'unkni' or 'sysent' is sytem entry from various traps.
                                <1>     ; The trap type is determined and an indirect jump is made to
                                <1>     ; the appropriate system call handler. If there is a trap inside
                                <1>     ; the system a jump to panic is made. All user registers are saved
                                <1>     ; and u.sp points to the end of the users stack. The sys (trap)
                                <1>     ; instructor is decoded to get the the system code part (see
                                <1>     ; trap instruction in the PDP-11 handbook) and from this
                                <1>     ; the indirect jump address is calculated. If a bad system call is
                                <1>     ; made, i.e., the limits of the jump table are exceeded, 'badsys'
                                <1>     ; is called. If the call is legitimate control passes to the
```

```
                                <1>     ; appropriate system routine.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       Through a trap caused by any sys call outside the system.
                                <1>     ; Arguments:
                                <1>     ;       Arguments of particular system call.
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;        System call number is in EAX register.
                                <1>     ;
                                <1>     ;        Other parameters are in EDX, EBX, ECX, ESI, EDI, EBP
                                <1>     ;       registers depending of function details.
                                <1>     ;
                                <1>     ; 16/04/2015
0000C8D6 368925[648E0100]       <1>         mov     [ss:u.sp], esp ; Kernel stack points to return address
                                <1>
                                <1>     ; save user registers
0000C8DD 1E                     <1>     push    ds
0000C8DE 06                     <1>     push    es
0000C8DF 0FA0                   <1>     push    fs
0000C8E1 0FA8                   <1>     push    gs
0000C8E3 60                     <1>     pushad  ; eax, ecx, edx, ebx, esp -before pushad-, ebp, esi, edi
                                <1>     ;
                                <1>     ; ESPACE = [ss:u.sp] - esp ; 4*12 = 48 ; 17/09/2015 ; 06/06/2016
                                <1>     ;       (ESPACE is size of space in kernel stack
                                <1>     ;       for saving/restoring user registers.)
                                <1>     ;
0000C8E4 50                     <1>     push    eax ; 01/07/2015
0000C8E5 66B81000               <1>     mov     ax, KDATA
0000C8E9 8ED8                   <1>         mov     ds, ax
0000C8EB 8EC0                   <1>         mov     es, ax
0000C8ED 8EE0                   <1>         mov     fs, ax
0000C8EF 8EE8                   <1>         mov     gs, ax
0000C8F1 A1[B0760100]           <1>     mov     eax, [k_page_dir]
0000C8F6 0F22D8                 <1>     mov     cr3, eax
0000C8F9 58                     <1>     pop     eax ; 01/07/2015
                                <1>     ; 19/10/2015
0000C8FA FC                     <1>     cld
                                <1>     ;
0000C8FB FE05[608E0100]         <1>     inc     byte [sysflg]
                                <1>             ; incb sysflg / indicate a system routine is in progress
0000C901 FB                     <1>         sti         ; 18/01/2014
                                <1>     ;jnz    panic ; 24/05/2013
                                <1>             ; beq 1f
                                <1>             ; jmp panic ; / called if trap inside system
                                <1>     ; 23/07/2022
0000C902 7405                   <1>     jz      short sysent0
0000C904 E910A5FFFF             <1>     jmp     panic
                                <1> ;1:
                                <1> sysent0:
                                <1>     ; 17/03/2017
0000C909 80642438FE             <1>     and     byte [esp+ESPACE+8], ~1 ; clear carry flag
                                <1>
                                <1>     ; 16/04/2015
0000C90E A3[6C8E0100]           <1>     mov     [u.r0], eax
0000C913 8925[688E0100]         <1>     mov     [u.usp], esp ; kernel stack points to user's registers
                                <1>
                                <1>     ; 13/01/2017 (TRDOS 386 Feaure only !)
0000C919 803D[E48E0100]00       <1>     cmp     byte [u.t_lock], 0 ; timer interrupt lock ?
                                <1>     ;ja     sysrele           ; yes, sys release only !!!
                                <1>     ; 23/07/2022
0000C920 7605                   <1>     jna     short sysent1
0000C922 E902020000             <1>     jmp     sysrele
                                <1>             ; mov $s.syst+2,clockp
                                <1>             ; mov r0,-(sp) / save user registers
                                <1>             ; mov sp,u.r0 / pointer to bottom of users stack
                                <1>                             ; / in u.r0
                                <1>             ; mov r1,-(sp)
                                <1>             ; mov r2,-(sp)
                                <1>             ; mov r3,-(sp)
                                <1>             ; mov r4,-(sp)
                                <1>             ; mov r5,-(sp)
                                <1>             ; mov ac,-(sp) / "accumulator" register for extended
                                <1>                             ; / arithmetic unit
                                <1>             ; mov mq,-(sp) / "multiplier quotient" register for the
                                <1>                             ; / extended arithmetic unit
                                <1>             ; mov sc,-(sp) / "step count" register for the extended
                                <1>                             ; / arithmetic unit
                                <1>             ; mov sp,u.sp / u.sp points to top of users stack
                                <1>             ; mov 18.(sp),r0 / store pc in r0
                                <1>             ; mov -(r0),r0 / sys inst in r0
                                <1>             ; sub $sys,r0 / get xxx code
                                <1> sysent1:
0000C927 C1E002                 <1>     shl     eax, 2
                                <1>             ; asl r0 / multiply by 2 to jump indirect in bytes
0000C92A 3DBC000000             <1>     cmp     eax, end_of_syscalls - syscalls
                                <1>             ; cmp r0,$2f-1f / limit of table (35) exceeded
                                <1>     ;jnb    short badsys
                                <1>             ; bhis badsys / yes, bad system call
0000C92F F5                     <1>     cmc
0000C930 9C                     <1>     pushf
0000C931 50                     <1>     push    eax
0000C932 8B2D[648E0100]         <1>     mov     ebp, [u.sp] ; Kernel stack at the beginning of sys call
0000C938 B0FE                   <1>     mov     al, 0FEh ; 11111110b
0000C93A 1400                   <1>     adc     al, 0 ; al = al + cf
0000C93C 204508                 <1>     and     [ebp+8], al ; flags (reset carry flag)
                                <1>             ; bic $341,20.(sp) / set users processor priority to 0
                                <1>                             ; / and clear carry bit
0000C93F 5D                     <1>     pop     ebp ; eax
0000C940 9D                     <1>     popf
0000C941 720B                   <1>         jc      short badsys ; 23/07/2022
0000C943 A1[6C8E0100]           <1>     mov     eax, [u.r0]
                                <1>     ; system call registers: EAX, EDX, ECX, EBX, ESI, EDI
0000C948 FFA5[9BC90000]         <1>     jmp     dword [ebp+syscalls]
                                <1>             ; jmp *1f(r0) / jump indirect thru table of addresses
                                <1>                         ; / to proper system routine.
                                <1>
                                <1>     ; 20/08/2024 (exit code)
                                <1>     ; 30/07/2022
                                <1>     ; 23/07/2022
                                <1> badsys:
                                <1>     ; 25/12/2016
                                <1>     ; 18/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 17/04/2011 (TRDOS v1.0, 'IFC.ASM')
                                <1>     ; 03/02/2011 ('trdos_ifc_routine')
                                <1>     ;
                                <1>     ; 16/04/2015 (Retro UNIX 386 v1, 'badsys')
                                <1>     ; (EIP, EAX values will be shown on screen with error message)
                                <1>     ; (EIP = 'CD 40h' instruction address -INT 40h-)
                                <1>     ; (EAX = Function number)
                                <1>     ;
0000C94E FE05[BC8E0100]         <1>     inc     byte [u.bsys]
                                <1>     ;
```

```
0000C954 8B1D[648E0100]      <1>     mov     ebx, [u.sp] ; esp at the beginning of 'sysent'
0000C95A 8B03                <1>     mov     eax, [ebx] ; EIP (return address, not 'INT 30h' address)
                             <1>     ;sub    eax, 2 ; CDh, ##h
                             <1>     ; 30/07/2022
0000C95C 48                  <1>     dec     eax
0000C95D 48                  <1>     dec     eax
0000C95E E89C78FFFF          <1>     call    dwordtohex
0000C963 8915[A4350100]      <1>     mov     [eip_str], edx
0000C969 A3[A8350100]        <1>     mov     [eip_str+4], eax
0000C96E A1[6C8E0100]        <1>     mov     eax, [u.r0]
0000C973 E88778FFFF          <1>     call    dwordtohex
0000C978 8915[93350100]      <1>     mov     [eax_str], edx
0000C97E A3[97350100]        <1>     mov     [eax_str+4], eax
                             <1>
0000C983 66C705[88350100]34- <1>     mov     word [int_num_str], SYSCALL_INT_NUM ; 25/12/2016
0000C98B 30                  <1>
                             <1>
0000C98C BE[5A350100]        <1>     mov     esi, ifc_msg ; "invalid function call !" msg (trdosk9.s)
                             <1>
0000C991 E824A4FFFF          <1>     call    print_msg
                             <1>
                             <1> ;; 20/08/2024 - temporary
                             <1> ;   mov     ebx, 07h
                             <1> ;   mov     ah, 0Eh
                             <1> ;p_fc_msg:
                             <1> ;   lodsb
                             <1> ;   and     al, al
                             <1> ;   jz      short p_fc_ms_ok
                             <1> ;   call    _int10h
                             <1> ;   jmp     short p_fc_msg
                             <1> ;p_fc_ms_ok:
                             <1>
                             <1>     ;jmp    sysexit
                             <1>
                             <1>     ; 20/08/2024 (exit code)
                             <1>     ;mov    bl, 0FFh ; -1
                             <1>     ;jmp    sysexit
                             <1>     ;
0000C996 E92C020000          <1>     jmp     sysexit_0FFh
                             <1>
                             <1> syscalls: ; 1:
                             <1>     ; 20/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; 31/12/2017
                             <1>     ; 28/02/2017
                             <1>     ; 20/02/2017
                             <1>     ; 19/02/2017
                             <1>     ; 15/10/2016
                             <1>     ; 20/05/2016
                             <1>     ; 19/05/2016
                             <1>     ; 16/05/2016
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 21/09/2015
                             <1>     ; 01/07/2015
                             <1>     ; 16/04/2015 (32 bit address modification)
0000C99B [670A0100]          <1>     dd sysver      ; 0 ; Get TRDOS 386 version number (v2.0)
0000C99F [CCCB0000]          <1>     dd sysexit     ; 1
0000C9A3 [F4CD0000]          <1>     dd sysfork     ; 2
0000C9A7 [C1D10000]          <1>     dd sysread     ; 3
0000C9AB [E0D10000]          <1>     dd syswrite    ; 4
0000C9AF [DECF0000]          <1>     dd sysopen     ; 5
0000C9B3 [97D10000]          <1>     dd sysclose    ; 6
0000C9B7 [58CD0000]          <1>     dd syswait     ; 7
0000C9BB [01CF0000]          <1>     dd syscreat    ; 8
0000C9BF [53170100]          <1>     dd sysrename   ; 9  ; TRDOS 386, Rename File (31/12/2017)
0000C9C3 [00130100]          <1>     dd sysdelete   ; 10 ; TRDOS 386, Delete File (29/12/2017)
0000C9C7 [8EFE0000]          <1>     dd sysexec     ; 11
0000C9CB [30140100]          <1>     dd syschdir    ; 12
0000C9CF [BD150100]          <1>     dd systime     ; 13 ; TRDOS 386, Get Sys Date&Time (30/12/2017)
0000C9D3 [5AD10000]          <1>     dd sysmkdir    ; 14
0000C9D7 [64140100]          <1>     dd syschmod    ; 15 ; TRDOS 386, Change Attributes (30/12/2017)
0000C9DB [67130100]          <1>     dd sysrmdir    ; 16 ; TRDOS 386, Remove Directory (29/12/2017)
0000C9DF [16030100]          <1>     dd sysbreak    ; 17
0000C9E3 [12150100]          <1>     dd sysdrive    ; 18 ; TRDOS 386, Get/Set Current Drv (30/12/2017)
0000C9E7 [85030100]          <1>     dd sysseek     ; 19
0000C9EB [9C030100]          <1>     dd systell     ; 20
0000C9EF [5E180100]          <1>     dd sysmem      ; 21 ; TRDOS 386, Get Total&Free Mem (31/12/2017)
0000C9F3 [94180100]          <1>     dd sysprompt   ; 22 ; TRDOS 386, Change Cmd Prompt (31/12/2017)
0000C9F7 [D5180100]          <1>     dd syspath     ; 23 ; TRDOS 386, Get/Set Run Path (31/12/2017)
0000C9FB [3A190100]          <1>     dd sysenv      ; 24 ; TRDOS 386, Get/Set Env Vars (31/12/2017)
0000C9FF [3E160100]          <1>     dd sysstime    ; 25 ; TRDOS 386, Set Sys Date&Time (30/12/2017)
0000CA03 [FB030100]          <1>     dd sysquit     ; 26
0000CA07 [E4030100]          <1>     dd sysintr     ; 27
0000CA0B [61150100]          <1>     dd sysdir      ; 28 ; TRDOS 386, Get Curr Drive&Dir (30/12/2017)
0000CA0F [57CA0000]          <1>     dd sysemt      ; 29
0000CA13 [9C150100]          <1>     dd sysldrvt    ; 30 ; TRDOS 386, Get Logical DOS DDT (30/12/2017)
0000CA17 [DBD30000]          <1>     dd sysvideo    ; 31 ; TRDOS 386 Video Functions (16/05/2016)
0000CA1B [62200100]          <1>     dd sysaudio    ; 32 ; TRDOS 386 Audio Functions (16/05/2016)
0000CA1F [46D20000]          <1>     dd systimer    ; 33 ; TRDOS 386 Timer Functions (18/05/2016)
0000CA23 [17040100]          <1>     dd syssleep    ; 34 ; Retro UNIX 8086 v1 feature only !
                             <1>                           ; 11/06/2014
0000CA27 [57040100]          <1>     dd sysmsg      ; 35 ; Retro UNIX 386 v1 feature only !
                             <1>                           ; 01/07/2015
0000CA2B [72050100]          <1>     dd sysgeterr   ; 36 ; Retro UNIX 386 v1 feature only !
                             <1>                           ; 21/09/2015 - get last error number
0000CA2F [D7120100]          <1>     dd sysfpstat   ; 37 ; TRDOS 386 FPU state option (28/02/2017)
0000CA33 [760A0100]          <1>     dd syspri      ; 38 ; change priority - TRDOS 386 (20/05/2016)
0000CA37 [29CB0000]          <1>     dd sysrele     ; 39 ; TRDOS 386 (19/05/2016) (0 -> 39)
0000CA3B [A10B0100]          <1>     dd sysfff      ; 40 ; Find First File - TRDOS 386 (15/10/2016)
0000CA3F [7B0C0100]          <1>     dd sysfnf      ; 41 ; Find Next File - TRDOS 386 (15/10/2016)
0000CA43 [C5110100]          <1>     dd sysalloc    ; 42 ; Allocate contiguous memory block/pages
                             <1>                           ; TRDOS 386 (19/02/2017) DMA buff fuctions
0000CA47 [7F120100]          <1>     dd sysdalloc   ; 43 ; Deallocate contiguous memory block/pages
                             <1>                           ; TRDOS 386 (19/02/2017) DMA buff fuctions
0000CA4B [BA120100]          <1>     dd syscalbac   ; 44 ; IRQ Callback and Signal Response Byte
                             <1>                           ; service setup - TRDOS 386 (20/02/2017)
                             <1>                           ; 28/08/2017 (20/08/2017)
0000CA4F [43290100]          <1>     dd sysdma      ; 45 ; TRDOS 386 - (ISA) DMA service
0000CA53 [7F2C0100]          <1>     dd sysstdio    ; 46 ; TRDOS 386 v2.0.9 (STDIN/STDOUT functions)
                             <1>
                             <1> end_of_syscalls:
                             <1>
                             <1> sysemt: ; enable (or disable) multi tasking -time sharing-
                             <1>     ;
                             <1>     ; 08/08/2022
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 23/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 14/05/2015 (Retro UNIX 386 v1)
                             <1>     ; 10/12/2013 - 20/04/2014 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; Retro UNIX 8086 v1 modification:
                             <1>     ;       'Enable Multi Tasking'  system call instead
                             <1>     ;       of 'Emulator Trap' in original UNIX v1 for PDP-11.
                             <1>     ;
                             <1>     ; Retro UNIX 8086 v1 feature only!
```

```
                            <1>     ;       Using purpose: Kernel will start without time-out
                            <1>     ;       (internal clock/timer) functionality.
                            <1>     ;       Then etc/init will enable clock/timer for
                            <1>     ;       multi tasking.
                            <1>     ;
                            <1>     ; INPUT ->
                            <1>     ;       BL = 0 -> disable multi tasking
                            <1>     ;       BL > 1 -> enable multi tasking (time sharing)
                            <1>     ; OUTPUT ->
                            <1>     ;       none
                            <1>     ;
                            <1>     ;  Note: Multi tasking is disabled during system
                            <1>     ;        initialization, it must be enabled by using
                            <1>     ;        this system call. (Otherwise, running proces
                            <1>     ;        will not be changed by another process within
                            <1>     ;        run time sequence/schedule, if running process
                            <1>     ;        will not 'release' itself. Only 'wakeup' procedure
                            <1>     ;        for waiting processes and programmed timer events
                            <1>     ;        for other processes can change running process
                            <1>     ;        while multi tasking is disabled.) ** 23/05/2016 **
                            <1>
0000CA57 803D[BE8E0100]00   <1>     cmp     byte [u.uid], 0 ; root ?
                            <1>     ;ja     short error
                            <1>     ; 23/07/2022
                            <1>     ;ja     short badsys ; 14/05/2015
                            <1>     ; 08/08/2022
0000CA5E 7605               <1>     jna     short sysemt_root
0000CA60 E9E9FEFFFF         <1>     jmp     badsys
                            <1> sysemt_root:        ; 08/08/2022
0000CA65 FA                 <1>     cli
0000CA66 881D[46830100]     <1>     mov     [multi_tasking], bl ; 0 to disable, >0 to enable
0000CA6C EB20               <1>     jmp     sysret
                            <1>
                            <1> error:
                            <1>     ; 18/05/2016
                            <1>     ; 13/05/2016
                            <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                            <1>     ; 16/04/2015 - 17/09/2015 (Retro UNIX 386 v1)
                            <1>     ; 10/04/2013 - 07/08/2013 (Retro UNIX 8086 v1)
                            <1>     ;
                            <1>     ; 'error' merely sets the error bit off the processor status (c-bit)
                            <1>     ; then falls right into the 'sysret', 'sysrele' return sequence.
                            <1>     ;
                            <1>     ; INPUTS -> none
                            <1>     ; OUTPUTS ->
                            <1>     ;       processor status - carry (c) bit is set (means error)
                            <1>     ;
                            <1>     ; 26/05/2013 (Stack pointer must be reset here!
                            <1>     ;             Because, jumps to error procedure
                            <1>     ;             disrupts push-pop nesting balance)
                            <1>     ;
0000CA6E 8B2D[648E0100]     <1>     mov     ebp, [u.sp] ; interrupt (system call) return (iretd) address
0000CA74 804D0801           <1>     or      byte [ebp+8], 1  ; set carry bit of flags register
                            <1>                              ; (system call will return with cf = 1)
                            <1>             ; bis $1,20.(r1) / set c bit in processor status word below
                            <1>                              ; / users stack
                            <1>     ; 17/09/2015
0000CA78 83ED30             <1>     sub     ebp, ESPACE ; 48 ; total size of stack frame ('sysdefs.inc')
                            <1>                              ; for saving/restoring user registers
                            <1>     ;cmp    ebp, [u.usp]
                            <1>     ;je     short err0
0000CA7B 892D[688E0100]     <1>     mov     [u.usp], ebp
                            <1> ;err0:
                            <1>     ; 01/09/2015
0000CA81 8B25[688E0100]     <1>     mov     esp, [u.usp]        ; Retro Unix 8086 v1 modification!
                            <1>                                 ; 10/04/2013
                            <1>                                 ; (If an I/O error occurs during disk I/O,
                            <1>                                 ; related procedures will jump to 'error'
                            <1>                                 ; procedure directly without returning to
                            <1>                                 ; the caller procedure. So, stack pointer
                            <1>                                      ; must be restored here.)
                            <1>     ; 13/05/2016
                            <1>     ; NOTE: (The last) error code is in 'u.error', it can be retrieved by
                            <1>     ;       'get last error' system call later.
                            <1>
                            <1>     ; 03/09/2015 - 09/06/2015 - 07/08/2013
0000CA87 C605[D28E0100]00   <1>     mov     byte [u.kcall], 0 ; namei_r, mkdir_w reset
                            <1>
                            <1> sysret: ; < return from system call>
                            <1>     ; 28/08/2024 - TRDOS 386 Kernel v2.0.9
                            <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                            <1>     ; 01/03/2017
                            <1>     ; 28/02/2017
                            <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                            <1>     ; 16/04/2015 - 10/09/2015 (Retro UNIX 386 v1)
                            <1>     ; 10/04/2013 - 23/02/2014 (Retro UNIX 8086 v1)
                            <1>     ;
                            <1>     ; 'sysret' first checks to see if process is about to be
                            <1>     ; terminated (u.bsys). If it is, 'sysexit' is called.
                            <1>     ; If not, following happens:
                            <1>     ;       1) The user's stack pointer is restored.
                            <1>     ;       2) r1=0 and 'iget' is called to see if last mentioned
                            <1>     ;          i-node has been modified. If it has, it is written out
                            <1>     ;          via 'ppoke'.
                            <1>     ;       3) If the super block has been modified, it is written out
                            <1>     ;          via 'ppoke'.
                            <1>     ;       4) If the dismountable file system's super block has been
                            <1>     ;          modified, it is written out to the specified device
                            <1>     ;          via 'ppoke'.
                            <1>     ;       5) A check is made if user's time quantum (uquant) ran out
                            <1>     ;          during his execution. If so, 'tswap' is called to give
                            <1>     ;          another user a chance to run.
                            <1>     ;       6) 'sysret' now goes into 'sysrele'.
                            <1>     ;           (See 'sysrele' for conclusion.)
                            <1>     ;
                            <1>     ; Calling sequence:
                            <1>     ;       jump table or 'br sysret'
                            <1>     ; Arguments:
                            <1>     ;       -
                            <1>     ; ..............................................................
                            <1>     ;
                            <1>     ; ((AX=r1 for 'iget' input))
                            <1>     ;
0000CA8E 31C0               <1>     xor     eax, eax ; 28/02/2017
                            <1> sysret0: ; 29/07/2015 (eax = 0, jump from sysexec)
0000CA90 FEC0               <1>     inc     al ; 04/05/2013
0000CA92 3805[BC8E0100]     <1>     cmp     [u.bsys], al ; 1
                            <1>             ; tstb u.bsys / is a process about to be terminated because
                            <1>         ;jnb       sysexit ; 04/05/2013
                            <1>             ; bne sysexit / of an error? yes, go to sysexit
                            <1>     ; 23/07/2022
0000CA98 7205               <1>     jb      short sysret1
                            <1>     ;jmp    sysexit
                            <1>     ; 22/08/2024
```

```
0000CA9A E92B010000          <1>     jmp    sysexit_@ ; BL = 0FFh ; -1
                             <1> sysret1:    ; 23/07/2022
                             <1>     ;mov   esp, [u.usp] ; 24/05/2013 (that is not needed here)
                             <1>            ; mov u.sp,sp / no point stack to users stack
0000CA9F FEC8                <1>     dec    al ; mov ax, 0
                             <1>            ; clr r1 / zero r1 to check last mentioned i-node
0000CAA1 E83D4F0000          <1>     call   iget
                             <1>            ; jsr r0,iget / if last mentioned i-node has been modified
                             <1>                          ; / it is written out
                             <1>     ; 10/01/2017
                             <1>     ; 09/01/2017
                             <1> ;sysrele: ; < release >
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 16/04/2015 - 14/10/2015 (Retro UNIX 386 v1)
                             <1>     ; 10/04/2013 - 07/03/2014 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; 'sysrele' first calls 'tswap' if the time quantum for a user is
                             <1>     ;  zero (see 'sysret'). It then restores the user's registers and
                             <1>     ; turns off the system flag. It then checked to see if there is
                             <1>     ; an interrupt from the user by calling 'isintr'. If there is,
                             <1>     ; the output gets flashed (see isintr) and interrupt action is
                             <1>     ; taken by a branch to 'intract'. If there is no interrupt from
                             <1>     ; the user, a rti is made.
                             <1>     ;
                             <1>     ; Calling sequence:
                             <1>     ;      Fall through a 'bne' in 'sysret' & ?
                             <1>     ; Arguments:
                             <1>     ;      -
                             <1>     ; ............................................................
                             <1>     ;
                             <1>     ; 23/02/2014 (swapret)
                             <1>     ; 22/09/2013
                             <1> sysrel0: ;1:
0000CAA6 803D[B48E0100]00    <1>     cmp    byte [u.quant], 0 ; 16/05/2013
                             <1>            ; tstb uquant / is the time quantum 0?
0000CAAD 7705                <1>         ja      short swapret
                             <1>            ; bne 1f / no, don't swap it out
                             <1> sysrelease: ; 07/12/2013 (jump from 'clock')
0000CAAF E87C3E0000          <1>     call   tswap
                             <1>            ; jsr r0,tswap / yes, swap it out
                             <1>
                             <1> ; Retro Unix 8086 v1 feature: return from 'swap' to 'swapret' address.
                             <1> swapret: ;1:
                             <1>     ; 10/09/2015
                             <1>     ; 01/09/2015
                             <1>     ; 14/05/2015
                             <1>     ; 16/04/2015 (Retro UNIX 386 v1 - 32 bit, pm modifications)
                             <1>     ; 26/05/2013 (Retro UNIX 8086 v1)
                             <1>     ; cli
                             <1>     ; 24/07/2015
                             <1>     ;
                             <1>     ;; 'esp' must be already equal to '[u.usp]' here !
                             <1>     ;; mov esp, [u.usp]
                             <1>
                             <1>     ; 22/09/2013
0000CAB4 E8234F0000          <1>     call   isintr
                             <1>     ; 20/10/2013
0000CAB9 7405                <1>     jz     short sysrel1
0000CABB E8F4000000          <1>     call   intract
                             <1>            ; jsr r0,isintr / is there an interrupt from the user
                             <1>            ;     br intract / yes, output gets flushed, take interrupt
                             <1>                          ; / action
                             <1> sysrel1:
0000CAC0 FA                  <1>     cli    ; 14/10/2015
                             <1> sysrel2:
                             <1>     ; 28/02/2017
                             <1>     ; Check if there is a (delayed) callback for current user/process
0000CAC1 A0[E78E0100]        <1>     mov    al, [u.irqwait]
0000CAC6 240F                <1>     and    al, 0Fh ; is there a waiting IRQ callback service ?
0000CAC8 7444                <1>     jz     short sysrel8 ; no
                             <1>
                             <1>     ; Set return to IRQ callback service and return from the service
0000CACA 0FB6D8              <1>     movzx  ebx, al
0000CACD 883D[E78E0100]      <1>     mov    [u.irqwait], bh ; 0 ; reset
0000CAD3 8A9B[B0350100]      <1>     mov    bl, [ebx+IRQenum] ; (available) IRQ index +1 (1 to 9)
                             <1>     ; 01/03/2017
0000CAD9 FECB                <1>     dec    bl ; IRQ index number, 0 to 8
0000CADB 7831                <1>     js     short sysrel8 ; 0 -> FFh (not in use!?)
                             <1>     ;
0000CADD A0[BD8E0100]        <1>     mov    al, [u.uno] ; current process (user) number
0000CAE2 3883[94880100]      <1>     cmp    [ebx+IRQ.owner], al
0000CAE8 7524                <1>     jne    short sysrel8 ; it is not the current user/process !?
0000CAEA F683[A6880100]01    <1>     test   byte [ebx+IRQ.method], 1 ; callback ?
0000CAF1 741B                <1>     jz     short sysrel8 ; not a callback method !?
                             <1>
0000CAF3 8B93[B8880100]      <1>     mov    edx, [ebx+IRQ.addr] ; IRQ callback service address (virtual)
0000CAF9 C605[E88E0100]01    <1>     mov    byte [u.r_lock], 1 ; IRQ callback service in progress flag
                             <1>
0000CB00 E8D23E0000          <1>     call   wswap ; save user's registers & status
                             <1>                  ;(for return from IRQ callback service)
                             <1>
0000CB05 8B2D[648E0100]      <1>     mov    ebp, [u.sp]; kernel's stack, points to EIP (user)
0000CB0B 895500              <1>     mov    [ebp], edx ; IRQ call back service address
                             <1> sysrel8:
0000CB0E FE0D[608E0100]      <1>     dec    byte [sysflg]
                             <1>            ; decb sysflg / turn system flag off
                             <1>
0000CB14 A1[C48E0100]        <1>     mov    eax, [u.pgdir]
0000CB19 0F22D8              <1>     mov    cr3, eax  ; 1st PDE points to Kernel Page Table 0 (1st 4 MB)
                             <1>                      ; (others are different than kernel page tables)
                             <1>     ; 10/09/2015
0000CB1C 61                  <1>     popad ; edi, esi, ebp, temp (icrement esp by 4), ebx, edx, ecx, eax
                             <1>            ; mov (sp)+,sc / restore user registers
                             <1>            ; mov (sp)+,mq
                             <1>            ; mov (sp)+,ac
                             <1>            ; mov (sp)+,r5
                             <1>            ; mov (sp)+,r4
                             <1>            ; mov (sp)+,r3
                             <1>            ; mov (sp)+,r2
                             <1>     ;
0000CB1D A1[6C8E0100]        <1>     mov    eax, [u.r0]  ; ((return value in EAX))
0000CB22 0FA9                <1>     pop    gs
0000CB24 0FA1                <1>     pop    fs
0000CB26 07                  <1>     pop    es
0000CB27 1F                  <1>     pop    ds
                             <1>     ;or    word [esp+8], 200h ; 22/01/2017 ; force enabling interrupts
0000CB28 CF                  <1>     iretd
                             <1>            ; rti / no, return from interrupt
                             <1>
                             <1> sysrele:
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 24/03/2017
                             <1>     ; 28/02/2017
                             <1>     ; 27/02/2017
```

```
                                <1>     ; 29/01/2017
                                <1>     ; 14/01/2017
                                <1>     ; 13/01/2017
                                <1>     ; 09/01/2017 - 10/01/2017 - 12/01/2017
                                <1>     ; Major modification for TRDOS 386 (CallBack return)
                                <1>     ;
                                <1>     ; 'sysrele' system call restores previously saved
                                <1>     ; registers and addresses of the process
                                <1>     ; (Main purpose -in TRDOS 386- is to return from
                                <1>     ; timer callback service routine in ring 3 -user mode-.)
                                <1>     ;
                                <1>     ; check if the process is in timer callback phase
0000CB29 803D[E48E0100]00       <1>     cmp    byte [u.t_lock], 0 ; TIMER INT LOCK
                                <1>     ;je    short sysrel0 ; classic (Retro UNIX 386 type) sysrele
0000CB30 7735                   <1>     ja     short sysrel3
                                <1>     ; 27/02/2017
0000CB32 803D[E88E0100]00       <1>     cmp    byte [u.r_lock], 0 ; IRQ callback lock
                                <1>     ;jna   sysrel0 ; classic sysrele ; 24/03/2017
                                <1>     ; 23/07/2022
0000CB39 7705                   <1>     ja     short sysrel9
0000CB3B E966FFFFFF             <1>     jmp    sysrel0
                                <1> sysrel9:   ; 23/07/2022
0000CB40 E859000000             <1>     call   sysrel7
0000CB45 803D[E88E0100]00       <1>     cmp    byte [u.r_lock], 0 ; IRQ callback service lock
0000CB4C 7628                   <1>     jna    short sysrel4
0000CB4E C605[E88E0100]00       <1>     mov    byte [u.r_lock], 0 ; reset
                                <1>     ;mov   byte [u.irqwait], 0 ; reset ; 28/02/2017
0000CB55 A0[E98E0100]           <1>     mov    al, [u.r_mode]
0000CB5A 08C0                   <1>     or     al, al
0000CB5C 7518                   <1>     jnz    short sysrel4
0000CB5E FEC8                   <1>     dec    al
0000CB60 A2[E98E0100]           <1>     mov    [u.r_mode], al ; 0FFh ; not necessary !?
0000CB65 EB32                   <1>     jmp    short sysrel6
                                <1> sysrel3:
                                <1>     ; 27/02/2017
0000CB67 E832000000             <1>     call   sysrel7
                                <1>     ; 14/01/2017
0000CB6C 28C0                   <1>     sub    al, al
0000CB6E 3805[E48E0100]         <1>     cmp    [u.t_lock], al ; 0 ; TIMER INT LOCK
0000CB74 770E                   <1>     ja     short sysrel5 ; yes
                                <1> sysrel4:
                                <1>     ; 29/01/2017
0000CB76 8B44241C               <1>     mov    eax, [esp+28] ; eax
0000CB7A A3[6C8E0100]           <1>     mov    [u.r0], eax
0000CB7F E93DFFFFFF             <1>     jmp    sysrel2
                                <1> sysrel5:
0000CB84 A2[E48E0100]           <1>     mov    [u.t_lock], al ; 0 ; reset
0000CB89 A0[E58E0100]           <1>     mov    al, [u.t_mode]
0000CB8E 20C0                   <1>     and    al, al
                                <1>     ;jnz   short sysrel2 ; 0FFh ; user mode
0000CB90 75E4                   <1>     jnz    short sysrel4 ; 29/01/2017
0000CB92 FEC8                   <1>     dec    al
0000CB94 A2[E58E0100]           <1>     mov    [u.t_mode], al ; 0FFh ; not necessary !?
                                <1> sysrel6:
                                <1>     ; cpu will continue from the interrupted sytem call addr
0000CB99 61                     <1>     popad          ; edi, esi, ebp, esp, ebx, edx, ecx, eax
0000CB9A 83C410                 <1>     add    esp, 16 ; pass segment segisters: ds, es, fs, gs
0000CB9D CF                     <1>     iretd          ; eip, cs, eflags
                                <1>
                                <1> sysrel7:
0000CB9E 0FB61D[BD8E0100]       <1>     movzx  ebx, byte [u.uno] ; current process number
                                <1>     ;shl   bx, 2
                                <1>     ; 23/07/2022
0000CBA5 C1E302                 <1>     shl    ebx, 2
                                <1>     ;cmp   [ebx+p.tcb-4], eax ; 0 ; is there callback address ?
                                <1>     ;jna   short sysrel0
                                <1>     ; yes, reset callback address then restore process registers
                                <1>     ;mov   [ebx+p.tcb-4], eax ; 0 ; reset
0000CBA8 8B83[AC8D0100]         <1>     mov     eax, [ebx+p.upage-4] ; UPAGE address
0000CBAE FA                     <1>     cli    ; disable interrupts till 'iretd'
0000CBAF E95B3E0000             <1>     jmp    rswap ; restore process 'u' structure
                                <1>
                                <1> intract: ; / interrupt action
                                <1>     ; 14/10/2015
                                <1>     ; 16/04/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 09/05/2013 - 07/12/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification !
                                <1>     ; (Process/task switching and quit routine by using
                                <1>     ; Retro UNIX 8086 v1 keyboard interrupt output.))
                                <1>     ;
                                <1>     ; input -> 'u.quit' (also value of 'u.intr' > 0)
                                <1>     ; output -> If value of 'u.quit' = FFFFh ('ctrl+brk' sign)
                                <1>     ;              'intract' will jump to 'sysexit'.
                                <1>     ;          Intract will return to the caller
                                <1>     ;              if value of 'u.quit' <> FFFFh.
                                <1>     ; 14/10/2015
0000CBB4 FB                     <1>     sti
                                <1>     ; 07/12/2013
0000CBB5 66FF05[BA8E0100]       <1>     inc    word [u.quit]
0000CBBC 7408                   <1>     jz     short intrct0 ; FFFFh -> 0
0000CBBE 66FF0D[BA8E0100]       <1>     dec    word [u.quit]
                                <1>     ; 16/04/2015
0000CBC5 C3                     <1>     retn
                                <1> intrct0:
0000CBC6 58                     <1>     pop    eax ; call intract -> retn
                                <1>     ;
                                <1>     ; 20/08/2024
                                <1>     ;xor   eax, eax
                                <1>     ;inc   al  ; mov ax, 1
                                <1> ;;;
                                <1>     ; UNIX v1 original 'intract' routine...
                                <1>     ; / interrupt action
                                <1>            ;cmp *(sp),$rti / are you in a clock interrupt?
                                <1>            ; bne 1f / no, 1f
                                <1>            ; cmp (sp)+,(sp)+ / pop clock pointer
                                <1>     ; 1: / now in user area
                                <1>            ; mov r1,-(sp) / save r1
                                <1>            ; mov u.ttyp,r1
                                <1>                   ; / pointer to tty buffer in control-to r1
                                <1>            ; cmpb 6(r1),$177
                                <1>                   ; / is the interrupt char equal to "del"
                                <1>            ; beq 1f / yes, 1f
                                <1>            ; clrb 6(r1)
                                <1>                    ; / no, clear the byte
                                <1>                   ; / (must be a quit character)
                                <1>            ; mov (sp)+,r1 / restore r1
                                <1>            ; clr u.quit / clear quit flag
                                <1>            ; bis $20,2(sp)
                                <1>                   ; / set trace for quit (sets t bit of
                                <1>                   ; / ps-trace trap)
                                <1>            ; rti   ;  / return from interrupt
                                <1>     ; 1: / interrupt char = del
                                <1>            ; clrb 6(r1) / clear the interrupt byte
```

```
                                <1>                         ; / in the buffer
                                <1>             ; mov (sp)+,r1 / restore r1
                                <1>             ; cmp u.intr,$core / should control be
                                <1>                             ; / transferred to loc core?
                                <1>             ; blo 1f
                                <1>             ; jmp *u.intr / user to do rti yes,
                                <1>                             ; / transfer to loc core
                                <1>     ; 1:
                                <1>             ; sys 1 / exit
                                <1>
                                <1>     ; 20/08/2024
                                <1>     ; ctrl+break -> exit code = -1
                                <1> sysexit_0FFh:
                                <1>     ; 20/08/2024
0000CBC7 31C0                   <1>     xor     eax, eax
0000CBC9 40                     <1>     inc     eax     ; mov eax, 1
                                <1> sysexit_@:  ; 22/08/2024
0000CBCA B3FF                   <1>     mov     bl, 0FFh ;  exit code ; -1
                                <1>
                                <1> sysexit: ; <terminate process>
                                <1>     ; 08/01/2026 - TRDOS 386 v2.0.10
                                <1>     ; 22/08/2024
                                <1>     ; 20/08/2024
                                <1>     ; 18/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 30/07/2022
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 14/11/2017
                                <1>     ; 27/05/2017
                                <1>     ; 10/04/2017
                                <1>     ; 26/02/2017 - 28/02/2017
                                <1>     ; 02/01/2017 - 23/01/2017
                                <1>     ; 06/06/2016 - 10/06/2016
                                <1>     ; 19/05/2016 - 23/05/2016
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 16/04/2015 - 01/09/2015 (Retro UNIX 386 v1)
                                <1>     ; 19/04/2013 - 14/02/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysexit' terminates a process. First each file that
                                <1>     ; the process has opened is closed by 'flose'. The process
                                <1>     ; status is then set to unused. The 'p.pid' table is then
                                <1>     ; searched to find children of the dying process. If any of
                                <1>     ; children are zombies (died by not waited for), they are
                                <1>     ; set free. The 'p.pid' table is then searched to find the
                                <1>     ; dying process's parent. When the parent is found, it is
                                <1>     ; checked to see if it is free or it is a zombie. If it is
                                <1>     ; one of these, the dying process just dies. If it is waiting
                                <1>     ; for a child process to die, it notified that it doesn't
                                <1>     ; have to wait anymore by setting it's status from 2 to 1
                                <1>     ; (waiting to active). It is awakened and put on runq by
                                <1>     ; 'putlu'. The dying process enters a zombie state in which
                                <1>     ; it will never be run again but stays around until a 'wait'
                                <1>     ; is completed by it's parent process. If the parent is not
                                <1>     ; found, process just dies. This means 'swap' is called with
                                <1>     ; 'u.uno=0'. What this does is the 'wswap' is not called
                                <1>     ; to write out the process and 'rswap' reads the new process
                                <1>     ; over the one that dies..i.e., the dying process is
                                <1>     ; overwritten and destroyed.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysexit or conditional branch.
                                <1>     ; Arguments:
                                <1>     ;       -
                                <1>     ; ..............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       System call number (=1) is in EAX register.
                                <1>     ;
                                <1>     ;       Other parameters are in EDX, EBX, ECX, ESI, EDI, EBP
                                <1>     ;       registers depending of function details.
                                <1>     ;
                                <1>     ; ('swap' procedure is mostly different than original UNIX v1.)
                                <1>     ;
                                <1> ; / terminate process
                                <1>     ; AX = 1
                                <1>     ;dec    ax ; 0
                                <1>     ; 18/08/2024
0000CBCC 48                     <1>     dec     eax
0000CBCD 66A3[B88E0100]         <1>     mov     [u.intr], ax ; 0
                                <1>             ; clr u.intr / clear interrupt control word
                                <1>             ; clr r1 / clear r1
                                <1>     ; 18/08/2024 - exit code (in EBX, BL)
0000CBD3 881D[EA8E0100]         <1>     mov     [u.exit], bl
                                <1> sysexit_0:
                                <1>     ; 08/01/2026 (BugFix)
                                <1>     ; 30/07/2022
                                <1>     ; 23/01/2017
                                <1>     ; 02/01/2017
                                <1>     ; 10/06/2016
                                <1>     ; 06/06/2016
                                <1>     ; 23/05/2016
                                <1>     ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; Check and stop/clear timer event(s) of this (dying) process
                                <1>     ; if there is.
                                <1>
                                <1>     ; 02/01/2017
0000CBD9 FA                     <1>     cli     ; disable interrupts
                                <1>     ; 23/01/2017 - reset timer frequency (to 18.2Hz)
0000CBDA B036                   <1>     mov     al, 00110110b ; 36h
0000CBDC E643                   <1>     out     43h, al
0000CBDE 28C0                   <1>     sub     al, al ; 0
0000CBE0 E640                   <1>     out     40h, al ; LB
0000CBE2 E640                   <1>     out     40h, al ; HB
                                <1>     ;
0000CBE4 0FB61D[BD8E0100]       <1>     movzx   ebx, byte [u.uno]
                                <1>     ;mov    bl, [u.uno] ; process number of dying process
0000CBEB 3883[EF8D0100]         <1>     cmp     byte [ebx+p.timer-1], al ; 0
0000CBF1 7639                   <1>     jna     short sysexit_12 ; no timer events for this process
0000CBF3 8883[EF8D0100]         <1>     mov     byte [ebx+p.timer-1], al ; 0 ; reset
                                <1>     ;mov    al, [timer_events]
                                <1>     ;or     al, al
                                <1>     ;jz     short sysexit_12 ; no timer events
                                <1>     ;mov    cl, al
0000CBF9 8A0D[47830100]         <1>     mov     cl, [timer_events] ; 14/11/2017
                                <1>     ;cli    ; disable interrupts
0000CBFF B410                   <1>     mov     ah, 16 ; number of available timer events
0000CC01 BE[808F0100]           <1>     mov     esi, timer_set ; beginning address of timer events
                                <1> sysexit_7:
0000CC06 8A06                   <1>     mov     al, [esi] ; process number (of timer event)
0000CC08 38D8                   <1>     cmp     al, bl ; process number comparison
0000CC0A 7413                   <1>     je      short sysexit_10
0000CC0C 20C0                   <1>     and     al, al
0000CC0E 7406                   <1>     jz      short sysexit_9
                                <1> sysexit_8:
0000CC10 FEC9                   <1>     dec     cl
```

```
0000CC12 7416                <1>     jz      short sysexit_11
                             <1>     ; 08/01/2026 (BugFix)
0000CC14 B000                <1>     mov     al, 0
                             <1> sysexit_9:
0000CC16 FECC                <1>     dec     ah
0000CC18 7412                <1>     jz      short sysexit_12
0000CC1A 83C610              <1>     add     esi, 16
0000CC1D EBE7                <1>     jmp     short sysexit_7
                             <1>
                             <1> sysexit_10:
                             <1>     ; set timer event as free
0000CC1F C60600              <1>     mov     byte [esi], 0 ; 08/01/2026
                             <1>     ;mov    word [esi], 0
                             <1>     ;mov    dword [esi+12], 0
                             <1>     ;
0000CC22 FE0D[47830100]      <1>     dec     byte [timer_events] ; 02/01/2017
                             <1>     ;
0000CC28 EBE6                <1>     jmp     short sysexit_8
                             <1>
                             <1> sysexit_11:
                             <1>     ;sub    ax, ax ; 0 ; 26/02/2017
                             <1>     ; 30/07/2022
0000CC2A 29C0                <1>     sub     eax, eax ; 0
                             <1> sysexit_12:
                             <1>     ; 26/02/2017 (Unlink IRQ callbacks belong to the user)
                             <1>     ;cmp    byte [u.irqc], 0 ; Count of IRQ callbacks
                             <1>     ; 08/01/2026
                             <1>     ; eax = 0
0000CC2C 3805[E68E0100]      <1>     cmp     byte [u.irqc], al ; 0 ?
0000CC32 7E53                <1>     jng     short sysexit_16 ; zero or invalid
                             <1>     ; 28/02/2017
                             <1>     ; clear IRQ callback flags (for 'sysrele' and 'sysret')
0000CC34 A2[E78E0100]        <1>     mov     [u.irqwait], al ; 0 ; force to clear waiting flag
0000CC39 A2[E88E0100]        <1>     mov     [u.r_lock], al ; 0 ; force to clear busy flag
0000CC3E BE[94880100]        <1>     mov     esi, IRQ.owner
                             <1> sysexit_13:
0000CC43 AC                  <1>     lodsb
0000CC44 3A05[BD8E0100]      <1>     cmp     al, [u.uno] ; owner = current user ?
0000CC4A 7531                <1>     jne     short sysexit_14
0000CC4C C646FF00            <1>     mov     byte [esi-1], 0 ; owner = 0 : Free
                             <1>
                             <1>     ;;;
                             <1>     ; 22/08/2024 - Reset Hardware Interrupt Handler
0000CC50 89F1                <1>     mov     ecx, esi
0000CC52 81E9[94880100]      <1>     sub     ecx, IRQ.owner
                             <1>     ; CL = (user configurable) IRQ index + 1 (1 to 9)
0000CC58 B8[B0350100]        <1>     mov     eax, IRQenum
                             <1> sysexit_18:
0000CC5D 3A08                <1>     cmp     cl, [eax]
0000CC5F 740A                <1>     je      short sysexit_19
0000CC61 3D[BF350100]        <1>     cmp     eax, IRQenum+15
0000CC66 730D                <1>     jnb     short sysexit_20
0000CC68 40                  <1>     inc     eax
0000CC69 EBF2                <1>     jmp     short sysexit_18
                             <1> sysexit_19:
0000CC6B 2D[B0350100]        <1>     sub     eax, IRQenum
                             <1>     ; AL = IRQ number (will be reset to system's IRQ handler)
                             <1>     ; AH = 0
                             <1>     ;mov    ah, 0 ; reset
0000CC70 E8A7510000          <1>     call    set_hardware_int_vector
                             <1> sysexit_20:
                             <1>     ;;;
                             <1>
0000CC75 FE0D[E68E0100]      <1>     dec     byte [u.irqc]
0000CC7B 7408                <1>     jz      short sysexit_15
                             <1> sysexit_14:
0000CC7D 81FE[9C880100]      <1>     cmp     esi, IRQ.owner + 8 ; the last IRQ index number ?
0000CC83 76BE                <1>     jna     short sysexit_13 ; no
                             <1> sysexit_15:
                             <1>     ;xor    al, al ; 0
                             <1>     ; 22/08/2024
0000CC85 31C0                <1>     xor     eax, eax
                             <1> sysexit_16: ; 2:
0000CC87 FB                  <1>     sti     ; enable interrupts
                             <1>     ;
                             <1>     ; AX = 0
                             <1> sysexit_1: ; 1:
                             <1>     ; AX = File descriptor
                             <1>             ; / r1 has file descriptor (index to u.fp list)
                             <1>             ; / Search the whole list
0000CC88 E8CE350000          <1>     call    fclose
                             <1>             ; jsr r0,fclose / close all files the process opened
                             <1>     ;; ignore error return
                             <1>             ; br .+2 / ignore error return
                             <1>     ;;inc   ax
                             <1>     ;inc    al
                             <1>     ; 22/08/2024
0000CC8D 40                  <1>     inc     eax
                             <1>             ; inc r1 / increment file descriptor
                             <1>     ;cmp    ax, 10
0000CC8E 3C0A                <1>     cmp     al, 10
                             <1>             ; cmp r1,$10. / end of u.fp list?
0000CC90 72F6                <1>     jb      short sysexit_1
                             <1>             ; blt 1b / no, go back
                             <1>
                             <1>     ; 22/08/2024
                             <1>     ; reset IRQs if owned by the current (dying) process
0000CC92 BE[94880100]        <1>     mov     esi, IRQ.owner
                             <1>
                             <1>     ;movzx  ebx, byte [u.uno]
0000CC97 8A1D[BD8E0100]      <1>     mov     bl, [u.uno] ; 02/01/2017
                             <1>             ; movb u.uno,r1 / yes, move dying process's number to r1
0000CC9D 88A3[9F8D0100]      <1>     mov     [ebx+p.stat-1], ah ; 0, SFREE
                             <1>             ; clrb p.stat-1(r1) / free the process
                             <1>     ; 10/04/2017
0000CCA3 381D[11890100]      <1>     cmp     [audio_user], bl
0000CCA9 7518                <1>     jne     short sysexit_17
                             <1>     ; reset audio device (current) owner and 'initializated' flag
0000CCAB 883D[11890100]      <1>     mov     [audio_user], bh ; 0
                             <1>     ; 27/05/2017
0000CCB1 8B0D[F8880100]      <1>     mov     ecx, [audio_buffer]
0000CCB7 09C9                <1>     or      ecx, ecx
0000CCB9 7408                <1>     jz      short sysexit_17
                             <1>     ; 'deallocate_user_pages' is not necessary in sysexit !!!
                             <1>     ;push   ebx
                             <1>     ;mov    ebx, ecx
                             <1>     ;mov    ecx, [audio_buff_size]
                             <1>     ;call   deallocate_user_pages
                             <1>     ;; (Modified Registers -> EAX, EDX, ESI, EDI, EBX, ECX, EBP)
0000CCBB 29C9                <1>     sub     ecx, ecx
0000CCBD 890D[F8880100]      <1>     mov     [audio_buffer], ecx ; 0
                             <1>     ;pop    ebx
                             <1> sysexit_17:
                             <1>     ;shl    bx, 1
```

```
0000CCC3 D0E3                <1>     shl    bl, 1
                             <1>     ; 22/08/2024
                             <1>     ;shl   ebx, 1
                             <1>            ; asl r1 / use r1 for index into the below tables
0000CCC5 668B8B[3E8D0100]    <1>     mov    cx, [ebx+p.pid-2]
                             <1>            ; mov p.pid-2(r1),r3 / move dying process's name to r3
0000CCCC 668B93[5E8D0100]    <1>     mov    dx, [ebx+p.ppid-2]
                             <1>            ; mov p.ppid-2(r1),r4 / move its parents name to r4
                             <1>     ;xor   bx, bx ; 0
0000CCD3 30DB                <1>     xor    bl, bl ; 0
                             <1>            ; clr r2
0000CCD5 31F6                <1>     xor    esi, esi ; 0
                             <1>            ; clr r5 / initialize reg
                             <1> sysexit_2: ; 1:
                             <1>             ; / find children of this dying process,
                             <1>            ; / if they are zombies, free them
                             <1>     ;add   bx, 2
0000CCD7 80C302              <1>     add    bl, 2
                             <1>            ; add $2,r2 / search parent process table
                             <1>                          ; / for dying process's name
0000CCDA 66398B[5E8D0100]    <1>     cmp    [ebx+p.ppid-2], cx
                             <1>            ; cmp p.ppid-2(r2),r3 / found it?
0000CCE1 7513                <1>     jne    short sysexit_4
                             <1>            ; bne 3f / no
                             <1>     ;shr   bx, 1
0000CCE3 D0EB                <1>     shr    bl, 1
                             <1>            ; asr r2 / yes, it is a parent
0000CCE5 80BB[9F8D0100]03    <1>     cmp    byte [ebx+p.stat-1], 3 ; SZOMB
                             <1>            ; cmpb p.stat-1(r2),$3 / is the child of this
                             <1>                                   ; / dying process a zombie
0000CCEC 7506                <1>     jne    short sysexit_3
                             <1>            ; bne 2f / no
0000CCEE 88A3[9F8D0100]      <1>     mov    [ebx+p.stat-1], ah ; 0, SFREE
                             <1>            ; clrb p.stat-1(r2) / yes, free the child process
                             <1> sysexit_3: ; 2:
                             <1>     ;shr   bx, 1
0000CCF4 D0E3                <1>     shl    bl, 1
                             <1>            ; asl r2
                             <1> sysexit_4: ; 3:
                             <1>            ; / search the process name table
                             <1>            ; / for the dying process's parent
0000CCF6 663993[3E8D0100]    <1>     cmp    [ebx+p.pid-2], dx
                             <1>            ; cmp p.pid-2(r2),r4 / found it?
0000CCFD 7502                <1>     jne    short sysexit_5
                             <1>            ; bne 3f / no
0000CCFF 89DE                <1>     mov    esi, ebx
                             <1>            ; mov r2,r5 / yes, put index to p.pid table (parents
                             <1>                     ; / process # x2) in r5
                             <1> sysexit_5: ; 3:
                             <1>     ;cmp   bx, nproc + nproc
0000CD01 80FB20              <1>     cmp    bl, nproc + nproc
                             <1>            ; cmp r2,$nproc+nproc / has whole table been searched?
0000CD04 72D1                <1>     jb     short sysexit_2
                             <1>            ; blt 1b / no, go back
                             <1>            ; mov r5,r1 / yes, r1 now has parents process # x2
0000CD06 21F6                <1>     and    esi, esi ; r5=r1
0000CD08 743F                <1>     jz     short sysexit_6
                             <1>            ; beq 2f / no parent has been found.
                             <1>                  ; / The process just dies
                             <1>     ;shr   si, 1
                             <1>     ; 23/07/2022
0000CD0A D1EE                <1>     shr    esi, 1
                             <1>            ; asr r1 / set up index to p.stat
0000CD0C 8A86[9F8D0100]      <1>     mov    al, [esi+p.stat-1]
                             <1>            ; movb p.stat-1(r1),r2 / move status of parent to r2
0000CD12 20C0                <1>     and    al, al
0000CD14 7433                <1>     jz     short sysexit_6
                             <1>            ; beq 2f / if its been freed, 2f
0000CD16 3C03                <1>     cmp    al, 3
                             <1>            ; cmp r2,$3 / is parent a zombie?
0000CD18 742F                <1>     je     short sysexit_6
                             <1>            ; beq 2f / yes, 2f
                             <1>
                             <1>     ; BH = 0
0000CD1A 8A1D[BD8E0100]      <1>     mov    bl, [u.uno]
                             <1>            ; movb u.uno,r3 / move dying process's number to r3
0000CD20 C683[9F8D0100]03    <1>     mov    byte [ebx+p.stat-1], 3 ; SZOMB
                             <1>            ; movb $3,p.stat-1(r3) / make the process a zombie
                             <1>
0000CD27 3C01                <1>     cmp    al, 1 ; SRUN
0000CD29 741E                <1>     je     short sysexit_6
                             <1>     ;cmp   al, 2
                             <1>            ; cmp r2,$2 / is the parent waiting for
                             <1>                     ; / this child to die
                             <1>     ;jne   short sysexit_6
                             <1>            ; bne 2f / yes, notify parent not to wait any more
                             <1>     ; p.stat = 2 --> waiting
                             <1>     ; p.stat = 4 --> sleeping
0000CD2B C686[9F8D0100]01    <1>     mov    byte [esi+p.stat-1], 1 ; SRUN
                             <1>     ;;;
                             <1>     ; 18/08/2024 - exit code
0000CD32 A0[EA8E0100]        <1>     mov    al, [u.exit] ; exit code of the child
                             <1>     ; 20/08/2024
0000CD37 8883[3F8E0100]      <1>     mov    [ebx+p.exitc-1], al ; save it to use by the parent
                             <1>     ;;;
                             <1>     ;dec   byte [esi+p.stat-1]
                             <1>            ; decb  p.stat-1(r1) / awaken it by putting it (parent)
                             <1>     ;;;
                             <1>     ;mov   ax, si ; r1  (process number in AL)
                             <1>     ; 18/08/2024
0000CD3D 89F0                <1>     mov    eax, esi
                             <1>     ;;;
                             <1>     ;mov   ebx, runq + 4
                             <1>            ; mov $runq+4,r2 / on the runq
0000CD3F BB[5C8E0100]        <1>     mov    ebx, runq+2 ; normal run queue ; 02/01/2017
0000CD44 E8FE3C0000          <1>     call   putlu
                             <1>            ; jsr r0, putlu
                             <1> sysexit_6:
                             <1>            ; / the process dies
0000CD49 C605[BD8E0100]00    <1>     mov    byte [u.uno], 0
                             <1>            ; clrb u.uno / put zero as the process number,
                             <1>                ; / so "swap" will
0000CD50 E8F53B0000          <1>     call   swap
                             <1>            ; jsr r0,swap / overwrite process with another process
                             <1> hlt_sys:
                             <1>     ;sti
                             <1> hlts0:
0000CD55 F4                  <1>     hlt
0000CD56 EBFD                <1>     jmp    short hlts0
                             <1>            ; 0 / and thereby kill it; halt?
                             <1>
                             <1> syswait: ; < wait for a processs to die >
                             <1>     ; 20/08/2024
                             <1>     ; 30/07/2022
```

```
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/09/2015
                                <1>     ; 02/09/2015
                                <1>     ; 01/09/2015
                                <1>     ; 16/04/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 24/05/2013 - 05/02/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'syswait' waits for a process die.
                                <1>     ; It works in following way:
                                <1>     ;    1) From the parent process number, the parent's
                                <1>     ;       process name is found. The p.ppid table of parent
                                <1>     ;       names is then searched for this process name.
                                <1>     ;       If a match occurs, r2 contains child's process
                                <1>     ;       number. The child status is checked to see if it is
                                <1>     ;       a zombie, i.e; dead but not waited for (p.stat=3)
                                <1>     ;       If it is, the child process is freed and it's name
                                <1>     ;       is put in (u.r0). A return is then made via 'sysret'.
                                <1>     ;       If the child is not a zombie, nothing happens and
                                <1>     ;       the search goes on through the p.ppid table until
                                <1>     ;       all processes are checked or a zombie is found.
                                <1>     ;    2) If no zombies are found, a check is made to see if
                                <1>     ;       there are any children at all. If there are none,
                                <1>     ;       an error return is made. If there are, the parent's
                                <1>     ;       status is set to 2 (waiting for child to die),
                                <1>     ;       the parent is swapped out, and a branch to 'syswait'
                                <1>     ;       is made to wait on the next process.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       ?
                                <1>     ; Arguments:
                                <1>     ;       -
                                <1>     ; Inputs: -
                                <1>     ; Outputs: if zombie found, it's name put in u.r0.
                                <1>     ; ............................................................
                                <1>     ;
                                <1>
                                <1> ; / wait for a process to die
                                <1>
                                <1> syswait_0:
0000CD58 0FB61D[BD8E0100]       <1>     movzx  ebx, byte [u.uno] ; 01/09/2015
                                <1>            ; movb u.uno,r1 / put parents process number in r1
0000CD5F D0E3                   <1>     shl    bl, 1
                                <1>     ;shl   bx, 1
                                <1>            ; asl r1 / x2 to get index into p.pid table
0000CD61 668B83[3E8D0100]       <1>     mov    ax, [ebx+p.pid-2]
                                <1>            ; mov p.pid-2(r1),r1 / get the name of this process
0000CD68 31F6                   <1>     xor    esi, esi
                                <1>            ; clr r2
0000CD6A 31C9                   <1>     xor    ecx, ecx ; 30/10/2013
                                <1>     ;xor   cl, cl
                                <1>            ; clr r3 / initialize reg 3
                                <1> syswait_1: ; 1:
                                <1>     ;add   si, 2
                                <1>     ; 23/07/2022
0000CD6C 46                     <1>     inc    esi
0000CD6D 46                     <1>     inc    esi
                                <1>            ; add $2,r2 / use r2 for index into p.ppid table
                                <1>                     ; / search table of parent processes
                                <1>                     ; / for this process name
0000CD6E 663B86[5E8D0100]       <1>     cmp    ax, [esi+p.ppid-2]
                                <1>            ; cmp p.ppid-2(r2),r1 / r2 will contain the childs
                                <1>                                   ; / process number
0000CD75 7542                   <1>     jne    short syswait_3
                                <1>            ;bne 3f / branch if no match of parent process name
                                <1>     ;inc   cx
0000CD77 FEC1                   <1>     inc    cl
                                <1>            ;inc r3 / yes, a match, r3 indicates number of children
                                <1>     ;shr   si, 1
                                <1>     ; 23/07/2022
0000CD79 D1EE                   <1>     shr    esi, 1
                                <1>            ; asr r2 / r2/2 to get index to p.stat table
                                <1>     ; The possible states ('p.stat' values) of a process are:
                                <1>     ;      0 = free or unused
                                <1>     ;      1 = active
                                <1>     ;      2 = waiting for a child process to die
                                <1>     ;      3 = terminated, but not yet waited for (zombie).
0000CD7B 80BE[9F8D0100]03       <1>     cmp    byte [esi+p.stat-1], 3 ; SZOMB, 05/02/2014
                                <1>            ; cmpb p.stat-1(r2),$3 / is the child process a zombie?
0000CD82 7533                   <1>     jne    short syswait_2
                                <1>            ; bne 2f / no, skip it
0000CD84 88BE[9F8D0100]         <1>     mov    [esi+p.stat-1], bh ; 0
                                <1>            ; clrb p.stat-1(r2) / yes, free it
                                <1>
                                <1>     ; 20/08/2024 (ebx = child's exit code)
0000CD8A 31C0                   <1>     xor    eax, eax
0000CD8C 8A86[3F8E0100]         <1>     mov    al, [esi+p.exitc-1] ; exit code
0000CD92 8B2D[688E0100]         <1>     mov    ebp, [u.usp]
                                <1>     ; 22/08/2024
0000CD98 894510                 <1>     mov    [ebp+16], eax ; ebx
                                <1>
                                <1>     ;shl   si, 1
                                <1>     ; 23/07/2022
0000CD9B D1E6                   <1>     shl    esi, 1
                                <1>            ; asl r2 / r2x2 to get index into p.pid table
                                <1>     ;movzx eax, word [esi+p.pid-2]
                                <1>     ; 20/08/2024
0000CD9D 668B86[3E8D0100]       <1>     mov    ax, [esi+p.pid-2]
0000CDA4 A3[6C8E0100]           <1>     mov    [u.r0], eax
                                <1>            ; mov p.pid-2(r2),*u.r0
                                <1>                           ; / put childs process name in (u.r0)
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification ! (17/09/2015)
                                <1>     ;
                                <1>     ; Parent process ID -p.ppid- field (of the child process)
                                <1>     ; must be cleared in order to prevent infinitive 'syswait'
                                <1>     ; system call loop from the application/program if it calls
                                <1>     ; 'syswait' again (mistakenly) while there is not a zombie
                                <1>     ; or running child process to wait. ('forktest.s', 17/09/2015)
                                <1>     ;
                                <1>     ; Note: syswait will return with error if there is not a
                                <1>     ;       zombie or running process to wait.
                                <1>
                                <1>     ;sub   ax, ax
                                <1>     ; 30/07/2022
0000CDA9 29C0                   <1>     sub    eax, eax ; 0
0000CDAB 668986[5E8D0100]       <1>     mov    [esi+p.ppid-2], ax ; 0 ; 17/09/2015
0000CDB2 E9D9FCFFFF             <1>     jmp    sysret0 ; ax = 0
                                <1>     ;
                                <1>     ;jmp   sysret
                                <1>            ; br sysret1 / return cause child is dead
                                <1> syswait_2: ; 2:
                                <1>     ;shl   si, 1
                                <1>     ; 23/07/2022
0000CDB7 D1E6                   <1>     shl    esi, 1
```

```
                                <1>             ; asl r2 / r2x2 to get index into p.ppid table
                                <1> syswait_3: ; 3:
0000CDB9 6683FE20               <1>     cmp     si, nproc+nproc
                                <1>             ; cmp r2,$nproc+nproc / have all processes been checked?
0000CDBD 72AD                   <1>     jb      short syswait_1
                                <1>             ; blt 1b / no, continue search
                                <1>     ;and    cx, cx
0000CDBF 20C9                   <1>     and     cl, cl
                                <1>             ; tst r3 / one gets here if there are no children
                                <1>                    ; / or children that are still active
                                <1>     ; 30/10/2013
0000CDC1 7515                   <1>     jnz     short syswait_4
                                <1>     ;jz     error
                                <1>             ; beq error1 / there are no children, error
0000CDC3 890D[6C8E0100]         <1>     mov     [u.r0], ecx ; 0
                                <1>
                                <1>     ; 20/08/2024 (ebx = child's exit code)
0000CDC9 8B2D[688E0100]         <1>     mov     ebp, [u.usp]
0000CDCF 49                     <1>     dec     ecx ; -1 ; 0FFFFFFFFh
                                <1>     ; 22/08/2024
0000CDD0 894D10                 <1>     mov     [ebp+16], ecx ; ebx
                                <1>
0000CDD3 E996FCFFFF             <1>     jmp     error
                                <1> syswait_4:
0000CDD8 8A1D[BD8E0100]         <1>     mov     bl, [u.uno]
                                <1>             ; movb u.uno,r1 / there are children so put
                                <1>                           ; / parent process number in r1
0000CDDE FE83[9F8D0100]         <1>     inc     byte [ebx+p.stat-1] ; 2, SWAIT, 05/02/2014
                                <1>             ; incb p.stat-1(r1) / it is waiting for
                                <1>                                 ; / other children to die
                                <1>     ; 04/11/2013
0000CDE4 E8613B0000             <1>     call    swap
                                <1>             ; jsr r0,swap / swap it out, because it's waiting
                                <1>
                                <1>     ; 23/08/2024 - bugfix
                                <1>     ; restore [u.usp]
                                <1>     ; (swap above changes [u.usp] just before wswap)
0000CDE9 8925[688E0100]         <1>     mov     [u.usp], esp
                                <1>             ; points to user's regs on top ofn system stack
                                <1>
0000CDEF E964FFFFFF             <1>     jmp     syswait_0
                                <1>             ; br syswait / wait on next process
                                <1>
                                <1> sysfork: ; < create a new process >
                                <1>     ; 19/08/2024 - TRDOS 386 Kernel v2.0.9 (STDIN/STDOUT)
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 02/01/2017 (TRDOS 386 modification)
                                <1>     ; 04/09/2015 - 18/05/2015
                                <1>     ; 28/08/2015 - 01/09/2015 - 02/09/2015
                                <1>     ; 09/05/2015 - 10/05/2015 - 14/05/2015
                                <1>     ; 06/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 24/05/2013 - 14/02/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysfork' creates a new process. This process is referred
                                <1>     ; to as the child process. This new process core image is
                                <1>     ; a copy of that of the caller of 'sysfork'. The only
                                <1>     ; distinction is the return location and the fact that (u.r0)
                                <1>     ; in the old process (parent) contains the process id (p.pid)
                                <1>     ; of the new process (child). This id is used by 'syswait'.
                                <1>     ; 'sysfork' works in the following manner:
                                <1>     ;    1) The process status table (p.stat) is searched to find
                                <1>     ;       a process number that is unused. If none are found
                                <1>     ;       an error occurs.
                                <1>     ;    2) when one is found, it becomes the child process number
                                <1>     ;       and it's status (p.stat) is set to active.
                                <1>     ;    3) If the parent had a control tty, the interrupt
                                <1>     ;       character in that tty buffer is cleared.
                                <1>     ;    4) The child process is put on the lowest priority run
                                <1>     ;       queue via 'putlu'.
                                <1>     ;    5) A new process name is gotten from 'mpid' (actually
                                <1>     ;       it is a unique number) and is put in the child's unique
                                <1>     ;       identifier; process id (p.pid).
                                <1>     ;    6) The process name of the parent is then obtained and
                                <1>     ;       placed in the unique identifier of the parent process
                                <1>     ;       name is then put in 'u.r0'.
                                <1>     ;    7) The child process is then written out on disk by
                                <1>     ;       'wswap',i.e., the parent process is copied onto disk
                                <1>     ;       and the child is born. (The child process is written
                                <1>     ;       out on disk/drum with 'u.uno' being the child process
                                <1>     ;       number.)
                                <1>     ;    8) The parent process number is then restored to 'u.uno'.
                                <1>     ;    9) The child process name is put in 'u.r0'.
                                <1>     ;   10) The pc on the stack sp + 18 is incremented by 2 to
                                <1>     ;       create the return address for the parent process.
                                <1>     ;   11) The 'u.fp' list as then searched to see what files
                                <1>     ;       the parent has opened. For each file the parent has
                                <1>     ;       opened, the corresponding 'fsp' entry must be updated
                                <1>     ;       to indicate that the child process also has opened
                                <1>     ;       the file. A branch to 'sysret' is then made.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       from shell ?
                                <1>     ; Arguments:
                                <1>     ;       -
                                <1>     ; Inputs: -
                                <1>     ; Outputs: *u.r0 - child process name
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       AX = r0 = PID (>0) (at the return of 'sysfork')
                                <1>     ;       = process id of child a parent process returns
                                <1>     ;       = process id of parent when a child process returns
                                <1>     ;
                                <1>     ;        In original UNIX v1, sysfork is called and returns as
                                <1>     ;       in following manner: (with an example: c library, fork)
                                <1>     ;
                                <1>     ;       1:
                                <1>     ;               sys     fork
                                <1>     ;                       br 1f  / child process returns here
                                <1>     ;               bes     2f     / parent process returns here
                                <1>     ;               / pid of new process in r0
                                <1>     ;               rts     pc
                                <1>     ;       2: / parent process condionally branches here
                                <1>     ;               mov     $-1,r0 / pid = -1 means error return
                                <1>     ;               rts     pc
                                <1>     ;
                                <1>     ;       1: / child process brances here
                                <1>     ;               clr     r0   / pid = 0 in child process
                                <1>     ;               rts     pc
                                <1>     ;
                                <1>     ;       In UNIX v7x86 (386) by Robert Nordier (1999)
                                <1>     ;               // pid = fork();
                                <1>     ;               //
                                <1>     ;               // pid == 0 in child process;
```

```
                                <1>     ;               // pid == -1 means error return
                                <1>     ;               // in child,
                                <1>     ;               //      parents id is in par_uid if needed
                                <1>     ;
                                <1>     ;               _fork:
                                <1>     ;                       mov     $.fork,eax
                                <1>     ;                       int     $0x30
                                <1>     ;                       jmp     1f
                                <1>     ;                       jnc     2f
                                <1>     ;                       jmp     cerror
                                <1>     ;               1:
                                <1>     ;                       mov     eax,_par_uid
                                <1>     ;                       xor     eax,eax
                                <1>     ;               2:
                                <1>     ;                       ret
                                <1>     ;
                                <1>     ;       In Retro UNIX 8086 v1,
                                <1>     ;       'sysfork' returns in following manner:
                                <1>     ;
                                <1>     ;               mov     ax, sys_fork
                                <1>     ;               mov     bx, offset @f ; routine for child
                                <1>     ;               int     20h
                                <1>     ;               jc      error
                                <1>     ;
                                <1>     ;       ; Routine for parent process here (just after 'jc')
                                <1>     ;               mov     word ptr [pid_of_child], ax
                                <1>     ;               jmp     next_routine_for_parent
                                <1>     ;
                                <1>     ;       @@: ; routine for child process here
                                <1>     ;                       ....
                                <1>     ;       NOTE: 'sysfork' returns to specified offset
                                <1>     ;                for child process by using BX input.
                                <1>     ;              (at first, parent process will return then
                                <1>     ;              child process will return -after swapped in-
                                <1>     ;              'syswait' is needed in parent process
                                <1>     ;              if return from child process will be waited for.)
                                <1>     ;
                                <1>
                                <1> ; / create a new process
                                <1>     ; EBX = return address for child process
                                <1>             ; (Retro UNIX 8086 v1 modification !)
0000CDF4 31F6                   <1>     xor     esi, esi
                                <1>             ; clr r1
                                <1> sysfork_1: ; 1: / search p.stat table for unused process number
0000CDF6 46                     <1>     inc     esi
                                <1>             ; inc r1
0000CDF7 80BE[9F8D0100]00       <1>     cmp     byte [esi+p.stat-1], 0 ; SFREE, 05/02/2014
                                <1>             ; tstb p.stat-1(r1) / is process active, unused, dead
0000CDFE 760B                   <1>     jna     short sysfork_2
                                <1>             ; beq 1f / it's unused so branch
0000CE00 6683FE10               <1>     cmp     si, nproc
                                <1>             ; cmp r1,$nproc / all processes checked
0000CE04 72F0                   <1>     jb      short sysfork_1
                                <1>             ; blt 1b / no, branch back
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1. modification:
                                <1>     ;       Parent process returns from 'sysfork' to address
                                <1>     ;       which is just after 'sysfork' system call in parent
                                <1>     ;       process. Child process returns to address which is put
                                <1>     ;       in BX register by parent process for 'sysfork'.
                                <1>     ;
                                <1>             ; add $2,18.(sp) / add 2 to pc when trap occured, points
                                <1>                             ; / to old process return
                                <1>             ; br error1 / no room for a new process
                                <1> sysfork_err:
0000CE06 E963FCFFFF             <1>     jmp     error
                                <1> sysfork_2: ; 1:
0000CE0B E81A8AFFFF             <1>     call    allocate_page
                                <1>     ;jc     error
                                <1>     ; 23/07/2022
0000CE10 72F4                   <1>     jc      short sysfork_err
0000CE12 50                     <1>     push    eax   ; UPAGE (user structure page) address
                                <1>     ; Retro UNIX 386 v1 modification!
0000CE13 E8028CFFFF             <1>     call    duplicate_page_dir
                                <1>             ; EAX = New page directory
0000CE18 730B                   <1>     jnc     short sysfork_3
0000CE1A 58                     <1>     pop     eax   ; UPAGE (user structure page) address
0000CE1B E8C98BFFFF             <1>     call    deallocate_page
0000CE20 E949FCFFFF             <1>     jmp     error
                                <1> sysfork_3:
                                <1>     ; Retro UNIX 386 v1 modification !
0000CE25 56                     <1>     push    esi
0000CE26 E8AC3B0000             <1>     call    wswap ; save current user (u) structure, user registers
                                <1>                   ; and interrupt return components (for IRET)
0000CE2B 8705[C48E0100]         <1>     xchg    eax, [u.pgdir] ; page directory of the child process
0000CE31 A3[C88E0100]           <1>     mov     [u.ppgdir], eax ; page directory of the parent process
0000CE36 5E                     <1>     pop     esi
0000CE37 58                     <1>     pop     eax   ; UPAGE (user structure page) address
                                <1>             ; [u.usp] = esp
0000CE38 89F7                   <1>     mov     edi, esi
                                <1>     ;shl    di, 2
                                <1>     ; 23/07/2022
0000CE3A C1E702                 <1>     shl     edi, 2
0000CE3D 8987[AC8D0100]         <1>     mov     [edi+p.upage-4], eax ; memory page for 'user' struct
0000CE43 A3[C08E0100]           <1>     mov     [u.upage], eax ; memory page for 'user' struct (child)
                                <1>     ;;;
                                <1>     ; 19/08/2024 - reset STDIN/STDOUT redirections (and ungetchar)
0000CE48 31C0                   <1>     xor     eax, eax ; 0
                                <1>     ; 19/08/2024
                                <1>     ;mov    [u.stdin], al ; 0
                                <1>     ;mov    [u.stdout], al ; 0
                                <1>     ;mov    [u.ungetc], al ; 0
                                <1>     ;mov    [u.getc], al ; 0
0000CE4A A3[D48E0100]           <1>     mov     [u.stdin], eax ; 0
                                <1>     ;;;
                                <1>
                                <1>     ; 28/08/2015
                                <1>     ;movzx  eax, byte [u.uno] ; parent process number
                                <1>     ; 19/08/2024
                                <1>     ; eax = 0
0000CE4F A0[BD8E0100]           <1>     mov     al, [u.uno] ; parent process number
                                <1>             ; movb u.uno,-(sp) / save parent process number
0000CE54 89C7                   <1>     mov     edi, eax
0000CE56 50                     <1>         push       eax ; **
0000CE57 8A87[7F8D0100]         <1>     mov      al, [edi+p.ttyc-1] ; console tty (parent)
                                <1>     ; 18/09/2015
                                <1>     ;;mov   [esi+p.ttyc-1], al ; set child's console tty
                                <1>     ;;mov   [esi+p.waitc-1], ah ; 0 ; reset child's wait channel
                                <1>     ;mov     [esi+p.ttyc-1], ax ; al - set child's console tty
                                <1>                                ; ah - reset child's wait channel
                                <1>     ; 23/07/2022
0000CE5D 8886[7F8D0100]         <1>     mov     [esi+p.ttyc-1], al ; set child's console tty
0000CE63 89F0                   <1>     mov     eax, esi
0000CE65 A2[BD8E0100]           <1>     mov     [u.uno], al ; child process number
```

```
                                <1>             ;movb r1,u.uno / set child process number to r1
0000CE6A FE86[9F8D0100]         <1>         inc     byte [esi+p.stat-1] ; 1, SRUN, 05/02/2014
                                <1>             ; incb p.stat-1(r1) / set p.stat entry for child
                                <1>                             ; / process to active status
                                <1>             ; mov u.ttyp,r2 / put pointer to parent process'
                                <1>                             ; / control tty buffer in r2
                                <1>                 ; beq 2f / branch, if no such tty assigned
                                <1>             ; clrb 6(r2) / clear interrupt character in tty buffer
                                <1>     ; 2:
0000CE70 53                     <1>     push    ebx  ; * return address for the child process
                                <1>                  ; * Retro UNIX 8086 v1 feature only !
                                <1>     ; (Retro UNIX 8086 v1 modification!)
                                <1>             ; mov $runq+4,r2
0000CE71 BB[5C8E0100]           <1>     mov     ebx, runq+2 ; normal run queue ; 02/01/2017
0000CE76 E8CC3B0000             <1>     call    putlu
                                <1>             ; jsr r0,putlu / put child process on lowest priority
                                <1>                         ; / run queue
                                <1>     ; 23/07/2022
0000CE7B D1E6                   <1>     shl     esi, 1
                                <1>     ;shl    si, 1
                                <1>             ; asl r1 / multiply r1 by 2 to get index
                                <1>                     ; / into p.pid table
0000CE7D 66FF05[548E0100]       <1>     inc     word [mpid]
                                <1>             ; inc mpid / increment m.pid; get a new process name
0000CE84 66A1[548E0100]         <1>     mov     ax, [mpid]
0000CE8A 668986[3E8D0100]       <1>     mov     [esi+p.pid-2], ax
                                <1>             ;mov mpid,p.pid-2(r1) / put new process name
                                <1>                                   ; / in child process' name slot
0000CE91 5A                     <1>     pop     edx  ; * return address for the child process
                                <1>                  ; * Retro UNIX 8086 v1 feature only !
0000CE92 5B                     <1>     pop     ebx  ; **
                                <1>     ;mov    ebx, [esp] ; ** parent process number
                                <1>             ; movb (sp),r2 / put parent process number in r2
                                <1>     ; 23/07/2022
0000CE93 D1E3                   <1>     shl     ebx, 1
                                <1>     ;shl    bx, 1
                                <1>             ;asl r2 / multiply by 2 to get index into below tables
                                <1>     ;movzx eax, word [ebx+p.pid-2]
0000CE95 668B83[3E8D0100]       <1>     mov     ax, [ebx+p.pid-2]
                                <1>             ; mov p.pid-2(r2),r2 / get process name of parent
                                <1>                                  ; / process
0000CE9C 668986[5E8D0100]       <1>     mov     [esi+p.ppid-2], ax
                                <1>             ; mov r2,p.ppid-2(r1) / put parent process name
                                <1>                     ; / in parent process slot for child
0000CEA3 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>             ; mov r2,*u.r0 / put parent process name on stack
                                <1>                            ; / at location where r0 was saved
0000CEA8 8B2D[648E0100]         <1>     mov     ebp, [u.sp] ; points to return address (EIP for IRET)
0000CEAE 895500                 <1>     mov     [ebp], edx ; *, CS:EIP -> EIP
                                <1>                         ; * return address for the child process
                                <1>             ; mov $sysret1,-(sp) /
                                <1>             ; mov sp,u.usp / contents of sp at the time when
                                <1>                            ; / user is swapped out
                                <1>             ; mov $sstack,sp / point sp to swapping stack space
                                <1>     ; 04/09/2015 - 01/09/2015
                                <1>     ; [u.usp] = esp
0000CEB1 68[8ECA0000]           <1>     push    sysret ; ***
0000CEB6 8925[688E0100]         <1>     mov     [u.usp], esp ; points to 'sysret' address (***)
                                <1>                          ; (for child process)
0000CEBC 31C0                   <1>     xor     eax, eax
0000CEBE 66A3[A08E0100]         <1>     mov     [u.ttyp], ax ; 0
                                <1>     ;
0000CEC4 E80E3B0000             <1>     call    wswap ; Retro UNIX 8086 v1 modification !
                                <1>             ; jsr r0,wswap / put child process out on drum
                                <1>             ; jsr r0,unpack / unpack user stack
                                <1>             ; mov u.usp,sp / restore user stack pointer
                                <1>             ; tst (sp)+ / bump stack pointer
                                <1>     ; Retro UNIX 386 v1 modification !
0000CEC9 58                     <1>     pop     eax ; ***
                                <1>     ;shl    bx, 1
                                <1>     ; 23/07/2022
0000CECA D1E3                   <1>     shl     ebx, 1
0000CECC 8B83[AC8D0100]         <1>     mov      eax, [ebx+p.upage-4] ; UPAGE address ; 14/05/2015
0000CED2 E8383B0000             <1>     call    rswap ; restore parent process 'u' structure,
                                <1>                   ; registers and return address (for IRET)
                                <1>             ;movb (sp)+,u.uno / put parent process number in u.uno
0000CED7 0FB705[548E0100]       <1>         movzx   eax, word [mpid]
0000CEDE A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>             ; mov mpid,*u.r0 / put child process name on stack
                                <1>                              ; / where r0 was saved
                                <1>             ; add $2,18.(sp) / add 2 to pc on stack; gives parent
                                <1>                              ; / process return
                                <1>     ;xor    ebx, ebx
0000CEE3 31F6                   <1>     xor      esi, esi
                                <1>             ;clr r1
                                <1> sysfork_4: ; 1: / search u.fp list to find the files
                                <1>                ; / opened by the parent process
                                <1>     ; 01/09/2015
                                <1>     ;xor    bh, bh
                                <1>     ;mov    bl, [esi+u.fp]
0000CEE5 8A86[768E0100]         <1>     mov     al, [esi+u.fp]
                                <1>             ; movb u.fp(r1),r2 / get an open file for this process
                                <1>         ;or bl, bl
0000CEEB 08C0                   <1>     or      al, al
0000CEED 7406                   <1>     jz      short sysfork_5
                                <1>             ; beq 2f / file has not been opened by parent,
                                <1>                     ; / so branch
                                <1>     ;mov    ah, 10 ; Retro UNIX 386 v1 fsp structure size = 10 bytes
                                <1>     ;mul    ah
                                <1>     ; 23/07/2022
                                <1>     ; Retro UNIX 386 v2 & TRDOS 386 v2.0.5 fsp struc size = 16 bytes
                                <1>     ;mov    ebx, eax
                                <1>     ;shl    ebx, 4 ; * 16
                                <1>     ;inc    byte [ebx+fsp-10]
                                <1>
                                <1>     ; 23/07/2022 (BugFix)
0000CEEF FE80[58840100]         <1>     inc     byte [OF_OPENCOUNT+eax]
                                <1>
                                <1>     ;;movzx ebx, ax
                                <1>     ;mov    bx, ax
                                <1>     ;shl    bx, 3
                                <1>             ; asl r2 / multiply by 8
                                <1>                     ; asl r2 / to get index into fsp table
                                <1>                     ; asl r2
                                <1>     ;inc    byte [ebx+fsp-2]
                                <1>             ; incb fsp-2(r2) / increment number of processes
                                <1>                          ; / using file, because child will now be
                                <1>                          ; / using this file
                                <1> sysfork_5: ; 2:
0000CEF5 46                     <1>         inc     esi
                                <1>             ; inc r1 / get next open file
                                <1>     ; 23/07/2022
0000CEF6 6683FE20               <1>     cmp     si, OPENFILES ; = 10
                                <1>     ;cmp    si, 10
```

```
                                <1>             ; cmp r1,$10. / 10. files is the maximum number which
                                <1>                         ; / can be opened
0000CEFA 72E9                   <1>     jb      short sysfork_4
                                <1>             ; blt 1b / check next entry
0000CEFC E98DFBFFFF             <1>     jmp     sysret
                                <1>             ; br sysret1
                                <1>
                                <1> syscreat: ; < create file >
                                <1>     ; 29/08/2024
                                <1>     ; 25/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 08/08/2022
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 13/11/2017
                                <1>     ; 27/10/2016
                                <1>     ; 25/10/2016 - 26/10/2016
                                <1>     ; 15/10/2016 - 16/10/2016 - 17/10/2016
                                <1>     ; 10/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;            -derived from INT_21H.ASM-
                                <1>     ;             ("loc_INT21h_create_file")
                                <1>         ;   10/07/2011 (12/03/2011)
                                <1>         ;   INT 21h Function AH = 3Ch
                                <1>         ;   Create File
                                <1>         ;   INPUT
                                <1>         ;      CX = Attributes
                                <1>         ;           DS:DX= Address of zero terminaned path name
                                <1>         ;
                                <1>     ; 27/12/2015 (Retro UNIX 386 v1.1)
                                <1>     ; 14/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 27/05/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'syscreat' called with two arguments; name and mode.
                                <1>     ; u.namep points to name of the file and mode is put
                                <1>     ; on the stack. 'namei' is called to get i-number of the file.
                                <1>     ; If the file aready exists, it's mode and owner remain
                                <1>     ; unchanged, but it is truncated to zero length. If the file
                                <1>     ; did not exist, an i-node is created with the new mode via
                                <1>     ; 'maknod' whether or not the file already existed, it is
                                <1>     ; open for writing. The fsp table is then searched for a free
                                <1>     ; entry. When a free entry is found, proper data is placed
                                <1>     ; in it and the number of this entry is put in the u.fp list.
                                <1>     ; The index to the u.fp (also know as the file descriptor)
                                <1>     ; is put in the user's r0.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       syscreate; name; mode
                                <1>     ; Arguments:
                                <1>     ;       name - name of the file to be created
                                <1>     ;       mode - mode of the file to be created
                                <1>     ; Inputs: (arguments)
                                <1>     ; Outputs: *u.r0 - index to u.fp list
                                <1>     ;                  (the file descriptor of new file)
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       'syscreate' system call has two arguments; so,
                                <1>     ;       * 1st argument, name is pointed to by BX register
                                <1>     ;       * 2nd argument, mode is in CX register
                                <1>     ;
                                <1>     ;       AX register (will be restored via 'u.r0') will return
                                <1>     ;       to the user with the file descriptor/number
                                <1>     ;       (index to u.fp list).
                                <1>     ;
                                <1>     ;call  arg2
                                <1>     ; * name - 'u.namep' points to address of file/path name
                                <1>     ;          in the user's program segment ('u.segmnt')
                                <1>     ;          with offset in BX register (as sysopen argument 1).
                                <1>     ; * mode - sysopen argument 2 is in CX register
                                <1>     ;          which is on top of stack.
                                <1>     ;
                                <1>     ; TRDOS 386 (10/10/2016)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;      CL = File Attributes
                                <1>     ;           bit 0 (1) - Read only file (R)
                                <1>     ;            bit 1 (1) - Hidden file (H)
                                <1>         ;           bit 2 (1) - System file (R)
                                <1>     ;            bit 3 (1) - Volume label/name (V)
                                <1>         ;           bit 4 (1) - Subdirectory (D)
                                <1>     ;           bit 5 (1) - File has been archived (A)
                                <1>         ;          EBX = Pointer to filename (ASCIIZ) -path-
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;          eax = File/Device Handle/Number (index) (AL)
                                <1>     ;          cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; Note: If the file is existing and it has not any one
                                <1>     ;       of S,H,R,V,D attributes, it will be truncated
                                <1>     ;       to zero length; otherwise, access error will be
                                <1>     ;       returned.
                                <1>
                                <1> sysmkdir_0:
0000CF01 F6C108                 <1>     test    cl, 08h ; Volume name
0000CF04 740A                   <1>     jz      short syscreat_0
                                <1>
                                <1>     ; Volume name or long name creation
                                <1>     ; is not permitted (in TRDOS 386)!
0000CF06 B80B000000             <1>     mov     eax, ERR_FILE_ACCESS  ; 11 ; 'permission denied !'
0000CF0B E9D8000000             <1>         jmp sysopen_dev_err ; 08/08/2022
                                <1> syscreat_0:
                                <1>         ;mov       [u.namep], ebx
0000CF10 51                     <1>     push    ecx
0000CF11 89DE                   <1>     mov     esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov    ax, 1
                                <1>     ;mov    [FFF_Valid], ah ; 0 ; reset ; 17/10/2016
                                <1>     ;call   set_working_path
0000CF13 E8BD4D0000             <1>     call    set_working_path_x ; 17/10/2016
                                <1>     ;jc     short syscreat_err
                                <1>     ; 23/07/2022
0000CF18 7305                   <1>     jnc     short syscreat_3
0000CF1A E9E2000000             <1>     jmp     syscreat_err
                                <1>
                                <1> syscreat_3:
                                <1>     ; 16/10/2016
0000CF1F 803D[6F830100]00       <1>     cmp     byte [SWP_inv_fname], 0
0000CF26 7776                   <1>     ja      short syscreat_inv_fname ; invalid file name !
                                <1>
                                <1>     ; Here, we have a valid path and also a valid file name
                                <1>     ; (Working dir has been changed if the path
                                <1>     ;  -file name string- had contained a dir name.)
                                <1>
                                <1>     ;xor    ax, ax
```

```
                                <1>     ; 25/08/2024
0000CF28 31C0                   <1>     xor    eax, eax
                                <1>     ;mov   esi, FindFile_Name
0000CF2A E84BBAFFFF             <1>     call   find_first_file
0000CF2F 59                     <1>     pop    ecx
                                <1>            ; ESI = Directory Entry (FindFile_DirEntry) Location
                                <1>            ; EDI = Directory Buffer Directory Entry Location
                                <1>            ; EAX = File Size
                                <1>            ;  BL = Attributes of The File/Directory
                                <1>            ;  BH = Long Name Yes/No Status (>0 is YES)
                                <1>            ;  DX > 0 : Ambiguous filename chars are used
0000CF30 7274                   <1>     jc     short syscreat_1 ; file not found (the good!)
                                <1>                              ; or another error (the bad!)
                                <1>
                                <1>     ; (& the uggly!) truncate file to zero length before open
                                <1>
                                <1>     ;'*' and '?' already checked at 'set_working_path' stage
                                <1>     ;and   dx, dx
                                <1>     ;jnz   short sysmkdir_err ; permission denied
                                <1>                               ; invalid filename chars
                                <1>
                                <1>     ;test  cl, 10h ; subdirectory ?
                                <1>     ;jnz   short sysmkdir_err
                                <1>
                                <1>     ; BL = File Attributes:
                                <1>     ;            bit 0 (1) - Read only file (R)
                                <1>     ;             bit 1 (1) - Hidden file (H)
                                <1>         ;            bit 2 (1) - System file (R)
                                <1>     ;            bit 3 (1) - Volume label/name (V)
                                <1>         ;            bit 4 (1) - Subdirectory (D)
                                <1>     ;            bit 5 (1) - File has been archived
                                <1>
                                <1>     ; * existing directory must not be truncated
                                <1>     ;   (we don't know it is empty or not, at this stage)
                                <1>     ; * existing volume name (or a long name) can not be
                                <1>     ;   re-created or truncated by 'syscreat'
                                <1>     ; * A file with S, H, R attributes must not be truncated
                                <1>     ;   (change attributes to normal, if you need truncate it)
                                <1>
0000CF32 F6C31F                 <1>     test   bl, 00011111b  ; check attributes of existing file
0000CF35 7559                   <1>     jnz    short sysmkdir_err
                                <1>
                                <1>     ;; normal file, OK to continue...
                                <1>
                                <1>     ; ESI = FindFile_DirEntry
0000CF37 668B4614               <1>     mov    ax, [esi+DirEntry_FstClusHI] ; 20
0000CF3B C1E010                 <1>     shl    eax, 16 ; 13/11/2017
0000CF3E 668B461A               <1>     mov    ax, [esi+DirEntry_FstClusLO] ; 26
                                <1>
                                <1>     ; 24/08/2024
0000CF42 21C0                   <1>     and    eax, eax
0000CF44 7426                   <1>     jz     short skip_truncate ; first cluster is 0 !
                                <1>
                                <1>     ; EAX = First cluster to be truncated/unlinked
0000CF46 57                     <1>     push   edi
0000CF47 51                     <1>     push   ecx
0000CF48 BE00010900             <1>     mov    esi, Logical_DOSDisks
0000CF4D 29C9                   <1>     sub    ecx, ecx
                                <1>     ;mov   ch, [Current_Drv] ; = [FindFile_Drv]
                                <1>     ; 03/09/2024
0000CF4F 8A2D[16800100]         <1>     mov    ch, [FindFile_Drv]
0000CF55 01CE                   <1>     add    esi, ecx
                                <1>     ; ESI = Logical dos drive description table address
0000CF57 E8C2F7FFFF             <1>     call   truncate_cluster_chain
0000CF5C 59                     <1>     pop    ecx
0000CF5D 5F                     <1>     pop    edi
0000CF5E 7237                   <1>     jc     short syscreate_truncate_err
                                <1>
                                <1>     ; 03/09/2024
0000CF60 66C747140000           <1>     mov    word [edi+DirEntry_FstClusHI], 0
0000CF66 66C7471A0000           <1>     mov    word [edi+DirEntry_FstClusLO], 0
                                <1>
                                <1>     ; 24/08/2024
                                <1> skip_truncate:
                                <1>     ; 26/10/2016
                                <1>     ; EDI = Directory entry address in directory buffer
                                <1>     ; Update directory entry
0000CF6C E88BDDFFFF             <1>     call   convert_current_date_time
                                <1>     ; OUTPUT -> DX = Date in dos dir entry format
                                <1>         ;       AX = Time in dos dir entry format
0000CF71 66894716               <1>     mov    [edi+DirEntry_WrtTime], ax
0000CF75 66895718               <1>     mov    [edi+DirEntry_WrtDate], dx
0000CF79 66895712               <1>     mov    [edi+DirEntry_LastAccDate], dx
0000CF7D 31C0                   <1>     xor    eax, eax ; file size = 0
0000CF7F 89471C                 <1>     mov    [edi+DirEntry_FileSize], eax ; 0
0000CF82 C605[9D7E0100]02       <1>     mov    byte [DirBuff_ValidData], 2 ; data changed sign
                                <1>     ;mov   esi, FindFile_DirEntry
                                <1>     ; 03/09/2024
0000CF89 B201                   <1>     mov    dl, 1 ; open file for writing
0000CF8B E9AB000000             <1>     jmp    sysopen_2 ; 08/08/2022
                                <1>
                                <1> sysmkdir_err:
                                <1>     ; 1 = write, 2 = read & write, >2 = invalid
0000CF90 B80B000000             <1>         mov eax, ERR_FILE_ACCESS  ; 11 ; 'permission denied !'
0000CF95 EB74                   <1>         jmp short sysopen_err
                                <1>
                                <1> syscreate_truncate_err:
0000CF97 B812000000             <1>     mov    eax, ERR_DRV_WRITE ; 18 ; 'disk write error !'
0000CF9C EB6D                   <1>         jmp short sysopen_err
                                <1>
                                <1> syscreat_inv_fname:  ; invalid file name chars
                                <1>     ; 16/10/2016
0000CF9E B81A000000             <1>     mov    eax, ERR_INV_FILE_NAME  ; 26 ; invalid file name chars
0000CFA3 59                     <1>     pop    ecx
0000CFA4 EB65                   <1>     jmp    short sysopen_err
                                <1>
                                <1> syscreat_1:
                                <1>     ; Error code in EAX
0000CFA6 3C02                   <1>         cmp al, 02h ; 'File not found' error
0000CFA8 7561                   <1>         jne short sysopen_err
                                <1>
0000CFAA F6C110                 <1>     test   cl, 10h ; Directory
                                <1>     ;jnz   sysmkdir_2
                                <1>     ; 23/07/2022
0000CFAD 7405                   <1>     jz     short syscreat_2
0000CFAF E9C3010000             <1>     jmp    sysmkdir_2
                                <1>
                                <1> syscreat_2:
0000CFB4 BE[58800100]           <1>     mov    esi, FindFile_Name
                                <1>         ;xor       edx, edx
0000CFB9 31C0                   <1>         xor eax, eax ; File Size  = 0
0000CFBB 31DB                   <1>     xor    ebx, ebx
0000CFBD 4B                     <1>     dec    ebx ; FFFFFFFFh -> create empty file
                                <1>                 ;              (only for FAT fs)
```

```
                                <1>     ; CL = File Attributes
0000CFBE E8B0ECFFFF             <1>     call    create_file
0000CFC3 7246                   <1>     jc      short sysopen_err
                                <1>             ; EAX = New file's first cluster
                                <1>             ; ESI = Logical Dos Drv Descr. Table Addr.
                                <1>             ; EBX = offset CreateFile_Size
                                <1>             ; ECX = Sectors per cluster (<256)
                                <1>             ; EDX = Directory entry index/number (<65536)
                                <1>             ; 29/08/2024
                                <1>             ; EBX = File Size (0 for a new, empty file)
                                <1>             ; ECX = Directory Entry Index/Number (<2048)
                                <1>             ;      (in directory cluster, not in directory)
                                <1>             ; EDX = Directory Cluster Number (of the file)
                                <1>
                                <1>     ; 26/10/2016
                                <1>     ;mov    esi, Directory_Buffer
                                <1>     ;shl    dx, 5 ; *32
                                <1>     ;add    esi, edx
                                <1>     ;; esi = directory entry address in directory buffer
                                <1>
                                <1>     ; Here, directory entry has been created but last
                                <1>     ; modification date & time of the parent dir has not
                                <1>     ; been updated, yet!
                                <1>     ; (Note: Directory and FAT buffers have been updated...)
                                <1>
0000CFC5 E850DEFFFF             <1>     call    update_parent_dir_lmdt ; now, it is OK too!
                                <1>
                                <1>     ; 25/10/2016
0000CFCA 66B80018               <1>     mov     ax, 1800h
0000CFCE BE[58800100]           <1>     mov     esi, FindFile_Name
0000CFD3 E8A2B9FFFF             <1>     call    find_first_file
0000CFD8 7231                   <1>     jc      short sysopen_err
                                <1>
                                <1>     ; Only possible error after here is
                                <1>     ; "too many open files !" error.
                                <1>     ;
                                <1>     ; If "syscreat" will return with that error,
                                <1>     ; (the file has been created but it could not be opened)
                                <1>     ; the user must retry to open this file again
                                <1>     ; or must close another file before using
                                <1>     ; "sysopen" system call.
                                <1>
0000CFDA B201                   <1>     mov     dl, 1 ; open file for writing
                                <1>     ; ESI = Directory Entry (FindFile_DirEntry) Location
                                <1>     ; EAX = File Size (= 0)
0000CFDC EB5D                   <1>     jmp     short sysopen_2
                                <1>
                                <1> sysopen: ;<open file>
                                <1>     ; 19/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 03/09/2024
                                <1>     ; 19/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;      (temporary modifications)
                                <1>     ; 26/10/2016
                                <1>     ; 24/10/2016
                                <1>     ; 17/10/2016
                                <1>     ; 15/10/2016
                                <1>     ; 06/10/2016, 07/10/2016, 08/10/2016
                                <1>     ; 05/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;            -derived from INT_21H.ASM-
                                <1>     ;             ("loc_INT21h_open_file")
                                <1>         ;   26/02/2011
                                <1>         ;   INT 21h Function AH = 3Dh
                                <1>         ;   Open File
                                <1>         ;   INPUT
                                <1>         ;      AL= File Access Value
                                <1>     ;             0- Open for reading
                                <1>     ;             1- Open for writing
                                <1>         ;            2- Open for reading and writing
                                <1>         ;          DS:DX= Pointer to filename (ASCIIZ)
                                <1>         ;
                                <1>     ; 14/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 22/05/2013 - 27/05/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysopen' opens a file in following manner:
                                <1>     ;    1) The second argument in a sysopen says whether to
                                <1>     ;       open the file ro read (0) or write (>0).
                                <1>     ;    2) I-node of the particular file is obtained via 'namei'.
                                <1>     ;    3) The file is opened by 'iopen'.
                                <1>     ;    4) Next housekeeping is performed on the fsp table
                                <1>     ;       and the user's open file list - u.fp.
                                <1>     ;       a) u.fp and fsp are scanned for the next available slot.
                                <1>     ;       b) An entry for the file is created in the fsp table.
                                <1>     ;       c) The number of this entry is put on u.fp list.
                                <1>     ;       d) The file descriptor index to u.fp list is pointed
                                <1>     ;          to by u.r0.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysopen; name; mode
                                <1>     ; Arguments:
                                <1>     ;       name - file name or path name
                                <1>     ;       mode - 0 to open for reading
                                <1>     ;              1 to open for writing
                                <1>     ; Inputs: (arguments)
                                <1>     ; Outputs: *u.r0 - index to u.fp list (the file descriptor)
                                <1>     ;                is put into r0's location on the stack.
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       'sysopen' system call has two arguments; so,
                                <1>     ;       * 1st argument, name is pointed to by BX register
                                <1>     ;       * 2nd argument, mode is in CX register
                                <1>     ;
                                <1>     ;       AX register (will be restored via 'u.r0') will return
                                <1>     ;       to the user with the file descriptor/number
                                <1>     ;       (index to u.fp list).
                                <1>     ;
                                <1>     ;call  arg2
                                <1>     ; * name - 'u.namep' points to address of file/path name
                                <1>     ;          in the user's program segment ('u.segmnt')
                                <1>     ;          with offset in BX register (as sysopen argument 1).
                                <1>     ; * mode - sysopen argument 2 is in CX register
                                <1>     ;          which is on top of stack.
                                <1>     ;
                                <1>     ; jsr r0,arg2 / get sys args into u.namep and on stack
                                <1>     ;
                                <1>             ; system call registers: ebx, ecx (through 'sysenter')
                                <1>     ;
                                <1>     ; TRDOS 386 (05/10/2016)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;      CL = File Access Value (Open Mode)
                                <1>     ;            0 - Open file for reading
```

```
                                <1>     ;             1 - Open file for writing
                                <1>         ;             2 - Open device for reading
                                <1>     ;             3 - Open device for writing
                                <1>         ;             EBX = Pointer to filename/devicename (ASCIIZ)
                                <1>     ; OUTPUT ->
                                <1>     ;          eax = File/Device Handle/Number (index) (AL)
                                <1>     ;          cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>
0000CFDE 80F901                 <1>     cmp    cl, 1 ; read file (0), write file (1)
0000CFE1 7614                   <1>     jna    short sysopen_0
                                <1>
                                <1>     ; 17/04/2021 (temporary)
                                <1>     ;cmp   cl, 3
                                <1>     ;jna   sysopen_device
                                <1>
                                <1>     ; Invalid access code
0000CFE3 B817000000             <1>     mov    eax, ERR_INV_PARAMETER
                                <1>     ;jmp   sysopen_dev_err
                                <1>
                                <1> sysopen_dev_err:
0000CFE8 A3[6C8E0100]           <1>     mov    [u.r0], eax
0000CFED A3[D88E0100]           <1>     mov    [u.error], eax
0000CFF2 E977FAFFFF             <1>     jmp    error
                                <1>
                                <1> sysopen_0:
                                <1>     ;mov   [u.namep], ebx
0000CFF7 51                     <1>     push   ecx
0000CFF8 89DE                   <1>     mov    esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov   ax, 1
                                <1>     ;mov   [FFF_Valid], ah ; 0 ; reset ; 17/10/2016
                                <1>     ;call  set_working_path
0000CFFA E8D64C0000             <1>     call   set_working_path_x ; 17/10/2016
0000CFFF 731E                   <1>     jnc    short sysopen_1
                                <1>
                                <1> syscreat_err: ; ecx = file attributes (for 'syscreat')
0000D001 59                     <1>     pop    ecx ; open mode
0000D002 21C0                   <1>     and    eax, eax  ; 0 -> Bad Path!
0000D004 7505                   <1>     jnz    short sysopen_err
                                <1>     ; eax = 0
0000D006 B80C000000             <1>     mov    eax, ERR_DIR_NOT_FOUND ; Directory not found !
                                <1> sysopen_err:
0000D00B A3[6C8E0100]           <1>     mov    [u.r0], eax
0000D010 A3[D88E0100]           <1>     mov    [u.error], eax
0000D015 E8904D0000             <1>     call   reset_working_path
0000D01A E94FFAFFFF             <1>     jmp    error
                                <1>
                                <1> sysopen_1:
                                <1>     ;mov   esi, FindFile_Name
0000D01F 66B80018               <1>     mov    ax, 1800h ; Only files
0000D023 E852B9FFFF             <1>     call   find_first_file
0000D028 5A                     <1>     pop    edx
0000D029 72E0                   <1>     jc     short sysopen_err ; eax = 2 (File not found !)
                                <1>
                                <1>     ; check_open_file_attr_access_code
                                <1>
0000D02B F6C307                 <1>     test   bl, 7  ; system, hidden, readonly
0000D02E 740B                   <1>     jz     short sysopen_2
                                <1>
0000D030 20D2                   <1>     and    dl, dl ; 0 = read mode
0000D032 7407                   <1>     jz     short sysopen_2
                                <1>
                                <1> sysopen_access_err:
                                <1>     ; 1 = write, 2 = read & write, >2 = invalid
0000D034 B80B000000             <1>     mov    eax, ERR_FILE_ACCESS ; 11 = 'permission denied !'
0000D039 EBD0                   <1>     jmp    short sysopen_err
                                <1>
                                <1> sysopen_2:
                                <1>     ; esi = Directory Entry (FindFile_DirEntry) Location
                                <1>     ;mov   ebx, esi
                                <1>     ; 03/09/2024
0000D03B 89FB                   <1>     mov    ebx, edi ; Directory entry in directory buffer
0000D03D 31F6                   <1>     xor     esi, esi ; 0
0000D03F 31FF                   <1>     xor     edi, edi ; 0
                                <1> sysopen_3: ; scan the list of entries in fsp table
0000D041 80BE[768E0100]00       <1>     cmp     byte [esi+u.fp], 0
0000D048 760E                   <1>     jna     short sysopen_4 ; empty slot
                                <1>     ;inc   si
                                <1>     ; 19/08/2024
0000D04A 46                     <1>     inc    esi
0000D04B 6683FE0A               <1>     cmp     si, 10
0000D04F 72F0                   <1>     jb     short sysopen_3
                                <1> toomanyf:
0000D051 B80D000000             <1>     mov    eax, ERR_TOO_MANY_FILES ; too many open files !
0000D056 EBB3                   <1>     jmp    short sysopen_err
                                <1>
                                <1> sysopen_4:
0000D058 80BF[18840100]00       <1>     cmp     byte [edi+OF_MODE], 0 ; Scan open files table
0000D05F 7609                   <1>     jna     short sysopen_5
                                <1>
                                <1>     ;inc   di
                                <1>     ; 19/08/2024
0000D061 47                     <1>     inc    edi
0000D062 6683FF20               <1>     cmp     di, OPENFILES ; max. number of open files (sytem)
0000D066 72F0                   <1>     jb     short sysopen_4
0000D068 EBE7                   <1>     jmp    short toomanyf
                                <1>
                                <1> sysopen_5:
                                <1>     ; 19/08/2024
                                <1>     ;;;
0000D06A 50                     <1>     push   eax ; size
0000D06B 668B4314               <1>     mov    ax, [ebx+DirEntry_FstClusHI]
0000D06F C1E010                 <1>     shl    eax, 16
0000D072 668B431A               <1>     mov    ax, [ebx+DirEntry_FstClusLO]
0000D076 5B                     <1>     pop    ebx ; size
                                <1>     ; eax = First Cluster
                                <1>     ; ebx = File Size
0000D077 80FA01                 <1>     cmp    dl, 1 ; is open mode = open for write ?
0000D07A 7535                   <1>     jne    short sysopen_8
                                <1>     ; check if the file is already open for write
0000D07C 57                     <1>     push   edi
0000D07D 29FF                   <1>     sub    edi, edi
0000D07F 8A15[16800100]         <1>     mov    dl, [FindFile_Drv]
                                <1> sysopen_5_@:
0000D085 80BF[18840100]02       <1>     cmp    byte [edi+OF_MODE], 2 ; open for write
0000D08C 7519                   <1>     jne    short sysopen_5_@@@
0000D08E 3A97[F8830100]         <1>     cmp    dl, [edi+OF_DRIVE]
0000D094 7511                   <1>     jne    short sysopen_5_@@@
0000D096 C1E702                 <1>     shl    edi, 2
0000D099 3B87[78830100]         <1>     cmp    eax, [edi+OF_FCLUSTER]
0000D09F 7503                   <1>     jne    short sysopen_5_@@
```

```
0000D0A1 5F                 <1>     pop     edi
0000D0A2 EB90               <1>     jmp     short sysopen_access_err
                            <1> sysopen_5_@@:
0000D0A4 C1EF02             <1>     shr     edi, 2
                            <1>     ;;;
                            <1> sysopen_5_@@@:
0000D0A7 47                 <1>     inc     edi
0000D0A8 6683FF20           <1>     cmp     di, OPENFILES
0000D0AC 72D7               <1>     jb      short sysopen_5_@
0000D0AE 5F                 <1>     pop     edi
0000D0AF B201               <1>     mov     dl, 1 ; open for write
                            <1> sysopen_8:
0000D0B1 FEC2               <1>     inc     dl
0000D0B3 8897[18840100]     <1>     mov     [edi+OF_MODE], dl
0000D0B9 8A15[16800100]     <1>     mov     dl, [FindFile_Drv]
0000D0BF 8897[F8830100]     <1>     mov     [edi+OF_DRIVE], dl ; Logical DOS drive number
                            <1>     ;shl    di, 2 ; *4 (dword offset)
                            <1>     ; 23/07/2022
0000D0C5 C1E702             <1>     shl     edi, 2
                            <1>
                            <1>     ; 19/08/2024
                            <1>     ;mov    [edi+OF_SIZE], eax ; File size in bytes
0000D0C8 899F[F8840100]     <1>     mov     [edi+OF_SIZE], ebx ; File size in bytes
                            <1>
                            <1>     ; 19/08/2024
                            <1>     ; eax = Fist Cluster
                            <1>     ;mov    ax, [ebx+DirEntry_FstClusHI]
                            <1>     ;shl    eax, 16
                            <1>     ;mov    ax, [ebx+DirEntry_FstClusLO]
                            <1>
0000D0CE 8987[78830100]     <1>     mov     [edi+OF_FCLUSTER], eax ; First cluster
0000D0D4 8987[F8860100]     <1>     mov     [edi+OF_CCLUSTER], eax ; Current cluster
                            <1>
0000D0DA 31DB               <1>     xor     ebx, ebx
0000D0DC 899F[78840100]     <1>     mov     [edi+OF_POINTER], ebx ; offset pointer (0)
0000D0E2 899F[78870100]     <1>     mov     [edi+OF_CCINDEX], ebx ; cluster index (0)
                            <1>
0000D0E8 A1[88800100]       <1>     mov     eax, [FindFile_DirFirstCluster]
0000D0ED 8987[78850100]     <1>     mov     [edi+OF_DIRFCLUSTER], eax
                            <1>
0000D0F3 A1[8C800100]       <1>     mov     eax, [FindFile_DirCluster]
0000D0F8 8987[F8850100]     <1>     mov     [edi+OF_DIRCLUSTER], eax
                            <1>
                            <1>     ; Get (& Save) Volume ID
                            <1>     ; Important for files of removable drives
                            <1>     ; (In order to check the drive has same volume/disk)
0000D0FE 88D7               <1>     mov     bh, dl
0000D100 81C300010900       <1>     add     ebx, Logical_DOSDisks
                            <1>
0000D106 8A4303             <1>     mov     al, [ebx+LD_FATType]
                            <1>
                            <1> ; 19/12/2025
                            <1> %if 0
                            <1>     cmp     al, 1
                            <1>     jb      short sysopen_6_fs
                            <1> %endif
0000D109 3C02               <1>     cmp     al, 2
0000D10B 7705               <1>     ja      short sysopen_6_fat32
                            <1> sysopen_6_fat:
0000D10D 8B432D             <1>     mov     eax, [ebx+LD_BPB+VolumeID]
0000D110 EB03               <1>     jmp     short sysopen_7
                            <1>
                            <1> ; 19/12/2025
                            <1> %if 0
                            <1> sysopen_6_fs:
                            <1>     mov     eax, [ebx+LD_FS_VolumeSerial]
                            <1>     jmp     short sysopen_7
                            <1> %endif
                            <1>
                            <1> sysopen_6_fat32:
0000D112 8B4349             <1>     mov     eax, [ebx+LD_BPB+FAT32_VolID]
                            <1> sysopen_7:
0000D115 A3[6C770100]       <1>     mov     [Current_VolSerial], eax
                            <1>
0000D11A 8987[78860100]     <1>     mov     [edi+OF_VOLUMEID], eax
                            <1>
                            <1>     ; 24/10/2016
                            <1>     ;shr    di, 1 ; 4/2, word offset
                            <1>     ; 23/07/2022
0000D120 D1EF               <1>     shr     edi, 1
0000D122 668B1D[90800100]   <1>     mov     bx, [FindFile_DirEntryNumber]
0000D129 66899F[F8870100]   <1>     mov     [edi+OF_DIRENTRY], bx
                            <1>
0000D130 31D2               <1>     xor     edx, edx
                            <1>     ;;shr   di, 2 ; /4 (byte offset)
                            <1>     ;shr    di, 1 ; 2/2, byte offset
                            <1>     ; 23/07/2022
0000D132 D1EF               <1>     shr     edi, 1
0000D134 8897[58840100]     <1>     mov     byte [edi+OF_OPENCOUNT], dl ; 0
0000D13A 8897[38840100]     <1>     mov     byte [edi+OF_STATUS], dl ; 0
                            <1>
0000D140 89FB               <1>     mov     ebx, edi
0000D142 FEC3               <1>     inc     bl
                            <1>
0000D144 889E[768E0100]     <1>     mov     [esi+u.fp], bl ; Open File Entry Number
0000D14A 8935[6C8E0100]     <1>     mov     [u.r0], esi ; move index to u.fp list
                            <1>                         ; into eax on stack
                            <1>
0000D150 E8554C0000         <1>     call    reset_working_path
                            <1>
0000D155 E934F9FFFF         <1>     jmp     sysret
                            <1>
                            <1>
                            <1> ; fsp table (original UNIX v1)
                            <1> ;
                            <1> ;Entry
                            <1> ;          15                                      0
                            <1> ;  1      |---|-----------------------------------------|
                            <1> ;         |r/w|       i-number of open file             |
                            <1> ;         |---|-----------------------------------------|
                            <1> ;         |               device number                 |
                            <1> ;         |-----------------------------------------|
                            <1> ;     (*) | offset pointer, i.e., r/w pointer to file |
                            <1> ;         |-----------------------------------------|
                            <1> ;         |  flag that says    | number of processes  |
                            <1> ;         |   file deleted     | that have file open  |
                            <1> ;         |-----------------------------------------|
                            <1> ;  2      |                                         |
                            <1> ;         |-----------------------------------------|
                            <1> ;         |                                         |
                            <1> ;         |-----------------------------------------|
                            <1> ;         |                                         |
                            <1> ;         |-----------------------------------------|
                            <1> ;         |                                         |
```

```
<1> ;        |-----------------------------------------|
<1> ;  3     |                                         |
<1> ;        |                                         |
<1> ;
<1> ; (*) Retro UNIX 386 v1 modification: 32 bit offset pointer
<1>
<1> ; 27/03/2020 - Retro UNIX 386 v2 - FSP (OPEN FILES) TABLE
<1>
<1> ;Entry
<1> ;         15                    7                  0
<1> ;  1     |-----------------------------------------|
<1> ;        |              i-number of open file      |
<1> ;        |-----------------------------------------|
<1> ;        |        high word of 32 bit i-number     |
<1> ;        |-----------------------------------------|
<1> ;        | open mode & status  |   device number   |
<1> ;        |-----------------------------------------|
<1> ;        |    reserved byte    |     open count    |
<1> ;        |-----------------------------------------|
<1> ;        | offset pointer, i.e., r/w pointer to file |
<1> ;        |-----------------------------------------|
<1> ;        |   64 bit file offset pointer (bit 16-31)  |
<1> ;        |-----------------------------------------|
<1> ;        |   64 bit file offset pointer (bit 32-47)  |
<1> ;        |-----------------------------------------|
<1> ;        |   64 bit file offset pointer (bit 48-63)  |
<1> ;        |-----------------------------------------|
<1> ;  2     |                                         |
<1> ;        |-----------------------------------------|
<1> ;        |                                         |
<1> ;        |-----------------------------------------|
<1> ;        |                                         |
<1> ;        |-----------------------------------------|
<1> ;        |                                         |
<1> ;        |-----------------------------------------|
<1> ;        |                                         |
<1>
<1> ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
<1> ; OPENFILES equ 10 (sysdefs.s)
<1> ;; 06/10/2016
<1> ;; Open File Parameters (trdoskx.s)
<1> ;OF_FCLUSTER:      resd OPENFILES  ; First clusters of open files
<1> ;OF_DRIVE:  resb OPENFILES  ; Logical DOS drive numbers of open files
<1> ;OF_MODE:   resb OPENFILES  ; Open mode (1 = read, 2 = write, 3 = r&w)
<1> ;OF_STATUS: resb OPENFILES  ; (bit 0 = read, bit 1 = write)
<1> ;OF_OPENCOUNT:     resb OPENFILES  ; Open counts of open files
<1> ;OF_POINTER:       resd OPENFILES ; File seek/read/write pointer
<1> ;OF_SIZE:   resd OPENFILES ; File sizes of open files (in bytes)
<1> ;OF_DIRFCLUSTER: resd OPENFILES  ; Directory First Clusters of open files
<1> ;OF_DIRCLUSTER:    resd OPENFILES  ; Directory (Entry) Clusters of open files
<1> ;OF_VOLUMEID:      resd OPENFILES  ; Vol ID for removable drives of open files
<1> ;OF_CCLUSTER:      resd OPENFILES  ; Current clusters of open files
<1> ;OF_CCINDEX:       resd OPENFILES  ; Cluster index numbers of current clusters
<1> ;; 24/10/2016
<1> ;OF_DIRENTRY:      resw OPENFILES  ; Directory entry index no. in dir cluster
<1>
<1> ; 17/04/2021
<1> ; ('sysopen_device' procedure is disabled as temporary)
<1>
<1> ;sysopen_device:
<1> ;   ; 15/10/2016
<1> ;   ; 08/10/2016
<1> ;   ; 07/10/2016 (TRDOS 386 = TRDOS v2.0)
<1> ;   push    ecx ; open mode
<1> ;   mov     ebp, esp
<1> ;   mov     ecx, 16 ; transfer length = 16 bytes
<1> ;   sub     esp, ecx
<1> ;   mov     edi, esp ; destination address
<1> ;   mov     esi, ebx ; dev name in user's memory space
<1> ;   call    transfer_from_user_buffer
<1> ;   jnc     short sysopen_dev_0
<1> ;   ; eax = ERR_OUT_OF_MEMORY = 4 = ERR_MINOR_IM
<1> ;   pop     ecx
<1> ;sysopen_dev_err:
<1> ;   mov     [u.r0], eax
<1> ;   mov     [u.error], eax
<1> ;   jmp     error
<1> ;sysopen_dev_0:
<1> ;   mov     esi, edi ; Device name addr (max. 16 bytes, ASCIIZ)
<1> ;                    ; for example: "tty, TTY, /dev/tty"
<1> ;   call    get_device_number
<1> ;   mov     esp, ebp
<1> ;   pop     ecx
<1> ;   jnc     short sysopen_dev_1
<1> ;   mov     eax, ERR_INV_DEV_NAME ; 24 ; 'invalid device name !'
<1> ;   jmp     short sysopen_dev_err
<1> ;sysopen_dev_1:
<1> ;   ; eax = Device Number (AL)
<1> ;   ;  cl = Open mode (2 = device read, 3 = device write)
<1> ;       xor     ebx, ebx ; 0
<1> ;sysopen_dev_2: ; scan the list of entries
<1> ;       cmp     [ebx+u.fp], bl ; 0
<1> ;       jna     short sysopen_dev_3 ; empty slot
<1> ;       inc     bl
<1> ;       cmp     bl, 10
<1> ;   jb      short sysopen_dev_2
<1> ;   ;
<1> ;   mov     eax, ERR_TOO_MANY_FILES ; too many open files !
<1> ;   jmp     short sysopen_dev_err
<1> ;sysopen_dev_3:
<1> ;   mov     [u.r0], ebx ; File/Device index/handle/descriptor
<1> ;   ; eax = device number (entry offset)
<1> ;   mov     ch, [eax+DEV_ACCESS] ; bit 0 = accessable by users
<1> ;                                ; bit 1 = read access perm
<1> ;                                ; bit 2 = write access perm
<1> ;                                ; bit 3 = IOCTL permit to users
<1> ;                                ; bit 4 = block device if set
<1> ;                                ; bit 5 = 16 bit or 1024 byte
<1> ;                                ; bit 6 = 32 bit or 2048 byte
<1> ;                                ; bit 7 = installable device drv
<1> ;   test    ch, 1 ; accessable by normal users (except root)
<1> ;   jnz     short sysopen_dev_4 ; yes, permission has been given
<1> ;   cmp     byte [u.uid], 0 ; root?
<1> ;   jna     short sysopen_dev_4 ; superuser can open all devices
<1> ;sysopen_dev_perm_err:
<1> ;   mov     eax, ERR_DEV_ACCESS  ; 11 = 'permission denied !'
<1> ;   jmp     short sysopen_dev_err
<1> ;sysopen_dev_4:
<1> ;   shr     ch, 1 ; result: 1 = read, 2 = write, 3 = r & w
<1> ;   dec     cl  ; result: 1 = read, 2 = write
<1> ;   test    cl, ch
<1> ;   jz      short sysopen_dev_perm_err
<1> ;
<1> ;   shl     ch, 1 ; bit 0 = 0
```

```
                                <1> ;   ; eax = device number (entry offset)
                                <1> ;   call    device_open
                                <1> ;   jc      short sysopen_dev_perm_err
                                <1> ;
                                <1> ;   ; eax = device number (entry offset)
                                <1> ;   or      al, 80h ; set device bit (set bit 7 to 1)
                                <1> ;   mov     ebx, [u.r0]
                                <1> ;   mov     [ebx+u.fp], al ; bit 7 (=1) points to device
                                <1> ;
                                <1> ;   jmp     sysret
                                <1>
                                <1> sysmkdir: ; < make directory >
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 15/10/2016
                                <1>     ; 10/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;            -derived from INT_21H.ASM-
                                <1>     ;             ("loc_INT21h_create_file")
                                <1>         ;   10/07/2011 (12/03/2011)
                                <1>         ;   INT 21h Function AH = 3Ch
                                <1>         ;   Create File
                                <1>         ;   INPUT
                                <1>         ;      CX = Attributes
                                <1>         ;           DS:DX= Address of zero terminaned path name
                                <1>         ;
                                <1>         ;
                                <1>     ; 14/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 27/05/2013 - 02/08/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysmkdir' creates an empty directory whose name is
                                <1>     ; pointed to by arg 1. The mode of the directory is arg 2.
                                <1>     ; The special entries '.' and '..' are not present.
                                <1>     ; Errors are indicated if the directory already exists or
                                <1>     ; user is not the super user.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysmkdir; name; mode
                                <1>     ; Arguments:
                                <1>     ;       name - points to the name of the directory
                                <1>     ;       mode - mode of the directory
                                <1>     ; Inputs: (arguments)
                                <1>     ; Outputs: -
                                <1>     ;    (sets 'directory' flag to 1;
                                <1>     ;    'set user id on execution' and 'executable' flags to 0)
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       'sysmkdir' system call has two arguments; so,
                                <1>     ;       * 1st argument, name is pointed to by BX register
                                <1>     ;       * 2nd argument, mode is in CX register
                                <1>     ;
                                <1>     ; TRDOS 386 (10/10/2016)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;      CL = Directory Attributes
                                <1>     ;              bit 0 (1) - Read only file/dir (R)
                                <1>     ;              bit 1 (1) - Hidden file/dir (H)
                                <1>         ;              bit 2 (1) - System file/dir (R)
                                <1>     ;              bit 3 (1) - Volume label/name (V)
                                <1>         ;              bit 4 (1) - Subdirectory (D)
                                <1>     ;              bit 5 (1) - File/Dir has been archived (A)
                                <1>     ;          CX = 0 -> create normal directory
                                <1>         ;           EBX = Pointer to directory name (ASCIIZ) -path-
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;          eax = First cluster of the new directory
                                <1>     ;          cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; Note: If the file or directory is existing
                                <1>     ;       an access error will be returned.
                                <1>
0000D15A 6621C9                 <1>     and     cx, cx ; if cx = 0 -> create a normal subdir
0000D15D 7414                   <1>     jz      short sysmkdir_1
                                <1>
0000D15F F6C110                 <1>     test    cl, 10h ; if dir flags set, also use other flags
                                <1>     ;jnz    sysmkdir_0 ; jump to head of 'syscreat'
                                <1>     ; 23/07/2022
0000D162 7405                   <1>     jz      short sysmkdir_invf
                                <1> sysmkdir_3:
0000D164 E998FDFFFF             <1>     jmp     sysmkdir_0
                                <1>
                                <1> sysmkdir_invf:
                                <1>     ; CX has wrong flags
0000D169 B817000000             <1>     mov     eax, ERR_INV_FLAGS
0000D16E E975FEFFFF             <1>     jmp     sysopen_dev_err
                                <1>
                                <1> sysmkdir_1:
0000D173 B110                   <1>     mov     cl, 10h ; set subdir flag and reset other flags
                                <1>     ;jmp    sysmkdir_0
                                <1>     ; 23/07/2022
0000D175 EBED                   <1>     jmp     short sysmkdir_3 ; jump to head of 'syscreat'
                                <1> sysmkdir_2:
                                <1>     ; jump from 'syscreat' ; from 'syscreat_1'
                                <1>     ;  CL = Directory attributes/flags
0000D177 BE[58800100]           <1>     mov     esi, FindFile_Name
0000D17C E8E0D8FFFF             <1>     call    make_sub_directory
                                <1>     ;jc     sysopen_err       ; NOTE: Old type (TRDOS 8086)
                                <1>                               ; error codes must be modified
                                <1>                               ; for next TRDOS 386 versions
                                <1>                               ; (10/10/2016)
                                <1>                               ; Old (MSDOS type)
                                <1>                               ; error codes (2011):
                                <1>                               ;  2 = file not found
                                <1>                               ;  3 = directory not found
                                <1>                               ;  5 = access denied
                                <1>                               ; 12 = no more files
                                <1>                               ; 19 = disk write protected
                                <1>                               ; 39 = insufficient disk space
                                <1>                               ; 'sysdefs.s' ; 10/10/2016
                                <1>     ; 23/07/2022
0000D181 7305                   <1>     jnc     short sysmkdir_4
0000D183 E983FEFFFF             <1>     jmp     sysopen_err
                                <1>
                                <1> sysmkdir_4:
0000D188 A3[6C8E0100]           <1>     mov     [u.r0], eax ; New sub dir's first cluster
0000D18D E8184C0000             <1>         call       reset_working_path
0000D192 E9F7F8FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysclose: ;<close file>
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 06/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; 14/05/2015 (Retro UNIX 386 v1 - Beginning)
```

```
                                <1>     ; 22/05/2013 - 26/05/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysclose', given a file descriptor in 'u.r0', closes the
                                <1>     ; associated file. The file descriptor (index to 'u.fp' list)
                                <1>     ; is put in r1 and 'fclose' is called.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysclose
                                <1>     ; Arguments:
                                <1>     ;       -
                                <1>     ; Inputs: *u.r0 - file descriptor
                                <1>     ; Outputs: -
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       The user/application program puts file descriptor
                                <1>     ;        in BX register as 'sysclose' system call argument.
                                <1>     ;       (argument transfer method 1)
                                <1>
                                <1>     ; TRDOS 386 (06/10/2016)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;     EBX = File Handle/Number (file index) (AL)
                                <1>     ; OUTPUT ->
                                <1>     ;         cf = 0 -> EAX = 0
                                <1>     ;         cf = 1 -> Error code in EAX (ERR_FILE_NOT_OPEN)
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>
0000D197 89D8                   <1>     mov     eax, ebx
0000D199 31DB                   <1>     xor     ebx, ebx
0000D19B 891D[6C8E0100]         <1>     mov     [u.r0], ebx ; 0  ; return value of EAX
0000D1A1 E8B5300000             <1>     call    fclose
                                <1>     ;jnc    sysret
                                <1>     ; 23/07/2022
0000D1A6 7205                   <1>     jc      short sysclose_err
0000D1A8 E9E1F8FFFF             <1>     jmp     sysret
                                <1> sysclose_err:
                                <1>     ; 24/04/2025
                                <1> seektell_err:
                                <1> device_rw_err:
0000D1AD B80A000000             <1>     mov     eax, ERR_FILE_NOT_OPEN ; file not open !
0000D1B2 A3[D88E0100]           <1>     mov     [u.error], eax ;
0000D1B7 A3[6C8E0100]           <1>     mov     [u.r0], eax ; ! invalid handle !
0000D1BC E9ADF8FFFF             <1>     jmp     error
                                <1>
                                <1> sysread: ; < read from file >
                                <1>     ; 27/09/2024
                                <1>     ; 18/09/2024
                                <1>     ; 03/09/2024 (TRDOS v2.0.9)
                                <1>     ; 11/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;           -derived from INT_21H.ASM-
                                <1>     ;            ("loc_INT21h_read_file")
                                <1>         ;   13/03/2011 (05/03/2011)
                                <1>         ;   INT 21h Function AH = 3Fh
                                <1>         ;   Read from a File
                                <1>         ;   INPUT
                                <1>     ;          BX = File Handle
                                <1>         ;      CX = Number of bytes to read
                                <1>         ;           DS:DX= Buffer address
                                <1>         ;
                                <1>     ; Note: TRDOS 386 'sysread' has been derived from
                                <1>     ;       Retro UNIX 386 v1 'sysread', except a few
                                <1>     ;       code modifications.
                                <1>     ;
                                <1>     ; 13/05/2015 (Retro UNIX 386 v1)
                                <1>     ; 11/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 23/05/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysread' is given a buffer to read into and the number of
                                <1>     ; characters to be read. If finds the file from the file
                                <1>     ; descriptor located in *u.r0 (r0). This file descriptor
                                <1>     ; is returned from a successful open call (sysopen).
                                <1>     ; The i-number of file is obtained via 'rw1' and the data
                                <1>     ; is read into core via 'readi'.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysread; buffer; nchars
                                <1>     ; Arguments:
                                <1>     ;       buffer - location of contiguous bytes where
                                <1>     ;                input will be placed.
                                <1>     ;       nchars - number of bytes or characters to be read.
                                <1>     ; Inputs: *u.r0 - file descriptor (& arguments)
                                <1>     ; Outputs: *u.r0 - number of bytes read.
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;        'sysread' system call has three arguments; so,
                                <1>     ;       * 1st argument, file descriptor is in BX register
                                <1>     ;       * 2nd argument, buffer address/offset in CX register
                                <1>     ;       * 3rd argument, number of bytes is in DX register
                                <1>     ;
                                <1>     ;       AX register (will be restored via 'u.r0') will return
                                <1>     ;       to the user with number of bytes read.
                                <1>     ;
                                <1>     ; TRDOS 386 (05/10/2016)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;     EBX = File handle (descriptor/index)
                                <1>     ;         ECX = Buffer address
                                <1>         ;         EDX = Number of bytes
                                <1>     ; OUTPUT ->
                                <1>     ;         EAX = Number of bytes have been read
                                <1>     ;         cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>
                                <1>     ; EBX = File descriptor
0000D1C1 E8DA300000             <1>     call    getf1
0000D1C6 723A                   <1>     jc      short device_read ; read data from device
                                <1>
                                <1>     ; EAX = First cluster of the file
                                <1>
0000D1C8 E837000000             <1>     call    rw1     ; 03/09/2024 (major modification)
0000D1CD 730A                   <1>     jnc     short sysread_0
                                <1>
                                <1> sysrw_err:  ; 03/09/2024
0000D1CF A3[6C8E0100]           <1>     mov     [u.r0], eax ; error code
0000D1D4 E995F8FFFF             <1>     jmp     error
                                <1>
                                <1> sysread_0:
0000D1D9 E8A8340000             <1>     call    readi
```

```
0000D1DE EB13                <1>     jmp    short rw0
                             <1>
                             <1> syswrite: ; < write to file >
                             <1>     ; 27/09/2024
                             <1>     ; 18/09/2024
                             <1>     ; 03/09/2024
                             <1>     ; 25/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; 23/10/2016
                             <1>     ; 11/10/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;            -derived from INT_21H.ASM-
                             <1>     ;             ("loc_INT21h_write_file")
                             <1>         ;   13/03/2011 (05/03/2011)
                             <1>         ;   INT 21h Function AH = 40h
                             <1>         ;   Write to a File
                             <1>         ;   INPUT
                             <1>     ;           BX = File Handle
                             <1>         ;       CX = Number of bytes to write
                             <1>         ;           DS:DX= Buffer address
                             <1>         ;
                             <1>     ; Note: TRDOS 386 'sysrwrite' has been derived from
                             <1>     ;       Retro UNIX 386 v1 'syswrite', except a few
                             <1>     ;       code modifications.
                             <1>     ;
                             <1>
                             <1>     ; 13/05/2015 (Retro UNIX 386 v1)
                             <1>     ; 11/05/2015 (Retro UNIX 386 v1 - Beginning)
                             <1>     ; 23/05/2013 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; 'syswrite' is given a buffer to write onto an output file
                             <1>     ; and the number of characters to write. If finds the file
                             <1>     ; from the file descriptor located in *u.r0 (r0). This file
                             <1>     ; descriptor is returned from a successful open or create call
                             <1>     ; (sysopen or syscreat). The i-number of file is obtained via
                             <1>     ; 'rw1' and buffer is written on the output file via 'write'.
                             <1>     ;
                             <1>     ; Calling sequence:
                             <1>     ;       syswrite; buffer; nchars
                             <1>     ; Arguments:
                             <1>     ;       buffer - location of contiguous bytes to be writtten.
                             <1>     ;       nchars - number of characters to be written.
                             <1>     ; Inputs: *u.r0 - file descriptor (& arguments)
                             <1>     ; Outputs: *u.r0 - number of bytes written.
                             <1>     ; ............................................................
                             <1>     ;
                             <1>     ; Retro UNIX 8086 v1 modification:
                             <1>     ;       'syswrite' system call has three arguments; so,
                             <1>     ;       * 1st argument, file descriptor is in BX register
                             <1>     ;       * 2nd argument, buffer address/offset in CX register
                             <1>     ;       * 3rd argument, number of bytes is in DX register
                             <1>     ;
                             <1>     ;       AX register (will be restored via 'u.r0') will return
                             <1>     ;       to the user with number of bytes written.
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>         ;       EBX = File handle (descriptor/index)
                             <1>     ;           ECX = Buffer address
                             <1>         ;           EDX = Number of bytes
                             <1>     ; OUTPUT ->
                             <1>     ;           EAX = Number of bytes have been written
                             <1>     ;           cf = 1 -> Error code in AL
                             <1>     ;
                             <1>     ; Modified Registers: EAX (at the return of system call)
                             <1>     ;
                             <1>
                             <1>     ; EBX = File descriptor
0000D1E0 E8BB300000          <1>     call   getf1
0000D1E5 721B                <1>     jc     short device_write ; write data to device
                             <1>     ; EAX = First cluster of the file
                             <1>     ; EBX = File number (Open file number) ; 23/10/2016
                             <1>
0000D1E7 E818000000          <1>     call   rw1    ; 03/09/2024 (major modification)
                             <1>     ; 03/09/2024
0000D1EC 72E1                <1>     jc     short sysrw_err
                             <1>
                             <1>     ; 25/08/2024 (*)
                             <1>     ;jnc   short syswrite_0
                             <1>     ;mov   [u.r0], eax ; error code
                             <1>     ;jmp   error
                             <1>
                             <1>     ; 25/08/2024 - TRDOS 386 v2.0.9 (bugfix) (*)
                             <1>     ; (if eax = 0, lets add a cluster at 'mget_w_0')
                             <1>     ; ('mget_w' and 'add_new_cluster' procs are modified)
                             <1>
                             <1> syswrite_0:
0000D1EE E86C3B0000          <1>     call   writei  ; 24/08/2024 ('mget_w' modification)
                             <1> rw0: ; 1:
0000D1F3 A1[988E0100]        <1>         mov eax, [u.nread]
0000D1F8 A3[6C8E0100]        <1>     mov    [u.r0], eax
0000D1FD E98CF8FFFF          <1>     jmp    sysret
                             <1>
                             <1>     ; 17/04/2021 (temporary)
                             <1> device_write:
                             <1> device_read:
                             <1>     ; 26/09/2024
                             <1>     ; 18/09/2024 - TRDOS 386 v2.0.9
                             <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                             <1>     ;       (temporary modifications)
                             <1>     ;
                             <1>     ; 11/10/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; cl = DEV_OPENMODE ; open mode
                             <1>     ; ch = DEV_ACCESS   ; access flags
                             <1>     ; al = DEV_DRIVER   ; device number (eax)
                             <1>
                             <1>     ; 24/04/2025 - TRDOS 386 v2.0.10
                             <1>     ;; 18/09/2024 (temporary)
                             <1>     ;call  rw2 ; file not open ; cf = 1
                             <1>     ;;jmp  error
                             <1>     ;; 26/09/2024
                             <1>     ;jmp   short sysrw_err
                             <1>     ; 24/04/2025
0000D202 EBA9                <1>     jmp    short device_rw_err
                             <1>
                             <1>
                             <1> ;   test   cl, 1 ; 1 = read, 2 = write, 3 = read&write
                             <1> ;   jz     short rw3
                             <1> ;
                             <1> ;   mov    ebx, eax
                             <1> ;   shl    bx, 2 ; *4
                             <1> ;
                             <1> ;   test   ch, 80h ; bit 7, installable device driver flag
                             <1> ;   jz     short d_read_2 ; Kernel device
                             <1> ;   ; installable device
                             <1> ;d_read_1:
                             <1> ;       jmp dword [ebx+IDEV_RADDR-4]
```

```
                                <1> ;d_read_2:
                                <1> ;   jmp     dword [ebx+KDEV_RADDR-4]
                                <1>
                                <1> ;device_write:
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;       (temporary modifications)
                                <1>     ;
                                <1>     ; 11/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; cl = DEV_OPENMODE ; open mode
                                <1>     ; ch = DEV_ACCESS   ; access flags
                                <1>     ; al = DEV_DRIVER   ; device number (eax)
                                <1>
                                <1>     ; 17/04/2021 (temporary)
                                <1>     ;jmp   short rw2 ; file not open
                                <1>
                                <1> ;   test    cl, 2 ; 1 = read, 2 = write, 3 = read&write
                                <1> ;   jz      short rw3
                                <1> ;
                                <1> ;   mov     ebx, eax
                                <1> ;   shl     bx, 2 ; *4
                                <1> ;
                                <1> ;   test    ch, 80h ; bit 7, installable device driver flag
                                <1> ;   jz      short d_write_2 ; Kernel device
                                <1> ;   ; installable device
                                <1> ;d_write_1:
                                <1> ;       jmp dword [ebx+IDEV_WADDR-4]
                                <1> ;d_write_2:
                                <1> ;   jmp     dword [ebx+KDEV_WADDR-4]
                                <1>
                                <1> rw1:
                                <1>     ; 27/09/2024
                                <1>     ; 03/09/2024 (TRDOS 386 v2.0.9)
                                <1>     ; 17/04/2021 (TRDOS 386 v2.0.4)
                                <1>     ; 11/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 14/05/2015 (Retro UNIX 386 v1)
                                <1>     ; 11/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 23/05/2013 - 24/05/2013 (Retro UNIX 8086 v1)
                                <1>     ; System call registers: ebx, ecx, edx (through 'sysenter')
                                <1>     ;
                                <1>     ; EBX = File descriptor
                                <1>     ;call  getf1 ; calling point in 'getf' from 'rw1'
                                <1>     ;jc    short device_rw ; read/write data from/to device
                                <1>     ; EAX = First cluster of the file
                                <1>
                                <1>     ; 03/09/2024
0000D204 09C0                   <1>     or      eax, eax
0000D206 7421                   <1>     jz      short rw7 ; eax = 0 -> empty file (OK for now)
                                <1>
0000D208 83F802                 <1>     cmp     eax, 2   ; is it valid cluster number ?
                                <1>     ;jb    short rw2
                                <1>     ; 03/09/2024
0000D20B 7307                   <1>     jnb     short rw6 ; yes, check upper limit
                                <1>
                                <1>     ;;;
                                <1>     ; eax = 1 -> invalid cluster number
                                <1> rw5:
0000D20D B823000000             <1>     mov     eax, ERR_CLUSTER ; 35 ; 'cluster not available !'
0000D212 EB2C                   <1>     jmp     short rw4 ; cf = 1
                                <1>
                                <1> rw6:; 03/09/2024 (check cluster number is valid or not)
0000D214 53                     <1>     push    ebx
                                <1>     ; ebx <= OPENFILES-1 ; 0-31
0000D215 8ABB[F8830100]         <1>     mov     bh, [ebx+OF_DRIVE] ; drive number * 256
0000D21B 28DB                   <1>     sub     bl, bl ; 27/09/2024
0000D21D 8B9B78010900           <1>     mov     ebx, [ebx+Logical_DOSDisks+LD_Clusters]
0000D223 43                     <1>     inc     ebx ; cluster count + 1 = last cluster number
0000D224 39C3                   <1>     cmp     ebx, eax
0000D226 5B                     <1>     pop     ebx
0000D227 72E4                   <1>     jb      short rw5
                                <1> rw7:
                                <1>     ;;;
                                <1>
0000D229 890D[908E0100]         <1>     mov     [u.base], ecx  ; buffer address/offset
                                <1>                            ;(in the user's virtual memory space)
0000D22F 8915[948E0100]         <1>     mov     [u.count], edx
0000D235 C705[D88E0100]0000-    <1>         mov dword [u.error], 0 ; reset the last error code
0000D23D 0000                   <1>
0000D23F C3                     <1>     retn
                                <1>
                                <1> ; 24/04/2025 - TRDOS 386 v2.0.10
                                <1> %if 0
                                <1>
                                <1> rw2:
                                <1>     mov     eax, ERR_FILE_NOT_OPEN ; file not open !
                                <1>     ;mov   dword [u.error], eax
                                <1>     ;retn
                                <1>
                                <1> %endif
                                <1>     ; 03/09/2024
                                <1>     ; 17/04/2021
                                <1>     ;jmp   short rw4
                                <1>
                                <1>     ; 03/09/2024
                                <1> rw3:
                                <1> ;   mov     eax, ERR_FILE_ACCESS ; permission denied !
                                <1> ;   stc
                                <1>
                                <1> rw4:; 17/04/2021
0000D240 A3[D88E0100]           <1>     mov     dword [u.error], eax
0000D245 C3                     <1>     retn
                                <1>
                                <1> systimer:
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 02/01/2017
                                <1>     ; 21/12/2016
                                <1>     ; 19/12/2016
                                <1>     ; 10/12/2016 (callback)
                                <1>     ; 10/06/2016
                                <1>     ; 07/06/2016
                                <1>     ; 06/06/2016
                                <1>     ; 21/05/2016
                                <1>     ; 19/05/2016
                                <1>     ; 18/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; (TRDOS 386 feature only!)
                                <1>     ;
                                <1>     ; (start or stop timer event(s))
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       BL = Signal return byte (response byte)
                                <1>     ;            (Any requested value between 0 and 255)
                                <1>     ;            (Kernel will put it at the requested address)
                                <1>     ;       BH = Time count unit
                                <1>     ;            0 = Stop timer event
                                <1>     ;            1 = 18.2 ticks per second
```

```
                                <1>     ;           2 = 10 milliseconds
                                <1>     ;           3 = 1 second (for real time clock interrupt)
                                <1>     ;           4 = time/tick count in current time count unit
                                <1>     ;          // 10/12/2016
                                <1>     ;          80h = Stop timer event (callback method)
                                <1>     ;          81h = 18.2 ticks per second, callback method
                                <1>     ;          82h = 10 milliseconds, callback method
                                <1>     ;          83h = 1 second (for RTC int), callback method
                                <1>     ;          84h = current time count unit, callback method
                                <1>     ;
                                <1>     ;          Note: Only 03h or 83h will set real time clock
                                <1>     ;                (RTC) events (Others are for PIT events)!
                                <1>     ;
                                <1>     ;      NOTE: If callback (user service) method is used,
                                <1>     ;          EDX will point to the return address (of service
                                <1>     ;          procedure) in user's space instead of signal
                                <1>     ;          response byte address. (TRDOS 386 kernel will
                                <1>     ;          direct the cpu to that address -in user's space-
                                <1>     ;          at the return of system call or interrupt
                                <1>     ;          just after the adjusted count/time is elapsed.)
                                <1>     ;          User's sevice routine must be ended with a
                                <1>     ;          'iret'. Normal return addresses from system
                                <1>     ;          calls or and interrupts will be kept same except
                                <1>     ;          the timer returns.
                                <1>             ;
                                <1>     ;      BH = 0 -> Stop timer event
                                <1>     ;      BL = Timer event number (1 to 255) if BH = 0
                                <1>     ;           If BL = 0, all timer events (which are belongs
                                <1>     ;            to running process) will be stopped
                                <1>     ;      ECX = Time/Tick count (depending on time count unit)
                                <1>     ;      EDX = Signal return (Response) byte address
                                <1>     ;            (virtual address in user's memory space)
                                <1>     ; OUTPUT ->
                                <1>     ;      AL = Timer event number      (1 to 255) (max. value = 16)
                                <1>     ;      IF BH Input = 0 & CF = 0 & AL = 0 ->
                                <1>     ;           timer event(s) has/have been stopped/finished
                                <1>     ;      CF = 1 & AL = 0 -> no timer setting space to set
                                <1>     ;      CF = 1 & AL > 0 -> timer count unit is not usable
                                <1>     ;
                                <1>     ;      NOTE: To modify a time count for a user function,
                                <1>     ;            at first, current timer event must be stopped
                                <1>     ;            then a new timer event (which is related with
                                <1>     ;            same user function) must be started.
                                <1>     ;
                                <1>     ;            Signal return (response) byte may be used for
                                <1>     ;            several purposes. Kernel will put this value
                                <1>     ;            to requested address during timer interrupt,
                                <1>     ;            program/user can check this value to understand
                                <1>     ;            which event has been occurred and what is changed.
                                <1>     ;            (Multi timer events can share same signal address)
                                <1>     ;
                                <1>     ;      NOTE: If the process is running while the time count
                                <1>     ;            is reached, kernel will put signal return (response)
                                <1>     ;            byte value at requested address during timer
                                <1>     ;            interrupt and the process will continue to run.
                                <1>     ;            Program/process must call (jump to) it's timer event
                                <1>     ;            function as required, for checking the timer event
                                <1>     ;            status via signal return (response) byte address.
                                <1>     ;
                                <1>     ;            If the process is not running (waiting or sleeping
                                <1>     ;            or released) while the time count is reached,
                                <1>     ;            it is restarted from where it left, to ensure
                                <1>     ;            proper multi media (video, audio, clock, timer)
                                <1>     ;            functionality.
                                <1>     ;
                                <1>     ;            (It is better to use 'syswait' or 'syssleep',
                                <1>     ;            or 'sysrele' system call just after the timer
                                <1>     ;            function. Otherwise, timer events may block other
                                <1>     ;            processes which are not using timer events.)
                                <1>     ;
                                <1>     ; Timer Event Structure: (max. 16 timer events, 16*16 bytes)
                                <1>     ;       Owner:         resb 1 ; 0 = free
                                <1>     ;                              ;>0 = process number (u.uno)
                                <1>     ;      Calback:       resb 1 ; 1 = callback, 0 = response byte
                                <1>     ;      Interrupt:      resb 1 ; 0 = Timer interrupt (or none)
                                <1>     ;                              ; 1 = Real Time Clock interrupt
                                <1>     ;      Response:       resb 1 ; 0 to 255, signal return value
                                <1>     ;      Count Limit:   resd 1 ; count of ticks (total/set)
                                <1>     ;      Current Count:        resd 1 ; count of ticks (current)
                                <1>     ;      Response Addr:  resd 1 ; response byte (pointer) address
                                <1>     ;
                                <1>
                                <1>     ; 19/12/2016 (timer callback)
0000D246 C605[50880100]00       <1>     mov     byte [tcallback], 0
0000D24D C605[51880100]00       <1>     mov     byte [trtc], 0
0000D254 C705[E08E0100]0000-    <1>     mov     dword [u.tcb], 0 ; this is not necessary...
0000D25C 0000                   <1>
                                <1>
0000D25E 80FF80                 <1>     cmp     bh, 80h
0000D261 7224                   <1>     jb      short systimer_cb2
0000D263 7704                   <1>     ja      short systimer_cb0
                                <1>
0000D265 31D2                   <1>     xor     edx, edx ; 0, reset callback address
0000D267 EB0B                   <1>     jmp     short systimer_cb1
                                <1>
                                <1> systimer_cb0:
0000D269 80FF84                 <1>     cmp     bh, 84h
0000D26C 7764                   <1>     ja      short systimer_5 ;  undefined, error
                                <1>
                                <1>     ;mov    byte [tcallback], 1 ; 19/12/2016
0000D26E FE05[50880100]         <1>     inc     byte [tcallback]
                                <1>
                                <1> systimer_cb1:
0000D274 0FB635[BD8E0100]       <1>     movzx   esi, byte [u.uno] ; process number
                                <1>     ;shl    si, 2
                                <1>     ; 23/07/2022
0000D27B C1E602                 <1>     shl     esi, 2
0000D27E 8996[FC8D0100]         <1>     mov     [esi+p.tcb-4], edx ; set process timer callback address
                                <1>                                ; (overwrite prev value if it is set!)
0000D284 80E77F                 <1>     and     bh, 7Fh
                                <1>
                                <1> systimer_cb2:
0000D287 80FF02                 <1>     cmp     bh, 2
0000D28A 7446                   <1>         je      short systimer_5  ; only 18.2 ticks per second is usable
                                <1>                                ; 10 milliseconds (100 Hertz) timer
                                <1>                                ; will be set later (18/05/2016)
0000D28C 774C                   <1>         ja      short systimer_6
                                <1>
0000D28E 20FF                   <1>     and     bh, bh
                                <1>     ;jz     systimer_9        ; stop timer event(s)
                                <1>     ; 23/07/2022
0000D290 7505                   <1>     jnz     short systimer_19
0000D292 E9BA000000             <1>     jmp     systimer_9
                                <1>
```

```
                                <1>     ; bh = 1 (timer interrupt, 18.2 Hz, IBM PC/AT ROMBIOS default)
                                <1>
                                <1> systimer_19:
0000D297 B00A                   <1>     mov     al, 10 ; (*)
                                <1>
                                <1> systimer_0:
0000D299 B710                   <1>     mov     bh, 16
                                <1>     ;
0000D29B 383D[47830100]         <1>     cmp     [timer_events], bh ; 16 ; 07/06/2016
0000D2A1 7319                   <1>     jnb     short systimer_3  ; max. 16 timer events
                                <1>     ;
0000D2A3 50                     <1>     push    eax ; (*)
                                <1>
0000D2A4 BF[808F0100]           <1>     mov     edi, timer_set  ; beginning address of timer events
                                <1>                             ; setting space
0000D2A9 30C0                   <1>     xor     al, al ; 0
                                <1> systimer_1:
0000D2AB FEC0                   <1>     inc     al
0000D2AD 803F00                 <1>     cmp     byte [edi], 0  ; is it free space ?
0000D2B0 7639                   <1>     jna     short systimer_7 ; yes
0000D2B2 FECF                   <1>     dec     bh
0000D2B4 7405                   <1>     jz      short systimer_2
0000D2B6 83C710                 <1>     add     edi, 16
0000D2B9 EBF0                   <1>     jmp     short  systimer_1 ; next event space
                                <1>
                                <1> systimer_2:
0000D2BB 58                     <1>     pop     eax ; (*) discard
                                <1> systimer_3:
0000D2BC C605[6C8E0100]00       <1>     mov     byte [u.r0], 0
                                <1> systimer_4:
0000D2C3 C705[D88E0100]1B00-    <1>         mov     dword [u.error], ERR_MISC
0000D2CB 0000                   <1>
                                <1>                                 ; one of miscellaneous/other errors
0000D2CD E99CF7FFFF             <1>     jmp     error ; cf -> 1
                                <1>
                                <1> systimer_5:
0000D2D2 883D[6C8E0100]         <1>     mov     [u.r0], bh ; Time count unit (=2 or >3)
0000D2D8 EBE9                   <1>     jmp     short systimer_4 ; 07/06/2016
                                <1>
                                <1> systimer_6:
0000D2DA 80FF04                 <1>     cmp     bh, 4
0000D2DD 77F3                   <1>         ja      short systimer_5  ; undefined time count unit
                                <1>     ;jb     short systimer_16
                                <1>
                                <1>     ;mov    al, 1   ; default (use current timer unit)
                                <1>                     ; countdown value is in ECX !
                                <1>                     ; max. value of ecx = 4294967296/10
                                <1>         ;jmp    short systimer_0
                                <1>     ;jmp    short systimer_19
0000D2DF 74B6                   <1>     je      short systimer_19
                                <1>
                                <1> systimer_16:
                                <1>     ; bh = 3
                                <1>     ; timer event via real time clock interrupt
                                <1>     ; interrupt/update frequency: 1 Hz (1 tick per second)
                                <1>
0000D2E1 B0B6                   <1>     mov     al, 182 ; (*) ; 18.2 * 10
0000D2E3 FE05[51880100]         <1>     inc     byte [trtc] ; timer event via real time clock
0000D2E9 EBAE                   <1>         jmp     short systimer_0
                                <1>
                                <1> systimer_7:
0000D2EB A2[6C8E0100]           <1>     mov     [u.r0], al ; timer event number
                                <1>     ;
                                <1>     ; edi = address of empty timer event area
0000D2F0 A0[BD8E0100]           <1>     mov     al, [u.uno]
0000D2F5 FA                     <1>     cli     ; disable interrupts
0000D2F6 AA                     <1>     stosb   ; process number
0000D2F7 A0[50880100]           <1>     mov     al, [tcallback] ; timer callback flag
0000D2FC AA                     <1>     stosb   ; 1= callback method, 0= signal response byte method
0000D2FD A0[51880100]           <1>     mov     al, [trtc] ; timer interrupt type
0000D302 AA                     <1>     stosb   ; 1= real time clock, 0= programmable interval timer
0000D303 88D8                   <1>     mov     al, bl ; Signal return (Response) value
0000D305 AA                     <1>     stosb    ; response byte
0000D306 58                     <1>     pop     eax ; (*) ; 10 or 182
0000D307 89D3                   <1>     mov     ebx, edx ; virtual address for response/signal byte
0000D309 F7E1                   <1>     mul     ecx
                                <1>     ; (eax = 10 * count of 18.2 Hz timer ticks)
                                <1>     ; (count down step = 10)
0000D30B AB                     <1>     stosd  ; count limit (reset value)
0000D30C AB                     <1>     stosd  ; current count value
                                <1>
                                <1>     ; 19/12/2016
0000D30D 803D[50880100]00       <1>     cmp     byte [tcallback], 0 ; timer callback method ?
0000D314 7604                   <1>     jna     short systimer_17 ; no
0000D316 89D8                   <1>     mov     eax, ebx ; virtual address for callback routine
0000D318 EB0D                   <1>     jmp     short systimer_18
                                <1>
                                <1> systimer_17: ; signal response byte method
                                <1>     ; ebx = virtual address
                                <1>     ; [u.pgdir] = page directory's physical address
                                <1>     ; 20/02/2017
0000D31A FE05[52880100]         <1>     inc      byte [no_page_swap] ; 1
                                <1>                     ; Do not add this page to swap queue
                                <1>                     ; and remove it from swap queue if it is
                                <1>                     ; on the queue.
0000D320 E8EB88FFFF             <1>     call    get_physical_addr
0000D325 721A                   <1>     jc      short systimer_8 ; 07/06/2016
                                <1>     ; eax = physical address of the virtual address in user's space
                                <1> systimer_18:
0000D327 AB                     <1>     stosd  ; response addr (physical) or callback addr (virtual)
0000D328 FE05[47830100]         <1>     inc     byte [timer_events] ; 07/06/201
                                <1>     ; 02/01/2017
0000D32E 0FB605[BD8E0100]       <1>     movzx   eax, byte [u.uno]
0000D335 FE80[EF8D0100]         <1>     inc     byte [eax+p.timer-1]
                                <1>     ;
0000D33B FB                     <1>     sti     ; enable interrupts
0000D33C E94DF7FFFF             <1>     jmp     sysret
                                <1>
                                <1> systimer_8:
                                <1>     ; 10/06/2016
                                <1>     ; 07/06/2016
0000D341 28C0                   <1>     sub     al, al ; 0
0000D343 8847F4                 <1>     mov     [edi-12], al ; clear process number (free timer event)
                                <1>     ;mov    dword [edi], eax ; 0
0000D346 FB                     <1>     sti
0000D347 A2[6C8E0100]           <1>     mov     [u.r0], al ; 0
0000D34C E91DF7FFFF             <1>     jmp     error
                                <1>
                                <1> systimer_9:
                                <1>     ; 10/06/2016
                                <1>     ; 07/06/2016
0000D351 28C0                   <1>     sub     al, al
0000D353 A2[6C8E0100]           <1>     mov     byte [u.r0], al ; 0
0000D358 3805[47830100]         <1>     cmp      byte [timer_events], al ;  0
```

```
0000D35E 7631                <1>     jna     short systimer_12
                             <1>
                             <1>     ; Note: ecx and edx are undefined here
                             <1>     ;       (for stop timer function)
                             <1>
0000D360 BE[808F0100]        <1>     mov     esi, timer_set  ; beginning address of timer events
                             <1>                             ; setting space
0000D365 A0[BD8E0100]        <1>     mov     al, [u.uno]
                             <1>
0000D36A B710                <1>     mov     bh, 16
                             <1>
0000D36C 08DB                <1>     or      bl, bl
0000D36E 7544                <1>     jnz     short systimer_15
                             <1>
                             <1>     ; clear timer event areas belong to current process
                             <1>     ; (for stopping all timer events belong to current process)
0000D370 FA                  <1>     cli     ; disable interrupts
                             <1> systimer_10:
                             <1>     ; 10/06/2016
                             <1>     ; 07/06/2016
0000D371 8A26                <1>     mov     ah, [esi]
0000D373 08E4                <1>     or      ah, ah ; 0 ?
0000D375 7411                <1>     jz      short systimer_11
0000D377 38C4                <1>     cmp     ah, al ; is the process number (owner) same ?
0000D379 750D                <1>         jne     short systimer_11 ; no
                             <1>
                             <1>     ;mov    byte [esi], 0
0000D37B 66C7060000          <1>     mov     word [esi], 0 ; clear
                             <1>     ;mov    dword [esi+12], 0 ; clear
                             <1>
0000D380 FE0D[47830100]      <1>     dec     byte [timer_events]
0000D386 7409                <1>     jz      short systimer_12
                             <1>
                             <1> systimer_11:
0000D388 FECF                <1>     dec     bh
0000D38A 7405                <1>     jz      short systimer_12
0000D38C 83C610              <1>     add     esi, 16
0000D38F EBE0                <1>     jmp     short systimer_10
                             <1>
                             <1> systimer_12:
0000D391 0FB635[BD8E0100]    <1>     movzx   esi, byte [u.uno]
0000D398 08DB                <1>     or      bl, bl ; all timer events or one timer event ?
0000D39A 740C                <1>     jz      short systimer_13
0000D39C 8A9E[EF8D0100]      <1>     mov     bl, [esi+p.timer-1]
0000D3A2 20DB                <1>     and     bl, bl  ; previous number of timer events for the process
0000D3A4 7408                <1>     jz      short systimer_14
0000D3A6 FECB                <1>     dec     bl  ; previous number of timer events for the process - 1
                             <1> systimer_13:
0000D3A8 889E[EF8D0100]      <1>     mov     [esi+p.timer-1], bl ; 0 ; no timer events for process
                             <1> systimer_14:
0000D3AE FB                  <1>     sti     ; enable interrupts
0000D3AF E9DAF6FFFF          <1>     jmp     sysret
                             <1>
                             <1> systimer_15:
0000D3B4 38FB                <1>     cmp     bl, bh ; 16
                             <1>         ;ja systimer_4      ; max. 16 timer events !
                             <1>     ; 23/07/2022
0000D3B6 7605                <1>     jna     short systimer_21
                             <1> systimer_20:
0000D3B8 E906FFFFFF          <1>     jmp     systimer_4
                             <1> systimer_21:        ; 23/07/2022
0000D3BD 88DA                <1>     mov     dl, bl
0000D3BF FECA                <1>     dec     dl  ; 16 -> 15 ... 1 -> 0
0000D3C1 C0E204              <1>     shl     dl, 4 ; * 16
0000D3C4 0FB6FA              <1>     movzx   edi, dl
0000D3C7 01F7                <1>     add     edi, esi ; timer_set
                             <1>
0000D3C9 3A07                <1>     cmp     al, [edi] ; process number
                             <1>         ;jne        systimer_4
                             <1>     ; 23/07/2022
0000D3CB 75EB                <1>     jne     short systimer_20 ; jmp systimer_4
                             <1>
                             <1>     ; same process ID
0000D3CD FA                  <1>     cli     ; disable interrupts
                             <1>     ; 10/06/2016 ; 02/01/2017
                             <1>     ;mov    byte [edi], 0
0000D3CE 66C7070000          <1>     mov     word [edi], 0 ; clear
                             <1>     ;mov    dword [edi+12], 0 ; clear
0000D3D3 FE0D[47830100]      <1>     dec     byte [timer_events]
0000D3D9 EBB6                <1>     jmp     short systimer_12
                             <1>
                             <1> sysvideo: ; VIDEO DATA TRANSFER FUNCTIONS
                             <1>     ; 11/08/2022
                             <1>     ; 08/08/2022
                             <1>     ; 23/07/2022 (TRDOS 386 v2.0.5)
                             <1>     ; 06/03/2021
                             <1>     ; 02/03/2021
                             <1>     ; 28/02/2021
                             <1>     ; 27/02/2021
                             <1>     ; 26/02/2021
                             <1>     ; 25/02/2021
                             <1>     ; 21/02/2021, 22/02/2021, 23/02/2021
                             <1>     ; 15/02/2021, 16/02/2021, 18/02/2021
                             <1>     ; 10/02/2021, 11/02/2021, 12/02/2021
                             <1>     ; 07/02/2021, 08/02/2021
                             <1>     ; 01/02/2021, 02/02/2021, 05/02/2021
                             <1>     ; 29/01/2021, 30/01/2021, 31/01/2021
                             <1>     ; 23/01/2021, 24/01/2021, 28/01/2021
                             <1>     ; 18/01/2021, 19/01/2021, 22/01/2021
                             <1>     ; 04/01/2021, 10/01/2021, 11/01/2021
                             <1>     ; 01/01/2021, 02/01/2021, 03/01/2021
                             <1>     ; 28/12/2020, 29/12/2020, 30/12/2020
                             <1>     ; 25/12/2020, 26/12/2020
                             <1>     ; 21/12/2020, 23/12/2020
                             <1>     ; 12/12/2020, 14/12/2020
                             <1>     ; 10/12/2020, 11/12/2020
                             <1>     ; 03/12/2020, 04/12/2020
                             <1>     ; 22/11/2020, 23/11/2020
                             <1>     ; 21/11/2020 (TRDOS 386 v2.0.3)
                             <1>     ; 12/05/2017
                             <1>     ; 11/07/2016
                             <1>     ; 13/06/2016
                             <1>     ; 16/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ;
                             <1>     ; VIDEO DATA TRANSFER FUNCTIONS:
                             <1>     ;
                             <1>     ; Inputs:
                             <1>     ;               ; 07/02/2021
                             <1>     ;       BH = 0 = VIDEO BIOS Mode 3, tty/text mode data transfers
                             <1>     ;            BL =
                             <1>     ;              Bits 0&1, Transfer direction
                             <1>     ;                      0 - System to system
                             <1>     ;                      1 - User to system
                             <1>     ;                      2 - System to user
```

```
<1>     ;                   3 - Exhange (Swap) - 28/01/2021
<1>     ;             Bits 2, Transfer Type
<1>     ;                   0 - Display page (complete) transfer
<1>     ;                   1 - Display page window (col,row) transfer
<1>     ;             ; 28/01/2021
<1>     ;             Bits 3..7 - Reserved, undefined (must be 0)
<1>     ;              ; 28/01/2021
<1>     ;           /// BL = 0 -> System to system (display page) transfer
<1>     ;              CL = Source page (0FFh = current video page)
<1>     ;              DL = Destination page (0FFh = current video page)
<1>     ;              (Note: Nothing to do if src & dest are same page)
<1>     ;           /// BL = 1&2 -> user to system & system to user transfer
<1>     ;              ECX = User's buffer address
<1>     ;              DL = Video page (0FFh = current video page)
<1>     ;           /// BL = 3 -> exchange (swap) display page ; 28/01/2021
<1>     ;              ECX = User's buffer address
<1>     ;              DL = Video page (0FFh = current video page)
<1>     ;              EDI = Swap address in user's memory (must be > 0)
<1>     ;           /// BL = 5&6&7 -> user to system, system to user transfer
<1>     ;             (system window is in current/active display page)
<1>     ;              ESI = User's buffer address
<1>     ;              ECX Low 16 bits = Top left column (X1 position)
<1>     ;              ECX High 16 bits = Top row (Y1 position)
<1>     ;              EDX Low 16 bits = Bottom right column (X2 position)
<1>     ;              EDX High 16 bits = Bottom row (Y2 position)
<1>         ;                If BL = 5 or BL bit 0 & bit 1 are 1 ; 28/01/2021
<1>     ;              EDI = Swap address (in user's memory space)
<1>     ;              (If swap address > 0, previous content of the window
<1>     ;              will be saved into swap area in user's memory space)
<1>     ;           /// BL = 4 -> system to system transfer
<1>     ;              ESI = System's source buffer (video page) address
<1>     ;              ECX Low 16 bits = Top left column (X1 position)
<1>     ;              ECX High 16 bits = Top row (Y1 position)
<1>     ;              EDX Low 16 bits = Bottom right column (X2 position)
<1>     ;              EDX High 16 bits = Bottom row (Y2 position)
<1>     ;              EDI = System's destination buffer (video page) addr
<1>     ;
<1>     ;             ; 06/02/2021
<1>     ;             ; 05/02/2021
<1>     ;             ; 01/02/2021, 02/02/2021
<1>     ;             ; 30/01/2021, 31/02/2021
<1>     ;             ; 29/01/2021 (major modification)
<1>     ;             ; 23/11/2020 (major modification)
<1>     ;             ; 22/11/2020 (bugfixes and extensions)
<1>     ;      BH = 1 = VGA Graphics (0A0000h) data transfers
<1>     ;             BL bit 7
<1>     ;                resolution (screen width) option
<1>     ;                0 = 320 pixels
<1>     ;                1 = 640 pixels
<1>     ;             .. followings are same with SVGA transfer function
<1>     ;             BL bit 6
<1>     ;                direction option
<1>     ;                0 = user to system (video memory)
<1>     ;                1 = system to user
<1>     ;             BL bit 5
<1>     ;                masked/direct (non-masked) operations
<1>     ;                1 = masked, 0 = non-masked (direct)
<1>     ;             BL bit 4
<1>     ;                page/window option
<1>     ;                1 = window, 0 = display page (screen)
<1>     ;             BL bit 0 to 3 (pixel operation types)
<1>     ;                0 = Copy pixels (colors) ((mask color))
<1>     ;                1 = Change (New, Fill) color
<1>     ;                2 = Add color
<1>     ;                3 = Sub color
<1>     ;                4 = OR color
<1>     ;                5 = AND color
<1>     ;                6 = XOR color
<1>     ;                7 = NOT color
<1>     ;                8 = NEG color
<1>     ;                9 = INC color
<1>     ;               10 = DEC color
<1>     ;               11 = Mix (Average) colors
<1>     ;               12 = Replace pixel colors
<1>     ;               13 = Copy pixel block(s)
<1>     ;               14 = Write line(s)
<1>     ;               15 = Write character (font)
<1>     ;
<1>     ;          Input Registers for pixel operations:
<1>     ;               Same with LFB data transfer function below
<1>     ;
<1>     ;             ; 25/02/2021
<1>     ;             ; 05/02/2021, 06/02/2021
<1>     ;             ; 01/02/2021, 02/02/2021
<1>     ;             ; 30/01/2021, 31/02/2021
<1>     ;             ; 29/01/2021 (major modification)
<1>     ;             ; 23/11/2020 (major modification)
<1>     ;             ; 22/11/2020 (bugfixes and extensions)
<1>     ;      BH = 2 = Super VGA, LINEAR FRAME BUFFER data transfers
<1>     ;             BL bit 7
<1>     ;                unused (invalid), must be 0
<1>     ;             BL bit 6
<1>     ;                direction option
<1>     ;                0 = user to system (video memory)
<1>     ;                1 = system to user
<1>     ;             BL bit 5
<1>     ;                masked/direct (non-masked) operations
<1>     ;                1 = masked, 0 = non-masked (direct)
<1>     ;             BL bit 4
<1>     ;                page/window option
<1>     ;                1 = window, 0 = display page (screen)
<1>     ;             BL bit 0 to 3 (pixel operation types)
<1>     ;                0 = Copy pixels (colors) ((mask color))
<1>     ;                1 = Change (New, Fill) color
<1>     ;                2 = Add color
<1>     ;                3 = Sub color
<1>     ;                4 = OR color
<1>     ;                5 = AND color
<1>     ;                6 = XOR color
<1>     ;                7 = NOT color
<1>     ;                8 = NEG color
<1>     ;                9 = INC color
<1>     ;               10 = DEC color
<1>     ;               11 = Mix (Average) colors
<1>     ;               12 = Replace pixel colors
<1>     ;               13 = Copy pixel block(s)
<1>     ;               14 = Write line(s)
<1>     ;               15 = Write character (font)
<1>     ;
<1>     ;          Note: If HW of EBX > 0, it is VESA VBE mode number
<1>     ;               otherwise, function will be applied
<1>     ;               to current (VESA VBE) video mode.
<1>     ;
<1>     ;          Input Registers for pixel operations:
```

```
<1>     ;               -- user to system & system to system --
<1>     ;               -- (BL = 0 to 0Fh) -- non-masked, screen --
<1>     ;               -- (BL = 10h to 1Fh) -- non-masked, window --
<1>     ;               -- (BL = 20h to 2Fh) -- masked, screen --
<1>     ;               -- (BL = 30h to 3Fh) -- masked, window --
<1>     ;               (*) window, (**) masked (***) sys to sys
<1>     ;               for BL bit 0 to 3
<1>     ;               00h: COPY PIXELS
<1>     ;                 If BL bit 4 = 0 ; 21/02/2021
<1>     ;                    full screen copy
<1>     ;                    ECX & EDX will not be used
<1>     ;                   (user buffer must fit to display page)
<1>     ;                 If BL bit 4 = 1 ; 21/02/2021
<1>     ;                   ECX = start position (row, column) (*)
<1>     ;                         (HW = row, CX = column)
<1>     ;                   EDX = size (rows, colums) (*)
<1>     ;                         (HW = rows, DX = columns)
<1>     ;                          (0 -> invalid)
<1>     ;                          (1 -> horizontal or vertical line)
<1>     ;                   ESI = user's buffer address
<1>     ;                   EDI = mask color (**) ; 25/02/2021
<1>     ;                         (this color will be excluded)
<1>     ;               01h: CHANGE PIXEL COLORS
<1>     ;               02h: ADD PIXEL COLORS
<1>     ;               03h: SUB PIXEL COLORS
<1>     ;               04h: OR PIXEL COLORS
<1>     ;               05h: AND PIXEL COLORS
<1>     ;               06h: XOR PIXEL COLORS
<1>     ;               0Bh: MIX PIXEL COLORS
<1>     ;                    CL = color (8 bit, 256 colors)
<1>     ;                   ECX = color (16 bit and true colors)
<1>     ;                   EDX = start position (row, column) (*)
<1>     ;                        (HW = row, DX = column)
<1>     ;                   ESI = size (rows, colums) (*)
<1>     ;                         (HW = rows, SI = columns)
<1>     ;                   EDI = mask color (**) ; 25/02/2021
<1>     ;                         (this color will be excluded)
<1>     ;               07h: NOT PIXEL COLORS
<1>     ;               08h: NEG PIXEL COLORS
<1>     ;               09h: INC PIXEL COLORS
<1>     ;               0Ah: DEC PIXEL COLORS
<1>     ;                   ECX = start position (row, column) (*)
<1>     ;                         (HW = row, CX = column)
<1>     ;                   EDX = size (rows, colums) (*)
<1>     ;                         (HW = rows, DX = columns)
<1>     ;                          (0 -> invalid)
<1>     ;                          (1 -> horizontal or vertical line)
<1>     ;                   EDI = mask color (**) ; 25/02/2021
<1>     ;                         (this color will be excluded)
<1>     ;               0Ch: REPLACE PIXEL COLORS
<1>     ;                    CL = current color (8 bit, 256 colors)
<1>     ;                   ECX = current color (16 bit and true colors)
<1>     ;                    DL = new color (8 bit, 256 colors)
<1>     ;                   EDX = new color (16 bit and true colors)
<1>     ;                   ESI = start position (row, column) (*)
<1>     ;                         (HW = row, SI = column)
<1>     ;                   EDI = size (rows, colums) (*)
<1>     ;                         (HW = rows, DI = columns)
<1>     ;               0Dh: COPY PIXEL BLOCK(S) -full screen-
<1>     ;                  -If BL bit 5 is 0-
<1>     ;                   ECX = start position (row, column) (*)
<1>     ;                         (HW = row, CX = column)
<1>     ;                   EDX = size (rows, colums) (*)
<1>     ;                         (HW = rows, DX = columns)
<1>     ;                          (0 -> invalid)
<1>     ;                          (1 -> horizontal or vertical line)
<1>     ;                   ESI = destination (row, column) (***)
<1>     ;                  -If BL bit 5 is 1-
<1>     ;                       CL = color (8 bit, 256 colors)
<1>     ;                     ECX = color (16 bit and true colors)
<1>     ;                     EDX = count of blocks (not bytes)
<1>     ;                           (limit: 2048 blocks)
<1>     ;                     ESI = user's buffer address
<1>     ;                         contains 64 bits block data
<1>     ;                         BLOCK ADDRESS - (row, col), dword
<1>     ;                         (first 32 bits)
<1>     ;                         BLOCK SIZE - (rows, cols), dword
<1>     ;                         (second 32 bits)
<1>     ;               ; 10/02/2021
<1>     ;               0Eh: WRITE LINE(s) -full screen-
<1>     ;                  -If BL bit 5 is 0-
<1>     ;                   CL = color (8 bit, 256 colors)
<1>     ;                  ECX = color (16 bit and true colors)
<1>     ;                   DX = low 12 bits - size (length)
<1>     ;                        high 4 bits - direction or type
<1>     ;                            0 - Horizontal line
<1>     ;                            1 - Vertical line
<1>     ;                            > 1 - undefined, invalid
<1>     ;                  ESI = start position (row, column)
<1>     ;                        (HW = row, SI = column)
<1>     ;                  -If BL bit 5 is 1-
<1>     ;                    CL = color (8 bit, 256 colors)
<1>     ;                   ECX = color (16 bit and true colors)
<1>     ;                    DX = number of lines (in user buffer)
<1>     ;                          (limit: 2048 lines)
<1>     ;                   ESI = user's buffer
<1>     ;                         contains 64 bit data for lines
<1>     ;                          START POINT: 32 bit (row, col)
<1>     ;                          LENGTH: 32 bit
<1>     ;                          high 16 bits - 0
<1>     ;                          bit 0-11 - length
<1>     ;                          bit 12-15 - type (length)
<1>     ;               0Fh: WRITE CHARACTER (FONT)
<1>     ;                    CL = char's color (8 bit, 256 colors)
<1>     ;                   ECX = char's color (16 bit and true colors)
<1>     ;                    DL = Character's ASCII code
<1>     ;                    DH bit 0 -> font height
<1>     ;                         0 -> 8x16 character font
<1>     ;                         1 -> 8x8 character font
<1>     ;                    DH bit 1 & 2 -> scale
<1>     ;                         0 = 1/1 (8 pixels per char row)
<1>     ;                         1 = 2/1 (16 pixels per char row)
<1>     ;                         2 = 3/1 (24 pixels per char row)
<1>     ;                         3 = 4/1 (32 pixels per char row)
<1>     ;                    DH bit 6 -> [ufont] option (1 = use [ufont])
<1>     ;                    If DH bit 7 = 1
<1>     ;                       USER FONT (from user buffer)
<1>     ;                          DL = 0 -> 8x8 (width: 1 byte per row)
<1>     ;                          DL = 1 -> 8x16
<1>     ;                          DL = 2 -> 16x16 (width: 2 bytes)
<1>     ;                          DL = 3 -> 16x32
<1>     ;                          DL = 4 -> 24x24 (width: 3 bytes)
<1>     ;                          DL = 5 -> 24x48
<1>     ;                          DL = 6 -> 32x32 (width: 4 bytes)
```

```
<1>     ;                       DL = 7 -> 32x64
<1>     ;                       DL > 7 -> invalid (unused)
<1>     ;                    EDI = user's font buffer address
<1>     ;                       (NOTE: byte order is as row0,row1,row2..)
<1>     ;                ESI = start position (row, column) (*)
<1>     ;                        (HW = row, SI = column)
<1>     ;
<1>     ;            -- system to user --
<1>     ;               BL (bit 0 to 7)
<1>     ;            40h: COPY PIXELS (full screen, display page)
<1>     ;                ESI = user's buffer address
<1>     ;            41h: COPY PIXELS (window)
<1>     ;                ECX = start position (row, column) (*)
<1>     ;                      (HW = row, CX = column)
<1>     ;                EDX = size (rows, colums) (*)
<1>     ;                      (HW = rows, DX = columns)
<1>     ;                       (<=1 -> horizontal or vertical line)
<1>     ;                EDI = user's buffer address
<1>     ;
<1>     ;            Example: (29/01/2021)
<1>     ;                ecx = 00400064h (start at row 64, column 100)
<1>     ;                edx = 00320048h (size: 50 rows, 72 columns)
<1>     ;                              (end at row 114, column 172)
<1>     ;                If video memory starts at 0A0000h
<1>     ;                and if resolution is 320x200 (256 colors) ..
<1>     ;                window start offset: (64*320)+100 = 20580
<1>     ;                window size: 16072 bytes (pixels)
<1>     ;                window end offset: 20580+16072 = 36652
<1>     ;                window start address: 0A0000h+564h = 0A5064h
<1>     ; Outputs:
<1>     ;      EAX = transfer/byte count
<1>     ;
<1>     ;      NOTE: If the source or destination address passes out of
<1>     ;      video pages (display memory limits), data will not be transferred
<1>     ;      and EAX will return as 0.
<1>     ;
<1>     ; 08/02/2021
<1>     ; 07/02/2021
<1>     ; 04/01/2021
<1>     ; PIXEL READ/WRITE (in current/active video mode)
<1>     ;
<1>     ;      BH = 3 = Read/Write pixel(s) -for all graphics modes-
<1>     ;           BL =
<1>     ;             0 = Read pixel
<1>     ;             1 = Write pixel
<1>     ;             2 = swap pixel colors
<1>     ;             3 = mix pixel colors
<1>     ;           29/01/2021
<1>     ;             4 = read pixels from user defined positions
<1>     ;             5 = write single color pixels to user defined positions
<1>     ;             6 = write multi color pixels to user defined positions
<1>     ;
<1>     ;            > 6 = invalid/unimplemented
<1>     ;
<1>     ;           .. for BL = 0 to 5
<1>     ;           CL = color for writing pixel(s) or
<1>     ;           ECX = color for writing pixel(s) in true color modes
<1>     ;
<1>     ;           EDX = Offset from start of video memory (0A0000h)
<1>     ;                 or start of linear frame buffer
<1>     ;
<1>     ;           07/02/2021
<1>     ;           .. for BL = 4
<1>     ;           EDI = user's destination buffer address for pixel colors
<1>     ;           29/01/2021
<1>     ;           .. for BL = 4 & 5
<1>     ;           ESI = user's source buffer address for BL = 4 & 5
<1>     ;           (buffer contains dword offset positions for pixels)
<1>     ;           EDX = number of pixels
<1>     ;           .. for BL = 6
<1>     ;           ESI = user's buffer address for BL = 6
<1>     ;           (buffer contains dword offset position and dword color
<1>     ;            value for each pixel)
<1>     ;           EDX = number of pixels
<1>     ;
<1>     ; Note:
<1>     ;      Pixel read/write will be performed in current video mode.
<1>     ;      If [CRT_MODE] < 0FFh, 0A0000h will be used
<1>     ;         as video memory and limit will be 65536
<1>     ;         (new/mix pixel color will be in CL)
<1>     ;      if [CRT_MODE] = 0FFh (VESA VBE video mode)
<1>     ;         LFB base address will be used as video memory
<1>     ;         and limit will be video page size
<1>     ;         (new/mix pixel color will be in CL)
<1>     ;
<1>     ; Outputs:
<1>     ;      EAX = pixel color (according to BL and ECX input)
<1>     ;      EAX = 0 (pixel color is 0 or there is an error)
<1>     ;            (BL will return as 0FFh if there is an error)
<1>     ;      ; 29/01/2021
<1>     ;      EAX = number of pixels (for BL input = 4&5&6)
<1>     ;
<1>     ; DIRECT (STANDARD VGA/CGA) DISPLAY MEMORY ACCESS FUNCTIONS:
<1>     ;
<1>     ;      BH = 4 = CGA direct video memory (0B8000h, 32K) access
<1>     ;              Page directory & page tables of the user's
<1>     ;              program will be updated to direct access to
<1>     ;              0B8000h (32K) video (CGA, color) memory; if
<1>     ;              there is not a permission conflict or lock!
<1>     ;               (User's program/process will have permission to
<1>     ;              access locked display memory if the owner is
<1>     ;              it's parent.)
<1>       ;
<1>     ;          Screen width = 320
<1>     ;
<1>     ;      BH = 5 = VGA direct video memory (0A0000h, 64K) access
<1>     ;              Page directory & page tables of the user's
<1>     ;              program will be updated to direct access to
<1>     ;              0A0000h (64K) video (VGA) memory; if there is not
<1>     ;              a permission conflict or lock!
<1>     ;               (User's program/process will have permission to
<1>     ;              access locked display memory if the owner is
<1>     ;              it's parent.)
<1>     ;
<1>     ;          ; 23/11/2020
<1>     ;          Screen width options = 320, 640, 800
<1>     ;
<1>     ; Outputs:
<1>     ;      EAX = Display memory address for direct access
<1>     ;            0A0000h for VGA, 0B8000h for CGA
<1>     ;      (Display memory size: 32K for CGA, 64K for VGA)
<1>     ;      EAX = 0 if display page access permission has been denied.
<1>     ;            (Locked!)
<1>     ;
```

```
<1>     ; LINEAR FRAME BUFFER ACCESS FUNCTIONS:
<1>     ;
<1>     ;       BH = 6 = Linear Frame Buffer direct video memory access
<1>     ;
<1>     ;               Page directory & page tables of the user's
<1>     ;               program will be updated to direct access to
<1>     ;               the configured LFB (Linear Frame Buffer) address,
<1>     ;               if there is not a permission conflict or lock!
<1>     ;                (User's program/process will have permission to
<1>     ;               access locked display memory if the owner is
<1>     ;               it's parent.)
<1>     ;
<1>     ;          ; 10/12/2020
<1>     ;          BL = 0FFh -> Direct LFB access for current video mode
<1>     ;          BL = XXh < 0FFh -> Direct LFB access
<1>     ;                             for VESA video mode 1XXh
<1>     ;
<1>     ;               Return: EAX = Linear Frame Buffer address
<1>     ;                      (EAX = 0 -> error)
<1>     ;                    If EAX > 0
<1>     ;                       EDX = Frame Buffer Size in bytes
<1>     ;                         BH = Requested Video Mode - 100h
<1>     ;                              (VESA VBE video modes)
<1>     ;                         BL = bits per pixel
<1>     ;                              8 = 256 colors, 8
<1>     ;                             16 = 65536 colors, 5-6(G)-5
<1>     ;                             24 = RGB, 16M colors, 8-8-8
<1>     ;                             32 = RGB + alpha bytes, 8-8-8-8
<1>     ;                    If BH = 0FFh
<1>     ;                       BL = VGA/CGA video mode (also EAX = 0)
<1>     ;
<1>     ;               ** Function will return with EAX = 0 if the mode
<1>     ;               is not a valid VESA VBE video mode as 1??h **
<1>     ;
<1>     ;               ECX = Pixel resolution
<1>     ;                     CX = Width (320, 640, 800, 1024, 1366, 1920)
<1>     ;                     High 16 bits of ECX = Height
<1>     ;
<1>     ;       23/11/2020
<1>     ; ***   GET VIDEO MODE & LINEAR FRAME BUFFER INFO
<1>     ;       (This function -7- also is used for VGA and CGA modes)
<1>     ;
<1>     ;       BH = 7 = Get Linear Frame Buffer info
<1>     ;
<1>     ;          ; 22/01/2021
<1>     ;          ; 10/12/2020
<1>     ;          BL = any -not used- (22/01/2021)
<1>     ;
<1>     ;               Return:
<1>     ;               EAX = Frame Buffer Address (0 = is not in use)
<1>     ;               EDX = Frame Buffer Size in bytes
<1>     ;                BH = Current Video Mode - 100h  ; 22/01/2021
<1>     ;                     (VESA VBE video modes)
<1>     ;                BL = bits per pixel
<1>     ;                       8 = 256 colors, 8
<1>     ;                      16 = 65536 colors, 5-6(G)-5
<1>     ;                      24 = RGB, 16M colors, 8-8-8
<1>     ;                      32 = RGB + alpha bytes, 8-8-8-8
<1>     ;               If BH = 0FFh
<1>     ;                  BL = VGA/CGA video mode (also EAX = 0)
<1>     ;
<1>     ;               Note:
<1>     ;               Alpha byte will be used as virtual color index.
<1>     ;               (32 bit pixel colors.. byte 0,1,2 rgb and 3 alpha)
<1>     ;
<1>     ;               ** Function will return with EAX = 0 if the mode
<1>     ;               is not a valid VESA VBE video mode as 1??h **
<1>     ;
<1>     ;               ECX = Pixel resolution
<1>     ;                     CX = Width (320, 640, 800, 1024, 1366, 1920)
<1>     ;                     High 16 bits of ECX = Height
<1>     ;
<1>     ;       NOTE: Each process will have it's own frame buffer
<1>     ;             address and resolution parameters in 'u' area.
<1>     ;             Then, if the current frame buffer & resolution
<1>     ;             is different, frame buffer r/w functions
<1>     ;             will use scale factor to convert process's
<1>     ;                 pixel coordinates to actual screen coordinates.
<1>     ;             resolution -> dimensional scale
<1>     ;             color size -> color scale
<1>     ;            * RGB (TRUE) colors to 256 colors conversion:
<1>     ;                 TRUE Colors -> 8,8,8 (R,G,B; byte 0 is R)
<1>     ;             256 colors -> 2,2,2,2 (R,G,B,L; bit 0&1 is R)
<1>     ;                bit 6&7 -> luminosity base level (0,1,2,3)
<1>     ;                bit 4&5 -> blue level (0,1,2,3)
<1>     ;                bit 2%3 -> green level (0,1,2,3)
<1>     ;                bit 0&1 -> red level (0,1,2,3)
<1>     ;             Example: total red level : luminosity + red level
<1>     ;             Luminosity base level: 0 -> 16
<1>     ;                                    1 -> 32
<1>     ;                                    2 -> 64
<1>     ;                                    3 -> 128
<1>     ;             Color level:
<1>     ;                                   0 -> 0
<1>     ;                                   1 -> luminosity level
<1>     ;                                   2 -> luminosity level + 64
<1>     ;                                   3 -> 255
<1>     ;            Luminosity base level = min (R,G,B)
<1>     ;                      if it is <16, it will be set to 16
<1>     ;            Color levels: Color values are fixed to (nearest)
<1>     ;                  one of all possible set level (step) values
<1>     ;                  (according to luminosity base level); then
<1>     ;                  color levels are set to R-L, G-L, B-L.
<1>     ;        For example: If luminosity base level is 32
<1>     ;                 all possible set values are 0, 32, 96, 255.
<1>     ;
<1>     ;           * RGB (TRUE) colors to 16 colors conversion:
<1>     ;           16 colors: R,B,G,L bits (4 bits)
<1>     ;                  If any one of R,G,B >= 128 L = 1
<1>     ;                  If max. value of (R,G,B) >= 32, it is 1
<1>     ;                     else all color bits (R&G&B&L) are 0
<1>     ;                  If the second value >= max. value / 2
<1>     ;                     it is 1
<1>     ;                  If the third value >= max. value / 2
<1>     ;                     it is 1
<1>     ;           Example: R = 132, G = 64, B = 78
<1>     ;                   L = 1, R = 1
<1>     ;                   G < 66 --> G = 0
<1>     ;                   B >= 66 --> B = 1
<1>     ;
<1>     ; 10/12/2020
<1>     ; SET VIDEO MODE (& RETURN LFB INFO for VESA VBE VIDEO MODES)
<1>     ;
<1>     ;       BH = 8 = Set Video Mode
```

```
<1>     ;
<1>     ;               BL = Requested Video Mode (method)
<1>     ;               If BL = 0FFh
<1>     ;                  CX = VESA VBE Video Mode
<1>     ;                  ; 11/12/2020
<1>     ;                  If EDX > 0 -> LFB INFO (user) buffer addr
<1>     ;               If BL < 0FFh, it is VGA/CGA video mode and
<1>     ;                      CX & EDX will not be used
<1>     ;
<1>     ;           NOTE: The last VESA VBE video mode is 11Bh but
<1>     ;           TRDOS 386 will permit to set video mode upto 11Fh.
<1>     ;           Above 11Fh, from 140h to 1FEh, it will be accepted
<1>     ;           as Bochs/Plex86 emulator video mode and it will be
<1>     ;           used only if [vbe3] = 2 and detected
<1>     ;           video bios is BOCHS/PLEX86/QEMU/VIRTUALBOX vbios.
<1>     ;
<1>     ; Outputs:
<1>     ;      EAX = Requested (Proper) video mode number + 1
<1>     ;            ("dec eax" by user will give requested video mode),
<1>     ;
<1>     ;      If BL input is 0FFh
<1>     ;         EDX = LFBINFO table/structure (in user's buffer addr)
<1>     ;         EDX = 0 -> Invalid LFBINFO (do not use it)
<1>     ;
<1>     ;      EAX = 0 -> Error (but EDX will not be changed)
<1>     ;
<1>     ; 03/12/2020
<1>     ; VESA VBE3 VIDEO BIOS (32 bit) PROTECTED MODE INTERFACE SETTINGS
<1>     ;
<1>     ;      BH = 9 = set/get VBE3 Protected Mode Interface parameters
<1>     ;
<1>     ;               BL = 0 - Disable protected mode interface
<1>     ;                      ([pmi32] = 0)
<1>     ;                      Return: AL = 1
<1>     ;               BL = 1 - Enable protected mode Interface
<1>     ;                      ([pmi32] = 1)
<1>     ;                      Return: AL = 2
<1>     ;               BL = 2 - Get protected mode interface status
<1>     ;                      Return: AL = [pmi32] + 1 (AL = 1 or 2)
<1>     ;
<1>     ;               If [vbe3] <> 3 --> AL = 0
<1>     ;
<1>     ;               ; 17/01/2021
<1>     ;               BL = 3 - Disable/Cancel restore permission to user
<1>     ;                      Return: AL = 1 (if disabled) or 0
<1>     ;               BL = 4 - Enable/Give restore permission to user
<1>     ;                      Return: AL = 2 (if enabled) or 0
<1>     ;               BL = 5 - Get video state save/restore status
<1>     ;                       (permission status)
<1>     ;                       Return: AL = Status (enabled = 1)
<1>     ;                              ; 22/01/2021
<1>     ;                              AH = state options ([srvso])
<1>     ;               BL = 6 - Return VESA VBE number/status
<1>     ;                       Return: AX = status
<1>     ;                             if AH =  2, AL > 0 : Emulator
<1>     ;                               AH =  3, VESA VBE3 video bios
<1>     ;               ; 28/02/2021
<1>     ;               BL = 7 - Set true color mode as 32bpp (default)
<1>     ;                      Return: AX = 32 (if VBE3)
<1>     ;               NOTE: Initial/default value is 32bpp for vbe3.
<1>     ;                      Return: AX = 0 -> error
<1>     ;               BL = 8 - Set true color mode as 24bpp (default)
<1>     ;                      Return: AX = 24
<1>     ;                      ;Return: AX = 0 -> error
<1>     ;               BL = 9 - Return default/current true color mode
<1>     ;                      Return: AX = 32 (32 bpp)
<1>     ;                      Return: AX = 24 (24 bpp)
<1>     ;                      Return: AX = 0 -> error (not VESA bios)
<1>     ;
<1>     ;               BL > 9 : not implemented (28/02/2021)
<1>     ;
<1>     ;      ; 19/01/2021 ([u.uid] check)
<1>     ;      Note: Enabling/Disabling are done by root ([u.uid] = 0)
<1>     ;            while [multi_tasking] = 0.
<1>     ;
<1>     ; 23/12/2020
<1>     ; VIDEO MEMORY MAPPING:
<1>     ;      BH = 10 = Map video memory to user's buffer
<1>     ;
<1>     ;         BL = 0 : CGA memory (0B8000h) map (32K)
<1>     ;         BL = 1 : VGA memory (0A0000h) map (64K)
<1>     ;         BL = 2 : SVGA memory (LFB) map to user's buffer
<1>     ;
<1>     ;      ECX = User's buffer addr (low 12 bits will be cleared)
<1>     ;      EDX = Buffer size in bytes (if BL = 2)
<1>     ;              (will be trimmed if LFB size < EDX)
<1>     ; Return:
<1>     ;   EAX = physical address of video memory (buffer)
<1>     ;      EBX = mapped (actual) size of video memory (bytes)
<1>     ;      ECX = virtual start address of user's video buffer
<1>     ;      EDX is same with EDX input
<1>     ;
<1>     ;      (Note: Memory page boundaries will be applied
<1>     ;       to buffer size and buff start addr by rounding down.
<1>     ;       Rounded size & address values must not be zero.)
<1>     ;      -Normally, it is expected to request mapping by using
<1>     ;       correct buffer size of current or desired video mode-
<1>     ;
<1>     ;      EAX = 0 -> error ! memory can not mapped to user
<1>     ;
<1>     ; 04/01/2021
<1>     ; SET/READ COLOR PALETTE (set/read DAC color registers)
<1>     ;      ((256 colors (8bpp) VGA/CGA video hardware feature))
<1>     ;
<1>     ;      BH = 11 = Set/Read DAC color registers
<1>     ;
<1>     ;         (BL<4 Original method for std VGA video hardware)
<1>     ;         BL = 0 : Read all DAC color registers (256 colors)
<1>     ;             (6 bit colors, in RGB order)
<1>     ;         BL = 1 : Set all DAC color registers (256 colors)
<1>     ;             (6 bit colors, in RGB order)
<1>     ;         BL = 2 : Read single DAC color register
<1>     ;             (6 bit color, in RGB order)
<1>     ;              ((EAX will return with color value))
<1>     ;              CL = DAC color register (index)
<1>     ;         BL = 3 : Set/Write single DAC color register
<1>     ;             (6 bit color, in RGB order, bit 6&7 are 0)
<1>     ;              ECX byte 0 - DAC color register
<1>     ;              ECX byte 1 - Red (6 bit)
<1>     ;              ECX byte 2 - Green (6 bit)
<1>     ;              ECX byte 3 - Blue (6 bit)
<1>     ;         (BL>3 Alternative method for BMP files etc.)
<1>     ;         BL = 4 : Read all DAC color registers (256 colors)
<1>     ;             (8 bit colors, in BGR order, bit 0&1 is 0)
```

```
<1>     ;         BL = 5 : Set all DAC color registers (256 colors)
<1>     ;             (8 bit colors, in BGR order, bit 0&1 is 0)
<1>     ;         BL = 6 : Read single DAC color register
<1>     ;             (8 bit color, in BGR order, bit 0&1 is 0)
<1>     ;             ((EAX will return with color value))
<1>     ;             CL = DAC color register (index)
<1>     ;         BL = 7 : Set/Write single DAC color register
<1>     ;             (8 bit color, bit 0&1 are 0)
<1>     ;             ECX byte 0 - DAC color register
<1>     ;             ECX byte 1 - Blue (8 bit)
<1>     ;             ECX byte 2 - Green (8 bit)
<1>     ;             ECX byte 3 - Red (8 bit)
<1>     ;
<1>     ;        BL > 7 : invalid (not implemented)
<1>     ;
<1>     ;   if BL bit 2 is 1, 6 bit colors converted to 8 bit colors
<1>     ;       (low two bits of color bytes will be 0)
<1>     ;     ((color byte 00111111b will be converted to 11111100b))
<1>     ;      and RGB byte order will be
<1>     ;             byte 0 - Blue (low 2 bits are 0)
<1>     ;             byte 1 - Green (low 2 bits are 0)
<1>     ;             byte 2 - Red (low 2 bits are 0)
<1>     ;             byte 3 - pad (or zero byte)
<1>     ;      and 256 colors buffer size must be 256*4 = 1024 bytes
<1>     ;   if BL bit 2 is 0, 6 bit colors will be used directly
<1>     ;        (high two bits of 8 bit color bytes will be 0)
<1>     ;     ((dac color 111111b will be converted to 00111111b))
<1>     ;             byte 0 - Red (high 2 bits are 0)
<1>     ;             byte 1 - Green (high 2 bits are 0)
<1>     ;             byte 2 - Blue (high 2 bits are 0)
<1>     ;      and 256 colors buffer size must be 256*3 = 768 bytes
<1>     ;
<1>     ;      ECX = User's buffer addr (256*3 = 768 bytes) or
<1>     ;            Color
<1>     ;
<1>     ; Return:
<1>       ;   EAX = buffer size (for BL input = 0,1,4,5)
<1>     ;            or color value (for BL input = 2,3,6,7)
<1>     ;
<1>     ; 10/01/2021
<1>     ; SET/READ FONT DATA
<1>     ;
<1>     ;      BH = 12 = Set/Read Character Font Data
<1>     ;
<1>     ;         BL = 0 : Disable system font overwrite
<1>     ;         BL = 1 : Enable system font overwrite
<1>     ;         BL = 2 : Read system font 8x8
<1>     ;         BL = 3 : Read system font 8x14
<1>     ;         BL = 4 : Read system font 8x16
<1>     ;         BL = 5 : Read user defined font 8x8
<1>     ;         BL = 6 : Read user defined font 8x16
<1>     ;         BL = 7 : Write system font 8x8
<1>     ;         BL = 8 : Write system font 8x14
<1>     ;         BL = 9 : Write system font 8x16
<1>     ;         BL = 10 : Write user defined font 8x8
<1>     ;         BL = 11 : Write user defined font 8x16
<1>     ;
<1>     ;        BL > 11 : invalid (not implemented)
<1>     ;
<1>     ;      For BL = 1 to 11
<1>     ;         ECX = number of characters (<= 256)
<1>     ;         EDX = first character (ascii code in DL)
<1>     ;         ESI = user's buffer address
<1>     ;
<1>     ; Return:
<1>       ;   EAX = number of characters (ecx input)
<1>     ;      EAX = 0 -> error
<1>     ;      (EAX = 256 for BL = 0 and 1 if successful)
<1>     ;
<1>     ; Note: system font overwrite permission will be
<1>     ;       given if [multi_tasking] = 0
<1>     ;      and [u.uid] = 0 (BL = 1) ; 19/01/2021
<1>     ;       and if [ufont] bit 7 is 1 (BL = 7,8,9)
<1>     ;
<1>     ; 18/01/2021
<1>     ; SAVE/RESTORE STANDARD VGA VIDEO STATE
<1>     ;
<1>     ;      BH = 13 = Save/Restore std VGA video state
<1>     ;
<1>     ;         BL = 0 : Save VGA state (without DAC regs)
<1>     ;             Return: EAX = VideoStateID (>0)
<1>     ;                     EAX = 0 (failed!)
<1>     ;             (size: 110 bytes for TRDOS 386 v2.0.3)
<1>     ;         BL = 1 : Restore VGA state (without DAC regs)
<1>     ;             ECX = VideoStateID (to be verified)
<1>     ;              Return: EAX = Restore size (>0)
<1>     ;         BL = 2 : Save VGA state (complete)
<1>     ;             Return: EAX = VideoStateID (>0)
<1>     ;                     EAX = 0 (failed!)
<1>     ;             (size: 882 bytes for TRDOS 386 v2.0.3)
<1>     ;         BL = 3 : Restore VGA state (complete)
<1>     ;             ECX = VideoStateID (to be verified)
<1>     ;              Return: EAX = Restore size (>0)
<1>     ;
<1>     ;      * Above options are for saving
<1>     ;      *       video state to system memory
<1>     ;      *      (location: VBE3VIDEOSTATE, 2048 bytes)
<1>     ;
<1>     ;         BL = 4 : Save VGA state (without DAC regs)
<1>     ;             ECX = buffer address
<1>     ;             Return: EAX = transfer count
<1>     ;             (size: 110 bytes for TRDOS 386 v2.0.3)
<1>     ;             ECX = buffer address
<1>     ;         BL = 5 : Restore VGA state (without DAC regs)
<1>     ;             ECX = buffer address
<1>     ;              Return: EAX = transfer count
<1>     ;         BL = 6 : Save VGA state (complete)
<1>     ;             ECX = buffer address
<1>     ;             Return: EAX = transfer count
<1>     ;             (size: 882 bytes for TRDOS 386 v2.0.3)
<1>     ;         BL = 7 : Restore VGA state (complete)
<1>     ;             ECX = buffer address
<1>     ;              Return: EAX = transfer count
<1>     ;
<1>     ;      * Above options are for saving
<1>     ;      *       video state to user's buffer
<1>     ;      *      (buffer size: 110 bytes or 882 bytes)
<1>     ;
<1>     ;        BL > 7 : invalid (not implemented)
<1>     ;
<1>     ; 18/01/2021
<1>     ; SAVE/RESTORE SUPER VGA (VESA VBE 2/3) VIDEO STATE
<1>     ;
<1>     ;      BH = 14 = Save/Restore SVGA video state
```

```
                                <1>     ;
                                <1>     ;         BL = options
                                <1>     ;             bit 0 - Save (0) or Restore (1)
                                <1>     ;             bit 1 - controller hardware state
                                <1>     ;             bit 2 - BIOS data state
                                <1>     ;             bit 3 - DAC state
                                <1>     ;             bit 4 - (extended) Register state
                                <1>     ;             bit 5 - system (0) or user (1) memory
                                <1>     ;             bit 6 - verify without transfer
                                <1>     ;             bit 7 - not used (must be 0)
                                <1>     ;
                                <1>     ;           if bit 0 = 0 and bit 5 = 0
                                <1>     ;             Return: EAX = VideoStateID (>0)
                                <1>     ;           if bit 0 = 1
                                <1>     ;             ECX = VideoStateID (bit 5 = 0)
                                <1>     ;             Return: EAX = restore (transfer) size
                                <1>     ;           if bit 5 = 1
                                <1>     ;             ECX = Buffer address
                                <1>     ;             Return: EAX = transfer count (size)
                                <1>     ;
                                <1>     ;           ECX = Buffer address or VideoStateID
                                <1>     ;
                                <1>     ;        BL > 127 : invalid (not implemented)
                                <1>     ;
                                <1>     ;  Note: Required buffer size may be > 2048 bytes
                                <1>     ;       (function fails when buff size > 2048 bytes)
                                <1>     ;       proper option must be used
                                <1>     ;
                                <1>     ; 18/01/2021
                                <1>     ; READ VESA EDID (EXTENDED DISPLAY IDENTIFICATION DATA)
                                <1>     ;
                                <1>     ;      BH = 15 = Read VESA EDID for connected monitor
                                <1>     ;                (copy EDID to user)
                                <1>     ;
                                <1>     ;         BL = any
                                <1>     ;
                                <1>     ;      Input:
                                <1>     ;          ECX = user's (EDID) buffer address
                                <1>     ;                (buffer size: 128 bytes)
                                <1>     ;      Output:
                                <1>     ;          EAX = 128 (EDID size)
                                <1>     ;          or EAX = 0 -> Error!
                                <1>     ;             (EDID not ready or buffer addr error)
                                <1>     ;
                                <1>
                                <1>     ; 16/05/2016
0000D3DB 31C0                   <1>     xor     eax, eax
0000D3DD A3[6C8E0100]           <1>     mov     [u.r0], eax
0000D3E2 20FF                   <1>     and     bh, bh
                                <1>     ;jnz    sysvideo_13 ; 11/07/2016
                                <1>     ; 23/07/2022
0000D3E4 7405                   <1>     jz      short sysvideo_40
0000D3E6 E974020000             <1>     jmp     sysvideo_13
                                <1>
                                <1> sysvideo_40:        ; 23/07/2022
                                <1>     ;; 21/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ;; tty/text mode transfers are only for video mode 3
                                <1>
                                <1>     ; 22/11/2020
                                <1>     ;cmp    byte [CRT_MODE], 3 ; 80x25 text, 16 colors
                                <1>     ;jne    sysret ; invalid (nothing to do), [u.r0] = 0
                                <1>
                                <1>     ; 23/11/2020
                                <1>     ; bit 7,6,5,4 of BL are reserved and it must be 0
                                <1>     ;               for current 'sysvideo' version
                                <1>     ;test   bl, 0F0h
                                <1>     ;;jnz   sysret ; invalid (undefined) !
                                <1>     ;; 28/01/2021
                                <1>     ;jnz    short sysvideo_1_2 ; invalid (undefined) !
                                <1>     ; 28/01/2021
0000D3EB 80FB07                 <1>     cmp     bl, 7
0000D3EE 776E                   <1>     ja      short sysvideo_1_2 ; invalid (undefined) !
                                <1>
                                <1>     ; Video mode 0, 80*25 text mode, CGA 16 colors
                                <1>     ; [CRT_MODE] = 3
                                <1>     ;mov    bh, bl
                                <1>     ;shr    bh, 2 ; 4..7 -> 1, 8..11 -> 2, 12..15 -> 3
                                <1>     ;; 21/11/2020
                                <1>     ;;and   bh, bh
                                <1>         ;jnz        sysvideo_4  ; Display page window transfer etc.
                                <1>
                                <1>     ; 28/01/2021
0000D3F0 F6C304                 <1>     test    bl, 4 ; bit 2
                                <1>     ;jnz    sysvideo_4 ; Display page window transfer
                                <1>     ; 23/07/2022
0000D3F3 7405                   <1>     jz      short sysvideo_41
0000D3F5 E9A1000000             <1>     jmp     sysvideo_4
                                <1> sysvideo_41:        ; 23/07/2022
                                <1>     ; Display page (complete) transfer
0000D3FA 80FA07                 <1>     cmp     dl, 7
                                <1>     ;jnz    sysret ; invalid (nothing to do), [u.r0] = 0
0000D3FD 760A                   <1>     jna     short sysvideo_0 ; 28/01/2021
0000D3FF FEC2                   <1>     inc     dl ; 0FFh -> 0 ("use current video page")
0000D401 755B                   <1>     jnz     short sysvideo_1_2  ; invalid
                                <1>     ; dl = 0 -> use current current page
0000D403 8A15[DE760100]         <1>     mov     dl, [ACTIVE_PAGE]
                                <1> sysvideo_0:
                                <1>     ; 28/01/2021
0000D409 80FB03                 <1>     cmp     bl, 3
0000D40C 7206                   <1>     jb      short sysvideo_0_0
0000D40E 09FF                   <1>     or      edi, edi
0000D410 744C                   <1>     jz      short sysvideo_1_2  ; invalid
0000D412 89FE                   <1>     mov     esi, edi ; save swap/exchange buffer addr
                                <1>     ; ecx = user buffer for new video page content
                                <1>     ; esi = user (swap) buffer for saving current video page
                                <1> sysvideo_0_0:
0000D414 BF00800B00             <1>     mov     edi, 0B8000h
                                <1>     ; dl = display page number, destination
0000D419 66B80010               <1>     mov     ax, 4096 ; 21/11/2020
0000D41D 20D2                   <1>     and     dl, dl
0000D41F 7408                   <1>     jz      short sysvideo_1
                                <1>     ; 07/02/2021
0000D421 88D6                   <1>     mov     dh, dl
                                <1> sysvideo_0_1:
                                <1>     ; page length = 4096 bytes (but page content is 80*25*2 bytes)
0000D423 01C7                   <1>     add     edi, eax ; 21//11/2020 ([CRT_LEN] = 1000h for mode 3)
0000D425 FECE                   <1>     dec     dh
0000D427 75FA                   <1>     jnz     short sysvideo_0_1
                                <1> sysvideo_1:
0000D429 80E303                 <1>     and     bl, 3
0000D42C 7535                   <1>     jnz     short sysvideo_2 ; user to system display page transfer
                                <1>     ; system to system video page (content) transfer
                                <1>     ; cl = display page number, source
0000D42E 80F907                 <1>     cmp     cl, 7
```

```
                                <1>     ;ja     sysret ; invalid (nothing to do), [u.r0] = 0
0000D431 760A                   <1>     jna     short sysvideo_1_0
0000D433 FEC1                   <1>     inc     cl ; 0FFh -> 0 ("use current video page")
0000D435 7527                   <1>     jnz     short sysvideo_1_2  ; invalid
0000D437 8A0D[DE760100]         <1>     mov     cl, [ACTIVE_PAGE]
                                <1> sysvideo_1_0:
                                <1>     ; 28/01/2021
0000D43D 38D1                   <1>     cmp     cl, dl
0000D43F 741D                   <1>     je      short sysvideo_1_2  ; same video page !
                                <1>
                                <1>     ; system to system video/display page transfer (mode 0)
                                <1>     ; 21/11/2020
                                <1>     ;mov    esi, 0B8000h
                                <1>     ;movzx  eax, cl
                                <1>     ;mov    edx, 4096 ; [CRT_LEN] = 1000h for video mode 3
                                <1>     ;mov    ecx, edx
                                <1>     ;mul    edx
                                <1>     ;add    esi, eax
                                <1>     ; 28/01/2021
                                <1>     ;movzx  esi, cl
                                <1>     ;shl    si, 12 ; * 4096
                                <1>     ;add    esi, 0B8000h
                                <1>
                                <1>     ; 28/01/2021
0000D441 A3[6C8E0100]           <1>     mov     [u.r0], eax ; 4096
0000D446 BE00800B00             <1>     mov     esi, 0B8000h
0000D44B 08C9                   <1>     or      cl, cl
0000D44D 7409                   <1>     jz      short sysvideo_1_1
                                <1>     ; 07/02/2021
                                <1>     ;mov    al, cl ; display/video page
                                <1>     ;xor    ah, ah
                                <1>     ;shl    ax, 12 ; * 4096
                                <1>     ; 23/07/2022
0000D44F 30C0                   <1>     xor     al, al
                                <1>     ;xor    eax, eax
0000D451 88CC                   <1>     mov     ah, cl ; CL * 256
0000D453 C1E004                 <1>     shl     eax, 4 ; * 16 = CL * 4096
0000D456 01C6                   <1>     add     esi, eax
                                <1> sysvideo_1_1:
                                <1>     ; 21/11/2020
                                <1>     ;;mov   ecx, 4096
                                <1>     ;mov    ecx, eax ; 4096
                                <1>     ;;mov   [u.r0], ecx ; 4096 bytes
                                <1>     ; 28/01/2021
                                <1>     ;mov    [u.r0], cx
0000D458 31C9                   <1>     xor     ecx, ecx
0000D45A B504                   <1>     mov     ch, 4 ; mov ecx, 1024
                                <1>     ;shr    cx, 2 ; / 4
0000D45C F3A5                   <1>     rep     movsd
                                <1> sysvideo_1_2:
0000D45E E92BF6FFFF             <1>     jmp     sysret
                                <1> sysvideo_2:
0000D463 80FB02                 <1>     cmp     bl, 2
                                <1>         ;ja sysret; invalid (user to user), [u.r0] = 0
                                <1>     ; 28/01/2021
0000D466 7226                   <1>     jb      short sysvideo_3 ; user to system
0000D468 7404                   <1>     je      short sysvideo_2_0 ; system to user
                                <1>     ; bl = 3
0000D46A 89CA                   <1>     mov     edx, ecx ; save user's buffer addr
0000D46C 89F1                   <1>     mov     ecx, esi ; save swap address
                                <1> sysvideo_2_0:
                                <1>     ; bl = 2 (or bl = 3, stage 1)
                                <1>     ; system to user video/display page transfer (mode 0)
0000D46E 89FE                   <1>     mov     esi, edi
0000D470 89CF                   <1>     mov     edi, ecx ; user buffer ; 28/01/2021
                                <1>     ; 21/11/2020
0000D472 89C1                   <1>     mov     ecx, eax ; 4096
0000D474 E8A2360000             <1>     call    transfer_to_user_buffer ; fast transfer
                                <1>     ;jc     sysret ; [u.r0] = 0
0000D479 72E3                   <1>     jc      short sysvideo_1_2 ; 28/01/2021
                                <1>     ; 28/01/2021
0000D47B 80FB03                 <1>     cmp     bl, 3
0000D47E 7408                   <1>     je      short sysvideo_2_2
                                <1> sysvideo_2_1:
                                <1>     ; 21/11/2020
0000D480 890D[6C8E0100]         <1>     mov     [u.r0], ecx
                                <1>     ;;mov   [u.r0], cx
                                <1>     ;jmp    sysret
0000D486 EBD6                   <1>     jmp     short sysvideo_1_2
                                <1>
                                <1> sysvideo_2_2:
                                <1>     ; bl = 3 (exchange/swap) complete display page
                                <1>     ; esi = video page start address
                                <1>     ; edx = user's buffer address
                                <1>
                                <1>     ;mov    ecx, 4096
0000D488 89F7                   <1>     mov     edi, esi ; video page start address
0000D48A 89D6                   <1>     mov     esi, edx ; user's (new page) buffer address
0000D48C EB04                   <1>     jmp     short sysvideo_2_3
                                <1> sysvideo_3:
                                <1>     ; bl = 1 (or bl = 3, stage 2)
                                <1>     ; user to system video/display page transfer (mode 0)
0000D48E 89CE                   <1>     mov     esi, ecx ; user buffer
                                <1>     ; edi = video page address
                                <1>     ; 21/11/2020
0000D490 89C1                   <1>     mov     ecx, eax ; 4096
                                <1> sysvideo_2_3:
0000D492 E8CE360000             <1>     call    transfer_from_user_buffer ; fast transfer
                                <1>     ;jc     sysret ; [u.r0] = 0
0000D497 72C5                   <1>     jc      short sysvideo_1_2 ; 28/01/2021
0000D499 EBE5                   <1>     jmp     short sysvideo_2_1
                                <1>
                                <1>     ; 21/11/2020
                                <1>     ;mov    [u.r0], ecx
                                <1>     ;;mov   [u.r0], cx
                                <1>     ;;jmp   sysret
                                <1>     ;jmp    short sysvideo_1_2 ; 28/01/2021
                                <1>
                                <1> sysvideo_4:
                                <1>     ; 23/07/2022
                                <1>     ; 23/11/2020 (TRDOS 386 v2.0.3)
                                <1>
                                <1>     ; Display page window transfer etc.
0000D49B 80E303                 <1>     and     bl, 3
                                <1>         ;jnz        sysvideo_9 ; user to system or system to user
                                <1>     ; 23/07/2022
0000D49E 7405                   <1>     jz      short sysvideo_4_0
0000D4A0 E9E3000000             <1>     jmp     sysvideo_9
                                <1> sysvideo_4_0:
                                <1>     ; 21/11/2020
                                <1>     ; system to system video/display page window transfer (mode 0)
0000D4A5 81FE00800B00           <1>     cmp     esi, 0B8000h   ; source buffer address (system)
                                <1>     ;jb     sysret
                                <1>     ; 23/07/2022
```

```
0000D4AB 7245                <1>     jb      short sysvideo_4_3  ; jmp sysret
0000D4AD 81FE00000C00        <1>     cmp     esi, 0B8000h+(4096*8)
                             <1>     ; 23/07/2022
                             <1>     ;jnb    sysret
0000D4B3 733D                <1>     jnb     short sysvideo_4_3  ; jmp sysret
0000D4B5 81FF00800B00        <1>     cmp     edi, 0B8000h   ; destination buffer address (system)
                             <1>     ;jb     sysret
                             <1>     ; 23/07/2022
0000D4BB 7235                <1>     jb      short sysvideo_4_3  ; jmp sysret
0000D4BD 81FF00000C00        <1>         cmp     edi, 0B8000h+(4096*8)
                             <1>     ; 23/07/2022
                             <1>     ;jnb    sysret
0000D4C3 732D                <1>     jnb     short sysvideo_4_3  ; jmp sysret
                             <1>     ;
0000D4C5 51                  <1>     push    ecx ; X1 and Y1 position - top left column, row
0000D4C6 0FB7C1              <1>     movzx   eax, cx ; top left column
                             <1>     ; 21/11/2020
0000D4C9 C1E910              <1>     shr     ecx, 16 ; top row
0000D4CC 740E                <1>     jz      short sysvideo_4_1 ; bypass following code
0000D4CE 52                  <1>     push    edx ; X2 and Y2 position - bottom right column, row
0000D4CF 50                  <1>     push    eax
0000D4D0 66B8A000            <1>     mov     ax, 80*2 ; 80 colums, 160 bytes per row
0000D4D4 F7E1                <1>     mul     ecx
                             <1>             ; eax = offset for start row number
0000D4D6 01C6                <1>     add     esi, eax
0000D4D8 01C7                <1>     add     edi, eax
0000D4DA 58                  <1>     pop     eax
0000D4DB 5A                  <1>     pop     edx
                             <1> sysvideo_4_1:
                             <1>     ;shl    ax, 1 ; * 2 ; convert start column number to offset
                             <1>     ; 23/07/2022
0000D4DC D1E0                <1>     shl     eax, 1
0000D4DE 7404                <1>     jz      short sysvideo_4_2
0000D4E0 01C6                <1>     add     esi, eax
0000D4E2 01C7                <1>     add     edi, eax
                             <1>             ; esi = source page window start offset
                             <1>             ; edi = destination page window start offset
                             <1> sysvideo_4_2:
0000D4E4 59                  <1>     pop     ecx
                             <1>     ;mov    eax, 0B8000h+(80*25*2*8)
                             <1>     ; 21/11/2020
0000D4E5 B800000C00          <1>     mov     eax, 0B8000h+(4096*8)
0000D4EA 39C6                <1>     cmp     esi, eax
                             <1>     ;jnb    sysret ; out of video page
                             <1>     ; 23/07/2022
0000D4EC 7304                <1>     jnb     short sysvideo_4_3  ; jmp sysret
0000D4EE 39C6                <1>     cmp     esi, eax
                             <1>     ;jnb    sysret  ;out of video page
                             <1>     ; 23/07/2022
0000D4F0 7205                <1>     jb      short sysvideo_4_4
                             <1> sysvideo_4_3:
0000D4F2 E997F5FFFF          <1>     jmp     sysret
                             <1> sysvideo_4_4:
0000D4F7 56                  <1>     push    esi ; ****
0000D4F8 57                  <1>     push    edi ; ***
0000D4F9 52                  <1>     push    edx ; **
0000D4FA 51                  <1>     push    ecx ; *
0000D4FB C1E910              <1>     shr     ecx, 16 ; top row
0000D4FE C1EA10              <1>     shr     edx, 16 ; bottom row
                             <1>     ; 21/11/2020
                             <1>     ;cmp    ecx, 24 ; max. 25 rows
0000D501 6683F918            <1>     cmp     cx, 24
0000D505 7778                <1>     ja      short sysvideo_6 ; invalid, [u.r0] = 0
                             <1>     ;cmp    edx, 24 ; max. 25 rows
0000D507 6683FA18            <1>     cmp     dx, 24
0000D50B 7772                <1>     ja      short sysvideo_6 ; invalid, [u.r0] = 0
0000D50D 28CA                <1>     sub     dl, cl ; end >= start
0000D50F 726E                <1>     jc      short sysvideo_6 ; invalid, [u.r0] = 0
                             <1>     ; 21/11/2020
0000D511 89D3                <1>     mov     ebx, edx ; row count - 1
0000D513 7415                <1>     jz      short sysvideo_4_5
0000D515 50                  <1>     push    eax ; *****
0000D516 B8A0000000          <1>     mov     eax, 80*2 ; bytes per row
0000D51B F7E3                <1>     mul     ebx ; 21/11/2020
                             <1>             ; eax = window end offset
                             <1>             ; (for the last row, before adding column bytes)
0000D51D 01C6                <1>     add     esi, eax
0000D51F 01C7                <1>     add     edi, eax
0000D521 58                  <1>     pop     eax ; ***** ; mode 3 video memory end (0C000h)
0000D522 39C6                <1>     cmp     esi, eax
                             <1>     ;ja     short sysvideo_6 ; invalid, [u.r0] = 0
0000D524 7359                <1>     jnb     short sysvideo_6 ; 21/11/2020
0000D526 39C7                <1>     cmp     edi, eax
                             <1>     ;ja     short sysvideo_6 ; invalid, [u.r0] = 0
0000D528 7355                <1>     jnb     short sysvideo_6 ; 21/11/2020
                             <1> sysvideo_4_5:
0000D52A 59                  <1>     pop     ecx ; *
0000D52B 5A                  <1>     pop     edx ; **
0000D52C 81E1FFFF0000        <1>     and     ecx, 0FFFFh
0000D532 81E2FFFF0000        <1>     and     edx, 0FFFFh
                             <1>     ; 21/11/2020
                             <1>     ;cmp    ecx, 79 ; max. 80 columns
0000D538 6683F94F            <1>     cmp     cx, 79
0000D53C 7743                <1>     ja      short sysvideo_7 ; invalid, [u.r0] = 0
                             <1>     ;cmp    edx, 79 ; max. 80 columns
0000D53E 6683FA4F            <1>     cmp     dx, 79
0000D542 773D                <1>     ja      short sysvideo_7 ; invalid, [u.r0] = 0
0000D544 28CA                <1>     sub     dl, cl
0000D546 7639                <1>     jna     short sysvideo_7 ; invalid, [u.r0] = 0
                             <1>     ; 21/11/2020
0000D548 740E                <1>     jz      short sysvideo_4_6
                             <1>     ; edx = column count (width) - 1
0000D54A D0E2                <1>     shl     dl, 1 ; * 2 ; byte offset (in end row)
0000D54C 01D6                <1>     add     esi, edx
0000D54E 01D7                <1>     add     edi, edx
                             <1>             ; esi = source page window end offset
                             <1>             ; edi = destination page window end offset
0000D550 39C6                <1>     cmp     esi, eax ; video memory end
                             <1>     ;ja     short sysvideo_7
0000D552 732D                <1>     jnb     short sysvideo_7 ; 21/11/2020
0000D554 39C7                <1>     cmp     edi, eax ; video memory end
                             <1>     ;ja     short sysvideo_7
0000D556 7329                <1>     jnb     short sysvideo_7 ; 21/11/2020
                             <1> sysvideo_4_6:
0000D558 5F                  <1>     pop     edi ; ***
0000D559 5E                  <1>     pop     esi ; ****
0000D55A FEC3                <1>     inc     bl ; row count - 1 -> row count
0000D55C FEC2                <1>     inc     dl ; column count
0000D55E 88D7                <1>     mov     bh, dl
0000D560 D0E2                <1>     shl     dl, 1 ; convert column count to byte offset
                             <1>     ; 21/11/2020
                             <1>     ; esi = source page window start offset
                             <1>     ; edi = destination page window start offset
0000D562 B8A0000000          <1>     mov     eax, 80*2 ; bytes per row
```

```
                                <1>     ; Note: 160 bytes per row (even if move count < 160)
                                <1> sysvideo_5:
0000D567 88F9                   <1>     mov     cl, bh ; move/transfer -word- count per row
0000D569 0115[6C8E0100]         <1>     add     [u.r0], edx ; transfer count in bytes
0000D56F 66F3A5                 <1>     rep     movsw
0000D572 01C6                   <1>     add     esi, eax ; + 160 bytes to next row
0000D574 01C7                   <1>     add     edi, eax ; + 160 bytes to next row
0000D576 FECB                   <1>     dec     bl ; remain count of rows
0000D578 75ED                   <1>     jnz     short sysvideo_5
0000D57A E90FF5FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysvideo_6:
0000D57F 59                     <1>     pop     ecx ; *
0000D580 5A                     <1>     pop     edx ; **
                                <1> sysvideo_7:
0000D581 5F                     <1>     pop     edi ; ***
0000D582 5E                     <1>     pop     esi ; ****
                                <1> sysvideo_8:
0000D583 E906F5FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysvideo_9:
                                <1>     ; user to system or system to user window transfer
                                <1>     ; 28/01/2021 (bl = 3 -> swap/exchange)
                                <1>     ;cmp    bl, 2
                                <1>     ;;ja    sysret  ; user to user transfer is invalid
                                <1>     ;               ; [u.r0] = 0
                                <1>     ;ja     short sysvideo_8 ; 26/12/2020
                                <1>
                                <1>     ; 28/01/2021
0000D588 80FB02                 <1>     cmp     bl, 2
0000D58B 7604                   <1>     jna     short sysvideo_9_8
                                <1>
                                <1>     ; swap/ exchange video memory and user mem windows
                                <1>     ; edi = swap address in user's memory space
0000D58D 21FF                   <1>     and     edi, edi
0000D58F 74F2                   <1>     jz      short sysvideo_8 ; invalid ; 28/01/2021
                                <1>
                                <1> sysvideo_9_8:
0000D591 56                     <1>     push    esi ; ****
0000D592 57                     <1>     push    edi ; ***
0000D593 52                     <1>     push    edx ; **
0000D594 51                     <1>     push    ecx ; *
                                <1>
0000D595 C1E910                 <1>     shr     ecx, 16 ; top row
0000D598 C1EA10                 <1>     shr     edx, 16 ; bottom row
                                <1>
                                <1>     ; 21/11/2020
                                <1>     ;cmp    ecx, 24 ; max. 25 rows
0000D59B 6683F918               <1>     cmp     cx, 24
0000D59F 77DE                   <1>     ja      short sysvideo_6 ; invalid, [u.r0] = 0
                                <1>     ;cmp    edx, 24 ; max. 25 rows
0000D5A1 6683FA18               <1>     cmp     dx, 24
0000D5A5 77D8                   <1>     ja      short sysvideo_6 ; invalid, [u.r0] = 0
0000D5A7 28CA                   <1>     sub     dl, cl
0000D5A9 72D4                   <1>     jc      short sysvideo_6 ; invalid, [u.r0] = 0
                                <1>
                                <1>     ;mov    ch, cl ; top row
                                <1>     ;mov    cl, [ACTIVE_PAGE]
                                <1>
                                <1>     ;mov    edi, 80*25*2 ; 4000
                                <1>     ; 21/11/2020
                                <1>     ;mov    edi, 4096 ; [CRT_LEN = 4096 for video mode 3
                                <1>     ;shl    edi, cl  ; ! wrong for page 2 to page 7 !
                                <1>     ;;add   edi, 0B8000h - 80*25*2
                                <1>     ;add    edi, 0B8000h - 1000h ; - 4096
                                <1>
                                <1>     ; 21/11/2020
                                <1>     ;xor    eax, eax
                                <1>     ;mov    edi, 0B8000h
                                <1>     ;and    cl, cl ; is video page = 0 ?
                                <1>     ;jz     short sysvideo_9_1 ; yes
                                <1>     ; eax = 0
                                <1>
                                <1> ;sysvideo_9_0:
                                <1>     ;add    ax, 4096
                                <1>     ;dec    cl
                                <1>     ;jnz    short sysvideo_9_0
                                <1>     ;add    edi, eax
                                <1>     ;       ; edi = video page start address
                                <1>
                                <1>     ; 21/11/2020
0000D5AB BF00800B00             <1>     mov     edi, 0B8000h
0000D5B0 803D[DE760100]00       <1>     cmp     byte [ACTIVE_PAGE], 0
0000D5B7 760C                   <1>     jna     short sysvideo_9_1
                                <1> stsvideo_9_0:
0000D5B9 B010                   <1>     mov     al, 16 ; 4096/256
0000D5BB F625[DE760100]         <1>     mul     byte [ACTIVE_PAGE]
0000D5C1 86E0                   <1>     xchg    ah, al ; * 256
0000D5C3 01C7                   <1>     add     edi, eax
                                <1>             ; edi = video page start address
                                <1> sysvideo_9_1:
                                <1>     ; bl = transfer direction
                                <1>     ;     (1 = from user, 2 = to user)
                                <1>     ;     (3 = swap) ; 28/01/2021
0000D5C5 88D7                   <1>     mov     bh, dl ; row count - 1
                                <1>     ;mov    dl, ch ; top row
                                <1>     ; 21/11/2020
0000D5C7 08C9                   <1>     or      cl, cl ; top row number
0000D5C9 7408                   <1>     jz      short sysvideo_9_2
                                <1>
                                <1>     ;mov    eax, 80*2
0000D5CB 66B8A000               <1>     mov     ax, 80*2 ; 160, bytes per row
0000D5CF F7E1                   <1>     mul     ecx ; 22/11/2020
0000D5D1 01C7                   <1>     add     edi, eax
                                <1>             ; edi = window start address for top row
                                <1> sysvideo_9_2:
0000D5D3 59                     <1>     pop     ecx ; *
0000D5D4 5A                     <1>     pop     edx ; **
0000D5D5 81E1FFFF0000           <1>     and     ecx, 0FFFFh
0000D5DB 81E2FFFF0000           <1>     and     edx, 0FFFFh
                                <1>     ; 21/11/2020
                                <1>     ;cmp    ecx, 79 ; max. 80 columns
0000D5E1 6683F94F               <1>     cmp     cx, 79
0000D5E5 779A                   <1>     ja      short sysvideo_7 ; invalid, [u.r0] = 0
                                <1>     ;cmp    edx, 79 ; max. 80 columns
0000D5E7 6683FA4F               <1>     cmp     dx, 79
0000D5EB 7794                   <1>     ja      short sysvideo_7 ; invalid, [u.r0] = 0
                                <1>
0000D5ED 28CA                   <1>     sub     dl, cl
0000D5EF 7290                   <1>     jc      short sysvideo_7 ; invalid, [u.r0] = 0
                                <1>
0000D5F1 08C9                   <1>     or      cl, cl ; left column
0000D5F3 7404                   <1>     jz      short sysvideo_9_3 ; 0
                                <1>
```

```
                                <1>     ; 21/11/2020
0000D5F5 D0E1                   <1>     shl     cl, 1  ; column * 2
0000D5F7 01CF                   <1>     add     edi, ecx
                                <1>             ; edi = window start addr for top left column
                                <1> sysvideo_9_3:
0000D5F9 88D1                   <1>     mov     cl, dl
0000D5FB FEC1                   <1>     inc     cl ; column count
0000D5FD D0E1                   <1>     shl     cl, 1 ; column count * 2
                                <1>             ; ecx = transfer count per row
                                <1>
0000D5FF 58                     <1>     pop     eax ; *** (swap address)
0000D600 5E                     <1>     pop     esi ; ****
                                <1>
0000D601 FEC7                   <1>     inc     bh  ; row count
                                <1>
                                <1>     ;mov    edx, 80*2
0000D603 B2A0                   <1>     mov     dl, 80*2  ; bytes per row
                                <1>     ;
                                <1>     ;cmp    bl, 1 ; transfer direction
                                <1>     ;ja     short sysvideo_11 ; system to user transfer
                                <1>     ; 28/01/2021
0000D605 F6C301                 <1>     test    bl, 1
0000D608 7439                   <1>     jz      short sysvideo_11 ; system to user transfer
                                <1>
                                <1>     ; user to system video/display page window transfer (mode 0)
0000D60A 21C0                   <1>     and     eax, eax ; swap address
0000D60C 741B                   <1>     jz      short sysvideo_10 ; no window swap
                                <1> sysvideo_9_7: ; 28/01/2021
                                <1>     ; save previous window content in user's buffer (swap address)
0000D60E 56                     <1>     push    esi ; user buffer
0000D60F 57                     <1>     push    edi ; beginning address of the window
                                <1>     ; 21/11/2020
0000D610 53                     <1>     push    ebx ; save bh
0000D611 89FE                   <1>     mov     esi, edi
0000D613 89C7                   <1>     mov     edi, eax
                                <1> sysvideo_9_4:
0000D615 E801350000             <1>     call    transfer_to_user_buffer ; fast transfer
0000D61A 7208                   <1>     jc      short sysvideo_9_5
                                <1>     ; ecx = actual transfer count (must be same with input)
0000D61C 01D6                   <1>     add     esi, edx ; next row address of (video page) window
0000D61E 01CF                   <1>     add     edi, ecx ; next row address of user's window
                                <1>             ; Note: ecx may be less than row length of video page
                                <1>             ; user's window uses offset according to window width
0000D620 FECF                   <1>     dec     bh
0000D622 75F1                   <1>     jnz     short sysvideo_9_4 ; repeat for next row
                                <1> sysvideo_9_5:
0000D624 5B                     <1>     pop     ebx ; restore bh
0000D625 5F                     <1>     pop     edi
0000D626 5E                     <1>     pop     esi
                                <1>     ;jnc    short sysvideo_10
0000D627 7215                   <1>     jc      short sysvideo_9_6 ; 28/01/2021
                                <1> ;sysvideo_9_6:
                                <1> ;   jmp     sysret ; [u.r0] = 0
                                <1>
                                <1> sysvideo_10:
                                <1>     ; user to system video/display page window transfer (mode 0)
                                <1>     ; esi =user buffer
0000D629 E837350000             <1>     call    transfer_from_user_buffer ; fast transfer
                                <1>     ;jc     sysret
0000D62E 720E                   <1>     jc      short sysvideo_9_6 ; 28/01/2021
                                <1>     ; ecx = actual transfer count (must be same with input)
0000D630 010D[6C8E0100]         <1>     add     [u.r0], ecx ; actual transfer count
0000D636 01D7                   <1>     add     edi, edx ; next row address of (video page) window
0000D638 01CE                   <1>     add     esi, ecx ; next row address of user's window
                                <1>             ; Note: ecx may be less than row length of video page
                                <1>             ; user's window uses offset according to window width
0000D63A FECF                   <1>     dec     bh
0000D63C 75EB                   <1>     jnz     short sysvideo_10 ; repeat for next row
                                <1>     ;jmp    sysret
                                <1> sysvideo_9_6:
0000D63E E94BF4FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysvideo_11:
                                <1>     ; system to user video/display page window transfer (mode 0)
0000D643 87FE                   <1>     xchg    edi, esi
                                <1> sysvideo_12:
                                <1>     ; esi = beginning addr of the (screen, video page) window
                                <1>     ; edi =user's buffer
0000D645 E8D1340000             <1>     call    transfer_to_user_buffer ; fast transfer
                                <1>     ;jc     sysret
                                <1>     ; 23/07/2022
0000D64A 72F2                   <1>     jc      short sysvideo_9_6 ; jmp sysret
                                <1>
                                <1>     ; ecx = actual transfer count (must be same with input)
0000D64C 010D[6C8E0100]         <1>     add     [u.r0], ecx
0000D652 01D6                   <1>     add     esi, edx ; next row (edx = 160)
0000D654 01CF                   <1>     add     edi, ecx ; next row of the user's window
                                <1>                     ; (ecx <= 160)
0000D656 FECF                   <1>     dec     bh
0000D658 75EB                   <1>     jnz     short sysvideo_12
                                <1> sysvideo_12_0:
0000D65A E92FF4FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysvideo_13:
                                <1>     ; 28/12/2020
0000D65F 80FF01                 <1>     cmp     bh, 1
0000D662 7752                   <1>         ja      short sysvideo_15 ; 23/11/2020
                                <1>
                                <1>     ; 25/02/2021
                                <1>     ; 12/02/2021
                                <1>     ; 29/01/2021, 31/01/2021
                                <1>     ; 23/11/2020 (TRDOS 386 v2.0.3)
                                <1>     ; (major modification, from mode 13h to all VGA modes,
                                <1>     ;  except super VGA modes and liner frame buffer method)
                                <1>
                                <1>     ; BH = 1 = VGA Graphics mode (0A0000h) data transfers
                                <1>
                                <1>     ; 29/01/2021
0000D664 66B84001               <1>     mov     ax, 320 ; 320 pixels
0000D668 F6C380                 <1>     test    bl, 80h ; bit 7 (screen width, 640 pixels)
0000D66B 7407                   <1>     jz      short sysvideo_13_0
                                <1>     ;shl    ax, 1  ; 640 pixels
                                <1>     ; 23/07/2022
0000D66D D1E0                   <1>     shl     eax, 1
                                <1>     ;
0000D66F 80E37F                 <1>     and     bl, 7Fh
0000D672 7405                   <1>     jz      short sysvideo_14
                                <1> sysvideo_13_0:
                                <1>     ; 29/01/2021
0000D674 80FB41                 <1>     cmp     bl, 41h
0000D677 77E1                   <1>     ja      short sysvideo_12_0 ; invalid (unknown) sub function
                                <1> sysvideo_14:
0000D679 66A3[8E9D0100]         <1>     mov     [v_width], ax  ; save screen width
0000D67F C705[929D0100]0000-    <1>     mov     dword [v_mem], 0A0000h  ; save video memory address
```

```
0000D687 0A00                <1>
0000D689 C705[969D0100]0000- <1>     mov    dword [v_siz], 65536  ; save video memory size
0000D691 0100                <1>
0000D693 C705[9E9D0100]0000- <1>     mov    dword [v_end], 0B0000h; save end of video page
0000D69B 0B00                <1>
0000D69D B708                <1>     mov    bh, 8
0000D69F 883D[919D0100]      <1>     mov    [v_bpp], bh ; 8       ; bits per pixel (256 colors)
0000D6A5 881D[909D0100]      <1>     mov    [v_ops], bl    ; VGA data transfer options
                             <1>     ;mov   [maskbuff], edi ; 25/02/2021
0000D6AB 893D[A29D0100]      <1>     mov    [maskcolor], edi ; 25/02/2021
                             <1>                    ; save mask color or bitmask buffer address
0000D6B1 E9BE000000          <1>     jmp    sysvideo_15_7
                             <1>
                             <1> sysvideo_15:
                             <1>     ; 23/07/2022
                             <1>     ; 28/12/2020
0000D6B6 80FF02              <1>     cmp    bh, 2
                             <1>         ;ja sysvideo_16
                             <1>     ; 23/07/2022
0000D6B9 7605                <1>     jna    short sysvideo_15_17
0000D6BB E9ED1E0000          <1>     jmp    sysvideo_16
                             <1> sysvideo_15_17:    ; 23/07/2022
                             <1>     ; 25/02/2021
                             <1>     ; 12/02/2021
                             <1>     ; 30/01/2021 - 31/01/2021
                             <1>     ; 01/01/2021 - 29/01/2021
                             <1>     ; 26/12/2020 - 27/12/2020
                             <1>     ; 25/12/2020 (TRDOS 386 v2.0.3)
                             <1>     ;
                             <1>     ; BH = 2 = SVGA (VESA VBE) Graphics mode (LFB) data transfers
                             <1>
                             <1>     ; 25/12/2020
                             <1>     ; resolution table entry will be saved into EBP register
                             <1>
0000D6C0 803D[7C090000]02    <1>     cmp    byte [vbe3], 2 ; VESA VBE 3 video bios
                             <1>                    ; or BOCHS/QEMU/VIRTUALBOX emu video bios
0000D6C7 724E                <1>     jb     short sysvideo_15_4 ; no, nothing to do !
0000D6C9 770B                <1>     ja     short sysvideo_15_0 ; yes
                             <1>
                             <1>     ; Only Bochs/Plex86 (emu) vbe2 video bios is usable in pmid
                             <1>     ; (if [vbe3] = 2)
0000D6CB A0[7D090000]        <1>     mov    al, [vbe2bios] ; Bochs vbios sign is from C0h to C5h
0000D6D0 24F0                <1>     and    al, 0F0h
0000D6D2 3CC0                <1>     cmp    al, 0C0h
0000D6D4 7541                <1>     jne    short sysvideo_15_4 ; unknown (vbe2) video bios
                             <1> sysvideo_15_0:
                             <1>     ; 29/01/2021
0000D6D6 80FB41              <1>     cmp    bl, 41h
0000D6D9 773C                <1>     ja     short sysvideo_15_4 ; invalid (unknown) sub function
                             <1>     ; 29/01/2021
0000D6DB 881D[909D0100]      <1>     mov    [v_ops], bl    ; SVGA data transfer options
                             <1>
0000D6E1 89D8                <1>     mov    eax, ebx ; hw of ebx is vesa vbe video mode
0000D6E3 C1E810              <1>     shr    eax, 16 ; ax = vesa vbe video mode
0000D6E6 7513                <1>     jnz    short sysvideo_15_2
                             <1>     ; ax = 0
                             <1>
                             <1>     ; check & use current video mode
0000D6E8 803D[2E680000]FF    <1>     cmp    byte [CRT_MODE], 0FFh ; extended (SVGA) mode ?
0000D6EF 7526                <1>     jne    short sysvideo_15_4 ; no
                             <1> sysvideo_15_1:
                             <1>     ; use current vbe (svga) video mode
0000D6F1 66A1[269D0100]      <1>     mov    ax, [video_mode] ; extended (SVGA, VESA VBE) mode
0000D6F7 6625FF01            <1>     and    ax, 1FFh ; vesa vbe video mode: 1XXh
                             <1> sysvideo_15_2:
                             <1>     ; 29/01/2021
                             <1>     ;mov   [maskbuff], edi ; 25/02/2021
0000D6FB 893D[A29D0100]      <1>     mov    [maskcolor], edi ; 25/02/2021
                             <1>                    ; save mask color or bitmask buffer address
0000D701 BD[BA6B0000]        <1>     mov    ebp, b_vbe_modes ; vbe mode table (in 'vidata.s')
                             <1> sysvideo_15_3:
0000D706 663B4500            <1>     cmp    ax, [ebp]
0000D70A 7410                <1>     je     short sysvideo_15_5
0000D70C 83C508              <1>     add    ebp, 8 ; vbe mode table entry size
0000D70F 81FD[7A6C0000]      <1>     cmp    ebp, end_of_b_vbe_modes
0000D715 72EF                <1>     jb     short sysvideo_15_3
                             <1> sysvideo_15_4:
                             <1>     ; desired video mode is not a valid (implemented)
                             <1>     ;        extended (VESA VBE, SVGA) video mode
                             <1>     ;
                             <1>     ; nothing to do !
                             <1>
                             <1>     ; [u.r0] = 0  ; return value of EAX
0000D717 E972F3FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysvideo_15_5:
                             <1>     ; get LFB address
0000D71C A1[349D0100]        <1>     mov    eax, [LFB_ADDR] ; [LFB_Info+LFBINFO.LFB_addr]
0000D721 09C0                <1>     or     eax, eax
0000D723 7509                <1>     jnz    short sysvideo_15_6
0000D725 66A1[100F0000]      <1>     mov    ax, [def_LFB_addr] ; default LFB addr
                             <1>                            ; (for vbe mode 118h)
0000D72B C1E010              <1>     shl    eax, 16
                             <1>     ; 27/12/2020
                             <1>     ;jz    short sysvideo_15_4
                             <1> sysvideo_15_6:
                             <1>     ; 29/01/2021
0000D72E A3[929D0100]        <1>     mov    [v_mem], eax ; save video memory address
                             <1>
                             <1>     ; 27/12/2020
                             <1>     ; 26/12/2020
0000D733 8B4502              <1>     mov    eax, [ebp+2] ; width, height
                             <1>     ; 29/01/2021
0000D736 66A3[8E9D0100]      <1>     mov    [v_width], ax ; save screen width
                             <1>     ; 28/12/2020
0000D73C 8A7D06              <1>     mov    bh, [ebp+6] ; bpp
                             <1>     ; 28/02/2021
                             <1>     ; check default truecolor bpp value and use
                             <1>     ; 32bpp instead of 24bpp if the default value
                             <1>     ; has been set to 32bpp.
0000D73F 80FF18              <1>     cmp    bh, 24
0000D742 750B                <1>     jne    short sysvideo_15_16
0000D744 803D[73730100]20    <1>     cmp    byte [truecolor], 32
                             <1>            ; Default truecolor bpp value,
                             <1>            ; it is 32 for VBE3 video bios
                             <1>            ; (it can be set to 32 or 24)
0000D74B 7502                <1>     jne    short sysvideo_15_16 ; not VBE3 !
                             <1>                            ; or it is set to 24
0000D74D B720                <1>     mov    bh, 32
                             <1>     ; 28/02/2021
                             <1> sysvideo_15_16:
                             <1>     ; 29/01/2021
0000D74F 883D[919D0100]      <1>     mov    [v_bpp], bh ; bits per pixel
                             <1>
```

```
0000D755 52                 <1>     push    edx ; *
0000D756 0FB7D0             <1>     movzx   edx, ax  ; width
0000D759 C1E810             <1>     shr     eax, 16  ; height
0000D75C F7E2               <1>     mul     edx
                            <1>     ; eax = linear frame buffer size (pixels)
                            <1>     ; 29/01/2021
0000D75E A3[969D0100]       <1>     mov     [v_siz], eax ; save video page size
0000D763 E8FD000000         <1>     call    pixels_to_byte_count
0000D768 0305[929D0100]     <1>     add     eax, [v_mem]
0000D76E A3[9E9D0100]       <1>     mov     [v_end], eax ; save end of video page
0000D773 5A                 <1>     pop     edx ; *
                            <1>
                            <1>     ; bh = bits per pixel
                            <1>     ; (bh will not be used after here, 29/01/2021)
                            <1>
                            <1>     ; bl = pixel operations & options
                            <1>     ; ecx, edx, esi, edi input parameters
                            <1>     ; [maskcolor] = edi input ; 25/02/2021
                            <1>
                            <1> sysvideo_15_7:
                            <1>     ; 29/01/2021
                            <1>     ;test   byte [v_ops], 40h  ; system to user ?
0000D774 F6C340             <1>     test    bl, 40h
0000D777 7517               <1>     jnz     short sysvideo_15_9
                            <1>
0000D779 31C0               <1>     xor     eax, eax
0000D77B 88D8               <1>     mov     al, bl
0000D77D BB[A3D80000]       <1>     mov     ebx, pixel_ops
0000D782 240F               <1>     and     al, 0Fh ; isolate 16 pixel operations
0000D784 C0E002             <1>     shl     al, 2 ; * 4 for dword table pointers
0000D787 01C3               <1>     add     ebx, eax
                            <1>
                            <1>     ; ebx = subroutine address
                            <1>
                            <1>     ; ecx, edx, esi, edi input parameters
                            <1>     ; [maskbuff] = edi input
                            <1>     ; [maskcolor] = edi input ; 25/02/2021
                            <1>
0000D789 FF13               <1>     call    [ebx]
                            <1> sysvideo_15_8:
0000D78B E9FEF2FFFF         <1>     jmp     sysret
                            <1>
                            <1> sysvideo_15_9:
                            <1>     ; system to user display page or window copy
                            <1>     ;test   byte [v_ops], 1 ; window copy ?
0000D790 F6C301             <1>     test    bl, 1
0000D793 7521               <1>     jnz     short sysvideo_15_10
                            <1>
                            <1>     ; display page (full screen copy)
0000D795 8B35[929D0100]     <1>     mov     esi, [v_mem] ; LFB start address
0000D79B A1[969D0100]       <1>     mov     eax, [v_siz]
0000D7A0 E8C0000000         <1>     call    pixels_to_byte_count
0000D7A5 89C1               <1>     mov     ecx, eax ; transfer count in bytes
                            <1>     ;edi = user's buffer address
0000D7A7 E86F330000         <1>     call    transfer_to_user_buffer
0000D7AC 72DD               <1>     jc      short sysvideo_15_8
0000D7AE 890D[6C8E0100]     <1>     mov     [u.r0], ecx
0000D7B4 EBD5               <1>     jmp     short sysvideo_15_8
                            <1>
                            <1> sysvideo_15_10:
0000D7B6 E820000000         <1>     call    sysvideo_15_12 ; window preparations
0000D7BB 72CE               <1>     jc      short sysvideo_15_8
                            <1>
0000D7BD 8B35[9A9D0100]     <1>     mov     esi, [v_str]
                            <1> sysvideo_15_11:
                            <1>     ; esi = window's current row address (video mem)
                            <1>     ; edi = current row (virtual) addr in user's buff
                            <1>     ; ecx = transfer count per row
0000D7C3 E853330000         <1>     call    transfer_to_user_buffer
0000D7C8 72C1               <1>     jc      short sysvideo_15_8
0000D7CA 010D[6C8E0100]     <1>     add     [u.r0], ecx
0000D7D0 4B                 <1>     dec     ebx
0000D7D1 74B8               <1>     jz      short sysvideo_15_8 ; ok.
                            <1>     ; next row
0000D7D3 01CF               <1>     add     edi, ecx ; next row in user's buffer
0000D7D5 01D6               <1>     add     esi, edx ; next row of window (system)
0000D7D7 EBEA               <1>     jmp     short sysvideo_15_11
                            <1>
                            <1> sysvideo_15_14:
0000D7D9 F9                 <1>     stc      ; error !
                            <1> sysvideo_15_15:
0000D7DA C3                 <1>     retn
                            <1>
                            <1> sysvideo_15_12:
                            <1>     ; 30/01/2021
                            <1>     ; 29/01/2021
                            <1>     ; window address preparations for window copy
0000D7DB 6621D2             <1>     and     dx, dx
0000D7DE 74F9               <1>     jz      short sysvideo_15_14 ; invalid (zero columns)
                            <1>     ;test   edx, 0FFFF0000h
                            <1>     ;jz     short sysvideo_15_14 ; invalid (zero rows)
0000D7E0 81FA00000100       <1>     cmp     edx, 65536
0000D7E6 72F2               <1>     jb      short sysvideo_15_15 ; invalid (zero rows)
0000D7E8 89C8               <1>     mov     eax, ecx ; start position (row, column)
0000D7EA E899000000         <1>     call    calc_pixel_offset
0000D7EF 3B05[969D0100]     <1>     cmp     eax, [v_siz]
0000D7F5 73E2               <1>     jnb     short sysvideo_15_14 ; out of display page
                            <1>                             ; nothing to do
0000D7F7 E869000000         <1>     call    pixels_to_byte_count
0000D7FC 0305[929D0100]     <1>     add     eax, [v_mem]
0000D802 A3[9A9D0100]       <1>     mov     [v_str], eax ; window start address
                            <1>                           ; (addr of top left corner)
                            <1>     ; check column limit
0000D807 89C8               <1>     mov     eax, ecx
0000D809 6601D0             <1>     add     ax, dx  ; add columns to start column
0000D80C 72CC               <1>     jc      short sysvideo_15_15 ; cf = 1
0000D80E 663B05[8E9D0100]   <1>     cmp     ax, [v_width]
0000D815 77C2               <1>     ja      short sysvideo_15_14
                            <1>
0000D817 89D0               <1>     mov     eax, edx ; size
0000D819 2D00000100         <1>     sub     eax, 65536 ; row count -> 0 based row #
0000D81E E865000000         <1>     call    calc_pixel_offset
0000D823 3B05[969D0100]     <1>     cmp     eax, [v_siz] ; video (display) page size
0000D829 77AE               <1>     ja      short sysvideo_15_14 ; out of display page
                            <1>                             ; nothing to do
0000D82B E835000000         <1>     call    pixels_to_byte_count
0000D830 0305[9A9D0100]     <1>     add     eax, [v_str] ; window start address
0000D836 3B05[9E9D0100]     <1>     cmp     eax, [v_end] ; window end address (+1)
                            <1>                     ; (addr of bottom right corner +1)
0000D83C 779B               <1>     ja      short sysvideo_15_14 ; out of display page
                            <1>                             ; nothing to do
0000D83E 89D3               <1>     mov     ebx, edx
0000D840 C1EB10             <1>     shr     ebx, 16
                            <1>     ; ebx = row count
```

```
0000D843 81E2FFFF0000        <1>     and     edx, 0FFFFh
                             <1>     ; edx = transfer count per row (from user's buffer)
                             <1>     ;       (in pixels, window width)
0000D849 89D0                <1>     mov     eax, edx
0000D84B A3[A69D0100]        <1>     mov     [pixcount], eax ; 27/02/2021
0000D850 E810000000          <1>     call    pixels_to_byte_count
0000D855 89C1                <1>     mov     ecx, eax
                             <1>     ; ecx = transfer count per row (from user's buffer)
                             <1>     ;       (in bytes, window width)
0000D857 66A1[8E9D0100]      <1>     mov     ax, [v_width]
0000D85D E803000000          <1>     call    pixels_to_byte_count
0000D862 89C2                <1>     mov     edx, eax
                             <1>     ; edx = byte count per row
0000D864 C3                  <1>     retn ; cf = 0
                             <1>
                             <1> pixels_to_byte_count:
                             <1>     ; 29/01/2021
                             <1>     ; INPUT:
                             <1>     ;   eax = pixel count
                             <1>     ; OUTPUT:
                             <1>     ;   eax = byte count
                             <1>     ;
0000D865 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8
0000D86C 7619                <1>     jna     short pixtobc_3 ; 8 bit colors
0000D86E 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24
0000D875 720A                <1>     jb      short pixtobc_1 ; 16 bit colors
0000D877 770B                <1>     ja      short pixtobc_2 ; 32 bit colors
                             <1>     ; 24 bit pixels
                             <1>     ; eax = eax * 3
                             <1>     ;push   edx
                             <1>     ;mov    edx, eax
                             <1>     ;shl    eax, 1
                             <1>     ;add    eax, edx
                             <1>     ;pop    edx
0000D879 50                  <1>     push    eax
0000D87A D1E0                <1>     shl     eax, 1
0000D87C 010424              <1>     add     [esp], eax
0000D87F 58                  <1>     pop     eax
0000D880 C3                  <1>     retn
                             <1> pixtobc_1:
                             <1>     ; 32 bit pixels
                             <1>     ; eax = eax * 2
0000D881 D1E0                <1>     shl     eax, 1
0000D883 C3                  <1>     retn
                             <1> pixtobc_2:
                             <1>     ; 16 bit pixels
                             <1>     ; eax = eax * 4
0000D884 C1E002              <1>     shl     eax, 2
                             <1> pixtobc_3:
0000D887 C3                  <1>     retn
                             <1>
                             <1> calc_pixel_offset:
                             <1>     ; 29/01/2021
                             <1>     ; INPUT:
                             <1>     ;   eax = pixel position (row, column)
                             <1>     ; OUTPUT:
                             <1>     ;   eax = pixel offset (linear address)
                             <1>     ;
0000D888 52                  <1>     push    edx
0000D889 50                  <1>     push    eax
0000D88A C1E810              <1>     shr     eax, 16
0000D88D 7409                <1>     jz      short cpixo_0
                             <1>     ; eax = row
0000D88F 0FB715[8E9D0100]    <1>     movzx   edx, word [v_width]
0000D896 F7E2                <1>     mul     edx
                             <1> cpixo_0:
                             <1>     ; eax = row * screen width
0000D898 5A                  <1>     pop     edx
0000D899 81E2FFFF0000        <1>     and     edx, 0FFFFh
                             <1>     ; edx = column
0000D89F 01D0                <1>     add     eax, edx
                             <1>     ; eax = (row * screen width) + column
0000D8A1 5A                  <1>     pop     edx
0000D8A2 C3                  <1>     retn
                             <1>
                             <1>     ; 02/02/2021
                             <1>     ; 29/01/2021
                             <1> pixel_ops:
0000D8A3 [E3D80000]          <1>     dd      pix_op_cpy ; copy pixels (user to system)
0000D8A7 [2EDE0000]          <1>     dd      pix_op_new ; change (new, fill) color
0000D8AB [4BD90000]          <1>     dd      pix_op_add ; add color (up to 0FFh)
0000D8AF [FDD90000]          <1>     dd      pix_op_sub ; sub color (down to 0)
0000D8B3 [18DC0000]          <1>     dd      pix_op_orc ; or color
0000D8B7 [CADC0000]          <1>     dd      pix_op_and ; and color
0000D8BB [7CDD0000]          <1>     dd      pix_op_xor ; xor color
0000D8BF [F2DE0000]          <1>     dd      pix_op_not ; not color
0000D8C3 [9CDF0000]          <1>     dd      pix_op_neg ; neg color
0000D8C7 [46E00000]          <1>     dd      pix_op_inc ; inc color
0000D8CB [F0E00000]          <1>     dd      pix_op_dec ; dec color
0000D8CF [AFDA0000]          <1>     dd      pix_op_mix ; mix color
0000D8D3 [76DB0000]          <1>     dd      pix_op_rpl ; replace color
0000D8D7 [9AE10000]          <1>     dd      pix_op_blk ; copy pixel block(s) (sys)
0000D8DB [48E20000]          <1>     dd      pix_op_lin ; write line(s)
0000D8DF [2BE60000]          <1>     dd      pix_op_chr ; write character (font)
                             <1>
                             <1> pix_op_cpy:
                             <1>     ; 21/02/2021
                             <1>     ; 06/02/2021
                             <1>     ; 30/01/2021
                             <1>     ; COPY PIXELS
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;  If bit 4 of BL or [v_ops] = 1 -window copy-
                             <1>     ;  ECX = start position (row, column)
                             <1>     ;        (HW = row, CX = column)
                             <1>     ;  EDX = size (rows, colums)
                             <1>     ;        (HW = rows, DX = columns)
                             <1>     ;       (0 -> invalid
                             <1>     ;        (1 -> horizontal or vertical line)
                             <1>     ;  If bit 4 of BL or [v_ops] = 0 -full screen-
                             <1>     ;     ECX and EDX will not be used
                             <1>     ;  ESI = user's buffer address
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
0000D8E3 F605[909D0100]10    <1>     test    byte [v_ops], 10h ; display page or window ?
0000D8EA 752E                <1>     jnz     short pix_op_cpy_w ; window
                             <1>
0000D8EC 8B3D[929D0100]      <1>     mov     edi, [v_mem] ; 21/02/2021
                             <1>
                             <1>     ; Copy user's buffer content do display page
                             <1>     ; (full screen copy)
```

```
0000D8F2 A1[969D0100]        <1>     mov     eax, [v_siz] ; video page size
0000D8F7 E869FFFFFF          <1>     call    pixels_to_byte_count
0000D8FC 89C1                <1>     mov     ecx, eax ; transfer count
                             <1>     ; esi = user's buffer address (virtual)
0000D8FE F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked copy ?
0000D905 7405                <1>     jz      short pix_op_cpy_0 ; no
0000D907 E96F0F0000          <1>     jmp     m_pix_op_cpy ; copy pixels except mask color
                             <1> pix_op_cpy_0:
                             <1>     ; esi = user buffer for full screen copy
                             <1>     ; edi = start of video memory
                             <1>     ;       (start of display page)
                             <1>     ; ecx = byte count (display page size in bytes)
0000D90C E854320000          <1>     call    transfer_from_user_buffer
0000D911 7206                <1>     jc      short pix_op_cpy_1
0000D913 890D[6C8E0100]      <1>     mov     [u.r0], ecx
                             <1> pix_op_cpy_1:
0000D919 C3                  <1>     retn    ; 06/02/2021
                             <1>
                             <1> pix_op_cpy_w:
0000D91A E8BCFEFFFF          <1>     call    sysvideo_15_12 ; window preparations
0000D91F 72F8                <1>     jc      short pix_op_cpy_1
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
0000D921 8B3D[9A9D0100]      <1>     mov     edi, [v_str]
0000D927 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked copy ?
0000D92E 7405                <1>     jz      short pix_op_cpy_w_0 ; no
0000D930 E909100000          <1>     jmp     m_pix_op_cpy_w ; window copy except mask color
                             <1> pix_op_cpy_w_0:
                             <1>     ; esi = current row (virtual) addr in user's buff
                             <1>     ; edi = window's current row address (video mem)
                             <1>     ; ecx = transfer count per row
0000D935 E82B320000          <1>     call    transfer_from_user_buffer
0000D93A 72DD                <1>     jc      short pix_op_cpy_1
0000D93C 010D[6C8E0100]      <1>     add     [u.r0], ecx
0000D942 4B                  <1>     dec     ebx
0000D943 74D4                <1>     jz      short pix_op_cpy_1 ; ok.
                             <1>     ; next row
0000D945 01CE                <1>     add     esi, ecx ; next row in user's buffer
0000D947 01D7                <1>     add     edi, edx ; next row of window (system)
0000D949 EBEA                <1>     jmp     short pix_op_cpy_w_0
                             <1>
                             <1> pix_op_add:
                             <1>     ; 31/01/2021
                             <1>     ; 30/01/2021
                             <1>     ; ADD COLOR
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   CL = color (8 bit, 256 colors)
                             <1>     ;  ECX = color (16 bit and true colors)
                             <1>     ;  EDX = start position (row, column)
                             <1>     ;        (HW = row, DX = column)
                             <1>     ;  ESI = size (rows, colums)
                             <1>     ;        (HW = rows, SI = columns)
                             <1>     ;
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
0000D94B F605[909D0100]10    <1>     test    byte [v_ops], 10h ; display page or window ?
0000D952 7555                <1>     jnz     short pix_op_add_w ; window
                             <1>
0000D954 8B3D[929D0100]      <1>     mov     edi, [v_mem]
0000D95A 89FE                <1>     mov     esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000D95C 89C8                <1>     mov     eax, ecx
0000D95E 8B0D[969D0100]      <1>     mov     ecx, [v_siz] ; display page pixel count
                             <1>
0000D964 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color adding ?
0000D96B 7405                <1>     jz      short pix_op_add_0 ; no
0000D96D E9CB100000          <1>     jmp     m_pix_op_add ; add color except mask color
                             <1> pix_op_add_0:
0000D972 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000D979 7707                <1>     ja      short pix_op_add_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000D97B E84C0A0000          <1>     call    pix_op_add_8
0000D980 EB1E                <1>     jmp     short pix_op_add_4
                             <1>
                             <1> pix_op_add_1:
0000D982 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000D989 7710                <1>     ja      short pix_op_add_3 ; 32bpp
0000D98B 7207                <1>     jb      short pix_op_add_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000D98D E85A0A0000          <1>     call    pix_op_add_24
0000D992 EB0C                <1>     jmp     short pix_op_add_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_add_2:
0000D994 E8410A0000          <1>     call    pix_op_add_16
0000D999 EB05                <1>     jmp     short pix_op_add_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_add_3:
0000D99B E86C0A0000          <1>     call    pix_op_add_32
                             <1> pix_op_add_4:
0000D9A0 29F7                <1>     sub     edi, esi
0000D9A2 893D[6C8E0100]      <1>     mov     [u.r0], edi
                             <1> pix_op_add_5:
0000D9A8 C3                  <1>     retn
                             <1>
                             <1> pix_op_add_w:
                             <1>     ; 31/01/2021
0000D9A9 51                  <1>     push    ecx ; * ; color
0000D9AA 89D1                <1>     mov     ecx, edx ; win start pos
0000D9AC 89F2                <1>     mov     edx, esi ; size (rows, cols)
0000D9AE E828FEFFFF          <1>     call    sysvideo_15_12 ; window preparations
0000D9B3 58                  <1>     pop     eax ; * ; color
0000D9B4 72F2                <1>     jc      short pix_op_add_5
                             <1>
0000D9B6 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color adding ?
0000D9BD 7405                <1>     jz      short pix_op_add_w_0 ; no
0000D9BF E927110000          <1>     jmp     m_pix_op_add_w
                             <1>                     ; window add color except mask color
                             <1> pix_op_add_w_0:
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>
0000D9C4 8B3D[9A9D0100]      <1>     mov     edi, [v_str]
0000D9CA 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
```

```
0000D9D1 7707                <1>     ja      short pix_op_add_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000D9D3 BD[CCE30000]        <1>     mov     ebp, pix_op_add_8
0000D9D8 EB1E                <1>     jmp     short pix_op_add_w_4
                             <1>
                             <1> pix_op_add_w_1:
0000D9DA 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000D9E1 7710                <1>     ja      short pix_op_add_w_3 ; 32bpp
0000D9E3 7207                <1>     jb      short pix_op_add_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000D9E5 BD[ECE30000]        <1>     mov     ebp, pix_op_add_24
0000D9EA EB0C                <1>     jmp     short pix_op_add_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_add_w_2:
0000D9EC BD[DAE30000]        <1>     mov     ebp, pix_op_add_16
0000D9F1 EB05                <1>     jmp     short pix_op_add_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_add_w_3:
0000D9F3 BD[0CE40000]        <1>     mov     ebp, pix_op_add_32
                             <1> pix_op_add_w_4:
0000D9F8 E95F010000          <1>     jmp     pix_op_add_w_x
                             <1>
                             <1> pix_op_sub:
                             <1>     ; 31/01/2021
                             <1>     ; SUB COLOR
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   CL = color (8 bit, 256 colors)
                             <1>     ;  ECX = color (16 bit and true colors)
                             <1>     ;  EDX = start position (row, column)
                             <1>     ;        (HW = row, DX = column)
                             <1>     ;  ESI = size (rows, colums)
                             <1>     ;        (HW = rows, SI = columns)
                             <1>     ;
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
0000D9FD F605[909D0100]10    <1>     test    byte [v_ops], 10h ; display page or window ?
0000DA04 7555                <1>     jnz     short pix_op_sub_w ; window
                             <1>
0000DA06 8B3D[929D0100]      <1>     mov     edi, [v_mem]
0000DA0C 89FE                <1>     mov     esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000DA0E 89C8                <1>     mov     eax, ecx
0000DA10 8B0D[969D0100]      <1>     mov     ecx, [v_siz] ; display page pixel count
                             <1>
0000DA16 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color subtract ?
0000DA1D 7405                <1>     jz      short pix_op_sub_0 ; no
0000DA1F E9FA100000          <1>     jmp     m_pix_op_sub ; sub color except mask color
                             <1> pix_op_sub_0:
0000DA24 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DA2B 7707                <1>     ja      short pix_op_sub_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DA2D E8E9090000          <1>     call    pix_op_sub_8
0000DA32 EB1E                <1>     jmp     short pix_op_sub_4
                             <1>
                             <1> pix_op_sub_1:
0000DA34 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DA3B 7710                <1>     ja      short pix_op_sub_3 ; 32bpp
0000DA3D 7207                <1>     jb      short pix_op_sub_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DA3F E8FA090000          <1>     call    pix_op_sub_24
0000DA44 EB0C                <1>     jmp     short pix_op_sub_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_sub_2:
0000DA46 E8E0090000          <1>     call    pix_op_sub_16
0000DA4B EB05                <1>     jmp     short pix_op_sub_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_sub_3:
0000DA4D E8060A0000          <1>     call    pix_op_sub_32
                             <1> pix_op_sub_4:
0000DA52 29F7                <1>     sub     edi, esi
0000DA54 893D[6C8E0100]      <1>     mov     [u.r0], edi
                             <1> pix_op_sub_5:
0000DA5A C3                  <1>     retn
                             <1>
                             <1> pix_op_sub_w:
                             <1>     ; 31/01/2021
0000DA5B 51                  <1>     push    ecx ; * ; color
0000DA5C 89D1                <1>     mov     ecx, edx ; win start pos
0000DA5E 89F2                <1>     mov     edx, esi ; size (rows, cols)
0000DA60 E876FDFFFF          <1>     call    sysvideo_15_12 ; window preparations
0000DA65 58                  <1>     pop     eax ; * ; color
0000DA66 72F2                <1>     jc      short pix_op_sub_5
                             <1>
0000DA68 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color subtract ?
0000DA6F 7405                <1>     jz      short pix_op_sub_w_0 ; no
0000DA71 E94B110000          <1>     jmp     m_pix_op_sub_w
                             <1>                     ; window sub color except mask color
                             <1> pix_op_sub_w_0:
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>
0000DA76 8B3D[9A9D0100]      <1>     mov     edi, [v_str]
0000DA7C 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DA83 7707                <1>     ja      short pix_op_sub_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DA85 BD[1BE40000]        <1>     mov     ebp, pix_op_sub_8
0000DA8A EB1E                <1>     jmp     short pix_op_sub_w_4
                             <1>
                             <1> pix_op_sub_w_1:
0000DA8C 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DA93 7710                <1>     ja      short pix_op_sub_w_3 ; 32bpp
0000DA95 7207                <1>     jb      short pix_op_sub_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DA97 BD[3EE40000]        <1>     mov     ebp, pix_op_sub_24
0000DA9C EB0C                <1>     jmp     short pix_op_sub_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_sub_w_2:
```

```
0000DA9E BD[2BE40000]        <1>     mov     ebp, pix_op_sub_16
0000DAA3 EB05                <1>     jmp     short pix_op_sub_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_sub_w_3:
0000DAA5 BD[58E40000]        <1>     mov     ebp, pix_op_sub_32
                             <1> pix_op_sub_w_4:
0000DAAA E9AD000000          <1>     jmp     pix_op_sub_w_x
                             <1>
                             <1> pix_op_mix:
                             <1>     ; 31/01/2021
                             <1>     ; MIX COLOR
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   CL = color (8 bit, 256 colors)
                             <1>     ;  ECX = color (16 bit and true colors)
                             <1>     ;  EDX = start position (row, column)
                             <1>     ;        (HW = row, DX = column)
                             <1>     ;  ESI = size (rows, colums)
                             <1>     ;        (HW = rows, SI = columns)
                             <1>     ;
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
0000DAAF F605[909D0100]10    <1>     test    byte [v_ops], 10h ; display page or window ?
0000DAB6 7555                <1>     jnz     short pix_op_mix_w ; window
                             <1>
0000DAB8 8B3D[929D0100]      <1>     mov     edi, [v_mem]
0000DABE 89FE                <1>     mov     esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000DAC0 89C8                <1>     mov     eax, ecx
0000DAC2 8B0D[969D0100]      <1>     mov     ecx, [v_siz] ; display page pixel count
                             <1>
0000DAC8 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color mix ?
0000DACF 7405                <1>     jz      short pix_op_mix_0 ; no
0000DAD1 E91E110000          <1>     jmp     m_pix_op_mix ; mix colors except mask color
                             <1> pix_op_mix_0:
0000DAD6 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DADD 7707                <1>     ja      short pix_op_mix_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DADF E8F3090000          <1>     call    pix_op_mix_8
0000DAE4 EB1E                <1>     jmp     short pix_op_mix_4
                             <1>
                             <1> pix_op_mix_1:
0000DAE6 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DAED 7710                <1>     ja      short pix_op_mix_3 ; 32bpp
0000DAEF 7207                <1>     jb      short pix_op_mix_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DAF1 E8FC090000          <1>     call    pix_op_mix_24
0000DAF6 EB0C                <1>     jmp     short pix_op_mix_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_mix_2:
0000DAF8 E8E6090000          <1>     call    pix_op_mix_16
0000DAFD EB05                <1>     jmp     short pix_op_mix_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_mix_3:
0000DAFF E80A0A0000          <1>     call    pix_op_mix_32
                             <1> pix_op_mix_4:
0000DB04 29F7                <1>     sub     edi, esi
0000DB06 893D[6C8E0100]      <1>     mov     [u.r0], edi
                             <1> pix_op_mix_5:
0000DB0C C3                  <1>     retn
                             <1>
                             <1> pix_op_mix_w:
                             <1>     ; 31/01/2021
0000DB0D 51                  <1>     push    ecx ; * ; color
0000DB0E 89D1                <1>     mov     ecx, edx ; win start pos
0000DB10 89F2                <1>     mov     edx, esi ; size (rows, cols)
0000DB12 E8C4FCFFFF          <1>     call    sysvideo_15_12 ; window preparations
0000DB17 58                  <1>     pop     eax ; * ; color
0000DB18 72F2                <1>     jc      short pix_op_mix_5
                             <1>
0000DB1A F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color mix ?
0000DB21 7405                <1>     jz      short pix_op_mix_w_0 ; no
0000DB23 E969110000          <1>     jmp     m_pix_op_mix_w
                             <1>                     ; window mix colors except mask color
                             <1> pix_op_mix_w_0:
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>
0000DB28 8B3D[9A9D0100]      <1>     mov     edi, [v_str]
0000DB2E 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DB35 7707                <1>     ja      short pix_op_mix_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DB37 BD[D7E40000]        <1>     mov     ebp, pix_op_mix_8
0000DB3C EB1E                <1>     jmp     short pix_op_mix_w_x
                             <1>
                             <1> pix_op_mix_w_1:
0000DB3E 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DB45 7710                <1>     ja      short pix_op_mix_w_3 ; 32bpp
0000DB47 7207                <1>     jb      short pix_op_mix_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DB49 BD[F2E40000]        <1>     mov     ebp, pix_op_mix_24
0000DB4E EB0C                <1>     jmp     short pix_op_mix_w_x
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_mix_w_2:
0000DB50 BD[E3E40000]        <1>     mov     ebp, pix_op_mix_16
0000DB55 EB05                <1>     jmp     short pix_op_mix_w_x
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_mix_w_3:
0000DB57 BD[0EE50000]        <1>     mov     ebp, pix_op_mix_32
                             <1>     ;jmp    short pix_op_mix_w_x
                             <1>
                             <1> pix_op_mix_w_x:
                             <1> pix_op_add_w_x:
                             <1> pix_op_sub_w_x:
                             <1> pix_op_rpl_w_x:
                             <1> pix_op_orc_w_x:
                             <1> pix_op_and_w_x:
                             <1> pix_op_xor_w_x:
                             <1>     ; 27/02/2021
                             <1>     ; 31/01/2021
```

```
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = windows (screen) width in bytes
                                <1>     ; ebx = row count
                                <1>     ; eax = color
                                <1>     ; ebp = pixel operation subroutine address
0000DB5C 52                     <1>     push    edx
0000DB5D 51                     <1>     push    ecx
0000DB5E 57                     <1>     push    edi
0000DB5F 8B0D[A69D0100]         <1>     mov     ecx, [pixcount] ; 27/02/2021
0000DB65 FFD5                   <1>     call    ebp ; call pixel-row operation
0000DB67 5F                     <1>     pop     edi
0000DB68 59                     <1>     pop     ecx ; bytes per row
0000DB69 010D[6C8E0100]         <1>     add     [u.r0], ecx
0000DB6F 5A                     <1>     pop     edx
0000DB70 01D7                   <1>     add     edi, edx ; next row
0000DB72 4B                     <1>     dec     ebx
0000DB73 75E7                   <1>     jnz     short pix_op_mix_w_x
0000DB75 C3                     <1>     retn
                                <1>
                                <1> pix_op_rpl:
                                <1>     ; 01/02/2021
                                <1>     ; REPLACE COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   CL = old/current color (8 bit, 256 colors)
                                <1>     ;  ECX = old/current color (16 bit and true colors)
                                <1>     ;   DL = new color (8 bit, 256 colors)
                                <1>     ;  EDX = new color (16 bit and true colors)
                                <1>     ;  ESI = start position (row, column)
                                <1>     ;        (HW = row, DX = column)
                                <1>     ;  EDI = size (rows, colums)
                                <1>     ;        (HW = rows, SI = columns)
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000DB76 F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000DB7D 754D                   <1>     jnz     short pix_op_rpl_w ; window
                                <1>
0000DB7F 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000DB85 89FE                   <1>     mov     esi, edi
                                <1>     ; ecx = old color (CL, CX, ECX) -to be replaced with-
                                <1>     ; edx = new color (CL, CX, ECX) -new one-
0000DB87 89D0                   <1>     mov     eax, edx ; new color
0000DB89 890D[A29D0100]         <1>     mov     [maskcolor], ecx ; old color
0000DB8F 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1> pix_op_rpl_0:
0000DB95 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DB9C 7707                   <1>     ja      short pix_op_rpl_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DB9E E82F0A0000             <1>     call    pix_op_rpl_8
0000DBA3 EB1E                   <1>     jmp     short pix_op_rpl_4
                                <1>
                                <1> pix_op_rpl_1:
0000DBA5 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DBAC 7710                   <1>     ja      short pix_op_rpl_3 ; 32bpp
0000DBAE 7207                   <1>     jb      short pix_op_rpl_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DBB0 E8400A0000             <1>     call    pix_op_rpl_24
0000DBB5 EB0C                   <1>     jmp     short pix_op_rpl_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_rpl_2:
0000DBB7 E8260A0000             <1>     call    pix_op_rpl_16
0000DBBC EB05                   <1>     jmp     short pix_op_rpl_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_rpl_3:
0000DBBE E8540A0000             <1>     call    pix_op_rpl_32
                                <1> pix_op_rpl_4:
0000DBC3 29F7                   <1>     sub     edi, esi
0000DBC5 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_rpl_5:
0000DBCB C3                     <1>     retn
                                <1>
                                <1> pix_op_rpl_w:
                                <1>     ; 01/02/2021
0000DBCC 890D[A29D0100]         <1>     mov     [maskcolor], ecx ; old color
0000DBD2 52                     <1>     push    edx ; * ; new color
0000DBD3 89F1                   <1>     mov     ecx, esi ; win start pos
0000DBD5 89FA                   <1>     mov     edx, edi ; size (rows, cols)
0000DBD7 E8FFFBFFFF             <1>     call    sysvideo_15_12 ; window preparations
0000DBDC 58                     <1>     pop     eax ; * ; new color
0000DBDD 72EC                   <1>     jc      short pix_op_rpl_5
                                <1>
                                <1>     ; replace window color
                                <1> pix_op_rpl_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>     ; eax = new color
                                <1>     ; [maskcolor] = old color
                                <1>
0000DBDF 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000DBE5 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DBEC 7707                   <1>     ja      short pix_op_rpl_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DBEE BD[D2E50000]           <1>     mov     ebp, pix_op_rpl_8
0000DBF3 EB1E                   <1>     jmp     short pix_op_rpl_w_4
                                <1>
                                <1> pix_op_rpl_w_1:
0000DBF5 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DBFC 7710                   <1>     ja      short pix_op_rpl_w_3 ; 32bpp
0000DBFE 7207                   <1>     jb      short pix_op_rpl_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DC00 BD[F5E50000]           <1>     mov     ebp, pix_op_rpl_24
0000DC05 EB0C                   <1>     jmp     short pix_op_rpl_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_rpl_w_2:
0000DC07 BD[E2E50000]           <1>     mov     ebp, pix_op_rpl_16
0000DC0C EB05                   <1>     jmp     short pix_op_rpl_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_rpl_w_3:
0000DC0E BD[17E60000]           <1>     mov     ebp, pix_op_rpl_32
                                <1> pix_op_rpl_w_4:
0000DC13 E944FFFFFF             <1>     jmp     pix_op_rpl_w_x
                                <1>
                                <1> pix_op_orc:
```

```
                                <1>     ; 31/01/2021
                                <1>     ; OR COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   CL = color (8 bit, 256 colors)
                                <1>     ;  ECX = color (16 bit and true colors)
                                <1>     ;  EDX = start position (row, column)
                                <1>     ;        (HW = row, DX = column)
                                <1>     ;  ESI = size (rows, colums)
                                <1>     ;        (HW = rows, SI = columns)
                                <1>     ;
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000DC18 F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000DC1F 7555                   <1>     jnz     short pix_op_or_w ; window
                                <1>
0000DC21 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000DC27 89FE                   <1>     mov     esi, edi
                                <1>     ; ecx = color (CL, CX, ECX)
0000DC29 89C8                   <1>     mov     eax, ecx
0000DC2B 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1>
0000DC31 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked color 'or' ?
0000DC38 7405                   <1>     jz      short pix_op_or_0 ; no
0000DC3A E945110000             <1>     jmp     m_pix_op_or ; 'or' color except mask color
                                <1> pix_op_or_0:
0000DC3F 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DC46 7707                   <1>     ja      short pix_op_or_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DC48 E81B080000             <1>     call    pix_op_or_8
0000DC4D EB1E                   <1>     jmp     short pix_op_or_4
                                <1>
                                <1> pix_op_or_1:
0000DC4F 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DC56 7710                   <1>     ja      short pix_op_or_3 ; 32bpp
0000DC58 7207                   <1>     jb      short pix_op_or_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DC5A E817080000             <1>     call    pix_op_or_24
0000DC5F EB0C                   <1>     jmp     short pix_op_or_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_or_2:
0000DC61 E808080000             <1>     call    pix_op_or_16
0000DC66 EB05                   <1>     jmp     short pix_op_or_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_or_3:
0000DC68 E818080000             <1>     call    pix_op_or_32
                                <1> pix_op_or_4:
0000DC6D 29F7                   <1>     sub     edi, esi
0000DC6F 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_or_5:
0000DC75 C3                     <1>     retn
                                <1>
                                <1> pix_op_or_w:
                                <1>     ; 31/01/2021
0000DC76 51                     <1>     push    ecx ; * ; color
0000DC77 89D1                   <1>     mov     ecx, edx ; win start pos
0000DC79 89F2                   <1>     mov     edx, esi ; size (rows, cols)
0000DC7B E85BFBFFFF             <1>     call    sysvideo_15_12 ; window preparations
0000DC80 58                     <1>     pop     eax ; * ; color
0000DC81 72F2                   <1>     jc      short pix_op_or_5
                                <1>
0000DC83 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked color 'or' ?
0000DC8A 7405                   <1>     jz      short pix_op_or_w_0 ; no
0000DC8C E980110000             <1>     jmp     m_pix_op_or_w
                                <1>                     ; window 'or' color except mask color
                                <1> pix_op_or_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>     ; eax = color
                                <1>
0000DC91 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
0000DC97 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DC9E 7707                   <1>     ja      short pix_op_or_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DCA0 BD[68E40000]           <1>     mov     ebp, pix_op_or_8
0000DCA5 EB1E                   <1>     jmp     short pix_op_or_w_4
                                <1>
                                <1> pix_op_or_w_1:
0000DCA7 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DCAE 7710                   <1>     ja      short pix_op_or_w_3 ; 32bpp
0000DCB0 7207                   <1>     jb      short pix_op_or_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DCB2 BD[76E40000]           <1>     mov     ebp, pix_op_or_24
0000DCB7 EB0C                   <1>     jmp     short pix_op_or_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_or_w_2:
0000DCB9 BD[6EE40000]           <1>     mov     ebp, pix_op_or_16
0000DCBE EB05                   <1>     jmp     short pix_op_or_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_or_w_3:
0000DCC0 BD[85E40000]           <1>     mov     ebp, pix_op_or_32
                                <1> pix_op_or_w_4:
0000DCC5 E992FEFFFF             <1>     jmp     pix_op_orc_w_x
                                <1>
                                <1> pix_op_and:
                                <1>     ; 31/01/2021
                                <1>     ; AND COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   CL = color (8 bit, 256 colors)
                                <1>     ;  ECX = color (16 bit and true colors)
                                <1>     ;  EDX = start position (row, column)
                                <1>     ;        (HW = row, DX = column)
                                <1>     ;  ESI = size (rows, colums)
                                <1>     ;        (HW = rows, SI = columns)
                                <1>     ;
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000DCCA F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
```

```
0000DCD1 7555                <1>     jnz     short pix_op_and_w ; window
                             <1>
0000DCD3 8B3D[929D0100]      <1>     mov     edi, [v_mem]
0000DCD9 89FE                <1>     mov     esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000DCDB 89C8                <1>     mov     eax, ecx
0000DCDD 8B0D[969D0100]      <1>     mov     ecx, [v_siz] ; display page pixel count
                             <1>
0000DCE3 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color 'and' ?
0000DCEA 7405                <1>     jz      short pix_op_and_0 ; no
0000DCEC E9D30F0000          <1>     jmp     m_pix_op_and ; 'and' color except mask color
                             <1> pix_op_and_0:
0000DCF1 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DCF8 7707                <1>     ja      short pix_op_and_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DCFA E88E070000          <1>     call    pix_op_and_8
0000DCFF EB1E                <1>     jmp     short pix_op_and_4
                             <1>
                             <1> pix_op_and_1:
0000DD01 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DD08 7710                <1>     ja      short pix_op_and_3 ; 32bpp
0000DD0A 7207                <1>     jb      short pix_op_and_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DD0C E88A070000          <1>     call    pix_op_and_24
0000DD11 EB0C                <1>     jmp     short pix_op_and_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_and_2:
0000DD13 E87B070000          <1>     call    pix_op_and_16
0000DD18 EB05                <1>     jmp     short pix_op_and_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_and_3:
0000DD1A E88B070000          <1>     call    pix_op_and_32
                             <1> pix_op_and_4:
0000DD1F 29F7                <1>     sub     edi, esi
0000DD21 893D[6C8E0100]      <1>     mov     [u.r0], edi
                             <1> pix_op_and_5:
0000DD27 C3                  <1>     retn
                             <1>
                             <1> pix_op_and_w:
                             <1>     ; 31/01/2021
0000DD28 51                  <1>     push    ecx ; * ; color
0000DD29 89D1                <1>     mov     ecx, edx ; win start pos
0000DD2B 89F2                <1>     mov     edx, esi ; size (rows, cols)
0000DD2D E8A9FAFFFF          <1>     call    sysvideo_15_12 ; window preparations
0000DD32 58                  <1>     pop     eax ; * ; color
0000DD33 72F2                <1>     jc      short pix_op_and_5
                             <1>
0000DD35 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color 'and' ?
0000DD3C 7405                <1>     jz      short pix_op_and_w_0 ; no
0000DD3E E90E100000          <1>     jmp     m_pix_op_and_w
                             <1>                     ; window 'and' color except mask color
                             <1> pix_op_and_w_0:
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>
0000DD43 8B3D[9A9D0100]      <1>     mov     edi, [v_str]
0000DD49 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DD50 7707                <1>     ja      short pix_op_and_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DD52 BD[8DE40000]        <1>     mov     ebp, pix_op_and_8
0000DD57 EB1E                <1>     jmp     short pix_op_and_w_4
                             <1>
                             <1> pix_op_and_w_1:
0000DD59 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DD60 7710                <1>     ja      short pix_op_and_w_3 ; 32bpp
0000DD62 7207                <1>     jb      short pix_op_and_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DD64 BD[9BE40000]        <1>     mov     ebp, pix_op_and_24
0000DD69 EB0C                <1>     jmp     short pix_op_and_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_and_w_2:
0000DD6B BD[93E40000]        <1>     mov     ebp, pix_op_and_16
0000DD70 EB05                <1>     jmp     short pix_op_and_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_and_w_3:
0000DD72 BD[AAE40000]        <1>     mov     ebp, pix_op_and_32
                             <1> pix_op_and_w_4:
0000DD77 E9E0FDFFFF          <1>     jmp     pix_op_and_w_x
                             <1>
                             <1> pix_op_xor:
                             <1>     ; 31/01/2021
                             <1>     ; XOR COLOR
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   CL = color (8 bit, 256 colors)
                             <1>     ;  ECX = color (16 bit and true colors)
                             <1>     ;  EDX = start position (row, column)
                             <1>     ;        (HW = row, DX = column)
                             <1>     ;  ESI = size (rows, colums)
                             <1>     ;        (HW = rows, SI = columns)
                             <1>     ;
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
0000DD7C F605[909D0100]10    <1>     test    byte [v_ops], 10h ; display page or window ?
0000DD83 7555                <1>     jnz     short pix_op_xor_w ; window
                             <1>
0000DD85 8B3D[929D0100]      <1>     mov     edi, [v_mem]
0000DD8B 89FE                <1>     mov     esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000DD8D 89C8                <1>     mov     eax, ecx
0000DD8F 8B0D[969D0100]      <1>     mov     ecx, [v_siz] ; display page pixel count
                             <1>
0000DD95 F605[909D0100]20    <1>     test    byte [v_ops], 20h ; masked color 'xor' ?
0000DD9C 7405                <1>     jz      short pix_op_xor_0 ; no
0000DD9E E9A1100000          <1>     jmp     m_pix_op_xor ; 'xor' color except mask color
                             <1> pix_op_xor_0:
0000DDA3 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DDAA 7707                <1>     ja      short pix_op_xor_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DDAC E801070000          <1>     call    pix_op_xor_8
```

```
0000DDB1 EB1E                <1>     jmp    short pix_op_xor_4
                             <1>
                             <1> pix_op_xor_1:
0000DDB3 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000DDBA 7710                <1>     ja     short pix_op_xor_3 ; 32bpp
0000DDBC 7207                <1>     jb     short pix_op_xor_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DDBE E8FD060000          <1>     call   pix_op_xor_24
0000DDC3 EB0C                <1>     jmp    short pix_op_xor_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_xor_2:
0000DDC5 E8EE060000          <1>     call   pix_op_xor_16
0000DDCA EB05                <1>     jmp    short pix_op_xor_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_xor_3:
0000DDCC E8FE060000          <1>     call   pix_op_xor_32
                             <1> pix_op_xor_4:
0000DDD1 29F7                <1>     sub    edi, esi
0000DDD3 893D[6C8E0100]      <1>     mov    [u.r0], edi
                             <1> pix_op_xor_5:
0000DDD9 C3                  <1>     retn
                             <1>
                             <1> pix_op_xor_w:
                             <1>     ; 31/01/2021
0000DDDA 51                  <1>     push   ecx ; * ; color
0000DDDB 89D1                <1>     mov    ecx, edx ; win start pos
0000DDDD 89F2                <1>     mov    edx, esi ; size (rows, cols)
0000DDDF E8F7F9FFFF          <1>     call   sysvideo_15_12 ; window preparations
0000DDE4 58                  <1>     pop    eax ; * ; color
0000DDE5 72F2                <1>     jc     short pix_op_xor_5
                             <1>
0000DDE7 F605[909D0100]20    <1>     test   byte [v_ops], 20h ; masked color 'xor' ?
0000DDEE 7405                <1>     jz     short pix_op_xor_w_0 ; no
0000DDF0 E9DC100000          <1>     jmp    m_pix_op_xor_w
                             <1>                    ; window 'xor' color except mask color
                             <1> pix_op_xor_w_0:
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>
0000DDF5 8B3D[9A9D0100]      <1>     mov    edi, [v_str]
0000DDFB 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000DE02 7707                <1>     ja     short pix_op_xor_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000DE04 BD[B2E40000]        <1>     mov    ebp, pix_op_xor_8
0000DE09 EB1E                <1>     jmp    short pix_op_xor_w_4
                             <1>
                             <1> pix_op_xor_w_1:
0000DE0B 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000DE12 7710                <1>     ja     short pix_op_xor_w_3 ; 32bpp
0000DE14 7207                <1>     jb     short pix_op_xor_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000DE16 BD[C0E40000]        <1>     mov    ebp, pix_op_xor_24
0000DE1B EB0C                <1>     jmp    short pix_op_xor_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> pix_op_xor_w_2:
0000DE1D BD[B8E40000]        <1>     mov    ebp, pix_op_xor_16
0000DE22 EB05                <1>     jmp    short pix_op_xor_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> pix_op_xor_w_3:
0000DE24 BD[CFE40000]        <1>     mov    ebp, pix_op_xor_32
                             <1> pix_op_xor_w_4:
0000DE29 E92EFDFFFF          <1>     jmp    pix_op_xor_w_x
                             <1>
                             <1> pix_op_new:
                             <1>     ; 31/01/2021
                             <1>     ; 30/01/2021
                             <1>     ; CHANGE COLOR
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   CL = color (8 bit, 256 colors)
                             <1>     ;  ECX = color (16 bit and true colors)
                             <1>     ;  EDX = start position (row, column)
                             <1>     ;        (HW = row, DX = column)
                             <1>     ;  ESI = size (rows, colums)
                             <1>     ;        (HW = rows, SI = columns)
                             <1>     ;
                             <1>     ;  [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
0000DE2E F605[909D0100]10    <1>     test   byte [v_ops], 10h ; display page or window ?
0000DE35 7554                <1>     jnz    short pix_op_new_w ; window
                             <1>
0000DE37 8B3D[929D0100]      <1>     mov    edi, [v_mem]
0000DE3D 89FE                <1>     mov    esi, edi
                             <1>     ; ecx = color (CL, CX, ECX)
0000DE3F 89C8                <1>     mov    eax, ecx
0000DE41 8B0D[969D0100]      <1>     mov    ecx, [v_siz] ; display page pixel count
                             <1>
0000DE47 F605[909D0100]20    <1>     test   byte [v_ops], 20h ; masked color change ?
0000DE4E 7405                <1>     jz     short pix_op_new_0 ; no
0000DE50 E90A0B0000          <1>     jmp    m_pix_op_new ; change color except mask color
                             <1> pix_op_new_0:
0000DE55 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000DE5C 7706                <1>     ja     short pix_op_new_2
                             <1>
                             <1>     ; 256 colors (8bpp)
                             <1> pix_op_new_1:
0000DE5E 88C4                <1>     mov    ah, al
0000DE60 D1E9                <1>     shr    ecx, 1
0000DE62 EB12                <1>     jmp    short pix_op_new_3
                             <1>
                             <1> pix_op_new_2:
0000DE64 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000DE6B 7713                <1>     ja     short pix_op_new_4 ; 32bpp
0000DE6D 7207                <1>     jb     short pix_op_new_3 ; 16bpp
                             <1>
                             <1>     ; 31/01/2021
                             <1>
                             <1>     ; 24 bit true colors
0000DE6F E849050000          <1>     call   pix_op_new_24
                             <1>
0000DE74 EB0C                <1>     jmp    short pix_op_new_5
                             <1>
```

```
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_new_3:
0000DE76 89C2                   <1>     mov     edx, eax
0000DE78 C1E010                 <1>     shl     eax, 16
0000DE7B 6689D0                 <1>     mov     ax, dx
0000DE7E D1E9                   <1>     shr     ecx, 1 ; dword counts
                                <1>     ; 32 bit true colors
                                <1> pix_op_new_4:
0000DE80 F3AB                   <1>     rep     stosd
                                <1> pix_op_new_5:
0000DE82 29F7                   <1>     sub     edi, esi
0000DE84 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_new_6:
0000DE8A C3                     <1>     retn
                                <1>
                                <1> pix_op_new_w:
                                <1>     ; 31/01/2021
                                <1>     ; 30/01/2021
0000DE8B 51                     <1>     push    ecx ; * ; color
0000DE8C 89D1                   <1>     mov     ecx, edx ; win start pos
0000DE8E 89F2                   <1>     mov     edx, esi ; size (rows, cols)
0000DE90 E846F9FFFF             <1>     call    sysvideo_15_12 ; window preparations
0000DE95 58                     <1>     pop     eax ; * ; color
0000DE96 72F2                   <1>     jc      short pix_op_new_6
                                <1>
0000DE98 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked color change ?
0000DE9F 7405                   <1>     jz      short pix_op_new_w_0 ; no
0000DEA1 E9470B0000             <1>     jmp     m_pix_op_new_w
                                <1>                     ; window chg color except mask color
                                <1> pix_op_new_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>     ; eax = color
                                <1>
0000DEA6 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000DEAC 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DEB3 7707                   <1>     ja      short pix_op_new_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DEB5 BD[B6E30000]           <1>     mov     ebp, pix_op_new_8
0000DEBA EB1E                   <1>     jmp     short pix_op_new_w_x
                                <1>
                                <1> pix_op_new_w_1:
0000DEBC 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DEC3 7710                   <1>     ja      short pix_op_new_w_3 ; 32bpp
0000DEC5 7207                   <1>     jb      short pix_op_new_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DEC7 BD[BDE30000]           <1>     mov     ebp, pix_op_new_24
0000DECC EB0C                   <1>     jmp     short pix_op_new_w_x
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_new_w_2:
0000DECE BD[B9E30000]           <1>     mov     ebp, pix_op_new_16
0000DED3 EB05                   <1>     jmp     short pix_op_new_w_x
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_new_w_3:
0000DED5 BD[C9E30000]           <1>     mov     ebp, pix_op_new_32
                                <1>     ;jmp    short pix_op_new_w_x
                                <1>
                                <1> pix_op_new_w_x:
                                <1> pix_op_not_w_x:
                                <1> pix_op_neg_w_x:
                                <1> pix_op_inc_w_x:
                                <1> pix_op_dec_w_x:
                                <1>     ; 27/02/2021
                                <1>     ; 01/02/2021
                                <1>     ; 31/01/2021
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = windows (screen) width in bytes
                                <1>     ; ebx = row count
                                <1>     ; eax = color
                                <1>     ; ebp = pixel operation subroutine address
                                <1>     ;push   edx ; 01/02/2021
0000DEDA 51                     <1>     push    ecx
0000DEDB 57                     <1>     push    edi
0000DEDC 8B0D[A69D0100]         <1>     mov     ecx, [pixcount] ; 27/02/2021
0000DEE2 FFD5                   <1>     call    ebp ; call pixel-row operation
0000DEE4 5F                     <1>     pop     edi
0000DEE5 59                     <1>     pop     ecx ; bytes per row
0000DEE6 010D[6C8E0100]         <1>     add     [u.r0], ecx
                                <1>     ;pop    edx ; 01/02/2021
0000DEEC 01D7                   <1>     add     edi, edx ; next row
0000DEEE 4B                     <1>     dec     ebx
0000DEEF 75E9                   <1>     jnz     short pix_op_new_w_x
0000DEF1 C3                     <1>     retn
                                <1>
                                <1> pix_op_not:
                                <1>     ; 31/01/2021
                                <1>     ; NOT COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;  ECX = start position (row, column)
                                <1>     ;        (HW = row, CX = column)
                                <1>     ;  EDX = size (rows, colums)
                                <1>     ;        (HW = rows, DX = columns)
                                <1>     ;       (0 -> invalid
                                <1>     ;        (1 -> horizontal or vertical line)
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;      [u.r0] will be > 0 if succesful
                                <1>
0000DEF2 F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000DEF9 7553                   <1>     jnz     short pix_op_not_w ; window
                                <1>
0000DEFB 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000DF01 89FE                   <1>     mov     esi, edi
0000DF03 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1>
0000DF09 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked color 'not' ?
0000DF10 7405                   <1>     jz      short pix_op_not_0 ; no
0000DF12 E9ED0F0000             <1>     jmp     m_pix_op_not ; 'not' color except mask color
                                <1> pix_op_not_0:
0000DF17 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DF1E 7707                   <1>     ja      short pix_op_not_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DF20 E8F5050000             <1>     call    pix_op_not_8
0000DF25 EB1E                   <1>     jmp     short pix_op_not_4
                                <1>
```

```
                                <1> pix_op_not_1:
0000DF27 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DF2E 7710                   <1>     ja      short pix_op_not_3 ; 32bpp
0000DF30 7207                   <1>     jb      short pix_op_not_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DF32 E8F1050000             <1>     call    pix_op_not_24
0000DF37 EB0C                   <1>     jmp     short pix_op_not_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_not_2:
0000DF39 E8E2050000             <1>     call    pix_op_not_16
0000DF3E EB05                   <1>     jmp     short pix_op_not_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_not_3:
0000DF40 E8EE050000             <1>     call    pix_op_not_32
                                <1> pix_op_not_4:
0000DF45 29F7                   <1>     sub     edi, esi
0000DF47 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_not_5:
0000DF4D C3                     <1>     retn
                                <1>
                                <1> pix_op_not_w:
                                <1>     ; 31/01/2021
                                <1>     ; ecx = win start pos (row, column)
                                <1>     ; edx = size (rows, columns)
0000DF4E E888F8FFFF             <1>     call    sysvideo_15_12 ; window preparations
0000DF53 72F8                   <1>     jc      short pix_op_not_5
                                <1>
0000DF55 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked color 'not' ?
0000DF5C 7405                   <1>     jz      short pix_op_not_w_0 ; no
0000DF5E E926100000             <1>     jmp     m_pix_op_not_w
                                <1>                     ; window 'not' color except mask color
                                <1> pix_op_not_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>
0000DF63 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000DF69 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DF70 7707                   <1>     ja      short pix_op_not_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DF72 BD[1AE50000]           <1>     mov     ebp, pix_op_not_8
0000DF77 EB1E                   <1>     jmp     short pix_op_not_w_4
                                <1>
                                <1> pix_op_not_w_1:
0000DF79 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DF80 7710                   <1>     ja      short pix_op_not_w_3 ; 32bpp
0000DF82 7207                   <1>     jb      short pix_op_not_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DF84 BD[28E50000]           <1>     mov     ebp, pix_op_not_24
0000DF89 EB0C                   <1>     jmp     short pix_op_not_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_not_w_2:
0000DF8B BD[20E50000]           <1>     mov     ebp, pix_op_not_16
0000DF90 EB05                   <1>     jmp     short pix_op_not_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_not_w_3:
0000DF92 BD[33E50000]           <1>     mov     ebp, pix_op_not_32
                                <1> pix_op_not_w_4:
0000DF97 E93EFFFFFF             <1>     jmp     pix_op_not_w_x
                                <1>
                                <1> pix_op_neg:
                                <1>     ; 31/01/2021
                                <1>     ; NEGATE COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;  ECX = start position (row, column)
                                <1>     ;        (HW = row, CX = column)
                                <1>     ;  EDX = size (rows, colums)
                                <1>     ;        (HW = rows, DX = columns)
                                <1>     ;       (0 -> invalid
                                <1>     ;        (1 -> horizontal or vertical line)
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000DF9C F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000DFA3 7553                   <1>     jnz     short pix_op_neg_w ; window
                                <1>
0000DFA5 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000DFAB 89FE                   <1>     mov     esi, edi
0000DFAD 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1>
0000DFB3 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked negate color ?
0000DFBA 7405                   <1>     jz      short pix_op_neg_0 ; no
0000DFBC E9FB0F0000             <1>     jmp     m_pix_op_neg ; 'neg' color except mask color
                                <1> pix_op_neg_0:
0000DFC1 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000DFC8 7707                   <1>     ja      short pix_op_neg_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000DFCA E86C050000             <1>     call    pix_op_neg_8
0000DFCF EB1E                   <1>     jmp     short pix_op_neg_4
                                <1>
                                <1> pix_op_neg_1:
0000DFD1 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000DFD8 7710                   <1>     ja      short pix_op_neg_3 ; 32bpp
0000DFDA 7207                   <1>     jb      short pix_op_neg_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000DFDC E868050000             <1>     call    pix_op_neg_24
0000DFE1 EB0C                   <1>     jmp     short pix_op_neg_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_neg_2:
0000DFE3 E859050000             <1>     call    pix_op_neg_16
0000DFE8 EB05                   <1>     jmp     short pix_op_neg_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_neg_3:
0000DFEA E86C050000             <1>     call    pix_op_neg_32
                                <1> pix_op_neg_4:
0000DFEF 29F7                   <1>     sub     edi, esi
0000DFF1 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_neg_5:
0000DFF7 C3                     <1>     retn
```

```
                                <1>
                                <1> pix_op_neg_w:
                                <1>     ; 31/01/2021
                                <1>     ; ecx = win start pos (row, column)
                                <1>     ; edx = size (rows, columns)
0000DFF8 E8DEF7FFFF             <1>     call    sysvideo_15_12 ; window preparations
0000DFFD 72F8                   <1>     jc      short pix_op_neg_5
                                <1>
0000DFFF F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked negate color ?
0000E006 7405                   <1>     jz      short pix_op_neg_w_0 ; no
0000E008 E934100000             <1>     jmp     m_pix_op_neg_w
                                <1>                     ; window 'neg' color except mask color
                                <1> pix_op_neg_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>
0000E00D 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000E013 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E01A 7707                   <1>     ja      short pix_op_neg_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000E01C BD[3BE50000]           <1>     mov     ebp, pix_op_neg_8
0000E021 EB1E                   <1>     jmp     short pix_op_neg_w_4
                                <1>
                                <1> pix_op_neg_w_1:
0000E023 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E02A 7710                   <1>     ja      short pix_op_neg_w_3 ; 32bpp
0000E02C 7207                   <1>     jb      short pix_op_neg_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000E02E BD[49E50000]           <1>     mov     ebp, pix_op_neg_24
0000E033 EB0C                   <1>     jmp     short pix_op_neg_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_neg_w_2:
0000E035 BD[41E50000]           <1>     mov     ebp, pix_op_neg_16
0000E03A EB05                   <1>     jmp     short pix_op_neg_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_neg_w_3:
0000E03C BD[5BE50000]           <1>     mov     ebp, pix_op_neg_32
                                <1> pix_op_neg_w_4:
0000E041 E994FEFFFF             <1>     jmp     pix_op_neg_w_x
                                <1>
                                <1> pix_op_inc:
                                <1>     ; 31/01/2021
                                <1>     ; INCREASE COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;  ECX = start position (row, column)
                                <1>     ;        (HW = row, CX = column)
                                <1>     ;  EDX = size (rows, colums)
                                <1>     ;        (HW = rows, DX = columns)
                                <1>     ;       (0 -> invalid
                                <1>     ;        (1 -> horizontal or vertical line)
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000E046 F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000E04D 7553                   <1>     jnz     short pix_op_inc_w ; window
                                <1>
0000E04F 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000E055 89FE                   <1>     mov     esi, edi
0000E057 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1>
0000E05D F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked increase color ?
0000E064 7405                   <1>     jz      short pix_op_inc_0 ; no
0000E066 E909100000             <1>     jmp     m_pix_op_inc ; 'inc' color except mask color
                                <1> pix_op_inc_0:
0000E06B 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E072 7707                   <1>     ja      short pix_op_inc_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000E074 E8EA040000             <1>     call    pix_op_inc_8
0000E079 EB1E                   <1>     jmp     short pix_op_inc_4
                                <1>
                                <1> pix_op_inc_1:
0000E07B 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E082 7710                   <1>     ja      short pix_op_inc_3 ; 32bpp
0000E084 7207                   <1>     jb      short pix_op_inc_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000E086 E8EF040000             <1>     call    pix_op_inc_24
0000E08B EB0C                   <1>     jmp     short pix_op_inc_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_inc_2:
0000E08D E8DB040000             <1>     call    pix_op_inc_16
0000E092 EB05                   <1>     jmp     short pix_op_inc_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_inc_3:
0000E094 E8F5040000             <1>     call    pix_op_inc_32
                                <1> pix_op_inc_4:
0000E099 29F7                   <1>     sub     edi, esi
0000E09B 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_inc_5:
0000E0A1 C3                     <1>     retn
                                <1>
                                <1> pix_op_inc_w:
                                <1>     ; 31/01/2021
                                <1>     ; ecx = win start pos (row, column)
                                <1>     ; edx = size (rows, columns)
0000E0A2 E834F7FFFF             <1>     call    sysvideo_15_12 ; window preparations
0000E0A7 72F8                   <1>     jc      short pix_op_inc_5
                                <1>
0000E0A9 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked increase color ?
0000E0B0 7405                   <1>     jz      short pix_op_inc_w_0 ; no
0000E0B2 E956100000             <1>     jmp     m_pix_op_inc_w
                                <1>                     ; window 'inc' color except mask color
                                <1> pix_op_inc_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>
0000E0B7 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000E0BD 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E0C4 7707                   <1>     ja      short pix_op_inc_w_1
                                <1>
```

```
                                <1>     ; 256 colors (8bpp)
0000E0C6 BD[63E50000]           <1>     mov     ebp, pix_op_inc_8
0000E0CB EB1E                   <1>     jmp     short pix_op_inc_w_4
                                <1>
                                <1> pix_op_inc_w_1:
0000E0CD 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E0D4 7710                   <1>     ja      short pix_op_inc_w_3 ; 32bpp
0000E0D6 7207                   <1>     jb      short pix_op_inc_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000E0D8 BD[7AE50000]           <1>     mov     ebp, pix_op_inc_24
0000E0DD EB0C                   <1>     jmp     short pix_op_inc_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_inc_w_2:
0000E0DF BD[6DE50000]           <1>     mov     ebp, pix_op_inc_16
0000E0E4 EB05                   <1>     jmp     short pix_op_inc_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_inc_w_3:
0000E0E6 BD[8EE50000]           <1>     mov     ebp, pix_op_inc_32
                                <1> pix_op_inc_w_4:
0000E0EB E9EAFDFFFF             <1>     jmp     pix_op_inc_w_x
                                <1>
                                <1> pix_op_dec:
                                <1>     ; 31/01/2021
                                <1>     ; DECREASE COLOR
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;  ECX = start position (row, column)
                                <1>     ;         (HW = row, CX = column)
                                <1>     ;  EDX = size (rows, colums)
                                <1>     ;         (HW = rows, DX = columns)
                                <1>     ;        (0 -> invalid
                                <1>     ;         (1 -> horizontal or vertical line)
                                <1>     ;  [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
0000E0F0 F605[909D0100]10       <1>     test    byte [v_ops], 10h ; display page or window ?
0000E0F7 7553                   <1>     jnz     short pix_op_dec_w ; window
                                <1>
0000E0F9 8B3D[929D0100]         <1>     mov     edi, [v_mem]
0000E0FF 89FE                   <1>     mov     esi, edi
0000E101 8B0D[969D0100]         <1>     mov     ecx, [v_siz] ; display page pixel count
                                <1>
0000E107 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked decrease color ?
0000E10E 7405                   <1>     jz      short pix_op_dec_0 ; no
0000E110 E92B100000             <1>     jmp     m_pix_op_dec ; 'dec' color except mask color
                                <1> pix_op_dec_0:
0000E115 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E11C 7707                   <1>     ja      short pix_op_dec_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000E11E E877040000             <1>     call    pix_op_dec_8
0000E123 EB1E                   <1>     jmp     short pix_op_dec_4
                                <1>
                                <1> pix_op_dec_1:
0000E125 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E12C 7710                   <1>     ja      short pix_op_dec_3 ; 32bpp
0000E12E 7207                   <1>     jb      short pix_op_dec_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000E130 E87C040000             <1>     call    pix_op_dec_24
0000E135 EB0C                   <1>     jmp     short pix_op_dec_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_dec_2:
0000E137 E868040000             <1>     call    pix_op_dec_16
0000E13C EB05                   <1>     jmp     short pix_op_dec_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_dec_3:
0000E13E E881040000             <1>     call    pix_op_dec_32
                                <1> pix_op_dec_4:
0000E143 29F7                   <1>     sub     edi, esi
0000E145 893D[6C8E0100]         <1>     mov     [u.r0], edi
                                <1> pix_op_dec_5:
0000E14B C3                     <1>     retn
                                <1>
                                <1> pix_op_dec_w:
                                <1>     ; 31/01/2021
                                <1>     ; ecx = win start pos (row, column)
                                <1>     ; edx = size (rows, columns)
0000E14C E88AF6FFFF             <1>     call    sysvideo_15_12 ; window preparations
0000E151 72F8                   <1>     jc      short pix_op_dec_5
                                <1>
0000E153 F605[909D0100]20       <1>     test    byte [v_ops], 20h ; masked decrease color ?
0000E15A 7405                   <1>     jz      short pix_op_dec_w_0 ; no
0000E15C E973100000             <1>     jmp     m_pix_op_dec_w
                                <1>                     ; window 'dec' color except mask color
                                <1> pix_op_dec_w_0:
                                <1>     ; ecx = bytes per row (to be applied)
                                <1>     ; edx = screen width in bytes
                                <1>     ; ebx = row count
                                <1>
0000E161 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>
0000E167 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E16E 7707                   <1>     ja      short pix_op_dec_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000E170 BD[9AE50000]           <1>     mov     ebp, pix_op_dec_8
0000E175 EB1E                   <1>     jmp     short pix_op_dec_w_4
                                <1>
                                <1> pix_op_dec_w_1:
0000E177 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E17E 7710                   <1>     ja      short pix_op_dec_w_3 ; 32bpp
0000E180 7207                   <1>     jb      short pix_op_dec_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000E182 BD[B1E50000]           <1>     mov     ebp, pix_op_dec_24
0000E187 EB0C                   <1>     jmp     short pix_op_dec_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> pix_op_dec_w_2:
0000E189 BD[A4E50000]           <1>     mov     ebp, pix_op_dec_16
0000E18E EB05                   <1>     jmp     short pix_op_dec_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> pix_op_dec_w_3:
0000E190 BD[C4E50000]           <1>     mov     ebp, pix_op_dec_32
                                <1> pix_op_dec_w_4:
```

```
0000E195 E940FDFFFF          <1>     jmp    pix_op_dec_w_x
                             <1>
                             <1> pix_op_blk:
                             <1>     ; 11/08/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 23/01/2021
                             <1>     ; 22/02/2021
                             <1>     ; 02/02/2021
                             <1>     ; COPY PIXEL BLOCK -system to system-
                             <1>     ; WRITE PIXEL BLOCKS -user to system-
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   -If BL bit 5 is 0-
                             <1>     ;    ECX = start position (row, column) (*)
                             <1>     ;    (HW = row, CX = column)
                             <1>     ;    EDX = size (rows, colums) (*)
                             <1>     ;    (HW = rows, DX = columns)
                             <1>     ;         (0 -> invalid)
                             <1>     ;         (1 -> horizontal or vertical line)
                             <1>     ;    ESI = destination (row, column) (***)
                             <1>     ;   -If BL bit 5 is 1-
                             <1>     ;     CL = color (8 bit, 256 colors)
                             <1>     ;    ECX = color (16 bit and true colors)
                             <1>     ;    EDX = count of blocks (not bytes)
                             <1>     ;        (limit: 2048 blocks/windows)
                             <1>     ;    ESI = user's buffer address
                             <1>     ;        contains 64 bit block data
                             <1>     ;        BLOCK ADDRESS - (row, col), dword
                             <1>     ;        (first 32 bits)
                             <1>     ;        BLOCK SIZE - (rows, cols), dword
                             <1>     ;        (second 32 bits)
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Window option ([v_ops] bit 4) will be ignored
                             <1>     ; (Function is used for display page coordinates)
                             <1>
0000E19A F605[909D0100]20    <1>     test   byte [v_ops], 20h ; masked or direct ?
0000E1A1 755A                <1>     jnz    short pix_op_blk_u ; blocks from user's buffer
                             <1>
0000E1A3 89F0                <1>     mov    eax, esi ; destination position (row, col)
0000E1A5 E8DEF6FFFF          <1>     call   calc_pixel_offset
0000E1AA 3B05[969D0100]      <1>     cmp    eax, [v_siz]
0000E1B0 734A                <1>     jnb    short pix_op_blk_retn ; out of display page
0000E1B2 89C6                <1>     mov    esi, eax
0000E1B4 E8ACF6FFFF          <1>     call   pixels_to_byte_count
0000E1B9 89C7                <1>     mov    edi, eax
0000E1BB 89D0                <1>     mov    eax, edx  ; size
0000E1BD E8C6F6FFFF          <1>     call   calc_pixel_offset
                             <1>     ; 22/02/2021
0000E1C2 3B05[969D0100]      <1>     cmp    eax, [v_siz]
0000E1C8 7732                <1>     ja     short pix_op_blk_retn ; out of display page
0000E1CA 01C6                <1>     add    esi, eax
0000E1CC 3B35[969D0100]      <1>     cmp    esi, [v_siz]
0000E1D2 7728                <1>     ja     short pix_op_blk_retn ; out of display page
                             <1>
0000E1D4 033D[929D0100]      <1>     add    edi, [v_mem] ; destination address
                             <1>
                             <1>     ; 23/01/2021
                             <1>     ;call  pixels_to_byte_count
                             <1>     ;add   edi, eax
                             <1>     ;jc    short pix_op_blk_retn ; out of display page
                             <1>     ;cmp   edi, [v_end]
                             <1>     ;ja    short pix_op_blk_retn ; out of display page
                             <1>     ;sub   edi, eax
                             <1>
0000E1DA E8FCF5FFFF          <1>     call   sysvideo_15_12 ; window preparations
0000E1DF 721B                <1>     jc     short pix_op_blk_retn ; something wrong !?
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = screen width in bytes
                             <1>     ; ebx = row count
                             <1>
0000E1E1 8B35[9A9D0100]      <1>     mov    esi, [v_str] ; source address
                             <1>
                             <1>     ; Note:
                             <1>     ; ecx & edx are already adjusted for pixel sizes
                             <1>     ; so, following code is proper all pixel sizes
                             <1>
0000E1E7 29CA                <1>     sub    edx, ecx ; screen width - window width
                             <1> pix_op_blk_0:
0000E1E9 89C8                <1>     mov    eax, ecx
0000E1EB 0105[6C8E0100]      <1>     add    [u.r0], eax
0000E1F1 F3A4                <1>     rep    movsb
0000E1F3 89C1                <1>     mov    ecx, eax
0000E1F5 01D6                <1>     add    esi, edx ; next row
0000E1F7 01D7                <1>     add    edi, edx ; next row
0000E1F9 4B                  <1>     dec    ebx
0000E1FA 75ED                <1>     jnz    short pix_op_blk_0
                             <1> pix_op_blk_retn:
0000E1FC C3                  <1>     retn
                             <1>
                             <1> pix_op_blk_u:
                             <1>     ; fill blocks (windows) with desired color
                             <1>     ; according to block definitions in user's buffer
0000E1FD 81FA00080000        <1>     cmp    edx, 2048
0000E203 7605                <1>     jna    short pix_op_blk_u_0
                             <1>     ; Maximum 2048 blocks
0000E205 BA00080000          <1>     mov    edx, 2048
                             <1> pix_op_blk_u_0:
0000E20A 8025[909D0100]DF    <1>     and    byte [v_ops], ~20h ; clear masked bit
0000E211 890D[A29D0100]      <1>     mov    [maskcolor], ecx ; save pixel color
                             <1>     ; 22/02/2021
                             <1>     ;mov   ebp, edx ; save blocks count
                             <1>     ;push  ebp
                             <1> pix_op_blk_u_next:
0000E217 52                  <1>     push   edx
                             <1>     ;mov   ecx, 8
                             <1>     ; 11/08/2022
0000E218 29C9                <1>     sub    ecx, ecx
0000E21A B108                <1>     mov    cl, 8
0000E21C BF[AA9D0100]        <1>     mov    edi, buffer8 ; 8 bytes small buffer
                             <1>     ; esi = user's buffer address
0000E221 E83F290000          <1>     call   transfer_from_user_buffer
0000E226 72D4                <1>     jc     short pix_op_blk_retn
0000E228 01CE                <1>     add    esi, ecx ; 22/02/2021
0000E22A 56                  <1>     push   esi
0000E22B 8B15[AA9D0100]      <1>     mov    edx, [buffer8] ; block start pos (row,col)
0000E231 8B35[AE9D0100]      <1>     mov    esi, [buffer8+4] ; block size (rows,cols)
0000E237 8B0D[A29D0100]      <1>     mov    ecx, [maskcolor]
0000E23D E849FCFFFF          <1>     call   pix_op_new_w ; new (change) color (window)
0000E242 5E                  <1>     pop    esi
                             <1>     ;pop   ebp
                             <1>     ;dec   ebp
0000E243 5A                  <1>     pop    edx
0000E244 4A                  <1>     dec    edx
```

```
0000E245 75D0                <1>     jnz    short pix_op_blk_u_next
0000E247 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin:
                             <1>     ; 11/08/2022
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 12/02/2021
                             <1>     ; 11/02/2021
                             <1>     ; 10/02/2021
                             <1>     ; 05/02/2021
                             <1>     ; 02/02/2021
                             <1>     ; WRITE LINE -direct-
                             <1>     ; WRITE LINE(S) -via user's buffer-
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   -If BL bit 5 is 0-
                             <1>     ;     CL = color (8 bit, 256 colors)
                             <1>     ;    ECX = color (16 bit and true colors)
                             <1>     ;     DX = low 12 bits - size (length)
                             <1>     ;          high 4 bits - direction or type
                             <1>     ;            0 - Horizontal line
                             <1>     ;            1 - Vertical line
                             <1>     ;          > 1 - undefined, invalid
                             <1>     ;    ESI = start position (row, column)
                             <1>     ;         (HW = row, SI = column)
                             <1>     ;   -If BL bit 5 is 1-
                             <1>     ;     CL = color (8 bit, 256 colors)
                             <1>     ;    ECX = color (16 bit and true colors)
                             <1>     ;     DX = number of lines (in user buffer)
                             <1>     ;          (limit: 2048 lines)
                             <1>     ;    ESI = user's buffer
                             <1>     ;          contains 64 bit data for lines
                             <1>     ;          START POINT: 32 bit (row, col)
                             <1>     ;          LENGTH: 32 bit
                             <1>     ;                  high 16 bits - 0
                             <1>     ;                  bit 0-11 - length
                             <1>     ;                  bit 12-15 - type (length)
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Window option ([v_ops] bit 4) will be ignored
                             <1>     ; (Function is used for display page coordinates)
                             <1>
                             <1>     ; 10/02/2021
0000E248 F605[909D0100]20    <1>     test   byte [v_ops], 20h ; masked or direct ?
0000E24F 7444                <1>     jz     short pix_op_lin_vh ; direct (v/h lines)
                             <1>
                             <1>     ; lines from user's buffer
                             <1> pix_op_lin_u:
                             <1>     ; draw lines with desired color
                             <1>     ; according to line definitions in user's buffer
0000E251 81FA00080000        <1>     cmp    edx, 2048
0000E257 7605                <1>     jna    short pix_op_lin_u_0
                             <1>     ; Maximum 2048 lines
0000E259 BA00080000          <1>     mov    edx, 2048
                             <1> pix_op_lin_u_0:
0000E25E 890D[A29D0100]      <1>     mov    [maskcolor], ecx ; save pixel color
0000E264 89D5                <1>     mov    ebp, edx ; save line count
                             <1> pix_op_lin_u_next:
                             <1>     ;mov   ecx, 8
                             <1>     ; 11/08/2022
0000E266 29C9                <1>     sub    ecx, ecx
0000E268 B108                <1>     mov    cl, 8
0000E26A BF[AA9D0100]        <1>     mov    edi, buffer8 ; 8 bytes small buffer
                             <1>     ; esi = user's buffer address
0000E26F E8F1280000          <1>     call   transfer_from_user_buffer
0000E274 721E                <1>     jc     short pix_op_lin_retn
0000E276 01CE                <1>     add    esi, ecx ; 11/02/2021
0000E278 56                  <1>     push   esi
0000E279 8B35[AA9D0100]      <1>     mov    esi, [buffer8] ; line start pos (row,col)
0000E27F 8B15[AE9D0100]      <1>     mov    edx, [buffer8+4] ; line length
0000E285 8B0D[A29D0100]      <1>     mov    ecx, [maskcolor]
0000E28B E805000000          <1>     call   pix_op_lin_vh ; new (change) color (window)
0000E290 5E                  <1>     pop    esi
0000E291 4D                  <1>     dec    ebp
0000E292 75D2                <1>     jnz    short pix_op_lin_u_next
                             <1> pix_op_lin_retn:
0000E294 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_vh:
0000E295 81FA38140000        <1>     cmp    edx, 1438h ; 1920*1080 (780hx438h) limit
0000E29B 7762                <1>     ja     short pix_op_lin_err1 ; invalid type
                             <1>                           ; (for current version)
0000E29D 66F7C2FF0F          <1>     test   dx, 0FFFh
0000E2A2 745B                <1>     jz     short pix_op_lin_err1 ; zero length!
                             <1>
0000E2A4 89F0                <1>     mov    eax, esi ; start point (row, col)
0000E2A6 E8DDF5FFFF          <1>     call   calc_pixel_offset
0000E2AB 3B05[969D0100]      <1>     cmp    eax, [v_siz]
0000E2B1 734C                <1>     jnb    short pix_op_lin_err1 ; out of display page!
0000E2B3 E8ADF5FFFF          <1>     call   pixels_to_byte_count
0000E2B8 89C7                <1>     mov    edi, eax ; start point offset
0000E2BA 033D[929D0100]      <1>     add    edi, [v_mem] ; LFB start address
0000E2C0 89C8                <1>     mov    eax, ecx ; color
                             <1>
0000E2C2 F6C610              <1>     test   dh, 10h
                             <1>     ;jz    pix_op_lin_h ; Horizontal line
                             <1>     ; 23/07/2022
0000E2C5 7505                <1>     jnz    short pix_op_lin_v
0000E2C7 E98A000000          <1>     jmp    pix_op_lin_h
                             <1>
                             <1> pix_op_lin_v:
                             <1>     ; Vertical line
0000E2CC 80E60F              <1>     and    dh, 0Fh ; low 12 bits
0000E2CF 51                  <1>     push   ecx ; color
0000E2D0 89D1                <1>     mov    ecx, edx
0000E2D2 0FB705[8E9D0100]    <1>     movzx  eax, word [v_width]
0000E2D9 89C3                <1>     mov    ebx, eax
                             <1>     ; 12/02/2021
0000E2DB F7E2                <1>     mul    edx ; rows * [v_width]
0000E2DD 01F8                <1>     add    eax, edi
0000E2DF 3B05[9E9D0100]      <1>     cmp    eax, [v_end]
0000E2E5 58                  <1>     pop    eax ; color
0000E2E6 7717                <1>     ja     short pix_op_lin_err1 ; out of display page
                             <1>     ; ecx = rows
0000E2E8 89CA                <1>     mov    edx, ecx
                             <1>
0000E2EA 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000E2F1 770D                <1>     ja     short pix_op_lin_v_2
                             <1>     ; 256 colors (1 byte per pixel)
0000E2F3 010D[6C8E0100]      <1>     add    [u.r0], ecx ; byte count
                             <1> pix_op_lin_v_1:
0000E2F9 8807                <1>     mov    [edi], al
0000E2FB 01DF                <1>     add    edi, ebx  ; next row
```

```
0000E2FD E2FA                <1>     loop    pix_op_lin_v_1
                             <1> pix_op_lin_err1:
0000E2FF C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_v_2:
0000E300 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E307 773A                <1>     ja      short pix_op_lin_v_6 ; 32bpp
0000E309 7226                <1>     jb      short pix_op_lin_v_4 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
                             <1>     ; * 3
0000E30B 53                  <1>     push    ebx ; screen width in pixels
0000E30C D1E3                <1>     shl     ebx, 1
0000E30E 011C24              <1>     add     [esp], ebx
0000E311 5B                  <1>     pop     ebx ; screen width in bytes
0000E312 010D[6C8E0100]      <1>     add     [u.r0], ecx
0000E318 D1E2                <1>     shl     edx, 1
0000E31A 0115[6C8E0100]      <1>     add     [u.r0], edx ; byte count
                             <1> pix_op_lin_v_3:
0000E320 668907              <1>     mov     [edi], ax
0000E323 C1C810              <1>     ror     eax, 16
0000E326 884702              <1>     mov     [edi+2], al
0000E329 C1C010              <1>     rol     eax, 16
0000E32C 01DF                <1>     add     edi, ebx  ; next row
0000E32E E2F0                <1>     loop    pix_op_lin_v_3
0000E330 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_v_4:
                             <1>     ; 16 bit (65536) colors
0000E331 D1E3                <1>     shl     ebx, 1
0000E333 D1E2                <1>     shl     edx, 1
0000E335 0115[6C8E0100]      <1>     add     [u.r0], edx
                             <1> pix_op_lin_v_5:
0000E33B 668907              <1>     mov     [edi], ax
0000E33E 01DF                <1>     add     edi, ebx  ; next row
0000E340 E2F9                <1>     loop    pix_op_lin_v_5
0000E342 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_v_6:
                             <1>     ; 32 bit true colors
0000E343 C1E302              <1>     shl     ebx, 2
0000E346 C1E202              <1>     shl     edx, 2
0000E349 0115[6C8E0100]      <1>     add     [u.r0], edx ; byte count
                             <1> pix_op_lin_v_7:
0000E34F 8907                <1>     mov     [edi], eax
0000E351 01DF                <1>     add     edi, ebx  ; next row
0000E353 E2FA                <1>     loop    pix_op_lin_v_7
0000E355 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_h:
                             <1>     ; Horizontal line
0000E356 80E60F              <1>     and     dh, 0Fh ; low 12 bits
0000E359 89D1                <1>     mov     ecx, edx
0000E35B 6601D6              <1>     add     si, dx ; start column + columns
0000E35E 663B35[8E9D0100]    <1>     cmp     si, [v_width] ; screen width
0000E365 7711                <1>     ja      short pix_op_lin_err2 ; out of columns limit
                             <1>
0000E367 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E36E 7709                <1>     ja      short pix_op_lin_h_1
                             <1>     ; 256 colors (1 byte per pixel)
0000E370 010D[6C8E0100]      <1>     add     [u.r0], ecx
0000E376 F3AA                <1>     rep     stosb
                             <1> pix_op_lin_err2:
0000E378 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_h_1:
0000E379 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E380 7728                <1>     ja      short pix_op_lin_h_4 ; 32bpp
0000E382 721A                <1>     jb      short pix_op_lin_h_3 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
                             <1>     ; * 3
0000E384 0115[6C8E0100]      <1>     add     [u.r0], edx
0000E38A D1E2                <1>     shl     edx, 1
0000E38C 0115[6C8E0100]      <1>     add     [u.r0], edx
                             <1> pix_op_lin_h_2:
0000E392 66AB                <1>     stosw
0000E394 C1C810              <1>     ror     eax, 16
0000E397 AA                  <1>     stosb
0000E398 C1C010              <1>     rol     eax, 16
0000E39B E2F5                <1>     loop    pix_op_lin_h_2
0000E39D C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_h_3:
                             <1>     ; 16 bit (65536) colors
0000E39E D1E2                <1>     shl     edx, 1
0000E3A0 0115[6C8E0100]      <1>     add     [u.r0], edx
0000E3A6 66F3AB              <1>     rep     stosw
0000E3A9 C3                  <1>     retn
                             <1>
                             <1> pix_op_lin_h_4:
                             <1>     ; 32 bit true colors
0000E3AA C1E202              <1>     shl     edx, 2
0000E3AD 0115[6C8E0100]      <1>     add     [u.r0], edx
0000E3B3 F3AB                <1>     rep     stosd
0000E3B5 C3                  <1>     retn
                             <1>
                             <1> pix_op_new_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; CHANGE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  al = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3B6 F3AA                <1>     rep     stosb
0000E3B8 C3                  <1>     retn
                             <1>
                             <1> pix_op_new_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; CHANGE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  ax = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3B9 66F3AB              <1>     rep     stosw
0000E3BC C3                  <1>     retn
                             <1>
                             <1> pix_op_new_24:
                             <1>     ; 24 bit true colors
                             <1>     ; CHANGE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = color
                             <1>     ; edi = start pixel address
                             <1>
```

```
0000E3BD 66AB                <1>     stosw
0000E3BF C1C810              <1>     ror    eax, 16
0000E3C2 AA                  <1>     stosb
0000E3C3 C1C010              <1>     rol    eax, 16
0000E3C6 E2F5                <1>     loop   pix_op_new_24
0000E3C8 C3                  <1>     retn
                             <1>
                             <1> pix_op_new_32:
                             <1>     ; 32 bit true colors
                             <1>     ; CHANGE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3C9 F3AB                <1>     rep    stosd
0000E3CB C3                  <1>     retn
                             <1>
                             <1> pix_op_add_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; ADD PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  al = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3CC 88C4                <1>     mov    ah, al
                             <1> pix_op_add_8_0:
0000E3CE 0207                <1>     add    al, [edi]
0000E3D0 7302                <1>     jnc    short pix_op_add_8_1
0000E3D2 B0FF                <1>     mov    al, 0FFh ; Max. value
                             <1> pix_op_add_8_1:
0000E3D4 AA                  <1>     stosb
0000E3D5 88E0                <1>     mov    al, ah
0000E3D7 E2F5                <1>     loop   pix_op_add_8_0
0000E3D9 C3                  <1>     retn
                             <1>
                             <1> pix_op_add_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; ADD PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  ax = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3DA 89C2                <1>     mov    edx, eax
                             <1> pix_op_add_16_0:
0000E3DC 660307              <1>     add    ax, [edi]
0000E3DF 7304                <1>     jnc    short pix_op_add_16_1
0000E3E1 66B8FFFF            <1>     mov    ax, 0FFFFh ; Max. value
                             <1> pix_op_add_16_1:
0000E3E5 66AB                <1>     stosw
0000E3E7 89D0                <1>     mov    eax, edx
0000E3E9 E2F1                <1>     loop   pix_op_add_16_0
0000E3EB C3                  <1>     retn
                             <1>
                             <1> pix_op_add_24:
                             <1>     ; 24 bit true colors
                             <1>     ; ADD PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = color
                             <1>     ; edi = start pixel address
                             <1>
0000E3EC 53                  <1>     push   ebx
0000E3ED BBFFFFFF00          <1>     mov    ebx, 0FFFFFFh
                             <1>     ;and   eax, ebx ; 0FFFFFFh
0000E3F2 89C2                <1>     mov    edx, eax
                             <1> pix_op_add_24_0:
0000E3F4 8B07                <1>     mov    eax, [edi]
0000E3F6 21D8                <1>     and    eax, ebx ; 0FFFFFFh
0000E3F8 01D0                <1>     add    eax, edx
0000E3FA 39D8                <1>     cmp    eax, ebx
0000E3FC 7602                <1>     jna    short pix_op_add_24_1
0000E3FE 89D8                <1>     mov    eax, ebx ; 0FFFFFFh  ; Max. value
                             <1> pix_op_add_24_1:
0000E400 66AB                <1>     stosw
0000E402 C1E810              <1>     shr    eax, 16
0000E405 AA                  <1>     stosb
0000E406 E2EC                <1>     loop   pix_op_add_24_0
0000E408 89D0                <1>     mov    eax, edx
0000E40A 5B                  <1>     pop    ebx
0000E40B C3                  <1>     retn
                             <1>
                             <1> pix_op_add_32:
                             <1>     ; 32 bit true colors
                             <1>     ; ADD PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = color
                             <1>     ; edi = start pixel address
                             <1>
0000E40C 89C2                <1>     mov    edx, eax
                             <1> pix_op_add_32_0:
0000E40E 0307                <1>     add    eax, [edi]
0000E410 7303                <1>     jnc    short pix_op_add_32_1
                             <1>     ;mov   eax, 0FFFFFFFFh ; Max. value
0000E412 29C0                <1>     sub    eax, eax
0000E414 48                  <1>     dec    eax
                             <1> pix_op_add_32_1:
0000E415 AB                  <1>     stosd
0000E416 89D0                <1>     mov    eax, edx
0000E418 E2F4                <1>     loop   pix_op_add_32_0
0000E41A C3                  <1>     retn
                             <1>
                             <1> pix_op_sub_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; SUBTRACT PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  al = color
                             <1>     ; edi = start pixel address
                             <1>
0000E41B 88C4                <1>     mov    ah, al
                             <1> pix_op_sub_8_0:
0000E41D 8A07                <1>     mov    al, [edi]
0000E41F 28E0                <1>     sub    al, ah
0000E421 7302                <1>     jnb    short pix_op_sub_8_1
0000E423 30C0                <1>     xor    al, al ; 0 ; Min. value
                             <1> pix_op_sub_8_1:
0000E425 AA                  <1>     stosb
0000E426 E2F5                <1>     loop   pix_op_sub_8_0
0000E428 88E0                <1>     mov    al, ah
0000E42A C3                  <1>     retn
                             <1>
                             <1> pix_op_sub_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; SUBTRACT PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ;  ax = color
```

```
                                <1>     ; edi = start pixel address
                                <1>
0000E42B 89C2                   <1>     mov     edx, eax
                                <1> pix_op_sub_16_0:
0000E42D 668B07                 <1>     mov     ax, [edi]
0000E430 6629D0                 <1>     sub     ax, dx
0000E433 7302                   <1>     jnb     short pix_op_sub_16_1
0000E435 31C0                   <1>     xor     eax, eax ; 0 ; Min. value
                                <1> pix_op_sub_16_1:
0000E437 66AB                   <1>     stosw
0000E439 E2F2                   <1>     loop    pix_op_sub_16_0
0000E43B 89D0                   <1>     mov     eax, edx
0000E43D C3                     <1>     retn
                                <1>
                                <1> pix_op_sub_24:
                                <1>     ; 24 bit true colors
                                <1>     ; SUBTRACT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1>     ;and    eax, 0FFFFFFh
0000E43E 89C2                   <1>     mov     edx, eax
                                <1> pix_op_sub_24_0:
0000E440 8B07                   <1>     mov     eax, [edi]
                                <1>     ; 27/02/2021
0000E442 25FFFFFF00             <1>     and     eax, 0FFFFFFh
0000E447 29D0                   <1>     sub     eax, edx
0000E449 7302                   <1>     jnb     short pix_op_sub_24_1
0000E44B 31C0                   <1>     xor     eax, eax ; 0 ; Min. value
                                <1> pix_op_sub_24_1:
0000E44D 66AB                   <1>     stosw
0000E44F C1E810                 <1>     shr     eax, 16
0000E452 AA                     <1>     stosb
0000E453 E2EB                   <1>     loop    pix_op_sub_24_0
0000E455 89D0                   <1>     mov     eax, edx
0000E457 C3                     <1>     retn
                                <1>
                                <1> pix_op_sub_32:
                                <1>     ; 32 bit true colors
                                <1>     ; SUBTRACT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E458 89C2                   <1>     mov     edx, eax
                                <1> pix_op_sub_32_0:
0000E45A 8B07                   <1>     mov     eax, [edi]
0000E45C 29D0                   <1>     sub     eax, edx
0000E45E 7302                   <1>     jnb     short pix_op_sub_32_1
0000E460 31C0                   <1>     xor     eax, eax ; 0 ; Min. value
                                <1> pix_op_sub_32_1:
0000E462 AB                     <1>     stosd
0000E463 E2F5                   <1>     loop    pix_op_sub_32_0
0000E465 89D0                   <1>     mov     eax, edx
0000E467 C3                     <1>     retn
                                <1>
                                <1> pix_op_or_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; OR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  al = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_or_8_0:
0000E468 0807                   <1>     or      [edi], al
0000E46A 47                     <1>     inc     edi
0000E46B E2FB                   <1>     loop    pix_op_or_8_0
0000E46D C3                     <1>     retn
                                <1>
                                <1> pix_op_or_16:
                                <1>     ; 16 bit colors (65536 colors)
                                <1>     ; OR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  ax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_or_16_0:
0000E46E 660907                 <1>     or      [edi], ax
0000E471 47                     <1>     inc     edi
0000E472 47                     <1>     inc     edi
0000E473 E2F9                   <1>     loop    pix_op_or_16_0
0000E475 C3                     <1>     retn
                                <1>
                                <1> pix_op_or_24:
                                <1>     ; 24 bit true colors
                                <1>     ; OR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E476 89C2                   <1>     mov     edx, eax
                                <1> pix_op_or_24_0:
0000E478 0B07                   <1>     or      eax, [edi]
0000E47A 66AB                   <1>     stosw
0000E47C C1E810                 <1>     shr     eax, 16
0000E47F AA                     <1>     stosb
0000E480 89D0                   <1>     mov     eax, edx
0000E482 E2F4                   <1>     loop    pix_op_or_24_0
0000E484 C3                     <1>     retn
                                <1>
                                <1> pix_op_or_32:
                                <1>     ; 32 bit true colors
                                <1>     ; OR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1>     ;mov    edx, eax
                                <1> pix_op_or_32_0:
                                <1>     ;or     eax, [edi]
                                <1>     ;stosd
                                <1>     ;mov    eax, edx
0000E485 0907                   <1>     or      [edi], eax
0000E487 83C704                 <1>     add     edi, 4
0000E48A E2F9                   <1>     loop    pix_op_or_32_0
0000E48C C3                     <1>     retn
                                <1>
                                <1> pix_op_and_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; AND PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  al = color
                                <1>     ; edi = start pixel address
```

```
                                <1>
                                <1> pix_op_and_8_0:
0000E48D 2007                   <1>     and     [edi], al
0000E48F 47                     <1>     inc     edi
0000E490 E2FB                   <1>     loop    pix_op_and_8_0
0000E492 C3                     <1>     retn
                                <1>
                                <1> pix_op_and_16:
                                <1>     ; 16 bit colors (65536 colors)
                                <1>     ; AND PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  ax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_and_16_0:
0000E493 662107                 <1>     and     [edi], ax
0000E496 47                     <1>     inc     edi
0000E497 47                     <1>     inc     edi
0000E498 E2F9                   <1>     loop    pix_op_and_16_0
0000E49A C3                     <1>     retn
                                <1>
                                <1> pix_op_and_24:
                                <1>     ; 24 bit true colors
                                <1>     ; AND PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E49B 89C2                   <1>     mov     edx, eax
                                <1> pix_op_and_24_0:
0000E49D 2307                   <1>     and     eax, [edi]
0000E49F 66AB                   <1>     stosw
0000E4A1 C1E810                 <1>     shr     eax, 16
0000E4A4 AA                     <1>     stosb
0000E4A5 89D0                   <1>     mov     eax, edx
0000E4A7 E2F4                   <1>     loop    pix_op_and_24_0
0000E4A9 C3                     <1>     retn
                                <1>
                                <1> pix_op_and_32:
                                <1>     ; 32 bit true colors
                                <1>     ; AND PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1>     ;mov    edx, eax
                                <1> pix_op_and_32_0:
                                <1>     ;and    eax, [edi]
                                <1>     ;stosd
                                <1>     ;mov    eax, edx
0000E4AA 2107                   <1>     and     [edi], eax
0000E4AC 83C704                 <1>     add     edi, 4
0000E4AF E2F9                   <1>     loop    pix_op_and_32_0
0000E4B1 C3                     <1>     retn
                                <1>
                                <1> pix_op_xor_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; XOR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  al = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_xor_8_0:
0000E4B2 3007                   <1>     xor     [edi], al
0000E4B4 47                     <1>     inc     edi
0000E4B5 E2FB                   <1>     loop    pix_op_xor_8_0
0000E4B7 C3                     <1>     retn
                                <1>
                                <1> pix_op_xor_16:
                                <1>     ; 16 bit colors (65536 colors)
                                <1>     ; XOR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ;  ax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_xor_16_0:
0000E4B8 663107                 <1>     xor     [edi], ax
0000E4BB 47                     <1>     inc     edi
0000E4BC 47                     <1>     inc     edi
0000E4BD E2F9                   <1>     loop    pix_op_xor_16_0
0000E4BF C3                     <1>     retn
                                <1>
                                <1> pix_op_xor_24:
                                <1>     ; 24 bit true colors
                                <1>     ; XOR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E4C0 89C2                   <1>     mov     edx, eax
                                <1> pix_op_xor_24_0:
0000E4C2 3307                   <1>     xor     eax, [edi]
0000E4C4 66AB                   <1>     stosw
0000E4C6 C1E810                 <1>     shr     eax, 16
0000E4C9 AA                     <1>     stosb
0000E4CA 89D0                   <1>     mov     eax, edx
0000E4CC E2F4                   <1>     loop    pix_op_xor_24_0
0000E4CE C3                     <1>     retn
                                <1>
                                <1> pix_op_xor_32:
                                <1>     ; 32 bit true colors
                                <1>     ; XOR PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
                                <1>     ;mov    edx, eax
                                <1> pix_op_xor_32_0:
                                <1>     ;xor    eax, [edi]
                                <1>     ;stosd
                                <1>     ;mov    eax, edx
0000E4CF 3107                   <1>     xor     [edi], eax
0000E4D1 83C704                 <1>     add     edi, 4
0000E4D4 E2F9                   <1>     loop    pix_op_xor_32_0
0000E4D6 C3                     <1>     retn
                                <1>
                                <1> pix_op_mix_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; MIX (AVERAGE) PIXEL COLORS
                                <1>     ; ecx = pixel count per row
                                <1>     ;  al = color
                                <1>     ; edi = start pixel address
                                <1>
0000E4D7 88C4                   <1>     mov     ah, al
```

```
                                <1> pix_op_mix_8_0:
0000E4D9 0207                   <1>     add     al, [edi]
0000E4DB D0D8                   <1>     rcr     al, 1
0000E4DD AA                     <1>     stosb
0000E4DE 88E0                   <1>     mov     al, ah
0000E4E0 E2F7                   <1>     loop    pix_op_mix_8_0
0000E4E2 C3                     <1>     retn
                                <1>
                                <1> pix_op_mix_16:
                                <1>     ; 16 bit colors (65536 colors)
                                <1>     ; MIX (AVERAGE) PIXEL COLORS
                                <1>     ; ecx = pixel count per row
                                <1>     ;  ax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E4E3 89C2                   <1>     mov     edx, eax
                                <1> pix_op_mix_16_0:
0000E4E5 660307                 <1>     add     ax, [edi]
0000E4E8 66D1D8                 <1>     rcr     ax, 1
0000E4EB 66AB                   <1>     stosw
0000E4ED 89D0                   <1>     mov     eax, edx
0000E4EF E2F4                   <1>     loop    pix_op_mix_16_0
0000E4F1 C3                     <1>     retn
                                <1>
                                <1> pix_op_mix_24:
                                <1>     ; 24 bit true colors
                                <1>     ; MIX (AVERAGE) PIXEL COLORS
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E4F2 53                     <1>     push    ebx
0000E4F3 BBFFFFFF00             <1>     mov     ebx, 0FFFFFFh
                                <1>     ;and    eax, ebx ; 0FFFFFFh
0000E4F8 89C2                   <1>     mov     edx, eax
                                <1> pix_op_mix_24_0:
0000E4FA 8B07                   <1>     mov     eax, [edi]
0000E4FC 21D8                   <1>     and     eax, ebx ; 0FFFFFFh
0000E4FE 01D0                   <1>     add     eax, edx
0000E500 D1E8                   <1>     shr     eax, 1
                                <1>     ;rcr    eax, 1
0000E502 66AB                   <1>     stosw
0000E504 C1E810                 <1>     shr     eax, 16
0000E507 AA                     <1>     stosb
0000E508 E2F0                   <1>     loop    pix_op_mix_24_0
0000E50A 89D0                   <1>     mov     eax, edx
0000E50C 5B                     <1>     pop     ebx
0000E50D C3                     <1>     retn
                                <1>
                                <1> pix_op_mix_32:
                                <1>     ; 32 bit true colors
                                <1>     ; MIX (AVERAGE) PIXEL COLORS
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1>
0000E50E 89C2                   <1>     mov     edx, eax
                                <1> pix_op_mix_32_0:
0000E510 0307                   <1>     add     eax, [edi]
0000E512 D1D8                   <1>     rcr     eax, 1
0000E514 AB                     <1>     stosd
0000E515 89D0                   <1>     mov     eax, edx
0000E517 E2F7                   <1>     loop    pix_op_mix_32_0
0000E519 C3                     <1>     retn
                                <1>
                                <1> pix_op_not_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; NOT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_not_8_0:
0000E51A F617                   <1>     not     byte [edi]
0000E51C 47                     <1>     inc     edi
0000E51D E2FB                   <1>     loop    pix_op_not_8_0
0000E51F C3                     <1>     retn
                                <1>
                                <1> pix_op_not_16:
                                <1>     ; 16 bit colors (65536 colors)
                                <1>     ; NOT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_not_16_0:
0000E520 66F717                 <1>     not     word [edi]
0000E523 47                     <1>     inc     edi
0000E524 47                     <1>     inc     edi
0000E525 E2F9                   <1>     loop    pix_op_not_16_0
0000E527 C3                     <1>     retn
                                <1>
                                <1> pix_op_not_24:
                                <1>     ; 24 bit true colors
                                <1>     ; NOT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_not_24_0:
0000E528 66F717                 <1>     not     word [edi]
0000E52B 47                     <1>     inc     edi
0000E52C 47                     <1>     inc     edi
0000E52D F617                   <1>     not     byte [edi]
0000E52F 47                     <1>     inc     edi
0000E530 E2F6                   <1>     loop    pix_op_not_24_0
0000E532 C3                     <1>     retn
                                <1>
                                <1> pix_op_not_32:
                                <1>     ; 32 bit true colors
                                <1>     ; NOT PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; eax = color
                                <1>     ; edi = start pixel address
                                <1> pix_op_not_32_0:
0000E533 F717                   <1>     not     dword [edi]
0000E535 83C704                 <1>     add     edi, 4
0000E538 E2F9                   <1>     loop    pix_op_not_32_0
0000E53A C3                     <1>     retn
                                <1>
                                <1> pix_op_neg_8:
                                <1>     ; 8 bit colors (256 colors)
                                <1>     ; NEG PIXEL COLOR
                                <1>     ; ecx = pixel count per row
                                <1>     ; edi = start pixel address
                                <1>
                                <1> pix_op_neg_8_0:
```

```
0000E53B F61F                <1>     neg     byte [edi]
0000E53D 47                  <1>     inc     edi
0000E53E E2FB                <1>     loop    pix_op_neg_8_0
0000E540 C3                  <1>     retn
                             <1>
                             <1> pix_op_neg_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; NEG PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_neg_16_0:
0000E541 66F71F              <1>     neg     word [edi]
0000E544 47                  <1>     inc     edi
0000E545 47                  <1>     inc     edi
0000E546 E2F9                <1>     loop    pix_op_neg_16_0
0000E548 C3                  <1>     retn
                             <1>
                             <1> pix_op_neg_24:
                             <1>     ; 24 bit true colors
                             <1>     ; NEG PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_neg_24_0:
0000E549 8B07                <1>     mov     eax, [edi]
0000E54B 25FFFFFF00          <1>     and     eax, 0FFFFFFh
0000E550 F7D8                <1>     neg     eax
0000E552 66AB                <1>     stosw
0000E554 C1E810              <1>     shr     eax, 16
0000E557 AA                  <1>     stosb
0000E558 E2EF                <1>     loop    pix_op_neg_24_0
0000E55A C3                  <1>     retn
                             <1>
                             <1> pix_op_neg_32:
                             <1>     ; 32 bit true colors
                             <1>     ; NEG PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = color
                             <1>     ; edi = start pixel address
                             <1> pix_op_neg_32_0:
0000E55B F71F                <1>     neg     dword [edi]
0000E55D 83C704              <1>     add     edi, 4
0000E560 E2F9                <1>     loop    pix_op_neg_32_0
0000E562 C3                  <1>     retn
                             <1>
                             <1> pix_op_inc_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; INCREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_inc_8_0:
0000E563 FE07                <1>     inc     byte [edi]
0000E565 7502                <1>     jnz     short pix_op_inc_8_1
                             <1>     ;mov    [edi], 0FFh ; Max. value
0000E567 FE0F                <1>     dec     byte [edi]
                             <1> pix_op_inc_8_1:
0000E569 47                  <1>     inc     edi
0000E56A E2F7                <1>     loop    pix_op_inc_8_0
0000E56C C3                  <1>     retn
                             <1>
                             <1> pix_op_inc_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; INCREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_inc_16_0:
0000E56D 66FF07              <1>     inc     word [edi]
0000E570 7503                <1>     jnz     short pix_op_inc_16_1
                             <1>     ;mov    word [edi], 0FFFFh ; Max. value
0000E572 66FF0F              <1>     dec     word [edi]
                             <1> pix_op_inc_16_1:
0000E575 47                  <1>     inc     edi
0000E576 47                  <1>     inc     edi
0000E577 E2F4                <1>     loop    pix_op_inc_16_0
0000E579 C3                  <1>     retn
                             <1>
                             <1> pix_op_inc_24:
                             <1>     ; 24 bit true colors
                             <1>     ; INCREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_inc_24_0:
0000E57A 8B07                <1>     mov     eax, [edi]
0000E57C 40                  <1>     inc     eax
0000E57D 25FFFFFF00          <1>     and     eax, 0FFFFFFh
0000E582 7501                <1>     jnz     short pix_op_inc_24_1
                             <1>     ;mov    eax, 0FFFFFFh  ; Max. value
0000E584 48                  <1>     dec     eax ; 0FFFFFFFFh
                             <1> pix_op_inc_24_1:
0000E585 66AB                <1>     stosw
0000E587 C1E810              <1>     shr     eax, 16
0000E58A AA                  <1>     stosb
0000E58B E2ED                <1>     loop    pix_op_inc_24_0
0000E58D C3                  <1>     retn
                             <1>
                             <1> pix_op_inc_32:
                             <1>     ; 32 bit true colors
                             <1>     ; INCREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_inc_32_0:
0000E58E FF07                <1>     inc     dword [edi]
0000E590 7502                <1>     jnz     short pix_op_inc_32_1
                             <1>     ;mov    dword [edi], 0FFFFFFFFh ; Max. value
0000E592 FF0F                <1>     dec     dword [edi]
                             <1> pix_op_inc_32_1:
0000E594 83C704              <1>     add     edi, 4
0000E597 E2F5                <1>     loop    pix_op_inc_32_0
0000E599 C3                  <1>     retn
                             <1>
                             <1> pix_op_dec_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; DECREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_dec_8_0:
0000E59A FE0F                <1>     dec     byte [edi]
0000E59C 7902                <1>     jns     short pix_op_dec_8_1
```

```
0000E59E FE07                <1>     inc     byte [edi] ; 0 ; Min. value
                             <1> pix_op_dec_8_1:
0000E5A0 47                  <1>     inc     edi
0000E5A1 E2F7                <1>     loop    pix_op_dec_8_0
0000E5A3 C3                  <1>     retn
                             <1>
                             <1> pix_op_dec_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; DECREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_dec_16_0:
0000E5A4 66FF0F              <1>     dec     word [edi]
0000E5A7 7903                <1>     jns     short pix_op_dec_16_1
0000E5A9 66FF07              <1>     inc     word [edi] ; 0 ; Min. value
                             <1> pix_op_dec_16_1:
0000E5AC 47                  <1>     inc edi
0000E5AD 47                  <1>     inc edi
0000E5AE E2F4                <1>     loop    pix_op_dec_16_0
0000E5B0 C3                  <1>     retn
                             <1>
                             <1> pix_op_dec_24:
                             <1>     ; 24 bit true colors
                             <1>     ; DECREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_dec_24_0:
0000E5B1 8B07                <1>     mov     eax, [edi]
0000E5B3 25FFFFFF00          <1>     and     eax, 0FFFFFFh
0000E5B8 7401                <1>     jz      short pix_op_dec_24_1
                             <1>                     ; 0 ; Min. value
0000E5BA 48                  <1>     dec     eax
                             <1> pix_op_dec_24_1:
0000E5BB 66AB                <1>     stosw
0000E5BD C1E810              <1>     shr     eax, 16
0000E5C0 AA                  <1>     stosb
0000E5C1 E2B7                <1>     loop    pix_op_inc_24_0
0000E5C3 C3                  <1>     retn
                             <1>
                             <1> pix_op_dec_32:
                             <1>     ; 32 bit true colors
                             <1>     ; DECREASE PIXEL COLOR
                             <1>     ; ecx = pixel count per row
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_dec_32_0:
0000E5C4 FF0F                <1>     dec     dword [edi]
0000E5C6 7902                <1>     jns     short pix_op_dec_32_1
0000E5C8 FF07                <1>     inc     dword [edi] ; 0 ; Min. value
                             <1> pix_op_dec_32_1:
0000E5CA 83C704              <1>     add     edi, 4
0000E5CD E2F5                <1>     loop    pix_op_dec_32_0
0000E5CF 89D0                <1>     mov     eax, edx
0000E5D1 C3                  <1>     retn
                             <1>
                             <1> pix_op_rpl_8:
                             <1>     ; 8 bit colors (256 colors)
                             <1>     ; REPLACE PIXEL COLORS
                             <1>     ; ecx = pixel count per row
                             <1>     ;  al = new color
                             <1>     ; byte [maskcolor] = old color
                             <1>     ; edi = start pixel address
                             <1>
0000E5D2 8A25[A29D0100]      <1>     mov     ah, [maskcolor]
                             <1> pix_op_rpl_8_0:
0000E5D8 3A27                <1>     cmp     ah, [edi]
0000E5DA 7502                <1>     jne     short pix_op_rpl_8_1
0000E5DC 8807                <1>     mov     [edi], al
                             <1> pix_op_rpl_8_1:
0000E5DE 47                  <1>     inc     edi
0000E5DF E2F7                <1>     loop    pix_op_rpl_8_0
0000E5E1 C3                  <1>     retn
                             <1>
                             <1> pix_op_rpl_16:
                             <1>     ; 16 bit colors (65536 colors)
                             <1>     ; REPLACE PIXEL COLORS
                             <1>     ; ecx = pixel count per row
                             <1>     ;  ax = new color
                             <1>     ; word [maskcolor] = old color
                             <1>     ; edi = start pixel address
                             <1>
0000E5E2 8B15[A29D0100]      <1>     mov     edx, [maskcolor]
                             <1> pix_op_rpl_16_0:
0000E5E8 663B17              <1>     cmp     dx, [edi]
0000E5EB 7503                <1>     jne     short pix_op_rpl_16_1
0000E5ED 668907              <1>     mov     [edi], ax
                             <1> pix_op_rpl_16_1:
0000E5F0 47                  <1>     inc     edi
0000E5F1 47                  <1>     inc     edi
0000E5F2 E2F4                <1>     loop    pix_op_rpl_16_0
0000E5F4 C3                  <1>     retn
                             <1>
                             <1> pix_op_rpl_24:
                             <1>     ; 24 bit true colors
                             <1>     ; REPLACE PIXEL COLORS
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = new color
                             <1>     ; [maskcolor] = old color
                             <1>     ; edi = start pixel address
                             <1>
                             <1> pix_op_rpl_24_0:
0000E5F5 8B17                <1>     mov     edx, [edi]
0000E5F7 81E2FFFFFF00        <1>     and     edx, 0FFFFFFh
0000E5FD 3B15[A29D0100]      <1>     cmp     edx, [maskcolor]
0000E603 7406                <1>     je      short pix_op_rpl_24_1
0000E605 83C703              <1>     add     edi, 3
0000E608 E2EB                <1>     loop    pix_op_rpl_24_0
0000E60A C3                  <1>     retn
                             <1> pix_op_rpl_24_1:
0000E60B AA                  <1>     stosb
0000E60C C1C808              <1>     ror     eax, 8
0000E60F 66AB                <1>     stosw
0000E611 C1C008              <1>     rol     eax, 8
0000E614 E2DF                <1>     loop    pix_op_rpl_24_0
0000E616 C3                  <1>     retn
                             <1>
                             <1> pix_op_rpl_32:
                             <1>     ; 32 bit true colors
                             <1>     ; REPLACE PIXEL COLORS
                             <1>     ; ecx = pixel count per row
                             <1>     ; eax = new color
                             <1>     ; [maskcolor] = old color
```

```
                                <1>     ; edi = start pixel address
                                <1>
0000E617 8B15[A29D0100]         <1>     mov     edx, [maskcolor]
                                <1> pix_op_rpl_32_0:
0000E61D 3B17                   <1>     cmp     edx, [edi]
0000E61F 7504                   <1>     jne     short pix_op_rpl_32_2
0000E621 AB                     <1>     stosd
0000E622 E2F9                   <1>     loop    pix_op_rpl_32_0
0000E624 C3                     <1>     retn
                                <1> pix_op_rpl_32_2:
0000E625 83C704                 <1>     add     edi, 4
0000E628 E2F3                   <1>     loop    pix_op_rpl_32_0
0000E62A C3                     <1>     retn
                                <1>
                                <1> pix_op_chr:
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 15/02/2021
                                <1>     ; 05/02/2021
                                <1>     ; WRITE CHARACTER (FONT)
                                <1>     ; 05/01/2021 ([ufont])
                                <1>     ; 01/01/2021
                                <1>     ;     CL = char's color (8 bit, 256 colors)
                                <1>     ;    ECX = char's color (16 bit and true colors)
                                <1>     ;     DL = Character's ASCII code
                                <1>     ;     DH bit 0 -> font height
                                <1>     ;        0 -> 8x16 character font
                                <1>     ;        1 -> 8x8 character font
                                <1>     ;     DH bit 1 & 2 -> scale
                                <1>     ;        0 = 1/1 (8 pixels per char row)
                                <1>     ;        1 = 2/1 (16 pixels per char row)
                                <1>     ;        2 = 3/1 (24 pixels per char row)
                                <1>     ;        3 = 4/1 (32 pixels per char row)
                                <1>     ;     DH bit 6 -> [ufont] option (1 = use [ufont])
                                <1>     ;     If DH bit 7 = 1
                                <1>     ;       USER FONT (from user buffer)
                                <1>     ;          DL = 0 -> 8x8 (width: 1 byte per row)
                                <1>     ;          DL = 1 -> 8x16
                                <1>     ;          DL = 2 -> 16x16 (width: 2 bytes)
                                <1>     ;          DL = 3 -> 16x32
                                <1>     ;          DL = 4 -> 24x24 (width: 3 bytes)
                                <1>     ;          DL = 5 -> 24x48
                                <1>     ;          DL = 6 -> 32x32 (width: 4 bytes)
                                <1>     ;          DL = 7 -> 32x64
                                <1>     ;          DL > 7 -> invalid (unused)
                                <1>     ;       EDI = user's font buffer address
                                <1>     ;          (NOTE: byte order is as row0,row1,row2..)
                                <1>     ;     ESI = start position (row, column) (*)
                                <1>     ;           (HW = row, SI = column)
                                <1>
0000E62B 89F0                   <1>     mov     eax, esi ; start position
0000E62D E856F2FFFF             <1>     call    calc_pixel_offset
0000E632 3B05[969D0100]         <1>     cmp     eax, [v_siz]
0000E638 736D                   <1>     jnb     short pix_op_chr_err ; out of display page!
0000E63A E826F2FFFF             <1>     call    pixels_to_byte_count
                                <1>     ; eax = font start offset
0000E63F 0305[929D0100]         <1>     add     eax, [v_mem] ; LFB start address
0000E645 A3[9A9D0100]           <1>     mov     [v_str], eax ; font start address
                                <1>
0000E64A 890D[A29D0100]         <1>     mov     [maskcolor], ecx ; save char's color
                                <1>
0000E650 8835[909D0100]         <1>     mov     [v_ops], dh
                                <1>
0000E656 81E6FFFF0000           <1>     and     esi, 0FFFFh
0000E65C 8935[AA9D0100]         <1>     mov     [buffer8], esi ; start column
                                <1>
0000E662 31DB                   <1>     xor     ebx, ebx ; 0
0000E664 31C0                   <1>     xor     eax, eax ; 15/02/2021
                                <1>
0000E666 F6C680                 <1>     test    dh, 80h
0000E669 7577                   <1>     jnz     short pix_op_chr_u ; user font
                                <1>
0000E66B 80E63F                 <1>     and     dh, 3Fh ; clear bit 6, [UFONT] option bit
0000E66E 7409                   <1>     jz      short pix_op_chr_0
                                <1>
0000E670 80FE07                 <1>     cmp     dh, 7
0000E673 7732                   <1>     ja      short pix_op_chr_err
                                <1>                     ; invalid (undefined) option
0000E675 88F4                   <1>     mov     ah, dh
0000E677 D0EC                   <1>     shr     ah, 1
                                <1>     ; ah = 0 to 3, scale
                                <1>     ;jmp    short pix_op_chr_font_pixels
                                <1>
                                <1> pix_op_chr_font_pixels:
                                <1>     ; 05/02/2021
                                <1>     ; write scaled font to buffer
                                <1>
                                <1>     ; DL = ASCII code of character
                                <1>     ; AH = scale
                                <1>     ; EDI = buffer address (kernel)
                                <1>
                                <1> pix_op_chr_0:
0000E679 88D3                   <1>     mov     bl, dl ; 15/02/2021
0000E67B 31C9                   <1>     xor     ecx, ecx
0000E67D B610                   <1>     mov     dh, 16
0000E67F F605[909D0100]01       <1>     test    byte [v_ops], 1 ; 8x8 font ?
0000E686 7428                   <1>     jz      short pix_op_chr_2 ; 8x16 font
0000E688 B608                   <1>     mov     dh, 8
0000E68A C1E303                 <1>     shl     ebx, 3 ; * 8
0000E68D F605[909D0100]40       <1>     test    byte [v_ops], 40h ; [ufont] option
0000E694 7412                   <1>     jz      short pix_op_chr_1  ; no
                                <1>     ; test 8x8 user font is ready flag
0000E696 F605[869D0100]01       <1>     test    byte [ufont], 1
0000E69D 7409                   <1>     jz      short pix_op_chr_1 ; no
0000E69F 81C300500900           <1>     add     ebx, VGAFONT8USER
0000E6A5 EB2C                   <1>     jmp     short pix_op_chr_fpos_0
                                <1> pix_op_chr_err:
0000E6A7 C3                     <1>     retn
                                <1> pix_op_chr_1:
0000E6A8 81C3[404D0100]         <1>     add     ebx, vgafont8 ; system font (8x8)
0000E6AE EB23                   <1>     jmp     short pix_op_chr_fpos_0
                                <1> pix_op_chr_2:
0000E6B0 C1E304                 <1>     shl     ebx, 4 ; * 16
0000E6B3 F605[909D0100]40       <1>     test    byte [v_ops], 40h ; [ufont] option
0000E6BA 7411                   <1>     jz      short pix_op_chr_3  ; no
                                <1>     ; test 8x16 user font is ready flag
0000E6BC F605[869D0100]02       <1>     test    byte [ufont], 2
0000E6C3 7408                   <1>     jz      short pix_op_chr_3 ; no
0000E6C5 81C300400900           <1>     add     ebx, VGAFONT16USER
0000E6CB EB06                   <1>     jmp     short pix_op_chr_fpos_0
                                <1> pix_op_chr_3:
0000E6CD 81C3[40630100]         <1>     add     ebx, vgafont16 ; system font (8x16)
                                <1> pix_op_chr_fpos_0:
0000E6D3 20E4                   <1>     and     ah, ah
0000E6D5 754A                   <1>     jnz     short pix_op_chr_fpos_1 ; scale > 1
```

```
                                <1>     ; no scale (scale = 1)
0000E6D7 89DE                   <1>     mov     esi, ebx ; 15/02/2021
0000E6D9 88F1                   <1>     mov     cl, dh ; rows/height (16 or 8)
0000E6DB B608                   <1>     mov     dh, 8  ; columns/width
0000E6DD E9CB000000             <1>     jmp     pix_op_chr_f2p
                                <1> pix_op_chr_u:
                                <1>     ; write user defined font
0000E6E2 80FE80                 <1>     cmp     dh, 80h
0000E6E5 75C0                   <1>     jne     short pix_op_chr_err
0000E6E7 80FA07                 <1>     cmp     dl, 7
0000E6EA 77BB                   <1>     ja      short pix_op_chr_err
                                <1>
                                <1>     ; 16/02/2021
0000E6EC 89FE                   <1>     mov     esi, edi ; user's font buffer
                                <1>
                                <1>     ;xor    eax, eax
                                <1>     ; eax = 0 ; 15/02/2021
0000E6EE 88D4                   <1>     mov     ah, dl
0000E6F0 D0EC                   <1>     shr     ah, 1
0000E6F2 FEC4                   <1>     inc     ah
                                <1>     ; ah =  1 to 4
0000E6F4 88E0                   <1>     mov     al, ah
0000E6F6 C0E003                 <1>     shl     al, 3 ; * 8
                                <1>     ; al = 8,16,24,32
0000E6F9 88C3                   <1>     mov     bl, al
0000E6FB 88C7                   <1>     mov     bh, al
0000E6FD F6E4                   <1>     mul     ah
                                <1>     ; ax = 8,32,72,128 bytes
0000E6FF F6C201                 <1>     test    dl, 1
0000E702 7404                   <1>     jz      short pix_op_chr_u_0
                                <1>     ;shl    ax, 1 ; *2
                                <1>     ; 23/07/2022
0000E704 D1E0                   <1>     shl     eax, 1
                                <1>     ; ax = 16,32,144,256 bytes
0000E706 D0E7                   <1>     shl     bh, 1
                                <1> pix_op_chr_u_0:
                                <1>     ; eax = byte count
0000E708 89C1                   <1>     mov     ecx, eax
0000E70A BF00760900             <1>     mov     edi, VBE3SAVERESTOREBLOCK
                                <1>     ; esi = user buffer
0000E70F E851240000             <1>     call    transfer_from_user_buffer
0000E714 7291                   <1>     jc      short pix_op_chr_err
                                <1>
0000E716 88F9                   <1>     mov     cl, bh ; rows/height
0000E718 88DE                   <1>     mov     dh, bl ; columns (width)
0000E71A 89FE                   <1>     mov     esi, edi ; VBE3SAVERESTOREBLOCK
0000E71C E98C000000             <1>     jmp     pix_op_chr_f2p
                                <1>
                                <1> pix_op_chr_fpos_1:
                                <1>     ; 18/02/2021
                                <1>     ; scale > 1
0000E721 88F5                   <1>     mov     ch, dh ; 16 or 8
0000E723 BF00760900             <1>     mov     edi, VBE3SAVERESTOREBLOCK
0000E728 89FE                   <1>     mov     esi, edi
0000E72A FECC                   <1>     dec     ah
0000E72C 7522                   <1>     jnz     short pix_op_chr_fpos_5 ; scale > 2
                                <1>     ; scale = 2
                                <1> pix_op_chr_fpos_2:
0000E72E B108                   <1>     mov     cl, 8
0000E730 8A13                   <1>     mov     dl, [ebx]
                                <1> pix_op_chr_fpos_3:
                                <1>     ;shl    ax, 2
                                <1>     ; 23/07/2022
0000E732 C1E002                 <1>     shl     eax, 2
0000E735 D0E2                   <1>     shl     dl, 1
0000E737 7302                   <1>     jnc     short pix_op_chr_fpos_4
0000E739 0C03                   <1>     or      al, 3
                                <1> pix_op_chr_fpos_4:
0000E73B FEC9                   <1>     dec     cl
0000E73D 75F3                   <1>     jnz     short pix_op_chr_fpos_3
0000E73F 66AB                   <1>     stosw
                                <1>     ; 18/02/2021
0000E741 66AB                   <1>     stosw
0000E743 43                     <1>     inc     ebx
0000E744 FECD                   <1>     dec     ch
0000E746 75E6                   <1>     jnz     short pix_op_chr_fpos_2
                                <1>     ; scale = 2
0000E748 88F1                   <1>     mov     cl, dh ; 16 or 8 (height/rows)
0000E74A D0E1                   <1>     shl     cl, 1  ; 32 or 16 rows
0000E74C B610                   <1>     mov     dh, 16 ; columns (width)
0000E74E EB5D                   <1>     jmp     short pix_op_chr_f2p
                                <1> pix_op_chr_fpos_5:
0000E750 FECC                   <1>     dec     ah
0000E752 7538                   <1>     jnz     short pix_op_chr_fpos_9 ; scale = 4
                                <1>     ; scale = 3
                                <1> pix_op_chr_fpos_6:
0000E754 B108                   <1>     mov     cl, 8
0000E756 8A13                   <1>     mov     dl, [ebx]
                                <1> pix_op_chr_fpos_7:
0000E758 C1E003                 <1>     shl     eax, 3
0000E75B D0E2                   <1>     shl     dl, 1 ; 18/02/2021
0000E75D 7302                   <1>     jnc     short pix_op_chr_fpos_8
0000E75F 0C07                   <1>     or      al, 7
                                <1> pix_op_chr_fpos_8:
0000E761 FEC9                   <1>     dec     cl
0000E763 75F3                   <1>     jnz     short pix_op_chr_fpos_7
0000E765 66AB                   <1>     stosw
                                <1>     ; 18/02/2021
0000E767 C1C810                 <1>     ror     eax, 16
0000E76A AA                     <1>     stosb
0000E76B C1C010                 <1>     rol     eax, 16
0000E76E 66AB                   <1>     stosw
0000E770 C1C810                 <1>     ror     eax, 16
0000E773 AA                     <1>     stosb
0000E774 C1C010                 <1>     rol     eax, 16
0000E777 66AB                   <1>     stosw
0000E779 C1E810                 <1>     shr     eax, 16 ; 27/02/2021
0000E77C AA                     <1>     stosb
0000E77D 43                     <1>     inc     ebx
                                <1>     ; 18/02/2021
0000E77E FECD                   <1>     dec     ch
0000E780 75D2                   <1>     jnz     short pix_op_chr_fpos_6
                                <1>     ; scale = 3
0000E782 88F1                   <1>     mov     cl, dh ; 16 or 8 (height/rows)
0000E784 D0E1                   <1>     shl     cl, 1
0000E786 00F1                   <1>     add     cl, dh ; 48 or 24 rows
0000E788 B618                   <1>     mov     dh, 24 ; columns (width)
0000E78A EB21                   <1>     jmp     short pix_op_chr_f2p
                                <1>
                                <1> pix_op_chr_fpos_9:
                                <1>     ; scale = 4
0000E78C B108                   <1>     mov     cl, 8
0000E78E 8A13                   <1>     mov     dl, [ebx]
                                <1> pix_op_chr_fpos_10:
```

```
                                <1>     ; 18/02/2021
0000E790 C1E004                 <1>     shl     eax, 4
0000E793 D0E2                   <1>     shl     dl, 1 ; 18/02/2021
0000E795 7302                   <1>     jnc     short pix_op_chr_fpos_11
0000E797 0C0F                   <1>     or      al, 0Fh ; or al, 15
                                <1> pix_op_chr_fpos_11:
0000E799 FEC9                   <1>     dec     cl
0000E79B 75F3                   <1>     jnz     short pix_op_chr_fpos_10
0000E79D AB                     <1>     stosd
                                <1>     ; 18/02/2021
0000E79E AB                     <1>     stosd
0000E79F AB                     <1>     stosd
0000E7A0 AB                     <1>     stosd
0000E7A1 43                     <1>     inc     ebx
0000E7A2 FECD                   <1>     dec     ch
0000E7A4 75E6                   <1>     jnz     short pix_op_chr_fpos_9
                                <1>     ; scale = 4
0000E7A6 88F1                   <1>     mov     cl, dh ; 16 or 8 (height/rows)
0000E7A8 C0E102                 <1>     shl     cl, 2 ; 64 or 32 rows
0000E7AB B620                   <1>     mov     dh, 32 ; columns (width)
                                <1>     ;jmp    short pix_op_chr_f2p
                                <1>
                                <1> pix_op_chr_f2p:
                                <1>     ; write font pixels
0000E7AD 8B3D[9A9D0100]         <1>     mov     edi, [v_str]
                                <1>     ; 15/02/2021
                                <1> pix_op_chr_f2p_next:
0000E7B3 80FE08                 <1>     cmp     dh, 8
0000E7B6 7706                   <1>     ja      short pix_op_chr_f2p_24
                                <1> pix_op_chr_f2p_8:
0000E7B8 AC                     <1>     lodsb
0000E7B9 C1E018                 <1>     shl     eax, 24 ; 15/02/2021
0000E7BC EB16                   <1>     jmp     short pix_op_chr_f2p_0
                                <1> pix_op_chr_f2p_24:
0000E7BE 80FE18                 <1>     cmp     dh, 24
0000E7C1 7710                   <1>     ja      short pix_op_chr_f2p_32
0000E7C3 7207                   <1>     jb      short pix_op_chr_f2p_16
                                <1>     ; 27/02/2021
0000E7C5 AD                     <1>     ;mov    eax, [esi]
0000E7C6 C1E008                 <1>     lodsd
                                <1>     shl     eax, 8
0000E7C9 4E                     <1>     ;add    esi, 3
0000E7CA EB08                   <1>     dec     esi
                                <1>     jmp     short pix_op_chr_f2p_0
                                <1> pix_op_chr_f2p_16:
0000E7CC 66AD                   <1>     lodsw
0000E7CE C1E010                 <1>     shl     eax, 16 ; 15/02/2021
0000E7D1 EB01                   <1>     jmp     short pix_op_chr_f2p_0
                                <1> pix_op_chr_f2p_32:
0000E7D3 AD                     <1>     lodsd
                                <1> pix_op_chr_f2p_0:
                                <1>     ; EAX = font row (8,16,24,32 pixels)
                                <1>     ;       (bits are shifted to left)
                                <1>     ; CL = rows
                                <1>     ; DH = bits per row (8,16,24,32)
0000E7D4 8B1D[AA9D0100]         <1>     mov     ebx, [buffer8] ; start column
0000E7DA 57                     <1>     push    edi ; *
0000E7DB 52                     <1>     push    edx ; **
                                <1> pix_op_chr_f2p_1:
0000E7DC E82E000000             <1>     call    pix_op_chr_w_pixel
                                <1> pix_op_chr_f2p_2:
0000E7E1 663B1D[8E9D0100]       <1>     cmp     bx, [v_width] ; current column
0000E7E8 7304                   <1>     jnb     short pix_op_chr_f2p_3
0000E7EA FECE                   <1>     dec     dh
0000E7EC 75EE                   <1>     jnz     short pix_op_chr_f2p_1 ; next bit
                                <1> pix_op_chr_f2p_3:
                                <1>     ;mov    ebx, [buffer8]
0000E7EE 5A                     <1>     pop     edx ; **
0000E7EF 58                     <1>     pop     eax ; *
0000E7F0 3B3D[9E9D0100]         <1>     cmp     edi, [v_end]
0000E7F6 7316                   <1>     jnb     short pix_op_chr_f2p_4
0000E7F8 FEC9                   <1>     dec     cl
0000E7FA 7412                   <1>     jz      short pix_op_chr_f2p_4
                                <1>     ; 27/02/2021
0000E7FC 89C7                   <1>     mov     edi, eax
0000E7FE 0FB705[8E9D0100]       <1>     movzx   eax, word [v_width]
0000E805 E85BF0FFFF             <1>     call    pixels_to_byte_count
0000E80A 01C7                   <1>     add     edi, eax ; next position
0000E80C EBA5                   <1>     jmp     short pix_op_chr_f2p_next
                                <1> pix_op_chr_f2p_4:
0000E80E C3                     <1>     retn
                                <1>
                                <1> pix_op_chr_w_pixel:
                                <1>     ; 15/02/2021
0000E80F 89C5                   <1>     mov     ebp, eax
0000E811 A1[A29D0100]           <1>     mov     eax, [maskcolor]
0000E816 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E81D 7711                   <1>     ja      short pix_op_chr_wp_2
                                <1>     ; 256 colors (1 byte per pixel)
0000E81F D1E5                   <1>     shl     ebp, 1
0000E821 7302                   <1>     jnc     short pix_op_chr_wp_0
0000E823 8807                   <1>     mov     [edi], al
                                <1> pix_op_chr_wp_0:
0000E825 47                     <1>     inc     edi
0000E826 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> pix_op_chr_wp_1:
0000E82C 43                     <1>     inc     ebx
0000E82D 89E8                   <1>     mov     eax, ebp
0000E82F C3                     <1>     retn
                                <1> pix_op_chr_wp_2:
0000E830 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000E837 772E                   <1>     ja      short pix_op_chr_wp_6 ; 32bpp
0000E839 721C                   <1>     jb      short pix_op_chr_wp_4 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ; * 3
0000E83B D1E5                   <1>     shl     ebp, 1
0000E83D 7309                   <1>     jnc     short pix_op_chr_wp_3
0000E83F 668907                 <1>     mov     [edi], ax
0000E842 C1E810                 <1>     shr     eax, 16
0000E845 884702                 <1>     mov     [edi+2], al
                                <1> pix_op_chr_wp_3:
0000E848 B803000000             <1>     mov     eax, 3 ; 27/02/2021
0000E84D 01C7                   <1>     add     edi, eax  ; add edi, 3
0000E84F 0105[6C8E0100]         <1>     add     [u.r0], eax ; +3
                                <1>
0000E855 EBD5                   <1>     jmp     short pix_op_chr_wp_1
                                <1>
                                <1> pix_op_chr_wp_4:
                                <1>     ; 16 bit (65536) colors
0000E857 D1E5                   <1>     shl     ebp, 1
0000E859 7303                   <1>     jnc     short pix_op_chr_wp_5
0000E85B 668907                 <1>     mov     [edi], ax
                                <1> pix_op_chr_wp_5:
0000E85E 47                     <1>     inc     edi
```

```
0000E85F FF05[6C8E0100]      <1>     inc    dword [u.r0] ; +1
0000E865 EBBE                <1>     jmp    short pix_op_chr_wp_0
                             <1>
                             <1> pix_op_chr_wp_6:
                             <1>     ; 32 bit true colors
0000E867 D1E5                <1>     shl    ebp, 1
0000E869 7302                <1>     jnc    short pix_op_chr_wp_7
0000E86B 8907                <1>     mov    [edi], eax
                             <1> pix_op_chr_wp_7:
0000E86D 31C0                <1>     xor    eax, eax
0000E86F B004                <1>     mov    al, 4
0000E871 01C7                <1>     add    edi, eax ; add edi, 4
0000E873 0105[6C8E0100]      <1>     add    [u.r0], eax ; +4
0000E879 EBB1                <1>     jmp    short pix_op_chr_wp_1
                             <1>
                             <1> m_pix_op_cpy:
                             <1>     ; 26/02/2021
                             <1>     ; 06/02/2021
                             <1>     ; MASKED COPY PIXELS (full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_cpy
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = transfer count (bytes)
                             <1>     ;   edi = [v_mem] = start address of LFB
                             <1>     ;   esi = user's buffer address (virtual)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;   [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Full screen masked copy
                             <1>
                             <1>     ; Modified regs: eax, ebx, edx, esi, edi, ecx
                             <1>
                             <1> m_pix_op_cpy_0:
                             <1>     ;push  ebx ; *** ; 26/02/2021
0000E87B 57                  <1>     push   edi ; **
0000E87C 51                  <1>     push   ecx ; *
0000E87D 81F9F8070000        <1>     cmp    ecx, 2040 ; (3*680) ; 26/02/2021
0000E883 7605                <1>     jna    short m_pix_op_cpy_1
0000E885 B9F8070000          <1>     mov    ecx, 2040
                             <1> m_pix_op_cpy_1:
0000E88A BF00760900          <1>     mov    edi, VBE3SAVERESTOREBLOCK ; temporary buff
0000E88F E8D1220000          <1>     call   transfer_from_user_buffer
0000E894 726C                <1>     jc     short m_pix_op_cpy_3
0000E896 01CE                <1>     add    esi, ecx
0000E898 89F5                <1>     mov    ebp, esi  ; save user's buffer address
0000E89A 89FE                <1>     mov    esi, edi
0000E89C 89CB                <1>     mov    ebx, ecx
0000E89E 59                  <1>     pop    ecx ; *
0000E89F 29D9                <1>     sub    ecx, ebx
0000E8A1 5F                  <1>     pop    edi ; **
0000E8A2 31D2                <1>     xor    edx, edx ; 26/02/2021
0000E8A4 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8
0000E8AB 7435                <1>     je     short m_pix_op_cpy_1_8
0000E8AD 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24
0000E8B4 776D                <1>     ja     short m_pix_op_cpy_1_32
0000E8B6 724D                <1>     jb     short m_pix_op_cpy_1_16
                             <1> m_pix_op_cpy_1_24:
                             <1>     ; 24 bit masked copy
                             <1>     ;mov   edx, 3
0000E8B8 B203                <1>     mov    dl, 3 ; 26/02/2021
                             <1> m_pix_op_cpy_1_24_0:
0000E8BA 66AD                <1>     lodsw
0000E8BC C1E010              <1>     shl    eax, 16
0000E8BF AC                  <1>     lodsb
0000E8C0 C1C010              <1>     rol    eax, 16
0000E8C3 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000E8C9 740F                <1>     je     short m_pix_op_cpy_1_24_1 ; exclude
0000E8CB 668907              <1>     mov    [edi], ax
0000E8CE C1E810              <1>     shr    eax, 16
0000E8D1 884702              <1>     mov    [edi+2], al
0000E8D4 0115[6C8E0100]      <1>     add    [u.r0], edx ; +3
                             <1> m_pix_op_cpy_1_24_1:
0000E8DA 01D7                <1>     add    edi, edx ; +3
0000E8DC 29D3                <1>     sub    ebx, edx ; sub ebx, 3
0000E8DE 77DA                <1>     ja     short m_pix_op_cpy_1_24_0
0000E8E0 EB15                <1>     jmp    short m_pix_op_cpy_2
                             <1>
                             <1> m_pix_op_cpy_1_8:
                             <1>     ; 8 bit masked copy
0000E8E2 AC                  <1>     lodsb
0000E8E3 3A05[A29D0100]      <1>     cmp    al, [maskcolor]
0000E8E9 7408                <1>     je     short m_pix_op_cpy_1_8_1 ; exclude
0000E8EB 8807                <1>     mov    [edi], al
0000E8ED FF05[6C8E0100]      <1>     inc    dword [u.r0] ; +1
                             <1> m_pix_op_cpy_1_8_1:
0000E8F3 47                  <1>     inc    edi ; +1
0000E8F4 4B                  <1>     dec    ebx
0000E8F5 75EB                <1>     jnz    short m_pix_op_cpy_1_8
                             <1> m_pix_op_cpy_2:
0000E8F7 89EE                <1>     mov    esi, ebp ; restore user's buffer addr
0000E8F9 09C9                <1>     or     ecx, ecx
                             <1>     ; 26/02/2021
0000E8FB 7407                <1>     jz     short m_pix_of_cpy_4
0000E8FD E979FFFFFF          <1>     jmp    m_pix_op_cpy_0
                             <1> m_pix_op_cpy_3:
0000E902 59                  <1>     pop    ecx ; *
0000E903 5F                  <1>     pop    edi ; **
                             <1> m_pix_of_cpy_4:
                             <1>     ;pop   ebx ; *** ; 26/02/2021
0000E904 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_cpy_1_16:
                             <1>     ; 16 bit masked copy
                             <1>     ;mov   edx, 2
0000E905 B202                <1>     mov    dl, 2 ; 26/02/2021
                             <1> m_pix_op_cpy_1_16_0:
0000E907 66AD                <1>     lodsw
0000E909 663B05[A29D0100]    <1>     cmp    ax, [maskcolor]
0000E910 7409                <1>     je     short m_pix_op_cpy_1_16_1 ; exclude
0000E912 668907              <1>     mov    [edi], ax
0000E915 0115[6C8E0100]      <1>     add    [u.r0], edx ; +2
                             <1> m_pix_op_cpy_1_16_1:
0000E91B 01D7                <1>     add    edi, edx ; +2
0000E91D 29D3                <1>     sub    ebx, edx ; sub ebx, 2
0000E91F 77E6                <1>     ja     short m_pix_op_cpy_1_16_0
0000E921 EBD4                <1>     jmp    short m_pix_op_cpy_2
                             <1>
                             <1> m_pix_op_cpy_1_32:
                             <1>     ; 32 bit masked copy
                             <1>     ;mov   edx, 4
0000E923 B204                <1>     mov    dl, 4 ; 26/02/2021
                             <1> m_pix_op_cpy_1_32_0:
```

```
0000E925 AD                  <1>     lodsd
0000E926 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000E92C 7408                <1>     je     short m_pix_op_cpy_1_32_1 ; exclude
0000E92E 8907                <1>     mov    [edi], eax
0000E930 0115[6C8E0100]      <1>     add    [u.r0], edx ; +4
                             <1> m_pix_op_cpy_1_32_1:
0000E936 01D7                <1>     add    edi, edx ; +4
0000E938 29D3                <1>     sub    ebx, edx ; sub ebx, 4
0000E93A 77E9                <1>     ja     short m_pix_op_cpy_1_32_0
0000E93C EBB9                <1>     jmp    short m_pix_op_cpy_2
                             <1>
                             <1> m_pix_op_cpy_w:
                             <1>     ; 26/02/2021
                             <1>     ; 06/02/2021
                             <1>     ; MASKED COPY PIXELS (window)
                             <1>     ;
                             <1>     ; jump from pix_op_cpy_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;   esi = user's buffer address
                             <1>     ;   [v_str] = window start address
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;   [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Window masked copy
                             <1>
                             <1> m_pix_op_cpy_w_0:
0000E93E 8B3D[9A9D0100]      <1>     mov    edi, [v_str]
                             <1> m_pix_op_cpy_w_1:
0000E944 57                  <1>     push   edi
0000E945 56                  <1>     push   esi
0000E946 53                  <1>     push   ebx
0000E947 52                  <1>     push   edx
0000E948 51                  <1>     push   ecx
0000E949 E82DFFFFFF          <1>     call   m_pix_op_cpy ; 26/02/2021
0000E94E 59                  <1>     pop    ecx
0000E94F 5A                  <1>     pop    edx
0000E950 5B                  <1>     pop    ebx
0000E951 5E                  <1>     pop    esi
0000E952 5F                  <1>     pop    edi
0000E953 7209                <1>     jc     short m_pix_op_cpy_w_2
0000E955 4B                  <1>     dec    ebx
0000E956 7406                <1>     jz     short m_pix_op_cpy_w_2 ; ok.
                             <1>     ; next row
0000E958 01CE                <1>     add    esi, ecx ; next row in user's buffer
0000E95A 01D7                <1>     add    edi, edx ; next row of window (system)
0000E95C EBE6                <1>     jmp    short m_pix_op_cpy_w_1
                             <1> m_pix_op_cpy_w_2:
0000E95E C3                  <1>     retn
                             <1>
                             <1> m_pix_op_new:
                             <1>     ; 06/02/2021
                             <1>     ; CHANGE COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_new
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   eax = color (AL, AX, EAX)
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Full screen
                             <1> m_pix_op_new_0:
0000E95F 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000E966 7717                <1>     ja     short m_pix_op_new_1
                             <1>     ; 256 colors (8bpp)
                             <1>     ;jmp   short m_pix_op_new_8
                             <1> m_pix_op_new_8:
                             <1>     ; 8 bit colors (256 colors)
0000E968 88C2                <1>     mov    dl, al ; new color
                             <1> m_pix_op_new_8_0:
0000E96A AC                  <1>     lodsb
0000E96B 3A05[A29D0100]      <1>     cmp    al, [maskcolor]
0000E971 7408                <1>     je     short m_pix_op_new_8_1 ; exclude
0000E973 8817                <1>     mov    [edi], dl
0000E975 FF05[6C8E0100]      <1>     inc    dword [u.r0]
                             <1> m_pix_op_new_8_1:
0000E97B 47                  <1>     inc    edi
0000E97C E2EC                <1>     loop   m_pix_op_new_8_0
0000E97E C3                  <1>     retn
                             <1> m_pix_op_new_1:
0000E97F 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000E986 774B                <1>     ja     short m_pix_op_new_3 ; 32bpp
0000E988 722C                <1>     jb     short m_pix_op_new_2 ; 16bpp
                             <1>     ; 24 bit true colors
                             <1>     ;jmp   short m_pix_op_new_24
                             <1> m_pix_op_new_24:
                             <1>     ; 24 bit true colors
0000E98A 89C2                <1>     mov    edx, eax ; new color
                             <1> m_pix_op_new_24_0:
0000E98C 66AD                <1>     lodsw
0000E98E C1E010              <1>     shl    eax, 16
0000E991 AC                  <1>     lodsb
0000E992 C1C010              <1>     rol    eax, 16
0000E995 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000E99B 7413                <1>     je     short m_pix_op_new_24_1 ; exclude
0000E99D 668917              <1>     mov    [edi], dx
0000E9A0 C1CA10              <1>     ror    edx, 16
0000E9A3 885702              <1>     mov    [edi+2], dl
0000E9A6 C1C210              <1>     rol    edx, 16
0000E9A9 8305[6C8E0100]03    <1>     add    dword [u.r0], 3
                             <1> m_pix_op_new_24_1:
0000E9B0 83C703              <1>     add    edi, 3
0000E9B3 E2D7                <1>     loop   m_pix_op_new_24_0
0000E9B5 C3                  <1>     retn
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_new_2:
                             <1>     ;jmp   short m_pix_op_new_16
                             <1> m_pix_op_new_16:
                             <1>     ; 16 bit colors (65536 colors)
0000E9B6 89C2                <1>     mov    edx, eax ; new color
                             <1> m_pix_op_new_16_0:
0000E9B8 66AD                <1>     lodsw
0000E9BA 663B05[A29D0100]    <1>     cmp    ax, [maskcolor]
0000E9C1 740A                <1>     je     short m_pix_op_new_16_1 ; exclude
```

```
0000E9C3 668917              <1>     mov     [edi], dx
0000E9C6 8305[6C8E0100]02    <1>     add     dword [u.r0], 2
                             <1> m_pix_op_new_16_1:
0000E9CD 83C702              <1>     add     edi, 2
0000E9D0 E2E6                <1>     loop    m_pix_op_new_16_0
0000E9D2 C3                  <1>     retn
                             <1> m_pix_op_new_3:
                             <1>     ; 32 bit true colors
                             <1>     ;jmp    short m_pix_op_new_32
                             <1> m_pix_op_new_32:
                             <1>     ; 32 bit true colors
0000E9D3 89C2                <1>     mov     edx, eax ; new color
                             <1> m_pix_op_new_32_0:
0000E9D5 AD                  <1>     lodsd
0000E9D6 3B05[A29D0100]      <1>     cmp     eax, [maskcolor]
0000E9DC 7409                <1>     je      short m_pix_op_new_32_1 ; exclude
0000E9DE 8917                <1>     mov     [edi], edx
0000E9E0 8305[6C8E0100]04    <1>     add     dword [u.r0], 4
                             <1> m_pix_op_new_32_1:
0000E9E7 83C704              <1>     add     edi, 4
0000E9EA E2E9                <1>     loop    m_pix_op_new_32_0
0000E9EC C3                  <1>     retn
                             <1>
                             <1> m_pix_op_new_w:
                             <1>     ; 06/02/2021
                             <1>     ; CHANGE COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_new_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;   eax = color
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Window
                             <1>     ;mov    edi, [v_str] ; LFB start address
                             <1>     ;mov    esi, edi
                             <1>
0000E9ED 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000E9F4 7707                <1>     ja      short m_pix_op_new_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000E9F6 BD[68E90000]        <1>     mov     ebp, m_pix_op_new_8
0000E9FB EB1E                <1>     jmp     short m_pix_op_new_w_x
                             <1>
                             <1> m_pix_op_new_w_1:
0000E9FD 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EA04 7710                <1>     ja      short m_pix_op_new_w_3 ; 32bpp
0000EA06 7207                <1>     jb      short m_pix_op_new_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000EA08 BD[8AE90000]        <1>     mov     ebp, m_pix_op_new_24
0000EA0D EB0C                <1>     jmp     short m_pix_op_new_w_x
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_new_w_2:
0000EA0F BD[B6E90000]        <1>     mov     ebp, m_pix_op_new_16
0000EA14 EB05                <1>     jmp     short m_pix_op_new_w_x
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_new_w_3:
0000EA16 BD[D3E90000]        <1>     mov     ebp, m_pix_op_new_32
                             <1>     ;jmp    short m_pix_op_new_w_x
                             <1>
                             <1> m_pix_op_new_w_x:
                             <1> m_pix_op_add_w_x:
                             <1> m_pix_op_sub_w_x:
                             <1> m_pix_op_mix_w_x:
                             <1> m_pix_op_and_w_x:
                             <1> m_pix_op_orc_w_x:
                             <1> m_pix_op_xor_w_x:
                             <1> m_pix_op_not_w_x:
                             <1> m_pix_op_neg_w_x:
                             <1> m_pix_op_inc_w_x:
                             <1> m_pix_op_dec_w_x:
                             <1>     ; 27/02/2021
                             <1>     ; 26/02/2021
                             <1>     ; 06/02/2021
                             <1>     ; ecx = bytes per row (to be applied)
                             <1>     ; edx = windows (screen) width in bytes
                             <1>     ; ebx = row count
                             <1>     ; eax = color
                             <1>     ; ebp = pixel operation subroutine address
                             <1>     ; edi = esi = window start address
                             <1>
0000EA1B 8B3D[9A9D0100]      <1>     mov     edi, [v_str] ; LFB start address
0000EA21 89FE                <1>     mov     esi, edi
                             <1> m_pix_op_w_x_next:
0000EA23 52                  <1>     push    edx
0000EA24 51                  <1>     push    ecx
0000EA25 56                  <1>     push    esi
0000EA26 57                  <1>     push    edi
0000EA27 50                  <1>     push    eax ; 26/02/2021
0000EA28 8B0D[A69D0100]      <1>     mov     ecx, [pixcount] ; 27/02/2021
0000EA2E FFD5                <1>     call    ebp ; call masked pixel-row operation
0000EA30 58                  <1>     pop     eax ; 26/02/2021
0000EA31 5F                  <1>     pop     edi
0000EA32 5E                  <1>     pop     esi
0000EA33 59                  <1>     pop     ecx
0000EA34 5A                  <1>     pop     edx
0000EA35 01D6                <1>     add     esi, edx ; next row
0000EA37 01D7                <1>     add     edi, edx ; next row
0000EA39 4B                  <1>     dec     ebx
0000EA3A 75E7                <1>     jnz     short m_pix_op_w_x_next
0000EA3C C3                  <1>     retn
                             <1>
                             <1> m_pix_op_add:
                             <1>     ; 06/02/2021
                             <1>     ; ADD COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_add
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   eax = color (AL, AX, EAX)
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
```

```
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_add_0:
0000EA3D 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EA44 771C                   <1>     ja      short m_pix_op_add_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_add_8
                                <1> m_pix_op_add_8:
                                <1>     ; 8 bit colors (256 colors)
0000EA46 88C2                   <1>     mov     dl, al ; new color
                                <1> m_pix_op_add_8_0:
0000EA48 AC                     <1>     lodsb
0000EA49 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000EA4F 740D                   <1>     je      short m_pix_op_add_8_1 ; exclude
0000EA51 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
0000EA57 0017                   <1>     add     [edi], dl
0000EA59 7303                   <1>     jnc     short m_pix_op_add_8_1
0000EA5B C607FF                 <1>     mov     byte [edi], 0FFh
                                <1> m_pix_op_add_8_1:
0000EA5E 47                     <1>     inc     edi
0000EA5F E2E7                   <1>     loop    m_pix_op_add_8_0
0000EA61 C3                     <1>     retn
                                <1> m_pix_op_add_1:
0000EA62 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EA69 775E                   <1>     ja      short m_pix_op_add_3 ; 32bpp
0000EA6B 7238                   <1>     jb      short m_pix_op_add_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_add_24
                                <1> m_pix_op_add_24:
                                <1>     ; 24 bit true colors
0000EA6D 89C2                   <1>     mov     edx, eax ; new color
0000EA6F 81CA000000FF           <1>     or      edx, 0FF000000h
                                <1> m_pix_op_add_24_0:
0000EA75 66AD                   <1>     lodsw
0000EA77 C1E010                 <1>     shl     eax, 16
0000EA7A AC                     <1>     lodsb
0000EA7B C1C010                 <1>     rol     eax, 16
0000EA7E 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EA84 7419                   <1>     je      short m_pix_op_add_24_2 ; exclude
0000EA86 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
0000EA8D 01D0                   <1>     add     eax, edx
0000EA8F 7305                   <1>     jnc     short m_pix_op_add_24_1
0000EA91 B8FFFFFF00             <1>     mov     eax, 0FFFFFFh
                                <1> m_pix_op_add_24_1:
0000EA96 668907                 <1>     mov     [edi], ax
0000EA99 C1E810                 <1>     shr     eax, 16
0000EA9C 884702                 <1>     mov     [edi+2], al
                                <1> m_pix_op_add_24_2:
0000EA9F 83C703                 <1>     add     edi, 3 ; +3
0000EAA2 E2D1                   <1>     loop    m_pix_op_add_24_0
0000EAA4 C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_add_2:
                                <1>     ;jmp    short m_pix_op_add_16
                                <1> m_pix_op_add_16:
                                <1>     ; 16 bit colors (65536 colors)
0000EAA5 89C2                   <1>     mov     edx, eax ; new color
                                <1> m_pix_op_add_16_0:
0000EAA7 66AD                   <1>     lodsw
0000EAA9 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000EAB0 7411                   <1>     je      short m_pix_op_add_16_1 ; exclude
0000EAB2 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
0000EAB9 660117                 <1>     add     [edi], dx
0000EABC 7305                   <1>     jnc     short m_pix_op_add_16_1
0000EABE 66C707FFFF             <1>     mov     word [edi], 0FFFFh
                                <1> m_pix_op_add_16_1:
0000EAC3 83C702                 <1>     add     edi, 2 ; +2
0000EAC6 E2DF                   <1>     loop    m_pix_op_add_16_0
0000EAC8 C3                     <1>     retn
                                <1> m_pix_op_add_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_add_32
                                <1> m_pix_op_add_32:
                                <1>     ; 32 bit true colors
0000EAC9 89C2                   <1>     mov     edx, eax ; new color
                                <1> m_pix_op_add_32_0:
0000EACB AD                     <1>     lodsd
0000EACC 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EAD2 7411                   <1>     je      short m_pix_op_add_32_1 ; exclude
0000EAD4 8305[6C8E0100]04       <1>     add     dword [u.r0], 4 ; +4
0000EADB 0117                   <1>     add     [edi], edx
0000EADD 7306                   <1>     jnc     short m_pix_op_add_32_1
0000EADF C707FFFFFFFF           <1>     mov     dword [edi], 0FFFFFFFFh
                                <1> m_pix_op_add_32_1:
0000EAE5 83C704                 <1>     add     edi, 4 ; +4
0000EAE8 E2E1                   <1>     loop    m_pix_op_add_32_0
0000EAEA C3                     <1>     retn
                                <1>
                                <1> m_pix_op_add_w:
                                <1>     ; 06/02/2021
                                <1>     ; ADD COLOR (MASKED, window)
                                <1>     ;
                                <1>     ; jump from pix_op_add_w
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = bytes per row (to be applied)
                                <1>     ;   edx = screen width in bytes
                                <1>     ;   ebx = row count
                                <1>     ;   eax = color
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; window
                                <1>     ;mov    edi, [v_str] ; LFB start address
                                <1>     ;mov    esi, edi
                                <1>
0000EAEB 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EAF2 7707                   <1>     ja      short m_pix_op_add_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000EAF4 BD[46EA0000]           <1>     mov     ebp, m_pix_op_add_8
0000EAF9 EB1E                   <1>     jmp     short m_pix_op_add_w_4
                                <1>
                                <1> m_pix_op_add_w_1:
0000EAFB 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EB02 7710                   <1>     ja      short m_pix_op_add_w_3 ; 32bpp
0000EB04 7207                   <1>     jb      short m_pix_op_add_w_2 ; 16bpp
                                <1>
```

```
                                <1>     ; 24 bit true colors
0000EB06 BD[6DEA0000]           <1>     mov     ebp, m_pix_op_add_24
0000EB0B EB0C                   <1>     jmp     short m_pix_op_add_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_add_w_2:
0000EB0D BD[A5EA0000]           <1>     mov     ebp, m_pix_op_add_16
0000EB12 EB05                   <1>     jmp     short m_pix_op_add_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> m_pix_op_add_w_3:
0000EB14 BD[C9EA0000]           <1>     mov     ebp, m_pix_op_add_32
                                <1> m_pix_op_add_w_4:
0000EB19 E9FDFEFFFF             <1>     jmp     m_pix_op_add_w_x
                                <1>
                                <1> m_pix_op_sub:
                                <1>     ; 02/03/2021
                                <1>     ; 06/02/2021
                                <1>     ; SUBTRACT COLOR (MASKED, full screen)
                                <1>     ;
                                <1>     ; jump from pix_op_sub
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   eax = color (AL, AX, EAX)
                                <1>     ;   ecx = [v_siz] ; display page pixel count
                                <1>     ;   esi = edi = [v_mem] ; LFB start address
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_sub_0:
0000EB1E 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EB25 771C                   <1>     ja      short m_pix_op_sub_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_sub_8
                                <1> m_pix_op_sub_8:
                                <1>     ; 8 bit colors (256 colors)
0000EB27 88C2                   <1>     mov     dl, al ; new color
                                <1> m_pix_op_sub_8_0:
0000EB29 AC                     <1>     lodsb
0000EB2A 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000EB30 740D                   <1>     je      short m_pix_op_sub_8_1 ; exclude
0000EB32 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
0000EB38 2817                   <1>     sub     [edi], dl
0000EB3A 7303                   <1>     jnb     short m_pix_op_sub_8_1
0000EB3C C60700                 <1>     mov     byte [edi], 0
                                <1> m_pix_op_sub_8_1:
0000EB3F 47                     <1>     inc     edi
0000EB40 E2E7                   <1>     loop    m_pix_op_sub_8_0
0000EB42 C3                     <1>     retn
                                <1> m_pix_op_sub_1:
0000EB43 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EB4A 7755                   <1>     ja      short m_pix_op_sub_3 ; 32bpp
0000EB4C 722F                   <1>     jb      short m_pix_op_sub_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_sub_24
                                <1> m_pix_op_sub_24:
                                <1>     ; 24 bit true colors
0000EB4E 89C2                   <1>     mov     edx, eax ; new color
                                <1>     ; 02/03/2021
                                <1> m_pix_op_sub_24_0:
0000EB50 66AD                   <1>     lodsw
0000EB52 C1E010                 <1>     shl     eax, 16
0000EB55 AC                     <1>     lodsb
0000EB56 C1C010                 <1>     rol     eax, 16
0000EB59 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EB5F 7416                   <1>     je      short m_pix_op_sub_24_2 ; exclude
0000EB61 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
0000EB68 29D0                   <1>     sub     eax, edx
0000EB6A 7302                   <1>     jnb     short m_pix_op_sub_24_1
0000EB6C 31C0                   <1>     xor     eax, eax ; 0
                                <1> m_pix_op_sub_24_1:
0000EB6E 668907                 <1>     mov     [edi], ax
0000EB71 C1E810                 <1>     shr     eax, 16
0000EB74 884702                 <1>     mov     [edi+2], al
                                <1> m_pix_op_sub_24_2:
0000EB77 83C703                 <1>     add     edi, 3 ; +3
0000EB7A E2D4                   <1>     loop    m_pix_op_sub_24_0
0000EB7C C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_sub_2:
                                <1>     ;jmp    short m_pix_op_sub_16
                                <1> m_pix_op_sub_16:
                                <1>     ; 16 bit colors (65536 colors)
0000EB7D 89C2                   <1>     mov     edx, eax ; new color
                                <1> m_pix_op_sub_16_0:
0000EB7F 66AD                   <1>     lodsw
0000EB81 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000EB88 7411                   <1>     je      short m_pix_op_sub_16_1 ; exclude
0000EB8A 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
0000EB91 662917                 <1>     sub     [edi], dx
0000EB94 7305                   <1>     jnb     short m_pix_op_sub_16_1
0000EB96 31C0                   <1>     xor     eax, eax
0000EB98 668907                 <1>     mov     [edi], ax ; 0
                                <1> m_pix_op_sub_16_1:
0000EB9B 83C702                 <1>     add     edi, 2 ; +2
0000EB9E E2DF                   <1>     loop    m_pix_op_sub_16_0
0000EBA0 C3                     <1>     retn
                                <1> m_pix_op_sub_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_sub_32
                                <1> m_pix_op_sub_32:
                                <1>     ; 32 bit true colors
0000EBA1 89C2                   <1>     mov     edx, eax ; new color
                                <1> m_pix_op_sub_32_0:
0000EBA3 AD                     <1>     lodsd
0000EBA4 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EBAA 740F                   <1>     je      short m_pix_op_sub_32_1 ; exclude
0000EBAC 8305[6C8E0100]04       <1>     add     dword [u.r0], 4 ; +4
0000EBB3 2917                   <1>     sub     [edi], edx
0000EBB5 7304                   <1>     jnb     short m_pix_op_sub_32_1
0000EBB7 31C0                   <1>     xor     eax, eax
0000EBB9 8907                   <1>     mov     [edi], eax ; 0
                                <1> m_pix_op_sub_32_1:
0000EBBB 83C704                 <1>     add     edi, 4 ; +4
0000EBBE E2E3                   <1>     loop    m_pix_op_sub_32_0
0000EBC0 C3                     <1>     retn
                                <1>
                                <1> m_pix_op_sub_w:
                                <1>     ; 06/02/2021
                                <1>     ; SUBTRACT COLOR (MASKED, window)
```

```
                                <1>     ;
                                <1>     ; jump from pix_op_sub_w
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = bytes per row (to be applied)
                                <1>     ;   edx = screen width in bytes
                                <1>     ;   ebx = row count
                                <1>     ;   eax = color
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; window
                                <1>     ;mov    edi, [v_str] ; LFB start address
                                <1>     ;mov    esi, edi
                                <1>
0000EBC1 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EBC8 7707                   <1>     ja      short m_pix_op_sub_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000EBCA BD[27EB0000]           <1>     mov     ebp, m_pix_op_sub_8
0000EBCF EB1E                   <1>     jmp     short m_pix_op_sub_w_4
                                <1>
                                <1> m_pix_op_sub_w_1:
0000EBD1 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EBD8 7710                   <1>     ja      short m_pix_op_sub_w_3 ; 32bpp
0000EBDA 7207                   <1>     jb      short m_pix_op_sub_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000EBDC BD[4EEB0000]           <1>     mov     ebp, m_pix_op_sub_24
0000EBE1 EB0C                   <1>     jmp     short m_pix_op_sub_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_sub_w_2:
0000EBE3 BD[7DEB0000]           <1>     mov     ebp, m_pix_op_sub_16
0000EBE8 EB05                   <1>     jmp     short m_pix_op_sub_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> m_pix_op_sub_w_3:
0000EBEA BD[A1EB0000]           <1>     mov     ebp, m_pix_op_sub_32
                                <1> m_pix_op_sub_w_4:
0000EBEF E927FEFFFF             <1>     jmp     m_pix_op_sub_w_x
                                <1>
                                <1> m_pix_op_mix:
                                <1>     ; 25/02/2021
                                <1>     ; 06/02/2021
                                <1>     ; MIX COLOR (MASKED, full screen)
                                <1>     ;
                                <1>     ; jump from pix_op_mix
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   eax = color (AL, AX, EAX)
                                <1>     ;   ecx = [v_siz] ; display page pixel count
                                <1>     ;   esi = edi = [v_mem] ; LFB start address
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_mix_0:
0000EBF4 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EBFB 771B                   <1>     ja      short m_pix_op_mix_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_mix_8
                                <1> m_pix_op_mix_8:
                                <1>     ; 8 bit colors (256 colors)
0000EBFD 88C2                   <1>     mov     dl, al ; new (mixing) color
                                <1> m_pix_op_mix_8_0:
0000EBFF AC                     <1>     lodsb
0000EC00 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000EC06 740C                   <1>     je      short m_pix_op_mix_8_1 ; exclude
0000EC08 00D0                   <1>     add     al, dl ; 25/02/2021
0000EC0A D0D8                   <1>     rcr     al, 1
0000EC0C 8807                   <1>     mov     [edi], al
0000EC0E FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> m_pix_op_mix_8_1:
0000EC14 47                     <1>     inc     edi
0000EC15 E2E8                   <1>     loop    m_pix_op_mix_8_0
0000EC17 C3                     <1>     retn
                                <1> m_pix_op_mix_1:
0000EC18 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EC1F 7752                   <1>     ja      short m_pix_op_mix_3 ; 32bpp
0000EC21 722D                   <1>     jb      short m_pix_op_mix_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_mix_24
                                <1> m_pix_op_mix_24:
                                <1>     ; 24 bit true colors
0000EC23 89C2                   <1>     mov     edx, eax ; new color
                                <1>     ;and    edx, 0FFFFFFh
                                <1> m_pix_op_mix_24_0:
0000EC25 66AD                   <1>     lodsw
0000EC27 C1E010                 <1>     shl     eax, 16
0000EC2A AC                     <1>     lodsb
0000EC2B C1C010                 <1>     rol     eax, 16
0000EC2E 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EC34 7414                   <1>     je      short m_pix_op_mix_24_1 ; exclude
0000EC36 01D0                   <1>     add     eax, edx
0000EC38 D1E8                   <1>     shr     eax, 1
0000EC3A 668907                 <1>     mov     [edi], ax
0000EC3D C1E810                 <1>     shr     eax, 16
0000EC40 884702                 <1>     mov     [edi+2], al
0000EC43 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
                                <1> m_pix_op_mix_24_1:
0000EC4A 83C703                 <1>     add     edi, 3 ; +3
0000EC4D E2D6                   <1>     loop    m_pix_op_mix_24_0
0000EC4F C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_mix_2:
                                <1>     ;jmp    short m_pix_op_mix_16
                                <1> m_pix_op_mix_16:
                                <1>     ; 16 bit colors (65536 colors)
0000EC50 89C2                   <1>     mov     edx, eax ; new color
                                <1>     ;and    edx, 0FFFFh
                                <1> m_pix_op_mix_16_0:
0000EC52 66AD                   <1>     lodsw
0000EC54 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000EC5B 7410                   <1>     je      short m_pix_op_mix_16_1 ; exclude
0000EC5D 6601D0                 <1>     add     ax, dx
0000EC60 66D1D8                 <1>     rcr     ax, 1
0000EC63 668907                 <1>     mov     [edi], ax
```

```
0000EC66 8305[6C8E0100]02    <1>     add    dword [u.r0], 2 ; +2
                             <1> m_pix_op_mix_16_1:
0000EC6D 83C702              <1>     add    edi, 2 ; +2
0000EC70 E2E0                <1>     loop   m_pix_op_mix_16_0
0000EC72 C3                  <1>     retn
                             <1> m_pix_op_mix_3:
                             <1>     ; 32 bit true colors
                             <1>     ;jmp   short m_pix_op_mix_32
                             <1> m_pix_op_mix_32:
                             <1>     ; 32 bit true colors
0000EC73 89C2                <1>     mov    edx, eax ; new color
                             <1> m_pix_op_mix_32_0:
0000EC75 AD                  <1>     lodsd
0000EC76 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000EC7C 740D                <1>     je     short m_pix_op_mix_32_1 ; exclude
0000EC7E 01D0                <1>     add    eax, edx
                             <1>     ; 02/03/2021
0000EC80 D1D8                <1>     rcr    eax, 1
0000EC82 8907                <1>     mov    [edi], eax
0000EC84 8305[6C8E0100]04    <1>     add    dword [u.r0], 4 ; +4
                             <1> m_pix_op_mix_32_1:
0000EC8B 83C704              <1>     add    edi, 4 ; +4
0000EC8E E2E5                <1>     loop   m_pix_op_mix_32_0
0000EC90 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_mix_w:
                             <1>     ; 06/02/2021
                             <1>     ; MIX COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_mix_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;   eax = color
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; window
                             <1>     ;mov   edi, [v_str] ; LFB start address
                             <1>     ;mov   esi, edi
                             <1>
0000EC91 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000EC98 7707                <1>     ja     short m_pix_op_mix_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000EC9A BD[FDEB0000]        <1>     mov    ebp, m_pix_op_mix_8
0000EC9F EB1E                <1>     jmp    short m_pix_op_mix_w_4
                             <1>
                             <1> m_pix_op_mix_w_1:
0000ECA1 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000ECA8 7710                <1>     ja     short m_pix_op_mix_w_3 ; 32bpp
0000ECAA 7207                <1>     jb     short m_pix_op_mix_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000ECAC BD[23EC0000]        <1>     mov    ebp, m_pix_op_mix_24
0000ECB1 EB0C                <1>     jmp    short m_pix_op_mix_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_mix_w_2:
0000ECB3 BD[50EC0000]        <1>     mov    ebp, m_pix_op_mix_16
0000ECB8 EB05                <1>     jmp    short m_pix_op_mix_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_mix_w_3:
0000ECBA BD[73EC0000]        <1>     mov    ebp, m_pix_op_mix_32
                             <1> m_pix_op_mix_w_4:
0000ECBF E957FDFFFF          <1>     jmp    m_pix_op_mix_w_x
                             <1>
                             <1> m_pix_op_and:
                             <1>     ; 06/02/2021
                             <1>     ; AND COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_and
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   eax = color (AL, AX, EAX)
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Full screen
                             <1> m_pix_op_and_0:
0000ECC4 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000ECCB 7717                <1>     ja     short m_pix_op_and_1
                             <1>     ; 256 colors (8bpp)
                             <1>     ;jmp   short m_pix_op_and_8
                             <1> m_pix_op_and_8:
                             <1>     ; 8 bit colors (256 colors)
0000ECCD 88C2                <1>     mov    dl, al ; new color
                             <1> m_pix_op_and_8_0:
0000ECCF AC                  <1>     lodsb
0000ECD0 3A05[A29D0100]      <1>     cmp    al, [maskcolor]
0000ECD6 7408                <1>     je     short m_pix_op_and_8_1 ; exclude
0000ECD8 2017                <1>     and    [edi], dl
0000ECDA FF05[6C8E0100]      <1>     inc    dword [u.r0] ; +1
                             <1> m_pix_op_and_8_1:
0000ECE0 47                  <1>     inc    edi
0000ECE1 E2EC                <1>     loop   m_pix_op_and_8_0
0000ECE3 C3                  <1>     retn
                             <1> m_pix_op_and_1:
0000ECE4 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000ECEB 774A                <1>     ja     short m_pix_op_and_3 ; 32bpp
0000ECED 722B                <1>     jb     short m_pix_op_and_2 ; 16bpp
                             <1>     ; 24 bit true colors
                             <1>     ;jmp   short m_pix_op_and_24
                             <1> m_pix_op_and_24:
                             <1>     ; 24 bit true colors
0000ECEF 89C2                <1>     mov    edx, eax ; new color
                             <1>     ;and   edx, 0FFFFFFh
                             <1> m_pix_op_and_24_0:
0000ECF1 66AD                <1>     lodsw
0000ECF3 C1E010              <1>     shl    eax, 16
0000ECF6 AC                  <1>     lodsb
0000ECF7 C1C010              <1>     rol    eax, 16
0000ECFA 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
```

```
0000ED00 7412                <1>     je      short m_pix_op_and_24_1 ; exclude
0000ED02 21D0                <1>     and     eax, edx
0000ED04 668907              <1>     mov     [edi], ax
0000ED07 C1E810              <1>     shr     eax, 16
0000ED0A 884702              <1>     mov     [edi+2], al
0000ED0D 8305[6C8E0100]03    <1>     add     dword [u.r0], 3 ; +3
                             <1> m_pix_op_and_24_1:
0000ED14 83C703              <1>     add     edi, 3 ; +3
0000ED17 E2D8                <1>     loop    m_pix_op_and_24_0
0000ED19 C3                  <1>     retn
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_and_2:
                             <1>     ;jmp    short m_pix_op_and_16
                             <1> m_pix_op_and_16:
                             <1>     ; 16 bit colors (65536 colors)
0000ED1A 89C2                <1>     mov     edx, eax ; new color
                             <1>     ;and    edx, 0FFFFh
                             <1> m_pix_op_and_16_0:
0000ED1C 66AD                <1>     lodsw
0000ED1E 663B05[A29D0100]    <1>     cmp     ax, [maskcolor]
0000ED25 740A                <1>     je      short m_pix_op_and_16_1 ; exclude
0000ED27 662117              <1>     and     [edi], dx
0000ED2A 8305[6C8E0100]02    <1>     add     dword [u.r0], 2 ; +2
                             <1> m_pix_op_and_16_1:
0000ED31 83C702              <1>     add     edi, 2 ; +2
0000ED34 E2E6                <1>     loop    m_pix_op_and_16_0
0000ED36 C3                  <1>     retn
                             <1> m_pix_op_and_3:
                             <1>     ; 32 bit true colors
                             <1>     ;jmp    short m_pix_op_and_32
                             <1> m_pix_op_and_32:
                             <1>     ; 32 bit true colors
0000ED37 89C2                <1>     mov     edx, eax ; new color
                             <1> m_pix_op_and_32_0:
0000ED39 AD                  <1>     lodsd
0000ED3A 3B05[A29D0100]      <1>     cmp     eax, [maskcolor]
0000ED40 7409                <1>     je      short m_pix_op_and_32_1 ; exclude
0000ED42 2117                <1>     and     [edi], edx ; 25/02/2021
0000ED44 8305[6C8E0100]04    <1>     add     dword [u.r0], 4 ; +4
                             <1> m_pix_op_and_32_1:
0000ED4B 83C704              <1>     add     edi, 4 ; +4
0000ED4E E2E9                <1>     loop    m_pix_op_and_32_0
0000ED50 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_and_w:
                             <1>     ; 06/02/2021
                             <1>     ; AND COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_and_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;   eax = color
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; window
                             <1>     ;mov    edi, [v_str] ; LFB start address
                             <1>     ;mov    esi, edi
                             <1>
0000ED51 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000ED58 7707                <1>     ja      short m_pix_op_and_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000ED5A BD[CDEC0000]        <1>     mov     ebp, m_pix_op_and_8
0000ED5F EB1E                <1>     jmp     short m_pix_op_and_w_4
                             <1>
                             <1> m_pix_op_and_w_1:
0000ED61 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000ED68 7710                <1>     ja      short m_pix_op_and_w_3 ; 32bpp
0000ED6A 7207                <1>     jb      short m_pix_op_and_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000ED6C BD[EFEC0000]        <1>     mov     ebp, m_pix_op_and_24
0000ED71 EB0C                <1>     jmp     short m_pix_op_and_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_and_w_2:
0000ED73 BD[1AED0000]        <1>     mov     ebp, m_pix_op_and_16
0000ED78 EB05                <1>     jmp     short m_pix_op_and_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_and_w_3:
0000ED7A BD[37ED0000]        <1>     mov     ebp, m_pix_op_and_32
                             <1> m_pix_op_and_w_4:
0000ED7F E997FCFFFF          <1>     jmp     m_pix_op_and_w_x
                             <1>
                             <1> m_pix_op_or:
                             <1>     ; 06/02/2021
                             <1>     ; OR COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_orc
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   eax = color (AL, AX, EAX)
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Full screen
                             <1> m_pix_op_or_0:
0000ED84 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000ED8B 7717                <1>     ja      short m_pix_op_or_1
                             <1>     ; 256 colors (8bpp)
                             <1>     ;jmp    short m_pix_op_or_8
                             <1> m_pix_op_or_8:
                             <1>     ; 8 bit colors (256 colors)
0000ED8D 88C2                <1>     mov     dl, al ; new color
                             <1> m_pix_op_or_8_0:
0000ED8F AC                  <1>     lodsb
0000ED90 3A05[A29D0100]      <1>     cmp     al, [maskcolor]
0000ED96 7408                <1>     je      short m_pix_op_or_8_1 ; exclude
0000ED98 0817                <1>     or      [edi], dl
0000ED9A FF05[6C8E0100]      <1>     inc     dword [u.r0] ; +1
                             <1> m_pix_op_or_8_1:
```

```
0000EDA0 47                  <1>     inc     edi
0000EDA1 E2EC                <1>     loop    m_pix_op_or_8_0
0000EDA3 C3                  <1>     retn
                             <1> m_pix_op_or_1:
0000EDA4 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EDAB 774A                <1>     ja      short m_pix_op_or_3 ; 32bpp
0000EDAD 722B                <1>     jb      short m_pix_op_or_2 ; 16bpp
                             <1>     ; 24 bit true colors
                             <1>     ;jmp    short m_pix_op_or_24
                             <1> m_pix_op_or_24:
                             <1>     ; 24 bit true colors
0000EDAF 89C2                <1>     mov     edx, eax ; new color
                             <1>     ;and    edx, 0FFFFFFh
                             <1> m_pix_op_or_24_0:
0000EDB1 66AD                <1>     lodsw
0000EDB3 C1E010              <1>     shl     eax, 16
0000EDB6 AC                  <1>     lodsb
0000EDB7 C1C010              <1>     rol     eax, 16
0000EDBA 3B05[A29D0100]      <1>     cmp     eax, [maskcolor]
0000EDC0 7412                <1>     je      short m_pix_op_or_24_1 ; exclude
0000EDC2 09D0                <1>     or      eax, edx
0000EDC4 668907              <1>     mov     [edi], ax
0000EDC7 C1E810              <1>     shr     eax, 16
0000EDCA 884702              <1>     mov     [edi+2], al
0000EDCD 8305[6C8E0100]03    <1>     add     dword [u.r0], 3 ; +3
                             <1> m_pix_op_or_24_1:
0000EDD4 83C703              <1>     add     edi, 3 ; +3
0000EDD7 E2D8                <1>     loop    m_pix_op_or_24_0
0000EDD9 C3                  <1>     retn
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_or_2:
                             <1>     ;jmp    short m_pix_op_or_16
                             <1> m_pix_op_or_16:
                             <1>     ; 16 bit colors (65536 colors)
0000EDDA 89C2                <1>     mov     edx, eax ; new color
                             <1>     ;and    edx, 0FFFFh
                             <1> m_pix_op_or_16_0:
0000EDDC 66AD                <1>     lodsw
0000EDDE 663B05[A29D0100]    <1>     cmp     ax, [maskcolor]
0000EDE5 740A                <1>     je      short m_pix_op_or_16_1 ; exclude
0000EDE7 660917              <1>     or      [edi], dx
0000EDEA 8305[6C8E0100]02    <1>     add     dword [u.r0], 2 ; +2
                             <1> m_pix_op_or_16_1:
0000EDF1 83C702              <1>     add     edi, 2 ; +2
0000EDF4 E2E6                <1>     loop    m_pix_op_or_16_0
0000EDF6 C3                  <1>     retn
                             <1> m_pix_op_or_3:
                             <1>     ; 32 bit true colors
                             <1>     ;jmp    short m_pix_op_or_32
                             <1> m_pix_op_or_32:
                             <1>     ; 32 bit true colors
0000EDF7 89C2                <1>     mov     edx, eax ; new color
                             <1> m_pix_op_or_32_0:
0000EDF9 AD                  <1>     lodsd
0000EDFA 3B05[A29D0100]      <1>     cmp     eax, [maskcolor]
0000EE00 7409                <1>     je      short m_pix_op_or_32_1 ; exclude
0000EE02 0917                <1>     or      [edi], edx ; 25/02/2021
0000EE04 8305[6C8E0100]04    <1>     add     dword [u.r0], 4 ; +4
                             <1> m_pix_op_or_32_1:
0000EE0B 83C704              <1>     add     edi, 4 ; +4
0000EE0E E2E9                <1>     loop    m_pix_op_or_32_0
0000EE10 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_or_w:
                             <1>     ; 06/02/2021
                             <1>     ; MIX COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_or_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;   eax = color
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; window
                             <1>     ;mov    edi, [v_str] ; LFB start address
                             <1>     ;mov    esi, edi
                             <1>
0000EE11 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EE18 7707                <1>     ja      short m_pix_op_or_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000EE1A BD[8DED0000]        <1>     mov     ebp, m_pix_op_or_8
0000EE1F EB1E                <1>     jmp     short m_pix_op_or_w_4
                             <1>
                             <1> m_pix_op_or_w_1:
0000EE21 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EE28 7710                <1>     ja      short m_pix_op_or_w_3 ; 32bpp
0000EE2A 7207                <1>     jb      short m_pix_op_or_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000EE2C BD[AFED0000]        <1>     mov     ebp, m_pix_op_or_24
0000EE31 EB0C                <1>     jmp     short m_pix_op_or_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_or_w_2:
0000EE33 BD[DAED0000]        <1>     mov     ebp, m_pix_op_or_16
0000EE38 EB05                <1>     jmp     short m_pix_op_or_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_or_w_3:
0000EE3A BD[F7ED0000]        <1>     mov     ebp, m_pix_op_or_32
                             <1> m_pix_op_or_w_4:
0000EE3F E9D7FBFFFF          <1>     jmp     m_pix_op_orc_w_x
                             <1>
                             <1> m_pix_op_xor:
                             <1>     ; 06/02/2021
                             <1>     ; XOR COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_xor
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   eax = color (AL, AX, EAX)
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
```

```
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_xor_0:
0000EE44 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EE4B 7717                   <1>     ja      short m_pix_op_xor_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_xor_8
                                <1> m_pix_op_xor_8:
                                <1>     ; 8 bit colors (256 colors)
0000EE4D 88C2                   <1>     mov     dl, al ; new color
                                <1> m_pix_op_xor_8_0:
0000EE4F AC                     <1>     lodsb
0000EE50 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000EE56 7408                   <1>     je      short m_pix_op_xor_8_1 ; exclude
0000EE58 3017                   <1>     xor     [edi], dl
0000EE5A FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> m_pix_op_xor_8_1:
0000EE60 47                     <1>     inc     edi
0000EE61 E2EC                   <1>     loop    m_pix_op_xor_8_0
0000EE63 C3                     <1>     retn
                                <1> m_pix_op_xor_1:
0000EE64 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EE6B 774A                   <1>     ja      short m_pix_op_xor_3 ; 32bpp
0000EE6D 722B                   <1>     jb      short m_pix_op_xor_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_xor_24
                                <1> m_pix_op_xor_24:
                                <1>     ; 24 bit true colors
0000EE6F 89C2                   <1>     mov     edx, eax ; new color
                                <1>     ;and    edx, 0FFFFFFh
                                <1> m_pix_op_xor_24_0:
0000EE71 66AD                   <1>     lodsw
0000EE73 C1E010                 <1>     shl     eax, 16
0000EE76 AC                     <1>     lodsb
0000EE77 C1C010                 <1>     rol     eax, 16
0000EE7A 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EE80 7412                   <1>     je      short m_pix_op_xor_24_1 ; exclude
0000EE82 31D0                   <1>     xor     eax, edx
0000EE84 668907                 <1>     mov     [edi], ax
0000EE87 C1E810                 <1>     shr     eax, 16
0000EE8A 884702                 <1>     mov     [edi+2], al
0000EE8D 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
                                <1> m_pix_op_xor_24_1:
0000EE94 83C703                 <1>     add     edi, 3 ; +3
0000EE97 E2D8                   <1>     loop    m_pix_op_xor_24_0
0000EE99 C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_xor_2:
                                <1>     ;jmp    short m_pix_op_xor_16
                                <1> m_pix_op_xor_16:
                                <1>     ; 16 bit colors (65536 colors)
0000EE9A 89C2                   <1>     mov     edx, eax ; new color
                                <1>     ;and    edx, 0FFFFh
                                <1> m_pix_op_xor_16_0:
0000EE9C 66AD                   <1>     lodsw
0000EE9E 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000EEA5 740A                   <1>     je      short m_pix_op_xor_16_1 ; exclude
0000EEA7 663117                 <1>     xor     [edi], dx
0000EEAA 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
                                <1> m_pix_op_xor_16_1:
0000EEB1 83C702                 <1>     add     edi, 2 ; +2
0000EEB4 E2E6                   <1>     loop    m_pix_op_xor_16_0
0000EEB6 C3                     <1>     retn
                                <1> m_pix_op_xor_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_xor_32
                                <1> m_pix_op_xor_32:
                                <1>     ; 32 bit true colors
0000EEB7 89C2                   <1>     mov     edx, eax ; new color
                                <1> m_pix_op_xor_32_0:
0000EEB9 AD                     <1>     lodsd
0000EEBA 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EEC0 7409                   <1>     je      short m_pix_op_xor_32_1 ; exclude
0000EEC2 3117                   <1>     xor     [edi], edx ; 25/02/2021
0000EEC4 8305[6C8E0100]04       <1>     add     dword [u.r0], 4 ; +4
                                <1> m_pix_op_xor_32_1:
0000EECB 83C704                 <1>     add     edi, 4 ; +4
0000EECE E2E9                   <1>     loop    m_pix_op_xor_32_0
0000EED0 C3                     <1>     retn
                                <1>
                                <1> m_pix_op_xor_w:
                                <1>     ; 06/02/2021
                                <1>     ; XOR COLOR (MASKED, window)
                                <1>     ;
                                <1>     ; jump from pix_op_xor_w
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = bytes per row (to be applied)
                                <1>     ;   edx = screen width in bytes
                                <1>     ;   ebx = row count
                                <1>     ;   eax = color
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; window
                                <1>     ;mov    edi, [v_str] ; LFB start address
                                <1>     ;mov    esi, edi
                                <1>
0000EED1 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EED8 7707                   <1>     ja      short m_pix_op_xor_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000EEDA BD[4DEE0000]           <1>     mov     ebp, m_pix_op_xor_8
0000EEDF EB1E                   <1>     jmp     short m_pix_op_xor_w_4
                                <1>
                                <1> m_pix_op_xor_w_1:
0000EEE1 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EEE8 7710                   <1>     ja      short m_pix_op_xor_w_3 ; 32bpp
0000EEEA 7207                   <1>     jb      short m_pix_op_xor_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000EEEC BD[6FEE0000]           <1>     mov     ebp, m_pix_op_xor_24
0000EEF1 EB0C                   <1>     jmp     short m_pix_op_xor_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_xor_w_2:
0000EEF3 BD[9AEE0000]           <1>     mov     ebp, m_pix_op_xor_16
0000EEF8 EB05                   <1>     jmp     short m_pix_op_xor_w_4
                                <1>
```

```
                                <1>     ; 32 bit true colors
                                <1> m_pix_op_xor_w_3:
0000EEFA BD[B7EE0000]           <1>     mov     ebp, m_pix_op_xor_32
                                <1> m_pix_op_xor_w_4:
0000EEFF E917FBFFFF             <1>     jmp     m_pix_op_xor_w_x
                                <1>
                                <1> m_pix_op_not:
                                <1>     ; 06/02/2021
                                <1>     ; NOT COLOR (MASKED, full screen)
                                <1>     ;
                                <1>     ; jump from pix_op_not
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = [v_siz] ; display page pixel count
                                <1>     ;   esi = edi = [v_mem] ; LFB start address
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_not_0:
0000EF04 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EF0B 7715                   <1>     ja      short m_pix_op_not_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_not_8
                                <1> m_pix_op_not_8:
                                <1>     ; 8 bit colors (256 colors)
0000EF0D AC                     <1>     lodsb
0000EF0E 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000EF14 7408                   <1>     je      short m_pix_op_not_8_1 ; exclude
0000EF16 F617                   <1>     not     byte [edi]
0000EF18 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> m_pix_op_not_8_1:
0000EF1E 47                     <1>     inc     edi
0000EF1F E2EC                   <1>     loop    m_pix_op_not_8
0000EF21 C3                     <1>     retn
                                <1> m_pix_op_not_1:
0000EF22 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EF29 7746                   <1>     ja      short m_pix_op_not_3 ; 32bpp
0000EF2B 7229                   <1>     jb      short m_pix_op_not_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_not_24
                                <1> m_pix_op_not_24:
                                <1>     ; 24 bit true colors
0000EF2D 66AD                   <1>     lodsw
0000EF2F C1E010                 <1>     shl     eax, 16
0000EF32 AC                     <1>     lodsb
0000EF33 C1C010                 <1>     rol     eax, 16
0000EF36 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EF3C 7412                   <1>     je      short m_pix_op_not_24_1 ; exclude
0000EF3E F7D0                   <1>     not     eax
0000EF40 668907                 <1>     mov     [edi], ax
0000EF43 C1E810                 <1>     shr     eax, 16
0000EF46 884702                 <1>     mov     [edi+2], al
0000EF49 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
                                <1> m_pix_op_not_24_1:
0000EF50 83C703                 <1>     add     edi, 3 ; +3
0000EF53 E2D8                   <1>     loop    m_pix_op_not_24
0000EF55 C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_not_2:
                                <1>     ;jmp    short m_pix_op_not_16
                                <1> m_pix_op_not_16:
                                <1>     ; 16 bit colors (65536 colors)
0000EF56 66AD                   <1>     lodsw
0000EF58 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000EF5F 740A                   <1>     je      short m_pix_op_not_16_1 ; exclude
0000EF61 66F717                 <1>     not     word [edi]
0000EF64 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
                                <1> m_pix_op_not_16_1:
0000EF6B 83C702                 <1>     add     edi, 2 ; +2
0000EF6E E2E6                   <1>     loop    m_pix_op_not_16
0000EF70 C3                     <1>     retn
                                <1> m_pix_op_not_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_not_32
                                <1> m_pix_op_not_32:
                                <1>     ; 32 bit true colors
0000EF71 AD                     <1>     lodsd
0000EF72 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000EF78 7409                   <1>     je      short m_pix_op_not_32_1 ; exclude
0000EF7A F717                   <1>     not     dword [edi]
0000EF7C 8305[6C8E0100]04       <1>     add     dword [u.r0], 4 ; +4
                                <1> m_pix_op_not_32_1:
0000EF83 83C704                 <1>     add     edi, 4 ; +4
0000EF86 E2E9                   <1>     loop    m_pix_op_not_32
0000EF88 C3                     <1>     retn
                                <1>
                                <1> m_pix_op_not_w:
                                <1>     ; 06/02/2021
                                <1>     ; NOT COLOR (MASKED, window)
                                <1>     ;
                                <1>     ; jump from pix_op_not_w
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = bytes per row (to be applied)
                                <1>     ;   edx = screen width in bytes
                                <1>     ;   ebx = row count
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; window
                                <1>     ;mov    edi, [v_str] ; LFB start address
                                <1>     ;mov    esi, edi
                                <1>
0000EF89 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000EF90 7707                   <1>     ja      short m_pix_op_not_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000EF92 BD[0DEF0000]           <1>     mov     ebp, m_pix_op_not_8
0000EF97 EB1E                   <1>     jmp     short m_pix_op_not_w_4
                                <1>
                                <1> m_pix_op_not_w_1:
0000EF99 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000EFA0 7710                   <1>     ja      short m_pix_op_not_w_3 ; 32bpp
0000EFA2 7207                   <1>     jb      short m_pix_op_not_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000EFA4 BD[2DEF0000]           <1>     mov     ebp, m_pix_op_not_24
```

```
0000EFA9 EB0C                <1>     jmp    short m_pix_op_not_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_not_w_2:
0000EFAB BD[56EF0000]        <1>     mov    ebp, m_pix_op_not_16
0000EFB0 EB05                <1>     jmp    short m_pix_op_not_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_not_w_3:
0000EFB2 BD[71EF0000]        <1>     mov    ebp, m_pix_op_not_32
                             <1> m_pix_op_not_w_4:
0000EFB7 E95FFAFFFF          <1>     jmp    m_pix_op_not_w_x
                             <1>
                             <1> m_pix_op_neg:
                             <1>     ; 06/02/2021
                             <1>     ; NEGATIVE COLOR (MASKED, full screen)
                             <1>     ;
                             <1>     ; jump from pix_op_neg
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = [v_siz] ; display page pixel count
                             <1>     ;   esi = edi = [v_mem] ; LFB start address
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; Full screen
                             <1> m_pix_op_neg_0:
0000EFBC 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000EFC3 7715                <1>     ja     short m_pix_op_neg_1
                             <1>     ; 256 colors (8bpp)
                             <1>     ;jmp   short m_pix_op_neg_8
                             <1> m_pix_op_neg_8:
                             <1>     ; 8 bit colors (256 colors)
0000EFC5 AC                  <1>     lodsb
0000EFC6 3A05[A29D0100]      <1>     cmp    al, [maskcolor]
0000EFCC 7408                <1>     je     short m_pix_op_neg_8_1 ; exclude
0000EFCE F61F                <1>     neg    byte [edi]
0000EFD0 FF05[6C8E0100]      <1>     inc    dword [u.r0] ; +1
                             <1> m_pix_op_neg_8_1:
0000EFD6 47                  <1>     inc    edi
0000EFD7 E2EC                <1>     loop   m_pix_op_neg_8
0000EFD9 C3                  <1>     retn
                             <1> m_pix_op_neg_1:
0000EFDA 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000EFE1 7746                <1>     ja     short m_pix_op_neg_3 ; 32bpp
0000EFE3 7229                <1>     jb     short m_pix_op_neg_2 ; 16bpp
                             <1>     ; 24 bit true colors
                             <1>     ;jmp   short m_pix_op_neg_24
                             <1> m_pix_op_neg_24:
                             <1>     ; 24 bit true colors
0000EFE5 66AD                <1>     lodsw
0000EFE7 C1E010              <1>     shl    eax, 16
0000EFEA AC                  <1>     lodsb
0000EFEB C1C010              <1>     rol    eax, 16
0000EFEE 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000EFF4 7412                <1>     je     short m_pix_op_neg_24_1 ; exclude
0000EFF6 F7D8                <1>     neg    eax
0000EFF8 668907              <1>     mov    [edi], ax
0000EFFB C1E810              <1>     shr    eax, 16
0000EFFE 884702              <1>     mov    [edi+2], al
0000F001 8305[6C8E0100]03    <1>     add    dword [u.r0], 3 ; +3
                             <1> m_pix_op_neg_24_1:
0000F008 83C703              <1>     add    edi, 3 ; +3
0000F00B E2D8                <1>     loop   m_pix_op_neg_24
0000F00D C3                  <1>     retn
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_neg_2:
                             <1>     ;jmp   short m_pix_op_neg_16
                             <1> m_pix_op_neg_16:
                             <1>     ; 16 bit colors (65536 colors)
0000F00E 66AD                <1>     lodsw
0000F010 663B05[A29D0100]    <1>     cmp    ax, [maskcolor]
0000F017 740A                <1>     je     short m_pix_op_neg_16_1 ; exclude
0000F019 66F71F              <1>     neg    word [edi]
0000F01C 8305[6C8E0100]02    <1>     add    dword [u.r0], 2 ; +2
                             <1> m_pix_op_neg_16_1:
0000F023 83C702              <1>     add    edi, 2 ; +2
0000F026 E2E6                <1>     loop   m_pix_op_neg_16
0000F028 C3                  <1>     retn
                             <1> m_pix_op_neg_3:
                             <1>     ; 32 bit true colors
                             <1>     ;jmp   short m_pix_op_neg_32
                             <1> m_pix_op_neg_32:
                             <1>     ; 32 bit true colors
0000F029 AD                  <1>     lodsd
0000F02A 3B05[A29D0100]      <1>     cmp    eax, [maskcolor]
0000F030 7409                <1>     je     short m_pix_op_neg_32_1 ; exclude
0000F032 F71F                <1>     neg    dword [edi]
0000F034 8305[6C8E0100]04    <1>     add    dword [u.r0], 4 ; +4
                             <1> m_pix_op_neg_32_1:
0000F03B 83C704              <1>     add    edi, 4 ; +4
0000F03E E2E9                <1>     loop   m_pix_op_neg_32
0000F040 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_neg_w:
                             <1>     ; 06/02/2021
                             <1>     ; NEGATIVE COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_neg_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;      [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; window
                             <1>     ;mov   edi, [v_str] ; LFB start address
                             <1>     ;mov   esi, edi
                             <1>
0000F041 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000F048 7707                <1>     ja     short m_pix_op_neg_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000F04A BD[C5EF0000]        <1>     mov    ebp, m_pix_op_neg_8
0000F04F EB1E                <1>     jmp    short m_pix_op_neg_w_4
                             <1>
```

```
                                <1> m_pix_op_neg_w_1:
0000F051 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F058 7710                   <1>     ja      short m_pix_op_neg_w_3 ; 32bpp
0000F05A 7207                   <1>     jb      short m_pix_op_neg_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000F05C BD[E5EF0000]           <1>     mov     ebp, m_pix_op_neg_24
0000F061 EB0C                   <1>     jmp     short m_pix_op_neg_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_neg_w_2:
0000F063 BD[0EF00000]           <1>     mov     ebp, m_pix_op_neg_16
0000F068 EB05                   <1>     jmp     short m_pix_op_neg_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> m_pix_op_neg_w_3:
0000F06A BD[29F00000]           <1>     mov     ebp, m_pix_op_neg_32
                                <1> m_pix_op_neg_w_4:
0000F06F E9A7F9FFFF             <1>     jmp     m_pix_op_neg_w_x
                                <1>
                                <1> m_pix_op_inc:
                                <1>     ; 06/02/2021
                                <1>     ; INCREASE COLOR (MASKED, full screen)
                                <1>     ;
                                <1>     ; jump from pix_op_inc
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = [v_siz] ; display page pixel count
                                <1>     ;   esi = edi = [v_mem] ; LFB start address
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_inc_0:
0000F074 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F07B 7719                   <1>     ja      short m_pix_op_inc_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_inc_8
                                <1> m_pix_op_inc_8:
                                <1>     ; 8 bit colors (256 colors)
0000F07D AC                     <1>     lodsb
0000F07E 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000F084 740C                   <1>     je      short m_pix_op_inc_8_1 ; exclude
0000F086 FE07                   <1>     inc     byte [edi]
0000F088 7502                   <1>     jnz     short m_pix_op_inc_8_0
0000F08A FE0F                   <1>     dec     byte [edi]
                                <1> m_pix_op_inc_8_0:
0000F08C FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> m_pix_op_inc_8_1:
0000F092 47                     <1>     inc     edi
0000F093 E2E8                   <1>     loop    m_pix_op_inc_8
0000F095 C3                     <1>     retn
                                <1> m_pix_op_inc_1:
0000F096 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F09D 7752                   <1>     ja      short m_pix_op_inc_3 ; 32bpp
0000F09F 7230                   <1>     jb      short m_pix_op_inc_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_inc_24
                                <1> m_pix_op_inc_24:
                                <1>     ; 24 bit true colors
0000F0A1 66AD                   <1>     lodsw
0000F0A3 C1E010                 <1>     shl     eax, 16
0000F0A6 AC                     <1>     lodsb
0000F0A7 C1C010                 <1>     rol     eax, 16
0000F0AA 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000F0B0 7419                   <1>     je      short m_pix_op_inc_24_1 ; exclude
0000F0B2 40                     <1>     inc     eax
0000F0B3 3DFFFFFF00             <1>     cmp     eax, 0FFFFFFh
0000F0B8 7601                   <1>     jna     short m_pix_op_inc_24_0
0000F0BA 48                     <1>     dec     eax
                                <1> m_pix_op_inc_24_0:
0000F0BB 668907                 <1>     mov     [edi], ax
0000F0BE C1E810                 <1>     shr     eax, 16
0000F0C1 884702                 <1>     mov     [edi+2], al
0000F0C4 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
                                <1> m_pix_op_inc_24_1:
0000F0CB 83C703                 <1>     add     edi, 3 ; +3
0000F0CE E2D1                   <1>     loop    m_pix_op_inc_24
0000F0D0 C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_inc_2:
                                <1>     ;jmp    short m_pix_op_inc_16
                                <1> m_pix_op_inc_16:
                                <1>     ; 16 bit colors (65536 colors)
0000F0D1 66AD                   <1>     lodsw
0000F0D3 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000F0DA 740F                   <1>     je      short m_pix_op_inc_16_1 ; exclude
0000F0DC 66FF07                 <1>     inc     word [edi]
0000F0DF 7503                   <1>     jnz     short m_pix_op_inc_16_0
0000F0E1 66FF0F                 <1>     dec     word [edi]
                                <1> m_pix_op_inc_16_0:
0000F0E4 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
                                <1> m_pix_op_inc_16_1:
0000F0EB 83C702                 <1>     add     edi, 2 ; +2
0000F0EE E2E1                   <1>     loop    m_pix_op_inc_16
0000F0F0 C3                     <1>     retn
                                <1> m_pix_op_inc_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_inc_32
                                <1> m_pix_op_inc_32:
                                <1>     ; 32 bit true colors
0000F0F1 AD                     <1>     lodsd
0000F0F2 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000F0F8 740D                   <1>     je      short m_pix_op_inc_32_1 ; exclude
0000F0FA FF07                   <1>     inc     dword [edi]
0000F0FC 7502                   <1>     jnz     short m_pix_op_inc_32_0
0000F0FE FF0F                   <1>     dec     dword [edi]
                                <1> m_pix_op_inc_32_0:
0000F100 8305[6C8E0100]04       <1>     add     dword [u.r0], 4 ; +4
                                <1> m_pix_op_inc_32_1:
0000F107 83C704                 <1>     add     edi, 4 ; +4
0000F10A E2E5                   <1>     loop    m_pix_op_inc_32
0000F10C C3                     <1>     retn
                                <1>
                                <1> m_pix_op_inc_w:
                                <1>     ; 06/02/2021
                                <1>     ; INCREASE COLOR (MASKED, window)
                                <1>     ;
                                <1>     ; jump from pix_op_inc_w
                                <1>     ;
                                <1>     ; INPUT:
```

```
                                <1>     ;   ecx = bytes per row (to be applied)
                                <1>     ;   edx = screen width in bytes
                                <1>     ;   ebx = row count
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; window
                                <1>     ;mov    edi, [v_str] ; LFB start address
                                <1>     ;mov    esi, edi
                                <1>
0000F10D 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F114 7707                   <1>     ja      short m_pix_op_inc_w_1
                                <1>
                                <1>     ; 256 colors (8bpp)
0000F116 BD[7DF00000]           <1>     mov     ebp, m_pix_op_inc_8
0000F11B EB1E                   <1>     jmp     short m_pix_op_inc_w_4
                                <1>
                                <1> m_pix_op_inc_w_1:
0000F11D 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F124 7710                   <1>     ja      short m_pix_op_inc_w_3 ; 32bpp
0000F126 7207                   <1>     jb      short m_pix_op_inc_w_2 ; 16bpp
                                <1>
                                <1>     ; 24 bit true colors
0000F128 BD[A1F00000]           <1>     mov     ebp, m_pix_op_inc_24
0000F12D EB0C                   <1>     jmp     short m_pix_op_inc_w_4
                                <1>
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_inc_w_2:
0000F12F BD[D1F00000]           <1>     mov     ebp, m_pix_op_inc_16
0000F134 EB05                   <1>     jmp     short m_pix_op_inc_w_4
                                <1>
                                <1>     ; 32 bit true colors
                                <1> m_pix_op_inc_w_3:
0000F136 BD[F1F00000]           <1>     mov     ebp, m_pix_op_inc_32
                                <1> m_pix_op_inc_w_4:
0000F13B E9DBF8FFFF             <1>     jmp     m_pix_op_inc_w_x
                                <1>
                                <1> m_pix_op_dec:
                                <1>     ; 06/02/2021
                                <1>     ; DECREASE COLOR (MASKED, full screen)
                                <1>     ;
                                <1>     ; jump from pix_op_dec
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;   ecx = [v_siz] ; display page pixel count
                                <1>     ;   esi = edi = [v_mem] ; LFB start address
                                <1>     ;
                                <1>     ;   [maskcolor] = mask color (to be excluded)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       [u.r0] will be > 0 if succesful
                                <1>
                                <1>     ; Full screen
                                <1> m_pix_op_dec_0:
0000F140 803D[919D0100]08       <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F147 7719                   <1>     ja      short m_pix_op_dec_1
                                <1>     ; 256 colors (8bpp)
                                <1>     ;jmp    short m_pix_op_dec_8
                                <1> m_pix_op_dec_8:
                                <1>     ; 8 bit colors (256 colors)
0000F149 AC                     <1>     lodsb
0000F14A 3A05[A29D0100]         <1>     cmp     al, [maskcolor]
0000F150 740C                   <1>     je      short m_pix_op_dec_8_1 ; exclude
0000F152 FE0F                   <1>     dec     byte [edi]
0000F154 7902                   <1>     jns     short m_pix_op_dec_8_0
0000F156 FE07                   <1>     inc     byte [edi]
                                <1> m_pix_op_dec_8_0:
0000F158 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; +1
                                <1> m_pix_op_dec_8_1:
0000F15E 47                     <1>     inc     edi
0000F15F E2E8                   <1>     loop    m_pix_op_dec_8
0000F161 C3                     <1>     retn
                                <1> m_pix_op_dec_1:
0000F162 803D[919D0100]18       <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F169 774D                   <1>     ja      short m_pix_op_dec_3 ; 32bpp
0000F16B 722B                   <1>     jb      short m_pix_op_dec_2 ; 16bpp
                                <1>     ; 24 bit true colors
                                <1>     ;jmp    short m_pix_op_dec_24
                                <1> m_pix_op_dec_24:
                                <1>     ; 24 bit true colors
0000F16D 66AD                   <1>     lodsw
0000F16F C1E010                 <1>     shl     eax, 16
0000F172 AC                     <1>     lodsb
0000F173 C1C010                 <1>     rol     eax, 16
0000F176 3B05[A29D0100]         <1>     cmp     eax, [maskcolor]
0000F17C 7414                   <1>     je      short m_pix_op_dec_24_1 ; exclude
0000F17E 48                     <1>     dec     eax
0000F17F 7901                   <1>     jns     short m_pix_op_dec_24_0
0000F181 40                     <1>     inc     eax
                                <1> m_pix_op_dec_24_0:
0000F182 668907                 <1>     mov     [edi], ax
0000F185 C1E810                 <1>     shr     eax, 16
0000F188 884702                 <1>     mov     [edi+2], al
0000F18B 8305[6C8E0100]03       <1>     add     dword [u.r0], 3 ; +3
                                <1> m_pix_op_dec_24_1:
0000F192 83C703                 <1>     add     edi, 3 ; +3
0000F195 E2D6                   <1>     loop    m_pix_op_dec_24
0000F197 C3                     <1>     retn
                                <1>     ; 65536 colors (16bpp)
                                <1> m_pix_op_dec_2:
                                <1>     ;jmp    short m_pix_op_dec_16
                                <1> m_pix_op_dec_16:
                                <1>     ; 16 bit colors (65536 colors)
0000F198 66AD                   <1>     lodsw
0000F19A 663B05[A29D0100]       <1>     cmp     ax, [maskcolor]
0000F1A1 740F                   <1>     je      short m_pix_op_dec_16_1 ; exclude
0000F1A3 66FF0F                 <1>     dec     word [edi]
0000F1A6 7903                   <1>     jns     short m_pix_op_dec_16_0
0000F1A8 66FF07                 <1>     inc     word [edi]
                                <1> m_pix_op_dec_16_0:
0000F1AB 8305[6C8E0100]02       <1>     add     dword [u.r0], 2 ; +2
                                <1> m_pix_op_dec_16_1:
0000F1B2 83C702                 <1>     add     edi, 2 ; +2
0000F1B5 E2E1                   <1>     loop    m_pix_op_dec_16
0000F1B7 C3                     <1>     retn
                                <1> m_pix_op_dec_3:
                                <1>     ; 32 bit true colors
                                <1>     ;jmp    short m_pix_op_dec_32
                                <1> m_pix_op_dec_32:
                                <1>     ; 32 bit true colors
0000F1B8 AD                     <1>     lodsd
```

```
0000F1B9 3B05[A29D0100]      <1>     cmp     eax, [maskcolor]
0000F1BF 740D                <1>     je      short m_pix_op_dec_32_1 ; exclude
0000F1C1 FF0F                <1>     dec     dword [edi]
0000F1C3 7902                <1>     jns     short m_pix_op_dec_32_0
0000F1C5 FF07                <1>     inc     dword [edi]
                             <1> m_pix_op_dec_32_0:
0000F1C7 8305[6C8E0100]04    <1>     add     dword [u.r0], 4 ; +4
                             <1> m_pix_op_dec_32_1:
0000F1CE 83C704              <1>     add     edi, 4 ; +4
0000F1D1 E2E5                <1>     loop    m_pix_op_dec_32
0000F1D3 C3                  <1>     retn
                             <1>
                             <1> m_pix_op_dec_w:
                             <1>     ; 06/02/2021
                             <1>     ; DECREASE COLOR (MASKED, window)
                             <1>     ;
                             <1>     ; jump from pix_op_dec_w
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;   ecx = bytes per row (to be applied)
                             <1>     ;   edx = screen width in bytes
                             <1>     ;   ebx = row count
                             <1>     ;
                             <1>     ;   [maskcolor] = mask color (to be excluded)
                             <1>     ;
                             <1>     ; OUTPUT:
                             <1>     ;       [u.r0] will be > 0 if succesful
                             <1>
                             <1>     ; window
                             <1>     ;mov    edi, [v_str] ; LFB start address
                             <1>     ;mov    esi, edi
                             <1>
0000F1D4 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F1DB 7707                <1>     ja      short m_pix_op_dec_w_1
                             <1>
                             <1>     ; 256 colors (8bpp)
0000F1DD BD[49F10000]        <1>     mov     ebp, m_pix_op_dec_8
0000F1E2 EB1E                <1>     jmp     short m_pix_op_dec_w_4
                             <1>
                             <1> m_pix_op_dec_w_1:
0000F1E4 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F1EB 7710                <1>     ja      short m_pix_op_dec_w_3 ; 32bpp
0000F1ED 7207                <1>     jb      short m_pix_op_dec_w_2 ; 16bpp
                             <1>
                             <1>     ; 24 bit true colors
0000F1EF BD[6DF10000]        <1>     mov     ebp, m_pix_op_dec_24
0000F1F4 EB0C                <1>     jmp     short m_pix_op_dec_w_4
                             <1>
                             <1>     ; 65536 colors (16bpp)
                             <1> m_pix_op_dec_w_2:
0000F1F6 BD[98F10000]        <1>     mov     ebp, m_pix_op_dec_16
0000F1FB EB05                <1>     jmp     short m_pix_op_dec_w_4
                             <1>
                             <1>     ; 32 bit true colors
                             <1> m_pix_op_dec_w_3:
0000F1FD BD[B8F10000]        <1>     mov     ebp, m_pix_op_dec_32
                             <1> m_pix_op_dec_w_4:
0000F202 E914F8FFFF          <1>     jmp     m_pix_op_dec_w_x
                             <1>
                             <1> sysvideo_39:
                             <1>     ; 15/02/2021
                             <1>     ; 07/02/2021, 08/02/2021
                             <1>     ; 03/01/2021, 04/01/2021
                             <1>     ; 23/11/2020
                             <1>     ; BH = 3
                             <1>     ; PIXEL READ/WRITE
                             <1>
                             <1>     ; 07/02/2021
                             <1>     ; 04/01/2021 (TRDOS 386 v2.0.3)
0000F207 80FB03              <1>     cmp     bl, 3
0000F20A 761A                <1>     jna     short sysvideo_39_1
                             <1>     ; 07/02/2021
0000F20C 80FB06              <1>     cmp     bl, 6
0000F20F 7705                <1>     ja      short sysvideo_39_0
0000F211 E91A010000          <1>     jmp     sysvideo_39_31
                             <1> sysvideo_39_0:
                             <1>     ; error
0000F216 B3FF                <1>     mov     bl, 0FFh
0000F218 8B2D[688E0100]      <1>     mov     ebp, [u.usp]  ; ebp points to user's registers
0000F21E 895D10              <1>     mov     [ebp+16], ebx ; EBX
0000F221 E968D8FFFF          <1>     jmp     sysret
                             <1> sysvideo_39_1:
0000F226 803D[2E680000]FF    <1>     cmp     byte [CRT_MODE], 0FFh
0000F22D 7312                <1>     jnb     short sysvideo_39_2 ; SVGA (VESA VBE) video mode
                             <1>
                             <1>     ; Std VGA or CGA mode
0000F22F 81C200000A00        <1>     add     edx, 0A0000h
0000F235 72DF                <1>     jc      short sysvideo_39_0
0000F237 81FAFFFF0A00        <1>     cmp     edx, 0AFFFFh
0000F23D 77D7                <1>     ja      short sysvideo_39_0
0000F23F EB1E                <1>     jmp     short sysvideo_39_3 ; 8bpp
                             <1>
                             <1> sysvideo_39_2:
                             <1>     ; use current vbe (svga) video mode
                             <1>
                             <1>     ; get LFB address
0000F241 A1[349D0100]        <1>     mov     eax, [LFB_ADDR] ; [LFB_Info+LFBINFO.LFB_addr]
0000F246 09C0                <1>     or      eax, eax
0000F248 74CC                <1>     jz      short sysvideo_39_0
0000F24A 3B15[389D0100]      <1>     cmp     edx, [LFB_SIZE]
0000F250 73C4                <1>     jnb     short sysvideo_39_0
                             <1>
0000F252 01C2                <1>     add     edx, eax
                             <1>     ;jc     short sysvideo_39_0
                             <1>
                             <1>     ; Pixel read/write in VESA VBE (2/3) video mode
                             <1>     ; Video memory at Linear Frame Buffer base address
                             <1>
0000F254 8A3D[409D0100]      <1>     mov     bh, [LFB_Info+LFBINFO.bpp]
                             <1>
0000F25A 80FF08              <1>     cmp     bh, 8 ; 8bpp
0000F25D 775D                <1>     ja      short sysvideo_39_17
                             <1>
                             <1>     ; 8 bits per pixel
                             <1> sysvideo_39_3:
0000F25F 80FB01              <1>     cmp     bl, 1 ; 1 = write pixel
0000F262 7406                <1>     je      short sysvideo_39_5
0000F264 7712                <1>     ja      short sysvideo_39_8
                             <1> sysvideo_39_4:
                             <1>     ; read pixel (8bpp)
0000F266 8A02                <1>     mov     al, [edx]
                             <1>     ;mov    [u.r0], al
                             <1>     ;jmp    sysret
0000F268 EB04                <1>     jmp     short sysvideo_39_7
```

```
                                <1> sysvideo_39_5:
                                <1>     ; write pixel (8bpp)
0000F26A 88C8                   <1>     mov     al, cl
                                <1> sysvideo_39_6:
0000F26C 8802                   <1>     mov     [edx], al
                                <1> sysvideo_39_7:
0000F26E A2[6C8E0100]           <1>     mov     [u.r0], al
0000F273 E916D8FFFF             <1>     jmp     sysret
                                <1> sysvideo_39_8:
0000F278 80FB03                 <1>     cmp     bl, 3 ; mix
0000F27B 7208                   <1>     jb      short sysvideo_39_9
                                <1>     ; mix  pixel colors (8bpp)
0000F27D 8A02                   <1>     mov     al, [edx]
0000F27F 00C8                   <1>     add     al, cl
0000F281 D0D8                   <1>     rcr     al, 1
0000F283 EBE7                   <1>     jmp     short sysvideo_39_6
                                <1> sysvideo_39_9:
                                <1>      ; swap pixel colors (8bpp)
0000F285 88C8                   <1>     mov     al, cl
0000F287 8602                   <1>     xchg    [edx], al
0000F289 EBE3                   <1>     jmp     short sysvideo_39_7
                                <1>
                                <1>     ; 16 bits per pixel
                                <1> sysvideo_39_10:
0000F28B 80FB01                 <1>     cmp     bl, 1 ; 1 = write pixel
0000F28E 7406                   <1>     je      short sysvideo_39_12
0000F290 7714                   <1>     ja      short sysvideo_39_15
                                <1> sysvideo_39_11:
                                <1>     ; read pixel (16bpp)
0000F292 8B02                   <1>     mov     eax, [edx]
                                <1>     ;mov    [u.r0], ax
                                <1>     ;jmp    sysret
0000F294 EB05                   <1>     jmp     short sysvideo_39_14
                                <1> sysvideo_39_12:
                                <1>     ; write pixel (16bpp)
0000F296 89C8                   <1>     mov     eax, ecx
                                <1> sysvideo_39_13:
0000F298 668902                 <1>     mov     [edx], ax
                                <1> sysvideo_39_14:
0000F29B 66A3[6C8E0100]         <1>     mov     [u.r0], ax
0000F2A1 E9E8D7FFFF             <1>     jmp     sysret
                                <1> sysvideo_39_15:
0000F2A6 80FB03                 <1>     cmp     bl, 3 ; mix
0000F2A9 720A                   <1>     jb      short sysvideo_39_16
                                <1>     ; mix  pixel colors (16bpp)
0000F2AB 8B02                   <1>     mov     eax, [edx]
0000F2AD 6601C8                 <1>     add     ax, cx
0000F2B0 66D1D8                 <1>     rcr     ax, 1
0000F2B3 EBE3                   <1>     jmp     short sysvideo_39_13
                                <1> sysvideo_39_16:
                                <1>      ; swap pixel colors (16bpp)
0000F2B5 89C8                   <1>     mov     eax, ecx
0000F2B7 668702                 <1>     xchg    [edx], ax
0000F2BA EBDF                   <1>     jmp     short sysvideo_39_14
                                <1> sysvideo_39_17:
0000F2BC 80FF18                 <1>     cmp     bh, 24
0000F2BF 7743                   <1>     ja      short sysvideo_39_24
0000F2C1 72C8                   <1>     jb      short sysvideo_39_10
                                <1>
                                <1>     ; 24 bits per pixel
0000F2C3 81E1FFFFFF00           <1>     and     ecx, 0FFFFFFh
0000F2C9 80FB01                 <1>     cmp     bl, 1 ; 1 = write pixel
0000F2CC 7406                   <1>     je      short sysvideo_39_19
0000F2CE 7712                   <1>     ja      short sysvideo_39_22
                                <1> sysvideo_39_18:
                                <1>     ; read pixel (24bpp)
0000F2D0 8B02                   <1>     mov     eax, [edx]
                                <1>     ;and    eax, 0FFFFFFh
                                <1>     ;mov    [u.r0], eax
                                <1>     ;jmp    sysret
0000F2D2 EB04                   <1>     jmp     short sysvideo_39_21
                                <1> sysvideo_39_19:
                                <1>     ; write pixel (24bpp)
0000F2D4 89C8                   <1>     mov     eax, ecx
                                <1> sysvideo_39_20:
                                <1>     ;and    eax, 0FFFFFFh
0000F2D6 8902                   <1>     mov     [edx], eax
                                <1> sysvideo_39_21:
0000F2D8 A3[6C8E0100]           <1>     mov     [u.r0], eax
0000F2DD E9ACD7FFFF             <1>     jmp     sysret
                                <1> sysvideo_39_22:
0000F2E2 80FB03                 <1>     cmp     bl, 3 ; mix
0000F2E5 720D                   <1>     jb      short sysvideo_39_23
                                <1>     ; mix  pixel colors (24bpp)
0000F2E7 8B02                   <1>     mov     eax, [edx]
0000F2E9 25FFFFFF00             <1>     and     eax, 0FFFFFFh
                                <1>     ;and    ecx, 0FFFFFFh
0000F2EE 01C8                   <1>     add     eax, ecx
0000F2F0 D1D8                   <1>     rcr     eax, 1
0000F2F2 EBE2                   <1>     jmp     short sysvideo_39_20
                                <1> sysvideo_39_23:
                                <1>      ; swap pixel colors (24bpp)
0000F2F4 89C8                   <1>     mov     eax, ecx
                                <1>     ;and    eax, 0FFFFFFh
0000F2F6 668702                 <1>     xchg    [edx], ax
0000F2F9 C1C810                 <1>     ror     eax, 16
0000F2FC 884202                 <1>     mov     [edx+2], al
0000F2FF C1C010                 <1>     rol     eax, 16
0000F302 EBD4                   <1>     jmp     short sysvideo_39_21
                                <1>
                                <1>     ; 32 bits per pixel
                                <1> sysvideo_39_24:
0000F304 80FB01                 <1>     cmp     bl, 1 ; 1 = write pixel
0000F307 7406                   <1>     je      short sysvideo_39_26
0000F309 7712                   <1>     ja      short sysvideo_39_29
                                <1> sysvideo_39_25:
                                <1>     ; read pixel (32bpp)
0000F30B 8B02                   <1>     mov     eax, [edx]
                                <1>     ;mov    [u.r0], eax
                                <1>     ;jmp    sysret
0000F30D EB04                   <1>     jmp     short sysvideo_39_28
                                <1> sysvideo_39_26:
                                <1>     ; write pixel (32bpp)
0000F30F 89C8                   <1>     mov     eax, ecx
                                <1> sysvideo_39_27:
0000F311 8902                   <1>     mov     [edx], eax
                                <1> sysvideo_39_28:
0000F313 A3[6C8E0100]           <1>     mov     [u.r0], eax
0000F318 E971D7FFFF             <1>     jmp     sysret
                                <1> sysvideo_39_29:
0000F31D 80FB03                 <1>     cmp     bl, 3 ; mix
0000F320 7208                   <1>     jb      short sysvideo_39_30
                                <1>     ; mix  pixel colors (32bpp)
0000F322 8B02                   <1>     mov     eax, [edx]
```

```
0000F324 01C8                <1>     add     eax, ecx
0000F326 D1D8                <1>     rcr     eax, 1
0000F328 EBE7                <1>     jmp     short sysvideo_39_27
                             <1> sysvideo_39_30:
                             <1>      ; swap pixel colors (32bpp)
0000F32A 89C8                <1>     mov     eax, ecx
0000F32C 8702                <1>     xchg    [edx], eax
0000F32E EBE3                <1>     jmp     short sysvideo_39_28
                             <1>
                             <1> sysvideo_39_31:
                             <1>     ; 06/03/2021
                             <1>     ; 08/02/2021
                             <1>     ; 07/02/2021
                             <1>     ; BL = 4 -> read pixels from user defined positions
                             <1>     ; BL = 5 -> write single color pixels to user defined pos.
                             <1>     ; BL = 6 -> write multi color pixels to user defined pos.
                             <1>     ; ECX = color (CL, CX, ECX)
                             <1>     ; EDX = number of pixels
                             <1>     ; ESI = user buffer contains dword pixel positions
                             <1>     ;       (and dword colors for BL input = 6)
                             <1>     ; EDI = user's pixel color buff (destination) for BL = 4
                             <1>
0000F330 890D[A29D0100]      <1>     mov     [maskcolor], ecx
0000F336 89D5                <1>     mov     ebp, edx ; number of pixels
0000F338 803D[2E680000]FF    <1>     cmp     byte [CRT_MODE], 0FFh ; SVGA flag
0000F33F 7317                <1>     jnb     short sysvideo_39_33 ; SVGA (VESA VBE mode)
                             <1>     ; Standard VGA mode
0000F341 B900000100          <1>     mov     ecx, 65536 ; Video page size (maximum)
0000F346 39CA                <1>     cmp     edx, ecx
0000F348 7709                <1>     ja      short sysvideo_39_32 ; abnormal value !
0000F34A B800000A00          <1>     mov     eax, 0A0000h ; Video page start address
0000F34F B708                <1>     mov     bh, 8  ; 8 bits per pixel (256 colors)
0000F351 EB35                <1>     jmp     short sysvideo_39_34
                             <1> sysvideo_39_32:
                             <1>     ; nonsense! (edx has abnormal value)
0000F353 E936D7FFFF          <1>     jmp     sysret
                             <1> sysvideo_39_33:
                             <1>     ; 06/03/2021
0000F358 8A3D[409D0100]      <1>     mov     bh, [LFB_Info+LFBINFO.bpp]
0000F35E 80FF08              <1>     cmp     bh, 8
0000F361 7412                <1>     je      short sysvideo_39_81 ; 8bpp
0000F363 89D0                <1>     mov     eax, edx
0000F365 80FF10              <1>     cmp     bh, 16
0000F368 7409                <1>     je      short sysvideo_39_80 ; 16bpp
0000F36A D1E2                <1>     shl     edx, 1
0000F36C 80FF20              <1>     cmp     bh, 32
0000F36F 7502                <1>     jne     short sysvideo_39_80 ; 24bpp
0000F371 D1E0                <1>     shl     eax, 1
                             <1> sysvideo_39_80:
0000F373 01C2                <1>     add     edx, eax
                             <1>     ; edx = number of bytes
                             <1> sysvideo_39_81:
                             <1>     ; get LFB address
0000F375 A1[349D0100]        <1>     mov     eax, [LFB_ADDR] ; [LFB_Info+LFBINFO.LFB_addr]
0000F37A 09C0                <1>     or      eax, eax
0000F37C 74D5                <1>     jz      short sysvideo_39_32 ; LFB is not ready !
0000F37E 8B0D[389D0100]      <1>     mov     ecx, [LFB_SIZE]
0000F384 39CA                <1>     cmp     edx, ecx
0000F386 77CB                <1>     ja      short sysvideo_39_32 ; abnormal value !
                             <1>
                             <1>     ; 02/03/2021
                             <1>     ; 08/02/2021
                             <1>     ;mov    ebp, edx ; pixel count
                             <1>     ;shl    ebp, 2 ; byte count (pixel pos: 4 bytes)
                             <1>
                             <1>     ; 06/03/2021
                             <1>     ; bits per pixel (pixel color size)
                             <1>     ;mov    bh, [LFB_Info+LFBINFO.bpp]
                             <1> sysvideo_39_34:
0000F388 C1E502              <1>     shl     ebp, 2 ; 15/02/2021 (byte count)
0000F38B A3[929D0100]        <1>     mov     [v_mem], eax ; Save video page start address
0000F390 88DE                <1>     mov     dh, bl ; sub function
                             <1>     ; 06/03/2021
0000F392 883D[919D0100]      <1>     mov     [v_bpp], bh ; bits per pixel (color size)
                             <1>     ;mov    ebx, [LFB_SIZE]
0000F398 89CB                <1>     mov     ebx, ecx ; [LFB_SIZE]
                             <1>
0000F39A B900080000          <1>     mov     ecx, 2048
0000F39F 39CD                <1>     cmp     ebp, ecx
0000F3A1 7302                <1>     jnb     short sysvideo_39_35
0000F3A3 89E9                <1>     mov     ecx, ebp ; fix to requested byte count
                             <1> sysvideo_39_35:
0000F3A5 80FE04              <1>     cmp     dh, 4 ; 08/02/2021
                             <1>     ;cmp    bl, 4 ; read pixels from user defined positions
0000F3A8 7605                <1>     jna     short sysvideo_39_36
0000F3AA E9B2000000          <1>     jmp     sysvideo_39_52
                             <1>     ; 08/02/2021
                             <1>     ;mov    [buffer8], edi  ; user's destination buff addr
                             <1> sysvideo_39_36:
                             <1>     ; 08/02/2021
                             <1>     ; read pixel positions
                             <1>     ; as defined in user's source buffer
0000F3AF 893D[AA9D0100]      <1>     mov     [buffer8], edi  ; user's destination buff addr
0000F3B5 BF00760900          <1>     mov     edi, VBE3SAVERESTOREBLOCK ; kernel buffer for
                             <1>                                       ; 2028 byte data
                             <1>     ; esi = user's source buffer for pixel positions
                             <1>     ; ecx = byte count
0000F3BA E8A6170000          <1>     call    transfer_from_user_buffer
0000F3BF 7292                <1>     jc      short sysvideo_39_32 ; error
                             <1>     ; ecx  = transfer count (bytes)
                             <1>
0000F3C1 57                  <1>     push    edi ; *
0000F3C2 56                  <1>     push    esi ; **
0000F3C3 51                  <1>     push    ecx ; ***
                             <1>
0000F3C4 89FE                <1>     mov     esi, edi ; kernel buffer
0000F3C6 8B15[929D0100]      <1>     mov     edx, [v_mem] ; video memory
0000F3CC C1E902              <1>     shr     ecx, 2 ; pixel count (within buffer capacity)
                             <1>
0000F3CF 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F3D6 7753                <1>     ja      short sysvideo_39_49
                             <1> sysvideo_39_37:
                             <1>     ; 8bpp
0000F3D8 AD                  <1>     lodsd
0000F3D9 39D8                <1>     cmp     eax, ebx ; < [LFB_SIZE]
0000F3DB 7309                <1>     jnb     short sysvideo_39_39
0000F3DD 0FB60402            <1>     movzx   eax, byte [edx+eax]
                             <1> sysvideo_39_38:
0000F3E1 AB                  <1>     stosd
0000F3E2 E2F4                <1>     loop    sysvideo_39_37
0000F3E4 EB49                <1>     jmp     short sysvideo_39_50
                             <1> sysvideo_39_39:
                             <1>     ; write black color for improper positions
0000F3E6 31C0                <1>     xor     eax, eax
```

```
0000F3E8 EBF7                <1>     jmp    short sysvideo_39_38
                             <1> sysvideo_39_40:
0000F3EA 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000F3F1 772A                <1>     ja     short sysvideo_39_47 ; 32bpp
0000F3F3 7216                <1>     jb     short sysvideo_39_44 ; 16bpp
                             <1> sysvideo_39_41:
                             <1>     ; 24bpp
0000F3F5 AD                  <1>     lodsd
0000F3F6 39D8                <1>     cmp    eax, ebx ; < [LFB_SIZE]
0000F3F8 730D                <1>     jnb    short sysvideo_39_43
0000F3FA 8B0402              <1>     mov    eax, [edx+eax]
0000F3FD 25FFFFFF00          <1>     and    eax, 0FFFFFFh
                             <1> sysvideo_39_42:
0000F402 AB                  <1>     stosd
0000F403 E2F0                <1>     loop   sysvideo_39_41
0000F405 EB28                <1>     jmp    short sysvideo_39_50
                             <1> sysvideo_39_43:
                             <1>     ; write black color for improper positions
0000F407 31C0                <1>     xor    eax, eax
0000F409 EBF7                <1>     jmp    short sysvideo_39_42
                             <1> sysvideo_39_44:
                             <1>     ; 16bpp
0000F40B AD                  <1>     lodsd
0000F40C 39D8                <1>     cmp    eax, ebx ; < [LFB_SIZE]
0000F40E 7309                <1>     jnb    short sysvideo_39_46
0000F410 0FB70402            <1>     movzx  eax, word [edx+eax]
                             <1> sysvideo_39_45:
0000F414 AB                  <1>     stosd
0000F415 E2F4                <1>     loop   sysvideo_39_44
0000F417 EB16                <1>     jmp    short sysvideo_39_50
                             <1> sysvideo_39_46:
                             <1>     ; write black color for improper positions
0000F419 31C0                <1>     xor    eax, eax
0000F41B EBF7                <1>     jmp    short sysvideo_39_45
                             <1> sysvideo_39_47:
                             <1>     ; 32bpp
0000F41D AD                  <1>     lodsd
0000F41E 39D8                <1>     cmp    eax, ebx ; < [LFB_SIZE]
0000F420 7309                <1>     jnb    short sysvideo_39_49
0000F422 0FB70402            <1>     movzx  eax, word [edx+eax]
                             <1> sysvideo_39_48:
0000F426 AB                  <1>     stosd
0000F427 E2F4                <1>     loop   sysvideo_39_47
0000F429 EB04                <1>     jmp    short sysvideo_39_50
                             <1> sysvideo_39_49:
                             <1>     ; write black color for improper positions
0000F42B 31C0                <1>     xor    eax, eax
0000F42D EBF7                <1>     jmp    short sysvideo_39_48
                             <1> sysvideo_39_50:
0000F42F 59                  <1>     pop    ecx ; transfer count in bytes
0000F430 5E                  <1>     pop    esi ; ** ; kernel buffer
                             <1>     ;mov   esi, VBE3SAVERESTOREBLOCK ; kernel buffer for
                             <1>                                     ; 2048 byte data
0000F431 8B3D[AA9D0100]      <1>     mov    edi, [buffer8]
                             <1>     ; edi = user's destination buffer for pixel colors
                             <1>     ; ecx = byte count
0000F437 E8DF160000          <1>     call   transfer_to_user_buffer
0000F43C 5E                  <1>     pop    esi ; *
0000F43D 7266                <1>     jc     short sysvideo_39_56 ; error
                             <1>     ; ecx  = transfer count (bytes)
0000F43F 89C8                <1>     mov    eax, ecx
0000F441 C1E802              <1>     shr    eax, 2
0000F444 0105[6C8E0100]      <1>     add    [u.r0], eax ; transfer count (in pixels)
                             <1>
0000F44A 29CD                <1>     sub    ebp, ecx
0000F44C 7657                <1>     jna    short sysvideo_39_56 ; completed/finished
0000F44E 01CE                <1>     add    esi, ecx ; next position in source buffer
                             <1>     ;add   [buffer8], ecx ; next pos in destination buff
0000F450 01CF                <1>     add    edi, ecx
0000F452 66B90008            <1>     mov    cx, 2048 ; new count, limit: kernel buff size
0000F456 39CD                <1>     cmp    ebp, ecx ; remain >= limit ?
0000F458 7302                <1>     jnb    short sysvideo_39_51 ; yes
0000F45A 89E9                <1>     mov    ecx, ebp ; fix byte count to remain bytes
                             <1> sysvideo_39_51:
0000F45C E94EFFFFFF          <1>     jmp    sysvideo_39_36
                             <1>
                             <1> sysvideo_39_52:
0000F461 80FE05              <1>     cmp    dh, 5 ; 08/02/2021
                             <1>     ;cmp   bl, 5 ; write pixels to user defined positions
0000F464 7605                <1>     jna    short sysvideo_39_53
0000F466 E9A1000000          <1>     jmp    sysvideo_39_66
                             <1> sysvideo_39_53:
                             <1>     ; single color pixel writing
0000F46B BF00760900          <1>     mov    edi, VBE3SAVERESTOREBLOCK ; kernel buffer for
                             <1>                                     ; 2028 byte data
                             <1>     ; esi = user's source buffer for pixel positions
                             <1>     ; ecx = byte count
0000F470 E8F0160000          <1>     call   transfer_from_user_buffer
0000F475 722E                <1>     jc     short sysvideo_39_56 ; error
                             <1>     ; ecx = transfer count (bytes)
                             <1>
                             <1>     ; write pixels by using (user) defined positions
                             <1>     ; ecx = byte count (1,2,3,4 times pixel count)
                             <1>     ; edi = system buffer address
                             <1>
0000F477 56                  <1>     push   esi ; *
0000F478 51                  <1>     push   ecx ; **
                             <1>
0000F479 89FE                <1>     mov    esi, edi
0000F47B 8B3D[929D0100]      <1>     mov    edi, [v_mem]
                             <1>
                             <1>     ; 08/02/2021
0000F481 C1E902              <1>     shr    ecx, 2 ; pixel count
0000F484 8B15[A29D0100]      <1>     mov    edx, [maskcolor]
                             <1>     ;mov   ebx, [v_siz]
                             <1>
0000F48A 803D[919D0100]08    <1>     cmp    byte [v_bpp], 8 ; 8bpp
0000F491 7717                <1>     ja     short sysvideo_39_57
                             <1> sysvideo_39_54:
                             <1>     ; 8bpp
0000F493 AD                  <1>     lodsd
0000F494 39D8                <1>     cmp    eax, ebx ; < [v_siz]
0000F496 7309                <1>     jnb    short sysvideo_39_55
0000F498 881407              <1>     mov    [edi+eax], dl
                             <1>     ; 06/03/2021
0000F49B FF05[6C8E0100]      <1>     inc    dword [u.r0]
                             <1> sysvideo_39_55:
0000F4A1 E2F0                <1>     loop   sysvideo_39_54
0000F4A3 EB50                <1>     jmp    short sysvideo_39_64
                             <1> sysvideo_39_56:
0000F4A5 E9E4D5FFFF          <1>     jmp    sysret
                             <1> sysvideo_39_57:
0000F4AA 803D[919D0100]18    <1>     cmp    byte [v_bpp], 24 ; 24bpp
0000F4B1 7732                <1>     ja     short sysvideo_39_62 ; 32bpp
```

```
0000F4B3 721D                <1>     jb      short sysvideo_39_60 ; 16bpp
                             <1> sysvideo_39_58:
                             <1>     ; 24bpp
0000F4B5 AD                  <1>     lodsd
0000F4B6 39D8                <1>     cmp     eax, ebx ; < [v_siz]
0000F4B8 7314                <1>     jnb     short sysvideo_39_59
0000F4BA 881407              <1>     mov     [edi+eax], dl
0000F4BD 40                  <1>     inc     eax
0000F4BE C1CA08              <1>     ror     edx, 8
0000F4C1 66891407            <1>     mov     [edi+eax], dx
0000F4C5 C1C208              <1>     rol     edx, 8
0000F4C8 FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_59:
0000F4CE E2E5                <1>     loop    sysvideo_39_58
0000F4D0 EB23                <1>     jmp     short sysvideo_39_64
                             <1> sysvideo_39_60:
                             <1>     ; 16bpp
0000F4D2 AD                  <1>     lodsd
0000F4D3 39D8                <1>     cmp     eax, ebx ; < [v_siz]
0000F4D5 730A                <1>     jnb     short sysvideo_39_61
0000F4D7 66891407            <1>     mov     [edi+eax], dx
0000F4DB FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_61:
0000F4E1 E2EF                <1>     loop    sysvideo_39_60
0000F4E3 EB10                <1>     jmp     short sysvideo_39_64
                             <1> sysvideo_39_62:
                             <1>     ; 32bpp
0000F4E5 AD                  <1>     lodsd
0000F4E6 39D8                <1>     cmp     eax, ebx ; < [v_siz]
0000F4E8 7309                <1>     jnb     short sysvideo_39_63
0000F4EA 891407              <1>     mov     [edi+eax], edx
0000F4ED FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_63:
0000F4F3 E2F0                <1>     loop    sysvideo_39_62
                             <1> sysvideo_39_64:
0000F4F5 59                  <1>     pop     ecx ; **
0000F4F6 5E                  <1>     pop     esi ; *
0000F4F7 29CD                <1>     sub     ebp, ecx
0000F4F9 76AA                <1>     jna     short sysvideo_39_56
0000F4FB 01CE                <1>     add     esi, ecx
0000F4FD 66B90008            <1>     mov     cx, 2048
0000F501 39CD                <1>     cmp     ebp, ecx
0000F503 7302                <1>     jnb     short sysvideo_39_65
0000F505 89E9                <1>     mov     ecx, ebp
                             <1> sysvideo_39_65:
0000F507 E95FFFFFFF          <1>     jmp     sysvideo_39_53
                             <1>
                             <1> sysvideo_39_66:
                             <1>     ; 15/02/2021
0000F50C D1E5                <1>     shl     ebp, 1 ; 8 bytes per pixel (position&color)
                             <1> sysvideo_39_67:
0000F50E 66B90008            <1>     mov     cx, 2048
0000F512 39CD                <1>     cmp     ebp, ecx
0000F514 7302                <1>     jnb     short sysvideo_39_68
0000F516 89E9                <1>     mov     ecx, ebp
                             <1> sysvideo_39_68:
                             <1>     ; multi colors pixel writing
0000F518 BF00760900          <1>     mov     edi, VBE3SAVERESTOREBLOCK ; kernel buffer for
                             <1>                                       ; 2048 byte data
                             <1>     ; esi = user's source buffer for pixel positions
                             <1>     ; ecx = byte count
0000F51D E843160000          <1>     call    transfer_from_user_buffer
0000F522 7281                <1>     jc      short sysvideo_39_56 ; error
                             <1>     ; ecx  = transfer count
                             <1>
                             <1>     ; write pixels & colors as defined in user buffer
                             <1>     ; ecx = byte count (2,4,6,8 times pixel count)
                             <1>     ; edi = system buffer address
                             <1>
0000F524 56                  <1>     push    esi ; **
0000F525 51                  <1>     push    ecx ; *
                             <1>
0000F526 89FE                <1>     mov     esi, edi
0000F528 8B3D[929D0100]      <1>     mov     edi, [v_mem]
                             <1>
                             <1>     ; 08/02/2021
0000F52E C1E903              <1>     shr     ecx, 3 ; pixel count
                             <1>
                             <1>     ;mov    ebx, [v_siz]
                             <1>
0000F531 803D[919D0100]08    <1>     cmp     byte [v_bpp], 8 ; 8bpp
0000F538 7715                <1>     ja      short sysvideo_39_71
                             <1> sysvideo_39_69:
                             <1>     ; 8bpp
0000F53A AD                  <1>     lodsd   ; position
0000F53B 89C2                <1>     mov     edx, eax
0000F53D AD                  <1>     lodsd   ; color
0000F53E 39DA                <1>     cmp     edx, ebx ; < [v_siz]
0000F540 7309                <1>     jnb     short sysvideo_39_70
0000F542 880417              <1>     mov     [edi+edx], al
                             <1>     ; 06/03/2021
0000F545 FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_70:
0000F54B E2ED                <1>     loop    sysvideo_39_69
0000F54D EB51                <1>     jmp     short sysvideo_39_78
                             <1> sysvideo_39_71:
0000F54F 803D[919D0100]18    <1>     cmp     byte [v_bpp], 24 ; 24bpp
0000F556 7735                <1>     ja      short sysvideo_39_76 ; 32bpp
0000F558 721D                <1>     jb      short sysvideo_39_74 ; 16bpp
                             <1> sysvideo_39_72:
                             <1>     ; 24bpp
0000F55A AD                  <1>     lodsd   ; position
0000F55B 89C2                <1>     mov     edx, eax
0000F55D AD                  <1>     lodsd   ; color
0000F55E 39DA                <1>     cmp     edx, ebx ; < [v_siz]
0000F560 7311                <1>     jnb     short sysvideo_39_73
0000F562 880417              <1>     mov     [edi+edx], al
0000F565 42                  <1>     inc     edx
0000F566 C1E808              <1>     shr     eax, 8
0000F569 66890417            <1>     mov     [edi+edx], ax
0000F56D FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_73:
0000F573 E2E5                <1>     loop    sysvideo_39_72
0000F575 EB29                <1>     jmp     short sysvideo_39_78
                             <1> sysvideo_39_74:
                             <1>     ; 16bpp
0000F577 AD                  <1>     lodsd   ; position
0000F578 89C2                <1>     mov     edx, eax
0000F57A AD                  <1>     lodsd   ; color
0000F57B 39DA                <1>     cmp     edx, ebx ; < [v_siz]
0000F57D 730A                <1>     jnb     short sysvideo_39_75
0000F57F 66890417            <1>     mov     [edi+edx], ax
0000F583 FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_75:
```

```
0000F589 E2EC                <1>     loop    sysvideo_39_74
0000F58B EB13                <1>     jmp     short sysvideo_39_78
                             <1> sysvideo_39_76:
                             <1>     ; 32bpp
0000F58D AD                  <1>     lodsd   ; position
0000F58E 89C2                <1>     mov     edx, eax
0000F590 AD                  <1>     lodsd   ; color
0000F591 39DA                <1>     cmp     edx, ebx ; < [v_siz]
0000F593 7309                <1>     jnb     short sysvideo_39_77
0000F595 890417              <1>     mov     [edi+edx], eax
0000F598 FF05[6C8E0100]      <1>     inc     dword [u.r0]
                             <1> sysvideo_39_77:
0000F59E E2ED                <1>     loop    sysvideo_39_76
                             <1> sysvideo_39_78:
0000F5A0 59                  <1>     pop     ecx ; *
0000F5A1 5E                  <1>     pop     esi ; **
                             <1>
0000F5A2 29CD                <1>     sub     ebp, ecx
0000F5A4 762A                <1>     jna     short sysvideo_39_79
0000F5A6 01CE                <1>     add     esi, ecx
0000F5A8 E961FFFFFF          <1>     jmp     sysvideo_39_67
                             <1> ;sysvideo_39_79:
                             <1> ;   jmp     sysret
                             <1>
                             <1> sysvideo_16:
                             <1>     ; 11/08/2022
                             <1>     ; 23/07/2022
                             <1>     ; 06/03/2021
                             <1>     ; 23/11/2020
0000F5AD 80FF04              <1>     cmp     bh, 4
                             <1>     ;jb     sysvideo_39 ; bh = 3, pixel r/w
                             <1>     ;ja     short sysvideo_17
                             <1>     ; 23/07/2022
0000F5B0 7407                <1>     je      short sysvideo_16_0
0000F5B2 7721                <1>     ja      short sysvideo_17
0000F5B4 E94EFCFFFF          <1>     jmp     sysvideo_39
                             <1> sysvideo_16_0:
                             <1>     ; BH = 4
                             <1>     ; Direct User Access for CGA video memory.
                             <1>     ; Setup user's page tables for direct access to 0B8000h.
                             <1>     ;
                             <1>     ; Permission checks are not implemented yet !
                             <1>     ; (11/07/2016)
                             <1>
0000F5B9 B800800B00          <1>     mov     eax, 0B8000h
                             <1>     ;mov    ecx, 8 ; 8 pages (8*4K=32K)
                             <1>     ; 11/08/2022
0000F5BE 31C9                <1>     xor     ecx, ecx
0000F5C0 B108                <1>     mov     cl, 8
0000F5C2 89C3                <1>     mov     ebx, eax ; 12/05/2017 ; virtual = physical
0000F5C4 E8066AFFFF          <1>     call    direct_memory_access
                             <1>     ;jc     sysret
0000F5C9 7205                <1>     jc      short sysvideo_39_79 ; 06/03/2021
                             <1>     ; eax = 0B8000h if there is not an error
0000F5CB A3[6C8E0100]        <1>     mov     [u.r0], eax
                             <1> sysvideo_39_79: ; 08/01/2021
0000F5D0 E9B9D4FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_17:
                             <1>     ; 23/07/2022
                             <1>     ; 23/12/2020
                             <1>     ; 11/12/2020
                             <1>     ; 10/12/2020
                             <1>     ; 23/11/2020
0000F5D5 80FF06              <1>     cmp     bh, 6
0000F5D8 7424                <1>     je      short sysvideo_17_0 ; 23/07/2022
0000F5DA 7205                <1>     jb      short sysvideo_18 ; 23/07/2022
0000F5DC E95E010000          <1>     jmp     sysvideo_20 ; ja
                             <1>
                             <1>     ; 23/07/2022
                             <1> sysvideo_18:
                             <1>     ; BH = 5
                             <1>     ; Direct User Access for VGA video memory.
                             <1>     ; Setup user's page tables for direct access to 0A0000h.
                             <1>     ;
                             <1>     ; Permission checks are not implemented yet !
                             <1>     ; (11/07/2016)
                             <1>
0000F5E1 B800000A00          <1>     mov     eax, 0A0000h
0000F5E6 B910000000          <1>     mov     ecx, 16 ; 16 pages (16*4K=64K)
0000F5EB 89C3                <1>     mov     ebx, eax ; 12/05/2017 ; virtual = physical
0000F5ED E8DD69FFFF          <1>     call    direct_memory_access
                             <1>     ;jc     sysret
                             <1>     ; 23/07/2022
0000F5F2 7205                <1>     jc      short sysvideo_18_0
                             <1>     ; eax = 0A0000h if there is not an error
0000F5F4 A3[6C8E0100]        <1>     mov     [u.r0], eax
                             <1> sysvideo_18_0:
0000F5F9 E990D4FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_17_0:
                             <1>     ; BH = 6
                             <1>     ; Direct User Access to Linear Frame Buffer.
                             <1>     ; Setup user's page tables for direct access to LFB.
                             <1>     ;
                             <1>     ; Permission checks are not implemented yet !
                             <1>     ; (10/12/2020)
                             <1>
0000F5FE 80FBFF              <1>     cmp     bl, 0FFh ; current video mode
0000F601 722C                <1>     jb      short sysvideo_17_2 ; for desired video mode
                             <1>
0000F603 381D[2E680000]      <1>     cmp     [CRT_MODE], bl ; VESA VBE video mode ?
0000F609 750E                <1>     jne     short sysvideo_17_1
0000F60B 668B0D[269D0100]    <1>     mov     cx, [video_mode]
0000F612 6681E1FF01          <1>     and     cx, 1FFh
0000F617 EB29                <1>     jmp     short sysvideo_17_3
                             <1> sysvideo_17_1:
                             <1>     ; 11/12/2020
0000F619 88DF                <1>     mov     bh, bl ; 0FFh
0000F61B 8A1D[2E680000]      <1>     mov     bl, [CRT_MODE] ; VGA/CGA video mode
0000F621 8B2D[688E0100]      <1>     mov     ebp, [u.usp]  ; ebp points to user's registers
                             <1>     ; 23/12/2020
0000F627 895D10              <1>     mov     [ebp+16], ebx ; return to user with EBX value
0000F62A E95FD4FFFF          <1>     jmp     sysret ; return to user with EAX = 0
                             <1> sysvideo_17_2:
                             <1>     ; bl = VESA video mode - 100h
0000F62F B701                <1>     mov     bh, 1 ; bx = 1XXh
0000F631 53                  <1>     push    ebx ; requested vesa video mode
0000F632 E88044FFFF          <1>     call    vbe_biosfn_return_current_mode
0000F637 59                  <1>     pop     ecx ; requested vesa video mode
0000F638 6681E3FF01          <1>     and     bx, 1FFh
0000F63D 6639D9              <1>     cmp     cx, bx
0000F640 7564                <1>     jne     short sysvideo_17_8
                             <1> sysvideo_17_3:
```

```
0000F642 663B0D[329D0100]    <1>     cmp     cx, [LFB_Info+LFBINFO.mode]
0000F649 755B                <1>     jne     short sysvideo_17_8
                             <1> sysvideo_17_4:
                             <1>     ; 11/12/2020
0000F64B A1[349D0100]        <1>     mov     eax, [LFB_Info+LFBINFO.LFB_addr]
                             <1>     ; 21/12/2020
0000F650 09C0                <1>     or      eax, eax
0000F652 744D                <1>     jz      short sysvideo_17_7
                             <1>     ;
0000F654 8B0D[389D0100]      <1>     mov     ecx, [LFB_Info+LFBINFO.LFB_size] ; buff size
0000F65A 89C3                <1>     mov     ebx, eax ; user's address = physical address
                             <1>     ;push   ebx
0000F65C 51                  <1>     push    ecx
                             <1>     ; 21/12/2020
0000F65D 81C1FF0F0000        <1>     add     ecx, 4095  ; PAGESIZE - 1
                             <1>     ; 14/12/2020
0000F663 C1E90C              <1>     shr     ecx, 12  ; convert bytes to pages
0000F666 E86469FFFF          <1>     call    direct_memory_access
0000F66B 5A                  <1>     pop     edx  ; linear frame buffer size in bytes
                             <1>     ;pop    eax  ; linear frame buffer address (physical)
0000F66C 7233                <1>     jc      short sysvideo_17_7 ; [u.r0] = eax = 0
                             <1> sysvideo_17_5:
0000F66E 668B0D[3E9D0100]    <1>     mov     cx, [LFB_Info+LFBINFO.Y_res] ; screen height
0000F675 C1E110              <1>     shl     ecx, 16
0000F678 668B0D[3C9D0100]    <1>     mov     cx,  [LFB_Info+LFBINFO.X_res] ; screen width
0000F67F 31DB                <1>     xor     ebx, ebx
0000F681 8A1D[409D0100]      <1>     mov     bl, [LFB_Info+LFBINFO.bpp] ; bits per pixel
0000F687 8A3D[329D0100]      <1>     mov     bh, [LFB_Info+LFBINFO.mode] ; XX part of 1XXh
                             <1> sysvideo_26_4: ; 23/12/2020
0000F68D 8B2D[688E0100]      <1>     mov     ebp, [u.usp]  ; ebp points to user's registers
0000F693 895514              <1>     mov     [ebp+20], edx ; return to user with EDX value
0000F696 895D10              <1>     mov     [ebp+16], ebx ; EBX
0000F699 894D18              <1>     mov     [ebp+24], ecx ; ECX
                             <1> sysvideo_17_6:
0000F69C A3[6C8E0100]        <1>     mov     [u.r0], eax ; LFB address
                             <1> sysvideo_17_7:
0000F6A1 E9E8D3FFFF          <1>     jmp     sysret
                             <1> sysvideo_17_8:
                             <1>     ; cx = mode
                             <1>     ; 21/12/2020
0000F6A6 80CD40              <1>     or      ch, 40h  ; Linear frame buffer flag
0000F6A9 E82C42FFFF          <1>     call    _vbe_biosfn_return_mode_info
0000F6AE 72F1                <1>     jc      short sysvideo_17_7
0000F6B0 EB99                <1>     jmp     short sysvideo_17_4
                             <1>
                             <1> sysvideo_19:
                             <1>     ; 23/07/2022
                             <1>     ; 22/01/2021
                             <1>     ; 12/12/2020
                             <1>     ; 11/12/2020
                             <1>     ; 23/11/2020
                             <1>     ; BH = 7
                             <1>     ; Get (Super/Extended VGA) mode
                             <1>     ; and Linear Frame Buffer info.
                             <1>
                             <1>     ; 22/01/2021
0000F6B2 B3FF                <1>     mov     bl, 0FFh
                             <1>     ; 11/12/2020
                             <1>     ;cmp    byte [CRT_MODE], 0FFh ; (extended mode?)
                             <1>     ; 22/01/2021
0000F6B4 381D[2E680000]      <1>     cmp     [CRT_MODE], bl  ; 0FFh
                             <1>     ;jb     short sysvideo_17_1 ; not a VESA VBE mode
                             <1>     ; 23/07/2022
                             <1>     ; 12/12/2020
0000F6BA 7305                <1>     jnb     short sysvideo_19_0
                             <1>     ; 23/07/2022
0000F6BC E958FFFFFF          <1>     jmp     sysvideo_17_1
                             <1>
                             <1> sysvideo_19_0:
0000F6C1 E8F143FFFF          <1>     call    vbe_biosfn_return_current_mode
0000F6C6 6681E3FF01          <1>     and     bx, 1FFh
0000F6CB 663B1D[329D0100]    <1>     cmp     bx, [LFB_Info+LFBINFO.mode]
0000F6D2 750D                <1>     jne     short sysvideo_19_2
                             <1> sysvideo_19_1:
0000F6D4 A1[349D0100]        <1>     mov     eax, [LFB_Info+LFBINFO.LFB_addr]
0000F6D9 8B15[389D0100]      <1>     mov     edx, [LFB_Info+LFBINFO.LFB_size]
0000F6DF EB8D                <1>     jmp     sysvideo_17_5
                             <1> sysvideo_19_2:
0000F6E1 E8F441FFFF          <1>     call    _vbe_biosfn_return_mode_info
0000F6E6 73EC                <1>     jnc     short sysvideo_19_1
0000F6E8 E9A1D3FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_20_1:
                             <1>     ; cx = vesa video mode
0000F6ED 6689C8              <1>     mov     ax, cx
0000F6F0 663D0001            <1>     cmp     ax, 100h
0000F6F4 725C                <1>     jb      short sysvideo_20_0 ; VGA/CGA mode
0000F6F6 663DFF01            <1>     cmp     ax, 1FFh
                             <1>     ;ja     short sysvideo_20_4 ; not valid
0000F6FA 773E                <1>     ja      short sysvideo_20_3
0000F6FC 50                  <1>     push    eax
0000F6FD 6689C3              <1>     mov     bx, ax
0000F700 66B8024F            <1>     mov     ax, 4F02h
                             <1>
                             <1>     ; simulate _int10h (int 31h) for func 4F02h
                             <1>     ;pushfd
                             <1>     ;push   cs
                             <1>     ;push   sysvideo_20_1_retn
                             <1>     ;push   es ; *
                             <1>     ;push   ds ; ** ; SAVE WORK AND PARAMETER REGISTERS
                             <1>     ;jmp    VBE_func
                             <1> ;sysvideo_20_1_retn:
                             <1>
0000F704 E81120FFFF          <1>     call    _int10h ; simulate int 10h (int 31h)
                             <1>
0000F709 6683F84F            <1>     cmp     ax, 004Fh
0000F70D 58                  <1>     pop     eax
0000F70E 752A                <1>     jne     short sysvideo_20_3 ; error
                             <1>     ;pop    eax
0000F710 40                  <1>     inc     eax
0000F711 A3[6C8E0100]        <1>     mov     [u.r0], eax ; video mode + 1
0000F716 09D2                <1>     or      edx, edx ; is LFBINFO requested by user ?
                             <1>     ;jz     short sysvideo_20_4
0000F718 7420                <1>     jz      short sysvideo_20_3 ; no
                             <1>
                             <1>     ; 11/12/2020
                             <1>     ; Check LFBINFO table/structure
                             <1>     ; (it is set by vbe2 'vbe_biosfn_set_mode'
                             <1>     ; but if vbe3 vbios pmi is in use,
                             <1>     ; it will not set LFBINFO table)
                             <1>
0000F71A 52                  <1>     push    edx
0000F71B 48                  <1>     dec     eax  ; video mode
0000F71C BE[329D0100]        <1>     mov     esi, LFB_Info
```

```
0000F721 663B06              <1>     cmp    ax, [esi+LFBINFO.mode]
0000F724 7407                <1>     je     short sysvideo_20_2
                             <1>
0000F726 E8AF41FFFF          <1>     call   _vbe_biosfn_return_mode_info
                             <1>     ;jnc   short sysvideo_20_2
0000F72B 723B                <1>     jc     short sysvideo_20_4 ; edx = 0
                             <1>
                             <1>     ;; clear LFBINFO table for invalidating
                             <1>     ;mov   ecx, LFBINFO.size ; 16
                             <1>     ;mov   edi, esi ; LFB_Info table address
                             <1>     ;xor   eax, eax
                             <1>     ;rep   stosb
                             <1>
                             <1> sysvideo_20_2:
                             <1>     ;pop   ecx
                             <1>     ;mov   edi, ecx ; user buffer
0000F72D 5F                  <1>     pop    edi
0000F72E B910000000          <1>     mov    ecx, LFBINFO.size ; 16
0000F733 E8E3130000          <1>     call   transfer_to_user_buffer ; fast transfer
0000F738 722F                <1>     jc     short sysvideo_20_5
                             <1>
                             <1>     ;jmp   sysret
                             <1> sysvideo_20_3:
                             <1>     ;pop   eax ; [u.r0] = 0
                             <1> ;sysvideo_20_4:
0000F73A E94FD3FFFF          <1>     jmp    sysret
                             <1>
                             <1>     ; 23/07/2022
                             <1> sysvideo_20:
                             <1>     ; 11/12/2020
                             <1>     ; 23/11/2020
0000F73F 80FF08              <1>     cmp    bh, 8
                             <1>     ; 08/08/2022
                             <1>     ;jb    short sysvideo_19 ; video mode & lfb info
0000F742 7407                <1>     je     short sysvideo_20_6
                             <1>     ; 23/07/2022
0000F744 7733                <1>     ja     short sysvideo_21 ; 12/12/2020
                             <1>     ; 08/08/2022 - jb
0000F746 E967FFFFFF          <1>     jmp    sysvideo_19
                             <1> sysvideo_20_6:
                             <1>     ; BH = 8
                             <1>     ; Set (Super/Extended VGA) mode & return LFB info
                             <1>
                             <1>     ; 11/12/2020
0000F74B 80FBFF              <1>     cmp    bl, 0FFh  ; CGA/VGA mode ?
0000F74E 739D                <1>     jnb    short sysvideo_20_1
                             <1>
                             <1>     ;xor   ah, ah
0000F750 88D8                <1>     mov    al, bl
                             <1> sysvideo_20_0:
0000F752 E8C31FFFFF          <1>     call   _int10h  ; uses vbe3 pmi32 option
0000F757 83F8FF              <1>     cmp    eax, 0FFFFFFFFh ; -1
0000F75A 74DE                <1>     je     short sysvideo_20_3 ; error
                             <1>
                             <1>     ; 11/12/2020
                             <1>     ; alternative (it does not use vbe3 pmi32)
                             <1>     ;push  eax
                             <1>     ;call  _set_mode
                             <1>     ;pop   eax
                             <1>     ;jc    short sysvideo_20_3
                             <1>
                             <1>     ;inc   eax
0000F75C FEC0                <1>     inc    al
                             <1>     ;mov   [u.r0], ax ; video mode + 1
0000F75E A2[6C8E0100]        <1>     mov    [u.r0], al
0000F763 E926D3FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysvideo_20_4:
0000F768 5A                  <1>     pop    edx
                             <1> sysvideo_20_5:
0000F769 31D2                <1>     xor    edx, edx ; 0
                             <1>     ; edx = 0 -> invalid LFBINFO data
0000F76B 8B2D[688E0100]      <1>     mov    ebp, [u.usp]  ; ebp points to user's registers
0000F771 895514              <1>     mov    [ebp+20], edx ; return to user with EDX value
0000F774 E915D3FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysvideo_21:
                             <1>     ; 23/07/2022
                             <1>     ; 04/01/2021
                             <1>     ; 03/12/2020
0000F779 80FF0A              <1>     cmp    bh, 10
                             <1>     ;jb    sysvideo_22 ; VESA VBE3 pmi parms
                             <1>     ; 23/07/2022
0000F77C 723F                <1>     jb     short sysvideo_21_19
                             <1>     ; 23/12/2020
                             <1>     ;je    sysvideo_26 ; Video memory mapping
                             <1>     ; 23/07/2022
0000F77E 7442                <1>     je     short sysvideo_21_20
                             <1>
                             <1>     ; 04/01/2020
0000F780 80FF0B              <1>     cmp    bh, 11
                             <1>     ;ja    sysvideo_27
                             <1>     ; 23/07/2022
0000F783 7742                <1>     ja     short sysvideo_21_21
                             <1>
                             <1>     ; BH = 11
                             <1>     ; set/read DAC color registers (for 8bpp)
                             <1>
0000F785 80FB04              <1>     cmp    bl, 4
                             <1>     ;jnb   sysvideo_21_7  ; BMP file type palette
                             <1>                           ; handling
                             <1>     ; 23/07/2022
                             <1>     ;jnb   short sysvideo_21_7
                             <1>     ; 08/08/2022
0000F788 7205                <1>     jb     short sysvideo_21_18
0000F78A E989000000          <1>     jmp    sysvideo_21_7
                             <1> sysvideo_21_18:
0000F78F F6C301              <1>     test   bl, 1
0000F792 7563                <1>     jnz    short sysvideo_21_4 ; set/write DAC colors
                             <1>
                             <1>     ; Read DAC color register or all DAC color registers
0000F794 F6C302              <1>     test   bl, 2  ; read single DAC color register
0000F797 7433                <1>     jz     short sysvideo_21_2 ; read all DAC color regs
                             <1>
                             <1>     ; read single DAC color register
                             <1>     ; CL = DAC color register (index)
                             <1>
0000F799 66BAC703            <1>     mov    dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
0000F79D 88C8                <1>     mov    al, cl  ; DAC color register
0000F79F 31C9                <1>     xor    ecx, ecx ; (this may not be necessary)
0000F7A1 EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C9h ; VGAREG_DAC_DATA
0000F7A2 B2C9                <1>     mov    dl, 0C9h
0000F7A4 EC                  <1>     in     al, dx
```

```
0000F7A5 88C4                <1>     mov     ah, al  ; red
0000F7A7 EC                  <1>     in      al, dx
0000F7A8 88C1                <1>     mov     cl, al  ; green
0000F7AA EC                  <1>     in      al, dx
0000F7AB 88C5                <1>     mov     ch, al  ; blue
0000F7AD C1E108              <1>     shl     ecx, 8
0000F7B0 88E1                <1>     mov     cl, ah  ; red
                             <1>     ; CL = Red, CH = Green, byte 3 = Blue, byte 4 = 0
                             <1> sysvideo_21_0:
0000F7B2 890D[6C8E0100]      <1>     mov     [u.r0], ecx
                             <1> sysvideo_21_1:
0000F7B8 E9D1D2FFFF          <1>     jmp     sysret
                             <1> sysvideo_21_19:
                             <1>     ; 23/07/2022
0000F7BD E97A010000          <1>     jmp     sysvideo_22
                             <1> sysvideo_21_20:
                             <1>     ; 23/07/2022
0000F7C2 E930020000          <1>     jmp     sysvideo_26
                             <1> sysvideo_21_21:
                             <1>     ; 23/07/2022
0000F7C7 E9B6020000          <1>     jmp     sysvideo_27
                             <1> sysvideo_21_2:
                             <1>     ; read all DAC color registers
0000F7CC 89CB                <1>     mov     ebx, ecx ; user's buffer address
0000F7CE BF00600900          <1>     mov     edi, VBE3STACKADDR
0000F7D3 89FE                <1>     mov     esi, edi
                             <1>     ;mov    ecx, 768 ; 256*3
                             <1>     ; 08/08/2022
0000F7D5 29C9                <1>     sub     ecx, ecx
0000F7D7 B503                <1>     mov     ch, 3
                             <1>     ; ecx = 768 = 300h
                             <1>     ;push   ecx
0000F7D9 66BAC703            <1>     mov     dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
0000F7DD 28C0                <1>     sub     al, al ; 0
0000F7DF EE                  <1>     out     dx, al
                             <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
0000F7E0 B2C9                <1>     mov     dl, 0C9h
                             <1> sysvideo_21_3:
0000F7E2 EC                  <1>     in      al, dx
0000F7E3 AA                  <1>     stosb
0000F7E4 EC                  <1>     in      al, dx
0000F7E5 AA                  <1>     stosb
0000F7E6 EC                  <1>     in      al, dx
0000F7E7 AA                  <1>     stosb
0000F7E8 E2F8                <1>     loop    sysvideo_21_3
                             <1>     ;pop    ecx
                             <1>     ; 18/08/2022
0000F7EA B503                <1>     mov     ch, 3
                             <1>     ; ecx = 768 = 300h
                             <1>
0000F7EC 89DF                <1>     mov     edi, ebx ; user's buffer address
                             <1>     ;mov    esi, VBE3STACKADDR
                             <1>     ;mov    ecx, 256*3 = 768
0000F7EE E828130000          <1>     call    transfer_to_user_buffer
0000F7F3 72C3                <1>     jc      short sysvideo_21_1
                             <1>     ;mov    [u.r0], ecx ; actual transfer count
0000F7F5 EBBB                <1>     jmp     short sysvideo_21_0
                             <1>
                             <1> sysvideo_21_4:
                             <1>     ; Set/Write DAC color register or all registers
0000F7F7 F6C302              <1>     test    bl, 2 ; write/set single DAC color register
0000F7FA 7456                <1>     jz      short sysvideo_21_5 ; set all DAC color regs
                             <1>
                             <1>     ; set single DAC color register
                             <1>     ; CL = DAC color register (index)
                             <1>     ; (byte 1 = Red, byte 2 = Green, byte 3 = Blue)
                             <1>
0000F7FC 66BAC803            <1>     mov     dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
0000F800 89C8                <1>     mov     eax, ecx ; DAC color register (index)
0000F802 C1E910              <1>     shr     ecx, 16  ; CL = green, AH = Red
0000F805 EE                  <1>     out     dx, al
                             <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
0000F806 FEC2                <1>     inc     dl
0000F808 88E0                <1>     mov     al, ah  ; Red
0000F80A EE                  <1>     out     dx, al
0000F80B 88C8                <1>     mov     al, cl  ; Green
0000F80D EE                  <1>     out     dx, al
0000F80E 88E8                <1>     mov     al, ch  ; Blue
0000F810 EE                  <1>     out     dx, al
                             <1>     ;rol    ecx, 8
0000F811 C1E108              <1>     shl     ecx, 8  ; 21/02/2021
0000F814 88E1                <1>     mov     cl, ah  ; Red
                             <1>             ; ecx = 00BBGGRRh
0000F816 EB9A                <1>     jmp     short sysvideo_21_0
                             <1>
                             <1>     ; 23/07/2022
                             <1> sysvideo_21_7:
                             <1>     ; BMP file type palette handling
                             <1>
0000F818 F6C301              <1>     test    bl, 1
                             <1>     ;jnz    short sysvideo_21_12 ; set/write DAC colors
                             <1>     ; 08/08/2022
0000F81B 7405                <1>     jz      short sysvideo_21_16
0000F81D E9A3000000          <1>     jmp     sysvideo_21_12
                             <1>
                             <1> sysvideo_21_16:
                             <1>     ; Read DAC color register or all DAC color registers
0000F822 F6C302              <1>     test    bl, 2  ; read single DAC color register
0000F825 7460                <1>     jz      short sysvideo_21_10 ; read all DAC color regs
                             <1>
                             <1>     ; read single DAC color register
                             <1>     ; CL = DAC color register (index)
                             <1>
0000F827 66BAC703            <1>     mov     dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
0000F82B 88C8                <1>     mov     al, cl  ; DAC color register
0000F82D 31C9                <1>     xor     ecx, ecx
0000F82F EE                  <1>     out     dx, al
                             <1>     ;mov    dx, 3C9h ; VGAREG_DAC_DATA
0000F830 B2C9                <1>     mov     dl, 0C9h
0000F832 EC                  <1>     in      al, dx
0000F833 C0E002              <1>     shl     al, 2
0000F836 88C5                <1>     mov     ch, al  ; red
0000F838 EC                  <1>     in      al, dx
0000F839 C0E002              <1>     shl     al, 2
0000F83C 88C1                <1>     mov     cl, al  ; green
0000F83E EC                  <1>     in      al, dx
0000F83F C0E002              <1>     shl     al, 2
                             <1>     ; 21/02/2021
0000F842 C1E108              <1>     shl     ecx, 8
0000F845 88C1                <1>     mov     cl, al  ; blue
                             <1>     ; CL = Blue, CH = Green, byte 3 = Red, byte 4 = 0
                             <1> sysvideo_21_8:
0000F847 890D[6C8E0100]      <1>     mov     [u.r0], ecx
                             <1> sysvideo_21_9:
```

```
0000F84D E93CD2FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysvideo_21_5:
                             <1>     ; write/set all DAC color registers
0000F852 89CE                <1>     mov    esi, ecx ; user's buffer address
0000F854 BF00600900          <1>     mov    edi, VBE3STACKADDR
0000F859 89FB                <1>     mov    ebx, edi
0000F85B B900030000          <1>     mov    ecx, 768 ; 256*3
0000F860 E800130000          <1>     call   transfer_from_user_buffer
                             <1>     ;jc    short sysvideo_21_1
                             <1>     ; 08/08/2022
0000F865 7305                <1>     jnc    short sysvideo_21_17
0000F867 E94CFFFFFF          <1>     jmp    sysvideo_21_1
                             <1> sysvideo_21_17:
0000F86C 890D[6C8E0100]      <1>     mov    [u.r0], ecx ; actual transfer count
                             <1>
0000F872 89DE                <1>     mov    esi, ebx ; VBE3STACKADDR
0000F874 66BAC803            <1>     mov    dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
0000F878 28C0                <1>     sub    al, al ; 0
0000F87A EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C9h ; VGAREG_DAC_DATA
0000F87B FEC2                <1>     inc    dl
                             <1> sysvideo_21_6:
0000F87D AC                  <1>     lodsb
0000F87E EE                  <1>     out    dx, al
0000F87F AC                  <1>     lodsb
0000F880 EE                  <1>     out    dx, al
0000F881 AC                  <1>     lodsb
0000F882 EE                  <1>     out    dx, al
0000F883 E2F8                <1>     loop   sysvideo_21_6
0000F885 EBC6                <1>     jmp    short sysvideo_21_9
                             <1>
                             <1> sysvideo_21_10:
                             <1>     ; read all DAC color registers
0000F887 89CD                <1>     mov    ebp, ecx ; user's buffer address
0000F889 BF00600900          <1>     mov    edi, VBE3STACKADDR
0000F88E 89FE                <1>     mov    esi, edi
0000F890 B900040000          <1>     mov    ecx, 1024 ; 256*4
0000F895 51                  <1>     push   ecx
0000F896 66BAC703            <1>     mov    dx, 3C7h ; VGAREG_DAC_READ_ADDRESS
0000F89A 28C0                <1>     sub    al, al ; 0
0000F89C EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C9h ; VGAREG_DAC_DATA
0000F89D B2C9                <1>     mov    dl, 0C9h
                             <1> sysvideo_21_11:
0000F89F 31DB                <1>     xor    ebx, ebx
0000F8A1 EC                  <1>     in     al, dx  ; Red
0000F8A2 C0E002              <1>     shl    al, 2
0000F8A5 88C7                <1>     mov    bh, al
0000F8A7 EC                  <1>     in     al, dx  ; Green
0000F8A8 C0E002              <1>     shl    al, 2
0000F8AB 88C3                <1>     mov    bl, al
0000F8AD EC                  <1>     in     al, dx  ; Blue
0000F8AE C0E002              <1>     shl    al, 2
0000F8B1 C1E308              <1>     shl    ebx, 8
0000F8B4 89D8                <1>     mov    eax, ebx ; 00RRGGBBh
0000F8B6 AB                  <1>     stosd
0000F8B7 E2E6                <1>     loop   sysvideo_21_11
0000F8B9 59                  <1>     pop    ecx
                             <1>
0000F8BA 89EF                <1>     mov    edi, ebp ; user's buffer address
                             <1>     ;mov   esi, VBE3STACKADDR
                             <1>     ;mov   ecx, 1024 = 4*256
0000F8BC E85A120000          <1>     call   transfer_to_user_buffer
0000F8C1 728A                <1>     jc     short sysvideo_21_9
                             <1>     ;mov   [u.r0], ecx ; actual transfer count
0000F8C3 EB82                <1>     jmp    short sysvideo_21_8
                             <1>
                             <1> sysvideo_21_12:
                             <1>     ; Set/Write DAC color register or all registers
0000F8C5 F6C302              <1>     test   bl, 2 ; write/set single DAC color register
0000F8C8 742A                <1>     jz     short sysvideo_21_13 ; set all DAC color regs
                             <1>
                             <1>     ; set single DAC color register
                             <1>     ; CL = DAC color register (index)
                             <1>     ; (byte 1 = Blue, byte 2 = Green, byte 3 = Red)
                             <1>
0000F8CA 66BAC803            <1>     mov    dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
0000F8CE 88C8                <1>     mov    al, cl  ; DAC color register (index)
0000F8D0 88EC                <1>     mov    ah, ch  ; Blue
0000F8D2 C1E910              <1>     shr    ecx, 16
0000F8D5 EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C9h ; VGAREG_DAC_DATA
0000F8D6 FEC2                <1>     inc    dl
0000F8D8 88E8                <1>     mov    al, ch  ; Red
0000F8DA C0E802              <1>     shr    al, 2
0000F8DD EE                  <1>     out    dx, al
0000F8DE 88C8                <1>     mov    al, cl  ; Green
0000F8E0 C0E802              <1>     shr    al, 2
0000F8E3 EE                  <1>     out    dx, al
0000F8E4 88E0                <1>     mov    al, ah  ; Blue
0000F8E6 C0E802              <1>     shr    al, 2
0000F8E9 EE                  <1>     out    dx, al
                             <1>     ;rol   ecx, 8
0000F8EA C1E108              <1>     shl    ecx, 8 ; 21/02/2021
0000F8ED 88E1                <1>     mov    cl, ah
0000F8EF E953FFFFFF          <1>     jmp    sysvideo_21_8 ; 08/08/2022
                             <1>
                             <1> sysvideo_21_13:
                             <1>     ; write/set all DAC color registers
0000F8F4 89CE                <1>     mov    esi, ecx ; user's buffer address
0000F8F6 BF00600900          <1>     mov    edi, VBE3STACKADDR
0000F8FB 89FB                <1>     mov    ebx, edi
0000F8FD B900040000          <1>     mov    ecx, 1024 ; 256*4
0000F902 E85E120000          <1>     call   transfer_from_user_buffer
0000F907 722E                <1>     jc     short sysvideo_21_15
0000F909 890D[6C8E0100]      <1>     mov    [u.r0], ecx ; actual transfer count
                             <1>
0000F90F 89DE                <1>     mov    esi, ebx ; VBE3STACKADDR
0000F911 66BAC803            <1>     mov    dx, 3C8h ; VGAREG_DAC_WRITE_ADDRESS
0000F915 28C0                <1>     sub    al, al ; 0
0000F917 EE                  <1>     out    dx, al
                             <1>     ;mov   dx, 3C9h ; VGAREG_DAC_DATA
0000F918 FEC2                <1>     inc    dl
                             <1> sysvideo_21_14:
0000F91A AD                  <1>     lodsd
                             <1>     ; byte 0 = Blue, byte 1 = Green, byte 2 = Red
                             <1>     ; 21/02/2021
0000F91B 89C3                <1>     mov    ebx, eax ; BL = Blue, BH = Green
0000F91D C1CB08              <1>     ror    ebx, 8 ; BL = Green, BH = Red
0000F920 88F8                <1>     mov    al, bh
0000F922 C0E802              <1>     shr    al, 2
0000F925 EE                  <1>     out    dx, al ; Red
0000F926 88D8                <1>     mov    al, bl
```

```
0000F928 C0E802              <1>     shr     al, 2
0000F92B EE                  <1>     out     dx, al ; Green
0000F92C C1C308              <1>     rol     ebx, 8 ; BL = Blue
0000F92F 88D8                <1>     mov     al, bl
0000F931 C0E802              <1>     shr     al, 2
0000F934 EE                  <1>     out     dx, al ; Blue
0000F935 E2E3                <1>     loop    sysvideo_21_14
                             <1> sysvideo_21_15:
0000F937 E952D1FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_22:
                             <1>     ; 28/02/2021
                             <1>     ; 22/01/2021
                             <1>     ; 17/01/2021
                             <1>     ; 04/12/2020
                             <1>     ; 03/12/2020
                             <1>     ; BH = 9
                             <1>     ; Set/Get VESA VBE3 protected mode interface params
                             <1>
                             <1>     ; 22/01/2021
                             <1>     ;cmp    byte [vbe3], 3
                             <1>     ;jne    short sysvideo_25 ; not applicable if
                             <1>                               ; vbe3 compatible video bios
                             <1>                               ; is not detected by kernel
0000F93C 80FB02              <1>     cmp     bl, 2
                             <1>     ;ja     short sysvideo_25 ; bl > 2 not implemented
                             <1>     ; 17/01/2021
0000F93F 7716                <1>     ja      short sysvideo_22_0 ; srvs flag sub function
                             <1>     ;jb     short sysvideo_23
                             <1>
                             <1>     ; 21/01/2021
0000F941 803D[7C090000]03    <1>     cmp     byte [vbe3], 3
                             <1>     ;jne    short sysvideo_25 ; not applicable if
                             <1>                               ; vbe3 compatible video bios
                             <1>                               ; is not detected by kernel
0000F948 75ED                <1>     jne     short sysvideo_21_15 ; 28/02/2021
                             <1>
0000F94A 80FB01              <1>     cmp     bl, 1
0000F94D 7673                <1>     jna     short sysvideo_23
                             <1>
0000F94F 8A1D[249D0100]      <1>     mov     bl, [pmi32] ; Video bios 32 bit PMI functions
0000F955 EB78                <1>     jmp     short sysvideo_24
                             <1>
                             <1> sysvideo_22_0:
                             <1>     ; 17/01/2021
                             <1>     ; save/restore video state user permission
0000F957 80FB05              <1>     cmp     bl, 5
0000F95A 771E                <1>     ja      short sysvideo_22_2
0000F95C 7208                <1>     jb      short sysvideo_22_1
                             <1>     ; get srvs flag value/status
0000F95E 8A1D[889D0100]      <1>     mov     bl, [srvsf] ; 0 = disabled, 1 = enabled
0000F964 EB2C                <1>     jmp     short sysvideo_22_3
                             <1>
                             <1> sysvideo_22_1:
                             <1>     ; permission (root and multi tasking) check
0000F966 E836000000          <1>     call    sysvideo_22_4
0000F96B 736A                <1>     jnc     short sysvideo_25 ; not permitted !
                             <1>     ; cf = 1
0000F96D 80EB03              <1>     sub     bl, 3 ; disable = 0, enable = 1
                             <1>     ; 22/01/2021
0000F970 881D[889D0100]      <1>     mov     [srvsf], bl
0000F976 FEC3                <1>     inc     bl ; 1 = disabled, 2 = enabled
0000F978 EB18                <1>     jmp     short sysvideo_22_3
                             <1>
                             <1> sysvideo_22_2:
0000F97A 80FB06              <1>     cmp     bl, 6
                             <1>     ;ja     short sysvideo_25 ; invalid/unimplemented
                             <1>     ; 28/02/2021
0000F97D 7733                <1>     ja      short sysvideo_22_6
                             <1>     ; get VESA VBE number/status
0000F97F 8A25[7C090000]      <1>     mov     ah, [vbe3] ; vbe3 = 3, vbe2 = 2, others = 0
0000F985 A0[7D090000]        <1>     mov     al, [vbe2bios] ; bochs/qemu/vbox emulator status
0000F98A 66A3[6C8E0100]      <1>     mov     [u.r0], ax
0000F990 EB45                <1>     jmp     short sysvideo_25
                             <1>
                             <1> sysvideo_22_3:
                             <1>     ; 22/01/2021
0000F992 8A3D[899D0100]      <1>     mov     bh, [srvso] ; state options (> 80h -> svga)
0000F998 66891D[6C8E0100]    <1>     mov     [u.r0], bx ; function result is return value
0000F99F EB36                <1>     jmp     short sysvideo_25
                             <1>
                             <1> sysvideo_22_4:
                             <1>     ; 17/01/2021 - permission will be given by root only
0000F9A1 803D[46830100]00    <1>     cmp     byte [multi_tasking], 0 ; in single user mode
0000F9A8 7707                <1>     ja      short sysvideo_22_5
                             <1>     ; 19/01/2021
0000F9AA 803D[BE8E0100]01    <1>     cmp     byte [u.uid], 1 ; ([u.uid] = 0 -> root)
                             <1> sysvideo_22_5:
                             <1>     ; [multi_tasking] = 0 & [u.uid] = 0 -> CF = 1
                             <1>     ; otherwise -> CF = 0
0000F9B1 C3                  <1>     retn
                             <1>
                             <1> sysvideo_22_6:
                             <1>     ; 28/02/2021
0000F9B2 80FB09              <1>     cmp     bl, 9
0000F9B5 7720                <1>     ja      short sysvideo_25 ; invalid/unimplemented
0000F9B7 7436                <1>     je      short sysvideo_22_9
0000F9B9 80FB08              <1>     cmp     bl, 8
0000F9BC 721E                <1>     jb      short sysvideo_22_7
                             <1>
                             <1>     ; BL = 8
                             <1>     ; Set default true color bpp to 24
                             <1>
0000F9BE B318                <1>     mov     bl, 24
                             <1>     ;mov    [truecolor], al       ; 24bpp (RRGGBBh)
                             <1>     ;mov    [u.r0], al
                             <1>     ;jmp    short sysvideo_25
0000F9C0 EB25                <1>     jmp     short sysvideo_22_8
                             <1>
                             <1> sysvideo_23:
                             <1>     ; 17/01/2021
                             <1>     ; permission (root and multi tasking) check
0000F9C2 E8DAFFFFFF          <1>     call    sysvideo_22_4
0000F9C7 730E                <1>     jnc     short sysvideo_25 ; not permitted !
                             <1>
0000F9C9 881D[249D0100]      <1>     mov     [pmi32], bl ; 1 = enabled, 0 = disabled
                             <1> sysvideo_24:
0000F9CF FEC3                <1>     inc     bl
                             <1> sysvideo_22_10:    ; 28/02/2021
0000F9D1 881D[6C8E0100]      <1>     mov     [u.r0], bl ; function result is return value
                             <1> sysvideo_25:
0000F9D7 E9B2D0FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_22_7:
```

```
                                <1>     ; BL = 7
                                <1>     ; Set default true color bpp to 32
                                <1>     ; (it will set if [VBE3]=3)
                                <1>
                                <1>     ; Note: This sub function is used to set 24bpp
                                <1>     ; VESA VBE video modes to 32bpp.. because,
                                <1>     ; old hardware uses 24 bpp but new video hardware
                                <1>     ; uses 32bpp for same VESA VBE truecolor modes.
                                <1>     ; (For example: VBE mode 112h is 640*480, 24bpp but
                                <1>     ; new hardware uses/apply it as 640*480, 32bpp.)
                                <1>     ; So, TRDOS 386 v2.0.3 kernel will check [truecolor]
                                <1>     ; status is 32 bpp or not and it will change 24bpp
                                <1>     ; to 32bpp if default [truecolor] value is 32, for
                                <1>     ; same video mode number.
                                <1>
0000F9DC 803D[7C090000]03       <1>     cmp     byte [vbe3], 3
0000F9E3 75F2                   <1>     jne     short sysvideo_25 ; Only applicable
                                <1>                               ; for VBE3 video hardware!
0000F9E5 B320                   <1>     mov     bl, 32
                                <1> sysvideo_22_8:
0000F9E7 881D[73730100]         <1>     mov     [truecolor], bl       ; 32bpp (00RRGGBBh)
                                <1>     ;mov    [u.r0], bl
                                <1>     ;jmp    short sysvideo_25
0000F9ED EBE2                   <1>     jmp     short sysvideo_22_10
                                <1>
                                <1> sysvideo_22_9:
                                <1>     ; BL = 9
                                <1>     ; Return default true color bpp
0000F9EF 8A1D[73730100]         <1>     mov     bl, [truecolor]
0000F9F5 EBDA                   <1>     jmp     short sysvideo_22_10
                                <1> ;sysvideo_22_10:
                                <1>     ;mov    [u.r0], bl
                                <1>     ;jmp    sysret
                                <1>
                                <1> sysvideo_26:
                                <1>     ; 23/12/2020
                                <1>     ; BH = 10
                                <1>     ; Map video memory to user's buffer
                                <1>     ; (multiuser/owner r/w permisions are ignored
                                <1>     ;  for current TRDOS 386 version !)
                                <1>
0000F9F7 6681E100F0             <1>     and     cx, ~4095  ; clear low 12 bits
0000F9FC 09C9                   <1>     or      ecx, ecx ; start address of user's buffer
0000F9FE 74D7                   <1>     jz      short sysvideo_25 ; error !
                                <1>
0000FA00 80FB01                 <1>     cmp     bl, 1  ; VGA memory mapping ?
0000FA03 740E                   <1>     je      short sysvideo_26_1
0000FA05 7718                   <1>     ja      short sysvideo_26_2
                                <1> sysvideo_26_0:
                                <1>     ; BL = 0 : CGA memory (0B8000h) map (32K)
0000FA07 B800800B00             <1>     mov     eax, 0B8000h
0000FA0C BB00800000             <1>     mov     ebx, 32768
0000FA11 EB37                   <1>     jmp     short sysvideo_26_3
                                <1> sysvideo_26_1:
                                <1>     ; BL = 1 : VGA memory (0A0000h) map (64K)
0000FA13 B800000A00             <1>     mov     eax, 0A0000h
0000FA18 BB00000100             <1>     mov     ebx, 65536
0000FA1D EB2B                   <1>     jmp     short sysvideo_26_3
                                <1> sysvideo_26_2:
                                <1>     ; BL = 2 : SVGA memory (LFB) map to user's buffer
0000FA1F 803D[7C090000]02       <1>     cmp     byte [vbe3], 2  ; VESA VBE 2/3 vbios ready ?
0000FA26 72AF                   <1>     jb      short sysvideo_25  ; no, error !
0000FA28 6681E200F0             <1>     and     dx, ~4095 ; clear low 12 bits
0000FA2D 09D2                   <1>     or      edx, edx  ; buffer size in bytes
0000FA2F 74A6                   <1>     jz      short sysvideo_25  ; error
0000FA31 89D3                   <1>     mov     ebx, edx
0000FA33 A1[349D0100]           <1>     mov     eax, [LFB_ADDR] ; [LFB_Info+LFBINFO.LFB_addr]
0000FA38 21C0                   <1>     and     eax, eax
0000FA3A 7425                   <1>     jz      short sysvideo_26_5
                                <1>                             ; (LFB parms are not set yet)
0000FA3C 3B1D[389D0100]         <1>     cmp     ebx, [LFB_SIZE] ; [LFB_Info+LFBINFO.LFB_size]
0000FA42 7606                   <1>     jna     short sysvideo_26_3
0000FA44 8B1D[389D0100]         <1>     mov     ebx, [LFB_SIZE]
                                <1> sysvideo_26_3:
0000FA4A 52                     <1>     push    edx
0000FA4B 53                     <1>     push    ebx     ; buffer size in bytes
0000FA4C 51                     <1>     push    ecx     ; user's buffer address
0000FA4D 87D9                   <1>     xchg    ebx, ecx
0000FA4F C1E90C                 <1>     shr     ecx, 12 ; convert buffer size to page count
0000FA52 E87865FFFF             <1>     call    direct_memory_access
0000FA57 59                     <1>     pop     ecx     ; user's buffer address
0000FA58 5B                     <1>     pop     ebx     ; buffer size
0000FA59 5A                     <1>     pop     edx
                                <1>     ;jc     short sysvideo_25 ; error !
                                <1>                               ; [u.r0] = 0
                                <1>     ; 28/02/2021
0000FA5A 7235                   <1>     jc      short sysvideo_27_0 ; error !
                                <1>
                                <1> ;sysvideo_26_4:
                                <1>     ;mov    ebp, [u.usp] ; ebp points to user's registers
                                <1>     ;mov    [ebp+20], edx ; return to user with EDX value
                                <1>     ;mov    [ebp+16], ebx ; EBX
                                <1>     ;mov    [ebp+24], ecx ; ECX
                                <1>     ; eax = physical address of video memory (LFB)
                                <1>     ;mov    [u.r0], eax
                                <1>     ;jmp    sysret
0000FA5C E92CFCFFFF             <1>     jmp     sysvideo_26_4
                                <1>
                                <1> sysvideo_26_5:
0000FA61 66A1[100F0000]         <1>     mov     ax, [def_LFB_addr] ; default LFB for mode 118h
                                <1>     ; ah must be 0C0h or 0D0h or E0h
                                <1>     ; others are nonsence !?
0000FA67 08E4                   <1>     or      ah, ah
                                <1>     ;jz     short sysvideo_25 ; invalid lfb addr or
                                <1>                               ; it is not a vbe2 -bochs emu-
                                <1>                               ; or vbe3 -real- video bios
                                <1>     ; 28/02/2021
0000FA69 7426                   <1>     jz      short sysvideo_27_0 ; invalid LFB address
                                <1>
0000FA6B 80FCF0                 <1>     cmp     ah, 0F0h
                                <1>     ;jnb    short sysvideo_25 ; nonsence !?
                                <1>     ; 28/02/2021
0000FA6E 7321                   <1>     jnb     short sysvideo_27_0 ; nonsence !?
                                <1>
0000FA70 C1E010                 <1>     shl     eax, 16
                                <1>     ;jz     short sysvideo_25 ; eax = 0
                                <1>
0000FA73 81FB00907E00           <1>     cmp     ebx, 1920*1080*4 ; maximum value of possible
                                <1>                               ; buffer sizes
0000FA79 76CF                   <1>     jna     short sysvideo_26_3  ; buffer size is proper
                                <1>     ; resize buffer to fit 4GB limit
0000FA7B BB00907E00             <1>     mov     ebx, 1920*1080*4
0000FA80 EBC8                   <1>     jmp     short sysvideo_26_3
                                <1>
```

```
                                <1> sysvideo_27:
                                <1>     ; 23/07/2022
                                <1>     ; 16/02/2021
                                <1>     ; 18/01/2021
0000FA82 80FF0C                 <1>     cmp    bh, 12
                                <1>     ;ja    sysvideo_28 ; 19/01/2021
                                <1>     ; 23/07/2022
0000FA85 7605                   <1>     jna    short sysvideo_27_24
0000FA87 E970010000             <1>     jmp    sysvideo_28
                                <1>
                                <1> sysvideo_27_24:    ; 23/07/2022
                                <1>     ; BH = 12
                                <1>     ; Font sub functions.
                                <1>     ; 12/02/2021
                                <1>     ; 11/01/2021
                                <1>     ; 10/01/2021
                                <1>     ;   BL = 0 : Disable system font overwrite
                                <1>     ;   BL = 1 : Enable system font overwrite
                                <1>     ;   BL = 2 : Read system font 8x8
                                <1>     ;   BL = 3 : Read system font 8x14
                                <1>     ;   BL = 4 : Read system font 8x16
                                <1>     ;   BL = 5 : Read user defined font 8x8
                                <1>     ;   BL = 6 : Read user defined font 8x16
                                <1>     ;   BL = 7 : Write system font 8x8
                                <1>     ;   BL = 8 : Write system font 8x14
                                <1>     ;   BL = 9 : Write system font 8x16
                                <1>     ;   BL = 10 : Write user defined font 8x8
                                <1>     ;   BL = 11 : Write user defined font 8x16
                                <1>     ;
                                <1>     ;  BL > 11 : invalid (not implemented)
                                <1>     ;
                                <1>     ; For BL = 1 to 11
                                <1>     ;   ECX = number of characters (<= 256)
                                <1>     ;   EDX = first character (ascii code in DL)
                                <1>     ;   ESI = user's buffer address
                                <1>     ;
                                <1>     ; Return: EAX = character count
                                <1>
0000FA8C 80FB0B                 <1>     cmp    bl, 11
0000FA8F 7605                   <1>     jna    short sysvideo_27_1
                                <1> sysvideo_27_0:
0000FA91 E9F8CFFFFF             <1>     jmp    sysret ; not implemented yet !
                                <1> sysvideo_27_1:
0000FA96 66B80001               <1>     mov    ax, 256
0000FA9A 08DB                   <1>     or     bl, bl
0000FA9C 750E                   <1>     jnz    short sysvideo_27_3
                                <1>
                                <1>     ; bl = 0
                                <1>     ; disable system font overwrite
                                <1>
0000FA9E 8025[869D0100]7F       <1>     and    byte [ufont], 7Fh ; clear bit 7
                                <1> sysvideo_27_2:
                                <1>     ;mov   word [u.r0], 256  ; > 0 -> successful
0000FAA5 A3[6C8E0100]           <1>     mov    [u.r0], eax ; 256
0000FAAA EBE5                   <1>     jmp    short sysvideo_27_0
                                <1> sysvideo_27_3:
0000FAAC 80FB01                 <1>     cmp    bl, 1
0000FAAF 7710                   <1>     ja     short sysvideo_27_4
                                <1>
                                <1>     ; bl = 1
                                <1>     ; enable system font overwrite
                                <1>     ;      if [multi_tasking]= 0 and [u.uid] = 0
                                <1>
                                <1>     ;cmp   byte [multi_tasking], 0
                                <1>     ;                     ; multi tasking enabled ?
                                <1>     ;ja    short sysvideo_27_0 ; yes
                                <1>     ;; 19/01/2021
                                <1>     ;; system maintenance or single user mode
                                <1>     ;cmp   byte [u.uid], 0  ; root ?
                                <1>     ;ja    short sysvideo_27_0 ; no
                                <1>
                                <1>     ; 19/01/2021
                                <1>     ; multi tasking & root check
0000FAB1 E8EBFEFFFF             <1>     call   sysvideo_22_4
0000FAB6 73D9                   <1>     jnc    short sysvideo_27_0 ; not permitted
                                <1>
                                <1>     ; [multi_tasking]= 0 and [u.uid] = 0
                                <1>
0000FAB8 800D[869D0100]80       <1>     or     byte [ufont], 80h ; set bit 7
                                <1>
0000FABF EBE4                   <1>     jmp    short sysvideo_27_2
                                <1>
                                <1> sysvideo_27_4:
0000FAC1 09C9                   <1>     or     ecx, ecx
0000FAC3 74CC                   <1>     jz     short sysvideo_27_0
0000FAC5 21D2                   <1>     and    edx, edx
0000FAC7 7410                   <1>     jz     short sysvideo_27_4_0
                                <1>     ;mov   ax, 256
0000FAC9 39C1                   <1>     cmp    ecx, eax ; 256
0000FACB 77C4                   <1>     ja     short sysvideo_27_0
0000FACD 48                     <1>     dec    eax
0000FACE 39C2                   <1>     cmp    edx, eax ; 255
0000FAD0 77BF                   <1>     ja     short sysvideo_27_0
0000FAD2 40                     <1>     inc    eax
0000FAD3 29D0                   <1>     sub    eax, edx ; 256 - DX
0000FAD5 39C8                   <1>     cmp    eax, ecx
0000FAD7 72B8                   <1>     jb     short sysvideo_27_0
                                <1>
                                <1> sysvideo_27_4_0:
0000FAD9 89F5                   <1>     mov    ebp, esi
                                <1>
0000FADB 80FB06                 <1>     cmp    bl, 6
0000FADE 7768                   <1>     ja     short sysvideo_27_13
0000FAE0 7210                   <1>     jb     short sysvideo_27_5
                                <1>     ; bl = 6
0000FAE2 F605[869D0100]10       <1>     test   byte [ufont], 16 ; 8x16 user font loaded ?
0000FAE9 74A6                   <1>     jz     short sysvideo_27_0
                                <1>     ; read 8x16 user defined font
0000FAEB BE00400900             <1>     mov    esi, VGAFONT16USER
0000FAF0 EB0C                   <1>     jmp    short sysvideo_27_6
                                <1> sysvideo_27_5:
0000FAF2 80FB04                 <1>     cmp    bl, 4
0000FAF5 7239                   <1>     jb     short sysvideo_27_11
0000FAF7 7721                   <1>     ja     short sysvideo_27_9
                                <1>     ; bl = 4
                                <1>     ; read 8x16 system font
0000FAF9 BE[40630100]           <1>     mov    esi, vgafont16
                                <1> sysvideo_27_6:
                                <1>     ; read 8x16 font
                                <1>     ;shl   dx, 4 ; * 16
                                <1>     ;shl   cx, 4 ; * 16 ; 16 bytes per char
                                <1>     ; 23/07/2022
0000FAFE C1E204                 <1>     shl    edx, 4 ; * 16
0000FB01 C1E104                 <1>     shl    ecx, 4 ; * 16
```

```
                                <1> sysvideo_27_7:
0000FB04 89EF                   <1>     mov     edi, ebp
                                <1>     ;add    edi, edx ; 16/02/2021
0000FB06 01D6                   <1>     add     esi, edx
                                <1>     ; ecx = byte count
                                <1>     ; esi = source (in system memory)
                                <1>     ; edi = destination (in user memory)
0000FB08 E80E100000             <1>     call    transfer_to_user_buffer
0000FB0D 7206                   <1>     jc      short sysvideo_27_8
0000FB0F 890D[6C8E0100]         <1>     mov     [u.r0], ecx
                                <1> sysvideo_27_8:
0000FB15 E974CFFFFF             <1>     jmp     sysret
                                <1> sysvideo_27_9:
                                <1>     ; bl = 5
0000FB1A F605[869D0100]08       <1>     test    byte [ufont], 8 ; 8x8 user font loaded ?
0000FB21 74F2                   <1>     jz      short sysvideo_27_8
                                <1>     ; read 8x8 user defined font
0000FB23 BE00500900             <1>     mov     esi, VGAFONT8USER
                                <1> sysvideo_27_10:
                                <1>     ; read 8x8 font
                                <1>     ;shl    dx, 3 ; * 8
                                <1>     ;shl    cx, 3 ; * 8 ; 8 bytes per char
                                <1>     ; 23/07/2022
0000FB28 C1E203                 <1>     shl     edx, 3 ; * 8
0000FB2B C1E103                 <1>     shl     ecx, 3 ; * 8
0000FB2E EBD4                   <1>     jmp     short sysvideo_27_7
                                <1>
                                <1> sysvideo_27_11:
0000FB30 80FB03                 <1>     cmp     bl, 3  ; 8x14 system font
0000FB33 720C                   <1>     jb      short sysvideo_27_12 ; 8x8 system font
                                <1>     ; bl = 3
                                <1>     ; read 8x14 system font
                                <1>     ;mov    al, 14
                                <1>     ;mul    dl
                                <1>     ;mov    dx, ax
                                <1>     ;push   edx
                                <1>     ;mov    ax, 14
                                <1>     ;mul    cx
                                <1>     ;mov    cx, ax
                                <1>     ;pop    edx
0000FB35 E8A5000000             <1>     call    sysvideo_27_14
0000FB3A BE[40550100]           <1>     mov     esi, vgafont14
0000FB3F EBC3                   <1>     jmp     short sysvideo_27_7
                                <1>
                                <1> sysvideo_27_12:
                                <1>     ; bl = 2
                                <1>     ; read 8x8 system font
0000FB41 BE[404D0100]           <1>     mov     esi, vgafont8
0000FB46 EBE0                   <1>     jmp     short sysvideo_27_10
                                <1>
                                <1> sysvideo_27_13:
                                <1>     ; overwrite font
0000FB48 80FB0A                 <1>     cmp     bl, 10
0000FB4B 776E                   <1>     ja      short sysvideo_27_22 ; 8x16 user font
0000FB4D 7224                   <1>     jb      short sysvideo_27_15
                                <1>     ; bl = 10
0000FB4F BF00500900             <1>     mov     edi, VGAFONT8USER
0000FB54 F605[869D0100]08       <1>     test    byte [ufont], 8 ; 8x8 user font loaded ?
0000FB5B 7556                   <1>     jnz     short sysvideo_27_21 ; yes
0000FB5D 08ED                   <1>     or      ch, ch ; cx = 256
                                <1>     ;jnz    short sysvideo_27_21 ; 256 chars
0000FB5F 7406                   <1>     jz      short sysvideo_27_13_0
0000FB61 66B90008               <1>     mov     cx, 8*256
0000FB65 EB35                   <1>     jmp     short sysvideo_27_18_0
                                <1> sysvideo_27_13_0:
                                <1>     ; copy system font to user font before overwrite
0000FB67 BE[404D0100]           <1>     mov     esi, vgafont8
                                <1>     ;push   edi
                                <1>     ;push   ecx
                                <1>     ;mov    cl, 64
                                <1>     ;rep    movsd
                                <1>     ;pop    ecx
                                <1>     ;pop    edi
                                <1>     ;mov    esi, ebp ; user's font buffer
0000FB6C E880000000             <1>     call    sysvideo_27_23
0000FB71 EB40                   <1>     jmp     short sysvideo_27_21
                                <1>
                                <1> sysvideo_27_15:
                                <1>     ; check system font overwrite permission
0000FB73 F605[869D0100]80       <1>     test    byte [ufont], 80h
0000FB7A 7499                   <1>     jz      short sysvideo_27_8
                                <1>
0000FB7C 80FB08                 <1>     cmp     bl, 8
0000FB7F 773A                   <1>     ja      short sysvideo_27_22 ; 8x16 system font
0000FB81 722B                   <1>     jb      short sysvideo_27_20 ; 8x8 system font
                                <1>     ; bl = 8
                                <1>     ; overwrite 8x14 system font
                                <1>     ;mov    al, 14
                                <1>     ;mul    dl
                                <1>     ;mov    dx, ax
                                <1>     ;push   edx
                                <1>     ;mov    ax, 14
                                <1>     ;mul    cx
                                <1>     ;mov    cx, ax
                                <1>     ;pop    edx
0000FB83 E857000000             <1>     call    sysvideo_27_14
0000FB88 BF[40550100]           <1>     mov     edi, vgafont14
0000FB8D EB0B                   <1>     jmp     short sysvideo_27_18
                                <1> sysvideo_27_16:
                                <1>     ; bl = 9
                                <1>     ; overwrite 8x16 system font
0000FB8F BF[40630100]           <1>     mov     edi, vgafont16
                                <1> sysvideo_27_17:
                                <1>     ; overwrite 8x16 font
                                <1>     ;shl    dx, 4 ; * 16
                                <1>     ;shl    cx, 4 ; * 16 ; 16 bytes per char
                                <1>     ; 23/07/2022
0000FB94 C1E204                 <1>     shl     edx, 4 ; * 16
0000FB97 C1E104                 <1>     shl     ecx, 4 ; * 16
                                <1> sysvideo_27_18:
0000FB9A 01D7                   <1>     add     edi, edx
                                <1>     ;add    esi, edx ; 16/02/2021
                                <1> sysvideo_27_18_0:
                                <1>     ; ecx = byte count
                                <1>     ; esi = source (in user memory)
                                <1>     ; edi = destination (in system memory)
0000FB9C E8C40F0000             <1>     call    transfer_from_user_buffer
0000FBA1 7206                   <1>     jc      short sysvideo_27_19
0000FBA3 890D[6C8E0100]         <1>     mov     [u.r0], ecx
                                <1> sysvideo_27_19:
0000FBA9 E9E0CEFFFF             <1>     jmp     sysret
                                <1> sysvideo_27_20:
                                <1>     ; bl = 7
                                <1>     ; overwrite 8x8 system font
```

```
0000FBAE BF[404D0100]        <1>     mov     edi, vgafont8
                             <1> sysvideo_27_21:
                             <1>     ; overwrite 8x8 font
                             <1>     ;shl    dx, 3 ; * 8
                             <1>     ;shl    cx, 3 ; * 8 ; 8 bytes per char
                             <1>     ; 23/07/2022
0000FBB3 C1E203              <1>     shl     edx, 3 ; * 8
0000FBB6 C1E103              <1>     shl     ecx, 3 ; * 8
0000FBB9 EBDF                <1>     jmp     short sysvideo_27_18
                             <1> sysvideo_27_22:
                             <1>     ; bl = 11
                             <1>     ; overwrite 8x16 user defined font
0000FBBB BF00400900          <1>     mov     edi, VGAFONT16USER
0000FBC0 F605[869D0100]10    <1>     test    byte [ufont], 16 ; 8x16 user font loaded ?
0000FBC7 75CB                <1>     jnz     short sysvideo_27_17 ; yes
0000FBC9 08ED                <1>     or      ch, ch ; cx = 256
                             <1>     ;jnz    short sysvideo_27_17 ; 256 chars
0000FBCB 7406                <1>     jz      short sysvideo_27_22_0
0000FBCD 66B90010            <1>     mov     cx, 16*256
0000FBD1 EBC9                <1>     jmp     short sysvideo_27_18_0
                             <1> sysvideo_27_22_0:
                             <1>     ; copy system font to user font before overwrite
0000FBD3 BE[40630100]        <1>     mov     esi, vgafont16
                             <1>     ;push   edi
                             <1>     ;push   ecx
                             <1>     ;mov    cl, 64
                             <1>     ;rep    movsd
                             <1>     ;pop    ecx
                             <1>     ;pop    edi
                             <1>     ;mov    esi, ebp ; user's font buffer
0000FBD8 E814000000          <1>     call    sysvideo_27_23
0000FBDD EBB5                <1>     jmp     short sysvideo_27_17
                             <1>
                             <1> sysvideo_27_14:
                             <1>     ; 16/02/2021
0000FBDF 52                  <1>     push    edx
0000FBE0 66B80E00            <1>     mov     ax, 14
0000FBE4 66F7E1              <1>     mul     cx
0000FBE7 89C1                <1>     mov     ecx, eax
0000FBE9 5A                  <1>     pop     edx
0000FBEA B00E                <1>     mov     al, 14
0000FBEC F6E2                <1>     mul     dl
0000FBEE 89C2                <1>     mov     edx, eax
0000FBF0 C3                  <1>     retn
                             <1>
                             <1>     ;mov    al, 14
                             <1>     ;mul    dl
                             <1>     ;mov    dx, ax
                             <1>     ;push   edx
                             <1>     ;; 12/02/2021
                             <1>     ;mov    ax, 14
                             <1>     ;;mov   eax, 14
                             <1>     ;;mul   cx
                             <1>     ;mul    ecx
                             <1>     ;;mov   cx, ax
                             <1>     ;mov    ecx, eax
                             <1>     ;pop    edx
                             <1>     ;retn
                             <1>
                             <1> sysvideo_27_23:
0000FBF1 57                  <1>     push    edi
0000FBF2 51                  <1>     push    ecx
0000FBF3 B140                <1>     mov     cl, 64
0000FBF5 F3A5                <1>     rep     movsd
0000FBF7 59                  <1>     pop     ecx
0000FBF8 5F                  <1>     pop     edi
0000FBF9 89EE                <1>     mov     esi, ebp ; user's font buffer
0000FBFB C3                  <1>     retn
                             <1>
                             <1> sysvideo_28:
                             <1>     ; 23/07/2022
                             <1>     ; 24/01/2021
                             <1>     ; 23/01/2021
                             <1>     ; 18/01/2021
0000FBFC 80FF0E              <1>     cmp     bh, 14
                             <1>     ;jb     sysvideo_29
                             <1>     ; 23/07/2022
0000FBFF 7222                <1>     jb      short sysvideo_28_29
                             <1>     ;ja     sysvideo_30
                             <1>     ; 23/07/2022
0000FC01 7725                <1>     ja      short sysvideo_28_30
                             <1>
                             <1>     ; BH = 14
                             <1>     ; Save/Restore Super VGA video state
                             <1>
                             <1>     ; BL = options
                             <1>     ;       bit 0 - Save (0) or Restore (1)
                             <1>     ;       bit 1 - controller hardware state
                             <1>     ;       bit 2 - BIOS data state
                             <1>     ;       bit 3 - DAC state
                             <1>     ;       bit 4 - (extended) Register state
                             <1>     ;       bit 5 - system (0) or user (1) memory
                             <1>     ;       bit 6 - verify without transfer
                             <1>     ;       bit 7 - not used (must be 0)
                             <1>
                             <1>     ; ECX = Buffer address or VideoStateID
                             <1>
0000FC03 803D[7C090000]02    <1>     cmp     byte [vbe3], 2 ;  VESA VBE2 or VBE3 ?
0000FC0A 7721                <1>     ja      short sysvideo_28_0 ; yes
0000FC0C 7210                <1>     jb      short sysvideo_28_16 ; not a SVGA sys !
                             <1>
                             <1>     ; == VBE2 ==
                             <1>     ; Check Bochs/Qemu/VirtualBox PC emulator
                             <1>     ; (vbe2 is usable only for emulator's vbios)
0000FC0E 8A25[7D090000]      <1>     mov     ah, [vbe2bios]
0000FC14 80FCC0              <1>     cmp     ah, 0C0h
0000FC17 7205                <1>     jb      short sysvideo_28_16 ; unknown vbios !
0000FC19 80FCC5              <1>     cmp     ah, 0C5h
0000FC1C 760F                <1>     jna     short sysvideo_28_0
                             <1>             ; Use kernel's vbios functions (video.s)
                             <1> sysvideo_28_16:
                             <1>     ; unknown vbios !
0000FC1E E96BCEFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_28_29:
                             <1>     ; 23/07/2022
0000FC23 E955010000          <1>     jmp     sysvideo_29
                             <1> sysvideo_28_30:
                             <1>     ; 23/07/2022
0000FC28 E935020000          <1>     jmp     sysvideo_30
                             <1>
                             <1> sysvideo_28_0:
0000FC2D 80FB7F              <1>     cmp     bl, 7Fh
0000FC30 77EC                <1>     ja      short sysvideo_28_16 ; unknown options
```

```
                                <1>
0000FC32 88DA                   <1>     mov     dl, bl
0000FC34 80E21F                 <1>     and     dl, 1Fh
0000FC37 D0EA                   <1>     shr     dl, 1
0000FC39 74E3                   <1>     jz      short sysvideo_28_16 ; invalid !
                                <1>     ; DL = VBE Function 4F04h Save/Restore options
                                <1>     ;  bit 0 : controller hardware state
                                <1>     ;  bit 1 : BIOS data state
                                <1>     ;  bit 2 : DAC state
                                <1>     ;  bit 3 : (extended) Register state
                                <1>
0000FC3B F6C320                 <1>     test    bl, 32 ; bit 5
                                <1>     ;jnz    sysvideo_28_7 ; user buffer
                                <1>     ; 23/07/2022
0000FC3E 7405                   <1>     jz      short sysvideo_28_17
0000FC40 E9B1000000             <1>     jmp     sysvideo_28_7
                                <1>
                                <1> sysvideo_28_17:     ; 23/07/2022
                                <1>     ; source or destination is kernel/system buffer
                                <1>
0000FC45 803D[889D0100]00       <1>     cmp     byte [srvsf], 0 ; srs permission flag
0000FC4C 76D0                   <1>     jna     short sysvideo_28_16 ; not permitted
                                <1>
0000FC4E F6C301                 <1>     test    bl, 1
0000FC51 743A                   <1>     jz      short sysvideo_28_4 ; Save
                                <1>
                                <1>     ; Restore
0000FC53 3B0D[8A9D0100]         <1>     cmp     ecx, [VideoStateID]
0000FC59 75C3                   <1>     jne     short sysvideo_28_16 ; not correct ID !
                                <1>
0000FC5B 0FB6CA                 <1>     movzx   ecx, dl
0000FC5E 80CA80                 <1>     or      dl, 80h
0000FC61 3A15[899D0100]         <1>     cmp     dl, [srvso]
0000FC67 75B5                   <1>     jne     short sysvideo_28_16 ; not correct !
                                <1>
0000FC69 88DA                   <1>     mov     dl, bl
                                <1>
                                <1>     ; ecx = cl = options
0000FC6B E80B3FFFFF             <1>     call    vbe_srs_gbs
                                <1>     ; ebx = state buffer size (data size)
                                <1>
0000FC70 891D[6C8E0100]         <1>     mov     [u.r0], ebx
                                <1>
0000FC76 F6C240                 <1>     test    dl, 64 ; verify without transfer
0000FC79 75A3                   <1>     jnz     short sysvideo_28_16 ; yes
                                <1>
0000FC7B BE00580900             <1>     mov     esi, VBE3VIDEOSTATE
0000FC80 BF00760900             <1>     mov     edi, VBE3SAVERESTOREBLOCK
0000FC85 87CB                   <1>     xchg    ecx, ebx
0000FC87 F3A4                   <1>     rep     movsb
                                <1>
0000FC89 88D9                   <1>     mov     cl, bl
                                <1>
                                <1>     ; 23/01/2021
0000FC8B EB44                   <1>     jmp     short sysvideo_28_10
                                <1>
                                <1> sysvideo_28_4:
0000FC8D 53                     <1>     push    ebx
                                <1>     ; 24/01/2021
0000FC8E 31DB                   <1>     xor     ebx, ebx ; 0 ; use kernel's buffer
0000FC90 881D[899D0100]         <1>     mov     [srvso], bl ; 0 ; invalidate
0000FC96 891D[8A9D0100]         <1>     mov     [VideoStateID], ebx ; 0 ; invalidate
0000FC9C 0FB6CA                 <1>     movzx   ecx, dl ; options
0000FC9F B201                   <1>     mov     dl, 1 ; save state
0000FCA1 E890000000             <1>     call    sysvideo_28_11 ; 23/01/2021
                                <1>     ; Note: VBE3 BIOS data save option will be
                                <1>     ;       disabled.. ; 24/01/2021
0000FCA6 89CA                   <1>     mov     edx, ecx ; state (save) options
0000FCA8 5B                     <1>     pop     ebx
                                <1>
0000FCA9 6683F84F               <1>     cmp     ax, 4Fh ; successful ?
0000FCAD 7536                   <1>     jne     short sysvideo_28_3 ; no !
                                <1>
0000FCAF F6C340                 <1>     test    bl, 64 ; verify without transfer
0000FCB2 7536                   <1>     jnz     short sysvideo_28_6 ; yes
                                <1>
                                <1>     ; ecx = cl = options
0000FCB4 E8C23EFFFF             <1>     call    vbe_srs_gbs
                                <1>     ; ebx = state buffer size (data size)
                                <1>
0000FCB9 BE00760900             <1>     mov     esi, VBE3SAVERESTOREBLOCK
0000FCBE BF00580900             <1>     mov     edi, VBE3VIDEOSTATE
0000FCC3 89D9                   <1>     mov     ecx, ebx
0000FCC5 F3A4                   <1>     rep     movsb
                                <1>
0000FCC7 88D1                   <1>     mov     cl, dl
0000FCC9 80C980                 <1>     or      cl, 80h ; SVGA (VESA VBE) flag
                                <1>     ;mov    [srvso], dl
                                <1>
0000FCCC E908010000             <1>     jmp     sysvideo_28_15
                                <1>
                                <1>     ; 23/01/2021
                                <1> sysvideo_28_10:
                                <1>     ; CL = VESA VBE3 Save/Restore options
                                <1>
0000FCD1 B202                   <1>     mov     dl, 2 ; restore state
                                <1>
0000FCD3 E85C000000             <1>     call    sysvideo_28_1
                                <1>
0000FCD8 6683F84F               <1>     cmp     ax, 4Fh ; successful ?
0000FCDC 7407                   <1>     je      short sysvideo_28_3
                                <1>     ;jmp    short sysvideo_28_9
                                <1>
                                <1> sysvideo_28_9:
                                <1>     ; return zero size (error) to user
0000FCDE 29C0                   <1>     sub     eax, eax
                                <1> sysvideo_28_5:
0000FCE0 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1> sysvideo_28_3:
0000FCE5 E9A4CDFFFF             <1>     jmp     sysret
                                <1>
                                <1> sysvideo_28_6:
                                <1>     ; use timer ticks as VideoStateID
0000FCEA A1[30770100]           <1>     mov     eax, [TIMER_LH]
0000FCEF 09C0                   <1>     or      eax, eax
0000FCF1 75ED                   <1>     jnz     short sysvideo_28_5
0000FCF3 40                     <1>     inc     eax
0000FCF4 EBEA                   <1>     jmp     short sysvideo_28_5
                                <1>
                                <1> sysvideo_28_7:
                                <1>     ; save/restore to/from user buffer
                                <1>
                                <1>     ; 23/01/2021
0000FCF6 89CE                   <1>     mov     esi, ecx ; user's vstate buffer
```

```
0000FCF8 BF00760900          <1>     mov     edi, VBE3SAVERESTOREBLOCK
                             <1>
0000FCFD 0FB6CA              <1>     movzx   ecx, dl ; VESA VBE func 4F04h options
                             <1>
                             <1>     ; source or destination is user buffer
0000FD00 F6C301              <1>     test    bl, 1
0000FD03 7444                <1>     jz      short sysvideo_28_12  ; Save
                             <1>
                             <1>     ; Restore
0000FD05 803D[889D0100]00    <1>     cmp     byte [srvsf], 0 ; srs permission flag
0000FD0C 766A                <1>     jna     short sysvideo_28_14 ; not permitted
                             <1>
0000FD0E 88DA                <1>     mov     dl, bl ; 'sysvideo' options
                             <1>
                             <1>     ; ecx = cl = options
0000FD10 E8663EFFFF          <1>     call    vbe_srs_gbs
                             <1>     ; ebx = state buffer size (data size)
                             <1>
0000FD15 891D[6C8E0100]      <1>     mov     [u.r0], ebx ; transfer count
                             <1>
0000FD1B F6C240              <1>     test    dl, 64 ; verify without transfer
0000FD1E 7558                <1>     jnz     short sysvideo_28_14 ; yes
                             <1>
0000FD20 6681FB0008          <1>     cmp     bx, 2048
0000FD25 73B7                <1>     jnb     short sysvideo_28_9 ; invalid
                             <1>
0000FD27 87CB                <1>     xchg    ecx, ebx
                             <1>     ; esi = user buffer
                             <1>     ; edi = VBE3SAVERESTOREBLOCK
                             <1>
0000FD29 E8370E0000          <1>     call    transfer_from_user_buffer
0000FD2E 72AE                <1>     jc      short sysvideo_28_9 ; error
                             <1>
0000FD30 89D9                <1>     mov     ecx, ebx ; Function 4F04h options
0000FD32 EB9D                <1>     jmp     short sysvideo_28_10 ; 23/01/2021
                             <1>
                             <1> sysvideo_28_1:
0000FD34 31DB                <1>     xor     ebx, ebx ; 0 ; use kernel's buffer
                             <1> sysvideo_28_11:
                             <1>     ; 24/01/2021
0000FD36 803D[7C090000]03    <1>     cmp     byte [vbe3], 3
0000FD3D 7405                <1>     je      short sysvideo_28_2
                             <1>
                             <1>     ; VESA VBE2 (BOCHS/QEMU/VBOX) video bios
0000FD3F E9A03DFFFF          <1>     jmp     _vbe_biosfn_save_restore_state
                             <1> sysvideo_28_2:
                             <1>     ;24/01/2021
                             <1>     ;mov    eax, 4F04h  ; Save/Restore vstate
                             <1>     ; VESA VBE3 video bios
0000FD44 E9051DFFFF          <1>     jmp     _vbe3_pmfn_save_restore_state
                             <1>
                             <1> sysvideo_28_12:
                             <1>     ; Save
                             <1>     ;mov    edi, VBE3SAVERESTOREBLOCK
                             <1>
                             <1>     ;movzx  ecx, dl ; options
0000FD49 56                  <1>     push    esi
0000FD4A 53                  <1>     push    ebx
                             <1>     ; 23/01/2021
0000FD4B B201                <1>     mov     dl, 1 ; save state
0000FD4D E8E2FFFFFF          <1>     call    sysvideo_28_1
0000FD52 5A                  <1>     pop     edx ; 'sysvideo' options
0000FD53 5F                  <1>     pop     edi ; user's video state buffer
                             <1>
0000FD54 6683F84F            <1>     cmp     ax, 4Fh ; successful ?
0000FD58 751E                <1>     jne     short sysvideo_28_14 ; no !
                             <1>
                             <1>     ; ecx = cl = options
0000FD5A E81C3EFFFF          <1>     call    vbe_srs_gbs
                             <1>     ; ebx = state buffer size (data size)
                             <1>
0000FD5F 89D9                <1>     mov     ecx, ebx ; transfer count
                             <1>
0000FD61 F6C240              <1>     test    dl, 64 ; verify without transfer
0000FD64 750C                <1>     jnz     short sysvideo_28_13 ; yes
                             <1>
                             <1>     ;mov    edi, esi
0000FD66 BE00760900          <1>     mov     esi, VBE3SAVERESTOREBLOCK
0000FD6B E8AB0D0000          <1>     call    transfer_to_user_buffer
0000FD70 7206                <1>     jc      short sysvideo_28_14
                             <1> sysvideo_28_13:
0000FD72 890D[6C8E0100]      <1>     mov     [u.r0], ecx
                             <1> sysvideo_28_14:
0000FD78 E911CDFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysvideo_29:
                             <1>     ; 18/01/2021
                             <1>     ; BH = 13
                             <1>     ; Save/Restore std VGA video state
                             <1>
                             <1>     ; bl = 0..3
                             <1>     ;       save to or restore from
                             <1>     ;       system buffer, VBE3VIDEOSTATE
                             <1>     ;       ECX = VideoStateID for restoring
                             <1>     ; bl = 4..7
                             <1>     ;       save to or restore from
                             <1>     ;       user buffer pointed by ECX
                             <1>
0000FD7D 80FB03              <1>     cmp     bl, 3
0000FD80 7776                <1>     ja      short sysvideo_29_6
                             <1>
                             <1>     ; source or destination is kernel/system buffer
                             <1>
0000FD82 803D[889D0100]00    <1>     cmp     byte [srvsf], 0 ; srs permission flag
0000FD89 7668                <1>     jna     short sysvideo_29_5 ; not permitted
                             <1>
0000FD8B F6C301              <1>     test    bl, 1
0000FD8E 7437                <1>     jz      short sysvideo_29_2  ; Save
                             <1>
                             <1>     ; Restore
0000FD90 3B0D[8A9D0100]      <1>     cmp     ecx, [VideoStateID]
0000FD96 755B                <1>     jne     short sysvideo_29_5 ; not correct ID !
0000FD98 80FB01              <1>     cmp     bl, 1
0000FD9B 7709                <1>     ja      short sysvideo_29_0
                             <1>     ; bl = 1
0000FD9D BB6E000000          <1>     mov     ebx, 110
0000FDA2 B103                <1>     mov     cl, 3 ; ctrl, vbios data
0000FDA4 EB07                <1>     jmp     short sysvideo_29_1
                             <1> sysvideo_29_0:
                             <1>     ; bl  = 3
0000FDA6 BB72030000          <1>     mov     ebx, 882
0000FDAB B107                <1>     mov     cl, 7 ; ctrl, vbios data, dac
                             <1> sysvideo_29_1:
0000FDAD 3A0D[899D0100]      <1>     cmp     cl, [srvso]
```

```
0000FDB3 753E                <1>     jne    short sysvideo_29_5 ; not correct !
                             <1>
0000FDB5 BE00580900          <1>     mov    esi, VBE3VIDEOSTATE ; 22/01/2021
0000FDBA E8503FFFFF          <1>     call   biosfn_restore_video_state
0000FDBF 891D[6C8E0100]      <1>     mov    [u.r0], ebx ; video state size (bytes)
                             <1>     ;jmp   sysret
0000FDC5 EB2C                <1>     jmp    short sysvideo_29_5
                             <1> sysvideo_29_2:
                             <1>     ;mov   esi, ecx
0000FDC7 BF00580900          <1>     mov    edi, VBE3VIDEOSTATE
                             <1>
0000FDCC B107                <1>     mov    cl, 7 ; ctrl, vbios data, dac
0000FDCE 08DB                <1>     or     bl, bl
0000FDD0 7502                <1>     jnz    short sysvideo_29_3 ; bl = 2
                             <1>     ; bl = 0
0000FDD2 B103                <1>     mov    cl, 3 ; ctrl, vbios data
                             <1> sysvideo_29_3:
0000FDD4 E8CE3DFFFF          <1>     call   biosfn_save_video_state
                             <1> sysvideo_28_15:
                             <1>     ; use timer ticks as VideoStateID
0000FDD9 A1[30770100]        <1>     mov    eax, [TIMER_LH]
0000FDDE 21C0                <1>     and    eax, eax
0000FDE0 7501                <1>     jnz    short sysvideo_29_4
0000FDE2 40                  <1>     inc    eax
                             <1> sysvideo_29_4:
0000FDE3 880D[899D0100]      <1>     mov    [srvso], cl
0000FDE9 A3[8A9D0100]        <1>     mov    [VideoStateID], eax
0000FDEE A3[6C8E0100]        <1>     mov    [u.r0], eax
                             <1> sysvideo_29_5:
0000FDF3 E996CCFFFF          <1>     jmp    sysret
                             <1>
                             <1> sysvideo_29_6:
0000FDF8 80FB07              <1>     cmp    bl, 7
0000FDFB 77F6                <1>     ja     short sysvideo_29_5 ; invalid sub function
                             <1>
0000FDFD 89CE                <1>     mov    esi, ecx
0000FDFF BF00760900          <1>     mov    edi, VBE3SAVERESTOREBLOCK
                             <1>
                             <1>     ; source or destination is user buffer
0000FE04 F6C301              <1>     test   bl, 1
0000FE07 7434                <1>     jz     short sysvideo_29_9  ; Save
                             <1>
                             <1>     ; Restore
0000FE09 803D[889D0100]00    <1>     cmp    byte [srvsf], 0 ; srs permission flag
0000FE10 76E1                <1>     jna    short sysvideo_29_5 ; not permitted
                             <1>
                             <1>     ;mov   esi, ecx
                             <1>     ;mov   edi, VBE3SAVERESTOREBLOCK
                             <1>
0000FE12 80FB07              <1>     cmp    bl, 7
0000FE15 7409                <1>     je     short sysvideo_29_7
                             <1>     ; bl = 5
0000FE17 B303                <1>     mov    bl, 3
0000FE19 B96E000000          <1>     mov    ecx, 110
0000FE1E EB05                <1>     jmp    short sysvideo_29_8
                             <1> sysvideo_29_7:
                             <1>     ; bl = 7
0000FE20 B972030000          <1>     mov    ecx, 882
                             <1> sysvideo_29_8:
0000FE25 E83B0D0000          <1>     call   transfer_from_user_buffer
0000FE2A 72C7                <1>     jc     short sysvideo_29_5
0000FE2C 890D[6C8E0100]      <1>     mov    [u.r0], ecx
0000FE32 88D9                <1>     mov    cl, bl ; mov cl,7 (mov cl,3)
0000FE34 89FE                <1>     mov    esi, edi ; VBE3SAVERESTOREBLOCK
                             <1>     ; cl = 3 or 7
0000FE36 E8D43EFFFF          <1>     call   biosfn_restore_video_state
0000FE3B EBB6                <1>     jmp    sysvideo_29_5
                             <1>     ;jmp   sysret
                             <1> sysvideo_29_9:
                             <1>     ; Save
                             <1>     ;mov   edi, VBE3SAVERESTOREBLOCK
                             <1>
0000FE3D 80FB06              <1>     cmp    bl, 6
0000FE40 7409                <1>     je     short sysvideo_29_10
                             <1>     ; bl = 4
0000FE42 BB6E000000          <1>     mov    ebx, 110
0000FE47 B103                <1>     mov    cl, 3 ; ctrl, vbios data
0000FE49 EB07                <1>     jmp    short sysvideo_29_11
                             <1> sysvideo_29_10:
                             <1>     ; bl = 6
0000FE4B BB72030000          <1>     mov    ebx, 882
0000FE50 B107                <1>     mov    cl, 7 ; ctrl, vbios data, dac
                             <1> sysvideo_29_11:
0000FE52 E8503DFFFF          <1>     call   biosfn_save_video_state
                             <1>
0000FE57 89D9                <1>     mov    ecx, ebx ; transfer count
0000FE59 89F7                <1>     mov    edi, esi
0000FE5B BE00760900          <1>     mov    esi, VBE3SAVERESTOREBLOCK
                             <1>
                             <1>     ;call  transfer_to_user_buffer
                             <1>     ;jc    short sysvideo_29_5
                             <1>     ;mov   [u.r0], ecx ; transfer count
                             <1>     ;;jmp  sysret
                             <1>     ;jmp   short sysvideo_29_5
                             <1>
0000FE60 EB1A                <1>     jmp    short sysvideo_29_12
                             <1>
                             <1> sysvideo_30:
0000FE62 80FF0F              <1>     cmp    bh, 15
0000FE65 7722                <1>     ja     short sysvideo_31 ; invalid function
                             <1>
                             <1>     ; BH = 15
                             <1>     ; Copy VESA EDID to user's buffer
                             <1>
0000FE67 803D[1D420000]4F    <1>     cmp    byte [edid], 4Fh
0000FE6E 7519                <1>     jne    short sysvideo_31 ; not ready !
                             <1>
                             <1>     ;and   ecx, ecx
                             <1>     ;jz    short sysvideo_31
                             <1>
                             <1>     ; ecx = user's buffer address
0000FE70 89CF                <1>     mov    edi, ecx
0000FE72 BE[A49C0100]        <1>     mov    esi, edid_info
0000FE77 B980000000          <1>     mov    ecx, 128 ; 128 bytes
                             <1> sysvideo_29_12:
0000FE7C E89A0C0000          <1>     call   transfer_to_user_buffer
0000FE81 7206                <1>     jc     short sysvideo_31
                             <1>
0000FE83 890D[6C8E0100]      <1>     mov    [u.r0], ecx ; EDID size, 128 bytes
                             <1> sysvideo_31:
0000FE89 E900CCFFFF          <1>     jmp    sysret
                             <1>
                             <1> sysexec:
                             <1>     ; 07/07/2025 - TRDOS 386 v2.0.10
```

```
                                <1>     ; 21/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 06/02/2022 - Retro UNIX 386 v1.2
                                <1>     ; 18/11/2017
                                <1>     ; 14/11/2017
                                <1>     ; 13/11/2017
                                <1>     ; 24/10/2016 - 04/01/2017
                                <1>     ; 24/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 23/06/2015 - 23/10/2015 (Retro UNIX 386 v1)
                                <1>     ; 03/06/2013 - 06/12/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysexec' initiates execution of a file whose path name if
                                <1>     ; pointed to by 'name' in the sysexec call.
                                <1>     ; 'sysexec' performs the following operations:
                                <1>     ;    1. obtains i-number of file to be executed via 'namei'.
                                <1>     ;    2. obtains i-node of file to be exceuted via 'iget'.
                                <1>     ;    3. sets trap vectors to system routines.
                                <1>     ;    4. loads arguments to be passed to executing file into
                                <1>     ;       highest locations of user's core
                                <1>     ;    5. puts pointers to arguments in locations immediately
                                <1>     ;       following arguments.
                                <1>     ;    6. saves number of arguments in next location.
                                <1>     ;    7. initializes user's stack area so that all registers
                                <1>     ;       will be zeroed and the PS is cleared and the PC set
                                <1>     ;       to core when 'sysret' restores registers
                                <1>     ;       and does an rti.
                                <1>     ;    8. initializes u.r0 and u.sp
                                <1>     ;    9. zeros user's core down to u.r0
                                <1>     ;   10. reads executable file from storage device into core
                                <1>     ;       starting at location 'core'.
                                <1>     ;   11. sets u.break to point to end of user's code with
                                <1>     ;       data area appended.
                                <1>     ;   12. calls 'sysret' which returns control at location
                                <1>     ;       'core' via 'rti' instruction.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysexec; namep; argp
                                <1>     ; Arguments:
                                <1>     ;       namep - points to pathname of file to be executed
                                <1>     ;       argp  - address of table of argument pointers
                                <1>     ;       argp1... argpn - table of argument pointers
                                <1>     ;       argp1:<...0> ... argpn:<...0> - argument strings
                                <1>     ; Inputs: (arguments)
                                <1>     ; Outputs: -
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification:
                                <1>     ;       User application runs in it's own virtual space
                                <1>     ;       which is izolated from kernel memory (and other
                                <1>     ;       memory pages) via 80386      paging in ring 3
                                <1>     ;       privilige mode. Virtual start address is always 0.
                                <1>     ;       User's core memory starts at linear address 400000h
                                <1>     ;       (the end of the 1st 4MB).
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       user/application segment and system/kernel segment
                                <1>     ;       are different and sysenter/sysret/sysrele routines
                                <1>     ;       are different (user's registers are saved to
                                <1>     ;       and then restored from system's stack.)
                                <1>     ;
                                <1>     ;       NOTE: Retro UNIX 8086 v1 'arg2' routine gets these
                                <1>     ;             arguments which were in these registers;
                                <1>     ;             but, it returns by putting the 1st argument
                                <1>     ;             in 'u.namep' and the 2nd argument
                                <1>     ;             on top of stack. (1st argument is offset of the
                                <1>     ;             file/path name in the user's program segment.)
                                <1>
                                <1>     ;call  arg2
                                <1>     ; * name - 'u.namep' points to address of file/path name
                                <1>     ;          in the user's program segment ('u.segmnt')
                                <1>     ;          with offset in BX register (as sysopen argument 1).
                                <1>     ; * argp - sysexec argument 2 is in CX register
                                <1>     ;          which is on top of stack.
                                <1>     ;
                                <1>             ; jsr r0,arg2 / arg0 in u.namep,arg1 on top of stack
                                <1>
                                <1>     ; 23/06/2015 (32 bit modifications)
                                <1>
                                <1>     ;; 13/11/2017
                                <1>     ;;mov  [u.namep], ebx ; argument 1
                                <1>         ; 18/10/2015
0000FE8E 890D[6C8F0100]         <1>     mov     [argv], ecx  ; * ; argument 2
                                <1>
                                <1>     ; 13/11/2017
0000FE94 89DE                   <1>     mov    esi, ebx
0000FE96 E83A1E0000             <1>     call   set_working_path_x
0000FE9B 7319                   <1>     jnc    short sysexec_0
                                <1>
                                <1>     ;; 'bad command or file name'
                                <1>     ;mov   eax, ERR_BAD_CMD_ARG ; 01h ; TRDOS 8086
                                <1>
                                <1>     ; 'file not found !' error
0000FE9D B802000000             <1>     mov    eax, ERR_NOT_FOUND ; 02h ; TRDOS 8086
                                <1> sysexec_not_found_err:
                                <1> sysexec_access_error:
                                <1> sysexec_ext_error:
0000FEA2 A3[6C8E0100]           <1>     mov    [u.r0], eax
0000FEA7 A3[D88E0100]           <1>     mov    [u.error], eax
0000FEAC E8F91E0000             <1>     call   reset_working_path
0000FEB1 E9B8CBFFFF             <1>     jmp    error
                                <1>
                                <1> sysexec_0:
                                <1>     ; 13/11/2017
                                <1>     ;mov   esi, FindFile_Name
0000FEB6 66B80018               <1>         mov ax, 1800h ; Only files
0000FEBA E8BB8AFFFF             <1>     call   find_first_file
0000FEBF 72E1                   <1>     jc     short sysexec_not_found_err ; eax = 2
                                <1>
                                <1>     ; check_ file attributes
                                <1>     ; (attribute bits = 00ADVSHR) ; 18h = Directory+Volume
                                <1>     ; BL = Attributes byte
                                <1>
0000FEC1 F6C306                 <1>         test       bl, 6  ; system file or hidden file (S+H)
                                <1>     ;jz    short sysexec_0ext
0000FEC4 7417                   <1>     jz     short sysexec_1 ; yes
                                <1>
                                <1>     ; 13/11/2017
                                <1>     ; /// TRDOS386 permission check for multiuser mode ///
                                <1>     ; SYSTEM file or HIDDEN file !!
                                <1>     ; (Only super user has permission to run this file.)
                                <1>
                                <1>     ; ([u.uid]=0 for super user or root in multiuser mode)
                                <1>     ; ([u.uid]=0 for any users in singleuser mode)
0000FEC6 803D[BE8E0100]00       <1>     cmp    byte [u.uid], 0 ; Super User ([u.uid]=0) ?
```

```
                                <1>     ;jna    short sysexec_0ext
0000FECD 760E                   <1>     jna     short sysexec_1 ; yes
                                <1>
                                <1>     ; 'permission denied !' error
0000FECF B80B000000             <1>         mov eax, ERR_FILE_ACCESS  ; 11 = ERR_PERM_DENIED
0000FED4 EBCC                   <1>         jmp short sysexec_access_error
                                <1>
                                <1> sysexec_not_exf:
                                <1>     ; 'not executable file !' error
0000FED6 B816000000             <1>     mov     eax, ERR_NOT_EXECUTABLE
0000FEDB EBC5                   <1>     jmp     sysexec_ext_error
                                <1>
                                <1> ;sysexec_0ext:
                                <1> sysexec_1:
                                <1>     ; 18/11/2017
0000FEDD BE[58800100]           <1>     mov     esi, FindFile_Name
                                <1>     ; 13/11/2017
                                <1>     ; check program file name extension
                                <1>     ; ('.PRG' for current TRDOS version)
0000FEE2 E8F9A3FFFF             <1>     call    check_prg_filename_ext
0000FEE7 72ED                   <1>     jc      short sysexec_not_exf
                                <1>
                                <1>     ; 18/11/2017
0000FEE9 3C50                   <1>     cmp     al, 'P'
0000FEEB 75E9                   <1>     jne     short sysexec_not_exf
                                <1>
                                <1>     ; '.PRG' extension is OK.
                                <1>     ; Only '.PRG' files are valid program files
                                <1>     ; for current TRDOS 386 version.
                                <1>
0000FEED 8B15[84800100]         <1>     mov     edx, [FindFile_DirEntry+DirEntry_FileSize]
0000FEF3 66A1[7C800100]         <1>     mov     ax, [FindFile_DirEntry+DirEntry_FstClusHI]
0000FEF9 C1E010                 <1>     shl     eax, 16
0000FEFC 66A1[82800100]         <1>     mov     ax, [FindFile_DirEntry+DirEntry_FstClusLO]
                                <1>     ; EAX = First Cluster number
                                <1>     ; EDX = File Size
                                <1>
0000FF02 A3[748F0100]           <1>     mov     [ii], eax
0000FF07 8915[788F0100]         <1>     mov     [i.size], edx
                                <1>
                                <1> ;sysexec_1:
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 06/02/2022 - Retro UNIX 386 v1.2
                                <1>     ; 13/11/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 24/06/2015 - 23/10/2015 (Retro UNIX 386 v1)
                                <1>         ; Moving arguments to the end of [u.upage]
                                <1>     ; (by regarding page borders in user's memory space)
                                <1>     ;
                                <1>     ; 10/10/2015
                                <1>     ; 21/07/2015
                                <1>     ;mov    ebp, esp ; (**)
                                <1>     ; 18/10/2015
                                <1>     ;mov    edi, ebp
                                <1>     ; 23/07/2022
0000FF0D 89E7                   <1>     mov     edi, esp ; (**)
0000FF0F B900010000             <1>     mov     ecx, MAX_ARG_LEN ; 256
                                <1>     ;sub    edi, MAX_ARG_LEN ; 256
0000FF14 29CF                   <1>     sub     edi, ecx
0000FF16 89FC                   <1>     mov     esp, edi ; *!*
0000FF18 31C0                   <1>     xor     eax, eax
0000FF1A A3[988E0100]           <1>     mov     [u.nread], eax ; 0
                                <1>     ; ([argc] must be cleared because previous 'sysexec'
                                <1>     ; may leave it with any value after an error))
                                <1>     ;mov    [argc], ax ; 0 ; 13/11/2017
                                <1>     ; 23/07/2022
0000FF1F A3[688F0100]           <1>     mov     [argc], eax ; 0
0000FF24 49                     <1>     dec     ecx ; 256 - 1
0000FF25 890D[948E0100]         <1>     mov     [u.count], ecx ; MAX_ARG_LEN - 1 ; 255
                                <1>     ;mov    dword [u.count], MAX_ARG_LEN - 1 ; 255
                                <1> sysexec_2:
0000FF2B 8B35[6C8F0100]         <1>     mov     esi, [argv] ; 18/10/2015
0000FF31 E86E020000             <1>     call    get_argp
                                <1>     ;mov    ecx, 4
                                <1>     ; 23/07/2022
0000FF36 31C9                   <1>     xor     ecx, ecx
0000FF38 B104                   <1>     mov     cl, 4
                                <1> sysexec_3:
0000FF3A 21C0                   <1>     and     eax, eax
0000FF3C 7453                   <1>         jz  short sysexec_6 ; 23/07/2022
                                <1>     ; 18/10/2015
0000FF3E 010D[6C8F0100]         <1>     add     [argv], ecx ; 4
                                <1>     ;;inc   word [argc]
                                <1>     ; 23/07/2022
                                <1>     ;inc    dword [argc]
0000FF44 FE05[688F0100]         <1>     inc     byte [argc]
                                <1>     ;
0000FF4A A3[908E0100]           <1>     mov     [u.base], eax
                                <1>     ; 23/10/2015
0000FF4F 66C705[D08E0100]00-    <1>     mov     word [u.pcount], 0
0000FF57 00                     <1>
                                <1> sysexec_4:
0000FF58 E85F0B0000             <1>     call    cpass ; get a character from user's core memory
0000FF5D 750B                   <1>         jnz short sysexec_5
                                <1>             ; (max. 255 chars + null)
                                <1>     ; 18/10/2015
0000FF5F 28C0                   <1>     sub     al, al
0000FF61 AA                     <1>     stosb
0000FF62 FF05[988E0100]         <1>     inc     dword [u.nread]
                                <1>     ; 23/07/2022
0000FF68 EB27                   <1>     jmp     short sysexec_6 ; 24/04/2016
                                <1> sysexec_5:
0000FF6A AA                     <1>     stosb
0000FF6B 20C0                   <1>     and     al, al
0000FF6D 75E9                   <1>     jnz     short sysexec_4
                                <1>     ;mov    ecx, 4
                                <1>     ; 23/07/2022
0000FF6F 31C9                   <1>     xor     ecx, ecx
0000FF71 B104                   <1>     mov     cl, 4
0000FF73 390D[648F0100]         <1>     cmp     [ncount], ecx ; 4
0000FF79 72B0                   <1>     jb      short sysexec_2
0000FF7B 8B35[608F0100]         <1>     mov     esi, [nbase]
0000FF81 010D[608F0100]         <1>     add     [nbase], ecx ; 4
                                <1>     ;sub    [ncount], cx
                                <1>     ; 23/07/2022
0000FF87 290D[648F0100]         <1>     sub     [ncount], ecx
0000FF8D 8B06                   <1>     mov     eax, [esi]
0000FF8F EBA9                   <1>     jmp     short sysexec_3
                                <1>
                                <1> sysexec_6:
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 19/11/2017
                                <1>     ; 18/11/2017
                                <1>     ; 14/11/2017
                                <1>     ; 13/11/2017
```

```
0000FF91 8925[6C8F0100]      <1>     mov    [argv], esp ; *!* ; start address of argument list
                             <1>
                             <1>     ; 04/01/2017
                             <1>     ; 24/10/2016
                             <1>     ;;02/05/2016
                             <1>     ; 23/04/2016 (TRDOS 386)
                             <1>     ; 18/10/2015 ('sysexec_6')
                             <1>     ; 23/06/2015
0000FF97 A1[C48E0100]        <1>     mov    eax, [u.pgdir] ; physical address of page directory
                             <1>     ;cmp   eax, [k_page_dir] ; TRDOS MainProg ?
                             <1>     ;je    short sysexec_7
                             <1>     ; 19/11/2017
0000FF9C 8B1D[C88E0100]      <1>     mov    ebx, [u.ppgdir] ; phy addr of the parent's page dir
0000FFA2 E8B359FFFF          <1>     call   deallocate_page_dir
                             <1> sysexec_7:
0000FFA7 E8E358FFFF          <1>     call   make_page_dir
                             <1>     ;jc    panic  ; allocation error
                             <1>     ;             ; after a deallocation would be nonsence !?
                             <1>     ; 23/07/2022
0000FFAC 7243                <1>     jc     short sysexec_panic
                             <1>
                             <1>     ; 24/07/2015
                             <1>     ; map kernel pages (1st 4MB) to PDE 0
                             <1>     ;     of the user's page directory
                             <1>     ;     (It is needed for interrupts!)
                             <1>     ; 18/10/2015
0000FFAE 8B15[B0760100]      <1>     mov    edx, [k_page_dir] ; Kernel's page directory
0000FFB4 8B02                <1>     mov    eax, [edx] ; physical address of
                             <1>                       ; kernel's first page table (1st 4 MB)
                             <1>                       ; (PDE 0 of kernel's page directory)
0000FFB6 8B15[C48E0100]      <1>     mov    edx, [u.pgdir]
0000FFBC 8902                <1>     mov    [edx], eax ; PDE 0 (1st 4MB)
                             <1>     ;
                             <1>     ; 20/07/2015
0000FFBE BB00004000          <1>     mov    ebx, CORE ; start address = 0 (virtual) + CORE
                             <1>     ; 18/10/2015
0000FFC3 BE[588F0100]        <1>     mov    esi, pcore ; physical start address
                             <1> sysexec_8:
0000FFC8 B907000000          <1>     mov    ecx, PDE_A_USER + PDE_A_WRITE + PDE_A_PRESENT
0000FFCD E8DB58FFFF          <1>     call   make_page_table
                             <1>     ;jc    panic
                             <1>     ; 23/07/2022
0000FFD2 721D                <1>     jc     short sysexec_panic
                             <1>     ;
                             <1>     ;mov   ecx, PTE_A_USER + PTE_A_WRITE + PTE_A_PRESENT
0000FFD4 E8E258FFFF          <1>     call   make_page ; make new page, clear and set the pte
                             <1>     ;jc    panic
                             <1>     ; 23/07/2022
0000FFD9 7216                <1>     jc     short sysexec_panic
                             <1>     ;
0000FFDB 8906                <1>     mov    [esi], eax ; 24/06/2015
                             <1>     ; ebx = virtual address (24/07/2015)
                             <1>     ; 23/07/2022
                             <1>     ;call  add_to_swap_queue
                             <1>     ; 18/10/2015
0000FFDD 81FE[5C8F0100]      <1>     cmp    esi, ecore ; user's stack (last) page ?
0000FFE3 7411                <1>     je     short sysexec_9 ; yes
0000FFE5 BE[5C8F0100]        <1>     mov    esi, ecore  ; physical address of the last page
                             <1>     ; 20/07/2015
0000FFEA BB00F0FFFF          <1>     mov    ebx, (ECORE - PAGE_SIZE) + CORE
                             <1>     ; ebx = virtual end address + segment base address - 4K
0000FFEF EBD7                <1>         jmp     short sysexec_8
                             <1> sysexec_panic:
                             <1>     ; 23/07/2022
0000FFF1 E9236EFFFF          <1>     jmp    panic
                             <1>
                             <1> sysexec_9:
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 19/11/2017
                             <1>     ; 24/04/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 25/06/2015 - 26/08/2015 - 18/10/2015
                             <1>     ; move arguments from kernel stack to [ecore]
                             <1>     ; (argument list/line will be copied from kernel stack
                             <1>     ; frame to the last (stack) page of user's core memory)
                             <1>     ; 18/10/2015
0000FFF6 8B3D[5C8F0100]      <1>     mov    edi, [ecore]
0000FFFC 81C700100000        <1>     add    edi, PAGE_SIZE
                             <1>     ; 19/11/2017
                             <1>     ;sub   edi, 4
                             <1>     ;mov   dword [edi], 0
                             <1>     ;mov   ebx, edi
                             <1>     ;
                             <1>     ;movzx eax, word [argc]
                             <1>     ;or    eax, eax
                             <1>     ;jz    short sysexec_13 ; 19/11/2017
                             <1>     ;;jnz  short sysexec_10
                             <1>     ;;mov  ebx, edi
                             <1>     ;;sub  ebx, 4
                             <1>     ;;mov  [ebx], eax ; 0
                             <1>     ;;jmp  short sysexec_13
                             <1>     ; 23/07/2022
                             <1>     ; [argc] < 32
00010002 A1[688F0100]        <1>     mov    eax, [argc]
00010007 09C0                <1>     or     eax, eax
00010009 7509                <1>     jnz    short sysexec_10
0001000B 89FB                <1>     mov    ebx, edi
0001000D 83EB04              <1>     sub    ebx, 4
00010010 8903                <1>     mov    [ebx], eax ; 0
00010012 EB47                <1>     jmp    short sysexec_13
                             <1>
                             <1> sysexec_10:
00010014 8B0D[988E0100]      <1>     mov    ecx, [u.nread]
                             <1>     ; 13/11/2017
                             <1>     ;;mov  esi, TextBuffer ; 'load_and_execute_file'
                             <1>     ;mov   esi, esp       ; 'sysexec'
0001001A 8B35[6C8F0100]      <1>     mov    esi, [argv] ; 24/04/2016 (TRDOS 386  = TRDOS v2.0)
                             <1>     ; 23/07/2022
00010020 29CF                <1>     sub    edi, ecx ; page end address - argument list length
                             <1>     ;sub   ebx, ecx ; 19/11/2017
                             <1>
                             <1>     ;;;;
                             <1>     ; 23/07/2022
                             <1>     ; (move edi -backward- to dword boundary)
                             <1>     ; ((this will prevent 'general protection fault' error
                             <1>     ;  as result of a lodsd or dword move instruction
                             <1>     ;  at the end of argument list))
00010022 83EF03              <1>     sub    edi, 3
00010025 83E7FC              <1>     and    edi, ~3 ; (*)
                             <1>     ;;;
                             <1>
00010028 89C2                <1>     mov    edx, eax
0001002A FEC2                <1>     inc    dl ; argument count + 1 for argc value
0001002C C0E202              <1>     shl    dl, 2  ; 4 * (argument count + 1)
                             <1>     ; edx <= 128
```

```
0001002F 89FB                <1>     mov     ebx, edi
                             <1>     ;mov    edi, ebx ; 19//11/2017
                             <1>     ; 23/07/2022 (*) - edi is already dword aligned -
                             <1>     ;and    bl, 0FCh ; 32 bit (dword) alignment
00010031 29D3                <1>     sub     ebx, edx
00010033 89FA                <1>     mov     edx, edi
00010035 F3A4                <1>     rep     movsb
00010037 89D6                <1>     mov     esi, edx
00010039 89DF                <1>     mov     edi, ebx
0001003B BA00F0BFFF          <1>     mov     edx, ECORE - PAGE_SIZE ; virtual addr. of the last page
00010040 2B15[5C8F0100]      <1>     sub     edx, [ecore] ; difference (virtual - physical)
00010046 AB                  <1>     stosd   ; eax = argument count
                             <1> sysexec_11:
00010047 89F0                <1>     mov     eax, esi
00010049 01D0                <1>     add     eax, edx
0001004B AB                  <1>     stosd   ; eax = virtual address
                             <1>     ; 23/07/2022
0001004C FE0D[688F0100]      <1>     dec     byte [argc]
                             <1>     ;dec    dword [argc]
                             <1>     ;;dec   word [argc] ; 14/11/2017
00010052 7407                <1>     jz      short sysexec_13
                             <1> sysexec_12:
00010054 AC                  <1>     lodsb
00010055 20C0                <1>     and     al, al
00010057 75FB                <1>     jnz     short sysexec_12
00010059 EBEC                <1>     jmp     short sysexec_11
                             <1> sysexec_13:
                             <1>     ; 24/10/2016
                             <1>     ; 24/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 23/06/2015 - 19/10/2015 (Retro UNIX 386 v1, 'sysexec_13')
                             <1>     ;
                             <1>     ; moving arguments to [ecore] is OK here..
                             <1>     ;
                             <1>     ; ebx = beginning addres of argument list pointers
                             <1>             ;       in user's stack
0001005B 2B1D[5C8F0100]      <1>     sub     ebx, [ecore]
00010061 81C300F0BFFF        <1>     add      ebx, (ECORE - PAGE_SIZE)
                             <1>                     ; end of core - 4096 (last page)
                             <1>                     ; (virtual address)
00010067 891D[6C8F0100]      <1>     mov     [argv], ebx
0001006D 891D[9C8E0100]      <1>     mov     [u.break], ebx ; available user memory
                             <1>     ;
00010073 29C0                <1>     sub     eax, eax
00010075 C705[948E0100]2000- <1>     mov     dword [u.count], 32 ; Executable file header size
0001007D 0000                <1>
0001007F C705[808E0100]-     <1>     mov     dword [u.fofp], u.off
00010085 [8C8E0100]          <1>
00010089 A3[8C8E0100]        <1>     mov     [u.off], eax ; 0
0001008E A3[908E0100]        <1>     mov     [u.base], eax ; 0, start of user's core (virtual)
                             <1>     ; 24/10/2016
00010093 A0[76770100]        <1>     mov     al, [Current_Drv]
00010098 A2[518E0100]        <1>     mov     [cdev], al
                             <1>     ;
0001009D A1[748F0100]        <1>     mov     eax, [ii] ; Fist Cluster of the Program (PRG) file
                             <1>     ; EAX = First cluster of the executable file
000100A2 E8DF050000          <1>     call    readi
                             <1>
000100A7 8B0D[9C8E0100]      <1>     mov     ecx, [u.break] ; top of user's stack (physical addr.)
000100AD 890D[948E0100]      <1>     mov     [u.count], ecx ; save for overrun check
                             <1>     ;
000100B3 8B0D[988E0100]      <1>     mov     ecx, [u.nread]
000100B9 890D[9C8E0100]      <1>     mov     [u.break], ecx ; virtual address (offset from start)
000100BF 80F920              <1>     cmp     cl, 32
000100C2 7540                <1>         jne     short sysexec_15
                             <1>     ;:
                             <1>     ; Retro UNIX 386 v1 (32 bit) executable file header format
000100C4 8B35[588F0100]      <1>     mov     esi, [pcore] ; start address of user's core memory
                             <1>                          ; (phys. start addr. of the exec. file)
000100CA AD                  <1>     lodsd
000100CB 663DEB1E            <1>     cmp     ax, 1EEBh ; EBH, 1Eh -> jump to +32
000100CF 7533                <1>     jne     short sysexec_15
000100D1 AD                  <1>     lodsd
000100D2 89C1                <1>     mov     ecx, eax ; text (code) section size
000100D4 AD                  <1>     lodsd
000100D5 01C1                <1>     add     ecx, eax ; + data section size (initialized data)
000100D7 89CB                <1>     mov     ebx, ecx
000100D9 AD                  <1>     lodsd
000100DA 01C3                <1>     add     ebx, eax ; + bss section size (for overrun checking)
000100DC 3B1D[948E0100]      <1>     cmp     ebx, [u.count]
000100E2 7711                <1>     ja      short sysexec_14  ; program overruns stack !
                             <1>     ;
                             <1>     ; add bss section size to [u.break]
000100E4 0105[9C8E0100]      <1>     add     [u.break], eax
                             <1>     ;
000100EA 83E920              <1>     sub     ecx, 32  ; header size (already loaded)
                             <1>     ;cmp    ecx, [u.count]
                             <1>     ;jnb    short sysexec_16
000100ED 890D[948E0100]      <1>     mov     [u.count], ecx ; required read count
000100F3 EB29                <1>     jmp     short sysexec_16
                             <1> sysexec_14:
                             <1>     ; insufficient (out of) memory
000100F5 C705[D88E0100]0400- <1>     mov     dword [u.error], ERR_MINOR_IM ; 1
000100FD 0000                <1>
000100FF E96AC9FFFF          <1>     jmp     error
                             <1> sysexec_15:
00010104 8B15[788F0100]      <1>         mov edx, [i.size] ; file size
0001010A 29CA                <1>     sub     edx, ecx ; file size - loaded bytes
0001010C 7626                <1>     jna     short sysexec_17 ; no need to next read
0001010E 01D1                <1>     add     ecx, edx ; [i.size]
00010110 3B0D[948E0100]      <1>     cmp     ecx, [u.count] ; overrun check (!)
00010116 77DD                <1>     ja      short sysexec_14
00010118 8915[948E0100]      <1>     mov     [u.count], edx
                             <1> sysexec_16:
0001011E A1[748F0100]        <1>     mov     eax, [ii] ; first cluster
00010123 E85E050000          <1>     call    readi
00010128 8B0D[988E0100]      <1>     mov     ecx, [u.nread]
0001012E 010D[9C8E0100]      <1>     add     [u.break], ecx
                             <1> sysexec_17:
                             <1>     ; 07/07/2025
                             <1>     ;mov    eax, [ii] ; first cluster
                             <1>     ;call   iclose
00010134 31C0                <1>     xor      eax, eax
                             <1>
                             <1>     ; 21/08/2024
                             <1>     ;inc    al
                             <1>     ;mov    [u.intr], ax ; 1 (interrupt/time-out is enabled)
                             <1>     ;mov    [u.quit], ax ; 1 ('crtl+brk' signal is enabled)
                             <1>     ; 23/07/2022
                             <1>     ;dec    al
                             <1>     ; 21/08/2024
00010136 48                  <1>     dec     eax
00010137 66A3[B88E0100]      <1>     mov     [u.intr], ax ; -1 ; 0FFFFh ; enable CTRL+CRK
0001013D 40                  <1>     inc     eax
0001013E 66A3[BA8E0100]      <1>     mov     [u.quit], ax ; 0 ; reset CTRL+BRK flag
```

```
                                <1>
                                <1>     ;cmp   dword [u.ppgdir], 0  ; is the caller MainProg (kernel) ?
00010144 3905[C88E0100]         <1>     cmp    [u.ppgdir], eax ; 0 ; 23/07/2022
0001014A 770C                   <1>     ja     short sysexec_18 ; no, the caller is user process
                                <1>     ; If the caller is kernel (MainProg), 'sysexec' will come here
0001014C 8B15[B0760100]         <1>     mov    edx, [k_page_dir] ; kernel's page directory
00010152 8915[C88E0100]         <1>     mov    [u.ppgdir], edx ; next time 'sysexec' must not come here
                                <1> sysexec_18:
                                <1>     ; 02/05/2016
                                <1>     ; 24/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; 18/10/2015 (Retro UNIX 386 v1)
                                <1>     ; 05/08/2015
                                <1>     ; 29/07/2015
                                <1>
                                <1> ;   ; **** arguments list test start - 19/11/2017
                                <1> ;   mov    ebp, [argv]
                                <1> ;   sub    ebp, ECORE - 4096
                                <1> ;   add    ebp, [ecore]
                                <1> ;
                                <1> ;   mov    ebx, [ebp]
                                <1> ;   mov    [argc], bx
                                <1> ;   add    ebp, 4
                                <1> ;   mov    byte [ccolor], 1Fh
                                <1> ;_zx0:
                                <1> ;   cmp    word [argc], 0
                                <1> ;   jna    short _zx2
                                <1> ;_zx1:
                                <1> ;   push   ebp
                                <1> ;   mov    esi, [ebp]
                                <1> ;
                                <1> ;   sub    esi, ECORE - 4096
                                <1> ;   add    esi, [ecore]
                                <1> ;
                                <1> ;   call   print_cmsg
                                <1> ;
                                <1> ;   dec    word [argc]
                                <1> ;   jz     short _zx2
                                <1> ;
                                <1> ;   mov    al, '.'
                                <1> ;   mov    bl, 07h
                                <1> ;   mov    bh, [u.ttyn]
                                <1> ;   call   _write_tty
                                <1> ;
                                <1> ;   pop    ebp
                                <1> ;   add    ebp, 4
                                <1> ;   jmp    short _zx1
                                <1> ;_zx2:
                                <1> ;   pop    ebp
                                <1> ;   mov    byte [ccolor], 07h
                                <1> ;   mov    eax, 1
                                <1> ;   ; **** arguments list test stop
                                <1> ;   Test result is OK! (there is not a wrong thing) - 19/11/2017
                                <1>
00010158 8B2D[6C8F0100]         <1>     mov    ebp, [argv] ; user's stack pointer must point to argument
                                <1>                        ; list pointers (argument count)
0001015E FA                     <1>     cli
0001015F 8B25[4C760100]         <1>         mov     esp, [tss.esp0] ; ring 0 (kernel) stack pointer
                                <1>     ;mov   esp, [u.sp] ; Restore Kernel stack
                                <1>                        ; for this process
                                <1>     ;add   esp, 20 ; --> EIP, CS, EFLAGS, ESP, SS
                                <1>     ;;xor  eax, eax ; 0
                                <1>     ; 23/07/2022
                                <1>     ;dec   al ; eax = 0
                                <1>     ; eax = 0
                                <1>
                                <1>     ;mov   edx, UDATA
                                <1>     ; 18/11/2017
00010165 6A23                   <1>     push   UDATA ; user's stack segment
                                <1>     ;push  edx
00010167 55                     <1>     push   ebp ; user's stack pointer
                                <1>                ; (points to number of arguments)
                                <1>
                                <1>     ; 04/01/2017
                                <1>     ; MainProg comes here while [sysflg]= 0FFh
                                <1>     ; (but sysexec comes here while [sysflg]= 0)
00010168 C605[608E0100]00       <1>     mov    byte [sysflg], 0 ; 04/01/2017
                                <1>                             ; (timer_int sysflg control)
0001016F FB                     <1>     sti
00010170 9C                     <1>     pushfd ; EFLAGS
                                <1>            ; Set IF for enabling interrupts in user mode
                                <1>     ;or    dword [esp], 200h
                                <1>     ;
                                <1>     ;mov   bx, UCODE
                                <1>     ;push  bx ; user's code segment
00010171 6A1B                   <1>     push   UCODE
                                <1>     ;push  0
00010173 50                     <1>     push   eax ; EIP (=0) - start address -
00010174 8925[648E0100]         <1>     mov    [u.sp], esp ; 29/07/2015
                                <1>     ; 05/08/2015
                                <1>     ; Remedy of a General Protection Fault during 'iretd' is here !
                                <1>     ; ('push dx' would cause to general protection fault,
                                <1>     ; after 'pop ds' etc.)
                                <1>     ;
                                <1>     ;; push dx ; ds (UDATA)
                                <1>     ;; push dx ; es (UDATA)
                                <1>     ;; push dx ; fs (UDATA)
                                <1>     ;; push dx ; gs (UDATA)
                                <1>     ;
                                <1>     ; This is a trick to prevent general protection fault
                                <1>     ; during 'iretd' intruction at the end of 'sysrele' (in u1.s):
0001017A 66BA2300               <1>     mov    dx, UDATA ; 19/11/2017
0001017E 8EC2                   <1>     mov    es, dx ; UDATA
00010180 06                     <1>     push   es ; ds (UDATA)
00010181 06                     <1>     push   es ; es (UDATA)
00010182 06                     <1>     push   es ; fs (UDATA)
00010183 06                     <1>     push   es ; gs (UDATA)
00010184 66BA1000               <1>     mov    dx, KDATA
00010188 8EC2                   <1>     mov    es, dx
                                <1>     ;
                                <1>     ;; pushad simulation
0001018A 89E5                   <1>     mov    ebp, esp ; esp before pushad
0001018C 50                     <1>     push   eax ; eax (0)
0001018D 50                     <1>     push   eax ; ecx (0)
0001018E 50                     <1>     push   eax ; edx (0)
0001018F 50                     <1>     push   eax ; ebx (0)
00010190 55                     <1>     push   ebp ; esp before pushad
00010191 50                     <1>     push   eax ; ebp (0)
00010192 50                     <1>     push   eax ; esi (0)
00010193 50                     <1>     push   eax ; edi (0)
                                <1>     ;
00010194 A3[6C8E0100]           <1>     mov    [u.r0], eax ; eax = 0
00010199 8925[688E0100]         <1>     mov    [u.usp], esp
                                <1>
                                <1>     ; 14/11/2017
```

```
0001019F E9ECC8FFFF          <1>     jmp    sysret0
                             <1>
                             <1> get_argp:
                             <1>     ; 08/08/2022
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 11/12/2021 - Retro UNIX 386 v1.2
                             <1>     ; 14/11/2017 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 18/10/2015 (nbase, ncount)
                             <1>     ; 21/07/2015
                             <1>     ; 24/06/2015 (Retro UNIX 386 v1)
                             <1>     ; Get (virtual) address of argument from user's core memory
                             <1>     ;
                             <1>     ; INPUT:
                             <1>     ;      esi = virtual address of argument pointer
                             <1>     ; OUTPUT:
                             <1>     ;      eax = virtual address of argument
                             <1>     ;
                             <1>     ; Modified registers: EAX, EBX, ECX, EDX, ESI
                             <1>     ;
000101A4 833D[C88E0100]00    <1>     cmp     dword [u.ppgdir], 0 ; /etc/init ?
                             <1>                                 ; (the caller is kernel)
                             <1>     ;jna   short get_argpk
                             <1>     ; 08/08/2022
000101AB 7705                <1>     ja     short get_argp5
000101AD E985000000          <1>     jmp    get_argpk
                             <1> get_argp5:
000101B2 89F3                <1>             mov    ebx, esi
000101B4 E8575AFFFF          <1>     call   get_physical_addr ; get physical address
000101B9 7253                <1>         jc      short get_argp_err ; 23/07/2022
000101BB A3[608F0100]        <1>     mov    [nbase], eax ; physical address
                             <1>     ;mov   [ncount], cx ; remain byte count in page (1-4096)
                             <1>     ; 23/07/2022
000101C0 890D[648F0100]      <1>     mov    [ncount], ecx
                             <1>     ;mov   eax, 4 ; 21/07/2015
000101C6 31C0                <1>     xor    eax, eax
000101C8 B004                <1>     mov    al, 4
                             <1>     ;cmp   cx, ax ; 4
                             <1>     ; 23/07/2022
000101CA 39C1                <1>     cmp    ecx, eax ; 4
000101CC 7354                <1>     jnb    short get_argp2
000101CE 89F3                <1>     mov    ebx, esi
000101D0 01CB                <1>     add    ebx, ecx
000101D2 E8395AFFFF          <1>     call   get_physical_addr ; get physical address
000101D7 7235                <1>     jc     short get_argp_err
                             <1>     ;push  esi
000101D9 89C6                <1>     mov    esi, eax
                             <1>     ;xchg  cx, [ncount]
                             <1>     ; 23/07/2022
000101DB 870D[648F0100]      <1>     xchg   ecx, [ncount]
000101E1 8735[608F0100]      <1>     xchg   esi, [nbase]
000101E7 B504                <1>     mov    ch, 4
000101E9 28CD                <1>     sub    ch, cl
                             <1> get_argp0:
000101EB AC                  <1>     lodsb
                             <1>     ;push  ax
                             <1>     ; 23/07/2022
000101EC 50                  <1>     push   eax
000101ED FEC9                <1>     dec    cl
000101EF 75FA                <1>         jnz     short get_argp0
000101F1 8B35[608F0100]      <1>     mov    esi, [nbase]
                             <1>     ; 21/07/2015
000101F7 0FB6C5              <1>     movzx  eax, ch
000101FA 0105[608F0100]      <1>     add    [nbase], eax
                             <1>     ;sub   [ncount], ax
                             <1>     ; 23/07/2022
00010200 2905[648F0100]      <1>     sub    [ncount], eax
                             <1> get_argp1:
00010206 AC                  <1>     lodsb
00010207 FECD                <1>     dec    ch
00010209 7445                <1>         jz      short get_argp3
                             <1>         ;push      ax
                             <1>     ; 23/07/2022
0001020B 50                  <1>     push   eax
0001020C EBF8                <1>     jmp     short get_argp1
                             <1> get_argp_err:
0001020E A3[D88E0100]        <1>     mov    [u.error], eax
                             <1>     ; 14/11/2017
00010213 B801000000          <1>     mov    eax, ERR_BAD_CMD_ARG ; 01h ; TRDOS 8086
00010218 A3[6C8E0100]        <1>     mov    [u.r0], eax
0001021D E94CC8FFFF          <1>     jmp    error
                             <1> get_argp2:
                             <1>     ; 21/07/2015
                             <1>     ;mov   eax, 4
00010222 8B15[608F0100]      <1>     mov    edx, [nbase] ; 18/10/2015
00010228 0105[608F0100]      <1>     add    [nbase], eax
                             <1>     ;sub   [ncount], ax
                             <1>     ; 23/07/2022
0001022E 2905[648F0100]      <1>     sub    [ncount], eax
                             <1>     ;
00010234 8B02                <1>     mov    eax, [edx]
00010236 C3                  <1>     retn
                             <1> get_argpk:
                             <1>     ; Argument is in kernel's memory space
00010237 66C705[648F0100]00- <1>     mov    word [ncount], PAGE_SIZE ; 4096
0001023F 10                  <1>
00010240 8935[608F0100]      <1>     mov    [nbase], esi
00010246 8305[608F0100]04    <1>     add    dword [nbase], 4
0001024D 8B06                <1>     mov    eax, [esi] ; virtual addr. = physical addr.
0001024F C3                  <1>     retn
                             <1> get_argp3:
00010250 B103                <1>     mov    cl, 3
                             <1> get_argp4:
00010252 C1E008              <1>     shl    eax, 8
                             <1>     ;pop   dx
                             <1>     ; 23/07/2022
00010255 5A                  <1>     pop    edx
00010256 88D0                <1>     mov    al, dl
00010258 E2F8                <1>         loop    get_argp4
                             <1>     ;pop   esi
0001025A C3                  <1>     retn
                             <1>
                             <1>     ; 23/07/2022
                             <1> %if 0
                             <1>
                             <1> sysstat:
                             <1>     ; 13/01/2017 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; temporary !
                             <1>     mov    eax, ERR_INV_FNUMBER ; 'invalid function number !'
                             <1>         mov     [u.error], eax
                             <1>         mov     [u.r0], eax
                             <1>     jmp    error
                             <1>
                             <1> sysfstat:
                             <1>     ; 13/01/2017 - TRDOS 386 (TRDOS v2.0)
```

```
                                <1>     ; temporary !
                                <1>     mov     eax, ERR_INV_FNUMBER ; 'invalid function number !'
                                <1>         mov     [u.error], eax
                                <1>         mov     [u.r0], eax
                                <1>     jmp     error
                                <1>
                                <1> %endif
                                <1>
                                <1> fclose:
                                <1>     ; 24/04/2025 - TRDOS 386 Kernel v2.0.10
                                <1>     ; 18/09/2024 - TRDOS 386 Kernel v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 06/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>     ; 18/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ;            (32 bit offset pointer modification)
                                <1>     ; 19/04/2013 - 12/01/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; Given the file descriptor (index to the u.fp list)
                                <1>     ; 'fclose' first gets the i-number of the file via 'getf'.
                                <1>     ; If i-node is active (i-number > 0) the entry in
                                <1>     ; u.fp list is cleared. If all the processes that opened
                                <1>     ; that file close it, then fsp etry is freed and the file
                                <1>     ; is closed. If not a return is taken.
                                <1>     ; If the file has been deleted while open, 'anyi' is called
                                <1>     ; to see anyone else has it open, i.e., see if it is appears
                                <1>     ; in another entry in the fsp table. Upon return from 'anyi'
                                <1>     ; a check is made to see if the file is special.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    r1 - contains the file descriptor (value=0,1,2...)
                                <1>     ;    u.fp - list of entries in the fsp table
                                <1>     ;    fsp - table of entries (4 words/entry) of open files.
                                <1>     ; OUTPUTS ->
                                <1>     ;    r1 - contains the same file descriptor
                                <1>     ;    r2 - contains i-number
                                <1>     ;
                                <1>     ; ((AX = R1))
                                <1>     ; ((Modified registers: EDX, EBX, ECX, ESI, EDI, EBP))
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification : CF = 1
                                <1>     ;              if i-number of the file is 0. (error)
                                <1>     ;
                                <1>     ; TRDOS 386 (06/10/2016)
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;      EAX = File Handle (File Descriptor, File Index)
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;      CF = 1 -> File not open !
                                <1>     ;      CF = 0 -> OK!
                                <1>     ;           EBX = File Number (System)
                                <1>     ;           [cdev] = Logical DOS Drive Number
                                <1>     ;           EAX = File Handle/Number (user)
                                <1>     ;
                                <1>     ; Modified Registers: EBX
                                <1>
0001025B 50                     <1>     push    eax ; File handle
                                <1>
0001025C E83D000000             <1>     call    getf
                                <1>     ;jc     device_close ; eax = device number
                                <1>     ; 17/04/2021 (temporary)
00010261 7302                   <1>     jnc     short _fclose_0
                                <1>     ; 24/04/2025 (BugFix for 'sysexit')
00010263 58                     <1>     pop     eax
                                <1>     ;jmp    rw2 ; file not open !
00010264 C3                     <1>     retn
                                <1> _fclose_0:
00010265 80BB[18840100]01       <1>     cmp     byte [ebx+OF_MODE], 1 ; open mode ; 0 = empty entry
0001026C 7227                   <1>     jb      short fclose_1       ; 1 = read, 2 = write
                                <1>
                                <1>     ; 18/09/2024 (EMPTY FILE BugFix)
                                <1>     ;cmp    eax, 1 ; is the first cluster number > 0
                                <1>     ;jb     short fclose_1 ; no, this is empty entry
                                <1>
                                <1> fclose_0:
0001026E FE8B[58840100]         <1>     dec     byte [ebx+OF_OPENCOUNT] ; decrement the number of processes
                                <1>                                     ; that have opened the file
00010274 791F                   <1>     jns     short fclose_1 ; jump if not negative (jump if bit 7 is 0)
                                <1>                     ; if all processes haven't closed the file, return
                                <1>     ;
                                <1>     ; eax ; First cluster
00010276 31C0                   <1>     xor     eax, eax ; 0
00010278 8883[18840100]         <1>     mov     [ebx+OF_MODE], al ; 0 = empty entry
                                <1>     ;mov    [ebx+OF_STATUS], al ; 0 = empty entry
                                <1>     ;shl    bx, 2
                                <1>     ; 23/07/2022
                                <1>     ;shl    bl, 2
0001027E C1E302                 <1>     shl     ebx, 2
00010281 8983[78830100]         <1>     mov     [ebx+OF_FCLUSTER], eax ; 0
00010287 8983[F8860100]         <1>     mov     [ebx+OF_CCLUSTER], eax ; 0
                                <1>     ;mov    [ebx+OF_CCINDEX], eax ; 0
                                <1>     ; 23/07/2022
                                <1>     ;mov    [ebx+OF_OPENCOUNT], al ; 0
0001028D A3[808E0100]           <1>     mov     [u.fofp], eax ; 0
                                <1>     ;shr    bx, 2
                                <1>     ; 23/07/2022
                                <1>     ;shr    bl, 2
00010292 C1EB02                 <1>     shr     ebx, 2
                                <1> fclose_1: ; 1:
00010295 58                     <1>     pop     eax ; File handle (File Descriptor, File Index)
00010296 C680[768E0100]00       <1>     mov     byte [eax+u.fp], 0 ; clear that entry in the u.fp list
0001029D C3                     <1>     retn
                                <1>
                                <1> getf:
                                <1>     ; 03/09/2024 - TRDOS 386 v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/04/2021 - TRDOS 386 v2.0.4
                                <1>     ;      (temporary modifications)
                                <1>     ; 12/10/2016
                                <1>     ; 11/10/2016
                                <1>     ; 08/10/2016
                                <1>     ; 06/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; / get the device number and the i-number of an open file
                                <1>     ; 13/05/2015
                                <1>     ; 11/05/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 19/04/2013 - 18/11/2013 (Retro UNIX 8086 v1)
                                <1>     ;
0001029E 89C3                   <1>     mov     ebx, eax
                                <1> getf1:
000102A0 83FB0A                 <1>     cmp     ebx, 10
000102A3 730A                   <1>         jnb short getf2
000102A5 8A9B[768E0100]         <1>     mov     bl, [ebx+u.fp]
000102AB 08DB                   <1>     or      bl, bl
```

```
000102AD 7504                <1>     jnz     short getf3
                             <1> getf2:
                             <1>     ; 'File not open !' error (ax=0)
000102AF 29C0                <1>     sub     eax, eax
                             <1>     ; 03/09/2024
000102B1 F9                  <1>     stc
000102B2 C3                  <1>     retn
                             <1> getf3:
                             <1>     ; 23/07/2022
                             <1>     ;test  bl, 80h
                             <1>     ;jnz   short getf5 ; device
000102B3 FECB                <1>     dec     bl ; 0 based
000102B5 8A83[F8830100]      <1>     mov     al, [ebx+OF_DRIVE]
000102BB A2[518E0100]        <1>     mov     [cdev], al
000102C0 C0E302              <1>     shl     bl, 2 ; *4 (dword offset)
                             <1>     ; 23/07/2022
                             <1>     ;shl   ebx, 2
000102C3 8B83[F8840100]      <1>     mov     eax, [ebx+OF_SIZE]
000102C9 A3[788F0100]        <1>     mov     [i.size], eax ; file size
000102CE 8D83[78840100]      <1>     lea     eax, [ebx+OF_POINTER] ; 12/10/2016
000102D4 A3[808E0100]        <1>     mov     [u.fofp], eax
000102D9 8B83[78830100]      <1>     mov     eax, [ebx+OF_FCLUSTER]
000102DF C0EB02              <1>     shr     bl, 2 ; /4 (byte offset)
                             <1>     ; 23/07/2022
                             <1>     ;shr   ebx, 2
                             <1>     ; 17/04/2021
000102E2 F8                  <1>     clc
                             <1> getf4:
000102E3 C3                  <1>     retn
                             <1> ;getf5:
                             <1>     ; 17/04/2021
                             <1>     ; (following code is disabled as temporary)
                             <1>     ;
                             <1>     ;; get device number
                             <1>     ;and   bl, 7Fh ; 1 to 7Fh
                             <1>     ;dec   bl ; 0 based (0 to 7Eh)
                             <1>     ;mov   al, [ebx+DEV_DRIVER]
                             <1>     ;mov   ch, [ebx+DEV_ACCESS]
                             <1>     ;mov   cl, [ebx+DEV_OPENMODE]
                             <1>     ;and   ch, 0FEh ; reset bit 0 ; dev_close
                             <1>     ;
                             <1>     ; 23/07/2022
                             <1>     ;stc ; cf = 1
                             <1>     ;retn
                             <1>
                             <1> trans_addr_nmbp:
                             <1>     ; 18/10/2015
                             <1>     ; 12/10/2015
000102E4 8B2D[888E0100]      <1>     mov     ebp, [u.namep]
                             <1> trans_addr_nm:
                             <1>     ; Convert virtual (pathname) address to physical address
                             <1>     ; (Retro UNIX 386 v1 feature only !)
                             <1>     ; 18/10/2015
                             <1>     ; 12/10/2015 (u.pnbase & u.pncount has been removed from code)
                             <1>     ; 02/07/2015
                             <1>     ; 17/06/2015
                             <1>     ; 16/06/2015
                             <1>     ;
                             <1>     ; INPUTS:
                             <1>     ;       ebp = pathname address (virtual) ; [u.namep]
                             <1>     ;       [u.pgdir] = user's page directory
                             <1>     ; OUTPUT:
                             <1>     ;        esi = physical address of the pathname
                             <1>     ;       ecx = remain byte count in the page
                             <1>     ;
                             <1>     ; (Modified registers: EAX, EBX, ECX, EDX, ESI)
                             <1>     ;
000102EA 833D[C88E0100]00    <1>         cmp     dword [u.ppgdir], 0  ; /etc/init ? (sysexec)
000102F1 7618                <1>     jna     short trans_addr_nmk ; the caller is os kernel;
                             <1>                                  ; it is already physical address
000102F3 50                  <1>     push    eax
000102F4 89EB                <1>     mov     ebx, ebp ; [u.namep] ; pathname address (virtual)
000102F6 E81559FFFF          <1>             call    get_physical_addr ; get physical address
000102FB 7204                <1>     jc      short tr_addr_nm_err
                             <1>     ; 18/10/2015
                             <1>     ; eax = physical address
                             <1>     ; cx = remain byte count in page (1-4096)
                             <1>             ; 12/10/2015 (cx = [u.pncount])
000102FD 89C6                <1>     mov     esi, eax ; 12/10/2015 (esi=[u.pnbase])
000102FF 58                  <1>     pop     eax
00010300 C3                  <1>     retn
                             <1>
                             <1> tr_addr_nm_err:
00010301 A3[D88E0100]        <1>     mov     [u.error], eax
                             <1>     ;pop   eax
00010306 E963C7FFFF          <1>     jmp     error
                             <1>
                             <1> trans_addr_nmk:
                             <1>     ; 12/10/2015
                             <1>     ; 02/07/2015
0001030B 8B35[888E0100]      <1>     mov     esi, [u.namep]  ; [u.pnbase]
00010311 66B90010            <1>     mov     cx, PAGE_SIZE ; 4096 ; [u.pncount]
00010315 C3                  <1>     retn
                             <1>
                             <1> sysbreak:
                             <1>     ; 06/09/2024 - TRDOS 386 v2.0.9
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 18/10/2015
                             <1>     ; 07/10/2015
                             <1>     ; 23/06/2015 (Retro UNIX 386 v1 - Beginning)
                             <1>     ; 20/06/2013 - 24/03/2014 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; 'sysbreak' sets the programs break points.
                             <1>     ; It checks the current break point (u.break) to see if it is
                             <1>     ; between "core" and the stack (sp). If it is, it is made an
                             <1>     ; even address (if it was odd) and the area between u.break
                             <1>     ; and the stack is cleared. The new breakpoint is then put
                             <1>     ; in u.break and control is passed to 'sysret'.
                             <1>     ;
                             <1>     ; Calling sequence:
                             <1>     ;       sysbreak; addr
                             <1>     ; Arguments: -
                             <1>     ;
                             <1>     ; Inputs: u.break - current breakpoint
                             <1>     ; Outputs: u.break - new breakpoint
                             <1>     ;       area between old u.break and the stack (sp) is cleared.
                             <1>     ;
                             <1>     ; ..............................................................
                             <1>     ; 06/09/2024 - TRDOS 386 v2.0.9 - Major Modification -
                             <1>     ;
                             <1>     ;       This system call performs "malloc" function as in C.
                             <1>     ; Input:
                             <1>     ;    EBX = new u.break address (virtual, user's space)
                             <1>     ; (Note: Default/Initial u.break address is the start of the BSS
```

```
                                <1>     ;       section. Or it is file size for PRG -flat image- files.)
                                <1>     ; If EBX = 0FFFFFFFFh ; -1
                                <1>     ;    Return current u.break in EAX
                                <1>     ; Output:
                                <1>     ;    If EBX input is -1, EAX = current u.break address (virtual)
                                <1>     ;      otherwise, EAX = new u.break address
                                <1>     ; Area between the new u.break and the old u.break is cleared.
                                <1>     ; (Note: If the new break address cross overs user's esp,
                                <1>     ;       this area <new break - old break> is not cleared.)
                                <1>     ; If CF=1 at return, it means Memory Allocation Error.
                                <1>     ;              (or "Insufficient Memory" error)
                                <1>     ;      EAX = 0
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;      The user/application program puts breakpoint address
                                <1>     ;       in BX register as 'sysbreak' system call argument.
                                <1>     ;      (argument transfer method 1)
                                <1>     ;
                                <1>     ;  NOTE: Beginning of core is 0 in Retro UNIX 8086 v1 !
                                <1>     ;      ((!'sysbreak' is not needed in Retro UNIX 8086 v1!))
                                <1>     ;  NOTE:
                                <1>     ;      'sysbreak' clears extended part (beyond of previous
                                <1>     ;      'u.break' address) of user's memory for original unix's
                                <1>     ;      'bss' compatibility with Retro UNIX 8086 v1 (19/11/2013)
                                <1>
                                <1>             ; mov u.break,r1 / move users break point to r1
                                <1>             ; cmp r1,$core / is it the same or lower than core?
                                <1>             ; blos 1f / yes, 1f
                                <1>     ; 23/06/2015
00010316 8B2D[9C8E0100]         <1>     mov     ebp, [u.break] ; virtual address (offset)
                                <1>
                                <1>     ; 06/09/2024
0001031C 31C0                   <1>     xor     eax, eax
0001031E A3[6C8E0100]           <1>     mov     [u.r0], eax ; 0 ; default ('memory allocation error')
00010323 48                     <1>     dec     eax ; -1
                                <1>
                                <1>     ; 06/09/2024 - TRDOS 386 v2.0.9
                                <1>     ; get u.break address (for 'malloc' in c compiler)
                                <1>     ; for PRG files:
                                <1>     ; default/initial u.break address is file size
00010324 39C3                   <1>     cmp     ebx, eax ; -1 ; 0FFFFFFFFh
00010326 7214                   <1>     jb      short sysbreak_@
                                <1>
                                <1> sysbreak_4:
00010328 89EB                   <1>     mov     ebx, ebp
                                <1>     ;(allocates a new page for user if it is not present)
0001032A E8E158FFFF             <1>     call    get_physical_addr ; get physical address
0001032F 72D0                   <1>     jc      short tr_addr_nm_err
                                <1>
00010331 892D[6C8E0100]         <1>     mov     [u.r0], ebp  ; start of bss space (to be allocated)
00010337 E952C7FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysbreak_@:
                                <1>     ;and    ebp, ebp
                                <1>     ;jz     short sysbreak_3
                                <1>     ; Retro UNIX 386 v1 NOTE: u.break points to virtual address !!!
                                <1>     ; (Even break point address is not needed for Retro UNIX 386 v1)
0001033C 8B15[648E0100]         <1>     mov     edx, [u.sp] ; kernel stack at the beginning of sys call
00010342 83C20C                 <1>     add     edx, 12 ; EIP -4-> CS -4-> EFLAGS -4-> ESP (user)
                                <1>     ; 07/10/2015
                                <1>     ;mov    [u.break], ebx ; virtual address !!!
                                <1>     ;
                                <1>     ; 06/09/2024
                                <1>     ; ebp = old/current u.break
                                <1>
00010345 3B1A                   <1>     cmp     ebx, [edx] ; compare new break point with
                                <1>                        ; with top of user's stack (virtual!)
00010347 732B                   <1>     jnb     short sysbreak_3
                                <1>             ; cmp r1,sp / is it the same or higher
                                <1>                     ; / than the stack?
                                <1>             ; bhis 1f / yes, 1f
00010349 89DE                   <1>     mov     esi, ebx
0001034B 29EE                   <1>     sub     esi, ebp ; new break point - old break point
                                <1>     ;jna    short sysbreak_3
                                <1>     ; 06/09/2024
0001034D 7706                   <1>     ja      short sysbreak_1
0001034F 74D7                   <1>     jz      short sysbreak_4
00010351 F7DE                   <1>     neg     esi ; convert to positive number (big-small)
00010353 89DD                   <1>     mov     ebp, ebx ; move small number (address) to ebp
                                <1>
                                <1> sysbreak_1:
                                <1>     ; 06/09/2024
00010355 53                     <1>     push    ebx
00010356 89EB                   <1>     mov     ebx, ebp
00010358 E8B358FFFF             <1>     call    get_physical_addr ; get physical address
0001035D 5B                     <1>     pop     ebx
0001035E 72A1                   <1>     jc      short tr_addr_nm_err ; 23/07/2022
                                <1>     ; 18/10/2015
00010360 89C7                   <1>     mov     edi, eax
00010362 29C0                   <1>     sub     eax, eax ; 0
                                <1>              ; ECX = remain byte count in page (1-4096)
00010364 39CE                   <1>     cmp     esi, ecx
00010366 7302                   <1>     jnb     short sysbreak_2
00010368 89F1                   <1>     mov     ecx, esi
                                <1> sysbreak_2:
0001036A 29CE                   <1>     sub     esi, ecx
0001036C 01CD                   <1>     add     ebp, ecx
0001036E F3AA                   <1>     rep     stosb
00010370 09F6                   <1>     or      esi, esi
00010372 75E1                   <1>     jnz     short sysbreak_1
                                <1>             ; bit $1,r1 / is it an odd address
                                <1>             ; beq 2f / no, its even
                                <1>             ; clrb (r1)+ / yes, make it even
                                <1>     ; 2: / clear area between the break point and the stack
                                <1>             ; cmp r1,sp / is it higher or same than the stack
                                <1>             ; bhis 1f / yes, quit
                                <1>             ; clr (r1)+ / clear word
                                <1>             ; br 2b / go back
                                <1>     ;pop    ebx
                                <1> sysbreak_3: ; 1:
                                <1>     ; 06/09/2024
00010374 891D[9C8E0100]         <1>     mov     [u.break], ebx ; virtual address !!!
                                <1>             ; jsr r0,arg; u.break / put the "address"
                                <1>                     ; / in u.break (set new break point)
                                <1>             ; br sysret4 / br sysret
                                <1>     ; 06/09/2024
0001037A 891D[6C8E0100]         <1>     mov     [u.r0], ebx ; return new break point in eax
00010380 E909C7FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysseek: ; / moves read write pointer in an fsp entry
                                <1>     ; 27/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 06/11/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
```

```
                                <1>     ; 07/07/2013 - 05/08/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysseek' changes the r/w pointer of (3rd word of in an
                                <1>     ; fsp entry) of an open file whose file descriptor is in u.r0.
                                <1>     ; The file descriptor refers to a file open for reading or
                                <1>     ; writing. The read (or write) pointer is set as follows:
                                <1>     ;       * if 'ptrname' is 0, the pointer is set to offset.
                                <1>     ;       * if 'ptrname' is 1, the pointer is set to its
                                <1>     ;         current location plus offset.
                                <1>     ;       * if 'ptrname' is 2, the pointer is set to the
                                <1>     ;         size of file plus offset.
                                <1>     ; The error bit (e-bit) is set for an undefined descriptor.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysseek; offset; ptrname
                                <1>     ; Arguments:
                                <1>     ;       offset - number of bytes desired to move
                                <1>     ;                the r/w pointer
                                <1>     ;       ptrname - a switch indicated above
                                <1>     ;
                                <1>     ; Inputs: r0 - file descriptor
                                <1>     ;       ; 27/12/2025
                                <1>     ;       EBX = File descriptor (number) -user-
                                <1>     ;       ECX = Offset (plus)
                                <1>     ;       EDX = 0 -> set to offset in ECX
                                <1>     ;           = 1 -> set to current offset + offset in ECX
                                <1>     ;           = 2 -> set to file size + offset in ECX
                                <1>     ; Outputs: -
                                <1>     ;       EAX = (new) File Offset ; 27/12/2025
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;        'sysseek' system call has three arguments; so,
                                <1>     ;       * 1st argument, file descriptor is in BX (BL) register
                                <1>     ;       * 2nd argument, offset is in CX register
                                <1>     ;       * 3rd argument, ptrname/switch is in DX (DL) register
                                <1>
0001038A 0305[908E0100]         <1>     call    seektell
                                <1>     ; EAX = Current R/W pointer of the file
                                <1>     ; EBX = [u.fofp]
                                <1>     ; [u.base] = offset (ECX input)
                                <1>
0001038A 0305[908E0100]         <1>     add     eax, [u.base]
00010390 8903                   <1>     mov     [ebx], eax
                                <1>     ; 27/12/2025
                                <1> sysseek_@:
00010392 A3[6C8E0100]           <1>     mov     [u.r0], eax ; return (new) file offset
00010397 E9F2C6FFFF             <1>     jmp     sysret
                                <1>
                                <1> systell: ; / get the r/w pointer
                                <1>     ; 27/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 06/11/2016 - TRDOS 386 (TRDOS v2.0) - temporary !-
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 07/07/2013 - 05/08/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ; ! 'systell' does not work in original UNIX v1,
                                <1>     ;           it returns with error !
                                <1>     ; Inputs: r0 - file descriptor
                                <1>     ;       ; 27/12/2025
                                <1>     ;       EBX = File descriptor (number) -user-
                                <1>     ;       ECX = ignored
                                <1>     ;       EDX = 0 -> return current file offset
                                <1>     ;           = 1 -> return current file offset
                                <1>     ;           = 2 -> return file size
                                <1>     ;           > 2 -> return current file offset
                                <1>     ;
                                <1>     ; Outputs: r0 - file r/w pointer
                                <1>
                                <1>     ;xor    ecx, ecx ; 0
                                <1>     ;
                                <1>     ; 27/12/2025
0001039C 83FA02                 <1>     cmp     edx, 2
0001039F 7405                   <1>     je      short systell_@
000103A1 BA01000000             <1>     mov     edx, 1 ; 05/08/2013
                                <1> systell_@:
                                <1>     ;call   seektell
000103A6 E808000000             <1>     call    seektell0 ; 05/08/2013
                                <1>     ;; 06/11/2016
                                <1>     ;;mov   eax, [ebx]
                                <1>     ; 27/12/2025
                                <1>     ;mov    [u.r0], eax
                                <1>     ;jmp    sysret
000103AB EBE5                   <1>     jmp     short sysseek_@
                                <1>
                                <1> ; Original unix v1 'systell' system call:
                                <1>             ; jsr r0,seektell
                                <1>             ; br error4
                                <1>
                                <1> seektell:
                                <1>     ; 24/04/2025 - TRDOS 386 v2.0.10
                                <1>     ; 06/11/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 03/01/2016
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 07/07/2013 - 05/08/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'seektell' puts the arguments from sysseek and systell
                                <1>     ; call in u.base and u.count. It then gets the i-number of
                                <1>     ; the file from the file descriptor in u.r0 and by calling
                                <1>     ; getf. The i-node is brought into core and then u.count
                                <1>     ; is checked to see it is a 0, 1, or 2.
                                <1>     ; If it is 0 - u.count stays the same
                                <1>     ;          1 - u.count = offset (u.fofp)
                                <1>     ;         2 - u.count = i.size (size of file)
                                <1>     ;
                                <1>     ; !! Retro UNIX 8086 v1 modification:
                                <1>     ;       Argument 1, file descriptor is in BX;
                                <1>     ;       Argument 2, offset is in CX;
                                <1>     ;       Argument 3, ptrname/switch is in DX register.
                                <1>     ;
                                <1>     ; ((Return -> eax = base for offset (position = base+offset))
                                <1>     ;
000103AD 890D[908E0100]         <1>     mov     [u.base], ecx ; offset
                                <1> seektell0:
000103B3 8915[948E0100]         <1>     mov     [u.count], edx
                                <1>     ; EBX = file descriptor (file number)
000103B9 E8E2FEFFFF             <1>     call    getf1
                                <1>     ; EAX = First cluster of the file
                                <1>     ; EBX = File number (Open file number)
                                <1>     ; [u.fofp] = Pointer to File pointer
                                <1>     ; [i.size] = File size
                                <1>
000103BE 09C0                   <1>     or      eax, eax
000103C0 7505                   <1>     jnz     short seektell1
```

```
                                <1>
                                <1>     ; 24/04/2025 - TRDOS 386 v2.0.10
                                <1>     ;mov    eax, ERR_FILE_NOT_OPEN
                                <1>     ;mov    [u.r0], eax
                                <1>     ;mov    dword [u.error], eax ; 'file not open !'
                                <1>     ;jmp    error
                                <1>     ; 24/04/2025
000103C2 E9E6CDFFFF             <1>     jmp     seektell_err ; sysclose_err
                                <1>
                                <1> seektell1:
000103C7 8B1D[808E0100]         <1>         mov     ebx, [u.fofp]
000103CD 803D[948E0100]01       <1>     cmp     byte [u.count], 1
000103D4 7705                   <1>     ja      short seektell2
000103D6 7409                   <1>     je      short seektell3
000103D8 31C0                   <1>     xor     eax, eax
000103DA C3                     <1>     retn
                                <1>
                                <1> seektell2:
000103DB A1[788F0100]           <1>         mov        eax, [i.size]
000103E0 C3                     <1>     retn
                                <1>
                                <1> seektell3:
000103E1 8B03                   <1>     mov     eax, [ebx]
000103E3 C3                     <1>     retn
                                <1>
                                <1> sysintr: ; / set interrupt handling
                                <1>     ; 21/08/2024 - TRDOS 386 v2.0.9 modification
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 07/07/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysintr' sets the interrupt handling value. It puts
                                <1>     ; argument of its call in u.intr then branches into 'sysquit'
                                <1>     ; routine. u.tty is checked if to see if a control tty exists.
                                <1>     ; If one does the interrupt character in the tty buffer is
                                <1>     ; cleared and 'sysret'is called. If one does not exits
                                <1>     ; 'sysret' is just called.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysintr; arg
                                <1>     ; Argument:
                                <1>     ;       arg - if 0, interrupts (ASCII DELETE) are ignored.
                                <1>     ;           - if 1, interrupts cause their normal result
                                <1>     ;                 i.e force an exit.
                                <1>     ;           - if arg is a location within the program,
                                <1>     ;               control is passed to that location when
                                <1>     ;               an interrupt occurs.
                                <1>     ; Inputs: -
                                <1>     ; Outputs: -
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       'sysintr' system call sets u.intr to value of BX
                                <1>     ;       then branches into sysquit.
                                <1>
                                <1>     ; 21/08/2024 - TRDOS 386
                                <1>     ; enable/disable terminate (CTRL+BREAK) interrupt
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;       bx = 0 -> disable CTRL+BREAK
                                <1>     ;              -u.intr will be set to 0, u.quit will be ignored-
                                <1>     ;       bx > 0 -> enable CTRL+BREAK (also default at start)
                                <1>     ;              -u.intr will be set to 0FFFFh, u.quit will be used-
                                <1>     ;       NOTE: u.quit is flag for CTRL+BREAK key press status
                                <1>     ;             -1 = pressed -termination request-, 0 = not pressed
                                <1>     ; OUTPUT:
                                <1>     ;       none
                                <1>
                                <1>     ;;;
                                <1>     ; 21/08/2024
000103E4 F7C3FFFF0000           <1>     test    ebx, 0FFFFh
000103EA 7403                   <1>     jz      short sysintr_@ ; 0
                                <1>     ;mov    bx, 0FFFFh
000103EC 31DB                   <1>     xor     ebx, ebx
000103EE 4B                     <1>     dec     ebx ; -1
                                <1> sysintr_@:
                                <1>     ;;;
000103EF 66891D[B88E0100]       <1>     mov     [u.intr], bx
                                <1>             ; jsr r0,arg; u.intr / put the argument in u.intr
                                <1>             ; br 1f / go into quit routine
000103F6 E993C6FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysquit:
                                <1>     ; 21/08/2024 - TRDOS 386 v2.0.9 modification
                                <1>     ;       get/reset QUIT (u.quit) status
                                <1>     ;
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 07/07/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'sysquit' turns off the quit signal. it puts the argument of
                                <1>     ; the call in u.quit. u.tty is checked if to see if a control
                                <1>     ; tty exists. If one does the interrupt character in the tty
                                <1>     ; buffer is cleared and 'sysret'is called. If one does not exits
                                <1>     ; 'sysret' is just called.
                                <1>     ;
                                <1>     ; Calling sequence:
                                <1>     ;       sysquit; arg
                                <1>     ; Argument:
                                <1>     ;       arg - if 0, this call diables quit signals from the
                                <1>     ;               typewriter (ASCII FS)
                                <1>     ;           - if 1, quits are re-enabled and cause execution to
                                <1>     ;               cease and a core image to be produced.
                                <1>     ;                 i.e force an exit.
                                <1>     ;           - if arg is an addres in the program,
                                <1>     ;               a quit causes control to sent to that
                                <1>     ;               location.
                                <1>     ; Inputs: -
                                <1>     ; Outputs: -
                                <1>     ; ............................................................
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification:
                                <1>     ;       'sysquit' system call sets u.quit to value of BX
                                <1>     ;       then branches into 'sysret'.
                                <1>
                                <1>     ; 21/08/2024
                                <1>     ;mov    [u.quit], bx
                                <1>     ;jmp    sysret
                                <1>             ; jsr r0,arg; u.quit / put argument in u.quit
                                <1>     ;1:
                                <1>             ; mov u.ttyp,r1 / move pointer to control tty buffer
                                <1>                             ; / to r1
                                <1>             ; beq sysret4 / return to user
                                <1>             ; clrb 6(r1) / clear the interrupt character
                                <1>                             ; / in the tty buffer
                                <1>             ; br sysret4 / return to user
```

```
                                <1>
                                <1>     ; 21/08/2024 - TRDOS 386
                                <1>     ; get/reset QUIT (u.quit) status
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;       none
                                <1>     ; OUTPUT:
                                <1>     ;       eax = 0 -> no CTRL+BREAK
                                <1>     ;       eax = -1 -> CTRL+BREAK status (but not applied)
                                <1>     ;               -u.intr may be 0-
                                <1>     ;       NOTE: u.quit will be reset to 0
                                <1>
000103FB 31C0                   <1>     xor     eax, eax ; 0
000103FD 663905[BA8E0100]       <1>     cmp     [u.quit], ax
00010404 7407                   <1>     jz      short sysquit_@ ; already 0
00010406 66A3[BA8E0100]         <1>     mov     [u.quit], ax ; 0 ; reset
0001040C 48                     <1>     dec     eax ; -1
                                <1> sysquit_@:
0001040D A3[6C8E0100]           <1>     mov     [u.r0], eax
00010412 E977C6FFFF             <1>     jmp     sysret
                                <1>
                                <1> %if 0
                                <1>
                                <1> anyi:
                                <1>     ; 23/07/2022
                                <1>     ; 06/10/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; Major Modification!
                                <1>     ; TRDOS 386 does not permit to delete a file while it is open
                                <1>     ; The role of 'anyi' procedure has beeen changed to ensure that.
                                <1>     ;
                                <1>     ; 22/06/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 25/04/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'anyi' is called if a file deleted while open.
                                <1>     ; "anyi" checks to see if someone else has opened this file.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    r1 - contains an i-number
                                <1>     ;    fsp - start of table containing open files
                                <1>     ;
                                <1>     ; OUTPUTS ->
                                <1>     ;    "deleted" flag set in fsp entry of another occurrence of
                                <1>     ;          this file and r2 points 1st word of this fsp entry.
                                <1>     ;    if file not found - bit in i-node map is cleared
                                <1>     ;                        (i-node is freed)
                                <1>     ;                all blocks related to i-node are freed
                                <1>     ;                all flags in i-node are cleared
                                <1>     ; ((AX = R1)) input
                                <1>     ;
                                <1>     ;    (Retro UNIX Prototype : 02/12/2012, UNIXCOPY.ASM)
                                <1>       ;    ((Modified registers: EDX, ECX, EBX, ESI, EDI, EBP))
                                <1>     ;
                                <1>     ; / r1 contains an i-number
                                <1>
                                <1>     ; TRDOS 386 (06/10/2016)
                                <1>     ;
                                <1>     ; INPUT:
                                <1>     ;       EAX = First Cluster
                                <1>     ;        DL = Logical DOS Drive Number
                                <1>     ;
                                <1>     ; OUTPUT:
                                <1>     ;       CF = 1 -> EBX = File Handle/Number/Index
                                <1>     ;       CF = 0 -> EBX = 0
                                <1>     ;
                                <1>     ; Modified Registers: EBX
                                <1>
                                <1>     xor     ebx, ebx
                                <1> anyi_0:
                                <1>     cmp     byte [ebx+OF_MODE], 0 ; 0 = empty entry
                                <1>     ja      short anyi_2 ; 1 (r), 2 (w) or 3 (r&w)
                                <1> anyi_1:
                                <1>     inc     bl
                                <1>     cmp     bl, OPENFILES ; max. count of open files
                                <1>     jb      short anyi_0
                                <1>     xor     eax, eax
                                <1>     retn
                                <1> anyi_2:
                                <1>     cmp     dl, [ebx+OF_DRIVE]
                                <1>     jne     short anyi_1
                                <1>     ;shl    bx, 2 ; *4 (dword offset)
                                <1>     shl     ebx, 2 ; 23/07/2022
                                <1>     cmp     eax, [ebx+OF_FCLUSTER]
                                <1>     je      short anyi_3
                                <1>     ;shr    bx, 2 ; /4 (byte offset)
                                <1>     shr     ebx, 2 ; 23/07/2022
                                <1>     jmp     short anyi_1
                                <1> anyi_3:
                                <1>     ;shr    bx, 2 ; /4 (bytes offset) (index)
                                <1>     shr     ebx, 2 ; 23/07/2022
                                <1>     stc
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel (v0.2) - SYS9.INC
                                <1> ; Last Modification: 09/12/2015
                                <1>
                                <1> syssleep:
                                <1>     ; 28/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 24/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 29/06/2015 - (Retro UNIX 386 v1)
                                <1>     ; 11/06/2014 - (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 feature only
                                <1>     ; (INPUT -> none)
                                <1>
                                <1>     ; Temporary - 24/07/2022
00010417 891D[6C8E0100]         <1>     mov     [u.r0], ebx
                                <1>     ;
0001041D 0FB61D[BD8E0100]       <1>     movzx   ebx, byte [u.uno] ; process number
00010424 8AA3[7F8D0100]         <1>     mov     ah, [ebx+p.ttyc-1] ; current/console tty
0001042A E8B4150000             <1>     call    sleep
                                <1>
                                <1>     ; 23/08/2024 - restore [u.usp]
                                <1>     ; swap changes [u.usp] to esp which points to
                                <1>     ; return of sleep
0001042F 8925[688E0100]         <1>     mov     [u.usp], esp
                                <1>             ; points to user's regs on top ofn system stack
                                <1>
                                <1>     ; 24/07/2022
00010435 FF05[6C8E0100]         <1>     inc     dword [u.r0] ; Temporary !
0001043B E94EC6FFFF             <1>     jmp     sysret
                                <1>
                                <1> _vp_clr:
```

```
                                <1>     ; Reset/Clear Video Page
                                <1>     ;
                                <1>     ; 24/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 30/06/2015 - (Retro UNIX 386 v1)
                                <1>     ; 21/05/2013 - 30/10/2013(Retro UNIX 8086 v1) (U0.ASM)
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 feature only !
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;   BH = video page number
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;   none
                                <1>     ; ((Modified registers: EAX, BH, ECX, EDX, ESI, EDI))
                                <1>     ;
                                <1>     ; 04/12/2013
00010440 28C0                   <1>     sub     al, al
                                <1>     ; al = 0 (clear video page)
                                <1>     ; bh = video page ; 13/05/2016
00010442 B407                   <1>     mov     ah, 07h
                                <1>     ; ah = 7 (attribute/color)
                                <1>     ;xor    cx, cx ; 0, left upper column (cl) & row (cl)
                                <1>     ;mov    dx, 184Fh ; right lower column & row (dl=24, dh=79)
                                <1>     ; 24/07/2022
00010444 31C9                   <1>     xor     ecx, ecx
00010446 BA4F180000             <1>     mov     edx, 184Fh
0001044B E8E11BFFFF             <1>     call    _scroll_up
                                <1>     ; bh = video page
                                <1>     ;xor    dx, dx ; 0 (cursor position)
                                <1>     ; 24/07/2022
00010450 31D2                   <1>     xor     edx, edx
00010452 E9201FFFFF             <1>     jmp     _set_cpos
                                <1>
                                <1> sysmsg:
                                <1>     ; 28/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 07/12/2020
                                <1>     ; 05/12/2020
                                <1>     ; 13/05/2016
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 01/07/2015 - 11/11/2015 (Retro UNIX 386 v1)
                                <1>     ; Print user-application message on user's console tty
                                <1>     ;
                                <1>     ; Input -> EBX = Message address
                                <1>     ;          ECX = Message length (max. 255)
                                <1>     ;          DL = Color (IBM PC Rombios color attributes)
                                <1>     ;
00010457 81F9FF000000           <1>     cmp     ecx, MAX_MSG_LEN ; 255
                                <1>     ;ja     sysret ; nothing to do with big message size
                                <1>     ; 23/07/2022
0001045D 7605                   <1>     jna     short sysmsg8
                                <1> sysmsg7:
0001045F E92AC6FFFF             <1>     jmp     sysret
                                <1> sysmsg8:    ; 23/07/2022
00010464 08C9                   <1>     or      cl, cl
                                <1>     ;jz     sysret
                                <1>     ; 23/07/2022
00010466 74F7                   <1>     jz      short sysmsg7
00010468 20D2                   <1>     and     dl, dl
0001046A 7502                   <1>     jnz     short sysmsg0
0001046C B207                   <1>     mov     dl, 07h ; default color
                                <1>             ; (black background, light gray character)
                                <1> sysmsg0:
0001046E 891D[908E0100]         <1>     mov     [u.base], ebx
00010474 8815[DF760100]         <1>     mov     [ccolor], dl ; color attributes
0001047A 89E5                   <1>     mov     ebp, esp ; save stack pointer
0001047C 31DB                   <1>     xor     ebx, ebx ; 0
0001047E 891D[988E0100]         <1>     mov     [u.nread], ebx ; 0
                                <1>     ;
00010484 381D[D28E0100]         <1>     cmp     [u.kcall], bl ; 0
0001048A 7770                   <1>     ja      short sysmsgk ; Temporary (01/07/2015)
                                <1>     ;
0001048C 890D[948E0100]         <1>     mov     [u.count], ecx
                                <1>     ;inc    ecx ; + 00h ; ASCIIZ
                                <1>     ;
                                <1>     ; 07/12/2020
                                <1>     ;add    ecx, 3
00010492 6683C103               <1>     add     cx, 3
00010496 80E1FC                 <1>     and     cl, ~3  ; not 3
                                <1>     ;
                                <1>     ;;;
                                <1>     ; 20/08/2024
                                <1>     ; for safety, against a possibility which if ecx+1
                                <1>     ; chars (without a 0 in ecx chars) overs stack frame
                                <1>     ; (for example 255 chars without 0, safe stack frame
                                <1>     ; size is 260.)
                                <1>     ; !!! 'sysmsg' will put a 0 at ecx+1 position.!!!
00010499 51                     <1>     push    ecx
                                <1>     ;;;
                                <1>     ;
0001049A 29CC                   <1>     sub     esp, ecx
0001049C 89E7                   <1>     mov     edi, esp
0001049E 89E6                   <1>     mov     esi, esp
000104A0 66891D[D08E0100]       <1>     mov     [u.pcount], bx ; reset page (phy. addr.) counter
                                <1>     ; 11/11/2015
000104A7 8A25[A08E0100]         <1>     mov     ah, [u.ttyp] ; recent open tty
                                <1>     ; 0 = none
000104AD FECC                   <1>     dec     ah
000104AF 790C                   <1>     jns     short sysmsg1
000104B1 8A1D[BD8E0100]         <1>     mov     bl, [u.uno] ; process number
000104B7 8AA3[7F8D0100]         <1>     mov     ah, [ebx+p.ttyc-1] ; user's (process's) console tty
                                <1> sysmsg1:
000104BD 8825[A28E0100]         <1>     mov     [u.ttyn], ah
                                <1> sysmsg2:
000104C3 E8F4050000             <1>     call    cpass
000104C8 7416                   <1>     jz      short sysmsg5
000104CA AA                     <1>     stosb
000104CB 20C0                   <1>     and     al, al
000104CD 75F4                   <1>     jnz     short sysmsg2
                                <1> sysmsg3:
000104CF 80FC07                 <1>     cmp     ah, 7 ; tty number
000104D2 7711                   <1>     ja      short sysmsg6 ; serial port
000104D4 E83E000000             <1>     call    print_cmsg ; 05/12/2020
                                <1> sysmsg4:
000104D9 89EC                   <1>     mov     esp, ebp ; restore stack pointer
000104DB E9AEC5FFFF             <1>     jmp     sysret
                                <1> sysmsg5:
000104E0 C60700                 <1>     mov     byte [edi], 0
000104E3 EBEA                   <1>     jmp     short sysmsg3
                                <1> sysmsg6:
000104E5 8A06                   <1>     mov     al, [esi]
000104E7 E8F7140000             <1>     call    sndc
000104EC 72EB                   <1>     jc      short sysmsg4
000104EE 803E00                 <1>     cmp     byte [esi], 0  ; 0 is stop character
```

```
000104F1 76E6                <1>     jna     short sysmsg4
000104F3 46                  <1>     inc     esi
000104F4 8A25[A28E0100]      <1>     mov     ah, [u.ttyn]
000104FA EBE9                <1>     jmp     short sysmsg6
                             <1>
                             <1> sysmsgk: ; Temporary (01/07/2015)
                             <1>     ; The message has been sent by Kernel (ASCIIZ string)
                             <1>     ; (ECX -character count- will not be considered)
000104FC 8B35[908E0100]      <1>     mov     esi, [u.base]
00010502 8A25[DE760100]      <1>     mov     ah, [ptty] ; present/current screen (video page)
00010508 8825[A28E0100]      <1>     mov     [u.ttyn], ah
0001050E C605[D28E0100]00    <1>     mov     byte [u.kcall], 0
00010515 EBB8                <1>     jmp     short sysmsg3
                             <1>
                             <1> print_cmsg:
                             <1>     ; 08/12/2020
                             <1>     ; 07/12/2020
                             <1>     ; 05/12/2020
                             <1>     ; 18/11/2017
                             <1>     ; 13/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 01/07/2015 (Retro UNIX 386 v1)
                             <1>     ;
                             <1>     ; print message (on user's console tty)
                             <1>     ;       with requested color
                             <1>     ;
                             <1>     ; INPUTS:
                             <1>     ;       esi = message address
                             <1>     ;       [u.ttyn] = tty number (0 to 7)
                             <1>     ;       [ccolor] = color attributes (IBM PC BIOS colors)
                             <1>     ;
                             <1>     ; Modified registers: eax, ebx, ecx, edx, esi, edi
                             <1>     ; (ebp must be preserved)
                             <1>
                             <1>     ;mov    bh, ah
00010517 8A3D[A28E0100]      <1>     mov     bh, [u.ttyn]
0001051D 8A1D[DF760100]      <1>     mov     bl, [ccolor] ; * ; 05/12/2020
                             <1>
                             <1>     ; 05/12/2020
00010523 803D[249D0100]00    <1>     cmp     byte [pmi32], 0 ; is vbios's 32 bit pmi enabled ?
0001052A 772E                <1>     ja      short pcmsg5 ; yes
                             <1> pcmsg1:
                             <1>     ; 08/12/2020
0001052C 8A1D[DF760100]      <1>     mov     bl, [ccolor] ; * (video.s 'u11'&'beep' change BL)
                             <1>
00010532 AC                  <1>     lodsb
00010533 20C0                <1>     and     al, al  ; 0
00010535 743A                <1>     jz      short pcmsg2
                             <1> pcmsg7:
00010537 56                  <1>     push    esi
                             <1>     ;mov    bl, [ccolor] ; * (video.s 'u11'&'beep' change BL)
                             <1>     ; 05/12/2020
                             <1>     ;;mov   bh, [u.ttyn]
                             <1>     ;call   _write_tty
                             <1>     ;pop    esi
                             <1>     ;jmp    short pcmsg1
                             <1> ;pcmsg2:
                             <1>     ;retn
                             <1>
                             <1>     ; 07/12/2020
00010538 803D[2E680000]03    <1>     cmp     byte [CRT_MODE], 3
0001053F 7708                <1>     ja      short pcmsg4
                             <1> pcmsg3:
00010541 E8AC1DFFFF          <1>     call    _write_tty_m3
00010546 5E                  <1>     pop     esi
00010547 EBE3                <1>     jmp     short pcmsg1
                             <1> pcmsg4:
00010549 803D[2E680000]07    <1>         cmp     byte [CRT_MODE], 7
00010550 76EF                <1>     jna     short pcmsg3
00010552 E8642AFFFF          <1>     call    vga_write_teletype
00010557 5E                  <1>     pop     esi
00010558 EBD2                <1>     jmp     short pcmsg1
                             <1> pcmsg5:
                             <1>     ; 07/12/2020
0001055A 803D[2E680000]07    <1>         cmp     byte [CRT_MODE], 7
00010561 76C9                <1>     jna     short pcmsg1
                             <1>
                             <1>     ; 05/12/2020
                             <1>     ; writing message by using
                             <1>     ; VESA VBE3 video bios protected mode interface
                             <1>
00010563 B40E                <1>     mov     ah, 0Eh
                             <1> pcmsg6:
00010565 AC                  <1>     lodsb
00010566 20C0                <1>     and     al, al  ; 0
00010568 7407                <1>     jz      short pcmsg2
                             <1>     ; bh = video page
                             <1>     ; ah = 0Eh
                             <1>     ; al = character
                             <1>     ; bl = color
0001056A E8B314FFFF          <1>     call    int10h_32bit_pmi
0001056F EBF4                <1>     jmp     short pcmsg6
                             <1> pcmsg2:
00010571 C3                  <1>     retn
                             <1>
                             <1> sysgeterr:
                             <1>     ; 09/12/2015
                             <1>     ; 21/09/2015 - (Retro UNIX 386 v1 feature only!)
                             <1>     ; Get last error number or page fault count
                             <1>     ; (for debugging)
                             <1>     ;
                             <1>     ; Input -> EBX = return type
                             <1>     ;         0 = last error code (which is in 'u.error')
                             <1>     ;         FFFFFFFFh = page fault count for running process
                             <1>     ;         FFFFFFFEh = total page fault count
                             <1>     ;         1 .. FFFFFFFDh = undefined
                             <1>     ;
                             <1>     ; Output -> EAX = last error number or page fault count
                             <1>     ;         (depending on EBX input)
                             <1>     ;
00010572 21DB                <1>     and     ebx, ebx
00010574 750B                <1>     jnz     short glerr_2
                             <1> glerr_0:
00010576 A1[D88E0100]        <1>     mov     eax, [u.error]
                             <1> glerr_1:
0001057B A3[6C8E0100]        <1>     mov     [u.r0], eax
00010580 C3                  <1>     retn
                             <1> glerr_2:
00010581 43                  <1>     inc     ebx ; FFFFFFFFh -> 0, FFFFFFFEh -> FFFFFFFFh
00010582 74FD                <1>     jz      short glerr_2 ; page fault count for process
00010584 43                  <1>     inc     ebx ; FFFFFFFFh -> 0
00010585 75EF                <1>     jnz     short glerr_0
00010587 A1[88900100]        <1>     mov     eax, [PF_Count] ; total page fault count
0001058C EBED                <1>         jmp     short glerr_1
                             <1> glerr_3:
```

```
0001058E A1[DC8E0100]        <1>     mov    eax, [u.pfcount]
00010593 EBE6                <1>     jmp    short glerr_1
                             <1>
                             <1> load_and_run_file:
                             <1>     ; 10/01/2026
                             <1>     ; 07/07/2025 - TRDOS 386 Kernel v2.0.10
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 18/11/2017
                             <1>     ; 22/01/2017
                             <1>     ; 04/01/2017 - 07/01/2017
                             <1>     ; 24/10/2016
                             <1>     ; 24/04/2016 - 02/05/2016 - 03/05/2016 - 06/05/2016
                             <1>     ; 23/04/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 23/10/2015 (Retro UNIX 386 v1, 'sysexec')
                             <1>     ; 23/06/2015 (Retro UNIX 386 v1 - Beginning)
                             <1>     ; 03/06/2013 - 06/12/2013 (Retro UNIX 8086 v1)
                             <1>     ; EAX = First Cluster number
                             <1>     ; EDX = File Size
                             <1>     ; ESI = Argument list address
                             <1>     ; [argc] = argument count
                             <1>     ; [u.nread] = argument list length
                             <1>     ; [esp] = return address to the caller (*)
                             <1>     ;
00010595 8935[6C8F0100]      <1>     mov    [argv], esi
0001059B 8915[788F0100]      <1>     mov    [i.size], edx
000105A1 A3[748F0100]        <1>     mov    [ii], eax
                             <1>
                             <1>     ;sti    ; 07/01/2017
                             <1>     ;mov    eax, [k_page_dir]
                             <1>     ;mov    [u.pgdir], eax
000105A6 31C0                <1>     xor    eax, eax ; clc ; *** ; 04/01/2017
                             <1>     ;mov    [u.r0], eax ; 0 ; 07/01/2017
                             <1>
                             <1>     ; 06/05/2016
                             <1>     ; Set 'sysexit' return order to MainProg
                             <1>     ;
000105A8 58                  <1>     pop    eax ; * 'loc_load_and_run_file_8:' address
                             <1>     ;; 22/01/2017
                             <1>     ;;cli ; 07/01/2017
000105A9 8B25[4C760100]      <1>     mov    esp, [tss.esp0]
                             <1>     ;
                             <1>     ; 'loc_load_run_file_8' address has
                             <1>     ; 'jmp loc_file_rw_restore_retn' instruction
                             <1>     ; 'loc_file_rw_restore_retn:' will return to
                             <1>     ; [mainprog_return_addr]
                             <1>     ; just after 'call command_interpreter'
                             <1>     ;
000105AF 68[986D0000]        <1>     push   _end_of_mainprog ; we must not return to here !
000105B4 FF35[34830100]      <1>     push   dword [mainprog_return_addr]
000105BA 89E5                <1>     mov    ebp, esp ; **
                             <1>     ;
000105BC 9C                  <1>     pushfd  ; EFLAGS      ; IRETD ; ***
000105BD 6A08                <1>     push   KCODE ; cs    ; IRETD
000105BF 50                  <1>     push   eax ; * (eip) ; IRETD
000105C0 8925[648E0100]      <1>     mov    [u.sp], esp
                             <1>     ;mov   byte [u.quant], time_count
000105C6 1E                  <1>     push   ds
000105C7 06                  <1>     push   es
000105C8 0FA0                <1>     push   fs
000105CA 0FA8                <1>     push   gs
                             <1>     ;mov   eax, [u.r0]
000105CC 29C0                <1>     sub    eax, eax
000105CE 60                  <1>     pushad
000105CF 68[8ECA0000]        <1>     push   sysret
                             <1>     ;push  sysrel1 ; 07/01/2017
000105D4 8925[688E0100]      <1>     mov    [u.usp], esp
                             <1>     ;
000105DA E8F8030000          <1>     call   wswap ; Save MainProg (process 1) 'u' structure
                             <1>                  ; and registers for return (from program)
000105DF 89EC                <1>     mov    esp, ebp ; **
                             <1>     ;;22/01/2017
                             <1>     ;;sti ; 07/01/2017
                             <1>     ; 23/07/2022
                             <1>     ;push  eax  ; * 'loc_load_and_run_file_8:' address
                             <1>     ;
                             <1>     ;;; 02/05/2016
                             <1>     ;;; Create a new process (parent: MainProg)
000105E1 31F6                <1>     xor    esi, esi
                             <1> cnpm_1: ; search p.stat table for unused process number
000105E3 46                  <1>     inc    esi
                             <1>     ; 10/01/2026
000105E4 8A86[9F8D0100]      <1>     mov    al, [esi+p.stat-1]
000105EA 3C00                <1>     cmp    al, 0
                             <1>     ;cmp   byte [esi+p.stat-1], 0 ; SFREE
                             <1>                          ; is process active, unused, dead
000105EC 762C                <1>     jna    short cnpm_2   ; it's unused so branch
                             <1>     ;;;;
                             <1>     ; 10/01/2026 - TRDOS 386 v2.0.10 (BugFix)
                             <1>     ; (if the parent proc's number = 1, the child's status = 3)
000105EE 3C03                <1>     cmp    al, 3 ; zombie ?
000105F0 751D                <1>     jne    short cnpm_3 ; al = 1
000105F2 D1E6                <1>     shl    esi, 1
000105F4 668B86[5E8D0100]    <1>     mov    ax, [esi+p.ppid-2] ; parent process's id
000105FB D1EE                <1>     shr    esi, 1
000105FD 663B05[408D0100]    <1>     cmp    ax, [p.pid] ; compare with process 1's process id
00010604 7509                <1>     jne    short cnpm_3 ; not MainProg (!?)
00010606 C686[9F8D0100]00    <1>     mov    byte [esi+p.stat-1], 0
0001060D EB0B                <1>     jmp    short cnpm_2
                             <1> cnpm_3:
                             <1>     ;;;;
0001060F 6683FE10            <1>     cmp    si, nproc      ; all processes checked
00010613 72CE                <1>     jb     short cnpm_1    ; no, branch back
                             <1> cnpm_panic:
00010615 E9FF67FFFF          <1>     jmp    panic
                             <1> cnpm_2:
0001061A A1[C48E0100]        <1>     mov    eax, [u.pgdir] ; page directory of MainProg
0001061F A3[C88E0100]        <1>     mov    [u.ppgdir], eax ; parent's page directory
00010624 E80152FFFF          <1>     call   allocate_page
                             <1>     ;jc    panic
                             <1>     ; 23/07/2022
00010629 72EA                <1>     jc     short cnpm_panic
                             <1>
                             <1>     ; EAX = UPAGE (user structure page) address
0001062B A3[C08E0100]        <1>     mov    [u.upage], eax ; memory page for 'user' struct (child)
00010630 89F7                <1>     mov    edi, esi
                             <1>     ;shl   di, 2
                             <1>     ; 23/07/2022
00010632 C1E702              <1>     shl    edi, 2
00010635 8987[AC8D0100]      <1>     mov    [edi+p.upage-4], eax ; memory page for 'user' struct
0001063B E85B52FFFF          <1>     call   clear_page ; 03/05/2016
                             <1>     ;;movzx eax, byte [p.ttyc] ; console tty (for MainProg)
                             <1>     ;sub   ax, ax ; 0
                             <1>     ; 23/07/2022
00010640 29C0                <1>     sub    eax, eax
```

```
00010642 668986[7F8D0100]    <1>     mov     [esi+p.ttyc-1], ax ; al - set child's console tty
                             <1>                                ; ah - reset child's wait channel
00010649 66A3[A08E0100]      <1>     mov    [u.ttyp], ax ; 0
                             <1>
0001064F 89F2                <1>     mov    edx, esi
00010651 8815[BD8E0100]      <1>     mov    [u.uno], dl ; child process number
00010657 FE86[9F8D0100]      <1>        inc     byte [esi+p.stat-1] ; 1, SRUN
                             <1>     ;shl   si, 1 ; multiply si by 2 to get index into p.pid table
                             <1>     ; 23/07/2022
0001065D D1E6                <1>     shl    esi, 1
0001065F 66FF05[548E0100]    <1>     inc    word [mpid] ; increment m.pid; get a new process name
                             <1>
                             <1>     ; 23/07/2022
                             <1>     ;mov   ax, [p.pid]  ; get process name of MainProg
                             <1>     ;mov   ax, 1
00010666 FEC0                <1>     inc    al ; eax = 1
00010668 668986[5E8D0100]    <1>     mov    [esi+p.ppid-2], ax ; put parent process name
                             <1>                               ; in parent process slot for child
0001066F A2[B68E0100]        <1>     mov    [u.pri], al ; 1       ; normal priority
                             <1>
                             <1>     ;dec   ax ; 0
                             <1>     ;mov   [u.ttyp], ax ; 0
                             <1>
00010674 66A1[548E0100]      <1>     mov    ax, [mpid]
0001067A 668986[3E8D0100]    <1>     mov    [esi+p.pid-2], ax ; put new process name
                             <1>                              ; in child process' name slot
                             <1>     ;;;
                             <1>     ; 07/07/2025
                             <1>     ;mov   eax, [ii]
                             <1>     ; 23/07/2022
                             <1>     ; Retro UNIX 386 v1, 'sysexec' (u2.s)
                             <1>     ;call  iopen
                             <1>     ; 06/06/2016
                             <1>     ;mov   byte [u.pri], 1 ; normal priority
                             <1>     ;
                             <1>     ;jmp   short sysexec_7 ; 02/05/2016
                             <1>     ; 23/07/2022
00010681 E921F9FFFF          <1>     jmp    sysexec_7
                             <1>
                             <1> ;   ; 02/05/2016
                             <1> ;   ;inc   byte [sysflg] ; 0FFh -> 0
                             <1> ;   ;mov   byte [sysflg], 0 ; 04/01/2017
                             <1> ;   movzx  ebx, byte [u.uno]
                             <1> ;   shl    bl, 1 ; 13/11/2017
                             <1> ;   cmp    word [ebx+p.ppid-2], 1 ; MainProg
                             <1> ;   ja     sysret0 ; 03/05/2016
                             <1> ;   push   sysret ; *
                             <1> ;   mov    [u.usp], esp
                             <1> ;   call   wswap ; save child process 'u' structure and
                             <1> ;          ; registers
                             <1> ;   add    dword [u.usp], 4 ; 03/05/2016
                             <1> ;sysexec_19: ; 02/05/2016
                             <1> ;   retn ; * 'sysret' ; byte [sysflg] -> 0FFh
                             <1>
                             <1> readi:
                             <1>     ; 09/08/2022
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 01/05/2016
                             <1>     ; 25/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 20/05/2015 - Retro UNIX 386 v1
                             <1>     ; 11/03/2013 - 31/07/2013 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; Reads from a file whose the first cluster number in EAX
                             <1>     ;
                             <1>     ; INPUTS ->
                             <1>     ;    EAX - First cluster number of the file
                             <1>     ;    u.count - byte count user desires
                             <1>     ;    u.base - points to user buffer
                             <1>     ;    u.fofp - points to dword with current file offset
                             <1>     ;    i.size - file size
                             <1>     ;    cdev - logical dos drive number of the file
                             <1>     ; OUTPUTS ->
                             <1>     ;    u.count - cleared
                             <1>     ;    u.nread - accumulates total bytes passed back
                             <1>     ;
                             <1>     ; ((EAX)) input/output
                             <1>     ; (Retro UNIX Prototype : 14/12/2012 - 01/03/2013, UNIXCOPY.ASM)
                             <1>       ; ((Modified registers: edx, ebx, ecx, esi, edi))
                             <1>
00010686 31D2                <1>     xor    edx, edx ; 0
00010688 8915[988E0100]      <1>     mov    [u.nread], edx ; 0
0001068E 668915[D08E0100]    <1>     mov    [u.pcount], dx ; 19/05/2015
00010695 3915[948E0100]      <1>     cmp    [u.count], edx ; 0
                             <1>     ;ja    short readi_1
                             <1>     ;retn
                             <1>     ; 09/08/2022
0001069B 765E                <1>     jna    short dskr_5 ; retn
                             <1> ;readi_1:
                             <1> dskr:
                             <1>     ; 01/05/2016
                             <1>     ; 25/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 24/05/2015 - 12/10/2015 (Retro UNIX 386 v1)
                             <1>     ; 26/04/2013 - 03/08/2013 (Retro UNIX 8086 v1)
                             <1> dskr_0:
0001069D 8B15[788F0100]      <1>         mov edx, [i.size]
000106A3 8B1D[808E0100]      <1>     mov    ebx, [u.fofp]
000106A9 2B13                <1>     sub    edx, [ebx]
000106AB 7647                <1>     jna    short dskr_4
                             <1>     ;
000106AD 50                  <1>     push   eax ; 01/05/2016
000106AE 3B15[948E0100]      <1>     cmp     edx, [u.count]
000106B4 7306                <1>     jnb    short dskr_1
000106B6 8915[948E0100]      <1>     mov    [u.count], edx
                             <1> dskr_1:
                             <1>     ; EAX = First Cluster
                             <1>     ; [Current_Drv] = Physical drive number
000106BC E83B000000          <1>     call   mget_r
                             <1>     ; NOTE: in 'mget_r', relevant sector will be read in buffer
                             <1>     ; if it is not already in buffer !
000106C1 BB[94900100]        <1>     mov    ebx, readi_buffer
000106C6 803D[D28E0100]00    <1>     cmp    byte [u.kcall], 0 ; the caller is 'namei' sign (=1)
000106CD 770F                <1>     ja     short dskr_3     ; zf=0 -> the caller is 'namei'
000106CF 66833D[D08E0100]00  <1>     cmp    word [u.pcount], 0
000106D7 7705                <1>     ja     short dskr_3
                             <1> dskr_2:
                             <1>     ; [u.base] = virtual address to transfer (as destination address)
000106D9 E869010000          <1>     call   trans_addr_w ; translate virtual address to physical (w)
                             <1> dskr_3:
                             <1>     ; EBX (r5) = system (I/O) buffer address -physical-
000106DE E8CC010000          <1>     call   sioreg
000106E3 87F7                <1>     xchg   esi, edi
                             <1>     ; EDI = file (user data) offset
                             <1>     ; ESI = sector (I/O) buffer offset
                             <1>     ; ECX = byte count
```

```
000106E5 F3A4                <1>     rep    movsb
                             <1>     ; eax = remain bytes in buffer
                             <1>         ;       (check if remain bytes in the buffer > [u.pcount])
000106E7 09C0                <1>     or     eax, eax
000106E9 75EE                <1>     jnz    short dskr_2 ; (page end before system buffer end!)
000106EB 58                  <1>     pop    eax  ; (first cluster number)
000106EC 390D[948E0100]      <1>     cmp    [u.count], ecx ; 0
000106F2 77A9                <1>     ja     short dskr_0
                             <1> dskr_4:
000106F4 C605[D28E0100]00    <1>     mov    byte [u.kcall], 0
                             <1> dskr_5:            ; 23/07/2022
000106FB C3                  <1>     retn
                             <1>
                             <1> mget_r:
                             <1>     ; 19/12/2025 - TRDOS 386 Kernel v2.0.10
                             <1>     ; 29/08/2023
                             <1>     ; 30/07/2022
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 24/10/2016
                             <1>     ; 22/10/2016
                             <1>     ; 12/10/2016
                             <1>     ; 29/04/2016
                             <1>     ; 25/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 03/06/2015 (Retro UNIX 386 v1, 'mget', u.5s)
                             <1>     ; 22/03/2013 - 31/07/2013 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; Get existing or (allocate) a new disk block for file
                             <1>     ;
                             <1>     ; INPUTS ->
                             <1>     ;    [u.fofp] = file offset pointer
                             <1>     ;    EAX = First Cluster
                             <1>     ;    [cdev] = Logical dos drive number
                             <1>     ;    ([u.off] = file offset)
                             <1>     ; OUTPUTS ->
                             <1>     ;    EAX = logical sector number
                             <1>     ;    ESI = Logical Dos Drive Description Table address
                             <1>     ;
                             <1>     ; Modified registers: EDX, EBX, ECX, ESI, EDI
                             <1>
000106FC 8B35[808E0100]      <1>     mov     esi, [u.fofp]
00010702 8B1E                <1>     mov    ebx, [esi] ; (u.off)
                             <1>
00010704 29C9                <1>     sub    ecx, ecx
00010706 8A2D[518E0100]      <1>     mov    ch, [cdev]
                             <1>
0001070C BE00010900          <1>     mov    esi, Logical_DOSDisks
00010711 01CE                <1>     add    esi, ecx
                             <1>
00010713 380D[E4820100]      <1>     cmp    [readi.valid], cl ; 0
00010719 7642                <1>     jna    short mget_r_0
                             <1>
0001071B 3A2D[E5820100]      <1>     cmp    ch, [readi.drv]
00010721 753A                <1>     jne    short mget_r_0
                             <1>
00010723 3B05[F8820100]      <1>     cmp    eax, [readi.fclust]
00010729 754E                <1>     jne    short mget_r_3
                             <1>
0001072B 89D8                <1>     mov    eax, ebx ; file offset
0001072D 668B0D[EC820100]    <1>     mov    cx, [readi.bpc]
00010734 41                  <1>     inc    ecx ; <= 65536
00010735 29D2                <1>     sub    edx, edx
00010737 F7F1                <1>     div    ecx
                             <1>
00010739 8B3D[F4820100]      <1>     mov    edi, [readi.c_index] ; cluster index
                             <1>
0001073F 39F8                <1>     cmp    eax, edi
00010741 7563                <1>         jne     short mget_r_4  ; (*)
                             <1>
                             <1>     ; edx = byte offset in cluster (<= 65535)
00010743 668915[EE820100]    <1>     mov    [readi.offset], dx
                             <1>     ; 23/07/2022
                             <1>     ;shr   dx, 9 ; / 512
0001074A C1EA09              <1>     shr    edx, 9
0001074D 8815[E7820100]      <1>     mov    [readi.s_index], dl ; sector index in cluster (0 to spc -1)
                             <1>
00010753 A1[F0820100]        <1>     mov    eax, [readi.cluster]  ; > 0 if [readi.valid] = 1
                             <1>     ; 19/12/2025
                             <1>     ;mov   edx, [readi.fs_index]
00010758 E989000000          <1>         jmp     mget_r_7
                             <1>
                             <1> mget_r_0:
0001075D 882D[E5820100]      <1>     mov    [readi.drv], ch ; physical drive number
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp    byte [esi+LD_FATType], 0
                             <1>     ja     short mget_r_1
                             <1>     ;mov   cl, [esi+LD_FS_BytesPerSec+1]
                             <1>     ;shr   cl, 1   ; 1 for 512 bytes, 4 for 2048 bytes
                             <1>     ; 19/12/2025
                             <1>     mov    cl, 1
                             <1>     jmp    short mget_r_2
                             <1> %endif
                             <1> mget_r_1:
00010763 8A4E13              <1>     mov    cl, [esi+LD_BPB+BPB_SecPerClust]
                             <1> mget_r_2:
00010766 880D[E6820100]      <1>     mov    [readi.spc], cl  ; sectors per cluster
                             <1>     ; NOTE: readi bytes per sector value is always 512 !
                             <1>     ; 23/07/2022
                             <1>     ;xor   ch, ch
                             <1>     ; 29/08/2023
0001076C C1E109              <1>     shl    ecx, 9
                             <1>     ;shl   cx, 9 ; * 512
                             <1>     ;dec   cx ; bytes per cluster - 1
                             <1>     ; 23/07/2022
0001076F 49                  <1>     dec    ecx
00010770 66890D[EC820100]    <1>     mov    [readi.bpc], cx
                             <1>     ;sub   cx, cx
                             <1>     ; 23/07/2022
00010777 29C9                <1>     sub    ecx, ecx
                             <1> mget_r_3:
00010779 A3[F8820100]        <1>     mov    [readi.fclust], eax ; first cluster (or FDT address)
0001077E 880D[E4820100]      <1>     mov    [readi.valid], cl ; 0
                             <1>     ;mov   [readi.s_index], cl ; 0
                             <1>     ;mov   [readi.offset], cx ; 0
00010784 890D[F4820100]      <1>     mov    [readi.c_index], ecx ; 0
0001078A 890D[F0820100]      <1>     mov    [readi.cluster], ecx ; 0
00010790 890D[E8820100]      <1>     mov    [readi.sector], ecx ; 0
                             <1>
00010796 89D8                <1>     mov    eax, ebx ; file offset
00010798 668B0D[EC820100]    <1>     mov    cx, [readi.bpc]
0001079F 41                  <1>     inc    ecx ; <= 65536
000107A0 29D2                <1>     sub    edx, edx
000107A2 F7F1                <1>     div    ecx
                             <1>     ;mov   edi, [readi.c_index] ; previous cluster index
```

```
000107A4 29FF                <1>     sub     edi, edi
                             <1> mget_r_4:
000107A6 A3[F4820100]        <1>     mov     [readi.c_index], eax ; cluster index
                             <1>     ; edx = byte offset in cluster (<= 65535)
000107AB 668915[EE820100]    <1>     mov     [readi.offset], dx
                             <1>     ; 23/07/2022
                             <1>     ;shr    dx, 9 ; / 512
000107B2 C1EA09              <1>     shr     edx, 9
000107B5 8815[E7820100]      <1>     mov     [readi.s_index], dl ; sector index in cluster (0 to spc -1)
                             <1>
000107BB 89C1                <1>     mov     ecx, eax ; current cluster index
000107BD A1[F8820100]        <1>     mov     eax, [readi.fclust]
000107C2 09C9                <1>     or      ecx, ecx ; cluster index
000107C4 741B                <1>     jz      short mget_r_6
                             <1>
000107C6 39CF                <1>     cmp     edi, ecx
000107C8 7710                <1>     ja      short mget_r_5 ; old cluster index is higher
000107CA 8B15[F0820100]      <1>     mov     edx, [readi.cluster]
000107D0 21D2                <1>     and     edx, edx
000107D2 7406                <1>     jz      short mget_r_5
                             <1>     ; valid 'readi' parameters (*)
000107D4 89D0                <1>     mov     eax, edx
000107D6 29F9                <1>     sub     ecx, edi
000107D8 740C                <1>     jz      short mget_r_7
                             <1> mget_r_5:
                             <1>     ; EAX = Beginning cluster
                             <1>     ; EDX = Sector index in disk/file section
                             <1>     ;       (Only for SINGLIX file system!)
                             <1>     ; ECX = Cluster sequence number after the beginning cluster
                             <1>     ; ESI = Logical DOS Drive Description Table address
000107DA E8DAC0FFFF          <1>     call    get_cluster_by_index
000107DF 7246                <1>     jc      short mget_r_err
                             <1>     ; EAX = Cluster number
                             <1> mget_r_6:
000107E1 A3[F0820100]        <1>     mov     [readi.cluster], eax ; FDT number for Singlix File System
                             <1> mget_r_7:
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     byte [esi+LD_FATType], 0
                             <1>     jna     short mget_r_12
                             <1> %endif
                             <1>     ;sub    eax, 2
                             <1>     ; 30/07/2022
000107E6 48                  <1>     dec     eax
000107E7 48                  <1>     dec     eax
000107E8 0FB615[E6820100]    <1>     movzx   edx, byte [readi.spc]
000107EF F7E2                <1>     mul     edx
                             <1>
000107F1 034668              <1>     add     eax, [esi+LD_DATABegin]
000107F4 8A15[E7820100]      <1>     mov     dl, [readi.s_index]
000107FA 01D0                <1>     add     eax, edx
                             <1> mget_r_8:
                             <1>     ; eax = logical sector number
000107FC 803D[E4820100]00    <1>     cmp     byte [readi.valid], 0
00010803 7608                <1>     jna     short mget_r_9
00010805 3B05[E8820100]      <1>     cmp     eax, [readi.sector]
0001080B 7435                <1>     je      short mget_r_11 ; sector is already in 'readi' buffer
                             <1> mget_r_9:
0001080D A3[E8820100]        <1>     mov     [readi.sector], eax
00010812 BB[94900100]        <1>     mov     ebx, readi_buffer ; buffer address
                             <1>     ;mov    ecx, 1
                             <1>     ; 30/07/2022
00010817 31C9                <1>     xor     ecx, ecx
00010819 FEC1                <1>     inc     cl
                             <1>     ; ecx = 1
                             <1>
                             <1>     ; 29/04/2016
                             <1>     ;xor    dl, dl
                             <1>
                             <1>     ; EAX = Logical sector number
                             <1>     ; ECX = Sector count
                             <1>     ; EBX = Buffer address
                             <1>     ; (EDX = 0)
                             <1>     ; ESI = Logical DOS drive description table address
                             <1>
0001081B E8D3110000          <1>     call    disk_read
00010820 7314                <1>     jnc     short mget_r_10
                             <1>
                             <1>     ; 22/10/2016 (15h -> 17)
00010822 B811000000          <1>     mov     eax, 17 ; Drive not ready or read error !
                             <1> mget_r_err:
00010827 A3[D88E0100]        <1>     mov     [u.error], eax
                             <1>     ; 12/10/2016
0001082C A3[6C8E0100]        <1>     mov     [u.r0], eax
00010831 E938C2FFFF          <1>     jmp     error
                             <1> mget_r_10:
00010836 C605[E4820100]01    <1>     mov     byte [readi.valid], 1 ; 24/10/2016
0001083D A1[E8820100]        <1>     mov     eax, [readi.sector]
                             <1> mget_r_11:
00010842 C3                  <1>     retn
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> mget_r_12:
                             <1>     ; EAX = FDT number
                             <1>     ; EDX = Sector index from FDT sector (0,1,2,3,4...)
                             <1>     inc     eax ; the first data sector in FS disk section
                             <1>     mov     [readi.fs_index], edx
                             <1>     add     eax, edx
                             <1>     jmp     short mget_r_8
                             <1> %endif
                             <1>
                             <1> trans_addr_r:
                             <1>     ; 12/10/2016
                             <1>     ; 02/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; Translate virtual address to physical address
                             <1>     ; for reading from user's memory space
                             <1>     ; 04/06/2015 - 18/10/2015 (Retro UNIX 386 v1)
                             <1>
00010843 31D2                <1>     xor     edx, edx ; 0 (read access sign)
00010845 EB04                <1>     jmp     short trans_addr_rw
                             <1>
                             <1> trans_addr_w:
                             <1>     ; 12/10/2016
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; Translate virtual address to physical address
                             <1>     ; for writing to user's memory space
                             <1>     ; 04/06/2015 - 18/10/2015 (Retro UNIX 386 v1)
                             <1>
00010847 29D2                <1>     sub     edx, edx
00010849 FEC2                <1>     inc     dl ; 1 (write access sign)
                             <1> trans_addr_rw:
0001084B 50                  <1>     push    eax
0001084C 53                  <1>     push    ebx
```

```
0001084D 52                  <1>     push    edx ; r/w sign (in DL)
                             <1>     ;
0001084E 8B1D[908E0100]      <1>     mov     ebx, [u.base]
00010854 E8B753FFFF          <1>     call    get_physical_addr ; get physical address
00010859 730F                <1>     jnc     short passc_0
0001085B A3[D88E0100]        <1>     mov     [u.error], eax
00010860 A3[6C8E0100]        <1>     mov     [u.r0], eax ; 12/10/2016
                             <1>     ;pop    edx
                             <1>     ;pop    ebx
                             <1>     ;pop    eax
00010865 E904C2FFFF          <1>     jmp     error
                             <1> passc_0:
0001086A F6C202              <1>     test    dl, PTE_A_WRITE ; writable page
0001086D 5A                  <1>     pop     edx
0001086E 751C                <1>     jnz     short passc_1
                             <1>
00010870 20D2                <1>     and     dl, dl
00010872 7418                <1>     jz      short passc_1
                             <1>     ; read only (duplicated) page -must be copied to a new page-
                             <1>     ; EBX = linear address
00010874 51                  <1>     push    ecx
00010875 E80753FFFF          <1>     call    copy_page
0001087A 59                  <1>     pop     ecx
0001087B 721E                <1>     jc      short passc_2
0001087D 50                  <1>     push    eax ; physical address of the new/allocated page
0001087E E88D53FFFF          <1>     call    add_to_swap_queue
00010883 58                  <1>     pop     eax
00010884 81E3FF0F0000        <1>     and     ebx, PAGE_OFF ; 0FFFh
                             <1>     ;mov    ecx, PAGE_SIZE
                             <1>     ;sub    ecx, ebx
0001088A 01D8                <1>     add     eax, ebx
                             <1> passc_1:
0001088C A3[CC8E0100]        <1>     mov     [u.pbase], eax ; physical address
00010891 66890D[D08E0100]    <1>     mov     [u.pcount], cx ; remain byte count in page (1-4096)
00010898 5B                  <1>     pop     ebx
00010899 58                  <1>     pop     eax
0001089A C3                  <1>     retn
                             <1> passc_2:
0001089B B804000000          <1>     mov     eax, ERR_MINOR_IM ; "Insufficient memory !" error
000108A0 A3[6C8E0100]        <1>     mov     [u.r0], eax ; 12/10/2016
000108A5 A3[D88E0100]        <1>     mov     [u.error], eax
                             <1>     ;pop    ebx
                             <1>     ;pop    eax
000108AA E9BFC1FFFF          <1>     jmp     error
                             <1>
                             <1> sioreg:
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 19/05/2015 - 25/07/2015 (Retro UNIX 386 v1)
                             <1>     ; 12/03/2013 - 22/07/2013 (Retro UNIX 8086 v1)
                             <1>     ; INPUTS ->
                             <1>     ;     EBX = system buffer (data) address (r5)
                             <1>     ;     [u.fofp] = pointer to file offset pointer
                             <1>     ;     [u.base] = virtual address of the user buffer
                             <1>     ;     [u.pbase] = physical address of the user buffer
                             <1>     ;     [u.count] = byte count
                             <1>     ;     [u.pcount] = byte count within page frame
                             <1>     ; OUTPUTS ->
                             <1>     ;     ESI = user data offset (r1)
                             <1>     ;     EDI = system (I/O) buffer offset (r2)
                             <1>     ;     ECX = byte count (r3)
                             <1>     ;     EAX = remain bytes after byte count within page frame
                             <1>     ;       (If EAX > 0, transfer will continue from the next page)
                             <1>         ;
                             <1>     ; ((Modified registers:  EDX))
                             <1>
000108AF 8B35[808E0100]      <1>         mov     esi, [u.fofp]
000108B5 8B3E                <1>         mov     edi, [esi]
000108B7 89F9                <1>     mov     ecx, edi
000108B9 81C900FEFFFF        <1>     or      ecx, 0FFFFFE00h
000108BF 81E7FF010000        <1>     and     edi, 1FFh
000108C5 01DF                <1>     add     edi, ebx ; EBX = system buffer (data) address
000108C7 F7D9                <1>     neg     ecx
000108C9 3B0D[948E0100]      <1>     cmp     ecx, [u.count]
000108CF 7606                <1>     jna     short sioreg_0
000108D1 8B0D[948E0100]      <1>     mov     ecx, [u.count]
                             <1> sioreg_0:
000108D7 803D[D28E0100]00    <1>     cmp     byte [u.kcall], 0
000108DE 7613                <1>     jna     short sioreg_1
                             <1>      ; the caller is 'mkdir' or 'namei'
000108E0 A1[908E0100]        <1>     mov     eax, [u.base]
000108E5 A3[CC8E0100]        <1>     mov     [u.pbase], eax ; physical address = virtual address
000108EA 66890D[D08E0100]    <1>     mov     word [u.pcount], cx ; remain bytes in buffer (1 sector)
000108F1 EB0B                <1>     jmp     short sioreg_2
                             <1> sioreg_1:
000108F3 0FB715[D08E0100]    <1>     movzx   edx, word [u.pcount]
000108FA 39D1                <1>     cmp     ecx, edx
000108FC 772A                <1>     ja      short sioreg_4 ; transfer count > [u.pcount]
                             <1> sioreg_2: ; 2:
000108FE 31C0                <1>     xor     eax, eax
                             <1> sioreg_3:
00010900 010D[988E0100]      <1>     add     [u.nread], ecx
00010906 290D[948E0100]      <1>     sub     [u.count], ecx
0001090C 010D[908E0100]      <1>     add     [u.base], ecx
00010912 010E                <1>         add         [esi], ecx
00010914 8B35[CC8E0100]      <1>     mov     esi, [u.pbase]
0001091A 66290D[D08E0100]    <1>     sub     [u.pcount], cx
00010921 010D[CC8E0100]      <1>     add     [u.pbase], ecx
00010927 C3                  <1>         retn
                             <1> sioreg_4:
                             <1>     ; transfer count > [u.pcount]
                             <1>     ; (ecx > edx)
00010928 89C8                <1>     mov     eax, ecx
0001092A 29D0                <1>     sub     eax, edx ; remain bytes for 1 sector (block) transfer
0001092C 89D1                <1>     mov     ecx, edx ; current transfer count = [u.pcount]
0001092E EBD0                <1>     jmp     short sioreg_3
                             <1>
                             <1> tswitch: ; Retro UNIX 386 v1
                             <1> tswap:
                             <1>     ; 16/01/2017
                             <1>     ; 21/05/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 10/05/2015 - 01/09/2015 (Retro UNIX 386 v1)
                             <1>     ; 14/04/2013 - 14/02/2014 (Retro UNIX 8086 v1)
                             <1>     ; time out swap, called when a user times out.
                             <1>     ; the user is put on the low priority queue.
                             <1>     ; This is done by making a link from the last user
                             <1>     ; on the low priority queue to him via a call to 'putlu'.
                             <1>     ; then he is swapped out.
                             <1>
                             <1>     ; TRDOS 386 (TRDOS v2.0) modification ->  ** 21/05/2016 **
                             <1>     ;     * when a high priority (event) process will be stopped
                             <1>     ;       (swapped out, swithched out/off), 'tswap/tswitch' will
                             <1>     ;       not add it to a run queue.
                             <1>     ;       /// What for: Process may be already in a run queue,
                             <1>     ;       it is unspeficied state because process might be started
```

```
                                <1>     ;       by a timer event which does not regard previous priority
                                <1>     ;       level and run queue of the process (for fast executing!).
                                <1>     ;       After the 'run for event', process will be sequenced
                                <1>     ;       to run by it's actual run queue. ///
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification ->
                                <1>     ;        swap (software task switch) is performed by changing
                                <1>     ;       user's page directory (u.pgdir) instead of segment change
                                <1>     ;       as in Retro UNIX 8086 v1.
                                <1>     ;
                                <1>     ; RETRO UNIX 8086 v1 modification ->
                                <1>     ;        'swap to disk' is replaced with 'change running segment'
                                <1>     ;       according to 8086 cpu (x86 real mode) architecture.
                                <1>     ;       pdp-11 was using 64KB uniform memory while IBM PC
                                <1>     ;       compatibles was using 1MB segmented memory
                                <1>     ;       in 8086/8088 times.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    u.uno - users process number
                                <1>     ;    runq+4 - lowest priority queue
                                <1>     ; OUTPUTS ->
                                <1>     ;    r0 - users process number
                                <1>     ;    r2 - lowest priority queue address
                                <1>     ;
                                <1>     ; ((AX = R0, BX = R2)) output
                                <1>     ; ((Modified registers: EDX, EBX, ECX, ESI, EDI))
                                <1>     ;
                                <1>
                                <1>     NOTE:
                                <1>     ;* [u.pri] priority level is specified by run queue which is process
                                <1>     ;  comes to run from.
                                <1>     ;* Initial [u.pri] is 1 ('normal/regular') for programs
                                <1>     ;  (which are launched by MainProg or 'sysexec'), it is changed
                                <1>     ;  to 2 ('high') by timer event, if program uses 'systimer' system call.
                                <1>     ;* Program (Process) also can change it's running priority
                                <1>     ;  from 1 to 0 or up to 2 by using 'syspri' system call; but,
                                <1>     ;  if program selects priority level 2 (high) for running, next time
                                <1>     ;  it is reduced to 1 (normal/regular) because 'syspri' adds this
                                <1>     ;  program to 'run for normal' queue while running duration is a bit
                                <1>     ;  protected from swap/switch out immediate, behalf of other high
                                <1>     ;  priority process in sequence. Program (with high priority) will not
                                <1>     ;  be swapped/switched out (by timer event) before it's time quantum
                                <1>     ;  will be elapsed, but, this will be temporary if program is not using
                                <1>     ;  timer event function.
                                <1>
                                <1>     ;For example:
                                <1>     ;If a process frequently gets a timer event, it runs at high priority
                                <1>     ;level but when it returns from running it returns to actual run queue,
                                <1>     ;not to 'run for event' queue again.
                                <1>     ;'tswap' will not change the sequence at return/stop(swap out) stage.
                                <1>     ;But if priority level not high (=2, 'run for event'), 'tswap/tswitch'
                                <1>     ;will add the stopping process to relevant run queue according to
                                <1>     ;[u.pri] priority level.
                                <1>
                                <1>     ; 16/01/2017
00010930 BB[5C8E0100]           <1>     mov     ebx, runq+2    ; 'runq_normal' ; normal/regular priority
                                <1>     ; 21/05/2016
                                <1>     ;cmp   byte [u.pri], 2       ; high priority (run for event) ?
                                <1>     ;jnb   short swap
                                <1>     ; 16/01/2017
                                <1>     ; (Normal and also high/event priority processes will be added to
                                <1>     ; normal priority run queue for ensuring circular running sequence!)
                                <1>     ; (Timer interrupt or 'syspri' system call may change priority and run
                                <1>     ; queue to high/event level.)
00010935 803D[B68E0100]00       <1>     cmp     byte [u.pri], 0
0001093C 7702                   <1>     ja      short tswap_1  ; normal priority run queue
                                <1>     ;
0001093E 43                     <1>     inc     ebx
0001093F 43                     <1>     inc     ebx            ; runq+4, 'runq_background', low priority
                                <1> tswap_1:
00010940 A0[BD8E0100]           <1>     mov     al, [u.uno]
                                <1>             ; movb u.uno,r1 / move users process number to r1
                                <1>             ; mov  $runq+4,r2
                                <1>                     ; / move lowest priority queue address to r2
                                <1>             ; ebx = run queue
00010945 E8FD000000             <1>     call    putlu
                                <1>             ; jsr r0,putlu / create link from last user on Q to
                                <1>                             ; / u.uno's user
                                <1>
                                <1> switch: ; Retro UNIX 386 v1
                                <1> swap:
                                <1>     ; 20/08/2024
                                <1>     ; 02/01/2017
                                <1>     ; 21/05/2016
                                <1>     ; 20/05/2016
                                <1>     ; 02/05/2016
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 10/05/2015 - 02/09/2015 (Retro UNIX 386 v1)
                                <1>     ; 14/04/2013 - 08/03/2014 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 'swap' is routine that controls the swapping of processes
                                <1>     ; in and out of core.
                                <1>     ;
                                <1>     ; TRDOS 386 (TRDOS v2.0) modification ->  ** 20/05/2016 **
                                <1>     ;     * 3 different priority level is applied
                                <1>     ;       (just as original unix v1)
                                <1>     ;       1) high priority (event) run queue, 'runq_event'
                                <1>     ;       2) normal priority (regular) run queue, 'runq_normal'
                                <1>     ;       3) low priority (background) run queue, 'runq_backgroud'
                                <1>     ;       'swap' code will run a process which has max. priority
                                <1>       ;       (for earliest event at first)
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification ->
                                <1>     ;        swap (software task switch) is performed by changing
                                <1>     ;       user's page directory (u.pgdir) instead of segment change
                                <1>     ;       as in Retro UNIX 8086 v1.
                                <1>     ;
                                <1>     ; RETRO UNIX 8086 v1 modification ->
                                <1>     ;        'swap to disk' is replaced with 'change running segment'
                                <1>     ;       according to 8086 cpu (x86 real mode) architecture.
                                <1>     ;       pdp-11 was using 64KB uniform memory while IBM PC
                                <1>     ;       compatibles was using 1MB segmented memory
                                <1>     ;       in 8086/8088 times.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    runq table - contains processes to run.
                                <1>     ;    p.link - contains next process in line to be run.
                                <1>     ;    u.uno - process number of process in core
                                <1>     ;    s.stack - swap stack used as an internal stack for swapping.
                                <1>     ; OUTPUTS ->
                                <1>     ;    (original unix v1 -> present process to its disk block)
                                <1>     ;    (original unix v1 -> new process into core ->
                                <1>     ;          Retro Unix 8086 v1 -> segment registers changed
                                <1>     ;          for new process)
```

```
                                <1>     ;    u.quant = 3 (Time quantum for a process)
                                <1>     ;      ((INT 1Ch count down speed -> 18.2 times per second)
                                <1>     ;    RETRO UNIX 8086 v1 will use INT 1Ch (18.2 times per second)
                                <1>     ;       for now, it will swap the process if there is not
                                <1>     ;       a keyboard event (keystroke) (Int 15h, function 4Fh)
                                <1>     ;       or will count down from 3 to 0 even if there is a
                                <1>     ;        keyboard event locking due to repetitive key strokes.
                                <1>     ;       u.quant will be reset to 3 for RETRO UNIX 8086 v1.
                                <1>     ;
                                <1>     ; ((Modified registers: EAX, EDX, EBX, ECX, ESI, EDI))
                                <1>
                                <1>     ;NOTE:
                                <1>     ;High priority queue is the first for selecting a process to run.
                                <1>     ;If there is not a process in high priority level run queue,
                                <1>     ;a process in normal priority run queue will be selected
                                <1>     ;or a proces in low priority run queue will be selected if normal
                                <1>     ;priority level run queue is empty.
                                <1>
                                <1>     ; 21/05/2016 -(3 priority levels, 3 run queues)
0001094A BE[5A8E0100]           <1>     mov    esi, runq ; 'runq_event' ; high priority, 'run for event'
0001094F C605[44830100]03       <1>     mov    byte [priority], 3 ; high priority + 1
00010956 31DB                   <1>     xor    ebx, ebx ; 02/01/2017
                                <1> swap_0: ; 1: / search runq table for highest priority process
00010958 66AD                   <1>     lodsw  ; mov ax, [esi], add esi+2
                                <1>     ;xor   ebx, ebx ; 02/05/2016
0001095A 6621C0                 <1>     and    ax, ax ; are there any processes to run in this Q entry
0001095D 750E                   <1>     jnz    short swap_2
                                <1>     ; 21/05/2026
                                <1>     ; runq_normal = runq+2, runq_background = runq+4
0001095F FE0D[44830100]         <1>     dec    byte [priority] ; 3 -> 3, 2 -> 1, 1-> 0
00010965 75F1                   <1>     jnz    short swap_0
                                <1>     ;cmp   esi, runq+6  ; if zero compare address to end of table
                                <1>     ;jb    short swap_0 ; if not at end, go back
                                <1> swap_1:
                                <1>     ; 02/05/2016
                                <1>     ; 29/04/2016 (TRDOS 386 = TRDOS v2.0)
                                <1>     ; No user process to run...
                                <1>     ; Run the kernel process... MainProg: Internal Command Interpreter
00010967 FEC0                   <1>     inc    al ; mov al, 1  ; process number of MainProg
00010969 FEC3                   <1>     inc    bl ; mov bl, al ; 1
0001096B EB1D                   <1>     jmp    short swap_4
                                <1> swap_2:
                                <1>     ; 21/05/2016
0001096D FE0D[44830100]         <1>     dec    byte [priority] ; priority level of present user/process
                                <1>                            ; 0, 1, 2
00010973 4E                     <1>     dec    esi
00010974 4E                     <1>         dec     esi
                                <1>     ;
00010975 88C3                   <1>     mov    bl, al
00010977 38E0                   <1>     cmp    al, ah ; is there only 1 process in the queue to be run
00010979 740A                   <1>     je     short swap_3 ; yes
0001097B 8AA3[8F8D0100]         <1>     mov    ah, [ebx+p.link-1]
00010981 8826                   <1>            mov    [esi], ah ; move next process in line into run queue
00010983 EB05                   <1>     jmp    short swap_4
                                <1> swap_3:
                                <1>     ;xor   dx, dx
                                <1>     ; 20/08/2024
00010985 31D2                   <1>     xor    edx, edx
00010987 668916                 <1>     mov    [esi], dx ; zero the entry; no processes on the Q
                                <1> swap_4:
0001098A 8A25[BD8E0100]         <1>     mov    ah, [u.uno]
00010990 38C4                   <1>     cmp    ah, al ; is this process the same as the process in core?
00010992 743B                   <1>            je     short swap_8 ; yes, don't have to swap
00010994 08E4                   <1>     or     ah, ah ; is the process # = 0
00010996 740D                   <1>            jz     short swap_6 ; 'sysexit'
                                <1>     ;cmp   ah, al ; is this process the same as the process in core?
                                <1>            ;je    short swap_8 ; yes, don't have to swap
00010998 8925[688E0100]         <1>     mov    [u.usp], esp ; return address for 'syswait' & 'sleep'
0001099E E834000000             <1>     call   wswap  ; write out core to disk
000109A3 EB1C                   <1>     jmp    short swap_7
                                <1> swap_6:
                                <1>     ; Deallocate memory pages belong to the process
                                <1>     ; which is being terminated.
                                <1>     ; (Retro UNIX 386 v1 modification !)
                                <1>     ;
000109A5 53                     <1>     push   ebx
000109A6 A1[C48E0100]           <1>     mov    eax, [u.pgdir]  ; page directory of the process
000109AB 8B1D[C88E0100]         <1>     mov    ebx, [u.ppgdir] ; page directory of the parent process
000109B1 E8A44FFFFF             <1>     call   deallocate_page_dir
000109B6 A1[C08E0100]           <1>     mov    eax, [u.upage] ; 'user' structure page of the process
000109BB E82950FFFF             <1>     call   deallocate_page
000109C0 5B                     <1>     pop    ebx
                                <1> swap_7:
000109C1 C0E302                 <1>     shl    bl, 2 ; * 4
                                <1>     ;;;
000109C4 8B83[AC8D0100]         <1>     mov    eax, [ebx+p.upage-4] ; the 'u' page of the new process
000109CA E840000000             <1>     call   rswap ; read new process into core
                                <1> swap_8:
                                <1>     ; Retro UNIX  8086 v1 modification !
000109CF C605[B48E0100]04       <1>     mov    byte [u.quant], time_count
000109D6 C3                     <1>     retn
                                <1>
                                <1> wswap:  ; < swap out, swap to disk >
                                <1>     ; 28/02/2017 (fnsave)
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 09/05/2015 (Retro UNIX 386 v1)
                                <1>     ; 26/05/2013 - 08/03/2014 (Retro UNIX 8086 v1)
                                <1>     ; 'wswap' writes out the process that is in core onto its
                                <1>     ; appropriate disk area.
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification ->
                                <1>     ;       User (u) structure content and the user's register content
                                <1>     ;      will be copied to the process's/user's UPAGE (a page for
                                <1>     ;      saving 'u' structure and user registers for task switching).
                                <1>     ;      u.usp - points to kernel stack address which contains
                                <1>     ;              user's registers while entering system call.
                                <1>     ;      u.sp  - points to kernel stack address
                                <1>     ;              to return from system call -for IRET-.
                                <1>     ;      [u.usp]+32+16 = [u.sp]
                                <1>     ;      [u.usp] -> edi, esi, ebp, esp (= [u.usp]+32), ebx,
                                <1>     ;              edx, ecx, eax, gs, fs, es, ds, -> [u.sp].
                                <1>     ;
                                <1>     ; Retro UNIX 8086 v1 modification ->
                                <1>     ;       'swap to disk' is replaced with 'change running segment'
                                <1>     ;      according to 8086 cpu (x86 real mode) architecture.
                                <1>     ;      pdp-11 was using 64KB uniform memory while IBM PC
                                <1>     ;      compatibles was using 1MB segmented memory
                                <1>     ;      in 8086/8088 times.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    u.break - points to end of program
                                <1>     ;    u.usp - stack pointer at the moment of swap
                                <1>     ;    core - beginning of process program
                                <1>     ;    ecore - end of core
```

```
                                <1>     ;    user - start of user parameter area
                                <1>     ;    u.uno - user process number
                                <1>     ;    p.dska - holds block number of process
                                <1>     ; OUTPUTS ->
                                <1>     ;    swp I/O queue
                                <1>     ;    p.break - negative word count of process
                                <1>     ;    r1 - process disk address
                                <1>     ;    r2 - negative word count
                                <1>     ;
                                <1>     ; RETRO UNIX 8086 v1 input/output:
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    u.uno - process number (to be swapped out)
                                <1>     ; OUTPUTS ->
                                <1>     ;    none
                                <1>     ;
                                <1>     ;   ((Modified registers: ECX, ESI, EDI))
                                <1>     ;
                                <1>
                                <1>     ; 28/02/2017
                                <1>     ;cmp   byte [multi_tasking], 0  ; Musti tasking mode ?
                                <1>     ;jna   short wswp
000109D7 803D[EB8E0100]00       <1>     cmp    byte [u.fpsave], 0 ; 28/02/2017
000109DE 7606                   <1>     jna    short wswp
000109E0 DD35[EC8E0100]         <1>     fnsave [u.fpregs] ; save floating point registers (94 bytes)
                                <1> wswp:
000109E6 8B3D[C08E0100]         <1>     mov    edi, [u.upage] ; process's user (u) structure page addr
000109EC B93D000000             <1>     mov    ecx, (U_SIZE + 3) / 4
000109F1 BE[648E0100]           <1>     mov    esi, user ; active user (u) structure
000109F6 F3A5                   <1>     rep    movsd
                                <1>     ;
000109F8 8B35[688E0100]         <1>     mov    esi, [u.usp] ; esp (system stack pointer,
                                <1>                         ;      points to user registers)
000109FE 8B0D[648E0100]         <1>     mov    ecx, [u.sp]  ; return address from the system call
                                <1>                         ; (for IRET)
                                <1>                         ; [u.sp] -> EIP (user)
                                <1>                         ; [u.sp+4]-> CS (user)
                                <1>                         ; [u.sp+8] -> EFLAGS (user)
                                <1>                         ; [u.sp+12] -> ESP (user)
                                <1>                         ; [u.sp+16] -> SS (user)
00010A04 29F1                   <1>     sub    ecx, esi     ; required space for user registers
00010A06 83C114                 <1>     add    ecx, 20      ; +5 dwords to return from system call
                                <1>                         ; (for IRET)
00010A09 C1E902                 <1>     shr    ecx, 2
00010A0C F3A5                   <1>     rep    movsd
00010A0E C3                     <1>     retn
                                <1>
                                <1> rswap:  ; < swap in, swap from disk >
                                <1>     ; 28/02/2017 (frstor)
                                <1>     ; 15/01/2017
                                <1>     ; 14/01/2017
                                <1>     ; 21/05/2016
                                <1>     ; 03/05/2016
                                <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 09/05/2015 - 15/09/2015 (Retro UNIX 386 v1)
                                <1>     ; 26/05/2013 - 08/03/2014 (Retro UNIX 8086 v1)
                                <1>     ; 'rswap' reads a process whose number is in r1,
                                <1>     ; from disk into core.
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 modification ->
                                <1>     ;       User (u) structure content and the user's register content
                                <1>     ;      will be restored from process's/user's UPAGE (a page for
                                <1>     ;      saving 'u' structure and user registers for task switching).
                                <1>     ;      u.usp - points to kernel stack address which contains
                                <1>     ;              user's registers while entering system call.
                                <1>     ;      u.sp  - points to kernel stack address
                                <1>     ;              to return from system call -for IRET-.
                                <1>     ;      [u.usp]+32+16 = [u.sp]
                                <1>     ;      [u.usp] -> edi, esi, ebp, esp (= [u.usp]+32), ebx,
                                <1>     ;              edx, ecx, eax, gs, fs, es, ds, -> [u.sp].
                                <1>     ;
                                <1>     ; RETRO UNIX 8086 v1 modification ->
                                <1>     ;       'swap to disk' is replaced with 'change running segment'
                                <1>     ;      according to 8086 cpu (x86 real mode) architecture.
                                <1>     ;      pdp-11 was using 64KB uniform memory while IBM PC
                                <1>     ;      compatibles was using 1MB segmented memory
                                <1>     ;      in 8086/8088 times.
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    r1 - process number of process to be read in
                                <1>     ;    p.break - negative of word count of process
                                <1>     ;    p.dska - disk address of the process
                                <1>     ;    u.emt - determines handling of emt's
                                <1>     ;    u.ilgins - determines handling of illegal instructions
                                <1>     ; OUTPUTS ->
                                <1>     ;    8 = (u.ilgins)
                                <1>     ;    24 = (u.emt)
                                <1>     ;    swp - bit 10 is set to indicate read
                                <1>     ;              (bit 15=0 when reading is done)
                                <1>     ;    swp+2 - disk block address
                                <1>     ;    swp+4 - negative word count
                                <1>     ;      ((swp+6 - address of user structure))
                                <1>     ;
                                <1>     ; RETRO UNIX 8086 v1 input/output:
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    AL - new process number (to be swapped in)
                                <1>     ; OUTPUTS ->
                                <1>     ;    none
                                <1>     ;
                                <1>     ;   ((Modified registers: EAX, ECX, ESI, EDI, ESP))
                                <1>     ;
                                <1>     ; Retro UNIX 386 v1 - modification ! 14/05/2015
00010A0F 89C6                   <1>     mov    esi, eax  ; process's user (u) structure page addr
00010A11 B93D000000             <1>     mov    ecx, (U_SIZE + 3) / 4
00010A16 BF[648E0100]           <1>     mov    edi, user ; active user (u) structure
00010A1B F3A5                   <1>     rep    movsd
00010A1D 58                     <1>     pop    eax    ; 'rswap' return address
                                <1>     ;
                                <1>     ;cli
00010A1E 8B3D[688E0100]         <1>     mov    edi, [u.usp] ; esp (system stack pointer,
                                <1>                         ;      points to user registers)
00010A24 89FC                   <1>     mov    esp, edi     ; 14/01/2017
00010A26 8B0D[648E0100]         <1>     mov    ecx, [u.sp]  ; return address from the system call
                                <1>                         ; (for IRET)
                                <1>                         ; [u.sp] -> EIP (user)
                                <1>                         ; [u.sp+4]-> CS (user)
                                <1>                         ; [u.sp+8] -> EFLAGS (user)
                                <1>                         ; [u.sp+12] -> ESP (user)
                                <1>                         ; [u.sp+16] -> SS (user)
00010A2C 29F9                   <1>     sub    ecx, edi     ; required space for user registers
00010A2E 83C114                 <1>     add    ecx, 20      ; +5 dwords to return from system call
                                <1>                         ; (for IRET)
00010A31 C1E902                 <1>     shr    ecx, 2
```

```
00010A34 F3A5                <1>     rep    movsd
                             <1>     ;mov   esp, [u.usp] ; 15/09/2015
                             <1>     ;sti
                             <1>     ; 28/02/2017
                             <1>     ;cmp   byte [multi_tasking], 0  ; Multi tasking mode ?
                             <1>     ;jna   short rswp_retn
00010A36 803D[EB8E0100]00    <1>     cmp    byte [u.fpsave], 0
00010A3D 7606                <1>     jna    short rswp_retn
00010A3F DD25[EC8E0100]      <1>     frstor [u.fpregs] ; restore floating point regs (94 bytes)
                             <1>                       ; 108 bytes (22/08/2024)
                             <1> rswp_retn:
00010A45 50                  <1>     push   eax    ; 'rswap' return address
00010A46 C3                  <1>     retn
                             <1>
                             <1> putlu:
                             <1>     ; 20/05/2016
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 10/05/2015 - 12/09/2015 (Retro UNIX 386 v1)
                             <1>     ; 15/04/2013 - 23/02/2014 (Retro UNIX 8086 v1)
                             <1>     ; 'putlu' is called with a process number in r1 and a pointer
                             <1>     ; to lowest priority Q (runq+4) in r2. A link is created from
                             <1>     ; the last process on the queue to process in r1 by putting
                             <1>     ; the process number in r1 into the last process's link.
                             <1>     ;
                             <1>     ; INPUTS ->
                             <1>     ;    r1 - user process number
                             <1>     ;    r2 - points to lowest priority queue
                             <1>     ;    p.dska - disk address of the process
                             <1>     ;    u.emt - determines handling of emt's
                             <1>     ;    u.ilgins - determines handling of illegal instructions
                             <1>     ; OUTPUTS ->
                             <1>     ;    r3 - process number of last process on the queue upon
                             <1>     ;         entering putlu
                             <1>     ;    p.link-1 + r3 - process number in r1
                             <1>     ;    r2 - points to lowest priority queue
                             <1>     ;
                             <1>     ; ((Modified registers: EDX, EBX))
                             <1>     ;
                             <1>     ; / r1 = user process no.; r2 points to lowest priority queue
                             <1>
                             <1>     ; EBX = r2
                             <1>     ; EAX = r1 (AL=r1b)
                             <1>
                             <1>     ; 20/05/2016
                             <1>     ; AL = process number (1 to 16) // Retro UNIX 8086, 386 v1 //
                             <1>     ;     (max. 16 processes available for current kernel version)
                             <1>     ; EBX = run queue address ; 20/05/2016 (TRDOS 386)
                             <1>             ; which is one of following addresses:
                             <1>             ;  1) 'runq_event' high priority run queue
                             <1>             ;  2) 'runq_normal' normal/regular priority run queue
                             <1>             ;  3) 'runq_background' low priority run queue
                             <1>
                             <1>     ;mov   ebx, runq
00010A47 0FB613              <1>     movzx  edx, byte [ebx]
00010A4A 43                  <1>     inc    ebx
00010A4B 20D2                <1>     and    dl, dl
                             <1>             ; tstb (r2)+ / is queue empty?
00010A4D 740A                <1>             jz     short putlu_1
                             <1>             ; beq 1f / yes, branch
00010A4F 8A13                <1>     mov    dl, [ebx] ; 12/09/2015
                             <1>             ; movb (r2),r3 / no, save the "last user" process number
                             <1>                            ; / in r3
00010A51 8882[8F8D0100]      <1>             mov    [edx+p.link-1], al
                             <1>             ; movb r1,p.link-1(r3) / put pointer to user on
                             <1>                            ; / "last users" link
00010A57 EB03                <1>     jmp    short putlu_2
                             <1>             ; br 2f /
                             <1> putlu_1: ; 1:
00010A59 8843FF              <1>     mov    [ebx-1], al
                             <1>                     ; movb r1,-1(r2) / user is only user;
                             <1>                         ; / put process no. at beginning and at end
                             <1> putlu_2: ; 2:
00010A5C 8803                <1>     mov    [ebx], al
                             <1>                     ; movb r1,(r2) / user process in r1 is now the last entry
                             <1>                            ; / on the queue
00010A5E 88C2                <1>     mov    dl, al
00010A60 88B2[8F8D0100]      <1>         mov     [edx+p.link-1], dh ; 0
                             <1>             ; dec r2 / restore r2
00010A66 C3                  <1>         retn
                             <1>             ; rts r0
                             <1>
                             <1> sysver:
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
00010A67 C705[6C8E0100]0002- <1>     mov    dword [u.r0], 200h ; AH = major version, AL = minor version
00010A6F 0000                <1>
00010A71 E918C0FFFF          <1>     jmp    sysret
                             <1>
                             <1> syspri: ; change running priority (of the process)
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 21/05/2016
                             <1>     ; 20/05/2026 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; INPUT ->
                             <1>     ;      BL = priority level
                             <1>     ;         0 = low running priority (running on background)
                             <1>     ;         1 = normal/regular priority (running as regular)
                             <1>     ;         2 = high/event priority (running for event)
                             <1>     ;         >2 = invalid, it will accepted as 2 (event)
                             <1>     ;         0FFh = get/return current running priority only
                             <1>     ; OUTPUT ->
                             <1>     ;      * if current [u.pri] < 2
                             <1>     ;        if BL input < 0FFh ->
                             <1>     ;           [u.pri] is updated as in BL input (0,1,2)
                             <1>     ;        if BL input = 0FFh -> AL = [u.pri] (current)
                             <1>     ;
                             <1>     ;      * if current [u.pri] = 2
                             <1>     ;        if BL input < 0FFh -> cf = 1 & AL = 2
                             <1>     ;        if BL input = 0FFh -> cf = 0 & AL = 2
                             <1>     ;
                             <1>     ;      NOTE:
                             <1>     ;      If [u.pri] = 2, it can not be changed to 1 or 0;
                             <1>     ;      because, run queue of the running process is unspecified
                             <1>     ;      at this stage. Process might be started by a timer event
                             <1>     ;      or priority might be changed to high by previous
                             <1>     ;      'syspri' system       call. In both cases, the process is in
                             <1>     ;      'runq_normal' or 'runq_background' queue.
                             <1>     ;      As result of this fact, when the [u.quant] time quantum
                             <1>     ;      of the process is elapsed or 'sysrele' system call is
                             <1>     ;      instructed by the process, 'tswap' ('tswitch') procedure
                             <1>     ;      will be called (to 'swap' or 'switch' out the procedure)
                             <1>     ;      and it will not call 'putlu' to add the (stopping)
                             <1>     ;      process to relevant run queue when [u.pri] = 2.
                             <1>     ;      (Otherwise, it would be possible to add process to
                             <1>     ;      a run queue while it is already in a run queue, wrongly.)
                             <1>     ;
```

```
                                <1>     ;       If [u.pri]< 2, 'tswap/tswitch' procedure will call
                                <1>     ;       'putlu' to add process to relevant run queue
                                <1>     ;       according to [u.pri] value. ('runq_normal' for 1,
                                <1>     ;       'runq_background' for 0).
                                <1>     ;
                                <1>     ;       If BL input >= 2 and < 0FFh while [u.pri] < 2,
                                <1>     ;       process will be added to 'runq_normal' queue and
                                <1>     ;       [u.pri] will be set to 2. (in 'syspri' system call)
                                <1>     ;
                                <1>
00010A76 29C0                   <1>     sub     eax, eax ; 0
00010A78 A3[D88E0100]           <1>     mov     [u.error], eax
                                <1>
00010A7D A0[B68E0100]           <1>     mov     al, [u.pri]
00010A82 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>
00010A87 FEC3                   <1>     inc     bl
                                <1>     ;jz     sysret ; 0FFh -> 0, get priority level
                                <1>     ; 23/07/2022
00010A89 742C                   <1>     jz      short syspri_2 ; jmp sysret
                                <1>
00010A8B 3C02                   <1>     cmp     al, 2
                                <1>     ;jnb    error ; CF = 1 & AL = 2 (& last error = 0)
                                <1>     ; 23/07/2022
00010A8D 7205                   <1>     jb      short syspri_0
00010A8F E9DABFFFFF             <1>     jmp     error
                                <1> syspri_0:
00010A94 FECB                   <1>     dec     bl
00010A96 80FB02                 <1>     cmp     bl, 2
00010A99 7602                   <1>     jna     short syspri_1
00010A9B B302                   <1>     mov     bl, 2
                                <1> syspri_1:
00010A9D 881D[B68E0100]         <1>     mov     [u.pri], bl
00010AA3 80FB02                 <1>     cmp     bl, 2
                                <1>     ;jb     sysret
                                <1>     ; 23/07/2022
00010AA6 720F                   <1>     jb      short syspri_2 ; jmp sysret
                                <1>
                                <1>     ; here...
                                <1>     ; Priority of current process has been changed to high
                                <1>     ; ('run for event') but current process will be added to
                                <1>     ; 'run as normal' queue. ('run for event' high priority
                                <1>     ; queue is under control of timer -& RTC- interrupt only!)
                                <1>     ;
                                <1>     ; (Otherwise, process can fall into black hole!
                                <1>     ; e.g. if it is not in waiting list and it has not got
                                <1>     ; a timer event and it is not in a run queue!
                                <1>     ; Because, when [u.pri] is 2, 'tswap/tswitch' will not
                                <1>     ; add the stopping process to a run queue.)
                                <1>
00010AA8 A0[BD8E0100]           <1>     mov     al, [u.uno]
00010AAD BB[5C8E0100]           <1>     mov     ebx, runq_normal ; normal priority !
                                <1>                              ; [u.pri] is set to high
                                <1>                              ; but 'runq_event' queue is set
                                <1>                              ; only by the kernel's timer
                                <1>                              ; event function (timer interrupt).
00010AB2 E890FFFFFF             <1>     call    putlu
                                <1> syspri_2:
00010AB7 E9D2BFFFFF             <1>     jmp     sysret
                                <1>
                                <1> cpass: ; / get next character from user area of core and put it in AL (r1)
                                <1>     ; 30/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 02/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 19/05/2015 - 18/10/2015 (Retro UNIX 386 v1)
                                <1>     ; 14/08/2013 - 20/09/2013 (Retro UNIX 8086 v1)
                                <1>     ; INPUTS ->
                                <1>     ;     [u.base] = virtual address in user area
                                <1>     ;     [u.count] = byte count (max.)
                                <1>     ;     [u.pcount] = byte count in page (0 = reset)
                                <1>     ; OUTPUTS ->
                                <1>     ;     AL = the character which is pointed by [u.base]
                                <1>     ;     zf = 1 -> transfer count has been completed
                                <1>         ;
                                <1>     ; ((Modified registers: EAX, EDX, ECX))
                                <1>
                                <1>     ; 30/07/2022
00010ABC 29C0                   <1>     sub     eax, eax
                                <1>
00010ABE 3905[948E0100]         <1>     cmp     [u.count], eax ; 0
                                <1>     ;cmp    dword [u.count], 0  ; have all the characters been transferred
                                <1>                             ; i.e., u.count, # of chars. left
00010AC4 763D                   <1>     jna     short cpass_3      ; to be transferred = 0?) yes, branch
00010AC6 FF0D[948E0100]         <1>     dec     dword [u.count]            ; no, decrement u.count
                                <1>         ; 19/05/2015
                                <1>     ;(Retro UNIX 386 v1 - translation from user's virtual address
                                <1>     ;                   to physical address
                                <1>     ; 30/07/2022
00010ACC 663905[D08E0100]       <1>     cmp     [u.pcount], ax ; 0
                                <1>     ;cmp    word [u.pcount], 0 ; byte count in page = 0 (initial value)
                                <1>                         ; 1-4095 --> use previous physical base address
                                <1>                         ; in [u.pbase]
00010AD3 770D                   <1>     ja      short cpass_1
                                <1>     ; 30/07/2022
00010AD5 3905[C88E0100]         <1>     cmp     [u.ppgdir], eax ; 0
                                <1>     ;cmp    dword [u.ppgdir], 0 ; is the caller os kernel
00010ADB 7427                   <1>         je      short cpass_k       ; (sysexec, '/etc/init') ?  (MainProg)
00010ADD E861FDFFFF             <1>     call    trans_addr_r
                                <1> cpass_1:
00010AE2 66FF0D[D08E0100]       <1>     dec     word [u.pcount]
                                <1> cpass_2:
00010AE9 8B15[CC8E0100]         <1>     mov     edx, [u.pbase]
00010AEF 8A02                   <1>     mov     al, [edx]      ; take the character pointed to
                                <1>                            ; by u.base and put it in r1
00010AF1 FF05[988E0100]         <1>     inc     dword [u.nread] ; increment no. of bytes transferred
00010AF7 FF05[908E0100]         <1>     inc     dword [u.base]  ; increment the buffer address to point to the
                                <1>                             ; next byte
00010AFD FF05[CC8E0100]         <1>     inc     dword [u.pbase]
                                <1> cpass_3:
00010B03 C3                     <1>     retn
                                <1> cpass_k:
                                <1>     ; 02/07/2015
                                <1>     ; The caller is os kernel
                                <1>     ; (get sysexec arguments from kernel's memory space)
00010B04 8B1D[908E0100]         <1>     mov     ebx, [u.base]
00010B0A 66C705[D08E0100]00-    <1>         mov     word [u.pcount], PAGE_SIZE ; 4096
00010B12 10                     <1>
00010B13 891D[CC8E0100]         <1>     mov     [u.pbase], ebx
00010B19 EBCE                   <1>     jmp     short cpass_2
                                <1>
                                <1> transfer_to_user_buffer: ; fast transfer
                                <1>     ; 27/05/2016
                                <1>     ; 16/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; INPUT ->
```

```
                                <1>     ;       ESI = source address in system space
                                <1>     ;       EDI = user's buffer address
                                <1>     ;       ECX = transfer (byte) count
                                <1>     ;       [u.pgdir] = user's page directory
                                <1>     ; OUTPUT ->
                                <1>     ;       ECX = actual transfer count
                                <1>     ;       cf = 1 -> error
                                <1>     ;       [u.count] = remain byte count
                                <1>     ;
                                <1>     ; Modified registers: eax, ecx
                                <1>     ;
                                <1>
00010B1B 21C9                   <1>     and     ecx, ecx
00010B1D 743B                   <1>     jz      short ttub_4
                                <1>
00010B1F 890D[948E0100]         <1>     mov     [u.count], ecx
                                <1>
00010B25 57                     <1>     push    edi
00010B26 56                     <1>     push    esi
00010B27 53                     <1>     push    ebx
00010B28 52                     <1>     push    edx
00010B29 51                     <1>     push    ecx
                                <1>
00010B2A 89FB                   <1>     mov     ebx, edi
00010B2C 81C300004000           <1>     add     ebx, CORE ; 27/05/2016
                                <1> ttub_1:
                                <1>     ; ebx = virtual (linear) address
                                <1>     ; [u.pgdir] = user's page directory
00010B32 E8DF50FFFF             <1>             call    get_physical_addr_x ; get physical address
00010B37 7222                   <1>     jc      short ttub_5
                                <1>     ; eax = physical address
                                <1>     ; ecx = remain byte count in page (1-4096)
00010B39 89C7                   <1>     mov     edi, eax
00010B3B A1[948E0100]           <1>     mov     eax, [u.count]
00010B40 39C1                   <1>     cmp     ecx, eax
00010B42 7602                   <1>     jna     short ttub_2
00010B44 89C1                   <1>     mov     ecx, eax
                                <1> ttub_2:
00010B46 29C8                   <1>     sub     eax, ecx
00010B48 01CB                   <1>     add     ebx, ecx
00010B4A F3A4                   <1>     rep     movsb
00010B4C A3[948E0100]           <1>     mov     [u.count], eax
00010B51 09C0                   <1>     or      eax, eax
00010B53 75DD                   <1>     jnz     short ttub_1
                                <1> ttub_retn:
                                <1> tfub_retn:
00010B55 59                     <1>     pop     ecx ; transfer count = actual transfer count
                                <1> ttub_3:
00010B56 5A                     <1>     pop     edx
00010B57 5B                     <1>     pop     ebx
00010B58 5E                     <1>     pop     esi
00010B59 5F                     <1>     pop     edi
                                <1> ttub_4:
00010B5A C3                     <1>     retn
                                <1> ttub_5:
00010B5B 59                     <1>     pop     ecx
00010B5C 2B0D[948E0100]         <1>     sub     ecx, [u.count] ; actual transfer count
00010B62 F9                     <1>     stc
00010B63 EBF1                   <1>     jmp     short ttub_3
                                <1>
                                <1> transfer_from_user_buffer: ; fast transfer
                                <1>     ; 27/05/2016
                                <1>     ; 16/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       ESI = user's buffer address
                                <1>     ;       EDI = destination address in system space
                                <1>     ;       ECX = transfer (byte) count
                                <1>     ;       [u.pgdir] = user's page directory
                                <1>     ; OUTPUT ->
                                <1>     ;       ecx = actual transfer count
                                <1>     ;       cf = 1 -> error
                                <1>     ;       [u.count] = remain byte count
                                <1>     ;
                                <1>     ; Modified registers: eax, ecx
                                <1>     ;
                                <1>
00010B65 21C9                   <1>     and     ecx, ecx
                                <1>     ;jz     short tfub_4
00010B67 74F1                   <1>     jz      short ttub_4
                                <1>
00010B69 890D[948E0100]         <1>     mov     [u.count], ecx
                                <1>
00010B6F 57                     <1>     push    edi
00010B70 56                     <1>     push    esi
00010B71 53                     <1>     push    ebx
00010B72 52                     <1>     push    edx
00010B73 51                     <1>     push    ecx
                                <1>
00010B74 89F3                   <1>     mov     ebx, esi
00010B76 81C300004000           <1>     add     ebx, CORE ; 27/05/2016
                                <1> tfub_1:
                                <1>     ; ebx = virtual (linear) address
                                <1>     ; [u.pgdir] = user's page directory
00010B7C E89550FFFF             <1>             call    get_physical_addr_x ; get physical address
                                <1>     ;jc     short tfub_5
00010B81 72D8                   <1>     jc      short ttub_5
                                <1>     ; eax = physical address
                                <1>     ; ecx = remain byte count in page (1-4096)
00010B83 89C6                   <1>     mov     esi, eax
00010B85 A1[948E0100]           <1>     mov     eax, [u.count]
00010B8A 39C1                   <1>     cmp     ecx, eax
00010B8C 7602                   <1>     jna     short tfub_2
00010B8E 89C1                   <1>     mov     ecx, eax
                                <1> tfub_2:
00010B90 29C8                   <1>     sub     eax, ecx
00010B92 01CB                   <1>     add     ebx, ecx
00010B94 F3A4                   <1>     rep     movsb
00010B96 A3[948E0100]           <1>     mov     [u.count], eax
00010B9B 09C0                   <1>     or      eax, eax
00010B9D 75DD                   <1>     jnz     short tfub_1
                                <1>
00010B9F EBB4                   <1>     jmp     short tfub_retn
                                <1>
                                <1> ;tfub_retn:
                                <1> ;   pop     ecx ; transfer count = actual transfer count
                                <1> ;tfub_3:
                                <1> ;   pop     edx
                                <1> ;   pop     ebx
                                <1> ;   pop     esi
                                <1> ;   pop     edi
                                <1> ;tfub_4:
                                <1> ;   retn
                                <1> ;tfub_5:
```

```
                                <1> ;   pop     ecx
                                <1> ;   sub     ecx, [u.count] ; actual transfer count
                                <1> ;   stc
                                <1> ;   jmp     short tfub_3
                                <1>
                                <1> sysfff: ; <Find First File>
                                <1>     ; 25/08/2024 (TRDOS 386 Kernel v2.0.9)
                                <1>     ; 08/08/2022
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 17/10/2016
                                <1>     ; 16/10/2016
                                <1>     ; 15/10/2016 TRDOS 386 (TRDOS v2.0) feature only !
                                <1>     ;           -derived from TRDOS v1.0, INT_21H.ASM-
                                <1>     ;            ("loc_INT21h_find_first_file")
                                <1>     ; TRDOS 8086 (v1.0)
                                <1>         ;   07/08/2011
                                <1>         ;   Find First File
                                <1>     ;       INPUT:
                                <1>         ;       CX= Attributes
                                <1>         ;       DS:DX= Pointer to filename
                                <1>     ;       MSDOS OUTPUT:
                                <1>     ;           DTA: (Default address: PSP offset 80h)
                                <1>     ;           Offset  Descrription
                                <1>     ;           0       Reserved for use find next file
                                <1>     ;           21      Attribute of file found
                                <1>     ;           22      Time stamp of file
                                <1>     ;           24      Date stamp of file
                                <1>     ;           26      File size in bytes
                                <1>     ;           30      Filename and extension (zero terminated)
                                <1>     ;       If cf = 1:
                                <1>     ;           Error Codes: (in AX)
                                <1>     ;               2 - File not found
                                <1>     ;              18 - No more files
                                <1>         ;
                                <1>     ; TRDOS 386 (v2.0)
                                <1>     ; 15/10/2016
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;       CL = File attributes
                                <1>     ;              bit 0 (1) - Read only file (R)
                                <1>     ;              bit 1 (1) - Hidden file (H)
                                <1>         ;              bit 2 (1) - System file (R)
                                <1>     ;              bit 3 (1) - Volume label/name (V)
                                <1>         ;              bit 4 (1) - Subdirectory (D)
                                <1>     ;              bit 5 (1) - File has been archived (A)
                                <1>     ;           CH = 0 -> Return basic parameters (24 bytes)
                                <1>     ;           CH > 0 -> Return FindFile structure/table (128 bytes)
                                <1>         ;           EBX = Pointer to filename (ASCIIZ) -path-
                                <1>     ;           EDX = File parameters buffer address
                                <1>     ;               (buffer size = 24 bytes if CH input = 0)
                                <1>     ;               (buffer size = 128 bytes if CH input > 0)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;           EAX = 0 if CH input > 0
                                <1>     ;           EAX = First cluster number of file if CH input = 0
                                <1>     ;           EDX = File parameters table/structure address
                                <1>     ;           Basic Parameters:
                                <1>     ;               Offset  Description
                                <1>     ;               ------  ---------------
                                <1>     ;               0       File Attributes
                                <1>     ;               1       Ambiguous filename chars are used sign
                                <1>     ;                       (0 = filename fits exactly with request)
                                <1>     ;                       (>0 = ambiguous filename chars are used)
                                <1>     ;               2       Time stamp of file
                                <1>     ;               4       Date stamp of file
                                <1>     ;               6       File size in bytes
                                <1>     ;               10      Short Filename (ASCIIZ, max. 13 bytes)
                                <1>     ;               23      Longname Length (1-255) if existing
                                <1>     ;
                                <1>     ;            cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; TR-DOS FindFile (FFF) Structure (128 bytes):
                                <1>     ; 09/10/2011 (DIR.ASM) - 10/02/2016 (trdoskx.s)
                                <1>     ;
                                <1>     ; Offset        Parameter               Size
                                <1>     ; ------        ------------------      --------
                                <1>     ; 0             FindFile_Drv            1 byte
                                <1>     ; 1             FindFile_Directory      65 bytes
                                <1>     ; 66            FindFile_Name           13 bytes
                                <1>     ; 79            FindFile_LongNameEntryLength 1 byte
                                <1>     ;Above 80 bytes form
                                <1>     ;TR-DOS Source/Destination File FullName Format/Structure
                                <1>     ; 80            FindFile_AttributesMask 1 word
                                <1>     ; 82            FindFile_DirEntry       32 bytes (*)
                                <1>     ; 114           FindFile_DirFirstCluster 1 double word
                                <1>     ; 118           FindFile_DirCluster     1 double word
                                <1>     ; 122           FindFile_DirEntryNumber 1 word
                                <1>     ; 124           FindFile_MatchCounter 1 word
                                <1>     ; 126           FindFile_Reserved       1 word
                                <1>     ; (*) MS-DOS, FAT 12-16-32 classic directory entry (32 bytes)
                                <1>
                                <1>     ;mov    [u.namep], ebx
                                <1>     ; 16/10/2016
00010BA1 8915[68830100]         <1>     mov     [FFF_UBuffer], edx
00010BA7 66890D[6D830100]       <1>     mov     [FFF_Attrib], cx ; [FFF_RType] = ch
                                <1>                     ; Attributes in CL, return data type in CH
00010BAE 89DE                   <1>     mov     esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov    ax, 1
                                <1>     ;mov    [FFF_Valid], ah ; 0 ; reset ; 17/10/2016
                                <1>     ;call   set_working_path
00010BB0 E820110000             <1>     call    set_working_path_x ; 17/10/2016
00010BB5 731D                   <1>     jnc     short sysfff_0
                                <1>
00010BB7 21C0                   <1>     and     eax, eax  ; 0 -> Bad Path!
00010BB9 7505                   <1>     jnz     short sysfff_err
                                <1>
                                <1>     ; eax = 0
00010BBB B80C000000             <1>     mov     eax, ERR_DIR_NOT_FOUND ; Directory not found !
                                <1> sysfff_err:
00010BC0 A3[6C8E0100]           <1>     mov     [u.r0], eax
00010BC5 A3[D88E0100]           <1>     mov     [u.error], eax
00010BCA E8DB110000             <1>         call        reset_working_path
00010BCF E99ABEFFFF             <1>     jmp     error
                                <1>
                                <1> sysfff_0:
                                <1>     ;;;
                                <1>     ; 25/08/2024 (bugfix)
                                <1>     ;mov    al, [esp] ; ???
00010BD4 A0[6D830100]           <1>     mov     al, [FFF_Attrib]
                                <1>     ;;;
00010BD9 08C0                   <1>     or      al, al
```

```
00010BDB 7412                <1>     jz      short sysfff_2
00010BDD B410                <1>     mov     ah, 10h
00010BDF A808                <1>     test    al, 08h
00010BE1 7503                <1>     jnz     short sysfff_1
00010BE3 80CC08              <1>     or      ah, 08h
                             <1> sysfff_1:
00010BE6 2410                <1>     and     al, 10h ; Directory
00010BE8 7405                <1>     jz      short sysfff_2
00010BEA 80E408              <1>     and     ah, 08h
00010BED 30C0                <1>     xor     al, al ; When a directory is searched,
                             <1>                    ; filename will be returned even if
                             <1>                    ; it is not a directory!
                             <1>                    ; Because: (in order to prevent
                             <1>                    ; creating a dir with existing file name)
                             <1>                    ; Dir and file names must not be same!
                             <1>                    ; (return attribute must be checked)
                             <1> sysfff_2:
                             <1>     ; AX = Attributes mask
                             <1>             ; AL = AND mask (result must be equal to AL)
                             <1>             ; AH = Negative AND mask (result must be ZERO)
                             <1>     ; ESI = FindFile_Name address
                             <1>
00010BEF E8867DFFFF          <1>     call    find_first_file
00010BF4 72CA                <1>     jc      short sysfff_err ; eax = 2 (File not found !)
                             <1>
                             <1>     ; ESI = Directory Entry (FindFile_DirEntry) Location
                             <1>     ; EDI = Directory Buffer Directory Entry Location
                             <1>     ; EAX = File Size
                             <1>     ;  BL = Attributes of The File/Directory
                             <1>     ;  BH = Long Name Yes/No Status (>0 is YES)
                             <1>     ;  DX > 0 : Ambiguous filename chars are used
                             <1>
                             <1> sysfff_3:
                             <1>     ; 16/10/2016
00010BF6 668B0D[6D830100]    <1>     mov     cx, [FFF_Attrib]
                             <1>     ; Attribs in CL, return data type in CH
                             <1>
                             <1>     ;or     cl, cl
                             <1>     ;jz     short sysfff_4 ; 0 = No filter
00010BFD 80F1FF              <1>     xor     cl, 0FFh
                             <1>     ; cl = negative attributes ; 25/08/2024
00010C00 20D9                <1>     and     cl, bl
00010C02 7409                <1>     jz      short sysfff_4
                             <1>
                             <1>     ;mov    eax, 2 ; 'file not found !' error
                             <1>     ;jmp    short sysfff_err_1
                             <1>
                             <1>     ; 16/10/2016
00010C04 E81E7EFFFF          <1>     call    find_next_file
00010C09 72B5                <1>     jc      short sysfff_err ; eax = 12 (no more files !)
00010C0B EBE9                <1>     jmp     short sysfff_3
                             <1>
                             <1> sysfff_4:
00010C0D 20ED                <1>     and     ch, ch ; [FFF_RType]
                             <1>     ;jz     short sysfff_5
                             <1>     ; 25/08/2024
00010C0F 755A                <1>     jnz     short sysfff_7
                             <1>
                             <1>     ;mov    ecx, 128 ; ; transfer length
                             <1>     ; 30/07/2022
                             <1>     ;sub    ecx, ecx
                             <1>     ;mov    cl, 128
                             <1>     ;mov    [FFF_Valid], cl
                             <1>     ; 25/08/2024
                             <1>     ;mov    byte [FFF_Valid], 128 ; (*)
                             <1>     ;jmp    short sysfff_7 ; sysfnf_11
                             <1>
                             <1> sysfff_5:
                             <1>     ;;mov   esi, FindFile_DirEntry
                             <1>     ;mov    ecx, 24  ; transfer length
                             <1>     ; 30/07/2022
                             <1>     ;sub    ecx, ecx
                             <1>     ;mov    cl, 24
                             <1>     ;mov    [FFF_Valid], cl
00010C11 C605[6C830100]18    <1>     mov     byte [FFF_Valid], 24 ; (*)
                             <1> sysfnf_12:
00010C18 BF[38880100]        <1>     mov     edi, DTA ; FFF data transfer address
                             <1>     ;mov    al, [esi+DirEntry_Attr] ; 11
00010C1D 88D8                <1>     mov     al, bl ; File/Dir Attributes
00010C1F 887F17              <1>     mov     [edi+23], bh ; Longname length (0= none)
00010C22 AA                  <1>     stosb
00010C23 88D0                <1>     mov     al, dl ; DL is for '?'
00010C25 00F0                <1>     add     al, dh ; DH is for '*'
                             <1>     ; AL > 0 if ambiguous file name wildcards are used
00010C27 AA                  <1>     stosb
00010C28 8B4616              <1>     mov     eax, [esi+DirEntry_WrtTime] ; 22
00010C2B AB                  <1>         stosd      ; DirEntry_WrtTime & DirEntry_WrtDate
00010C2C 8B461C              <1>         mov eax, [esi+DirEntry_FileSize] ; 28
00010C2F AB                  <1>         stosd
00010C30 668B4614            <1>     mov     ax, [esi+DirEntry_FstClusHI] ; 20
                             <1>     ;shl    ax, 16
                             <1>     ; 23/07/2022 (BugFix)
00010C34 C1E010              <1>     shl     eax, 16
00010C37 668B461A            <1>     mov     ax, [esi+DirEntry_FstClusLO] ; 26
00010C3B A3[6C8E0100]        <1>     mov     [u.r0], eax ; First Cluster
                             <1>
                             <1>         ;mov       esi, FindFile_DirEntry
00010C40 E8A2110000          <1>     call    get_file_name
                             <1>
                             <1>     ; 25/08/2024
                             <1>     ; ecx <= 7 (from 'get_file_name')
                             <1>     ;mov    cl, [FFF_Valid] ; (*)
00010C45 BE[38880100]        <1>             mov     esi, DTA ; FFF data transfer address
                             <1>
                             <1> sysfff_6:
                             <1>     ; 25/08/2024
                             <1>     ;sub    ecx, ecx
00010C4A 8A0D[6C830100]      <1>     mov     cl, [FFF_Valid] ; (*) ecx <= 128
                             <1>
00010C50 8B3D[68830100]      <1>     mov     edi, [FFF_UBuffer] ; user's buffer address (edx)
00010C56 E8C0FEFFFF          <1>     call    transfer_to_user_buffer
                             <1>
00010C5B 890D[6C8E0100]      <1>     mov     [u.r0], ecx ; actual transfer count
00010C61 E844110000          <1>         call       reset_working_path
00010C66 E923BEFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysfff_7:
                             <1>     ; 25/08/2024
00010C6B C605[6C830100]80    <1>     mov     byte [FFF_Valid], 128 ; (*)
                             <1> sysfnf_11:
                             <1>     ; ecx = 128
                             <1>     ; 25/08/2024 (bugfix)
                             <1>     ; 08/08/2022
00010C72 BE[16800100]        <1>     mov     esi, FindFile_Drv
```

```
                                <1>     ; 25/08/2024
00010C77 29C9                   <1>     sub    ecx, ecx ; 0
00010C79 EBCF                   <1>     jmp    short sysfff_6
                                <1>
                                <1> sysfnf: ; <Find Next File>
                                <1>     ; 19/12/2025 (TRDOS 386 v2.0.10)
                                <1>     ; 25/08/2024 (TRDOS 386 Kernel v2.0.9)
                                <1>     ; 29/08/2023 (TRDOS 386 Kernel v2.0.6)
                                <1>     ; 08/08/2022
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 16/10/2016 TRDOS 386 (TRDOS v2.0) feature only !
                                <1>     ;           -derived from TRDOS v1.0, INT_21H.ASM-
                                <1>     ;            ("loc_INT21h_find_next_file")
                                <1>     ; TRDOS 8086 (v1.0)
                                <1>         ;   07/08/2011
                                <1>         ;   Find First File
                                <1>     ;       INPUT:
                                <1>         ;       none
                                <1>     ;       MSDOS OUTPUT:
                                <1>     ;           DTA: (Default address: PSP offset 80h)
                                <1>     ;           Offset  Descrription
                                <1>     ;           0      Reserved for use find next file
                                <1>     ;           21     Attribute of file found
                                <1>     ;           22     Time stamp of file
                                <1>     ;           24     Date stamp of file
                                <1>     ;           26     File size in bytes
                                <1>     ;           30     Filename and extension (zero terminated)
                                <1>     ;       If cf = 1:
                                <1>     ;           Error Codes: (in AX)
                                <1>     ;              18 - No more files
                                <1>         ;
                                <1>     ; TRDOS 386 (v2.0)
                                <1>     ; 16/10/2016
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>             ;         none
                                <1>     ; OUTPUT ->
                                <1>     ;         EAX = 0 if CH input of 'Find First File' > 0
                                <1>     ;         EAX = First cluster number of file
                                <1>     ;               if CH input of 'Find First File' = 0
                                <1>     ;         EDX = File parameters table/structure address
                                <1>     ;
                                <1>     ;          cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>
                                <1>     ; Note: If byte [FFF_Valid] = 0
                                <1>     ;       'sysfnf' will return with 'no more files' error.
                                <1>     ;       If byte [FFF_Valid] = 24
                                <1>     ;       'sysfnf' will return with 24 bytes basic parameters
                                <1>     ;       at the address which is in EDX.
                                <1>     ;       If byte [FFF_Valid] = 128
                                <1>     ;       'sysfnf' will return with 128 bytes Find File
                                <1>     ;       Structure/Table at the address which is in EDX.
                                <1>
00010C7B 803D[6C830100]00       <1>     cmp    byte [FFF_Valid], 0
00010C82 7713                   <1>     ja     short stsfnf_0
                                <1>     ; 'no more files !' error
                                <1>     ;mov   eax, ERR_NO_MORE_FILES ; 12
                                <1>     ; 30/07/2022
00010C84 29C0                   <1>     sub    eax, eax
00010C86 B00C                   <1>     mov    al, ERR_NO_MORE_FILES ; 12
00010C88 A3[6C8E0100]           <1>     mov    [u.r0], eax
00010C8D A3[D88E0100]           <1>     mov    [u.error], eax
00010C92 E9D7BDFFFF             <1>     jmp    error
                                <1>
                                <1> stsfnf_0:
                                <1>     ;cmp   byte [FFF_Valid], 128
                                <1>     ;je    short stsfnf_1
                                <1>     ;cmp   byte [FFF_Valid], 24
                                <1>     ;je    short stsfnf_1
                                <1>     ;mov   [FFF_Valid], 24 ; Default
                                <1> stsfnf_1:
00010C97 0FB61D[76770100]       <1>     movzx  ebx, byte [Current_Drv]
00010C9E 66891D[72830100]       <1>     mov    [SWP_DRV], bx
00010CA5 8A15[16800100]         <1>     mov    dl, [FindFile_Drv]
00010CAB 38DA                   <1>     cmp    dl, bl
00010CAD 7532                   <1>     jne    short stsfnf_2
00010CAF 86FB                   <1>     xchg   bh, bl
00010CB1 BE00010900             <1>     mov    esi, Logical_DOSDisks
00010CB6 01DE                   <1>     add    esi, ebx
00010CB8 EB34                   <1>     jmp    short sysfnf_3
                                <1>
                                <1> sysfnf_8:
00010CBA E886B4FFFF             <1>     call   load_FAT_sub_directory
00010CBF 7269                   <1>     jc     short sysfnf_err_1 ; read error (no FNF stop)
                                <1>
                                <1> sysfnf_9:
00010CC1 E8617DFFFF             <1>     call   find_next_file
00010CC6 7257                   <1>     jc     short sysfnf_5
                                <1>     ; 25/08/2024
                                <1>     ; esi = Directory Entry (FindFile_DirEntry) Location
                                <1>     ;; 08/08/2022
                                <1>     ;; esi = FindFile_Drv ; wrong ! ; 25/08/2024
                                <1>
00010CC8 A0[6D830100]           <1>     mov    al, [FFF_Attrib]
                                <1>     ;or    al, al
                                <1>     ;jz    short sysfnf_10 ; 0 = No filter
00010CCD 34FF                   <1>     xor    al, 0FFh
00010CCF 20D8                   <1>     and    al, bl
00010CD1 75EE                   <1>     jnz    short sysfnf_9 ; search for next file until
                                <1>                           ; an error return from
                                <1>                           ; find_next_file procedure
                                <1> sysfnf_10:
                                <1>         ;movzx     ecx, byte [FFF_Valid]
                                <1>     ;cmp   cl, 128 ; complete FindFile structure/table
                                <1>     ;je    sysfnf_11
                                <1>     ;;cmp  cl, 24  ; basic parameters
                                <1>     ;;je   sysfnf_12
                                <1>     ;jmp   sysfnf_12
                                <1>     ; 30/07/2022
                                <1>     ;movzx ecx, byte [FFF_Valid]
                                <1>     ; 25/08/2024
                                <1>     ;cmp   cl, 128
00010CD3 803D[6C830100]80       <1>     cmp    byte [FFF_Valid], 128
                                <1>     ;jne   short sysfnf_12
                                <1>     ;jmp   short sysfnf_11 (*)
                                <1>     ;; 08/08/2022
                                <1>     ;;je   short sysfnf_6 ; esi = FindFile_Drv
                                <1>     ; 29/08/2023 (BugFix)
                                <1>     ;je    short sysfff_6 ; esi = FindFile_Drv
                                <1>     ; 25/08/2024 (BugFix of BugFix) (*)
00010CDA 7496                   <1>     je     short sysfnf_11 ; esi <> FindFile_Drv
                                <1>
```

```
00010CDC E937FFFFFF          <1>     jmp     sysfnf_12
                             <1>
                             <1> stsfnf_2:
00010CE1 FE05[73830100]      <1>     inc     byte [SWP_DRV_chg]
                             <1>
00010CE7 E8046AFFFF          <1>     call    change_current_drive
00010CEC 723C                <1>     jc      short sysfnf_err_1 ; read error !
                             <1>                                 ; (do not stop, because
                             <1>                                 ; we don't have a
                             <1>                                 ; 'no more files'
                             <1>                                 ; -file not found- error,
                             <1>                                 ; next sysfnf system call
                             <1>                                 ; may solve the problem,
                             <1>                                 ; after re-placing the disk)
                             <1> sysfnf_3:
00010CEE A1[8C800100]        <1>     mov     eax, [FindFile_DirCluster]
00010CF3 21C0                <1>     and     eax, eax
00010CF5 7547                <1>     jnz     short sysfnf_6
                             <1>
00010CF7 803D[75770100]02    <1>     cmp     byte [Current_FATType], 2
00010CFE 7723                <1>     ja      short sysfnf_err_0 ; invalid, we neeed to stop !?
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     byte [Current_FATType], 1
                             <1>     jb      short sysfnf_err_0 ; invalid, we neeed to stop !?
                             <1> %endif
                             <1>
00010D00 3805[9D7E0100]      <1>     cmp     byte [DirBuff_ValidData], al ; 0
00010D06 7608                <1>     jna     short sysfnf_4
                             <1>
00010D08 3B05[A27E0100]      <1>     cmp     eax, [DirBuff_Cluster] ; 0 ?
00010D0E 74B1                <1>     je      short sysfnf_9
                             <1>
                             <1>     ;cmp    byte [Current_Dir_Level], 0
                             <1>         ;ja short sysfnf_4
                             <1>         ;jna       short sysfnf_9
                             <1>
                             <1> sysfnf_4:
00010D10 FE05[73830100]      <1>     inc     byte [SWP_DRV_chg]
00010D16 E8ACB3FFFF          <1>     call    load_FAT_root_directory
00010D1B 73A4                <1>     jnc     short sysfnf_9
                             <1>     ; eax = error code (17, 'drv not ready or read error')
00010D1D EB0B                <1>     jmp     short sysfnf_err_1 ; read error ! (no FNF stop)
                             <1>                                 ; (if you want, try again,
                             <1>                                 ;  after re-placing the disk)
                             <1> sysfnf_5:
00010D1F 3C0C                <1>     cmp     al, 12 ; 'no more files' error
00010D21 7507                <1>     jne     short sysfnf_err_1 ; (no FNF stop -sysfnf will try
                             <1>                                 ;  to read the directory again,
                             <1>                                 ;  if the user calls sysfnf
                             <1>                                 ;  just after this error return-)
                             <1>     ; (FNF stop -sysfnf will not try
                             <1>     ;  to read the directory again-)
                             <1>
                             <1> sysfnf_err_0:
00010D23 C605[6C830100]00    <1>     mov     byte [FFF_Valid], 0 ; FNF stop sign
                             <1> sysfnf_err_1:
00010D2A A3[6C8E0100]        <1>     mov     [u.r0], eax
00010D2F A3[D88E0100]        <1>     mov     [u.error], eax
00010D34 E871100000          <1>     call    reset_working_path
00010D39 E930BDFFFF          <1>     jmp     error
                             <1>
                             <1> sysfnf_6:
00010D3E 803D[9D7E0100]00    <1>     cmp     byte [DirBuff_ValidData], 0
00010D45 760D                <1>     jna     short sysfnf_7
                             <1>
00010D47 3B05[A27E0100]      <1>     cmp     eax, [DirBuff_Cluster]
                             <1>     ;je     short sysfnf_9
                             <1>     ; 08/08/2022
00010D4D 7505                <1>     jne     short sysfnf_7
00010D4F E96DFFFFFF          <1>     jmp     sysfnf_9
                             <1> sysfnf_7:
00010D54 FE05[73830100]      <1>     inc     byte [SWP_DRV_chg]
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp     byte [Current_FATType], 1
                             <1>     ;jnb    short sysfnf_8
                             <1>     ; 08/08/2022
                             <1>     jb      short sysfnf_13
                             <1> %endif
00010D5A E95BFFFFFF          <1>     jmp     sysfnf_8
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1> sysfnf_13:
                             <1>     ; Singlix (TRFS) File System
                             <1>     ; (access via compatibility buffer)
                             <1>     call    load_FS_sub_directory
                             <1>     ;jnc    short sysfnf_9
                             <1>     ;jmp    short sysfnf_err_1 ; read error (no FNF stop)
                             <1>     ; 08/08/2022
                             <1>     jc      short sysfnf_err_1
                             <1>     jmp     sysfnf_9
                             <1> %endif
                             <1>
                             <1> writei:
                             <1>     ; 02/09/2024
                             <1>     ; 27/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; 08/08/2022
                             <1>     ; 30/07/2022
                             <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                             <1>     ; 26/10/2016
                             <1>     ; 25/10/2016
                             <1>     ; 23/10/2016
                             <1>     ; 22/10/2016
                             <1>     ; 19/10/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; 19/05/2015 - 20/05/2015 (Retro UNIX 386 v1)
                             <1>     ; 12/03/2013 - 31/07/2013 (Retro UNIX 8086 v1)
                             <1>     ;
                             <1>     ; Write data to file with first cluster number in EAX
                             <1>     ;
                             <1>     ; INPUTS ->
                             <1>     ;    EAX - First cluster number of the file
                             <1>     ;    EBX - File number (Open file index number)
                             <1>     ;    u.count - byte count to be written
                             <1>     ;    u.base - points to user buffer
                             <1>     ;    u.fofp - points to dword with current file offset
                             <1>     ;    i.size - file size
                             <1>     ;    cdev - logical dos drive number of the file
                             <1>     ; OUTPUTS ->
                             <1>     ;    u.count - cleared
                             <1>     ;    u.nread - accumulates total bytes passed back
```

```
                                <1>     ;    i.size - new file size (if file byte offset overs file size)
                                <1>     ;    u.fofp - points to u.off (with new offset value)
                                <1>     ;
                                <1>     ; (Retro UNIX Prototype : 11/11/2012 - 18/11/2012, UNIXCOPY.ASM)
                                <1>     ; ((Modified registers: eax, edx, ebx, ecx, esi, edi, ebp))
                                <1>
00010D5F 31C9                   <1>     xor    ecx, ecx
00010D61 890D[988E0100]         <1>     mov    [u.nread], ecx  ; 0
00010D67 66890D[D08E0100]       <1>     mov    [u.pcount], cx ; 19/05/2015
00010D6E 390D[948E0100]         <1>     cmp    [u.count], ecx
00010D74 7701                   <1>     ja     short writei_1 ; 08/08/2022
00010D76 C3                     <1>     retn
                                <1>     ; 23/07/2022
                                <1>     ;jna   short dskw_8 ; retn
                                <1> writei_1:
                                <1>     ; 02/09/2024
00010D77 C705[10830100]FFFF-    <1>     mov    dword [writei.fclust], 0FFFFFFFFh ; -1 ; reset
00010D7F FFFF                   <1>
                                <1> dskw:
00010D81 881D[20830100]         <1>     mov    [writei.ofn], bl ; Open file number
00010D87 880D[63830100]         <1>     mov    [setfmod], cl ; 0 ; reset 'update lm date&time' sign
                                <1> dskw_0:
                                <1>     ; 26/10/2016
                                <1>     ; 22/10/2016, 23/10/2016, 25/10/2016
                                <1>     ; 19/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 31/05/2015 - 25/07/2015 (Retro UNIX 386 v1)
                                <1>     ; 26/04/2013 - 20/09/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; 01/08/2013 (mkdir_w check)
00010D8D E8D0000000             <1>     call   mget_w
                                <1>     ; eax = sector/block number
                                <1>
00010D92 8B1D[808E0100]         <1>     mov     ebx, [u.fofp]
00010D98 8B13                   <1>     mov    edx, [ebx]
00010D9A 81E2FF010000           <1>     and    edx, 1FFh  ; / test the lower 9 bits of the file offset
00010DA0 750C                   <1>     jnz    short dskw_1 ; / if its non-zero, branch
                                <1>                         ; if zero, file offset = 0,
                                <1>                         ; / 512, 1024,...(i.e., start of new block)
00010DA2 813D[948E0100]0002-    <1>     cmp    dword [u.count], 512
00010DAA 0000                   <1>
                                <1>                           ; / if zero, is there enough data to fill
                                <1>                           ; / an entire block? (i.e., no. of
00010DAC 7331                   <1>     jnb    short dskw_2 ; / bytes to be written greater than 512.?
                                <1>                         ; / Yes, branch. Don't have to read block
                                <1> dskw_1: ; in as no past info. is to be saved
                                <1>     ; (the entire block will be overwritten).
                                <1>     ; 23/10/2016
                                <1>
00010DAE BB[9C920100]           <1>     mov    ebx, writei_buffer
                                <1>     ; esi = logical dos drive description table address
                                <1>     ; eax = sector number
                                <1>     ; ebx = buffer address (in kernel's memory space)
                                <1>     ; ecx = sector count
                                <1>     ; 30/07/2022
                                <1>     ;mov   ecx, 1
00010DB3 31C9                   <1>     xor    ecx, ecx
00010DB5 FEC1                   <1>     inc    cl
                                <1>     ; ecx = 1
00010DB7 E8370C0000             <1>     call   disk_read
                                <1>     ;call  dskrd   ; / no, must retain old info..
                                <1>                    ; / Hence, read block 'r1' into an I/O buffer
00010DBC 7321                   <1>     jnc    short dskw_2
                                <1>
                                <1>     ; disk read error
                                <1>     ; 30/07/2022
                                <1>     ;mov   eax, 17 ; drive not ready or READ ERROR !
00010DBE 29C0                   <1>     sub    eax, eax
00010DC0 B011                   <1>     mov    al, 17
                                <1> dskw_err: ; jump from disk write error
00010DC2 A3[6C8E0100]           <1>     mov    [u.r0], eax
00010DC7 A3[D88E0100]           <1>     mov    [u.error], eax
                                <1>
00010DCC 803D[63830100]00       <1>     cmp    byte [setfmod], 0
                                <1>     ;jna   error
                                <1>     ; 23/07/2022
00010DD3 7605                   <1>     jna    short writei_err
                                <1>
00010DD5 E887020000             <1>     call   update_file_lmdt ; update last modif. date&time of the file
                                <1>     ;mov   byte [setfmod], 0
                                <1> writei_err:
00010DDA E98FBCFFFF             <1>     jmp    error
                                <1>
                                <1> dskw_2: ; 3:
                                <1>     ; 23/10/2016
00010DDF C605[FC820100]01       <1>     mov    byte [writei.valid], 1 ; writei buffer contains valid data
00010DE6 56                     <1>     push   esi ; logical dos drive description table address
                                <1>     ; EAX (r1) = block/sector number
                                <1>     ;call  wslot
                                <1>            ; jsr r0,wslot / set write and inhibit bits in I/O queue,
                                <1>                         ; / proc. status=0, r5 points to 1st word of data
00010DE7 803D[D28E0100]00       <1>     cmp    byte [u.kcall], 0
00010DEE 770F                   <1>     ja     short dskw_4 ; zf=0 -> the caller is 'mkdir'
                                <1>     ;
00010DF0 66833D[D08E0100]00     <1>     cmp    word [u.pcount], 0
00010DF8 7705                   <1>     ja     short dskw_4
                                <1> dskw_3:
                                <1>     ; [u.base] = virtual address to transfer (as source address)
00010DFA E844FAFFFF             <1>     call   trans_addr_r ; translate virtual address to physical (r)
                                <1> dskw_4:
00010DFF BB[9C920100]           <1>     mov    ebx, writei_buffer
                                <1>     ; EBX (r5) = system (I/O) buffer address
00010E04 E8A6FAFFFF             <1>     call   sioreg
                                <1>     ; ESI = file (user data) offset
                                <1>     ; EDI = sector (I/O) buffer offset
                                <1>     ; ECX = byte count
                                <1>     ;
00010E09 F3A4                   <1>     rep    movsb
                                <1>     ; 25/07/2015
                                <1>     ; eax = remain bytes in buffer
                                <1>         ;       (check if remain bytes in the buffer > [u.pcount])
00010E0B 09C0                   <1>     or     eax, eax
00010E0D 75EB                   <1>     jnz    short dskw_3 ; (page end before system buffer end!)
                                <1>
                                <1>     ; 23/10/2016
00010E0F B101                   <1>     mov    cl, 1
00010E11 5E                     <1>     pop    esi
00010E12 A1[00830100]           <1>     mov    eax, [writei.sector]
                                <1>     ; esi = logical dos drive description table address
                                <1>     ; eax = sector number
                                <1>     ; ebx = writei buffer address
                                <1>     ; ecx = sector count
00010E17 E8C80B0000             <1>     call   disk_write ; / yes, write the block
00010E1C 7313                   <1>     jnc    short dskw_5
                                <1>
```

```
                                <1>     ;mov    eax, 18 ; drive not ready or WRITE ERROR !
                                <1>     ; 30/08/2022
00010E1E 29C0                   <1>     sub     eax, eax
00010E20 B012                   <1>     mov     al, 18
00010E22 EB9E                   <1>     jmp     short dskw_err
                                <1>
                                <1> dskw_7:
                                <1>     ; update last modif. date&time of the file
                                <1>     ; (also updates file size as OF_SIZE)
00010E24 E838020000             <1>     call    update_file_lmdt
                                <1>     ;mov    byte [setfmod], 0
                                <1>
                                <1>     ; 03/08/2013
00010E29 C605[D28E0100]00       <1>     mov     byte [u.kcall], 0
                                <1>     ; 23/10/2016
                                <1>     ;mov    eax, [writei.fclust]
                                <1> dskw_8:             ; 23/07/2022
00010E30 C3                     <1>     retn
                                <1>
                                <1> dskw_5:
                                <1>     ; 26/10/2016
00010E31 0FB61D[20830100]       <1>     movzx   ebx, byte [writei.ofn] ; open file number
00010E38 C0E302                 <1>     shl     bl, 2 ; *4
00010E3B 8B83[78840100]         <1>     mov     eax, [ebx+OF_POINTER]
00010E41 3B83[F8840100]         <1>     cmp     eax, [ebx+OF_SIZE]
00010E47 7606                   <1>     jna     short dskw_6
00010E49 8983[F8840100]         <1>     mov     [ebx+OF_SIZE], eax
                                <1> dskw_6:
                                <1>     ;shr    bl, 2
00010E4F 833D[948E0100]00       <1>         cmp     dword [u.count], 0 ; / any more data to write?
00010E56 76CC                   <1>     jna     short dskw_7
00010E58 A1[10830100]           <1>     mov     eax, [writei.fclust]
00010E5D E92BFFFFFF             <1>     jmp     dskw_0 ; / yes, branch
                                <1>
                                <1> mget_w:
                                <1>     ; 19/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 03/09/2024
                                <1>     ; 02/09/2024
                                <1>     ; 25/08/2024 - TRDOS 386 v2.0.9
                                <1>     ; 08/08/2022
                                <1>     ; 25/07/2022
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 02/11/2016
                                <1>     ; 01/11/2016
                                <1>     ; 23/10/2016, 31/10/2016
                                <1>     ; 22/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 03/06/2015 (Retro UNIX 386 v1, 'mget', u.5s)
                                <1>     ; 22/03/2013 - 31/07/2013 (Retro UNIX 8086 v1)
                                <1>     ;
                                <1>     ; Get existing or (allocate) a new disk block for file
                                <1>     ;
                                <1>     ; INPUTS ->
                                <1>     ;    [u.fofp] = file offset pointer
                                <1>     ;    [i.size] = file size
                                <1>     ;    [u.count] = byte count
                                <1>     ;    EAX = First cluster
                                <1>     ;    [cdev] = Logical dos drive number
                                <1>     ;    [writei.ofn] = File Number
                                <1>     ;                   (Open file index, 0 based)
                                <1>     ;    ([u.off] = file offset)
                                <1>     ; OUTPUTS ->
                                <1>     ;    EAX = logical sector number
                                <1>     ;    ESI = Logical Dos Drive Description Table address
                                <1>     ;
                                <1>     ; Modified registers: EDX, EBX, ECX, ESI, EDI, EBP
                                <1>
00010E62 8B35[808E0100]         <1>         mov     esi, [u.fofp]
00010E68 8B2E                   <1>     mov     ebp, [esi] ; u.off (or EBX*4+OF_POINTER)
                                <1>
00010E6A 29C9                   <1>     sub     ecx, ecx
00010E6C 8A2D[518E0100]         <1>     mov     ch, [cdev]
                                <1>
00010E72 BE00010900             <1>     mov     esi, Logical_DOSDisks
00010E77 01CE                   <1>     add     esi, ecx
                                <1>
                                <1>     ; 31/10/2016
00010E79 89C3                   <1>     mov     ebx, eax ; First Cluster or FDT address
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp     byte [esi+LD_FATType], 0
                                <1>     ;jna    mget_w_14 ; Singlix FS
                                <1>     ; 23/07/2022
                                <1>     ja      short mget_w_20
                                <1>     jmp     mget_w_14 ; Singlix FS
                                <1> %endif
                                <1>
                                <1> mget_w_20:
                                <1>     ;movzx  eax, word [esi+LD_BPB+BytesPerSec]
                                <1>     ; 19/12/2025
00010E7B B800020000             <1>     mov     eax, 512
00010E80 0FB65613               <1>     movzx   edx, byte [esi+LD_BPB+SecPerClust]
00010E84 8815[FE820100]         <1>     mov     [writei.spc], dl  ; sectors per cluster
00010E8A F7E2                   <1>     mul     edx
                                <1>     ; edx = 0
                                <1>     ; eax = bytes per cluster (<= 65536)
                                <1>
                                <1>     ; 02/11/2016
00010E8C 89C1                   <1>     mov     ecx, eax
00010E8E 48                     <1>     dec     eax
00010E8F 66A3[04830100]         <1>     mov     [writei.bpc], ax
                                <1>
00010E95 89E8                   <1>     mov     eax, ebp
00010E97 0305[948E0100]         <1>     add     eax, [u.count] ; next file position
00010E9D 3B05[788F0100]         <1>     cmp     eax, [i.size] ; <= file size ?
                                <1>     ;jna    mget_w_4 ; no
                                <1>     ; 23/07/2022
00010EA3 7705                   <1>     ja      short mget_w_21
                                <1>     ; 02/09/2024
                                <1>     ; ebx  = first cluster (input)
00010EA5 E925010000             <1>     jmp     mget_w_4
                                <1> mget_w_21:
00010EAA F7F1                   <1>     div     ecx
00010EAC A3[0C830100]           <1>     mov     [writei.c_index], eax ; cluster index
                                <1>     ; edx = byte offset in cluster (<= 65535)
                                <1>     ;mov    [writei.offset], dx
                                <1>     ;shr    dx, 9 ; / 512
                                <1>     ;mov    [writei.s_index], dl ; sector index in cluster (0 to spc -1)
                                <1>
00010EB1 29D2                   <1>     sub     edx, edx ; 01/11/2016
00010EB3 8915[00830100]         <1>     mov     [writei.sector], edx ; 0
00010EB9 668915[06830100]       <1>     mov     [writei.offset], dx  ; byte offset in cluster
00010EC0 8815[FF820100]         <1>     mov     [writei.s_index], dl ; sector index in cluster (0 to spc -1)
                                <1>
```

```
00010EC6 89D8                <1>     mov    eax, ebx ; First Cluster
                             <1>
                             <1>     ; is this the 1st mget_w or a next mget_w call ? (by 'writei')
00010EC8 3815[FC820100]      <1>     cmp    byte [writei.valid], dl ; 0
00010ECE 7624                <1>     jna    short mget_w_0
                             <1>
00010ED0 8815[FC820100]      <1>     mov    byte [writei.valid], dl ; 0 ; reset ('writei' will set it)
                             <1>
00010ED6 3B05[10830100]      <1>     cmp    eax, [writei.fclust]
00010EDC 7516                <1>     jne    short mget_w_0
                             <1>
00010EDE 8A0D[518E0100]      <1>     mov    cl, [cdev]
00010EE4 3A0D[FD820100]      <1>     cmp    cl, [writei.drv]
00010EEA 7508                <1>     jne    short mget_w_0
                             <1>     ; [writei.l_clust] & [writei.l_index] are valid,
                             <1>     ;  we don't need to get last cluster & last cluster index
00010EEC 8B0D[1C830100]      <1>     mov    ecx, [writei.l_index]
00010EF2 EB7F                <1>     jmp    short mget_w_2
                             <1> mget_w_0:
00010EF4 A3[10830100]        <1>     mov    [writei.fclust], eax ; first cluster
                             <1>     ; edx = 0
00010EF9 A3[08830100]        <1>     mov    [writei.cluster], eax ; first cluster ; 01/11/2016
00010EFE 8915[14830100]      <1>     mov    [writei.fs_index], edx ; 0 ; current cluster index
                             <1>
                             <1>     ;;;
                             <1>     ; 25/08/2024 - TRDOS 386 v2.0.9
00010F04 09C0                <1>     or     eax, eax ; is first cluster number = 0 ?
00010F06 751A                <1>     jnz    short mget_w_27 ; no
                             <1>
                             <1>     ; eax = 0 (*)
                             <1>     ; ((a directory entry with zero file size)) (*)
                             <1>     ; (*) ((('syswrite' may be used for writing to an empty file.
                             <1>     ; Before this modification, 'syscreat' and 'syswrite' system calls
                             <1>     ; did wrong things together... 'writetest.s' file, 24/08/2024.)))
                             <1>
00010F08 E8FFB8FFFF          <1>     call   add_new_cluster ; (*) 2024 (v2.0.9) modification
00010F0D 721A                <1>     jc     short mget_w_errj ; eax = error code
                             <1>     ; eax = (new) first cluster
                             <1>     ; skip 'get_last_cluster' and set ecx to -1
                             <1>     ;
                             <1>     ; 02/09/2024
00010F0F A3[10830100]        <1>     mov    [writei.fclust], eax ; first cluster
00010F14 A3[08830100]        <1>     mov    [writei.cluster], eax
                             <1>     ; 03/09/2024
00010F19 A3[64830100]        <1>     mov    [setfclust], eax
                             <1>     ;
00010F1E 31C9                <1>     xor    ecx, ecx
00010F20 EB12                <1>     jmp    short mget_w_28
                             <1>     ;;;
                             <1>
                             <1> mget_w_27:
                             <1>     ; FAT file system (FAT12, FAT16, FAT32)
00010F22 E8C8B7FFFF          <1>     call   get_last_cluster
                             <1>     ;jc    short mget_w_err ; eax = error code
                             <1>     ; 08/08/2022
00010F27 7305                <1>     jnc    short mget_w_25
                             <1> mget_w_errj:       ; 25/08/2024
00010F29 E992000000          <1>     jmp    mget_w_err
                             <1>
                             <1> mget_w_25:
                             <1>     ; 25/08/2024
00010F2E 8B0D[44810100]      <1>     mov    ecx, [glc_index] ; last cluster index
                             <1> mget_w_28:  ; 02/09/2024
00010F34 890D[1C830100]      <1>     mov    [writei.l_index], ecx
00010F3A A3[18830100]        <1>     mov    [writei.lclust], eax ; last cluster
                             <1>
00010F3F A0[20830100]        <1>     mov    al, [writei.ofn]
00010F44 FEC0                <1>     inc    al
00010F46 A2[63830100]        <1>     mov    [setfmod], al ; update lm date&time sign
                             <1>
                             <1> mget_w_1:
00010F4B 3B0D[0C830100]      <1>     cmp    ecx, [writei.c_index] ; last cluster index
00010F51 7320                <1>     jnb    short mget_w_2 ; 01/11/2016
                             <1>
00010F53 A1[18830100]        <1>     mov    eax, [writei.lclust]
                             <1>     ; EAX = Last cluster
00010F58 E8AFB8FFFF          <1>     call   add_new_cluster
00010F5D 7261                <1>     jc     short mget_w_err ; eax = error code
                             <1>     ; edx = 0
00010F5F A3[18830100]        <1>     mov    [writei.lclust], eax ; (new) last cluster
00010F64 8B0D[1C830100]      <1>     mov    ecx, [writei.l_index]
00010F6A 41                  <1>     inc    ecx ; add 1 to last cluster index
00010F6B 890D[1C830100]      <1>     mov    [writei.l_index], ecx ; current last cluster index
                             <1>
00010F71 EBD8                <1>     jmp    short mget_w_1
                             <1>
                             <1> mget_w_2:
00010F73 89E9                <1>     mov    ecx, ebp
00010F75 030D[948E0100]      <1>     add    ecx, [u.count]
00010F7B 890D[788F0100]      <1>     mov    [i.size], ecx ; save new file size
                             <1>     ;sub   edx, edx ; 0
                             <1>
00010F81 A0[518E0100]        <1>     mov    al, [cdev]
00010F86 A2[FD820100]        <1>     mov    [writei.drv], al ; physical drive number
                             <1>     ; edx = 0
00010F8B 89E8                <1>     mov    eax, ebp ; file offset
00010F8D 0FB70D[04830100]    <1>     movzx  ecx, word [writei.bpc] ; bytes per cluster - 1
00010F94 41                  <1>     inc    ecx ; bytes per cluster
00010F95 F7F1                <1>     div    ecx
                             <1>     ; edx = byte offset in cluster (<= 65535)
                             <1>     ; eax = cluster index
00010F97 A3[0C830100]        <1>     mov    [writei.c_index], eax
00010F9C 668915[06830100]    <1>     mov    [writei.offset], dx
                             <1>     ;shr   dx, 9 ; / 512
                             <1>     ; 23/07/2022
00010FA3 C1EA09              <1>     shr    edx, 9
00010FA6 8815[FF820100]      <1>     mov    [writei.s_index], dl ; sector index in cluster (0 to spc -1)
                             <1>
                             <1> mget_w_3:
00010FAC 3B05[1C830100]      <1>     cmp    eax, [writei.l_index] ; last cluster index
00010FB2 753E                <1>     jne    short mget_w_5
                             <1>
00010FB4 A3[14830100]        <1>     mov    [writei.fs_index], eax ; cluster index (for next check)
                             <1>
00010FB9 A1[18830100]        <1>     mov    eax, [writei.lclust] ; last cluster
00010FBE EB74                <1>     jmp    short mget_w_10
                             <1>
                             <1> mget_w_err:
00010FC0 A3[D88E0100]        <1>     mov    [u.error], eax
00010FC5 A3[6C8E0100]        <1>     mov    [u.r0], eax
00010FCA E99FBAFFFF          <1>     jmp    error
                             <1>
                             <1> mget_w_4: ; 02/11/2016
                             <1>     ; eax = next file position
```

```
00010FCF 2B05[948E0100]      <1>     sub    eax, [u.count] ; current file position
                             <1>     ; edx = 0
                             <1>     ; ecx = bytes per cluster
00010FD5 F7F1                <1>     div    ecx
00010FD7 A3[0C830100]        <1>     mov    [writei.c_index], eax ; cluster index
00010FDC 668915[06830100]    <1>     mov    [writei.offset], dx
                             <1>     ;shr   dx, 9 ; / 512
                             <1>     ; 23/07/2022
00010FE3 C1EA09              <1>     shr    edx, 9
00010FE6 8815[FF820100]      <1>     mov    [writei.s_index], dl ; sector index in cluster (0 to spc -1)
                             <1>
                             <1>     ; 02/09/2024
00010FEC 891D[10830100]      <1>     mov    [writei.fclust], ebx
                             <1>
                             <1> mget_w_5:
00010FF2 21C0                <1>     and    eax, eax ; 0 = First Cluster's index number
00010FF4 750C                <1>     jnz    short mget_w_6
                             <1>
00010FF6 A3[14830100]        <1>     mov    [writei.fs_index], eax ; cluster index (for next check)
                             <1>
00010FFB A1[10830100]        <1>     mov    eax, [writei.fclust] ; first cluster
00011000 EB32                <1>     jmp    short mget_w_10
                             <1>
                             <1> mget_w_6:
00011002 3B05[14830100]      <1>     cmp    eax, [writei.fs_index] ; current cluster index (>0)
00011008 7507                <1>     jne    short mget_w_7
0001100A A1[08830100]        <1>     mov    eax, [writei.cluster] ; current cluster
0001100F EB34                <1>     jmp    short mget_w_11
                             <1>
                             <1> mget_w_7:
00011011 89C1                <1>     mov    ecx, eax
00011013 2B0D[14830100]      <1>     sub    ecx, [writei.fs_index]
00011019 730D                <1>     jnc    short mget_w_8
                             <1>     ; get cluster by index from the first cluster
0001101B A1[10830100]        <1>     mov    eax, [writei.fclust]
00011020 8B0D[0C830100]      <1>     mov    ecx, [writei.c_index]
00011026 EB05                <1>     jmp    short mget_w_9
                             <1>
                             <1> mget_w_8:
00011028 A1[08830100]        <1>     mov    eax, [writei.cluster] ; beginning cluster
                             <1>     ; ecx = cluster sequence number after the beginning cluster
                             <1>     ; sub  edx, edx ; 0
                             <1>
                             <1> mget_w_9:
                             <1>     ; EAX = Beginning cluster
                             <1>     ; EDX = Sector index in disk/file section
                             <1>     ;       (Only for SINGLIX file system!)
                             <1>     ; ECX = Cluster sequence number after the beginning cluster
                             <1>     ; ESI = Logical DOS Drive Description Table address
0001102D E887B8FFFF          <1>     call   get_cluster_by_index
00011032 728C                <1>     jc     short mget_w_err ; error code in EAX
                             <1>
                             <1>     ; EAX = Cluster number
                             <1> mget_w_10:
00011034 A3[08830100]        <1>     mov    [writei.cluster], eax ; FDT number for Singlix File System
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>     cmp    byte [esi+LD_FATType], 0
                             <1>     jna    short mget_w_13
                             <1> %endif
                             <1>     ; 01/11/2016
00011039 8B15[0C830100]      <1>     mov    edx, [writei.c_index]
0001103F 8915[14830100]      <1>     mov    [writei.fs_index], edx
                             <1> mget_w_11:
                             <1>     ;sub   eax, 2
                             <1>     ; 23/07/2022
00011045 48                  <1>     dec    eax
00011046 48                  <1>     dec    eax
00011047 0FB615[FE820100]    <1>     movzx  edx, byte [writei.spc]
0001104E F7E2                <1>     mul    edx
                             <1>
00011050 034668              <1>     add    eax, [esi+LD_DATABegin]
00011053 8A15[FF820100]      <1>     mov    dl, [writei.s_index]
00011059 01D0                <1>     add    eax, edx
                             <1> mget_w_12:
0001105B A3[00830100]        <1>     mov    [writei.sector], eax
                             <1>     ;; buffer validation must be done in writei
                             <1>     ;;mov  byte [writei.valid], 1
00011060 C3                  <1>     retn
                             <1>
                             <1> ; 19/12/2025
                             <1> %if 0
                             <1>
                             <1> mget_w_13:
                             <1>     ; EAX = FDT number (Current Section)
                             <1>     ; EDX = Sector index from the first section (0,1,2,3,4...)
                             <1>     sub    edx, [writei.fs_index]
                             <1>     ; EDX = Sector index from current section
                             <1>     mov    [writei.fs_index], edx
                             <1>     inc    eax ; the first data sector in FS disk section
                             <1>     add    eax, edx
                             <1>     jmp    short mget_w_12
                             <1>
                             <1> mget_w_14:
                             <1>     ;mov   cl, [esi+LD_FS_BytesPerSec+1]
                             <1>     ;shr   cl, 1 ;  ; 1 for 512 bytes, 4 for 2048 bytes
                             <1>     ;mov   [writei.spc], cl  ; sectors per cluster
                             <1>     ; 19/12/2025
                             <1>     mov    byte [writei.spc], 1
                             <1>     ; NOTE: writei bytes per sector value is always 512 !
                             <1>     mov    word [writei.bpc], 512
                             <1>
                             <1>     mov    ecx, ebp
                             <1>     add    ecx, [u.count] ; next file position
                             <1>     cmp    ecx, [i.size] ; <= file size ?
                             <1>     ;jna   mget_w_19 ; no
                             <1>     ja     short mget_w_22
                             <1>     ; 23/07/2022
                             <1>     jmp    mget_w_19
                             <1>
                             <1> mget_w_22:
                             <1>     sub    edx, edx ; 0
                             <1>     mov    [writei.sector], edx ; 0
                             <1>     mov    [writei.offset], dx  ; byte offset in cluster
                             <1>     mov    [writei.s_index], dl ; sector index in cluster (0 to spc -1)
                             <1>
                             <1>     shr    ecx, 9 ; 1 cluster = 512 bytes
                             <1>     mov    [writei.c_index], ecx ; section/cluster index
                             <1>
                             <1>     mov    eax, ebx ; FDT number (First FDT address)
                             <1>
                             <1>     ; is this the 1st mget_w or a next mget_w call ? (by 'writei')
                             <1>     cmp    byte [writei.valid], dl ; 0
```

```
<1>     jna     short mget_w_15
<1>
<1>     mov     byte [writei.valid], dl ; 0 ; reset ('writei' will set it)
<1>
<1>     cmp     eax, [writei.fclust]
<1>     jne     short mget_w_15
<1>
<1>     mov     cl, [cdev]
<1>     cmp     cl, [writei.drv]
<1>     jne     short mget_w_15
<1>     ; [writei.l_clust] & [writei.l_index] are valid,
<1>     ;  we don't need to get last cluster & last cluster index
<1>     mov     ecx, [writei.l_index]
<1>     jmp     short mget_w_17
<1> mget_w_15:
<1>     mov     [writei.fclust], eax ; first section (FDT number)
<1>     ; edx = 0
<1>     mov     [writei.cluster], edx ; 0 ; current section
<1>     mov     [writei.fs_index], edx ; 0 ; curret section index
<1>
<1>     ; eax = FDT number (section 0 header address)
<1>     call    get_last_section
<1>     ;jc     short mget_w_err ; eax = error code
<1>     ; 08/08/2022
<1>     jnc     short mget_w_26
<1>     jmp     mget_w_err
<1> mget_w_26:
<1>     mov     [writei.fs_index], edx ; sector index in last section
<1>
<1>     mov     [writei.lclust], eax ; last section address
<1>
<1>     mov     ecx, [glc_index] ; last section index
<1>     mov     [writei.l_index], ecx
<1>
<1>     mov     al, [writei.ofn]
<1>     inc     al
<1>     mov     [setfmod], al ; update lm date&time sign
<1>
<1> mget_w_16:
<1>     ; edx = (existing) last section (sector) index
<1>     mov     ecx, [writei.c_index] ; final section (sector) index
<1>     sub     ecx, edx
<1>     jna     short mget_w_19
<1>     ; ecx = sector count
<1> mget_w_17:
<1>     mov     eax, [writei.lclust]
<1>     ; ESI = Logical dos drv desc. table address
<1>         ; EAX = Last section
<1>         ; (ECX = 0 for directory)
<1>         ; ECX = sector count (except FDT)
<1>     call    add_new_fs_section
<1>     jnc     short mget_w_18
<1>
<1>     ; If error number = 27h (insufficient disk space)
<1>     ; it is needed to check free consequent sectors
<1>     ; (1 data sector at least and +1 section header sector)
<1>
<1>     cmp     eax, 27h
<1>     ;jne    mget_w_err ; eax = error code
<1>     ; 25/07/2022
<1>     je      short mget_w_24
<1> mget_w_23:
<1>     jmp     mget_w_err
<1> mget_w_24:
<1>     ; ecx = count of free consequent sectors
<1>     ; ecx must be > 1 (1 data + 1 header sector)
<1>     dec     ecx
<1>     ;jz     short mget_w_err
<1>     ;jmp    short mget_w_17
<1>     ; 23/07/2022
<1>     jnz     short mget_w_17
<1>     ;jmp    mget_w_err
<1>     ; 25/07/2022
<1>     jmp     short mget_w_23
<1>
<1> mget_w_18:
<1>     mov     [writei.lclust], eax ; (new) last section
<1>     ; ecx = sector count (except section header)
<1>     mov     edx, [writei.l_index]
<1>     add     edx, ecx ; add sector count to index
<1>     mov     [writei.l_index], edx
<1>     jmp     short mget_w_16
<1>
<1> mget_w_19:
<1>     mov     ecx, ebp
<1>     add     ecx, [u.count]
<1>     mov     [i.size], ecx ; save new file size
<1>     ;sub    edx, edx ; 0
<1>
<1>     mov     al, [cdev]
<1>     mov     [writei.drv], al ; physical drive number
<1>     ; edx = 0
<1>     mov     eax, ebp ; file offset
<1>     mov     edx, eax
<1>     ; 1 cluster = 512 bytes (for Singlix FS)
<1>     shr     eax, 9  ; / 512
<1>     and     edx, 1FFh
<1>     ; edx = byte offset in cluster/sector (<= 511)
<1>     ; eax = section (sector/cluster) index
<1>     mov     [writei.c_index], eax
<1>     mov     [writei.offset], dx
<1>     ;mov    byte [writei.s_index], 0 ; sector index in cluster
<1>     jmp     mget_w_3
<1>
<1> %endif
<1>
<1> update_file_lmdt: ; & update file size
<1>     ; 19/12/2025 - TRDOS 386 v2.0.10
<1>     ; 03/09/2024
<1>     ; 27/08/2024 - TRDOS 386 v2.0.9
<1>     ; 30/07/2022
<1>     ; 23/07/2022 - TRDOS 386 v2.0.5
<1>     ; 26/10/2016
<1>     ; 24/10/2016
<1>     ; 23/10/2016
<1>     ; 22/10/2016 - TRDOS 386 (TRDOS v2.0)
<1>     ;
<1>     ; Update last modification date&time of file
<1>     ; (call from syswrite -> writei)
<1>     ; ((also updates file size)) // 26/10/2016
<1>     ;
<1>     ; INPUT:
<1>     ;       byte [setfmod] = open file number
<1>     ; OUTPUT:
```

```
                                <1>     ;       cf = 0 -> success !
                                <1>     ;       cf = 1 -> lmdt update has been failed!
                                <1>     ;
                                <1>     ; Modified registers: eax, ebx, ecx, edx, esi, edi
                                <1>     ;
                                <1>
                                <1>     ;cmp    byte [setfmod], 0
                                <1>     ;jna    short uflmdt_2 ; nothing to do
                                <1>
                                <1>     ; 27/08/2024
                                <1>     ; if [ebx+OF_FCLUSTER] = 0 and [setfclust] > 0
                                <1>     ; [setfclust] will be copied to Directory entry's
                                <1>     ; first cluster fields (LO/HI).
                                <1>     ; (and [setfclust] will be cleared.)
                                <1>
00011061 31C0                   <1>     xor     eax, eax
                                <1>
00011063 0FB61D[63830100]       <1>     movzx   ebx, byte [setfmod]
0001106A FECB                   <1>     dec     bl ; open file index number (0 based)
                                <1>
0001106C 8AA3[F8830100]         <1>     mov     ah, [ebx+OF_DRIVE]
00011072 BE00010900             <1>     mov     esi, Logical_DOSDisks
00011077 01C6                   <1>     add     esi, eax
00011079 C0E302                 <1>     shl     bl, 2 ; *4
0001107C 8B8B[78830100]         <1>     mov     ecx, [ebx+OF_FCLUSTER] ; first cluster
00011082 8B93[F8850100]         <1>     mov     edx, [ebx+OF_DIRCLUSTER] ; dir cluster
                                <1>
00011088 D0EB                   <1>     shr     bl, 1 ; /2
0001108A 0FB7BB[F8870100]       <1>     movzx   edi, word [ebx+OF_DIRENTRY]
                                <1>
                                <1>     ; 03/09/2024
00011091 80C441                 <1>     add     ah, 'A'
00011094 8A4603                 <1>     mov     al, [esi+LD_FATType]
                                <1>
00011097 803D[9D7E0100]01       <1>     cmp     byte [DirBuff_ValidData], 1
0001109E 727E                   <1>     jb      short uflmdt_4
                                <1>
                                <1>     ;mov    al, [DirBuff_DRV]
                                <1>     ;sub    al, 'A'
                                <1>     ;cmp    al, ah
                                <1>     ; 03/09/2024
                                <1>     ; 27/08/2024
                                <1>     ;add    ah, 'A'
                                <1>     ;mov    al, [esi+LD_FATType]
000110A0 3A25[9B7E0100]         <1>     cmp     ah, [DirBuff_DRV]
000110A6 7576                   <1>     jne     short uflmdt_4 ; different drive
                                <1>     ;mov    al, [esi+LD_FATType]
000110A8 3A05[9C7E0100]         <1>     cmp     al, [DirBuff_FATType]
000110AE 756E                   <1>     jne     short uflmdt_5 ; different FS type
000110B0 3B15[A27E0100]         <1>     cmp     edx, [DirBuff_Cluster]
000110B6 7566                   <1>     jne     short uflmdt_5 ; different cluster
                                <1>
                                <1> uflmdt_1:
                                <1>     ; Directory buffer is ready here!
                                <1>     ; OF_FCLUSTER must be compared/verified
000110B8 BE00000800             <1>     mov     esi, Directory_Buffer
                                <1>     ;shl    di, 5 ; dir entry index * 32
                                <1>     ; 23/07/2022
000110BD C1E705                 <1>     shl     edi, 5
000110C0 01FE                   <1>     add     esi, edi ; offset
                                <1>     ;
                                <1>     ; 03/09/2024
                                <1>     ;test   byte [esi+DirEntry_Attr], 18h ; Vol & Dir
                                <1>     ;jnz    short uflmdt_2 ; not a valid file !
000110C2 668B4614               <1>     mov     ax, [esi+DirEntry_FstClusHI]
000110C6 C1E010                 <1>     shl     eax, 16
000110C9 668B461A               <1>     mov     ax, [esi+DirEntry_FstClusLO]
                                <1>
                                <1>     ; 03/09/2024 (short jump)
000110CD F6460B18               <1>     test    byte [esi+DirEntry_Attr], 18h ; Vol & Dir
000110D1 7571                   <1>     jnz     short uflmdt_2 ; not a valid file !
                                <1>
000110D3 39C8                   <1>     cmp     eax, ecx ; same first cluster ?
                                <1>     ;je     short uflmdt_3 ; yes, it is OK !!!
                                <1>     ; 23/07/2022
000110D5 756D                   <1>     jne     short uflmdt_2
                                <1>
                                <1>     ;;;
                                <1>     ; 03/09/2024
000110D7 D0E3                   <1>     shl     bl, 1 ; *2
                                <1>     ; 27/08/2024
000110D9 21C0                   <1>     and     eax, eax
000110DB 751B                   <1>     jnz     short uflmdt_3 ; not empty file
000110DD 8705[64830100]         <1>     xchg    eax, [setfclust]
000110E3 09C0                   <1>     or      eax, eax
000110E5 7411                   <1>     jz      short uflmdt_3
                                <1>     ; 03/09/2024
000110E7 8983[78830100]         <1>     mov     [ebx+OF_FCLUSTER], eax
                                <1>     ;
000110ED 6689461A               <1>     mov     [esi+DirEntry_FstClusLO], ax
000110F1 C1E810                 <1>     shr     eax, 16
000110F4 66894614               <1>     mov     [esi+DirEntry_FstClusHI], ax
                                <1>     ;;;
                                <1>
                                <1> uflmdt_3:
                                <1>     ; Update directory entry
                                <1>     ; 03/09/2024
                                <1>     ; 26/10/2016
                                <1>     ;shl    bl, 1 ; *2
000110F8 8B83[F8840100]         <1>     mov     eax, [ebx+OF_SIZE] ; file size
000110FE 89461C                 <1>     mov     [esi+DirEntry_FileSize], eax
                                <1>     ;
00011101 E8F69BFFFF             <1>     call    convert_current_date_time
                                <1>     ; OUTPUT -> DX = Date in dos dir entry format
                                <1>         ;       AX = Time in dos dir entry format
00011106 66894616               <1>     mov     [esi+DirEntry_WrtTime], ax
0001110A 66895618               <1>     mov     [esi+DirEntry_WrtDate], dx
0001110E 66895612               <1>     mov     [esi+DirEntry_LastAccDate], dx
00011112 C605[9D7E0100]02       <1>     mov     byte [DirBuff_ValidData], 2
                                <1>     ;call   save_directory_buffer
                                <1>     ;retn
                                <1>     ; 23/07/2022
00011119 E9739CFFFF             <1>     jmp     save_directory_buffer
                                <1>
                                <1> uflmdt_4:
                                <1>     ; Directory buffer sector read&write
                                <1>     ; 23/10/2016
                                <1>     ; 27/08/2024
                                <1>     ;mov    al, [esi+LD_FATType]
                                <1> uflmdt_5:
0001111E BB[A4940100]           <1>     mov     ebx, rw_buffer ; Common r/w sector buffer addr
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
```

```
                                <1>     and     al, al ; 0 = Singlix FS
                                <1>     jz      short uflmdt_11
                                <1> %endif
                                <1>
                                <1> ;uflmdt_12:
00011123 21D2                   <1>     and     edx, edx
00011125 7527                   <1>     jnz     short uflmdt_9
                                <1>
00011127 3C02                   <1>     cmp     al, 2  ; 3 = FAT32
00011129 7720                   <1>     ja      short uflmdt_8
                                <1>
0001112B 89F8                   <1>     mov     eax, edi ; directory entry index number
                                <1>     ;shr    ax, 4 ; 16 entries per sector
                                <1>     ; 23/07/2022
0001112D C1E804                 <1>     shr     eax, 4
00011130 034664                 <1>     add     eax, [esi+LD_ROOTBegin]
                                <1>     ; eax = root directory sector
                                <1> uflmdt_6:
00011133 50                     <1>     push    eax ; * ; disk sector address
00011134 51                     <1>     push    ecx ; first cluster
00011135 B901000000             <1>     mov     ecx, 1
                                <1>     ; ecx = sector count
0001113A E8B4080000             <1>     call    disk_read
0001113F 59                     <1>     pop     ecx
00011140 7320                   <1>     jnc     short uflmdt_10
00011142 58                     <1>     pop     eax ; *
                                <1> uflmdt_7:
00011143 C3                     <1>     retn
                                <1>
                                <1> uflmdt_2:
                                <1>     ; save directory buffer if has modified/changed sign
                                <1>     ; (It is good to save dir buff even if the searched
                                <1>     ; directory entry is not found !?)
00011144 E8489CFFFF             <1>     call    save_directory_buffer
00011149 F9                     <1>     stc     ; update failed
0001114A C3                     <1>     retn
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1> uflmdt_11:
                                <1>     ; 24/10/2016
                                <1>     ; Update last modification date & time of a file
                                <1>     ; on a disk with Singlix File System.
                                <1>     ;
                                <1>     ; (Method: Read the FDT -File Description Table-
                                <1>     ; sector of the file and update the lmdt data fields,
                                <1>     ; then write FDT sector to the disk.
                                <1>     ; /// It is easy but there is compatibility buffer
                                <1>     ; method also for changing directory entry data and
                                <1>     ; also there are some programming issues for Singlix
                                <1>     ; file system (TRFS), which are not completed yet!)
                                <1>     ;
                                <1>     ; Not ready yet ! (24/10/2016)
                                <1>     ; /// Temporary code for error return ! ///
                                <1>     xor     eax, eax
                                <1>     stc
                                <1>     retn
                                <1> %endif
                                <1>
                                <1> uflmdt_8:
0001114B 8B5632                 <1>     mov     edx, [esi+LD_BPB+FAT32_RootFClust]
                                <1> uflmdt_9:
0001114E 83FA02                 <1>     cmp     edx, 2
00011151 72F0                   <1>     jb      short uflmdt_7 ; invalid, nothing to do
                                <1>
                                <1>     ;sub    edx, 2
                                <1>     ; 30/07/2022
00011153 4A                     <1>     dec     edx
00011154 4A                     <1>     dec     edx
00011155 89D0                   <1>     mov     eax, edx
00011157 0FB65613               <1>     movzx   edx, byte [esi+LD_BPB+SecPerClust]
0001115B F7E2                   <1>     mul     edx
0001115D 034668                 <1>     add     eax, [esi+LD_DATABegin]
                                <1>     ; eax = sub directory (data) sector
00011160 EBD1                   <1>     jmp     short uflmdt_6
                                <1>
                                <1> uflmdt_10:
                                <1>     ; Directory sector buffer is ready here!
                                <1>     ; OF_FCLUSTER must be compared/verified
                                <1>     ; edi = dir entry index number (<= 2047)
00011162 6683E70F               <1>     and     di, 0Fh ; 16 entries per sector
                                <1>     ;shl    di, 5 ; dir entry index * 32
                                <1>     ; 23/07/2022
00011166 C1E705                 <1>     shl     edi, 5
00011169 81C7[A4940100]         <1>     add     edi, rw_buffer
                                <1>     ;
0001116F F6470B18               <1>     test    byte [edi+DirEntry_Attr], 18h ; Vol & Dir
00011173 75CF                   <1>     jnz     short uflmdt_2 ; not a valid file !
00011175 668B5714               <1>     mov     dx, [edi+DirEntry_FstClusHI]
00011179 C1E210                 <1>     shl     edx, 16
0001117C 668B571A               <1>     mov     dx, [edi+DirEntry_FstClusLO]
00011180 39CA                   <1>     cmp     edx, ecx ; same first cluster ?
00011182 75C0                   <1>     jne     short uflmdt_2 ; no !?
                                <1>
                                <1>     ;;;
                                <1>     ; 27/08/2024
00011184 21D2                   <1>     and     edx, edx
00011186 7515                   <1>     jnz     short uflmdt_12 ; not empty file
00011188 8715[64830100]         <1>     xchg    edx, [setfclust]
0001118E 09D2                   <1>     or      edx, edx
00011190 740B                   <1>     jz      short uflmdt_12
00011192 6689571A               <1>     mov     [edi+DirEntry_FstClusLO], dx
00011196 C1EA10                 <1>     shr     edx, 16
00011199 66895714               <1>     mov     [edi+DirEntry_FstClusHI], dx
                                <1> uflmdt_12:
                                <1>     ;;;
                                <1>
                                <1>     ; Update directory entry
0001119D E85A9BFFFF             <1>     call    convert_current_date_time
                                <1>     ; OUTPUT -> DX = Date in dos dir entry format
                                <1>         ;       AX = Time in dos dir entry format
000111A2 66894716               <1>     mov     [edi+DirEntry_WrtTime], ax
000111A6 66895718               <1>     mov     [edi+DirEntry_WrtDate], dx
000111AA 66895712               <1>     mov     [edi+DirEntry_LastAccDate], dx
                                <1>
000111AE 58                     <1>     pop     eax ; *
                                <1>
000111AF BB[A4940100]           <1>     mov     ebx, rw_buffer ; Common r/w sector buffer addr
000111B4 B901000000             <1>     mov     ecx, 1
                                <1>     ; esi = logical dos description table address
                                <1>     ; eax = disk sector number/address (LBA)
                                <1>     ; ecx = sector count
                                <1>     ; ebx = buffer address
000111B9 E826080000             <1>     call    disk_write
```

```
000111BE 7284                <1>     jc      short uflmdt_2
                             <1> ;uflmdt_12:
                             <1>     ; save directory buffer if has modified/changed sign
                             <1>     ;call  save_directory_buffer
                             <1>     ;retn
                             <1>     ; 23/07/2022
000111C0 E9CC9BFFFF          <1>     jmp    save_directory_buffer
                             <1>
                             <1> sysalloc:
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 14/10/2017
                             <1>     ; 20/08/2017 - 01/09/2017
                             <1>     ; 20/02/2017 - 04/03/2017 - 15/05/2017
                             <1>     ; 19/02/2017 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; (TRDOS 386 feature only!)
                             <1>     ;
                             <1>     ; Allocate Contiguous Memory Block/Pages (for user)
                             <1>     ; (System call for DMA Buffer allocation etc.)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;      EBX = Virtual address (for user)
                             <1>     ;           (Physical memory block/aperture
                             <1>     ;           will be mapped to this virtual address)
                             <1>     ;      ECX = Byte Count
                             <1>     ;           (will be rounded up to page border)
                             <1>     ;      If ECX = 0
                             <1>     ;          System call will return with an error (cf=1)
                             <1>     ;          but ECX will contain maximum size of
                             <1>     ;          available memory aperture and physical
                             <1>     ;          (beginning) address of that aperture
                             <1>     ;          (which have maximum size) will be in EAX.
                             <1>     ;      EDX = Upper limit of the requested physical memory
                             <1>     ;            block/pages.
                             <1>     ;           (The last byte address of the memory aperture
                             <1>     ;            must not be equal to or above this limit.)
                             <1>     ;      If EDX = 0
                             <1>     ;         there is NOLIMIT !
                             <1>     ;      If EDX = 0FFFFFFFFh (-1)
                             <1>     ;         ESI = Lower Limit !
                             <1>     ;             (Beginning of the block must not be 'less'
                             <1>     ;             than this.) (Must be equal to or above...)
                             <1>     ;         EDI = Upper Limit !
                             <1>     ;             (End of the block must be !less! than this)
                             <1>     ;             (The last byte addr of the memory aperture
                             <1>     ;             must not be equal to or above this limit.)
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;      If CF = 0
                             <1>     ;      EAX = Physical address of the allocated memory block
                             <1>     ;      ECX = Allocated bytes (as rounded up to page borders)
                             <1>     ;      EBX = Virtual address (as rounded up)
                             <1>     ;      IF CF = 1
                             <1>     ;          Requested (size of) Memory block could not be
                             <1>     ;          allocated to the user!
                             <1>     ;      IF CF = 1 & EAX = 0 (Insufficient memory error!)
                             <1>     ;         ECX = Total number of free bytes
                             <1>     ;               (not size of available contiguous bytes!)
                             <1>     ;      If CF = 1 & EAX > 0
                             <1>     ;         there is not a memory aperture with requested size
                             <1>     ;         but total free mem is not less than requested size.
                             <1>     ;         EAX = Physical addr of available memory aperture
                             <1>     ;              with max size
                             <1>     ;              (but it doesn't fit to the conditions!)
                             <1>     ;         ECX = Size of available memory aperture in bytes.
                             <1>     ;      If CF = 1 -> EAX = 0FFFFFFFFh
                             <1>     ;         Conditions/Parameters are wrong !
                             <1>     ;         ECX is same with input value.
                             <1>     ;
                             <1>     ; Note: Previously allocated pages will be deallocated if
                             <1>     ;       new allocation conditions are met.
                             <1>     ;
                             <1>     ; Note: u.break control may be included in future versions
                             <1>     ;
                             <1>
000111C5 31C0                <1>     xor    eax, eax ; 0
                             <1>     ; 14/10/2017
000111C7 4A                  <1>     dec    edx ; is there a limit ?
000111C8 7810                <1>     js     short sysalloc_1 ;  0 -> 0FFFFFFFFh -> NO LIMIT
000111CA 42                  <1>     inc    edx ; > 0
                             <1>     ; Check upper address limit
                             <1>     ;(round up to page borders)
000111CB 81C1FF0F0000        <1>     add    ecx, PAGE_SIZE-1 ; 4095
000111D1 6681E100F0          <1>     and    cx, ~PAGE_OFF ; not 4095
000111D6 39CA                <1>     cmp    edx, ecx ; upper limit - block size
000111D8 7224                <1>     jb     short sysalloc_err
                             <1> sysalloc_1:
                             <1>     ; EAX = Beginning address (physical)
                             <1>     ; EAX = 0 -> Allocate mem block from the 1st proper aperture
                             <1>     ; ECX = Number of bytes to be allocated
000111DA E8664BFFFF          <1>     call   allocate_memory_block
000111DF 721D                <1>     jc     short sysalloc_err
                             <1>     ; 01/09/2017
000111E1 29C2                <1>     sub    edx, eax ; upper limit address - beginning address
000111E3 760F                <1>     jna    short sysalloc_3 ; begin addr not less than the limit
000111E5 39CA                <1>     cmp    edx, ecx
000111E7 720B                <1>     jb     short sysalloc_3 ; end address overs the limit
                             <1> sysalloc_2:
                             <1>     ; EAX = Beginning (physical) addr of the allocated mem block
                             <1>     ; ECX = Num of allocated bytes (rounded up to page borders)
000111E9 50                  <1>     push   eax ; * ; 04/03/2017
                             <1>     ; Here, requested contiquous memory pages have been allocated
                             <1>     ; on Memory Allocation Table but user's page directory
                             <1>     ; and page tables have not been updated yet!
000111EA 51                  <1>     push   ecx ; **
                             <1>     ; ebx = virtual address (will be rounded up to page border)
                             <1>     ; ecx = number of bytes to be deallocated
                             <1>     ;      will be adjusted to ebx+ecx round down - ebx round up
000111EB E8AD4EFFFF          <1>     call   deallocate_user_pages
000111F0 731F                <1>     jnc    short sysalloc_4 ; EAX = Deallocated memory bytes
000111F2 59                  <1>     pop    ecx ; **
000111F3 58                  <1>     pop    eax ; *
                             <1> sysalloc_3:
                             <1>     ; error !
                             <1>     ; restore Memory Allocation Table Content
000111F4 E8664DFFFF          <1>     call   deallocate_memory_block
000111F9 31C0                <1>     xor    eax, eax ; 0
000111FB 48                  <1>     dec    eax ; 0FFFFFFFFh ; 15/05/2017
000111FC EB09                <1>     jmp    short sysalloc_wrong
                             <1> sysalloc_err:
000111FE 8B2D[688E0100]      <1>     mov    ebp, [u.usp]  ; ebp points to user's registers
00011204 894D18              <1>     mov    [ebp+24], ecx ; return to user with ecx value
                             <1> sysalloc_wrong:
                             <1>     ; eax = 0FFFFFFFFh
00011207 A3[6C8E0100]        <1>     mov    [u.r0], eax
```

```
0001120C E95DB8FFFF          <1>     jmp    error
                             <1> sysalloc_4:
00011211 8B2D[688E0100]      <1>     mov    ebp, [u.usp]  ; ebp points to user's registers
00011217 894518              <1>     mov    [ebp+24], eax ; return to user with ecx value
0001121A 895D10              <1>     mov    [ebp+16], ebx ; new value of ebx (rounded up)
0001121D 89C1                <1>     mov    ecx, eax ; byte count (from 'deallocate_user_pages')
0001121F 5A                  <1>     pop    edx ; ** ; discard (another) byte count
00011220 58                  <1>     pop    eax ; *
00011221 A3[6C8E0100]        <1>     mov    [u.r0], eax ; physical address
                             <1>
00011226 51                  <1>     push   ecx ; 20/08/2017
                             <1>     ;
                             <1>     ; Write newly allocated contiguous (physical) pages
                             <1>     ; on page dir and page tables of current user/process
                             <1>     ; as PRESENT, USER, WRITABLE
                             <1>     ; (then clear allocated pages)
00011227 E8594FFFFF          <1>     call   allocate_user_pages
                             <1>     ;jnc   sysret ; OK! return to process with success...
                             <1>
                             <1>     ; 20/08/2017 ('sysdma' modification)
0001122C 59                  <1>     pop    ecx
0001122D A1[6C8E0100]        <1>     mov    eax, [u.r0]   ; physical address (of the block)
                             <1>
00011232 7219                <1>     jc     short sysalloc_6
                             <1>
00011234 833D[308D0100]FF    <1>     cmp    dword [dma_addr], 0FFFFFFFFh ; -1
                             <1>     ;jb    sysret
                             <1>     ; 23/07/2022
0001123B 720B                <1>     jb     short sysalloc_5 ; jmp sysret
                             <1>
0001123D A3[308D0100]        <1>     mov    [dma_addr], eax ; save dma address for sysdma
00011242 890D[348D0100]      <1>     mov    [dma_size], ecx ; save dma buff size for sysdma
                             <1> sysalloc_5:
00011248 E941B8FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysalloc_6:
                             <1>     ;
                             <1>     ; unexpected error ! insufficient memory !? conflict !?
                             <1>     ; (!!?there is not a free page for a new page table?!!)
                             <1>     ; We need to terminate process with error message !!!
                             <1>     ;
0001124D 8B2D[688E0100]      <1>     mov    ebp, [u.usp]  ; ebp points to user's registers
00011253 8B4D18              <1>     mov    ecx, [ebp+24] ; byte count
                             <1>
                             <1>     ; 20/08/2017
                             <1>     ;mov   eax, [u.r0]   ; physical address (of the block)
                             <1>
                             <1>     ;
                             <1>     ; restore Memory Allocation Table Content
00011256 E8044DFFFF          <1>     call   deallocate_memory_block
                             <1>     ;
0001125B 803D[2E680000]03    <1>     cmp    byte [CRT_MODE], 3 ; 80x25 text mode?
00011262 7407                <1>     je     short sysalloc_7 ; yes
                             <1>     ; Current mode is VGA (or CGA graphics) mode,
                             <1>     ; We need to return to text mode for displaying
                             <1>     ; error message just before 'sysexit'.
00011264 B003                <1>     mov    al, 3
00011266 E81A09FFFF          <1>     call   _set_mode
                             <1> sysalloc_7:
0001126B BE[F8310100]        <1>     mov    esi, beep_Insufficient_Memory ; error message
00011270 E8455BFFFF          <1>     call   print_msg ; print/display the message
00011275 B801000000          <1>     mov    eax, 1 ; ax=1 is needed for 'sysexit' procedure
0001127A E94DB9FFFF          <1>     jmp    sysexit ; and terminate the process !
                             <1>
                             <1> sysdalloc:
                             <1>     ; 19/02/2017 - TRDOS 386 (TRDOS v2.0)
                             <1>     ; (TRDOS 386 feature only!)
                             <1>     ;
                             <1>     ; Deallocate Memory Block/Pages (for user)
                             <1>     ; (Complementary call for sysalloc.)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;      EBX = Virtual address (for user)
                             <1>     ;            (will be rounded up to page border)
                             <1>     ;      ECX = Byte Count
                             <1>     ;           (will be adjusted to page borders)
                             <1>     ;      If ICX = 0
                             <1>     ;         nothing to do
                             <1>     ;      If EBX + ECX > User's ESP
                             <1>     ;         nothing to do
                             <1>     ;
                             <1>     ; Note: u.break control may be included in future versions
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;      If CF = 0
                             <1>     ;         EAX = Deallocated memory bytes
                             <1>     ;         EBX = Virtual address (as rounded up)
                             <1>     ;      IF CF = 1
                             <1>     ;         EAX = 0
                             <1>     ;
                             <1>     ; Note: Main purpose of this call is to deallocate/release
                             <1>     ;      previously allocated (physically) contiguous memory
                             <1>     ;      pages but beginning (virtual) address may not be
                             <1>     ;      followed by physically contiguous pages. So, this
                             <1>     ;      system call will deallocate user's virtually
                             <1>     ;      contiguous memory pages. Also, there is not any
                             <1>     ;      objections to use this system call without sysalloc
                             <1>     ;      system call; only possible objection is to lost data
                             <1>     ;      within user's memory space, if the beginning address
                             <1>     ;      and size is not proper.
                             <1>     ;
                             <1>     ; Note: Empty page tables will not be deallocated!!!
                             <1>     ;       (they will be deallocated at process termination)
                             <1>     ;
                             <1>     ; Note: When the program terminates itself or when it is
                             <1>     ;      terminated by operating system kernel, all allocated
                             <1>     ;      memory pages will be deallocated during termination
                             <1>     ;      stage. So, 'sysdalloc' is not necessary except
                             <1>     ;      forgiving memory block to other programs/processes.
                             <1>     ;
0001127F 8B15[648E0100]      <1>     mov    edx, [u.sp]
00011285 8B420C              <1>     mov    eax, [edx+12] ; user's stack pointer
00011288 29C8                <1>     sub    eax, ecx ; esp - byte count
0001128A 24FC                <1>     and    al, 0FCh ; dword alignment
0001128C 39D8                <1>     cmp    eax, ebx
0001128E 7220                <1>     jb     short sysdalloc_err ; deallocation overlaps with stack
                             <1>
00011290 31C0                <1>     xor    eax, eax
00011292 21C9                <1>     and    ecx, ecx
00011294 7407                <1>     jz     short sysdalloc_2
                             <1>
00011296 E8024EFFFF          <1>     call   deallocate_user_pages
0001129B 7213                <1>     jc     short sysdalloc_err
                             <1>
```

```
                                <1> sysdalloc_2:
0001129D A3[6C8E0100]           <1>     mov     [u.r0], eax
000112A2 8B2D[688E0100]         <1>     mov     ebp, [u.usp]
000112A8 895D10                 <1>     mov     [ebp+16], ebx ; new value of ebx
000112AB E9DEB7FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdalloc_err:
000112B0 A3[6C8E0100]           <1>     mov     [u.r0], eax ; 0
000112B5 E9B4B7FFFF             <1>     jmp     error
                                <1>
                                <1> syscalbac:
                                <1>     ; SYS CALLBACK
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 03/08/2020
                                <1>     ; 16/04/2017
                                <1>     ; 14/04/2017
                                <1>     ; 13/04/2017
                                <1>     ; 28/02/2017
                                <1>     ; 26/02/2017
                                <1>     ; 24/02/2017
                                <1>     ; 21/02/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; (TRDOS 386 feature only!)
                                <1>     ;
                                <1>     ; Link or unlink IRQ callback service to/from user (ring 3)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;       BL = IRQ number (Hardware interrupt request number)
                                <1>     ;            (0 t0 15 but IRQ 0,1,2,6,8,14,15 are prohibited)
                                <1>     ;            IRQ numbers 3,4,5,7,9,10,11,12,13 are valid
                                <1>     ;            (numbers >15 are invalid)
                                <1>     ;
                                <1>     ;       BH = 0 = Unlink IRQ (in BL) from user (ring 3) service
                                <1>     ;            1 = Link IRQ by using Signal Response Byte method
                                <1>     ;            2 = Link IRQ by using Callback service method
                                <1>     ;            3 = Link IRQ by using Auto Increment S.R.B. method
                                <1>     ;           >3 = invalid
                                <1>     ;
                                <1>     ;       CL = Signal Return/Response Byte value
                                <1>     ;
                                <1>     ;       If BH = 3, kernel will put a counter value  ; 03/08/2020
                                <1>     ;                 (into the S.R.B. addr)
                                <1>     ;                 between 0 to 255. (start value = CL+1)
                                <1>     ;
                                <1>     ;       NOTE: counter value, for example: even and odd numbers
                                <1>     ;             may be used for -audio- DMA buffer switch
                                <1>     ;             within double buffer method, etc.
                                <1>     ;
                                <1>     ;       EDX = Signal return (Response) byte address
                                <1>     ;                         - or -
                                <1>     ;             Interrupt/Callback service/routine address
                                <1>     ;
                                <1>     ;             (virtual address in user's memory space)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 0 & EAX = 0 -> Successful setting
                                <1>     ;       CF = 1 & EAX > 0 -> IRQ is prohibited or locked
                                <1>     ;                           by another process
                                <1>     ;                 eax = ERR_PERM_DENIED -> prohibited or locked
                                <1>     ;                 eax = ERR_INV_PARAMETER ->
                                <1>     ;                       invalid parameter/option or bad address
                                <1>     ;
                                <1>     ;       NOTE: Timer callbacks are set by using 'systimer'
                                <1>     ;             system call (IRQ 0, PIT and IRQ 8, RTC)
                                <1>     ;
                                <1>     ;             Direct keyboard access is performed by using
                                <1>     ;             Keyboard Interrupt (INT 32h)
                                <1>     ;
                                <1>     ;             It is prohibited here because:
                                <1>     ;              1) Signal Response Byte method has not advantage
                                <1>     ;                 against INT 32h, function AH = 1. Also,
                                <1>     ;                 keyboard service interrupt will return with
                                <1>     ;                 ascii and scan codes (AL, AH) while
                                <1>     ;                 SRB method has only 1 byte space for ascii code
                                <1>     ;                 or scan code. One byte signal response is used
                                <1>     ;                 for ensuring very simple and very fast
                                <1>     ;                 virtual to physical memory address conversion
                                <1>     ;                 without any memory page crossover risk.
                                <1>     ;                 (Otherwise double page conversion or word
                                <1>     ;                 alignment would be needed.)
                                <1>     ;              2) Badly written user code (callback code)
                                <1>     ;                 can prevent keyboard and timesharing functions
                                <1>     ;                 of the operating system via continuous and long
                                <1>     ;                 keyboard event handling by callback service.
                                <1>     ;                 (It can cause to lose immediate keystroke
                                <1>     ;                 response from hardware to user.)
                                <1>     ;              3) If user will check any keyboard events, 'getkey'
                                <1>     ;                 (or 'getchar') must have more priority than other
                                <1>     ;                 (video etc.) events because only control ability
                                <1>     ;                 on a procedural infinite loop is a keyboard or
                                <1>     ;                 mouse event. So user can use keyboard function
                                <1>     ;                 at the end or at the beginning of a loop.
                                <1>     ;                 In this case, INT 32h is used for that purpose
                                <1>     ;                 and timer interrupt etc. callbacks can be used
                                <1>     ;                 for dynamic and synchronized data refresh/transfer
                                <1>     ;                 while cpu is in a static loop (without polling).
                                <1>     ;                 Keyboard Int callback is not more useful because
                                <1>     ;                 already a manual check (a key is pressed or not)
                                <1>     ;                 can be performed (via INT 32h, AH = 1) efficiently
                                <1>     ;                 in a loop to prevent a locked infinitive loop.
                                <1>     ;
                                <1>     ;          Disk IRQs (6,14,15) have been phohibited from ring 3
                                <1>     ;          callback because, disk operations (file system services
                                <1>     ;          etc.) are independent from user program, for fast disk r/w.
                                <1>     ;          They are not more useful at ring 3 while they are in use
                                <1>     ;          by standard diskio functions which are mandatory part of
                                <1>     ;          (monolithic) OS kernel and mainprog command interpreter.
                                <1>     ;          INT 33h diskio functions are enough for user level disk
                                <1>     ;          r/w.
                                <1>     ;
                                <1>     ; TRDOS 386 - IRQ CALLBACK structures (parameters):
                                <1>     ;
                                <1>     ;         [u.irqlock] = 1 word, IRQ flags (0-15) that indicates
                                <1>     ;                     which IRQs are locked by (that) user.
                                <1>     ;                      Lock and unlock (by user) will change
                                <1>     ;                     these flags or 'terminate process' (sysexit)
                                <1>     ;                     will clear these flags and unlock those IRQs.
                                <1>     ;
                                <1>     ;                     Bit 0 is for IRQ 0 and Bit 15 is for IRQ 15
                                <1>     ;
                                <1>     ;         IRQ(x).owner        : 1 byte, user, [u.uno], 0 = free (unlocked)
                                <1>     ;
                                <1>     ;         IRQ(x).method : 1 byte for callback method & status
                                <1>     ;                       0 = Signal Response Byte method
                                <1>     ;                       1 = Callback service method
```

```
                                <1>     ;                       >1 = invalid for current 'syscalback'.
                                <1>     ;                    or(+) 80h = IRQ is in use by system (ring 0)
                                <1>     ;                                function (audio etc.) or
                                <1>     ;                               a device driver.
                                <1>     ;                    (system function will ignore the lock/owner)
                                <1>     ;
                                <1>     ;         IRQ(x).srb  : 1 byte, Signal Return/Response byte value
                                <1>     ;                       (a fixed value by user or a counter value
                                <1>     ;                      from 0 to 255, which is increased by every
                                <1>     ;                      interrupt just before putting it into
                                <1>     ;                      the Signal Response byte address
                                <1>     ;                      (This is not used in callback serv method)
                                <1>     ;
                                <1>     ;         IRQ(x).addr : 1 dword
                                <1>     ;                       Signal Response Byte address (physical)
                                <1>     ;                                    -or-
                                <1>     ;                       Callback service address (virtual)
                                <1>     ;
                                <1>     ;         IRQ(x).dev  : 1 byte
                                <1>     ;                       0 = Default device or kernel function
                                <1>     ;                                    -or-
                                <1>     ;                       1-255 = Assigned device driver number
                                <1>     ;
                                <1>     ;         (x) = 3,4,5,7,9,10,11,12,13
                                <1>     ;
                                <1>     ;       NOTE: If user's process/program calls the kernel (INT 40h)
                                <1>     ;             while it is already running in a (ring 3) callback
                                <1>     ;             service, kernel will force (convert) system call to
                                <1>     ;             'sysrele' (sys release). So, this feature provides
                                <1>     ;             easy and simple usage of callback services without
                                <1>     ;             falling into deepless <please 'callback me' then
                                <1>     ;             let me 'callback you'> cycles! (User must return
                                <1>     ;             from callback service by using 'sysrele' system
                                <1>     ;             call, without a significant delay. Otherwise user
                                <1>     ;             process/program may be late to catch the next event
                                <1>     ;             within same callback purpose.
                                <1>     ;
                                <1>
000112BA 30C0                   <1>     xor     al, al ; the caller is 'syscalbac' sign/flag
000112BC E8C1150000             <1>     call    set_irq_callback_service
                                <1>     ; 16/04/2017
000112C1 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>     ;jnc    sysret
                                <1>     ; 23/07/2022
000112C6 7205                   <1>     jc      short syscalbac_err
000112C8 E9C1B7FFFF             <1>     jmp     sysret
                                <1> syscalbac_err:
000112CD A3[D88E0100]           <1>     mov     dword [u.error], eax
000112D2 E997B7FFFF             <1>     jmp     error
                                <1>
                                <1> sysfpstat:
                                <1>     ; 28/02/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; (TRDOS 386 feature only!)
                                <1>     ;
                                <1>     ; Set or reset FPU registers save/restore option (for user)
                                <1>     ;              (during software task switching, wswap-rswap)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      BL = 0 -> reset
                                <1>     ;      BL = 1 -> set (FPU register will be saved and restored)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      cf = 0 -> no error, FPU is ready...
                                <1>     ;                (EAX = 0)
                                <1>     ;      Cf = 1 -> error, 80387 FPU is not ready !
                                <1>     ;                (EAX = 0FFFFFFFFh)
                                <1>
000112D7 31C0                   <1>     xor     eax, eax
000112D9 803D[74830100]00       <1>     cmp     byte [fpready], 0
000112E0 7613                   <1>     jna     short sysfpstat_err
                                <1>
000112E2 80E301                 <1>     and     bl, 1 ; use BIT 0 only !
000112E5 881D[EB8E0100]         <1>     mov     [u.fpsave], bl
000112EB A3[6C8E0100]           <1>     mov     [u.r0], eax ; 0
000112F0 E999B7FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysfpstat_err:
000112F5 48                     <1>     dec     eax ; 0FFFFFFFFh
000112F6 A3[6C8E0100]           <1>     mov     [u.r0], eax ; -1
000112FB E96EB7FFFF             <1>     jmp     error
                                <1>
                                <1> sysdelete: ; Delete (Remove, Unlink) File
                                <1>     ; 29/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;         EBX = File name (ASCIIZ string) address
                                <1>     ; OUTPUT ->
                                <1>     ;         cf = 0 -> eax = 0
                                <1>     ;         cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>
00011300 89DE                   <1>     mov     esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov    ax, 1
                                <1>     ;mov    [FFF_Valid], ah ; 0 ; reset
                                <1>     ;call   set_working_path
00011302 E8CE090000             <1>     call    set_working_path_x
00011307 731D                   <1>     jnc     short sysdelete_1
                                <1>
00011309 21C0                   <1>     and     eax, eax  ; 0 -> Bad Path!
0001130B 7505                   <1>     jnz     short sysdelete_err
                                <1>     ; eax = 0
                                <1> sysdelete_path_err:
0001130D B813000000             <1>     mov     eax, ERR_INV_PATH_NAME ; 'bad path name !'
                                <1> sysdelete_err:
00011312 A3[6C8E0100]           <1>     mov     [u.r0], eax
00011317 A3[D88E0100]           <1>     mov     [u.error], eax
0001131C E8890A0000             <1>     call    reset_working_path
00011321 E948B7FFFF             <1>     jmp     error
                                <1> sysdelete_1:
                                <1>     ;mov    esi, FindFile_Name
00011326 66B80018               <1>     mov     ax, 1800h ; Only files
0001132A E84B76FFFF             <1>     call    find_first_file
0001132F 72E1                   <1>     jc      short sysdelete_err
                                <1> sysdelete_2:
                                <1>     ; check file attributes
                                <1>
                                <1>     ;test   bl, 17 ; system, hidden, readonly, directory
00011331 F6C307                 <1>         test       bl, 7 ; system, hidden, readonly
00011334 7407                   <1>         jz  short sysdelete_3
                                <1>
```

```
00011336 B80B000000          <1>         mov eax, ERR_FILE_ACCESS ; 11 = 'permission denied !'
0001133B EBD5                <1>         jmp short sysdelete_err
                             <1> sysdelete_3:
0001133D 6621D2              <1>     and     dx, dx ; Ambiguous filename chars used sign (DX>0)
00011340 7407                <1>     jz      short sysdelete_4
00011342 B81A000000          <1>     mov     eax, ERR_INV_FILE_NAME ; 26 = 'invalid file name !'
00011347 EBC9                <1>         jmp short sysdelete_err
                             <1> sysdelete_4:
                             <1>     ;mov    bh, [LongName_EntryLength]
00011349 883D[DE800100]      <1>     mov     [DelFile_LNEL], bh ; Long name entry length (if > 0)
                             <1>     ; edi = Directory Entry Offset (DirBuff)
                             <1>     ; esi = Directory Entry (FFF Structure)
0001134F E8559DFFFF          <1>     call    remove_file
00011354 72BC                <1>     jc      short sysdelete_err
                             <1> sysrmdir_5:
00011356 31C0                <1>     xor     eax, eax ; 0
00011358 A3[6C8E0100]        <1>     mov     [u.r0], eax
                             <1>     ;mov    [u.error], eax
0001135D E8480A0000          <1>     call    reset_working_path
00011362 E927B7FFFF          <1>     jmp     sysret
                             <1>
                             <1> sysrmdir: ; Remove (Unlink) Directory
                             <1>     ; 17/07/2025 - TRDOS 386 v2.0.10
                             <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>     ; 19/01/2021
                             <1>     ; 29/12/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>         ; INPUT ->
                             <1>         ;          EBX = Pointer to directory name
                             <1>     ; OUTPUT ->
                             <1>     ;          cf = 0 -> eax = 0
                             <1>     ;          cf = 1 -> Error code in AL
                             <1>     ;
                             <1>     ; Modified Registers: EAX (at the return of system call)
                             <1>     ;
                             <1>
                             <1>     ; 19/01/2021
00011367 803D[BE8E0100]00    <1>     cmp     byte [u.uid], 0  ; root (super user) ?
0001136E 7614                <1>     jna     short sysrmdir_0
                             <1>
                             <1>     ;mov    dword [u.r0], ERR_PERM_DENIED
00011370 B80B000000          <1>     mov     eax, ERR_PERM_DENIED ; ERR_NOT_SUPERUSER
00011375 A3[6C8E0100]        <1>     mov     [u.r0], eax
0001137A A3[D88E0100]        <1>     mov     [u.error], eax
0001137F E9EAB6FFFF          <1>     jmp     error
                             <1>
                             <1> sysrmdir_0:
00011384 89DE                <1>     mov     esi, ebx
                             <1>
                             <1>     ;;;;
                             <1>     ; 17/07/2025 - TRDOS 386 v2.0.10
00011386 A0[76770100]        <1>     mov     al, [Current_Drv]
0001138B A2[1C810100]        <1>     mov     [rmdir_drv], al
00011390 A1[70770100]        <1>     mov     eax, [Current_Dir_FCluster]
00011395 A3[18810100]        <1>     mov     [rmdir_dir_fcluster], eax
                             <1>     ;;;;
                             <1>
                             <1>     ; file name is forced, change directory as temporary
                             <1>     ;mov    ax, 1
                             <1>     ;mov    [FFF_Valid], ah ; 0 ; reset
                             <1>     ;call   set_working_path
0001139A E836090000          <1>     call    set_working_path_x
0001139F 7308                <1>     jnc     short sysrmdir_1
                             <1>
000113A1 21C0                <1>     and     eax, eax  ; 0 -> Bad Path!
000113A3 7513                <1>     jnz     short sysrmdir_err
                             <1>     ; eax = 0
                             <1> sysrmdir_not_found:
                             <1>     ;mov    eax, ERR_DIR_NOT_FOUND ; Directory not found !
                             <1>     ; 23/07/2022
000113A5 B00C                <1>     mov     al, ERR_DIR_NOT_FOUND
000113A7 EB0F                <1>     jmp     short sysrmdir_err
                             <1> ;sysrmdir_err:
                             <1> ;   mov     [u.r0], eax
                             <1> ;   mov     [u.error], eax
                             <1> ;   call    reset_working_path
                             <1> ;   jmp     error
                             <1> sysrmdir_1:
                             <1>     ;mov    esi, FindFile_Name
000113A9 66B81008            <1>     mov     ax, 0810h ; Only directories
000113AD E8C875FFFF          <1>     call    find_first_file
000113B2 7318                <1>     jnc     short sysrmdir_2
                             <1>
                             <1>     ; eax = 2 (File not found !)
000113B4 3C02                <1>     cmp     al, 2 ; ERR_NOT_FOUND
000113B6 74ED                <1>     je      short sysrmdir_not_found
                             <1>     ;jmp    short sysrmdir_err
                             <1>     ; 23/07/2022
                             <1> sysrmdir_err:
000113B8 A3[6C8E0100]        <1>     mov     [u.r0], eax
000113BD A3[D88E0100]        <1>     mov     [u.error], eax
                             <1> sysrmdir_8: ; 23/07/2022
000113C2 E8E3090000          <1>     call    reset_working_path
000113C7 E9A2B6FFFF          <1>     jmp     error
                             <1> sysrmdir_2:
                             <1>     ; check directory attributes
                             <1>
000113CC F6C307              <1>         test       bl, 7 ; system, hidden, readonly
000113CF 7407                <1>         jz  short sysrmdir_3
                             <1>
000113D1 B80B000000          <1>         mov eax, ERR_DIR_ACCESS ; 11 = 'permission denied !'
000113D6 EBE0                <1>         jmp short sysrmdir_err
                             <1> sysrmdir_3:
000113D8 6621D2              <1>     and     dx, dx ; Ambiguous filename chars used sign (DX>0)
000113DB 7407                <1>     jz      short sysrmdir_4
                             <1>     ;mov    eax, ERR_NOT_DIR ; 'not a valid directory !'
000113DD B813000000          <1>     mov     eax, ERR_INV_PATH_NAME ; 'bad path name !'
000113E2 EBD4                <1>         jmp short sysrmdir_err
                             <1> sysrmdir_4:
                             <1>     ;mov    bh, [LongName_EntryLength]
000113E4 883D[DE800100]      <1>     mov     [DelFile_LNEL], bh ; Long name entry length (if > 0)
                             <1>     ; edi = Directory Entry Offset (DirBuff)
                             <1>     ; esi = Directory Entry (FFF Structure)
000113EA E8F77BFFFF          <1>     call    delete_sub_directory
                             <1>     ;jnc    sysrmdir_5
                             <1>     ; 23/07/2022
000113EF 7205                <1>     jc      short sysrmdir_6
000113F1 E960FFFFFF          <1>     jmp     sysrmdir_5
                             <1>
                             <1> ;   jc      short sysrmdir_6
                             <1> ;
                             <1> ;   xor     eax, eax ; 0
                             <1> ;sysrmdir_5:
                             <1> ;   mov     [u.r0], eax
```

```
                                <1> ;   ;mov    [u.error], eax
                                <1> ;   call    reset_working_path
                                <1> ;   jmp     sysret
                                <1>
                                <1> sysrmdir_6:
000113F6 A3[6C8E0100]           <1>     mov     [u.r0], eax
000113FB A3[D88E0100]           <1>     mov     [u.error], eax
                                <1>
00011400 09C0                   <1>     or      eax, eax ; EAX = 0 -> Directory not empty!
00011402 7414                   <1>     jz      short sysrmdir_9
                                <1>
                                <1>     ; EAX > 0 -> Error code in AL (or AX or EAX)
                                <1>
00011404 833D[927E0100]01       <1>     cmp     dword [FAT_ClusterCounter], 1
0001140B 72B5                   <1>     jb      short sysrmdir_8
                                <1> sysrmdir_7:
                                <1>     ; ESI = Logical DOS Drive Description Table address
0001140D 66BB00FF               <1>     mov     bx, 0FF00h ; BH = FFh -> use ESI for Drive parameters
                                <1>                 ; BL = 0 -> Recalculate free cluster count
00011411 E85BB1FFFF             <1>     call    calculate_fat_freespace
                                <1>     ; 23/07/2022
00011416 EBAA                   <1>     jmp     short sysrmdir_8
                                <1> ;sysrmdir_8:
                                <1> ;   call    reset_working_path
                                <1> ;   jmp     error
                                <1>
                                <1> sysrmdir_9:
00011418 A1[927E0100]           <1>     mov     eax, [FAT_ClusterCounter]
0001141D 09C0                   <1>     or      eax, eax ; 0 ?
                                <1>     ;jz     short sysrmdir_err
                                <1>     ; 23/07/2022
0001141F 74A1                   <1>     jz      short sysrmdir_8
                                <1>
                                <1>     ; ESI = Logical DOS Drive Description Table address
00011421 66BB01FF               <1>     mov     bx, 0FF01h ; BH = FFh -> use ESI for Drive parameters
                                <1>                 ; BL = 1 -> add free clusters
00011425 E847B1FFFF             <1>     call    calculate_fat_freespace
0001142A 09C9                   <1>     or      ecx, ecx
0001142C 7494                   <1>         jz  short sysrmdir_8 ; ecx = 0 -> OK
                                <1>     ; ecx > 0 -> Error (Recalculation is needed)
0001142E EBDD                   <1>     jmp     short sysrmdir_7
                                <1>
                                <1> syschdir: ; Change Current (Working) Drive & Directory (for user)
                                <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;           EBX = Directory name (ASCIIZ string) address
                                <1>     ; OUTPUT ->
                                <1>     ;           cf = 0 -> eax = 0
                                <1>     ;           cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; NOTE: If drive name is not included, only the working
                                <1>     ; directory (for user, not for drive/OS) will be chanded.
                                <1>     ; If there is a drive name (as A:, B:, C:, D: etc.)
                                <1>     ; at the beginning of the ASCIIZ (directory) string,
                                <1>     ; working drive and working directory (for user)
                                <1>     ; will be changed together.
                                <1>     ; (When the program is terminated, MainProg -internal
                                <1>     ; shell- will reset working directory to the previous
                                <1>     ; -current- logical drive's current directory again.)
                                <1>
00011430 89DE                   <1>     mov     esi, ebx
                                <1>     ; file name is not forced, change directory as temporary
00011432 31C0                   <1>     xor     eax, eax
                                <1>     ;mov    [FFF_Valid], ah ; 0 ; reset
                                <1>     ;call   set_working_path
00011434 E8A0080000             <1>     call    set_working_path_xx
00011439 731D                   <1>     jnc     short syschdir_ok
0001143B 21C0                   <1>     and     eax, eax  ; 0 -> Bad Path!
0001143D 7505                   <1>     jnz     short syschdir_err
                                <1>     ; eax = 0
                                <1> syschdir_not_found:
0001143F B80C000000             <1>     mov     eax, ERR_DIR_NOT_FOUND ; Directory not found !
                                <1> syschdir_err:
00011444 A3[6C8E0100]           <1>     mov     [u.r0], eax
00011449 A3[D88E0100]           <1>     mov     [u.error], eax
0001144E E857090000             <1>     call    reset_working_path
00011453 E916B6FFFF             <1>     jmp     error
                                <1> syschdir_ok:
00011458 31C0                   <1>     xor     eax, eax ; 0
0001145A A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>     ;mov    [u.error], eax
0001145F E92AB6FFFF             <1>     jmp     sysret
                                <1>
                                <1>
                                <1> syschmod: ; Get & Change File (or Directory) Attributes
                                <1>     ; 26/09/2024 - TRDOS 386 v2.0.9
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 19/01/2021
                                <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;           EBX = File/Directory (ASCIIZ) name address
                                <1>     ;           CL = New attributes (if CL < 40h)
                                <1>     ;           CL >= 40h -> Get File Attributes
                                <1>     ; OUTPUT ->
                                <1>     ;           cf = 0 -> EAX = File attributes (in AL)
                                <1>     ;           cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; MSDOS File Attributes:    (bit value of attrib byte)
                                <1>     ;       ATTR_READ_ONLY =      01h  (bit 0, 'R')
                                <1>     ;       ATTR_HIDDEN    =      02h  (bit 1, 'H')
                                <1>     ;       ATTR_SYSTEM    =      04h  (bit 2, 'S')
                                <1>     ;       ATTR_VOLUME_ID =      08h  (bit 3)
                                <1>     ;       ATTR_DIRECTORY =      10h  (bit 4)
                                <1>     ;       ATTR_ARCHIVE   =      20h  (bit 5, 'A')
                                <1>     ;       ATTR_LONG_NAME =      ATTR_READONLY |
                                <1>     ;                             ATTR_HIDDEN |
                                <1>     ;                             ATTR_SYSTEM |
                                <1>     ;                             ATTR_VOLUME_ID
                                <1>     ;       The upper two bits of attributes must be 0.
                                <1>
                                <1>     ; Note: * If ATTR_DIRECTORY is set, only directory names
                                <1>     ;         will be searched (and S,H,R,A attributeds of
                                <1>     ;         the directory will be changed.)
                                <1>     ;       * If ATTR_VOLUME_ID is set, 'syschmod' system call
                                <1>     ;         will return with 'permission denied' error.
                                <1>     ;       * If ATTR_DIRECTORY is not set, only file names
                                <1>     ;         will be searched (and S,H,R,A attributes of the
                                <1>     ;         file will be changed.)
```

```
                                <1>     ;
                                <1>     ; (Ony Super User can change S,H,R attributes.)
                                <1>
00011464 80F940                 <1>     cmp    cl, 40h
00011467 7327                   <1>     jnb    short syschmod_0
                                <1>
00011469 F6C108                 <1>     test   cl, 08h ; ATTR_VOLUME_ID
0001146C 750E                   <1>     jnz    short syschmod_perm_err
                                <1>
                                <1>     ; 19/01/2021
0001146E 803D[BE8E0100]00       <1>     cmp    byte [u.uid], 0  ; root (super user) ?
00011475 7619                   <1>     jna    short syschmod_0
                                <1>
                                <1>     ; Not super user..
00011477 F6C107                 <1>     test   cl, 07h ; S,H,R attributes
0001147A 7414                   <1>     jz     short syschmod_0
                                <1>
                                <1> syschmod_perm_err:
                                <1>     ;mov   dword [u.r0], ERR_PERM_DENIED
0001147C B80B000000             <1>     mov    eax, ERR_PERM_DENIED ; 'permission denied !'
00011481 A3[6C8E0100]           <1>     mov    [u.r0], eax
00011486 A3[D88E0100]           <1>     mov    [u.error], eax
0001148B E9DEB5FFFF             <1>     jmp    error
                                <1>
                                <1> syschmod_0:
00011490 880D[30810100]         <1>     mov    [Attributes], cl
00011496 89DE                   <1>     mov    esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov   ax, 1
                                <1>     ;mov   [FFF_Valid], ah ; 0 ; reset
                                <1>     ;call  set_working_path
00011498 E838080000             <1>     call   set_working_path_x
0001149D 7308                   <1>     jnc    short syschmod_1
0001149F 21C0                   <1>     and    eax, eax  ; 0 -> Bad Path!
000114A1 751E                   <1>     jnz    short syschmod_err
                                <1>     ; eax = 0
                                <1> syschmod_path_not_found:
                                <1>     ;mov   eax, ERR_INV_PATH_NAME ; 'Bad path name !'
                                <1>     ; 23/07/2022
000114A3 B013                   <1>     mov    al, ERR_INV_PATH_NAME
                                <1> ;syschmod_err:
                                <1>     ;mov   [u.r0], eax
                                <1>     ;mov   [u.error], eax
                                <1>     ;call  reset_working_path
                                <1>     ;jmp   error
                                <1>     ; 23/07/2022
000114A5 EB1A                   <1>     jmp    short syschmod_err
                                <1> syschmod_1:
000114A7 B008                   <1>     mov    al, 08h ; Except volume labels (& long names)
000114A9 A0[30810100]           <1>     mov    al, [Attributes]
000114AE 2410                   <1>     and    al, 10h ;
                                <1>     ;mov   esi, FindFile_Name
                                <1>     ;mov   ax, 1800h ; Only files
                                <1>     ;mov   ax, 0810h ; Only directories
000114B0 E8C574FFFF             <1>     call   find_first_file
                                <1>     ;jnc   short syschmod_2
000114B5 720A                   <1>     jc     short syschmod_err
                                <1>
                                <1>     ;; eax = 2 (File not found !)
                                <1>     ;cmp   al, 2 ; ERR_NOT_FOUND
                                <1>     ;jne   short syschmod_err
                                <1>
                                <1>     ;and   byte [Attributes], 10h
                                <1>     ;jz    short syschmod_err
                                <1>
                                <1>     ;; Directory not found !
                                <1>     ;mov   al, 3 ; ERR_PATH_NOT_FOUND
                                <1>     ;jmp   short syschmod_err
                                <1>
                                <1> syschmod_2:
000114B7 6621D2                 <1>     and    dx, dx ; Ambiguous filename chars used sign (DX>0)
000114BA 7419                   <1>     jz     short syschmod_3
000114BC B81A000000             <1>     mov    eax, ERR_INV_FILE_NAME ; 'invalid file name !'
                                <1>         ;jmp       short syschmod_err
                                <1> syschmod_err:
000114C1 A3[6C8E0100]           <1>     mov    [u.r0], eax
000114C6 A3[D88E0100]           <1>     mov    [u.error], eax
000114CB E8DA080000             <1>     call   reset_working_path
000114D0 E999B5FFFF             <1>     jmp    error
                                <1> syschmod_3:
                                <1>     ; EDI = Directory buffer entry offset/address
                                <1>     ; BL = File (or Directory) Attributes
                                <1>     ; mov  bl, [EDI+0Bh]
                                <1>
                                <1>     ; check directory attributes
000114D5 8A3D[30810100]         <1>     mov    bh, [Attributes] ; new attributes
000114DB 80FF40                 <1>     cmp    bh, 40h  ;>=40 -> get file/directory attributes
000114DE 7325                   <1>     jnb    short syschmod_6
                                <1>
                                <1>     ; set file/directory attributes
000114E0 F6C307                 <1>     test   bl, 7 ; system, hidden, readonly
000114E3 7409                   <1>         jz  short syschmod_4
                                <1>
                                <1>     ; 19/01/2021
000114E5 803D[BE8E0100]00       <1>     cmp    byte [u.uid], 0  ; root (super user) ?
000114EC 778E                   <1>     ja     short syschmod_perm_err
                                <1> syschmod_4:
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1>     cmp    word [edi+DirEntry_NTRes], 01A1h ; Singlix FS
                                <1>     je     short syschmod_7
                                <1> %endif
000114EE 887F0B                 <1>     mov    [edi+0Bh], bh    ; Attributes (New!)
                                <1>
000114F1 C605[9D7E0100]02       <1>     mov    byte [DirBuff_ValidData], 2 ; modified sign
                                <1>                                        ; to force write
000114F8 E89498FFFF             <1>     call   save_directory_buffer
000114FD 72C2                   <1>     jc     short syschmod_err
                                <1>
                                <1> syschmod_5:
000114FF 8A1D[30810100]         <1>     mov    bl, [Attributes]
                                <1> syschmod_6:
00011505 0FB6C3                 <1>     movzx  eax, bl
00011508 A3[6C8E0100]           <1>     mov    [u.r0], eax
                                <1>     ;mov   dword [u.error], 0
0001150D E97CB5FFFF             <1>     jmp    sysret
                                <1>
                                <1> ; 19/12/2025
                                <1> %if 0
                                <1> syschmod_7:
                                <1>     sub    eax, eax
                                <1>         mov     ah, [DirBuff_DRV]
                                <1>     ; 26/09/2024 (BugFix)
                                <1>     sub    ah, 'A'
```

```
                                <1>     mov     esi, Logical_DOSDisks
                                <1>         add     esi, eax
                                <1>         cmp     byte [esi+LD_FSType], 0A1h
                                <1>     jnc     short syschmod_8
                                <1>     mov     al, ERR_INV_DATA ; 29 = Invalid Data
                                <1>     jmp     short syschmod_err
                                <1>
                                <1> syschmod_8:
                                <1>     ; BH = New MS-DOS File Attributes
                                <1>     mov     al, bh ; File/Directory Attributes
                                <1>     xor     ah, ah ; Attributes in MS-DOS format sign
                                <1>     call    change_fs_file_attributes
                                <1>     ;jc     syschmod_err
                                <1>     ;jmp    short syschmod_5
                                <1>     ; 23/07/2022
                                <1>     jnc     short syschmod_5
                                <1>     jmp     syschmod_err ; 26/09/2024
                                <1> %endif
                                <1>
                                <1> sysdrive: ; Get/Set Current (Working) Drive (for user)
                                <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;         BL = Logical DOS Drive number (0=A: ... 2=C:)
                                <1>     ;         If BL = 0FFh -> Get Current Drive
                                <1>     ; OUTPUT ->
                                <1>     ;         cf = 0 ->
                                <1>     ;                AL = Current Drive number
                                <1>     ;                AH = The Last Logical DOS Drive no.
                                <1>     ;         cf = 1 -> Error code in AL
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; NOTE: If the requested logical dos drive is ready,
                                <1>     ;      it's current current directory will be the user's
                                <1>     ;      (program's) current directory.
                                <1>     ;      (When the program is terminated, MainProg -internal
                                <1>     ;      shell- will reset the previous -current- logical drive
                                <1>     ;       as current drive again).
                                <1>
00011512 80FBFF                 <1>     cmp     bl, 0FFh
00011515 7435                   <1>     je      short sysdrive_ok
00011517 3A1D[8D2E0100]         <1>     cmp     bl, [Last_DOS_DiskNo]
0001151D 771E                   <1>     ja      short sysdrive_err
                                <1>
                                <1>     ; Save current drive and reset mode
                                <1>     ; for 'reset_working_path' procedure (for MainProg)
0001151F 30C0                   <1>     xor     al, al
00011521 66A3[70830100]         <1>     mov     [SWP_Mode], ax ; ah = 0
00011527 A0[76770100]           <1>     mov     al, [Current_Drv]
0001152C FEC4                   <1>     inc     ah ; mov ah, 1
0001152E 66A3[72830100]         <1>     mov     [SWP_DRV], ax
                                <1>
00011534 88DA                   <1>     mov     dl, bl
00011536 E8B561FFFF             <1>     call    change_current_drive
0001153B 730F                   <1>     jnc     short sysdrive_ok
                                <1> sysdrive_err:
0001153D C705[6C8E0100]0F00-    <1>     mov     dword [u.r0], ERR_DRV_NOT_RDY ; 'drive not ready !'
00011545 0000                   <1>
00011547 E922B5FFFF             <1>     jmp     error
                                <1> sysdrive_ok:
0001154C A0[76770100]           <1>     mov     al, [Current_Drv]
00011551 8A25[8D2E0100]         <1>     mov     ah, [Last_DOS_DiskNo]
00011557 A3[6C8E0100]           <1>     mov     [u.r0], eax
0001155C E92DB5FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdir: ; Get Current (Working) Drive & Directory (for user)
                                <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;         EBX = Current directory name buffer address
                                <1>     ;             (Buffer length = 92 bytes)
                                <1>     ; OUTPUT ->
                                <1>     ;         AL = Current drive (0=A: .. 2=C:)
                                <1>     ;         If CF = 1 -> AL = error code
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; Note: Required directory name buffer length may be
                                <1>     ;      <= 92 bytes for current TRDOS 386 version.
                                <1>     ;      (7*12 name chars + 7 slash + 0)
                                <1>
00011561 89E5                   <1>     mov     ebp, esp
00011563 83EC60                 <1>     sub     esp, 96
00011566 53                     <1>     push    ebx ; User's buffer address
00011567 30D2                   <1>     xor     dl, dl ; 0 = current drive
00011569 E8238EFFFF             <1>     call    get_current_directory
0001156E 72CD                   <1>     jc      short sysdrive_err ; 'drive not ready !' error
00011570 89E6                   <1>     mov     esi, esp ; System's buffer address
00011572 5F                     <1>     pop     edi  ; User's buffer address
                                <1>     ; ecx = transfer (byte) count (<=92)
00011573 E8A3F5FFFF             <1>     call    transfer_to_user_buffer
00011578 89EC                   <1>     mov     esp, ebp
0001157A 730F                   <1>     jnc     short sysdir_ok
                                <1> sysdir_err:
0001157C C705[6C8E0100]2E00-    <1>     mov     dword [u.r0], ERR_BUFFER  ; 'buffer error !'
00011584 0000                   <1>
00011586 E9E3B4FFFF             <1>     jmp     error
                                <1> sysdir_ok:
0001158B 8A0D[76770100]         <1>     mov     cl, [Current_Drv]
00011591 890D[6C8E0100]         <1>     mov     [u.r0], ecx
00011597 E9F2B4FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysldrvt: ; Get copy of Logical DOS Drive Description Table
                                <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>     ;         BL = Logical DOS drive number (zero based)
                                <1>         ;         ECX = Logical DOS drv desc table buffer addr
                                <1>     ;             (Buffer length = 256 bytes)
                                <1>     ; OUTPUT ->
                                <1>     ;         cf = 0 ->
                                <1>     ;                AL = Current Drive number
                                <1>     ;                AH = The Last Logical DOS Drive no.
                                <1>     ;         cf = 1 -> Error code in AL
                                <1>     ;                AH = The Last Logical DOS Drive no.
                                <1>     ;
                                <1>     ; Modified Registers: EAX (at the return of system call)
                                <1>     ;
                                <1>     ; Note: Required description table buffer length is
                                <1>     ;      256 bytes for current TRDOS 386 version.
                                <1>
0001159C 89CF                   <1>     mov     edi, ecx ; Destination address (user space)
0001159E 88DC                   <1>     mov     ah, bl
```

```
000115A0 30C0                <1>     xor    al, al
000115A2 BE00010900          <1>     mov    esi, Logical_DOSDisks
000115A7 01C6                <1>     add    esi, eax ; Source address (system space)
000115A9 B900010000          <1>     mov    ecx, 256 ; Byte count
                             <1>                     ; Logical Dos Drv Desc Table size
000115AE E868F5FFFF          <1>     call   transfer_to_user_buffer
000115B3 72C7                <1>     jc     short sysdir_err
000115B5 8A2D[8D2E0100]      <1>     mov    ch, [Last_DOS_DiskNo]
000115BB EBCE                <1>     jmp    short sysdir_ok
                             <1>
                             <1> systime: ; Get System Date&Time
                             <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>        ; INPUT -> BL =
                             <1>     ;          0 = Get Date&Time in Unix/Epoch format
                             <1>     ;          1 = Get Time in MSDOS format
                             <1>     ;          2 = Get Date in MSDOS format
                             <1>     ;          3 = Get Date&Time in MSDOS format
                             <1>     ;          4 & other values =
                             <1>     ;             System timer ticks will be returned
                             <1>     ;             in EAX and Carry Flag will be set.
                             <1>     ;             (CF will not be set if BL = 4)
                             <1>     ; OUTPUT ->
                             <1>     ;      For BL input = 3
                             <1>     ;          EAX = Current Time (RTC)
                             <1>     ;             AL = Second (DL in MSDOS)
                             <1>     ;             AH = Minute (CL in MSDOS)
                             <1>     ;             HW of EAX = Hour (CH in MSDOS)
                             <1>     ;         EDX = Current System Date (RTC)
                             <1>     ;             DL = Day (DL in MSDOS)
                             <1>     ;             DH = Month (DH in MSDOS)
                             <1>     ;             HW of EDX = Year (CX in MSDOS)
                             <1>     ;
                             <1>     ;      For BL input = 2
                             <1>     ;         EAX = Current System Date (RTC)
                             <1>     ;             DL = Day (DL in MSDOS)
                             <1>     ;             DH = Month (DH in MSDOS)
                             <1>     ;             HW of EDX = Year (CX in MSDOS)
                             <1>     ;
                             <1>     ;      For BL input = 1
                             <1>     ;          EAX = Current Time (RTC)
                             <1>     ;             AL = Second (DL in MSDOS)
                             <1>     ;             AH = Minute (CL in MSDOS)
                             <1>     ;             HW of EAX = Hour (CH in MSDOS)
                             <1>     ;
                             <1>     ;      For BL input = 0
                             <1>     ;          EAX = Unix (Epoch) Time Ticks/Seconds
                             <1>     ;
                             <1>     ;      For BL input  = 4
                             <1>     ;         EAX = System timer ticks
                             <1>     ;
                             <1>     ;      If CF = 1 (for other values of BL input)
                             <1>     ;         EAX = System timer ticks (no error code!)
                             <1>     ;
                             <1>     ; Modified Registers: EAX, (EDX)
                             <1>     ;              (at the return of system call)
                             <1>     ;
                             <1>
000115BD 20DB                <1>     and    bl, bl
000115BF 750F                <1>     jnz    short systime_1
000115C1 E81B59FFFF          <1>     call   epoch
                             <1> systime_0:
000115C6 A3[6C8E0100]        <1>     mov    [u.r0], eax
000115CB E9BEB4FFFF          <1>     jmp    sysret
                             <1> systime_1:
000115D0 80FB04              <1>     cmp    bl, 4
000115D3 7211                <1>     jb     short systime_2
000115D5 A1[30770100]        <1>     mov    eax, [TIMER_LH] ; 18.2 Hz timer ticks
                             <1>                            ; Note: [TIMER_LH] may be set
                             <1>                            ; to wrong timer value due to
                             <1>                            ; program functions.
                             <1>                            ; (This value must not be
                             <1>                            ; accepted as [TIMER_LH]/18.2
                             <1>                            ; seconds since the midnight.)
000115DA 76EA                <1>     jna    short systime_0
000115DC A3[6C8E0100]        <1>     mov    [u.r0], eax
000115E1 E988B4FFFF          <1>     jmp    error ; cf = 1 & [u.r0] = eax = timer ticks
                             <1>
                             <1> systime_2:
                             <1>     ;push  ebx
000115E6 E87058FFFF          <1>     call   get_rtc_date_time
                             <1>     ;pop   ebx
000115EB F6C301              <1>     test   bl, 1
000115EE 7429                <1>     jz     short systime_4
000115F0 30E4                <1>     xor    ah, ah
000115F2 A0[7A730100]        <1>     mov    al, [hour]
000115F7 88C2                <1>     mov    dl, al
000115F9 C1E010              <1>     shl    eax, 16
000115FC A0[7E730100]        <1>     mov    al, [second]
00011601 8A25[7C730100]      <1>     mov    ah, [minute]
00011607 F6C302              <1>     test   bl, 2
0001160A 74BA                <1>     jz     short systime_0
                             <1>     ; Check time & date match risk
                             <1>     ; (23:59:59 may cause to wrong
                             <1>     ; date -new day with previous date-...)
0001160C 80FA17              <1>     cmp    dl, 23
0001160F 7206                <1>     jb     short systime_3
00011611 663D3B3B            <1>     cmp    ax, (59*256)+59 ; if hour is 23:59:59
00011615 73CF                <1>     jnb    short systime_2 ; wait for 1 second
                             <1> systime_3:
                             <1>     ; eax = time
00011617 89C6                <1>     mov    esi, eax
                             <1> systime_4:
00011619 66A1[74730100]      <1>     mov    ax, [year]
0001161F C1E010              <1>     shl    eax, 16
00011622 A0[78730100]        <1>     mov    al, [day]
00011627 8A25[76730100]      <1>     mov    ah, [month]
                             <1>     ; eax = date
0001162D 80E301              <1>     and    bl, 1
00011630 7494                <1>     jz     short systime_0
00011632 96                  <1>     xchg   esi, eax
                             <1>     ; eax = time, esi = date
00011633 8B2D[688E0100]      <1>     mov    ebp, [u.usp]  ; EBP points to user's registers
                             <1>     ; (user) edx <-- (system) esi
00011639 897514              <1>     mov    [ebp+20], esi ; return to user with EDX value
0001163C EB88                <1>     jmp    short systime_0
                             <1>
                             <1> sysstime: ; Set System Date&Time
                             <1>     ; 31/12/2017
                             <1>     ; 30/12/2017 (TRDOS 386 = TRDOS v2.0)
                             <1>     ;
                             <1>        ; INPUT -> BL =
                             <1>     ;          0 = Set Date&Time in Unix/Epoch format
                             <1>     ;          1 = Set Time in MSDOS format
```

```
                                <1>     ;          2 = Set Date in MSDOS format
                                <1>     ;          3 = Set Date&Time in MSDOS format
                                <1>     ;          4 = Set System Timer (Ticks)
                                <1>     ;          5 = Convert/Save current time to/as
                                <1>     ;             18.2 Hz system timer ticks
                                <1>     ;          6 = Convert MSDOS Date&Time to UNIX format
                                <1>     ;             without setting system date&time ; (test)
                                <1>     ;          7 = Convert UNIX Date&Time to MSDOS format
                                <1>     ;             without setting system date&time ; (test)
                                <1>     ;         8-0FFh = invalid !
                                <1>     ;        ECX = Time (or Timer) value in selected format
                                <1>     ;        EDX = Date value in MSDOS format if BL=2,3,6
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      If CF = 0 ->
                                <1>     ;          EAX = Set value
                                <1>     ;      If CF = 1 -> (invalid BL input)
                                <1>     ;         EAX = Ticks count [TIMER_LH]
                                <1>     ;
                                <1>
0001163E 20DB                   <1>     and    bl, bl ; 0
00011640 7511                   <1>     jnz    short sysstime_0
00011642 89C8                   <1>     mov    eax, ecx
00011644 E81D59FFFF             <1>     call   convert_from_epoch
00011649 E8BD59FFFF             <1>     call   set_rtc_date_time
0001164E E93BB4FFFF             <1>     jmp    sysret
                                <1> sysstime_0:
00011653 80FB08                 <1>     cmp    bl, 8
00011656 722D                   <1>     jb     short sysstime_1
                                <1>     ; invalid input (>7)
00011658 A1[30770100]           <1>     mov    eax, [TIMER_LH] ; 18.2 Hz timer ticks
                                <1>                            ; Note: [TIMER_LH] may be set
                                <1>                            ; to wrong timer value due to
                                <1>                            ; program functions.
                                <1>                            ; (This value must not be
                                <1>                            ; accepted as [TIMER_LH]/18.2
                                <1>                            ; seconds since the midnight.)
0001165D A3[6C8E0100]           <1>     mov    [u.r0], eax
00011662 E907B4FFFF             <1>     jmp    error ; cf = 1 & [u.r0] = eax = timer ticks
                                <1>
                                <1> sysstime_8:
                                <1>     ; BL = 7
00011667 89C8                   <1>     mov    eax, ecx ; seconds since 1/1/1970 00:00:00
00011669 E8F858FFFF             <1>     call   convert_from_epoch
0001166E 30E4                   <1>     xor    ah, ah
00011670 A0[7A730100]           <1>     mov    al, [hour]
00011675 C1E010                 <1>     shl    eax, 16
00011678 A0[7E730100]           <1>     mov    al, [second]
0001167D 8A25[7C730100]         <1>     mov    ah, [minute]
00011683 EB92                   <1>     jmp    short systime_3
                                <1>
                                <1> sysstime_1:
00011685 80FB04                 <1>     cmp    bl, 4
00011688 743F                   <1>     je     short sysstime_2 ; set system timer ticks
0001168A 80FB05                 <1>     cmp    bl, 5
0001168D 754B                   <1>     jne    short sysstime_4
                                <1>     ; convert current time to system timer ticks (18.2Hz)
0001168F E8C757FFFF             <1>     call   get_rtc_date_time
00011694 0FB60D[7A730100]       <1>     movzx  ecx, byte [hour]
0001169B B8100E0000             <1>     mov    eax, 60*60 ; 1 hour = 3600 seconds
000116A0 F7E1                   <1>     mul    ecx
000116A2 89C3                   <1>     mov    ebx, eax
000116A4 B13C                   <1>     mov    cl, 60  ; 1 minute = 60 seconds
000116A6 0FB605[7C730100]       <1>     movzx  eax, byte [minute]
000116AD F7E1                   <1>     mul    ecx
000116AF 01D8                   <1>     add    eax, ebx
000116B1 8A0D[7E730100]         <1>     mov    cl, [second]
000116B7 01C8                   <1>     add    eax, ecx
000116B9 B1B6                   <1>     mov    cl, 182
000116BB F7E1                   <1>     mul    ecx
000116BD 83C009                 <1>     add    eax, 9
000116C0 83D200                 <1>     adc    edx, 0
000116C3 B10A                   <1>     mov    cl, 10
000116C5 F7F1                   <1>     div    ecx
                                <1>     ; eax = ((182*seconds)+9)/10
000116C7 89C1                   <1>     mov    ecx, eax
                                <1> sysstime_2:
000116C9 890D[30770100]         <1>     mov    [TIMER_LH], ecx ; 18.2 * seconds
                                <1> sysstime_3:
000116CF 890D[6C8E0100]         <1>     mov    [u.r0], ecx
000116D5 E9B4B3FFFF             <1>     jmp    sysret
                                <1> sysstime_4:
000116DA 80FB06                 <1>     cmp    bl, 6
000116DD 7788                   <1>     ja     short sysstime_8
                                <1>
000116DF 890D[6C8E0100]         <1>     mov    [u.r0], ecx
                                <1>
000116E5 880D[7E730100]         <1>     mov    [second], cl
000116EB 882D[7C730100]         <1>     mov    [minute], ch
000116F1 C1E910                 <1>     shr    ecx, 16
000116F4 880D[7A730100]         <1>     mov    [hour], cl
                                <1>     ; BL = 1,2,3,6
000116FA 80FB01                 <1>     cmp    bl, 1
000116FD 762A                   <1>     jna    short sysstime_5
                                <1>     ; BL = 2,3,6
000116FF 8815[78730100]         <1>     mov    [day], dl
00011705 8835[76730100]         <1>     mov    [month], dh
0001170B C1EA10                 <1>     shr    edx, 16
0001170E 668915[74730100]       <1>     mov    [year], dx
00011715 80E303                 <1>     and    bl, 3
00011718 742D                   <1>     jz     short sysstime_7 ; 6
                                <1>     ; BL = 2,3
0001171A F6C301                 <1>     test   bl, 1
0001171D 7419                   <1>     jz     short sysstime_6 ; 2
                                <1>     ; BL = 3
0001171F E8E758FFFF             <1>     call   set_rtc_date_time
00011724 E965B3FFFF             <1>     jmp    sysret
                                <1> sysstime_5:
                                <1>     ; BL = 1
00011729 E81E59FFFF             <1>     call   set_time_bcd
0001172E E8584CFFFF             <1>     call   set_rtc_time
00011733 E956B3FFFF             <1>     jmp    sysret
                                <1> sysstime_6:
                                <1>     ; BL = 2
00011738 E8E258FFFF             <1>     call   set_date_bcd
0001173D E8AC4CFFFF             <1>     call   set_rtc_date
00011742 E947B3FFFF             <1>     jmp    sysret
                                <1> sysstime_7:
                                <1>     ; BL = 6
                                <1>     ; [year], [month], [day],
                                <1>     ; [hour], [minute], [second]
00011747 E89A57FFFF             <1>     call   convert_to_epoch
0001174C 89C1                   <1>     mov    ecx, eax ; seconds since 1/1/1970 00:00:00
0001174E E97CFFFFFF             <1>     jmp    sysstime_3
```

```
                                <1>
                                <1> sysrename: ; Rename File (or Directory)
                                <1>     ; 19/12/2025 - TRDOS 386 v2.0.10
                                <1>     ; 08/08/2022
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 19/01/2021
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>         ;          EBX = File/Directory (ASCIIZ) name address
                                <1>     ;         ECX = New name (in same dir, no path name)
                                <1>     ; OUTPUT ->
                                <1>     ;          cf = 0 -> EAX = 0
                                <1>     ;          cf = 1 -> Error code in AL
                                <1>
                                <1>     ; 19/01/2021
00011753 803D[BE8E0100]00       <1>     cmp    byte [u.uid], 0  ; root (super user) ?
                                <1>     ;jna   short sysrename_0
                                <1>     ; 23/07/2022
0001175A 7717                   <1>     ja     short sysrename_perm_err
                                <1>
                                <1> ;sysrename_perm_err:
                                <1> ;   ;mov   dword [u.r0], ERR_PERM_DENIED
                                <1> ;   mov    eax, ERR_PERM_DENIED ; 'permission denied !'
                                <1> ;   mov    [u.r0], eax
                                <1> ;   mov    [u.error], eax
                                <1> ;   jmp    error
                                <1>
                                <1> sysrename_0:
0001175C 51                     <1>     push   ecx ; new file name address (in user space)
0001175D 89DE                   <1>     mov    esi, ebx
                                <1>     ; file name is forced, change directory as temporary
                                <1>     ;mov   ax, 1
                                <1>     ;mov   [FFF_Valid], ah ; 0 ; reset
                                <1>     ;call  set_working_path
0001175F E871050000             <1>     call   set_working_path_x
00011764 7321                   <1>     jnc    short sysrename_1
00011766 21C0                   <1>     and    eax, eax  ; 0 -> Bad Path!
00011768 753B                   <1>     jnz    short sysrename_err
                                <1>     ; eax = 0
                                <1> sysrename_path_not_found:
0001176A B813000000             <1>     mov    eax, ERR_INV_PATH_NAME ; 'Bad path name !'
                                <1>     ; 23/07/2022
0001176F B013                   <1>     mov    al, ERR_INV_PATH_NAME
00011771 EB32                   <1>     jmp    short sysrename_err
                                <1> ;sysrename_err:
                                <1> ;   pop    ecx ; new file name address (in user space)
                                <1> ;sysrename_error:
                                <1> ;   mov    [u.r0], eax
                                <1> ;   mov    [u.error], eax
                                <1> ;   call   reset_working_path
                                <1> ;   jmp    error
                                <1>
                                <1>     ; 23/07/2022
                                <1> sysrename_perm_err:
                                <1>     ;mov   dword [u.r0], ERR_PERM_DENIED
00011773 B80B000000             <1>     mov    eax, ERR_PERM_DENIED ; 'permission denied !'
00011778 A3[6C8E0100]           <1>     mov    [u.r0], eax
0001177D A3[D88E0100]           <1>     mov    [u.error], eax
00011782 E9E7B2FFFF             <1>     jmp    error
                                <1>
                                <1> sysrename_1:
00011787 B008                   <1>     mov    al, 08h ; Except volume labels (& long names)
00011789 A0[30810100]           <1>     mov    al, [Attributes]
0001178E 2410                   <1>     and    al, 10h ;
                                <1>     ;mov   esi, FindFile_Name
                                <1>     ;mov   ax, 1800h ; Only files
                                <1>     ;mov   ax, 0810h ; Only directories
00011790 66B80008               <1>     mov    ax, 0800h ; Find File or Directory
00011794 E8E171FFFF             <1>     call   find_first_file
                                <1>     ;jnc   short sysrename_2
00011799 720A                   <1>     jc     short sysrename_err
                                <1> sysrename_2:
                                <1>     ; ESI = Directory Entry (FindFile_DirEntry) Location
                                <1>     ; EDI = Directory Buffer Directory Entry Location
                                <1>     ; EAX = File Size
                                <1>     ;  BL = Attributes of The File/Directory
                                <1>     ;  BH = Long Name Yes/No Status (>0 is YES)
                                <1>     ;  DX > 0 : Ambiguous filename chars are used
                                <1>
0001179B 6621D2                 <1>     and    dx, dx ; Ambiguous filename chars used sign (DX>0)
0001179E 741A                   <1>     jz     short sysrename_3
000117A0 B81A000000             <1>     mov    eax, ERR_INV_FILE_NAME ; 'invalid file name !'
                                <1>     ;jmp   short sysrename_err
                                <1>     ; 23/07/2022
                                <1> sysrename_err:
000117A5 59                     <1>     pop    ecx ; new file name address (in user space)
                                <1> sysrename_error:
000117A6 A3[6C8E0100]           <1>     mov    [u.r0], eax
000117AB A3[D88E0100]           <1>     mov    [u.error], eax
000117B0 E8F5050000             <1>     call   reset_working_path
000117B5 E9B4B2FFFF             <1>     jmp    error
                                <1> sysrename_3:
                                <1>     ; EDI = Directory buffer entry offset/address
                                <1>     ; BL = File (or Directory) Attributes
                                <1>     ; mov  bl, [EDI+0Bh]
                                <1>
000117BA 5A                     <1>     pop    edx ; new file name address (in user space)
                                <1>
                                <1>     ; check file/directory attributes
000117BB F6C307                 <1>     test   bl, 7 ; system, hidden, readonly
000117BE 75B3                   <1>         jnz short sysrename_perm_err
                                <1> sysrename_4:
                                <1>     ; 19/12/2025
                                <1>     ;cmp   word [edi+DirEntry_NTRes], 01A1h ; Singlix FS
                                <1>     ;je    short sysrename_perm_err ; -temporary!-
                                <1>
                                <1>     ; save old file name & file info (FFF structure)
000117C0 BE[16800100]           <1>     mov    esi, FindFile_Drv
000117C5 BF[60810100]           <1>     mov    edi, SourceFile_Drv
                                <1>     ;mov   ecx, 128/4
                                <1>     ; 23/07/2022
000117CA 29C9                   <1>     sub    ecx, ecx
000117CC B120                   <1>     mov    cl, 128/4
000117CE F3A5                   <1>     rep    movsd
                                <1>
000117D0 89D6                   <1>     mov    esi, edx ; new file name address (in user space)
000117D2 BF[E0810100]           <1>     mov    edi, DestinationFile_Drv
000117D7 E8A291FFFF             <1>     call   parse_path_name
000117DC 72C8                   <1>     jc     short sysrename_error ; eax = 1 (Bad file name)
                                <1>
                                <1>     ; same drive ?
000117DE A0[16800100]           <1>     mov    al, [FindFile_Drv]
000117E3 3A05[E0810100]         <1>     cmp    al, [DestinationFile_Drv]
```

```
                                <1>     ;jne    short sysrename_perm_err ; Permission denied
000117E9 7509                   <1>     jne     short sysrename_5 ; Bad file name
                                <1>
                                <1>     ; no path name !? (rename file in same directory)
000117EB 803D[E1810100]20       <1>     cmp     byte [DestinationFile_Directory], 20h
000117F2 7607                   <1>     jna     short sysrename_6
                                <1> sysrename_5:
000117F4 B801000000             <1>     mov     eax, ERR_BAD_CMD_ARG ; 1 = Bad file name
                                <1>                                  ; (Bad argument)
000117F9 EBAB                   <1>     jmp     short sysrename_error
                                <1> sysrename_6:
000117FB 803D[22820100]20       <1>     cmp     byte [DestinationFile_Name], 20h
00011802 76F0                   <1>     jna     short sysrename_5
                                <1>
00011804 BE[22820100]           <1>     mov     esi, DestinationFile_Name
00011809 E8CB74FFFF             <1>     call    check_filename ; is it a valid msdos file name?
0001180E 7296                   <1>     jc      short sysrename_error ; 26 = ERR_INV_FILE_NAME
                                <1>
                                <1>     ;mov    esi, DestinationFile_Name
00011810 66B80008               <1>     mov     ax, 0800h ; Find File or Directory
00011814 E86171FFFF             <1>     call    find_first_file
00011819 7207                   <1>     jc      short sysrename_7
                                <1>
0001181B B80E000000             <1>     mov     eax, ERR_FILE_EXISTS  ; file already exists !
                                <1> jmp_sysrename_err: ; 08/08/2022
00011820 EB84                   <1>     jmp     sysrename_error
                                <1> sysrename_7:
                                <1>     ; eax = 2 (File not found !)
00011822 3C02                   <1>     cmp     al, 2 ; ERR_NOT_FOUND
                                <1>     ;jne    short sysrename_error
                                <1>     ; 08/08/2022
00011824 75FA                   <1>     jne     short jmp_sysrename_err
                                <1>
                                <1>     ; 31/12/2017
                                <1>     ; Following code is also part of 'rename_file' in
                                <1>     ; 'trdosk3.s' (MainProg's 'rename' command) ; 13/11/2017
00011826 BE[22820100]           <1>     mov     esi, DestinationFile_Name ; (Rename_NewName)
0001182B 668B0D[DA810100]       <1>     mov     cx, [SourceFile_DirEntryNumber]
00011832 66A1[C6810100]         <1>     mov     ax, [SourceFile_DirEntry+20] ; First Cluster, HW
00011838 C1E010                 <1>     shl     eax, 16
0001183B 66A1[CC810100]         <1>     mov     ax, [SourceFile_DirEntry+26] ; First Cluster, LW
00011841 0FB61D[AF810100]       <1>     movzx   ebx, byte [SourceFile_LongNameEntryLength]
00011848 E8E098FFFF             <1>     call    rename_directory_entry
                                <1>     ;jc     short sysrename_error
                                <1>     ; 08/08/2022
0001184D 72D1                   <1>     jc      short jmp_sysrename_err
                                <1>     ;xor    eax, eax
0001184F A3[6C8E0100]           <1>     mov     [u.r0], eax ; 0
                                <1>     ;mov    [u.error], eax
00011854 E851050000             <1>     call    reset_working_path
00011859 E930B2FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysmem: ; Get Total&Free Memory amount
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>     ;       none
                                <1>     ; OUTPUT ->
                                <1>     ;       EAX = Total memory count (in bytes)
                                <1>     ;       EBX = Virtually available memory amount (in bytes)
                                <1>     ;             = 4GB - CORE (4MB)
                                <1>     ;       ECX = Free memory count (in bytes)
                                <1>     ;       EDX = Calculated free memory count (in bytes)
                                <1>
0001185E A1[B4760100]           <1>     mov     eax, [memory_size] ; in pages
00011863 C1E00C                 <1>     shl     eax, 12            ; in bytes
00011866 A3[6C8E0100]           <1>     mov     [u.r0], eax
0001186B E8CB29FFFF             <1>     call    calc_free_mem
                                <1>     ; edx = calculated free pages
                                <1>     ; ecx = 0
00011870 8B2D[688E0100]         <1>     mov     ebp, [u.usp]  ; EBP points to user's registers
00011876 C745100000C0FF         <1>     mov     dword [ebp+16], ECORE ; EBX (for user)
                                <1>                             ; 0FFC00000h ; 4GB - 4MB
0001187D C1E20C                 <1>     shl     edx, 12
00011880 895514                 <1>     mov     [ebp+20], edx ; EDX (for user)
00011883 8B0D[B8760100]         <1>     mov     ecx, [free_pages]
00011889 C1E10C                 <1>     shl     ecx, 12 ; free bytes
0001188C 894D18                 <1>     mov     [ebp+24], ecx ; ECX (for user)
                                <1>     ;mov    [free_pages], edx
0001188F E9FAB1FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysprompt:
                                <1>     ; Set TRDOS 386 Command Interpreter (MainProg) prompt
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>     ;       EBX = 0 -> use default prompt
                                <1>     ;       EBX > 0 -> prompt string (ASCIIZ) address
                                <1>     ;                  (Max. 11 characters except ZERO tail)
                                <1>     ; OUTPUT ->
                                <1>     ;       (EAX = 0)
                                <1>     ;       CF = 0 -> Successful
                                <1>     ;       CF = 1 -> Failed
                                <1>
00011894 21DB                   <1>     and     ebx, ebx
00011896 750A                   <1>     jnz     short sysprompt_0
                                <1>
00011898 E8C46AFFFF             <1>     call    default_command_prompt ; ['+'TRDOS'+']'
0001189D E9ECB1FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysprompt_0:
000118A2 31C0                   <1>     xor     eax, eax
000118A4 A3[6C8E0100]           <1>     mov     [u.r0], eax
000118A9 89DE                   <1>     mov     esi, ebx
                                <1>     ;mov    ecx, 12
                                <1>     ; 30/07/2022
000118AB 31C9                   <1>     xor     ecx, ecx
000118AD B10C                   <1>     mov     cl, 12
000118AF 89E5                   <1>     mov     ebp, esp
000118B1 29CC                   <1>     sub     esp, ecx
000118B3 49                     <1>     dec     ecx ; 11
000118B4 89E7                   <1>     mov     edi, esp
000118B6 E8AAF2FFFF             <1>     call    transfer_from_user_buffer
000118BB 7211                   <1>     jc      short sysprompt_err
000118BD 803E20                 <1>     cmp     byte [esi], 20h
000118C0 760C                   <1>     jna     short sysprompt_err
000118C2 E8AC6AFFFF             <1>     call    set_command_prompt
000118C7 89EC                   <1>     mov     esp, ebp
000118C9 E9C0B1FFFF             <1>     jmp     sysret
                                <1> sysprompt_err:
                                <1> syspath_err:
000118CE 89EC                   <1>     mov     esp, ebp
000118D0 E999B1FFFF             <1>     jmp     error
```

```
                                <1>
                                <1> syspath:
                                <1>     ; Get/Set Run Path
                                <1>     ;
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>     ;       EBX = 0 -> get path (to buffer address in ECX)
                                <1>     ;       EBX > 0 -> set path
                                <1>     ;           EBX = Path string buffer address (ASCIIZ)
                                <1>     ;                 (Path description except 'PATH=')
                                <1>     ;       ECX = Buffer address (if EBX = 0)
                                <1>     ;             (ECX will not be used if EBX > 0)
                                <1>     ;       DL = Buffer size (0 = 256 byte)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 0 -> Successful (EAX = String length)
                                <1>     ;       CF = 1 -> Failed (EAX = 0)
                                <1>     ;
                                <1>     ; NOTE: 'PATH=' or 'PATH' must be excluded
                                <1>     ;  (It must not be at the beginning of the string.)
                                <1>
000118D5 89E5                   <1>     mov     ebp, esp
000118D7 81EC00010000           <1>     sub     esp, 256
000118DD 89E7                   <1>     mov     edi, esp
                                <1>
000118DF 31C0                   <1>     xor     eax, eax
000118E1 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>
000118E6 21DB                   <1>     and     ebx, ebx
000118E8 752A                   <1>     jnz     short syspath_0
                                <1>
                                <1>     ; EBX = 0 -> get run path
000118EA 89CB                   <1>     mov     ebx, ecx ; buffer addr (in user's mem space)
000118EC BE[632F0100]           <1>     mov     esi, Cmd_Path  ; 'PATH' address
000118F1 0FB6CA                 <1>     movzx   ecx, dl
                                <1>     ;sub    cl, 1 ; 0 -> 255, 1 -> 0
                                <1>     ;adc    cx, 1 ; 255 -> 256, 0 -> 1
                                <1>     ; 30/07/2022
000118F4 FEC9                   <1>     dec     cl ; 0 -> 255, 1 -> 0
000118F6 41                     <1>     inc     ecx ; 255 -> 256, 0 -> 1
                                <1>     ; EDI = Output buffer
                                <1>     ; CX = Buffer length
                                <1>     ; AL = 0 -> use ASCIIZ word in [ESI]
                                <1>     ; ESI = 'PATH' address (with zero tail)
000118F7 E8BB81FFFF             <1>     call    get_environment_string
000118FC 72D0                   <1>     jc      short syspath_err
000118FE 89DF                   <1>     mov     edi, ebx ; User's buffer address
00011900 89E6                   <1>     mov     esi, esp
                                <1>     ; EDI = User's buffer address
                                <1>     ; ECX = transfer (byte) count
00011902 E814F2FFFF             <1>     call    transfer_to_user_buffer
00011907 72C5                   <1>     jc      short syspath_err
00011909 890D[6C8E0100]         <1>     mov     [u.r0], ecx
0001190F E97AB1FFFF             <1>     jmp     sysret
                                <1>
                                <1> syspath_0:
00011914 89DE                   <1>     mov     esi, ebx
00011916 0FB6CA                 <1>     movzx   ecx, dl
                                <1>     ;sub    cl, 1 ; 0 -> 255, 1 -> 0
                                <1>     ;adc    cx, 1 ; 255 -> 256, 0 -> 1
                                <1>     ; 30/07/2022
00011919 FEC9                   <1>     dec     cl ; 0 -> 255, 1 -> 0
0001191B 41                     <1>     inc     ecx ; 255 -> 256, 0 -> 1
0001191C E844F2FFFF             <1>     call    transfer_from_user_buffer
00011921 72AB                   <1>     jc      short syspath_err
                                <1>     ;(*) 'PATH=' will be added to
                                <1>     ;         the head of the string
00011923 83EC08                 <1>     sub     esp, 8 ;(*)
00011926 89FE                   <1>     mov     esi, edi ;(*)
00011928 E86E81FFFF             <1>     call    set_path_x ;(*)
0001192D 729F                   <1>     jc      short syspath_err
0001192F 8915[6C8E0100]         <1>     mov     [u.r0], edx ; run path string length
00011935 E954B1FFFF             <1>     jmp     sysret
                                <1>
                                <1> sysenv:
                                <1>     ; Get/Set Environment Variables
                                <1>     ;
                                <1>     ; 30/07/2022
                                <1>     ; 23/07/2022 - TRDOS 386 v2.0.5
                                <1>     ; 31/12/2017 (TRDOS 386 = TRDOS v2.0)
                                <1>     ;
                                <1>         ; INPUT ->
                                <1>     ;       EBX = 0 -> get (all) environment variables
                                <1>     ;             (Required Buffer length = 512 bytes)
                                <1>     ;       EBX > 0 -> set (one) environment variable
                                <1>     ;             (If there is not a '=' after
                                <1>     ;             the environment variable name, it will
                                <1>     ;             accepted as 'get environment variable'.)
                                <1>     ;              EBX = Buffer address
                                <1>     ;       ECX = Buffer address (if EBX = 0)
                                <1>     ;             (ECX will not be used if EBX > 0)
                                <1>     ;             (Note: Buffer size is 512 bytes.)
                                <1>     ;       DL = Buffer size (0 = 256 byte)
                                <1>     ;            (For one envrionment variable)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       (EAX = 0)
                                <1>     ;       CF = 0 -> Successful (EAX = String length)
                                <1>     ;       CF = 1 -> Failed (EAX = 0)
                                <1>     ;
                                <1>     ; Note: Environment variable name, for example,
                                <1>     ;       'PATH=' must be included at the beginning
                                <1>     ;       of the environment string. If the variable
                                <1>     ;       name is as 'PATH' but it is not as 'PATH='
                                <1>     ;       the variable string (row) will be returned.
                                <1>     ;       If variable name is as 'PATH=' but there is
                                <1>     ;       not a following text after the variable name,
                                <1>     ;       the environment variable will be reset/deleted.
                                <1>
0001193A 89E5                   <1>     mov     ebp, esp
0001193C 81EC00020000           <1>     sub     esp, 512
00011942 89E7                   <1>     mov     edi, esp
                                <1>
00011944 31C0                   <1>     xor     eax, eax
00011946 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>
0001194B 21DB                   <1>     and     ebx, ebx
0001194D 751F                   <1>     jnz     short sysenv_0
                                <1>
                                <1>     ; EBX = 0 -> get (all) environment variables
0001194F 89EC                   <1>     mov     esp, ebp
00011951 BE00300900             <1>     mov     esi, Env_Page  ; Environment page
```

```
00011956 89CF                <1>     mov    edi, ecx ; buffer addr (in user's mem space)
                             <1>     ;mov   ecx, 512
                             <1>     ; 30/07/2022
00011958 29C9                <1>     sub    ecx, ecx
0001195A B502                <1>     mov    ch, 2
                             <1>     ; ecx = 512
0001195C E8BAF1FFFF          <1>     call   transfer_to_user_buffer
                             <1>     ;jc    error
                             <1>     ; 23/07/2022
00011961 725B                <1>     jc     short sysenv_error
00011963 890D[6C8E0100]      <1>     mov    [u.r0], ecx
00011969 E920B1FFFF          <1>     jmp    sysret
                             <1>
                             <1> sysenv_0:
0001196E 89DE                <1>     mov    esi, ebx ; * ; user's buffer address
00011970 0FB6CA              <1>     movzx  ecx, dl
                             <1>     ;sub   cl, 1 ; 0 -> 255, 1 -> 0
                             <1>     ;adc   cx, 1 ; 255 -> 256, 0 -> 1
                             <1>     ; 30/07/2022
00011973 FEC9                <1>     dec    cl ; 0 -> 255, 1 -> 0
00011975 41                  <1>     inc    ecx ; 255 -> 256, 0 -> 1
00011976 E8EAF1FFFF          <1>     call   transfer_from_user_buffer
0001197B 723F                <1>     jc     short sysenv_err
0001197D 89FE                <1>     mov    esi, edi
0001197F 8A06                <1>     mov    al, [esi]
00011981 3C20                <1>     cmp    al, 20h
00011983 7637                <1>     jna    short sysenv_err
00011985 3C3D                <1>     cmp    al, '='
00011987 7433                <1>     je     short sysenv_err
00011989 56                  <1>     push   esi
                             <1> sysenv_1:
0001198A 46                  <1>     inc    esi
0001198B 803E3D              <1>     cmp    byte [esi], '='
0001198E 7433                <1>     je     short sysenv_3
00011990 803E20              <1>     cmp    byte [esi], 20h
00011993 73F5                <1>     jnb    short sysenv_1
00011995 C60600              <1>     mov    byte [esi], 0
00011998 5E                  <1>     pop    esi
                             <1>     ; EDI = Output buffer
                             <1>     ; CX = Buffer length
00011999 30C0                <1>     xor    al, al
                             <1>     ; AL = 0 -> use ASCIIZ word in [ESI]
                             <1>     ; ESI = Environment variable name address
0001199B E81781FFFF          <1>     call   get_environment_string
000119A0 721A                <1>     jc     short sysenv_err
000119A2 89DF                <1>     mov    edi, ebx ; * ; user's buffer address
000119A4 89C1                <1>     mov    ecx, eax ; String length
000119A6 89E6                <1>     mov    esi, esp
                             <1>     ; ESI = system buffer address
                             <1>     ; EDI = User's buffer address
                             <1>     ; ECX = transfer (byte) count
000119A8 E86EF1FFFF          <1>     call   transfer_to_user_buffer
000119AD 720D                <1>     jc     short sysenv_err
000119AF 890D[6C8E0100]      <1>     mov    [u.r0], ecx ; transfer (byte) count
                             <1> sysenv_2:
000119B5 89EC                <1>     mov    esp, ebp
000119B7 E9D2B0FFFF          <1>     jmp    sysret
                             <1> sysenv_err:
000119BC 89EC                <1>     mov    esp, ebp
                             <1> sysenv_error:      ; 23/07/2022
000119BE E9ABB0FFFF          <1>     jmp    error
                             <1> sysenv_3:
000119C3 46                  <1>     inc    esi
000119C4 803E20              <1>     cmp    byte [esi], 20h
000119C7 73FA                <1>     jnb    short sysenv_3
000119C9 C60600              <1>     mov    byte [esi], 0
000119CC 5E                  <1>     pop    esi
000119CD E8A881FFFF          <1>     call   set_environment_string
000119D2 72E8                <1>     jc     short sysenv_err
000119D4 8915[6C8E0100]      <1>     mov    [u.r0], edx
000119DA EBD9                <1>     jmp    short sysenv_2
                             <1>
                             <1> isintr:
                             <1>     ; 21/08/2024 - TRDOS 386 v2.0.9
                             <1>     ; check terminate (CTRL+BREAK)
                             <1>     ;      interrupt is enabled or disabled
                             <1>     ;test  word [u.intr], 0FFFFh
000119DC F605[B88E0100]FF    <1>     test   byte [u.intr], 0FFh
                             <1>     ; zf = 1 -> terminate interrupt disabled
                             <1>     ; zf = 0 -> terminate interrupt enabled
                             <1>
                             <1> ; 19/12/2025
                             <1> iget:
                             <1> sndc:
                             <1> sleep:
000119E3 C3                  <1>     retn
                             <1>
                             <1> ; 23/07/2022 - TRDOS 386 v2.0.5
                             <1>
                             <1> ; 22/01/2021
                             <1> ; temporary - 24/01/2016
                             <1>
                             <1> ; 19/12/2025
                             <1> ;iget:
                             <1> ;   ;retn
                             <1> ;;iopen:
                             <1> ;;  ;retn
                             <1> ;;iclose:
                             <1> ;;  ;retn
                             <1> ;sndc:
                             <1> ;   ;retn
                             <1> ;access:
                             <1> ;   ;retn
                             <1> ;sleep:
                             <1> ;   retn
                                 %include 'trdosk7.s' ; 24/01/2016
                             <1> ; ****************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.6) - DISK READ&WRITE : trdosk7.s
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Last Update: 29/08/2023  (Previous: 25/07/2022 - Kernel v2.0.5)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Beginning: 24/01/2016
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.11 (trdos386.s)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                             <1> ; DISK_IO.ASM (20/07/2011)
                             <1> ; ****************************************************************************
                             <1> ; DISK_IO.ASM (c) 2009-2011 Erdogan TAN [ 04/07/2009 ] Last Update: 20/07/2011
                             <1>
                             <1> disk_write:
                             <1>     ; 25/02/2016
                             <1>     ; 24/02/2016
                             <1>     ; 23/02/2016
```

```
000119E4 807E0500            <1>     cmp    byte [esi+LD_LBAYes], 0
000119E8 7774                <1>         ja      short lba_write
                             <1>
                             <1> chs_write:
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 25/02/2016
                             <1>     ; 23/02/2016
000119EA C605[657F0100]03    <1>     mov    byte [disk_rw_op], 3 ; CHS write
000119F1 EB0D                <1>     jmp    short chs_rw
                             <1>
                             <1> disk_read:
                             <1>     ; 25/02/2016
                             <1>     ; 24/02/2016
                             <1>     ; 23/02/2016
                             <1>     ; 17/02/2016
                             <1>     ; 14/02/2016
                             <1>     ; 31/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 17/10/2010
                             <1>     ; 18/04/2010
                             <1>     ;
                             <1>     ; INPUT -> EAX = Logical Block Address
                             <1>     ;         ESI = Logical Dos Disk Table Offset (DRV)
                             <1>     ;         ECX = Sector Count
                             <1>     ;         EBX = Destination Buffer
                             <1>     ; OUTPUT ->
                             <1>     ;         cf = 0 or cf = 1
                             <1>     ; (Modified registers: EAX, EBX, ECX, EDX)
                             <1>
000119F3 807E0500            <1>     cmp    byte [esi+LD_LBAYes], 0
000119F7 776E                <1>         ja      short lba_read
                             <1>
                             <1> chs_read:
                             <1>     ; 29/08/2023 (TRDOS 386 Kernel v2.0.6)
                             <1>     ; 25/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 25/02/2016
                             <1>     ; 24/02/2016
                             <1>     ; 23/02/2016
                             <1>     ; 31/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 20/07/2011
                             <1>     ; 04/07/2009
                             <1>     ;
                             <1>     ; INPUT -> EAX = Logical Block Address
                             <1>     ;         ECX = Number of sectors to read
                             <1>     ;         ESI = Logical Dos Disk Table Offset (DRV)
                             <1>     ;         EBX = Destination Buffer
                             <1>     ; OUTPUT ->
                             <1>     ;         cf = 0 or cf = 1
                             <1>     ; (Modified registers: EAX, EBX, ECX, EDX)
                             <1>
                             <1>     ; 23/02/2016
000119F9 C605[657F0100]02    <1>     mov    byte [disk_rw_op], 2 ; CHS read
                             <1>
                             <1> chs_rw:
                             <1>     ;;movzx edx, word [esi+LD_BPB+SecPerTrack]
                             <1>     ;movzx  edx, byte [esi+LD_BPB+SecPerTrack] ; <= 63
                             <1>     ;mov   [disk_rw_spt], dl
                             <1>
                             <1> chs_read_next_sector:
00011A00 C605[667F0100]04    <1>     mov    byte [retry_count], 4
                             <1>
                             <1> chs_read_retry:
                             <1>     ;mov   [sector_count], ecx ; 23/02/2016
                             <1>
00011A07 50                  <1>     push   eax                    ; Linear sector #
00011A08 51                  <1>     push   ecx                    ; # of FAT/FILE/DIR sectors
                             <1>
00011A09 0FB74E1E            <1>     movzx  ecx, word [esi+LD_BPB+SecPerTrack]
                             <1>     ;movzx  ecx, byte [disk_rw_spt] ; 23/02/2016
00011A0D 29D2                <1>     sub    edx, edx
00011A0F F7F1                <1>     div    ecx
                             <1>     ; eax = track, dx (dl ) = sector (on track)
                             <1>     ;sub   cl, dl ; 24/02/2016 (spt - sec)
                             <1>     ;push  ecx ; *
                             <1>     ; 25/07/2022
                             <1>     ;mov   cx, dx                 ; Sector (zero based)
                             <1>     ;inc   cx                     ; To make it 1 based
                             <1>     ;push  cx
                             <1>     ; 29/08/2023
                             <1>     ;mov   ecx, edx
                             <1>     ;inc   ecx
                             <1>     ;push  ecx
00011A11 42                  <1>     inc    edx
00011A12 52                  <1>     push   edx ; dl = sector number (on track)
                             <1>     ;
00011A13 668B4E20            <1>     mov    cx, [esi+LD_BPB+Heads]
                             <1>     ;sub   dx, dx
                             <1>     ; 25/07/2022
00011A17 29D2                <1>     sub    edx, edx
00011A19 F7F1                <1>     div    ecx                    ; Convert track to head & cyl
                             <1>     ; eax (ax) = cylinder, dx (dl) = head (max. FFh)
00011A1B 88D6                <1>     mov    dh, dl
                             <1>     ;pop   cx                     ; AX=Cyl, DH=Head, CX=Sector
                             <1>     ; 25/07/2022
00011A1D 59                  <1>     pop    ecx ; sector number (on track)
00011A1E 8A5602              <1>     mov    dl, [esi+LD_PhyDrvNo]
                             <1>
00011A21 88C5                <1>     mov    ch, al                 ; NOTE: max. 1023 cylinders !
00011A23 C0CC02              <1>     ror    ah, 2                  ; Rotate 2 bits right
00011A26 08E1                <1>     or     cl, ah
                             <1>
                             <1>     ; 24/02/2016
                             <1>     ;pop   eax ; * (spt - sec) (example: 63 - 0 = 63)
                             <1>     ;cmp   eax, [sector_count]
                             <1>     ;jb    short chs_write_sectors
                             <1>     ;je    short chs_read_sectors
                             <1>     ;; (# of sectors to read is more than remaining sectors on the track)
                             <1>     ;mov   al, [sector_count]
                             <1> ;chs_read_sectors: ; read or write !
00011A28 B001                <1>     mov    al, 1 ; 25/02/2016
00011A2A 8A25[657F0100]      <1>     mov    ah, [disk_rw_op]  ; 02h = chs read, 03h = chs write
                             <1>     ;
00011A30 E8DA35FFFF          <1>     call   int13h                 ; BIOS Service func ( ah ) = 2
                             <1>                                          ; Read disk sectors
                             <1>                                          ; AL-sec num CH-track CL-sec
                             <1>                                          ; DH-head DL-drive ES:BX-buffer
                             <1>                                          ; CF-flag AH-stat AL-sec read
                             <1>                                       ; If CF = 1 then (If AH > 0)
00011A35 8825[677F0100]      <1>     mov    [disk_rw_err], ah
                             <1>
00011A3B 59                  <1>     pop    ecx
00011A3C 58                  <1>     pop    eax
00011A3D 7314                <1>     jnc    short chs_read_ok
                             <1>
00011A3F 803D[677F0100]09    <1>     cmp    byte [disk_rw_err], 09h ; DMA crossed 64K segment boundary
```

```
00011A46 7408                <1>     je      short chs_read_error_retn
                             <1>
00011A48 FE0D[667F0100]      <1>     dec     byte [retry_count]
00011A4E 75B7                <1>     jnz     short chs_read_retry
                             <1>
                             <1> chs_read_error_retn:
00011A50 F9                  <1>     stc
                             <1>     ;retn
00011A51 EB67                <1>     jmp     short update_drv_error_byte
                             <1>
                             <1> ;chs_write_sectors: ; read or write
                             <1>     ;; (# of sectors to read is less than remaining sectors on the track)
                             <1>     ;mov    [sector_count], al
                             <1>     ;jmp    short chs_read_sectors
                             <1>
                             <1> chs_read_ok:
                             <1>     ;; 23/02/2016
                             <1>     ;movzx  edx, byte [sector_count] ; sector count (<= spt)
                             <1>         ;sub    ecx, edx  ; remaining sector count
                             <1>     ;jna    short update_drv_error_byte
                             <1>     ;add    eax, edx ; next disk sector
                             <1>     ;shl    edx, 9 ; 512 * sector count
                             <1>     ;add    ebx, edx ; next buffer byte address
                             <1>         ;jmp       chs_read_next_sector
                             <1>     ; 25/02/2016
00011A53 40                  <1>     inc     eax ; next sector
00011A54 81C300020000        <1>     add     ebx, 512
00011A5A E2A4                <1>     loop    chs_read_next_sector
00011A5C EB5C                <1>     jmp     short update_drv_error_byte
                             <1>
                             <1> lba_write:
                             <1>     ; 23/02/2016
00011A5E C605[657F0100]1C    <1>     mov     byte [disk_rw_op], 1Ch ; LBA write
00011A65 EB07                <1>     jmp     short lba_rw
                             <1>
                             <1> lba_read:
                             <1>     ; 14/07/2022
                             <1>     ; 23/02/2016
                             <1>     ; 17/02/2016
                             <1>     ; 14/02/2016
                             <1>     ; 13/02/2016
                             <1>     ; 31/01/2016 (TRDOS 386 = TRDOS v2.0)
                             <1>     ; 10/07/2015 (Retro UNIX 386 v1)
                             <1>     ;
                             <1>     ; INPUT -> EAX = Logical Block Address
                             <1>     ;         ESI = Logical Dos Disk Table Offset (DRV)
                             <1>     ;         ECX = Sector Count
                             <1>     ;         EBX = Destination Buffer
                             <1>     ; OUTPUT ->
                             <1>     ;         cf = 0 or cf = 1
                             <1>     ; (Modified registers: EAX, EBX, ECX, EDX)
                             <1>
                             <1>     ; LBA read/write (with private LBA function)
                             <1>     ;((Retro UNIX 386 v1 - DISK I/O code by Erdogan Tan))
                             <1>
                             <1>     ; 23/02/2016
00011A67 C605[657F0100]1B    <1>     mov     byte [disk_rw_op], 1Bh ; LBA read
                             <1>
                             <1> lba_rw:
                             <1>     ; 17/02/2016
00011A6E 57                  <1>     push    edi
                             <1>
00011A6F 890D[687F0100]      <1>     mov     [sector_count], ecx ; total sector (read) count
                             <1>
00011A75 8A5602              <1>     mov     dl, [esi+LD_PhyDrvNo]
                             <1>     ; dl = physical drive number (0, 1, 80h, 81h, 82h, 83h)
                             <1>
                             <1> lba_read_next:
                             <1>     ; 14/07/2022
00011A78 BF00010000          <1>     mov     edi, 256
00011A7D 39F9                <1>     cmp     ecx, edi
                             <1>     ;cmp    ecx, 256
00011A7F 7602                <1>     jna     short lba_read_rsc
                             <1>     ;mov    ecx, 256 ; 17/02/2016
00011A81 89F9                <1>     mov     ecx, edi
                             <1> lba_read_rsc:
00011A83 290D[687F0100]      <1>     sub     [sector_count], ecx ; remain sectors
                             <1>
00011A89 89CF                <1>     mov     edi, ecx
00011A8B 89C1                <1>     mov     ecx, eax ; sector number/address
                             <1>
00011A8D C605[667F0100]04    <1>     mov     byte [retry_count], 4
                             <1> lba_read_retry:
00011A94 89F8                <1>     mov     eax, edi
                             <1>     ;
                             <1>     ; ecx = sector number
                             <1>     ; al = sector count (0 - 255) /// (0 = 256)
                             <1>     ; dl = drive number
                             <1>     ; ebx = buffer offset
                             <1>     ;
                             <1>     ; Function 1Bh = LBA read, 1Ch = LBA write
                             <1>     ; 23/02/2016
00011A96 8A25[657F0100]      <1>     mov     ah, [disk_rw_op] ; 1Bh = LBA read, 1Ch = LBA write
00011A9C E86E35FFFF          <1>     call    int13h
                             <1>     ; al = ? (changed)
                             <1>     ; ah = error code
00011AA1 8825[677F0100]      <1>     mov     [disk_rw_err], ah
00011AA7 7332                <1>     jnc     short lba_read_ok
00011AA9 80FC80              <1>     cmp     ah, 80h ; time out?
00011AAC 740A                <1>         je      short lba_read_stc_retn
00011AAE FE0D[667F0100]      <1>     dec     byte [retry_count]
00011AB4 7FDE                <1>     jg      short lba_read_retry
00011AB6 7438                <1>     jz      short lba_read_reset
                             <1>     ; sf = 1
                             <1>
                             <1> lba_read_stc_retn:
00011AB8 F9                  <1>     stc
                             <1> lba_read_retn:
00011AB9 5F                  <1>     pop     edi
                             <1>
                             <1> update_drv_error_byte:
00011ABA 9C                  <1>     pushf
00011ABB 53                  <1>     push    ebx
                             <1>     ;push   cx
00011ABC 51                  <1>     push    ecx ; 14/07/2022
                             <1>     ;or     ecx, ecx
                             <1>     ;jz     short udrv_errb0
00011ABD 8A0D[677F0100]      <1>     mov     cl, [disk_rw_err]
                             <1> udrv_errb0:
00011AC3 0FB65E02            <1>     movzx   ebx, byte [esi+LD_PhyDrvNo]
00011AC7 80FB02              <1>     cmp     bl, 2
00011ACA 7203                <1>         jb      short udrv_errb1
00011ACC 80EB7E              <1>     sub     bl, 7Eh
                             <1>     ;cmp    bl, 5
```

```
                                <1>     ;ja     short udrv_errb2
                                <1> udrv_errb1:
00011ACF 81C3[B5660000]         <1>         add     ebx, drv.error ; 13/02/2016
00011AD5 880B                   <1>     mov     [ebx], cl ; error code
                                <1> udrv_errb2:
                                <1>     ;pop    cx
00011AD7 59                     <1>     pop     ecx  ; 14/07/2022
00011AD8 5B                     <1>     pop     ebx
00011AD9 9D                     <1>     popf
00011ADA C3                     <1>     retn
                                <1>
                                <1> lba_read_ok:
00011ADB 89C8                   <1>     mov     eax, ecx ; sector number
00011ADD 01F8                   <1>     add     eax, edi ; sector number (next)
00011ADF C1E709                 <1>     shl     edi, 9 ; sector count * 512
00011AE2 01FB                   <1>     add     ebx, edi ; next buffer offset
                                <1>
00011AE4 8B0D[687F0100]         <1>     mov     ecx, [sector_count] ; remaining sectors
00011AEA 09C9                   <1>     or      ecx, ecx
00011AEC 758A                   <1>     jnz     short lba_read_next
00011AEE EBC9                   <1>     jmp     short lba_read_retn
                                <1>
                                <1> lba_read_reset:
00011AF0 B40D                   <1>     mov     ah, 0Dh ; Alternate reset
00011AF2 E81835FFFF             <1>         call       int13h
                                <1>     ; al = ? (changed)
                                <1>     ; ah = error code
00011AF7 739B                   <1>     jnc     short lba_read_retry
00011AF9 EBBE                   <1>     jmp     short lba_read_retn
                                    %include 'trdosk8.s' ; 24/01/2016
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - MAIN PROGRAM : trdosk8.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 28/01/2025  (Previous: 29/12/2024)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; u0.s (20/11/2015), u4.s (14/10/2015)
                                <1> ; ****************************************************************************
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; DIR.ASM (c) 2004-2011 Erdogan TAN  [07/01/2004] Last Update: 09/10/2011
                                <1>
                                <1> set_run_sequence:
                                <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 23/12/2016
                                <1>     ; 10/06/2016
                                <1>     ; 22/05/2016
                                <1>     ; 20/05/2016
                                <1>     ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; TRDOS 386 feature only !
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      AL = process number (next process)
                                <1>     ;
                                <1>     ;      this process must be added to run sequence
                                <1>     ;
                                <1>     ;      [u.pri] = priority of present process
                                <1>     ;
                                <1>     ;      DL = priority (queue)
                                <1>     ;           0 = background (low) ; run on background
                                <1>     ;           1 = regular (normal) ; run as regular
                                <1>     ;           2 = event (high)     ; run for event
                                <1>     ;
                                <1>     ;      1) If the requested process is already running:
                                <1>     ;         a) If present priority is high ([u.pri]=2)
                                <1>     ;            and requested priority is also high,
                                <1>     ;            there is nothing to do! Because it has been
                                <1>     ;            done already (before this attempt).
                                <1>     ;         b) If present priority is high ([u.pri]=2)
                                <1>     ;            and requested priority is not high, there is
                                <1>     ;            nothing to do! Because, it's current
                                <1>     ;            run queue is unspecified, here. (It may be in
                                <1>     ;            a waiting list or in a run queue; if the new
                                <1>     ;            priority would be used to add it to relavant
                                <1>         ;            run queue, this would be wrong, unnecessary
                                <1>     ;            and destabilizing duplication!)
                                <1>     ;         c) If present priority is not high ([u.pri]<2)
                                <1>         ;            and requested priority is high (event),
                                <1>     ;            process will be added to present priority's
                                <1>     ;            run queue and then, priority will be changed
                                <1>     ;            to high ([u.pri]=2).
                                <1>     ;         d) If present priority is not high ([u.pri]<2)
                                <1>     ;            and requested priority is not high, [u.pri]
                                <1>     ;            value will be changed. There is nothing to do
                                <1>     ;            in addition. (The new priority value will be
                                <1>     ;            used by 'tswap/tswitch' procedure at 'sysret'
                                <1>     ;            or 'sysrele' stage.)
                                <1>     ;
                                <1>     ;      2) If the requested process is not running:
                                <1>     ;         a) If requested priority of the requested
                                <1>     ;            (next) process is high (event) and priority
                                <1>     ;            of present process is not high, the requested
                                <1>     ;            process will be added to ('runq_event') high
                                <1>     ;            priority run queue and then present (running)
                                <1>     ;            process will be stopped (swapped/switched out)
                                <1>     ;            immediately if it is in user mode, or it's
                                <1>     ;            [u.quant] value will be reset to 0 and (then)
                                <1>     ;            it will be stopped at 'sysret' stage.
                                <1>     ;         b) If requested priority of the requested
                                <1>     ;            (next) process is high (event) and priority
                                <1>     ;            of present process is also high, the requested
                                <1>     ;            process will be added to ('runq_event') high
                                <1>     ;            priority run queue and present (running)
                                <1>     ;            process will be allowed to run until it's
                                <1>     ;            time quantum will be elapsed ([u.quant]=0).
                                <1>     ;         c) If requested priority of the requested
                                <1>     ;            (next) process is not high ('run for event'),
                                <1>     ;            there is nothing to do. Because, it's current
                                <1>     ;            run queue is unspecified, here. (It may be in
                                <1>     ;            a waiting list or in a run queue; if the new
                                <1>     ;            priority would be used to add it to relavant
                                <1>         ;            run queue, this would be wrong, unnecessary
                                <1>     ;            and destabilizing duplication!)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;      none
                                <1>     ;
                                <1>     ;      [u.pri] = priority of present process
                                <1>     ;
```

```
                            <1>     ;       cf = 1, if the request could not be fulfilled.
                            <1>     ;
                            <1>     ;       NOTE:
                            <1>         ;           * Processes in 'run as regular' queue can run
                            <1>     ;             if there is no process in 'run for event' queue
                            <1>     ;             ('run for event' processes have higher priority)
                            <1>     ;           * When [u.quant] time quantum of a process is
                            <1>     ;             elapsed, it's high priority ('run for event')
                            <1>     ;             status will be disabled, it can be run in sequence
                            <1>     ;             of it's actual run queue.
                            <1>     ;           * A 'run on background' process will always be
                            <1>     ;             sequenced in 'run on background' (low priority)
                            <1>     ;             queue, it can run only when other priority queues
                            <1>     ;             are empty. (idle time processes, e.g. printing)
                            <1>     ;
                            <1>     ; Modified registers: eax, ebx, edx
                            <1>     ;
                            <1> srunseq_0:
00011AFB 3A05[BD8E0100]     <1>         cmp     al, [u.uno]     ; same process ?
00011B01 750C               <1>     jne     short srunseq_2 ; no
                            <1>
00011B03 8A25[B68E0100]     <1>     mov     ah, [u.pri]     ; present/current priority
00011B09 80FC02             <1>     cmp     ah, 2           ; 'run for event' priority level
00011B0C 7220               <1>     jb      short srunseq_6 ; no
                            <1>
                            <1> srunseq_1:
                            <1>     ; there is nothing to do!
00011B0E C3                 <1>     retn
                            <1>
                            <1> srunseq_2:
                            <1>     ;;this not necessary ! 23/12/2016
                            <1>         ;;cmp      al, nproc      ; number of processes = 16
                            <1>     ;;jnb  short srunseq_5       ; error ! invalid process number
                            <1>
                            <1>     ; dl = priority
00011B0F 80FA02             <1>     cmp     dl, 2           ; event queue
00011B12 72FA               <1>     jb      short srunseq_1 ; requested process is not present
                            <1>                             ; process and priority of requested
                            <1>                             ; process is not high (event),
                            <1>                             ; there is nothing to do!
                            <1>
                            <1>     ; requested process is not present process
                            <1>     ; & priority of requested process is high
00011B14 3A15[B68E0100]     <1>     cmp     dl, [u.pri]     ; priority of present process
00011B1A 7606               <1>     jna     short srunseq_3 ; is high, also
                            <1>     ;
                            <1>     ; present process will be swapped/switched out
00011B1C FE05[45830100]     <1>     inc     byte [p_change] ; 1
                            <1>
                            <1> srunseq_3:
                            <1>     ; add process to 'runq_event' queue for new event
00011B22 BB[5A8E0100]       <1>     mov     ebx, runq_event ; high priority run queue
                            <1>
                            <1> srunseq_4:
                            <1>     ; al = process number
                            <1>     ; ebx = run queue
                            <1>     ;call  putlu
                            <1>     ;retn
                            <1>     ; 29/07/2022
00011B27 E91BEFFFFF         <1>     jmp     putlu
                            <1>
                            <1> srunseq_5:
00011B2C F5                 <1>     cmc
00011B2D C3                 <1>     retn
                            <1>
                            <1> srunseq_6:
                            <1>     ; present priority of the process is not high
                            <1>
00011B2E 8815[B68E0100]     <1>     mov     [u.pri], dl ; new priority
                            <1>                         ; (will be used by 'tswap')
                            <1>
00011B34 80FA02             <1>     cmp     dl, 2           ; high priority ?
00011B37 72F3               <1>     jb      short srunseq_5 ; no, there is nothing to do
                            <1>                             ; in addition
                            <1>
                            <1>     ; process must be added to relevant run queue, here!
                            <1>     ; (new priority is high/event priority and process
                            <1>     ; will not be added to a run queue by 'tswap')
                            <1>
00011B39 BB[5C8E0100]       <1>     mov     ebx, runq_normal ; 'run as regular' queue
                            <1>
00011B3E 20E4               <1>     and     ah, ah  ;  previous value of [u.pri]
00011B40 75E5               <1>     jnz     short srunseq_4
                            <1>
00011B42 43                 <1>     inc     ebx
00011B43 43                 <1>     inc     ebx
                            <1>     ; ebx = runq_background ; 'run on backgroud' queue
                            <1>
00011B44 EBE1               <1>     jmp     short srunseq_4
                            <1> clock:
                            <1>     ; 21/08/2024 - TRDOS 386 v2.0.9
                            <1>     ; 23/05/2016
                            <1>     ; 22/05/2016
                            <1>     ; 20/05/2016
                            <1>     ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                            <1>     ; 14/05/2015 - 14/10/2015 (Retro UNIX 386 v1)
                            <1>     ; 07/12/2013 - 10/04/2014 (Retro UNIX 8086 v1)
                            <1>
                            <1>     ;;;
                            <1>     ; 21/08/2024
                            <1>     ;cmp    word [u.quit], 0FFFFh
00011B46 803D[BA8E0100]FF   <1>     cmp     byte [u.quit], 0FFh
00011B4D 721A               <1>     jb      short clk_0
                            <1>     ; CTRL+BRK keys pressed
                            <1>     ;cmp    word [u.intr], 0
00011B4F 803D[B88E0100]00   <1>     cmp     byte [u.intr], 0
00011B56 7611               <1>     jna     short clk_0
00011B58 C605[45830100]FF   <1>     mov     byte [p_change], 0FFh ; -1 ; CTRL+BREAK sign
00011B5F F605[B48E0100]FF   <1>     test    byte [u.quant], 0FFh  ; if [u.quant] number > 0
00011B66 752D               <1>     jnz     short clk_1 ; decrease [u.quant] number
                            <1>     ; [u.quant] = 0
00011B68 C3                 <1>     retn
                            <1>     ;;;
                            <1> clk_0:
00011B69 803D[B48E0100]00   <1>     cmp     byte [u.quant], 0
00011B70 7723               <1>     ja      short clk_1
                            <1>     ;
00011B72 803D[BD8E0100]01   <1>     cmp      byte [u.uno], 1 ; /etc/init ? (for Retro UNIX 8086 & 386 v1)
                            <1>                              ; MainProg (Kernel's Command Interpreter)
                            <1>                              ; for TRDOS 386.
00011B79 761A               <1>     jna     short clk_1 ; yes, do not swap out
                            <1>     ;
00011B7B 803D[608E0100]FF   <1>     cmp      byte [sysflg], 0FFh ; user or system space ?
```

```
00011B82 7517                <1>     jne    short clk_2        ; system space (sysflg <> 0FFh)
                             <1>     ;
                             <1>     ; 21/08/2024
                             <1>     ;cmp   word [u.intr], 0
                             <1>     ;jna   short clk_2
                             <1>     ;
                             <1>     ; 23/05/2016
00011B84 803D[46830100]00    <1>     cmp    byte [multi_tasking], 0
00011B8B 760E                <1>     jna    short clk_2
                             <1>
                             <1>     ; 21/08/2024
                             <1>     ;inc   byte [p_change] ; it is time to change running process
00011B8D C605[45830100]01    <1>     mov    byte [p_change], 1
00011B94 C3                  <1>     retn
                             <1> clk_1:
00011B95 FE0D[B48E0100]      <1>     dec    byte [u.quant]
                             <1> clk_2:
00011B9B C3                  <1>     retn   ; return to (hardware) timer interrupt routine
                             <1>
                             <1> ; 12/10/2017
                             <1> ; 15/01/2017
                             <1> ; 14/01/2017
                             <1> ; 07/01/2017
                             <1> ; 02/01/2017
                             <1> ; 17/08/2016
                             <1> ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1> int34h: ; #IOCTL# (I/O port access support for ring 3)
                             <1>     ;
                             <1>     ; 23/05/2016
                             <1>     ; 20/06/2016
                             <1>     ; 29/04/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>     ;
                             <1>     ; INPUT ->
                             <1>     ;      AH = 0 -> read port  (physical IO port) -byte-
                             <1>     ;      AH = 1 -> write port (physical IO port) -byte-
                             <1>     ;              AL = data byte
                             <1>     ;      AH = 2 -> read port  (physical IO port) -word-
                             <1>     ;      AH = 3 -> write port (physical IO port) -word-
                             <1>     ;              BX = data word
                             <1>     ;      AH = 4 -> read port  (physical IO port) -dword-
                             <1>     ;      AH = 5 -> write port (physical IO port) -dword-
                             <1>     ;              EBX = data dword
                             <1>     ;      ; 12/10/2017
                             <1>     ;      AH = 6 -> read port (physical IO port) twice -byte-
                             <1>     ;      AH = 7 -> write port (physical IO port) twice -byte-
                             <1>     ;              BX = data word
                             <1>     ;
                             <1>     ;      DX = Port number (<= 0FFFFh)
                             <1>     ;
                             <1>     ; OUTPUT ->
                             <1>     ;      AL = data byte (in al, dx)
                             <1>     ;      AX = data word (in ax, dx)
                             <1>     ;      EAX = data dword (in eax, dx)
                             <1>     ;
                             <1>     ;      (ECX = actual TRANSFER COUNT for string functions)
                             <1>     ;
                             <1>     ;
                             <1>     ; Modified registers: EAX
                             <1>     ;
                             <1>
                             <1>     ;cmp   ah, 5
                             <1>     ;ja    short int34h_5 ; invalid function !
                             <1>
                             <1>     ; 12/10/2017
00011B9C 80FC07              <1>     cmp    ah, 7
00011B9F 7743                <1>     ja     short int34h_5 ; invalid function !
                             <1>
                             <1>     ;; 15/01/2017
                             <1>     ; 14/01/2017
                             <1>     ; 02/01/2017
                             <1>     ;;mov  byte [ss:intflg], 34h ; IOCTL interrupt
00011BA1 FB                  <1>     sti
                             <1>
                             <1>     ;sti   ; enable interrupts
00011BA2 80642408FE          <1>     and    byte [esp+8], 11111110b      ; clear carry bit of eflags register
                             <1>
00011BA7 80FC01              <1>     cmp    ah, 1
00011BAA 7205                <1>     jb     short int34h_0
00011BAC 7705                <1>     ja     short int34h_1
                             <1>
00011BAE EE                  <1>     out    dx, al
                             <1>     ;iretd
00011BAF EB01                <1>     jmp    short int34h_iret
                             <1>
                             <1> int34h_0:
00011BB1 EC                  <1>     in     al, dx
                             <1>     ;iretd
                             <1> int34h_iret:
                             <1>     ;cli   ; 07/01/2017
                             <1>     ;; 15/01/2017
                             <1>     ;;mov  byte [ss:intflg], 0 ; reset
00011BB2 CF                  <1>     iretd
                             <1>
                             <1> int34h_1:
00011BB3 F6C401              <1>     test   ah, 1
00011BB6 7516                <1>     jnz    short int34h_3 ; out
                             <1>
                             <1>     ; in
00011BB8 80FC02              <1>     cmp    ah, 2
00011BBB 7707                <1>     ja     short int34h_2
                             <1>
00011BBD 6689D8              <1>     mov    ax, bx
00011BC0 66ED                <1>     in     ax, dx
                             <1>     ;iretd
00011BC2 EBEE                <1>     jmp    short int34h_iret
                             <1>
                             <1> int34h_2:
00011BC4 80FC04              <1>     cmp    ah, 4
00011BC7 772C                <1>     ja     short int34h_7 ; 12/10/2017
                             <1>     ; ah = 4
00011BC9 89D8                <1>     mov    eax, ebx
00011BCB ED                  <1>     in     eax, dx
                             <1>     ;iretd
00011BCC EBE4                <1>     jmp    short int34h_iret
                             <1>
                             <1> int34h_3:
00011BCE 80FC03              <1>     cmp    ah, 3
00011BD1 7707                <1>     ja     short int34h_4
                             <1>
00011BD3 6689D8              <1>     mov    ax, bx
00011BD6 66EF                <1>     out    dx, ax
                             <1>     ;iretd
00011BD8 EBD8                <1>     jmp    short int34h_iret
                             <1>
```

```
                                <1> int34h_4:
00011BDA 80FC05                 <1>     cmp     ah, 5
00011BDD 770B                   <1>     ja      short int34h_6 ; 12/10/2017
                                <1>     ; ah = 5
00011BDF 89D8                   <1>     mov     eax, ebx
00011BE1 EF                     <1>     out     dx, eax
                                <1>     ;iretd
00011BE2 EBCE                   <1>     jmp     short int34h_iret
                                <1>
                                <1> int34h_5:
00011BE4 804C240801             <1>     or      byte [esp+8], 1       ; set carry bit of eflags register
00011BE9 CF                     <1>     iretd
                                <1>
                                <1>     ; 12/10/2017
                                <1> int34h_6:
00011BEA 6689D8                 <1>     mov     ax, bx
00011BED EE                     <1>     out     dx, al
00011BEE EB00                   <1>     jmp     short $+2
00011BF0 86E0                   <1>     xchg    ah, al
00011BF2 EE                     <1>     out     dx, al
                                <1>     ;xchg   al, ah
                                <1>     ;iretd
00011BF3 EB06                   <1>     jmp     short int34h_8
                                <1> int34h_7:
00011BF5 EC                     <1>     in      al, dx
00011BF6 EB00                   <1>     jmp     short $+2
00011BF8 88C4                   <1>     mov     ah, al
00011BFA EC                     <1>     in      al, dx
                                <1> int34h_8:
00011BFB 86C4                   <1>     xchg    al, ah
00011BFD CF                     <1>     iretd
                                <1>
                                <1> INT4Ah:
                                <1>     ; 24/01/2016
                                <1>     ; this procedure will be called by 'RTC_INT' (in 'timer.s')
00011BFE C3                     <1>     retn
                                <1>
                                <1> ; u0.s
                                <1> ; Retro UNIX 386 v1 Kernel (v0.2) - SYS0.INC
                                <1> ; Last Modification: 20/11/2015
                                <1>
                                <1> com2_int:
                                <1>     ; 07/11/2015
                                <1>     ; 24/10/2015
                                <1>     ; 23/10/2015
                                <1>     ; 14/03/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 28/07/2014 (Retro UNIX 8086 v1)
                                <1>     ; < serial port 2 interrupt handler >
                                <1>     ;
00011BFF 890424                 <1>     mov     [esp], eax ; overwrite call return address
                                <1>     ;push   eax
00011C02 66B80900               <1>     mov     ax, 9
00011C06 EB07                   <1>     jmp     short comm_int
                                <1> com1_int:
                                <1>     ; 07/11/2015
                                <1>     ; 24/10/2015
00011C08 890424                 <1>     mov     [esp], eax ; overwrite call return address
                                <1>     ; 23/10/2015
                                <1>     ;push   eax
00011C0B 66B80800               <1>     mov     ax, 8
                                <1> comm_int:
                                <1>     ; 20/11/2015
                                <1>     ; 18/11/2015
                                <1>     ; 17/11/2015
                                <1>     ; 16/11/2015
                                <1>     ; 09/11/2015
                                <1>     ; 08/11/2015
                                <1>     ; 07/11/2015
                                <1>     ; 06/11/2015 (serial4.asm, 'serial')
                                <1>     ; 01/11/2015
                                <1>     ; 26/10/2015
                                <1>     ; 23/10/2015
00011C0F 53                     <1>     push    ebx
00011C10 56                     <1>     push    esi
00011C11 57                     <1>     push    edi
00011C12 1E                     <1>     push    ds
00011C13 06                     <1>     push    es
                                <1>     ; 18/11/2015
00011C14 0F20DB                 <1>     mov     ebx, cr3
00011C17 53                     <1>     push    ebx ; ****
                                <1>     ;
00011C18 51                     <1>     push    ecx ; ***
00011C19 52                     <1>     push    edx ; **
                                <1>     ;
00011C1A BB10000000             <1>     mov     ebx, KDATA
00011C1F 8EDB                   <1>     mov     ds, bx
00011C21 8EC3                   <1>     mov     es, bx
                                <1>     ;
00011C23 8B0D[B0760100]         <1>     mov     ecx, [k_page_dir]
00011C29 0F22D9                 <1>     mov     cr3, ecx
                                <1>     ; 20/11/2015
                                <1>     ; Interrupt identification register
00011C2C 66BAFA02               <1>     mov     dx, 2FAh ; COM2
                                <1>     ;
00011C30 3C08                   <1>     cmp     al, 8
00011C32 7702                   <1>     ja      short com_i0
                                <1>     ;
                                <1>     ; 20/11/2015
                                <1>     ; 17/11/2015
                                <1>     ; 16/11/2015
                                <1>     ; 15/11/2015
                                <1>     ; 24/10/2015
                                <1>     ; 14/03/2015 (Retro UNIX 386 v1 - Beginning)
                                <1>     ; 28/07/2014 (Retro UNIX 8086 v1)
                                <1>     ; < serial port 1 interrupt handler >
                                <1>     ;
00011C34 FEC6                   <1>     inc     dh ; 3FAh ; COM1 Interrupt id. register
                                <1> com_i0:
                                <1>     ;push   eax ; *
                                <1>     ; 07/11/2015
00011C36 A2[1A770100]           <1>     mov     byte [ccomport], al
                                <1>     ; 09/11/2015
00011C3B 0FB7D8                 <1>     movzx   ebx, ax ; 8 or 9
                                <1>     ; 17/11/2015
                                <1>     ; reset request for response status
00011C3E 88A3[10770100]         <1>     mov     [ebx+req_resp-8], ah ; 0
                                <1>     ;
                                <1>     ; 20/11/2015
00011C44 EC                     <1>     in      al, dx          ; read interrupt id. register
00011C45 EB00                   <1>     JMP     $+2             ; I/O DELAY
00011C47 2404                   <1>     and     al, 4           ; received data available?
00011C49 7470                   <1>     jz      short com_eoi   ; (transmit. holding reg. empty)
                                <1>     ;
                                <1>     ; 20/11/2015
```

```
00011C4B 80EA02              <1>     sub    dl, 3FAh-3F8h  ; data register (3F8h, 2F8h)
00011C4E EC                  <1>     in     al, dx         ; read character
                             <1>     ;JMP   $+2            ; I/O DELAY
                             <1>     ; 08/11/2015
                             <1>     ; 07/11/2015
00011C4F 89DE                <1>     mov    esi, ebx
00011C51 89DF                <1>     mov    edi, ebx
00011C53 81C6[14770100]      <1>     add    esi, rchar - 8 ; points to last received char
00011C59 81C7[16770100]      <1>     add    edi, schar - 8 ; points to last sent char
00011C5F 8806                <1>     mov    [esi], al ; received char (current char)
                             <1>     ; query
00011C61 20C0                <1>     and    al, al
00011C63 7527                <1>     jnz    short com_i2
                             <1>     ; response
                             <1>     ; 17/11/2015
                             <1>     ; set request for response status
00011C65 FE83[10770100]      <1>         inc     byte [ebx+req_resp-8] ; 1
                             <1>     ;
00011C6B 6683C205            <1>     add    dx, 3FDh-3F8h  ; (3FDh, 2FDh)
00011C6F EC                  <1>     in     al, dx         ; read line status register
00011C70 EB00                <1>     JMP    $+2            ; I/O DELAY
00011C72 2420                <1>     and    al, 20h        ; transmitter holding reg. empty?
00011C74 7445                <1>     jz     short com_eoi  ; no
00011C76 B0FF                <1>     mov    al, 0FFh       ; response
00011C78 6683EA05            <1>     sub    dx, 3FDh-3F8h  ; data port (3F8h, 2F8h)
00011C7C EE                  <1>     out    dx, al         ; send on serial port
                             <1>     ; 17/11/2015
00011C7D 803F00              <1>     cmp    byte [edi], 0   ; query ? (schar)
00011C80 7502                <1>     jne    short com_i1    ; no
00011C82 8807                <1>     mov    [edi], al      ; 0FFh (responded)
                             <1> com_i1:
                             <1>     ; 17/11/2015
                             <1>     ; reset request for response status (again)
00011C84 FE8B[10770100]      <1>         dec     byte [ebx+req_resp-8] ; 0
00011C8A EB2F                <1>     jmp    short com_eoi
                             <1> com_i2:
                             <1>     ; 08/11/2015
00011C8C 3CFF                <1>     cmp    al, 0FFh       ; (response ?)
00011C8E 7417                <1>     je     short com_i3   ; (check for response signal)
                             <1>     ; 07/11/2015
00011C90 3C04                <1>     cmp    al, 04h ; EOT
00011C92 751C                <1>     jne    short com_i4
                             <1>     ; EOT = 04h (End of Transmit) - 'CTRL + D'
                             <1>     ;(an EOT char is supposed as a ctrl+brk from the terminal)
                             <1>     ; 08/11/2015
                             <1>            ; ptty -> tty 0 to 7 (pseudo screens)
00011C94 861D[DE760100]      <1>     xchg   bl, [ptty]  ; tty number (8 or 9)
00011C9A E89F51FFFF          <1>     call   ctrlbrk
00011C9F 861D[DE760100]      <1>     xchg   [ptty], bl ; (restore ptty value and BL value)
                             <1>     ;mov   al, 04h ; EOT
                             <1>     ; 08/11/2015
00011CA5 EB09                <1>     jmp    short com_i4
                             <1> com_i3:
                             <1>     ; 08/11/2015
                             <1>     ; If 0FFh has been received just after a query
                             <1>     ; (schar, ZERO), it is a response signal.
                             <1>     ; 17/11/2015
00011CA7 803F00              <1>         cmp     byte [edi], 0 ; query ? (schar)
00011CAA 7704                <1>     ja     short com_i4 ; no
                             <1>     ; reset query status (schar)
00011CAC 8807                <1>     mov    [edi], al ; 0FFh
00011CAE FEC0                <1>     inc    al ; 0
                             <1> com_i4:
                             <1>     ; 27/07/2014
                             <1>     ; 09/07/2014
00011CB0 D0E3                <1>     shl    bl, 1
00011CB2 81C3[E0760100]      <1>     add    ebx, ttychr
                             <1>     ; 23/07/2014 (always overwrite)
                             <1>     ;;cmp  word [ebx], 0
                             <1>     ;;ja   short com_eoi
                             <1>     ;
00011CB8 668903              <1>     mov    [ebx], ax   ; Save ascii code
                             <1>                        ; scan code = 0
                             <1> com_eoi:
                             <1>     ;mov   al, 20h
                             <1>     ;out   20h, al    ; end of interrupt
                             <1>     ;
                             <1>     ; 07/11/2015
                             <1>            ;pop   eax ; *
00011CBB A0[1A770100]        <1>     mov    al, byte [ccomport] ; current COM port
                             <1>      ; al = tty number (8 or 9)
00011CC0 E80F000000          <1>         call       wakeup
                             <1> com_iret:
                             <1>     ; 23/10/2015
00011CC5 5A                  <1>     pop    edx ; **
00011CC6 59                  <1>     pop    ecx ; ***
                             <1>     ; 18/11/2015
                             <1>     ;pop   eax ; ****
                             <1>     ;mov   cr3, eax
                             <1>     ;jmp   iiret
00011CC7 E90EF1FEFF          <1>     jmp    iiretp
                             <1>
                             <1> ;iiretp: ; 01/09/2015
                             <1> ;   ; 28/08/2015
                             <1> ;   pop    eax ; (*) page directory
                             <1> ;   mov    cr3, eax
                             <1> ;iiret:
                             <1> ;   ; 22/08/2014
                             <1> ;   mov    al, 20h ; END OF INTERRUPT COMMAND TO 8259
                             <1> ;   out    20h, al ; 8259 PORT
                             <1> ;   ;
                             <1> ;   pop    es
                             <1> ;   pop    ds
                             <1> ;   pop    edi
                             <1> ;   pop    esi
                             <1> ;   pop    ebx ; 29/08/2014
                             <1> ;   pop    eax
                             <1> ;   iretd
                             <1>
                             <1> ; 21/11/2023
                             <1> %if 0
                             <1>
                             <1> sp_init:
                             <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                             <1>     ; 07/11/2015
                             <1>     ; 29/10/2015
                             <1>     ; 26/10/2015
                             <1>     ; 23/10/2015
                             <1>     ; 29/06/2015
                             <1>     ; 14/03/2015 (Retro UNIX 386 v1 - 115200 baud)
                             <1>     ; 28/07/2014 (Retro UNIX 8086 v1 - 9600 baud)
                             <1>     ; Initialization of Serial Port Communication Parameters
                             <1>     ; (COM1 base port address = 3F8h, COM1 Interrupt = IRQ 4)
                             <1>     ; (COM2 base port address = 2F8h, COM1 Interrupt = IRQ 3)
```

```
<1>     ;
<1>     ; ((Modified registers: EAX, ECX, EDX, EBX))
<1>     ;
<1>     ; INPUT:  (29/06/2015)
<1>     ;      AL = 0 for COM1
<1>     ;           1 for COM2
<1>     ;      AH = Communication parameters
<1>     ;
<1>     ;  (*) Communication parameters (except BAUD RATE):
<1>     ;      Bit    4      3      2      1      0
<1>     ;             -PARITY--   STOP BIT  -WORD LENGTH-
<1>     ;  this one -->       00 = none    0 = 1 bit  11 = 8 bits
<1>     ;             01 = odd     1 = 2 bits       10 = 7 bits
<1>     ;             11 = even
<1>     ;  Baud rate setting bits: (29/06/2015)
<1>     ;             Retro UNIX 386 v1 feature only !
<1>     ;      Bit    7    6    5  | Baud rate
<1>     ;             ------------------------
<1>     ;      value  0    0    0  | Default (Divisor = 1)
<1>     ;             0    0    1  | 9600 (12)
<1>     ;             0    1    0  | 19200 (6)
<1>     ;             0    1    1  | 38400 (3)
<1>     ;             1    0    0  | 14400 (8)
<1>     ;             1    0    1  | 28800 (4)
<1>     ;             1    1    0  | 57600 (2)
<1>     ;             1    1    1  | 115200 (1)
<1>
<1>     ; References:
<1>     ; (1) IBM PC-XT Model 286 BIOS Source Code
<1>     ;     RS232.ASM --- 10/06/1985 COMMUNICATIONS BIOS (RS232)
<1>     ; (2) Award BIOS 1999 - ATORGS.ASM
<1>     ; (3) http://wiki.osdev.org/Serial_Ports
<1>     ;
<1>     ; Set communication parameters for COM1 (= 03h)
<1>     ;
<1>     mov     ebx, com1p            ; COM1 parameters
<1>     mov     dx, 3F8h              ; COM1
<1>      ; 29/10/2015
<1>     mov     cx, 301h  ; divisor = 1 (115200 baud)
<1>     call    sp_i3  ; call A4
<1>     test    al, 80h
<1>     jz      short sp_i0 ; OK..
<1>             ; Error !
<1>     ;mov    dx, 3F8h
<1>     sub     dl, 5 ; 3FDh -> 3F8h
<1>     mov     cx, 30Eh  ; divisor = 12 (9600 baud)
<1>     call    sp_i3  ; call A4
<1>     test    al, 80h
<1>     jnz     short sp_i1
<1> sp_i0:
<1>         ; (Note: Serial port interrupts will be disabled here...)
<1>         ; (INT 14h initialization code disables interrupts.)
<1>     ;
<1>     mov     byte [ebx], 0E3h ; 11100011b
<1>     call    sp_i5 ; 29/06/2015
<1> sp_i1:
<1>     inc     ebx
<1>     mov     dx, 2F8h              ; COM2
<1>      ; 29/10/2015
<1>     mov     cx, 301h  ; divisor = 1 (115200 baud)
<1>     call    sp_i3  ; call A4
<1>     test    al, 80h
<1>     jz      short sp_i2 ; OK..
<1>             ; Error !
<1>     ;mov    dx, 2F8h
<1>     sub     dl, 5 ; 2FDh -> 2F8h
<1>     mov     cx, 30Eh  ; divisor = 12 (9600 baud)
<1>     call    sp_i3  ; call A4
<1>     test    al, 80h
<1>     jnz     short sp_i7
<1> sp_i2:
<1>     mov     byte [ebx], 0E3h ; 11100011b
<1> sp_i6:
<1>     ;; COM2 - enabling IRQ 3
<1>     ; 29/07/2022
<1>     ; 07/11/2015
<1>     ; 26/10/2015
<1>     pushf
<1>     cli
<1>     ;
<1>     mov     dx, 2FCh              ; modem control register
<1>     in      al, dx                ; read register
<1>     JMP     $+2                   ; I/O DELAY
<1>     or      al, 8                 ; enable bit 3 (OUT2)
<1>     out     dx, al                ; write back to register
<1>     JMP     $+2                   ; I/O DELAY
<1>     ;mov    dx, 2F9h              ; interrupt enable register
<1>     ; 29/07/2022
<1>     mov     dl, 0F9h
<1>     in      al, dx                ; read register
<1>     JMP     $+2                   ; I/O DELAY
<1>     ;or     al, 1                 ; receiver data interrupt enable and
<1>     or      al, 3                 ; transmitter empty interrupt enable
<1>     out     dx, al                ; write back to register
<1>     JMP     $+2                   ; I/O DELAY
<1>     in      al, 21h               ; read interrupt mask register
<1>     JMP     $+2                   ; I/O DELAY
<1>     and     al, 0F7h              ; enable IRQ 3 (COM2)
<1>     out     21h, al               ; write back to register
<1>     ;
<1>     ; 23/10/2015
<1>     mov     eax, com2_int
<1>     mov     [com2_irq3], eax
<1>     ; 26/10/2015
<1>     popf
<1> sp_i7:
<1>     retn
<1>
<1> sp_i3:
<1> ;A4:        ;-----  INITIALIZE THE COMMUNICATIONS PORT
<1>     ; 28/10/2015
<1>     inc     dl      ; 3F9h (2F9h)  ; 3F9h, COM1 Interrupt enable register
<1>     mov     al, 0
<1>     out     dx, al                ; disable serial port interrupt
<1>     JMP     $+2                   ; I/O DELAY
<1>     add     dl, 2   ; 3FBh (2FBh)  ; COM1 Line control register (3FBh)
<1>     mov     al, 80h
<1>     out     dx, al                ; SET DLAB=1 ; divisor latch access bit
<1>     ;-----  SET BAUD RATE DIVISOR
<1>     ; 26/10/2015
<1>     sub     dl, 3   ; 3F8h (2F8h) ; register for least significant byte
<1>                                   ; of the divisor value
<1>     mov     al, cl  ; 1
<1>     out     dx, al                ; 1 = 115200 baud (Retro UNIX 386 v1)
```

```
<1>                                     ; 2 = 57600 baud
<1>                                     ; 3 = 38400 baud
<1>                                     ; 6 = 19200 baud
<1>                                     ; 12 = 9600 baud (Retro UNIX 8086 v1)
<1>     JMP     $+2                     ; I/O DELAY
<1>     sub     al, al
<1>     inc     dl      ; 3F9h (2F9h) ; register for most significant byte
<1>                                     ; of the divisor value
<1>     out     dx, al ; 0
<1>     JMP     $+2                     ; I/O DELAY
<1>     ;
<1>     mov     al, ch ; 3              ; 8 data bits, 1 stop bit, no parity
<1>     ;and    al, 1Fh ; Bits 0,1,2,3,4
<1>     add     dl, 2   ; 3FBh (2FBh)  ; Line control register
<1>     out     dx, al
<1>     JMP     $+2                     ; I/O DELAY
<1>     ; 29/10/2015
<1>     dec     dl      ; 3FAh (2FAh)  ; FIFO Control register (16550/16750)
<1>     xor     al, al                  ; 0
<1>     out     dx, al                  ; Disable FIFOs (reset to 8250 mode)
<1>     JMP     $+2
<1> sp_i4:
<1> ;A18:       ;----- COMM PORT STATUS ROUTINE
<1>     ; 29/06/2015 (line status after modem status)
<1>     add     dl, 4   ; 3FEh (2FEh)  ; Modem status register
<1> sp_i4s:
<1>     in      al, dx                  ; GET MODEM CONTROL STATUS
<1>     JMP     $+2                     ; I/O DELAY
<1>     mov     ah, al                  ; PUT IN (AH) FOR RETURN
<1>     dec     dl      ; 3FDh (2FDh)  ; POINT TO LINE STATUS REGISTER
<1>                                     ; dx = 3FDh for COM1, 2FDh for COM2
<1>     in      al, dx                  ; GET LINE CONTROL STATUS
<1>     ; AL = Line status, AH = Modem status
<1>     retn
<1>
<1> sp_status:
<1>     ; 29/06/2015
<1>     ; 27/06/2015 (Retro UNIX 386 v1)
<1>     ; Get serial port status
<1>     mov     dx, 3FEh                ; Modem status register (COM1)
<1>     sub     dh, al                  ; dh = 2 for COM2 (al = 1)
<1>                                     ; dx = 2FEh for COM2
<1>     jmp     short sp_i4s
<1>
<1> sp_setp: ; Set serial port communication parameters
<1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
<1>     ; 07/11/2015
<1>     ; 29/10/2015
<1>     ; 29/06/2015
<1>     ; Retro UNIX 386 v1 feature only !
<1>     ;
<1>     ; INPUT:
<1>     ;       AL = 0 for COM1
<1>     ;            1 for COM2
<1>     ;       AH = Communication parameters (*)
<1>     ; OUTPUT:
<1>     ;       CL = Line status
<1>     ;       CH = Modem status
<1>     ;   If cf = 1 -> Error code in [u.error]
<1>     ;                'invalid parameter !'
<1>     ;                        or
<1>     ;                'device not ready !' error
<1>     ;
<1>     ;  (*) Communication parameters (except BAUD RATE):
<1>     ;       Bit     4       3       2       1       0
<1>     ;               -PARITY--   STOP BIT  -WORD LENGTH-
<1>     ;  this one -->       00 = none    0 = 1 bit  11 = 8 bits
<1>     ;               01 = odd     1 = 2 bits       10 = 7 bits
<1>     ;               11 = even
<1>     ;  Baud rate setting bits: (29/06/2015)
<1>     ;               Retro UNIX 386 v1 feature only !
<1>     ;       Bit     7    6    5  | Baud rate
<1>     ;               ------------------------
<1>     ;       value   0    0    0  | Default (Divisor = 1)
<1>     ;               0    0    1  | 9600 (12)
<1>     ;               0    1    0  | 19200 (6)
<1>     ;               0    1    1  | 38400 (3)
<1>     ;               1    0    0  | 14400 (8)
<1>     ;               1    0    1  | 28800 (4)
<1>     ;               1    1    0  | 57600 (2)
<1>     ;               1    1    1  | 115200 (1)
<1>     ;
<1>     ; (COM1 base port address = 3F8h, COM1 Interrupt = IRQ 4)
<1>     ; (COM2 base port address = 2F8h, COM1 Interrupt = IRQ 3)
<1>     ;
<1>     ; ((Modified registers: EAX, ECX, EDX, EBX))
<1>     ;
<1>     mov     dx, 3F8h
<1>     mov     ebx, com1p ; COM1 control byte offset
<1>     cmp     al, 1
<1>     ja      short sp_invp_err
<1>     jb      short sp_setp1 ;  COM1 (AL = 0)
<1>     dec     dh ; 2F8h
<1>     inc     ebx ; COM2 control byte offset
<1> sp_setp1:
<1>     ; 29/10/2015
<1>     mov     [ebx], ah
<1>     movzx   ecx, ah
<1>     shr     cl, 5 ; -> baud rate index
<1>     and     ah, 1Fh ; communication parameters except baud rate
<1>     mov     al, [ecx+b_div_tbl]
<1>     mov     cx, ax
<1>     call    sp_i3
<1>     mov     cx, ax ; CL = Line status, CH = Modem status
<1>     test    al, 80h
<1>     jz      short sp_setp2
<1>         mov     byte [ebx], 0E3h ; Reset to initial value (11100011b)
<1> stp_dnr_err:
<1>     mov     dword [u.error], ERR_DEV_NOT_RDY ; 'device not ready !'
<1>     ; CL = Line status, CH = Modem status
<1>     stc
<1>     retn
<1> sp_setp2:
<1>     cmp     dh, 2 ; COM2 (2F?h)
<1>     ;jna    sp_i6
<1>                     ; COM1 (3F?h)
<1>     ; 29/07/2022
<1>     ja      short sp_i5
<1>     jmp     sp_i6
<1> sp_i5:
<1>     ; 29/07/2022
<1>     ; 07/11/2015
<1>     ; 26/10/2015
<1>     ; 29/06/2015
```

```
                                <1>     ;
                                <1>     ;; COM1 - enabling IRQ 4
                                <1>     pushf
                                <1>     cli
                                <1>     mov     dx, 3FCh                ; modem control register
                                <1>     in      al, dx                  ; read register
                                <1>     JMP     $+2                     ; I/O DELAY
                                <1>     or      al, 8                   ; enable bit 3 (OUT2)
                                <1>     out     dx, al                  ; write back to register
                                <1>     JMP     $+2                     ; I/O DELAY
                                <1>     ;mov    dx, 3F9h                ; interrupt enable register
                                <1>     ; 29/07/2022
                                <1>     mov     dl, 0F9h
                                <1>     in      al, dx                  ; read register
                                <1>     JMP     $+2                     ; I/O DELAY
                                <1>     ;or     al, 1                   ; receiver data interrupt enable and
                                <1>     or      al, 3                   ; transmitter empty interrupt enable
                                <1>     out     dx, al                  ; write back to register
                                <1>     JMP     $+2                     ; I/O DELAY
                                <1>     in      al, 21h                 ; read interrupt mask register
                                <1>     JMP     $+2                     ; I/O DELAY
                                <1>     and     al, 0EFh                ; enable IRQ 4 (COM1)
                                <1>     out     21h, al                 ; write back to register
                                <1>     ;
                                <1>     ; 23/10/2015
                                <1>     mov     eax, com1_int
                                <1>     mov     [com1_irq4], eax
                                <1>     ; 26/10/2015
                                <1>     popf
                                <1>     retn
                                <1>
                                <1> sp_invp_err:
                                <1>     mov     dword [u.error], ERR_INV_PARAMETER ; 'invalid parameter !'
                                <1>     xor     ecx, ecx
                                <1>     dec     ecx ; 0FFFFh
                                <1>     stc
                                <1>     retn
                                <1>
                                <1> ; 29/10/2015
                                <1> b_div_tbl: ; Baud rate divisor table (115200/divisor)
                                <1>     db 1, 12, 6, 3, 8, 4, 1
                                <1>
                                <1> %endif
                                <1>
                                <1> ; 23/10/2015
                                <1> com1_irq4:
00011CCC [D41C0100]             <1>     dd dummy_retn
                                <1> com2_irq3:
00011CD0 [D41C0100]             <1>     dd dummy_retn
                                <1>
                                <1> ; 21/11/2023
                                <1> dummy_retn:
                                <1>     ;retn
                                <1> wakeup:
                                <1>     ; 24/01/2016
00011CD4 C3                     <1>     retn
                                <1>
                                <1> set_working_path_x:
                                <1>             ; 17/10/2016 (TRDOS 386 - FFF & FNF)
                                <1>             ;mov    ax, 1
                                <1>                     ; File name is needed/forced (AL=1)
                                <1>                     ; Change directory as temporary (AH=0)
                                <1>             ; 29/07/2022
00011CD5 31C0                   <1>             xor     eax, eax
00011CD7 FEC0                   <1>             inc     al
                                <1>             ; eax = 1
                                <1> set_working_path_xx: ; 30/12/2017 (syschdir)
                                <1>             ; This is needed for preventing wrong Find Next File
                                <1>             ; system call after sysopen, syscreate, sysmkdir etc.
                                <1>             ; Find Next File must immediate follow Find First File)
                                <1>
00011CD9 8825[6C830100]         <1>             mov     [FFF_Valid], ah ; 0 ; reset ; 17/10/2016
                                <1>
                                <1> set_working_path:
                                <1>             ; 08/08/2022
                                <1>             ; 29/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>             ; 16/10/2016
                                <1>             ; 12/10/2016
                                <1>             ; 10/10/2016
                                <1>             ; 05/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>             ;
                                <1>             ; TRDOS v1.0 (DIR.ASM, "proc_set_working_path")
                                <1>                ; 27/01/2011 - 08/02/2011
                                <1>             ; Set/Changes current drive, directory and file
                                <1>             ; depending on command tail
                                <1>             ; (procedure is derivated from CMD_INTR.ASM
                                <1>             ; file or dir locating code of internal commands)
                                <1>             ; (This procedure is prepared for INT 21H file/dir
                                <1>             ; functions and also to get compact code for
                                <1>             ; internal mainprog -command interpreter- commands)
                                <1>             ;
                                <1>             ; INPUT: DS:SI -> Command tail (ASCIIZ string)
                                <1>             ; AL = 0 -> any, AL > 0 -> file name is forced
                                <1>             ; AH = CD -> Change directory permanently
                                <1>             ; AH <> CD -> Change directory as temporary
                                <1>             ;
                                <1>             ; OUTPUT: ES=DS, FindFile structure has been set
                                <1>             ;       RUN_CDRV points previous current drive
                                <1>             ;       DS:SI = FindFile structure address
                                <1>             ;       (DS=CS)
                                <1>             ;       AX, BX, CX, DX, DI will be changed
                                <1>             ;   cf = 1 -> Error code in AX (AL)
                                <1>             ;       stc & AX = 0 -> Bad command or path name
                                <1>             ; ---------------------------------------------
                                <1>             ;
                                <1>             ; TRDOS 386 (05/10/2016)
                                <1>             ; INPUT:
                                <1>             ;      ESI = File/Directory Path (ASCIIZ string)
                                <1>             ;            address in user's memory space
                                <1>             ;      AL = 0 -> any
                                <1>             ;      AL > 0 -> file name is forced
                                <1>             ;      AH = CD -> change directory as permanent
                                <1>             ;      AH <> CD -> change directory as temporary
                                <1>             ;
                                <1>             ; OUTPUT:
                                <1>             ;      FindFile structure has been set
                                <1>             ;      RUN_CDRV points previous current drive
                                <1>             ;      ESI = FindFile_Name address ; 12/10/2016
                                <1>             ;
                                <1>             ;      cf = 1 -> Error code in EAX (AL)
                                <1>             ;      stc & EAX = 0 -> Bad command or path name
                                <1>             ;
                                <1>             ; Modified registers: EAX, EBX, ECX, EDX, ESI, EDI
                                <1>
```

```
00011CDF 66A3[70830100]      <1>             mov     [SWP_Mode], ax
00011CE5 A0[76770100]        <1>             mov     al, [Current_Drv]
00011CEA 30E4                <1>             xor     ah, ah
00011CEC 66A3[72830100]      <1>             mov     [SWP_DRV], ax
                             <1>
                             <1>             ; TRDOS 386 ring 3 (user's page directory)
                             <1>             ; to ring 0 (kernel's page directory)
                             <1>             ; transfer modifications (05/10/2016).
                             <1>
00011CF2 55                  <1>             push    ebp
00011CF3 89E5                <1>             mov     ebp, esp
                             <1>
                             <1>             ;mov    ecx, 128 ; maximum path length = 128 bytes
                             <1>             ; 29/07/2022
00011CF5 31C9                <1>             xor     ecx, ecx
00011CF7 B180                <1>             mov     cl, 128
00011CF9 29CC                <1>             sub     esp, ecx ; reserve 128 bytes (buffer) on stack
00011CFB 89E7                <1>             mov     edi, esp ; destination address (kernel space)
                             <1>             ; esi = source address (virtual, in user's memory space)
00011CFD E863EEFFFF          <1>             call    transfer_from_user_buffer
00011D02 720D                <1>             jc      short loc_swp_xor_retn
                             <1>
00011D04 89E6                <1>             mov     esi, esp ; temporary buffer (the path) on stack
                             <1> loc_swp_fchar:
00011D06 8A06                <1>             mov     al, [esi]
00011D08 3C20                <1>             cmp     al, 20h
00011D0A 7711                <1>             ja      short loc_swp_parse_path_name
                             <1>             ;je     short loc_swp_fchar_next
                             <1>             ; 29/07/2022
00011D0C 7203                <1>             jb      short loc_swp_xor_retn
                             <1>
                             <1> loc_swp_fchar_next:
00011D0E 46                  <1>             inc     esi
00011D0F EBF5                <1>             jmp     short loc_swp_fchar
                             <1>
                             <1> loc_swp_xor_retn:
00011D11 31C0                <1>             xor     eax, eax
00011D13 F9                  <1>             stc
                             <1> loc_swp_retn:
00011D14 89EC                <1>             mov     esp, ebp
00011D16 5D                  <1>             pop     ebp
                             <1>
                             <1>             ;mov    esi, FindFile_Drv
00011D17 BE[58800100]        <1>             mov     esi, FindFile_Name ; 12/10/2016
00011D1C C3                  <1>             retn
                             <1>
                             <1> ;loc_swp_fchar_next:
                             <1> ;           inc     esi
                             <1> ;           jmp     short loc_swp_fchar
                             <1>
                             <1> loc_swp_parse_path_name:
00011D1D BF[16800100]        <1>             mov     edi, FindFile_Drv
00011D22 E8578CFFFF          <1>             call    parse_path_name
00011D27 72EB                <1>             jc      short loc_swp_retn
                             <1>
                             <1> loc_swp_checkfile_name:
00011D29 803D[70830100]00    <1>             cmp     byte [SWP_Mode], 0
00011D30 761E                <1>             jna     short loc_swp_drv
                             <1>
                             <1>             ; 10/10/2016 (valid file name checking)
00011D32 BE[58800100]        <1>             mov     esi, FindFile_Name
00011D37 803E20              <1>             cmp     byte [esi], 20h
00011D3A 76D5                <1>             jna     short loc_swp_xor_retn
                             <1>
                             <1>             ; 16/10/2016
00011D3C C605[6F830100]00    <1>             mov     byte [SWP_inv_fname], 0 ; reset
                             <1>             ; esi = file name address (ASCIIZ)
00011D43 E8916FFFFF          <1>             call    check_filename
00011D48 7306                <1>             jnc     short loc_swp_drv
                             <1>
00011D4A FE05[6F830100]      <1>             inc     byte [SWP_inv_fname] ; set
                             <1> loc_swp_drv:
00011D50 8A35[76770100]      <1>             mov     dh, [Current_Drv]
                             <1>                 ;mov       [RUN_CDRV], dh
                             <1>
00011D56 8A15[16800100]      <1>             mov     dl, [FindFile_Drv]
                             <1>                 ;cmp       dl, dh
00011D5C 3A15[76770100]      <1>             cmp     dl, [Current_Drv]
00011D62 740D                <1>             je      short loc_swp_change_directory
                             <1>
00011D64 FE05[73830100]      <1>             inc     byte [SWP_DRV_chg]
00011D6A E88159FFFF          <1>             call    change_current_drive
00011D6F 72A3                <1>             jc      short loc_swp_retn ; eax = error code
                             <1>             ; eax = 0
                             <1>
                             <1> loc_swp_change_directory:
00011D71 803D[17800100]21    <1>             cmp     byte [FindFile_Directory], 21h
00011D78 F5                  <1>             cmc
00011D79 7399                <1>             jnc     short loc_swp_retn
                             <1>
00011D7B FE05[73830100]      <1>             inc     byte [SWP_DRV_chg]
00011D81 FE05[8E2E0100]      <1>             inc     byte [Restore_CDIR]
00011D87 BE[17800100]        <1>             mov     esi, FindFile_Directory
00011D8C 8A25[71830100]      <1>             mov     ah, [SWP_Mode+1]
00011D92 E85486FFFF          <1>             call    change_current_directory
                             <1>             ;jc     short loc_swp_retn ; eax = error code
                             <1>             ; 08/08/2022
00011D97 7305                <1>             jnc     short loc_swp_change_prompt_dir_string
00011D99 E976FFFFFF          <1>             jmp     loc_swp_retn
                             <1>
                             <1> loc_swp_change_prompt_dir_string:
                             <1>             ; esi = PATH_Array
                             <1>             ; eax = Current Directory First Cluster
                             <1>             ; edi = Logical DOS Drive Description Table
00011D9E E87685FFFF          <1>             call    change_prompt_dir_str
00011DA3 29C0                <1>             sub     eax, eax ; 0
00011DA5 E96AFFFFFF          <1>             jmp     loc_swp_retn
                             <1>
                             <1> reset_working_path:
                             <1>             ; 06/10/2016 - TRDOS 386 (TRDOS v2.0)
                             <1>             ;
                             <1>             ; TRDOS v1.0 (DIR.ASM, "proc_reset_working_path")
                             <1>             ; 05/02/2011 - 08/02/2011
                             <1>             ;
                             <1>             ; Restores current drive and directory
                             <1>             ;
                             <1>             ; INPUT: none
                             <1>             ; OUTPUT: DL = SWP_DRV, EAX = 0 -> OK
                             <1>             ;
                             <1>             ;    AX = 0 -> ESI = Logical Dos Drv Desc. Table
                             <1>             ;
                             <1>             ;    EAX, EBX, ECX, EDX, ESI, EDI will be changed
                             <1>             ;
                             <1>
```

```
                                <1>
00011DAA 31C0                   <1>             xor     eax, eax
00011DAC 48                     <1>             dec     eax
                                <1>
00011DAD 668B15[72830100]       <1>             mov     dx, [SWP_DRV]
00011DB4 08F6                   <1>             or      dh, dh
00011DB6 742E                   <1>             jz      short loc_rwp_return
                                <1>
00011DB8 3A15[76770100]         <1>             cmp     dl, [Current_Drv]
00011DBE 7407                   <1>             je      short loc_rwp_restore_cdir
                                <1> loc_rwp_restore_cdrv:
00011DC0 E82B59FFFF             <1>             call    change_current_drive
00011DC5 EB10                   <1>             jmp     short loc_rwp_restore_ok
                                <1> loc_rwp_restore_cdir:
00011DC7 31DB                   <1>             xor     ebx, ebx
00011DC9 88D7                   <1>             mov     bh, dl
00011DCB BE00010900             <1>             mov     esi, Logical_DOSDisks
00011DD0 01DE                   <1>             add     esi, ebx
                                <1>
00011DD2 E8CA59FFFF             <1>             call    restore_current_directory
                                <1>
                                <1> loc_rwp_restore_ok:
00011DD7 668B15[72830100]       <1>             mov     dx, [SWP_DRV]
00011DDE 31C0                   <1>             xor     eax, eax
00011DE0 66A3[73830100]         <1>             mov     [SWP_DRV_chg], ax
                                <1> loc_rwp_return:
00011DE6 C3                     <1>             retn
                                <1>
                                <1> get_file_name:
                                <1>             ; 25/08/2024 (TRDOS 386 Kernel v2.0.9)
                                <1>             ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>             ; 15/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>             ; Convert file name
                                <1>             ;       from directory entry format
                                <1>                 ;  to (8.3) dot file name format
                                <1>             ;
                                <1>             ; TRDOS v1.0 (DIR.ASM, "get_file_name")
                                <1>                 ; 2005 - 09/10/2011
                                <1>             ; INPUT:
                                <1>             ;       DS:SI -> Directory Entry Format File Name
                                <1>             ;        ES:DI -> DOS Dot File Name Address
                                <1>             ; OUTPUT:
                                <1>             ;       DS:SI -> DOS Dot File Name Address
                                <1>                 ;  ES:DI -> Directory Entry Format File Name
                                <1>             ;
                                <1>             ; TRDOS 386 (15/10/2016)
                                <1>             ; INPUT:
                                <1>             ;       ESI = File name addr in dir entry format
                                <1>             ;       EDI = Dot file name address (destination)
                                <1>             ; OUTPUT:
                                <1>             ;       File name is converted and moved
                                <1>             ;       to destination (as 8.3 dot filename)
                                <1>             ;
                                <1>             ; Modified registers: EAX, ECX
                                <1>
                                <1>                 ; 2005 (TRDOS 8086) - 2016 (TRDOS 386)
                                <1>
00011DE7 57                     <1>             push    edi
00011DE8 56                     <1>             push    esi
00011DE9 AC                     <1>             lodsb
                                <1>             ; 25/08/2024
00011DEA 31C9                   <1>             xor     ecx, ecx ; 0
00011DEC 3C20                   <1>             cmp     al, 20h
00011DEE 7626                   <1>             jna     short pass_gfn_ext
                                <1>             ; 25/08/2024
                                <1>             ;push   esi
00011DF0 AA                     <1>             stosb
                                <1>             ; 25/08/2024
                                <1>             ; 29/07/2022
                                <1>             ;xor    ecx, ecx
                                <1>             ; ecx <= 128 ; 25/08/2024
00011DF1 B107                   <1>             mov     cl, 7
                                <1>             ; 25/08/2024
00011DF3 01CE                   <1>             add     esi, ecx ; add esi, 7
00011DF5 56                     <1>             push    esi ; (*)
                                <1> loc_gfn_next_char:
00011DF6 AC                     <1>             lodsb
00011DF7 3C20                   <1>             cmp     al, 20h
00011DF9 7603                   <1>             jna     short pass_gfn_fn
00011DFB AA                     <1>             stosb
00011DFC E2F8                   <1>             loop    loc_gfn_next_char
                                <1> pass_gfn_fn:
                                <1>             ;pop    esi
                                <1>             ;add    esi, 7
                                <1>             ; 25/08/2024
00011DFE 5E                     <1>             pop     esi ; (*)
                                <1>
00011DFF AC                     <1>             lodsb
00011E00 3C20                   <1>             cmp     al, 20h
00011E02 7612                   <1>             jna     short pass_gfn_ext
00011E04 B42E                   <1>             mov     ah, '.'
00011E06 86E0                   <1>             xchg    ah, al
00011E08 66AB                   <1>             stosw
00011E0A AC                     <1>             lodsb
00011E0B 3C20                   <1>             cmp     al, 20h
00011E0D 7607                   <1>             jna     short pass_gfn_ext
00011E0F AA                     <1>             stosb
00011E10 AC                     <1>             lodsb
00011E11 3C20                   <1>             cmp     al, 20h
00011E13 7601                   <1>             jna     short pass_gfn_ext
00011E15 AA                     <1>             stosb
                                <1> pass_gfn_ext:
00011E16 30C0                   <1>             xor     al, al
00011E18 AA                     <1>             stosb
00011E19 5E                     <1>             pop     esi
00011E1A 5F                     <1>             pop     edi
                                <1>             ; 25/08/2024
                                <1>             ; ecx <= 7
00011E1B C3                     <1>             retn
                                <1>
                                <1> set_hardware_int_vector:
                                <1>             ; 18/03/2017
                                <1>             ; 03/03/2017
                                <1>             ; 28/02/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>             ;
                                <1>             ; SET/RESET HARDWARE INTERRUPT GATE
                                <1>             ;
                                <1>             ; Changes interrupt gate descriptor table
                                <1>             ; (without changing default interrupt list)
                                <1>             ;
                                <1>             ; INPUT:
                                <1>             ;       AL = IRQ number (0 to 15)
                                <1>             ;       AH > 0 -> set
                                <1>             ;       AH = 0 -> reset
```

```
                                <1>             ;
                                <1>             ; Modified registers: eax, ebx, edx, edi
                                <1>             ;
                                <1>
00011E1C C0E002                 <1>             shl     al, 2 ; IRQ number * 4
00011E1F 0FB6D8                 <1>             movzx   ebx, al
                                <1>
00011E22 08E4                   <1>             or      ah, ah
00011E24 7508                   <1>             jnz     short shintv_1 ; set (for user call service)
                                <1>
                                <1>             ; 18/03/2017
00011E26 81C3[78360100]         <1>             add     ebx, IRQ_list ; reset to default interrupt list
00011E2C EB06                   <1>             jmp     short shintv_2
                                <1> shintv_1:
00011E2E 81C3[551E0100]         <1>             add     ebx, IRQ_u_list
                                <1> shintv_2:
00011E34 8B13                   <1>             mov     edx, [ebx] ; IRQ handler address
                                <1>
                                <1>             ; 03/03/2017
00011E36 D0E0                   <1>             shl     al, 1 ; IRQ number * 8
                                <1>             ; 18/03/2017
00011E38 0FB6F8                 <1>             movzx   edi, al
00011E3B 81C7[C8740100]         <1>             add     edi, idt + (8*32) ; IRQ 0 offset = idt + 256
                                <1>
00011E41 89D0                   <1>             mov     eax, edx ; IRQ handler address
00011E43 BB00000800             <1>             mov     ebx, 80000h
                                <1>
                                <1>             ;mov    edx, eax
00011E48 66BA008E               <1>             mov     dx, 8E00h
00011E4C 6689C3                 <1>             mov     bx, ax
00011E4F 89D8                   <1>             mov     eax, ebx ; /* selector = 0x0008 = cs */
                                <1>                                      ; /* interrupt gate - dpl=0, present */
00011E51 AB                     <1>             stosd   ; selector & offset bits 0-15
00011E52 8917                   <1>             mov     [edi], edx ; attributes & offset bits 16-23
                                <1>
00011E54 C3                     <1>             retn
                                <1> IRQ_u_list:
                                <1>             ; 28/02/2017
00011E55 [7E090000]             <1>             dd      timer_int
00011E59 [E3100000]             <1>             dd      kb_int
00011E5D [6A0B0000]             <1>             dd      irq2
00011E61 [951E0100]             <1>             dd      IRQ_service3
00011E65 [9F1E0100]             <1>             dd      IRQ_service4
00011E69 [A91E0100]             <1>             dd      IRQ_service5
00011E6D [A44E0000]             <1>             dd      fdc_int
00011E71 [B31E0100]             <1>             dd      IRQ_service7
00011E75 [F20A0000]             <1>             dd      rtc_int
00011E79 [BD1E0100]             <1>             dd      IRQ_service9
00011E7D [C71E0100]             <1>             dd      IRQ_service10
00011E81 [D11E0100]             <1>             dd      IRQ_service11
00011E85 [DB1E0100]             <1>             dd      IRQ_service12
00011E89 [E51E0100]             <1>             dd      IRQ_service13
00011E8D [E3570000]             <1>             dd      hdc1_int
00011E91 [06580000]             <1>             dd      hdc2_int
                                <1>
                                <1>             ; 03/03/2017
                                <1>             ; 27/02/2017
                                <1> IRQ_service3:
00011E95 36C605[E0880100]03     <1>             mov     byte [ss:IRQnum], 3
00011E9D EB4E                   <1>             jmp     short IRQ_service
                                <1> IRQ_service4:
00011E9F 36C605[E0880100]04     <1>             mov     byte [ss:IRQnum], 4
00011EA7 EB44                   <1>             jmp     short IRQ_service
                                <1> IRQ_service5:
00011EA9 36C605[E0880100]05     <1>             mov     byte [ss:IRQnum], 5
00011EB1 EB3A                   <1>             jmp     short IRQ_service
                                <1> IRQ_service7:
00011EB3 36C605[E0880100]07     <1>             mov     byte [ss:IRQnum], 7
00011EBB EB30                   <1>             jmp     short IRQ_service
                                <1> IRQ_service9:
00011EBD 36C605[E0880100]09     <1>             mov     byte [ss:IRQnum], 9
00011EC5 EB26                   <1>             jmp     short IRQ_service
                                <1> IRQ_service10:
00011EC7 36C605[E0880100]0A     <1>             mov     byte [ss:IRQnum], 10
00011ECF EB1C                   <1>             jmp     short IRQ_service
                                <1> IRQ_service11:
00011ED1 36C605[E0880100]0B     <1>             mov     byte [ss:IRQnum], 11
00011ED9 EB12                   <1>             jmp     short IRQ_service
                                <1> IRQ_service12:
00011EDB 36C605[E0880100]0C     <1>             mov     byte [ss:IRQnum], 12
00011EE3 EB08                   <1>             jmp     short IRQ_service
                                <1> IRQ_service13:
00011EE5 36C605[E0880100]0D     <1>             mov     byte [ss:IRQnum], 13
                                <1>             ;jmp    short IRQ_service
                                <1> IRQ_service:
                                <1>             ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>             ; 13/06/2017
                                <1>             ; 11/06/2017
                                <1>             ; 10/06/2017
                                <1>             ; 01/03/2017, 04/03/2017
                                <1>             ; 27/02/2017, 28/02/2017
00011EED 1E                     <1>             push    ds
00011EEE 06                     <1>             push    es
00011EEF 0FA0                   <1>             push    fs
00011EF1 0FA8                   <1>             push    gs
                                <1>
00011EF3 60                     <1>             pushad  ; eax,ecx,edx,ebx,esp,ebp,esi,edi
00011EF4 66B91000               <1>             mov     cx, KDATA
00011EF8 8ED9                   <1>             mov     ds, cx
00011EFA 8EC1                   <1>             mov     es, cx
00011EFC 8EE1                   <1>             mov     fs, cx
00011EFE 8EE9                   <1>             mov     gs, cx
                                <1>
00011F00 0F20D8                 <1>             mov     eax, cr3
00011F03 A3[DC880100]           <1>             mov     [IRQ_cr3], eax
                                <1>
00011F08 A1[B0760100]           <1>             mov     eax, [k_page_dir]
00011F0D 0F22D8                 <1>             mov     cr3, eax
                                <1>
00011F10 A0[E0880100]           <1>             mov     al, [IRQnum]
                                <1>
                                <1>             ;mov    cl, [sysflg]
                                <1>             ;mov    [u.r_mode], cl  ; system (0) or user mode (FFh)
                                <1> IRQsrv_0:
00011F15 0FB6D8                 <1>             movzx   ebx, al
00011F18 8A9B[B0350100]         <1>             mov     bl, [ebx+IRQenum] ; IRQ (available) index number + 1
                                <1>             ; 01/03/2017
00011F1E FECB                   <1>             dec     bl  ; IRQ index number, 0 to 8
                                <1>             ;js     IRQsrv_5 ; not available to use here!?
                                <1>             ; 29/07/2022
00011F20 785E                   <1>             js      short IRQsrv_j5  ; (jump to IRQsrv_5)
                                <1>             ;
00011F22 80BB[A6880100]80       <1>             cmp     byte [ebx+IRQ.method], 80h ; using by a dev or kernel?
00011F29 7205                   <1>             jb      short IRQsrv_1 ; no
```

```
                                <1>
                                <1>             ; If the IRQ service is already owned by TRDOS 386 kernel
                                <1>             ;       or a Device driver
                                <1>             ; we need to call 'dev_IRQ_service'
                                <1>
                                <1>             ; IRQ number in AL
00011F2B E80E010000             <1>             call   dev_IRQ_service        ; IRQ service for device drivers
                                <1>             ; IRQ number in AL
                                <1> IRQsrv_1:
                                <1>             ; check user callback service status
                                <1>             ; AL = IRQ number
                                <1>             ; EBX = IRQ (Available) Index number
                                <1>
00011F30 A2[E78E0100]           <1>             mov    [u.irqwait], al ; set waiting IRQ flag
                                <1>
00011F35 8A83[94880100]         <1>             mov    al, [ebx+IRQ.owner]
00011F3B 20C0                   <1>             and    al, al
                                <1>             ;jz    IRQsrv_5 ; it is not owned by a user/proc
                                <1>             ; 29/07/2022
00011F3D 7441                   <1>             jz     short IRQsrv_j5  ; (jump to IRQsrv_5)
                                <1>
                                <1>             ; 03/03/2017
00011F3F 89DA                   <1>             mov    edx, ebx
00011F41 C0E202                 <1>             shl    dl, 2
00011F44 8B92[B8880100]         <1>             mov    edx, [edx+IRQ.addr] ; S.R.B. or Callback service addr
                                <1>
00011F4A 8AA3[A6880100]         <1>             mov    ah, [ebx+IRQ.method]
00011F50 F6C401                 <1>             test   ah, 1
00011F53 7530                   <1>             jnz    short IRQsrv_4 ; Callback service method
                                <1>
                                <1>             ; Signal Response Byte method
                                <1>             ;mov   edx, [edx+IRQ.addr] ; Signal Response Byte address
                                <1>             ;                          ; (Physical address, non-swappable)
00011F55 80E402                 <1>             and    ah, 2 ; bit 1, (S.R.B.) counter (auto increment) method
00011F58 8AA3[AF880100]         <1>             mov    ah, [ebx+IRQ.srb] ; Signal Response Byte value
00011F5E 7408                   <1>             jz     short IRQsrv_2 ; fixed S.R.B. value
                                <1>             ; counter method (auto increment)
00011F60 FEC4                   <1>             inc    ah
00011F62 88A3[AF880100]         <1>             mov    [ebx+IRQ.srb], ah ; next (count) number
                                <1> IRQsrv_2:
00011F68 8822                   <1>             mov    [edx], ah ; put S.R.B. val to the user's S.R.B. addr
00011F6A C605[E78E0100]00       <1>             mov    byte [u.irqwait], 0 ; clear waiting IRQ flag
                                <1>
00011F71 3A05[BD8E0100]         <1>             cmp    al, [u.uno]
                                <1>             ;je    IRQsrv_5 ; the owner is current user/process
                                <1>             ; 29/07/2022
00011F77 7407                   <1>             je     short IRQsrv_j5  ; (jump to IRQsrv_5)
                                <1> IRQsrv_3:
                                <1>             ; the owner is not current user/process
                                <1>             ; AL = process number
00011F79 B202                   <1>             mov    dl, 2 ; priority, 2 = event (high)
00011F7B E87BFBFFFF             <1>             call   set_run_sequence
                                <1>
                                <1>             ; [u.irqwait] = waiting IRQ number for callback service
                                <1> IRQsrv_j5:          ; 29/07/2022
00011F80 E998000000             <1>             jmp    IRQsrv_5
                                <1> IRQsrv_4:
00011F85 3A05[BD8E0100]         <1>             cmp    al, [u.uno]  ; is the owner is current user/process?
00011F8B 75EC                   <1>             jne    short IRQsrv_3 ; no !
                                <1>
                                <1>             ; Check if an IRQ callback service already in progress
00011F8D 803D[E88E0100]00       <1>             cmp    byte [u.r_lock], 0
                                <1>             ;ja    IRQsrv_5 ; nothing to do !
                                <1>                            ; (we need to complete prev callback)
                                <1>             ; 29/07/2022
00011F94 77EA                   <1>             ja     short IRQsrv_j5  ; (jump to IRQsrv_5)
                                <1>
00011F96 803D[E48E0100]00       <1>             cmp    byte [u.t_lock], 0
00011F9D 777E                   <1>             ja     short IRQsrv_5 ; nothing to do !
                                <1>                            ; (we need to complete timer callback)
                                <1>
                                <1>             ; 04/03/2017
00011F9F C605[E78E0100]00       <1>             mov    byte [u.irqwait], 0 ; reset/clear waiting IRQ flag
                                <1>
00011FA6 FE05[E88E0100]         <1>             inc    byte [u.r_lock] ; 'IRQ callback service in progress' flag
                                <1>
00011FAC 8A0D[608E0100]         <1>             mov    cl, [sysflg]   ; (system call) mode flag (kernel/user)
00011FB2 880D[E98E0100]         <1>             mov    [u.r_mode], cl ; system mode (0) or user mode (FFh)
                                <1>
                                <1>             ;
00011FB8 8B2D[4C760100]         <1>             mov    ebp, [tss.esp0] ; kernel stack address (for ring 0)
00011FBE 83ED14                 <1>             sub    ebp, 20        ; eip, cs, eflags, esp, ss
00011FC1 892D[648E0100]         <1>             mov    [u.sp], ebp
00011FC7 8925[688E0100]         <1>             mov    [u.usp], esp
                                <1>
                                <1>             ;or    word [ebp+8], 200h ; 22/01/2017, force enabling interrupts
                                <1>
00011FCD 8B44241C               <1>             mov    eax, [esp+28] ; pushed eax
00011FD1 A3[6C8E0100]           <1>             mov    [u.r0], eax
                                <1>
00011FD6 E8FCE9FFFF             <1>             call   wswap ; save user's registers & status
                                <1>
                                <1>             ; software int is in ring 0 but IRQ handler must return to ring 3
                                <1>             ; so, ring 3 return address and stack registers
                                <1>             ; (eip, cs, eflags, esp, ss)
                                <1>             ; must be copied to IRQ handler return
                                <1>             ; eip will be replaced by callback service routine address
                                <1>
00011FDB C605[608E0100]FF       <1>             mov    byte [sysflg], 0FFh ; user mode
                                <1>
                                <1>             ; system mode (system call)
                                <1>             ;mov   ebp, [u.sp] ; EIP (u), CS (UCODE), EFLAGS (u),
                                <1>                                ; ESP (u), SS (UDATA)
                                <1>
00011FE2 8B4510                 <1>             mov    eax, [ebp+16]  ; SS (UDATA)
00011FE5 89E6                   <1>             mov    esi, esp
00011FE7 50                     <1>             push   eax
00011FE8 50                     <1>             push   eax
00011FE9 89E7                   <1>             mov    edi, esp
00011FEB 893D[688E0100]         <1>             mov    [u.usp], edi
00011FF1 B908000000             <1>             mov    ecx, ((ESPACE/4) - 4) ; except DS, ES, FS, GS
00011FF6 F3A5                   <1>             rep    movsd
00011FF8 B104                   <1>             mov    cl, 4
00011FFA F3AB                   <1>             rep    stosd
00011FFC 893D[648E0100]         <1>             mov    [u.sp], edi
00012002 89EE                   <1>             mov    esi, ebp
00012004 B105                   <1>             mov    cl, 5 ; EIP (u), CS (UCODE), EFLAGS (u), ESP (u), SS (UDATA)
00012006 F3A5                   <1>             rep    movsd
                                <1>             ;
                                <1>
00012008 8B0D[C48E0100]         <1>             mov    ecx, [u.pgdir]
0001200E 890D[DC880100]         <1>             mov    [IRQ_cr3], ecx
                                <1>
                                <1> set_IRQ_callback_addr:
```

```
                                <1>             ;
                                <1>             ; This routine sets return address
                                <1>             ; to start of user's interrupt
                                <1>             ; service (callback) address
                                <1>             ;
                                <1>             ; INPUT:
                                <1>             ;      EDX = callback routine/service address
                                <1>             ;            (virtual, not physical address!)
                                <1>             ;      [u.sp] = kernel stack, points to
                                <1>             ;               user's EIP,CS,EFLAGS,ESP,SS
                                <1>             ;               registers.
                                <1>             ; OUTPUT:
                                <1>             ;      EIP (user) = callback (service) address
                                <1>             ;      CS (user) = UCODE
                                <1>             ;      EFLAGS (user) = flags before callback
                                <1>             ;       ESP (user) = ESP-4 (user, before callback)
                                <1>             ;      [ESP](user) = EIP (user) before callback
                                <1>             ;
                                <1>             ; Note: If CPU was in user mode while entering
                                <1>             ;      the timer interrupt service routine,
                                <1>             ;      'IRET' will get return to callback routine
                                <1>             ;      immediately. If CPU was in system/kernel mode
                                <1>             ;      'iret' will get return to system call and
                                <1>             ;      then, callback routine will be return address
                                <1>             ;      from system call. (User's callback/service code
                                <1>             ;      will be able to return to normal return address
                                <1>             ;      via a 'sysrele' system call at the end.)
                                <1>             ;
                                <1>             ; Note: User's IRQ callback service code must be ended
                                <1>             ;      with a 'sysrele' system call !
                                <1>             ;
                                <1>             ;      For example:
                                <1>             ;
                                <1>             ;      audio_IRQ_callback:
                                <1>             ;          ...
                                <1>             ;          <load DMA buffer with audio data>
                                <1>             ;          ...
                                <1>             ;          mov eax, 39 ; 'sysrele'
                                <1>             ;          int 40h ; TRDOS 386 system call (interrupt)
                                <1>             ;
                                <1>
                                <1>             ;mov    edx, [edx+IRQ.addr] ; Callback service address
                                <1>             ;                           ; (Virtual address)
                                <1>
00012014 8B2D[648E0100]         <1>             mov     ebp, [u.sp]; kernel's stack, points to EIP (user)
0001201A 895500                 <1>             mov     [ebp], edx
                                <1> IRQsrv_5:
                                <1>             ; EOI & return
                                <1>             ; 01/08/2020
                                <1>             ; 11/06/2017
                                <1>             ; 10/06/2017
                                <1>             ;mov    al, [IRQnum]
0001201D B020                   <1>             mov     al, 20h ; 01/08/2020
0001201F FA                     <1>             cli
                                <1>             ;cmp    al, 7
00012020 803D[E0880100]07       <1>             cmp     byte [IRQnum], 7 ; 01/08/2020
00012027 7602                   <1>             jna     short IRQsrv_6
                                <1>             ;
                                <1>             ;;mov   al, EOI ; end of interrupt
                                <1>             ;mov    al, 20h ; 01/08/2020
                                <1>             ;cli            ; disable interrupts till stack cleared
                                <1>             ;out    INTB00, al ; For controll2 #2
00012029 E6A0                   <1>             out     0A0h, al
                                <1> IRQsrv_6:
                                <1>             ;mov    byte [IRQnum], 0 ; reset
                                <1>             ;;mov   al, EOI ; end of interrupt
                                <1>             ;mov    al, 20h ; 01/08/2020
                                <1>             ;cli            ; disable interrupts till stack cleared
                                <1>             ;out    INTA00, al ; end of interrupt to 8259 - 1
0001202B E620                   <1>             out     20h, al
                                <1> IRQsrv_7:
                                <1>             ;; 13/06/2017
                                <1>             ;or     word [ebp+8], 200h ; force enabling interrupts
                                <1>             ;
0001202D 8B0D[DC880100]         <1>             mov     ecx, [IRQ_cr3] ; previous content of cr3 register
00012033 0F22D9                 <1>             mov     cr3, ecx       ; restore cr3 register content
                                <1>             ;
00012036 61                     <1>             popad ; edi,esi,ebp,(icrement esp by 4),ebx,edx,ecx,eax
                                <1>             ;
00012037 0FA9                   <1>             pop     gs
00012039 0FA1                   <1>             pop     fs
0001203B 07                     <1>             pop     es
0001203C 1F                     <1>             pop     ds
                                <1>             ;
0001203D CF                     <1>             iretd   ; return from interrupt
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('get_device_number' procedure is disabled as temporary)
                                <1>
                                <1> ;get_device_number:
                                <1> ;           ; 08/10/2016
                                <1> ;           ; 07/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;           ;
                                <1> ;           ; This procedure compares name of requested
                                <1> ;           ; device with kernel device names and
                                <1> ;           ; installable device names. If names match,
                                <1> ;           ; the relevant device index (entry) number
                                <1> ;           ; will be returned the caller (sysopen)
                                <1> ;           ; for the requested device.
                                <1> ;           ;
                                <1> ;           ; NOTE: Installable device drivers must
                                <1> ;           ; be loaded before using 'sysopen'
                                <1> ;           ; (opendev) system call.
                                <1> ;           ;
                                <1> ;           ; INPUT:
                                <1> ;           ;    ESI = device name address (ASCIIZ)
                                <1> ;           ;         (in kernel's memory space)
                                <1> ;           ;    max name length = 8 without '/dev/')
                                <1> ;           ;    Device name will be capitalized
                                <1> ;           ;    and if there is, '/dev/' will be
                                <1> ;           ;    removed from name before comparising)
                                <1> ;           ;
                                <1> ;           ; OUTPUT:
                                <1> ;           ;    cf = 0 ->
                                <1> ;           ;      EAX (AL) = device entry/index number
                                <1> ;           ;    cf = 1 -> device not found (installed)
                                <1> ;           ;              or invalid device name
                                <1> ;           ;              (AL=0)
                                <1> ;           ;    device_name = device name address (asciiz)
                                <1> ;           ;       ;
                                <1> ;           ; Modified registers: EAX, EBX, ESI, EDI
                                <1> ;
                                <1> ;           mov     edi, device_name
                                <1> ;           call    lodsb_capitalize
```

```
<1> ;           mov     ah, al
<1> ;           cmp     al, '/'
<1> ;           jne     short gdn_1
<1> ;           mov     edi, device_name
<1> ;           call    lodsb_capitalize
<1> ;gdn_0:
<1> ;           and     al, al ; 0 ?
<1> ;           jz      short gdn_err ; null name after '/'
<1> ;gdn_1:
<1> ;           cmp     al, 'D'
<1> ;           jne     short gdn_2
<1> ;           call    lodsb_capitalize
<1> ;           cmp     al, 'E'
<1> ;           jne     short gdn_2
<1> ;           call    lodsb_capitalize
<1> ;           cmp     al, 'V'
<1> ;           jne     short gdn_2
<1> ;           lodsb
<1> ;           cmp     al, '/'
<1> ;           je      short gdn_4
<1> ;gdn_2:
<1> ;           cmp     ah, '/'
<1> ;           jne     short gdn_5
<1> ;gdn_err:
<1> ;           ; invalid device name or device not found
<1> ;           xor     eax, eax ; 0
<1> ;           stc
<1> ;           retn
<1> ;gdn_3:
<1> ;           cmp     al, '/'
<1> ;           jne     short gdn_5
<1> ;gdn_4:
<1> ;           mov     edi, device_name
<1> ;           jmp     short gdn_6
<1> ;gdn_5:
<1> ;           cmp     al, 0
<1> ;           je      short gdn_7
<1> ;gdn_6:
<1> ;           call    lodsb_capitalize
<1> ;           cmp     edi, device_name + 8
<1> ;           jb      short gdn_3
<1> ;           cmp     al, 0
<1> ;           jne     short gdn_err
<1> ;           cmp     edi, device_name + 1
<1> ;           jna     short gdn_err ; null name after '/'
<1> ;gdn_7:
<1> ;           stosb
<1> ;           ; zero padding ("NAME",0,0,0,0)
<1> ;           cmp     edi, device_name + 8
<1> ;           jb      short gdn_7
<1> ;gdn_8:
<1> ;           ; search for kernel device names
<1> ;           mov     esi, device_name
<1> ;           mov     edi, KDEV_NAME
<1> ;           xor     eax, eax
<1> ;gdn_9:
<1> ;           cmpsd
<1> ;           jne     short gdn_10
<1> ;           cmpsd
<1> ;           jne     short gdn_11
<1> ;           jmp     short gdn_17 ; match
<1> ;gdn_10:
<1> ;           cmpsd  ; add esi, 4 & add edi, 4
<1> ;gdn_11:
<1> ;           mov     esi, device_name
<1> ;           inc     al
<1> ;           cmp     al, NumOfKernelDevNames
<1> ;           jb      short gdn_9
<1> ;gdn_12:
<1> ;           ; search for installable device names
<1> ;           ; esi = offset device_name
<1> ;           mov     edi, IDEV_NAME
<1> ;           sub     al, al ; 0
<1> ;gdn_13:
<1> ;           cmpsd
<1> ;           jne     short gdn_14
<1> ;           cmpsd
<1> ;           jne     short gdn_15
<1> ;           jmp     short gdn_19 ; match
<1> ;gdn_14:
<1> ;           cmpsd  ; add esi, 4 & add edi, 4
<1> ;gdn_15:
<1> ;           mov     esi, device_name
<1> ;           inc     al
<1> ;           cmp     al, NumOfInstallableDevices
<1> ;           jb      short gdn_13
<1> ;
<1> ;gdn_16:    ; error: invalid device name (not found) !
<1> ;           xor     al, al
<1> ;           stc
<1> ;           retn
<1> ;
<1> ;gdn_17:    ; name match (with one of kernel device names)
<1> ;           ;
<1> ;           ; convert KDEV_NAME index to
<1> ;           ; KDEV_NUMBER index
<1> ;           ; (different names are used for same devices)
<1> ;           ; (example: "COM1" & "TTY8" = device number 18)
<1> ;           mov     ebx, eax ; < 256
<1> ;           mov     al, [KDEV_NUMBER+ebx]
<1> ;
<1> ;           ; check if empty dev entry in the list
<1> ;           cmp     byte [DEV_OPENMODE+eax], 0
<1> ;           ja      short gdn_18 ; it must be already set
<1> ;
<1> ;           ; (re)set device name and access flags
<1> ;           ; (remain open work will be easy after that)
<1> ;           ; (NOTE: here, data will be copied to bss section)
<1> ;           mov     bl, al
<1> ;           sub     edi, 8 ; kernel device name address (data)
<1> ;           shl     bx, 2
<1> ;           mov     [DEV_NAME_PTR+ebx], edi ; (all) device names
<1> ;           mov     bl, [KDEV_ACCESS+eax] ; kernel dev list (data)
<1> ;           mov     [DEV_ACCESS+eax], bl ; (all) device list (bss)
<1> ;gdn_18:
<1> ;           inc     al ; 1 to NumOfKernelDevNames (<=7Fh)
<1> ;           ; eax = device index/entry number
<1> ;           retn
<1> ;
<1> ;gdn_19:    ; name match (with one of installable device names)
<1> ;           ;
<1> ;           ; al = 0 to NumOfInstallableDevices - 1 (<=7Fh)
<1> ;
<1> ;           mov     ebx, eax
```

```
                                <1> ;           add     bl, NumOfKernelDevices       ; < NUMOFDEVICES
                                <1> ;
                                <1> ;           ; check if empty dev entry in the list
                                <1> ;           cmp     byte [DEV_OPENMODE+ebx], 0
                                <1> ;           ja      short gdn_20 ; it must be already set
                                <1> ;
                                <1> ;           ; (re)set device name and access flags
                                <1> ;           ; (remain open work will be easy after that)
                                <1> ;           sub     edi, 8 ; installable device name address
                                <1> ;           shl     bx, 2 ;*4
                                <1> ;           mov     [DEV_NAME_PTR+ebx], edi ; (all) device names
                                <1> ;           shr     bx, 2
                                <1> ;           mov     al, [IDEV_FLAGS+eax] ; installable dev list
                                <1> ;           mov     [DEV_ACCESS+ebx], al ; (all) device list
                                <1> ;gdn_20:
                                <1> ;           mov     al, bl
                                <1> ;           ; eax = device index/entry number ; < NUMOFDEVICES
                                <1> ;           retn
                                <1>
                                <1> ;lodsb_capitalize:
                                <1> ;   ; 07/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;   ; INPUT -> [esi] = character
                                <1> ;   ;          edi = destination
                                <1> ;   ; OUTPUT -> AL contains capitalized character
                                <1> ;   ;         esi = esi+1
                                <1> ;   ;         edi = edi+1
                                <1> ;   ;
                                <1> ;   lodsb
                                <1> ;   cmp     al, 61h
                                <1> ;           jb      short lodsb_cap_retn
                                <1> ;           cmp     al, 7Ah
                                <1> ;           ja      short lodsb_cap_retn
                                <1> ;           and     al, 0DFh
                                <1> ;lodsb_cap_retn:
                                <1> ;   stosb
                                <1> ;   retn
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('device_open' procedure is disabled as temporary)
                                <1>
                                <1> ;device_open:
                                <1> ;   ; 08/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;   ; Complete device opening work for sysopen (device)
                                <1> ;   ;
                                <1> ;   ; INPUT ->
                                <1> ;   ;       EAX = Device Number (AL)
                                <1> ;   ;        CL = Open mode (1 = read, 2 = write)
                                <1> ;   ;        CH = Device access byte (bit 0 = 0)
                                <1> ;   ; OUTPUT ->
                                <1> ;   ;       EAX = Device Number
                                <1> ;   ;       CF = 0 -> device has been opened
                                <1> ;   ;       CF = 1 -> device could not be opened
                                <1> ;   ;
                                <1> ;   ;  Modified registers: ebx, (edx, ecx, esi, edi, ebp)
                                <1> ;   ;
                                <1> ;
                                <1> ;   mov     ebx, eax
                                <1> ;   shl     bx, 2 ; *4
                                <1> ;
                                <1> ;   test    ch, 80h ; bit 7, installable device driver flag
                                <1> ;   jz      short d_open_2 ; Kernel device
                                <1> ;   ; installable device
                                <1> ;d_open_1:
                                <1> ;           jmp dword [ebx+IDEV_OADDR-4]
                                <1> ;d_open_2:
                                <1> ;   jmp     dword [ebx+KDEV_OADDR-4]
                                <1>
                                <1> ; 17/04/2021
                                <1> ; ('device_close' procedure is disabled as temporary)
                                <1>
                                <1> ;device_close:
                                <1> ;   ; 08/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;   ; Complete device closing work for sysclose (device)
                                <1> ;   ;
                                <1> ;   ; INPUT ->
                                <1> ;   ;       EAX = Device Number (AL)
                                <1> ;   ;        CL = Open mode (1 = read, 2 = write)
                                <1> ;   ;        CH = Device access byte (bit 0 = 0)
                                <1> ;   ; OUTPUT ->
                                <1> ;   ;       EAX = Device Number
                                <1> ;   ;       CF = 0 -> device has been closed
                                <1> ;   ;       CF = 1 -> device could not be closed
                                <1> ;   ;
                                <1> ;   ; Modified registers: ebx, (edx, ecx, esi, edi, ebp)
                                <1> ;   ;
                                <1> ;
                                <1> ;   mov     ebx, eax
                                <1> ;   shl     bx, 2 ; *4
                                <1> ;
                                <1> ;   test    ch, 80h ; bit 7, installable device driver flag
                                <1> ;   jz      short d_close_2 ; Kernel device
                                <1> ;   ; installable device
                                <1> ;d_close_1:
                                <1> ;           jmp dword [ebx+IDEV_CADDR-4]
                                <1> ;d_close_2:
                                <1> ;   jmp     dword [ebx+KDEV_CADDR-4]
                                <1>
                                <1> ;rnull:
                                <1> ;   ; 07/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;   ; read null (read from null device)
                                <1> ;   retn
                                <1>
                                <1> ;wnull:
                                <1> ;   ; 07/10/2016 - TRDOS 386 (TRDOS v2.0)
                                <1> ;   ; write null (write to null device)
                                <1> ;   retn
                                <1>
                                <1> dev_IRQ_service:
                                <1>     ; 12/05/2017
                                <1>     ; 13/04/2017
                                <1>     ; 27/02/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; INPUT ->
                                <1>     ;       AL = IRQ Number (0 to 15)
                                <1>     ;
0001203E 53                     <1>     push    ebx
0001203F 0FB6D8                 <1>     movzx   ebx, al
00012042 C0E302                 <1>     shl     bl, 2 ; * 4
00012045 8B9B[54880100]         <1>     mov     ebx, [ebx+DEV_INT_HNDLR]
0001204B 21DB                   <1>     and     ebx, ebx
0001204D 7404                   <1>         jz  short dIRQ_s_retn
0001204F 50                     <1>     push    eax
                                <1>
00012050 FFD3                   <1>     call    ebx
                                <1>
```

```
00012052 58                <1>     pop     eax
                           <1> dIRQ_s_retn:
00012053 5B                <1>     pop     ebx
00012054 C3                <1>     retn
                           <1>
                           <1> set_dev_IRQ_service:
                           <1>     ; 09/08/2022
                           <1>     ; 29/07/2022 (TRDOS 386 Kernel v2.0.5)
                           <1>     ; 13/04/2017 - TRDOS 386 (TRDOS v2.0)
                           <1>     ;
                           <1>     ; Set Device Interrupt Service
                           <1>     ;
                           <1>     ; INPUT ->
                           <1>     ;       AL = IRQ Number
                           <1>     ;       EBX = Hardware Interrupt Service Address
                           <1>     ;
                           <1>     ; Note: There is not a validation check here
                           <1>     ;       because this procedure is called by
                           <1>     ;       TRDOS 386 kernel !
                           <1>     ;        (Even if a device driver does not exist
                           <1>     ;       this setting may be used by sysaudio
                           <1>     ;       and other system calls for hardware
                           <1>     ;       components which use IRQ method for I/O.)
                           <1>     ;
                           <1>     ;push   esi
00012055 0FB6F0            <1>     movzx   esi, al
                           <1>     ;shl    si, 2 ; * 4
                           <1>     ; 09/08/2022
00012058 C1E602            <1>     shl     esi, 2 ; * 4
0001205B 899E[54880100]    <1>     mov     [esi+DEV_INT_HNDLR], ebx
                           <1>     ;pop    esi
00012061 C3                <1>     retn
                           <1>
                           <1> sysaudio: ; AUDIO FUNCTIONS
                           <1>     ; 28/01/2025
                           <1>     ; 27/01/2025
                           <1>     ; 15/01/2025
                           <1>     ; 12/01/2025
                           <1>     ; 11/01/2025 (TRDOS 386 v2.0.10)
                           <1>     ; 29/12/2024
                           <1>     ; 19/12/2024
                           <1>     ; 23/08/2024 (TRDOS 386 v2.0.9)
                           <1>     ; 05/06/2024
                           <1>     ; 04/06/2024
                           <1>     ; 23/05/2024 (TRDOS 386 v2.0.8)
                           <1>     ; 19/11/2023 (TRDOS 386 v2.0.7)
                           <1>     ; 29/07/2022 (TRDOS 386 v2.0.5)
                           <1>     ; 12/02/2021 (TRDOS 386 v2.0.3)
                           <1>     ; 28/07/2020
                           <1>     ; 27/07/2020
                           <1>     ; 10/10/2017
                           <1>     ; 22/06/2017
                           <1>     ; 28/05/2017, 04/06/2017, 05/06/2017, 10/06/2017
                           <1>     ; 01/05/2017, 12/05/2017, 15/05/2017, 20/05/2017
                           <1>     ; 21/04/2017, 22/04/2017, 23/04/2017, 24/04/2017
                           <1>     ; 10/04/2017, 13/04/2017, 14/04/2017, 16/04/2017
                           <1>     ; 03/04/2017 (VIA VT8237R)
                           <1>     ; 01/04/2016 (trdosk6.s -> tdosk8.s)
                           <1>     ; 16/05/2016 - TRDOS 386 (TRDOS v2.0)
                           <1>     ;
                           <1>     ; Inputs:
                           <1>     ;
                           <1>     ;       BH = 0 -> Beep (PC Speaker)
                           <1>     ;            BL = Duration Counter (1 for 1/64 second)
                           <1>     ;            CX = Frequency Divisor (1193180/Frequency)
                           <1>     ;                (1331 for 886 Hz)
                           <1>     ;
                           <1>     ;       01/04/2017
                           <1>     ;
                           <1>     ;       BH = 1 -> DETECT (& ENABLE) AUDIO DEVICE
                           <1>     ;            BL = 0 : PC SPEAKER
                           <1>     ;                 1 : SOUND BLASTER 16
                           <1>     ;                 2 : INTEL AC'97
                           <1>     ;                 3 : VIA VT8237R (VT8233)
                           <1>     ;                 4 : INTEL HDA
                           <1>     ;             5-FEh : unknown/invalid
                           <1>     ;               ; 04/06/2017
                           <1>     ;               FFh : Get current audio device id
                           <1>     ;
                           <1>     ;       BH = 2 -> ALLOCATE AUDIO BUFFER (for user)
                           <1>     ;               ECX = Audio Buffer Size (must be equal to
                           <1>     ;                     the half of DMA buffer size)
                           <1>     ;               EDX = Virtual Address of the buffer
                           <1>     ;                     (This is not DMA buffer!)
                           <1>     ;
                           <1>     ;       BH = 3 -> INITIALIZE AUDIO DEVICE
                           <1>     ;            BL = 0,2 -> for Signal Response Byte
                           <1>     ;               CL = Signal Response Byte Value (fixed)
                           <1>     ;                            if BL = 0
                           <1>     ;                     auto increment of S.R.B. value
                           <1>     ;                            if BL = 2
                           <1>     ;               EDX = Signal Response (Return) Byte Address
                           <1>     ;
                           <1>     ;            BL = 1 for CallBack Method
                           <1>     ;               EDX = CallBack Service Address (Virtual)
                           <1>     ;
                           <1>     ;            BL > 2 -> invalid function
                           <1>     ;
                           <1>     ;           (Audio buffer must be allocated before
                           <1>     ;            initialization.)
                           <1>     ;
                           <1>     ;       BH = 4 -> START TO PLAY
                           <1>     ;            BL = Mode
                           <1>     ;                 Bit 0 = mono/stereo (1 = stereo)
                           <1>     ;                 Bit 1 = 8 bit / 16 bit (1 = 16 bit)
                           <1>     ;            CX = Sampling Rate (Hz)
                           <1>     ;
                           <1>     ;       BH = 5 -> PAUSE
                           <1>     ;            BL = Any
                           <1>     ;
                           <1>     ;       BH = 6 -> CONTINUE TO PLAY
                           <1>     ;            BL = Any
                           <1>     ;
                           <1>     ;       BH = 7 -> STOP
                           <1>     ;            BL = Any
                           <1>     ;
                           <1>     ;       BH = 8 -> RESET
                           <1>     ;            BL = Any
                           <1>     ;
                           <1>     ;       BH = 9 -> CANCEL (CALLBACK or S.R.B. SERVICE)
                           <1>     ;            BL = Any
                           <1>     ;
                           <1>     ;       BH = 10 -> DEALLOCATE AUDIO BUFFER (for user)
                           <1>     ;            BL = Any
```

```
<1>     ;
<1>     ;       BH = 11 -> SET VOLUME LEVEL
<1>     ;            BL: (Bit 0 to 6)
<1>     ;                0 = Master (Playback, Lineout) volume
<1>     ;                1 = PCM out volume ; 23/05/2024
<1>     ;            CL = Left Channel Volume (0 to 31 max)
<1>     ;            CH = Right Channel Volume (0 to 31 max)
<1>     ;
<1>     ;            Note: If BL >= 80h (Bit 7 of BL is set),
<1>     ;            volume level will be set for next playing
<1>     ;            (actual volume level will not be changed
<1>     ;            immediately)
<1>     ;
<1>     ;       BH = 12 -> DISABLE AUDIO DEVICE
<1>     ;            (reset audio device and unlink dma buffer)
<1>     ;            BL = Any
<1>     ;
<1>     ;       12/05/2017
<1>     ;       BH = 13 -> MAP DMA BUFFER TO USER
<1>     ;           (for direct access to system's dma buffer)
<1>     ;
<1>     ;            ECX = map size in bytes
<1>     ;                (will be rounded up to page borders)
<1>     ;            EDX = Virtual Address of the buffer
<1>     ;                (Will be rounded up to page borders)
<1>     ;
<1>     ;       05/06/2017
<1>     ;       04/06/2017
<1>     ;       BH = 14 -> GET AUDIO DEVICE INFO
<1>     ;            BL: 0 = Audio Controller Info
<1>     ;            ; 19/11/2023
<1>     ;            BL: 1 = Audio (AC'97) Codec Info
<1>     ;            BL > 1 = Invalid for now!
<1>     ;
<1>     ;       22/06/2017
<1>     ;       BH = 15 -> GET CURRENT SOUND DATA (for graphics)
<1>     ;            BL: 0 -> PCM OUT data
<1>     ;              > 0 -> Invalid for now!
<1>     ;            ECX = 0 -> Get DMA Buffer Pointer
<1>     ;                EDX = Not Used
<1>     ;            ECX > 0 -> Byte count for buffer (EDX)
<1>     ;                EDX = Buffer Address (Virtual)
<1>     ;
<1>     ;       10/10/2017
<1>     ;       BH = 16 -> UPDATE DMA BUFFER DATA
<1>     ;                  (by using the Audio Buffer content)
<1>     ;            BL = 0 : Update dma half buffer in sequence
<1>     ;                     (automatic destination)
<1>     ;                 1 : Update 1st half of the dma buffer
<1>     ;                 2 : Update 2nd half of the dma buffer
<1>     ;                 3-FEh: Invalid!
<1>     ;                 FFh = Get current flag value
<1>     ;                       (Half buffer number -1)
<1>     ;
<1>     ;       24/05/2024
<1>     ;       BH = 17 -> GET VOLUME LEVEL
<1>     ;            BL:  0 = Master (Playback, Lineout) volume
<1>     ;                 1 = PCM out volume
<1>     ;
<1>     ; Outputs:
<1>     ;
<1>     ;       For BH = 0 -> Beep
<1>     ;           None
<1>     ;
<1>     ;       01/04/2017
<1>     ;
<1>     ;       For BH = 1 -> DETECT (& ENABLE) AUDIO DEVICE
<1>     ;           AH = 0 : PC SPEAKER
<1>     ;                1 : SOUND BLASTER 16
<1>     ;                2 : INTEL AC'97
<1>     ;                3 : VIA VT8237R (VT8233)
<1>     ;                4 : INTEL HDA
<1>     ;              5-FFh : unknown/invalid
<1>     ;           AL = mode status
<1>     ;                bit 0 = mono /stereo (1 = stereo)
<1>     ;                bit 1 = 8 bit / 16 bit (1 = 16 bit)
<1>     ;           04/06/2017
<1>     ;           EBX = PCI DEVICE/VENDOR ID (if >0)
<1>     ;                (BX = VENDOR ID)
<1>     ;           (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 2 -> ALLOCATE AUDIO BUFFER (for user)
<1>     ;           EAX = Physical Address of the buffer
<1>     ;           (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 3 -> INITIALIZE AUDIO DEVICE
<1>     ;           (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 4 -> START TO PLAY
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 5 -> PAUSE
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 6 -> CONTINUE TO PLAY
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 7 -> STOP
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 8 -> RESET
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 9 -> CANCEL (CALLBACK or S.R.B. SERVICE)
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 10 -> DEALLOCATE AUDIO BUFFER (for user)
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 11 -> SET VOLUME LEVEL
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       For BH = 12 -> DISABLE AUDIO DEVICE
<1>     ;           none (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       12/05/2017
<1>     ;       For BH = 13 -> MAP DMA BUFFER TO USER
<1>     ;           EAX = Physical Address of the buffer
<1>     ;           (if CF = 1 -> Error code in EAX)
<1>     ;
<1>     ;       04/06/2017
<1>     ;       For BH = 14 -> GET AUDIO DEVICE INFO
<1>     ;       (for BL = 0) ; 05/06/2017
<1>     ;           EAX = IRQ Number in AL
```

```
                                <1>     ;               Audio Device Number in AH
                                <1>     ;           EBX = DEV/VENDOR ID
                                <1>     ;               (DDDDDDDDDDDDDDDDVVVVVVVVVVVVVVVV)
                                <1>     ;           ECX = BUS/DEV/FN
                                <1>     ;               (00000000BBBBBBBBDDDDDFFF00000000)
                                <1>     ;           EDX = NABMBAR/NAMBAR (for AC97)
                                <1>     ;                (Low word, DX = NAMBAR address)
                                <1>     ;           EDX = Base IO Addr (DX) for SB16 & VT8233
                                <1>     ;           (if CF = 1 -> Error code in EAX)
                                <1>     ;                        (ERR_DEV_NOT_RDY = 15)
                                <1>     ;       (for BL = 1) -for AC97- ; 19/11/2023
                                <1>     ;           EAX = Extended Audio ID (MX28) in AX
                                <1>     ;            AX bit 0 - VRA bit
                                <1>     ;            HW of EAX = PCM output sample rate (HZ)
                                <1>     ;           EBX = VENDOR ID 1 (BX), VENDOR ID 2 (HW)
                                <1>     ;           (if CF = 1 -> Error code in EAX)
                                <1>     ;                        (ERR_DEV_NOT_RDY = 15)
                                <1>     ;           Note: EAX & EBX = 0 (for SB16,VIA,HDA)
                                <1>     ;
                                <1>     ;       22/06/2017
                                <1>     ;       For BH = 15 -> GET CURRENT SOUND DATA
                                <1>     ;                       (for graphics)
                                <1>     ;       (for BL = 0)
                                <1>     ;        If ECX input is 0
                                <1>     ;           EAX = DMA Buffer Current Position (Offset)
                                <1>     ;        If ECX input >  0
                                <1>     ;           EAX = Actual transfer count
                                <1>     ;           (Sound samples will be copied from
                                <1>     ;            Current DMA Buffer Position to EDX
                                <1>     ;            virtual address as EAX bytes.)
                                <1>     ;        ((If CF = 1 ->  Error code in EAX))
                                <1>     ;
                                <1>     ;       10/10/2017
                                <1>     ;       For BH = 16 -> UPDATE DMA BUFFER DATA
                                <1>     ;           EAX = 0, if the updated (or current)
                                <1>     ;                    half buffer is DMA half buffer 1
                                <1>     ;           EAX = 1, if the updated (or current)
                                <1>     ;                    half buffer is DMA half buffer 2
                                <1>     ;           (If CF = 1 -> Error code in EAX)
                                <1>     ;
                                <1>     ;       24/05/2024
                                <1>     ;       For BH = 17 -> GET VOLUME LEVEL
                                <1>     ;           IF BL input is 0
                                <1>     ;               Master (Playback, Lineout) volume
                                <1>     ;           If BL input is 1
                                <1>     ;               PCM out volume
                                <1>     ;           If BL input > 1
                                <1>     ;               Invalid for now ; 24/05/2024
                                <1>     ;
                                <1>     ;           CL = Left Channel Volume (0 to 31 max)
                                <1>     ;           CH = Right Channel Volume (0 to 31 max)
                                <1>     ;
                                <1>
00012062 80FF12                 <1>     cmp     bh, AUDIO1L/4
00012065 0F8323AAFFFF           <1>     jnb     sysret
                                <1>
0001206B C0E702                 <1>     shl     bh, 2 ; *4
0001206E 0FB6F7                 <1>     movzx   esi, bh
                                <1>
                                <1>     ; 22/04/2017
00012071 31C0                   <1>     xor     eax, eax
00012073 A3[6C8E0100]           <1>     mov     [u.r0], eax  ; 0
                                <1>
00012078 FF96[83200100]         <1>     call    dword [esi+AUDIO1]
                                <1>     ;jc     error
0001207E E90BAAFFFF             <1>     jmp     sysret
                                <1>
00012083 [C1230000]             <1> AUDIO1:     dd      _beep ; 12/02/2021
                                <1>     ;dd     beep ; FUNCTION = 0 (bl = Duration Counter
                                <1>                  ;               cx = Frequency Divisor
00012087 [CB200100]             <1>     dd      soundc_detect
0001208B [60210100]             <1>     dd      sound_alloc
0001208F [23220100]             <1>     dd      soundc_init
00012093 [07240100]             <1>     dd      sound_play
00012097 [94240100]             <1>     dd      sound_pause
0001209B [BE240100]             <1>     dd      sound_continue
0001209F [E8240100]             <1>     dd      sound_stop
000120A3 [13250100]             <1>     dd      soundc_reset
000120A7 [46250100]             <1>     dd      soundc_cancel
000120AB [6C250100]             <1>     dd      sound_dalloc
000120AF [97250100]             <1>     dd      sound_volume
000120B3 [D7250100]             <1>     dd      soundc_disable
000120B7 [55260100]             <1>     dd      sound_dma_map
000120BB [C3260100]             <1>     dd      soundc_info
000120BF [47270100]             <1>     dd      sound_data
000120C3 [E9270100]             <1>     dd      sound_update
000120C7 [6B280100]             <1>     dd      sound_getvol ; 24/05/2024
                                <1> %if 0
                                <1>     dd      sound_dmaclear ; 29/12/2024
                                <1> %endif
                                <1>
                                <1> AUDIO1L     EQU     $ - AUDIO1
                                <1>
                                <1> soundc_detect:
                                <1>     ; FUNCTION = 1
                                <1>     ; bl = Audio device type number
                                <1>     ; (0 = pc speaker, 1 = sound blaster 16, 2 = intel ac97
                                <1>     ;  3 = via vt823x, 4 = intel HDA, 0FFh = any)
                                <1>
                                <1>     ; 04/06/2017
                                <1>     ;mov    ah, [audio_device]
                                <1>     ; 11/01/2025
000120CB A0[E5880100]           <1>     mov     al, [audio_device]
000120D0 80FBFF                 <1>     cmp     bl, 0FFh ; get current audio device id
000120D3 7408                   <1>     je      short sysaudio0
                                <1>
                                <1>     ;and    ah, ah
                                <1>     ; 11/01/2025
000120D5 20C0                   <1>     and     al, al
000120D7 741F                   <1>     jz      short soundc_get_dev
                                <1>
                                <1>     ;cmp    ah, bl
                                <1>     ; 11/01/2025
000120D9 38D8                   <1>     cmp     al, bl
000120DB 7568                   <1>     jne     short soundc_dev_err
                                <1>
                                <1> sysaudio0:
                                <1>     ;mov    al, [audio_mode]
                                <1>     ; 11/01/2025
000120DD 8A25[1D890100]         <1>     mov     ah, [audio_mode]
                                <1> sysaudio1:
000120E3 A3[6C8E0100]           <1>     mov     [u.r0], eax
000120E8 8B1D[F0880100]         <1>     mov     ebx, [audio_vendor] ; (DEVICE/VENDOR ID)
000120EE 8B2D[688E0100]         <1>     mov     ebp, [u.usp]
```

```
000120F4 895D10              <1>     mov     [ebp+16], ebx  ; ebx
000120F7 C3                  <1>     retn
                             <1>
                             <1> soundc_get_dev:
                             <1>     ; 28/05/2017
                             <1>     ; 03/04/2017, 24/04/2017
000120F8 C605[E4880100]00    <1>     mov     byte [audio_pci], 0
000120FF 80FB03              <1>     cmp     bl, 3 ; VIA VT8233 (VT8237R) Audio Controller & AC97 Codec
                             <1>     ;jne    short soundc_get_dev_sb
                             <1>     ; 28/05/2017
00012102 7220                <1>     jb      short soundc_get_dev_sb
00012104 773F                <1>     ja      short soundc_dev_err  ; temporary (28/05/2017)
                             <1>     ;
00012106 E892180000          <1>     call    DetectVT8233
0001210B 7238                <1>     jc      short soundc_dev_err
                             <1>     ; eax = 0
                             <1>
                             <1>     ;mov    ebx, [audio_vendor]
                             <1>     ; ebx = DEVICE/VENDOR ID
                             <1>     ;       DDDDDDDDDDDDDDDDVVVVVVVVVVVVVVVV
                             <1>
0001210D B003                <1>     mov     al, 3  ; VIA VT8237R (VT3233) Audio Controller
0001210F 88C4                <1>     mov     ah, al
                             <1>
                             <1> soundc_get_pci_dev_ok: ; 28/05/2017
00012111 FE05[E4880100]      <1>     inc     byte [audio_pci] ; = 1
                             <1> soundc_get_dev_ok:
                             <1>
                             <1> soundc_get_dev_sb16_ok:
00012117 A2[E5880100]        <1>     mov     [audio_device], al
0001211C 8825[1D890100]      <1>     mov     [audio_mode], ah ; stereo (bit0), 16 bit (bit1) capability
00012122 EBBF                <1>     jmp     short sysaudio1
                             <1>
                             <1> soundc_get_dev_sb:
                             <1>     ; 24/04/2017
00012124 80FB01              <1>     cmp     bl, 1 ; Sound Blaster 16
00012127 750E                <1>     jne     short soundc_get_dev_ich ; 28/05/2017
                             <1>     ;
00012129 E8431D0000          <1>     call    DetectSB
0001212E 7215                <1>     jc      short soundc_dev_err
00012130 B801030000          <1>     mov     eax, 0301h ; Sound Blaster 16
00012135 EBE0                <1>     jmp     short soundc_get_dev_sb16_ok
                             <1>
                             <1> soundc_get_dev_ich:
                             <1>     ; 28/05/2017
                             <1>     ;cmp    bl, 2 ; Intel AC'97 Audio Controller (ICH)
                             <1>     ;jne    short soundc_dev_err ; Temporary  (28/05/2017)
                             <1>     ;                            ; (Here will be modified just after
                             <1>     ;                            ; new sound card code will be ready!)
00012137 E84C180000          <1>     call    DetectICH
0001213C 7207                <1>     jc      short soundc_dev_err
                             <1>     ;
0001213E B802030000          <1>     mov     eax, 0302h ; AC'97 (ICH)
00012143 EBCC                <1>     jmp     short soundc_get_pci_dev_ok
                             <1>
                             <1> soundc_dev_err:
00012145 B80F000000          <1>     mov     eax, ERR_DEV_NOT_RDY ; Device not ready !
                             <1>     ; 29/07/2022
                             <1>     ;sub    eax, eax
                             <1>     ;mov    al, ERR_DEV_NOT_RDY
0001214A EB05                <1>     jmp     short sysaudio_err
                             <1>
                             <1> soundc_respond_err:
                             <1>     ; ERR_TIME_OUT ; 'time out !' error
0001214C B819000000          <1>     mov     eax, ERR_DEV_NOT_RESP ; 'device not responding !' error
                             <1>     ; 29/07/2022
                             <1>     ;sub    eax, eax
                             <1>     ;mov    al, ERR_DEV_NOT_RESP
                             <1> sysaudio_err:
00012151 A3[6C8E0100]        <1>     mov     [u.r0], eax
00012156 A3[D88E0100]        <1>     mov     [u.error], eax
0001215B E90EA9FFFF          <1>     jmp     error
                             <1>
                             <1> sound_alloc:
                             <1>     ; FUNCTION = 2
                             <1>     ; ecx = audio buffer size (in bytes)
                             <1>     ; edx = audio buffer address (virtual)
                             <1>     ; 28/01/2025
                             <1>     ; 27/01/2025 (BugFix)
                             <1>     ; 15/01/2025
                             <1>     ; 12/01/2025
                             <1>     ; 25/11/2023
                             <1>     ; 27/07/2020
                             <1>     ; 28/05/2027
                             <1>     ; 01/05/2017, 15/05/2017
                             <1>     ; 21/04/2017, 24/04/2017
00012160 803D[E4880100]00    <1>     cmp     byte [audio_pci], 0
00012167 770F                <1>     ja      short snd_alloc_0
                             <1>     ; Max. 64KB DMA buffer !!!
00012169 81F900800000        <1>     cmp     ecx, 32768
0001216F 760F                <1>     jna     short snd_alloc_6
                             <1>     ; 25/11/2023
                             <1> sound_buff_error:
00012171 B82E000000          <1>     mov     eax, ERR_BUFFER ; Buffer error !
                             <1>     ; 29/07/2022
                             <1>     ;sub    eax, eax
                             <1>     ;mov    al, ERR_BUFFER
00012176 EBD9                <1>     jmp     short sysaudio_err
                             <1> snd_alloc_0:
                             <1>     ; 25/11/2023
                             <1>     ; Max 128KB DMA buffer size (2 half buffers)
00012178 81F900000100        <1>     cmp     ecx, 65536
0001217E 77F1                <1>     ja      short sound_buff_error
                             <1> snd_alloc_6:
                             <1>     ; 15/05/2017
                             <1>     ;cmp    ecx, 4096 ; PAGE_SIZE
                             <1>     ;jb     short sound_buff_error
                             <1>     ; 15/01/2025
00012180 81F940010000        <1>     cmp     ecx, 320
00012186 72E9                <1>     jb      short sound_buff_error
                             <1>     ;
                             <1>     ; 28/01/2025
                             <1>     ; 12/01/2025
                             <1>     ;mov    ebx, edx ; (new) virtual address of audio_buffer
                             <1>     ;
00012188 A1[F8880100]        <1>     mov     eax, [audio_buffer] ; audio buffer address (current)
0001218D 09C0                <1>     or      eax, eax
                             <1>     ;jz     short snd_alloc_2
                             <1>     ; 12/01/2025
0001218F 743E                <1>     jz      short snd_alloc_1 ; first time
                             <1>     ; audio buffer exists !
00012191 8A1D[BD8E0100]      <1>     mov     bl, [u.uno]
00012197 3A1D[11890100]      <1>     cmp     bl, [audio_user]
                             <1>     ;jne    sndc_owner_error ; not owner !
```

```
                                <1>     ; 25/11/2023
                                <1>     ;je    short snd_alloc_7
                                <1>     ; 12/01/2025
0001219D 7405                   <1>     je     short snd_alloc_2
0001219F E9F3000000             <1>     jmp    sndc_owner_error ; not owner !
                                <1>
                                <1>     ; 12/01/2025 - TRDOS 386 v2.0.10
                                <1>     ; Important modification:
                                <1>     ;; Always deallocate user's audio_buffer address with
                                <1>     ;; buffer size and then allocate a new one
                                <1>     ;; (address range may be used before)
                                <1> snd_alloc_2:
                                <1>     ;mov   ecx, [audio_buff_size]
                                <1>     ; 28/01/2025
000121A4 870D[00890100]         <1>     xchg   ecx, [audio_buff_size] ; *
000121AA 89C3                   <1>     mov    ebx, eax ; audio buffer address (current)
                                <1> ;snd_alloc_7:
                                <1>     ; 12/01/2025
                                <1>     ;cmp   eax, edx ; same virtual buffer address ?
                                <1>     ;jne   short snd_alloc_1
                                <1>     ;cmp   ecx, [audio_buff_size]
                                <1>     ;je    short snd_alloc_3 ; Nothing to do !
                                <1>     ;                        ; Buffer has been set already!
                                <1> ;snd_alloc_1:
                                <1>     ; 28/01/2025
                                <1>     ;push  ecx
000121AC 52                     <1>     push   edx
                                <1>     ; 12/01/2025
                                <1>     ;mov   ebx, eax ; audio buffer address (current)
                                <1>     ;mov   ecx, [audio_buff_size]
                                <1>     ; ebx = audio buffer address (virtual)
                                <1>     ; ecx = audio buffer size in bytes
000121AD E8EB3EFFFF             <1>     call   deallocate_user_pages
000121B2 5A                     <1>     pop    edx
                                <1>     ;pop   ecx
                                <1>     ; 28/01/2025
000121B3 8B0D[00890100]         <1>     mov    ecx, [audio_buff_size] ; * ; requested size
000121B9 31C0                   <1>     xor    eax, eax ; 0
000121BB A3[F8880100]           <1>     mov    [audio_buffer], eax  ; 0
000121C0 A3[FC880100]           <1>     mov    [audio_p_buffer], eax  ; 0
000121C5 A3[00890100]           <1>     mov    [audio_buff_size], eax
000121CA A2[11890100]           <1>     mov    [audio_user], al ; 0
                                <1>
                                <1>     ; 27/01/2025 (BugFix)
                                <1> snd_alloc_1:
                                <1>     ; 12/01/2025
                                <1> ;snd_alloc_2:
000121CF 89D3                   <1>     mov    ebx, edx
                                <1>     ; 01/05/2017
                                <1>     ;mov   edx, ~PAGE_OFF ; truncating page offsets
                                <1>     ;                     ; for aligning to page borders
                                <1>     ;;and  eax, edx
                                <1>     ;and   ebx, edx
                                <1>     ; 26/11/2023
                                <1>     ;and   ecx, edx
000121D1 81E300F0FFFF           <1>     and    ebx, ~PAGE_OFF
000121D7 89CA                   <1>     mov    edx, ecx
                                <1>     ; 15/05/2017
                                <1>     ; EAX = Beginning address (physical)
                                <1>     ; EAX = 0 -> Allocate mem block from the 1st proper aperture
                                <1>     ; ECX = Number of bytes to be allocated
000121D9 E8673BFFFF             <1>     call   allocate_memory_block
000121DE 7291                   <1>     jc     short sound_buff_error
                                <1>
                                <1>     ; EAX = Physical address of the allocated memory block
                                <1>     ; ECX = Allocated bytes (as rounded up to page border)
                                <1>     ; EBX = Virtual address (as truncated to page border)
000121E0 52                     <1>     push   edx ; 26/11/2023
000121E1 50                     <1>     push   eax
000121E2 53                     <1>     push   ebx
000121E3 51                     <1>     push   ecx
000121E4 E89C3FFFFF             <1>     call   allocate_user_pages
000121E9 59                     <1>     pop    ecx
000121EA 5B                     <1>     pop    ebx
000121EB 58                     <1>     pop    eax
000121EC 5A                     <1>     pop    edx ; 26/11/2023
000121ED 722A                   <1>     jc     short snd_alloc_4 ; insufficient memory, buff error
                                <1>     ; eax = physical address of the user's audio buffer
                                <1>     ; ebx = virtual address of the user's audio buffer
                                <1>     ; 26/11/2023
                                <1>     ; ecx = allocated buffer size (in bytes)
                                <1>     ;               -rounded up to page boundary-
                                <1>     ; edx = requested buffer size (in bytes)
000121EF A3[FC880100]           <1>     mov    [audio_p_buffer], eax
000121F4 891D[F8880100]         <1>     mov    [audio_buffer], ebx
                                <1>     ; 26/11/2023
                                <1>     ;mov   ecx, edx
                                <1>     ;; 25/11/2023
                                <1>     ;cmp   ecx, 65536
                                <1>     ;jb    short snd_alloc_5
                                <1>     ;dec   ecx
                                <1>     ;; ecx = 65535
                                <1>     ;; (DMA half buffer's sample count must be < 65536)
                                <1> ;snd_alloc_5:
                                <1>     ;mov   [audio_buff_size], ecx
                                <1>     ; 26/11/2023
000121FA 8915[00890100]         <1>     mov    [audio_buff_size], edx
                                <1>     ;
00012200 8A15[BD8E0100]         <1>     mov    dl, [u.uno]
00012206 8815[11890100]         <1>     mov    [audio_user], dl
0001220C A3[6C8E0100]           <1>     mov    [u.r0], eax
                                <1> snd_alloc_3:
                                <1>     ; 27/07/2020
00012211 C605[10890100]00       <1>     mov    byte [audio_flag], 0 ; clear dma half buffer flag
                                <1>     ;
00012218 C3                     <1>     retn
                                <1> snd_alloc_4:
                                <1>     ; 15/05/2017
                                <1>     ; EAX = Beginning address (physical)
                                <1>     ; ECX = Number of bytes to be deallocated
00012219 E8413DFFFF             <1>     call   deallocate_memory_block
0001221E E94EFFFFFF             <1>     jmp    sound_buff_error ; insufficient memory, buff error
                                <1>
                                <1> soundc_init:
                                <1>     ; FUNCTION = 3
                                <1>     ; bl = method (0= s.r.b., 1= callback, 2= auto incr s.r.b.)
                                <1>     ; cl = signal response byte (initial or fixed) value
                                <1>     ; edx = signal response byte or callback address
                                <1>
                                <1>     ; 11/01/2025
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024
                                <1>     ; 02/12/2023, 04/12/2022
                                <1>     ; 29/07/2022, 07/08/2022
```

```
                                <1>     ; 27/07/2020
                                <1>     ; 12/05/2017, 20/05/2017, 28/05/2027
                                <1>     ; 22/04/2017, 23/04/2017, 24/04/2017
                                <1>     ; 13/04/2017, 14/04/2017, 16/04/2017, 21/04/2017
                                <1>     ; 03/04/2017, 10/04/2017
                                <1>
00012223 A0[E5880100]           <1>     mov     al, [audio_device]
00012228 20C0                   <1>     and     al, al
0001222A 754A                   <1>     jnz     short sndc_init6
                                <1>     ;
0001222C C605[E4880100]00       <1>     mov     byte [audio_pci], 0
00012233 52                     <1>     push    edx
00012234 53                     <1>     push    ebx
00012235 51                     <1>     push    ecx
00012236 E8361C0000             <1>     call    DetectSB
0001223B 7213                   <1>     jc      short sndc_init8
0001223D 66B80103               <1>     mov     ax, 0301h ; Sound Blaster 16
00012241 EB1E                   <1>     jmp     short sndc_init7
                                <1>
                                <1> sndc_init11:
                                <1>     ; 28/05/2017
00012243 E840170000             <1>     call    DetectICH ; Detect AC'97 (ICH) Audio Controller
00012248 7217                   <1>     jc      short sndc_init7
0001224A 66B80203               <1>     mov     ax, 0302h ; Intel AC'97 Audio Device
0001224E EB0B                   <1>     jmp     short sndc_init12 ; (PCI device)
                                <1>
                                <1> sndc_init8:
00012250 E848170000             <1>     call    DetectVT8233
                                <1>     ;jc     short sndc_init7
                                <1>     ; 29/07/2022
00012255 72EC                   <1>     jc      short sndc_init11 ; 28/05/2017
                                <1>
                                <1>     ; eax = 0
00012257 B003                   <1>     mov     al, 3   ; VIA VT8237R (VT8233) Audio Controller
00012259 88C4                   <1>     mov     ah, al
                                <1>
                                <1> sndc_init12:
0001225B FE05[E4880100]         <1>     inc     byte [audio_pci] ; = 1
                                <1> sndc_init7:
00012261 59                     <1>     pop     ecx
00012262 5B                     <1>     pop     ebx
00012263 5A                     <1>     pop     edx
                                <1>     ;jc     soundc_dev_err
                                <1>     ; 29/07/2022
00012264 7305                   <1>     jnc     short sndc_init14
00012266 E9DAFEFFFF             <1>     jmp     soundc_dev_err
                                <1> sndc_init14:
0001226B A2[E5880100]           <1>     mov     [audio_device], al
00012270 8825[1D890100]         <1>     mov     [audio_mode], ah ; stereo (bit0), 16 bit (bit1) capability
                                <1>
                                <1> sndc_init6:
00012276 833D[F8880100]00       <1>     cmp     dword [audio_buffer], 0
                                <1>     ;jna    sound_buff_error
                                <1>     ; 29/07/2022
0001227D 7705                   <1>     ja      short sndc_init19
0001227F E9EDFEFFFF             <1>     jmp     sound_buff_error ; 07/08/2022
                                <1> sndc_init19:
00012284 A0[BD8E0100]           <1>     mov     al, [u.uno]
00012289 8A25[11890100]         <1>     mov     ah, [audio_user]
0001228F 08E4                   <1>     or      ah, ah
00012291 7418                   <1>     jz      short sndc_init0
00012293 38E0                   <1>     cmp     al, ah
00012295 7419                   <1>     je      short sndc_init1
                                <1>
                                <1> sndc_owner_error:
00012297 B80B000000             <1>     mov     eax, ERR_NOT_OWNER ; 'permission denied !' error
                                <1>     ; 29/07/2022
                                <1>     ;sub    eax, eax
                                <1>     ;mov    al, ERR_NOT_OWNER
                                <1> sndc_perm_error:
0001229C A3[6C8E0100]           <1>     mov     [u.r0], eax
000122A1 A3[D88E0100]           <1>     mov     [u.error], eax
000122A6 E9C3A7FFFF             <1>     jmp     error
                                <1> sndc_init0:
000122AB A2[11890100]           <1>     mov     [audio_user], al
                                <1> sndc_init1:
000122B0 8915[14890100]         <1>     mov     [audio_cb_addr], edx
000122B6 881D[12890100]         <1>     mov     [audio_cb_mode], bl
000122BC 880D[13890100]         <1>     mov     [audio_srb], cl
                                <1>
                                <1>     ; 27/07/2020
                                <1>     ;mov    byte [audio_flag], 0  ; clear dma half buffer flag
                                <1>
                                <1>     ; 24/04/2017
000122C2 803D[E5880100]03       <1>     cmp     byte [audio_device], 3 ; VT8233 (VT8237R)
000122C9 7435                   <1>     je      short sndc_init9
                                <1>     ;ja     short soundc_respond_err ; temporary (28/05/2017)
000122CB 803D[E5880100]01       <1>     cmp     byte [audio_device], 1 ; SB 16
000122D2 7510                   <1>     jne     short sndc_init13
000122D4 BB[83400100]           <1>     mov     ebx, sb16_int_handler
                                <1>     ; Note: 'SbInit' is at 'Start to Play' stage
                                <1>     ; 20/05/2017
000122D9 66C705[20890100]08- <1>     mov     word [audio_master_volume], 0808h ; 2/8
000122E1 08                     <1>
000122E2 EB34                   <1>     jmp     short sndc_init10
                                <1> sndc_init13:
                                <1>     ; 28/05/2017
000122E4 803D[E5880100]02       <1>     cmp     byte [audio_device], 2 ; AC 97 (ICH)
                                <1>     ;jne    soundc_respond_err ; temporary (28/05/2017)
                                <1>     ; 29/07/2022
000122EB 7405                   <1>     je      short sndc_init16
                                <1> sndc_init15:
000122ED E95AFEFFFF             <1>     jmp     soundc_respond_err
                                <1> sndc_init16:
000122F2 E8D41E0000             <1>     call    ac97_codec_config
                                <1>     ;jc     soundc_respond_err ; codec error !
                                <1>     ; 29/07/2022
000122F7 72F4                   <1>     jc      short sndc_init15
                                <1>
000122F9 BB[65440100]           <1>     mov     ebx, ac97_int_handler
000122FE EB18                   <1>     jmp     short sndc_init10
                                <1>
                                <1> sndc_init9:
                                <1>     ;call   reset_codec
                                <1>     ;; eax = 1
                                <1>     ;call   codec_io_w16 ; w32
00012300 E8EC170000             <1>     call    init_codec ; 28/05/2017
                                <1>     ;jc     soundc_respond_err ; codec error !
                                <1>     ; 29/07/2022
00012305 72E6                   <1>     jc      short sndc_init15
                                <1>
00012307 E8FA190000             <1>     call    channel_reset
                                <1>
                                <1>     ; setup the Codec (actually mixer registers)
```

```
0001230C E823190000          <1>         call    codec_config  ; unmute codec, set rates.
                             <1>     ;jc    soundc_respond_err ; codec error !
                             <1>     ; 29/07/2022
00012311 72DA                <1>     jc     short sndc_init15
                             <1>
00012313 BB[9A3C0100]        <1>     mov    ebx, vt8233_int_handler
                             <1> sndc_init10:
                             <1>     ; 13/04/2017
00012318 A0[E6880100]        <1>     mov    al, [audio_intr] ; IRQ number
0001231D E833FDFFFF          <1>     call   set_dev_IRQ_service
                             <1>
                             <1>     ; SETUP (audio) INTERRUPT CALLBACK SERVICE
00012322 8A1D[E6880100]      <1>     mov    bl, [audio_intr] ; IRQ number
00012328 8A3D[12890100]      <1>     mov    bh, [audio_cb_mode]
0001232E FEC7                <1>     inc    bh  ; 1 = Signal Response Byte method (fixed value)
                             <1>                ; 2 = Callback service method
                             <1>                ; 3 = Auto Increment S.R.B. method
00012330 8A0D[13890100]      <1>     mov    cl, [audio_srb]
00012336 8B15[14890100]      <1>     mov    edx, [audio_cb_addr]
0001233C A0[11890100]        <1>     mov    al, [audio_user]
                             <1>     ; 14/04/2017
00012341 E83C050000          <1>     call   set_irq_callback_service
                             <1>     ; 16/04/2017
00012346 A3[6C8E0100]        <1>     mov    [u.r0], eax
                             <1>     ;jnc   sysret
0001234B 7316                <1>     jnc    short sndc_init2 ; 21/04/2017
                             <1>     ;
0001234D A3[D88E0100]        <1>     mov    dword [u.error], eax
                             <1>
00012352 A0[E6880100]        <1>     mov    al, [audio_intr] ; IRQ number
00012357 31DB                <1>     xor    ebx, ebx ; reset IRQ handler address
00012359 E8F7FCFFFF          <1>     call   set_dev_IRQ_service
                             <1>
0001235E E90BA7FFFF          <1>     jmp    error
                             <1>
                             <1> sndc_init2:
                             <1>     ; 21/04/2017
00012363 8B0D[00890100]      <1>     mov    ecx, [audio_buff_size] ; audio buffer size
                             <1>     ; 05/06/2024
                             <1>     ;shl   ecx, 1 ; *2
                             <1>
                             <1>     ; 05/06/2024
00012369 A0[E5880100]        <1>     mov    al, [audio_device]
0001236E 3C01                <1>     cmp    al, 1 ; SB16
00012370 760A                <1>     jna    short sndc_init22
00012372 80E1FE              <1>     and    cl, ~1 ; truncated for word alignment
00012375 3C03                <1>     cmp    al, 3 ; VT8233
00012377 7303                <1>     jnb    short sndc_init22
                             <1>     ; al = 2 ; AC'97
00012379 80E1F8              <1>     and    cl, ~7 ; truncated for 8 byte (8x) alignment
                             <1> sndc_init22:
                             <1>     ; 05/06/2024
0001237C 890D[0C890100]      <1>     mov    [dma_hbuff_size], ecx  ; DMA half buffer size
00012382 D1E1                <1>     shl    ecx, 1  ; add ecx, ecx ; * 2
                             <1>
                             <1>     ;;;
                             <1>     ; 04/06/2024
00012384 81C1FF0F0000        <1>     add    ecx, 4095  ; PAGE_SIZE - 1
0001238A 6681E100F0          <1>     and    cx, ~4095  ; ~PAGE_OFF
                             <1>     ; ecx = page border aligned DMA buffer size (required) (*)
                             <1>     ;;;
                             <1>
0001238F A1[04890100]        <1>     mov    eax, [audio_dma_buff]
00012394 21C0                <1>     and    eax, eax
00012396 741C                <1>     jz     short sndc_init3 ; no need to compare dma buff size
                             <1>
00012398 8B15[08890100]      <1>     mov    edx, [audio_dmabuff_size] ; dma buffer size
0001239E 39D1                <1>     cmp    ecx, edx
000123A0 742B                <1>     je     short sndc_init5
                             <1>
                             <1>     ; 04/12/2023
000123A2 3D00000500          <1>     cmp    eax, sb16_dma_buffer  ; reserved buffer ?
000123A7 7409                <1>     je     short sndc_init20 ; it isn't an allocated mem buff
                             <1>
000123A9 87CA                <1>     xchg   ecx, edx
                             <1>     ; 26/11/2023
                             <1>     ; round up (always -rounded up- page count is allocated)
                             <1>     ; ((so deallocation must be done for the rounded up value))
                             <1>     ;add   ecx, PAGE_SIZE - 1   ; 4095
                             <1>     ;call  deallocate_memory_block
                             <1>     ; 04/06/2024
                             <1>     ;call  deallocate_memory_block_x
                             <1>     ;              ; deallocate ((ecx+4095)>>12) pages
000123AB E8AF3BFFFF          <1>     call   deallocate_memory_block
                             <1>
000123B0 87D1                <1>     xchg   edx, ecx
                             <1> sndc_init20:
000123B2 31C0                <1>     xor    eax, eax
                             <1> sndc_init3:
                             <1>     ; 05/06/2024
000123B4 890D[08890100]      <1>     mov    [audio_dmabuff_size], ecx ; (*)
                             <1>     ; 12/05/2017
000123BA 803D[E5880100]01    <1>     cmp    byte [audio_device], 1 ; SB 16
000123C1 750B                <1>     jne    short sndc_init4
000123C3 C705[04890100]0000- <1>     mov    dword [audio_dma_buff], sb16_dma_buffer
000123CB 0500                <1>
                             <1>     ; 05/06/2024
                             <1>     ;mov   dword [audio_dmabuff_size], 65536
                             <1>     ;xor   eax, eax
                             <1>     ;mov   [u.r0], eax ; 0 = no error, successful
                             <1> sndc_init5: ; 29/07/2022
000123CD C3                  <1>     retn
                             <1>
                             <1> sndc_init4:
                             <1>     ; 02/12/2023 - TRDOS 386 v2.0.7
000123CE 81F900000100        <1>     cmp    ecx, 65536
000123D4 7707                <1>     ja     short sndc_init21
000123D6 B800000500          <1>     mov    eax, sb16_dma_buffer ; use already reserved buffer
000123DB EB0C                <1>     jmp    short sndc_init17
                             <1>
                             <1> sndc_init21:
                             <1>     ; EAX = Beginning address (physical)
                             <1>     ; EAX = 0 -> Allocate mem block from the 1st proper aperture
                             <1>     ; ECX = Number of bytes to be allocated     (>0)
000123DD E86339FFFF          <1>     call   allocate_memory_block
                             <1>     ;jc    sound_buff_error
                             <1>     ; 29/07/2022
000123E2 7305                <1>     jnc    short sndc_init17
000123E4 E988FDFFFF          <1>     jmp    sound_buff_error
                             <1>
                             <1> sndc_init17:       ; 29/07/2022
                             <1>     ; set dma buffer address and size parameters
000123E9 A3[04890100]        <1>     mov    [audio_dma_buff], eax ; dma buffer address
                             <1>     ; 05/06/2024
```

```
                            <1>     ;mov    [audio_dmabuff_size], ecx ; dma buffer size
                            <1>
                            <1>     ;;;
                            <1>     ; 05/06/2024
000123EE 8B0D[0C890100]     <1>     mov     ecx, [dma_hbuff_size] ; DMA half (1-2) buffer size
                            <1>     ; 04/06/2024
                            <1>     ;mov    ecx, [audio_buff_size] ; audio buffer size in bytes
                            <1>     ;;;
                            <1>
                            <1> ;   ; EAX = Beginning (physical) addr of the allocated mem block
                            <1> ;   ; ECX = Num of allocated bytes (rounded up to page borders)
                            <1> ;   cmp     byte [audio_pci], 0 ; AC97 audio controller ?
                            <1> ;   ja      short sndc_init4
                            <1> ;
                            <1> ;   ; Sound Blaster 16 uses classic DMA
                            <1> ;   mov     edx, eax
                            <1> ;   add     edx, ecx
                            <1> ;   cmp     edx, 1000000h ; 1st 16 MB
                            <1> ;   jna     short sndc_init4
                            <1> ;
                            <1> ;   ; error !
                            <1> ;   ; restore Memory Allocation Table Content
                            <1> ;   ; EAX = Beginning address (physical)
                            <1> ;   ; ECX = Number of bytes to be deallocated
                            <1> ;   call    deallocate_memory_block
                            <1> ;   ; reset dma buffer address and size parameters
                            <1> ;   xor     eax, eax ; 0
                            <1> ;   mov     [audio_dma_buff], eax ; 0
                            <1> ;   mov     [audio_dmabuff_size], ecx ; 0
                            <1> ;   jmp     sound_buff_error
                            <1> ;
                            <1> ;sndc_init4:
000123F4 803D[E5880100]03   <1>     cmp     byte [audio_device], 3
                            <1>     ;jne    short sndc_init5
                            <1>     ; 29/07/2022
000123FB 7505               <1>     jne     short sndc_init18 ; 28/05/2017
                            <1>
                            <1> ; 29/07/2022
                            <1> ;   call    set_vt8233_bdl
                            <1> ;sndc_init5:
                            <1> ;   ;sub    eax, eax ; 0
                            <1> ;   ;mov    [u.r0], eax ; 0 = no error, successful
                            <1> ;   retn
                            <1>
                            <1>     ; 05/06/2024
                            <1>     ;;;
                            <1>     ; 04/06/2024 (use truncated buffer size for BDL setup)
                            <1>     ; NOTE: Round up adds spurious bytes (noise) to the DMA buff;
                            <1>     ;       so, round down the size is better than the round up.
                            <1>     ;       ((BDL/SGD feature needs word aligned buffer address))
                            <1>     ;
                            <1>     ;mov    ecx, [audio_buff_size] ; audio buffer size in bytes
                            <1>     ;and    cl, ~1
                            <1>     ; 05/06/2024
                            <1>     ;mov    [dma_hbuff_size], ecx
                            <1>     ; ecx = DMA half buffer size as word aligned
                            <1>     ;       (in fact, half buffer is one of the two DMA buffers)
                            <1>     ;;;
                            <1>
000123FD E93E190000         <1>     jmp     set_vt8233_bdl
                            <1>
                            <1> sndc_init18:
                            <1>     ;call   set_ac97_bdl
                            <1>     ;;jmp   short sndc_init5
                            <1>     ;retn
                            <1>
                            <1>     ; 05/06/2024
                            <1>     ;;;
                            <1>     ;;;
                            <1>     ; 04/06/2024 (use truncated buffer size for BDL setup)
                            <1>     ; NOTE: Round up adds spurious bytes (noise) to the DMA buff;
                            <1>     ;       so, round down the size is better than the round up.
                            <1>     ;       ((BDL feature needs 8 byte aligned buffer address))
                            <1>     ;
                            <1>     ;mov    ecx, [audio_buff_size] ; audio buffer size in bytes
                            <1>     ;and    cl, ~7
                            <1>     ; 05/06/2024
                            <1>     ;mov    [dma_hbuff_size], ecx
                            <1>     ; ecx = DMA half buffer size as truncated for 8 byte alignment
                            <1>     ;       (in fact, half buffer is one of the two DMA buffers)
                            <1>     ;;;
                            <1>
                            <1>     ; 29/07/2022
00012402 E96A1F0000         <1>     jmp     set_ac97_bdl
                            <1>
                            <1> sound_play:
                            <1>     ; FUNCTION = 4
                            <1>     ; bl = Mode
                            <1>     ;       bit 0 = mono/stereo (1 = stereo)
                            <1>     ;       bit 1 = 8 bit / 16 bit (1 = 16 bit)
                            <1>     ; cx = Sampling Rate (Hz)
                            <1>
                            <1>     ; 13/06/2017
                            <1>     ; Note: Even if Mode bits are not 11b,
                            <1>     ;       AC'97 Audio Controller (&Codec)
                            <1>     ;       will play audio samples as 16 bit, stereo
                            <1>     ;       samples.
                            <1>     ;       (Program must fill the audio buffer
                            <1>     ;       as required; 8 bit samples must be converted
                            <1>     ;       to 16 bit samples and mono samples must be
                            <1>     ;       converted to stereo samples...)
                            <1>
                            <1>     ; 19/12/2024 (BugFix)
                            <1>     ; 05/06/2024
                            <1>     ; 04/06/2024
                            <1>     ; 30/07/2022
                            <1>     ; 28/07/2020
                            <1>     ; 27/07/2020
                            <1>     ; 28/05/2017
                            <1>     ; 15/05/2017, 20/05/2017
                            <1>     ; 21/04/2017, 24/04/2017
                            <1>     ; ... device check at first
00012407 A0[E5880100]       <1>     mov     al, [audio_device]
0001240C 08C0               <1>     or      al, al ; 0 ; pc speaker or invalid
                            <1>     ;jz     beeper_gfx ; 'video.s' ; temporary
                            <1>     ; 30/07/022
0001240E 7505               <1>     jnz     short snd_play_7
00012410 E9BDFFFEFF         <1>     jmp     beeper_gfx
                            <1>
                            <1> snd_play_7:
                            <1> ;   cmp     al, 3 ; VIA VT 8237R (vt8233)
                            <1> ;   je      short snd_play_1
                            <1> ;   cmp     al, 1 ; SB 16
                            <1> ;   jne     soundc_dev_err ; temporary !
```

```
                                <1> ;snd_play_0:
                                <1>     ; ... buffer & (buffer) owner check at second
00012415 833D[F8880100]00       <1>     cmp    dword [audio_buffer], 0
                                <1>     ;jna   sound_buff_error
                                <1>     ; 30/07/2022
0001241C 763C                   <1>     jna    short snd_play_4 ; jmp sound_buff_error
0001241E A0[BD8E0100]           <1>     mov    al, [u.uno]
00012423 3A05[11890100]         <1>     cmp    al, [audio_user]
                                <1>     ;jne   sndc_owner_error
                                <1>     ; 30/07/2022
00012429 7405                   <1>     je     short snd_play_3
0001242B E967FEFFFF             <1>     jmp    sndc_owner_error
                                <1> snd_play_3:
00012430 66890D[1A890100]       <1>     mov    [audio_freq], cx ; sample frequency (Hertz)
00012437 88D8                   <1>     mov    al, bl
00012439 2401                   <1>     and    al, 1 ; mono/stereo (1= stereo)
0001243B FEC0                   <1>     inc    al ; channels
0001243D A2[19890100]           <1>     mov    [audio_stmo], al ; sound channels (1 or 2)
00012442 B008                   <1>     mov    al, 8
00012444 F6C302                 <1>     test   bl, 2 ; bits per sample (1= 16 bit)
00012447 7402                   <1>     jz     short snd_play_bps
00012449 D0E0                   <1>     shl    al, 1
                                <1> snd_play_bps:
0001244B A2[18890100]           <1>     mov    [audio_bps], al
                                <1>
                                <1>     ; Transfer ring 3 (user's) audio buffer content to dma buffer
00012450 8B3D[04890100]         <1>     mov    edi, [audio_dma_buff] ; dma buffer (ring 0)
00012456 09FF                   <1>     or     edi, edi
                                <1>     ;jz    sound_buff_error
                                <1>     ; 30/07/2022
00012458 7505                   <1>     jnz    short snd_play_5
                                <1> snd_play_4:
0001245A E912FDFFFF             <1>     jmp    sound_buff_error
                                <1> snd_play_5:
                                <1>     ; 27/07/2020
0001245F 8B35[FC880100]         <1>     mov    esi, [audio_p_buffer] ; physical address (ring 3)
                                <1>     ;mov   ecx, [audio_buff_size] ; 15/05/2017
                                <1>
                                <1> ; 04/06/2024
                                <1> %if 0
                                <1>     mov    ecx, [audio_dmabuff_size] ; 27/07/2020
                                <1>     ;or    ecx, ecx
                                <1>     ;jz    sound_buff_error
                                <1>     ; 28/07/2020
                                <1>     shr    ecx, 1  ; dma half buffer size
                                <1> %else
                                <1>     ; 05/06/2024
00012465 8B0D[0C890100]         <1>     mov    ecx, [dma_hbuff_size]  ; DMA half buffer size
                                <1>
                                <1>     ; 04/06/2024
                                <1>     ;mov   ecx, [audio_buff_size]
                                <1>     ; 19/12/2024 (BugFix)
0001246B A0[E5880100]           <1>     mov    al, [audio_device]
                                <1>     ;
                                <1>     ;cmp   al, 1 ; Sound Blaster 16
                                <1>     ;je    short snd_play_8
                                <1>     ;and   cl, ~1
                                <1>     ;cmp   al, 3 ; VT8233 (VT8237R)
                                <1>     ;je    short snd_play_8
                                <1>     ; AC'97
                                <1>     ;and   cl, ~7
                                <1> snd_play_8:
                                <1> %endif
                                <1>
00012470 8035[10890100]01       <1>     xor    byte [audio_flag], 1 ;  0 -> 1, 1 -> 0
00012477 7502                   <1>     jnz    short snd_play_0 ; [audio_flag] = 1
                                <1>                             ; fill dma half buffer 1
                                <1>     ; [audio_flag] = 0
                                <1>
                                <1>     ; fill dma half buffer 2
00012479 01CF                   <1>     add    edi, ecx
                                <1>
                                <1> snd_play_0:
                                <1>     ; 05/06/2024
                                <1>     ;;rep  movsb
                                <1>     ;shr   ecx, 2 ; convert byte count to dword count
                                <1>     ;rep   movsd  ;
0001247B F3A4                   <1>     rep    movsb  ; SB16, AC97, VT8233
                                <1>
                                <1>     ; here, if [audio_flag] = 0, interrupt handler will update
                                <1>                             ; dma half buffer 2
                                <1>                             ; (user's audio buffer data will be
                                <1>                             ; copied into dma half buffer 2)
                                <1>     ;; 20/05/2017
                                <1>     ;mov   byte [audio_flag], 1 ; next half (on next time)
                                <1>
                                <1>     ; 24/04/2017
                                <1>     ;mov   al, [audio_device]
                                <1>     ; 04/06/2024
                                <1>     ; al = [audio_device]
                                <1>
0001247D 3C03                   <1>     cmp    al, 3 ; VT8233 (VT8237R)
0001247F 7409                   <1>     je     short snd_play_1
00012481 3C01                   <1>     cmp    al, 1 ; Sound Blaster 16
00012483 750A                   <1>     jne    short snd_play_2  ; 28/05/2017
                                <1>
                                <1> ; 30/07/2022
                                <1> ;   call   SbInit_play
                                <1> ;   ;jc    soundc_respond_err
                                <1> ;   ;retn
                                <1> ;   ; 30/07/2022
                                <1> ;   jnc    short snd_play_6  ; retn
                                <1> ;   jmp    soundc_respond_err
                                <1>
                                <1>     ; 30/07/2022
00012485 E9A71A0000             <1>     jmp    SbInit_play    ; sb16_start_play
                                <1>
                                <1> snd_play_1:
                                <1>     ;call  vt8233_start_play
                                <1>     ;retn
                                <1>     ; 30/07/2022
0001248A E9E5180000             <1>     jmp    vt8233_start_play
                                <1>
                                <1> snd_play_2:
                                <1>     ; 28/05/2017
                                <1>     ;cmp   al, 2 ; AC'97
                                <1>     ;jne   short snd_play_3
                                <1>
                                <1>     ;call  ac97_start_play
                                <1>     ;retn
                                <1>     ; 30/07/2022
0001248F E90E1F0000             <1>     jmp    ac97_start_play
                                <1>
                                <1> ;snd_play_3:
```

```
                                <1> ;   ;call  hda_start_play
                                <1> ;   retn
                                <1>     ; 30/07/2022
                                <1>     ;jmp   hda_start_play
                                <1>
                                <1> ; 04/06/2024
                                <1> ;snd_play_6:
                                <1> ;   ; 30/07/2022
                                <1> ;   retn
                                <1>
                                <1> sound_pause:
                                <1>     ; FUNCTION = 5
                                <1>     ; Pause
                                <1>     ; 28/05/2017
                                <1>     ; 24/04/2017
                                <1>     ; 22/04/2017
00012494 E82B030000             <1>     call   snd_dev_check
00012499 7277                   <1>     jc     short snd_nothing ; temporary.
0001249B E831030000             <1>     call   snd_buf_check
000124A0 7270                   <1>     jc     short snd_nothing ; temporary.
000124A2 A0[E5880100]           <1>     mov    al, [audio_device]
000124A7 3C03                   <1>     cmp    al, 3 ; VIA VT 8237R (vt8233)
000124A9 7409                   <1>     je     short snd_pause_1
000124AB 3C01                   <1>     cmp    al, 1 ; Sound Blaster 16
000124AD 750A                   <1>     jne    short snd_pause_2 ; 28/05/2017
000124AF E9501C0000             <1>     jmp    sb16_pause
                                <1> snd_pause_1:
000124B4 E970190000             <1>     jmp    vt8233_pause
                                <1> snd_pause_2:
                                <1>     ; 28/05/2017
                                <1>     ;cmp   al, 2 ; AC'97
                                <1>     ;jne   short snd_nothing ; temporary.
000124B9 E9E01F0000             <1>     jmp    ac97_pause
                                <1>
                                <1> sound_continue:
                                <1>     ; FUNCTION = 6
                                <1>     ; Continue to play
                                <1>     ; 28/05/2017
                                <1>     ; 22/04/2017
000124BE E801030000             <1>     call   snd_dev_check
000124C3 724D                   <1>     jc     short snd_nothing ; temporary.
000124C5 E807030000             <1>     call   snd_buf_check
000124CA 7246                   <1>     jc     short snd_nothing ; temporary.
000124CC A0[E5880100]           <1>     mov    al, [audio_device]
000124D1 3C03                   <1>     cmp    al, 3 ; VIA VT 8237R (vt8233)
000124D3 7409                   <1>     je     short snd_cont_1
000124D5 3C01                   <1>     cmp    al, 1 ; Sound Blaster 16
000124D7 750A                   <1>     jne    short snd_cont_2 ; 28/05/2017
000124D9 E9481C0000             <1>     jmp    sb16_continue
                                <1> snd_cont_1:
000124DE E9F8180000             <1>     jmp    vt8233_play
                                <1> snd_cont_2:
                                <1>     ; 28/05/2017
                                <1>     ;cmp   al, 2 ; AC'97
                                <1>     ;;jne  short snd_nothing ; temporary.
                                <1>     ; 30/07/2022
                                <1>     ;jne   short snd_cont_3
000124E3 E9F71E0000             <1>     jmp    ac97_play
                                <1> ;snd_cont_3:
                                <1>     ;jmp   hda_play
                                <1>
                                <1> sound_stop:
                                <1>     ; FUNCTION = 7
                                <1>     ; Stop playing
                                <1>     ; 30/07/2022
                                <1>     ; 28/05/2017
                                <1>     ; 24/05/2017
                                <1>     ; 21/04/2017, 22/04/2017, 24/04/2017
000124E8 E8D7020000             <1>     call   snd_dev_check
000124ED 7223                   <1>     jc     short snd_nothing ; temporary.
                                <1>     ;call  snd_buf_check
000124EF E8E6020000             <1>     call   snd_user_check ; 24/05/2017
000124F4 721C                   <1>     jc     short snd_nothing ; temporary.
                                <1>
000124F6 A0[E5880100]           <1>     mov    al, [audio_device]
000124FB 3C03                   <1>     cmp    al, 3 ; VIA VT 8237R (vt8233)
                                <1>     ;je    vt8233_stop
                                <1>     ;; 28/05/2017
                                <1>     ;;ja   short snd_nothing
                                <1>     ; 30/07/2022
000124FD 7505                   <1>     jne    short snd_stop_1
000124FF E933180000             <1>     jmp    vt8233_stop
                                <1> snd_stop_1:
00012504 3C01                   <1>     cmp    al, 1 ; Sound Blaster 16
                                <1>     ;je    sb16_stop
                                <1>     ; 30/07/2022
00012506 7505                   <1>     jne    short snd_stop_2
00012508 E93B1C0000             <1>     jmp    sb16_stop
                                <1> snd_stop_2:
                                <1>     ;cmp   al, 2
                                <1>     ;;je   short ac97_stop
                                <1>     ; 30/07/2022
                                <1>     ;jne   short snd_stop_3
0001250D E9831F0000             <1>     jmp    ac97_stop ; temporary.
                                <1> ;snd_stop_3:
                                <1>     ;jmp   hda_stop
                                <1>
                                <1> sndc_cancel_ok:
                                <1>     ; 30/07/2022
                                <1> sndc_reset_ok:
                                <1>     ; 30/07/2022
                                <1> snd_nothing:
                                <1>     ; 21/04/2017
00012512 C3                     <1>     retn
                                <1>
                                <1> soundc_reset:
                                <1>     ; FUNCTION = 8
                                <1>     ; Reset Audio Controller
                                <1>     ; 30/07/2022
                                <1>     ; 28/05/2017
                                <1>     ; 22/04/2017
00012513 E8AC020000             <1>     call   snd_dev_check
00012518 72F8                   <1>     jc     short snd_nothing ; temporary.
0001251A E8B2020000             <1>     call   snd_buf_check
0001251F 72F1                   <1>     jc     short snd_nothing ; temporary.
                                <1>
00012521 A0[E5880100]           <1>     mov    al, [audio_device]
                                <1>     ; 30/07/2022
00012526 3C03                   <1>     cmp    al, 3 ; VIA VT 8237R (vt8233)
00012528 7207                   <1>     jb     short sndc_reset_1
                                <1>     ;je    vt8233_reset
0001252A 77E6                   <1>     ja     short snd_nothing ; temporary.
                                <1>     ;;ja   hda_reset
                                <1>     ;ja    short sndc_reset_3
```

```
                                <1>     ; 30/07/2022
0001252C E904190000             <1>     jmp     vt8233_reset
                                <1> sndc_reset_1:
00012531 3C01                   <1>     cmp     al, 1 ; Sound Blaster 16
                                <1>     ;jne    ac97_reset
                                <1>     ; 30/07/2022
00012533 750C                   <1>     jne     short sndc_reset_2
00012535 E85F1C0000             <1>     call    sb16_reset
                                <1>     ;jc     soundc_respond_err
                                <1>     ;retn
                                <1>     ; 30/07/2022
0001253A 73D6                   <1>     jnc     short sndc_reset_ok
0001253C E90BFCFFFF             <1>     jmp     soundc_respond_err
                                <1> sndc_reset_2:
                                <1>     ; 30/07/2022
                                <1>     ;cmp    al, 2
                                <1>     ;ja     short sndc_reset_3
00012541 E914200000             <1>     jmp     ac97_reset
                                <1> ;sndc_reset_3:
                                <1>     ;jmp    hda_reset
                                <1>
                                <1> soundc_cancel:
                                <1>     ; FUNCTION = 9
                                <1>     ; Cancel audio callback service
                                <1>     ; 30/07/2022
                                <1>     ; 22/04/2017
00012546 A0[11890100]           <1>     mov     al, [audio_user]
0001254B 3A05[BD8E0100]         <1>     cmp     al, [u.uno]
00012551 75BF                   <1>     jne     short snd_nothing
                                <1>     ; RESET (audio) INTERRUPT CALLBACK SERVICE
00012553 8A1D[E6880100]         <1>     mov     bl, [audio_intr] ; IRQ number
00012559 A0[BD8E0100]           <1>     mov     al, [u.uno]
0001255E 28FF                   <1>     sub     bh, bh ; 0 ; unlink IRQ from user service
00012560 E81D030000             <1>     call    set_irq_callback_service
                                <1>     ;jc     sndc_perm_error ; 'permission denied' error
                                <1>     ;retn
                                <1>     ; 30/07/2022
00012565 73AB                   <1>     jnc     short sndc_cancel_ok
00012567 E930FDFFFF             <1>     jmp     sndc_perm_error
                                <1>
                                <1> sound_dalloc:
                                <1>     ; FUNCTION = 10
                                <1>     ; Deallocate (ring 3) audio buffer
                                <1>     ; 22/04/2017
0001256C A0[11890100]           <1>     mov     al, [audio_user]
00012571 3A05[BD8E0100]         <1>     cmp     al, [u.uno]
00012577 7599                   <1>     jne     short snd_nothing
00012579 8B1D[F8880100]         <1>     mov     ebx, [audio_buffer]
                                <1>     ;or     ebx, ebx
                                <1>     ;jz     short snd_nothing
0001257F 8B0D[00890100]         <1>     mov     ecx, [audio_buff_size]
00012585 E8133BFFFF             <1>     call    deallocate_user_pages
0001258A 31C0                   <1>     xor     eax, eax
0001258C A3[F8880100]           <1>     mov     [audio_buffer], eax ; 0
00012591 A2[11890100]           <1>     mov     [audio_user], al ; 0
                                <1> ;sndc_cancel_ok:
00012596 C3                     <1>     retn
                                <1>
                                <1> sound_volume:
                                <1>     ; FUNCTION = 11
                                <1>     ; Set sound volume level
                                <1>     ; 23/05/2024
                                <1>     ; 30/07/2022
                                <1>     ; 28/05/2017
                                <1>     ; 20/05/2017
                                <1>     ; 22/04/2017, 24/04/2017
                                <1>     ; bl = component
                                <1>     ;              0 = master/playback/lineout volume
                                <1>     ;              1 = PCM out volume ; 23/05/2024
                                <1>     ; cl = left channel volume level (0 to 31)
                                <1>     ; ch = right channel volume level (0 to 31)
                                <1>
00012597 80FB80                 <1>     cmp     bl, 80h
0001259A 720A                   <1>     jb      short snd_vol_1
                                <1>     ;ja     snd_nothing ; temporary.
                                <1>     ; 30/07/2022
0001259C 7707                   <1>     ja      short snd_vol_0
                                <1>     ; Set volume level for next play (BL>= 80h)
0001259E 66890D[20890100]       <1>     mov     [audio_master_volume], cx
                                <1> snd_vol_0:
000125A5 C3                     <1>     retn
                                <1> snd_vol_1:
                                <1>     ; set volume level immediate (BL< 80h)
                                <1>     ;cmp    bl, 0
                                <1>     ;;ja    snd_nothing ; temporary.
                                <1>     ; 30/07/2022
                                <1>     ;ja     short snd_vol_0
                                <1>     ; 23/05/2024
000125A6 80FB01                 <1>     cmp     bl, 1
000125A9 77FA                   <1>     ja      short snd_vol_0 ; temporary
                                <1>
000125AB E814020000             <1>     call    snd_dev_check
                                <1>     ;jc     snd_nothing ; temporary.
                                <1>     ; 30/07/2022
000125B0 72F3                   <1>     jc      short snd_vol_0
000125B2 E81A020000             <1>     call    snd_buf_check
                                <1>     ;jc     snd_nothing ; temporary.
                                <1>     ; 30/07/2022
000125B7 72EC                   <1>     jc      short snd_vol_0
                                <1>
000125B9 A0[E5880100]           <1>     mov     al, [audio_device]
000125BE 3C03                   <1>     cmp     al, 3 ; VIA VT 8237R (vt8233)
                                <1>     ;je     vt8233_volume
                                <1>     ; 30/07/2022
000125C0 7207                   <1>     jb      short snd_vol_2
                                <1>     ; 28/05/2017
                                <1>     ;ja     snd_nothing ; temporary.
                                <1>     ; 30/07/2022
000125C2 77E1                   <1>     ja      short snd_vol_0
                                <1>     ;ja     hda_volume
                                <1>     ; 30/07/2022
000125C4 E985180000             <1>     jmp     vt8233_volume
                                <1> snd_vol_2:
                                <1>     ; Sound Blaster 16
000125C9 3C01                   <1>     cmp     al, 1 ; SB 16
                                <1>     ;je     sb16_volume
                                <1>     ; 30/07/2022
000125CB 7705                   <1>     ja      short snd_vol_3
000125CD E9011B0000             <1>     jmp     sb16_volume
                                <1> snd_vol_3:
                                <1>     ; 30/07/2022
                                <1>     ;cmp    al, 2
                                <1>     ;ja     short snd_vol_4
000125D2 E9541E0000             <1>     jmp     ac97_volume
```

```
                                <1> ;snd_vol_4:
                                <1>     ;jmp   hda_volume
                                <1>
                                <1> soundc_disable:
                                <1>     ; FUNCTION = 12
                                <1>     ; Disable audio device (and unlink DMA memory)
                                <1>     ; 23/08/2024
                                <1>     ; 04/06/2024
                                <1>     ; 30/07/2022
                                <1>     ; 28/05/2017
                                <1>     ; 24/05/2017
                                <1>     ; 22/04/2017
000125D7 E8E8010000             <1>     call   snd_dev_check
                                <1>     ;jc    soundc_dev_err ; temporary.
                                <1>     ; 30/07/2022
000125DC 7305                   <1>     jnc    short snd_disable_4
000125DE E962FBFFFF             <1>     jmp    soundc_dev_err
                                <1> snd_disable_4:
                                <1>     ;call  snd_buf_check
                                <1>     ;;jc   sndc_owner_error ; temporary.
                                <1>     ; 30/07/2022
                                <1>     ;jnc   short snd_disable_5
                                <1>     ;jmp   sndc_owner_error
                                <1> ;snd_disable_5:
000125E3 A0[E5880100]           <1>     mov    al, [audio_device]
000125E8 3C03                   <1>     cmp    al, 3 ; VIA VT 8237R (vt8233)
000125EA 7414                   <1>     je     short snd_disable_1
                                <1>     ;ja    snd_nothing ; temporary.
                                <1>     ; 30/07/2022
000125EC 7766                   <1>     ja     short snd_disable_3 ; retn
000125EE 3C01                   <1>     cmp    al, 1 ; Sound Blaster 16
000125F0 7507                   <1>     jne    short snd_disable_0
000125F2 E8511B0000             <1>     call   sb16_stop
000125F7 EB0C                   <1>     jmp    short snd_disable_2
                                <1> snd_disable_0:
000125F9 E8971E0000             <1>     call   ac97_stop
000125FE EB05                   <1>     jmp    short snd_disable_2
                                <1> snd_disable_1:
00012600 E832170000             <1>     call   vt8233_stop
                                <1> snd_disable_2:
00012605 A0[E6880100]           <1>     mov    al, [audio_intr]
                                <1>
                                <1>     ; 15/01/2025
0001260A 20C0                   <1>     and    al, al
0001260C 740E                   <1>     jz     short snd_disable_5
                                <1>
0001260E 29DB                   <1>     sub    ebx, ebx ; 0 = reset
00012610 E840FAFFFF             <1>     call   set_dev_IRQ_service
                                <1>
                                <1>     ;mov   al, [audio_intr]
00012615 28E4                   <1>     sub    ah, ah ; 0 = reset
00012617 E800F8FFFF             <1>     call   set_hardware_int_vector
                                <1>
                                <1> snd_disable_5:     ; 15/01/2025
0001261C 31C0                   <1>     xor    eax, eax
0001261E A2[E5880100]           <1>     mov    byte [audio_device], al
00012623 A2[E6880100]           <1>     mov    byte [audio_intr], al
00012628 8705[04890100]         <1>     xchg   eax, [audio_dma_buff]
                                <1>
                                <1>     ; 15/01/2025
0001262E 09C0                   <1>     or     eax, eax
00012630 7422                   <1>     jz     short snd_disable_3
                                <1>
                                <1>     ; 24/05/2017
                                <1>     ;or    eax, eax
                                <1>     ;jz    short snd_disable_3
                                <1>     ;cmp   eax, sb16_dma_buffer ; default DMA buffer
                                <1>     ;je    short snd_disable_3
00012632 803D[E4880100]00       <1>     cmp    byte [audio_pci], 0 ; AC97 audio controller ?
00012639 7619                   <1>     jna    short snd_disable_3
0001263B C605[E4880100]00       <1>     mov    byte [audio_pci], 0
                                <1>
                                <1>     ; 23/08/2024 - bugfix
00012642 3D00000500             <1>     cmp    eax, sb16_dma_buffer ; reserved buffer ?
00012647 740B                   <1>     je     short snd_disable_3 ; it isn't an allocated mem buff
                                <1>
                                <1>     ;sub   ecx, ecx
                                <1>     ;xchg  ecx, [audio_dmabuff_size]
00012649 8B0D[08890100]         <1>     mov    ecx, [audio_dmabuff_size]
                                <1>     ; 26/11/2023
                                <1>     ; round up (always -rounded up- page count is allocated)
                                <1>     ; ((so deallocation must be done for the rounded up value))
                                <1>     ;add   ecx, PAGE_SIZE - 1   ; 4095
                                <1>     ;call  deallocate_memory_block
                                <1>     ; 04/06/2024
                                <1>     ;call  deallocate_memory_block_x
                                <1>                    ; deallocate ((ecx+4095)>>12) pages
0001264F E80B39FFFF             <1>     call   deallocate_memory_block
                                <1> snd_disable_3:
00012654 C3                     <1>     retn
                                <1>
                                <1> sound_dma_map:
                                <1>     ; FUNCTION = 13
                                <1>     ; Map audio dma buff addr to user's buffer addr
                                <1>     ; 30/07/2022
                                <1>     ; 12/05/2017
00012655 21C9                   <1>     and    ecx, ecx
                                <1>     ;jz    sound_buff_error
                                <1>     ; 30/07/2022
00012657 7505                   <1>     jnz    short snd_dma_map_3
00012659 E913FBFFFF             <1>     jmp    sound_buff_error
                                <1> snd_dma_map_3:
0001265E 803D[E5880100]01       <1>     cmp    byte [audio_device], 1
00012665 722A                   <1>     jb     short snd_dma_map_1
                                <1> snd_dma_map_0:
00012667 A1[04890100]           <1>     mov    eax, [audio_dma_buff]
0001266C 21C0                   <1>     and    eax, eax
0001266E 7421                   <1>     jz     short snd_dma_map_1
                                <1>     ;
00012670 8A1D[11890100]         <1>     mov    bl, [audio_user]
00012676 08DB                   <1>     or     bl, bl
00012678 7417                   <1>     jz     short snd_dma_map_1
0001267A 3A1D[BD8E0100]         <1>     cmp    bl, [u.uno]
                                <1>     ;jne   sndc_owner_error
                                <1>     ; 30/07/2022
00012680 7405                   <1>     je     short snd_dma_map_4
00012682 E910FCFFFF             <1>     jmp    sndc_owner_error
                                <1> snd_dma_map_4:
00012687 8B1D[08890100]         <1>     mov    ebx, [audio_dmabuff_size]
0001268D 21DB                   <1>     and    ebx, ebx
0001268F 750A                   <1>     jnz    short snd_dma_map_2
                                <1> snd_dma_map_1:
00012691 B800000500             <1>     mov    eax, sb16_dma_buffer
00012696 BB00000100             <1>     mov    ebx, 65536
```

```
                                <1> snd_dma_map_2:
0001269B 81C1FF0F0000           <1>     add     ecx, PAGE_SIZE-1 ; 4095
000126A1 6681E100F0             <1>     and     cx, ~PAGE_OFF ; not 4095
000126A6 39D9                   <1>     cmp     ecx, ebx
                                <1>     ;ja     sound_buff_error
                                <1>     ; 30/07/2022
000126A8 7605                   <1>     jna     short snd_dma_map_6
                                <1> snd_dma_map_5:
000126AA E9C2FAFFFF             <1>     jmp     sound_buff_error
                                <1> snd_dma_map_6:
000126AF 50                     <1>     push    eax
000126B0 89D3                   <1>     mov     ebx, edx
000126B2 C1E90C                 <1>     shr     ecx, 12 ; byte count to page count
                                <1>     ; eax = physical address of (audio) dma buffer
                                <1>     ; ebx = virtual address of (audio) dma buffer (user's pgdir)
                                <1>     ; ecx = page count (>0)
000126B5 E81539FFFF             <1>     call    direct_memory_access
000126BA 58                     <1>     pop     eax
                                <1>     ;jc     sound_buff_error
                                <1>     ; 30/07/2022
000126BB 72ED                   <1>     jc      short snd_dma_map_5
000126BD A3[6C8E0100]           <1>     mov     [u.r0], eax
000126C2 C3                     <1>     retn
                                <1>
                                <1> soundc_info:
                                <1>     ; FUNCTION = 14
                                <1>     ; Get Audio Controller Info
                                <1>     ; 19/11/2023
                                <1>     ; 30/07/2022
                                <1>     ; 10/06/2017
                                <1>     ; 05/06/2017
                                <1>
                                <1>     ;and    bl, bl ; 0
                                <1>     ;jz     short sndc_info_0
                                <1>
                                <1>     ; 19/11/2023
000126C3 80FB01                 <1>     cmp     bl, 1
000126C6 7609                   <1>     jna     short sndc_info_0
                                <1>
                                <1> ; invalid parameter !
                                <1> ; 30/07/2022
                                <1>     ;mov    eax, ERR_INV_PARAMETER ; 23
                                <1> ;sndc_inf_error:
                                <1> ;   mov     [u.r0], eax
                                <1> ;   mov     [u.error], eax
                                <1> ;   jmp     error
                                <1>     ; 30/07/2022
000126C8 29C0                   <1>     sub     eax, eax
000126CA B017                   <1>     mov     al, ERR_INV_PARAMETER ; 23
000126CC E980FAFFFF             <1>     jmp     sysaudio_err
                                <1>
                                <1> sndc_info_0:
000126D1 E8EE000000             <1>     call    snd_dev_check
                                <1>     ;jc     soundc_dev_err
                                <1>     ; 30/07/2022
000126D6 7305                   <1>     jnc     short sndc_info_3
                                <1> snd_data_dev_err:
000126D8 E968FAFFFF             <1>     jmp     soundc_dev_err
                                <1> sndc_info_3:
                                <1>     ; 19/11/2023
                                <1>     ;cmp    bl, 1
                                <1>     ;je     short sndc_info_4
000126DD 20DB                   <1>     and     bl, bl
000126DF 7546                   <1>     jnz     short sndc_info_4
                                <1>
000126E1 8B1D[F0880100]         <1>     mov     ebx, [audio_vendor]
000126E7 8B0D[EC880100]         <1>     mov     ecx, [audio_dev_id]
                                <1>     ;mov    al,  [audio_device]
000126ED 3C02                   <1>     cmp     al, 2 ; AC'97 (ICH)
000126EF 7513                   <1>     jne     short sndc_info_1
                                <1>     ; Intel AC97 (ICH) Audio Controller (=2)
000126F1 668B15[EA880100]       <1>     mov     dx, [NABMBAR]
000126F8 C1E210                 <1>     shl     edx, 16
000126FB 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00012702 EB07                   <1>     jmp     short sndc_info_2
                                <1> sndc_info_1:
                                <1>     ; 05/06/2017
                                <1>     ; Note: Intel HDA code (here) is not ready yet!
                                <1>     ; !!! SB16 or VT8233 (VT8237R) !!!
00012704 0FB715[EA880100]       <1>     movzx   edx, word [audio_io_base]
                                <1> sndc_info_2:
0001270B 88C4                   <1>     mov     ah, al ; [audio_device]
0001270D A0[E6880100]           <1>     mov     al, [audio_intr]
                                <1>
                                <1>     ; EAX = IRQ Number in AL
                                <1>     ;       Audio Device Number in AH
                                <1>     ; EBX = DEV/VENDOR ID
                                <1>     ;       (DDDDDDDDDDDDDDDDVVVVVVVVVVVVVVVV)
                                <1>     ; ECX = BUS/DEV/FN
                                <1>     ;       (00000000BBBBBBBBDDDDDFFF00000000)
                                <1>     ; EDX = NABMBAR/NAMBAR (for AC97)
                                <1>     ;       (Low word, DX = NAMBAR address)
                                <1>     ; EDX = Base IO Addr (DX) for SB16 & VT8233
                                <1>
                                <1>     ; 10/06/2017
00012712 A3[6C8E0100]           <1>     mov     [u.r0], eax
00012717 8B2D[688E0100]         <1>     mov     ebp, [u.usp]
0001271D 895D10                 <1>     mov     [ebp+16], ebx  ; ebx
00012720 895514                 <1>     mov     [ebp+20], edx  ; edx
00012723 894D18                 <1>     mov     [ebp+24], ecx  ; ecx
                                <1>
00012726 C3                     <1>     retn
                                <1>
                                <1> sndc_info_4:
                                <1>     ; 19/11/2023
00012727 3C02                   <1>     cmp     al, 2 ; Intel AC97 (ICH) Audio Controller
00012729 7413                   <1>     je      short sndc_info_7
                                <1> sndc_info_5:
                                <1>     ; return ZERO if it is not AC97 audio controller codec
0001272B 31C0                   <1>     xor     eax, eax ; 0
0001272D 31DB                   <1>     xor     ebx, ebx ; 0
                                <1> sndc_info_6:
0001272F A3[6C8E0100]           <1>     mov     [u.r0], eax
00012734 8B2D[688E0100]         <1>     mov     ebp, [u.usp]
0001273A 895D10                 <1>     mov     [ebp+16], ebx  ; ebx
0001273D C3                     <1>     retn
                                <1> sndc_info_7:
0001273E E8F01F0000             <1>     call    ac97_codec_info
00012743 72E6                   <1>     jc      short sndc_info_5
                                <1>
                                <1>     ; 26/11/2023 - temporary
                                <1>     ;and    al, 0FEh       ; clear VRA support bit for test
                                <1>
00012745 EBE8                   <1>     jmp     short sndc_info_6
```

```
                                <1>
                                <1> sound_data:
                                <1>     ; FUNCTION = 15
                                <1>     ; Get Current Sound data for graphics
                                <1>     ; 30/07/2022
                                <1>     ; 22/06/2017
                                <1>     ;
00012747 E878000000             <1>     call    snd_dev_check
                                <1>     ;jc     soundc_dev_err ; Device not ready !
                                <1>     ; 30/07/2022
0001274C 728A                   <1>     jc      short snd_data_dev_err
                                <1>
0001274E 80FB00                 <1>     cmp     bl, 0
00012751 760A                   <1>     jna     short sound_data_0
                                <1>
                                <1>     ; Only PCM OUT buffer data is valid for now!
00012753 B817000000             <1>     mov     eax, ERR_INV_PARAMETER  ; 23
00012758 E9F4F9FFFF             <1>     jmp     sysaudio_err
                                <1>
                                <1> sound_data_0:
0001275D A1[04890100]           <1>     mov     eax, [audio_dma_buff]
00012762 09C0                   <1>     or      eax, eax
                                <1>     ;jz     sound_buff_error
                                <1>     ; 30/07/2022
00012764 741A                   <1>     jz      short sound_data_5
                                <1>
00012766 803D[E5880100]04       <1>     cmp     byte [audio_device], 4 ; Intel HDA
0001276D 744C                   <1>     je      short sound_data_4 ; temporary ! (22/06/2017)
                                <1>
0001276F 21C9                   <1>     and     ecx, ecx
                                <1>     ;jnz    short sound_data_1 ; sample transfer
                                <1>
                                <1>     ; Return only DMA Buffer pointer/offset...
                                <1>     ; (If DMA Buffer has been mapped to user's
                                <1>     ;  memory space; program can get graphics
                                <1>     ;  data by using only this pointer value.)
                                <1>
                                <1>     ;call   get_dma_buffer_offset
                                <1>     ;; eax = DMA buffer offset
                                <1>     ;;      (!not half buffer offset!)
                                <1>     ;mov    [u.r0], eax
                                <1>     ;retn
                                <1>
                                <1>     ;jz     get_dma_buffer_offset
                                <1>     ; 30/07/2022
00012771 7505                   <1>     jnz     short sound_data_1
00012773 E95C1F0000             <1>     jmp     get_dma_buffer_offset
                                <1>
                                <1> sound_data_1:
                                <1>     ;mov    eax, [audio_dmabuff_size]
                                <1>     ;shr    eax, 1 ; half buffer size
                                <1>     ;cmp    ecx, eax
                                <1>     ;ja     short sound_buff_error
                                <1>
00012778 3B0D[08890100]         <1>     cmp     ecx, [audio_dmabuff_size]
                                <1>     ;ja     sound_buff_error
                                <1>     ; 30/07/2022
0001277E 7605                   <1>     jna     short sound_data_6
                                <1> sound_data_5:
00012780 E9ECF9FFFF             <1>     jmp     sound_buff_error
                                <1> sound_data_6:
00012785 89D0                   <1>     mov     eax, edx
00012787 25FF0F0000             <1>     and     eax, PAGE_OFF ; 4095 (0FFFh)
0001278C 81F900100000           <1>     cmp     ecx, 4096
00012792 7604                   <1>     jna     short sound_data_2
                                <1>     ;mov    ecx, 4096 ; max. 1 page
                                <1>     ; 30/07/2022
00012794 31C9                   <1>     xor     ecx, ecx
00012796 B510                   <1>     mov     ch, 16
                                <1>     ; ecx = 4096
                                <1> sound_data_2:
00012798 01C8                   <1>     add     eax, ecx
0001279A 3D00100000             <1>     cmp     eax, 4096
0001279F 7606                   <1>     jna     short sound_data_3
000127A1 6625FF0F               <1>     and     ax, PAGE_OFF ; 4095 (0FFFh)
000127A5 29C1                   <1>     sub     ecx, eax
                                <1>     ; here, ECX has been adjusted to fit
                                <1>     ;       in page border..  (<= 4096, >0)
                                <1> sound_data_3:
000127A7 51                     <1>     push    ecx
000127A8 52                     <1>     push    edx
000127A9 89D3                   <1>     mov     ebx, edx
000127AB E86034FFFF             <1>     call    get_physical_addr
000127B0 5A                     <1>     pop     edx
000127B1 59                     <1>     pop     ecx
                                <1>     ;jc     sound_buff_error
                                <1>     ; 30/07/2022
000127B2 72CC                   <1>     jc      short sound_data_5
                                <1>
                                <1>     ; eax = physical address of user's buffer
000127B4 89C3                   <1>     mov     ebx, eax
                                <1>     ; ecx = byte (transfer) count
                                <1>     ;call   get_current_sound_data
                                <1>     ;retn
000127B6 E96E1E0000             <1>     jmp     get_current_sound_data
                                <1>
                                <1> sound_data_4:
                                <1>     ; Intel HDA code is not ready yet !
                                <1>     ; 22/06/2017
000127BB 31C0                   <1>     xor     eax, eax
000127BD 48                     <1>     dec     eax
000127BE A3[6C8E0100]           <1>     mov     [u.r0], eax ; 0FFFFFFFFh
000127C3 C3                     <1>     retn
                                <1>
                                <1> snd_dev_check:
                                <1>     ; 10/06/2017
                                <1>     ; 05/06/2017
                                <1>     ; 24/05/2017
                                <1>     ; 22/04/2017
                                <1>     ; 21/04/2017
                                <1>     ; ... device check at first
000127C4 A0[E5880100]           <1>     mov     al, [audio_device]
000127C9 3C01                   <1>     cmp     al, 1 ; SB 16
000127CB 7203                   <1>     jb      short snd_dev_chk_retn ; error !
                                <1>     ;cmp    al, 4 ; Intel HDA
                                <1>     ;ja     short snd_dbchk_stc ; invalid !
                                <1>     ; 10/06/2017
000127CD 3C05                   <1>     cmp     al, 5
000127CF F5                     <1>     cmc
                                <1> snd_dev_chk_retn:
000127D0 C3                     <1>     retn
                                <1>
                                <1> snd_buf_check:
                                <1>     ; 10/06/2017
```

```
                                <1>     ; 22/04/2017
                                <1>     ; 21/04/2017
                                <1>     ; ... buffer & (buffer) owner check at second
000127D1 833D[F8880100]00       <1>     cmp     dword [audio_buffer], 0
000127D8 760D                   <1>     jna     short snd_dbchk_stc
                                <1> snd_user_check:
000127DA A0[BD8E0100]           <1>     mov     al, [u.uno]
000127DF 3A05[11890100]         <1>     cmp     al, [audio_user]
                                <1>     ;jne    short snd_dbchk_stc
                                <1>     ;retn
000127E5 74E9                   <1>     je      short snd_dev_chk_retn
                                <1>
                                <1> snd_dbchk_stc:
000127E7 F9                     <1>     stc
000127E8 C3                     <1>     retn
                                <1>
                                <1> sound_update:
                                <1>     ; FUNCTION = 16
                                <1>     ; bl =
                                <1>     ;    0 = automatic (sequental) update (with flag switch!)
                                <1>     ;    1 = update dma half buffer 1 (without flag switch!)
                                <1>     ;    2 = update dma half buffer 2 (without flag switch!)
                                <1>     ;  FFh = get current flag value
                                <1>     ;      0 = dma half buffer 1 (will be played next)
                                <1>     ;      1 = dma half buffer 2 (will be played next)
                                <1>     ;
                                <1>     ; 29/12/2024
                                <1>     ; Note: Requested half buffer is updated and
                                <1>     ;       flag is set to other value for other half buffer
                                <1>     ;
                                <1>     ; For example:
                                <1>     ;     * if BL input is 1, half buffer 1 is loaded and
                                <1>     ;         flag is switched to 1 -half buff 2 is in order-
                                <1>     ;     (next time half buffer 2 is updated automaticly)
                                <1>     ;     * if BL input is 2, half buffer 2 is updated/loaded
                                <1>     ;         and flag is switched to 0
                                <1>     ;
                                <1>     ; Return value for BL input = 0,1,2
                                <1>     ;     Current flag value (just after switched)
                                <1>     ;       -flag will have same value-
                                <1>     ;     If BL input is 1, return value will be 1 (1 xor 0)
                                <1>     ;     IF BL input is 2, return value will be 0 (1 xor 1)
                                <1>
                                <1>     ; 28/12/2024
                                <1>     ; 05/06/2024
                                <1>     ; 30/07/2022
                                <1>     ; 10/10/2017
                                <1>     ; ... device check at first
000127E9 A0[E5880100]           <1>     mov     al, [audio_device]
000127EE 08C0                   <1>     or      al, al ; 0 ; pc speaker or invalid
                                <1>     ;jz     soundc_dev_err
                                <1>     ; 30/07/2022
000127F0 7505                   <1>     jnz     short snd_update_4
000127F2 E94EF9FFFF             <1>     jmp     soundc_dev_err
                                <1> snd_update_4:
                                <1>     ; ... buffer & (buffer) owner check at second
000127F7 833D[F8880100]00       <1>     cmp     dword [audio_buffer], 0
                                <1>     ;jna    sound_buff_error
                                <1>     ; 30/07/2022
000127FE 761C                   <1>     jna     short snd_update_6 ; jmp sound_buff_error
00012800 A0[BD8E0100]           <1>     mov     al, [u.uno]
00012805 3A05[11890100]         <1>     cmp     al, [audio_user]
                                <1>     ;jne    sndc_owner_error
                                <1>     ; 30/07/2022
0001280B 7405                   <1>     je      short snd_update_5
0001280D E985FAFFFF             <1>     jmp     sndc_owner_error
                                <1> snd_update_5:
                                <1>     ; Transfer ring 3 (user's) audio buffer content to dma buffer
00012812 8B3D[04890100]         <1>     mov     edi, [audio_dma_buff] ; dma buffer (ring 0)
00012818 09FF                   <1>     or      edi, edi
                                <1>     ;jz     sound_buff_error
                                <1>     ; 30/07/2022
0001281A 7505                   <1>     jnz     short snd_update_7
                                <1> snd_update_6:
0001281C E950F9FFFF             <1>     jmp     sound_buff_error
                                <1> snd_update_7:
00012821 8B35[FC880100]         <1>     mov     esi, [audio_p_buffer] ; physical address (ring 3)
                                <1>     ; 05/06/2024
                                <1>     ;mov    ecx, [audio_buff_size]
                                <1>     ;
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ;mov    al, [audio_device]
                                <1>     ;cmp    al, 1
                                <1>     ;je     short snd_update_8 ; SB16
                                <1>     ;and    cl, ~1 ; word alignment
                                <1>     ;cmp    al, 3
                                <1>     ;je     short snd_update_8 ; VIA VT8233
                                <1>     ;; al = 2 ; AC97
                                <1>     ;and    cl, ~7 ; 8 byte alignment
                                <1> ;snd_update_8:
                                <1>     ;;;
                                <1>
                                <1>     ; 05/06/2024
00012827 8B0D[0C890100]         <1>     mov     ecx, [dma_hbuff_size] ; DMA half buffer size
                                <1>
                                <1>     ;movzx  eax, byte [audio_flag]
0001282D A0[10890100]           <1>     mov     al, [audio_flag]
00012832 FEC3                   <1>     inc     bl
00012834 742F                   <1>     jz      short snd_update_3 ; bl = 0FFh
00012836 FECB                   <1>     dec     bl
00012838 7417                   <1>     jz      short snd_update_0 ; bl = 0
                                <1>
                                <1>     ; 28/12/2024
0001283A 80FB02                 <1>     cmp     bl, 2
0001283D 7609                   <1>     jna     short snd_update_8
                                <1>     ;cmp    bl, 2
                                <1>     ;je     short snd_update_1 ; dma half buffer 2
                                <1>     ;jb     short snd_update_2 ; dma half buffer 1
                                <1>     ; 28/12/2024
                                <1>     ;jz     short  snd_update_2
                                <1>
                                <1>     ; invalid parameter !
                                <1> ; 30/07/2022
                                <1>     ;mov    eax, ERR_INV_PARAMETER ; 23
                                <1> ;   mov     [u.r0], eax
                                <1> ;   mov     [u.error], eax
                                <1> ;   jmp     error
                                <1>
                                <1> snd_update_err:
                                <1>     ; 30/07/2022
0001283F 29C0                   <1>     sub     eax, eax
00012841 B017                   <1>     mov     al, ERR_INV_PARAMETER ; 23
00012843 E909F9FFFF             <1>     jmp     sysaudio_err
```

```
                                <1>
                                <1>     ; 29/12/2024
                                <1> snd_update_8:
00012848 88D8                   <1>     mov     al, bl  ; 1 or 2
0001284A FEC8                   <1>     dec     al
0001284C A2[10890100]           <1>     mov     [audio_flag], al ; 0 or 1
                                <1> snd_update_0:
00012851 8035[10890100]01       <1>     xor     byte [audio_flag], 1 ; update flag !!!
                                <1> snd_update_9:
00012858 3C01                   <1>     cmp     al, 1
0001285A 7202                   <1>     jb      short snd_update_2 ; dma half buffer 1
                                <1> snd_update_1:
                                <1>     ; dma half buffer 2
0001285C 01CF                   <1>     add     edi, ecx
                                <1> snd_update_2:
                                <1>     ;rep    movsb
                                <1>     ; 05/06/2024
                                <1>     ;shr    ecx, 2
                                <1>     ;rep    movsd
0001285E F3A4                   <1>     rep     movsb ; SB16, AC97, VT8233
                                <1>     ; 29/12/2024
00012860 A0[10890100]           <1>     mov     al, [audio_flag]
                                <1> snd_update_3:
00012865 A3[6C8E0100]           <1>     mov     [u.r0], eax
                                <1>
0001286A C3                     <1>     retn
                                <1>
                                <1> sound_getvol:
                                <1>     ; FUNCTION = 17
                                <1>     ; Get sound volume level
                                <1>     ; 24/05/2024
                                <1>     ; bl = component
                                <1>     ;              0 = master/playback/lineout volume
                                <1>     ;              1 = PCM out volume
                                <1>     ; Return:
                                <1>     ; cl = left channel volume level (0 to 31)
                                <1>     ; ch = right channel volume level (0 to 31)
                                <1>
0001286B 80FB01                 <1>     cmp     bl, 1
0001286E 7709                   <1>     ja      short snd_gvol_0 ; temporary ; 24/05/2024
00012870 7408                   <1>     je      short snd_gvol_1
00012872 668B0D[20890100]       <1>     mov     cx, [audio_master_volume]
                                <1> snd_gvol_0:
00012879 C3                     <1>     retn
                                <1> snd_gvol_1:
0001287A 668B0D[22890100]       <1>     mov     cx, [audio_pcmo_volume]
00012881 C3                     <1>     retn
                                <1>
                                <1> %if 0
                                <1>
                                <1> sound_dmaclear:
                                <1>     ; FUNCTION = 18
                                <1>     ; Clear DMA buffer
                                <1>     ;       (may be useful for non-VRA AC97 hardware)
                                <1>     ; bl = cleaning byte value
                                <1>     ;      0 or 80h (not a necessary value, 0 is most proper)
                                <1>     ;
                                <1>     ; Return value
                                <1>     ;      eax = DMA half buffer size (DMA buff size / 2)
                                <1>     ;  If eax = 0
                                <1>     ;      there is not an active DMA buffer for user)
                                <1>     ;
                                <1>     ; 28/12/2024
                                <1>     mov     edi, [audio_dma_buff]
                                <1>     and     edi, edi
                                <1>     jz      short snd_dma_clear_ok
                                <1>     mov     ecx, [audio_dmabuff_size]
                                <1>     or      ecx, ecx
                                <1>     jz      short snd_dma_clear_ok
                                <1>     ; security (is necessary for multitasking)
                                <1>     mov     cl, [audio_user]
                                <1>     or      cl, cl
                                <1>     jz      short snd_dma_clear_ok
                                <1>     cmp     cl, [u.uno]
                                <1>     jne     short snd_dma_clear_ok
                                <1>     mov     ecx, [dma_hbuff_size]
                                <1>     and     ecx, ecx
                                <1>     jz      short snd_dma_clear_ok
                                <1>
                                <1>     mov     [u.r0], ecx
                                <1>     shl     ecx, 1 ; full buffer size in bytes
                                <1>     mov     al, bl
                                <1>     rep     stosb
                                <1>
                                <1> snd_dma_clear_ok:
                                <1>     retn
                                <1>
                                <1> %endif
                                <1>
                                <1> set_irq_callback_service:
                                <1>     ; 23/11/2023
                                <1>     ; 20/11/2023 (TRDOS 386 Kernel v2.0.7)
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 03/08/2020
                                <1>     ; 10/06/2017
                                <1>     ; 12/05/2017
                                <1>     ; 24/04/2017
                                <1>     ; 22/04/2017
                                <1>     ; caller: 'syscalbac' or 'sysaudio' or ...
                                <1>     ; 13/04/2017, 14/04/2017, 17/04/2017
                                <1>     ; 24/02/2017, 26/02/2017, 28/02/2017
                                <1>     ; 21/02/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; Link or unlink IRQ callback service to/from user (ring 3)
                                <1>     ;
                                <1>     ; INPUT ->
                                <1>     ;      If AL = 0, the caller is 'syscalbac';
                                <1>     ;         otherwise, the caller is 'sysaudio' or ...
                                <1>     ;         (AL = user number)
                                <1>     ;
                                <1>     ;      BL = IRQ number (Hardware interrupt request number)
                                <1>     ;           (0 to 15 but IRQ 0,1,2,6,8,14,15 are prohibited)
                                <1>     ;           IRQ numbers 3,4,5,7,9,10,11,12,13 are valid
                                <1>     ;           (numbers >15 are invalid)
                                <1>     ;
                                <1>     ;      BH = 0 = Unlink IRQ (in BL) from user (ring 3) service
                                <1>     ;           1 = Link IRQ by using Signal Response Byte method
                                <1>     ;           2 = Link IRQ by using Callback service method
                                <1>     ;           3 = Link IRQ by using Auto Increment S.R.B. method
                                <1>     ;          >3 = invalid
                                <1>     ;               (syscallback version will return to user)
                                <1>     ;
                                <1>     ;      CL = Signal Return/Response Byte value
                                <1>     ;
```

```
                                <1>     ;       If BH = 3, kernel will put a counter value ; 03/08/2020
                                <1>     ;                (into the S.R.B. addr)
                                <1>     ;                between 0 to 255. (start value = CL+1)
                                <1>     ;
                                <1>     ;       NOTE: counter value, for example: even and odd numbers
                                <1>     ;             may be used for -audio- DMA buffer switch
                                <1>     ;             within double buffer method, etc.
                                <1>     ;
                                <1>     ;       EDX = Signal return (Response) byte address
                                <1>     ;                         - or -
                                <1>     ;             Interrupt/Callback service/routine address
                                <1>     ;
                                <1>     ;             (virtual address in user's memory space)
                                <1>     ;
                                <1>     ; OUTPUT ->
                                <1>     ;       CF = 0 & EAX = 0 -> Successful setting
                                <1>     ;       CF = 1 & EAX > 0 -> IRQ is prohibited or locked
                                <1>     ;                           by another process
                                <1>     ;                 eax = ERR_PERM_DENIED -> prohibited or locked
                                <1>     ;                 eax = ERR_INV_PARAMETER ->
                                <1>     ;                       invalid parameter/option or bad address
                                <1>     ;
                                <1>     ; TRDOS 386 - IRQ CALLBACK structures (parameters):
                                <1>     ;
                                <1>     ;         [u.irqlock] : 1 word, IRQ flags (0-15) that indicates
                                <1>     ;                       which IRQs are locked by (that) user.
                                <1>     ;                        Lock and unlock (by user) will change
                                <1>     ;                       these flags or 'terminate process' (sysexit)
                                <1>     ;                       will clear these flags and unlock those IRQs.
                                <1>     ;
                                <1>     ;                       Bit 0 is for IRQ 0 and Bit 15 is for IRQ 15
                                <1>     ;
                                <1>     ;         IRQ(x).owner       : 1 byte, user, [u.uno], 0 = free (unlocked)
                                <1>     ;
                                <1>     ;         IRQ(x).method : 1 byte for callback method & status
                                <1>     ;                         0 = Signal Response Byte method
                                <1>     ;                         1 = Callback service method
                                <1>     ;                         >1 = invalid for current 'syscalback'.
                                <1>     ;                       or(+) 80h = IRQ is in use by system (ring 0)
                                <1>     ;                                   function (audio etc.) or
                                <1>     ;                                  a device driver.
                                <1>     ;                       (system function will ignore the lock/owner)
                                <1>     ;
                                <1>     ;         IRQ(x).srb  : 1 byte, Signal Return/Response byte value
                                <1>     ;                        (a fixed value by user or a counter value
                                <1>     ;                       from 0 to 255, which is increased by every
                                <1>     ;                       interrupt just before putting it into
                                <1>     ;                       the Signal Response byte address)
                                <1>     ;                       (This is not used in callback serv method)
                                <1>     ;
                                <1>     ;         IRQ(x).addr : 1 dword
                                <1>     ;                        Signal Response Byte address (physical)
                                <1>     ;                                    -or-
                                <1>     ;                        Callback service address (virtual)
                                <1>     ;
                                <1>     ;         IRQ(x).dev  : 1 byte
                                <1>     ;                        0 = Default device or kernel function
                                <1>     ;                                    -or-
                                <1>     ;                        1-255 = Assigned device driver number
                                <1>     ;
                                <1>     ;         (x) = 3,4,5,7,9,10,11,12,13
                                <1>     ;
                                <1>
00012882 80FB0F                 <1>     cmp     bl, 15
00012885 7727                   <1>     ja      short scbs_2
                                <1>
00012887 80FF03                 <1>     cmp     bh, 3
0001288A 7722                   <1>     ja      short scbs_2  ; invalid parameter
                                <1>
0001288C 0FB6FB                 <1>     movzx   edi, bl ; save IRQ number
                                <1>
                                <1>             ; IRQ 0,1,2,6,8,14,15 are prohibited
                                <1>     ;IRQenum: ; 'trdosk9.s'
                                <1>     ;       db  0,0,0,1,2,3,0,4,0,5,6,7,8,9,0,0
                                <1>
0001288F 0FB6B7[B0350100]       <1>     movzx   esi, byte [edi+IRQenum] ; IRQ availability
                                <1>                                     ; enumeration/index
                                <1>     ; 20/11/2023
00012896 4E                     <1>     dec     esi
                                <1>     ;dec    si
00012897 780F                   <1>     js      short scbs_1 ;  0 -> 0FFFFh
                                <1>
                                <1>     ; ESI = IRQ callback parameters index number (0 to 8)
                                <1>
00012899 08FF                   <1>     or      bh, bh
0001289B 7417                   <1>     jz      short scbs_4 ; unlink the IRQ (in BL)
                                <1>
0001289D FECF                   <1>     dec     bh
                                <1>     ; bh = method (0 = signal response byte, 1 = callback)
                                <1>     ;             (2 = auto increment of signal response byte)
                                <1>
0001289F 80BE[94880100]00       <1>     cmp     byte [esi+IRQ.owner], 0 ; locked ?
000128A6 7635                   <1>     jna     short scbs_6   ; no... OK...
                                <1>
                                <1> scbs_1:
                                <1>     ; permission denied (prohibited IRQ)
                                <1>     ;mov    eax, ERR_PERM_DENIED
                                <1>     ; 30/07/2022
000128A8 29C0                   <1>     sub     eax, eax
000128AA B00B                   <1>     mov     al, ERR_PERM_DENIED
000128AC F9                     <1>     stc
000128AD C3                     <1>     retn
                                <1> scbs_2:
                                <1>     ;stc
                                <1> scbs_3:
                                <1>     ;mov    eax, ERR_INV_PARAMETER
                                <1>     ; 30/07/2022
000128AE 29C0                   <1>     sub     eax, eax
000128B0 B017                   <1>     mov     al, ERR_INV_PARAMETER
000128B2 F9                     <1>     stc
000128B3 C3                     <1>     retn
                                <1>
                                <1> scbs_4: ; unlink the requested IRQ (if it belongs to current user)
                                <1>     ; 10/06/2017
                                <1>     ; 22/04/2017
                                <1>     ; 14/04/2017
                                <1>     ; If AL = 0 -> The caller is 'syscalbac'
000128B4 8AA6[94880100]         <1>     mov     ah, [esi+IRQ.owner]
000128BA 3A25[BD8E0100]         <1>     cmp     ah, [u.uno]
000128C0 75E6                   <1>     jne     short scbs_1
                                <1>
000128C2 FE0D[E68E0100]         <1>     dec     byte [u.irqc] ; decrease IRQ count (in use)
                                <1>
                                <1>     ;sub    ah, ah
```

```
                                <1>     ;mov    [esi+IRQ.owner], ah ; 0 ; free !!!
                                <1>     ;and    byte [esi+IRQ.method], 80h
                                <1>     ;mov    [esi+IRQ.srb], ah ; 0
                                <1>     ;mov    [esi+IRQ.dev], ah ; 0
                                <1>     ;mov    dword [esi+IRQ.addr], 0
                                <1>     ;mov    dword [u.r0], 0
                                <1>
                                <1>     ;mov    byte [esi+IRQ.owner], 0
                                <1>
                                <1>     ; 22/04/2017
000128C8 29C0                   <1>     sub     eax, eax
000128CA 8886[94880100]         <1>     mov     [esi+IRQ.owner], al ; 0
                                <1>     ; 10/06/2017
000128D0 8686[A6880100]         <1>     xchg    al, [esi+IRQ.method]
000128D6 2480                   <1>     and     al, 80h
000128D8 745D                   <1>     jz      short scbs_12
                                <1>     ; Audio device must be disabled -later- ! ([IRQ.medhod] = 80h)
                                <1>
                                <1> ;   cmp     byte [esi+IRQ.method], 80h ; device drv or kernel extension ?
                                <1> ;   jb      short scbs_12 ; bh = 0 reset to default IRQ handler
                                <1> ;
                                <1> ;   and     al, al
                                <1> ;   jz      short scbs_5  ; the caller is 'syscalbac'
                                <1> ;   ; The caller is 'sysaudio' or ...
000128DA 30C0                   <1>     xor     al, al
                                <1> ;   mov     [esi+IRQ.method], al ; 0 ; reset kernel extension flag
                                <1> ;scbs_5:
                                <1> ;   sub     ah, ah
                                <1>     ;mov    [u.r0], eax ; 0
000128DC C3                     <1>     retn
                                <1>
                                <1> scbs_6:
                                <1>     ; 14/04/2017
000128DD 20C0                   <1>     and     al, al
000128DF 7405                   <1>     jz      short scbs_7  ; the caller is 'syscalbac'
                                <1>     ; AL = user number ([u.uno] or [audio.user] or ...)
                                <1>     ; The caller is 'sysaudio' or ...
                                <1>     ;
                                <1>     ; bh = method (0 = signal response byte, 1 = callback)
                                <1>     ;             (2 = auto increment of signal response byte)
                                <1>
000128E1 80CF80                 <1>     or      bh, 80h        ; Kernel extension flag !
000128E4 EB0A                   <1>     jmp     short scbs_8
                                <1> scbs_7:
000128E6 8A86[A6880100]         <1>     mov     al, [esi+IRQ.method] ; >= 80h = kernel is using this IRQ
000128EC 2480                   <1>     and     al, 80h ; use only bit 7 (kernel function flag)
000128EE 08C7                   <1>     or      bh, al          ; method
                                <1>                             ; 0 = signal response byte, 1 = callback
                                <1>                             ; 2 = auto increment of s.r.b.
                                <1> scbs_8:
000128F0 A0[BD8E0100]           <1>     mov     al, [u.uno] ; user (process) number (1 to 16)
000128F5 8886[94880100]         <1>     mov     [esi+IRQ.owner], al ; lock the IRQ for user
000128FB 88BE[A6880100]         <1>     mov     [esi+IRQ.method], bh
                                <1>
                                <1> ;   test    bh, 1
                                <1> ;   jnz     short scbs_9    ; Callback method, CX will not be used
                                <1> ;
                                <1> ;   test    bh, 2           ; use auto increment (counter) method
                                <1> ;   jz      short scbs_10   ; (count can be used for buffer switch)
                                <1> ;scbs_9:
                                <1> ;   xor     ecx, ecx ; 0
                                <1>
                                <1> scbs_10:
                                <1>     ;mov    [esi+IRQ.method], bh
00012901 888E[AF880100]         <1>     mov     [esi+IRQ.srb], cl
00012907 C686[9D880100]00       <1>     mov     byte [esi+IRQ.dev], 0 ; device number is always 0
                                <1>                             ; for this system call
                                <1>     ;test   bh, 1
0001290E 80E701                 <1>     and     bh, 1 ; 17/04/2017
00012911 7513                   <1>     jnz     short scbs_11   ; callback method, use virtual address
                                <1>
00012913 53                     <1>     push    ebx ; IRQ number (in BL)
00012914 89D3                   <1>     mov     ebx, edx
                                <1>     ; ebx = virtual address
                                <1>     ; [u.pgdir] = page directory's physical address
00012916 FE05[52880100]         <1>     inc      byte [no_page_swap] ; 1
                                <1>                    ; Do not add this page to swap queue
                                <1>                    ; and remove it from swap queue if it is
                                <1>                    ; on the queue.
0001291C E8EF32FFFF             <1>     call    get_physical_addr
00012921 5B                     <1>     pop     ebx
00012922 728A                   <1>     jc      short scbs_3 ; invalid address !
                                <1>     ; eax = physical address of the virtual address in user's space
00012924 89C2                   <1>     mov     edx, eax
                                <1> scbs_11:
                                <1>     ;shl    si, 2           ; byte (index) to dword (offset)
                                <1>     ; 23/11/2023
00012926 C1E602                 <1>     shl     esi, 2
00012929 8996[B8880100]         <1>     mov     [esi+IRQ.addr], edx
                                <1>
0001292F FE05[E68E0100]         <1>     inc     byte [u.irqc]  ; increase IRQ (in use) count
                                <1>
00012935 FEC7                   <1>     inc     bh ; 17/04/2017
                                <1>     ; bh > 0 -> set to requested IRQ handler (IRQ_u_list)
                                <1> scbs_12:
00012937 88D8                   <1>     mov     al, bl ; IRQ number
00012939 88FC                   <1>     mov     ah, bh ; 0 = reset, >0 = set
0001293B E8DCF4FFFF             <1>     call    set_hardware_int_vector
                                <1>
00012940 31C0                   <1>     xor     eax, eax
                                <1>     ;mov    [u.r0], eax ; 0
                                <1>
00012942 C3                     <1>     retn    ; return with success (cf=0, eax=0)
                                <1>
                                <1> sysdma: ; DMA FUNCTIONS
                                <1>     ; 08/08/2022
                                <1>     ; 30/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 02/09/2017
                                <1>     ; 28/08/2017
                                <1>     ; 20/08/2017 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;
                                <1>     ; Inputs:
                                <1>     ;       BH = 0 -> Allocate DMA buffer
                                <1>     ;          BL = 0 -> Use the system's default DMA
                                <1>     ;                    (SB16) Buffer
                                <1>     ;               Buffer Size (max.) = 65536 bytes
                                <1>     ;          BL > 0 -> Allocate (a new) DMA buffer
                                <1>     ;          ECX = DMA Buffer Size in bytes (<=128KB)
                                <1>     ;          EDX = Virtual Address of DMA buffer
                                <1>     ;
                                <1>     ;       BH = 1 -> Initialize (Start) DMA service
                                <1>     ;            BL, bit 0 to 3 = Channel Number (0 to 7)
                                <1>     ;            BL, bit 7 = Auto Initialized Mode
                                <1>     ;                        (If bit 7 is set)
```

```
                                <1>     ;               bit 6 = Record (read) mode (0= playback)
                                <1>     ;            ECX = byte count (0 = use dma buffer size)
                                <1>     ;            EDX = physical buffer address
                                <1>     ;                  (0 = use dma buffer -start- address)
                                <1>     ;
                                <1>     ;      BH = 2 -> Get Current DMA Buffer Offset
                                <1>     ;           BL = DMA channel number
                                <1>     ;
                                <1>     ;      BH = 3 -> Get Current DMA count down value
                                <1>     ;           BL = DMA channel number (0 tO 7)
                                <1>     ;
                                <1>     ;      BH = 4 -> Get Current DMA channel (in progress)
                                <1>     ;
                                <1>     ;      BH = 5 -> Get System's Default DMA Buffer Address
                                <1>     ;
                                <1>     ;      BH = 6 -> Get Current DMA Buffer Address
                                <1>     ;
                                <1>     ;      BH = 7 -> Stop DMA service
                                <1>     ;
                                <1>     ; Outputs:
                                <1>     ;
                                <1>     ;      For BH = 0 ; Allocate DMA buffer
                                <1>     ;          EAX = Physical address of DMA buffer
                                <1>     ;          ECX = Allocated buffer size in bytes
                                <1>     ;                - page count * 4096 -
                                <1>     ;                (may be bigger than requested)
                                <1>     ;          If BL input > 0,
                                <1>     ;             'sysalloc:' system call will be used with
                                <1>     ;             EBX (for 'sysalloc') = EDX (for 'sysdma')
                                <1>     ;             ECX is same, byte count (buffer size)
                                <1>     ;             EDX = 1024*1024*16 ; 16 MB upper limit
                                <1>     ;          If BL input = 0,
                                <1>     ;             Default DMA buffer (SB16 buffer) will be
                                <1>     ;             checked and if it is free, it's address
                                <1>     ;             will be returned in EAX and it's size
                                <1>     ;             will be returned in ECX (as 65536)
                                <1>     ;
                                <1>     ;          If CF = 1, error code is in EAX
                                <1>     ;              EAX = -1 ; DMA buffer allocation error!
                                <1>     ;              EAX = 11 ; 'Permission Denied' error !
                                <1>     ;
                                <1>     ;              Note: 'sysalloc' error return method
                                <1>     ;                    will be applied if BL input > 0 !
                                <1>     ;
                                <1>     ;        For BH = 1 ; Initialize (Start) DMA
                                <1>     ;            EAX = 0 (Successful)
                                <1>     ;            If CF = 1, error code is in EAX
                                <1>     ;
                                <1>     ;        For BH = 2 ; Get Current DMA Buffer Offset
                                <1>     ;            EAX = DMA Buffer Offset (in bytes)
                                <1>     ;            ;
                                <1>     ;             AX = DMA buffer offset
                                <1>     ;            EAX bits 16 to 23 = Page register value
                                <1>     ;
                                <1>     ;        For BH = 3 ; Get Current DMA count down value
                                <1>     ;            EAX = Count down value (remain bytes)
                                <1>     ;
                                <1>     ;        For BH = 4 ; Get Current DMA channel (in progress)
                                <1>     ;            EAX = DMA channel number (0 to 7)
                                <1>     ;              AH = 0 if the owner is the caller process
                                <1>     ;              AH > 0 if the dma channel is in use by
                                <1>     ;                     another user/process
                                <1>     ;            EAX = -1 (0FFFFFFFFh)
                                <1>     ;                  if DMA service is not in use
                                <1>     ;                  (stopped or not initialized/started)
                                <1>     ;
                                <1>     ;        For BH = 5 ; Get System's Default DMA Buff Addr
                                <1>     ;            EAX = Default DMA Buffer Address (Physical)
                                <1>     ;                = offset 'sb16_dma_buffer:'
                                <1>     ;            ECX = Buffer size
                                <1>     ;                = 65536
                                <1>     ;
                                <1>     ;        For BH = 6 ; Get Current DMA Buffer Address
                                <1>     ;            EAX = Current DMA buffer address (Physical)
                                <1>     ;            ECX = Current DMA buffer size (setting value)
                                <1>     ;            Note: These values are for current dma channel
                                <1>     ;                  settings for the user/process
                                <1>     ;            ** For now (for current TRDOS 386 version)
                                <1>     ;              only one user/process can use only one
                                <1>     ;              dma channel & one dma buffer at same time
                                <1>     ;              (no multi tasking on DMA service) !!! **
                                <1>     ;            (Once, current DMA user must stop it's own DMA
                                <1>     ;             DMA service, than another user/program
                                <1>     ;             can use DMA service with same dma channel
                                <1>     ;             or with another DMA channel.)
                                <1>     ;
                                <1>     ;        For BH = 7 ; Stop DMA service (for current user
                                <1>     ;            and current DMA channel)
                                <1>     ;            EAX = 0 ; successful
                                <1>     ;            CF = 1 & EAX > 0 (= -1) -> Error
                                <1>
00012943 80FF07                 <1>     cmp     bh, 7
00012946 7612                   <1>     jna     short sysdma_0
                                <1>
                                <1> sysdma_err:
00012948 31C0                   <1>     xor     eax, eax
0001294A 48                     <1>     dec     eax ; -1
                                <1> sysdma_perm_err:
0001294B A3[6C8E0100]           <1>     mov     [u.r0], eax
00012950 A3[D88E0100]           <1>     mov     [u.error], eax ; DMA service error !
00012955 E914A1FFFF             <1>     jmp     error
                                <1>
                                <1> sysdma_0:
0001295A 08FF                   <1>     or      bh, bh
0001295C 7537                   <1>     jnz     short sysdma_1 ; 30/07/2022
                                <1>
0001295E 20DB                   <1>     and     bl, bl
00012960 7416                   <1>     jz      short sysdma_01
                                <1>
                                <1>     ; redirect system call to 'sysalloc'
00012962 89D3                   <1>     mov     ebx, edx ; virtual address of DMA buffer
                                <1>     ;ecx = Buffer size in bytes
                                <1>     ; DMA buffer address <= 16MB upper limit
00012964 BA00000001             <1>     mov     edx, 1024*1024*16 ; 16MB limit for DMA buff
                                <1>
00012969 C705[308D0100]FFFF-    <1>     mov     dword [dma_addr], 0FFFFFFFFh ; -1
00012971 FFFF                   <1>
                                <1>
00012973 E94DE8FFFF             <1>     jmp     sysalloc
                                <1>
                                <1> sysdma_01:
00012978 B800000500             <1>     mov     eax, sb16_dma_buffer
                                <1>
0001297D 803D[E5880100]01       <1>     cmp     byte [audio_device], 1
```

```
00012984 724A                <1>     jb      short sysdma_03
                             <1>
00012986 3B05[04890100]      <1>     cmp     eax, [audio_dma_buff]
0001298C 7527                <1>     jne     short sysdma_02
                             <1>
                             <1> sysdma_0_err:
0001298E B80B000000          <1>     mov     eax, ERR_PERM_DENIED
00012993 EBB6                <1>     jmp     short sysdma_perm_err
                             <1>
                             <1>     ; 30/07/2022
                             <1> sysdma_1:
00012995 80FF01              <1>     cmp     bh, 1
                             <1>     ;ja     sysdma_5
                             <1>     ; 30/07/2022
00012998 7605                <1>     jna     short sysdma_10
0001299A E90C010000          <1>     jmp     sysdma_5
                             <1>
                             <1> sysdma_10:
0001299F F6C340              <1>     test    bl, 40h ; record (read) mode -BL, bit 6-
                             <1>     ;jnz    sysdma_err ; not ready yet!
                             <1>     ; 30/07/2022
000129A2 757C                <1>     jnz     short sysdma_06 ; jmp sysdma_err
                             <1>
000129A4 A1[308D0100]        <1>     mov     eax, [dma_addr] ; physical address of dma buffer
000129A9 21C0                <1>     and     eax, eax
                             <1>     ;jz     sysdma_err
                             <1>     ; 30/07/2022
000129AB 7473                <1>     jz      short sysdma_06 ; jmp sysdma_err
                             <1>
000129AD 09D2                <1>     or      edx, edx
000129AF 7574                <1>     jnz     short sysdma_11
                             <1>
000129B1 89C2                <1>     mov     edx, eax
000129B3 EB74                <1>     jmp     short sysdma_12
                             <1>
                             <1> sysdma_02:
                             <1>     ; Only one user is permitted for audio/dma functions
                             <1>
000129B5 833D[04890100]00    <1>     cmp     dword [audio_dma_buff], 0
000129BC 7612                <1>     jna     short sysdma_03
                             <1>
000129BE 8A1D[11890100]      <1>     mov     bl, [audio_user]
000129C4 08DB                <1>     or      bl, bl
000129C6 7408                <1>     jz      short sysdma_03
                             <1>
000129C8 3A1D[BD8E0100]      <1>     cmp     bl, [u.uno]
000129CE 75BE                <1>     jne     short sysdma_0_err
                             <1>
                             <1> sysdma_03:
000129D0 8A1D[2D8D0100]      <1>     mov     bl, [dma_user]
000129D6 20DB                <1>     and     bl, bl
000129D8 750E                <1>     jnz     short sysdma_04
                             <1>
000129DA 8A1D[BD8E0100]      <1>     mov     bl, [u.uno]
000129E0 881D[2D8D0100]      <1>     mov     [dma_user], bl
                             <1>
000129E6 EB15                <1>     jmp     short sysdma_05
                             <1>
                             <1> sysdma_04:
000129E8 8B35[308D0100]      <1>     mov     esi, [dma_addr]
000129EE 21F6                <1>     and     esi, esi
000129F0 740B                <1>     jz      short sysdma_05
                             <1>
000129F2 46                  <1>     inc     esi ; -1 -> 0
000129F3 7408                <1>     jz      short sysdma_05
                             <1>
000129F5 3A1D[BD8E0100]      <1>     cmp     bl, [u.uno]
000129FB 7591                <1>     jne     short sysdma_0_err
                             <1>
                             <1> sysdma_05:
                             <1>     ; edx = virtual address (user's buffer address)
                             <1>     ;
000129FD 81F900000100        <1>     cmp     ecx, 65536   ; byte count (buffer size)
                             <1>     ;ja     sysdma_err
                             <1>     ; 30/07/2022
00012A03 771B                <1>     ja      short sysdma_06 ; jmp sysdma_err
                             <1>     ;
00012A05 81C1FF0F0000        <1>     add     ecx, PAGE_SIZE-1 ; 4095
00012A0B 6681E100F0          <1>     and     cx, ~PAGE_OFF ; not 4095
                             <1>     ;cmp    ecx, 65536
                             <1>     ;ja     sysdma_err
00012A10 51                  <1>     push    ecx     ; buffer size (allocated pages * 4096)
00012A11 50                  <1>     push    eax     ; offset sb16_dma_buffer
00012A12 89D3                <1>     mov     ebx, edx
00012A14 C1E90C              <1>     shr     ecx, 12 ; byte count to page count
                             <1>     ; eax = physical address of (audio) dma buffer
                             <1>     ; ebx = virtual address of (audio) dma buffer (user's pgdir)
                             <1>     ; ecx = page count (>0)
00012A17 E8B335FFFF          <1>     call    direct_memory_access
00012A1C 58                  <1>     pop     eax
00012A1D 59                  <1>     pop     ecx
                             <1>     ;jc     sysdma_err
                             <1>     ; 30/07/2022
00012A1E 7315                <1>     jnc     short sysdma_07
                             <1> sysdma_06:
00012A20 E923FFFFFF          <1>     jmp     sysdma_err
                             <1>
                             <1> sysdma_11:
00012A25 39C2                <1>     cmp     edx, eax
                             <1>     ;jb     sysdma_err
                             <1>     ; 30/07/2022
00012A27 72F7                <1>     jb      short sysdma_06 ; jmp sysdma_err
                             <1> sysdma_12:
00012A29 21C9                <1>     and     ecx, ecx
00012A2B 7515                <1>     jnz     short sysdma_13
                             <1>
00012A2D 8B0D[348D0100]      <1>     mov     ecx, [dma_size]
00012A33 EB15                <1>     jmp     short sysdma_14
                             <1>
                             <1> sysdma_07:
00012A35 A3[308D0100]        <1>     mov     [dma_addr], eax
00012A3A 890D[348D0100]      <1>     mov     [dma_size], ecx ; dma buffer size (in bytes)
                             <1>
                             <1>     ;mov    [u.r0], eax ; DMA Buffer Address (Physical)
                             <1>
                             <1>     ;mov    ebp, [u.usp]  ; ebp points to user's registers
                             <1>     ;mov    [ebp+24], ecx ; return to user with ecx value
                             <1>
                             <1>     ;jmp    sysret
                             <1>
                             <1>     ; 28/08/2017
00012A40 EB7A                <1>     jmp     sysdma_51
                             <1>
                             <1> sysdma_13:
```

```
00012A42 3B0D[348D0100]      <1>     cmp     ecx, [dma_size]
                             <1>     ;ja     sysdma_err
                             <1>     ; 30/07/2022
00012A48 77D6                <1>     ja      short sysdma_06 ; jmp sysdma_err
                             <1> sysdma_14:
00012A4A 89C6                <1>     mov     esi, eax
00012A4C 0335[348D0100]      <1>     add     esi, [dma_size]
                             <1>
00012A52 89D0                <1>     mov     eax, edx
00012A54 01C8                <1>     add     eax, ecx
                             <1>     ;jc     sysdma_err ; 02/09/2017
                             <1>     ; 30/07/2022
00012A56 72C8                <1>     jc      short sysdma_06 ; jmp sysdma_err
                             <1>
00012A58 39F0                <1>     cmp     eax, esi
                             <1>     ;ja     sysdma_err
                             <1>     ; 30/07/2022
00012A5A 77C4                <1>     ja      short sysdma_06 ; jmp sysdma_err
                             <1>
00012A5C 8B3D[04890100]      <1>     mov     edi, [audio_dma_buff]
00012A62 8B35[308D0100]      <1>     mov     esi, [dma_addr]
                             <1>
00012A68 09FF                <1>     or      edi, edi
00012A6A 7425                <1>     jz      short sysdma_16
                             <1>
00012A6C 803D[E5880100]01    <1>     cmp     byte [audio_device], 1
00012A73 7209                <1>     jb      short sysdma_15
                             <1>
                             <1>     ; Sound Blaster 16
00012A75 39FE                <1>     cmp     esi, edi
                             <1>     ;je     sysdma_0_err ; permmission denied !
                             <1>     ; 30/07/2022
00012A77 7505                <1>     jne     short sysdma_15
00012A79 E910FFFFFF          <1>     jmp     sysdma_0_err
                             <1> sysdma_15:
00012A7E C605[2F8D0100]48    <1>     mov     byte [dma_mode], 48h ; single mode playback
                             <1>
00012A85 F6C380              <1>     test    bl, 80h ; DMA mode - BL, bit 7, auto init -
00012A88 7407                <1>     jz      short sysdma_16
                             <1>     ; Auto initialized playback (write) mode
00012A8A 8005[2F8D0100]10    <1>     add     byte [dma_mode], 10h ; = 58h
                             <1> sysdma_16:
00012A91 80E307              <1>     and     bl, 07h
00012A94 881D[2E8D0100]      <1>     mov     [dma_channel], bl
00012A9A 8915[388D0100]      <1>     mov     [dma_start], edx
00012AA0 890D[3C8D0100]      <1>     mov     [dma_count], ecx
                             <1>
                             <1>     ; 28/08/2017
                             <1>     ;call   dma_init
                             <1>     ;jmp    sysret
00012AA6 E945010000          <1>     jmp     dma_init
                             <1>
                             <1> sysdma_5:
00012AAB 80FF05              <1>     cmp     bh, 5
00012AAE 726D                <1>     jb      short sysdma_3
00012AB0 7759                <1>     ja      short sysdma_6
                             <1>
                             <1>     ; Get the system's default dma buffer addr and size
00012AB2 B800000500          <1>     mov     eax, sb16_dma_buffer
00012AB7 B900000100          <1>     mov     ecx, 65536 ; Buffer size in bytes
                             <1>
                             <1> sysdma_51:
                             <1>     ; 0 = there is not a dma buffer (in use or available)
00012ABC A3[6C8E0100]        <1>     mov     [u.r0], eax
                             <1>
00012AC1 8B2D[688E0100]      <1>     mov     ebp, [u.usp]  ; ebp points to user's registers
00012AC7 894D18              <1>     mov     [ebp+24], ecx ; return to user with ecx value
                             <1>
00012ACA E9BF9FFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysdma_2:
                             <1>     ; Get current dma buffer offset (& page)
                             <1>     ; 28/08/2017
00012ACF 0FB635[2E8D0100]    <1>     movzx   esi, byte [dma_channel]
00012AD6 0FB696[E8350100]    <1>     movzx   edx, byte [dma_flip+esi]
00012ADD EE                  <1>     out     dx, al                  ; flip-flop clear
00012ADE 8A96[C0350100]      <1>     mov     dl, [dma_adr+esi]
00012AE4 EC                  <1>     in      al, dx                  ; get dma position
00012AE5 0FB6D8              <1>     movzx   ebx, al
00012AE8 EC                  <1>     in      al, dx
00012AE9 88C7                <1>     mov     bh, al
                             <1>
00012AEB 6683FE04            <1>     cmp     si, 4   ; channel number ?
00012AEF 7202                <1>     jb      short sysdma_21 ; 8 bit dma channel
                             <1>
00012AF1 D1E3                <1>     shl     ebx, 1  ; word offset to byte offset
                             <1>
                             <1> sysdma_21:
00012AF3 891D[6C8E0100]      <1>     mov     [u.r0], ebx
                             <1>
00012AF9 8A96[D0350100]      <1>     mov     dl, [dma_page+esi]
00012AFF EC                  <1>     in      al, dx                  ; get dma page
                             <1>
                             <1>     ;add    [u.ro+2], al
00012B00 0805[6E8E0100]      <1>     or      [u.r0+2], al
                             <1>
00012B06 E9839FFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysdma_6:
00012B0B 80FF06              <1>     cmp     bh, 6
00012B0E 775C                <1>     ja      short sysdma_7
                             <1>
                             <1>     ; 28/08/2017
                             <1>     ; Get current DMA buffer addr and size
00012B10 A1[308D0100]        <1>     mov     eax, [dma_addr] ; dma buffer address
00012B15 8B0D[348D0100]      <1>     mov     ecx, [dma_size] ; dma buffer size (in bytes)
                             <1>
00012B1B EB9F                <1>     jmp     short sysdma_51
                             <1>
                             <1> sysdma_3:
00012B1D 80FF03              <1>     cmp     bh, 3
00012B20 72AD                <1>     jb      short sysdma_2
00012B22 772F                <1>     ja      short sysdma_4
                             <1>
                             <1>     ; Get current dma count down value (remain bytes)
                             <1>     ; 28/08/2017
00012B24 0FB635[2E8D0100]    <1>     movzx   esi, byte [dma_channel]
00012B2B 0FB696[E8350100]    <1>     movzx   edx, byte [dma_flip+esi]
00012B32 EE                  <1>     out     dx, al                  ; flip-flop clear
00012B33 8A96[C8350100]      <1>     mov     dl, [dma_cnt+esi] ; dma count register addr
00012B39 EC                  <1>     in      al, dx
00012B3A 0FB6D8              <1>     movzx   ebx, al
00012B3D EC                  <1>     in      al, dx
00012B3E 88C7                <1>     mov      bh, al
```

```
                                <1>
00012B40 6683FE04               <1>     cmp     si, 4   ; channel number ?
00012B44 7202                   <1>     jb      short sysdma_31 ; 8 bit dma channel
                                <1>
00012B46 D1E3                   <1>     shl     ebx, 1 ; word count to byte count
                                <1>
                                <1> sysdma_31:
00012B48 891D[6C8E0100]         <1>     mov     [u.r0], ebx
                                <1>
00012B4E E93B9FFFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdma_4:
                                <1>     ; Get current DMA channel number
                                <1>     ; 28/08/2017
00012B53 8A25[2D8D0100]         <1>     mov     ah, [dma_user]
00012B59 20E4                   <1>     and     ah, ah
00012B5B 7539                   <1>     jnz     short sysdma_42
                                <1>
                                <1> sysdma_41:
                                <1>     ; Not a valid dma channel (in use)
00012B5D C705[6C8E0100]FFFF-    <1>     mov     dword [u.r0], -1 ; 0FFFFFFFFh
00012B65 FFFF                   <1>
00012B67 E9229FFFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdma_7:
                                <1>     ; DMA service STOP
00012B6C A0[BD8E0100]           <1>     mov     al, [u.uno]
00012B71 3A05[2D8D0100]         <1>     cmp     al, [dma_user]
00012B77 7543                   <1>     jne     short sysdma_72
                                <1>
00012B79 28C0                   <1>     sub     al, al ; 0
                                <1>
00012B7B A2[2D8D0100]           <1>     mov     [dma_user], al ; clear user
                                <1>
00012B80 8605[2F8D0100]         <1>     xchg    al, [dma_mode]
00012B86 20C0                   <1>     and     al, al
                                <1>     ;jz     short sysdma_err
00012B88 754E                   <1>     jnz     short sysdma_73
                                <1>
                                <1> sysdma_71:
00012B8A 31C0                   <1>     xor     eax, eax
00012B8C A3[6C8E0100]           <1>     mov     [u.r0], eax; 0
00012B91 E9F89EFFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdma_42:
00012B96 8B35[308D0100]         <1>     mov     esi, [dma_addr]
00012B9C 21F6                   <1>     and     esi, esi
00012B9E 74BD                   <1>     jz      short sysdma_41
                                <1>
00012BA0 46                     <1>     inc     esi ; -1 -> 0
00012BA1 74BA                   <1>     jz      short sysdma_41
                                <1>
00012BA3 A0[2E8D0100]           <1>     mov     al, [dma_channel]
                                <1>
00012BA8 3A25[BD8E0100]         <1>     cmp     ah, [u.uno]
00012BAE 7502                   <1>     jne     short sysdma_43
                                <1>
00012BB0 30E4                   <1>     xor     ah, ah ; DMA channel in use by current user
                                <1>
                                <1> sysdma_43:
00012BB2 A3[6C8E0100]           <1>     mov     [u.r0], eax ; AL = dma channel number
                                <1>                         ; AH > 0 if the the channel
                                <1>                         ; in use by another user/process
00012BB7 E9D29EFFFF             <1>     jmp     sysret
                                <1>
                                <1> sysdma_72:
                                <1>     ; 28/08/2017
00012BBC 803D[2D8D0100]00       <1>     cmp     byte [dma_user], 0
00012BC3 76C5                   <1>     jna     short sysdma_71 ; Nothing to do !
                                <1>
00012BC5 833D[308D0100]00       <1>     cmp     dword [dma_addr], 0
                                <1>     ;ja     sysdma_0_err
                                <1>     ; 30/07/2022
00012BCC 7605                   <1>     jna     short sysdma_74
00012BCE E9BBFDFFFF             <1>     jmp     sysdma_0_err
                                <1>
                                <1> sysdma_74:
00012BD3 A2[2D8D0100]           <1>     mov     [dma_user], al ; reset to current user
                                <1>
                                <1> sysdma_73:
                                <1>     ; 28/08/2017
00012BD8 0FB635[2E8D0100]       <1>     movzx   esi, byte [dma_channel]
00012BDF 0FB696[D8350100]       <1>     movzx   edx, byte [dma_mask+esi]
00012BE6 A0[2E8D0100]           <1>     mov     al, [dma_channel]
00012BEB 0C04                   <1>     or      al, 4
00012BED EE                     <1>     out     dx, al
                                <1>
00012BEE EB9A                   <1>     jmp     short sysdma_71
                                <1>
                                <1> dma_init:
                                <1>     ; 30/07/2022 (TRDOS 386 Kernel v2.0.5)
                                <1>     ; 28/08/2017
                                <1>     ; 20/08/2017
                                <1>     ; DMA initialization
                                <1>     ; 14/08/2017
                                <1>     ; 03/08/2017, 06/08/2017, 08/08/2017
                                <1>     ; 02/07/2017, 13/07/2017, 16/07/2017, 30/07/2017
                                <1>     ; (Derived from 'DMA_INIT' procedure in SB16MOD.ASM)
                                <1>     ; Modified for TRDOS 386 DMA buffer allocation & initialization !
                                <1>
00012BF0 8B1D[388D0100]         <1>     mov     ebx, [dma_start]
00012BF6 8B0D[3C8D0100]         <1>     mov     ecx, [dma_count]
                                <1>
00012BFC 0FB635[2E8D0100]       <1>     movzx   esi, byte [dma_channel]
                                <1>
00012C03 6683FE04               <1>     cmp     si, 4
00012C07 7204                   <1>     jb      short gdmi1
                                <1>     ; 08/08/2017
                                <1>     ;shr    cx, 1 ; word count
                                <1>     ; 30/07/2022
00012C09 D1E9                   <1>     shr     ecx, 1
00012C0B D1EB                   <1>     shr     ebx, 1 ; convert byte offset to word offset
                                <1> gdmi1:
                                <1>     ;mov    [dma_poff], bx ; 08/08/2017
                                <1>     ;dec    cx                      ; dma size = block size - 1
                                <1>     ; 30/07/2022
00012C0D 49                     <1>     dec     ecx
                                <1>
00012C0E 0FB696[D8350100]       <1>     movzx   edx, byte [dma_mask+esi] ; 30/07/2017
00012C15 A0[2E8D0100]           <1>     mov     al, [dma_channel]
00012C1A 0C04                   <1>     or      al, 4
00012C1C EE                     <1>     out     dx, al                  ; dma channel mask
                                <1>
00012C1D 30C0                   <1>     xor     al, al ; 0 ; any value ! 08/08/2017
```

```
00012C1F 8A96[E8350100]      <1>     mov     dl, [dma_flip+esi]
00012C25 EE                  <1>     out     dx, al                  ; flip-flop clear
                             <1>
00012C26 8A96[E0350100]      <1>     mov     dl, [dma_mod+esi]
00012C2C A0[2E8D0100]        <1>     mov     al, [dma_channel]  ; 13/07/2017
00012C31 2403                <1>     and     al, 3
                             <1>     ; 08/08/2017
00012C33 0A05[2F8D0100]      <1>     or      al, [dma_mode] ; 58h    ; dma mode for SB16
00012C39 EE                  <1>     out     dx, al
                             <1>
00012C3A 8A96[C0350100]      <1>     mov     dl, [dma_adr+esi]
00012C40 88D8                <1>     mov     al, bl
00012C42 EE                  <1>     out     dx, al                  ; offset low
                             <1>
00012C43 88F8                <1>     mov     al, bh
00012C45 EE                  <1>     out     dx, al                  ; offset high
                             <1>
00012C46 8A96[C8350100]      <1>     mov     dl, [dma_cnt+esi]
00012C4C 88C8                <1>     mov     al, cl
00012C4E EE                  <1>     out     dx, al                  ; size low
                             <1>
00012C4F 88E8                <1>     mov     al, ch
00012C51 EE                  <1>     out     dx, al                  ; size high
                             <1>
00012C52 8A96[D0350100]      <1>     mov     dl, [dma_page+esi]
                             <1>     ; 14/08/2017
00012C58 6683FE04            <1>     cmp     si, 4
00012C5C 7305                <1>     jnb     short gdmi2
00012C5E C1EB10              <1>     shr     ebx, 16
00012C61 EB06                <1>     jmp     short gdmi3
                             <1> gdmi2:
                             <1>     ; 09/08/2017
00012C63 C1EB0F              <1>     shr     ebx, 15 ; complete 16 bit shift
00012C66 80E3FE              <1>     and     bl, 0FEh ; clear bit 0 (not necessary)
                             <1> gdmi3:
00012C69 88D8                <1>     mov     al, bl
00012C6B EE                  <1>     out     dx, al                  ; page
                             <1>
00012C6C 8A96[D8350100]      <1>     mov     dl, [dma_mask+esi]
00012C72 A0[2E8D0100]        <1>     mov     al, [dma_channel]  ; 13/07/2017
00012C77 2403                <1>     and     al, 3
00012C79 EE                  <1>     out     dx, al                  ; dma channel unmask
                             <1>
                             <1>     ;retn
                             <1>     ; 28/08/2017
00012C7A E90F9EFFFF          <1>     jmp     sysret
                             <1>
                             <1> sysstdio: ; STDIN/STDOUT/STDERR functions
                             <1>     ; 18/09/2024
                             <1>     ; 07/09/2024
                             <1>     ;       (STDAUX/STDPRN functions, pre-definions)
                             <1>     ;       (!these functions are not ready!)
                             <1>     ; 24/08/2024
                             <1>     ; 23/08/2024
                             <1>     ; 20/08/2024 - TRDOS 386 Kernel v2.0.9
                             <1>     ;
                             <1>     ; Inputs:
                             <1>     ;       BL = 0 -> read a character on stdin (wait)
                             <1>     ;       BL = 1 -> read a character on stdin (no wait)
                             <1>     ;       BL = 2 -> write a character onto stdout (redirection)
                             <1>     ;       BL = 3 -> write a character onto stderr (no redirection)
                             <1>     ;       BL = 4 -> redirect stdin to file (if cl > 0)
                             <1>     ;       BL = 5 -> redirect stdout to file (if cl > 0)
                             <1>     ;       BL = 6 -> read character (ascii and scancode) on stdin
                             <1>     ;                                 -no redirection, wait-
                             <1>     ;       BL = 7 -> read character (ascii and scancode) on stdin
                             <1>     ;                                 -no redirection, no wait-
                             <1>     ;       BL = 8 -> write character and color onto stdout
                             <1>     ;                                 -no redirection-
                             <1>     ;       BL = 9 -> ungetchar (put back the ascii code in u.getc)
                             <1>     ;
                             <1>     ;       For BL=2,3,8,9
                             <1>     ;           CL = character (ascii) code
                             <1>     ;       For BL=8
                             <1>     ;           CH = color (Attribute) -CGA-
                             <1>     ;       For BL=4,5
                             <1>     ;           CL = file descriptor number + 1
                             <1>     ;           (File must be open and that number is 'u.fp' index)
                             <1>     ;
                             <1>     ;       07/09/2024 (these subfunctions will not be handled
                             <1>     ;                   by kernel v2.0.9 for now)
                             <1>     ;        -I am writing them here for C compiler compatibility,
                             <1>     ;         for now. Ref: SCC STDIO.H-
                             <1>     ;       BL = 10 -> get STDAUX status
                             <1>     ;       BL = 11 -> get/select STDAUX (COM) port
                             <1>     ;                                 (clears redirection)
                             <1>     ;       BL = 12 -> redirect STDAUX
                             <1>     ;       BL = 13 -> STDAUX IOCTL (control functions)
                             <1>     ;       BL = 14 -> read byte/character from STDAUX
                             <1>     ;       BL = 15 -> write byte/character to STDAUX
                             <1>     ;       BL = 16 -> get STDPRN status (LPT printer only)
                             <1>     ;       BL = 17 -> redirect STDPRN
                             <1>     ;       BL = 18 -> STDPRN IOCTL (init/control functions)
                             <1>     ;       BL = 19 -> write byte/character to STDPRN
                             <1>     ;
                             <1>     ;       For BL = 11
                             <1>     ;           If CL = 0 -> get STDAUX (COM) port number
                             <1>     ;              CL = 1 -> COM1
                             <1>     ;              CL = 2 -> COM2
                             <1>     ;              CL > 4 -> invalid
                             <1>     ;       For BL = 12
                             <1>     ;           If ECX = 0 -> clear redirection
                             <1>     ;           If  CH & 7Fh = 0 -> redirect INPUT
                             <1>     ;               CH & 7Fh > 0 -> redirect OUTPUT
                             <1>     ;           If  CH & 80h = 80h
                             <1>     ;              CL = file handle (descriptor index)
                             <1>     ;           If  CH & 80h = 0
                             <1>     ;              CL = console (pseudo) tty number,
                             <1>     ;                   1 to 8 for tty0 to tty7.
                             <1>     ;       For BL = 13
                             <1>     ;           If CL = 0 -> get control byte
                             <1>     ;           If CL > 1 -> set control byte (in CL)
                             <1>     ;              bit 0 = 1
                             <1>     ;              bit 1 = data bits (set=8, clear=7)
                             <1>     ;              bit 2 = stop bit (set=2, clear=1)
                             <1>     ;              bit 3 = parity bit (set=yes, clear=no)
                             <1>     ;              bit 4 = parity bit (set=even, clear=odd)
                             <1>     ;              bit 5-6-7 = data rate
                             <1>     ;                      000 = 9600
                             <1>     ;                       001 = 19200
                             <1>     ;                      010 = 38400
                             <1>     ;                      011 = 57600
                             <1>     ;                      100 = 115200
                             <1>     ;       For BL = 15, 19
```

```
                                <1>     ;           CL = byte/character to r/w
                                <1>     ;       For BL = 17
                                <1>     ;           If ECX = 0 -> clear redirection
                                <1>     ;           If  CH & 80h = 80h
                                <1>     ;               CL = file handle (descriptor index)
                                <1>     ;           If  CH & 80h = 0
                                <1>     ;               CL = console (pseudo) tty number,
                                <1>     ;                    1 to 8 for tty0 to tty7.
                                <1>     ;       For BL = 18
                                <1>     ;           If CL = 0 -> initialize printer (command=0)
                                <1>     ;              CL > 0 -> configuration/command byte
                                <1>     ;                       (reserved, not used)
                                <1>     ;
                                <1>     ; Outputs:
                                <1>     ;
                                <1>     ;       For BL=0,1,2,3,6,7,8,9
                                <1>     ;           AL = character (ascii) code
                                <1>     ;       For BL=6,7
                                <1>     ;           AH = scan code
                                <1>     ;       For BL=4,5
                                <1>     ;           AL = file descriptor (CL input - 1)
                                <1>     ;
                                <1>     ;       If CF=1
                                <1>     ;          AL (EAX) = error code
                                <1>     ;          If EAX = 0 -> EOF ; 18/09/2024
                                <1>     ;
                                <1>     ;       07/09/2024
                                <1>     ;       For BL = 10 to 19
                                <1>     ;           IF CF = 1, EAX/AL = error code
                                <1>     ;    .. If CF = 0 ...
                                <1>     ;       For BL = 10 & 16
                                <1>     ;           AL (EAX) = status
                                <1>     ;       For BL = 11
                                <1>     ;           EAX/AL = COM (serial) port (1 to 4)
                                <1>     ;       For BL = 12, 15, 17, 18, 19
                                <1>     ;           EAX/AL = 0
                                <1>     ;       For BL = 13
                                <1>     ;           If CL input = 0
                                <1>     ;                EAX/AL = current ctrl/cfg byte
                                <1>     ;            If CL input > 0
                                <1>     ;                EAX/AL = ctrl/cfg byte
                                <1>     ;       For BL = 14
                                <1>     ;           EAX/AL = byte/character
                                <1>     ;
                                <1>
00012C7F 80FB0A                 <1>     cmp     bl, (end_of_stdiofuncs-stdiofuncs)>>2
00012C82 7214                   <1>     jb      short sysstdio_0
                                <1>
                                <1>     ; invalid parameter (invalid sub function number)
00012C84 B817000000             <1>     mov     eax, ERR_INV_PARAMETER
                                <1> sysstdio_err:
00012C89 A3[6C8E0100]           <1>     mov     [u.r0], eax
00012C8E A3[D88E0100]           <1>     mov     [u.error], eax
00012C93 E9D69DFFFF             <1>     jmp     error
                                <1>
                                <1> sysstdio_0:
00012C98 31C0                   <1>     xor     eax, eax ; 0
00012C9A A3[D88E0100]           <1>     mov     [u.error], eax ; clear previous error code
00012C9F A3[6C8E0100]           <1>     mov     [u.r0], eax ; clear previous return code
                                <1>
00012CA4 0FB6F3                 <1>     movzx   esi, bl
00012CA7 C1E602                 <1>     shl     esi, 2
00012CAA FFA6[B02C0100]         <1>     jmp     dword [esi+stdiofuncs]
                                <1>
                                <1> stdiofuncs:
00012CB0 [3A2D0100]             <1>     dd      readstdinw
00012CB4 [3A2D0100]             <1>     dd      readstdinnw
00012CB8 [512E0100]             <1>     dd      writestdout
00012CBC [082E0100]             <1>     dd      writestderr
00012CC0 [D82C0100]             <1>     dd      redirstdin
00012CC4 [162D0100]             <1>     dd      redirstdout
                                <1> stdinacsc:
00012CC8 [262E0100]             <1>     dd      readstdin2w
00012CCC [262E0100]             <1>     dd      readstdin2nw
                                <1> stdoutcc:
00012CD0 [5C2E0100]             <1>     dd      writestdoutcc
00012CD4 [732E0100]             <1>     dd      ungetchar
                                <1> end_of_stdiofuncs:
                                <1>
                                <1> redirstdin:
                                <1>     ; CL = file handle (descriptor/index) + 1
00012CD8 08C9                   <1>     or      cl, cl
00012CDA 7508                   <1>     jnz     short redirstdin_0
00012CDC 880D[D48E0100]         <1>     mov     [u.stdin], cl ; 0 ; reset STDIN to keyboard
00012CE2 EB24                   <1>     jmp     short redirstdin_3
                                <1> redirstdin_0:
00012CE4 88C8                   <1>     mov     al, cl
00012CE6 E8B3D5FFFF             <1>     call    getf
                                <1>     ; eax = first cluster
                                <1>     ; ebx = system (not user) open file index
00012CEB 80BB[18840100]01       <1>     cmp     byte [ebx+OF_MODE], 1 ; open for read
00012CF2 7407                   <1>     je      short redirstdin_1
                                <1> redir_acc_denied:
00012CF4 B805000000             <1>     mov     eax, ERR_ACCESS_DENIED
00012CF9 EB8E                   <1>     jmp     short sysstdio_err
                                <1> redirstdin_1:
00012CFB 880D[D48E0100]         <1>     mov     [u.stdin], cl
                                <1> redirstdout_2:
00012D01 49                     <1>     dec     ecx
00012D02 880D[6C8E0100]         <1>     mov     [u.r0], cl ; return file descriptor
                                <1> redirstdin_3:
                                <1> redirstdout_3:
00012D08 66C705[D68E0100]00-    <1>     mov     word [u.ungetc], 0
00012D10 00                     <1>
                                <1>             ; reset u.ungetc and u.getc
00012D11 E9789DFFFF             <1>     jmp     sysret
                                <1>
                                <1> redirstdout:
                                <1>     ; CL = file handle (descriptor/index) + 1
00012D16 08C9                   <1>     or      cl, cl
00012D18 7508                   <1>     jnz     short redirstdout_0
00012D1A 880D[D58E0100]         <1>     mov     [u.stdout], cl ; 0 ; reset STDOUT to keyboard
00012D20 EBE6                   <1>     jmp     short redirstdout_3
                                <1> redirstdout_0:
00012D22 88C8                   <1>     mov     al, cl
00012D24 E875D5FFFF             <1>     call    getf
                                <1>     ; eax = first cluster
                                <1>     ; ebx = system (not user) open file index
00012D29 80BB[18840100]02       <1>     cmp     byte [ebx+OF_MODE], 2 ; open for write
00012D30 75C2                   <1>     jne     short redir_acc_denied
                                <1> redirstdout_1:
00012D32 880D[D58E0100]         <1>     mov     [u.stdout], cl
00012D38 EBC7                   <1>     jmp     short redirstdout_2
                                <1>
```

```
                                <1> readstdinw:
                                <1> readstdinnw:
                                <1>     ; read a character on stdin (wait)
                                <1>     ;sub    eax, eax
00012D3A 3805[D68E0100]         <1>     cmp     byte [u.ungetc], al ; 0
00012D40 761E                   <1>     jna     short readstdinw_0
                                <1>     ; read the character in u.getc buffer (put by ungetchar)
00012D42 A2[D68E0100]           <1>     mov     [u.ungetc], al ; 0
00012D47 8605[D78E0100]         <1>     xchg    al, [u.getc]
                                <1> readstdinw_retn:
00012D4D A2[6C8E0100]           <1>     mov     [u.r0], al
00012D52 E9379DFFFF             <1>     jmp     sysret
                                <1>
                                <1> readstdinw_3:
                                <1>     ; 23/08/2024
00012D57 B410                   <1>     mov     ah, 10h
00012D59 E8B4E1FEFF             <1>     call    int16h
00012D5E EBED                   <1>     jmp     short readstdinw_retn
                                <1>
                                <1> readstdinw_0:
00012D60 A0[D48E0100]           <1>     mov     al, [u.stdin]
00012D65 08C0                   <1>     or      al, al
00012D67 745F                   <1>     jz      short readstdinw_1
                                <1>
                                <1> readstdinf:
                                <1>     ; 24/08/2024
00012D69 48                     <1>     dec     eax
                                <1>
                                <1>     ; file
00012D6A 89C3                   <1>     mov     ebx, eax ; File handle (descriptor/index)
                                <1>     ; 18/09/2024
00012D6C B9[8B2E0100]           <1>     mov     ecx, charbuf ; buffer address
00012D71 BA01000000             <1>     mov     edx, 1  ; 1 character only
                                <1>
                                <1>     ; 18/09/2024
                                <1>     ;;;
00012D76 E825D5FFFF             <1>     call    getf1
00012D7B 722B                   <1>     jc      short redirect_fno_err
                                <1>
                                <1> readstdinf_@:
                                <1>     ; eax = first cluster
                                <1>     ; ebx = file descriptor (system, open files)
                                <1>
00012D7D E882A4FFFF             <1>     call    rw1
00012D82 7229                   <1>     jc      short redirection_err
                                <1>
00012D84 FE05[D28E0100]         <1>     inc     byte [u.kcall]
                                <1>     ;mov    byte [u.kcall], 1 ; >0 means system buffer
                                <1>                             ; (buff addr is not virtual)
00012D8A E8F7D8FFFF             <1>     call    readi
                                <1>
                                <1> redirected_rw_OK:
00012D8F 31C0                   <1>     xor     eax, eax
00012D91 3905[988E0100]         <1>     cmp     [u.nread], eax ; 0
00012D97 7623                   <1>     jna     short readstdinf_EOF
                                <1>
00012D99 A0[8B2E0100]           <1>     mov     al, [charbuf]
                                <1> ;redirected_w_not_OK:
00012D9E A3[6C8E0100]           <1>     mov     [u.r0], eax
00012DA3 E9E69CFFFF             <1>     jmp     sysret
                                <1>
                                <1> redirect_fno_err:
00012DA8 B80A000000             <1>     mov     eax, ERR_FILE_NOT_OPEN ; file not open !
                                <1> redirection_err:
00012DAD A3[D88E0100]           <1>     mov     [u.error], eax
                                <1> readstdinf_EOF_@:
00012DB2 A3[6C8E0100]           <1>     mov     [u.r0], eax ; error code ; 0 = EOF
00012DB7 E9B29CFFFF             <1>     jmp     error
                                <1>
                                <1> readstdinf_EOF:
                                <1>     ; 'end of file !' error
00012DBC C705[D88E0100]1000-    <1>     mov     dword [u.error], ERR_FILE_EOF ; 16
00012DC4 0000                   <1>
00012DC6 EBEA                   <1>     jmp     short readstdinf_EOF_@
                                <1>     ;;;
                                <1>
                                <1> readstdinw_1:
                                <1>     ; 18/09/2024
00012DC8 20DB                   <1>     and     bl, bl
00012DCA 748B                   <1>     jz      short readstdinw_3 ; wait (int16h, 10h)
                                <1>     ; no wait (int16h, 11h)
00012DCC B411                   <1>     mov     ah,11h  ; EXTENDED ASCII STATUS
00012DCE E83FE1FEFF             <1>     call    int16h
                                <1>     ; ah = scan code, al = ascii code
                                <1>     ;jnz    short readstdinw_retn
                                <1>     ; 23/08/2024
00012DD3 7582                   <1>     jnz     short readstdinw_3
                                <1> readstdinw_2:
                                <1>
                                <1> ; 23/08/2024
                                <1> ;   ; if zf=1 at here
                                <1> ;   ; it means 'no code available' for function 11h
                                <1> ;   and     bl, bl  ; 0 ?
                                <1> ;   jz      short  short readstdinw_retn ; function 10h
                                <1> ;   ; function 11h
                                <1> ;   ; [u.r0] = 0 = eax return
                                <1> readstdin2w@_retn:
00012DD5 E9B49CFFFF             <1>     jmp     sysret
                                <1>
                                <1> writestdoutf:
                                <1>     ; 24/08/2024
00012DDA 48                     <1>     dec     eax
                                <1>
                                <1>     ; file
00012DDB 89C3                   <1>     mov     ebx, eax ; File handle (descriptor/index)
                                <1>     ; 18/09/2024
00012DDD 880D[8B2E0100]         <1>     mov     byte [charbuf], cl
00012DE3 B9[8B2E0100]           <1>     mov     ecx, charbuf
00012DE8 BA01000000             <1>     mov     edx, 1  ; 1 character only
                                <1>
                                <1>     ; 18/09/2024
                                <1>     ;;;
00012DED E8AED4FFFF             <1>     call    getf1
00012DF2 72B4                   <1>     jc      short redirect_fno_err
                                <1>
                                <1> writestdoutf_@:
                                <1>     ; eax = first cluster
                                <1>     ; ebx = file descriptor (system, open files)
                                <1>
00012DF4 E80BA4FFFF             <1>     call    rw1
00012DF9 72B2                   <1>     jc      short redirection_err
                                <1>
00012DFB FE05[D28E0100]         <1>     inc     byte [u.kcall]
                                <1>     ;mov    byte [u.kcall], 1 ; >0 means system buffer
```

```
                                <1>                         ; (buff addr is not virtual)
00012E01 E859DFFFFF             <1>     call    writei
                                <1>
                                <1>     ;xor    eax, eax
                                <1>     ;cmp    [u.nread], eax ; 0
                                <1>     ;jna    short redirected_w_not_OK
00012E06 EB87                   <1>     jmp     short redirected_rw_OK
                                <1>     ;;;
                                <1>
                                <1> writestderr:        ; skip redirection
                                <1>             ; STDERR = STDOUT (as file redirection disabled)
00012E08 88C8                   <1>     mov     al, cl  ; character
00012E0A A2[6C8E0100]           <1>     mov     [u.r0], al
00012E0F B40E                   <1>     mov     ah, 0Eh ; write a character (as tty write)
00012E11 BB07000000             <1>     mov     ebx, 07h ; video page 0 (and color/attrib 07h)
                                <1>     ; 23/08/2024
00012E16 8A3D[DE760100]         <1>     mov     bh, [ptty] ; ACTIVE_PAGE
00012E1C E8F9E8FEFF             <1>     call    _int10h
00012E21 E9689CFFFF             <1>     jmp     sysret
                                <1>
                                <1> readstdin2w:
                                <1> readstdin2nw:
00012E26 66C705[D68E0100]00-    <1>     mov     word [u.ungetc], 0
00012E2E 00                     <1>
                                <1>             ; reset u.ungetc and u.getc
00012E2F B410                   <1>     mov     ah, 10h  ; Keyboard, EXTENDED READ
                                <1>     ;cmp    bl, 6
00012E31 80FB06                 <1>     cmp     bl, (stdinacsc-stdiofuncs)>>2
00012E34 740B                   <1>     je      short readstdin2w_1 ; wait (int16h, 10h)
                                <1>     ; no wait (int16h, 11h)
00012E36 FEC4                   <1>     inc     ah  ; function 11h ; EXTENDED ASCII STATUS
                                <1>
                                <1>     ; 24/08/2024
00012E38 E8D5E0FEFF             <1>     call    int16h
                                <1>     ; ah = scan code, al = ascii code
                                <1>     ;jnz    short readstdin2w_retn
                                <1>     ; 23/08/2024
00012E3D 7496                   <1>     jz      short readstdin2w@_retn
                                <1> ;readstdin2w_1:
                                <1>
                                <1> ; 23/08/2024
                                <1> ;   ; if zf=1 at here
                                <1> ;   ; it means 'no code available' for function 11h
                                <1> ;   ;cmp    bl, 6
                                <1> ;   cmp     bl, (stdinacsc-stdiofuncs)>>2
                                <1> ;   ;je     short readstdin2w_retn ; function 10h
                                <1> ;   ; function 11h
                                <1> ;   ; [u.r0] = 0 = eax return
                                <1> ;   ;jmp    sysret
                                <1> ;   jne     short readstdin2w@_retn
                                <1>
                                <1>     ; 23/08/2024
00012E3F B410                   <1>     mov     ah, 10h
                                <1> readstdin2w_1:      ; 18/09/2024
00012E41 E8CCE0FEFF             <1>     call    int16h
                                <1>
                                <1> readstdin2w_retn:
00012E46 66A3[6C8E0100]         <1>     mov     [u.r0], ax
00012E4C E93D9CFFFF             <1>     jmp     sysret
                                <1>
                                <1> writestdout:
                                <1>     ; 18/09/2024
00012E51 A0[D58E0100]           <1>     mov     al, [u.stdout]
00012E56 20C0                   <1>     and     al, al
00012E58 7580                   <1>     jnz     short writestdoutf ; 18/09/2024
                                <1>
                                <1> writestdout_1:
                                <1>     ;mov    byte [ccolor], 07h ; default color (CGA)
                                <1>                     ; (black background, light gray character)
                                <1>     ; 18/09/2024
00012E5A B507                   <1>     mov     ch, 07h
                                <1> writestdoutcc:
00012E5C BE[6C8E0100]           <1>     mov     esi, u.r0 ; buffer
                                <1>     ; 18/09/2024
00012E61 880E                   <1>     mov     [esi], cl ; character
                                <1>     ;;cmp   bl, 8 ; write char and color
                                <1>     ;cmp    bl, (stdoutcc-stdiofuncs)>>2
                                <1>     ;jne    short writestdout_2
00012E63 882D[DF760100]         <1>     mov     byte [ccolor], ch ; color/attribute (CGA)
                                <1> writestdout_2:
00012E69 E8A9D6FFFF             <1>     call    print_cmsg
                                <1>     ; 18/09/2024
                                <1>     ;mov    byte [ccolor], 07h ; set default color again
                                <1>
                                <1>     ; [u.r0] = written character (AL)
00012E6E E91B9CFFFF             <1>     jmp     sysret
                                <1>
                                <1> ungetchar:
                                <1>     ; put a character back in u.getc STDIN buffer
00012E73 C605[D68E0100]01       <1>     mov     byte [u.ungetc], 1
00012E7A 880D[D78E0100]         <1>     mov     [u.getc], cl
00012E80 880D[6C8E0100]         <1>     mov     [u.r0],cl
00012E86 E9039CFFFF             <1>     jmp     sysret
                                <1>
                                <1>     ; 18/09/2024
00012E8B 00                     <1> charbuf: db 0
                                <1>
                                <1> otty:
                                <1> sret:
                                <1> ocvt:
                                <1> ctty:
                                <1> cret:
                                <1> ccvt:
                                <1> rtty:
                                <1> wtty:
                                <1> rmem:
                                <1> wmem:
                                <1> rfd:
                                <1> rhd:
                                <1> wfd:
                                <1> whd:
                                <1> rlpt:
                                <1> wlpt:
                                <1> rcvt:
                                <1> xmtt:
00012E8C C3                     <1>     retn
                                    %include 'trdosk9.s' ; 04/01/2016
                                <1> ; ******************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - INITIALIZED DATA : trdosk9.s
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Last Update: 10/02/2026 (Previous: 29/12/2024 - Kernel v2.0.9)
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Beginning: 04/01/2016
                                <1> ; ------------------------------------------------------------------------------
```

```
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ---------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ***************************************************************************
                                <1> ; DRV_INIT.ASM [26/09/2009] Last Update: 07/08/2011
                                <1> ; MAINPROG.ASM [17/01/2004] Last Update: 09/11/2011
                                <1> ; CMD_INTR.ASM [29/01/2005] Last Update: 09/11/2011
                                <1> ; FILE.ASM [29/10/2009] Last Update: 09/10/2011
                                <1>
                                <1> ; 12/02/2016
                                <1> Last_DOS_DiskNo:
00012E8D 01                     <1>             db 1 ; A: = 0 & B: = 1
                                <1>
                                <1> Restore_CDIR:
00012E8E FF                     <1>             db 0FFh ; Initial value -> any number except 0
                                <1>
                                <1> msg_CRLF_temp:
00012E8F 070D0A00               <1>             db 07h, 0Dh, 0Ah, 0
                                <1>
                                <1> Magic_Bytes:
00012E93 04                     <1>             db 4
00012E94 01                     <1>             db 1
                                <1> mainprog_Version:
00012E95 07                     <1>             db 7
00012E96 5B5452444F535D204D-    <1>             db "[TRDOS] Main Program v2.0.10 (10/02/2026)"
00012E9F 61696E2050726F6772-    <1>
00012EA8 616D2076322E302E31-    <1>
00012EB1 30202831302F30322F-    <1>
00012EBA 3230323629             <1>
00012EBF 0D0A                   <1>             db 0Dh, 0Ah
00012EC1 286329204572646F67-    <1>             db "(c) Erdogan Tan 2005-2026"
00012ECA 616E2054616E203230-    <1>
00012ED3 30352D32303236         <1>
00012EDA 0D0A00                 <1>             db 0Dh, 0Ah, 0
                                <1>
00012EDD 4D41494E50524F472E-    <1> MainProgCfgFile: ; 14/04/2016
00012EE6 43464700               <1>             db "MAINPROG.CFG", 0
                                <1>
                                <1> TRDOSPromptLabel:
00012EEA 5452444F53             <1>             db "TRDOS"
00012EEF 00                     <1>             db 0
00012EF0 00<rep 5h>             <1>                 times 5 db 0
00012EF5 00                     <1>             db 0
                                <1>
                                <1> ; INTERNAL COMMANDS
                                <1> Command_List:
00012EF6 44495200               <1> Cmd_Dir:    db "DIR", 0
00012EFA 434400                 <1> Cmd_Cd:             db "CD", 0
00012EFD 433A00                 <1> Cmd_Drive:  db "C:", 0
00012F00 56455200               <1> Cmd_Ver:    db "VER", 0
00012F04 4558495400             <1> Cmd_Exit:   db "EXIT", 0
00012F09 50524F4D505400         <1> Cmd_Prompt: db "PROMPT", 0
00012F10 564F4C554D4500         <1> Cmd_Volume: db "VOLUME", 0
00012F17 4C4F4E474E414D4500     <1> Cmd_LongName:       db "LONGNAME", 0
00012F20 4441544500             <1> Cmd_Date:   db "DATE", 0
00012F25 54494D4500             <1> Cmd_Time:   db "TIME", 0
00012F2A 52554E00               <1> Cmd_Run:    db "RUN", 0
00012F2E 53455400               <1> Cmd_Set:    db "SET", 0
00012F32 434C5300               <1> Cmd_Cls:    db "CLS", 0
00012F36 53484F5700             <1> Cmd_Show:   db "SHOW", 0
00012F3B 44454C00               <1> Cmd_Del:    db "DEL", 0
00012F3F 41545452494200         <1> Cmd_Attrib: db "ATTRIB", 0
00012F46 52454E414D4500         <1> Cmd_Rename: db "RENAME", 0
00012F4D 524D44495200           <1> Cmd_Rmdir:  db "RMDIR", 0
00012F53 4D4B44495200           <1> Cmd_Mkdir:  db "MKDIR", 0
00012F59 434F505900             <1> Cmd_Copy:   db "COPY", 0
00012F5E 4D4F564500             <1> Cmd_Move:   db "MOVE", 0
00012F63 5041544800             <1> Cmd_Path:   db "PATH", 0
00012F68 4D454D00               <1> Cmd_Mem:    db "MEM", 0
00012F6C 00                     <1>             db 0
00012F6D 46494E4400             <1> Cmd_Find:   db "FIND", 0
00012F72 4543484F00             <1> Cmd_Echo:   db "ECHO", 0
00012F77 2A00                   <1> Cmd_Remark: db "*", 0
00012F79 3F00                   <1> Cmd_Help:   db "?", 0
00012F7B 44455649434500         <1> Cmd_Device: db "DEVICE", 0
00012F82 4445564C49535400       <1> Cmd_DevList:        db "DEVLIST", 0
00012F8A 434844495200           <1> Cmd_Chdir:  db "CHDIR", 0
00012F90 4245455000             <1> Cmd_Beep:   db "BEEP", 0
                                <1>
00012F95 00                     <1>             db 0
                                <1>
                                <1> ; 15/02/2016 (FILE.ASM, 09/10/2011)
                                <1> invalid_fname_chars:
00012F96 222728292A2B2C2F       <1>             db 22h, 27h, 28h, 29h, 2Ah, 2Bh, 2Ch, 2Fh
00012F9E 3A3B3C3D3E3F40         <1>             db 3Ah, 3Bh, 3Ch, 3Dh, 3Eh, 3Fh, 40h
00012FA5 5B5C5D5E60             <1>             db 5Bh, 5Ch, 5Dh, 5Eh, 60h
                                <1> sizeInvFnChars  equ ($ - invalid_fname_chars)
                                <1> ;
                                <1>
                                <1> Msg_Enter_Date:
00012FAA 456E746572206E6577-    <1>                 db 'Enter new date (dd-mm-yy): '
00012FB3 206461746520286464-    <1>
00012FBC 2D6D6D2D7979293A20     <1>
00012FC5 00                     <1>                 db 0
                                <1> Msg_Show_Date:
00012FC6 43757272656E742064-    <1>                 db   'Current date is '
00012FCF 61746520697320         <1>
00012FD6 30                     <1> Day:            db   '0'
00012FD7 30                     <1>             db   '0'
00012FD8 2F                     <1>                 db   '/'
00012FD9 30                     <1> Month:          db   '0'
00012FDA 30                     <1>             db   '0'
00012FDB 2F                     <1>                 db   '/'
00012FDC 30                     <1> Century:        db   '0'
00012FDD 30                     <1>                 db   '0'
00012FDE 30                     <1> Year:           db   '0'
00012FDF 30                     <1>             db   '0'
00012FE0 0D0A00                 <1>                 db   0Dh, 0Ah, 0
                                <1>
                                <1> Msg_Enter_Time:
00012FE3 456E746572206E6577-    <1>             db 'Enter new time: '
00012FEC 2074696D653A20         <1>
00012FF3 00                     <1>             db 0
                                <1> Msg_Show_Time:
00012FF4 43757272656E742074-    <1>             db   'Current time is '
00012FFD 696D6520697320         <1>
00013004 30                     <1> Hour:           db   '0'
00013005 30                     <1>             db   '0'
00013006 3A                     <1>             db   ':'
00013007 30                     <1> Minute:         db   '0'
00013008 30                     <1>             db   '0'
```

```
00013009 3A                  <1>             db   ':'
0001300A 30                  <1> Second:         db   '0'
0001300B 30                  <1>             db   '0'
0001300C 0D0A00              <1>             db   0Dh, 0Ah, 0
                             <1>
                             <1> ;VolSize_Unit1:   dd 0
                             <1> ;VolSize_Unit2:   dd 0
                             <1>
                             <1> VolSize_KiloBytes:
0001300F 206B696C6F62797465- <1>             db " kilobytes", 0Dh, 0Ah, 0
00013018 730D0A00            <1>
                             <1> VolSize_Bytes:
0001301C 2062797465730D0A00  <1>             db " bytes", 0Dh, 0Ah, 0
                             <1> Volume_in_drive:
00013025 0D0A                <1>             db 0Dh, 0Ah
                             <1> Vol_FS_Name:
00013027 54522046533120      <1>             db "TR FS1 "
0001302E 566F6C756D6520696E- <1>             db "Volume in drive "
00013037 20647269766520      <1>
0001303E 30                  <1> Vol_Drv_Name:   db 30h
0001303F 3A                  <1>             db ":"
00013040 20697320            <1>             db " is "
00013044 0D0A00              <1>             db 0Dh, 0Ah, 0
                             <1> Dir_Drive_Str:
00013047 54522D444F53204472- <1>                 db "TR-DOS Drive "
00013050 69766520            <1>
                             <1> Dir_Drive_Name:
00013054 303A                <1>                 db "0:"
00013056 0D0A                <1>                 db  0Dh, 0Ah
                             <1> Vol_Str_Header:
00013058 566F6C756D65204E61- <1>                 db "Volume Name: "
00013061 6D653A20            <1>
                             <1> Vol_Name:
00013065 00<rep 40h>         <1>             times 64 db 0
000130A5 00                  <1>             db 0
                             <1> Vol_Serial_Header:
000130A6 0D0A                <1>             db 0Dh, 0Ah
000130A8 566F6C756D65205365- <1>             db "Volume Serial No: "
000130B1 7269616C204E6F3A20  <1>
                             <1> Vol_Serial1:
000130BA 30303030            <1>             db "0000"
000130BE 2D                  <1>             db "-"
                             <1> Vol_Serial2:
000130BF 30303030            <1>             db "0000"
000130C3 0D0A00              <1>             db 0Dh, 0Ah, 0
                             <1>
                             <1> ;Vol_Tot_Sec_Str_Start:
                             <1> ;           dd 0
                             <1> Vol_Total_Sector_Header:
000130C6 0D0A                <1>             db 0Dh, 0Ah
000130C8 566F6C756D65205369- <1>             db "Volume Size : ", 0
000130D1 7A65203A2000        <1>
                             <1> ;Vol_Tot_Sec_Str:
                             <1> ;           db "0000000000"
                             <1> ;Vol_Tot_Sec_Str_End:
                             <1> ;           db 0
                             <1> ;Vol_Free_Sectors_Str_Start:
                             <1> ;           dd 0
                             <1> Vol_Free_Sectors_Header:
000130D7 467265652053706163- <1>             db "Free Space  : ", 0
000130E0 6520203A2000        <1>
                             <1> ;Vol_Free_Sectors_Str:
                             <1> ;           db "0000000000"
                             <1> ;Vol_Free_Sectors_Str_End:
                             <1> ;           db 0
                             <1>
                             <1> Dir_Str_Header:
000130E6 4469726563746F7279- <1>                 db "Directory: "
000130EF 3A20                <1>
000130F1 2F                  <1> Dir_Str_Root:   db "/"
000130F2 00<rep 40h>         <1> Dir_Str:        times 64 db 0
00013132 00000000            <1>                 dd 0
00013136 00                  <1>                 db 0
                             <1>
                             <1> Msg_Bad_Command:
00013137 42616420636F6D6D61- <1>                 db "Bad command or file name!"
00013140 6E64206F722066696C- <1>
00013149 65206E616D6521      <1>
00013150 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> msgl_drv_not_ready:
00013153 070D0A              <1>             db 07h, 0Dh, 0Ah
                             <1>
                             <1> ; CMD_INTR.ASM - 09/11/2011 - Messages
                             <1>
                             <1> Msg_Not_Ready_Read_Err:
00013156 4472697665206E6F74- <1>                 db "Drive not ready or read error!"
0001315F 207265616479206F72- <1>
00013168 207265616420657272- <1>
00013171 6F7221              <1>
00013174 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_Not_Ready_Write_Err:
00013177 4472697665206E6F74- <1>                 db "Drive not ready or write error!"
00013180 207265616479206F72- <1>
00013189 207772697465206572- <1>
00013192 726F7221            <1>
00013196 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_Dir_Not_Found:
00013199 4469726563746F7279- <1>                 db "Directory not found!"
000131A2 206E6F7420666F756E- <1>
000131AB 6421                <1>
000131AD 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_File_Not_Found:
000131B0 46696C65206E6F7420- <1>                 db "File not found!"
000131B9 666F756E6421        <1>
000131BF 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_File_Directory_Not_Found:
000131C2 46696C65206F722064- <1>                 db "File or directory not found!"
000131CB 6972656374 6F727920- <1>
000131D4 6E6F7420666F756E64- <1>
000131DD 21                  <1>
000131DE 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_LongName_Not_Found:
000131E1 4C6F6E67206E616D65- <1>                 db "Long name not found!"
000131EA 206E6F7420666F756E- <1>
000131F3 6421                <1>
000131F5 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> beep_Insufficient_Memory: ; 20/02/2017
```

```
000131F8 0D0A                <1>             db 0Dh, 0Ah
000131FA 07                  <1>             db 07h
                             <1> Msg_Insufficient_Memory:
000131FB 496E73756666696369- <1>                 db "Insufficient memory!"
00013204 656E74206D656D6F72- <1>
0001320D 7921                <1>
0001320F 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_Error_Code:
00013212 436F6D6D616E642066- <1>                 db 'Command failed! Error code : '
0001321B 61696C656421204572- <1>
00013224 726F7220636F646520- <1>
0001322D 3A20                <1>
0001322F 303068              <1> error_code_hex: db '00h'
00013232 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> ; 19/12/2025
                             <1> ;align 2
                             <1>
                             <1> ; 10/02/2016
                             <1> ; DIR.ASM - 09/10/2011
                             <1>
00013235 3C4449523E20202020- <1> Type_Dir:       db '<DIR>     ' ; 10 bytes
0001323E 20                  <1>
                             <1>
                             <1> File_Name:
0001323F 20<rep Ch>          <1>                 times 12 db 20h
0001324B 20                  <1>             db 20h
                             <1> Dir_Or_FileSize:
0001324C 20<rep Ah>          <1>                 times 10 db 20h
00013256 20                  <1>             db 20h
                             <1> File_Attribute:
00013257 20202020            <1>             dd 20202020h
0001325B 20                  <1>             db 20h
                             <1> File_Day:
0001325C 3030                <1>                 db '0','0'
0001325E 2F                  <1>             db '/'
                             <1> File_Month:
0001325F 3030                <1>                 db '0','0'
00013261 2F                  <1>             db '/'
                             <1> File_Year:
00013262 30303030            <1>                 db '0','0','0','0'
00013266 20                  <1>             db 20h
                             <1> File_Hour:
00013267 3030                <1>                 db '0','0'
00013269 3A                  <1>             db ':'
                             <1> File_Minute:
0001326A 3030                <1>                 db '0','0'
0001326C 00                  <1>             db 0
                             <1>
                             <1> Decimal_File_Count_Header:
0001326D 0D0A                <1>             db 0Dh, 0Ah
                             <1> Decimal_File_Count:
0001326F 00<rep 6h>          <1>             times 6 db 0
                             <1>
00013275 2066696C6528732920- <1> str_files:  db " file(s) & "
0001327E 2620                <1>
                             <1> Decimal_Dir_Count:
00013280 00<rep 6h>          <1>             times 6 db 0
                             <1> str_dirs:
00013286 206469726563746F72- <1>             db " directory(s) "
0001328F 7928732920          <1>
00013294 0D0A00              <1>             db 0Dh, 0Ah, 0
                             <1>
00013297 206279746528732920- <1> str_bytes:  db " byte(s) in file(s)"
000132A0 696E2066696C652873- <1>
000132A9 29                  <1>
000132AA 0D0A00              <1>             db 0Dh, 0Ah, 0
                             <1>
                             <1> ; CMD_INTR.ASM - 09/11/2011
                             <1> ; 07/10/2010
                             <1> Msg_invalid_name_chars:
000132AD 496E76616C69642066- <1>                 db "Invalid file or directory name characters!"
000132B6 696C65206F72206469- <1>
000132BF 726563746F7279206E- <1>
000132C8 616D65206368617261- <1>
000132D1 637465727321        <1>
000132D7 0D0A00              <1>             db 0Dh, 0Ah, 0
                             <1> ; 21/02/2016
000132DA 46696C65206F722064- <1> Msg_Name_Exists: db "File or directory name exists!"
000132E3 69726563746F727920- <1>
000132EC 6E616D652065786973- <1>
000132F5 747321              <1>
000132F8 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1> Msg_DoYouWantMkdir:
000132FB 446F20796F75207761- <1>                 db "Do you want to make directory ", 0
00013304 6E7420746F206D616B- <1>
0001330D 65206469726563746F- <1>
00013316 72792000            <1>
0001331A 2028592F4E29203F20- <1> Msg_YesNo:      db " (Y/N) ? ", 0
00013323 00                  <1>
00013324 000D0A00            <1> Y_N_nextline:      db 0, 0Dh, 0Ah, 0
00013328 4F4B2E0D0A00        <1> Msg_OK:            db "OK.", 0Dh, 0Ah, 0
                             <1>
                             <1> ; 27/02/2016
                             <1> Msg_DoYouWantRmDir:
0001332E 446F20796F75207761- <1>                 db "Do you want to delete directory ", 0
00013337 6E7420746F2064656C- <1>
00013340 657465206469726563- <1>
00013349 746F72792000        <1>
                             <1> Msg_Dir_Not_Empty:
0001334F 4469726563746F7279- <1>                 db "Directory not empty!"
00013358 206E6F7420656D7074- <1>
00013361 7921                <1>
00013363 0D0A00              <1>                 db 0Dh, 0Ah, 0
                             <1>
                             <1> Msg_DoYouWantDelete:
00013366 446F20796F75207761- <1>                 db "Do you want to delete file ",0
0001336F 6E7420746F2064656C- <1>
00013378 6574652066696C6520- <1>
00013381 00                  <1>
                             <1>
00013382 44656C657465642E2E- <1> Msg_Deleted:    db "Deleted...", 0Dh, 0Ah, 0
0001338B 2E0D0A00            <1>
                             <1>
                             <1> Msg_Permission_Denied:
0001338F 07                  <1>                 db 7
00013390 5065726D697373696F- <1>                 db "Permission denied!", 0Dh, 0Ah, 0
00013399 6E2064656E69656421- <1>
000133A2 0D0A00              <1>
                             <1>
                             <1> ; 04/03/2016
000133A5 4E657720            <1> Msg_New:        db "New "
000133A9 00                  <1>                 db 0
```

```
                                <1> Str_Attributes:
000133AA 417474726962757465-    <1>                 db "Attributes : "
000133B3 73203A20               <1>
000133B7 4E4F524D414C           <1> Attr_Chars:     db "NORMAL"
000133BD 00                     <1>                 db 0
                                <1>
                                <1> ; 06/03/2016
                                <1> ; CMD_INTR.ASM - 16/11/2010
                                <1> Msg_DoYouWantRename:
000133BE 446F20796F75207761-    <1>                 db "Do you want to rename ", 0
000133C7 6E7420746F2072656E-    <1>
000133D0 616D652000             <1>
000133D5 66696C652000           <1> Rename_File:    db "file ", 0
000133DB 6469726563746F7279-    <1> Rename_Directory: db "directory ", 0
000133E4 2000                   <1>
000133E6 00<rep Dh>             <1> Rename_OldName: times 13 db 0
000133F3 20617320               <1> Msg_File_rename_as: db " as "
000133F7 00<rep Dh>             <1> Rename_NewName: times 13 db 0
                                <1>
                                <1> ; 08/03/2016
                                <1> ; CMD_INTR.ASM - 01/08/2010 - 23/04/2011
                                <1> msg_not_same_drv:
00013404 4E6F742073616D6520-    <1>                 db "Not same drive!"
0001340D 647269766521           <1>
00013413 0D0A00                 <1>                 db 0Dh, 0Ah, 0
                                <1>
                                <1> Msg_DoYouWantMoveFile:
00013416 446F20796F75207761-    <1>                 db "Do you want to move file", 0
0001341F 6E7420746F206D6F76-    <1>
00013428 652066696C6500         <1>
                                <1>
                                <1> msg_insufficient_disk_space:
0001342F 496E73756666696369-    <1>                 db "Insufficient disk space!"
00013438 656E74206469736B20-    <1>
00013441 737061636521           <1>
00013447 0D0A00                 <1>                 db 0Dh, 0Ah, 0
                                <1>
                                <1> ; 01/08/2010
                                <1> msg_source_file:
0001344A 0D0A536F7572636520-    <1>             db 0Dh, 0Ah, "Source file name      :   "
00013453 66696C65206E616D65-    <1>
0001345C 2020202020203A2020-    <1>
00013465 20                     <1>
                                <1> msg_source_file_drv:
00013466 203A00                 <1>             db " :", 0
                                <1> msg_destination_file:
00013469 0D0A44657374696E61-    <1>             db 0Dh, 0Ah, "Destination file name :   "
00013472 74696F6E2066696C65-    <1>
0001347B 206E616D65203A2020-    <1>
00013484 20                     <1>
                                <1> msg_destination_file_drv:
00013485 203A00                 <1>             db " :", 0
                                <1> msg_copy_nextline:
00013488 0D0A00                 <1>             db 0Dh, 0Ah, 0
                                <1>
                                <1> ; 15/03/2016
                                <1> ; CMD_INTR.ASM
                                <1>
                                <1> Msg_DoYouWantOverWriteFile:
0001348B 446F20796F75207761-    <1>                 db "Do you want to overwrite file ",0
00013494 6E7420746F206F7665-    <1>
0001349D 727772697465206669-    <1>
000134A6 6C652000               <1>
                                <1>
                                <1> Msg_DoYouWantCopyFile:
000134AA 446F20796F75207761-    <1>                 db "Do you want to copy file",0
000134B3 6E7420746F20636F70-    <1>
000134BC 792066696C6500         <1>
                                <1>
                                <1> Msg_read_file_error_before_EOF:
000134C3 46696C652072656164-    <1>             db "File reading error! (before EOF)"
000134CC 696E67206572726F72-    <1>
000134D5 2120286265666F7265-    <1>
000134DE 20454F4629             <1>
000134E3 0A0A00                 <1>             db 0Ah, 0Ah, 0
                                <1>
                                <1> ; 18/03/2016
                                <1> ; TRDOS 386 (v2.0) mainprog copy procedure
                                <1> msg_reading:
000134E6 52656164696E672E2E-    <1>             db "Reading... ", 0
000134EF 2E2000                 <1>
                                <1> msg_writing:
000134F2 57726974696E672E2E-    <1>             db "Writing... ", 0
000134FB 2E2000                 <1>
                                <1> percentagestr:
000134FE 2020202500             <1>             db "   %", 0  ; "  0%" .. "100%"
                                <1> ; 11/04/2016
                                <1> Msg_No_Set_Space:
00013503 496E73756666696369-    <1>                 db "Insufficient environment space!"
0001350C 656E7420656E766972-    <1>
00013515 6F6E6D656E74207370-    <1>
0001351E 61636521               <1>
00013522 0D0A00                 <1>                 db 0Dh, 0Ah, 0
                                <1> ; 18/04/2016
                                <1> isc_msg:
00013525 0D0A                   <1>             db 0Dh, 0Ah
00013527 494E56414C49442053-    <1>             db "INVALID SYSTEM CALL", 0
00013530 595354454D2043414C-    <1>
00013539 4C00                   <1>
                                <1> usi_msg:
0001353B 0D0A                   <1>             db 0Dh, 0Ah
0001353D 554E444546494E4544-    <1>             db "UNDEFINED SOFTWARE INTERRUPT", 0
00013546 20534F465457415245-    <1>
0001354F 20494E544552525550-    <1>
00013558 5400                   <1>
                                <1> ifc_msg:
0001355A 0D0A                   <1>             db 0Dh, 0Ah
0001355C 494E56414C49442046-    <1>             db "INVALID FUNCTION CALL"
00013565 554E4354494F4E2043-    <1>
0001356E 414C4C                 <1>
                                <1> inv_msg_for_trdos_v2:
00013571 20                     <1>             db 20h
00013572 666F72205452444F53-    <1>             db "for TRDOS v2!"
0001357B 20763221               <1>
0001357F 07                     <1>             db 07h
00013580 0D0A                   <1>             db 0Dh, 0Ah
00013582 0D0A                   <1>             db 0Dh, 0Ah
00013584 494E5420               <1>             db "INT "
00013588 303068                 <1> int_num_str:        db "00h"
0001358B 0D0A                   <1>             db 0Dh, 0Ah
0001358D 454158203A20           <1>             db "EAX : "
00013593 303030303030303068-    <1> eax_str:    db "00000000h", 0Dh, 0Ah
0001359C 0D0A                   <1>
0001359E 454950203A20           <1>             db "EIP : "
```

```
000135A4 303030303030303068- <1> eip_str:   db "00000000h", 0Dh, 0Ah, 0
000135AD 0D0A00              <1>
                             <1>
                             <1> ; 17/04/2021
                             <1> ; ('KDEV' device parameters are disabled as temporary)
                             <1>
                             <1> ; 07/10/2016
                             <1> ; Device names & parameters (for kernel devices)
                             <1>
                             <1> align 2
                             <1> ;KDEV_NAME:
                             <1> ;           db 'TTY',0,0,0,0,0 ; 1
                             <1> ;           db 'MEM',0,0,0,0,0 ; 2
                             <1> ;           db 'FD0',0,0,0,0,0 ; 3
                             <1> ;           db 'FD1',0,0,0,0,0 ; 4
                             <1> ;           db 'HD0',0,0,0,0,0 ; 5
                             <1> ;           db 'HD1',0,0,0,0,0 ; 6
                             <1> ;           db 'HD2',0,0,0,0,0 ; 7
                             <1> ;           db 'HD3',0,0,0,0,0 ; 8
                             <1> ;           db 'LPT',0,0,0,0,0 ; 9
                             <1> ;           db 'TTY0',0,0,0,0 ; 10
                             <1> ;           db 'TTY1',0,0,0,0 ; 11
                             <1> ;           db 'TTY2',0,0,0,0 ; 12
                             <1> ;           db 'TTY3',0,0,0,0 ; 13
                             <1> ;           db 'TTY4',0,0,0,0 ; 14
                             <1> ;           db 'TTY5',0,0,0,0 ; 15
                             <1> ;           db 'TTY6',0,0,0,0 ; 16
                             <1> ;           db 'TTY7',0,0,0,0 ; 17
                             <1> ;           db 'TTY8',0,0,0,0 ; 18
                             <1> ;           db 'TTY9',0,0,0,0 ; 19
                             <1> ;           db 'COM1',0,0,0,0 ; 18
                             <1> ;           db 'COM2',0,0,0,0 ; 19
                             <1> ;           ;db 'CONSOLE',0         ; 1
                             <1> ;           ;db 'PRINTER',0   ; 9
                             <1> ;           ;db 'CDROM'      ; 20
                             <1> ;           ;db 'CDROM0'     ; 20
                             <1> ;           ;db 'CDROM1'     ; 21
                             <1> ;           ;db 'DVD'        ; 22
                             <1> ;           ;db 'DVD0'       ; 23
                             <1> ;           ;db 'DVD1'       ; 24
                             <1> ;           ;db 'USB'        ; 24
                             <1> ;           ;db 'USB0'       ; 25
                             <1> ;           ;db 'USB1'       ; 26
                             <1> ;           ;db 'USB2'        ; 27
                             <1> ;           ;db 'USB3'       ; 1
                             <1> ;           ;db 'KEYBOARD'   ; 28
                             <1> ;           ;db 'MOUSE'      ; 29
                             <1> ;           ;db 'SOUND'      ; 30
                             <1> ;           ;db 'VGA',0,0,0,0 ; 31
                             <1> ;           ;db 'CGA',0,0,0,0 ; 29
                             <1> ;           ;db 'AUDIO',0,0,0 ; 32
                             <1> ;           ;db 'VIDEO',0,0,0 ; 33
                             <1> ;           ;db 'MUSIC',0,0,0 ; 34
                             <1> ;           ;db 'ETHERNET'   ; 35
                             <1> ;           ;db 'SD0',0,0,0,0,0 ; 36
                             <1> ;           ;db 'SD1',0,0,0,0,0 ; 37
                             <1> ;           ;db 'SD2',0,0,0,0,0 ; 38
                             <1> ;           ;db 'SD3',0,0,0,0,0 ; 35
                             <1> ;           ;db 'SATA0'      ; 36
                             <1> ;           ;db 'SATA1'      ; 37
                             <1> ;           ;db 'SATA2'        ; 38
                             <1> ;           ;db 'SATA3'        ; 5
                             <1> ;           ;db 'PATA0',0,0,0  ; 6
                             <1> ;           ;db 'PATA1',0,0,0  ; 7
                             <1> ;           ;db 'PATA2',0,0,0  ; 8
                             <1> ;           ;db 'PATA3',0,0,0  ; 39
                             <1> ;           ;db 'WIRELESS'   ; 40
                             <1> ;           ;db 'HDMI',0,0,0,0 ; 0
                             <1> ;           db 'NULL',0,0,0,0 ; 0
                             <1>
                             <1> ;NumOfKernelDevNames equ ($-KDEV_NAME) / 8 ; 20 (07/10/2016)
                             <1>
                             <1> ;KDEV_NUMBER:
                             <1> ;           db 1,2,3,4,5,6,7,8,9
                             <1> ;           db 10,11,12,13,14,15,16,17,18,19
                             <1> ;           db 18,19,0
                             <1>
                             <1> ;NumOfKernelDevices equ $ - KDEV_NUMBER
                             <1>
                             <1> ;KDEV_OADDR:
                             <1> ;           dd otty ;tty  ; 1
                             <1> ;           dd sret ;mem  ; 2
                             <1> ;           dd sret ;fd0  ; 3
                             <1> ;           dd sret ;fd1  ; 4
                             <1> ;           dd sret ;hd0  ; 5
                             <1> ;           dd sret ;hd1  ; 6
                             <1> ;           dd sret ;hd2  ; 7
                             <1> ;           dd sret ;hd3  ; 8
                             <1> ;           dd sret ;lpt  ; 9
                             <1> ;           dd ocvt ;tty0 ; 10
                             <1> ;           dd ocvt ;tty1 ; 11
                             <1> ;           dd ocvt ;tty2 ; 12
                             <1> ;           dd ocvt ;tty3 ; 13
                             <1> ;           dd ocvt ;tty4 ; 14
                             <1> ;           dd ocvt ;tty5 ; 15
                             <1> ;           dd ocvt ;tty6 ; 16
                             <1> ;           dd ocvt ;tty7 ; 17
                             <1> ;           dd ocvt ;tty8 ; 18
                             <1> ;           dd ocvt ;tty9 ; 19
                             <1> ;           ;dd ocvt ;com1 ; 18
                             <1> ;           ;dd ocvt ;com2 ; 19
                             <1> ;           dd sret ;null ; 20
                             <1> ;KDEV_CADDR:
                             <1> ;           dd ctty ;tty  ; 1
                             <1> ;           dd cret ;mem  ; 2
                             <1> ;           dd cret ;fd0  ; 3
                             <1> ;           dd cret ;fd1  ; 4
                             <1> ;           dd cret ;hd0  ; 5
                             <1> ;           dd cret ;hd1  ; 6
                             <1> ;           dd cret ;hd2  ; 7
                             <1> ;           dd cret ;hd3  ; 8
                             <1> ;           dd cret ;lpt  ; 9
                             <1> ;           dd ocvt ;tty0 ; 10
                             <1> ;           dd ccvt ;tty1 ; 11
                             <1> ;           dd ccvt ;tty2 ; 12
                             <1> ;           dd ccvt ;tty3 ; 13
                             <1> ;           dd ccvt ;tty4 ; 14
                             <1> ;           dd ccvt ;tty5 ; 15
                             <1> ;           dd ccvt ;tty6 ; 16
                             <1> ;           dd ccvt ;tty7 ; 17
                             <1> ;           dd ccvt ;tty8 ; 18
                             <1> ;           dd ccvt ;tty9 ; 19
                             <1> ;           ;dd ccvt ;com1 ; 18
                             <1> ;           ;dd ccvt ;com2 ; 19
```

```
                                <1> ;           dd cret ;null ; 20
                                <1> ;
                                <1> ;KDEV_RADDR:
                                <1> ;           dd rtty ;tty  ; 1
                                <1> ;           dd rmem ;mem  ; 2
                                <1> ;           dd rfd  ;fd0  ; 3
                                <1> ;           dd rfd  ;fd1  ; 4
                                <1> ;           dd rhd  ;hd0  ; 5
                                <1> ;           dd rhd  ;hd1  ; 6
                                <1> ;           dd rhd  ;hd2  ; 7
                                <1> ;           dd rhd  ;hd3  ; 8
                                <1> ;           dd rlpt ;lpt  ; 9
                                <1> ;           dd rcvt ;tty0 ; 10
                                <1> ;           dd rcvt ;tty1 ; 11
                                <1> ;           dd rcvt ;tty2 ; 12
                                <1> ;           dd rcvt ;tty3 ; 13
                                <1> ;           dd rcvt ;tty4 ; 14
                                <1> ;           dd rcvt ;tty5 ; 15
                                <1> ;           dd rcvt ;tty6 ; 16
                                <1> ;           dd rcvt ;tty7 ; 17
                                <1> ;           dd rcvt ;tty8 ; 18
                                <1> ;           dd rcvt ;tty9 ; 19
                                <1> ;           ;dd rcvt ;com1 ; 18
                                <1> ;           ;dd rcvt ;com2 ; 19
                                <1> ;           dd rnull ;null ; 20
                                <1> ;KDEV_WADDR:
                                <1> ;           dd wtty ;tty  ; 1
                                <1> ;           dd wmem ;mem  ; 2
                                <1> ;           dd wfd  ;fd0  ; 3
                                <1> ;           dd wfd  ;fd1  ; 4
                                <1> ;           dd whd  ;hd0  ; 5
                                <1> ;           dd whd  ;hd1  ; 6
                                <1> ;           dd whd  ;hd2  ; 7
                                <1> ;           dd whd  ;hd3  ; 8
                                <1> ;           dd wlpt ;lpt  ; 9
                                <1> ;           dd xmtt ;tty0 ; 10
                                <1> ;           dd xmtt ;tty1 ; 11
                                <1> ;           dd xmtt ;tty2 ; 12
                                <1> ;           dd xmtt ;tty3 ; 13
                                <1> ;           dd xmtt ;tty4 ; 14
                                <1> ;           dd xmtt ;tty5 ; 15
                                <1> ;           dd xmtt ;tty6 ; 16
                                <1> ;           dd xmtt ;tty7 ; 17
                                <1> ;           dd xmtt ;tty8 ; 18
                                <1> ;           dd xmtt ;tty9 ; 19
                                <1> ;           ;dd xmtt ;com1 ; 18
                                <1> ;           ;dd xmtt ;com2 ; 19
                                <1> ;           dd wnull ;null ; 20
                                <1>
                                <1> ; DEV_ACCESS bits:
                                <1>     ; bit 0 = accessable by normal users
                                <1>     ; bit 1 = read access permission
                                <1>     ; bit 2 = write access permission
                                <1>     ; bit 3 = IOCTL permission to users
                                <1>     ; bit 4 = block device if it is set
                                <1>     ; bit 5 = 16 bit or 1024 byte data
                                <1>     ; bit 6 = 32 bit or 2048 byte data
                                <1>     ; bit 7 = installable device driver
                                <1>
                                <1> ;KDEV_ACCESS: ; 08/10/2016
                                <1> ;           db  00000111b  ; tty, 1
                                <1> ;           db  00000111b  ; mem, 2
                                <1> ;           db  10001111b  ; fd0, 3
                                <1> ;           db  10001111b  ; fd1, 4
                                <1> ;           db  10001111b  ; hd0, 5
                                <1> ;           db  10001111b  ; hd1, 6
                                <1> ;           db  10001111b  ; hd2, 7
                                <1> ;           db  10001111b  ; hd3, 8
                                <1> ;           db  00000111b   ; lpt, 9
                                <1> ;           db  00000111b  ; tty0, 10
                                <1> ;           db  00000111b  ; tty1, 11
                                <1> ;           db  00000111b  ; tty2, 12
                                <1> ;           db  00000111b  ; tty3, 13
                                <1> ;           db  00000111b  ; tty4, 14
                                <1> ;           db  00000111b  ; tty5, 15
                                <1> ;           db  00000111b  ; tty6, 16
                                <1> ;           db  00000111b  ; tty7, 17
                                <1> ;           db  00000111b  ; tty8, 18
                                <1> ;           db  00000111b  ; tty9, 19
                                <1> ;           ;db 00000111b  ; com1, 18
                                <1> ;           ;db 00000111b  ; com2, 19
                                <1> ;           db  00000000b   ; null, 0
                                <1>
                                <1> ; 07/10/2016
                                <1> ;NumOfInstallableDevices equ 8
                                <1> ;NUMIDEV    equ NumOfInstallableDevices ; 8
                                <1> ;NUMOFDEVICES      equ NumOfKernelDevices + NumOfInstallableDevices
                                <1>
                                <1> ; 26/02/2017
                                <1> ; IRQ Callback (& Signal Response Byte) service availibity
                                <1> ; 'syscalbac'
                                <1> ; ***************************************************
                                <1> ; IRQ 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
                                <1> ; --- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
                                <1> ; --- 00 00 00 01 02 03 00 04 00 05 06 07 08 09 00 00
                                <1> ; ***************************************************
                                <1> IRQenum:
 000135B0 000000010203000400-   <1>     db  0,0,0,1,2,3,0,4,0,5,6,7,8,9,0,0
 000135B9 05060708090000        <1>
                                <1>
                                <1> ; 28/08/2017
                                <1> ; 20/08/2017
                                <1> ; DMA Registers (for 'sysdma')
                                <1> ; 02/07/2017 (sb16mod.s)
 000135C0 00020406C0C4C8CC      <1> dma_adr:    db 0,2,4,6,0C0h,0C4h,0C8h,0CCh
 000135C8 01030507C2C6CACE      <1> dma_cnt:    db 1,3,5,7,0C2h,0C6h,0CAh,0CEh
 000135D0 878381828F8B898A      <1> dma_page:   db 87h,83h,81h,82h,8Fh,8Bh,89h,8Ah ; 03/08/2017
 000135D8 0A0A0A0AD4D4D4D4      <1> dma_mask:   db 0Ah,0Ah,0Ah,0Ah,0D4h,0D4h,0D4h,0D4h
 000135E0 0B0B0B0BD6D6D6D6      <1> dma_mod:    db 0Bh,0Bh,0Bh,0Bh,0D6h,0D6h,0D6h,0D6h
 000135E8 0C0C0C0CD8D8D8D8      <1> dma_flip:   db 0Ch,0Ch,0Ch,0Ch,0D8h,0D8h,0D8h,0D8h

                                    ; 27/08/2014
                                    scr_row:
 000135F0 E0810B00                      dd 0B8000h + 0A0h + 0A0h + 0A0h ; Row 3
                                    scr_col:
 000135F4 00000000                      dd 0

                                    Align 4
                                        ; 15/04/2016
                                        ; TRDOS 386 (TRDOS v2.0)

                                        ; 21/08/2014
                                    ilist:
                                        ;times 32 dd cpu_except ; INT 0 to INT 1Fh
```

```
                                        ;
                                        ; Exception list
                                        ; 25/08/2014
000135F8 [140C0000]                     dd      exc0    ; 0h,  Divide-by-zero Error
000135FC [1B0C0000]                     dd      exc1
00013600 [220C0000]                     dd      exc2
00013604 [290C0000]                     dd      exc3
00013608 [2D0C0000]                     dd      exc4
0001360C [310C0000]                     dd      exc5
00013610 [350C0000]                     dd      exc6    ; 06h,  Invalid Opcode
00013614 [390C0000]                     dd      exc7
00013618 [3D0C0000]                     dd      exc8
0001361C [410C0000]                     dd      exc9
00013620 [450C0000]                     dd      exc10
00013624 [490C0000]                     dd      exc11
00013628 [4D0C0000]                     dd      exc12
0001362C [510C0000]                     dd      exc13   ; 0Dh, General Protection Fault
00013630 [550C0000]                     dd      exc14   ; 0Eh, Page Fault
00013634 [590C0000]                     dd      exc15
00013638 [5D0C0000]                     dd      exc16
0001363C [610C0000]                     dd      exc17
00013640 [650C0000]                     dd      exc18
00013644 [690C0000]                     dd      exc19
00013648 [6D0C0000]                     dd      exc20
0001364C [710C0000]                     dd      exc21
00013650 [750C0000]                     dd      exc22
00013654 [790C0000]                     dd      exc23
00013658 [7D0C0000]                     dd      exc24
0001365C [810C0000]                     dd      exc25
00013660 [850C0000]                     dd      exc26
00013664 [890C0000]                     dd      exc27
00013668 [8D0C0000]                     dd      exc28
0001366C [910C0000]                     dd      exc29
00013670 [950C0000]                     dd      exc30
00013674 [990C0000]                     dd      exc31
                                    IRQ_list: ; 28/02/2017 ('syscalbac')
                                        ; Interrupt list
00013678 [7E090000]                     dd      timer_int       ; INT 20h
                                                ;dd     irq0
0001367C [E3100000]                     dd      kb_int          ; 24/01/2016
                                                ;dd     irq1
00013680 [6A0B0000]                     dd      irq2
                                                ; COM2 int
00013684 [6E0B0000]                     dd      irq3
                                                ; COM1 int
00013688 [790B0000]                     dd      irq4
0001368C [840B0000]                     dd      irq5
                                    ;DISKETTE_INT: ;06/02/2015
00013690 [A44E0000]                     dd      fdc_int         ; 16/02/2015, IRQ 6 handler
                                                ;dd     irq6
                                    ; Default IRQ 7 handler against spurious IRQs (from master PIC)
                                    ; 25/02/2015 (source: http://wiki.osdev.org/8259_PIC)
00013694 [F00E0000]                     dd      default_irq7    ; 25/02/2015
                                                ;dd     irq7
                                    ; Real Time Clock Interrupt
00013698 [F20A0000]                     dd      rtc_int         ; 23/02/2015, IRQ 8 handler
                                                ;dd     irq8    ; INT 28h
0001369C [940B0000]                     dd      irq9
000136A0 [980B0000]                     dd      irq10
000136A4 [9C0B0000]                     dd      irq11
000136A8 [A00B0000]                     dd      irq12
000136AC [A40B0000]                     dd      irq13
                                    ;HDISK_INT1:  ;06/02/2015
000136B0 [E3570000]                     dd      hdc1_int        ; 21/02/2015, IRQ 14 handler
                                                ;dd     irq14
                                    ;HDISK_INT2:  ;06/02/2015
000136B4 [06580000]                     dd      hdc2_int        ; 21/02/2015, IRQ 15 handler
                                                ;dd     irq15   ; INT 2Fh
                                                ; 14/08/2015
                                        ;dd     sysent          ; INT 30h (system calls)

                                        ; 15/04/2016
                                        ; TRDOS 386(TRDOS v2.0) Software Interrupts

000136B8 [15370100]                     dd      int30h          ; Reserved for
                                                                ; !!! Retro UNIX (RUNIX) !!!
                                                                ; !!! SINGLIX !!! System Calls
000136BC [72170000]                     dd      int31h          ; Video BIOS (IBM PC/AT, Int 10h)
000136C0 [3F0F0000]                     dd      int32h          ; Keyboard Functions (IBM PC/AT, Int 16h)
000136C4 [464F0000]                     dd      int33h          ; DISK I/O (IBM PC/AT, Int 13h)
000136C8 [9C1B0100]                     dd      int34h          ; #IOCTL# (I/O port access support for ring 3)
000136CC [F1620000]                     dd      int35h          ; Time/Date Functions (IBM PC/AT, Int 1Ah)
000136D0 [A40D0000]                     dd      ignore_int      ; INT 36h : Timer Functions
000136D4 [A40D0000]                     dd      ignore_int      ; INT 37h
000136D8 [A40D0000]                     dd      ignore_int      ; INT 38h
000136DC [A40D0000]                     dd      ignore_int      ; INT 39h
000136E0 [A40D0000]                     dd      ignore_int      ; INT 3Ah
000136E4 [A40D0000]                     dd      ignore_int      ; INT 3Bh
000136E8 [A40D0000]                     dd      ignore_int      ; INT 3Ch
000136EC [A40D0000]                     dd      ignore_int      ; INT 3Dh
000136F0 [A40D0000]                     dd      ignore_int      ; INT 3Eh
000136F4 [A40D0000]                     dd      ignore_int      ; INT 3Fh
000136F8 [D6C80000]                     dd      sysent          ; INT 40h : !!! TRDOS 386 System Calls !!!
                                        ;dd     ignore_int
000136FC 00000000                       dd      0

                                    ; 20/08/2014
                                      ; /* This is the default interrupt "handler" :-) */
                                      ; Linux v0.12 (head.s)
                                    int_msg:
00013700 556E6B6E6F776E2069-            db "Unknown interrupt ! ", 0
00013709 6E7465727275707420-
00013712 212000

                                    ; 15/04/2016
                                    ; TRDOS 386 (TRDOS v2.0)

                                    ; 29/04/2016
                                    int30h:
                                    trdos_isc_routine:
                                        ; 02/05/2016
                                        ; 01/05/2016
                                        ; 29/04/2016
                                        ; 18/04/2016
                                        ; 15/04/2016 (TRDOS 386 = TRDOS v2.0)
                                        ; 17/04/2011 (TRDOS v1.0, 'IFC.ASM')
                                        ; 03/02/2011 ('trdos_ifc_routine')
                                        ;
00013715 8B1C24                         mov     ebx, [esp] ; EIP (next)
00013718 83EB02                         sub     ebx, 2 ; EIP (CD ##h)

0001371B 89C1                           mov     ecx, eax
0001371D 8A4301                         mov     al, [ebx+1] ; CDh ##h
```

```
00013720 66BA1000                    mov     dx, KDATA
00013724 8EDA                        mov     ds, dx
00013726 8EC2                        mov     es, dx

00013728 FC                          cld
00013729 8B15[B0760100]              mov     edx, [k_page_dir]
0001372F 0F22DA                      mov     cr3, edx

00013732 E8880AFFFF                  call    bytetohex
00013737 66A3[88350100]              mov     [int_num_str], ax

0001373D 89D8                        mov     eax, ebx ; EIP
0001373F E8BB0AFFFF                  call    dwordtohex
00013744 8915[A4350100]              mov     [eip_str], edx
0001374A A3[A8350100]                mov     [eip_str+4], eax

0001374F 89C8                        mov     eax, ecx
00013751 E8A90AFFFF                  call    dwordtohex
00013756 8915[93350100]              mov     [eax_str], edx
0001375C A3[97350100]                mov     [eax_str+4], eax

00013761 43                          inc     ebx
00013762 8A03                        mov     al, [ebx] ; Interrupt number

                                 trdos_isc_handler:
00013764 80FE30                      cmp     dh, 30h ; Retro UNIX, SINGLIX System calls
00013767 7507                        jne     short trdos_usi_handler
00013769 BE[25350100]                mov     esi, isc_msg
0001376E EB05                        jmp     short loc_write_inv_system_call_msg

                                 trdos_usi_handler:
00013770 BE[3B350100]                mov     esi, usi_msg

                                 loc_write_inv_system_call_msg:
00013775 E84036FFFF                  call    print_msg
                                     ; 29/04/2016
0001377A BE[71350100]                mov     esi, inv_msg_for_trdos_v2
0001377F E83636FFFF                  call    print_msg

                                 loc_ifc_terminate_process:
                                     ; u.uno = process number
                                     ; 29/04/2016

                                     ; 02/05/2016
00013784 FE05[608E0100]              inc     byte [sysflg] ; 0FFh -> 0

0001378A B801000000                  mov     eax, 1
0001378F E93894FFFF                  jmp     sysexit

                                 ; 07/03/2015
                                 ; Temporary Code
                                 display_disks:
00013794 803D[8C660000]00            cmp     byte [fd0_type], 0
0001379B 7605                        jna     short ddsks1
0001379D E87D000000                  call    pdskm
                                 ddsks1:
000137A2 803D[8D660000]00            cmp     byte [fd1_type], 0
000137A9 760C                        jna     short ddsks2
000137AB C605[EF380100]31            mov     byte [dskx], '1'
000137B2 E868000000                  call    pdskm
                                 ddsks2:
000137B7 803D[8E660000]00            cmp     byte [hd0_type], 0
000137BE 7654                        jna     short ddsk6
000137C0 66C705[ED380100]68-         mov     word [dsktype], 'hd'
000137C8 64
000137C9 C605[EF380100]30            mov     byte [dskx], '0'
000137D0 E84A000000                  call    pdskm
                                 ddsks3:
000137D5 803D[8F660000]00            cmp     byte [hd1_type], 0
000137DC 7636                        jna     short ddsk6
000137DE C605[EF380100]31            mov     byte [dskx], '1'
000137E5 E835000000                  call    pdskm
                                 ddsks4:
000137EA 803D[90660000]00            cmp     byte [hd2_type], 0
000137F1 7621                        jna     short ddsk6
000137F3 C605[EF380100]32            mov     byte [dskx], '2'
000137FA E820000000                  call    pdskm
                                 ddsks5:
000137FF 803D[91660000]00            cmp     byte [hd3_type], 0
00013806 760C                        jna     short ddsk6
00013808 C605[EF380100]33            mov     byte [dskx], '3'
0001380F E80B000000                  call    pdskm
                                 ddsk6:
00013814 BE[17390100]                mov     esi, nextline
00013819 E806000000                  call    pdskml
                                 pdskm_ok:
0001381E C3                          retn
                                 pdskm:
0001381F BE[EB380100]                mov     esi, dsk_ready_msg
                                 pdskml:
00013824 AC                          lodsb
00013825 08C0                        or      al, al
00013827 74F5                        jz      short pdskm_ok
00013829 56                          push    esi
                                     ; 13/05/2016
0001382A BB07000000                      mov     ebx, 7  ; Black background,
                                                         ; light gray forecolor
                                                         ; Video page 0 (bh=0)
0001382F E8ABEAFEFF                  call    _write_tty
00013834 5E                          pop     esi
00013835 EBED                        jmp     short pdskml

00013837 90                      Align 2
                                     ; 21/08/2014
                                 exc_msg:
00013838 435055206578636570-         db "CPU exception ! "
00013841 74696F6E202120
                                 excnstr:            ; 25/08/2014
00013848 3F3F68202045495020-         db "??h", "  EIP : "
00013851 3A20
                                 EIPstr: ; 29/08/2014
00013853 00<rep Ch>                  times 12 db 0

                                     ; 23/02/2015
                                     ; 25/08/2014
                                 ;scounter:
                                 ;   db 5
                                 ;   db 19

                                 ; 06/11/2014
                                 ; Memory Information message
                                 ; 14/08/2015
                                 msg_memory_info:
0001385F 07                          db      07h
```

```
00013860 0D0A                        db      0Dh, 0Ah
                                     ;db     "MEMORY ALLOCATION INFO", 0Dh, 0Ah, 0Dh, 0Ah
00013862 546F74616C206D656D-         db      "Total memory : "
0001386B 6F7279203A20
                                 mem_total_b_str: ; 10 digits
00013871 303030303030303030-         db      "0000000000 bytes", 0Dh, 0Ah
0001387A 302062797465730D0A
00013883 202020202020202020-         db      "               ", 20h, 20h, 20h
0001388C 202020202020202020
                                 mem_total_p_str: ; 7 digits
00013895 303030303030302070-         db      "0000000 pages", 0Dh, 0Ah
0001389E 616765730D0A
000138A4 0D0A                        db      0Dh, 0Ah
000138A6 46726565206D656D6F-         db      "Free memory  : "
000138AF 727920203A20
                                 free_mem_b_str:  ; 10 digits
000138B5 3F3F3F3F3F3F3F3F3F-         db      "?????????? bytes", 0Dh, 0Ah
000138BE 3F2062797465730D0A
000138C7 202020202020202020-         db      "               ", 20h, 20h, 20h
000138D0 202020202020202020
                                 free_mem_p_str:  ; 7 digits
000138D9 3F3F3F3F3F3F2070-           db      "??????? pages", 0Dh, 0Ah
000138E2 616765730D0A
000138E8 0D0A00                      db      0Dh, 0Ah, 0

                                 dsk_ready_msg:
000138EB 0D0A                        db      0Dh, 0Ah
                                 dsktype:
000138ED 6664                        db      'fd'
                                 dskx:
000138EF 30                          db      '0'
000138F0 20                          db      20h
000138F1 697320524541445920-         db      'is READY ...'
000138FA 2E2E2E
000138FD 00                          db      0

                                 setup_error_msg:
000138FE 0D0A                        db      0Dh, 0Ah
00013900 4469736B2053657475-         db      'Disk Setup Error !'
00013909 70204572726F722021
00013912 0D0A00                      db      0Dh, 0Ah,0

                                 next2line: ; 08/02/2016
00013915 0D0A                        db      0Dh, 0Ah
                                 nextline:
00013917 0D0A00                      db      0Dh, 0Ah, 0

                                 ; temporary
                                 ; 19/12/2020
                                 msg_lfb_addr:
                                     ;db     0Dh, 0Ah
0001391A 4C696E656172206672-         db      "Linear frame buffer at "
00013923 616D65206275666665-
0001392C 7220617420
                                 lfb_addr_str: ; 8 (hex) digits
00013931 303030303030303068-         db      "00000000h", 0Dh, 0Ah
0001393A 0D0A
0001393C 0D0A00                      db      0Dh, 0Ah, 0

                                 ; KERNEL - SYSINIT Messages
                                 ; 24/08/2015
                                 ; 13/04/2015 - (Retro UNIX 386 v1 Beginning)
                                 ; 14/07/2013
                                 ;kernel_init_err_msg:
                                 ;   db 0Dh, 0Ah
                                 ;   db 07h
                                 ;   db 'Kernel initialization ERROR !'
                                 ;   db 0Dh, 0Ah, 0

                                 ;welcome_msg:
                                 ;   db 0Dh, 0Ah
                                 ;   db 07h
                                 ;   db 'Welcome to TRDOS 386 Operating System !'
                                 ;   db 0Dh, 0Ah
                                 ;   db 'by Erdogan Tan - 31/12/2017 (v2.0.0)'
                                 ;   db 0Dh, 0Ah, 0

                                 panic_msg:
0001393F 0D0A07                      db 0Dh, 0Ah, 07h
00013942 4552524F523A204B65-         db 'ERROR: Kernel Panic !'
0001394B 726E656C2050616E69-
00013954 632021
00013957 0D0A00                      db 0Dh, 0Ah, 0

                                 ;msgl_drv_not_ready:
                                 ;   db 07h, 0Dh, 0Ah
                                 ;       db 'Drive not ready or read error !'
                                 ;       db 0Dh, 0Ah, 0

                                 starting_msg:
                                     ;;;;;;;;db "Turkish Rational DOS v2.0 [18/04/2021] ...", 0
                                     ;;;;;;;db "Turkish Rational DOS v2.0 [11/08/2022] ...", 0
                                     ;;;;;;db "Turkish Rational DOS v2.0 [30/08/2023] ...", 0
                                     ;;;;;db "Turkish Rational DOS v2.0 [07/12/2023] ...", 0
                                     ;;;;db "Turkish Rational DOS v2.0 [23/06/2024] ...", 0
                                     ;;;db "Turkish Rational DOS v2.0 [29/12/2024] ...", 0
                                     ;;db "Turkish Rational DOS v2.0 [28/01/2025] ...", 0
                                     ;db "Turkish Rational DOS v2.0 [28/12/2025] ...", 0
0001395A 5475726B6973682052-         db "Turkish Rational DOS v2.0 [10/02/2026] ...", 0
00013963 6174696F6E616C2044-
0001396C 4F532076322E30205B-
00013975 31302F30322F323032-
0001397E 365D202E2E2E2E00

                                 NextLine:
00013985 0D0A00                      db 0Dh, 0Ah, 0

                                 %include 'audio.s' ; 03/04/2017
                             <1> ; ******************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.10 - audio.s
                             <1> ; ------------------------------------------------------------------------------
                             <1> ; Last Update: 28/01/2025  (Previous: 06/06/2024 - Kernel v2.0.9)
                             <1> ; ------------------------------------------------------------------------------
                             <1> ; Beginning: 03/04/2017
                             <1> ; ------------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.15 (trdos386.s)
                             <1> ; ******************************************************************************
                             <1>
                             <1> ; AUDIO CONTROLLER & CODEC DEFINITIONS & CODE FOR TRDOS 386
                             <1>
                             <1> ;==============================================================================
                             <1> ;               EQUATES
                             <1> ;==============================================================================
                             <1>
```

```
                                <1> ; PCI EQUATES
                                <1>
                                <1> BIT0  EQU 1
                                <1> BIT1  EQU 2
                                <1> BIT2  EQU 4
                                <1> BIT3  EQU 8
                                <1> BIT4  EQU 10h
                                <1> BIT5  EQU 20h
                                <1> BIT6  EQU 40h
                                <1> BIT7  EQU 80h
                                <1> BIT8  EQU 100h
                                <1> BIT9  EQU 200h
                                <1> BIT10 EQU 400h
                                <1> BIT11 EQU 800h
                                <1> BIT12 EQU 1000h
                                <1> BIT13 EQU 2000h
                                <1> BIT14 EQU 4000h
                                <1> BIT15 EQU 8000h
                                <1> BIT16 EQU 10000h
                                <1> BIT17 EQU 20000h
                                <1> BIT18 EQU 40000h
                                <1> BIT19 EQU 80000h
                                <1> BIT20 EQU 100000h
                                <1> BIT21 EQU 200000h
                                <1> BIT22 EQU 400000h
                                <1> BIT23 EQU 800000h
                                <1> BIT24 EQU 1000000h
                                <1> BIT25 EQU 2000000h
                                <1> BIT26 EQU 4000000h
                                <1> BIT27 EQU 8000000h
                                <1> BIT28 EQU 10000000h
                                <1> BIT29 EQU 20000000h
                                <1> BIT30 EQU 40000000h
                                <1> BIT31 EQU 80000000h
                                <1> NOT_BIT31 EQU 7FFFFFFFh
                                <1>
                                <1> ; PCI equates
                                <1> ; PCI function address (PFA)
                                <1> ; bit 31 = 1
                                <1> ; bit 23:16 = bus number     (0-255)
                                <1> ; bit 15:11 = device number  (0-31)
                                <1> ; bit 10:8 = function number (0-7)
                                <1> ; bit 7:0 = register number  (0-255)
                                <1>
                                <1> IO_ADDR_MASK    EQU     0FFFEh    ; mask off bit 0 for reading BARs
                                <1> PCI_INDEX_PORT  EQU     0CF8h
                                <1> PCI_DATA_PORT   EQU     0CFCh
                                <1> PCI32           EQU     BIT31     ; bitflag to signal 32bit access
                                <1> PCI16           EQU     BIT30     ; bitflag for 16bit access
                                <1> NOT_PCI32_PCI16    EQU    03FFFFFFFh ; NOT BIT31+BIT30 ; 19/03/2017
                                <1>
                                <1> PCI_FN0         EQU     0 << 8
                                <1> PCI_FN1         EQU     1 << 8
                                <1> PCI_FN2         EQU     2 << 8
                                <1> PCI_FN3         EQU     3 << 8
                                <1> PCI_FN4         EQU     4 << 8
                                <1> PCI_FN5         EQU     5 << 8
                                <1> PCI_FN6         EQU     6 << 8
                                <1> PCI_FN7         EQU     7 << 8
                                <1>
                                <1> PCI_CMD_REG EQU     04h     ; reg 04, command reg
                                <1> IO_ENA              EQU     BIT0    ; i/o decode enable
                                <1> MEM_ENA             EQU     BIT1    ; memory decode enable
                                <1> BM_ENA              EQU      BIT2   ; bus master enable
                                <1>
                                <1> ; VIA VT8233 EQUATES
                                <1>
                                <1> VIA_VID             equ 1106h      ; VIA's PCI vendor ID
                                <1> VT8233_DID      equ 3059h ; VT8233 (VT8235) device ID
                                <1>
                                <1> PCI_IO_BASE          equ 10h
                                <1> AC97_INT_LINE        equ 3Ch
                                <1> VIA_ACLINK_CTRL      equ 41h
                                <1> VIA_ACLINK_STAT      equ 40h
                                <1> VIA_ACLINK_C00_READY equ 01h ; primary codec ready
                                <1>
                                <1> VIA_REG_AC97            equ 80h ; dword
                                <1>
                                <1> VIA_ACLINK_CTRL_ENABLE    equ   80h ; 0: disable, 1: enable
                                <1> VIA_ACLINK_CTRL_RESET     equ   40h ; 0: assert, 1: de-assert
                                <1> VIA_ACLINK_CTRL_SYNC      equ   20h ; 0: release SYNC, 1: force SYNC hi
                                <1> VIA_ACLINK_CTRL_VRA       equ   08h ; 0: disable VRA, 1: enable VRA
                                <1> VIA_ACLINK_CTRL_PCM       equ   04h ; 0: disable PCM, 1: enable PCM
                                <1>                                     ; 3D Audio Channel slots 3/4
                                <1> VIA_ACLINK_CTRL_INIT      equ  (VIA_ACLINK_CTRL_ENABLE +                              VIA_ACLINK_CTRL_RESET +
VIA_ACLINK_CTRL_PCM +                         VIA_ACLINK_CTRL_VRA)
                                <1>
                                <1> CODEC_AUX_VOL             equ   04h
                                <1> VIA_REG_AC97_BUSY  equ   01000000h ;(1<<24)
                                <1> VIA_REG_AC97_CMD_SHIFT    equ   10h ; 16
                                <1> VIA_REG_AC97_PRIMARY_VALID equ 02000000h ;(1<<25)
                                <1> VIA_REG_AC97_READ  equ   00800000h ;(1<<23)
                                <1> VIA_REG_AC97_CODEC_ID_SHIFT   equ  1Eh ; 30
                                <1> VIA_REG_AC97_CODEC_ID_PRIMARY equ  0
                                <1> VIA_REG_AC97_DATA_SHIFT equ   0
                                <1> VIADEV_PLAYBACK         equ   0
                                <1> VIA_REG_OFFSET_STATUS   equ   0    ;; byte - channel status
                                <1> VIA_REG_OFFSET_CONTROL  equ   01h  ;; byte - channel control
                                <1> VIA_REG_CTRL_START equ   80h  ;; WO
                                <1> VIA_REG_CTRL_TERMINATE  equ   40h  ;; WO
                                <1> VIA_REG_CTRL_PAUSE      equ   08h  ;; RW
                                <1> VIA_REG_CTRL_RESET      equ   01h  ;; RW - probably reset? undocumented
                                <1> VIA_REG_OFFSET_STOP_IDX equ   08h  ;; dword - stop index, channel type, sample rate
                                <1> VIA8233_REG_TYPE_16BIT  equ   200000h ;; RW
                                <1> VIA8233_REG_TYPE_STEREO equ   100000h ;; RW
                                <1> VIA_REG_OFFSET_CURR_INDEX equ 0Fh ;; byte - channel current index (for via8233 only)
                                <1> VIA_REG_OFFSET_TABLE_PTR equ  04h  ;; dword - channel table pointer
                                <1> VIA_REG_OFFSET_CURR_PTR equ   04h  ;; dword - channel current pointer
                                <1> VIA_REG_OFS_PLAYBACK_VOLUME_L equ  02h ;; byte
                                <1> VIA_REG_OFS_PLAYBACK_VOLUME_R equ  03h ;; byte
                                <1> VIA_REG_CTRL_AUTOSTART    equ   20h
                                <1> VIA_REG_CTRL_INT_EOL      equ   02h
                                <1> VIA_REG_CTRL_INT_FLAG     equ   01h
                                <1> VIA_REG_CTRL_INT   equ  (VIA_REG_CTRL_INT_FLAG +                              VIA_REG_CTRL_INT_EOL +
VIA_REG_CTRL_AUTOSTART)
                                <1>
                                <1> VIA_REG_STAT_STOP_IDX     equ   10h    ;; RO ; 27/07/2020
                                <1>                                 ; current index = stop index
                                <1> VIA_REG_STAT_STOPPED      equ   04h    ;; RWC
                                <1> VIA_REG_STAT_EOL   equ   02h    ;; RWC
                                <1> VIA_REG_STAT_FLAG  equ   01h    ;; RWC
                                <1> VIA_REG_STAT_ACTIVE       equ   80h    ;; RO
                                <1> ; 28/11/2016
                                <1> VIA_REG_STAT_LAST  equ   40h    ;; RO
```

```
<1> VIA_REG_STAT_TRIGGER_QUEUED equ 08h  ;; RO
<1> VIA_REG_CTRL_INT_STOP     equ   04h  ; Interrupt on Current Index = Stop Index
<1>                                 ; and End of Block
<1>
<1> VIA_REG_OFFSET_CURR_COUNT equ 0Ch ;; dword - channel current count, index
<1>
<1> PORTB              EQU     061h
<1> REFRESH_STATUS     EQU     010h    ; Refresh signal status
<1>
<1> ; AC97 Codec registers.
<1>
<1> ; 22/07/2020
<1> ; REALTEK ALC655 and ADI SOUNDMAX AD1980 CODEC MIXER REGISTERS
<1>
<1> ; each codec/mixer register is 16bits
<1>
<1> CODEC_RESET_REG                 equ     00h     ; reset codec
<1> CODEC_MASTER_VOL_REG            equ     02h     ; master volume
<1> CODEC_HP_VOL_REG                equ     04h     ; headphone volume ; AD1980
<1> CODEC_MASTER_MONO_VOL_REG       equ     06h     ; master mono volume (mono-out)
<1> ;CODEC_MASTER_TONE_REG          equ     08h     ; master tone (R+L) ; (not used)
<1> CODEC_PCBEEP_VOL_REG            equ     0Ah     ; PC beep volume ; ALC655
<1> CODEC_PHONE_VOL_REG             equ     0Ch     ; phone volume
<1> CODEC_MIC_VOL_REG               equ     0Eh     ; mic volume
<1> CODEC_LINE_IN_VOL_REG           equ     10h     ; line in volume
<1> CODEC_CD_VOL_REG                equ     12h     ; CD volume
<1> ;CODEC_VID_VOL_REG              equ     14h     ; video volume ; (not used)
<1> CODEC_AUX_VOL_REG               equ     16h     ; aux volume
<1> CODEC_PCM_OUT_REG               equ     18h     ; PCM out volume
<1> CODEC_RECORD_SELECT_REG         equ     1Ah     ; record select
<1> CODEC_RECORD_VOL_REG            equ     1Ch     ; record volume (record gain)
<1> ;CODEC_RECORD_MIC_VOL_REG       equ     1Eh     ; record mic volume ; (not used)
<1> CODEC_GP_REG                    equ     20h     ; general purpose
<1> ;CODEC_3D_CONTROL_REG           equ     22h     ; 3D control
<1> ;;CODEC_AUDIO_INT_PAGING_REG    equ      24h    ; audio int & paging ; (not used)
<1> CODEC_POWER_CTRL_REG            equ     26h     ; power down control
<1> CODEC_EXT_AUDIO_REG             equ     28h     ; extended audio ID
<1> CODEC_EXT_AUDIO_CTRL_REG        equ     2Ah     ; extended audio status/control
<1> CODEC_PCM_FRONT_DACRATE_REG     equ     2Ch     ; PCM front sample rate
<1> CODEC_PCM_SURND_DACRATE_REG     equ     2Eh     ; PCM surround sample rate
<1> CODEC_PCM_LFE_DACRATE_REG       equ     30h     ; PCM Center/LFE sample rate
<1> CODEC_LR_ADCRATE_REG            equ     32h     ; PCM input sample rate
<1> CODEC_MIC_ADCRATE_REG           equ     34h     ; mic in sample rate  ; AD1980
<1> CODEC_PCM_LFE_VOL_REG           equ     36h     ; PCM Center/LFE volume
<1> CODEC_PCM_SURND_VOL_REG         equ     38h     ; PCM surround volume
<1> ;CODEC_SPDIF_CTRL_REG           equ     3Ah     ; S/PDIF control
<1> ; 22/07/2020
<1> CODEC_MISC_CRTL_BITS_REG  equ     76h     ; misc control bits ; AD1980
<1> ;
<1> CODEC_VENDOR_ID1          equ     7Ch     ; REALTEK: 414Ch, ADI: 4144h
<1> CODEC_VENDOR_ID2          equ     7Eh     ; REALTEK: 4760h, ADI: 5370h
<1>
<1> ; VT8233 SGD bits (21/04/2017)
<1> FLAGEQU BIT30
<1> EOL EQU BIT31
<1>
<1> ; INTEL ICH EQUATES
<1> ; 28/05/2017
<1> INTEL_VID   equ     8086h   ; Intel's PCI vendor ID
<1> ; 20/11/2023
<1> ; (playwav2.com, ac97.inc, Erdogan Tan, 11/11/2023)
<1> ; 03/11/2023 - Erdogan Tan (Ref: MenuetOS AC97 WAV Player source code, 2004)
<1> SIS_VID             equ     1039h
<1> NVIDIA_VID  equ     10DEh    ; Ref: MPXPLAY/SBEMU/KOLIBRIOS AC97 source c.
<1> AMD_VID             equ     1022h
<1> ;
<1> ICH_DID             equ     2415h   ; ICH (82801AA) device ID
<1> ; 20/11/2023
<1> ; (playwav2.com, ac97.inc, Erdogan Tan, 11/11/2023)
<1> ; 17/02/2017 (Erdogan Tan, ref: ALSA Device IDs, ALSA project)
<1> ICH0_DID        equ     2425h     ; ICH0
<1> ICH2_DID        equ     2445h     ; ICH2
<1> ICH3_DID    equ     2485h  ; ICH3
<1> ICH4_DID        equ     24C5h     ; ICH4
<1> ICH5_DID    equ     24D5h  ; ICH5
<1> ICH6_DID    equ     266Eh  ; ICH6
<1> ESB6300_DID equ     25A6h  ; 6300ESB
<1> ESB631X_DID equ     2698h  ; 631XESB
<1> ICH7_DID    equ     27DEh   ; ICH7
<1> ; 03/11/2023 - Erdogan Tan (Ref: MenuetOS AC97 WAV Player source code, 2004)
<1> MX82440_DID equ     7195h
<1> SI7012_DID  equ     7012h
<1> NFORCE_DID  equ     01B1h
<1> NFORCE2_DID equ     006Ah
<1> AMD8111_DID equ     746Dh
<1> AMD768_DID  equ     7445h
<1> ; 03/11/2023 - Erdogan Tan - Ref: MPXPLAY/SBEMU/KOLIBRIOS AC97 source code
<1> CK804_DID   equ     0059h   ; NFORCE4
<1> MCP04_DID   equ     003Ah
<1> CK8_DID             equ     008Ah
<1> NFORCE3_DID equ     00DAh
<1> CK8S_DID    equ     00EAh
<1>
<1> NAMBAR_REG      equ       10h     ; native audio mixer Base Address Register
<1> NABMBAR_REG     equ       14h     ; native audio bus mastering Base Addr Reg
<1>
<1> PI_CR_REG       equ     0Bh     ; PCM in Control Register
<1> PO_CR_REG   equ     1Bh     ; PCM out Control Register
<1> MC_CR_REG   equ     2Bh     ; MIC in Control Register
<1>
<1> PI_SR_REG   equ     6       ; PCM in Status register
<1> PO_SR_REG   equ     16h     ; PCM out Status register
<1> MC_SR_REG   equ     26h     ; MIC in Status register
<1>
<1> IOCE                equ     BIT4    ; interrupt on complete enable.
<1> FEIFE               equ     BIT3    ; set if you want an interrupt to fire
<1> LVBIE               equ     BIT2    ; last valid buffer interrupt enable.
<1> RR          equ     BIT1    ; reset registers. Nukes all regs
<1>                                 ; except bits 4:2 of this register.
<1>                                 ; Only set this bit if BIT 0 is 0
<1> RPBM        equ     BIT0    ; Run/Pause
<1>                             ; set this bit to start the codec!
<1>
<1> PI_BDBAR_REG        equ     0       ; PCM in buffer descriptor BAR
<1> PO_BDBAR_REG        equ     10h     ; PCM out buffer descriptor BAR
<1> MC_BDBAR_REG        equ     20h     ; MIC in buffer descriptor BAR
<1>
<1> PI_CIV_REG  equ     4       ; PCM in current Index value (RO)
<1> PO_CIV_REG  equ     14h     ; PCM out current Index value (RO)
<1> MC_CIV_REG  equ     24h     ; MIC in current Index value (RO)
<1>
<1> PI_LVI_REG  equ     5       ; PCM in Last Valid Index
<1> PO_LVI_REG  equ     15h     ; PCM out Last Valid Index
<1> MC_LVI_REG  equ     25h     ; MIC in Last Valid Index
```

```
                                <1>
                                <1> IOC         equ     BIT31  ; Fire an interrupt whenever this
                                <1>                            ; buffer is complete.
                                <1> BUP         equ     BIT30  ; Buffer Underrun Policy.
                                <1>
                                <1> GLOB_CNT_REG       equ     2Ch     ; Global Control Register
                                <1> GLOB_STS_REG       equ     30h     ; Global Status register (RO)
                                <1>
                                <1> CTRL_ST_CREADY     equ   BIT8+BIT9+BIT28 ; Primary Codec Ready
                                <1>
                                <1> CODEC_REG_POWERDOWN   equ 26h
                                <1> CODEC_REG_ST          equ 26h
                                <1>
                                <1> ; 22/06/2017
                                <1> PO_PICB_REG equ 18h ; PCM Out Position In Current Buffer Register
                                <1>
                                <1> ; 19/11/2023
                                <1> AC97_EA_VRA equ BIT0
                                <1> ; 24/11/2023
                                <1> BCIS        equ BIT3  ; Buffer Completion Interrupt Status
                                <1>
                                <1> ;==============================================================================
                                <1> ;               CODE
                                <1> ;==============================================================================
                                <1>
                                <1> ; CODE for INTEL ICH AC'97 AUDIO CONTROLLER
                                <1>
                                <1> DetectICH:
                                <1>     ; 22/11/2023
                                <1>     ; 19/11/2023
                                <1>     ; 01/11/2023 - TRDOS 386 Kernel v2.0.7
                                <1>     ;; 10/06/2017
                                <1>     ;; 05/06/2017
                                <1>     ;; 29/05/2017
                                <1>     ;; 28/05/2017
                                <1>     ;mov    eax, (ICH_DID << 16) + INTEL_VID
                                <1>         ;call      pciFindDevice
                                <1>         ;jnc       short d_ac97_1
                                <1>     ;
                                <1>     ;; 01/11/2023
                                <1>     ;mov    eax, (NFORCE_DID << 16) + NFORCE_VID
                                <1>         ;call      pciFindDevice
                                <1>         ;jnc       short d_ac97_1
                                <1>
                                <1>     ; 19/11/2023
00013988 BE[AB470100]           <1>     mov     esi, valid_ids ; address of Valid ICH (AC97) Device IDs
0001398D B915000000             <1>     mov     ecx, valid_id_count
                                <1> pfd_1:
00013992 AD                     <1>     lodsd
00013993 E857000000             <1>     call    pciFindDevice
00013998 730F                   <1>     jnc     short d_ac97_1
0001399A E2F6                   <1>     loop    pfd_1
                                <1>     ;stc
                                <1>
                                <1> d_ac97_0:
                                <1> ; couldn't find the audio device!
0001399C C3                     <1>     retn
                                <1>
                                <1> ; CODE for VIA VT8233 AUDIO CONTROLLER
                                <1>
                                <1> DetectVT8233:
                                <1>     ; 22/11/2023
                                <1>     ; 02/11/2023
                                <1>     ; 01/11/2023 - TRDOS 386 Kernel v2.0.7
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 10/06/2017
                                <1>     ; 05/06/2017
                                <1>     ; 29/05/2017
                                <1>     ; 03/04/2017
0001399D B806115930             <1>     mov     eax, (VT8233_DID << 16) + VIA_VID
000139A2 E848000000             <1>         call    pciFindDevice
                                <1> ;       jnc     short d_vt8233_0
                                <1> ; couldn't find the audio device!
                                <1> ;   retn
000139A7 72F3                   <1>     jc      short d_ac97_0  ; 28/05/2017
                                <1> d_vt8233_0:
                                <1>     ; 24/03/2017 ('player.asm')
                                <1>     ; 12/11/2016
                                <1>     ; Erdogan Tan - 8/11/2016
                                <1>     ; References: Kolibrios - vt823x.asm (2016)
                                <1>     ;             VIA VT8235 V-Link South Bridge (VT8235-VIA.PDF)(2002)
                                <1>     ;             lowlevel.eu - AC97 (2016)
                                <1>     ;             .wav player for DOS by Jeff Leyda (2002) -this file-
                                <1>     ;             Linux kernel - via82xx.c (2016)
                                <1> d_ac97_1:
                                <1>     ; eax = BUS/DEV/FN
                                <1>     ;       00000000BBBBBBBBDDDDDFFF00000000
                                <1>     ; edx = DEV/VENDOR
                                <1>     ;       DDDDDDDDDDDDDDDDVVVVVVVVVVVVVVVV
                                <1>
000139A9 A3[EC880100]           <1>     mov     [audio_dev_id], eax
000139AE 8915[F0880100]         <1>     mov     [audio_vendor], edx
                                <1>
                                <1>     ; 22/11/2023
                                <1>     ;; init controller
                                <1>     ;mov    al, PCI_CMD_REG ; command register (04h)
                                <1>     ;call   pciRegRead32
                                <1>     ;
                                <1>     ;; eax = BUS/DEV/FN/REG
                                <1>     ;; edx = STATUS/COMMAND
                                <1>     ;;      SSSSSSSSSSSSSSSSCCCCCCCCCCCCCCCC
                                <1>     ;mov    [audio_stats_cmd], edx
                                <1>
000139B4 B010                   <1>     mov     al, PCI_IO_BASE ; IO base address register (10h)
                                <1>     ;mov    al, NAMBAR_REG ; Native Audio Mixer BAR (10h)
000139B6 E8BF000000             <1>     call    pciRegRead32
                                <1>
                                <1>     ;cmp    word [audio_vendor], INTEL_VID ; 8086h ; AC'97 ?
                                <1>     ;;jne   short d_vt8233_1
                                <1>     ;; 01/11/2023
                                <1>     ;je     short d_ac97_3
                                <1>     ;cmp    word [audio_vendor], NFORCE_VID ; 10DEh ; AC'97
                                <1>     ;jne    short d_vt8233_1
                                <1>     ; 02/11/2023
000139BB 66813D[F0880100]06-    <1>     cmp     word [audio_vendor], VIA_VID  ; 1106h ; VT8233-VT8237R
000139C3 11                     <1>
000139C4 7411                   <1>     je      short d_vt8233_1
                                <1> d_ac97_3:
                                <1>     ;and    dx, 0FFFEh ; Audio Codec IO_ADDR_MASK
                                <1>     ; 06/08/2022
000139C6 80E2FE                 <1>     and     dl, 0FEh
000139C9 668915[E8880100]       <1>     mov     [NAMBAR], dx
                                <1>
000139D0 B014                   <1>     mov     al, NABMBAR_REG ; Native Audio Bus Mastering BAR (14h)
```

```
000139D2 E8A3000000          <1>     call    pciRegRead32
                             <1>
                             <1>     ;;and   dx, 0FFC0h ; Audio Controller IO_ADDR_MASK
                             <1>     ;; 06/08/2022
                             <1>     ;and    dl, 0C0h
                             <1>     ;mov    [NABMBAR], dx
                             <1>         ;;mov      [audio_io_base], dx
                             <1>     ;
                             <1>     ;jmp    short d_ac97_2
                             <1>     ; 02/11/2023
                             <1>     ; NABMBAR = audio_io_base
                             <1>
                             <1> d_vt8233_1:
                             <1>     ;and    dx, 0FFC0h ; Audio Controller IO_ADDR_MASK
                             <1>     ; 06/08/2022
000139D7 80E2C0              <1>     and     dl, 0C0h
000139DA 668915[EA880100]    <1>         mov     [audio_io_base], dx
                             <1>
                             <1> d_ac97_2:
                             <1>     ; 10/06/2017
000139E1 B03C                <1>     mov     al, AC97_INT_LINE ; Interrupt Line Register (3Ch)
                             <1>     ;call   pciRegRead32
000139E3 E87F000000          <1>     call    pciRegRead8
                             <1>
                             <1>     ;;and   edx, 0FFh
                             <1>     ; 06/08/2022
                             <1>     ;and    dx, 0FFh
                             <1>
000139E8 8815[E6880100]      <1>     mov     [audio_intr], dl
                             <1>
000139EE C3                  <1>     retn
                             <1>
                             <1>     ; (Note: Interrupts are already enabled by TRDOS 386 kernel!)
                             <1>     ;mov    cx, dx
                             <1>     ;
                             <1>     ;in     al, 0A1h ; irq 8-15
                             <1>     ;mov    ah, al
                             <1>     ;in     al, 21h  ; irq 0-7
                             <1>     ;btr    ax, dx   ; unmask ; 17/03/2017
                             <1>     ;;bts   ax, dx   ; MASK interrupt ; 10/06/2017
                             <1>     ;out    21h, al  ; irq <= 7
                             <1>     ;mov    al, ah
                             <1>     ;out    0A1h, al ; irq > 7
                             <1>     ;
                             <1>
                             <1>     ; 10/06/2017
                             <1>     ; === Intel ICH I/O Controller Hub Datasheet, Section 8.1.16 ===
                             <1>     ; PRQ[n]_ROUT Register (61h, PRQB) Bit 7:
                             <1>     ; Interrupt Routing Enable (IRQEN).
                             <1>     ; 0 = The corresponding PIRQ is routed to one of the ISA-compatible
                             <1>     ;     interrupts specified in bits[3:0].
                             <1>     ; 1 = The PIRQ is not routed to the 8259.
                             <1>     ; Note: If the PIRQ is intended to cause an interrupt to the ICH's
                             <1>     ;       integrated I/O APIC, then this bit should be set to 0 and
                             <1>     ;       the APIC_EN bit should be set to 1.
                             <1>     ;       The IRQEN must be set to 0 and the PIRQ routed to
                             <1>     ;       an 8259 interrupt via the IRQ Routing filed (bits[3:0).
                             <1>     ;       The corresponding 8259 interrupt must be masked via the
                             <1>     ;       appropriated bit in the 8259's OCW1 (Interrupt Mask)
                             <1>     ;       register. The IOAPIC must then be enabled by setting
                             <1>     ;       the APIC_EN bit in the GEN_CNTL register.
                             <1>
                             <1>     ;mov    eax, 0F861h  ; D31:F0
                             <1>     ;       ; AL=61h : PIRQ[B] Routing Control Reg, LPC interface
                             <1>     ;;mov   dl, [audio_intr]
                             <1>     ;call   pciRegWrite8
                             <1>     ;;mov   al, 0D0h       ; General Control Register (GEN_CTL)
                             <1>     ;;call  pciRegRead32
                             <1>     ;;or    edx, 100h      ; Bit 8, APIC_EN (Enable I/O APIC)
                             <1>     ;;call  pciRegWrite32
                             <1>     ;;and   edx, ~100h
                             <1>     ;;call  pciRegWrite32  ; Bit 8, APIC_EN (Disable I/O APIC)
                             <1>     ;
                             <1>
                             <1>     ;mov    dx, 4D1h       ; 8259 ELCR2
                             <1>     ;in     al, dx
                             <1>     ;mov    ah, al
                             <1>     ;;mov   dx, 4D0h       ; 8259 ELCR1
                             <1>     ;dec    dl
                             <1>     ;in     al, dx
                             <1>     ;bts    ax, cx
                             <1>     ;;mov   dx, 4D0h
                             <1>     ;out    dx, al         ; set level-triggered mode
                             <1>     ;mov    al, ah ; 29/05/2017
                             <1>     ;;mov   dx, 4D1h
                             <1>     ;inc    dl
                             <1>     ;out    dx, al         ; set level-triggered mode
                             <1>
                             <1>     ;xor    eax, eax ; 0
                             <1>
                             <1>     ;retn
                             <1>
                             <1> ; CODE for PCI
                             <1>
                             <1> pciFindDevice:
                             <1>     ; 19/11/2023
                             <1>     ; 03/04/2017 ('pci.asm', 20/03/2017)
                             <1>     ;
                             <1>     ; scan through PCI space looking for a device+vendor ID
                             <1>     ;
                             <1>     ; Entry: EAX=Device+Vendor ID
                             <1>     ;
                             <1>     ; Exit: EAX=PCI address if device found
                             <1>     ;       EDX=Device+Vendor ID
                             <1>     ;        CY clear if found, set if not found. EAX invalid if CY set.
                             <1>     ;
                             <1>     ; Destroys: ebx, edi ; 19/11/2023
                             <1>
                             <1>     ;push   ecx
                             <1>     ;push   eax ; * ; 19/11/2023
                             <1>     ;push   esi
                             <1>     ;push   edi
                             <1>     ;
                             <1>     ;mov    esi, eax               ; save off vend+device ID
                             <1>         ; 19/11/2023
000139EF 89C3                <1>     mov     ebx, eax
                             <1>     ;mov    edi, (80000000h - 100h)       ; start with bus 0, dev 0 func 0
000139F1 BF00000080          <1>     mov     edi, 80000000h
                             <1> nextPCIdevice:
                             <1>     ;add    edi, 100h
                             <1>     ;cmp    edi, 80FFF800h         ; scanned all devices?
                             <1>     ;stc
                             <1>     ;je     short PCIScanExit      ; not found
                             <1>
```

```
000139F6 89F8                <1>     mov     eax, edi                ; read PCI registers
000139F8 E87D000000          <1>     call    pciRegRead32
                             <1>     ; 19/11/2023
000139FD 39DA                <1>     cmp     edx, ebx
                             <1>     ;cmp    edx, esi                ; found device?
                             <1>     ;jne    short nextPCIdevice
                             <1>     ;;clc
000139FF 7412                <1>     je      short PCIScanExit       ; found
                             <1>     ; 19/11/2023
00013A01 81FF00F8FF80        <1>     cmp     edi, 80FFF800h
00013A07 7308                <1>     jnb     short pfd_nf            ; not found
00013A09 81C700010000        <1>     add     edi, 100h
00013A0F EBE5                <1>     jmp     short nextPCIdevice
                             <1> pfd_nf:
00013A11 F9                  <1>     stc
00013A12 C3                  <1>     retn
                             <1> PCIScanExit:
                             <1>     ;pushf
00013A13 B8FFFFFF7F          <1>     mov     eax, NOT_BIT31          ; 19/03/2017
00013A18 21F8                <1>     and     eax, edi        ; return only bus/dev/fn #
                             <1>     ;popf
                             <1>     ;
                             <1>     ;pop    edi
                             <1>     ;pop    esi
                             <1>     ;pop    edx ; * ; 19/11/2023
                             <1>     ;pop    ecx
                             <1>     ;
00013A1A C3                  <1>     retn
                             <1>
                             <1> ; 26/11/2023
                             <1> %if 1
                             <1>
                             <1> pciRegRead:
                             <1>     ; 03/04/2017 ('pci.asm', 20/03/2017)
                             <1>     ;
                             <1>     ; 8/16/32bit PCI reader
                             <1>     ;
                             <1>     ; Entry: EAX=PCI Bus/Device/fn/register number
                             <1>     ;           BIT30 set if 32 bit access requested
                             <1>     ;           BIT29 set if 16 bit access requested
                             <1>     ;           otherwise defaults to 8 bit read
                             <1>     ;
                             <1>     ; Exit:  DL,DX,EDX register data depending on requested read size
                             <1>     ;
                             <1>     ; Note1: this routine is meant to be called via pciRegRead8,
                             <1>     ;       pciRegread16 or pciRegRead32, listed below.
                             <1>     ;
                             <1>     ; Note2: don't attempt to read 32 bits of data from a non dword
                             <1>     ;       aligned reg number. Likewise, don't do 16 bit reads from
                             <1>     ;       non word aligned reg #
                             <1>
00013A1B 53                  <1>     push    ebx
00013A1C 51                  <1>     push    ecx
00013A1D 89C3                <1>         mov     ebx, eax            ; save eax, dh
00013A1F 88F1                <1>         mov     cl, dh
                             <1>
00013A21 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16        ; clear out data size request
00013A26 0D00000080          <1>         or      eax, BIT31          ; make a PCI access request
00013A2B 24FC                <1>         and     al, ~3 ; NOT 3              ; force index to be dword
                             <1>
00013A2D 66BAF80C            <1>         mov     dx, PCI_INDEX_PORT
00013A31 EF                  <1>         out dx, eax                 ; write PCI selector
                             <1>
00013A32 66BAFC0C            <1>         mov     dx, PCI_DATA_PORT
00013A36 88D8                <1>         mov     al, bl
00013A38 2403                <1>         and     al, 3                       ; figure out which port to
00013A3A 00C2                <1>         add     dl, al                      ; read to
                             <1>
00013A3C F7C3000000C0        <1>     test    ebx, PCI32+PCI16
00013A42 7507                <1>         jnz     short _pregr0
00013A44 EC                  <1>     in      al, dx                  ; return 8 bits of data
00013A45 88C2                <1>         mov dl, al
00013A47 88CE                <1>     mov     dh, cl                  ; restore dh for 8 bit read
00013A49 EB12                <1>     jmp     short _pregr2
                             <1> _pregr0:
00013A4B F7C300000080        <1>     test    ebx, PCI32
00013A51 7507                <1>         jnz short _pregr1
00013A53 66ED                <1>     in      ax, dx
00013A55 6689C2              <1>         mov     dx, ax                      ; return 16 bits of data
00013A58 EB03                <1>     jmp     short _pregr2
                             <1> _pregr1:
00013A5A ED                  <1>     in      eax, dx                 ; return 32 bits of data
00013A5B 89C2                <1>     mov     edx, eax
                             <1> _pregr2:
00013A5D 89D8                <1>     mov     eax, ebx                ; restore eax
00013A5F 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16        ; clear out data size request
00013A64 59                  <1>     pop     ecx
00013A65 5B                  <1>     pop     ebx
00013A66 C3                  <1>     retn
                             <1>
                             <1> pciRegRead8:
00013A67 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16        ; set up 8 bit read size
00013A6C EBAD                <1>         jmp     short pciRegRead  ; call generic PCI access
                             <1>
                             <1> pciRegRead16:
00013A6E 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16        ; set up 16 bit read size
00013A73 0D00000040          <1>         or      eax, PCI16          ; call generic PCI access
00013A78 EBA1                <1>         jmp     short pciRegRead
                             <1>
                             <1> pciRegRead32:
00013A7A 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16        ; set up 32 bit read size
00013A7F 0D00000080          <1>         or      eax, PCI32          ; call generic PCI access
00013A84 EB95                <1>         jmp     pciRegRead
                             <1>
                             <1> pciRegWrite:
                             <1>     ; 03/04/2017 ('pci.asm', 29/11/2016)
                             <1>     ;
                             <1>     ; 8/16/32bit PCI writer
                             <1>     ;
                             <1>     ; Entry: EAX=PCI Bus/Device/fn/register number
                             <1>     ;           BIT31 set if 32 bit access requested
                             <1>     ;           BIT30 set if 16 bit access requested
                             <1>     ;           otherwise defaults to 8bit read
                             <1>     ;        DL/DX/EDX data to write depending on size
                             <1>     ;
                             <1>     ; Note1: this routine is meant to be called via pciRegWrite8,
                             <1>     ;       pciRegWrite16 or pciRegWrite32 as detailed below.
                             <1>     ;
                             <1>     ; Note2: don't attempt to write 32bits of data from a non dword
                             <1>     ;       aligned reg number. Likewise, don't do 16 bit writes from
                             <1>     ;       non word aligned reg #
                             <1>
00013A86 53                  <1>     push    ebx
00013A87 51                  <1>     push    ecx
```

```
00013A88 89C3                <1>         mov     ebx, eax          ; save eax, edx
00013A8A 89D1                <1>         mov     ecx, edx
00013A8C 25FFFFFF3F          <1>     and     eax, NOT_PCI32_PCI16  ; clear out data size request
00013A91 0D00000080          <1>         or      eax, BIT31        ; make a PCI access request
00013A96 24FC                <1>         and     al, ~3 ; NOT 3           ; force index to be dword
                             <1>
00013A98 66BAF80C            <1>         mov     dx, PCI_INDEX_PORT
00013A9C EF                  <1>         out dx, eax               ; write PCI selector
                             <1>
00013A9D 66BAFC0C            <1>         mov     dx, PCI_DATA_PORT
00013AA1 88D8                <1>         mov     al, bl
00013AA3 2403                <1>         and     al, 3                     ; figure out which port to
00013AA5 00C2                <1>         add     dl, al                    ; write to
                             <1>
00013AA7 F7C3000000C0        <1>     test    ebx, PCI32+PCI16
00013AAD 7505                <1>         jnz     short _pregw0
00013AAF 88C8                <1>     mov    al, cl                 ; put data into al
00013AB1 EE                  <1>     out    dx, al
00013AB2 EB12                <1>     jmp    short _pregw2
                             <1> _pregw0:
00013AB4 F7C300000080        <1>     test    ebx, PCI32
00013ABA 7507                <1>         jnz     short _pregw1
00013ABC 6689C8              <1>     mov    ax, cx                 ; put data into ax
00013ABF 66EF                <1>     out    dx, ax
00013AC1 EB03                <1>     jmp    short _pregw2
                             <1> _pregw1:
00013AC3 89C8                <1>     mov    eax, ecx               ; put data into eax
00013AC5 EF                  <1>     out    dx, eax
                             <1> _pregw2:
00013AC6 89D8                <1>         mov     eax, ebx          ; restore eax
00013AC8 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16     ; clear out data size request
00013ACD 89CA                <1>         mov     edx, ecx          ; restore dx
00013ACF 59                  <1>     pop    ecx
00013AD0 5B                  <1>     pop    ebx
00013AD1 C3                  <1>     retn
                             <1>
                             <1> pciRegWrite8:
00013AD2 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16     ; set up 8 bit write size
00013AD7 EBAD                <1>         jmp short pciRegWrite     ; call generic PCI access
                             <1>
                             <1> pciRegWrite16:
00013AD9 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16     ; set up 16 bit write size
00013ADE 0D00000040          <1>         or      eax, PCI16        ; call generic PCI access
00013AE3 EBA1                <1>         jmp short pciRegWrite
                             <1>
                             <1> pciRegWrite32:
00013AE5 25FFFFFF3F          <1>         and     eax, NOT_PCI32_PCI16     ; set up 32 bit write size
00013AEA 0D00000080          <1>         or      eax, PCI32        ; call generic PCI access
00013AEF EB95                <1>         jmp pciRegWrite
                             <1>
                             <1> %endif
                             <1>
                             <1> ; 26/11/2023 - temporary
                             <1> %if 0
                             <1>
                             <1> ; PLAYWAV3.COM, ac97_vra.asm, 19/11/2023, Erdogan Tan
                             <1>
                             <1> ;==============================================================
                             <1> ; 8/16/32bit PCI reader
                             <1> ;
                             <1> ; Entry: EAX=PCI Bus/Device/fn/register number
                             <1> ;           BIT30 set if 32 bit access requested
                             <1> ;           BIT29 set if 16 bit access requested
                             <1> ;           otherwise defaults to 8 bit read
                             <1> ;
                             <1> ; Exit:  DL,DX,EDX register data depending on requested read size
                             <1> ;
                             <1> ; Note: this routine is meant to be called via pciRegRead8, pciRegread16,
                             <1> ;   or pciRegRead32, listed below.
                             <1> ;
                             <1> ; Note2: don't attempt to read 32bits of data from a non dword aligned reg
                             <1> ;    number. Likewise, don't do 16bit reads from non word aligned reg #
                             <1> ;
                             <1> pciRegRead:
                             <1>     push   ebx
                             <1>     push   ecx
                             <1>         mov     ebx, eax                      ; save eax, dh
                             <1>         mov     cl, dh
                             <1>         and     eax, (~PCI32)+PCI16           ; clear out data size request
                             <1>         or      eax, BIT31                    ; make a PCI access request
                             <1>         and     al, ~3 ; NOT 3                ; force index to be dword
                             <1>
                             <1>         mov     dx, PCI_INDEX_PORT
                             <1>         out     dx, eax                       ; write PCI selector
                             <1>
                             <1>         mov     dx, PCI_DATA_PORT
                             <1>         mov     al, bl
                             <1>         and     al, 3                         ; figure out which port to
                             <1>         add     dl, al                        ; read to
                             <1>
                             <1>     in      eax, dx                       ; do 32bit read
                             <1>         test    ebx, PCI32
                             <1>         jz      short _pregr1
                             <1>
                             <1>         mov     edx, eax                      ; return 32bits of data
                             <1> _pregr1:
                             <1>     mov     dx, ax                        ; return 16bits of data
                             <1>         test    ebx, PCI32+PCI16
                             <1>         jnz     short _pregr2
                             <1>         mov     dh, cl                        ; restore dh for 8 bit read
                             <1> _pregr2:
                             <1>         mov     eax, ebx                      ; restore eax
                             <1>         and     eax, (~PCI32)+PCI16           ; clear out data size request
                             <1>     pop    ecx
                             <1>     pop    ebx
                             <1>     retn
                             <1>
                             <1> pciRegRead8:
                             <1>         and     eax, (~PCI16)+PCI32           ; set up 8 bit read size
                             <1>         jmp     short pciRegRead         ; call generic PCI access
                             <1>
                             <1> pciRegRead16:
                             <1>         and     eax, (~PCI16)+PCI32           ; set up 16 bit read size
                             <1>         or      eax, PCI16               ; call generic PCI access
                             <1>         jmp     short pciRegRead
                             <1>
                             <1> pciRegRead32:
                             <1>         and     eax, (~PCI16)+PCI32           ; set up 32 bit read size
                             <1>         or      eax, PCI32               ; call generic PCI access
                             <1>         jmp     short pciRegRead
                             <1>
                             <1> ;==============================================================
                             <1> ; 8/16/32bit PCI writer
                             <1> ;
```

```
                                <1> ; Entry: EAX=PCI Bus/Device/fn/register number
                                <1> ;           BIT31 set if 32 bit access requested
                                <1> ;           BIT30 set if 16 bit access requested
                                <1> ;           otherwise defaults to 8bit read
                                <1> ;        DL/DX/EDX data to write depending on size
                                <1> ;
                                <1> ;
                                <1> ; note: this routine is meant to be called via pciRegWrite8, pciRegWrite16,
                                <1> ;   or pciRegWrite32 as detailed below.
                                <1> ;
                                <1> ; Note2: don't attempt to write 32bits of data from a non dword aligned reg
                                <1> ;    number. Likewise, don't do 16bit writes from non word aligned reg #
                                <1> ;
                                <1> pciRegWrite:
                                <1>     push    ebx
                                <1>     push    ecx
                                <1>         mov     ebx, eax                        ; save eax, dx
                                <1>         mov     cx, dx
                                <1>         or      eax, BIT31                      ; make a PCI access request
                                <1>         and     eax, ~PCI16 ; NOT PCI16         ; clear out data size request
                                <1>         and     al, ~3 ; NOT 3                  ; force index to be dword
                                <1>
                                <1>         mov     dx, PCI_INDEX_PORT
                                <1>         out     dx, eax                         ; write PCI selector
                                <1>
                                <1>         mov     dx, PCI_DATA_PORT
                                <1>         mov     al, bl
                                <1>         and     al, 3                           ; figure out which port to
                                <1>         add     dl, al                          ; write to
                                <1>
                                <1>         mov     eax, edx                        ; put data into eax
                                <1>         mov     ax, cx
                                <1>
                                <1>         out     dx, al
                                <1>         test    ebx, PCI16+PCI32                ; only 8bit access? bail
                                <1>         jz      short _pregw1
                                <1>
                                <1>         out     dx, ax                          ; write 16 bit value
                                <1>         test    ebx, PCI16                      ; 16bit requested?  bail
                                <1>         jnz     short _pregw1
                                <1>
                                <1>         out     dx, eax                         ; write full 32bit
                                <1> _pregw1:
                                <1>         mov     eax, ebx                        ; restore eax
                                <1>         and     eax, (~PCI32)+PCI16             ; clear out data size request
                                <1>         mov     dx, cx                          ; restore dx
                                <1>     pop     ecx
                                <1>     pop     ebx
                                <1>     ret
                                <1>
                                <1> pciRegWrite8:
                                <1>         and     eax, (~PCI16)+PCI32             ; set up 8 bit write size
                                <1>         jmp     short pciRegWrite        ; call generic PCI access
                                <1>
                                <1> pciRegWrite16:
                                <1>         and     eax, (~PCI16)+PCI32             ; set up 16 bit write size
                                <1>         or      eax, PCI16               ; call generic PCI access
                                <1>         jmp     short pciRegWrite
                                <1>
                                <1> pciRegWrite32:
                                <1>         and     eax, (~PCI16)+PCI32             ; set up 32 bit write size
                                <1>         or      eax, PCI32               ; call generic PCI access
                                <1>         jmp     short pciRegWrite
                                <1>
                                <1> ;==============================================================
                                <1>
                                <1> %endif
                                <1>
                                <1> init_codec:
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 05/06/2017
                                <1>     ; 28/05/2017 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                                <1>     ;
00013AF1 A1[EC880100]           <1>     mov     eax, [audio_dev_id]
00013AF6 B041                   <1>     mov     al, VIA_ACLINK_CTRL
00013AF8 E86AFFFFFF             <1>     call    pciRegRead8
                                <1>     ; ?
00013AFD B040                   <1>     mov     al, VIA_ACLINK_STAT
00013AFF E863FFFFFF             <1>     call    pciRegRead8
00013B04 F6C201                 <1>     test    dl, VIA_ACLINK_C00_READY
00013B07 7508                   <1>         jnz     short _codec_ready_1
00013B09 E80D000000             <1>     call    reset_codec
00013B0E 7305                   <1>     jnc     short _codec_ready_2 ; eax = 1
00013B10 C3                     <1>     retn
                                <1> _codec_ready_1:
                                <1>     ;mov    eax, 1
                                <1>     ; 06/08/2022
00013B11 29C0                   <1>     sub     eax, eax
00013B13 FEC0                   <1>     inc     al
                                <1>     ; eax = 1
                                <1> _codec_ready_2:
00013B15 E880000000             <1>     call    codec_io_w16
                                <1> detect_codec:
00013B1A C3                     <1>     retn
                                <1>
                                <1> reset_codec:
                                <1>     ; 16/04/2017
                                <1>     ; 23/03/2017
                                <1>     ; ('codec.asm')
                                <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
00013B1B A1[EC880100]           <1>     mov     eax, [audio_dev_id]
00013B20 B041                   <1>     mov     al, VIA_ACLINK_CTRL
00013B22 B2E0                   <1>             mov     dl, VIA_ACLINK_CTRL_ENABLE + VIA_ACLINK_CTRL_RESET + VIA_ACLINK_CTRL_SYNC
00013B24 E8A9FFFFFF             <1>     call    pciRegWrite8
                                <1>
00013B29 E843000000             <1>     call    delay_100ms    ; wait 100 ms
                                <1> _rc_cold:
00013B2E E811000000             <1>         call    cold_reset
00013B33 7301                   <1>         jnc     short _reset_codec_ok
                                <1>
                                <1>     ; 16/04/2017
                                <1>         ;xor       eax, eax       ; timeout error
                                <1>             ;stc
00013B35 C3                     <1>     retn
                                <1>
                                <1> _reset_codec_ok:
                                <1>     ; 01/09/2020
                                <1>     ; 15/08/2020
                                <1>     ; 27/07/2020
                                <1>     ; also reset codec by using index control register 0 of AD1980 or ALC655
                                <1>     ; (to fix line out -2 channels audio playing- problem on AD1980 codec)
                                <1>
00013B36 29C0                   <1>     sub     eax, eax
                                <1>     ; 02/11/2023
```

```
                            <1>     ;mov    edx, CODEC_RESET_REG ; 00h ; Reset register
00013B38 31D2               <1>     xor     edx, edx ; 00h ; Reset register
00013B3A E8BE000000         <1>     call    codec_write
                            <1>
                            <1>     ;sub    eax, eax
                            <1>     ; 01/09/2020
                            <1>     ; 15/08/2020
                            <1>     ; AD1980 BugFix
                            <1>     ; (set HPSEL -headphone amp to be driven from mixer- and
                            <1>     ;      CLDIS -center and LFE disable- bits)
                            <1>     ;mov    eax, 0C00h ; HPSEL = bit 10, CLDIS = bit 11 ; 01/09/2020
                            <1>     ;mov    edx, CODEC_MISC_CRTL_BITS_REG ; 76h ; Misc Ctrl Bits ; AD1980
                            <1>     ;call   codec_write
                            <1>
00013B3F 31C0               <1>         xor     eax, eax
                            <1>         ;mov       al, VIA_ACLINK_C00_READY ; 1
00013B41 FEC0               <1>         inc al
00013B43 C3                 <1>     retn
                            <1>
                            <1> cold_reset:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                            <1>     ; 16/04/2017
                            <1>     ; 23/03/2017
                            <1>     ; ('codec.asm')
                            <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                            <1>     ;mov    eax, [audio_dev_id]
                            <1>     ;mov    al, VIA_ACLINK_CTRL
00013B44 30D2               <1>     xor     dl, dl ; 0
00013B46 E887FFFFFF         <1>     call    pciRegWrite8
                            <1>
00013B4B E821000000         <1>     call    delay_100ms    ; wait 100 ms
                            <1>
                            <1>     ;; ACLink on, deassert ACLink reset, VSR, SGD data out
                            <1>         ;; note - FM data out has trouble with non VRA codecs !!
                            <1>
                            <1>     ;mov    eax, [audio_dev_id]
                            <1>     ;mov    al, VIA_ACLINK_CTRL
00013B50 B2CC               <1>     mov     dl, VIA_ACLINK_CTRL_INIT
00013B52 E87BFFFFFF         <1>     call    pciRegWrite8
                            <1>
                            <1>     ;mov    ecx, 16 ; total 2s
                            <1>     ; 06/08/2022
                            <1>     ;sub    ecx, ecx
                            <1>     ; 22/11/2023
                            <1>     ; ecx = 0 (from 'delay_100ms')
00013B57 B110               <1>     mov     cl, 16
                            <1> _crst_wait:
                            <1>     ;mov    eax, [audio_dev_id]
00013B59 B040               <1>     mov     al, VIA_ACLINK_STAT
00013B5B E807FFFFFF         <1>     call    pciRegRead8
                            <1>
00013B60 F6C201             <1>         test    dl, VIA_ACLINK_C00_READY
00013B63 750B               <1>         jnz     short _crst_ok
                            <1>
00013B65 51                 <1>     push    ecx
00013B66 E806000000         <1>     call    delay_100ms
00013B6B 59                 <1>     pop     ecx
                            <1>
00013B6C 49                 <1>         dec     ecx
00013B6D 75EA               <1>         jnz     short _crst_wait
                            <1>
                            <1> _crst_fail:
00013B6F F9                 <1>         stc
                            <1> _crst_ok:
00013B70 C3                 <1>     retn
                            <1>
                            <1> delay_100ms:
                            <1>     ; 29/05/2017
                            <1>     ; 24/03/2017 ('codec.asm')
                            <1>     ; wait 100 ms
00013B71 B990010000         <1>     mov     ecx, 400  ; 400*0.25ms
                            <1> _delay_x_ms:
00013B76 E803000000         <1>     call    delay1_4ms
00013B7B E2F9               <1>         loop       _delay_x_ms
00013B7D C3                 <1>     retn
                            <1>
                            <1> ;       delay1_4ms - Delay for 1/4 millisecond.
                            <1> ;       1ms = 1000us
                            <1> ;       Entry:
                            <1> ;         None
                            <1> ;       Exit:
                            <1> ;     None
                            <1> ;
                            <1> ;       Modified:
                            <1> ;         None
                            <1>
                            <1>     ; 29/05/2017
                            <1>     ; 23/04/2017
                            <1>     ; 05/03/2017 (TRDOS 386)
                            <1>     ; ('UTILS.ASM')
                            <1> delay1_4ms:
00013B7E 50                 <1>         push    eax
00013B7F 51                 <1>         push    ecx
00013B80 B110               <1>         mov cl, 16         ; close enough.
                            <1>
00013B82 E461               <1>     in      al, PORTB ; 61h
                            <1>
00013B84 2410               <1>     and     al, REFRESH_STATUS ; 10h
00013B86 88C5               <1>     mov     ch, al         ; Start toggle state
                            <1> _d4ms1:
00013B88 E461               <1>     in      al, PORTB      ; Read system control port
                            <1>
00013B8A 2410               <1>     and     al, REFRESH_STATUS ; Refresh toggles 15.085 microseconds
00013B8C 38C5               <1>     cmp     ch, al
00013B8E 74F8               <1>     je      short _d4ms1   ; Wait for state change
                            <1>
00013B90 88C5               <1>     mov     ch, al         ; Update with new state
00013B92 FEC9               <1>     dec     cl
00013B94 75F2               <1>     jnz     short _d4ms1
                            <1>
00013B96 F8                 <1>     clc     ; 29/05/2017
                            <1>
00013B97 59                 <1>         pop     ecx
00013B98 58                 <1>         pop     eax
00013B99 C3                 <1>         retn
                            <1>
                            <1> ; 10/04/2017 (TRDOS 386)
                            <1> ; 12/11/2016
                            <1>
                            <1> codec_io_w16: ;w32
                            <1>     ; ('codec.asm')
00013B9A 668B15[EA880100]   <1>         mov dx, [audio_io_base]
00013BA1 6681C28000         <1>         add     dx, VIA_REG_AC97
00013BA6 EF                 <1>     out     dx, eax
```

```
00013BA7 C3                  <1>         retn
                             <1>
                             <1> codec_io_r16: ;r32
                             <1>     ; ('codec.asm')
00013BA8 668B15[EA880100]    <1>         mov     dx, [audio_io_base]
00013BAF 6681C28000          <1>         add     dx, VIA_REG_AC97
00013BB4 ED                  <1>         in  eax, dx
00013BB5 C3                  <1>         retn
                             <1>
                             <1> ctrl_io_w8:
                             <1>     ; ('codec.asm')
00013BB6 660315[EA880100]    <1>         add     dx, [audio_io_base]
00013BBD EE                  <1>         out dx, al
00013BBE C3                  <1>         retn
                             <1>
                             <1> ctrl_io_r8:
                             <1>     ; ('codec.asm')
00013BBF 660315[EA880100]    <1>         add     dx, [audio_io_base]
00013BC6 EC                  <1>         in  al, dx
00013BC7 C3                  <1>         retn
                             <1>
                             <1> ctrl_io_w32:
                             <1>     ; ('codec.asm')
00013BC8 660315[EA880100]    <1>         add     dx, [audio_io_base]
00013BCF EF                  <1>         out dx, eax
00013BD0 C3                  <1>         retn
                             <1>
                             <1> ctrl_io_r32:
                             <1>     ; ('codec.asm')
00013BD1 660315[EA880100]    <1>         add dx, [audio_io_base]
00013BD8 ED                  <1>     in      eax, dx
                             <1> _cr_not_rdy:       ; 06/08/2022
00013BD9 C3                  <1>         retn
                             <1>
                             <1> codec_read:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                             <1>         ; Use only primary codec.
                             <1>         ; eax = register
00013BDA C1E010              <1>         shl     eax, VIA_REG_AC97_CMD_SHIFT
00013BDD 0D00008002          <1>         or      eax, VIA_REG_AC97_PRIMARY_VALID + VIA_REG_AC97_READ
                             <1>
00013BE2 E8B3FFFFFF          <1>     call    codec_io_w16
                             <1>
                             <1>             ; codec_valid
00013BE7 E825000000          <1>     call    codec_check_ready
                             <1>     ;jnc    short _cr_ok
                             <1>     ;retn
                             <1>     ; 06/08/2022
00013BEC 72EB                <1>     jc      short _cr_not_rdy
                             <1>     ; ecx <= 20
                             <1> _cr_ok:
                             <1>     ; wait 25 ms
                             <1>     ;mov    ecx, 80 ; (100*0.25 ms)
                             <1>     ; 06/08/2022
                             <1>     ;xor    ecx, ecx
00013BEE B150                <1>     mov     cl, 80
                             <1>     ; ecx = 80
                             <1> _cr_wloop:
00013BF0 E889FFFFFF          <1>     call    delay1_4ms
00013BF5 E2F9                <1>     loop    _cr_wloop
                             <1>
00013BF7 E8ACFFFFFF          <1>         call    codec_io_r16
                             <1>     ; 06/08/2022
                             <1>     ;and    eax, 0FFFFh
00013BFC C3                  <1>         retn
                             <1>
                             <1> codec_write:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                             <1>         ; Use only primary codec.
                             <1>
                             <1>     ; eax = data (volume)
                             <1>     ; edx = register (mixer register)
                             <1>
00013BFD C1E210              <1>     shl     edx, VIA_REG_AC97_CMD_SHIFT
                             <1>
                             <1>     ; 02/11/2023 (shl eax, 0)
                             <1>     ;shl    eax, VIA_REG_AC97_DATA_SHIFT ; shl eax, 0
00013C00 09C2                <1>         or      edx, eax
                             <1>
00013C02 B800000000          <1>         mov     eax, VIA_REG_AC97_CODEC_ID_PRIMARY
00013C07 C1E01E              <1>         shl     eax, VIA_REG_AC97_CODEC_ID_SHIFT
00013C0A 09D0                <1>         or      eax, edx
                             <1>
00013C0C E889FFFFFF          <1>         call    codec_io_w16
                             <1>         ;mov    [codec.regs+esi], ax
                             <1>
                             <1>         ;call      codec_check_ready
                             <1>             ;retn
                             <1>     ;jmp    short _codec_check_ready
                             <1>
                             <1> codec_check_ready:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                             <1>
                             <1> _codec_check_ready:
                             <1>     ;mov    ecx, 20 ; total 2s
                             <1>     ; 06/08/2022
00013C11 29C9                <1>     sub     ecx, ecx
                             <1>     ;mov    cl, 20
                             <1>     ; 02/11/2023
00013C13 B10A                <1>     mov     cl, 10  ; wait 1s
                             <1> _ccr_wait:
00013C15 51                  <1>     push    ecx
                             <1>
00013C16 E88DFFFFFF          <1>         call    codec_io_r16
00013C1B A900000001          <1>         test    eax, VIA_REG_AC97_BUSY
00013C20 740B                <1>         jz      short _ccr_ok
                             <1>
00013C22 E84AFFFFFF          <1>     call    delay_100ms
                             <1>
00013C27 59                  <1>     pop     ecx
                             <1>
00013C28 49                  <1>     dec     ecx
00013C29 75EA                <1>         jnz     short _ccr_wait
                             <1>
00013C2B F9                  <1>         stc
00013C2C C3                  <1>         retn
                             <1>
                             <1> _ccr_ok:
00013C2D 59                  <1>     pop     ecx
00013C2E 25FFFF0000          <1>     and     eax, 0FFFFh
00013C33 C3                  <1>         retn
```

```
                                <1>
                                <1> codec_config:
                                <1>     ; 02/11/2023 - TRDOS 386 Kernel v2.0.7
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 10/06/2017
                                <1>     ; 29/05/2017
                                <1>     ; 24/04/2017
                                <1>     ; 21/04/2017
                                <1>     ; 16/04/2017 (TRDOS 386 Kernel)
                                <1>     ; 15/11/2016 ('codec.asm', 'player.com')
                                <1>     ; 14/11/2016
                                <1>     ; 12/11/2016 - Erdogan Tan
                                <1>     ;           (Ref: KolibriOS, 'setup_codec', codec.inc)
                                <1>
00013C34 B802020000             <1>     mov     eax, 0202h
00013C39 66A3[20890100]         <1>     mov     [audio_master_volume], ax
                                <1>
                                <1>     ;mov    ax, 1F1Fh ; 31,31
                                <1>     ; 02/11/2023
00013C3F 66B80B0B               <1>     mov     ax, 0B0Bh
                                <1>     ;mov    edx, CODEC_MASTER_VOL_REG ; 02h ; Line Out
                                <1>     ; 06/08/2022
00013C43 29D2                   <1>     sub     edx, edx
00013C45 B202                   <1>     mov     dl, CODEC_MASTER_VOL_REG ; 02h ; Line Out
00013C47 E8B1FFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 0202h
00013C4C 66B80202               <1>     mov     ax, 0202h
                                <1>     ;mov    edx, CODEC_PCM_OUT_REG ; 18h ; Wave Output (Stereo)
                                <1>     ; 06/08/2022
00013C50 29D2                   <1>     sub     edx, edx
00013C52 B218                   <1>     mov     dl, CODEC_PCM_OUT_REG ; 18h ; Wave Output (Stereo)
00013C54 E8A4FFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 0202h
00013C59 66B80202               <1>     mov     ax, 0202h
                                <1>     ;mov    edx, CODEC_AUX_VOL ; 04h ; CODEC_HP_VOL_REG ; HeadPhone
                                <1>     ; 06/08/2022
00013C5D 29D2                   <1>     sub     edx, edx
00013C5F B204                   <1>     mov     dl, CODEC_AUX_VOL ; 04h ; CODEC_HP_VOL_REG ; HeadPhone
00013C61 E897FFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 08h
                                <1>         ;mov    ax, 08h
00013C66 66B80880               <1>     mov     ax, 8008h ; Mute
                                <1>     ;mov    edx, 0Ch  ; AC97_PHONE_VOL ; TAD Input (Mono)
                                <1>     ; 06/08/2022
00013C6A 29D2                   <1>     sub     edx, edx
00013C6C B20C                   <1>     mov     dl, 0Ch  ; AC97_PHONE_VOL ; TAD Input (Mono)
00013C6E E88AFFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 0808h
00013C73 66B80808               <1>     mov     ax, 0808h
                                <1>     ;mov    edx, CODEC_LINE_IN_VOL_REG ; 10h ; Line Input (Stereo)
                                <1>     ; 06/08/2022
00013C77 29D2                   <1>     sub     edx, edx
00013C79 B210                   <1>     mov     dl, CODEC_LINE_IN_VOL_REG ; 10h ; Line Input (Stereo)
00013C7B E87DFFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 0808h
00013C80 66B80808               <1>     mov     ax, 0808h
                                <1>     ;mov    edx, CODEC_CD_VOL_REG ; 12h ; CR Input (Stereo)
                                <1>     ; 06/08/2022
00013C84 29D2                   <1>     sub     edx, edx ; 02/11/2023
00013C86 B212                   <1>     mov     dl, CODEC_CD_VOL_REG ; 12h ; CR Input (Stereo)
00013C88 E870FFFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1>     ;mov    eax, 0808h
00013C8D 66B80808               <1>     mov     ax, 0808h
                                <1>         ;mov       edx, CODEC_AUX_VOL_REG ; 16h ; Aux Input (Stereo)
                                <1>     ; 06/08/2022
00013C91 29D2                   <1>     sub     edx, edx
00013C93 B216                   <1>     mov     dl, CODEC_AUX_VOL_REG ; 16h ; Aux Input (Stereo)
                                <1>     ;call   codec_write
                                <1>     ;;jc    short cconfig_error
00013C95 E963FFFFFF             <1>     jmp     codec_write ; 10/06/2017
                                <1>
                                <1> ;   ; Extended Audio Status (2Ah)
                                <1> ;   mov     eax, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
                                <1> ;   call    codec_read
                                <1> ;   and     eax, 0FFFFh - 2              ; clear DRA (BIT1)
                                <1> ;   ;or     eax, 1                       ; set VRA (BIT0)
                                <1> ;   or      eax, 5         ; VRA (BIT0) & S/PDIF (BIT2) ; 14/11/2016
                                <1> ;   mov     edx, CODEC_EXT_AUDIO_CTRL_REG
                                <1> ;   call    codec_write
                                <1> ;   ;jc     short cconfig_error
                                <1> ;
                                <1> ;set_sample_rate:
                                <1> ;   ;movzx  eax, word [audio_freq]
                                <1> ;   mov     ax, [audio_freq]
                                <1> ;   mov     edx, CODEC_PCM_FRONT_DACRATE_REG ; 2Ch ; PCM Front DAC Rate
                                <1> ;   ;call   codec_write
                                <1> ;   ;retn
                                <1> ;   jmp     codec_write
                                <1>
                                <1> ;cconfig_error:
                                <1> ;   retn
                                <1>
                                <1> vt8233_int_handler:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 27/07/2020
                                <1>     ; 22/07/2020
                                <1>     ; Interrupt Handler for VIA VT8237R Audio Controller
                                <1>     ; Note: called by 'dev_IRQ_service'
                                <1>     ; 14/10/2017
                                <1>     ; 09/10/2017, 10/10/2017, 12/10/2017
                                <1>     ; 13/06/2017
                                <1>     ; 21/04/2017 (TRDOS 386 kernel, 'audio.s')
                                <1>     ; 24/03/2017 - 'PLAYER.COM' ('player.asm')
                                <1>
                                <1>     ;push   eax ; * must be saved !
                                <1>     ;push   edx
                                <1>     ;push   ecx
                                <1>     ;push   ebx ; * must be saved !
                                <1>     ;push   esi
                                <1>     ;push   edi
                                <1>
```

```
                            <1>     ;cmp   byte [audio_busy], 1
                            <1>     ;jnb   short _ih0 ; 09/10/2017
                            <1>
                            <1>     ;mov   byte [audio_flag_eol], 0
                            <1>
00013C9A 66BA0000           <1>         mov     dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_STATUS
00013C9E E81CFFFFFF         <1>         call    ctrl_io_r8
                            <1>
00013CA3 A880               <1>     test    al, VIA_REG_STAT_ACTIVE
00013CA5 7417               <1>         jz      short _ih0 ; 09/10/2017
                            <1>
00013CA7 2407               <1>         and     al, VIA_REG_STAT_EOL + VIA_REG_STAT_FLAG + VIA_REG_STAT_STOPPED
00013CA9 A2[1F890100]       <1>     mov    [audio_flag_eol], al
00013CAE 740E               <1>         jz  short _ih0 ; 09/10/2017
                            <1>
                            <1>     ; 09/10/2017
                            <1>     ;mov   byte [audio_busy], 1
                            <1>
00013CB0 803D[1E890100]01   <1>     cmp    byte [audio_play_cmd], 1
00013CB7 7315               <1>     jnb    short _ih1 ; 10/10/2017
                            <1>
00013CB9 E848000000         <1>     call   channel_reset
                            <1> _ih0:
                            <1>     ; 09/10/2017
00013CBE A0[1F890100]       <1>         mov     al, [audio_flag_eol]   ;; ack ;;
00013CC3 66BA0000           <1>         mov     dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_STATUS
00013CC7 E8EAFEFFFF         <1>         call    ctrl_io_w8
00013CCC EB37               <1>     jmp    short _ih4
                            <1> _ih1:
                            <1> vt8233_tuneLoop:
00013CCE A0[1F890100]       <1>         mov     al, [audio_flag_eol]   ;; ack ;;
00013CD3 66BA0000           <1>         mov     dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_STATUS
00013CD7 E8DAFEFFFF         <1>         call    ctrl_io_w8
                            <1>
                            <1>     ; 22/07/2020
                            <1>     ;; 12/10/2017
                            <1>     ;mov   byte [audio_flag], 0 ; Reset
                            <1>
                            <1>     ; 10/10/2017
                            <1>     ; 09/10/2017
                            <1>     ;test  byte [audio_flag_eol], VIA_REG_STAT_FLAG
                            <1>     ;jz    short _ih2 ; EOL
                            <1>
                            <1>     ; 22/07/2020
                            <1>     ; 14/10/2017
                            <1>     ;test  byte [audio_flag_eol], VIA_REG_STAT_EOL
                            <1>     ;jnz   short _ih2 ; EOL
                            <1>     ;                 ; (Half Buffer 2 has been completed
                            <1>     ;                 ; and Half Buffer 1 will be played.)
                            <1>
                            <1>     ; FLAG
                            <1>     ; (Half Buffer 1 has been completed
                            <1>     ;  and Half Buffer 2 will be played.)
                            <1>
                            <1>     ; 14/10/2017
                            <1>     ;; (Continue to play.)
                            <1>     ;mov   al, VIA_REG_CTRL_INT
                            <1>            ;or    al, VIA_REG_CTRL_START
                            <1>            ;mov   dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_CONTROL
                            <1>         ;call      ctrl_io_w8
                            <1>     ; 12/10/2017
                            <1>     ;mov   byte [audio_flag], 1
                            <1>
                            <1>     ; 22/07/2020
                            <1>     ;inc   byte [audio_flag] ; = 1
                            <1> _ih2:
                            <1>     ; 10/10/2017
00013CDC 8B3D[04890100]     <1>     mov    edi, [audio_dma_buff]
                            <1>
                            <1>     ; 05/06/2024
                            <1>     ; 04/06/2024
                            <1>     ;;mov  ecx, [audio_dmabuff_size]
                            <1>     ;;shr  ecx, 1 ; dma buff size / 2 = half buffer size
                            <1>     ;mov   ecx, [audio_buff_size]
                            <1>     ;and   cl, ~1 ; word aligned
                            <1>     ; 05/06/2024
00013CE2 8B0D[0C890100]     <1>     mov    ecx, [dma_hbuff_size] ; half buffer size
                            <1>
                            <1>     ; 22/07/2020
                            <1>     ; 12/10/2017
                            <1>     ;cmp   byte [audio_flag], 0
                            <1>     ;ja    short _ih3 ; Playing Half Buffer 2 (Current: FLAG)
                            <1>
                            <1>     ; 27/07/2020
                            <1>     ; 22/07/2020
00013CE8 F605[10890100]01   <1>     test   byte [audio_flag], 1  ; Current flag value
00013CEF 7402               <1>     jz     short _ih3 ; Half Buffer 1 must be filled
                            <1>
                            <1>     ; Half Buffer 2 must be filled
00013CF1 01CF               <1>     add    edi, ecx
                            <1> _ih3:
                            <1>     ; Update half buffer 2 while playing half buffer 1
                            <1>     ; Update half buffer 1 while playing half buffer 2
                            <1>
00013CF3 8B35[FC880100]     <1>     mov    esi, [audio_p_buffer] ; phy addr of audio buff
                            <1>     ; 05/06/2024
                            <1>     ;shr   ecx, 2 ; half buff size / 4
                            <1>     ;rep   movsd
00013CF9 D1E9               <1>     shr    ecx, 1 ; word aligned buffer
00013CFB 66F3A5             <1>     rep    movsw
                            <1>
                            <1>     ; switch flag value ;
00013CFE 8035[10890100]01   <1>     xor    byte [audio_flag], 1
                            <1>     ; 12/10/2017
                            <1>     ; [audio_flag] = 0 : Playing dma half buffer 2
                            <1>                        ; Next buffer (to update) is dma half buff 1
                            <1>     ;              = 1 : Playing dma half buffer 1
                            <1>                        ; Next buffer (to update) is dma half buff 2
                            <1> _ih4:
                            <1>     ; 28/05/2017
                            <1>     ;mov   byte [audio_busy], 0 ; 09/10/2017
                            <1>     ;
                            <1>     ;pop   edi
                            <1>     ;pop   esi
                            <1>     ;pop   ebx ; * must be restored !
                            <1>     ;pop   ecx
                            <1>     ;pop   edx
                            <1>     ;pop   eax ; * must be restored !
                            <1>
00013D05 C3                 <1>     retn
                            <1>
                            <1> channel_reset:
                            <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                            <1>     ; 24/06/2017
```

```
                            <1>     ; 29/05/2017
                            <1>     ; 23/03/2017
                            <1>     ; 14/11/2016 - Erdogan Tan
                            <1>     ; 12/11/2016 - Erdogan Tan (Ref: KolibriOS, vt823x.asm)
                            <1>         ;mov       edx, VIA_REG_OFFSET_CONTROL
                            <1>     ; 06/08/2022
00013D06 29D2               <1>     sub    edx, edx
00013D08 B201               <1>     mov    dl, VIA_REG_OFFSET_CONTROL
                            <1>     ;mov   eax, VIA_REG_CTRL_PAUSE + VIA_REG_CTRL_TERMINATE + VIA_REG_CTRL_RESET
00013D0A B848000000         <1>         mov eax, VIA_REG_CTRL_PAUSE + VIA_REG_CTRL_TERMINATE ; 24/06/2017
00013D0F E8A2FEFFFF         <1>     call    ctrl_io_w8
                            <1>
                            <1>         ;mov       edx, VIA_REG_OFFSET_CONTROL
                            <1>         ;call   ctrl_io_r8
                            <1>
                            <1>     ; wait for 50 ms
                            <1>     ;mov   ecx, 160 ; (200*0.25 ms) ; 29/05/2017
                            <1>     ; 06/08/2022
00013D14 31C9               <1>     xor    ecx, ecx
00013D16 B1A0               <1>     mov    cl, 160
                            <1> _ch_rst_wait:
00013D18 E861FEFFFF         <1>     call   delay1_4ms
00013D1D 49                 <1>     dec    ecx
00013D1E 75F8               <1>     jnz    short _ch_rst_wait
                            <1>
                            <1>         ; disable interrupts
                            <1>     ;mov   edx, VIA_REG_OFFSET_CONTROL
                            <1>         ; 06/08/2022
00013D20 29D2               <1>     sub    edx, edx
00013D22 B201               <1>     mov    dl, VIA_REG_OFFSET_CONTROL
00013D24 31C0               <1>     xor     eax, eax
00013D26 E88BFEFFFF         <1>         call    ctrl_io_w8
                            <1>
                            <1>         ; clear interrupts
                            <1>         ;mov       edx, VIA_REG_OFFSET_STATUS
                            <1>     ; 06/08/2022
00013D2B 29D2               <1>     sub    edx, edx
                            <1>     ;mov   dl, VIA_REG_OFFSET_STATUS ; 0
                            <1>     ; edx = 0
                            <1>     ;mov   eax, 3
                            <1>         ; 06/08/2022
00013D2D 29C0               <1>     sub    eax, eax
00013D2F B003               <1>     mov    al, 3
                            <1>     ; eax = 3
00013D31 E880FEFFFF         <1>     call   ctrl_io_w8
                            <1>
                            <1>     ;mov   edx, VIA_REG_OFFSET_CURR_PTR
                            <1>     ;xor   eax, eax
                            <1>     ;call  ctrl_io_w32
                            <1>
00013D36 C3                 <1>         retn
                            <1>
                            <1> vt8233_stop: ; 22/04/2017
00013D37 C605[1E890100]00   <1>     mov    byte [audio_play_cmd], 0 ; stop !
                            <1> _tlp2:
                            <1>     ; 24/06/2017
                            <1>         ; finished with song, stop everything
                            <1>     ;mov   al, VIA_REG_CTRL_INT
                            <1>         ;or al, VIA_REG_CTRL_TERMINATE
                            <1>     ;mov   dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_CONTROL
                            <1>         ;call      ctrl_io_w8
                            <1>
                            <1>         ;call      channel_reset
                            <1>     ;retn
                            <1>
00013D3E EBC6               <1>     jmp    short channel_reset
                            <1>
                            <1> set_vt8233_bdl: ; Set VT8237R Buffer Descriptor List
                            <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                            <1>     ; 22/07/2020 - TRDOS 386 v2.0.2
                            <1>     ; 28/05/2017
                            <1>     ; 21/04/2017 (TRDOS 386 kernel, 'audio.s')
                            <1>     ; 24/03/2017 - 'PLAYER.COM' ('via_wav.asm' - 29/11/2016)
                            <1>
                            <1>     ; eax = dma buffer address = [audio_DMA_buff]
                            <1>     ; ecx = dma buffer buffer size = [audio_dmabuff_size]
                            <1>
                            <1>     ; 04/06/2024
                            <1>     ; ecx = DMA half buffer size (same as audio buffer size)
                            <1>
                            <1>     ;shr   ecx, 1 ; dma half buffer size
00013D40 89CE               <1>     mov    esi, ecx
                            <1>
00013D42 BF[28890100]       <1>         mov     edi, audio_bdl_buff      ; get BDL address
                            <1>     ;mov   ecx, 32 / 2           ; make 32 entries in BDL
                            <1>     ; 06/08/2022
00013D47 29C9               <1>     sub    ecx, ecx
00013D49 B110               <1>     mov    cl, 16
                            <1>
00013D4B EB05               <1>     jmp    short s_vt8233_bdl1
                            <1>
                            <1> s_vt8233_bdl0:
                            <1>     ; set buffer descriptor 0 to start of data file in memory
                            <1>
00013D4D A1[04890100]       <1>     mov    eax, [audio_dma_buff] ; Physical address of DMA buffer
                            <1>
                            <1> s_vt8233_bdl1:
00013D52 AB                 <1>     stosd                        ; store dmabuffer1 address
                            <1>
00013D53 89C2               <1>     mov    edx, eax
                            <1>
                            <1> ; VIA VT8235.PDF: (Page 110) (Erdogan Tan, 29/11/2016)
                            <1>     ;
                            <1>     ;      Audio SGD Table Format
                            <1>     ;      -------------------------------
                            <1>     ;      63   62    61-56    55-32  31-0
                            <1>     ;      --   --   --------  -----  ----
                            <1>     ;      EOL FLAG -reserved- Base   Base
                            <1>     ;                         Count  Address
                            <1>     ;                          [23:0] [31:0]
                            <1>     ;      EOL: End Of Link.
                            <1>     ;           1 indicates this block is the last of the link.
                            <1>     ;           If the channel “Interrupt on EOL” bit is set, then
                            <1>     ;           an interrupt is generated at the end of the transfer.
                            <1>     ;
                            <1>     ;      FLAG: Block Flag. If set, transfer pauses at the end of this
                            <1>     ;            block. If the channel “Interrupt on FLAG” bit is set,
                            <1>     ;            then an interrupt is generated at the end of this block.
                            <1>
00013D55 89F0               <1>     mov    eax, esi ; DMA half buffer size
                            <1>
                            <1>            ; 04/06/2024 - Erdogan Tan
                            <1>            ; NOTE: I have changed DMA half buffer size to
```

```
                                <1>             ; (word aligned) audio buffer size for smooth audio playing
                                <1>             ; (it was page border aligned before June 4, 2024)
                                <1>             ;
                                <1>             ; For example:
                                <1>             ;    For 65416 bytes audio buffer (22kHz, 16bit samples)
                                <1>             ;       (user's audio buff virtual addr is mapped to physical)
                                <1>             ;      ((kernel's and user's audio buff pages are same))
                                <1>             ;    1) Memory allocation (of it) is 65536 bytes
                                <1>             ;    2) DMA half buffer size is 65416 bytes
                                <1>             ;      (it would be 65536 bytes before this modification)
                                <1>             ;      ((additional 140 bytes would cause to a noise))
                                <1>             ;    3) Total DMA buffer size is 131072 bytes
                                <1>             ;       (the last 240 bytes will not be used for playing)
                                <1>
00013D57 01C2                   <1>     add     edx, eax
00013D59 0D00000040             <1>     or      eax, FLAG
                                <1>     ;or     eax, EOL
00013D5E AB                     <1>     stosd
                                <1>
                                <1> ; 2nd buffer:
                                <1>
00013D5F 89D0                   <1>         mov eax, edx ; Physical address of the 2nd half of DMA buffer
00013D61 AB                     <1>     stosd           ; store dmabuffer2 address
                                <1>
                                <1> ; set length to [audio_dmabuff_size]/2
                                <1> ; Set control (bits 31:16) to BUP, bits 15:0=number of samples
                                <1> ;
00013D62 89F0                   <1>     mov     eax, esi ; DMA half buffer size
                                <1>     ; 22/07/2020
                                <1>     ;or     eax, EOL
00013D64 0D00000040             <1>     or      eax, FLAG
00013D69 AB                     <1>     stosd
                                <1>
00013D6A E2E1                   <1>     loop    s_vt8233_bdl0
                                <1>
                                <1>     ; 22/07/2020
00013D6C 814FFC00000080         <1>     or      dword [edi-4], EOL
                                <1>
00013D73 C3                     <1>     retn
                                <1>
                                <1> vt8233_start_play:
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 01/09/2020
                                <1>     ; 22/07/2020
                                <1>     ; start to play audio data via VT8233 audio controller
                                <1>     ; 13/06/2017
                                <1>     ; 10/06/2017
                                <1>     ; 24/04/2017
                                <1>     ; 21/04/2017 (TRDOS 386 kernel, 'audio.s')
                                <1>     ; 24/03/2017 - 'PLAYER.COM' ('via_wav.asm' - 29/11/2016)
                                <1>     ; write buffer descriptor list address
                                <1>
                                <1>     ; Extended Audio Status (2Ah)
00013D74 B82A000000             <1>     mov     eax, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
00013D79 E85CFEFFFF             <1>     call    codec_read
00013D7E 25FDFF0000             <1>     and      eax, 0FFFFh - 2              ; clear DRA (BIT1)
                                <1>     ;or      eax, 1               ; set VRA (BIT0)
                                <1>     ;or     eax, 5         ; VRA (BIT0) & S/PDIF (BIT2) ; 14/11/2016
00013D83 0C05                   <1>     or      al, 5
                                <1>     ; 01/09/2020
                                <1>     ;or     eax, 3805h ; AD1980 (PRK, PRJ, PRI = 1 .. only front DAC)
                                <1>     ; 01/09/2020
                                <1>     ;mov    edx, CODEC_EXT_AUDIO_CTRL_REG
                                <1>     ;cmp    word [audio_freq], 0BB80h ; 48 kHz
                                <1>     ;jne    short set_extd_audio_status_1
                                <1>     ;and    al, 0FEh ; disable VRA bit (set sample rate to 48000 Hz)
                                <1>     ;jmp    short set_extd_audio_status_2
                                <1> ;set_extd_audio_status_1:
00013D85 BA2A000000             <1>     mov     edx, CODEC_EXT_AUDIO_CTRL_REG
00013D8A E86EFEFFFF             <1>     call    codec_write
                                <1>     ;jc     short cconfig_error
                                <1>
                                <1> set_sample_rate:
                                <1>     ;movzx  eax, word [audio_freq]
00013D8F 66A1[1A890100]         <1>     mov     ax, [audio_freq]
00013D95 BA2C000000             <1>     mov     edx, CODEC_PCM_FRONT_DACRATE_REG ; 2Ch ; PCM Front DAC Rate
                                <1> ;set_extd_audio_status_2:
00013D9A E85EFEFFFF             <1>     call    codec_write
                                <1>
                                <1>     ; 01/09/2020
                                <1>     ; set AD1980 MCB register (Index 76h) to 0C00h
                                <1>     ; (CLDIS, HPSEL)
                                <1>     ;mov    ax, 0C00h
                                <1>     ;mov    edx, CODEC_MISC_CRTL_BITS_REG ; 76h
                                <1>     ;                       ; Miscellaneous Control Bit Register
                                <1>     ;call   codec_write
                                <1>     ;
                                <1>
00013D9F B8[28890100]           <1>         mov eax, audio_bdl_buff
                                <1>
                                <1>     ; 12/11/2016 - Erdogan Tan
                                <1>     ; (Ref: KolibriOS, vt823x.asm, 'create_primary_buff')
                                <1>     ;mov    edx, VIADEV_PLAYBACK + VIA_REG_OFFSET_TABLE_PTR
                                <1>         ; 06/08/2022
00013DA4 29D2                   <1>     sub     edx, edx
                                <1>     ;mov    dl, VIADEV_PLAYBACK + VIA_REG_OFFSET_TABLE_PTR
                                <1>     ; edx = 0
00013DA6 E81DFEFFFF             <1>     call    ctrl_io_w32
                                <1>
                                <1>     ;call   codec_check_ready
                                <1>
00013DAB 66BA0200               <1>     mov     dx, VIADEV_PLAYBACK + VIA_REG_OFS_PLAYBACK_VOLUME_L
                                <1>         ;mov       eax, 2  ; 31
00013DAF B01F                   <1>     mov     al, 31
00013DB1 2A05[20890100]         <1>         sub al, [audio_master_volume_l]
00013DB7 E8FAFDFFFF             <1>     call    ctrl_io_w8
                                <1>
                                <1>     ;call   codec_check_ready
                                <1>
00013DBC 66BA0300               <1>         mov     dx, VIADEV_PLAYBACK + VIA_REG_OFS_PLAYBACK_VOLUME_R
                                <1>         ;mov       ax, 2   ; 31
00013DC0 B01F                   <1>     mov     al, 31
00013DC2 2A05[21890100]         <1>         sub al, [audio_master_volume_r]
00013DC8 E8E9FDFFFF             <1>     call     ctrl_io_w8
                                <1>
                                <1>     ;call   codec_check_ready
                                <1> ;
                                <1> ;
                                <1> ; All set. Let's play some music.
                                <1> ;
                                <1> ;
                                <1>             ;mov    dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_STOP_IDX
                                <1>         ;mov    ax, VIA8233_REG_TYPE_16BIT or VIA8233_REG_TYPE_STEREO or 0xfffff or 0xff000000
                                <1>         ;call   ctrl_io_w32
```

```
                                <1>
                                <1>     ;call  codec_check_ready
                                <1>
                                <1>     ; 08/12/2016
                                <1>     ; 07/10/2016
                                <1>         ;;mov    al, 1
                                <1>         ;mov      al, 31
                                <1>     ; 22/07/2020
00013DCD B0FF                   <1>     mov    al, 0FFh
00013DCF E813000000             <1>     call    set_VT8233_LastValidIndex
                                <1>
00013DD4 C605[1E890100]01       <1>     mov    byte [audio_play_cmd], 1 ; play command (do not stop) !
                                <1>
                                <1>     ; 22/07/2020
                                <1>     ;mov   byte [audio_flag], 0  ; clear half buffer flag
                                <1>
                                <1> vt8233_play: ; continue to play
                                <1>     ; 22/04/2017
                                <1>     ;mov   al, VIA_REG_CTRL_INT
                                <1>            ;or    al, VIA_REG_CTRL_START
                                <1>         ;;mov      al, VIA_REG_CTRL_AUTOSTART + VIA_REG_CTRL_START
                                <1>     ; 22/07/2020
00013DDB B0A1                   <1>     mov    al, VIA_REG_CTRL_AUTOSTART + VIA_REG_CTRL_START + VIA_REG_CTRL_INT_FLAG
                                <1>
00013DDD 66BA0100               <1>     mov     dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_CONTROL
00013DE1 E8D0FDFFFF             <1>         call    ctrl_io_w8
                                <1>     ;call  codec_check_ready
                                <1>     ;retn
                                <1>     ;jmp   codec_check_ready
00013DE6 C3                     <1>     retn
                                <1>
                                <1> ;input AL = index # to stop on
                                <1> set_VT8233_LastValidIndex:
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 23/07/2020
                                <1>     ; 10/06/2017
                                <1>     ; 21/04/2017 (TRDOS 386 kernel, 'audio.s')
                                <1>     ; 24/03/2017 - 'PLAYER.COM' ('via_wav.asm' - 29/11/2016)
                                <1>     ; 19/11/2016
                                <1>     ; 14/11/2016 - Erdogan Tan (Ref: VIA VT8235.PDF, Page 110)
                                <1>     ; 12/11/2016 - Erdogan Tan
                                <1>     ; (Ref: KolibriOS, vt823x.asm, 'create_primary_buff')
                                <1>     ;push  edx
                                <1>     ;push  ax
00013DE7 50                     <1>     push   eax ; 23/07/2020
                                <1>     ;push  ecx
00013DE8 0FB705[1A890100]       <1>     movzx  eax, word [audio_freq] ; Hertz
00013DEF BA00001000             <1>     mov    edx, 100000h ; 2^20 = 1048576
00013DF4 F7E2                   <1>     mul    edx
00013DF6 B980BB0000             <1>     mov    ecx, 48000
00013DFB F7F1                   <1>     div    ecx
                                <1>     ;and   eax, 0FFFFFh
                                <1>     ;pop   ecx
                                <1>     ;pop   dx
00013DFD 5A                     <1>     pop    edx ; 23/07/2020
00013DFE C1E218                 <1>     shl    edx, 24  ; STOP Index Setting: Bit 24 to 31
00013E01 09D0                   <1>     or     eax, edx
                                <1>     ; 19/11/2016
00013E03 803D[18890100]10       <1>     cmp    byte [audio_bps], 16
00013E0A 7505                   <1>     jne    short sLVI_1
00013E0C 0D00002000             <1>     or     eax, VIA8233_REG_TYPE_16BIT
                                <1> sLVI_1:
00013E11 803D[19890100]02       <1>     cmp    byte [audio_stmo], 2
00013E18 7505                   <1>     jne    short sLVI_2
00013E1A 0D00001000             <1>     or     eax, VIA8233_REG_TYPE_STEREO
                                <1> sLVI_2:
                                <1>     ;mov   edx, VIADEV_PLAYBACK + VIA_REG_OFFSET_STOP_IDX
                                <1>     ; 06/08/2022
00013E1F 29D2                   <1>     sub    edx, edx
00013E21 B208                   <1>     mov    dl, VIADEV_PLAYBACK + VIA_REG_OFFSET_STOP_IDX
00013E23 E8A0FDFFFF             <1>     call    ctrl_io_w32
                                <1>     ;call  codec_check_ready
                                <1>     ;pop   edx
00013E28 C3                     <1>     retn
                                <1>
                                <1> vt8233_pause: ; pause
                                <1>     ; 10/06/2017
                                <1>     ; 22/04/2017
                                <1>     ;mov   al, VIA_REG_CTRL_INT
                                <1>         ;or al, VIA_REG_CTRL_PAUSE
                                <1>     ; 23/07/2020
00013E29 B029                   <1>     mov    al, VIA_REG_CTRL_PAUSE+VIA_REG_CTRL_INT_FLAG+VIA_REG_CTRL_AUTOSTART
                                <1>
00013E2B 66BA0100               <1>     mov     dx, VIADEV_PLAYBACK + VIA_REG_OFFSET_CONTROL
00013E2F E882FDFFFF             <1>         call    ctrl_io_w8
                                <1>     ;call  codec_check_ready
                                <1>     ;retn
                                <1>     ;jmp   codec_check_ready
00013E34 C3                     <1>     retn
                                <1>
                                <1> vt8233_reset:
                                <1>     ; 22/04/2017
                                <1>     ; reset VT8237R (vt8233) Audio Controller
                                <1>     ;cmp   byte [audio_play_cmd], 1
                                <1>     ;jna   short vt8233_rst_0
00013E35 C605[1E890100]00       <1>     mov    byte [audio_play_cmd], 0 ; stop !
                                <1> vt8233_rst_0:
00013E3C E8DAFCFFFF             <1>     call   reset_codec
00013E41 720A                   <1>     jc     short vt8233_rst_1 ; codec error !
                                <1>     ; eax = 1
00013E43 E852FDFFFF             <1>     call   codec_io_w16 ; w32
00013E48 E8B9FEFFFF             <1>     call   channel_reset
                                <1> vt8233_rst_1:
                                <1> vt8233_vol_1:      ; 06/08/2022
00013E4D C3                     <1>     retn
                                <1>
                                <1> vt8233_volume:
                                <1>     ; set VT8237R (vt8233) sound volume level
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 24/04/2017
                                <1>     ; 22/04/2017
                                <1>     ; bl = component (0 = master/playback/lineout volume)
                                <1>     ; cl = left channel volume level (0 to 31)
                                <1>     ; ch = right channel volume level (0 to 31)
                                <1>
00013E4E 08DB                   <1>     or     bl, bl
00013E50 75FB                   <1>     jnz    short vt8233_vol_1 ; temporary !
00013E52 66B81F1F               <1>     mov    ax, 1F1Fh ; 31,31
00013E56 38C1                   <1>     cmp    cl, al
00013E58 77F3                   <1>     ja     short vt8233_vol_1 ; temporary !
00013E5A 38E5                   <1>     cmp    ch, ah
00013E5C 77EF                   <1>     ja     short vt8233_vol_1 ; temporary !
00013E5E 66890D[20890100]       <1>     mov    [audio_master_volume], cx
00013E65 6629C8                 <1>     sub    ax, cx
```

```
                            <1>     ;mov    edx, CODEC_MASTER_VOL_REG ; 02h ; Line Out
                            <1>     ; 06/08/2022
00013E68 29D2               <1>     sub     edx, edx
00013E6A B202               <1>     mov     dl, CODEC_MASTER_VOL_REG ; 02h ; Line Out
                            <1>     ; 06/08/2022
00013E6C E98CFDFFFF         <1>     jmp     codec_write
                            <1>     ;call   codec_write
                            <1> ;vt8233_vol_1:
                            <1>     ;retn
                            <1>
                            <1> ; CODE for SOUND BLASTER 16
                            <1>
                            <1> DetectSB:
                            <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                            <1>     ; 24/04/2017
                            <1>     ;pushad
                            <1> ScanPort:
                            <1>     ; 06/08/2022
00013E71 66BB1002           <1>     mov     bx, 0210h       ; start scanning ports
                            <1>                             ; 210h, 220h, .. 260h
                            <1>     ; 06/08/2022
00013E75 31C9               <1>     xor     ecx, ecx
00013E77 88FE               <1>     mov     dh, bh
                            <1> ResetDSP:
                            <1>     ;mov    dx, bx          ; try to reset the DSP.
                            <1>     ;add    dx, 06h
                            <1>     ; 06/08/2022
00013E79 88DA               <1>     mov     dl, bl
00013E7B 80C206             <1>     add     dl, 06h
                            <1>
00013E7E B001               <1>     mov     al, 1
00013E80 EE                 <1>     out     dx, al
                            <1>
00013E81 EC                 <1>     in      al, dx
00013E82 EC                 <1>     in      al, dx
00013E83 EC                 <1>     in      al, dx
00013E84 EC                 <1>     in      al, dx
                            <1>
00013E85 30C0               <1>     xor     al, al
00013E87 EE                 <1>     out     dx, al
                            <1>
                            <1>     ;add    dx, 08h
                            <1>     ; 06/08/2022
00013E88 80C208             <1>     add     dl, 08h
                            <1>     ;mov    cx, 100
00013E8B B164               <1>     mov     cl, 100
                            <1> WaitID:
00013E8D EC                 <1>     in      al, dx
00013E8E 08C0               <1>     or      al, al
00013E90 7804               <1>     js      short GetID
00013E92 E2F9               <1>     loop    WaitID
00013E94 EB0D               <1>     jmp     short NextPort
                            <1> GetID:
                            <1>     ;sub    dx, 04h
                            <1>     ; 06/08/2022
00013E96 80EA04             <1>     sub     dl, 04h
00013E99 EC                 <1>     in      al, dx
00013E9A 3CAA               <1>     cmp     al, 0AAh
00013E9C 740F               <1>     je      short Found
                            <1>     ;add    dx, 04h
                            <1>     ; 06/08/2022
00013E9E 80C204             <1>     add     dl, 04h
00013EA1 E2EA               <1>     loop    WaitID
                            <1> NextPort:
                            <1>     ;add    bx, 10h         ; if not response,
                            <1>     ; 06/08/2022
00013EA3 80C310             <1>     add     bl, 10h
                            <1>     ;cmp    bx, 260h        ; try the next port.
00013EA6 80FB60             <1>     cmp     bl, 60h
00013EA9 76CE               <1>     jbe     short ResetDSP
00013EAB F9                 <1>     stc
00013EAC C3                 <1>     retn
                            <1> Found:
00013EAD 66891D[EA880100]   <1>     mov     [audio_io_base], bx   ; SB Port Address Found!
                            <1> ScanIRQ:
                            <1> SetIrqs:
00013EB4 28C0               <1>     sub     al, al ; 0
00013EB6 A2[E0880100]       <1>     mov     [IRQnum], al ; reset
00013EBB A2[E6880100]       <1>     mov     [audio_intr], al ; reset
                            <1>
                            <1>     ; ah > 0 -> set IRQ vector
                            <1>     ; al = IRQ number
                            <1>     ;mov    ax, 103h ; IRQ 3
                            <1>     ;call   set_hardware_int_vector
                            <1>     ;mov    ax, 104h ; IRQ 4
                            <1>     ;call   set_hardware_int_vector
00013EC0 66B80501           <1>     mov     ax, 105h ; IRQ 5
00013EC4 E853DFFFFF         <1>     call    set_hardware_int_vector
00013EC9 66B80701           <1>     mov     ax, 107h ; IRQ 7
00013ECD E84ADFFFFF         <1>     call    set_hardware_int_vector
                            <1>
00013ED2 668B15[EA880100]   <1>     mov     dx, [audio_io_base] ; tells to the SB to
                            <1>     ;add    dx, 0Ch             ; generate a IRQ!
                            <1>     ; 06/08/2022
00013ED9 80C20C             <1>     add     dl, 0Ch
                            <1> WaitSb:
00013EDC EC                 <1>     in      al, dx
00013EDD 08C0               <1>     or      al, al
00013EDF 78FB               <1>     js      short WaitSb
00013EE1 B0F2               <1>     mov     al, 0F2h
00013EE3 EE                 <1>     out     dx, al
                            <1>
00013EE4 31C9               <1>     xor     ecx, ecx        ; wait until IRQ level
                            <1> WaitIRQ:
00013EE6 A0[E0880100]       <1>     mov     al, [IRQnum]
00013EEB 3C00               <1>     cmp     al, 0 ; is changed or timeout.
00013EED 7706               <1>     ja      short IrqOk
00013EEF 6649               <1>     dec     cx
00013EF1 75F3               <1>     jnz     short WaitIRQ
00013EF3 EB14               <1>     jmp     short RestoreIrqs
                            <1> IrqOk:
00013EF5 A2[E6880100]       <1>     mov     [audio_intr], al ; set
00013EFA 668B15[EA880100]   <1>     mov     dx, [audio_io_base]
                            <1>     ;add    dx, 0Eh
                            <1>     ; 06/08/2022
00013F01 80C20E             <1>     add     dl, 0Eh
00013F04 EC                 <1>     in      al, dx  ; SB acknowledge.
00013F05 B020               <1>     mov     al, 20h
00013F07 E620               <1>     out     20h, al ; Hardware acknowledge.
                            <1>
                            <1> RestoreIrqs:
                            <1>     ; ah = 0 -> reset IRQ vector
                            <1>     ; al = IRQ number
                            <1>     ;mov    ax, 3 ; IRQ 3
```

```
                                <1>     ;call   set_hardware_int_vector
                                <1>     ;mov    ax, 4 ; IRQ 4
                                <1>     ;call   set_hardware_int_vector
00013F09 66B80500               <1>     mov     ax, 5 ; IRQ 5
00013F0D E80ADFFFFF             <1>     call    set_hardware_int_vector
00013F12 66B80700               <1>     mov     ax, 7 ; IRQ 7
00013F16 E801DFFFFF             <1>     call    set_hardware_int_vector
                                <1>
00013F1B 31D2                   <1>     xor     edx, edx
00013F1D 8915[EC880100]         <1>     mov     [audio_dev_id], edx ; 0
00013F23 8915[F0880100]         <1>     mov     [audio_vendor], edx ; 0
                                <1>     ; 22/11/2023
                                <1>     ;mov    [audio_stats_cmd], edx ; 0
                                <1>
                                <1>     ;popad
                                <1>
00013F29 803D[E6880100]01       <1>     cmp     byte [audio_intr], 1 ; IRQ level was changed?
                                <1>
00013F30 C3                     <1>     retn
                                <1>
                                <1> %macro      SbOut   1
                                <1> %%Wait:
                                <1>     in      al, dx
                                <1>     or      al, al
                                <1>     js      short %%Wait
                                <1>     mov     al, %1
                                <1>     out     dx, al
                                <1> %endmacro
                                <1>
                                <1> SbInit_play:
                                <1>     ; 28/01/2025 - TRDOS 386 Kernel v2.0.10
                                <1>     ; 06/08/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 22/10/2017
                                <1>     ; 20/10/2017
                                <1>     ; 06/10/2017
                                <1>     ; 13/07/2017 - 09/08/2017
                                <1>     ; 24/04/2017 - 15/05/2017 - 24/06/2017
                                <1>     ;pushad
                                <1> SetBuffer:
                                <1>     ;mov    byte [DmaFlag], 0
                                <1>
00013F31 8B1D[04890100]         <1>     mov     ebx, [audio_dma_buff] ; physical addr of DMA buff
00013F37 89DF                   <1>     mov     edi, ebx
                                <1>     ;mov    ecx, [audio_dmabuff_size]
                                <1>     ; 28/01/2025 (BugFix)
00013F39 8B0D[0C890100]         <1>     mov     ecx, [dma_hbuff_size]
                                <1>     ;shl    ecx, 1 ; * 2 ; *!*
                                <1>
00013F3F 803D[18890100]10       <1>     cmp     byte [audio_bps], 16
00013F46 752F                   <1>     jne     short sbInit_0 ; set 8 bit DMA buffer
                                <1>
                                <1>     ; 09/08/2017
                                <1>     ; convert byte count to word count
                                <1>     ; 28/01/2025
                                <1>     ;shr    ecx, 1 ; *!*
                                <1>
00013F48 49                     <1>     dec     ecx ; word count - 1
                                <1>     ; convert byte offset to word offset
00013F49 D1EB                   <1>     shr     ebx, 1
                                <1>
                                <1>     ; 16 bit DMA buffer setting (DMA channel 5)
00013F4B B005                   <1>     mov     al, 05h  ; set mask bit for channel 5  (4+1)
00013F4D E6D4                   <1>     out     0D4h, al
                                <1>
00013F4F 30C0                   <1>     xor     al, al   ; stops all DMA processes on selected channel
00013F51 E6D8                   <1>     out     0D8h, al ; clear selected channel register
                                <1>
00013F53 88D8                   <1>     mov     al, bl  ; byte 0 of DMA buffer offset in words (physical)
00013F55 E6C4                   <1>     out     0C4h, al ; DMA channel 5 port number
                                <1>
00013F57 88F8                   <1>     mov     al, bh   ; byte 1 of DMA buffer offset in words (physical)
00013F59 E6C4                   <1>     out     0C4h, al
                                <1>
                                <1>     ; 09/08/2017
00013F5B C1EB0F                 <1>     shr     ebx, 15  ; complete 16 bit shift
00013F5E 80E3FE                 <1>     and     bl, 0FEh ; clear bit 0 (not necessary, it will be ignored)
                                <1>
00013F61 88D8                   <1>     mov     al, bl   ; byte 2 of DMA buffer address (physical)
00013F63 E68B                   <1>     out     8Bh, al  ; page register port addr for channel 5 ; 13/07/2017
                                <1>
00013F65 88C8                   <1>     mov     al, cl   ; low byte of DMA count - 1
00013F67 E6C6                   <1>     out     0C6h, al ; count register port addr for channel 1
                                <1>
00013F69 88E8                   <1>     mov     al, ch   ; high byte of DMA count - 1
00013F6B E6C6                   <1>     out     0C6h, al
                                <1>
                                <1>     ; channel 5, read, autoinitialized, single mode
                                <1>     ;mov    al, 49h
00013F6D B059                   <1>     mov     al, 59h  ; 06/10/2017
00013F6F E6D6                   <1>     out     0D6h, al ; DMA mode register port address
                                <1>
00013F71 B001                   <1>     mov     al, 01h  ; clear mask bit for channel 1
00013F73 E6D4                   <1>     out     0D4h, al ; DMA mask register port address
                                <1>
00013F75 EB2A                   <1>     jmp     short ClearBuffer
                                <1>
                                <1> sbInit_0:
                                <1>     ; 28/01/2025
00013F77 D1E1                   <1>     shl     ecx, 1 ; half buffer size * 2 ; *!*
                                <1>
00013F79 49                     <1>     dec     ecx    ; 09/08/2017
                                <1>
                                <1>     ; 8 bit DMA buffer setting (DMA channel 1)
00013F7A B005                   <1>     mov     al, 05h ; set mask bit for channel 1  (4+1)
00013F7C E60A                   <1>     out     0Ah, al ; DMA mask register
                                <1>
00013F7E 30C0                   <1>     xor     al, al  ; stops all DMA processes on selected channel
00013F80 E60C                   <1>     out     0Ch, al ; clear selected channel register
                                <1>
00013F82 88D8                   <1>     mov     al, bl ; byte 0 of DMA buffer address (physical)
00013F84 E602                   <1>     out     02h, al ; DMA channel 1 port number
                                <1>
00013F86 88F8                   <1>     mov     al, bh  ; byte 1 of DMA buffer address (physical)
00013F88 E602                   <1>     out     02h, al
                                <1>
00013F8A C1EB10                 <1>     shr     ebx, 16
                                <1>
00013F8D 88D8                   <1>     mov     al, bl  ; byte 2 of DMA buffer address (physical)
00013F8F E683                   <1>     out     83h, al ; page register port addr for channel 1
                                <1>
00013F91 88C8                   <1>     mov     al, cl  ; low byte of DMA count - 1
00013F93 E603                   <1>     out     03h, al ; count register port addr for channel 1
                                <1>
00013F95 88E8                   <1>     mov     al, ch  ; high byte of DMA count - 1
```

```
2003 00013F97 E603                <1>     out    03h, al
2004                              <1>
2005                              <1>     ; channel 1, read, autoinitialized, single mode
2006                              <1>     ;mov   al, 49h
2007 00013F99 B059                <1>     mov    al, 59h ; 06/10/2017
2008 00013F9B E60B                <1>     out    0Bh, al ; DMA mode register port address
2009                              <1>
2010 00013F9D B001                <1>     mov     al, 01h ; clear mask bit for channel 1
2011 00013F9F E60A                <1>     out    0Ah, al ; DMA mask register port address
2012                              <1>
2013                              <1> ClearBuffer:
2014                              <1>     ;;mov  edi, [audio_dma_buff]
2015                              <1>     ;;mov  ecx, [audio_dmabuff_size]
2016                              <1>     ;inc   ecx
2017                              <1>     ;mov   al, 80h
2018                              <1>     ;;cld
2019                              <1>     ;rep   stosb
2020                              <1> SetIrq:
2021                              <1>     ;mov   ebx, SbIrqhandler
2022                              <1>     ;mov   al, [audio_intr] ; IRQ number
2023                              <1>     ;call  set_dev_IRQ_service
2024                              <1>     ;; SETUP (audio) INTERRUPT CALLBACK SERVICE
2025                              <1>     ;mov   bl, [audio_intr] ; IRQ number
2026                              <1>     ;mov   bh, [audio_cb_mode]
2027                              <1>     ;inc   bh  ; 1 = Signal Response Byte method (fixed value)
2028                              <1>     ;          ; 2 = Callback service method
2029                              <1>     ;          ; 3 = Auto Increment S.R.B. method
2030                              <1>     ;mov   cl, [audio_srb]
2031                              <1>     ;mov   edx, [audio_cb_addr]
2032                              <1>     ;mov   al, [audio_user]
2033                              <1>     ;call  set_irq_callback_service
2034                              <1> ResetDsp:
2035 00013FA1 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
2036                              <1>     ;add   dx, 06h
2037                              <1>     ; 06/08/2022
2038 00013FA8 80C206              <1>     add    dl, 06h
2039 00013FAB B001                <1>     mov     al, 1
2040 00013FAD EE                  <1>     out    dx, al
2041                              <1>
2042 00013FAE EC                  <1>     in     al, dx
2043 00013FAF EC                  <1>     in     al, dx
2044 00013FB0 EC                  <1>     in     al, dx
2045 00013FB1 EC                  <1>     in     al, dx
2046                              <1>
2047 00013FB2 30C0                <1>     xor     al, al
2048 00013FB4 EE                  <1>     out    dx, al
2049                              <1>
2050                              <1>     ;mov   cx, 100
2051                              <1>     ; 06/08/2022
2052 00013FB5 29C9                <1>     sub    ecx, ecx
2053 00013FB7 B164                <1>     mov    cl, 100
2054 00013FB9 28E4                <1>     sub    ah, ah ; 0
2055                              <1> WaitId:
2056 00013FBB 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
2057                              <1>     ;add   dx, 0Eh
2058                              <1>     ; 06/08/2022
2059 00013FC2 80C20E              <1>     add    dl, 0Eh
2060 00013FC5 EC                  <1>     in     al, dx
2061 00013FC6 08C0                <1>     or      al, al
2062 00013FC8 7807                <1>     js      short sb_GetId
2063 00013FCA E2EF                <1>     loop    WaitId
2064 00013FCC E9B1000000          <1>     jmp     sb_Exit
2065                              <1> sb_GetId:
2066 00013FD1 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
2067                              <1>     ;add   dx, 0Ah
2068                              <1>     ; 06/08/2022
2069 00013FD8 80C20A              <1>     add    dl, 0Ah
2070 00013FDB EC                  <1>     in     al, dx
2071 00013FDC 3CAA                <1>     cmp     al, 0AAh
2072 00013FDE 7407                <1>     je      short SbOk
2073 00013FE0 E2D9                <1>     loop    WaitId
2074 00013FE2 E99B000000          <1>     jmp    sb_Exit
2075                              <1> SbOk:
2076 00013FE7 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
2077                              <1>     ;add   dx, 0Ch
2078                              <1>     ; 06/08/2022
2079 00013FEE 80C20C              <1>     add    dl, 0Ch
2080                              <1>     SbOut   0D1h ; Turn on speaker
1900                              <2> %%Wait:
1901 00013FF1 EC                  <2>  in al, dx
1902 00013FF2 08C0                <2>  or al, al
1903 00013FF4 78FB                <2>  js short %%Wait
1904 00013FF6 B0D1                <2>  mov al, %1
1905 00013FF8 EE                  <2>  out dx, al
2081                              <1>     SbOut   41h ; 8h bit or 16 bit transfer
1900                              <2> %%Wait:
1901 00013FF9 EC                  <2>  in al, dx
1902 00013FFA 08C0                <2>  or al, al
1903 00013FFC 78FB                <2>  js short %%Wait
1904 00013FFE B041                <2>  mov al, %1
1905 00014000 EE                  <2>  out dx, al
2082 00014001 668B1D[1A890100]    <1>     mov    bx, [audio_freq] ; sampling rate (Hz)
2083                              <1>     SbOut  bh ; sampling rate high byte
1900                              <2> %%Wait:
1901 00014008 EC                  <2>  in al, dx
1902 00014009 08C0                <2>  or al, al
1903 0001400B 78FB                <2>  js short %%Wait
1904 0001400D 88F8                <2>  mov al, %1
1905 0001400F EE                  <2>  out dx, al
2084                              <1>     SbOut  bl ; sampling rate low byte
1900                              <2> %%Wait:
1901 00014010 EC                  <2>  in al, dx
1902 00014011 08C0                <2>  or al, al
1903 00014013 78FB                <2>  js short %%Wait
1904 00014015 88D8                <2>  mov al, %1
1905 00014017 EE                  <2>  out dx, al
2085                              <1>
2086                              <1>     ; 22/05/2017
2087 00014018 E8BD000000          <1>     call   sb16_volume_initial ; 15/05/2017
2088                              <1>     ; 20/05/2017
2089                              <1>     ;call  sb16_volume
2090                              <1>
2091                              <1> StartDma:
2092                              <1>     ; autoinitialized mode
2093 0001401D 803D[18890100]10    <1>     cmp    byte [audio_bps], 16 ; 16 bit samples
2094 00014024 7411                <1>     je     short sb_play_1
2095                              <1>     ; 8 bit samples
2096 00014026 66BBC600            <1>     mov    bx, 0C6h ; 8 bit output (0C6h)
2097 0001402A 803D[19890100]02    <1>     cmp    byte [audio_stmo], 2 ; 1 = mono, 2 = stereo
2098 00014031 7214                <1>     jb     short sb_play_2
2099 00014033 B720                <1>     mov    bh, 20h ; 8 bit stereo (20h)
2100 00014035 EB10                <1>     jmp    short sb_play_2
2101                              <1> sb_play_1:
2102                              <1>     ; 16 bit samples
```

```
00014037 66BBB610            <1>     mov     bx, 10B6h ; 16 bit output (0B6h)
0001403B 803D[19890100]02    <1>     cmp     byte [audio_stmo], 2 ; 1 = mono, 2 = stereo
00014042 7203                <1>     jb      short sb_play_2
00014044 80C720              <1>     add     bh, 20h ; 16 bit stereo (30h)
                             <1> sb_play_2:
                             <1>     ; PCM output (8/16 bit mono autoinitialized transfer)
                             <1>     SbOut   bl ; bCommand
                             <2> %%Wait:
00014047 EC                  <2>  in al, dx
00014048 08C0                <2>  or al, al
0001404A 78FB                <2>  js short %%Wait
0001404C 88D8                <2>  mov al, %1
0001404E EE                  <2>  out dx, al
                             <1>     SbOut   bh ; bMode
                             <2> %%Wait:
0001404F EC                  <2>  in al, dx
00014050 08C0                <2>  or al, al
00014052 78FB                <2>  js short %%Wait
00014054 88F8                <2>  mov al, %1
00014056 EE                  <2>  out dx, al
00014057 8B1D[08890100]      <1>     mov     ebx, [audio_dmabuff_size]  ; 15/05/2017
0001405D D1EB                <1>     shr     ebx, 1 ; half buffer size
                             <1>     ; 20/10/2017
0001405F 803D[18890100]10    <1>     cmp     byte [audio_bps], 16 ; 16 bit DMA
00014066 7502                <1>     jne     short sb_play_3
00014068 D1EB                <1>     shr     ebx, 1 ; byte count to word count
                             <1> sb_play_3:
                             <1>     ;dec    bx  ; wBlkSize is one less than the actual size
                             <1>     ; 06/08/2022
0001406A 4B                  <1>     dec     ebx
                             <1>     SbOut   bl
                             <2> %%Wait:
0001406B EC                  <2>  in al, dx
0001406C 08C0                <2>  or al, al
0001406E 78FB                <2>  js short %%Wait
00014070 88D8                <2>  mov al, %1
00014072 EE                  <2>  out dx, al
                             <1>     SbOut   bh
                             <2> %%Wait:
00014073 EC                  <2>  in al, dx
00014074 08C0                <2>  or al, al
00014076 78FB                <2>  js short %%Wait
00014078 88F8                <2>  mov al, %1
0001407A EE                  <2>  out dx, al
                             <1>
0001407B C605[1E890100]01    <1>     mov     byte [audio_play_cmd], 1 ; playing !
                             <1>
                             <1>     ;; Set Voice and master volumes
                             <1>     ;mov    dx, [audio_io_base]
                             <1>     ;add    dl, 4 ; Mixer chip Register Address Port
                             <1>     ;SbOut  30h   ; select Master Volume Register (L)
                             <1>     ;inc    dl    ; Mixer chip Register Data Port
                             <1>     ;SbOut  0F8h  ; Max. volume value is 31 (31*8)
                             <1>     ;dec    dl
                             <1>     ;SbOut  31h   ; select Master Volume Register (R)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F8h  ; Max. volume value is 31 (31*8)
                             <1>     ;dec    dl
                             <1>     ;SbOut  32h   ; select Voice Volume Register (L)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F8h  ; Max. volume value is 31 (31*8)
                             <1>     ;dec    dl
                             <1>     ;SbOut  33h   ; select Voice Volume Register (R)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F8h  ; Max. volume value is 31 (31*8)
                             <1>     ;;
                             <1>     ;dec    dl
                             <1>     ;SbOut  44h   ; select Treble Register (L)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F0h  ; Max. Treble value is 15 (15*16)
                             <1>     ;dec    dl
                             <1>     ;SbOut  45h   ; select Treble Register (R)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F0h  ; Max. Treble value is 15 (15*16)
                             <1>     ;dec    dl
                             <1>     ;SbOut  46h   ; select Bass Register (L)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F0h  ; Max. Bass value is 15 (15*16)
                             <1>     ;dec    dl
                             <1>     ;SbOut  47h   ; select Bass Register (R)
                             <1>     ;inc    dl
                             <1>     ;SbOut  0F0h  ; Max. Bass value is 15 (15*16)
                             <1>
                             <1> sb_Exit:
                             <1>     ;popad
00014082 C3                  <1>     retn
                             <1>
                             <1> sb16_int_handler:
                             <1>     ; Interrupt Handler for Sound Blaster 16 Audio Card
                             <1>     ; Note: called by 'dev_IRQ_service'
                             <1>     ; 28/01/2025
                             <1>     ;       TRDOS 386 Kernel v2.0.10
                             <1>     ; 20/10/2017
                             <1>     ; 12/10/2017
                             <1>     ; 10/10/2017
                             <1>     ; 12/05/2017, 09/10/2017
                             <1>     ; 24/04/2017 (TRDOS 386 kernel, 'audio.s')
                             <1>     ; 10/03/2017 - 'PLAYWAV.PRG' ('playwav.s')
                             <1>
                             <1>     ;push   eax ; * must be saved !
                             <1>     ;push   ebx ; * must be saved !
                             <1>     ;push   ecx
                             <1>     ;push   edx
                             <1>     ;push   esi
                             <1>     ;push   edi
                             <1>
00014083 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
                             <1>     ; 20/10/2017
0001408A 80C20F              <1>     add     dl, 0Fh ; 2xFh (DSP 16 bit intr ack)
0001408D 803D[18890100]10    <1>     cmp     byte [audio_bps], 16
00014094 7402                <1>     je      short sb_irq_16bit_ack
                             <1> sb_irq_8bit_ack:
00014096 FECA                <1>     dec     dl  ; 2xEh (DSP 8 bit intr ack)
                             <1> sb_irq_16bit_ack:
00014098 EC                  <1>     in      al, dx
                             <1>
                             <1>     ;cmp    byte [audio_busy], 0
                             <1>     ;ja     short sb_irq_h3
                             <1>
                             <1>     ;mov    byte [audio_busy], 1
                             <1>
0001409A 803D[1E890100]01    <1>     cmp     byte [audio_play_cmd], 1
000140A0 7307                <1>     jnb     short sb_irq_h1
                             <1> sb_irq_h0:
```

```
000140A2 E8A1000000          <1>     call    sb16_stop
000140A7 EB29                <1>     jmp     short sb_irq_h3
                             <1> sb_irq_h1:
                             <1>     ;call   sb16_tuneloop
                             <1>     ; 09/10/2017
                             <1> sb16_tuneloop:
000140A9 8B3D[04890100]      <1>     mov     edi, [audio_dma_buff]
                             <1>     ;mov    ecx, [audio_dmabuff_size]
                             <1>     ;shr    ecx, 1 ; dma buff size / 2 = half buffer size
                             <1>     ; 28/01/2025 (BugFix)
000140AF 8B0D[0C890100]      <1>     mov     ecx, [dma_hbuff_size]
                             <1>
                             <1>     ; 22/05/2017
000140B5 F605[10890100]01    <1>     test    byte [audio_flag], 1  ; Current flag value
000140BC 7402                <1>     jz      short sb_tlp1 ; EOL (Half Buffer 1 must be filled)
                             <1>     ; FLAG (Half Buffer 2 must be filled)
000140BE 01CF                <1>     add     edi, ecx
                             <1>     ; 15/05/2017
                             <1> sb_tlp1:
000140C0 8B35[FC880100]      <1>     mov     esi, [audio_p_buffer] ; phy addr of audio buff
                             <1>     ;rep    movsb
000140C6 C1E902              <1>     shr     ecx, 2 ; half buff size / 4
000140C9 F3A5                <1>     rep     movsd
                             <1>     ;retn
                             <1>
                             <1>     ; 10/10/2017
                             <1>     ; switch flag value
000140CB 8035[10890100]01    <1>     xor     byte [audio_flag], 1
                             <1>
                             <1>     ; 12/10/2017
                             <1>     ; [audio_flag] = 0 : Playing dma half buffer 2 (odd intr count)
                             <1>                        ; Next buffer (to update) is dma half buff 1
                             <1>     ;              = 1 : Playing dma half buffer 1 (even intr count)
                             <1>                        ; Next buffer (to update) is dma half buff 2
                             <1>
                             <1> sb_irq_h3:
                             <1>     ;mov    byte [audio_busy], 0
                             <1>
                             <1>     ;pop    edi
                             <1>     ;pop    esi
                             <1>     ;pop    edx
                             <1>     ;pop    ecx
                             <1>     ;pop    ebx ; * must be restored !
                             <1>     ;pop    eax ; * must be restored !
                             <1>
000140D2 C3                  <1>     retn
                             <1>
                             <1> sb16_volume:
                             <1>     ; 06/08/2022 (TRDOS 386 v2.0.5)
                             <1>     ; 22/10/2017
                             <1>     ; mov [audio_master_volume_l], cl
                             <1>     ; mov [audio_master_volume_h], ch
000140D3 66890D[20890100]    <1>     mov     [audio_master_volume], cx
                             <1> sb16_volume_initial:
                             <1>     ;push   dx ; DX (port address) must be saved
                             <1>     ; 06/08/2022
000140DA 52                  <1>     push    edx
000140DB 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
                             <1>     ;add    dx, 4 ; Mixer chip address port
                             <1>     ; 06/08/2022
000140E2 80C204              <1>     add     dl, 4
000140E5 B022                <1>     mov     al, 22h ; master volume
000140E7 EE                  <1>     out     dx, al
                             <1>     ;inc    dx
                             <1>     ; 06/08/2022
000140E8 42                  <1>     inc     edx
000140E9 8A25[20890100]      <1>     mov     ah, [audio_master_volume_l]
000140EF C0EC02              <1>     shr     ah, 2 ; 32 -> 8 level
000140F2 C0E405              <1>     shl     ah, 5 ; bit 5 to 7
000140F5 A0[21890100]        <1>     mov     al, [audio_master_volume_r]
000140FA C0E802              <1>     shr     al, 2 ; 32 -> 8 level
                             <1>     ;and    al, 0Fh
000140FD D0E0                <1>     shl     al, 1 ; bit 1 to 3
000140FF 08E0                <1>     or      al, ah
00014101 EE                  <1>     out     dx, al
                             <1>     ;pop    dx ; DX (port address) must be restored
                             <1>     ; 06/08/2022
00014102 5A                  <1>     pop     edx
00014103 C3                  <1>     retn
                             <1>
                             <1> sb16_pause:
                             <1>     ; 06/08/2022 (TRDOS 386 v2.0.5)
00014104 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
                             <1>     ;add    dx, 0Ch ; Command & Data Port
                             <1>     ; 06/08/2022
0001410B 80C20C              <1>     add     dl, 0Ch
0001410E 803D[18890100]10    <1>     cmp     byte [audio_bps], 16 ; 16 bit samples
00014115 7404                <1>     je      short sb_pause_1
                             <1>     ; 8 bit samples
00014117 B3D0                <1>     mov     bl, 0D0h ; 8 bit DMA mode
00014119 EB02                <1>     jmp     short sb_pause_2
                             <1> sb_pause_1:
                             <1>     ; 16 bit samples
0001411B B3D5                <1>     mov     bl, 0D5h ; 16 bit DMA mode
                             <1> sb_pause_2:
                             <1>     SbOut   bl ; bCommand
                             <2> %%wait:
0001411D EC                  <2>  in al, dx
0001411E 08C0                <2>  or al, al
00014120 78FB                <2>  js short %%wait
00014122 88D8                <2>  mov al, %1
00014124 EE                  <2>  out dx, al
                             <1> sb_pause_3:
00014125 C3                  <1>     retn
                             <1>
                             <1> sb16_continue:
                             <1>     ; 06/08/2022 (TRDOS 386 v2.0.5)
00014126 668B15[EA880100]    <1>     mov     dx, [audio_io_base]
                             <1>     ;add    dx, 0Ch ; Command & Data Port
                             <1>     ; 06/08/2022
0001412D 80C20C              <1>     add     dl, 0Ch
00014130 803D[18890100]10    <1>     cmp     byte [audio_bps], 16 ; 16 bit samples
00014137 7404                <1>     je      short sb_cont_1
                             <1>     ; 8 bit samples
00014139 B3D4                <1>     mov     bl, 0D4h ; 8 bit DMA mode
0001413B EB02                <1>     jmp     short sb_cont_2
                             <1> sb_cont_1:
                             <1>     ; 16 bit samples
0001413D B3D6                <1>     mov     bl, 0D6h ; 16 bit DMA mode
                             <1> sb_cont_2:
                             <1>     SbOut   bl ; bCommand
                             <2> %%wait:
0001413F EC                  <2>  in al, dx
00014140 08C0                <2>  or al, al
```

```
00014142 78FB                <2>  js short %%Wait
00014144 88D8                <2>  mov al, %1
00014146 EE                  <2>  out dx, al
                             <1> sb_cont_3:
00014147 C3                  <1>     retn
                             <1>
                             <1> sb16_stop:
                             <1>     ; 06/08/2022 (TRDOS 386 v2.0.5)
                             <1>     ; 24/04/2017
00014148 803D[1E890100]00    <1>     cmp    byte [audio_play_cmd], 0
0001414F 7647                <1>     jna    short sb16_stop_4
                             <1>
                             <1>     ; 22/05/2017
00014151 668B15[EA880100]    <1>     mov    dx, [audio_io_base]
                             <1>     ;add   dx, 0Ch
                             <1>     ; 06/08/2022
00014158 80C20C              <1>     add    dl, 0Ch
                             <1>
0001415B B3D9                <1>     mov    bl, 0D9h ; exit auto-initialize 16 bit transfer
                             <1>     ; stop  autoinitialized DMA transfer mode
0001415D 803D[18890100]10    <1>     cmp    byte [audio_bps], 16 ; 16 bit samples
00014164 7402                <1>     je     short sb16_stop_1
                             <1>     ;mov   bl, 0DAh ; exit auto-initialize 8 bit transfer
00014166 FEC3                <1>     inc    bl
                             <1> sb16_stop_1:
                             <1>     SbOut  bl ; exit auto-initialize transfer command
                             <2> %%Wait:
00014168 EC                  <2>  in al, dx
00014169 08C0                <2>  or al, al
0001416B 78FB                <2>  js short %%Wait
0001416D 88D8                <2>  mov al, %1
0001416F EE                  <2>  out dx, al
                             <1>
00014170 30C0                <1>     xor     al, al ; stops all DMA processes on selected channel
                             <1>
00014172 803D[18890100]10    <1>     cmp    byte [audio_bps], 16 ; 16 bit samples
00014179 7404                <1>     je     short sb16_stop_2
0001417B E60C                <1>     out    0Ch, al ; clear selected channel register
0001417D EB02                <1>     jmp    short sb16_stop_3
                             <1>
                             <1> sb16_stop_2:
0001417F E6D8                <1>     out    0D8h, al ; clear selected channel register
                             <1>
                             <1> sb16_stop_3:
00014181 C605[1E890100]00    <1>     mov    byte [audio_play_cmd], 0 ; stop !
                             <1> SbDone:
                             <1>     ;mov   dx, [audio_io_base]
                             <1>     ;add   dx, 0Ch
                             <1>     SbOut   0D0h
                             <2> %%Wait:
00014188 EC                  <2>  in al, dx
00014189 08C0                <2>  or al, al
0001418B 78FB                <2>  js short %%Wait
0001418D B0D0                <2>  mov al, %1
0001418F EE                  <2>  out dx, al
                             <1>     SbOut   0D3h
                             <2> %%Wait:
00014190 EC                  <2>  in al, dx
00014191 08C0                <2>  or al, al
00014193 78FB                <2>  js short %%Wait
00014195 B0D3                <2>  mov al, %1
00014197 EE                  <2>  out dx, al
                             <1> sb16_stop_4:
00014198 C3                  <1>     retn
                             <1>
                             <1> sb16_reset:
                             <1>     ; 06/08/2022 (TRDOS 386 v2.0.5)
                             <1>     ; 24/04/2017
00014199 668B15[EA880100]    <1>     mov     dx, [audio_io_base] ; try to reset the DSP.
                             <1>     ;add   dx, 06h
                             <1>     ; 06/08/2022
000141A0 80C206              <1>     add    dl, 06h
000141A3 B001                <1>     mov    al, 1
000141A5 EE                  <1>     out    dx, al
                             <1>
000141A6 EC                  <1>     in     al, dx
000141A7 EC                  <1>     in     al, dx
000141A8 EC                  <1>     in     al, dx
000141A9 EC                  <1>     in     al, dx
                             <1>
000141AA 30C0                <1>     xor     al, al
000141AC EE                  <1>     out    dx, al
                             <1>
                             <1>     ;add   dx, 08h
                             <1>     ; 06/08/2022
000141AD 80C208              <1>     add    dl, 08h
                             <1>     ;mov   cx, 100
000141B0 29C9                <1>     sub    ecx, ecx
000141B2 B164                <1>     mov    cl, 100
                             <1> sbrstWaitID:
000141B4 EC                  <1>     in     al, dx
000141B5 08C0                <1>     or      al, al
000141B7 7804                <1>     js      short sbrstGetID
000141B9 E2F9                <1>     loop    sbrstWaitID
000141BB F9                  <1>     stc
000141BC C3                  <1>     retn
                             <1> sbrstGetID:
                             <1>     ;sub   dx, 04h
                             <1>     ; 06/08/2022
000141BD 80EA04              <1>     sub    dl, 04h
000141C0 EC                  <1>     in     al, dx
000141C1 3CAA                <1>     cmp     al, 0AAh
000141C3 7405                <1>     je      short sb_rst_retn
                             <1>     ;add   dx, 04h
                             <1>     ; 06/08/2022
000141C5 80C204              <1>     add    dl, 04h
000141C8 E2EA                <1>     loop    sbrstWaitID
                             <1> sb_rst_retn:
000141CA C3                  <1>     retn
                             <1>
                             <1> ac97_codec_config:
                             <1>     ; 06/06/2024
                             <1>     ; 04/06/2024
                             <1>     ; 03/06/2024
                             <1>     ; 02/06/2024
                             <1>     ; 01/06/2024 (TRDOS 386 v2.0.8)
                             <1>     ; 26/11/2023
                             <1>     ; 21/11/2023
                             <1>     ; 20/11/2023
                             <1>     ; 19/11/2023 (TRDOS 386 v2.0.7)
                             <1>     ; 10/06/2017
                             <1>     ; 05/06/2017
                             <1>     ; 29/05/2017
                             <1>     ; 28/05/2017 (TRDOS 386, 'audio.s')
```

```
                                <1>     ; 07/11/2016 (Erdogan Tan)
                                <1>     ; Derived from 'codecConfig' procedure in 'CODEC.ASM'
                                <1>     ; .wav player for DOS by Jeff Leyda (02/09/2002)
                                <1>
                                <1>     ;; 'PLAYER.ASM'
                                <1>     ;; get ICH base address regs for mixer and bus master
                                <1>
                                <1> init_ac97_controller: ; 10/06/2017
000141CB A1[EC880100]           <1>     mov     eax, [audio_dev_id]
                                <1>         ;mov       al, NAMBAR_REG
                                <1>         ;;call  pciRegRead16                    ; read PCI registers 10-11
                                <1>         ;call      pciRegRead32
                                <1>     ;and   dx, IO_ADDR_MASK             ; mask off BIT0
                                <1>     ;;and  edx, IO_ADDR_MASK
                                <1>
                                <1>         ;mov       [NAMBAR], dx                 ; save audio mixer base addr
                                <1>
                                <1>         ;mov       al, NABMBAR_REG
                                <1>         ;;call     pciRegRead16
                                <1>         ;call      pciRegRead32
                                <1>     ;and   dx, 0FFC0h ; IO_ADDR_MASK
                                <1>     ;;and  edx, 0FFC0h
                                <1>
                                <1>         ;mov    [NABMBAR], dx                   ; save bus master base addr
                                <1>
                                <1>     ;mov   eax, [audio_dev_id]
000141D0 B004                   <1>         mov al, PCI_CMD_REG
                                <1>         ;call      pciRegRead8                  ; read PCI command register
000141D2 E897F8FFFF             <1>         call       pciRegRead16
000141D7 80CA05                 <1>     or     dl, IO_ENA+BM_ENA              ; enable IO and bus master
                                <1>         ;call      pciRegWrite8
000141DA E8FAF8FFFF             <1>     call   pciRegWrite16
                                <1>
                                <1>     ; 'CODEC.ASM'
                                <1>
                                <1>     ; enable codec, unmute stuff, set output rate
                                <1> ;   ; entry: [audio_freq] = desired sample rate
                                <1>
                                <1> ;   mov    dx, [NAMBAR]
                                <1> ;   add    dx, CODEC_EXT_AUDIO_CTRL_REG         ; 2Ah
                                <1> ;   in     ax, dx
                                <1> ;   or     ax, 1
                                <1> ;   out    dx, ax                       ; Enable variable rate audio
                                <1>
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1>
                                <1> ;   mov    ax, [audio_freq]             ; sample rate
                                <1>
                                <1> ;   mov    dx, [NAMBAR]
                                <1> ;   add    dx, CODEC_PCM_FRONT_DACRATE_REG      ; 2Ch
                                <1> ;   out    dx, ax                       ; out sample rate
                                <1>
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1> ;       ;call      delay1_4ms
                                <1>
                                <1>     ;mov   dx, [NAMBAR]                 ; mixer base address
                                <1>         ;add       dx, CODEC_RESET_REG          ; reset register
                                <1>         ;mov       ax, 42
                                <1>     ;out   dx, ax                          ; reset
                                <1>
                                <1>     ;mov   dx, [NABMBAR]                ; bus master base address
                                <1>         ;add       dx, GLOB_STS_REG
                                <1>         ;mov       ax, 2
                                <1>     ;out   dx, ax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 16/05/2024
                                <1>     ; 02/12/2023
                                <1>     ;call  delay_100ms ; 29/05/2017
                                <1>
                                <1>     ; 02/12/2023
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>
                                <1> init_ac97_codec:
                                <1>     ; 26/11/2023
                                <1>     ; 19/11/2023
                                <1>     ;   (playwav3.com, ac97_vra.asm, Erdogan Tan, 19/11/2023)
                                <1>     ; 10/06/2017
                                <1>     ; 29/05/2017
                                <1>     ; 28/05/2017 - Erdogan Tan (Ref: KolibriOS, intelac97.asm)
                                <1>
                                <1>     ;; 23/11/2023 - temporary
                                <1>     ;push  ebx
                                <1>     ;mov   ebx, 0B8000h
                                <1>     ;mov   al, '?'
                                <1>     ;mov   ah, 4Eh
                                <1>     ;mov   [ebx], ax
                                <1>     ;pop   ebx
                                <1>
                                <1>     ; 01/06/2024
000141DF C605[2C8D0100]01       <1>     mov    byte [reset], 1
                                <1>
                                <1>     ; 19/11/2023
000141E6 BD28000000             <1>     mov    ebp, 40 ; 21/11/2023
                                <1> _initc_1:
                                <1>     ; 26/05/2024
000141EB 66BA3000               <1>     mov    dx, GLOB_STS_REG ; 30h
000141EF 660315[EA880100]       <1>     add    dx, [NABMBAR]
000141F6 ED                     <1>     in     eax, dx
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 02/12/2023
                                <1>     ;call  delay1_4ms
                                <1>
000141F7 83F8FF                 <1>     cmp    eax, 0FFFFFFFFh ; -1
000141FA 750A                   <1>     jne    short _initc_3
                                <1> _initc_2:
000141FC 4D                     <1>     dec    ebp    ; 21/11/2023
000141FD 7425                   <1>     jz     short _ac97_codec_ready
                                <1>
000141FF E86DF9FFFF             <1>     call   delay_100ms
00014204 EBE5                   <1>     jmp    short _initc_1
                                <1> _initc_3:
00014206 A900030010             <1>     test   eax, CTRL_ST_CREADY
0001420B 7517                   <1>     jnz    short _ac97_codec_ready
                                <1>
```

```
                                <1>     ; 01/06/2024
0001420D 803D[2C8D0100]00       <1>     cmp    byte [reset], 0
00014214 76E6                   <1>     jna    short _initc_2
                                <1>
00014216 E849030000             <1>     call   reset_ac97_codec
                                <1>     ;jc    short _initc_2
                                <1>     ; 01/06/2024
0001421B C605[2C8D0100]00       <1>     mov    byte [reset], 0
                                <1>     ; 26/11/2023
00014222 EBD8                   <1>     jmp    short _initc_2
                                <1>
                                <1> _ac97_codec_ready:
00014224 668B15[E8880100]       <1>     mov    dx, [NAMBAR]
                                <1>     ;add   dx, 0 ; ac_reg_0 ; reset register
0001422B 66EF                   <1>     out    dx, ax
                                <1>
                                <1>     ; 06/06/2024
                                <1>     ; 01/06/2024
                                <1>     ;;;
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>     ;;;
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 26/05/2024
                                <1>     ; 19/11/2023
                                <1>     ;call  delay_100ms
                                <1>     ; 24/11/2023 - temporary
                                <1>     ;call  delay_100ms
                                <1>     ;call  delay_100ms
                                <1>     ;call  delay_100ms
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ;;;
0001422D 09ED                   <1>     or     ebp, ebp  ; 21/11/2023
0001422F 753E                   <1>     jnz    short _ac97_codec_init_ok
                                <1>     ;;;
                                <1>
00014231 31C0                   <1>     xor    eax, eax ; 0
00014233 668B15[E8880100]       <1>     mov    dx, [NAMBAR]
0001423A 6683C226               <1>     add    dx, CODEC_REG_POWERDOWN
0001423E 66EF                   <1>     out    dx, ax
                                <1>
                                <1>     ; 10/06/2017
                                <1>     ; 29/05/2017
                                <1>     ; wait for 1 second
                                <1>     ; 01/06/2024
                                <1>     ; 16/05/2024
00014240 B9E8030000             <1>     mov    ecx, 1000 ; 1000*0.25ms
                                <1>     ; 20/11/2023
                                <1>     ;mov   ecx, 10
                                <1>     ; 23/05/2024
                                <1>     ;mov   cl, 10
                                <1>     ; 24/11/2023 - temporary
                                <1>     ;mov   cl, 40
                                <1>     ; ecx = 40 ; 23/05/2024
                                <1> _ac97_codec_rloop:
                                <1>     ; 01/06/2024
00014245 E834F9FFFF             <1>     call   delay1_4ms
                                <1>     ; 06/06/2024
0001424A E82FF9FFFF             <1>     call   delay1_4ms
0001424F E82AF9FFFF             <1>     call   delay1_4ms
00014254 E825F9FFFF             <1>     call   delay1_4ms
                                <1>     ; 01/06/2024
                                <1>     ; 19/11/2023
                                <1>     ; 22/11/2023
                                <1>     ;push  ecx
                                <1>     ;call  delay_100ms
                                <1>     ;pop   ecx
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ;;;
00014259 668B15[E8880100]       <1>     mov    dx, [NAMBAR]
00014260 6683C226               <1>     add    dx, CODEC_REG_POWERDOWN
                                <1>     ;;;
00014264 66ED                   <1>     in     ax, dx
                                <1>
                                <1>     ; 06/06/2024
                                <1>     ;call  delay1_4ms
                                <1>
                                <1>     ;and   ax, 0Fh
                                <1>     ; 21/11/2023
00014266 240F                   <1>     and    al, 0Fh
00014268 3C0F                   <1>     cmp    al, 0Fh
0001426A 7403                   <1>     je     short _ac97_codec_init_ok
                                <1>     ; 24/11/2023 - temporary
                                <1>     ;je    short _ac97_codec_init_ok_
                                <1> _ac97_codec_yloop:
0001426C E2D7                   <1>     loop   _ac97_codec_rloop
                                <1>     ; 22/11/2023
                                <1>     ; cf = 1
                                <1> init_ac97_codec_err1:
                                <1>     ;stc
                                <1> init_ac97_codec_err2:
0001426E C3                     <1>     retn
                                <1>
                                <1> ;_ac97_codec_init_ok_:
                                <1>     ;mov   dx, GLOB_STS_REG ; 30h
                                <1>     ;add   dx, [NABMBAR]
                                <1>     ;in    eax, dx
                                <1>     ;test  eax, CTRL_ST_CREADY
                                <1>     ;jnz   short _ac97_codec_init_ok
                                <1>     ;stc
                                <1>     ;retn
                                <1>
                                <1> _ac97_codec_init_ok:
                                <1>     ; 06/06/2024 (temporary)
                                <1>     ; (this may not be needed)
                                <1>     ;;;
                                <1>     ; 24/05/2024
                                <1>     ; 26/11/2023
                                <1>     ; 23/11/2023
0001426F 31C0                   <1>     xor    eax, eax
                                <1>     ; 21/11/2023 - temporary
                                <1>     ; 19/11/2023
00014271 B002                   <1>     mov    al, 2 ; force set 16-bit 2-channel PCM
00014273 66BA2C00               <1>     mov    dx, GLOB_CNT_REG ; 2Ch
00014277 660315[EA880100]       <1>     add    dx, [NABMBAR]
0001427E EF                     <1>     out    dx, eax
                                <1>     ;
0001427F E8FAF8FFFF             <1>     call   delay1_4ms
                                <1>     ;;;
```

```
                                <1>
                                <1>     ; 23/11/2023 - temporary
                                <1>     ;call   delay_100ms
                                <1>     ; 06/06/2024
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     ; 10/06/2017
00014284 E884020000             <1>     call    reset_ac97_controller
                                <1>
                                <1>     ; 21/11/2023
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 24/05/2024
                                <1>     ; 23/05/2024
                                <1>     ; 21/11/2023 - temporary
00014289 E8E3F8FFFF             <1>     call    delay_100ms ; 06/06/2024
                                <1>
                                <1> ;   call    setup_ac97_codec
                                <1> ;
                                <1> ;detect_ac97_codec:
                                <1> ;   retn
                                <1>
                                <1> setup_ac97_codec:
                                <1>     ; 06/06/2024 - TRDOS 386 v2.0.8
                                <1>     ;;;
0001428E 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00014295 6683C22A               <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
00014299 66ED                   <1>     in      ax, dx
                                <1>
                                <1>     ;and    al, ~BIT1 ; Clear DRA
0001429B 24FC                   <1>     and     al, ~(BIT1+BIT0) ; Clear DRA+VRA
                                <1>     ;or     al, BIT0  ; Set VRA
                                <1>
0001429D 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
000142A4 6683C22A               <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
000142A8 66EF                   <1>     out     dx, ax
                                <1>     ;;;
                                <1>
                                <1>     ; 24/11/2023
                                <1>     ; 19/11/2023 - TRDOS 386 v2.0.7
000142AA C605[1C890100]01       <1>     mov     byte [VRA], 1
                                <1>
                                <1>     ; 25/11/2023 - temporary
                                <1>     ;jmp    short vra_not_supported
                                <1>
                                <1>     ; 23/05/2024
                                <1>     ; 18/05/2024
                                <1>     ;call   delay1_4ms ; 06/06/2024
                                <1>     ;call   delay_100ms
                                <1>
                                <1> ; 06/06/2024
                                <1> ; 01/06/2024
                                <1> %if 0
                                <1>     ; 24/05/2024
                                <1>     ; 23/05/2024
                                <1>     mov     dx, [NAMBAR]
                                <1>     add     dx, CODEC_EXT_AUDIO_REG       ; 28h
                                <1>     in      ax, dx
                                <1>
                                <1>     ; 18/05/2024
                                <1>     ; 02/12/2023
                                <1>     call    delay1_4ms
                                <1>     ; 17/05/2024
                                <1>     ;call   delay_100ms
                                <1>     ;
                                <1>     test    al, 1 ; BIT0 ; Variable Rate Audio bit
                                <1>     jz      short vra_not_supported
                                <1>
                                <1>     mov     dx, [NAMBAR]
                                <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
                                <1>     in      ax, dx
                                <1>
                                <1>     ; 06/06/2024
                                <1>     ; 23/05/2024
                                <1>     ; 02/12/2023
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay_100ms ; 18/05/2024
                                <1>     ; 24/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     ;and    al, ~BIT1 ; Clear DRA
                                <1>     ;;;
                                <1>     ; 01/06/2024
                                <1>     and     al, ~(BIT1+BIT0) ; Clear DRA+VRA
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ;mov    dx, [NAMBAR]
                                <1>     ;add    dx, CODEC_EXT_AUDIO_CTRL_REG ; 2Ah
                                <1>     ;;;
                                <1>     out     dx, ax
                                <1> %endif
                                <1>
                                <1>     ; 04/06/2024
000142B1 E8C8F8FFFF             <1>     call    delay1_4ms
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; check VRA
000142B6 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
000142BD 6683C228               <1>     add     dx, CODEC_EXT_AUDIO_REG       ; 28h
000142C1 66ED                   <1>     in      ax, dx
                                <1>
                                <1>     ; 06/06/2024
                                <1>     ;call   delay1_4ms
                                <1>
000142C3 A801                   <1>     test    al, 1 ; BIT0  ; Variable Rate Audio bit
000142C5 746F                   <1>     jz      short vra_not_supported
                                <1>
000142C7 6640                   <1>     inc     ax ; 0FFh -> 0
000142C9 753D                   <1>     jnz     short not_alc ; skip ALC850 (and ALC655) checking
                                <1>     ; ax = -1 -> 0
                                <1>
                                <1>     ;;;;
                                <1>     ; 06/06/2024 (temporary!?)
                                <1>     ; check ALC850
                                <1>     ; (which does not support VRA but it may not be detected)
                                <1>     ;
000142CB E8AEF8FFFF             <1>     call    delay1_4ms
                                <1>     ;
000142D0 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
000142D7 6683C27C               <1>     add     dx, CODEC_VENDOR_ID1 ; 7Ch
000142DB 66ED                   <1>     in      ax, dx
```

```
000142DD C1E010              <1>     shl     eax, 16 ; *
                             <1>     ;cmp    ax, 414Ch ; VENDOR_ID - 'AL'
                             <1>     ;jne    short not_alc
000142E0 668B15[E8880100]    <1>     mov     dx, [NAMBAR]
000142E7 6683C27E            <1>     add     dx, CODEC_VENDOR_ID2 ; 7Eh
000142EB 66ED                <1>     in      ax, dx
                             <1>     ;inc    edx
                             <1>     ;inc    edx
                             <1>     ;in     ax, dx ; VENDOR ID2
                             <1>
000142ED 40                  <1>     inc     eax ; 0FFFFFFFFh -> 0 ; 0 -> 1
000142EE 7446                <1>     jz      short vra_not_supported
000142F0 48                  <1>     dec     eax ; 1 -> 0
000142F1 7443                <1>     jz      short vra_not_supported
                             <1>
000142F3 C1C810              <1>     ror     eax, 16
000142F6 663D4C41            <1>     cmp     ax, 414Ch ; VENDOR_ID - 'AL'
000142FA 750C                <1>     jne     short not_alc
000142FC C1C010              <1>     rol     eax, 16
                             <1>
                             <1>     ; ALC850
000142FF 80FC47              <1>     cmp     ah, 47h ; 'G'
00014302 7504                <1>     jne     short not_alc
00014304 3C90                <1>     cmp     al, 90h
                             <1>     ;cmp    ax, 4790h ; VENDOR ID - 'G' ; bit 8-15
                             <1>                       ; CHIP ID - 1001b ; bit 4-7
                             <1>                       ; Version Number - 0 ; bit 0-3
                             <1>     ;jne    short not_alc850 ; not_alc
00014306 742E                <1>     je      short vra_not_supported
                             <1>     ;
                             <1> ; 06/06/2024
                             <1> %if 0
                             <1> not_alc850:
                             <1>     ; ALC655
                             <1>     cmp     al, 60h
                             <1>     ;cmp    ax, 4760h ; VENDOR ID - 'G' ; bit 8-15
                             <1>                       ; CHIP ID - 0110b ; bit 4-7
                             <1>                       ; Version Number - 0 ; bit 0-3
                             <1>     ;jne    short not_alc655
                             <1>     je      short vra_not_supported
                             <1>     ;
                             <1> not_alc655:
                             <1> %endif
                             <1> not_alc:
                             <1>     ; 06/06/2024
                             <1>
00014308 668B15[E8880100]    <1>     mov     dx, [NAMBAR]
0001430F 6683C22A            <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG         ; 2Ah
00014313 66ED                <1>     in      ax, dx
                             <1>
                             <1>     ;and    al, ~BIT1 ; Clear DRA
                             <1>
00014315 0C01                <1>     or      al, AC97_EA_VRA ; 1 ; 04/11/2023
                             <1>
                             <1>     ; 06/06/2024
                             <1>     ; 04/06/2024
                             <1>     ;mov    dx, CODEC_EXT_AUDIO_CTRL_REG         ; 2Ah
                             <1>     ;add    dx, [NAMBAR]
                             <1>
00014317 66EF                <1>     out     dx, ax          ; Enable variable rate audio
                             <1>
                             <1>     ; 01/06/2024
00014319 B90A000000          <1>     mov     ecx, 10
                             <1>     ; 21/11/2023
                             <1>     ;mov    cl, 10
                             <1> check_vra:
                             <1>     ; 01/06/2024
                             <1>     ;call   delay_100ms
0001431E E85BF8FFFF          <1>     call    delay1_4ms
                             <1>
                             <1>     ;;;
                             <1>     ; 04/06/2024
00014323 668B15[E8880100]    <1>     mov     dx, [NAMBAR]
0001432A 6683C22A            <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG         ; 2Ah
                             <1>     ;;;
                             <1>
0001432E 66ED                <1>     in      ax, dx
                             <1>
                             <1>     ; 24/05/2024
                             <1>     ; 18/05/2024
                             <1>     ; 02/12/2023
                             <1>     ;call   delay1_4ms
                             <1>
                             <1>     ;test   al, AC97_EA_VRA ; 1
                             <1>     ; 04/06/2024
00014330 2401                <1>     and     al, AC97_EA_VRA ; 1
00014332 7508                <1>     jnz     short vra_ok
                             <1>
00014334 E2E8                <1>     loop    check_vra
                             <1>
                             <1> vra_not_supported: ; 24/11/2023
                             <1>     ; VRA is not usable
                             <1>     ;mov    byte [VRA], 0
00014336 FE0D[1C890100]      <1>     dec     byte [VRA]
                             <1>
                             <1> ; 03/06/2024
                             <1> %if 0
                             <1>     ; 02/06/2024
                             <1>     jmp     short set_volume
                             <1>
                             <1>     ; 02/06/2024
                             <1> set_sampling_rate:
                             <1>     mov     dx, [NAMBAR]
                             <1>     add     dx, CODEC_PCM_FRONT_DACRATE_REG
                             <1>     out     dx, ax
                             <1>     retn
                             <1>
                             <1>     ; 02/06/2024
                             <1> get_sampling_rate:
                             <1>     mov     dx, CODEC_PCM_FRONT_DACRATE_REG
                             <1>     add     dx, [NAMBAR]
                             <1>     in      ax, dx
                             <1>     retn
                             <1> %endif
                             <1>
                             <1> vra_ok:
                             <1>
                             <1> ; 03/06/2024
                             <1> %if 0
                             <1>     ; 02/06/2024
                             <1>     ; a second test for verifying VRA status
                             <1>     ; (may be needed for ALC850 codec and CK804 controller)
                             <1>
                             <1>     call    get_sampling_rate
```

```
                                <1>     or      ax, ax
                                <1>     jz      short vra_not_supported
                                <1>     cmp     ax, 48000
                                <1>     jne     short set_volume
                                <1>     mov     ax, 24000
                                <1>     call    set_sampling_rate
                                <1>     call    get_sampling_rate
                                <1>     cmp     ax, 48000
                                <1>     je      short vra_not_supported
                                <1>     mov     ax, 48000
                                <1>     call    set_sampling_rate
                                <1>     ;call   get_sampling_rate
                                <1> %endif
                                <1>
                                <1>     ; 02/06/2024
                                <1> set_volume:
                                <1>     ; 24/05/2024
                                <1>     ; 18/05/2024 - TRDOS 386 v2.0.8
                                <1>     ; 20/11/2023
                                <1>     ; 19/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 22/07/2020
                                <1>     ; 10/06/2017
                                <1>     ; 29/05/2017
                                <1>
                                <1>     ;mov    eax, 0202h
                                <1>     ; 21/11/2023
                                <1>     ;mov    eax, 0B0Bh
                                <1>     ; 23/11/2023
0001433C B81D1D0000             <1>     mov     eax, 1D1Dh
00014341 66A3[20890100]         <1>     mov     [audio_master_volume], ax
                                <1>     ; 24/05/2024
00014347 66A3[22890100]         <1>     mov     [audio_pcmo_volume], ax
                                <1>     ;mov    ax, 1F1Fh ; 31, 31
                                <1>
                                <1>     ; 21/11/2023
                                <1>     ;push   eax
                                <1>
                                <1>     ; 20/11/2023
                                <1>     ;mov    ax, 0B0Bh
                                <1>     ; 23/11/2023
0001434D 66B80202               <1>     mov     ax, 0202h
                                <1>
00014351 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00014358 6683C202               <1>     add     dx, CODEC_MASTER_VOL_REG      ;02h
                                <1>     ;xor    ax, ax  ; volume attenuation = 0 (max. volume)
                                <1>     ; 06/08/2022
                                <1>     ;xor    eax, eax
                                <1>     ; 19/11/2023
0001435C 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 23/05/2024
                                <1>     ; 16/05/2024
                                <1>     ; 20/11/2023
0001435E E81BF8FFFF             <1>     call    delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ; 21/11/2023 temporary
                                <1>     ;call   delay_100ms ; 18/05/2024
                                <1>
                                <1>     ; 21/11/2023
                                <1>     ;pop    eax
                                <1>
                                <1>     ; 23/11/2023
                                <1>     ; 21/11/2023
                                <1>     ;mov    ax, 0202h
                                <1>
00014363 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
0001436A 6683C218               <1>     add     dx, CODEC_PCM_OUT_REG         ;18h
                                <1>     ;xor    ax, ax
0001436E 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 23/05/2024
                                <1>     ; 16/05/2024
                                <1>     ; 20/11/2023
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ; 21/11/2023 - temporary
                                <1>     ;call   delay_100ms ; 18/05/2024
                                <1>
                                <1> ; 23/11/2023
                                <1> %if 0
                                <1>     mov     dx, [NAMBAR]
                                <1>     add     dx, CODEC_MASTER_MONO_VOL_REG ;06h
                                <1>     ;xor    ax, ax
                                <1>     out     dx, ax
                                <1>
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     mov     dx, [NAMBAR]
                                <1>     add     dx, CODEC_PCBEEP_VOL_REG      ;0Ah
                                <1>     ;xor    ax, ax
                                <1>     out     dx, ax
                                <1>
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     mov     ax, 8008h ; Mute
                                <1>     mov     dx, [NAMBAR]
                                <1>     ; 22/07/2020
                                <1>     add     dx, CODEC_PHONE_VOL_REG               ;0Ch
                                <1>                             ; AC97_PHONE_VOL ; TAD Input (Mono)
                                <1>     out     dx, ax
                                <1>
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>         mov ax, 0808h
                                <1>     mov     dx, [NAMBAR]
                                <1>         add dx, CODEC_LINE_IN_VOL_REG ;10h ; Line Input (Stereo)
                                <1>     out     dx, ax
                                <1>
```

```
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>
                                <1>     ;mov   ax, 0808h
                                <1>     mov    dx, [NAMBAR]
                                <1>         add dx, CODEC_CD_VOL_REG ;12h ; CR Input (Stereo)
                                <1>     out    dx, ax
                                <1>
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>     ;call  delay1_4ms
                                <1>
                                <1>     ;mov   ax, 0808h
                                <1>     mov    dx, [NAMBAR]
                                <1>         add dx, CODEC_AUX_VOL_REG ;16h ; Aux Input (Stereo)
                                <1>     out    dx, ax
                                <1>
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>         ;call      delay1_4ms
                                <1>
                                <1>     ; 21/11/2023 - temporary
                                <1>     ;call  delay_100ms
                                <1> %endif
                                <1>     ; 16/05/2024
                                <1>     ;clc
                                <1>
                                <1> set_volume_ok:
                                <1>
                                <1> ;detect_ac97_codec:
00014370 C3                     <1>     retn
                                <1>
                                <1> set_ac97_bdl: ; Set AC97 (ICH) Buffer Descriptor List
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 17/06/2017
                                <1>     ; 11/06/2017
                                <1>     ; 28/05/2017
                                <1>     ; eax = dma buffer address = [audio_DMA_buff]
                                <1>     ;; ecx = dma buffer buffer size = [audio_dmabuff_size]
                                <1>
                                <1>     ; 04/06/2024
                                <1>     ; ecx = DMA half buffer size as truncated for 8 byte alignment
                                <1>
                                <1>     ;shr   ecx, 1 ; dma half buffer size
00014371 89CE                   <1>     mov    esi, ecx
                                <1>
00014373 BF[28890100]           <1>         mov     edi, audio_bdl_buff      ; get BDL address
                                <1>     ;mov   ecx, 32 / 2           ; make 32 entries in BDL
                                <1>     ; 06/08/2022
00014378 29C9                   <1>     sub    ecx, ecx
0001437A B110                   <1>     mov    cl, 16
                                <1>
0001437C EB05                   <1>     jmp    short s_ac97_bdl1
                                <1>
                                <1> s_ac97_bdl0:
                                <1>     ; set buffer descriptor 0 to start of data file in memory
                                <1>
0001437E A1[04890100]           <1>     mov    eax, [audio_dma_buff] ; Physical address of DMA buffer
                                <1>
                                <1> s_ac97_bdl1:
00014383 AB                     <1>     stosd                        ; store dmabuffer1 address
                                <1>
00014384 89C2                   <1>     mov    edx, eax
                                <1>
                                <1> ;
                                <1> ; Buffer Descriptors List
                                <1> ; As stated earlier, each buffer descriptor list is a set of (up to) 32
                                <1> ; descriptors, each 8 bytes in length. Bytes 0-3 of a descriptor entry point
                                <1> ; to a chunk of memory to either play from or record to. Bytes 4-7 of an
                                <1> ; entry describe various control things detailed below.
                                <1> ;
                                <1> ; Buffer pointers must always be aligned on a Dword boundry.
                                <1> ;
                                <1>
                                <1> ;IOCequ     BIT31                ; Fire an interrupt whenever this
                                <1>                                  ; buffer is complete.
                                <1>
                                <1> ;BUPequ     BIT30                ; Buffer Underrun Policy.
                                <1>                                  ; if this buffer is the last buffer
                                <1>                                  ; in a playback, fill the remaining
                                <1>                                  ; samples with 0 (silence) or not.
                                <1>                                  ; It's a good idea to set this to 1
                                <1>                                  ; for the last buffer in playback,
                                <1>                                  ; otherwise you're likely to get a lot
                                <1>                                  ; of noise at the end of the sound.
                                <1>
                                <1> ;
                                <1> ; Bits 15:0 contain the length of the buffer, in number of samples, which
                                <1> ; are 16 bits each, coupled in left and right pairs, or 32bits each.
                                <1> ; Luckily for us, that's the same format as .wav files.
                                <1> ;
                                <1> ; A value of FFFF is 65536 samples. Running at 44.1Khz, that's just about
                                <1> ; 1.5 seconds of sample time. FFFF * 32bits is 1FFFFh bytes or 128k of data.
                                <1> ;
                                <1> ; A value of 0 in these bits means play no samples.
                                <1> ;
                                <1>
00014386 89F0                   <1>     mov    eax, esi ; DMA half buffer size
                                <1>
                                <1>            ; 04/06/2024 - Erdogan Tan
                                <1>            ; NOTE: I have changed DMA half buffer size to truncated
                                <1>            ; (8 byte aligned) audio buffer size for smooth audio playing
                                <1>            ; (it was page border aligned before June 4, 2024)
                                <1>            ;
                                <1>            ; For example:
                                <1>            ;    For 65416 bytes audio buffer (22kHz, 16bit samples)
                                <1>            ;       (user's audio buff virtual addr is mapped to physical)
                                <1>            ;      ((kernel's and user's audio buff pages are same))
                                <1>            ;    1) Memory allocation (of it) is 65536 bytes
                                <1>            ;    2) DMA half buffer size is 65416 bytes
                                <1>            ;      (it would be 65536 bytes before this modification)
                                <1>            ;      ((additional 140 bytes would cause to a noise))
                                <1>            ;    3) Total DMA buffer size is 131072 bytes
                                <1>            ;       (the last 240 bytes will not be used for playing)
                                <1>            ;
                                <1>            ; (buffer will be truncated if the size is not a multiple of 8)
                                <1>
00014388 01C2                   <1>     add    edx, eax
0001438A D1E8                   <1>     shr    eax, 1 ; count of 16 bit samples
```

```
                                <1>     ; 19/11/2023
0001438C 0D000000C0             <1>     or      eax, IOC+BUP
                                <1>     ;or     eax, IOC ; 11/06/2017
00014391 AB                     <1>     stosd
                                <1>
                                <1> ; 2nd buffer:
                                <1>
00014392 89D0                   <1>         mov eax, edx ; Physical address of the 2nd half of DMA buffer
00014394 AB                     <1>     stosd           ; store dmabuffer2 address
                                <1>
                                <1> ; set length to [audio_dmabuff_size]/2
                                <1> ; Set control (bits 31:16) to BUP, bits 15:0=number of samples
                                <1> ;
00014395 89F0                   <1>     mov     eax, esi ; DMA half buffer size
00014397 D1E8                   <1>     shr     eax, 1 ; count of 16 bit samples
                                <1>     ; 19/11/2023
00014399 0D000000C0             <1>     or      eax, IOC+BUP
                                <1>     ;or     eax, IOC ; 11/06/2017
0001439E AB                     <1>     stosd
                                <1>
0001439F E2DD                   <1>     loop    s_ac97_bdl0
                                <1>
000143A1 C3                     <1>     retn
                                <1>
                                <1> ac97_start_play:
                                <1>     ; 26/05/2024 - TRDOS 386 v2.0.8
                                <1>     ; 26/11/2023
                                <1>     ; 20/11/2023
                                <1>     ; 19/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 28/05/2017
                                <1>     ; Derived from 'playWav' procedure in 'ICHWAV.ASM'
                                <1>     ; .wav player for DOS by Jeff Leyda (02/09/2002)
                                <1>
                                <1>     ; set output rate
                                <1>     ; entry: [audio_freq] = desired sample rate
                                <1>
                                <1>     ; 21/11/2023 - temporary
                                <1>     ;call   ac97_codec_config
                                <1>
                                <1> ; 20/11/2023
                                <1> %if 0
                                <1>     AC97_EA_VRA equ 0001h ; 04/11/2023
                                <1>
                                <1>     mov     dx, [NAMBAR]
                                <1>     add     dx, CODEC_EXT_AUDIO_CTRL_REG        ; 2Ah
                                <1>     in      ax, dx
                                <1>
                                <1>     ; 23/05/2024
                                <1>     call    delay1_4ms
                                <1>
                                <1>     and     al, ~BIT1 ; Clear DRA
                                <1>     or      al, AC97_EA_VRA ; 1 ; 04/11/2023
                                <1>     out     dx, ax                ; Enable variable rate audio
                                <1>
                                <1>        ;call       delay1_4ms
                                <1>        ;call       delay1_4ms
                                <1>        ;call       delay1_4ms
                                <1>        ;call       delay1_4ms
                                <1>
                                <1>     ; 23/05/2024
                                <1>     call    delay_100ms
                                <1> %endif
                                <1>
                                <1>     ; 26/11/2023
                                <1>     ; 24/11/2023 - temporary
                                <1>     ;mov    ax, 48000
                                <1>
                                <1>     ; 20/11/2023
000143A2 803D[1C890100]01       <1>     cmp     byte [VRA], 1
000143A9 7213                   <1>     jb      short skip_set_rate
                                <1>
000143AB 66A1[1A890100]         <1>     mov     ax, [audio_freq]             ; sample rate
                                <1>
                                <1> ;skip_set_rate:    ; 24/11/2023
000143B1 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
000143B8 6683C22C               <1>     add     dx, CODEC_PCM_FRONT_DACRATE_REG      ; 2Ch
000143BC 66EF                   <1>     out     dx, ax                       ; out sample rate
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 27/11/2023
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ; 27/11/2023 - temporary
                                <1>     ;call   delay_100ms
                                <1>
                                <1>     ; 26/11/2023
                                <1> skip_set_rate:     ; 20/11/2023
                                <1>
                                <1>     ;; 23/11/2023 - temporary
                                <1>     ;push   ebx
                                <1>     ;mov    ebx, 0B8000h
                                <1>     ;mov    al, 'x'
                                <1>     ;mov    ah, 4Eh
                                <1>     ;mov    [ebx], ax
                                <1>     ;pop    ebx
                                <1> ;
                                <1> ; register reset the DMA engine. This may cause a pop noise on the output
                                <1> ; lines when the device is reset. Prolly a better idea to mute output, then
                                <1> ; reset.
                                <1> ;
                                <1>     ;; 21/11/2023 - temporary
                                <1>     ;; 20/11/2023
                                <1>     ;mov    dx, [NABMBAR]
                                <1>     ;add    dx, PO_CR_REG                ; set pointer to Cntl reg
                                <1>     ;mov    al, RR                       ; set reset
                                <1>     ;out    dx, al                       ; self clearing bit
                                <1>
                                <1>     ; 23/11/2023 - temporary
                                <1>     ;call   delay_100ms
                                <1>
                                <1> ;   mov     edi, audio_bdl_buff
                                <1> ;   mov     edx, [audio_dmabuff_size]
                                <1> ;   shr     edx, 1
                                <1> ;   mov     ecx, 32/2
                                <1> ;ac97_set_bdl_buffer:
                                <1> ;   ; 1st half of DMA buffer
                                <1> ;   mov     eax, [audio_dma_buff]
                                <1> ;   push    eax
                                <1> ;   stosd
                                <1> ;   mov     eax, edx ; dma buffer size / 2
                                <1> ;   or      eax, IOC+BUP
```

```
                                <1> ;   stosd
                                <1> ;   pop     eax
                                <1> ;   ; 2nd half of DMA buffer
                                <1> ;   add     eax, edx
                                <1> ;   stosd
                                <1> ;   mov     eax, edx ; dma buffer size / 2
                                <1> ;   or      eax, IOC+BUP
                                <1> ;   stosd
                                <1> ;   loop    ac97_set_bdl_buffer
                                <1>
                                <1> ; tell the DMA engine where to find our list of Buffer Descriptors.
                                <1> ; this 32bit value is a flat mode memory offset (ie no segment:offset)
                                <1> ;
                                <1> ; write NABMBAR+10h with offset of buffer descriptor list
                                <1> ;
000143BE B8[28890100]           <1>         mov eax, audio_bdl_buff
000143C3 668B15[EA880100]       <1>     mov     dx, [NABMBAR]
000143CA 6683C210               <1>     add     dx, PO_BDBAR_REG
000143CE EF                     <1>     out     dx, eax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 24/05/2024
                                <1>     ; 23/11/2023 - temporary
                                <1>     ;call   delay_100ms
                                <1>     ; 23/05/2024
                                <1>     ;call   delay1_4ms
                                <1> ;
                                <1> ; All set. Let's play some music.
                                <1> ;
                                <1>     ;mov    eax, 31
                                <1>     ; 06/08/2022
000143CF 29C0                   <1>     sub     eax, eax
000143D1 B01F                   <1>     mov     al, 31
000143D3 E846000000             <1>     call     set_ac97_LastValidIndex
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 24/05/2024
                                <1>     ; 23/11/2023 - temporary
                                <1>     ;call   delay_100ms
                                <1>     ; 23/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ; 27/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
000143D8 C605[1E890100]01       <1>     mov     byte [audio_play_cmd], 1 ; play command (do not stop) !
                                <1>
                                <1> ac97_play: ; continue to play (after pause)
                                <1>     ; 27/05/2024
                                <1>     ;;;
                                <1>     ; 19/05/2024 - temporary
                                <1>     ; 24/11/2023 - temporary
                                <1>     ;mov    ax, 0202h
                                <1>     ; 23/05/2024
000143DF 66B81F1F               <1>     mov     ax, 1F1Fh
000143E3 662B05[20890100]       <1>     sub     ax, [audio_master_volume]
                                <1>
000143EA 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
000143F1 6683C202               <1>     add     dx, CODEC_MASTER_VOL_REG  ; 02h ; Line Out
000143F5 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 19/05/2024 - temporary
                                <1>     ;call   delay_100ms
                                <1>     ; 23/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ; 24/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>
                                <1>     ;mov    ax, 0202h
                                <1>     ; 23/05/2024
                                <1>     ;mov    ax, 1F1Fh
                                <1>     ;sub    ax, [audio_master_volume]
                                <1>
                                <1>     ; 27/05/2024
000143F7 66B81F1F               <1>     mov     ax, 1F1Fh
000143FB 662B05[22890100]       <1>     sub     ax, [audio_pcmo_volume]
                                <1>
00014402 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00014409 6683C218               <1>     add     dx, CODEC_PCM_OUT_REG ; 18h ; PCM Out
0001440D 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ; 01/06/2024
                                <1>     ; 24/05/2024
                                <1>     ; 23/05/2024
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ;call   delay1_4ms
                                <1>     ; 24/05/2024
                                <1>     ;call   delay_100ms
                                <1>     ;;;
                                <1>
                                <1>     ; 11/06/2017
                                <1>     ; 29/05/2017
                                <1>     ; 28/05/2017
0001440F 668B15[EA880100]       <1>         mov     dx, [NABMBAR]
00014416 6683C21B               <1>         add     dx, PO_CR_REG            ; PCM out control register
                                <1>     ; 26/11/2023
0001441A B011                   <1>     mov     al, IOCE+RPBM ; 29/05/2017
                                <1>     ; 24/11/2023
                                <1>     ;mov    al, 1Dh ; (Ref: KolibriOS, intelac97.asm, 'play:')
0001441C EE                     <1>     out      dx, al                ; set start!
                                <1>
                                <1>     ;mov    byte [audio_play_cmd], 1 ; play command (do not stop) !
                                <1>
0001441D C3                     <1>     retn
                                <1>
                                <1> ;input AL = index # to stop on
                                <1> set_ac97_LastValidIndex:
                                <1>     ; 28/05/2017
                                <1>     ; Derived from 'setLastValidIndex' procedure in 'ICHWAV.ASM'
                                <1>     ; .wav player for DOS by Jeff Leyda (02/09/2002)
0001441E 668B15[EA880100]       <1>     mov     dx, [NABMBAR]
00014425 6683C215               <1>     add     dx, PO_LVI_REG
00014429 EE                     <1>         out     dx, al
                                <1>     ;mov    [audio_lvi], al ; for ac97_int_handler
0001442A C3                     <1>     retn
                                <1>
                                <1> ac97_volume:
                                <1>     ; 23/05/2024
```

```
                                <1>     ; 28/05/2017
                                <1>     ; bl = component (0 = master/playback/lineout volume)
                                <1>     ;                 (1 = PCM Out volume) ; 23/05/2024
                                <1>     ; cl = left channel volume level (0 to 31)
                                <1>     ; ch = right channel volume level (0 to 31)
                                <1>
                                <1>     ; 24/05/2024
0001442B 80FB01                 <1>     cmp     bl, 1
0001442E 7727                   <1>     ja      short ac97_vol_2 ; temporary !
                                <1>
00014430 66B81F1F               <1>     mov     ax, 1F1Fh ; 31,31
00014434 38C1                   <1>     cmp     cl, al
00014436 771F                   <1>     ja      short ac97_vol_2
00014438 38E5                   <1>     cmp     ch, ah
0001443A 771B                   <1>     ja      short ac97_vol_2
                                <1>
0001443C 08DB                   <1>     or      bl, bl
0001443E 7518                   <1>     jnz     short ac97_vol_1
                                <1>
                                <1>     ; 23/05/2024
00014440 66BA0200               <1>     mov     dx, CODEC_MASTER_VOL_REG  ; 02h ; Line Out
                                <1>     ; 24/05/2024
00014444 66890D[20890100]       <1>     mov     [audio_master_volume], cx
                                <1> ac97_vol_0:
0001444B 6629C8                 <1>     sub     ax, cx
                                <1>     ;mov    dx, [NAMBAR]
                                <1>     ;add    dx, CODEC_MASTER_VOL_REG
                                <1>     ; 23/05/2024
0001444E 660315[E8880100]       <1>     add     dx, [NAMBAR]
00014455 66EF                   <1>     out      dx, ax
                                <1>     ; 23/05/2024
                                <1> ac97_vol_2:
                                <1>     ; 21/11/2023
                                <1> ;_ac97_ih5: ; 06/08/2022
00014457 C3                     <1>     retn
                                <1>
                                <1> ac97_vol_1:
                                <1>     ; 24/05/2024
                                <1>     ; 23/05/2024
                                <1>     ;cmp    bl, 1
                                <1>     ;ja     short ac97_vol_2 ; temporary !
                                <1>     ; BL = 1
                                <1>     ; PCM OUT Volume
00014458 66BA1800               <1>     mov     dx, CODEC_PCM_OUT_REG ; 18h ; PCM out
                                <1>     ; 24/05/2024
0001445C 66890D[22890100]       <1>     mov     [audio_pcmo_volume], cx
00014463 EBE6                   <1>     jmp     short ac97_vol_0
                                <1>
                                <1> ac97_int_handler:
                                <1>     ; 04/06/2024
                                <1>     ; 03/06/2024
                                <1>     ; 02/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 27/11/2023
                                <1>     ; 24/11/2023
                                <1>     ; 21/11/2023
                                <1>     ; 20/11/2023 - TRDOS 386 v2.0.7
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 12/10/2017
                                <1>     ; 10/10/2017
                                <1>     ; 09/10/2017
                                <1>     ; 13/06/2017, 13/06/2017
                                <1>     ; 10/06/2017, 11/06/2017
                                <1>     ; Interrupt Handler for AC97 (ICH) Audio Controller
                                <1>     ; Note: called by 'dev_IRQ_service'
                                <1>     ; 28/05/2017
                                <1>
                                <1>     ;push   eax ; * must be saved !
                                <1>     ;push   edx
                                <1>     ;push   ecx
                                <1>     ;push   ebx ; * must be saved !
                                <1>     ;push   esi
                                <1>     ;push   edi
                                <1>
                                <1> ; 02/06/2024
                                <1> %if 0
                                <1>
                                <1>     ;cmp    byte [audio_busy], 1
                                <1>     ;jnb    _ac97_ih2 ; busy !
                                <1>
                                <1>     ; 24/11/2023
                                <1>         ;mov       dx, [NABMBAR]
                                <1>     ;mov    dx, PO_SR_REG
                                <1>     ;in     ax, dx
                                <1>     ;test   al, BCIS ; bit 3, 8
                                <1>     ;jz     short _ac97_ih5
                                <1>
                                <1>     ; 24/11/2023
                                <1>     mov     dx, GLOB_STS_REG
                                <1>         add dx, [NABMBAR]
                                <1>     in      eax, dx
                                <1>
                                <1>     cmp     eax, 0FFFFFFFFh ; -1
                                <1>     ;je     _ac97_ih3 ; exit
                                <1>     ; 06/08/2022
                                <1>     je      short _ac97_ih5
                                <1>
                                <1>     ;test   eax, 40h ; PCM Out Interrupt
                                <1>     ; 06/08/2022
                                <1>     test    al, 40h
                                <1>     jnz     short _ac97_ih0
                                <1>
                                <1>     ; 24/11/2023
                                <1>     test    eax, eax
                                <1>     ;jz     _ac97_ih3 ; exit
                                <1>     ; 06/08/2022
                                <1>     jz      short _ac97_ih5
                                <1>
                                <1>     ;mov    dx, GLOB_STS_REG
                                <1>         ;add       dx, [NABMBAR]
                                <1>     out     dx, eax
                                <1>
                                <1>     ;jmp    _ac97_ih3 ; exit
                                <1>
                                <1>     ; 06/08/2022
                                <1>     retn
                                <1>     ; 21/11/2023
                                <1>     ;jmp    short _ac97_ih5
                                <1>
                                <1> _ac97_ih0:
                                <1>     ; 24/11/2023
                                <1>     ;push   eax
                                <1>     ; 09/10/2017
                                <1>     cmp     byte [audio_play_cmd], 1
                                <1>     ;jb     short _ac97_ih4 ; stop command !
```

```
                                <1>     ; 24/11/2023
                                <1>     jnb    short _ac97_ih4
                                <1>
                                <1>     ;mov   dx, GLOB_STS_REG
                                <1>         ;add       dx, [NABMBAR]
                                <1>     out    dx, eax
                                <1>     ;jmp   short _ac97_stop
                                <1>
                                <1>     ; 24/11/2023
                                <1> ac97_stop:
                                <1>     ; 28/05/2017
                                <1>     mov    byte [audio_play_cmd], 0 ; stop !
                                <1>
                                <1> %else
                                <1>     ; 02/06/2024
                                <1> _ac97_ih0:
00014465 803D[1E890100]01       <1>     cmp    byte [audio_play_cmd], 1
0001446C 7334                   <1>     jnb    short _ac97_ih4
                                <1> %endif
                                <1>
                                <1> _ac97_stop: ; 09/10/2017
                                <1>     ; 29/05/2017
                                <1>     ;mov   dx, [NABMBAR]
                                <1>     ;add   dx, PO_CR_REG
                                <1>     ;mov   al, 0
                                <1>     ;out   dx, al
                                <1>
                                <1>     ; 11/06/2017
0001446E 30C0                   <1>     xor    al, al ; 0
00014470 E813000000             <1>     call   ac97_po_cmd
                                <1>
                                <1>     ; (Ref: KolibriOS, intelac97.asm, 'stop:')
                                <1>     ; Clear FIFOE, BCIS, LVBCI (Ref: Intel ICH hub manual)
00014475 66B81C00               <1>     mov     ax, 1Ch
00014479 668B15[EA880100]       <1>     mov     dx, [NABMBAR]
00014480 6683C216               <1>     add     dx, PO_SR_REG
00014484 66EF                   <1>     out     dx, ax
                                <1>
                                <1>     ;retn
                                <1>
                                <1>     ; 11/06/2017
00014486 B002                   <1>     mov     al, RR
                                <1> ac97_po_cmd:
                                <1>      ;11/06/2017
                                <1>     ; 29/05/2017
00014488 668B15[EA880100]       <1>     mov     dx, [NABMBAR]
0001448F 6683C21B               <1>         add     dx, PO_CR_REG            ; PCM out control register
00014493 EE                     <1>     out    dx, al
00014494 C3                     <1>     retn
                                <1>
                                <1> ; 02/06/2024
                                <1> %if 1
                                <1>     ; 24/11/2023
                                <1> ac97_stop:
                                <1>     ; 28/05/2017
00014495 C605[1E890100]00       <1>     mov    byte [audio_play_cmd], 0 ; stop !
0001449C EBD0                   <1>     jmp    short _ac97_stop
                                <1> %endif
                                <1>
                                <1> ac97_pause:
                                <1>     ; 11/06/2017
                                <1>     ; 29/05/2017
0001449E B010                   <1>     mov    al, IOCE
000144A0 EBE6                   <1>     jmp    short ac97_po_cmd
                                <1>
                                <1> _ac97_ih4:
                                <1>     ;mov   byte [audio_busy], 1
                                <1>
                                <1> ; 02/06/2024
                                <1> %if 1
                                <1>     ; 24/11/2023 (TRDOS386 'audio.s')
000144A2 668B15[EA880100]       <1>         mov dx, [NABMBAR]
000144A9 6683C216               <1>     add    dx, PO_SR_REG
000144AD 66ED                   <1>     in     ax, dx
                                <1>
000144AF A808                   <1>     test   al, BCIS ; bit 3, 8
000144B1 744C                   <1>     jz     short _ac97_ih2       ; 02/06/2024
                                <1>
                                <1>     ; 02/06/2024
000144B3 50                     <1>     push   eax ; *
                                <1> %else
                                <1>     ; 27/05/2024
                                <1>     ; 24/11/2023
                                <1>     push   eax
                                <1>
                                <1>     mov    ax, 1Ch ; FIFOE(=16)+BCIS(=8)+LVBCI(=4)
                                <1>     mov    dx, PO_SR_REG
                                <1>         add dx, [NABMBAR]
                                <1>     out    dx, ax
                                <1>
                                <1>     mov    dx, PO_CIV_REG
                                <1>         add dx, [NABMBAR]
                                <1>     in     al, dx
                                <1>
                                <1>     ;cmp   al, [audio_civ] ; [audio_flag]
                                <1>     ;je    short _ac97_ih2
                                <1>
                                <1>     ; 21/11/2023
                                <1>     ;mov   [audio_civ], al
                                <1>     ; 20/11/2023
                                <1>     mov    ah, al
                                <1>     dec    al
                                <1>     ;inc   al ; 11/06/2017
                                <1>     and    al, 1Fh
                                <1> %endif
                                <1>     ; 02/06/2024
000144B4 66BA1400               <1>     mov    dx, PO_CIV_REG
000144B8 660315[EA880100]       <1>         add dx, [NABMBAR]
                                <1>     ;in    ax, dx
                                <1>     ; 03/06/2024
000144BF EC                     <1>     in     al, dx
                                <1>                     ; al = CVI, ah = LVI
                                <1>     ;xchg  ah, al
                                <1>     ;              ; al = LVI, ah = CVI
                                <1>     ;cmp   al, ah
                                <1>     ;jne   short _ac97_ih5
                                <1>     ; 03/06/2024
000144C0 88C4                   <1>     mov    ah, al
                                <1>
000144C2 FEC8                   <1>     dec    al
000144C4 241F                   <1>     and    al, 1Fh
                                <1>
                                <1>     ;mov   dx, PO_LVI_REG
                                <1>     ;add   dx, [NABMBAR]
```

```
000144C6 6642                <1>         inc dx      ; 02/06/2024
000144C8 EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 12/10/2017
                             <1>     ;mov    al, [audio_civ]
                             <1>     ;inc    al
                             <1>     ;and    al, 1
                             <1>     ;mov    [audio_flag], al
                             <1>     ; 27/11/2023
                             <1>     ; 20/11/2023
                             <1>     ;inc    ah
                             <1> _ac97_ih5:  ; 02/06/2024
000144C9 80E401              <1>     and     ah, 1
000144CC 8825[10890100]      <1>     mov     [audio_flag], ah
                             <1>
                             <1>     ;; [audio_flag] : 0 = Buffer 1, 1 = Buffer 2
                             <1>     ;
                             <1>     ; 21/11/2023
                             <1>     ;call   ac97_tuneloop
                             <1>
                             <1>     ; 24/11/2023
                             <1> ac97_tuneloop:
                             <1>     ; 09/10/2017
000144D2 8B3D[04890100]      <1>     mov     edi, [audio_dma_buff]
                             <1>     ; 04/06/2024
                             <1>     ;mov    ecx, [audio_dmabuff_size]
                             <1>     ;shr    ecx, 1 ; dma buff size / 2 = half buffer size
000144D8 8B0D[00890100]      <1>     mov     ecx, [audio_buff_size]
000144DE 80E1F8              <1>     and     cl, ~7 ; 8 byte aligned
                             <1>
                             <1>     ; 12/10/2017
000144E1 803D[10890100]00    <1>     cmp     byte [audio_flag], 0
000144E8 7702                <1>     ja      short _ac97_ih1  ; Playing Half Buffer 2 (Current: FLAG)
                             <1>     ; Playing Half Buffer 1 (Current: EOL)
000144EA 01CF                <1>     add     edi, ecx
                             <1> _ac97_ih1:
                             <1>     ; Update half buffer 2 while playing half buffer 1 (next: FLAG)
                             <1>     ; Update half buffer 1 while playing half buffer 2 (next: EOL)
                             <1>
000144EC 8B35[FC880100]      <1>     mov     esi, [audio_p_buffer] ; phy addr of audio buff
000144F2 C1E902              <1>     shr     ecx, 2 ; half buff size / 4
000144F5 F3A5                <1>     rep     movsd
                             <1>
                             <1>     ; 10/10/2017
                             <1>     ; switch flag value
000144F7 8035[10890100]01    <1>     xor     byte [audio_flag], 1
                             <1>
                             <1> ; 02/06/2024
                             <1> %if 1
                             <1>     ; 02/06/2024
000144FE 58                  <1>     pop     eax ; *
                             <1> _ac97_ih2:
                             <1>     ;mov    ax, 1Ch ; FIFOE(=16)+BCIS(=8)+LVBCI(=4)
000144FF 66BA1600            <1>     mov     dx, PO_SR_REG
00014503 660315[EA880100]    <1>     add     dx, [NABMBAR]
0001450A 66EF                <1>     out     dx, ax
                             <1> %else
                             <1>     ; 27/11/2023
                             <1>     ; 21/11/2023 - temporary
                             <1>     ;push   ebx
                             <1>     ;mov    ebx, 0B8002h
                             <1>     ;mov    al, [audio_flag]
                             <1>     ;add    al, '1'
                             <1>     ;mov    ah, 0Fh
                             <1>     ;mov    [ebx], ax
                             <1>     ;pop    ebx
                             <1>
                             <1>     ; 12/10/2017
                             <1>     ; [audio_flag] = 0 : Playing dma half buffer 2 (even index value)
                             <1>                          ; Next buffer (to update) is dma half buff 1
                             <1>     ;              = 1 : Playing dma half buffer 1 (odd index value)
                             <1>                          ; Next buffer (to update) is dma half buff 2
                             <1>     ; 24/11/2023
                             <1>     ;retn
                             <1>
                             <1>     ;
                             <1>     ; 24/11/2023
                             <1>     pop     eax
                             <1>     ; 24/11/2023
                             <1>     ; 20/11/2023
                             <1>     ;and    eax, 40h
                             <1>         mov dx, [NABMBAR]
                             <1>     add     dx, GLOB_STS_REG
                             <1>     out     dx, eax
                             <1>
                             <1>     ;; 13/06/2017
                             <1>     ;mov    al, 11h ; IOCE + RPBM
                             <1>     ;mov    dx, PO_CR_REG
                             <1>         ;add       dx, [NABMBAR]
                             <1>     ;out    dx, al
                             <1>
                             <1>     ; 24/11/2023
                             <1>     ;mov    dx, [NABMBAR]
                             <1>     ;add    dx, PO_SR_REG  ; set pointer to Status reg
                             <1>     ;mov    ax, 1Ch
                             <1>     ;out    dx, ax
                             <1>
                             <1>     ; 24/11/2023
                             <1> _ac97_ih2:
                             <1>     ;mov    byte [audio_busy], 0
                             <1> %endif
                             <1>
                             <1> _ac97_ih3:
                             <1>     ;pop    edi
                             <1>     ;pop    esi
                             <1>     ;pop    ebx ; * must be restored !
                             <1>     ;pop    ecx
                             <1>     ;pop    edx
                             <1>     ;pop    eax ; * must be restored !
                             <1>
0001450C C3                  <1>     retn
                             <1>
                             <1> reset_ac97_controller:
                             <1>     ; 06/06/2024
                             <1>     ; 01/06/2024
                             <1>     ; 27/05/2024
                             <1>     ; 16/05/2024
                             <1>     ; 10/06/2017
                             <1>     ; 29/05/2017
                             <1>     ; 28/05/2017
                             <1>     ; reset AC97 audio controller registers
0001450D 31C0                <1>     xor     eax, eax
0001450F 66BA0B00            <1>         mov dx, PI_CR_REG
00014513 660315[EA880100]    <1>     add     dx, [NABMBAR]
```

```
0001451A EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
0001451B 66BA1B00            <1>         mov     dx, PO_CR_REG
0001451F 660315[EA880100]    <1>     add     dx, [NABMBAR]
00014526 EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
00014527 66BA2B00            <1>         mov     dx, MC_CR_REG
0001452B 660315[EA880100]    <1>     add     dx, [NABMBAR]
00014532 EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
00014533 B002                <1>         mov     al, RR
00014535 66BA0B00            <1>         mov     dx, PI_CR_REG
00014539 660315[EA880100]    <1>     add     dx, [NABMBAR]
00014540 EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
00014541 66BA1B00            <1>         mov     dx, PO_CR_REG
00014545 660315[EA880100]    <1>     add     dx, [NABMBAR]
0001454C EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
0001454D 66BA2B00            <1>         mov     dx, MC_CR_REG
00014551 660315[EA880100]    <1>     add     dx, [NABMBAR]
00014558 EE                  <1>     out     dx, al
                             <1>
                             <1>     ; 27/05/2024
                             <1>     ; 16/05/2024
                             <1>     ;call   delay1_4ms
                             <1>
00014559 C3                  <1>     retn
                             <1>
                             <1> ac97_reset:
                             <1>     ; 27/05/2024 - TRDOS 386 v2.0.8
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 10/06/2017
                             <1>     ; 29/05/2017
                             <1>     ; 28/05/2017
0001455A E8AEFFFFFF          <1>     call    reset_ac97_controller
                             <1>     ; 29/05/2017
                             <1>     ;jmp    reset_ac97_codec
                             <1>     ; 27/05/2024
0001455F E80DF6FFFF          <1>     call    delay_100ms
                             <1> reset_ac97_codec:
                             <1>     ; 28/05/2017 - Erdogan Tan (Ref: KolibriOS, intelac97.asm)
00014564 66BA2C00            <1>     mov     dx, GLOB_CNT_REG ; 2Ch
00014568 660315[EA880100]    <1>     add     dx, [NABMBAR]
0001456F ED                  <1>     in      eax, dx
                             <1>
                             <1>     ;test   eax, 2
                             <1>     ; 06/08/2022
00014570 A802                <1>     test    al, 2
00014572 7407                <1>     jz      short _r_ac97codec_cold
                             <1>
00014574 E80F000000          <1>     call    warm_ac97codec_reset
00014579 7308                <1>     jnc     short _r_ac97codec_ok
                             <1> _r_ac97codec_cold:
0001457B E83B000000          <1>         call    cold_ac97codec_reset
00014580 7301                <1>         jnc     short _r_ac97codec_ok
                             <1>
                             <1>     ; 16/04/2017
                             <1>         ;xor       eax, eax       ; timeout error
                             <1>             ;stc
00014582 C3                  <1>     retn
                             <1>
                             <1> _r_ac97codec_ok:
                             <1>         xor     eax, eax
                             <1>         ;mov       al, VIA_ACLINK_C00_READY ; 1
00014585 FEC0                <1>         inc al
00014587 C3                  <1>     retn
                             <1>
                             <1> warm_ac97codec_reset:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 28/05/2017 - Erdogan Tan (Ref: KolibriOS, intelac97.asm)
                             <1>         ;mov       eax, 6
                             <1>     ; 06/08/2022
00014588 29C0                <1>     sub     eax, eax
0001458A B006                <1>     mov     al, 6
0001458C 66BA2C00            <1>     mov     dx, GLOB_CNT_REG ; 2Ch
00014590 660315[EA880100]    <1>     add     dx, [NABMBAR]
00014597 EF                  <1>     out     dx, eax
                             <1>
                             <1>     ;mov    ecx, 10 ; total 1s
                             <1>     ; 06/08/2022
00014598 31C9                <1>     xor     ecx, ecx
0001459A B10A                <1>     mov     cl, 10
                             <1> _warm_ac97c_rst_wait:
0001459C 51                  <1>     push    ecx
0001459D E8CFF5FFFF          <1>     call    delay_100ms
000145A2 59                  <1>     pop     ecx
                             <1>
000145A3 66BA3000            <1>     mov     dx, GLOB_STS_REG ; 30h
000145A7 660315[EA880100]    <1>     add     dx, [NABMBAR]
000145AE ED                  <1>     in      eax, dx
                             <1>
000145AF A900030010          <1>     test    eax, CTRL_ST_CREADY
000145B4 7504                <1>     jnz     short _warm_ac97c_rst_ok
                             <1>
000145B6 49                  <1>         dec     ecx
000145B7 75E3                <1>         jnz     short _warm_ac97c_rst_wait
                             <1>
                             <1> _warm_ac97c_rst_fail:
000145B9 F9                  <1>         stc
                             <1> _warm_ac97c_rst_ok:
000145BA C3                  <1>     retn
                             <1>
                             <1> cold_ac97codec_reset:
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 28/05/2017 - Erdogan Tan (Ref: KolibriOS, intelac97.asm)
                             <1>         ;mov       eax, 2
                             <1>     ; 06/08/2022
000145BB 31C0                <1>     xor     eax, eax
```

```
000145BD B002                <1>     mov     al, 2
000145BF 66BA2C00            <1>     mov     dx, GLOB_CNT_REG ; 2Ch
000145C3 660315[EA880100]    <1>     add     dx, [NABMBAR]
000145CA EF                  <1>     out     dx, eax
                             <1>
000145CB E8A1F5FFFF          <1>     call    delay_100ms    ; wait 100 ms
000145D0 E89CF5FFFF          <1>     call    delay_100ms    ; wait 100 ms
000145D5 E897F5FFFF          <1>     call    delay_100ms    ; wait 100 ms
000145DA E892F5FFFF          <1>     call    delay_100ms    ; wait 100 ms
                             <1>
                             <1>     ;mov    ecx, 16 ; total 20*100 ms = 2s
                             <1>     ; 06/08/2022
000145DF 31C9                <1>     xor     ecx, ecx
000145E1 B110                <1>     mov     cl, 16
                             <1> _cold_ac97c_rst_wait:
000145E3 66BA3000            <1>     mov     dx, GLOB_STS_REG ; 30h
000145E7 660315[EA880100]    <1>     add     dx, [NABMBAR]
000145EE ED                  <1>     in      eax, dx
                             <1>
000145EF A900030010          <1>     test    eax, CTRL_ST_CREADY
000145F4 750B                <1>     jnz     short _cold_ac97c_rst_ok
                             <1>
000145F6 51                  <1>     push    ecx
000145F7 E875F5FFFF          <1>     call    delay_100ms
000145FC 59                  <1>     pop     ecx
                             <1>
000145FD 49                  <1>         dec     ecx
000145FE 75E3                <1>         jnz     short _cold_ac97c_rst_wait
                             <1>
                             <1> _cold_ac97c_rst_fail:
00014600 F9                  <1>         stc
                             <1> _cold_ac97c_rst_ok:
00014601 C3                  <1>     retn
                             <1>
                             <1> sb16_current_sound_data:
                             <1>     ; 05/06/2024
                             <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                             <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                             <1>     ; 20/08/2017
                             <1>     ; 24/06/2017
                             <1>     ; 22/06/2017
                             <1>     ; get current sound (PCM out) data for graphics
                             <1>     ; (for Sound Blaster 16)
                             <1>     ; ebx = Physical address (on page boundary)
                             <1>     ; ecx = Byte count
                             <1>     ; [audio_buff_size]
                             <1>
                             <1>     ;;mov   edi, [audio_buff_size]
                             <1>     ;mov    edi, [audio_dmabuff_size]
                             <1>     ;mov    esi, [audio_dma_buff]
                             <1>
00014602 39CF                <1>     cmp     edi, ecx
00014604 7302                <1>     jnb     short sb16_gcd_0
00014606 89F9                <1>     mov     ecx, edi
                             <1> sb16_gcd_0:
                             <1>     ; 06/08/2022
00014608 31C0                <1>     xor     eax, eax
                             <1>     ; 20/08/2017
0001460A 803D[18890100]10    <1>     cmp     byte [audio_bps], 16
00014611 750C                <1>     jne     short sb16_gcd_1 ; 8 bit DMA channel
00014613 E4C6                <1>     in      al, 0C6h ; DMA channel 5 count register
                             <1>     ;mov    dl, al
                             <1>     ; 06/08/2022
00014615 88C4                <1>     mov     ah, al
00014617 E4C6                <1>     in      al, 0C6h
                             <1>     ;mov    dh, al
                             <1>     ;movzx  eax, dx
                             <1>     ; 06/08/2022
00014619 86E0                <1>     xchg    ah, al
0001461B D1E0                <1>     shl     eax, 1 ; word count -> byte count
0001461D EB4C                <1>     jmp     short sb16_gcd_2
                             <1> sb16_gcd_1:
0001461F E403                <1>     in      al, 03h ; DMA channel 1 count register
                             <1>     ;mov    dl, al
                             <1>     ; 06/08/2022
00014621 88C4                <1>     mov     ah, al
00014623 E403                <1>     in      al, 03h
                             <1>     ;mov    dh, al
                             <1>     ;movzx  eax, dx
                             <1>     ; 06/08/2022
00014625 86E0                <1>     xchg    ah, al
00014627 EB42                <1>     jmp     short sb16_gcd_2
                             <1> ;sb16_gcd_2:
                             <1> ;   cmp     eax, ecx
                             <1> ;   jnb     short sb16_gcd_3
                             <1> ;   ; remain count < graphics bytes
                             <1> ;   mov     eax, ecx ; fix remain count to data size
                             <1> ;sb16_gcd_3:
                             <1> ;   sub     edi, eax
                             <1> ;   jna     short sb16_gcd_4
                             <1> ;   add     esi, edi ; dma buffer offset
                             <1> ;sb16_gcd_4:
                             <1> ;   mov     edi, ebx ; buffer address (for graphics)
                             <1> ;   mov     [u.r0], ecx
                             <1> ;   rep     movsb
                             <1> ;   retn
                             <1>
                             <1> get_current_sound_data:
                             <1>     ; 05/06/2024
                             <1>     ; 04/06/2024
                             <1>     ; 24/06/2017
                             <1>     ; 22/06/2017
                             <1>     ; get current sound (PCM out) data for graphics
                             <1>     ;
                             <1>     ; ebx = Physical address (on page boundary)
                             <1>     ; ecx = Byte count
                             <1>     ; [audio_buff_size]
                             <1>
                             <1>     ;mov    edi, [audio_buff_size]
                             <1>     ;;;
                             <1>     ; 04/06/2024
                             <1>     ;mov    edi, [audio_dmabuff_size]
                             <1>     ; 05/06/2024
                             <1>     ;mov    edi, [audio_buff_size]
00014629 8B3D[0C890100]      <1>     mov     edi, [dma_hbuff_size]
                             <1>     ;;;
0001462F 8B35[04890100]      <1>     mov     esi, [audio_dma_buff]
                             <1>
                             <1>     ;;;
                             <1>     ; 04/06/2024
                             <1>     ;cmp    byte [audio_device], 2
                             <1>     ;jb     short sb16_current_sound_data ; = 1
                             <1>     ;shr    edi, 1
                             <1>
```

```
                                <1>     ; 04/06/2024
00014635 803D[E5880100]01       <1>     cmp     byte [audio_device], 1
0001463C 7704                   <1>     ja      short gcd_1
0001463E D1E7                   <1>     shl     edi, 1 ; [audio_dmabuff_size] = 2 * edi
                                <1>     ; 05/06/2024
                                <1>     ; edi = [dma_hbuff_size] * 2
00014640 EBC0                   <1>     jmp     short sb16_current_sound_data
                                <1> gcd_1:
                                <1>     ;;;
                                <1>
                                <1> ; 04/06/2024
                                <1> %if 0
                                <1>     cmp     edi, ecx
                                <1>     jnb     short gcd_0
                                <1>     mov     ecx, edi
                                <1> gcd_0:
                                <1> %endif
00014642 803D[E5880100]03       <1>     cmp     byte [audio_device], 3
00014649 7237                   <1>     jb      short ac97_current_sound_data ; = 2
                                <1>     ; = 3
                                <1> vt8233_current_sound_data:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 22/06/2017
                                <1>     ; 21/06/2017
                                <1>     ; get current sound (PCM out) data for graphics
                                <1>     ; (for VT 8233, VT 8237R)
                                <1>     ; ebx = Physical address (on page boundary)
                                <1>     ; ecx = Byte count
                                <1>     ; [audio_buff_size]
                                <1>
                                <1>     ;;mov   edi, [audio_buff_size]
                                <1>     ;mov    edi, [audio_dmabuff_size]
                                <1>     ;mov    esi, [audio_dma_buff]
                                <1>     ;shr    edi, 1
                                <1>     ;cmp    edi, ecx
                                <1>     ;jnb    short vt8233_gcd_1
                                <1>     ;mov    ecx, edi
                                <1>
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ; edi = [audio_buff_size]
                                <1>     ; 05/06/2024
                                <1>     ;and    di, ~1  ; word alignment
                                <1>     ; edi = [dma_hbuff_size]
0001464B 39CF                   <1>     cmp     edi, ecx
0001464D 7302                   <1>     jnb     short vt8233_gcd_1
0001464F 89F9                   <1>     mov     ecx, edi
                                <1>     ;;;
                                <1>
                                <1> vt8233_gcd_1:
                                <1>     ;mov    edx, VIA_REG_OFFSET_CURR_COUNT
                                <1>     ; 06/08/2022
00014651 31D2                   <1>     xor     edx, edx
00014653 B20C                   <1>     mov     dl, VIA_REG_OFFSET_CURR_COUNT
00014655 E877F5FFFF             <1>     call    ctrl_io_r32
0001465A 89C2                   <1>     mov     edx, eax ; remain count (bits 23-0),
                                <1>                      ; SGD index (bits 31-24)
0001465C 81E200000001           <1>     and     edx, 1000000h ; SGD index (0 = 1st half)
00014662 7402                   <1>     jz      short vt8233_gcd_2
                                <1>     ; the second half of DMA buffer
00014664 01FE                   <1>     add     esi, edi
                                <1> vt8233_gcd_2:
00014666 25FFFFFF00             <1>     and     eax, 0FFFFFFh ; bits 23-0
                                <1> ac97_gcd_2:
                                <1> sb16_gcd_2:
0001466B 39C8                   <1>     cmp     eax, ecx
0001466D 7302                   <1>     jnb     short vt8233_gcd_3
                                <1>     ; remain count < graphics bytes
0001466F 89C8                   <1>     mov     eax, ecx ; fix remain count to data size
                                <1> vt8233_gcd_3:
00014671 29C7                   <1>     sub     edi, eax
00014673 7602                   <1>     jna     short vt8233_gcd_4
00014675 01FE                   <1>     add     esi, edi ; dma buffer offset
                                <1> vt8233_gcd_4:
00014677 89DF                   <1>     mov     edi, ebx ; buffer address (for graphics)
00014679 890D[6C8E0100]         <1>     mov     [u.r0], ecx
0001467F F3A4                   <1>     rep     movsb
                                <1> vt8233_gcd_5:
00014681 C3                     <1>     retn
                                <1>
                                <1> ac97_current_sound_data:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 23/06/2017
                                <1>     ; 22/06/2017
                                <1>     ; get current sound (PCM out) data for graphics
                                <1>     ; (for AC'97, ICH)
                                <1>     ; ebx = Physical address (on page boundary)
                                <1>     ; ecx = Byte count
                                <1>     ; [audio_buff_size]
                                <1>
                                <1>     ;;mov   edi, [audio_buff_size]
                                <1>     ;mov    edi, [audio_dmabuff_size]
                                <1>     ;mov    esi, [audio_dma_buff]
                                <1>     ;shr    edi, 1
                                <1>     ;cmp    edi, ecx
                                <1>     ;jnb    short ac97_gcd_0
                                <1>     ;mov    ecx, edi
                                <1>
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ; edi = [audio_buff_size]
                                <1>     ; 05/06/2024
                                <1>     ;and    di, ~7  ; 8 byte alignment
                                <1>     ; edi = [dma_hbuff_size]
00014682 39CF                   <1>     cmp     edi, ecx
00014684 7302                   <1>     jnb     short ac97_gcd_0
00014686 89F9                   <1>     mov     ecx, edi
                                <1>     ;;;
                                <1>
                                <1> ac97_gcd_0:
00014688 66BA1400               <1>     mov     dx, PO_CIV_REG ; Position In Current Buff Reg
0001468C 660315[EA880100]       <1>     add     dx, [NABMBAR]
00014693 EC                     <1>     in      al, dx ; current index value
00014694 A801                   <1>     test    al, 1
00014696 7402                   <1>     jz      short ac97_gcd_1
00014698 01FE                   <1>     add     esi, edi
                                <1> ac97_gcd_1:
0001469A 31C0                   <1>     xor     eax, eax
0001469C 66BA1800               <1>     mov     dx, PO_PICB_REG ; Position In Current Buff Reg
000146A0 660315[EA880100]       <1>     add     dx, [NABMBAR]
                                <1>     ;in     ax, dx ; remain dwords
```

```
                                <1>     ;shl    eax, 2 ; remain bytes ; 23/06/2017
                                <1>     ;;;
                                <1>     ; 04/06/2024
000146A7 66ED                   <1>     in      ax, dx ; remain words
000146A9 D1E0                   <1>     shl     eax, 1 ; remain bytes
                                <1>     ;;;
000146AB EBBE                   <1>     jmp     short ac97_gcd_2
                                <1> ;   cmp     eax, ecx
                                <1> ;   jnb     short ac97_gcd_2
                                <1> ;   ; remain count < graphics bytes
                                <1> ;   mov     eax, ecx ; fix remain count to data size
                                <1> ;ac97_gcd_2:
                                <1> ;   sub     edi, eax
                                <1> ;   jna     short ac97_gcd_3
                                <1> ;   add     esi, edi ; dma buffer offset
                                <1> ;ac97_gcd_3:
                                <1> ;   mov     edi, ebx ; buffer address (for graphics)
                                <1> ;   mov     [u.r0], ecx
                                <1> ;   rep     movsb
                                <1> ;   retn
                                <1>
                                <1> sb16_get_dma_buff_off:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 28/10/2017
                                <1>     ; 24/06/2017
                                <1>     ; 22/06/2017
                                <1>     ; get current (PCM OUT DMA buffer) pointer
                                <1>     ; (for Sound Blaster 16)
                                <1>
                                <1>     ;mov    ecx, [audio_dmabuff_size]
                                <1>     ;xor    ebx, ebx
                                <1>     ;shr    ecx, 1
                                <1>
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ; ecx = audio buffer size
                                <1>     ; 05/06/2024
                                <1>     ; ecx = DMA half buffer size
000146AD D1E1                   <1>     shl     ecx, 1 ; * 2
                                <1>     ; ecx = DMA buffer size
                                <1>     ;;;
                                <1>
                                <1> sb16_gdmabo_0:
                                <1>     ; 28/10/2017
000146AF 803D[18890100]10       <1>     cmp     byte [audio_bps], 16
000146B6 750F                   <1>     jne     short sb16_gdmabo_1 ; 8 bit DMA channel
                                <1>     ; 16 bit DMA channel
000146B8 E4C6                   <1>     in      al, 0C6h ; DMA channel 5 count register
000146BA 88C2                   <1>     mov     dl, al
000146BC E4C6                   <1>     in      al, 0C6h
000146BE 88C6                   <1>     mov      dh, al
000146C0 0FB7C2                 <1>     movzx   eax, dx
000146C3 D1E0                   <1>     shl     eax, 1 ; word count -> byte count
000146C5 EB3A                   <1>     jmp     short sb16_gdmabo_2
                                <1> sb16_gdmabo_1:
000146C7 E403                   <1>     in      al, 03h ; DMA channel 1 count register
000146C9 88C2                   <1>     mov     dl, al
000146CB E403                   <1>     in      al, 03h
000146CD 88C6                   <1>     mov      dh, al
000146CF 0FB7C2                 <1>     movzx   eax, dx
000146D2 EB2D                   <1>     jmp     short sb16_gdmabo_2
                                <1>
                                <1> get_dma_buffer_offset:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024
                                <1>     ; 24/06/2017
                                <1>     ; 22/06/2017
                                <1>     ; get current sound (PCM out) data for graphics
                                <1>     ;
                                <1>     ; ebx = Physical address (on page boundary)
                                <1>     ; ecx = Byte count
                                <1>     ; [audio_buff_size]
                                <1>
                                <1>     ;mov    ecx, [audio_dmabuff_size]
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ;mov    ecx, [audio_buff_size]
                                <1>     ; 05/06/2024
000146D4 8B0D[0C890100]         <1>     mov     ecx, [dma_hbuff_size]
                                <1>     ;;;
000146DA 31DB                   <1>     xor     ebx, ebx
                                <1> gdmabo_0:
000146DC 803D[E5880100]02       <1>     cmp     byte [audio_device], 2
000146E3 72C8                   <1>     jb      short sb16_get_dma_buff_off
000146E5 7427                   <1>     je      short ac97_get_dma_buff_off
                                <1>
                                <1> vt8233_get_dma_buff_off:
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 06/08/2022 - TRDOS 386 v2.0.5
                                <1>     ; 24/06/2017
                                <1>     ; 22/06/2017
                                <1>     ; get current (PCM OUT DMA buffer) pointer
                                <1>     ; (for VT 8233, VT 8237R)
                                <1>
                                <1>     ;mov    ecx, [audio_dmabuff_size]
                                <1>     ;xor    ebx, ebx
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ;shr    ecx, 1
                                <1>     ; 05/06/2024
                                <1>     ;and    cl, ~1
                                <1>     ; ecx = DMA Half Buffer Size (word aligned)
                                <1>     ;;;
                                <1> vt8233_gdmabo_0:
                                <1>     ;mov    edx, VIA_REG_OFFSET_CURR_COUNT
                                <1>     ; 06/08/2022
000146E7 31D2                   <1>     xor     edx, edx
000146E9 B20C                   <1>     mov     dl, VIA_REG_OFFSET_CURR_COUNT
000146EB E8E1F4FFFF             <1>     call    ctrl_io_r32
000146F0 89C2                   <1>     mov     edx, eax ; remain count (bits 23-0),
                                <1>                      ; SGD index (bits 31-24)
000146F2 81E200000001           <1>     and     edx, 1000000h ; SGD index (0 = 1st half)
000146F8 7402                   <1>     jz      short vt8233_gdmabo_1
                                <1>     ; the second half of DMA buffer
000146FA 89CB                   <1>     mov     ebx, ecx
                                <1> vt8233_gdmabo_1:
000146FC 25FFFFFF00             <1>     and     eax, 0FFFFFFh ; bits 23-0
                                <1> sb16_gdmabo_2:
                                <1> ac97_gdmabo_2:
00014701 29C1                   <1>     sub     ecx, eax
00014703 7602                   <1>     jna     short vt8233_gdmabo_2
00014705 01CB                   <1>     add     ebx, ecx ; dma buffer offset
```

```
                                <1> vt8233_gdmabo_2:
00014707 891D[6C8E0100]         <1>     mov     [u.r0], ebx
0001470D C3                     <1>     retn
                                <1>
                                <1> ac97_get_dma_buff_off:
                                <1>     ; 06/06/2024
                                <1>     ; 05/06/2024
                                <1>     ; 04/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 24/06/2017
                                <1>     ; 22/06/2017
                                <1>     ; get current (PCM OUT DMA buffer) pointer
                                <1>     ; (for AC'97, ICH)
                                <1>     ; ebx = Physical address (on page boundary)
                                <1>     ; ecx = Byte count
                                <1>     ; [audio_buff_size]
                                <1>
                                <1>     ;mov   ecx, [audio_dmabuff_size]
                                <1>     ;xor   ebx, ebx
                                <1>     ;;;
                                <1>     ; 04/06/2024
                                <1>     ;shr   ecx, 1
                                <1>     ; 05/06/2024
                                <1>     ;and   cl, ~7 ; (truncate bytes if out of 8x)
                                <1>     ; ecx = DMA Half Buffer Size (8 byte aligned)
                                <1>     ;;;
                                <1> ac97_gdmabo_0:
0001470E 66BA1400               <1>     mov     dx, PO_CIV_REG ; Position In Current Buff Reg
00014712 660315[EA880100]       <1>     add     dx, [NABMBAR]
00014719 EC                     <1>     in      al, dx ; current index value
0001471A A801                   <1>     test    al, 1
0001471C 7402                   <1>     jz      short ac97_gdmabo_1
0001471E 89CB                   <1>     mov     ebx, ecx
                                <1> ac97_gdmabo_1:
00014720 31C0                   <1>     xor     eax, eax
00014722 66BA1800               <1>     mov     dx, PO_PICB_REG ; Position In Current Buff Reg
00014726 660315[EA880100]       <1>     add     dx, [NABMBAR]
0001472D 66ED                   <1>     in      ax, dx ; remain words (samples)
                                <1>     ;;;
                                <1>     ; 06/06/2024
                                <1>     ; audio samples are counted as words
                                <1>     ; 04/06/2024 (BugFix)
0001472F D1E0                   <1>     shl     eax, 1 ; remain bytes
                                <1>     ;;;
00014731 EBCE                   <1>     jmp     short ac97_gdmabo_2
                                <1>
                                <1> ac97_codec_info:
                                <1>     ; 06/06/2024
                                <1>     ; 05/06/2024 - TRDOS 386 v2.0.8
                                <1>     ; 19/11/2023
                                <1>     ; ENTRY: none
                                <1>     ; RETURN:
                                <1>     ;       eax = Extended Audio ID (MX28) in ax
                                <1>     ;        ax bit 0 - VRA bit
                                <1>     ;        hw of eax = PCM output sample rate (48000)
                                <1>     ;       ebx = VENDOR ID 1, VENDOR ID 2 (bx)
                                <1>     ;       cf = 1 -> error
                                <1>
                                <1>     ; 06/06/2024
                                <1>     ;;;
00014733 66BA3000               <1>     mov     dx, GLOB_STS_REG ; 30h
00014737 660315[EA880100]       <1>     add     dx, [NABMBAR]
0001473E ED                     <1>     in      eax, dx
                                <1>
                                <1>     ;cmp   eax, 0FFFFFFFFh ; -1
0001473F 40                     <1>     inc     eax
00014740 7408                   <1>     jz      short ac97_c_inf_err
00014742 48                     <1>     dec     eax
                                <1>
00014743 2500030010             <1>     and     eax, CTRL_ST_CREADY
00014748 7502                   <1>     jnz     short ac97_c_inf_1
                                <1>
                                <1> ac97_c_inf_err:
0001474A F9                     <1>     stc
0001474B C3                     <1>     retn
                                <1>     ;;;
                                <1>
                                <1> ac97_c_inf_1:
0001474C 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00014753 6683C22C               <1>     add     dx, CODEC_PCM_FRONT_DACRATE_REG
00014757 66ED                   <1>     in      ax, dx  ; PCM Output Sample Rate
                                <1>                     ; 48000 Hz
00014759 C1E010                 <1>     shl     eax, 16
0001475C E810F4FFFF             <1>     call    delay_100ms
00014761 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
00014768 6683C228               <1>     add     dx, CODEC_EXT_AUDIO_REG ; 28h
0001476C 66ED                   <1>     in      ax, dx
0001476E 50                     <1>     push    eax
0001476F E8FDF3FFFF             <1>     call    delay_100ms
00014774 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
0001477B 6683C27C               <1>     add     dx, CODEC_VENDOR_ID1
0001477F 66ED                   <1>     in      ax, dx
00014781 C1E010                 <1>     shl     eax, 16
00014784 668B15[E8880100]       <1>     mov     dx, [NAMBAR]
0001478B 6683C27E               <1>     add     dx, CODEC_VENDOR_ID2
0001478F 66ED                   <1>     in      ax, dx
00014791 89C3                   <1>     mov     ebx, eax
00014793 58                     <1>     pop     eax
                                <1>     ;;;;
                                <1>     ; 06/06/2024 (temporary!?)
                                <1>     ; (ALC850 & ALC655 BugFix)
00014794 83FBFF                 <1>     cmp     ebx, 0FFFFFFFFh ; -1
00014797 74B1                   <1>     je      short ac97_c_inf_err ; invalid
00014799 09DB                   <1>     or      ebx, ebx
0001479B 74AD                   <1>     jz      short ac97_c_inf_err ; invalid
0001479D 81FB90474C41           <1>     cmp     ebx, 414C4790h ; ALC850
                                <1>     ;je    short ac97_c_inf_2
                                <1>     ;cmp   ebx, 414C4760h ; ALC655
000147A3 7505                   <1>     jne     short ac97_c_inf_3
                                <1>     ;mov   eax, 0BB8009C6h ; AC655 default (Read only)
                                <1>     ;retn
                                <1> ac97_c_inf_2:
                                <1>     ; ref: ALC850 & ALC655 data sheet
000147A5 B8C40980BB             <1>     mov     eax, 0BB8009C4h ; AC850 default (Read only)
                                <1> ac97_c_inf_3:
                                <1>     ;;;;
000147AA C3                     <1>     retn
                                <1>
                                <1> ; 19/11/2023
                                <1> ; Valid ICH device IDs
                                <1>
                                <1> valid_ids:
000147AB 86801524               <1> dd  (ICH_DID << 16) + INTEL_VID    ; 8086h:2415h
000147AF 86802524               <1> dd  (ICH0_DID << 16) + INTEL_VID   ; 8086h:2425h
000147B3 86804524               <1> dd  (ICH2_DID << 16) + INTEL_VID   ; 8086h:2445h
```

```
000147B7 86808524            <1> dd  (ICH3_DID << 16) + INTEL_VID   ; 8086h:2485h
000147BB 8680C524            <1> dd  (ICH4_DID << 16) + INTEL_VID   ; 8086h:24C5h
000147BF 8680D524            <1> dd  (ICH5_DID << 16) + INTEL_VID   ; 8086h:24D5h
000147C3 86806E26            <1> dd  (ICH6_DID << 16) + INTEL_VID   ; 8086h:266Eh
000147C7 8680A625            <1> dd  (ESB6300_DID << 16) + INTEL_VID  ; 8086h:25A6h
000147CB 86809826            <1> dd  (ESB631X_DID << 16) + INTEL_VID  ; 8086h:2698h
000147CF 8680DE27            <1> dd  (ICH7_DID << 16) + INTEL_VID   ; 8086h:27DEh
000147D3 86809571            <1> dd  (MX82440_DID << 16) + INTEL_VID  ; 8086h:7195h
000147D7 39101270            <1> dd  (SI7012_DID << 16)  + SIS_VID  ; 1039h:7012h
000147DB DE10B101            <1> dd  (NFORCE_DID << 16)  + NVIDIA_VID ; 10DEh:01B1h
000147DF DE106A00            <1> dd  (NFORCE2_DID << 16) + NVIDIA_VID ; 10DEh:006Ah
000147E3 22106D74            <1> dd  (AMD8111_DID << 16) + AMD_VID         ; 1022h:746Dh
000147E7 22104574            <1> dd  (AMD768_DID << 16)  + AMD_VID         ; 1022h:7445h
000147EB DE105900            <1> dd  (CK804_DID << 16) + NVIDIA_VID        ; 10DEh:0059h
000147EF DE103A00            <1> dd  (MCP04_DID << 16) + NVIDIA_VID        ; 10DEh:003Ah
000147F3 DE108A00            <1> dd  (CK8_DID << 16) + NVIDIA_VID   ; 1022h:008Ah
000147F7 DE10DA00            <1> dd  (NFORCE3_DID << 16) + NVIDIA_VID ; 10DEh:00DAh
000147FB DE10EA00            <1> dd  (CK8S_DID << 16) + NVIDIA_VID  ; 10DEh:00EAh
                             <1>
                             <1> valid_id_count:    equ ($ - valid_ids)>>2 ; 19/11/2023

000147FF 90                     align 4

                                %include 'vgadata.s' ; 04/07/2016
                             <1> ; ********************************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.3 - vgadata.s  (palette and font data)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Last Update: 01/01/2021
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Beginning: 16/01/2016
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.15 (trdos386.s)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Turkish Rational DOS
                             <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                             <1> ;
                             <1> ; Derived from 'Plex86/Bochs VGABios' source code, vgabios-0.7a (2011)
                             <1> ; by the LGPL VGABios Developers Team (2001-2008), 'vgatables.h'
                             <1> ;
                             <1> ; Oracle VirtualBox 5.0.24 VGABios Source Code
                             <1> ; ('vgabios.c', 'vgatables.h', 'vgafonts.h', 'vgarom.asm')
                             <1> ;
                             <1> ; Palette and font data in assembly language format:
                             <1> ; 'VBoxVgaBiosAlternative.asm'
                             <1>
                             <1> ; ********************************************************************************************
                             <1>
                             <1> ; 25/11/2020 (TRDOS 386 v2.0.3)
                             <1> ; ('vgatables.h' - 30/12/2019 - vruppert)
                             <1>
                             <1> ; 04/07/2016
                             <1> ; COLOR DATA
                             <1>
                             <1> palette0: ; (63+1)*3
00014800 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00014809 00000000000000      <1>
00014810 000000000000000002A- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014819 2A2A2A2A2A2A2A      <1>
00014820 2A2A2A2A2A2A2A2A2A- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014829 2A2A2A2A2A2A2A      <1>
00014830 2A2A2A2A2A2A2A2A2A- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014839 2A2A2A2A2A2A2A      <1>
00014840 2A2A2A2A2A2A2A2A3F- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh
00014849 3F3F3F3F3F3F3F      <1>
00014850 3F3F3F3F3F3F3F3F3F- <1>     db  03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh
00014859 3F3F3F3F3F3F3F      <1>
00014860 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00014869 00000000000000      <1>
00014870 000000000000000002A- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014879 2A2A2A2A2A2A2A      <1>
00014880 2A2A2A2A2A2A2A2A2A- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014889 2A2A2A2A2A2A2A      <1>
00014890 2A2A2A2A2A2A2A2A2A- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah
00014899 2A2A2A2A2A2A2A      <1>
000148A0 2A2A2A2A2A2A2A2A3F- <1>     db  02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 02ah, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh
000148A9 3F3F3F3F3F3F3F      <1>
000148B0 3F3F3F3F3F3F3F3F3F- <1>     db  03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh, 03fh
000148B9 3F3F3F3F3F3F3F      <1>
                             <1> palette1: ; (63+1)*3
000148C0 00000000002A002A00- <1>     db  000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah, 000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah
000148C9 002A2A2A00002A      <1>
000148D0 002A2A15002A2A2A00- <1>     db  000h, 02ah, 02ah, 015h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah
000148D9 000000002A002A      <1>
000148E0 00002A2A2A00002A00- <1>     db  000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah, 000h, 02ah, 02ah, 015h, 000h, 02ah, 02ah, 02ah
000148E9 2A2A15002A2A2A      <1>
000148F0 15151515153F153F15- <1>     db  015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh, 015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh
000148F9 153F3F3F15153F      <1>
00014900 153F3F3F153F3F3F15- <1>     db  015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh
00014909 151515153F153F      <1>
00014910 15153F3F3F15153F15- <1>     db  015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh
00014919 3F3F3F153F3F3F      <1>
00014920 00000000002A002A00- <1>     db  000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah, 000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah
00014929 002A2A2A00002A      <1>
00014930 002A2A15002A2A2A00- <1>     db  000h, 02ah, 02ah, 015h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah
00014939 000000002A002A      <1>
00014940 00002A2A2A00002A00- <1>     db  000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah, 000h, 02ah, 02ah, 015h, 000h, 02ah, 02ah, 02ah
00014949 2A2A15002A2A2A      <1>
00014950 15151515153F153F15- <1>     db  015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh, 015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh
00014959 153F3F3F15153F      <1>
00014960 153F3F3F153F3F3F15- <1>     db  015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh
00014969 151515153F153F      <1>
00014970 15153F3F3F15153F15- <1>     db  015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh
00014979 3F3F3F153F3F3F      <1>
                             <1> palette2: ; (63+1)*3
00014980 00000000002A002A00- <1>     db  000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah, 000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah
00014989 002A2A2A00002A      <1>
00014990 002A2A2A002A2A2A00- <1>     db  000h, 02ah, 02ah, 02ah, 000h, 02ah, 02ah, 02ah, 000h, 000h, 015h, 000h, 000h, 03fh, 000h, 02ah
00014999 001500003F002A      <1>
000149A0 15002A3F2A00152A00- <1>     db  015h, 000h, 02ah, 03fh, 02ah, 000h, 015h, 02ah, 000h, 03fh, 02ah, 02ah, 015h, 02ah, 02ah, 03fh
000149A9 3F2A2A152A2A3F      <1>
000149B0 00150000152A003F00- <1>     db  000h, 015h, 000h, 000h, 015h, 02ah, 000h, 03fh, 000h, 000h, 03fh, 02ah, 02ah, 015h, 000h, 02ah
000149B9 003F2A2A15002A      <1>
000149C0 152A2A3F002A3F2A00- <1>     db  015h, 02ah, 02ah, 03fh, 000h, 02ah, 03fh, 02ah, 000h, 015h, 015h, 000h, 015h, 03fh, 000h, 03fh
000149C9 151500153F003F      <1>
000149D0 15003F3F2A15152A15- <1>     db  015h, 000h, 03fh, 03fh, 02ah, 015h, 015h, 02ah, 015h, 03fh, 02ah, 03fh, 015h, 02ah, 03fh, 03fh
000149D9 3F2A3F152A3F3F      <1>
000149E0 15000015002A152A00- <1>     db  015h, 000h, 000h, 015h, 000h, 02ah, 015h, 02ah, 000h, 015h, 02ah, 02ah, 03fh, 000h, 000h, 03fh
000149E9 152A2A3F00003F      <1>
000149F0 002A3F2A003F2A2A15- <1>     db  000h, 02ah, 03fh, 02ah, 000h, 03fh, 02ah, 02ah, 015h, 000h, 015h, 015h, 000h, 03fh, 015h, 02ah
000149F9 001515003F152A      <1>
00014A00 15152A3F3F00153F00- <1>     db  015h, 015h, 02ah, 03fh, 03fh, 000h, 015h, 03fh, 000h, 03fh, 03fh, 02ah, 015h, 03fh, 02ah, 03fh
00014A09 3F3F2A153F2A3F      <1>
00014A10 15150015152A153F00- <1>     db  015h, 015h, 000h, 015h, 015h, 02ah, 015h, 03fh, 000h, 015h, 03fh, 02ah, 03fh, 015h, 000h, 03fh
00014A19 153F2A3F15003F      <1>
```

```
00014A20 152A3F3F003F3F2A15- <1>     db  015h, 02ah, 03fh, 03fh, 000h, 03fh, 03fh, 02ah, 015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh
00014A29 151515153F153F      <1>
00014A30 15153F3F15153F15- <1>     db  015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh
00014A39 3F3F3F153F3F3F      <1>
                             <1> palette3: ; 256*3
00014A40 00000000002A002A00- <1>     db  000h, 000h, 000h, 000h, 000h, 02ah, 000h, 02ah, 000h, 000h, 02ah, 02ah, 02ah, 000h, 000h, 02ah
00014A49 002A2A2A00002A      <1>
00014A50 002A2A15002A2A2A15- <1>     db  000h, 02ah, 02ah, 015h, 000h, 02ah, 02ah, 02ah, 015h, 015h, 015h, 015h, 015h, 03fh, 015h, 03fh
00014A59 151515153F153F      <1>
00014A60 15153F3F15153F15- <1>     db  015h, 015h, 03fh, 03fh, 03fh, 015h, 015h, 03fh, 015h, 03fh, 03fh, 03fh, 015h, 03fh, 03fh, 03fh
00014A69 3F3F3F153F3F3F      <1>
00014A70 000000050505080808- <1>     db  000h, 000h, 000h, 005h, 005h, 005h, 008h, 008h, 008h, 00bh, 00bh, 00bh, 00eh, 00eh, 00eh, 011h
00014A79 0B0B0B0E0E0E11      <1>
00014A80 11111414141818181C- <1>     db  011h, 011h, 014h, 014h, 014h, 018h, 018h, 018h, 01ch, 01ch, 01ch, 020h, 020h, 020h, 024h, 024h
00014A89 1C1C2020202424      <1>
00014A90 242828282D2D2D3232- <1>     db  024h, 028h, 028h, 028h, 02dh, 02dh, 02dh, 032h, 032h, 032h, 038h, 038h, 038h, 03fh, 03fh, 03fh
00014A99 323838383F3F3F      <1>
00014AA0 00003F10003F1F003F- <1>     db  000h, 000h, 03fh, 010h, 000h, 03fh, 01fh, 000h, 03fh, 02fh, 000h, 03fh, 03fh, 000h, 03fh, 03fh
00014AA9 2F003F3F003F3F      <1>
00014AB0 002F3F001F3F00103F- <1>     db  000h, 02fh, 03fh, 000h, 01fh, 03fh, 000h, 010h, 03fh, 000h, 000h, 03fh, 010h, 000h, 03fh, 01fh
00014AB9 00003F10003F1F      <1>
00014AC0 003F2F003F3F002F3F- <1>     db  000h, 03fh, 02fh, 000h, 03fh, 03fh, 000h, 02fh, 03fh, 000h, 01fh, 03fh, 000h, 010h, 03fh, 000h
00014AC9 001F3F00103F00      <1>
00014AD0 003F00003F10003F1F- <1>     db  000h, 03fh, 000h, 000h, 03fh, 010h, 000h, 03fh, 01fh, 000h, 03fh, 02fh, 000h, 03fh, 03fh, 000h
00014AD9 003F2F003F3F00      <1>
00014AE0 2F3F001F3F00103F1F- <1>     db  02fh, 03fh, 000h, 01fh, 03fh, 000h, 010h, 03fh, 01fh, 01fh, 03fh, 027h, 01fh, 03fh, 02fh, 01fh
00014AE9 1F3F271F3F2F1F      <1>
00014AF0 3F371F3F3F1F3F3F1F- <1>     db  03fh, 037h, 01fh, 03fh, 03fh, 01fh, 03fh, 03fh, 01fh, 037h, 03fh, 01fh, 02fh, 03fh, 01fh, 027h
00014AF9 373F1F2F3F1F27      <1>
00014B00 3F1F1F3F271F3F2F1F- <1>     db  03fh, 01fh, 01fh, 03fh, 027h, 01fh, 03fh, 02fh, 01fh, 03fh, 037h, 01fh, 03fh, 03fh, 01fh, 037h
00014B09 3F371F3F3F1F37      <1>
00014B10 3F1F2F3F1F273F1F1F- <1>     db  03fh, 01fh, 02fh, 03fh, 01fh, 027h, 03fh, 01fh, 01fh, 03fh, 01fh, 01fh, 03fh, 027h, 01fh, 03fh
00014B19 3F1F1F3F271F3F      <1>
00014B20 2F1F3F371F3F3F1F37- <1>     db  02fh, 01fh, 03fh, 037h, 01fh, 03fh, 03fh, 01fh, 037h, 03fh, 01fh, 02fh, 03fh, 01fh, 027h, 03fh
00014B29 3F1F2F3F1F273F      <1>
00014B30 2D2D3F312D3F362D3F- <1>     db  02dh, 02dh, 03fh, 031h, 02dh, 03fh, 036h, 02dh, 03fh, 03ah, 02dh, 03fh, 03fh, 02dh, 03fh, 03fh
00014B39 3A2D3F3F2D3F3F      <1>
00014B40 2D3A3F2D363F2D313F- <1>     db  02dh, 03ah, 03fh, 02dh, 036h, 03fh, 02dh, 031h, 03fh, 02dh, 02dh, 03fh, 031h, 02dh, 03fh, 036h
00014B49 2D2D3F312D3F36      <1>
00014B50 2D3F3A2D3F3F2D3A3F- <1>     db  02dh, 03fh, 03ah, 02dh, 03fh, 03fh, 02dh, 03ah, 03fh, 02dh, 036h, 03fh, 02dh, 031h, 03fh, 02dh
00014B59 2D363F2D313F2D      <1>
00014B60 2D3F2D2D3F312D3F36- <1>     db  02dh, 03fh, 02dh, 02dh, 03fh, 031h, 02dh, 03fh, 036h, 02dh, 03fh, 03ah, 02dh, 03fh, 03fh, 02dh
00014B69 2D3F3A2D3F3F2D      <1>
00014B70 3A3F2D363F2D313F00- <1>     db  03ah, 03fh, 02dh, 036h, 03fh, 02dh, 031h, 03fh, 000h, 000h, 01ch, 007h, 000h, 01ch, 00eh, 000h
00014B79 001C07001C0E00      <1>
00014B80 1C15001C1C001C1C00- <1>     db  01ch, 015h, 000h, 01ch, 01ch, 000h, 01ch, 01ch, 000h, 015h, 01ch, 000h, 00eh, 01ch, 000h, 007h
00014B89 151C000E1C0007      <1>
00014B90 1C00001C07001C0E00- <1>     db  01ch, 000h, 000h, 01ch, 007h, 000h, 01ch, 00eh, 000h, 01ch, 015h, 000h, 01ch, 01ch, 000h, 015h
00014B99 1C15001C1C0015      <1>
00014BA0 1C000E1C00071C0000- <1>     db  01ch, 000h, 00eh, 01ch, 000h, 007h, 01ch, 000h, 000h, 01ch, 000h, 000h, 01ch, 007h, 000h, 01ch
00014BA9 1C00001C07001C      <1>
00014BB0 0E001C15001C1C0015- <1>     db  00eh, 000h, 01ch, 015h, 000h, 01ch, 01ch, 000h, 015h, 01ch, 000h, 00eh, 01ch, 000h, 007h, 01ch
00014BB9 1C000E1C00071C      <1>
00014BC0 0E0E1C110E1C150E1C- <1>     db  00eh, 00eh, 01ch, 011h, 00eh, 01ch, 015h, 00eh, 01ch, 018h, 00eh, 01ch, 01ch, 00eh, 01ch, 01ch
00014BC9 180E1C1C0E1C1C      <1>
00014BD0 0E181C0E151C0E111C- <1>     db  00eh, 018h, 01ch, 00eh, 015h, 01ch, 00eh, 011h, 01ch, 00eh, 00eh, 01ch, 011h, 00eh, 01ch, 015h
00014BD9 0E0E1C110E1C15      <1>
00014BE0 0E1C180E1C1C0E181C- <1>     db  00eh, 01ch, 018h, 00eh, 01ch, 01ch, 00eh, 018h, 01ch, 00eh, 015h, 01ch, 00eh, 011h, 01ch, 00eh
00014BE9 0E151C0E111C0E      <1>
00014BF0 0E1C0E0E1C110E1C15- <1>     db  00eh, 01ch, 00eh, 00eh, 01ch, 011h, 00eh, 01ch, 015h, 00eh, 01ch, 018h, 00eh, 01ch, 01ch, 00eh
00014BF9 0E1C180E1C1C0E      <1>
00014C00 181C0E151C0E111C14- <1>     db  018h, 01ch, 00eh, 015h, 01ch, 00eh, 011h, 01ch, 014h, 014h, 01ch, 016h, 014h, 01ch, 018h, 014h
00014C09 141C16141C1814      <1>
00014C10 1C1A141C1C141C1C14- <1>     db  01ch, 01ah, 014h, 01ch, 01ch, 014h, 01ch, 01ch, 014h, 01ah, 01ch, 014h, 018h, 01ch, 014h, 016h
00014C19 1A1C14181C1416      <1>
00014C20 1C14141C16141C1814- <1>     db  01ch, 014h, 014h, 01ch, 016h, 014h, 01ch, 018h, 014h, 01ch, 01ah, 014h, 01ch, 01ch, 014h, 01ah
00014C29 1C1A141C1C141A      <1>
00014C30 1C14181C14161C1414- <1>     db  01ch, 014h, 018h, 01ch, 014h, 016h, 01ch, 014h, 014h, 01ch, 014h, 014h, 01ch, 016h, 014h, 01ch
00014C39 1C14141C16141C      <1>
00014C40 18141C1A141C1C141A- <1>     db  018h, 014h, 01ch, 01ah, 014h, 01ch, 01ch, 014h, 01ah, 01ch, 014h, 018h, 01ch, 014h, 016h, 01ch
00014C49 1C14181C14161C      <1>
00014C50 000010040010080010- <1>     db  000h, 000h, 010h, 004h, 000h, 010h, 008h, 000h, 010h, 00ch, 000h, 010h, 010h, 000h, 010h, 010h
00014C59 0C001010001010      <1>
00014C60 000C10000810000410- <1>     db  000h, 00ch, 010h, 000h, 008h, 010h, 000h, 004h, 010h, 000h, 000h, 010h, 004h, 000h, 010h, 008h
00014C69 00001004001008      <1>
00014C70 00100C001010000C10- <1>     db  000h, 010h, 00ch, 000h, 010h, 010h, 000h, 00ch, 010h, 000h, 008h, 010h, 000h, 004h, 010h, 000h
00014C79 00081000041000      <1>
00014C80 001000001004001008- <1>     db  000h, 010h, 000h, 000h, 010h, 004h, 000h, 010h, 008h, 000h, 010h, 00ch, 000h, 010h, 010h, 000h
00014C89 00100C00101000      <1>
00014C90 0C1000081000041008- <1>     db  00ch, 010h, 000h, 008h, 010h, 000h, 004h, 010h, 008h, 008h, 010h, 00ah, 008h, 010h, 00ch, 008h
00014C99 08100A08100C08      <1>
00014CA0 100E08101008101008- <1>     db  010h, 00eh, 008h, 010h, 010h, 008h, 010h, 010h, 008h, 00eh, 010h, 008h, 00ch, 010h, 008h, 00ah
00014CA9 0E10080C10080A      <1>
00014CB0 100808100A08100C08- <1>     db  010h, 008h, 008h, 010h, 00ah, 008h, 010h, 00ch, 008h, 010h, 00eh, 008h, 010h, 010h, 008h, 00eh
00014CB9 100E081010080E      <1>
00014CC0 10080C10080A100808- <1>     db  010h, 008h, 00ch, 010h, 008h, 00ah, 010h, 008h, 008h, 010h, 008h, 008h, 010h, 00ah, 008h, 010h
00014CC9 100808100A0810      <1>
00014CD0 0C08100E081010080E- <1>     db  00ch, 008h, 010h, 00eh, 008h, 010h, 010h, 008h, 00eh, 010h, 008h, 00ch, 010h, 008h, 00ah, 010h
00014CD9 10080C10080A10      <1>
00014CE0 0B0B100C0B100D0B10- <1>     db  00bh, 00bh, 010h, 00ch, 00bh, 010h, 00dh, 00bh, 010h, 00fh, 00bh, 010h, 010h, 00bh, 010h, 010h
00014CE9 0F0B10100B1010      <1>
00014CF0 0B0F100B0D100B0C10- <1>     db  00bh, 00fh, 010h, 00bh, 00dh, 010h, 00bh, 00ch, 010h, 00bh, 00bh, 010h, 00ch, 00bh, 010h, 00dh
00014CF9 0B0B100C0B100D      <1>
00014D00 0B100F0B10100B0F10- <1>     db  00bh, 010h, 00fh, 00bh, 010h, 010h, 00bh, 00fh, 010h, 00bh, 00dh, 010h, 00bh, 00ch, 010h, 00bh
00014D09 0B0D100B0C100B      <1>
00014D10 0B100B0B100C0B100D- <1>     db  00bh, 010h, 00bh, 00bh, 010h, 00ch, 00bh, 010h, 00dh, 00bh, 010h, 00fh, 00bh, 010h, 010h, 00bh
00014D19 0B100F0B10100B      <1>
00014D20 0F100B0D100B0C1000- <1>     db  00fh, 010h, 00bh, 00dh, 010h, 00bh, 00ch, 010h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00014D29 00000000000000      <1>
00014D30 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00014D39 00000000000000      <1>
                             <1>
                             <1>
                             <1> ; 04/07/2016
                             <1> ; FONT DATA
                             <1>
                             <1> CRT_CHAR_GEN:
                             <1> vgafont8:
00014D40 00000000000000007E- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07eh, 081h, 0a5h, 081h, 0bdh, 099h, 081h, 07eh
00014D49 81A581BD99817E      <1>
00014D50 7EFFDBFFC3E7FF7E6C- <1>     db  07eh, 0ffh, 0dbh, 0ffh, 0c3h, 0e7h, 0ffh, 07eh, 06ch, 0feh, 0feh, 0feh, 07ch, 038h, 010h, 000h
00014D59 FEFEFE7C381000      <1>
00014D60 10387CFE7C38100038- <1>     db  010h, 038h, 07ch, 0feh, 07ch, 038h, 010h, 000h, 038h, 07ch, 038h, 0feh, 0feh, 07ch, 038h, 07ch
00014D69 7C38FEFE7C387C      <1>
00014D70 1010387CFE7C387C00- <1>     db  010h, 010h, 038h, 07ch, 0feh, 07ch, 038h, 07ch, 000h, 000h, 018h, 03ch, 03ch, 018h, 000h, 000h
00014D79 00183C3C180000      <1>
00014D80 FFFFE7C3C3E7FFFF00- <1>     db  0ffh, 0ffh, 0e7h, 0c3h, 0c3h, 0e7h, 0ffh, 0ffh, 000h, 03ch, 066h, 042h, 042h, 066h, 03ch, 000h
00014D89 3C664242663C00      <1>
00014D90 FFC399BDBD99C3FF0F- <1>     db  0ffh, 0c3h, 099h, 0bdh, 0bdh, 099h, 0c3h, 0ffh, 00fh, 007h, 00fh, 07dh, 0cch, 0cch, 0cch, 078h
00014D99 070F7DCCCCCC78      <1>
00014DA0 3C6666663C187E183F- <1>     db  03ch, 066h, 066h, 066h, 03ch, 018h, 07eh, 018h, 03fh, 033h, 03fh, 030h, 030h, 070h, 0f0h, 0e0h
00014DA9 333F303070F0E0      <1>
00014DB0 7F637F636367E6C099- <1>     db  07fh, 063h, 07fh, 063h, 063h, 067h, 0e6h, 0c0h, 099h, 05ah, 03ch, 0e7h, 0e7h, 03ch, 05ah, 099h
00014DB9 5A3CE7E73C5A99      <1>
```

```
133 00014DC0 80E0F8FEF8E0800002- <1>     db  080h, 0e0h, 0f8h, 0feh, 0f8h, 0e0h, 080h, 000h, 002h, 00eh, 03eh, 0feh, 03eh, 00eh, 002h, 000h
133 00014DC9 0E3EFE3E0E0200      <1>
134 00014DD0 183C7E18187E3C1866- <1>     db  018h, 03ch, 07eh, 018h, 018h, 07eh, 03ch, 018h, 066h, 066h, 066h, 066h, 066h, 000h, 066h, 000h
134 00014DD9 66666666006600      <1>
135 00014DE0 7FDBDB7B1B1B1B003E- <1>     db  07fh, 0dbh, 0dbh, 07bh, 01bh, 01bh, 01bh, 000h, 03eh, 063h, 038h, 06ch, 06ch, 038h, 0cch, 078h
135 00014DE9 63386C6C38CC78      <1>
136 00014DF0 000000007E7E7E0018- <1>     db  000h, 000h, 000h, 000h, 07eh, 07eh, 07eh, 000h, 018h, 03ch, 07eh, 018h, 07eh, 03ch, 018h, 0ffh
136 00014DF9 3C7E187E3C18FF      <1>
137 00014E00 183C7E181818180018- <1>     db  018h, 03ch, 07eh, 018h, 018h, 018h, 018h, 000h, 018h, 018h, 018h, 018h, 07eh, 03ch, 018h, 000h
137 00014E09 1818187E3C1800      <1>
138 00014E10 00180CFE0C18000000- <1>     db  000h, 018h, 00ch, 0feh, 00ch, 018h, 000h, 000h, 000h, 030h, 060h, 0feh, 060h, 030h, 000h, 000h
138 00014E19 3060FE60300000      <1>
139 00014E20 0000C0C0C0FE000000- <1>     db  000h, 000h, 0c0h, 0c0h, 0c0h, 0feh, 000h, 000h, 000h, 024h, 066h, 0ffh, 066h, 024h, 000h, 000h
139 00014E29 2466FF66240000      <1>
140 00014E30 00183C7EFFFF000000- <1>     db  000h, 018h, 03ch, 07eh, 0ffh, 0ffh, 000h, 000h, 000h, 0ffh, 0ffh, 07eh, 03ch, 018h, 000h, 000h
140 00014E39 FFFF7E3C180000      <1>
141 00014E40 000000000000000030- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 030h, 078h, 078h, 030h, 030h, 000h, 030h, 000h
141 00014E49 78783030003000      <1>
142 00014E50 6C6C6C00000000006C- <1>     db  06ch, 06ch, 06ch, 000h, 000h, 000h, 000h, 000h, 06ch, 06ch, 0feh, 06ch, 0feh, 06ch, 06ch, 000h
142 00014E59 6CFE6CFE6C6C00      <1>
143 00014E60 307CC0780CF8300000- <1>     db  030h, 07ch, 0c0h, 078h, 00ch, 0f8h, 030h, 000h, 000h, 0c6h, 0cch, 018h, 030h, 066h, 0c6h, 000h
143 00014E69 C6CC183066C600      <1>
144 00014E70 386C3876DCCC760060- <1>     db  038h, 06ch, 038h, 076h, 0dch, 0cch, 076h, 000h, 060h, 060h, 0c0h, 000h, 000h, 000h, 000h, 000h
144 00014E79 60C00000000000      <1>
145 00014E80 183060606030180060- <1>     db  018h, 030h, 060h, 060h, 060h, 030h, 018h, 000h, 060h, 030h, 018h, 018h, 018h, 030h, 060h, 000h
145 00014E89 30181818306000      <1>
146 00014E90 00663CFF3C66000000- <1>     db  000h, 066h, 03ch, 0ffh, 03ch, 066h, 000h, 000h, 000h, 030h, 030h, 0fch, 030h, 030h, 000h, 000h
146 00014E99 3030FC30300000      <1>
147 00014EA0 000000000030306000- <1>     db  000h, 000h, 000h, 000h, 000h, 030h, 030h, 060h, 000h, 000h, 000h, 0fch, 000h, 000h, 000h, 000h
147 00014EA9 0000FC00000000      <1>
148 00014EB0 000000000030300006- <1>     db  000h, 000h, 000h, 000h, 000h, 030h, 030h, 000h, 006h, 00ch, 018h, 030h, 060h, 0c0h, 080h, 000h
148 00014EB9 0C183060C08000      <1>
149 00014EC0 7CC6CEDEF6E67C0030- <1>     db  07ch, 0c6h, 0ceh, 0deh, 0f6h, 0e6h, 07ch, 000h, 030h, 070h, 030h, 030h, 030h, 030h, 0fch, 000h
149 00014EC9 7030303030FC00      <1>
150 00014ED0 78CC0C3860CCFC0078- <1>     db  078h, 0cch, 00ch, 038h, 060h, 0cch, 0fch, 000h, 078h, 0cch, 00ch, 038h, 00ch, 0cch, 078h, 000h
150 00014ED9 CC0C380CCC7800      <1>
151 00014EE0 1C3C6CCCFE0C1E00FC- <1>     db  01ch, 03ch, 06ch, 0cch, 0feh, 00ch, 01eh, 000h, 0fch, 0c0h, 0f8h, 00ch, 00ch, 0cch, 078h, 000h
151 00014EE9 C0F80C0CCC7800      <1>
152 00014EF0 3860C0F8CCCC7800FC- <1>     db  038h, 060h, 0c0h, 0f8h, 0cch, 0cch, 078h, 000h, 0fch, 0cch, 00ch, 018h, 030h, 030h, 030h, 000h
152 00014EF9 CC0C1830303000      <1>
153 00014F00 78CCCC78CCCC780078- <1>     db  078h, 0cch, 0cch, 078h, 0cch, 0cch, 078h, 000h, 078h, 0cch, 0cch, 07ch, 00ch, 018h, 070h, 000h
153 00014F09 CCCC7C0C187000      <1>
154 00014F10 003030000030300000- <1>     db  000h, 030h, 030h, 000h, 000h, 030h, 030h, 000h, 000h, 030h, 030h, 000h, 000h, 030h, 030h, 060h
154 00014F19 30300000303060      <1>
155 00014F20 183060C06030180000- <1>     db  018h, 030h, 060h, 0c0h, 060h, 030h, 018h, 000h, 000h, 000h, 0fch, 000h, 000h, 0fch, 000h, 000h
155 00014F29 00FC0000FC0000      <1>
156 00014F30 6030180C1830600078- <1>     db  060h, 030h, 018h, 00ch, 018h, 030h, 060h, 000h, 078h, 0cch, 00ch, 018h, 030h, 000h, 030h, 000h
156 00014F39 CC0C1830003000      <1>
157 00014F40 7CC6DEDEDEC0780030- <1>     db  07ch, 0c6h, 0deh, 0deh, 0deh, 0c0h, 078h, 000h, 030h, 078h, 0cch, 0cch, 0fch, 0cch, 0cch, 000h
157 00014F49 78CCCCFCCCCC00      <1>
158 00014F50 FC66667C6666FC003C- <1>     db  0fch, 066h, 066h, 07ch, 066h, 066h, 0fch, 000h, 03ch, 066h, 0c0h, 0c0h, 0c0h, 066h, 03ch, 000h
158 00014F59 66C0C0C0663C00      <1>
159 00014F60 F86C6666666CF800FE- <1>     db  0f8h, 06ch, 066h, 066h, 066h, 06ch, 0f8h, 000h, 0feh, 062h, 068h, 078h, 068h, 062h, 0feh, 000h
159 00014F69 6268786862FE00      <1>
160 00014F70 FE6268786860F0003C- <1>     db  0feh, 062h, 068h, 078h, 068h, 060h, 0f0h, 000h, 03ch, 066h, 0c0h, 0c0h, 0ceh, 066h, 03eh, 000h
160 00014F79 66C0C0CE663E00      <1>
161 00014F80 CCCCCCFCCCCCCC0078- <1>     db  0cch, 0cch, 0cch, 0fch, 0cch, 0cch, 0cch, 000h, 078h, 030h, 030h, 030h, 030h, 030h, 078h, 000h
161 00014F89 30303030307800      <1>
162 00014F90 1E0C0C0CCCCC7800E6- <1>     db  01eh, 00ch, 00ch, 00ch, 0cch, 0cch, 078h, 000h, 0e6h, 066h, 06ch, 078h, 06ch, 066h, 0e6h, 000h
162 00014F99 666C786C66E600      <1>
163 00014FA0 F06060606266FE00C6- <1>     db  0f0h, 060h, 060h, 060h, 062h, 066h, 0feh, 000h, 0c6h, 0eeh, 0feh, 0feh, 0d6h, 0c6h, 0c6h, 000h
163 00014FA9 EEFEFED6C6C600      <1>
164 00014FB0 C6E6F6DECEC6C60038- <1>     db  0c6h, 0e6h, 0f6h, 0deh, 0ceh, 0c6h, 0c6h, 000h, 038h, 06ch, 0c6h, 0c6h, 0c6h, 06ch, 038h, 000h
164 00014FB9 6CC6C6C66C3800      <1>
165 00014FC0 FC66667C6060F00078- <1>     db  0fch, 066h, 066h, 07ch, 060h, 060h, 0f0h, 000h, 078h, 0cch, 0cch, 0cch, 0dch, 078h, 01ch, 000h
165 00014FC9 CCCCCCDC781C00      <1>
166 00014FD0 FC66667C6C66E60078- <1>     db  0fch, 066h, 066h, 07ch, 06ch, 066h, 0e6h, 000h, 078h, 0cch, 0e0h, 070h, 01ch, 0cch, 078h, 000h
166 00014FD9 CCE0701CCC7800      <1>
167 00014FE0 FCB4303030307800CC- <1>     db  0fch, 0b4h, 030h, 030h, 030h, 030h, 078h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 0fch, 000h
167 00014FE9 CCCCCCCCCCFC00      <1>
168 00014FF0 CCCCCCCCCC783000C6- <1>     db  0cch, 0cch, 0cch, 0cch, 0cch, 078h, 030h, 000h, 0c6h, 0c6h, 0c6h, 0d6h, 0feh, 0eeh, 0c6h, 000h
168 00014FF9 C6C6D6FEEEC600      <1>
169 00015000 C6C66C38386CC600CC- <1>     db  0c6h, 0c6h, 06ch, 038h, 038h, 06ch, 0c6h, 000h, 0cch, 0cch, 0cch, 078h, 030h, 030h, 078h, 000h
169 00015009 CCCC7830307800      <1>
170 00015010 FEC68C183266FE0078- <1>     db  0feh, 0c6h, 08ch, 018h, 032h, 066h, 0feh, 000h, 078h, 060h, 060h, 060h, 060h, 060h, 078h, 000h
170 00015019 60606060607800      <1>
171 00015020 C06030180C06020078- <1>     db  0c0h, 060h, 030h, 018h, 00ch, 006h, 002h, 000h, 078h, 018h, 018h, 018h, 018h, 018h, 078h, 000h
171 00015029 18181818187800      <1>
172 00015030 10386CC60000000000- <1>     db  010h, 038h, 06ch, 0c6h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh
172 00015039 000000000000FF      <1>
173 00015040 303018000000000000- <1>     db  030h, 030h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 078h, 00ch, 07ch, 0cch, 076h, 000h
173 00015049 00780C7CCC7600      <1>
174 00015050 E060607C6666DC0000- <1>     db  0e0h, 060h, 060h, 07ch, 066h, 066h, 0dch, 000h, 000h, 000h, 078h, 0cch, 0c0h, 0cch, 078h, 000h
174 00015059 0078CCC0CC7800      <1>
175 00015060 1C0C0C7CCCCC760000- <1>     db  01ch, 00ch, 00ch, 07ch, 0cch, 0cch, 076h, 000h, 000h, 000h, 078h, 0cch, 0fch, 0c0h, 078h, 000h
175 00015069 0078CCFCC07800      <1>
176 00015070 386C60F06060F00000- <1>     db  038h, 06ch, 060h, 0f0h, 060h, 060h, 0f0h, 000h, 000h, 000h, 076h, 0cch, 0cch, 07ch, 00ch, 0f8h
176 00015079 0076CCCC7C0CF8      <1>
177 00015080 E0606C766666E60030- <1>     db  0e0h, 060h, 06ch, 076h, 066h, 066h, 0e6h, 000h, 030h, 000h, 070h, 030h, 030h, 030h, 078h, 000h
177 00015089 00703030307800      <1>
178 00015090 0C000C0C0CCCCC78E0- <1>     db  00ch, 000h, 00ch, 00ch, 00ch, 0cch, 0cch, 078h, 0e0h, 060h, 066h, 06ch, 078h, 06ch, 0e6h, 000h
178 00015099 60666C786CE600      <1>
179 000150A0 703030303030780000- <1>     db  070h, 030h, 030h, 030h, 030h, 030h, 078h, 000h, 000h, 000h, 0cch, 0feh, 0feh, 0d6h, 0c6h, 000h
179 000150A9 00CCFEFED6C600      <1>
180 000150B0 0000F8CCCCCCCC0000- <1>     db  000h, 000h, 0f8h, 0cch, 0cch, 0cch, 0cch, 000h, 000h, 000h, 078h, 0cch, 0cch, 0cch, 078h, 000h
180 000150B9 0078CCCCCC7800      <1>
181 000150C0 0000DC66667C60F000- <1>     db  000h, 000h, 0dch, 066h, 066h, 07ch, 060h, 0f0h, 000h, 000h, 076h, 0cch, 0cch, 07ch, 00ch, 01eh
181 000150C9 0076CCCC7C0C1E      <1>
182 000150D0 0000DC766660F00000- <1>     db  000h, 000h, 0dch, 076h, 066h, 060h, 0f0h, 000h, 000h, 000h, 07ch, 0c0h, 078h, 00ch, 0f8h, 000h
182 000150D9 007CC0780CF800      <1>
183 000150E0 10307C303034180000- <1>     db  010h, 030h, 07ch, 030h, 030h, 034h, 018h, 000h, 000h, 000h, 0cch, 0cch, 0cch, 0cch, 076h, 000h
183 000150E9 00CCCCCCCC7600      <1>
184 000150F0 0000CCCCCC78300000- <1>     db  000h, 000h, 0cch, 0cch, 0cch, 078h, 030h, 000h, 000h, 000h, 0c6h, 0d6h, 0feh, 0feh, 06ch, 000h
184 000150F9 00C6D6FEFE6C00      <1>
185 00015100 0000C66C386CC60000- <1>     db  000h, 000h, 0c6h, 06ch, 038h, 06ch, 0c6h, 000h, 000h, 000h, 0cch, 0cch, 0cch, 07ch, 00ch, 0f8h
185 00015109 00CCCCCC7C0CF8      <1>
186 00015110 0000FC983064FC001C- <1>     db  000h, 000h, 0fch, 098h, 030h, 064h, 0fch, 000h, 01ch, 030h, 030h, 0e0h, 030h, 030h, 01ch, 000h
186 00015119 3030E030301C00      <1>
187 00015120 1818180018181800E0- <1>     db  018h, 018h, 018h, 000h, 018h, 018h, 018h, 000h, 0e0h, 030h, 030h, 01ch, 030h, 030h, 0e0h, 000h
187 00015129 30301C3030E000      <1>
188 00015130 76DC00000000000000- <1>     db  076h, 0dch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 000h
188 00015139 10386CC6C6FE00      <1>
189 00015140 78CCC0CC78180C7800- <1>     db  078h, 0cch, 0c0h, 0cch, 078h, 018h, 00ch, 078h, 000h, 0cch, 000h, 0cch, 0cch, 0cch, 07eh, 000h
189 00015149 CC00CCCCCC7E00      <1>
190 00015150 1C0078CCFCC078007E- <1>     db  01ch, 000h, 078h, 0cch, 0fch, 0c0h, 078h, 000h, 07eh, 0c3h, 03ch, 006h, 03eh, 066h, 03fh, 000h
190 00015159 C33C063E663F00      <1>
191 00015160 CC00780C7CCC7E00E0- <1>     db  0cch, 000h, 078h, 00ch, 07ch, 0cch, 07eh, 000h, 0e0h, 000h, 078h, 00ch, 07ch, 0cch, 07eh, 000h
191 00015169 00780C7CCC7E00      <1>
192 00015170 3030780C7CCC7E0000- <1>     db  030h, 030h, 078h, 00ch, 07ch, 0cch, 07eh, 000h, 000h, 000h, 078h, 0c0h, 0c0h, 078h, 00ch, 038h
192 00015179 0078C0C0780C38      <1>
193 00015180 7EC33C667E603C00CC- <1>     db  07eh, 0c3h, 03ch, 066h, 07eh, 060h, 03ch, 000h, 0cch, 000h, 078h, 0cch, 0fch, 0c0h, 078h, 000h
193 00015189 0078CCFCC07800      <1>
194 00015190 E00078CCFCC07800CC- <1>     db  0e0h, 000h, 078h, 0cch, 0fch, 0c0h, 078h, 000h, 0cch, 000h, 070h, 030h, 030h, 030h, 078h, 000h
194 00015199 00703030307800      <1>
```

```
000151A0 7CC6381818183C00E0- <1>     db  07ch, 0c6h, 038h, 018h, 018h, 018h, 03ch, 000h, 0e0h, 000h, 070h, 030h, 030h, 030h, 078h, 000h
000151A9 00703030307800      <1>
000151B0 C6386CC6FEC6C60030- <1>     db  0c6h, 038h, 06ch, 0c6h, 0feh, 0c6h, 0c6h, 000h, 030h, 030h, 000h, 078h, 0cch, 0fch, 0cch, 000h
000151B9 300078CCFCCC00      <1>
000151C0 1C00FC607860FC0000- <1>     db  01ch, 000h, 0fch, 060h, 078h, 060h, 0fch, 000h, 000h, 000h, 07fh, 00ch, 07fh, 0cch, 07fh, 000h
000151C9 007F0C7FCC7F00      <1>
000151D0 3E6CCCFECCCCCE0078- <1>     db  03eh, 06ch, 0cch, 0feh, 0cch, 0cch, 0ceh, 000h, 078h, 0cch, 000h, 078h, 0cch, 0cch, 078h, 000h
000151D9 CC0078CCCC7800      <1>
000151E0 00CC0078CCCC780000- <1>     db  000h, 0cch, 000h, 078h, 0cch, 0cch, 078h, 000h, 000h, 0e0h, 000h, 078h, 0cch, 0cch, 078h, 000h
000151E9 E00078CCCC7800      <1>
000151F0 78CC00CCCCCC7E0000- <1>     db  078h, 0cch, 000h, 0cch, 0cch, 0cch, 07eh, 000h, 000h, 0e0h, 000h, 0cch, 0cch, 0cch, 07eh, 000h
000151F9 E000CCCCCC7E00      <1>
00015200 00CC00CCCC7C0CF8C3- <1>     db  000h, 0cch, 000h, 0cch, 0cch, 07ch, 00ch, 0f8h, 0c3h, 018h, 03ch, 066h, 066h, 03ch, 018h, 000h
00015209 183C66663C1800      <1>
00015210 CC00CCCCCCCC780018- <1>     db  0cch, 000h, 0cch, 0cch, 0cch, 0cch, 078h, 000h, 018h, 018h, 07eh, 0c0h, 0c0h, 07eh, 018h, 018h
00015219 187EC0C07E1818      <1>
00015220 386C64F060E6FC00CC- <1>     db  038h, 06ch, 064h, 0f0h, 060h, 0e6h, 0fch, 000h, 0cch, 0cch, 078h, 0fch, 030h, 0fch, 030h, 030h
00015229 CC78FC30FC3030      <1>
00015230 F8CCCCFAC6CFC6C70E- <1>     db  0f8h, 0cch, 0cch, 0fah, 0c6h, 0cfh, 0c6h, 0c7h, 00eh, 01bh, 018h, 03ch, 018h, 018h, 0d8h, 070h
00015239 1B183C1818D870      <1>
00015240 1C00780C7CCC7E0038- <1>     db  01ch, 000h, 078h, 00ch, 07ch, 0cch, 07eh, 000h, 038h, 000h, 070h, 030h, 030h, 030h, 078h, 000h
00015249 00703030307800      <1>
00015250 001C0078CCCC780000- <1>     db  000h, 01ch, 000h, 078h, 0cch, 0cch, 078h, 000h, 000h, 01ch, 000h, 0cch, 0cch, 0cch, 07eh, 000h
00015259 1C00CCCCCC7E00      <1>
00015260 00F800F8CCCCCC00FC- <1>     db  000h, 0f8h, 000h, 0f8h, 0cch, 0cch, 0cch, 000h, 0fch, 000h, 0cch, 0ech, 0fch, 0dch, 0cch, 000h
00015269 00CCECFCDCCC00      <1>
00015270 3C6C6C3E007E000038- <1>     db  03ch, 06ch, 06ch, 03eh, 000h, 07eh, 000h, 000h, 038h, 06ch, 06ch, 038h, 000h, 07ch, 000h, 000h
00015279 6C6C38007C0000      <1>
00015280 30003060C0CC780000- <1>     db  030h, 000h, 030h, 060h, 0c0h, 0cch, 078h, 000h, 000h, 000h, 000h, 0fch, 0c0h, 0c0h, 000h, 000h
00015289 0000FCC0C00000      <1>
00015290 000000FC0C0C0000C3- <1>     db  000h, 000h, 000h, 0fch, 00ch, 00ch, 000h, 000h, 0c3h, 0c6h, 0cch, 0deh, 033h, 066h, 0cch, 00fh
00015299 C6CCDE3366CC0F      <1>
000152A0 C3C6CCDB376FCF0318- <1>     db  0c3h, 0c6h, 0cch, 0dbh, 037h, 06fh, 0cfh, 003h, 018h, 018h, 000h, 018h, 018h, 018h, 018h, 000h
000152A9 18001818181800      <1>
000152B0 003366CC6633000000- <1>     db  000h, 033h, 066h, 0cch, 066h, 033h, 000h, 000h, 000h, 0cch, 066h, 033h, 066h, 0cch, 000h, 000h
000152B9 CC663366CC0000      <1>
000152C0 228822882288228855- <1>     db  022h, 088h, 022h, 088h, 022h, 088h, 022h, 088h, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah
000152C9 AA55AA55AA55AA      <1>
000152D0 DB77DBEEDB77DBEE18- <1>     db  0dbh, 077h, 0dbh, 0eeh, 0dbh, 077h, 0dbh, 0eeh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
000152D9 18181818181818      <1>
000152E0 18181818F818181818- <1>     db  018h, 018h, 018h, 018h, 0f8h, 018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 0f8h, 018h, 018h, 018h
000152E9 18F818F8181818      <1>
000152F0 36363636F636363600- <1>     db  036h, 036h, 036h, 036h, 0f6h, 036h, 036h, 036h, 000h, 000h, 000h, 000h, 0feh, 036h, 036h, 036h
000152F9 000000FE363636      <1>
00015300 0000F818F818181836- <1>     db  000h, 000h, 0f8h, 018h, 0f8h, 018h, 018h, 018h, 036h, 036h, 0f6h, 006h, 0f6h, 036h, 036h, 036h
00015309 36F606F6363636      <1>
00015310 363636363636363600- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 000h, 000h, 0feh, 006h, 0f6h, 036h, 036h, 036h
00015319 00FE06F6363636      <1>
00015320 3636F606FE00000036- <1>     db  036h, 036h, 0f6h, 006h, 0feh, 000h, 000h, 000h, 036h, 036h, 036h, 036h, 0feh, 000h, 000h, 000h
00015329 363636FE000000      <1>
00015330 1818F818F800000000- <1>     db  018h, 018h, 0f8h, 018h, 0f8h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0f8h, 018h, 018h, 018h
00015339 000000F8181818      <1>
00015340 181818181F00000018- <1>     db  018h, 018h, 018h, 018h, 01fh, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 0ffh, 000h, 000h, 000h
00015349 181818FF000000      <1>
00015350 00000000FF18181818- <1>     db  000h, 000h, 000h, 000h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 01fh, 018h, 018h, 018h
00015359 1818181F181818      <1>
00015360 00000000FF00000018- <1>     db  000h, 000h, 000h, 000h, 0ffh, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 0ffh, 018h, 018h, 018h
00015369 181818FF181818      <1>
00015370 18181F181F18181836- <1>     db  018h, 018h, 01fh, 018h, 01fh, 018h, 018h, 018h, 036h, 036h, 036h, 036h, 037h, 036h, 036h, 036h
00015379 36363637363636      <1>
00015380 363637303F00000000- <1>     db  036h, 036h, 037h, 030h, 03fh, 000h, 000h, 000h, 000h, 000h, 03fh, 030h, 037h, 036h, 036h, 036h
00015389 003F3037363636      <1>
00015390 3636F700FF00000000- <1>     db  036h, 036h, 0f7h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0f7h, 036h, 036h, 036h
00015399 00FF00F7363636      <1>
000153A0 363637303736363600- <1>     db  036h, 036h, 037h, 030h, 037h, 036h, 036h, 036h, 000h, 000h, 0ffh, 000h, 0ffh, 000h, 000h, 000h
000153A9 00FF00FF000000      <1>
000153B0 3636F700F736363618- <1>     db  036h, 036h, 0f7h, 000h, 0f7h, 036h, 036h, 036h, 018h, 018h, 0ffh, 000h, 0ffh, 000h, 000h, 000h
000153B9 18FF00FF000000      <1>
000153C0 36363636FF00000000- <1>     db  036h, 036h, 036h, 036h, 0ffh, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0ffh, 018h, 018h, 018h
000153C9 00FF00FF181818      <1>
000153D0 00000000FF36363636- <1>     db  000h, 000h, 000h, 000h, 0ffh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 03fh, 000h, 000h, 000h
000153D9 3636363F000000      <1>
000153E0 18181F181F00000000- <1>     db  018h, 018h, 01fh, 018h, 01fh, 000h, 000h, 000h, 000h, 000h, 01fh, 018h, 01fh, 018h, 018h, 018h
000153E9 001F181F181818      <1>
000153F0 000000003F36363636- <1>     db  000h, 000h, 000h, 000h, 03fh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 0ffh, 036h, 036h, 036h
000153F9 363636FF363636      <1>
00015400 1818FF18FF18181818- <1>     db  018h, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0f8h, 000h, 000h, 000h
00015409 181818F8000000      <1>
00015410 000000001F181818FF- <1>     db  000h, 000h, 000h, 000h, 01fh, 018h, 018h, 018h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
00015419 FFFFFFFFFFFFFF      <1>
00015420 00000000FFFFFFFFF0- <1>     db  000h, 000h, 000h, 000h, 0ffh, 0ffh, 0ffh, 0ffh, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h
00015429 F0F0F0F0F0F0F0      <1>
00015430 0F0F0F0F0F0F0FFFF- <1>     db  00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h, 000h, 000h
00015439 FFFFFF00000000      <1>
00015440 000076DCC8DC760000- <1>     db  000h, 000h, 076h, 0dch, 0c8h, 0dch, 076h, 000h, 000h, 078h, 0cch, 0f8h, 0cch, 0f8h, 0c0h, 0c0h
00015449 78CCF8CCF8C0C0      <1>
00015450 00FCCCC0C0C0C00000- <1>     db  000h, 0fch, 0cch, 0c0h, 0c0h, 0c0h, 0c0h, 000h, 000h, 0feh, 06ch, 06ch, 06ch, 06ch, 06ch, 000h
00015459 FE6C6C6C6C6C00      <1>
00015460 FCCC603060CCFC0000- <1>     db  0fch, 0cch, 060h, 030h, 060h, 0cch, 0fch, 000h, 000h, 000h, 07eh, 0d8h, 0d8h, 0d8h, 070h, 000h
00015469 007ED8D8D87000      <1>
00015470 00666666667C60C000- <1>     db  000h, 066h, 066h, 066h, 066h, 07ch, 060h, 0c0h, 000h, 076h, 0dch, 018h, 018h, 018h, 018h, 000h
00015479 76DC1818181800      <1>
00015480 FC3078CCCC7830FC38- <1>     db  0fch, 030h, 078h, 0cch, 0cch, 078h, 030h, 0fch, 038h, 06ch, 0c6h, 0feh, 0c6h, 06ch, 038h, 000h
00015489 6CC6FEC66C3800      <1>
00015490 386CC6C66C6CEE001C- <1>     db  038h, 06ch, 0c6h, 0c6h, 06ch, 06ch, 0eeh, 000h, 01ch, 030h, 018h, 07ch, 0cch, 0cch, 078h, 000h
00015499 30187CCCCC7800      <1>
000154A0 00007EDBDB7E000006- <1>     db  000h, 000h, 07eh, 0dbh, 0dbh, 07eh, 000h, 000h, 006h, 00ch, 07eh, 0dbh, 0dbh, 07eh, 060h, 0c0h
000154A9 0C7EDBDB7E60C0      <1>
000154B0 3860C0F8C060380078- <1>     db  038h, 060h, 0c0h, 0f8h, 0c0h, 060h, 038h, 000h, 078h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 000h
000154B9 CCCCCCCCCCCC00      <1>
000154C0 00FC00FC00FC000030- <1>     db  000h, 0fch, 000h, 0fch, 000h, 0fch, 000h, 000h, 030h, 030h, 0fch, 030h, 030h, 000h, 0fch, 000h
000154C9 30FC303000FC00      <1>
000154D0 603018306000FC0018- <1>     db  060h, 030h, 018h, 030h, 060h, 000h, 0fch, 000h, 018h, 030h, 060h, 030h, 018h, 000h, 0fch, 000h
000154D9 3060301800FC00      <1>
000154E0 0E1B1B181818181818- <1>     db  00eh, 01bh, 01bh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0d8h, 0d8h, 070h
000154E9 18181818D8D870      <1>
000154F0 303000FC0030300000- <1>     db  030h, 030h, 000h, 0fch, 000h, 030h, 030h, 000h, 000h, 076h, 0dch, 000h, 076h, 0dch, 000h, 000h
000154F9 76DC0076DC0000      <1>
00015500 386C6C380000000000- <1>     db  038h, 06ch, 06ch, 038h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h
00015509 00001818000000      <1>
00015510 00000000180000000F- <1>     db  000h, 000h, 000h, 000h, 018h, 000h, 000h, 000h, 00fh, 00ch, 00ch, 00ch, 0ech, 06ch, 03ch, 01ch
00015519 0C0C0CEC6C3C1C      <1>
00015520 786C6C6C6C00000070- <1>     db  078h, 06ch, 06ch, 06ch, 06ch, 000h, 000h, 000h, 070h, 018h, 030h, 060h, 078h, 000h, 000h, 000h
00015529 18306078000000      <1>
00015530 00003C3C3C3C000000- <1>     db  000h, 000h, 03ch, 03ch, 03ch, 03ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00015539 00000000000000      <1>
                             <1> vgafont14:
00015540 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00015549 00000000000000      <1>
00015550 7E81A58181BD99817E- <1>     db  07eh, 081h, 0a5h, 081h, 081h, 0bdh, 099h, 081h, 07eh, 000h, 000h, 000h, 000h, 000h, 07eh, 0ffh
00015559 00000000007EFF      <1>
00015560 DBFFFFC3E7FF7E0000- <1>     db  0dbh, 0ffh, 0ffh, 0c3h, 0e7h, 0ffh, 07eh, 000h, 000h, 000h, 000h, 000h, 000h, 06ch, 0feh, 0feh
00015569 000000006CFEFE      <1>
00015570 FEFE7C381000000000- <1>     db  0feh, 0feh, 07ch, 038h, 010h, 000h, 000h, 000h, 000h, 000h, 000h, 010h, 038h, 07ch, 0feh, 07ch
```

```
257 00015579 000010387CFE7C      <1>
258 00015580 381000000000000018- <1>     db  038h, 010h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 03ch, 03ch, 0e7h, 0e7h, 0e7h, 018h, 018h
258 00015589 3C3CE7E7E71818      <1>
259 00015590 3C0000000000183C7E- <1>     db  03ch, 000h, 000h, 000h, 000h, 000h, 018h, 03ch, 07eh, 0ffh, 0ffh, 07eh, 018h, 018h, 03ch, 000h
259 00015599 FFFF7E18183C00      <1>
260 000155A0 00000000000000183C- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 03ch, 03ch, 018h, 000h, 000h, 000h, 000h, 000h
260 000155A9 3C180000000000      <1>
261 000155B0 FFFFFFFFFFE7C3C3E7- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0e7h, 0c3h, 0c3h, 0e7h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h
261 000155B9 FFFFFFFFFF0000      <1>
262 000155C0 00003C664242663C00- <1>     db  000h, 000h, 03ch, 066h, 042h, 042h, 066h, 03ch, 000h, 000h, 000h, 000h, 0ffh, 0ffh, 0ffh, 0ffh
262 000155C9 000000FFFFFFFF      <1>
263 000155D0 C399BDBD99C3FFFFFF- <1>     db  0c3h, 099h, 0bdh, 0bdh, 099h, 0c3h, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h, 01eh, 00eh, 01ah, 032h
263 000155D9 FF00001E0E1A32      <1>
264 000155E0 78CCCCCC7800000000- <1>     db  078h, 0cch, 0cch, 0cch, 078h, 000h, 000h, 000h, 000h, 000h, 03ch, 066h, 066h, 066h, 03ch, 018h
264 000155E9 003C6666663C18      <1>
265 000155F0 7E181800000000003F- <1>     db  07eh, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 03fh, 033h, 03fh, 030h, 030h, 030h, 070h, 0f0h
265 000155F9 333F30303070F0      <1>
266 00015600 E000000000007F637F- <1>     db  0e0h, 000h, 000h, 000h, 000h, 000h, 07fh, 063h, 07fh, 063h, 063h, 063h, 067h, 0e7h, 0e6h, 0c0h
266 00015609 63636367E7E6C0      <1>
267 00015610 000000001818DB3CE7- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 0dbh, 03ch, 0e7h, 03ch, 0dbh, 018h, 018h, 000h, 000h, 000h
267 00015619 3CDB1818000000      <1>
268 00015620 000080C0E0F8FEF8E0- <1>     db  000h, 000h, 080h, 0c0h, 0e0h, 0f8h, 0feh, 0f8h, 0e0h, 0c0h, 080h, 000h, 000h, 000h, 000h, 000h
268 00015629 C080000000000       <1>
269 00015630 02060E3EFE3E0E0602- <1>     db  002h, 006h, 00eh, 03eh, 0feh, 03eh, 00eh, 006h, 002h, 000h, 000h, 000h, 000h, 000h, 018h, 03ch
269 00015639 0000000000183C      <1>
270 00015640 7E1818187E3C180000- <1>     db  07eh, 018h, 018h, 018h, 07eh, 03ch, 018h, 000h, 000h, 000h, 000h, 000h, 066h, 066h, 066h, 066h
270 00015649 00000066666666      <1>
271 00015650 666600666600000000- <1>     db  066h, 066h, 000h, 066h, 066h, 000h, 000h, 000h, 000h, 000h, 07fh, 0dbh, 0dbh, 0dbh, 07bh, 01bh
271 00015659 007FDBDBDB7B1B      <1>
272 00015660 1B1B1B000000007CC6- <1>     db  01bh, 01bh, 01bh, 000h, 000h, 000h, 000h, 07ch, 0c6h, 060h, 038h, 06ch, 0c6h, 0c6h, 06ch, 038h
272 00015669 60386CC6C66C38      <1>
273 00015670 0CC67C000000000000- <1>     db  00ch, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 0feh, 0feh, 000h
273 00015679 000000FEFEFE00      <1>
274 00015680 00000000183C7E1818- <1>     db  000h, 000h, 000h, 000h, 018h, 03ch, 07eh, 018h, 018h, 018h, 07eh, 03ch, 018h, 07eh, 000h, 000h
274 00015689 187E3C187E0000      <1>
275 00015690 0000183C7E18181818- <1>     db  000h, 000h, 018h, 03ch, 07eh, 018h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h, 000h
275 00015699 18180000000000      <1>
276 000156A0 1818181818187E3C18- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 07eh, 03ch, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
276 000156A9 00000000000000      <1>
277 000156B0 180CFE0C1800000000- <1>     db  018h, 00ch, 0feh, 00ch, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 030h, 060h
277 000156B9 00000000003060      <1>
278 000156C0 FE6030000000000000- <1>     db  0feh, 060h, 030h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0c0h, 0c0h, 0c0h
278 000156C9 00000000C0C0C0      <1>
279 000156D0 FE0000000000000000- <1>     db  0feh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 028h, 06ch, 0feh, 06ch, 028h, 000h
279 000156D9 00286CFE6C2800      <1>
280 000156E0 000000000000001038- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 010h, 038h, 038h, 07ch, 07ch, 0feh, 0feh, 000h, 000h
280 000156E9 387C7CFEFE0000      <1>
281 000156F0 0000000000FEFE7C7C- <1>     db  000h, 000h, 000h, 000h, 000h, 0feh, 0feh, 07ch, 07ch, 038h, 038h, 010h, 000h, 000h, 000h, 000h
281 000156F9 38381000000000      <1>
282 00015700 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
282 00015709 00000000000000      <1>
283 00015710 183C3C1818001818- <1>     db  018h, 03ch, 03ch, 03ch, 018h, 018h, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 066h, 066h, 066h
283 00015719 00000000666666      <1>
284 00015720 240000000000000000- <1>     db  024h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 06ch, 06ch, 0feh, 06ch
284 00015729 0000006C6CFE6C      <1>
285 00015730 6C6CFE6C6C00000018- <1>     db  06ch, 06ch, 0feh, 06ch, 06ch, 000h, 000h, 000h, 018h, 018h, 07ch, 0c6h, 0c2h, 0c0h, 07ch, 006h
285 00015739 187CC6C2C07C06      <1>
286 00015740 86C67C181800000000- <1>     db  086h, 0c6h, 07ch, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 0c2h, 0c6h, 00ch, 018h, 030h, 066h
286 00015749 00C2C60C183066      <1>
287 00015750 C60000000000386C6C- <1>     db  0c6h, 000h, 000h, 000h, 000h, 000h, 038h, 06ch, 06ch, 038h, 076h, 0dch, 0cch, 0cch, 076h, 000h
287 00015759 3876DCCCCC7600      <1>
288 00015760 000000303030600000- <1>     db  000h, 000h, 000h, 030h, 030h, 030h, 060h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
288 00015769 00000000000000      <1>
289 00015770 00000C183030303030- <1>     db  000h, 000h, 00ch, 018h, 030h, 030h, 030h, 030h, 030h, 018h, 00ch, 000h, 000h, 000h, 000h, 000h
289 00015779 180C0000000000      <1>
290 00015780 30180C0C0C0C0C1830- <1>     db  030h, 018h, 00ch, 00ch, 00ch, 00ch, 00ch, 018h, 030h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
290 00015789 00000000000000      <1>
291 00015790 663CFF3C6600000000- <1>     db  066h, 03ch, 0ffh, 03ch, 066h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h
291 00015799 00000000001818      <1>
292 000157A0 7E1818000000000000- <1>     db  07eh, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
292 000157A9 00000000000000      <1>
293 000157B0 181818300000000000- <1>     db  018h, 018h, 018h, 030h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 000h, 000h, 000h
293 000157B9 000000FE000000      <1>
294 000157C0 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h
294 000157C9 00000000181800      <1>
295 000157D0 0000000002060C1830- <1>     db  000h, 000h, 000h, 000h, 002h, 006h, 00ch, 018h, 030h, 060h, 0c0h, 080h, 000h, 000h, 000h, 000h
295 000157D9 60C08000000000      <1>
296 000157E0 00007CC6CEDEF6E6C6- <1>     db  000h, 000h, 07ch, 0c6h, 0ceh, 0deh, 0f6h, 0e6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h
296 000157E9 C67C0000000000      <1>
297 000157F0 18387818181818187E- <1>     db  018h, 038h, 078h, 018h, 018h, 018h, 018h, 018h, 07eh, 000h, 000h, 000h, 000h, 000h, 07ch, 0c6h
297 000157F9 00000000007CC6      <1>
298 00015800 060C183060C6FE0000- <1>     db  006h, 00ch, 018h, 030h, 060h, 0c6h, 0feh, 000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 006h, 006h
298 00015809 0000007CC60606      <1>
299 00015810 3C0606C67C00000000- <1>     db  03ch, 006h, 006h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 00ch, 01ch, 03ch, 06ch, 0cch, 0feh
299 00015819 000C1C3C6CCCFE      <1>
300 00015820 0C0C1E0000000000FE- <1>     db  00ch, 00ch, 01eh, 000h, 000h, 000h, 000h, 000h, 0feh, 0c0h, 0c0h, 0c0h, 0fch, 006h, 006h, 0c6h
300 00015829 C0C0C0FC0606C6      <1>
301 00015830 7C00000000003860C0- <1>     db  07ch, 000h, 000h, 000h, 000h, 000h, 038h, 060h, 0c0h, 0c0h, 0fch, 0c6h, 0c6h, 0c6h, 07ch, 000h
301 00015839 C0FCC6C6C67C00      <1>
302 00015840 00000000FEC6060C18- <1>     db  000h, 000h, 000h, 000h, 0feh, 0c6h, 006h, 00ch, 018h, 030h, 030h, 030h, 030h, 000h, 000h, 000h
302 00015849 30303030000000      <1>
303 00015850 00007CC6C6C67CC6C6- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 07ch, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h
303 00015859 C67C0000000000      <1>
304 00015860 7CC6C6C67E06060C78- <1>     db  07ch, 0c6h, 0c6h, 0c6h, 07eh, 006h, 006h, 00ch, 078h, 000h, 000h, 000h, 000h, 000h, 000h, 018h
304 00015869 00000000000018      <1>
305 00015870 180000001818000000- <1>     db  018h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h
305 00015879 00000000181800      <1>
306 00015880 000018183000000000- <1>     db  000h, 000h, 018h, 018h, 030h, 000h, 000h, 000h, 000h, 000h, 006h, 00ch, 018h, 030h, 060h, 030h
306 00015889 00060C18306030      <1>
307 00015890 180C06000000000000- <1>     db  018h, 00ch, 006h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07eh, 000h, 000h, 07eh, 000h
307 00015899 00007E00007E00      <1>
308 000158A0 000000000000603018- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 060h, 030h, 018h, 00ch, 006h, 00ch, 018h, 030h, 060h, 000h
308 000158A9 0C060C18306000      <1>
309 000158B0 000000007CC6C60C18- <1>     db  000h, 000h, 000h, 000h, 07ch, 0c6h, 0c6h, 00ch, 018h, 018h, 000h, 018h, 018h, 000h, 000h, 000h
309 000158B9 18001818000000      <1>
310 000158C0 00007CC6C6DEDEDEDC- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0deh, 0deh, 0deh, 0dch, 0c0h, 07ch, 000h, 000h, 000h, 000h, 000h
310 000158C9 C07C0000000000      <1>
311 000158D0 10386CC6C6FEC6C6C6- <1>     db  010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h, 000h, 0fch, 066h
311 000158D9 0000000000FC66      <1>
312 000158E0 66667C666666FC0000- <1>     db  066h, 066h, 07ch, 066h, 066h, 066h, 0fch, 000h, 000h, 000h, 000h, 000h, 03ch, 066h, 0c2h, 0c0h
312 000158E9 0000003C66C2C0      <1>
313 000158F0 C0C0C2663C00000000- <1>     db  0c0h, 0c0h, 0c2h, 066h, 03ch, 000h, 000h, 000h, 000h, 000h, 0f8h, 06ch, 066h, 066h, 066h, 066h
313 000158F9 00F86C66666666      <1>
314 00015900 666CF80000000000FE- <1>     db  066h, 06ch, 0f8h, 000h, 000h, 000h, 000h, 000h, 0feh, 066h, 062h, 068h, 078h, 068h, 062h, 066h
314 00015909 66626878686266      <1>
315 00015910 FE0000000000FE6662- <1>     db  0feh, 000h, 000h, 000h, 000h, 000h, 0feh, 066h, 062h, 068h, 078h, 068h, 060h, 060h, 0f0h, 000h
315 00015919 6878686060F000      <1>
316 00015920 000000003C66C2C0C0- <1>     db  000h, 000h, 000h, 000h, 03ch, 066h, 0c2h, 0c0h, 0c0h, 0deh, 0c6h, 066h, 03ah, 000h, 000h, 000h
316 00015929 DEC6663A000000      <1>
317 00015930 0000C6C6C6C6FEC6C6- <1>     db  000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h, 000h
317 00015939 C6C60000000000      <1>
318 00015940 3C181818181818183C- <1>     db  03ch, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 000h, 01eh, 00ch
318 00015949 00000000001E0C      <1>
319 00015950 0C0C0C0CCCCC780000- <1>     db  00ch, 00ch, 00ch, 00ch, 0cch, 0cch, 078h, 000h, 000h, 000h, 000h, 000h, 0e6h, 066h, 06ch, 06ch
```

```
319 00015959 000000E6666C6C      <1>
320 00015960 786C6C66E600000000- <1>     db  078h, 06ch, 06ch, 066h, 0e6h, 000h, 000h, 000h, 000h, 000h, 0f0h, 060h, 060h, 060h, 060h, 060h
320 00015969 00F06060606060      <1>
321 00015970 6266FE0000000000C6- <1>     db  062h, 066h, 0feh, 000h, 000h, 000h, 000h, 000h, 0c6h, 0eeh, 0feh, 0feh, 0d6h, 0c6h, 0c6h, 0c6h
321 00015979 EEFEFED6C6C6C6      <1>
322 00015980 C60000000000C6E6F6- <1>     db  0c6h, 000h, 000h, 000h, 000h, 000h, 0c6h, 0e6h, 0f6h, 0feh, 0deh, 0ceh, 0c6h, 0c6h, 0c6h, 000h
322 00015989 FEDECEC6C6C600      <1>
323 00015990 00000000386CC6C6C6- <1>     db  000h, 000h, 000h, 000h, 038h, 06ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 06ch, 038h, 000h, 000h, 000h
323 00015999 C6C66C38000000      <1>
324 000159A0 0000FC6666667C6060- <1>     db  000h, 000h, 0fch, 066h, 066h, 066h, 07ch, 060h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h, 000h
324 000159A9 60F00000000000      <1>
325 000159B0 7CC6C6C6C6D6DE7C0C- <1>     db  07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0d6h, 0deh, 07ch, 00ch, 00eh, 000h, 000h, 000h, 000h, 0fch, 066h
325 000159B9 0E00000000FC66      <1>
326 000159C0 66667C6C6666E60000- <1>     db  066h, 066h, 07ch, 06ch, 066h, 066h, 0e6h, 000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0c6h, 060h
326 000159C9 0000007CC6C660      <1>
327 000159D0 380CC6C67C00000000- <1>     db  038h, 00ch, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 07eh, 07eh, 05ah, 018h, 018h, 018h
327 000159D9 007E7E5A181818      <1>
328 000159E0 18183C0000000000C6- <1>     db  018h, 018h, 03ch, 000h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h
328 000159E9 C6C6C6C6C6C6C6      <1>
329 000159F0 7C0000000000C6C6C6- <1>     db  07ch, 000h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 06ch, 038h, 010h, 000h
329 000159F9 C6C6C66C381000      <1>
330 00015A00 00000000C6C6C6C6D6- <1>     db  000h, 000h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0d6h, 0d6h, 0feh, 07ch, 06ch, 000h, 000h, 000h
330 00015A09 D6FE7C6C000000      <1>
331 00015A10 0000C6C66C3838386C- <1>     db  000h, 000h, 0c6h, 0c6h, 06ch, 038h, 038h, 038h, 06ch, 0c6h, 0c6h, 000h, 000h, 000h, 000h, 000h
331 00015A19 C6C60000000000      <1>
332 00015A20 666666663C1818183C- <1>     db  066h, 066h, 066h, 066h, 03ch, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 000h, 0feh, 0c6h
332 00015A29 0000000000FEC6      <1>
333 00015A30 8C183060C2C6FE0000- <1>     db  08ch, 018h, 030h, 060h, 0c2h, 0c6h, 0feh, 000h, 000h, 000h, 000h, 000h, 03ch, 030h, 030h, 030h
333 00015A39 0000003C303030      <1>
334 00015A40 303030303C00000000- <1>     db  030h, 030h, 030h, 030h, 03ch, 000h, 000h, 000h, 000h, 000h, 080h, 0c0h, 0e0h, 070h, 038h, 01ch
334 00015A49 0080C0E070381C      <1>
335 00015A50 0E060200000000003C- <1>     db  00eh, 006h, 002h, 000h, 000h, 000h, 000h, 000h, 03ch, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch
335 00015A59 0C0C0C0C0C0C0C      <1>
336 00015A60 3C00000010386CC600- <1>     db  03ch, 000h, 000h, 000h, 010h, 038h, 06ch, 0c6h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
336 00015A69 00000000000000      <1>
337 00015A70 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h
337 00015A79 0000000000FF00      <1>
338 00015A80 303018000000000000- <1>     db  030h, 030h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
338 00015A89 00000000000000      <1>
339 00015A90 000000780C7CCCCC76- <1>     db  000h, 000h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 000h, 0e0h, 060h
339 00015A99 0000000000E060      <1>
340 00015AA0 60786C6666667C0000- <1>     db  060h, 078h, 06ch, 066h, 066h, 066h, 07ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07ch
340 00015AA9 0000000000007C      <1>
341 00015AB0 C6C0C0C67C00000000- <1>     db  0c6h, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 01ch, 00ch, 00ch, 03ch, 06ch, 0cch
341 00015AB9 001C0C0C3C6CCC      <1>
342 00015AC0 CCCC76000000000000- <1>     db  0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c6h
342 00015AC9 00007CC6FEC0C6      <1>
343 00015AD0 7C0000000000386C64- <1>     db  07ch, 000h, 000h, 000h, 000h, 000h, 038h, 06ch, 064h, 060h, 0f0h, 060h, 060h, 060h, 0f0h, 000h
343 00015AD9 60F0606060F000      <1>
344 00015AE0 0000000000000076CC- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 076h, 0cch, 0cch, 0cch, 07ch, 00ch, 0cch, 078h, 000h
344 00015AE9 CCCC7C0CCC7800      <1>
345 00015AF0 0000E060606C766666- <1>     db  000h, 000h, 0e0h, 060h, 060h, 06ch, 076h, 066h, 066h, 066h, 0e6h, 000h, 000h, 000h, 000h, 000h
345 00015AF9 66E60000000000      <1>
346 00015B00 18180038181818183C- <1>     db  018h, 018h, 000h, 038h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 000h, 006h, 006h
346 00015B09 00000000000606      <1>
347 00015B10 000E0606060666663C- <1>     db  000h, 00eh, 006h, 006h, 006h, 006h, 066h, 066h, 03ch, 000h, 000h, 000h, 0e0h, 060h, 060h, 066h
347 00015B19 000000E0606066      <1>
348 00015B20 6C786C66E600000000- <1>     db  06ch, 078h, 06ch, 066h, 0e6h, 000h, 000h, 000h, 000h, 000h, 038h, 018h, 018h, 018h, 018h, 018h
348 00015B29 00381818181818      <1>
349 00015B30 18183C000000000000- <1>     db  018h, 018h, 03ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ech, 0feh, 0d6h, 0d6h, 0d6h
349 00015B39 0000ECFED6D6D6      <1>
350 00015B40 C60000000000000000- <1>     db  0c6h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0dch, 066h, 066h, 066h, 066h, 066h, 000h
350 00015B49 DC666666666600      <1>
351 00015B50 000000000000007CC6- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h
351 00015B59 C6C6C67C000000      <1>
352 00015B60 0000000000DC666666- <1>     db  000h, 000h, 000h, 000h, 000h, 0dch, 066h, 066h, 066h, 07ch, 060h, 060h, 0f0h, 000h, 000h, 000h
352 00015B69 7C6060F0000000      <1>
353 00015B70 00000076CCCCCC7C0C- <1>     db  000h, 000h, 000h, 076h, 0cch, 0cch, 0cch, 07ch, 00ch, 00ch, 01eh, 000h, 000h, 000h, 000h, 000h
353 00015B79 0C1E0000000000      <1>
354 00015B80 00DC76666060F00000- <1>     db  000h, 0dch, 076h, 066h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07ch
354 00015B89 0000000000007C      <1>
355 00015B90 C6701CC67C00000000- <1>     db  0c6h, 070h, 01ch, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 010h, 030h, 030h, 0fch, 030h, 030h
355 00015B99 00103030FC3030      <1>
356 00015BA0 30361C000000000000- <1>     db  030h, 036h, 01ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch
356 00015BA9 0000CCCCCCCCCC      <1>
357 00015BB0 760000000000000000- <1>     db  076h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 066h, 066h, 066h, 066h, 03ch, 018h, 000h
357 00015BB9 666666663C1800      <1>
358 00015BC0 00000000000000C6C6- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 0d6h, 0d6h, 0feh, 06ch, 000h, 000h, 000h
358 00015BC9 D6D6FE6C000000      <1>
359 00015BD0 0000000000C66C3838- <1>     db  000h, 000h, 000h, 000h, 000h, 0c6h, 06ch, 038h, 038h, 06ch, 0c6h, 000h, 000h, 000h, 000h, 000h
359 00015BD9 6CC60000000000      <1>
360 00015BE0 000000C6C6C6C67E06- <1>     db  000h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 07eh, 006h, 00ch, 0f8h, 000h, 000h, 000h, 000h, 000h
360 00015BE9 0CF80000000000      <1>
361 00015BF0 00FECC183066FE0000- <1>     db  000h, 0feh, 0cch, 018h, 030h, 066h, 0feh, 000h, 000h, 000h, 000h, 000h, 00eh, 018h, 018h, 018h
361 00015BF9 0000000E181818      <1>
362 00015C00 701818180E00000000- <1>     db  070h, 018h, 018h, 018h, 00eh, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 000h, 018h
362 00015C09 00181818180018      <1>
363 00015C10 181818000000000070- <1>     db  018h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 070h, 018h, 018h, 018h, 00eh, 018h, 018h, 018h
363 00015C19 1818180E181818      <1>
364 00015C20 70000000000076DC00- <1>     db  070h, 000h, 000h, 000h, 000h, 000h, 076h, 0dch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
364 00015C29 00000000000000      <1>
365 00015C30 00000000000010386C- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 000h, 000h, 000h, 000h
365 00015C39 C6C6FE00000000      <1>
366 00015C40 00003C66C2C0C0C266- <1>     db  000h, 000h, 03ch, 066h, 0c2h, 0c0h, 0c0h, 0c2h, 066h, 03ch, 00ch, 006h, 07ch, 000h, 000h, 000h
366 00015C49 3C0C067C000000      <1>
367 00015C50 CCCC00CCCCCCCCCC76- <1>     db  0cch, 0cch, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 00ch, 018h, 030h
367 00015C59 000000000C1830      <1>
368 00015C60 007CC6FEC0C67C0000- <1>     db  000h, 07ch, 0c6h, 0feh, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 010h, 038h, 06ch, 000h, 078h
368 00015C69 000010386C0078      <1>
369 00015C70 0C7CCCCC7600000000- <1>     db  00ch, 07ch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 000h, 0cch, 0cch, 000h, 078h, 00ch, 07ch
369 00015C79 00CCCC00780C7C      <1>
370 00015C80 CCCC76000000006030- <1>     db  0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 060h, 030h, 018h, 000h, 078h, 00ch, 07ch, 0cch, 0cch
370 00015C89 1800780C7CCCCC      <1>
371 00015C90 7600000000386C3800- <1>     db  076h, 000h, 000h, 000h, 000h, 038h, 06ch, 038h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 076h, 000h
371 00015C99 780C7CCCCC7600      <1>
372 00015CA0 0000000000003C6660- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 03ch, 066h, 060h, 066h, 03ch, 00ch, 006h, 03ch, 000h, 000h
372 00015CA9 663C0C063C0000      <1>
373 00015CB0 0010386C007CC6FEC0- <1>     db  000h, 010h, 038h, 06ch, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h
373 00015CB9 C67C0000000000      <1>
374 00015CC0 CCCC007CC6FEC0C67C- <1>     db  0cch, 0cch, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 060h, 030h, 018h
374 00015CC9 00000000603018      <1>
375 00015CD0 007CC6FEC0C67C0000- <1>     db  000h, 07ch, 0c6h, 0feh, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 066h, 066h, 000h, 038h
375 00015CD9 00000066660038      <1>
376 00015CE0 181818183C00000000- <1>     db  018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 018h, 03ch, 066h, 000h, 038h, 018h, 018h
376 00015CE9 183C6600381818      <1>
377 00015CF0 18183C000000006030- <1>     db  018h, 018h, 03ch, 000h, 000h, 000h, 000h, 060h, 030h, 018h, 000h, 038h, 018h, 018h, 018h, 018h
377 00015CF9 18003818181818      <1>
378 00015D00 3C00000000C6C61038- <1>     db  03ch, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 000h
378 00015D09 6CC6C6FEC6C600      <1>
379 00015D10 0000386C3800386CC6- <1>     db  000h, 000h, 038h, 06ch, 038h, 000h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 000h, 000h, 000h
379 00015D19 C6FEC6C6000000      <1>
380 00015D20 18306000FE66607C60- <1>     db  018h, 030h, 060h, 000h, 0feh, 066h, 060h, 07ch, 060h, 066h, 0feh, 000h, 000h, 000h, 000h, 000h
380 00015D29 66FE0000000000      <1>
381 00015D30 0000CC76367ED8D86E- <1>     db  000h, 000h, 0cch, 076h, 036h, 07eh, 0d8h, 0d8h, 06eh, 000h, 000h, 000h, 000h, 000h, 03eh, 06ch
```

```
00015D39 00000000003E6C      <1>
00015D40 CCCCFECCCCCCCE0000- <1>     db  0cch, 0cch, 0feh, 0cch, 0cch, 0cch, 0ceh, 000h, 000h, 000h, 000h, 010h, 038h, 06ch, 000h, 07ch
00015D49 000010386C007C      <1>
00015D50 C6C6C6C67C00000000- <1>     db  0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 000h, 07ch, 0c6h, 0c6h
00015D59 00C6C6007CC6C6      <1>
00015D60 C6C67C000000006030- <1>     db  0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 060h, 030h, 018h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h
00015D69 18007CC6C6C6C6      <1>
00015D70 7C000000003078CC00- <1>     db  07ch, 000h, 000h, 000h, 000h, 030h, 078h, 0cch, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h
00015D79 CCCCCCCCCC7600      <1>
00015D80 00000060301800CCCC- <1>     db  000h, 000h, 000h, 060h, 030h, 018h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h
00015D89 CCCCCC76000000      <1>
00015D90 0000C6C600C6C6C6C6- <1>     db  000h, 000h, 0c6h, 0c6h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 07eh, 006h, 00ch, 078h, 000h, 000h, 0c6h
00015D99 7E060C780000C6      <1>
00015DA0 C6386CC6C6C6C66C38- <1>     db  0c6h, 038h, 06ch, 0c6h, 0c6h, 0c6h, 0c6h, 06ch, 038h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 000h
00015DA9 00000000C6C600      <1>
00015DB0 C6C6C6C6C6C67C0000- <1>     db  0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 018h, 018h, 03ch, 066h, 060h
00015DB9 000018183C6660      <1>
00015DC0 60663C181800000000- <1>     db  060h, 066h, 03ch, 018h, 018h, 000h, 000h, 000h, 000h, 038h, 06ch, 064h, 060h, 0f0h, 060h, 060h
00015DC9 386C6460F06060      <1>
00015DD0 60E6FC000000000066- <1>     db  060h, 0e6h, 0fch, 000h, 000h, 000h, 000h, 000h, 066h, 066h, 03ch, 018h, 07eh, 018h, 07eh, 018h
00015DD9 663C187E187E18      <1>
00015DE0 1800000000F8CCCCF8- <1>     db  018h, 000h, 000h, 000h, 000h, 0f8h, 0cch, 0cch, 0f8h, 0c4h, 0cch, 0deh, 0cch, 0cch, 0c6h, 000h
00015DE9 C4CCDECCCCC600      <1>
00015DF0 0000000E1B1818187E- <1>     db  000h, 000h, 000h, 00eh, 01bh, 018h, 018h, 018h, 07eh, 018h, 018h, 018h, 018h, 0d8h, 070h, 000h
00015DF9 18181818D87000      <1>
00015E00 0018306000780C7CCC- <1>     db  000h, 018h, 030h, 060h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 00ch
00015E09 CC76000000000C      <1>
00015E10 18300038181818183C- <1>     db  018h, 030h, 000h, 038h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 018h, 030h, 060h
00015E19 00000000183060      <1>
00015E20 007CC6C6C6C67C0000- <1>     db  000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 018h, 030h, 060h, 000h, 0cch
00015E29 000018306000CC      <1>
00015E30 CCCCCCCC7600000000- <1>     db  0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h, 000h, 076h, 0dch, 000h, 0dch, 066h, 066h
00015E39 0076DC00DC6666      <1>
00015E40 66666600000076DC00- <1>     db  066h, 066h, 066h, 000h, 000h, 000h, 076h, 0dch, 000h, 0c6h, 0e6h, 0f6h, 0feh, 0deh, 0ceh, 0c6h
00015E49 C6E6F6FEDECEC6      <1>
00015E50 C6000000003C6C6C3E- <1>     db  0c6h, 000h, 000h, 000h, 000h, 03ch, 06ch, 06ch, 03eh, 000h, 07eh, 000h, 000h, 000h, 000h, 000h
00015E59 007E0000000000      <1>
00015E60 000000386C6C38007C- <1>     db  000h, 000h, 000h, 038h, 06ch, 06ch, 038h, 000h, 07ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00015E69 00000000000000      <1>
00015E70 0000303000303060C6- <1>     db  000h, 000h, 030h, 030h, 000h, 030h, 030h, 060h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h, 000h
00015E79 C67C0000000000      <1>
00015E80 00000000FEC0C0C000- <1>     db  000h, 000h, 000h, 000h, 0feh, 0c0h, 0c0h, 0c0h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00015E89 00000000000000      <1>
00015E90 0000FE060606000000- <1>     db  000h, 000h, 0feh, 006h, 006h, 006h, 000h, 000h, 000h, 000h, 000h, 0c0h, 0c0h, 0c6h, 0cch, 0d8h
00015E99 0000C0C0C6CCD8      <1>
00015EA0 3060DC860C183E0000- <1>     db  030h, 060h, 0dch, 086h, 00ch, 018h, 03eh, 000h, 000h, 0c0h, 0c0h, 0c6h, 0cch, 0d8h, 030h, 066h
00015EA9 C0C0C6CCD83066      <1>
00015EB0 CE9E3E060600000018- <1>     db  0ceh, 09eh, 03eh, 006h, 006h, 000h, 000h, 000h, 018h, 018h, 000h, 018h, 018h, 03ch, 03ch, 03ch
00015EB9 180018183C3C3C      <1>
00015EC0 180000000000000036- <1>     db  018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 036h, 06ch, 0d8h, 06ch, 036h, 000h, 000h, 000h
00015EC9 6CD86C36000000      <1>
00015ED0 000000000000D86C36- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 0d8h, 06ch, 036h, 06ch, 0d8h, 000h, 000h, 000h, 000h, 000h
00015ED9 6CD80000000000      <1>
00015EE0 114411441144114411- <1>     db  011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 055h, 0aah
00015EE9 441144114455AA      <1>
00015EF0 55AA55AA55AA55AA55- <1>     db  055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 0ddh, 077h, 0ddh, 077h
00015EF9 AA55AADD77DD77      <1>
00015F00 DD77DD77DD77DD77DD- <1>     db  0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 018h, 018h, 018h, 018h, 018h, 018h
00015F09 77181818181818      <1>
00015F10 181818181818181818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0f8h
00015F19 181818181818F8      <1>
00015F20 181818181818181818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 0f8h, 018h, 018h
00015F29 1818F818F81818      <1>
00015F30 181818183636363636- <1>     db  018h, 018h, 018h, 018h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 0f6h, 036h, 036h, 036h, 036h
00015F39 3636F636363636      <1>
00015F40 363600000000000000- <1>     db  036h, 036h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 036h, 036h, 036h, 036h, 036h, 036h
00015F49 FE363636363636      <1>
00015F50 0000000000F818F818- <1>     db  000h, 000h, 000h, 000h, 000h, 0f8h, 018h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h, 036h, 036h
00015F59 18181818183636      <1>
00015F60 363636F606F6363636- <1>     db  036h, 036h, 036h, 0f6h, 006h, 0f6h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00015F69 36363636363636      <1>
00015F70 363636363636363636- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 000h, 000h, 000h, 000h, 000h, 0feh
00015F79 360000000000FE      <1>
00015F80 06F636363636363636- <1>     db  006h, 0f6h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 0f6h, 006h, 0feh
00015F89 36363636F606FE      <1>
00015F90 000000000000363636- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 0feh, 000h, 000h
00015F99 36363636FE0000      <1>
00015FA0 000000001818181818- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 0f8h, 000h, 000h, 000h, 000h
00015FA9 F818F800000000      <1>
00015FB0 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h
00015FB9 F8181818181818      <1>
00015FC0 181818181818181F00- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 01fh, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h
00015FC9 00000000001818      <1>
00015FD0 1818181818FF000000- <1>     db  018h, 018h, 018h, 018h, 018h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00015FD9 00000000000000      <1>
00015FE0 000000FF1818181818- <1>     db  000h, 000h, 000h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00015FE9 18181818181818      <1>
00015FF0 181F18181818181800- <1>     db  018h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh
00015FF9 000000000000FF      <1>
00016000 000000000000181818- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0ffh, 018h, 018h
00016009 18181818FF1818      <1>
00016010 181818181818181818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 01fh, 018h, 01fh, 018h, 018h, 018h, 018h
00016019 1F181F18181818      <1>
00016020 181836363636363636- <1>     db  018h, 018h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 037h, 036h, 036h, 036h, 036h, 036h, 036h
00016029 37363636363636      <1>
00016030 363636363637303F00- <1>     db  036h, 036h, 036h, 036h, 036h, 037h, 030h, 03fh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016039 00000000000000      <1>
00016040 0000003F3037363636- <1>     db  000h, 000h, 000h, 03fh, 030h, 037h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016049 36363636363636      <1>
00016050 36F700FF0000000000- <1>     db  036h, 0f7h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh
00016059 000000000000FF      <1>
00016060 00F736363636363636- <1>     db  000h, 0f7h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 037h, 030h, 037h
00016069 36363636373037      <1>
00016070 363636363636000000- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0ffh, 000h, 000h
00016079 0000FF00FF0000      <1>
00016080 000000003636363636- <1>     db  000h, 000h, 000h, 000h, 036h, 036h, 036h, 036h, 036h, 0f7h, 000h, 0f7h, 036h, 036h, 036h, 036h
00016089 F700F736363636      <1>
00016090 36361818181818FF00- <1>     db  036h, 036h, 018h, 018h, 018h, 018h, 018h, 0ffh, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h
00016099 FF000000000000      <1>
000160A0 36363636363636FF00- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000160A9 00000000000000      <1>
000160B0 000000FF00FF181818- <1>     db  000h, 000h, 000h, 0ffh, 000h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h
000160B9 18181800000000      <1>
000160C0 000000FF3636363636- <1>     db  000h, 000h, 000h, 0ffh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
000160C9 36363636363636      <1>
000160D0 363F00000000000018- <1>     db  036h, 03fh, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 018h, 018h, 018h, 01fh, 018h, 01fh
000160D9 181818181F181F      <1>
000160E0 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 01fh, 018h, 01fh, 018h, 018h
000160E9 00001F181F1818      <1>
000160F0 181818180000000000- <1>     db  018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 03fh, 036h, 036h, 036h, 036h
000160F9 00003F36363636      <1>
00016100 363636363636363636- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 0ffh, 036h, 036h, 036h, 036h, 036h, 036h
00016109 FF363636363636      <1>
00016110 1818181818FF18FF18- <1>     db  018h, 018h, 018h, 018h, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
```

```
00016119 18181818181818      <1>
00016120 1818181818F8000000- <1>     db  018h, 018h, 018h, 018h, 018h, 0f8h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016129 00000000000000      <1>
00016130 0000001F1818181818- <1>     db  000h, 000h, 000h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
00016139 18FFFFFFFFFFFF      <1>
00016140 FFFFFFFFFFFFFFF00- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh
00016149 000000000000FF      <1>
00016150 FFFFFFFFFFFF0F0F0- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h
00016159 F0F0F0F0F0F0F0      <1>
00016160 F0F0F0F00F0F0F0F0F- <1>     db  0f0h, 0f0h, 0f0h, 0f0h, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh
00016169 0F0F0F0F0F0F0F      <1>
00016170 0F0FFFFFFFFFFFFFFF- <1>     db  00fh, 00fh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016179 00000000000000      <1>
00016180 000000000076DCD8D8- <1>     db  000h, 000h, 000h, 000h, 000h, 076h, 0dch, 0d8h, 0d8h, 0dch, 076h, 000h, 000h, 000h, 000h, 000h
00016189 DC760000000000      <1>
00016190 00007CC6FCC6C6FCC0- <1>     db  000h, 000h, 07ch, 0c6h, 0fch, 0c6h, 0c6h, 0fch, 0c0h, 0c0h, 040h, 000h, 000h, 000h, 0feh, 0c6h
00016199 C040000000FEC6      <1>
000161A0 C6C0C0C0C0C0C00000- <1>     db  0c6h, 0c0h, 0c0h, 0c0h, 0c0h, 0c0h, 0c0h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 06ch
000161A9 0000000000FE6C      <1>
000161B0 6C6C6C6C6C00000000- <1>     db  06ch, 06ch, 06ch, 06ch, 06ch, 000h, 000h, 000h, 000h, 000h, 0feh, 0c6h, 060h, 030h, 018h, 030h
000161B9 00FEC660301830      <1>
000161C0 60C6FE000000000000- <1>     db  060h, 0c6h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07eh, 0d8h, 0d8h, 0d8h, 0d8h
000161C9 00007ED8D8D8D8      <1>
000161D0 700000000000000066- <1>     db  070h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 066h, 066h, 066h, 066h, 07ch, 060h, 060h, 0c0h
000161D9 6666667C6060C0      <1>
000161E0 00000000000076DC18- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 076h, 0dch, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h
000161E9 18181818000000      <1>
000161F0 00007E183C6666663C- <1>     db  000h, 000h, 07eh, 018h, 03ch, 066h, 066h, 066h, 03ch, 018h, 07eh, 000h, 000h, 000h, 000h, 000h
000161F9 187E0000000000      <1>
00016200 386CC6C6FEC6C66C38- <1>     db  038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 06ch, 038h, 000h, 000h, 000h, 000h, 000h, 038h, 06ch
00016209 0000000000386C      <1>
00016210 C6C6C66C6C6CEE0000- <1>     db  0c6h, 0c6h, 0c6h, 06ch, 06ch, 06ch, 0eeh, 000h, 000h, 000h, 000h, 000h, 01eh, 030h, 018h, 00ch
00016219 0000001E30180C      <1>
00016220 3E6666663C00000000- <1>     db  03eh, 066h, 066h, 066h, 03ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07eh, 0dbh, 0dbh
00016229 000000007EDBDB      <1>
00016230 7E0000000000000003- <1>     db  07eh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 003h, 006h, 07eh, 0dbh, 0dbh, 0f3h, 07eh, 060h
00016239 067EDBDBF37E60      <1>
00016240 C000000000001C3060- <1>     db  0c0h, 000h, 000h, 000h, 000h, 000h, 01ch, 030h, 060h, 060h, 07ch, 060h, 060h, 030h, 01ch, 000h
00016249 607C6060301C00      <1>
00016250 00000000007CC6C6C6- <1>     db  000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h
00016259 C6C6C6C6000000      <1>
00016260 000000FE0000FE0000- <1>     db  000h, 000h, 000h, 0feh, 000h, 000h, 0feh, 000h, 000h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h
00016269 FE000000000000      <1>
00016270 0018187E18180000FF- <1>     db  000h, 018h, 018h, 07eh, 018h, 018h, 000h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 030h, 018h
00016279 00000000003018      <1>
00016280 0C060C1830007E0000- <1>     db  00ch, 006h, 00ch, 018h, 030h, 000h, 07eh, 000h, 000h, 000h, 000h, 000h, 00ch, 018h, 030h, 060h
00016289 0000000C183060      <1>
00016290 30180C007E00000000- <1>     db  030h, 018h, 00ch, 000h, 07eh, 000h, 000h, 000h, 000h, 000h, 00eh, 01bh, 01bh, 018h, 018h, 018h
00016299 000E1B1B181818      <1>
000162A0 181818181818181818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0d8h, 0d8h
000162A9 1818181818D8D8      <1>
000162B0 700000000000001818- <1>     db  070h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h, 07eh, 000h, 018h, 018h, 000h, 000h
000162B9 007E0018180000      <1>
000162C0 00000000000076DC00- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 076h, 0dch, 000h, 076h, 0dch, 000h, 000h, 000h, 000h, 000h
000162C9 76DC0000000000      <1>
000162D0 00386C6C3800000000- <1>     db  000h, 038h, 06ch, 06ch, 038h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000162D9 00000000000000      <1>
000162E0 000000001818000000- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000162E9 00000000000000      <1>
000162F0 000000180000000000- <1>     db  000h, 000h, 000h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 00fh, 00ch, 00ch, 00ch, 00ch
000162F9 00000F0C0C0C0C      <1>
00016300 0CEC6C3C1C00000000- <1>     db  00ch, 0ech, 06ch, 03ch, 01ch, 000h, 000h, 000h, 000h, 0d8h, 06ch, 06ch, 06ch, 06ch, 06ch, 000h
00016309 D86C6C6C6C6C00      <1>
00016310 0000000000000070D8- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 070h, 0d8h, 030h, 060h, 0c8h, 0f8h, 000h, 000h, 000h
00016319 3060C8F8000000      <1>
00016320 0000000000000007C- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 07ch, 07ch, 07ch, 07ch, 07ch, 07ch, 000h, 000h
00016329 7C7C7C7C7C0000      <1>
00016330 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016339 00000000000000      <1>
                             <1> vgafont16:
00016340 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016349 00000000000000      <1>
00016350 00007E81A58181BD99- <1>     db  000h, 000h, 07eh, 081h, 0a5h, 081h, 081h, 0bdh, 099h, 081h, 081h, 07eh, 000h, 000h, 000h, 000h
00016359 81817E00000000      <1>
00016360 00007EFFDBFFFFC3E7- <1>     db  000h, 000h, 07eh, 0ffh, 0dbh, 0ffh, 0ffh, 0c3h, 0e7h, 0ffh, 0ffh, 07eh, 000h, 000h, 000h, 000h
00016369 FFFF7E00000000      <1>
00016370 000000006CFEFEFEFE- <1>     db  000h, 000h, 000h, 000h, 06ch, 0feh, 0feh, 0feh, 0feh, 07ch, 038h, 010h, 000h, 000h, 000h, 000h
00016379 7C381000000000      <1>
00016380 0000000010387CFE7C- <1>     db  000h, 000h, 000h, 000h, 010h, 038h, 07ch, 0feh, 07ch, 038h, 010h, 000h, 000h, 000h, 000h, 000h
00016389 38100000000000      <1>
00016390 000000183C3CE7E7E7- <1>     db  000h, 000h, 000h, 018h, 03ch, 03ch, 0e7h, 0e7h, 0e7h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016399 18183C00000000      <1>
000163A0 000000183C7EFFFF7E- <1>     db  000h, 000h, 000h, 018h, 03ch, 07eh, 0ffh, 0ffh, 07eh, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
000163A9 18183C00000000      <1>
000163B0 00000000000183C3C- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 018h, 03ch, 03ch, 018h, 000h, 000h, 000h, 000h, 000h, 000h
000163B9 18000000000000      <1>
000163C0 FFFFFFFFFFFE7C3C3- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0e7h, 0c3h, 0c3h, 0e7h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
000163C9 E7FFFFFFFFFFFF      <1>
000163D0 00000000003C664242- <1>     db  000h, 000h, 000h, 000h, 000h, 03ch, 066h, 042h, 042h, 066h, 03ch, 000h, 000h, 000h, 000h, 000h
000163D9 663C0000000000      <1>
000163E0 FFFFFFFFFC399BDBD- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0c3h, 099h, 0bdh, 0bdh, 099h, 0c3h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
000163E9 99C3FFFFFFFFFF      <1>
000163F0 00001E0E1A3278CCCC- <1>     db  000h, 000h, 01eh, 00eh, 01ah, 032h, 078h, 0cch, 0cch, 0cch, 0cch, 078h, 000h, 000h, 000h, 000h
000163F9 CCCC7800000000      <1>
00016400 00003C666666663C18- <1>     db  000h, 000h, 03ch, 066h, 066h, 066h, 066h, 03ch, 018h, 07eh, 018h, 018h, 000h, 000h, 000h, 000h
00016409 7E181800000000      <1>
00016410 00003F333F30303030- <1>     db  000h, 000h, 03fh, 033h, 03fh, 030h, 030h, 030h, 030h, 070h, 0f0h, 0e0h, 000h, 000h, 000h, 000h
00016419 70F0E000000000      <1>
00016420 00007F637F63636363- <1>     db  000h, 000h, 07fh, 063h, 07fh, 063h, 063h, 063h, 063h, 067h, 0e7h, 0e6h, 0c0h, 000h, 000h, 000h
00016429 67E7E6C0000000      <1>
00016430 0000001818DB3CE73C- <1>     db  000h, 000h, 000h, 018h, 018h, 0dbh, 03ch, 0e7h, 03ch, 0dbh, 018h, 018h, 000h, 000h, 000h, 000h
00016439 DB181800000000      <1>
00016440 0080C0E0F0F8FEF8F0- <1>     db  000h, 080h, 0c0h, 0e0h, 0f0h, 0f8h, 0feh, 0f8h, 0f0h, 0e0h, 0c0h, 080h, 000h, 000h, 000h, 000h
00016449 E0C08000000000      <1>
00016450 0002060E1E3EFE3E1E- <1>     db  000h, 002h, 006h, 00eh, 01eh, 03eh, 0feh, 03eh, 01eh, 00eh, 006h, 002h, 000h, 000h, 000h, 000h
00016459 0E060200000000      <1>
00016460 0000183C7E1818187E- <1>     db  000h, 000h, 018h, 03ch, 07eh, 018h, 018h, 018h, 07eh, 03ch, 018h, 000h, 000h, 000h, 000h, 000h
00016469 3C180000000000      <1>
00016470 000066666666666666- <1>     db  000h, 000h, 066h, 066h, 066h, 066h, 066h, 066h, 066h, 000h, 066h, 066h, 000h, 000h, 000h, 000h
00016479 00666600000000      <1>
00016480 00007FDBDBDB7B1B1B- <1>     db  000h, 000h, 07fh, 0dbh, 0dbh, 0dbh, 07bh, 01bh, 01bh, 01bh, 01bh, 01bh, 000h, 000h, 000h, 000h
00016489 1B1B1B00000000      <1>
00016490 007CC660386CC6C66C- <1>     db  000h, 07ch, 0c6h, 060h, 038h, 06ch, 0c6h, 0c6h, 06ch, 038h, 00ch, 0c6h, 07ch, 000h, 000h, 000h
00016499 380CC67C000000      <1>
000164A0 0000000000000000FE- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 0feh, 0feh, 0feh, 000h, 000h, 000h, 000h
000164A9 FEFEFE00000000      <1>
000164B0 0000183C7E1818187E- <1>     db  000h, 000h, 018h, 03ch, 07eh, 018h, 018h, 018h, 07eh, 03ch, 018h, 07eh, 000h, 000h, 000h, 000h
000164B9 3C187E00000000      <1>
000164C0 0000183C7E18181818- <1>     db  000h, 000h, 018h, 03ch, 07eh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h
000164C9 18181800000000      <1>
000164D0 000018181818181818- <1>     db  000h, 000h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 07eh, 03ch, 018h, 000h, 000h, 000h, 000h
000164D9 7E3C1800000000      <1>
000164E0 0000000000180CFE0C- <1>     db  000h, 000h, 000h, 000h, 000h, 018h, 00ch, 0feh, 00ch, 018h, 000h, 000h, 000h, 000h, 000h, 000h
000164E9 18000000000000      <1>
```

```
000164F0 00000000003060FE60- <1>     db  000h, 000h, 000h, 000h, 000h, 030h, 060h, 0feh, 060h, 030h, 000h, 000h, 000h, 000h, 000h, 000h
000164F9 30000000000000      <1>
00016500 000000000000C0C0C0- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 0c0h, 0c0h, 0c0h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h
00016509 FE000000000000      <1>
00016510 00000000002466FF66- <1>     db  000h, 000h, 000h, 000h, 000h, 024h, 066h, 0ffh, 066h, 024h, 000h, 000h, 000h, 000h, 000h, 000h
00016519 24000000000000      <1>
00016520 000000001038387C7C- <1>     db  000h, 000h, 000h, 000h, 010h, 038h, 038h, 07ch, 07ch, 0feh, 0feh, 000h, 000h, 000h, 000h, 000h
00016529 FEFE0000000000      <1>
00016530 00000000FEFE7C7C38- <1>     db  000h, 000h, 000h, 000h, 0feh, 0feh, 07ch, 07ch, 038h, 038h, 010h, 000h, 000h, 000h, 000h, 000h
00016539 38100000000000      <1>
00016540 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016549 00000000000000      <1>
00016550 0000183C3C3C181818- <1>     db  000h, 000h, 018h, 03ch, 03ch, 03ch, 018h, 018h, 018h, 000h, 018h, 018h, 000h, 000h, 000h, 000h
00016559 00181800000000      <1>
00016560 006666662400000000- <1>     db  000h, 066h, 066h, 066h, 024h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016569 00000000000000      <1>
00016570 0000006C6CFE6C6C6C- <1>     db  000h, 000h, 000h, 06ch, 06ch, 0feh, 06ch, 06ch, 06ch, 0feh, 06ch, 06ch, 000h, 000h, 000h, 000h
00016579 FE6C6C00000000      <1>
00016580 18187CC6C2C07C0606- <1>     db  018h, 018h, 07ch, 0c6h, 0c2h, 0c0h, 07ch, 006h, 006h, 086h, 0c6h, 07ch, 018h, 018h, 000h, 000h
00016589 86C67C18180000      <1>
00016590 00000000C2C60C1830- <1>     db  000h, 000h, 000h, 000h, 0c2h, 0c6h, 00ch, 018h, 030h, 060h, 0c6h, 086h, 000h, 000h, 000h, 000h
00016599 60C68600000000      <1>
000165A0 0000386C6C3876DCCC- <1>     db  000h, 000h, 038h, 06ch, 06ch, 038h, 076h, 0dch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
000165A9 CCCC7600000000      <1>
000165B0 003030306000000000- <1>     db  000h, 030h, 030h, 030h, 060h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000165B9 00000000000000      <1>
000165C0 00000C183030303030- <1>     db  000h, 000h, 00ch, 018h, 030h, 030h, 030h, 030h, 030h, 030h, 018h, 00ch, 000h, 000h, 000h, 000h
000165C9 30180C00000000      <1>
000165D0 000030180C0C0C0C0C- <1>     db  000h, 000h, 030h, 018h, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch, 018h, 030h, 000h, 000h, 000h, 000h
000165D9 0C183000000000      <1>
000165E0 0000000000663CFF3C- <1>     db  000h, 000h, 000h, 000h, 000h, 066h, 03ch, 0ffh, 03ch, 066h, 000h, 000h, 000h, 000h, 000h, 000h
000165E9 66000000000000      <1>
000165F0 000000000018187E18- <1>     db  000h, 000h, 000h, 000h, 000h, 018h, 018h, 07eh, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h
000165F9 18000000000000      <1>
00016600 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 018h, 030h, 000h, 000h, 000h
00016609 18181830000000      <1>
00016610 00000000000000FE00- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016619 00000000000000      <1>
00016620 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h
00016629 00181800000000      <1>
00016630 0000000002060C1830- <1>     db  000h, 000h, 000h, 000h, 002h, 006h, 00ch, 018h, 030h, 060h, 0c0h, 080h, 000h, 000h, 000h, 000h
00016639 60C08000000000      <1>
00016640 00003C66C3C3DBDBC3- <1>     db  000h, 000h, 03ch, 066h, 0c3h, 0c3h, 0dbh, 0dbh, 0c3h, 0c3h, 066h, 03ch, 000h, 000h, 000h, 000h
00016649 C3663C00000000      <1>
00016650 000018387818181818- <1>     db  000h, 000h, 018h, 038h, 078h, 018h, 018h, 018h, 018h, 018h, 018h, 07eh, 000h, 000h, 000h, 000h
00016659 18187E00000000      <1>
00016660 00007CC6060C183060- <1>     db  000h, 000h, 07ch, 0c6h, 006h, 00ch, 018h, 030h, 060h, 0c0h, 0c6h, 0feh, 000h, 000h, 000h, 000h
00016669 C0C6FE00000000      <1>
00016670 00007CC606063C0606- <1>     db  000h, 000h, 07ch, 0c6h, 006h, 006h, 03ch, 006h, 006h, 006h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016679 06C67C00000000      <1>
00016680 00000C1C3C6CCCFE0C- <1>     db  000h, 000h, 00ch, 01ch, 03ch, 06ch, 0cch, 0feh, 00ch, 00ch, 00ch, 01eh, 000h, 000h, 000h, 000h
00016689 0C0C1E00000000      <1>
00016690 0000FEC0C0C0FC0606- <1>     db  000h, 000h, 0feh, 0c0h, 0c0h, 0c0h, 0fch, 006h, 006h, 006h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016699 06C67C00000000      <1>
000166A0 00003860C0C0FCC6C6- <1>     db  000h, 000h, 038h, 060h, 0c0h, 0c0h, 0fch, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
000166A9 C6C67C00000000      <1>
000166B0 0000FEC606060C1830- <1>     db  000h, 000h, 0feh, 0c6h, 006h, 006h, 00ch, 018h, 030h, 030h, 030h, 030h, 000h, 000h, 000h, 000h
000166B9 30303000000000      <1>
000166C0 00007CC6C6C67CC6C6- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
000166C9 C6C67C00000000      <1>
000166D0 00007CC6C6C67E0606- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 07eh, 006h, 006h, 006h, 00ch, 078h, 000h, 000h, 000h, 000h
000166D9 060C7800000000      <1>
000166E0 000000001818000000- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 000h
000166E9 18180000000000      <1>
000166F0 000000001818000000- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 018h, 018h, 030h, 000h, 000h, 000h, 000h
000166F9 18183000000000      <1>
00016700 000000060C18306030- <1>     db  000h, 000h, 000h, 006h, 00ch, 018h, 030h, 060h, 030h, 018h, 00ch, 006h, 000h, 000h, 000h, 000h
00016709 180C0600000000      <1>
00016710 00000000007E00007E- <1>     db  000h, 000h, 000h, 000h, 000h, 07eh, 000h, 000h, 07eh, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016719 00000000000000      <1>
00016720 0000006030180C060C- <1>     db  000h, 000h, 000h, 060h, 030h, 018h, 00ch, 006h, 00ch, 018h, 030h, 060h, 000h, 000h, 000h, 000h
00016729 18306000000000      <1>
00016730 00007CC6C60C181818- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 00ch, 018h, 018h, 018h, 000h, 018h, 018h, 000h, 000h, 000h, 000h
00016739 00181800000000      <1>
00016740 0000007CC6C6DEDEDE- <1>     db  000h, 000h, 000h, 07ch, 0c6h, 0c6h, 0deh, 0deh, 0deh, 0dch, 0c0h, 07ch, 000h, 000h, 000h, 000h
00016749 DCC07C00000000      <1>
00016750 000010386CC6C6FEC6- <1>     db  000h, 000h, 010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00016759 C6C6C600000000      <1>
00016760 0000FC6666667C6666- <1>     db  000h, 000h, 0fch, 066h, 066h, 066h, 07ch, 066h, 066h, 066h, 066h, 0fch, 000h, 000h, 000h, 000h
00016769 6666FC00000000      <1>
00016770 00003C66C2C0C0C0C0- <1>     db  000h, 000h, 03ch, 066h, 0c2h, 0c0h, 0c0h, 0c0h, 0c0h, 0c2h, 066h, 03ch, 000h, 000h, 000h, 000h
00016779 C2663C00000000      <1>
00016780 0000F86C6666666666- <1>     db  000h, 000h, 0f8h, 06ch, 066h, 066h, 066h, 066h, 066h, 066h, 06ch, 0f8h, 000h, 000h, 000h, 000h
00016789 666CF800000000      <1>
00016790 0000FE666268786860- <1>     db  000h, 000h, 0feh, 066h, 062h, 068h, 078h, 068h, 060h, 062h, 066h, 0feh, 000h, 000h, 000h, 000h
00016799 6266FE00000000      <1>
000167A0 0000FE666268786860- <1>     db  000h, 000h, 0feh, 066h, 062h, 068h, 078h, 068h, 060h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h
000167A9 6060F000000000      <1>
000167B0 00003C66C2C0C0DEC6- <1>     db  000h, 000h, 03ch, 066h, 0c2h, 0c0h, 0c0h, 0deh, 0c6h, 0c6h, 066h, 03ah, 000h, 000h, 000h, 000h
000167B9 C6663A00000000      <1>
000167C0 0000C6C6C6C6FEC6C6- <1>     db  000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
000167C9 C6C6C600000000      <1>
000167D0 00003C181818181818- <1>     db  000h, 000h, 03ch, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
000167D9 18183C00000000      <1>
000167E0 00001E0C0C0C0C0CCC- <1>     db  000h, 000h, 01eh, 00ch, 00ch, 00ch, 00ch, 00ch, 0cch, 0cch, 0cch, 078h, 000h, 000h, 000h, 000h
000167E9 CCCC7800000000      <1>
000167F0 0000E666666C78786C- <1>     db  000h, 000h, 0e6h, 066h, 066h, 06ch, 078h, 078h, 06ch, 066h, 066h, 0e6h, 000h, 000h, 000h, 000h
000167F9 6666E600000000      <1>
00016800 0000F0606060606060- <1>     db  000h, 000h, 0f0h, 060h, 060h, 060h, 060h, 060h, 060h, 062h, 066h, 0feh, 000h, 000h, 000h, 000h
00016809 6266FE00000000      <1>
00016810 0000C3E7FFFFDBC3C3- <1>     db  000h, 000h, 0c3h, 0e7h, 0ffh, 0ffh, 0dbh, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 000h, 000h, 000h, 000h
00016819 C3C3C300000000      <1>
00016820 0000C6E6F6FEDECEC6- <1>     db  000h, 000h, 0c6h, 0e6h, 0f6h, 0feh, 0deh, 0ceh, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00016829 C6C6C600000000      <1>
00016830 00007CC6C6C6C6C6C6- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016839 C6C67C00000000      <1>
00016840 0000FC6666667C6060- <1>     db  000h, 000h, 0fch, 066h, 066h, 066h, 07ch, 060h, 060h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h
00016849 6060F000000000      <1>
00016850 00007CC6C6C6C6C6C6- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0d6h, 0deh, 07ch, 00ch, 00eh, 000h, 000h
00016859 D6DE7C0C0E0000      <1>
00016860 0000FC6666667C6C66- <1>     db  000h, 000h, 0fch, 066h, 066h, 066h, 07ch, 06ch, 066h, 066h, 066h, 0e6h, 000h, 000h, 000h, 000h
00016869 6666E600000000      <1>
00016870 00007CC6C660380C06- <1>     db  000h, 000h, 07ch, 0c6h, 0c6h, 060h, 038h, 00ch, 006h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016879 C6C67C00000000      <1>
00016880 0000FFDB9918181818- <1>     db  000h, 000h, 0ffh, 0dbh, 099h, 018h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016889 18183C00000000      <1>
00016890 0000C6C6C6C6C6C6C6- <1>     db  000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016899 C6C67C00000000      <1>
000168A0 0000C3C3C3C3C3C3C3- <1>     db  000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h, 000h, 000h, 000h, 000h
000168A9 663C1800000000      <1>
000168B0 0000C3C3C3C3C3DBDB- <1>     db  000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 0dbh, 0dbh, 0ffh, 066h, 066h, 000h, 000h, 000h, 000h
000168B9 FF666600000000      <1>
000168C0 0000C3C3663C18183C- <1>     db  000h, 000h, 0c3h, 0c3h, 066h, 03ch, 018h, 018h, 03ch, 066h, 0c3h, 0c3h, 000h, 000h, 000h, 000h
000168C9 66C3C300000000      <1>
```

```
000168D0 0000C3C3C3663C1818- <1>     db  000h, 000h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
000168D9 18183C00000000      <1>
000168E0 0000FFC3860C183060- <1>     db  000h, 000h, 0ffh, 0c3h, 086h, 00ch, 018h, 030h, 060h, 0c1h, 0c3h, 0ffh, 000h, 000h, 000h, 000h
000168E9 C1C3FF00000000      <1>
000168F0 00003C303030303030- <1>     db  000h, 000h, 03ch, 030h, 030h, 030h, 030h, 030h, 030h, 030h, 030h, 03ch, 000h, 000h, 000h, 000h
000168F9 30303C00000000      <1>
00016900 00000080C0E070381C- <1>     db  000h, 000h, 000h, 080h, 0c0h, 0e0h, 070h, 038h, 01ch, 00eh, 006h, 002h, 000h, 000h, 000h, 000h
00016909 0E060200000000      <1>
00016910 00003C0C0C0C0C0C0C- <1>     db  000h, 000h, 03ch, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch, 00ch, 03ch, 000h, 000h, 000h, 000h
00016919 0C0C3C00000000      <1>
00016920 10386CC60000000000- <1>     db  010h, 038h, 06ch, 0c6h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016929 00000000000000      <1>
00016930 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 000h
00016939 00000000FF0000      <1>
00016940 303018000000000000- <1>     db  030h, 030h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016949 00000000000000      <1>
00016950 0000000000780C7CCC- <1>     db  000h, 000h, 000h, 000h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016959 CCCC7600000000      <1>
00016960 0000E06060786C6666- <1>     db  000h, 000h, 0e0h, 060h, 060h, 078h, 06ch, 066h, 066h, 066h, 066h, 07ch, 000h, 000h, 000h, 000h
00016969 66667C00000000      <1>
00016970 00000000007CC6C0C0- <1>     db  000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0c0h, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016979 C0C67C00000000      <1>
00016980 00001C0C0C3C6CCCCC- <1>     db  000h, 000h, 01ch, 00ch, 00ch, 03ch, 06ch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016989 CCCC7600000000      <1>
00016990 00000000007CC6FEC0- <1>     db  000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016999 C0C67C00000000      <1>
000169A0 0000386C6460F06060- <1>     db  000h, 000h, 038h, 06ch, 064h, 060h, 0f0h, 060h, 060h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h
000169A9 6060F000000000      <1>
000169B0 000000000076CCCCCC- <1>     db  000h, 000h, 000h, 000h, 000h, 076h, 0cch, 0cch, 0cch, 0cch, 0cch, 07ch, 00ch, 0cch, 078h, 000h
000169B9 CCCC7C0CCC7800      <1>
000169C0 0000E060606C766666- <1>     db  000h, 000h, 0e0h, 060h, 060h, 06ch, 076h, 066h, 066h, 066h, 066h, 0e6h, 000h, 000h, 000h, 000h
000169C9 6666E600000000      <1>
000169D0 000018180038181818- <1>     db  000h, 000h, 018h, 018h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
000169D9 18183C00000000      <1>
000169E0 00000606000E060606- <1>     db  000h, 000h, 006h, 006h, 000h, 00eh, 006h, 006h, 006h, 006h, 006h, 006h, 066h, 066h, 03ch, 000h
000169E9 06060666663C00      <1>
000169F0 0000E06060666C7878- <1>     db  000h, 000h, 0e0h, 060h, 060h, 066h, 06ch, 078h, 078h, 06ch, 066h, 0e6h, 000h, 000h, 000h, 000h
000169F9 6C66E600000000      <1>
00016A00 000038181818181818- <1>     db  000h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016A09 18183C00000000      <1>
00016A10 0000000000E6FFDBDB- <1>     db  000h, 000h, 000h, 000h, 000h, 0e6h, 0ffh, 0dbh, 0dbh, 0dbh, 0dbh, 0dbh, 000h, 000h, 000h, 000h
00016A19 DBDBDB00000000      <1>
00016A20 0000000000DC666666- <1>     db  000h, 000h, 000h, 000h, 000h, 0dch, 066h, 066h, 066h, 066h, 066h, 066h, 000h, 000h, 000h, 000h
00016A29 66666600000000      <1>
00016A30 00000000007CC6C6C6- <1>     db  000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016A39 C6C67C00000000      <1>
00016A40 0000000000DC666666- <1>     db  000h, 000h, 000h, 000h, 000h, 0dch, 066h, 066h, 066h, 066h, 066h, 07ch, 060h, 060h, 0f0h, 000h
00016A49 66667C6060F000      <1>
00016A50 000000000076CCCCCC- <1>     db  000h, 000h, 000h, 000h, 000h, 076h, 0cch, 0cch, 0cch, 0cch, 0cch, 07ch, 00ch, 00ch, 01eh, 000h
00016A59 CCCC7C0C0C1E00      <1>
00016A60 0000000000DC766660- <1>     db  000h, 000h, 000h, 000h, 000h, 0dch, 076h, 066h, 060h, 060h, 060h, 0f0h, 000h, 000h, 000h, 000h
00016A69 6060F000000000      <1>
00016A70 00000000007CC66038- <1>     db  000h, 000h, 000h, 000h, 000h, 07ch, 0c6h, 060h, 038h, 00ch, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016A79 0CC67C00000000      <1>
00016A80 0000103030FC303030- <1>     db  000h, 000h, 010h, 030h, 030h, 0fch, 030h, 030h, 030h, 030h, 036h, 01ch, 000h, 000h, 000h, 000h
00016A89 30361C00000000      <1>
00016A90 0000000000CCCCCCCC- <1>     db  000h, 000h, 000h, 000h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016A99 CCCC7600000000      <1>
00016AA0 0000000000C3C3C3C3- <1>     db  000h, 000h, 000h, 000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h, 000h, 000h, 000h, 000h
00016AA9 663C1800000000      <1>
00016AB0 0000000000C3C3C3DB- <1>     db  000h, 000h, 000h, 000h, 000h, 0c3h, 0c3h, 0c3h, 0dbh, 0dbh, 0ffh, 066h, 000h, 000h, 000h, 000h
00016AB9 DBFF6600000000      <1>
00016AC0 0000000000C3663C18- <1>     db  000h, 000h, 000h, 000h, 000h, 0c3h, 066h, 03ch, 018h, 03ch, 066h, 0c3h, 000h, 000h, 000h, 000h
00016AC9 3C66C300000000      <1>
00016AD0 0000000000C6C6C6C6- <1>     db  000h, 000h, 000h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07eh, 006h, 00ch, 0f8h, 000h
00016AD9 C6C67E060CF800      <1>
00016AE0 0000000000FECC1830- <1>     db  000h, 000h, 000h, 000h, 000h, 0feh, 0cch, 018h, 030h, 060h, 0c6h, 0feh, 000h, 000h, 000h, 000h
00016AE9 60C6FE00000000      <1>
00016AF0 00000E181818701818- <1>     db  000h, 000h, 00eh, 018h, 018h, 018h, 070h, 018h, 018h, 018h, 018h, 00eh, 000h, 000h, 000h, 000h
00016AF9 18180E00000000      <1>
00016B00 000018181818001818- <1>     db  000h, 000h, 018h, 018h, 018h, 018h, 000h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h
00016B09 18181800000000      <1>
00016B10 0000701818180E1818- <1>     db  000h, 000h, 070h, 018h, 018h, 018h, 00eh, 018h, 018h, 018h, 018h, 070h, 000h, 000h, 000h, 000h
00016B19 18187000000000      <1>
00016B20 000076DC0000000000- <1>     db  000h, 000h, 076h, 0dch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016B29 00000000000000      <1>
00016B30 0000000010386CC6C6- <1>     db  000h, 000h, 000h, 000h, 010h, 038h, 06ch, 0c6h, 0c6h, 0c6h, 0feh, 000h, 000h, 000h, 000h, 000h
00016B39 C6FE0000000000      <1>
00016B40 00003C66C2C0C0C0C2- <1>     db  000h, 000h, 03ch, 066h, 0c2h, 0c0h, 0c0h, 0c0h, 0c2h, 066h, 03ch, 00ch, 006h, 07ch, 000h, 000h
00016B49 663C0C067C0000      <1>
00016B50 0000CC0000CCCCCCCC- <1>     db  000h, 000h, 0cch, 000h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016B59 CCCC7600000000      <1>
00016B60 000C1830007CC6FEC0- <1>     db  000h, 00ch, 018h, 030h, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016B69 C0C67C00000000      <1>
00016B70 0010386C00780C7CCC- <1>     db  000h, 010h, 038h, 06ch, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016B79 CCCC7600000000      <1>
00016B80 0000CC0000780C7CCC- <1>     db  000h, 000h, 0cch, 000h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016B89 CCCC7600000000      <1>
00016B90 0060301800780C7CCC- <1>     db  000h, 060h, 030h, 018h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016B99 CCCC7600000000      <1>
00016BA0 00386C3800780C7CCC- <1>     db  000h, 038h, 06ch, 038h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016BA9 CCCC7600000000      <1>
00016BB0 000000003C66606066- <1>     db  000h, 000h, 000h, 000h, 03ch, 066h, 060h, 060h, 066h, 03ch, 00ch, 006h, 03ch, 000h, 000h, 000h
00016BB9 3C0C063C000000      <1>
00016BC0 0010386C007CC6FEC0- <1>     db  000h, 010h, 038h, 06ch, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016BC9 C0C67C00000000      <1>
00016BD0 0000C600007CC6FEC0- <1>     db  000h, 000h, 0c6h, 000h, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016BD9 C0C67C00000000      <1>
00016BE0 00603018007CC6FEC0- <1>     db  000h, 060h, 030h, 018h, 000h, 07ch, 0c6h, 0feh, 0c0h, 0c0h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016BE9 C0C67C00000000      <1>
00016BF0 000066000038181818- <1>     db  000h, 000h, 066h, 000h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016BF9 18183C00000000      <1>
00016C00 00183C660038181818- <1>     db  000h, 018h, 03ch, 066h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016C09 18183C00000000      <1>
00016C10 006030180038181818- <1>     db  000h, 060h, 030h, 018h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016C19 18183C00000000      <1>
00016C20 00C60010386CC6C6FE- <1>     db  000h, 0c6h, 000h, 010h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00016C29 C6C6C600000000      <1>
00016C30 386C3800386CC6C6FE- <1>     db  038h, 06ch, 038h, 000h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00016C39 C6C6C600000000      <1>
00016C40 18306000FE66607C60- <1>     db  018h, 030h, 060h, 000h, 0feh, 066h, 060h, 07ch, 060h, 060h, 066h, 0feh, 000h, 000h, 000h, 000h
00016C49 6066FE00000000      <1>
00016C50 00000000006E3B1B7E- <1>     db  000h, 000h, 000h, 000h, 000h, 06eh, 03bh, 01bh, 07eh, 0d8h, 0dch, 077h, 000h, 000h, 000h, 000h
00016C59 D8DC7700000000      <1>
00016C60 00003E6CCCCCFECCCC- <1>     db  000h, 000h, 03eh, 06ch, 0cch, 0cch, 0feh, 0cch, 0cch, 0cch, 0cch, 0ceh, 000h, 000h, 000h, 000h
00016C69 CCCCCE00000000      <1>
00016C70 0010386C007CC6C6C6- <1>     db  000h, 010h, 038h, 06ch, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016C79 C6C67C00000000      <1>
00016C80 0000C600007CC6C6C6- <1>     db  000h, 000h, 0c6h, 000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016C89 C6C67C00000000      <1>
00016C90 00603018007CC6C6C6- <1>     db  000h, 060h, 030h, 018h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016C99 C6C67C00000000      <1>
00016CA0 003078CC00CCCCCCCC- <1>     db  000h, 030h, 078h, 0cch, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016CA9 CCCC7600000000      <1>
```

```
00016CB0 0060301800CCCCCCCC- <1>     db  000h, 060h, 030h, 018h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016CB9 CCCC7600000000      <1>
00016CC0 0000C60000C6C6C6C6- <1>     db  000h, 000h, 0c6h, 000h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07eh, 006h, 00ch, 078h, 000h
00016CC9 C6C67E060C7800      <1>
00016CD0 00C6007CC6C6C6C6C6- <1>     db  000h, 0c6h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016CD9 C6C67C00000000      <1>
00016CE0 00C600C6C6C6C6C6C6- <1>     db  000h, 0c6h, 000h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016CE9 C6C67C00000000      <1>
00016CF0 0018187EC3C0C0C0C3- <1>     db  000h, 018h, 018h, 07eh, 0c3h, 0c0h, 0c0h, 0c0h, 0c3h, 07eh, 018h, 018h, 000h, 000h, 000h, 000h
00016CF9 7E181800000000      <1>
00016D00 00386C6460F0606060- <1>     db  000h, 038h, 06ch, 064h, 060h, 0f0h, 060h, 060h, 060h, 060h, 0e6h, 0fch, 000h, 000h, 000h, 000h
00016D09 60E6FC00000000      <1>
00016D10 0000C3663C18FF18FF- <1>     db  000h, 000h, 0c3h, 066h, 03ch, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 018h, 000h, 000h, 000h, 000h
00016D19 18181800000000      <1>
00016D20 00FC66667C62666F66- <1>     db  000h, 0fch, 066h, 066h, 07ch, 062h, 066h, 06fh, 066h, 066h, 066h, 0f3h, 000h, 000h, 000h, 000h
00016D29 6666F300000000      <1>
00016D30 000E1B1818187E1818- <1>     db  000h, 00eh, 01bh, 018h, 018h, 018h, 07eh, 018h, 018h, 018h, 018h, 018h, 0d8h, 070h, 000h, 000h
00016D39 181818D8700000      <1>
00016D40 0018306000780C7CCC- <1>     db  000h, 018h, 030h, 060h, 000h, 078h, 00ch, 07ch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016D49 CCCC7600000000      <1>
00016D50 000C18300038181818- <1>     db  000h, 00ch, 018h, 030h, 000h, 038h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h
00016D59 18183C00000000      <1>
00016D60 00183060007CC6C6C6- <1>     db  000h, 018h, 030h, 060h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016D69 C6C67C00000000      <1>
00016D70 0018306000CCCCCCCC- <1>     db  000h, 018h, 030h, 060h, 000h, 0cch, 0cch, 0cch, 0cch, 0cch, 0cch, 076h, 000h, 000h, 000h, 000h
00016D79 CCCC7600000000      <1>
00016D80 000076DC00DC666666- <1>     db  000h, 000h, 076h, 0dch, 000h, 0dch, 066h, 066h, 066h, 066h, 066h, 066h, 000h, 000h, 000h, 000h
00016D89 66666600000000      <1>
00016D90 76DC00C6E6F6FEDECE- <1>     db  076h, 0dch, 000h, 0c6h, 0e6h, 0f6h, 0feh, 0deh, 0ceh, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00016D99 C6C6C600000000      <1>
00016DA0 003C6C6C3E007E0000- <1>     db  000h, 03ch, 06ch, 06ch, 03eh, 000h, 07eh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016DA9 00000000000000      <1>
00016DB0 00386C6C38007C0000- <1>     db  000h, 038h, 06ch, 06ch, 038h, 000h, 07ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016DB9 00000000000000      <1>
00016DC0 0000303000303060C0- <1>     db  000h, 000h, 030h, 030h, 000h, 030h, 030h, 060h, 0c0h, 0c6h, 0c6h, 07ch, 000h, 000h, 000h, 000h
00016DC9 C6C67C00000000      <1>
00016DD0 000000000000FEC0C0- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 0feh, 0c0h, 0c0h, 0c0h, 0c0h, 000h, 000h, 000h, 000h, 000h
00016DD9 C0C00000000000      <1>
00016DE0 000000000000FE0606- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 0feh, 006h, 006h, 006h, 006h, 000h, 000h, 000h, 000h, 000h
00016DE9 06060000000000      <1>
00016DF0 00C0C0C2C6CC183060- <1>     db  000h, 0c0h, 0c0h, 0c2h, 0c6h, 0cch, 018h, 030h, 060h, 0ceh, 09bh, 006h, 00ch, 01fh, 000h, 000h
00016DF9 CE9B060C1F0000      <1>
00016E00 00C0C0C2C6CC183066- <1>     db  000h, 0c0h, 0c0h, 0c2h, 0c6h, 0cch, 018h, 030h, 066h, 0ceh, 096h, 03eh, 006h, 006h, 000h, 000h
00016E09 CE963E06060000      <1>
00016E10 00001818001818183C- <1>     db  000h, 000h, 018h, 018h, 000h, 018h, 018h, 018h, 03ch, 03ch, 03ch, 018h, 000h, 000h, 000h, 000h
00016E19 3C3C1800000000      <1>
00016E20 0000000000366CD86C- <1>     db  000h, 000h, 000h, 000h, 000h, 036h, 06ch, 0d8h, 06ch, 036h, 000h, 000h, 000h, 000h, 000h, 000h
00016E29 36000000000000      <1>
00016E30 0000000000D86C366C- <1>     db  000h, 000h, 000h, 000h, 000h, 0d8h, 06ch, 036h, 06ch, 0d8h, 000h, 000h, 000h, 000h, 000h, 000h
00016E39 D8000000000000      <1>
00016E40 114411441144114411- <1>     db  011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h, 011h, 044h
00016E49 44114411441144      <1>
00016E50 55AA55AA55AA55AA55- <1>     db  055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah, 055h, 0aah
00016E59 AA55AA55AA55AA      <1>
00016E60 DD77DD77DD77DD77DD- <1>     db  0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h, 0ddh, 077h
00016E69 77DD77DD77DD77      <1>
00016E70 181818181818181818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016E79 18181818181818      <1>
00016E80 18181818181818F818- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016E89 18181818181818      <1>
00016E90 1818181818F818F818- <1>     db  018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016E99 18181818181818      <1>
00016EA0 36363636363636F636- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 0f6h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016EA9 36363636363636      <1>
00016EB0 00000000000000FE36- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0feh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016EB9 36363636363636      <1>
00016EC0 0000000000F818F818- <1>     db  000h, 000h, 000h, 000h, 000h, 0f8h, 018h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016EC9 18181818181818      <1>
00016ED0 3636363636F606F636- <1>     db  036h, 036h, 036h, 036h, 036h, 0f6h, 006h, 0f6h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016ED9 36363636363636      <1>
00016EE0 363636363636363636- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016EE9 36363636363636      <1>
00016EF0 0000000000FE06F636- <1>     db  000h, 000h, 000h, 000h, 000h, 0feh, 006h, 0f6h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016EF9 36363636363636      <1>
00016F00 3636363636F606FE00- <1>     db  036h, 036h, 036h, 036h, 036h, 0f6h, 006h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F09 00000000000000      <1>
00016F10 36363636363636FE00- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 0feh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F19 00000000000000      <1>
00016F20 1818181818F818F800- <1>     db  018h, 018h, 018h, 018h, 018h, 0f8h, 018h, 0f8h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F29 00000000000000      <1>
00016F30 00000000000000F818- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0f8h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016F39 18181818181818      <1>
00016F40 181818181818181F00- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 01fh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F49 00000000000000      <1>
00016F50 18181818181818FF00- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F59 00000000000000      <1>
00016F60 00000000000000FF18- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016F69 18181818181818      <1>
00016F70 181818181818181F18- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016F79 18181818181818      <1>
00016F80 00000000000000FF00- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016F89 00000000000000      <1>
00016F90 181818181818181FF18- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016F99 18181818181818      <1>
00016FA0 18181818181F181F18- <1>     db  018h, 018h, 018h, 018h, 018h, 01fh, 018h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00016FA9 18181818181818      <1>
00016FB0 363636363636363736- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 037h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016FB9 36363636363636      <1>
00016FC0 363636363637303F00- <1>     db  036h, 036h, 036h, 036h, 036h, 037h, 030h, 03fh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016FC9 00000000000000      <1>
00016FD0 00000000003F303736- <1>     db  000h, 000h, 000h, 000h, 000h, 03fh, 030h, 037h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016FD9 36363636363636      <1>
00016FE0 3636363636F700FF00- <1>     db  036h, 036h, 036h, 036h, 036h, 0f7h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00016FE9 00000000000000      <1>
00016FF0 0000000000FF00F736- <1>     db  000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0f7h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00016FF9 36363636363636      <1>
00017000 363636363637303736- <1>     db  036h, 036h, 036h, 036h, 036h, 037h, 030h, 037h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00017009 36363636363636      <1>
00017010 0000000000FF00FF00- <1>     db  000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017019 00000000000000      <1>
00017020 3636363636F700F736- <1>     db  036h, 036h, 036h, 036h, 036h, 0f7h, 000h, 0f7h, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00017029 36363636363636      <1>
00017030 1818181818FF00FF00- <1>     db  018h, 018h, 018h, 018h, 018h, 0ffh, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017039 00000000000000      <1>
00017040 36363636363636FF00- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017049 00000000000000      <1>
00017050 0000000000FF00FF18- <1>     db  000h, 000h, 000h, 000h, 000h, 0ffh, 000h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00017059 18181818181818      <1>
00017060 00000000000000FF36- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
00017069 36363636363636      <1>
00017070 363636363636363F00- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 03fh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017079 00000000000000      <1>
00017080 18181818181F181F00- <1>     db  018h, 018h, 018h, 018h, 018h, 01fh, 018h, 01fh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017089 00000000000000      <1>
```

```
00017090 00000000001F181F18- <1>     db  000h, 000h, 000h, 000h, 000h, 01fh, 018h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00017099 18181818181818      <1>
000170A0 000000000000003F36- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 03fh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
000170A9 36363636363636      <1>
000170B0 36363636363636FF36- <1>     db  036h, 036h, 036h, 036h, 036h, 036h, 036h, 0ffh, 036h, 036h, 036h, 036h, 036h, 036h, 036h, 036h
000170B9 36363636363636      <1>
000170C0 1818181818FF18FF18- <1>     db  018h, 018h, 018h, 018h, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
000170C9 18181818181818      <1>
000170D0 18181818181818F800- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 0f8h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000170D9 00000000000000      <1>
000170E0 000000000000001F18- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 01fh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
000170E9 18181818181818      <1>
000170F0 FFFFFFFFFFFFFFFFFF- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
000170F9 FFFFFFFFFFFFFF      <1>
00017100 00000000000000FFFF- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh
00017109 FFFFFFFFFFFFFF      <1>
00017110 F0F0F0F0F0F0F0F0F0- <1>     db  0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h, 0f0h
00017119 F0F0F0F0F0F0F0      <1>
00017120 0F0F0F0F0F0F0F0F0F- <1>     db  00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh, 00fh
00017129 0F0F0F0F0F0F0F      <1>
00017130 FFFFFFFFFFFFF0000- <1>     db  0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017139 00000000000000      <1>
00017140 000000000076DCD8D8- <1>     db  000h, 000h, 000h, 000h, 000h, 076h, 0dch, 0d8h, 0d8h, 0d8h, 0dch, 076h, 000h, 000h, 000h, 000h
00017149 D8DC7600000000      <1>
00017150 000078CCCCCCD8CCC6- <1>     db  000h, 000h, 078h, 0cch, 0cch, 0cch, 0d8h, 0cch, 0c6h, 0c6h, 0c6h, 0cch, 000h, 000h, 000h, 000h
00017159 C6C6CC00000000      <1>
00017160 0000FEC6C6C0C0C0C0- <1>     db  000h, 000h, 0feh, 0c6h, 0c6h, 0c0h, 0c0h, 0c0h, 0c0h, 0c0h, 0c0h, 0c0h, 000h, 000h, 000h, 000h
00017169 C0C0C000000000      <1>
00017170 00000000FE6C6C6C6C- <1>     db  000h, 000h, 000h, 000h, 0feh, 06ch, 06ch, 06ch, 06ch, 06ch, 06ch, 06ch, 000h, 000h, 000h, 000h
00017179 6C6C6C00000000      <1>
00017180 000000FEC660301830- <1>     db  000h, 000h, 000h, 0feh, 0c6h, 060h, 030h, 018h, 030h, 060h, 0c6h, 0feh, 000h, 000h, 000h, 000h
00017189 60C6FE00000000      <1>
00017190 00000000007ED8D8D8- <1>     db  000h, 000h, 000h, 000h, 000h, 07eh, 0d8h, 0d8h, 0d8h, 0d8h, 0d8h, 070h, 000h, 000h, 000h, 000h
00017199 D8D87000000000      <1>
000171A0 000000006666666666- <1>     db  000h, 000h, 000h, 000h, 066h, 066h, 066h, 066h, 066h, 07ch, 060h, 060h, 0c0h, 000h, 000h, 000h
000171A9 7C6060C0000000      <1>
000171B0 0000000076DC181818- <1>     db  000h, 000h, 000h, 000h, 076h, 0dch, 018h, 018h, 018h, 018h, 018h, 018h, 000h, 000h, 000h, 000h
000171B9 18181800000000      <1>
000171C0 0000007E183C666666- <1>     db  000h, 000h, 000h, 07eh, 018h, 03ch, 066h, 066h, 066h, 03ch, 018h, 07eh, 000h, 000h, 000h, 000h
000171C9 3C187E00000000      <1>
000171D0 000000386CC6C6FEC6- <1>     db  000h, 000h, 000h, 038h, 06ch, 0c6h, 0c6h, 0feh, 0c6h, 0c6h, 06ch, 038h, 000h, 000h, 000h, 000h
000171D9 C66C3800000000      <1>
000171E0 0000386CC6C6C66C6C- <1>     db  000h, 000h, 038h, 06ch, 0c6h, 0c6h, 0c6h, 06ch, 06ch, 06ch, 06ch, 0eeh, 000h, 000h, 000h, 000h
000171E9 6C6CEE00000000      <1>
000171F0 00001E30180C3E6666- <1>     db  000h, 000h, 01eh, 030h, 018h, 00ch, 03eh, 066h, 066h, 066h, 066h, 03ch, 000h, 000h, 000h, 000h
000171F9 66663C00000000      <1>
00017200 00000000007EDBDBDB- <1>     db  000h, 000h, 000h, 000h, 000h, 07eh, 0dbh, 0dbh, 0dbh, 07eh, 000h, 000h, 000h, 000h, 000h, 000h
00017209 7E000000000000      <1>
00017210 00000003067EDBDBF3- <1>     db  000h, 000h, 000h, 003h, 006h, 07eh, 0dbh, 0dbh, 0f3h, 07eh, 060h, 0c0h, 000h, 000h, 000h, 000h
00017219 7E60C000000000      <1>
00017220 00001C3060607C6060- <1>     db  000h, 000h, 01ch, 030h, 060h, 060h, 07ch, 060h, 060h, 060h, 030h, 01ch, 000h, 000h, 000h, 000h
00017229 60301C00000000      <1>
00017230 0000007CC6C6C6C6C6- <1>     db  000h, 000h, 000h, 07ch, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 0c6h, 000h, 000h, 000h, 000h
00017239 C6C6C600000000      <1>
00017240 00000000FE0000FE00- <1>     db  000h, 000h, 000h, 000h, 0feh, 000h, 000h, 0feh, 000h, 000h, 0feh, 000h, 000h, 000h, 000h, 000h
00017249 00FE0000000000      <1>
00017250 0000000018187E1818- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 07eh, 018h, 018h, 000h, 000h, 0ffh, 000h, 000h, 000h, 000h
00017259 0000FF00000000      <1>
00017260 00000030180C060C18- <1>     db  000h, 000h, 000h, 030h, 018h, 00ch, 006h, 00ch, 018h, 030h, 000h, 07eh, 000h, 000h, 000h, 000h
00017269 30007E00000000      <1>
00017270 0000000C1830603018- <1>     db  000h, 000h, 000h, 00ch, 018h, 030h, 060h, 030h, 018h, 00ch, 000h, 07eh, 000h, 000h, 000h, 000h
00017279 0C007E00000000      <1>
00017280 00000E1B1B18181818- <1>     db  000h, 000h, 00eh, 01bh, 01bh, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h
00017289 18181818181818      <1>
00017290 1818181818181818D8- <1>     db  018h, 018h, 018h, 018h, 018h, 018h, 018h, 018h, 0d8h, 0d8h, 0d8h, 070h, 000h, 000h, 000h, 000h
00017299 D8D87000000000      <1>
000172A0 000000001818007E00- <1>     db  000h, 000h, 000h, 000h, 018h, 018h, 000h, 07eh, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 000h
000172A9 18180000000000      <1>
000172B0 000000000076DC0076- <1>     db  000h, 000h, 000h, 000h, 000h, 076h, 0dch, 000h, 076h, 0dch, 000h, 000h, 000h, 000h, 000h, 000h
000172B9 DC000000000000      <1>
000172C0 00386C6C3800000000- <1>     db  000h, 038h, 06ch, 06ch, 038h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000172C9 00000000000000      <1>
000172D0 000000000000001818- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000172D9 00000000000000      <1>
000172E0 000000000000000018- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 018h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
000172E9 00000000000000      <1>
000172F0 000F0C0C0C0C0CEC6C- <1>     db  000h, 00fh, 00ch, 00ch, 00ch, 00ch, 00ch, 0ech, 06ch, 06ch, 03ch, 01ch, 000h, 000h, 000h, 000h
000172F9 6C3C1C00000000      <1>
00017300 00D86C6C6C6C6C0000- <1>     db  000h, 0d8h, 06ch, 06ch, 06ch, 06ch, 06ch, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017309 00000000000000      <1>
00017310 0070D83060C8F80000- <1>     db  000h, 070h, 0d8h, 030h, 060h, 0c8h, 0f8h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017319 00000000000000      <1>
00017320 000000007C7C7C7C7C- <1>     db  000h, 000h, 000h, 000h, 07ch, 07ch, 07ch, 07ch, 07ch, 07ch, 07ch, 000h, 000h, 000h, 000h, 000h
00017329 7C7C0000000000      <1>
00017330 000000000000000000- <1>     db  000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h
00017339 00000000000000      <1>
                             <1>
                             <1> ; 01/01/2021 (TRDOS 386 v2.0.3)
                             <1>
                             <1> %if 0
                             <1>
                             <1> vgafont14alt:
                             <1>     db  01dh, 000h, 000h, 000h, 000h, 024h, 066h, 0ffh, 066h, 024h, 000h, 000h, 000h, 000h, 000h, 022h
                             <1>     db  000h, 063h, 063h, 063h, 022h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 02bh, 000h
                             <1>     db  000h, 000h, 018h, 018h, 018h, 0ffh, 018h, 018h, 018h, 000h, 000h, 000h, 000h, 02dh, 000h, 000h
                             <1>     db  000h, 000h, 000h, 000h, 0ffh, 000h, 000h, 000h, 000h, 000h, 000h, 000h, 04dh, 000h, 000h, 0c3h
                             <1>     db  0e7h, 0ffh, 0dbh, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 000h, 000h, 000h, 054h, 000h, 000h, 0ffh, 0dbh
                             <1>     db  099h, 018h, 018h, 018h, 018h, 018h, 03ch, 000h, 000h, 000h, 056h, 000h, 000h, 0c3h, 0c3h, 0c3h
                             <1>     db  0c3h, 0c3h, 0c3h, 066h, 03ch, 018h, 000h, 000h, 000h, 057h, 000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h
                             <1>     db  0dbh, 0dbh, 0ffh, 066h, 066h, 000h, 000h, 000h, 058h, 000h, 000h, 0c3h, 0c3h, 066h, 03ch, 018h
                             <1>     db  03ch, 066h, 0c3h, 0c3h, 000h, 000h, 000h, 059h, 000h, 000h, 0c3h, 0c3h, 066h, 03ch, 018h
                             <1>     db  018h, 018h, 03ch, 000h, 000h, 000h, 05ah, 000h, 0ffh, 0c3h, 086h, 00ch, 018h, 030h, 061h
                             <1>     db  0c3h, 0ffh, 000h, 000h, 000h, 06dh, 000h, 000h, 000h, 000h, 000h, 0e6h, 0ffh, 0dbh, 0dbh, 0dbh
                             <1>     db  0dbh, 000h, 000h, 000h, 076h, 000h, 000h, 000h, 000h, 000h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h
                             <1>     db  000h, 000h, 000h, 077h, 000h, 000h, 000h, 000h, 000h, 0c3h, 0c3h, 0dbh, 0dbh, 0ffh, 066h, 000h
                             <1>     db  000h, 000h, 091h, 000h, 000h, 000h, 000h, 06eh, 03bh, 01bh, 07eh, 0d8h, 0dch, 077h, 000h, 000h
                             <1>     db  000h, 09bh, 000h, 018h, 018h, 07eh, 0c3h, 0c0h, 0c0h, 0c3h, 07eh, 018h, 018h, 000h, 000h, 000h
                             <1>     db  09dh, 000h, 000h, 0c3h, 066h, 03ch, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 000h, 000h, 000h, 09eh
                             <1>     db  000h, 0fch, 066h, 066h, 07ch, 062h, 066h, 06fh, 066h, 066h, 0f3h, 000h, 000h, 000h, 0f1h, 000h
                             <1>     db  000h, 018h, 018h, 018h, 0ffh, 018h, 018h, 018h, 000h, 0ffh, 000h, 000h, 000h, 0f6h, 000h, 000h
                             <1>     db  018h, 018h, 000h, 000h, 0ffh, 000h, 000h, 018h, 018h, 000h, 000h, 000h, 000h
                             <1> vgafont16alt:
                             <1>     db  01dh, 000h, 000h, 000h, 000h, 000h, 024h, 066h, 0ffh, 066h, 024h, 000h, 000h, 000h, 000h, 000h
                             <1>     db  000h, 030h, 000h, 000h, 03ch, 066h, 0c3h, 0c3h, 0dbh, 0dbh, 0c3h, 0c3h, 066h, 03ch, 000h, 000h
                             <1>     db  000h, 000h, 04dh, 000h, 000h, 0c3h, 0e7h, 0ffh, 0ffh, 0dbh, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 000h
                             <1>     db  000h, 000h, 000h, 054h, 000h, 000h, 0ffh, 0dbh, 099h, 018h, 018h, 018h, 018h, 018h, 018h, 03ch
                             <1>     db  000h, 000h, 000h, 000h, 056h, 000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 0c3h, 066h, 03ch
                             <1>     db  018h, 000h, 000h, 000h, 000h, 057h, 000h, 000h, 0c3h, 0c3h, 0c3h, 0c3h, 0dbh, 0dbh, 0dbh, 0ffh
                             <1>     db  066h, 066h, 000h, 000h, 000h, 000h, 058h, 000h, 000h, 0c3h, 0c3h, 066h, 03ch, 018h, 018h, 03ch
                             <1>     db  066h, 0c3h, 0c3h, 000h, 000h, 000h, 000h, 059h, 000h, 000h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h
                             <1>     db  018h, 018h, 018h, 03ch, 000h, 000h, 000h, 000h, 05ah, 000h, 000h, 0ffh, 0c3h, 086h, 00ch, 018h
                             <1>     db  030h, 060h, 0c1h, 0c3h, 0ffh, 000h, 000h, 000h, 000h, 06dh, 000h, 000h, 000h, 000h, 000h, 0e6h
                             <1>     db  0ffh, 0dbh, 0dbh, 0dbh, 0dbh, 0dbh, 000h, 000h, 000h, 076h, 000h, 000h, 000h, 000h, 000h, 000h
                             <1>     db  0c3h, 0c3h, 0c3h, 0c3h, 066h, 03ch, 018h, 000h, 000h, 000h, 000h, 077h, 000h, 000h, 000h, 000h
```

```
                                <1>     db  000h, 0c3h, 0c3h, 0c3h, 0dbh, 0dbh, 0ffh, 066h, 000h, 000h, 000h, 000h, 078h, 000h, 000h, 000h
                                <1>     db  000h, 000h, 0c3h, 066h, 03ch, 018h, 03ch, 066h, 0c3h, 000h, 000h, 000h, 000h, 091h, 000h, 000h
                                <1>     db  000h, 000h, 000h, 06eh, 03bh, 01bh, 07eh, 0d8h, 0dch, 077h, 000h, 000h, 000h, 000h, 09bh, 000h
                                <1>     db  018h, 018h, 07eh, 0c3h, 0c0h, 0c0h, 0c0h, 0c3h, 07eh, 018h, 018h, 000h, 000h, 000h, 000h, 09dh
                                <1>     db  000h, 000h, 0c3h, 066h, 03ch, 018h, 0ffh, 018h, 0ffh, 018h, 018h, 018h, 000h, 000h, 000h, 000h
                                <1>     db  09eh, 000h, 0fch, 066h, 066h, 07ch, 062h, 066h, 06fh, 066h, 066h, 066h, 0f3h, 000h, 000h, 000h
                                <1>     db  000h, 0abh, 000h, 0c0h, 0c0h, 0c2h, 0c6h, 0cch, 018h, 030h, 060h, 0ceh, 09bh, 006h, 00ch, 01fh
                                <1>     db  000h, 000h, 0ach, 000h, 0c0h, 0c0h, 0c2h, 0c6h, 0cch, 018h, 030h, 066h, 0ceh, 096h, 03eh, 006h
                                <1>     db  006h, 000h, 000h, 000h
                                <1>
                                <1> %endif

                                    ; 20/11/2020
                                    vbe2_bochs_vbios:
                                    ;   db "BOCHS/QEMU"
00017340 424F4348532F51454D-        db "BOCHS/QEMU/VIRTUALBOX"  ; 26/11/2020
00017349 552F5649525455414C-
00017352 424F58

                                    ;vbe_vnumber equ vbe2_bochs_vbios + 28 ; "3" or "2"
                                    ; 26/11/2020
                                    vbe_vnumber equ vbe2_bochs_vbios + 30 ; "3" or "2"
                                    ; 15/11/2020
                                    vesa_vbe3_bios_msg:
                                        ;db " VESA VBE version 3 Video BIOS ..."
00017355 205645534120564245-        db " VESA VBE3 Video BIOS ..." ; 26/11/2020
0001735E 3320566964656F2042-
00017367 494F53202E2E2E

0001736E 0D0A0D0A00                 db 0Dh, 0Ah, 0Dh, 0Ah, 0

                                    ; 28/02/2021
00017373 18                         truecolor: db 24

                                    align 2

                                    ; EPOCH Variables
                                    ; 13/04/2015 - Retro UNIX 386 v1 Beginning
                                    ; 09/04/2013 epoch variables
                                    ; Retro UNIX 8086 v1 Prototype: UNIXCOPY.ASM, 10/03/2013
                                    ;
00017374 B207                       year:      dw 1970
00017376 0100                       month:     dw 1
00017378 0100                       day:       dw 1
0001737A 0000                       hour:      dw 0
0001737C 0000                       minute: dw 0
0001737E 0000                       second: dw 0

                                    DMonth:
00017380 0000                           dw 0
00017382 1F00                           dw 31
00017384 3B00                           dw 59
00017386 5A00                           dw 90
00017388 7800                           dw 120
0001738A 9700                           dw 151
0001738C B500                           dw 181
0001738E D400                           dw 212
00017390 F300                           dw 243
00017392 1101                           dw 273
00017394 3001                           dw 304
00017396 4E01                           dw 334

                                    ; 15/11/2020
00017398 00                         db  0
                                    kernel_version_msg: ; 17/04/2021
                                    ;;;;;;db "TRDOS (386) Kernel v2.0.4 by Erdogan Tan"
                                    ;;;;;db    "TRDOS (386) Kernel v2.0.5 by Erdogan Tan" ; 11/08/2022
                                    ;;;;db     "TRDOS (386) Kernel v2.0.6 by Erdogan Tan" ; 29/08/2023
                                    ;;;db      "TRDOS (386) Kernel v2.0.7 by Erdogan Tan" ; 20/10/2023
                                    ;;db"TRDOS (386) Kernel v2.0.8 by Erdogan Tan" ; 16/05/2024
                                    ;db "TRDOS (386) Kernel v2.0.9 by Erdogan Tan" ; 20/08/2024
00017399 5452444F5320283338-        db  "TRDOS (386) Kernel v2.0.10 by Erdogan Tan" ; 11/01/2025
000173A2 3629204B65726E656C-
000173AB 2076322E302E313020-
000173B4 6279204572646F6761-
000173BD 6E2054616E
000173C2 00                         db  0

                                    ; 20/02/2017
                                    KERNELFSIZE  equ $  ; 04/07/2016

                                    bss_start:

                                    ABSOLUTE bss_start

000173C3 ??????????                 alignb 8 ; 25/12/2016

                                        ; 15/04/2016
                                        ; TRDOS 386 (TRDOS v2.0)
                                        ;       80 interrupts
                                        ; 11/03/2015
                                        ; Interrupt Descriptor Table (20/08/2014)
                                    idt:
                                        ;resb  64*8 ; INT 0 to INT 3Fh
                                        ; 15/04/2016
000173C8 <res 280h>                     resb   80*8 ; INT 0 to INT 4Fh

                                    idt_end:

                                    ;alignb 4

                                    task_state_segment:
                                        ; 24/03/2015
00017648 ????                       tss.link:   resw 1
0001764A ????                               resw 1
                                    ; tss offset 4
0001764C ????????                   tss.esp0:   resd 1
00017650 ????                       tss.ss0:    resw 1
00017652 ????                               resw 1
00017654 ????????                   tss.esp1:   resd 1
00017658 ????                       tss.ss1:    resw 1
0001765A ????                               resw 1
0001765C ????????                   tss.esp2:   resd 1
00017660 ????                       tss.ss2:    resw 1
00017662 ????                               resw 1
                                    ; tss offset 28
00017664 ????????                   tss.CR3:    resd 1
00017668 ????????                   tss.eip:    resd 1
0001766C ????????                   tss.eflags: resd 1
                                    ; tss offset 40
00017670 ????????                   tss.eax:    resd 1
00017674 ????????                   tss.ecx:    resd 1
00017678 ????????                   tss.edx:    resd 1
0001767C ????????                   tss.ebx:    resd 1
00017680 ????????                   tss.esp:    resd 1
00017684 ????????                   tss.ebp:    resd 1
```

```
00017688 ????????                tss.esi:    resd 1
0001768C ????????                tss.edi:    resd 1
                                 ; tss offset 72
00017690 ????                    tss.ES:     resw 1
00017692 ????                            resw 1
00017694 ????                    tss.CS:        resw 1
00017696 ????                            resw 1
00017698 ????                    tss.SS:        resw 1
0001769A ????                            resw 1
0001769C ????                    tss.DS:        resw 1
0001769E ????                            resw 1
000176A0 ????                    tss.FS:        resw 1
000176A2 ????                            resw 1
000176A4 ????                    tss.GS:        resw 1
000176A6 ????                            resw 1
000176A8 ????                    tss.LDTR:   resw 1
000176AA ????                            resw 1
                                 ; tss offset 100
000176AC ????                            resw 1
000176AE ????                    tss.IOPB:   resw 1
                                 ; tss offset 104
                                 tss_end:

000176B0 ????????                k_page_dir:  resd 1 ; Kernel's (System) Page Directory address
                                                ; (Physical address = Virtual address)
000176B4 ????????                memory_size: resd 1 ; memory size in pages
000176B8 ????????                free_pages:  resd 1 ; number of free pages
000176BC ????????                next_page:   resd 1 ; offset value in M.A.T. for
                                                ; first free page search
000176C0 ????????                last_page:   resd 1 ; offset value in M.A.T. which
                                                ; next free page search will be
                                                ; stopped after it. (end of M.A.T.)
000176C4 ????????                first_page:  resd 1 ; offset value in M.A.T. which
                                                ; first free page search
                                                ; will be started on it. (for user)
000176C8 ????????                mat_size:    resd 1 ; Memory Allocation Table size in pages

                                 ; 20/11/2020
                                 ;vbe2bios:    resw 1 ; VBE2 video bios ID (bochs/qemu)
                                 ;               ; (0B0C4h or 0B0C5h for bochs/plex86 vgabios)

                                 ; 02/09/2014 (Retro UNIX 386 v1)
                                 ; 04/12/2013 (Retro UNIX 8086 v1)
000176CC ????                    CRT_START:   resw 1          ; starting address in regen buffer
                                                     ; NOTE: active page only
000176CE <res 10h>               CURSOR_POSN: resw 8 ; cursor positions for video pages
                                 ACTIVE_PAGE:
000176DE ??                      ptty:           resb 1 ; current tty
                                 ; 01/07/2015 - 29/01/2016
000176DF ??                      ccolor:         resb 1 ; current color attribute
                                 ; 26/10/2015
                                 ; 07/09/2014
000176E0 <res 14h>               ttychr:      resw ntty+2 ; Character buffer (multiscreen)

                                 ; 18/05/2015 (03/06/2013 - Retro UNIX 8086 v1 feature only!)
000176F4 ????????                p_time:      resd 1     ; present time (for systime & sysmdate)

                                 ; 18/05/2015 (16/08/2013 - Retro UNIX 8086 v1 feature only !)
                                 ; (open mode locks for pseudo TTYs)
                                 ; [ major tty locks (return error in any conflicts) ]
000176F8 <res 14h>               ttyl:        resw ntty+2 ; opening locks for TTYs.

                                 ; 15/04/2015 (Retro UNIX 386 v1)
                                 ; 22/09/2013 (Retro UNIX 8086 v1)
0001770C <res Ah>                wlist:       resb ntty+2 ; wait channel list (0 to 9 for TTYs)
                                 ; 15/04/2015 (Retro UNIX 386 v1)
                                 ;; 12/07/2014 -> sp_init set comm. parameters as 0E3h
                                 ;; 0 means serial port is not available
                                 ;;comprm: ; 25/06/2014
00017716 ??                      com1p:       resb 1  ;;0E3h
00017717 ??                      com2p:       resb 1  ;;0E3h

                                 ; 17/11/2015
                                 ; request for response (from the terminal)
00017718 ????                    req_resp:    resw 1
                                 ; 07/11/2015
0001771A ??                      ccomport:    resb 1 ; current COM (serial) port
                                                ; (0= COM1, 1= COM2)
                                 ; 09/11/2015
0001771B ??                      comqr:          resb 1 ; 'query or response' sign (u9.s, 'sndc')
                                 ; 07/11/2015
0001771C ????                    rchar:          resw 1 ; last received char for COM 1 and COM 2
0001771E ????                    schar:          resw 1 ; last sent char for COM 1 and COM 2

                                 ; 22/08/2014 (RTC)
                                 ; (Packed BCD)
00017720 ??                      time_seconds: resb 1
00017721 ??                      time_minutes: resb 1
00017722 ??                      time_hours:   resb 1
00017723 ??                      date_wday:    resb 1
00017724 ??                      date_day:     resb 1
00017725 ??                      date_month:   resb 1
00017726 ??                      date_year:    resb 1
00017727 ??                      date_century: resb 1

                                 ; 24/01/2016
00017728 ????????                RTC_LH:           resd 1
0001772C ??                      RTC_WAIT_FLAG: resb 1
0001772D ??                      USER_FLAG:     resb 1
                                 ; 19/05/2016
                                 ;RTC_second:
0001772E ??                      RTC_2Hz:       resb 1 ;  from 2Hz interrupt to 1Hz timer event function

                                 %include 'diskbss.s'      ; UNINITIALIZED DISK (BIOS) DATA
                             <1> ; ****************************************************************************
                             <1> ; TRDOS386.ASM (TRDOS 386 Kernel) - v2.0.5 - diskbss.s
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Last Update: 06/08/2022 (Previous: 24/01/2016)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Beginning: 24/01/2016
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Assembler: NASM version 2.15 (trdos386.s)
                             <1> ; ----------------------------------------------------------------------------
                             <1> ; Turkish Rational DOS
                             <1> ; Operating System Project v2.0 by ERDOGAN TAN (Beginning: 04/01/2016)
                             <1> ;
                             <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                             <1> ; diskbss.inc (10/07/2015)
                             <1> ;
                             <1> ; Derived from 'IBM PC-XT-286' BIOS source code (1986)
                             <1> ; ****************************************************************************
                             <1>
                             <1> ; Retro UNIX 386 v1 Kernel - DISKBSS.INC
                             <1> ; Last Modification: 10/07/2015
```

```
                                <1> ;   (Unitialized Disk Parameters Data section for 'DISKIO.INC')
                                <1>
0001772F ??                     <1> alignb 2
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   TIMER DATA AREA               :
                                <1> ;----------------------------------------
                                <1>
                                <1> TIMER_LH:   ; 16/02/2015
00017730 ????                   <1> TIMER_LOW:      resw      1               ; LOW WORD OF TIMER COUNT
00017732 ????                   <1> TIMER_HIGH:     resw      1               ; HIGH WORD OF TIMER COUNT
00017734 ??                     <1> TIMER_OFL:      resb      1               ; TIMER HAS ROLLED OVER SINCE LAST READ
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   DISKETTE DATA AREAS           :
                                <1> ;----------------------------------------
                                <1>
00017735 ??                     <1> SEEK_STATUS:       resb   1
00017736 ??                     <1> MOTOR_STATUS:      resb   1
00017737 ??                     <1> MOTOR_COUNT:       resb   1
00017738 ??                     <1> DSKETTE_STATUS:    resb   1
00017739 ?????????????          <1> NEC_STATUS: resb   7
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   ADDITIONAL MEDIA DATA         :
                                <1> ;----------------------------------------
                                <1>
00017740 ??                     <1> LASTRATE:   resb   1
00017741 ??                     <1> HF_STATUS:  resb   1
                                <1> ; 06/08/2022
                                <1> ;HF_ERROR:  resb   1
00017742 ??                     <1> HF_INT_FLAG:       resb   1
                                <1> ; 06/08/2022
                                <1> ;HF_CNTRL:  resb   1
                                <1> ;DSK_STATE: resb   4
                                <1> ; 06/08/2022
00017743 ????                   <1> DSK_STATE:  resb   2
00017745 ????                   <1> DSK_TRK:    resb   2
                                <1>
                                <1> ;----------------------------------------
                                <1> ;   FIXED DISK DATA AREAS         :
                                <1> ;----------------------------------------
                                <1>
00017747 ??                     <1> DISK_STATUS1:      resb   1              ; FIXED DISK STATUS
00017748 ??                     <1> HF_NUM:            resb   1              ; COUNT OF FIXED DISK DRIVES
00017749 ??                     <1> CONTROL_BYTE:      resb   1              ; HEAD CONTROL BYTE
                                <1> ;@PORT_OFF  resb   1              ; RESERVED (PORT OFFSET)
                                <1> ;port1_off  resb   1              ; Hard disk controller 1 - port offset
                                <1> ;port2_off  resb   1              ; Hard disk controller 2 - port offset
                                <1>
0001774A ????                   <1> alignb 4
                                <1>
                                <1> ;HF_TBL_VEC:       resd   1              ; Primary master disk param. tbl. pointer
                                <1> ;HF1_TBL_VEC:      resd   1              ; Primary slave disk param. tbl. pointer
                                <1> HF_TBL_VEC: ; 22/12/2014
0001774C ????????               <1> HDPM_TBL_VEC:      resd   1              ; Primary master disk param. tbl. pointer
00017750 ????????               <1> HDPS_TBL_VEC:      resd   1              ; Primary slave disk param. tbl. pointer
00017754 ????????               <1> HDSM_TBL_VEC:      resd   1              ; Secondary master disk param. tbl. pointer
00017758 ????????               <1> HDSS_TBL_VEC:      resd   1              ; Secondary slave disk param. tbl. pointer
                                <1>
                                <1> ; 03/01/2015
0001775C ??                     <1> LBAMode:           resb   1
                                <1>
                                <1> ; ******************************************************************************

                                    ;;; Real Mode Data (10/07/2015 - BSS)

                                    ;alignb 2

                                    ; 10/01/2016
                                    %include 'trdoskx.s'      ; UNINITIALIZED KERNEL (Logical Drive & FS) DATA
                                <1> ; ******************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.10) - UNINITIALIZED DATA : trdoskx.s
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Last Update: 10/02/2026 (Previous: 01/09/2024 - Kernel v2.0.9)
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Beginning: 04/01/2016
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.11 (trdos386.s)
                                <1> ; ------------------------------------------------------------------------------
                                <1> ; Derived from TRDOS Operating System v1.0 (8086) source code by Erdogan Tan
                                <1> ; TRDOS2.ASM (09/11/2011)
                                <1> ; ******************************************************************************
                                <1> ; DRV_INIT.ASM [26/09/2009] Last Update: 07/08/2011
                                <1> ; MAINPROG.ASM [17/01/2004] Last Update: 09/11/2011
                                <1> ; DIR.ASM      [17/01/2004] Last Update: 09/10/2011
                                <1> ; CMD_INTR.ASM [29/01/2005] Last update: 09/11/2011
                                <1> ; DRV_FAT.ASM  [07/07/2009] Last update: 21/08/2011
                                <1>
0001775D ??????                 <1> alignb 4
                                <1>
                                <1> ; 10/02/2026 - TRDOS 386 v2.0.10
00017760 ????????               <1> cr3prev: resd 1 ; for saving and restoring previous (user's) cr3
                                <1>            ; (page directory address) -video.s-
                                <1>
                                <1> ; MAINPROG.ASM
00017764 ????????               <1> MainProgCfg_FileSize:   resd 1 ; 14/04/2016
00017768 ????????               <1> MainProgCfg_LineOffset: resd 1 ; 14/04/2016
                                <1>
0001776C ????????               <1> Current_VolSerial: resd 1
                                <1>
00017770 ????????               <1> Current_Dir_FCluster: resd 1
                                <1>
00017774 ??                     <1> Current_Dir_Level: resb 1
00017775 ??                     <1> Current_FATType: resb 1
00017776 ??                     <1> Current_Drv: resb 1
00017777 ??                     <1> Current_Dir_Drv:   resb 1 ; '?'
00017778 ??                     <1>                    resb 1 ; ':'
00017779 ??                     <1> Current_Dir_Root:  resb 1 ; '/'
0001777A <res 5Ah>              <1> Current_Directory: resb 90
000177D4 ??                     <1> End_Of_Current_Dir_Str: resb 1
000177D5 ??                     <1> Current_Dir_StrLen: resb 1
                                <1>
000177D6 ??                     <1> CursorColumn:      resb 1
000177D7 ??                     <1> CmdArgStart:    resb 1
                                <1>
                                <1> ; 03/02/2016
000177D8 <res 4Eh>              <1> Remark:            resb 78
                                <1>
00017826 <res 50h>              <1> CommandBuffer:     resb 80
                                <1>
                                <1> ;TextBuffer:       ;resb 256
                                <1> ; 04/07/2025
00017876 <res 80h>              <1> TextBuffer: resb 128
```

```
000178F6 <res 80h>          <1> RunPathBuffer:     resb 128
                            <1>
                            <1> MasterBootBuff:
00017976 <res 1BEh>         <1> MasterBootCode: resb 1BEh
00017B34 <res 40h>          <1> PartitionTable: resb 64
00017B74 ????               <1> MBIDCode: resw 1
                            <1>
                            <1> PTable_Buffer:
00017B76 <res 40h>          <1> PTable_hd0: resb 64
00017BB6 <res 40h>          <1> PTable_hd1: resb 64
00017BF6 <res 40h>          <1> PTable_hd2: resb 64
00017C36 <res 40h>          <1> PTable_hd3: resb 64
                            <1> ; 15/07/2020
                            <1> ;PTable_ep0: resb 64
                            <1> ;PTable_ep1: resb 64
                            <1> ;PTable_ep2: resb 64
                            <1> ;PTable_ep3: resb 64
                            <1>
                            <1> ; 22/05/2024
00017C76 ????????           <1> HD_LBA_yes: resd 1
                            <1> ; 13/08/2020
00017C7A ??                 <1> scount: resb 1 ; 16/05/2016 (diskio.s, 'int33h:')
                            <1> ; 22/05/2024
                            <1>     ;resb 1
                            <1>     ;resb 1
                            <1>     ;resb 1
                            <1> ;HD_LBA_yes: resb 1
00017C7B ??                 <1> PP_Counter: resb 1
00017C7C ??                 <1> EP_Counter: resb 1
                            <1> ; 13/08/2020
00017C7D ??                 <1> LD_Counter: resb 1
                            <1>
                            <1> ; 30/08/2020
00017C7E ????????           <1> MBR_EP_StartSector: resd 1
                            <1>             ; Extd partition start sector as in MBR
00017C82 ????????           <1> EP_StartSector: resd 1 ; next extd partition start sector
                            <1>     ; 15/07/2020
                            <1>                 ;resd 1
                            <1>
                            <1> ; 20/07/2020
00017C86 <res 200h>         <1> DOSBootSectorBuff: resb 512
                            <1> ; 15/07/2020
                            <1> ;DOSBootSectorBuff: resb 446 ; 1BEh
                            <1> ;MiniPartitionTable: resb 64 ;  40h
                            <1> ;MiniPartitionMagic: resw  1 ;  02h
                            <1>
                            <1> FAT_BuffDescriptor:
00017E86 ????????           <1> FAT_CurrentCluster: resd 1
00017E8A ??                 <1> FAT_BuffValidData: resb 1
00017E8B ??                 <1> FAT_BuffDrvName: resb 1
00017E8C ????               <1> FAT_BuffOffset: resw 1
00017E8E ????????           <1> FAT_BuffSector: resd 1
                            <1>
00017E92 ????????           <1> FAT_ClusterCounter: resd 1
00017E96 ????????           <1> LastCluster: resd 1
                            <1>
                            <1> ; 16/05/2016
                            <1> ;; 18/03/2016 (TRDOS v2.0)
                            <1> ;ClusterBuffer_Valid: resb 1
                            <1>
                            <1> ; 29/07/2022
                            <1> ;resb 1
                            <1> ; 02/12/2023
00017E9A ??                 <1> P_TIMER: resb 1 ; diskette change check (2 seconds)
                            <1>
                            <1> Dir_BuffDescriptor:
00017E9B ??                 <1> DirBuff_DRV: resb 1
00017E9C ??                 <1> DirBuff_FATType: resb 1
00017E9D ??                 <1> DirBuff_ValidData: resb 1
00017E9E ????               <1> DirBuff_CurrentEntry: resw 1
00017EA0 ????               <1> DirBuff_LastEntry: resw 1
00017EA2 ????????           <1> DirBuff_Cluster: resd 1
00017EA6 ????               <1> DirBuffer_Size: resw 1
                            <1> ;DirBuff_EntryCounter: resw 1
                            <1>
                            <1> ; 01/02/2016
                            <1> ; these are on (real mode) segment 8000h and later
                            <1> ; FAT_Buffer:      resb 1536 ; 3 sectors
                            <1> ; Dir_Buffer:      resb 512*32
                            <1> ; Logical_DOSDisks:  resb 6656 ; 26 * 256 bytes
                            <1>
                            <1> ; 18/01/2016
                            <1>
00017EA8 ????????           <1> FreeClusterCount: resd 1
                            <1>
00017EAC ????????           <1> VolSize_Unit1:   resd 1
00017EB0 ????????           <1> VolSize_Unit2:   resd 1
                            <1>
00017EB4 ????????           <1> Vol_Tot_Sec_Str_Start:        resd 1
00017EB8 <res Ah>           <1> Vol_Tot_Sec_Str:       resb 10
00017EC2 ??                 <1> Vol_Tot_Sec_Str_End:          resb 1
00017EC3 ??                 <1> resb 1
00017EC4 ????????           <1> Vol_Free_Sectors_Str_Start: resd 1
00017EC8 <res Ah>           <1> Vol_Free_Sectors_Str:         resb 10
00017ED2 ??                 <1> Vol_Free_Sectors_Str_End:   resb 1
                            <1>
                            <1> ; 10/02/2016
00017ED3 ??                 <1> RUN_CDRV: resb 1 ; CMD_INTR.ASM  ; 09/11/2011
                            <1>
                            <1> ; 24/01/2016
00017ED4 <res 80h>          <1> PATH_Array:     resb 128 ; DIR.ASM ; 09/10/2011
                            <1> ; 06/02/2016
00017F54 ????????           <1> CCD_DriveDT:       resd 1 ; DIR.ASM ; (word)
00017F58 ??                 <1> CCD_Level:  resb 1 ; DIR.ASM
00017F59 ??                 <1> Last_Dir_Level:    resb 1 ; DIR.ASM
                            <1> ;
00017F5A ????               <1> CDLF_AttributesMask: resw 1 ; DIR.ASM
00017F5C ????????           <1> CDLF_FNAddress:    resd 1 ; DIR.ASM (word)
00017F60 ????               <1> CDLF_DEType:       resw 1 ; DIR.ASM
                            <1> ;
00017F62 ??                 <1> CD_COMMAND: resb 1 ; DIR.ASM
                            <1>
00017F63 ??                 <1> alignb 4
                            <1>
                            <1> ; 29/01/2016
00017F64 ??                 <1> Program_Exit:      resb 1 ; CMD_INTR.ASM  ; 09/11/2011
                            <1>
                            <1> ;alignb 4
                            <1> ; 23/02/2016
00017F65 ??                 <1> disk_rw_op: resb 1 ; 0 = disk read, 1 = disk write
                            <1> ;disk_rw_spt:      resb 1 ; sectors per track (<= 63) /// (<256)
                            <1> ; 31/01/2016
00017F66 ??                 <1> retry_count:       resb 1 ; DISK_IO.ASM ; 20/07/2011 (CHS_RetryCount)
```

```
00017F67 ??                <1> disk_rw_err:       resb 1 ; DISK_IO.ASM ; (Disk_IO_err_code)
00017F68 ????????          <1> sector_count:      resd 1 ; DISK_IO.ASM ; (Disk_RW_SectorCount)
                           <1>
                           <1> ; 06/02/2016 (long name)
00017F6C ????              <1> FDE_AttrMask:         resw 1 ; DIR.ASM
00017F6E ????              <1> AmbiguousFileName: resw 1 ; DIR.ASM
00017F70 ??                <1> PreviousAttr:         resb 1 ; DIR.ASM
                           <1> ;
00017F71 ??                <1> LongNameFound:   resb 1     ; DIR.ASM
00017F72 ??                <1> LFN_EntryLength: resb 1   ; DIR.ASM
00017F73 ??                <1> LFN_CheckSum:    resb 1   ; DIR.ASM
00017F74 <res 84h>         <1> LongFileName:    resb 132 ; DIR.ASM
                           <1>
                           <1> ;PATH_Array_Ptr: resw 1 ; DIR.ASM
00017FF8 ??                <1> PATH_CDLevel:       resb 1 ; DIR.ASM
00017FF9 ??                <1> PATH_Level:  resb 1 ; DIR.ASM
                           <1>
                           <1> ; 07/02/2016
00017FFA <res Dh>          <1> Dir_File_Name:     resb 13 ; DIR.ASM ; 09/10/2011
                           <1>
                           <1> ; 10/02/2016
00018007 <res Dh>          <1> Dir_Entry_Name:    resb 13 ; DIR.ASM
                           <1>
                           <1> alignb 2
                           <1>
00018014 ????              <1> AttributesMask: resw 1 ; CMD_INTR.ASM ; 09/11/2011
                           <1>
                           <1> ; 10/02/2016 (128 bytes -> 126 bytes)
                           <1> ; 08/02/2016
                           <1> ;FFF Structure (128 bytes) ; DIR.ASM ; 09/10/2011
00018016 ??                <1> FindFile_Drv:               resb 1
00018017 <res 41h>         <1> FindFile_Directory:         resb 65
00018058 <res Dh>          <1> FindFile_Name:              resb 13
                           <1> FindFile_LongNameEntryLength:
00018065 ??                <1> FindFile_LongNameYes:       resb 1 ; Sign for longname procedures
                           <1> ;Above 80 bytes form
                           <1> ;TR-DOS Source/Destination File FullName Format/Structure
00018066 ????              <1> FindFile_AttributesMask:  resw 1
00018068 <res 20h>         <1> FindFile_DirEntry:   resb 32
00018088 ????????          <1> FindFile_DirFirstCluster: resd 1
0001808C ????????          <1> FindFile_DirCluster:        resd 1
00018090 ????              <1> FindFile_DirEntryNumber:  resw 1
00018092 ????              <1> FindFile_MatchCounter:      resw 1
00018094 ????              <1> FindFile_Reserved:   resw 1 ; 06/03/2016
                           <1>
00018096 ????????          <1> First_Path_Pos: resd 1    ; DIR.ASM ; 09/10/2011
0001809A ????????          <1> Last_Slash_Pos: resd 1    ; DIR.ASM
                           <1>
                           <1> ; 10/02/2016
0001809E ????              <1> File_Count:     resw 1    ; DIR.ASM ; 09/10/2011
000180A0 ????              <1> Dir_Count:      resw 1
000180A2 ????????          <1> Total_FSize:    resd 1
000180A6 ????????          <1> TFS_Dec_Begin:  resd 1
000180AA <res Ah>          <1>                 resb 10
000180B4 ??                <1> TFS_Dec_End:    resb 1
                           <1>
000180B5 ??                <1> PrintDir_RowCounter: resb 1
                           <1>
000180B6 ????              <1> alignb 4
                           <1> ; 15/02/2015 ('show' command variables)
000180B8 ????????          <1> Show_FDT:   resd 1
000180BC ????????          <1> Show_LDDDT: resd 1
000180C0 ????????          <1> Show_Cluster:      resd 1
000180C4 ????????          <1> Show_FileSize:     resd 1
000180C8 ????????          <1> Show_FilePointer: resd 1
000180CC ????              <1> Show_ClusterPointer: resw 1
000180CE ????              <1> Show_ClusterSize: resw 1
000180D0 ??                <1> Show_RowCount:     resb 1
                           <1>
000180D1 ??????            <1> alignb 4
                           <1> ; 21/02/2016
000180D4 ????????          <1> DelFile_FNPointer: resd 1 ; ; CMD_INTR.ASM (word) ; 09/11/2011
                           <1> ; 27/02/2016
                           <1> ; DIR.ASM (09/10/2011)
000180D8 ????????          <1> DelFile_FCluster:  resd 1
000180DC ????              <1> DelFile_EntryCounter:     resw 1
000180DE ??                <1> DelFile_LNEL:             resb 1
                           <1> ; 17/07/2025
                           <1> ;resb 1
                           <1> ; 22/02/2016
000180DF ??                <1> UPDLMDT_CDirLevel: resb 1
000180E0 ????????          <1> UPDLMDT_CDirFCluster:     resd 1
                           <1>
                           <1> ; DIR.ASM
000180E4 ????????          <1> mkdir_DirName_Offset:     resd 1
000180E8 ????????          <1> mkdir_FFCluster:   resd 1
000180EC ????????          <1> mkdir_LastDirCluster:     resd 1
000180F0 ????????          <1> mkdir_FreeSectors: resd 1
000180F4 ????              <1> mkdir_attrib:             resw 1
000180F6 ??                <1> mkdir_SecPerClust: resb 1
000180F7 ??                <1> mkdir_add_new_cluster:    resb 1
000180F8 <res Dh>          <1> mkdir_Name:        resb 13
00018105 ????              <1> resw 1 ; 01/03/2016
                           <1> ; 27/02/2016
00018107 ??                <1> RmDir_MultiClusters:      resb 1
00018108 ????????          <1> RmDir_DirEntryOffset:     resd 1 ; 01/03/2016 (word -> dword)
0001810C ????????          <1> RmDir_ParentDirCluster: resd 1
00018110 ????????          <1> RmDir_DirLastCluster:   resd 1
00018114 ????????          <1> RmDir_PreviousCluster:  resd 1
                           <1>
                           <1> ; 17/07/2025 - TRDOS 386 v2.0.10
00018118 ????????          <1> rmdir_dir_fcluster:       resd 1
0001811C ??                <1> rmdir_drv:         resb 1
                           <1>
0001811D ??????            <1> alignb 4
                           <1> ; DRV_FAT.ASM ; 21/08/2011
00018120 ????????          <1> gffc_next_free_cluster:  resd 1
00018124 ????????          <1> gffc_first_free_cluster: resd 1
00018128 ????????          <1> gffc_last_free_cluster:  resd 1
                           <1>
                           <1> ;29/04/2016
                           <1> Cluster_Index: ; resd 1
                           <1> ; 22/02/2016
0001812C ????????          <1> ClusterValue:      resd 1
                           <1> ; 04/03/2016
00018130 ??                <1> Attributes: resb 1
                           <1> ;;CFS_error:  resb 1 ;; 01/03/2016
00018131 ??                <1> resb 1
00018132 ??                <1> CFS_OPType: resb 1
00018133 ??                <1> CFS_Drv:    resb 1
00018134 ????????          <1> CFS_CC:         resd 1
00018138 ????????          <1> CFS_FAT32FSINFOSEC: resd 1
0001813C ????????          <1> CFS_FAT32FC: resd 1
                           <1>
```

```
                                <1> ; 27/02/2016
                                <1> ;alignb 4
00018140 ????????               <1> glc_prevcluster: resd 1 ; DRV_FAT.ASM (21/08/2011)
                                <1> ; 22/10/2016
00018144 ????????               <1> glc_index:   resd 1 ; Last Cluster Index (22/10/2016)
                                <1>
                                <1> ; DIR.ASM
00018148 ????                   <1> DLN_EntryNumber: resw 1
0001814A ??                     <1> DLN_40h:     resb 1
                                <1> ; 28/02/2016
0001814B ??                     <1> TCC_FATErr:  resb 1 ; DRV_FAT.ASM
                                <1>
                                <1> alignb 4
                                <1> ; DIR.ASM (09/10/2011)
0001814C ????                   <1> LCDE_EntryIndex: resw 1 ; LCDE_EntryOffset
0001814E ????                   <1> LCDE_ClusterSN:  resw 1
00018150 ????????               <1> LCDE_Cluster:       resd 1
00018154 ????????               <1> LCDE_ByteOffset: resd 1
                                <1>
                                <1> ;alignb4
                                <1> ; 06/03/2016 (word -> dword)
                                <1> ; CMD_INTR.ASM (01/08/2010)
00018158 ????????               <1> SourceFilePath:         resd 1
0001815C ????????               <1> DestinationFilePath: resd 1
                                <1>
                                <1> ;alignb 4
                                <1> ; 06/03/2016
                                <1> ; FILE.ASM (09/10/2011)
                                <1> ;Source File Structure (same with 'Find File' Structure)
00018160 ??                     <1> SourceFile_Drv:                  resb 1
00018161 <res 41h>              <1> SourceFile_Directory:            resb 65
000181A2 <res Dh>               <1> SourceFile_Name:          resb 13
                                <1> SourceFile_LongNameEntryLength:
000181AF ??                     <1> SourceFile_LongNameYes:          resb 1 ; Sign for longname procedures
                                <1> ;Above 80 bytes
                                <1> ;is TR-DOS Source File FullName Format/Structure
000181B0 ????                   <1> SourceFile_AttributesMask: resw 1
000181B2 <res 20h>              <1> SourceFile_DirEntry:             resb 32
000181D2 ????????               <1> SourceFile_DirFirstCluster:      resd 1
000181D6 ????????               <1> SourceFile_DirCluster:           resd 1
000181DA ????                   <1> SourceFile_DirEntryNumber: resw 1
000181DC ????                   <1> SourceFile_MatchCounter:  resw 1
                                <1> ; 16/03/2016
000181DE ??                     <1> SourceFile_SecPerClust:          resb 1
000181DF ??                     <1> SourceFile_Reserved:             resb 1
                                <1> ; Above is 128 bytes
                                <1>
                                <1> ;Destination File Structure (same with 'Find File' Structure)
000181E0 ??                     <1> DestinationFile_Drv:             resb 1
000181E1 <res 41h>              <1> DestinationFile_Directory:       resb 65
00018222 <res Dh>               <1> DestinationFile_Name:            resb 13
                                <1> DestinationFile_LongNameEntryLength:
0001822F ??                     <1> DestinationFile_LongNameYes:     resb 1 ; Sign for longname procedures
                                <1> ;Above 80 bytes
                                <1> ;is TR-DOS Destination File FullName Format/Structure
00018230 ????                   <1> DestinationFile_AttributesMask: resw 1
00018232 <res 20h>              <1> DestinationFile_DirEntry: resb 32
00018252 ????????               <1> DestinationFile_DirFirstCluster: resd 1
00018256 ????????               <1> DestinationFile_DirCluster:      resd 1
0001825A ????                   <1> DestinationFile_DirEntryNumber: resw 1
0001825C ????                   <1> DestinationFile_MatchCounter:    resw 1
                                <1> ; 16/03/2016
0001825E ??                     <1> DestinationFile_SecPerClust:     resb 1
0001825F ??                     <1> DestinationFile_Reserved: resb 1
                                <1> ; Above is 128 bytes
                                <1>
                                <1> ; 24/04/2016
00018260 ????                   <1> resw 1
                                <1>
                                <1> ; 10/03/2016
                                <1> ; FILE.ASM
00018262 ??                     <1> move_cmd_phase:       resb 1
00018263 ??                     <1> msftdf_sf_df_drv:  resb 1
00018264 ????????               <1> msftdf_drv_offset: resd 1
                                <1>
                                <1> ; 11/03/2016
                                <1> ; DRV_FAT.ASM (21/08/2011)
00018268 ????????               <1> FAT_anc_LCluster:  resd 1
0001826C ????????               <1> FAT_anc_FFCluster: resd 1
                                <1>
                                <1> ;alignb 4
                                <1>
                                <1> ; 14/03/2016
                                <1> ; TRDOS 386 = TRDOS v2.0 feature only !
                                <1> ; 'allocate_memory_block' in 'memory.s'
00018270 ????????               <1> mem_ipg_count:     resd 1 ; page count (for contiguous allocation)
00018274 ????????               <1> mem_pg_count:      resd 1 ; page count (for count down)
00018278 ????????               <1> mem_aperture:      resd 1 ; contiguous free pages (current)
0001827C ????????               <1> mem_max_aperture: resd 1 ; maximum value of contiguous free pages
00018280 ????????               <1> mem_pg_pos: resd 1 ; mem. position (page #) of current aperture
00018284 ????????               <1> mem_max_pg_pos: resd 1 ; mem. position (page #) of max. aperture
                                <1>
                                <1> ; 15/03/2016
                                <1> ; FILE.ASM ('copy_source_file_to_destination_file')
00018288 ??                     <1> copy_cmd_phase:       resb 1
00018289 ??                     <1> csftdf_rw_err:           resb 1
0001828A ??                     <1> DestinationFileFound: resb 1
0001828B ??                     <1> csftdf_cdrv:             resb 1
0001828C ????????               <1> csftdf_filesize:      resd 1
                                <1> ; TRDOS386 (TRDOS v2.0)
                                <1> MainProgCfg_mem_addr: ; 19/12/2025
00018290 ????????               <1> csftdf_sf_mem_addr:   resd 1
                                <1> MainProgCfg_mem_bsize: ; 19/12/2025
00018294 ????????               <1> csftdf_sf_mem_bsize:  resd 1
                                <1> ;
                                <1> MainProgCfg_cluster: ; 19/12/2025
00018298 ????????               <1> csftdf_sf_cluster:    resd 1 ; 16/03/2016
0001829C ????????               <1> csftdf_df_cluster:    resd 1
                                <1> MainProgCfg_r_size: ; 19/12/2025
                                <1> ; 16/03/2016
000182A0 ????????               <1> csftdf_r_size:        resd 1
000182A4 ????????               <1> csftdf_w_size:        resd 1
                                <1> MainProgCfg_rbytes: ; 19/12/2025
000182A8 ????????               <1> csftdf_sf_rbytes:     resd 1
000182AC ????????               <1> csftdf_df_wbytes:     resd 1
000182B0 ??                     <1> csftdf_percentage:    resb 1
                                <1> ; 17/03/2016
000182B1 ??                     <1> csftdf_videopage:     resb 1
000182B2 ????                   <1> csftdf_cursorpos:     resw 1
000182B4 ????????               <1> csftdf_sf_drv_dt:     resd 1
000182B8 ????????               <1> csftdf_df_drv_dt:     resd 1
                                <1> ; 01/09/2024
                                <1> ; 29/08/2024
                                <1> ;csftdf_df_dclust:    resd 1
```

```
                                <1> ;csftdf_df_dindex:    resd 1
                                <1>
                                <1> ; 21/03/2016
                                <1> ; 20/03/2016
                                <1> ; FILE.ASM
000182BC ????????               <1> createfile_Name_Offset:  resd 1
000182C0 ????????               <1> createfile_FreeSectors:  resd 1
000182C4 ????????               <1> createfile_size:         resd 1
000182C8 ????????               <1> createfile_FFCluster:    resd 1 ; 11/03/2016
000182CC ????????               <1> createfile_LastDirCluster: resd 1
000182D0 ????????               <1> createfile_Cluster:      resd 1
000182D4 ????????               <1> createfile_PCluster:     resd 1
000182D8 ??                     <1> createfile_attrib:  resb 1
000182D9 ??                     <1> createfile_SecPerClust:  resb 1
000182DA ????                   <1> createfile_DirIndex:     resw 1
000182DC ????????               <1> createfile_CCount:  resd 1
                                <1> ; 19/12/2025
                                <1> ;createfile_BytesPerSec: resw 1 ; 23/03/2016
000182E0 ??                     <1> createfile_wfc:             resb 1
000182E1 ??                     <1> createfile_UpdatePDir:     resb 1 ; 31/03/2016
                                <1>
                                <1> ;alignb 4
                                <1>
                                <1> ; 11/04/2016
000182E2 ????                   <1> env_var_length:    resw 1
                                <1>
                                <1> alignb 4
                                <1>
                                <1> ; 25/04/2016
000182E4 ??                     <1> readi.valid:       resb 1 ; valid data (>0 = valid for readi)
000182E5 ??                     <1> readi.drv:  resb 1 ; drive number (0, 1,2,3,4..)
000182E6 ??                     <1> readi.spc:  resb 1 ; sectors per cluster for 'readi' drive
000182E7 ??                     <1> readi.s_index:  resb 1 ; sector index in current cluster (buffer)
000182E8 ????????               <1> readi.sector:      resd 1 ; current disk sector
000182EC ????                   <1> readi.bpc:  resw 1 ; bytes per cluster - 1
000182EE ????                   <1> readi.offset:      resw 1 ; byte offset in cluster buffer
000182F0 ????????               <1> readi.cluster:  resd 1 ; current cluster number
000182F4 ????????               <1> readi.c_index:     resd 1 ; cluster index of the current cluster (0,1,2,3..)
000182F8 ????????               <1> readi.fclust:      resd 1 ; first cluster of the current cluster
                                <1> ; 19/12/2025
                                <1> ;readi.fs_index: resd 1 ; sector index in disk/file section (for Singlix FS)
                                <1> ;readi.buffer:     resd 1 ; readi sector buffer address
                                <1>
                                <1> ;alignb 4
                                <1>
000182FC ??                     <1> writei.valid:      resb 1 ; valid data (>0 = valid for writei)
000182FD ??                     <1> writei.drv: resb 1 ; drive number (0, 1,2,3,4..)
000182FE ??                     <1> writei.spc: resb 1 ; sectors per cluster for 'writei' drive
000182FF ??                     <1> writei.s_index: resb 1 ; sector index in current cluster (buffer)
00018300 ????????               <1> writei.sector:     resd 1 ; current disk sector
00018304 ????                   <1> writei.bpc: resw 1 ; bytes per cluster - 1
00018306 ????                   <1> writei.offset:     resw 1 ; byte offset in cluster buffer
00018308 ????????               <1> writei.cluster: resd 1 ; current cluster number
0001830C ????????               <1> writei.c_index:    resd 1 ; cluster index of the current cluster (0,1,2,3..)
00018310 ????????               <1> writei.fclust:  resd 1 ; first cluster of the current cluster
00018314 ????????               <1> writei.fs_index: resd 1 ; sector index in disk/file section (for Singlix FS)
                                <1> ;writei.buffer:    resd 1 ; writei sector buffer address
00018318 ????????               <1> writei.lclust:     resd 1 ; writei last cluster (mget_w) ; 23/10/2016
0001831C ????????               <1> writei.l_index:    resd 1 ; writei last cluster index (mget_w) ; 23/10/2016
00018320 ??                     <1> writei.ofn: resb 1 ; open file number (to be written) ; 23/10/2016
                                <1>
00018321 ??????                 <1> alignb 4
                                <1>
                                <1> ; 29/04/2016
00018324 ????????               <1> Run_CDirFC: resd 1
                                <1> ;Run_Auto_Path:    resb 1
                                <1> ; 03/07/2025
                                <1> Run_Path_Length:
00018328 ????????               <1>            resd 1
0001832C ????????               <1> Run_Auto_Path:     resd 1
                                <1> Run_Manual_Path:
00018330 ??                     <1>            resb 1 ; 0 -> auto path sequence needed
00018331 ??                     <1> EXE_ID:            resb 1
00018332 ??                     <1> EXE_dot:   resb 1
00018333 ??                     <1>            resb 1 ; 03/07/2025
                                <1>
                                <1> ; 06/05/2016
00018334 ????????               <1> mainprog_return_addr: resd 1
00018338 ????????               <1> last_error: resd 1  ; this will be used to return error code to MainProg
                                <1>                     ; 'lasterror' keyword will be used later to get the
                                <1>                     ; last error code/number/status.
                                <1> ; 12/05/2016
0001833C ????????               <1> video_eax:  resd 1  ; eax return value of video function
                                <1>
                                <1> ; 01/06/2016
00018340 ????????               <1> user_buffer:       resd 1  ; 'diskio.s' (INT 33h, Function 08h, floppy disk type)
                                <1>
                                <1> ; 21/05/2016 - TRDOS 386 ('swap/switch', 'rswap', [u.pri])
00018344 ??                     <1> priority:   resb 1  ; running priority level of process (0,1,2)
                                <1>                     ; (run queue which is process comes from)
                                <1> ; 22/05/2016 - TRDOS 386 ('set_run_sequence', 'rtc_int', 'u_timer')
00018345 ??                     <1> p_change:   resb 1  ; process change status (for timer events)
                                <1> ; 23/05/2016 - TRDOS 386 ('clock')
00018346 ??                     <1> multi_tasking:     resb 1   ; Multi Tasking status (0 = disabled, >0 = enabled)
                                <1>                     ; (EBX will return with user buffer addr or disk type)
                                <1> ; 07/06/2016
00018347 ??                     <1> timer_events:      resb 1  ; number of (active) timer events, <= 16
                                <1>
                                <1> ; 24/06/2016
00018348 ??                     <1> w_str_cmd:  resb 1 ; WRITE_STRING command (0,1,2,3) ; video.s
00018349 ??                     <1> p_crt_mode: resb 1  ; previous video mode (=3 or 0), backup mark/sign
                                <1> ; 26/06/2016
0001834A ??                     <1> p_crt_page: resb 1  ; previous active page (for 'set_mode')
                                <1> ; 04/07/2016
0001834B ??                     <1> noclearmem: resb 1  ; if set, 'SET MODE' (INT 31h) function (AH = 4)
                                <1>                     ; will not clear the video memory
                                <1>                     ; (usable for graphics modes only)
                                <1> alignb 2
0001834C ????                   <1> CRT_LEN:   resw 1  ; length of regen buffer in bytes
0001834E <res 10h>              <1> cursor_pposn:      resw 8  ; cursor positions backup
                                <1>
                                <1> ; 10/07/2016 ('VGA_FONT_SETUP', INT 43H address for x86 real mode bios)
0001835E ????????               <1> VGA_INT43H: resd 1 ; 0 = default (not configured by user)
                                <1>                     ; 0FFFFFFFFh = user defined fonts
                                <1>                     ; address:
                                <1>                     ;      vgafont8
                                <1>                     ;      vgafont16
                                <1>                     ;      vgafont14
                                <1>
                                <1> ; 25/07/2016
00018362 ??                     <1> VGA_MTYPE:  resb 1  ; 0=CTEXT,1=MTEXT,2=CGA,3=PLANAR1,4=PLANAR4,5=LINEAR
                                <1>
                                <1> ; 23/10/2016
00018363 ??                     <1> setfmod:   resb 1 ; update last modification date&time sign (if >0)
```

```
                                <1>                     ; (it is Open File Number + 1, if > 0)
                                <1> ; 27/08/2024
00018364 ??                     <1> setfclust:  resb 1  ; first cluster of file
                                <1>                     ; (is used by update lmdt proc)
00018365 ??????                 <1> alignb 4
                                <1>
                                <1> ; 16/10/2016
00018368 ????????               <1> FFF_UBuffer:       resd 1  ; User's buffer address for FFF & FNF system calls
                                <1> ; 15/10/2016
0001836C ??                     <1> FFF_Valid:  resb 1  ; Find First File Structure validation byte
                                <1>                     ; 0  = invalid (Find Next File can't use FFF struct)
                                <1>                     ; >0 = valid, return type for FFF and Find Next File
                                <1>                     ; 24 = basic parameters, 24 bytes
                                <1>                     ; 128 = entire FFF structure/table, 128 bytes
                                <1> ; 16/10/2016 (FFF_Attrib: resw 1)
0001836D ??                     <1> FFF_Attrib: resb 1  ; Find First File attributes for Find Next File (LB)
0001836E ??                     <1> FFF_RType:  resb 1  ; FFF return type (0 = Basic, >0 = complete) (HB)
                                <1> ; 16/10/2016 - 05/10/2016 (Set Working Path)
0001836F ??                     <1> SWP_inv_fname:     resb 1 ; Set Working Path - Invalid File Name
00018370 ????                   <1> SWP_Mode:   resw 1 ; Set Working Path - Mode
00018372 ??                     <1> SWP_DRV:    resb 1 ; Set Working Path - Drive
00018373 ??                     <1> SWP_DRV_chg:       resb 1 ; Set Working Path - Drive Change
                                <1>
                                <1> ; 27/02/2017
00018374 ??                     <1> fpready:    resb 1 ; '80387 fpu is ready' flag
                                <1>
                                <1> ; 17/04/2021
                                <1> ; (DEVICE parameters is disabled as temporary)
                                <1>
                                <1> ; 08/10/2016
                                <1> ;device_name:      resb 9  ; capitalized (and zero padded) device name
                                <1>                     ; (example: "TTY0",0,0,0,0,0")
                                <1>
00018375 ??????                 <1> alignb 4
                                <1>
                                <1> ; 08/10/2016
                                <1> ; 07/10/2016
                                <1> ; Table of kernel devices (which do not use installable device drivers)
                                <1> ; has been coded into KERNEL (trdosk9.s)
                                <1> ; 07/10/2016
                                <1> ; 8 installable device drivers available to install (NUMIDEV)
                                <1> ;IDEV_PGDIR: resd NUMIDEV
                                <1>                     ; Page directories of installable device drivers
                                <1>                     ;
                                <1>                     ; Note: Virtual start address is always 400000h
                                <1>                     ; (end of the 1st 4MB). [org 400000h]
                                <1>                     ; Segments: KCODE, KDATA
                                <1>                     ; Method: call 400000h (after changing page dir)
                                <1>                     ; Query code located at the start (400000h).
                                <1>                     ; Query code returns with
                                <1>                     ;   eax = device type and driver version
                                <1>                     ;         AL = Device Type minor
                                <1>                     ;         AH = Device Type major
                                <1>                     ;         Byte 16-23 : Version minor
                                <1>                     ;        Byte 24-31 : Version major - 1
                                <1>                     ;                     (0:0 -> 1.0)
                                <1>                     ;   ebx = initialization code address
                                <1>                     ;   ecx = configuration table address
                                <1>                     ;   edx = description table address
                                <1>                     ;   esi = device (default) name address (ASCIIZ)
                                <1>                     ;       (name has "/DEV/" prefix)
                                <1>                     ;   edi = dispatch table address
                                <1>                     ;       (for calling kernel-device functions)
                                <1>                     ;   ebp = address table address
                                <1>                     ; Initialization code returns with
                                <1>                     ;   eax = open code address
                                <1>                     ;   ecx = close code address
                                <1>                     ;   ebx = read code address
                                <1>                     ;   edx = write code address
                                <1>                     ;   esi = IOCTL code address
                                <1>                     ;   edi = dispatch table address
                                <1>                     ;   ebp = address table address
                                <1>                     ; Address Table:
                                <1>                     ;    Offset 0  : open code address
                                <1>                     ;    Offset 4  : read code address
                                <1>                     ;    Offset 8  : write code address
                                <1>                     ;    Offset 12 : close code address
                                <1>                     ;    Offset 16 : IOCTL code address
                                <1>                     ;    Offset 20 : initialization code address
                                <1>                     ;    Offset 24 : description table address
                                <1>                     ;    Offset 28 : configuration table address
                                <1>                     ;    Offset 32 : device name address
                                <1>                     ;    Offset 36 : dispatch table address
                                <1>                     ;          (for calling kernel-device functions)
                                <1>
                                <1> ;IDEV_NAME:  resb 8*NUMIDEV
                                <1>                       ; 8 byte names of installable device drivers
                                <1>
                                <1> ;IDEV_TYPE:  resb NUMIDEV ; Driver type of installable device drivers
                                <1> ;IDEV_FLAGS: resb NUMIDEV ; Device access parameters for installable
                                <1>                           ; device drivers (These values are set while
                                <1>                       ; the device driver is being loaded.)
                                <1> ;IDEV_OADDR: resd NUMIDEV ; open function addr for installable dev driver
                                <1> ;IDEV_CADDR: resd NUMIDEV ; close function addr for installable dev driver
                                <1> ;IDEV_RADDR: resd NUMIDEV ; read function addr for installable dev driver
                                <1> ;IDEV_WADDR: resd NUMIDEV ; write function addr for installable dev driver
                                <1>
                                <1> ; 08/10/2016
                                <1> ; 07/10/2016
                                <1> ; Device Open and Access parameters
                                <1> ;DEV_ACCESS:       resb NUMOFDEVICES    ; bit 0 = accessable by normal users
                                <1>                                 ; bit 1 = read access permission
                                <1>                                 ; bit 2 = write access permission
                                <1>                                 ; bit 3 = IOCTL permission to users
                                <1>                                 ; bit 4 = block device if it is set
                                <1>                                 ; bit 5 = 16 bit or 1024 byte data
                                <1>                                 ; bit 6 = 32 bit or 2048 byte data
                                <1>                                 ; bit 7 = installable device driver
                                <1> ;DEV_R_OWNER:      resb NUMOFDEVICES    ; Reading owner no (u.uid) of devices
                                <1> ;DEV_R_OPENCOUNT: resb NUMOFDEVICES  ; Reading open count
                                <1> ;DEV_W_OWNER:      resb NUMOFDEVICES    ; Writing owner no (u.uid) of devices
                                <1> ;DEV_W_OPENCOUNT: resb NUMOFDEVICES  ; Writing open count
                                <1> ;DEV_DRIVER:       resb NUMOFDEVICES    ; device driver number (1 to 7Fh)
                                <1>                                 ; *if bit 7 is set (80 to FFh)
                                <1>                                 ; *if it is installable device driver
                                <1>                                 ; *index (0 to 7Fh)
                                <1>                                 ; otherwise it is kernel device index
                                <1> ;DEV_OPENMODE:     resb NUMOFDEVICES    ; 1 = read mode
                                <1>                                 ; 2 = write mode
                                <1>                                 ; 3 = read & write
                                <1>                                 ; 0 = not open (free)
                                <1> ;DEV_NAME_PTR:     resd NUMOFDEVICES    ; pointers to name addresses of drivers
                                <1>                                 ; Address base: KDEV_NAME+
                                <1>                                 ; or IDEV_NAME+
```

```
                                <1> ;DEV_R_POINTER:    resd NUMOFDEVICES    ; reading pointer, writing pointer
                                <1> ;DEV_W_POINTER:    resd NUMOFDEVICES    ; sector number if block device
                                <1>                                         ; character offset if char device
                                <1> alignb 4
                                <1>
                                <1> ; 06/10/2016
                                <1> ; Open File Parameters
00018378 <res 80h>              <1> OF_FCLUSTER:       resd OPENFILES  ; First clusters of open files
000183F8 <res 20h>              <1> OF_DRIVE:   resb OPENFILES  ; Logical DOS drive numbers of open files
00018418 <res 20h>              <1> OF_MODE:    resb OPENFILES  ; Open mode (1 = read, 2 = write, 3 = r&w)
00018438 <res 20h>              <1> OF_STATUS:  resb OPENFILES  ; (bit 0 = read, bit 1 = write)
00018458 <res 20h>              <1> OF_OPENCOUNT:      resb OPENFILES  ; Open counts of open files
00018478 <res 80h>              <1> OF_POINTER: resd OPENFILES ; File seek/read/write pointer
000184F8 <res 80h>              <1> OF_SIZE:    resd OPENFILES ; File sizes of open files (in bytes)
00018578 <res 80h>              <1> OF_DIRFCLUSTER:    resd OPENFILES  ; Directory First Clusters of open files
000185F8 <res 80h>              <1> OF_DIRCLUSTER:     resd OPENFILES  ; Directory (Entry) Clusters of open files
00018678 <res 80h>              <1> OF_VOLUMEID:       resd OPENFILES  ; Vol ID for removable drives of open files
000186F8 <res 80h>              <1> OF_CCLUSTER:       resd OPENFILES  ; Current clusters of open files
00018778 <res 80h>              <1> OF_CCINDEX: resd OPENFILES  ; Cluster index numbers of current clusters
                                <1> ; 24/10/2016
000187F8 <res 40h>              <1> OF_DIRENTRY:       resw OPENFILES  ; Directory entry index no. in dir cluster
                                <1>                            ; Sector index = entry index / 16
                                <1> ;alignb 2
                                <1>
                                <1> DTA:        ;resd 24       ; Find First File data transfer area
00018838 <res 18h>              <1>             resb 24        ; 29/07/2022
                                <1>
                                <1> ; 19/12/2016
00018850 ??                     <1> tcallback:  resb 1         ; Timer callback method flag for 'systimer'
00018851 ??                     <1> trtc:               resb 1         ; Timer interrupt type flag for 'systimer'
                                <1> ; 20/02/2017
00018852 ??                     <1> no_page_swap:      resb 1         ; Swap lock for Signal Response Byte pages
                                <1> ;;15/01/2017
                                <1> ; 02/01/2017
                                <1> ;;intflg:   resb 1         ; software interrupt in progress signal
                                <1>                            ; (for timer interrupt)
00018853 ??                     <1> alignb 4
                                <1> ; 13/04/2017
                                <1> ;DEV_INTR:  resb NUMOFDEVICES ; Device Interrupt (IRQ) number + 1
                                <1>                            ; (0= not available, 1= IRQ 0, 16= IRQ 15)
00018854 <res 40h>              <1> DEV_INT_HNDLR:     resd 16        ; Device Interrupt Handler addr, if > 0
                                <1>
                                <1> ;alignb 4
                                <1>
                                <1> ; 26/02/2017 ; IRQ Callback parameters ('syscalbac')
                                <1> ;Index: ; 0 to 8
                                <1> ;   0 = IRQ3, 1 = IRQ4, 2 = IRQ5, 3 = IRQ7
                                <1> ;   4 = IRQ9, 5 = IRQ10, 6 = IRQ11, 7 = IRQ12, 8 = IRQ13
00018894 <res 9h>               <1> IRQ.owner:  resb 9         ; owner, 0 = free, >0 = [u.uno]
0001889D <res 9h>               <1> IRQ.dev:    resb 9         ; 0 = default/kernel, >0 = device number
000188A6 <res 9h>               <1> IRQ.method: resb 9         ; 0 = Signal Response Byte, 1 = Callback
000188AF <res 9h>               <1> IRQ.srb:    resb 9         ; Signal Response/Return Byte value
000188B8 <res 24h>              <1> IRQ.addr:   resd 9         ; Rignal Response Byte address (physical)
                                <1>                            ; or Callback service address (virtual)
                                <1> ; 28/02/2017
000188DC ????????               <1> IRQ_cr3:    resd 1         ; for saving cr3 register in IRQ handler
000188E0 ??                     <1> IRQnum:             resb 1         ; IRQ number for IRQ handler (trdosk8.s)
                                <1>
                                <1> ; 10/04/2017
                                <1> ; 03/04/2017
                                <1> ; UNINITIALIZED AUDIO DATA
000188E1 ??????                 <1> alignb 4
000188E4 ??                     <1> audio_pci:  resb 1
000188E5 ??                     <1> audio_device:      resb 1
                                <1> ;audio_mode:       resb 1
000188E6 ??                     <1> audio_intr: resb 1
000188E7 ??                     <1> audio_busy: resb 1  ; Busy flag for audio irq ; 21/04/2017
                                <1> ;audio_reserved: resb 1
                                <1> ; 20/11/2023
000188E8 ????                   <1> NAMBAR:             resw 1 ; Native Audio Mixer Base Address
                                <1> NABMBAR:    ; 02/10/2023 (NABMBAR = audio_io_base)
000188EA ????                   <1> audio_io_base:     resw 1 ; Base I/O address of audio device
000188EC ????????               <1> audio_dev_id:      resd 1 ; BUS/DEV/FN ; 00000000BBBBBBBBDDDDDFFF00000000
000188F0 ????????               <1> audio_vendor:      resd 1
000188F4 ????????               <1> audio_stats_cmd: resd 1
                                <1> ;
000188F8 ????????               <1> audio_buffer:      resd 1 ; virtual address of user's audio buffer
000188FC ????????               <1> audio_p_buffer:    resd 1 ; Physical address of user's audio buffer
00018900 ????????               <1> audio_buff_size: resd 1 ; user's audio buffer size (half buffer size)
00018904 ????????               <1> audio_dma_buff: resd 1  ; dma buffer address
00018908 ????????               <1> audio_dmabuff_size: resd 1 ; dma buffer size (2 * half buffer size)
                                <1> ; 05/06/2024
0001890C ????????               <1> dma_hbuff_size: resd 1 ; dma half buffer size
                                <1> ;
00018910 ??                     <1> audio_flag: resb 1  ; dma buffer flag (1st half = 0, 2nd half = 1)
00018911 ??                     <1> audio_user: resb 1 ; user number of the owner
00018912 ??                     <1> audio_cb_mode:     resb 1 ; 0 = signal response byte method
                                <1>                            ; 1 = callback method
                                <1>                            ; 2 = s.r.b. method with auto increment
00018913 ??                     <1> audio_srb:  resb 1 ; signal response byte value
00018914 ????????               <1> audio_cb_addr:     resd 1  ; callback service address or s.r.b. address
                                <1>                            ; (s.r.b. addr is physical, cbs addr is virtual)
                                <1>
00018918 ??                     <1> audio_bps:  resb 1  ; selected mode: 8 bit, 16 bit
00018919 ??                     <1> audio_stmo: resb 1 ; selected mode: mono /stereo
0001891A ????                   <1> audio_freq:        resw 1 ; sampling rate
                                <1> ; 20/11/2023
0001891C ??                     <1> VRA:        resb 1
0001891D ??                     <1> audio_mode: resb 1
                                <1>
                                <1> ; 21/04/2017
0001891E ??                     <1> audio_play_cmd: resb 1  ; Play/Stop command (1 = play, 0 = stop)
                                <1> ; 21/11/2023
                                <1> ;audio_civ: ; 28/05/2017 ; Current Buffer Index (AC'97)
                                <1> ; 23/05/2024
                                <1> LVI:                ; AC'97 Last Valid Buffer Index
0001891F ??                     <1> audio_flag_eol:    resb 1  ; End of Link status (vt8233, EOL/FLAG)
                                <1>
                                <1> audio_master_volume:
00018920 ??                     <1> audio_master_volume_l: resb 1 ; sound volume (lineout) left channel
00018921 ??                     <1> audio_master_volume_r: resb 1 ; sound volume (lineout) right channel
                                <1>
                                <1> ; 24/05/2024
                                <1> audio_pcmo_volume:
00018922 ??                     <1> audio_pcmo_volume_l: resb 1 ; PCM out volume left channel
00018923 ??                     <1> audio_pcmo_volume_r: resb 1 ; PCM out volume right channel
                                <1>
                                <1> ; 02/06/2024
                                <1> ;alignb 4
                                <1>
                                <1> ; 20/11/2023
                                <1> ; 28/05/2017
                                <1> ; AC'97 Audio Controller Base Adress Registers
                                <1> ;NAMBAR:    resw 1 ; Native Audio Mixer Base Address
```

```
                                <1> ;NABMBAR:   resw 1  ; Native Audio Bus Mastering Base Address
                                <1>
                                <1> ; 02/06/2024
00018924 ????????               <1> alignb 8
                                <1>
                                <1> ; 21/04/2017
00018928 <res 400h>             <1> audio_bdl_buff:    resd 32*8 ; VT8233 (AC97) BDL Buffer Size
                                <1> ; 12/05/2017
00018D28 ????????               <1> base_addr:  resd 1  ; 'direct_memory_access' (memory.s)
                                <1>
                                <1> ; 01/06/2024
00018D2C ??                     <1> reset:              resb 1  ; AC97 init
                                <1>
                                <1> ; 28/08/2017
                                <1> ; 20/08/2017
                                <1>             ;resb 1 ;
00018D2D ??                     <1> dma_user:   resb 1  ; user number for sysdma
00018D2E ??                     <1> dma_channel:        resb 1  ; dma channel for sysdma
00018D2F ??                     <1> dma_mode:   resb 1  ; dma mode for sysdma
00018D30 ????????               <1> dma_addr:   resd 1  ; dma buffer physical addr for sysdma
00018D34 ????????               <1> dma_size:   resd 1  ; dma buffer size (in bytes) for sysdma
00018D38 ????????               <1> dma_start:  resd 1  ; dma start address for sysdma
00018D3C ????????               <1> dma_count:  resd 1  ; dma count (in bytes) for sysdma
                                <1>
                                <1> ; 04/12/2023
                                <1> %if 0
                                <1>
                                <1> alignb 65536
                                <1> ; 09/08/2017
                                <1> ; 12/05/2017
                                <1> sb16_dma_buffer: resb 65536 ; DMA buffer for sb16 audio playing.
                                <1>
                                <1> %endif
                                    ; 24/01/2016
                                    %include 'ubss.s'  ; UNINITIALIZED KERNEL (USER) DATA
                                <1> ; ****************************************************************************
                                <1> ; TRDOS386.ASM (TRDOS 386 Kernel - v2.0.9) - UNINITIALIZED USER DATA : ubss.s
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Last Update: 19/08/2024  (Previous: 07/08/2022)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Beginning: 24/01/2016
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Assembler: NASM version 2.15 (trdos386.s)
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from 'Retro UNIX 386 Kernel - v0.2.1.0' source code by Erdogan Tan
                                <1> ; ux.s (04/12/2015)
                                <1> ; ****************************************************************************
                                <1>
                                <1> ; Retro UNIX 386 v1 Kernel - ux.s
                                <1> ; Last Modification: 04/12/2015
                                <1> ;
                                <1> ; /////// RETRO UNIX 386 V1 SYSTEM DEFINITIONS ///////////////
                                <1> ; (Modified from
                                <1> ;   Retro UNIX 8086 v1 system definitions in 'UNIX.ASM', 01/09/2014)
                                <1> ; ((UNIX.ASM (RETRO UNIX 8086 V1 Kernel), 11/03/2013 - 01/09/2014))
                                <1> ; ----------------------------------------------------------------------------
                                <1> ; Derived from UNIX Operating System (v1.0 for PDP-11)
                                <1> ; (Original) Source Code by Ken Thompson (1971-1972)
                                <1> ; <Bell Laboratories (17/3/1972)>
                                <1> ; <Preliminary Release of UNIX Implementation Document>
                                <1> ; (Section E10 (17/3/1972) - ux.s)
                                <1> ; ****************************************************************************
                                <1> ; Ref: Retro UNIX 386 v1.2 Kernel (v0.2.2.3) - ux.s - 15/07/2022
                                <1>
                                <1> alignb 2
                                <1>
                                <1> %if 0
                                <1>
                                <1> inode:
                                <1>     ;; 11/03/2013.
                                <1>     ;;Derived from UNIX v1 source code 'inode' structure (ux).
                                <1>     ;;i.
                                <1>     ;
                                <1>     ;i.flgs: resw 1
                                <1>     ;i.nlks: resb 1
                                <1>     ;i.uid:  resb 1
                                <1>         ;i.size: resw 1 ; size
                                <1>     ;i.dskp: resw 8 ; 16 bytes
                                <1>     ;i.ctim: resd 1
                                <1>     ;i.mtim: resd 1
                                <1>     ;i.rsvd: resw 1 ; Reserved (ZERO/Undefined word for UNIX v1)
                                <1>
                                <1>     ; 26/01/2020
                                <1>     ; Retro UNIX 386 v2.0 - Modified UNIX v7 inode model
                                <1>     ;       (15/09/2029 .. 18/12/2019)
                                <1>
                                <1>     i.flgs:   resw 1      ; /* mode and type of file */
                                <1>     i.nlks:   resw 1      ; /* number of links to file */
                                <1>     i.uid:    resw 1      ; /* owner's user id */  - 0 to 65535 -
                                <1>     i.gid:    resb 1      ; /* owner's group id */ - o to 255 -
                                <1>     i.size_h: resb 1      ; /* number of bytes in file */ ; byte 5
                                <1>     i.size:   resd 1 ; size       ; /* number of bytes in file */
                                <1>     i.dskp:   resd 10 ; 40 bytes ; /* disk block addresses */
                                <1>     i.atim:   resd 1      ; /* time last accessed */
                                <1>     i.mtim:   resd 1      ; /* time last modified */
                                <1>     i.ctim:   resd 1      ; /* time created */
                                <1>
                                <1> I_SIZE      equ $ - inode
                                <1>
                                <1> %endif
                                <1>
                                <1> process:
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 27/02/2022
                                <1>     ; 12/01/2022 - Retro UNIX 386 v1.2
                                <1>     ; 19/12/2016
                                <1>     ; 21/05/2016
                                <1>     ; 19/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ; 06/05/2015 - Retro UNIX 386 v1
                                <1>     ; 11/03/2013 - 05/02/2014 (Retro UNIX 8086 v1)
                                <1>     ;Derived from UNIX v1 source code 'proc' structure (ux).
                                <1>     ;p.
                                <1>
00018D40 <res 20h>              <1>         p.pid:   resw nproc
00018D60 <res 20h>              <1>         p.ppid:  resw nproc
                                <1>     ;p.break: resw nproc ; 12/01/2022 (p.break is not used)
00018D80 <res 10h>              <1>         p.ttyc:  resb nproc ; console tty in Retro UNIX 8086 v1.
                                <1>     ; 27/02/2022 (p.waitc is not used)
                                <1>     ;p.waitc: resb nproc ; waiting channel in Retro UNIX 8086 v1.
00018D90 <res 10h>              <1>     p.link:  resb nproc
00018DA0 <res 10h>              <1>     p.stat:  resb nproc
                                <1>
                                <1>     ; 06/05/2015 (Retro UNIX 386 v1 feature only !)
00018DB0 <res 40h>              <1>     p.upage: resd nproc ; Physical address of the process's
```

```
                                <1>                         ; 'user' structure
                                <1>     ; 21/05/2016
                                <1>     ; 19/05/2016 (TRDOS 386 feature only!)
00018DF0 <res 10h>              <1>     p.timer: resb nproc ; number of timer events of the processs
                                <1>
                                <1>     ; 19/12/2016
00018E00 <res 40h>              <1>     p.tcb:  resd nproc ; timer callback service address (if > 0)
                                <1>
                                <1>     ; 18/08/2024 (TRDOS 386 v2.0.9)
00018E40 <res 10h>              <1>     p.exitc: resb nproc ; exit code of the child
                                <1>
                                <1> P_SIZE     equ $ - process
                                <1>
                                <1> ; fsp table (original UNIX v1)
                                <1> ;
                                <1> ;Entry
                                <1> ;          15                                      0
                                <1> ;  1     |---|-----------------------------------------|
                                <1> ;        |r/w|       i-number of open file             |
                                <1> ;        |---|-----------------------------------------|
                                <1> ;        |               device number                |
                                <1> ;        |--------------------------------------------|
                                <1> ;    (*) | offset pointer, i.e., r/w pointer to file |
                                <1> ;        |--------------------------------------------|
                                <1> ;        |  flag that says    | number of processes   |
                                <1> ;        |   file deleted     | that have file open   |
                                <1> ;        |--------------------------------------------|
                                <1> ;  2     |                                            |
                                <1> ;        |--------------------------------------------|
                                <1> ;        |                                            |
                                <1> ;        |--------------------------------------------|
                                <1> ;        |                                            |
                                <1> ;        |--------------------------------------------|
                                <1> ;        |                                            |
                                <1> ;  3     |                                            |
                                <1> ;        |                                            |
                                <1> ;
                                <1> ; (*) Retro UNIX 386 v1 modification: 32 bit offset pointer
                                <1>
                                <1> ; 27/03/2020 - Retro UNIX 386 v2 - FSP (OPEN FILES) TABLE
                                <1>
                                <1> ;Entry
                                <1> ;          15                    7                    0
                                <1> ;  1     |---------------------------------------------|
                                <1> ;        |              i-number of open file           |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |        high word of 32 bit i-number         |
                                <1> ;        |---------------------------------------------|
                                <1> ;        | open mode & status  |   device number       |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |    reserved byte    |     open count        |
                                <1> ;        |---------------------------------------------|
                                <1> ;        | offset pointer, i.e., r/w pointer to file |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |   64 bit file offset pointer (bit 16-31)   |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |   64 bit file offset pointer (bit 32-47)   |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |   64 bit file offset pointer (bit 48-63)   |
                                <1> ;        |---------------------------------------------|
                                <1> ;  2     |                                             |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |                                             |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |                                             |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |                                             |
                                <1> ;        |---------------------------------------------|
                                <1> ;        |                                             |
                                <1>
                                <1> %if 0
                                <1>
                                <1> ; (Retro UNIX 386 v1.2 - ux.s - 15/07/2022)
                                <1> ; 22/11/2021
                                <1> ; 21/07/2021 - Retro UNIX 386 v2 open file structure revision
                                <1>
                                <1> struc file  ; open files (fsp) structure ; (*)
                                <1>   .inode:  resw 1  ; inode number of open file (32 bit)
                                <1>   .i32:       resw 1  ; higher word of inode number (reserved)
                                <1>   .drive:  resb 1  ; logical drive (disk) number
                                <1>   .flags:  resb 1  ; open mode and status
                                <1>   .count:  resb 1  ; number of processes that have file open
                                <1>   ;.rsvd:  resb 1  ; reserved byte (for next versions)
                                <1>   .mnt:    resb 1  ; mnttab index+1 (0 = not mounted)
                                <1>   .offset: resd 1  ; file offset/pointer (64 bit)
                                <1>   .o64:       resd 1  ; higher 32 bit of file offset
                                <1>  .size:  ; = 16
                                <1> endstruc
                                <1>
                                <1> %endif
                                <1>
                                <1> ; 23/07/2022
                                <1> ;fsp:      resb nfiles * 16 ; (*)
                                <1>
00018E50 ??                     <1> idev:      resb 1
00018E51 ??                     <1> cdev:      resb 1
                                <1>
                                <1> ; 15/04/2015
                                <1> ;fsp:      resb nfiles * 10 ; 11/05/2015 (8 -> 10)
                                <1> ;idev:     resw 1 ; device number is 1 byte in Retro UNIX 8086 v1 !
                                <1> ;cdev:     resw 1 ; device number is 1 byte in Retro UNIX 8086 v1 !
                                <1>
                                <1> ; 18/05/2015
                                <1> ; 26/04/2013 device/drive parameters (Retro UNIX 8086 v1 feature only!)
                                <1> ; 'UNIX' device numbers (as in 'cdev' and 'u.cdrv')
                                <1> ;   0 -> root device (which has Retro UNIX 8086 v1 file system)
                                <1> ;   1 -> mounted device (which has Retro UNIX 8086 v1 file system)
                                <1> ; 'Retro UNIX 8086 v1' device numbers: (for disk I/O procedures)
                                <1> ;   0 -> fd0 (physical drive, floppy disk 1), physical drive number = 0
                                <1> ;   1 -> fd1 (physical drive, floppy disk 2), physical drive number = 1
                                <1> ;   2 -> hd0 (physical drive, hard disk 1), physical drive number = 80h
                                <1> ;   3 -> hd1 (physical drive, hard disk 2), physical drive number = 81h
                                <1> ;   4 -> hd2 (physical drive, hard disk 3), physical drive number = 82h
                                <1> ;   5 -> hd3 (physical drive, hard disk 4), physical drive number = 83h
                                <1>
00018E52 ??                     <1> rdev:      resb 1 ; root device number ; Retro UNIX 8086 v1 feature only!
                                <1>             ; as above, for physical drives numbers in following table
00018E53 ??                     <1> mdev:      resb 1 ; mounted device number ; Retro UNIX 8086 v1 feature only!
                                <1>
                                <1> ; 23/07/2022
                                <1> ;; 15/04/2015
                                <1> ;active: resb 1
                                <1> ;    resb 1 ; 09/06/2015
```

```
                                <1>
                                <1> ; 23/07/2022
                                <1> ;mnti:       resw 1
                                <1> ; 07/08/2022
00018E54 ????                   <1> mpid:        resw 1
                                <1> ;rootdir: resw 1
00018E56 ????????               <1> rootdir: resd 1
                                <1>
                                <1> ; 21/05/2016 - TRDOS 386 (TRDOS v2.0) - priority levels, 3 run queues
                                <1> runq:
00018E5A ????                   <1> runq_event:  resw 1 ; high priority, 'run for event'            ; 2
00018E5C ????                   <1> runq_normal:        resw 1 ; normal/regular priority, 'run as reqular' ; 1
00018E5E ????                   <1> runq_background: resw 1 ; low priority, 'run on background'         ; 0
                                <1> ;
                                <1> ; 23/07/2022
                                <1> ;imod:      resb 1
                                <1> ;smod:      resb 1
                                <1> ;mmod:      resb 1
00018E60 ??                     <1> sysflg:     resb 1
00018E61 ??                     <1>     resb 1
                                <1>
00018E62 ????                   <1> alignb 4
                                <1>
                                <1> user:
                                <1>     ; 23/07/2022 - TRDOS 386 Kernel v2.0.5
                                <1>     ; 04/12/2021 - Retro UNIX 386 v1.2
                                <1>     ; 13/01/2017
                                <1>     ; 19/12/2016
                                <1>     ; 21/05/2016 - TRDOS 386 (TRDOS v2.0)
                                <1>     ;                [u.pri] usage method modification
                                <1>     ; 04/12/2015
                                <1>     ; 18/10/2015
                                <1>     ; 12/10/2015
                                <1>     ; 21/09/2015
                                <1>     ; 24/07/2015
                                <1>     ; 16/06/2015
                                <1>     ; 09/06/2015
                                <1>     ; 11/05/2015
                                <1>     ; 16/04/2015 (Retro UNIX 386 v1 - 32 bit modifications)
                                <1>     ; 10/10/2013
                                <1>     ; 11/03/2013.
                                <1>     ;Derived from UNIX v1 source code 'user' structure (ux).
                                <1>     ;u.
                                <1>
00018E64 ????????               <1>     u.sp:    resd 1 ; esp (kernel stack at the beginning of 'sysent')
00018E68 ????????               <1>     u.usp:   resd 1 ; esp (kernel stack points to user's registers)
00018E6C ????????               <1>     u.r0:    resd 1 ; eax
00018E70 ????                   <1>     u.cdir:  resw 1
00018E72 ????                   <1>              resw 1 ; 23/07/2022
00018E74 ??                     <1>     u.cdrv:  resb 1 ; 23/07/2022
00018E75 ??                     <1>              resb 1
00018E76 <res Ah>               <1>     u.fp:    resb 10
                                <1>     ;u.fp:   resb OPENFILES ; 23/07/202
                                <1>     u.fsp:   ; 23/07/2022
00018E80 ????????               <1>     u.fofp:  resd 1
00018E84 ????????               <1>     u.dirp:  resd 1
00018E88 ????????               <1>     u.namep:  resd 1
00018E8C ????????               <1>     u.off:   resd 1
                                <1>     ;        resd 1 ; 23/07/2022 (64 bit fptr)
00018E90 ????????               <1>     u.base:  resd 1
00018E94 ????????               <1>     u.count:  resd 1
00018E98 ????????               <1>     u.nread:  resd 1
00018E9C ????????               <1>     u.break:  resd 1 ; break
                                <1>     ; 10/01/2017 (TRDOS 386, relocation and dword alignment)
                                <1>     ; tty number (rtty, rcvt, wtty)
00018EA0 ????                   <1>     u.ttyp:  resw 1
00018EA2 ??                     <1>     u.ttyn:  resb 1 ; 28/07/2013 - Retro Unix 8086 v1 feature only !
00018EA3 ??                     <1>     u.mode:   resb 1 ; 23/07/2022
                                <1>     ;u.resb:  resb 1 ; 10/01/2017 (TRDOS 386, temporary)
00018EA4 <res 10h>              <1>     u.dirbuf: resb 16 ; 04/12/2015 (10 -> 16)
                                <1>     ;u.pri:  resw 1 ; 14/02/2014
00018EB4 ??                     <1>     u.quant:  resb 1 ; Retro UNIX 8086 v1 Feature only ! (uquant)
00018EB5 ??                     <1>              resb 1 ; 23/07/2022
00018EB6 ??                     <1>     u.pri:   resb 1 ; Modification: 21/05/2016 (priority levels: 0, 1, 2)
00018EB7 ??                     <1>              resb 1 ; 23/07/2022
00018EB8 ????                   <1>     u.intr:  resw 1
00018EBA ????                   <1>     u.quit:  resw 1
                                <1>     ;u.emt:  resw 1 ; 10/10/2013
                                <1>     ;u.ilgins: resw 1 ; 10/01/2017
                                <1>     ;u.cdrv:  resw 1 ; cdev
00018EBC ??                     <1>     u.bsys:  resb 1
00018EBD ??                     <1>     u.uno:   resb 1
                                <1>     ; 23/07/2022
                                <1>     ;u.uid:  resw 1 ; uid ; 27/03/2021 - Retro UNIX 386 v2
                                <1>     ;u.ruid:  resw 1      ; 16 bit uid
                                <1>     ;u.gid:  resb 1 ; gid        ; 27/03/2021 - Retro UNIX 386 v2
                                <1>     ;u.rgid:  resb 1
                                <1>     ; 23/07/2022
                                <1>     ;u.procp: resd 1 ; /* pointer to proc structure */
00018EBE ??                     <1>     u.uid:   resb 1 ; uid
00018EBF ??                     <1>     u.ruid:   resb 1
00018EC0 ????????               <1>         u.upage:  resd 1 ; 16/04/2015 - Retro Unix 386 v1 feature only !
00018EC4 ????????               <1>     u.pgdir:  resd 1 ; 09/03/2015 (page dir addr of process)
00018EC8 ????????               <1>     u.ppgdir: resd 1 ; 06/05/2015 (page dir addr of the parent process)
00018ECC ????????               <1>     u.pbase:  resd 1 ; 20/05/2015 (physical base/transfer address)
00018ED0 ????                   <1>     u.pcount: resw 1 ; 20/05/2015 (byte -transfer- count for page)
                                <1>     ;u.pncount: resw 1
                                <1>             ; 16/06/2015 (byte -transfer- count for page, 'namei', 'mkdir')
                                <1>     ;u.pnbase:  resd 1
                                <1>             ; 16/06/2015 (physical base/transfer address, 'namei', 'mkdir')
                                <1>                     ; 09/06/2015
00018ED2 ??                     <1>     u.kcall:  resb 1 ; The caller is 'namei' (dskr) or 'mkdir' (dskw) sign
00018ED3 ??                     <1>     u.brwdev: resb 1 ; Block device number for direct I/O (bread & bwrite)
                                <1>                     ; 24/07/2015 - 24/06/2015
                                <1>     ;u.args:  resd 1 ; arguments list (line) offset from start of [u.upage]
                                <1>                     ; (arg list/line is from offset [u.args] to 4096 in [u.upage])
                                <1>                     ; ([u.args] points to argument count -argc- address offset)
                                <1>                     ; 24/06/2015
                                <1>     ;u.core:  resd 1 ; physical start address of user's memory space (for sys exec)
                                <1>     ;u.ecore: resd 1 ; physical end address of user's memory space (for sys exec)
                                <1>     ;;;
                                <1>     ; 19/08/2024    ; TRDOS 386 v2.0.9 ; STDIN/STDOUT/STDERR
00018ED4 ??                     <1>     u.stdin:  resb 1 ; 0 is tty-r or > 0 is file (redirection)
00018ED5 ??                     <1>     u.stdout: resb 1 ; 0 is tty-w or > 0 is file (redirection)
00018ED6 ??                     <1>     u.ungetc: resb 1 ; u.getc is valid if u.ungetc > 0
00018ED7 ??                     <1>     u.getc:  resb 1 ; last char read on stdin
                                <1>     ;;;
                                <1>     ; last error number
00018ED8 ????????               <1>     u.error:  resd 1 ; 28/07/2013 - 09/03/2015
                                <1>                     ; Retro UNIX 8086/386 v1 feature only!
                                <1>                     ; 21/09/2015 (debugging - page fault analyze)
00018EDC ????????               <1>     u.pfcount: resd 1 ; page fault count for (this) process (for sys geterr)
                                <1>             ; 19/12/2016 (TRDOS 386)
00018EE0 ????????               <1>     u.tcb:   resd 1 ; Timer callback address/flag which will be used by timer int
```

```
                                <1>             ; 13/01/2017 (TRDOS 386)
00018EE4 ??                     <1>     u.t_lock: resb 1 ; Timer interrupt (callback) lock (unlocked by 'sysrele')
00018EE5 ??                     <1>     u.t_mode: resb 1 ; running mode during timer interrupt (0= system, 0FFh= user)
                                <1>             ; 26/02/2017 (TRDOS 386)
00018EE6 ??                     <1>     u.irqc:  resb 1  ; Count of IRQ callback services (IRQs in use)
                                <1>             ; 28/02/2017 (TRDOS 386)
00018EE7 ??                     <1>     u.irqwait: resb 1 ; IRQ waiting for callback service flag (IRQ number, If > 0)
00018EE8 ??                     <1>     u.r_lock: resb 1 ; 'IRQ callback service is in progress' flag (IRQ lock)
00018EE9 ??                     <1>     u.r_mode: resb 1 ; running mode during hadware interrupt
                                <1>     ; 23/07/2022
00018EEA ??                     <1>     u.exit:  resb 1 ; exit code
                                <1>     ; 27/02/2017 (TRDOS 386)
00018EEB ??                     <1>     u.fpsave: resb 1 ; TRDOS 386, 'save/restore FPU registers' flag
                                <1> alignb 4
                                <1>     ; !! wrong sizing in TRDOS 386 v2.0.4 (in 'ubss.s', 28/02/2017) !!
                                <1>     ;u.fpregs: resb 94 ; 94 byte area for saving and restoring FPU registers
                                <1>     ; 23/07/2022
00018EEC <res 6Ch>              <1>     u.fpregs: resb 108 ; 108 byte area for saving and restoring FPU registers
                                <1>
                                <1> alignb 4
                                <1>
                                <1> U_SIZE      equ $ - user
                                <1>
                                <1> ; 18/10/2015 - Retro UNIX 386 v1 (local variables for 'namei' and 'sysexec')
00018F58 ????????               <1> pcore:      resd 1  ; physical start address of user's memory space (for sys exec)
00018F5C ????????               <1> ecore:      resd 1  ; physical address of user's stack/last page (for sys exec)
00018F60 ????????               <1> nbase:      resd 1 ; physical base address for 'namei' & 'sysexec'
                                <1> ; 23/07/202 - TRDOS 386 Kernel v2.0.5
                                <1> ;ncount: resw 1
00018F64 ????????               <1> ncount: resd 1     ; remain byte count in page for 'namei' & 'sysexec'
                                <1> ;argc:      resw 1
00018F68 ????????               <1> argc:       resd 1 ; argument count for 'sysexec'
00018F6C ????????               <1> argv:       resd 1 ; argument list (recent) address for 'sysexec'
                                <1>
                                <1> ; 03/06/2015 - Retro UNIX 386 v1 Beginning
                                <1> ; 07/04/2013 - 31/07/2013 - Retro UNIX 8086 v1
00018F70 ??                     <1> rw: resb 1 ;; Read/Write sign (iget)
                                <1> ; 23/07/2022
00018F71 ??????                 <1>     resb 3
                                <1>
                                <1> alignb 4
                                <1>
                                <1> ; 24/04/2016
00018F74 ????????               <1> ii: resd 1 ; first cluster of the program file
00018F78 ????????               <1> i.size:     resd 1 ; size of the program file

                                    alignb 4

                                    ; 23/05/2016 (TRDOS 386)
                                    ; 14/10/2015 (Retro UNIX 386 v1, 'unix386.s')
00018F7C ????????                   cr3reg:      resd 1  ; cr3 register content at the beginning of the timer
                                                         ; (or RTC) interrupt handler.

                                    ; 10/12/2016 (callback)
                                    ; 10/06/2016
                                    ; 19/05/2016
                                    ; 18/05/2016 - TRDOS 386 feature only !
00018F80 <res 100h>                 timer_set: resd 16*4   ; 256 bytes memory space for 16 timer events
                                        ; Timer Event Structure: (max. 16 timer events, 16*16 bytes)
                                        ;       Owner:         resb 1 ; 0 = free
                                        ;                              ;>0 = process number (u.uno)
                                        ;       Callback:      resb 1 ; 0 = response byte address (phy)
                                        ;                               1 = callback address (virtual)
                                        ;       Interrupt:      resb 1 ; 0 = Timer interrupt (or none)
                                        ;                              ; 1 = Real Time Clock interrupt
                                        ;       Response:       resb 1 ; 0 to 255, signal return value
                                        ;       Count Limit:   resd 1 ; count of ticks (total/set)
                                        ;       Current Count:        resd 1 ; count of ticks (current)
                                        ;       Response Addr:  resd 1 ; response byte (pointer) address
                                        ;                              ; (or callback -user service- address)


                                    ; 17/04/2021
                                    ; (memory page swap parameters are disabled as temporary)
                                    ;
                                    ; Memory (swap) Data (11/03/2015)
                                    ; 09/03/2015
                                    ;swpq_count: resw 1 ; count of pages on the swap queue
                                    ;swp_drv:    resd 1 ; logical drive description table address of the swap drive/disk
                                    ;swpd_size:  resd 1 ; size of swap drive/disk (volume) in sectors (512 bytes).
                                    ;swpd_free:  resd 1 ; free page blocks (4096 bytes) on swap disk/drive (logical)
                                    ;swpd_next:  resd 1 ; next free page block
                                    ;swpd_last:  resd 1 ; last swap page block

                                    alignb 4

                                    ; 10/07/2015
                                    ; 28/08/2014
00019080 ????????                   error_code: resd 1
                                    ; 29/08/2014
00019084 ????????                   FaultOffset:       resd 1
                                    ; 21/09/2015
00019088 ????????                   PF_Count:   resd 1 ; total page fault count
                                                       ; (for debugging - page fault analyze)
                                                       ; 'page_fault_handler' (memory.inc)
                                                       ; 'sysgeterr' (u9.s)

                                    ; 29/04/2016 (TRDOS 386 = TRDOS v2.0)
                                    ; 22/08/2015 (Retro UNIX 386 v1)
                                    buffer:
0001908C ???????????????                resb   8
                                    readi_buffer:
00019094 <res 200h>                     resb   512
00019294 ???????????????                resb   8
                                    writei_buffer:
0001929C <res 200h>                     resb   512
                                    ; 24/10/2016
0001949C ???????????????                resb   8
                                    rw_buffer:
000194A4 <res 800h>                     resb   2048  ; general purposed, r/w sector buffer

                                    %if 1
                                    ; 17/01/2021
                                    edid_info:  resb 128 ; VESA EDID (monitor capabilities) info
                                    ; 28/11/2020
00019D24 ??                         pmi32:             resb 1 ; (>0) use VESA VBE3 protected mode calls
00019D25 ??                         vbe_mode_x: resb 1  ; VESA VBE3 video bios mode set options
00019D26 ????                       video_mode: resw 1 ; VESA VBE3 video mode (with option flags)
                                    ; 30/11/2020
00019D28 ????????                   vbe3bios_addr:     resd 1 ; new (writable mem) address of VBE3 bios
                                    ; 02/12/2020
00019D2C ????????                   pmid_addr:  resd 1 ; PMInfoBlock ('PMID') linear address
                                    ; 14/01/2021
                                    ; 06/12/2020              ; VESA VBE 3 video state
```

```
                                ;vbe3stbufsize: resw 1    ; video regs/dac/bios state buffer size
                                ;                 ; block size in bytes
                                ; 16/01/2021
00019D30 ????                   vbe3stbsflags: resw 1     ; video regs/dac/bios state buffer size
                                ;                 ; pointer flags for buffer state options
                                %endif

                                %if 1
                                ; 10/12/2020
                                LFB_Info:
00019D32 <res 10h>                          resb 16 ; Linear Frame Buffer info block

                                ;24/11/2020 - TRDOS 386 v2.0.3
                                ; BOCHS/PLEX86 VESA VBE3 MODE INFO extension to TRDOS 386 v2 kernel
                                MODE_INFO_LIST:
00019D42 <res 44h>                          resb 68 ; mode + 66 byte VESA vbe3 mode info (4F01h)
                                %endif

                                ; 05/01/2021
00019D86 ??                     ufont:            resb 1 ; (VGA graphics) user font flags
                                                  ; bit 7 - permission flag for int 31h
                                                  ; bit 4 - 8x16 user font ready/loaded flag
                                                  ; bit 3 - 8x8 user font ready/loaded flag
                                                  ; bit 1 - select 8x16 user font (sysvideo)
                                                  ; bit 0 - select 8x8 user font (sysvideo)
00019D87 ??                                 resb 1 ; 19/01/2021
                                ; 17/01/2021
00019D88 ??                     srvsf:            resb 1 ; 'save restore video state' permission flag
                                ; 18/01/2021
00019D89 ??                     srvso:            resb 1 ; video state buffer save/restore option
00019D8A ????????               VideoStateID:     resd 1 ; used to verify state saved by same prog
                                ; 29/01/2021
00019D8E ????                   v_width:   resw 1 ; screen (display page) width
00019D90 ??                     v_ops:            resb 1 ; 'sysvideo' graphics data transfer option
00019D91 ??                     v_bpp:            resb 1 ; bits per pixels ('sysvideo')
00019D92 ????????               v_mem:            resd 1 ; video memory ('sysvideo')
00019D96 ????????               v_siz:            resd 1 ; video page size ('sysvideo')
00019D9A ????????               v_str:            resd 1 ; window start adress ('sysvideo')
00019D9E ????????               v_end:            resd 1 ; window end (end+1) adress ('sysvideo')
                                ; 31/01/2021
                                ; 01/01/2021
                                ;maskbuff:     ;resd 1    ; user's bitmask buffer addr ('sysvideo')
00019DA2 ????????               maskcolor:  resd 1 ; VGA/SVGA pixel mask color ('sysvideo')
                                ; 27/02/2021
00019DA6 ????????               pixcount:   resd 1 ; pixel count ('sysvideo' window ops)
                                ; 02/02/2021
00019DAA ????????????????       buffer8:   resd 2 ; 8 bytes small buffer for 'sysvideo'

                                bss_end:

                                ; 27/12/2013
                                _end:  ; end of kernel code
```